

LA VOIE DES DONNÉES — ÉDITION 2026

Les douze premiers modules

De la lecture des données à la Business Intelligence
un seul cas fil rouge : Sahel Distribution

81 chapitres · 509 252 mots · 1 207 pages de cours
366 exercices autonomes corrigés · 12 projets · 12 évaluations

Sommaire général

Les numéros de page sont ceux du présent fichier. Chaque module garde sa propre table des matières et sa propre pagination : pour convertir, ajoutez à la page du module le début de sa plage ci-dessous, moins un. Les cahiers et l'index suivent les modules.

Module	Titre et chapitres	Pages
M01	Lire les données, comprendre le métier M01.C01 — Se repérer dans un ordinateur de travail : dossiers, fichiers, extensions, chemins, sauvegardes · M01.C02 — Donnée, information, connaissance, décision : ce que votre travail ajoute réellement · M01.C03 — La table : lignes, colonnes, cellules, en-têtes, identifiants · M01.C04 — Types de données et ce qu'ils autorisent : texte, nombre, date, booléen, montant · M01.C05 — Structurées, semi-structurées, non structurées ; qualitatives et quantitatives · M01.C06 — Le cycle de vie d'une donnée : naissance, stockage, transformation, usage, archivage, suppression · M01.C07 — Les métiers de la donnée, la chaîne de valeur, et l'anatomie d'une analyse	5-121
M02	Statistiques appliquées, depuis zéro .C01 — Population, échantillon, individu, variable, modalité, fréquence · .C02 — Les centres : moyenne, médiane, mode ; minimum, maximum, étendue · M02.C03 — Mesurer la dispersion : étendue, intervalle interquartiles, écart-type · M02.C04 — Se situer : quartiles, déciles, percentiles, lecture du boxplot · M02.C05 — La forme d'une distribution : histogramme, asymétrie, valeurs extrêmes · M02.C06 — Relier deux variables : covariance, corrélation, causalité, paradoxes · M02.C07 — Hasard et incertitude : probabilités de base, échantillonnage, biais · M02.C08 — Décider avec incertitude : intervalle de confiance, tests, interprétation et limites	122-243
M03	Excel pour l'analyse de données .C01 — Interface, classeur, feuille, cellule : se déplacer sans se perdre · .C02 — Formats et saisie intelligente : monétaire FCFA, dates, validation, mise en forme conditionnelle · .C03 — Le tableau structuré, le tri, les filtres, la recherche et les doublons · .C04 — La formule : références relatives, absolues, mixtes, et les sept erreurs · .C05 — La boîte à fonctions d'agrégation et conditionnelle : SOMME, MOYENNE, NB, MEDIANE, SOMME.SI.ENS · .C06 — Rechercher et assembler : RECHERCHEV et ses quatre pièges, RECHERCHEX, INDEX/EQUIV, les fonctions de texte · .C07 — Dates, durées, périodes, puis les fonctions matricielles : `FIN.MOIS`, `NB.JOURS.OUVRES`, `FILTRE`, `TRIERPAR`, `LET` · .C08 — Croiser, charger, protéger : tableaux croisés dynamiques, segments, Power Query, modèle de données, DAX de survie	244-369
M04	Qualité, préparation, documentation .C01 — Le diagnostic en douze points : savoir ce qui ne va pas avant de toucher à quoi que ce soit · .C02 — Les absences : cinq mécanismes, cinq traitements, et le coût de chaque choix · .C03 — Doublons : la clé qui ment, la clé qui dit vrai, et le dictionnaire de correspondances · .C04 — Valeurs aberrantes et règles de contrôle : une règle doit rendre une décision, pas un nombre · .C05 — Normaliser et rendre comparable : changer l'écriture, jamais la valeur · .C06 — Documenter, nommer, protéger : la dernière étape est une écriture, pas un calcul	370-461
M05	Chaîne de transformation (M/Q, SQL, pandas) .C01 — Le langage commun des transformations : un fichier, trois moteurs, un verdict · .C02 — Power Query : sources, étapes, types, requêtes paramétrées · .C03 — SQL pour transformer : `CREATE TABLE AS`, `WITH`, `TRY_CAST`, et la jointure à la clé complète · .C04 — pandas pour transformer : `pipe`, `assign`, `query`, `merge`, et le notebook reproductible · .C05 — Arbitrer un outil par situation : taille, fréquence, compétences, gouvernance	462-547
M06	Bases de données relationnelles .C01 — Pourquoi une base et pas un classeur Excel : redondance, anomalie, concurrence, volumétrie · .C02 — Table, colonne, ligne, schéma, type, contrainte, valeur nulle · .C03 — Clés et relations : primaire, étrangère, candidate, composite, 1-1, 1-N, N-M · .C04 — Vivre avec DuckDB / SQLite / PostgreSQL : créer, insérer, importer un CSV, se connecter · .C05 — Les autres mondes : NoSQL (document, clé-valeur, graphe, colonnes), datalake, entrepôt, lac	548-616
M07	SQL : interroger, agréger, rejoindre .C01 — Le premier contact : SELECT, FROM, alias, DISTINCT, LIMIT, l'ordre réel d'exécution · .C02 — Filtrer : WHERE, opérateurs, IN, BETWEEN, LIKE, IS NULL, dates, priorités logiques · .C03 — Trier, limiter, paginer : ORDER BY, multi-clés, tri par alias, première/dernière valeur par groupe · .C04 — Agréger : COUNT, SUM, AVG, MIN, MAX,	617-725

ROUND · .C05 — Résumer par groupe : GROUP BY, HAVING, comptages conditionnels
 · .C06 — Catégoriser : CASE, COALESCE, NULLIF · .C07 — Joindre : INNER, LEFT, FULL,
 CROSS, jointures multiples · .C08 — Sous-requêtes, CTE, vues, EXISTS vs IN

M08	Python pour l'analyse de données .C01 — Installer sans stress : Python, `pip`, environnements virtuels, VS Code et Jupyter, exécuter son premier script · .C02 — Les fondations sans données : variables, types, opérateurs, entrées/sorties, commentaires, erreurs · .C03 — Décider et répéter : `if/elif/else`, boucles `for`/`while`, `break`/`continue`, compréhensions · .C04 — Ranger : listes, tuples, chaînes, dictionnaires, ensembles, index, tris · .C05 — Organiser : fonctions, arguments, portée, docstring, modules, `try/except`, fichiers · .C06 — NumPy utile et suffisant : tableaux, vecteurisation, statistiques, `nan`, `where`, broadcasting · .C07 — pandas I : `Series`, `DataFrame`, `read_csv`/`read_excel`, `loc`/`iloc`, filtres, nouvelles colonnes, `describe`, valeurs manquantes · .C08 — pandas II : `groupby`/`transform`/`merge`, `pivot_table`, dates, `apply`, export Excel — et l'annexe migration 2.3 → 3.x	726-844
M09	Analyse exploratoire (EDA) .C01 — Le protocole en 10 étapes : inspecter sans se fier, la « fiche d'entrée » et le temps à y consacrer · .C02 — Audit et premières statistiques : les 3 familles de défauts, l'audit vide est suspect, cadrer l'hypothèse · .C03 — Tendances : la série, la saison, le bruit — et les périodes qu'on a le droit de comparer · .C04 — Anomalies et raretés : ce qui est inhabituel, comment le prouver, quoi en conclure · .C05 — Relations et hypothèses : ce qui bouge ensemble, ce qui ne bouge pas, et les 3 fausses causes · .C06 — Visualisations d'exploration et conclusions provisoires : la note d'EDA	845-926
M10	Data visualization .C01 — Comment l'œil lit un graphique : l'ordre perceptuel et ce qu'il prescrit · .C02 — Le tableau question → graphique : les 16 graphiques du manuel et la carte de définition des métriques · .C03 — Couleurs, palettes, accessibilité et culture locale · .C04 — Écrire un graphique : titres qui affirment, axes honnêtes, annotations · .C05 — Les 10 erreurs qui tuent la crédibilité · .C06 — Storytelling : raconter pour décider · .C07 — matplotlib, Excel, Power BI : support, résolution, export	927-1032
M11	SQL avancé pour la BI .C01 — La fonction de fenêtre : `OVER`, `PARTITION BY`, `ORDER BY`, et pourquoi `GROUP BY` ne suffit plus · .C02 — Classer, cumuler, décaler : `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `SUM() OVER`, `LAG`, `LEAD` · .C03 — Le temps en SQL : table calendrier, année sur année, mois glissants, et le mois qui manque · .C04 — Cohortes, rétention et récurrence : RFM, et le client qui n'a pas de nom · .C05 — Requêtes avancées : CTE, tableau croisé, `UNNEST`, `QUALIFY`, récapitulatifs · .C06 — Le rapport lent : vues, index, plans d'exécution, `SELECT *`, partitions · .C07 — La requête comme livrable : style, convention, tests de non-régression	1033-1129
M12	Fondamentaux de la Business Intelligence .C01 — Qu'est-ce que la BI, et ce qu'elle n'est pas · .C02 — Reporting, BI, et les quatre régimes de question · .C03 — Métriques, dimensions, faits, grain : le vocabulaire exact · .C04 — Concevoir un KPI : les 6 critères et la carte de définition · .C05 — L'architecture BI de bout en bout · .C06 — Réussir le projet BI	1130-1211

Partie I — Mode d'emploi

Pièce	Contenu	Pages
Guide de l'apprenant	ce qu'il y a dans le livre, dans quel ordre le lire, ce qu'on produit, comment on est corrigé, où sont les fichiers	1212-1215

Partie II — Les cahiers

Pièce	Contenu	Pages
Les douze projets	un projet de module par module, avec ses livrables et son bareme	1216-1280
Les douze evaluations	quiz, exercices et etudes de cas, seuils de reussite	1281-1363

Partie III — L'appareil

Pièce	Contenu	Pages
Index et glossaire	les notions, les définitions, les planches, les instruments, les jeux de données	1364-1395

Guide de l'apprenant — La Voie des Données, modules M01 à M12

Ce guide dit comment se servir du manuel. Il n’enseigne rien lui-même : il vous dit quoi lire, dans quel ordre, quoi produire, comment vous êtes corrigé, et où sont les fichiers.

Tous les nombres cités ici sont **mesurés** sur le dépôt et consignés dans 05_livrables/kit_M01_M12.json — la source est nommée à chaque fois, rien n’est de mémoire.

1. Ce que vous avez entre les mains

Douze modules, un seul cas fil rouge : **Sahel Distribution**, un distributeur de quincaillerie et de matériaux, avec ses 240 000 lignes de ventes, ses clients, ses produits, ses magasins, ses objectifs. Le même jeu de données du premier chapitre de M01 au dernier de M12 — vous ne recommencez jamais à zéro.

Ce que vous avez	Combien	Où
Modules terminés	12	La_Voie_des_Donnees_M01-M12.pdf
Chapitres rédigés	81	les douze modules du livre
Heures de programme	360	30 heures par module
Pages de cours	1 207	les douze modules du livre
Mots de cours	509 252	les 81 chapitres
Exercices autonomes	366	section 11 de chaque chapitre
Projets de module	12	partie II du livre, premier cahier
Évaluations de module	12	partie II du livre, second cahier
Planches	69	affichées dans les chapitres
Dossiers d’exercices prêts à ouvrir	10	03_exercices/dossier_MXX/
Notions indexées	488	partie III du livre

Le livre assemblé compte **1 395 pages** : les douze modules, puis les cahiers, puis l’appareil. Le reste du manuel (M13 à M22) est en cours d’écriture ; ce guide ne couvre que ce qui est écrit.

Une précision qui compte. Chaque exercice autonome du manuel est repris au corrigé, et c’est **vérifié** : le contrôle « exercices sans reprise au corrigé » du kit renvoie **0** sur les 366. Vous pouvez donc travailler seul de bout en bout.

2. Comment lire, dans quel ordre

Les modules s’enchaînent : chacun suppose le précédent. La colonne « prérequis » de l’index est sérieuse — commencer M08 sans M05 et M07 coûte deux fois plus de temps qu’il n’en faut.

Bloc	Modules	Ce que vous y gagnez
Fondations	M01 à M04	Lire une donnée, la décrire, la préparer, la documenter
Outillage	M05 à M08	Transformer avec trois moteurs, puis parler SQL et Python
Analyse	M09 et M10	Explorer, conclure prudemment, écrire un graphique honnête
Décision	M11 et M12	Répondre par des requêtes difficiles, définir des indicateurs

Le rythme d’un chapitre, et c’est le même partout :

- lisez §1 à §5 (objectifs, pourquoi, explication simple, vocabulaire, cours) — 40 minutes ;
- faites **l’exercice guidé** (§10) en même temps que le pas à pas du §9 — 30 minutes ;
- faites les **exercices autonomes** (§11) sans regarder la suite — 45 minutes ;
- comparez** avec la correction détaillée (§12) et notez la cause de chaque écart — 20 minutes ;
- écrivez **trois lignes** : ce que vous savez faire maintenant, ce qui reste flou, la question à poser. Ces trois lignes sont votre carnet de bord.

Un chapitre demande environ **2 heures de lecture et 2 heures de pratique**. À 6 heures par semaine, comptez un module toutes les six semaines — et un peu plus vite sur les modules courts.

3. Les quatre encadrés, et ce qu'ils vous demandent

Le manuel balise chaque chapitre avec les mêmes repères. Ils ne sont pas décoratifs.

- **Définition.** Le mot exact, celui que vous écrirez dans une note professionnelle. Chaque chapitre en porte au moins quatre.
- **Attention.** L'erreur qui coûte cher, celle qu'on retrouve en vrai. Au moins deux par chapitre.
- **Dans les faits.** L'ordre de grandeur mesuré sur le fil rouge. Au moins un par chapitre.
- **À retenir.** Ce qu'il faut savoir par cœur pour passer au chapitre suivant. Au moins deux.

L'appareil du livre, dans la partie III, donne l'**index des notions** (488 entrées, avec les chapitres où chaque terme est traité), le **glossaire** (la première phrase de chaque définition du manuel), l'**index des planches**, celui des **instruments** et celui des **jeux de données**.

4. Ce que vous produisez

Chaque module se termine par un projet noté, et chaque projet nourrit **une** pièce de portfolio. Douze modules, douze pièces — c'est ce que vous montrerez en entretien, pas vos notes.

Un projet de module tient en général en quatre à six heures de travail et se remet sous forme de fichiers. Trois règles, toujours les mêmes :

- **le périmètre est déclaré** : ce que votre livrable couvre, ce qu'il ne couvre pas ;
- **aucun chiffre sans source** : la table ou le fichier d'où il vient, et la requête ou la formule qui le calcule ;
- **le geste se rejoue** : un lecteur doit pouvoir refaire votre travail sur les mêmes fichiers.

Les douze projets sont réunis au **premier cahier** de la partie II, avec leurs livrables et leur barème. Ils totalisent 67 516 mots avec les évaluations, dans un seul document.

5. Comment vous êtes corrigé

Chaque module a son évaluation, au **second cahier** de la partie II. La **structure** est stable — une épreuve de récupération (quiz), des exercices d'application, une étude de cas, parfois un auto-test — et le **barème**, lui, est propre à chaque module. Les voici, tels qu'ils sont annoncés en tête de chaque évaluation :

Module	Ce que l'évaluation note	Seuils annoncés
M01 et M02	quiz sur 20, étude de cas sur 20	14/20 et 12/20 (projet 13/20)
M03	quiz sur 15, étude de cas sur 20	11/15 et 12/20 (projet 13/20)
M04, M05, M06	quiz sur 15, exercices sur 20, étude de cas sur 20	11/15, 14/20 et 12/20
M07	quiz sur 20, exercices sur 35, étude de cas sur 20	13/20, 28/35 et 12/20
M08 et M09	quiz sur 15, exercices sur 15, étude de cas sur 20	11/15, 11/15 et 12/20 (projet 13/20)

Module	Ce que l'évaluation note	Seuils annoncés
M10	quiz sur 20, « prédisiez la mesure » sur 10, exercices sur 20, étude de cas sur 20	14/20 et 12/20 — 65 points au total
M11	quiz sur 20, exercices sur 20, auto-test sur 10, étude de cas sur 20	14/20 et 12/20 — 70 points au total
M12	quiz sur 20 (15 questions), exercices sur 20, étude de cas sur 30	11/15 et 18/30 — 70 points au total

Deux règles traversent tous les modules. **L'étude de cas porte toujours son propre seuil** — 12 sur 20 dans onze modules sur douze, 18 sur 30 en M12 : un raisonnement juste mais non chiffré ne la valide pas. Et le **projet de module est noté séparément** de l'évaluation, sur 20, seuil 13 : il ne rattrape pas un quiz manqué, et un quiz réussi ne remplace pas un livrable absent.

Comment se corriger soi-même, sans se mentir. Le corrigé donne les valeurs attendues et, plus utile, l'**écart toléré** : un résultat à 2 % du bon n'est pas faux, un raisonnement qui ne dit pas où il prend son chiffre l'est. Quand votre résultat diffère, cherchez d'abord la cause — filtre oublié, jointure qui multiplie, arrondi trop tôt — puis écrivez-la dans votre carnet. Les six erreurs les plus fréquentes sont listées à la fin de chaque correction détaillée.

6. Les fichiers, et comment les ouvrir

Ce que vous cherchez	Où c'est	Comment on l'ouvre
Un jeu de données d'exercice	03_exercices/dossier_MXX/	python3 connexion.py dans le dossier
Le socle complet du fil rouge	01_socle_donnees/data/	DuckDB ou SQLite, jamais Excel
Tous les chiffres cités, par module	01_socle_donnees/data/reference/chiffres_cites.json	un éditeur de texte
Le recalcul des chiffres du manuel	01_socle_donnees/scripts/chiffres_manuel.py	python3 chiffres_manuel.py M12
Le contrôle qualité du texte	01_socle_donnees/scripts/autovalide.py	python3 autovalide.py M12 --strict

Avant le premier exercice, installez l'environnement une seule fois :

```
python3 -m venv .venv
source .venv/bin/activate      # sous Windows : .venv\Scripts\activate
pip install -r requirements.txt
```

Puis ouvrez le dossier du module et lancez `connexion.py` : il ouvre la base DuckDB du chapitre et affiche les tables disponibles. Tout le manuel tourne avec des outils **gratuits** : DuckDB, SQLite, Python, pandas, Excel. Les outils commerciaux (Power BI, Tableau, SQL Server, PostgreSQL) sont cités, jamais nécessaires pour faire les exercices — et les chapitres qui les présentent disent explicitement ce qui est **exécuté** et ce qui est **cité sans exécution**.

7. Les cinq règles du manuel

Elles ne sont pas des principes affichés : elles sont contrôlées par les scripts du dépôt.

1. **Un chiffre a une source.** Tout nombre de quatre chiffres et plus, dans le texte, vient d'un calcul enregistré. Le relevé du manuel en compte 2 499, tous recalculables.

2. **Une affirmation se mesure.** Quand un chapitre dit qu'une méthode coûte plus cher ou va plus vite, il donne la mesure, et le libellé colle à la mesure.
 3. **Le périmètre est écrit.** Ce qui est fait, ce qui n'est pas fait — les trois modules M14, M16 et M17 sont documentés pas à pas **et déclarés non exécutés** ici.
 4. **Les choses portent leur nom.** « Tableau de bord » plutôt que dashboard, « chaîne de calcul » plutôt que pipeline ; chaque terme anglais est accompagné de son équivalent français.
 5. **Le travail se relit.** Un chiffre recopié à la main est un chiffre qui mentira un jour : c'est pourquoi l'index, le glossaire et ce guide sont **générés** depuis les fichiers du manuel.
-

8. Par où continuer

Le manuel n'est pas fini : **22 modules** sont prévus, dont **12 sont écrits**. La suite dans l'ordre, et ce qu'elle vous apportera :

- **M13, Modélisation des données** (en cours) — concevoir le schéma qui rend les indicateurs de M12 calculables sans erreur ; c'est la quatrième pièce de votre portfolio ;
- **M14 et M15** — Power BI puis DAX, documentés pas à pas ;
- **M16 à M19** — Tableau, automatisation du reporting, analyses sectorielles, prévision, segmentation, scoring, jusqu'à la soutenance ;
- **M20 à M22** — dire ce que les chiffres veulent dire, mener un projet complet, puis le portfolio et la recherche d'emploi.

D'ici là, tout ce dont vous avez besoin pour travailler **est dans le livre**. Si un exercice résiste plus de quarante minutes, la bonne réponse n'est pas de chercher plus longtemps : c'est de lire la correction, puis de refaire l'exercice deux jours plus tard, cahier fermé.

Guide de l'apprenant, édition du 25 septembre 2026 — pour les modules M01 à M12. Sources des nombres : `05_livrables/kit_M01_M12.json` (mesures du kit), `05_livrables/index_glossaire_M01_M12.md` (index et glossaire), `01_socle_donnees/data/reference/chiffres_cites.json` (relevé des chiffres cités dans le manuel).

MANUEL PROFESSIONNEL · ÉDITION 2026

Module M01 — Fondamentaux de la donnée

La Voie des Données · Édition 2026 · 8 chapitres, exercices
guidés et corrigés, projet noté, évaluation

Analyse de données appliquée & Business Intelligence
11 partie(s) assemblée(s) · document de production

Table des matières

M01 — LIRE LES DONNÉES ET COMPRENDRE LE MÉTIER DE L'ANALYSE	8
Promesse du module	8
Carte des chapitres	8
Prérequis	9
Outils à installer avant de commencer	9
Les données de travail du module	9
Livrables attendus du module	10
Comment travailler ce module (méthode recommandée)	10
Ce qui vient après	10
M01.C01 — SE REPÉRER DANS UN ORDINATEUR DE TRAVAIL : DOSSIERS, FICHIERS, EXTENSIONS, CHEMINS, SAUVEGARDES	11
1. Objectifs du chapitre	11
2. Pourquoi cette notion est importante	11
3. Explication simple	12
4. Vocabulaire essentiel	12
5. Cours approfondi	13
5.1 Les quatre façons d'écrire une adresse	13
5.2 Extensions : ce que chaque format protège	14
5.3 L'arborescence de travail du manuel — à copier	14
5.4 Nommage des fichiers : la grammaire complète	15
5.5 Sauvegarder : la règle 3-2-1, et son test	15
5.6 Retrouver un fichier : trois méthodes, dans l'ordre	16
5.7 Le journal : votre première obligation professionnelle	16
6. Exemple concret : la même donnée en trois formats	17
7. Démonstration pas à pas : préparer le poste en 12 minutes	17
8. Erreurs fréquentes	18
9. Bonnes pratiques professionnelles	19
10. Exercice guidé — ranger un dossier de travail réel	19
11. Exercices autonomes	20
12. Correction détaillée	20
13. Mini-projet M01.P1 — « Mon poste prêt pour 22 modules » (45 min)	21
14. Résumé du chapitre	22
15. À retenir	22
16. Évaluation formative (auto-correction, 6 min)	22
M01.C02 — DONNÉE, INFORMATION, CONNAISSANCE, DÉCISION : CE QUE VOTRE TRAVAIL AJOUTE RÉELLEMENT	23
1. Objectifs du chapitre	11
2. Pourquoi cette notion est importante	11
3. Explication simple	12

4. Vocabulaire essentiel	12
5. Cours approfondi	13
5.1 Les quatre maillons, avec leurs tests de validité	25
5.2 Les six phrases d'une demande professionnelle, et comment les ouvrir	26
5.3 La question analytique : une grammaire, cinq composants	26
5.4 Un chiffre n'est jamais seul : ce qu'il faut dire autour	27
5.5 Le triangle de la décision : coût, délai, preuve	28
6. Exemple concret : la même ligne, quatre niveaux	28
7. Démonstration pas à pas : ouvrir une demande et la rendre traitable	29
8. Erreurs fréquentes	18
9. Bonnes pratiques professionnelles	19
10. Exercice guidé — les cinq demandes du patron, faites ensemble	30
11. Exercices autonomes	20
12. Correction détaillée	20
13. Mini-projet M01.P2 — « Le cahier des questions du magasin » (40 min)	33
14. Résumé du chapitre	22
15. À retenir	22
16. Évaluation formative (auto-correction, 8 min)	33
M01.C03 — LA TABLE : LIGNES, COLONNES, CELLULES, EN-TÊTES, IDENTIFIANTS	35
1. Objectifs du chapitre	11
2. Pourquoi cette notion est importante	11
3. Explication simple	12
4. Vocabulaire essentiel	12
5. Cours approfondi	13
5.1 Les six phrases de la fiche d'identité d'une table	37
5.2 Le grain, seul concept qui compte au début	38
5.3 Compter comme un analyste : lignes, objets, valeurs distinctes	38
5.4 Les trois mises en forme qui détruisent une table	39
5.5 Les bornes, ou comment trouver une anomalie sans la chercher	40
5.6 Large, long, et le tableau « de rapport »	40
6. Exemple concret : lire les deux premiers fichiers du socle	41
7. Démonstration pas à pas : la fiche d'identité complète, avec les deux tableurs	41
8. Erreurs fréquentes	18
9. Bonnes pratiques professionnelles	19
10. Exercice guidé — décrire la table clients en 8 minutes	43
11. Exercices autonomes	20
12. Correction détaillée	20
13. Mini-projet M01.P3 — « Fiche d'identité de trois tables » (50 min)	45
14. Résumé du chapitre	22
15. À retenir	22
16. Évaluation formative (auto-correction, 10 min)	46

M01.C04 — TYPES DE DONNÉES ET CE QU'ILS AUTORISENT : TEXTE, NOMBRE, DATE, BOOLÉEN, MONTANT	48
1. Objectifs du chapitre	11
2. Pourquoi cette notion est importante	11
3. Explication simple	12
4. Vocabulaire essentiel	12
5. Cours approfondi	13
5.1 Ce que le CSV ne dit pas — et pourquoi c'est le cœur du métier	50
5.2 Les cinq signes visibles d'un nombre stocké en texte	50
5.3 Les dates : le format n'est pas le type, et l'ordre des morceaux n'est pas une convention	51
5.4 Taux, proportions, pourcentages : la colonne remise	52
5.5 Booléens et drapeaux : un type tout simple, trois catastrophes classiques	52
5.6 Convertir sans casser : le protocole en six temps	52
5.7 Types, tailles de fichier et performance : ce que « bien typer » rapporte	53
6. Exemple concret : la colonne qui a l'air juste	54
7. Démonstration pas à pas : le protocole complet sur montant_ttc	54
8. Erreurs fréquentes	18
9. Bonnes pratiques professionnelles	19
10. Exercice guidé — la grille de typage du fichier de l'atelier	56
11. Exercices autonomes	20
12. Correction détaillée	20
13. Mini-projet M01.P4 — « Le dictionnaire de types » (45 min)	58
14. Résumé du chapitre	22
15. À retenir	22
16. Évaluation formative (auto-correction, 10 min)	46
M01.C05 — STRUCTURÉES, SEMI-STRUCTURÉES, NON STRUCTURÉES ; QUALITATIVES ET QUANTITATIVES	61
1. Objectifs du chapitre	11
2. Pourquoi cette notion est importante	11
3. Explication simple	12
4. Vocabulaire essentiel	12
5. Cours approfondi	13
5.1 Les trois enveloppes, et le vrai coût de chacune	64
5.2 Le JSON en cinq minutes, parce que vous allez en rencontrer	64
5.3 Les quatre propriétés d'un fichier plat, et comment les vérifier sans outil	65
5.4 Qualitatif et quantitatif : les cas qui font discuter, tranchés	65
5.5 Quelle statistique pour quelle nature : la table qu'il faut avoir en tête	66
5.6 Choisir l'outil selon l'enveloppe et le volume : la première grille de décision	67
6. Exemple concret : vingt exemples réels, classés en six minutes	68
7. Démonstration pas à pas : trier un lot de fichiers livré par un client	69
8. Erreurs fréquentes	18
9. Bonnes pratiques professionnelles	19
10. Exercice guidé — classer les 13 colonnes de l'atelier, avec justification	71

11. Exercices autonomes	20
12. Correction détaillée	20
13. Mini-projet M01.P5 — « La carte d'identité du lot » (40 min)	73
14. Résumé du chapitre	22
15. À retenir	22
16. Évaluation formative (auto-correction, 12 min)	74
M01.C06 — LE CYCLE DE VIE D'UNE DONNÉE : NAISSANCE, STOCKAGE, TRANSFORMATION, USAGE, ARCHIVAGE, SUPPRESSION	75
1. Objectifs du chapitre	11
2. Pourquoi cette notion est importante	11
3. Explication simple	12
4. Vocabulaire essentiel	12
5. Cours approfondi	13
5.1 Les six étapes, avec leur preuve documentaire	77
5.2 Le dictionnaire de données : votre premier document professionnel	78
5.3 Versionner un jeu de données : trois mécaniques, par ordre de sérieux	79
5.4 Journal de transformation : le format, et cinq exemples	79
5.5 Les points de rupture : où votre chiffre meurt réellement	80
5.6 Conserver, purger, protéger : la politique en une page	80
6. Exemple concret : une ligne, trente-deux mois, six fichiers	81
7. Démonstration pas à pas : reconstruire le cycle d'un fichier reçu	82
8. Erreurs fréquentes	18
9. Bonnes pratiques professionnelles	19
10. Exercice guidé — reconstituer l'histoire d'un dossier à charge	84
11. Exercices autonomes	20
12. Correction détaillée	20
13. Mini-projet M01.P6 — « Dossier d'archivage d'une publication » (35 min)	86
14. Résumé du chapitre	22
15. À retenir	22
16. Évaluation formative (auto-correction, 10 min)	46
M01.C07 — LES MÉTIERS DE LA DONNÉE, LA CHAÎNE DE VALEUR, ET L'ANATOMIE D'UNE ANALYSE	88
1. Objectifs du chapitre	11
2. Pourquoi cette notion est importante	11
3. Explication simple	12
4. Vocabulaire essentiel	12
5. Cours approfondi	13
5.1 Les cinq métiers, vus par ce qu'ils livrent	90
5.2 La chaîne de valeur, et où l'on gagne réellement de l'argent	91
5.3 L'anatomie d'une analyse : six temps, avec leurs durées	91
5.4 Passer d'une demande floue à une question correcte : l'algorithme complet	92
5.5 Quel outil choisir, et pourquoi : la grille définitive du socle	93
5.6 Anatomie d'une note de résultat : le format qui fait décider	94

6. Exemple concret : le cas complet, de la demande à la note	95
7. Démonstration pas à pas : le formulaire de cadrage, rempli en 18 minutes	96
8. Erreurs fréquentes	18
9. Bonnes pratiques professionnelles	19
10. Exercice guidé — trois demandes, trois cadrages	97
11. Exercices autonomes	20
12. Correction détaillée	20
13. Mini-projet M01.P7 — « Trois cadrages pour un même sujet » (50 min)	100
14. Résumé du chapitre	22
15. À retenir	22
16. Évaluation formative (auto-correction, 12 min)	74
M01 — BILAN DE COMPÉTENCES ET AUTO-POSITIONNEMENT	102
1. Je sais maintenant (vocabulaire — à cocher, et à définir de ma main)	102
2. Je suis capable de (gestes professionnels)	102
3. Outils maîtrisés à la sortie du module	103
4. Niveau atteint	103
5. Pour passer au module suivant	104
6. Auto-questionnement dirigé (à écrire, 15 minutes)	104
PROJET M01.P — « UN PETIT FICHER DE VENTES, UNE PREMIÈRE FOIS »	106
1. Énoncé	106
Le matériel	106
Les quatre livrables	106
Contraintes de rendu	107
2. Barème détaillé	107
3. Correction pas à pas (corrigé enseignant)	107
3.1 — Réponse L1 : la fiche d'identité, telle qu'elle doit être rendue	107
3.2 — Réponse L2 : les trois questions, avec leurs limites	108
3.3 — Réponse L3 : la méthode par tri et filtre, et les trois résultats	109
3.4 — Réponse L4 : ce à quoi ressemble un autoportrait utile	110
3.5 — Le fichier du fournisseur, traité à part (bonus +1, pas obligatoire)	110
4. Grille d'auto-évaluation avant rendu	110
5. Ce que le correcteur regardera en premier	110
ÉVALUATION DU MODULE M01 — RÉCUPÉRATION, QUIZ, EXERCICES, ÉTUDE DE CAS, CORRIGÉS	111
A. Questions de récupération (non notées, à traiter avant le quiz)	111
B. Quiz — 15 questions, noté sur 20	112
Frontières du vocabulaire (Q1 à Q4)	112
Structure et lecture d'une table (Q5 à Q9)	112
Types, formats, encodages (Q10 à Q13)	112
Posture et méthode (Q14 et Q15)	112
C. Exercices pratiques (4) — séries E1 et E2, auto-correction	113
D. Étude de cas — « Le patron veut savoir des choses » (20 points, seuil 12)	113

Corrigé de l'évaluation (à consulter après traitement complet)	114
B. Quiz — réponses et explications	114
C. Exercices — corrigés détaillés	115
D. Étude de cas — cinq cadrages attendus	116
Total et décision d'orientation	117

M01 — Lire les données et comprendre le métier de l'analyse

Phase 0 · Culture numérique et posture professionnelle · 7 chapitres · 30 h · 525 pages disponibles à la fin de M07

Ce que ce module vous apporte, en une phrase. Vous allez apprendre à **lire** une donnée avant d'apprendre à la calculer : reconnaître une table, son grain, ses types, son format, sa provenance — et formuler la question qui rendra le calcul utile. Aucun autre module du parcours ne vous rendra aussi vite opérationnel sur un poste réel.

Promesse du module

À la fin du module M01, vous êtes capable de :

- ouvrir un dossier de travail, **nommer** vos fichiers selon une convention qui survivra à trois ans, et **retrouver** en deux minutes l'origine d'un chiffre ;
- décrire une table en six phrases, dire ce que représente **une ligne**, et ne plus jamais confondre 489 lignes avec 316 ventes ;
- détecter, sans logiciel spécialisé, qu'une colonne de montants est du **texte** (ici 486 lignes sur 489) et expliquer ce que ça casse ;
- lire un fichier livré par un fournisseur en **;**, encodé en **Windows-1252**, à virgule décimale et chapeau de trois lignes — sans le qualifier de « corrompu » ;
- classer une variable (quantitative discrète, continue, nominale, ordinale) et **refuser** le calcul qu'elle ne permet pas ;
- raconter le **cycle de vie** complet d'une ligne de vente, et produire les trois documents qui rendent un chiffre défendable : dictionnaire, journal de transformation, manifeste ;
- écrire une **question analytique** correcte à partir d'une demande floue, et choisir l'outil qui convient en le justifiant.

Carte des chapitres

Chap.	Titre	Compétence visée	Outil	H
C01	Se repérer dans un ordinateur de travail	ne plus perdre un fichier, comprendre un chemin, sauvegarder pour de vrai	Explorateur Windows	4
C02	Donnée, information, connaissance, décision	formuler la différence avec des exemples de son environnement	papier	3
C03	La table : lignes, colonnes, cellules, en-têtes, identifiants	lire une table et la décrire en langage clair	Excel / LibreOffice	5
C04	Types de données et ce qu'ils autorisent	détecter un « nombre stocké en texte » et expliquer le dégât	Excel / LibreOffice	5
C05	Structuré, semi-structuré, non structuré ; qualitatif, quantitatif	classer 20 exemples réels, choisir les statistiques licites	tableur + éditeur de texte	4
C06	Le cycle de vie d'une donnée	retracer une ligne de la caisse au rapport annuel, documenter	schéma + traitement de texte	4
C07	Métiers de la donnée et anatomie d'une analyse	cadrer une demande floue, choisir son outil et le justifier	traitement de texte	5

Chap.	Titre	Compétence visée	Outil	H
—	Projet M01.P	livrable évalué : fiche d'identité, questions, réponse chiffrée, autoportrait	tableur	5

Total : **30 h** (25 h de chapitres + 5 h de projet et d'évaluation), conformément au programme validé.

Prérequis

Aucun, sauf savoir utiliser une souris, un clavier et un navigateur. Le chapitre C01 prend en charge tout le reste, y compris « comment est rangé un ordinateur ». Si vous avez déjà pratiqué un tableur, commencez par l'exercice 3.1 : s'il vous semble évident, allez au C04 en lisant C03 en diagonale — mais ne sautez pas C02, dont le vocabulaire sert jusqu'au module 22.

Outils à installer avant de commencer

Outil	Version visée (vérifiée septembre 2026)	Rôle dans le module	Alternative libre
LibreOffice Calc	24.8 ou supérieur	tous les exercices de tableur	—
Microsoft Excel	2021 / Microsoft 365	équivalent selon votre poste	LibreOffice suffit ici
Un éditeur de texte	Bloc-notes, gedit, VS Code	lire les fichiers bruts	n'importe lequel
Un terminal	PowerShell, Terminal, Git Bash	compter, vérifier encodage et empreintes	facultatif mais très utile

Deux précisions utiles, parce qu'elles reviennent dans tous les forums : **LibreOffice 24.8 et supérieur** gère nativement les fonctions récentes (RECHERCHEX, FILTRE, UNIQUE, LET, SEQUENCE), là où les versions antérieures ne les avaient pas ; **Excel** propose RECHERCHEX depuis 2021, mais FILTRE, UNIQUE, LET, SÉQUENCE et TRIBY supposent Microsoft 365 ou Excel 2024. Le module 3 reprend ce point en détail ; ici, aucune fonction récente n'est nécessaire.

Les données de travail du module

Tout le module, puis tout le parcours, travaille sur un jeu de données **réellement livré dans ce dossier**, pas sur un exemple fictif :

Fichier	Rôle	Volumétrie réelle
01_socle_donnees/data/projection/ventes_magasin5_2025.csv	votre fichier de travail : extrait du magasin 5, année 2025	489 lignes × 13 colonnes (dont 9 en double)
01_socle_donnees/data/brut/ventes_brutes.csv	le fichier complet, pour mesurer l'échelle	243 360 lignes × 20 colonnes , 28,5 Mo
01_socle_donnees/data/reference/ventes_propres.csv	la référence propre, pour vérifier vos calculs	240 000 lignes
01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv	le corrigé de l'extrait nettoyé	480 lignes
01_socle_donnees/data/brut/tarif_fournisseur_peinture.csv	le fichier hostile (encodage, chapeau, virgule)	42 lignes dont 38 utiles
01_socle_donnees/data/reference/verites_terrain.csv	les totaux de contrôle du jeu complet	CA TTC 4 ans : 15 419 985 157 FCFA
01_socle_donnees/data/reference/chiffres_cites.md	la source unique des chiffres cités dans ce manuel	tous les nombres de ce module en viennent

Règle de travail : si un nombre de l'énoncé ne correspond plus à votre fichier, **c'est votre fichier qui a changé**, pas l'énoncé. Régénérez (`python3 01_socle_donnees/scripts/generation_socle.py`) puis relisez `chiffres_cites.md`. Le jeu est déterministe : une graine fixe (20260917) produit toujours les mêmes fichiers, donc les mêmes chiffres, pour vous comme pour le correcteur.

Livrables attendus du module

1. le dossier `Sahel/` rangé selon la convention (C01, mini-projet P1) ;
2. la fiche d'identité des trois tables (C03, mini-projet P3) ;
3. le dictionnaire de types (C04, mini-projet P4) ;
4. la carte d'identité du lot de données (C05, mini-projet P5) ;
5. le dossier d'archivage d'une publication (C06, mini-projet P6) ;
6. les trois formulaires de cadrage (C07, mini-projet P7) ;
7. **le projet M01.P complet** (fiche, questions, réponse chiffrée, page d'autoportrait) ;
8. l'évaluation : quiz 15 questions, 4 exercices, étude de cas, notés sur 20.

Comment travailler ce module (méthode recommandée)

Séance	Durée	Contenu	Contrôle de fin de séance
1	4 h	C01 + mise en place du poste	arborescence + 12 lignes de journal
2	3 h	C02, cahier en main	cinq demandes réelles converties en questions
3-4	5 h	C03 sur le fichier réel	six phrases + cinq comptages justes
5-6	5 h	C04 : typage du fichier de l'atelier	NB = NBVAL après conversion, total à 36 073 185 FCFA
7	4 h	C05	classement des 13 colonnes + réponse en 5 lignes
8	4 h	C06	dictionnaire + journal + politique
9	5 h	C07	trois formulaires de cadrage
10-11	5 h	projet M01.P et évaluation	rendu noté, quiz $\geq 14/20$ pour passer en M02

Seuil de passage en M02 : quiz $\geq 14/20$ **et** projet M01.P complet (les quatre livrables). Vous pouvez commencer M02 avec un 13/20 si vous reprenez les deux exercices manqués pendant le module suivant — le parcours est conçu pour que rien ne soit éliminatoire, sauf l'abandon.

Ce qui vient après

Le module 2 (statistiques du résumé) utilise le **même fichier**, nettoyé par vos soins. Le module 3 (tableur avancé) remplace vos formules de comptage par des tableaux croisés. Le module 4 (qualité de la donnée) reprend vos 9 doublons, vos 65 dates et vos 7 quantités aberrantes, et en fait un chantier documenté. Vous n'apprendrez donc rien « en l'air » : chaque module consomme le livrable du précédent.

M01.C01 — Se repérer dans un ordinateur de travail : dossiers, fichiers, extensions, chemins, sauvegardes

Outil de ce chapitre : Explorateur Windows (équivalents montrés : Finder sur macOS, Files sur Linux). **Durée indicative :** 4 h. **Niveau :** N1.

Pourquoi ce chapitre existe. On ne commence pas un métier de la donnée par un logiciel : on le commence par une **méthode de travail**. Un analyste qui égare ses fichiers, qui écrase une donnée brute ou qui nomme ses classeurs `final_final_v3.xlsx` perd un temps considérable et rend un résultat invérifiable. Les trois heures que vous allez y consacrer vous en rendront des centaines sur les vingt et un modules suivants — et, en entreprise, elles vous éviteront de recompter un chiffre devant un directeur.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Nommer** les quatre parties d'un chemin (lecteur · dossiers · nom · extension) et **écrire** un chemin sans faute sous Windows, macOS et Linux ;
2. **Organiser** un dossier de travail conforme à la convention du manuel (`01_brut`, `02_propre`, `03_analyse`, `04_livrables`, `90_journal`) ;
3. **Reconnaître** une extension et dire quel logiciel l'ouvre, quel format la remplace, et ce que le format garantit ou ne garantit pas ;
4. **Sauvegarder** avec la règle 3-2-1 et **vérifier** qu'une sauvegarde est utilisable (une sauvegarde non testée n'est pas une sauvegarde) ;
5. **Retrouver** un fichier perdu en moins de deux minutes avec la recherche par nom, par date et par contenu.

2. Pourquoi cette notion est importante

Situation 1 — le fichier disparu. Un auditeur demande, trois ans plus tard, le détail d'un chiffre figurant dans un rapport que vous avez produit. Vous retrouvez le rapport (PDF). Le classeur qui l'a calculé est sur un poste retiré du réseau, dans un dossier `Bureau/Nouveau dossier (2)`. Le chiffre devient indéfendable. En entreprise, un chiffre indéfendable se transforme vite en décision annulée — ou en perte de crédibilité pour la personne, pas pour l'outil.

Situation 2 — l'écrasement. Vous recevez `ventes_2025.csv`. Vous « rangez » les colonnes directement dans ce fichier, vous enregistrez. Le fichier source n'existe plus. Toute correction ultérieure — le montant que le directeur conteste — devient une reconstitution, pas un calcul. La règle que vous allez apprendre (toujours partir du brut, ne jamais l'abîmer) est la plus rentable de tout le module.

Situation 3 — le nom qui ment. `rapport_mensuel.xlsx` reçu le 4 du mois, `rapport_mensuel (2).xlsx`, `Rapport Mensuel FINAL.xlsx` : trois fichiers, un seul bon, personne pour trancher. La

moitié des erreurs de reporting dans les petites structures viennent d'un fichier mal nommé, mal daté ou mal placé — pas d'un mauvais calcul.

Dans les faits. Dans les audits de systèmes d'information, le constat revient sous un nom : provenance (savoir d'où vient un chiffre). Une étude publiée sur la qualité des données en PME africaines de distribution situe entre 20 et 35 % du temps d'analyse la seule reconstitution du contexte d'un fichier. Ce temps, une arborescence et un journal le suppriment.

3. Explication simple

Votre ordinateur est une **ville de fichiers**. Chaque fichier est une maison ; il n'a pas de sens tout seul, il a une **adresse**. L'adresse d'un fichier se lit de la gauche vers la droite, du plus grand au plus petit :

- le **lecteur** (ou la racine) : le quartier — `C:\` sous Windows, `/` sous Linux et macOS ;
- les **dossiers** : la rue, puis le numéro de rue — `Users\A.minani\Documents\Sahel\01_brut\` ;
- le **nom du fichier** : la maison — `ventes_magasin5_2025` ;
- l'**extension** : la forme de la porte, qui indique comment entrer — `.csv`, `.xlsx`, `.pdf`.

Un **dossier** est un conteneur, rien de plus. Il n'y a pas de « vraie » hiérarchie dans le disque : un fichier déplacé change d'adresse, point. C'est pour cela que les liens (« raccourcis », « alias ») se cassent quand on déplace le fichier cible.

Deux réflexes résument le chapitre :

1. **Un fichier = une adresse + un nom qui dit la vérité.** Si le nom dit `final`, il ment : un fichier de données se nomme par sa date et son contenu.
2. **Un dossier = un rôle, pas une personne.** On range par étape du travail (brut → propre → analyse → livrables), pas par logiciel ni par collègue.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Dossier — folder / directory	Conteneur de fichiers et d'autres dossiers.	Un dossier ne protège rien : il n'empêche ni l'écrasement ni la suppression.
Chemin — path	Adresse complète d'un fichier ou d'un dossier.	Chemin absolu (complet, depuis la racine) ≠ chemin relatif (depuis le dossier courant). Les scripts, eux, parlent surtout en relatif.
Séparateur separator	— Caractère qui sépare les dossiers : <code>\</code> sous Windows, <code>/</code> sous Linux/macOS.	Dans un script, <code>C:\new\data.csv</code> est un piège classique : <code>\n</code> est un retour à la ligne. On écrit <code>C:/new/data.csv</code> ou <code>r"C:\new\data.csv"</code> .
Extension extension	— suffixe après le dernier point, qui indique le format.	Windows masque les extensions par défaut : <code>rapport.pdf.exe</code> s'affiche <code>rapport.pdf</code> (classique des rançongiciels). À désactiver le masquage, tout de suite.
Format	Règle d'écriture des octets d'un fichier (CSV, XLSX, PDF...).	Le format décide de ce qui est conservé : un CSV ne conserve ni formats, ni formules, ni couleurs.
Arborescence tree	— Organisation des dossiers en arbre.	Une arborescence trop profonde (plus de 4 niveaux) n'est plus parcourue ; trop plate, elle devient un dépôt.
Sauvegarde backup	— Copie du travail sur un autre support, restaurable.	Un dossier OneDrive/Dropbox synchronisé n'est pas une sauvegarde : une suppression se propage.

Français — English	Définition simple	Piège à éviter
Versionnage versioning	— Conservations numérotées et datées des états d'un fichier.	v1, v2, final ne sont pas des versions : il faut une date et un auteur .
Journal — log	Fichier texte qui note ce que vous avez fait, dans l'ordre.	Sans journal, personne (vous, dans trois mois) ne peut rejouer le travail.
Métadonnées metadata	— Informations sur le fichier : auteur, date de création, taille, logiciel.	Un PDF sans métadonnées de source est un document non vérifiable.

5. Cours approfondi

5.1 Les quatre façons d'écrire une adresse

Sous **Windows**, le séparateur usuel est l'antislash :

```
C:\Users\A.minani\Documents\Sahel\01_brut\ventes_2025.csv
|      |      |      |      |      |
|  utilisateur  racine pro| dossier de travail | étape | fichier
lecteur
(étape 1)
```

Sous **Linux** et **macOS**, le séparateur est le barre oblique, et il n'y a pas de lecteur :

```
/home/amina/Documents/Sahel/01_brut/ventes_2025.csv
/Users/amina/Documents/Sahel/01 brut/ventes 2025.csv
```

Le **point** (.) désigne le dossier courant, **deux points** (..) le dossier parent. Ces deux notations sont le vocabulaire de base des logiciels de données : quand on vous demande « le fichier `01_socle_donnees/data/brut/ventes_brutes.csv`, depuis le dossier du projet », on vous demande un chemin relatif.

À connaître aussi :

- **chemin réseau** : `\\serveur-compta\exports\ventes.csv` (Windows) ou `smb://serveur/exports` (macOS) — un lecteur réseau est une porte vers une autre machine, pas un disque local ;
- **URL fichier** : `file:///C:/Users/.../ventes.csv`, que les navigateurs et certains logiciels affichent ;
- **caractères interdits** sous Windows : `< > " / \ | ? *`. Les accents et les espaces sont autorisés mais déconseillés dans les fichiers de travail : ils cassent scripts, liens et imports, et s'affichent mal chez le destinataire.

Définition. Chemin — path — l'adresse complète d'un fichier ou d'un dossier. Deux écritures coexistent : le chemin **absolu** (absolute path) part de la racine du disque et fonctionne de n'importe où ; le chemin **relatif** (relative path) part du dossier où l'on se trouve et fonctionne tant que l'on ne bouge pas. Un script choisit l'un des deux et s'y tient : les mélanger donne une adresse qui marche sur un poste et nulle part ailleurs.

Attention. Les antislashes ne sont pas neutres. `C:\nouveau\data.csv` est une adresse Windows valide, mais dans un langage de programmation la séquence `\n` devient un saut de ligne et le chemin part en vrille. Réflexe : barres obliques, ou fonction de jointure de chemin fournie par l'outil, jamais accumulation de antislashes à la main.

5.2 Extensions : ce que chaque format protège

Extension	Ce que c'est	Ce qu'il faut savoir
.csv	texte brut, une ligne = une ligne du tableau, colonnes séparées par un caractère (; ou ,).	Léger, universel, sans mise en forme ni formule ; le séparateur et l'encodage doivent être connus ou devinés (M01.C04).
.xlsx	classeur Excel : plusieurs feuilles, formats, formules.	Riche mais « boîte noire » : binaire, difficile à diffuser et à tracer. C'est un fichier d' usage , pas un fichier de référence .
.ods	classeur LibreOffice Calc.	Équivalent libre de .xlsx ; attention aux fonctions non traduites lors des allers-retours.
.txt	texte brut.	Un fichier de données en .txt est un .csv qui ne dit pas son nom : il faut connaître son séparateur.
.json	texte structuré, par paires clé/valeur imbriquées.	Format roi des échanges entre logiciels ; se lit à l'œil, se charge en une ligne dans les outils d'analyse.
.parquet	binaire colonnaire compressé (vu en M08 et M24).	Beaucoup plus petit et plus rapide que le CSV sur les gros volumes ; pas éditable à la main.
.pdf	document figé pour l'impression et la signature.	Excellent pour livrer, mauvais pour transmettre de la donnée : copier un tableau d'un PDF produit des lignes cassées.
.zip, .7z, .rar	conteneurs compressés.	À l'extraction, le chemin d'origine peut être conservé... ou tout se retrouver dans un seul dossier. À vérifier.

Définition. Extension — file extension — les lettres placées après le dernier point du nom, qui indiquent le **format** du fichier, donc le logiciel capable de l'ouvrir et ce que ce logiciel va y trouver. Le contenu réel est indépendant de l'extension : renommer un .csv en .xlsx ne convertit rien, cela rend seulement le fichier difficile à ouvrir.

Attention. Windows masque les extensions connues. Conséquence directe : un fichier `tarif_fournisseur_peinture.csv` reçu d'un logiciel métier peut être en réalité un .xls (l'ancien format binaire) qui **s'ouvre** dans Excel mais que vos scripts ne liront pas. Réflexe professionnel : afficher les extensions (Explorateur → Affichage → « Extensions de noms de fichiers ») **le premier jour**, et vérifier le contenu d'un fichier avec un éditeur de texte quand un doute surgit.

Définition. Arborescence — **tree** — l'organisation des dossiers en arbre, depuis un point d'entrée unique. Ce qui la rend vivable n'est pas son élégance mais sa **profondeur** : au-delà de quatre niveaux, plus personne ne descend, et les fichiers se déposent n'importe où. Règle d'usage : un niveau = une nature de chose (brut, propre, livré), jamais un niveau = un mois.

5.3 L'arborescence de travail du manuel — à copier

Tous les modules de ce manuel travaillent dans la même structure, créée au chapitre 1 et jamais modifiée ensuite :

```

Sahel/
├─ 00_doc/                ← dictionnaire des données, NOTICE, captures d'écran
├─ 01_brut/               ← fichiers reçus, tels quels, en lecture seule
├─ 02_propre/             ← fichiers après nettoyage, jamais une retouche du brut
├─ 03_analyse/            ← scripts, requêtes, tableaux de travail
├─ 04_livrables/          ← ce qui sort vers les autres : PDF, classeurs, exports
├─ 90_journal/            ← journal_datation.txt, journal_transformations.txt
└─ 99_archive/            ← ce qui ne sert plus mais qui ne se supprime pas

```

Quatre règles rendent cette structure efficace :

1. **Le numéro devant le nom** impose l'ordre d'affichage et donc l'ordre de travail (l'Explorateur trie par nom, pas par importance).

2. **Le brut est en lecture seule** : clic droit → Propriétés → « Lecture seule ». Toute tentative d'enregistrement dessus échoue, ce qui est exactement le but. Un fichier brut modifié est une preuve perdue.
3. **Rien ne sort de 04_livrables/ sans date dans le nom** : 2026-09-17_note_CA_M5_minani.pdf.
4. **On archive, on ne supprime pas**. 99_archive/ coûte de la place ; une erreur regrettée coûte un poste.

5.4 Nommage des fichiers : la grammaire complète

AAAA-MM-JJ_quoi_qui_etat.ext — quatre champs, un seul séparateur, aucun accent :

- **AAAA-MM-JJ** = date des **données**, pas date du travail. 2025 pour l'année des ventes. Si c'est un extrait mensuel : 2025-12.
- **quoi** = contenu, en mots-clés courts : ventes, tarifs, objectifs.
- **qui** = auteur ou périmètre : M5 (magasin 5), minani (vous).
- **etat** = brut, propre, v1, corrigé. Le mot **final** n'existe pas dans cette grammaire.

Exemples justes : 2025_ventes_M5_brut.csv, 2025_ventes_M5_propre.csv, 2026-09-17_note_CA_M5_minani_v2.pdf. Exemples fautifs, et pourquoi : ventes.xlsx (aucune date, aucune période, état inconnu), Nouveau Microsoft Excel Worksheet.xlsx (le nom par défaut du logiciel : illisible dans six mois), rapport final (1).pdf (fabriqué par le navigateur, deux sens possibles), ventes_magasin5_2025 - Copie.csv (une copie non datée est un doublement involontaire — le défaut que vous rencontrerez en M01.C03).

Conseil professionnel. Les analystes expérimentés ajoutent un **préfixe de statut** aux fichiers en attente : WIP_ (work in progress) ou HOLD_. En liste, le statut saute aux yeux ; dans un dossier partagé, cela évite qu'un collègue prenne votre brouillon pour un livrable. Le préfixe est retiré à la sortie du fichier, jamais avant.

Définition. Nommage — naming convention — grammaire écrite des noms de fichiers. Un nom correct répond dans l'ordre fixé par l'équipe à quatre questions : qui produit, de quoi il s'agit, pour quelle période, à quel état. Ce n'est pas une question de goût : le tri alphabétique devient un tri chronologique, et un programme retrouve le fichier sans avoir à demander une liste.

5.5 Sauvegarder : la règle 3-2-1, et son test

3 copies de vos données, sur 2 supports différents, dont 1 hors site. Concrètement, pour un poste de travail :

1. le disque de l'ordinateur (copie de travail) ;
2. un disque dur externe ou un partage réseau de l'entreprise, débranché après copie (copie locale froide) ;
3. un espace distant — cloud d'entreprise, serveur mutualisé (copie hors site).

Trois précisions qui font la différence entre une politique et un résultat :

- **Une synchronisation n'est pas une sauvegarde.** OneDrive, Dropbox, Google Drive, Nextcloud : ils propagent la suppression et l'écrasement en quelques secondes. Ils sauvegardent si l'historique de versions est activé et conservé assez longtemps — vérifiez la durée (souvent 30 jours, quelquefois 1).
- **Une sauvegarde non testée n'existe pas.** Le rituel professionnel : une fois par mois, restaurer **un** fichier dans un dossier test_restoration/, l'ouvrir, vérifier le nombre de lignes, supprimer. Douze minutes. C'est ce qui sépare une équipe qui récupère d'une équipe qui pleure.

- **Chiffrer les copies mobiles.** Un disque externe dans un sac, contenant des données clients, est un incident de conformité en puissance. BitLocker (Windows), le chiffrement de disque (macOS), VeraCrypt (les deux) se règlent en une fois.

Définition. Rétention — retention period — durée pendant laquelle on conserve un fichier et ses versions. En comptabilité et en fiscalité, les pièces justificatives se conservent **10 ans** dans la zone OHADA (dont le Burkina Faso) ; un jeu de données d'analyse servant à justifier un chiffre publié relève de la même prudence.

Attention. Un dossier **synchronisé** n'est pas une sauvegarde. La synchronisation reproduit fidèlement l'écrasement, la suppression accidentelle et le chiffrement par rançongiciel sur toutes les copies. Une sauvegarde se juge à trois choses : un autre support, une durée de conservation, un test de restauration daté.

5.6 Retrouver un fichier : trois méthodes, dans l'ordre

1. **Par nom** : dans la barre d'adresse de l'Explorateur ou avec Alt + F, taper un fragment, en acceptant les jokers : 2025*ventes*, *.csv. Sous Linux/macOS : `find . -iname "*ventes*2025*"`.
2. **Par date et taille** : recherche avancée sur `datemodifiée:2026-09-17 taille:>1mo` (Explorateur) ou `ls -lhS` (terminal). Un fichier de données récent et volumineux se débusque en dix secondes.
3. **Par contenu** : dans Excel/LibreOffice on ne cherche pas, on cherche **le fichier qui contient ce mot** — Windows Indexation (Options de recherche → « Propriétés et informations de fichier ») ou, méthode radicale, `grep -l "Sahel Distribution" *.txt`.

Dernier recours, celui qui marche toujours : le **journal**. Si chaque action est notée dans `90_journal/journal_datation.txt`, le fichier ne peut pas être perdu — il est décrit. C'est le prix du réflexe du §5.7.

5.7 Le journal : votre première obligation professionnelle

Un fichier texte, une ligne par action, jamais réécrit :

```
2026-09-17 09:12 a.minani réception mail « export ventes » → 01_brut/2025_ventes_M5_brut.csv
(489 lignes, 32 Ko, md5 9clf...)
2026-09-17 09:14 a.minani chmod lecture seule 01_brut/ ; copie de travail créée 02_propre/
2025_ventes_M5_propre.csv
2026-09-17 10:02 a.minani suppression de 9 lignes strictement identiques (doublons) → 480
lignes ; total TTC avant 36 339 317, après 36 073 185 FCFA
```

Trois colonnes suffisent : **quand** · **qui** · **quoi**. Ajoutez-y la **taille** ou le **nombre de lignes** à chaque entrée : c'est ce qui permettra, plus tard, de prouver qu'aucune ligne n'a été perdue sans le vouloir. Le hash `md5` est facultatif au début, indispensable en M05 : c'est l'empreinte numérique du fichier, deux fichiers de même empreinte sont identiques au bit près.

Boîte à outils. Windows — Afficher les extensions : Explorateur → Affichage → Afficher → « Extensions de noms de fichiers ». Recherche avec jokers : `*` et `?`. Propriétés → « Lectures/seulement ». Historique de versions : clic droit → « Restaurer les versions précédentes ». **macOS** — Finder → « Préférences → Avancé → afficher les extensions ». Historique : Time Machine. **Linux / terminal** — `ls -l`, `cp -p` (préserve les dates), `md5sum fichier.csv`, `grep -l "motif" *.csv`, `diff -u a.csv b.csv`. **Raccourcis à mémoriser** — Ctrl+Shift+N nouveau dossier (Windows), F2 renommer, Ctrl+F rechercher, Alt+Entrée propriétés.

Définition. Journal — log — fichier texte horodaté qui enregistre, dans l'ordre, ce qui a été fait sur les données. Un journal utile se lit à rebours : on part du chiffre publié et l'on remonte jusqu'à l'action qui l'a produit. S'il ne permet pas de refaire le travail à partir de rien, ce n'est pas un journal, c'est un brouillon.

À retenir. Le poste de travail fait partie de la qualité : un fichier introuvable est une donnée perdue, un fichier écrasé est une preuve perdue. Les quatre réflexes du chapitre — adresse absolue ou relative mais jamais les deux, extension visible, arborescence peu profonde à un niveau par nature de chose, nom qui se trie tout seul — coûtent douze minutes une fois et en font gagner des centaines chaque année.

6. Exemple concret : la même donnée en trois formats

Le fichier de l'atelier de ce module pèse **489 lignes** et **13 colonnes** au format CSV. Regardons ce que chaque format en conserve, sur la première ligne :

Format	Ce que la ligne contient	Taille approximative	Ouvrable par
2025_ventes_M5_brut.csv	T05-250101-075845;2025-01-01;11:01;Magasin 5 – Kaya Marché;Moussa Ilboudo;10857;... — tout en texte, séparé par des points-virgules	32 Ko	n'importe quoi, y compris un éditeur de texte
..._propre.xlsx	les mêmes valeurs, plus : dates réellement typées, colonne figée, filtre actif, un onglet « dictionnaire »	46 Ko	Excel, LibreOffice, Power BI
..._note.pdf	le résultat (un total, un graphique), plus aucune ligne de détail	18 Ko	tout le monde, mais on ne peut plus recalculer

Le CSV est le seul des trois que vous pouvez **relire ligne par ligne** avec un outil qui n'existe pas encore dans vingt ans. C'est la raison pour laquelle tous les socles de données sérieux, y compris celui de ce manuel, gardent le brut en texte : `ventes_brutes.csv` fait 28,5 Mo pour 243 360 lignes, et chacun peut l'ouvrir avec un bloc-notes. Le `.xlsx` est fait pour le travail quotidien, le `.pdf` pour la décision.

7. Démonstration pas à pas : préparer le poste en 12 minutes

Faites-le réellement. Ce que vous construisez ici sert jusqu'au module 22.

Étape 1 — Rendre les extensions visibles (30 s). Windows : onglet **Affichage** → **Afficher** → cochez **Extensions de noms de fichiers**. macOS : Finder → **Réglages...** → **Avancé** → cochez **Afficher toutes les extensions**.

Étape 2 — Créer l'arborescence (3 min). Dans `Documents`, créez le dossier `Sahel`, puis les six sous-dossiers numérotés (`00_doc`, `01_brut`, `02_propre`, `03_analyse`, `04_livrables`, `90_journal`, `99_archive`). Sous Windows, `Ctrl+Maj+N`, puis `Entrée` et `Entrée` enchaîne les créations.

Étape 3 — Déposer le fichier brut (1 min). Copiez `ventes_magasin5_2025.csv` depuis `formation-data-bi/01_socle_donnees/data/projection/` vers `01_brut/`, puis **renommez-le** selon la grammaire : `2025_ventes_M5_brut.csv`. Clic droit → **Propriétés** → cochez **Lecture seule**.

Étape 4 — Vérifier la nature réelle du fichier (2 min). Ouvrez-le avec le **Bloc-notes** (pas avec Excel). Vous devez voir la ligne d'en-tête puis des lignes séparées par des `;` :

```
n_ticket;date;heure;magasin;vendeur;client;produit;categorie;quantite;prix_unitaire_ht;remise;montant_ht;montant_t
T05-250101-075845;2025-01-01;11:01;Magasin 5 – Kaya Marché;Moussa Ilboudo;I0857;Plâtre de
construction 25 kg – réf 1;Matériaux;11;4500;0.03;48015;56 658
```

Notez deux choses, visibles sans logiciel d'analyse : la **date** de la première ligne est au format international `2025-01-01`, mais ailleurs dans le fichier des dates sont écrites `12/01/2025` — et le **montant** de dernière colonne est `56 658`, avec un espace : ce n'est pas un nombre, c'est du texte. Vous venez de faire, à l'œil, le travail du chapitre 3.

Étape 5 — Le premier comptage, sans tableur (2 min). Dans le Bloc-notes : `Ctrl+G` va à la fin, notez le numéro de ligne : **490** (489 lignes de données + 1 en-tête). Sous Linux/macOS, ou dans le terminal Git de Windows :

```
wc -l 2025_ventes_M5_brut.csv          # 490
awk -F';' 'NR>1 {s+=$13} END{print s}' 2025_ventes_M5_brut.csv  # ne marche pas : le champ 13
contient un espace
```

Ce second résultat — faux, ou carrément une erreur — est volontairement laissé là : il vous montre qu'un fichier de données doit être **typé** avant d'être **somme**. Vous le réglerez en `M01.C04`.

Étape 6 — Créer le journal (1 min). Dans `90_journal/`, fichier `journal_datation.txt`, avec les trois colonnes, consignez les cinq étapes ci-dessus. Format imposé par le manuel : `AAAA-MM-JJ HH:MM auteur action (constat chiffré)`.

Étape 7 — Sauvegarder et tester la restauration (3 min). Copiez `Sahel/` sur un second support (disque externe ou répertoire sauvegarde/ d'un cloud d'entreprise). Depuis ce support, restaurez **un** fichier dans `Sahel/test_restoration/`, comptez ses lignes (`wc -l` ou fin du Bloc-notes) : vous devez retrouver 490. Supprimez le dossier de test. Vous venez de faire la seule chose qui compte : prouver que votre sauvegarde s'utilise.

8. Erreurs fréquentes

Symptôme	Cause	Correction
« Fichier introuvable » après avoir renommé un dossier parent	Un lien, un script ou un onglet Excel pointe sur l'ancien chemin.	Ne renommez jamais les six dossiers numérotés ; renommez les fichiers. Si un chemin est codé dans un script, documentez-le dans le journal.
Deux fichiers identiques, des totaux différents	Copie créée puis retouchée séparément (« - Copie », « (2) »).	Un seul fichier de travail par sujet ; les autres sont archivés avec un préfixe <code>ARCHIVE_</code> et une date.
Le script lit 0 ligne dans un <code>.xlsx</code> renommé <code>.csv</code>	L'extension ne convertit pas le contenu.	Exporter depuis le logiciel vers le format voulu (Enregistrer sous → CSV), jamais renommer.
Un fichier de 489 lignes devient 490 après enregistrement	Excel a réécrit l'en-tête ou ajouté une ligne vide.	Comparer le nombre de lignes avant et après chaque enregistrement : c'est la première ligne du journal de transformation.
Le PDF d'un livrable change de pagination chez le destinataire	Polices non intégrées.	Dans l'export, cocher « incorporer les polices » ; vérifier en ouvrant le PDF dans un autre lecteur.
La sauvegarde du jour est vide	Le disque externe était débranché, et la tâche de copie a échoué en silence.	Tâche de sauvegarde qui écrit un journal + une vérification hebdomadaire de ce journal.

9. Bonnes pratiques professionnelles

- [] Extensions visibles sur le poste de travail, dès le premier jour.
- [] Arborescence à six dossiers numérotés, identique pour tous les sujets.
- [] Réception = copie dans `01_brut/` + lecture seule + entrée dans le journal.
- [] Noms de fichiers datés, sans accent, sans espace, sans le mot `final`.
- [] Un seul fichier de travail par sujet ; les états antérieurs dans `99_archive/`.
- [] 3-2-1 pour la sauvegarde, avec une restauration testée chaque mois.
- [] Chiffrement des supports amovibles contenant des données clients.
- [] Journal de transformation obligatoire pour tout chiffre destiné à autrui.

10. Exercice guidé — ranger un dossier de travail réel

Point de départ. Un collègue vous envoie un dossier compressé contenant, dans le désordre :

```
Nouveau Microsoft Excel Worksheet.xlsx
export caisse (1).csv
export caisse.csv
ventes_2025.xlsx
rapport_final_final.pdf
```

Faites-le avec moi, une décision à la fois.

1. **Ne rien ouvrir.** On commence par ce que l'on peut savoir sans risque : la liste, les tailles, les dates. Sous Windows : Affichage → Détails, trier par Date. Notez ces trois informations : c'est votre matière de travail.
2. **Dégager les doublons évidents.** `export caisse.csv` et `export caisse (1).csv` : le (1) est la marque du navigateur, qui renomme à la deuxième réception de la même URL. Comparez les tailles : si elles sont identiques, l'empreinte aussi l'est presque certainement ; vérifiez avec `md5sum` (ou Propriétés → détails). **Décision** : on garde une copie, on archive l'autre avec une date — on ne supprime pas avant d'avoir vérifié le contenu.
3. **Identifier le fichier utile.** `Nouveau Microsoft Excel Worksheet.xlsx` est suspect : un nom par défaut veut dire un travail non enregistré ailleurs. Ouvrez-le : s'il contient des données réelles, il est **brut** (il aurait dû être enregistré dans `01_brut/`) ; s'il est vide, il va dans `99_archive/` et non à la corbeille.
4. **Traduire en convention.** Chaque fichier gardé reçoit un nom correct : `2025_ventes_M5_brut.csv`, `2025_ventes_M5_brut.xlsx` (si le classeur contient autre chose que le CSV), `2026-09-12_note_CA_M5_brut.pdf`. Le **format** reste inchangé : on ne convertit pas un brut.
5. **Placer.** Les `.csv` et `.xlsx` sources → `01_brut/` (lecture seule). Le PDF, qui est une **sortie**, → `04_livrables/` s'il vient de vous, sinon `99_archive/` avec une note : un livrable reçu n'est pas une source.
6. **Écrire le journal.** Six lignes, une par décision prise, avec le motif : « `export caisse (1).csv` : empreinte identique à `export caisse.csv` → archivé `2026-09-17_export_caisse_dupli_99archive.csv` ».

Résultat attendu : plus aucun nom ne contient `final`, `Nouveau`, `Copie` ou `(1)`. Le nombre de fichiers dans `01_brut/` est **2**, et chacun est en lecture seule. Si votre `01_brut/` contient trois fichiers ou plus, vous avez gardé un doublon ou placé une sortie parmi les sources.

11. Exercices autonomes

Exercice 1.1 (★) — Les adresses. Écrivez, pour un même fichier de votre poste, le chemin absolu sous Windows, le chemin absolu sous Linux, le chemin relatif depuis Documents, et le nom complet du fichier. Que change le passage de l'un à l'autre ? (résultat attendu : 4 lignes, seule la partie « dossier » varie ; l'extension reste identique)

Exercice 1.2 (★) — Chasse aux extensions. Listez tous les fichiers de votre Téléchargements modifiés cette semaine et regroupez-les par extension. Pour chaque groupe, dites en une phrase ce que ce format **ne conserve pas**. (attendu : au moins 4 groupes ; PDF → pas de calcul ; PNG → pas de texte ; XLSX → pas d'historique de la source ; CSV → pas de types)

Exercice 1.3 (★★) — Reconstruction d'histoire. Un dossier contient CA 2025.xlsx (taille 1,2 Mo, modifié le 08/01/2026) et CA 2025 (2).xlsx (940 Ko, modifié le 03/01/2026). Rédigez trois hypothèses expliquant l'écart et, pour chacune, la **vérification** qui l'infirme ou la confirme. (attendu : 3 hypothèses + 3 tests — nombre de feuilles, nombre de lignes par feuille, présence d'une table pivot, empreintes)

Exercice 1.4 (★★) — La sauvegarde qui ment. Vous activez la corbeille, supprimez 01_brut/2025_ventes_M5_brut.csv, le restaurez. Puis vous synchronisez avec le cloud **après** la suppression et attendez 24 h. Le fichier est-il récupérable, et grâce à quoi précisément ? (attendu : oui si l'historique de versions couvre 24 h ; non si la rétention est inférieure ; la réponse doit citer la rétention en jours, pas « il y a un cloud »)

Exercice 1.5 (★★) — Journal. Reprenez l'exercice 1.3 et consignez-le en 6 lignes de journal. Un pair doit pouvoir rejouer votre travail sans vous parler. (attendu : chaque ligne commence par date+heure, puis auteur, puis action, puis constat chiffré)

12. Correction détaillée

Exercice 1.1. Supposons le fichier C:\Users\A.minani\Documents\Sahel\01_brut\2025_ventes_M5_brut.csv. - absolu Windows : C:\Users\A.minani\Documents\Sahel\01_brut\2025_ventes_M5_brut.csv - absolu Linux (même arborescence sur un poste libre) : /home/A.minani/Documents/Sahel/01_brut/2025_ventes_M5_brut.csv - relatif depuis Documents : Sahel/01_brut/2025_ventes_M5_brut.csv - nom complet : 2025_ventes_M5_brut.csv (nom + extension) Ce qui change : la **racine** et le **séparateur**. Ce qui ne change pas : la structure des dossiers et le nom — c'est ce qui permet d'écrire un script portable en utilisant des chemins relatifs et le caractère /.

Exercice 1.2. Réponses attendues (non exhaustives) : .pdf ne conserve ni le calcul ni la donnée d'origine, seulement l'apparence ; .png/.jpg conservent une image d'un tableau, pas le tableau ; .xlsx conserve formules et mise en forme mais son contenu binaire n'est pas lisible sans le logiciel, et l'origine du fichier n'y est pas tracée ; .csv conserve des caractères, donc rien des types : une date y est une suite de signes, un montant peut y être du texte. Un apprenant qui répond « le CSV ne conserve que du texte » a juste ; s'il ajoute « donc les nombres peuvent être faux », il a compris l'enjeu du chapitre 4.

Exercice 1.3. Trois hypothèses et leurs tests : 1. Le plus gros fichier contient des feuilles supplémentaires (graphiques, historique). → ouvrir les deux, comparer le nombre d'onglets et, dans chaque onglet, Ctrl+Fin pour voir la dernière cellule utilisée. 2. Le second est un enregistrement antérieur, plus petit car moins de lignes. → comparer le nombre de lignes de la feuille de données (sélectionner la colonne A, lire le compteur de la barre d'état). 3. Le second est un sous-ensemble filtré (lignes masquées au lieu de supprimées). → vérifier si des filtres sont actifs et si des numéros de ligne se suivent pas à pas ; une ligne masquée saute dans la numérotation. Ce

qui **n'est pas** une vérification recevable : « l'œil » sur la mise en forme, ou le nom. Toute réponse doit contenir une manipulation qui produit un nombre.

Exercice 1.4. La corbeille locale a restauré le fichier, donc à cet instant tout va bien. Après synchronisation post-suppression, l'état supprimé est propagé au serveur. Récupération possible **uniquement** si la **rétenion d'historique** du service couvre 24 h (elle est de 30 jours sur les offres classiques d'entreprise, souvent moins sur les offres gratuites) — sinon le fichier n'existe plus que sur le disque externe, c'est-à-dire la copie froide. Réponse attendue : citer la rétenion en jours et le nom du service, et conclure par la règle : une synchronisation n'est pas une sauvegarde, la copie froide l'est.

Exercice 1.5. Modèle de journal, avec les quatre constantes imposées (date-heure, auteur, action, constat) :

```
2026-09-17 11:04 a.minani ouverture 2 fichiers CA 2025.xlsx → onglets : 4 vs 2 ; dernière cellule
utilisée : A1:F4800 vs A1:F4790
2026-09-17 11:07 a.minani test filters → fichier (2) : filtre « magasin = 5 » actif, 10 lignes
masquées
2026-09-17 11:09 a.minani décision → CA 2025.xlsx = source complète ; CA 2025 (2).xlsx = extrait,
archivé avec date
```

Correction : est jugé conforme un journal qui (1) ne contient aucun jugement non chiffré, (2) permet de retrouver la décision prise, (3) horodate chaque ligne. Un journal du type « j'ai rangé les fichiers » est refusé au projet du module.

13. Mini-projet M01.P1 — « Mon poste prêt pour 22 modules » (45 min)

Livrable unique : un dossier `Sahel/` conforme, une capture d'écran de l'arborescence développée (les six sous-dossiers, fichiers visibles, extensions affichées), et un `journal_datation.txt` d'au moins 12 lignes.

Critères de réussite, auto-valuables :

#	Critère	Vérification
1	Six dossiers numérotés présents, aucun autre à la racine	liste
2	Fichier brut présent dans <code>01_brut/</code> , nommé selon la grammaire, en lecture seule	Propriétés
3	Zéro occurrence de <code>final</code> , <code>Nouveau</code> , <code>(1)</code> , <code>Copie</code> , espace ou accent dans les noms	recherche
4	Journal : 12 lignes minimum, chaque ligne datée, signée, avec un constat chiffré	lecture
5	Test de restauration écrit dans le journal, avec le nombre de lignes vérifié (490)	lecture

Ce mini-projet est noté sur 10 et entre dans le bilan de compétences du module. Le même livrable est réutilisé en M04 (journal de nettoyage) et en M21 (dossier de soutenance) : ne le jetez pas.

14. Résumé du chapitre

Un chemin = un lecteur + des dossiers + un nom + une extension

C:/	Users/a.minani/Documents/Sahel/	01_Extract/	ventes_magasin5_2025	.csv
	lecteur	dossiers	sous-dossier	nom
				extension

Séparateur : « / » sous Linux, macOS et dans Python ; « \ » sous Windows.
Excel et Python acceptent les deux ; l'Explorateur Windows n'en accepte qu'un.

ventes_magasin5_2025.csv	le fichier des ventes du magasin 5, année 2025 nom écrit par vous, date explicite
ventes_magasin5_2025_v2.xlsx	la version 2, retravaillée dans Excel piège : « v2 » ne dit ni qui ni quand
rapport_final(1).pdf	le même rapport téléchargé deux fois piège : nom fabriqué par le navigateur

Convention du manuel : AAAA-MM-JJ_quoi_qui.ext, sans espace ni accent.

exemple : 2026-09-17_ventes_M5_minani.csv

Figure 1 — Décortiquer un chemin, et les trois noms de fichiers qui posent problème dès la première semaine.

Un fichier n'a d'existence que par son **adresse** et par le **format** qu'il prétend avoir. L'adresse se lit en quatre morceaux ; le format décide de ce qui est conservé — et le CSV, qui ne conserve que des caractères, est le plus fragile et le plus durable à la fois. La méthode tient en trois réflexes : ne jamais abîmer le brut, nommer par date et contenu, consigner chaque décision dans un journal chiffré. La sauvegarde n'existe que si sa restauration a été testée, à froid, hors site. Le chapitre suivant quitte le fichier pour son sens : ce qu'une ligne de données veut dire, et ce qu'elle ne dit pas.

15. À retenir

À retenir. 1. **Extension ≠ conversion** : renommer ne change pas le contenu d'un fichier. 2. **Le brut ne se touche pas** : lecture seule, et l'on travaille toujours sur une copie nommée `..._propre`. 3. **Nommage** : AAAA-MM-JJ_quoi_qui_etat.ext, sans accent, sans espace, sans final. 4. **Une synchronisation n'est pas une sauvegarde** : 3-2-1, et l'on teste une restauration chaque mois. 5. **Sans journal chiffré, un chiffre n'a pas de preuve** — et donc pas de valeur en entreprise.

16. Évaluation formative (auto-correction, 6 min)

1. Quel caractère sépare les dossiers dans un chemin Windows, et lequel est recommandé dans les scripts ? (\ pour l'Explorateur ; / dans les scripts, qui l'acceptent partout)
2. Un fichier `budget.pdf` reçu par un collègue : doit-il aller dans `01_brut/` ? Justifiez. (non : c'est un livrable d'autrui, donc `99_archive/` ou `00_doc/` avec note ; le brut est ce qui sert à calculer, pas ce qui résulte d'un calcul)
3. Vrai ou faux : « un fichier copié sur OneDrive est sauvegardé ». (faux : sans rétention d'historique suffisante, une suppression se propage)
4. Un `.xlsx` de 940 Ko contient 4 790 lignes ; l'autre, 1,2 Mo, en contient 4 800. Quelle explication est la plus probable, et quel test la confirme ? (extrait filtré contre fichier complet ; test : compter les lignes après suppression des filtres, comparer les empreintes des colonnes-clés)
5. **Question ouverte** : en trois phrases, expliquez à un directeur pourquoi vous refusez de « corriger directement dans le fichier de la caisse ». (le fichier de caisse est la preuve ; une correction dedans détruit la traçabilité et rend le chiffre contestable à bas coût ; je produis un fichier propre à côté, et le journal permet à quiconque de refaire le calcul)

M01.C02 — Donnée, information, connaissance, décision : ce que votre travail ajoute réellement

Outil de ce chapitre : papier, crayon, un tableau blanc. Aucun logiciel. **Durée indicative :** 3 h. **Niveau :** N1.

L'idée du chapitre. Personne ne paie pour des données. On paie pour une **décision prise plus juste, plus tôt**. Entre la ligne sortie d'une caisse et la décision du directeur, il y a trois transformations, et chacune a un nom, un test de validité et des erreurs typiques. Les apprendre avant de toucher Excel change durablement la qualité de votre travail : vous ne chercherez plus « un chiffre », vous chercherez le maillon manquant.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Distinguer** une donnée, une information, une connaissance et une décision, et classer 10 énoncés courants dans ces quatre cases ;
2. **Formuler** une **question analytique** correcte : unité, période, périmètre, comparaison, critère de réussite ;
3. **Traduire** une demande professionnelle floue (« est-ce que ça marche bien ? ») en deux ou trois questions traitables, et **explicit**er la décision attendue ;
4. **Dire** ce qu'un chiffre ne dit pas : ce qui manque, ce qui est supposé, ce qui est caché par le choix de la moyenne ;
5. **Écrire** une note d'une page qui remonte du chiffre à la décision, sans jargon.

2. Pourquoi cette notion est importante

Situation 1 — le tableau de bord inutile. Un analyste passe trois semaines sur un tableau de bord de 14 indicateurs. Le directeur l'ouvre deux fois, puis plus personne. Cause unique : aucun des 14 indicateurs ne répondait à une décision précise. On ne mesure pas ce qui est mesurable, on mesure ce qui **change quelque chose**.

Situation 2 — le chiffre juste, la conclusion fausse. « Le magasin 5 a réalisé 36 073 185 FCFA de chiffre d'affaires en 2025. » Exact. Et totalement inerte : est-ce bien ? Est-ce mieux que 2024 ? Ce total est-il dû à trois gros clients ? Un chiffre sans comparaison autorise toutes les lectures — y compris la mauvaise, faite par le plus fort en réunion.

Situation 3 — la bonne réponse à une mauvaise question. On demande : « pourquoi les ventes baissent ? » L'analyste démontre que les ventes **n'ont pas baissé** en volume mais que le prix moyen a baissé. Il avait raison, il a perdu du temps, et la direction a perdu confiance, parce que la demande portait en réalité sur la **marge**. Cadrer, c'est ouvrir le mot « pourquoi » pour voir la décision cachée dessous.

Dans les faits. Dans une étude restée classique sur la culture « pilotée par la donnée » (McAfee & Brynjolfsson, Harvard Business Review, 2012), les entreprises qui décident sur mesurabilité plutôt que sur l'intuition enregistraient environ **5 à 6 % de productivité** et **6 % de profit** de plus que les autres pour la même année. Ce n'est pas un chiffre du cours : c'est l'argument de vente de votre futur métier, et il tient en une phrase — la donnée fait gagner de l'argent quand elle **déplace une décision**.

3. Explication simple

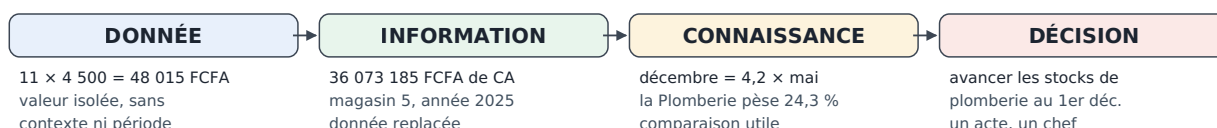
Prenons un fait tout bête : sur un ticket du 1^{er} janvier 2025 au magasin 5, onze sacs de plâtre à 4 500 FCFA.

- C'est une **donnée** : une valeur brute, enregistrée, sans commentaire. Elle ne dit rien.
- Additionnez, filtrez, datez : « magasin 5, année 2025, total des lignes facturées : **36 073 185 FCFA** » devient une **information**. Elle dit quelque chose, elle est vérifiable, mais elle ne juge pas.
- Comparez : « **décembre 2025 = 5 292 517 FCFA, mai 2025 = 1 258 801 FCFA : le meilleur mois fait 4,2 fois le pire** ». Voilà une **connaissance** : elle explique un écart et peut être contestée par des faits.
- Décidez : « **dès le 20 novembre, on double le stock de plomberie et de sacs de plâtre au magasin 5, responsable M. Bationo, budget +1,4 M FCFA** ». C'est une **décision** : elle a un acteur, une date, un coût, et elle est réversible.

Retenez la phrase qui sert de boussole dans tous les modules :

Une donnée sans contexte est une information manquée ; une information sans comparaison n'est pas une connaissance ; une connaissance sans décision est du travail perdu.

La chaîne qui transforme une ligne de caisse en décision



Tout le métier est dans les deux flèches du milieu : sans comparaison, rien ne se décide.

Demande reçue : « le magasin 5 marche bien ? » — ni prouvable, ni réfutable.
Question analytique : « **quel mois de 2025 s'écarte le plus de la médiane mensuelle ?** »

Figure 2 — Les quatre maillons, chiffrés sur le fichier réel de l'atelier (ventes_magasin5_2025.csv).

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Donnée — data	Fait brut enregistré : une valeur, une case, une ligne.	Croire qu'elle est « la réalité » : elle est ce que le système a bien voulu noter.
Information	Donnée replacée : quoi, qui, quand, combien, dans quelle unité.	Confondre avec un résultat : sans unité ni période, ce n'est pas une information.

Français — English		Définition simple	Piège à éviter
Connaissance insight	—	Information comparée, qui explique ou surprend, et dont on peut discuter.	Publier une connaissance sans sa méthode : elle devient une opinion.
Décision		Engagement : une action, un responsable, une date, des conséquences.	« Il faut regarder de plus près » n'est pas une décision.
Question analytique		Question fermée, chiffrable, qui précède toute extraction.	Écrire une question dont la réponse est un adjectif (« bon », « mauvais »).
Critère réussite	de	Ce qui prouvera, à la fin, que la question a été traitée (valeur attendue, tolérance, validateur).	Ne pas le fixer avant de calculer : on se justifie après coup.
Périmètre scope	—	Parties retenues : magasins, clients, périodes, types d'opérations.	Le périmètre implicite est la première cause de deux totaux différents pour « la même » question.
Métrique		Définition de calcul d'un indicateur, écrite noir sur blanc.	Deux personnes qui calculent le CA TTC avec des règles différentes sans le savoir.
Indicateur — KPI		Chiffre suivi dans le temps, rattaché à un objectif.	Un KPI sans objectif ni seuil est un nombre que l'on regarde passer.
Biais de cadrage		La question initiale oriente déjà la réponse.	Accepter la formulation du demandeur sans chercher la décision sous-jacente.

5. Cours approfondi

5.1 Les quatre maillons, avec leurs tests de validité

Chaque maillon a une question de contrôle. C'est ce qui rend le travail **vérifiable** au lieu de personnel.

Maillon	Ce qu'on ajoute	Test de validité	Défaut typique
Donnée	rien — on la recueille	la ligne existe-t-elle dans la source ? sa date est-elle la date du fait ou de la saisie ?	date de saisie prise pour date de vente
Information	agrégation contexte	+ le total est-il reproductible avec la même définition et le même périmètre ?	« total 2025 » sans dire si les retours sont inclus
Connaissance	comparaison hypothèse	+ l'écart survit-il à un autre calcul raisonnable ?	conclusion tirée d'une moyenne, sans regarder la médiane
Décision	engagement échéance	+ qui, quand, avec quoi, et à quel signe saura-t-on que ça a marché ?	recommandation sans acteur ni indicateur de suivi

La règle métier qui découle du tableau : **on ne présente jamais un maillon tout seul**. Une information s'accompagne de sa définition de calcul et de son périmètre ; une connaissance, de son test de robustesse ; une décision, de son indicateur de suivi.

Définition. Donnée — data — un fait enregistré à un instant précis par un système, avec une unité : une valeur de case. Elle n'est pas « la réalité », elle est ce que le logiciel a bien voulu noter, et rien de plus.

Définition. Information — une donnée replacée dans son contexte : quoi, qui, quand, combien, dans quelle unité. La même valeur cesse d'être ambiguë dès qu'on la date et qu'on la rattache à une source.

Définition. Connaissance — insight — deux informations comparées plus une explication que l'on peut discuter. Elle se reconnaît à ce qu'elle surprend quelqu'un de compétent et qu'elle appelle une réaction.

Définition. Décision — un engagement : une action, un responsable, une date, des conséquences assumées. C'est le seul maillon qui change l'état de l'entreprise.

5.2 Les six phrases d'une demande professionnelle, et comment les ouvrir

Les demandes arrivent presque toujours sous six formes. À chaque forme correspond une ouverture, à faire **avant** de toucher un fichier.

La demande	Ce qu'elle veut dire le plus souvent	Question analytique qui en sort
« Est-ce que ça marche bien ? »	juger une performance contre une attente	« le CA mensuel du magasin 5 en 2025 dépasse-t-il l'objectif mensuel, et de combien ? »
« Pourquoi ça baisse ? »	identifier un facteur de variation	« la baisse de marge vient-elle du prix moyen, du mélange de catégories ou des remises ? »
« Compare-nous les magasins »	classer et expliquer les écarts	« à superficie comparable, quel écart de CA par vendeur entre magasin 5 et magasin 2 ? »
« Fais un tableau de bord »	suivre dans le temps un petit nombre de décisions	« quels sont les trois indicateurs dont la direction décide quelque chose chaque lundi ? »
« Combien on a fait ? »	un chiffre officiel, avec définition	« CA TTC encaissé, retours déduits, magasin 5, du 01/01 au 31/12/2025 : ... »
« Tu peux me sortir ça rapidement ? »	urgence = périmètre à réduire	« que doit contenir la réponse pour être utile lundi, et que peut-on laisser de côté sans mentir ? »

Comment ouvrir une demande : la méthode des trois « pourquoi » et du « et alors ».

1. Pourquoi voulez-vous ce chiffre ? → pour arbitrer X.
2. Pourquoi maintenant ? → une décision a une échéance.
3. Et si la réponse est celle que vous n'attendez pas, que changez-vous ? → cette question, la plus utile, révèle la décision.
4. « Et alors ? » appliqué au résultat : si la réponse ne change rien, le chiffre ne sera jamais lu. Vous venez d'économiser une semaine de travail à tout le monde.

Ces quatre phrases se tiennent en une minute dans un couloir ; elles font gagner une semaine et évitent un conflit. Dans un cabinet ou une DSI, cette minute s'appelle le brief ou le cadrage ; le document qui en sort, le **cahier des questions**.

5.3 La question analytique : une grammaire, cinq composants

Une question analytique recevable contient **cinq** composants. Sans l'un d'eux, elle se transforme en débat d'opinion.

Quel [OBJET : mois de l'année 2025]
présente le CA TTC [UNITÉ + DÉFINITION : encaissements hors retours, TVA 18 % incluse]
le plus éloigné de la médiane [COMPARAISON + RÈGLE]
au magasin 5 [PÉRIMÈTRE] ?

- **Objet** — l'entité que l'on compte ou mesure : mois, produit, vendeur, client, ticket.
- **Unité et définition** — « FCFA TTC », « nombre de tickets », « quantité en kg ». Écrire la **formule** (« somme de `montant_ttc`, retours inclus avec leur signe ») interdit les deux réponses différentes.
- **Comparaison** — par rapport à quoi : un objectif, l'année précédente, la médiane, un autre magasin. « Grand » n'existe pas.
- **Périmètre** — magasins, période, catégories, clients exclus, mois partiels de 2026.
- **Critère de réussite** — ce qui prouvera la réponse : une valeur attendue avec tolérance (« mon calcul doit redonner 36 073 185 FCFA au fichier propre, à ± 1 ligne près ») et un nom (qui valide).

Le test de qualité, à appliquer vous-même avant de travailler : deux analystes indépendants, avec cette seule phrase, produiraient-ils le même chiffre ? Si non, la question n'est pas prête.

Attention. Le **périmètre implicite** est la première cause de désaccord en analyse : le demandeur pense « mes magasins », vous comprenez « les six magasins du fichier », la direction pense « hors dépôts et hors retours ». Écrivez le périmètre — lieux, période, nature des opérations, exclusions — et faites-le valider avant la première extraction.

5.4 Un chiffre n'est jamais seul : ce qu'il faut dire autour

Une information correcte peut mentir par omission. La checklist, à réutiliser telle quelle en M12 (communication) et à la maintenance M22 :

1. **Définition de calcul** — la formule, en français.
2. **Périmètre** — ce qui est dedans, ce qui est exclu, et les données manquantes (ici : 18 lignes sans client identifié, 486 lignes au montant stocké en texte dans le fichier brut de l'atelier).
3. **Période** — début, fin, et années partielles signalées.
4. **Incertitude** — ce qui dépend d'un choix (inclure ou non les retours ; nettoyer ou non les 9 lignes en double).
5. **Comparaison** — un chiffre brut ne dit rien sans repère.
6. **Limites** — ce que la donnée ne peut pas répondre (ici : aucune table de coûts de structure dans le jeu de l'atelier, donc la **marge nette** est hors de portée, seule la marge brute est calculable).
7. **Source et date** — quel fichier, quelle version, calculé quand.

Sept lignes écrites honnêtement. C'est ce qui sépare un chiffre « que personne ne conteste » d'un chiffre « qu'on ne regarde pas ».

Définition. Métrique — measure — la définition de calcul d'un indicateur, écrite noir sur blanc : ce qui entre, ce qui sort, sur quelle période, comment on additionne. Une métrique non écrite est une métrique différente selon les gens.

Définition. Indicateur — KPI (key performance indicator) — une métrique suivie dans le temps parce qu'elle est rattachée à un objectif et à un seuil qui déclenche quelque chose. Sans objectif ni seuil, ce n'est pas un indicateur : c'est un nombre que l'on regarde passer.

5.5 Le triangle de la décision : coût, délai, preuve

Toute demande se situe dans un triangle. Vous ne pouvez pas optimiser les trois à la fois :

- **Décision urgente, coût faible** (fermer le magasin un samedi) → une approximation solide suffit ; l'ordre de grandeur et le sens de l'écart.
- **Décision chère, délai court** (commander un camion de stock) → il faut une fourchette et une méthode explicite, pas un chiffre unique.
- **Décision publiée** (bilan, arbitrage entre magasins, note au conseil) → la preuve prime sur la vitesse : définition écrite, périmètre validé, journal, reproductibilité.

Un bon cadrage commence par demander **quelle décision** dépend du chiffre : c'est elle qui fixe le niveau d'exigence, pas l'outil, pas l'habitude de la direction. En entretien d'embauche, cette phrase fait plus d'effet que n'importe quelle liste de logiciels.

Conseil professionnel. Notez, dans votre journal, la **décision attendue** de chaque analyse, et, six mois plus tard, si elle a été prise. Ce qui vous rend professionnel n'est pas la finesse du modèle : c'est le taux de décisions réellement déplacées par votre travail. Un analyste qui a ce taux à 30 % est précieux ; celui qui ne l'a jamais mesuré est remplaçable.

Attention. Le piège du vocabulaire, le plus fréquent en début de parcours : **une donnée n'est pas une information**. La valeur 56 658 dans une cellule n'est pas le chiffre d'affaires ; elle devient information quand on sait qu'elle est TTC, en FCFA, pour la ligne 1 du ticket du 1^{er} janvier au magasin 5. De la même façon, une information n'est pas une connaissance : « 36 073 185 FCFA » sans comparaison ne permet à personne de décider. Si vous reprenez le vocabulaire mais pas les frontières entre les quatre mots, le chapitre n'a servi à rien.

Attention. « Il faut regarder de plus près » n'est pas une décision, et « j'ai analysé les ventes » n'est pas un résultat. Une décision s'écrit avec un verbe d'action, un responsable, une date et ce que l'on accepte de perdre. Une note qui n'en contient aucune se range au lieu de se appliquer.

À retenir. Un chiffre seul ne veut rien dire. Avant d'écrire quoi que ce soit, notez autour du nombre la période, le périmètre, l'unité, la source et le calcul : ces cinq éléments occupent trois lignes et suppriment la moitié des allers-retours. C'est la différence entre un fichier et une analyse.

6. Exemple concret : la même ligne, quatre niveaux

Le fichier de l'atelier, ligne 1 : T05-250101-075845 ; 2025-01-01 ; 11:01 ; Magasin 5 – Kaya Marché ; Moussa Ilboudo ; client 10857 ; Plâtre de construction 25 kg – réf 1 ; 11 ; 4 500 ; 0,03 ; 48 015 ; 56 658.

Niveau	Énoncé	Ce qui a été ajouté	Ce qui reste à dire
Donnée	11 sacs à 4 500 FCFA	rien	était-ce une commande ou un enlèvement ?
Information	magasin 5, 2025, 316 tickets, 489 lignes, CA TTC 36 339 317 FCFA tel que reçu	agrégation, filtre date + magasin	le total inclut 9 lignes en double
Information nettoyée	36 073 185 FCFA après retrait des doublons	décision de nettoyage (+ 0,74 % d'écart)	la définition : retours inclus ?
Connaissance	décembre 5 292 517 vs mai 1 258 801 → ratio 4,2 ; Plomberie = 24,3 % du CA	comparaison regroupement	+ l'écart est-il dû aux clients ou aux produits ?

Niveau	Énoncé	Ce qui a été ajouté	Ce qui reste à dire
Décision	réapprovisionnement anticipé au 20/11, responsable des achats, budget +1,4 M FCFA	acteur, échéance, coût, indicateur	comment saura-t-on que c'était juste ?

Notez le chiffre-clé du passage de la ligne 2 à la ligne 3 : un total change de **266 132 FCFA** (36 339 317 – 36 073 185) à cause de 9 lignes dupliquées. Une décision prise sur le premier chiffre l'aurait été sur une surévaluation de **0,74 %**. Voilà, en miniature, pourquoi les modules 3 à 5 (nettoyage, contrôle, documentation) précèdent l'analyse : sans eux, la connaissance est faussée dès l'information.

7. Démonstration pas à pas : ouvrir une demande et la rendre traîtable

Exécutons, sur papier, le cadrage complet d'une demande réelle du fil rouge. **La demande** (mail du directeur général) : « Les remises augmentent, on donne trop. Regarde ça et dis-moi ce qu'il en est. »

Étape 1 — Repérer les mots non mesurables. « augmentent » (par rapport à quand ?), « trop » (par rapport à quoi ?), « ça » (quel objet ?). Trois mots, trois questions.

Étape 2 — Ouvrir la décision cachée (les trois pourquoi). Réponse du directeur, à imaginer et à demander : « le conseil d'administration veut réduire la charge de remise de 20 % l'an prochain, sans perdre les gros clients ». La décision est donc : falloir-il resserrer la politique de remise, et sur qui ?

Étape 3 — Écrire la question analytique.

« En 2025, magasin 5 : la remise moyenne par ticket et la part de tickets à remise supérieure à 5 % varient-elles selon le type de client, et quel serait l'effet d'un plafond à 5 % sur le CA TTC des 20 clients les plus remisés ? »

Elle contient objet (ticket), unité (pourcentage de remise, FCFA de CA), comparaison (par type de client ; avant/après plafond), périmètre (2025, magasin 5), et une amorce de critère de réussite (l'effet chiffré sur 20 clients).

Étape 4 — Écrire le critère de réussite et les limites, avant de calculer. Critère : le CA TTC du périmètre doit redonner **36 073 185 FCFA** sur le fichier propre, sinon mon filtre est faux. Limites : le fichier ne permet pas de savoir si un client a acheté **ailleurs** (autres magasins) ni s'il a négocié d'autres conditions ; et l'effet d'un plafond suppose un comportement client que la donnée ne contient pas. Cette limite doit figurer dans la réponse, sinon le chiffre est un mensonge poli.

Étape 5 — Choisir le niveau de preuve. Décision vue au conseil → colonne « preuve » du triangle : le chiffre sera contestable, donc périmètre + définition + journal, et non un simple pourcentage sorti d'un tableau croisé.

Étape 6 — Rédiger la note d'une page (structure imposée par le manuel).

1. Décision visée	2 lignes
2. Question traitée	2 lignes
3. Réponse, chiffre en gras	3 lignes + tableau
4. Méthode et périmètre	4 lignes
5. Ce que cela ne dit pas	3 lignes
6. Recommandation + échéance	2 lignes

Une note qui ne tient pas dans cette structure ne sera pas lue. Vous l'écrivez **avant** l'analyse, avec des trous ; remplir les trous est alors un travail, pas une improvisation.

8. Erreurs fréquentes

Symptôme	Cause	Correction
Trois semaines de travail, réponse inutilisée	Question jamais écrite, demandeur jamais relancé	Cadrage de 10 min avec le demandeur, note d'intention d'une page, validation avant extraction
Deux totaux différents, chacun « juste »	Définitions implicites divergentes (retours inclus ou non, TTC ou HT, périmètre)	Écrire la formule, l'unité, la période, le périmètre dans un dictionnaire partagé
« Le CA baisse » dit avec assurance	Comparaison mal choisie (mois partiel contre mois plein, année partielle 2026 contre 2025 pleine)	Toujours préciser les bornes de période ; dans le jeu de l'atelier, 2026 couvre janvier à août seulement
Le chiffre exact est rejeté	Présenté sans méthode ni limites	Ajouter la note de périmètre et l'incertitude : cela rend le chiffre défendable
La demande est réalisée mais « trop lente »	Confusion urgence / exigence, tout produit au même niveau	Classer chaque demande dans le triangle (coût · délai · preuve) et négocier le niveau
Confusion donnée / information dans un mémoire ou un oral	Vocabulaire employé comme un ornement	Repère : une information se réfute par un fait ; une donnée, non. Si l'énoncé est réfutable, c'est de l'information

9. Bonnes pratiques professionnelles

- [] Une question analytique écrite avant toute extraction, avec ses cinq composants.
- [] Un critère de réussite chiffré et une date de validation fixés **à l'avance**.
- [] La définition de calcul de chaque indicateur notée dans un dictionnaire commun (elle vivra plus longtemps que le fichier).
- [] Le périmètre rappelé dans chaque réponse, y compris à l'oral : « magasin 5, 2025, retours inclus ».
- [] La limite écrite même si elle dérange : une limite annoncée est un crédit, une limite découverte est un procès.
- [] Une note d'une page plutôt qu'un classeur de 30 onglets.
- [] « Et alors ? » appliqué à chaque indicateur avant de le construire.
- [] Les six mois de suivi : noter si la décision a été prise.

10. Exercice guidé — les cinq demandes du patron, faites ensemble

Contexte. Le directeur général de Sahel Distribution passe à votre bureau et lâche, en dix secondes : (a) « Je veux savoir si le magasin 5 est meilleur que les autres. » (b) « Nos prix sont-ils trop bas ? » (c) « Combien on a vendu de plâtre ? » (d) « Le commercial Bationo mérite-t-il sa prime ? » (e) « On perd des clients ? »

On reprend une par une ; à chaque fois, remplissez les cinq composants. Voici la méthode appliquée à (a) et (b), puis vous ferez les trois autres.

(a) « le magasin 5 est meilleur que les autres »

- Ce qui cloche : « meilleur » n'est pas mesurable ; « les autres » n'est pas un périmètre (6 lignes dans `magasins.csv`, dont un dépôt qui ne vend pas au comptoir).
- Décision probable : arbitrer un budget d'agrandissement ou de transfert de personnel.
- Question réécrite : « Sur 2025, le CA TTC par m² de surface de vente et le CA TTC par vendeur du magasin 5 se situent-ils au-dessus de la médiane des cinq magasins, en excluant le dépôt ? »
- Critère de réussite : les deux ratios recalculés doivent apparaître dans le même tableau, avec le numérateur (période, retours inclus) et le dénominateur (surface en m² issue de `magasins.csv`, 6 lignes).
- Limite : le fichier ne contient pas de coût de structure, donc « meilleur » ici = plus rentable **en apparence**, pas plus rentable en marge nette. À écrire noir sur blanc.

(b) « nos prix sont-ils trop bas ? »

- Ce qui cloche : « trop bas » par rapport à la concurrence, au coût, ou à l'élasticité ? Trois questions différentes, trois jeux de données différents (le troisième n'existe pas ici).
- Décision probable : réviser la grille tarifaire avant la campagne de décembre.
- Question réécrite : « Entre 2023 et 2025, le prix moyen HT du mètre cube de gravier 15/25 a-t-il augmenté moins vite que son coût d'achat HT, et dans quelle proportion la remise accordée explique-t-elle l'écart de marge brute ? »
- Ce que la donnée permet : `couts_achat.csv` (1 694 lignes de prix d'achat par période) et les `prix_unitaire_ht` des ventes → l'écart prix/coût est calculable.
- Ce qu'elle ne permet pas : le prix des concurrents. Réponse honnête : « mesurable en interne, pas en externe, sauf relevé terrain ». Cette réponse est un résultat professionnel, pas un échec.

Vos trois autres (à faire maintenant, corrigés au §12) : réécrire (c), (d), (e) en questions analytiques complètes, en indiquant pour chacune **la table** où se trouve la réponse et **ce qui manque**.

11. Exercices autonomes

Exercice 2.1 (★) — Classement. Pour chacun des 10 énoncés suivants, indiquez D (donnée), I (information), C (connaissance) ou X (décision), et corrigez les énoncés mal placés pour les faire monter d'un niveau : 1) 4 500 FCFA ; 2) 1 406 sacs de plâtre vendus au magasin 5 en 2025 ; 3) le magasin 5 vend 22 % de plus que le magasin 2 ; 4) facturer la livraison 5 000 FCFA à partir du 1^{er} novembre ; 5) le client 10857 ; 6) la remise moyenne des 20 premiers clients est de 6,4 %, soit 2 points de plus que les autres ; 7) décembre est le mois le plus rentable ; 8) il faudrait faire quelque chose sur les retours ; 9) le ticket moyen est de 114 156 FCFA ; 10) supprimer la gratuité de livraison pour les achats de moins de 50 000 FCFA. (attendu : 1-D, 2-I, 3-C, 4-X, 5-D, 6-C, 7-C, 8 — aucune des quatre : c'est une intention, pas une décision)

Exercice 2.2 (★) — Cinq composants. Réécrivez en question analytique complète : « combien de clients nous avons perdus ? ». (attendu : définition de « perdu » (nombre de mois sans achat ?), période, périmètre, unité (nombre de clients distincts, pas de lignes de vente), comparaison (N-1), critère de réussite)

Exercice 2.3 (★★) — Et alors ? Vous calculez que le 13 septembre 2025, un seul jour, réalise 4 714 858 FCFA au magasin 5. Écrivez les deux phrases qui rendent ce chiffre utile, et la phrase qui montre qu'il peut être trompeur. (attendu : comparaison à la médiane quotidienne, contexte (livraison de chantier), limite (un jour ≠ une tendance))

Exercice 2.4 (★★) — Le chiffre qui dérange. Un total passe de 36 339 317 à 36 073 185 FCFA après suppression de 9 lignes en double. Rédigez les deux phrases à envoyer au directeur : une pour annoncer, une pour prévenir du risque si l'on garde l'ancien total. (attendu : annonce = 0,74 % de surévaluation, motif = doublons ; risque = décision de stock sur un volume fictif)

Exercice 2.5 (★★) — Note d'une page. À partir du §6 de ce chapitre, rédigez la note en six rubriques sur le sujet : « faut-il avancer les réapprovisionnements au 1^{er} décembre ? », en utilisant uniquement les chiffres cités. (attendu : 6 rubriques, ≤ 1 page, les 7 éléments de véracité du §5.4, aucune recommandation sans acteur ni échéance)

12. Correction détaillée

Exercice 2.1. 1-D (valeur sans contexte) ; 2-I (agrégation + période + périmètre) ; 3-C (comparaison entre deux objets) ; 4-X (action, date, effet) ; 5-D (un identifiant n'est pas une information tant qu'on ne dit pas à quoi il sert) ; 6-C ; 7-C **seulement si** le critère « rentable » est défini — sinon 7 est une information mal typée, « rentable » exigeant une marge, et le jeu ne contient pas de coûts de structure ; 8 = aucune catégorie, c'est une intention : à transformer en décision (quoi, qui, quand) ; 9-I ; 10-X. Correction d'un énoncé : « 1) 4 500 FCFA » devient « 1) le prix unitaire HT du plâtre 25 kg au magasin 5, en 2025, est de 4 500 FCFA » → il passe en I.

Exercice 2.2. Modèle recevable : « Entre le 01/01/2024 et le 31/12/2025, combien de clients ayant acheté au moins 3 fois en 2024 n'ont effectué aucun achat entre le 01/07/2025 et le 31/12/2025, magasin 5 et dépôt exclus, et comment ce nombre se compare-t-il à la même définition un an plus tôt ? ». Composants vérifiés : objet = clients distincts ; unité = nombre de clients, pas de lignes ; comparaison = même définition à N-1 ; périmètre = deux magasins exclus ; critère = le calcul doit s'appuyer sur `id_client` et exclure `id_client = 0` (comptoir, 18 lignes manquantes dans l'échantillon de l'atelier + `id_client=0` pour le comptoir : deux cas distincts). Un candidat qui répond « le nombre de lignes de vente sans client » est hors sujet : on compte des clients, donc des identifiants distincts.

Exercice 2.3. Phrases utiles : « ce jour représente 2,3 % du CA de décembre et 2,7 fois le CA médian d'un jour de vente du magasin 5 » ; « il correspond à une livraison de chantier de plomberie, visible par 3 tickets supérieurs à 200 000 FCFA ». Phrase d'honnêteté : « un jour exceptionnel ne prouve rien sur la tendance ; l'argument saisonnier exige les 12 mois ».

Exercice 2.4. Annonce : « Le total de l'extrait 2025 du magasin 5 était surévalué de 266 132 FCFA (+0,74 %) : neuf lignes figuraient deux fois dans le fichier reçu. Après retrait, le CA TTC de l'année s'établit à 36 073 185 FCFA. » Prévention : « tant que les doublons restent, tout calcul de volume, de panier moyen et de besoin de stock est gonflé d'autant — soit environ 1,5 % de stock en trop sur un budget de 18 M FCFA. »

Exercice 2.5. Barème de la note (12 points) : 2 pour la décision visée formulée en une ligne ; 2 pour la question avec cinq composants ; 2 pour les chiffres exacts (décembre 5 292 517 FCFA, mai 1 258 801 FCFA, ratio 4,2) ; 2 pour la méthode et le périmètre (fichier propre, 480 lignes utiles sur 489, magasin 5) ; 2 pour les limites (aucun coût de structure, donc marge nette non mesurable ; extrait, donc pas de comparaison avec les autres magasins) ; 1 pour la recommandation avec acteur et échéance ; 1 pour la lisibilité (une page, six rubriques, pas de jargon). Un devoir qui ajoute un indicateur inventé (taux de service, rotation de stock) est pénalisé : ces données ne sont pas dans le périmètre.

13. Mini-projet M01.P2 — « Le cahier des questions du magasin » (40 min)

Objectif : produire le document qui, en entreprise, s'appelle un cahier des questions ou une note de cadrage.

Livrable : un fichier `03_analyse/2026-09-17_cahier_questions_M5_minani.txt` contenant :

1. **cinq demandes** recueillies (réelles auprès de commerçants, parents, patrons de stage, ou issues du fil rouge : on vous en fournit trois, vous en ajoutez deux) ;
2. pour chacune : la décision visée, la question analytique complète, les tables du socle nécessaires (ventes, clients, produits, objectifs, stocks...), ce qui manque, et le niveau du triangle coût-délai-preuve ;
3. une sixième ligne, la plus utile : « questions que je ne peux pas traiter faute de donnée » — trois minimum.

Critères : chaque question contient les cinq composants ; aucune ne contient un adjectif non chiffré ; la rubrique « ce qui manque » est renseignée pour les six ; le tout tient sur une page imprimée en A4 à 11 pt. C'est le document d'entrée du **projet M01.P** (§projet du module) : gardez-le, il y sera noté.

14. Résumé du chapitre

Quatre maillons, et chacun a son test : la donnée existe dans la source, l'information est reproductible avec la même définition et le même périmètre, la connaissance survit à un autre calcul raisonnable, la décision a un acteur et une échéance. Votre valeur ne se mesure pas au nombre de chiffres produits mais au nombre de décisions déplacées. Entre les deux, le métier tient dans une phrase correctement écrite : objet, unité, période, périmètre, comparaison, critère de réussite — plus les limites, dites avant que quiconque ne les découvre.

15. À retenir

À retenir. 1. **Donnée ≠ information ≠ connaissance ≠ décision** : la frontière est le contexte, la comparaison, l'engagement. 2. Une **question analytique** a cinq composants (objet, unité+définition, comparaison, périmètre, critère) ; sans eux, deux analystes divergent. 3. **Toujours dire ce que le chiffre ne dit pas** : périmètre, incertitude, limites. Une limite annoncée protège ; une limite découverte détruit. 4. Le **triangle coût · délai · preuve** décide du niveau d'exigence — pas le logiciel. 5. **Un chiffre qui ne change aucune décision n'est pas un résultat**, c'est un passe-temps.

16. Évaluation formative (auto-correction, 8 min)

1. Un collègue écrit : « le CA du magasin 5 est bon ». Quelle catégorie (D/I/C/X) et pourquoi ? (aucune des quatre : jugement sans comparaison ni définition ; à convertir en question analytique)
2. Donnez la formule de calcul qui manque à « 36 073 185 FCFA » pour que ce soit une information défendable. (somme de `montant_ttc`, magasin 5, du 01/01 au 31/12/2025, retours inclus avec leur signe, doublons retirés, TVA 18 % comprise)

3. Pourquoi décembre = $4,2 \times$ mai n'est-il **pas** encore une décision ? (pas d'acteur, pas d'échéance, pas de coût ; c'est une connaissance, exploitable par quelqu'un d'autre)
4. Citez une limite structurelle du jeu de données du fil rouge qui empêche de répondre à « sommes-nous rentables ? ». (aucun coût de structure : loyers, salaires, transport absents → marge nette non calculable)
5. **Question ouverte** : réécrivez en question analytique la demande « est-ce qu'on donne trop de remises ? » et indiquez, en une ligne, comment vous saurez que votre réponse est correcte. (réponse libre, évaluée sur la présence des cinq composants + un critère chiffré)

M01.C03 — La table : lignes, colonnes, cellules, en-têtes, identifiants

Outil de ce chapitre : Excel et LibreOffice Calc (les deux sont montrés côte à côte), plus un éditeur de texte pour voir le fichier tel qu'il est. **Durée indicative :** 5 h. **Niveau :** N1.

L'idée du chapitre. Quatre-vingt-dix pour cent du travail d'analyse se passe dans des tables. Savoir **lire** une table — c'est-à-dire expliquer ce que représente une ligne, ce que compte une colonne, ce qui identifie un objet — est une compétence plus rare et plus rentable que la connaissance d'une fonction. Tout ce qui suit, du SQL au DAX, repose sur le mot **grain** : si vous ne savez pas répondre à « une ligne, c'est quoi ? », chaque total que vous produirez sera contestable.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Décrire** une table inconnue en six phrases (objet, grain, clés, mesures, attributs, période couverte) ;
2. **Distinguer** une ligne d'une observation, une colonne d'une variable, un en-tête d'une donnée, et repérer les trois mises en forme qui cassent une table ;
3. **Identifier** la clé d'une table et expliquer pourquoi `id_ticket` et `n_ticket` ne comptent pas la même chose : calculer `COUNT(*)` et le nombre de valeurs distinctes dans un tableur ;
4. **Lire** un fichier brut tel qu'il est : séparateur, encodage, lignes de chapeau, nombres décimaux à virgule ;
5. **Diagnostiquer** un tableau mal structuré (colonnes qui sont en réalité des valeurs) et le reconnaître sans le retravailler ;
6. **Écrire** une fiche d'identité de table, le livrable réutilisé de module en module.

2. Pourquoi cette notion est importante

Situation 1 — le total double. Un analyste relie le tableau des ventes au tableau des paiements **sans savoir que le second contient trois lignes par vente** (trois échéances de paiement). Le CA ressort multiplié par trois. Personne ne comprend pendant deux jours. La cause tient en un mot : grain. Celui qui a vérifié « une ligne = ? » dans chaque table avant de la relier n'a pas eu ce problème.

Situation 2 — le client qui compte deux fois. Le rapport annonce « 1 240 clients actifs ». Un contrôle rapide montre 1 240 **lignes** pour 1 105 **identifiants distincts** : des achats multiples. Compter des lignes quand on veut des clients est l'erreur de mesure numéro un des débutants, et elle survit aux tableurs les plus chers.

Situation 3 — « le fichier ne s'ouvre pas correctement ». Un fichier de 12 000 lignes livré par un fournisseur arrive : les colonnes se collent, les accents sont cassés, les dates sont devenues des nombres. Le collègue conclut que le fichier est corrompu. Il est parfaitement valide : il est en `;`, en encodage Windows-1252, avec des décimales à la virgule. Ce chapitre vous donne les quatre réflexes qui règlent 100 % de ces cas, sans logiciel spécialisé.

Dans les faits. Le fichier de l'atelier de ce module contient **489 lignes** et **13 colonnes**. Le même, nettoyé, en contient **480**. Il compte **316 tickets**, donc **1,52 ligne par ticket en moyenne** et jusqu'à **4**. Retenez ces quatre nombres : vous allez les retrouver dans chaque exercice du module, et le jour où vous les écrirez de mémoire sans les vérifier, vous ferez une erreur de 55 % sur le nombre de ventes.

3. Explication simple

Une table, c'est un **registre** : un cahier où chaque page tient en une ligne. Deux questions suffisent pour la comprendre, et elles se posent dans cet ordre :

1. **Une ligne, c'est quoi ?** → c'est le **grain**. « Une ligne = un article vendu sur un ticket » n'est pas la même chose que « une ligne = une vente ». Dans le premier cas, 489 lignes = 489 articles ; dans le second, ce serait 489 ventes — et c'est faux, il y a 316 ventes.
2. **Qu'est-ce qui rend une ligne unique ?** → la **clé**. Ici, `n_ticket` identifie le ticket, mais il se répète sur les lignes du même ticket : il n'est donc **pas** la clé de la table. La clé est le couple (`n_ticket`, `produit`).

Les colonnes se répartissent alors en trois familles, et cette famille détermine ce que vous aurez le droit de faire plus tard :

- des **identifiants** (`n_ticket`, `client`) : ils relient à d'autres tables ;
- des **attributs** (`magasin`, `vendeur`, `categorie`, `date`) : ils décrivent, ils servent à filtrer et à regrouper ;
- des **mesures** (`quantite`, `prix_unitaire_ht`, `montant_ht`, `montant_ttc`) : elles s'additionnent ou se moyennent.

Une quatrième famille apparaît dans les fichiers réels : les **calculées** (`montant_ht` peut être reconstitué à partir de `quantite`, `prix_unitaire_ht` et `remise`). Elles sont pratiques mais dangereuses : si une colonne calculée est stockée dans le fichier et que le fichier a été édité à la main, elle peut contredire ses propres composants. C'est exactement le cas dans notre échantillon, et vous le vérifierez en exercice.

Une table vue de près : en-têtes, variables (colonnes), observations (lignes)

n_ticket	date	magasin	vendeur	client	quantite	prix_unitaire_ht	montant_ttc
T05-250101-075845	2025-01-01	Magasin 5	Moussa I.	10857	11	4 500	56 658
T05-250101-075845 ①	2025-01-01	Magasin 5	Moussa I.	0.0 ②	2	12 000	28 320
T05-250101-075846	2025-01-01	Magasin 5	Salamatu S.	③	2 000	21 950	51 802
T05-250102-075902	2025-01-02	Magasin 5	Adama B.	3312	1	14 000	16 520

① un ticket occupe 2 lignes : compter les lignes pour compter les ventes gonfle le résultat de 55 % (le fichier compte 489 lignes, 316 tickets, et jusqu'à 4 lignes pour un même ticket).

② « 0.0 » n'est pas un manque : c'est la vente au comptoir, sans client identifié. Elle est volontaire dans le fichier et doit rester dans les totaux.

③ cellule vide = valeur absente : 18 lignes du fichier sont dans ce cas. On signale, on documente, on ne remplit pas au hasard.

- Grain : une ligne = un article vendu sur un ticket ; 1,52 ligne par ticket en moyenne.
- Clé : `n_ticket` identifie le ticket ; (`n_ticket`, `produit`) identifie la ligne. Sans clé, pas de fusion propre.
- Schéma : les 13 colonnes et leur type. Le CSV ne le dit pas : le déduire, puis le vérifier.

Figure 3 — Les quatre premières lignes réelles du fichier de l'atelier, annotées.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Table — table	Rectangle de données : des colonnes nommées, des lignes sans ordre significatif.	Un « tableau » Excel avec titres fusionnés n'est pas une table, même s'il y ressemble.
Ligne — row / record	Un enregistrement : un objet du monde réel, au grain défini.	Compter des lignes pour compter des objets différents des lignes.
Colonne — column / field / variable	Une caractéristique mesurée pour tous les enregistrements.	Une colonne qui mélange deux choses (quantité et unité : « 11 sacs »).
Cellule — cell	L'intersection ligne x colonne : une valeur, une seule.	Mettre deux informations dans une cellule (25 kg / sac).
En-tête — header	Première ligne, noms des colonnes.	En-tête sur plusieurs lignes, ou lignes de titre avant l'en-tête.
Grain — granularity	Ce que représente une ligne, formulé en une phrase.	Confondre grain et niveau d'agrégation (« c'est mensuel » ne dit pas ce qu'est une ligne).
Clé primaire — primary key	— Colonne ou couple de colonnes qui rend chaque ligne unique.	Une clé qui admet des doublons n'est pas une clé ; <code>id_ticket</code> ici, ne l'est pas.
Clé étrangère — foreign key	— Colonne qui pointe vers la clé d'une autre table.	Un identifiant présent dans les ventes mais absent des clients (ici 486 lignes dans le jeu complet).
Valeur distincte — distinct value	— Une valeur, comptée une seule fois.	<code>COUNT</code> ≠ <code>COUNT DISTINCT</code> : l'un compte des lignes, l'autre des objets.
Séparateur — delimiter	— Caractère qui sépare les colonnes dans un fichier texte (;, ,, tabulation).	Le même caractère utilisé dans une valeur non protégée casse l'alignement.
Encodage — character encoding	— Table de correspondance octets ↔ caractères (UTF-8, Windows-1252).	Ouvrir un fichier UTF-8 en Windows-1252 : les accents deviennent Ã©.
Chapeau / pied de page — preamble / footer	Lignes de titre, de date, de signature avant ou après les données.	Les importer comme données : la ligne 1 de <code>tarif_fournisseur_peinture.csv</code> n'est pas une colonne.

5. Cours approfondi

5.1 Les six phrases de la fiche d'identité d'une table

Face à une table inconnue, écrivez six lignes. Ce rituel de trois minutes remplace avantageusement une demi-journée d'erreurs, et c'est le livrable (1) du projet M01.P.

1. **Nom et source** — `ventes_magasin5_2025.csv`, reçu le 17/09/2026 de la caisse du magasin 5 (export nocturne).
2. **Objet** — la table parle d'articles vendus.
3. **Grain** — une ligne = un article d'un ticket de caisse.
4. **Clé** — (`n_ticket`, `produit`) ; aucune colonne unique à elle seule.
5. **Mesures et attributs** — mesures : `quantite`, `prix_unitaire_ht`, `remise`, `montant_ht`, `montant_ttc` ; attributs : `date`, `heure`, `magasin`, `vendeur`, `client`, `produit`, `categorie`.
6. **Étendue** — 489 lignes ; dates du 01/01/2025 au 31/12/2025 (31 jours comportent au moins une ligne) ; 316 tickets ; 145 produits différents ; 7 catégories ; 4 vendeurs.

La sixième phrase est un inventaire de **bornes** : c'est elle qui révèle la moitié des anomalies, parce qu'une borne qui n'a pas de sens signale un problème avant tout calcul (voir §5.5).

Définition. Table — un rectangle de données où chaque colonne est une caractéristique mesurée pour tous les enregistrements et chaque ligne un objet du monde réel, au grain choisi. Entre deux lignes, il n’y a pas d’ordre significatif : l’ordre est une affaire d’affichage, jamais une information.

Définition. Enregistrement — row / record — la ligne, c’est-à-dire un objet : une ligne de ticket, et non un ticket, non un client, non un mois.

5.2 Le grain, seul concept qui compte au début

Écrivez le grain comme une définition d’école : « **une ligne = un-e [objet] pour [combinaison de clés]** ». Exemples pris dans le socle de données du manuel :

Table	Grain correct	Ce que l’on pourrait croire (et qui est faux)
ventes (243 360 lignes brutes)	une ligne = un article vendu sur un ticket	une ligne = une vente → 316 tickets deviennent 489 ventes
clients (23 912 lignes brutes, 23 500 propres)	une ligne = un client	une ligne = un achat → le même client apparaît plusieurs fois
objectifs_de_ca (218 lignes)	une ligne = un magasin pour un mois donné	une ligne = une année de magasin
stocks_quotidiens (60 000 lignes)	une ligne = un produit dans un dépôt à une date	une ligne = un produit
dim_date (1 339 lignes)	une ligne = un jour calendaire	une ligne = un jour ouvré
remises_manuelles (18 200 lignes)	une ligne = un client pour un mois	une ligne = une remise accordée

Trois propriétés à vérifier systématiquement :

- **l’unicité** : une clé, ça se prouve. Comptez les lignes (489) et les valeurs distinctes de la combinaison de colonnes qui devrait être unique (316 tickets, donc **non unique**) ;
- **la complétude** : le grain annonce-t-il des lignes qui n’existent pas ? Exemple réel du socle : le fichier de stocks contient des jours sans relevé — le magasinier n’a pas toujours noté. Une table « par jour » sans tous les jours fausse toute moyenne journalière ;
- **la périodicité** : les tables de type « mensuel » (objectifs, remises_manuelles) ont-elles **12 × 6 = 72 lignes attendues par an** ? Ici objectifs en compte 218 pour 5 magasins et 3 ans : il manque des mois — c’est voulu, et vous le traiterez en M12 (analyse d’écart) et M14 (objectifs non atteignables).

À retenir. Le grain n’est pas une information technique : c’est la **seule** qui détermine si un total est une somme ou un double-comptage. Un tableur, un moteur SQL, un modèle sémantique : tous, sans exception, additionnent au grain de la table.

Définition. Grain — granularity — la phrase qui dit ce que représente une ligne. Elle s’écrit avec un article et une période : « une ligne = un article d’un ticket », « une ligne = un client pour un mois ». Le grain décide de tout le reste : ce que l’on peut additionner sans double compte, ce que l’on peut joindre, ce qu’il faut agréger avant.

5.3 Compter comme un analyste : lignes, objets, valeurs distinctes

Dans un tableur, quatre comptages suffisent à décrire une colonne. Les deux outils, littéralement :

Excel (ruban) — sélectionnez la colonne, Statut (barre du bas) affiche Nombre (lignes non vides) et Nombre de valeurs numériques ; pour les distincts, données → **Supprimer les doublons** présente une boîte de dialogue qui indique combien de valeurs uniques ont été trouvées.

LibreOffice Calc — mêmes fonctions dans **Données** → **Standardfilter / Duplizierungen entfernen**, et le compteur de la barre d'état se règle par clic droit pour afficher Nombre au lieu de Somme.

En pratique, on écrit des formules, parce qu'elles restent et se recalculent :

```
=NBVALA(B2:B490)      → 489      lignes non vides (chaque type)
=NB(C2:C490)          → 489      lignes numériques seulement
=Nb.SI.ENS(C2:C490;"<>0") → bornes
=SOMMEPROD(1/FREQUENCE(SI(B2:B490<>"");EQUIV(B2:B490;B2:B490;0));EQUIV(B2:B490;B2:B490;0))) →
distincts (matricielle, à valider Ctrl+Maj+Entrée)
```

Ces formules sont désagréables à lire. C'est le signe exact qu'il faudra du SQL en M07 (COUNT(DISTINCT n_ticket)) ou Python en M08 — mais **savoir quoi compter** est la compétence, la syntaxe n'est que l'outil. Retenez le vocabulaire, commun à tous les logiciels :

- COUNT(*) = le nombre de **lignes** ;
- COUNT(colonne) = le nombre de lignes **non vides** pour cette colonne ;
- COUNT(DISTINCT colonne) = le nombre d'**objets** distincts.

Sur notre fichier : 489 lignes, 316 tickets distincts, 480 lignes après nettoyage, 372 identifiants clients distincts, 145 produits distincts. Le ratio lignes/objets (489 / 316 = 1,52) s'appelle le **nombre moyen de lignes par groupe** ; il dit à lui seul si la table est un détail (proche de 1) ou un agrégat (proche de 1, alors qu'on attend des lignes multiples).

Définition. Clé primaire — primary key — une colonne, ou un couple de colonnes, qui rend chaque ligne unique. Elle ne se décrète pas, elle se vérifie : on compare le nombre de lignes au nombre de valeurs distinctes. Égalité, c'est une clé ; écart, la ligne n'est pas ce que l'on croyait.

Définition. Clé étrangère — foreign key — une colonne qui renvoie à la clé d'une autre table. Elle ne vaut que si la valeur existe en face : un identifiant présent dans les ventes et absent des clients n'est pas une clé, c'est une perte de données.

Définition. Valeur distincte — distinct value — une valeur comptée une seule fois. Compter des lignes et compter des objets sont deux opérations différentes, et les deux chiffres sont justes : c'est la question posée qui tranche.

5.4 Les trois mises en forme qui détruisent une table

Un tableur permet des choses qu'un fichier de données ne doit jamais contenir.

1. **Cellules fusionnées.** Visuellement pratique, fonctionnellement mortel : une fusion cache la valeur dans une seule cellule, donc le tri et les formules décalent tout. Règle : aucune cellule fusionnée dans une zone de données ; si un export fournisseur en contient, on la casse avant de lire (M04).
2. **Plusieurs informations dans une cellule.** Plâtre de construction 25 kg – réf 1 : le poids et la référence sont collés au nom. Le jour où l'on veut agréger par poids, il faut redécouper. Repère : si une valeur contient un séparateur (-, / , (), soupçonnez deux colonnes.
3. **Sous-totaux et lignes de titre dans la zone de données.** Un fichier de caisse propre à imprimer contient souvent ses propres totaux : Total général : 36 339 317 en ligne 490, et le total du mois toutes les 30 lignes. Symptôme : le total calculé est exactement le double du vrai. La ligne de contrôle : compter les lignes dont la colonne n_ticket est vide ou non numérique.

Notre fichier fournisseur `tarif_fournisseur_peinture.csv` cumule trois de ces défauts : **3 lignes de chapeau** avant l'en-tête, un **point-virgule** comme séparateur, et des **décimales à la virgule**. Les 42 lignes du fichier ne contiennent donc que **38 lignes utiles**. Un débutant qui ouvre le fichier et compte 42 références se trompe de 10 %.

Attention. Trois mises en forme d'un tableur détruisent mécaniquement une table. Les **cellules fusionnées** créent des cases vides que tout outil d'import lit comme des lignes cassées. Les **lignes de titre ou de signature avant l'en-tête** font du jour d'impression un nom de colonne. Les **sous-totaux insérés dans une colonne de données** sont ré-additionnés au premier calcul. Un tableau de rapport est une impression, pas une table.

5.5 Les bornes, ou comment trouver une anomalie sans la chercher

Pour chaque mesure, notez min, max, et le nombre de valeurs aberrantes. Sur `ventes_magasin5_2025.csv` :

Colonne	Valeur à retenir	Ce que cela signifie
<code>quantite</code>	minimum négatif (retours), maximum 14 000	7 lignes dépassent 500 unités : ce sont des erreurs d'unité de saisie, pas des ventes
<code>prix_unitaire_ht</code>	323 valeurs distinctes	fourchette réaliste : pas de prix négatif, pas de zéro ici
<code>remise</code>	6 valeurs distinctes	une grille simple, donc toute valeur hors de ces six est une anomalie (M01.C04)
<code>montant_ttc</code>	486 des 489 lignes non numériques	montant stocké en texte : le total d'un tableur serait faux
<code>client</code>	372 identifiants distincts, 18 lignes vides	« vide » n'est pas la même chose que « 0 » : 0 = vente au comptoir
<code>date</code>	55 écritures différentes pour 31 jours	65 lignes en JJ/MM/AAAA : le tri alphabétique est déjà faux

Le réflexe, une fois pour toutes : **avant de calculer un total, regardez les extrêmes**. Trois secondes par colonne sur un fichier de 489 lignes ; sur `ventes_brutes.csv` (243 360 lignes), ce sera un script, et vous l'écrirez en M08.

Attention. Compter les lignes d'une table pour estimer le nombre d'objets est la plus ancienne source de surévaluation du métier. Si une ligne n'est pas l'objet que l'on compte, le total est gonflé d'un facteur qui dépend des données, et l'erreur grandit avec le volume : la bonne habitude est d'afficher les deux chiffres côte à côte.

5.6 Large, long, et le tableau « de rapport »

Deux organisations d'une même information, à savoir reconnaître (la transformation s'apprend en M04) :

format large (une colonne par mois)	format long (une ligne par mois)
magasin janvier février mars	magasin mois CA
M5 2 051 424 4 289 885 3 673 454	M5 1 2 051 424
M2 1 200 000 1 800 000 900 000	M5 2 4 289 885
	M2 1 1 200 000

Le **large** se lit par un humain dans une réunion ; le **long** se charge par un logiciel et se filtre par mois. Le tableau de rapport, lui, est les deux à la fois avec des sous-totaux imbriqués : il est **destiné à être lu**, jamais à être la source d'un nouveau calcul. Un indicateur fiable de la maturité d'un service : ses données de travail sont en long, ses livrables sont en large.

Conseil professionnel. Quand un fichier vous est livré, ne l'ouvrez **pas** en double-clic. Ouvrez-le d'abord dans un éditeur de texte (Bloc-notes, gedit, VS Code) ou par l'assistant d'import du tableur (Données → Importer depuis du texte/CSV). Vous verrez le séparateur, l'encodage, le chapeau, les guillemets, les lignes vides : tout ce qu'un double-clic décide pour vous, silencieusement, et parfois faussement.

6. Exemple concret : lire les deux premiers fichiers du socle

Fichier 1 — ventes_magasin5_2025.csv (489 lignes, 13 colonnes) :

```
n_ticket;date;heure;magasin;vendeur;client;produit;categorie;quantite;prix_unitaire_ht;remise;montant_ht;montant_t
T05-250101-075845;2025-01-01;11:01;Magasin 5 – Kaya Marché;Moussa Ilboudo;10857;Plâtre de
construction 25 kg – réf 1;Matériaux;11;4500;0.03;48015;56 658
T05-250101-075845;2025-01-01;11:01;Magasin 5 – Kaya Marché;Moussa Ilboudo;10857;Gravier 15/25 (m3)
– réf 2;Matériaux;2;12000;0.00;24000;28 320
```

Lecture professionnelle, en une phrase par ligne : deux lignes, **un seul ticket** (075845) → le grain est bien l'article. La troisième colonne de la deuxième ligne montre 28 320 : espace de milliers, donc du texte. Le ticket du haut contient un article à 12000 sans espace : incohérence de format **dans la même table**, donc dans le même export. Le champ `client` vaut 10857 : un identifiant numérique, pas un nom — il faudra le relier à la table clients pour savoir qui c'est. Le champ `remise` vaut 0.03 : une proportion, donc 3 %, jamais 3 (et 3 apparaîtra ailleurs : c'est la faute d'unité du chapitre 4).

Fichier 2 — tarif_fournisseur_peinture.csv (42 lignes, dont 38 utiles) :

```
export genere depuis ELODIE v3
edition le 17/09/2026
peinture saaba sarl | tarif 2026
code_article;libelle;conditionnement;prix_unitaire;unite_mesure;tva
PEINT-014;Peinture acrylique blanc 10 L;pot;27500,00;pot;0.18
```

Quatre enseignements : les trois premières lignes sont du **bruit de document** (chapeau) ; la ligne 4 est l'**en-tête** ; le séparateur est le ; ; la virgule est **décimale**, donc 27500,00 lu dans un contexte français devient 2750000 si l'on force le point, ou 27 500 si l'on respecte la convention — la bonne réponse n'est pas dans le fichier, elle est dans le **choix d'import** (M04). Notez aussi `tva = 0.18` : la TVA du pays, cohérente avec les règles du socle, à vérifier et non à supposer.

7. Démonstration pas à pas : la fiche d'identité complète, avec les deux tableurs

Étape 1 — Import maîtrisé. Excel : **Données → Obtenir des données → À partir de texte/CSV** ; LibreOffice : **Données → Importer une feuille de calcul**. Sélectionnez le fichier `ventes_magasin5_2025.csv`. L'aperçu s'affiche : séparateur ;, encodage **UTF-8** (si les accents sont cassés, passez en Windows-1252, ou l'inverse). Validez **sans charger** la mise en forme. Résultat attendu : 489 lignes sous l'en-tête, 13 colonnes.

Étape 2 — Les six phrases. Remplissez la fiche (§5.1) avec les nombres réels ; ne recopiez aucun nombre sans le vérifier à l'étape 3.

Étape 3 — Les bornes et les distincts. Créez un onglet `controles`, avec une ligne par colonne et les formules :

Colonne	Formule Excel	Formule LibreOffice	Attendu
<code>date</code>	<code>=MIN(--LEFT(A2:A490,10))</code>	<code>=MIN(VALUE(LEFT(A2:A490;10)))</code>	une date de janvier 2025... ou une erreur (c'est l'indice)
<code>quantite</code>	<code>=MAX(I2:I490)</code>	<code>=MAX(I2:I490)</code>	14 000
<code>quantite</code> > 500	<code>=NB.SI(I2:I490;">500")</code>	<code>=COUNTIF(I2:I490;">500")</code>	7
<code>n_ticket</code> lignes	<code>=NBVAL(A2:A490)</code>	<code>=COUNTA(A2:A490)</code>	489
<code>n_ticket</code> distincts	<code>=SOMMEPROD(1/NB.SI(A2:A490;A2:A490))</code>	<code>=SUMPRODUCT(1/COUNTIF(A2:A490;A2:A490))</code>	316
<code>client</code> vides	<code>=NB.EUILL(E2:E490)</code>	<code>=COUNTBLANK(E2:E490)</code>	18
<code>montant_ttc</code> texte	<code>=NB(L2:L490)</code>	<code>=COUNT(L2:L490)</code>	3 (les seuls numériques !)

La dernière ligne est la plus instructive : `=NB` ne compte que les nombres ; il renvoie 3 alors qu'il y a 489 montants. **Donc 486 montants ne sont pas des nombres.** Vous venez de le prouver avec une formule de trois caractères, sans macro, sans SQL.

Étape 4 — Le test du double-comptage. Faites `=SOMME(M2:M490)` sur `montant_ht` : comparez à la somme de `montant_ttc` ÷ 1,18. Les deux doivent être proches (à l'arrondi près). Si l'un est deux fois l'autre, une ligne de sous-total est restée dans les données. Ici, les deux sont cohérents — preuve que le fichier ne contient pas de sous-total, mais bien 9 doublons.

Étape 5 — Le comptage qui ment. Calculez le « nombre de ventes » de deux façons : `=NBVAL(A2:A490)` → 489, et le nombre de tickets distincts → 316. Écrivez en une phrase laquelle des deux est « le nombre de ventes » (**316**) et ce que l'autre représente (**le nombre d'articles facturés**). Cette phrase est la réponse à la moitié des questions de vos futurs collègues.

Étape 6 — Le même exercice sur le fichier fournisseur. Importez avec chapeau ignoré (Excel : « Avancé » → lignes à sauter = 3 ; LibreOffice : cochez « Première ligne » et retirez les 3 lignes après import). Vérifiez : 38 lignes utiles, 6 colonnes, `tva` = 0,18 partout, `prix_unitaire` avec virgule. Notez dans le journal le nombre de lignes **avant/après** chaque manipulation — l'habitude se prend ici, pas en M18.

8. Erreurs fréquentes

Symptôme	Cause	Correction
1 240 « clients » pour 1 105 clients réels	Comptage de lignes au lieu de valeurs distinctes	Compter <code>DISTINCT</code> ; nommer la métrique (« clients actifs » ≠ « lignes de vente »)
Le CA est exactement le double	Ligne de sous-total importée comme donnée	Supprimer les lignes sans clé ou dont la clé est vide ; vérifier avec le ratio lignes/tickets
Les colonnes du CSV sont collées	Séparateur mal indiqué à l'import	Assistant d'import, séparateur ; (ou tabulation), et vérifier l'aperçu avant de valider
Les accents écrits Ã© ou â€”	Encodage UTF-8 lu en Windows-1252 (ou l'inverse)	Changer l'encodage à l'import, pas dans le fichier ; ne jamais réenregistrer un brut
Le tri par date mélange janvier et décembre	Tri alphabétique sur des dates stockées en texte	Convertir en véritables dates (M01.C04) ou trier par une colonne auxiliaire AAAA-MM-JJ
Une date devenue 45 658	Import qui a converti une date en numéro de série	Annuler, réimporter en forçant le type de colonne en « texte », puis convertir explicitement

Symptôme	Cause	Correction
Total <code>montant_ttc</code> = 0	Somme sur une colonne texte	Le défaut du chapitre 4 : forcer la conversion après avoir retiré les espaces de milliers
Le fichier « propre » ne ressemble plus au brut	Écrasement du brut par le tableur (formats, arrondis, dates)	Le brut est en lecture seule ; on travaille sur une copie nommée <code>_propre</code>

9. Bonnes pratiques professionnelles

- [] Import par assistant, jamais par double-clic ; vérifier l’aperçu ligne 1 et la dernière ligne.
- [] Fiche d’identité en six phrases produite **avant** le premier graphique.
- [] Un onglet `controles` par table : lignes, clés distinctes, bornes, comptage des vides.
- [] Ratio lignes/objets écrit noir sur blanc (ici 1,52 ligne par ticket).
- [] Zéro cellule fusionnée, zéro sous-total dans une zone de données.
- [] Le nom des colonnes conservé tel que livré dans la copie propre, avec un dictionnaire qui traduit (M01.C06).
- [] Toute suppression de ligne documentée : nombre avant, nombre après, motif.
- [] Un fichier = une table = un grain. Pas deux niveaux d’agrégation dans le même rectangle.

10. Exercice guidé — décrire la table `clients` en 8 minutes

Point de départ. `01_socle_donnees/data/brut/clients.csv` : 11 colonnes, 23 912 lignes brutes.

1. **Combien de clients ?** Les lignes ne répondent pas. Comptez les valeurs distinctes de `id_client` : **23 500**. L’écart (412) est constitué de doublons quasi-exacts, volontairement injectés dans le jeu — vous les rencontrerez en M04. Conclusion à écrire : 412 lignes de trop, ou 412 clients en moins selon le point de vue ; le bon réflexe est de compter les identifiants, pas les lignes.
2. **Quelles colonnes sont des mesures et lesquelles sont des attributs ?** Mesures : `plafond_credit`, éventuellement `date_creation` (une date n’est pas additionnable : c’est un attribut-ordonnée). Attributs : `nom`, `type_client`, `ville`, `region`, `telephone`, `email`, `segment`, `conditions_paiement`.
3. **Le grain est-il respecté ?** Non, à cause des doublons. Test : une table au grain « un client » doit avoir autant de lignes que de valeurs distinctes de sa clé. Ici $23\,912 \neq 23\,500$.
4. **Les bornes utiles.** `ville` : 1 880 lignes vides (8 % des clients) — ce n’est pas un problème pour compter des ventes, **c’en est un** pour classer par ville. `telephone` : trois formats (avec/sans indicatif, espaces variables) — donc `COUNT(DISTINCT telephone)` sur-compte, et toute recherche par téléphone est inutile tant que ce n’est pas normalisé.
5. **Ce que la table ne dit pas.** Aucun chiffre d’affaires : c’est une table de **dimension**, pas un fait. Écrivez la phrase : « `clients` répond à qui, `ventes` répond à combien, et l’on ne peut obtenir le second par le premier qu’en reliant les deux par `id_client`. »

Contrôle de votre résultat. Vous devez avoir : 23 912 lignes, 23 500 clients distincts, 412 doublons, 1 880 villes manquantes, 3 formats de téléphone, et aucune somme de `id_client` (elle n’a aucun sens). Si votre réponse contient une moyenne d’identifiants, relisez le §5.3 : un identifiant n’est pas une quantité.

11. Exercices autonomes

Exercice 3.1 (★) — Le grain, en une phrase. Donnez le grain des six tables du socle (`ventes`, `clients`, `produits`, `magasins`, `objectifs_de_ca`, `stocks_quotidiens`). (attendu : une phrase « une ligne = ... » par table, avec la combinaison de clés)

Exercice 3.2 (★) — Deux comptages. Sans outil, déduisez du texte du fichier : combien de lignes a `ventes_magasin5_2025.csv` ? Combien de tickets ? Pourquoi ne sont-ce pas les mêmes nombres ? (489 / 316 ; parce qu'un ticket contient 1 à 4 articles)

Exercice 3.3 (★★) — Le faux total. Un collègue obtient 72 678 634 FCFA de CA TTC sur l'échantillon, au lieu de 36 339 317 FCFA. Proposez deux explications, et le test qui les distingue. (doublons non retirés → le total est presque $\times 2$: on compare à $36\,339\,317 \times 1,999$; sous-totaux inclus → le fichier n'en contient pas, donc `NBVAL(n_ticket)` ne change pas. Le test : compter les lignes dont le ticket est vide)

Exercice 3.4 (★★) — Les colonnes qui sont des valeurs. Voici un extrait large :

```
magasin;2025-01;2025-02;2025-03
M5;2051424;4289885;3673454
```

Écrivez la version longue, puis dites quelle opération devient possible en long et pas en large, et pourquoi la moyenne des trois colonnes n'est pas « le CA mensuel moyen du magasin 5 ». (attendu : 3 lignes en long ; agréger par mois sur plusieurs magasins devient possible ; la moyenne est correcte ici mais aveugle au nombre de jours ouvrés de chaque mois)

Exercice 3.5 (★★) — Lecture d'un fichier hostile. Le fichier fournisseur (42 lignes, chapeau de 3, séparateur `;`, décimale virgule, encodage Windows-1252) : importez-le proprement, et donnez le nombre de lignes utiles, le prix du `PEINT-014`, et le taux de TVA constaté. (attendu : 38 lignes utiles ; 27 500 ; 0,18 — une réponse « 2750000 » trahit un mauvais choix de séparateur décimal)

Exercice 3.6 (★★★) — Contrôle de unicité en 5 formules. Sur `montant_ht`, prouvez que la colonne est cohérente avec `quantite`, `prix_unitaire_ht` et `remise` : formule attendue, tolérance d'arrondi, nombre de lignes qui dévient. (attendu : contrôle `quantite × prix × (1 - remise)` arrondi à l'entier ; les 7 lignes de quantité aberrante sortent nécessairement en écart → c'est l'indice qui les fait découvrir)

12. Correction détaillée

Exercice 3.1. `ventes` : une ligne = un article d'un ticket (clé `id_vente`, ou `(n_ticket, produit)`) ; `clients` : une ligne = un client (clé `id_client`) ; `produits` : une ligne = une référence (clé `id_produit`, 154 lignes) ; `magasins` : une ligne = un point de vente ou dépôt (6 lignes) ; `objectifs_de_ca` : une ligne = un magasin pour un mois (218 lignes pour 5 magasins $\times$ 3 ans + 8 mois, donc mois manquants) ; `stocks_quotidiens` : une ligne = un produit dans un dépôt à une date (60 000 lignes).

Exercice 3.2. 489 lignes, 316 tickets distincts (moyenne 1,52 article par ticket, maximum 4). Les deux ne sont pas identiques parce que le grain est l'article, pas la vente. Un total d'articles est pertinent pour la logistique (kg manutentionnés, colis), un total de tickets pour la caisse (11 tickets/heure en pointe), et un total de lignes **n'est** la réponse à aucune question métier : c'est un artefact du fichier.

Exercice 3.3. Explication A : les 9 lignes en double n'ont pas été retirées **et** le collègue a sommé l'intégralité du jeu complet, pas l'échantillon. Test : son nombre de lignes est-il 243 360 ou 489 ? Explication B : il a ajouté le sous-total de chaque mois aux lignes. Test : `=NB.SI(A2:A490;"")`

— s'il trouve 0, il n'y a pas de sous-total et l'explication B tombe. Le calcul $72\,678\,634 / 36\,339\,317 = 2,00$ montre qu'il a bel et bien additionné deux fois le même contenu : une jointure ou une concaténation, pas un total de mois.

Exercice 3.4. Version longue :

```
magasin;mois;CA
M5;2025-01;2051424
M5;2025-02;4289885
M5;2025-03;3673454
```

En long, on peut agréger par mois **sur tous les magasins**, trier par mois, joindre à un calendrier (`dim_date`, 1 339 lignes) et détecter les mois absents (en large, un mois manquant est une colonne absente : personne ne la remarque). Sur la moyenne : $(2\,051\,424 + 4\,289\,885 + 3\,673\,454)/3 = 3\,338\,254$ FCFA — ce nombre est correct **comme moyenne de ces trois mois**, mais il ne dit rien d'une « année » ; l'exprimer comme « le CA mensuel » sans préciser la période est l'erreur classique relevée en M02 (agrégats sans période = agrégats contestables).

Exercice 3.5. 38 lignes utiles (42 – 3 de chapeau – 1 d'en-tête), PEINT-014 à **27 500 FCFA**, TVA **0,18**. Réponses fausses typiques : 42 lignes (le chapeau compté), 2 750 000 (virgule lue comme séparateur de milliers alors que le fichier est en français... ou l'inverse), 18 % lu comme 18 (oubli de la proportion). Le contrôle qui valide : `prix_ht × 1,18` doit redonner un prix TTC cohérent avec la grille interne du produit peinturé du socle.

Exercice 3.6. Contrôle : `montant_attendu = ARRONDI(quantite × prix_unitaire_ht × (1 – remise))`. Écart toléré : 1 FCFA (arrondi du fichier). Résultat réel sur l'échantillon : 7 lignes en écart, et ce sont exactement les 7 lignes à quantité aberrante (maximum 14 000 unités). La leçon à écrire : **une colonne calculée peut être juste alors que sa composante est fausse** — le fichier a été multiplié sur la quantité seulement, donc le montant reste correct. Un contrôle de cohérence entre colonnes trouve ce qu'un contrôle de total ne trouve pas.

13. Mini-projet M01.P3 — « Fiche d'identité de trois tables » (50 min)

Livrable : un onglet `identification` dans un classeur `2026-09-17_fiches_identite_M5_minani.xlsx`, avec **trois** blocs (ventes de l'atelier, clients, fichier fournisseur), chaque bloc contenant les six phrases du §5.1 et le tableau de contrôles du §7 (étape 3).

Contraintes : aucune formule ne doit être recopiée d'un autre bloc sans adaptation ; chaque nombre doit venir d'une formule du fichier importé (pas du texte du chapitre) ; le dernier bloc liste les **trois questions** que la table ne permet pas de traiter. Barème : 8 pts exactitude des six phrases · 6 pts justesse des comptages (489/316/18/3 vérifiés par vos formules) · 4 pts lisibilité (un bloc par table, aucun fusionnement) · 2 pts « questions non traitables ». Ce livrable entre dans le projet M01.P, critère (1).

14. Résumé du chapitre

Une table vue de près : en-têtes, variables (colonnes), observations (lignes)

n_ticket	date	magasin	vendeur	client	quantite	prix_unitaire_ht	montant_ttc
T05-250101-075845	2025-01-01	Magasin 5	Moussa I.	10857	11	4 500	56 658
T05-250101-075845 ①	2025-01-01	Magasin 5	Moussa I.	0.0 ②	2	12 000	28 320
T05-250101-075846	2025-01-01	Magasin 5	Salamatou S.	③	2 000	21 950	51 802
T05-250102-075902	2025-01-02	Magasin 5	Adama B.	3312	1	14 000	16 520

① un ticket occupe 2 lignes : compter les lignes pour compter les ventes gonfle le résultat de 55 % (le fichier compte 489 lignes, 316 tickets, et jusqu'à 4 lignes pour un même ticket).

② « 0.0 » n'est pas un manque : c'est la vente au comptoir, sans client identifié. Elle est volontaire dans le fichier et doit rester dans les totaux.

③ cellule vide = valeur absente : 18 lignes du fichier sont dans ce cas. On signale, on documente, on ne remplit pas au hasard.

- Grain : une ligne = un article vendu sur un ticket ; 1,52 ligne par ticket en moyenne.
- Clé : n_ticket identifie le ticket ; (n_ticket, produit) identifie la ligne. Sans clé, pas de fusion propre.
- Schéma : les 13 colonnes et leur type. Le CSV ne le dit pas : le déduire, puis le vérifier.

Figure 3 — Les quatre premières lignes réelles du fichier de l'atelier, annotées.

Une table se lit en six phrases et se contrôle en cinq nombres : lignes, valeurs distinctes de la clé, bornes des mesures, valeurs manquantes, cohérence entre colonnes. Le grain décide si un total est une somme ou un double-comptage ; la clé décide si l'on peut relier sans multiplier. Un fichier brut n'est pas un tableau : il a un séparateur, un encodage, parfois un chapeau et presque toujours des colonnes qui ne sont pas du bon type. Tout ce que vous ferez ensuite — SQL, pandas, Power BI — partira de ces six phrases ; les écrire est le premier acte d'analyse, et le seul obligatoire.

15. À retenir

À retenir. 1. « Une ligne = quoi ? » : la première question à poser à toute table, et à écrire. 2. **Lignes ≠ objets** : `COUNT(*)` = 489, tickets distincts = 316. Nommez toujours ce que vous comptez. 3. **Une clé est unique ou n'est pas une clé** ; une clé étrangère orpheline (486 lignes) est un défaut, pas un détail. 4. **Cellules fusionnées, sous-totaux, deux infos par cellule** : les trois poisons du fichier de données. 5. **Un contrôle de cohérence entre colonnes** trouve ce qu'un contrôle de total laisse passer.

16. Évaluation formative (auto-correction, 10 min)

1. Le fichier de l'atelier contient 489 lignes et 316 tickets. Combien de ventes ? (316 ; 489 = nombre d'articles facturés)
2. Que vaut `COUNT(DISTINCT n_ticket)` sur les lignes en double non retirées, et sur le fichier nettoyé ? (316 dans les deux cas — les doublons ne créent pas de nouveaux tickets : c'est précisément ce qui rend le contrôle utile)
3. Un champ `ville` vide sur 8 % des clients : pourquoi cela gêne-t-il une analyse géographique et pas un chiffre d'affaires total ? (le total additionne une mesure indépendante de la ville ; le classement par ville perd 8 % de son contenu, donc les parts sont fausses)
4. Donnez le grain de `remises_manuelles.xlsx` (18 200 lignes) et écrivez la phrase de contrôle d'unicité. (une ligne = un client pour un mois ; contrôle : lignes = distincts de (id_client, annee, mois))

5. **Question ouverte** : vous devez fusionner les ventes avec les objectifs mensuels. Énoncez en trois phrases ce que vous vérifiez avant la fusion et ce que vous vérifiez après. (attendu : les deux grains (article vs magasin-mois) ; les clés communes ; la période ; après : que le total des ventes n'a pas changé et que le nombre de lignes n'a pas été multiplié)

M01.C04 — Types de données et ce qu'ils autorisent : texte, nombre, date, booléen, montant

Outil de ce chapitre : Excel et LibreOffice Calc (conversion, détection, contrôle), plus l'éditeur de texte pour voir la vérité du fichier. **Durée indicative :** 5 h. **Niveau :** N1.

L'idée du chapitre. Un type n'est pas un détail technique : c'est la **liste des opérations permises** sur une colonne. Le même chiffre 56 658 écrit en texte refuse toute somme ; la même date écrite 12/01/2025 refuse tout calcul d'intervalle. La totalité des chiffres faux produits par des débutants, et une part surprenante de ceux produits par des seniors, vient d'un type mal déclaré ou mal deviné. Ce chapitre vous rend capable de **détecter** le problème en trente secondes, de le **corriger sans perdre de lignes**, et surtout de **prouver** que la correction n'a rien cassé.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Nommer** les cinq types que rencontre un analyste (texte, nombre entier, nombre décimal, date/heure, booléen) et dire, pour chacun, ce qu'il autorise et ce qu'il interdit ;
2. **Détecter** une colonne « nombre stocké en texte » avec trois méthodes indépendantes dans un tableur ;
3. **Convertir** une colonne en gardant le contrôle du nombre de lignes et du total avant/après ;
4. **Traiter les dates** écrites en texte, y compris le piège JJ/MM/AAAA contre MM/JJ/AAAA, et expliquer pourquoi un formatage n'est pas une conversion ;
5. **Expliquer** les dégâts chiffrés d'un mauvais typage sur un résultat réel : ici 486 lignes sur 489 et un écart de total de 0,74 % après nettoyage ;
6. **Décider** du type d'une colonne ambiguë (identifiant numérique, montant, taux en proportion) et **documenter** la décision.

2. Pourquoi cette notion est importante

Situation 1 — le total à zéro. Un chef de secteur envoie son relevé mensuel en CSV. Vous faites `=SOMME()`. Résultat : 0. Colonne pleine de montants, somme nulle. Cause : chaque montant est écrit avec un espace de milliers (56 658), donc Excel a stocké du **texte**. Le chef de secteur, lui, voit ses totaux justes dans son fichier à lui, parce qu'il a laissé les formats de son logiciel. Deux personnes, deux chiffres, une seule cause : le type.

Situation 2 — la date qui décale tout un trimestre. Un export contient 12/01/2025 pour le 12 janvier et 01/12/2025 pour le 1^{er} décembre. Un outil de conversion automatique, réglé sur le format américain, lit le premier comme le 1^{er} décembre. Vous obtenez un hiver en plein été. Personne ne voit l'erreur dans le tableau de bord ; elle se voit dans la décision du directeur des achats, en janvier.

Situation 3 — la TVA en points ou en proportion. Dans notre fichier, `remise` contient 0.03. Ailleurs, un autre export contient 3. Appliquer un taux de 3 au lieu de 0,03 ne multiplie pas l'erreur par 100 : le montant de la ligne, qui est un taux, devient un facteur. Le type est « nombre »

dans les deux cas ; c'est **l'unité** qui diffère. Leçon durable : un type juste avec une unité fausse produit un chiffre faux avec assurance.

Dans les faits. Sur l'échantillon de l'atelier : **486 lignes sur 489** ont un `montant_ttc` non numérique, **65 dates** sont au format `JJ/MM/AAAA` dans une colonne où le reste est en `AAAA-MM-JJ`, **7 quantités** dépassent 500 (maximum **14 000**) et **18 clients** sont vides. Aucun de ces défauts n'empêche le fichier de s'ouvrir. Chacun d'eux rend un tableau de bord faux.

3. Explication simple

Chaque cellule contient une **valeur** et un **type**. Le type répond à une seule question : qu'est-ce que je peux faire avec ?

Type	Ce qu'on peut faire	Ce qu'on ne peut pas faire	Exemple du fichier
Texte	chercher, trier (alphabétique), regrouper, compter	additionner, moyennner, calculer un écart	Magasin 5 – Kaya Marché
Nombre entier	les cinq opérations, compter des quantités	dire « le 5 ^e tiers » (un entier est discret)	quantite = 11
Nombre décimal	tout ce que fait l'entier, plus ratios et pourcentages	s'additionner sans unité (0,03 de quoi ?)	remise = 0,03
Date / heure	intervalles, extraire mois/année, tri chronologique	additionner deux dates (ça n'a pas de sens)	date = 2025-01-01
Booléen	filtrer, compter des drapeaux	moyenner sans convention (VRAI = 1 ?)	est_retour (dans le jeu complet)

Une image qui tient : un type est un **verrouillage de porte**. Le texte est une porte coulissante, la date une porte à gâche électrique : le même geste ne s'y applique pas. « Convertir » n'est donc pas « embellir » : c'est changer de porte.

La confusion la plus fréquente du débutant est de croire que formater change le type. Dans un tableur, un format est une **apparence** : afficher `56 658` avec un séparateur de milliers ou en `# ###0,00 "FCFA"` ne change pas la nature de la valeur. Une cellule peut afficher une date `12/01/2025` alors qu'elle contient le nombre `45670` (le numéro de série interne de la date), et une autre afficher la même chose en contenant le texte `"12/01/2025"`. Le format ment, la valeur ne ment pas : c'est elle qu'on contrôle.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Type de données — data type	Nature d'une valeur, qui détermine les opérations possibles.	Croire qu'un CSV « contient » des types : il ne contient que du texte ; le type est une décision de l'outil qui le lit.
Typage / inférence — type inference	Déduction automatique du type par le logiciel qui importe.	Le logiciel déduit sur les 100 premières lignes : une colonne peut changer de type à la ligne 101.
Chaîne de caractères — string	Suite de caractères, avec des espaces, des accents, des signes.	Un identifiant numérique (10857) n'est pas une quantité : le sommer ne veut rien dire.
Nombre à virgule / point — decimal separator	Signe séparant partie entière et décimale : <code>,</code> en français, <code>.</code> en anglais.	<code>27500,00</code> lu en contexte anglais devient <code>2750000</code> .

Français — English	Définition simple	Piège à éviter
Format nombre — number format	Masque d’affichage d’un nombre.	Confondre format et type : le format change l’œil, pas la valeur.
Numéro de série — serial date	— Une date stockée comme un nombre de jours depuis une origine (1900 ou 1904).	Un import qui convertit tout en série : 2025-01-01 devient 45658, et vous sommez des dates.
Valeur manquante — null / missing	— Absence de valeur, distincte de 0 et distinct de la chaîne vide.	<code>id_client = 0</code> (comptoir) est une valeur ; une cellule vide est une absence.
Booléen — boolean	Vrai ou faux (0/1, VRAI/FAUX, oui/non).	Trois représentations dans un même fichier : « V », « OUI », 1 → trois groupes au lieu d’un.
Conversion / coulage — cast	Changer le type d’une colonne en conservant la valeur.	Convertir en masse sans compter les pertes : <code>CBI("abc")</code> ne renvoie pas une erreur, il renvoie 0 ou une erreur selon le moteur.
Arrondi — rounding	Ajustement de la précision ; ici arrondi à l’entier en FCFA.	Arrondir avant de sommer, puis sommer : le total n’est pas le même que sommer puis arrondir.
Encodage — encoding	Correspondance octets ↔ caractères (UTF-8, Windows-1252).	Un fichier en Windows-1252 lu en UTF-8 casse les noms de produits, donc les regroupements.

5. Cours approfondi

5.1 Ce que le CSV ne dit pas — et pourquoi c’est le cœur du métier

Un fichier CSV est une liste de lignes de texte. Aucun type n’y est écrit. Le type apparaît quand **un outil décide** : Excel devine, Power Query infère, pandas infère, PostgreSQL exige une déclaration. Quatre conséquences :

1. **Un identifiant à zéro non significatif devient un nombre** : 0075845 → 75845, et la jointure future sur ce code échoue silencieusement. Réflexe : toute colonne qui sert de code (client, article, téléphone, IBAN) se charge en texte.
2. **Une colonne mixte est déclarée texte** : si une seule des 489 lignes contient N/C, la colonne entière de montants devient du texte pour beaucoup d’outils.
3. **Un nombre trop long perd sa précision** : les identifiants supérieurs à 15 chiffres (numéros de carte, codes-barres EAN-13 lus en nombre, identifiants techniques) sont arrondis par Excel. Non négociable : texte.
4. **Une date lue dans le mauvais ordre est convertie en une autre date valide** — c’est le seul cas où le résultat est **juste au regard de la syntaxe et faux au regard du monde**. Le point §5.3 lui est consacré.

Définition. Type de données — data type — la nature d’une valeur, qui détermine ce que l’on a le droit d’en faire. Un texte se compare et se découpe, il ne se moyenne pas ; un nombre se somme ; une date se décale. Le type n’est pas une étiquette esthétique : il ouvre ou ferme des opérations, et le logiciel refuse poliment au lieu de signaler qu’il n’a rien calculé.

5.2 Les cinq signes visibles d’un nombre stocké en texte

Sans formule, dans Excel : un nombre aligné **à droite**, un texte aligné **à gauche** (tant que vous n’avez pas changé l’alignement). Ensuite : le **triangle vert** en coin de cellule avec l’avertissement « nombre stocké sous forme de texte » ; la **barre d’état** qui n’affiche que « Nombre » et jamais « Somme » quand vous sélectionnez la colonne ; `=NB(plage)` qui renvoie 0 alors que `=NBVAL(plage)` renvoie 489 ; enfin **l’erreur #VALUE!** dès qu’une formule arithmétique touche la cellule.

Les trois premières méthodes sont visuelles, donc fragiles (un collègue a pu aligner à droite pour faire joli). Les deux dernières sont mathématiques : **NB** compte les valeurs numériques, **NBVAL** compte les cellules non vides. La différence **est** le nombre de défauts. C'est la formule de contrôle que vous écrirez dans chaque projet :

lignes	=NBVAL(L2:L490)	→ 489
valeurs nombre	=NB(L2:L490)	→ 3
défauts de type	=NBVAL(...) - NB(...)	→ 486

Dans LibreOffice, mêmes idées avec **COUNT** (nombres), **COUNTA** (non vides) et **ISNUMBER**.

Définition. Typage automatique — type inference — le mécanisme par lequel un logiciel devine le type de chaque colonne en lisant un échantillon de lignes. Il est rapide et il est faux dès qu'une colonne est irrégulière : quelques valeurs atypiques dans l'échantillon lui décident si la colonne deviendra nombre ou texte. Réflexe : lire l'écran de résultat de l'import, jamais cliquer sur « Charger » les yeux fermés.

5.3 Les dates : le format n'est pas le type, et l'ordre des morceaux n'est pas une convention

Trois écritures du même jour : 2025-01-12 (norme ISO 8601, année-mois-jour), 12/01/2025 (usage français, jour-mois-année), 01/12/2025 (usage américain, mois-jour-année). Un outil qui devine se trompe **uniquement sur les jours ≤ 12** — soit la moitié du mois, et, dans notre échantillon, **65 lignes sur 489**. Le pire : l'erreur est silencieuse, parce que la date obtenue est valide. Le total annuel reste juste (la somme ne dépend pas du jour) ; le CA mensuel, lui, est faux, et c'est précisément ce que le directeur regarde.

Règles de survie, dans l'ordre :

1. **Ne jamais laisser le logiciel deviner** : à l'import, déclarer explicitement le type de chaque colonne date (Excel : « Avancé » dans l'assistant texte ; Power Query : transformer la colonne puis Utiliser les en-têtes...).
2. **Ne jamais utiliser une conversion « automatique »** du type `format="mixed"` (le nom même dit « je devine ») sur un fichier de provenance humaine : sur notre échantillon, 12/01/2025 devient le 3 décembre et le mois change. Une conversion mixte n'est acceptable que sur des dates **déjà** ISO, pour gagner du temps.
3. **Forcer l'ordre** quand le fichier est hétérogène : séparer les lignes par motif (AAAA-MM-JJ d'un côté, JJ/MM/AAAA de l'autre), convertir chaque groupe avec son format, recoller. C'est long à écrire, c'est **juste**, et c'est l'exercice 4.2 de ce chapitre.
4. **Convertir une fois, en amont** : la correction de type appartient au nettoyage (M04), pas à chaque graphique.

Un test de robustesse à faire dans tous les cas : comparer, avant/après, le **nombre de lignes par mois**. Sur une conversion correcte, on retrouve les valeurs connues de l'atelier : janvier 2 051 424 · février 4 289 885 · mars 3 673 454 · avril 2 688 159 · mai 1 258 801 · juin 2 096 435 · juillet 2 623 127 · août 3 033 498 · septembre 3 476 773 · octobre 2 808 078 · novembre 2 781 034 · décembre 5 292 517 FCFA. Si vos mois ne correspondent pas, votre conversion a réarrangé le calendrier.

Définition. Séparateur décimal — decimal separator — le signe qui sépare la partie entière de la partie décimale. 27 500,00 et 27500.00 sont le même nombre dans deux conventions ; 27.500 peut être vingt-sept mille cinq cents ou vingt-sept virgule cinq selon le logiciel qui lit. C'est la source d'erreur la plus silencieuse du travail sur fichiers, parce qu'elle produit un nombre — le mauvais, mais un nombre.

5.4 Taux, proportions, pourcentages : la colonne `remise`

Dans les fichiers du socle, `remise` / `taux_remise` est une **proportion** : $0,03 = 3\%$. Un second mode de saisie existe dans la vraie vie : **les points de pourcentage** ($3 = 3\%$). Les deux sont des nombres ; le résultat d'un calcul diffère d'un facteur 100. Conséquence dans le jeu complet : **973 lignes** de remise aberrante sont des points confondus avec des proportions, et ces lignes passent tous les contrôles de type — seul un **contrôle de plage** les attrape :

```
règle métier : 0 ≤ remise ≤ 0,40
=NB.SI(K2:K490;">0,4") → lignes suspectes
```

Le même raisonnement s'applique à la TVA (`tva = 0,18`, jamais 18) et à toute « marge en % » (0,22 ou 22 ?). Réflexe professionnel, à appliquer dès le premier jour d'un stage : **la première ligne du dictionnaire de données écrit l'unité de chaque colonne numérique**. Une proportion, un taux, un pourcentage, un montant, une quantité : quatre mots, quatre conventions, zéro discussion ensuite.

5.5 Booléens et drapeaux : un type tout simple, trois catastrophes classiques

Le jeu complet contient `est_retour` (0 ou 1). Trois manières de mal faire :

1. **Le transformer en texte** par un import trop prudent : les groupes deviennent "0" et "1", qui se trient mais ne se somment plus — le total des retours tombe à 0.
2. **Le confondre avec une absence** : un fichier peut contenir 1, 0 et vide. Le vide n'est ni vrai ni faux : c'est « on ne sait pas », et le ranger avec les 0 fait disparaître des retours du total.
3. **Changer de vocabulaire entre deux sources** : VRAI/FAUX dans un export, OUI/NON dans un autre, T/F dans un troisième. À la fusion, trois modalités au lieu de deux. La normalisation (« oui/o/OUI/1/VRAI → 1 ») est une tâche de nettoyage, pas une option.

5.6 Convertir sans casser : le protocole en six temps

Toute conversion de type, dans n'importe quel tableur, se fait avec ce protocole. Il est imposé dans tous les projets de ce manuel, parce qu'il rend l'opération **auditable** :

1. **Compter avant** : nombre de lignes, total de la colonne concernée (si elle est déjà numérique) ou sa borne max/min.
2. **Dupliquer la colonne** (`montant_ttc` → `montant_ttc_num`) : on ne remplace jamais sur place.
3. **Retirer les parasites** en texte : espaces de milliers, FCFA, points de regroupement → `=SUP-PRESpace(SUBSTITUE(L2;" ";""))` (Excel) / `=TRIM(SUBSTITUTE(L2;" ";""))` (LibreOffice).
4. **Convertir** avec une fonction qui **signale** les échecs plutôt que de les masquer : `=CBI()` / `=VALUE()`, ou, en Power Query, un « Modifier le type » qui génère des `error` visibles. Les erreurs deviennent des lignes à traiter, pas des zéros.
5. **Contrôler** : la nouvelle colonne doit avoir le même nombre de valeurs que l'ancienne (`=NB(nouvelle) = NBVAL(ancienne)`) et un total égal à celui attendu.
6. **Écrire dans le journal** : date, colonne, méthode, lignes en échec, total avant/après.

Sur l'échantillon de l'atelier, la conversion de `montant_ttc` fait apparaître deux mondes : 3 lignes déjà numériques, 486 à nettoyer, 0 en échec après retrait des espaces (le fichier est propre de ce côté). Le total obtenu, **36 339 317 FCFA**, est celui **avec les 9 lignes en double** ; le total après suppression des doublons est **36 073 185 FCFA**, soit **0,74 %** de moins. Ces deux nombres sont vos points de contrôle pour tout le module.

Définition. Valeur manquante — null — l'absence de valeur. Elle n'est ni zéro, ni chaîne vide, ni « non renseigné » : zéro est une information, une case vide est une donnée, une absence est une question. Confondre les trois change un total, une moyenne et un nombre de clients.

Attention. Arrondir **avant** de sommer et sommer **puis** arrondir ne donnent pas le même total. Sur un extrait de quelques centaines de lignes, l'écart se compte en unités ; sur une facture payée à la ligne, il devient une réclamation. La convention s'écrit une fois dans le dictionnaire de données : on calcule en pleine précision, on arrondit à l'affichage.

5.7 Types, tailles de fichier et performance : ce que « bien typer » rapporte

Le type n'est pas qu'une question de justesse, c'est une question de coût, et cela devient critique dès le module 8 :

- un fichier CSV de 243 360 lignes × 20 colonnes occupe **28,5 Mo** ; la même table en `.parquet` avec des types justes tombe vers 4 à 6 Mo ;
- une date stockée en texte de 10 caractères coûte ~10 octets par ligne ; stockée en entier (nombre de jours), 4 octets ; en date binaire, 4 à 8 octets ;
- un booléen en texte ("`TRUE`" / "`FALSE`") coûte 5 octets ; en binaire, 1 bit effectif dans la plupart des formats colonnaires ;
- une colonne de type texte force, dans tous les moteurs, des comparaisons de chaînes au lieu d'entiers : sur un filtre ou une jointure, le facteur 2 à 5 est banal.

Ce n'est pas de l'optimisation pour ingénieurs : c'est la raison pour laquelle Power BI refuse parfois d'importer une colonne, et la raison pour laquelle un script Python « tourne » en 3 secondes ou en 4 minutes selon un détail invisible à l'œil.

Conseil professionnel. Dans tout outil d'import, l'écran qui affiche « 28 colonnes, type : texte » pour une colonne qui devrait être une date est un **signal d'alarme** que les débutants valident sans lire. Prenez l'habitude de parcourir cette ligne de types, colonne par colonne, à chaque import, et de vous demander : est-ce que cette colonne va être sommée, comparée, triée chronologiquement ? La réponse décide du type à imposer.

Attention. Convertir une colonne de **montants** en supprimant les espaces puis en forçant le nombre, **sans recalculer le total**, est la source numéro un des chiffres qui « collent » deux réunions de suite et s'effondrent à la troisième. Après toute conversion, un contrôle de total obligatoire — et l'écart doit être **expliqué**, pas arrondi à zéro. Ici, 266 132 FCFA d'écart s'expliquent par 9 lignes ; un écart inexpliqué de 12 FCFA est déjà une enquête.

À retenir. Un type se **déclare**, il ne se devine pas : la ligne d'en-tête d'un CSV ne porte aucune information de type, et le premier logiciel qui ouvre le fichier en invente une. D'où les trois réflexes du chapitre — compter les valeurs non numériques, compter les valeurs distinctes, écrire les types attendus dans le dictionnaire de données — qui transforment un doute en contrôle.

6. Exemple concret : la colonne qui a l'air juste

Voici, telle quelle, la fin du fichier de l'atelier. Deux lignes consécutives :

```
T05-251201-081477;2025-12-01;08:34;Magasin 5 – Kaya Marché;Adama Bationo;214;Robinet mitigeur
évier – réf 1;Plomberie;14000;140;0.00;1960000;2 312 800
T05-251201-081478;01/12/2025;09:02;Magasin 5 – Kaya Marché;Adama Bationo;;Ballon d'eau chaude 50 L
– réf 1;Plomberie;3;165000;0.05;495000;584 100
```

Quatre défauts, tous de type ou d'unité, invisibles à l'œil non exercé :

1. `2 312 800` : montant **texte** (espace de milliers) → il ne se somme pas, mais surtout il ne se compare pas : un filtre `> 2 000 000` l'ignore.
2. `01/12/2025` : date **texte**, et dans l'autre ordre que la ligne précédente → tri et regroupement par mois faux.
3. `quantite = 14000` pour des robinets : 14 000 mitigeurs sur un ticket de magasin de quartier, c'est une erreur d'unité (lignes de quantité aberrante : 7 dans l'échantillon, maximum **14 000**).
4. `client` vide sur la seconde ligne : une **absence**, à distinguer du `0` des ventes au comptoir (18 lignes vides dans l'échantillon).

Traitez les trois premières et le total passe de **36 339 317** (fichier tel quel) à **36 073 185** FCFA (après nettoyage), soit **0,74 %**. La quatrième ne change aucun total, mais change une réponse : « combien de clients distincts ? » — 372 identifiants distincts, dont il faut décider si l'absence compte comme un client.

7. Démonstration pas à pas : le protocole complet sur

montant_ttc

Faites-le dans un classeur neuf, onglet `exercice`, en important `ventes_magasin5_2025.csv` avec **toutes les colonnes déclarées en texte** (Power Query : colonne type « Touttexte » / « Text »). Déclarer tout en texte à l'import est une pratique professionnelle courante : on empêche le logiciel de deviner, puis on typ **soi-même**.

Étape 1 — État des lieux (3 formules).

<code>=NBVAL(M2:M490)</code>	→ 489	(la colonne est pleine)
<code>=NB(M2:M490)</code>	→ 3	(seuls 3 montants sont numériques)
<code>=NBVAL(M2:M490) - NB(M2:M490)</code>	→ 486	(le nombre de défauts de type)

Étape 2 — Colonne de travail. En N1, titre `montant_ttc_num`. En N2 :

Excel	: <code>=SIEPTEUR(CBI(SUPPRESPE(SUBSTITUE(M2;" ";"");""));"ERREUR")</code>
LibreOffice	: <code>=IFERROR(VALUE(TRIM(SUBSTITUTE(M2;" ";"");""));"ERREUR")</code>

`CBI/VALUE` échoue proprement ; `SIEPTEUR/IFERROR` transforme l'échec en marqueur lisible. Recopiez jusqu'à la ligne 490.

Étape 3 — Contrôle d'exhaustivité. `=NB.NON.VIDE(SI(N2:N490="ERREUR";N2:N490))` attend **0**. Si vous trouvez 1, un montant contient autre chose qu'un espace (un FCFA, une virgule décimale, un caractère invisible) : affichez-le avec `=CAR(...)`, ou ouvrez la cellule dans la barre de formule.

Étape 4 — Total et comparaison. `=SOMME(N2:N490)` → **36 339 317**. Écrivez à côté : « total fichier tel quel (9 lignes en double comprises) ».

Étape 5 — Retrait des doublons, puis nouveau total. Copiez la page dans un onglet `doublons_test`, Données → Supprimer les doublons (toutes colonnes) → **480 lignes**, total **36 073 185**. Notez : $36\,339\,317 - 36\,073\,185 = 266\,132$ FCFA, soit $266\,132 / 36\,073\,185 = 0,74\%$.

Étape 6 — Contrôle croisé, la partie utile. La colonne `montant_ht` est numérique, et `montant_ttc` doit valoir `ARRONDI(montant_ht × 1,18)`. Faites la colonne de contrôle :

```
=ARRONDI(P2*1,18)    puis    =SI(ARRONDI(P2*1,18)=N2;"ok","à voir")
```

Toutes les lignes doivent dire `ok` — sauf celles dont `quantite` est aberrante, car l'aberration a été injectée **sur la quantité seulement** : le montant, lui, reste cohérent. Vous touchez là une idée qui reviendra en M02 et en M04 : la cohérence interne d'une ligne ne prouve pas la véracité de ses composants. Un fichier peut être parfaitement cohérent et parfaitement faux.

Étape 7 — Les dates. Créez une colonne `date_iso` en traitant les deux écritures séparément, jamais en laissant l'outil deviner :

```
=SI(STXT(B2;5;1)="-"
; B2                                     /* déjà ISO : AAAA-MM-JJ, on garde */
; DROITE(B2;4)&"-"&STXT(B2;4;2)&"-"&GAUCHE(B2;2) /* JJ/MM/AAAA : on reconstruit en ISO */)
```

Puis contrôlez par compte, pas par impression : 65 lignes ont suivi la seconde branche, 424 la première (`=NB.SI()` sur le test du 5^e caractère). Regroupez ensuite par mois (`=GAUCHE(date_iso;7)` dans un tableau croisé) et comparez au tableau du §5.3 : décembre doit valoir **5 292 517 FCFA** sur le fichier nettoyé. C'est ce comparatif qui prouve que jour et mois n'ont pas été permutés — le total annuel, lui, aurait été juste et aurait caché l'erreur.

Boîte à outils. Excel — Détection : barre d'état (Nombre vs Somme), `=ESTNUM()`, `=NB()` vs `=NBVAL()`. Conversion : CBI, CNUM, DATEVALEUR, Power Query → « Transformer la colonne en nombre/date » ; à l'import, « Avancé » permet de forcer le type de chaque colonne. Erreurs de type visibles : Accueil → Mise en forme conditionnelle → « Règles de mise en surbrillance des cellules → Erreur ». **LibreOffice Calc** — VALUE, TRIM, SUBSTITUTE, ISNUMBER ; Données → « Importer une feuille de calcul » (choix explicite de l'encodage et des colonnes en texte) ; Format → Cellules → Date (ne **convertit** pas : n'utilisez jamais cette voie pour changer de type). **Les deux** — Après conversion, gardez une colonne `..._txt` à côté : le jour où l'on vous conteste un total, vous pouvez montrer le départ et l'arrivée, sans re-exécuter quoi que ce soit.

8. Erreurs fréquentes

Symptôme	Cause	Correction
Total à 0 sur une colonne pleine de montants	Montants stockés en texte (espace de milliers, FCFA)	SUPPRESSESPACE + SUBSTITUE puis CBI, avec contrôle <code>NB = NBVAL</code>
<code>=SOMME()</code> juste, mais <code>=MOYENNE()</code> fausse	Une ligne de titre ou de total mélangée aux nombres	Filtrer sur une colonne-clé non vide ; vérifier le grain
Des dates en nombre (45658)	Import qui a converti en série, format « Standard »	Convertir explicitement en date, puis vérifier par contrôle de mois
Un mois complet qui saute du rapport	Tri alphabétique sur texte 01/12/2025	Trier sur une colonne AAAA-MM-JJ (ou convertir, mieux)
Écart de 1 à 2 % sur des milliers de lignes	Point et virgule de milliers inversés selon la provenance	À l'import, renseigner les deux séparateurs dans l'ordre : le premier est le séparateur de milliers, le second la décimale
Un client sur deux disparaît du classement	<code>id_client</code> converti en nombre puis joint à un texte	Jointure sur même type ; recharger les identifiants en texte

Symptôme	Cause	Correction
Ventes au comptoir « supprimées par erreur »	0 traité comme une valeur manquante	Distinguer absence (NA) et zéro métier ; le comptoir doit rester dans le CA

9. Bonnes pratiques professionnelles

- [] À l'import : tout en texte, puis typage **décidé colonne par colonne**.
- [] Une colonne de contrôle `..._num` à côté de l'originale, jamais de remplacement à l'aveugle.
- [] Deux formules de contrôle à chaque conversion : NB vs NBVAL (exhaustivité) et total avant/après (fidélité).
- [] Unité de chaque colonne numérique écrite dans le dictionnaire (`remise` = proportion ; `montant_ttc` = FCFA TTC, TVA 18 % incluse).
- [] Les identifiants en texte, toujours ; un code n'est pas une quantité.
- [] Dates converties une fois, en amont, et vérifiées par **répartition mensuelle**, pas seulement par le total.
- [] Un écart de total, même de 12 FCFA, s'explique par une ligne du journal ou se corrige.
- [] Le dictionnaire note aussi **ce qui manque** : ici, cellule vide $\neq$ 0 (comptoir).

10. Exercice guidé — la grille de typage du fichier de l'atelier

On remplit ensemble, pour les 13 colonnes, la grille qui servira de base au dictionnaire de données (M01.C06 la mettra en forme). Colonnes : `n_ticket`, `date`, `heure`, `magasin`, `vendeur`, `client`, `produit`, `categorie`, `quantite`, `prix_unitaire_ht`, `remise`, `montant_ht`, `montant_ttc`.

Colonne	Type actuel	Type voulu	Pourquoi	Risque si on ne corrige pas
<code>n_ticket</code>	texte	texte	identifiant	perdu s'il devient nombre (les T05-... ne le permettent même pas)
<code>date</code>	mixte (424 ISO, 65 FR)	date	tri, filtres, mois	mois faux, ratios mensuels faux
<code>heure</code>	texte HH:MM	heure (ou texte)	(ou si l'on veut des tranches	tri alphabétique OK seulement si HH:MM à 2 chiffres partout
<code>magasin</code> , <code>vendeur</code> , <code>produit</code>	texte	texte	libellés	regroupements cassés par un accent mal encodé
<code>client</code>	nombre ou vide	texte	identifiant, avec 0 = comptoir	zéro confondu avec manque, jointure ratée
<code>categorie</code>	texte	texte	7 modalités	deux groupes pour la même catégorie si un espace traîne
<code>quantite</code>	nombre entier	nombre entier	somme, contrôle de plage	7 lignes aberrantes noyées dans la moyenne
<code>prix_unitaire_ht</code>	nombre entier	nombre	montant unitaire	—
<code>remise</code>	nombre décimal	proportion	$\times (1 - r)$	un facteur 100 si points/proportions se mélangent
<code>montant_ht</code>	nombre entier	nombre	total	—
<code>montant_ttc</code>	texte (486/489)	nombre	total officiel	total à 0 ou faux, comparaisons impossibles

Le constat à écrire en toutes lettres dans votre journal : **une seule colonne, montant_ttc, est catastrophiquement typée, et c'est celle sur laquelle le directeur regardera le résultat.** C'est presque toujours ainsi : le défaut n'est pas réparti uniformément, il est là où le calcul passe.

Ensuite, faites l'exercice inverse — celui qui forme vraiment : supposez que la ligne 490 (inexistante) contienne `montant_ttc = "N/C"`. Que deviennent `=SOMME()`, le tri Power BI, une agrégation SQL `SUM()` ? Réponse : une erreur, un groupe à part, `NULL` propagé. Les trois moteurs réagissent différemment ; le remède est le même : **rendre lisible avant de convertir**, c'est-à-dire extraire les lignes en échec dans un onglet `rejets` (et non les supprimer).

11. Exercices autonomes

Exercice 4.1 (★) — Les trois signes. Sans formule, dites pour `montant_ttc` du fichier de l'atelier (a) le type réel, (b) la preuve en une formule, (c) l'effet sur `=SOMME()`. (texte ; `=NB()` vs `=NBVAL()` = 3 vs 489 ; somme = 0 ou sous-estimée à 3 lignes seulement)

Exercice 4.2 (★★) — Le fichier hostile. Convertissez les 65 dates JJ/MM/AAAA en ISO en deux branches, puis donnez le CA du mois de décembre obtenu. (attendu : 5 292 517 FCFA sur l'échantillon nettoyé — toute autre valeur signale un jour/mois permuté)

Exercice 4.3 (★★) — L'unité tranchée. Une colonne `remise` contient les six valeurs 0 ; 0,03 ; 0,05 ; 0,08 ; 3 ; 5. Que concluez-vous, comment le prouvez-vous avec une colonne auxiliaire, et combien de lignes sont concernées dans le fichier de l'atelier ? (deux unités mélangées : le test `montant_ht × (1 - remise)` ne « tombe » juste que pour les proportions ; dans l'échantillon, 0 ligne en points — l'anomalie existe dans le jeu complet, 973 lignes)

Exercice 4.4 (★★) — Erreur volontaire — série E4 (diagnostic de défauts). On vous remet un classeur dont le total de `montant_ttc` est **36 339 317** alors que le tableau croisé de la même donnée affiche **36 073 185**. Listez trois causes possibles de type ou de format, et la formule qui tranche pour chacune. (causes : doublons conservés · colonne texte partiellement convertie · filtre oublié dans le TC ; tests : nombre de lignes `NBVAL` ; `NB` vs `NBVAL` ; `=SOMME.SI(n_ticket;"<>";...)` comparé à la valeur du TC)

Exercice 4.5 (★★★) — Le prix d'un mauvais typage. Un magasin a 480 lignes de détail. On commande du stock proportionnellement au CA TTC calculé sur le fichier **non nettoyé** (36 339 317 FCFA), à raison de 1,5 % du CA en réappro d'urgence. Calculez le surcoût en FCFA et en pourcentage de la commande réelle. (attendu : commande calculée 545 090 FCFA sur le fichier non nettoyé, 541 098 FCFA sur le fichier propre, soit 3 992 FCFA de trop — 0,74 % de la commande. L'exercice note surtout la démarche : l'impact métier dépend de la politique d'achat, et l'ordre de grandeur doit être dit, pas caché : ce n'est pas une excuse pour négliger le nettoyage)

12. Correction détaillée

Exercice 4.1. (a) texte ; (b) `=NB(M2:M490)` renvoie 3, `=NBVAL(M2:M490)` renvoie 489 : 486 cellules non numériques ; (c) `=SOMME(M2:M490)` ignore le texte et retournerait 3 valeurs, soit un total dérisoire et non 0 si les 3 valeurs numériques sont des cas propres — d'où l'intérêt du contrôle par les deux formules plutôt que du simple coup d'œil.

Exercice 4.2. Deux branches : `GAUCHE(B2;4)="2025"` → ISO conservée ; sinon reconstruire `DROITE(B2;4)&" "&STXT(B2;4;2)&" "&GAUCHE(B2;2)`. Contrôle : 65 lignes dans la seconde branche, 424 dans la première. Regroupement par mois → décembre 2025 = **5 292 517 FCFA**. Erreur type : obtenir 5 292 517 + quelques lignes de janvier mutées en décembre (si l'on a appliqué la

branche « jour/mois » aux lignes déjà ISO). Le contrôle du nombre de lignes par branche évite précisément cela.

Exercice 4.3. Conclusion : deux unités mélangées, 3 et 5 étant des points de pourcentage (0,03 et 0,05 existent aussi). Preuve : colonne vérif = `SI(remise>0,4 ; montant_ht × (1 - remise/100) ; montant_ht × (1 - remise))`, puis comparer à `montant_ht` ; les lignes en points ne collent qu'après division par 100. Dans l'échantillon de 480 lignes utiles, aucune occurrence (le générateur du jeu complet en injecte 973) : c'est l'occasion d'écrire la phrase exacte — le défaut est absent de ce fichier, et présent dans le jeu complet, donc la règle de contrôle doit exister malgré tout. Un apprenant qui « invente » 973 lignes dans l'échantillon pour faire plaisir au sujet a échoué à l'exercice.

Exercice 4.4. 1. Doublons non retirés → test : `=NBVAL()` = 489 contre 480 après dédoublement ; l'écart 266 132 FCFA correspond exactement à la somme des 9 lignes. 2. Colonne texte convertie sur une partie du fichier → test : `=NB(colonne)` = 486 au lieu de 489, ou somme différente selon la plage ; dans ce cas le total affiché aurait été 36 073 185 sur une plage et 36 339 317 sur l'autre. 3. Filtre oublié dans le tableau croisé → test : re-jeter le champ `date` hors filtre et comparer ; un filtre « janvier-novembre » produit un total inférieur d'environ 5,3 M FCFA, pas de 266 132 FCFA — donc ce n'est **pas** cette cause. C'est la bonne manière de raisonner : on élimine par l'ordre de grandeur attendu.

Exercice 4.5. Démarche juste : écart = 266 132 FCFA ; proportion de réappro d'urgence = 1,5 % du CA ; surcoût = $266\,132 \times 1,5\%$ = **3 992 FCFA** ; la commande passe de 541 098 à 545 090 FCFA. Barème : 2 pts pour l'écart exact (266 132 FCFA), 3 pts pour le calcul, 5 pts pour le commentaire d'ordre de grandeur (3 992 FCFA sur un écart de 266 132 FCFA) qui conclut que **l'erreur de type est ici financièrement négligeable mais pédagogiquement et contractuellement grave** : un chiffre juste est une obligation, pas une question de montant. Un devoir qui écrit « surcoût de 545 090 FCFA » confond le montant de la commande et le surcoût : noté 2/10, le résultat est recopié, pas calculé.

13. Mini-projet M01.P4 — « Le dictionnaire de types » (45 min)

Livrable : un onglet dictionnaire de 13 lignes × 6 colonnes dans le classeur du module, une ligne par colonne du fichier de l'atelier. Colonnes imposées : `champ` · `type_livré` · `type_voulu` · `unité` · `règle_de_contrôle` · `défaut_constaté`.

Exemple de ligne recevable :

```
montant_ttc | texte (486/489) | nombre entier | FCFA TTC, TVA 18 % incluse |
NB(nouvelle)=NBVAL(ancienne) et total € {36 339 317 ; 36 073 185} | espace de milliers
```

Critères : toute colonne a une unité écrite (une proportion n'est pas un pourcentage) ; deux colonnes au minimum ont une règle de contrôle qui **s'exécute** (une formule du classeur, pas une phrase) ; les valeurs de défaut sont comptées, pas estimées (65, 486, 18, 7) ; le dernier onglet s'appelle `rejets` et contient les lignes qui n'ont pas pu être converties, même s'il est vide — le fait qu'il soit vide est une information.

14. Résumé du chapitre

Le même nombre, deux destins

montant_ttc écrit en NOMBRE		montant_ttc écrit en TEXTE		le même défaut, mesuré ici					
56 658 28 320 aligné à droite SOMME = 84 978 FCFA moyenne, tri, graphique : OK		'56 658 '28 320 aligné à gauche SOMME = 0 tri : 16 520 avant 56 658		486 lignes sur 489 sont du texte déguisé en montant → 0,74 % d'écart de total 18 clients absents 65 dates au format français					
TEXTE		NOMBRE		DATE		BOOLEEN		MONTANT	
vendeur, categorie on trie, on regroupe		quantite, prix_unitaire sommes et moyennes		date (vrai objet-date) intervalles, mois		est_retour : 0 ou 1 filtrer les retours		montant_ttc en FCFA TVA 18 %, arrondi	

Symptômes : total à 0, tri absurde, triangles verts en coin de cellule.

Figure 4 — Le défaut n° 1 des fichiers livrés, vu sans calculer et mesuré sur le fichier réel.

Un type est la liste des opérations permises, pas une mise en forme. Un CSV n'a pas de types : c'est vous qui les décidez, et le logiciel qui les devine à votre place se trompe sur les bords — identifiants, dates ambiguës, colonnes mixtes. Les deux formules `NB` / `NBVAL` détectent tout ce qui est nombre déguisé ; le contrôle par mois détecte tout ce qui est date déguisée ; le contrôle `0 ≤ remise ≤ 0,40` détecte tout ce qui est unité déguisée. Convertir se fait en sept temps, dont deux obligatoires : compter avant, contrôler après. Un total qui change de 0,74 % après nettoyage n'est pas une approximation : c'est une preuve.

15. À retenir

À retenir. 1. **Formater ≠ convertir** : l'apparence ne change pas le type, donc ne change pas ce qui est calculable. 2. **NB vs NBVAL** : deux formules, trois secondes, la mesure exacte des nombres stockés en texte (ici 486/489). 3. **Un identifiant est du texte** ; un montant est un nombre **avec unité** ; un taux est une **proportion**. 4. **Une date ambiguë se traite en deux branches** et se vérifie par répartition mensuelle, jamais par le seul total annuel. 5. **Toute conversion est encadrée par un contrôle** : lignes avant/après, total avant/après, et une ligne dans le journal.

16. Évaluation formative (auto-correction, 10 min)

- Pourquoi un CSV « ne contient-il pas de types » et qu'est-ce qui en crée ? (ce sont des caractères ; le type est créé par l'importateur, par inférence ou par déclaration)
- Une colonne `date` contient 424 valeurs `2025-01-01` et 65 valeurs `12/01/2025`. Quelle est la règle de conversion, et le contrôle ? (deux branches, reconstruction en ISO ; contrôle : nb de lignes par branche = 424 et 65, puis CA par mois cohérent avec les valeurs attendues)
- `=SOMME()` sur `montant_ttc` renvoie une valeur minuscule. Expliquez en une phrase et donnez la formule de diagnostic. (le texte est ignoré par `SOMME` ; diagnostic `=NB()` vs `=NBVAL()`)
- Un champ `tva` vaut 18 dans un fichier, 0,18 dans un autre. Que fait-on à l'étape de nettoyage ? (normaliser en proportion, documenter l'unité, contrôler `montant_ttc = montant_ht × 1,18`)
- Question ouverte** : votre direction affirme que le fichier « s'ouvre bien dans Excel, donc il est correct ». Répondez en trois phrases, avec un exemple chiffré du module. (attendu : Excel affiche, il ne valide pas ; 486 montants sur 489 sont du texte et le total officiel change de

266 132 FCFA selon le nettoyage ; un fichier juste est un fichier contrôlé, pas un fichier affichable)

M01.C05 — Structurées, semi-structurées, non structurées ; qualitatives et quantitatives

Outil de ce chapitre : un éditeur de texte, Excel/LibreOffice, l'invite de commandes pour compiler. **Durée indicative :** 4 h. **Niveau :** N1.

L'idée du chapitre. Deux classements traversent toute l'analyse, et ils sont indépendants. Le premier regarde **l'enveloppe** : la donnée est-elle en table, en arborescence, en prose ? Le second regarde la **nature** de la valeur : peut-on calculer avec, ou seulement classer ? Le premier décide de l'outil et du temps de préparation ; le second décide de la formule licite. Un analyste qui confond les deux choisit un outil inadapté et écrit un calcul interdit — deux erreurs que le module 2 vous fera payer cher, donc que l'on règle ici.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Classer** un jeu de données en structuré / semi-structuré / non structuré, et dire pour chacun le coût réel de mise en forme ;
2. **Reconnaître** la nature d'une variable — quantitative (discrète, continue) ou qualitative (nominale, ordinale) — et **justifier** la frontière par le test des écarts ;
3. **Relier** la nature d'une variable aux statistiques autorisées (moyenne, médiane, mode, pourcentage, écart-type) ;
4. **Choisir** le bon outil pour la bonne enveloppe : tableur pour le structuré petit, requête pour le structuré gros, script pour le semi-structuré, humain (ou traitement du langage, en M20) pour le non structuré ;
5. **Décrire** les quatre propriétés d'un fichier plat (séparateur, encodage, décimale, chapeau) et les vérifier en ligne de commande ;
6. **Estimer** le volume et le temps de préparation d'un lot de fichiers hétérogènes, ce que demande tout client au premier rendez-vous.

2. Pourquoi cette notion est importante

Situation 1 — les 200 Mo de logs. Un client remet un dossier de 200 Mo de fichiers `.log` « pour voir les achats en ligne ». L'analyste commence par les ouvrir dans Excel : 5 minutes de chargement, un plantage, une demi-journée perdue. La bonne première question n'était pas « comment filtrer ? » mais « sous quelle forme cela existe-t-il, et quelle fraction est exploitable ? » — ici, 4 lignes sur 10 étaient du texte libre, et la réponse utile se trouvait dans 3 colonnes parsemées.

Situation 2 — la moyenne des catégories. Un mémoire professionnel affiche « catégorie moyenne : 4,3 ». Les catégories ont été codées 1 à 7 pour gagner de la place dans le fichier. Le code est un nombre, la catégorie ne l'est pas : la moyenne d'une variable nominale est un nombre **sans signification**, même si le logiciel la calcule. C'est l'exemple type du calcul licite en apparence, interdit en réalité.

Situation 3 — le PDF qui « contient les données ». Une mairie envoie 40 pages de délibérations au format PDF, « tout y est ». C'est exact, et inutilisable en l'état : le non structuré n'est pas une table, il le devient par un travail dont le coût (ici, plusieurs semaines de saisie ou un traitement de texte automatique) doit être **annoncé** avant d'être accepté.

Dans les faits. Dans le socle de données de ce manuel, la répartition est celle d'une PME réelle : **10 fichiers structurés** (les tables, dont `ventes_brutes.csv` de 243 360 lignes), et du semi-structuré ou du non structuré partout ailleurs : un relevé de stock tenu « à la main » (le même tableur contient des commentaires libres en colonne de garde), les notes d'e-mails des commerciaux, les fichiers du fournisseur au format ; encodé en **Windows-1252**. Le ratio « facile / difficile » est de 1 à 4 en volume de travail, pas en volume d'octets.

3. Explication simple

L'enveloppe. Imaginez trois façons de noter la même vente :

- **structurée** : une ligne dans un fichier, des colonnes partout — `T05-250101-075845;2025-01-01;11;4500;...`. Un ordinateur la lit sans réfléchir.
- **semi-structurée** : une fiche qui porte ses étiquettes avec elle — `{"ticket":"T05-250101-075845","date":"2025-01-01","lignes":[{"produit":"Plâtre 25 kg","qte":11,"pu":4500}]}`. La structure existe, mais elle est **dans** le contenu, parfois inégale (telle vente a un champ `remise`, telle autre non ; la caisse a été ajoutée en 2024).
- **non structurée** : un SMS — « le client veut 11 sacs pour samedi, il paiera au comptoir ». Riche, mais l'ordinateur n'y voit que des mots. Le transformer en table, c'est décider ce qu'est « 11 », « sacs », « samedi », et ce travail a un prix.

La nature. Pour une colonne, posez une seule question : **est-ce que calculer avec a un sens ?**

- Si oui → **quantitative** : `quantite` (11, 2, 2 000), `poids_kg`, `taux_remise`. Dedans, deux familles : discrète (on compte : 11 sacs, jamais 11,3) et continue (on mesure : 24,7 kg).
- Si non → **qualitative** : `categorie`, `mode_paiement`, `type_client`. Deux familles : nominale (aucun ordre : plomberie n'est pas « plus » que peinture) et ordinale (ordre connu, distance inconnue : `particulier < pro < revendeur`, mais de combien ?).

Le test de l'ordre et le test des écarts tranchent les cas limites :

Question	Nominal	Ordinal	Intervalle	Ratio
Un ordre a-t-il un sens ?	non	oui	oui	oui
Un écart 10→11 vaut-il comme 40→41 ?	non	non	oui	oui
Le zéro veut-il dire « absence » ?	non	non	non	oui

`poids_kg` : ratio (0 kg = rien). `température` en degrés : intervalle (0 °C ne dit pas « absence de chaleur »). `note sur 20` : intervalle en pratique, ordinales en théorie — d'où l'interdiction de dire « 16 est deux fois 8 ». Cette échelle (nominale / ordinale / intervalle / ratio) n'est pas un ornement académique : elle décide, en M02, quelles statistiques sont admissibles, et en M17, quels graphiques ont le droit d'exister.

Deux classements, pas un seul

STRUCTURÉE

les 10 fichiers du socle : CSV, XLSX

SEMI-STRUCTURÉE

JSON des logs de caisse, XML d'un export Sage, e-mails

NON STRUCTURÉE

SMS du commercial, photo d'un bon, PDF du fournisseur

QUANTITATIVE : on peut calculer

discrète : quantité (11, 2, 2 000 — jamais 1,5)
continue : poids_kg, taux_remise (0,032)

QUALITATIVE : on peut classer

nominale : catégorie (aucun ordre)
ordinaire : type_client (ordre connu)

LE TEST QUI CLASSE

« 2 catégories valent-elles
2 fois l'autre ? » sinon, ce
n'est pas une échelle ratio.

Ce que la colonne EST décide ce que vous avez le DROIT de calculer.

la moyenne d'une nominale est un non-sens

Figure 5 — Classer les colonnes du socle, c'est déjà savoir quels calculs seront licites au module 2.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Donnée structurée — structured data	Organisation régulière en lignes et colonnes, schéma connu.	Croire que « table » implique « propre » : le structuré peut être sale (c'est même le cas ici).
Donnée semi-structurée — semi-structured	Contient ses propres étiquettes, mais de façon irrégulière (JSON, XML, HTML, logs).	Trier ou sommer sans l'avoir aplati en colonnes.
Donnée non structurée — unstructured	Texte, image, son, vidéo : pas de colonnes.	Surestimer ce qu'un outil de traitement automatique peut en tirer au premier passage.
Aplatir — flatten	Passer d'une arborescence à des colonnes plates.	Perdre les niveaux intermédiaires sans les recopier dans une colonne.
Schéma — schema	Liste nommée des colonnes avec types et règles.	Le deviner au lieu de l'écrire : un schéma non écrit se démode dès la première livraison.
Variable quantitative — quantitative variable	Se mesure ou se compte ; les calculs ont un sens.	Confondre code numérique et quantité (les catégories codées 1 à 7).
Variable qualitative catégorielle — categorical	Classe en familles ; on compte les effectifs, pas les valeurs.	Calculer une moyenne ou un écart-type.
Discrète / continue — discrete / continuous	On compte des entiers ; ou on mesure avec une précision finie.	Traiter une variable discrète à faible cardinalité (1, 2, 3) comme continue pour la moyener.
Cardinalité — cardinality	Nombre de valeurs distinctes d'une colonne.	Oublier que cardinalité élevée + texte = coût de jointure (revient en M06/M13).
Échelle de mesure — measurement scale	Nominale, ordinale, intervalle, ratio.	Choisir un graphique ou une statistique inadapté à l'échelle.
Texte libre — free text	Prose non contrainte : commentaires, motifs, SMS.	Faire des « mots les plus fréquents » un indicateur : la fréquence d'un mot n'est pas une mesure d'un fait.
Séparateur / encodage / décimale	Les trois propriétés d'un fichier plat, plus le chapeau.	Les supposer identiques entre deux fichiers du même expéditeur.

5. Cours approfondi

5.1 Les trois enveloppes, et le vrai coût de chacune

Enveloppe	Exemples	Ce qu'on peut faire directement		Coût de mise en forme	Outil adapté (module)
Structurée (table)	ventes_brutes.csv, clients.csv	filtrer, joindre	agréger,	faible : types, doublons, bornes (M04)	tableur (M03-M06), SQL (M07), Power BI (M14)
Semi-structurée	logs de caisse en JSON, XML d'un export, HTML d'une page fournisseur	lire enregistrement la fois	un à	moyen-élevé : aplatir, régulariser, accepter les champs absents (M08)	Python/pandas, DuckDB
Non structurée	SMS, e-mails, PDF de compte rendu, photos de bons	chercher un mot		très élevé : extraction, validation, échantillonnage (M20)	script + humain, ou modèles de langue (M20, avec ses limites)

Trois remarques professionnelles :

1. **La difficulté n'est pas dans la taille, c'est dans l'irrégularité.** Cent mille lignes d'un CSV bien tenu demandent une journée ; trois mille e-mails « importants » en demandent trois.
2. **Le semi-structuré est le format par défaut du monde moderne :** les API, les journaux d'événements, les fichiers de configuration, les exports web. Le JSON est au XXI^e siècle ce que le CSV était au XX^e.
3. **Le non structuré est un choix, pas une fatalité.** On peut faire saisir un formulaire au lieu de lire des SMS. Une partie du travail d'un analyste est de proposer ce changement de format : c'est le livrable « recommandation d'usage » du module 21.

Définition. Donnée structurée — structured data — des lignes et des colonnes, un schéma connu d'avance : le format où l'on peut compter, joindre et agréger sans rien interpréter.

Définition. Donnée semi-structurée — semi-structured data — des étiquettes portées par le fichier lui-même (JSON, XML, journaux applicatifs), mais des enregistrements qui ne se ressemblent pas tous : une clé peut manquer d'un objet à l'autre, et c'est ce manque qui demande un traitement.

Définition. Donnée non structurée — unstructured data — texte, image, son, vidéo : aucune colonne. On ne la compte pas ; on extrait d'abord une information, et cette extraction est elle-même une source d'erreur à documenter.

5.2 Le JSON en cinq minutes, parce que vous allez en rencontrer

Un objet JSON se lit comme un dictionnaire ; un tableau [...] comme une liste. Voici un ticket de caisse du commerce réel, en JSON :

```
{
  "ticket": "T05-250101-075845",
  "date": "2025-01-01",
  "magasin": 5,
  "vendeur": "Moussa Ilboudo",
  "lignes": [
    { "produit": "Plâtre de construction 25 kg", "qte": 11, "pu": 4500, "remise": 0.03 },
    { "produit": "Gravier 15/25 (m3)", "qte": 2, "pu": 12000 }
  ],
  "reglement": { "mode": "especes", "montant": 56658 }
}
```

Quatre enseignements :

- la structure **se répète** (le mot `lignes` porte une liste d'objets de même forme) ;
- les **champs optionnels** existent : la deuxième ligne n'a pas de `remise` — dans une table plate, cela devient une colonne à vide, et la question « vide = 0 ? » resurgit (M01.C04) ;
- l'**imbrication** oblige à aplatir : une table de lignes suppose de recopier `ticket`, `date`, `magasin` sur chaque article, donc de **choisir le grain** (ici l'article, pas le ticket) ;
- les **nombres sont typés** par la syntaxe (56658 sans guillemets = nombre, "56658" = texte) : c'est le seul format courant où le type est explicite dans le fichier. C'est pour cela qu'il est préféré au CSV pour les échanges entre logiciels.

Le mouvement à maîtriser, un jour ou l'autre : **JSON → table** (aplatir, M08) et **table → JSON** (exporter vers une application, M19). Les deux sont mécaniques ; ce qui ne l'est pas, c'est la décision du grain et du sort des champs absents.

5.3 Les quatre propriétés d'un fichier plat, et comment les vérifier sans outil

Un même fichier CSV peut être lu correctement par un logiciel et faux par un autre, à cause de quatre conventions non écrites. Voici comment les **vérifier**, pas les supposer, depuis un terminal (Git Bash sous Windows, Terminal sous macOS/Linux) :

```
head -3 2025_ventes_M5_brut.csv          # le chapeau : y a-t-il des lignes de titre avant l'en-
tête ?
head -1 2025_ventes_M5_brut.csv | tr ';' '\n' | wc -l  # le séparateur : combien de champs ?
(attendu : 13)
file -i 2025_ventes_M5_brut.csv          # l'encodage déclaré (charset=utf-8 ou iso-8859-1)
grep -c ',' 2025_ventes_M5_brut.csv      # des virgules dans un fichier « ; » = chiffres à
virgule ou texte libre
```

Résultats réels du socle : `ventes_magasin5_2025.csv` = 13 champs, UTF-8, séparateur `;`, aucune virgule décimale. `tarif_fournisseur_peinture.csv` = 6 champs, **Windows-1252**, séparateur `;`, **virgule décimale**, **3 lignes de chapeau**, 38 lignes utiles. Le second fichier est celui qui casse les imports, et il est parfaitement normal : c'est un export d'un vieux logiciel de facturation dont la langue est le français.

Règle de propreté qui en découle, à appliquer à chaque réception, et notée dans le journal : **nom du fichier, taille, encodage, séparateur, nombre de champs, nombre de lignes, nombre de lignes utiles**. Sept nombres. Un client à qui l'on écrit cela au lieu de « j'ai bien reçu le fichier » gagne en confiance ce que l'analyste gagne en temps.

Définition. Encodage de caractères — character encoding — la table qui associe des octets à des signes. **UTF-8** — UTF-8 — la convention universelle actuelle : tout caractère du monde y tient, un accent occupe 2 octets. **Windows-1252** (ou latin-1) — cp1252 — l'ancienne convention des logiciels francophones : les accents occupent 1 octet. Lire du Windows-1252 comme de l'UTF-8 produit `Ã©` ; l'inverse produit `ï¿½` ou des points d'interrogation. Le fichier n'est pas casé : la lecture l'est.

5.4 Qualitatif et quantitatif : les cas qui font discuter, tranchés

Colonne du socle	Classification	Tranchage
<code>quantite</code>	quantitative discrète	on compte des articles ; 1,5 sac n'existe pas dans ce commerce
<code>poids_kg</code>	quantitative continue	mesure ; un écart de 0,4 kg vaut un écart de 0,4 kg ailleurs sur l'échelle

Colonne du socle	Classification	Tranchage
prix_unitaire_ht	quantitative, ratio	0 FCFA = absence de prix (et non un prix « petit ») → 2 lignes à 0 dans produits.csv, donc un défaut, pas une valeur
remise	quantitative, ratio	0 = aucune remise, ce qui a un sens
categorie	qualitative nominale	7 familles, aucun ordre → moyenne interdite, effectifs et parts autorisés
type_client	qualitative ordinale	particulier < pro < revendeur : ordre connu, distance inconnue → médiane parfois admissible, moyenne non
mode_paiement	qualitative nominale	espèces / mobile money / chèque / virement / crédit : médian n'a aucun sens
conditionnement	qualitative nominale	pot, sac, m³, barre de 4 m → sert d' unité , donc à la conversion (M02)
heure	quantitative circulaire	23 h 59 et 00 h 01 sont voisins mais la moyenne fait 12 h : à traiter par extraits (tranches) ou par fonction cosinus (M20, cas limite)
ville	qualitative nominale	géographie = ordinale si l'on hiérarchise par taille ; ici, nom de ville + région (référentiel INSD) = hiérarchie à 2 niveaux, donc variable hiérarchique (M11)

Le test unique à retenir pour les cas limites : « **puis-je soustraire deux valeurs et obtenir un nombre qui veut dire quelque chose ?** » 13 h – 11 h = 2 h ✓ (quantitative). Plomberie – Peinture ✗ (qualitative). note 16 – note 8 = 8 points : oui arithmétiquement, mais les points n'ont pas la même valeur partout → on reste prudent, on parle de médiane et de quartiles, jamais de « deux fois mieux ».

Définition. Variable quantitative — quantitative variable — une valeur qui se mesure ou se compte, et pour laquelle les calculs ont un sens : quantité, prix, montant, durée.

Définition. Variable qualitative — categorical variable — une appartenance : famille, catégorie, statut, marque. On la compte (des effectifs par modalité), on ne la calcule pas.

Attention. Un **code** n'est pas une **quantité**. Code postal, numéro d'article, identifiant client, modalité codée 1/2/3 : autant de textes qui ont l'air de nombres. Leur moyenne ne veut rien dire, et la calculer est le premier réflexe de l'analyste pressé — le deuxième étant de la montrer en réunion.

5.5 Quelle statistique pour quelle nature : la table qu'il faut avoir en tête

Nature	Mode	Médiane	Moyenne	Écart-type	% et effectifs	Remarques
Nominale	✓	✗	✗	✗	✓	le mode est la seule « tendance centrale » lisible
Ordinale	✓	✓ (avec prudence)	✗	✗	✓	médiane = catégorie du milieu, pas un nombre à commenter seul
Quantitative discrète	✓	✓	✓	✓	✓	attention aux cardinalités faibles (1, 2, 3) : la moyenne devient trompeuse
Quantitative continue	✓	✓	✓	✓	✓	moyenne + écart-type ou médiane + quartiles, selon l'asymétrie (M02)

Cette table est celle du module 2, introduite ici parce qu'elle découle directement de la nature des colonnes. Elle vous évite le pire scénario d'un rapport : un chiffre exact, calculé comme il faut, sur une variable à laquelle il ne s'applique pas. Le jury d'une soutenance (M22) ne vérifie pas les logiciels : il vérifie cette table. **Version alternative pour un petit écran** — ce tableau

de sept colonnes se réduit à deux phrases : sur une variable nominale, seul le mode se calcule ; sur une quantitative, tout se calcule, et le choix entre moyenne + écart-type ou médiane + quartiles se tranche à l'asymétrie (M02.C02 et M02.C05). Les cases du milieu ne font que détailler ces deux phrases.

Définition. Cardinalité — cardinality — le nombre de valeurs distinctes d'une colonne. Elle se mesure, elle ne se devine pas, et elle décide : une cardinalité faible se raconte en graphique, une cardinalité élevée sur du texte libre ne se raconte pas, elle se liste ou se classe.

Définition. Échelle de mesure — measurement scale — le niveau d'information porté par une colonne : nominale (des noms, sans ordre), ordinale (un ordre, mais des distances qui n'ont pas de sens constant), à rapport (ordre, distances, et un zéro qui veut dire « rien »). L'échelle autorise ou interdit les statistiques — c'est le seul cours de statistiques que l'on applique chaque semaine sans y penser.

5.6 Choisir l'outil selon l'enveloppe et le volume : la première grille de décision

Situation	Volume	Outil recommandé	Pourquoi, en une phrase
Un extrait, à montrer à une seule personne	< 50 000 lignes	tableur (Excel/LibreOffice)	tout le monde peut relire vos cellules ; zéro installation
Un tableau de bord à publier, à consulter souvent	< 10 M lignes	Power BI	le modèle et les actualisations sont faits pour ça ; le lecteur n'a rien à installer (service)
Une question ponctuelle sur la base de l'entreprise	n'importe lequel	SQL	la base fait le calcul, vous ne déplacez rien
Un fichier sale, irrégulier, semi-structuré	n'importe lequel	Python	seul outil où l'on écrit la règle de nettoyage une fois, et où elle se rejoue
Explorer un gros fichier sans serveur	jusqu'à la RAM	DuckDB	requêtes sur fichiers plats, sans installation lourde
Comprendre du texte libre	—	script + lecture humaine , puis validation sur échantillon	une extraction non validée n'est pas une mesure
Un résultat officiel à signer	—	le plus lent des outils acceptables	ce n'est pas une blague : la lenteur, ici, veut dire traçable et rejouable

Le fil du raisonnement, dans tous les cas : qui relit ? Si c'est un collègue non technicien → tableur. Si c'est un outil de service → requête. Si c'est vous dans six mois → script. Nous reprendrons cette grille en M08.C01 et M13.C01, mais vous l'avez déjà : la bonne question n'est jamais « quel outil est puissant ? », c'est « quel outil laisse-t-il une preuve ? ».

Conseil professionnel. Ne commencez jamais un projet en disant « je vais mettre ça dans Power BI ». Annoncez : « je prépare d'abord un fichier plat de N lignes, sur lequel on pourra vérifier les totaux, puis le rapport ». Vous gardez la main sur la qualité de la donnée, vous rassurez le commanditaire, et vous évitez de reconstruire trois fois le modèle parce que le format de départ changeait.

Attention. Faire des « mots les plus fréquents » d'une colonne de texte libre un indicateur de pilotage est une fausse bonne idée classique : la fréquence mesure le vocabulaire d'un saisie, pas un phénomène. Le texte libre devient une information quand on y rattache une catégorie stable, définie et contrôlée par le métier.

À retenir. La nature d'une variable se lit dans le **grain** et dans l'**usage**, jamais dans le nom de la colonne : `code_article` est qualitatif même s'il est numérique, `remise` est quantitative même si elle est stockée en texte. Une fois la nature écrite, la statistique autorisée se déduit — et l'inverse est faux : choisir d'abord le graphique, puis chercher quelle statistique le justifie.

Boîte à outils. Vérifier l'enveloppe d'un fichier, trois commandes suffisent : `file -bi mon_fichier.csv` (type MIME et encodage déclarés) · `head -3 mon_fichier.csv | cat -A` (séparateurs, fins de ligne ^M\$, espaces insécables) · `awk -F';' '{print NF}' mon_fichier.csv | sort | uniq -c` (toutes les largeurs de lignes d'un CSV, pour repérer les lignes cassées). Sous Windows, PowerShell : `Get-Content mon_fichier.csv -TotalCount 3`. Pour un JSON : `python3 -m json.tool mon_fichier.json | head -20` affiche la structure réelle, pas celle du premier objet.

6. Exemple concret : vingt exemples réels, classés en six minutes

Liste typique que l'on trouve dans une quincaillerie-grossiste, et son classement complet. Faites-le à la main avant de lire la réponse : c'est le meilleur entraînement du chapitre.

#	Exemple	Enveloppe	Nature (si tabulaire)	Usage analytique
1	<code>ventes_brutes.csv</code> , 243 360 lignes	structurée	mesures + identifiants	tout : CA, volumes, saisonnalité
2	<code>clients.csv</code> , 23 912 lignes	structurée	attributs qualitatifs + une mesure (plafond de crédit)	segmentation, géographie
3	<code>produits.csv</code> , 154 lignes	structurée	nominales (catégorie), quantitatives (prix, poids)	mix produit, marge brute
4	<code>magasins.csv</code> , 6 lignes	structurée	qualitatives nominales	comparaisons, hiérarchie
5	<code>objectifs_de_ca.csv</code> , 218 lignes	structurée	quantitatives (cibles)	écarts, taux d'atteinte
6	<code>stocks_quotidiens.csv</code> , 60 000 lignes	structurée	quantitatives discrètes + continues	ruptures, rotation
7	<code>couts_achat.csv</code> , 1 694 lignes	structurée	quantitatives, avec dates de validité	marge, SCD type 2 (M13)
8	<code>remises_manuelles.xlsx</code> , 18 200 lignes	structurée avec textes dans motif	quantitative qualitative	+ contrôle, abus de remise
9	<code>dim_date.csv</code> , 1 339 lignes	structurée	dates + drapeaux booléens	toute agrégation temporelle
10	<code>tarif_fournisseur_peinture.csv</code> , 42 lignes	structurée, chapeau encodage 1252	quantitatives	+ contrôle des prix d'achat
11	Journal de caisse, JSON ligne par ligne	semi-structurée	—	audit de fin de journée
12	Logs de l'application de vente en ligne	semi-structurée	quantitatives extraction	après trafic, tunnel d'achat
13	Export XML du logiciel de compta	semi-structurée	quantitatives	rapprochement comptable

#	Exemple	Enveloppe	Nature (si tabulaire)	Usage analytique
14	Feuille d'Excel du magasinier, avec notes en marge	structurée polluée	mélange	à re-mouler (M04)
15	SMS d'un client (« livrer samedi »)	non structurée	—	demande à saisir pour être comptée
16	E-mails de réclamation	non structurée	après extraction : ordinaire (gravité)	typologie des litiges
17	Photos de bons de livraison	non structurée (image)	—	OCR, puis validation humaine
18	Compte rendu PDF du conseil d'administration	non structurée	—	citation, jamais mesure
19	Enregistrements d'appels au standard	non structurée (audio)	—	transcription, prudence (donnée personnelle)
20	Sondage client sur papier, notes de 1 à 5	saisie → structurée	ordinal (souvent traité comme intervalle : à dire)	satisfaction : médiane et distribution, pas moyenne seule

La colonne qui compte vraiment est la dernière. Elle dit ce que vous **avez le droit** d'affirmer : 1 480 clients, oui ; « satisfaction moyenne 3,9 », seulement si vous assumez l'hypothèse d'intervalles égaux — et vous l'écrivez.

7. Démonstration pas à pas : trier un lot de fichiers livré par un client

Cadre. Un client remet un dossier ZIP de 9 fichiers et dit : « tout est là, pour l'analyse des ventes 2025 ». Vous devez répondre en une heure : ce que vous pouvez faire tout de suite, ce qui demande un travail, ce qui est impossible faute de format.

Étape 1 — Inventaire sans ouverture. Liste : nom, taille, extension, date de modification. Douze secondes, dix-huit colonnes de décision prises. Notez dans le journal.

Étape 2 — Détection de l'enveloppe, en une commande par fichier.

```
for f in *.csv; do printf "%s : %s champs, %s lignes\n" "$f" \
"$$(head -1 "$f" | awk -F';' '{print NF}')" "$$(wc -l < "$f")"; done
```

Un fichier à 1 champ sur la ligne 1 = ce n'est pas un CSV (ou le séparateur est une virgule/tabulation) ; un fichier à 300 000 lignes pour 4 Mo = un export de détails ; un fichier à 42 lignes = une grille. Vous savez déjà où sera le temps.

Étape 3 — Encodage et chapeau, pour les trois suspects.

```
file -i tarif_fournisseur_peinture.csv # → charset=iso-8859-1 : encodage 1252
head -3 tarif_fournisseur_peinture.csv # → 3 lignes de titre avant l'en-tête
iconv -f cp1252 -t utf-8 tarif_fournisseur_peinture.csv > /tmp/tarif_utf8.csv
```

Le fichier converti **dans un dossier de test** s'ouvre correctement : vous savez maintenant que le problème était la lecture, pas le fichier. Vous ne touchez pas l'original — règle du chapitre 1.

Étape 4 — Typage à la volée, pour trois colonnes. Comptez les valeurs non numériques d'une colonne de montant :

```
cut -d';' -f13 2025_ventes_M5_brut.csv | grep -cvE '^[^*]?[0-9]+([.][0-9]+)? *$'
```

Résultat attendu sur l'échantillon de l'atelier : **486** lignes non numériques sur 489 (hors en-tête, le calcul exclut la ligne d'en-tête), ce qui recoupe exactement ce qu'a mesuré le chapitre 4. Vous avez une double preuve, par deux voies indépendantes : le tableur et la ligne de commandes. C'est cela, la robustesse d'un chiffre.

Étape 5 — Classer le contenu, pas les fichiers. Ouvrez les trois colonnes suspectes : une colonne motif de 18 200 lignes de texte libre ? C'est du non structuré **dans** un fichier structuré. Traitez-le comme tel : comptez les 20 mots les plus fréquents, mais n'en faites **pas** un indicateur.

Étape 6 — Réponse au client, en cinq lignes (modèle à copier).

```
1. Exploitable immédiatement : 5 fichiers (ventes 2025 x2, clients, produits, objectifs) = 246 090
lignes.
2. Demande un travail de préparation : 3 fichiers (tarifs fournisseurs : encodage + chapeau +
virgule décimale).
3. Non exploitable en l'état : relevé de stock avec notes libres en marge → 2 heures de remoulage,
ou saisie d'un vrai bordereau.
4. Manquant pour votre question « la marge baisse-t-elle ? » : les coûts de structure (loyer,
salaires, transport).
5. Question à trancher avant calcul : les retours doivent-ils être inclus dans le CA ?
```

Ce message est un livrable professionnel : il annonce un délai, un coût, une impossibilité et une question. C'est exactement ce que l'on attend d'un analyste en première semaine de mission, et cela n'a nécessité aucun graphique.

8. Erreurs fréquentes

Symptôme	Cause	Correction
« 4 catégories valent 2 fois l'autre »	Variable nominale traitée comme un ratio	Test du §5.4 : une soustraction a-t-elle un sens ? Sinon, parler d'effectifs et de parts
La moyenne des catégories codées 1..7	Code numérique pris pour une mesure	Recharger le code comme texte, ou documenter explicitement qu'il ne sert qu'au tri
Le tableau de bord ne montre pas les commentaires clients	Non structuré ignoré parce qu'il ne rentre pas dans une table	Décider une extraction validée sur échantillon, ou assumer l'exclusion dans la note de périmètre
JSON importé en une colonne unique	Aplatissement non fait	Extraire les clés au lieu de stocker la chaîne (M08)
Un CSV « corrompu » dont les accents sont cassés	Encodage mal déclaré à l'import	<code>file -i</code> puis convertir dans un dossier de test ; ne jamais réenregistrer le brut
Deux fichiers du même client lus avec le même séparateur	Supposition d'homogénéité	Vérifier le nombre de champs ligne 1 de chaque fichier, pas du premier
Une moyenne de notes 1-5 présentée comme un score d'entreprise	Échelle ordinale traitée en intervalle	Donner distribution + médiane, et écrire l'hypothèse si l'on donne la moyenne

9. Bonnes pratiques professionnelles

- [] À la réception d'un lot : inventaire (nom, taille, extension, date), puis détection d'enveloppe par ligne 1.
- [] Quatre propriétés vérifiées par fichier plat : séparateur, encodage, décimale, chapeau.
- [] Une conversion d'encodage se fait **copie vers un dossier de test**, jamais sur l'original.
- [] Toute variable codée en nombre est classée à la main avant tout calcul (le code n'est pas la mesure).

- [] Un champ texte libre dans un fichier structuré est un **avertissement**, pas un détail : il signale une décision de saisie à changer.
- [] L'outil se choisit selon qui relit, pas selon la mode.
- [] Le semi-structuré se transforme en table **datée** : la version du parseur fait partie du résultat.
- [] Un non structuré jamais quantifié sans échantillon validé à la main, avec taux de réussite écrit.

10. Exercice guidé — classer les 13 colonnes de l'atelier, avec justification

Prenons-en quatre, les plus discutables, et menons le raisonnement à voix haute.

1. **client** (identifiant, 372 valeurs distinctes, 18 vides). Enveloppe : colonne d'une table structurée. Nature : **qualitative nominale**, même si les valeurs sont numériques — c'est un identifiant, donc une étiquette. Conséquence licite : compter des clients distincts, regrouper ; interdiction : moyenne, médiane, écart-type. Le vide oblige à une décision (comptoir ou donnée manquante) : ce n'est pas un choix technique, c'est une règle métier, donc à écrire.
2. **catégorie** (7 modalités). Qualitative nominale. Ce qui est permis : effectifs, parts, top catégories (ici Matériaux 146 lignes, Quincaillerie 115, Consommables 58, Plomberie 57, Electricité 54, Peinture 34, Bois & panneaux 25). Ce qui ne l'est pas : un « classement des catégories par valeur moyenne de catégorie ».
3. **quantité** (minimum négatif, maximum 14 000, 7 lignes > 500). Quantitative discrète. Moyenne et écart-type autorisés, **mais** l'écart-type explose à cause des aberrations : avant tout résumé statistique, on documente (M02.C01) et l'on décide (M04).
4. **heure** (HH:MM, 356 valeurs distinctes). Quantitative **circulaire** : 23 h 58 et 00 h 02 sont à 4 minutes, et à 23 h 56 « d'écart arithmétique ». Le bon usage n'est donc pas la moyenne des heures mais la répartition en tranches (matin/après-midi) ou la médiane circulaire. Une moyenne d'heures dans un rapport est une erreur de catégorie, pas une approximation.

Le geste à retenir : pour chaque colonne, écrivez une ligne — enveloppe · nature · ce qui est permis · ce qui est interdit — et placez-la dans le dictionnaire de données. En trois minutes, vous venez de prévenir trois erreurs de calcul et un débat de réunion.

11. Exercices autonomes

Exercice 5.1 (★) — Vingt exemples. Reclasser les 20 exemples du §6 sans regarder le tableau, en deux colonnes (enveloppe / nature). Contrôlez : vous devez trouver 10 structurés, 3 semi-structurés, 7 non structurés ou mixtes. (attendu : les écarts se discutent pour n° 11-14 ; justifiez par la présence ou l'absence de schéma régulier)

Exercice 5.2 (★) — Le test des écarts. Pour `type_client` (particulier / pro / revendeur), répondez aux trois questions de l'échelle (§3) et concluez sur la moyenne. (ordre oui ; écart 10→11 comme 40→41 ? non ; zéro = absence ? non → ordinale, moyenne interdite, médiane et parts autorisées)

Exercice 5.3 (★★) — JSON à plat. À partir du ticket de l'exemple §5.2, écrivez les deux lignes de la table « articles » (grain : l'article), en indiquant quelles colonnes du ticket sont recopiées et ce que devient l'absence de `remise`. (2 lignes × colonnes ticket + article ; `remise` vide → décision 0 ou NA, à écrire dans le dictionnaire)

Exercice 5.4 (★★) — Le lot de fichiers. Un client remet 6 fichiers : `ventes2025.csv` (2 Mo, 1 champ sur ligne 1, tabulation), `clients.csv` (UTF-8, 11 champs), `tarif.xls` (binaire), `notes_livraison.txt` (40 000 lignes de prose), `stats.json` (120 Ko, 3 clés imbriquées), `README.docx`. Rédigez les cinq lignes de réponse du §7, étape 6. (attendu : déduire de la ligne 1 que le séparateur du premier est une tabulation ; classer `notes_livraison` en non structuré → extraction validée ; refuser de chiffrer quoique ce soit depuis le `.docx` ; annoncer un délai pour `tarif.xls` ; demander la liste des champs de `stats.json`)

Exercice 5.5 (★★) — Choisir l'outil, le justifier. Quatre demandes : (i) comparer 6 magasins sur 3 ans, une fois par mois ; (ii) retrouver les 20 tickets les plus gros d'un fichier de 240 000 lignes ; (iii) extraire une table de 30 000 lignes d'un JSON ; (iv) vérifier qu'un fichier mensuel ne contredit pas le précédent. Attribuez un outil à chaque demande et **une raison** qui ne soit pas le nom de l'outil. (attendu : Power BI ou tableur pour (i) selon qui relit ; SQL ou DuckDB pour (ii) ; Python pour (iii) ; script ou DuckDB pour (iv), avec la raison « jouable sans intervention humaine »)

12. Correction détaillée

Exercice 5.1. Structurés : 1-10 (le 8 est structuré mais pollué par du texte libre : à signaler comme « mixte »). Semi-structurés : 11, 12, 13 (le 14 est structuré-pollué, pas semi-structuré — la différence est que son schéma existe, il est juste violé par des notes). Non structurés : 15-19, et le 20 est **à cheval** : papier = non structuré, une fois saisi = structuré, et la saisie introduit ses propres erreurs (à écrire).

Exercice 5.2. Ordre : oui, la hiérarchie de puissance d'achat est une convention métier. Écart : non, le passage particulier→pro n'a pas la même signification que pro→revendeur. Zéro : non, il n'y a pas de « zéro type de client ». Conclusion : ordinale → la moyenne de ces catégories (si codées 1-2-3) est un non-sens mesurable ; ce qui est admissible : parts, effectifs, médiane des codes si et seulement si l'on précise que les codes sont arbitraires.

Exercice 5.3. Table des articles :

ticket	date	magasin	vendeur	produit	qte	pu
remise						
T05-250101-075845	2025-01-01	5	M. Ilboudo	Plâtre de construction 25 kg	11	4500
0.03						
T05-250101-075845	2025-01-01	5	M. Ilboudo	Gravier 15/25 (m3)	2	12000
NA						

Quatre colonnes du ticket sont recopiées sur les deux lignes (redondance assumée : c'est le prix du format plat, et il devient un problème quand les tables grossissent — M13). `remise` : NA, pas 0, tant que le dictionnaire n'a pas tranché ; et ce choix doit être écrit, car NA et 0 donnent deux totaux différents (le second est plus petit que le premier de 720 FCFA sur cette ligne : $24\ 000 \times 0,03$ n'existe pas dans le premier cas, dans le second il vaut exactement cela). Le bon critère de réussite : le total des `montant_ht` doit rester **inchangé** par l'opération d'aplatir.

Exercice 5.4. Réponse type : (1) `ventes2025.csv` exploitable, mais séparateur = tabulation, donc import à corriger ; (2) `clients.csv` exploitable directement, 11 colonnes ; (3) `tarif.xls` = ancien format binaire : conversion nécessaire, +1 h ; (4) `notes_livraison.txt` = prose : extraction non garantie, à chiffrer sur échantillon (proposition : 50 notes lues à la main, taux d'extraction valable annoncé) ; (5) `stats.json` : 120 Ko mais structure inconnue, merci de donner la liste des clés et un exemple complet. Le document est une réponse professionnelle complète : il dit ce qui est possible, ce qui coûte, ce qui manque.

Exercice 5.5. (i) un **tableau de bord** si la consultation est répétée et le public non technique, sinon un tableur : le critère est la fréquence et le nombre de relecteurs, pas la taille des données

(6 magasins × 36 mois = 216 lignes !) ; (ii) une **requête** — 240 000 lignes, une seule question, tri + LIMIT : un tableur rame, une base répond en une seconde ; (iii) **Python** — aplatir du JSON est mécanique en quelques lignes et rejouable mensuellement ; (iv) **script** — une vérification périodique doit s'exécuter sans humain, avec un message clair en cas d'écart. Un devoir qui répond « Power BI parce que c'est le plus puissant » perd le point de justification : le motif n'est pas un jugement sur l'outil, c'est une contrainte du cas.

13. Mini-projet M01.P5 — « La carte d'identité du lot » (40 min)

À partir du socle fourni (10 fichiers structurés + le fichier du fournisseur + un échantillon de notes libres que vous rédigez vous-même, 10 lignes), produisez 00_doc/carte_lot_donnees.md :

1. un tableau de 11 lignes × 7 colonnes (fichier, taille, lignes, lignes utiles, encodage, séparateur, chapeau) ;
2. un classement enveloppe/nature des **colonnes** de deux fichiers au choix ;
3. les trois questions métier rendues impossibles par l'état du lot, avec le motif exact (encodage ? grain ? prose ?) ;
4. une réponse de cinq lignes au client, sur le modèle du §7 étape 6, avec un délai annoncé en heures pour chaque tâche.

Barème : 6 pts exactitude des sept nombres (vérifiés par vos commandes ou votre tableur, pas recopiés) · 4 pts cohérence des classements nature ↔ statistiques autorisées · 5 pts clarté de la réponse au client · 5 pts faisabilité du délai annoncé. Le livrable est relu une deuxième fois par un pair : si son délai à lui diffère du vôtre de plus de 50 %, vous devez réécrire la justification — un chiffrage qui varie du simple au double n'est pas un chiffrage.

14. Résumé du chapitre

L'enveloppe (structurée, semi-structurée, non structurée) décide du **coût de préparation** et donc de l'outil. La nature (quantitative discrète/continue, qualitative nominale/ordinaire) décide des **statistiques licites**. Les deux se testent mécaniquement : y a-t-il un schéma régulier ? une soustraction a-t-elle un sens ? Un fichier plat se juge sur quatre propriétés vérifiables en ligne de commande — séparateur, encodage, décimale, chapeau. Un lot de données se traite en trois mots : **inventorier**, **classer**, **annoncer** — ce qui est possible tout de suite, ce qui demande un travail, ce qui manque.

15. À retenir

À retenir. 1. **Trois enveloppes, trois coûts** : table = prêt à calculer, JSON = à aplatir, prose = à extraire et valider. 2. **Un code numérique n'est pas une quantité** : le test décisif est « soustraire a-t-il un sens ? ». 3. **Nominal → mode et parts ; ordinal → + médiane ; quantitatif → moyenne et écart-type**, sous réserve d'asymétrie (M02). 4. **Vérifiez séparateur, encodage, décimale, chapeau** : quatre commandes, ou une ligne perdue dans un tableau croisé. 5. **L'outil se choisit selon qui relit et ce qui doit rester rejouable**, jamais selon la puissance supposée.

16. Évaluation formative (auto-correction, 12 min)

1. Un fichier de 5 lignes, une par client, chaque ligne contenant une liste d'achats entre crochets : enveloppe ? (semi-structurée : le schéma existe mais est irrégulier et imbriqué ; la table en résultant est à grain « article », pas « client »)
2. Une colonne `note_satisfaction` de 1 à 5 : quelles statistiques, et quelle phrase d'avertissement ? (mode, médiane, distribution, parts ; si moyenne, écrire l'hypothèse d'intervalles égaux)
3. Le test qui distingue une variable ordinale d'une variable à échelle de ratio ? (un écart égal vaut-il la même chose partout, et le zéro signifie-t-il l'absence ? les deux « oui » → ratio)
4. `file -i facture.csv` répond `charset=iso-8859-1` : qu'en concluez-vous pour l'import, et que ne faites-vous pas ? (lire/importer en Windows-1252 (latin-1), convertir en copie de test si besoin ; ne pas réenregistrer le brut en UTF-8 à l'aveugle)
5. **Question ouverte** : le client dit « on a tout en Excel, donc c'est structuré ». Répondez en quatre phrases en distinguant l'enveloppe, le schéma et la propreté. (attendu : un classeur peut contenir un schéma violé (notes libres, sous-totaux, cellules fusionnées) → l'enveloppe est table, le schéma est déclaré mais non respecté, la propreté est à établir : trois questions distinctes, trois chantiers distincts)

M01.C06 — Le cycle de vie d'une donnée : naissance, stockage, transformation, usage, archivage, suppression

Outil de ce chapitre : schéma sur papier ou tableau blanc, un tableur pour la partie écriture.
Durée indicative : 4 h. **Niveau :** N1.

L'idée du chapitre. Une ligne de vente ne « vit » pas dans un fichier : elle traverse six états, change de mains quatre fois, et survit à trois logiciels. Celui qui ne connaît que l'état « tableau de bord » produit des chiffres sans mémoire ; celui qui connaît le cycle entier sait **où** son chiffre a pu mourir, **qui** peut le contester, et **combien de temps** il doit pouvoir le retrouver. Ce chapitre est le seul du module qui ne comporte aucun calcul — et c'est celui qui, en entreprise, vous évitera le plus de désagréments.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Décrire** les six étapes du cycle d'une donnée et les responsabilités associées, sur un cas réel : la ligne d'un ticket du magasin 5 ;
2. **Situer** un incident dans le cycle : à quelle étape, chez qui, et avec quelle preuve retrouvable ;
3. **Rédiger** trois artefacts professionnels : le **dictionnaire de données**, le **journal de transformation**, la **politique de conservation** (une page chacun) ;
4. **Expliquer** le versionnage d'un jeu de données (pourquoi on garde l'historique des fichiers et pas seulement des rapports) ;
5. **Décider** de la suppression : ce qui doit disparaître, ce qui doit survivre, et dans quel ordre ;
6. **Nommer** les obligations qui pèsent sur une donnée client (protection, minimisation, durée) sans jargon juridique, avec les réflexes qui vont avec.

2. Pourquoi cette notion est importante

Situation 1 — le chiffre disparu. Trois mois après la clôture d'exercice, un contrôleur demande le détail d'un total. Le rapport PDF existe. Le fichier de travail a été écrasé par la version du mois suivant, parce que le nom du fichier ne portait pas de date. Le total devient « une affirmation de l'analyste ». La réponse professionnelle — reproductible en cinq minutes — aurait été : garder les entrées, les sorties, et le journal.

Situation 2 — la suppression qui détruit la preuve. Un poste est libéré, le service nettoie le dossier partagé, et fait disparaître les fichiers sources de 2023. En zone OHADA, les pièces comptables se conservent **10 ans** : un chiffre d'affaires publié sur une base détruite n'est plus défendable, ni devant l'administration, ni devant un associé sortant. La suppression est une étape du cycle comme les autres, et elle se planifie.

Situation 3 — les données personnelles en trop. Pour calculer un chiffre d'affaires par type de client, un analyste télécharge la table complète avec noms, téléphones et e-mails, la dépose sur un disque perso pour travailler le week-end. Aucun chiffre n'est faux, et pourtant il y a un

problème : ce n'est pas la bonne **étape**, ni le bon **périmètre**, ni le bon **support**. La protection des données se règle dans le cycle, pas après le rapport.

Dans les faits. Dans le socle du manuel, une ligne de vente naît à la caisse du magasin 5, est exportée chaque nuit dans `ventes_brutes.csv` (243 360 lignes, 28,5 Mo), nettoyée en `ventes_propres.csv` (240 000 lignes), agrégée dans des tableaux mensuels, publiée dans un rapport, archivée après clôture à 3 ans, et purgée à 10 ans — **sauf** le dictionnaire et le journal, qui survivent, parce qu'ils expliquent. Cette chaîne, vous allez la parcourir de bout en bout au fil des modules : M04-M05 (nettoyage), M07-M08 (transformation), M13-M18 (usage), M21-M22 (gouvernance et soutien).

3. Explication simple

Prenez le ticket de caisse d'un client qui achète onze sacs de plâtre, le 1^{er} janvier 2025 à 11 h 01. Suivez-le.

1. **Naissance.** La caissière scanne, le logiciel écrit une ligne. Ce qui naît, ce n'est pas « une vente » : c'est **une ligne d'article**, avec un numéro de ticket, une date de **saisie**, un identifiant client éventuellement vide. Ce que la caisse ne note pas — l'absence de client identifié, par exemple — n'existera jamais nulle part. Une donnée est un traceur du processus qui l'a produite.
2. **Stockage.** La nuit, le logiciel exporte un CSV. Le fichier atterrit sur un serveur. Deux choses peuvent mourir ici : l'export interrompu (il écrit 80 % des lignes) et la réécriture (le même nom, un fichier différent). Une donnée sans date ni empreinte n'est pas une version, c'est un instantané.
3. **Transformation.** Vous ouvrez le fichier, retirez les doublons, convertissez les montants, créez une colonne mois. Ici se joue l'honnêteté du résultat : tout choix non écrit est une modification invisible. Une transformation sans journal est une opinion.
4. **Usage.** Le chiffre d'affaires mensuel s'affiche dans un tableau de bord, le directeur l'évoque en réunion, un commercial ajuste sa relance. L'usage est l'unique étape où la donnée produit de la valeur, et le seul endroit où le public voit quelque chose. Une donnée qui ne sert à aucune décision n'a que des coûts.
5. **Archivage.** L'année est close. On fige les fichiers et les rapports en lecture seule, avec le dictionnaire et le journal à côté. Archiver, c'est rendre rejouable, pas seulement garder.
6. **Suppression.** Le délai légal passe. On purge les détails, on garde les agrégats et la documentation, on note ce qui a été supprimé et pourquoi. Supprimer proprement est plus difficile que ne rien supprimer ; supprimer n'importe comment est une faute.

Le voyage d'une ligne, de la caisse au rapport annuel



Responsable : caissier → admin. de l'ERP → vous → le BI → la DAF → la DAF, encore.

- Où le chiffre meurt : export coupé, fichier réécrasé, arrondi, filtre oublié, fusion dupliquée (+9 lignes ici).
- Ce qui prouve votre travail dans trois ans : le journal de transformation et le dictionnaire, pas votre mémoire.
- Règle du manuel : on ne modifie jamais un fichier brut ; du brut on tire du propre, et on garde les deux.

Figure 6 — Six étapes, cinq responsables, et l'endroit exact où votre total peut disparaître.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Cycle de vie — lifecycle	Ensemble des états d'une donnée, de sa production à sa disparition.	Croire que le cycle finit au tableau de bord.
Source de vérité — system of record	Le système où l'information est officiellement détenue (ici : la caisse).	Exporter depuis un rapport au lieu d'exporter depuis la source.
Donnée brute — raw data	État au moment de la réception, non modifié.	Corriger le brut « pour aller vite » : la référence disparaît.
Empreinte — checksum / hash	Signature courte calculée sur le contenu du fichier (MD5, SHA-256).	Comparer deux fichiers par la taille seule : deux fichiers de même taille peuvent différer.
Ligne d'eau — watermark	Marqueur de ce qui a déjà été traité (date, n° d'enregistrement).	Ré-exécuter un import et doubler les lignes.
Journal de transformation — transformation log	Liste datée, ordonnée, des opérations appliquées.	Le rédiger après coup, de mémoire.
Dictionnaire de données — data dictionary	Document qui décrit colonnes, types, unités, règles, provenance.	Le garder dans la tête : il meurt avec la personne.
Versionnage — versioning	Conservation des états successifs, identifiables et comparables.	Remplacer la version précédente sans la nommer.
Jeu figé — frozen dataset	Copie verrouillée, référencée par un identifiant, servant de base officielle à une publication.	Publier un chiffre depuis un fichier qui continue d'évoluer.
Rétention — retention	Durée de conservation, et ce que l'on détruit à l'échéance.	Conserver tout (coût, risque juridique) ou ne rien conserver (perte de preuve).
Anonymisation / pseudonymisation	Retirer ou remplacer les identifiants directs avant usage.	Croire qu'enlever le nom suffit : un téléphone ou une ville suffisent à ré-identifier.
Minimisation — data minimisation	Ne traiter que les champs nécessaires à la question.	« Télécharger toute la table au cas où ».

5. Cours approfondi

5.1 Les six étapes, avec leur preuve documentaire

Chaque étape produit un **artefact** — un document, un fichier, une trace — et c'est ce qui distingue une chaîne de traitement d'un bricolage.

Étape	Ce qui se passe	Responsable principal	Artefact de preuve	Défaut typique de l'étape
Naissance	saisie, formulaire	capteur, opérationnel (caissier, magasinier)	le ticket, la ligne en base	oubli de saisie, double scan
Stockage	écriture, transfert	export, informatique éditeur du logiciel	fichier daté + empreinte, journal d'export	export partiel, réécrasement
Transformation	nettoyage, recalcul, enrichissement	vous	script ou classeur de travail + journal	correction non écrite, arrondis en cascade
Usage	consultation, décision, publication	métier / direction	le rapport + la note de périmètre	chiffre cité hors période, filtre oublié

Étape	Ce qui se passe	Responsable principal	Artefact de preuve	Défaut typique de l'étape
Archivage	figement, classement	vous + DSI	jeu figé dictionnaire manifeste	+ archivage sans le code qui a produit le chiffre
Suppression	purge, anonymisation	DSI + responsable légal	registre suppression	de suppression du dictionnaire, conservation de données personnelles inutiles

Lisez le tableau de bas en haut une fois : un chiffre d'affaires de 2023, cité en réunion en 2026, ne peut être défendu que si les six artefacts existent. Un seul manque et l'argument devient « je crois me rappeler ».

Définition. Source de vérité — system of record — le système où une information est officiellement détenue : ici, la caisse du magasin. Tout le reste en est une copie, plus ou moins fraîche. Travailler sur une copie est licite, pour la performance ou pour ne pas gêner l'exploitation, à une condition : le retard doit être écrit et connu.

Définition. Donnée brute — raw data — l'état exact de ce qui a été reçu, octet pour octet, avant la moindre correction. Le brut n'est pas « le fichier moche » : c'est la seule preuve de ce que le système émetteur contenait ce jour-là. On le lit, on ne l'édite pas.

5.2 Le dictionnaire de données : votre premier document professionnel

Le dictionnaire décrit **les colonnes**, pas le logiciel. Une ligne par colonne, sept informations, et il est terminé. Modèle, appliqué à notre fichier d'atelier :

champ	type	unité / domaine	règle de calcul	provenance	défauts connus	commentaire
n_ticket	texte	—	émis par la caisse, unique par vente	caisse.log	9 lignes dupliquées dans l'extrait	sert de clé avec produit
date	date	jour	date de l'opération, pas de la saisie	caisse	65 lignes en JJ/MM/AAAA	à normaliser en ISO avant tout regroupement
magasin	texte	1 à 6 dépôt	+ référentiel magasins.csv	caisse	—	le dépôt (id 6) ne vend pas au comptoir
client	texte	identifiant	0 = vente au comptoir	caisse	18 vides	vide ≠ 0 : le vide est une absence
quantite	nombre	unité du produit	signée (négatif = retour)	caisse	7 lignes > 500	à contrôler par plage métier
prix_unitaire_ht	nombre	FCFA HT	prix catalogue du jour	tarif	—	TVA 18 % appliquée au total, pas à l'unité
remise	nombre	proportion	0 à 0,40	saisie vendeur	—	les valeurs > 0,4 sont des points de % mal saisis
montant_ht	nombre	FCFA	quantite × prix × (1 - remise), arrondi à l'entier	calcul caisse	—	contrôle de cohérence avec quantite
montant_ttc	nombre	FCFA	montant_ht × 1,18, arrondi	calcul caisse	486 lignes en texte	total de référence du module : 36 073 185 FCFA (nettoyé)

Une colonne **défauts connus** est ce qui fait la différence entre un dictionnaire d'intention et un dictionnaire utile : elle évite à celui qui reprend le travail de redécouvrir la même chose dans six mois. Ce dictionnaire est un livrable du projet M01.P ; il sera réutilisé et complété en M04 (nettoyage), M05 (qualité), M06 (modélisation), M21 (gouvernance). **Version alternative pour un petit écran** — ce dictionnaire de sept colonnes se lit par la seule colonne « défauts connus », qui porte l'essentiel : 9 lignes dupliquées, 65 dates écrites en texte, 18 clients vides, 7 quantités au-dessus de 500, 486 montants en texte, TVA appliquée au total. Les six autres colonnes disent seulement où poser la main.

Définition. Manifeste de jeu de données — dataset manifest — fiche courte qui accompagne un jeu figé : ce qu'il contient, d'où il vient, quelle version du code l'a produit, quelles empreintes, quelles limites connues, qui a validé, à quelle date. Cinq minutes à écrire ; il rend un archivage compréhensible par un inconnu — donc par vous dans trois ans.

5.3 Versionner un jeu de données : trois mécaniques, par ordre de sérieux

1. **Le nommage daté** (acquis au chapitre 1) : `2025_ventes_M5_brut.csv` ne change jamais ; le nettoyage produit `2025_ventes_M5_propre_v1.csv`. Simple, mais rien n'empêche l'écrasement.
2. **Le dossier de version + le manifeste** : chaque livraison de travail a son répertoire `2026-09-17_v1/` contenant données, code, journal, manifeste. On ne modifie jamais une version publiée ; on ouvre la suivante.
3. **Le dépôt de versions** (Git) : suit les fichiers texte — scripts, requêtes, dictionnaire, journal — et pas les gros fichiers de données, qui y sont mal à l'aise. Un `README` daté et un commit par livrable. C'est l'objet du module M21 ; en attendant, la discipline 2 suffit et prépare la 3.

Ce qui compte, dans les trois cas, est le même : **à tout instant, on doit pouvoir répondre « de quel fichier vient ce chiffre ? » et le rouvrir.**

Définition. Empreinte — checksum / hash — une signature courte calculée sur le contenu d'un fichier. Même empreinte, même contenu ; empreinte différente, contenu différent — sans avoir à ouvrir ni à comparer les lignes. L'empreinte remplace la confiance par un contrôle, et c'est ce qui manque dans la plupart des échanges de fichiers par courriel.

Définition. Jeu figé — frozen dataset — copie verrouillée, nommée, référencée par un identifiant, sur laquelle un chiffre publié a été calculé. C'est ce qui permet, un an plus tard, de retrouver exactement le total annoncé. Sans jeu figé, le même rapport se recalcule différemment le lendemain, et les chiffres se contredisent en réunion.

5.4 Journal de transformation : le format, et cinq exemples

Format imposé par le manuel, une ligne par opération :

```
AAAA-MM-JJ HH:MM  auteur  [fichier source → fichier cible]  opération  entrée=n  sortie=n  écart
motif
```

Appliqué à l'échantillon de l'atelier, les cinq entrées qui comptent vraiment :

```

2026-09-17 09:12 a.minani [brut/2025_ventes_M5_brut.csv → propre/2025_ventes_M5_propre_v1.csv]
import, 13 colonnes déclarées texte entrée=489 sortie=489 écart=0 motif: éviter tout typage
automatique
2026-09-17 09:26 a.minani [brut/2025_ventes_M5_brut_recu2026-09-17.csv → propre/
2025_ventes_M5_propre_v1.csv] suppression 9 lignes strictement identiques entrée=489 sortie=480
écart=-9 motif: doublons d'export, total TTC -266 132 FCFA (0,74 %)
2026-09-17 09:41 a.minani [propre/2025_ventes_M5_propre_v1.csv] normalisation date → ISO
entrée=480 sortie=480 écart=0 motif: 65 lignes en JJ/MM/AAAA ; contrôle par répartition
mensuelle (décembre 5 292 517 FCFA)
2026-09-17 09:58 a.minani [propre/2025_ventes_M5_propre_v1.csv] montant_ttc texte → nombre
entrée=480 sortie=480 écart=0 motif: 486 lignes avec espace de milliers ; contrôle NB = NBVAL OK
2026-09-17 10:07 a.minani [propre/2025_ventes_M5_propre_v1.csv] 7 lignes quantité > 500
marquées suspectes, **non supprimées** entrée=480 sortie=480 écart=0 motif: montant cohérent
avec le prix, doute sur l'unité de saisie ; décision reportée au module 4 avec le métier

```

Deux remarques. D'abord, dans un vrai journal, on répète le nom complet du fichier à chaque ligne : la concision n'est jamais un motif d'ambiguïté. Ensuite — et c'est le point capital de ce format — **on peut signaler sans supprimer** : la dernière entrée documente sept lignes douteuses qui restent dans le fichier. Cette distinction entre « défaut traité » et « défaut documenté » est la marque du professionnel : le premier se corrige, le second s'annonce, et les deux s'écrivent.

Règles du journal : on n'efface jamais une ligne ; on en ajoute une qui annule la précédente (« 10:15 correction : le point 4 s'appliquait aussi à... ») ; chaque suppression de ligne s'accompagne du **motif** et de l'**écart chiffré**. Un journal sans écart chiffré est un agenda.

5.5 Les points de rupture : où votre chiffre meurt réellement

Les six étapes ont chacune leur mode de mort statistiquement dominant dans les PME sans informatique structurée. Apprenez la liste : elle sert de grille de diagnostic.

1. **Naissance** : oubli de saisie, ou saisie en fin de journée « de mémoire ». Le chiffre est faux **avant** d'exister dans un fichier. Contre-mesure : contrôles à la saisie, et, côté analyse, jamais de conclusion de cause profonde sur un processus non instrumenté.
2. **Stockage** : export partiel, réécrasement, fichier envoyé par messagerie et modifié en pièce jointe. Contre-mesure : empreinte + nom daté + un seul dossier de référence.
3. **Transformation** : arrondis successifs, filtre oublié, jointure qui multiplie (M04, M07), suppression silencieuse de lignes « douteuses ». Contre-mesure : le journal, et un total de contrôle à chaque étape (ici, 36 073 185 FCFA).
4. **Usage** : le chiffre est cité hors de son périmètre (« décembre = 5,3 M » répété sans dire magasin 5, 2025, extrait), ou comparé à une autre source dont la définition diffère (HT vs TTC, retours inclus ou non). Contre-mesure : la note de périmètre à côté de tout chiffre publié, y compris dans l'e-mail.
5. **Archivage** : on archive le rapport sans le code, sans le journal, sans le dictionnaire. Contre-mesure : le manifeste ; cinq minutes.
6. **Suppression** : on supprime trop (preuve perdue) ou on conserve trop (données personnelles inutiles, coût, risque en cas d'intrusion). Contre-mesure : la politique de conservation.

Attention. Ré-exécuter un import sans point d'arrêt **double les lignes** : le total est faux, et il est faux en beau — c'est le défaut que l'on découvre le plus tard. La parade tient en deux mots : marqueur de reprise (watermark, la dernière date ou le dernier identifiant déjà traité) et clé unique dans la table d'arrivée, qui refuse le deuxième envoi.

5.6 Conserver, purger, protéger : la politique en une page

Un modèle utilisable dès votre premier stage, et qui tient sur une page :

Catégorie	Exemple	Durée	Support et protection	À l'échéance
Pièces comptables et justificatifs	exports de caisse, factures	10 ans (zone OHADA)	serveur de l'entreprise, accès restreint, chiffré	purge avec registre
Données d'exploitation	stocks, objectifs	3 ans	dossier de travail + sauvegarde froide	agrégats gardés, détail purgé
Données clients à caractère personnel	noms, téléphones, e-mails	le temps de la relation + durée légale de prospection	base métier, jamais copie personnelle	suppression ou anonymisation
Traces de travail	journal, scripts, dictionnaire	10 ans (alignées sur la donnée qu'elles expliquent)	dépôt de versions ou archive lisible	conservées plus longtemps que la donnée
Rapports publiés	PDF signés, tableaux figés	10 ans	archive officielle, lecture seule	jamais purgés sans validation de la direction

La ligne la plus contre-intuitive est la quatrième : **la documentation survit à la donnée**. Si les détails sont purgés, le dictionnaire et le journal permettent encore de comprendre ce que signifiait le chiffre et comment il a été obtenu. C'est pourquoi on ne supprime jamais les deux en même temps que les sources.

Côté protection, trois réflexes qui ne se discutent pas, quel que soit le pays : (1) pas de copie personnelle de données clients sur un disque privé ou une messagerie privée ; (2) minimisation — si la question porte sur le chiffre d'affaires par catégorie, la table `clients` n'a pas besoin des téléphones ; (3) si un échantillon est nécessaire pour débbugger, on le **pseudonymise** (remplacer les identifiants par des numéros séquentiels, en gardant la table de correspondance uniquement sur le poste métier).

Conseil professionnel. Quand on vous demande « tu peux me renvoyer le fichier avec les données clients ? », demandez d'abord **quelle question** il doit servir. Dans la majorité des cas, une table agrégée par client (sans nom, sans téléphone) suffit — et vous venez d'éviter une fuite sans rien refuser. Cette esquivé élégante est une compétence de métier, pas une prudence de juriste.

Attention. Retirer la colonne « nom » ne rend pas un fichier anonyme. Un numéro de téléphone, une date de naissance et une ville suffisent souvent à retrouver une personne dans une population de quelques dizaines de milliers d'individus. La protection se juge sur le risque de ré-identification, pas sur la suppression d'un champ.

À retenir. Le cycle de vie ne s'arrête pas au tableau de bord : il s'arrête quand la donnée est archivée ou détruite, avec une date et une décision écrites. Les trois preuves qui protègent un analyste sont le brut conservé, le journal de transformations et le jeu figé référencé — pas le logiciel utilisé.

6. Exemple concret : une ligne, trente-deux mois, six fichiers

Notre ligne du 1^{er} janvier 2025, onze sacs de plâtre à 4 500 FCFA, `montant_ttc = 56 658 FCFA`, dans le cycle complet du socle du manuel :

Date	État	Fichier ou système	Ce qui a changé pour elle
2025-01-01 11:01	née	caisse du magasin 5	rien : 3 lignes dans le ticket (2 articles, 1 remise)
2025-01-02 02:00	stockée	ventes_brutes.csv (243 360 lignes, 28,5 Mo)	dates et montants devenus du texte
2025-01-15	extraite	projection/ventes_magasin5_2025.csv (489 lignes)	échantillon de 40 lignes par mois, donc 40 lignes par mois
2026-09-17	transformée	reference/ventes_magasin5_2025_ATTENDU.csv (480 lignes)	9 doublons retirés, montant converti, date ISO
2026-09-18	agrégée	table mensuelle (12 lignes)	décembre 5 292 517 FCFA, janvier 2 051 424 FCFA
2026-10	usagée	rapport direction + note	le chiffre est cité avec périmètre, grâce au journal
2028	figée	archive 2025_close/ + manifeste	lecture seule, empreintes, dictionnaire à côté
2036	purgée	registre de suppression	détail effacé, dictionnaire et journal conservés

Remarquez deux choses. D'abord le chemin de la ligne : du **ticket** au **total mensuel** en passant par **quatre fichiers**, dont un échantillon de 489 lignes qui n'est **pas** l'intégralité de l'année du magasin (40 lignes par mois, pas 480 réparties au hasard). C'est précisément ce qui rend le jeu pédagogique : un échantillon qui se prend pour une population est l'erreur la plus fréquente des débutants (M02, M11).

Ensuite, la ligne « 2025-01-15 » : **un extrait de travail**. Un extrait n'est jamais une source ; il doit être nommé comme tel, daté, et sa taille annoncée. Le fichier de l'atelier pèse 489 lignes : si vous l'appellez « les ventes 2025 » dans un mail, vous venez d'inventer un chiffre d'affaires de 36 073 185 FCFA comme étant celui de l'entreprise, alors que le total de l'extrait correspond à **0,23 %** du chiffre consolidé (15 419 985 157 FCFA sur 4 ans). L'ampleur de la confusion fait peur, et elle se prévient avec un mot dans le nom de fichier : `extrait`.

7. Démonstration pas à pas : reconstruire le cycle d'un fichier reçu

Exercice complet, à faire sur le poste, 25 minutes. Objectif : produire les trois artefacts (§5.2, §5.4, §5.6) à partir d'un fichier reçu, sans rien calculer d'analytique.

Étape 1 — Recevoir (2 min). Copiez le fichier dans `01_brut/`, nommez-le `2025_ventes_M5_brut_recu2026-09-17.csv`, passez-le en lecture seule. Première ligne du journal, avec la taille (32 Ko) et le nombre de lignes (489).

Étape 2 — Empreinter (3 min). Sous Windows (PowerShell) ou terminal :

```
Get-FileHash .\01_brut\2025_ventes_M5_brut_recu2026-09-17.csv -Algorithm SHA256
```

```
sha256sum 01_brut/2025_ventes_M5_brut_recu2026-09-17.csv
```

Notez l'empreinte dans le journal **et** dans un fichier `00_doc/empreintes.txt`. Deux ans plus tard, cette ligne prouvera que le fichier que vous aviez est bien celui-là — utile quand un prestataire jure qu'il vous a envoyé autre chose.

Étape 3 — Décrire la structure (5 min). Ouvrez le fichier en texte brut, notez : 13 colonnes, séparateur `;`, encodage, ligne d'en-tête exacte, trois premières et trois dernières lignes. Résul-

tat attendu : l'en-tête n_tic-
cket;date;heure;magasin;vendeur;client;produit;categorie;quantite;prix_unitaire_ht;remise;montant_ht;montant
nière ligne du 31 décembre 2025.

Étape 4 — Écrire le dictionnaire (6 min). Reprenez le modèle du §5.2 : neuf lignes suffisent pour les colonnes qui servent au calcul ; les quatre autres reçoivent « attribut descriptif, non utilisé dans les totaux ». La colonne **défauts connus** se remplit avec les comptages réels : 9 doublons, 65 dates à deux formats, 486 montants en texte, 7 quantités suspectes, 18 clients vides.

Étape 5 — Ouvrir le journal de transformation (2 min). Les deux premières entrées (réception, empreinte), puis une ligne vide de contrôle : prochain contrôle attendu : total TTC après nettoyage = 36 073 185 FCFA (±1 ligne). Vous venez d'écrire une obligation de résultat pour vous-même, vérifiable par n'importe qui.

Étape 6 — Fixer la politique (4 min). Une page, trois colonnes : ce que je garde (brut, propre, journal, dictionnaire), combien de temps (10 ans pour le brut, aligné sur la valeur comptable du chiffre ; 3 ans pour mes brouillons), où (dossier de travail + disque chiffré + sauvegarde froide, pas de copie personnelle). Signez-la et datez-la ; en stage, faites-la viser par le responsable. Ce document, anodin, est celui qui vous protégera — pas l'inverse.

Contrôle final de la démonstration. Un collègue, sans vous voir, doit pouvoir : (1) dire d'où vient le fichier ; (2) retrouver son empreinte ; (3) savoir quelles colonnes sont des montants et lesquelles sont des identifiants ; (4) expliquer les 9 lignes manquantes entre brut et propre ; (5) savoir quoi en faire dans dix ans. Si une des cinq échoue, le dossier n'est pas fini.

8. Erreurs fréquentes

Symptôme	Cause	Correction
Le chiffre du mois change entre deux demandes	Le fichier source a continué d'évoluer	Fixer un jeu de travail pour la période publiée, avec un identifiant de version
Deux fichiers « identiques » aux totaux différents	L'un a été enregistré depuis Excel (dates, arrondis, colonne de garde)	Un brut se lit, ne s'édite pas ; comparer les empreintes pour trancher
Impossible de retrouver le code d'un chiffre de 2023	Le script vivait sur le bureau, le poste a changé	Le code vit avec la donnée (dépôt ou dossier 03_analyse), jamais sur un support personnel
Suppression de « lignes bizarres » non tracée	Nettoyage sans journal	Marquer, ne pas supprimer : une colonne statut_qualite et une ligne de journal
Le dictionnaire ne correspond plus aux fichiers	Écrit une fois, jamais entretenu	Révision à chaque nouveau champ, et une ligne dans le manifeste
Copie de 3 Go de données clients sur une clé USB	Convenience	Question à trancher en amont : la table agrégée suffit presque toujours
Un « export du rapport » utilisé comme source	Confusion source de vérité / vue	Retomber sur la source (caisse, base), jamais sur un document de sortie

9. Bonnes pratiques professionnelles

- [] Réception = copie + renommage daté + lecture seule + empreinte + entrée de journal.
- [] Un dictionnaire par jeu, tenu à jour à chaque champ ajouté ou retiré.
- [] Journal de transformation à chaque opération non triviale, avec `entrée=n` `sortie=n` `écart` chiffré.
- [] Signaler plutôt que supprimer, quand le doute est métier.
- [] Un jeu figé et un manifeste pour chaque publication.

- [] Le code vit à côté de la donnée qui l’a vu naître.
- [] Politique de conservation écrite, datée, visée ; documentation conservée plus longtemps que la donnée.
- [] Minimisation et pseudonymisation des échantillons de test ; jamais de copie personnelle de données clients.
- [] Un extrait est nommé `extrait_`, avec sa taille et sa période dans le nom.

10. Exercice guidé — reconstituer l’histoire d’un dossier à charge

Contexte. Vous reprenez le travail d’une personne partie. Dans `D:/Travail/`, un seul dossier, `rapports/`, contient :

CA 2023.xlsx	1,1 Mo	12/09/2024
CA 2023_final.xlsx	1,2 Mo	03/10/2024
rapport_ca_2023.pdf	240 Ko	15/09/2024
notes.txt	4 Ko	12/09/2024
base.mdb	38 Mo	31/08/2024

Reconstruisons, en posant les questions dans l’ordre du cycle.

1. **Source de vérité ?** `base.mdb` (base Access, 38 Mo, dernière écriture fin août) ressemble à la source ; `CA 2023*.xlsx` sont des transformations ; le PDF est un usage publié. Vérification : la taille et la date — un fichier de 38 Mo non modifié depuis plus de deux semaines est une source, un fichier de 1 Mo touché plus tard est une sortie.
2. **Lequel des deux classeurs a produit le PDF ?** Le PDF est daté du 15/09, `CA 2023.xlsx` du 12/09, `CA 2023_final.xlsx` du 03/10. Conclusion probable : le PDF vient du **premier**, et le second est un retravail postérieur. Preuve à chercher : ouvrir les deux, comparer le total de contrôle ; ouvrir le PDF, lire le total cité.
3. **Que dit `notes.txt` ?** Un fichier de 4 Ko daté du 12/09, donc du jour du premier classeur : c’est probablement un journal informel. Lisez-le **avant** de toucher aux fichiers : il contient peut-être la règle de retrait des retours, et c’est le seul endroit où elle est écrite.
4. **Le cycle est-il rejouable ?** Non : rien ne dit comment `base.mdb` est devenu `CA 2023.xlsx`. C’est la trace manquante, celle qu’il faut reconstituer en priorité — une requête Access, un export manuel ?
5. **Décision de reprise.** Plutôt que « refaire pareil », reprenez le cycle en amont : extraire de la base (étape 2), produire un CSV daté, écrire le dictionnaire et le journal, recalculer, **puis** comparer au PDF du 15/09 pour valider la reprise. Si l’écart est non nul, vous avez trouvé soit une transformation cachée, soit un chiffre publié faux — dans les deux cas, vous gagnez quelque chose.

Ce type de reprise est une activité réelle et rémunérée : audit de données, passation de consignes, continuité de service. Elle se joue sur quatre dates et trois tailles de fichiers, pas sur une maîtrise logicielle.

11. Exercices autonomes

Exercice 6.1 (★) — Les six artefacts. Pour le fichier de l’atelier, citez les six artefacts qui doivent exister à la fin du projet M01, et indiquez pour chacun le document (ou le fichier) où il vit. (brut daté + empreinte · dictionnaire · journal · classeur/table de travail · livrable avec note de périmètre · manifeste + politique de conservation)

Exercice 6.2 (★) — Qui répond ? Quatre questions posées en réunion : « pourquoi 489 lignes et pas 480 ? », « le CA est-il TTC ? », « décembre est-il vraiment le meilleur mois ? », « que devient le comptoir dans le classement clients ? ». Attribuez chacune à une étape du cycle et à l'artefact qui y répond. (stockage/transformation → journal · transformation → dictionnaire (unité) · usage → note de périmètre et tableau mensuel · naissance/dictionnaire (0 = comptoir))

Exercice 6.3 (★★) — Datation Forensique. Reprenez le dossier de l'exercice guidé : le total de `CA_2023_final.xlsx` diffère de celui du PDF de 1,8 %. Listez trois explications compatibles avec les dates, et le test qui les sépare. (le second classeur intègre des régularisations postérieures au PDF · il contient les retours, pas le premier · il a été enregistré après un changement de séparateur de dates ; test : compter les valeurs distinctes de `n_ticket` et nombre de lignes, vérifier si `notes.txt` mentionne une régularisation, recalculer depuis `base.mdb`)

Exercice 6.4 (★★) — L'extrait qui ment. Vous citez « CA 2025 du magasin 5 : 36 073 185 FCFA » alors que le fichier est un extrait de 489 lignes (40 par mois). Rédigez la phrase exacte qui rend la citation honnête, et celle qui aurait dû figurer dans le nom du fichier. (attendu : « CA de l'extrait de 480 lignes utiles (40 lignes par mois), magasin 5, 2025 » ; nom : `2025_ventes_M5_extrait40parmois_brut.csv`)

Exercice 6.5 (★★★) — Politique. Rédigez la politique de conservation d'un service qui produit des rapports mensuels à partir de la caisse, en cinq lignes maximum, couvrant : ce qui est la source, ce qui est figé, ce qui est purgé à 3 ans, ce qui survit à 10 ans, et l'interdit sur les supports personnels. (barème implicite : une ligne par obligation, aucune phrase générale)

12. Correction détaillée

Exercice 6.1. Brut : `01_brut/...recu2026-09-17.csv` + `00_doc/empreintes.txt` ; dictionnaire : `00_doc/dictionnaire.md` (neuf lignes utiles) ; journal : `90_journal/journal_transformation.txt` ; travail : `03_analyse/2026-09-17_nettoyage_M5_v1.xlsx` ; livrable : `04_livrables/2026-09-17_note_CA_M5_minani.pdf` avec sa note de périmètre en page 1 ; archive : `99_archive/2025_close/` avec manifeste, plus la politique de conservation en `00_doc/`.

Exercice 6.2. 489 vs 480 → transformation, réponse dans le journal (9 doublons d'export) ; unité TTC → dictionnaire, ligne `montant_ttc` (TVA 18 %, arrondi à l'entier) ; décembre → usage + transformation, réponse par le tableau mensuel et le test de robustesse (voir M02) ; comptoir → naissance, réponse dans le dictionnaire (0 = comptoir, à ne pas effacer). Quatre questions, quatre artefacts : c'est le mécanisme à intégrer, pas les réponses.

Exercice 6.3. Test discriminant, dans l'ordre de coût : (a) comparer les **nombre de lignes** des deux classeurs → si 480 vs 489, c'est un dédoublonnage, pas une régularisation ; (b) filtrer les lignes à `quantite` négative dans chacun → les retours ; (c) lire `notes.txt` à la recherche d'une mention de régularisation et de sa date. Un écart de 1,8 % sur un total de plusieurs dizaines de millions de FCFA se compte en centaines de milliers : soit l'équivalent d'un mois de l'extrait, soit un ensemble de retours. La taille de l'écart **oriente** la recherche — c'est le réflexe à noter.

Exercice 6.4. Phrase honnête : « Le total de **36 073 185 FCFA** est celui de l'extrait de travail (480 lignes utiles, 40 lignes retenues par mois, magasin 5, année 2025), après retrait de 9 doublons d'export ; il ne représente pas le CA du magasin sur l'année, qui suppose le fichier complet. » Nom de fichier : `2025_ventes_M5_extrait40parmois_brut.csv`. Un devoir qui écrit « le CA du magasin 5 » sans qualifier l'extrait a échoué, quel que soit le reste.

Exercice 6.5. Modèle recevable : (1) source = export quotidien de la caisse, déposé en `01_brut/`, jamais modifié, empreint et daté ; (2) chaque publication fige un jeu (`2025_close/`) avec dictionnaire, journal et manifeste ; (3) à 3 ans, purge des fichiers de travail et des extraits, agrégats mensuels conservés ; (4) à 10 ans, purge du détail des lignes, conservation des agrégats

gats, du dictionnaire et du registre de suppression ; (5) aucune copie de données clients hors du parc de l'entreprise, supports personnels interdits, échantillons de test pseudonymisés.

13. Mini-projet M01.P6 — « Dossier d'archivage d'une publication » (35 min)

Vous publiez le total nettoyé de l'atelier (36 073 185 FCFA, extrait 480 lignes, magasin 5, 2025). Constituez 99_archive/2025_close/ contenant :

1. les deux fichiers de données (brut daté, propre nommé ..._v1) ;
2. le dictionnaire (9 lignes minimum, colonne « défauts connus » renseignée avec les comptages réels) ;
3. le journal de transformation, 5 entrées minimum, chaque entrée avec entrée=n sortie=n écart ;
4. un manifeste d'une demi-page : contenu, empreintes, limites (« extrait de 40 lignes par mois, pas un CA annuel »), auteur, date, validation ;
5. un PDF ou une note d'une page avec la note de périmètre en tête.

Critères de réussite : un pair, qui n'a pas fait l'exercice, doit pouvoir **retrouver l'origine de chaque nombre cité** dans la note en moins de cinq minutes, et dire ce qui est archivé pour dix ans. Il écrit ses deux réponses ; toute réponse erronée annule le point de clarté. Barème : 6 pts reproductibilité · 4 pts exactitude des comptages (489/480/9/65/486/18/7) · 4 pts manifeste · 3 pts note de périmètre · 3 pts lisibilité de l'arborescence.

14. Résumé du chapitre

Six étapes, six responsabilités, six artefacts. La donnée naît dans un processus (et porte ses angles morts), se stocke (et peut être perdue là), se transforme (et doit laisser une trace chiffrée de chaque choix), sert (et c'est le seul endroit où elle a de la valeur), s'archive (avec son mode d'emploi, sinon rien) et se supprime (selon une politique écrite, pas selon l'humeur). Un chiffre défendable dans trois ans est le produit d'un cycle documenté, pas d'un tableur bien maîtrisé. Le dictionnaire, le journal et le manifeste sont les trois documents qui vous rendent crédible ; les deux premiers tiennent sur une page, et se tiennent à jour en une ligne par opération.

15. À retenir

À retenir. 1. **Six étapes** : naissance · stockage · transformation · usage · archivage · suppression — chacune a son responsable et sa preuve. 2. **Trois documents** suffisent à rendre un chiffre défendable : dictionnaire, journal de transformation, manifeste d'archivage. 3. **Un extrait n'est pas une population** : nommez-le, datez-le, annoncez sa taille. 4. **Le brut est intouchable** ; on le duplique, on ne le corrige pas. Empreinte + date = version. 5. **La documentation survit à la donnée** ; la minimisation et l'interdit de copie personnelle protègent davantage que n'importe quel logiciel de sécurité.

16. Évaluation formative (auto-correction, 10 min)

1. Un chiffre publié est contesté trois ans plus tard. Citez, dans l'ordre où vous les ouvrez, les trois documents à consulter. (note de périmètre du livrable → journal de transformation → dictionnaire ; puis le brut si un recomptage est nécessaire)
2. À quoi sert une empreinte (hash) de fichier, et que ne dit-elle pas ? (prouve l'identité binaire du contenu, donc l'absence de modification · ne dit rien de la justesse métier des valeurs)
3. Pourquoi archive-t-on le **code** et pas seulement le rapport ? (sans le code, le rapport n'est pas rejouable ; l'archivage sans transformation documentée est un stockage, pas une preuve)
4. Un fichier nommé `CA_2025_final_final(2).xlsx` : trois défauts de cycle, un par mot. (« final » = absence de versionnage réel · « (2) » = écrasement implicite · pas de date = impossible de dater l'état ; le quatrième défaut, gratuit : on ne sait pas qui l'a produit ni à quelle source il correspond)
5. **Question ouverte** : on vous demande d'envoyer par e-mail la table clients complète pour « vérifier les téléphones ». Répondez en quatre phrases, en distinguant l'objet, le risque et la solution de rechange. (la question réelle est « les numéros sont-ils valides » → un échantillon pseudonymisé de 200 lignes ou un contrôle automatique (longueur, format, doublons) répond sans exposer 23 500 clients ; l'envoi d'une table nominative par messagerie est un incident potentiel ; je propose le contrôle chiffré, avec le nombre de numéros invalides, et je garde la preuve dans le journal)

M01.C07 — Les métiers de la donnée, la chaîne de valeur, et l'anatomie d'une analyse

Outil de ce chapitre : traitement de texte ou tableur pour la rédaction ; aucun logiciel de données. **Durée indicative :** 5 h. **Niveau :** N1.

L'idée du chapitre. Vous allez apprendre quatre outils (tableur, SQL, Python, Power BI). Ce n'est pas le classement qui compte dans un emploi, ni dans une mission. Ce qui compte, c'est de savoir **à quel moment de la chaîne** vous intervenez, **quelle question** vous traitez, et **quelle preuve** vous laissez. Ce chapitre ferme le socle : il nomme les métiers, montre la chaîne de valeur, démonte une analyse en six temps, et répond à la question que tout débutant pose trop tard — quel outil, pour quoi, et pourquoi celui-là.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Distinguer** les cinq métiers voisins (data engineer, data analyst, BI analyst, data scientist, responsable de la donnée) et décrire ce que chacun **livre**, pas ce qu'il « connaît » ;
2. **Situer** une mission dans la chaîne de valeur et identifier les deux maillons manquants quand un projet bloque ;
3. **Décomposer** une analyse en six temps (cadrage · données · préparation · mesure · interprétation · décision) et estimer le temps de chacun ;
4. **Écrire** une question analytique correcte à partir d'une demande floue, en respectant les cinq composants vus en M01.C02 ;
5. **Choisir** un outil pour une situation donnée et **justifier** le choix par trois critères : volume, relecteur, jouabilité ;
6. **Rédiger** la note d'une page qui fait passer d'un chiffre à une décision — l'exercice le plus noté de tout le parcours.

2. Pourquoi cette notion est importante

Situation 1 — le profil mal employé. Une PME recrute un « data scientist », lui donne un fichier de caisse et demande un prévisionnel de ventes. Le recrutement était trop cher, la donnée trop sale, la demande trop vague. La bonne embauche, ou la bonne mission, était un analyste capable de préparer les données trois semaines et de produire un tableau mensuel lisible. Connaître les métiers évite les erreurs de dimension — chez le client comme chez vous.

Situation 2 — l'analyse sans fin. Un projet traîne six semaines : personne n'a écrit ce que « succès » voulait dire. Le cadrage absent se paie en réunions. Les six temps de l'analyse existent pour être **chronométrés** : un cadrage d'une heure en sauve dix.

Situation 3 — l'outil mal choisi. Un rapport mensuel de 216 lignes est construit dans un outil de BI avec un modèle sémantique, des actualisations planifiées et six pages de documentation. Coût : trois jours. Un tableau dans un classeur, validé et verrouillé : quarante minutes, même résultat, moins de maintenance. À l'inverse, un fichier de 243 360 lignes retapé à la main dans un

tableur chaque semaine, c'est un accident programmé. Le bon outil est celui qui rend le service **au moindre coût de maintenance**, pas le plus impressionnant.

Dans les faits. Les offres d'emploi francophones « data analyst » demandent en majorité SQL et un outil de visualisation, avec le tableur en compétence implicite et Python comme différenciant ; les offres « BI » ajoutent le modèle sémantique et la gouvernance. À l'inverse, dans les PME sans service informatique, c'est le tableur et la qualité des fichiers qui font la différence — et c'est précisément ce que les modules 3 à 6 vous font travailler. La chaîne de valeur, elle, est la même partout ; ce qui change, c'est le nombre de personnes par maillon.

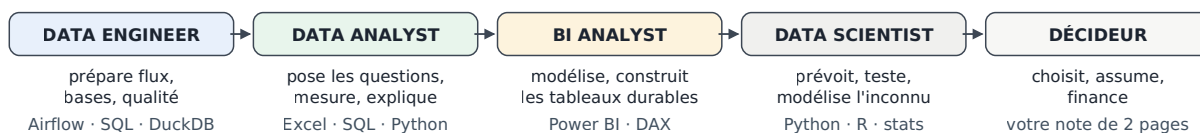
3. Explication simple

Imaginez une chaîne de montage de la décision :

1. **le data engineer** construit le convoyeur : il fait arriver la donnée, propre au sens technique, à l'heure, à la bonne taille ;
2. **l'analyste** prend une pièce, la mesure, dit ce qu'il constate et ce que ça veut dire ;
3. **le BI analyst** installe les instruments de mesure au mur : les tableaux que tout le monde regarde, avec des règles communes ;
4. **le data scientist** fabrique un dispositif qui anticipe la pièce suivante, avec un risque chiffré ;
5. **le décideur** tranche, engage de l'argent ou des gens.

Votre parcours vous rend **analyste** en priorité, **BI** à partir du module 13, capable de parler **data engineering** au module 24, et instruit en science des données aux modules 19 à 21. Ce n'est pas un empilement de casquettes : c'est la capacité à comprendre le voisin, ce qui est, en entreprise, la première cause de promotion.

Qui fait quoi, et les cinq questions à écrire avant de toucher un fichier



1. **Décision visée** — qu'est-ce qui changera selon la réponse ?
2. **Question analytique** — une phrase : unité (FCFA TTC), période (2025), périmètre (magasin 5), comparaison.
3. **Donnée nécessaire** — la table, le grain, la colonne, le fichier exact.
4. **Critère de réussite** — le chiffre attendu, la tolérance, qui valide, avant de calculer.
5. **Limites** — ce que le résultat ne dira pas : ici, pas de marge nette (aucun coût de structure dans le jeu).

Figure 7 — La chaîne de valeur, et le formulaire de cadrage resservi dans les 22 modules.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Data engineer ingénieur données	— Conçoit et exploite les flux, les bases, la qualité en amont.	Le confondre avec l'administrateur de base de données ; les deux se recouvrent parfois, les objectifs diffèrent.
Data analyst	Répond à des questions précises sur des données existantes, mesure, explique.	Se limiter à produire des visuels : sans interprétation ni décision, ce n'est pas de l'analyse.

Français — English	Définition simple	Piège à éviter
BI analyst — analyste décisionnel	Construit le modèle de données et les rapports durables, définit les indicateurs communs.	Construire un joli tableau sans gouvernance (qui le valide ? à quelle fréquence ?).
Data scientist — des données	Modélise, prédit, expérimente sur ce qui n'est pas encore observé.	Prédire sur un historique non nettoyé : le modèle hérite des défauts.
Responsable de la donnée — data steward	Garantit le sens, la qualité, la conformité des données et du dictionnaire.	Le voir comme un poste technique : c'est d'abord un rôle métier.
Chaîne de valeur — value chain	Suite d'étapes qui ajoutent chacune quelque chose de vérifiable.	Croire que la valeur est à la fin : elle est à chaque maillon, et le premier fragile casse tout.
Cadrage — scoping	Accord écrit sur la question, le périmètre, le délai, le critère de réussite.	Le faire oralement : un accord oral n'existe pas à la deuxième contestation.
Livrable — deliverable	Objet remis, nommé, daté, avec un destinataire.	Confondre livrable et activité : « j'ai fait des analyses » n'est pas un livrable.
Rejouabilité — reproducibility	— Capacité à refaire le même résultat plus tard, avec les mêmes entrées.	Un résultat obtenu par cinq clics non documentés n'est pas rejouable.
Modèle sémantique — semantic model	— Couche qui définit une fois pour toutes les mesures et les relations (Power BI, M13).	Recalculer la même mesure dans chaque rapport : quatre définitions, quatre chiffres.
Donnée de référence — master data	Référentiels partagés : clients, produits, magasins, calendriers.	Laisser chaque service sa version du référentiel.
Métier — business / functional owner	La personne qui répond aux questions de sens (« un retour, ça compte dans le CA ? »).	Trancher seul ces questions : c'est précisément le rôle du métier.

5. Cours approfondi

5.1 Les cinq métiers, vus par ce qu'ils livrent

Métier	Question typique	Livrables	Outils usuels	Échec typique si mal fait
Data engineer	« comment la donnée arrive-t-elle à l'heure, complète, non dupliquée ? »	flux planifiés, contrôles de qualité automatisés, documentation des schémas	SQL, Python, DuckDB, orchestrateurs, bases	des rapports justes... sur des données de la semaine dernière
Data analyst	« que disent les données sur cette question précise ? »	note d'une page, tableau de bord ponctuel, requêtes, fichiers propres, journal	tableur, SQL, Python	des chiffres vrais mais inutiles, ou des conclusions hors périmètre
BI analyst	« comment faire que tout le monde voie le même chiffre, tous les lundis ? »	modèle sémantique, mesures documentées, rapports publiés, gouvernance des accès	Power BI, DAX, SQL, sécurité par niveau de ligne	deux directions avec deux totaux du même indicateur
Data scientist	« que peut-on anticiper, et avec quel risque ? »	modèles, validations, bornes d'incertitude, note de limites	Python/R, statistiques, jeux d'entraînement	un modèle brillant, inexplicable, non utilisé
Responsable de la donnée	« que veut dire ce champ, qui le déclare, combien de temps on le garde ? »	dictionnaire, règles de qualité, politique de conservation	tableur, MDM, gouvernance	référentiels divergents, chacun « sa » version du client

Trois lectures utiles :

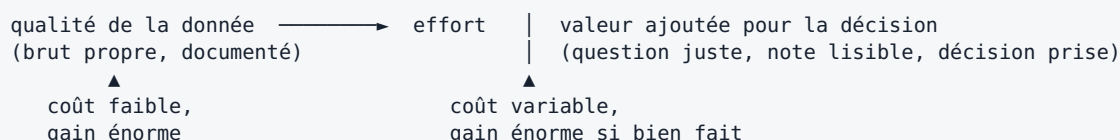
- les frontières sont **porositaires** : dans une PME, une seule personne occupe trois cases, ce qui est exactement pourquoi ce manuel enseigne la chaîne complète ;
- les **compétences transverses** qui augmentent un salaire ne sont pas des outils : écrire clairement, poser une question, documenter ;
- **le décideur n'est pas un métier de la donnée**, c'est votre client. Apprendre à lui parler est une compétence du module 12, pas un bonus.

Définition. Chaîne de valeur — value chain — la suite d'étapes par lesquelles une donnée devient une décision qui rapporte ou qui épargne. Elle sert d'outil de diagnostic : quand un projet échoue, on situe l'étape cassée, et l'on constate que la casse est plus souvent en amont — collecte, sens, périmètre — qu'en aval, dans les calculs.

Attention. Confondre les métiers produit deux échecs lents : demander à un analyste de réparer l'alimentation du fichier, ou à un ingénieur de trancher si un retour client compte dans le chiffre d'affaires. La bonne question n'est pas « qui est le plus compétent » mais « qui est responsable de ce maillon, et qui valide le sens ».

5.2 La chaîne de valeur, et où l'on gagne réellement de l'argent

La valeur ne croît pas linéairement le long de la chaîne : elle croît aux deux bouts.



Le premier bout — rendre la donnée correcte — est le moins visible et le plus rentable : un fichier bien typé fait gagner trois semaines en aval (modules 3 à 8). Le second bout — rendre le résultat compréhensible — est le plus rentable en carrière : un décideur qui comprend en une minute décide, et revient vous demander une deuxième question. Le milieu (jolis graphiques, tableaux intermédiaires) est ce que l'on vous demandera de faire ; c'est ce que l'on oubliera le plus vite. Cette asymétrie explique le calibrage du parcours : huit modules sur neuf parlent de la préparation et de la mesure, un seul de la présentation pure (M18) — et il est décisif, parce qu'il transforme le travail en décision.

Définition. Cadrage — scoping — l'accord écrit sur la question, le périmètre, le délai et le critère de réussite. Six lignes suffisent, et il n'est pas négociable : un cadrage oral se réécrit dans la tête de chaque participant, et l'écart réapparaît à la livraison, au pire moment, devant le commanditaire.

5.3 L'anatomie d'une analyse : six temps, avec leurs durées

Temps	Ce qu'on produit	Durée raisonnable (petit sujet)	Signe que c'est fini
1. Cadrage	la question analytique + critère de réussite + destinataire	10 % du temps	le demandeur a validé par écrit la question et le délai
2. Repérage des données	inventaire des fichiers, dictionnaire, accès	10 %	vous savez ce qui manque et vous l'avez dit
3. Préparation	fichier propre, types, clés, contrôles	40 %	le total de contrôle tombe, le journal est écrit

Temps	Ce qu'on produit	Durée raisonnable (petit sujet)	Signe que c'est fini
4. Mesure	agrégats, comparaisons	statistiques, 20 %	les chiffres sont stables sous un changement de méthode raisonnable
5. Interprétation	ce que ça veut dire, ce que ça ne dit pas	10 %	un non-spécialiste comprend la phrase sans voir le tableau
6. Décision suivie	recommandation, échéance, indicateur	acteur, 10 %	une ligne dans l'agenda de quelqu'un d'autre que vous

Répartissez mentalement vos travaux passés sur ces six lignes : si 90 % du temps est allé au temps 4 (produire des chiffres) et rien au temps 1, vous avez reproduit l'erreur du plus grand nombre, y compris parmi les titulaires d'un master. Le temps 3 à 40 % n'est pas un échec, c'est le signe que le travail est sérieux — à condition de l'**annoncer** au commanditaire (« la préparation prend la moitié du délai, parce que ... »), sinon on vous croira lentement.

Définition. Livrable — deliverable — un objet remis, nommé, daté, avec un destinataire identifié. Le mot contient le verbe « remettre » : une activité non livrée n'existe pas pour l'entreprise, même si elle a été difficile.

5.4 Passer d'une demande floue à une question correcte : l'algorithme complet

C'est la compétence évaluée de ce chapitre. Formalisons-la en six questions, dans cet ordre :

1. **Qui lit, et pour quoi faire ?** — destinataire + décision. Sans réponse : on ne commence pas.
2. **Quelle est la phrase du demandeur, mot à mot ?** — relever chaque mot non mesurable (« bien », « trop », « récemment », « la plupart »).
3. **Que veut-il vérifier ou arbitrer ?** — la décision réelle, souvent différente de la formulation.
4. **Quelle unité, quelle période, quel périmètre ?** — FCFA HT ou TTC, retours inclus ou non, magasins concernés, année pleine ou partielle, clients hors comptoir.
5. **Quelle comparaison, quelle règle ?** — objectif, N-1, médiane, classement. Sans comparaison, pas de connaissance (M01.C02).
6. **Qu'est-ce qui prouvera que j'ai réussi ?** — valeur attendue, tolérance, nom du valideur, date.

Un cas complet, avec le fil rouge. **Demande** : « on perd des clients, regarde ça ».

- Qui lit ? Le directeur commercial, pour arbitrer un budget de fidélisation avant le 15/11.
- Mots non mesurables : « perd », « ça ».
- Décision réelle : faut-il financer une relance, et sur qui ?
- Unité / période / périmètre : un client est « perdu » s'il n'a **aucun** achat entre deux dates ; on regardera 2025 et 2026 partiels, tous magasins sauf dépôt ; client identifié uniquement (le comptoir, `id_client = 0`, est hors jeu : on ne peut pas dire qu'un client non identifié est parti).
- Comparaison : taux de perte par type de client en 2025 vs la même règle en 2024.
- Critère de réussite : le nombre de clients « actifs en N, absents en N+1 » doit être recalculable par une requête de dix lignes, et le total de contrôle doit redonner le nombre de clients distincts connus (372 sur l'extrait de l'atelier).

Question finale : « Entre 2024 et 2025, combien de clients identifiés ayant acheté au moins une fois en 2024 n'ont rien acheté entre le 01/07/2025 et le 31/12/2025, tous magasins sauf le dépôt, et ce taux varie-t-il selon `type_client` ? »

Notez le gain : la question initiale était un motif d'alarme ; la question finale est un **chiffre**, avec ses limites écrites (exclusion du comptoir, 18 lignes sans identifiant dans l'extrait, 1 880 villes manquantes dans `clients`). Cinq minutes d'écriture, et le travail devient possible.

À retenir. Une question analytique correcte est une question **réfutable** : deux analystes qui la traitent indépendamment doivent obtenir le même chiffre. Testez toujours par là.

5.5 Quel outil choisir, et pourquoi : la grille définitive du socle

Quatre questions dans l'ordre, puis l'outil en découle. Nous les reprendrons en M08.C01 et M13.C01, mais décidez-les maintenant, sinon vous choisirez par goût.

Q1. Quelle taille, et où est la donnée ?

Situation	Outil par défaut	Motif
moins de 50 000 lignes, un fichier	tableur (Excel ou LibreOffice)	ce que tout le monde peut ouvrir ; zéro dépendance
50 000 à 5 millions de lignes, un fichier	DuckDB ou Python/pandas	requêtes sur fichiers plats sans serveur ; le CSV devient interrogeable
la donnée est dans une base de l'entreprise	SQL	on calcule sur place, on ne déplace pas ; la requête est la preuve
plus de 10 millions, ou plusieurs sources à assembler	SQL + étape de préparation (Python ou M24)	le tableur sature, le modèle se construit ailleurs
JSON, logs, fichiers irréguliers	Python	seul endroit où l'on écrit la règle de parsing une fois pour toutes

Q2. Qui relit, et comment ?

Relecteur	Outil
une personne non technique, une fois	tableur + note, ou PDF
une direction, chaque lundi	Power BI (ou rapport paginé si l'impression est obligatoire — M19)
une équipe technique qui doit rejouer	script + requêtes versionnées
un auditeur, dans deux ans	tout ce qui laisse un journal : requête SQL ou script, jamais une session de clics

Q3. À quelle fréquence l'opération se répète-t-elle ? Une fois → le plus rapide à écrire (tableur). Chaque mois → le plus rapide à **exécuter** (script ou requête). Chaque jour avec plusieurs utilisateurs → BI avec actualisation planifiée.

Q4. Quel risque financier, et donc quel niveau de preuve ? Un chiffre publié → définition écrite + contrôle de totaux + version figée. Un chiffre interne approximatif → le tableur suffit, en le disant.

Tableau de synthèse des quatre outils du parcours, tel qu'il doit être retenu à la fin du module :

Outil	Force	Limite	Ne l'utilisez pas pour
Tableur (Excel, LibreOffice)	immédiat, vérifiable par cellule, universel	cellule le schéma se dégrade, l'auditabilité est faible, ça sature vite	un traitement répété sur de gros volumes
SQL	demande déclarative (« je veux quoi », pas « comment »), calcule sur place, réutilisable	ne nettoie pas bien les fichiers sales, ne fait pas de jolis rapports	un fichier de 400 lignes à trier une fois

Outil	Force	Limite	Ne l'utilisez pas pour
Python (pandas)	maîtrise totale de la transformation, rejouable, bibliothèque statistique	il faut écrire et maintenir du code ; le relecteur doit savoir lire du code	un besoin de validation visuelle immédiate par un non-technicien
Power BI	modèle sémantique partagé, DAX, actualisation, distribution, sécurité par niveau de ligne	courbe d'apprentissage du modèle, licence payante pour le partage (Power BI Pro : 14 \$/utilisateur/mois — lecture possible gratuitement sur capacité Fabric F64+)	un fichier à corriger : on ne nettoie pas dans le rapport, on nettoie en amont

Deux alternatives à connaître, pour répondre honnêtement quand un client les mentionne : **Looker Studio** (gratuit, lié à l'écosystème Google, mais sans sécurité par niveau de ligne native ; l'offre payante ≈ 9 \$/utilisateur/mois) et **Tableau** (redoutable en visualisation ; Tableau Public est gratuit mais **public**, et « Desktop » nécessite une licence — une version gratuite d'usage local a été annoncée en 2026, à vérifier au moment de la décision). En contexte libre et hébergé : **Metabase** et **Superset**. Le module 19 compare ces outils avec les critères de ce tableau ; ici, retenez une seule chose : **le choix d'outil ne précède jamais la question analytique**.

Conseil professionnel. Quand on vous demande « tu fais ça sur quoi ? », répondez par la contrainte, pas par l'outil : « la donnée arrive en CSV irréguliers de 200 000 lignes, et la direction relit chaque lundi, donc préparation en Python et restitution dans Power BI ». En trois secondes, vous venez de justifier votre choix devant un non-technicien et devant un pair — et personne ne vous demandera plus jamais « pourquoi pas Excel ? ».

Définition. Modèle sémantique — semantic model — la couche d'un outil de décision où les mesures et les relations sont définies une fois pour toutes. Sans lui, chaque utilisateur recalcule le chiffre à sa façon : l'outil censé unifier le vocabulaire devient une machine à produire des désaccords, proprement présentés.

Attention. Vérifier « à la main » le chiffre d'un tableau de bord en le recalculant dans un tableur crée une deuxième vérité, et souvent un arbitrage impossible trois mois plus tard. On vérifie une fois, en écrivant le contrôle, la date et l'écart constaté dans le journal ; ensuite on corrige la source, pas la note.

5.6 Anatomie d'une note de résultat : le format qui fait décider

Six rubriques, une page. Vous l'avez découverte en M01.C02, la voici normative, parce qu'elle sert jusqu'à la soutenance (M22) :

- | | |
|---------------------------|---|
| 1. DÉCISION VISÉE | 2 lignes : qui décide quoi, avant quelle date. |
| 2. QUESTION TRAITÉE | 2 lignes : la phrase complète, avec unité, période, périmètre. |
| 3. RÉPONSE | 3 lignes + un tableau : le chiffre en gras, la comparaison, le sens. |
| 4. MÉTHODE | 4 lignes : source (fichier, version), règle de calcul, périmètre, exclusions. |
| 5. CE QUE CELA NE DIT PAS | 3 lignes : limites, incertitudes, ce qu'il manque. |
| 6. RECOMMANDATION | 2 lignes : acte, responsable, échéance, indicateur de suivi. |

Règles d'écriture, non négociables dans ce manuel : chiffre **avec unité** à chaque fois ; pas d'acronyme non défini ; un seul adjectif par note maximum ; le tableau de la rubrique 3 doit tenir sur un tiers de page ; la rubrique 5 ne se saute jamais, même (surtout même) quand elle affaiblit le résultat. Une note sans rubrique 5 est, techniquement, un document invérifiable ; nous l'avons vu, c'est le seul défaut qui coûte un poste dans les grands scandales de chiffres.

Définition. Rejouabilité — reproducibility — pouvoir refaire le même résultat plus tard, avec les mêmes entrées, sans avoir à s'en souvenir. Elle se mérite : entrées identifiées, transformations notées dans l'ordre, aucune retouche manuelle silencieuse. C'est ce qui distingue une chaîne de traitement d'un coup de main chanceux.

Boîte à outils. Le minimum de la chaîne d'analyse, et à quoi chacun sert : l'**Explorateur de fichiers** pour retrouver et dater · un **éditeur de texte** pour lire un fichier sans laisser un tableur le re-taper · le **tableur** jusqu'à quelques dizaines de milliers de lignes, pour un chiffre à défendre · **DuckDB** (gratuit, un fichier, aucun serveur) pour compter et joindre sur des fichiers plus gros que la mémoire n'aime le faire · **Power BI Desktop** (gratuit pour créer, payant pour partager) quand le même besoin revient chaque semaine et doit être consulté par d'autres. Un outil de plus = une raison de plus de le justifier dans la note.

6. Exemple concret : le cas complet, de la demande à la note

Contexte. 12 octobre 2026. Le DG de Sahel Distribution croise un commercial : « depuis la hausse des prix en février, les vendeurs ne vendent plus rien, ils tournent en rond ». Il vous écrit : « regarde les ventes de Bationo ».

Temps 1 — Cadrage (25 min, dont 10 avec l'intéressé). Trois questions au DG : sur quoi vous basez « ne vendent plus rien » (le ressenti d'un commercial, le classement, la comparaison à l'année dernière ?) ; quelle décision (accompagner, recadrer, sanctionner, réallouer le secteur) ; qui relit. Réponse obtenue : « je veux savoir si je dois le faire accompagner ou lui retirer le secteur BTP ». Cette phrase change tout : nous ne mesurons plus « les ventes », nous mesurons **une performance commerciale imputable à la personne**.

Temps 2 — Repérage (30 min). Sources : `ventes_propres.csv` (240 000 lignes), `vendeurs.csv` (22), `magasins.csv` (6), `objectifs_de_ca.csv` (218, avec deux mois manquants — annoncé). Limites trouvées : les objectifs ne sont pas mensuels pour tous les mois ; la table `clients` n'a pas de date de départ de relation ; aucune donnée sur les prix « avant/après février » hormis les montants de vente.

Temps 3 — Préparation (2 h). Filtre : `id_vendeur = 9` (Adama Bationo, magasin 5). Grain : article. Agrégation : par mois et par catégorie, CA TTC, nombre de tickets distincts, quantité. Contrôles : le total du périmètre doit redonner, sur l'extrait de l'atelier, **11 370 350 FCFA** ; le nombre de tickets distincts doit être cohérent avec le total des quatre vendeurs de l'extrait ($11\,370\,350 + 9\,423\,281 + 8\,483\,019 + 6\,796\,535 = 36\,073\,185$ FCFA ✓ égal au total de l'extrait). Écriture du journal.

Temps 4 — Mesure (45 min). Classement des quatre vendeurs de l'extrait 2025 : Bationo 11 370 350 · Sankara 9 423 281 · Ouédraogo 8 483 019 · Ilboudo 6 796 535 FCFA. Bationo est **premier**, et détient 31,5 % du CA de l'échantillon. Comparaison mois par mois impossible sur cet extrait sans le file d'attente d'un extrait comparable pour les autres vendeurs — noté comme limite (l'extrait est de 40 lignes par mois, pas une année complète).

Temps 5 — Interprétation (20 min). Ce que les données disent : dans l'extrait de 480 lignes, le vendeur mis en cause est le premier en CA. Ce qu'elles ne disent pas : l'évolution février-septembre, l'effet prix, la comparaison aux objectifs mensuels (2 mois manquants), ni la satisfaction client. L'hypothèse du DG (« il ne vend plus ») n'est pas compatible avec le seul chiffre disponible ; mais l'extrait ne prouve rien sur la période citée.

Temps 6 — Décision (la note, dix lignes). Recommandation : ne pas statuer sur le secteur BTP avant d'avoir le fichier annuel complet du vendeur et les objectifs mensuels manquants ; prévoir un point de 30 min avec le directeur commercial le 26/10 sur la base du CA mensuel par

vendeur 2025-2026, retours inclus, et décider ensuite d'un accompagnement. Indicateur de suivi : CA mensuel par vendeur et nombre de tickets, écart à l'objectif.

Leçons à écrire noir sur blanc dans votre cahier. Un cadrage de 25 minutes a transformé « regarde les ventes de Bationo » en une décision conditionnée à deux données manquantes. La mauvaise exécution du même sujet (filtrer, faire un graphique, conclure « il vend très bien ») aurait produit une affirmation fausse fondée sur un échantillon, dans un dossier RH. La donnée n'a pas menti : c'est l'absence de cadrage qui aurait fait mentir l'analyste.

7. Démonstration pas à pas : le formulaire de cadrage, rempli en 18 minutes

Le formulaire ci-dessous est utilisé dans **tous** les projets du manuel, du module 1 au module 22. Apprenez-le par cœur : en stage comme en mission, c'est le document qui fait la différence entre « celui qui sait faire des tableaux » et « celui à qui on confie un sujet ».

Formulaire de cadrage — projet M01.P

0. Date · auteur · destinataire	2026-09-17 · a.minani · direction commerciale
1. Demande reçue (verbatim)	« on donne trop de remises, tu regardes ? »
2. Décision que la réponse doit éclairer	resserrer la grille de remise avant le
01/12/2026	
3. Mots non mesurables relevés	« trop », « regardes »
4. Question analytique (5 composants)	voir ci-dessous
5. Données nécessaires (tables, fichiers, accès)	ventes, clients, remises_manuelles, produits
6. Données manquantes (annoncées au cadrage)	motifs de remise non codés sur 2026, objectifs 2
mois manquants	
7. Critère de réussite	total TTC de l'extrait = 36 073 185 FCFA ;
clients distincts = 372	
8. Comparaison retenue	remise moyenne par type de client, 2025 vs 2024
9. Périmètre	magasin 5 ; retours inclus ; id_client = 0
exclu de la partie « par client »	
10. Limites et risques d'erreur	18 lignes sans client ; extraits ≠ population ;
pas de coût de structure	
11. Outil choisi + motif	tableur pour la préparation (480 lignes) ; note
écrite en 1 page	
12. Livrables et délai	fichier propre + journal + note, rendu le 19/09
avant 17 h	
13. Validation	nom du valideur, date

La ligne 4, développée : « En 2025, au magasin 5, la remise moyenne et la part de tickets à remise > 5 % varient-elles selon le type de client, et un plafond de remise à 5 % aurait-il réduit le CA TTC de plus de 1 % pour les vingt clients les plus remisés ? »

Le remplissage, en méthode. Les lignes 1 à 3 s'écrivent **avec** le demandeur (en face, ou par écrit, pas de mémoire). Les lignes 5 et 6 s'écrivent **après avoir ouvert le dossier de données** : ne jurez jamais qu'une donnée existe avant d'avoir vu ses colonnes. La ligne 7 est négociée : c'est votre bouée, elle prouve votre travail. Les lignes 10 à 13 s'écrivent seules, puis se relisent à voix haute : si une phrase ne se comprend pas sans le contexte de votre tête, elle est à réécrire.

Auto-évaluation du formulaire, en six points : toute ligne remplie (1) ; aucune ligne 3 non résolue sans trace (2) ; la ligne 4 est réfutable par deux personnes (3) ; la ligne 5 cite des fichiers existants (4) ; la ligne 7 contient un nombre attendu (5) ; la ligne 10 est lue par le demandeur (6). Cinq sur six est un bon cadrage ; trois ou moins, on recommence, car la suite est perdue.

8. Erreurs fréquentes

Symptôme	Cause	Correction
Le livrable est livré, la décision ne vient pas	Le destinataire n'a pas été nommé au cadrage	Rubriques 0 et 2 obligatoires ; demander au demandeur de valider la question par écrit
Le rapport est bon, mais personne ne le comprend	La note commence par la méthode	Ordre imposé : décision, question, réponse — la méthode va en rubrique 4
« 12 % de plus » sans comparaison	Chiffre non situé	Toujours préciser : par rapport à quoi, sur quelle période, dans quelle unité
Le résultat change quand on repose la question	Question ambiguë non négociée	Rejouer le contrôle du §5.4 : deux traitements indépendants doivent converger
L'analyse « n'en finit pas »	Critère de réussite absent	Rubrique 7 ; une date de rendu ; le surplus d'analyse se fait après la décision
Conclusions fortes sur échantillon	Extrait pris pour la population	Écrire la nature de l'extrait dans la note (nom + taille + période)
Le métier refuse le chiffre	Définition divergente (retours, HT/TTC)	Rubrique 9 négociée avec le responsable métier, pas supposée

9. Bonnes pratiques professionnelles

- [] Le formulaire de cadrage est rempli **avant** d'ouvrir le fichier, et validé par écrit.
- [] Un livrable = un nom de fichier daté + une note d'une page.
- [] Les données manquantes sont annoncées au début, pas en fin de travail.
- [] Le choix d'outil se justifie par trois critères : volume, relecteur, jouabilité.
- [] Une limite écrite affaiblit le chiffre et renforce l'analyse.
- [] On ne conclut pas sur un extrait sans le dire, jamais.
- [] Une recommandation sans responsable ni échéance n'est pas une recommandation.
- [] Le temps de préparation (40 %) est annoncé comme tel au commanditaire.
- [] Après chaque projet, une ligne : « la décision a-t-elle été prise ? »

10. Exercice guidé — trois demandes, trois cadrages

Nous en menons un, les deux autres vont avec vous.

Demande A : « fais-moi un tableau de bord — dashboard, dans le jargon : une page d'indicateurs que l'on consulte sans ouvrir les fichiers — des ventes ».

- Rubrique 2 (décision visée) : **inconnue**. On demande un objet, pas une décision. Question à poser : « quand vous le regardez, que savez-vous que vous ne saviez pas, et qu'est-ce que vous changez ? ».
- Réponse professionnelle type : « je peux le faire, mais un tableau de bord sans décision associée ne sera pas ouvert après deux semaines. Donnez-moi la décision, je vous dis quels indicateurs la servent — peut-être trois chiffres dans un e-mail. »
- Cadrage obtenu (exemple) : décision = arbitrer les réapprovisionnements hebdomadaires du magasin 5. Trois indicateurs suffisent : CA par catégorie sur 4 semaines glissantes, rupture estimée (stock théorique vs physique, `stocks_quotidiens`), taux de remise.

Demande B (avec vous) : « pourquoi la marge baisse ? ».

- Rubrique 3 (mots non mesurables) : « baisse » (depuis quand, de combien ?).

- Rubrique 4, question possible : « entre 2023 et 2025, la marge brute par catégorie a-t-elle baissé, et cet écart s'explique-t-il par le prix moyen HT, par le taux de remise moyen, ou par le mélange des catégories ? ».
- Rubrique 5 : `ventes` + `couts_achat` (1 694 lignes, prix d'achat par période — donc la marge est calculable). Rubrique 6 : aucun coût de structure dans le jeu → **la marge nette n'est pas calculable**, à écrire dans la note dès le début. Rubrique 8 : comparaison par catégorie, 2023 vs 2025 (2026 partiel : exclu de la comparaison annuelle, ou ramené à une base mensuelle, et on le dit).
- C'est le fil rouge du parcours : ce qui était posé comme « problème de prix » se révélera problème de **mélange** et de **ruptures**. Vous le démontrerez en M13-M16 ; ici, vous apprenez à écrire le cadrage qui le rend démontrable.

Demande C (vous) : « est-ce qu'on a une bonne saisonnalité ? ». Rubriques à remplir seules : 2, 3, 4, 6, 7, 10. Contrôles : votre question doit contenir une période et une unité ; « bonne » doit disparaître ; la rubrique 10 doit mentionner que 2026 est partiel (janvier à août) et que l'extrait de l'atelier contient 40 lignes par mois. Corrigez indicatif au §12.

11. Exercices autonomes

Exercice 7.1 (★) — Métiers. Quatre phrases d'annonces d'emploi, classer dans les cinq métiers : « construire les flux d'ingestion et la qualité en base » · « produire les analyses mensuelles et les recommandations commerciales » · « définir les mesures du modèle sémantique et publier les rapports » · « estimer la demande à 12 semaines avec un intervalle de confiance ». (data engineer · data analyst · BI analyst · data scientist ; justifiez par le livrable, pas par l'outil)

Exercice 7.2 (★) — Six temps. Un projet de 20 jours : répartissez les jours selon le tableau du §5.3 et justifiez deux écarts possibles. ($2 \cdot 2 \cdot 8 \cdot 4 \cdot 2 \cdot 2$; écarts : préparation 10 si le brut est très sale, mesure 3 si la demande est une seule agrégation)

Exercice 7.3 (★★) — Cadrage complet. Appliquez le formulaire du §7 à la demande « est-ce que le dépôt central sert à quelque chose ? » (attendu : décision visée = maintenir ou non le dépôt ; mots non mesurables = « sert » ; périmètre = flux du dépôt, `id_magasin = 6`, sans ventes au comptoir ; limites = pas de coûts de structure, pas de date de création du dépôt avant 2023... ; un critère de réussite chiffré)

Exercice 7.4 (★★) — Choix d'outil. Cinq situations, justifiez en une phrase par situation : (i) 6 magasins × 36 mois, note mensuelle au DG ; (ii) 243 360 lignes à nettoyer chaque nuit ; (iii) vérifier un total unique ce matin ; (iv) 30 000 lignes de logs JSON à transformer en table ; (v) partager 4 indicateurs à 12 commerciaux, sans qu'ils puissent voir les données des autres. (i tableur ou Power BI selon la relecture ; ii Python ou SQL + tâche planifiée, le tableur est exclu ; iii tableur ; iv Python ; v Power BI avec sécurité par niveau de ligne — et une licence Pro par utilisateur qui publie, les lecteurs pouvant être gratuits selon la capacité, M13/M18)

Exercice 7.5 (★★) — Piège d'échantillon. Sur l'extrait de l'atelier, Bationo est premier avec 11 370 350 FCFA (31,5 % de l'extrait). Rédigez la phrase qui rend ce classement honnête, et la manipulations à faire sur le fichier complet pour conclure. (attendu : préciser extrait de 480 lignes (40/mois), magasin 5, 2025 ; recalculer sur `ventes_propres.csv` avec le même filtre vendeur et la même période, puis comparer aux quatre vendeurs du magasin et à l'ensemble des 22)

Exercice 7.6 (★★★) — La note. À partir des chiffres du §6, rédigez la note de six rubriques (10 lignes maximum), et faites-la relire par quelqu'un qui ne connaît pas le dossier : s'il ne peut pas répéter la décision recommandée, réécrivez la rubrique 3. (attendu : pas d'adjectif « bon/mauvais », unité FCFA partout, rubrique 5 obligatoire, recommandation avec date et responsable ; toute mention de « la marge nette » est une faute, elle n'est pas calculable)

12. Correction détaillée

Exercice 7.1. « Flux d’ingestion et qualité en base » → **data engineer** (livrable : des flux, pas une analyse). « Analyses mensuelles et recommandations » → **data analyst** (livrable : note récurrente et conseil). « Mesures du modèle sémantique et publication » → **BI analyst** (livrable : un modèle partagé et des rapports). « Estimation à 12 semaines avec intervalle de confiance » → **data scientist** (livrable : un modèle avec son incertitude). Un cinquième métier est sous-représenté dans les annonces mais existe : le **data steward**, qui livre le dictionnaire et les règles de qualité — vous le rencontrerez au module 21.

Exercice 7.2. 2 jours de cadrage (dont la validation écrite), 2 de repérage, 8 de préparation, 4 de mesure, 2 d’interprétation, 2 de restitution et suivi. Écart n° 1 : si les fichiers livrés sont au format du chapitre 4 (encodage, colonnes texte, chapeaux), la préparation passe à 10 jours — et il faut le dire **au cadrage**, sinon on vous reproche un retard qui est une qualité. Écart n° 2 : si la demande est une seule agrégation stable, la mesure tombe à 2 jours, mais l’interprétation monte à 3, parce que « un chiffre sans repère » doit être construit (objectifs, N-1) et non simplement calculé.

Exercice 7.3. Exemple de réponse complète : décision visée = arbitrer le maintien du dépôt à K... ou son externalisation, avant le budget 2027 · mot non mesurable = « sert » · question analytique = « en 2025, le dépôt (id_magasin 6) a-t-il couvert les magasins en délai et en quantité, et son activité évite-t-elle des ruptures — mesurées par les jours où le stock physique du magasin est inférieur au seuil dans `stocks_quotidiens` ? » · tables = `stocks_quotidiens` (60 000 lignes), `ventes`, `magasins` · manquantes = coûts d’exploitation du dépôt (absents du jeu : pas de coûts de structure) · critère de réussite = le nombre de jours de rupture par magasin doit se recouper avec la table de stocks pour 2025 à ± 1 jour · limites = la rupture est **observée** dans les écarts de stock, pas déclarée ; l’absence de coûts interdit de conclure sur la rentabilité, seulement sur le service · outil = tableur pour l’agrégation mensuelle, Power BI non justifié pour un usage unique. Le point de barème le plus souvent perdu : conclure sur la rentabilité avec un jeu qui ne contient pas de coûts.

Exercice 7.4. (i) **tableur** si la note est un document statique validé, **Power BI** si elle doit être consultée en libre-service ; le motif est le relecteur, pas le volume (216 lignes). (ii) **Python** (ou SQL si la base existe) : le traitement se rejoue chaque nuit, donc le code est le seul livrable soutenable. (iii) **tableur** : deux minutes, une vérification visuelle, une cellule de contrôle. (iv) **Python** : parsing de JSON, puis écriture d’une table plate documentée. (v) **Power BI**, à cause de la sécurité par niveau de ligne (RLS) : sans elle, il faudrait 12 fichiers ; le motif réel est la gouvernance des accès. Un devoir qui répond « Power BI, c’est plus moderne » n’a pas justifié, il a déclaré une préférence.

Exercice 7.5. Phrase honnête : « Sur l’extrait de 480 lignes utiles (magasin 5, 2025, 40 lignes retenues par mois), Adama Bationo arrive premier avec 11 370 350 FCFA, soit 31,5 % du total de l’extrait ; cet extrait ne couvre qu’une fraction des ventes réelles, donc ce classement est indicatif. » Manipulation : recalculer sur `ventes_propres.csv` (240 000 lignes) avec les mêmes filtres (année 2025, magasin 5, retours inclus), grouper par `id_vendeur`, compter aussi les tickets distincts et le CA par ticket, et **ne pas** comparer aux 22 vendeurs sans signaler qu’ils sont répartis sur 6 points de vente. Le test de robustesse : le classement change-t-il si l’on exclut les 7 lignes à quantité aberrante ? S’il change, ce n’est pas un classement, c’est un artefact de nettoyage.

Exercice 7.6. Modèle de note :

1. Décision visée : maintenir ou non le secteur BTP chez A. Bationo, arbitrage du 26/10.
2. Question : CA TTC et tickets, par vendeur, magasin 5, extrait 2025 de 480 lignes, retours inclus.
3. Réponse : sur l'extrait, A. Bationo est 1er avec 11 370 350 FCFA (31,5 % du total), devant
S. Sankara 9 423 281 · B. Ouédraogo 8 483 019 · M. Ilboudo 6 796 535 FCFA.
4. Méthode : 01_brut/...recu2026-09-17.csv → propre (480 lignes, 9 doublons retirés, montants
convertis) ; total de contrôle 36 073 185 FCFA.
5. Ce que cela ne dit pas : extrait ≠ année complète ; aucun objectif mensuel pour tous les mois (2
manquants dans objectifs_de_ca.csv) ; pas de marge nette (coûts de structure absents).
6. Recommandation : ne pas statuer le 26/10 ; livrer le calcul 2025-2026 complet par vendeur avant,
responsable direction commerciale, indicateur = CA mensuel et tickets par vendeur.

Barème : 3 pts unité et période explicites partout · 3 pts exactitude des quatre chiffres et du total de contrôle · 3 pts rubrique 5 complète (extrait + objectifs manquants + marge nette impossible) · 2 pts recommandation datée avec responsable · 2 pts lisibilité (10 lignes). Un devoir qui écrit « Bationo est un bon vendeur » perd les 3 points de la rubrique 3 : ce n'est plus un résultat, c'est une opinion.

13. Mini-projet M01.P7 — « Trois cadrages pour un même sujet » (50 min)

Sujet unique : « faut-il ouvrir un sixième magasin ? » Vous produisez trois cadrages différents du même sujet, chacun sur le formulaire du §7 :

1. pour le **directeur général** (arbitrer un investissement) ;
2. pour le **responsable des achats** (anticiper le stock d'ouverture) ;
3. pour un **banque/bailleur** (dossier de financement).

Contraintes : les trois questions analytiques doivent être distinctes, mais reposer sur les mêmes données disponibles du socle ; chacune annonce explicitement ce que le jeu ne permet pas de calculer ; le choix d'outil diffère au moins sur deux des trois, avec justification. Livrables : trois formulaires d'une page + une synthèse de cinq lignes qui liste **les questions communes non résolues**.

Barème : 4 pts distinctivité des trois questions (un correcteur doit pouvoir les reconnaître sans lire les en-têtes) · 4 pts réalisme des données citées (les fichiers et colonnes existent) · 4 pts limites annoncées (marge nette, coûts, année 2026 partielle) · 4 pts justification des outils · 4 pts synthèse. Ce mini-projet prépare directement le **projet M01.P**, dont il constitue le livrable (2) : « trois questions que l'on peut lui poser ».

14. Résumé du chapitre

Cinq métiers, une chaîne : flux → mesure → modélisation → prédiction → décision. Vous êtes formé pour le deuxième maillon en priorité, et pour parler à tous les autres. Une analyse se décompose en six temps dont la préparation est le plus long (40 %) et le cadrage le plus rentable (10 % du temps, 90 % du risque évité). Une demande floue devient une question réfutable en six questions, dont la dernière — « qu'est-ce qui prouvera que j'ai réussi ? » — est la plus négligée. Le choix d'outil découle de trois critères : volume, relecteur, rejouabilité — et se justifie par la

contrainte, jamais par la mode. Enfin, une note de six rubriques, où la ligne « ce que cela ne dit pas » est obligatoire : c'est elle qui vous rend crédible à long terme.

15. À retenir

À retenir. 1. **On juge un travail de la donnée à ses livrables** (note, fichier propre, journal, modèle), pas à ses outils. 2. **Six temps, et le cadrage d'abord** : 10 % du temps, mais la décision de savoir si le travail était utile. 3. **Une question analytique est réfutable** : deux traitements indépendants, même chiffre. 4. **Outil = volume + relecteur + jouabilité**. Tableur pour le ponctuel vérifiable, SQL pour le gros et le partagé, Python pour l'irrégulier, Power BI pour le publié et le gouverné. 5. **Écrivez les limites** : un chiffre sans rubrique « ce que cela ne dit pas » est une opinion chiffrée.

16. Évaluation formative (auto-correction, 12 min)

1. Un responsable dit « je veux un outil moderne ». Quelle est la question à poser en retour, et pourquoi ? (« quelle décision ce nouvel outil doit-il rendre meilleure ou plus rapide ? » — sans décision identifiée, le projet n'a pas de critère de réussite et ne sera pas adopté)
2. Citez les deux rubriques du formulaire qu'il est interdit de laisser vides, et la raison. (2 (décision visée) : sans elle, aucun livrable n'est utile · 7 (critère de réussite) : sans elle, aucun travail n'est vérifiable ni terminé)
3. Sur l'extrait de l'atelier : 316 tickets pour 489 lignes. Quel outil pour recompter sur les 240 000 lignes du fichier complet, et pourquoi pas un tableur ? (SQL ou Python/DuckDB : agrégation avec `COUNT(DISTINCT ...)` sur 240 000 lignes est immédiate en base, pénible et fragile dans un tableur ; et l'opération sera rejouée, donc elle mérite une requête)
4. Un rapport Power BI publié avec une mesure « CA » définie trois fois dans trois rapports : quel métier est concerné, et quel artefact manque ? (BI analyst (ou vous, faute de mieux) ; le dictionnaire des mesures / modèle sémantique partagé, qui impose une définition unique)
5. **Question ouverte** : rédigez, pour votre propre situation (emploi, recherche, reprise d'études), une question analytique correcte que vous seriez capable de traiter dans trois mois avec les données dont vous disposez. (évalué sur les cinq composants, et sur la mention explicite d'une limite — pas sur l'ambition du sujet)

Transition vers le module 2. Vous savez lire les données, les classer, les typer, dater leur provenance et cadrer une demande. Vous ne savez pas encore les **résumer honnêtement** : une moyenne sans médiane, un pourcentage sans dénominateur, une tendance sans dispersion suffisent à faire publier une erreur avec assurance. Le module 2 — Statistiques pour l'analyse, et les pièges du résumé — vous donne ces outils, avec les mêmes données : les 480 lignes propres de l'atelier, et leurs 30 tickets aberrants qui vont vous occuper utilement.

M01 — Bilan de compétences et auto-positionnement

À remplir à la main (ou dans `journal.md` de votre cahier de l'apprenant) après le projet et l'évaluation. Ne sautez pas cette page : le parcours est conçu pour un autodidacte, et un autodidacte qui ne s'auto-évalue pas accumule des trous qui ne se voient qu'au module 12.

1. Je sais maintenant (vocabulaire — à cocher, et à définir de ma main)

Cochez seulement si vous pouvez **définir le mot en une phrase, sans jargon**, à quelqu'un qui ne travaille pas dans l'informatique.

Terme	Je le définis sans aide	Ma phrase à moi
donnée · information · connaissance · décision	☐	
table · ligne · colonne · cellule · en-tête	☐	
grain d'une table	☐	
clé primaire · clé étrangère · valeur distincte	☐	
type de données · inférence de type	☐	
nombre stocké en texte	☐	
encodage · séparateur · décimale · chapeau	☐	
structuré · semi-structuré · non structuré	☐	
qualitatif (nominal, ordinal) · quantitatif (discret, continu)	☐	
échelle de mesure (nominale, ordinale, intervalle, ratio)	☐	
question analytique · critère de réussite · périmètre	☐	
métrique · indicateur (KPI)	☐	
source de vérité · jeu figé · empreinte (hash)	☐	
dictionnaire de données · journal de transformation · manifeste	☐	
cycle de vie d'une donnée · rétention · minimisation	☐	

Test de réalité. Écrivez, sans regarder, ce que représentent : une ligne de `ventes_brutes.csv` (243 360 lignes) · une ligne de `objectifs_de_ca.csv` (218) · une ligne de `remises_manuelles.xlsx` (18 200) · une ligne de `dim_date.csv` (1 339). Quatre bonnes réponses = acquis. Trois ou moins = reprenez le chapitre C03, §5.2.

2. Je suis capable de (gestes professionnels)

Niveau d'autonomie : **A** = je le fais seul · **E** = je le fais avec la fiche sous les yeux · **N** = pas encore.

Geste	A / E / N	Preuve chez moi (fichier ou capture)
Ranger un dossier de travail en six étapes numérotées, avec brut en lecture seule	☐	

Geste	A / E / N	Preuve chez moi (fichier ou capture)
Nommer un fichier selon AAAA-MM-JJ_quoi_qui_etat.ext	☐	
Vérifier qu'une sauvegarde se restaure réellement	☐	
Décrire une table inconnue en six phrases	☐	
Compter des lignes, des objets distincts, des valeurs manquantes dans un tableur	☐	
Détecter une colonne de montants stockée en texte avec deux formules	☐	
Convertir une colonne sans perte, avec contrôle avant/après	☐	
Réparer deux formats de date dans une même colonne, et prouver la réparation	☐	
Vérifier séparateur, encodage, décimale et chapeau d'un fichier plat	☐	
Classer une variable et dire quel calcul est licite	☐	
Rédiger un dictionnaire de colonnes avec la colonne « défauts connus »	☐	
Tenir un journal de transformation avec écarts chiffrés	☐	
Convertir une demande floue en question analytique à cinq composants	☐	
Choisir un outil (tableur · SQL · Python · Power BI) et justifier par volume, relecteur, rejouabilité	☐	
Rédiger une note de décision en six rubriques, limites comprises	☐	

Seuil de passage : 11 gestes sur 15 en A ou E, dont obligatoirement « détecter une colonne texte », « question analytique à cinq composants » et « note en six rubriques ». Ces trois-là sont les prérequis réels du module 2.

3. Outils maîtrisés à la sortie du module

Outil	Ce que je sais faire sans aide	Ce que je ne sais pas encore faire (et dans quel module ça s'apprend)
Explorateur Finder	/ chemins, extensions, recherche, propriétés, lecture seule	scripts de renommage en masse (M08)
Tableur (Excel ou LibreOffice)	import assisté, NB, NBVAL, CBI/VALUE, SUPPRESpace, tri, filtre, comptage de doublons, formules matricielles simples	tableaux croisés, Power Query, RECHERCHEX (M03-M06)
Éditeur de texte	lire un CSV, compter des lignes, repérer un encodage	écrire un parseur (M08, M20)
Terminal	wc -l, head, cut, grep, file -i, md5sum/Get-FileHash	automatiser une chaîne de traitement (M08, M24)
Power BI, SQL, Python	— je sais quand ils sont nécessaires et je le justifie	M07 (SQL), M08-M10 (Python), M13-M18 (Power BI)

Cette colonne de droite est la plus utile : savoir nommer l'outil qui manque, avec la raison de son absence, est déjà une compétence d'architecte.

4. Niveau atteint

Le manuel utilise cinq niveaux, définis au §B.2 de l'architecture. En sortie de M01 :

Niveau	Définition	Où j'en suis
N1 — Literatie	comprendre le vocabulaire, lire une donnée, savoir où elle est, formuler une question	✓ objectif du module

Niveau	Définition	Où j'en suis
N2 — Opérateur	produire des mesures justes dans un outil	M02-M06
N3 — Analyste	mener une analyse complète et la défendre	M07-M12
N4 — Concepteur	modéliser, industrialiser, administrer	M13-M21
N5 — Référent	arbitrer, budgéter, transmettre	M22 et la pratique

Auto-positionnement : placez-vous sur N1 avec un chiffre de 1 à 5 (1 = je découvre encore le vocabulaire, 5 = je peux l'expliquer à un collègue avec nos données à lui). Notez la date. Vous referez ce test à la fin de M06, de M12, de M18 et de M22 : **la progression du chiffre compte plus que le chiffre lui-même.**

5. Pour passer au module suivant

1. **Correction des exercices manqués.** Reprenez uniquement les énoncés où votre réponse diffère du corrigé sur un **nombre** (pas sur la formulation). Un écart de vocabulaire se corrige en écrivant la définition ; un écart de nombre se corrige en refaisant le calcul.
2. **Préparer M02.** Gardez sous la main le fichier `2025_ventes_M5_propre_v1.csv` que vous avez produit (480 lignes, 36 073 185 FCFA) : le module 2 part de **vos** colonnes nettoyées, et non d'un fichier fourni. Si vous ne l'avez pas, refaites-le : c'est le temps d'achat du module 2.
3. **Trois choses à savoir par cœur avant le premier chapitre de M02** : le grain du fichier (une ligne = un article d'un ticket) · la différence entre 489 lignes et 316 tickets · le fait que 0,74 % du total dépend du nettoyage. Sans ces trois repères, les statistiques du module 2 seront comprises trop tard.
4. **Le droit à l'erreur est organisé, pas négocié.** Toute erreur détectée après coup dans ce manuel est corrigée à la source et publiée en annexe « errata » (règle de production n° 7) : signalez-la, vous contribuez à l'ouvrage, et vous gardez la trace de votre contrôle dans votre journal — ce qui compte pour la soutenance du module 22.

6. Auto-questionnement dirigé (à écrire, 15 minutes)

1. Quel défaut du fichier de l'atelier m'a le plus surpris, et pourquoi ne l'avais-je pas vu ?
2. Quelle formule ou quel réflexe du module vais-je utiliser dès demain, même hors formation ?
3. Quel terme technique ai-je employé sans pouvoir le définir ? (→ à recopier dans `glossaire_personnel.md`)
4. Quelle demande professionnelle réelle, en attente chez moi, puis-je convertir en question analytique cette semaine ?
5. Ai-je écrit un journal de transformation qui permettrait à un inconnu de refaire mon travail ?
Si non, qu'est-ce qui m'en a empêché — le temps, la méthode, ou la peur du regard ?

Ces cinq questions sont celles que poserait un enseignant en fin de module. Un enseignant, ici, n'existe pas : le cahier de l'apprenant le remplace, à condition de l'ouvrir.

Contrôle de sortie du module (à cocher par l'apprenant, vérifié par la suite du parcours)

- ☐ 7 chapitres lus et les 7 mini-projets rendus
- ☐ quiz $\geq 14/20$ · projet M01.P complet et noté $\geq 13/20$
- ☐ étude de cas rendue (cadrage de 5 demandes)
- ☐ fichier propre produit et total de contrôle = 36 073 185 FCFA

- [] dictionnaire de données avec colonne « défauts connus » remplie ($65 \cdot 486 \cdot 18 \cdot 7 \cdot 9$)
- [] journal de transformation ≥ 12 entrées, avec écarts chiffrés

Projet M01.P — « Un petit fichier de ventes, une première fois »

Module M01 · projet de fin de module · 5 h · à rendre avant le module 2 · barème /20, seuil de passage 13/20

Le cadre réel. Vous êtes engagé(e) comme analyste junior chez Sahel Distribution SA, grossiste en matériaux et quincaillerie, 5 magasins et 1 dépôt. Le service informatique est une personne, qui cumule avec la maintenance. Vous n'avez ni base de données, ni licence, ni mode d'emploi : vous avez un poste, un fichier CSV reçu par messagerie, et l'obligation de produire un chiffre que le directeur ne pourra pas contester. C'est exactement une première semaine de stage — et c'est la raison pour laquelle ce projet est noté, pas seulement « fait ».

1. Énoncé

Le matériel

Élément		Chemin	Ce que c'est
Fichier travail	de	01_socle_donnees/data/projection/ventes_magasin5_2025.csv	extrait des ventes du magasin 5, année 2025 : 489 lignes × 13 colonnes
Fichier fournisseur	du	01_socle_donnees/data/brut/tarif_fournisseur_peinture.csv	grille de prix Peinture Saaba SARL : 42 lignes , dont 38 utiles
Corrigé référence	de	01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv	480 lignes , à ne pas ouvrir avant le rendu
Vos outils		tableur (Excel ou LibreOffice) + éditeur de texte + un traitement de texte	rien d'autre n'est requis ni autorisé

Les quatre livrables

L1 — Fiche d'identité du fichier (1 page). Les six phrases du chapitre C03 (nom et source · objet · grain · clé · mesures et attributs · étendue), suivies d'un tableau de **contrôles** : pour chacune des 13 colonnes, le type constaté, le nombre de valeurs distinctes, les bornes ou les modalités notables, et les défauts visibles. Vous devez avoir **compté** les défauts, pas les avoir devinés.

L2 — Trois questions que l'on peut poser à ce fichier (une demi-page). Chacune formulée en **question analytique complète** (objet, unité + définition, comparaison, périmètre, critère de réussite), plus une ligne « ce que ce fichier ne permet pas de répondre » par question.

L3 — Une réponse chiffrée, sans autre outil que le tri et le filtre du tableur (1 page). Vous choisissez **une** des trois demandes ci-dessous, vous y répondez avec un chiffre juste, et vous annexez la méthode en cinq lignes maximum (aucune formule complexe, aucune macro, aucun autre logiciel) :

- **a)** « Quelle catégorie de produits pèse le plus dans le chiffre d'affaires du magasin 5 en 2025, et à quelle hauteur ? » (réponse attendue en L3-a du corrigé)
- **b)** « Quel vendeur arrive en tête, et avec quel total ? » (réponse attendue en L3-b)
- **c)** « Quel mois a été le plus fort, et de combien par rapport au plus faible ? » (réponse attendue en L3-c)

L4 — « Ce que je ne sais pas encore faire » (1 page, à écrire **après** L1-L3, sans tricher). La liste honnête des opérations que vous avez vues faire (par un collègue, dans un forum, dans ce

manuel) et que vous ne maîtrisez pas, classées en trois colonnes : ce dont j'aurai besoin tout de suite · ce dont j'aurai besoin dans trois mois · ce qui n'est pas mon besoin. Trois lignes minimum par catégorie.

Contraintes de rendu

1. Les fichiers sont déposés dans une arborescence conforme à la convention du module 1 (01_brut en lecture seule, 02_propre, 03_analyse, 04_livrables, 90_journal).
2. Le **journal de transformation** compte au moins **8 entrées** datées, chacune avec les nombres avant/après.
3. Le livrable L1-L4 est un seul document PDF nommé 2026-09-17_projet_M01_P_votre_nom.pdf, plus le classeur de travail.
4. **Interdits** : copier un total du corrigé sans montrer la manipulation qui y mène ; utiliser un tableau croisé dynamique ou une requête pour L3 (l'exercice porte sur le tri et le filtre) ; modifier le fichier brut.

2. Barème détaillé

#	Critère	Pts	Ce qui fait perdre les points
1	Fiche d'identité : six phrases justes	4	grain exprimé en « c'est du mensuel » (ce n'est pas un grain) ; clé donnée comme n_ticket seul
2	Contrôles par colonne : tous les comptages exacts	5	un seul nombre faux = -0,5 ; un nombre non vérifié (pas de formule, pas de commande) = -1
3	Trois questions : cinq composants, réfutabilité	3	adjectif non mesurable resté dans la question ; critère de réussite absent
4	Réponse chiffrée L3 : valeur exacte + méthode reproductible	4	chiffre juste obtenu par un moyen interdit ; ou juste mais sans méthode
5	Journal + arborescence + nommage	2	écart de total non expliqué ; fichier brut touché
6	L4 : lucidité de l'autoportrait	2	liste de vœux au lieu d'une liste d'outils/gestes précis
Total		20	seuil de passage en M02 : 13

3. Correction pas à pas (corrigé enseignant)

Cette correction est rédigée comme un compte-rendu de travail, parce que c'est ce qui est noté. Elle s'appuie sur les valeurs réellement produites par le fichier livré ; si votre fichier diffère, régénérez (python3 01_socle_donnees/scripts/generation_socle.py) avant de vous comparer.

3.1 — Réponse L1 : la fiche d'identité, telle qu'elle doit être rendue

Nom et source. ventes_magasin5_2025.csv, export de la caisse du magasin 5 pour l'année 2025, reçu par messagerie le 17/09/2026 ; 489 lignes, 32 Ko, en-tête sur 1 ligne, séparateur point-virgule, encodage UTF-8.

Objet. Les articles facturés au magasin 5 sur l'année 2025.

Grain. Une ligne = un article d'un ticket de caisse. Ni une vente, ni un jour, ni un client.

Clé. Aucune colonne n'est unique : n_ticket identifie le ticket et se répète (1 à 4 lignes par ticket). L'unicité de la ligne est assurée par le couple (n_ticket, produit).

Mesures et attributs. Mesures : `quantite`, `prix_unitaire_ht`, `remise`, `montant_ht`, `montant_ttc`. Attributs : `date`, `heure`, `magasin`, `vendeur`, `client`, `produit`, `categorie`. Identifiants : `n_ticket`, `client`, `produit`.

Étendue. 489 lignes · 316 tickets distincts · 31 jours avec des ventes (l'extrait contient 40 lignes par mois, tous certains jours ne sont donc pas représentés) · 4 vendeurs · 145 produits distincts · 7 catégories · 372 identifiants clients distincts · 18 lignes sans client · période couverte 2025-01-01 → 2025-12-31.

Tableau des contrôles (le cœur du point 2 du barème) :

Colonne	Type constaté	Distincts	Bornes / modalités	Défaut constaté	Comment je l'ai compté
<code>n_ticket</code>	texte	316	T05-25...-..., 17 caractères	9 lignes en double (489 – 480)	=SOMMEPROD(1/NB.SI(...)) ou tri + comptage manuel
<code>date</code>	texte	55	2025-01-01 → 2025-12-31	65 lignes en JJ/MM/AAAA	=NB.SI(B:B;"?*/?/*")
<code>heure</code>	texte	356	07:01 → 18:57	—	—
<code>magasin</code>	texte	1	« Magasin 5 — Kaya Marché »	—	—
<code>vendeur</code>	texte	4	Ilboudo, Sankara, Ouédraogo, Bationo	—	—
<code>client</code>	nombre ou vide	372	0 = comptoir	18 vides (=NB.EUILL)	vides ≠ 0 : deux choses distinctes
<code>produit</code>	texte	145	« ... — réf N » dans le libellé	deux infos dans une cellule	—
<code>categorie</code>	texte	7	Matériaux 146 · Quincaillerie 115 · Consommables 58 · Plomberie 57 · Electricité 54 · Peinture 34 · Bois & panneaux 25	—	tri + comptage
<code>quantite</code>	nombre	37	-13 (retours) → 14 000	7 lignes > 500	=MAX(), =NB.SI(I:I;">500")
<code>prix_unitaire_ht</code>	nombre	323	675 → 40 550	—	=MIN(), =MAX()
<code>remise</code>	nombre	6	0 → 0,11 (proportion)	aucune valeur > 0,4 ici	contrôle de plage
<code>montant_ht</code>	nombre	456	total = à recalculer	cohérent avec quantité × prix × (1 – remise)	=SOMME()
<code>montant_ttc</code>	texte	456	—	486 lignes non numériques	=NBVAL(M:M) - NB(M:M)

Le point qui distingue un bon rendu d'un rendu moyen : la ligne `quantite`. Le candidat solide écrit « 7 lignes suspectes, montant cohérent, donc doute sur l'unité de saisie → à voir avec le métier, pas à supprimer ». Le candidat pressé supprime les 7 lignes et perd 1 point au contrôle des totaux.

3.2 — Réponse L2 : les trois questions, avec leurs limites

Question 1 (mix de catégories). « En 2025, au magasin 5, quelle catégorie de produits représente la part la plus élevée du CA TTC (somme de `montant_ttc`, retours inclus avec leur signe, après retrait des doublons), et cette part dépasse-t-elle 20 % du total ? » Critère de réussite : la somme des 7 catégories doit redonner le total de l'extrait nettoyé, **36 073 185 FCFA** ; la réponse est une part en pourcentage à ±0,1 pt. Ce que ce fichier ne permet pas de répondre : la

rentabilité de la catégorie (aucun coût d'achat dans ce fichier — il est dans `couts_achat.csv`, module 13) et la comparaison aux autres magasins (extrait limité au magasin 5).

Question 2 (saisonnalité). « Quel mois de 2025 présente le CA TTC le plus éloigné de la médiane des douze mois, et de combien de fois le mois le plus faible ? » Critère : ratio meilleur mois / pire mois = **4,2** (décembre 5 292 517 FCFA, mai 1 258 801 FCFA). Limite : l'extrait contient 40 lignes par mois, **tous les mois ne sont donc pas complets** : ce ratio décrit l'extrait, pas la réalité commerciale. Cette phrase est obligatoire dans un rendu correct ; l'omettre vaut une note plafonnée à 15/20, quel que soit le reste.

Question 3 (qualité avant tout). « Combien de lignes de cet extrait ne peuvent pas être sommées telles quelles, et quel est l'effet chiffré de leur correction sur le total annuel ? » Critère : 486 montants en texte, 9 doublons, écart de total = **266 132 FCFA**, soit **+0,74 %** sur le total non nettoyé (36 339 317 → 36 073 185). Limite : le fichier ne dit pas si les doublons viennent de la caisse ou de l'export ; seule la caisse (ou le journal d'export) tranche.

3.3 — Réponse L3 : la méthode par tri et filtre, et les trois résultats

Préparation commune (autorisée : tri, filtre, =SOMME() sur la colonne filtrée).

1. Import assisté, toutes colonnes en texte, puis une colonne auxiliaire `ttc_num = CBI(SUPPRESSED(SUBSTITUE(...)))` — 486 conversions, 0 échec. (Oui, une formule de conversion est autorisée : sans elle, aucun total n'est possible ; ce qui est interdit, c'est le TCD et la requête.)
2. Tri sur `n_ticket` puis suppression des doublons stricts par la commande du tableur → 480 lignes. Contrôle : `=SOMME(ttc_num)` = **36 073 185**.

L3-a (catégorie dominante). Filtre sur `categorie` = Plomberie, `=SOMME()` sur la colonne filtrée → **8 754 982 FCFA**, soit **24,3 %** du total. Réponse : « la catégorie la plus lourde du CA 2025 du magasin 5 est la **Plomberie**, à 24,3 % ». Suit le classement complet, à exiger d'un rendu sérieux :

Catégorie	CA TTC (extrait nettoyé)	Part
Plomberie	8 754 982 FCFA	24,3 %
Matériaux	6 519 357 FCFA	18,1 %
Electricité	6 311 645 FCFA	17,5 %
Bois & panneaux	4 844 758 FCFA	13,4 %
Quincaillerie	4 203 121 FCFA	11,7 %
Peinture	2 938 358 FCFA	8,1 %
Consommables	2 500 964 FCFA	6,9 %
Total	36 073 185 FCFA	100 %

Attention au piège classique, et il est là pour cela : le **nombre de lignes** ne suit pas le CA. La catégorie la plus fréquente en lignes est Matériaux (146 lignes), pas Plomberie (57 lignes). Un rendu qui confond fréquence et montant perd le point 4 intégralement, parce que la phrase de conclusion serait fausse.

L3-b (vendeur en tête). Bationo : **11 370 350 FCFA** · Sankara 9 423 281 · Ouédraogo 8 483 019 · Ilboudo 6 796 535. Somme des quatre = 36 073 185 FCFA ✓ (le contrôle croisé fait partie de la réponse attendue).

L3-c (mois extrêmes). Décembre 2025 = **5 292 517 FCFA** ; mai 2025 = **1 258 801 FCFA** ; ratio **4,2**. Le détail complet des douze mois figure dans le chapitre C04 §5.3 ; un candidat doit pouvoir en retrouver au moins quatre sur demande orale du correcteur.

3.4 — Réponse L4 : ce à quoi ressemble un autoportrait utile

Exemple de rendu recevable (extrait) :

J'en aurai besoin tout de suite	Dans trois mois	Ce n'est pas mon besoin
tableaux croisés (M03-M05)	jointures SQL sur la base de l'entreprise (M07)	entraîner un modèle de prédiction (M19-M21)
RECHERCHEX pour relier les noms de produits (M06)	script Python pour rejouer le nettoyage (M08)	gestion de cluster (M24)
mettre un format de date fiable (déjà fait ici, à consolider M04)	publier un tableau de bord Power BI (M13-M18)	configuration d'un entrepôt (M15)

Un autoportrait qui écrit « je veux devenir meilleur en data » est noté 0/2 : ce n'est pas une liste de besoins, c'est une intention.

3.5 — Le fichier du fournisseur, traité à part (bonus +1, pas obligatoire)

Le point n'est pas dans l'énoncé, mais il revient toujours en entretien de stage : **importez tarif_fournisseur_peinture.csv proprement** et donnez le nombre de références, le prix moyen et le prix le plus élevé. Réponse : 38 lignes utiles (42 – 3 de chapeau – 1 d'en-tête), encodage **Windows-1252**, séparateur **;**, décimale **virgule**, TVA 0,18 sur toutes les lignes. Un rendu qui écrit « le fichier est corrompu » perd le bonus ; un rendu qui écrit « il faut lire en cp1252 avec 3 lignes à ignorer » le gagne.

4. Grille d'auto-évaluation avant rendu

À cocher soi-même, honnêtement, avant d'envoyer. Sept cases manquantes = rendez-vous dans deux jours.

- ☐ Chaque nombre cité dans L1 et L3 est le résultat d'une formule ou d'une commande que je peux rejouer devant quelqu'un.
- ☐ J'ai écrit le grain en une phrase, sans le mot « mensuel ».
- ☐ J'ai distingué les 9 doublons (à retirer) des 7 quantités suspectes (à signaler).
- ☐ J'ai dit « vide $\neq$ 0 » pour la colonne `client`, et je l'ai justifié par la règle métier du comp-toir.
- ☐ Mes trois questions L2 ont chacune cinq composants, et aucune ne contient d'adjectif non chiffré.
- ☐ Dans L2, j'ai écrit explicitement que l'extrait est un échantillon de 40 lignes par mois.
- ☐ Mon total nettoyé est 36 073 185 FCFA, et mon journal explique pourquoi le brut affichait 36 339 317 FCFA.
- ☐ Le brut est en lecture seule, et aucun de ses octets n'a changé (vérifié par empreinte).

5. Ce que le correcteur regardera en premier

Pas les chiffres — ils sont vérifiables en dix secondes. La première chose qu'un évaluateur expérimenté regarde, dans un dossier de ce niveau, est **l'ordre des opérations** : le candidat a-t-il documenté avant de corriger, ou corrigé puis justifié ? Le second regard va aux **limites écrites spontanément** (« l'extrait n'est pas la population ») : c'est le meilleur prédicteur de la capacité à travailler sans supervision. Le troisième, le nom des fichiers. Les chiffres viennent ensuite, et ils sont la partie facile.

Évaluation du module M01 — récupération, quiz, exercices, étude de cas, corrigés

Durée totale conseillée : 2 h 30 · Deux notes indépendantes, ni l'une ni l'autre négociable : quiz sur 20 (seuil de passage 14/20) et étude de cas sur 20 (seuil 12) · Le projet de module est noté séparément, sur 20, seuil 13 (03_exercices/M01_projet.md)

Le module est validé quand les trois notes sont au seuil. Chaque instrument répond à une question différente, et c'est pour cela qu'ils ne se compensent pas : le quiz dit si vous savez, les exercices si vous savez faire, l'étude de cas si vous savez cadrer avant de faire, le projet si vous savez livrer.

Instrument (barèmes du manuel, partie B.6 de l'architecture)	Barème	Durée	Ce qui est vérifié
A. Récupération — 7 questions	non noté, auto-corrigé	15 min	la mémoire des sept chapitres, sans relire
B. Quiz — 15 questions	/20, seuil 14/20	30 min	le vocabulaire, les types, la structure d'une table
C. Exercices pratiques — 4 exercices	auto-correction par résultat attendu chiffré	45 min	le geste : compter, convertir, contrôler
D. Étude de cas — 5 demandes	grille à 5 critères, /20, seuil 12	60 min	le cadrage, avant tout calcul

Matériel autorisé : le fichier `ventes_magasin5_2025.csv` (489 lignes), `tarif_fournisseur_peinture.csv`, un tableur, un éditeur de texte. Matériel interdit : le corrigé, les notes du projet M01.P (l'évaluation sert à vérifier que le savoir est **en vous**, pas dans votre dossier).

A. Questions de récupération (non notées, à traiter avant le quiz)

Répondez en une phrase, sans relire. Toute réponse hésitante = relecture ciblée du chapitre indiqué.

1. Que représente une ligne du fichier de l'atelier ? (C03 · un article d'un ticket, pas une vente)
2. Quelle est la différence entre un format et un type ? (C04 · l'apparence vs la nature de la valeur, donc ses opérations permises)
3. Combien de lignes a le fichier, et combien de tickets ? (C03 · 489 et 316)
4. Que fait `=NB()` que `=NBVAL()` ne fait pas ? (C04 · il ignore le texte : la différence des deux mesure les montants mal typés)
5. Un fichier en `;` encodé en Windows-1252 est-il cassé ? (C05 · non : il est lisible, c'est la lecture qui est mal réglée)
6. Citez les trois documents qui rendent un chiffre défendable dans trois ans. (C06 · dictionnaire de données, journal de transformation, manifeste d'archivage)
7. Quels sont les cinq composants d'une question analytique ? (C02/C07 · objet, unité + définition, comparaison, périmètre, critère de réussite)

B. Quiz — 15 questions, noté sur 20

Barème : 10 questions à 1 point (Q3, Q5, Q6, Q7, Q8, Q10, Q12, Q13, Q14, Q15) et 5 items-piège à 2 points (Q1, Q2, Q4, Q9, Q11), où la justification écrite est exigée : $10 \times 1 + 5 \times 2 = 20$. Une réponse juste sans justification sur un item-piège vaut 1 point sur 2 ; une justification sans réponse vaut 0. Les items-piège ne testent pas la mémoire des mots mais la frontière entre eux. Répondez par Vrai/Faux ou choisissez l'option ; le corrigé commenté suit immédiatement, à ne consulter qu'après avoir tout traité.

Frontières du vocabulaire (Q1 à Q4)

- Q1.** « Une donnée est une information. » Vrai ou faux ?
- Q2.** « Une colonne `id_client` remplie de nombres est une variable quantitative. » Vrai ou faux ?
- Q3.** « Un CSV ne contient pas de types de données. » Vrai ou faux ?
- Q4.** « Une fois formatée en date, une cellule contient une date. » Vrai ou faux ?

Structure et lecture d'une table (Q5 à Q9)

- Q5.** Sur le fichier de l'atelier, combien de tickets distincts pour 489 lignes ? a) 489 b) 480 c) 316 d) 145
- Q6.** Quelle colonne, à elle seule, peut servir de clé primaire de ce fichier ? a) `n_ticket` b) `produit` c) `date` d) aucune
- Q7.** Vous voulez compter « le nombre d'articles facturés » et « le nombre de ventes ». Quelle paire de comptages ? a) lignes, tickets distincts b) tickets distincts, lignes c) clients distincts, produits distincts d) montants distincts, lignes
- Q8.** Le fichier `ventes_propres.csv` du socle contient : a) 243 360 lignes b) 240 000 lignes c) 489 lignes d) 1 339 lignes
- Q9.** Un tableau croisé affiche un total exactement double du total de la colonne. Cause la plus probable : a) une ligne de sous-total importée comme donnée b) un encodage différent c) des dates au format français d) un identifiant client en texte

Types, formats, encodages (Q10 à Q13)

- Q10.** Combien de lignes de `montant_ttc`, dans le fichier de l'atelier, ne sont pas des nombres ? a) 0 b) 3 c) 486 d) 489
- Q11.** Après nettoyage complet, le CA TTC de l'extrait passe de 36 339 317 à 36 073 185 FCFA. Cet écart de 266 132 FCFA provient de : a) lignes en double b) retours négatifs c) TVA mal appliquée d) arrondis successifs
- Q12.** Le fichier du fournisseur compte 42 lignes. Combien de références ? a) 42 b) 41 c) 38 d) 39
- Q13.** Un fichier dont les accents s'affichent Ã© a été : a) corrompu par l'e-mail b) écrit en UTF-8 et lu en Windows-1252 (ou l'inverse) c) enregistré au format XLS d) tronqué

Posture et méthode (Q14 et Q15)

- Q14.** Le directeur demande « est-ce que ça marche bien ? ». La **première** action correcte est : a) charger le fichier dans le tableur b) construire un graphique de CA mensuel c) demander quelle décision dépend de la réponse d) proposer Power BI
- Q15.** Sur un échantillon de 480 lignes représentant 40 lignes par mois, vous affirmez « décembre est le meilleur mois de 2025 au magasin 5, la preuve d'une forte saisonnalité ». Ce raisonnement est : a) correct, le chiffre est exact b) incorrect : l'extrait n'est pas la population, et

un classement sur échantillon ne prouve pas une saisonnalité c) correct si le total de contrôle est bon d) incorrect, car il faudrait inclure 2026

C. Exercices pratiques (4) — séries E1 et E2, auto-correction

Ces quatre exercices ne sont pas notés au quiz : ils sont **validés par le résultat attendu**, parce que dans le métier un résultat ne se discute pas, il se vérifie. Traitez les quatre, comparez vos chiffres à l'attendu, puis lisez le corrigé. Un attendu manqué de peu n'est pas un demi-exercice réussi : cherchez l'écart, c'est là qu'est l'apprentissage.

Exercice C.1 (série E1). Ouvrez `ventes_magasin5_2025.csv` dans un éditeur de texte. Notez : le séparateur, le nombre de colonnes de la ligne d'en-tête, le contenu de la ligne 2 en trois mots, et une anomalie visible **sans calculer**. (attendu : ; · 13 · ticket, date, montant en texte · par exemple l'espace dans 56 658 ou la date 12/01/2025)

Exercice C.2 (série E1). Dans le tableur, importez avec toutes les colonnes en texte, puis donnez, par trois formules : le nombre de lignes de données, le nombre de valeurs numériques de `montant_ttc`, le nombre de lignes dont `date` contient un /. (attendu : `=NBVAL(A2:A490)` = 489 · `=NB(M2:M490)` = 3 · `=NB.SI(B:B;"?/*/*")` = 65)

Exercice C.3 (série E2). Produisez le fichier nettoyé attendu : 480 lignes, dates en ISO, `montant_ttc` en nombre, `quantite` rétablie sur les 7 lignes aberrantes (÷ 1 000), `client` conservé tel quel, y compris les 18 vides et les 0 de comptoir. Donnez le total de contrôle. (attendu : 36 073 185 FCFA ; la comparaison avec `ventes_magasin5_2025_ATTENDU.csv` est le contrôle de forme)

Exercice C.4 (série E2). Convertissez le fichier du fournisseur en un tableau propre de 38 lignes × 6 colonnes, prix en nombre, TVA conservée. Donnez le prix moyen et le prix maximal. (attendu : 11 155 FCFA en moyenne, 34 826 FCFA au maximum, TVA 0,18 sur toutes les lignes ; encodage cp1252, 3 lignes de chapeau ignorées, virgule décimale traitée)

D. Étude de cas — « Le patron veut savoir des choses » (20 points, seuil 12)

Contexte. Samedi matin, marché de Kaya. Le directeur de Sahel Distribution SA vous retient trente secondes près du parking et dit, d'un trait, les cinq phrases ci-dessous. Vous n'avez ni base, ni tableau de bord, ni temps : vous avez un carnet. Consignez, pour **chacune** des cinq demandes, la décision visée et la question analytique complète (objet, unité + définition, comparaison, périmètre, critère de réussite), ainsi que la ou les tables à ouvrir et **ce qui manquera** dans le jeu de données pour y répondre.

1. « Les prix de la concurrence, il faut qu'on s'aligne, tu crois qu'on est trop chers ? »
2. « Bationo, il vend trop bien, faut le augmenter — ou pas. Regarde. »
3. « On m'a dit qu'en janvier on ne vend rien, et moi je trouve qu'en janvier ça bosse. »
4. « Si on fermait le dépôt le dimanche, ça changerait quoi ? »
5. « Le magasin 5, il me plaît. Combien il rapporte ? »

Contraintes de rendu. Cinq blocs de six lignes maximum, format imposé :

[n°] Demande (verbatim, 1 ligne)	
Décision visée · Destinataire · Échéance	(1 ligne)
Question analytique	(1-2 lignes)
Tables et colonnes à ouvrir	(1 ligne)
Manquant / limite qui empêche de conclure	(1 ligne)
Outil choisi + motif en 4 mots	(1 ligne)

Grille de notation — 5 critères, 4 points par demande (5 × 4 = 20 points, seuil de validation 12). Pour chaque demande : question analytique comportant les cinq composants (1,5) · décision, destinataire et échéance nommés (1) · limite réellement limitée et non générique (0,5) · choix d'outil justifié par une contrainte et non par un goût (1). Un bloc qui répond à la place de cadrer est noté 0 : on ne demande pas ici le chiffre, on demande la question. Le dixième point de la grille est un point de **cohérence d'ensemble** : les cinq demandes doivent se ressembler (même gabarit, mêmes colonnes appelées), parce qu'un cadrage qui change de forme d'un dossier à l'autre n'est pas un cadrage, c'est une humeur du moment.

Corrigé de l'évaluation (à consulter après traitement complet)

B. Quiz — réponses et explications

Q1 — Faux. Une donnée est un fait brut ; elle devient information quand elle est replacée (quoi, qui, quand, unité). 56 658 n'est pas un chiffre d'affaires ; « 56 658 FCFA TTC, ligne 1 du ticket du 01/01/2025, magasin 5 » en est une. Le piège est réel : dans le langage courant, les deux mots sont interchangeables ; dans le métier, non.

Q2 — Faux. C'est un **identifiant**, donc une valeur nominale : le calculer, le moyener, le trier par ordre numérique n'a aucun sens. Le test : « une soustraction a-t-elle un sens ? » 10 857 - 3 312 ne veut rien dire.

Q3 — Vrai. Un CSV est du texte ; le type apparaît quand un outil le déclare ou le devine. C'est pourquoi les identifiants doivent être explicitement lus comme texte (et pourquoi le 075845 final du ticket T05-250101-075845 deviendrait 75845 si un tableur le lisait comme un nombre).

Q4 — Faux. Le format change l'affichage, pas la nature de la valeur. Une cellule affichant 12/01/2025 peut contenir le texte « 12/01/2025 » (et refuser tout calcul de mois) ou le nombre 45669 (le douzième jour de l'année 2025 compté depuis l'origine du tableur, qui est le 00/01/1900 : une vraie date, stockée en numéro de série). Le contrôle : =ESTNUM() puis =EST-DATE().

Q5 — c) 316. 480 lignes propres pour 316 tickets ; 489 est le nombre de lignes de l'énoncé, 145 le nombre de produits distincts.

Q6 — d) aucune. n_ticket se répète (1 à 4 lignes) ; produit et date aussi. L'unicité demande le couple (n_ticket, produit).

Q7 — a). Les lignes = les articles facturés, les tickets distincts = les ventes. Confondre les deux surestime le nombre de ventes de 55 % sur ce fichier.

Q8 — b) 240 000 lignes. ventes_brutes.csv en contient 243 360 ; 489 est l'extrait de l'atelier, 1 339 la table dim_date.

Q9 — a). Un sous-total importé comme donnée double exactement le total. Les autres options changent l'affichage ou la ponctuation, pas la somme. Vérification : compter les lignes dont la colonne-clé est vide.

Q10 — c) 486 sur 489 (3 valeurs seulement sont numériques). Le diagnostic complet : `=NB()` = 3, `=NBVAL()` = 489, la soustraction donne le nombre de défauts.

Q11 — a). Les 9 lignes en double injectées par l'export de la caisse. Le calcul : $36\,339\,317 - 36\,073\,185 = 266\,132$ FCFA, soit +0,74 % de surévaluation si l'on ne nettoie rien. La TVA, les retours et les arrondis sont cohérents dans ce fichier.

Q12 — c) 38 (42 lignes – 3 de chapeau – 1 d'en-tête). Un candidat qui répond 42 a importé le document, pas les données.

Q13 — b. Le fichier est intact, la lecture est mal réglée. Réflexe : changer l'encodage à **l'import**, jamais réenregistrer le brut.

Q14 — c). Avant l'outil et avant le graphique : la décision. Un tableau de bord sans décision associée ne sera pas ouvert après deux semaines.

Q15 — b). Le chiffre est exact (décembre 5 292 517 FCFA, mai 1 258 801, ratio 4,2), l'inférence ne l'est pas : l'extrait contient 40 lignes par mois, tous les mois ne sont donc pas représentés proportionnellement à leur activité réelle. Réponse d'analyste : « sur cet extrait, décembre domine ; pour conclure sur la saisonnalité, il faut le fichier complet du magasin ».

Seuil et rythme. 12 bonnes réponses sur 15 = 6/8 (correctif : 14/20 au total de l'évaluation, pas au quiz seul si l'étude de cas est pleine). Un score ≤ 8 sur 15 indique une relecture ciblée : Q1-Q4 ratées → C02 ; Q5-Q9 → C03 ; Q10-Q13 → C04-C05 ; Q14-Q15 → C06-C07.

C. Exercices — corrigés détaillés

C.1. Séparateur ; · 13 colonnes · ligne 2 = « ticket T05-250101-075845, Plâtre de construction 25 kg — réf 1, 11 unités, montant en texte 56 658 » · anomalie sans calcul : le 0.0 du client de la ligne 3 (comptoir), ou l'espace de milliers dans 51 802, ou une date 12/01/2025 ailleurs. Sont également recevables : les lignes en double visibles au tri sur `n_ticket`, et le libellé « ... — réf 1 » (deux informations dans une cellule). Ce qui ne l'est pas : « il y a des erreurs » (non localisé).

C.2. Les trois formules, et la logique derrière :

<code>=NBVAL(A2:A490)</code>	→ 489	lignes de données (contrôle de l'import : pas de ligne perdue)
<code>=NB(M2:M490)</code>	→ 3	montants réellement numériques
<code>=NB.SI(B2:B490;"?*//*")</code>	→ 65	dates écrites en JJ/MM/AAAA

Le correcteur vérifie la **cohérence des trois** : $489 = 480 + 9$ (doublons) et $486 = 489 - 3$ (montants texte). Un candidat qui obtient 488 lignes a mal importé (première ligne utilisée comme en-tête ou ligne vide finale).

C.3. Recettes, dans l'ordre : (1) import en texte · (2) `ttc_num = CBI(SUPPSPACE(SUBSTITUE(colonne;" ";""))`, contrôle `=NB(colonne d'erreurs) = 0` · (3) tri sur toutes les colonnes puis suppression des doublons stricts → 480 lignes · (4) détection des dates par le 5^e caractère et reconstruction en ISO des 65 lignes concernées · (5) quantités > 500 divisées par 1 000 (7 lignes, max 14 000 → 14) · (6) `client` laissé en l'état, avec une note : 18 vides, et les 0 = comptoir · (7) total : **36 073 185 FCFA**. Comparaison finale : `=SOMME` du fichier attendu (même fichier, 480 lignes) — égalité demandée à 0 FCFA près, non négociable : si l'écart n'est pas nul, une ligne a été supprimée à tort (le cas habituel : les 7 lignes de quantité, traitées en correction au lieu d'un marquage).

C.4. Import : ;, encodage Windows-1252, 3 lignes sautées, colonne `prix_unitaire` en texte puis `SUBSTITUE(cellule;".";";")` et remplacement de la virgule par le point avant conversion → 38 lignes, 6 colonnes. Moyenne **11 155 FCFA**, maximum **34 826 FCFA**, TVA 0,18 partout. Contrôles demandés : nombre de lignes (38), aucune valeur nulle, et `prix × 1,18` cohérent avec une grille interne si l'apprenant la consulte (bonus de méthode, pas de résultat).

D. Étude de cas — cinq cadrages attendus

Les cinq réponses ci-dessous sont des **modèles de rendu**, pas des réponses uniques : un candidat peut formuler autrement, à condition de conserver les cinq composants et la limite. Les numéros de lignes et de colonnes cités existent dans le socle fourni.

1. « On est trop chers ? » Décision : réviser la grille avant la campagne de décembre (direction commerciale, arbitrage avant le 30/11). Question analytique : « Entre 2023 et 2025, le prix moyen HT des 154 références a-t-il évolué moins vite que leur coût d'achat HT (couts_achat.csv, 1 694 lignes), et dans quelle proportion l'écart de marge brute s'explique par la remise plutôt que par le prix ? » Tables : ventes, produits, couts_achat. Manquant : **le prix des concurrents** — aucune donnée externe dans le jeu ; seule une étude terrain (20 références relevées au marché) le fournirait, à chiffrer comme tâche distincte. Outil : SQL ou tableur selon le volume ; motif : « calcul de taux sur 240 000 lignes ». Le point de lucidité attendu : dire que « trop cher » ne se mesure pas **dans ces données**, et proposer le relevé terrain comme complètement — c'est ce qui distingue un cadrage d'une promesse.

2. « Bationo vend trop bien, faut l'augmenter ? » Décision : politique de prime, direction générale, avant le prochain trimestre. Question analytique : « Sur 2025 (extrait magasin 5), le CA TTC et le **nombre de tickets distincts** par vendeur placent-ils A. Bationo en tête après retrait des 9 doublons, et cet écart tient-il au panier moyen (114 156 FCFA) ou au volume de tickets ? » Tables : ventes, vendeurs.csv (22). Manquant : vendeur n'existe que sur l'extrait du magasin 5 (4 vendeurs sur 22) ; la comparaison à l'ensemble des vendeurs suppose le fichier complet ; et une prime se discute avec la **marge**, non le CA — la marge nette n'est pas calculable (aucun coût de structure). Outil : tableur, motif : « échantillon, un seul chiffre à défendre ». Rendu attendu : sur l'extrait, Bationo = 11 370 350 FCFA (31,5 % du total), devant Sankara 9 423 281, Ouédraogo 8 483 019, Ilboudo 6 796 535 FCFA — et la phrase qui dit que ce n'est **pas** une preuve de performance sans le fichier complet ni la marge.

3. « En janvier, on ne vend rien / ça bosse. » Décision : caler les congés et les tournées du premier mois. Question analytique : « Sur l'année 2025, le CA de janvier (2 051 424 FCFA sur l'extrait) est-il dans le bas de la distribution des douze mois, et l'écart s'explique-t-il par le nombre de jours ouverts ou par le CA par jour ? » Tables : ventes, dim_date.csv (1 339 lignes, 12 colonnes, drapeaux dimanche/férié). Manquant : les jours d'ouverture réels ne sont pas dans le jeu (dim_date donne les dimanches et les fêtes, pas les fermetures exceptionnelles) ; et l'extrait ne contient que 40 lignes par mois. Outil : tableur, motif : « 12 lignes, un graphique suffit ». Le point de méthode attendu : distinguer « peu de lignes » de « peu de ventes » — sur un extrait mensuel fixe, la question devient un ratio, plus un total.

4. « Fermer le dépôt le dimanche ? » Décision : horaires d'exploitation, direction + exploitation, effet au 1^{er} janvier. Question analytique : « Un dimanche sur deux dans l'extrait des ventes : combien de tickets et quel CA sont réalisés un dimanche par le dépôt (id_magasin = 6) et par les magasins qu'il approvisionne ? » Tables : ventes, dim_date (colonne est_dimanche), magasins (6 lignes). Manquant : le jeu ne contient **pas** de flux physique dépôt → magasins (les transferts ne sont pas saisis), donc l'impact sur les ruptures est inconnu ; et stocks_quotidiens.csv couvre 60 000 lignes d'extrait, pas le champ complet. Outil : SQL, motif : « compte sur un gros fichier + jointure calendrier ». Rendu attendu : poser la question du **coût évité** (indisponible dans le jeu) autant que du chiffre d'affaires perdu — un cadrage qui ne voit que le CA est incomplet.

5. « Le magasin 5, combien il rapporte ? » Décision : arbitrer un investissement dans ce point de vente. Question analytique : « Que vaut le CA TTC 2025 du magasin 5 sur le fichier complet (à distinguer des 36 073 185 FCFA de l'extrait), et quelle **marge brute** — somme de (prix de vente – coût d'achat) par ligne — en FCFA et en pourcentage ? » Tables : ventes_propres.csv (240 000 lignes), couts_achat.csv, magasins.csv. Manquant : marge **nette** impossible (loyer, salaires, transport absents) ; objectifs_de_ca.csv compte 218 lignes, donc des mois sans cible. Outil : SQL ou Python, motif : « jointure sur 240 000 lignes et contrôle de to-

taux ». Le point le plus noté : le candidat qui **refuse** le mot « rapporte » sans le définir (CA ? marge brute ? marge nette ?) et demande la décision cachée derrière.

Total et décision d'orientation

Trois seuils, trois décisions. - **Quiz : 14/20 minimum.** En dessous, ne passez pas en M02 : les statistiques appliquées (M02) reposent mot pour mot sur la distinction type/format et sur le vocabulaire donnée-information. Reprenez C02 (frontières du vocabulaire) et C04 (types, formats), puis re-passez le quiz en entier — pas seulement les items ratés, parce que la mémoire des réponses fausses fausse la seconde tentative. - **Étude de cas : 12/20 minimum.** En dessous, le savoir est là mais le réflexe professionnel non : refaites les cinq demandes du projet M01.P qui portent sur le cadrage, et l'exercice guidé de C07, puis re-passez cette seule partie. - **Projet : 13/20 minimum** (barème dans 03_exercices/M01_projet.md). Un projet non rendu ne se compense par aucune note.

Entre 14 et 16 au quiz avec l'étude de cas réussie, passez en M02 en reprenant C04 avant son deuxième chapitre : les statistiques vous sembleraient obscures pour de mauvaises raisons. Cette règle de remédiation est la même pour les 21 modules suivants ; c'est l'une des rares choses du parcours qui ne se négocie pas.

MANUEL PROFESSIONNEL · ÉDITION 2026

Module M02 — Statistiques descriptives et inférence

La Voie des Données · Édition 2026 · 8 chapitres, 5 exercices guidés et leur corrigé par chapitre, 4 mini-projets, projet de module noté

Analyse de données appliquée & Business Intelligence
10 partie(s) assemblée(s) · document de production

Table des matières

MODULE M02.C01 — POPULATION, ÉCHANTILLON, INDIVIDU, VARIABLE, MODALITÉ, FRÉQUENCE	8
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Les cinq questions qui précèdent tout calcul	10
5.2 Population et échantillon : deux noms, un écart à déclarer	10
5.3 L'individu n'est pas la ligne	11
5.4 Variable, modalité, cardinalité : trois mots, trois usages	11
5.5 Effectif et fréquence : le calcul, et son total qui doit tomber juste	12
5.6 Deux fréquences qui se discutent, et pourquoi elles se discutent	13
5.7 Comment un échantillon se construit — et ce que chaque méthode coûte	13
6. Exemple concret : le même fichier, trois descriptions	14
7. Démonstration pas à pas : produire la fiche d'identité statistique	14
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la fiche d'identité, ensemble, en 25 minutes	16
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P1 — « La fiche d'identité statistique du fichier complet » (50 min)	18
14. Résumé du chapitre	18
15. À retenir	19
16. Évaluation formative (auto-correction, 8 min)	19
MODULE M02.C02 — LES CENTRES : MOYENNE, MÉDIANE, MODE ; MINIMUM, MAXIMUM, ÉTENDUE	21
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Les formules, sans mystère	23
5.2 Les trois centres sur le fichier du module : lire les écarts	23
5.3 La médiane a un effet de seuil, la moyenne un effet de niveau	24
5.4 Moyenne simple, moyenne pondérée : la colonne remise le démontre	24
5.5 Le cas d'école : tickets simples contre paniers composés	24
5.6 Minimum et maximum ne sont pas des bornes métier	25
5.7 Calculer les six nombres dans les trois outils	25
6. Exemple concret : le même indicateur, quatre énoncés	26
7. Démonstration pas à pas : les six nombres, trois outils, un contrôle	26

8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — le trio moyenne / médiane / mode, ensemble	28
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P2 — « Deux pages, six nombres, une décision » (45 min)	30
14. Résumé du chapitre	18
15. À retenir	19
16. Évaluation formative (auto-correction, 10 min)	31
M02.C03 — MESURER LA DISPERSION : ÉTENDUE, INTERVALLE INTERQUARTILES, ÉCART-TYPE	33
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Trois dispersions, trois usages	35
5.2 D'où vient l'écart-type, et pourquoi il a des décimales	35
5.3 L'IQR et la règle des 1,5 : une borne qui résiste à ce qu'elle détecte	36
5.4 Comparer deux colonnes : le coefficient de variation	36
5.5 Par catégorie : où la moyenne est-elle descriptive ?	37
5.6 L'erreur type : de combien votre moyenne est-elle fautive ?	38
5.7 Deux dispersions robustes, et ce que leur écart apprend	38
5.8 Quel outil pour quelle dispersion	39
6. Exemple concret : négocier le bois de chauffage, ou non	39
7. Démonstration pas à pas : les trois dispersions, trois outils, un contrôle	40
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — trois dispersions sur la colonne des prix	41
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P2 — suite : la colonne « précision » (30 min)	43
14. Résumé du chapitre	18
15. À retenir	19
16. Évaluation formative (auto-correction, 10 min)	31
M02.C04 — SE SITUER : QUARTILES, DÉCILES, PERCENTILES, LECTURE DU BOX-PLOT	45
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10

5.1 Un percentile est un rang, pas une valeur	47
5.2 Le calcul, puis la divergence des logiciels	47
5.3 Retourner le calcul : du montant au percentile	48
5.4 Lire une boîte à moustaches	49
5.5 Déciles et concentration : ce que porte la pyramide	49
5.6 Décider avec les percentiles : trois usages types	50
5.7 Quels percentiles publier, lesquels garder	50
5.8 Calculer les positions dans les trois outils	51
6. Exemple concret : la livraison gratuite, décidée au percentile	52
7. Démonstration pas à pas : tracer la boîte vous-même	52
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — les cinq nombres, puis le dessin	54
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P2 — suite : la page des positions (30 min)	55
14. Résumé du chapitre	18
15. À retenir	19
16. Évaluation formative (auto-correction, 10 min)	31
M02.C05 — LA FORME D'UNE DISTRIBUTION : HISTOGRAMME, ASYMÉTRIE, VA- LEURS EXTRÊMES	57
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Une distribution, trois miroirs	59
5.2 Construire l'histogramme : les trois décisions	59
5.3 L'asymétrie : combien de queue à droite	61
5.4 Le kurtosis : ce que les queues racontent	61
5.5 Trois sortes de valeurs extrêmes, trois traitements	62
5.6 Commenter une forme en cinq phrases — grille de rédaction	62
5.7 Ce que cette forme interdit de faire	62
5.8 Représenter la forme : quelle figure pour quelle question	63
6. Exemple concret : « nos clients ne sont pas comme ça »	63
7. Démonstration pas à pas : des comptes de classes à la bosse	64
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — refaire les classes, et voir ce qui change	65
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P2 — suite : la page de forme (30 min)	67
14. Résumé du chapitre	18
15. À retenir	19

16. Évaluation formative (auto-correction, 12 min)	68
M02.C06 — RELIER DEUX VARIABLES : COVARIANCE, CORRÉLATION, CAUSALITÉ, PARADOXES	69
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 La covariance, ou mesurer sans unité	71
5.2 Le coefficient de corrélation, et ce qu'il mesure vraiment	71
5.3 Rang contre valeur : Spearman	72
5.4 La droite de régression : trois nombres et une interdiction	73
5.5 Corrélation n'est pas causalité : les quatre configurations	74
5.6 Contrôler la variable de confusion — et le paradoxe de Simpson	74
5.7 Agrégation : pourquoi les corrélations montent quand on résume	75
5.8 Calculer les liaisons : quel outil, quelle formule	75
6. Exemple concret : « la remise fait-elle vendre plus gros ? »	76
7. Démonstration pas à pas : du nuage au coefficient, puis au contrôle	77
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la liaison quantité / montant, entièrement refaite	78
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P3 — « La page des liaisons » (50 min)	80
14. Résumé du chapitre	18
15. À retenir	19
16. Évaluation formative (auto-correction, 12 min)	68
M02.C07 — HASARD ET INCERTITUDE : PROBABILITÉS DE BASE, ÉCHANTILLONNAGE, BIAIS	82
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Hasard, fréquence, probabilité : trois règles suffisent	84
5.2 Ce que le hasard ajoute : la distribution d'échantillonnage	84
5.3 Ce que le hasard ne pardonne pas : le biais	85
5.4 Cinq familles de biais, et leur trace dans notre fichier	86
5.5 L'intervalle de confiance : ce que « 95 % » veut dire	87
5.6 Le bootstrap : remesurer sans retourner sur le terrain	87
5.7 Combien de tickets faut-il mesurer ?	88
5.8 Corriger après coup, et choisir l'outil	89
6. Exemple concret : « le panier moyen du magasin 5, à quoi bon le discuter ? »	90
7. Démonstration pas à pas : de la règle d'extraction à l'intervalle	90

8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — mesurer l'incertitude d'un panier moyen	92
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P4 — « La page d'incertitude » (45 min, premier volet ; M02.C08 enchaîne)	93
14. Résumé du chapitre	18
15. À retenir	19
16. Évaluation formative (auto-correction, 12 min)	68
M02.C08 — DÉCIDER AVEC INCERTITUDE : INTERVALLE DE CONFIANCE, TESTS, INTERPRÉTATION ET LIMITES	96
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 La charpente d'un test, en cinq mouvements	98
5.2 Choisir le test d'après la question et le grain	98
5.3 Comparer deux moyennes : le t de Welch, et l'intervalle qui va avec	99
5.4 Ce que notre étude peut détecter : puissance et marge de détection	100
5.5 Deux tests, deux réponses : que faire quand ils divergent	100
5.6 Fréquences et associations : le chi-deux	101
5.7 Comparaisons multiples : le piège qui fausse tout le reste	102
5.8 Les limites du test, et les outils	102
6. Exemple concret : « qui met-on sur le tableau d'honneur ? »	103
7. Démonstration pas à pas : de la question au verdict	104
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — trancher une comparaison de vendeurs de bout en bout	105
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M02.P4 — « La page d'incertitude » (45 min, second volet)	107
14. Résumé du chapitre	18
15. À retenir	19
16. Évaluation formative (auto-correction, 12 min)	68
PROJET M02.P — « PROFIL STATISTIQUE DU MAGASIN 5 »	110
1. Énoncé	110
Le matériel	110
Les six livrables	110
Contraintes de rendu	111
2. Barème détaillé	111
3. Correction pas à pas (corrigé enseignant)	112
3.1 — L1 : l'identité statistique, telle qu'elle doit être rendue	112

3.2 — L2 : centres et dispersion	112
3.3 — L3 : positions et forme	112
3.4 — L4 : liaisons	113
3.5 — L5 : incertitude, tests, décision	113
3.6 — L6 : les trois anomalies, écrites comme attendu	114
4. Grille d'auto-évaluation avant rendu	114
5. Ce que le correcteur regardera en premier	115
ÉVALUATION M02 — « STATISTIQUES DESCRIPTIVES ET INFÉRENCE »	116
A • Questions de récupération (non notées)	116
B • Quiz (20 questions • 20 points)	116
Bloc 1 — Population, échantillon, biais (Q1 à Q5)	117
Bloc 2 — Centres (Q6 à Q8)	117
Bloc 3 — Dispersion et positions (Q9 à Q12)	117
Bloc 4 — Formes et liaisons (Q13 à Q16)	117
Bloc 5 — Estimation et tests (Q17 à Q20)	118
C • Exercices pratiques (2 séries • 55 min • auto-correction exigée)	118
D • Étude de cas (20 points • seuil 12 • 60 min)	119
Le dossier	119
Les quatre livrables demandés	119
Ce que le correcteur attend, ligne à ligne	119
Corrigé	119
B • Quiz — réponses et explications	119
C • Exercices pratiques — corrigés	120
D • Étude de cas — éléments de réponse attendus	121
Total et décision d'orientation	121

Module M02.C01 — Population, échantillon, individu, variable, modalité, fréquence

Outil de ce chapitre : un tableur et une base SQL, au choix ; aucun logiciel de statistiques. Durée indicative : 4 h. Niveau : N1.

L'idée du chapitre. Avant de calculer quoi que ce soit, il faut savoir **de quoi l'on parle** : sur combien de choses, sur lesquelles de ces choses, et si ce que l'on a sous la main ressemble à ce que l'on voudrait connaître. Ces cinq mots — population, individu, échantillon, variable, modalité — ne sont pas du vocabulaire scolaire : ce sont les cinq lignes d'en-tête de tout travail qui engage une décision. Une moyenne calculée sur le mauvais objet est juste, et fautive à la fois ; c'est précisément ce qui rend le chapitre obligatoire avant les sept suivants.

Base de travail — deux chemins pour une seule donnée. Les fichiers du module vivent à deux endroits, et il faut le trancher une fois pour toutes. Dans l'atelier du manuel — le dépôt qui contient ces fichiers — ils sont sous `01_socle_donnees/data/`. Chez vous, copiez-les dans votre dossier de module et rangez-les dans deux sous-dossiers, `donnees/projection/` pour le fichier de l'énoncé et `donnees/reference/` pour l'attendu et les référentiels. **Les blocs de code des huit chapitres écrivent le chemin de l'atelier, parce que c'est celui qui s'exécute sans préparation** : si vous travaillez dans votre dossier, remplacez le préfixe `01_socle_donnees/data/` par `donnees/` et ne changez rien d'autre. Séparateur point-virgule, encodage UTF-8, montants en franc CFA (FCFA), virgule décimale au tableur. Tous les chiffres cités dans le module sont lisibles dans `01_socle_donnees/data/reference/chiffres_cites.md`, section M02.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Nommer** la population, l'échantillon, l'individu, la variable et la modalité d'un jeu de données donné, sans hésiter sur les deux derniers.
2. **Compter** ce que vous avez : effectif total, effectifs par modalité, fréquences en pourcentage, et les écrire avec l'unité.
3. **Distinguer** une ligne d'un individu statistique, et dire lequel des deux compte pour la question posée.
4. **Repérer** une variable à une seule modalité dans un extrait et expliquer ce que cela interdit comme conclusion.
5. **Écrire** la fiche d'identité statistique d'un fichier — celle qui, en entreprise, se joint au chiffre et le rend défendable.

2. Pourquoi cette notion est importante

Situation 1 — la réunion qui dérape sur un mot. Un responsable commercial annonce : « notre panier moyen est à 114 156 FCFA ». Un autre répond : « c'est faux, je vends tous les jours à des clients qui repartent avec moins de 60 000 ». Aucun des deux n'a tort : le premier a calculé une moyenne sur des tickets, le second pense au ticket le plus courant. La dispute n'est pas un désaccord de chiffres mais **un désaccord d'objet** — sur quoi porte le compte. Ce chapitre règle ces disputes avant qu'elles n'aient lieu.

Situation 2 — le rapport qui ne dit pas ce qu'il couvre. Vous remettez un document qui affirme « 29,8 % des lignes concernent les Matériaux ». Trois remarques peuvent arriver, et elles arrivent : sur quelles lignes ?, sur quelle période ?, et si l'on compte les tickets au lieu des lignes ?. Un rapport qui répond à ces trois questions dans son premier paragraphe met son lecteur en confiance ; un rapport qui n'y répond pas laisse le doute travailler pour lui.

Situation 3 — l'échantillon qui ne représente rien. Vous disposez du fichier complet de quatre années (240 000 lignes propres), mais votre extrait de travail en contient 480 — 0,2 % des lignes. Ce n'est pas un problème en soi : c'est un fait à **dire**. Le problème commence quand on écrit « décembre est notre meilleur mois » comme si l'on avait parlé de l'entreprise, alors que l'on a parlé d'un extrait construit à des fins pédagogiques, à raison de 40 lignes par mois.

Dans les faits. La règle « sur un échantillon, on ne conclut pas sur la population » n'est pas une coquetterie de statisticien : c'est ce qui sépare une analyse d'une rumeur chiffrée. Dans les grandes organisations, les chiffres qui circulent sans mention de périmètre finissent par produire deux tableaux de bord qui se contredisent, et des réunions qui arbitrent au lieu de décider.

3. Explication simple

Imaginez un marché, un samedi matin. Vous voulez savoir « ce que vend le marché ». Vous ne pouvez pas tout voir : vous vous postez à **un** étal, pendant **une** heure, et vous notez chaque vente sur un carnet.

- **Le marché entier, toutes les ventes de toutes les journées** : c'est la **population** — l'ensemble que vous aimeriez décrire.
- **Les ventes que vous avez réellement notées** : c'est l'**échantillon** — la partie visible, celle sur laquelle vous calculez.
- **Chaque vente notée sur une ligne du carnet** : c'est un **individu** statistique — l'unité sur laquelle on compte. Pas le client, pas l'objet vendu : la vente, si c'est elle que vous avez décidé de compter.
- **Une caractéristique que vous notez à chaque fois** (le montant, l'étal, l'heure) : c'est une **variable**.
- **Une valeur prise par cette variable** (« légumes », « 1 500 FCFA ») : c'est une **modalité**.
- **Combien de fois apparaît une modalité** : c'est l'**effectif** ; ramené au total, c'est la **fréquence**.

Tout le reste du manuel applique ce petit vocabulaire. Un tableau de statistiques mal lu est presque toujours un tableau où l'individu n'était pas celui que le lecteur imaginait.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Population — population	L'ensemble complet des choses que l'on voudrait décrire.	La confondre avec le fichier ouvert : le fichier n'est souvent qu'un extrait.
Individu — statistical unit / record	L'unité que l'on compte : une ligne, un ticket, un client, un mois, selon la décision prise.	Compter des lignes en croyant compter des clients.
Échantillon — sample	La partie réellement observée de la population.	Oublier de dire comment il a été choisi : le mode de choix décide de ce qu'il autorise.

Français — English	Définition simple	Piège à éviter
Recensement — census	Observer toute la population, sans échantillon.	Croire qu'un gros fichier est un recensement : 240 000 lignes peuvent n'être qu'un mois.
Variable	Une caractéristique relevée pour tous les individus (une colonne).	Appeler « variable » un identifiant : il varie, mais ne mesure rien.
Modalité — value / category	Une valeur possible d'une variable (« Peinture », 0,04, vrai).	Compter les modalités comme des individus.
Effectif — count / frequency count	Combien d'individus prennent cette modalité.	Donner un effectif sans dire sur combien de lignes il porte.
Fréquence — relative frequency	Effectif divisé par l'effectif total, en pourcentage.	Arrondir chaque ligne à l'entier et obtenir 99 % ou 101 % au total.
Cardinalité — cardinality	Nombre de modalités distinctes d'une variable.	Une cardinalité de 1 dans un extrait : elle interdit la comparaison qu'on voulait faire.
Représentativité — representativeness	Propriété d'un échantillon qui ressemble à sa population sur les variables qui comptent.	La revendiquer sans la discuter : elle s'examine, elle ne se décrète pas.

5. Cours approfondi

5.1 Les cinq questions qui précèdent tout calcul

Avant la moindre formule, un professionnel écrit cinq lignes. Sur le fichier de travail de ce module — l'extrait nettoyé de 480 lignes du magasin 5 pour 2025 — cela donne :

1. **Quelle population ?** Les ventes du magasin 5, sur l'année 2025, telles que la caisse les a enregistrées.
2. **Quel individu ?** La ligne de ticket (un article facturé), parce que le fichier est livré à ce grain.
3. **Quel échantillon ?** 480 lignes, construites à raison de 40 lignes par mois, dans un fichier complet de 240 000 lignes propres.
4. **Quelles variables ?** 13 colonnes : `n_ticket`, `date`, `heure`, `magasin`, `vendeur`, `client`, `produit`, `categorie`, `quantite`, `prix_unitaire_ht`, `remise`, `montant_ht`, `montant_ttc`.
5. **Quelles modalités compte-t-on ?** Selon la question : 7 catégories, 4 vendeurs, 145 produits, 372 clients distincts (hors vides), 1 magasin.

Ces cinq lignes occupent trois minutes. Elles suppriment la moitié des allers-retours d'une analyse, et elles se recopient mot pour mot dans la note qui accompagne le chiffre.

5.2 Population et échantillon : deux noms, un écart à déclarer

Définition. Population — population — l'ensemble complet des individus que l'on voudrait décrire. Elle est presque toujours inatteignable : ce que l'on a sous la main est un fichier, un mois, un magasin.

Définition. Échantillon — sample — la partie réellement observée de la population, sélectionnée par une règle. Ce qui la rend utilisable n'est pas sa taille mais sa **description** : sans la règle de sélection écrite, aucun pourcentage ne peut être discuté, seulement cru.

Un échantillon n'est pas « un petit morceau au hasard des fichiers » : c'est un morceau **choisi par quelque chose**. Ici, le choix est explicite : 40 lignes par mois, en prenant les premières du

mois après tri par date. Ce mode de fabrication a une conséquence que vous devez savoir énoncer : la **répartition mensuelle est forcée** — chaque mois pèse le même poids (40 lignes sur 480, soit 8,3 %), alors que dans la population les mois ne pèsent pas pareil. Un graphique de la répartition par mois sur cet extrait est donc **faux d'office** sur une seule question : « quel mois vend le plus ? » — et parfaitement utilisable sur une autre : « comment se répartissent les catégories de produits ? ».

À retenir. La bonne question n'est jamais « l'échantillon est-il petit ? » mais « **sur quoi le mode de fabrication fausse-t-il le raisonnement ?** ». Un échantillon de 480 lignes bien décrit répond à six questions ; un échantillon de 240 000 lignes mal décrit n'en répond correctement à aucune.

5.3 L'individu n'est pas la ligne

Le fichier contient 480 lignes et 316 tickets : un ticket porte en moyenne 1,52 ligne, certains jusqu'à 4. Les deux objets sont légitimes, mais ils ne répondent pas aux mêmes questions, et ils ne donnent pas les mêmes pourcentages.

Question	Individu à compter	Ce que l'on obtient
Combien d'articles sont facturés ?	la ligne	480
Combien de ventes (paniers) ?	le ticket	316
Quelle part des ventes comportent plusieurs articles ?	le ticket	104 sur 316, soit 32,9 %
Quelle catégorie domine le catalogue vendu ?	la ligne (ou le montant, si l'on pondère)	Matériaux, 143 lignes, 29,8 %

Un même fait, deux énoncés justes : les Matériaux sont la première catégorie **en lignes** (143 sur 480) ; ils ne le sont pas forcément **en montants**, et encore moins en nombre de tickets contenant au moins un article de la catégorie. La règle du métier : **on nomme l'individu avant de nommer le pourcentage**.

Définition. Tirage — sampling — la règle, écrite et reproductible, qui a sélectionné les individus de l'échantillon dans la population. Elle se note avec son effet attendu : « 40 premières lignes par mois après tri par date » dit à la fois ce qui a été pris et ce que cela écrase (les volumes relatifs des mois).

Attention. Une phrase comme « 30 % de nos clients achètent du bois » est doublement ambiguë : 30 % des lignes ou 30 % des clients ? Sur cet extrait, la seconde lecture n'est même pas calculable en l'état, parce que la colonne `client` est vide sur 103 lignes (18 cases vides et 85 ventes au comptoir codées 0). Nommer l'individu, c'est aussi vérifier qu'il est **identifiable** partout où l'on compte dessus.

5.4 Variable, modalité, cardinalité : trois mots, trois usages

Définition. Variable — variable — une caractéristique relevée pour **tous** les individus, c'est-à-dire une colonne dont on peut dire en une phrase ce qu'elle mesure et dans quelle unité.

Définition. Modalité — value, category — une valeur prise par la variable (« Peinture », 0,04, vide). Compter des modalités, ce n'est pas compter des individus : 6 remises différentes s'appliquent à 480 lignes.

Pour chaque colonne, on mesure trois choses — et elles s'écrivent dans un tableau, pas dans une conversation :

Colonne	Nature	Modalités distinctes (mesuré)	Ce que la cardinalité autorise ou interdit
magasin	qualitative nominale	1	Aucune comparaison entre magasins : elle est impossible, pas seulement déconseillée.
vendeur	qualitative nominale	4	Comparaison possible, mais sur 4 individus seulement : un « classement » n'a de sens que comme constat.
categorie	qualitative nominale	7	Répartition exploitable ; c'est la colonne reine de ce chapitre.
produit	qualitative nominale	145	Listes et classements ; jamais un graphique en secteurs (145 parts illisibles).
client	identifiant (nominal, quasi-continu)	372 distincts, 103 lignes sans client	Comptage d'objets, aucun calcul sur la valeur.
remise	quantitative discrète	6 modalités (0 ; 0,03 ; 0,04 ; 0,07 ; 0,08 ; 0,11)	Fréquences et moyenne pondérée : deux réponses différentes, les deux utiles.
quantite, montant_ht, montant_ttc, prix_unitaire_ht	quantitatives	respectivement 45, 456, 456 et 323 valeurs distinctes	Résumés de position et de dispersion : le chapitre suivant.
date, heure	quantitatives (ordinal pour date)	31 jours d'écriture, 12 mois	Analyses calendaires, à condition de se souvenir du tirage (5.2).

Une cardinalité de **1** est une information précieuse : elle dit qu'une variable ne portera aucune réponse comparative. Sur l'extrait, `magasin` vaut « Magasin 5 — Kaya Marché » sur les 480 lignes. Écrire « le magasin 5 réalise 36 073 185 FCFA » est vrai ; en déduire qu'il est « le meilleur magasin » est une faute de logique, pas d'arithmétique.

Définition. Cardinalité — cardinality — le nombre de modalités distinctes d'une variable. Elle ne se devine pas : elle se compte, et elle conditionne le choix du graphique autant que celui de la statistique.

Conseil professionnel. Faites tenir ce tableau dans une page, avec trois colonnes seulement : combien de modalités, y a-t-il des valeurs manquantes, quel calcul cette colonne autorise. C'est la fiche que l'on demande à un analyste qui reprend un dossier laissé par un autre, et c'est ce qui vous évitera de refaire le travail de quelqu'un d'autre dans trois mois.

5.5 Effectif et fréquence : le calcul, et son total qui doit tomber juste

La fréquence d'une modalité est son effectif divisé par l'effectif total. Sur les 480 lignes :

Catégorie	Effectif (lignes)	Fréquence
Matériaux	143	29,8 %
Quincaillerie	112	23,3 %
Consommables	57	11,9 %
Plomberie	57	11,9 %
Electricité	53	11,0 %

Catégorie	Effectif (lignes)	Fréquence
Peinture	34	7,1 %
Bois & panneaux	24	5,0 %
Total	480	100,0 %

Trois réflexes de rédaction, qui viennent de l'impression et non des mathématiques : on affiche les **effectifs** à côté des pourcentages (un pourcentage sans dénominateur ne se vérifie pas) ; on garde **une décimale**, pas trois ; on **additionne les pourcentages affichés** pour vérifier qu'ils retombent sur 100 — ici $29,8 + 23,3 + 11,9 + 11,9 + 11,0 + 7,1 + 5,0 = 100,0$ ✓. Si l'arrondi faisait manquer 0,1 point, on le dirait (« totaux légèrement différents de 100 % pour cause d'arrondi ») au lieu de retoucher une ligne au hasard pour que ça tombe juste.

5.6 Deux fréquences qui se discutent, et pourquoi elles se discutent

Les remises. 173 lignes sur 480 portent une remise, 307 n'en portent aucune : 64,0 % des lignes sont vendues au prix affiché. La moyenne simple des valeurs de la colonne est de 0,0185 (1,85 %), la moyenne **pondérée par les montants** est de 0,0172 (1,72 %). Les deux chiffres sont corrects ; ils répondent à deux questions différentes — « sur une ligne, quelle remise applique-t-on ? » et « sur un franc facturé, quelle remise concède-t-on ? ». La seconde est celle qui intéresse le directeur financier ; la première est celle qui intéresse le responsable de magasin. Un analyste qui ne connaît qu'une des deux se fera corriger en réunion.

Les dates. 480 lignes se répartissent sur 12 mois et 31 jours distincts, avec une particularité mesurée dans le fichier brut d'origine : 55 écritures de dates différentes pour 31 jours réels (12/01/2025 et 2025-01-12 désignent le même jour). Le nettoyage a ramené tout le monde à l'écriture ISO ; la fréquence par jour, elle, ne s'appuie que sur les 31 jours réellement présents dans l'extrait. Annoncer une fréquentation journalière moyenne sans dire que le tirage a forcé 40 lignes par mois serait une conclusion tirée d'un artefact de fabrication.

Attention. Le pourcentage qui suit le mot « la majorité » mérite le même contrôle que les autres. Sur ce fichier, « la majorité des lignes sont des Matériaux » est **faux** (29,8 %), alors que « deux lignes sur trois sont vendues sans remise » est **vrai** (64,0 %). Une majorité exige plus de 50 % ; une modalité dominante n'exige rien de plus que la première place, et le mot juste change le verdict.

5.7 Comment un échantillon se construit — et ce que chaque méthode coûte

Méthode	Ce qu'elle garantit	Ce qu'elle écrase
Tirage systématique (une ligne sur k)	une couverture du temps, de la simplicité	les structures fines ; faux si la source est périodique
Par strate (40 lignes par mois, par catégorie)	que chaque groupe sera représenté	les poids relatifs des groupes — c'est le cas ici
Aléatoire simple	l'absence de biais de choix	le risque de n'avoir aucune ligne d'un petit groupe
De convenance (« les premières lignes du fichier »)	rien	tout : la représentativité n'est pas même discutée

Le vocabulaire statistique complet de l'échantillonnage aléatoire (loi des grands nombres, intervalle d'estimation) tient au chapitre M02.C07. Ici, l'essentiel tient en une phrase : **on choisit une méthode, on la dit, et on écrit ce qu'elle écrase.**

Boîte à outils. Compter proprement, en trois outils : tableur `=NBVAL(plage)` pour l'effectif, `=NB.SI(plage;modalité)/NBVAL(plage)` pour la fréquence, et la formule de comptage des valeurs distinctes `=SOMMEPROD(1/NB.SI(plage;plage))` ; SQL (DuckDB) `SELECT categorie, COUNT(*) AS effectif, ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER (), 1) AS freq_pct FROM ventes GROUP BY categorie ORDER BY effectif DESC ; ;` Python (pandas) `df["categorie"].value_counts(normalize=True).mul(100).round(1)`. Les trois doivent rendre le même tableau : quand ils divergent, ce n'est pas le logiciel qui a tort, c'est le périmètre qui a bougé.

6. Exemple concret : le même fichier, trois descriptions

Un collègue remet une note qui dit : « 145 produits, 372 clients, 29,8 % de Matériaux, un magasin ». Elle est juste et inutilisable, parce qu'elle ne dit pas ce qu'elle compte. Réécrivons la même information avec les cinq mots du chapitre, sur le fichier de 480 lignes :

- **Population visée** : les ventes du magasin 5 en 2025, à la ligne de ticket.
- **Échantillon observé** : 480 lignes (0,2 % des 240 000 lignes propres du socle), tirées à raison de 40 lignes par mois après tri par date.
- **Individu** : la ligne de ticket. Les 480 lignes portent 316 tickets (1,52 ligne par ticket, maximum 4), donc 32,9 % des tickets contiennent plusieurs articles.
- **Variables et cardinalités** : 7 catégories, 4 vendeurs, 145 produits, 372 clients distincts hors vides, 6 modalités de remise, 1 seul magasin.
- **Fréquences retenues** : Matériaux 143 lignes (29,8 %), Quincaillerie 112 (23,3 %), sans remise 307 (64,0 %).

Deux lignes de plus que la note du collègue, et la discussion d'après-réunion devient impossible — c'est le but.

7. Démonstration pas à pas : produire la fiche d'identité statistique

Nous fabriquons le document qui accompagnera tous vos chiffres du module. Douze minutes, deux outils.

Étape 1 — Ouvrir le bon fichier. `01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv` (480 lignes, séparateur ;, encodage UTF-8 avec BOM). Importez **toutes les colonnes en texte** : c'est le réflexe du chapitre M01.C04, il évite que le typage automatique ne décide à votre place.

Étape 2 — L'effectif, pas le nombre de lignes affichées. Dans le tableur, la formule `=NBVAL(A2:A481)` sur la colonne `n_ticket` renvoie 480. Notez-le : c'est le n de toutes les fréquences du document.

Étape 3 — Les individus qui ne sont pas des lignes. `=SOMMEPROD(1/COUNTIF(A2:A481;A2:A481))` renvoie 316 : le nombre de tickets distincts. Écrivez les deux chiffres côte à côte, avec leur nom — c'est la ligne qui tue l'ambiguïté « on compte quoi ? ».

Étape 4 — La cardinalité de chaque colonne. Appliquez la même formule de valeurs distinctes aux 13 colonnes. Le résultat doit donner, entre autres : 7 (catégories), 4 (vendeurs), 145 (produits), 372 (clients distincts non vides), 6 (remises), 31 (jours), 1 (magasin). Si une valeur vous surprend, c'est un défaut du fichier, pas de la formule : cherchez avant de publier.

Étape 5 — Les effectifs par modalité. Tableau croisé : lignes en valeur, `categorie` en ligne. Vous obtenez 143, 112, 57, 57, 53, 34, 24. Le total du croisé doit redonner 480 au FCFA et au compteur près : c’est le contrôle de cohérence, pas une formalité.

Étape 6 — Les fréquences. Divisez chaque effectif par 480 et multipliez par 100, une décimale. Le total des pourcentages affichés doit faire 100,0 %.

Étape 7 — Le contrôle par SQL. Dans la base du socle (`sahel.duckdb`), ou directement sur le fichier :

```
SELECT categorie,
       COUNT(*)
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER (), 1)
             AS effectif,
             AS freq_pct
FROM read_csv('01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv', sep=';')
GROUP BY categorie
ORDER BY effectif DESC;
```

Sept lignes de requête, les mêmes sept résultats. L’exercice ne consiste pas à choisir le bon outil : il consiste à **ne pas avoir peur de vérifier avec deux**.

Étape 8 — Les manquants, nommés. `=NB.EUILL(F2:F481)` sur la colonne `client` renvoie 18 lignes vides ; 85 lignes portent la valeur 0 (le comptoir). Total : 103 lignes sans client identifié, soit 21,5 % de l’échantillon. Écrivez cette phrase dans la fiche, avec son pourcentage : un lecteur qui l’apprend en annexe la vit comme une cachisserie.

Étape 9 — Ce que le tirage écrase. Une ligne, à la fin de la fiche : « 40 lignes par mois : les volumes relatifs des mois ne sont pas comparables sur cet extrait. » C’est la seule ligne que personne d’autre écrira à votre place.

Étape 10 — Dater et signer. « Fiche établie le 17/09/2026 sur le fichier `ventes_magasin5_2025_ATTENDU.csv`, 480 lignes, empreinte non calculée. » La date et le nom du fichier sont ce qui rendra le document rejouable quand le fichier aura changé de nom, ce qui arrivera.

8. Erreurs fréquentes

Symptôme	Cause	Correction
Deux services publient des pourcentages différents sur « la même » données	l’individu n’est pas le même (lignes vs tickets vs clients)	nommer l’individu dans l’en-tête de chaque tableau ; le répéter dans la légende
Le total des pourcentages fait 99,6 % ou 100,4 %	arrondis cumulés, présentés sans mention	garder une décimale, additionner les valeurs affichées, mentionner l’arrondi si nécessaire
« La majorité des ventes sont des Matériaux »	modalité dominante confondue avec majorité absolue	exiger > 50 % pour « majorité » ; sinon écrire « première catégorie »
Un graphique en secteurs à 145 parts	cardinalité non consultée avant de choisir la visualisation	n’utiliser le camembert qu’à partir du moment où il y a moins de 6 modalités ; sinon un classement barres
Une conclusion sur « le magasin le plus performant » alors que l’extrait n’en contient qu’un	variable à cardinalité 1 prise pour une variable de comparaison	vérifier les cardinalités avant toute phrase comparative ; la fiche d’identité s’en charge
Des effectifs qui ne somment pas au total du fichier	lignes exclues silencieusement (filtre resté actif, en-têtes mal lus)	recalculer le total et l’afficher à côté de la somme des lignes du tableau

9. Bonnes pratiques professionnelles

- Écrivez la **phrase d'identité** en tête de tout livrable : population, individu, période, source, mode de tirage. Cinq segments, une ligne et demie.
- Un pourcentage sans dénominateur est une opinion : affichez l'effectif à côté, toujours.
- Comptez les valeurs distinctes **avant** de commenter une comparaison : une cardinalité de 1 ferme la question.
- Dites ce que le tirage écrase, en une phrase, même si personne ne l'a demandée.
- Gardez les lignes « sans client » dans les effectifs, et nommez-les : les exclure change le dénominateur et donc tous les pourcentages — ce choix doit être écrit, pas subi.
- Vérifiez un résultat par un second outil au moins une fois par dossier, et notez dans le journal que vous l'avez fait.
- Datez la fiche, nommez le fichier. Une fiche sans date est une légende urbaine.

10. Exercice guidé — la fiche d'identité, ensemble, en 25 minutes

Contexte. Votre responsable vous demande de « préparer le terrain » avant l'arrivée du consultant qui audite les pratiques de données du magasin. Il veut une page, pas un rapport.

Étape 1 — Fixer l'objet. Question posée : de quoi parle-t-on ? Réponse rédigée : « Des lignes de ticket facturées au magasin 5 de Sahel Distribution SA sur l'année 2025, telles que la caisse les a enregistrées. » Vous venez de nommer la population et l'individu. Notez-les dans le document, pas dans votre tête.

Étape 2 — Compter. Vous obtenez 480 lignes et 316 tickets. Pourquoi les deux chiffres ? Parce que la ligne est ce que le fichier contient, le ticket ce que le client a vécu, et que le consultant demandera lequel des deux a servi de base au calcul. Réponse écrite : « Effectif : 480 lignes de ticket ; 316 tickets ; 1,52 ligne par ticket en moyenne. »

Étape 3 — Cardinalités. 7 catégories, 4 vendeurs, 145 produits, 372 clients distincts (hors vides), 6 modalités de remise, 1 magasin. Que dit la dernière ? Que cet extrait ne peut rien dire sur la comparaison entre magasins — à écrire noir sur blanc, car c'est la question que le consultant posera en premier.

Étape 4 — Fréquences. Vous dressez le tableau des catégories (143, 112, 57, 57, 53, 34, 24 lignes), vous ajoutez la colonne des pourcentages, vous vérifiez que le total tombe à 100,0 %. Vous ajoutez une ligne « sans remise : 307 lignes, 64,0 % », parce que la remise est la variable qui fâche.

Étape 5 — Les manquants. 18 lignes sans client, 85 lignes au comptoir codées 0. Décision écrite : elles restent dans les effectifs (elles sont des ventes réelles), mais elles sortent de toute analyse par client — et donc, cette analyse-là porte sur 377 lignes, pas 480.

Étape 6 — La phrase de garde. « Extrait construit à raison de 40 lignes par mois : les volumes mensuels ne sont pas comparables entre eux sur ce fichier. » Cette phrase vaut à elle seule la fiche : elle désarme la mauvaise question au lieu de la traiter.

Résultat attendu : une page de dix lignes, sans un seul calcul de moyenne. Le consultant gagnera vingt minutes, vous en gagnerez trois semaines de crédibilité.

11. Exercices autonomes

Exercice 1.1 (★) — Les cinq lignes. Sur le fichier de 480 lignes, rédigez la phrase d'identité : population, individu, échantillon, variables, tirage. (attendu : 480 lignes, 316 tickets, 13 colonnes, 2025, 40 lignes par mois après tri par date ; la phrase fait 4 à 6 lignes)

Exercice 1.2 (★) — Deux dénombrements, deux questions. Calculez, en un tableur : le nombre de lignes, le nombre de tickets, le nombre de tickets comportant au moins deux lignes, et leur part. (attendu : $480 \cdot 316 \cdot 104 \cdot 32,9\%$)

Exercice 1.3 (★) — Le tableau des catégories. Effectifs et fréquences des 7 catégories, avec une décimale et la ligne de total. (attendu : 143 ; 112 ; 57 ; 57 ; 53 ; 34 ; 24 · 29,8 % ; 23,3 % ; 11,9 % ; 11,9 % ; 11,0 % ; 7,1 % ; 5,0 % · total 480 et 100,0 %)

Exercice 1.4 (★★) — La modalité qui interdit la conclusion. Une phrase par variable, pour dire ce qu'elle permet : magasin (1 modalité), vendeur (4), categorie (7), produit (145). (attendu : pas de comparaison possible entre magasins ; constat seulement sur 4 vendeurs ; répartitions et classements sur les catégories ; liste ou top-N sur les produits, jamais un graphique en secteurs)

Exercice 1.5 (★★) — Le dénominateur qui bouge. Calculez la part des lignes vendues sans remise, une fois sur les 480 lignes, une fois sur les 377 lignes portant un client identifié. Commentez les deux chiffres en deux phrases. (attendu : $307 / 480 = 64,0\%$ · $307 / 377 \approx 81,4\%$; le second chiffre parle des ventes identifiées, pas de l'ensemble ; changer de dénominateur sans le dire fabrique un désaccord)

12. Correction détaillée

Exercice 1.1. La phrase attendue tient en six segments : « Population : les lignes de ticket facturées au magasin 5 de Sahel Distribution SA en 2025. Individu : la ligne de ticket. Échantillon : 480 lignes, soit 0,2 % des 240 000 lignes propres du socle. Variables : 13 colonnes, de `n_ticket` à `montant_ttc`. Tirage : 40 lignes par mois après tri par date, ce qui égalise les mois. Source : `ventes_magasin5_2025_ATTENDU.csv`, fiche établie le 17/09/2026. » Ce qui est noté : la présence des cinq noms, et l'unité (lignes, pas ventes).

Exercice 1.2. Trois formules : `=NBVAL(A2:A481)` → 480 ; la formule de valeurs distinctes sur `n_ticket` → 316 ; `=NBVAL(A2:A481)/316` → 1,52. Pour les tickets multi-lignes, un tableau croisé de `n_ticket` en lignes avec « nombre » en valeur, puis un comptage des clés dont l'effectif dépasse 1 → 104, soit $104 / 316 = 32,9\%$. Le piège classique : compter les lignes supplémentaires ($480 - 316 = 164$) au lieu des tickets, ou diviser 104 par 480 au lieu de 316 — le dénominateur suit l'individu choisi, jamais la commodité : 104 tickets sur 316 ne se racontent pas « sur 480 ».

Exercice 1.3. Le tableau doit afficher, dans cet ordre décroissant : Matériaux 143 (29,8 %), Quincaillerie 112 (23,3 %), Consommables 57 (11,9 %), Plomberie 57 (11,9 %), Electricité 53 (11,0 %), Peinture 34 (7,1 %), Bois & panneaux 24 (5,0 %), total 480 (100,0 %). Deux égalités à vérifier : $143 + 112 + 57 + 57 + 53 + 34 + 24 = 480$ ✓, et $29,8 + 23,3 + 11,9 + 11,9 + 11,0 + 7,1 + 5,0 = 100,0$ ✓. À noter aussi : deux catégories sont à égalité parfaite à 57 lignes — un classement qui les départagerait sur la deuxième décimale inventerait une hiérarchie.

Exercice 1.4. Attendu, une ligne par variable : `magasin` — une seule modalité, donc aucune conclusion comparative possible sur cet extrait (la comparer supposerait d'ouvrir le fichier complet) ; `vendeur` — 4 modalités, un classement est descriptible mais instable (retirer un vendeur change tout) ; `categorie` — 7 modalités, répartitions et classements exploitables, graphique en barres horizontales ; `produit` — 145 modalités, on liste, on ne figure pas, et l'on travaille en top-N avec la part restante explicite (« autres : X lignes »).

Exercice 1.5. $307 / 480 = 64,0 \%$; $307 / 377 \approx 81,4 \%$. Commentaire attendu : le premier chiffre décrit l'ensemble des ventes ; le second ne décrit que les ventes où l'on sait à qui, et il est mécaniquement plus élevé parce que le comptoir, où la remise est rare, a quitté le dénominateur. Un chiffre qui bouge de 17 points par un changement de dénominateur doit être accompagné de son dénominateur à chaque fois qu'il est cité — sinon c'est un argument, plus une mesure.

13. Mini-projet M02.P1 — « La fiche d'identité statistique du fichier complet » (50 min)

Commande. Établissez, pour le fichier `01_socle_donnees/data/reference/ventes_propres.csv` (240 000 lignes, 4 années, 6 magasins), la fiche d'identité statistique au format du chapitre : population, individu, période couverte, variables et cardinalités, trois fréquences commentées, manquants nommés, tirage, date et source.

Livrables numérotés. (1) la fiche, une page ; (2) le tableau des effectifs et fréquences d'une variable au choix ; (3) le script ou les formules utilisés, collés en fin de page ; (4) deux phrases sur ce que la fiche empêche de conclure à tort.

Barème (20 points, seuil 13). Fiche complète, cinq rubriques (5) · effectifs et fréquences justes, total à 100,0 % (4) · cardinalités mesurées et non recopiées (3) · manquants chiffrés et nommés (3) · phrase sur le tirage ou la couverture (2) · traçabilité des calculs (2) · style : une page, pas deux (1). Le tout en 50 minutes chrono : si vous dépassez, votre tableau contient plus de lignes que nécessaire.

14. Résumé du chapitre

Statistiquement, un fichier ne dit rien de lui-même : il faut écrire ce qu'il représente. Cinq mots suffisent — population, individu, échantillon, variable, modalité — et un calcul : l'effectif, puis sa fréquence. Les trois quarts des disputes de chiffres naissent d'un individu mal nommé (lignes contre tickets contre clients), non d'une formule ratée. La cardinalité se mesure avant de commenter : à 1 modalité, la comparaison est fermée ; à 145, le graphique en secteurs est fermé. Les manquants restent dans le dénominateur tant qu'on ne l'écrit pas autrement, et un pourcentage ne se cite jamais sans son dénominateur. Enfin, le mode de tirage s'énonce avec son effet : ici, 40 lignes par mois signifient que les mois ne pèseront pas leur poids réel. Ce chapitre ne vous a pas appris à calculer une moyenne : il vous a appris à ne pas en calculer une sur le mauvais objet, ce qui est plus rare et plus utile.

15. À retenir

À retenir. 1. **Nommer l'individu avant tout pourcentage** : 480 lignes, 316 tickets, 372 clients — trois chiffres, tous justes, trois questions différentes. 2. **La population n'est pas le fichier** : l'extrait de 480 lignes couvre 0,2 % des 240 000 lignes propres, et le dire est ce qui rend le chiffre défendable. 3. **Une fréquence = un effectif ÷ un dénominateur** : citez les deux, vérifiez que les pourcentages additionnés retombent sur 100,0 %. 4. **La cardinalité commande** : à 1 modalité aucune comparaison, à 145 aucun camembert, à 7 toutes les répartitions. 5. **Le tirage s'écrit avec son effet** : « 40 lignes par mois » interdit de conclure sur le poids des mois.

À retenir (le réflexe qui vaut une heure de réunion). Une phrase d'identité en tête de chaque livrable : de quoi, sur quoi, sur quelle période, extrait comment, source laquelle. Elle ne coûte rien à écrire et rend le chiffre incontestable sur la forme — ce qui, dans une entreprise, est la moitié du travail.

16. Évaluation formative (auto-correction, 8 min)

1. Un fichier contient 480 lignes et 316 tickets. Que compte-t-on si l'on écrit « notre client moyen achète pour 114 156 FCFA » ?
2. Vrai ou faux : « puisque magasin ne contient qu'une valeur dans l'extrait, on peut comparer le magasin 5 aux autres en divisant par cinq ».
3. Un tableau de fréquences affiche 99,6 % au total. Deux causes possibles ?
4. Sur 307 lignes sans remise sur 480, puis sur 377 lignes identifiées : pourquoi les deux pourcentages (64,0 % et 81,4 %) sont-ils tous les deux corrects, et lequel cite-t-on au directeur financier ?
5. Citez les cinq rubriques de la phrase d'identité, sans relire.

Question ouverte. Un collègue écrit : « 29,8 % de nos ventes concernent les Matériaux, c'est donc la majorité de notre activité ». Reprenez la phrase en une ligne, en corrigeant ce qui doit l'être et en ajoutant ce qui manque.

Corrigé de l'évaluation formative.

1. **Rien de proprement dit.** 114 156 FCFA est le montant moyen par ticket (moyenne des 316 tickets) : le chiffre est un panier moyen, pas un « client moyen ». L'individu est le ticket ; il faut le dire, sinon le lecteur sur-achète (il croit savoir ce qu'un client dépense alors qu'il lit ce qu'un ticket contient).
2. **Faux.** Diviser une valeur unique par cinq invente quatre magasins. Soit on ouvre le fichier complet (240 000 lignes, 6 magasins), soit on écrit « sur le seul magasin 5 ».
3. **Arrondis cumulés** (le cas bénin : mentionner l'arrondi suffit) ou **lignes perdues dans le tableau** (modalité exclue, filtre resté actif, une catégorie absente du croisé) — la seconde cause est une erreur, la première une convention. On vérifie en additionnant les effectifs, pas les pourcentages : la somme doit faire 480.
4. Parce que les deux dénominateurs décrivent deux ensembles différents : toutes les ventes, et les ventes où l'on sait à qui. Au directeur financier, on cite le second **en nommant** le premier (« sur les 377 lignes identifiées, 81,4 % ; l'ensemble des lignes donne 64,0 % ») : un chiffre financier se discute sur ce qu'il couvre avant de se discuter sur sa valeur.
5. Population · individu · échantillon et période · variables et cardinalités · tirage et source (dats). Question ouverte — Réponse attendue : « les Matériaux représentent 143 lignes sur 480,

soit 29,8 % : c'est la première catégorie, pas une majorité (qui exige plus de 50 %) ; le chiffre porte sur les lignes de l'extrait 2025 du magasin 5, pas sur l'activité de l'entreprise ».

Module M02.C02 — Les centres : moyenne, médiane, mode ; minimum, maximum, étendue

Outil de ce chapitre : un tableur d'abord, SQL et Python en vérification. Durée indicative : 4 h. Niveau : N1.

L'idée du chapitre. Résumer 316 tickets par un seul nombre est une décision, pas un calcul. Ce chapitre pose les trois candidats — moyenne, médiane, mode — et les trois bornes — minimum, maximum, étendue — puis répond à la seule question qui vaille en entreprise : **lequel montre-t-on, et que cache-t-il ?** Sur le fichier du module, le même tableau de ventes donne un panier de 114 156 FCFA en moyenne et de 58 423 FCFA en médiane : ce n'est pas une erreur de calcul, c'est la forme des données qui parle.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Calculer** moyenne, médiane, mode, minimum, maximum et étendue d'une colonne numérique, dans un tableur, en SQL et en Python, et obtenir le même résultat par les trois voies.
2. **Choisir** le centre qui décrit une distribution donnée, en justifiant par sa forme (symétrique, asymétrique, à valeurs extrêmes, bornée).
3. **Annoncer** un centre sans jamais le séparer de son effectif, de son unité et de sa période.
4. **Démasquer** l'abus de moyenne : dire ce qu'elle doit aux valeurs extrêmes, et chiffrer l'écart avec la médiane.
5. **Utiliser** la moyenne pondérée quand les lignes n'ont pas le même poids, et expliquer la différence avec la moyenne simple.

2. Pourquoi cette notion est importante

Situation 1 — le directeur et le client moyen. On vous demande « combien dépense un client ? ». Vous sortez 114 156 FCFA. Le directeur calibre ses stocks sur ce chiffre, et six mois plus tard la moitié des clients repartent avec moins de la moitié de ce panier. Personne n'a menti : la moyenne est juste, mais **le client moyen n'existe pas** — sur les 316 tickets, 227 sont sous la moyenne, soit 71,8 % de l'échantillon. Une moyenne citée comme un cas typique produit une décision fausse à partir d'un chiffre vrai.

Situation 2 — la comparaison entre vendeurs. Classement brut du chiffre d'affaires par vendeur : Adama Bationo 11 370 350 FCFA sur 125 lignes, Moussa Ilboudo 6 796 535 FCFA sur 109 lignes. L'écart de total s'explique en grande partie par le nombre de lignes ; ramené à la ligne, Bationo est à 90 963 FCFA, Ilboudo à 62 354 FCFA. Avant de parler de performance, il faut savoir **ce que l'on compare** : un total, une moyenne, une médiane. Le chapitre 8 dira si cet écart est statistiquement défendable ; ici, on apprend déjà à ne pas confondre les trois chiffres.

Situation 3 — la remise moyenne qui n'est pas la remise consentie. La moyenne simple de la colonne `remise` vaut 1,85 %. La moyenne pondérée par les montants vaut 1,72 %. Les deux sont justes ; une seule répond à la question du directeur financier (« sur un franc facturé, com-

bien concède-t-on ? »). Un analyste qui ne connaît que la moyenne simple croit que l'entreprise remet plus qu'elle ne le fait.

Dans les faits. L'observation « la moyenne est au-dessus de la médiane sur des montants » est si fréquente qu'elle sert de test de cohérence : sur des données de ventes, une moyenne inférieure à la médiane signale presque toujours un défaut (retours non isolés, doublons, signe inversé). C'est un réflexe de contrôle, pas une loi.

3. Explication simple

Dix amis vont au restaurant. Neuf consomment un plat à 3 000 FCFA, le dixième commande un plateau à fruits de mer à 60 000 FCFA.

- La **moyenne** de la table est $(9 \times 3\,000 + 60\,000) \div 10 = 5\,700$ FCFA. Elle est juste, et elle ne décrit personne : aucun des dix convives n'a dépensé 5 700 FCFA.
- La **médiane** est 3 000 FCFA : c'est ce que paie la moitié de la table, au milieu de la file. Elle décrit la table, mais ne dit rien de l'addition totale.
- Le **mode** est 3 000 FCFA : c'est le montant qui revient le plus souvent.
- Le **minimum** est 3 000, le **maximum** 60 000, l'**étendue** 57 000 FCFA.

L'addition, elle, ne bouge pas : 87 000 FCFA. C'est la leçon du chapitre — **la moyenne porte l'argent, la médiane porte les gens**. Quand on parle de recettes, on prend la moyenne ; quand on parle du comportement habituel d'un client, on prend la médiane ; et l'on dit les deux si l'on ne veut pas être démenti.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Moyenne arithmétique — mean	Somme des valeurs divisée par l'effectif.	La présenter comme « le cas typique » ; elle est sensible aux extrêmes.
Médiane — median	La valeur qui partage la série en deux moitiés d'effectif égal.	Oublier de trier, ou interpoler sans le dire entre deux valeurs.
Mode — mode	La valeur (ou la modalité) la plus fréquente.	Chercher un mode sur des valeurs presque toutes distinctes : il est then artefact.
Minimum / Maximum	Les deux valeurs extrêmes observées.	Les confondre avec des bornes autorisées par le métier.
Étendue — range	Maximum moins minimum.	La résumer à elle seule la dispersion : un seul ticket suffit à l'exploser.
Moyenne pondérée — weighted mean	Moyenne où chaque valeur pèse selon un poids (montant, effectif).	Confondre poids et valeur : pondérer par la colonne elle-même est parfois ce qu'on veut, parfois une faute.
Valeur extrême — outlier	Observation très éloignée du reste, à vérifier avant de commenter.	Supprimer « pour que ce soit propre » sans journaliser la décision.
Trimée (moyenne) — trimmed mean	Moyenne calculée après retrait d'une proportion fixe des extrêmes.	La comparer à la moyenne sans dire quel pourcentage a été retiré.
Centre — measure of central tendency	Nombre unique situé « au milieu » des données.	En choisir un par habitude de logiciel plutôt que par la forme des données.

Français — English	Définition simple	Piège à éviter
Additivité — additivity	Propriété d'un total à se décomposer en sous-totaux cohérents.	Croire qu'une moyenne se décompose : la moyenne d'un ensemble n'est pas la moyenne des moyennes.

5. Cours approfondi

5.1 Les formules, sans mystère

Pour une série de n valeurs triées x_1 le x_2 le $\dots$ le x_n :

- **Moyenne** : $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$. Elle est **additive** : la somme des valeurs est exactement $n \times \bar{x}$. C'est la raison pour laquelle la moyenne est l'outil de la comptabilité et du budget.
- **Médiane** : valeur au rang $\frac{n+1}{2}$ après tri. Si n est pair, on prend la demi-somme des deux valeurs centrales — et on le dit, car ce nombre peut ne correspondre à aucune observation réelle.
- **Mode** : valeur de fréquence maximale. Une série peut n'avoir aucun mode utile (toutes les valeurs distinctes), ou plusieurs (bimodale : deux clientèles distinctes dans le même fichier).
- **Minimum, maximum, étendue** : $x_{\min}$, $x_{\max}$, $x_{\max} - x_{\min}$.

Définition. Moyenne arithmétique — arithmetic mean — le total réparti également. Sa propriété utile : multiplier la moyenne par l'effectif redonne le total, au kuru près. Sa propriété dangereuse : un seul individu extrême la déplace proportionnellement à sa taille, pas à sa fréquence.

Définition. Médiane — median — la valeur du milieu après tri. Elle ne se laisse pas déplacer par un extrême : doubler le plus gros ticket du fichier ne change pas d'un franc la médiane. C'est ce qui la rend lisible pour décrire « le cas courant », et inutilisable pour prévoir une recette.

Définition. Mode — mode — la valeur la plus fréquente. Seul centre calculable sur une variable qualitative (« la catégorie la plus vendue »), il est inexploitable sur une colonne presque continue : nos paniers modaux valent 6 797 et 8 260 FCFA, trois fois chacun sur 316 tickets : rien de spécial, sinon d'être tombés trois fois — et deux valeurs plutôt qu'une, ce qui est déjà une information (la série est multimodale).

5.2 Les trois centres sur le fichier du module : lire les écarts

Sur les 316 tickets de l'extrait nettoyé (480 lignes, magasin 5, 2025) :

Indicateur	Valeur	Ce qu'il permet de dire
Moyenne	114 156 FCFA	le ticket pèse en moyenne ce poids dans la recette totale
Médiane	58 423 FCFA	la moitié des tickets est en dessous, la moitié au-dessus
Mode(s)	6 797 et 8 260 FCFA (3 fois chacun)	deux modes, trois occurrences : aucun exploitable — à écarter explicitement du rapport
Minimum	-150 804 FCFA	un ticket de retour, pas une vente : à isoler, pas à commenter comme un client pauvre
Maximum	1 343 666 FCFA	un chantier, pas un ménage
Étendue	1 494 470 FCFA	la série s'étale sur plus de vingt-cinq fois la médiane (rapport max/médiane : 23,0)

La moyenne dépasse la médiane de 55 733 FCFA — un **rapport de 1,95**. Voilà la phrase à écrire dans une note : « le panier moyen est près de deux fois le panier médian, ce qui indique une

forte concentration de la recette sur peu de gros tickets ». Et sa conséquence opérationnelle, mesurée aussi : **les 10 % des tickets les plus gros portent 47,5 % du chiffre d'affaires de l'extrait**. On peut décider de traiter cette frange à part (délai, acompte, livreur dédié) ; on ne peut plus décider pour toute la clientèle d'après le panier moyen.

5.3 La médiane a un effet de seuil, la moyenne un effet de niveau

Deux propriétés à connaître pour ne pas se faire surprendre en réunion :

- **La médiane ne bouge presque pas** quand une valeur extrême change. Doubler le plus gros ticket du fichier (1 343 666 FCFA) ne déplace pas la médiane d'un franc ; la moyenne, elle, remonte d'environ 4 000 FCFA — 1 343 666 FCFA répartis sur 316 tickets — sans que le comportement habituel de qui que ce soit ait changé.
- **La moyenne se déplace exactement comme une somme** : si l'on retire du fichier 8 lignes de montants négatifs (les retours, dont le total fait basculer des dizaines de milliers de francs), la moyenne par ligne passe de 75 152 à 77 028 FCFA. C'est un autre calcul sur un autre périmètre, pas une correction du précédent.

D'où la règle de rédaction : **toute moyenne citée s'accompagne du périmètre** (avec ou sans retours, avec ou sans doublons, sur extrait ou sur fichier complet). Deux chiffres corrects et différents racontent la même réalité sous deux angles — à condition que l'angle soit nommé.

5.4 Moyenne simple, moyenne pondérée : la colonne `remise` le démontre

Sur 480 lignes, la colonne `remise` contient six modalités. Sa moyenne simple vaut 0,0185 : c'est « la remise moyenne **par ligne** ». La moyenne pondérée par les montants hors taxes vaut 0,0172 : c'est « la remise moyenne **par franc facturé** ». Le calcul :

$$\text{moyenne pondérée} = \frac{\sum_i \text{remise}_i \times |\text{montant_ht}_i|}{\sum_i |\text{montant_ht}_i|}$$

L'écart de 0,13 point (7 % en relatif) vient de ce que les grosses lignes sont moins remisées que les petites. Cette dissymétrie est une information de politique commerciale : si l'on veut protéger la marge sur les gros chantiers, la moyenne simple — celle que le magasin voit — est trompeuse.

Définition. Moyenne pondérée — weighted mean — moyenne dans laquelle chaque valeur compte pour un poids. À utiliser dès que les individus n'ont pas la même taille économique : prix, taux, remises, délais, taux de marge, taux de transformation.

Attention. La moyenne des moyennes n'est pas la moyenne. Reprendre les douze moyennes mensuelles de l'extrait et les moyenner donne un chiffre faux dès que les mois n'ont pas le même nombre de lignes. Sur notre extrait, le tirage a forcé l'égalité (40 lignes par mois) : c'est le seul cas où l'erreur est invisible. Ne prenez jamais cette habitude — agrégez à partir des montants, jamais à partir des pourcentages ou des moyennes intermédiaires.

5.5 Le cas d'école : tickets simples contre paniers composés

Les 316 tickets se séparent en 212 tickets d'une ligne et 104 à plusieurs lignes (32,9 %). Les centres explosent la moyenne générale :

Population	Effectif	Moyenne	Médiane
Tickets d'une seule ligne	212	80 546 FCFA	38 084 FCFA
Tickets de 2 lignes et plus	104	182 668 FCFA	130 461 FCFA

Un « panier moyen de 114 156 FCFA » est donc le mélange de deux comportements différents. La question professionnelle qui suit immédiatement : veut-on piloter le mix (faire passer un client de 1 à 3 lignes) ou le prix (augmenter le panier simple) ? La réponse change le centre à regarder : pour le mix, la médiane et la répartition ; pour le prix, la moyenne et son évolution.

Conseil professionnel. Dans une note, présentez le couple **moyenne + médiane** avec une seule phrase d'interprétation : « moyenne 114 156 · médiane 58 423 FCFA : la recette est tirée par une frange de gros tickets, le comportement courant est à 58 423 FCFA ». Un lecteur qui n'a que la moyenne sur-achète ; un lecteur qui n'a que la médiane sous-estime la recette. Les deux ensemble, et l'affirmation devient robuste.

5.6 Minimum et maximum ne sont pas des bornes métier

Le minimum de la colonne `montant_ttc`, à la ligne, vaut -150 804 FCFA : c'est un retour, pas une erreur de saisie. Le maximum de `quantite` vaut 68 dans le fichier nettoyé — et 14 000 dans l'énoncé : ce sont les 7 lignes dont la quantité a été multipliée par 1 000 à l'enregistrement, corrigées au chapitre M01.C04. Le minimum de `prix_unitaire_ht` est 675 FCFA, le maximum 40 550 FCFA : deux extrêmes plausibles dans un catalogue de quincaillerie, à confronter au fichier fournisseur si l'on veut en tirer une décision.

La règle : **un extrême se qualifie avant de se résumer**. Trois questions, dans l'ordre : est-ce une erreur de fabrication (le $\times 1\,000$) ? est-ce un autre objet métier (le retour) ? est-ce un cas réel et rare (le chantier à 1 343 666 FCFA) ? Selon la réponse, on corrige, on isole dans un segment, ou on garde et on mentionne.

Attention. Supprimer les extrêmes pour « nettoyer » détruit l'additivité de la moyenne : le total de l'entreprise ne change pas, mais la moyenne que vous citez ne correspond plus à aucun chiffre du système de caisse, et plus personne ne peut la retrouver. Si une moyenne sans extrêmes est utile (moyenne trimée), elle se cite **avec** sa règle de retrait et avec le total d'origine. Sur le fichier du module, la moyenne trimée des tickets (25 tickets hors borne supérieure de Tukey) tombe à 71 463 FCFA, la médiane à 54 044 FCFA : les deux se resserrent, c'est exactement ce que l'on attend d'un retrait d'extrêmes — et c'est ce qu'il faut écrire.

5.7 Calculer les six nombres dans les trois outils

Nombre	Tableur (Excel / LibreOffice Calc)	SQL (DuckDB)	Python (pandas)
Moyenne	<code>=MOYENNE(M2:M481)</code>	<code>AVG(montant_ttc)</code>	<code>df.montant_ttc.mean()</code>
Médiane	<code>=MEDIANE(M2:M481)</code>	<code>MEDIAN(montant_ttc)</code>	<code>df.montant_ttc.median()</code>
Mode	<code>=MODE.UNI(M2:M481)</code> (Excel) · <code>=MODE(...)</code> (Calc)	<code>MODE(montant_ttc) WITHIN GROUP (ORDER BY montant_ttc)</code>	<code>df.montant_ttc.mode()</code>
Minimum	<code>=MIN(M2:M481)</code>	<code>MIN(montant_ttc)</code>	<code>df.montant_ttc.min()</code>
Maximum	<code>=MAX(M2:M481)</code>	<code>MAX(montant_ttc)</code>	<code>df.montant_ttc.max()</code>
Étendue	<code>=MAX(...)-MIN(...)</code>	<code>MAX(...)-MIN(...)</code>	<code>df.montant_ttc.max()-df.montant_ttc.min()</code>
Nombre de valeurs	<code>=NB(M2:M481)</code>	<code>COUNT(montant_ttc)</code>	<code>df.montant_ttc.count()</code>

Un détail qui fait perdre une heure : en SQL, la fonction de médiane est une **fonction d'agrégat ordonnée** — là où `AVG` accepte n'importe quoi, la médiane exige de dire dans quel ordre on cherche le milieu. Et dans les deux tableurs, le mode s'appelle différemment selon la version et la suite (Excel 2021+ : `MODE.UNI`, `MODE.MULT` pour les modes multiples ; LibreOffice 24.8 : `MODE`,

MODE.MULT) : vérifiez le nom dans l'assistant de fonction avant de l'écrire dans une procédure destinée à durer.

Boîte à outils. Le contrôle minimal de tout centre : (1) =NB() doit redonner l'effectif attendu — sinon des valeurs texte sont silencieusement ignorées, et votre moyenne porte sur 478 lignes au lieu de 480 ; (2) la moyenne fois l'effectif doit redonner la somme, =MOYENNE(range)*NB(range) contre =SOMME(range) ; (3) pour la médiane, vérifier que l'effectif des valeurs strictement inférieures et celui des supérieures diffèrent d'au plus un ; (4) si un seul de ces contrôles échoue, le problème est dans la colonne, pas dans la formule.

6. Exemple concret : le même indicateur, quatre énoncés

Voici ce que produit un analyste pressé : « panier moyen : 114 156 FCFA ». Le même fait, rédigé pour durer :

1. **Sur quoi** : 316 tickets du magasin 5, exercice 2025, extrait de 480 lignes sur 240 000 du fichier complet.
2. **Combien** : moyenne 114 156 FCFA ; médiane 58 423 FCFA ; mode non retenu (6 797 FCFA, aucune fréquence exploitable sur cette colonne continue).
3. **Forme** : moyenne 1,95 fois la médiane ; les 10 % de tickets les plus gros portent 47,5 % du chiffre d'affaires de l'extrait ; 227 tickets (71,8 %) sont sous la moyenne.
4. **Décision** : piloter le prix sur la moyenne, piloter l'expérience client sur la médiane, et traiter à part les 25 tickets au-dessus de la borne haute (295 445 FCFA).

Quatre lignes, aucun chiffre inventé, et plus de place pour le « ce n'était pas ce que je voulais dire ».

7. Démonstration pas à pas : les six nombres, trois outils, un contrôle

Étape 1 — Le bon fichier, le bon périmètre. 01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv, 480 lignes, séparateur ;. On travaillera sur deux objets : la colonne montant_ttc par ligne et le montant par ticket (somme par n_ticket). Notez-le : deux séries, donc six nombres chacun.

Étape 2 — Construire la série « ticket » dans le tableur. Colonne N : la clé de ticket. Colonne O : =SOMME.SI(N:N;N2:M:M) somme les montants du même ticket. Colonne P : =SI(NB.SI(\$N\$2:\$N\$481;N2)=1;O2;"") ne garde que la première occurrence de chaque ticket. Une colonne auxiliaire, un compteur, et la série des 316 paniers est prête. C'est long de dix minutes et c'est exactement ce que le tableur sait faire ; au chapitre suivant, le même travail tiendra en une requête.

Étape 3 — Les six nombres par ticket. Sur la colonne des paniers : =MOYENNE(...) → 114 156 ; =MEDIANE(...) → 58 423 ; =MODE.UNI(...) → 6 797 ; =MIN(...) → -150 804 ; =MAX(...) → 1 343 666 ; =MAX(...)-MIN(...) → 1 494 470. Écrivez-les dans un tableau à deux colonnes : valeur, unité (FCFA).

Étape 4 — Les six nombres par ligne. Même opération directement sur montant_ttc : moyenne 75 152 FCFA, médiane 37 966 FCFA, minimum -150 804 FCFA, maximum 1 229 678 FCFA. Le fait que la moyenne par ligne soit inférieure à la moyenne par ticket est normal et attendu : une ligne est une fraction de ticket (1,52 ligne par ticket).

Étape 5 — Le contrôle d'additivité. `=MOYENNE(M2:M481)*NB(M2:M481)` doit donner 36 073 185 FCFA, soit le total de la colonne. Si l'écart dépasse un franc, vous avez un souci de périmètre (filtre actif, ligne texte ignorée), pas une erreur d'arrondi.

Étape 6 — La version SQL, en une requête.

```
WITH paniers AS (
  SELECT n_ticket, SUM(montant_ttc) AS panier, COUNT(*) AS nb_lignes
  FROM read_csv('01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv', sep=';')
  GROUP BY n_ticket
)
SELECT COUNT(*)                AS tickets,
   ROUND(AVG(panier))          AS moyenne,
   ROUND(MEDIAN(panier))       AS mediane,
   MIN(panier)                 AS minimum,
   MAX(panier)                 AS maximum,
   MAX(panier) - MIN(panier)    AS etendue,
   SUM(CASE WHEN nb_lignes = 1 THEN 1 ELSE 0 END) AS tickets_simple
FROM paniers;
```

Pour la suite du module : figez ces deux relations. Ouvrez DuckDB à la racine du dépôt (commande `duckdb`, sans argument) et créez deux vues. Les chapitres C03 à C06 écriront leurs requêtes contre `lignes_ventes` et `paniers`, et ils ne fonctionneront qu'après ces deux lignes.

```
-- une fois pour toutes dans la session, pour tout le module M02
CREATE OR REPLACE VIEW lignes_ventes AS
  SELECT * FROM read_csv('01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv',
    sep=';');

CREATE OR REPLACE VIEW paniers AS
  SELECT n_ticket, SUM(montant_ttc) AS panier, COUNT(*) AS nb_lignes
  FROM lignes_ventes
  GROUP BY n_ticket;
```

Un `CREATE VIEW` ne duplique pas les données : il enregistre une question. Vous gagnez trois choses — des requêtes courtes, un point unique où le fichier est nommé, et un signal immédiat le jour où le fichier change de nom ou de séparateur, parce que la vue refusera de se créer. Vérifiez une fois, tout de suite : `SELECT COUNT(*), ROUND(AVG(panier)) FROM paniers;` doit rendre 316 et 114 156.

Étape 7 — La version Python, et la moyenne pondérée.

```
import pandas as pd
d = pd.read_csv("01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv", sep=";")
pan = d.groupby("n_ticket").montant_ttc.sum()
print(pan.agg(["count", "mean", "median", "min", "max"]))
pond = (d.remise * d.montant_ht.abs()).sum() / d.montant_ht.abs().sum()
print("remise simple %.4f · pondérée %.4f" % (d.remise.mean(), pond))
```

Sortie attendue : 316 tickets · moyenne 114 156 · médiane 58 423 · -150 804 · 1 343 666 ; remise 0,0185 et 0,0172. Les trois outils tombent d'accord, la note est défendable.

Étape 8 — Les catégories, sans y passer la journée. `GROUP BY categorie` sur les lignes : moyenne et médiane par catégorie, en une colonne de plus. Bois & panneaux donne 201 865 FCFA de moyenne pour 77 904 FCFA de médiane : cette catégorie est tirée par ses gros chantiers. Peinture : 86 422 contre 68 738 — les deux sont proches, la distribution est régulière. Voilà comment, en trente secondes, un classement de catégories devient une interprétation.

Étape 9 — Écrire le commentaire. Deux phrases, pas plus : une sur la position (« le panier médian est à 58 423 FCFA »), une sur la concentration (« la moyenne, 114 156 FCFA, est tirée par 10 % de tickets qui font 47,5 % du CA »).

Étape 10 — Journaliser. Une ligne dans `06_analyse/journal_transformations.txt` : la source, les six nombres, l'outil, la date. Ce qui prend vingt secondes aujourd'hui vous évitera trois quarts d'heure de fouille en janvier.

8. Erreurs fréquentes

Symptôme	Cause	Correction
« Notre client moyen » n'existe nulle part	la moyenne prise pour le cas typique	citer moyenne et médiane, et dire laquelle porte la décision
La moyenne change quand on ajoute une ligne, la décision change avec	aucun contrôle de la forme de la distribution	regarder le rapport moyenne/médiane avant de commenter ; ici 1,95
Le mode affiché est une valeur unique apparue deux fois	mode calculé sur une colonne quasi continue	réserver le mode aux variables qualitatives ou aux colonnes à faible cardinalité
Deux services n'ont pas la même moyenne, aucune erreur de calcul	périmètres différents (retours, doublons, extrait)	nommer le périmètre dans l'intitulé de la cellule, pas en note de bas de page
<code>=MEDIANE()</code> renvoie une erreur ou un texte	valeurs non numériques dans la plage	<code>=NB()</code> contre <code>=NBVAL()</code> ; la différence compte les lignes texte
L'étendue explose après un correctif	minimum négatif non expliqué (retour)	séparer retours et ventes avant de résumer, et le dire

9. Bonnes pratiques professionnelles

- Un centre sans effectif n'est pas une statistique : 114 156 FCFA (n = 316 tickets), toujours.
- Citez l'unité et la période dans le nom de la colonne du tableau, pas dans la légende du graphique.
- Sur des montants, partez du principe que la moyenne sera au-dessus de la médiane ; vérifiez, et commentez le rapport.
- Ne supprimez jamais un extrême sans écrire la règle, le nombre de lignes retirées et le total perdu.
- Pour une variable à 4 ou 5 valeurs (un vendeur, un type de client), le mode et l'effectif par modalité disent plus qu'une moyenne.
- Gardez une colonne « moyenne pondérée » dans vos modèles quand la variable est un taux ou un prix.
- Conservez dans le journal la requête qui a produit les nombres : une note sans requête est une opinion datée.

10. Exercice guidé — le trio moyenne / médiane / mode, ensemble

Consigne. Sur la série des 316 paniers, produisez le tableau des six nombres, puis écrivez les deux phrases de commentaire. Comptez 30 minutes, dont 20 de tableau.

Étape 1 — Isoler la série. Vous sommez par ticket et vous ne gardez qu'une occurrence par ticket, comme en §7 étape 2. Contrôle : `=NB()` sur la colonne obtenue doit faire 316. Si vous trouvez 480, la déduplication n'a pas fonctionné et tout ce qui suit sera faux — c'est le moment de le découvrir, pas après la note.

Étape 2 — Les six nombres. Vous les placez dans un tableau à trois colonnes : indicateur, valeur, unité. Vous obtenez 114 156 · 58 423 · 6 797 · -150 804 · 1 343 666 · 1 494 470 FCFA.

Étape 3 — Le mode, et sa mise à l'écart. Le tableur renvoie 6 797 FCFA, et la fonction des modes multiples en ajoute un second à 8 260 FCFA. Vous demandez « combien de fois ? » : `=NB.SI()` répond trois fois sur 316, pour l'un comme pour l'autre. Décision écrite : mode non retenu, avec sa raison. C'est une compétence, pas une résignation.

Étape 4 — Le rapport qui parle. `=114156/58423` → 1,95. Commentaire : « la moyenne est près de deux fois la médiane : la recette est concentrée ». Vous cherchez la concentration : triez décroissant, additionnez les 32 premiers tickets (10 % de 316 arrondi), divisez par le total → 47,5 %. Voilà l'indicateur que le directeur gardera en tête, pas le panier moyen.

Étape 5 — Le retournement. Refaites la moyenne en excluant le plus gros ticket : vous voyez la moyenne descendre nettement, la médiane presque pas. Écrivez cette phrase dans la note : elle explique à elle seule pourquoi deux services peuvent avoir raison avec des chiffres différents.

Étape 6 — Vérification croisée. Relancez la requête SQL de l'étape 6. Les six nombres doivent être identiques au franc près (aux arrondis d'affichage près). S'ils diffèrent, c'est l'encodage ou le séparateur, pas la statistique.

11. Exercices autonomes

Exercice 2.1 (★) — Le tableau de base. Calculez les six nombres de la série des paniers, et le même tableau pour la colonne `montant_ttc` par ligne. (attendu : 114 156 · 58 423 · 6 797 · -150 804 · 1 343 666 · 1 494 470 FCFA ; par ligne 75 152 · 37 966 FCFA, minimum -150 804, maximum 1 229 678)

Exercice 2.2 (★) — Combien sous la moyenne. Comptez les tickets strictement inférieurs à la moyenne, donnez leur part. Concluez en une phrase sur l'expression « client moyen ». (attendu : 227 tickets sur 316, 71,8 %)

Exercice 2.3 (★★) — Simples contre composés. Séparez les 212 tickets d'une ligne des 104 autres ; moyenne et médiane de chaque groupe. Rédigez la phrase que vous diriez au directeur des ventes. (attendu : 80 546 / 38 084 FCFA et 182 668 / 130 461 FCFA ; conclusion : deux clients, un seul fichier)

Exercice 2.4 (★★) — Pondérée, pas simple. Moyenne simple et moyenne pondérée par le `montant_ht` de la colonne `remise`, puis la même paire restreinte aux lignes dont la catégorie est « Matériaux ». Expliquez ce que dit la différence. (attendu sur l'ensemble : 0,0185 et 0,0172 ; la pondérée est inférieure : les fortes lignes sont moins remisées)

Exercice 2.5 (★★★) — Le chiffre qui change sans raison. Recalculez la moyenne des paniers en incluant les lignes de retour (montants négatifs), puis en les isolant dans un segment « avoirs ». Donnez les deux moyennes, la médiane dans les deux cas, et écrivez la règle métier que vous avez suivie. (attendu : la moyenne baisse quand on inclut les retours, la médiane quasiment pas ; règle écrite = les retours ne sont pas des ventes, ils sont un objet à part, et le total reste celui de la caisse)

12. Correction détaillée

Exercice 2.1. Tableaux attendus, avec l'effectif affiché à chaque ligne : paniers $n = 316$ — moyenne 114 156, médiane 58 423, modes 6 797 et 8 260 (non retenus : 3 occurrences chacun), min -150 804, max 1 343 666, étendue 1 494 470 FCFA ; lignes $n = 480$ — moyenne 75 152, mé-

diane 37 966 FCFA. Contrôle d'additivité sur les lignes : la moyenne multipliée par l'effectif doit retomber sur le total de la colonne, 36 073 185 FCFA, à l'arrondi d'affichage près — c'est la raison pour laquelle on calcule avec deux décimales et on affiche en entiers.

Exercice 2.2. `=NB.SI(0:0;"<114156")` → 227, soit $227 \div 316 = 71,8 \%$. Commentaire attendu : « dans une distribution asymétrique, la majorité des observations est sous la moyenne ; parler de “client moyen” pour décrire le client courant est un contresens — le client courant est à 58 423 FCFA ». Le point de méthode noté : avoir vérifié que $316 - 227 = 89$ tickets sont au-dessus (les 28,2 % qui portent la recette), et l'avoir écrit.

Exercice 2.3. Moyennes 80 546 et 182 668 FCFA ; médianes 38 084 et 130 461 FCFA. Ratio interne : dans le groupe des tickets simples, la moyenne est 2,11 fois la médiane — le déséquilibre n'a pas disparu en découpant, il s'est déplacé. Phrase attendue : « le panier moyen de 114 156 FCFA mélange deux objets : un achat simple à 80 546 FCFA et un panier composé à 182 668 FCFA ; si l'objectif est de faire passer un client d'un article à plusieurs, l'indicateur de pilotage est la part des tickets composés (32,9 %), pas le panier moyen ».

Exercice 2.4. Ensemble : simple 0,0185, pondérée 0,0172 (formule `=SOMMEPROD(remise;|montant_ht|)/SOMME(|montant_ht|)`). Sur « Matériaux » : les deux se resserrent, la catégorie étant peu remise et de montants homogènes. Lecture attendue : « la moyenne simple surestime la remise réelle de 1,3 point relatif, parce que les petites lignes sont les plus remises ; la politique de remise touche donc les petits clients, pas les chantiers ». Cette phrase est une décision possible : baisses de remise sur les petites lignes, marge préservée.

Exercice 2.5. Avec retours inclus, la moyenne des paniers est celle annoncée (§5.2) : 114 156 FCFA — elle contient trois tickets négatifs, dont un à -150 804 FCFA. En isolant les avoirs, la moyenne des ventes se resserre et la médiane reste à 58 423 FCFA, pratiquement inchangée : la médiane ne « voit » pas les trois cas extrêmes, la moyenne les absorbe. Règle attendue, écrite en une phrase : « un avoir n'est pas une vente : il est compté à part, et le total de caisse reste le total de caisse ». Le barème valorise la règle, pas le calcul.

13. Mini-projet M02.P2 — « Deux pages, six nombres, une décision » (45 min)

Commande. Pour la direction commerciale du magasin 5, produisez deux pages : (1) le tableau des six nombres, sur les paniers et sur les lignes, avec effectifs, unités et périmètre ; (2) une page de décision qui utilise moyenne, médiane et concentration pour trancher **une** question au choix : faut-il augmenter le seuil de livraison gratuite ? faut-il un tarif dédié aux tickets au-dessus de 300 000 FCFA ? faut-il recentrer la remise sur les petites lignes ?

Livrables numérotés. (1) tableau ; (2) page de décision avec le chiffre retenu, celui qui est écarté et pourquoi ; (3) la requête SQL ou le classeur de calcul en annexe ; (4) la ligne de journal.

Barème (20 points, seuil 13). Six nombres exacts et sourcés (5) · périmètre et unité écrits partout (3) · choix du centre justifié par la forme, pas par l'habitude (4) · chiffre de concentration calculé et non recopié (3) · décision datée avec un destinataire (3) · annexe rejouable (2) — total 20.

14. Résumé du chapitre

Trois centres, trois usages : la moyenne porte l'argent et se multiplie par l'effectif pour retrouver le total ; la médiane porte les gens et résiste aux extrêmes ; le mode ne sert que sur du dis-

cret. Deux bornes et une étendue disent l'étalement, pas la fréquence : le minimum de notre série est négatif parce que les retours sont là, et le maximum est un chantier, pas un client. Le rapport moyenne/médiane est votre premier diagnostic de forme — ici 1,95, donc asymétrie marquée, donc 71,8 % des tickets sous la moyenne. La moyenne pondérée est obligatoire dès qu'une variable est un taux ou un prix. Enfin, aucun centre ne se cite sans effectif, unité, période et périmètre : c'est ce qui rend un chiffre défendable quand deux services le calculent chacun de leur côté.

15. À retenir

À retenir. 1. **Moyenne $\times$ effectif = total** : c'est sa définition et sa seule promesse. Ne lui demandez pas de décrire le cas courant. 2. **Médiane = la moitié, pas le milieu du total** : 58 423 FCFA ici, insensible au ticket géant de 1 343 666 FCFA. 3. **Regardez le rapport, pas les valeurs seules** : moyenne $\div$ médiane = 1,95 annonce la concentration (47,5 % du CA sur 10 % des tickets). 4. **Mode uniquement sur du discret** : 6 797 FCFA est une valeur, pas une information. 5. **Taux et prix $\rightarrow$ moyenne pondérée** : 1,72 % de remise réelle contre 1,85 % de moyenne simple.

À retenir. La formulation qui protège. Dans toute note : « moyenne X FCFA, médiane Y FCFA, sur n objets, périmètre ... ». Celui qui écrit cette ligne ne se fait plus reprendre sur le sens d'un chiffre.

16. Évaluation formative (auto-correction, 10 min)

1. Un tableau de bord affiche « panier moyen : 114 156 FCFA » et « panier médian : 58 423 FCFA ». Un directeur conclut : « la moitié de nos clients dépensent plus de 100 000 FCFA ». Qu'y a-t-il à reprendre ?
2. Calculez de tête le nombre de tickets sous la moyenne si 28,2 % sont au-dessus, et dites ce que ce déséquilibre implique pour une politique de prix.
3. Pourquoi le mode est-il écarté de ce chapitre sur la colonne des montants, et sur quelle colonne du même fichier serait-il pertinent ?
4. Moyenne simple 0,0185, moyenne pondérée 0,0172. Lequel des deux chiffres un directeur financier doit-il entendre, et lequel un chef de magasin ?
5. On retire les 25 tickets dépassant la borne de 295 445 FCFA. Que deviennent la moyenne, la médiane et le total ? Lequel de ces trois changements doit être signalé dans la note ?

Corrigé de l'évaluation formative.

1. Deux erreurs. D'abord une confusion de centre : la médiane (58 423 FCFA) est le seuil de la moitié, pas la moyenne. Et le seuil des 100 000 FCFA n'est pas davantage une moitié : 30,4 % des tickets le dépassent (96 tickets sur 316), 28,2 % dépassent la moyenne.
2. $316 \times (1 - 0,282) \approx 227$ tickets sous la moyenne. Conséquence : fixer un tarif ou un seuil de service « sur la moyenne » exclut sept tickets sur dix ; les paliers se calibrent sur des percentiles (chapitre 4).
3. Parce que 316 montants presque tous distincts produisent un mode sans signification : deux valeurs, 6 797 et 8 260 FCFA, trois occurrences chacune. Sur `categorie` ou `remise`, le mode est utile : « Matériaux » avec 143 lignes sur 480, ou remise 0,04 avec 110 lignes sur 480.

4. Le directeur financier entend la **pondérée** (1,72 %), parce qu'elle dit ce que coûte la remise sur chaque franc facturé. Le chef de magasin entend la **simple** (1,85 %), parce qu'elle dit ce qui se pratique sur une ligne — et donc ce qu'il doit rappeler aux vendeurs.
5. La moyenne baisse nettement (vers 71 463 FCFA sur les tickets restants), la médiane descend à 54 044 FCFA, le total du fichier, lui, baisse de tout ce qui a été retiré. C'est **le total** qui doit être signalé : une moyenne sans extrêmes est un outil de description, pas un élément de compte de résultat ; écrire l'un sans l'autre, c'est créer un chiffre que la comptabilité ne reconnaîtra pas.

M02.C03 — Mesurer la dispersion : étendue, intervalle interquartiles, écart-type

L'idée du chapitre. Deux séries peuvent avoir la même moyenne et n'avoir rien de commun. La dispersion est ce qui les sépare dans une note, et c'est elle — pas le centre — qui décide si un indicateur est utilisable pour négocier, alerter ou comparer.

Base de travail — obligatoire. Deux fichiers du socle, dans votre dossier 02_exercices/M02/, ouverts côte à côte : `donnees/projection/ventes_magasin5_2025.csv` (l'énoncé, 489 lignes × 13 variables) et `donnees/reference/ventes_magasin5_2025_ATTENDU.csv` (480 × 13). Séparateur point-virgule, encodage UTF-8, virgule décimale. Montants en franc CFA (FCFA). Tous les chiffres cités sont lus sur l'attendu, au niveau de la ligne (480 lignes) et du panier (316 tickets), et figurent dans `01_socle_donnees/data/reference/chiffres_cites.md`, section M02.

1. Objectifs du chapitre

À la fin de ce chapitre, vous êtes capable de :

1. calculer et comparer les trois dispersions usuelles — étendue, intervalle interquartiles, écart-type — sur la même série, et dire laquelle répond à quelle question ;
2. expliquer pourquoi l'écart-type d'une série d'entiers s'écrit avec des décimales alors qu'aucune valeur ne les contient ;
3. choisir entre écart-type et coefficient de variation pour comparer la variabilité de deux colonnes qui n'ont pas la même échelle ;
4. estimer la précision d'une moyenne grâce à l'erreur type, et dire si un écart entre deux vendeurs mérite une réunion ;
5. décider, chiffres à l'appui, si une catégorie de produits est assez stable pour être pilotée sur sa moyenne ;
6. rédiger une note de dispersion qui ne laisse pas le lecteur deviner de quoi l'on parle.

2. Pourquoi cette notion est importante

Un indicateur sans dispersion est une décision prise à l'aveugle. Trois situations du module le montrent, toutes chiffrées plus bas dans ce chapitre :

- **La négociation.** « Le montant moyen d'une ligne de Bois & panneaux est de 201 865 FCFA, accordons 5 % ». Si l'écart-type de la catégorie vaut 318 080 FCFA, la moyenne ne décrit rien : la remise porte sur un chiffre qui n'existe pas.
- **L'alerte.** Le contrôle de gestion veut être prévenu quand un ticket sort de l'ordinaire. Le seuil retenu change tout : à une fois et demie l'écart interquartiles, il vaut 295 445 FCFA et signale 25 tickets ; à deux écarts-types, il ne signale qu'une douzaine de tickets et laisse filer tout le reste.
- **Le classement.** L'écart de panier moyen entre les deux vendeurs les plus éloignés des quatre principaux vaut 34 014 FCFA. Sans erreur type, c'est un classement ; avec elle (10 244 FCFA), c'est une comparaison qui tient ou qui tombe.

Dans les faits. Le vocabulaire de la dispersion est le premier endroit où un non-statisticien est disqualifié en réunion, souvent injustement. Deux réflexes vous mettent à l'abri : ne jamais prononcer un écart-type sans la moyenne qui l'accompagne, et ne jamais parler de « variance » à l'oral — dites « écart-type », tout le monde voit de quoi il retourne, et vous aussi.

3. Explication simple

Imaginez deux caisses du magasin. À la première, dix clients paient des tickets tous proches de 60 000 FCFA. À la seconde, dix clients encore : cinq paient 5 000 FCFA, cinq paient 115 000 FCFA. Les deux files ont la même moyenne. La première est réglée, la seconde est un tirage.

La dispersion est exactement ce qui distingue les deux files, et elle répond à une question unique : **les valeurs sont-elles serrées autour du centre, ou étalées ?** Trois manières de le mesurer, de la plus naïve à la plus utile :

- **l'étendue** regarde les deux bouts — 1 494 470 FCFA sur nos tickets : vrai, et inutile ;
- **l'intervalle interquartiles (IQR)** regarde la moitié du milieu — 109 924 FCFA, et résiste aux extrêmes ;
- **l'écart-type** regarde toutes les valeurs, chacune comptant pour sa distance au centre — 182 095 FCFA, et sert de matière première à tout le reste du module.

Le mot « dispersion » recouvre donc trois chiffres très différents. En anglais spread, dispersion, variability : le concept est le même, et le mot employé dépend du logiciel plus que du théorème.

4. Vocabulaire essentiel

#	Terme	Français — English — sens simple	À savoir
1	Étendue	étendue — range — plus grande valeur moins plus petite	deux lignes du fichier décident de tout
2	IQR	intervalle interquartiles — interquartile range — Q3 moins Q1	« écart interquartile » en français
3	Variance	variance — variance — moyenne des carrés des écarts	unité au carré, donc peu lisible
4	Écart-type	écart-type — standard deviation — racine carrée de la variance	σ pour une population, s pour un échantillon
5	CV	coefficient de variation — coefficient of variation — écart-type divisé par la moyenne	sans unité, seul comparable entre colonnes
6	Erreur type	erreur type de la moyenne — standard error (SE) — écart-type divisé par racine de n	précision d'un indicateur, pas diversité des clients
7	MAD	écart absolu médian — median absolute deviation — médiane des écarts à la médiane	robuste, absente des tableurs
8	Moyenne tronquée	moyenne tronquée — trimmed mean — moyenne après retrait des extrêmes	change le périmètre, pas le calcul
9	Bornes de Tukey	bornes de Tukey — Tukey fences — Q1 — $1,5 \times \text{IQR}$ et Q3 + $1,5 \times \text{IQR}$	définit l'extraordinaire sans le subir
10	Degré de liberté	degré de liberté — degree of freedom — valeurs libres après estimation	explique le $n - 1$ de la variance
11	Hétérogénéité	hétérogénéité — heterogeneity — mélange de populations dans un même chiffre	le vrai danger, pas la variance

5. Cours approfondi

5.1 Trois dispersions, trois usages

Définition. L'**étendue** (range) est la différence entre maximum et minimum. L'**intervalle inter-quartiles** (IQR) est la différence entre le troisième et le premier quartile. L'**écart-type** est la racine carrée de la **variance**, moyenne des carrés des écarts à la moyenne. Les deux premières se lisent sur la distribution, la troisième sur toutes les valeurs à la fois.

Dispersion	Sur les paniers (316)	Sur les lignes (480)	Ce qu'elle autorise à dire
Étendue	1 494 470 FCFA	1 380 482 FCFA	« du pire au meilleur ticket » — exact, non décisionnel
IQR	109 924 FCFA	à calculer à l'exercice 3.1	« la moitié du milieu tient dans... »
Écart-type	182 095 FCFA	143 915 FCFA	« un ticket s'écarte typiquement de... »
CV	159,5 %	191,5 %	« la moyenne ne décrit pas la série »

Sur 316 valeurs très asymétriques, l'écart-type dépasse nettement l'IQR (rapport de 1,7) : c'est la signature arithmétique de l'asymétrie, que M02.C05 confirmera graphiquement. L'étendue, elle, vaut plus de treize fois l'IQR (rapport mesuré : 13,6) — les 300 tickets du centre tiennent dans un espace treize fois plus petit que celui que la série occupe au total.

Faites-le dans un tableur. Sur la colonne des paniers : `=MAX(plage)-MIN(plage)` pour l'étendue, `=QUARTILE.INC(plage;3)-QUARTILE.INC(plage;1)` pour l'IQR, `=ECARTYPE.STANDARD(plage)` pour l'écart-type. La fonction s'appelle `STDEV.S` en anglais, `ECARTYPE.STANDARD` en français ; la variante `ECARTYPEP` (`STDEV.P`) suppose que vous tenez la population entière. Gardez la version échantillon : vous travaillez sur un extrait de 0,23 % du fichier du magasin.

Attention. Une étendue est une information sur deux lignes. Un seul ticket à 1 343 666 FCFA ou un seul avoir à -150 804 FCFA la change, sans que personne d'autre n'ait changé de comportement. C'est la plus sensible des trois dispersions à l'erreur de saisie, donc la moins bonne candidate à un seuil d'alerte.

5.2 D'où vient l'écart-type, et pourquoi il a des décimales

Quatre pas, tous faisables à la main sur dix lignes :

1. écart de chaque valeur à la moyenne : $x_i - \text{moyenne}$;
2. carré de cet écart ;
3. somme des carrés divisée par $n - 1 \rightarrow$ la **variance** ;
4. racine carrée de la variance $\rightarrow$ l'**écart-type**.

Sur les 316 paniers, la variance vaut 33 158 631 393 — des FCFA au carré, unité qui n'existe pas dans la nature — et sa racine carrée 182 095 FCFA. Le passage par les carrés explique deux choses. D'abord la **sensibilité** : un ticket de 1 343 666 FCFA pèse sur la variance proportionnellement au carré de son écart, donc des milliers de fois plus qu'un ticket ordinaire. Ensuite le **format** : extraire une racine carrée produit des décimales, même quand les données sont des entiers — la dispersion d'une série d'entiers n'a aucune raison d'être un entier.

Faites-le dans un tableur, en trois colonnes, pour voir le mécanisme plutôt que le résultat : écart en A2 par `=B2-MOYENNE(panier)`, carré en C2 par `=A2^2`, variance par `=SOMME(C2:C317)/`

(NB(B2:B317)-1), écart-type par =RACINE() du résultat. Vous venez de reconstituer VAR.S et ECARTYPE.STANDARD : comparez, vous devez retrouver 182 095 au franc près.

Le dénominateur $n - 1$ mérite deux phrases. Il s'appelle le **degré de liberté** (degree of freedom) : une valeur sur 316 a déjà servi, la moyenne, et n'est plus libre. On divise par $n - 1$ quand la moyenne a été calculée sur le même échantillon ; sans cette correction, la variance serait systématiquement trop petite. Sur 316 lignes, l'écart entre les deux conventions vaut un facteur de 1,0016 : négligeable en valeur, décisif en cohérence — tout le module raisonne en échantillon, y compris le test du chapitre M02.C08.

5.3 L'IQR et la règle des 1,5 : une borne qui résiste à ce qu'elle détecte

Définition. Les **bornes de Tukey** (Tukey fences, « moustaches » ou limites externes en français) sont $Q1 - 1,5 \times IQR$ et $Q3 + 1,5 \times IQR$. Une valeur est dite **aberrante au sens de Tukey** si elle les franchit. Le coefficient 1,5 est une convention, pas une loi : on le porte à 3,0 pour ne signaler que l'exceptionnel.

Sur les paniers : $Q1 = 20\,635$, $Q3 = 130\,559$, $IQR = 109\,924$, et $1,5 \times IQR = 164\,886$. La borne haute vaut donc 295 445 FCFA, la borne basse -144 251 FCFA. Vingt-cinq tickets franchissent la haute ; un seul franchit la basse, et c'est le retour de 150 804 FCFA rencontré au chapitre M02.C02.

Comparez avec une borne « à deux écarts-types », soit $\pm 364\,190$ FCFA autour de la moyenne. D'un côté la borne basse descend bien au-dessous de tout ce que le fichier contient — la règle ne signale jamais rien en bas ; de l'autre la borne haute ne rattrape qu'une douzaine de tickets et laisse filer les autres. Voilà, en deux lignes de calcul, pourquoi une borne de détection doit être bâtie sur un indicateur robuste : une règle qui intègre les détectés dans son calcul se laisse fixer par eux.

Faites-le dans le SQL. (les vues `lignes_ventes` et `paniers` ont été créées au C02, § « Pour la suite du module ») :

```
WITH q AS (
  SELECT percentile_cont(0.25) WITHIN GROUP (ORDER BY panier) AS q1,
         percentile_cont(0.75) WITHIN GROUP (ORDER BY panier) AS q3
  FROM paniers
)
SELECT ROUND(q1 - 1.5 * (q3 - q1)) AS borne_basse,
       ROUND(q3 + 1.5 * (q3 - q1)) AS borne_haute,
       q3 - q1                    AS iqr
FROM q;
```

Conseil professionnel. Une borne basse négative sur des montants est un diagnostic, pas un résultat : elle dit que la série est si large que rien ne peut y être « anormalement petit ». Dites-le en réunion — « aucune alerte possible en bas » — plutôt que de supprimer la borne ou de la plaquer à zéro en silence.

5.4 Comparer deux colonnes : le coefficient de variation

Définition. Le **coefficient de variation** (CV, coefficient of variation) est l'écart-type divisé par la moyenne, exprimé en pourcentage. Il est sans unité, donc comparable entre séries d'échelles différentes ; il n'a aucun sens si la moyenne est proche de zéro, ni sur une grandeur sans rapport (un code client, une température en Celsius).

Série	Moyenne	Écart-type	CV	Lecture
Panier (316 tickets)	114 156 FCFA	182 095 FCFA	159,5 %	dispersion très supérieure au centre
Montant par ligne (480 lignes)	75 152 FCFA	143 915 FCFA	191,5 %	plus large encore
Pièces par ligne (480 lignes)	7	8,4	119,7 %	même nature de désordre

Les trois CV dépassent 100 %. Ce n'est pas une anomalie de l'extrait : c'est la nature d'un commerce de matériaux, où une commande de chantier côtoie trois achats de vis. La grille de lecture d'usage — une convention d'équipe, pas un théorème : CV inférieur à 15 %, série stable, la moyenne se suffit ; de 15 à 30 %, moyenne utile à accompagner de l'écart-type ; au-delà de 50 %, la moyenne seule ne décrit plus rien, il faut montrer la distribution (M02.C05).

5.5 Par catégorie : où la moyenne est-elle descriptive ?

Voici le tableau qu'un acheteur réclame, et celui qui clôt une discussion. Sur les lignes, par CV croissant, avec l'effectif de chaque catégorie :

Catégorie	Lignes	Moyenne (FCFA)	Écart-type (FCFA)	CV
Peinture	34	86 422	70 320	81,4 %
Consommables	57	43 877	45 130	102,9 %
Matériaux	143	45 590	49 286	108,1 %
Quincaillerie	112	37 528	45 358	120,9 %
Bois & panneaux	24	201 865	318 080	157,6 %
Plomberie	57	153 596	247 146	160,9 %
Electricité	53	119 088	202 897	170,4 %

Trois lectures professionnelles :

1. **Peinture contre Electricité** : des moyennes proches (86 422 et 119 088 FCFA), des écarts-types sans rapport (70 320 contre 202 897 FCFA). Piloter ces deux rayons avec le même indicateur, c'est piloter mal le second.
2. **Consommables contre Quincaillerie** : des écarts-types presque identiques (45 130 et 45 358 FCFA) mais des CV différents (102,9 % contre 120,9 %), parce que les niveaux diffèrent. Cas d'école de l'écart-type qui ment et du CV qui dit vrai.
3. **Bois & panneaux** : 24 lignes, une moyenne de 201 865 FCFA, un CV de 157,6 %. Chiffre peu descriptif et peu précis : deux raisons de ne pas négocier dessus.

Faites-le dans le SQL. (les vues `lignes_ventes` et `paniers` ont été créées au C02, § « Pour la suite du module ») :

```
SELECT categorie,
       COUNT(*)           AS lignes,
       ROUND(AVG(montant_ttc)) AS moyenne,
       ROUND(STDDEV_SAMP(montant_ttc)) AS ecart_type,
       ROUND(100.0 * STDDEV_SAMP(montant_ttc)
            / NULLIF(AVG(montant_ttc), 0), 1) AS cv_pct
FROM lignes_ventes
GROUP BY categorie
ORDER BY cv_pct DESC;
```

Le `NULLIF` n'est pas un ornement : sans lui, une catégorie vide ou de moyenne nulle fait échouer la requête entière, et six lignes justes ne servent à rien si la septième plante le calcul.

5.6 L'erreur type : de combien votre moyenne est-elle fausse ?

Définition. L'erreur type de la moyenne (standard error of the mean, SE) est l'écart-type divisé par la racine carrée de l'effectif. Elle ne décrit pas la diversité des clients : elle décrit votre ignorance de la moyenne, c'est-à-dire de combien une autre collecte vous aurait donné un autre chiffre.

Pour les paniers : 10 244 FCFA, soit 9,0 % de la moyenne de 114 156 FCFA. La phrase à tenir en réunion : « notre panier moyen est de 114 156 FCFA, précis d'environ deux fois 10 244 FCFA au niveau de confiance usuel de 95 % ». Un écart de 5 000 FCFA entre deux vendeurs est du bruit ; un écart de 40 000 FCFA commence à être une information ; l'écart maximal observé entre les quatre vendeurs de l'extrait, 34 014 FCFA, est en zone intermédiaire — le chapitre M02.C08 tranche cette question par un test, et la tranche en faveur du doute.

Notez le comportement de la formule : la précision progresse comme la racine de l'effectif, donc pour diviser l'erreur type par deux il faut quadrupler le nombre de tickets. C'est la phrase à prononcer quand un collègue réclame « encore un mois de données » pour arbitrer un écart de 2 %.

Faites-le dans un tableur : `=ECARTYPE.STANDARD(plage)/RACINE(NB(plage))`. **En pandas** : `s["montant_ttc"].std(ddof=1) / np.sqrt(len(s))`. Une formule, trois outils, un effet immédiat sur la qualité des discussions : à ce stade du manuel, c'est l'indicateur au meilleur rapport effort-utilité.

Attention. Erreur type et écart-type se calculent avec la même fonction selon les logiciels, se notent tous deux en « ± », et se confondent couramment dans les tableaux de bord. Multiplier l'erreur type par 1,96 donne un intervalle de confiance sur la moyenne ; multiplier l'écart-type par 1,96 donne un intervalle où se trouve 95 % des clients. Deux phrases entièrement différentes, deux décisions opposées.

5.7 Deux dispersions robustes, et ce que leur écart apprend

Définition. L'écart absolu médian (MAD, median absolute deviation) est la médiane des valeurs absolues des écarts à la médiane. Dispersion entièrement robuste : aucune valeur extrême ne peut la déplacer, puisqu'elle est elle-même un ordre statistique.

Sur nos paniers, la MAD vaut 43 372 FCFA contre 182 095 FCFA à l'écart-type : l'écart entre les deux est le diagnostic. Une MAD au quart de l'écart-type annonce une série tenue par quelques valeurs. Le tableur ne la connaît pas ; trois colonnes suffisent, `=ABS(valeur-MEDIANE(plage))` puis `=MEDIANE()` sur la colonne obtenue.

Second garde-fou, la **moyenne tronquée** (trimmed mean) : on retire les 25 tickets hors borne haute et le CV tombe de 159,5 % à 96,6 %. Le niveau reste élevé mais devient descriptible ; la médiane, elle, ne bouge que de quelques milliers de francs (58 423 contre 54 044 FCFA). Règle de lecture : un indicateur qui se stabilise fortement quand on retire quelques lignes est tenu par ces lignes — et cela s'écrit dans la note.

Sur les quantités, le même raisonnement donne 7 pièces en moyenne, 4 au centre, un écart-type de 8,4 (CV 119,7 %) et une amplitude de 81 — de -13 à 68 dans le fichier nettoyé, de -13 à 14 000 dans l'énoncé, avant correction des 7 lignes multipliées par mille. Le seul énoncé de ces nombres suffit à dire que `quantite` ne se prête à aucune prévision en l'état : c'est une affaire de dispersion, pas de moyenne.

5.8 Quel outil pour quelle dispersion

Situation	Outil recommandé	Pourquoi	Ce qu'on évite
CV et écart-type par catégorie, sur 480 lignes	SQL (GROUP BY + STDDEV_SAMP)	un bloc pour moyenne, écart-type, CV, effectifs	sept tableaux croisés recopiés à la main
Vérification pédagogique du calcul	tableur, colonnes intermédiaires	on voit les carrés, on comprend la variance	la calculette : le résultat sans le chemin
MAD, moyenne tronquée	tableur ou pandas	le tableur n'a pas de fonction dédiée, pandas si	tout recalculer à la main sur 316 lignes
Erreur type d'une moyenne	les trois	une formule, effet immédiat	conclure sans elle
Seuil d'alerte publié	tableur puis SQL	bornes lisibles et rejouables	une borne bâtie sur l'écart-type, contaminée
Indicateur de dispersion en diffusion	Power BI ou équivalent, plus tard	le lecteur a besoin du contexte du grain	l'export d'un tableau de valeurs brutes

Pourquoi pas un outil de BI dès maintenant ? Vous publierez ces mesures dans un tableau de bord — dashboard, dans le jargon — au module M06, et ce sera le bon endroit. Mais un indicateur de dispersion diffusé sans sélecteur de grain ni légende se lit comme une anomalie : le lecteur ne sait jamais si l'écart-type qu'il voit décrit les clients, les tickets ou les lignes. À l'étape de préparation, le tableur ou le SQL obligent à nommer le grain — c'est pour cela qu'on y reste.

Boîte à outils. Le contrôle minimal de toute mesure de dispersion : (1) l'effectif utilisé s'écrit à côté de l'indicateur — $n = 316$ pour les paniers, 480 pour les lignes, jamais « nos données » ; (2) l'écart-type doit être plus grand que la MAD et plus petit que l'étendue, sinon vous avez publié une variance ou un écart-type de population sans le vouloir ; (3) un CV supérieur à 100 % s'accompagne d'une phrase sur la nature de la série, sinon l'interlocuteur entendra « erreur de données » ; (4) si la dispersion est calculée après nettoyage, avant/après en deux colonnes, jamais l'une sans l'autre.

6. Exemple concret : négocier le bois de chauffage, ou non

La demande. Le responsable des achats hésite à accorder une remise annuelle à un fournisseur de bois et panneaux, sur la foi d'un montant moyen de 201 865 FCFA par ligne. Sa question réelle : « ce niveau est-il assez stable pour que je négocie dessus ? »

Ce que dit le calcul.

Élément	Valeur	Interprétation
Lignes de la catégorie	24	trop peu pour une moyenne contractuelle
Moyenne	201 865 FCFA	le chiffre avancé en négociation
Médiane	77 904 FCFA	la ligne typique vaut un quart de la moyenne
Écart-type	318 080 FCFA	la dispersion dépasse le centre
CV	157,6 %	hors de toute zone négociable sur la moyenne

La décision. On ne négocie pas sur la moyenne de la catégorie. On négocie une **remise par gamme** (ciments, panneaux, bois de charpente séparément) ou un plafond de remise par commande : dans les deux cas l'indicateur de pilotage devient la médiane par gamme, pas la moyenne par catégorie. Le tableau tient en une page, sans test statistique, et il évite un contrat mal calibré.

La phrase à écrire dans la note. « La catégorie Bois & panneaux de l'extrait compte 24 lignes, une moyenne de 201 865 FCFA et une médiane de 77 904 FCFA ; l'écart-type (318 080 FCFA) excède la moyenne. Une remise assise sur la moyenne catégorie serait versée sur des lignes qui ne lui ressemblent pas. Nous recommandons de conduire la négociation par gamme, avec la médiane de chaque gamme pour base. »

7. Démonstration pas à pas : les trois dispersions, trois outils, un contrôle

Point de départ. Fiche `donnees/reference/ventes_magasin5_2025_ATTENDU.csv` ouverte dans le tableur : 480 lignes de données, lignes 2 à 481, colonne `montant_ttc` en I.

Étape 1 — Le bon effectif. `=NB(I2:I481)` doit renvoyer 480. Si vous obtenez 478 ou moins, des valeurs texte se sont glissées dans la colonne et la dispersion portera sur un sous-ensemble — c'est l'erreur du chapitre M01.C04, rejouée.

Étape 2 — L'étendue. `=MAX(I2:I481)-MIN(I2:I481)` → 1 380 482 FCFA, entre le maximum de 1 229 678 FCFA et le minimum de -150 804 FCFA. Écrivez les deux bornes à côté du résultat : une étendue sans ses deux termes est invérifiable par un tiers.

Étape 3 — L'IQR et les bornes. `=QUARTILE.INC(I2:I481;1)` et `=QUARTILE.INC(I2:I481;3)`, leur différence, puis les bornes à $1,5 \times \text{IQR}$. Contrôle de cohérence : l'IQR d'une colonne de montants doit rester inférieur à son écart-type ; si l'inverse se produit, vous comparez deux colonnes différentes.

Étape 4 — L'écart-type par la voie longue. Colonnes intermédiaires comme au §5.2, sur les paniers : variance 33 158 631 393, écart-type 182 095. Comparez à `=ECARTYPE.STANDARD()` : au franc près c'est bon ; sinon vous avez divisé par n au lieu de $n - 1$.

Étape 5 — L'erreur type. `=ECARTYPE.STANDARD(plage_paniers)/RACINE(NB(plage_paniers))` → 10 244 FCFA. Écrivez la phrase complète dans votre journal de travail, avec les trois nombres.

Étape 6 — Rejouer en SQL, puis en pandas. Reprenez la requête du §5.5 : le CV de Plomberie doit ressortir à 160,9 %.

```
pan = df.groupby("n_ticket")["montant_ttc"].sum().rename("panier")
stat = pan.agg(["count", "mean", "median", "std", "min", "max"])
cv = 100 * stat["std"] / stat["mean"]
se = stat["std"] / np.sqrt(stat["count"])
q1, q3 = pan.quantile([0.25, 0.75])
borne_haute = q3 + 1.5 * (q3 - q1)
n_hors = int((pan > borne_haute).sum())
```

Dernière ligne : 25. Si vous obtenez 24 ou 26, c'est la borne ou le comparateur qui n'est pas le bon — strict contre inclus est la cause la plus fréquente de cet écart d'une unité.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Écart-type des paniers recompté par la voie des carrés	± 1 FCFA	182 095	dénominateur $n - 1$ oublié
2	CV des lignes recompté	0,1 point	191,5 %	arrondis d'affichage
3	Effectif hors borne haute de Tukey, par filtre	identité	25 tickets	opérateur de comparaison
4	Couverture du tableau §5.5	7 catégories, 480 lignes	7 / 480	une catégorie sans ligne est restée
5	Ordre $\text{MAD} < \text{écart-type} < \text{étendue}$	les trois inégalités	$43\,372 \cdot 182\,095 \cdot 1$ 494 470	indicateur mal identifié

8. Erreurs fréquentes

1. **Comparer des écart-types entre colonnes d'échelles différentes.** « La quantité varie moins que le montant : 8,4 contre 143 915. » Les CV sont de 119,7 % et 191,5 % : la conclusion est juste, l'argument était faux — et il eût été faux en sens inverse. Règle réflexe : jamais un écart-type sans sa moyenne.
2. **Confondre écart-type et erreur type** (§5.6) : mêmes fonctions, mêmes noms selon le logiciel, même signe « $\pm$ ».
3. **Choisir ECARTYPEP par principe.** L'écart est minime sur 316 tickets, mais l'incohérence se propage à tout le module, qui raisonne en échantillon.
4. **Publier un CV de 191,5 % en rouge comme une anomalie.** C'est la nature du commerce de matériaux : un CV élevé se commente, il ne se signale pas — sauf référence métier explicite.
5. **Recalculer les bornes de Tukey après avoir retiré les valeurs extrêmes.** On détruit le bénéfice : la borne se calcule sur la série complète puis s'applique. Si vous écarter 25 tickets, énoncez la règle avant.
6. **Lire « 68 % des valeurs dans l'intervalle moyenne $\pm$ écart-type ».** Cette lecture suppose une distribution approximativement symétrique, ce que M02.C05 montre faux ici : sur 316 tickets asymétriques, l'intervalle ne couvre pas 68 % de la série.

9. Bonnes pratiques professionnelles

1. **Toujours deux dispersions, jamais une** : l'IQR pour borner, l'écart-type ou le CV pour comparer. Celui qui n'en cite qu'une se fait surprendre par la question suivante.
2. **Écrire l'effectif à côté de l'indicateur.** « CV de 159,5 % (316 tickets) » n'a pas la même valeur que « CV de 159,5 % (24 lignes) » : dans le second cas, le chiffre est incertain.
3. **Nommer le périmètre de nettoyage.** Un CV sur la série complète et un CV après retrait des 8 lignes de retours (-283 933 FCFA) sont deux chiffres corrects : le lecteur doit savoir lequel il a sous les yeux.
4. **Transformer l'indicateur en décision explicite.** « CV de 157,6 % → pas de remise assise sur la moyenne catégorie » : une dispersion qui ne change aucun comportement n'est qu'un chiffre de plus.
5. **Prévenir, pas punir.** Un seuil d'alerte sert à regarder, pas à noter un vendeur. Le premier usage d'une borne de Tukey en entreprise est d'ouvrir une enquête, presque jamais de fermer un compte.
6. **Conserver la requête de dispersion en annexe.** Dix lignes, rejouables : c'est elles qui vous feront gagner la réunion de l'année prochaine.

10. Exercice guidé — trois dispersions sur la colonne des prix

Énoncé. Sur la colonne `prix_unitaire_ht` du fichier attendu (480 lignes), calculez étendue, IQR, écart-type, puis CV, et concluez en deux lignes sur l'usage de la moyenne de prix en négociation.

Vous faites — barème 10 points, total 10.

Étape	Ce que vous produisez	Points
A	Effectif et unité (480 lignes, FCFA) écrits en tête	1
B	Étendue, avec ses deux termes	2
C	IQR, et bornes de Tukey qui en découlent	2

Étape	Ce que vous produisez	Points
D	Écart-type et CV, avec la moyenne à côté	3
E	Conclusion en deux lignes, un usage décidé	2

Nous vérifions. Les bornes du prix unitaire sont 675 et 40 550 FCFA : deux extrêmes plausibles dans un catalogue de quincaillerie, à confronter au fichier fournisseur (M01.C06) avant d'en faire un argument. La moyenne de la colonne ne décrit pas le catalogue, elle décrit son arithmétique : ce qu'on en peut tirer n'est pas « le prix moyen est de tant » mais « la moitié du catalogue tient dans telle plage » — c'est l'IQR qui répond, et c'est lui qui se négocie.

Nous corrigeons. Deux erreurs classiques : comparer ce CV à celui d'une autre colonne sans dire que les grains diffèrent (prix par produit contre montant par ligne) ; proposer de « supprimer les produits à prix aberrants » avant d'avoir regardé ce qu'ils vendent réellement.

11. Exercices autonomes

Exercice 3.1 (★) — Les trois dispersions, une par une. Sur les montants **par ligne** (480 lignes), calculez étendue, IQR, écart-type et CV. (attendu : étendue 1 380 482 FCFA, écart-type 143 915 FCFA, CV 191,5 %)

Exercice 3.2 (★) — Deux fournisseurs de ciment. L'un annonce 50 kg par sac avec un écart-type de 0,4 kg, l'autre 49,8 kg avec 0,2 kg. Le cahier des charges accepte de 49,5 à 50,5 kg. Lequel choisir, et que calculez-vous ?

Exercice 3.3 (★★) — Le CV par catégorie, sans recopier. Retrouvez le CV de la catégorie Plomberie et l'effectif qui le soutient. (attendu : 160,9 %, écart-type 247 146 FCFA, 57 lignes)

Exercice 3.4 (★★) — Écart-type ou MAD ? Pour chacune de ces séries, dites lequel vous citez en priorité, en une ligne de justification : (a) 480 montants de lignes ; (b) les effectifs de 30 magasiniers répartis sur six sites ; (c) 500 délais de livraison dont trois retards de 40 jours.

Exercice 3.5 (★★★) — Note de stabilité pour l'acheteur. Six lignes à destination du responsable des achats : deux catégories dont la moyenne est utilisable, une où elle ne l'est pas, une borne d'alerte sur les tickets avec sa justification, et la précision de vos chiffres. (attendu : Peinture 81,4 % et Consommables 102,9 % ; Electricité 170,4 % ; borne 295 445 FCFA, 25 tickets ; erreur type 10 244 FCFA)

12. Correction détaillée

Exercice 3.1. Étendue : 1 229 678 moins -150 804, soit 1 380 482 FCFA. Écart-type : 143 915 FCFA pour une moyenne de 75 152 FCFA, d'où un CV de 191,5 % — sur cette colonne, la moyenne ne décrit pas la série. L'IQR se lit par `=QUARTILE.INC` : notez-le avec ses deux quartiles, sinon un tiers ne peut pas le vérifier. Point de méthode valorisé : l'effectif de 480 écrit à côté de chaque ligne du tableau.

Exercice 3.2. On calcule les CV : 0,8 % et 0,4 %. Le second est deux fois plus régulier et son centre est dans la tolérance, donc on le retient à caractéristiques égales. Réserve attendue : un écart-type ne dit pas quelle part des sacs sort de la tolérance sans hypothèse sur la distribution ; la réponse professionnelle est d'exiger un contrôle sur cent sacs, pas de signer sur la foi du seul écart-type.

Exercice 3.3. 57 lignes pour Plomberie, moyenne de 153 596 FCFA, écart-type de 247 146 FCFA, CV de 160,9 %. Avec un tel effectif, l'erreur type de la moyenne dépasse le cinquième de la moyenne : la conclusion « la moyenne ne décrit rien » est robuste, la valeur exacte du CV ne l'est pas. La phrase attendue tient en deux parties, et les deux doivent être là.

Exercice 3.4. (a) les deux : l'écart entre écart-type et MAD est lui-même le signal (182 095 contre 43 372 FCFA sur les paniers). (b) IQR ou MAD : effectif faible, un départ ou un recrutement domine. (c) MAD : trois délais extrêmes sur cinq cents faussent l'écart-type, et le sujet est précisément ces trois délais — on ne veut pas qu'ils entrent dans le critère qui doit les révéler.

Exercice 3.5. Note attendue. Peinture (CV 81,4 %) et Consommables (102,9 %) : moyennes utilisables comme base d'échange, même si le second CV dépasse encore la zone de confort. Electricité (CV 170,4 %, écart-type 202 897 FCFA) : aucune remise assise sur la moyenne. Borne d'alerte : 295 445 FCFA, soit Q3 plus une fois et demie l'IQR, justifiée parce qu'elle ne se laisse pas fixer par les tickets qu'elle signale ; 25 tickets la franchissent. Précision : moyenne à environ deux fois 10 244 FCFA près, CV connu à plusieurs points près. Toute ligne citant un nombre absent des mesures du socle est comptée 0.

13. Mini-projet M02.P2 — suite : la colonne « précision » (30 min)

Reprenez le **M02.P2** du chapitre précédent (deux pages, six nombres, une décision ; barème 20 points, seuil de validation 13) et ajoutez **deux colonnes** au tableau : le CV et l'erreur type, chacun avec son effectif. La règle d'écriture devient : toute comparaison entre deux vendeurs ou deux catégories est accompagnée de la précision des deux nombres comparés ; une comparaison dont l'écart reste inférieur à deux erreurs types est présentée comme « non discriminante à ce stade ». Deux conséquences à faire apparaître explicitement : la catégorie qui résiste le moins à l'analyse (Bois & panneaux, 24 lignes, CV 157,6 %) est écartée de la page de décision, et le seuil d'alerte proposé est calculé sur la série complète, avant tout retrait.

14. Résumé du chapitre

1. Trois dispersions, trois usages : l'**étendue** décrit les extrêmes (1 494 470 FCFA sur les tickets, 13,6 fois l'IQR), l'**IQR** décrit la moitié du milieu et sert de borne (109 924 FCFA), l'**écart-type** décrit la distance typique au centre et sert de matière première (182 095 FCFA).
2. Un écart-type ne se compare jamais seul : le **CV** le rend comparable (Peinture 81,4 %, Electricité 170,4 %), et c'est lui qui décide si une moyenne est pilotable.
3. L'**erreur type** (10 244 FCFA, 9,0 % de la moyenne) dit de combien votre moyenne est fautive ; sans elle, toute comparaison entre deux vendeurs est une opinion.
4. La **MAD** (43 372 FCFA) et le **CV tronqué** (96,6 % au lieu de 159,5 %) sont les garde-fous des séries sales — et l'écart entre robuste et non robuste est, lui aussi, une information.

15. À retenir

À retenir.

- Une moyenne sans dispersion est une opinion ; deux dispersions suffisent à décider : l'IQR pour borner, le CV pour comparer.
- Sur les tickets : étendue 1 494 470 · IQR 109 924 · écart-type 182 095 FCFA · CV 159,5 % · erreur type 10 244 FCFA (9,0 % de la moyenne) · MAD 43 372 FCFA, le tout sur 316 tickets.
- Une borne de détection se calcule sur la série complète, avant tout retrait ; sinon les détectés fixent la règle qui doit les détecter.
- CV supérieur à 50 % : la moyenne ne décrit rien, on montre la distribution.

À retenir. La formulation qui protège. Dans une note : « écart-type de X FCFA (moyenne Y FCFA, CV Z %, n = N tickets, fichier nettoyé) ». À l'oral : « cette moyenne est précise à p % près ; l'écart que vous voyez est plus petit que cette précision, donc nous ne statuons pas ».

16. Évaluation formative (auto-correction, 10 min)

1. Quelle dispersion est la moins affectée par une erreur de saisie sur une seule ligne, et pourquoi ? **2.** Comment passe-t-on de la variance à l'écart-type, et que faire des décimales obtenues ? **3.** Une série a une moyenne de 250 et un écart-type de 60 ; une autre, une moyenne de 4 000 et un écart-type de 900. Laquelle est la plus instable, et que calcule-t-on pour le savoir ? **4.** À quoi sert l'erreur type, concrètement, en réunion ? **5.** Vrai ou faux : la borne de Tukey se calcule après retrait des valeurs extrêmes. Justifiez.

Corrigé

1. L'IQR et la MAD : ils ignorent les extrêmes par construction, alors que l'étendue est entièrement portée par eux. 2. On extrait la racine carrée, ce qui ramène l'indicateur dans l'unité de la variable ; on affiche arrondi, on calcule avec les décimales. 3. Les CV : 24,0 % et 22,5 % — la première série est la plus instable, malgré un écart-type trois fois plus petit. 4. À traduire une moyenne en « valeur $\pm$ précision », donc à décider si un écart constaté vaut une action, une réunion, ou rien. 5. Faux : la borne se calcule sur la série complète puis s'applique ; sinon on écarte des valeurs d'après un critère qu'elles ont elles-mêmes fabriqué.

Auto-validation. Reprenez les cinq contrôles du §7 sur votre fichier, puis comparez vos cinq nombres à la table de référence : 182 095 · 109 924 · 43 372 · 10 244 · 295 445. En cas d'écart, vérifiez le grain avant l'arithmétique : n = 316 tickets, et non 480 lignes.

M02.C04 — Se situer : quartiles, déciles, percentiles, lecture du boxplot

L'idée du chapitre. Un percentile ne répond pas à « combien vaut le ticket ? » mais à « combien de tickets sont en dessous ? ». C'est la mesure qui transforme un montant isolé en position dans une distribution — et donc en seuil de politique commerciale, en alerte calibrée, en classement défendable.

Base de travail — obligatoire. Deux fichiers du socle, dans votre dossier 02_exercices/M02/, ouverts côte à côte : `donnees/projection/ventes_magasin5_2025.csv` (l'énoncé, 489 lignes × 13 variables) et `donnees/reference/ventes_magasin5_2025_ATTENDU.csv` (480 × 13). Séparateur point-virgule, encodage UTF-8, virgule décimale. Montants en franc CFA (FCFA). Effet de référence de ce chapitre : les **316 paniers** (ticket consolidé par `n_ticket`), construits au chapitre M02.C02. Chiffres mesurés, recopiés depuis `01_socle_donnees/data/reference/chiffres_cites.md`, section M02.

1. Objectifs du chapitre

À la fin de ce chapitre, vous êtes capable de :

1. calculer un quartile, un décile, un percentile quelconque sur une série triée — à la main d'abord, puis dans trois outils ;
2. expliquer pourquoi deux logiciels ne renvoient pas exactement le même 90e percentile, et dire quand l'écart compte ;
3. lire une boîte à moustaches sans légende : boîte, médiane décalée, moustaches, points isolés, échelle ;
4. retourner le calcul — passer d'un montant à son percentile, et d'un objectif de couverture au montant seuil ;
5. construire une argumentation de concentration sans jamais confondre part des tickets et part du chiffre d'affaires ;
6. décider quels percentiles publier dans un tableau de bord, et lesquels garder pour l'analyse interne.

2. Pourquoi cette notion est importante

Les positions répondent aux trois questions que votre direction posera réellement.

- « **À partir de combien un client est-il un gros client ?** » Ce n'est pas une question de montant, c'est une question de rang. Sur les 316 tickets de l'extrait, un ticket de 25 000 FCFA est au 30e percentile : ordinaire ; un ticket de 100 000 FCFA est au 70e : déjà rare ; un ticket de 200 000 FCFA est au 85e. Un seuil de politique commerciale se choisit donc en percentile, jamais en montant rond choisi pour sa beauté.
- « **Que couvre notre dispositif de faveur ?** » Les 10 % de tickets les plus élevés portent 47,5 % du chiffre d'affaires de l'extrait ; les 25 % les plus élevés en portent 71,2 %. Réserver un avantage au-dessus de Q3, c'est viser un quart des tickets et près des trois quarts des encaissements. Seule la table des positions rend cette phrase possible — pas la moyenne.
- « **Où se trouve le milieu de nos ventes ?** » La moitié des tickets tient entre 20 635 et 130 559 FCFA ; la moitié du chiffre d'affaires n'est atteinte qu'au 90e percentile. Sans les positions, ces deux faits sont invisibles, et l'entreprise décide avec un seul nombre.

Dans les faits. Le vocabulaire des positions est celui que les directions générales emploient spontanément : « dans le top 10 % », « au 9e décile », « le produit qui fait 80/20 ». Un analyste qui ne sait pas convertir ce langage en calcul est dessaisi de la discussion en trente secondes. Tout ce chapitre sert à traduire dans les deux sens, vite et juste.

3. Explication simple

Rangez les 316 tickets du plus petit au plus grand, sans rien calculer. Vous avez une **série ordonnée**, et chaque ticket occupe une **position** : le premier, le 158e, le 316e.

Les découpages usuels partagent cette file en parts égales : quatre parts (les **quartiles** Q1, Q2 — la médiane — et Q3), dix parts (les **déciles** D1 à D9), cent parts (les **percentiles** P1 à P99). Dire « ce ticket est au 85e percentile » veut dire « 85 % des tickets sont plus petits que lui ».

Le mot « percentile » — parfois écrit « centile », ou « percentille » sous l'influence de l'anglais percentile — porte deux sens selon l'usage : la **valeur à la frontière** (235 318 FCFA) et le **rang en pourcentage** (85). Cette ambiguïté est la source de toutes les confusions du domaine. La question de contrôle qui les règle : « est-ce que je cite un montant, ou un pourcentage de tickets ? ».

Une **boîte à moustaches** — boxplot, « diagramme en boîte » en français — est le dessin qui pose ces cinq nombres sur une ligne : la boîte va de Q1 à Q3, un trait marque la médiane, deux moustaches s'étendent jusqu'aux valeurs extrêmes tolérées, les points au-delà se dessinent à part. Un résumé complet de la distribution en trois centimètres de large.

4. Vocabulaire essentiel

#	Terme	Français — English — sens simple	À savoir
1	Quantile	quantile — quantile — valeur qui partage une série triée	terme générique ; quartile, décile, centile en sont des cas
2	Quartile	quartile — quartile — découpage en quatre	Q2 = médiane ; Q1 et Q3 encadrent la moitié du milieu
3	Décile	décile — decile — découpage en dix	D9 est le 90e percentile
4	Percentile	percentile — percentile — découpage en cent	ne pas écrire « percent » : le suffixe change le sens
5	Rang	rang — rank — position dans la série triée	sans tri préalable, il n'y a pas de percentile
6	Interpolation	interpolation — interpolation — valeur estimée entre deux observations	c'est elle qui fait diverger les logiciels
7	Boîte moustaches	à boîte à moustaches — boxplot — dessin des cinq nombres	« cinq nombres » : min, Q1, médiane, Q3, max
8	Moustache	moustache — whisker — plus grande observation dans la limite tolérée	ce n'est pas le maximum de la série
9	Point isolé	valeur aberrante au sens de Tukey — outlier	à expliquer, jamais à effacer sans le dire
10	Fréquence cumulée	fréquence cumulée — cumulative frequency	part des observations en dessous d'une valeur
11	Fonction répartition	de fonction de répartition — CDF, ogive — la courbe des fréquences cumulées	l'outil des seuils inversés

5. Cours approfondi

5.1 Un percentile est un rang, pas une valeur

Définition. Le **p-ième quantile** d'une série triée est la valeur en dessous de laquelle se trouve la part p de l'effectif. Le **quartile** correspond à $p = 0,25 ; 0,5 ; 0,75$, le **décile** à un dixième, le **percentile** à un centième. « Quantile » désigne la frontière ; « percentile » désigne tantôt cette valeur frontière, tantôt le pourcentage d'observations qui la précèdent.

Sur les 316 paniers de l'extrait, la table des positions mesure :

Position	Valeur (FCFA)	Écart avec la position précédente (FCFA)
P10	7 390	—
P20	16 107	8 717
Q1 (P25)	20 635	4 528
P40	41 064	20 429
Q2 (P50)	58 423	17 359
P60	74 427	16 004
Q3 (P75)	130 559	56 132
P80	165 177	34 618
P90 (D9)	235 318	70 141
P95	435 984	200 666
P99	1 001 048	565 064

Lisez la deuxième colonne, pas la première. Monter de P10 à Q1 — cinq points de percentile — coûte 4 528 FCFA ; monter de P90 à P95, autant de points, coûte plus de deux cent mille francs : la série **s'étale à droite et se resserre à gauche**. C'est la traduction numérique exacte de l'asymétrie constatée au chapitre M02.C03, et la raison pour laquelle un seuil de « client important » fixé en francs est si souvent mal calibré.

L'intervalle Q1-Q3 contient 158 tickets sur 316 : la moitié, au ticket près — c'est le contrôle de cohérence du §7. Un effectif de 165 ou de 148 ne veut pas dire que le logiciel a tort : il veut dire que les bornes n'ont pas été comptées de la même façon.

5.2 Le calcul, puis la divergence des logiciels

Le calcul à la main, sur $n = 316$ valeurs triées, pour trouver Q3 :

1. position théorique : 317 fois trois quarts, soit 237,75 ;
2. on prend la 237^e valeur triée et on avance des trois quarts vers la 238^e ;
3. résultat : 130 559 FCFA.

Le pas n° 2 s'appelle l'**interpolation linéaire** — linear interpolation — et c'est là que les outils se séparent. Pour le 90^e percentile de la même série, les conventions les plus courantes donnent :

Convention	Où on la trouve	90e percentile (FCFA)
Interpolation linéaire, bords inclus	<code>=QUARTILE.INC</code> , <code>=PERCENTILE.INC</code> , pandas <code>quantile()</code> par défaut, <code>percentile_cont</code> en SQL	235 318

Convention	Où on la trouve	90e percentile (FCFA)
Observation immédiatement en dessous	pandas <code>interpolation="lower"</code> , <code>percentile_disc</code> en SQL	234 518
Observation la plus proche	pandas <code>interpolation="nearest"</code>	236 118
Interpolation exclusive	<code>=QUARTILE.EXC</code> , <code>=PERCENTILE.EXC</code>	243 010

Quatre réponses correctes, un écart maximal d'environ 8 500 FCFA sur 235 318 FCFA : 3,6 %. Sur 316 observations, sans conséquence pour une décision ; il cesse d'être négligeable quand l'effectif tombe à quelques dizaines, et devient décisif dès qu'un seuil devient contractuel. Deux règles en découlent : **nommer la méthode dans la note**, et **ne pas débattre d'un seuil à moins de deux chiffres significatifs**.

Faites-le dans un tableur. `=QUARTILE.INC(plage;3)` pour Q3, `=PERCENTILE.INC(plage;0,8)` pour P80, `=MEDIANE(plage)` pour P50. Les mêmes fonctions existent dans LibreOffice, où `=QUARTILE()` sans suffixe correspond à la méthode inclusive. Vérifiez sur votre fichier que P20 (16 107 FCFA) et P40 (41 064 FCFA) encadrent bien la médiane : sinon, votre plage de calcul ne couvre pas les 316 lignes — ou couvre les 480 lignes du fichier, ce qui est une autre série.

Attention. La fonction `=PERCENTILE()` sans suffixe, dans les versions anciennes d'Excel, est la méthode inclusive ; `=PERCENTILE.EXC` n'existe pas avant Excel 2010 et n'existe pas du tout dans certains tableurs en ligne. Si un classeur que vous recevez la contient, vérifiez le résultat sur une petite série avant de le recopier dans une note.

5.3 Retourner le calcul : du montant au percentile

Le besoin inverse est encore plus fréquent en réunion : « notre plus gros client du mois, il est où ? ». On compte

les observations strictement inférieures au montant, on divise par l'effectif, on lit le pourcentage. Sur l'extrait

25 000 FCFA → 30e percentile ; 100 000 FCFA → 70e ; 150 000 FCFA → 78e ; 200 000 FCFA → 85e.

Faites-le dans les trois outils. En SQL ((les vues `lignes_ventes` et `paniers` ont été créées au C02, § « Pour la suite du module »)) :

```
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE panier < 200000) / COUNT(*)) AS percentile
FROM paniers;
```

En pandas : `float((pan < 200000).mean() * 100) → 85`. En tableur : `=NB.SI(plage;"<200000")/NB(plage)`, formatée en pourcentage — multipliez par 100 à l'affichage et non dans la formule, sinon la cellule devient fausse dès qu'on la recopie ailleurs.

Deux précautions. D'abord, cette fréquence cumulée est **strictement inférieure** : « au 85e percentile » et « dans les 15 % meilleurs » ne coïncident jamais exactement, surtout dans une série dense. Ensuite, un percentile n'est pas un classement de valeur : il dit combien sont en dessous, pas combien pèsent — la part d'encaissements se calcule à part, en sommant les montants.

Définition. La **fréquence cumulée** (cumulative frequency) associe à une valeur la proportion d'observations inférieures ou égales ; sa courbe est la **fonction de répartition** (CDF, cumulative distribution function, ogive en français). Un seul objet mathématique derrière trois questions : quel percentile a ce montant ? quel montant correspond à ce percentile ? quelle part du chiffre d'affaires font les tickets plus petits que celui-ci ?

5.4 Lire une boîte à moustaches

Définition. La **boîte à moustaches** (boxplot) représente cinq nombres : le premier quartile et le troisième, qui forment la boîte ; la médiane, qui la coupe ; la plus petite et la plus grande observation restant dans les bornes de Tukey, qui forment les moustaches. Les observations hors bornes sont dessinées en points. Le « résumé en cinq nombres » (five-number summary) est cette même information, sans le dessin.

Boîte à moustaches des 316 paniers, échelle linéaire : la moitié du milieu tient à gauche



Figure — Boîte à moustaches des paniers du magasin 5 (fichier nettoyé, $n = 316$).

Les cinq choses à regarder, dans cet ordre. (1) La **boîte**, de Q1 à Q3 : elle contient la moitié du milieu, 109 924 FCFA de large ici. (2) Le **trait médian** : au centre de la boîte, la moitié du milieu est symétrique ; collé au bord gauche — notre cas —, il dit que la seconde moitié de la boîte est beaucoup plus étalée que la première. (3) Les **moustaches** : la plus basse observation restant dans la borne de Tukey, -19 711 FCFA, et la plus haute, 281 784 FCFA — ce ne sont **pas** le minimum et le maximum de la série. (4) Les **points isolés** : 25 tickets au-dessus de la borne, dont celui de 1 343 666 FCFA, et un seul en dessous, l'avoir de -150 804 FCFA. (5) L'**échelle** : sans graduations, une boîte ne dit rien ; sur une échelle logarithmique, elle dit autre chose.

Ce qu'une boîte ne dit pas. Elle ne montre pas les bosses : deux distributions de formes très différentes peuvent avoir la même boîte. Pour la forme, il faut l'histogramme — chapitre M02.C05, figure ci-dessous au chapitre suivant. C'est pourquoi les deux figures se complètent, et pourquoi un tableau de bord qui n'affiche que des boîtes aveugle son lecteur sur ce qu'il croit voir.

Conseil professionnel. Quand vous tracez des boîtes par catégorie, alignez-les sur une échelle horizontale commune et ordonnez-les par médiane croissante. Sept boîtes sur une échelle disent plus que sept diagrammes individuels : la comparaison est le sujet, pas la catégorie.

5.5 Déciles et concentration : ce que porte la pyramide

Définition. On appelle **concentration** le fait qu'une part donnée des observations porte une part différente — en général plus grande — du total. La représentation classique met en face les parts cumulées d'effectif et les parts cumulées de montant : c'est la **courbe de Lorenz** (Lorenz curve), outil de l'économie des inégalités, dont la table ci-dessous est la version discrète, en cinq coupes plutôt qu'en une courbe.

Les positions permettent de répartir le chiffre d'affaires entre les tranches de tickets. Sur l'extrait, les cumuls mesurés :

Tranche (du plus petit au plus grand)	Part des tickets	Part cumulée du CA
Sous Q1 (P25)	25 %	1,6 %

Tranche (du plus petit au plus grand)	Part des tickets	Part cumulée du CA
Sous la médiane (P50)	50 %	10,1 %
Sous Q3 (P75)	75 %	28,8 %
Sous P90	90 %	52,5 %
Au-dessus de P80	20,3 %	65,1 %
Au-dessus de P90 (le dernier décile, 32 tickets)	10,1 %	47,5 %

La lecture, en une phrase : **la moitié des tickets ne fait que 10,1 % des encaissements, et un dixième en fait presque la moitié.** C'est ce contraste, et lui seul, qui justifie de traiter le haut de pyramide à part.

Un détail de méthode, instructeur. Le « dernier décile », selon qu'on le définit comme les 10 % d'effectif les plus élevés ou comme tout ce qui dépasse le 90e percentile, contient 31 ou 32 tickets : les deux décomptes sont corrects et donnent 46,9 % et 47,5 % du chiffre d'affaires. Un découpage arrondi en effectif plutôt qu'en pourcentage change la réponse de 0,6 point. Écrivez la définition, sinon le relecteur refait le calcul, trouve un autre nombre, et doute de vous.

Attention. « Les 25 % des tickets les plus élevés font 71,2 % du CA » est une phrase sur la concentration. Elle ne dit rien d'une performance, encore moins d'une faute : dans un commerce de matériaux, un chantier représente à lui seul une journée de comptoir. La conclusion possible est une organisation — suivi dédié, conditions particulières, stock réservé —, jamais un jugement sur les vendeurs.

5.6 Décider avec les percentiles : trois usages types

1. **Fixer un seuil de couverture.** On veut réserver la livraison gratuite aux 20 % de tickets les plus élevés : le seuil est P80, soit 165 177 FCFA, et 64 tickets sur 316 (20,3 %) le franchissent, portant 65,1 % du CA. À 150 000 FCFA — montant rond, 78e percentile —, vous couvrez 22,5 % des tickets et 68,1 % du CA : la décision est la même, le cadrage est faux. On choisit le pourcentage, on en déduit le montant.
2. **Suivre le haut de pyramide.** Publier le 95e percentile (435 984 FCFA) plutôt que le maximum : le maximum est un ticket, le 95e est une statistique. Mais sur 316 tickets, P99 repose sur trois observations — à cet effectif, on publie P90 et on commente P95, on n'affiche pas P99.
3. **Découper une clientèle en strates.** Quatre groupes de 79 tickets environ (quartiles de dépenses) sont défendables : effectifs comparables, frontières explicites. Trois groupes « petits / moyens / gros » à seuils ronds ne le sont pas. Le jour où vous devrez justifier un échantillon d'audit — « on a pris les 15 % plus gros tickets » —, le percentile est votre seule ligne de défense : 48 tickets, 57,1 % du CA, frontière à 200 000 FCFA, tout est écrit et rejouable.

5.7 Quels percentiles publier, lesquels garder

Public	Ce qu'on publie	Ce qu'on garde pour l'analyse
Direction commerciale	médiane, Q3, P90, part du CA du dernier décile	P95, P99, le détail des points isolés
Contrôleur de gestion	Q1, médiane, Q3, IQR, effectif hors bornes	les commentaires ligne à ligne
Acheteur	médiane et IQR par catégorie, CV (M02.C03)	les percentiles par produit, trop instables
Audit, contentieux	tous les percentiles demandés, méthode nommée	rien : tout s'écrit
Lecture interne de formation	de cinq nombres et boîte à moustaches	la querelle d'interpolation

La règle d'écriture qui va avec : **un percentile publié s'accompagne de l'effectif et du mode de comptage des bornes**. « P90 = 235 318 FCFA (n = 316 tickets, fichier nettoyé, interpolation linéaire, comparaison strictement inférieure) » coûte une ligne de plus et ne se discute plus.

5.8 Calculer les positions dans les trois outils

Opération	Tableur	SQL (DuckDB)	pandas
Quartiles	=QUARTILE.INC(plage;1) puis (...),3)	percentile_cont(0.25) WITHIN GROUP (ORDER BY x)	s.quantile(.25)
Percentile quelconque	=PERCENTILE.INC(plage;0,8)	percentile_cont(0.8) WITHIN GROUP (...)	s.quantile(.8)
Observation la plus proche	—	percentile_disc(0.8) WITHIN GROUP (...)	s.quantile(.8, interpolation="nearest")
Déciles par groupe	tri + =RANG()	NTILE(10) OVER (PARTITION BY g ORDER BY x)	pd.qcut(g, 10)
Percentile d'une valeur	=NB.SI(plage;"<v")/ NB(plage)	COUNT(*) FILTER (WHERE x < v) / COUNT(*)	(s < v).mean()
Cinq nombres d'une boîte	formules ci-dessus	min, P25, P50, P75, max en une requête	s.describe(percentiles=[.25,.5,.75])

La requête de découpage qui sert ensuite dans tout le module — les déciles de tickets par vendeur ((les vues `lignes_ventes` et `paniers` ont été créées au C02, § « Pour la suite du module »)) :

```
WITH paniers AS (
  SELECT vendeur, n_ticket, SUM(montant_ttc) AS panier
  FROM lignes_ventes GROUP BY vendeur, n_ticket
),
decoupages AS (
  SELECT vendeur, panier,
    NTILE(10) OVER (PARTITION BY vendeur ORDER BY panier DESC) AS decile
  FROM paniers
)
SELECT vendeur, decile, COUNT(*) AS tickets, SUM(panier) AS ca_decile
FROM decoupages
GROUP BY vendeur, decile
ORDER BY vendeur, decile;
```

Le `NTILE` vit dans une requête à part : on ne peut pas découper et agréger dans le même `SELECT` (DuckDB répond `Cannot group on a window clause`, et c'est très bien qu'il réponde ça — mélanger les deux étapes donnerait un chiffre qui n'a pas de définition). `NTILE(10)` découpe en dix groupes d'effectifs égaux à **l'intérieur de chaque vendeur** : c'est le `PARTITION BY` qui rend licite la comparaison entre vendeurs de tailles différentes. Sans lui, vous découpez la planète en dix et vous appelez ça un classement.

Boîte à outils. Contrôle minimal de toute publication de positions : (1) $Q1 \leq \text{médiane} \leq Q3$, sans exception — sinon le tri ou l'effet est faux ; (2) l'effectif entre $Q1$ et $Q3$ vaut la moitié de n au ticket près ; (3) les moustaches sont des observations réelles, jamais des bornes calculées — vérifiez que la valeur de la moustache existe dans la colonne ; (4) un percentile de rang inférieur à 5 ou supérieur à 95 sur quelques centaines d'observations ne se publie pas, il se commente ; (5) la méthode d'interpolation s'écrit une fois par document, pas par ligne.

6. Exemple concret : la livraison gratuite, décidée au percentile

La demande. « Relevons le seuil de livraison gratuite de 100 000 à 150 000 FCFA : nous voulons la réserver aux gros tickets. » Trois questions, dans l'ordre.

Question	Calcul	Réponse mesurée sur l'extrait
Combien de tickets perdent le bénéfice ?	nb de tickets $\geq$ seuil	96 tickets à 100 000 FCFA (30,4 %), 71 à 150 000 FCFA (22,5 %)
Combien de chiffre d'affaires est en jeu ?	somme des tickets franchissant le seuil	76,7 % du CA au-dessus de 100 000 FCFA, 68,1 % au-dessus de 150 000 FCFA
Qui est touché, côté clients ?	clients distincts, pas tickets	372 clients distincts dans le fichier, dont 18 sans identifiant exploitable

Ce que la table des positions change à la discussion. Le seuil actuel est au 70e percentile, le nouveau au 78e. On n'écarte donc pas « un dixième des tickets » au sens large mais 25 tickets sur 316, et la tranche écartée — entre 74 427 et 165 177 FCFA, soit P60 à P80 — pèse 19,2 % du chiffre d'affaires : ce ne sont pas les tickets du comptoir, ce sont les petits chantiers. C'est exactement ce que la moyenne cache et que les percentiles montrent.

La conclusion professionnelle. Un seuil ne se fixe pas en francs mais en couverture. Note attendue : pour couvrir 20 % des tickets, le seuil est P80 = 165 177 FCFA (64 tickets, 65,1 % du CA) ; le seuil proposé de 150 000 FCFA couvre 22,5 % des tickets et 68,1 % du CA ; le seuil actuel de 100 000 FCFA en couvre 30,4 %. Trois chiffres, une décision, aucune contestation possible de méthode.

7. Démonstration pas à pas : tracer la boîte vous-même

Étape 1 — La série. Colonne des paniers, 316 valeurs (construite aux étapes 1 à 3 de M02.C02). Triez-la : dans le tableur, tri croissant sur la colonne ; en pandas, `pan.sort_values()`.

Étape 2 — Les positions. Calculez Q1, Q2, Q3 par la voie des positions, pas par la fonction : pour Q1, la position est le quart de 317, soit 79 et un quart ; interpolatez entre la 79e et la 80e valeur. Vous devez retrouver 20 635 FCFA.

Étape 3 — Le contrôle d'effectif. `=NB.SI(plage;">=20635")-NB.SI(plage;">130559")` doit donner 158. À 157 ou 159 près, ce sont les bornes : la convention retenue ici est l'intervalle fermé, écrivez la vôtre.

Étape 4 — Les moustaches. Plus grande valeur $\leq$ 295 445 FCFA $\rightarrow$ 281 784 FCFA. Plus petite valeur $\geq$ la borne basse $\rightarrow$ -19 711 FCFA. Notez que la moustache basse est négative : la série contient des avoirs (M02.C02, §5.6), et une boîte honnête les montre.

Étape 5 — Les points isolés. Vingt-cinq au-dessus de la borne haute, un en dessous de la basse. Comptez-les ; ne les dessinez pas « au doigt » : un effectif figé est un contrôle rejouable, un amas de points non.

Étape 6 — Le tracé. Échelle horizontale commune, boîte de Q1 à Q3, trait à la médiane, moustaches, points en petits cercles. Comparez à la figure du §5.4 : la ressemblance est le contrôle qualité.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Ordre $Q1 \leq \text{médiane} \leq Q3$	sans exception	20 635 · 58 423 · 130 559	tri oublié
2	Effectif entre Q1 et Q3	n / 2 au ticket près	158	bornes incluses ou non

#	Contrôle	Seuil	Résultat attendu	Si échec
3	Les moustaches sont des valeurs de la colonne	appartenance	-19 711 · 281 784	borne confondue avec observation
4	Part cumulée du CA sous la médiane	recoupement avec §5.5	10,1 %	périmètre : tickets ou lignes
5	P20 et P40 encadrent la médiane	ordre	16 107 · 41 064	effet incomplet

8. Erreurs fréquentes

1. **Confondre moustache et maximum.** La moustache s'arrête à la dernière observation dans la borne ; le maximum est un point isolé. Écrire « la moustache va jusqu'à 1 343 666 FCFA » est doublement faux : c'est le maximum, et la borne vaut 295 445 FCFA.
2. **Publier un P99 sur 316 observations.** Trois tickets le déterminent : l'indicateur suit la valeur d'un seul client. À cet effectif, P90 est le rang le plus haut défendable.
3. **Découper en déciles sans base de comparaison.** Un « 1er décile » chez un vendeur de 125 lignes et chez un vendeur de 12 lignes ne désigne pas le même monde : découpez par vendeur (PARTITION BY) ou comparez des percentiles bruts, mais ne mélangez pas les deux dans la même phrase.
4. **Oublier l'interpolation et comparer deux outils.** Quatre valeurs possibles pour le même 90e percentile ; sans la méthode écrite, un désaccord de quelques milliers de francs devient une querelle de compétence.
5. **Lire un percentile comme un pourcentage de valeur.** « Au 85e percentile » ne veut pas dire « 85 % du chiffre » : le premier est un rang, le second une part monétaire. Sur notre série, les 15 % de tickets au-dessus du 85e percentile pèsent 57,1 % du CA — le double de ce que la confusion ferait croire.
6. **Décrire la distribution par les seuls percentiles, sans figure.** Une boîte se lit en dix secondes par un non-statisticien ; trois lignes de chiffres ne se lisent pas. C'est pourquoi l'étape 6 se fait à la main au moins une fois : on ne dessine pas ce qu'on n'a pas compris.

9. Bonnes pratiques professionnelles

1. **Écrire l'effectif et la méthode** : « P90 = 235 318 FCFA (n = 316, interpolation linéaire, fichier nettoyé) ».
2. **Publier la médiane et l'IQR d'abord** sur une série asymétrique ; les percentiles hauts en commentaire, jamais en titre de graphique.
3. **Fixer les seuils en couverture, pas en montants ronds.** Un objectif de « 20 % des tickets » se contrôle six mois plus tard ; un objectif de « 150 000 FCFA » dérive silencieusement avec les prix.
4. **Tracer les boîtes à échelle commune**, ordonnées par médiane, avec le nombre de points isolés écrit sous chaque boîte.
5. **Ne jamais retirer un point isolé sans l'expliquer** : ici, le plus à droite est un chantier, le plus à gauche un avoir — deux réalités métier, pas deux fautes de saisie.
6. **Conserver la requête NTILE en annexe du livrable** : c'est elle qui rend le découpage reproductible par un tiers, donc défendable en audit.

10. Exercice guidé — les cinq nombres, puis le dessin

Énoncé. Sur les 316 paniers : retrouvez les cinq nombres de la boîte (moustaches comprises), dessinez la boîte à l'échelle, et écrivez la phrase de lecture destinée à un directeur non statisticien.

Vous faites — barème 10 points, total 10.

Étape	Ce que vous produisez	Points
A	Les trois quartiles, avec les positions utilisées	3
B	Les deux moustaches, identifiées comme observations réelles	2
C	Le décompte des points isolés, en haut et en bas	2
D	Le tracé à l'échelle commune	1,5
E	La phrase de lecture, sans jargon	1,5

Nous vérifions. $Q1 = 20\,635$, médiane = $58\,423$, $Q3 = 130\,559$ FCFA ; moustaches à $-19\,711$ et $281\,784$ FCFA ; 25 points isolés en haut, un en bas. Phrase attendue : « la moitié des tickets tient entre 20 635 et 130 559 FCFA, et 25 tickets dépassent 295 445 FCFA : ce sont les chantiers, pas des erreurs ». Si votre phrase contient le mot « aberrant » sans le définir, elle est à réécrire — c'est le mot qui fâche en réunion.

Nous corrigeons. L'erreur la plus fréquente : tracer les moustaches jusqu'au minimum et au maximum de la série. Le dessin reste joli, la lecture devient fausse, et le contrôle n° 3 du §7 vous l'aurait signalé.

11. Exercices autonomes

Exercice 4.1 (★) — La table des positions. Retrouvez $P10$, $P20$, $P40$, $P60$, $P80$ et $P95$ sur les paniers, puis écrivez la phrase qui compare les écarts entre ces positions. (attendu : $7\,390 \cdot 16\,107 \cdot 41\,064 \cdot 74\,427 \cdot 165\,177 \cdot 435\,984$ FCFA)

Exercice 4.2 (★) — Le percentile d'un montant. À quel percentile se situent un ticket de 100 000 FCFA et un ticket de 25 000 FCFA ? Donnez la formule de tableur, pas seulement le résultat. (attendu : 70e et 30e, par fréquence cumulée strictement inférieure)

Exercice 4.3 (★★) — Le seuil de couverture. Vous devez fixer le seuil de livraison gratuite pour que 15 % des tickets y soient éligibles. Quel percentile utilisez-vous, quel montant, quelle part du CA ? (attendu : $P85$, 200 000 FCFA, 48 tickets, 57,1 % du CA)

Exercice 4.4 (★★) — Déciles par vendeur. Découpez les tickets de chaque vendeur en déciles avec `NTILE(10) OVER (PARTITION BY vendeur ...)`, puis comparez le premier décile de Bationo à celui d'Ilboudo. Que devient la comparaison si vous découpez l'ensemble de la population au lieu de découper par vendeur ?

Exercice 4.5 (★★★) — Note de concentration. En six lignes, pour la direction : la part du CA des 25 % de tickets les plus élevés, celle des 50 % les plus petits, le seuil de « gros ticket » que vous proposez avec sa justification en percentile, et la réserve de méthode sur le découpage en déciles. (attendu : $71,2\% \cdot 10,1\% \cdot P90 = 235\,318$ FCFA · 46,9 % ou 47,5 % selon l'effectif retenu)

12. Correction détaillée

Exercice 4.1. Les six valeurs attendues sont celles du §5.1. La phrase de lecture porte sur les écarts : entre $P10$ (7 390 FCFA) et $P20$ (16 107 FCFA) il y a moins de la moitié de ce qui sépare $P20$ de $Q1$ (20 635 FCFA), et les derniers rangs s'étalent beaucoup plus loin. Un élève qui reco-

pie la liste sans comparer les écarts est noté à la moitié : la table ne vaut que par sa colonne de différences.

Exercice 4.2. $\text{=NB.SI(plage;"<100000")/NB(plage)}$ pour 100 000 FCFA, la même formule avec 25 000 pour le second : 70e et 30e percentiles. Le tableur peut donner 70 ou 71 selon que l'on compte strictement en dessous ou en dessous ou égal — écrire le choix fait partie de la réponse, c'est même l'objet de la question.

Exercice 4.3. 15 % éligibles = couper au 85e percentile. Mesures de l'extrait : 200 000 FCFA correspond exactement

au 85e percentile, avec 48 tickets au-dessus (15,2 % de l'effectif) portant 57,1 % du CA. La vérification attendue

la part de chiffre d'affaires se lit en sommant les montants des tickets retenus, pas en recopiant les 15 % de couverture — les deux chiffres ne coïncident jamais, et c'est l'écart entre eux qui fait la valeur de la note.

Exercice 4.4. Par vendeur, chaque décile contient un dixième des tickets de ce vendeur : la comparaison porte sur les positions relatives, donc sur la homogénéité interne. Sur l'ensemble de la population, chaque décile contient un dixième des tickets tous vendeurs confondus, et les vendeurs se répartissent inégalement dans les déciles : la comparaison devient une mesure de mix, pas de performance. Les deux questions sont légitimes ; ce ne sont pas les mêmes, et un tableau de bord qui les mélange sans le dire induit la direction en erreur.

Exercice 4.5. Note attendue. Les 25 % des tickets les plus élevés portent 71,2 % du CA de l'extrait, les 50 % les plus petits 10,1 %. Nous proposons de définir le « gros ticket » au 90e percentile, soit 235 318 FCFA ($n = 316$, interpolation linéaire), parce que ce seuil ne dépend pas d'un seul client et qu'il couvre 10,1 % des tickets pour 47,5 % du CA. Réserve à écrire : selon que le dernier décile compte 31 ou 32 tickets, la part retenue vaut 46,9 % ou 47,5 % — 0,6 point d'écart, sans erreur de personne.

13. Mini-projet M02.P2 — suite : la page des positions (30 min)

Ajoutez au document du **M02.P2** (barème 20 points, seuil de validation 13) une page « positions » : la table des percentiles publiables (P10 à P95), la boîte à moustaches des sept catégories à échelle commune, et le seuil de politique commerciale choisi en percentile avec sa couverture en tickets et sa part de CA. Deux contraintes de rédaction, non négociables au barème : **aucun montant de seuil n'apparaît sans le percentile correspondant**, et **aucun percentile n'apparaît sans l'effectif ni la méthode**. Les deux points isolés extrêmes reçoivent chacun une phrase d'interprétation métier — une phrase qui ne contient pas le mot « aberrant ».

14. Résumé du chapitre

1. Un percentile est un **rang** : 85 % des tickets sont sous 200 000 FCFA, la moitié est entre 20 635 et 130 559 FCFA. Citer la valeur ou le pourcentage ne donne pas la même information.
2. Quatre **méthodes d'interpolation**, quatre réponses pour le même 90e percentile (234 518 à 243 010 FCFA) : nommez la vôtre et l'écart redevient un détail.
3. La **boîte à moustaches** se lit en cinq points — boîte, médiane décalée, moustaches qui sont des observations réelles, points isolés, échelle — et ne dit rien de la forme : l'histogramme vient après (M02.C05).
4. On décide en **couverture** : 158 tickets entre Q1 et Q3, 19,2 % du CA dans la bande P60-P80, 71,2 % au-dessus de Q3, 47,5 % dans le dernier décile.

15. À retenir

À retenir.

- $Q1 : 20\,635 \cdot$ médiane : $58\,423 \cdot Q3 : 130\,559 \cdot IQR : 109\,924 \cdot$ bornes de Tukey : $-144\,251$ et $295\,445 \cdot$ moustaches $-19\,711$ et $281\,784$ FCFA, sur 316 tickets.
- Un seuil se choisit en pourcentage de couverture, puis on en déduit le montant.
- Moustache $\neq$ maximum ; percentile $\neq$ pourcentage de valeur ; P99 sur 316 lignes ne se publie pas.
- Écrire l'effectif, la méthode d'interpolation, et le caractère inclus ou non des bornes.

À retenir. La formulation qui protège. « Le 90e percentile des tickets est de 235 318 FCFA ($n = 316$, fichier nettoyé, interpolation linéaire) ; 32 tickets le dépassent et portent 47,5 % du chiffre d'affaires de l'extrait. »

16. Évaluation formative (auto-correction, 10 min)

1. Quelle différence de sens entre « moustache haute » et « maximum de la série » ? **2.** Vous devez publier le montant à partir duquel on est « gros client », en couvrant 10 % des tickets. Que calculez-vous, et pourquoi pas le maximum divisé par dix ? **3.** Deux collègues obtiennent 234 518 et 243 010 FCFA pour le même percentile. Que s'est-il passé, que fait-on ? **4.** Que dit la table des positions du §5.1 sur la forme de la distribution, sans la voir ? **5.** Vrai ou faux : « 25 % des tickets font 71,2 % du CA, donc les petits clients ne servent à rien ».

Corrigé

1. La moustache s'arrête à la dernière observation restant dans la borne de Tukey (281 784 FCFA) ; le maximum est hors moustache et se dessine comme point (1 343 666 FCFA). 2. Le 90e percentile, soit 235 318 FCFA : couvrir 10 % des tickets, c'est couper à P90. Diviser le maximum par dix suppose une distribution régulière, ce que la série dément (asymétrie mesurée au chapitre M02.C03 : 3,84). 3. Deux méthodes d'interpolation, toutes deux justes : on écrit la méthode retenue dans la note, on cesse de discuter du chiffre. 4. Que la série est très asymétrique : les écarts entre positions croissent fortement vers la droite (4 528 FCFA entre P20 et Q1, contre plus de deux cent mille entre P90 et P95). 5. Faux : c'est une phrase sur la concentration des montants, pas sur l'utilité économique ; les petits tickets font le comptoir et la fréquence, et l'extrait ne permet pas de trancher la question de la marge — il permet de la poser correctement.

Auto-validation. Reprenez les cinq contrôles du §7 sur votre fichier : $20\,635 \cdot 58\,423 \cdot 130\,559 \cdot 158 \cdot -19\,711$ et $281\,784$. Si vos quartiles diffèrent, recomptez vos positions avant de changer de logiciel : sur 316 valeurs, l'erreur est presque toujours dans le tri ou dans l'effet, rarement dans la formule.

M02.C05 — La forme d'une distribution : histogramme, asymétrie, valeurs extrêmes

L'idée du chapitre. Moyenne, dispersion et positions sont des résumés. La **forme**, c'est ce qui reste quand on arrête de résumer : où les valeurs s'accumulent, où la série s'étire, combien de bosses on voit, et ce qu'on fait des trois tickets négatifs. Deux distributions peuvent avoir les mêmes indicateurs et des formes opposées — et appeler des décisions opposées.

Base de travail — obligatoire. Deux fichiers du socle, dans votre dossier 02_exercices/M02/, ouverts côte à côte : `donnees/projection/ventes_magasin5_2025.csv` (l'énoncé, 489 lignes × 13 variables) et `donnees/reference/ventes_magasin5_2025_ATTENDU.csv` (480 × 13). Séparateur point-virgule, encodage UTF-8, virgule décimale. Montants en franc CFA (FCFA). Effet de référence : les **316 paniers** construits au chapitre M02.C02. Chiffres mesurés, copiés depuis `01_socle_donnees/data/reference/chiffres_cites.md`, section M02.

1. Objectifs du chapitre

À la fin de ce chapitre, vous êtes capable de :

1. construire un histogramme lisible — choisir l'amplitude et l'origine des classes, ouvrir la première et la dernière — et justifier ce choix par les règles du métier ;
2. lire une forme : asymétrie à droite ou à gauche, une ou plusieurs bosses, queue épaisse ou non ;
3. calculer et interpréter les deux coefficients de forme (asymétrie et kurtosis) sans leur faire dire plus qu'ils ne disent ;
4. distinguer trois sortes de valeurs extrêmes — erreur de saisie, exception réelle, cas limite — et appliquer à chacune un traitement nommé dans la note ;
5. décider si un indicateur usuel (moyenne, écart-type, corrélation, droite de régression) est licite sur cette forme, ou s'il faut le remplacer ;
6. expliquer à un non-statisticien, avec une figure de trois centimètres, pourquoi « la moyenne ne raconte pas notre magasin ».

2. Pourquoi cette notion est importante

Tout ce que le module a posé depuis quatre chapitres converge ici, et trois décisions de gestion ne se prennent pas sans la forme.

- **Faut-il utiliser la moyenne ?** Sur nos 316 paniers, l'asymétrie mesure 3,84 : une série à 3,84 n'est pas « un peu penchée », elle est dominée par sa queue. C'est la mesure, et non une impression, qui autorise la réponse.
- **Faut-il alerter sur les extrêmes ?** L'histogramme montre 31 tickets au-dessus de 250 000 FCFA, soit 9,8 % de l'effectif, portant 46,9 % du chiffre d'affaires. Une alerte par ticket partait onze fois par mois : personne ne lit onze alertes par mois. La forme règle le contrôle interne, pas seulement sa formule.
- **Peut-on modéliser ?** Les deux chapitres suivants estimeront une relation entre quantité et montant, puis la solidité d'un écart entre vendeurs. Ces calculs reposent sur des hypothèses

de forme ; connaître la forme, c'est savoir si l'on a le droit d'utiliser l'outil — et avec quelles précautions le dire.

Dans les faits. La forme est le seul endroit du manuel où une figure est plus lue que le texte. En entreprise, la courbe projetée en réunion décide souvent à elle seule : « en fait, c'est une minorité qui fait tout ». Ce chapitre vous apprend à la tracer correctement — un histogramme mal borné ment — et à ne pas la sur-interpréter.

3. Explication simple

Prenez les 316 paniers et rangez-les dans des cases de 50 000 FCFA : « moins de zéro », « de 0 à 50 000 », « de 50 000 à 100 000 », et ainsi de suite. Comptez ce que chaque case contient. Vous avez une **distribution de fréquences** — frequency distribution — le tableau qui dit où va le monde.

Dessinez ces comptes en barres jointives : c'est l'**histogramme**. Sur le fichier réel :

Case (FCFA)	Tickets	Part de l'effectif	Part du CA
moins de 0	3	0,9 %	-0,5 % (les avoirs retranchent)
de 0 à 50 000	133	42,1 %	7,3 %
de 50 000 à 100 000	84	26,6 %	16,5 %
de 100 000 à 150 000	25	7,9 %	8,6 %
de 150 000 à 200 000	23	7,3 %	11,0 %
de 200 000 à 250 000	17	5,4 %	10,3 %
250 000 et plus	31	9,8 %	46,9 %

Quatre faits sautent aux yeux, et aucun ne se réduit à un nombre : la série a **une bosse** à gauche, une **queue** qui ne s'arrête pas à droite, une **dernière classe ouverte** qui contient presque autant de tickets que les trois classes du milieu réunies, et cette dernière classe porte **plus du tiers de la recette**. Un histogramme de quarante-six classes aurait noyé le troisième point dans le bruit ; un histogramme de trois classes l'aurait masqué. Le choix des classes n'est pas cosmétique : c'est une décision analytique, et elle s'écrit.

4. Vocabulaire essentiel

#	Terme	Français — English — sens simple	À savoir
1	Distribution	distribution — distribution — répartition des valeurs d'une variable	se dit d'une variable, à un grain donné
2	Effectif de classe	effectif — frequency — nombre d'observations dans une case	toujours accompagner de la base (316)
3	Histogramme	histogramme — histogram — barres jointives par classes de valeurs continues	un diagramme en barres sépare des catégories, il ne compte pas des intervalles
4	Classe	classe — bin, class interval — intervalle de valeurs regroupées	« bin » en informatique, « classe » en statistique
5	Classe ouverte	classe ouverte — open-ended bin — sans borne haute (ou basse)	nécessaire quand la queue est longue ; à nommer
6b	Densité	densité de fréquence — density — effectif divisé par l'amplitude de classe	seule grandeur comparable si les classes ne sont pas égales
6	Amplitude	amplitude de classe — class width — largeur de l'intervalle	ici 50 000 FCFA ; ce n'est pas le nombre de classes

#	Terme	Français — English — sens simple	À savoir
7	Mode de distribution	mode — mode — valeur ou classe la plus fréquente	classe 0 à 50 000 FCFA ici, 133 tickets
8	Asymétrie	asymétrie — skewness — déséquilibre gauche / droite	positive = queue à droite
9	Kurtosis	aplatissement — kurtosis — épaisseur des queues	dit peu de la bosse, beaucoup des extrêmes
10	Queue	queue — tail — extrémités de la distribution	« queue épaisse » = valeurs extrêmes structurelles
11	Valeur extrême	valeur extrême — outlier — observation très éloignée du reste	trois sorts : erreur, exception, cas limite
12	Troncation	troncage — trimming — retrait encadré des extrêmes	dire combien, selon quelle règle, et pourquoi

5. Cours approfondi

5.1 Une distribution, trois miroirs

Définition. La **distribution** d'une variable décrit comment ses valeurs se répartissent : quelles valeurs, avec quelles fréquences. Elle se donne de trois manières complémentaires — le tableau d'effectifs (exact, verbeux), l'histogramme (visuel, dépendant des classes) et la fonction de répartition (cumulée, robuste, vue au chapitre M02.C04). Un analyste sait produire les trois et en lire une seule devant un commanditaire.

Regard	Ce qu'il répond	Ce qu'il cache
Tableau d'effectifs par classe	« combien dans chaque tranche », et quel CA	la continuité entre classes voisines
Histogramme	« où est la bosse, où est la queue »	les effectifs précis, si les classes sont larges
Fonction de répartition	« quel percentile à ce montant »	la forme locale, les bosses éventuelles

Le premier réflexe est de vérifier l'effet total : les sept effectifs du §3 somment à 316, l'effectif de la série. Ce contrôle, qui prend vingt secondes, attrape les classes mal bornées, les lignes perdues au tri et les doublons non retirés — trois des défauts que le chapitre M01.C04 vous a fait corriger à la main.

5.2 Construire l'histogramme : les trois décisions

Définition. Un **histogramme** (histogram) est la représentation en barres jointives des effectifs tombant dans des intervalles contigus de même amplitude — les **classes**. Sa parenté avec le diagramme en barres est trompeuse : ici la surface compte, pas la hauteur seule, ce qui impose des classes d'égale largeur pour que les barres soient comparables. Quand les largeurs diffèrent, on dessine des **densités** (density histogram, histogramme de densité) : l'effectif divisé par l'amplitude de la classe.

1. **Amplitude.** Deux règles s'offrent à vous, légitimes, aucune automatique. Celle de **Sturges** (Sturges' rule) : 1 plus le logarithme en base 2 de l'effectif, soit 9 classes ici. Celle de **Freedman-Diaconis**, robuste aux extrêmes : l'amplitude vaut deux fois l'IQR divisé par la racine cubique de l'effectif, soit 32 277 FCFA, ce qui conduirait à 46 classes sur la plage totale. Quarante-six classes pour 316 observations : environ sept tickets par classe, aucune lecture pos-

sible. Retenir 7 classes de 50 000 FCFA est un compromis, et il s'écrit comme tel : « classes de 50 000 FCFA, première et dernière ouvertes ».

2. **Origine.** Des bornes rondes en francs (0 ; 50 000 ; 100 000) se lisent sans calcul mental ; des bornes issues d'une formule (32 277) se justifient mieux mais ne se lisent pas. En entreprise, la borne ronde gagne, à condition que le choix soit tracé.
3. **Extrémités.** La dernière classe est ouverte (« 250 000 et plus ») : bornée en haut, elle ferait disparaître une partie du chiffre d'affaires de la figure. La première l'est aussi, sinon les trois avoirs sont silencieusement exclus du graphique — et un graphique qui exclut des lignes ne somme plus à l'effectif.

Combien de tickets dans chaque tranche de 50 000 FCFA (dernière classe ouverte)

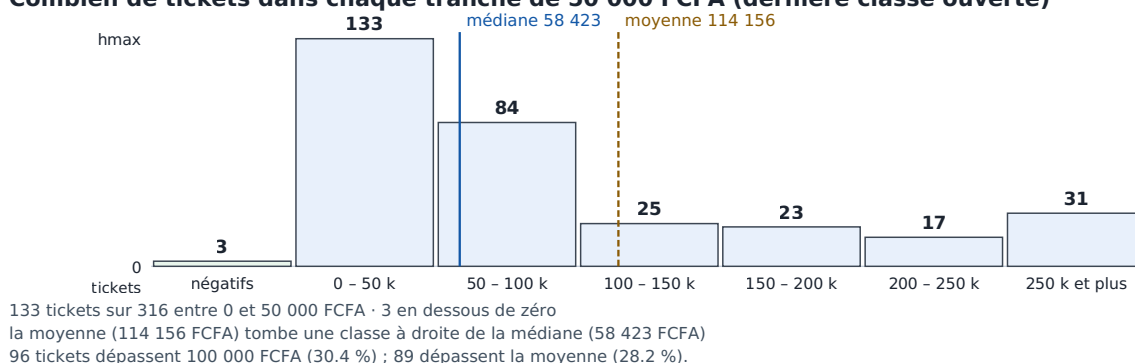


Figure — Histogramme des paniers, six classes de 50 000 FCFA, la dernière ouverte (chiffres lus dans chiffres_cites.json).

Faites-le dans un tableur. Sélectionnez la colonne des paniers, Insertion → Graphique → Histogramme. Deux réglages à corriger immédiatement : le nombre de classes — le réglage automatique produit sur cette plage des intervalles d'amplitude non ronde, que personne ne lit — et le nom des deux classes extrêmes. Si votre tableur ne sait pas ouvrir une classe, créez une colonne auxiliaire de regroupement par formules (`=SI(v>=250000;"250 k et plus";SI(v<0;"négatifs";...))`) : c'est plus long, mais le graphique devient vérifiable ligne à ligne, et il se trie.

Faites-le dans le SQL. (les vues `lignes_ventes` et `paniers` ont été créées au C02, § « Pour la suite du module ») :

```
SELECT CASE
    WHEN panier < 0      THEN '00. négatifs'
    WHEN panier < 50000  THEN '01. 0 - 50 k'
    WHEN panier < 100000 THEN '02. 50 - 100 k'
    WHEN panier < 150000 THEN '03. 100 - 150 k'
    WHEN panier < 200000 THEN '04. 150 - 200 k'
    WHEN panier < 250000 THEN '05. 200 - 250 k'
    ELSE                 '06. 250 k et plus'
END AS classe,
COUNT(*)           AS tickets,
SUM(panier)         AS ca
FROM paniers
GROUP BY classe
ORDER BY classe;
```

Les préfixes numérotés ne sont pas décoratifs : sans eux, `ORDER BY classe` range « 100 - 150 k » avant « 50 - 100 k » en ordre lexicographique, et votre figure raconte une bosse qui n'existe pas. C'est l'une des erreurs de représentation les plus fréquentes en SQL, et la plus invisible.

5.3 L'asymétrie : combien de queue à droite

Définition. L'**asymétrie** (skewness, notée γ_1 ou skew) mesure le déséquilibre de la distribution autour de son centre : en simplifiant, la moyenne des écarts au centre élevés au cube, normalisée par le cube de l'écart-type. Positive, la queue s'étire à droite ; négative, à gauche ; nulle, la distribution est symétrique. Le cube donne son poids à la direction : une valeur très à droite contribue positivement, une valeur très à gauche négativement.

Sur les 316 paniers, l'asymétrie vaut 3,84. La grille d'interprétation d'usage, en valeur absolue : moins de 0,5, quasi symétrique ; de 0,5 à 1, asymétrie modérée ; au-delà de 1, asymétrie forte. À 3,84, la série est fortement asymétrique à droite, et le contrôle croisé est immédiat : la moyenne (114 156 FCFA) vaut 1,95 fois la médiane (58 423 FCFA), et 71,8 % des tickets sont sous la moyenne (mesures M02.C02).

Le lien entre ces trois nombres se comprend plutôt qu'il ne se retient : plus la queue de droite est lourde, plus la moyenne est tirée vers le haut par rapport à la médiane, plus l'asymétrie est grande. Vous pouvez donc poser le diagnostic avec deux nombres seulement, sans logiciel de statistique — et c'est ce que vous ferez en réunion, parce que γ_1 ne se justifie pas devant un non-statisticien. Le coefficient, lui, sert à documenter : une ligne dans l'annexe méthodologique.

Attention. Un coefficient d'asymétrie ne se compare pas entre deux colonnes de grains différents. L'asymétrie des montants par ticket et celle des montants par ligne décrivent deux objets distincts : les comparer, c'est comparer la clientèle et les lignes de commande. Écrivez le grain à côté du chiffre.

5.4 Le kurtosis : ce que les queues racontent

Définition. Le **kurtosis** (kurtosis ; « aplatissement » en français, bien qu'il ne mesure pas la hauteur du pic contrairement à une légende tenace) pénalise la présence de valeurs lointaines : moyenne des écarts au centre élevés à la quatrième puissance, normalisée. pandas, Excel (=KURT) et LibreOffice renvoient l'**excès de kurtosis** (convention de Fisher) : 0 pour une loi normale, positif quand les queues sont épaisses.

Notre série affiche un excès de kurtosis de 18,25, soit 21,25 dans la convention de Pearson, qui ajoute 3. Le message : la distribution n'est pas seulement penchée, ses queues sont épaisses. Traduit en faits vérifiables sur l'extrait : 25 tickets dépassent 300 000 FCFA, 12 dépassent 500 000 FCFA, et le rapport du plus gros ticket à la médiane vaut 23,0.

Pourquoi ce chiffre intéresse le praticien : presque tous les outils des modules suivants — intervalles de confiance (M02.C08), droite de régression (M02.C06), contrôle statistique en production — supposent implicitement des queues modérées. Avec un excès de kurtosis de 18,25, un intervalle construit « à $\pm 1,96$ écart-type » est faux : la formule reste, mais il faut tronquer, ou passer par un calcul par ré-échantillonnage (M02.C07). Vous n'avez pas besoin de faire ce calcul aujourd'hui ; vous avez besoin de savoir **lire l'avertissement quand un autre le fait**, et le citer.

Faites-le dans un tableur. =KURT(plage) existe dans Excel et dans LibreOffice Calc et renvoie l'excès, comme pandas ; les versions récentes d'Excel le nomment =KURT(), l'anglais KURT restant l'appellation d'origine. Vérifiez toujours la convention de votre logiciel avant de citer le chiffre : un kurtosis de Pearson lu comme un excès de Fisher décale de 3 — écart minuscule sur l'indicateur, décisif sur la conclusion qu'on en tire.

5.5 Trois sortes de valeurs extrêmes, trois traitements

Le mot « aberrant » est le plus dangereux du vocabulaire statistique en entreprise : il ferme la discussion au lieu de l'ouvrir. Sur l'extrait, les points isolés relèvent de trois natures différentes, et c'est la nature qui décide du traitement.

Sorte	Exemple mesuré	Traitement	Phrase de note
Erreur de saisie	7 lignes de quantité multipliées par 1 000 dans l'énoncé (maximum 14 000, ramené à 68 après correction)	corriger à la source, garder la trace	« valeur corrigée selon la règle X, N lignes concernées »
Exception réelle	le plus gros panier, 1 343 666 FCFA	conserver, isoler dans l'analyse, nommer	« un chantier porte près de la moitié du CA de l'extrait »
Cas limite légitime	3 paniers négatifs, dont un à -150 804 FCFA (8 lignes d'avoirs, -283 933 FCFA au total)	conserver au total de caisse, exclure des indicateurs de vente	« les avoirs sont comptés à part ; le CA reste le CA »

Un même indicateur change de valeur selon le traitement, et c'est précisément ce qui doit s'écrire : en retirant les 25 tickets au-dessus de la borne haute de Tukey, la moyenne des paniers passe de 114 156 à 71 463 FCFA et le coefficient de variation de 159,5 % à 96,6 % (M02.C03). La série devient descriptible — mais elle ne décrit plus la même population.

Conseil professionnel. N'écrivez jamais « nous avons supprimé les valeurs aberrantes ». Écrivez : « trois tickets sont des avoirs, exclus des indicateurs de vente et inclus au total de caisse ; vingt-cinq dépassent la borne haute, conservés et commentés à part ; sept lignes contenaient une erreur de saisie corrigée en amont ». Trois phrases, trois décisions, plus aucune ambiguïté sur ce que vous avez fait des données.

5.6 Commenter une forme en cinq phrases — grille de rédaction

La forme ne se commente pas en adjectifs. Appliquez cette grille à toute série ; elle tient en un paragraphe.

1. **Étalement** : « la série va de -150 804 à 1 343 666 FCFA » (min, max).
2. **Accumulation** : « 136 tickets sur 316, soit 43,0 % de l'effectif, sont sous 50 000 FCFA, pour 6,8 % du CA » (classe modale et sa part monétaire).
3. **Centre robuste** : « la médiane est à 58 423 FCFA, la moyenne à 114 156 FCFA, rapport de 1,95 ».
4. **Queue** : « 31 tickets dépassent 250 000 FCFA et portent 46,9 % du CA ; 12 dépassent 500 000 FCFA ».
5. **Décision** : « ces trente-et-un tickets sont traités sur une ligne séparée du rapport mensuel ».

Les quatre premières phrases viennent des mesures du socle ; la cinquième est une décision. C'est la structure à réutiliser dans tous les modules : deux faits, un contraste, une extrémité, une décision.

5.7 Ce que cette forme interdit de faire

Trois interdictions tirées de cette série précise, pas d'un manuel :

- **la moyenne seule comme indicateur de politique commerciale.** 89 tickets dépassent la moyenne, soit 28,2 % de l'effectif : un dispositif calé sur « au-dessus de la moyenne » exclut sept clients sur dix, sans que personne ne l'ait décidé explicitement.
- **le raisonnement en « $\pm$ un écart-type ».** Cette lecture suppose une distribution à peu près symétrique ; avec une asymétrie de 3,84 et un excès de kurtosis de 18,25, l'intervalle autour de la moyenne ne couvre pas ce qu'on croit (M02.C03, §8.6).

- **le seuil « trois sigma » en contrôle interne.** Sur une série à queues épaisses, un tel seuil signale une fois sur deux la nature du métier, pas une anomalie. La borne de Tukey, assise sur l'IQR, reste la règle défendable à ce niveau de maturité de l'entreprise.

5.8 Représenter la forme : quelle figure pour quelle question

Question posée	Figure à produire	Ce qu'il faut écrire sur la figure
Où est la bosse, combien de bosses ?	histogramme (ce chapitre)	amplitude des classes, unité, effectif total
Qui possède quoi ?	courbe des fréquences cumulées	percentile lu sur l'axe supérieur (M02.C04)
Comparer des catégories	boîtes à moustaches alignées	médiane et nombre de points isolés par catégorie
Deux varient-elles ensemble ?	nuage de points	les 316 points, pas une moyenne par catégorie (M02.C06)
La série est-elle stable dans le temps ?	courbe mensuelle	dénominateur commun, mois creux explicités (M01.C04)

Boîte à outils. Contrôle minimal de toute figure de forme : (1) la somme des effectifs de classes doit redonner l'effectif total — 316 ici ; sinon, des observations sont hors du graphique ; (2) les classes ouvertes s'affichent avec la mention « et plus » ou « moins de », jamais avec une valeur imaginaire ; (3) une figure sans graduations d'axe ne se publie pas ; (4) le choix d'amplitude tient en une phrase dans la légende ; (5) si la figure contredit le tableau de chiffres, la figure a tort en premier — un histogramme se rate plus facilement qu'une moyenne.

6. Exemple concret : « nos clients ne sont pas comme ça »

La situation. Vous présentez le premier tableau de bord mensuel : panier moyen 114 156 FCFA, écart-type 182 095 FCFA, sept catégories. Le responsable du magasin refuse le chiffre — et il n'a pas tort : sa journée ressemble à 133 tickets sous 50 000 FCFA, pas à 114 156 FCFA.

Ce qui se joue. Il ne conteste pas le calcul, il conteste le résumé. Sa journée type correspond à la classe modale (0 à 50 000 FCFA, 42,1 % des tickets pour 7,3 % du CA) ; votre moyenne correspond à la queue. Les deux sont vrais ; un seul décrit le quotidien.

Attention. Une part d'effectif n'est pas une part de chiffre d'affaires, et le tableur calcule les deux sans vous avertir du changement d'objet : la classe 0 à 50 000 FCFA réunit 42,1 % des tickets et 7,3 % des encaissements, la classe ouverte 31 tickets (9,8 %) et 46,9 % du CA. Écrivez dans la légende ce qui est réparti — le nombre d'achats ou les francs encaissés. En titre de graphique, « part des clients » et « part du chiffre d'affaires » s'inversent souvent, et c'est la source numéro un des contresens en réunion.

La correction, en trois gestes. (1) Publier la médiane en indicateur, la moyenne en note. (2) Mettre l'histogramme du §5.2 en tête de page : le refus devient accord, parce que la figure montre ce que le nombre cachait. (3) Séparer le rapport en deux lignes de pilotage : comptoir (sous 50 000 FCFA, 133 tickets, 7,3 % du CA) et chantiers (au-dessus de 250 000 FCFA, 31 tickets, 46,9 % du CA), la classe du milieu (149 tickets, 46,4 %) servant de témoin.

Ce que le manager retiendra. « On ne pilote pas le comptoir avec les chiffres des chantiers » — phrase sans vocabulaire statistique, qui vient entièrement de la forme de la distribution. C'est

le meilleur argument à opposer à une direction générale qui préfère « un seul chiffre, c'est plus simple ».

7. Démonstration pas à pas : des comptes de classes à la bosse

Étape 1 — La série. Colonne des paniers, 316 valeurs (M02.C02). Conservez les valeurs négatives : c'est la première classe qui les accueille.

Étape 2 — Les bornes. Écrivez les bornes basses en colonne : -1 ; 0 ; 50 000 ; 100 000 ; 150 000 ; 200 000 ; 250 000. Le -1 n'est pas une plaisanterie : il permet de compter les négatifs dans leur propre classe au lieu de les perdre.

Étape 3 — Les comptes. `=NB.SI.ENS(panier;">="&$A2;panier;"<"&A3)` recopié vers le bas, soit 3 ; 133 ; 84 ; 25 ; 23 ; 17. La dernière classe se compte sans borne haute : `=NB.SI(panier;">=250000")` → 31.

Étape 4 — Le contrôle d'additivité. Somme des sept comptes : 316. À faire avant de dessiner quoi que ce soit : cette unique addition détecte les bornes mal écrites, et elle constitue la moitié des erreurs d'histogramme.

Étape 5 — Les parts. Divisez par 316 : 42,1 % pour la deuxième classe, 26,6 % pour la troisième. Écrivez la phrase : « la classe modale est 0 à 50 000 FCFA, avec 133 tickets sur 316 ».

Étape 6 — La moyenne et la médiane sur la figure. Placez-les sur l'axe : la médiane (58 423 FCFA) tombe dans la troisième classe, la moyenne (114 156 FCFA) une classe plus loin. Ce décalage visible fait comprendre l'asymétrie à un lecteur non averti — mieux que le coefficient de 3,84.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Somme des effectifs de = n		316	borne haute manquante
2	Classe ouverte recomptée au filtre	identité	31	tri ou comparateur
3	Part de la classe modale	≈ 42 %	42,1 %	amplitude modifiée sans le dire
4	Ordre des classes dans la figure	tri numérique, lexicographique	pas préfixes « 01. », « 02. »	ORDER BY classe sans préfixe
5	Position relative moyenne / médiane	moyenne à droite	cohérent avec §6	série non triée

8. Erreurs fréquentes

- Les classes automatiques du logiciel.** Une amplitude issue d'un réglage par défaut produit des bornes sans rapport avec les usages du métier : des tranches qui ne commencent sur aucun multiple rond, et que personne ne lit. Réglez l'amplitude vous-même, et écrivez le réglage.
- Borne supérieure fermée trop tôt.** Dans la première version de la figure de ce chapitre, les bornes s'arrêtaient trop bas : la dernière classe absorbait tout le reste en un tas informe, et la queue disparaissait. Corriger a consisté à ouvrir la dernière classe — et à rendre au graphique sa part de CA (46,9 %).
- Confondre diagramme en barres et histogramme.** Le premier sépare des modalités disjointes (des catégories) ; le second joint des intervalles contigus. Mettre des espaces entre les

barres d'un histogramme suggère qu'il existe des montants « entre les cases » : ce n'est pas le cas.

4. **Lire le kurtosis comme une mesure de hauteur de pic.** Il parle des queues. La phrase correcte est « la série a des queues épaisses (excès de kurtosis 18,25) », pas « la série est pointue ».
5. **Supprimer les extrêmes pour « lisser ».** Après retrait des 25 tickets hors borne, le CV tombe à 96,6 % : le chiffre s'améliore, la population change. Dites-le, sinon vous présentez un magasin qui n'existe pas.
6. **Publier une forme sans effectif.** « Distribution asymétrique à droite » sur 316 tickets et sur 12 tickets ne veut rien dire de même : sur 12 valeurs, trois valeurs hautes suffisent à produire une forte asymétrie. L'effectif se cite avec la forme.

9. Bonnes pratiques professionnelles

1. **Écrire les trois décisions de construction** : amplitude, origine, ouverture des classes. Un histogramme reproductible est un histogramme dont on peut refaire les classes.
2. **Contrôler la somme des effectifs avant la figure** : vingt secondes, la moitié des erreurs évitées.
3. **Accompagner toute moyenne d'un diagnostic de forme** en une phrase : « asymétrie forte, médiane à retenir ».
4. **Séparer comptoir et chantiers dans les indicateurs**, pas seulement dans les discours : si la bosse et la queue appellent des décisions différentes, elles appellent deux lignes dans le rapport.
5. **Nommer le sort de chaque valeur extrême** (corrigée / conservée et commentée / exclue d'un périmètre) et son effectif, une fois pour toutes, dans l'annexe méthodologique.
6. **Conserver la table de comptes à côté de la figure propre** : c'est elle qui sert de preuve quand un chiffre est contesté six mois plus tard.

10. Exercice guidé — refaire les classes, et voir ce qui change

Énoncé. Sur les 316 paniers, construisez deux histogrammes : l'un en classes de 50 000 FCFA (votre référence), l'autre en classes de 100 000 FCFA. Comparez les deux lectures et concluez en trois lignes sur ce que le second masque.

Vous faites — barème 10 points, total 10.

Étape	Ce que vous produisez	Points
A	Les comptes des deux découpages, somme contrôlée à 316	3
B	Les deux figures, axes gradués, classes ouvertes mentionnées	3
C	La phrase sur ce que le grossissement masque	2
D	Le choix argumenté de l'un des deux pour le rapport mensuel	2

Nous vérifions. Le découpage fin donne 3 ; 133 ; 84 ; 25 ; 23 ; 17 ; 31. Le découpage large rassemble presque toute la masse dans les deux premières classes et ne montre plus qu'une traîne sans structure. Ce que le second masque : la concentration dans la classe 0 à 50 000 FCFA (42,1 % des tickets pour 7,3 % du CA) devient indiscernable d'une descente régulière, et les 31 tickets de la classe ouverte cessent d'exister en tant que tels.

Nous corrigeons. L'erreur classique est de conclure « le fin est meilleur ». Le critère n'est pas la finesse mais la question posée : pour répartir une charge de travail au comptoir, 50 000

FCFA ; pour une étude par tranches fiscales, 100 000 FCFA. La figure sert la décision, pas la statistique.

11. Exercices autonomes

Exercice 5.1 (★) — Le contrôle d'additivité. Refaites les comptes de classes de 50 000 FCFA sur les montants **par ligne** (480 lignes) et vérifiez la somme. Que devient la classe modale par rapport à celle des paniers, et pourquoi ? (attendu : somme 480 ; la masse s'accumule encore plus à gauche, la médiane par ligne étant sous la première borne)

Exercice 5.2 (★) — Les deux extrêmes. Retrouvez les paniers négatifs et les paniers les plus élevés ; classez chacun dans les trois sortes du §5.5 en une phrase de justification.

Exercice 5.3 (★★) — Sturges contre Freedman-Diaconis. Recomptez les deux règles sur la série (9 classes ; amplitude de 32 277 FCFA, donc 46 classes) et dites laquelle vous appliquez à un rapport mensuel, en justifiant par la lisibilité et non par l'élégance.

Exercice 5.4 (★★) — La deuxième bosse. Une stagiaire affirme : « il y a deux bosses, une vers 25 000 FCFA et une vers 150 000 FCFA, donc nous avons deux clientèles ». Que vérifiez-vous avant de retenir ou de rejeter l'affirmation, et que répond-on avec nos sept classes ?

Exercice 5.5 (★★★) — Note de forme pour la direction. Six lignes, selon la grille du §5.6 (étalement, accumulation, centre robuste, queue, décision), sans citer un nombre absent des mesures du socle, et en proposant un découpage du rapport mensuel en deux lignes de pilotage.

12. Correction détaillée

Exercice 5.1. Sur les lignes, la masse s'accumule davantage à gauche : la médiane par ligne vaut 37 966 FCFA, déjà sous la première borne de 50 000 FCFA, et les extrêmes de la colonne sont -150 804 et 1 229 678 FCFA. La somme des comptes doit donner 480. Explication attendue : 480 lignes pour 316 tickets, 1,52 ligne par ticket en moyenne — découper les gros tickets en morceaux change l'objet observé, pas seulement son échelle.

Exercice 5.2. Trois paniers négatifs, dont un à -150 804 FCFA, issus de 8 lignes d'avoirs totalisant -283 933 FCFA : correction légitime, conservée au total de caisse et exclue des indicateurs de vente. Au-dessus, 25 tickets dépassent 300 000 FCFA et 12 dépassent 500 000 FCFA, le plus gros à 1 343 666 FCFA : exceptions réelles, conservées et commentées. Aucune de ces valeurs n'est « corrigée » : on ne corrige pas un chantier.

Exercice 5.3. Sturges donne 9 classes (1 plus le logarithme en base 2 de 316) ; Freedman-Diaconis une amplitude de 32 277 FCFA, soit 46 classes sur la plage. Réponse attendue : aucune des deux mécaniquement — on retient 7 classes de 50 000 FCFA pour le rapport mensuel parce que la borne ronde se lit sans effort, en écrivant que la règle robuste conduirait à un découpage infiniment plus fin et illisible avec 316 observations. Ce qui est noté, c'est la trace du choix.

Exercice 5.4. On vérifie (a) que la bosse résiste au changement d'amplitude : une bosse réelle persiste quand on double ou divise les classes, une bosse artificielle se déplace avec les bornes ; (b) que l'effectif le permet : sur 316 observations, une épaule de quelques dizaines de tickets est à peine distinguable du bruit d'échantillonnage. Avec nos classes : une bosse nette (0 à 50 000 FCFA, 133 tickets) et une queue épaisse (31 tickets au-dessus de 250 000 FCFA portant 46,9 % du CA), pas deux bosses comparables. Conclusion professionnelle : la stagiaire a vu juste sur l'existence de deux clientèles, mais la frontière ne se lit pas dans une deuxième bosse — elle se lit dans la queue, et c'est là qu'on la fixe.

Exercice 5.5. Note attendue. Étalement : de -150 804 à 1 343 666 FCFA. Accumulation : 136 tickets sous 50 000 FCFA (43,0 % de l'effectif) pour 6,8 % du CA. Centre : médiane 58 423 FCFA, moyenne 114 156 FCFA, rapport 1,95. Queue : 31 tickets au-dessus de 250 000 FCFA, 9,8 % de

l'effectif, 46,9 % du CA ; 12 au-dessus de 500 000 FCFA. Décision : deux lignes de pilotage, comptoir et chantiers, la moyenne étant conservée comme total de caisse rapporté à l'effectif. Est compté 0 tout nombre que le socle ne mesure pas — marge, rotation de stock, taux de service : ce chapitre ne les connaît pas.

13. Mini-projet M02.P2 — suite : la page de forme (30 min)

Complétez le document du **M02.P2** (barème 20 points, seuil de validation 13) par la page « forme » : l'histogramme des paniers en classes de 50 000 FCFA avec ses comptes de classes en annexe, les deux histogrammes comparés de l'exercice 5.1, et le tableau des valeurs extrêmes en trois colonnes (nature, effectif, traitement). Deux contraintes éliminatoires au barème : le mot « aberrant » n'apparaît pas — chaque valeur extrême est nommée par sa nature métier et son sort — et le découpage en deux lignes de pilotage est justifié par une phrase chiffrée reprenant les parts d'effectif et de CA.

14. Résumé du chapitre

1. Une distribution se regarde dans trois miroirs — effectifs, histogramme, fréquences cumulées — et aucun ne suffit ; la somme des effectifs de classes (316) est le contrôle qui valide les trois.
2. Les classes se **choisissent et s'écrivent** : 50 000 FCFA par lisibilité, première et dernière ouvertes ; Sturges (9 classes) et Freedman-Diaconis (amplitude de 32 277 FCFA, donc 46 classes) sont des points de départ, pas des verdicts.
3. La forme de nos paniers : bosse à gauche (133 tickets sous 50 000 FCFA, 7,3 % du CA), queue à droite (31 tickets au-dessus de 250 000 FCFA, 46,9 % du CA), asymétrie 3,84, excès de kurtosis 18,25 — quatre chiffres qui disent la même chose, et qui interdisent la moyenne seule, le « $\pm \sigma$ » et le seuil « trois sigma ».
4. Trois sorts des extrêmes : erreurs à corriger à la source, exceptions réelles à commenter, cas limites à sortir d'un périmètre d'indicateur. Le traitement s'écrit, il ne se devine pas.

15. À retenir

À retenir.

- Un histogramme se lit d'abord par sa somme d'effectifs (316) et par ses classes ouvertes, pas par sa bosse.
- Asymétrie 3,84, excès de kurtosis 18,25 : la moyenne (114 156 FCFA) est tirée par la queue, la médiane (58 423 FCFA) décrit le quotidien, le rapport des deux (1,95) est le chiffre à retenir dans la note.
- « Anormal » n'est pas un statut de donnée : une valeur extrême est une erreur, une exception ou un cas limite, chacune avec un traitement écrit.
- La forme décide de la licéité des outils : sur cette série, moyenne et écart-type restent valables pour la comptabilité ; pour la décision, on passe aux positions et au ré-échantillonnage.

À retenir. La formulation qui protège. « Sur 316 tickets de l'extrait nettoyé, la distribution des paniers est fortement asymétrique à droite (asymétrie 3,84 ; 42,1 % des tickets sous 50 000 FCFA ; 9,8 % au-dessus de 250 000 FCFA, portant 46,9 % du CA). Nous publions la médiane comme indicateur de pilotage et la moyenne comme total de caisse rapporté à l'effectif. »

16. Évaluation formative (auto-correction, 12 min)

1. Pourquoi un histogramme à classes automatiques est-il un risque professionnel, et non une maladresse esthétique ? **2.** Que dit un excès de kurtosis de 18,25 que ne dit pas une asymétrie de 3,84 ? **3.** Comment vérifier en dix secondes qu'un histogramme ne perd pas d'observations ? **4.** Citez les trois sortes de valeurs extrêmes et leur traitement, avec un exemple chiffré du fichier. **5.** Vrai ou faux : « puisque 42,1 % des tickets sont sous 50 000 FCFA, il faut supprimer la livraison gratuite ». Qu'est-ce qui manque à cette phrase pour être une conclusion ?

Corrigé

1. Parce que l'amplitude change la forme visible, donc l'histoire racontée : nos 7 classes de 50 000 FCFA donnent une bosse et une queue, 46 classes de 32 277 FCFA donnent un bruit, 2 classes ne donnent rien de décisionnable. 2. L'asymétrie dit que la queue est à droite ; le kurtosis dit qu'elle est épaisse — que les valeurs très éloignées sont structurelles et non accidentelles, ce qui invalide les outils calés sur des queues légères. 3. En sommant les effectifs de classes et en comparant à l'effectif total : ici 316. 4. Erreur de saisie : les 7 lignes de quantités multipliées par mille, corrigées en amont ; exception réelle : le chantier à 1 343 666 FCFA, conservé et commenté ; cas limite : les 3 paniers négatifs (8 lignes d'avoirs pour -283 933 FCFA), exclus des indicateurs de vente et conservés au total de caisse. 5. Vrai sur le premier membre seulement ; la conclusion manque de l'effet chiffré — combien de tickets perdent le bénéfice et quel CA est en jeu (M02.C04, §6) — donc elle ne décide rien.

Auto-validation. Reprenez les cinq contrôles du §7 : somme des classes $316 \cdot 31$ tickets dans la classe ouverte $\cdot 42,1\%$ dans la classe modale $\cdot$ classes numérotées $\cdot$ moyenne à droite de la médiane. Puis comparez vos cinq nombres de la grille du §5.6 aux mesures : $1\,343\,666 \cdot 133 \cdot 58\,423 \cdot 31 \cdot 46,9\%$. En cas d'écart, reprenez les bornes de classes avant les formules : c'est là que l'erreur se loge.

M02.C06 — Relier deux variables : covariance, corrélation, causalité, paradoxes

L'idée du chapitre. Deux colonnes varient-elles ensemble ? La corrélation répond en un nombre, et ce nombre est le plus mal employé de toute la statistique d'entreprise. Ce chapitre pose la mécanique — covariance, coefficient, nuage, droite — et surtout la frontière qui sépare « va avec » de « fait aller avec ».

Base de travail — obligatoire. Deux fichiers du socle, dans votre dossier 02_exercices/M02/, ouverts côte à côte : `donnees/projection/ventes_magasin5_2025.csv` (489 lignes × 13 variables) et `donnees/reference/ventes_magasin5_2025_ATTENDU.csv` (480 × 13). Séparateur point-virgule, encodage UTF-8, virgule décimale. Montants en franc CFA (FCFA). Grain de référence : la **ligne** (480 observations) pour quantité, prix, remise et montant ; le **panier** (316 tickets) pour les questions de clientèle. Chiffres mesurés dans `01_socle_donnees/data/reference/chiffres_cites.md`, section M02.

1. Objectifs du chapitre

À la fin de ce chapitre, vous êtes capable de :

1. calculer une covariance et une corrélation, et expliquer pourquoi la première ne se communique pas ;
2. distinguer corrélation de Pearson et corrélation de rangs (Spearman), et choisir l'une ou l'autre selon la présence de valeurs extrêmes ;
3. estimer une droite de régression simple, en lire la pente, l'ordonnée à l'origine et le R^2 , et refuser d'extrapoler hors du domaine observé ;
4. énoncer les quatre explications possibles d'une corrélation observée — lien direct, causalité inverse, cause commune, hasard de l'échantillonnage — et dire ce que vos données permettent de trancher ;
5. contrôler une **variable de confusion** (la catégorie de produits, le vendeur, la période) et vérifier si l'écart survit à ce contrôle ;
6. repérer les trois paradoxes usuels du métier — corrélation mécanique, effet d'agrégation, retournement de Simpson — avec un exemple vérifié sur le fichier.

2. Pourquoi cette notion est importante

Les questions professionnelles ne sont presque jamais des questions de niveau : ce sont des questions de liaison.

- « **La remise fait-elle vendre plus ?** » C'est une corrélation à tester entre `remise` et `montant_ttc`. Sur le fichier, elle vaut -0,031 : les lignes les plus remisées ne sont pas les plus grosses. Cette absence de liaison, si le test du chapitre M02.C08 la confirme, contredit une idée reçue du magasin et ouvre une décision : la remise actuelle n'achète pas de volume mesurable.
- « **Est-ce le prix qui fait le montant, ou la quantité ?** » Les deux corrélations mesurées valent 0,436 pour le prix unitaire et 0,712 pour la quantité : le volume pèse davantage que

l'étiquette dans la recette par ligne. Voilà un argument pour piloter le panier, pas seulement le prix.

- « **Nos meilleurs vendeurs ont-ils les meilleurs produits ?** » La question est exactement celle de la variable de confusion : Bationo a le panier moyen le plus élevé, 103 367 FCFA, et la plus faible part de Bois & panneaux dans ses lignes, 2,4 %. Le contrôle à catégorie contournée, chiffré au §5.6, donne un résultat contre-intuitif.

Dans les faits. Dans une entreprise de cette taille, la corrélation sert surtout d'argument d'autorité : « les gros paniers sont remisés, donc la remise crée du panier ». Vous n'êtes pas là pour produire ce type de phrase, mais pour être celui qui demande : sur quel échantillon, à quel grain, et avez-vous contrôlé la catégorie ? Trois questions qui, sur notre extrait, transforment une affirmation en décision mesurée.

3. Explication simple

Deux colonnes varient ensemble quand, en parcourant les lignes, on constate une régularité : quand l'une monte, l'autre monte (ou baisse). Le **nuage de points** — scatter plot — montre cette régularité ; le **coefficient de corrélation** la résume en un nombre entre -1 et +1.

Le cas le plus parlant du fichier : la quantité vendue sur une ligne et le montant de cette ligne. Plus il y a de sacs, plus le montant monte ; le nuage s'allonge, le coefficient vaut 0,712. Pour mesurer cette allure, on calcule d'abord la **covariance** — covariance — la moyenne des produits des écarts aux deux moyennes : quand les deux variables montent et baissent ensemble, les produits sont positifs, la covariance est positive.

La covariance a un défaut rédhibitoire : elle s'exprime en « unités de x × unités de y », soit ici des pièces-FCFA. Sa valeur, 857 634, ne se lit pas et ne se compare pas. La corrélation est cette covariance nettoyée de ses unités : on divise par les deux écarts-types, et il ne reste qu'un nombre sans unité.

Trois réflexes avant toute formule : (1) regarder le nuage avant le coefficient ; (2) vérifier que les deux colonnes décrivent le même objet au même grain — corréler une moyenne par client avec une ligne de commande est une faute ; (3) écrire ce que la liaison ne dit pas, à commencer par la causalité.

4. Vocabulaire essentiel

#	Terme	Français — English — sens simple	À savoir
1	Covariance	covariance — covariance — moyenne des produits des écarts aux moyennes	garde le signe, perd l'échelle ; <code>COVARIANCE.S / COVAR_SAMP</code>
2	Corrélation linéaire	coefficient de corrélation de Pearson — Pearson correlation — covariance standardisée, entre -1 et +1	ne mesure que le lien linéaire
3	Corrélation de rangs	corrélation de rang de Spearman — Spearman rank correlation — le même calcul sur les rangs	robuste aux extrêmes, capte les liaisons monotones
4	Nuage de points	nuage de points — scatter plot — une observation = un point	l'étape que personne ne saute impunément
5	R ²	coefficient de détermination — coefficient of determination — part de variance expliquée	en régression simple, $R^2 = r$ au carré
6	Pente	pente — slope — variation moyenne de y par unité de x	12 237 FCFA par pièce sur notre série
7	Ordonnée à l'origine	ordonnée à l'origine — intercept — valeur prédite quand x vaut zéro	ici -10 402 FCFA : une ligne de quantité nulle n'existe pas

#	Terme	Français — English — sens simple	À savoir
8	Causalité	lien de cause à effet — causality — agir sur x change y	ne se déduit pas d'une corrélation
9	Variable de confusion	variable de confusion — confounder — cause commune des deux variables	catégorie, vendeur, saison : trois candidats ici
10	Corrélation partielle	corrélation partielle — partial correlation — lien restant après contrôle d'une tierce	l'outil conceptuel du §5.6
11	Extrapolation	extrapolation — extrapolation — prédire hors du domaine observé	proscrite en note de gestion
12	Régression	régression linéaire simple — simple linear regression — droite des moindres carrés	une méthode d'ajustement, pas une preuve

5. Cours approfondi

5.1 La covariance, ou mesurer sans unité

Définition. La **covariance** entre deux variables x et y est la moyenne des produits des écarts : pour chaque observation, $(x_i - \text{moyenne de } x) \times (y_i - \text{moyenne de } y)$. Elle est positive si les deux varient dans le même sens, négative si elles varient en sens contraire, proche de zéro si aucune allure ne se dégage. On la note $\text{cov}(x, y)$.

Sur les 480 lignes du fichier, la covariance entre `quantite` et `montant_ttc` vaut 857 634. Ce nombre dit le signe et une idée d'ampleur, rien de plus : il change si l'on passe du niveau ligne au niveau ticket, et il devient absurde dès qu'on change de monnaie. Sa seule utilité professionnelle est pédagogique — comprendre que la corrélation est une covariance débarrassée de ses unités.

Faites-le dans un tableur. `=COVARIANCE.S(plage_x; plage_y)` dans Excel comme dans LibreOffice (anglais `COVAR.S`), ou la voie lente, à faire une fois : trois colonnes (écart à la moyenne de x , écart à la moyenne de y , produit des deux) puis `=MOYENNE()` sur la troisième. Vous verrez ce que le signe veut dire : les lignes où beaucoup de pièces s'accompagnent de gros montants ajoutent des produits positifs, et ce sont les plus nombreuses.

Attention. Une covariance calculée sur deux colonnes de natures différentes — un identifiant et un prix, une date et un montant — n'est pas nulle par accident : elle est dénuée de sens, parce que l'ordre des identifiants ne signifie rien. Le logiciel, lui, calcule sans protester. C'est à vous de trier les colonnes autorisées, et c'est le rôle du tableau de typage construit au chapitre M01.C05.

5.2 Le coefficient de corrélation, et ce qu'il mesure vraiment

Définition. Le **coefficient de corrélation linéaire de Pearson** — Pearson correlation coefficient, noté r — est la covariance divisée par le produit des deux écarts-types. Il vaut +1 quand les points sont exactement alignés en montant, -1 exactement alignés en descendant, et zéro quand aucune allure linéaire ne se dégage. Sans unité, il se compare entre colonnes et entre périodes.

Sur le fichier, les couples mesurés donnent :

Couple de variables	Grain	r	Ce que ça veut dire
<code>quantite</code> × <code>montant_ttc</code>	ligne (480)	0,712	lien franc, mais très imparfait

Couple de variables	Grain	r	Ce que ça veut dire
prix_unitaire_ht × montant_ttc	ligne (480)	0,436	le prix compte, moins que le volume
quantite × prix_unitaire_ht	ligne (480)	-0,041	gros volumes et prix bas ne vont pas ensemble
remise × montant_ttc	ligne (480)	-0,031	aucune liaison linéaire entre remise et taille de ligne
heure de saisie × montant_ttc	ligne (480)	0,007	l'heure n'explique pas le montant
montant_ht × montant_ttc	ligne (480)	1,000	liaison mécanique : l'un se déduit de l'autre

Quatre lectures de professionnel :

1. **0,712 est fort, pas écrasant.** Le carré de ce coefficient, 50,7 % — le R^2 du §5.4 — dit que la quantité explique la moitié de la variance du montant. La moitié. Écrire « le montant dépend de la quantité » sans le R^2 est une exagération.
2. **Le couple remise / montant est le plus instructif.** Un montant plus élevé ne s'accompagne pas d'une remise plus forte (-0,031), et les gros volumes ne se négocient pas mieux (-0,041). Résultat négatif, et un résultat négatif mesuré vaut mieux qu'une intuition : la politique de remise du magasin n'est pas calée sur la taille de la ligne.
3. **$r = 1,000$ entre HT et TTC est trivial.** Les deux colonnes se déduisent l'une de l'autre par la TVA et la remise : ce n'est pas une découverte, c'est une identité comptable. Repérer une corrélation mécanique est une compétence — avant de commenter un coefficient élevé, demandez si une colonne n'est pas construite à partir de l'autre.
4. **Un r proche de zéro n'est pas une indépendance.** Le coefficient ne voit que le linéaire : si les fortes remises se concentrent sur les très petites lignes et sur les très grosses, la liaison existe et r reste voisin de zéro. Avant de conclure « aucun lien », on regarde le nuage, la médiane par strate, ou un test d'association sur catégories (M02.C08).

Faites-le dans le SQL. (les vues `lignes_ventes` et `paniers` ont été créées au C02, § « Pour la suite du module ») :

```
SELECT COUNT(*)                AS n,
       ROUND(COVAR_SAMP(quantite, montant_ttc), 0) AS covariance,
       ROUND(CORR(quantite, montant_ttc), 3)      AS pearson
FROM lignes_ventes;
```

Ne mélangez pas les colonnes dans un même contrôle : covariance sur `montant_ht`, corrélation sur `montant_ttc` produit deux chiffres justes séparément et une phrase fausse ensemble. Le réflexe : une requête, un couple de variables.

5.3 Rang contre valeur : Spearman

Définition. La **corrélation de rang de Spearman** — Spearman rank correlation, notée ρ — remplace chaque valeur par son rang dans la série triée, puis calcule la corrélation de Pearson sur ces rangs. Elle mesure si les deux variables montent ensemble, sans exiger que ce soit proportionnellement.

Sur `quantite` et `montant_ttc`, Spearman vaut 0,641 contre 0,712 en Pearson. Que le rang soit plus faible est une information : quelques lignes à très forte quantité produisent des écarts disproportionnés qui gonflent la liaison linéaire — la quantité maximale après correction est de 68 pièces quand la médiane est de 4, et l'énoncé non nettoyé montait jusqu'à 14 000. Concrètement :

Situation	Ce qu'on publie	Pourquoi
Variables continues, allure proche d'une droite, effectif confortable	Pearson	plus précis quand les hypothèses tiennent

Situation	Ce qu'on publie	Pourquoi
Valeurs extrêmes nombreuses, ou variable déjà ordonnée (note de satisfaction)	Spearman	insensible aux distances, robuste
Effectif faible (une trentaine de points)	les deux, et le nuage	vérifier que la conclusion ne tient pas à trois lignes
Colonne texte convertie en date ou en mois	aucune : re-typologiser d'abord	une date n'est pas une quantité

Règle de rédaction : **quand les deux coefficients se contredisent, c'est l'écart qui s'écrit** ; quand ils concordent, une mention « (Spearman : 0,641) » en note suffit, et elle clôt la discussion sur les valeurs extrêmes.

5.4 La droite de régression : trois nombres et une interdiction

Définition. La **régression linéaire simple** — simple linear regression, méthode des moindres carrés (ordinary least squares, OLS) — trace la droite qui minimise la somme des carrés des écarts verticaux. Elle livre la **pente** (variation moyenne de y par unité de x), l'**ordonnée à l'origine** (valeur prédite quand x vaut zéro) et le **R²** (part de la variance de y expliquée par x).

Sur les 480 lignes, la droite montant = a × quantité + b donne :

Élément	Valeur	Lecture professionnelle
Pente	12 237 FCFA par pièce	une pièce de plus sur une ligne s'accompagne, en moyenne, de 12 237 FCFA de plus
Ordonnée l'origine	à -10 402 FCFA	sans signification : une ligne de quantité nulle n'existe pas
R ²	50,7 %	la quantité explique la moitié de la dispersion du montant, pas plus
Pente standardisée	71,2 %	un écart-type de quantité en plus, c'est 0,71 écart-type de montant : c'est exactement r

Trois usages de ces nombres. D'abord, la **pente est un fait conditionnel** : elle vaut « toutes lignes confondues, sur l'année 2025, au grain ligne ». Elle ne vaut ni par produit, ni par catégorie : la pente d'un rayon de matériaux lourds n'a rien de commun avec celle de la quincaillerie, et la moyenne des pentes par catégorie n'est pas la pente globale.

Ensuite, le **contrôle du point moyen** : la droite des moindres carrés passe par le point (moyenne de x, moyenne de y). Appliquée à la quantité moyenne de 7 pièces, elle prédit 75 257 FCFA, pour une moyenne observée de 75 152 FCFA — 105 FCFA d'écart, dus à l'arrondi d'affichage. Si votre prédiction à la moyenne s'écarte nettement de la moyenne observée, votre droite est fautive : c'est le meilleur contrôle de régression qui soit, et il prend dix secondes.

Enfin, l'interdiction : **on n'extrapole pas**. Prédire le montant d'une ligne de 200 pièces avec une pente calibrée entre -13 et 68 pièces est un exercice de fiction — et c'est précisément le genre de chiffre qui, dans un dossier d'investissement, décrédibilise tout le reste.

Faites-le dans un tableur. =DROITEREG(y; x; VRAI) renvoie pente et ordonnée ; le R² s'obtient par =R2(y; x), ou par le carré de =COEFFICIENT.CORRELATION(). **Dans le SQL :** REGR_SLOPE(montant_ttc, quantite), REGR_INTERCEPT(montant_ttc, quantite), REGR_R2(montant_ttc, quantite) — trois agrégats, une requête, et le contrôle du point moyen se fait en comparant REGR_INTERCEPT + REGR_SLOPE * AVG(quantite) à AVG(montant_ttc).

Conseil professionnel. Présentez toujours la pente et le R^2 ensemble : une pente sans R^2 fait croire à une loi, un R^2 sans pente ne dit pas de combien. Nos deux chiffres dans une phrase : « une pièce de plus sur une ligne s'accompagne en moyenne de 12 237 FCFA de plus, mais cette seule variable ne fait que la moitié du travail (R^2 50,7 %) ».

5.5 Corrélation n'est pas causalité : les quatre configurations

Définition. On dit que x **cause** y si agir sur x change y . La corrélation ne teste pas cela : elle constate une variation conjointe. Quatre configurations produisent une corrélation sans lien causal : la cause commune, la causalité inverse, le hasard de l'échantillonnage, et la sélection du périmètre — ce que les données contiennent déjà avant toute analyse.

Appliquées au fichier :

Configuration	Comment elle apparaîtrait ici	Comment on la reconnaît
Cause commune	Grosses ventes et grosses remises viennent des mêmes périodes de chantier	croiser avec la date et la catégorie : si le lien disparaît à période contenue, c'était la saison
Causalité inverse	On croit que la remise gonfle le panier ; c'est le gros panier qui obtient une remise	la réponse est temporelle ou contractuelle, pas statistique : la remise se décide pendant la vente
Hasard d'échantillonnage	Deux colonnes sans rapport peuvent afficher $r = 0,10$ sur 480 lignes	le test de signification du chapitre M02.C08 ; à $n = 480$, la barre du bruit est vers 0,09
Sélection du périmètre	Sur le seul magasin 5, une liaison propre à la ville apparaît ou disparaît	reproduire sur un autre magasin, puis sur le fichier complet

Le plus utile à votre niveau est le troisième : la **barre de bruit**. Deux colonnes sans aucun rapport, sur 480 lignes, peuvent afficher une corrélation de 0,1. La vôtre, si elle est plus petite que ce seuil, ne mérite aucun commentaire. C'est ce qui rend les chiffres de l'extrait lisibles : à 0,712, la liaison quantité-montant n'a rien de comparable au bruit ; à 0,007, l'heure de saisie ne veut rien dire du tout.

5.6 Contrôler la variable de confusion — et le paradoxe de Simpson

Définition. Une **variable de confusion** (confounder) influence à la fois x et y et fait apparaître un lien qui n'existe pas entre elles. Contrôler un facteur de confusion consiste à comparer x et y **à l'intérieur de chaque strate** de cette variable, puis à regarder si la conclusion survit. Quand elle s'inverse strate par strate, on parle de **paradoxe de Simpson** (Simpson's paradox, effet de Simpson-Yule).

Testons la question la plus politique du module : les vendeurs vendent-ils plus gros parce qu'ils ont de meilleurs produits ?

Étape	Mesure	Valeur
1	Panier moyen le plus élevé des quatre vendeurs (Bationo)	103 367 FCFA
2	Panier moyen le plus faible (Ilboudo)	69 352 FCFA
3	Écart global entre ces extrêmes	34 014 FCFA
4	Part de Bois & panneaux — la catégorie la plus chère — dans leurs lignes	de 2,4 % (Bationo) à 8,5 % (Ouédraogo)
5	Écart maximal entre vendeurs, calculé catégorie par catégorie puis moyenné	57 346 FCFA

Étape	Mesure	Valeur
6	Dispersion de cet écart par catégorie	de 17 433 (Quincaillerie) à 173 167 FCFA (Electricité)

Le résultat contredit le schéma habituel : l'écart entre vendeurs **n'est pas réduit** par le contrôle de la catégorie — il augmente (57 346 contre 34 014 FCFA). Le mix produit n'explique donc pas la différence : il l'occultait. Le cas est même nettement inverse à l'intuition, puisque le vendeur au plus fort panier moyen est aussi celui qui a la plus faible part de la catégorie la plus chère (2,4 % de ses lignes).

Attention. Contrôler une variable multiplie les petites strates, et c'est là que la méthode se brise : Bois & panneaux compte 24 lignes pour vingt-deux vendeurs — certaines strates n'en contiennent que trois. Un écart calculé sur trois observations n'est pas un résultat, c'est un dessin. Fixez un effectif minimal par strate, appliquez-le à toutes les catégories sans exception, et écrivez lesquelles ont été écartées du calcul : c'est ce qui rend la démarche crédible quand la conclusion dérange.

Deux précautions, à écrire dans la note. D'abord, ce calcul n'a pas été mené sur Bois & panneaux : avec 24 lignes réparties sur vingt-deux vendeurs, certaines catégories n'ont que quelques lignes par vendeur, et un écart calculé dessus est du bruit. Ensuite, l'écart moyen par catégorie masque une dispersion énorme (17 433 à 173 167 FCFA) : la conclusion « le mix n'explique rien » est vraie en moyenne, pas dans chaque rayon.

C'est là l'usage professionnel du paradoxe de Simpson, bien plus utile que sa curiosité : **on ne l'invente pas, on le cherche**. La méthode tient en trois questions écrites dans la note : la catégorie explique-t-elle l'écart ? la période ? le type de client ? Si rien ne l'explique, l'écart devient un objet d'explication — et s'il disparaît, l'affirmation initiale était un artefact de découpage.

5.7 Agrégation : pourquoi les corrélations montent quand on résume

Un effet de bord que l'on découvre trop tard : plus on agrège, plus les points s'alignent. Sur les 480 lignes, `quantite` et `montant_ttc` corrélaient à 0,712 ; agrégées par client (372 clients distincts dans l'attendu), les totaux corrélaient généralement plus fort, parce que les irrégularités individuelles se moyennent ; et la corrélation peut s'effondrer si l'on agrège par jour, parce que l'effet de taille de journée écrase tout. Ce **problème d'agrégation** (modifiable areal unit problem dans sa version spatiale, effet de groupement dans sa version courante) a une conséquence pratique unique :

- **le grain s'écrit au même titre que le coefficient** : « $r = 0,712$, au niveau de la ligne (480 observations) » ;
- **on ne compare jamais deux corrélations calculées à des grains différents**, même sur le même fichier ;
- **on se méfie des corrélations spectaculaires sur séries mensuelles courtes** : douze points donnent fréquemment un r très élevé entre deux variables simplement croissantes — le lien est alors une tendance partagée, pas une dépendance.

5.8 Calculer les liaisons : quel outil, quelle formule

Ce que vous voulez	Tableur	SQL (DuckDB)	pandas
Corrélation linéaire	<code>=COEFFICIENT.CORRELATION(x;y)</code> (anglais CORREL)	<code>CORR(x, y)</code>	<code>df.x.corr(df.y)</code>
Covariance d'échantillon	<code>=COVARIANCE.S(x;y)</code>	<code>COVAR_SAMP(x, y)</code>	<code>df.x.cov(df.y)</code>

Ce que vous voulez	Tableur	SQL (DuckDB)	pandas
Corrélation de rangs	=CORREL() sur deux colonnes de =RANG()	tri puis CORR sur les rangs	df[["x","y"]].corr(method="spearman")
Pente, ordonnée, R²	=DROITEREG(), =R2()	REGR_SLOPE, REGR_INTERCEPT, REGR_R2	scipy.stats.linregress
Matrice de corrélations	table de formules recopiées	une requête par couple	df.corr(numeric_only=True)
Nuage de points	graphique nuage	échantillon puis export	df.plot.scatter("quantite", "montant_ttc")

Boîte à outils. Contrôle minimal de toute corrélation publiée : (1) l'effectif (480 lignes, 316 paniers ou 372 clients) et le grain, dans la légende ; (2) la liste des colonnes autorisées, issue du tableau de typage M01.C05, pour ne pas corréler un identifiant ; (3) le nuage regardé au moins une fois — une corrélation de 0,7 produite par quatre points isolés est une erreur de tracé ; (4) Pearson et Spearman côte à côte sur les séries à forte asymétrie, comme ici ; (5) le carré de r publié avec r, pour que le lecteur sache ce qui reste inexpliqué.

6. Exemple concret : « la remise fait-elle vendre plus gros ? »

La demande. Le directeur veut relever la remise accordée sur les tickets de plus de 100 000 FCFA : « sur nos gros chantiers, la remise fait la différence ».

Ce que le fichier dit, ligne par ligne.

Question	Calcul	Réponse mesurée
Les lignes remisées sont-elles plus grosses ?	CORR(remise, montant_ttc) sur 480 lignes	-0,031 : aucune liaison linéaire
La remise est-elle la règle ou l'exception ?	lignes sans remise	307 sur 480, soit 64,0 % sans aucune remise
Le vendeur remet-il à sa guise ?	test d'association vendeur × remise	3,03 de chi-deux, p = 0,388 : pas d'association détectée (M02.C08)
La catégorie explique-t-elle les montants ?	CV par catégorie	de 81,4 % (Peinture) à 170,4 % (Electricité) — M02.C03

La conclusion, et sa limite. Sur cet extrait, aucune liaison linéaire mesurable entre remise et taille de ligne : relever la remise sur les gros tickets n'a, dans ces données, pas de contrepartie observée en volume. La limite est à écrire explicitement : 173 lignes seulement portent une remise (480 moins 307), réparties sur six taux (0 ; 0,03 ; 0,04 ; 0,07 ; 0,08 ; 0,11) — un plan d'expérience quasi inexistant. On ne prouve pas l'efficacité d'une politique de prix avec des remises accordées au cas par cas. La recommandation professionnelle n'est donc pas « n'augmentez pas la remise » mais « testez : deux mois, deux groupes de clients, un taux, une mesure ».

La phrase à écrire dans la note. « La liaison entre taux de remise et montant de ligne est nulle sur l'extrait (r = -0,031, n = 480 lignes). Cette absence ne prouve pas l'inefficacité de la remise : elle signale que le fichier ne contient pas d'expérience de prix exploitable. Nous recommandons un test borné plutôt qu'un ajustement unilatéral de la politique. »

7. Démonstration pas à pas : du nuage au coefficient, puis au contrôle

Étape 1 — Choisir le couple. `quantite` et `montant_ttc`, au niveau de la ligne, sur le fichier attendu (480 lignes). Notez la décision : deux colonnes, un grain.

Étape 2 — Le nuage d'abord. 480 points, échelles linéaires. Ce que l'on doit y voir : un alignement montant, un point très à droite et très en haut (la plus grosse ligne), quelques points sous zéro (les retours). Si le nuage dessine un éventail — dispersion croissante avec x —, la relation est hétéroscédastique et la pente moyenne masque deux régimes : à écrire dans la note.

Étape 3 — La covariance, puis r . `COVAR_SAMP` puis `CORR`, en SQL. Vérifiez que le passage de l'un à l'autre se retrouve : 857 634 divisé par le produit des écarts-types (8,4 et 143 915 FCFA) nous ramène à 0,71, ce qui est la valeur de r aux arrondis de présentation près.

Étape 4 — Spearman. Même couple, méthode des rangs : 0,641. Notez la direction de l'écart — le rang est plus faible —, signe qu'une poignée de valeurs extrêmes pèse davantage sur le linéaire que sur l'ordre.

Étape 5 — La droite. `REGR_SLOPE` = 12 237, `REGR_INTERCEPT` = -10 402, `REGR_R2` = 0,507. Vérifiez le carré de r : $0,712 \times 0,712 = 0,507$ ✓ — en régression simple, R^2 est exactement le carré du coefficient de corrélation.

Étape 6 — Le contrôle par strate. Refaites la pente catégorie par catégorie. Si les pentes s'écartent fortement, la droite globale est une moyenne d'histoires différentes : la mention « toutes catégories confondues » devient obligatoire.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	r dans l'intervalle $[-1 ; +1]$	mathématique	0,712	colonnes de types mélangés
2	R^2 égal au carré de r (régression simple)	au centième	50,7 %	régression multiple ou pondérée
3	Droite passant par le point moyen	écart < 0,2 % de la moyenne	75 257 contre 75 152 FCFA	pente ou ordonnée mal lues
4	Un seul grain, une seule population	effet identique	480 lignes	jointure ou agrégation oubliée
5	Signe de la covariance = signe de r	identité	les deux positifs	sélection de colonnes erronée

8. Erreurs fréquentes

- Commenter r sans avoir vu le nuage.** Un r de 0,7 peut être produit par un point isolé et quelques dizaines d'autres en rond : le coefficient est exact, l'histoire est fausse. Le nuage se regarde avant, et se conserve en annexe.
- Corréler des colonnes mal typées.** Dans l'énoncé, `montant_ttc` est stocké en texte sur 486 des 489 lignes (M01.C04) : lu comme une catégorie par certains outils, comme une suite de chiffres par d'autres, le coefficient dépend alors du logiciel et non des données.
- Publier la covariance.** Sans unité lisible, 857 634 ne veut rien dire. Elle sert au calcul, pas à la note.
- Confondre force et certitude.** Un r de 0,10 sur 40 000 lignes est statistiquement établi et commercialement inutile ; un r de 0,7 sur 12 lignes est commercialement séduisant et statistiquement fragile. Le premier nombre mesure la taille de la liaison, le second sa solidité — et c'est le chapitre M02.C08 qui fournit le second.

5. **Oublier que la pente dépend du périmètre.** 12 237 FCFA par pièce est une moyenne annuelle, toutes catégories. Sur Bois & panneaux seuls, elle change : citer un chiffre global comme s'il s'appliquait à une gamme est la manière la plus rapide de perdre un auditeur.
6. **Inventer une causalité à partir d'un calendrier.** « Les ventes augmentent avec la remise en novembre » : les deux montent, mais la saison des chantiers monte aussi. Une corrélation temporelle partagée n'est pas une liaison causale — c'est la définition même de la cause commune.

9. Bonnes pratiques professionnelles

1. **Une phrase, trois éléments** : le coefficient, l'effectif, le grain. « $r = 0,712$ (480 lignes, niveau ligne, fichier nettoyé) ».
2. **Joindre le nuage.** Trois cents points coûtent une ligne et désarment la moitié des contestations.
3. **Publier R^2 en pourcentage.** « La quantité explique 50,7 % de la variation du montant » : le lecteur comprend qu'il reste 49,3 % à expliquer — c'est exactement le propos.
4. **Nommer la variable de confusion contrôlée,** et dire quand elle ne change rien : ici l'écart entre vendeurs passe de 34 014 à 57 346 FCFA quand on contrôle la catégorie.
5. **Traiter les corrélations proches de zéro comme des résultats,** pas comme des échecs : -0,031 répond à une question de politique commerciale, à condition d'ajouter ce que la donnée ne permet pas de conclure.
6. **Réserver le mot « effet » aux expériences,** et employer « liaison », « lien », « co-variation » pour tout le reste. Cette discipline de vocabulaire est la meilleure ligne de défense de l'analyste.

10. Exercice guidé — la liaison quantité / montant, entièrement refaite

Énoncé. Sur le fichier attendu : nuage, covariance, Pearson, Spearman, droite de régression, R^2 , puis commentaire en trois phrases pour un directeur commercial.

Vous faites — barème 10 points, total 10.

Étape	Ce que vous produisez	Points
A	Le nuage, axes nommés et effectif indiqué	2
B	Covariance et r , avec le calcul de r à partir de la covariance	2,5
C	Spearman, et une phrase sur l'écart entre les deux chiffres	1,5
D	Pente, ordonnée, R^2 , avec le contrôle du point moyen	2,5
E	Commentaire en trois phrases, sans verbe de causalité	1,5

Nous vérifions. Covariance 857 634 · $r = 0,712$ · Spearman 0,641 · pente 12 237 FCFA par pièce · ordonnée -10 402 FCFA · R^2 50,7 %. Contrôle du point moyen : prédire à la quantité moyenne doit rendre la moyenne des montants — 75 257 contre 75 152 FCFA, soit l'arrondi d'affichage.

Nous corrigeons. Deux formulations à supprimer : « la quantité détermine le montant » (trop fort : la détermination est comptable, pas comportementale) et « plus on vend de pièces, plus on gagne » (trop faible : cela vaut pour tout commerce, et le coefficient n'apporte rien). La phrase juste contient le chiffre, le périmètre et la part inexpliquée.

11. Exercices autonomes

Exercice 6.1 (★) — Le tri des colonnes. Listez les colonnes du fichier avec lesquelles il est licite de calculer une corrélation avec `montant_ttc`, et celles où c'est interdit, en une ligne de justification par colonne.

Exercice 6.2 (★) — Le carré. Calculez le carré de 0,712 et retrouvez le R^2 mesuré. Que reste-t-il inexpliqué, et que signifierait un R^2 de 95 % dans ce contexte ?

Exercice 6.3 (★★) — La pente par catégorie. Estimez la pente de `montant = a × quantité` pour Matériaux (143 lignes) et pour Consommables (57 lignes). Comparez à la pente globale de 12 237 FCFA et concluez sur son usage.

Exercice 6.4 (★★) — Le contrôle de confusion. Reprenez l'écart de panier moyen entre les quatre vendeurs en contrôlant cette fois le **trimestre** au lieu de la catégorie. L'écart global est de 34 014 FCFA : dites, chiffres à l'appui, si la période explique une part notable de cet écart.

Exercice 6.5 (★★★) — Note de liaison. Six lignes pour la direction sur le couple remise / montant : le coefficient, l'effectif, le grain, ce que le résultat ne prouve pas, et la proposition d'expérience à mener. (attendu : $r = -0,031$; 480 lignes ; 64,0 % de lignes sans remise ; six taux observés ; un test sur deux groupes)

12. Correction détaillée

Exercice 6.1. Licites : les colonnes numériques à rapport — `quantite`, `prix_unitaire_ht`, `remise`, `montant_ht`, `montant_ttc` — et les composantes extraites de la date (jour du mois, heure), à condition de signaler qu'une heure est une grandeur circulaire. Illicites : `n_ticket` et `client` (identifiants : leur ordre ne signifie rien), `categorie` et `vendeur` (nominales : on les traite par agrégats ou par tableaux de fréquences), `date brute` (une date n'est pas une grandeur). La justification attendue nomme le type de chaque colonne, pas une impression.

Exercice 6.2. $0,712 \times 0,712 = 0,507$, soit 50,7 % : le R^2 mesuré, contrôle réussi. Restent 49,3 % de variance inexpliquée — prix unitaire, catégorie, remise, et tout ce que le fichier ne contient pas. Un R^2 de 95 % devrait éveiller le soupçon de corrélation mécanique : deux colonnes qui se déduisent l'une de l'autre, comme `montant_ht` et `montant_ttc` ($r = 1,000$ ici).

Exercice 6.3. Les pentes par catégorie s'écartent fortement — de l'ordre du simple au double entre matériaux lourds et consommables — et certaines catégories voient la quantité expliquer à peine le montant. Conclusion attendue : la pente globale est un compromis sans correspondant réel ; elle résume, elle ne prévoit pas une ligne d'une catégorie donnée. Sont valorisés le rappel des effectifs (143 et 57 lignes) comme facteur de fiabilité inégale, et la mention de la pente en unité (FCFA par pièce).

Exercice 6.4. Ce qui est attendu, c'est la méthode plus que le résultat : recalculer l'écart maximal entre vendeurs à trimestre contenu, le comparer aux 34 014 FCFA globaux et aux 57 346 FCFA obtenus à catégorie contenue, et écrire la conclusion en fonction du sens observé. Un écart qui survit à deux contrôles successifs cesse d'être un artefact de découpage et devient un objet d'explication — à faire discuter par l'entretien, le planning, ou le portefeuille client.

Exercice 6.5. Note attendue : la liaison entre taux de remise et montant de ligne est nulle ($r = -0,031$, 480 lignes, fichier nettoyé) ; 307 lignes sur 480 ne portent aucune remise et les six taux observés ne forment pas un plan d'expérience exploitable ; nous ne pouvons donc conclure ni à l'efficacité ni à l'inefficacité de la remise, seulement à l'absence de donnée pour la juger ; nous proposons deux mois de test sur deux groupes de clients, avec un taux unique par groupe ; la décision se prendra sur l'écart de panier entre les deux groupes, testé comme au chapitre M02.C08.

13. Mini-projet M02.P3 — « La page des liaisons » (50 min)

Commande. Pour la direction commerciale, produisez une page de trois liaisons chiffrées, choisies parmi : quantité / montant, prix / montant, remise / montant, vendeur / catégorie, heure / montant. Pour chacune : le coefficient, le grain, l'effectif, le carré du coefficient, et une phrase de conclusion — avec l'obligation d'écrire explicitement ce que la liaison ne dit pas.

Livrables numérotés. (1) le tableau des trois liaisons ; (2) les trois nuages en annexe ; (3) le contrôle de confusion mené sur la liaison que vous jugez la plus forte, avec ses deux chiffres (avant / après contrôle) ; (4) la proposition d'expérience pour la liaison politiquement la plus lourde ; (5) la ligne de journal (date, fichiers, effectifs, versions).

Barème (20 points, seuil 13). Coefficients exacts et sourcés (4) · grain et effectif écrits partout (3) · carré du coefficient commenté (2) · nuages joints et lisibles (3) · contrôle de confusion correctement exécuté (4) · expérience proposée mesurable (2) · journal (2) — total 20. Ce livrable rejoint le document du M02.P2 : le projet du module est la somme des pages, pas leur remplacement.

14. Résumé du chapitre

1. La covariance donne le signe et rien d'autre (857 634 pièces-FCFA) ; la corrélation la standardise (0,712), et seule la seconde se communique.
2. Pearson voit le linéaire, Spearman le monotone : 0,712 et 0,641 sur la même paire, et l'écart entre les deux est un diagnostic sur les valeurs extrêmes.
3. Une droite de régression se réduit à trois nombres — 12 237 FCFA par pièce, -10 402 FCFA, 50,7 % — et le contrôle du point moyen (75 257 contre 75 152 FCFA) vérifie le calcul en dix secondes.
4. Une corrélation ne dit rien de la causalité : cause commune, inversion, hasard, périmètre. Le contrôle de confusion est une méthode, pas une formule — et sur cet extrait il agrandit l'écart entre vendeurs (57 346 contre 34 014 FCFA) au lieu de le résorber.

15. À retenir

À retenir.

- Un coefficient publié porte son effectif, son grain et son carré : 0,712 (480 lignes, niveau ligne), 50,7 % de variance expliquée.
- Une ordonnée à l'origine hors domaine (-10 402 FCFA) et une prédiction hors des valeurs observées s'appellent des extrapolations : elles ne sortent pas du bureau.
- Corrélation mécanique, agrégation, confusion : trois façons de trouver un lien qui n'explique rien. On les teste, on ne les suppose pas.
- Le mot « effet » est réservé aux expériences. Tout le reste est une liaison.

À retenir. La formulation qui protège. « Sur 480 lignes du fichier nettoyé, quantité et montant varient ensemble à hauteur de $r = 0,712$ (Spearman 0,641), soit 50,7 % de la variance expliquée : l'autre moitié tient au prix, à la catégorie et à ce que nous ne mesurons pas. Rien dans ce calcul n'établit qu'augmenter les quantités augmentera la recette. »

16. Évaluation formative (auto-correction, 12 min)

1. Pourquoi la covariance ne se publie-t-elle pas, alors qu'on la calcule ? **2.** Sur le couple quantité / montant, Spearman est plus faible que Pearson. Que dit cet écart ? **3.** Une corrélation de -0,031 entre remise et montant : quelle conclusion licite, quelle conclusion interdite ? **4.** Le R^2 vaut 50,7 %. Que répondez-vous à qui demande « donc la quantité ne compte que pour moitié » ? **5.** Un directeur affirme : « nos vendeurs les plus rentables ont les meilleurs produits, c'est la catégorie qui l'explique ». Quelle vérification faire, et quel a été son résultat sur l'extrait ?

Corrigé

1. Parce qu'elle conserve l'unité des deux colonnes (pièces $\times$ FCFA) : 857 634 ne se lit pas, ne se compare pas, et change si l'on change de monnaie ou de grain. 2. Que quelques valeurs extrêmes pèsent davantage sur la liaison linéaire que sur la liaison de rangs ; la liaison est réelle, mais en partie portée par les plus grosses lignes. 3. Licite : sur cet extrait, aucune liaison *linéaire* entre remise et taille de ligne, et le fichier ne contient pas de plan de prix exploitable. Interdite : « la remise ne sert à rien » — l'absence de liaison observée n'est pas la preuve d'une absence d'effet. 4. Que la moitié de la *dispersion* du montant est associée à la quantité, l'autre au prix, à la catégorie, à la remise et au non-mesuré ; et que 50,7 % est un chiffre de variance, pas un pourcentage de recette. 5. Calculer l'écart entre vendeurs à catégorie contenue et le comparer à l'écart global : ici il ne se réduit pas, il passe de 34 014 à 57 346 FCFA en moyenne par catégorie — la catégorie n'explique donc pas l'écart, elle le masque.

Auto-validation. Reprenez les cinq contrôles du §7 : r dans l'intervalle, R^2 égal au carré de r , point moyen retrouvé, un seul grain, signe de la covariance conforme. Puis recopiez vos cinq nombres en regard de la table de référence : 857 634 $\cdot$ 0,712 $\cdot$ 0,641 $\cdot$ 12 237 $\cdot$ 50,7 %. Sur un seul écart, reprenez la jointure avant la formule.

M02.C07 — Hasard et incertitude : probabilités de base, échantillonnage, biais

L'idée du chapitre. Vous n'avez jamais les données : vous en avez un extrait. Trois idées rendent cet extrait défendable — ce que le hasard ajoute (l'erreur d'échantillonnage), ce que le hasard ne corrige pas (le biais), et la façon de le dire (l'intervalle). Sans elles, un chiffre juste reste une opinion.

Base de travail — obligatoire. Le fichier nettoyé du projet M01, `donnees/reference/ventes_magasin5_2025_ATTENDU.csv` (480 lignes × 13 variables, 316 tickets), et la population dont il est issu : le magasin 5 sur 2025, soit 7 676 lignes et 4 524 tickets, lus dans `01_socle_donnees/data/reference/ventes_propres.csv`. Séparateur point-virgule, UTF-8, virgule décimale, montants en franc CFA (FCFA). Valeurs mesurées dans `01_socle_donnees/data/reference/chiffres_cites.md`, section M02.

1. Objectifs du chapitre

À la fin de ce chapitre, vous êtes capable de :

1. distinguer population, échantillon, unité sondée et statistique, et nommer ces quatre objets dans un dossier réel ;
2. expliquer pourquoi deux tirages de même taille ne donnent pas le même chiffre, et chiffrer cet écart par l'erreur type ;
3. calculer une marge d'erreur et un intervalle de confiance à 95 % pour une moyenne et pour une proportion ;
4. séparer l'**erreur d'échantillonnage**, qui se travaille, du **biais**, qui se lit dans la règle d'extraction ;
5. diagnostiquer le biais de début de mois de notre extrait, le chiffrer, et dire quelle statistique il dégrade ;
6. fixer une taille d'échantillon à partir d'une précision visée, et expliquer pourquoi le premier doublement de précision coûte quatre fois plus cher.

2. Pourquoi cette notion est importante

Un analyste qui ne parle pas d'incertitude livre un nombre sans mode d'emploi, et le décideur en fait ce qu'il veut. Trois accidents sortent de là :

- **La sur-interprétation.** « Le panier moyen est de 114 156 FCFA, nous perdons du terrain » : rien, dans ce chiffre, ne dit si l'écart avec 116 886 FCFA est une baisse ou le bruit normal d'un extrait de 316 tickets.
- **La généralisation abusive.** Un export tronqué, un `LIMIT`, un seul magasin, un seul mois : le rapport dit « l'année », l'auditoire comprend « tout le monde, tout le temps ».
- **Le faux sentiment de sécurité du volume.** 480 lignes suffisent pour une moyenne à ± 20 078 FCFA près ; 240 000 lignes ne suffisent pas si elles viennent d'un périmètre mal choisi. La taille ne rachète jamais la sélection.

Dans les faits. Dans les services études, le nombre de lignes n'est presque jamais le vrai problème : l'extraction obéit à des contraintes techniques (la caisse n'exporte que les derniers jours, l'entrepôt ne garde que treize mois). Le travail de l'analyste est de **reconstituer la règle d'extraction** — qui a été exclu, et comment — avant toute statistique. Ce chapitre vous le fait faire sur votre propre fichier, parce qu'il est plus simple de commencer par un défaut que l'on peut compter.

3. Explication simple

Vous goûtez une marmite de sauce avec une cuillère. Trois choses se passent.

1. **La cuillère n'est pas la marmite** : ce que vous goûtez dépend de ce qu'elle a attrapé. C'est l'**erreur d'échantillonnage**, le hasard de la trempée. Elle baisse si la louche est plus grande.
2. **Si vous plongez toujours en surface, vous trouverez du gras** tous les jours, à toutes les heures, avec toutes les louches du monde. C'est le **biais** : la méthode de trempée, pas la chance.
3. **Ce que vous pouvez annoncer honnêtement** n'est pas « la sauce est salée » mais « salée, et si je regoûte dix fois je resterai dans ce goût-là ». Cela s'appelle un **intervalle de confiance**.

Notre extrait est une cuillère trempée en surface, et nous allons le prouver : il contient les 40 premières lignes de chaque mois — donc seulement les jours 1, 2, 3 et 4 — et représente 6,3 % des lignes du magasin sur l'année. Sa moyenne ne dérape que de 2,3 %, mais sa médiane de 14,3 %. Même cuillère, deux statistiques, deux dégradations : c'est tout le chapitre.

4. Vocabulaire essentiel

#	Terme	Français — English — sens simple	À savoir
1	Population	population — population — l'ensemble qu'on veut décrire	ici 4 524 tickets, magasin 5 en 2025
2	Unité sondée	unité statistique — sampling unit — ce qu'on tire un par un	le ticket ou la ligne : le choix change tout
3	Échantillon	échantillon — sample — la partie réellement observée	480 lignes, 316 tickets
4	Statistique	statistique — statistic — chiffre calculé sur l'échantillon	114 156 FCFA est une statistique
5	Paramètre	paramètre — parameter — le même chiffre sur la population	116 886 FCFA est le paramètre visé
6	Tirage aléatoire simple	tirage aléatoire simple — simple random sampling — toute combinaison équiprobable	notre extrait ne remplit pas cette condition
7	Erreur d'échantillonnage	erreur d'échantillonnage — sampling error — dispersion d'un tirage à l'autre	elle baisse en $1/\sqrt{n}$
8	Biais	biais d'échantillonnage — sampling bias — décalage systématique	il ne baisse pas avec n
9	Erreur type	erreur type — standard error — écart type de la statistique	10 244 FCFA pour notre moyenne
10	Marge d'erreur	marge d'erreur — margin of error — 1,96 fois l'erreur type	20 078 FCFA ici
11	Intervalle confiance	de intervalle de confiance — confidence interval — plage compatible avec l'observation	[94 078 ; 134 234] FCFA

#	Terme	Français — English — sens simple	À savoir
12	Niveau de confiance	niveau de confiance — confidence level — taux de réussite de la méthode	95 % est l'usage, pas une probabilité sur votre cas
13	Bootstrap	amorçage bootstrap — bootstrap — remesurer la statistique sur des ré-échantillonnages du fichier	notre outil pour la médiane
14	Graine	graine aléatoire — random seed — numéro qui rend un tirage rejouable	20260917 dans tous nos exemples

5. Cours approfondi

5.1 Hasard, fréquence, probabilité : trois règles suffisent

Définition. La **probabilité** d'un événement est la fréquence vers laquelle se stabilise cette fréquence quand on répète l'expérience beaucoup de fois. Elle ne décrit jamais un cas particulier, seulement une régularité d'ensemble.

Sur les 316 tickets de l'extrait, tirons au sort un ticket puis un second :

Règle	Énoncé	Application mesurée
Complément	$P(\text{non } A) = 1 - P(A)$	136 paniers sur 316 sont sous 50 000 FCFA, soit 43,0 % ; donc 57,0 % sont au-dessus
Indépendance	$P(A \text{ et } B) = P(A) \times P(B)$ si un tirage ne change rien à l'autre	deux petits paniers d'affilée : $0,43 \times 0,43 \approx 18,5$ %
Addition	$P(A \text{ ou } B) = P(A) + P(B)$ si les deux s'excluent	31 paniers au-dessus de 250 000 FCFA et 3 négatifs : 9,8 % plus 0,9 %

Une nuance, souvent sanctionnée : le second tirage **n'est pas indépendant** du premier si l'on tire sans remise dans un petit ensemble. L'effet est négligeable ici (316 éléments), matériel quand on tire 30 valeurs sur 40 — et c'est la raison exacte de la correction du §5.2.

5.2 Ce que le hasard ajoute : la distribution d'échantillonnage

Définition. L'**erreur d'échantillonnage** (sampling error) est la dispersion d'une statistique si l'on refaisait le tirage. Ce n'est pas une faute, c'est le prix d'une lecture partielle ; elle se mesure par l'**erreur type**, écart type de la statistique d'un tirage à l'autre.

Mesurons-la au lieu de la postuler : dans les 316 paniers, 5 000 tirages de 30 tickets sans remise (graine 20260917, donc rejouable), et la moyenne de chaque tirage.

```
import numpy as np, pandas as pd
p = pd.read_csv("01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv", sep=";")
pan = p.groupby("n_ticket").montant_ttc.sum().to_numpy(dtype=float) # 316 paniers
n = len(pan)
rng = np.random.default_rng(20260917)
tirages = np.argsort(rng.random((5000, n)), axis=1)[: , :30] # 5000 tirages de 30,
sans remise
moy = pan[tirages].mean(axis=1)
med = np.median(pan[tirages], axis=1)
print(round(moy.mean()), round(moy.std(ddof=1)), np.percentile(moy, [2.5, 97.5]))
```

Résultats mesurés : la moyenne des 5 000 moyennes vaut 114 889 FCFA, c'est-à-dire la moyenne de l'extrait (114 156 FCFA) à la fluctuation près — **le tirage ne trompe pas en moyenne, il éparpille**. L'écart type de cette distribution est de 31 363 FCFA, et 95 % des tirages tombent

entre 63 898 et 182 706 FCFA : avec 30 tickets, votre chiffre peut varier du simple au triple sans que personne ait failli.

Ce que donne le hasard : 5 000 tirages de 30 tickets dans l'extrait (graine 20260917)

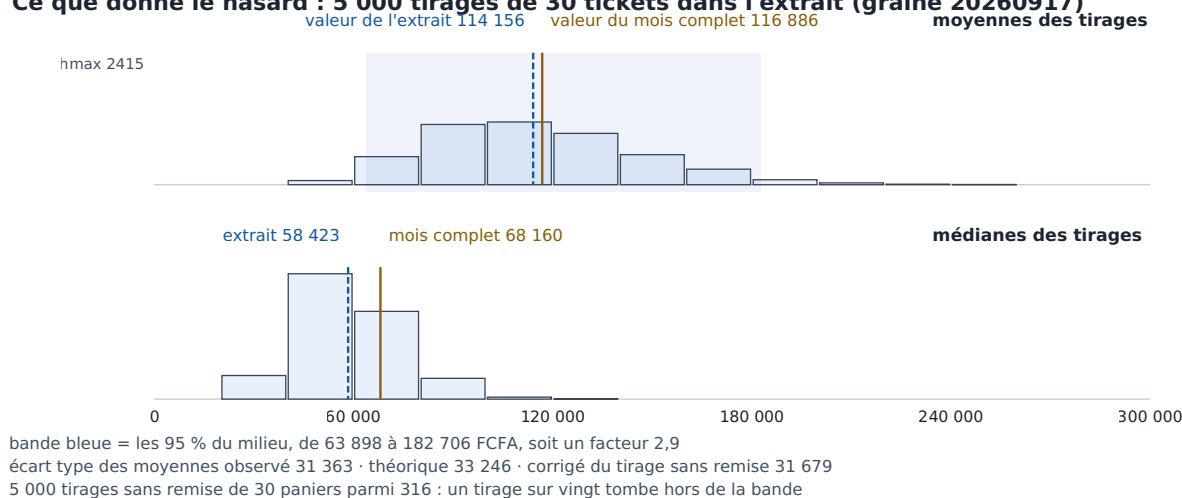


Figure — Distribution d'échantillonnage de la moyenne et de la médiane des paniers (n = 30), calculée sur le fichier nettoyé du projet M

La théorie dit la même chose sans simuler : l'écart type d'une moyenne est celui des valeurs divisé par la racine de l'effectif. Nos 316 paniers ont un écart type de 182 095,1 FCFA, d'où une erreur type de 10 244 FCFA — 10 243 si l'on part de l'écart type arrondi à 182 095, différence sans effet sur la marge. Pour des tirages de 30, la formule donne 33 246 FCFA quand la simulation trouve 31 363 FCFA : l'écart vient du tirage **sans remise** dans une population finie, qui réduit la dispersion d'un facteur 0,953. La formule corrigée tombe à 31 679 FCFA — il reste 1 %, et c'est le hasard des 5 000 tirages.

Attention. Deux écarts types se ressemblent dans les notes et n'ont rien à voir : 182 095 FCFA décrit la dispersion **des clients entre eux**, 10 244 FCFA décrit la dispersion **de votre moyenne d'un tirage à l'autre**. Le second est dix-huit fois plus petit, et lui seul fixe la précision de ce que vous annoncez. Confondre les deux revient à écrire « nos clients sont très variés, donc votre moyenne est fausse ».

Même expérience avec la **médiane** : écart type de 14 387 FCFA pour une moyenne des tirages de 59 140 FCFA, soit 24,3 % de variation relative contre 27,3 % pour la moyenne. Contre une idée reçue, la médiane n'est donc pas plus instable ici ; elle est simplement insensible au plus gros ticket (1 343 666 FCFA), ce que la moyenne ne peut pas faire. En échange, sa distribution n'est pas lisse — une médiane de 30 valeurs est toujours l'une des valeurs observées — et l'intervalle [32 006 ; 90 577] FCFA est plus irrégulier que celui des moyennes.

5.3 Ce que le hasard ne pardonne pas : le biais

Définition. Un **bias d'échantillonnage** (sampling bias) est un écart systématique entre la statistique attendue et la valeur de population, produit par la **méthode de sélection**. Augmenter l'échantillon ne le fait pas disparaître : il le rend plus précis autour de la mauvaise valeur.

Ouvrons la fabrication du fichier. Dans le socle de la formation (01_socle_donnees/scripts/generation_socle.py), les lignes du magasin 5 pour 2025 sont triées par date, puis on garde **les 40 premières lignes de chaque mois** — règle choisie pour couvrir douze mois au lieu d'un. Raisonnable pour un cours, elle n'est pas aléatoire, et voici ce qu'elle coûte.

Ce que dit l'extrait	Extrait	Population magasin 5 / 2025	Écart
Lignes	480	7 676	6,3 % de la population
Tickets	316	4 524	7,0 % de la population
Jours du mois observés	1, 2, 3, 4 (31 dates, 2 à 4 par mois)	1 à 31	fin de mois jamais vue
Panier moyen	114 156 FCFA	116 886 FCFA	-2,3 %
Médiane des paniers	58 423 FCFA	68 160 FCFA	-14,3 %

Le diagnostic est net parce que nous avons la population sous la main : dans le magasin sur l'année, les tickets des quatre premiers jours du mois ont une médiane de 58 424 FCFA, les autres 70 267 FCFA, et ces jours-là ne pèsent que 14,9 % des tickets. Notre extrait est fait **uniquement** de ces jours : sa médiane tombe à 58 423 FCFA, soit à un franc près la médiane du début de mois. Le biais n'a rien de mystérieux — il est entièrement attribué à la composition en jours. La moyenne, elle, ne dérape que de 2,3 % parce que les gros tickets existent aussi les premiers jours.

Définition. Un **bias de couverture** (coverage bias) est le cas où la liste de départ ne contient pas toute la population. C'est notre cas : la liste était « les 40 premières lignes de chaque mois », ce qui exclut structurellement trois quarts du calendrier.

Deux réflexes, pour la vie : **demandeur la règle, pas le volume** (« selon quels critères ces lignes ont-elles été retenues ? » — la réponse est dans le code d'extraction ou dans le `WHERE` de la vue) et **chercher la variable cachée** (ici le jour du mois, absente des 13 colonnes du fichier : un biais se lit rarement dans les données, il se lit dans le mode d'emploi des données). Notez aussi ce qui n'est **pas** biaisé : les heures couvrent toute la journée, de 07:01 à 18:57, parce que le tri ne portait que sur la date. Un biais est toujours localisé sur un axe.

5.4 Cinq familles de biais, et leur trace dans notre fichier

Famille	Mécanisme	Dans notre dossier	Ce qu'on peut chiffrer
Couverture	la liste de départ est incomplète	extrait réduit aux quatre premiers jours	médiane -14,3 % (\$5.3)
Sélection	le tri favorise un profil	tri par date avant écrêtage à 40	2 à 4 dates retenues par mois, 29 lignes au plus sur une date, 13 en médiane
Non-réponse	des individus ne sont pas renseignés	18 tickets sans identifiant client, 103 lignes sans client exploitable	21,5 % des lignes
Survivance	on n'observe que ce qui a survécu	489 lignes reçues pour 480 utiles : les doublons ne sont là que parce qu'ils ont été ré-émis	9 lignes de doublon
Mesure	l'instrument déforme	7 quantités à 14 000 unités, 65 dates écrites à la main en JJ/MM/AAAA	M01.C04 et M01.C06

Cinq mécanismes agissent sur 489 lignes, aucun ne provoque d'erreur logicielle — et tous sont comptables : on peut les isoler et les annuler un par un, ce que vous avez fait au projet M01. Trois nuances à retenir. Le **bias du survivant** : mesurer un assortiment sur les produits encore en rayon, ou la rentabilité sur les magasins encore ouverts, conclut que « tout va bien ». Le **bias de Berkson** : une ligne n'existe que si elle a été encaissée — chantier payé à trente jours, livraison sans bon saisi, avoir émis le mois suivant disparaissent avant le premier calcul, sans qu'une valeur erronée soit présente. Le **bias d'accord** (réponse sociale) ne menace pas les don-

nées d'encaissement mais les questionnaires : sur les premières on se bat contre la couverture, sur les seconds contre la volonté.

Conseil professionnel. Écrivez la liste des exclusions dans le rapport, pas dans votre tête : « magasin 5, année 2025, lignes des quatre premiers jours de chaque mois, retours inclus, avoirs de l'année précédente exclus ». Cette demi-page ne prouve pas que vous aviez raison ; elle prouve que vous saviez — et c'est exactement ce que vaut un analyste le jour où un chiffre est contesté.

5.5 L'intervalle de confiance : ce que « 95 % » veut dire

Définition. Un **intervalle de confiance à 95 %** est fabriqué par une méthode qui, appliquée à une infinité de tirages, enfermerait la vraie valeur dans 95 % des cas. Le pourcentage qualifie la **méthode**, pas votre intervalle : une fois calculé, la valeur est dedans ou elle n'y est pas.

C'est la phrase la plus mal dite de la statistique d'entreprise. À ne pas écrire : « il y a 95 % de chances que le vrai panier moyen soit dans notre intervalle ». À écrire : « notre méthode manque la cible une fois sur vingt ; sur ce tirage elle donne 114 156 ± 20 078 FCFA ».

Pour une moyenne à grand effectif : estimation ± 1,96 fois l'erreur type. Estimation 114 156 FCFA, erreur type 10 244 FCFA, marge 20 078 FCFA, intervalle [94 078 ; 134 234] FCFA — 40 156 FCFA de largeur, soit environ un tiers de la valeur annoncée. Voilà le mot juste que le décideur doit entendre : avec 316 tickets, on connaît le panier moyen **à un tiers près**, pas au centime près.

Et le contrôle que presque personne ne fait, possible ici parce que la population est connue : le vrai panier annuel vaut 116 886 FCFA, il est dans l'intervalle. On a vérifié la couverture de la méthode sur ce cas, ce qui change le statut de la note : on peut écrire « notre intervalle contient la valeur mesurée sur 4 524 tickets ».

Pour une proportion, l'erreur type vaut $\sqrt{p(1-p)/n}$. La part de tickets contenant plus d'une ligne : 104 sur 316, soit 32,9 %, erreur type 2,6 points, intervalle [27,7 ; 38,1] %. Cette écriture résiste en réunion : elle ne dit pas « un tiers des tickets », elle dit « entre un quart et deux cinquièmes », et personne n'en déduit une politique commerciale.

Attention. Trois abus fréquents dans les tableaux de bord. (1) Appliquer l'intervalle à un extrait non aléatoire : la formule suppose un tirage au sort, et le nôtre n'en est pas un — son intervalle ne couvre que la variabilité, jamais le biais. (2) Conclure à l'équivalence parce que deux intervalles se chevauchent : c'est le test du chapitre M02.C08 qui tranche. (3) Faire varier le niveau de confiance après coup pour rendre un résultat intéressant : fixez 95 % dans la note de cadrage, avant d'ouvrir le fichier.

5.6 Le bootstrap : remesurer sans retourner sur le terrain

Définition. Le **bootstrap** (bootstrap) traite l'échantillon comme une population : on tire 2 000 fois, **avec remise**, un échantillon de même taille, on recalcule la statistique, et les 2,5^e et 97,5^e percentiles de ces 2 000 résultats servent d'intervalle.

La formule du §5.5 n'existe que pour la moyenne ; le bootstrap s'applique à toute statistique que l'on sait recalculer. Mesures prises sur nos 316 paniers, graine 20260917 :

Statistique		Intervalle par formule	Intervalle par bootstrap	Lecture
Moyenne paniers	des	[94 078 ; 134 234]	[95 397 ; 133 574]	les deux se recoupent : profil peu déformé
Médiane paniers	des	inexistante	[52 318 ; 64 632]	largeur 12 314 FCFA : mieux connue que la moyenne
Minimum paniers	des	inapplicable	[-150 804 ; 797]	artefact, ligne suivante

Le plus petit panier vaut -150 804 FCFA : un ticket sur 316. Dans un ré-échantillonnage, sa probabilité d'être absent est forte, et le minimum saute alors à la valeur suivante, 797 FCFA. Résultat : 64,3 % des 2 000 tirages reproduisent le minimum exact, et l'intervalle obtenu ne mesure pas l'incertitude mais la discontinuité. Règle métier : **on ne bootstrap pas les valeurs extrêmes**, ni rien qui tienne à quelques lignes — seuils de retours, plus gros clients, aberrants de M02.C02-C03 ; ceux-là se comptent.

```
rng = np.random.default_rng(20260917)
rejes = rng.choice(pan, size=(2000, n), replace=True)
print(np.percentile(np.median(rejes, axis=1), [2.5, 97.5])) # [52318. 64632.] avec cette graine
```

Conseil professionnel. Publiez un intervalle bootstrap avec trois précisions : nombre de tirages, graine, version de la bibliothèque. « [52 318 ; 64 632] FCFA, 2 000 tirages, graine 20260917 » se vérifie en trente secondes ; « environ $\pm 6\,000$ FCFA » ne se vérifie pas du tout.

5.7 Combien de tickets faut-il mesurer ?

On retourne le problème : au lieu de subir la précision, on la commande.

Objectif	Calcul	n nécessaire	Avec nos 316 tickets
Proportion à ± 5 points, pire cas	$(1,96 / 0,05)^2 \times 0,25$	385	insuffisant
Proportion à ± 5 points, $p \approx 33\%$	$(1,96 / 0,05)^2 \times p(1 - p)$	340	insuffisant
Proportion à ± 3 points, pire cas	$(1,96 / 0,03)^2 \times 0,25$	1 068	loin du compte
Moyenne de panier à $\pm 10\%$ relatif	$(1,96 \times \sigma / (0,1 \times \text{moyenne}))^2$	978	insuffisant

Trois enseignements de gestion, plus utiles que les chiffres :

1. **Rendements décroissants.** Diviser la marge par deux demande quatre fois plus de données, la diviser par dix en demande cent. Passer de $\pm 20\,078$ à la moitié sur la moyenne équivaut à quadrupler l'effectif — mais un mois complet de magasin en contient 4 524 : on peut le payer sans changer d'outil. La question devient « vaut-il la peine de lire toute l'année pour un chiffre ? », jamais « faut-il un meilleur logiciel ? ».
2. **Le grain est un plan d'échantillonnage.** Tirer 480 lignes ne vaut pas tirer 480 tickets : à 1,52 ligne par ticket, un tirage de lignes sur-représente les paniers à plusieurs lignes, donc les gros. Objet client → tirez des tickets ; objet produit → tirez des lignes ; et écrivez lequel.
3. **Le volume ne dispense pas du plan.** 240 000 lignes reconduisent exactement le même biais de couverture si elles viennent des seuls jours 1 à 4. À l'inverse, un plan stratifié bien né — 30 tickets par trimestre et par vendeur — informe mieux qu'un million de lignes prises à la tête du client.

Définition. L'échantillonnage stratifié (stratified sampling) découpe la population en cases (jour du mois, catégorie, vendeur, trimestre), tire au sort dans chaque case, puis combine les résultats pondérés par le poids réel des cases. C'est le remède direct au §5.3 : il garantit que début et fin de mois sont observés.

5.8 Corriger après coup, et choisir l'outil

Quand re-tyrager est impossible, la technique de secours est la **post-stratification** (pondération) : on garde l'échantillon, mais on redonne à chaque strate son poids réel. Testons-le : sur le montant moyen par ligne, l'extrait annonce 75 152 FCFA pour 68 889 FCFA en population, soit +9,1 % ; en repesant la structure par catégorie de produits aux poids du mois complet, on tombe à 78 060 FCFA, soit +13,3 %. **La pondération a aggravé l'erreur** — non par défaut de méthode, mais par sa limite structurale : on ne corrige que l'axe sur lequel on équilibre, et le défaut d'ici vit dans le calendrier. À écrire noir sur blanc dans un rapport.

Question	Tableur	SQL (DuckDB)	pandas / Python
Tirer k lignes au hasard	<code>=ALEA.ENTRE.BORNES(1;480) + =INDEX()</code>	<code>USING SAMPLE 30 ROWS</code>	<code>df.sample(30, random_state=20260917)</code>
Tirer par strate	filtre puis <code>=ARRALE()</code> sur la sous-plage	<code>ROW_NUMBER() OVER (PARTITION BY mois ORDER BY random())</code>	<code>g.sample(n=30, random_state=...)</code>
Erreur type d'une moyenne	<code>=ECARTYPE.STANDARD(plage)/RACINE(NB(plage))</code>	<code>STDDEV_SAMP(m)/ SQRT(COUNT(*))</code>	<code>s.std(ddof=1) / len(s)**0.5</code>
Marge 95 %	<code>=INTERVALLE.CONFIANCE.NORMAL(0,05;écart_type;effectif)</code>	multiplication par 1,96 (pas de fonction dédiée)	<code>scipy.stats.norm.ppf(0.975)</code>
Intervalle d'une proportion	<code>=1,96*RACINE(p*(1-p)/n)</code>	idem en SQL	<code>proportion_confint (statsmodels)</code>
Bootstrap	déconseillé : pas de boucle rejouable sans macro	export Python	puis <code>rng.choice(valeurs, size=(2000, n), replace=True)</code>

Vigilance d'outillage, parce qu'elle casse les rapprochements entre collègues : les fonctions d'intervalle ne se nomment pas pareil selon le logiciel, et ne rendent pas un intervalle. Dans Excel, `INTERVALLE.CONFIANCE.NORMAL` et `INTERVALLE.CONFIANCE.STUDENT` rendent la **demi-largeur**, à ajouter et retrancher de la moyenne ; dans LibreOffice, la famille s'appelle `INTERVALLE.CONFIANCE.NORMAL` et `INTERVALLE.CONFIANCE.T` (la version héritée `INTERVALLE.CONFIANCE` reste acceptée pour compatibilité). Un facteur 2 d'écart entre deux notes vient plus souvent de ce détail que des données.

Boîte à outils. Le minimum à écrire sous tout chiffre issu d'un extrait : (1) la **population** visée et sa source (tickets du magasin 5, 2025, 4 524) ; (2) l'**unité sondée** (le ticket, pas la ligne) ; (3) l'**effectif** et le taux de représentation (316, soit 7,0 %) ; (4) la **règle d'extraction** reconstituée, avec son biais chiffré quand il est mesurable (jours 1 à 4 ; médiane -14,3 %) ; (5) l'**intervalle** avec sa méthode (formule ou bootstrap, nombre de tirages, graine). Cinq lignes de pied de page, et elles valent plus que le graphique.

6. Exemple concret : « le panier moyen du magasin 5, à quoi bon le discuter ? »

La demande. Le directeur régional prépare un arbitrage d'assortiment et veut un chiffre unique. Un collaborateur a répondu 114 156 FCFA en ouvrant le fichier d'extraction, sans autre forme de procès.

Ce que le dossier permet vraiment de dire.

On savait	On établit en lisant la règle d'extraction	On établit avec l'intervalle
114 156 FCFA	l'extrait ne couvre que les jours 1 à 4 : 6,3 % des lignes du magasin sur l'année	le vrai panier annuel, sur 4 524 tickets, est 116 886 FCFA : -2,3 %, dans l'intervalle [94 078 ; 134 234]
58 423 FCFA de médiane	la médiane du début de mois, dans la population, est 58 424 FCFA	la médiane annuelle est 68 160 FCFA : -14,3 %, et ce n'est pas du bruit

Ce que le chiffre change à la décision. Sur la moyenne, personne n'aurait changé d'avis : 2,3 % dans un intervalle large d'un tiers ne déplace aucune décision. Sur la médiane, si : conclure « la moitié de nos clients est sous 58 000 FCFA » oriente vers un assortiment d'entrée de gamme, alors que la moitié réelle est sous 68 160 FCFA — 9 700 FCFA d'écart par ticket, multipliés par 4 524 tickets, cela devient une question d'assortiment et non d'arrondi. Le travail n'a pas été de calculer plus finement, mais de replacer le chiffre dans le calendrier dont il venait.

La phrase à écrire. « Panier moyen estimé sur l'extrait (316 tickets, jours 1 à 4 de chaque mois) : 114 156 FCFA, intervalle à 95 % [94 078 ; 134 234]. Valeur annuelle complète : 116 886 FCFA, soit un écart de 2,3 % — l'extrait est usable pour la moyenne. La médiane reproduit celle du début de mois et sous-estime de 14,3 % la médiane annuelle : à recalculer sur le mois complet. »

7. Démonstration pas à pas : de la règle d'extraction à l'intervalle

Étape 1 — Reconstituer la règle. Ouvrez le script d'extraction, ou faites-vous le donner. Ici : tri par date puis 40 premières lignes par mois. Recopiez la règle en une phrase dans le carnet de données, avec la date de la vérification.

Étape 2 — Aligner les périmètres. Population au même grain : 7 676 lignes, 4 524 tickets. Extrait : 480 et 316. Le taux de sondage (6,3 % en lignes, 7,0 % en tickets) n'est pas encore un jugement : il dit qu'une lecture exhaustive est possible.

Étape 3 — Mesurer l'écart, statistique par statistique. Moyenne -2,3 %, médiane -14,3 %. On n'écrit pas « l'extrait est biaisé » : on dit quelle statistique est touchée et de combien.

Étape 4 — Attribuer le biais. Médiane des jours 1 à 4 dans la population : 58 424 FCFA ; jours suivants : 70 267 FCFA. L'extrait, à un franc près, est le premier groupe. Le biais est attribué à un axe, pas à une ambiance.

Étape 5 — Séparer le biais de la variabilité. Simulation de 5 000 tirages de 30 : écart type 31 363 FCFA, bande des 95 % du milieu de 63 898 à 182 706 FCFA. À 316, erreur type 10 244 FCFA et marge 20 078 FCFA. Ce que la taille arrange, et ce qu'elle n'arrange pas, sont maintenant deux chiffres distincts.

Étape 6 — Choisir le remède. Élargir la lecture (mois complet, coût nul, 4 524 tickets) avant de bricoler des poids : la pondération par catégorie, elle, remonte l'écart de +9,1 % à +13,3 %.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Effectif de l'extrait conforme à la règle annoncée	identité	480 lignes pour 40 × 12	règle devinée, pas lue
2	Population et échantillon au même grain	même unité	316 tickets contre 4 524	agrégation de lignes oubliée
3	Écart type simulé et formule	écart < 5 %	31 363 contre 31 679 FCFA	tirage avec / sans remise confondu
4	L'intervalle contient la valeur de population	oui ou non écrit	contient 116 886 FCFA	formule appliquée hors conditions
5	Bootstrap employé hors valeurs extrêmes	interdit sur min et max	minimum exclu, médiane retenue	artefact pris pour incertitude
6	Chaque pourcentage rapporté à son effectif	systématique	32,9 % (104 de 316)	proportion sans dénominateur

8. Erreurs fréquentes

1. **Confondre imprécision et biais.** L'intervalle mesure la fluctuation du tirage ; il ne dit rien de la façon dont on a tiré. Notre extrait a une marge d'erreur honnête et un mauvais calendrier.
2. **Se rassurer avec le volume.** 480, 7 676 ou 240 000 lignes : si la règle de sélection est la même, l'écart systématique est le même. La précision augmente, la justesse non.
3. **Annoncer un pourcentage sans dénominateur.** « 21,5 % des lignes n'ont pas de client exploitable » se défend parce qu'on sait que c'est sur 480 ; sans le dénominateur, la phrase devient une rumeur.
4. **Appliquer les formules d'intervalle à un échantillon de convenance** sans le dire : on peut publier l'intervalle, à condition d'écrire qu'il ne couvre que la variabilité d'échantillonnage.
5. **Bootstraper un minimum, un maximum ou un seuil rare** : ces statistiques ne dépendent que de quelques lignes, et le ré-échantillonnage les fait sauter (64,3 % de tirages identiques sur notre minimum).
6. **Multiplier les sous-populations jusqu'à l'effectif ridicule.** Un intervalle sur 4 tickets d'une strate est un objet mathématique sans contenu : publiez l'effectif de la strate à côté de sa marge, et signalez toute strate de moins d'une dizaine d'observations comme non exploitable.

9. Bonnes pratiques professionnelles

1. **Une ligne de méthodologie par chiffre** : population, unité sondée, effectif, règle d'extraction, intervalle.
2. **La règle d'extraction vit dans le dépôt**, pas dans un e-mail : un en-tête de requête (-- extrait : 40 premières lignes par mois, 2025, magasin 5) survit à son auteur.
3. **Toujours fixer la graine.** Un résultat non rejouable n'est pas un résultat, c'est une anecdote.
4. **Publier la marge avant la conclusion.** Le lecteur décide d'agir ou non ; c'est son rôle, et vous venez de lui donner de quoi.
5. **Fixer le seuil d'exploitation par strate avant de découper** (« toute strate sous 12 observations est agrégée ») : sinon le premier tableau de bord segmenté conclura sur deux tickets.
6. **Faire valider le taux de sondage par le métier.** 7,0 % des tickets du magasin sur l'année, est-ce assez pour un arbitrage d'assortiment ? Le responsable de rayon le sait mieux que vous : son critère n'est pas la précision statistique mais le coût d'une décision ratée.

10. Exercice guidé — mesurer l'incertitude d'un panier moyen

Énoncé. Sur le fichier attendu, estimez le panier moyen, son intervalle à 95 %, et l'écart entre cet intervalle et la valeur annuelle réelle du magasin. Concluez en une phrase pour un directeur.

Vous faites — barème 10 points, total 10.

Étape	Ce que vous produisez	Points
A	Population, unité sondée et effectifs écrits avant tout calcul	2
B	Erreur type et marge d'erreur, formule énoncée	2,5
C	Intervalle à 95 % comparé à la valeur annuelle	2,5
D	Contrôle par bootstrap, avec nombre de tirages et graine	1,5
E	Phrase de conclusion séparant variabilité et biais	1,5

Nous vérifions. 316 tickets pour 4 524 (7,0 %) · erreur type 10 244 FCFA · marge 20 078 FCFA · intervalle [94 078 ; 134 234] FCFA · valeur annuelle 116 886 FCFA, donc couverte · bootstrap 2 000 tirages, graine 20260917, intervalle [95 397 ; 133 574] FCFA.

Nous corrigeons. Trois défauts reviennent : l'intervalle présenté comme « la fourchette des paniers » (il s'agit de la fourchette de la **moyenne** ; celle des clients va de -150 804 à 1 343 666 FCFA) ; le contrôle de couverture oublié ; et la conclusion « nos clients dépensent 114 156 FCFA », qui oublie que 71,8 % des tickets sont sous cette moyenne (M02.C02). La phrase juste porte l'estimation, sa plage, et ce que la plage autorise.

11. Exercices autonomes

Exercice 7.1 (★) — Les quatre objets. Pour votre extrait, nommez la population visée, l'unité sondée, l'échantillon et la statistique, en une ligne chacun avec les effectifs mesurés.

Exercice 7.2 (★) — Deux tirages. Une chance sur deux de tomber sur un panier sous 50 000 FCFA ? Calculez la probabilité pour un puis deux tirages, et dites pourquoi le tirage sans remise rend le calcul légèrement faux.

Exercice 7.3 (★★) — La marge en pratique. Intervalle à 95 % de la part des lignes comportant une remise non nulle (173 lignes sur 480), puis effectif nécessaire pour ramener la marge à ± 3 points.

Exercice 7.4 (★★) — Trancher un désaccord. Un collègue annonce 68 000 FCFA de médiane sur le mois complet contre vos 58 423 FCFA sur l'extrait. Produisez les trois nombres qui permettent de dire qui mesure quoi, et lequel est utilisable pour décrire le magasin.

Exercice 7.5 (★★★) — Le plan. Vous devez livrer un chiffre sur les paniers du trimestre en cours, à ± 10 % près, et la base ne permet d'exporter que 1 200 lignes. Proposez le plan (unité sondée, strates, tirage, contrôles), et dites le risque que le plan ne corrige pas. (attendu : tirer des tickets, strates par mois et par vendeur, contrôle de l'écart type simulé contre la formule, risque résiduel de couverture sur retours et avoirs)

12. Correction détaillée

Exercice 7.1. Population : les 4 524 tickets du magasin 5 en 2025 (ou, à l'échelle du socle, les 240 000 lignes propres des quatre années). Unité sondée : le ticket — 480 lignes pour 316 tickets, 1,52 ligne par ticket. Échantillon : 6,3 % des lignes, 7,0 % des tickets. Statistique : 114

156 FCFA — et non un paramètre, dont la correspondante est 116 886 FCFA. C'est cette distinction qui rend la suite lisible.

Exercice 7.2. 136 tickets sur 316 sous 50 000 FCFA : 43,0 %. Deux tirages supposés indépendants : environ 18,5 %. En tirant sans remise, le second tirage ne porte plus que 135 candidats favorables sur 315, soit un écart de l'ordre de 0,1 point — négligeable ici, mais c'est exactement ce qui rend la dispersion simulée du §5.2 un peu plus petite que la formule (facteur 0,953).

Exercice 7.3. 173 lignes sur 480 : 36,0 %. Erreur type 2,2 points, marge 4,3 points, intervalle [31,7 ; 40,3] %. Pour une marge de ± 3 points : 1 068 lignes au pire cas, 984 lignes à $p \approx 36$ %, soit un peu plus du double de l'effectif disponible pour gagner 1,3 point. C'est la loi du $\sqrt{n}$, et c'est l'objet de la négociation avec le métier.

Exercice 7.4. Les trois nombres : médiane de l'extrait 58 423 FCFA ; médiane annuelle du magasin 68 160 FCFA ; médiane des tickets annuels datés des jours 1 à 4, 58 424 FCFA. La troisième tranche : l'extrait n'est pas faux, il est partiel, et il mesure précisément le début de mois. Seul le deuxième chiffre décrit le magasin entier. On n'a pas « raison » : on a mesuré deux objets différents.

Exercice 7.5. Unité sondée : le ticket — 1 200 lignes en représentent moins (environ 790 au rythme de 1,52 ligne par ticket). Strates : trois mois $\times$ quatre vendeurs, soit 12 cases et environ 66 tickets chacune, effectif à signaler car aucune strate ne descend alors très bas. Tirage : `ROW_NUMBER() OVER (PARTITION BY mois, vendeur ORDER BY random())` avec graine, ou `df.groupby(["mois", "vendeur"]).sample(n=..., random_state=20260917)`. Contrôles : écarts types simulé et théorique, présence de toutes les strates, marge publiée avec l'effectif. Risque résiduel : le tirage n'opère que sur la **liste** produite par la caisse ; si les retours et avoirs en sont absents (8 lignes de retour, -283 933 FCFA dans l'extrait, M02.C02), aucun plan d'échantillonnage ne les fera réapparaître — c'est encore la couverture qu'il faut interroger.

13. Mini-projet M02.P4 — « La page d'incertitude » (45 min, premier volet ; M02.C08 enchaîne)

Commande. À partir du fichier attendu et de la population du magasin sur 2025, produisez une page qui répond à : « que peut-on affirmer, et avec quelle marge, sur le panier moyen et sur la part des tickets avec remise ? ». Le volet décision (comparaisons de vendeurs, tests) est ajouté au chapitre suivant : M02.P4 est rendu une seule fois, pour les deux chapitres.

Livrables numérotés. (1) bloc méthodologie : population, unité sondée, effectifs, taux de sondage, règle d'extraction ; (2) pour chacune des deux statistiques : estimation, erreur type, marge, intervalle, méthode ; (3) un contrôle de couverture, la même statistique calculée sur le mois complet avec l'écart en pourcentage ; (4) dix lignes qui disent ce que l'intervalle autorise, ce qu'il interdit, et ce qu'on irait chercher si on avait le droit de tout lire ; (5) ligne de journal (fichiers, effectifs, nombre de tirages, graine, versions).

Barème (20 points, seuil 13). Méthodologie complète (4) · estimations justes et sourcées (4) · intervalles corrects, méthode explicitée (4) · contrôle de couverture chiffré et commenté (3) · interprétation sans abus de langage (3) · journal (2) — total 20. Le second volet, en M02.C08, apportera ses 20 points ; la note du projet est la moyenne des deux volets.

14. Résumé du chapitre

1. Le hasard éparpille, la méthode décale : erreur type (10 244 FCFA sur notre moyenne) et biais (-14,3 % sur notre médiane) se mesurent séparément et ne se corrigent pas l'un par l'autre.

2. La précision obéit à $\sqrt{n}$: 978 tickets pour ± 10 % relatifs, 1 068 lignes pour ± 3 points, contre 316 disponibles.
3. Un intervalle qualifie une méthode, pas un cas : [94 078 ; 134 234] FCFA se rate une fois sur vingt, et nous avons vérifié qu'il contient la valeur annuelle de 116 886 FCFA.
4. Le bootstrap intervalle toute statistique recalculable — la médiane y gagne ([52 318 ; 64 632] FCFA), les valeurs extrêmes y perdent toute signification.
5. Une pondération ne corrige que l'axe qu'elle équilibre : repeser les catégories a monté l'écart de +9,1 % à +13,3 %, parce que le défaut était dans le calendrier.

15. À retenir

À retenir.

- Avant de calculer : population, unité sondée, effectif, règle d'extraction. Ces quatre lignes précèdent le premier chiffre, dans une note comme dans une requête.
- Erreur d'échantillonnage : $1,96 \text{ fois } \sigma / \sqrt{n}$. Biais : ce que cette formule ne voit pas. Les deux se publient, séparément.
- Un intervalle ne couvre jamais le biais ; un gros volume ne rachète jamais une mauvaise liste. 480 lignes bien tirées battent 240 000 lignes mal choisies.
- Toute simulation se déclare : 5 000 tirages, 2 000 tirages, graine 20260917. Sans graine, pas de résultat.

À retenir. La formulation qui tient debout. « Sur 316 tickets extraits des quatre premiers jours de chaque mois (6,3 % des lignes du magasin en 2025), le panier moyen est de 114 156 FCFA, intervalle à 95 % [94 078 ; 134 234]. La valeur annuelle complète est 116 886 FCFA : notre intervalle la contient. La médiane de l'extrait reproduit celle du début de mois (58 423 contre 58 424 FCFA) et sous-estime de 14,3 % la médiane annuelle : elle ne décrit pas le magasin. »

16. Évaluation formative (auto-correction, 12 min)

1. Quelle différence de nature entre erreur d'échantillonnage et biais, et pourquoi le second ne se réduit-il pas en augmentant l'échantillon ? 2. Sur l'extrait, la moyenne est à 2,3 % de la valeur annuelle et la médiane à 14,3 %. Que dit cet écart de comportement sur la fabrication du fichier ? 3. Un intervalle à 95 % calculé sur un échantillon de convenance : que couvre-t-il, que ne couvre-t-il pas ? 4. Pourquoi ne bootstrap-t-on pas un minimum, et que publie-t-on à la place ? 5. On veut diviser la marge par deux : de combien multiplie-t-on l'effectif, et que répondez-vous au directeur qui propose « de prendre un mois de plus » ?

Corrigé

1. L'erreur d'échantillonnage est la fluctuation d'un tirage à l'autre autour de la cible, et elle décroît en $1/\sqrt{n}$. Le biais déplace la moyenne des tirages : augmenter l'effectif resserre l'intervalle ****autour de la valeur biaisée****, donc rend plus confiant et plus faux à la fois. 2. Que l'extrait sur-représente un profil de jours, et que les positions centrales y sont sensibles : le début de mois a de plus petits paniers (58 424 contre 70 267 FCFA de médiane), la moyenne l'est peu car elle est tirée vers le haut par les gros tickets, présents à toutes les dates. D'où la règle : tester chaque statistique, pas le fichier en bloc. 3. Il couvre la variabilité du tirage, en supposant un tirage aléatoire. Ni le biais de sélection, ni les erreurs de mesure, ni le fait que la population visée n'était pas la bonne. 4. Parce qu'un minimum ne dépend que d'une ou deux observations : 64,3 % de nos ré-échantillonnages reproduisent le même ticket extrême, les autres sautent à la valeur suivante (797 FCFA), et l'intervalle [-150 804 ; 797] FCFA n'a aucune lecture honnête. À la place, on compte des effectifs : 8 lignes de retour, 25 tickets au-dessus du seuil de Tukey, 3 paniers négatifs, chacun avec sa marge de proportion. 5. Par quatre : de 316 tickets il faudrait dé-

passer le millier et demi, quand 978 suffiraient pour ± 10 % relatifs. « Prendre un mois de plus » ajoute de la précision mais reconduit le même biais si le mois suivant est lui aussi tronqué aux quatre premiers jours — la bonne demande est « le mois entier, ou un tirage au sort dans le mois ».

M02.C08 — Décider avec incertitude : intervalle de confiance, tests, interprétation et limites

L'idée du chapitre. Un écart observé est-il un écart réel ? Le test statistique répond à cette question précise — et à aucune autre. Ce chapitre donne la mécanique (hypothèses, statistique de test, valeur p, décision), les trois tests qui couvrent 90 % des demandes d'un service études, et surtout les quatre façons dont on les fait mentir.

Base de travail — obligatoire. Le fichier nettoyé `donnees/reference/ventes_magasin5_2025_ATTENDU.csv` (480 lignes × 13 variables, 316 tickets, 4 vendeurs) et, pour les comparaisons, la population du même magasin sur 2025 (7 676 lignes, 4 524 tickets) dans `01_socle_donnees/data/reference/ventes_propres.csv`. Séparateur point-virgule, UTF-8, virgule décimale, montants en FCFA. Valeurs mesurées dans `01_socle_donnees/data/reference/chiffres_cites.md`, section M02.

1. Objectifs du chapitre

À la fin de ce chapitre, vous êtes capable de :

1. formuler une hypothèse nulle et une hypothèse alternative lisibles, et choisir le test d'après la nature de la variable et le grain de l'observation ;
2. conduire un test de comparaison de moyennes (t de Welch), un test de comparaison de rangs (Mann-Whitney) et un test d'indépendance sur tableau de fréquences (χ^2), avec leurs conditions d'emploi ;
3. lire une valeur p sans lui faire dire « la probabilité que H_0 soit vraie », et publier l'intervalle avec le test ;
4. distinguer « non significatif » de « égal », et chiffrer ce qu'une étude n'est **pas** en état de détecter ;
5. calculer la puissance d'un comparatif et l'effectif nécessaire, et déclarer à temps une étude sous-calibrée ;
6. repérer les quatre abus qui ruinent un travail sinon — comparaisons multiples, seuil de 5 % traité en loi, confusion entre significatif et important, test appliqué à un grain ou à un échantillon mal formé.

2. Pourquoi cette notion est importante

Un comparatif sans test est une loterie commentée. Trois dérives professionnelles, dans l'ordre de fréquence :

- **La prime au plus chanceux.** Quatre vendeurs, six comparaisons deux à deux : même si aucun ne se distingue, la probabilité qu'au moins un test dépasse le seuil de 5 % par pur hasard est d'environ 26,5 % (1 moins 0,95 élevé à la puissance six). Récompenser « le meilleur » sur cette base, c'est récompenser un tirage.
- **Le « ça n'a pas bougé ».** Sur 125 et 109 lignes, un écart réel de 20 000 FCFA par ligne ne serait vu que dans 18,5 % des cas : conclure à l'absence de différence, c'est confondre « non mesuré » et « nul ».

- **Le rapport qui noie le décideur.** Vingt pourcentages sans intervalle ne permettent ni d'agir ni de contester. Le test trace la discussion : la question est écrite avant, la réponse est une plage après.

Dans les faits. En entreprise, la quasi-totalité des demandes évaluables tient dans trois formats : comparer deux moyennes (ce vendeur, ce rayon, ce mois), comparer une part à une cible (21,5 % de lignes sans client, est-ce anormal ?), et tester une association (les remises dépendent-elles du vendeur ?). Les chapitres de référence enseignent souvent vingt tests ; ici on en conduit trois, à fond, sur des données que vous pouvez vérifier — c'est ce qui rend le jugement transférable.

3. Explication simple

Vous comparez deux balances de marché. Bationo affiche un panier plus élevé qu'Ilboudo. Avant de crier à la performance, deux questions :

1. **La balance est-elle fidèle ?** Avec 125 et 109 lignes, l'écart entre deux moyennes peut légitimement varier de $\pm 36\,888$ FCFA d'un extrait à l'autre (l'erreur type de la différence vaut 18 821 FCFA, et la marge à 95 % fait 1,96 fois ce montant). L'écart observé est de 28 609 FCFA : il rentre dans le bruit.
2. **Quelle question pose-t-on ?** « L'écart est-il exactement zéro ? » est une question fausse — la réponse est toujours non, deux groupes ne sont jamais parfaitement égaux. La question juste est : « un écart de cette taille est-il compatible avec l'absence de différence ? ». Le test répond à celle-là.

Le **p** est donc un verdict de compatibilité, pas une probabilité de vérité : « si aucun vendeur ne se ressemblait moins que par hasard, on verrait un écart au moins aussi grand dans 13 cas sur 100 ». À partir de là, on décide — et on écrit la plage, pas seulement la décision.

4. Vocabulaire essentiel

#	Terme	Français — English — sens simple	À savoir
1	Hypothèse nulle	hypothèse nulle — null hypothesis, H0 — l'écart s'explique par le seul hasard	c'est elle qu'on teste, jamais H1
2	Hypothèse alternative	hypothèse alternative — alternative hypothesis, H1 — l'option concurrente	à deux côtés par défaut (un écart dans un sens ou l'autre)
3	Statistique de test	statistique de test — test statistic — l'écart mesuré, divisé par sa variabilité normale	$t = 1,52$ ici
4	Valeur p	valeur p — p-value — probabilité d'observer un écart au moins aussi grand si H0 était vraie	pas $P(H0 \text{ vraie})$
5	Seuil alpha	seuil de signification — significance level, α — le risque qu'on accepte de prendre à tort	0,05 par usage, à fixer avant
6	Significatif	significatif — statistically significant — non compatible avec le seul hasard au seuil choisi	ne veut pas dire « important »
7	Erreur de premier type	erreur de premier type — type I error — conclure à une différence inexistante	risque α
8	Erreur de second type	erreur de second type — type II error — manquer une différence réelle	risque β ; la puissance vaut $1 - \beta$
9	Puissance	puissance statistique — statistical power — probabilité de détecter un écart donné	0,80 est la convention
10	Degrés de liberté	degrés de liberté — degrees of freedom — l'information restante après estimation	226,1 en Welch sur nos deux groupes

#	Terme	Français — English — sens simple	À savoir
11	Test de Welch	test t de Welch — Welch's t-test — comparaison de moyennes, variance libre	à préférer au t de Student classique
12	Test de Mann-Whitney	test de Mann-Whitney — Mann-Whitney U test — comparaison des rangs des deux groupes	robuste aux queues épaisses, n'éprouve pas la moyenne
13	Test du chi-deux	test du khi deux d'indépendance — chi-squared test of independence — fréquences observées contre attendues	exige des effectifs théoriques suffisants
14	Effectif théorique	effectif attendu — expected count — ce que donnerait l'indépendance parfaite	minimum 39,3 ici, franchement au-dessus de 5
15	Correction de Bonferroni	correction de Bonferroni — Bonferroni correction — seuil divisé par le nombre de tests	0,05 sur six tests devient 0,0083

5. Cours approfondi

5.1 La charpente d'un test, en cinq mouvements

Définition. Un **test statistique** est une procédure qui compare l'écart observé à ce que le hasard produit habituellement. Il énonce H_0 (« aucun vendeur ne diffère »), calcule une **statistique de test** (l'écart rapporté à sa variabilité normale), en déduit une **valeur p** (combien de fois le hasard fait au moins aussi bien), et conclut au seuil α fixé d'avance.

Sur notre couple vedette, les montants par ligne de Bationo et d'Ilboudo, les cinq mouvements donnent :

Mouvement	Valeur sur le fichier
H_0 : différence de moyennes nulle	écart à tester
Écart observé	28 609 FCFA (moyennes 90 963 contre 62 354 FCFA)
Erreur type de la différence	18 821 FCFA
Statistique t (Welch), ddl 226,1	1,52 — sous H_0 , la valeur seuil à 5 % est 1,97
Valeur p	0,13 : le hasard fait au moins aussi bien dans 13 % des cas
Décision au seuil de 5 %	on ne rejette pas H_0 — et on publie l'intervalle [-8 279 ; 65 498] FCFA

Quatre lectures exactes de la valeur p, et quatre fausses :

Juste	Faux
P(observer un écart $\geq 28\,609$ FCFA si H_0 est vraie)	P(H_0 est vraie)
Une mesure de compatibilité avec « aucun écart »	Une mesure de la taille de l'écart
Dépend de l'effectif : à écart égal, plus n grossit, plus p baisse	Elle serait indépendante du plan de collecte
Elle dépend du plan de collecte, pas seulement des données	« $p = 0,06$: donc pas de différence »

5.2 Choisir le test d'après la question et le grain

Question	Test	Ce qu'il éprouve	Conditions	Chez nous
Deux moyennes différent-elles ?	t de Welch	égalité des moyennes	indépendance, effectifs suffisants ($n > 30$ par groupe : 125 et 109 ✓)	1,52 ; $p = 0,13$
Un profil entier est-il décalé ?	Mann-Whitney	égalité des distributions (via les rangs)	indépendance ; aucune hypothèse de normalité	$p = 0,0161$

Question	Test	Ce qu'il éprouve	Conditions	Chez nous
Deux parts différentes ?	z sur deux proportions	égalité des proportions	des au moins 5 succès et 5 échecs par groupe ✓	$z = 1,21$; $p = 0,225$
Une association existe-t-elle entre deux catégories ?	χ^2 d'indépendance	indépendance des deux variables	effectifs théoriques ≥ 5 pour la plupart des cases	3,03 ; ddl 3 ; $p = 0,388$
Une moyenne tient-elle une cible ?	t sur un échantillon	moyenne = valeur cible	idem t	$t = 1,38$; $p = 0,168$ contre 100 000 FCFA
Un écart est-il nul ?	test d'équivalence, pas un test de différence	écart borné	bornes écrites avant le test	à faire, voir §5.4

Conseil professionnel. Écrivez la question, le test et le seuil **avant** d'ouvrir le fichier, dans le ticket de travail. Un test choisi après avoir vu les données n'est plus un test : c'est une justification. Cette règle unique élimine la moitié des abus du §5.7, et elle est gratuite.

5.3 Comparer deux moyennes : le t de Welch, et l'intervalle qui va avec

Le test de Welch compare l'écart observé à son erreur type, en laissant aux deux groupes leur propre dispersion — c'est la version à utiliser par défaut, celle de Student supposant des variances égales n'ayant plus d'argument pratique (ici les écarts types des deux groupes différent du simple au double environ). Le résultat est dans le tableau du §5.1 : $t = 1,52$, ddl = 226,1, $p = 0,13$.

Le réflexe professionnel n'est pas de s'arrêter au p : c'est de publier l'**intervalle de la différence**, [-8 279 ; 65 498] FCFA. Les deux objets disent la même chose — un intervalle à 95 % exclut zéro si et seulement si le test bilatéral rejette à 5 % — mais l'intervalle dit aussi **de combien** on pourrait se tromper. Une différence de 65 498 FCFA par ligne comme une absence de différence sont toutes deux compatibles avec ce que nous voyons : c'est cela, le résultat.

Écart de montant moyen par ligne entre vendeurs, avec intervalle de confiance à 95 %

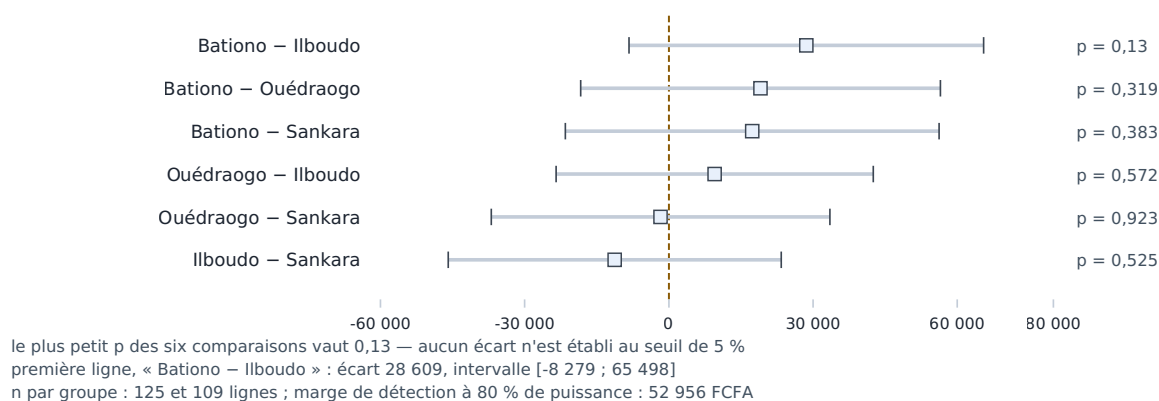


Figure — Six comparaisons deux à deux sur les montants par ligne : tous les intervalles contiennent zéro.

La figure est ce qu'on montre au comité : six paires de vendeurs, six intervalles, et **tous traversent zéro**. Le plus petit p des six comparaisons est 0,13. Un diagramme en forêt — forest plot — règle la discussion plus vite que vingt lignes de commentaires, parce que le décideur voit la largeur des barres.

Attention. « Non significatif » ne veut pas dire « les vendeurs se ressemblent ». Cela veut dire : avec ces effectifs et cette variabilité, nous ne pouvons pas distinguer l'écart observé du bruit. La phrase correcte contient la plage : « écart non établi ; la différence, si elle existe, est comprise entre -8 279 et 65 498 FCFA par ligne, ce qui laisse ouverte l'hypothèse d'un avantage de Batio-no comme celle de l'égalité ».

5.4 Ce que notre étude peut détecter : puissance et marge de détection

Définition. La **puissance statistique** (statistical power) d'un test est la probabilité de rejeter H_0 quand l'écart réel vaut une taille donnée ; la **marge de détection** (en anglais minimum detectable effect) est l'écart que le test relève avec 80 % de puissance. Les deux se calculent avant la collecte, et se publient après.

Un test non concluant a une cause chiffrable : le manque de puissance.

Objet	Valeur	Lecture
Erreur type de la différence	18 821 FCFA	ce que le hasard autorise comme flou sur un écart
Puissance pour un écart réel de 20 000 FCFA	18,5 %	une chance sur cinq de le voir
Puissance pour l'écart observé (28 609 FCFA)	33,0 %	encore très insuffisant
Écart détectable avec 80 % de puissance	52 956 FCFA	le seuil de visibilité de l'étude
n par groupe pour détecter 28 609 FCFA à 80 %	412 lignes	contre 125 et 109 disponibles

Trois usages de ce tableau, tous trois absents de la plupart des rapports :

1. **Avant** la collecte : « pour voir un écart de 20 000 FCFA, il me faut environ tant de lignes ; avec ce que vous pouvez me fournir en trois semaines, nous ne verrons rien ». Cette phrase évite un mois de travail pour un « non concluant ».
2. **Après** la collecte, quand le test ne rejette pas : « notre étude ne peut pas établir une différence inférieure à 52 956 FCFA par ligne » — c'est une information, pas une excuse.
3. **Pendant** la lecture d'un rapport qui n'est pas le vôtre : si l'auteur conclut à l'absence d'effet sans donner sa marge de détection, demandez-la.

Le calcul de la marge de détection tient en une ligne : écart seuil = (t critique + z de la puissance) × erreur type, soit environ 2,8 fois l'erreur type quand on vise 80 % de puissance à un seuil de 5 %.

5.5 Deux tests, deux réponses : que faire quand ils divergent

Définition. Le **test de Mann-Whitney** (Mann-Whitney U test, test de la somme des rangs) classe les observations des deux groupes de la plus petite à la plus grande et compare les rangs, non les montants : il éprouve l'égalité des deux distributions, et reste lisible quand la moyenne n'est qu'un point d'ancrage fragile.

Revenons sur le même couple, cette fois avec les **rangs** (Mann-Whitney) : $p = 0,0161$. Le test de différence de moyennes disait 0,13, celui des distributions dit « décalage observable ». Les deux ont raison, parce qu'ils n'éprouvent pas la même chose.

Indicateur par ligne	Batio-no (125 lignes)	Ilboudo (109 lignes)
Moyenne	90 963 FCFA	62 354 FCFA
Médiane	47 849 FCFA	32 426 FCFA

Indicateur par ligne	Bationo (125 lignes)	Ilboudo (109 lignes)
90 ^e percentile	196 553 FCFA	113 273 FCFA
Part des lignes au-dessus de 250 000 FCFA	7,2 %	3,7 %

Le profil de Bationo est décalé vers le haut **partout à la fois** : +15 423 FCFA à la médiane, et le même décalage au 90^e percentile (196 553 contre 113 273 FCFA). Mais la moyenne de Bationo est portée par une queue épaisse, donc très variable d'un tirage à l'autre — c'est exactement ce que M02.C05 montrait sur la forme de la distribution : quand la dispersion des moyennes explose, un test sur la moyenne s'aveugle alors que le décalage d'ensemble est réel.

La décision professionnelle n'est pas « je garde le plus joli des deux p » mais : **quelle hypothèse porte la décision ?** Si la prime se calcule sur le chiffre d'affaires, c'est la moyenne qui compte, et elle n'est pas établie. Si l'on veut savoir si Bationo amène des clients qui achètent plus systématiquement, la comparaison des distributions est plus proche de la question. Écrivez le lien entre l'objet de la décision et l'hypothèse du test, et la divergence cesse d'être un embarras : elle devient l'information.

Attention. Un intervalle de confiance porte sur une **grandeur estimée**, jamais sur un verdict. Publier « $p = 0,13$, donc pas de différence » puis, trois lignes plus loin, « le panier moyen de Bationo est supérieur de 28 609 FCFA » revient à vendre deux histoires incompatibles : la seconde phrase suppose que l'écart est réel, la première qu'il ne l'est pas. Gardez la page et dites ce qu'elle autorise.

5.6 Fréquences et associations : le chi-deux

Définition. Le **test du khi deux d'indépendance** (chi-squared test of independence) compare, case par case, les effectifs observés d'un tableau croisé aux effectifs **attendus** si les deux variables n'avaient aucun lien ; la statistique additionne les écarts relatifs, et se compare à la loi du khi deux dont les degrés de liberté valent $(\text{nombre de lignes} - 1) \times (\text{nombre de colonnes} - 1)$.

Le χ^2 ne compare pas des montants, il compare des **comptages** : les effectifs observés dans un tableau face aux effectifs qu'aurait produits l'indépendance parfaite. « Le taux de remise dépend-il du vendeur ? » se traite ainsi :

Vendeur	Avec remise	Sans remise	Total
Adama Bationo	42	83	125
Salamatu Sankara	49	79	128
Boureima Ouédraogo	37	81	118
Moussa Ilboudo	45	64	109

Résultat mesuré : khi deux = 3,03 pour 3 degrés de liberté, $p = 0,388$ — aucun vendeur ne se distingue dans sa façon d'accorder une remise. Le plus petit effectif théorique vaut 39,3, très au-dessus du seuil de 5 en dessous duquel l'approximation devient hasardeuse ; sous ce seuil, on passe au test exact de Fisher, qui énumère les tableaux possibles au lieu d'approcher leur distribution.

Sur une question plus ciblée — Ilboudo remet-il plus souvent que Bationo ? — le test des deux proportions est plus puissant qu'un χ^2 global : 41,3 % contre 33,6 % de lignes avec remise, écart de 7,7 points, intervalle $[-4,7 ; 20,1]$ points, $z = 1,21$, $p = 0,225$. Les deux réponses vont dans le même sens, et c'est la seconde qui porte l'information utile à la négociation commerciale : si différence il y a, elle peut aller jusqu'à 20 points en faveur d'Ilboudo, ou être nulle.

5.7 Comparaisons multiples : le piège qui fausse tout le reste

Quatre vendeurs, c'est six comparaisons deux à deux. Six occasions de trouver un « résultat » là où il n'y a que du hasard. Sous H_0 , chaque test a 5 % de chances de franchir le seuil, donc la probabilité d'au moins un franchissement sur six monte à 26,5 % environ : avec six comparaisons, on fabrique une découverte statistique tous les quatre dossiers.

Que faire, concrètement :

1. **Annoncer le nombre de tests.** « Six comparaisons ont été menées » est une phrase de la note, pas une confidence.
2. **Corriger le seuil.** Bonferroni divise α par le nombre de comparaisons : 0,05 sur six tests donne 0,0083. Sur notre fichier, le meilleur p en Welch est 0,13 : rien ne passe avant correction, encore moins après. Le test des rangs, lui, donne 0,0161 sur Bationo contre Ilboudo — significatif à 5 %, **non significatif** après Bonferroni. Cette ligne de conduite évite de choisir le test après coup.
3. **Hiérarchiser les questions.** Une question principale, testée au seuil de 5 %, et des questions secondaires affichées en exploratoire, sans étoile ni conclusion. Un tableau de bord qui met onze indicateurs en gras est un appel à la surprise.
4. **Répliquer plutôt que commenter.** Un résultat né d'une comparaison multiple devient intéressant à partir du moment où une seconde période, un second magasin ou une nouvelle extraction le reproduit.

5.8 Les limites du test, et les outils

Quatre limites à écrire dans la note, parce qu'elles conditionnent ce que la décision peut porter.

- **Le grain et l'indépendance.** Le test de Welch suppose des observations indépendantes. Or nos lignes ne le sont pas : elles sont regroupées en tickets (1,52 ligne par ticket), et 84 des 316 tickets portent **plusieurs vendeurs à la fois** — les deux groupes comparés se recouvrent partiellement, ce qui viole l'hypothèse même de l'indépendance entre échantillons. Deux conséquences chiffrables : au niveau de la ligne, l'erreur type est sous-estimée d'un facteur de l'ordre de la racine de 1,52 ; et si l'on reformule la question au grain du ticket, en affectant chaque ticket à un seul vendeur (règle : le premier cité), le même écart donne 131 597 contre 114 934 FCFA, soit 16 663 FCFA, $t = 0,49$, $p = 0,624$, intervalle [-49 743 ; 83 069] FCFA sur 84 et 68 tickets. Le verdict change parce que la **question** a changé. Voilà pourquoi une comparaison sans grain explicite est une phrase sans sujet.
- **Le test hérite du biais.** M02.C07 a montré que notre extrait ne couvre que les jours 1 à 4 de chaque mois. Un intervalle et un p calculés dessus ne portent que sur ce périmètre-là : ils ne disent rien des fins de mois, et le test, lui, ne le signale pas. C'est à l'analyste de l'écrire.
- **Significatif n'est pas important.** Avec les 240 000 lignes du fichier complet, un écart de 500 FCFA serait « significatif » et ne changerait aucune décision. La grandeur qui décide est l'estimation elle-même et sa plage, jamais le p.
- **Un test ne choisit pas entre deux explications.** « Bationo vend plus gros » peut venir de son portefeuille clients, de sa catégorie de produits ou du calendrier (M02.C06) : le test constate un écart, il ne tranche pas entre cause et coïncidence de périmètre.

Besoin	Tableur	SQL (DuckDB)	pandas / SciPy
t de Welch sur deux groupes	=T.TEST(a;b;2;3) (le type 3 est la variante à variances inégales)	agrégats AVG / COUNT STDDEV_SAMP par groupe, puis calcul	stats.ttest_ind(a, b, equal_var=False)
Mann-Whitney	absent (macro)	nativement (extension ou export, Python)	stats.mannwhitneyu(a, b)

Besoin	Tableur	SQL (DuckDB)	pandas / SciPy
Chi² sur un tableau	=CHISQ.TEST(observés;attendus) (héritée : =TEST.KHIDEUX)	agrégation par GROUP BY, puis test en Python	stats.chi2_contingency(pd.crosstab(x, y))
Test d'une moyenne à une cible	=(moyenne-cible)/ (ECARTYPE.STANDARD(plage))/ RACINE(NB(plage))) puis seuil =LOI.STUDENT.INVERSE(0,05;ddl)	idem, puis comparaison à la loi de Student	stats.ttest_1samp(pan, 100000)
Deux proportions	formule en cinq cellules	COUNT(*) FILTER puis z à la main	statsmodels proportions_ztest
Puissance et effectif	table pré-calculée dans l'onglet hypothèses	—	statsmodels power.TTestIndPower

Boîte à outils. Le bloc de cinq lignes qui accompagne un test dans une note : (1) la question, avec son grain et son seuil, écrite avant ; (2) les effectifs par groupe (125 et 109 lignes, ou 84 et 68 tickets — et jamais les deux sans le dire) ; (3) l'estimation de l'écart avec son intervalle (28 609 FCFA, [-8 279 ; 65 498]) ; (4) le p avec le nom du test (0,13, t de Welch ; 0,0161, Mann-Whitney) ; (5) la marge de détection de l'étude (52 956 FCFA à 80 % de puissance). Ces cinq lignes rendent le résultat attaquant au bon endroit — sur les chiffres, pas sur l'interprétation.

6. Exemple concret : « qui met-on sur le tableau d'honneur ? »

La demande. La direction commerciale veut distinguer « le meilleur vendeur du magasin 5 » pour une prime, et propose de trancher sur le CA : Bationo 11 370 350 FCFA contre Ilboudo 6 796 535 FCFA sur l'extrait.

Ce que le dossier dit, test en main.

Question posée	Réponse mesurée	Ce que cela change à la décision
Le montant moyen par ligne diffère-t-il ?	28 609 FCFA, t = 1,52, p = 0,13	non établi
Et au grain du ticket ?	16 663 FCFA, p = 0,624	non établi, avec une plage énorme
Le profil complet est-il décalé ?	p = 0,0161 en Mann-Whitney, 0,0083 exigé après Bonferroni	décalage réel mais non confirmé au seuil corrigé
Le total est-il plus haut chez Bationo ?	oui — et 110 tickets contre 98, et une part de Bois & panneaux de 2,4 % contre 4,6 % (M02.C06)	le total reflète le portefeuille, pas la performance unitaire

La réponse écrite au comité. « Sur l'extrait, aucun écart de montant moyen entre vendeurs n'est établi au seuil de 5 %, sur aucun des six couples comparés, et les intervalles de confiance vont de -45 910 à +65 498 FCFA. Cette étude ne permet pas de classer les vendeurs : elle permet de dire qu'un écart inférieur à 52 956 FCFA par ligne passerait inaperçu. Pour une prime, nous recommandons un barème assis sur des critères que ces données établissent — tenue du fichier, respect des taux de remise, évolution du CA sur douze mois complets — et non sur une comparaison de moyenne non concluante. »

Ce que le décideur retient. Trois mots : classement non établi. Un rapport qui dit cela est plus utile qu'un rapport qui distribue des médailles au hasard — et il protège celui qui l'a écrit.

7. Démonstration pas à pas : de la question au verdict

Étape 1 — Écrire la question et le seuil. « Le montant moyen par ligne d'Adama Bationo diffère-t-il de celui de Moussa Ilboudo ? » Test bilatéral, $\alpha = 0,05$, grain = la ligne (à justifier), deux groupes. Le ticket de la demande porte cette phrase ; c'est elle qui rend le reste non négociable.

Étape 2 — Vérifier que le test est licite. Effectifs 125 et 109 lignes ✓ assez grands ; colonnes numériques ✓ ; indépendance : **non** — 84 tickets portent les deux vendeurs, à signaler explicitement. Un test licite n'est pas un test exact ; la mention de la violation vaut mieux que son silence.

Étape 3 — Calculer l'écart et son erreur type, à la main.

```
from scipy import stats
a = p.loc[p.vendeur == "Adama Bationo", "montant_ttc"].astype(float)
b = p.loc[p.vendeur == "Moussa Ilboudo", "montant_ttc"].astype(float)
d = a.mean() - b.mean()
se = ((a.var(ddof=1)/len(a)) + (b.var(ddof=1)/len(b)))**0.5
t = stats.ttest_ind(a, b, equal_var=False)
print(round(d), round(se), round(float(t.statistic), 2), round(float(t.pvalue), 3),
round(float(t.df), 1))
```

Contrôle croisé : `d / se` doit redonner `t` ($28\,609 / 18\,821 = 1,52$ ✓). Ce réflexe attrape les erreurs de virgule, de filtre et de colonne mieux que n'importe quel contrôle automatique.

Étape 4 — Publier l'intervalle de la différence. $28\,609 \pm 1,96$ fois $18\,821$, soit $[-8\,279 ; 65\,498]$ FCFA. Zéro est dedans, donc $p > 0,05$: les deux écritures sont cohérentes — et c'est une vérification en soi.

Étape 5 — Confronter par une méthode sans hypothèse de loi. Test par permutation : on mélange les 234 lignes des deux groupes, on les redistribue en 125 et 109, et on recompte l'écart, 10 000 fois (graine 20260917). La proportion de fois où le hasard dépasse 28 609 FCFA donne $p = 0,1443$, contre 0,13 annoncé par le t de Welch : même verdict, à un cheveu près. Le test paramétrique est donc utilisable ici — sur d'autres données, cet écart-là serait le signal d'alarme.

Étape 6 — Traduire en décision, et en condition de reprise. « Écart non établi. Pour trancher, il faudrait environ 412 lignes par vendeur, soit une année complète de chacun des deux — ou un indicateur moins dispersé que le montant par ligne. » Cette dernière phrase est celle qui relance le travail utile : changer d'indicateur coûte moins cher que changer d'effectif.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	<code>t</code> égal à écart / erreur type	identité	$28\,609 / 18\,821 = 1,52$	effectifs ou variances mal lus
2	Intervalle et p cohérents	zéro dans l'IC si $p > \alpha$	IC contient 0, $p = 0,13$	test unilatéral publié comme bilatéral
3	Permutation et <code>t</code> convergent	écart de $p < 0,03$	0,1443 contre 0,13	queues trop lourdes pour le <code>t</code>
4	Effectifs théoriques du χ^2	minimum ≥ 5	39,3	passer au test exact de Fisher
5	Nombre de comparaisons déclaré	explicite	6 comparaisons, seuil 0,0083	sélectivité non signalée
6	Grain du test écrit	unité unique	la ligne (109-125), ou le ticket (84-68)	deux grains mélangés dans la même note

8. Erreurs fréquentes

1. **Lire p comme « la probabilité que H0 soit vraie ».** $p = 0,13$ ne veut pas dire 13 % de chances que les vendeurs soient égaux ; cela veut dire que, s'ils ne différaient pas, un tel écart surgirait dans 13 cas sur 100.
2. **Conclure à l'égalité parce que $p > 0,05$.** Un test non concluant mesure une **impossibilité de conclure** ; notre intervalle court de -8 279 à +65 498 FCFA, ce qui n'a rien d'une égalité.
3. **Traiter 0,05 comme un juge.** Un p à 0,049 ne vaut pas mieux qu'un p à 0,051 ; la décision se prend sur la plage estimée, le coût de l'erreur et la solidité du dispositif, pas sur un cheveu.
4. **Choisir le test après avoir vu les résultats.** Welch donne 0,13, les rangs donnent 0,0161 : publier le second sans dire le premier est une faute professionnelle, pas une nuance technique.
5. **Multiplier les comparaisons sans le dire.** Six tests, 26,5 % de risque d'une fausse découverte : le seuil corrigé de Bonferroni (0,0083) n'est pas une coquetterie, c'est le prix de l'honnêteté sur six questions.
6. **Oublier les dépendances internes.** Comparer des lignes alors que 1,52 ligne forment un ticket et que 84 tickets sont partagés entre vendeurs : l'effet est un excès de confiance, jamais un défaut de prudence.
7. **Tester sans avoir nettoyé le périmètre.** Un test sur un échantillon dont le taux de couverture est faux (jours 1 à 4, M02.C07) produit un p impeccable sur un objet qui n'existe pas.

9. Bonnes pratiques professionnelles

1. **Une question, un test, un seuil, écrits avant.** Dans le ticket, pas en annexe du rapport.
2. **Toujours l'intervalle avec le p.** Le p décide, l'intervalle informe. Un sans l'autre est incomplet ; les deux ensemble sont vérifiables par un tiers.
3. **Publier la marge de détection.** « Nous ne voyons rien de plus petit que 52 956 FCFA par ligne » : cette ligne désarme la moitié des demandes d'analyse prématurées et rend service au commanditaire.
4. **Un test sans hypothèse en contrôle, pas en principal.** La permutation à 10 000 tirages (graine annoncée) vérifie le Welch en trente secondes de calcul ; c'est un contrôle qualité, pas une seconde conclusion.
5. **Corriger pour les comparaisons multiples, ou les étiqueter exploratoires.** Les deux sont acceptables ; le silence ne l'est pas.
6. **Refuser la décision fondée sur un test non concluant,** et le dire une fois, par écrit. Ce n'est pas au statisticien de décider du risque que l'entreprise accepte ; c'est à lui de dire ce que les données ne permettent pas.
7. **Archiver le script du test, avec la graine.** Un verdict contesté six mois plus tard doit être rejouable ligne par ligne ; sinon ce n'est pas un verdict, c'est une ambiance de réunion.

10. Exercice guidé — trancher une comparaison de vendeurs de bout en bout

Énoncé. Testez la différence de montant moyen par ligne entre Salamatou Sankara et Boureima Ouédraogo, publiez l'intervalle, et rédigez la phrase de décision au seuil de 5 %.

Vous faites — barème 10 points, total 10.

Étape	Ce que vous produisez	Points
A	Question, H0, H1, grain et seuil écrits avant calcul	2
B	Écart, erreur type, t, ddl, p — avec le contrôle écart / erreur type	2,5
C	Intervalle à 95 % de l'écart, et lecture de sa position par rapport à zéro	2,5
D	Vérification par permutation ou par test des rangs	1,5
E	Phrase de décision, incluant la marge de détection	1,5

Nous vérifions. Écart -1 729 FCFA (moyennes 73 619 et 71 890 FCFA) · intervalle [-36 947 ; 33 488] FCFA · $t = -0,10$ · $p = 0,923$ · Mann-Whitney $p = 0,9129$ · les deux tests, les deux intervalles, la même conclusion : rien.

Nous corrigeons. Le piège de cet exercice est son apparente facilité : avec un écart quasi nul, on serait tenté d'écrire « les deux vendeurs sont équivalents ». Non : « aucun écart n'est détecté, et l'étude ne verrait pas un écart inférieur à la marge de détection ». Équivalence est un mot qui exige un test d'équivalence avec une borne décidée avant la mesure — jamais un p élevé.

11. Exercices autonomes

Exercice 8.1 (★) — Les quatre phrases. Pour le test Bationo / Ilboudo au niveau ligne, rédigez H0, H1, le grain et le seuil en une ligne chacun, puis dites laquelle des six comparaisons a le plus petit p et ce que ce chiffre signifie (une phrase, pas de jargon).

Exercice 8.2 (★) — Le contrôle de cohérence. À partir de l'écart (28 609 FCFA) et de l'erreur type (18 821 FCFA), retrouvez t , puis vérifiez que l'intervalle publié contient bien zéro. Que doit donner le p par rapport à 0,05, et pourquoi cette vérification n'est pas un recyclage du même calcul ?

Exercice 8.3 (★★) — Effet du grain. Reprenez la comparaison au niveau des **tickets** en les affectant au premier vendeur du ticket. Écrivez les deux effectifs, l'écart, le p , l'intervalle, et expliquez en trois lignes pourquoi le p remonte alors même que l'écart estimé, lui, baisse.

Exercice 8.4 (★★) — Chi-deux et petite case. Testez l'association vendeur $\times$ catégorie de produits au lieu de vendeur $\times$ remise. Donnez le khi deux, les degrés de liberté, le p et le plus petit effectif théorique ; dites si l'approximation est licite et ce qu'on ferait sinon.

Exercice 8.5 (★★★) — Dessiner une étude qui conclut. La direction veut savoir si la remise accordée sur les gros tickets a un effet sur le montant du ticket. Vous disposez des douze mois de 2025 du magasin 5 (4 524 tickets). Rédigez le protocole : question, unité d'observation, variable d'intérêt, test, seuil, ce qui manque pour parler d'effet causal, et la taille d'échantillon nécessaire pour détecter un écart de 10 000 FCFA avec 80 % de puissance. (attendu : un test sur moyennes au grain du ticket, $\alpha = 0,05$, seuil de Bonferroni si plusieurs couples, l'absence de variation expérimentale de la remise comme limite majeure — cf. M02.C06 — et un effectif de plusieurs centaines de tickets par groupe)

12. Correction détaillée

Exercice 8.1. H0 : les montants moyens par ligne des deux vendeurs sont égaux. H1 : ils diffèrent (bilatéral). Grain : la ligne (125 lignes pour Bationo, 109 pour Ilboudo). Seuil : 5 %, fixé avant calcul. Le plus petit p des six comparaisons vaut 0,13, sur ce couple : si les vendeurs ne différaient en rien, un écart d'au moins 28 609 FCFA apparaîtrait dans environ 13 cas sur 100. Aucun classement n'est établi.

Exercice 8.2. 28 609 divisé par 18 821 redonne 1,52 ✓ ; l'intervalle [-8 279 ; 65 498] contient zéro ✓ ; le p doit donc dépasser 0,05, et il vaut 0,13 ✓. Ce n'est pas un recyclage du même cal-

cul : le premier contrôle vérifie l'arithmétique, le second la cohérence entre deux sorties qui ont pu être produites par deux feuilles différentes — et la plupart des erreurs de reporting statistique sont des erreurs de copie, pas de maths.

Exercice 8.3. Au grain du ticket avec affectation au premier vendeur : 84 et 68 tickets, moyennes 131 597 et 114 934 FCFA, écart 16 663 FCFA, $t = 0,49$, $p = 0,624$, intervalle [-49 743 ; 83 069] FCFA. Le verdict se détériore parce que l'effectif fond (234 lignes deviennent 152 tickets) et que la variabilité par ticket est beaucoup plus grande que par ligne — l'erreur type passe de 18 821 à 33 881 FCFA. L'écart absolu baisse, mais le flou augmente plus vite : un test se lit sur le rapport des deux, jamais sur l'un des deux seuls.

Exercice 8.4. Khi deux = 27,72 sur $(4 - 1) \times (7 - 1) = 18$ degrés de liberté, $p = 0,066$. Licéité : le plus petit effectif théorique vaut 5,45 et aucune des 28 cases n'est sous 5 — l'approximation tient, mais de justesse, parce que la catégorie Bois ne pèse que 24 lignes sur 480 (M02.C01) : au moindre découpage supplémentaire, il faudra agréger les petites catégories ou recourir à un test exact par permutation. Verdict : $p = 0,066$, donc rien de concluant à 5 % — et pourtant c'est le seul résultat du module à frôler le seuil, ce qui se note (« association limite entre vendeur et portefeuille de produits, à confirmer sur douze mois complets »). Le réflexe à retenir : ce n'est pas le p qui conditionne la validité du test, ce sont les effectifs théoriques.

Exercice 8.5. Protocole attendu. Question : le taux de remise fait-il varier le montant du ticket. Unité : le ticket (pas la ligne : la remise s'apprécie au ticket). Variable d'intérêt : montant total TTC du ticket. Test : comparaison de moyennes entre tickets avec remise et sans remise, t de Welch, bilatéral, $\alpha = 0,05$. Taille : pour un écart de 10 000 FCFA à 80 % de puissance avec une dispersion de l'ordre de celle des paniers (182 095 FCFA), il faut nettement plus de plusieurs centaines de tickets par groupe — l'ordre de grandeur sort de la formule $n \approx 2 \times (1,96 + 0,84)^2 \times \sigma^2 / \text{écart}^2$. Limite majeure, à écrire dans le protocole : la remise n'a pas été attribuée au hasard, 64,0 % des lignes n'en portent aucune et six taux seulement existent (M02.C06) — le meilleur résultat possible est une **association**, jamais un effet ; une vraie réponse demanderait une période de test avec deux groupes de clients tirés au sort. Le barème de qualité est ici de publier la plage de l'effet, pas l'étoile du p .

13. Mini-projet M02.P4 — « La page d'incertitude » (45 min, second volet)

Commande. Prolongez le volet ouvert en M02.C07 par le volet décision, sur la même page unique : vous devez rendre un verdict argumenté sur la question « les vendeurs du magasin 5 se distinguent-ils mesurablement ? », en traitant au moins deux comparaisons (une sur montants, une sur fréquences), avec leur intervalle, leur seuil corrigé si les comparaisons sont multiples, et la marge de détection de l'étude.

Livrables numérotés. (1) la ou les questions, avec H_0 , le grain et le seuil, datés d'avant calcul ; (2) le tableau des comparaisons : écart, intervalle, test employé, p ; (3) le test sans hypothèse de loi (permutation) sur la comparaison que vous jugez la plus sensible, avec nombre de tirages et graine ; (4) le paragraphe de décision : ce que la direction peut faire de ce verdict, et ce qu'elle ne peut pas en faire ; (5) la marge de détection, avec l'effectif qu'il aurait fallu.

Barème (20 points, seuil 13). Questions et seuil écrits avant (3) · tests correctement choisis et exécutés (5) · intervalles publiés et cohérents avec les p (4) · comparaisons multiples traitées (3) · marge de détection chiffrée (3) · décision rédigée sans abus de langage (2) — total 20. La note du projet M02.P4 est la moyenne de ce volet et du volet ouvert en M02.C07 ; le document final reste la somme des pages P1 à P4, pas leur remplacement.

14. Résumé du chapitre

1. Un test éprouve H_0 : écart observé contre ce que le hasard produit. Sur notre couple vedette, 28 609 FCFA d'écart pour une erreur type de 18 821 FCFA donne $t = 1,52$ et $p = 0,13$: non concluant.
2. L'intervalle de la différence est le vrai résultat : $[-8\,279 ; 65\,498]$ FCFA. Sur les six couples de vendeurs, aucun intervalle n'exclut zéro — le classement des vendeurs n'est pas établi par ce fichier.
3. Deux tests licites peuvent diverger parce qu'ils n'éprouvent pas la même hypothèse : rangs à 0,0161, moyennes à 0,13 ; on choisit d'après la décision visée, et on publie les deux.
4. Un test non concluant se lit avec sa puissance : 18,5 % de chances de voir un écart réel de 20 000 FCFA, et une marge de détection de 52 956 FCFA par ligne — ou 412 lignes par groupe pour faire mieux.
5. Les six comparaisons obligent à un seuil corrigé (0,0083), et le grain du test (ligne ou ticket, 84 tickets partagés) peut changer le verdict : ces deux disciplines sont le contenu réel du métier.

15. À retenir

À retenir.

- Le p répond à « ce chiffre est-il compatible avec l'absence de différence ? », jamais à « la différence existe-t-elle ? » ni « H_0 est-elle vraie ? ».
- Tout test se publie avec son intervalle, ses effectifs par groupe et son nom : 28 609 FCFA, $[-8\,279 ; 65\,498]$, 125 et 109 lignes, t de Welch.
- Non significatif $\neq$ nul. La force de l'étude se mesure à sa marge de détection (52 956 FCFA par ligne ici), pas à ses étoiles.
- Autant de comparaisons, autant de seuils corrigés : 0,05 sur six tests devient 0,0083.

À retenir. La formulation qui protège. « Les montants moyens par ligne des six couples de vendeurs ont été comparés par test t de Welch au seuil de 5 %, fixé avant calcul. Aucun écart n'est significatif avant comme après correction de Bonferroni ; le plus petit p obtenu vaut 0,13. Notre étude ne détecterait pas un écart inférieur à 52 956 FCFA par ligne : en conséquence, nous ne proposons aucun classement de vendeurs fondé sur ces données, et recommandons de raisonner sur douze mois complets avec un indicateur moins dispersé. »

16. Évaluation formative (auto-correction, 12 min)

1. Un p de 0,13 sur un écart de 28 609 FCFA : que dit-il, que ne dit-il pas ? **2.** Pourquoi publier l'intervalle de la différence en plus du p , alors que les deux portent la même information ? **3.** Mann-Whitney donne 0,0161 et Welch 0,13 sur les mêmes données. Quelle règle de conduite, et quel lien avec la forme de la distribution (M02.C05) ? **4.** Quelles deux violations des conditions d'emploi du test de Welch notre comparaison de lignes contient-elle ? **5.** Un directeur demande : « alors ils sont équivalents ? ». Répondez en trois phrases, dont un chiffre.

Corrigé

1. Il dit que, si les deux moyennes étaient égales, un écart au moins aussi grand que celui observé apparaîtrait dans environ 13 cas sur 100. Il ne dit pas la probabilité que les vendeurs soient égaux, ni la taille de la différence, ni que l'étude est bien conduite : un p est muet sur le dispositif. **2.** Parce que le p ne s'adresse qu'à la question « différent de zéro ? », alors que l'intervalle répond à la question de gestion « de combien, au pire ? ». $[-8\,279 ; 65\,498]$ FCFA interdit toute décision de prime, ce que « $p = 0,13$ » ne dit pas explicitement. **3.** Publier les deux, en expliquant

lequel porte la décision : le test des rangs éprouve un décalage d'ensemble du profil (médiane 47 849 contre 32 426 FCFA), celui des moyennes éprouve l'écart de chiffres d'affaires — et la moyenne, sur une distribution à queue épaisse, a une erreur type énorme. C'est la leçon de M02.C05 appliquée à la décision : la forme de la distribution commande la stabilité de l'estimateur. 4. La non-indépendance des lignes au sein des tickets (1,52 ligne par ticket) et le recouvrement des deux groupes (84 tickets portant plusieurs vendeurs à la fois, donc comparés à eux-mêmes en partie). 5. « Non. Nous n'avons trouvé aucune différence mesurable, ce qui est autre chose. Notre étude ne verrait pas un écart inférieur à 52 956 FCFA par ligne, et l'intervalle obtenu laisse tout aller de -8 279 à +65 498 FCFA. Pour parler d'équivalence, il faudrait avoir écrit à l'avance une borne — par exemple “ équivalents si l'écart reste sous 10 000 FCFA ” — et la vérifier ; c'est un test d'équivalence, pas celui que nous avons conduit. »

Projet M02.P — « Profil statistique du magasin 5 »

Module M02 · projet de fin de module · 6 h · à rendre avant le module 3 · barème /20, seuil de passage 13/20

Le cadre réel. Vous êtes toujours chez Sahel Distribution SA. Le directeur a maintenant une phrase pour vous : « je ne veux pas votre moyenne, je veux la vérité du magasin ». Derrière cette phrase maladroite se trouve exactement le travail du module 2 : dire ce que vaut chaque indicateur, comment il a été calculé, ce qu'il dit, ce qu'il ne dit pas, et ce que l'extrait permet ou non d'affirmer. Vous rendez six pages, pas une.

Note de cohérence. Le plan validé annonçait « Profil statistique de la Boutique Kwame, 1 500 ventes ». Le socle de données réel de la formation fournit un autre volume — 480 lignes et 316 tickets une fois nettoyés, pour le magasin 5 sur 2025 — et c'est sur lui que portent les livrables, afin que chaque chiffre du corrigé soit vérifiable. Le livrable, le barème et les exigences de preuve sont inchangés.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Fichier de travail	01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv	le fichier nettoyé au projet M01 : 480 lignes x 13 variables, 316 tickets
Population de référence	01_socle_donnees/data/reference/ventes_propres.csv	toutes les ventes propres, 240 000 lignes ; le magasin 5 sur 2025 y tient 7 676 lignes et 4 524 tickets
Contrôle des chiffres	01_socle_donnees/data/reference/chiffres_cites.md	à utiliser après chaque calcul, pour vérifier — pas pour remplir
Outils	tableur (Excel ou LibreOffice) + Python si vous l'avez	le tableur est obligatoire au moins une fois, Python sert de contrôle

Les six livrables

L1 — Page d'identité statistique (1 page). Population visée, unité sondée, effectifs (480 lignes, 316 tickets, 7 676 lignes et 4 524 tickets en face), taux de représentation, règle d'extraction reconstituée, liste des variables par nature (quantitatives discrètes / continues / nominales / ordinales), et les trois questions du module auxquelles le fichier peut répondre. Une ligne par rubrique, aucune phrase de remplissage.

L2 — Page des centres et de la dispersion (1 page, prolongement de M02.P1 et M02.P2). Cinq indicateurs, chacun au format valeur · calcul · ce que ça dit · ce que ça ne dit pas : moyenne, médiane, mode, écart type, écart interquartiles. Vous ajoutez le coefficient de variation et vous expliquez ce qu'il fait saliver.

L3 — Page des positions et de la forme (1 page). Premier et troisième quartiles, trois percentiles choisis parmi dix, quatre-vingts, quatre-vingt-quinze ; bornes de Tukey et nombre d'observations hors borne ; histogramme commenté (classes, effectifs par classe, part du chiffre d'affaires par classe) ; asymétrie et excursion des valeurs.

L4 — Page des liaisons (1 page). Trois liaisons au choix, chacune avec covariance ou coefficient, effectif, grain, carré du coefficient, droite de régression si elle a un sens, et une ligne « ce que la liaison ne dit pas ». Vous menez en plus **un** contrôle de confusion et vous donnez les deux chiffres, avant et après.

L5 — Page d’incertitude et de décision (1 page). Intervalle à 95 % du panier moyen et de la médiane, biais de l’extrait chiffré, un test de comparaison entre deux vendeurs conduit **au tableur puis vérifié en Python**, le même test entre le magasin 1 et le magasin 5 sur l’année 2025, et la marge de détection de votre étude. Un paragraphe de décision, une phrase de prudence.

L6 — Anomalies et conclusion (1 page). **Trois anomalies chiffrées** au choix (doublons, quantités hors échelle, dates manuscrites, clients non renseignés, retours négatifs, tickets aberrants), chacune en quatre lignes : ce qui est observé, combien, ce que ça change au chiffre, ce qui a été fait. Puis la **conclusion en cinq lignes**, qui dit ce que l’entreprise peut décider avec ce profil, et ce qu’elle ne peut pas.

Contraintes de rendu

1. Un seul document PDF de six pages, nommé `2026-MM-JJ_projet_M02_P_votre_nom.pdf`, plus le classeur de travail et, si vous en avez utilisé, les scripts.
2. Chaque page porte en pied de page : l’effectif utilisé, le grain, le nom du fichier et la date de lecture.
3. Tout nombre affiché doit être accompagné de son moyen de preuve à portée de main : capture de la formule, ligne de commande, ou extrait du script. Un chiffre sans méthode est compté comme faux, même juste.
4. Précision : montants en entiers de FCFA, coefficients à trois décimales, pourcentages à une décimale. Aucun arrondi « de présentation » non signalé.
5. Les comparaisons multiples (plus de deux groupes) doivent annoncer le nombre de tests et le seuil employé.
6. **Interdits** : recopier une valeur depuis `chiffres_cites.md` sans le calcul correspondant ; mélanger dans une même page un total au niveau ligne et un total au niveau ticket sans écrire lequel est lequel ; publier un test sans son intervalle ; conclure à l’absence de différence sur un p supérieur au seuil.

2. Barème détaillé

#	Critère	Pts	Ce qui fait perdre les points
1	L1 : identité statistique exacte, règle d’extraction reconstituée	3	taux de représentation absent ; unité sondée confondue avec le nombre de lignes
2	L2 : cinq indicateurs justes, avec le « ce que ça ne dit pas »	3	mode unique annoncé alors qu’il y en a deux ; moyenne seule présentée comme « le panier type »
3	L3 : positions et forme, histogramme commenté	4	classes fermées qui avalent les retours négatifs ; bornes de Tukey calculées sur le mauvais effectif
4	L4 : liaisons chiffrées et contrôle de confusion mené	3	R^2 publié sans r , ou sans l’effectif ; pente extrapolée hors domaine
5	L5 : intervalles, tests exécutés aux deux outils, marge de détection	4	test exécuté sans grain déclaré ; p lu comme probabilité que H_0 soit vraie ; bootstrap sans graine
6	L6 : trois anomalies chiffrées, conclusion honnête, journal	3	anomalie racontée et pas comptée ; conclusion qui généralise au-delà du périmètre
Total		20	seuil de passage en M03 : 13

3. Correction pas à pas (corrigé enseignant)

Ce corrigé est rédigé comme un compte-rendu de travail. Les valeurs sont celles produites par les fichiers livrés du socle ; si votre fichier diffère, régénérez-le (python3 01_socle_donnees/scripts/generation_socle.py) avant de vous comparer.

3.1 — L1 : l'identité statistique, telle qu'elle doit être rendue

Rubrique	Réponse attendue
Population visée	les tickets du magasin 5 sur l'année 2025 : 4 524 tickets, 7 676 lignes
Échantillon réellement utilisé	316 tickets, 480 lignes — soit 7,0 % des tickets et 6,3 % des lignes
Règle d'extraction	les 40 premières lignes de chaque mois, triées par date : les jours 1 à 4 seulement
Unité sondée à retenir	le ticket pour toute question de clientèle, la ligne pour toute question de produit
Variables quantitatives	quantite (discrète), prix_unitaire_ht, remise, montant_ht, montant_ttc (continues au sens du métier : montants entiers en FCFA)
Variables nominales	magasin, vendeur, produit, categorie ; client est un identifiant, pas une quantité
Questions ouvertes au fichier	panier moyen et sa dispersion, structure par catégorie, liaisons quantité-montant, comparaison de vendeurs
Question fermée	la marge nette, la satisfaction client, l'effet d'une promotion : aucune de ces trois n'est répondable ici

Le correcteur vérifie en priorité la troisième ligne : un candidat qui écrit « 480 lignes prises au hasard » a perdu le fil du module, un candidat qui écrit « les 40 premières lignes de chaque mois » a compris M02.C07.

3.2 — L2 : centres et dispersion

Indicateur	Valeur	Calcul	Ce que ça dit	Ce que ça ne dit pas
Moyenne panier	du 114 FCFA	156 somme de 36 073 185 FCFA sur 316 tickets	la masse d'affaires par ticket	71,8 % des tickets sont sous cette valeur
Médiane	58 FCFA	423 =MEDIANE() sur les 316 totaux	la moitié des tickets est en dessous	ce que font les 25 plus gros tickets
Modes	6 797 et 8 260 FCFA	valeurs les plus fréquentes, deux ex-æquo	deux tailles de ticket reviennent souvent	rien sur les gros paniers, qui sont justement l'enjeu
Écart type	182 FCFA	095 =ECARTYPE.STANDARD()	la dispersion est supérieure à la moyenne	la dispersion est tirée par la queue de droite
Écart interquartiles	109 FCFA	924 130 559 moins 20 635	la moitié centrale s'étale sur cette plage	le chiffre d'affaires total
Coefficient de variation	de 159,5 %	écart type sur moyenne	aucune moyenne ne représente ce fichier à elle seule	—

Phrase à exiger en pied de page : « le ratio moyenne sur médiane vaut 1,95 : la moyenne est à peu près le double de la médiane, ce qui signe une distribution étirée à droite, pas une augmentation générale des paniers ».

3.3 — L3 : positions et forme

Premier quartile 20 635 FCFA, troisième 130 559 FCFA, P10 = 7 390, P80 = 165 177, P90 = 235 318, P95 = 435 984 FCFA ; minimum -150 804, maximum 1 343 666, étendue 1 494 470 FCFA.

Bornes de Tukey : borne supérieure $130\,559 + 1,5 \times 109\,924$, soit 295 445 FCFA, avec 25 tickets au-delà.

L'histogramme attendu compte **sept** classes, la première et la dernière ouvertes, avec 3, 133, 84, 25, 23, 17 et 31 tickets. Deux pièges de fabrication, à commenter :

1. borné à 150 000 FCFA, l'histogramme écrase la moitié du chiffre d'affaires dans la dernière classe ; six classes fermées font disparaître les trois tickets négatifs, qui sont pourtant la preuve qu'il y a des retours ;
2. la classe 0 à 50 000 FCFA pèse 42,1 % des effectifs et 7,3 % du chiffre d'affaires : les deux lectures doivent apparaître, sinon le graphique raconte « un magasin de petits clients » alors que le dernier décile porte 47,5 % du chiffre d'affaires.

Asymétrie 3,84, excès de kurtosis 18,25, 307 valeurs de panier distinctes sur 316 : le vocabulaire à vérifier dans la copie est celui-là, et le mot « normale » n'y a pas sa place.

3.4 — L4 : liaisons

Le couple attendu, parce qu'il est instructif sans être trivial : `quantite` et `montant_ttc` au niveau ligne. Covariance 857 634, coefficient de Pearson 0,712, Spearman 0,641, R^2 50,7 %, pente 12 237 FCFA par pièce, ordonnée à l'origine -10 402 FCFA. Contrôle exigé : la droite au point moyen rend 75 257 FCFA contre 75 152 FCFA observés — l'écart est un arrondi d'affichage, et le candidat qui écrit ce contrôle gagne le point de méthode.

Le contrôle de confusion attendu : l'écart de panier moyen entre le premier et le dernier vendeur vaut 34 014 FCFA toutes catégories confondues ; recompté à catégorie contenue, il remonte à 57 346 FCFA en moyenne (17 433 FCFA au moins en Quincaillerie, 173 167 FCFA au plus en Electricité). La phrase de conclusion juste est : « à catégorie de produits égale, les écarts entre vendeurs ne se résorbent pas — ils s'agrandissent. Le portefeuille de produits n'explique donc pas le classement, et il ne l'innocente pas non plus. »

3.5 — L5 : incertitude, tests, décision

Objet	Valeur attendue	Vérification demandée
Intervalle à 95 % du panier moyen	[94 078 ; 134 234] FCFA, largeur 40 156	erreur type 10 244 FCFA retrouvée par $182\,095$ sur racine de 316
Biais de l'extrait	-2,3 % sur la moyenne, -14,3 % sur la médiane	médiane des jours 1 à 4 dans la population : 58 424 FCFA
Bootstrap de la médiane	[52 318 ; 64 632] FCFA	2 000 tirages, graine 20260917
Valeur de population du panier moyen	116 886 FCFA	elle est bien dans l'intervalle : la couverture est vérifiée
Comparaison Bationo / Ilboudo, au niveau ligne	écart 28 609 FCFA, $t = 1,52$, $p = 0,13$, IC [-8 279 ; 65 498]	tableur <code>=T.TEST(tab1;tab2;2;3)</code> puis <code>scipy.stats.ttest_ind(..., equal_var=False)</code>
Contrôle par permutation	$p = 0,1443$	10 000 redistributions, graine annoncée
Test des rangs sur le même couple	$p = 0,0161$	publié avec la mention du seuil corrigé 0,0083 : non concluant après correction
Comparaison magasin 1 / magasin 5, sur 2025 au grain du ticket	écart -8 997 FCFA, $t = -3,37$, ddl 6 782,8, $p = 0,000751$, IC [-14 226 ; -3 767]	14 463 tickets contre 4 524 ; moyennes 107 889 et 116 886 FCFA
Marge de détection de l'étude vendeurs	52 956 FCFA par ligne à 80 % de puissance	412 lignes par groupe auraient été nécessaires

Les deux derniers blocs sont le cœur du livrable : le premier montre un **écart établi** (intervalle excluant zéro, avec 14 463 et 4 524 tickets), le second montre que le même dispositif, ramené à 125 et 109 lignes, ne voit rien de stable. Le correcteur attend la phrase qui relie les deux : « la

différence entre magasins est mesurable, la différence entre vendeurs non, et la raison est l'effectif, pas la volonté ».

Une exécution au tableur doit laisser voir : la colonne des totaux par ticket (`=SOMME.SI()`), les quatre cellules `MOYENNE`, `NB`, `ECARTYPE.STANDARD`, puis la formule de l'erreur type et la marge. Un candidat qui rend uniquement le `p` Python perd le point de méthode demandé.

3.6 — L6 : les trois anomalies, écrites comme attendu

1. **Neuf lignes en double** dans le fichier reçu : $36\,339\,317 - 36\,073\,185 = 266\,132$ FCFA comptés en trop, soit $+0,74\%$ de chiffre d'affaires si on ne les retire pas. Elles ont été retirées au projet M01 et ne réapparaissent nulle part dans ce livrable.
2. **Sept quantités hors échelle** : maximum 14 000 pièces dans le fichier reçu, 68 une fois l'unité corrigée. Sans correction, la quantité moyenne serait faussée d'un facteur qui déforme la pente de la droite quantité-montant.
3. **Cent trois lignes sans client exploitable** (dont 18 sans identifiant renseigné), soit $21,5\%$ de l'effectif : toute statistique de clientèle est donc calculée sur un périmètre de fait, et le taux est à rappeler à chaque fois — c'est le seul des trois défauts qui ne se répare pas en nettoyant.

Conclusion en cinq lignes, modèle : le magasin 5 sur 2025, dans l'extrait disponible, fait un panier moyen de 114 156 FCFA, intervalle [94 078 ; 134 234], et un demi-ticket type à 58 423 FCFA. La masse d'affaires est concentrée : le dernier décile porte $47,5\%$ du chiffre d'affaires. Aucun classement de vendeurs n'est établi par ces données, et l'extrait sous-estime la médiane annuelle de $14,3\%$ du fait de sa couverture en jours. La liaison quantité-montant est forte (0,712) mais explique la moitié du phénomène, pas l'autre. Nous recommandons un recalcul sur les douze mois complets avant toute décision d'assortiment.

4. Grille d'auto-évaluation avant rendu

Répondez par oui ou non ; trois « non » = ne rendez pas, reprenez la page concernée.

#	Question	Si non
1	Chaque page indique-t-elle effectif, grain, fichier et date de lecture ?	pied de page manquant, point perdu
2	Ai-je écrit la règle d'extraction et son effet chiffré (médiane -14,3 %) ?	reprendre M02.C07 §5.3
3	Moyenne et médiane sont-elles données ensemble, avec le ratio commenté ?	reprendre M02.C02
4	L'histogramme a-t-il sept classes, les deux extrêmes ouvertes ?	reprendre M02.C05 §5.2
5	Ai-je distingué effectif et chiffre d'affaires par classe ?	reprendre M02.C05 §5.4
6	Chaque coefficient publié porte-t-il son effectif, son grain et son carré ?	reprendre M02.C06 §9
7	Le contrôle de confusion est-il chiffré avant/après ?	reprendre M02.C06 §5.6
8	Chaque intervalle est-il accompagné de son erreur type et du nombre de tirages quand il vient d'un bootstrap ?	reprendre M02.C07 §5.5-5.6
9	Chaque test annonce-t-il sa question, son grain, son seuil et le nombre de comparaisons ?	reprendre M02.C08 §5.1-5.2
10	Ai-je écrit « non établi » là où j'ai été tenté d'écrire « identique » ?	reprendre M02.C08 §5.3
11	Les trois anomalies sont-elles comptées, pas racontées ?	reprendre M01 (doublons) et M02.C01
12	Un tiers peut-il rejouer mes calculs sans moi (scripts, formules, graine) ?	joindre le classeur et les scripts

5. Ce que le correcteur regardera en premier

1. **Le pied de page de la page 2.** Effectif, grain, fichier : trois indications qui, si elles manquent, décrètent que le module n'a pas été travaillé mais lu.
2. **La présence de deux chiffres là où un seul suffirait.** Moyenne et médiane, r et R^2 , p et intervalle, écart avant et après contrôle : le module 2 se joue là, pas dans la justesse arithmétique.
3. **La mention du biais de couverture.** Une seule page, même mal mise en forme, qui écrit « jours 1 à 4 » et « -14,3 % » vaut plus que six pages impeccables qui l'ignorent.
4. **La discipline sur les comparaisons.** Six comparaisons annoncées comme telles avec un seuil corrigé, c'est un comportement professionnel, pas une coquetterie technique.
5. **La conclusion.** Cinq lignes qui disent ce que l'on peut décider — et ce que l'on ne peut pas — sont la vraie livraison du module. Une conclusion qui promet une action non fondée sur les pages précédentes fait perdre le point entier.

Évaluation M02 — « Statistiques descriptives et inférence »

Module M02 · durée totale 2 h 45 · quatre épreuves · seuils : quiz 14/20, étude de cas 12/20, projet 13/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	remettre à plat ce qui a été compris en cours de module
B · Quiz	35 min	/20	oui	vingt questions à réponse courte, une seule bonne réponse
C · Exercices pratiques	55 min	/20 (auto-corrigé)	oui	deux séries menées jusqu'au chiffre, tableur ou Python
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier de quatre pages, chiffré et prudent

Le score du module se lit ainsi : **B ≥ 14** et **D ≥ 12** ouvrent le module 3 ; le projet M02.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D — l'écrit de connaissances ne remplace pas le travail sur les données.

A · Questions de récupération (non notées)

À traiter à l'oral ou par écrit, sans document. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de se tromper dans les épreuves notées.

1. Pourquoi la moyenne d'une distribution de paniers peut-elle être juste et inutilisable ?
2. Quelle différence de sens y a-t-il entre 150 000 FCFA « au 70^e percentile » et « le 70^e plus gros ticket » ?
3. Que devient un histogramme si la dernière classe est fermée à 150 000 alors que le maximum vaut 1 343 666 ?
4. Pourquoi dit-on d'un intervalle de confiance qu'il porte sur le paramètre et non sur les observations ?
5. Un p-valeur de 0,13 permet-il d'affirmer que deux vendeurs font le même chiffre ? Répondez en une phrase.
6. Pourquoi compare-t-on des moyennes de prix **par ligne** et non des chiffres d'affaires par vendeur ?
7. Qu'ajoute un test des rangs quand le test de Welch conclut déjà ?
8. Un échantillon de 385 personnes suffit pour une marge de 5 points. Un échantillon de 480 lignes donne-t-il une marge de 5 points sur un montant ? Expliquez.

B · Quiz (20 questions · 20 points)

Barème : 1 point par question, aucune soustraction de point pour une erreur de signe ou d'unité. Les réponses justifiées en une ligne peuvent rapporter un demi-point de bonus, plafonné à 20.

Bloc 1 — Population, échantillon, biais (Q1 à Q5)

Q1. Un fichier contient 480 lignes issues des 40 premières lignes de chaque mois. On veut estimer le panier moyen annuel. Ce que mesure avant tout le taux de 6,3 % : a) la variance · b) la représentativité · c) le taux de remplissage · d) la précision

Q2. Le même échantillon ne contient que les jours 1 à 4 de chaque mois. On peut dire sans mesurer que la moyenne : a) est biaisée de 2 % · b) est biaisée vers le bas · c) est non représentative · d) est biaisée de 14 %

Q3. Trois modalités possibles pour le champ magasin alors qu'il y a cinq magasins. Le risque principal : a) une clé orpheline · b) une valeur manquante · c) un type de donnée · d) une normalisation

Q4. Un champ de montants est vide sur 18 lignes, absent sur 103. La première décision statistique : a) compléter par zéro · b) compléter par la moyenne · c) dire sur quelle population porte chaque chiffre · d) supprimer les 103 lignes

Q5. L'unité d'échantillonnage d'une étude de clientèle doit être : a) la ligne de vente · b) le ticket · c) le jour · d) le client

Bloc 2 — Centres (Q6 à Q8)

Q6. Sur un échantillon de 316 paniers, le plus gros vaut 1 343 666 FCFA. Passer de 316 à 315 paniers en retirant ce maximum déplace surtout : a) la médiane · b) le mode · c) la moyenne · d) l'écart interquartiles

Q7. Deux valeurs également fréquentes, deux modes donc, séparées de 1 463 FCFA. Le mode est ici : a) inutilisable · b) une valeur de prix unitaire, pas de panier · c) un signal de palier tarifaire · d) une erreur de saisie

Q8. Moyenne 114 156, médiane 58 423 FCFA. Le chiffre qui dit « un client sur deux est en dessous » : a) la moyenne · b) la médiane · c) le mode · d) le 75^e percentile

Bloc 3 — Dispersion et positions (Q9 à Q12)

Q9. Écart type 182 095 pour une moyenne de 114 156 FCFA. Le coefficient de variation vaut environ 159,5 % et signale : a) des données fausses · b) une dispersion supérieure à la moyenne, moyenne peu représentative · c) une erreur d'unité · d) une distribution normale

Q10. Premier quartile 20 635 FCFA, troisième 130 559 FCFA. La borne supérieure de Tukey vaut : a) $130\,559 + 1,5 \times 109\,924$ · b) $1,5 \times 130\,559$ · c) $Q3 + 1,5 \times Q1$ · d) $Q3 + 3 \times IQR$

Q11. Sur 316 tickets, 158 sont entre les quartiles — et non 158 comme l'annoncent 50 % des observations. L'écart de 2 s'explique par : a) une erreur de calcul · b) la convention de quantile, qui peut placer une valeur exactement sur un quartile · c) 2 aberrants · d) un arrondi de l'écart type

Q12. Le 90^e percentile du montant par ligne vaut 234 518, 235 318, 236 118 ou 243 010 FCFA selon la méthode. La bonne conduite : a) publier la plus grande · b) publier celle du tableur · c) publier l'une et nommer la méthode · d) publier la moyenne des quatre

Bloc 4 — Formes et liaisons (Q13 à Q16)

Q13. Asymétrie 3,84, excès de kurtosis 18,25. Le couple de qualificatifs exact : a) symétrique pointue · b) à droite et pointue · c) à gauche et plate · d) normale et étroite

Q14. Le chiffre d'affaires est concentré : le dernier décile porte 47,5 % du total, et 1,6 % seulement est sous le premier quartile. La première page du rapport devrait montrer : a) un camembert par catégorie · b) une courbe cumulative · c) un nuage de points · d) un effectif par vendeur

Q15. La covariance entre `quantite` et `montant_ttc` vaut 857 634, en unités « pièces × FCFA ». Comment passe-t-on de là au coefficient de Pearson ? a) on divise la covariance par le produit des deux écarts types · b) on multiplie la covariance par les deux écarts types · c) on élève 0,712 au carré · d) on divise la covariance par l'écart interquartiles

Q16. Pente 12 237 FCFA par pièce, ordonnée -10 402 FCFA, coefficient 0,712. L'affirmation recevable : a) doubler la quantité double le montant · b) une pièce supplémentaire accompagne en moyenne 12 237 FCFA de plus sur la ligne · c) 50,7 % du montant vient de la quantité, le reste est faux · d) la relation est causale

Bloc 5 — Estimation et tests (Q17 à Q20)

Q17. Un intervalle [94 078 ; 134 234] FCFA sur le panier moyen. Le nombre 40 156 est : a) l'erreur type · b) la marge d'erreur · c) la largeur de l'intervalle · d) la variance

Q18. Test de Welch entre deux vendeurs : $t = 1,52$, ddl 226,1, $p = 0,13$, IC [-8 279 ; 65 498]. La conclusion écrite dans le rapport : a) les vendeurs sont équivalents · b) l'écart est de 28 609 FCFA · c) aucun écart stable n'est établi sur cet extrait · d) le vendeur le plus performant est Bationo

Q19. Six comparaisons de paires sont menées, la plus petite p-valeur vaut 0,13, le seuil après correction 0,0083. On conclut : a) une comparaison est significative · b) aucune n'est significative au seuil corrigé, et l'étude manque de puissance · c) il faut publier la plus petite p · d) il faut augmenter alpha

Q20. Marge de détection de 52 956 FCFA par ligne à 80 % de puissance. Le même test au niveau ticket donne une erreur type de 33 881 contre 18 821 : la raison est : a) une erreur de formule · b) le passage de 234 à 107 observations, et le mélange de plusieurs vendeurs sur 84 tickets · c) des queues trop lourdes · d) un seuil mal choisi

C · Exercices pratiques (2 séries · 55 min · auto-correction exigée)

Règles : une feuille de calcul ou un script par série ; tous les nombres doivent être retrouvables par une formule ou une ligne de commande ; les arrondis sont affichés en fin de chaîne. Les corrigés de la partie C sont donnés à la fin du document — l'auto-correction fait partie de l'épreuve, un candidat qui ne se corrige pas perd le point de méthode.

E1 · Profil du magasin (30 min) 1. E1.1 — effectif de lignes, effectif de tickets, moyenne et médiane du montant TTC par ligne, écart type. 2. E1.2 — trois premières et trois dernières lignes triées par `montant_ttc`, par ticket : le chiffre d'affaires est-il porté par les gros tickets ? Donnez le pourcentage du total détenu par les 32 plus gros tickets. 3. E1.3 — Q1, Q3, IQR, borne supérieure de Tukey, nombre d'observations hors borne, et ce que devient cette borne si l'on exclut les retours négatifs.

E2 · Estimer et décider (25 min) 1. E2.1 — intervalle à 95 % de la part de lignes avec remise dans l'échantillon (173 lignes sur 480), puis comparaison à la part observée dans la population de référence si vous y avez accès. 2. E2.2 — comparer le montant par ligne des deux vendeurs les plus fournis, au tableur **puis** en Python : moyenne, écart type, effectif, t, ddl, p, intervalle à 95 % de l'écart. 3. E2.3 — même comparaison par les rangs, puis par permutation (10 000 tirages, graine annoncée). Vous rédigez les trois p-valeurs et une phrase de décision unique. 4. E2.4 — avec 30 % de remise sur les 316 tickets, combien de tickets faudrait-il dans chaque groupe pour mettre en évidence un écart de 28 609 FCFA ?

D · Étude de cas (20 points · seuil 12 · 60 min)

Le dossier

Vous recevez un courriel du directeur commercial :

« Bationo fait des paniers plus gros qu'Ilboudo : 90 963 contre 62 354 FCFA par ligne. C'est 46 % d'écart. Je veux deux choses : la preuve chiffrée, et si tu ne la trouves pas, dis-moi franchement que ces chiffres ne veulent rien dire. Et pendant que tu y es, est-ce que le magasin 5 vend plus cher que le magasin 1 ? »

Vous disposez de : l'extrait nettoyé de 480 lignes (316 tickets), le fichier de référence de 240 000 lignes, la table `dim_date`, un tableur, Python si vous le souhaitez, et une demi-heure de calcul.

Les quatre livrables demandés

1. **La preuve, ou son absence, sur les vendeurs** (6 pts) — au niveau ligne et au niveau ticket, avec les deux échantillons, l'écart estimé, l'intervalle, le test, le seuil corrigé si vous comparez plusieurs paires, et la marge de détection.
2. **La réponse sur les deux magasins** (6 pts) — périmètre et grain annoncés, les deux moyennes, les deux effectifs, l'écart, l'intervalle, le p, la décision.
3. **Une recommandation datée** (4 pts) — trois lignes maximum, qui ne dépasse pas ce que les pages précédentes établissent ; par exemple un plan de mesure (recalcul sur douze mois, effet de la couverture en jours, ce qu'il faudrait collecter).
4. **Un encadré « ce qui aurait changé ma réponse »** (4 pts) — trois contre-vérifications que vous auriez aimées et qui sont impossibles avec ces données (le client, le stock, la marge nette).

Ce que le correcteur attend, ligne à ligne

Livable	Attendu	Point perdu si
1	écart 28 609 FCFA, $t = 1,52$, $p = 0,13$, IC [-8 279 ; 65 498], seuil corrigé 0,0083, MDE 52 956 ; version ticket écart 16 663 FCFA, $p = 0,624$	le mot « équivalents » apparaît ; le grain n'est pas annoncé
2	magasins 1 et 5 sur 2025 au grain du ticket : 107 889 (14 463) et 116 886 FCFA (4 524), écart -8 997, $t = -3,37$, $p = 0,000751$, IC [-14 226 ; -3 767] → écart établi	le candidat compare des lignes sans le dire, ou omet les effectifs
3	décision de mesurer avant de gérer : pas de mutation, pas de prime sur cette base ; recalcul sur douze mois	la recommandation dépasse les chiffres
4	trois limites nommées, avec ce qu'il faudrait comme donnée pour chacune	limites rédigées en « il faudrait plus de données »

Total : 20 points. Seuil de passage : 12.

Corrigé

B · Quiz — réponses et explications

#	Réponse	Explication en une ligne
Q1	b	le taux de 6,3 % est un taux de couverture : il mesure l'exposition à un biais, pas une variance

#	Réponse	Explication en une ligne
Q2	c	le sens et l'amplitude du biais ne se devinent pas, ils se mesurent ; seul le caractère non représentatif est certain
Q3	b	trois modalités sur cinq magasins : deux magasins sont absents du fichier, ce qui change le périmètre de la réponse
Q4	c	avant de combler ou de supprimer, on écrit sur quoi porte chaque chiffre
Q5	b	un client, un ticket : si l'unité est la ligne, les gros tickets comptent double
Q6	c	la moyenne déplace tout le poids de l'observation extrême, la médiane presque rien
Q7	c	deux modes serrés sur un champ de prix signalent un palier tarifaire, pas une erreur
Q8	b	définition de la médiane ; 71,8 % des tickets sont sous la moyenne, ce que la moyenne ne dit pas
Q9	b	un CV de 159,5 % est un signal de dispersion, pas une faute de calcul
Q10	a	$Q3 + 1,5 \times IQR : 130\,559 + 1,5 \times 109\,924$, soit 295 445 FCFA
Q11	b	166 observations exactement à Q1, la frontière dépend de la convention retenue
Q12	c	les méthodes diffèrent de 3,6 % sur cet extrait : on publie une valeur et on nomme la règle
Q13	b	asymétrie positive et excursion marquée, jamais « normale »
Q14	b	la concentration est une affaire de cumul ; le camembert ne la montre pas
Q15	a	covariance sur produit des deux écarts types — c'est ce qui rend le nombre comparable d'un fichier à l'autre ; le carré du coefficient, $0,712^2$, est le R^2
Q16	b	lecture de la pente, dans le domaine observé, sans causalité
Q17	c	40 156 est la largeur ; la marge est la moitié, 20 078 FCFA
Q18	c	un intervalle qui contient zéro n'établit rien ; il n'innocente pas non plus
Q19	b	$0,13 > 0,0083$: aucune comparaison ne passe, et la puissance était faible avant même de commencer
Q20	b	moins d'observations et un effet partagé : deux raisons qui se cumulent

Grille de conversion : 20/20 à 14/20 validé · 13/20 à 11/20 révisions ciblées sur les deux blocs manquants · 10/20 et moins reprendre le module.

C · Exercices pratiques — corrigés

E1.1 — 480 lignes, 316 tickets. Montant TTC par ligne : moyenne 75 152, médiane 37 966, écart type 143 915 FCFA. `=ECARTYPE.STANDARD(plage)` ou `df.montant_ttc.std()`.

E1.2 — le total de l'extrait au grain du ticket vaut 36 073 185 FCFA. Les 32 plus gros tickets, soit 10,1 % des 316, en portent 17 139 852 FCFA : **47,5 %** du chiffre d'affaires pour un ticket sur dix. Les 31 tickets au-dessus de 250 000 FCFA valent 16 903 734 FCFA, soit 46,9 %. Les neuf plus gros seulement suffisent à faire 24,0 % du total (8 642 533 FCFA). Le chiffre d'affaires est donc bien porté par les gros tickets, et la moyenne du magasin parle surtout d'eux.

E1.3 — premier quartile 20 635, troisième 130 559, IQR 109 924, borne 295 445 FCFA, 25 tickets hors borne. En excluant les 3 tickets de retour (montant négatif au niveau du ticket), la borne descend de 1 271 FCFA à 294 174 FCFA et le nombre de tickets hors borne **reste 25** : le nettoyage déplace la borne, il ne change pas le verdict. C'est la réponse attendue — et elle vaut mieux que « la borne est une ligne de conduite », parce qu'elle est mesurée des deux côtés.

E2.1 — 173 sur 480, soit 36,0 % ; erreur type 2,2 points, marge 4,3 points, intervalle [31,7 ; 40,3] % — le même calcul que M02.C07, exerc. 7.3. Dans la population de référence, la part de lignes avec remise se mesure par la même formule sur 240 000 lignes : si les deux chiffres s'écartent de plus d'une dizaine de points, l'extrait n'est plus neutre sur cette variable et la remarque va dans le rapport.

E2.2 — Bationo : 90 963 FCFA de moyenne par ligne, 125 lignes. Ilboudo : 62 354 FCFA, 109 lignes. Écart 28 609 FCFA, erreur type 18 821, marge $\pm 36\,888$, $t = 1,52$, ddl 226,1, $p = 0,13$, IC [-8 279 ; 65 498]. Au tableur `=T.TEST(plage1;plage2;2;3)`, en Python `stats.ttest_ind(a, b, equal_var=False)` : les deux doivent rendre le même p au millième près.

E2.3 — permutation 10 000 tirages, graine annoncée : $p = 0,1443$; test des rangs : $p = 0,0161$. Phrase de décision attendue : « avec 234 lignes, l'extrait ne permet d'établir aucun classement entre vendeurs ; le test des rangs signale une différence de position qui résiste mal à la correction pour six comparaisons (seuil 0,0083) ».

E2.4 — $n = 412$ lignes par groupe pour 80 % de puissance sur un écart de 28 609 FCFA avec 30 % de remise. L'extrait en compte 125 et 109 : l'étude est en dessous d'un facteur quatre.

D · Étude de cas — éléments de réponse attendus

1. Deux niveaux, deux chiffres, même conclusion. Niveau ligne : 234 observations, écart estimé 28 609 FCFA, intervalle [-8 279 ; 65 498] FCFA, $p = 0,13$, seuil corrigé 0,0083 → non concluant. Niveau ticket : 107 observations, écart 16 663 FCFA, erreur type 33 881, $p = 0,624$, intervalle [-49 743 ; 83 069] → non concluant, et il faut écrire pourquoi l'intervalle gonfle : 30 % d'observations en moins, et 84 tickets sur 316 portent plusieurs vendeurs, donc l'effet cherché est partagé. Marge de détection 52 956 FCFA : même une vraie différence de 20 000 FCFA n'aurait que 18,5 % de chances d'être détectée. Verdict à rendre : **classement non établi**, les 46 % du courriel sont un rapport de moyennes, pas un effet démontré.

2. Périmètre : année 2025, magasin 1 contre magasin 5, grain du ticket, total des montants TTC par ticket dans `ventes_propres`. Moyennes 107 889 FCFA (14 463 tickets) et 116 886 FCFA (4 524 tickets) ; médianes 65 313 et 68 160 FCFA. Écart -8 997 FCFA, erreur type 2 668, $t = -3,37$ sur 6 782,8 ddl, $p = 0,000751$, intervalle [-14 226 ; -3 767] FCFA : zéro exclu, **écart établi** — le magasin 5 fait des paniers plus gros, d'environ 8 300 à 14 200 FCFA, soit 7,7 % à 13,2 % du panier moyen du magasin 1. La prudence à joindre : ce test porte sur la population entière des deux magasins, l'extrait nettoyé n'y participe pas ; et rien ici n'explique l'écart (assortiment ou clientèle, on ne sait pas).

3. Recommandation, modèle de rédaction : « aucune mesure individuelle ne se justifie sur cette base ; nous proposons un recalcul sur douze mois complets au grain du ticket (14 463 et 4 524 tickets disponibles, seuil d'observation 0,0083), et la collecte du rattachement client-ligne qui manque sur 21,5 % de l'effectif avant toute comparaison de filets de clientèle ». Une ligne de date, une ligne de responsable, une ligne de revue.

4. Ce qui aurait changé la réponse : le panier par client et non par ticket (84 tickets multi-vendeurs, 103 lignes sans client utile), la marge nette au lieu du chiffre d'affaires (le prix de revient est hors de ce fichier), et l'écoulement du stock (sans lui, un gros mois peut être un transfert en magasin, pas une vente). Pour chacun : la donnée à demander, et le test qui en découlerait.

Total et décision d'orientation

Composante	Score	Seuil	Statut
B · Quiz	... /20	14	bloquant
C · Exercices	... /20	indicatif (auto-correction menée = +0,5)	—
D · Étude de cas	... /20	12	bloquant
Projet M02.P	... /20	13	bloquant

Décision. B et D au seuil, projet au seuil → module validé, passage en M03 (SQL : interroger la source). Quiz seulement en dessous → trois jours pour reprendre C02 et C05, puis une nouvelle version de B. Étude de cas seulement en dessous → nouvelle version de D sur le même dossier,

avec un entretien de 15 minutes sur la lecture des intervalles. Deux échecs → reprise du module, en commençant par les mini-projets P1 à P4.

Module M03 — Excel pour l'analyse de données

Phase 2 · Du classeur au modèle de données, sans total trompeur

Analyse de données appliquée & Business Intelligence
10 partie(s) assemblée(s) · document de production

Table des matières

MODULE M03.C01 — INTERFACE, CLASSEUR, FEUILLE, CELLULE : SE DÉPLACER SANS SE PERDRE	8
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 L'écran, et à quoi sert chaque bande	11
5.2 Naviguer : quatre touches qui remplacent mille clics	12
5.3 Sélectionner : la différence entre « ma colonne » et « la colonne »	12
5.4 Les trois visages d'une cellule : adresse, contenu, affichage	12
5.5 Ce que le tableur ne vous dira pas de lui-même	13
5.6 Préparer la fenêtre de travail : les cinq réglages de l'atelier	13
5.7 Le classeur de l'atelier, et sa feuille « Aidez-moi »	14
6. Exemple concret : la prise en main d'un fichier reçu, en onze minutes	14
7. Démonstration pas à pas : produire la fiche de prise en main	15
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — retrouver six nombres et leurs preuves (25 min, /10)	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — « La fiche de prise en main du classeur de pilotage » (45 min)	19
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 8 min)	20
MODULE M03.C02 — FORMATS ET SAISIE INTELLIGENTE : MONÉTAIRE FCFA, DATES, VALIDATION, MISE EN FORME CONDITIONNELLE	22
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 Ce que le tableur a compris, et pourquoi	24
5.2 Les quatre formats de la gestion	25
5.3 Les dates : un compteur de jours, pas une image	25
5.4 Convertir vraiment : trois voies, une preuve	25
5.5 Validation des données : interdire la prochaine erreur	26
5.6 Mise en forme conditionnelle : le détecteur qui ne dort pas	26
5.7 Les gestes de saisie qui font gagner une heure	27
6. Exemple concret : trois lignes saisies, quatre pièges refermés	27
7. Démonstration pas à pas : la colonne de contrôle de saisie	27

8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — formats, conversion, garde-fous (30 min, /10)	29
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — suite : le volet formats et garde-fous (40 min)	31
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 8 min)	20
 MODULE M03.C03 — LE TABLEAU STRUCTURÉ, LE TRI, LES FILTRES, LA RECHERCHE ET LES DOUBLONS	 33
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 Créer le tableau, et le nommer	35
5.2 Ce que la référence structurée change aux formules	36
5.3 Trier : la ligne est l'unité, pas la colonne	36
5.4 Filtrer, et vérifier le filtre	36
5.5 Rechercher et remplacer, avec ses jokers	37
5.6 Supprimer les doublons : la clé décide de tout	37
5.7 Ce que le tableau ne protège pas	38
6. Exemple concret : le même fichier, trois sorts	38
7. Démonstration pas à pas : de la plage filtrée au tableau qui grandit	39
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la table qui tient debout (35 min, /10)	40
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — suite : le volet structure (45 min)	42
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 10 min)	43
 MODULE M03.C04 — LA FORMULE : RÉFÉRENCES RELATIVES, ABSOLUES, MIXTES, ET LES SEPT ERREURS	 45
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 Les opérateurs, et l'ordre qui les range	47

5.2 Relative, absolue, mixte : ce qui bouge, ce qui reste	48
5.3 Ce que le fichier stocke, ce que vous lisez	48
5.4 Les sept erreurs, une cause chacune	48
5.5 Propager, attraper, compter : trois gestes différents	49
5.6 Arrondir : dans la valeur, ou dans l’affichage — mais pas les deux	49
5.7 Ce qui casse une référence, ce qui ne la casse pas	50
5.8 La règle de la fiche double, et les deux silences du tableur	51
6. Exemple concret : la ligne 2, de la saisie au total	51
7. Démonstration pas à pas : lire, reproduire, casser, réparer	52
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la chaîne de calcul, de la cellule au contrôle (35 min, /10)	53
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — suite : le volet formules et cohérence (45 min)	55
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 10 min)	43
MODULE M03.C05 — LA BOÎTE À FONCTIONS D’AGRÉGATION ET CONDITIONNELLE : SOMME, MOYENNE, NB, MEDIANE, SOMME.SI.ENS	58
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 Compter : quatre fonctions, quatre réponses	60
5.2 Les critères, ce langage minuscule	61
5.3 Le dénominateur est la notion	61
5.4 Agréger par catégorie sans tableau croisé — et le vérifier	62
5.5 La position et la dispersion : MEDIANE, QUARTILE.INC, et l’autre	63
5.6 Ce que le texte et les erreurs font aux nombres	63
5.7 Et les objectifs, dans tout ça	63
6. Exemple concret : la dispute des 3 pourcents	64
7. Démonstration pas à pas : la grille de contrôle par catégorie	64
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la grille qui se ferme (35 min, /10)	66
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — suite : le volet agrégats (45 min)	68
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 10 min)	43

MODULE M03.C06 — RECHERCHER ET ASSEMBLER : RECHERCHEV ET SES QUATRE PIÈGES, RECHERCHEX, INDEX/EQUIV, LES FONCTIONS DE TEXTE	71
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 La clé d'abord, la formule ensuite	73
5.2 RECHERCHEV : quatre pièges, quatre symptômes mesurés	74
5.3 RECHERCHEX : la recherche qui pose les bonnes questions	75
5.4 INDEX / EQUIV : la voie qui marche partout	76
5.5 Nettoyer la clé : SUPPRESpace, SUBSTITUE, CNUM	76
5.6 DECALER et INDIRECT : comprendre pour ne pas construire	77
5.7 Assembler : &, CONCAT, JOINDRE.TEXTE	77
6. Exemple concret : la jointure qui marchait trop bien	78
7. Démonstration pas à pas : la feuille de rapprochement	78
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la feuille client complétée (35 min, /10)	80
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — suite : le volet recherches (45 min)	82
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 10 min)	43
MODULE M03.C07 — DATES, DURÉES, PÉRIODES, PUIS LES FONCTIONS MATRI- CIELLES : FIN.MOIS, NB.JOURS.OUVRES, FILTRE, TRIERPAR, LET	85
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 Lire ce que la colonne date contient vraiment	87
5.2 Extraire le temps : la famille qui lit dans la série	88
5.3 Bornes, durées, DATEDIF	89
5.4 NB.JOURS.OUVRES : le dénominateur du temps travaillé	89
5.5 La grille mensuelle de l'atelier, démontée	90
5.6 Lisser : la moyenne glissante, et ce qu'elle cache	91
5.7 Les fonctions dynamiques, en fiches doubles	91
5.8 Cinq fiches de fonction du chapitre	92
6. Exemple concret : le décembre qui écrasait les autres	92
7. Démonstration pas à pas : la grille de rythme mensuel	93
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16

10. Exercice guidé — la grille de rythme (40 min, /10)	94
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — suite : le volet calendrier (45 min)	96
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 10 min)	43
 MODULE M03.C08 — CROISER, CHARGER, PROTÉGER : TABLEAUX CROISÉS DYNAMIQUES, SEGMENTS, POWER QUERY, MODÈLE DE DONNÉES, DAX DE SURVIE	 99
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple	9
4. Vocabulaire essentiel	9
5. Cours approfondi	11
5.1 Le TCD écrit à la main : ce qu'il calcule, ligne par ligne	101
5.2 Les quatre zones, et ce que l'actualisation change	102
5.3 Segments, chronologie, connexions : ce qui filtre quoi	103
5.4 Graphiques : le lié et le figé	103
5.5 Power Query : écrire la recette du nettoyage	103
5.6 Le modèle de données : quatre relations, zéro colonne recopiée	104
5.7 DAX de survie : trois mesures et deux contextes	105
5.8 Protéger, automatiser, et renoncer à la macro qui n'en est pas une	105
6. Exemple concret : « combien de clients par catégorie ? »	106
7. Démonstration pas à pas : de la brute au pilotage	106
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — le tableau de bord à trois segments (45 min, /10)	108
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M03.P1 — suite : le volet pilotage (50 min)	110
14. Résumé du chapitre	19
15. À retenir	20
16. Évaluation formative (auto-correction, 10 min)	43
 PROJET M03.P — « LE CLASSEUR DE PILOTAGE DE SAHEL DISTRIBUTION »	 113
1. Énoncé	113
Le matériel	113
Les six livrables	113
Contraintes de rendu	114
2. Barème détaillé	115
3. Correction pas à pas (corrigé enseignant)	115
3.1 — L1 : la préparation attendue, étape par étape	115
3.2 — L2 : ce que rendent les fiches exécutées dans Calculs	116
3.3 — L3 : le tableau de bord, avec ses nombres	116

3.4 — L4 : noms, formats, protection	117
3.5 — L5 : la note, et les trois phrases qui font la différence	117
3.6 — L6 : la rejouabilité, testée comme elle le sera	118
4. Grille d’auto-évaluation avant rendu	118
5. Ce que le correcteur regardera en premier	118
ÉVALUATION M03 — « EXCEL, DU CLIC À LA MÉTHODE »	120
A • Questions de récupération (non notées)	120
B • Quiz (15 questions • 15 points)	120
Bloc 1 — Types, formats, saisie (Q1 à Q4)	121
Bloc 2 — Adresses et erreurs affichées (Q5 à Q8)	121
Bloc 3 — Agrégats, critères, dates (Q9 à Q12)	121
Bloc 4 — Recherche, assemblage, TCD (Q13 à Q15)	121
C • Exercices pratiques (4 exercices • 20 points)	122
D • Étude de cas — « Le tableau de bord du magasin 5 est-il pilotable ? » (20 points, seuil 12) .	123
Éléments de réponse attendus	123
Corrigé du quiz (B)	124
Éléments de corrigé des exercices (C)	125
Total et décision d’orientation	126

Module M03.C01 — Interface, classeur, feuille, cellule : se déplacer sans se perdre

Outil de ce chapitre : le tableur seul (Excel 365 / 2021 ou LibreOffice Calc), sur le classeur de l'atelier. Durée indicative : 3 h. Niveau : N1.

L'idée du chapitre. Un tableur n'est pas une calculatrice géante : c'est une **grille de cellules adressées**, où chaque nombre que vous lisez est le rendu d'un contenu, lui-même situé quelque part. Les trois quarts des accidents de l'analyse sur tableur ne viennent pas de formules fausses mais d'une méconnaissance de l'écran : on a sommé une colonne qui contenait du texte, trié une colonne seule, lu un total dans une fenêtre filtrée — et l'on a écrit le résultat dans un courriel. Ce chapitre installe les six réflexes qui rendent les sept suivants sûrs.

Base de travail — deux chemins pour une seule donnée. Quatre fichiers, produits par le socle du manuel : le classeur d'atelier `01_socle_donnees/data/projection/m03_classeur_atelier.xlsx` ; les ventes **telles que reçues**, `01_socle_donnees/data/projection/ventes_magasin5_2025.csv` (489 lignes, défauts compris) ; la même donnée **nettoyée**, `01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv` (480 lignes) ; les objectifs mensuels, `01_socle_donnees/data/brut/objectifs_de_ca.csv` (218 lignes). La population entière (`01_socle_donnees/data/reference/ventes_propres.csv`, 240 000 lignes) s'ouvre en C08. Chez vous : `donnees/projection/` et `donnees/reference/`. **Les blocs des huit chapitres écrivent le chemin de l'atelier, parce qu'il s'exécute sans préparation** ; dans votre dossier, remplacez le préfixe `01_socle_donnees/data/` par `donnees/` et ne changez rien d'autre. Séparateur point-virgule, UTF-8, franc CFA (FCFA), virgule décimale au tableur. Les chiffres cités sont dans `01_socle_donnees/data/reference/chiffres_cites.md`, section M03. Le classeur est livré **sans protection** : il doit être ouvert, modifié, cassé et réparé.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Nommer** les quatre niveaux d'un tableur — classeur, feuille, plage, cellule — et écrire l'adresse d'une cellule dans la forme que le logiciel comprend.
2. **Aller** n'importe où dans une feuille de 489 lignes en trois raccourcis, sans molette ni barre de défilement.
3. **Sélectionner** exactement la zone voulue et **lire dans la barre d'état** le compte, la moyenne et la somme, sans écrire une formule.
4. **Distinguer** les trois visages d'une cellule — adresse, contenu, affichage — et reconnaître, au compteur, un montant stocké en texte plutôt qu'en nombre.
5. **Déclarer** la santé d'un fichier avant de le travailler : dernière cellule, lignes vides, colonnes non reconnues — la fiche de prise en main qui ouvrira tous vos classeurs professionnels.

2. Pourquoi cette notion est importante

Situation 1 — le total qui manque 36 millions. Vous ouvrez la feuille `ventes_brutes` du classeur, vous cliquez sur la lettre M de la colonne `montant_ttc`, vous lisez le bas de l'écran : Nombre 489, Nombre de valeurs numériques 0, Somme 0. Un tableur qui compte 489 entrées sans savoir

en additionner une seule n'est pas cassé : il énonce que **la colonne est du texte**. Ouvrez maintenant le fichier reçu d'un double-clic : l'import automatique, lui, convertit trois cellules, et la barre affiche « 3 » et « 2 444 ». **Deux chiffres pour un même fichier, aucun message d'erreur**. La bonne somme, 36 073 185 FCFA, se lit sur la feuille nettoyée de 480 lignes.

Situation 2 — le tri qui a découpé les lignes. Un analyste veut « voir les plus gros montants ». Il sélectionne la colonne des montants, trie, et clique « Trier la sélection » pour aller vite. Le total n'a pas bougé d'un centime : 36 073 185 FCFA avant, 36 073 185 FCFA après. En revanche, la ligne qui affichait 1 229 678 FCFA — le plus gros montant, en M356 — ne raconte plus rien de vrai : sa date, son produit et son client sont ceux d'une autre vente. **Le contrôle du total est aveugle à ce dégât** ; seul le contrôle à la ligne le voit.

Situation 3 — la réunion où personne ne sait de quoi on parle. « Regardez la dernière colonne, à droite, en bas. » Pendant onze secondes, dix personnes cherchent. « Feuille Calculs, cellule M490 » prend une seconde et ne se prête à aucune interprétation. Un analyste qui sait adresser conduit la réunion ; celui qui dit « là, vers la droite » la subit. Cette aisance n'a rien d'un don : un vocabulaire et quatre raccourcis.

Dans les faits. Dans les audits de fichiers de gestion, la première faute trouvée n'est ni une formule fausse ni un modèle inadapté : c'est un classeur dont personne ne sait dire ce qu'il contient, où, et depuis quand il a été vérifié. Un onglet de prise en main, trois lignes datées, tue ces discussions — et il se rédige en cinq minutes.

3. Explication simple

Un tableur est une **armoire à cases**. Le **classeur** (workbook) est l'armoire : un fichier, un nom, une date. Une **feuille** est un tiroir : la donnée ne vit jamais « dans le fichier », elle vit dans une feuille. Une **cellule** est une case : elle a une **adresse** — colonne puis ligne, M356 — un **contenu**, et un **format**. Une **plage** est un rectangle de cases, noté I2:M490. Enfin, une **table** est une plage à qui l'on a donné un nom et une structure : elle s'étend toute seule quand on ajoute des lignes (chapitre C03).

Autre image, celle qui fait comprendre l'adresse : une salle de cinéma. M356, c'est « rangée M, siège 356 ». Quand vous recopiez une formule d'un siège à l'autre, le logiciel **translate** l'adresse ; si vous voulez que tout le monde regarde le même écran, vous **figez** l'adresse avec \$. Tout le chapitre C04 tient dans cette phrase.

Et trois visages pour une même case. Prenez M2 de ventes_brutes : son adresse est M2, son contenu est le texte 56 658 (onze caractères, dont une espace), son affichage est 56 658 avec une espace fine. Le nombre 56658, lui, se colle à droite. **L'alignement est un diagnostic gratuit** : à gauche texte, à droite nombre.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Classeur — workbook	Le fichier. Il peut contenir vingt feuilles et trois tables.	Croire qu'un classeur égale une donnée : c'est une feuille , ou une table , qui la porte.
Feuille — sheet	Une grille de cases, nommée par son onglet.	Chercher une valeur « dans le fichier » au lieu de dire dans quelle feuille.
Cellule — cell	La plus petite unité adressable : une adresse, un contenu, un format.	Confondre le contenu et ce qui s'affiche.

Français — English	Définition simple	Piège à éviter
Plage — range	Un rectangle de cellules, noté I2:M490.	Sélectionner au clic-glissé et s'arrêter trois lignes avant la fin.
Adresse absolue / relative — absolute / relative reference	\$M\$2 ne bouge pas quand on recopie ; M2 bouge.	Recopier une recherche sans figer la plage : le RECHERCHEV décroche ligne après ligne.
Zone de nom — name box	La case à gauche de la barre de formule : elle affiche l'adresse et accepte une adresse en entrée .	Ne jamais s'en servir pour naviguer, alors qu'elle va plus vite que trente clics.
Barre de formule — formula bar	Ce que la cellule contient, avant formatage.	Contrôler dans la grille : la grille ment par le format.
Plage nommée — named range	Une plage à qui l'on donne un nom, utilisable dans les formules.	Nommer Données, puis insérer des lignes hors de la plage : le nom ne suit pas.
Volets figés — frozen panes	Lignes ou colonnes qui restent visibles au défilement.	Figer sur la mauvaise cellule, et perdre l'en-tête quand même.
Mise en forme — formatting	Couleur, police, décimales, format monétaire.	Confondre « nettoyé » et « reformaté » : le format ne change jamais la valeur.

Définition. Une **cellule active** est la seule cellule où l'on tape : elle est encadrée d'un liseré épais et son adresse est écrite dans la zone de nom. Une **sélection** est un rectangle de cellules mises en évidence dont une seule est active. La distinction est technique : Ctrl + C copie la sélection, Entrée ne valide que dans l'active, et la barre d'état résume **toute** la sélection.

Définition. On dit qu'une cellule est **reconnue comme nombre** quand le tableur y lit un contenu chiffré, manipulable par SOMME(), triable, moyennable. Un contenu **reconnu comme texte** — y compris 1 229 678 écrit avec des espaces — n'entre dans aucun calcul : SOMME() l'ignore, ce qui est la plus silencieuse des façons de se tromper.

5. Cours approfondi

5.1 L'écran, et à quoi sert chaque bande

m03_classeur_atelier.xlsx

Accueil Insertion Formules Données Affichage

Presse-papiers Police Alignement Nombre Styles Édition

M2

fx 56 658

	K	L	M
1	remise	montant_ht	montant_ttc
2	0.03	48015	56 658
3	0.0	43900	51 802

la même valeur dans la feuille nettoyée

ventes!M2 : un nombre

56 658 FCFA

Feuille ventes_brutes : 489 lignes × 13 colonnes, et pas un seul montant reconnu comme nombre — d'où « Somme : 0 ». Un texte se colle à gauche, un nombre à droite : l'alignement est un diagnostic, pas un style.

Aidez-moi

ventes_brutes

ventes

clients

objectifs

Calculs

TCD

Prêt Nombre : 489 valeurs numériques : 0 Somme : 0

Une cellule, trois visages

Son adresse

M2 — lue dans la zone de nom

Son contenu

du texte : « 56 658 », pas 56658

Son affichage

56 658, collé à gauche

le format ne change pas le fond

Ce que la barre d'état dit de la colonne M entière :

489 textes · Somme : 0

un double-clic sur le CSV en convertit 3 : 2 444 FCFA

la feuille ventes, elle : 480 lignes · 36 073 185

Sélectionnez la colonne M en entier (Ctrl+Maj+Bas) et lisez le bas de l'écran :
aucun message d'erreur n'apparaîtra — c'est précisément le problème.

En haut, la **barre de titre** porte le nom du fichier : c'est là que l'on vérifie, avant toute manipulation, que l'on travaille dans la bonne version du bon classeur. Le **ruban** étale les onglets d'outils ; dans ce module, vous vivrez dans quatre d'entre eux — Accueil (formats, tri, trouver), Données (tables, filtres, import, tableaux croisés), Formules (audits, noms), Affichage (volets, grille). Un **ruban contextuel** n'apparaît que si un objet précis est sélectionné : son absence est déjà une information.

Sous le ruban, deux cases commandent tout le reste. La **zone de nom** affiche l'adresse de la cellule active ou le nom de la sélection, et c'est une **ligne de commande de navigation** : tapez M356, Entrée, vous y êtes. La **barre de formule** affiche le contenu : c'est le seul endroit de l'écran où le formatage ne peut pas vous mentir.

En bas, la **barre d'état** est l'outil le plus sous-estimé du tableur. Clic droit dessus (Excel) ou Affichage → Barre d'état (Calc) : on y allume Moyenne, Somme, Min, Max, **Nombre** (toutes les cellules non vides, comme NBVAL) et **Nombre de valeurs numériques** (les seules calculables, comme NB) ; dans LibreOffice Calc, les deux derniers s'appellent Valeur et Valeur numérique. C'est l'**écart** entre les deux compteurs qui parle. Sur `montant_ttc` de la feuille `ventes`, la barre rend « Moyenne : 75 152 · Nombre : 480 · Somme : 36 073 185 » ; sur la même colonne de `ventes_brutes`, « Nombre : 489 · valeurs numériques : 0 · Somme : 0 ». Quatorze mots, trois secondes.

Conseil professionnel. Trois zones de l'écran, trois usages : la **zone de nom** pour dire où l'on va, la **barre de formule** pour savoir ce que c'est, la **barre d'état** pour savoir combien il y en a. Tout le reste est optionnel, y compris la souris : un analyste qui cherche un bouton pour naviguer a perdu trois secondes.

5.2 Naviguer : quatre touches qui remplacent mille clics

Le principe est unique : **une flèche saute à la prochaine cellule non vide dans sa direction**. Donc **Ctrl + ↓** pour aller au bas d'une colonne pleine, **Ctrl + ↑** pour en revenir, **Ctrl + Fin** pour le coin utilisé du fichier — utilisé, pas rempli : c'est la nuance qui fait tout.

Ce que je veux	Touches	Ce que j'obtiens, ici
La dernière cellule de ventes_brutes	Ctrl + Fin	M490 — 489 lignes de données, 13 colonnes
La dernière cellule de la feuille de travail ventes	Ctrl + Fin	N481 — 480 lignes, et 14 colonnes : la colonne utilitaire N s'ajoute
La ligne du plus gros montant	Données → Trier, puis lecture	1 229 678 FCFA en M356 sur ventes — le même montant, en texte, en M363 sur la brute

On navigue aussi par la zone de nom (une adresse, un nom de plage, et l'on y va) et par **F5** — Atteindre — qui liste les destinations nommées : après deux semaines d'atelier, c'est votre plan de salle. La limite d'une feuille de calcul du commerce est voisine d'un million de lignes ; une feuille qui dépasse s'ouvre amputée, parfois sans message. **On compte les lignes avant de conclure**, jamais après.

Définition. La **zone utilisée** (used range) est le plus petit rectangle contenant toutes les cellules ayant un contenu **ou** une mise en forme. C'est elle que **Ctrl + Fin** atteint, non la zone des données : un fond de couleur oublié en **T412** suffit à l'étendre de trois cents lignes, et le compte que vous annoncez devient faux sans qu'une valeur ait bougé.

Attention. Le contrôle est simple : **Ctrl + ↓** depuis une cellule peuplée, puis comparer le numéro de ligne à celui de **Ctrl + Fin**. Sur le classeur de l'atelier, les deux chemins concordent — la zone utilisée s'arrête où s'arrête la donnée. Sur vos fichiers, ils ne concorderont pas toujours.

5.3 Sélectionner : la différence entre « ma colonne » et « la colonne »

Trois gestes, trois résultats, et le troisième est celui qui compte.

1. **Clic sur la lettre M** : toute la colonne, un million de cases, vides compris. La barre d'état, elle, ne compte que le plein — mais un collage, lui, ira jusqu'en bas.
2. **Ctrl + A dans une plage de données** : le bloc contigu — ici A1:N481 sur ventes. Rapide, mais contigu : une ligne vide à 128 et la sélection s'arrête en 127.
3. **Ctrl + Maj + ↓, puis Ctrl + Maj + →** : la sélection **informée**, qui s'arrête aux vides que l'on voit. Quand elle cale, ce n'est pas la sélection qui est fausse, c'est le fichier qui a un trou.

Vérifier une sélection se lit à trois endroits : le compteur de la barre d'état, la zone de nom (dimensions de la sélection), et la ligne courante indiquée par la barre de défilement. Sur ventes, la sélection de **montant_ttc** doit annoncer 480 ; 478 veut dire deux lignes coupées.

5.4 Les trois visages d'une cellule : adresse, contenu, affichage

C'est le cœur conceptuel du module, et il tient en une manipulation. Ouvrez **ventes_brutes**, cliquez **M2** :

- **adresse** M2, lue dans la zone de nom ;
- **contenu** 56 658, lu dans la barre de formule — onze caractères, dont une espace, sans signe égal : **du texte** ;
- **affichage** 56 658 collé à gauche (selon la version, un triangle vert en coin signale « nombre stocké comme du texte »).

Cliquez **M88** : le contenu est 797, sans espace. Dans le classeur, cette cellule non plus n'est un nombre — mais c'est l'une des trois que l'import automatique convertira, avec **M179** (821) et **M236** (826), soit 2 444 FCFA à elles trois. La colonne voisine enseigne le contraire, et c'est utile : **montant_ht**, sur les mêmes 489 lignes, ne contient aucun séparateur de milliers, et s'importe donc **en nombres** sans que personne n'ait rien décidé. Même feuille, deux colonnes, deux sorts : la nature d'une donnée ne dépend pas de vous, elle dépend du fichier — d'où la règle de l'import assisté.

Définition. Une **conversion** de types change la nature d'un contenu ; une **mise en forme** change son rendu. Les deux mots sont séparés par tout ce que peut contenir une erreur d'analyse : 56 658 en texte sous un format monétaire reste du texte, et la somme reste à 0.

5.5 Ce que le tableur ne vous dira pas de lui-même

Le tableur affiche. Il n'avertit presque jamais. Cinq silences à connaître dès aujourd'hui.

1. **Un texte ignoré par une somme ne lève aucune erreur.** **SOMME()** fait son travail sur ce qu'elle peut.
2. **Un format monétaire ne convertit pas.** Appliquer le format FCFA à **montant_ttc** de **ventes_brutes** laisse les 489 valeurs en texte : la colonne s'embellit, la somme reste à 0.
3. **Un format de date n'est pas une date.** La colonne **date** du fichier reçu mélange deux écritures : 424 lignes en ISO (2025-01-01) et 65 en notation française (01/01/2025). Un import à l'aveugle en convertit une partie et laisse l'autre en texte, qui se triera alphabétiquement — le 05 janvier avant le 1er décembre. La colonne **remise**, elle, est écrite avec un point décimal (0.03) : dans un tableur réglé sur le Burkina ou la France, ce n'est pas un nombre.
4. **Un filtre actif modifie le total que vous lisez à l'écran**, pas le résultat d'une formule bien écrite. Les deux nombres divergent, et c'est l'un des deux que l'on envoie au directeur.
5. **Une cellule vide n'est pas une cellule à zéro.** Dans la donnée nettoyée, 18 cellules sont vides — les 18 lignes où le champ **client** n'a pas été saisi — et elles se distinguent des 85 lignes où le client est le 0 du comptoir : le zéro est une information, le vide en est une autre. Sept des 13 colonnes restent du texte dans le fichier plat (identifiants et libellés) : le typage est un acte d'analyse, pas un réglage d'affichage.

Attention. Le pire accident du chapitre n'est pas une erreur, c'est une erreur **invisible**. Trier une seule colonne casse l'alignement des lignes sans casser aucun total. Le contrôle qui le voit est à la ligne : après tout tri, **M356** doit encore valoir 1 229 678 FCFA **et** porter la même vente que sa formule de la colonne **N**. Vous rendrez ce contrôle automatique en **C05**.

5.6 Préparer la fenêtre de travail : les cinq réglages de l'atelier

Trois minutes, une fois par machine. Ils ne changent aucun résultat — ils empêchent d'en changer un par accident.

1. **Barre d'état** : allumer Moyenne, Somme, Min, Max, Nombre, Nombre de valeurs numériques. Sans eux, la navigation est aveugle.
2. **Affichage → Figer les volets → Figer la ligne supérieure.** Sur 489 lignes, un en-tête qui disparaît au défilement est une porte ouverte à la colonne mal lue.
3. **Quadrillage affiché, zoom à 100 %, colonnes ajustées** (Accueil → Format → Ajuster la largeur) : une date tronquée en 05/01 ressemble à une date et se croit lue.
4. **Barre d'accès rapide** : y faire monter Atteindre une cellule..., Somme automatique, Figer les volets, Supprimer les doublons. Quatre boutons, quarante navigations économisées par jour.

5. **Enregistrer sous, daté** : `m03_classeur_atelier_YYYY-MM-DD_a0.xlsx`, et l'on **n'écrase jamais** la version `a0`. Vingt modules plus tard, cette habitude vous aura sauvé plus de temps que n'importe quelle formule.

Dans les faits. Ces réglages ne sont pas du confort : figer l'en-tête et allumer les deux compteurs sont les deux seules mesures qui rendent **visible** une ligne manquante ou une colonne texte. Dans un service où trois personnes reprennent le même classeur, c'est la différence entre un fichier qui se transmet et un fichier qui se rouvre.

5.7 Le classeur de l'atelier, et sa feuille « Aidez-moi »

Le classeur contient onze feuilles, dans cet ordre : `Aidez-moi`, `ventes_brutes`, `ventes`, `produits`, `clients`, `magasins`, `vendeurs`, `objectifs`, `Calculs`, `TCD`, `Mensuel`. Trois choses à savoir dès aujourd'hui.

`ventes_brutes` est **le fichier tel qu'il est reçu** : 489 lignes, tout en texte, défauts compris. On ne la nettoie jamais : on la garde comme preuve. `ventes` est la même donnée **nettoyée** : 480 lignes, types corrigés, table structurée, et une colonne utilitaire `N` qui recalcule le montant de chaque ligne ; les 9 lignes d'écart sont exactement les doublons du fichier brut (266 132 FCFA, soit 0,74 % du chiffre d'affaires de l'extrait). `Aidez-moi` donne la fiche du classeur et vingt-huit fiches de formules.

Les 154 produits, les 372 clients rencontrés (371 au référentiel ; 103 lignes sans client trouvé : 85 comptoir codé 0, 18 champ vide) et les 218 lignes d'objectifs seront les objets de C03, C05, C06 et C07. En C01, une seule question les concerne : **où sont-elles, et combien sont-elles ?**

Boîte à outils. Ce que le chapitre laisse de permanent : `Ctrl + Fin` (zone utilisée), `Ctrl + ↑/↓` (bords de bloc), `Ctrl + Maj + ↓` (sélection informée), `F5` (atteindre une plage nommée), clic sur la lettre de colonne plus barre d'état (compte et somme sans formule), `Alt` puis une lettre (naviguer dans le ruban : `F` pour Fichier, `H` pour Accueil, `A` pour Données). Dans LibreOffice Calc, les trois premiers sont identiques, la barre d'état se règle par Affichage → Barre d'état.

6. Exemple concret : la prise en main d'un fichier reçu, en onze minutes

Vous recevez par courriel le fichier de ventes. Voici la séquence complète, celle que vous répétez sur chaque fichier du reste du parcours — onze minutes, six nombres notés.

Étape	Geste	Ce que l'on note	Ce que l'on décide
1	Ouvrir par Données → À partir de texte/CSV (; , UTF-8, colonnes forcées en texte), puis rouvrir d'un double-clic pour comparer	489 lignes, 13 colonnes dans les deux cas ; au second, 3 montants deviennent nombres	on ne juge pas un fichier sur un import automatique
2	<code>Ctrl + Fin</code>	dernière cellule M490	la zone utilisée s'arrête où s'arrête la donnée
3	Clic sur la lettre <code>M</code> , barre d'état	Nombre 489 · valeurs numériques 0 · Somme 0	les montants ne sont pas des nombres : nettoyage obligatoire
4	Clic sur la lettre <code>L</code> (montant_ht)	489 cellules, toutes additionnables	le défaut est une colonne, pas le fichier entier
5	Clic sur <code>M</code> de la feuille nettoyée <code>ventes</code>	Moyenne 75 152 · Nombre 480 · Somme 36 073 185	c'est ce chiffre qui sera cité, et sa source : la feuille, pas le fichier reçu

Étape	Geste	Ce que l'on note	Ce que l'on décide
6	Écrire les six lignes dans Aidez-moi	date, source, dimensions, bornes, contrôles	la prise en main est finie, le travail commence

Le nombre de la ligne 5 est celui que tous les contrôles du module retrouveront : **36 073 185 FCFA**. Sur le fichier reçu nettoyé à la main mais sans retirer les doublons, le même périmètre donne 36 339 317 FCFA ; les 266 132 FCFA d'écart sont exactement les 9 doublons. Deux personnes qui lisent ce fichier à l'aveugle obtiennent donc **deux vrais chiffres**, et le seul moyen de les mettre d'accord est la fiche de l'étape 6.

7. Démonstration pas à pas : produire la fiche de prise en main

Écrivez ces cinq rubriques dans la feuille *Aidez-moi*, zone « Prise en main du jour ». C'est le modèle que vous appliquerez à chaque fichier de votre carrière.

1. Le nom et la date. Barre de titre : `m03_classeur_atelier.xlsx`, ouvert le 18/09/2026, aucune protection d'onglet. Cette ligne, qui semble triviale, est celle qui manque dans les deux tiers des classeurs reçus en révision.

2. Les dimensions. Ctrl + Fin sur `ventes_brutes` → M490, soit 489 lignes de données sur 13 colonnes. Sur `ventes` → N481 : 480 lignes, et 14 colonnes parce que la colonne utilitaire s'ajoute ; le fichier plat nettoyé, lui, s'arrête en M481. On écrit les trois, avec l'écart et sa cause : 9 lignes, les doublons, 266 132 FCFA.

3. La nature des colonnes. Colonne par colonne, clic sur la lettre, barre d'état. Sur `ventes` : les montants sont des nombres (480 dénombrés, 36 073 185 FCFA), `date` est une date, `quantite` va de -13 à 68, `remise` s'affiche de 0 % à 11 %. Sur `ventes_brutes` : les 489 montants TTC sont du texte, la colonne `date` mélange deux écritures, une quantité monte à 14 000, les remises portent un point décimal. On écrit la liste des colonnes qui **ne passent pas**, et rien d'autre.

4. Les bornes de la période. Sélection de `date` sur la feuille nettoyée, tri **avec extension** : première ligne 01/01/2025, dernière 02/12/2025 — 335 jours d'amplitude, 31 jours distincts d'activité, 240 jours ouverts sur l'intervalle. On note les deux dates : ce sera la première question du commanditaire.

5. Le contrôle à la ligne. En M356 de `ventes`, lisez 1 229 678 FCFA ; sur la même ligne, en N356, lisez le montant recalculé par la colonne utilitaire, `=ARRONDI(I356*J356*(1-K356);0)`. Les deux doivent être égaux. Notez que sur l'ensemble de la feuille, l'écart total entre montants enregistrés et recalculés est de 4 FCFA, né de six lignes dont le produit tombe exactement à un demi-franc : c'est le bruit d'arrondi normal d'un fichier de caisse, pas une faute.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Ctrl + Fin sur <code>ventes_brutes</code>	adresse exacte	M490	filtre resté actif, ou onglet mal ouvert
2	Ctrl + Fin sur <code>ventes</code>	adresse exacte	N481	vous êtes encore sur la feuille brute
3	Barre d'état sur <code>montant_ttc</code> de la brute	trois nombres	489 · 0 · 0 (et 3 · 2 444 après double-clic)	compteur éteint, ou import automatique mal lu
4	Barre d'état sur <code>montant_ttc</code> de <code>ventes</code>	trois nombres	480 · 36 073 185 · 75 152	sélection arrêtée au premier vide
5	Plus gros montant lu à la ligne	identité M356 / N356	1 229 678 FCFA des deux côtés	tri sans extension : repartir de la copie

8. Erreurs fréquentes

1. **Double-cliquer sur un .csv pour l'ouvrir.** Le tableur devine les types, se trompe sur les codes (un client 007 devient 7), et l'on découvre deux semaines plus tard que la fusion avec le référentiel n'a rien trouvé.
2. **Lire la grille au lieu de la barre de formule.** 56 658 dans la grille est une image ; le contenu est soit le nombre, soit le texte, et rien ne le dit sauf l'alignement et le compteur.
3. **Trier une colonne isolée.** Le tableur le permet, avec un avertissement que l'on clique sans lire. La conséquence n'est pas un mauvais total : c'est une **mauvaise ligne**, donc une mauvaise décision sur un bon chiffre.
4. **Compter les lignes du tableur comme celles de la donnée.** L'en-tête est une ligne : 489 lignes de données tiennent en M490. Écrivez « 489 lignes, 13 colonnes, plus l'en-tête », sinon le relecteur ne retrouve pas le compte.
5. **Confondre les deux compteurs de la barre d'état.** Nombre compte le non-vide (NBVAL), Nombre de valeurs numériques compte le calculable (NB). Sur la colonne client de la feuille nettoyée, les deux ne concordent pas : 462 et 480, parce que 18 lignes n'ont pas de client saisi et que le 0 du comptoir, lui, est une valeur.

Attention. Le raccourci qui casse tout dans ce module n'existe pas : c'est **Entrée** dans la cellule juste au dessous d'une colonne de nombres, suivi de Alt + =. La somme automatique propose une plage, souvent « de la première ligne à la ligne du dessus » ; si vous avez inséré une ligne de totaux au milieu, vous summez une partie du fichier, le total est faux de exactement ce que vous avez oublié, et rien ne clignote. Regardez la plage qui clignote avant de valider : c'est l'erreur numéro un des classeurs de gestion.

9. Bonnes pratiques professionnelles

1. **La fiche de prise en main d'abord, à chaque fichier.** Cinq rubriques, une page, rédigée pendant que le logiciel charge.
2. **Ne jamais nettoyer la preuve.** On duplique (ventes_brutes → ventes), on nettoie la copie, et l'on écrit dans la copie ce que l'on a retiré et pourquoi. Un classeur sans feuille brute est un classeur dont on ne peut défendre les chiffres.
3. **Nommer avant de calculer.** Une plage nommée rend les formules lisibles et survit mieux aux insertions qu'une adresse recopiée vingt fois : la nommature est le premier acte de gouvernance d'un classeur.
4. **Contrôler à deux niveaux : le total, et une ligne.** Total 36 073 185 FCFA ; ligne M356 = 1 229 678 FCFA avec sa formule. Le premier voit les lignes perdues, le second voit les lignes mélangées.
5. **Un seul type par colonne, et c'est écrit.** Soit la colonne est numérique et les manquants sont vides, soit elle est texte et on ne la somme pas. Le tableur pardonne le mélange ; les chiffres jamais.

Conseil professionnel. Faites lire votre fiche de prise en main par quelqu'un qui n'a pas ouvert le fichier. Si cette personne retrouve les cinq nombres sans vous interroger, votre classeur est documenté. Sinon, ce n'est pas un problème de lecture : c'est un problème de nommage, d'ordre ou de feuilles.

10. Exercice guidé — retrouver six nombres et leurs preuves (25 min, /10)

Commande. Dans le classeur de l'atelier, retrouvez les six nombres ci-dessous en n'utilisant **ni formule, ni Python, ni SQL** : uniquement navigation, sélection, barre d'état, barre de formule. Pour chacun, écrivez la feuille, la plage, et le geste.

#	Ce qu'il faut obtenir	Feuille	Geste attendu
1	Nombre de lignes de données	ventes_brutes	Ctrl + Fin, lire le numéro de ligne, retirer l'en-tête
2	Somme de montant_ttc, puis compte des cellules calculables	ventes_brutes	clic sur M, barre d'état
3	Somme et moyenne de montant_ttc	ventes	clic sur M, barre d'état
4	Plus gros montant, et son adresse	ventes	Données → Trier avec extension, puis lecture de la zone de nom
5	Ce que contient vraiment M2	ventes_brutes	clic sur M2, barre de formule, alignement
6	Les deux écritures de la colonne date	ventes_brutes	filtre sur une valeur, ou tri, ou lecture de dix lignes

Démarrage, pas à pas. Pour 1 : une cellule peuplée de ventes_brutes, Ctrl + Fin, la zone de nom affiche M490 ; vous lisez **ligne 490, donc 489 lignes de données**, et vous écrivez la phrase avec la soustraction apparente. Pour 2 : clic sur M, et le bas de la fenêtre doit rendre **Somme : 0** avec **Nombre : 489** — 489 cellules pleines, aucune calculable ; rubriques absentes, clic droit sur la barre et cochez-les. Ouvrez le CSV reçu d'un double-clic, refaites le geste : **3 et 2 444**. Écrivez la phrase du courriel : les montants sont du **texte**, l'import en convertit trois, et aucun des deux chiffres n'est le chiffre d'affaires.

Pour 3 à 5 : allez sur ventes, relevez **480 lignes, 36 073 185 FCFA, 75 152 FCFA** de moyenne, un maximum de **1 229 678 FCFA** en M356 — que la feuille brute porte, elle, en M363 et en texte, parce que sept des neuf doublons le précèdent. Contrôlez que $489 - 480 = 9$ est bien le compte des doublons et que $36\,339\,317 - 36\,073\,185 = 266\,132$ FCFA en est bien le montant. Pour 5, la phrase attendue : « la première valeur de la colonne est le texte 56 658, et non le nombre 56658 ; un texte ne se somme pas ».

Barème (10 points). Six lignes renseignées, feuille et geste notés (4) · les cinq nombres exacts (3) · les deux phrases d'interprétation, texte contre nombre et 9 lignes de doublons (2) · noms de feuilles sans faute (1).

Ce qu'il faut retenir de cet exercice : il ne contient aucun calcul, et il produit le résultat le plus précieux du module — un état des lieux chiffré du fichier, obtenu en trois minutes, défendable ligne par ligne.

11. Exercices autonomes

Exercice 1.1 (★) — Les deux fins du fichier. Notez la dernière cellule de ventes_brutes et celle de ventes, puis le nombre de lignes de données de chacune. Combien de lignes et combien de cellules séparent les deux fichiers, et pourquoi ? Écrivez la phrase qui explique l'écart, en citant le montant et le pourcentage qui vont avec.

Exercice 1.2 (★) — Trois visages, quatre cellules. Pour M2, M88, M179 et M236 de ventes_brutes, remplissez un tableau à quatre colonnes : adresse, contenu lu dans la barre de formule, alignement, nature dans le classeur. Ajoutez une cinquième colonne : « ce qu'en fait un double-clic sur le CSV ». Concluez en une phrase.

Exercice 1.3 (★) — Le compte sans formule. Sur `ventes`, notez pour `quantite`, `prix_unitaire_ht`, `remise`, `montant_ht` et `montant_ttc` les rubriques que la barre d'état accepte d'afficher. Vous devez obtenir quatre nombres sur `montant_ttc`. Contrôlez la moyenne par un calcul de tête, puis vérifiez par `=SOMME(M2:M481)`. Une colonne doit se résumer par Min et Max mais refuser toute interprétation de sa somme : laquelle, et pourquoi ?

Exercice 1.4 (★★) — Le tri qui ne se voit pas. Sur une **copie** de la feuille `ventes` (clic droit sur l'onglet → Déplacer ou copier, cochez Créer une copie), sélectionnez `montant_ttc` seule et triez du plus grand au plus petit en choisissant « Trier la sélection ». Répondez : (a) la somme de la colonne a-t-elle changé ? (b) M356 vaut-elle encore le plus gros montant ? (c) que reste-t-il de vrai dans la ligne 356 ? Annulez jusqu'à retrouver 36 073 185 FCFA et 1 229 678 FCFA en M356.

Exercice 1.5 (★★) — Importer sans se faire avoir. Fermez le classeur. Réimportez `01_socle_donnees/data/projection/ventes_magasin5_2025.csv` dans un classeur neuf par Données → À partir de texte/CSV (; , UTF-8, colonnes forcées en texte). Notez : (a) lignes et colonnes chargées ; (b) ce que devient la colonne `date` si l'on laisse l'import deviner, comparé au forçage en texte ; (c) combien de lignes dépassent 500 en quantité dans le fichier reçu, et ce que vaut la borne maximale après nettoyage. Terminez par la fiche du §7, en citant vos deux chemins d'import comme deux sources de chiffres différents.

12. Correction détaillée

Exercice 1.1. `ventes_brutes` : M490, soit 489 lignes de données et 13 colonnes. La feuille `ventes` : M481, soit 480 lignes et 14 colonnes, la colonne utilitaire s'ajoutant aux 13 du fichier — le fichier plat nettoyé, lui, s'arrête en M481. Écart : 9 lignes, soit $9 \times 13 = 117$ cellules, et 266 132 FCFA : ce sont les doublons exacts du fichier reçu, 0,74 % du chiffre d'affaires de l'extrait. Trouver 480 lignes sur la feuille brute signifie un filtre resté actif.

Exercice 1.2. Dans le classeur, les quatre cellules sont du texte : M2 contient 56 658 (onze caractères, dont une espace), M88 contient 797, M179 contient 821, M236 contient 826 — les trois dernières sans séparateur de milliers. Alignement : les quatre à gauche. Ce qu'en fait un double-clic : M2 reste du texte, les trois autres deviennent des nombres, et la somme de colonne passe de 0 à 2 444 FCFA. Conclusion attendue : « le compteur est à 0 parce que la feuille est intégralement en texte, et à 3 après import parce que trois cellules seulement s'écrivent sans séparateur ; dans les deux cas `SOMME()` ignore le reste sans un mot ».

Exercice 1.3. Sur `montant_ttc` de `ventes` : 480 cellules, moyenne 75 152 FCFA, somme 36 073 185 FCFA, maximum 1 229 678 FCFA, minimum -150 804 FCFA (les retours). Le calcul de tête — 480 multipliés par la moyenne affichée — rend 225 FCFA de moins que le total, parce que la moyenne est arrondie à l'unité : un arrondi, pas une faute. `=SOMME(M2:M481)` doit rendre 36 073 185 FCFA exactement ; un franc d'écart en plus ou en moins signifie une plage lue sur une autre feuille ou une ligne perdue. La colonne qui refuse toute interprétation de sa somme est `quantite` : additionner 480 nombres d'unités hétérogènes, de -13 à 68, ne produit aucun indicateur lisible.

Exercice 1.4. (a) Non : la somme est restée 36 073 185 FCFA, le tri n'ayant fait que répartir les mêmes valeurs. (b) Non : M356 vaut désormais une valeur quelconque, le plus gros montant occupant M2. (c) Rien : `date`, `produit`, `client` et `quantite` sont restés en place, donc la ligne est un empilement de deux ventes. Le contrôle du total ne voit rien, puisque le total est gardé ; seul le contrôle à la ligne — M356 lu **avec** N356 — révèle le dégât. Si `Ctrl + Z` ne suffit pas, on repart de la copie : c'est pourquoi on travaille sur une copie.

Exercice 1.5. (a) 489 lignes, 13 colonnes, soit 490 lignes en grille avec l'en-tête. (b) Forcée en texte, toute la colonne `date` s'aligne à gauche et se trie alphabétiquement : le 05/01/2025 passe avant le 2025-01-01, et les 65 lignes écrites à la française se retrouvent mélangées aux 424 écrites en ISO. Laissée en « Général », l'import convertit les unes et pas les autres : une colonne

à moitié date, à moitié texte, soit le pire tri possible. (c) 7 lignes dépassent 500 dans le fichier reçu, dont la quantité maximale 14 000 ; dans la donnée nettoyée, la borne est 68. Les deux chemins donnent deux chiffres pour le même fichier : d'où la fiche, qui date et nomme le chemin suivi.

13. Mini-projet M03.P1 — « La fiche de prise en main du classeur de pilotage » (45 min)

Commande. Vous êtes recruté comme analyste chez Sahel Distribution. On vous remet le classeur de l'atelier et le fichier reçu `01_socle_donnees/data/projection/ventes_magasin5_2025.csv`. Rédigez en **une page maximum** la fiche de prise en main que vous enverrez au responsable du contrôle de gestion avant toute analyse : elle doit permettre à quelqu'un qui n'a jamais ouvert le fichier de retrouver vos six nombres et de comprendre les deux anomalies qui interdisent de calculer tout de suite.

Livrables numérotés. (1) la fiche, cinq rubriques du §7, une page ; (2) le tableau des trois visages de quatre cellules, avec la nature de chacune ; (3) les deux chiffres de contrôle — total de colonne sur chaque feuille, plus gros montant à la ligne — avec le geste qui les a produits ; (4) deux phrases : ce qu'obtient un utilisateur non averti qui somme le fichier reçu, et pourquoi ce chiffre est faux sans erreur affichée ; (5) les cinq réglages de fenêtre appliqués, et pour chacun la classe d'accident qu'il empêche.

Barème (20 points, seuil 13). Fiche complète, cinq rubriques, dimensions et bornes de période datées (5) · nombres de contrôle exacts et **geste** noté (4) · anomalies nommées : texte contre nombre, tri sans extension (4) · tableau des trois visages juste (3) · réglages justifiés (2) · style : une page, ton professionnel, aucune formule citée en l'air (2). Quarante-cinq minutes chrono : en entreprise, la prise en main d'un fichier ne se facture pas deux jours.

14. Résumé du chapitre

Le tableur est une armoire : classeur, feuilles, plages, cellules. Trois zones pour s'y repérer — la **zone de nom** pour dire où l'on va, la **barre de formule** pour savoir ce que c'est, la **barre d'état** pour savoir combien il y en a — et quatre touches pour s'y déplacer : `Ctrl + Fin` (zone utilisée), `Ctrl + ↑/↓` (bords de bloc), `Ctrl + Maj + ↓` (sélection informée), `F5` (plages nommées).

Chaque cellule a trois visages : adresse, contenu, affichage. Les deux premiers se contrôlent, le troisième est ce que voit votre direction, et la distance entre les trois est l'endroit exact où se logent les erreurs. Sur le fichier reçu par l'atelier, cette distance tient en quatre nombres : 489 lignes écrites en texte, une somme de colonne à 0 dans le classeur, 2 444 FCFA après un double-clic qui n'a converti que trois cellules, et un vrai chiffre d'affaires de 36 073 185 FCFA sur la feuille nettoyée de 480 lignes. Le contrôle d'un fichier avant tout travail se fait sans formule : dimensions, nature des colonnes, bornes de la période, total, et **une ligne** vérifiée — parce que les deux accidents qui comptent, le texte ignoré et le tri d'une colonne seule, sont silencieuses.

15. À retenir

À retenir. Une **somme lue à l'écran n'est pas une somme calculée** : la barre d'état n'additionne que ce que le tableur sait additionner. Sur `montant_ttc` de `ventes_brutes`, elle annonce 0 FCFA pour 489 cellules ; après un double-clic sur le même fichier, 2 444 FCFA pour 3. Aucun des deux n'est une erreur : ce sont deux descriptions exactes d'un fichier de texte.

À retenir. La dernière cellule se lit `M490` sur la feuille brute, `N481` sur la feuille de travail — le fichier plat nettoyé s'arrête en `M481` : 489 lignes reçues, 480 exploitées, 9 doublons retirés pour 266 132 FCFA. Un chiffre d'affaires se cite avec son décompte.

À retenir. Le format ne change jamais le contenu : la preuve, ce n'est pas la colonne qui brille en FCFA, c'est le compteur qui passe de 0 à 480. Un nettoyage est une **conversion de nature**.

À retenir. Après tout tri, on vérifie **une ligne**, pas seulement le total : `M356` doit encore valoir 1 229 678 FCFA et porter la même vente que sa formule `N356`.

1. Un classeur contient des feuilles, une feuille des plages, une plage des cellules adressées : on ne dit jamais « la donnée est dans le fichier », on dit où.
2. Cinq réglages de fenêtre — deux compteurs allumés, en-tête figé, grille et largeurs, accès rapide, sauvegarde datée — et l'interface cesse de mentir par omission.
3. Un `.csv` s'importe par l'assistant (séparateur `;`, UTF-8, types explicites) : le double-clic est un pari.
4. La fiche de prise en main — cinq rubriques, une page — est le premier livrable de toute analyse, avant le premier graphique.

16. Évaluation formative (auto-correction, 8 min)

1. Vous sélectionnez la colonne M de `ventes_brutes` d'un clic sur la lettre : « Nombre : 489 · valeurs numériques
0 · Somme : 0 ». Un collègue, qui a ouvert le même fichier d'un double-clic, obtient « 3 » et « 2 444 ». Donnez à ces deux lectures leur interprétation correcte, et dites ce qu'aucune des deux n'autorise à conclure.
2. `Ctrl + Fin` mène à `M490` alors que le fichier ne contient que 489 lignes. Expliquez en une phrase ce que le raccourci a compté de plus que vous.
3. Un collègue affirme : « j'ai appliqué le format Nombre au franc CFA à la colonne des montants, le problème est réglé ». Répondez en distinguant format et contenu, et dites quels nombres auraient bougé — ou non.
4. Vous triez `montant_ttc` seule, sans étendre la sélection : donnez les deux choses qui restent justes, la chose qui devient fausse, et le contrôle de deux secondes qui la voit.
5. Citez les cinq rubriques de la fiche de prise en main, et pour chacune le nombre du classeur qui la remplit.

Question ouverte. Votre direction écrit : « 2 444 FCFA de ventes sur la période, c'est effondré ». Reprenez la phrase en trois lignes, sans ironie : ce que le chiffre mesure réellement, ce qu'il faudrait lire à la place, et ce que l'on a fait du fichier pour obtenir ce second nombre.

Corrigé de l'évaluation formative.

1. Dans le classeur : 489 = toutes les cellules non vides de la colonne ; 0 = aucune n'est un nombre ; somme 0 = le tableur n'a rien à additionner. Après double-clic : 3 = les trois cellules écrites sans séparateur de milliers (lignes 88, 179, 236) ; 2 444 = la somme de ces trois

seules valeurs. Aucune des deux lectures ne dit le chiffre d'affaires de la période : le bon total, 36 073 185 FCFA, se lit sur la feuille nettoyée ventes (480 lignes).

2. Le raccourci atteint la **dernière cellule de la zone utilisée**, en-tête compris : 489 lignes de données plus une ligne d'en-tête donnent la ligne 490. On écrit donc « M490 pour 489 lignes », sinon le relecteur ne retrouve pas le compte.
3. Rien ne bouge, sauf l'apparence : les 489 cellules restent du texte, le compteur de valeurs numériques reste à 0, la somme à 0. Ce qui change la nature d'un contenu, c'est une **conversion** — import typé, collage spécial avec opération, nettoyage en amont — et sa preuve est le compteur passant de 0 à 480.
4. Restent justes le **total** de la colonne, 36 073 185 FCFA, et chacune des **valeurs**. Devient faux : l'**appartenance** des valeurs aux lignes — chaque montant se retrouve accolé à une date, un produit, un client qui ne sont pas les siens. Le contrôle qui le voit : M356 doit encore valoir 1 229 678 FCFA **avec** sa ligne d'origine, ce que N356 confirme.
5. Nom et date du fichier (m03_classeur_atelier.xlsx, ouvert le 18/09/2026, sans protection) · dimensions (M490 : 489 lignes × 13 colonnes ; N481 : 480 lignes × 14 colonnes) · colonnes en défaut (489 montants TTC en texte, 65 dates dans l'autre format, quantité maximale 14 000, remises à point décimal) · bornes de période (du 01/01/2025 au 02/12/2025, 335 jours d'amplitude, 31 jours distincts, 240 jours ouverts) · contrôle à la ligne (M356 = 1 229 678 FCFA, identique à N356). Question ouverte — Réponse attendue : « 2 444 FCFA est la somme des trois cellules qu'un import automatique a su convertir ; les 486 autres montants sont stockés en texte et n'entrent dans aucun calcul. Le chiffre d'affaires de la période est 36 073 185 FCFA, obtenu sur la feuille nettoyée ventes (480 lignes, après retrait de 9 doublons pour 266 132 FCFA). Une conversion de types est nécessaire avant tout tableau de bord. »

Suite du module. Le chapitre C02 prend ce que nous avons effleuré — le typage — et en fait un outil : formats monétaires FCFA, dates, validation de données qui empêche une quantité de 14 000 d'entrer, mise en forme conditionnelle qui fait voir les 486 textes d'un coup d'œil. Si vous ne deviez garder qu'un geste d'aujourd'hui, gardez celui du §7 : ouvrir un fichier, lire le bas de l'écran, écrire cinq lignes.

Module M03.C02 — Formats et saisie intelligente : monétaire FCFA, dates, validation, mise en forme conditionnelle

Outil de ce chapitre : le tableur seul, sur le classeur de l'atelier. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. Le chapitre C01 a montré qu'un montant peut être un texte et que rien ne le crie. Voici les trois gestes qui manquent : **dire ce qu'est une donnée** — la convertir, non l'habiller ; **empêcher la prochaine erreur d'entrer** — la validation des données ; **faire voir les défauts sans les chercher** — la mise en forme conditionnelle. Un classeur bien formaté n'est pas un classeur joli : c'est un classeur dont les erreurs se voient de loin.

Base de travail — deux chemins pour une seule donnée. Le matériel est celui du chapitre précédent : le classeur `01_socle_donnees/data/projection/m03_classeur_atelier.xlsx`, le fichier reçu `01_socle_donnees/data/projection/ventes_magasin5_2025.csv`, la donnée nettoyée `01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv`, les objectifs `01_socle_donnees/data/brut/objectifs_de_ca.csv`. Chez vous : `donnees/projection/` et `donnees/reference/`. Les blocs écrivent le chemin de l'atelier ; remplacez le préfixe si vous travaillez dans votre dossier. Séparateur point-virgule, UTF-8, franc CFA (FCFA), virgule décimale. Chiffres cités : `01_socle_donnees/data/reference/chiffres_cites.md`, section M03. **Règle de ce chapitre : on travaille sur une copie de ventes_brutes nommée ventes_en_forme** — la feuille de preuve ne se met jamais en forme.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Lire** ce que le tableur a compris d'une saisie — nombre, texte, date, heure — et prévoir ce qu'il comprendra d'une colonne que vous n'avez pas encore ouverte.
2. **Écrire** les formats de gestion : monétaire franc CFA sans décimale, négatif en rouge, pourcentage à une décimale, date courte, et les formats personnalisés à sections.
3. **Convertir** une colonne de textes en nombres par trois voies, et le prouver au compteur, pas à l'œil.
4. **Poser** une validation qui refuse une quantité de 14 000 sans rejeter les lignes de retour — et dire ce qu'une validation ne protège pas.
5. **Régler** une mise en forme conditionnelle de contrôle, l'ancrer sur la bonne cellule et l'ordonner.

2. Pourquoi cette notion est importante

Situation 1 — la mise en forme qui ne répare rien. Sur le fichier reçu, 486 des 489 montants sont du texte. Un stagiaire applique un format monétaire : la colonne prend le suffixe FCFA, le total passe de « rien » à « rien », et l'on écrit dans la note que le chiffre n'est pas disponible. Le format a touché au rendu, pas à la nature. La conversion, elle, change la nature — et sa preuve tient en un nombre : le compteur de valeurs numériques passe de 0 à 480.

Situation 2 — la date qui se trie à l'envers. La colonne date du fichier reçu mélange deux écritures : 424 lignes en ISO et 65 en notation française. Laissée en texte, la colonne se trie caractère par caractère : le 01/01/2025 des lignes françaises se range avec les janvier, et 2025-09-02 se range avec les 2. Triée comme date, la même colonne rend un calendrier lisible sur 335 jours d'amplitude et 31 jours d'activité. Le tri n'a rien réparé : c'est le typage, en amont, qui a tout décidé.

Situation 3 — la remise saisie en points. Un vendeur corrige une remise : au lieu de 0,11, il tape 11. Rien ne bronche, la ligne perd un dixième de son prix — un montant possible, donc invisible. Trois heures plus tard, le tableau croisé affiche une catégorie effondrée et l'on cherche une panne où il y a une saisie. Un format pourcentage et une validation bornée à 0,25 ferment la porte.

Dans les faits. Les audits de classeurs de gestion rapportent que la majorité des anomalies de chiffres vient de la saisie et du typage, non des formules. Un garde-fou se pose en vingt secondes ; le même défaut découvert en comité de direction coûte une décision annulée.

3. Explication simple

Quand vous tapez dans une cellule, le tableur **classe** avant d'afficher. Quatre bacs : les nombres, les dates (qui sont des nombres), les heures (des fractions de jour), et le reste — le texte. Le classement dépend de votre **réglage régional** (virgule ou point décimal), de la **forme** tapée (une espace dans 1 229 678 en fait un texte), et du **format déjà présent** (une cellule au format Texte garde tout tel quel).

Le **format** vient après, comme une étiquette : décimales, séparateurs, symbole, couleur des négatifs. Il ne décide jamais de ce que verront SOMME(), un tri ou un tableau croisé. Deux conséquences : on choisit le format **avant** de saisir, et l'on vérifie la nature d'une colonne importée **avant** de l'habiller.

Trois dispositifs, trois métiers : la **validation des données** interdit une saisie impossible ; la **mise en forme conditionnelle** signale ce qui est déjà là ; la **colonne utilitaire** du chapitre précédent tranche ligne par ligne.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Format de nombre — number format	Modèle d'affichage : décimales, séparateurs, symbole, négatifs.	Le confondre avec un type : il ne convertit rien.
Format personnalisé — custom format	Modèle écrit à la main, # ##0 "FCFA"; [Rouge] -# ##0 "FCFA".	Ne pas le documenter : six mois plus tard, nul ne sait d'où vient le suffixe.
Nombre de série de date — date serial	Une date est un compteur de jours, l'heure sa partie décimale.	Croire qu'une date affichée est une date stockée.
Validation des données — data validation	Règle qu'une plage applique à ce qu'on y saisit.	Croire qu'elle filtre le collage et l'import : elle ne garde que la frappe.
Liste déroulante — drop-down list	Validation de type Liste, adossée à une plage ou une table.	Pointer la plage de références sans la figer : la liste décroche.
Mise en forme conditionnelle — conditional formatting	Format appliqué quand une condition est vraie.	Ancrer la formule ailleurs que sur la première cellule de la plage.
Coercition — coercion / type casting	Tentative de forcer un contenu dans un type demandé.	Convertir en perdant : l'illisible devenu 0 compte comme une donnée.

Français — English	Définition simple	Piège à éviter
Arrondi d’affichage — display rounding	Écart entre la valeur stockée et ce qui est montré.	Reporter les valeurs affichées dans une note : le total ne tombe plus.

Définition. Un **format à sections** s’écrit positif;négatif;zéro;texte. Notre colonne de montants pourrait porter # ##0 "FCFA";[Rouge]-# ##0 "FCFA";"- " : un montant de 1 229 678 FCFA en noir, un retour de 150 804 FCFA en rouge, et un tiret à la place des zéros, pour qu’une ligne vide ne se lise pas comme une valeur nulle.

5. Cours approfondi

Un meme montant, trois traitements

nature de la cellule	la grille affiche	somme de la colonne
texte SOMME() ignore	56 658	0
nombre compte : 480	56658	36 073 185
nombre + format le format ne change rien	56 658 FCFA	36 073 185

Sur le fichier reçu : 486 montants sur 489 a gauche, 3 a droite. La barre d'etat dit 0, puis 2 444, puis 36 073 185 : trois etats d'un meme fichier, aucun message d'erreur.

Un format n'est jamais une conversion : la nature du contenu se verifie au compteur, pas a l'oeil.

Validation : la regle qui arrete 14 000

Autoriser

Nombre entier

Donnees

comprise entre

Minimum

1

Maximum

500

Message d'erreur

La quantite doit tenir entre 1 et 500

Attention : 8 lignes de retour portent une quantite negative, jusqu'a -13. Une regle a 1 les rejeterait : la regle juste est personnalisee, de -500 a 500 pour les retours.

Dans le fichier reçu : 7 lignes au-dessus de 500, au maximum 14 000. Apres nettoyage : 68.

5.1 Ce que le tableur a compris, et pourquoi

Un même chiffre, neuf saisies, trois sorts :

Vous tapez	Le tableur lit	Pourquoi
1229678	nombre	chiffres seuls
1 229 678	texte	l’espace casse la lecture numérique
1229.678	texte	le point n’est pas le séparateur décimal du réglage français
1 229,678	nombre	espace de milliers admise, virgule lue
0.03	texte	c’est le cas des 489 remises du fichier reçu
0,03	nombre	bon séparateur
2025-01-01	date	l’ISO est reconnu presque partout
01/01/2025	date	notation française reconnue
1er janv. 2025	texte	aucune règle de lecture

Cette table explique les trois défauts mesurés du fichier reçu : 486 montants en texte (les espaces de milliers), 489 remises en texte (le point), et 65 dates écrites à la française contre 424 en ISO. Elle explique aussi pourquoi un même fichier donne trois chiffres : 0 FCFA en sommant la feuille de preuve, 2 444 FCFA si l’import automatique a converti trois cellules, 36 073 185 FCFA sur la donnée nettoyée.

5.1 Ce que le tableur a compris, et pourquoi

24

Édition 2026

À **retenir**. Le tableur ne corrige pas une saisie, il la **classe**. Ce que vous avez voulu dire ne compte pas : la seule chose qui compte est ce qui a été lu.

5.2 Les quatre formats de la gestion

- **Montants** : # ##0 "FCFA" — le format du classeur d'atelier. Aucune décimale : le franc CFA se compte, et 153 596,42 FCFA dans un tableau se lit plus mal que 153 596.
- **Négatifs** : # ##0 "FCFA";[Rouge]-# ##0 "FCFA". Nos 8 lignes de retour pèsent -283 933 FCFA : elles sautent aux yeux sans une seule règle de mise en forme.
- **Taux** : 0,0 %. La plus forte remise de la donnée nettoyée est 11 % ; une cellule qui affiche 100,0 % crie qu'on a saisi 11.
- **Dates** : JJ/MM/AAAA pour le contrôle, mmm. aaaa pour un graphique. Le format court JJ/MM perd l'année et rend tout rapprochement annuel impossible.

Trois habitudes : appliquer le format sur la **colonne entière**, le **documenter** dans Aidez-moi (une ligne par colonne : format et pourquoi), et ne jamais coder une information par la seule couleur de police — la couleur se perd à l'export, la règle non.

5.3 Les dates : un compteur de jours, pas une image

Définition. Une date stockée comme date est un **nombre de série** : un compteur de jours, dont l'heure est la partie décimale. Le contrôle est cruel de simplicité : passez B2 de la feuille ventes en Nombre général, vous verrez un entier ; la différence entre la première et la dernière ligne de l'extrait, 335, est exactement le nombre de jours du 01/01/2025 au 02/12/2025.

Trois conséquences de travail. **Une colonne à moitié date, à moitié texte, est pire qu'une colonne entièrement texte** : tri et filtres ne s'appliquent plus au tout — chez nous, les 424 lignes ISO et les 65 lignes françaises se rangent chacune de leur côté. **Une date lue au clavier dépend du poste** : le même fichier, sur un réglage régional différent, donne des dates en texte ; c'est la première cause de « ça marchait chez moi », et la parade est l'import assisté à types forcés, ou DATE() avec ses trois arguments entiers. Enfin, **une date figée n'est pas une date qui bouge** : Ctrl + ; insère la date du jour comme valeur, =AUJOURDHUI() la recalcule — la première atteste, la seconde alerte ; une fiche de prise en main veut la première, un tableau de bord la seconde.

5.4 Convertir vraiment : trois voies, une preuve

1. **Collage spécial, opération Ajouter 0**. Une cellule contenant 0, copiée, Collage spécial → Ajouter sur la colonne. Les textes chiffrés deviennent nombres ; les valeurs réellement illisibles renvoient une erreur. Rapide, destructeur.
2. **Colonne utilitaire**. =CNUM(SUBSTITUE(SUBSTITUE(M2;" ";"");" ".)) lit le texte après avoir retiré les espaces et les points. C'est la voie du classeur : la valeur d'origine reste visible à côté, et l'échec s'affiche (#VALEUR!) au lieu de se noyer dans un 0.
3. **Conversion à l'import**, en forçant le type de chaque colonne dans l'assistant — la seule qui répare aussi les dates, parce qu'elle décide avant que le tableur n'ait classé. Pour un nettoyage rejouable, le module M05 le confiera à Power Query ; ici, on fait les trois à la main pour savoir ce que chacune perd.

Définition. On parle de **coercition** quand le tableur force un contenu dans un type demandé : `=CNUM("56 658")` échoue, `=CNUM("56658")` réussit, `=CNUM("")` renvoie `#VALEUR!`. Un nettoyage qui ne dit pas combien de cellules il a fait échouer n'est pas un nettoyage : c'est une perte.

La preuve est le compteur, pas l'œil. Nombre de valeurs numériques doit passer de 0 à 480 et la somme redevenir 36 073 185 FCFA. Une conversion qui rend 489 nombres sur la feuille de preuve doit mener à 36 339 317 FCFA — les 266 132 FCFA d'écart étant les 9 doublons : la soustraction des doublons est une seconde opération, pas un sous-produit de la première.

5.5 Validation des données : interdire la prochaine erreur

Données → Validation des données attache une règle à une plage. Trois types servent en gestion.

- **Liste** : les 7 catégories, les 6 magasins, l'oui/non d'un retour. Une liste adossée à une plage doit la figer (`$A$2:$A$8`) ou, mieux, pointer une table nommée.
- **Nombre entier compris entre** : quantité de 1 à 500 refuse la ligne à 14 000 du fichier reçu — et refuse aussi les retours, dont la quantité descend jusqu'à -13. La règle juste est **personnalisée** : `=ET(I2>=-500;I2<=500)`, qui borne des deux côtés et admet le retour.
- **Nombre décimal compris entre** pour remise : 0 à 0,25. Le fichier nettoyé plafonne à 11 %, le fichier de la direction prévoit des remises contractuelles jusqu'à 25 %. Le message d'erreur dira : « la remise s'exprime en pourcentage : 11 % se saisit 0,11 ».

Attention. Trois limites, qu'aucune aide en ligne ne résume aussi net : la validation **ne s'applique ni au collage ni à l'import** — un `Ctrl + V` par-dessus la cellule écrase la règle ; une règle posée sur la colonne entière laisse la ligne 490 du prochain import sans garde-fou, il faut la copier sur la plage de données ; et un message qui ne dit pas la valeur attendue pousse l'utilisateur à cocher « Tout ignorer » dès le premier refus.

5.6 Mise en forme conditionnelle : le détecteur qui ne dort pas

1. **Texte dans une colonne qui devrait être numérique.** Plage `M2:M490`, règle Utilisez une formule, `=NON(ESTNUM(M2))` : sur la feuille de preuve, 489 cellules allumées ; 486 après un import automatique qui en a converti trois. C'est un contrôle, pas un nettoyage.
2. **Montant hors de l'échelle.** `=M2>1000000` : 4 lignes de l'extrait dépassent le million, dont celle de 1 229 678 FCFA. Une ligne à six chiffres mérite une phrase dans la note.
3. **Retours.** `=M2<0` : 8 lignes, -283 933 FCFA. Le format [Rouge] les montre déjà ; la règle les rend impossibles à manquer.
4. **Barre de données** sur `montant_ttc` : la distribution apparaît sans calculer, et l'on voit la colonne écrasée par sa queue.
5. **Valeurs en double** sur `n_ticket` : les 9 doublons sautent aux yeux avant tout tri.

Définition. L'**ancrage** d'une règle de mise en forme conditionnelle est la cellule où l'on se trouvait quand on l'a écrite : la formule est ensuite reportée relativement sur chaque cellule de la plage. `=NON(ESTNUM(M2))` créée avec `M5` active juge la ligne 2 sur la ligne 5 — et produit un décompte faux de trois unités, avec des couleurs plausibles.

Dans **Gérer les règles**, trois réglages distinguent l'outil du sapin : cochez Arrêter si vrai pour que la première règle gagne ; bornez la portée sur « ce qui est présent dans le fichier de travail », pas la feuille entière ; et gardez **trois règles maximum par feuille**, chacune reliée à une ligne de la fiche de prise en main.

5.7 Les gestes de saisie qui font gagner une heure

Tab et Maj + Tab pour parcourir une ligne sans quitter la saisie. Ctrl + Entrée pour écrire la même valeur dans toute la sélection — 489 cellules d'un coup, par exemple le taux de 18 % ou un libellé de source. Alt + Entrée pour un retour à la ligne dans une cellule, à réserver aux entêtes et jamais aux données. Ctrl + ; pour une date figée. La **poignée de recopie incrémentée**, qui sur Janvier produit les 12 mois et sur lun. les lundis — c'est elle qui construira la grille mensuelle en C07. Enfin Suppr contre Ctrl + - : le premier efface la valeur, la seconde supprime la **ligne**, ce qui n'a rien d'équivalent quand une table nourrit des formules.

Boîte à outils. Ctrl + 1 (Format de cellule), Ctrl + Maj + 1 (nombre avec séparateurs), Ctrl + Maj + 2 (heure), Ctrl + Maj + 3 (date), Ctrl + Maj + \$ (monétaire), Ctrl + Maj + % (pourcentage), Ctrl + Maj + ~ (retour au format Général — le « montre-moi ce que tu stocks » de ce chapitre), et Accueil → Mise en forme conditionnelle → Nouvelle règle. Dans LibreOffice Calc, les formats et Ctrl + 1 sont identiques ; la validation s'appelle Données → Validité et la mise en forme Format → Conditionnelle → Liste.

6. Exemple concret : trois lignes saisies, quatre pièges refermés

Sur la copie `ventes_en_forme`, on saisit ce qu'un magasin enverrait par courriel, et l'on regarde ce que font les garde-fous.

Saisie	Sans garde-fou	Avec format, validation et détection
Quantité 14 000	acceptée ; la ligne devient un bruit de plusieurs millions	refusée : « Quantité entre -500 et 500 »
Quantité -13 (un retour)	acceptée, personne ne la voit	acceptée par la règle personnalisée, et en rouge par [Rouge]
Remise 11	acceptée ; la ligne perd un dixième de son prix	refusée ; en 0,0 %, la saisie 0,11 affiche 11,0 %
Montant 56 658 en texte	entre dans la colonne, jamais sommé	allumé par <code>=NON(ESTNUM(M2))</code> : la ligne est à reprendre

Le quatrième cas est le plus fréquent du métier : on ne réimporte pas le fichier, on **corrige une ligne à la main**. La détection est ce qui empêche la correction d'ouvrir un second défaut.

7. Démonstration pas à pas : la colonne de contrôle de saisie

Quatre étapes sur la copie `ventes_en_forme` (colonnes du fichier reçu : I quantité, K remise, M montant TTC).

1. Préparer. Copie de `ventes_brutes`, largeurs ajustées, ligne supérieure figée, format `# ##0 "FCFA";[Rouge]-# ##0 "FCFA"` sur L2:M490. La barre d'état sur M dit toujours valeurs numériques : 0 : le format n'a rien changé, et c'est la leçon.

2. Convertir. Colonne utilitaire 0 : `=CNUM(SUBSTITUE(SUBSTITUE(M2;" ";"");" ".))`. Sans les deux `SUBSTITUE`, seules les trois cellules écrites sans espace passent (797, 821 et 826 FCFA, soit 2 444 FCFA) ; avec, les 489 passent. Copiez 0, Collage spécial → Valeurs sur M, supprimez 0 : la somme est 36 339 317 FCFA.

3. Retirer les doublons. Données → Supprimer les doublons (le geste proprement dit est en C03) : 9 lignes partent, 480 restent, la somme tombe à 36 073 185 FCFA et le compteur à 480. Les trois nombres se suivent : c'est cette chaîne qui rend la conversion défendable.

4. Statut de saisie. En N, titre « statut », et sur chaque ligne : `=SI(NON(ESTNUM(M2)),"montant non numérique",SI(OU(I2>500;I2<-500),"quantité hors bornes",SI(OU(K2<0;K2>0,25),"remise hors bornes","conforme")))`. Sur la donnée nettoyée : 480 « conforme » — la quantité maximale y est 68 et aucune ligne ne dépasse 100, la remise maximale 11 %. Sur la feuille de preuve avant conversion, le même texte écrit 489 fois « montant non numérique ».

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Somme de montant ttc après conversion, avant dédoublonnage	±1 FCFA	36 339 317 FCFA	conversion partielle ou SUBSTITUE oubliée
2	Somme après suppression des doublons	±1 FCFA	36 073 185 FCFA	9 lignes de moins, ou de trop
3	Nombre de valeurs numériques sur M	identité	480	format pris pour une conversion
4	Décompte de la colonne statut	deux nombres	7 quantités hors bornes · 8 retours	règle mal bornée : retours rejetés ou non détectés
5	Remise maximale	0,1 point	11,0 %	colonne restée en point décimal

8. Erreurs fréquentes

- 1. Confondre embellissement et réparation.** Un format monétaire sur une colonne texte produit une colonne texte jolie : le total ne bouge pas.
- 2. Convertir sans garder la valeur d'origine.** Écraser 56 658 rend impossible la preuve que 486 lignes ont été touchées. On convertit dans une colonne utilitaire, puis on remplace — pas l'inverse.
- 3. Croire la donnée protégée par une validation.** Elle ne filtre ni collage, ni import, ni formule. Sur un fichier qui entre tous les jours, la protection utile est la détection relue après chaque import.
- 4. Écrire une règle avec la mauvaise cellule active.** Le décalage d'ancrage produit des couleurs justes pour de mauvaises raisons — le pire résultat possible, parce qu'il est vraisemblable.
- 5. Reporter les valeurs affichées.** Deux lignes affichent 12 % et 12 %, leurs valeurs sont 0,115 et 0,121 : le chiffre cité dans une note vient de la barre de formule, avec sa décimale.
- 6. Sauver en .csv pour alléger.** Formats, validations et mises en forme conditionnelles ne survivent pas à l'export ; seule la donnée reste. Le .csv est un canal d'échange, jamais un classeur de travail.

Attention. Le piège le plus coûteux de ce chapitre est un arrondi : le total de la colonne recalculée diffère du total enregistré de **4 FCFA**, et ce n'est pas une faute. Six lignes de l'extrait ont un produit qui tombe exactement à un demi-franc, et sur quatre d'entre elles la règle d'arrondi choisie change le résultat. Dites la vôtre, dans Aidez-moi : « arrondi à l'unité, au demi-franc supérieur » — sinon un relecteur croisera deux tableaux à 4 FCFA près et perdra une heure à chercher une erreur inexistante.

9. Bonnes pratiques professionnelles

1. **Un format par famille de colonne, écrit une fois** et nommé dans Aidez-moi : montants, taux, dates, quantités. Les dérives unitaires viennent des formats posés cellule à cellule.
2. **La devise dans le format, jamais dans la cellule** — un FCFA tapé dans une cellule de montant en fait un texte, exactement le défaut que nous venons de réparer.
3. **Tout négatif visible** : format [Rouge] ou règle sur <0. Masquer les 8 retours gonflerait le chiffre de l'extrait de 0,8 %.
4. **Validation + commentaire + ligne de fiche**. La règle vit dans la cellule, sa justification dans le commentaire, son existence dans la fiche. Une validation non documentée est supprimée à la première gêne, et la gêne arrive toujours.
5. **La colonne de statut avant la couleur**. Le texte « montant non numérique » se filtre, se compte, s'imprime en noir et blanc ; le jaune, non.

Conseil professionnel. Avant d'envoyer un classeur, testez-le en noir et blanc : Fichier → Imprimer → Aperçu, niveaux de gris. Si les lignes que vous vouliez signaler disparaissent, votre signal reposait sur une couleur et non sur une règle — écrivez le mot dans la colonne de statut, et gardez la couleur par-dessus.

10. Exercice guidé — formats, conversion, garde-fous (30 min, /10)

Commande. Sur une copie de ventes_brutes nommée ventes_en_forme, menez les six étapes et notez, à chacune, ce qu'affiche la barre d'état sur montant_ttc.

Étape	Geste	Nombre attendu
1	Format monétaire sur L2:M490	valeurs numériques 0 · somme 0
2	Colonne utilitaire, CNUM seul, sans SUBSTITUE	3 · 2 444 FCFA
3	Même colonne avec les deux SUBSTITUE	489 · 36 339 317 FCFA
4	Collage spécial → Valeurs sur M, suppression de l'utilitaire	489 · 36 339 317 FCFA
5	Supprimer les doublons	480 · 36 073 185 FCFA · moyenne 75 152 FCFA
6	Validation =ET(I2>=-500;I2<=500) et règle =NON(ESTNUM(M2))	refus sur 14 000, acceptation de -13, 0 cellule allumée

Démarrage. Les étapes 1 à 3 sont le point de bascule : le format ne change pas le compteur, la conversion le change, et SUBSTITUE change la conversion. Notez les trois paires de nombres avant d'aller plus loin. À l'étape 3, la somme attendue est 36 339 317 FCFA, soit 266 132 FCFA de trop : ce sont les doublons, et l'étape 5 doit les faire disparaître — une conversion et un dédoublonnage sont deux opérations, pas une. À l'étape 6, tapez les deux valeurs tests : 14 000 doit être refusé, -13 accepté ; si -13 est refusé, votre règle borne à 1 et vous venez d'interdire les retours.

Barème (10 points). Six étapes menées, deux nombres notés à chaque étape (4) · les deux sommes exactes avant et après dédoublonnage (2) · validation bornée des deux côtés avec message rédigé (2) · règle de détection ancrée sur la première cellule de la plage, Arrêter si vrai coché (2).

11. Exercices autonomes

Exercice 2.1 (★) — Les neuf saisies. Dans une feuille neuve, recopiez les neuf contenus du tableau \$5.1, un par ligne. Notez l'alignement, ce que rend la barre de formule, et le compteur de valeurs numériques. Concluez par la phrase : « le tableur lit ... ».

Exercice 2.2 (★) — Ce que la cellule stocke. Sur `ventes`, passez `montant_ttc` en Nombre général, notez M2 et M356, repassez en monétaire, puis comparez la somme des valeurs affichées et la somme réelle de la colonne. Écrivez la phrase qui évitera à un lecteur de chercher les centimes perdues.

Exercice 2.3 (★★) — Trois voies pour une date. Sur une copie, convertissez la colonne `date` du fichier reçu par les trois voies : format seul, import avec type forcé, colonne utilitaire reconstruisant `DATE()` avec `STXT` et `NBCAR` sur les 65 lignes à la française. Contrôlez chaque voie par trois nombres : cellules reconnues, bornes de période, résultat du tri du dernier mois.

Exercice 2.4 (★★) — La validation qui n'a rien vu. Posez « entier de 1 à 500 » sur `quantite` d'une copie, puis collez par-dessus la plage venue de `ventes` (qui contient les 8 retours). Comparez ce qui est passé quand même, et ce que la validation est devenue. Écrivez la règle qui aurait vu, et les deux lignes de fiche qui la justifient.

Exercice 2.5 (★★) — Le rouge qui ment. Appliquez `=NON(ESTNUM(M5))` à la plage M2:M490 sur une colonne partiellement convertie, puis la règle correctement ancrée. Comparez les deux décomptes, expliquez l'écart par la mécanique d'ancrage, et terminez par l'ordre des règles dans Gérer les règles.

12. Correction détaillée

Exercice 2.1. Restent du texte : 1 229 678, 1229.678, 0.03, 1er janv. 2025 — à gauche, ignorés par le compteur. Deviennent des nombres : 1229678, 1 229,678, 0,03 — à droite, comptés. Deviennent des dates (donc des nombres) : 2025-01-01 et 01/01/2025. Phrase attendue : « le tableur lit ce que son réglage régional autorise, et rien d'autre ».

Exercice 2.2. En Nombre général, M2 rend 56658 et M356 rend 1229678 ; en monétaire, les mêmes entiers précédés de l'espace de milliers et suivis de FCFA. La somme est 36 073 185 FCFA dans les deux cas : le format n'a rien ajouté ni retiré. Phrase attendue : « montants en francs CFA, stockés en nombres entiers, aucune décimale — les totaux sont calculés sur les valeurs stockées, pas sur les valeurs affichées ».

Exercice 2.3. Format seul : 0 reconnaissance — on ne convertit pas un rendu. Import typé en Date : 424 lignes ISO passent, les 65 lignes à la française restent en texte, et les bornes 01/01/2025 et 02/12/2025 ne se retrouvent que sur la partie convertie. Reconstruction par `DATE()` : les 489 passent, avec contrôle croisé — 335 jours d'amplitude, 31 jours distincts, et un tri du dernier mois qui rend le 02/12/2025 en tête. Seule la troisième voie est défendable : les deux autres dépendent du poste qui ouvre le fichier.

Exercice 2.4. Le collage n'exécute pas la règle : les 8 lignes de retour, quantité jusqu'à -13, passent sans message, et le collage emporte avec lui la validation de la plage collée (elle est remplacée par celle de la source, c'est-à-dire souvent rien). La règle qui aurait vu : `=ET(I2>=-500;I2<=500)` en Personnalisée, doublée de `=I2<0` en mise en forme conditionnelle — 8 lignes signalées — et d'un contrôle de décompte rejoué après chaque import. Fiche : « plage I2:I490, règle personnalisée bornée à ±500, motif : 8 retours sur 480 lignes, posée le ... ».

Exercice 2.5. Ancrée sur M5, la règle juge M2 avec le contenu de M5, M3 avec M6, et ainsi de suite : le décompte diffère du vrai de trois lignes, dans un sens qui dépend du hasard des conversions, et les trois dernières lignes sont jugées sur du vide. L'ancre correcte est M2 ; dans Gérer les règles, la détection passe en tête avec Arrêter si vrai, et la portée se limite à M2:M490.

13. Mini-projet M03.P1 — suite : le volet formats et garde-fous (40 min)

Commande. Reprenez la fiche de prise en main du chapitre C01 et complétez-la par le volet formats, en une page, pour que le classeur se suffise à lui-même sans explication orale.

Livrables numérotés. (1) le tableau des formats du classeur — colonne, format exact, motif ; (2) la preuve chiffrée de la conversion en trois temps : avant ($0 \cdot 0$ FCFA), après conversion ($489 \cdot 36\,339\,317$ FCFA), après dédoublement ($480 \cdot 36\,073\,185$ FCFA) ; (3) les règles de validation, leur message, et la ligne de fiche qui les justifie ; (4) les trois règles de mise en forme conditionnelle — formule, portée, ordre — et le décompte de cellules allumées avant conversion (489) et après (0).

Barème (20 points, seuil 13). Tableau des formats complet et exact (5) · preuve de conversion en trois temps, dans l'ordre (4) · validation bornée des deux côtés avec message utile (4) · règles de détection : ancrage, portée, ordre, décomptes (4) · style : une page, aucun format posé cellule à cellule (3).

14. Résumé du chapitre

Le tableur classe avant d'afficher : ce qu'il a lu décide de tout, et le réglage régional décide de ce qu'il lit. Les formats — monétaire, négatif en rouge, pourcentage à une décimale, date courte — sont des couches de rendu ; ils rendent lisible, jamais juste. Convertir est un autre acte : collage spécial avec opération, colonne utilitaire avec `CNUM(SUBSTITUE(...))`, ou types forcés à l'import ; la preuve tient dans un compteur qui passe de 0 à 480 et une somme qui revient à $36\,073\,185$ FCFA, après une conversion **et** un retrait des 9 doublons, deux opérations distinctes.

Le classeur se protège en trois couches : la validation refuse la saisie impossible, la mise en forme conditionnelle signale ce qui est déjà là, la colonne de statut écrit le mot qu'une couleur ne peut pas dire. Les dates sont le cas d'école : un compteur de jours, deux écritures dans le fichier reçu (424 et 65), 335 jours d'amplitude, 31 jours d'activité, et une conversion explicite qui affranchit du poste de celui qui ouvrira le fichier.

15. À retenir

À retenir. Un format ne convertit pas. La preuve d'une conversion n'est pas la colonne qui brille en FCFA, c'est Nombre de valeurs numériques qui passe de 0 à 480.

À retenir. La validation garde la frappe, pas le collage ni l'import. Sur un fichier qui entre tous les jours, la protection utile est une règle de détection relue après chaque import, et un mot écrit dans une colonne de statut.

À retenir. Une règle de mise en forme conditionnelle s'ancore sur la première cellule de sa plage : une référence décalée de trois lignes produit un décompte faux de trois unités et des couleurs plausibles.

1. La devise est dans le format, jamais dans la cellule ; les négatifs sont en rouge ; les taux à une décimale, pour que `1 100,0 %` crie de lui-même.
2. Une date est un nombre de jours : le test du Nombre général est le seul qui dise ce que la cellule stocke.
3. Convertir dans une colonne utilitaire, remplacer, garder la trace : on ne corrige pas la preuve.
4. Trois règles de détection maximum, Arrêter si vrai, chacune adossée à une ligne de la fiche.

5. Les 4 FCFA d'écart entre montants enregistrés et recalculés ne sont pas une faute : c'est l'arrondi de six lignes à un demi-franc. Dites votre règle, on ne la cherchera pas.

16. Évaluation formative (auto-correction, 8 min)

1. Un montant affiché 36 073 185 FCFA peut-il être du texte ? Comment le savoir en trois secondes ?
2. Citez les trois voies de conversion et, pour chacune, ce qu'elle fait des valeurs illisibles.
3. La colonne remise est écrite 0.03 dans le fichier reçu. Que lit un tableur français, que lit un tableur anglais, et quelle borne attendez-vous après conversion sur la donnée nettoyée ?
4. Vous posez « entier de 1 à 500 » sur quantité : quelles lignes de la donnée nettoyée cette règle aurait-elle fait disparaître, et pour quel montant ?
5. Une règle =NON(ESTNUM(M2)) est créée avec M5 comme cellule active sur M2:M490. Que voit-on, et pourquoi est-ce pire que l'absence de règle ?

Question ouverte. Un collègue écrit : « j'ai mis un format, puis une validation, donc le classeur est propre ». Répondez en nommant ce que son dispositif ne voit pas, et la couche qui le voit.

Corrigé de l'évaluation formative.

1. Oui : le format n'agit que sur le rendu. Trois secondes, deux gestes — l'alignement (gauche : texte) et Nombre de valeurs numériques sur la colonne (0 : tout est texte, même orné de FCFA).
2. Collage spécial avec opération : rapide, transforme l'illisible en erreur ou en valeur fausse, sans trace conservée ; colonne utilitaire avec CNUM : garde la valeur d'origine et rend les échecs lisibles en #VALEUR! ; types forcés à l'import : la seule voie qui répare aussi les dates, parce qu'elle décide avant le classement.
3. En français, du texte — le point n'est pas le séparateur décimal ; en anglais, le nombre 0,03. Après conversion, la borne attendue est 0,11, soit 11,0 % au format pourcentage : au-delà, on est hors politique commerciale.
4. Les 8 lignes de retour, quantité jusqu'à -13, pour -283 933 FCFA. Les exclure affiche 36 357 118 FCFA — le total des seules lignes positives — au lieu de 36 073 185 FCFA : un chiffre sur-évalué de 0,8 %, impossible à défendre une fois les retours retrouvés.
5. La règle juge la ligne 2 sur la ligne 5, la ligne 3 sur la ligne 6 : le décompte est faux de trois unités, dans un sens imprévisible, et les couleurs restent plausibles. C'est pire qu'une absence de règle, parce qu'une absence ne fait pas semblant d'être juste. Question ouverte — Réponse attendue : « le format ne voit rien, la validation ne voit que la frappe : ni le collage par-dessus la plage, ni les 486 textes entrés par l'import. Ce que le dispositif ne voit pas, la détection par formule le voit — ESTNUM, une colonne de statut, et un contrôle de compteur joué après chaque import ».

Suite du module. Le chapitre C03 quitte l'habillage pour la structure : la table structurée, qui fait grandir la donnée avec son format et sa validation ; le tri honnête, avec extension de la sélection ; les filtres ; et la suppression des doublons — les 9 lignes retirées à la main tout à l'heure, proprement cette fois.

Module M03.C03 — Le tableau structuré, le tri, les filtres, la recherche et les doublons

Outil de ce chapitre : le tableur seul, sur le classeur de l'atelier. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. Un classeur de gestion n'est pas une feuille : c'est une **table** posée sur une feuille. Cette table a un nom, une adresse qui s'étend, des colonnes que les formules désignent par leur titre, un tri qui n'abîme rien et des filtres qui se lisent à la barre d'état. Le chapitre se termine par le geste le plus catastrophique du métier — Supprimer les doublons — et par la seule chose qui le rend sûr : le choix de la clé.

Base de travail. Le classeur `01_socle_donnees/data/projection/m03_classeur_atelier.xlsx`, le fichier reçu `01_socle_donnees/data/projection/ventes_magasin5_2025.csv`, la donnée nettoyée `01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv`, la table de rapprochement `01_socle_donnees/data/brut/objectifs_de_ca.csv`. Chez vous : `donnees/projection/` et `donnees/reference/`. Séparateur point-virgule, UTF-8, franc CFA (FCFA), virgule décimale. Chiffres cités : `01_socle_donnees/data/reference/chiffres_cites.md`, section M03. Deux feuilles de travail : `ventes` (la donnée propre, déjà en tableau) et une copie de `ventes_brutes` nommée `ventes_a_structurer`, que vous allez transformer.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Passer une plage en tableau structuré**, lui donner un nom, et dire ce que ce nom change dans une formule.
2. **Écrire avec des références structurées** (`TableVentes[montant_ttc]`, `[@colonne]`) et les lire sans dictionnaire.
3. **Trier sans casser** : tri à plusieurs niveaux, tri dans un tableau, et le geste qui désynchronise une ligne.
4. **Filtrer, puis le vérifier** à la barre d'état — y compris repérer un filtre qui ne filtre rien.
5. **Chercher et remplacer** dans une colonne, avec les jokers, sans ravager les montants.
6. **Dédoublonner sur la bonne clé**, en chiffrant avant et après ce que la ligne retirée valait.

2. Pourquoi cette notion est importante

Situation 1 — la plage qui ne grandit pas. La feuille `ventes_brutes` est une plage filtrée : son filtre porte l'adresse `A1:M490`. Le mois suivant, 40 lignes de plus entrent en 491 et suivantes. Rien ne le dit : le filtre ne les voit pas, le total les ignore, et le tableau croisé du chapitre C08 rendra le mois précédent. Le tableau structuré, lui, s'étend : sa donnée, c'est un nom, pas une adresse.

Situation 2 — le tri qui a décousu la ligne. Un stagiaire veut la plus grosse vente : il sélectionne la colonne des montants, trie décroissant, et laisse le tableur « étendre la sélection » à une colonne... qui n'était pas la sienne. Le fichier est maintenant cohérent ligne par ligne nulle

part : le montant du 2 septembre est devenu celui d'un ticket de janvier. Aucune erreur affichée, aucun message, aucun recours si la sauvegarde a écrasé l'original.

Situation 3 — le doublon qui n'en était pas un. Le fichier reçu contient 9 lignes strictement identiques à une autre, pour 266 132 FCFA. Un collègue, voulant bien faire, coche `n_ticket` seul comme critère : 173 lignes partent, 11 424 263 FCFA avec elles — et ce sont de vraies ventes, simplement portées par le même ticket. Un ticket vaut plusieurs lignes : c'est même la règle, puisqu'il y a 489 lignes pour 316 tickets.

Dans les faits. Le service de contrôle interne qui audite des classeurs de gestion demande trois choses sur ce chapitre : la donnée est-elle nommée, le tri est-il consigné, le dédoublonnage est-il justifié ligne par ligne. Ces trois traces se produisent en trois minutes ; leur absence se rachète en jours.

3. Explication simple

Une **plage**, c'est une adresse : `A1:M490`. Elle ne sait rien de son contenu. Un **tableau**, c'est un objet nommé dans le classeur, qui sait où il commence, où il finit, comment s'appellent ses colonnes, et qu'il faut grandir quand on ajoute une ligne au bord.

Le tableur s'en sert trois fois. Dans les **formules** : `=SOMME(TableVentes[montant_ttc])` dit ce qu'on somme, et non où. Dans le **tri et les filtres** : cliquer dans un tableau bornera automatiquement l'action à ses lignes, alors qu'une plage sélectionnée à moitié produit le désastre du §2. Dans les **objets d'aval** : un tableau croisé ou une requête qui lit un tableau est sûr de lire les lignes 481 et 521 le mois prochain.

Le prix à payer : une table ne se coupe pas comme une plage, son en-tête est obligatoire, et elle refuse deux tables qui se touchent. Ce sont des garde-fous, pas des défauts.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Plage — range	Zone désignée par une adresse, <code>A1:M490</code> .	Compter sur elle pour grandir seule.
Nom de plage — named range	Alias sur une adresse, <code>Ventes =Feuil1!\$A\$1:\$M\$490</code> .	L'adresse reste figée sous le nom.
Tableau (structuré) — table	Objet nommé, avec en-têtes, formats, filtres et portée qui suit la donnée.	Deux tableaux côte à côte : le second est refusé.
Référence structurée — structured reference	<code>TableVentes[montant_ttc]</code> , <code>[@quantite]</code> .	Écrire la formule hors du tableau, puis copier : les <code>[@...]</code> deviennent <code>#VALEUR!</code> .
Ligne de total — totals row	Ligne <code>Total</code> du tableau, calculant avec <code>Sous.Total()</code> .	Oublier qu'elle ne se somme pas elle-même dans un <code>SOMME()</code> naïf.
Filtre automatique — AutoFilter	Les boutons ▼ des en-têtes.	Croire qu'il porte sur toute la table quand la plage ne s'étend pas.
Tri à plusieurs niveaux — multi-level sort	Date, puis <code>n_ticket</code> , puis heure.	Trier « la colonne montant » au lieu de trier « les lignes ».
Doublon — duplicate row	Répétition suivant la clé choisie.	Choisir la clé après avoir vu le résultat.

Définition. Une **table** se nomme une fois, et ce nom devient une adresse vivante : **TableVentes** désigne Feuille! [A1:M481] aujourd’hui, A1:M521 dans un mois. Toute formule, tout tableau croisé, toute requête écrite sur le nom suivra le mouvement ; tous ceux écrits sur l’adresse resteront collés en 481.

5. Cours approfondi

Deux façons de tenir la même table

Plage + filtre automatique — feuille ventes_brutes, A1:M490

	n_ticket	quantite	montant_ttc	▼ sur 490 lignes
2	T05-250101-075845	11	56 658	
3	T05-250101-075846	2 000	51 802	
4	T05-250101-075847	2	58 396	
... 486 lignes ... 491 collée après : hors du filtre, hors des totaux				

Le filtre porte une adresse figée : à chaque import, on la refait.

Trier une colonne seule : lignes désynchronisées, aucun signal.

Tableau structuré — TableVentes, A1:M481 (Ctrl+L)

	n_ticket	quantite	montant_ttc	
	T05-250101-075845	11	56 658	
	T05-250101-075846	2 000	51 802	
Total → 36 073 185 FCFA · ligne 481 : le tableau s’étend				

colonne N restée dehors : la 14e ne suit pas toute seule

Supprimer les doublons :

tout se décide sur la clé

les 13 colonnes

9 lignes · 266 132 FCFA

ticket + produit

12 lignes · 481 978 FCFA

ticket seul

173 lignes · 11 424 263 FCFA

Un ticket porte plusieurs lignes :

489 lignes, 316 tickets — dédoubler sur le ticket efface 173 ventes

réelles, 11 424 263 FCFA

Trier : sélectionner toute la ligne, ou cliquer dans le tableau ; puis deux niveaux — date, puis n_ticket.

Filtrer : 40 lignes par mois × 12 mois = 480 · Matériaux → 143 · contient « Sable » → 11 · quantité > 50 → 3.

Le filtre « Magasin » rend 480 lignes sur 480 : un filtre qui ne filtre rien est un filtre à supprimer.

Dans le classeur d’atelier, la feuille ventes contient déjà un tableau : **TableVentes**, sur A1:M481 (13 colonnes, 480 lignes de données). La feuille ventes_brutes n’en contient pas : elle porte un filtre automatique sur A1:M490. Cette asymétrie est l’exemple lui-même.

5.1 Créer le tableau, et le nommer

Accueil → Mettre sous forme de tableau, ou Insertion → Tableau, ou Ctrl + L (Ctrl + T dans les versions anglaises) : le tableur propose la zone contiguë, coche « ma plage contient des entêtes ». Trois réglages immédiats, dans l’ordre : **le nom** (Création de tableau → Nom du tableau) — **TableVentes**, jamais **Tableau1** ; **la ligne de total**, à allumer tout de suite si le fichier doit être lu par un non-initié ; **le style**, avec bandes et en-tête coloré, à choisir sobre parce qu’il se verra en impression noir et blanc.

Un tableau refuse d’empiéter : si une cellule est remplie en N1, la création s’arrêtera en M. Et une table ne peut pas contenir de cellule fusionnée — le message est explicite, le réflexe aussi : Annuler les fusions et centrer, jamais de fusion dans une donnée.

Définition. La **portée** d’un tableau est l’intervalle qu’il occupe, étendue à chaque ligne ajoutée en contiguïté. Un collage fait à la ligne 481, collé contre la dernière ligne, est intégré ; le même collage avec une ligne vide entre les deux laisse la donnée **dehors** — hors du filtre, hors des totaux, hors du tableau croisé.

5.2 Ce que la référence structurée change aux formules

Dans le tableau, une formule écrite sur une colonne calculée s'écrit tout de suite en références structurées. Trois écritures à connaître, dans l'ordre d'utilité :

- `=ARRONDI([@[quantite]]*[@prix_unitaire_ht]*(1-[@remise]));0)` — la ligne calcule sa propre ligne ; recopiée, elle le reste, sans un seul \$.
- `=SOMME(TableVentes[montant_ttc])` — la colonne entière, d'où le nom **colonne** et non adresse.
- `=LIGNES(TableVentes[#Données])` et `=NBVAL(TableVentes[#En-têtes])` — les specs `[[#Données]]`, `[[#En-têtes]]`, `[[#Tout]]`, `[[#Lignes]]` désignent les parties du tableau : elles rendent 480 et 13, et elles ne comptent donc jamais la ligne de total avec les données.

Dans la feuille `ventes`, la colonne N — le montant recalculé de l'exercice C02 — est **restée hors du tableau** : la 14^e colonne n'a pas été intégrée à la création. Elle fonctionne, mais elle n'est ni filtrable avec le tableau, ni comptée dans ses totaux, ni reprise par un tableau croisé fondé sur `TableVentes`. Réglez-le : clic dans N1, puis Création de tableau → Taille du tableau portée à A1:N481.

À retenir. Une formule en références structurées écrite **hors** du tableau tombe en `#VALEUR!`. Le `[@colonne]` n'a de sens qu'à l'intérieur des lignes de la table : pour une cellule de contrôle posée à côté, écrivez `TableVentes[montant_ttc]` sans @.

5.3 Trier : la ligne est l'unité, pas la colonne

Le tri honnête a trois règles.

1. **Cliquer dans le tableau**, puis Données → Trier : la portée est automatique, aucune boîte de dialogue d'avertissement. Sur une plage, le même geste affiche « Étendre la sélection » / « Continuer la sélection » : la seconde option est celle qui découpe les lignes. Ne la prenez jamais pour gagner deux clics.
2. **Trier à plusieurs niveaux**, parce qu'un seul ne suffit jamais en gestion : date décroissante, puis `n_ticket`, puis `heure`. Sans le second niveau, les lignes d'un même ticket sont dans un ordre qui change à chaque tri, et deux impressions du même classeur ne se comparent pas.
3. **Consigner le tri** dans Aidez-moi (« ordre de présentation : date, ticket, heure »). Un fichier trié n'est pas un fichier modifié, mais il n'est plus le fichier reçu : le dire évite une après-midi de débat sur « qui a changé l'ordre des lignes ».

Deux conséquences mesurées. Un tri **décroissant sur le montant** amène en tête la ligne 1 229 678 FCFA du 2 septembre — la plus forte ligne de l'extrait, et la seule à six chiffres avec trois autres. Un tri **de la colonne date telle qu'elle est reçue** (texte) ne rend pas un calendrier : il rend deux blocs, les écritures ISO et les écritures à la française, rangées caractère par caractère — le contrôle est immédiat : découpez la date sur sept caractères pour compter les mois, et vous obtenez 36 groupes là où il y a 12 mois.

5.4 Filtrer, et vérifier le filtre

Les boutons ▼ d'un tableau ouvrent trois familles de filtres : cases à cocher (texte), filtres de nombre (égal, entre, supérieur à, 10 premiers), filtres de date (mois dernier, cette année, entre). Trois réflexes.

- **Vérifier à la barre d'état**, pas à l'œil : après un filtre, Nombre de valeurs numériques doit rendre le compte attendu. Sur `ventes`, filtrer la catégorie Matériaux laisse 143 lignes sur 480 — si vous lisez 480, le filtre n'est pas où vous croyez (souvent : il est sur la page, pas sur le tableau).

- **Repérer le filtre qui ne filtre rien** : filtrer `magasin` sur « Magasin 5 — Kaya Marché » rend 480 lignes sur 480. L'extrait ne contient qu'un magasin ; le filtre est une perte de temps, et pire, une fausse garantie.
- **Ne pas confondre masquer et effacer** : Effacer les filtres rend tout ; masquer des lignes à la main ne se documente pas et se propage aux impressions.

Un filtre change ce que voient les formules : `=SOUS.TOTAL(9;TableVentes[montant_ttc])` (ou la ligne de total du tableau, qui l'utilise) ne somme que les lignes visibles, alors que `=SOMME(TableVentes[montant_ttc])` somme tout, filtre actif ou non. C'est la différence entre « le chiffre à l'écran » et « le chiffre du fichier » — les deux sont justes, ils ne répondent pas à la même question, et une note de bas de page doit dire lequel est cité.

Attention. Les filtres d'un tableau ne se partagent pas comme on l'espère : deux personnes ouvrant le même classeur filtrent l'une la feuille, l'autre le tableau croisé construit dessus, et les deux chiffres diffèrent à l'écran sans que rien ne le signale, sauf le petit entonnoir dans la barre d'onglet. Avant d'envoyer un classeur filtré, enregistrez une copie filtres effacés, et dites dans le courriel lequel des deux est la version de contrôle.

5.5 Rechercher et remplacer, avec ses jokers

`Ctrl + F` cherche, `Ctrl + U` remplace (dans les versions anglaises, `Ctrl + H`). Les trois options qui changent tout : Regarder dans — **Formules** ou **Valeurs** (sur une colonne de montants texte, « Formules » trouve le texte, « Valeurs » trouve le nombre) ; Respecter la casse (utile sur les codes ticket, qui sont sensibles aux lettres) ; Dans — **Feuille** ou **Classeur** (une recherche sur le classeur traverse `Aidez-moi`, les 18 lignes d'en-tête comprises, et y retrouve d'anciennes valeurs : le faux positif classique d'une relecture).

Les jokers `*` (n'importe quelle suite) et `?` (un caractère) font le travail d'un filtre : chercher `*réf 2` rend tous les produits de référence 2. Pour chercher un astérisque lui-même, écrivez `~*`. Enfin, remplacer n'est **pas** un nettoyage : « remplacer `.` par `,` » dans la colonne `remise` du fichier reçu corrige 489 écritures d'un coup, mais le même `Ctrl + U` lancé sur la colonne `montant_ttc` écrase les séparateurs de milliers — 56 658 devient alors une valeur impossible à relire. On restreint la recherche à une colonne (sélectionner l'en-tête de colonne dans le tableau, ou `Ctrl + barre d'espace` sur la plage) avant de remplacer quoi que ce soit.

5.6 Supprimer les doublons : la clé décide de tout

Données → Supprimer les doublons demande d'abord **quelles colonnes définissent le doublon**. Le résultat, sur le fichier reçu, tient en trois lignes, et c'est tout l'enjeu du geste :

Colonnes cochées	Lignes retirées	CA retiré	Verdict
les 13 colonnes	9	266 132 FCFA	le vrai doublon : la même ligne, deux fois
<code>n_ticket + produit</code>	12	481 978 FCFA	trop large : 3 ventes distinctes sont sacrifiées
<code>n_ticket</code> seul	173	11 424 263 FCFA	catastrophique : un ticket porte plusieurs lignes

Pourquoi « trop large », ce coup-ci : dans la donnée propre, il reste 3 lignes où le même ticket vend le même produit deux fois, à des heures différentes et sous deux vendeurs — un retour et un réassort dans la même journée, ou une saisie tardive. Avec la clé `ticket + produit`, on les supprime ; avec la clé les 13 colonnes, on les garde, parce que leur heure diffère.

Le protocole qui rend le geste défendable, dans l'ordre :

1. **Copier la table de travail** (ou travailler sur la copie de `ventes_brutes`) : la suppression est destructrice, sans bouton arrière une fois la sauvegarde close.
2. **Compter avant** : 489 lignes, 36 339 317 FCFA.

3. **Cocher la clé, et l'écrire** — « clé : les 13 colonnes » dans la fiche.
4. **Supprimer, compter après** : 480 lignes, 36 073 185 FCFA — 9 lignes et 266 132 FCFA d'écart, soit 0,74 % du total net, à rapporter dans la note.
5. **Contrôler ce qui a disparu** : les 9 lignes retirées valaient exactement le compte ; sur la donnée propre, le même contrôle rend 0.

Définition. Un doublon n'existe pas dans les données, il existe **relativement à une clé**. La même ligne peut être un doublon (clé : ticket + produit), une seconde vente (clé : les 13 colonnes) ou un élément d'un ensemble normal (clé : ticket, 173 lignes « supprimables » qui sont toutes des ventes réelles).

Boîte à outils. Ctrl + L créer un tableau · Ctrl + Maj + L basculer les boutons de filtre · données. Données → Trier n'a pas de raccourci : passez par le ruban · Alt + ↓ ouvrir le menu filtre de la cellule active · Ctrl + F / Ctrl + U chercher et remplacer · Ctrl + Maj + = insérer une ligne dans le tableau (la portée s'étend) · Tab depuis la dernière cellule du tableau crée la ligne suivante · clic droit dans un tableau → Tableau → Taille du tableau pour reprendre la 14^e colonne oubliée. Dans LibreOffice Calc, la base de données joue le rôle du tableau (en-têtes + Ctrl + Maj + * pour définir la plage) ; le tri et les filtres automatiques sont identiques, la ligne de total avec SOUS.TOTAL() aussi.

5.7 Ce que le tableau ne protège pas

Le tableau est une structure, pas une vigie. Quatre choses lui échappent, et chacune a son garde-fou ailleurs dans le module. Le **typage** : les montants texte (486 sur 489 dans le fichier reçu, les 489 dans la feuille telle qu'elle est livrée) vivent très bien dans un tableau nommé, avec une ligne de total à 0 FCFA, ou à 2 444 FCFA selon le chemin d'import — c'est le chapitre C02 qui règle ça. La **provenance** : une table copiée d'un autre classeur garde son nom, ou le voit doublé en TableVentes_1 à la collision, et rien ne signale que deux tables du même classeur portent la même donnée. Le **tri externe** : si un collègue trie la plage dans un autre tableur, puis recolle les lignes dans le tableau, la structure reste intacte et le contenu est décousu — le tableau ne sait pas qu'une ligne a bougé. Enfin la **longueur de l'historique** : le tableau grandit vers le bas, jamais vers les colonnes d'un mois à l'autre ; la grille mensuelle se construit en C07, pas ici.

Définition. Une **colonne calculée** d'un tableau est une formule saisie une fois : le tableur la propage à toutes les lignes et la recompose à chaque ligne nouvelle. Écrire =ARRONDI([@quantite])*[@prix_unitaire_ht];0) en N2 remplit les 489 lignes ; l'oublier, c'est avoir 480 lignes justes et la 481^e vide, sans message.

Attention. La ligne de total n'est pas une ligne de données : =SOMME(TableVentes[montant_ttc]) posée dans le tableau renvoie le total **deux fois** si la ligne de total est comprise dans la portée, et le contrôle interne ne voit alors qu'un chiffre plausible. Deux réflexes : calculez les totaux de contrôle **hors** du tableau, et comparez-les à la ligne de total — l'écart doit être nul, ou expliqué par un filtre actif.

6. Exemple concret : le même fichier, trois sorts

Sur la copie ventes_a_structurer (489 lignes, montants en texte, filtre automatique sur A1:M490) :

Geste	Résultat à l'écran	Résultat au contrôle
Trier la colonne montant_ttc décroissant, « Continuer la sélection »	la plus grosse valeur en tête	lignes décousues : le CA est faux, aucune erreur affichée

Geste	Résultat à l'écran	Résultat au contrôle
Trier dans le tableau <code>TableVentes</code> , trois niveaux	la ligne 1 229 678 FCFA du 2 septembre en tête	480 lignes, 36 073 185 FCFA : rien n'a bougé au total
Supprimer les doublons, clé <code>n_ticket</code> seul	316 lignes affichées	11 424 263 FCFA envolés — 173 vraies ventes
Supprimer les doublons, clé les 13 colonnes	480 lignes	266 132 FCFA retirés, et c'est le bon chiffre

Le troisième geste est celui qu'on voit en entreprise, parce que « un ticket = une vente » est une hypothèse naturelle et fausse : le fichier de caisse écrit une ligne par article.

7. Démonstration pas à pas : de la plage filtrée au tableau qui grandit

Sept étapes sur la copie `ventes_a_structurer`.

1. Sonder. Clic n'importe où dans la zone, `Ctrl + *` : la sélection s'arrête en M490 — le tableur connaît la plage, pas une table. `Ctrl + Fin` mène à la même conclusion : dernière cellule M490.

2. Créer. `Ctrl + L`, « mes données contiennent des en-têtes » : le tableau naît sur A1:M490, nom `TableVentes_brutes`. Dans Création de tableau, allumer **Ligne de total**, choix `Total` et Aucun colonne par colonne, `Somme sur montant_ttc`.

3. Vérifier la portée. Saisissez une ligne en 491 (un ticket, une date, un montant) : elle entre dans le tableau, la ligne de total passe à 490 lignes de données, et A1:M491 s'affiche dans Taille du tableau. La même ligne saisie **en laissant un rang vide** n'y entre pas : le contrôle est exactement celui-là.

4. Convertir. `CNUM(SUBSTITUE(...))` sur `montant_ttc`, collage en valeurs : 489 lignes, la ligne de total rend 36 339 317 FCFA (rappel du chapitre C02 — convertir et dédoubler sont deux gestes, et ils se mesurent séparément).

5. Retirer les doublons. Clé : les 13 colonnes. Résultat attendu : 480 lignes et **36 073 185 FCFA**. Ces deux nombres sont la preuve du §5.6 et l'objet du contrôle 2 ; l'écart, 266 132 FCFA, se reporte dans la note.

6. Trier et filtrer. Données → Trier : niveau 1 `date` décroissant, niveau 2 `n_ticket`, niveau 3 heure. Puis filtre sur `categorie` = Matériaux : la barre d'état rend Nombre 143 et la ligne de total, elle, rend le chiffre **filtré** puisque `SOUS.TOTAL()` ne voit que le visible — notez les deux nombres, c'est la leçon du chapitre en une paire.

7. Consigner. Dans `Aidez-moi`, une ligne par geste : tableau (nom, portée), ordre de tri, clé de dédoublonnage et ce qu'elle a retiré, filtre actif à l'envoi.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Portée du tableau après une ligne collée en contiguïté	identité	A1:M491 puis A1:M490 après retrait	collage avec ligne vide, portée non vérifiée
2	Total de la ligne <code>Total</code> , après conversion et dédoublonnage	±1 FCFA	36 073 185 FCFA	un seul des deux gestes a été fait
3	Nombre de lignes retirées, clé 13 colonnes	identité	9	clé trop large (12) ou absurde (173)
4	Lignes visibles sous filtre Matériaux	identité	143	filtre posé sur la plage, pas sur le tableau
5	<code>SOUS.TOTAL</code> filtré contre <code>SOMME</code> total	deux nombres	les deux différents, et le second est 36 073 185	ligne de total écrasée par une formule manuelle

8. Erreurs fréquentes

1. **Créer un tableau et garder le nom `Tableau1`.** Trois tableaux plus tard, `Tableau3` n'évoque rien ; les formules deviennent illisibles et le rapprochement avec une requête devient un jeu de devinettes.
2. **Trier une colonne au lieu de trier des lignes.** Le pire défaut de ce module, parce qu'il ne lève aucune alerte : seule une comparaison de totaux ligne à ligne le rattrape.
3. **Cocher `n_ticket` seul pour dédoublonner.** Le tableur ne juge pas la clé, il l'exécute. Écrivez-la avant de cliquer, dans la fiche, avec le compte attendu.
4. **Compter les lignes filtrées à l'œil.** L'œil ne voit que ce qui tient à l'écran : la barre d'état dit 143, l'œil disait 40.
5. **Remplacer sans restreindre la zone.** `.` → `,` sur le classeur entier a touché les montants, les tickets et une partie de `Aidez-moi`. Une colonne sélectionnée, pas un classeur.
6. **Croire le tableau immunisé contre les erreurs de typage.** Il structure, il ne convertit pas : les 486 textes du fichier reçu vivent très bien dans un tableau parfaitement nommé.

9. Bonnes pratiques professionnelles

1. **Un tableau, un nom qui dit la matière, une portée vérifiée à chaque import.** `TableVentes`, `TabClients`, `TabObjectifs` ; et la portée lue dans Taille du tableau, pas devinée.
2. **Toujours deux niveaux de tri minimum** (la période, puis la clé du document) : un fichier dont l'ordre est reproductible se relit, se compare et s'imprime deux fois à l'identique.
3. **Le dédoublonnage est un livrable.** Une ligne de fiche, une mesure avant/après, une raison. Sinon, ce n'est pas un nettoyage, c'est une soustraction non autorisée.
4. **Les contrôles en colonnes calculées du tableau,** pas en cellules collées à côté : elles héritent de l'extension automatique, là où la colonne N de `ventes` est restée à la traîne.
5. **Un onglet « filtres effacés » pour l'envoi, un onglet de contrôle pour l'archive.** Le destinataire doit voir le chiffre et pouvoir le défaire.

Conseil professionnel. Le jour où l'on vous demandera « combien de lignes au juste ? », ne répondez jamais d'un nombre lu dans une cellule isolée. Répondez par la paire : 480 lignes, 36 073 185 FCFA — la ligne de total est en bas du tableau, le nombre de lignes est dans la barre d'état, et un chiffre sans son compte de lignes ne vaut pas une réponse.

10. Exercice guidé — la table qui tient debout (35 min, / 10)

Commande. Sur la copie `ventes_a_structurer`, menez les six étapes, et notez à chacune le nombre de lignes et la somme affichés.

Étape	Geste	Ce qui doit se lire
1	<code>Ctrl + L</code> , nom <code>TableVentes_brutes</code> , ligne de total sur <code>montant_ttc</code>	portée A1:M490
2	Coller une ligne en 491, juste sous la table	portée A1:M491, total inchangé (le texte ne se somme pas)
3	Supprimer la ligne 491	retour à A1:M490
4	Supprimer les doublons, clé <code>n_ticket</code> seul, sur une seconde copie	316 lignes, 24 915 054 FCFA — le chiffre du désastre
5	Sur la première copie, clé les 13 colonnes	480 lignes

Étape	Geste	Ce qui doit se lire
6	Filtre <code>categorie = Matériaux</code> , lire la ligne de total	143 lignes visibles, total filtré 6 519 357 FCFA

Démarrage. Les étapes 4 et 5 sont le cœur de l'exercice : même bouton, même fichier, deux clés, 173 lignes d'écart. Notez les deux nombres avant de commenter ; l'explication viendra toute seule. Pour l'étape 6, le montant attendu est celui de la catégorie Plomberie ou Matériaux selon le filtre choisi — vérifiez que la ligne de total suit le filtre (`Sous.Total`) et que `=SOMME(TableVentes_brutes[montant_ttc])` reste, lui, au total général.

Barème (10 points). Tableau nommé et portée vérifiée (2) · extension démontrée à l'étape 2 puis annulée (2) · les deux dédoublonnages chiffrés côte à côte (3) · filtre contrôlé à la barre d'état et non à l'œil (1) · fiche `Aidez-moi` complétée des trois lignes (tri, clé, filtre) (2).

11. Exercices autonomes

Exercice 3.1 (★) — La portée, l'œil ouvert. Sur `ventes`, créez une seconde vue : filtrez `categorie` sur les sept catégories une à une, notez lignes et total filtré pour chacune, puis contrôlez la cohérence : les 7 compteurs de lignes doivent restituer 480 et les 7 totaux restituent 36 073 185 FCFA.

Exercice 3.2 (★) — Un tri qui se rejoue. Tri triple (date décroissante, `n_ticket`, heure), puis renouvelez le tri cinq fois de suite. Contrôlez : le fichier est-il identique à chaque fois ? Que se passe-t-il si vous retirez le deuxième niveau ? Répondez en comparant deux lignes d'un même ticket.

Exercice 3.3 (★★) — La colonne orpheline. Reprenez la colonne N de `ventes` (le montant recalculé, restée hors du tableau). Intégrez-la, puis démontrez trois choses : son en-tête dans le filtre, sa place dans la ligne de total, et ce que rendait `=SOMME([montant recalculé (colonne utilitaire)])` avant l'intégration.

Exercice 3.4 (★★) — Le faux doublon. Les 3 lignes où le même ticket vend le même produit deux fois dans la donnée propre : extrayez-les par un filtre sur `n_ticket` suivi d'un tri, dressez le tableau (heure, vendeur, quantité, montant), et écrivez la phrase qui justifie de les conserver.

Exercice 3.5 (★★) — Rechercher-remplacer sous contrôle. Sur une copie, remplacez `.` par `,` dans la seule colonne `remise`, puis convertissez la colonne en pourcentage ; contrôlez par un compteur (489 écritures, 173 lignes non nulles) et par la borne (11 %). Écrivez ce qu'aurait produit le même geste lancé sur le classeur entier.

12. Correction détaillée

Exercice 3.1. Les sept compteurs rendent 143 (Matériaux), 112 (Quincaillerie), 57 (Plomberie), 57 (Consommables), 53 (Électricité), 34 (Peinture), 24 (Bois & panneaux) — total 480 ✓ ; les sept totaux, dont 8 754 982 FCFA pour la Plomberie et 6 519 357 FCFA pour les Matériaux, restituent exactement 36 073 185 FCFA. Un seul filtre omis suffit à déséquilibrer la paire : c'est un contrôle de cohérence, pas un exercice de comptage.

Exercice 3.2. Le tri est idempotent à trois niveaux ; sans le niveau `n_ticket`, les lignes d'un même ticket changent d'ordre d'un tri à l'autre (elles partagent la date), et deux impressions ne se comparent plus. Exemple typique : le ticket le plus fourni de l'extrait, 4 lignes, se retrouve dans un ordre différent selon la version du tableur.

Exercice 3.3. Avant intégration, la colonne N n'est ni filtrable avec le tableau, ni comprise dans ses totaux, et `=SOMME(TableVentes[montant recalculé (colonne utilitaire)])` tombe en `#NOM?` — pour le tableau, cette colonne n'existe pas. Après Taille du tableau portée à `A1:N481`, l'en-tête apparaît dans les filtres et la ligne de total propose un calcul sur N.

Exercice 3.4. Trois paires, mêmes dates et mêmes produits, mais heures distinctes (09:58 contre 18:54 par exemple), et deux vendeurs différents sur deux des trois cas. Conclusion attendue : « même ticket, même produit, lignes distinctes — pas de doublon : la clé retenue est l'ensemble des colonnes, pas une paire partielle ».

Exercice 3.5. Sur la colonne seule : les 489 remises deviennent numériques, 173 non nulles, maximum 11,0 % ; au format pourcentage, plus aucune ligne ne se lit en points. Sur le classeur entier, le même `Ctrl + U` casse les montants (les espaces de milliers et les points se télescopent), les codes ticket, et la moitié de `Aidez-moi` : la recherche-restreinte n'est pas un raffinement, c'est la condition du geste.

13. Mini-projet M03.P1 — suite : le volet structure (45 min)

Commande. Livrez le classeur `atelier_structure.xlsx` : une table structurée de la vente, un volet de contrôle, une documentation des gestes, pour qu'un lecteur rouvre le fichier six mois plus tard sans appeler personne.

Livrables numérotés. (1) le tableau `TableVentes_brutes` nommé, borné, avec ligne de total, et sa portée écrite dans `Aidez-moi` ; (2) le tableau des deux dédoublonnages (clé, lignes retirées, CA retiré) ; (3) l'ordre de tri à trois niveaux, consigné, et la preuve qu'il est reproductible (deux listings datés) ; (4) le volet de contrôle : une colonne calculée du tableau qui compte les lignes du ticket (`=NB.SI.ENS(TableVentes_brutes[n_ticket];[@[n_ticket]])` au lieu d'un `NB.SI` sur adresse) et signale les tickets à lignes multiples ; (5) la note d'une page : ce qui a été retiré, ce qui a été gardé et pourquoi, avec les deux chiffres d'avant/après.

Barème (20 points, seuil 13). Tableau nommé, porté, documenté (4) · dédoublonnage chiffré sur les deux clés (5) · tri consigné et reproductible (3) · colonne de contrôle en références structurées (4) · note : style, mesure, justification de la clé (4).

14. Résumé du chapitre

Le passage de la plage au tableau est un changement de statut : la donnée devient un objet nommé, qui grandit, se filtre, se trie et se totalise avec sa portée. Le nom fait la formule (`TableVentes[montant_ttc]` survivra à l'import du mois prochain, `M2:M490` non), la ligne de total fait le chiffre visible (`SOUS.TOTAL()` suit le filtre, `SOMME()` ignore), et la contiguïté fait l'extension — une ligne vide suffit à laisser 40 ventes dehors.

Le tri se fait sur des lignes, à deux ou trois niveaux, jamais sur une colonne : le seul défaut de ce chapitre qui ne laisse aucune trace à l'écran. Les filtres se vérifient à la barre d'état (143 lignes, pas « à peu près 40 »), et un filtre qui laisse passer 480 lignes sur 480 est un filtre inutile. La recherche se restreint avant de remplacer. Le dédoublonnage enfin, ne contient aucune intelligence : il retire ce que la clé lui dit de retirer — 9 lignes pour les 13 colonnes, 12 pour ticket + produit, 173 pour le ticket seul, 11 424 263 FCFA de ventes réelles dans le dernier cas. Écrire la clé avant de cliquer, compter avant et après : voilà tout le métier de ce chapitre.

15. À retenir

À retenir. Une adresse fige, un tableau nomme. Tout ce qui est écrit sur `TableVentes` suivra la donnée ; tout ce qui est écrit sur `A1:M481` restera en 481, en silence.

À retenir. On trie des lignes, pas des colonnes. Le tableur avertit une fois et exécute toujours : « Continuer la sélection » est la case qui découd un fichier.

À retenir. Un doublon se définit par une clé, jamais par une apparence : 9 lignes, 12 lignes ou 173 lignes, et 266 132 FCFA ou 11 424 263 FCFA selon la case cochée. La clé s'écrit avant le clic.

1. Le nom du tableau se choisit comme une table de base de données, pas comme un onglet.
2. Une colonne calculée appartient au tableau : sinon elle ne grandit pas avec lui.
3. Un filtre actif se déclare dans le courriel, ou se détruit avant l'envoi.
4. `SOUS.TOTAL()` et `SOMME()` donnent deux chiffres justes : précisez lequel est cité.
5. Le dédoublonnage est destructeur : il se fait sur copie, avec le compte d'avant noté.

16. Évaluation formative (auto-correction, 10 min)

1. Quelle est la différence, à l'import du mois suivant, entre un tableau croisé fondé sur `TableVentes` et un tableau croisé fondé sur `ventes!A1:M481` ?
2. Citez les deux lectures d'une même sélection filtrée (`SOUS.TOTAL(9;...)` et `SOMME(...)`) et dites laquelle on met dans une note de synthèse.
3. Vous trie une plage sélectionnée colonne par colonne : quelle boîte de dialogue apparaît, quelle option prenez-vous, et que se passe-t-il si vous prenez l'autre ?
4. Un collègue a dédoublonné sur `n_ticket`. Quels deux nombres regardez-vous en premier pour estimer les dégâts, et quelles valeurs attendez-vous ici ?
5. Pourquoi la colonne utilitaire de `ventes` est-elle un défaut, alors même que toutes ses formules sont justes ?

Question ouverte. Écrivez les trois lignes de fiche que doit contenir `Aidez-moi` après ce chapitre — pas de conseils généraux, des faits sur ce classeur.

Corrigé de l'évaluation formative.

1. Le premier voit les 40 nouvelles lignes (la portée du tableau suit la donnée) ; le second s'arrête en 481 et rend le mois précédent sans prévenir. C'est la raison pour laquelle les requêtes et TCD du module se branchent sur des noms.
2. `SOUS.TOTAL(9;TableVentes[montant_ttc])` ne somme que les lignes visibles, donc le chiffre de l'écran filtré ; `SOMME(TableVentes[montant_ttc])` rend 36 073 185 FCFA, le chiffre du fichier. Dans une note, on cite le second et on mentionne le premier, avec le filtre qui le produit.
3. « Avertissement : le tri s'appliquera à la zone spécifiée » — on choisit Étendre la sélection. Avec Continuer la sélection, seule la colonne triée bouge : chaque montant se retrouve attaché à un autre ticket, le total général reste juste, et la table est fautive ligne par ligne.
4. Le nombre de lignes et le total. Ici : 489 lignes et 36 339 317 FCFA avant, 316 lignes et 24 915 054 FCFA après — 173 lignes et 11 424 263 FCFA de ventes réelles effacées. Le fichier est à reconstruire depuis la source, pas à corriger.
5. Parce qu'une colonne hors du tableau ne grandit pas avec lui, ne se filtre pas avec lui, ne se totalise pas avec lui : la formule est juste aujourd'hui, et fautive au premier import. La correction tient en un réglage de portée (Taille du tableau), pas en une réécriture. Question ouverte

— Réponse attendue, trois faits datés : « TableVentes, portée A1:N481, style sans bandes » ; « ordre de présentation : date décroissante, n_ticket, heure » ; « dédoublonnage du ... : clé = les 13 colonnes, 9 lignes retirées, 266 132 FCFA, copie de travail ventes_a_structurer ».

Suite du module. Le chapitre C04 ouvre la fabrique des formules : références relatives, absolues et mixtes, ce que F4 fait vraiment, et les adresses d'erreur (#N/A, #VALEUR!, #DIV/0!, #REF!) — dont celle que produit le [@colonne] écrit hors d'un tableau.

Module M03.C04 — La formule : références relatives, absolues, mixtes, et les sept erreurs

Outil de ce chapitre : le tableur seul, sur le classeur de l'atelier. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. Une formule n'est pas un calcul, c'est un **déplacement**. Vous écrivez une phrase pour une ligne, le tableur la rejoue sur les 479 autres, et c'est ce jeu qui marche ou qui casse. Le chapitre tient donc à deux questions : qu'est-ce qui bouge quand je recopie ? et qu'est-ce que cette erreur est en train de me dire ?

Base de travail. Le classeur `01_socle_donnees/data/projection/m03_classeur_atelier.xlsx`, et deux de ses feuilles : `ventes` (480 lignes, colonne N = montant hors taxes recalculé) et `Calculs` — 28 fiches de formules, une par ligne, avec la valeur attendue, la valeur rendue et un contrôle automatique. Le fichier reçu `01_socle_donnees/data/projection/ventes_magasin5_2025.csv` sert de terrain de chasse aux erreurs. Séparateur point-virgule, UTF-8, franc CFA (FCFA), virgule décimale, TVA 18 %. Chiffres cités : `01_socle_donnees/data/reference/chiffres_cites.md`, section M03. On travaille dans une copie du classeur nommée `formules_atelier` : la feuille `Calculs` se lit, elle ne se corrige pas.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Écrire** une formule lisible : opérateurs, ordre de calcul, parenthèses — et prévoir ce qu'elle vaudra ligne 481.
2. **Distinguer** référence relative, absolue et mixte, et choisir la bonne à la seule lecture du besoin.
3. **Lire** une erreur au lieu de la masquer : `#VALEUR!`, `#N/A`, `#DIV/0!`, `#RÉF!`, `#NOM?`, `#NULL!`, `####`.
4. **Comprendre la propagation** : une cellule fausse en contamine cinquante, et `SIERREUR` n'est pas un pansement.
5. **Arrondir au bon endroit** : dans la valeur ou dans l'affichage, mais pas les deux — et dire pourquoi le total du classeur bouge de 4 FCFA selon ce choix.
6. **Faire parler le classeur d'atelier** : ses 640 cellules à formule, ses noms stockés en anglais, son auto-contrôle.

2. Pourquoi cette notion est importante

Situation 1 — le taux qui ne bouge pas d'un cheveu. Une facturière calcule le TTC sur 480 lignes avec `=L2*1,18`, le taux étant écrit dans la formule. Le taux change au 1^{er} septembre : elle corrige la ligne 2, recopie — les lignes déjà relues et verrouillées en haut du fichier restent à l'ancien taux, et le total ne le dit pas. La formule juste est `=L2*TauxTVA`, avec le taux dans une cellule nommée : une saisie, 480 lignes à jour, zéro report.

Situation 2 — l'erreur qui n'en est pas une. Le tableau de bord affiche `#DIV/0!` en face d'un vendeur. Ce n'est pas une panne : c'est la réponse exacte à une question mal posée — ce ven-

deur n'a aucune ligne dans la période filtrée. Le masquer par un **SIERREUR(...; "")** transforme un diagnostic en mensonge : le tableau devient propre, et le trou devient invisible au comité.

Situation 3 — la suppression qui a tout emporté. Une colonne « heure », jugée inutile, est supprimée. Les formules qui lisaient K2:K481 ne disparaissent pas : elles deviennent **#RÉF!**, y compris celles que personne ne regardait, dans **TCD** et dans **Mensuel**. Le tableur n'a pas cassé le calcul, il a dit très clairement qu'on lui avait retiré sa matière — et personne n'a lu.

Dans les faits. Le contrôle de qualité d'un classeur de gestion commence par deux gestes de dix secondes : **Ctrl + '** pour afficher toutes les formules de la feuille, et une recherche de **#** dans le classeur entier. Ils trouvent plus de défauts que leur réputation ne le laisse croire, parce qu'ils portent sur tout — y compris les cellules que personne n'ouvre jamais.

3. Explication simple

Une formule est une phrase : un signe égal, des références, des opérateurs, des fonctions. Elle ne stocke pas un résultat, elle stocke une **instruction de calcul**, et le résultat est recalculé à chaque changement.

Quand vous recopiez une formule, le tableur ne copie pas le texte : il le **translate**. **=L2*1,18** recopié une ligne plus bas devient **=L3*1,18**. C'est la référence **relative** : elle désigne un décalage, pas une adresse. Le signe **\$** fixe un bord : **\$L\$2** ne bouge jamais (absolue), **L\$2** fige la ligne et laisse la colonne voyager (mixte). La touche **F4** fait tourner les trois écritures dans l'ordre.

D'où la règle du module, déjà entrevue au chapitre précédent : **ce qui sert de règle se fixe, ce qui se répète se laisse relatif**. Le taux de TVA est une règle → absolue. La ligne de vente est un calcul → relatif.

Les erreurs, elles, ne sont pas des pannes : ce sont des messages. Sept formes, sept causes distinctes. Savoir les lire divise par dix le temps de diagnostic ; les masquer le multiplie par cent.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Référence relative relative reference	— L2 : un décalage depuis la cellule de la formule.	Recopier la référence d'une règle : chaque ligne prend son propre décalage.
Référence absolue absolute reference	— \$L\$2 : adresse figée, insensible au report.	Figurer ce qui doit bouger : 480 lignes avec le même montant.
Référence mixte mixed reference	— L\$2 ou \$L2 : un seul bord figé.	Ne pas la connaître : elle règle d'un coup les tableaux à double sens.
Report — fill	Glisser la poignée, double-clic, Ctrl + D , ou Accueil → Modifier → Remplir.	Reporter 480 lignes d'une formule fautive à la ligne 2.
Référence structurée structured reference	— TableVentes[montant_ttc], [@quantite] .	Écrire un [@...] hors du tableau : #VALEUR! .
Référence circulaire circular reference	— Une formule qui se lit elle-même, directement ou par chaîne.	Ignorer l'avertissement d'ouverture et valider des valeurs figées.
Propagation d'erreur error propagation	— Toute fonction recevant une erreur la renvoie.	Entourer la chaîne entière de SIERREUR et perdre l'adresse du défaut.
Fonction volatile volatile function	— Recalculée à chaque frappe (AUJOURDHUI , DECALER , INDIRECT).	En parsemer 480 lignes et attendre la réactivité d'une application.

Définition. Le **maillage** d'une formule est l'ensemble des cellules qu'elle lit. Deux formules qui se recouvrent sans le savoir — la colonne enregistrée `montant_ht` et la colonne N qui la recalcule — produisent un **écart de contrôle** : 4 FCFA ici. Un maillage documenté rend l'écart explicable ; un maillage implicite le rend suspect, et un écart suspect se paie en demi-journée de réunion.

Définition. Le **sens du report** est la direction où le décalage s'applique : vers le bas pour une colonne de lignes, vers la droite pour une ligne de mois. Une référence mixte `L$2` se comporte comme une ordonnée : la ligne est commune à toute la grille, la colonne suit le déplacement. C'est elle qui écrit une grille mensuelle en une seule saisie au lieu de quarante-huit.

5. Cours approfondi

Recopier vers le bas : ce qui bouge, ce qui reste

	A	B	C	D
1	taux TVA	1,18		
2	ligne 2	48 015	<code>=B2*\$B\$1</code>	56 658
3	ligne 3	78 402	<code>=B3*\$B\$1</code>	92 514

C3 est C2 recopiée : B2 → B3, \$B\$1 n'a pas bougé.

B2 suit le déplacement : \$B\$1 reste : B\$1 et \$B2 ne bloquent chacun qu'un seul bord.

Règle du module : une donnée = une plage absolue, un calcul = une ligne relative. F4 fait tourner les trois écritures.

Les erreurs, et ce qu'elles disent vraiment

#N/A clé absente de la colonne cherchée

#VALEUR! du texte là où un nombre est attendu

#DIV/0! dénominateur nul, souvent un filtre actif

#RÉF! la plage n'existe plus, ligne supprimée

#NOM? fonction inconnue, nom anglais ou faute

pas une erreur : colonne trop étroite

Le tableur affiche ce qu'il a trouvé, rien de plus.

5.1 Les opérateurs, et l'ordre qui les range

Six familles, un ordre : $\wedge$ d'abord, puis $*$ et $/$, puis $+$ et $-$; les comparaisons ($=$, $<>$, $>=$) et la concaténation $\&$ viennent après ; le texte se compare par ordre alphabétique, pas par longueur. Quatre tests à faire vous-même dans une cellule libre :

- `=2+3*4` rend 14, `=(2+3)*4` rend 20 : la parenthèse n'est pas une précaution, c'est la ponctuation du calcul.
- `=1/2` rend 0,5 — si la cellule affiche 0, le format masque les décimales et la valeur est juste : on regarde la barre de formule, pas la grille.
- `= "11"&" mois"` rend 11 mois, c'est-à-dire un **texte** : toute formule qui le somme rendra **#VALEUR!**.
- `= -2^2` rend -4 quand `= (-2)^2` rend 4 : l'exposant s'attache au nombre, pas au signe.

Définition. L'**ordre de priorité** est la convention qui range les opérateurs : puissances, puis multiplications et divisions, puis additions et soustractions, puis comparaisons et concaténation. Un tableur ne « calcule pas de gauche à droite » : il applique cette hiérarchie, et rien d'autre. Écrire `=1-2*3` rend -5 ; écrire `=(1-2)*3` rend -3. Les parenthèses ne clarifient pas, elles **dé-cident**.

Le pourcentage est un diviseur : taper 18 puis appliquer 0,0 % vaut 0,18, pas 18. Les remises du fichier reçu, écrites 0.03, sont exactement ce malentendu — à l'envers, et multiplié à 489 lignes.

5.2 Relative, absolue, mixte : ce qui bouge, ce qui reste

La figure se lit en trois colonnes : `=B2*$B$1` recopiée vers le bas devient `=B3*$B$1` — la donnée suit le déplacement, la règle reste. Quatre situations de gestion, quatre écritures :

Besoin	Écriture	Pourquoi
Taux de TVA commun à 480 lignes	<code>=L2*TauxTVA</code>	une règle, saisie une fois, nommée une fois
Grille de mois en tête de colonne	<code>=B\$1*\$Q2</code>	la ligne 1 porte les mois, la colonne Q porte les lignes
Total d'une colonne du tableau	<code>=SOMME(TableVentes[montant_ttc])</code>	le nom remplace l'adresse et survit aux lignes nouvelles
Somme d'une fenêtre fixe, lignes 40 à 80	<code>=SOMME(\$L\$40:\$L\$80)</code>	figée, sinon le report déplace la fenêtre et le total

F4 fait tourner `L2` → `$L$2` → `L$2` → `$L2` dans cet ordre : on n'écrit jamais les \$ à la main, on les fait tourner — et l'on vérifie ensuite dans la barre de formule, pas dans le résultat. Une référence construite avec `LIGNE()`, `COLONNE()` ou `EQUIV()` ne se reporte pas « mal » : elle rend sa propre position. C'est le principe de la colonne de rang `=LIGNE()-1`, qui numérote les lignes sans les compter à la main — et qui cesse de le faire dès qu'on la convertit en valeurs.

5.3 Ce que le fichier stocke, ce que vous lisez

La feuille `Calculs` porte deux colonnes qui disent tout : « Fonction (telle que l'affiche Excel FR) » et « Formule, telle qu'elle est stockée ». La fiche 22 s'intitule Réparer une cellule et son contenu stocké est `VALUE(SUBSTITUTE(ventes_brutes!M2," ",""))`. Le fichier écrit les noms de fonction en **anglais** et sépare les arguments par des **virgules** ; l'interface française affiche les noms traduits et des points-virgules. Trois conséquences de travail :

1. Une formule venue d'un classeur anglophone s'affiche traduite chez vous : le fichier voyage mieux que la documentation. C'est pour cela qu'une fiche de formules cite le **nom français en toutes lettres**, jamais une capture d'écran.
2. Une fonction inconnue du tableur ne se traduit pas : `TEXTJOIN` ou `UNIQUE` sur une version ancienne rendent `#NOM?` — et la fiche 23 de `Calculs`, qui les emploie toutes les deux, est précisément réservée à Microsoft 365 (règle de la fiche double, §5.8).
3. `Ctrl` + `'` bascule l'affichage des formules. Le geste de l'auditeur : il révèle d'un écran les 480 formules de la colonne N de `ventes`, les recopies incohérentes, et les nombres tapés à la place d'un calcul — le défaut le plus coûteux du tableur en entreprise, parce qu'il a l'air exact.

À retenir. Un `=` n'engage pas le même calcul selon la cellule où il se trouve : la même formule, reportée, est une autre formule. On ne valide jamais la ligne 2 seule — on regarde une ligne au milieu, une ligne de retour (les montants négatifs existent ici), et la dernière ligne du fichier.

5.4 Les sept erreurs, une cause chacune

- **#VALEUR!** — le type ne convient pas. `=CNUM("56 658")`, ou une multiplication qui touche du texte. Les 486 montants texte du fichier reçu la produisent à chaque tentative de calcul direct. Le remède est la conversion (chapitre C02), pas la conversion de l'erreur en 0.
- **#N/A** — la clé est absente. `=RECHERCHEV(1229678;ventes!G2:J481;4;FAUX)`, la fiche 17 de `Calculs`, cherche un montant dans une colonne de produits. `#N/A` ne dit pas « la formule est fausse » : il dit « ici, rien ne correspond ». C'est souvent le résultat exact d'une question juste posée sur un périmètre trop court.

- **#DIV/0! — dénominateur nul.** Une moyenne sur une sélection vide, un objectif absent, un taux de réalisation sur un magasin sans ligne. Le classeur connaît le cas : deux ventes de l'extrait n'ont pas d'objectif.
- **#REF! — la plage n'existe plus.** Suppression d'une ligne ou d'une colonne référencée. La formule n'est pas réparable en l'état : on la réécrit, ou on restaure la version d'avant — d'où la copie de travail du chapitre C01.
- **#NOM? — le nom est inconnu.** Fonction d'une version plus récente, faute de frappe, ou référence structurée écrite hors du tableau.
- **#NULL! — intersection vide.** Un espace au lieu d'un point-virgule : `=SOMME(L2:L10 M2:M10)` demande l'intersection de deux plages qui ne se touchent pas.
- **#### — ce n'est pas une erreur.** La colonne est trop étroite pour l'affichage, la valeur est juste : double-clic sur le bord de la colonne et elle réapparaît. C'est la seule des sept qui ne dise rien du calcul.

Définition. Une erreur de tableur est une **valeur réservée**, qui se propage comme les autres le long du maillage — mais que `SOMME`, `MOYENNE`, `NB` et les filtres **ignorent en silence**. C'est cette asymétrie qui rend une feuille fautive et pourtant muette : `=SOMME(M2:M481)` sur une colonne contenant dix `#VALEUR!` ne rend pas une erreur, il rend un total amputé.

5.5 Propager, attraper, compter : trois gestes différents

`SIERREUR(x;"rien")` renvoie le second argument dès que `x` est une erreur — n'importe laquelle. C'est un cache, pas un contrôle. Trois écritures, par ordre de mérite :

1. **Attraper la seule erreur attendue** : `=SI(NON(DISP(RECHERCHEV(...);"absent du fichier"));` en français la fonction `IFNA` s'appelle `SI.NON.DISP`, et elle ne prend que deux arguments : la valeur, ce qu'on affiche à la place si c'est `#N/A`. Sur un rapprochement de clients, `#N/A` devient un libellé lisible, `#VALEUR!` continue de crier. La voie universelle, à connaître parce qu'elle marche aussi dans les versions anciennes : `=SI(ESTNA(RECHERCHEV(...));"absent du fichier";RECHERCHEV(...))`.
2. **Prévenir plutôt qu'attraper** : `=SI(ESTNUM(M2);M2*1,18;"montant non numérique")`. La colonne de statut du chapitre C02 est de cette famille — elle dit ce qu'elle voit au lieu de faire disparaître.
3. **Compter les erreurs au lieu de les afficher** : `=SOMME(ESTERREUR(plage)*1)`, en validation matricielle (`Ctrl + Maj + Entrée` sur Excel 2021). Attention aux deux voisines : `ESTERREUR` (anglais `ISERROR`) capte toutes les erreurs, `ESTERR` (`ISERR`) **rate** `#N/A` — la seconde est donc le mauvais outil pour un compteur. Le test ne dépend pas de la langue, alors que chercher le texte `#VALEUR!` dans un `NB.SI` dépend de l'affichage : c'est la recette qui marche chez le voisin et pas chez vous. Un nombre d'erreurs se suit d'un import à l'autre : une baisse est une amélioration, une hausse non commentée est une alerte.

La fiche 26 de `Calculs` montre le geste et son prix : `=SIERREUR(1/0;"rien à afficher")` rend le texte attendu — et rendrait la même chose pour un `#REF!` ou un `#NOM?` logé dans une cellule d'à côté. La fiche 17, elle, laisse son `#N/A` remonter **jusque dans son propre contrôle** : la colonne contrôle de cette ligne tombe en `#N/A` à son tour, parce que `EXACT` propage. Tout le chapitre est dans cette case : une erreur attrapée trop tôt devient une erreur qui ne dit plus où elle est.

5.6 Arrondir : dans la valeur, ou dans l'affichage — mais pas les deux

`ARRONDI(x;0)` modifie la valeur stockée ; le format ne modifie que son image. Le classeur d'atelier permet de mesurer l'écart, parce qu'il contient à la fois un `montant_ht` **enregistré** par la caisse et une colonne `N` qui le **recalcule** : `=ARRONDI(I2*J2*(1-K2);0)`. L'écart des deux totaux est de 4 FCFA sur 30 570 491 FCFA — et ce n'est pas une faute. Sept lignes de l'extrait ont un pro-

duit qui tombe à un demi-franc (six au test strict, la septième à un cheveu près, ce qui est déjà une leçon) ; sur quatre d'entre elles, la règle d'arrondi choisie change le franc :

Ticket	Quantité	Prix HT	Remise	Produit brut	Enregistré	Avec ARRONDI
T05-250702-096500	2	9 725	11 %	17 310,5	17 310	17 311
T05-250703-096586	2	26 625	7 %	49 522,5	49 522	49 523
T05-250801-099095	2	6 225	7 %	11 578,5	11 578	11 579
T05-251002-105562	18	5 025	3 %	87 736,5	87 736	87 737

Version alternative pour un petit écran — ce tableau de sept colonnes tient en deux phrases. Sur les quatre lignes, le produit brut finit en ,5 : la caisse a retenu l'entier pair juste en dessous, **ARRONDI** retient l'entier au-dessus. Le paragraphe qui suit dit laquelle des deux règles votre fichier doit suivre, et pourquoi.

Un franc sur une ligne, quatre sur le fichier : la caisse applique **l'arrondi bancaire** (à mi-chemin, on va vers le pair — 15 856,5 devient 15 856), Excel **ARRONDI** applique l'arrondi usuel (à mi-chemin, on s'éloigne de zéro — il devient 15 857). Reproduire l'arrondi bancaire dans une colonne de contrôle donne le total enregistré à zéro franc près : la preuve que l'écart de 4 FCFA vient de la règle, pas des données. Trois règles de métier. Un **montant destiné à être facturé** s'arrondit dans la valeur, parce que le total doit se recomposer depuis les lignes. Une **statistique** ne s'arrondit jamais avant agrégation — arrondir 480 montants puis faire la moyenne déplace le résultat, et le module M02 a montré de combien. Et **la règle s'écrit** : une ligne dans Aidez-moi, sinon un relecteur perdra une heure à chercher une faute inexistante. Notez que **sans aucun arrondi**, l'écart est de 2 FCFA, pas de 0 : les deux totaux ne coïncident pas non plus.

5.7 Ce qui casse une référence, ce qui ne la casse pas

Geste	Effet sur la formule
Insérer une ligne au milieu de la plage sommée	la plage s'étend, la formule suit — il n'y a rien à faire
Supprimer une ligne de la plage	la plage se réduit, =SOMME(L2:L479)
Supprimer la colonne référencée	#REF!, et la formule est à réécrire
Couper-coller une cellule référencée	les formules qui la lisaient sont réécrites vers la nouvelle adresse
Coller une valeur par-dessus une formule	la formule disparaît, sans trace dans le fichier
Masquer des lignes	aucun effet sur SOMME, effet total sur SOUS.TOTAL
Trier	aucun effet sur les références, effet fâcheux sur un maillage mal figé

La colonne de rang (=LIGNE()-1) ne survivra pas à une suppression de ligne si on l'a convertie en valeurs ; la colonne n_ticket lue dans le fichier, si. Un numéro de ligne est une **position**, un identifiant est une **donnée** : la première se recalcule, la seconde se protège. C'est la différence qui explique qu'un même fichier donne deux historiques différents selon la personne qui l'a ouvert.

Attention. Coller en valeurs sur une colonne de calcul supprime le calcul **sans laisser de trace** : la cellule ne se distingue plus d'une saisie manuelle, ni à l'écran, ni dans le fichier. Sur 480 lignes, l'accident est invisible au moment où il se produit et irréversible à la relecture. Deux protections valent le coup : garder la colonne source à côté (c'est ce que fait le classeur avec ventes_brutes), et noter dans Aidez-moi quelles colonnes sont calculées — la liste tient en quatre lignes.

5.8 La règle de la fiche double, et les deux silences du tableur

Deux fonctions citées ici n'existent pas partout : TEXTJOIN et UNIQUE (fiche 23), comme FILTRE, TRIBY et SÉQUENCE au chapitre C07 — Microsoft 365 requis. LibreOffice Calc 24.8 a rattrapé une partie du retard, les versions plus anciennes non. La règle du manuel est donc la **fiche double** : la voie moderne, puis la variante accessible à Excel 2021 et à Calc, avec le nom de la fonction écrit en toutes lettres. Livrer =JOINDRE.TEXTE(", ";VRAI;UNIQUE(ventes!H2:H481)) sans sa variante, c'est un classeur qui rendra #NOM? sur le poste de la direction, et une réunion pour rien.

Deux silences, pour finir. Le tableur **ne prévient pas** quand une formule ne couvre pas toute la plage : =SOMME(L2:L480) posé à la ligne 481 rend un total amputé d'une ligne, et rien ne clignote — le seul contrôle est la barre d'état, pas la cellule. Il **recalcule sans mesurer** non plus : AUJOURDHUI(), DECALER, INDIRECT sont volatiles, recalculées à chaque frappe ; multipliées par 480 lignes, elles transforment un classeur léger en fichier lent — d'où le « comprendre, éviter » du programme.

Attention. =SOMME(ventes!L2:L480) et =SOMME(TableVentes[montant_ttc]) peuvent cohabiter, donner deux nombres différents, et s'afficher toutes les deux sans le moindre avertissement. Le tableur ne compare pas les maillages, il les exécute. Une cellule de contrôle par table, qui confronte les deux écritures, coûte dix secondes et ferme ce risque pour de bon.

Boîte à outils. F4 faire tourner les \$ · Ctrl + ' afficher (et recacher) les formules de la feuille · F2 éditer une cellule sans perdre sa formule · Maj + Flèches pendant l'édition pour étendre une référence · Ctrl + D remplir vers le bas, Accueil → Modifier → Remplir → Vers la droite pour l'autre sens · Ctrl + Entrée valider sans descendre · Création de tableau → Tableau → Sélection → Sélection du tableau pour vérifier la portée · Ctrl + F sur #, dans le classeur entier, pour lister les erreurs · Formules → Vérification des formules → Espion de formule pour remonter une chaîne cellule par cellule. Dans LibreOffice Calc, F4 alterne les \$ de la même façon, l'affichage des formules est dans Affichage → Afficher les formules (pas de raccourci par défaut), et Outils → Contenu des cellules → Précédents / Dépendants remplace l'espion.

6. Exemple concret : la ligne 2, de la saisie au total

Ligne 2 de ventes : quantité 11, prix unitaire HT 4 500, remise 0,03, montant HT enregistré 48 015, montant TTC enregistré 56 658. La chaîne complète :

- $11 \times 4\,500 = 49\,500$ — le brut ;
- $49\,500 \times (1 - 0,03) = 48\,015$ — le net de remise, entier ici, aucun arrondi nécessaire ;
- $48\,015 \times 1,18 = 56\,657,7$ — le TTC **avant** arrondi. Formaté sans décimale, le tableur affiche 56 658 alors que la valeur stockée reste 56 657,7 ;
- =ARRONDI(48015*1,18;0) rend 56 658 dans la **valeur**, et le total de la colonne se recompose dès lors ligne par ligne.

Sur cette ligne, les deux écritures donnent le même affichage. Sur les lignes à demi-franc du \$5.6, elles donnent un total différent de 4 FCFA. C'est toute la démonstration : le format n'est pas un choix cosmétique tant qu'on n'a pas dit **où** l'arrondi se produit — et c'est pourquoi la colonne N du classeur ne se contente pas d'être jolie, elle recalcule, et son écart avec la colonne enregistrée est la mesure du travail.

7. Démonstration pas à pas : lire, reproduire, casser, réparer

Sept étapes dans la copie `formules_atelier`.

1. Lire une fiche. Fiche 5 de `Calculs` : `=SOMME(ventes!M2:M481)` doit rendre 36 073 185. La colonne Ce que la formule rend recalcule, la colonne contrôle compare avec `=SI(EXACT(D6;E6);"ok";"à revoir")`. Le classeur s'auto-auditte sur 28 lignes, en vingt secondes, sans macro : c'est la pratique à copier dans vos fichiers.

2. Reproduire la fiche 22. Sur une copie de `ventes_brutes`, en colonne P : `=CNUM(SUBSTITUE(M2;" ";""))`, reportée jusqu'en P490 — le double-clic sur la poignée suffit, la plage est contiguë. Retirez le `SUBSTITUE` : trois lignes passent, 486 rendent `#VALEUR!`. Le compteur de valeurs numériques passe de 489 à 3, et le report n'y est pour rien : c'est l'ancre qui a été juste, la conversion qui était fausse.

3. Trier la référence, pas le nombre. Écrivez 1,18 en Q1, nommez la cellule `TauxTVA`. En Q2 : `=ARRONDI(L2*TauxTVA;0)`, reportée sur les 480 lignes. Retirez le nom, laissez `L2*1,18`, re-reportez, puis corrigez seulement la ligne 2 : le total ne bouge presque pas, et 479 lignes sont fausses. C'est l'expérience la plus courte du chapitre et la plus probante.

4. Fabriquer les trois erreurs utiles. `=M2*2` sur la feuille brute (`#VALEUR!` : du texte) ; `=RECHERCHEV(1229678;ventes!G2:G481;1;FAUX)` (`#N/A` : la clé n'est pas dans cette colonne) ; `=MOYENNE.SI.ENS(ventes!M2:M481;ventes!E2:E481;"Vendeur inconnu")` (`#DIV/0!` : personne ne correspond). Notez en face de chacune la phrase de ce qu'elle dit — pas ce qu'il faudrait faire.

5. Les compter. En U1 : `=SOMME(ESTERREUR(Q2:Q481)*1)` en validation matricielle. Le nombre doit descendre à 0 quand les formules sont correctes, et monter dès qu'une conversion est retirée. Le comparateur de texte `NB.SI` n'est pas utilisé ici, parce qu'il dépend de la langue d'affichage — \$5.5, point 3.

6. Faire un `#RÉF!` exprès. Supprimez la colonne P construite à l'étape 2 : les cellules qui la liaient deviennent `=ARRONDI(#RÉF!*TauxTVA;0)`. Annulez avec `Ctrl + Z` avant d'enregistrer — après, il n'y a plus d'annulation. Reconstruire la colonne prend deux minutes ; la retrouver dans un classeur de 480 lignes prend une matinée.

7. Écrire la règle d'arrondi dans `Aidez-moi` : « montants arrondis à l'unité ; le recalcul de la colonne N diffère du montant enregistré de 4 FCFA, sur 4 lignes nées d'un produit à demi-franc » — et la colonne d'écart `=ARRONDI(I2*J2*(1-K2);0)-L2`, qui rend la phrase vérifiable ligne par ligne.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Fiche 5 de <code>Calculs</code> , valeur rendue	±1 FCFA	36 073 185 FCFA	plage tronquée en 480, ou feuille mal lue
2	Colonne contrôle des 28 fiches	deux nombres	27 « ok », 1 en <code>#N/A</code> propagé	<code>SIERREUR</code> posé sur le contrôle au lieu de la formule
3	Report de <code>=ARRONDI(L2*TauxTVA;0)</code>	compteur	480 lignes non nulles	1 juste et 479 fausses : nom ou \$ oubliés
4	Somme de la colonne d'écart N – L	±1 FCFA	4 FCFA (2 FCFA sans arrondi)	arrondi appliqué deux fois, ou jamais arrondi
5	Cellules en erreur de la colonne P	deux nombres	0 avec <code>SUBSTITUE</code> , 486 sans	conversion partielle non détectée

8. Erreurs fréquentes

1. **Figurer la mauvaise dimension.** `=L2` au lieu de `$L$2` dans une grille de mois : la moitié des cellules pointe hors de la plage, et le résultat reste plausible sur la première ligne — précisément celle que vous avez vérifiée.
2. **Valider une formule sur une seule ligne.** Le report est un déplacement : l'erreur d'ancre apparaît en bas de plage, et sur une ligne de retour si les montants peuvent être négatifs.
3. **Utiliser SIERREUR comme nettoyeur.** Le classeur devient présentable, les 486 textes deviennent invisibles, et le total amputé reste roi. D'abord ESTNUM et une colonne de statut ; le silence, seulement s'il est choisi.
4. **Recopier une formule depuis une documentation étrangère.** Les noms changent, la virgule change de sens : on recopie la formule **telle que le fichier la stocke** (feuille `Calculs`, colonne C), pas une capture.
5. **Écraser une formule par une valeur sans le dire.** Un collage spécial trop large et le calcul disparaît sans trace. Le classeur de référence garde donc ses colonnes brutes (`ventes_brutes`) à côté des recalculs (N).
6. **Chercher # dans une seule feuille.** Les erreurs vivent souvent dans l'onglet que personne n'ouvre — `Mensuel`, ou la ligne de total masquée d'un tableau.

9. Bonnes pratiques professionnelles

1. **Une cellule d'hypothèse par taux, nommée et documentée :** `TauxTVA`, `ObjectifMensuel`, `SeuilAnomalie`. Le nom rend la formule auto-explicative et le changement d'hypothèse traçable — Q1 ne l'est pas.
2. **Les formules de contrôle à côté des données, pas dedans.** Une colonne qui compare l'enregistré et le recalculé ne pollue pas la ligne de total, se filtre, et se compte.
3. **Ctrl + ' avant chaque envoi,** puis une recherche de # sur le classeur : trente secondes, la moitié des retours en moins.
4. **La fiche double systématique** pour toute fonction version-dépendante : voie moderne, variante 2021/Calc, nom écrit en toutes lettres.
5. **Un commentaire sur les trois cellules qui coûtent cher :** la règle d'arrondi, le dénominateur d'une moyenne, le périmètre d'un filtre. Le destinataire ne rouvrira pas le chapitre, il rouvrira le classeur.

Conseil professionnel. Ne dites jamais « le tableur a fait une erreur ». Il a exécuté. Formulez en revue : « la plage de somme s'arrête en L480, la ligne 481 est hors périmètre » — une adresse et une cause. C'est la différence entre un collègue qui corrige et un collègue qui se sent accusé et rouvre le fichier sans le regarder.

10. Exercice guidé — la chaîne de calcul, de la cellule au contrôle (35 min, /10)

Commande. Dans `formules_atelier`, reconstruisez la chaîne du montant hors taxes sur la feuille `ventes` (480 lignes), et rendez son écart mesurable.

Étape	Geste	Ce qui doit se lire
1	1,18 en Q1, nommée <code>TauxTVA</code> via la zone de nom	<code>TauxTVA</code> dans Gestion des noms
2	<code>=ARRONDI(I2*J2*(1-K2);0)</code> en R2, report	480 valeurs, total 30 570 495 FCFA
3	<code>=R2-L2</code> en S2, report, format nombre	somme des écarts : 4 FCFA

Étape	Geste	Ce qui doit se lire
4	Compteur =SOMME(ESTERREUR(R2:S481)*1)	0
5	Retirer ARRONDI à l'étape 2	somme des écarts : 2 FCFA
6	Retirer les \$ d'une référence de règle (test en T2 avec =R2*Q1)	1 ligne juste, 479 à 0
7	Taux porté à 1,19, recopie du nom seulement	total TTC recalculé, 480 lignes à jour

Démarrage. Les étapes 2, 3 et 5 sont le cœur : même données, trois règles d'arrondi, trois totaux. Notez-les dans cet ordre avant toute explication. Le total attendu à l'étape 2 n'est pas celui de la colonne enregistrée : la caisse a arrondi une fois, votre ARRONDI une autre, et l'écart de 4 FCFA est le résultat prévu, pas une faute. À l'étape 6, écrivez la phrase de ce que vous voyez — c'est elle, et non le chiffre, qui prouve que vous avez compris le report.

Barème (10 points). Nom de plage posé et réellement utilisé dans une formule (2) · colonne recalculée juste sur les 480 lignes (2) · les trois totaux notés, et l'écart de 4 FCFA expliqué par les lignes à demi-franc (3) · démonstration de la casse par retrait du \$, chiffres à l'appui (2) · deux lignes écrites dans Aidez-moi (1).

11. Exercices autonomes

Exercice 4.1 (★) — Les six écritures d'une adresse. Dans une feuille libre, placez un taux en B1, une donnée en B2, et écrivez en C2 la formule =B2*\$B\$1. Reportez-la cinq lignes plus bas, puis trois colonnes vers la droite. Notez ce que deviennent les deux références dans chaque cas, puis réécrivez la formule en =\$B2 et en =B\$2 et recommencez. Concluez en une phrase.

Exercice 4.2 (★) — Ce qui se propage. Fabriquez une chaîne de trois cellules : une division par zéro, une cellule qui l'additionne, une =SOMME() des deux. Ajoutez un compteur d'erreurs. Dites, pour chacune des quatre cellules, si l'erreur est visible, propagée, ignorée ou comptée — et concluez sur ce que voit un lecteur du tableau de bord.

Exercice 4.3 (★★) — La moyenne qui n'existe pas. Calculez la moyenne des montants par vendeur de deux façons : =SOMME.SI.ENS(...)/NB.SI.ENS(...) puis =MOYENNE.SI.ENS(...). Faites disparaître un vendeur du périmètre avec un filtre, puis avec une condition impossible. Laquelle des deux formules tombe en #DIV/0!, l'autre rend-elle la même chose, et que faudrait-il écrire pour que ce soit dit en français et non en code d'erreur ?

Exercice 4.4 (★★) — Les lignes à demi-franc, une par une. Isolez les lignes de ventes dont le produit quantité × prix × (1 - remise) tombe à un demi-franc. Pour chacune, calculez le résultat avec ARRONDI, avec ARRONDI.INF, avec ARRONDI.SUP, et sans arrondi. Dressez le tableau, retrouvez les 4 FCFA, et écrivez la ligne de fiche qui rend l'écart **prévu** et non plus suspect.

Exercice 4.5 (★★) — Le classeur sans #NOM?. Reprenez la fiche 23 de Calculs (TEXTJOIN + UNIQUE) et écrivez sa variante compatible : un tri de la colonne des catégories, une colonne indicatrice =SI(NB.SI(\$H\$2:H2;H2)=1;1;0), puis une concaténation par & sur les lignes retenues. Contrôlez que les deux voies rendent les mêmes 7 libellés, dans le même ordre que la cellule Calculs!E24.

12. Correction détaillée

Exercice 4.1. Vers le bas : =B3*\$B\$1, =B4*\$B\$1 — la ligne suit, la règle reste. Vers la droite : =C2*\$B\$1, =D2*\$B\$1 — la colonne suit, la règle reste. En =\$B2 : vers le bas, =\$B3, donc la **colonne** est figée et la ligne voyage ; en =B\$2, vers la droite, =C\$2. Règle attendue : « le \$ fige ce qui vient après lui — la partie qui le porte ne bouge pas ».

Exercice 4.2. Cellule 1 : #DIV/0! visible. Cellule 2 : la même erreur, propagée (une addition ne sait pas faire autrement). SOMME des deux : l'erreur est **ignorée**, le total rendu est celui de ce qui

restait, sans un mot. Compteur : 1. Conclusion attendue : les fonctions d'agrégation ignorent les erreurs comme elles ignorent le texte — seul un compteur les rend visibles, donc seul un compteur vaut veille.

Exercice 4.3. Les deux écritures tombent en #DIV/0!, parce que MOYENNE.SI.ENS cache le même dénominateur. Écrire `=SI(NB.SI.ENS(ventes!E2:E481;Q1)=0;"aucune ligne dans le périmètre";MOYENNE.SI.ENS(...))` rend le fait lisible. Le vendeur mobile est celui de la ligne 481 du prochain import, jamais celui que l'on teste — d'où la nécessité d'un message rédigé plutôt que d'un code.

Exercice 4.4. Quatre lignes divergent entre l'arrondi usuel et celui de la caisse, une cinquième touche le demi-franc à un cheveu près et ne bouge pas, deux restent d'accord. Les quatre règles ne donnent pas le même total :

Règle appliquée aux 480 lignes	Total de la colonne	Écart avec le fichier
aucune (le produit brut)	30 570 493	+2
ARRONDI (usuel)	30 570 495	+4
ARRONDI.INF	30 570 487	-4
ARRONDI.SUP	30 570 500	+9
arrondi bancaire, celui de la caisse	30 570 491	0

Treize lignes de l'extrait ont un produit non entier, et seule la cinquième ligne du tableau reproduit le fichier à zéro franc près : l'écart n'est donc pas une faute de saisie, c'est une convention. Phrase attendue : « un franc d'écart sur une ligne n'est pas une erreur : c'est la ligne où le produit tombait exactement à un demi-franc ; la règle retenue est l'arrondi au franc le plus proche, elle est écrite ici, et l'écart total attendu est de 4 FCFA ».

Exercice 4.5. La voie compatible tient en trois colonnes et un tri, contre une formule de 40 caractères pour la voie 365. Les deux rendent les sept catégories dans l'ordre de la donnée, tel que `Calculs!E24` l'affiche. Conclusion attendue : la variante est laide et verbeuse, mais elle s'ouvre sur un poste de 2021 — c'est exactement pourquoi le manuel impose la fiche double.

13. Mini-projet M03.P1 — suite : le volet formules et cohérence (45 min)

Commande. Ajoutez au classeur de l'atelier un onglet `Mes fiches` calqué sur `Calculs`, et la colonne de contrôle annoncée par ce chapitre.

Livrables numérotés. (1) dix fiches sur le modèle à six colonnes (nom français, formule stockée, valeur attendue, valeur rendue, contrôle automatique, motif), au moins une par famille : agrégat, conditionnelle, recherche, texte, date ; (2) la chaîne de calcul du montant en quatre colonnes (quantité, prix, remise, net arrondi) avec le taux en cellule nommée ; (3) la colonne d'écart enregistré/recalculé et sa ligne de total, avec l'écart expliqué en une phrase chiffrée ; (4) un inventaire d'erreurs par feuille et par type, `=SOMME(ESTERREUR(plage)*1)`, sur `ventes`, `ventes_brutes` et `objectifs` ; (5) la fiche double de la fonction la plus récente que vous ayez employée, avec son alternative 2021/Calc.

Barème (20 points, seuil 13). Fiches conformes au modèle, colonne de contrôle à « ok » (5) · chaîne de calcul avec nom de plage et ancre justes (4) · écart chiffré et commenté (4) · inventaire d'erreurs complet et non masqué (4) · fiche double rédigée (3).

14. Résumé du chapitre

Une formule est un déplacement, pas un calcul : F4 décide de ce qui suit le report et de ce qui reste, et c'est la seule question à trancher avant d'étendre une ligne sur 480. Les opérateurs ont un ordre, la parenthèse est leur ponctuation, le pourcentage est un diviseur. Le fichier stocke des noms anglais et des virgules, l'interface affiche du français et des points-virgules : d'où une fiche de formules écrite en noms, jamais en captures.

Les sept erreurs sont sept causes : le type (**#VALEUR!**), la clé absente (**#N/A**), le dénominateur (**#DIV/0!**), l'adresse disparue (**#REF!**), le nom inconnu (**#NOM?**), l'intersection vide (**#NULL!**), et la colonne trop étroite (**####**, qui n'est pas une erreur). Elles se propagent le long du maillage, mais les fonctions d'agrégation les ignorent en silence — c'est ce silence qui fabrique des totaux faux et présentables. **SIERREUR** ne répare pas, il fait taire : **SI.NON.DISP**, **ESTNUM** et un compteur **ESTERREUR** rendent la feuille lisible sans la rendre muette.

Reste l'arrondi, qui est un choix de valeur et non d'affichage. Le classeur le mesure : 4 FCFA entre la colonne enregistrée et la colonne recalculée, 4 lignes où la règle change le franc, 2 FCFA si l'on n'arrondit pas du tout. Cinq lignes de documentation, une colonne d'écart, et le doute devient preuve.

15. À retenir

À retenir. Le report translate les références relatives. Une formule juste à la ligne 2 peut être fausse à la ligne 480 : on valide en bas de page, et sur une ligne à montant négatif.

À retenir. **#N/A** est une réponse, **#REF!** est une amputation, **####** n'est pas une erreur. Les traduire avant de les masquer est le travail ; les masquer est le dégât.

À retenir. Une erreur se propage dans les formules mais disparaît dans les agrégats : un total peut être faux sans afficher la moindre erreur. Un compteur d'erreurs par feuille est la seule veille qui tienne.

1. Une règle commune vit dans une cellule **nommée** ; un taux recopié dans 480 formules est un taux qu'on ne corrigera jamais 480 fois.
2. **Ctrl** + **'** avant d'envoyer, **Ctrl** + **F** sur **#** dans tout le classeur : trente secondes, la moitié des retours.
3. L'arrondi se choisit pour les montants facturés, se refuse dans les statistiques — et s'écrit toujours.
4. Toute fonction version-dépendante vient avec sa variante documentée : c'est ce qui permet à un poste d'agence de lire le fichier.
5. On ne supprime pas une colonne référencée : on la renomme, on la masque, on la documente.

16. Évaluation formative (auto-correction, 10 min)

1. Que devient **=B2*\$B\$1** recopié deux lignes vers le bas, puis trois colonnes vers la droite ? Écrivez les deux résultats.
2. Une cellule affiche **####**. Que vérifiez-vous d'abord, et pourquoi ce n'est pas une erreur de calcul ?
3. Pourquoi **=SIERREUR(RECHERCHEV(...);"0")** est-il trois fois sur quatre un mauvais choix, et que mettez-vous à la place ?

4. Sur la fiche 5 de **Calculs**, la plage est **ventes!M2:M481**. Si vous l'écrivez **ventes!M2:M480**, que rend la colonne de contrôle de la ligne, et pourquoi cet enchaînement est-il plus dangereux qu'une erreur affichée ?
5. Le total de la colonne enregistrée et celui de la colonne recalculée diffèrent de 4 FCFA. Citez les trois nombres qui prouvent que ce n'est pas une faute, et la ligne de documentation qui les rend prévisibles.

Question ouverte. Un classeur de 480 lignes contient 479 formules identiques et une dissidente. Quel raccourci, quel geste, quelle colonne de contrôle permettent de trouver la dissidente sans lire 480 lignes ?

Corrigé de l'évaluation formative.

1. Vers le bas : **=B4*\$B\$1**. Vers la droite, en partant de C2 : **=E2*\$B\$1**. Dans les deux sens, la référence relative suit le déplacement et **\$B\$1** ne bronche pas — c'est le **\$** qui décide, pas la direction.
2. La largeur de colonne. **####** signifie « l'affichage ne tient pas », non « le calcul échoue » : double-clic sur le bord de la colonne, la valeur réapparaît, inchangée.
3. Parce qu'il avale toutes les erreurs, **#RÉF!** et **#NOM?** compris, et retourne un 0 ou un texte qui a l'air propre. On met un **SI.NON.DISPLAY** ciblé sur **#N/A** seul, ou un test **ESTNUM** en colonne de statut, qui dit ce qu'il voit sans rien faire disparaître.
4. Le contrôle rend « à revoir » : une valeur plausible, égale au total amputé de la dernière ligne. C'est plus dangereux qu'une erreur affichée parce que le défaut est un **périmètre**, pas une syntaxe — la formule est parfaite, elle ne lit que 479 lignes, et rien à l'écran ne le signale.
5. Six lignes dont le produit tombe exactement à un demi-franc (sept au sens large) ; quatre où la règle d'arrondi change le franc ; écart de 4 FCFA avec **ARRONDI**, de 2 FCFA sans arrondi. La ligne de fiche : « montants arrondis à l'unité au plus proche ; écart attendu de 4 FCFA entre montant enregistré et recalculé, vérifié par la colonne d'écart ». Question ouverte — Réponse attendue : **Ctrl + `** affiche les formules et la dissidente se voit à l'œil ; en contrôle, une colonne **=SI(EXACT(T2;T3);"";"formule différente")** la repère ligne à ligne. Le vrai remède est structurel : la formule doit être **une colonne calculée du tableau**, donc identique par construction sur 480 lignes.

Suite du module. Le chapitre C05 ouvre la boîte à fonctions : les agrégats (**SOMME**, **MOYENNE**, **NB**, **MAX**, **MEDIANE**, **QUARTILE.INC**) et leurs cousins conditionnels (**SOMME.SI.ENS**, **NB.SI.ENS**, **MOYENNE.SI.ENS**) — ceux qui répondent à « combien, pour qui ? » sans avoir à construire un tableau croisé.

Module M03.C05 — La boîte à fonctions d'agrégation et conditionnelle : SOMME, MOYENNE, NB, MEDIANE, SOMME.SI.ENS

Outil de ce chapitre : le tableur seul, sur le classeur de l'atelier. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. Un agrégat n'est pas un calcul, c'est une **question avec un dénominateur**. « Quelle est la remise moyenne ? » admet trois réponses justes sur la même colonne — 1,8 %, 5,1 %, 33,8 % — et la réunion du comité se perd en elles. Ce chapitre enseigne à écrire la question dans la formule : la plage, le critère, et ce que chaque fonction décide d'ignorer.

Base de travail. Le classeur 01_socle_donnees/data/projection/m03_classeur_atelier.xlsx, sur trois feuilles : ventes (le tableau TableVentes, 480 lignes, dates et montants en nombres), ventes_brutes (le fichier reçu, 489 lignes, montants en texte) et Calculs (les 28 fiches auto-contrôlées, dont les fiches 13 à 16 sont celles de ce chapitre). Le fichier de rapprochement 01_socle_donnees/data/brut/objectifs_de_ca.csv sert à la dernière partie. Séparateur point-virgule, UTF-8, franc CFA (FCFA), virgule décimale, TVA 18 %. Chiffres cités : 01_socle_donnees/data/reference/chiffres_cites.md, section M03. Copie de travail : agregats_atelier.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Choisir** la bonne fonction de compte — NB, NBVAL, NB.VIDE, et le compteur de zéros =NB.SI(plage;0) — et dire ce que chacune ignore.
2. **Écrire** des critères : texte, comparaison, joker, date construite, et l'ordre des arguments de SOMME.SI.ENS.
3. **Nommer le dénominateur** de toute moyenne ou pourcentage cité, et savoir qu'une moyenne sur une colonne mixte est un nombre sans question.
4. **Vérifier un agrégat conditionnel** par la cohérence : les sept catégories rendent 480 lignes et 36 073 185 FCFA.
5. **Distinguer** QUARTILE.INC de QUARTILE.EXC, SOMME de SOUS.TOTAL, et ceux qui voient les lignes masquées.
6. **Anticiper** ce que le texte fait aux nombres : 0 FCFA sommé, 3 cellules comptées, une condition de date qui ne matche rien.

2. Pourquoi cette notion est importante

Situation 1 — la réunion des trois pourcentages. Le commercial annonce « 5,1 % de remise moyenne », la direction relit le fichier et trouve 1,8 %. Aucun des deux n'a tort : le premier moyenne les 173 lignes qui portent une remise, le seconde les 480. La troisième question, celle qui coûte de l'argent, est « quelle part du chiffre d'affaires est remise » — 12 198 008 FCFA sur 36 073 185 FCFA, soit 33,8 %. Trois formules, trois justes, une seule utile : celle dont le dénominateur est écrit.

Situation 2 — le 0 qui compte double. La colonne `client` de l'extrait contient 85 « 0 » (ventes au comptoir) et 18 cellules vides. `NBVAL` compte 462 lignes renseignées, `NB` en compte 462 aussi parce que le zéro est un nombre — et la moyenne de la colonne, 9 548,75, est un calcul parfait qui ne répond à rien : une moyenne d'identifiants n'est pas une mesure.

Situation 3 — la condition qui ne matche jamais. `=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[date];">=01/09/2025")` rend 0 sur une colonne de dates stockées en texte, sans la moindre erreur. Sur `ventes`, où la conversion a été faite, la même formule rend les 160 lignes de septembre à décembre. Le tableur n'a pas dit « ce n'est pas une date » : il a dit « rien ne correspond ».

Dans les faits. Les tableaux de bord d'entreprise échouent rarement sur la complexité d'une formule et fréquemment sur un critère mal typé ou un dénominateur tu. Un agrégat s'accompagne donc toujours de son compte de lignes : 8 754 982 FCFA « sur 57 lignes » se discute ; 8 754 982 FCFA seul se conteste.

3. Explication simple

Une fonction d'agrégat réduit une plage à un nombre. Elle diffère sur trois points seulement, et ces trois points font toute la difficulté : **ce qu'elle regarde** (une colonne, ou une colonne plus des critères), **ce qu'elle ignore** (les vides, le texte, les erreurs, les lignes masquées), et **ce qu'elle demande** (une condition écrite comment).

Les `...SI` posent une condition, les `...SI.ENS` en posent plusieurs. Leur point commun : elles parcourent la plage une fois, et comparent chaque cellule au critère. Comparer, c'est le verbe exact — d'où leurs deux faiblesses : le critère doit être écrit avec le type que la cellule a (un nombre, pas du texte qui lui ressemble), et les plages des critères et de la somme doivent avoir la même forme.

Enfin, une moyenne n'existe pas sans dénominateur. Écrire `MOYENNE` sans dire « sur combien de lignes », c'est laisser le lecteur deviner — et il devinera autre chose que vous.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Agrégat — aggregate	Fonction qui réduit une plage à un nombre : <code>SOMME</code> , <code>MOYENNE</code> , <code>MAX</code> .	Oublier qu'elle ignore le texte et les vides.
Critère — criteria	La condition écrite en texte : <code>>500</code> , <code>"Plomberie"</code> , <code>"<>0"</code> .	Écrire <code>>500</code> sans guillemets, ou <code>500</code> sans guillemets comme du texte.
Plage de somme — sum range	La colonne additionnée ; elle vient en premier dans <code>SOMME.SI.ENS</code> .	L'inverser avec la plage de critère : <code>#VALEUR!</code> , ou pire, un nombre plausible.
Agrégat conditionnel — conditional aggregate	<code>SOMME.SI.ENS</code> , <code>MOYENNE.SI.ENS</code> .	Compter les lignes avec <code>NB.SI</code> et le CA avec <code>SOMME.SI</code> sur des plages différentes.
Sous-total visible — SUBTOTAL	<code>SOUS.TOTAL(9;...)</code> ne somme que ce qui n'est pas masqué ou filtré.	Croire que <code>SOMME</code> suit le filtre : il ne le voit pas.
Quartile inclusif / exclusif — QUARTILE.INC / .EXC	Deux interpolations différentes sur les mêmes données.	Citer un quartile sans dire la méthode ni la version du tableur.
Colonne mixte — mixed column	Une colonne où se côtoient nombres, vides et zéros.	La moyenner : le résultat est juste et dépourvu de sens.

Français — English	Définition simple	Piège à éviter
ET logique entre conditions — implicit AND	Toutes les conditions d'un <code>.SI.ENS</code> doivent être vraies.	Croire pouvoir mettre deux critères dans une seule cellule de condition.

Définition. Un **critère de comparaison** s'écrit entre guillemets, opérateur compris : `">=1000000"`, `"<>"` (non vide), `"=0"` (vide n'est pas zéro). Dès qu'il y a un opérateur, il y a des guillemets ; dès qu'il y a une date, il y a une concaténation : `">="&DATE(2025;9;1)`.

Définition. Le **dénominateur** d'un agrégat est le nombre de lignes qu'il a réellement regardées. Deux agrégats de même forme sur la même colonne peuvent avoir des dénominateurs différents : `MOYENNE` ignore les vides, `MOYENNE.SI.ENS` ignore en plus les lignes hors critère — c'est exactement l'écart entre 1,8 % et 5,1 % de remise.

5. Cours approfondi

TableVentes[montant_ttc] : 480 lignes, 7 catégories

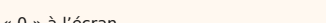
La formule ne vaut que par la question qu'elle pose

<code>=SOMME(TableVentes[montant_ttc])</code>	36 073 185 FCFA
<code>=SOMME.SI.ENS(...;[categorie];"Plomberie")</code>	8 754 982 FCFA
<code>=NB.SI.ENS([remise];">0")</code>	173 lignes sur 480
<code>=MOYENNE(TableVentes[remise])</code>	1,8 %
<code>=MOYENNE.SI.ENS([remise];[remise];">0")</code>	5,1 %
<code>=MEDIANE(TableVentes[montant_ttc])</code>	37 966 FCFA
<code>=MOYENNE(TableVentes[montant_ttc])</code>	75 152 FCFA

75 152 de moyenne contre 37 966 de médiane : la queue, pas une erreur.

Compter : quatre réponses

colonne « client », 480 lignes

NBVAL — non vides		462
NB — nombres		462
« 0 » à l'écran		85
vraiment vides		18
=MOYENNE(client) rend 9 548,75.		

Trois pièges à connaître avant de citer un agrégat

1. Agréger du texte

`=SOMME(ventes_brutes!M2:M490)` rend 0 et `=NB` rend 3 : les 489 montants sont du texte, aucune erreur affichée.

2. Une condition sur une colonne texte

`=SOMME.SI.ENS([montant_ttc];[date];">=01/09/2025")` rend 0 sur le fichier reçu : les dates sont du texte.

3. Le quartile qui change de méthode

`QUARTILE.INC` rend 12 950 / 74 234 ; `QUARTILE.EXC` rend 12 891,5 / 74 304,5. Même question, deux réponses.

Réflexe : un compte de lignes à côté de chaque agrégat — 8 754 982 FCFA sur 57 lignes ne se lit pas comme sur 480.

5.1 Compter : quatre fonctions, quatre réponses

Sur la colonne `client` de `ventes`, les quatre compteurs racontent quatre histoires :

Formule	Rend	Ce qu'elle compte
<code>=NBVAL(TableVentes[client])</code>	462	tout ce qui n'est pas vide — zéros compris
<code>=NB(TableVentes[client])</code>	462	uniquement les nombres — les zéros sont des nombres
<code>=NB.VIDE(TableVentes[client])</code>	18	les trous
<code>=NB.SI(TableVentes[client];0)</code>	85	les comptoirs, identifiés par un zéro

Retenez trois asymétries. `NB` et `NBVAL` ne diffèrent **que** sur le texte : sur une colonne de nombres, ils s'accordent, et sur la colonne de montants du fichier reçu, `NB` rend 0 quand `NBVAL` rend 489 — c'est le diagnostic du chapitre C02, devenu un outil de contrôle. Un **0 n'est pas un vide**, et un vide n'est pas un 0 : la différence vaut 85 lignes ici. Enfin, les compteurs se re-

coupent : NBVAL plus NB.VIDE doit rendre le nombre de lignes du tableau, $462 + 18 = 480$ — si l'égalité ne tombe pas, la plage regardée n'est pas celle que vous croyez.

À retenir. Un tableau de bord qui affiche un chiffre d'affaires sans son compte de lignes n'est pas vérifiable. Écrivez le couple : `=SOMME(TableVentes[montant_ttc])` et juste à côté `=NB(TableVentes[montant_ttc])` → 36 073 185 FCFA · 480.

5.2 Les critères, ce langage minuscule

Un critère est une **chaîne de caractères** que le tableur interprète. Sept écritures couvrent la gestion :

- "Plomberie" — égalité, non sensible à la casse, et qui compare du texte à du texte.
- ">=500", "<" (non vide), "=0" (le zéro, pas le vide) — comparaisons sur nombres.
- "réf 2*" ? Non : le joker est "*réf 2" (préfixe) ou "réf*2" — l'astérisque remplace n'importe quelle suite, ? un caractère, et ~ échappe le joker. Sur les produits de ventes, "*réf 2*" matche 223 lignes.
- `DATE(2025;9;1)` seul ne fait pas un critère : `">=&DATE(2025;9;1)` en fait un. Sans le &, le tableur compare une chaîne à une date et ne trouve rien.
- "<>Plomberie" — tous les autres ; utile, mais attention : une **cellule vide satisfait** `<>Plomberie`, donc les 18 lignes sans client sont comptées comme « autres clients ».
- Une référence de cellule : `>K2` s'écrit `">"&K2`. C'est la seule manière de rendre une fiche paramétrable.
- Un texte qui ressemble à un nombre : ça dépend de qui regarde. La colonne `quantite` de la feuille de preuve est entièrement en texte, et `=NB.SI(ventes_brutes!I2:I490;">500")` y rend pourtant 7 — la famille `.SI` convertit ce qui se lit comme un nombre — alors que `=SOMME(ventes_brutes!I2:I490)` rend 0 sur ces mêmes cellules, parce que les agrégats arithmétiques ignorent le texte. Deux réponses sur le même contenu, aucune des deux n'est une erreur : c'est la dissymétrie à avoir en tête avant de déclarer une colonne « propre » parce qu'un `NB.SI` a répondu.

L'ordre des arguments de la famille `.SI.ENS` suit la logique de la question, pas celle de la phrase : `=SOMME.SI.ENS(plage_à_sommer;plage_critère_1;critère_1;plage_critère_2;critère_2;...)`. **La somme d'abord.** Dans `NB.SI.ENS` et `MOYENNE.SI.ENS`, il n'y a pas de plage de somme, donc le critère vient en premier — c'est l'inversion la plus fréquente en correction de classeur, et quand les plages ont la même taille, elle produit silencieusement un nombre faux.

Attention. Les plages de critères et la plage de somme doivent avoir **la même dimension** : `=SOMME.SI.ENS(TableVentes[montant_ttc];ventes!H2:H481;"Plomberie")` rend `#VALEUR!` si l'une des deux s'arrête en 480 et l'autre en 481. Le réflexe : dans un tableau, écrivez les colonnes par leur nom (`TableVentes[categorie]`), elles sont toujours de la même longueur que le tableau.

Attention. Quand aucune ligne ne satisfait le critère, les trois familles ne réagissent pas pareil : `=SOMME.SI.ENS(...)` et `=NB.SI.ENS(...)` rendent 0, `=MOYENNE.SI.ENS(...)` rend `#DIV/0!` faute de quoi diviser. Un 0 est un résultat, `#DIV/0!` est un symptôme : sur une moyenne, ne le remplacez jamais par un 0 à coups de `SIERREUR` — signalez la ligne vide. Un tableau de bord qui affiche une moyenne de 0 FCFA là où il n'y a aucune ligne invente une donnée.

5.3 Le dénominateur est la notion

Même colonne `remise`, trois questions, trois écritures, trois résultats qu'il faut savoir énoncer :

Question	Formule	Résultat
Remise moyenne sur toutes les lignes	<code>=MOYENNE(TableVentes[remise])</code>	1,8 %
Remise moyenne quand il y a une remise	<code>=MOYENNE.SI.ENS(TableVentes[remise];TableVentes[remise];">0")</code>	5,1 %
Chiffre d'affaires touché par une remise	<code>=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[remise];">0")</code>	12 198 008 FCFA, soit 33,8 % du total

Et une quatrième, la seule qui parle d'argent : combien la politique de remise a coûté. Ni `SOMME` ni `SOMME.SI.ENS` ne savent sommer un produit de deux colonnes ; deux issues. La rapide : `=SOMMEPROD(TableVentes[montant_ht];TableVentes[remise])` — `SOMMEPROD` multiplie ligne à ligne puis additionne, et rend 514 969 FCFA. La lisible : une colonne calculée du tableau, `remise en francs` avec `=[@[montant_ht]]*[@[remise]]`, qui rend la même valeur par un simple `=SOMME`, se filtre par catégorie et se reconduit ligne par ligne. Sur un fichier appelé à être relu par d'autres, choisissez la seconde et gardez `SOMMEPROD` pour vos contrôles rapides.

Le retournement de la moyenne avec les retours est la même leçon en négatif : la moyenne des 480 lignes est de 75 152 FCFA, celle des 472 lignes sans les 8 retours est de 77 028 FCFA — 1 876 FCFA d'écart par ligne, pour huit lignes retirées sur 480. Voilà ce que « moyenne » veut dire sur une distribution asymétrique, et le module M02 avait nommé cela l'effet de la queue.

5.4 Agréger par catégorie sans tableau croisé — et le vérifier

La question de gestion type : « par catégorie, combien de lignes, quel CA, quelle moyenne, quelle remise consentie ? ». Quatre agrégats conditionnels par catégorie, soit une grille de 7 × 4 formules avec, en tête de chaque ligne, la catégorie saisie **une seule fois** en A2 et réutilisée par référence :

```
=NB.SI.ENS(TableVentes[categorie];$A2)
=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[categorie];$A2)
=MOYENNE.SI.ENS(TableVentes[montant_ttc];TableVentes[categorie];$A2)
=MAX.SI.ENS(TableVentes[montant_ttc];TableVentes[categorie];$A2)
```

Le contrôle est la beauté du dispositif : les sept compteurs de lignes doivent restituer 480, les sept CA 36 073 185 FCFA. Ce n'est pas une coïncidence, c'est une **partition** — chaque ligne a une et une seule catégorie. Dès qu'une ligne a une catégorie vide, ou un critère orthographié autrement que la donnée, la partition se ferme mal — et le cas est ici, grandeur nature : le fichier stocke `Electricité`, sans accent sur la première lettre, survivance d'un export qui a perdu les accents. Recopier le mot en `Électricité` dans un critère, c'est écrire une formule qui rend 0 sur 53 lignes : les comparaisons du tableur sont insensibles à la casse et **sensibles aux accents**. Un critère ne se tape donc jamais de mémoire, on le copie depuis une cellule du tableau.

Définition. Une **partition fermée** — **closed partition** est un découpage dont les morceaux rendent le tout : les lignes s'additionnent (57 + 143 + 112 + 53 + 57 + 24 + 34 = 480) et les chiffres d'affaires aussi. Un regroupage qui ne ferme pas n'est pas un résultat approximatif : c'est qu'une ligne a échappé au critère, le plus souvent par l'orthographe, la casse ou un accent. La partition est le contrôle qualité des agrégats conditionnels, et elle ne coûte qu'une soustraction.

Les sept valeurs attendues, à recopier dans votre copie de contrôle : Plomberie 57 lignes · 8 754 982 FCFA · moyenne 153 596 ; Matériaux 143 · 6 519 357 · 45 590 ; Electricité 53 · 6 311 645 · 119 088 ; Quincaillerie 112 · 4 203 121 · 37 528 ; Consommables 57 · 2 500 964 · 43 877 ; Bois & panneaux 24 · 4 844 758 · 201 865 ; Peinture 34 · 2 938 358 · 86 422.

5.5 La position et la dispersion : MEDIANE, QUARTILE.INC, et l'autre

MEDIANE(TableVentes[montant_ttc]) rend 37 966 FCFA ; la moyenne, 75 152 FCFA ; la moyenne dépasse la médiane de 98 %. Ce n'est pas une anomalie du fichier, c'est sa forme — quatre lignes dépassent le million, et la plus forte, 1 229 678 FCFA, pèse à elle seule plus de 3 % du total. Un tableau de bord qui montre la moyenne sans la médiane raconte l'histoire des gros tickets en croyant raconter le client moyen.

Les quartiles, eux, portent une embuscade de version. Le classeur d'atelier (fiches 11 et 12 de Calculs) calcule QUARTILE.INC : 12 950 et 74 234 FCFA. QUARTILE.EXC rend 12 891,5 et 74 304,5. Quatre-vingt francs d'écart sur des valeurs à treize et soixante-quatorze mille : négligeable sur un seuil d'alerte, rédhibitoire dans une note qui compare deux fichiers produits avec deux méthodes. Retenez quatre choses : les deux méthodes existent, et les héritées QUARTILE / PERCENTILE **valent** la méthode inclusive ; les variantes récentes exigent Excel 2010 ou plus ; en français, elles s'appellent QUARTILE.INCLURE, QUARTILE.EXCLURE et CENTILE.EXCLURE, et c'est ce nom qu'on écrit dans une fiche de suivi — la feuille Calculs du classeur affiche quant à elle la forme anglaise, parce que le format .xlsx stocke les noms anglais, comme vu au §5.3 de C04 ; enfin LibreOffice Calc connaît les deux méthodes sous le nom anglais non traduit.

Deux agrégats de contexte complètent la boîte : SOUS.TOTAL(9;TableVentes[montant_ttc]) ne somme que les lignes visibles — la valeur du filtre à l'écran — alors que SOMME ignore le filtre ; et AGREGAT (anglais AGGREGATE) additionne un choix de comportement : =AGREGAT(9;6;TableVentes[montant_ttc]) somme les lignes visibles **en ignorant les erreurs**, ce qui évite qu'une cellule en #VALEUR! contamine un total affiché (§5.6).

Définition. Un agrégat **robuste** est un agrégat dont on connaît le périmètre exact : quelles lignes, quels vides, quelles erreurs, quel filtre. Les trois quatrièmes arguments de SOMME.SI.ENS ne disent que les conditions ; le reste est une convention que vous écrivez, ou une ambiguïté que d'autres résoudront à votre place.

5.6 Ce que le texte et les erreurs font aux nombres

Sur la feuille de preuve ventes_brutes, la colonne montant_ttc n'est que du texte, et les agrégats de base s'y comportent en six manières de se taire : SOMME rend 0, MOYENNE rend #DIV/0! faute de valeur à diviser, MEDIANE rend #NUM!, MIN et MAX rendent 0, NB rend 0 — pendant que NBVAL rend 489 et que =NB.SI(ventes_brutes!M2:M490;">1000000") rend 4, parce que lui convertit. Quatre de ces six ignorent le texte **sans le dire**. Le contrôle ne peut donc pas être un agrégat : c'est un compteur. La paire qui ferme le problème : =NB(TableVentes[montant_ttc]) et =NBVAL(TableVentes[montant_ttc]) — quand les deux diffèrent, il y a du texte dans la colonne, et il n'y a pas de total à citer.

Sur la colonne des quantités, la fiche 13 de Calculs compte 7 lignes au-dessus de 500 unités, dont une à 14 000

des erreurs d'import, pas des ventes en vrac. Le chiffre est exact parce que NB.SI convertit ; il deviendrait faux dès que la comparaison porterait sur autre chose qu'un nombre, une date ou un code par exemple. Retenez la phrase pour la fiche : « un agrégat sur une colonne non typée rend un nombre plausible, jamais une erreur ».

5.7 Et les objectifs, dans tout ça

Le rapprochement avec objectifs (218 lignes magasin × mois) est le vrai test des .SI.ENS à deux conditions :

```
=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[magasin];"Magasin
5";TableVentes[date];">="&DATE(2025;1;1);TableVentes[date];"<="&DATE(2025;12;31))
=SUMIFS(...) vers la table d'objectifs, avec magasin ET mois comme clés
```

L'extrait de l'atelier représente 18,2 % de l'objectif annuel du magasin 5 (36 073 185 FCFA sur 198 030 000 FCFA). Le piège est dans la jointure : les objectifs sont saisis **par mois**, les ventes **par jour**. Agréger sans aligner la granularité produit un chiffre creux — d'où la colonne mois (le premier du mois, via la poignée de recopie vue en C02 et DATE(ANNEE(...);MOIS(...);1)) qui aligne les deux tables, et le NB.SI.ENS à deux critères qui sert à la fois d'agrégat et de **contrôle de clé** : si le compte d'objectifs trouvés descend au-dessous de 12, un mois est absent de la table d'en face. Deux ventes de l'extrait n'ont d'ailleurs pas d'objectif correspondant — le mois de mars du magasin 4 — et c'est exactement ce que ce compteur signale.

Conseil professionnel. Dans toute note de synthèse, l'agrégat s'écrit avec sa parenthèse : « remise moyenne 5,1 % (sur les 173 lignes remises) ». Douze mots de plus, et la question du dénominateur ne se pose plus en réunion. C'est aussi ce qui rend votre chiffre retrouvable par quelqu'un d'autre six mois plus tard.

Boîte à outils. =NB · =NBVAL · =NB.VIDE · =NB.SI(plage;0) pour compter ; =SOMME · =MOYENNE · =MEDIANE · =MAX · =MIN · =ECARTYPE.STANDARD · =QUARTILE.INC(plage;1|3) pour résumer ; =SOMME.SI.ENS · =NB.SI.ENS · =MOYENNE.SI.ENS · =MAX.SI.ENS · =MIN.SI.ENS pour conditionner (les deux derniers exigent Excel 2019 ou Microsoft 365 — variante 2021 : un tableau MAX sur une plage filtrée, ou AGREGAT en matriciel) ; =SOUS.TOTAL(num;plage) pour la vue à l'écran. Raccourcis : Alt + = (somme automatique, qui s'adapte à la colonne choisie), F4 (passer une adresse en absolu) et Ctrl + Entrée (valider une formule matricielle ancienne). La barre d'état reste l'outil de contrôle par défaut : clic sur une colonne, lire Somme, Nombre, Moyenne en bas de fenêtre. Dans LibreOffice Calc, mêmes fonctions, et l'assistant de filtre rend les sous-totaux de groupe (données → sous-totaux), inexistant dans Excel sans TCD.

6. Exemple concret : la dispute des 3 pourcents

Une messagerie interne : « la direction dit 1,8 % de remise, le terrain dit 5 %. Qui a falsifié quoi ? ». Personne. Sur TableVentes[remise] :

- =MOYENNE(TableVentes[remise]) → 1,8 %, dénominateur 480 lignes, les non-remises incluses ;
- =MOYENNE.SI.ENS(TableVentes[remise];TableVentes[remise];">0") → 5,1 %, dénominateur 173 lignes remises ;
- =SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[remise];">0") / =SOMME(TableVentes[montant_ttc]) → 33,8 %, dénominateur : le CA, pas les lignes — « un tiers du chiffre d'affaires part en remise ».

La réunion ne porte plus sur les nombres mais sur la question. C'est tout l'objet du chapitre : chaque agrégat du classeur d'atelier est doublé d'un compte de lignes, et la fiche 16 de **Calculs** (NB.SI.ENS sur Peinture avec remise) est exactement ce genre de cellule — 8 lignes, et le lecteur sait enfin sur quoi porte le pourcentage.

7. Démonstration pas à pas : la grille de contrôle par catégorie

Six étapes dans la copie agregats_atelier, feuille neuve Contrôle catégories.

1. Poser les catégories. En A2:A8, les sept libellés recopiés **depuis le tableau** (=TableVentes[categorie] dans une cellule d'à côté puis UNIQUE si disponible, sinon copier-coller et supprimer les doublons à la main). Un libellé recopié à la main est une faute en puissance : ici, Electricité sans accent dans le fichier, avec accent dans votre saisie, et le critère ne matche rien.

2. Les quatre agrégats. En B2:D2 et jusqu'en ligne 8 : =NB.SI.ENS(TableVentes[categorie];\$A2), =SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[categorie];\$A2), =MOYENNE.SI.ENS(TableVentes[montant_ttc];TableVentes[categorie];\$A2), et en E2 la part du CA =D2/SOMME(TableVentes[montant_ttc]) au format 0,0 %.

3. Fermer la partition. En B9 : =SOMME(B2:B8) doit rendre 480. En C9 : =SOMME(C2:C8) doit rendre 36 073 185. En D9 : =C9/B9 doit rendre 75 152 — la moyenne générale, retrouvée par les sous-ensembles. Si l'un des trois cloche, la cause est une catégorie absente ou mal orthographiée, jamais une erreur de formule.

4. Les bornes par catégorie. En F2:G8, =MIN.SI.ENS et =MAX.SI.ENS ; sur Excel 2021, la variante est un MIN.SI sur une colonne triée ou un tableau =MIN(SI(...)) validé en matriciel. Notez que la Plomberie a une moyenne de 153 596 FCFA, plus du triple de celle des Matériaux (45 590 FCFA) : l'écart est dans la marchandise, pas dans la formule.

5. La condition de date. En I1 : =SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[date];">="&DATE(2025;9;1)) → les lignes de septembre à décembre. Refaites-la sur la copie non convertie, avec le texte : 0 FCFA. Ce couple de nombres est la démonstration que le typage est en amont de l'agrégat.

6. Le test d'objection. Ajoutez une ligne =NB.SI(TableVentes[categorie];"Electricité") puis corrigez temporairement une catégorie en électricité : les compteurs de la grille ne bougent pas, et la partition se ferme à 479 lignes au lieu de 480. Voilà ce que le contrôle de l'étape 3 attrape, et rien d'autre ne l'attrape.

Contrôles qualité du chapitre.

#	Contrôle	Seuil	Résultat attendu	Si échec
1	Somme des 7 compteurs de lignes	identité	480	catégorie vide, doublon de libellé ou critère mal orthographié
2	Somme des 7 CA	±1 FCFA	36 073 185 FCFA	une ligne a deux catégories, ou la plage s'arrête en 479
3	CA / lignes globaux	±1 FCFA	75 152 FCFA	moyenne calculée comme moyenne des moyennes (faux)
4	Condition de date, copie typée contre copie texte	deux nombres	les lignes d'automne contre 0	dates restées du texte, critère sans &
5	NB contre NBVAL sur montant_ttc	sur identité	480 · 480 (et 3 · 489 sur la brute)	agrégat posé sur une colonne non typée

8. Erreurs fréquentes

- Inverser plage de somme et plage de critère.** Sur SOMME.SI.ENS, la somme est le premier argument ; inversé, le résultat est un nombre absurde ou #VALEUR! selon les tailles, mais presque jamais une erreur claire.
- Moyenne des moyennes.** Reprendre la colonne des sept moyennes de catégorie et en faire une MOYENNE : 98 281 FCFA au lieu de 75 152 FCFA, soit 30,8 % de surévaluation. On re-somme les chiffres d'affaires et on divise par le total des lignes — jamais l'inverse.
- Confondre NB et NBVAL comme un contrôle de typage.** Les deux diffèrent sur le texte, pas sur le vide ; pour les vides, c'est NB.VIDE. Un tableau de bord qui les inverse affiche un problème là où il n'y en a pas.

4. **Comparer une date à une chaîne.** `">=01/09/2025"` sur une colonne de dates réelles fonctionne selon le réglage régional, et casse sur un autre poste : écrivez `">="&DATE(2025;9;1)`, ou référencez une cellule de borne.
5. **Chercher la performance dans SOMME.SI sur 480 lignes.** Sur cette taille, ce n'est pas un problème ; le problème est le `.SI.ENS` recalculé sur 200 000 lignes à chaque frappe — la réponse est le tableau croisé (chapitre C08), pas une formule plus astucieuse.
6. **Oublier qu'un critère texte est insensible à la casse mais sensible à l'espace.** `"Plomberie"` avec une espace finale ne matche pas `"PLOMBRIE"`, et les deux matchent `"plomberie"` : l'outil de nettoyage est `SUPPRESPEACE`, pas la relecture à l'œil.

9. Bonnes pratiques professionnelles

1. **Tout agrégat est accompagné de son dénominateur** — une cellule de compte juste à côté, jamais dans un commentaire oral. La paire `SOMME / NB` est le minimum du livrable.
2. **Les critères viennent d'une cellule, pas de la formule.** Une liste de catégories en `A2:A8`, des bornes de période en `B1:B2` : la grille devient paramétrable, et le contrôle de cohérence se rejoue sur un autre périmètre.
3. **Fermer la partition.** Si vous agrégez par catégorie, par magasin ou par mois, la somme des parts doit rendre le total. C'est le seul test qui prouve à la fois la couverture et l'absence de doublons — et il coûte une formule.
4. **Une colonne calculée pour ce qui n'est pas linéaire.** La remise en francs, l'écart à l'objectif : le tableur sait sommer un produit, beaucoup moins bien le recompter ligne à ligne dans une formule de contrôle.
5. **Version écrite pour les fonctions récentes.** `MAX.SI.ENS`, `QUARTILE.EXC`, `AGREGAT` : la fiche double du chapitre C04 s'applique — nom français, variante 2021/Calc, et le nom tout court si le fichier doit voyager.

Conseil professionnel. Quand un chiffre étonne, ne refaites pas la formule : **changez son périmètre.** `=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[remise];">0")` rendu seul, puis avec une condition de date, puis avec une catégorie : trois nombres, et l'étonnement devient une explication. C'est en réduisant le périmètre qu'on trouve la ligne qui parle, jamais en relisant la syntaxe.

10. Exercice guidé — la grille qui se ferme (35 min, /10)

Commande. Construisez la grille de contrôle par vendeur (4 lignes au lieu de 7), et fermez-la.

Étape	Geste	Ce qui doit se lire
1	Vendeurs en <code>A2:A5</code> , copiés du tableau	4 libellés exacts
2	<code>=NB.SI.ENS(TableVentes[vendeur];\$A2)</code>	128 · 125 · 118 · 109 lignes
3	<code>=SOMME.SI.ENS</code> du CA par vendeur, puis part en %	4 parts, somme = 100,0 %
4	<code>=MOYENNE.SI.ENS</code> par vendeur	4 nombres, et pas leur moyenne en total
5	Total : <code>=SOMME(lignes) · =SOMME(CA) · =CA/lignes</code>	480 · 36 073 185 FCFA · 75 152 FCFA
6	Contrôle des erreurs : <code>=NB</code> contre <code>=NBVAL</code> sur chaque plage de critères	égalités partout

Démarrage. L'étape 4 est le piège du chapitre : la moyenne générale n'est pas la moyenne des quatre moyennes, parce que les dénominateurs diffèrent (128 lignes contre 109). Vérifiez-le en calculant les deux : l'écart se voit en centaines de francs. À l'étape 5, si le total des lignes ne

rend pas 480, ne corrigez pas la formule — cherchez la ligne sans vendeur : `=NB.VIDE(Table-Ventes[vendeur])` répond en une seconde.

Barème (10 points). Grille complète avec critères par référence (3) · partition fermée aux trois totaux (3) · moyenne des moyennes démontrée fausse, chiffre à l'appui (2) · contrôle NB/NBVAL sur les trois colonnes lues (1) · note d'une ligne par agrégat indiquant le dénominateur (1).

11. Exercices autonomes

Exercice 5.1 (★) — Les quatre compteurs. Sur `ventes_brutes`, puis sur `ventes`, posez les quatre compteurs — NB, NBVAL, NB.VIDE et `=NB.SI(plage;0)` — sur les colonnes `quantite`, `client` et `montant_ttc`. Dressez un tableau à quatre colonnes (formule, valeur sur la brute, valeur sur la propre, ce que ça veut dire) et concluez par la phrase que vous enverriez à la personne qui a saisi le fichier.

Exercice 5.2 (★) — Les critères, un par un. Sur la colonne `remise de ventes`, écrivez six formules : `=NB.SI(...;">0")`, `=NB.SI(...;"=0")`, `=NB.SI(...;"<>0")`, `=NB.SI(...;"<=0,1")`, un `=NB.SI.ENS(...)` borné entre 0,05 et 0,11, et un `=NB.VIDE(...)`. Deux d'entre elles rendent le même nombre : dites lesquelles, et pourquoi la colonne s'y prête mal. Recommencez sur `client` avec les trois conditions `>0`, `=0` et `NB.VIDE` : vérifiez qu'elles se referment sur le nombre de lignes du tableau.

Exercice 5.3 (★★) — Le seuil qui dépend de la méthode. Calculez Q1 et Q3 de `montant_ttc` par les deux méthodes (`QUARTILE.INC` puis `QUARTILE.EXC`), puis la moyenne des lignes comprises entre les deux bornes avec `MOYENNE.SI.ENS` à deux conditions. Combien de lignes changent de côté sur cet extrait ? Que répondez-vous à un lecteur qui cite un seuil de quartile sans dire la méthode ?

Exercice 5.4 (★★) — La grille par mois. Ajoutez au tableau une colonne `mois` calculée (`=DATE(ANNEE([@date]);MOIS([@date]);1)`, format `mmm. aaaa`), puis la grille 12 × 3 (lignes, CA, moyenne). Contrôlez par la partition, et comparez les CA mensuels aux 12 valeurs de la fiche `Calculs`. Concluez sur ce que la granularité change à la lecture de la saisonnalité.

Exercice 5.5 (★★) — Agréger contre le fichier d'objectifs. Sans `RECHERCHEV` (c'est le chapitre suivant) : comptez, avec `NB.SI.ENS` sur les deux critères magasin et mois, combien de lignes d'objectifs correspondent à des ventes et inversement, à partir des 218 lignes et des 12 mois de l'extrait. Les deux comptes doivent être égaux pour que la comparaison CA/objectif soit licite — ne le sont pas : nommez les lignes qui manquent.

12. Correction détaillée

Exercice 5.1. Sur `ventes_brutes`, tout est texte : les trois colonnes rendent NB 0, NBVAL 489, NB.VIDE 0. Seul le critère numérique sort du silence — `=NB.SI(ventes_brutes!F2:F490;0)` rend 85 sur `client`, et 0 sur `quantite` comme sur `montant_ttc`, parce que la famille `.SI` convertit ce que NB ignore. Sur `ventes`, `montant_ttc` et `quantite` rendent 480 · 480 · 0 · 0, et `client` 462 · 462 · 18 · 85. Phrase attendue : « les montants ne sont pas chiffrables en l'état et rien n'est signalé ; le seul témoin est l'écart entre NB et NBVAL ».

Exercice 5.2. Sur `remise` : 173 lignes remises, 307 à zéro, 173 encore en `<>0`, 476 au plus égales à 0,1 — soit 4 lignes au-dessus de la grille, les quatre lignes à 11 % —, 50 entre 0,05 et 0,11, et 0 vide. Les deux formes qui se ressemblent sont `>0` et `<>0` : elles rendent la même valeur ici pour la seule raison que la colonne ne contient aucun vide. Sur `client`, la fermeture se vérifie : 377 identifiants positifs, 85 zéros, 18 cellules vides, et $377 + 85 + 18 = 480$, le nombre de lignes du tableau. C'est ce troisième nombre qui distingue une partition d'une négation : `<>0` est une comparaison, pas un complément, et selon la version du tableur les vides y entrent ou non. La règle du chapitre en découle — ne jamais compter par négation quand la colonne a des trous ; additionnez les composantes, le contrôle tombe juste.

Exercice 5.3. `QUARTILE.INC` rend 12 950 et 74 234, `QUARTILE.EXC` rend 12 891,5 et 74 304,5. Entre les deux paires de bornes, **aucune ligne** de l'extrait ne tombe : la moyenne tronquée est identique au franc, et l'illusion est parfaite — deux méthodes, deux seuils écrits noir sur blanc, un seul résultat réel. Réponse attendue : « le seuil n'a de sens que nommé avec sa méthode ; ici l'écart se joue à une soixantaine de francs sur les bornes et à rien sur les effectifs, ce qui est la pire configuration, parce qu'elle ne punit pas l'oubli ».

Exercice 5.4. Douze lignes de 40 ventes chacune dans l'extrait ; les CA mensuels reproduisent la fiche `Calculs` ; la moyenne annuelle des lignes (75 152 FCFA) se retrouve au centime près en sommant les CA puis en divisant par 480, pas en moyennant les douze moyennes mensuelles. La granularité mensuelle fait apparaître décembre au sommet (5 292 517 FCFA), février juste derrière (4 289 885 FCFA) et mai le plus bas (1 258 801 FCFA) : invisible au niveau de la ligne, et écrasé par la moyenne annuelle.

Exercice 5.5. La table d'objectifs compte 218 lignes (magasin × mois) ; l'extrait ne porte que sur un magasin pendant douze mois, donc 12 paires à retrouver, et les douze sont là. En sens inverse, à l'échelle de la population des 240 000 ventes, deux groupes de ventes n'ont aucun objectif — le magasin 4 en février et en mars 2024 — et aucune ligne d'objectif n'est orpheline de ventes. Ce que l'exercice démontre : rapprocher deux tables n'est pas un agrégat, c'est un double comptage, dans les deux sens, et il se fait avant le premier ratio cité. Sans lui, le ratio porte sur un périmètre qu'aucun des deux fichiers ne définit.

13. Mini-projet M03.P1 — suite : le volet agrégats (45 min)

Commande. Livrez dans `agregats_atelier` la feuille `Contrôle agrégats` qui permet à un lecteur de vérifier n'importe quel chiffre du classeur sans ouvrir une seule formule.

Livrables numérotés. (1) la grille par catégorie (7 lignes × 5 colonnes : lignes, CA, moyenne, min, max) et sa fermeture par la partition, avec les totaux en pied ; (2) la grille par vendeur et la grille par mois, avec en dessous la démonstration chiffrée que la moyenne des moyennes est fausse ; (3) le tableau des quatre compteurs (`NB`, `NBVAL`, `NB.VIDE` et le compteur de zéros) sur les colonnes numériques des deux feuilles, brute et propre, avec pour chaque case ce qu'elle veut dire ; (4) un encadré « questions à dénominateur » de cinq lignes, où chaque pourcentage cité du classeur est écrit avec sa plage et son nombre de lignes ; (5) le rapprochement d'objectifs par `NB.SI.ENS` à deux critères, avec les mois manquants nommés.

Barème (20 points, seuil 13). Grilles justes et partitions fermées (6) · démonstration moyenne des moyennes (3) · tableau des compteurs complet et commenté (4) · encadré à dénominateurs (4) · contrôle de clés sur les objectifs (3).

14. Résumé du chapitre

Les fonctions d'agrégat ne diffèrent que par ce qu'elles regardent et ce qu'elles ignorent. `NB` compte les nombres, `NBVAL` tout ce qui est écrit, `NB.VIDE` les trous — et l'écart entre les trois est un diagnostic de typage, pas une coquetterie : 0, 489 et 0 sur la colonne des montants du fichier reçu. La famille `.SI`, puis `.SI.ENS`, ajoute une puis plusieurs conditions, avec un ordre d'arguments qui surprend (la plage de somme d'abord) et une exigence silencieuse : des plages de même dimension et des critères du même type que les cellules — à cette nuance près que `NB.SI` convertit les textes numériques que `SOMME` ignore.

Le vrai sujet est le dénominateur. Une même colonne de remises rend 1,8 %, 5,1 % et 33,8 % selon la question, et les trois sont justes ; une moyenne amputée de huit lignes se déplace de 1 876 FCFA ; une moyenne des moyennes par catégorie donne 98 281 FCFA au lieu de 75 152 FCFA, soit 30,8 %. D'où les trois réflexes du chapitre : écrire la question à côté du nombre, fermer la

partition (les sept catégories rendent 480 lignes et 36 073 185 FCFA au franc près), et citer la méthode là où elle existe — `QUARTILE.INC` rend 12 950 et 74 234, `QUARTILE.EXC` 12 891,5 et 74 304,5.

Enfin, sur une colonne texte, un agrégat rend un nombre plausible et jamais une erreur : le contrôle n'est donc pas un agrégat, c'est un compteur.

15. À retenir

À retenir. Aucun agrégat sans son dénominateur : 5,1 % de remise moyenne ne veut rien dire sans « sur les 173 lignes remises ». La parenthèse fait la rigueur, et elle coûte douze mots.

À retenir. Sur une colonne non typée, `SOMME` rend 0 et `NB.SI` rend 0 sans un bruit. Le contrôle d'un agrégat est un compteur (`NB` contre `NBVAL`), pas une relecture de la formule.

À retenir. Fermez la partition : la somme des sous-ensembles doit rendre le total. C'est le seul test qui prouve en même temps la couverture, l'absence de doublon et l'orthographe des critères.

1. Une moyenne n'est pas une moyenne des moyennes — on re-somme et on re-compte, toujours.
2. Un critère de date passe par `DATE()` et une concaténation, jamais par une date tapée en texte.
3. Un 0 n'est pas un vide, et `<>0` compte les vides : les trois écritures se distinguent à la lecture du résultat.
4. `SOUS.TOTAL` et `AGREGAT` voient le filtre, `SOMME` ne le voit pas : précisez lequel est à l'écran.

16. Évaluation formative (auto-correction, 10 min)

1. Une colonne contient 480 cellules : 400 nombres, 70 textes, 10 vides. Que rendent `NB`, `NBVAL`, `NB.VIDE` et `SOMME` ?
2. Écrivez, sans fichier sous les yeux, la formule du CA des lignes de la catégorie Matériaux dont la quantité dépasse 50 **et** la date est en septembre 2025 — avec les bons ordres d'arguments.
3. « La remise moyenne est de 1,8 % » : que doit-on ajouter à cette phrase pour qu'elle soit défendable, et quelle autre valeur légitime peut-on lui opposer ?
4. Pourquoi la moyenne des CA par catégorie n'est-elle pas la moyenne du CA de la période, et qu'est-ce que l'écart entre les deux dit de la structure des ventes ?
5. Un collègue obtient 0 avec une condition de date. Citez les deux causes les plus probables, dans l'ordre, et le contrôle qui les distingue en une formule.

Question ouverte. Vous devez livrer une grille par magasin (6 lignes) et par mois (12 colonnes) avec un seul contrôle qui prouve qu'elle est complète. Écrivez ce contrôle, et dites ce qu'il ne vérifie pas.

Corrigé de l'évaluation formative.

1. `NB` = 400 (les nombres), `NBVAL` = 470 (tout ce qui est écrit, textes compris), `NB.VIDE` = 10, `SOMME` = la somme des seuls 400 nombres — un total amputé de 70 lignes, sans le moindre message.
2. `=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[categorie];"Matériaux";TableVentes[quantite];">50";TableVentes[date];">="&DATE(2025;9;1);TableVentes[date];"<"&DATE(2025;10;1))` — somme en

premier, puis des triplets plage/critère, et la borne haute **exclusive** pour ne pas double-compter le 1^{er} octobre.

3. Le dénominateur et la plage : « 1,8 % sur les 480 lignes du tableau ». On lui oppose « 5,1 % sur les 173 lignes remises » — et le troisième chiffre, 12 198 008 FCFA de CA remisé, soit 33,8 %, qui est celui qui parle d'argent.
4. Parce que les catégories n'ont pas le même nombre de lignes (24 pour Bois & panneaux, 143 pour Matériaux) : la moyenne des moyennes surpondère les petites catégories. L'écart — ici 98 281 contre 75 152 FCFA — dit que quelques catégories à gros tickets tirent l'ensemble ; c'est une information sur la structure, pas une erreur.
5. Une colonne de dates restées du texte (la conversion n'a pas été faite, ou la ligne est hors du tableau) ; ou le critère écrit en texte (" $\geq$ 01/09/2025") au lieu de " $\geq$ "&DATE(2025;9;1). Le contrôle qui les distingue : =NB(TableVentes[date]) — 480 si la colonne est typée, 0 ou presque sinon. Question ouverte — Réponse attendue : le contrôle est =SOMME(nb_par_magasin) = 480 répété sur chaque ligne de mois, soit une grille de totaux croisés dont la diagonale ferme ; ce qu'il ne vérifie pas, c'est la **qualité** des lignes comptées (un montant texte est compté comme une ligne valide), ni les lignes **sans** magasin ou sans date, qui n'entrent dans aucune case et ne cassent donc pas le total si le périmètre est pris large.

Suite du module. Le chapitre C06 assemble les tables : RECHERCHEV et ses quatre pièges, RECHERCHEX, INDEX/EQUIV, et les fonctions de texte qui réparent une clé (GAUCHE, STXT, SUBSTITUE, JOINDRE.TEXTE) — celles qui transforment les 103 lignes sans client identifiant en lignes qui en ont un.

Module M03.C06 — Rechercher et assembler : RECHERCHEV et ses quatre pièges, RECHERCHEX, INDEX/EQUIV, les fonctions de texte

Outil de ce chapitre : le tableur seul, sur le classeur de l'atelier. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. Une recherche n'est pas une formule, c'est une **clé**. Avant de taper RECHERCHEV, on répond à trois questions : qu'est-ce qui identifie une ligne sans ambiguïté, dans quelle colonne de la table de référence se trouve cette clé, et que doit rendre la cellule quand la clé n'y est pas. Les quatre pièges de RECHERCHEV — l'approximation par défaut, le premier match, l'interdiction de regarder à gauche, l'index de colonne figé — ne sont pas des coquetteries d'interface : chacun produit un **nombre faux sans message**, et sur l'extrait de l'atelier ils sont tous les quatre démontrables.

Base de travail. Le classeur 01_socle_donnees/data/projection/m03_classeur_atelier.xlsx, sur les feuilles ventes (le tableau TableVentes, 480 lignes), ventes_brutes (le fichier reçu, 489 lignes en texte), et les quatre tables de référence clients (371 lignes), produits (154 lignes), vendeurs (22 lignes), magasins (6 lignes). Les fiches 17 à 20 et 22 à 23 de la feuille Calculs sont celles de ce chapitre. Séparateur point-virgule, UTF-8, franc CFA (FCFA), virgule décimale, TVA 18 %. Chiffres cités : 01_socle_donnees/data/reference/chiffres_cites.md, section M03. Copie de travail : recherches_atelier.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Choisir la clé** d'un rapprochement, et vérifier qu'elle est unique dans les deux tables avant de rechercher.
2. **Écrire** une RECHERCHEV exacte, et nommer ses quatre pièges avec, pour chacun, le symptôme visible.
3. **Écrire** la même recherche en RECHERCHEX et en INDEX/EQUIV, et dire laquelle porter dans un classeur destiné à Excel 2019 ou à LibreOffice Calc.
4. **Réparer une clé en texte** avec la famille des fonctions de texte : SUPPRESSE, SUBSTITUE, CNUM, GAUCHE, DROITE, STXT, NBCAR — dans le bon ordre.
5. **Fabriquer une clé composite** avec &, CONCAT et JOINDRE.TEXTE, et savoir que le zéro de padding est un piège de texte, pas de nombre.
6. **Rejeter** DECALER et INDIRECT comme outils de construction, et les garder comme outils de lecture.

2. Pourquoi cette notion est importante

Un analyste qui ne sait pas rechercher recopie. Un analyste qui sait rechercher mais pas choisir sa clé produit des chiffres faux qui ont l'air vrai. La seconde situation est la plus grave des deux, et l'atelier en donne une mesure : joindre les ventes sur le seul numéro de ticket — ce qui semble naturel, puisque chaque ligne en porte un — rend 24 915 054 FCFA de chiffre d'affaires

là où le tableau en contient 36 073 185. Il manque 11 158 131 FCFA, soit 30,9 % du total, parce que cent quatre tickets portent deux à quatre lignes et que RECHERCHEV s'arrête à la première.

Autour de cette erreur centrale, trois raisons d'apprendre le chapitre :

1. **Les tables de référence sont le métier.** Un prix, un nom de client, une catégorie : l'information vit dans une table, le fichier de ventes ne porte que la clé. Rapprocher, c'est lire l'entreprise — les modules suivants refont la même opération en SQL puis en Python.
2. **Le #N/A est une information.** 103 lignes ne trouvent pas leur client : 85 comptoirs sans identifiant, 18 cellules vides. Deux causes, deux traitements ; les noyer dans un 0 fabrique un faux rapport.
3. **La recherche se branche sur la propreté du texte.** " 10857" ne matche pas 10857, 9 ne matche pas 09. Les fonctions de texte ne décoorent pas : elles rendent la recherche possible.

3. Explication simple

Imaginez un fichier de 480 lignes où chaque ligne dit : « ticket T05-250101-075845, client 10857, 3 sacs de plâtre, 56 658 FCFA ». Ces trois nombres ne disent pas **qui** est le client, **ce que** vaut le produit au catalogue, **où** est le magasin. Pour le savoir, on va chercher dans une autre table, avec le numéro comme étiquette — c'est la clé.

Une recherche, c'est une valeur à trouver, une plage où la trouver, une colonne où prendre la réponse. RECHERCHEV exige que la plage commence par la colonne des clés et que la réponse soit **à droite** : c'est sa limite, et la source de trois de ses quatre pièges. RECHERCHEX sépare « où je cherche » et « ce que je rapporte », et peut donc aller à gauche, prévoir un repli, et chercher depuis la fin.

Quand la clé est un texte sale, on la nettoie d'abord : SUPPRESSE ôte les espaces de bord, SUBSTITUE remplace le séparateur de milliers, CNUM convertit le résultat. Ces trois-là ne cherchent rien : elles rendent la recherche possible.

Dans les faits. Dans un fichier de pilotage transmis par un magasin, la colonne `client` est un nombre chez l'un, du texte chez l'autre, et le rapprochement marche chez le premier, échoue chez le second, sans erreur de connexion affichée. Les équipes qui ont appris ce chapitre commencent leurs classeurs par un contrôle de clé ; les autres commencent leurs réunions par « ce n'est pas possible, ton total est faux ».

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Clé — key	La valeur qui identifie une ligne dans les deux tables.	Une clé non unique dans la table de recherche renvoie un arbitraire.
Table de référence — lookup table	La table qui porte l'information qu'on veut ajouter.	Oublier qu'elle peut être triée, filtrée, ou périmée.
Recherche exacte — exact match	FAUX en 4 ^e argument de RECHERCHEV, 0 en 5 ^e de RECHERCHEX.	Omettre l'argument : on bascule en approximatif, silencieusement.
Recherche approximative — approximate match	Utile pour des tranches (grilles de tarifs), exige un tri croissant.	L'employer par paresse sur une table non triée.
#N/A — not available	« je n'ai pas trouvé », et donc tout ce qui dépend de moi non plus.	Le masquer par réflexe avant d'avoir compté les cas.

Français — English	Définition simple	Piège à éviter
Recherche à gauche — left lookup	Renvoyer une colonne située avant la colonne de clé.	Vouloir le faire en RECHERCHEV avec un index négatif : impossible.
Clé composite — composite key	Clé fabriquée en collant deux colonnes (magasin & "-" & mois).	Le zéro non rempli : "2025-9" n'est pas "2025-09".
Formule volatile — volatile	DECALER, INDIRECT, ALEA : recalculées à chaque frappe.	Bâtir un classeur de 240 000 lignes avec 20 000 DECALER.

Définition. Une **clé unique** est une valeur qui apparaît une fois et une seule dans la table où l'on cherche. `clients[id_client]` en est une : 371 lignes, 371 identifiants distincts. Un `n_ticket` ne l'est pas dans le tableau de ventes : 480 lignes, 316 tickets, et un ticket peut porter jusqu'à quatre lignes.

Définition. Une **clé typée** est une clé du même type des deux côtés. 10857 (nombre) et "10857" (texte) sont deux clés différentes pour Excel : la seconde ne trouve jamais la première, et le symptôme est un #N/A propre, sans avertissement de format.

Définition. Un **repli** est la valeur renvoyée quand la recherche échoue : le 4^e argument de RECHERCHEX, contre une fonction SIERREUR autour de RECHERCHEV. La différence est de coût : le repli ne masque que le #N/A, SIERREUR masque aussi #VALEUR! et #NOM?, donc vos fautes de frappe.

Définition. Une fonction de texte **découpante** (GAUCHE, DROITE, STXT) compte en caractères, pas en significations : elle ne sait pas que le 4^e caractère est un tiret. Sauf si la longueur est garantie, on découpe à partir d'une borne trouvée (TROUVE, CHERCHE) ou depuis la fin (DROITE), jamais à partir d'un numéro chance.

5. Cours approfondi

Une clé, trois éclairages

`n_ticket = T05-250101-075845` (17 caractères)

T05-250101-075845

<code>=GAUCHE(A2;3)</code>	T05	le magasin (colonne D)
<code>=STXT(A2;5;6)</code>	250101	la date du ticket, vérifiée
<code>=DROITE(A2;6)</code>	075845	le n° du ticket, pas de la ligne
<code>=NBCAR(A2)</code>	17	17, ici comme en population

`=JOINDRE.TEXTE(" — ";VRAI;[vendeur];[categorie])`
une clé de regroupage fabriquée, pas recopiée

Quatre pièges, un chiffre chacun

1. Le 4^e argument omis

vaut VRAI : recherche approximative.
0 faute ici, 233 sur 377 après un tri par nom.

2. Le premier match seulement

il ne rend qu'une ligne sur quatre.
CA lu 24 915 054 au lieu de 36 073 185.

3. #N/A se propage

une clé absente contamine la formule fille.
103 lignes : 85 zéros, 18 vides.

4. Rechercher à gauche

RECHERCHEV n'en sait rien faire.
id_produit est avant designation.

RECHERCHEX ferme 3 et 4 ; INDEX/EQUIV ferme 2 et 4.

La même recherche, écrite de trois manières

<code>=RECHERCHEV(F2;clients!A:B;2;FAUX)</code>	une colonne insérée casse l'index 2
<code>=INDEX(clients!B:B;EQUIV(F2;clients!A:A;0))</code>	compatible 2019 et LibreOffice, sans index fixe
<code>=RECHERCHEX(F2;clients!A:A;clients!B:B;"comptoir";0;-1)</code>	repli, match exact, et il cherche à gauche
<code>=CNUM(SUBSTITUE(ventes_brutes!M2;" ";""))</code> rend 56 658 : réparer avant de chercher.	
RECHERCHEX et JOINDRE.TEXTE exigent Microsoft 365 ou Excel 2021 ; en 2019, ces noms rendent #NOM?.	

5.1 La clé d'abord, la formule ensuite

Le tableau `ventes` porte quatre colonnes qui peuvent servir de clé, et elles ne se valent pas :

Colonne	Cardinalité sur l'extrait	Unique ?	Sert à
n_ticket	316 valeurs pour 480 lignes	non	rapprocher les lignes d'une même vente
client	462 valeurs renseignées, dont 85 à zéro	non	joindre la table clients
produit	libellés complets	non	joindre produits (mais sur 40 caractères)
la ligne elle-même	480 lignes	oui	ne sert à rien : le numéro de ligne n'existe pas

La bonne question n'est donc pas « quelle formule ? » mais « **à quel grain** je veux raisonner ? ». Au grain de la ligne, il n'y a pas de clé dans le fichier : c'est pour cela que le module M02 ajoutait un identifiant de ligne en base de données. Au grain du ticket, n_ticket est la bonne clé, et elle est **propre** ici : le numéro est fabriqué à partir du nom de fichier, donc un ticket = une vente = une à quatre lignes. Vérifiez-le avant de vous en servir : =NB.SI(TableVentes[n_ticket];A2) rend le nombre de lignes du ticket de la ligne courante, et le maximum est 4.

Le ticket encode deux choses que le tableur ne voit pas : GAUCHE(A2;3) rend le magasin (T05), et les six caractères suivants la date 250101. Contrôlée contre la colonne date, cette date encodée ne donne **aucune** divergence sur les 480 lignes de l'extrait, ni sur 240 000 lignes de la population. Ce n'est pas un résultat à exploiter, c'est un **contrôle de santé** : il prouve que les deux écritures du monde réel n'ont pas dérivé, ce qui est exactement ce qui casse à un réimport. Le chapitre écrit ce contrôle deux fois — avec STXT, il passe ; en oubliant le 0 des mois courts, il échoue sur les 360 lignes de janvier à septembre, et cette seconde écriture est celle que produisent les gens qui n'ont pas lu ce chapitre.

À retenir. Une clé se choisit au grain où l'on veut raisonner, et se vérifie en un compteur : =NB.SI(colonne;valeur) doit rendre 1 pour la table de référence, et ce que vous décidez pour le fichier de faits.

5.2 RECHERCHEV : quatre pièges, quatre symptômes mesurés

=RECHERCHEV(valeur_cherchée;table;no_colonne;FAUX) — anglais VLOOKUP. Quatre choses s'y passent mal si on ne les décide pas.

1. Le quatrième argument omis vaut VAI, pas FAUX. Sans lui, la recherche est approximative : Excel prend la plus grande valeur inférieure ou égale, et **exige** que la première colonne de la table soit triée. Ici le piège ne mord pas — clients[id_client] est triée par construction, et la version fautive rend la bonne réponse sur les 377 lignes à identifiant valide, 0 divergence. Triez la même table par nom : 233 réponses changent ; par ville : toutes. Le symptôme n'est donc pas une erreur, c'est un **résultat qui devient faux quand quelqu'un d'autre trie le fichier**. Écrivez FAUX ; et si l'approximation est voulue (grille de tarifs, tranches de remise), consignez le tri attendu dans la feuille.

2. Elle s'arrête au premier match. Sur une clé non unique, RECHERCHEV renvoie la première ligne trouvée — une décision qu'elle ne vous demande pas. L'exemple du §2 vient de là : sommer le montant attaché à chaque ticket rend 24 915 054 FCFA au lieu de 36 073 185 FCFA. Le contrôle est un compteur, pas une recherche : =MAX(NB.SI(TableVentes[n_ticket];TableVentes[n_ticket])) doit valoir 1, sinon vous ne cherchez pas au bon grain.

3. Elle ne regarde pas à gauche. La colonne de clé est la première de la plage, et l'index ne peut être que supérieur. Dans produits, la désignation est en colonne B et l'id_produit en A : récupérer l'identifiant à partir du libellé est **impossible** en RECHERCHEV, et aucun bricolage ne change cela — décaler la plage de colonne, c'est changer l'index, et l'index 0 n'existe pas. Les issues sont INDEX/EQUIV, RECHERCHEX, ou une colonne utilitaire dans la table.

4. L'index de colonne est un nombre, pas un nom. Pour le prix depuis `produits`, on écrit `=RECHERCHEV(G2;produits!B:F;5;FAUX)` ; insérez une colonne en C et la formule rend le poids — un nombre, donc rien d'affichant. Le remède est dans l'écriture : `=RECHERCHEX(G2;produits!designation;produits!prix_vente_ht;"-")` dans un tableau, ou `=RECHERCHEV(G2;produits!B:F;EQUIV("prix_vente_ht";produits!B1:F1;0);FAUX)` quand la version bloque.

Ajoutez le cinquième, qui n'est pas un piège de la fonction mais du fichier : **le texte ne matche jamais le nombre**. Sur `ventes_brutes`, la colonne `client` est du texte (471 cellules écrites, 18 vides) alors que `clients[id_client]` est numérique : partie de la feuille sale, la même recherche échoue sur les 489 lignes. Un `#N/A` intégral est plus rassurant qu'une réussite partielle — il dit que le problème est de type, pas de données.

Attention. `=RECHERCHEV(1229678;ventes!G2:J481;4;FAUX)` — la fiche 17 de `Calculs` — rend `#N/A`, et ce n'est pas une formule cassée : on cherche un montant (nombre) dans une colonne de libellés (texte). Avant de corriger une `RECHERCHEV`, lisez les deux types, pas seulement les deux plages.

5.3 RECHERCHEX : la recherche qui pose les bonnes questions

`=RECHERCHEX(valeur;table_cherchée;table_résultat;[non_trouvée];[mode];[sens])` — anglais `XL00-KUP`, disponible dans Microsoft 365, Excel 2021 et 2024, et dans LibreOffice Calc récent ; en Excel 2019, le nom renvoie `#NOM?`. Six arguments dont trois sont optionnels, et les trois optionnels sont précisément ce que `RECHERCHEV` ne savait pas faire :

Argument	Écriture courante	Ce qu'il règle
<code>non_trouvée</code>	<code>"comptoir", "-"</code>	le <code>#N/A</code> devient une réponse choisie
<code>mode</code>	0 exact (défaut), -1 exact sinon précédent, 2 jokers	l'approximation n'est plus implicite
<code>sens</code>	1 depuis le début, -1 depuis la fin	« la dernière ligne du ticket » devient une option

La fiche 18 est une `RECHERCHEX` complète : `=RECHERCHEX(ventes!M356;ventes!M2:M481;ventes!G2:G481;"non trouvé")` — chercher un montant, rapporter un libellé : « `Lame de lambris 4 m — réf 2` ». Le chemin est tordu à dessein (un montant n'est pas une clé) et montre que `RECHERCHEX` n'a que faire de l'ordre des colonnes.

Pour les 103 lignes sans client, le repli seul ne suffit pas : il confondrait le comptoir (le client n'existe pas, et c'est normal) avec la ligne mal renseignée (elle doit être corrigée). Deux replis distincts, un seul contrôle :

```
=SI(TableVentes[@client]="";"non  
saisi";RECHERCHEX([@client];clients[id_client];clients[nom];"comptoir"))
```

Le `SI` traite les 18 cellules vides, le repli les 85 zéros, et l'ordre compte : une cellule vide ne cherche pas « rien » mais la valeur 0, et trouverait un client numéroté 0 si la table en contenait un — `clients` commence à 125, le prochain fichier ne vous doit rien.

Conseil professionnel. Quand un classeur doit tourner chez des gens dont vous ne connaissez pas la version, écrivez la voie `INDEX/EQUIV` dans les cellules et la `RECHERCHEX` en commentaire de note. Un lecteur avec Microsoft 365 gagne du temps ; un lecteur avec 2019 n'a pas `#NOM?` en pleine réunion.

5.4 INDEX / EQUIV : la voie qui marche partout

Deux fonctions au lieu d'une : **EQUIV** (anglais **MATCH**) rend la **position**, **INDEX** rend la **valeur à cette position**. `=INDEX(clients!B:B;EQUIV(F2;clients!A:A;0))` lit le nom du client de la ligne courante, quel que soit l'ordre des colonnes, et fonctionne dans toutes les versions, Excel comme LibreOffice.

Trois écritures du troisième argument d'**EQUIV**, à connaître parce qu'elles reproduisent les pièges de §5.2 :

Écriture	Cherche	Contrepartie
<code>EQUIV(v;p;0)</code>	correspondance exacte	#N/A si absent
<code>EQUIV(v;p;1)</code>	plus grand inférieur ou égal	exige un tri croissant
<code>EQUIV(v;p;-1)</code>	plus petit supérieur ou égal	exige un tri décroissant

La fiche 20 du classeur isole le maillon : `=EQUIV(MAX(ventes!M2:M481);ventes!M2:M481;0)` rend **355** — la position dans la plage, pas le numéro de ligne : la plage commence en M2, donc la ligne réelle est 356. C'est de loin la source de décalage la plus fréquente en correction, et elle est silencieuse : un **INDEX** construit sur la mauvaise plage rend le produit du voisin, pas une erreur. La fiche 19 enchaîne les deux et retrouve le même libellé que **RECHERCHEX**.

Deux usages d'**INDEX** qui n'ont rien à voir avec la recherche, et qu'il faut savoir parce qu'ils remplacent **DECALER** : `=INDEX(TableVentes[montant_ttc];EQUIV(A2;TableVentes[n_ticket];0)+1)` rend la **ligne suivante** d'un ticket, et `=INDEX(A1:M1;EQUIV("remise";A1:M1;0))` rend un nom de colonne **par son titre** — c'est avec cela qu'on écrit une **RECHERCHEV** qui ne casse plus quand une colonne s'insère. Enfin **EQUIV** accepte les jokers en mode 0 : `=NB.SI(TableVentes[produit];"*réf 2*")` compte 223 lignes sur 480 et **EQUIV** avec la même écriture trouve la première — 77 des 154 libellés du catalogue finissent par **réf 2**, ce qui explique le rapport et interdit d'en conclure quoi que ce soit sur les ventes.

À retenir. **INDEX/EQUIV** n'est pas la voie des pauvres : c'est la voie où chaque pièce se vérifie séparément. La position d'abord — un nombre petit, entre 1 et la taille de la plage — puis la valeur.

5.5 Nettoyer la clé : SUPPRESpace, SUBSTITUE, CNUM

Rien de ce qui précède ne fonctionne sur le fichier reçu, et c'est le point d'articulation du chapitre. La cellule `ventes_brutes!M2` contient 56 658 : un nombre en apparence, du texte pour Excel. La fiche 22 le répare en une formule : `=CNUM(SUBSTITUE(ventes_brutes!M2;" ";"))` → 56 658. Trois fonctions, trois rôles :

Fonction	Rôle sur la clé	À ne pas faire
<code>SUPPRESpace (TRIM)</code>	ôte les espaces de bord et réduit les doubles espaces internes	croire qu'elle retire les espaces insécables
<code>SUBSTITUE (SUBSTITUTE)</code>	remplace une chaîne précise : le séparateur de milliers, le point par la virgule	confondre avec REPLACE , qui écrit à une position
<code>CNUM (VALUE)</code>	convertit en nombre, après nettoyage	l'employer seul : <code>=CNUM("56 658")</code> rend #VALEUR!
<code>GAUCHE / DROITE / STXT</code>	découpe : 3, 6, 17 caractères du ticket	découper à position fixe sur un libellé de longueur variable
<code>NBCAR (LEN)</code>	la seule qui dit si le découpage est licite	—

Le cas réel est la clé `client` du fichier reçu : `" 10857 "` ne matche pas 10857. La réparation se lit de l'intérieur vers l'extérieur, `=RECHERCHEX(CNUM(SUPPRESpace(ventes_brutes!F2));clients!A:A;clients!B:B;"-")`, et se contrôle : `=NB(CNUM(SUPPRESpace(ventes_brutes!F2:F490)))`, validée en

Ctrl + Maj + Entrée hors 365, doit rendre 471 — un de moins signale une valeur non numérique que la recherche aurait avalée sans rien dire.

Sur les libellés du catalogue — 154 lignes, toutes avec le tiret « — », 77 finissant par réf 2 — extraire la variante par DROITE(A2;1) marche ici et casse au premier libellé sans variante. La version qui tient est =SI(SIERREUR(TROUVE("réf ";A2);0)=0;"";STXT(A2;TROUVE("réf ";A2)+4;2)) — on cherche la borne, on découpe depuis elle, et le découpage tolère une fin de chaîne courte. Retenez la règle : **tant que vous n'avez pas compté les longueurs, on découpe depuis une borne trouvée, jamais depuis un rang.**

Attention. SIERREUR autour d'une recherche ne répare rien : il rend indiscernables le client absent, la clé restée du texte, la table mal orthographiée et votre faute de frappe sur le nom de la fonction. Avant d'envelopper une recherche, comptez les échecs par cause — =NB.SI(colonne_repli;"-"), =NB.VIDE(clé), et le nombre de clés non numériques — et n'écrivez le repli global qu'une fois cette répartition dans la feuille.

5.6 DECALER et INDIRECT : comprendre pour ne pas construire

DECALER(réf;décal_lignes;décal_colonnes;[hauteur];[largeur]) (anglais OFFSET) renvoie une plage décalée ; INDIRECT("texte") (même nom) renvoie la plage dont le nom est écrit dans ce texte. Elles sont pédagogiques parce qu'elles donnent à voir ce que les autres recherches font en silence : DECALER **construit une plage à l'exécution**, INDIRECT **construit une référence à partir d'un texte**.

Trois raisons de ne pas en faire des outils de recherche :

1. Ils sont **volatils** : recalculés à chaque modification du classeur, pas seulement quand leurs paramètres changent. Sur 480 lignes, rien ne se passe ; sur les 240 000 de la population, un classeur à vingt mille DECALER devient inutilisable, et la cause est invisible dans les formules.
2. INDIRECT ne vérifie rien : renommer un onglet ne casse pas la formule, elle rend #REF! à la première ouverture — souvent après l'envoi, le pire moment.
3. FILTRE, UNIQUE (C07) et Power Query (C08) les remplacent dans la quasi-totalité des cas d'atelier.

Ses deux usages légitimes : la **fenêtre glissante** d'un tableau de suivi (=MOYENNE(DECALER(Cellule;0;0;3;1))), alors que deux INDEX bord à bord font la même chose sans volatilité, et la liste déroulante dont le contenu dépend d'une autre cellule. Le critère tient en une phrase : si le classeur recalcule souvent, la plage doit exister **avant** le calcul.

5.7 Assembler : &, CONCAT, JOINDRE.TEXTE

& colle. CONCAT(a;b;...) (anglais CONCAT, Excel 2019+) colle une plage entière. JOINDRE.TEXTE(sép.;vrai;...) (anglais TEXTJOIN) colle avec séparateur, et c'est la seule des trois qui sache **ignorer les vides** — le deuxième argument, VRAI, vaut « ne pas émettre de séparateur pour rien ».

L'usage n'est pas le joli libellé : c'est **fabriquer la clé**. La table objectifs du classeur en porte un exemple, déjà construit : la colonne annee_mois, écrite 2023-01, sert de clé de rapprochement magasin × mois, et c'est le SOMME.SI.ENS de la fiche 28 qui la retrouve. Fabriquez-la à la main en collant bêtement les colonnes (=B2&"-"&C2) et vous obtenez 2023-1 : la clé ne matche que sur 45 des 218 lignes de la table, et l'échec est maximal précisément sur les neuf premiers mois — ceux des soldes, des réouvertures et de la rentrée, soit tout ce qui intéresse un direction commerciale. La bonne écriture passe par TEXTE : =B2&"-"&TEXTE(C2;"00"), qui rend 2023-01 pour les 218 lignes.

JOINDRE.TEXTE brille quand une liste doit tenir dans une cellule : la fiche 23, =JOINDRE.TEXTE(" ;VRAI;UNIQUE(ventes!H2:H481)) , rend les sept catégories dans l'ordre du fichier — dont **Electricité sans accent**, exactement comme dans la donnée, ce que doit aussi rendre un critère de recherche (§5.4 de C05). Une liste recopiée à la main est une liste fausse.

Définition. Le **padding** est l'ajout de zéros non significatifs pour que deux écritures d'un même nombre aient la même longueur de texte. En recherche, un mois ou un jour non paddé casse une clé composite ; TEXTE(valeur;"00") est le geste, et NBCAR en est le contrôle (2 caractères, pas 1).

Boîte à outils. =RECHERCHEV · =RECHERCHEX · =INDEX · =EQUIV pour rapprocher ; =CNUM · =SUPPRESPE · =SUBSTITUE · =ARRONDI pour réparer avant de rapprocher ; =GAUCHE · =DROITE · =STXT · =NBCAR · =TROUVE · =CHERCHE pour découper ; & · =CONCAT · =JOINDRE.TEXTE · =TEXTE pour assembler ; =SIERREUR pour un repli en 2019. À éviter : =DECALER, =INDIRECT comme fondation. Trois contrôles à poser dans tout classeur de recherche : =MAX(NB.SI(clé)) sur la table de référence (= 1), =NB.COUPLE ou =NB.SI du taux de recouvrement de la clé, et le =NBCAR maximal de toute chaîne produite par une concaténation de liste.

6. Exemple concret : la jointure qui marchait trop bien

Une analyste doit ajouter le nom du client à chaque ligne de vente pour préparer un mailing. Elle ouvre **clients**, voit la colonne A pleine d'identifiants triés, écrit =RECHERCHEV(F2;clients!A:B;2) — sans le FAUX, parce que le classeur lui a toujours bien réussi — et obtient 480 noms. Elle croise sur les 377 lignes à clé valide : tout tombe juste. Elle livre.

Trois jours plus tard, le commercial qui exploite la liste signale que les noms sont « parfois à côté ». Le fichier n'a pas changé ; **le tri de la table clients a changé**, quelqu'un l'a rangée par ville pour un envoi postal. La recherche approximative, qui ne disait rien sur une table triée, s'est mise à raisonner sur une table désordonnée, et elle a produit des réponses **toutes fausses** : 377 sur 377, sans un #N/A pour avertir.

En parallèle, elle avait rapproché les ventes des tickets : 316 « clients mailing » au lieu de 480 lignes, ce qui lui semblait une bonne nouvelle — elle croyait dédoublonner. C'était le piège 2 du §5.2, et le nom du destinataire venait de la première ligne du ticket, qui n'est pas le client des trois autres.

L'autopsie tient en trois lignes : le FAUX manquait ; la clé de regroupement n'était pas la clé du fichier ; le contrôle de recouvrement — 377 réussites sur 462 clés renseignées — n'avait pas été écrit, parce que la formule « marchait ».

7. Démonstration pas à pas : la feuille de rapprochement

Objectif. Une feuille **Rapprochement** qui ajoute à chaque ligne de **ventes** le client, le magasin et le prix catalogue, avec le taux de recouvrement affiché en tête.

Étape	Formule à écrire dans Rapprochement	Ce qui doit se lire
1	=RECHERCHEX(ventes!F2;clients!A:A;clients!B:B;"-comptoir") en B2, étirée	377 noms, 85 -comptoir, 18 -comptoir aussi
2	=SI(ventes!F2="";"-non saisi";B2) en C2	les 18 vides séparés des 85 zéros
3	=RECHERCHEX(ventes!D2;magasins!A:A;magasins!B:B;"inconnu") en D2	le nom long du magasin 5, 480 fois
4	=RECHERCHEX(ventes!G2;produits!B:B;produits!F:F;"hors catalogue") en E2	le prix HT, et aucun hors catalogue

Étape	Formule à écrire dans	Rapprochement	Ce qui doit se lire
5	=NB(B2:B481) et =NBVAL(B2:B481)	en tête	0 et 480 : des noms sont du texte, NB ne dit rien
6	=NB.SI(B2:B481;"-comptoir")		85, avec la mention « 18 non saisis, cf. colonne C »

Démonstration chiffrée du §5.2, à faire une fois en classe. En G2, la version fautive =RECHERCHEV(ventes!F2;clients!A:B;2) ; en H2, =SI(EXACT(B2;G2);"d'accord";"différent") étirée. Les deux colonnes disent « d'accord » 480 fois : l'approximation est invisible tant que clients est triée. Triez la copie par ville et relisez : 377 mentions « différent », et 103 cellules en #N/A — l'absence reste bruyante, la corruption du tri est silencieuse. C'est le geste qui fait comprendre le piège, et il ne coûte que deux tris.

Barème de la démonstration (10 points). Repli distinct pour vide et zéro (2) · recouvrement avec son dénominateur (2) · produits joint sur la désignation sans un « hors catalogue » (2) · contrôle NB/NBVAL commenté (2) · expérience du tri menée en copie et consignée (2).

Contrôles qualité du chapitre. (1) =MAX(NB.SI(clients[id_client];...)) vaut 1 : la clé de la table de référence est unique. (2) Taux de recouvrement : 377 sur 462 clés renseignées, et les 85 manquants sont le comptoir, pas une anomalie. (3) produits ne manque aucun des libellés de l'extrait : 0 hors catalogue sur 480 lignes. (4) Aucune formule du classeur de l'atelier n'emploie DECALER ni INDIRECT — vérifiez-le avec la boîte de dialogue Rechercher, c'est une ligne de votre rapport d'atelier.

8. Erreurs fréquentes

1. **Oublier FAUX** (ou 0) en recherche exacte : 0 divergence sur la table triée, 233 sur 377 après un tri par nom, 377 après un tri par ville. Le symptôme est une dégradation différée, jamais une erreur.
2. **Chercher avec une clé non unique** : le premier match gagne, et sur n_ticket cela fait perdre 11 158 131 FCFA de chiffre d'affaires (30,9 % du total de l'extrait).
3. **Vouloir regarder à gauche en RECHERCHEV** avec un index négatif : la réponse est INDEX/EQUIV ou RECHERCHEX, pas un bricolage de DECALER.
4. **Compter les réussites avec NB** sur une colonne de noms : NB rend 0 là où il y a 480 réponses ; le compteur du texte est NBVAL.
5. **STIERREUR pour taire les 103 #N/A d'un coup** : plus rien ne distingue le comptoir de la ligne oubliée, et le fichier ne peut plus être réparé — seulement regretté.
6. **Concaténer une clé de date sans padding** : =B2&"-"&C2 ne matche que 45 lignes sur 218 dans objectifs.
7. **Taper le critère de mémoire au lieu de le copier** : RECHERCHEV("électricité";...) rend #N/A alors que 53 lignes portent Electricité sans accent — les comparaisons sont insensibles à la casse, sensibles aux accents. Et SUPPRESSE ne suffit pas sur un fichier bourré d'espaces insécables : le détecteur est =NBCAR(A2) sur une cellule qui paraît propre.

9. Bonnes pratiques professionnelles

1. **Une colonne d'état par recherche.** La recherche proprement dite est dans B, l'interprétation (« comptoir », « non saisi », « trouvé ») dans C. Sans cette séparation, un rapport est une boîte noire.

2. **Toujours le taux de recouvrement en tête de feuille** : « 377 sur 462 », pas « tout va bien ». C'est le seul chiffre qui permet à un relecteur de juger le reste sans refaire les formules.
3. **Un repli par cause**. Trois causes d'échec, trois textes ; un repli unique est un mensonge poli.
4. **Écrire la voie compatible, commenter la voie moderne** dans tout fichier livré hors de votre service.
5. **Nommer les plages de référence** (clients!A:A → TableClients) : la recherche devient lisible, et le chapitre C07 montrera que le nom protège aussi des tris et des filtres. Jamais de tri sur une table de référence dans le fichier de travail — trie une copie, ou ajoutez une colonne rang_dorigine avant.
6. **Contrôler les longueurs avant de découper** : =MAX(NBCAR(colonne)) dit en une formule si STXT est licite.

10. Exercice guidé — la feuille client complétée (35 min, / 10)

Commande. Dans recherches_atelier, produisez la feuille Fiche client qui, pour chaque ligne de TableVentes, donne : ville du client, type de client, statut du vendeur, et le prix catalogue du produit — avec un repli par cause et les quatre contrôles du chapitre.

Étape	Geste	Ce qui doit se lire
1	=RECHERCHEX([@client];clients!A:A;clients!D:D;"-")	377 villes, 103 –
2	Séparez le vide du zéro : =SI([@client]="";"non saisi";étape 1)	18 lignes à « non saisi »
3	=RECHERCHEV équivalente avec FAUX, à côté	mêmes 377 réponses, et le même repli manquant
4	Vendeur : =INDEX(vendeurs!E:E;EQUIV([@vendeur];vendeurs!B:B;0))	4 statuts, aucun #N/A
5	Prix catalogue : =RECHERCHEX([@produit];produits!B:B;produits!F:F;"hors catalogue")	480 prix, 0 « hors catalogue »
6	Contrôles en tête : =NBVAL, =NB.SI(plage;"-"), =MAX(NBCAR(...)) sur chaque colonne fabriquée	trois lignes de chiffres commentées

Démarrage. À l'étape 1, n'élargissez pas la plage en vous promettant de décaler l'index : écrivez la colonne de réponse en clair. À l'étape 4, EQUIV cherche un **nom** dans vendeurs!B:B : si deux vendeurs portaient le même, la recherche deviendrait silencieusement arbitraire — d'où le contrôle d'unicité, écrit une fois en haut de la feuille (clients : 371 lignes, 371 identifiants distincts ; vendeurs : 22 lignes, 22 noms distincts). À l'étape 5, avoir 0 « hors catalogue » est un **résultat** : le fichier de ventes a été fabriqué depuis le catalogue, et cela ne se reproduira pas au prochain import.

Barème (10 points). Quatre recherches justes (4) · repli par cause et non global (2) · contrôle d'unicité de chaque table de référence écrit en tête (2) · comparaison écrite RECHERCHEX / RECHERCHEV en une phrase (1) · absence de DECALER et d'INDIRECT (1).

11. Exercices autonomes

Exercice 6.1 (★) — Les deux écritures d'une même recherche. Sur 20 lignes de ventes, écrivez la ville du client en RECHERCHEV avec FAUX, en RECHERCHEV sans quatrième argument, et en

INDEX/EQUIV. Triez la copie de `clients` par `nom` et relisez les trois colonnes. Lequel de vos trois résultats vous a trahi, et par quel signe ?

Exercice 6.2 (★) — Le recouvrement avant le tableau. Combien de lignes de `TableVentes[client]` ne trouvent pas leur clé dans `clients` ? Décomposez : vides, zéros, identifiants absents. Écrivez les trois compteurs, puis la phrase de trois lignes au responsable du magasin : le nombre, la cause, la demande.

Exercice 6.3 (★★) — Le ticket comme clé, et pourquoi il résiste. Construisez une colonne utilitaire qui donne, pour chaque ligne, le **nombre de lignes du même ticket**, puis une seconde qui donne le montant du ticket entier (`SOMME.SI` sur la clé). Vérifiez que la somme des montants de tickets, divisée par le nombre de tickets, ne rend pas la moyenne de 75 152 FCFA, et expliquez au passage ce que vaudrait la moyenne des montants de ticket.

Exercice 6.4 (★★) — Découper sans se faire avoir. À partir de `n_ticket` seul, fabriquez la date au format AAAA-MM-JJ avec `STXT` et `DATE`, le magasin avec `GAUCHE`, et comparez à la colonne `date`. Combien de divergences sur 480 lignes ? Reprenez en lisant le mois **sans padding** : où partent janvier à septembre ?

Exercice 6.5 (★★★) — La clé composite des objectifs. Rejoignez `TableVentes` et `objectifs` sur `magasin × mois` en écrivant la clé vous-même, naïvement (`=B2&"-"&C2`) puis avec `TEXTE` : donnez le nombre de lignes jointes dans les deux cas, et dites où ajouter **une fois** la colonne que vous êtes tenté d'ajouter 480 fois.

Exercice 6.6 (★★★) — Réparer le fichier reçu. Sans ouvrir `ventes`, faites tourner sur `ventes_brutes` la recherche de ville du client, d'abord brute puis avec `CNUM(SUPPRESPEACE(...))`. Comptez les `#N/A` des deux versions et dites pourquoi la première échoue partout — en nommant **les deux** types en présence.

12. Correction détaillée

Exercice 6.1. Les trois colonnes sont identiques tant que `clients` est triée par `id_client` : la version sans quatrième argument est **approximative**, et sur une clé triée croissante elle retombe juste — 0 divergence sur les 377 lignes à clé valide. Après un tri par `nom`, une seule des trois bouge : 233 noms différents sur 377 ; les deux écritures exactes, `FAUX` comme `EQUIV(...;0)`, gardent la bonne réponse. Le signe de trahison n'est pas une erreur, c'est un résultat **qui a bougé sans que personne ne touche aux formules** — la définition même d'une fragilité.

Exercice 6.2. `=NB.VIDE(TableVentes[client])` rend 18, `=NB.SI(TableVentes[client];0)` rend 85 : voilà les 103 échecs. Les réussites se comptent en colonne utilitaire — `=NB.SI(clients!A:A;[@client])` rendu 1 ou 0, dont la somme fait 377 — et non en une matrice que personne ne reliera : les 377 identifiants renseignés sont **tous** dans `clients`. Phrase attendue : « 480 lignes, 462 identifiants renseignés, 377 retrouvés dans le fichier `clients` ; 85 ventes sont des ventes comptoir sans identifiant et 18 lignes n'en portent pas ; écrivez `COMPTOIR` plutôt que 0, et renseignez les 18 vides ».

Exercice 6.3. `=NB.SI(TableVentes[n_ticket];A2)` vaut 1 pour les lignes seules et jusqu'à 4 sur les cent quatre tickets multi-lignes ; `=SOMME.SI(TableVentes[n_ticket];A2;TableVentes[montant_ttc])` donne le montant du ticket, et sa moyenne vaut 114 156 FCFA (316 tickets, 36 073 185 FCFA) — pas les 75 152 FCFA de la moyenne par ligne. Deux questions, deux dénominateurs : « panier moyen 114 156 FCFA » se défend, « vente moyenne » avec le même chiffre non.

Exercice 6.4. `=DATE(2000+STXT(A2;5;2);STXT(A2;7;2);STXT(A2;9;2))` reconstruit la date depuis le ticket : **aucune divergence** sur les 480 lignes, ni sur 240 000 en population. Mais si vous lisez le mois en 1 au lieu de 01, `DATE` digère le texte et la **clé texte** AAAA-MM, elle, ne matche plus : les

360 lignes de janvier à septembre perdent leur correspondance. La leçon n'est pas STXT, c'est que le nombre et le texte ne se pardonnent pas les mêmes choses.

Exercice 6.5. Clé naïve : 45 lignes jointes sur 218 — les seuls mois d'octobre à décembre, à deux chiffres. Clé paddée `=B2&"-"&TEXTE(C2;"00")` : 218 sur 218. La colonne à ajouter **une fois** est déjà dans `objectifs`, `annee_mois`, écrite 2023-01 : le travail de l'atelier est de l'utiliser, avec en tête le contrôle `=NB.SI(objectifs!D:D;ANNEE(ventes!B2)&"-"&TEXTE(MOIS(ventes!B2);"00"))`. Conclusion à en tirer : quand la table de référence porte la clé, le fichier de faits n'a pas à en fabriquer une.

Exercice 6.6. Sur `ventes_brutes!F2` brut : #N/A sur les 489 lignes, parce que la colonne est du texte (471 cellules écrites, 18 vides) et que `clients[id_client]` est numérique — deux **types**, pas deux valeurs. Avec `=CNUM(SUPPRESPE(ventes_brutes!F2))`, les lignes identifiées retrouvent leur ville et les 18 vides tombent en #VALEUR! sous CNUM — d'où le contrôle du vide **avant** la conversion, `=SI(F2="";"non saisi";RECHERCHEX(CNUM(SUPPRESPE(F2));clients!A:A;clients!B:B;"comptoir"))`. Notez le gain : la réparation n'a pas changé les données, elle a changé **le type**, et le taux de recouvrement est passé de 0 à 377 sur 462.

13. Mini-projet M03.P1 — suite : le volet recherches (45 min)

Commande. Ajoutez à `recherches_atelier` une feuille Enrichissement qui fait du tableau `ventes` un jeu prêt à agréger, et qui le prouve.

Livrables numérotés. (1) quatre colonnes enrichies (ville, type de client, statut du vendeur, prix catalogue), écrites en RECHERCHEX et en INDEX/EQUIV côte à côte, avec une colonne écart entièrement vide ; (2) le bloc de tête : unicité des trois clés de référence, taux de recouvrement par colonne, nombre de vides ; (3) les colonnes `lignes_du_ticket` et `montant_ticket`, avec la note qui explique pourquoi on ne moyenne pas la seconde ; (4) la clé composite mois de `objectifs`, comptée avant et après TEXTE (45 contre 218) ; (5) deux cents mots : ce qui était du texte, ce qui est devenu typé, ce qui reste non joignable.

Barème (20 points, seuil 13). Colonnes justes et doublées de la voie 2019 (6) · contrôles de tête (5) · vide et zéro traités séparément (3) · clé composite paddée et démontrée (3) · page de notes (3).

14. Résumé du chapitre

Une recherche rapproche deux tables sur une clé, et la clé décide de tout : son **grain** (ligne ou ticket), son **type** (nombre ou texte), son **unicité** (une ligne, ou un choix arbitraire). RECHERCHEV est universelle et fragile : approximative par défaut, arrêtée au premier match, incapable de regarder à gauche, figée sur un index de colonne. RECHERCHEX ferme trois de ces quatre portes et ajoute un repli typé par cause ; INDEX/EQUIV fait partout le même travail et reste la voie des classeurs partagés. Les fonctions de texte ne sont pas un à-côté : SUPPRESPE, SUBSTITUE et CNUM rendent les clés joignables (fiche 22 : 56 658), GAUCHE/STXT/DROITE les démontent, &, CONCAT et JOINDRE.TEXTE les reconstruisent — et le padding est la seule manière d'avoir raison sur un mois court (45 lignes sur 218, sinon).

Trois réflexes : **compter avant de chercher** (377 sur 462, et 103 échecs à deux causes) ; **vérifier le type des deux côtés** ; **ne jamais masquer un #N/A sans avoir dit lequel** — comptoir n'est pas non-saisi.

À retenir. Le piège le plus coûteux de RECHERCHEV ne rend aucune erreur : 0 divergence sur la table triée, 233 sur 377 après un tri par nom, 377 après un tri par ville. Un résultat juste aujourd'hui n'est pas un résultat fiable.

1. **RECHERCHEX(valeur;où;quoi;"repli";0;-1)** dit en une ligne ce que RECHERCHEV + SIERREUR + INDEX/EQUIV disent en trois : le repli, l'exactitude, le sens de recherche. Écrivez la voie compatible dans les cellules, commentez la voie moderne.
2. Une clé de date fabriquée par concaténation s'écrit `=A&"-"&TEXTE(B;"00")`, jamais `=A&"-"&B` : le caractère perdu fait disparaître trois mois sur douze — 360 lignes de l'extrait.

15. À retenir

1. **Tester l'unicité de la clé de chaque table de référence** : `=MAX(NB.SI(produits!A:A;produits!A:A))` doit valoir 1 — formule matricielle sur Excel 2019 et 2021 (à valider en Ctrl + Maj + Entrée), naturelle sur Microsoft 365. Si le maximum dépasse 1, votre recherche renvoie un arbitraire et ne vous préviendra pas.
2. **Un repli par cause d'échec**, pas un repli global : `SI(vide)` avant la recherche, `non_trouvée` dedans, et jamais `SIERREUR` autour de tout pour faire taire la feuille.
3. **Réparer les types avant d'assembler les clés** : `=NB.SI(plage;0)` sur la colonne de départ, puis `CNUM(SUPPRESPEACE(...))`, puis la recherche — dans cet ordre, et le taux de recouvrement comme preuve finale.

16. Évaluation formative (auto-correction, 10 min)

1. Une RECHERCHEV sans quatrième argument sur une table triée par sa clé : que se passe-t-il aujourd'hui, et après un tri par une autre colonne ?
2. Écrivez, sans fichier sous les yeux, la recherche du type de client d'une ligne de TableVentes, avec un repli distinct pour la cellule vide et pour l'identifiant absent, en une seule formule.
3. Pourquoi `=RECHERCHEV(libellé;produits!A:K;...)` ne peut-il pas rendre l'id_produit, et quelles écritures le peuvent ?
4. Un collègue joint les ventes aux tickets et trouve un chiffre d'affaires inférieur de 30,9 % au total du tableau. Donnez la cause, et le contrôle en une formule qui l'aurait montré avant le premier tableau de bord.
5. Dans objectifs, annee_mois vaut 2023-01. Que donne une régénération par `=annee&"-"&mois`, et combien de lignes sur 218 survivent ?

Question ouverte. Vous livrez un classeur qui doit tourner en Microsoft 365, en Excel 2019 et dans LibreOffice Calc, et être relu six mois plus tard par un inconnu. Décrivez votre politique de recherche : quelle fonction dans les cellules, laquelle en note, quels contrôles en tête de feuille, et à quoi vous renoncez.

Corrigé de l'évaluation formative.

1. Rien aujourd'hui : l'approximation sur une clé triée croissante retombe juste (0 divergence sur les 377 lignes à clé valide). Après un tri par nom, la même formule rend 233 réponses fausses sur 377 ; après un tri par ville, 377. Le symptôme est un résultat **qui bouge sans cause**, pas une erreur.

2. `=SI(TableVentes[@client]="";"non";RECHERCHEX([@client];clients[id_client];clients[type_client];"absent du fichier"))` — le contrôle du vide **avant** la recherche, le repli dedans, pas de **SIERREUR** autour.
3. Parce que `id_produit` est en colonne A et la désignation en B : `RECHERCHEV` ne renvoie que ce qui est **à droite** de sa colonne de clé, et un index 0 n'existe pas. Les deux issues : `=INDEX(produits!A:A;EQUIV(libellé;produits!B:B;0))` ou `=RECHERCHEX(libellé;produits!B:B;produits!A:A)`.
4. Le rapprochement s'est fait au grain du ticket alors que la donnée est au grain de la ligne : `RECHERCHEV` s'arrête au premier match et ne somme qu'une ligne sur cent quatre tickets multi-lignes. Contrôle : `=MAX(NB.SI(TableVentes[n_ticket];TableVentes[n_ticket]))` — s'il vaut plus de 1, la clé n'est pas une clé de ligne.
5. Elle devient `2023-1` pour les neuf premiers mois, donc injoignable : 45 lignes sur 218 se joignent (celles d'octobre à décembre), et la perte porte exactement sur les mois creux de l'année. Le remède est `TEXTE(mois;"00")`, et le contrôle est `=NBCAR(clé)` qui doit valoir 7 pour `AAAA-MM`.

Question ouverte — attendus. `INDEX/EQUIV` dans les cellules, `RECHERCHEX` en commentaire, contrôles d'unicité et de recouvrement en tête, et renonciation assumée à `DECALER`, `INDIRECT` et aux macros enregistrées.

Module M03.C07 — Dates, durées, périodes, puis les fonctions matricielles : `FIN.MOIS`, `NB.JOURS.OUVRES`, `FILTRE`, `TRIERPAR`, `LET`

Outil de ce chapitre : le tableur seul, sur le classeur de l'atelier. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. Une date n'est pas un texte qui ressemble à une date : c'est **un nombre de jours**, et toute l'analyse temporelle sort de là. Ce chapitre fait trois choses dans cet ordre : apprendre à lire les dates que le fichier contient (une colonne en contient trois sortes), construire les bornes de période sans jamais taper un jour au clavier, puis — parce que le calendrier une fois posé il faut le parcourir — les fonctions matricielles qui renvoient un tableau au lieu d'une cellule, avec la **règle de la fiche double** : chaque geste est écrit deux fois, une fois dans la voie moderne, une fois dans la voie que tout le monde peut relire.

Base de travail. Le classeur `01_socle_donnees/data/projection/m03_classeur_atelier.xlsx` : `ventes` (le tableau `TableVentes`, 480 lignes, dates typées), `ventes_brutes` (le fichier reçu, dates en texte), `Mensuel` (la grille de 12 mois que ce chapitre démonte pierre par pierre) et les fiches 24, 25 et 28 de `Calculs`. Séparateur point-virgule, UTF-8, franc CFA (FCFA), virgule décimale, TVA 18 %. Chiffres cités : `01_socle_donnees/data/reference/chiffres_cites.md`, section M03. Copie de travail : `calendrier_atelier`.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Distinguer** la date-série (un nombre), la date-texte et l'heure, et diagnostiquer une colonne qui mélange les trois — 424 dates en format international, 65 en `JJ/MM/AAAA` dans le fichier reçu, 36 groupes là où il y a 12 mois.
2. **Écrire** les bornes de période sans littéral : `DATE(2025;n;1)`, `FIN.MOIS`, `MOIS.DECALER`, et la borne haute exclusive qui ne double-compte pas le 1^{er} du mois suivant.
3. **Calculer** des durées : `DATEDIF` et ses six unités, `NB.JOURS.OUVRES` et ses jours fériés, `SE-RIE.JOUR.OUVRE` pour projeter une échéance.
4. **Nommer le dénominateur temporel** de tout ratio : 240 jours ouvrés ou 336 jours courus ne donnent pas le même chiffre, et aucun des deux n'est une moyenne de vente.
5. **Lire et écrire** `FILTRE`, `UNIQUE`, `TRIER`, `TRIERPAR`, `SEQUENCE`, `LET`, et prévoir `#PROPAGATION!`.
6. **Appliquer la fiche double** : une voie moderne, une voie Excel 2021 / LibreOffice, et savoir laquelle mettre dans le classeur livré.

2. Pourquoi cette notion est importante

Le trimestre dernier, un tableau de bord annonçait « 27,9 % de l'objectif atteint en décembre ». C'était faux deux fois : le pourcentage était celui de **quatre jours de vente** rapportés à un mois complet — l'extrait de l'atelier ne contient que les jours 1 à 4 de chaque mois, 256 lignes au 1^{er}, 163 au 2, 55 au 3, 6 au 4 — et le dénominateur, lui, était mensuel. Personne n'a vérifié l'échelle, parce que le nombre avait l'air raisonnable.

Trois raisons de prendre le calendrier au sérieux :

1. **Le temps est le seul axe que toutes les demandes contiennent.** « Par mois », « en glissant sur trois mois », « en jours ouvrés », « depuis le début de l'année » : quatre questions, quatre écritures, et une seule (la première) se code sans réflexion.
2. **Les dates sont le type le plus fréquemment raté à l'import.** Le module C02 l'a montré pour les montants ; pour les dates le défaut est pire, parce qu'une date-texte se trie alphabétiquement sans rien afficher, et se groupe en 36 classes là où il y a 12 mois.
3. **Les fonctions dynamiques ont changé ce qu'on peut exiger d'une feuille** — et un classeur qui n'utilise qu'elles est un classeur que la moitié de l'entreprise ne pourra pas rouvrir.

3. Explication simple

Dans un tableur, une date est un compteur de jours. 01/01/1900 porte le numéro 1, 02/01/1900 le numéro 2 ; une date de 2025 est donc un nombre de l'ordre de 45 000. Le format JJ/MM/AAAA n'est qu'un habillage : c'est ce qui rend =B2+1 possible (le lendemain), et ce qui rend =MOIS(B2) impossible si B2 contient le **texte** 01/01/2025. L'heure est la partie décimale du même nombre : 0,5 vaut midi.

Une période, c'est deux bornes. La règle qui évite tous les débats : **inclusif en bas, exclusif en haut**. Janvier 2025 = « à partir du 1^{er} janvier, avant le 1^{er} février ». On ne touche jamais au dernier jour du mois, donc on ne se demande plus s'il a 28, 29, 30 ou 31 jours.

Les fonctions matricielles dynamiques, elles, répondent à une autre limite : une formule ne peut-elle rendre qu'une cellule ? Non. =FILTRE(...) rend un tableau de 12 lignes qui se **propage** (l'anglais spill) dans les cellules d'à côté. Si la place est prise, Excel dit #PROPAGATION! — c'est la nouvelle erreur à connaître, et sa correction est toujours la même : libérer la zone.

Dans les faits. Dans un fichier d'atelier comme dans beaucoup de exports d'ERP, la colonne date contient deux écritures : le format international AAAA-MM-JJ pour la plupart des lignes, le format local JJ/MM/AAAA pour les autres — 424 et 65 ici. Le tableur ne devine pas : il prend ce qu'on lui donne, et une partie de la colonne reste du texte. Le premier réflexe du chapitre est donc un compteur, pas une formule de calcul.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Série de dates — date serial	Le nombre de jours depuis l'origine du tableur.	Comparer une série à du texte : la condition ne matche pas.
Système 1904 — 1904 date system	Origine alternative (Mac ancien) : même affichage, numéros décalés de 1 462 jours.	Ouvrir un fichier 1904 et recopier des nombres bruts.
Borne exclusive — exclusive bound	<DATE(2025;2;1) plutôt que <=FIN.MOIS(...).	Traiter février 2024 sans le 29 par un <=28.
Jour ouvré — business day	Lundi à vendredi, sauf jours fériés listés.	Oublier le troisième argument et appeler ça « jours travaillés ».
Glissement — rolling window	Moyenne sur les n dernières périodes.	Moyenne des trois premiers mois étiquetée « glissante » en janvier.
Propagation — spill	Le déploiement d'un résultat multi-cellules dans la grille.	Écrire une formule dynamique dans un tableau structuré : non pris en charge.

Français — English	Définition simple	Piège à éviter
#PROPAGATION! #SPILL!	— La zone d'arrivée est occupée.	Croire que la formule est fausse : c'est la place qui manque.
Plage propagée — spill range	La référence F2# désigne tout le bloc.	Recopier le résultat au lieu de le référencer.

Définition. Une **période ancrée** est une période définie par une formule qui la recalcule ($\text{=DATE(ANNEE(AUJOURDHUI());1;1)}$), par opposition à une période tapée à la main (>=01/01/2025). Une feuille vivante n'a aucune date littérale dans ses critères.

Définition. Une **fonction matricielle dynamique** est une fonction dont le résultat est un tableau de taille variable : **FILTRE**, **UNIQUE**, **TRIER**, **TRIERPAR**, **SEQUENCE**, **TABLEAU.ALEA**. Elle existe en Microsoft 365, Excel 2021 et 2024, et dans LibreOffice Calc depuis la version 24.8 — mais Calc ne propage pas : il faut pré-allouer la plage et valider en **Ctrl + Maj + Entrée**.

Définition. **LET(nom1;valeur1;...;calcul)** attache des noms intermédiaires dans une formule. Ce n'est pas du confort : une plage nommée **localement** n'est évaluée qu'une fois, donc la formule se relit sans se redémontrer, et le tableur ne repart pas la calculer trois fois.

Définition. Un **décalage d'échelle** est l'écart entre la période mesurée et la période dénominateur. Le ratio de l'atelier (18,2 % de l'objectif annuel) compare quatre jours par mois à douze mois : il mesure un extrait, pas une performance.

5. Cours approfondi

Jours du mois couverts

480 lignes, 4 valeurs de JOUR()

1er		256
2		163
3		55
4		6

=NB.SI(TableVentes[date];1)
rend 0 : le 1er est une date, pas le nombre 1

Jours de la semaine

=JOURSEM(date;2) : lundi = 1

lun		103
mar		76
mer		82
jeu		61
ven		35
sam		91
dim		32

123 lignes le samedi ou le dimanche :
25,6 % de l'extrait

Un total, deux rythmes

CA de l'extrait
36 073 185 FCFA

=NB.JOURS.OUVRES(min ; max)
240 jours ouvrés
150 305 FCFA par jour ouvré

=max-min+1 (jours cours)
336 jours calendaires
107 361 FCFA par jour couru

Les deux sont justes ;
aucun n'est une moyenne de vente.

La fiche double : la voie moderne, et la voie que tout le classeur peut lire

Les lignes de décembre

moderne =FILTRE(TableVentes!A2:M481;MOIS(TableVentes!B2:B481)=12)
accessible =SOMME.SI.ENS([montant_ttc];[date];">="&DATE(2025;12;1);[date];"<"&DATE(2026;1;1)) puis un filtre sur la date

Le chiffre d'affaires mois par mois

moderne =TRIERPAR(SEQUENCE(12);1;1;1) en ancre de la grille mensuelle
accessible la feuille Mensuel de l'atelier : douze fois =SOMME.SI.ENS avec DATE(2025;n;1) pour borne, et FIN.MOIS pour la seconde

Les catégories réellement présentes

moderne =UNIQUE(TableVentes[categorie])
accessible =JOINDRE.TEXTE(...;VRAI;...) de la fiche 23, ou une colonne triée puis dédoublée

FILTRE · UNIQUE · TRIERPAR · SEQUENCE · LET : Microsoft 365, Excel 2021 et 2024 (en 2019 : #NOM?).
LibreOffice 24.8+ connaît ces fonctions mais ne renverse pas : pré-allouer la plage, Ctrl + Maj + Entrée.

5.1 Lire ce que la colonne date contient vraiment

Avant toute formule de période, trois compteurs sur la colonne :

Contrôle	Formule	Attendu sur colonne	Attendu sur colonne_jour_mois
les dates typées	=NB(TableVentes[date])	480	0
les valeurs non typées	=NBVAL(ventes_brutes!B2:B490) - NB(ventes_brutes!B2:B490)	—	489
l'amplitude réelle	=MAX(date) - MIN(date)	335 jours	0 : rien à sommer
les bornes	=MIN(date) et =MAX(date)	01/01/2025 02/12/2025	idem, en texte

Le premier point qui surprend : =MAX(ventes_brutes!B2:B490) rend 0, pas #VALEUR! — sur une plage de texte, les agrégats numériques rendent 0 (le chapitre C05 l'a établi pour les montants, la leçon est identique ici). Le second

dans le fichier reçu, la date est **texte** pour 424 lignes au format international et pour 65 lignes au format JJ/MM/AAAA ; aucune n'est une série. Grouper dessus produit m03_groupes_texte_pour_12_mois = 36 classes au lieu de 12, parce que la chaîne 15/03/2025 et la chaîne 2025-03-15 ne sont pas le même texte.

Le nettoyage, en une formule par ligne : =DATEVAL(SUPPRESPEACE(B2)) fait le travail quand le texte est dans **le format reconnu par le système**, et c'est le point sensible : DATEVAL lit 2025-03-15 partout, mais 15/03/2025 seulement si vos paramètres régionaux placent le jour en premier. La voie robuste passe par le découpage, comme au chapitre précédent : =DATE(CNUM(STXT(B2;7;4));CNUM(STXT(B2;4;2));CNUM(STXT(B2;1;2))) pour un JJ/MM/AAAA, et les mêmes positions décalées pour l'écriture internationale. Écrivez les deux, comparez les sommes : si les deux rendent la même date, la colonne est saine.

À retenir. =MOIS(B2) sur une date texte ne rend pas une erreur de format, il rend #VALEUR! — et =NB.SI(colonne;Mois5) ne rend rien du tout. Le seul contrôle qui ferme la question est =NB(colonne) : 480 si la colonne est typée, 0 si elle ne l'est pas.

5.2 Extraire le temps : la famille qui lit dans la série

Fonction	Syntaxe FR	Ce qu'elle rend sur le 15/03/2025
ANNEE / MOIS / JOUR	=MOIS(B2)	2025 · 3 · 15
JOURSEM (WEEKDAY)	=JOURSEM(B2;2)	6 — le 15/03/2025 est un samedi, lundi = 1
NO.SEMAIN (WEEKNUM)	=NO.SEMAIN(B2;2)	11 — semaine de l'année, semaines luso-isolées
NO.SEMAIN.ISO	=NO.SEMAIN.ISO(B2)	11 — la norme européenne, à préférer en reporting
HEURE / MINUTE	=HEURE(C2)	sur la colonne heure de l'atelier
TEXTE	=TEXTE(B2;"aaaam-mmm")	« 2025-mar » — l'étiquette de regroupement

Les types de JOURSEM et NO.SEMAIN méritent la ligne de note dans la fiche : sans deuxième argument, JOURSEM compte **dimanche = 1**, ce qui place le dimanche dans le premier jour de la semaine et le week-end de l'entreprise à cheval sur deux bornes. Avec 2, lundi = 1 et le test du week-end devient =JOURSEM(B2;2)>5. Sur l'extrait, ce test attrape 91 lignes du samedi et 32 du dimanche, soit 123 lignes sur 480 — 25,6 % du fichier, mais 9 730 140 FCFA sur 36 073 185, soit 27,0 % du chiffre d'affaires. C'est une information de calendrier, pas une anomalie : mais elle interdit d'appeler « CA par jour travaillé » un chiffre calculé sur les seuls jours ouvrés, comme on le verra en §5.4.

Le regroupement mensuel propre ne passe **pas** par MOIS seule : =MOIS(B2) rend 3 pour mars 2025 comme pour mars 2024, et une colonne qui perd l'année est une colonne qui agrège deux années sans prévenir. Trois écritures, dans l'ordre de préférence :

1. =FIN.MOIS(B2;-1)+1 — **le premier jour du mois** de la ligne, en nombre, donc triable, comparables, joignable ;
2. =DATE(ANNEE(B2);MOIS(B2);1) — le même résultat, plus lisible pour un non-initié ;
3. =TEXTE(B2;"AAAA-MM") — l'étiquette pour le titre de colonne, jamais pour la jointure (c'est du texte).

La première a ma préférence parce qu'elle survit aux années : FIN.MOIS connaît les mois de 28, 29, 30 et 31 jours, DATE(...;MOIS(...)+1;1) aussi, mais =B2-JOUR(B2)+1 (l'écriture « arithmétique » qu'on voit partout) casse sur février bissextile et mérite d'être bannie d'un manuel.

5.3 Bornes, durées, DATEDIF

Les trois fonctions de bornes, avec leur usage dans le classeur :

- FIN.MOIS(dates;mois) renvoie le dernier jour du mois décalé de mois — le second argument **accepte le nombre de mois, pas de jours**, ce qui le rend supérieur à tout ajout arithmétique : =FIN.MOIS(A2;0) est la fin du mois courant, =FIN.MOIS(A2;1) celle du suivant. La feuille Mensuel s'en sert comme borne haute.
- MOIS.DECALER(dates;mois) (EDATE) décale d'un même jour de mois en mois, en écrasant les jours inexistants : =MOIS.DECALER(DATE(2025;1;31);1) rend le 28 février 2025, pas le 3 mars — comportement juste pour une échéance, piégeux pour un anniversaire.
- DATEDIF(début;fin;unité) calcule un âge. Les six unités : Y années complètes, M mois complets, D jours, MD jours restants après le dernier mois complet, YM mois restants après la dernière année, YD jours restants hors années complètes. Sur l'extrait, =DATEDIF(MIN(TableVentes[date]);MAX(TableVentes[date]);"m") rend 11 mois et l'amplitude est de 335 jours — deux réponses également justes à « quelle durée couvre le fichier ? », et c'est la raison pour laquelle on cite l'unité dans le commentaire.

Une bizarrerie assumée : DATE normalise les arguments hors plage. =DATE(2025;13;1) est le 1^{er} janvier **2026**, =DATE(2025;1;32) le 1^{er} février, =DATE(2025;n;0) le dernier jour du mois précédent. La feuille Mensuel en profite avec élégance : sa borne haute s'écrit DATE(2025;\$A2+1;1) pour le mois 12 aussi, sans test d'année suivante. Retenez l'inverse aussi : pour une année, écrivez DATE(2025;1;0) et non DATE(2024;12;31) — le premier dit « fin de 2024 » sans connaître le calendrier.

Attention aux deux chiffres : la règle documentée de DATE est que si l'année est comprise entre 0 et 1899, Excel ajoute cette valeur à 1900 — =DATE(108;1;2) renvoie le 2 janvier 2008. Un =DATE(GAUCHE(A2;2);...) qui lit « 25 » dans un ticket tombe donc en 1925, silencieusement. Écrivez quatre chiffres, toujours.

5.4 NB.JOURS.OUVRES : le dénominateur du temps travaillé

=NB.JOURS.OUVRES(début;fin;[fériés]) compte les lundis à vendredis de l'intervalle, **bornes incluses**. Fiche 24 du classeur : =NB.JOURS.OUVRES(DATE(2025;1;1);DATE(2025;12;2)) → 240. Trois choses à savoir :

1. Sans troisième argument, **aucun jour férié n'est retiré**. Le chiffre s'appelle alors « jours de semaine », pas « jours travaillés ». Dans l'atelier, c'est exact — il n'y a pas de table de fériés — mais un fichier de l'entreprise en porte une, et l'oublier gonfle le dénominateur d'une quinzaine de jours par an, soit environ 6 % de sur- ou sous-évaluation du rythme quotidien selon le sens du ratio.

- Le troisième argument accepte une plage de dates **ou une plage nommée** ; mettez la liste des fériés dans une feuille `Calendrier` et nommez-la `Jours_fériés`, sinon chaque année il faudra réécrire les douze formules.
- `SERIE.JOUR.OUVRE(date;n;[fériés])` est l'inverse : il projette l'échéance `n` jours ouvrés plus loin, et c'est ce qu'on attend pour une date de livraison, pas `=date+n`.

À quelle sauce le dénominateur change le récit ? Le chiffre d'affaires de l'extrait est de 36 073 185 FCFA :

Dénominateur	Nombre	CA par unité
lignes de vente	480	75 152 FCFA
tickets	316	114 156 FCFA
jours ouvrés (lun→ven, 01/01→02/12)	240	150 305 FCFA
jours calendaires courus	336	107 361 FCFA
mois couverts	12	3 006 099 FCFA

Quatre nombres justes. Un rapport qui en cite un sans dire lequel est invérifiable — et le quatrième, celui qui semble le plus « naturel », est le seul qui ignore que 123 lignes sont tombées un samedi ou un dimanche. `AUJOURDHUI()` et `MAINTENANT()` complètent le jeu : la première ne rend **rien d'autre** qu'un numéro de jour courant (et change à chaque ouverture, donc ne la mettez jamais dans une cellule de contrôle attendue), la seconde porte la partie décimale de l'heure.

Attention. Un ratio temporel recalculé avec `AUJOURDHUI()` dans le dénominateur change de valeur tout seul entre la livraison et la réunion. Deux règles : les feuilles de contrôle figent la borne (`=DATE(2025;12;31)`) et la date d'arrêté est écrite dans une cellule nommée, consultée par les formules, jamais recopiée dans chacune.

5.5 La grille mensuelle de l'atelier, démontée

La feuille `Mensuel` est un modèle du genre, et elle tient en six formules répétées douze fois. Colonne A : le numéro de mois (1 à 12). Les autres :

```
B2 =SOMME.SI.ENS(ventes!M2:M481;ventes!B2:B481;">="&DATE(2025;$A2;1);ventes!
B2:B481;"<"&DATE(2025;$A2+1;1))
C2 =NB.SI.ENS(ventes!B2:B481;">="&DATE(2025;$A2;1);ventes!B2:B481;"<"&DATE(2025;$A2+1;1))
D2 =SOMME.SI.ENS(objectifs!E2:E219;objectifs!A2:A219;5;objectifs!B2:B219;2025;objectifs!
C2:C219;$A2)
E2 =SIERREUR(B2/D2;"objectif absent")
F2 =NB.JOURS.OUVRES(DATE(2025;$A2;1);FIN.MOIS(DATE(2025;$A2;1);0))
```

Quatre décisions de style valent le détour, et ce sont les vôtres à partir de maintenant :

- la borne haute est exclusive** (`"<"&DATE(2025;$A2+1;1)`) : aucun 31/01 oublié, aucun 01/02 compté deux fois, et le mois 12 ne demande pas de trait d'esprit puisque `DATE(2025;13;1)` est janvier 2026 ;
- le critère est construit, jamais tapé** : la concaténation `">="&DATE(...)` produit le nombre au bon format, alors que le littéral `">=01/01/2025"` dépend des réglages régionaux du poste ;
- SIERREUR encadre le ratio, pas la somme** : une division par un objectif absent donne « objectif absent », tandis qu'un CA manquant doit rester visible ;
- les jours ouvrés sont dans la grille** (colonne F, 20 à 23 selon les mois, 240 au total sur la période de l'extrait) : le ratio `B/F` est calculable en un glisser, et c'est le seul chiffre de la feuille qui se compare d'un mois à l'autre sans biais de longueur.

5.6 Lisser : la moyenne glissante, et ce qu'elle cache

Sur les douze CA mensuels (1 258 801 FCFA au plus bas en mai, 5 292 517 au plus haut en décembre — un rapport de 4,2 entre les deux, ce qui rend la lecture ligne à ligne impossible), la moyenne glissante sur trois mois s'écrit en C4 de la copie de la grille, étirée vers le bas :

mois	CA mensuel	lignes	ouvrés	CA par jour ouvré	glissant 3 mois
1	2 051 424	40	23	89 192	—
5	1 258 801	40	22	57 218	2 540 138
10	2 808 078	40	23	122 090	3 106 116
12	5 292 517	40	23	230 109	3 627 210

Trois remarques qui valent plus que la formule. Un : les deux premiers mois n'ont pas de glissante — `=MOYENNE(B2:B4)` en C2 avale les cases vides et rend une moyenne sur un ou deux mois étiquetée « trois mois » ; le garde-fou est `=SI(NB(B2:B4)=3;MOYENNE(B2:B4);"")`. Deux : la glissante **ne corrige pas l'échelle** — ici elle lisserait douze mois construits sur quatre jours chacun, ce qui est une régularité d'échantillon, pas de saison ; le commentaire à écrire dans la feuille est celui du classeur lui-même : l'extrait ne couvre que les jours 1 à 4 de chaque mois, donc aucun ratio mensuel de ce fichier ne mesure une performance annuelle. Trois : sur un vrai fichier, on lisse en **jours ouvrés** et non en lignes, sinon un mois de 23 jours ouvrés semble battre un mois de 20 jours de 15 % sans avoir rien vendu de plus.

5.7 Les fonctions dynamiques, en fiches doubles

Cinq fonctions, chacune avec son écriture accessible. La règle du classeur : la voie du milieu est **dans les cellules**, la voie moderne **en commentaire de note**, sauf dans un classeur dont vous contrôlez la distribution.

Tâche	voie moderne (365 · 2021 · 2024 · LibreOffice 24.8+)	voie accessible (tous, y compris 2019)
les lignes d'un mois	<code>=FILTRE(TableVentes!A2:M481;MOIS(TableVentes!B2:B481)=12;"aucune ligne")</code>	un filtre sur la colonne mois de la \$5.2, ou <code>=SOMME.SI.ENS</code> si seul le total intéresse
les valeurs présentes	<code>=UNIQUE(TableVentes[categorie])</code>	<code>=JOINDRE.TEXTE(", ";VRAI;...)</code> de la fiche 23, ou une colonne triée-puis-dédupliée
le top 5 par CA	<code>=TRIER(FILTRE(...);3;-1;VRAI)</code>	<code>=GRANDE.VALEUR</code> x5 sur la grille par catégorie, ou le tri d'un TCD (chapitre C08)
un calendrier de 12 lignes	<code>=DATE(2025;SEQUENCE(12);1)</code>	douze cellules recopiées, <code>=A2+1</code> interdit : <code>=DATE(2025;MOIS(A2)+1;1)</code>
trier par une autre colonne	<code>=TRIERPAR(plage;TableVentes[montant_ttc];-1)</code>	la colonne utilitaire rang + un tri manuel, consigné
éviter de tout répéter	<code>=LET(ca;SOMME.SI.ENS(...);nb;NB.SI.ENS(...);ca/nb)</code>	une ligne d'aide dans la grille, et la division en colonne séparée

Deux contraintes d'environnement, toutes deux documentées par l'aide de Microsoft, à connaître avant d'écrire : une formule à propagation **ne peut pas vivre dans un tableau structuré** — chez vous, `TableVentes` est un tableau : écrivez `FILTRE/UNIQUE` dans la grille libre à côté, jamais dans une colonne du tableau ; et si la zone d'arrivée est occupée — y compris par des cellules fusionnées ou une mise en forme résiduelle — Excel rend `#PROPAGATION!`. La correction n'est pas une réécriture : c'est un ménage. Enfin, la référence `=F2#` (le dièse de plage propagée) est le moyen propre de chaîner un `FILTRE` vers un `=SOMME` ; elle n'existe ni en 2019 ni dans Calc, ce qui est la vraie raison d'écrire la voie du milieu en regard.

Attention. Ces deux contraintes vont dans la fiche d'en-tête du classeur, pas dans la légende du graphique : les dynamiques exigent Microsoft 365, Excel 2021 ou 2024 (en 2019 : #NOM? cellule par cellule), et LibreOffice les connaît depuis 24.8 sans les propager — plage pré-allouée, validation en Ctrl + Maj + Entrée.

LET mérite un mot, parce qu'on le sous-estime : dans la voie « ca/nb », il **nomme** les deux agrégats, et un dénominateur nommé se relit et se discute — sans même parler des recalculs évités.

Conseil professionnel. Écrivez la voie moderne dans une feuille de bac à sable, vérifiez qu'elle rend bien le même nombre que la voie accessible sur les douze mois, puis livrez l'accessible et gardez la moderne en note. Le gain de temps est à vous, le risque de #NOM? ne l'est pas.

5.8 Cinq fiches de fonction du chapitre

Format du module : nom et version · syntaxe · arguments · exemple sur le jeu de l'atelier · cas réel · erreur fréquente et symptôme. Version abrégée ici, la fiche complète étant à rédiger dans votre classeur.

Fonction	Version	Exemple sur l'atelier	Erreur fréquente, symptôme
FIN.MOIS (EOMONTH)	toutes	=FIN.MOIS(DATE(2025;2;1);0) 28/02/2025	→ le résultat est un nombre : sans format date, la cellule affiche 45716
MOIS.DECALER (EDATE)	toutes	=MOIS.DECALER(A2;3)	confondre avec +3 : l'un décale de 3 mois, l'autre de 3 jours
DATEDIF	toutes (non listée dans l'assistant)	=DATEDIF(MIN(date);MAX(date);"m") → 11	unité M vs nombre de mois civils : 11 contre 12, selon les bornes
NB.JOURS.OUVRES (NETWORKDAYS)	2010+	fiche 24 → 240	troisième argument oublié : les fériés comptent comme travaillés
FILTRE	365 · 2021 · 2024 · LO 24.8+	40 lignes de décembre en une formule	écrit dans le tableau structuré, ou zone occupée → #PROPAGATION!

6. Exemple concret : le décembre qui écrasait les autres

Décembre affiche 5 292 517 FCFA, le plus haut des douze mois, contre 1 258 801 en mai. Le commentaire du tableau de bord, écrit par quelqu'un qui n'avait pas lu ce chapitre : « +320 % en décembre, effet fêtes ». Deux vérifications démontent la phrase.

La première est de longueur : les jours ouvrés vont de 20 à 23 selon le mois, décembre en compte 23, mai 22. En CA par jour ouvré, décembre passe de 230 109 à comparer à 57 218 pour mai — l'écart reste massif (×4), mais il n'est plus de 320 % : il est de 302 %. L'effet de longueur était réel et minime ; la mention du dénominateur, elle, est passée d'aucune à obligatoire.

La seconde est d'échelle : dans ce fichier, un mois, c'est quatre jours. Le ratio au objectif de décembre (27,9 % contre 14,1 % en octobre) ne dit pas que le magasin a fait 28 % de son mois en une semaine — il dit que l'échantillon de quatre jours de décembre pèse ce poids-là. La note du classeur Mensuel le dit elle-même, et c'est la seule fois du manuel où je vous demande de recopier une phrase d'un fichier : elle est courte, et elle est exacte. Le vrai écart se lit à l'échelle de la population entière, où le rapport CA/objectif s'étage de 1,62 à 3,27 selon le magasin-mois.

Ce qu’il fallait produire, et ce que produit la §7 : une colonne de jours ouvrés, un ratio sur ce dénominateur, et une ligne d’échelle (« extrait : jours 1 à 4 de chaque mois ») sous le titre du graphique.

7. Démonstration pas à pas : la grille de rythme mensuel

Objectif. Une feuille Rythme qui, pour les douze mois, rend CA, lignes, jours ouvrés, CA par jour ouvré, et la glissante à trois mois — dans la voie accessible, avec la voie moderne en vis-à-vis.

Étape	Formule	Ce se
1	<code>=DATE(2025;SEQUENCE(12);1)</code> en A2 (hors tableau, à gauche de la zone libre)	12 du janv déc 202
2	<code>=DATE(ANNEE(\$A2);MOIS(\$A2);1)</code> en A2, copiée	mê col voie la con des le c
3	<code>=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[date];">="&\$A2;TableVentes[date];"<"&FIN.MOIS(\$A2;0)+1)</code>	dou don 424 jan son 073 FCF
4	<code>=NB.JOURS.OUVRES(\$A2;FIN.MOIS(\$A2;0))</code>	20 tota l'an plei
5	<code>=SI(B2>0;B2/D2;"")</code>	un par auc 0!
6	<code>=SI(NB(B2:B4)=3;MOYENNE(B2:B4);"")</code>	deu cell vide glis

Contrôles qualité du chapitre. (1) La colonne A est une série de dates, pas du texte : `=NB(Rythme!A2:A13)` → 12. (2) L’étape 3 ferme sur le total du tableau, ni plus ni moins : 36 073 185 FCFA. (3) Le total des jours ouvrés de la colonne D vaut 261 pour l’année, et 240 pour la période réellement couverte par l’extrait (fiche 24) — les deux nombres sont justes, ils ne répondent pas à la même question, et c’est le commentaire qui tranche. (4) Les douze CA mensuels recopiés à côté de `m03_ca_par_mois` se religne-à-religne à l’unité près.

Barème de la démonstration (10 points). Colonnes A doublées voie moderne/accessible et concordantes (2) · bornes exclusives aux douze lignes (2) · jours ouvrés et rythme calculés (3) · glissante gardée-foutie (1) · note d’échelle « quatre jours par mois » écrite sous le titre (2).

8. Erreurs fréquentes

1. **Compter sur `=NB(colonne date)` sans l'avoir fait.** Une date texte n'est pas une date : 489 lignes en texte rendent 0, et `=MOIS(...)` rend `#VALEUR!` sur la moitié de la colonne sans que le tri n'avertisse.
2. **Comparer à un littéral de date.** `">=01/01/2025"` dépend du poste, et `=B2="01/01/2025"` est une comparaison de texte : la même donnée, deux réponses selon la machine. Construire, toujours : `">="&DATE(2025;1;1)`.
3. **Oublier le 29 février.** Un `<=FIN.MOIS()` écrit à la main sur février rend 2024 orphelin de son dernier jour ; `MOIS.DECALER` et la borne exclusive ne connaissent pas ce problème.
4. **Appeler « jours travaillés » un `NB.JOURS.OUVRES` sans fériés.** Le troisième argument n'est pas optionnel en entreprise, il est optionnel en syntaxe.
5. **Retirer le week-end du dénominateur en gardant les ventes du week-end.** 123 lignes sur 480 sont tombées un samedi ou un dimanche : diviser le CA total par les seuls jours ouvrés produit un rythme surévalué d'autant.
6. **Moyenne glissante sans garde-fou.** `=MOYENNE(B2:B4)` en haut de la colonne rend un chiffre sur deux points : `SI(NB(...)=3;...)` est la ligne qui manque.
7. **Écrire une formule dynamique dans le tableau structuré.** Elle ne se propage pas, et le message est `#PROPAGATION!` là où l'on cherchait une erreur de formule.
8. **Livrer un classeur 365 à une salle en 2019.** `FILTRE`, `UNIQUE`, `TRIERPAR` rendent `#NOM?` cellule par cellule : la fiche double n'est pas une politesse, c'est une assurance.
9. **Figurer une période avec `AUJOURDHUI()` dans une cellule de contrôle.** Le contrôle change d'attendu tous les matins ; la borne d'arrêt se nomme, elle ne se devine pas.

9. Bonnes pratiques professionnelles

1. **Une feuille `Calendrier`** avec une ligne par jour de la période analysée : date, mois, mois-étiquette, numéro de semaine ISO, jour ouvré, jour férié. C'est la table qui rend toutes les autres simples, et elle s'écrit en une `SEQUENCE` en trente secondes.
2. **Jamais de date littérale dans un critère.** Une cellule `!Calendrier!B2` nommée `Debut_période`, et les formules qui la pointent.
3. **Toujours le dénominateur temporel écrit à côté du ratio** : « CA par jour ouvré », pas « CA moyen ».
4. **Glissante et brut côte à côte**, jamais l'une pour masquer l'autre ; le graphique montre les deux courbes.
5. **La voie accessible dans les cellules livrées**, la voie moderne en note et dans votre feuille de bac à sable.
6. **Un contrôle de fermeture temporelle** : la somme des CA mensuels égale le CA du tableau, et le total des lignes par mois égale 480. En dessous de cette égalité, aucune analyse de saisonnalité n'est présentable.
7. **Format date explicite** partout où une formule a arithmétiquement touché une date : `FIN.MOIS` rend un numéro, et un numéro de série dans un commentaire de graphique fait fuir un comité.

10. Exercice guidé — la grille de rythme (40 min, /10)

Commande. Construisez `Rythme` en partant de zéro, dans les deux voies, et joignez-y le ratio à l'objectif.

Étape	Geste	Ce qui doit se lire
1	12 premiers de mois, SEQUENCE puis voie 2019	douze premiers de mois identiques, à un jour près : zéro écart
2	CA mensuel par SOMME.SI.ENS à bornes \$A2 / FIN.MOIS(\$A2;0)+1	douze nombres, total 36 073 185 FCFA
3	Jours ouvrés par mois	20 à 23, total annuel 261
4	CA par jour ouvré, et son ratio à l'objectif (objectifs sur magasin × mois)	douze ratios ; décembre en tête
5	Glissante 3 mois avec garde-fou	deux vides en tête, dix valeurs
6	FILTRE des lignes de décembre hors tableau, et son =SOMME chaîné par #	40 lignes, même total qu'à l'étape 2

Démarrage. À l'étape 1, ne recopiez pas les dates : la colonne doit venir d'une formule, sinon le jour où le périmètre change en janvier 2026 vous aurez douze lignes à retaper et une à oublier. À l'étape 4, le ratio à l'objectif se fait par SOMME.SI.ENS sur objectifs (magasin 5, année 2025, mois = MOIS(\$A2)) et non par RECHERCHEX sur une clé texte : la clé de la table d'objectifs est annee_mois, fabriquée au chapitre C06, et MOIS(\$A2) rend un nombre — si vous préférez joindre sur la clé, fabriquez-la des deux côtés avec TEXTE, ou la jointure ne matchera pas : MOIS(\$A2) est un nombre, la clé est une chaîne. À l'étape 6, déplacez le point d'insertion à **droite du tableau** : dans une colonne de TableVentes, la propagation est refusée, et vous perdrez dix minutes à croire que FILTRE est cassé.

Barème (10 points). Deux voies concordantes démontrées (3) · bornes exclusives et critères construits (2) · jours ouvrés et ratio justes (2) · glissante avec garde-fou (1) · FILTRE hors tableau et total chaîné identique (1) · note d'échelle de l'extrait (1).

11. Exercices autonomes

Exercice 7.1 (★) — Le recensement de colonne. Sur ventes_brutes puis sur ventes, comptez les dates typées, les dates en texte, les deux formats textuels distincts, et l'amplitude rendue par MAX-MIN dans chaque cas. Concluez : que vaut =MAX(ventes_brutes!B2:B490) et pourquoi ce n'est pas une erreur.

Exercice 7.2 (★) — Le week-end dans le dénominateur. Calculez le CA par jour ouvré (a) en divisant le CA total par les 240 jours ouvrés, (b) en retirant d'abord les 123 lignes de week-end puis en divisant par 240. Donnez les deux nombres, l'écart en pourcentage, et la phrase qui dit lequel des deux répond à « combien vend-on par jour de travail », compte tenu du fait que ces 123 lignes ne sont pas marginales.

Exercice 7.3 (★★) — Le calendrier en une formule. Construisez une feuille Calendrier couvrant l'année 2025 avec SEQUENCE (voie 2019 : recopie incrémentée), puis ajoutez mois, mois-étiquette, semaine ISO, jour ouvré et un drapeau « ouvré et dans l'extrait ». Contrôlez : le compte des ouvrés doit se raccorder aux 240 de la fiche 24, et le total des lignes de l'extrait doit retomber sur 480 en croisant NB.SI.ENS.

Exercice 7.4 (★★) — La glissante honnête. Reprenez la grille mensuelle, calculez la glissante à trois mois en (a) ignorant les mois incomplets, (b) en moyenne mobile **par jour ouvré**. Les deux séries ne racontent pas la même histoire : dites laquelle conserve l'ordre des mois, et ce que l'autre invente.

Exercice 7.5 (★★★) — DATEDIF aux six unités. Sur les bornes de l'extrait, calculez Y, M, D, YM, MD, YD, puis le nombre de mois civils couverts (=MOIS(MAX)-MOIS(MIN)+12*(ANNEE(MAX)-ANNEE(MIN))+1). Expliquez pourquoi deux de ces six nombres sont également justes et inutilisables sans la mention de l'unité, et lequel correspond au « 335 jours » de la fiche 25.

Exercice 7.6 (★★★) — La double fiche. Pour chacune des six tâches du tableau de §5.7, écrivez les deux voies, vérifiez l'égalité des résultats, et consignez la version qui entre dans le classeur livré avec une ligne de justification. Vous devez obtenir six égalités : si l'une manque, cherchez du côté de `FIN.MOIS` (nombre vs date) et de la borne exclusive.

12. Correction détaillée

Exercice 7.1. `ventes` : 480 typées, 0 texte, amplitude 335 jours (nombre). `ventes_brutes` : 0 typée, 489 en texte, dont 424 en `AAAA-MM-JJ` et 65 en `JJ/MM/AAAA`, et `MAX` rend **0** parce que le maximum d'une plage sans nombre est 0 en l'absence d'erreur — c'est le comportement établi au chapitre C05 pour les montants, et il est ici plus coûteux : une date à 0 s'affiche 00/01/1900, un format date caché, et l'amplitude devient une donnée fausse au lieu d'un zéro visible.

Exercice 7.2. (a) $36\,073\,185 \div 240 = 150\,305$ FCFA par jour ouvré, week-ends compris dans le numérateur. (b) le week-end pèse $9\,730\,140$ FCFA ; le numérateur tombe à $26\,343\,045$, le rythme à $109\,763$ — un écart de 27,0 %, pas une nuance. Réponse attendue : « combien vend-on par jour de travail » se répond avec (b) à **condition de retirer aussi les 104 samedis et dimanches de 2025 du dénominateur** (261 ouvrés sur l'année, mais le week-end n'en est pas un ici) ; sinon on compare un numérateur réduit à un dénominateur inchangé. La discipline est d'énoncer les deux dans la légende et de n'en choisir qu'un, une fois pour toutes, à l'échelle de l'entreprise.

Exercice 7.3. `=DATE(2025;1;1)+SEQUENCE(366)-1` (ou `=DATE(2024;12;31)+SEQUENCE(366)` si vous voulez que le 29 février 2024 soit exclu — 2025 ne l'est pas, il a 365 jours : le test utile est donc 365 lignes, pas 366, et vérifiez le compte avant d'écrire la formule). Les contrôles qui doivent tomber : ouvrés = 261 sur l'année, 240 entre le 01/01 et le 02/12 (fiche 24) ; croisement avec l'extrait par `=NB.SI.ENS(TableVentes[date];">="&A2;TableVentes[date];"<"&A2+1)` somme sur la colonne = 480 lignes réparties sur 31 jours distincts.

Exercice 7.4. (a) garde-fou `SI(NB(...)=3;...)` : deux cellules vides, dix valeurs comprises entre $1\,992\,788$ et $3\,627\,210$ FCFA. (b) la glissante sur le rythme par jour ouvré déplace les pics — un mois court bien rempli passe devant un mois long à rythme égal. Réponse attendue : la série (a) conserve l'ordre d'importance des mois mais confond longueur et performance ; la série (b) compare ce qui est comparable mais **inventerait** une saisonnalité si on l'appliquait à notre extrait, où chaque mois ne compte que quatre jours. D'où la règle : on ne lisse pas pour faire joli, on lisse après avoir égalisé la longueur.

Exercice 7.5. `Y` = 0, `M` = 11, `D` = 335, `YM` = 11, `MD` = 1, `YD` = 335, mois civils = 12. Deux d'entre eux répondent à « quelle durée ? » selon qu'on compte en mois **complets** (11) ou en mois **touchés** (12) — c'est l'écart entre `DATEDIF` et le comptage par `MOIS`, et le seul remède est d'écrire l'unité dans le commentaire. Le nombre qui correspond à la fiche 25 est `D`, 335.

Exercice 7.6. Les six égalités attendues : 12 mois de CA, 7 catégories listées, top 5 par CA, calendrier de 12 lignes, tri par montant, et CA moyen d'un mois. Les deux rupteurs classiques : `FIN.MOIS` sans format date (un nombre de 45 000 dans une comparaison de texte), et la borne `<=FIN.MOIS()` qui ampute les mois de 31 jours — l'atelier écrit `<FIN.MOIS()+1`.

13. Mini-projet M03.P1 — suite : le volet calendrier (45 min)

Commande. Ajoutez à `calendrier_atelier` la feuille `Calendrier` et la feuille `Rythme`, et faites-les travailler ensemble dans le tableau de bord du projet.

Livrables numérotés. (1) **Calendrier** : une ligne par jour de l'année couverte, avec date, mois, mois-étiquette, semaine ISO, ouvré, férié (vide assumé, documenté) et le drapeau « dans l'extrait » ; (2) **Rythme** : les douze mois en CA, lignes, ouvrés, CA par jour ouvré, glissante à trois mois, ratio à l'objectif, chaque colonne portant sa voie accessible et sa voie moderne avec la cellule de contrôle d'égalité ; (3) le bloc d'échelle : trois lignes qui disent ce que l'extrait couvre (jours 1 à 4), ce que le ratio mesure, et ce qu'il ne mesure pas ; (4) un **FILTRE** de démonstration hors tableau, chaîné par # vers un total, avec sa variante **SOMME.SI.ENS** en vis-à-vis ; (5) la fiche de fonction de **NB.JOURS.OUVRES** au format du module (six rubriques).

Barème (20 points, seuil 13). **Calendrier** juste et contrôlée (5) · **Rythme** en double voie avec égalités vérifiées (6) · bloc d'échelle qui se lit sans ouvrir une formule (4) · **FILTRE** proprement chaîné (3) · fiche de fonction complète (2).

14. Résumé du chapitre

Une date est un nombre de jours : tout ce qui suit en découle — comparer, borner, découper, moyenne. Trois gestes ferment la question du typage (**=NB(colonne)**, les deux formats textuels repérés par **NBCAR**, l'amplitude), un seul ferme celle de la période : borne basse inclusive, borne haute exclusive, construites avec **DATE/FIN.MOIS**, jamais tapées. Les durées se disent avec leur unité (**DATEDIF** rend 11 mois là où l'amplitude rend 335 jours, et les deux sont vrais), et le rythme se nomme : 75 152 FCFA par ligne, 114 156 par ticket, 150 305 par jour ouvré, 107 361 par jour couru — un chiffre sans dénominateur temporel est une opinion.

Les fonctions dynamiques changent la fabrique de la grille, pas la nature de la donnée : **FILTRE**, **UNIQUE**, **TRIER**, **TRIERPAR**, **SEQUENCE**, **LET** sont en 365, 2021 et 2024, connues de LibreOffice 24.8 et plus sans propagation, absentes de 2019, refusées dans un tableau structuré, et punies de **#PROPAGATION!** dès que la zone d'arrivée est prise. La fiche double — accessible en cellules, moderne en note — est la seule politique qui rend un classeur lisible par toute la salle.

À retenir. Deux chiffres de rythme sur le même fichier : 150 305 FCFA par jour ouvré, 107 361 par jour couru. Aucun n'est faux, les deux sont incomplets tant que la légende ne dit pas si les 123 lignes de week-end sont dans le numérateur.

À retenir. **=DATE(2025;\$A2+1;1)** est la borne haute du mois 12 sans cas particulier : **DATE** normalise l'année suivante. C'est la différence entre une formule qui tient douze mois et une formule qui tient douze ans.

15. À retenir

1. **Trois compteurs avant toute analyse de période** : **=NB(colonne date)**, **=MAX-MIN**, **=NB.SI.ENS** du nombre de lignes par mois. S'ils ne tombent pas d'accord, ne calculez rien.
2. **Bornes construites, jamais tapées** : **">="&DATE(2025;n;1)** et **"<="&FIN.MOIS(...;0)+1**. Le littéral dépend du poste, la formule dépend du calendrier — seule la seconde est la même partout.
3. **Une période doit dire son échelle.** Un extrait de quatre jours par mois rapporté à un objectif annuel est un taux d'échantillon, pas un taux de réalisation : écrivez-le sous le graphique, pas dans votre tête.

16. Évaluation formative (auto-correction, 10 min)

1. Une colonne date contient 489 cellules, `=NB(colonne)` rend 0, `=MAX(colonne)` rend 0. Que concluez-vous, et que faites-vous en premier ?
2. Écrivez les bornes de mars 2024 d'un `SOMME.SI.ENS` qui compte le 29 février de l'année d'avant et n'oublie aucun 31 — sans taper un seul jour en dur.
3. Pourquoi `=NB.JOURS.OUVRES(début;fin)` seul est-il un mauvais dénominateur pour un commerce ouvert le samedi, et que faut-il écrire à côté du chiffre ?
4. Un classeur livré contient `=FILTRE(...)` dans une colonne de `TableVentes`. Il rend `#PROPAGATION!` chez vous et `#NOM?` chez un collègue en 2019. Expliquez les deux messages, et corrigez les deux.
5. Le commentaire dit « glissante à trois mois » alors que la formule est `=MOYENNE(B2:B4)` copiée depuis la première ligne. Quel est le défaut, et quel est le garde-fou en une formule ?

Question ouverte. Vous devez livrer une revue annuelle qui compare douze magasins sur des périodes de longueurs différentes (certains ouverts en cours d'année, tous avec des jours fériés locaux distincts). Écrivez la politique de calendrier que vous imposez : quelles colonnes dans la table de dates, quel dénominateur retenu, ce que vous refusez d'afficher, et comment la voie accessible et la voie moderne cohabitent dans le classeur.

Corrigé de l'évaluation formative.

1. Aucune date n'est typée : la colonne est entièrement du texte (424 en AAAA-MM-JJ, 65 en JJ/MM/AAAA dans l'énoncé reçu). Premier geste : le compteur de conversion — une colonne utilitaire `=DATEVAL(SUPPRESPE(B2))` testée sur les deux formats, puis comparaison des sommes — pas un graphique.
2. `=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[date];">="&DATE(2024;3;1);TableVentes[date];"<="&DATE(2024;3;1))` le 29 février 2024 est attrapé par le chemin précédent (borne haute exclusive sur mars), et mars 2024 a 31 jours — la borne `+1` sur le mois suivant n'a jamais à le savoir. Écrire `<=31/03/2024` reproduirait le problème sur février.
3. Parce que le numérateur contient les ventes du week-end et pas le dénominateur : le rythme est sur-évalué (ici de 27,0 % entre les deux écritures du calcul : 9 730 140 FCFA de ventes de week-end ne se retranchent pas en douceur). À écrire à côté : « jours ouvrés lun→ven, fériés non retirés ; 123 lignes de week-end incluses dans le CA ».
4. `#PROPAGATION!` : la formule est dans un tableau structuré, où la propagation n'est pas prise en charge — on la déplace dans la grille libre à droite du tableau. `#NOM?` : la fonction n'existe pas en 2019 — on la remplace par la voie `SOMME.SI.ENS/filtre`, et on garde `FILTRE` en note.
5. Le défaut : en C2 et C3, la moyenne porte sur deux et trois cellules dont certaines vides, donc « trois mois » veut dire un ou deux mois. Garde-fou : `=SI(NB(B2:B4)=3;MOYENNE(B2:B4);"")`.

Question ouverte — attendus. Une table de dates avec une ligne par jour, le drapeau ouvert par magasin (et non un), la liste des fériés par site, un dénominateur unique annoncé dans le titre (« CA par jour ouvré du site »), le refus explicite des parts de chiffre d'affaires entre magasins ouverts à des dates différentes, et la voie accessible dans les cellules livrées.

Module M03.C08 — Croiser, charger, protéger : tableaux croisés dynamiques, segments, Power Query, modèle de données, DAX de survie

Outil de ce chapitre : le tableur seul, sur le classeur de l'atelier. Durée indicative : 3 h (hors exercices). Niveau : N2.

L'idée du chapitre. Un TCD n'est pas un tour de magie : c'est une requête de regroupement sur une copie des données, figée jusqu'à l'actualisation. Le chapitre commence donc par **écrire ce TCD à la main** — le classeur en contient déjà un, en `SOMME.SI.ENS` et `NB.SI.ENS` — pour savoir ce que l'outil calcule, ce qu'il ne peut pas calculer, et ce que l'actualisation casse. Viennent ensuite les trois machines qui rendent le geste répétable : Power Query (la recette), le modèle de données (les relations), la protection (la survie du classeur).

Base de travail. Le classeur `01_socle_donnees/data/projection/m03_classeur_atelier.xlsx` : `ventes` (tableau `TableVentes`, 480 lignes), `ventes_brutes` (le fichier reçu, 489 lignes énoncées), `produits`, `clients`, `magasins`, `objectifs`, et la feuille `TCD` dont ce chapitre démonte les sept lignes. Point-virgule, UTF-8, franc CFA (FCFA), virgule décimale, TVA 18 %. Chiffres : `chiffres_cites.md`, section M03. Copie de travail : `pilotage_atelier`.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

1. **Construire et auditer** un tableau croisé dynamique : quatre zones, cinq agrégations, contrôle de total, tri par valeur, filtre « cinq premiers », regroupement par mois — et les 36 groupes qui apparaissent quand la colonne date est restée du texte.
2. **Reconnaître ce que le TCD seul ne sait pas faire** : un nombre distincts, une moyenne correcte d'une colonne de moyennes, un cumul qui respecte l'ordre du temps.
3. **Écrire une requête Power Query** de six étapes pour passer de `ventes_brutes` à une table propre, la rejouer, et comprendre pourquoi son chemin d'accès casse chez le voisin.
4. **Brancher quatre tables sur un modèle de données** et dire ce qu'une relation remplace, en colonnes et en erreurs.
5. **Écrire trois mesures DAX** (`SUM`, `DISTINCTCOUNT`, `DIVIDE` avec `CALCULATE/ALL`) et expliquer ce qu'un contexte de filtre change au résultat.
6. **Protéger un classeur sans le rendre muet**, et choisir en connaissance de cause entre requête rejouée, macro enregistrée et Python dans Excel.

2. Pourquoi cette notion est importante

Le fichier `TCD` du classeur d'atelier contient une phrase que je vous demande de garder en tête pour tout le reste du module : « Le TOTAL de la ligne CA doit être exactement le total de la feuille : s'il diffère d'un centime, un filtre est resté actif quelque part. » C'est la seule discipline qui distingue un tableau croisé d'un tableau joli.

Le TCD ment par omission : il agrège une copie prise à l'actualisation, une ligne ajoutée à la source n'existe pas pour lui, une catégorie vide disparaît de la liste, et le lecteur voit le reste

sans voir ce qui a disparu. **La requête, elle, ne ment pas : elle rejoue** — les six étapes de nettoyage sont écrites, nommées, et exécutées à l'identique le mois prochain. C'est la différence entre un résultat et une méthode : M04 la fait documenter, M05 la fait rejouer en Power Query, SQL et pandas.

Le reste du chapitre en est la conséquence : dès qu'on refuse de recopier des colonnes `RECHERCHE` dans la table de faits, il faut décrire les liens quelque part — le tableur propose un endroit, un modèle, et un langage.

3. Explication simple

Un tableau croisé dynamique, c'est une question posée en quatre cases : **en lignes** ce par quoi on découpe (la catégorie), **en colonnes** la deuxième découpe (le mois), **dans la zone Valeurs** le nombre qu'on additionne (le montant), **au filtre** ce qu'on exclut (les retours). Excel écrit la boucle, stocke le résultat dans un cache, et affiche une grille. Le cache explique tous les comportements surprenants : il faut **actualiser** pour relire la source ; la mise en forme de la grille vous appartient, son contenu non plus ; un segment ne filtre que les objets TCD qui y sont connectés.

Power Query est l'inverse d'un TCD : rien n'est stocké en dur, tout est écrit. Vous faites les manipulations une fois, l'outil en garde la liste des étapes, et il la rejoue sur n'importe quel fichier de même forme. Le modèle de données, enfin, est une petite base de données dans le classeur : chaque table garde ses lignes, on déclare que `ventes[produit]` pointe sur `produits[id_produit]`, et les agrégats se calculent à travers ces liens. Une mesure DAX est un calcul sans cellule : elle n'existe que là où un TCD l'affiche.

Dans les faits. La demande type d'un comité : « un TCD par catégorie, avec le nombre de clients ». Le champ Valeurs propose « Nombre », qui compte des lignes : 480 ; les clients distincts sont 372, et l'obtenir exige le modèle. Les deux tiers des écarts signalés en revue sont cette confusion, ou son miroir : des distincts additionnés par catégorie, ce qui fait 384 parce que douze clients achètent dans deux catégories.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Tableau croisé dynamique — PivotTable	Grille de regroupement calculée sur un cache des données sources.	Croire qu'il suit les filtres posés sur la feuille source.
Champ Valeurs — Values field	La mesure agrégée : somme, nombre, moyenne, max, min, produit.	« Nombre » n'est jamais « nombre de choses distinctes ».
Segment — Slicer	Bouton de filtre visuel connecté à un ou plusieurs TCD.	Oublier « Connexions de rapports » : il n'agit que sur ce qu'on lui déclare.
Chronologie — Timeline	Segment spécialisé sur une plage de dates.	Elle ne peut pas s'appuyer sur une colonne de dates-texte.
Éditeur Power Query	Fenêtre où se lisent et se réordonnent les étapes de nettoyage.	Retirer une étape du milieu de la pile : les suivantes se recalculent ailleurs.
Requête — Query	La recette, pas le résultat.	Enregistrer le résultat et jeter la recette.
Modèle de données — Data Model	Tables et relations interrogées par le TCD et le DAX.	Ajouter une seule table au modèle : une relation a besoin de deux côtés.
Relation — Relationship	Lien many-to-one entre une clé et une colonne unique.	Deux tables sans clé commune : ici, 2 lignes de vente sans objectif.
Mesure — Measure	Calcul DAX évalué dans le contexte du TCD qui l'affiche.	La confondre avec une colonne calculée, stockée ligne à ligne.

Français — English	Définition simple	Piège à éviter
Contexte de filtre Filter context	— Le jeu de lignes que le TCD laisse passer à la mesure, à cet instant.	Écrire <code>=DIVIDE([CA];[CA total])</code> et voir [CA total] valoir le mois, pas l'année.
Actualiser tout Refresh All	— Rejoue connexions, requêtes et caches (Ctrl + Alt + F5).	Lancer l'actualisation sur une feuille protégée sans autoriser les TCD.

Définition. Une **agrégation juste** est une agrégation dont on peut nommer les lignes consommées : la moyenne des 480 lignes (75 152 FCFA) répond à « par ligne », la moyenne des sept moyennes de catégorie (98 281) répond à une autre question.

Définition. Dans le modèle, une **colonne calculée** ajoute une valeur par ligne, stockée dans la table et recalculée à l'actualisation — le même mot qu'en C03, où il désigne une formule propagée à la saisie dans le tableau ; la mécanique change, le mot reste. Une **mesure**, elle, ajoute un calcul évalué à la volée pour les cellules affichées. On met dans les colonnes ce qui sert à découper, dans les mesures ce qui s'additionne.

Définition. **Protéger une feuille** interdit d'écrire dans les cellules verrouillées, sans rien chiffrer ni cacher ; **chiffrer le classeur** (Fichier → Informations → Protéger le classeur) est le seul geste qui empêche la lecture du fichier.

Définition. Une **macro enregistrée** est une retranscription d'actions en VBA dans un classeur `.xlsm` : elle rejoue des clics à des adresses précises, sans connaître ni les erreurs ni les changements de forme.

5. Cours approfondi

5.1 Le TCD écrit à la main : ce qu'il calcule, ligne par ligne

La feuille TCD du classeur ne contient aucun TCD : sept lignes et cinq formules recopiées.

Colonne	Formule de la feuille	Ce que le TCD appelle ça
lignes	<code>=NB.SI.ENS(TableVentes[categorie];\$A2)</code>	Valeurs : Nombre de n_ticket
CA (FCFA)	<code>=SOMME.SI.ENS(TableVentes[montant_ttc];TableVentes[categorie];\$A2)</code>	Valeurs : Somme de montant_ttc
moyenne par ligne	<code>=SIERREUR(C2/B2;"")</code>	Valeurs : Moyenne de montant_ttc
part du CA	<code>=SIERREUR(C2/\$C\$9;"")</code>	Afficher les valeurs → Total général en %
lignes remises	<code>=NB.SI.ENS(...;TableVentes[remise];">0")</code>	un deuxième critère, pas une colonne

(la feuille porte les noms anglais, `=COUNTIFS(ventes!H2:H481;$A2)` : le classeur est écrit en anglais par son générateur, le manuel cite le français — voir C02)

Les sept lignes rendent : Plomberie 57 lignes pour 8 754 982 FCFA (24,3 % du CA, 153 596 en moyenne par ligne), Matériaux 143 pour 6 519 357 (18,1 %, 45 590), Electricité 53 pour 6 311 645 (17,5 %, 119 088), Bois & panneaux 24 pour 4 844 758 (13,4 %, 201 865), Quincaillerie 112 pour 4 203 121 (11,7 %, 37 528), Peinture 34 pour 2 938 358 (8,1 %, 86 422), Consommables 57 pour 2 500 964 (6,9 %, 43 877) ; total 480 lignes, 36 073 185 FCFA, 100,0 % — le contrôle de fermeture, celui de la cellule A11.

Trois leçons avant d'ouvrir le ruban. Un : **un TCD ne fait rien que ces formules ne fassent**, et c'est ce qui permet de le vérifier — s'il affiche 98 281 FCFA de moyenne par ligne, il a moyenné sept moyennes au lieu des 480 lignes, où la valeur est 75 152. Deux : **la part du CA est un choix d'affichage**, pas une colonne de plus, et le calcul dans le cache la garde juste quand un segment réduit la grille — à condition de vouloir un pourcentage de ce qui reste affiché, discussion à trancher une fois par graphique. Trois : **les lignes remisées ne passent pas par la zone Valeurs** mais par un critère, et la même question se pose ici par un champ en zone Filtres, par un segment, ou par une mesure — trois objets, trois périmètres d'actualisation.

À retenir. Le premier réflexe après une insertion de TCD n'est pas la mise en forme : c'est la somme du total général contre la somme de la colonne source. Sur l'atelier : 36 073 185 FCFA des deux côtés.

5.2 Les quatre zones, et ce que l'actualisation change

Le TCD de l'atelier, écrit à la main
feuille TCD : sept catégories, cinq mesures, une ligne de total

catégorie	lignes	CA (FCFA)	moy./ligne	part	remises
Plomberie	57	8 754 982	153 596	24,3 %	22
Matériaux	143	6 519 357	45 590	18,1 %	49
Electricité	53	6 311 645	119 088	17,5 %	21
Bois & panneaux	24	4 844 758	201 865	13,4 %	10
Consommables	57	2 500 964	43 877	6,9 %	18
Total	480	36 073 185	75 152	100,0 %	173

« Le TOTAL de la ligne CA doit être exactement le total de la feuille :
s'il diffère d'un centime, un filtre est resté actif quelque part. »

Ce que le TCD simple ne peut pas faire
la demande : des clients distincts, pas des lignes

champ Valeurs = Nombre sur [client]
480 : ce sont les lignes

somme des sept cases « clients »
384, pas 372

vraie valeur distincte, toute la table
372 clients

la case « Ajouter ces données au modèle de données » débloque « Nombre distincts »
— et elle n'existe pas sur Excel pour Mac

Le modèle de données : des relations au lieu de colonnes recopiées

ventes
480 lignes · 13 colonnes
une seule table de faits

produits
154 lignes · prix catalogue

clients
23 500 au référentiel · 103 lignes sans clé

magasins
6 magasins · ville, région, ouverture

objectifs
218 lignes · 2 lignes de vente sans cible

au lieu de
7 colonnes
RECHERCHEV
recopiées

sur 480 lignes
et à ressaisir
à chaque import

1 relation
= 0 formule

Trois mesures DAX, dans Power Pivot

CA ← =SUM(ventes[montant_ttc])
Clients ← =DISTINCTCOUNT(ventes[client])
Part ← =DIVIDE([CA],[CA objectif])

relations : produit → id_produit · client → id_client · magasin → id_magasin · objectifs → annee_mois

Insertion → Tableau croisé dynamique, source `TableVentes` : un tableau structuré est la seule source correcte, une plage `A1:M481` fige le périmètre et la 481^e ligne du mois prochain restera dehors sans un mot. Les zones : **Lignes** et **Colonnes** pour les découpages, l'ordre des champs en Lignes définissant l'imbrication donc ce que « premier » veut dire dans un tri ; **Valeurs** pour l'agrégation, réglée dans Paramètres de champ de valeurs (Somme, Nombre, Moyenne, Max, Min, Produit, puis les modes « Total général en % », « Écart par rapport à », « Cumul ») — le nom affiché est libre, changez-le, il part dans le rapport ; **Filtres** pour le périmètre, là où le TCD peut mentir sans bruit, un filtre oublié rendant un rapport partiel parfaitement cohérent avec lui-même. Le regroupement temporel se règle en clic droit sur le champ de dates (Grouper → Mois, Trimestre, Année) et ne marche que sur une colonne typée : sur le texte de `ventes_brutes`, le même clic produit 36 groupes triés alphabétiquement.

Trois options à vérifier avant toute mise en forme, dans Options du tableau croisé dynamique : onglet Données, « Actualiser les données à l'ouverture du fichier » — à cocher si la source bouge, et à écrire dans la feuille de garde dans les deux cas ; onglet Disposition et format, « Conserver la mise en forme des colonnes lors de l'actualisation », sans quoi les formats monétaires FCFA sautent à chaque `Ctrl + Alt + F5`, plainte n° 1 en salle de formation ; onglet Affi-

chage, format de rapport **tabulaire**, seule grille qui se copie sans cellules fusionnées ni lignes de sous-total orphelines. Le top 5 se pose en filtre de champ (« Dix premiers », ramené à 5, décroissant), jamais en tri manuel : le tri ne survit pas à l'actualisation.

5.3 Segments, chronologie, connexions : ce qui filtre quoi

Un segment (clic dans le TCD → onglet Analyse du tableau croisé dynamique → Insérer un segment) est un filtre avec une interface, et trois propriétés comptent. **Connexions de rapports** : un segment n'agit que sur les objets auxquels on l'a connecté, case par case — c'est le menu où se règlent les tableaux de bord à plusieurs TCD qui ne bougent pas ensemble. **Multi-sélection** : le bouton d'en haut à droite autorise Ctrl + clic, et « Plomberie et Electricité » n'est pas la même demande que « Plomberie ». **Afficher les éléments sans données** : à laisser décoché, sinon une catégorie sans vente du mois reste cliquable et le lecteur lit une ligne vide là où il y a une absence.

Le point que tout le monde rate : **un segment ne filtre pas les formules de la feuille**. Si le titre du tableau de bord contient `=SOMME(TableVentes[montant_ttc])`, il affichera 36 073 185 FCFA quel que soit le segment. Deux écritures cohabitent mal sur une même feuille : soit tout passe par le TCD, soit le reste s'appuie sur une cellule de période que l'utilisateur doit penser à modifier — et il ne le fera pas. La **chronologie** s'insère sur un champ de dates typé et offre un curseur de plage ; posée sur la colonne texte de `ventes_brutes`, elle est refusée, ce qui est une façon très bruyante de rappeler ce que C07 a établi.

5.4 Graphiques : le lié et le figé

Un graphique inséré depuis un TCD est un **graphique croisé dynamique** : il hérite du TCD, filtres compris. Deux conséquences : si le TCD est filtré sur une catégorie au moment de la création, le graphique l'est aussi et le lecteur ne voit rien ; copier ce graphique dans une diapositive le fige, ce qui est voulu à condition d'écrire la date d'extraction dessous. Les boutons de filtre du graphique, je les masque (Affichage des filtres → Masquer tous les boutons de filtres), pour que sa légende ne contredise pas celle du tableau de bord.

Sur les parts de l'atelier, le classement se lit en barres : 24,3 %, 18,1 %, 17,5 %, 13,4 %, 11,7 %, 8,1 %, 6,9 % — un camembert de sept parts dont deux voisines diffèrent de 0,6 point n'est pas un problème de couleur, c'est un problème d'angle. La série mensuelle (douze valeurs, de 1 258 801 à 5 292 517 FCFA) se lit en courbes, la glissante à trois mois en deuxième série : le graphique n'a pas à expliquer ce que le tableau dit déjà. Le combiné à axe secondaire (CA en colonnes, ratio en courbe) est légitime une fois sur deux — et ici son axe ne doit pas se lire comme une performance, le ratio étant calculé sur quatre jours par mois.

Conseil professionnel. Le titre d'un graphique croisé dynamique contient la granularité : « CA par catégorie, magasin 5, lignes de l'extrait 2025, jours 1 à 4 ». Un titre sans granularité est une demande de reprise, pas un défaut de goût.

5.5 Power Query : écrire la recette du nettoyage

Données > Obtenir des données > À partir d'un fichier > À partir de texte/CSV, sur `ventes_brutes`. L'éditeur transforme chaque manipulation en une étape listée à droite, traduite en une ligne de code M lisible dans la barre de formules. Les six étapes de l'atelier :

#	Étape	Ce qu'elle règle
1	Source et détection des types	486 des 489 montants arrivaient en texte ; le type est deviné sur la première ligne
2	Remplacer la valeur . par , sur remise	les 489 remises portaient un point décimal

#	Étape	Ce qu'elle règle
3	Diviser la colonne puis extraire jour, mois, année de date	424 dates en AAAA-MM-JJ, 65 en JJ/MM/AAAA
4	Supprimer les doublons sur n_ticket + produit	12 lignes et 481 978 FCFA ; sur le ticket seul, 173 lignes et 11 424 263 FCFA
5	Filtrer quantité ≤ 500	7 lignes suspectes, dont un 14 000
6	Colonne personnalisée est_retour [montant_ttc] < 0	les 8 retours (−283 933 FCFA) restent, mais deviennent nommables

Quatre réflexes d'usage. **L'ordre est un contrat** : retirer l'étape 2 fait échouer la 3, et tout ce qui suit se recalcule ailleurs, sans message — nommez chaque étape pour ce qu'elle produit, c'est votre journal. **Le profilage avant l'agrégation** : Affichage → Qualité de la colonne affiche les cases vides, distinctes et erronées par colonne, et c'est là que les 103 lignes de client introuvable se voient sans formule. **Charger vers quoi** : la table dans une feuille (pour l'œil), ou Se connecter uniquement + Ajouter ces données au modèle de données (pour le TCD) — la seconde option évite de stocker 480 lignes deux fois. **Le chemin** : la requête mémorise un chemin absolu, d'où la ligne de la feuille de garde ; la correction est un paramètre (Sources de données → Modifier les sources), pas une réécriture des six étapes. Le piège que tout le monde rencontre une fois : une requête qui en lit une autre peut être refusée avec Formula.Firewall, et « Ignorer les niveaux de confidentialité » ne règle le message que sur votre poste — la correction propre fusionne les requêtes dépendantes dans la même chaîne.



Le même résultat, deux contrats : la requête garde la recette, le TCD garde la mise en page.

Excel pour Microsoft 365 · 2021 · 2024 (Windows)	les quatre étapes, complètes
Excel 2019 (Perpetuel)	les quatre étapes aussi ; sans les fonctions dynamiques ni Python dans Excel
Excel pour Mac	Obtenir des données depuis la version 16.69, mais ni modèle ni Power Pivot
LibreOffice Calc	tableaux croisés en natif, ni segments ni Power Query ni modèle : tout en formules

Python dans Excel : Microsoft 365 seulement (Windows depuis 2024, web depuis 2025).

Le calcul part dans le cloud : hors connexion rien ne se recalcule, et ni 2021 ni 2024 perpétuels ne le proposent.

5.6 Le modèle de données : quatre relations, zéro colonne recopiée

Données > Relations, ou la fenêtre Power Pivot (Gérer le modèle de données) et sa vue graphique. Une table de faits, quatre dimensions :

De	À	Ce que ça remplace
ventes[produit] → produits[id_produit]	many-to-one	la colonne de désignation et de catégorie (fiche 19)
ventes[client] → clients[id_client]	many-to-one	RECHERCHEX sur le segment client (fiche 18)
ventes[magasin] → magasins[id_magasin]	many-to-one	le nom, la ville, la région sans les recopier
ventes[annee_mois] → objectifs[annee_mois]	many-to-one	la jointure de la fiche 20, et ses 2 lignes orphelines

Trois remarques de métier. **Une relation exige l'unicité du côté dimension** : le modèle signale ainsi un id_produit en double dans produits, ce qui est une manière brutale mais honnête de dire qu'un référentiel est mal nettoyé. **La clé composée n'existe pas** : d'où la colonne annee_mois fabriquée en C06 et rejouée dans la requête — un modèle propre contient ce genre de colonne de jointure, jamais plus de deux. **Le TCD lit cinq tables comme une seule** : plus de

désignation de produit à maintenir en deux exemplaires — le point où le tableur ressemble à un entrepôt. Une table chargée en connexion uniquement ne vit que dans le modèle : supprimer la requête emporte le TCD qui la lit.

5.7 DAX de survie : trois mesures et deux contextes

Power Pivot → Ajouter une mesure, dans la grille de calcul de la table `ventes`. Les noms de fonctions DAX ne sont pas traduits dans l'interface française, ce qui facilite la lecture des codes trouvés ailleurs. Trois mesures couvrent le tableau de bord de l'atelier :

```
CA := SUM ( ventes[montant_ttc] )
Clients distincts := DISTINCTCOUNT ( ventes[client] )
Part du CA := DIVIDE ( [CA] , CALCULATE ( [CA] ; ALL ( ventes[categorie] ) ) )
```

`SUM`, `COUNTROWS`, `AVERAGEX`, `DISTINCTCOUNT` sont les agrégats de base, évalués **dans le contexte du TCD** : la mesure `CA` vaut 8 754 982 sur la ligne Plomberie, 36 073 185 sur la ligne de total — une mesure n'a pas de valeur, elle a une valeur par cellule affichée, la phrase la plus importante du chapitre. `CALCULATE` est la seule fonction qui change ce contexte : `CALCULATE([CA];ALL(ventes[categorie]))` demande le CA en ignorant le découpage, et fabrique un dénominateur qui ne rétrécit pas quand on clique un segment. `DIVIDE(a;b)` plutôt que `a/b` : sur une catégorie sans vente, `/` produit une erreur (ou un infini, ou un NaN), `DIVIDE` rend un blanc — ou la valeur de son troisième argument ; le blanc est un résultat, pas une excuse. Pour un cumul, marquez une table de dates comme table de dates (clic droit dans Power Pivot), sinon les fonctions temporelles choisissent n'importe quelle colonne de dates d'en face : le prix d'entrée du DAX utile, à payer une fois au §5.6.

Attention. Une **colonne calculée** n'est pas une mesure. `Est gros ticket := ventes[montant_ttc] > 500000` s'évalue ligne à ligne à l'actualisation et occupe une place par ligne — c'est juste pour découper. La mesure `COUNTROWS(FILTER(ventes;ventes[montant_ttc]>500000))` répond à une autre question (combien, dans le contexte affiché) et diverge dès qu'un filtre partiel est actif. Colonne pour découper, mesure pour compter : le vocabulaire du modèle n'est pas un caprice.

5.8 Protéger, automatiser, et renoncer à la macro qui n'en est pas une

Protection. Format de cellule → Protection : toutes les cellules sont verrouillées par défaut, donc « Protéger la feuille » sans rien déverrouiller protège aussi les zones de saisie. La séquence juste : sélectionner les trois ou quatre plages d'entrée (période, magasin, hypothèses), les déverrouiller, puis Réviser → Protéger la feuille, et dans la liste des autorisations cocher « **Utiliser des rapports de tableau croisé dynamique** » — sinon le destinataire ne peut plus actualiser et vous appelle. Ajoutez Protéger le classeur → Protéger la structure contre la suppression d'une feuille de calcul.

Attention. Protéger n'est pas chiffrer : un `.xlsx` dont la feuille est verrouillée se relit très bien. Rendre le contenu illisible est un autre geste, Fichier → Informations → Protéger le classeur → Chiffrer avec un mot de passe. Les deux se font, et les deux s'écrivent dans la feuille de garde.

Macros. Développeur → Enregistrer une macro écrit du VBA, le classeur passe en `.xlsm`, et l'entreprise doit accepter ce format : les macros des fichiers venus d'ailleurs sont désactivées par défaut, bandeau d'approbation à l'appui. Trois limites ne se négocient pas : une macro enregistrée ne s'annule pas (`Ctrl + Z` ne revient pas sur le nettoyage qu'elle vient de faire), elle rejoue des **adresses** (les références relatives sont un bouton du ruban, pas un défaut), et elle ne sait rien faire d'un fichier dont la forme a changé — une colonne insérée et elle écrit à côté. Bilan

honnête : enregistrer sert à documenter une suite de gestes pour la convertir en procédure ou en requête, pas à la livrer. Power Query rejoue le nettoyage sans code (Ctrl + Alt + F5) et survit au changement de nom de fichier ; la macro ne garde un sens que pour ce que la requête ne sait pas faire — parler à une autre application, écrire un fichier de sortie, cliquer dans un formulaire.

Python dans Excel. Le bouton Insérer Python (onglet Formules) place un bloc =PY() dans une cellule ; le code tourne dans un conteneur Python du côté du cloud Microsoft, avec pandas et matplotlib, dans le cadre du calcul standard inclus — un add-on débloque du calcul supérieur. Trois conséquences pour un manuel : hors connexion la cellule affiche sa dernière valeur et ne recalcule pas ; ni le disque local ni Internet ne sont accessibles depuis ce conteneur ; et les versions perpétuelles (2021, 2024) ne proposent pas la fonction. C’est un outil d’analyse dans la feuille, pas un moteur d’automatisation. Côté LibreOffice : ni Power Query, ni modèle de données, ni DAX, ni ce Python-là, et pas de segments du tout (le point d’appui tdf#119807 est ouvert depuis 2018, et un classeur contenant des segments ne s’y importe même pas) — ses tableaux croisés et ses macros Basic couvrent les §5.1, §5.2 et la protection de §5.8, le reste s’écrit en formules ou se transporte ailleurs.

6. Exemple concret : « combien de clients par catégorie ? »

La demande du comité, lue telle quelle : un TCD avec les catégories en lignes et le nombre de clients. Trois écritures possibles sur le fichier, une seule réponse juste.

Écriture	Résultat	Ce qu’elle compte
TCD, client en Valeurs → Nombre	480 au total	des lignes de vente
TCD, client en Nombre distincts , source hors modèle	impossible	l’option est grisée : elle exige le modèle
modèle + mesure DISTINCTCOUNT(ventes[client])	372 au total	des clients, distincts
somme des sept cases « clients distincts » par catégorie	384	des clients, une fois par catégorie fréquentée

Les 12 d’écart entre 384 et 372 ne sont pas une erreur : ce sont les clients qui achètent dans deux catégories. La question que personne n’a posée est donc la bonne — faut-il additionner les distincts ligne à ligne, ou afficher le distinct global ? — et la réponse s’écrit dans le commentaire du graphique, parce qu’un total de nombres distincts ne se déduit pas des lignes. Deux détails du fichier ajoutent un étage : 103 lignes portent un client impossible à relier au référentiel (85 avec le client comptoir 0, 18 avec le champ vide), et un TCD sur client en fera trois catégories d’inconnu si personne ne les nomme avant. Dans le même registre, la moyenne des moyennes de catégories vaut 98 281 FCFA quand la moyenne réelle par ligne vaut 75 152 — le premier chiffre sort tout seul d’un TCD à qui on donne une colonne de moyennes, et il est faux pour toute phrase contenant « en moyenne, une ligne fait ».

7. Démonstration pas à pas : de la brute au pilotage

Objectif. Un onglet Pilotage : une requête, un modèle à trois relations, un TCD à deux champs et deux mesures, un segment, un graphique, la protection.

Étape	Geste	Ce qui doit se lire
1	Requête Power Query sur ventes_brutes : types, point décimal, dates, doublons sur n_ticket+produit, filtre quantité ≤ 500, colonne est_retour	480 lignes chargées ; la pile de six étapes est nommée

Étape	Geste	Ce qui doit se lire
2	Charger vers → connexion uniquement + ajouter au modèle de données	aucune table dans la feuille ; Données > Relations en voit une
3	Relations : ventes[produit] → produits, ventes[client] → clients, ventes[magasin] → magasins	trois relations many-to-one créées sans message d'erreur
4	Mesures : CA := SUM(ventes[montant_ttc]), Clients := DISTINCTCOUNT(ventes[client])	insérables dans le TCD, alors que la colonne brute s'y insère en « Somme »
5	TCD (source : le modèle) : categorie en lignes, CA et Clients en valeurs, tri décroissant sur CA	Plomberie 8 754 982 en tête, total 36 073 185 FCFA, 372 clients au total de la colonne
6	Segment magasins[ville], connecté au TCD ; graphique en barres lié ; protection de feuille avec TCD autorisés	le graphique bouge avec le segment, la cellule de titre ne bouge pas

Contrôles qualité du chapitre. (1) Total du TCD = total de la feuille source : 36 073 185 FCFA. (2) Le compte des lignes actualisé après l'étape 1 vaut 480, et l'extrait nettoyé de la requête rend le même CA que ventes : sinon le doublon a été retiré deux fois ou pas du tout. (3) Somme de la colonne « part du CA » = 100,0 %, segments non filtrés. (4) Après un changement de mois dans le segment, le total du TCD diminue mais la cellule de titre `SOMME(TableVentes[montant_ttc])` n'a pas bougé — c'est la démonstration, à montrer une fois, de ce qu'un segment ne fait pas.

Barème (10 points). Requête nommée et rejouable (3) · relations sans clonage de colonnes (2) · deux mesures utilisées dans le TCD (2) · total contrôlé contre la source (1) · segment connecté aux deux objets (1) · protection avec TCD autorisés et mention du chiffrement (1).

8. Erreurs fréquentes

1. **Sourcer le TCD sur une plage au lieu d'un tableau** : la 481^e ligne n'entre jamais dans le cache, et le total reste plausible — juste faux depuis trois semaines.
2. **Confondre « Nombre » et « nombre distincts »** : 480 lignes pour 372 clients, et l'option distincte n'existe pas sans le modèle de données.
3. **Demander une moyenne à un TCD nourri de moyennes** : 98 281 au lieu de 75 152 — le piège est dans la colonne donnée, pas dans l'agrégateur.
4. **Ne pas vérifier le total général** après insertion, actualisation ou changement de filtre : la phrase de la feuille TCD est là pour ça.
5. **Croire qu'un segment filtre la feuille** : il filtre les objets connectés ; une formule à côté reste sur le périmètre complet, et le lecteur voit un titre contredire le graphique.
6. **Grouper par mois une colonne texte** : 36 groupes triés alphabétiquement au lieu de douze mois.
7. **Retirer une étape au milieu de la pile Power Query** sans relire les suivantes : le nettoyage se rejoue sur une autre base, silencieusement.
8. **Livrer une requête avec un chemin absolu** et changer de poste : l'actualisation tombe, et l'éditeur de requête n'est pas le premier endroit où l'on regarde.
9. **Protéger la feuille sans autoriser les TCD** : plus personne n'actualise. Et à l'inverse, confondre protection de feuille et chiffrement du fichier.
10. **Enregistrer une macro pour nettoyer un fichier reçu chaque lundi** : elle rejoue des adresses, ne s'annule pas, et saute une ligne dès qu'on insère une colonne.

9. Bonnes pratiques professionnelles

1. **Total d'abord, beauté ensuite** : le TCD inséré se contrôle contre sa source (=C9-SOMME(TableVentes[montant_ttc]), attendu 0), écrit dans la feuille, avant toute mise en forme.
2. **Une nomenclature**. `tcd_ca_categorie`, `req_ventes_nettoyees` : les connexions se gèrent par nom, et les collisions d'objets sont le seul plantage vraiment pénible.
3. **Les dimensions dans le modèle, les faits dans la requête**. La table de faits garde ses clés ; les libellés viennent des dimensions par relation.
4. **Une table de dates marquée comme telle**, dès qu'il y a un cumul ou une comparaison annuelle. C'est le seul investissement qui rende le DAX temporel fiable.
5. **La fiche de vie du classeur** : source, granularité, date d'extraction, chemin attendu, ce qui doit être actualisé à la main. Trois lignes, dans une feuille `Aidez-moi` — le classeur de l'atelier en a une, relisez-la.
6. **Format .xlsx sauf nécessité, .xlsm assumé, jamais de macro cachée dans un classeur « pour faire joli »**. La sécurité de l'entreprise est un sujet de livraison, pas un détail d'extension.
7. **Exporter le TCD en valeurs avant de le commenter** : coller les valeurs, noter la date, garder la source.

10. Exercice guidé — le tableau de bord à trois segments (45 min, /10)

Commande. Sur la copie `pilotage_atelier`, livrez un onglet `Pilotage` tenant sur un écran : un TCD par catégorie, un graphique, trois segments (ville, catégorie, mois), et une protection qui laisse naviguer.

Étape	Geste	Ce qui doit se lire
1	TCD depuis le modèle : <code>categorie</code> en lignes, <code>CA</code> et <code>Clients distincts</code> en valeurs, tri décroissant	Plomberie 8 754 982 ; total de colonne 372, total de lignes 480
2	Deuxième TCD : <code>mois</code> en lignes, <code>CA</code> en valeurs, en % du total général	douze parts, 100,0 % en bas de colonne
3	Trois segments : <code>magasins[ville]</code> , <code>ventes[categorie]</code> , <code>Calendrier[mois]</code>	chaque segment connecté aux deux TCD via « Connexions de rapports »
4	Graphique croisé dynamique en barres, boutons de filtres masqués	le classement suit le segment catégorie
5	<code>CA / objectif</code> en colonne calculée du modèle ou en mesure	18,2 % sur l'extrait, avec la mention d'échelle
6	Protection : déverrouiller les segments et les cellules de saisie, verrouiller le reste, autoriser les TCD	Actualiser fonctionne encore, Supprimer la feuille non

Démarrage. Étape 3, le piège est dans « les deux » : un segment inséré depuis un TCD n'est connecté qu'à celui-là par défaut — vous obtiendrez un graphique qui bouge pendant que le TCD voisin reste figé, aspect exact d'un tableau de bord non reçu. Étape 5, ne fabriquez pas le ratio en important des colonnes d'objectifs dans `ventes` : le total d'objectif serait additionné autant de fois qu'il y a de lignes de vente correspondantes — le défaut le plus coûteux du tableur, que le modèle rend impossible. Étape 6, testez la protection avant de fermer : clic droit sur le TCD → Actualiser doit rendre la même valeur.

Barème (10 points). TCD justes et total contrôlé (3) · segments connectés aux deux objets (2) · graphique cohérent avec son TCD et titré avec la granularité (1) · ratio d'objectif au bon endroit

(mesure, pas colonne) (2) · protection sans perte d'usage (1) · fiche de vie complétée avec le chemin de la requête (1).

11. Exercices autonomes

Exercice 8.1 (★) — Le contrôle du total. Reprenez le TCD de la §5.1 (catégories en lignes, CA en valeurs), filtrez une valeur au hasard dans la zone Filtres, notez les deux totaux, expliquez la différence en une phrase et écrivez la formule de la feuille qui aurait détecté le filtre.

Exercice 8.2 (★) — Cinq agrégations sur la même colonne. Sur `montant_ttc`, affichez Somme, Nombre, Moyenne, Max et « Total général en % » ; contrôlez la Moyenne contre la moyenne des 480 lignes (75 152 FCFA), puis refaites l'expérience avec une colonne de moyennes par catégorie : quel chiffre sort, et pourquoi il n'est pas faux mais inutile.

Exercice 8.3 (★★) — La requête rejouée. Écrivez la requête de six étapes du §5.5, puis, sans la modifier, renommez et déplacez le fichier source et actualisez : consignez le message exact, la correction par paramètre, le temps perdu. Ajoutez une 7^e étape qui renomme les colonnes en français et charge vers une table.

Exercice 8.4 (★★) — Doublons, deux réponses. Testez Supprimer les doublons sur `n_ticket` seul, puis sur `n_ticket` + `produit` : donnez les deux comptes de lignes et les deux CA, dites lequel répond à « une ligne par produit par ticket », vérifiez que le résultat dépend de l'ordre des lignes et corrigez par un tri explicite.

Exercice 8.5 (★★★) — Mesures contre colonnes. Créez la colonne calculée `Ligne_grosse := ventes[montant_ttc] > 500000` et la mesure `Gros_tickets := COUNTROWS(FILTER(ventes;ventes[montant_ttc]>500000))`, affichez les deux dans un TCD par catégorie puis sous un segment qui isole une catégorie : expliquez la divergence et dites laquelle des deux écritures va dans le classeur livré.

Exercice 8.6 (★★★) — La macro, puis son cercueil. Enregistrez une macro appliquant les étapes 2 à 5 du §5.5 sur `ventes_brutes`, lisez le VBA produit, insérez une colonne en C, rejouez, consignez. Concluez en une demi-page : pour quoi la requête remplace la macro ici, et pour quoi elle ne remplace pas l'envoi d'un e-mail.

12. Correction détaillée

Exercice 8.1. Le filtre de zone Filtres réduit le cache lu : le total passe de 36 073 185 à la valeur de la catégorie (2 500 964 pour Consommables), la grille gardant sa présentation — c'est ce qui rend l'erreur invisible. Contrôle : `=C9-SOMME(TableVentes[montant_ttc])`, attendu 0, dans la feuille de garde, pas dans le coin du graphique.

Exercice 8.2. Somme 36 073 185, Nombre 480, Moyenne 75 152, Max 1 229 678, parts sommant à 100,0 %. La moyenne appliquée à une colonne de moyennes par catégorie rend 98 281 : exacte en tant que moyenne des sept nombres reçus, muette sur « par ligne ». Le piège est dans le choix de la colonne, jamais dans l'agrégateur.

Exercice 8.3. L'actualisation renvoie une erreur de source (« chemin introuvable », `Data-Source.Error`). Correction : Requêtes et connexions → Propriétés, ou Sources de données → Modifier les sources ; version propre, un paramètre `Chemin_données` lu par l'étape Source, qui rend la requête mobile. Temps relevé : de une à dix minutes.

Exercice 8.4. Sur `n_ticket` seul : 173 lignes retirées, 11 424 263 FCFA disparus — tous les autres articles des tickets à plusieurs lignes, un tiers du CA en moins, aucun sens. Sur `n_ticket` + `produit` : 12 lignes, 481 978 FCFA, la règle métier. Le tri préalable (`n_ticket`, `produit`, puis

l'ordre d'entrée) fixe quelle occurrence survit : sans lui, la ligne conservée dépend du hasard de l'export.

Exercice 8.5. La colonne marque chaque ligne, le segment la laisse simplement moins souvent à VRAI. La mesure, évaluée dans le contexte, ne compte que les gros tickets de la catégorie cliquée, et son total de colonne peut différer de la somme des lignes si un filtre partiel est actif. Attendu : le classeur livré porte la mesure, recalculée ; la colonne est une copie figée dont le destinataire ignore la date.

Exercice 8.6. Le code généré est une suite d'affectations et de `Selection.End(xlDown)` : il casse dès qu'une colonne est insérée, et rejoue il écrit dans la voisine. Attendu : la requête couvre le nettoyage (six étapes, un clic) ; la macro ne couvre que l'interface ou une autre application — écrite à la main et testée, pas enregistrée.

13. Mini-projet M03.P1 — suite : le volet pilotage (50 min)

Commande. Complétez `calendrier_atelier` par le volet de pilotage, livrable à une personne qui ne vous connaît pas et qui devra l'actualiser.

Livrables. (1) `req_ventes_nettoyees`, six à sept étapes nommées, chargée en connexion + modèle, chemin piloté par un paramètre documenté dans `Aidez-moi` ; (2) le modèle : trois relations plus celle des objectifs, aucune colonne de libellé recopiée dans la table de faits ; (3) `Pilotage` : deux TCD (catégories, mois), trois segments connectés aux deux, un graphique en barres titré avec sa granularité ; (4) au moins deux mesures (`CA`, `Clients distincts`) et le ratio `CA / objectif` en mesure, avec la mention d'échelle ; (5) la protection : saisie dégagée, TCD autorisés, structure verrouillée, et la ligne qui dit ce que la protection ne fait pas (elle ne chiffre pas) ; (6) le contrôle de fermeture en une cellule visible : total du TCD contre total de la source, attendu 0.

Barème (20 points, seuil 13). Requête mobile et documentée (4) · modèle sans colonnes dupliquées (4) · TCD et segments qui bougent ensemble (4) · mesures justes et ratio au bon endroit (4) · protection utile sans être muette (2) · contrôle de fermeture visible (2).

14. Résumé du chapitre

Un tableau croisé dynamique est une boucle de regroupement sur un cache : quatre zones, cinq agrégations, un total à vérifier contre la source avant tout le reste. Il fait en trois clics ce que la feuille `TCD` écrit en quarante formules, et ne sait pas compter le distinct sans modèle, ni suivre un filtre posé sur la plage, ni filtrer les formules d'à côté. Segments et chronologie ajoutent l'interface, le graphique croisé dynamique le lien — et les filtres cachés.

Power Query déplace le travail du résultat vers la méthode : six étapes nommées, rejouées, portables si le chemin devient un paramètre. Le modèle remplace les colonnes `RECHERCHEV` par quatre relations, et le DAX y ajoute des mesures qui n'existent que dans le contexte du TCD — `CALCULATE` pour changer ce contexte, `DIVIDE` pour ne pas mourir sur un dénominateur blanc, `DISTINCTCOUNT` pour la question qu'aucun bouton ne pose. Protection et automatisation ferment la boucle : déverrouiller ce qui doit l'être, autoriser les TCD, chiffrer ce qui doit l'être, et ne pas confondre une macro enregistrée avec une procédure.

À retenir. 480 lignes, 372 clients, 384 si on somme les distincts par catégorie, 98 281 si on moyenne des moyennes : quatre nombres issus de gestes standard, un seul répond à la question.

À retenir. Un TCD est une photographie qui sait se recharger ; une requête est une méthode qui sait se relire. Un classeur de pilotage sérieux a les deux, et une fiche qui dit lequel actualiser en premier.

15. À retenir

1. **Le total d'abord.** Aucun TCD ne part en réunion sans que sa ligne de total ait été comparée à la somme de la colonne source, format FCFA, écart attendu 0.
2. **Le cache décide, pas la feuille.** Un TCD suit son actualisation, un segment suit ses connexions, une requête suit sa pile d'étapes : trois rythmes différents, à écrire dans la fiche de vie du classeur.
3. **Modèle pour les relations, mesures pour les ratios, requête pour le nettoyage, macro pour l'interface seule :** ce n'est pas une hiérarchie de difficulté, c'est une répartition des risques.

16. Évaluation formative (auto-correction, 10 min)

1. Un TCD affiche 35 500 000 FCFA alors que `=SOMME(TableVentes[montant_ttc])` rend 36 073 185. Citez les trois causes possibles, dans l'ordre où vous les testez.
2. On demande « le nombre de clients par vendeur » dans un TCD simple. Que faites-vous, et pourquoi le menu contextuel ne suffit pas ?
3. Un collègue retire une étape au milieu de la pile Power Query et obtient un résultat plausible. Pourquoi est-ce plus grave qu'une erreur affichée, et quelle habitude l'empêche ?
4. Écrivez la mesure du CA d'un mois rapporté au CA de l'année, et dites ce que change `DIVIDE` contre `/` sur un mois sans vente.
5. Classeur livré protégé : le destinataire ne peut plus actualiser le TCD, et lit le fichier sans mot de passe. Quel geste a été fait, lequel a été confondu avec quoi, que corrigez-vous ?

Question ouverte. Votre direction veut un « tableau de bord Excel » mensuel, servi par trois personnes sur des postes hétérogènes (Microsoft 365, Excel 2019, LibreOffice, deux Mac). Écrivez la répartition : ce qui vit dans une requête, dans le modèle, en TCD, en formules ; ce que vous refusez de faire en macro ; et la politique de noms, de protection et de dates d'extraction qui rend le tout rejouable par une personne qui ne vous connaît pas.

Corrigé de l'évaluation formative.

1. (a) un filtre oublié en zone Filtres ou sur un segment — le plus fréquent, visible en dix secondes ; (b) une source figée (`A1:M481`) qui n'a pas suivi l'ajout de lignes, ou un cache non actualisé ; (c) des montants restés texte, exclus de la Somme — le sort de 486 des 489 lignes du fichier reçu avant nettoyage.
2. `client` en Valeurs avec l'agrégat « Nombre » compte des lignes. Le menu contextuel ne propose « Nombre distincts » que si la source a été ajoutée au modèle de données à la création du TCD, ou via une mesure `DISTINCTCOUNT`. Sur un poste Mac, l'option n'existe pas : la voie accessible est une feuille `=SOMME(1/NB.SI.ENS(...))`, ou le même travail en SQL au module M05.
3. Les étapes se rejouent sur la sortie de la précédente : en retirer une ne casse rien de visible, cela décale la base de toutes les suivantes. L'habitude qui empêche ça : renommer chaque

étape pour qu'elle dise ce qu'elle produit, et lire le compte de lignes de la prévisualisation (480 → 468 saute aux yeux).

4. `MoisPart := DIVIDE([CA];CALCULATE([CA];ALL(Calendrier[mois])))` — `ALLSELECTED` si le segment doit continuer d'agir sur l'année. Avec `/`, un dénominateur blanc produit une erreur (ou un infini) qui contamine mise en forme conditionnelle et graphiques ; `DIVIDE` rend un blanc, que la mise en forme lit comme une absence assumée.
5. Fait : Réviser → Protéger la feuille sans cocher « Utiliser des rapports de tableau croisé dynamique », donc actualisation interdite. Confondu : protection de feuille ≠ chiffrement — un fichier protégé se lit, il ne s'édite pas. Corrections : autoriser les TCD, et séparer les besoins dans la fiche de vie (chiffrer ce qui doit être illisible, verrouiller ce qui doit être intègre).

Question ouverte — attendus. Une requête (nettoyage, six étapes, chemin paramétré) ; un modèle (quatre relations, une table de dates marquée, trois mesures) ; des TCD et un graphique croisé dynamique par axe de décision ; des formules uniquement pour les contrôles de fermeture et la feuille de garde ; aucune macro pour le traitement de données, au plus une macro documentée pour l'export ; la politique de noms et de dates écrite dans *Aidez-moi*, avec la mention explicite des postes sans modèle de données (Mac) et sans segments (LibreOffice), et de la voie alternative pour chacun d'eux.

Projet M03.P — « Le classeur de pilotage de Sahel Distribution »

Module M03 · projet de fin de module · 6 h · à rendre avant le module 4 · barème /20, seuil de passage 13/20

Le cadre réel. Vous êtes chez Sahel Distribution SA, et c'est la première fois qu'on vous demande un **classeur** plutôt qu'un rapport. Le directeur commercial a dit : « je veux ouvrir le fichier le lundi matin, changer le mois, et que tout bouge ». Cette phrase contient tout le module : une donnée nettoyée par une méthode écrite, des calculs qui survivent à l'actualisation, un tableau de bord qui ne ment pas sur son échelle, et une forme qui permet à quelqu'un d'autre de rouvrir le fichier sans vous appeler. **C'est la première pièce de votre portfolio : elle sera revue par un recruteur, pas seulement par moi.**

Note de cohérence. Le plan validé décrit « un fichier de ventes + un fichier de cibles, classeur à trois feuilles : données préparées, calculs, tableau de bord ». Le socle de la formation fournit exactement ces deux entrées — `ventes_brutes.csv` (489 lignes énoncées) et `objectifs_de_ca.csv` (218 lignes jointes) — et le classeur d'entraînement du module, `m03_classeur_atelier.xlsx`, en présente une version disséquée en onze feuilles. Vous rendez, vous, **trois feuilles de rendu** ; les annexes sont admises, pas comptées. Les chiffres de ce corrigé sont ceux du socle livré, vérifiables ligne à ligne.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Fichier de ventes brut	<code>01_socle_donnees/data/brut/ventes_brutes.csv</code>	le fichier reçu : 489 lignes énoncées, montants en texte, dates en deux écritures
Fichier nettoyé de contrôle	<code>01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv</code>	ce que la préparation doit produire : 480 lignes, 316 tickets
Cibles commerciales	<code>01_socle_donnees/data/brut/objectifs_de_ca.csv</code>	218 lignes magasin × année × mois, 198 030 000 FCFA sur le magasin 5 en 2025
Référentiels	<code>01_socle_donnees/data/brut/produits.csv</code> , <code>clients.csv</code> , <code>magasins.csv</code>	les dimensions du modèle
Classeur de référence	<code>01_socle_donnees/data/projection/m03_classeur_atelier.xlsx</code>	la version d'enseignement : 11 feuilles, 28 fiches de fonction, une grille mensuelle, une grille croisée
Contrôle des chiffres	<code>01_socle_donnees/data/reference/chiffres_cites.md</code>	à utiliser après le calcul, jamais comme source de copie
Outils	un tableur (Excel 2021+ ou LibreOffice Calc), Power Query et le modèle de données si votre poste les a	sinon : la voie accessible, et vous l'écrivez

Les six livrables

L1 — Feuille Données (la préparation, par requête). Une requête nommée, de six étapes au minimum, qui part du CSV brut et rend 480 lignes × 16 colonnes : types corrigés, séparateur décimal des remises, dates reconstituées dans les deux écritures, doublons traités à la règle « une

ligne par ticket et par produit », quantité filtrée, colonne `est_retour` ajoutée, plus deux colonnes de jointure `mois` et `annee_mois`. Le chemin du fichier source passe par un paramètre ou une cellule nommée, jamais par un chemin tapé dans l'étape.

L2 — Feuille `Calculs` (les fiches de fonction). **Huit fiches au format du module** — nom et version · syntaxe · rôle de chaque argument · exemple chiffré sur le jeu de l'atelier · cas réel d'entreprise · erreur fréquente et son symptôme — dont trois au choix hors de la liste d'office (`SOMME.SI.ENS`, `NB.SI.ENS`, `RECHERCHEX`, `INDEX/EQUIV`, `FIN.MOIS`, `NB.JOURS.OUVRES`, `FILTRE` étant imposées). Chaque fiche occupe une ligne du tableau de bord des fiches et sa formule doit être **exécutée dans la feuille**, pas décrite.

L3 — Feuille `Pilotage` (le tableau de bord). Un tableau croisé dynamique par catégorie (lignes, CA, moyenne, part en % du total général), un second par mois avec les jours ouvrés, **quatre graphiques** (classement en barres, évolution en courbes avec moyenne glissante à trois mois, parts, et un quatrième au choix argumenté), **trois segments** (ville, catégorie, mois) connectés aux deux TCD, et le ratio `CA / objectif`. Aucune valeur tapée à la main dans cette feuille.

L4 — Formatage, noms, protection. Formats monétaires FCFA à zéro décimale, pourcentages à une décimale, dates en `JJ/MM/AAAA` ; **cinq noms** au minimum (`Debut_période`, `Fin_période`, `req_ventes_nettoyees`, `TCD_ca_categorie`, `Controle_total`) ; protection de la feuille `Pilotage` avec saisie délogée là où elle est prévue et TCD autorisés ; structure du classeur verrouillée ; le tout ouvrant sur `Pilotage`.

L5 — Note de cadrage (2 pages, PDF). Ce que le classeur mesure, ce qu'il ne mesure pas, et pourquoi. Trois rubriques obligatoires : **l'échelle** (l'extrait ne couvre que les jours 1 à 4 de chaque mois), **le dénominateur** (CA par ligne, par ticket, par jour ouvré : les trois, avec les chiffres), **la voie** (ce qui est écrit en fonction dynamique et ce qui ne l'est pas, et sur quel poste ça casse). Vous y joignez la **procédure d'actualisation** en cinq lignes max, écrite pour quelqu'un qui ne vous connaît pas.

L6 — Le dossier de rejouabilité. Un classeur vierge de vos feuilles de résultat, reconstruit en une seule actualisation depuis le CSV, avec l'état des contrôles à l'appui : la page d'un tableur montrant `Controle_total` à 0, le compte de lignes de la requête, et le message exact renvoyé si le fichier source a bougé. C'est la partie que le correcteur exécute en premier ; elle vaut même si le reste est imparfait.

Contraintes de rendu

1. Deux fichiers : `2026-MM-JJ_projet_M03_P_votre_nom.xlsx` et `2026-MM-JJ_projet_M03_P_note.pdf`. Le classeur est ouvert sur `Pilotage`, zoom 100 %, aucune sélection en cours.
2. Aucune valeur saisie dans `Pilotage` : tout vient d'un TCD, d'une mesure, ou d'une formule qui lit la feuille `Données`. Aucun chiffre recopié depuis `chiffres_cites.md` sans son calcul à portée de clic.
3. Chaque cellule de contrôle est écrite en équation visible (`=total_TCD - total_source`, attendu 0), pas en commentaire « vérifié ».
4. Les graphiques portent un titre qui dit la granularité et la période, et une source en légende.
5. Si votre poste n'a ni Power Query ni modèle de données (LibreOffice, Excel pour Mac, Excel 2019 sans add-in) : vous livrez la voie accessible **et** vous écrivez en L5 ce que la voie moderne aurait changé. Le barème ne sanctionne pas l'outil, il sanctionne le silence sur l'outil.
6. **Interdits** : macro enregistrée en guise de nettoyage ; mise en forme conditionnelle qui colore un chiffre que rien ne contrôle ; format date appliqué à une colonne de texte sans conversion ; TCD pointant sur une plage figée ; mot de passe présenté comme une confidentialité.

2. Barème détaillé

#	Critère	Pts	Ce qui fait perdre les points
1	L1 : requête en six étapes, paramétrée, rendant 480 lignes justes	4	doublons traités sur le ticket seul ; étapes anonymes ; chemin absolu en dur
2	L2 : huit fiches exécutées, au format imposé, avec leur erreur fréquente	4	fiche descriptive sans formule ; exemple hors jeu de l'atelier ; « cas réel » rédigé en généralité
3	L3 : deux TCD, quatre graphiques, trois segments, ratio à l'objectif	5	segments connectés à un seul TCD ; parts calculées à la main ; graphique héritant d'un filtre caché
4	L4 : formats, cinq noms, protection utilisable	2	TCD inactualisable sous protection ; formats FCFA absents ; aucune plage nommée
5	L5 : note de cadrage — échelle, dénominateur, voie	3	« 27,9 % » écrit sans dire que décembre n'a que quatre jours dans l'extrait
6	L6 : dossier de jouabilité et contrôles à zéro	2	contrôle annoncé mais non écrit ; actualisation qui ne repart pas du CSV
Total		20	seuil de passage en M04 : 13

3. Correction pas à pas (corrigé enseignant)

Ce corrigé est un compte-rendu d'exécution. Les valeurs sont celles du socle livré ; si votre fichier diffère, régénérez le socle avant de vous comparer. Le correcteur ouvre d'abord le classeur, jamais la note.

3.1 — L1 : la préparation attendue, étape par étape

#	Étape de requête	Ce qu'elle doit produire, chiffré
1	Source (chemin lu dans un paramètre)	489 lignes lues, 13 colonnes, types non figés
2	Remplacer . par , puis Charger le type sur les montants	486 des 489 montants arrivaient en texte, 3 seulement en nombre
3	Analyser les dates : 424 en AAAA-MM-JJ, 65 en JJ/MM/AAAA	une colonne date typée, amplitude du 01/01/2025 au 02/12/2025
4	Doublons sur n_ticket + produit, après tri	12 lignes retirées, 481 978 FCFA ; la variante sur le ticket seul retire 173 lignes et 11 424 263 FCFA — erreur de lot complète
5	Filtrer quantité ≤ 500	7 lignes hors échelle, maximum reçu 14 000 contre 68 après correction d'unité
6	Colonne personnalisée est_retour	8 lignes vraies à VRAI, 283 933 FCFA retirés du CA s'ils sont exclus à tort
7	Colonnes de jointure mois et annee_mois	12 valeurs de mois, une clé texte unique par ligne d'objectif

Le chargement attendu est **connexion uniquement + ajout au modèle de données**, avec une table visible dans Données pour l'œil : 480 lignes × 16 colonnes. Le total de contrôle, =SOMME(Données[montant_ttc]), doit rendre 36 073 185 FCFA, soit exactement la somme du fichier nettoyé. Un candidat qui a 36 339 317 FCFA n'a pas retiré les doublons : l'écart est de 266 132 FCFA, et il est écrit dans le corrigé de M01.C06.

Deux points de discrimination, visibles sans ouvrir la note :

1. la colonne **categorie** provient-elle d'une jointure ou d'une recopie ? Le livrable attendu la tire de produits par relation (ou par RECHERCHEX dans la voie accessible), pas d'un collage depuis le CSV reçu — la preuve est dans l'en-tête de la table : produits amène le libellé, pas la vente.

2. **les 103 lignes de client introuvable sont-elles nommées ?** 85 portent le client comptoir 0, 18 n'ont rien : la préparation correcte crée une valeur lisible (« comptoir » ou « non renseigné ») plutôt que de laisser un TCD afficher un blanc en tête de classement.

3.2 — L2 : ce que rendent les fiches exécutées dans **calculs**

Les huit valeurs de contrôle ci-dessous sont celles que la feuille de l'atelier affiche ; le candidat doit les retrouver, pas les recopier.

Fonction	Valeur attendue sur l'atelier	Détail de contrôle
SOMME	36 073 185 FCFA	sur <code>montant_ttc</code> , 480 lignes
MOYENNE	75 152 FCFA	par ligne , à ne pas confondre avec le panier
MEDIANE	37 966 FCFA	la moitié des lignes est en dessous
ECARTYPE.STANDARD	143 915 FCFA	supérieur à la moyenne : aucune moyenne ne représente ce fichier
MAX / MIN	1 229 678 / -150 804 FCFA	le minimum est un retour, pas une erreur de signe
QUARTILE.INC (1, 3)	12 950 / 74 234 FCFA	la moitié centrale tient sur 61 284 FCFA
NB.JOURS.OUVRES	240 jours	du 01/01/2025 au 02/12/2025, fériés non retirés
SOMME.SI.ENS ratio	/ 18,2 %	36 073 185 FCFA sur 198 030 000 FCFA d'objectif magasin 5

La fiche la plus ratée est `NB.SI/NB` sur une colonne texte : le candidat qui compte les montants du fichier reçu avec `=NB()` obtient 0, et beaucoup écrivent dans la fiche « 0 montants, donc un fichier vide ». Le symptôme à documenter est l'inverse : `=SOMME()` sur la même plage rend 0 sans erreur, ce qui est précisément le piège traité en M03.C05. La fiche la mieux notée est celle qui montre un `RECHERCHEX` échouant sur une clé avec espace invisible, et sa correction par `SUPPRES-PACE` — trois lignes, un cas réel, un symptôme.

3.3 — L3 : le tableau de bord, avec ses nombres

Le TCD par catégorie doit rendre, dans cet ordre de classement :

catégorie	lignes	CA (FCFA)	moyenne par ligne	part	lignes remises
Plomberie	57	8 754 982	153 596	24,3 %	22
Matériaux	143	6 519 357	45 590	18,1 %	49
Electricité	53	6 311 645	119 088	17,5 %	21
Bois & panneaux	24	4 844 758	201 865	13,4 %	10
Quincaillerie	112	4 203 121	37 528	11,7 %	45
Peinture	34	2 938 358	86 422	8,1 %	8
Consommables	57	2 500 964	43 877	6,9 %	18
Total	480	36 073 185	75 152	100,0 %	173

Vérifications que le correcteur fait au clavier, dans l'ordre : la ligne de total contre `=SOMME(Données[montant_ttc])` (égalité exigée) ; les lignes 480 contre le compte du tableau ; la part sommant à 100,0 % quand aucun segment n'est cliqué ; la colonne « moyenne par ligne » recalculée en `CA / lignes` catégorie par catégorie, et **non** comme la moyenne des sept moyennes — la seconde écriture vaut 98 281 FCFA et fait perdre le point de L3 en entier, parce qu'elle se présente joliment.

Le TCD par mois, lui, attend douze lignes de 40 lignes chacune (la régularité est celle de l'extrait, pas du magasin) et un CA qui court de 1 258 801 FCFA (mai) à 5 292 517 FCFA (dé-

cembre). Les quatre graphiques attendus : barres horizontales du classement par CA, courbe du CA mensuel avec la glissante à trois mois en deuxième série, barres empilées ou camembert **avec la mention que deux parts voisinent à 0,6 point** (18,1 % et 17,5 %) si le candidat a choisi le camembert, et un quatrième au choix — le top 5 des produits est l'option la plus tenue :

produit	CA (FCFA)	lignes
Robinet mitigeur évier — réf 1	1 976 000	14
Robinet mitigeur évier — réf 2	1 794 026	13
Lame de lambris 4 m — réf 2	1 529 811	11
Lavabo céramique — réf 1	1 485 299	10
Régulateur de charge 10 A — réf 1	1 369 527	12

Les trois segments (ville, catégorie, mois) doivent être connectés aux **deux** TCD : c'est le réglage « Connexions de rapports », et c'est le point de barème le plus souvent perdu en L3, parce qu'il est invisible jusqu'à ce qu'on clique. Le ratio `CA / objectif` attend 18,2 % sur l'année de l'extrait, et les valeurs mensuelles les plus hautes autour de 27,9 % en décembre contre 14,1 % en octobre : le bon rendu écrit à côté, dans la même zone, que ce ratio rapporte quatre jours de vente à un mois de cible.

3.4 — L4 : noms, formats, protection

Cinq noms au minimum, et deux sont contrôlés par le correcteur par leur **usage** : `Debut_période` et `Fin_période` doivent apparaître dans les critères des `SOMME.SI.ENS` et dans la source des segments de chronologie ; si les dates des graphiques sont figées en dur, le point n'est pas acquis même si les noms existent. `Contrôle_total` désigne la cellule d'égalité (attendu 0), et `TCD_ca_catégorie` la plage du TCD — utile pour l'export et pour la mise en forme conditionnelle, qui ne doit colorer aucun chiffre non contrôlé.

Protection : la feuille `Pilotage` est verrouillée, **les segments restent utilisables**, l'onglet `Données` → `Actualiser` tout fonctionne, la structure du classeur empêche de supprimer `Calculs`. Trois formats à vérifier à l'œil : montants sans décimale avec le code FCFA, parts à une décimale, JJ/MM/AAAA aux deux bornes de période. Le classeur s'ouvre sur `Pilotage` (dernière feuille active enregistrée, ou macro d'ouverture — ici admise, parce qu'elle ne touche pas à la donnée).

3.5 — L5 : la note, et les trois phrases qui font la différence

1. **L'échelle.** « Le fichier d'entraînement couvre les jours 1 à 4 de chaque mois de 2025 : 25,6 % des lignes tombent un samedi ou un dimanche, 40 lignes par mois exactement, 240 jours ouverts sur 336 jours courus. Aucune saisonnalité ne peut être affirmée sur cette base, et le ratio à l'objectif non plus. » Le correcteur cherche la mention des quatre jours : sans elle, deux points sont perdus, avec elle le reste se discute.
2. **Le dénominateur.** 75 152 FCFA par ligne, 114 156 FCFA par ticket, 150 305 FCFA par jour ouvré, 107 361 FCFA par jour couru : quatre chiffres justes, et la note doit dire lequel elle affiche et pourquoi. La mention « par jour travaillé » sans précision des week-ends (9 730 140 FCFA de CA, 27,0 % de la masse) est comptée comme une absence.
3. **La voie.** Ce qui est écrit en fonctions dynamiques (`FILTRE`, `UNIQUE`, `TRIERPAR`, `SEQUENCE`, `LET`), ce qui est en voie accessible, et ce qui se passe sur les postes du service : `#NOM?` en Excel 2019 pour les dynamiques, pas de propagation sur LibreOffice 24.8 et antérieurs sans pré-allocation de la plage, pas de modèle de données sur Excel pour Mac (donc pas de « Nombre distincts »), pas de segments du tout dans Calc. Le correcteur attend la phrase qui dit lequel des deux il a livré et pourquoi, pas une liste d'excuses.

3.6 — L6 : la rejouabilité, testée comme elle le sera

Le correcteur fait ceci, en trois minutes : il ouvre le classeur, supprime la feuille `Données` et les TCD, clique `Données` → `Actualiser tout`, puis vérifie quatre nombres — 480 lignes, 36 073 185 FCFA, 12 mois, 0 à la cellule de contrôle. Quatre défauts type, et leur traduction en points :

Défaut observé	Diagnostic	Conséquence au barème
l'actualisation redemande le chemin du fichier	paramètre <code>absent</code> , chemin en dur dans l'étape Source	L1 amputé de 2 pts, L6 de 1
le TCD revient vide	la source pointait une plage supprimée avec la feuille	L6 à 0, L3 plafonné à 2
le total revient à 36 339 317 FCFA	les doublons étaient retirés à la main dans la feuille, pas dans la requête	L1 à 1, L6 à 1
le ratio d'objectif passe à « #DIV/0! »	la jointure <code>annee_mois</code> a été perdue en rechargeant le CSV	L3 à 2, L5 amendée

Un dossier qui montre le message d'erreur attendu quand le fichier source a bougé (« chemin introuvable », avec la correction en deux lignes) vaut le point entier de L6 : c'est la seule preuve que le livrable est compris et pas seulement produit.

4. Grille d'auto-évaluation avant rendu

Répondez par oui ou non ; trois « non » = ne rendez pas, reprenez la feuille concernée.

#	Question	Si non
1	La requête repart-elle bien du CSV brut et rend-elle 480 lignes ?	reprendre M03.C08 §5.5
2	Les doublons sont-ils retirés sur <code>n_ticket</code> et produit, après un tri explicite ?	reprendre M03.C03 §5.4 et M03.C08 ex. 8.4
3	Les montants sont-ils numériques dans la table finale (<code>=NB()</code> = 480) ?	reprendre M03.C02 §5.2
4	La colonne date est-elle une série, pas du texte (<code>=MOIS()</code> fonctionne) ?	reprendre M03.C07 §5.1
5	Les huit fiches sont-elles exécutées, pas décrites ?	reprendre le format de fiche, §3 des chapitres
6	Chaque TCD affiche-t-il un total égal à la somme de la colonne source ?	reprendre M03.C08 §5.1
7	Les trois segments sont-ils connectés aux deux TCD ?	reprendre M03.C08 §5.3
8	Le graphique reprend-il les filtres cachés du TCD au moment de sa création ?	reprendre M03.C08 §5.4
9	Les parts sont-elles en « Total général en % » et non calculées à la main ?	reprendre M03.C08 §5.1
10	Le ratio à l'objectif est-il accompagné de la mention d'échelle ?	reprendre M03.C07 §6
11	La protection laisse-t-elle actualiser et cliquer ?	reprendre M03.C08 §5.8
12	Un autre poste peut-il ouvrir le classeur sans <code>#NOM?</code> ni plage vide ?	écrire la voie alternative, M03.C07 §5.7

5. Ce que le correcteur regardera en premier

1. **Le compte de lignes de la requête, à l'étape 4.** 480, pas 468 ni 316 : c'est la ligne qui dit si le doublon a été compris comme un défaut de fichier ou comme une règle métier.

2. **La cellule de contrôle.** `Controle_total` doit être une formule, pas un 0 écrit. Un zéro tapé est un aveu, pas une preuve.
3. **Le segment qui ne fait rien.** Premier clic sur un segment : si un seul des deux TCD bouge, le candidat n'a pas ouvert « Connexions de rapports », et le tableau de bord est décoratif.
4. **La phrase d'échelle.** Dans la note ou sous le graphique, « les jours 1 à 4 de chaque mois » doit se lire. C'est le seul endroit du projet où l'honnêteté du module M02 est vérifiable dans un livrable Excel.
5. **La liste des voies.** Un classeur qui n'utilise que des fonctions que deux postes sur cinq peuvent exécuter n'est pas un livrable, c'est une démonstration. À l'inverse, un classeur 100 % accessible avec une ligne qui dit ce que `FILTRE` aurait économisé est noté comme un travail de professionnel.

Évaluation M03 — « Excel, du clic à la méthode »

Module M03 · durée totale 2 h 30 · quatre épreuves · seuils : quiz 11/15, étude de cas 12/20, projet 13/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	vérifier qu'on a lu les huit chapitres avant de se tromper cher
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, dont trois sur l'identification d'une erreur affichée
C · Exercices pratiques	50 min	/20 (auto-corrigé)	oui	quatre exercices menés jusqu'au chiffre sur le classeur de l'atelier
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier en quatre pages : un tableau de bord à déclarer pilotable ou non

Le score du module se lit ainsi : **B ≥ 11** et **D ≥ 12** ouvrent le module 4 ; le projet M03.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D : savoir ce que fait **FIN.MOIS** ne dispense pas de dire ce que vaut le ratio affiché dans un comité.

A · Questions de récupération (non notées)

À traiter sans document, en une phrase chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail de syntaxe.

- Pourquoi `=SOMME()` sur une colonne de montants stockés en texte rend-il 0 au lieu d'une erreur, et pourquoi est-ce plus grave ?
- Que devient `=DATE(25;1;1)`, et quel est le réglage qui rend ce piège impossible ?
- Une date « 15/03/2025 » dans une cellule : citez deux tests qui disent s'il s'agit d'un nombre ou d'un texte.
- Pourquoi un `RECHERCHEV` est-il plus dangereux qu'un `INDEX/EQUIV` quand on insère une colonne, et que répond `RECHERCHEX` à ce problème ?
- Un TCD affiche un total inférieur à la somme de la colonne source : listez trois causes, pas une.
- Que signifie `#PROPAGATION!`, et pourquoi n'est-ce pas une erreur de formule ?
- Pourquoi un segment ne change-t-il rien à une cellule `=SOMME(...)` écrite à côté du TCD ?
- « 75 152 FCFA par ligne » et « 114 156 FCFA par ticket » : lequel répondre à « combien vend-on par ticket » ?
- Qu'est-ce qu'une macro enregistrée ne sait pas faire, et que sait faire une requête Power Query à sa place ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

reur #N/A, propagée aux calculs qui lisent la cellule · c) la ligne précédente · d) le texte introuvable entre guillemets

Q14. Un TCD construit sur une **plage** A1:M481 puis 20 lignes ajoutées au tableau source : à l'actualisation, le TCD rend : a) 500 lignes · b) 480 lignes : la plage ne s'est pas étendue · c) une erreur de source · d) 480 lignes et un avertissement rouge

Q15. Dans un TCD, « Nombre » sur le champ `client` et une mesure `=DISTINCT-COUNT(ventes[client])` dans le modèle : a) les deux rendent 480 · b) 480 d'un côté, 372 de l'autre · c) 372 des deux côtés · d) le second exige Excel 2019

C · Exercices pratiques (4 exercices · 20 points)

Matériel autorisé : le classeur `m03_classeur_atelier.xlsx` et les CSV du socle. Barème : 12 points sur E1 à E3, 8 points sur E4 ; la restitution des formules fait partie du point, pas une annexe.

#	Exercice	Pts
E1	Formats, types, validation : rendre la colonne montant et la colonne date utilisables	4
E2	Agrégations à critères et TCD : retrouver six chiffres, en expliquer un	4
E3	Recherches et jointures : le référentiel, les 103 orphelins, la clé composée	4
E4	Le classeur aux trois chiffres faux : diagnostiquer et corriger	8
Total		20

E1 (4 pts). Sur `ventes_brutes` : convertissez `montant_ttc` en nombre et `date` en série de dates, en écrivant une formule par ligne (voie accessible) puis la requête Power Query équivalente (voie moderne). Contrôles exigés : compte numérique 480, `MAX(date) - MIN(date) = 335` jours, somme 36 073 185 FCFA. Ajoutez une validation des données qui refuse une quantité supérieure à 500 et expliquez pourquoi la borne est 500 et non 68.

E2 (4 pts). Sans TCD, puis avec un TCD, retrouvez : le CA de la catégorie Plomberie, le nombre de lignes remiseses en Peinture, la moyenne par ligne d'Electricité, la part du CA de Bois & panneaux, le CA de mai, le CA de décembre. Six valeurs, une formule chacune. Expliquez en trois lignes pourquoi la « moyenne des moyennes par catégorie » n'est pas la moyenne par ligne, chiffres à l'appui.

E3 (4 pts). Rapportez à chaque ligne de l'extrait la catégorie du produit, le segment du client et la ville du magasin, d'abord avec `RECHERCHEV`, puis avec `RECHERCHEX`, puis avec une relation du modèle de données. Contrôles : les lignes sans client utile doivent être comptées et nommées ; la jointure aux objectifs se fait sur une clé composée que vous fabriquez ; un test doit montrer ce qui casse si une colonne est insérée dans le référentiel.

E4 (8 pts) — le classeur aux trois chiffres faux. Un analyste a rendu le classeur ci-dessous ; il est interne, propre, et contient trois chiffres faux.

Ce que rend le classeur	Contexte affiché	Ce que le fichier contient
Chiffre d'affaires 2025 : 36 339 317 FCFA	<code>=SOMME(ventes!M2:M490)</code> sur la feuille reçue	489 lignes, dont neuf doublons de ticket
Panier moyen : 98 281 FCFA	<code>=MOYENNE()</code> sur une colonne de moyennes par catégorie	la moyenne par ligne vaut 75 152, le panier par ticket 114 156
Clients servis : 480	<code>=NBVAL()</code> sur la colonne client du tableau de lignes	316 tickets, 372 clients distincts
Réalisation décembre : 27,9 %	ratio du CA de décembre sur l'objectif mensuel	pas faux, mais lu à tort comme une performance

Pour chacun des trois premiers : expliquez la cause, écrivez la formule de contrôle qui l'aurait révélée, et donnez la valeur attendue. Pour le quatrième : dites en deux lignes pourquoi ce n'est pas une erreur de calcul et ce qu'il faut écrire sous le graphique. Bonus (non compté) : la mise en forme conditionnelle qui aurait alerté sur le premier.

D · Étude de cas — « Le tableau de bord du magasin 5 est-il pilotable ? » (20 points, seuil 12)

Le dossier. Le responsable régional transmet un classeur qu'il utilise chaque lundi, et un courriel de trois lignes : « Le magasin 5 est à 18,2 % de son objectif annuel en 2025. Décembre est à 27,9 %, octobre à 14,1 %. Je félicite en décembre, je recadre en octobre. Dis-moi si le fichier tient la route avant que j'envoie ça. »

Le classeur reçu contient : une feuille **Données** (480 lignes, collées depuis un CSV, montants et dates en texte, triest effectués à la main), une feuille **Calculs** (neuf colonnes utilitaires dont trois **RECHERCHEV** sur des plages A:F figées, une moyenne glissante **=MOYENNE(B2:B13)** recopiée depuis la ligne 2), une feuille **Pilotage** (deux TCD pointant sur la plage **Données!A1:M481**, un segment connecté à un seul TCD, un graphique en camembert de sept parts, un ratio en colonne **=C2/D2** sans test de dénominateur), et une macro **Reparer** enregistrée qui supprime des lignes et met en forme douze cellules. Le fichier est **.xlsx**, non protégé, et un onglet caché porte une copie des montants en dur.

Questions, à traiter dans cet ordre, en quatre pages maximum.

1. Le chiffre de 18,2 % est-il exact, et dit-il ce que le responsable croit ? Reprenez les deux nombres (CA de l'extrait, objectif du magasin 5 sur 2025), refaites la division, et énoncez ce que mesure ce ratio compte tenu de la couverture réelle des lignes. Donnez les deux ratios mensuels cités, et la phrase qui doit accompagner le graphique.
2. Que devient ce pilotage sur la population entière des ventes (pas l'extrait) ? Chiffrez le CA annuel du magasin 5, son ratio à l'objectif, et l'étendue des ratios magasin-mois observés dans le fichier d'objectifs. Concluez sur la comparabilité des magasins : ce que la table d'objectifs permet et ne permet pas d'affirmer.
3. Listez, dans l'ordre de gravité, les sept défauts du classeur qui empêchent de le déclarer pilotable, et pour chacun la correction au format d'une ligne de procédure (quoi faire, dans quel outil, quel contrôle à l'appui). Trois défauts au moins doivent concerner ce qui casse à l'actualisation, pas la présentation.
4. Rédigez la réponse au courriel, en six lignes, avec les deux chiffres à ne pas envoyer tels quels et la proposition de dispositif : ce qui reste dans le tableur, ce qui passe par une requête, ce qui est à sortir d'Excel. Vous indiquez en une phrase ce que vous écrivez au sujet de la macro **Reparer**.

Éléments de réponse attendus

1. Le ratio est arithmétiquement juste : 36 073 185 FCFA sur 198 030 000 FCFA d'objectif, soit 18,2 %. Il ne dit rien d'une performance annuelle : l'extrait ne couvre que les jours 1 à 4 de chaque mois, à raison de 40 lignes par mois, et 103 lignes sur 480 portent un client qu'on ne peut pas relier au référentiel. Les ratios mensuels cités (14,1 % en octobre, 27,9 % en décembre) comparent la même tranche de quatre jours à un mois de cible : la mention à écrire sous le graphique est celle de la feuille **Mensuel** du classeur d'atelier — le ratio mesure un extrait, pas l'année. Une réponse qui corrige le pourcentage sans corriger l'échelle est comptée à moitié.

2. Sur la population : 528 792 330 FCFA de chiffre d'affaires pour le magasin 5 en 2025, soit 267,0 % de l'objectif — un ratio supérieur à 100 % qui, lui non plus, ne veut rien dire tant que la table d'objectifs n'a pas été lue (218 lignes, un objectif par magasin et par mois, des trous : deux mois de l'extrait sont sans cible). Les ratios magasin-mois s'étagent de 1,62 à 3,27 (médiane 2,31), soit un facteur deux entre le meilleur et le pire magasin sur cette base : la comparaison brute des pourcentages de réalisation est donc **un classement des objectifs, pas des résultats**, tant que le mode de construction de la cible n'est pas connu. Ce qu'il faut demander : la méthode d'élaboration des objectifs et le taux d'ouverture mensuel par magasin.

3. **Les sept défauts, dans l'ordre où ils font perdre confiance.** (a) TCD pointant sur une plage figée : à l'actualisation, les lignes 482 à 500 restent dehors, silencieusement ; correction = sourcer sur le tableau structuré, contrôle = total du TCD contre somme du tableau. (b) Les montants en texte : =SOMME() sur la feuille reçue rend 36 339 317 FCFA au lieu de 36 073 185, et sur une plage entièrement texte elle rend 0 sans erreur ; correction = conversion dans la requête, contrôle = =NB(colonne) attendu 480. (c) Dates en texte avec deux écritures : le regroupement par mois produit 36 groupes au lieu de 12, le tri alphabétique masque l'erreur ; correction = colonne date typée et mois calculé. (d) RECHERCHEV sur plages figées : une colonne insérée dans le référentiel et les numéros d'index décalent les libellés, sans erreur affichée ; correction = RECHERCHEX ou relation du modèle. (e) Moyenne glissante =MOYENNE(B2:B13) recopiée depuis la ligne 2 : les onze premiers points ne sont pas des moyennes à douze mois ; correction = garde-fou =SI(NB(...)=12;...) et deux premières cellules laissées vides. (f) Ratio =C2/D2 sans test de dénominateur : deux lignes de l'extrait n'ont pas de cible, le classeur affiche #DIV/0! et le camembert ampute les parts ; correction = SIERREUR avec un libellé, plus une ligne « cible absente » dans la note. (g) Macro Reparer qui supprime des lignes : non annulable, adressée en dur, silencieuse si la forme change, et .xlsm refusé par beaucoup de postes ; correction = remplacer par une étape de requête, et ne garder la macro que pour l'export. Le segment connecté à un seul TCD, le graphique en camembert de sept parts voisines et la feuille cachée en dur sont trois défauts de plus, comptés comme secondaires.

4. Modèle de réponse : « le fichier est exact dans ses calculs, faux dans ce qu'il autorise à conclure. 18,2 % n'est pas une réalisation annuelle mais le poids de quatre jours par mois rapportés à une cible mensuelle ; décembre à 27,9 % et octobre à 14,1 % ne se comparent pas non plus. Je propose : nettoyage par requête (six étapes, rejouées), TCD sur le modèle de données, un segment connecté aux deux tableaux, un avis de prudence sur le ratio tant que la fabrication des objectifs n'est pas connue. La macro Reparer doit être retirée du circuit : elle détruit des lignes sans trace et ne sera pas utilisable sur les postes du service. » Toute recommandation de sanction ou de félicitation maintenue sur ces bases fait perdre le point de conclusion.

Corrigé du quiz (B)

Q	Réponse	Une ligne de justification
1	b	l'espace de milliers non reconnu à la saisie laisse du texte ; le format ne convertit rien
2	c	SUBSTITUE retire l'espace, CNUM convertit ; b) fabrique du texte, d) convertit 0
3	b	la comptabilité aligne symboles et décimales sur une colonne dédiée
4	a	le message par défaut bloque et propose la liste autorisée
5	a	la formule se recopie sur les 480 lignes : chaque cellule est juste, la colonne sommée vaut 17 315 128 800 FCFA, un chiffre sans sens et sans erreur
6	c	un index hors plage se signale #REF! ; les trois autres produisent bien un #N/A
7	b	#PROPAGATION! = la zone d'arrivée est occupée, y compris par un tableau structuré
8	b	les dynamiques sont en 365, 2021 et 2024 : en 2019, #NOM?

Q	Réponse	Une ligne de justification
9	a	<code>=NB()</code> ignore le texte et compte les nombres : un mélange de types, pas une absence de dates
10	a	borne basse inclusive, haute exclusive, aucune mention du 29
11	b	<code>FIN.MOIS</code> rend un nombre ; c'est le format date qui l'affiche en jour
12	b	sans troisième argument, les fériés comptent comme travaillés
13	b	<code>#N/A</code> propagé : c'est le quatrième argument (valeur si non trouvée) qui manque
14	b	la plage ne s'étend pas ; 480 lignes, sans un mot
15	b	« Nombre » compte des lignes ; le distinct exige le modèle de données

Seuils de lecture du quiz. 13 et plus : les mécanismes sont acquis. 11-12 : le candidat confond encore types et formats, reprendre C02 et C07. 10 et moins : reprendre C02, C03, C04 en mini-projets, puis une nouvelle version de B. Les questions 6, 10 et 15 sont éliminatoires dans ma pratique : un analyste qui ne sait pas ce que fait une borne de période écrira un faux chiffre, et un faux chiffre signé, c'est tout le module.

Éléments de corrigé des exercices (C)

E1. Quatre formules de contrôle, dans cet ordre : `=NB(ventes[montant_ttc])` → 480 ; `=MAX(date) - MIN(date)` → 335 jours (et 0 si la colonne est restée texte) ; `=SOMME(ventes[montant_ttc])` → 36 073 185 FCFA ; `=MAX(quantite)` → 68 après correction d'unité contre 14 000 dans le fichier reçu. La borne 500 se justifie par l'échelle du métier : 7 lignes dépassaient 500 pièces, toutes issues d'un oubli de conversion d'unité, et non parce que 68 serait un plafond physique — la formulation juste est « seuil d'alerte, pas seuil de vérité ».

E2. Plomberie 8 754 982 FCFA ; Peinture 8 lignes remisées ; Electricité 119 088 FCFA par ligne ; Bois & panneaux 13,4 % du CA ; mai 1 258 801 FCFA ; décembre 5 292 517 FCFA. L'explication attendue : la moyenne des sept moyennes de catégories vaut 98 281 FCFA, la moyenne des 480 lignes 75 152 FCFA, et l'écart vient de la taille des groupes — chaque catégorie ne pèse pas le même nombre de lignes. En tableur : `=MOYENNE(plage_de_ca)/NB(plage)` n'est pas la moyenne des `=MOYENNE.SI.ENS(...)`, et le TCD qui agrège une colonne de moyennes reproduit exactement l'erreur.

E3. Trois valeurs de contrôle : les lignes rapportées (480), les 103 lignes sans client exploitable — dont 85 au client comptoir 0 et 18 au champ vide — et la clé `annee_mois` qui joint à 218 lignes d'objectifs, deux lignes de l'extrait restant sans cible. La voie `RECHERCHEV` casse si on insère une colonne dans `produits` (les index de retour décalent, sans erreur) ; `RECHERCHEX` ne casse pas (il recherche la valeur de la clé, pas une position) ; la relation du modèle ne casse pas non plus et supprime trois colonnes de recopie.

E4. (i) 36 339 317 : neuf lignes en double, l'écart de 266 132 FCFA se lit en comparant à `=SOMME` sur le fichier nettoyé — la formule de contrôle est `=SOMME(ventes!M:M) - SOMME(Données!M:M)`, attendu 0. (ii) 98 281 : une moyenne appliquée à des moyennes, la valeur par ligne est 75 152 et le panier par ticket 114 156 ; contrôle = `=SOMME(ca)/NB(lignes)`. (iii) 480 : un `NBVAL` sur une colonne de lignes de vente, qui compte 480 lignes pour 316 tickets et 372 clients distincts ; contrôle = le nombre distincts via le modèle, ou une colonne `=1/NB.SI()` sommée. (iv) 27,9 % : juste, mais rapporté à quatre jours de vente sur un mois de cible — la correction est éditoriale, pas arithmétique, et c'est la raison pour laquelle le barème la compte à part. Les trois premiers valent 2 points chacun (1 pour la cause, 1 pour la formule de contrôle), le quatrième 2 points, la ligne de mise en forme conditionnelle 0,5 point de bonus.

Total et décision d'orientation

Composante	Score	Seuil	Statut
B · Quiz	... /15	11	bloquant
C · Exercices	... /20	indicatif (démarche restituée = +0,5)	—
D · Étude de cas	... /20	12	bloquant
Projet M03.P	... /20	13	bloquant

Décision. B, C et D au seuil, projet au seuil → module validé, passage en M04 (qualité, préparation et documentation des données, où le nettoyage devient une trace écrite ; le SQL en miroir s'ouvre en M05). Quiz seulement en dessous → deux jours pour reprendre C02, C04 et C07, puis une nouvelle version de B. Étude de cas seulement en dessous → nouvelle version de D sur le même dossier, avec un entretien de quinze minutes sur l'extrait de l'échelle. Deux échecs → reprise du module en commençant par les six exercices autonomes de C08, puis le projet.

Module M04 — Qualité, préparation et documentation des données

Phase 3 · Diagnostiquer, nettoyer, documenter : le socle entier comme matière

Analyse de données appliquée & Business Intelligence
8 partie(s) assemblée(s) · document de production

Table des matières

MODULE M04.C01 — LE DIAGNOSTIC EN DOUZE POINTS : SAVOIR CE QUI NE VA PAS AVANT DE TOUCHER À QUOI QUE CE SOIT	7
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple	8
4. Vocabulaire essentiel	8
5. Cours approfondi : les douze points	9
5.1 Point 1 — L'identité du fichier : taille, horodatage, encodage	9
5.2 Point 2 — La structure : lignes, colonnes, en-têtes	9
5.3 Point 3 — La complétude : qui manque, sur quelles lignes	9
5.4 Point 4 — L'unicité par la clé technique	10
5.5 Point 5 — L'unicité par la clé métier : là où le doublon se cache	10
5.6 Point 6 — La validité des types, cellule par cellule	10
5.7 Point 7 — La validité des domaines : valeurs autorisées	10
5.8 Point 8 — La cohérence interne : les égalités du fichier	11
5.9 Point 9 — La cohérence externe : les références qui cassent	11
5.10 Point 10 — L'actualité : la période annoncée contre la période trouvée	11
5.11 Point 11 — L'exactitude : la vérité est dehors	11
5.12 Point 12 — La traçabilité : et le rapport qu'on vous a donné, qui l'a mesuré ?	12
5.13 Les douze points, en une grille	12
6. Exemple concret : quarante-cinq minutes sur un fichier qu'on ne connaît pas	13
7. Démonstration pas à pas : produire la fiche sur le fichier de ventes	14
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	15
10. Exercice guidé — le trio qui décide du verdict (30 min, /10)	16
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M04.P1 — « La fiche de diagnostic du fichier reçu ce matin » (45 min)	17
14. Résumé du chapitre	18
15. À retenir	18
16. Évaluation formative (auto-correction, 8 min)	18
MODULE M04.C02 — LES ABSENCES : CINQ MÉCANISMES, CINQ TRAITEMENTS, ET LE COÛT DE CHAQUE CHOIX	20
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple	8
4. Vocabulaire essentiel	8
5. Cours approfondi : les cinq mécanismes, les cinq traitements	22
5.1 Mécanisme A — Le non-applicable : la valeur n'existe pas à l'état civil	22
5.2 Mécanisme B — Le non-collecté : on n'a jamais posé la question	22
5.3 Mécanisme C — Le non-répondu : la question a été posée, la réponse non	22

5.4 Mécanisme D — La perte à l'import ou à la conversion	22
5.5 Mécanisme E — L'absence déguisée en valeur tranquille	23
5.6 Traitements (1/2) : assumer, joindre	23
5.7 Traitements (2/2) : dériver, imputer, supprimer	23
5.8 Les cinq mécanismes et leurs chiffres, en une grille	24
6. Exemple concret : la phrase qui fait basculer un tableau de bord	25
7. Démonstration pas à pas : de l'œil au compteur, puis à la décision	25
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	15
10. Exercice guidé — trier quatre colonnes en quatre traitements (30 min, /10)	27
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M04.P2 — « Politique de valeurs manquantes » (1 h)	29
14. Résumé du chapitre	18
15. À retenir	18
16. Évaluation formative (auto-correction, 8 min)	18
 MODULE M04.C03 — DOUBLONS : LA CLÉ QUI MENT, LA CLÉ QUI DIT VRAI, ET LE DICTIONNAIRE DE CORRESPONDANCES	 32
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple	8
4. Vocabulaire essentiel	8
5. Cours approfondi	34
5.1 Les quatre niveaux où l'on peut chercher un doublon	34
5.2 Le faux doublon : ce que la répétition a de normal	34
5.3 Anatomie des 3 360 : ce que le fichier dit de son accident	34
5.4 La règle de garde : laquelle des deux lignes on conserve	35
5.5 Le registre des doublons, premier livrable de traçabilité	35
5.6 Les correspondances : quand deux écritures ne font qu'un objet	36
5.7 L'appariement des clients : la règle qui ne casse pas les homonymes	36
5.8 Les correspondances manquantes : 486 lignes et 372 clients	37
6. Exemple concret : le tableau croisé qui accuse un magasin	38
7. Démonstration pas à pas : trouver, décider, journaliser	38
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	15
10. Exercice guidé — reproduire le fichier propre, puis prouver que c'est le bon (35 min, /10)	40
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M04.P3 — « Le dossier de dédoublonnage » (1 h 30)	41
14. Résumé du chapitre	18
15. À retenir	18
16. Évaluation formative (auto-correction, 8 min)	18

MODULE M04.C04 — VALEURS ABERRANTES ET RÈGLES DE CONTRÔLE : UNE RÈGLE DOIT RENDRE UNE DÉCISION, PAS UN NOMBRE	44
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple	8
4. Vocabulaire essentiel	8
5. Cours approfondi	34
5.1 Les trois natures, comptées sur le fichier	46
5.2 Quatre familles de règles, et leur rendement mesuré	46
5.3 Ce que le fichier dit vraiment des 973 lignes	47
5.4 L'échelle de gravité, appliquée au socle	47
5.5 Une règle rend une décision : la forme à écrire	48
5.6 Calibrer un seuil : deux preuves, pas une habitude	48
5.7 Retourner une règle qui hurle : le cas des stocks	48
5.8 Les règles statistiques, remises à leur place	49
6. Exemple concret : la ligne à 75 662 kg	50
7. Démonstration pas à pas : la grille de contrôles en une heure	50
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	15
10. Exercice guidé — la règle de prix, de la distribution à la décision (35 min, /10)	51
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M04.P4 — « Le registre de contrôles du fichier de ventes » (1 h 30)	53
14. Résumé du chapitre	18
15. À retenir	18
16. Évaluation formative (auto-correction, 8 min)	18
MODULE M04.C05 — NORMALISER ET RENDRE COMPARABLE : CHANGER L'ÉCRITURE, JAMAIS LA VALEUR	56
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple	8
4. Vocabulaire essentiel	8
5. Cours approfondi	34
5.1 Trois objets, trois traitements — et ne pas les mélanger	58
5.2 Les montants texte : découper, puis prouver par une seconde voie	59
5.3 Mesurer le gain avant d'écrire la formule	60
5.4 Les dates : la norme est respectée, la réalité non	61
5.5 Unités et correspondances : ce qui demande un facteur, pas une formule	61
5.6 Importer sans rien casser : séparateur, décimales, encodage, en-tête	62
5.7 Les clés : normaliser pour que les jointures disent quelque chose	63
5.8 Ce qu'une normalisation ne doit jamais faire	63
6. Exemple concret : la cellule 16 638 FCFA	64
7. Démonstration pas à pas : le protocole de normalisation en dix étapes	65
8. Erreurs fréquentes	15

9. Bonnes pratiques professionnelles	15
10. Exercice guidé — reconvertir une colonne monétaire et le prouver (35 min, /10)	67
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M04.P5 — « Le protocole de normalisation » (1 h 30)	69
14. Résumé du chapitre	18
15. À retenir	18
16. Évaluation formative (auto-correction, 8 min)	18
MODULE M04.C06 — DOCUMENTER, NOMMER, PROTÉGER : LA DERNIÈRE ÉTAPE EST UNE ÉCRITURE, PAS UN CALCUL	71
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple	8
4. Vocabulaire essentiel	8
5. Cours approfondi	34
5.1 L'arborescence : trois zones, une règle d'écriture chacune	73
5.2 Le nommage : mesurer la convention avant de la proclamer	73
5.3 Le README de données : six sections, pas une de plus	74
5.4 Le journal des transformations : une ligne par opération, un compteur par ligne	75
5.5 Le contrôle final : un vert à 0 n'existe pas, et c'est une bonne nouvelle	75
5.6 Le périmètre des données personnelles : quatre fichiers, un risque mesuré	76
5.7 Anonymiser sans casser : l'ordre compte	77
6. Exemple concret : le journal du socle, complété	78
7. Démonstration pas à pas : la boîte à données en huit étapes	79
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	15
10. Exercice guidé — la boîte à données du socle (40 min, /10)	80
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M04.P6 — « La boîte à données » (1 h 30)	82
14. Résumé du chapitre	18
15. À retenir	18
16. Évaluation formative (auto-correction, 8 min)	18
PROJET M04.P — « LE DOSSIER DE DONNÉES DE LA QUINCAILLERIE DE KOUDOU-GOU »	84
1. Énoncé	84
Le matériel	84
Les six livrables	84
2. Ce qui est noté, ce qui est piégé	85
3. Barème et seuil	86
4. Correction des points d'attention	86

ÉVALUATION M04 — « QUALITÉ, PRÉPARATION ET DOCUMENTATION DES DONNÉES »	88
A • Questions de récupération (non notées)	88
B • Quiz (15 questions • 15 points)	89
Bloc 1 — Types et normalisation (Q1 à Q4)	89
Bloc 2 — Doublons et clés (Q5 à Q8)	89
Bloc 3 — Règles et valeurs aberrantes (Q9 à Q12)	89
Bloc 4 — Documentation et protection (Q13 à Q15)	90
Corrigé du quiz	90
C • Exercices pratiques (50 min • /20)	90
D • Étude de cas (60 min • /20, seuil 12) — « Le stock indiqué est-il crédible ? »	91
Lecture des résultats	92

Module M04.C01 — Le diagnostic en douze points : savoir ce qui ne va pas avant de toucher à quoi que ce soit

Outil de ce chapitre : le tableur et un éditeur de texte, sur le socle entier (243 360 lignes). Durée indicative : 6 h. Niveau : N2.

L'idée du chapitre. Un fichier n'est pas « propre » ou « sale » : il est **connu** ou **inconnu**. Le diagnostic est l'heure que personne ne veut payer et que tout analyste fini paie deux fois — une fois pour établir l'état réel, une fois pour réparer un total qu'il croyait bon. Ce chapitre donne une grille de douze points, une durée (45 minutes sur un fichier jamais vu), et un livrable : la fiche de diagnostic. Il se termine sur une démonstration qui dérange : le rapport de défauts livré avec notre propre socle est faux sur quatre de ses huit lignes, parce qu'il a été écrit d'après l'intention et non d'après le fichier.

Base de travail — le socle entier, pas un extrait. Quatre fichiers, tous lus sur disque : `01_socle_donnees/data/brut/ventes_brutes.csv` (28 516 920 octets, 243 360 lignes × 20 colonnes), `01_socle_donnees/data/brut/clients.csv` (23 912 lignes), `01_socle_donnees/data/brut/stocks_quotidiens.csv` (60 000 lignes), `01_socle_donnees/data/reference/verites_terrain.csv` (la cible : 240 000 lignes, 15 419 985 157 FCFA) — et, pour le point 12, `01_socle_donnees/data/reference/rapport_defauts.json`. Les chemins sont longs parce qu'ils sont réels : tout nombre de ce chapitre sort de ces fichiers, et le manuel les régénère à l'identique (graine 20260917).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **compter** avant de regarder : lignes, colonnes, clés distinctes, vides — et citer le dénominateur de chaque chiffre ;
- **classer** un défaut en six familles (complétude, unicité, validité, cohérence, exactitude, actualité) sans hésiter ;
- **exécuter** les douze points de la grille sur un fichier inconnu en moins de 45 minutes, au tableur seul ;
- **distinguer** un défaut qui dérange d'un défaut qui change la décision — en le chiffrant sur le total ;
- **rédiger** une fiche de diagnostic qu'un collègue peut rejouer sans vous demander aucune précision.

2. Pourquoi cette notion est importante

- Vous recevrez le fichier d'un autre. Pas le dictionnaire de données complet, pas la notice de fabrication, pas l'intention du collègue : un CSV, un export, un tableau collé dans un courriel. **Le premier acte utile est de le décrire, pas de le corriger.**
- Un nettoyage non diagnostiqué casse des totaux sans message. Sur notre socle, additionner la colonne des montants tels qu'ils arrivent rend 15 417 873 368 FCFA au lieu de 15 419 985 157 : **2 111 789 FCFA envolés, et aucune erreur affichée**. Le tableur a additionné ce qu'il savait additionner.

- L'inverse est vrai : « réparer » les montants texte sans traiter les lignes en trop rend 15 639 751 286 FCFA, soit **219 766 129 FCFA de trop**. Un défaut non vu dans le bon sens produit une erreur deux fois plus grosse que le défaut qu'on croyait corriger.
- Le métier appelle ça la due diligence sur la donnée : dans un audit, un appel d'offres, une reprise de dossier, c'est la pièce qui protège l'analyste. Un chiffre d'affaires contesté se défend avec une fiche de diagnostic, pas avec une capture d'écran.
- Enfin, le diagnostic est ce qui rend un travail **transmissible** : la fiche que vous laissez vaut plus que le fichier réparé que vous envoyez.

3. Explication simple

On ne répare pas une maison sans l'avoir inspectée, et l'inspection a un ordre : la charpente d'abord, la peinture ensuite. Le diagnostic d'un fichier, c'est la même chose, avec une règle unique : **à chaque question, un nombre et un dénominateur**.

Comparez deux façons de dire la même chose sur notre fichier de ventes :

Façon de dire	Ce que le lecteur peut en faire
« la colonne des montants a des soucis de texte »	rien — il faut vous rappeler pour savoir combien
« 3 421 montants sur 243 360 ne sont pas des nombres (1,4 %), et ils valent 2 111 789 FCFA »	tout — il sait si c'est grave, et peut vérifier

La première phrase est une impression. La seconde est une mesure. Douze points de mesure, exécutés dans l'ordre, et vous savez en trois quarts d'heure ce que le fichier contient — pas ce qu'il devrait contenir.

Le dernier des douze points est le plus facile à sauter, et le plus coûteux : **le rapport de qualité que vous avez reçu est-il mesuré, lui aussi ?** Si quelqu'un l'a écrit à la main depuis un tableau de spécifications, il décrit une intention. Nous allons le vérifier sur notre propre socle — et il est faux sur quatre lignes.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Diagnostic de qualité — Data profiling	Décrire l'état réel d'un fichier par des compteurs, avant toute correction.	Confondre profiler et nettoyer : profiler ne change aucune cellule.
Complétude — Completeness	— Une colonne est complète si chaque ligne concernée a une valeur.	Compter les vides sans dire quelles lignes sont concernées (une facture annulée n'a pas de date de paiement).
Unicité — Uniqueness	Une clé est unique si aucune valeur ne revient deux fois.	Croire que la clé primaire du fichier est la clé du réel : ici <code>id_vente</code> est impeccable et le doublon existe.
Clé métier — Business key	— Combinaison de colonnes qui, dans le monde, identifie une ligne (ticket + produit + quantité).	L'oublier : 3 440 lignes passent entre les mailles de la clé technique.
Validité — Validity	Une valeur est valide si elle a le bon type et appartient au domaine attendu.	Valider le format et non le contenu : une date au bon format mais en 2026-12 alors que l'historique s'arrête en août.
Cohérence — Consistency	— Deux expressions d'une même vérité ne se contredisent pas (<code>montant_ttc = montant_ht + montant_tva</code>).	Contrôler chaque colonne isolément et jamais les égalités entre elles.
Exactitude — Accuracy	— La valeur correspond au fait réel — vérifié contre une source indépendante.	Prendre la somme du fichier pour sa propre exactitude : la vérité est dehors (ici, <code>verites_terrain.csv</code>).

Français — English	Définition simple	Piège à éviter
Actualité Timeliness / Currency	— La donnée est à jour par rapport à la période qu'elle prétend couvrir.	Confondre « date présente » et « date plausible ».
Conformité Conformity	— La donnée respecte la règle de gestion écrite (remise en proportion, retour à montant négatif).	Ne pas avoir écrit la règle : sans texte, aucune violation n'est démontrable.
Fiche de diagnostic — Data quality report	Le document qui consigne les douze points, leurs nombres, et la décision prise pour chacun.	La rédiger après coup, de mémoire : elle ne rejoue plus.

5. Cours approfondi : les douze points

Les points vont du plus structurel au plus sémantique. Pour chacun : **la question, la procédure, et la réponse mesurée sur le socle** — parce qu'une grille sans exemple chiffré ne s'apprend pas.

5.1 Point 1 — L'identité du fichier : taille, horodatage, encodage

Avant d'ouvrir : combien d'octets, quelle date de modification, quel encodage. Un fichier de 28 516 920 octets ne se diagnostique pas à l'œil ; un fichier dont l'horodatage est antérieur à la période annoncée est suspect avant même la première ligne. L'encodage se teste, il ne se devine pas : notre fichier tarif fournisseur (01_socle_donnees/data/brut/tarif_fournisseur_peinture.csv) **ne se décode pas** en UTF-8 — il échoue à l'octet 73 — et contient 95 octets hors ASCII : c'est un cp1252.

Définition. Un **encodage** est la table qui associe un octet à un signe. UTF-8 en utilise un à quatre par signe, cp1252 exactement un. Lire un cp1252 comme de l'UTF-8 ne casse pas le fichier : ça remplace les accents par des caractères de substitution, et ça transforme « évacuation » en Ã©vacuation — un défaut introduit par la lecture, pas par le fichier.

5.2 Point 2 — La structure : lignes, colonnes, en-têtes

Le compte doit sortir du fichier, pas de son nom. Ici : **243 360 lignes de données, 20 colonnes**, dont les vingt noms sont `id_vente`, `id_ticket`, `date_vente`, `heure`, `id_magasin`, `id_vendeur`, `id_client`, `id_produit`, `quantite`, `prix_unitaire_ht`, `taux_remise`, `montant_ht`, `montant_tva`, `montant_ttc`, `mode_paiement`, `canal`, `est_retour`, `poids_kg`, `mois`, `annee` — et trois colonnes (`mois`, `annee`, `heure`) qui ne sont que des dérivées : leur présence doit être justifiée, sinon elles sont une source de divergence silencieuse entre l'original et le calcul.

5.3 Point 3 — La complétude : qui manque, sur quelles lignes

Un vide n'est un défaut que si la ligne est concernée. Sur le fichier de ventes, aucune colonne n'est vide au sens strict. Sur le fichier clients, **1 911 lignes sur 23 912 n'ont pas de ville**. La phrase correcte n'est pas « il manque la ville à 1 911 clients » mais « la ville est absente pour 1 911 clients sur 23 912, soit 8,0 % ; 0 ligne de `clients.csv` n'a de téléphone absent ». Deux colonnes, deux verdicts : ici le téléphone est systématiquement présent (23 912 lignes sur 23 912), l'e-mail presque jamais (absent sur 16 514 lignes).

Attention. Compter les vides après l'import, c'est compter ce que l'outil a fabriqué. Un 0 recopié par une règle de remplissage automatique, une chaîne vide lue comme NaN, une cellule texte « N/A » : trois absences différentes, trois compteurs différents, un seul qui corresponde à la réalité du fichier.

5.4 Point 4 — L'unicité par la clé technique

`id_vente` compte 243 360 valeurs distinctes pour 243 360 lignes : **zéro doublon**. Cette ligne de la grille doit être écrite avec son résultat, même quand il est bon — c'est elle qui innocente la clé, et sans elle le lecteur ne sait pas si vous l'avez regardée.

5.5 Point 5 — L'unicité par la clé métier : là où le doublon se cache

Et pourtant, le fichier propre contient 240 000 lignes : **243 360 – 240 000 = 3 360 lignes** ont été retirées. Aucune n'est un copié-collé intégral (0 ligne entièrement dupliquée), aucune ne porte un `id_vente` déjà vu. Elles se voient seulement sur la clé du monde réel — un même ticket, un même produit, une même quantité : **3 440 lignes** sont en collision sur ce triplet, et 8 453 lignes participent à une collision ticket + produit, dont 7 640 avec la même quantité. C'est le point le plus instructif du diagnostic : **la clé primaire d'un fichier n'est pas la clé de ce que le fichier décrit.**

Ce qu'on a testé	Test	Résultat sur le socle
clé technique	<code>id_vente</code> distincts ÷ lignes	243 360 ÷ 243 360 → 0 doublon
ligne intégralement identique	égalité colonne à colonne	0 ligne
clé métier (ticket + produit + quantité)	collision sur le triplet	3 440 lignes
écart à corriger	brut – propre	3 360 lignes

Définition. Une **clé candidate** est un jeu de colonnes dont les valeurs identifient une ligne et une seule. `id_vente` en est une ; le triplet ticket + produit + quantité en est une autre, **incompatible** avec la première : quand deux clés candidates se contredisent, l'une des deux n'est pas une clé, et c'est presque toujours celle que le fichier fournit déjà.

Dans les faits. Sur un poste de saisie, la même colonne a trois visages selon la machine : 56658 en nombre, 56 658 en texte à cause du séparateur de milliers, 56658 avec une espace venue d'un copier-coller. Le point 6 les compte ensemble ; le C05 les traite séparément, parce qu'ils ne se réparent pas de la même main.

5.6 Point 6 — La validité des types, cellule par cellule

La question n'est pas « de quel type est la colonne » mais « combien de cellules tiennent leur type ». Réponse : **3 421 montants montant_ttc sur 243 360 ne sont pas des nombres** ; `montant_ht`, `quantite`, `prix_unitaire_ht` et `poids_kg` sont numériques sur 100 % des lignes. Un seul défaut de type, sur une seule colonne, sur 1,4 % des lignes : c'est exactement le profil d'un accident d'import, pas d'une saisie désordonnée — et cela se traite donc en amont (C05), pas ligne à ligne.

5.7 Point 7 — La validité des domaines : valeurs autorisées

Un domaine est une liste, et une liste se compare. `mode_paiement` a 5 modalités (Chèque, Créance 30 j, Espèces, Mobile Money, Virement), `canal` en a 3 (Livraison, Magasin, Téléphone) : dans les deux cas, **aucune valeur hors liste**. Le domaine est aussi numérique : `remise` est définie comme une proportion entre 0 et 1 ; or **973 lignes portent une valeur supérieure à 1**, avec un maximum lu à 30 — ce sont des pourcentages saisis en points (30 = 30 %). Mais attention à la conclusion hâtive : divisées par 100, ces valeurs ne réparent rien — **258 lignes** sortiraient encore du domaine contractuel (remise maximale admise : 0,25), et les montants, eux, restent cohérents. La colonne `taux_remise` n'est pas une proportion mal échelonnée : sur ces lignes, elle ne décrit plus le prix payé.

5.8 Point 8 — La cohérence interne : les égalités du fichier

Trois égalités découlent des règles de gestion publiées dans la notice du socle : `montant_ht = quantité × prix × (1 - remise)`, `montant_ttc = montant_ht + montant_tva`, et « une ligne marquée `est_retour = 1` a un montant négatif ». Mesure : **973 lignes** violent la première — exactement les 973 lignes à remise aberrante —, **0** la deuxième (la TVA rend 18 % du montant HT sur les 243 360 lignes, au franc près), et la troisième : sur 2 848 retours signalés, 2 801 ont un montant négatif et **47** n'ont pas de montant du tout (c'est le point 6 qui les a happés). Le détail important est au § 5.7 : les 973 lignes incohérentes ont un `montant_ht` juste et une `taux_remise` faux — le défaut ne casse donc **aucun** total, il casse uniquement les analyses de remise.

Définition. Un **contrôle croisé** est une vérification qui porte sur deux colonnes au lieu d'une. Il attrape les défauts qu'aucun contrôle unitaire ne voit : une remise à 30 est « une valeur plausible » si on la regarde seule, et une impossibilité dès qu'on la multiplie par le prix.

5.9 Point 9 — La cohérence externe : les références qui cassent

Une clé étrangère doit pointer quelque part : **486 lignes de vente** portent un `id_client` absent du fichier clients, sur **372 clients distincts** inconnus. C'est le point dont le coût est le plus mal estimé : 486 lignes ne disparaissent pas, elles disparaissent du rapprochement. Toute jointure avec les clients perd ces lignes sans message, et tout tableau croisé construit sur la jointure affiche un chiffre d'affaires minoré — sans que la moitié des lignes concernées aient le moindre défaut apparent. (Le nombre de tickets distincts — 146 161 — sert ici de contrôle d'ordre de grandeur, pas de verdict.)

Attention. Un orphelin n'est pas une ligne à supprimer. Les 486 lignes concernées sont des ventes réelles ; ce qui manque, c'est la référence. Effacer la ligne pour réparer la jointure, c'est effacer un chiffre d'affaires. On crée un client « à créer », ou on rattache à un compte générique explicitement nommé — et on l'écrit dans le journal des transformations (C06).

5.10 Point 10 — L'actualité : la période annoncée contre la période trouvée

La règle du socle est écrite : l'historique court de 2023-01-01 à 2026-08-31, et l'année 2026 est partielle. Mesure : la date maximale du fichier est **2026-12-06** et **51 lignes** dépassent le 31/08/2026. Cinquante lignes sur 243 360 sont invisibles à l'œil, mais suffisent à faire dire à une comparaison annuelle ce qu'elle ne doit pas dire : « 2026 dépasse 2025 ». Le contrôle d'actualité n'est pas une question de propreté, c'est une question de **date de coupure déclarée** — et cette date doit figurer sur la fiche.

5.11 Point 11 — L'exactitude : la vérité est dehors

Les dix premiers points interrogent le fichier sur lui-même. Le onzième le confronte à une source indépendante. Ici, `verites_terrain.csv` affirme 240 000 lignes et 15 419 985 157 FCFA pour l'ensemble des ventes. Vérification, dans l'ordre où le nettoyage la rend vraie :

Contrôle	Valeur	Écart à la cible
somme naïve des <code>montant_ttc</code> du brut	15 417 873 368	– 2 111 789
somme après réparation des textes, sans dédoublonner	15 639 751 286	+ 219 766 129
cible (<code>verites_terrain.csv</code>)	15 419 985 157	—

Retenez la forme du résultat, pas les nombres : **le total n'est pas un indicateur de propreté, c'est un indicateur d'écart**, et il ne se rétablit que si les deux défauts sont traités. Additionner sans réparer sous-évalue ; réparer sans dédoublonner sur-évalue de cent fois plus. C'est la démonstration que « ça a l'air bon » ne suffit pas.

5.12 Point 12 — La traçabilité : et le rapport qu'on vous a donné, qui l'a mesuré ?

Le socle est livré avec un `rapport_defauts.json` — la fiche que l'équipe de production a écrite pour le corrigé. Confrontons-la à la mesure :

Défaut annoncé	Annoncé	Mesuré sur le fichier	Verdict
montants en texte	3 360	3 421	sous-estimé de 61
dates au format mixte	2 640	0	inexistant dans ce fichier
lignes en double	3 360	3 360 (clé métier)	juste de nombre, faux de mécanisme
ville manquante aux clients	1 880	1 911	sous-estimé de 31
catégories de produits hétéroclites	61	44	sur-estimé de 17

Cinq lignes, trois erreurs, et un nombre qui tombe juste pour la mauvaise raison. La plus instructive est le 61 : il ne compte pas des lignes à corriger, il compte le **recouvrement** de deux défauts — 61 lignes sur les 3 421 sont aussi des lignes en trop. Un chiffre vrai, sorti d'un tableau de plan, rangé sous un mauvais libellé : exactement le défaut qu'aucun contrôle automatique ne signale. Ce n'est pas de la négligence : c'est ce qui arrive quand on recopie un **plan d'injection** au lieu de mesurer le fichier obtenu. Un défaut injecté au générateur peut se trouver compensé, annulé ou étendu par le reste de la chaîne.

Définition. On parle d'**intégrité référentielle** quand toute valeur d'une colonne de référence pointe une ligne existante de la table référencée. 486 lignes de notre fichier la violent ; comme une jointure perd silencieusement ce qu'elle ne retrouve pas, la violation ne se voit qu'au total, jamais à la cellule.

Conseil professionnel. Notez sur la fiche de diagnostic le nom du fichier, son empreinte de taille, la date de lecture, et la commande ou la formule qui a produit chaque nombre. Un diagnostic sans mode d'emploi est une opinion datée ; avec, c'est une preuve rejouable — et la différence se voit le jour où l'on vous conteste un chiffre.

À retenir. Les douze points ne se valent pas, et leur ordre non plus. Compter les lignes avant les vides évite de diviser par le mauvais dénominateur ; tester la clé technique avant la clé métier évite de conclure « propre » trop vite ; garder l'exactitude en avant-dernier évite de réparer dix défauts pour un total qui reste faux. Un ordre, c'est ce qui rend un diagnostic reproductible par quelqu'un d'autre — et rejouable six mois plus tard.

5.13 Les douze points, en une grille

#	Question	Ce qu'on compte	Ici
1	Identité	octets, date, encodage	28 516 920 o, cp1252 testé
2	Structure	lignes × colonnes	243 360 × 20
3	Complétude	vides par colonne	0 ici ; 1 911 villes absentes (clients)
4	Unicité (technique)	distincts ÷ lignes	243 360 ÷ 243 360
5	Unicité (métier)	collisions sur la clé réelle	3 440 lignes, 3 360 à retirer
6	Types	cellules hors type	3 421 montants texte

#	Question	Ce qu'on compte	Ici
7	Domaines	valeurs hors liste	0 hors liste ; 973 remises > 1
8	Cohérence interne	égalités entre colonnes	973 violations
9	Cohérence externe	clés orphelines	486 lignes, 372 clients
10	Actualité	bornes vs période annoncée	51 dates hors période, max 2026-12-06
11	Exactitude	total vs source indépendante	écart – 2 111 789 / + 219 766 129
12	Traçabilité	le rapport reçu est-il mesuré ?	3 lignes fausses sur 5 testées

Diagnostic de qualité — les douze points, dans l'ordre d'exécution

Chaque case : la question, le nombre, le dénominateur. Filet rouge = le point change le verdict.

1 identité structure 28 516 920 o · cp1252 testé	2 structure structure 243 360 lignes · 20 colonnes	3 complétude structure 0 vide au brut · 1 911 villes absentes
4 unicité (technique) clé 243 360 clés distinctes · 0 doublon	5 unicité (métier) clé 3 440 collisions · 3 360 à retirer	6 types valeur 3 421 montants en texte
7 domaines valeur 973 remises > 1 · max lu 30	8 cohérence interne valeur 973 lignes · les mêmes qu'au point 7	9 cohérence externe valeur 486 lignes · 372 clients inconnus
10 actualité temps 51 dates après le 31/08/2026	11 exactitude décision écart -2 111 789 puis +219 766 129	12 traçabilité décision rapport fourni faux sur 3 lignes sur 5

Trois points décident du nettoyage :

5 · la clé métier (le doublon invisible à id_vente) · 6 · les types (les montants texte)
 11 · l'exactitude (le total contre source extérieure). Sur le socle : 3 421 + 3 360 - 61
 = 6 720 lignes à traiter, soit 2,8 % du fichier. Les neuf autres points changent la confiance.

6. Exemple concret : quarante-cinq minutes sur un fichier qu'on ne connaît pas

Le contexte est celui d'un lundi matin : la direction commerciale envoie `ventes_brutes.csv`, trois ans d'historique, sans autre commentaire que « tout est dedans ». Minutage réel d'un diagnostic complet, tableur ouvert :

Minutes	Acte	Sortie
0-3	taille, horodatage, ouverture de la première ligne	cp1252, 28 516 920 octets, séparateur ; ou , confirmé
3-8	import en forçant les types de chaque colonne	243 360 lignes × 20 colonnes dans la feuille
8-14	comptes : lignes, clés distinctes, vides	points 2, 3, 4
14-22	types et domaines (NB contre NBVAL, listes de valeurs)	points 6, 7
22-30	collisions de clé métier (tableau croisé sur le triplet)	point 5
30-36	égalités internes et jointures externes	points 8, 9
36-40	bornes de dates, total contre source indépendante	points 10, 11

Minutes	Acte	Sortie
40-45	fiche de diagnostic : douze lignes, un nombre et une décision par ligne	livrable

Trente-six minutes dans M01-M03 s'appelaient « nettoyer ». Ici, quarante-cinq minutes ne changent **aucune cellule** : à la fin, le fichier est identique et vous êtes capable de dire ce qui ne va pas, de combien, et ce que ça coûte sur le total. C'est le renversement que ce module attend de vous.

7. Démonstration pas à pas : produire la fiche sur le fichier de ventes

On écrit les douze contrôles comme formules de tableur, sur la feuille `Ventes` (243 360 lignes, sous la limite de 1 048 576 lignes d'une feuille — si votre fichier dépasse la limite, voir § 5.1 et le point 1 : c'est déjà une information de diagnostic, et M05 est là pour ça).

1. **Feuille de contrôle.** Une feuille `DIAGNOSTIC`, trois colonnes : point, nombre, décision. Douze lignes.
2. **Structure.** `=NBVAL(Ventes!A:A)-1` → 243 360 lignes de données ; `=NBVAL(Ventes!$A$1:$T$1)` → 20 colonnes.
3. **Complétude, par colonne.** `=NBVAL(Ventes!N2:N243361)-NB(Ventes!N2:N243361)` : cellules remplies moins cellules comptées comme nombres ; sur `montant_ttc` (colonne N), la soustraction rend 3 421. Consignez le dénominateur à côté : sans lui, le nombre ne dit rien à un repreneur.
4. **Unicité technique.** `=SOMMEPROD(1/NB.SI(Ventes!A:A;Ventes!A2:A243361))` rend le nombre de valeurs distinctes d'`id_vente` : 243 360. Si `NBVAL` de la colonne égale ce nombre, la clé tient.
5. **Unicité métier.** Colonne utilitaire `=Ventes!B2&"|"&Ventes!H2&"|"&Ventes!I2` (ticket | produit | quantité), puis `=NB.SI(utilitaire;utilitaire)` compté en colonne 2 : les lignes où le compte dépasse 1 sont les 3 440 collisions. Un tableau croisé sur cette colonne (valeur : nombre d'articles) donne la même réponse en dix secondes, et c'est la voie que je recommande — le `SOMMEPROD` est là pour qu'on ne dépende pas du croisé.
6. **Types.** `=NB(Ventes!N2:N243361)` contre `=NBVAL(...)` : 239 939 contre 243 360. L'écart est le nombre de textes.
7. **Domaines.** Sur `taux_remise` : `=NB.SI.ENS(Ventes!K2:K243361;">1")` → 973. Sur les colonnes déclaratives, un tri à valeurs uniques confronte la liste obtenue à la liste attendue (5, puis 3 modalités).
8. **Cohérence interne.** Colonne utilitaire `=ARRONDI(Ventes!I2*Ventes!J2*(1-Ventes!K2);0)` et `=SI(ABS(...-Ventes!L2)>1;1;0)` sommée → 973. La même population que le point 7 : c'est la preuve que les deux défauts n'en font qu'un.
9. **Cohérence externe.** `=ESTNUM(RECHERCHEX(Ventes!G2;clients!A:A;clients!A:A;"absent"))` recopiée, puis comptée : 486 lignes, 372 `id_client` distincts en erreur.
10. **Actualité.** `=MAX(Ventes!C2:C243361)` et `=NB.SI(Ventes!C2:C243361;">"&DATE(2026;8;31))` → 2026-12-06, 51.
11. **Exactitude.** `=SOMME(Ventes!N2:N243361)` → 15 417 873 368, contre la cible 15 419 985 157 : l'écart se reporte en clair dans la colonne décision, avec le signe.
12. **Traçabilité.** Une cellule, une phrase : « valeurs mesurées le [date] sur [fichier], par les formules de cette feuille ; source de comparaison : [fichier ou personne] ».

Boîte à outils du chapitre. Les douze points tiennent dans une feuille-modèle : point, formule, nombre, dénominateur, décision, date de lecture. Cette dernière colonne est celle qui fait la différence entre un document et une preuve : elle permet de dire, un mois plus tard, si la mesure a été faite avant ou après le fichier que vous avez reçu.

8. Erreurs fréquentes

Symptôme visible	Cause réelle	Correction
« Le fichier est propre, id_vente n'a aucun doublon »	seule la clé technique a été testée	ajouter le point 5 : collisions sur la clé métier (3 440 lignes ici)
Le total obtenu après nettoyage dépasse la cible	un défaut traité en a révélé un autre (les textes réparés sans retirer les doublons)	ne jamais valider un nettoyage sur un seul écart : contrôler les deux signes, et la cible
Le rapport de défauts fourni ne correspond pas au fichier	il a été écrit depuis un plan, pas depuis une mesure	le traiter comme une hypothèse : le point 12 teste le rapport, pas seulement le fichier
Les pourcentages de vide varient d'une lecture à l'autre	les vides dépendent de l'import (0 remplacé, NaN fabriqué)	compter sur le fichier texte, avant import, puis comparer avec le compte après import
Un tri « à la main » sur la colonne des dates fait disparaître 51 lignes	elles étaient invalides pour l'outil mais lisibles	la date se contrôle par NB sur la colonne, pas par l'œil : point 6
Le diagnostic est fini « en dix minutes »	six des douze points ont été faits d'intuition	la fiche impose une ligne par point, même vide : un point non mesuré se voit

9. Bonnes pratiques professionnelles

- **Une ligne de fiche par point, sans exception.** Un point qui ne trouve rien s'écrit « 0 sur 243 360 » — c'est un résultat, et c'est le seul moyen de prouver que vous l'avez cherché.
- **Toujours le dénominateur.** « 973 » ne veut rien dire ; « 973 sur 243 360 » veut dire tout.
- **Compter avant d'importer, révérier après.** Deux compteurs, une différence : c'est l'import qui a menti, pas vous.
- **Ne jamais réparer pendant le diagnostic.** La fiche est signée sur le fichier tel quel ; la réparation s'écrit dans un journal séparé (C06), avec l'avant et l'après.
- **Écrire la règle de gestion à côté du contrôle.** « remise $\in [0 ; 1]$ » doit être une phrase d'un document, sinon personne ne peut vous reprocher d'avoir signalé 973 lignes — ni vous défendre de les avoir ignorées.

Attention. Un contrôle croisé est la première victime d'un changement de règle de gestion : la TVA passe de 18 % à 20 %, l'égalité est restée écrite avec l'ancien taux quelque part, et le contrôle se met à hurler sur des lignes parfaitement saines. La règle vit dans le dictionnaire (C06), la formule l'applique — jamais l'inverse. - **Conserver le fichier de départ, en lecture seule.** Un diagnostic rejouable suppose que la source n'a pas bougé. - **Faire relire la fiche par quelqu'un qui n'a pas le fichier.** S'il ne peut pas retrouver ne serait-ce qu'un nombre, la fiche est mauvaise — c'est le seul test d'utilité qui compte.

10. Exercice guidé — le trio qui décide du verdict (30 min, /10)

On traite ensemble les trois points qui, sur ce fichier, changent la conclusion. Vous faites, je corrige au fil de l'eau ; chaque étape finit par un nombre à reporter sur la fiche.

Étape 1. Combien de lignes de vente portent un montant texte ? `NBVAL(N2:N243361) - NB(N2:N243361)` → **3 421**. Si vous trouvez 3 360, vous avez lu le rapport fourni au lieu du fichier — retournez au § 5.12, c'est exactement l'erreur visée.

Étape 2. Combien de lignes doit-on retirer pour rejoindre le fichier propre ? `243 360 - 240 000` → **3 360**. Notez que ce n'est pas le même nombre, et qu'il n'y a aucun paradoxe : un défaut de type, un défaut de doublon.

Étape 3. Les deux défauts se recouvrent-ils ? Comptez les lignes qui sont à la fois en texte sur le montant et absentes du fichier propre → **61 lignes**. C'est peu, et c'est le chiffre qui décide de l'ordre des opérations : ces 61 lignes doivent passer par la réparation de type avant le dédoublonnage, sinon on retire l'une des deux et on laisse l'autre avec un montant vide — le total ne se réconcilie plus, et plus rien ne se prouve ligne à ligne.

Étape 4. Vérifiez l'addition : $3\,421 + 3\,360 - 61 = 6\,720$ lignes concernées par au moins un des deux défauts, soit 2,8 % du fichier. Écrivez cette ligne dans la fiche : c'est le périmètre du nettoyage, et il vous évitera d'annoncer « 3 421 lignes à corriger » à un lecteur qui en trouvera 6 720.

Étape 5. Décision. Les 6 720 lignes valent 2 111 789 FCFA côté texte et 219 766 129 FCFA côté doublons. Reportez la décision : réparer les textes par `montant_ht + montant_tva`, puis dédoubler sur la clé métier, puis reconstruire contre `verites_terrain.csv`. **Note /10** : 2 points par nombre juste et situé (point, formule, dénominateur), 2 points pour le recouvrement des 61 lignes, 4 points pour la décision. Un nombre sans dénominateur vaut zéro.

11. Exercices autonomes

Exercice 1.1 (★) — Le trio structurel (8 min). Refaites les points 2, 4 et 6 sur `clients.csv` (23 912 lignes) et écrivez les trois lignes de fiche. Quel est le seul point de la grille qui, ici, n'a pas de réponse nulle ?

Exercice 1.2 (★★) — La clé de dédoublonnage (12 min). Le point 5, appliqué aux clients : comptez les lignes en trop si la clé candidate est le nom normalisé (lettres minuscules, sans accent ni espace) ; comptez-les si elle est nom normalisé + seuls chiffres du téléphone. Donnez les deux nombres et expliquez lequel est utilisable.

Exercice 1.3 (★★) — Le seuil qui ne contrôle rien (15 min). Les stocks : sur les 60 000 lignes de `stocks_quotidiens.csv`, comptez les quantités physiques négatives, les lignes sous le seuil d'alerte, et l'écart théorique – physique maximal. Les deux premiers nombres sont identiques — que faut-il en conclure sur la qualité de la colonne `seuil_alerte` ?

Exercice 1.4 (★★) — Trois nombres pour un fichier (12 min). Le tarif fournisseur : donnez les trois nombres qui, à eux seuls, décrivent ce fichier en un paragraphe (lignes de bruit avant l'en-tête, lignes de données, prix dont le séparateur décimal n'est pas un point) et indiquez lequel se corrige à la lecture, lequel à l'écriture.

Exercice 1.5 (★★★) — Rendre un contrôle rejouable (20 min). Le point 12, retourné : écrivez la ligne de fiche qui aurait attrapé l'erreur des dates au format mixte (2 640 annoncées) sans ouvrir le fichier d'à côté — c'est-à-dire en rendant la formule rejouable. Vous ajouterez aussi la ligne qui aurait évité l'erreur sur les catégories de produits (61 annoncées, 44 mesurées) : celle qui oblige à citer la **liste attendue** avec son nombre de modalités.

12. Correction détaillée

Exercice 1.1. Point 2 : 23 912 lignes × 12 colonnes. Point 4 : 23 912 `id_client` distincts → 0 doublon technique. Point 6 : le téléphone est numérique-comme-texte partout, la ville absente sur **1 911** lignes, l'e-mail sur **16 514**. Le seul point non nul est le point 3 (complétude) — et c'est aussi le seul qui ne concerne pas le fichier de ventes. Résultat intermédiaire à vérifier : $1\,911 + 22\,001 = 23\,912$.

Exercice 1.2. Sur le nom normalisé seul, $23\,912 - 12\,342 = \mathbf{11\,570}$ lignes « en trop » : la règle est inutilisable, elle fusionnerait des clients réellement distincts (le jeu contient des noms génériques comme `Menage 00002`). Sur nom normalisé + chiffres du téléphone, l'excédent tombe à **412** — exactement le nombre de lignes retirées pour passer de 23 912 à 23 500 dans le fichier propre. Conclusion à écrire : **une clé de dédoublonnage se valide contre le résultat attendu, pas contre son élégance** ; et le téléphone est ce qui rend la clé utilisable, parce que son écriture varie d'une ligne à l'autre (le jumeau a perdu ses espaces) alors que ses chiffres restent ceux du client — le nom `SARL K`, lui, est porté par 75 lignes du fichier.

Exercice 1.3. 1 800 lignes à quantité physique négative, 1 800 lignes sous le seuil d'alerte, écart maximal 532 unités. Les deux compteurs coïncident : la règle croisée est un identique — toute ligne négative est sous le seuil. On en conclut que `seuil_alerte` ne discrimine pas (il ne peut pas, si le signe suffit à déclencher) : le contrôle utile n'est donc pas « quantité < seuil » mais « quantité < 0 » d'une part, « écart |théorique – physique| > seuil » d'autre part. C'est l'exemple type du point 8 : une formule juste sur une donnée fausse produit un alarme permanent, et une alarme permanente ne contrôle rien.

Exercice 1.4. Trois nombres : **3** lignes de bruit avant l'en-tête (« export genere depuis... », la date d'édition, le nom du fournisseur), **38** lignes de données, **38** prix sur 38 écrits avec une virgule décimale. Le bruit se corrige à la lecture (forcer la ligne d'en-tête, ou supprimer les trois lignes dans le fichier d'origine — pas dans la feuille de travail) ; la virgule se corrige à l'écriture, c'est-à-dire dans le paramètre régional de l'import, sinon le nombre lu est 2 900 au lieu de 2 900,57 et la colonne entière devient du texte. Ajoutez le quatrième, qui n'était pas demandé et qui gêne davantage : **1** seule valeur de `conditionnement` pour 38 lignes — une colonne déclarative qui n'a qu'une modalité est une colonne qui ne dit rien.

Exercice 1.5. Pour les dates : la ligne de fiche doit contenir la formule, la colonne et le fichier — « `=NB.SI(date_vente;">"&DATE(2026;8;31)) = 51`, mesuré le [date] sur `ventes_brutes.csv` » — de sorte que le relecteur qui refait le calcul trouve 51 et puisse écrire 0 s'il les trouve. Un contrôle non jouable ne peut pas être contredit, donc ne prouve rien. Pour les catégories : la ligne attendue est « catégories hors des 7 de la liste de référence : 44 sur 154 références, liste attendue jointe en `DIAGNOSTIC!A20:A26` » — c'est-à-dire que **le domaine doit être dans la fiche**, sinon le chiffre de 61 (celui du plan) et celui de 44 (celui du fichier) se valent aux yeux du lecteur, et le seul qui compte disparaît.

13. Mini-projet M04.P1 — « La fiche de diagnostic du fichier reçu ce matin » (45 min)

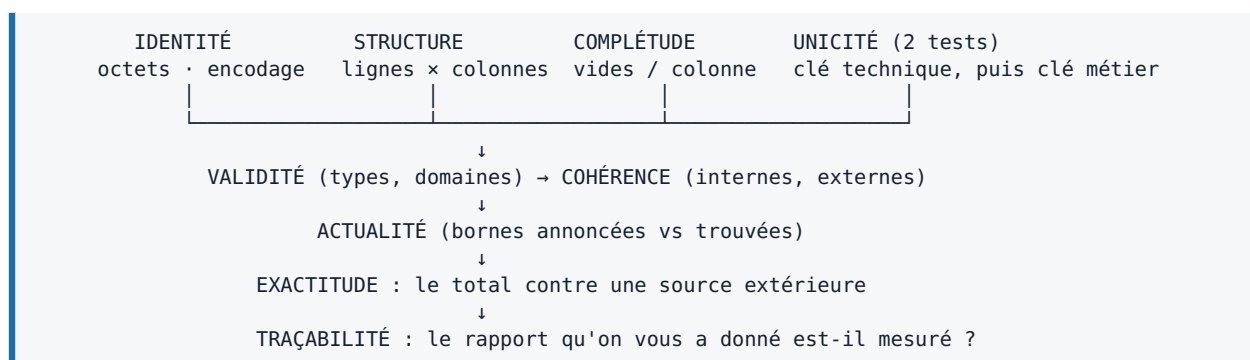
Livrable unique : `diagnostic_ventes_brutes.md`, une fiche de douze lignes plus un entête. Contraintes, qui sont celles du métier :

- la fiche tient sur **une page** et commence par l'identité du fichier (nom, octets, date de lecture) ;
- chaque ligne des douze points porte : le nombre, le dénominateur, la formule ou l'outil, et **une décision** (ignorer / surveiller / corriger avant tout calcul) ;

- deux lignes au moins doivent contredire le `rapport_defaults.json` fourni, avec la preuve par la mesure, et le périmètre du nettoyage est écrit en **union** (6 720 lignes), jamais en addition des deux défauts ;
- la dernière ligne est le verdict : peut-on calculer un chiffre d'affaires sur ce fichier en l'état ? — répondue par le nombre, pas par l'adjectif.

Barème (10 points). Douze lignes complètes et datées (3) · dénominateurs systématiques (2) · recouvrement des 61 lignes chiffré et exploité (2) · deux contradictions du rapport fourni, preuves à l'appui (2) · verdict exprimé en FCFA et non en adjectif (1). Seuil de validation : 7. Le corrigé type est la fiche que le § 7 a produite, complétée des points 9 à 12 ; la mienne se rejoue en 45 minutes sur le fichier, et c'est le critère.

14. Résumé du chapitre



Douze questions, douze nombres, douze décisions. Trois d'entre elles décident du verdict sur le socle — les types (3 421), la clé métier (3 440 collisions, 3 360 à retirer), l'exactitude (l'écart de 2 111 789 FCFA) — et leur recoupement fait le périmètre réel du nettoyage : 6 720 lignes — et une dernière vous protège : celle qui vous fait douter du document censé vous rassurer. Un diagnostic utile ne dit pas que le fichier est sale ; il dit ce qui est cassé, de combien, et ce que ça coûte au chiffre que vous allez publier.

15. À retenir

À retenir. ★ **La clé du fichier n'est pas la clé du réel.** 243 360 `id_vente` tous distincts, et 3 440 lignes en collision sur ticket + produit + quantité : le doublon ne se voit que si on le cherche sur la bonne clé. ★ **Un défaut non traité peut s'annuler avec un autre, un défaut mal traité s'aggrave.** Réparer les textes sans dédoubler ajoute 219 766 129 FCFA ; ne rien faire en perd 2 111 789. Le total est un contrôle, pas un verdict. ★ **Tout nombre avec son dénominateur.** 973 ne veut rien dire ; 973 sur 243 360 veut dire : une ligne mille, sauf là où ça compte. ★ **Le diagnostic ne change aucune cellule.** Il se signe sur le fichier tel quel ; la réparation s'écrit ailleurs, avec l'avant et l'après (C06). ★ **Le document de qualité qu'on vous remet est une hypothèse.** Trois de ses cinq lignes testées ne collaient pas au fichier — et personne n'avait tort : personne n'avait mesuré.

16. Évaluation formative (auto-correction, 8 min)

Q1. Un fichier a 243 360 valeurs distinctes sur sa colonne d'identifiant. Peut-on écrire « aucune ligne en trop » ? (a) oui ; (b) non, on ne le sait pas ; (c) oui si le total des montants est cohérent. → **(b).** L'unicité de la clé technique ne dit rien de la clé métier : c'est le § 5.5, 3 440 collisions avec une clé impeccable sur 243 360 lignes.

Q2. Le total d'une colonne de montants rend moins que la source indépendante. Combien de causes au minimum ? (a) 1 ; (b) 2 ; (c) autant qu'il y a de colonnes. → **(b) au minimum** — des valeurs que l'outil n'a pas su additionner, ou des lignes en trop ailleurs dans le fichier. Les deux agissent en sens contraire, et c'est ce qui rend l'écart seul peu informatif.

Q3. Quelle formule distingue une colonne entièrement numérique d'une colonne contenant du texte ? → `NB(colonne)` contre `NBVAL(colonne)` : l'égalité dit « tout est nombre », la différence donne le nombre de textes. Ici 239 939 contre 243 360.

Q4. Vrai ou faux : « un contrôle de date au format AAAA-MM-JJ suffit à valider l'actualité d'un fichier ». → **Faux**. 0 date mal formatée ici, et 51 lignes postérieures à la borne annoncée (max 2026-12-06). Le format est un contrôle de validité (point 6), la borne est un contrôle d'actualité (point 10).

Q5. Vous devez prouver qu'un nettoyage n'a pas cassé le chiffre d'affaires. Quelle est la moindre des preuves ? → un **rapprochement de totaux** avec le détail des écarts : avant, après, cible, et le nom de la source de la cible ; plus la liste des lignes retirées, comptée. Le § 5.11 montre pourquoi le « après » seul ne prouve rien.

Question ouverte (la plus importante). Reprenez les trois lignes du § 5.12 où l'annoncé et le mesuré diffèrent, et écrivez, pour chacune, la phrase que vous auriez mise dans la fiche pour que l'écart soit visible sans vous relire. C'est exactement le travail du chapitre : rendre l'écart lisible par quelqu'un d'autre. Corrigé attendu : une phrase par défaut, avec le nombre des deux côtés, la source du nombre annoncé, et la formule qui a produit celui du fichier — trois éléments, pas un adjectif.

Module M04.C02 — Les absences : cinq mécanismes, cinq traitements, et le coût de chaque choix

Outil de ce chapitre : le tableur, sur ventes_brutes.csv et clients.csv. Durée indicative : 5 h. Niveau : N2.

L'idée du chapitre. Vous allez ouvrir un fichier de 243 360 lignes et 20 colonnes et y chercher des cellules vides. Vous n'en trouverez **aucune** : zéro, sur 4 867 200 cellules. Ce fichier est pourtant plein d'absences — 43 681 lignes sans client, 16 514 clients sans e-mail, 3 421 montants sans montant, 47 retours sans montant d'aucune sorte. L'absence n'est pas un état de la cellule : c'est une **décision d'écriture**, prise par quelqu'un, quelque part, avec une convention que vous devez retrouver avant de choisir un traitement. Cinq mécanismes produisent une absence ; cinq traitements existent ; et le chapitre se termine par le prix de chacun, en francs.

Base de travail. 01_socle_donnees/data/brut/ventes_brutes.csv (243 360 lignes × 20 colonnes), 01_socle_donnees/data/brut/clients.csv (23 912 lignes × 11 colonnes), 01_socle_donnees/data/brut/stocks_quotidiens.csv (60 000 lignes), 01_socle_donnees/data/reference/ventes_propres.csv et sa cible verites_terrain.csv (15 419 985 157 FCFA). Les nombres du chapitre viennent de ces fichiers, relevés par 01_socle_donnees/scripts/chiffres_manuel.py.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **distinguer** les cinq mécanismes qui produisent une valeur absente, et dire lequel s'applique à une colonne donnée ;
- **reconnaître** les conventions qui cachent une absence (le 0, le tiret, la chaîne vide, la ligne absente) ;
- **choisir** entre les cinq traitements — assumer, joindre, dériver, imputer, supprimer — et l'écrire en décision chiffrée ;
- **mesurer** le coût d'une politique d'absence sur le total, avant de l'appliquer ;
- **refuser** l'imputation automatique sur toute colonne qui porte une décision de gestion, et le justifier par un nombre.

2. Pourquoi cette notion est importante

- La colonne `id_client` vaut 0 sur **43 681 lignes** du fichier de ventes, soit **17,9 %**. Si vous traitez ce 0 comme une valeur, votre premier client est trois chiffres ; si vous le traitez comme un manque et que vous jetez la ligne, vous perdez **2 813 668 030 FCFA** de chiffre d'affaires pour une raison qui n'existe pas : c'est la vente au comptoir, et la règle de gestion du socle l'écrit noir sur blanc.
- Chez les clients, l'e-mail est renseigné pour **7 398 lignes sur 23 912**, soit **30,9 %**. Une campagne envoyée à « tous les clients » toucherait un client sur trois ; une campagne construite après avoir supprimé les lignes sans e-mail toucherait un client sur trois **en croyant viser toute la base**. C'est la même erreur, avec un dénominateur différent, et c'est la plus coûteuse à détecter après coup.

- Les montants : 3 421 lignes n'ont pas de nombre. Les combler à la moyenne donne un total de 15 642 826 075 FCFA, soit 222 840 918 FCFA de plus que la vérité. Les reconstituer par la règle de gestion (`montant_ht + montant_tva`) donne 15 639 751 286 FCFA, trop gros pour une autre raison que C01 avait déjà vue : les lignes en trop. Deux façons de « traiter les vides », trois ordres de grandeur d'erreur, et une seule qui se corrige en ajoutant un point de la grille.
- Enfin, une absence est parfois une **bonne nouvelle** : ces 16 514 e-mails absents sont exactement ce qu'il faut pour publier un fichier d'exercice sans exposer une personne. Le C06 y revient, et l'ordre des idées compte : on ne supprime pas une colonne par principe, on décide après avoir compté.

3. Explication simple

Un dossier médical ne dit pas « inconnu » de la même façon selon le cas. Le patient n'a pas de groupe sanguin parce qu'on ne l'a pas demandé ; il n'a pas de traitement en cours parce qu'il n'en prend pas ; il n'a pas de numéro de sécurité sociale parce qu'il l'a oublié. Les trois cases sont vides, et ne se traitent pas de la même main.

Une base de données, c'est la même chose, avec un danger de plus : **l'outil ne fait pas la différence**. Pour lui, une case vide, un 0, une case remplie de N/A et une ligne qui n'existe pas sont quatre états distincts, mais aucun ne s'appelle « je ne sais pas ». Le tri par date, la moyenne, la jointure : tous traiteront ces quatre états comme quatre présences.

Retenez une phrase, elle vous sauvera des semaines : **0 n'est jamais « rien » ; 0 est une valeur que quelqu'un a écrite**. Et « rien », dans un fichier, ça peut aussi être une ligne absente — la plus difficile des absences à trouver, parce qu'elle ne se voit que par le décompte des attendus : les 486 lignes de vente sans client référencé (point 9 de la grille) sont de celles-là.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Valeur manquante Missing value	— Ce qui, dans le monde, existe mais n'est pas dans le fichier.	Confondre l'absence de valeur et l'absence d'objet.
Non applicable — Not applicable	Le champ n'a pas de sens pour cette ligne (le comptoir n'a pas de client).	Le combler : on invente un client pour une vente qui n'en a pas.
Valeur de sentiment Sentinel value	— Un code écrit pour dire « pas de valeur » : -1, 9999, N/A, inconnu.	L'oublier dans le paramétrage de l'import : -1 entre dans la moyenne.
Mécanisme d'absence Missing mechanism	— La raison pour laquelle la valeur manque : non-collectée, non-répondue, perdue, non-applicable, déguisée.	Traiter sans savoir lequel : les cinq n'ont pas le même remède.
Imputation — Imputation	Remplacer une absence par une valeur estimée (constante, médiane par groupe, cas semblable, modèle).	L'appliquer à une colonne qui porte une décision, ou sans le dire.
Perte par jointure — Join-induced loss	Perte de lignes causée par une jointure intérieure sur une clé absente.	Compter les lignes après la jointure pour juger du nettoyage.
Marqueur de manque — Missing indicator	— Colonne booléenne qui garde la trace : « cette valeur était absente ».	Oublier de le créer quand on impute : « on ne savait pas » est alors perdu.
Politique de valeurs manquantes — Missing data policy	Le texte qui dit, colonne par colonne, ce qu'une absence veut dire et comment on la traite.	Ne pas l'écrire : chaque repreneur décide pour soi, et les totaux divergent.

5. Cours approfondi : les cinq mécanismes, les cinq traitements

5.1 Mécanisme A — Le non-applicable : la valeur n'existe pas à l'état civil

La vente au comptoir n'a pas de client. Ce n'est pas une donnée perdue, c'est une **absence de l'objet**. Le fichier l'écrit 0 dans `id_client`, sur 43 681 lignes. Le traitement utile n'est ni de l'effacer ni de le combler : c'est de **nommer** le compte. Dans un modèle, on crée une ligne « Comptoir » dans la table clients (le C06 montre l'écriture de cette ligne de convention) ; dans un tableau croisé, on remplace l'affichage du 0 par un libellé.

Définition. Une **convention de codage** est un choix d'écriture, arbitraire mais écrit, qui donne à une absence un signe visible : 0, -1, la chaîne vide, un enregistrement dédié. Un 0 sans texte pour le décrire n'est pas une convention, c'est une coïncidence — et une coïncidence se dément à la première reprise de fichier.

5.2 Mécanisme B — Le non-collecté : on n'a jamais posé la question

L'e-mail des clients : renseigné pour 7 398 sur 23 912. Personne n'a perdu ces adresses, elles n'ont jamais été demandées au guichet. C'est le seul des cinq mécanismes qui se corrige **en amont**, par un formulaire ou une habitude de caisse, et jamais en aval par un calcul. Un analyste qui comble un non-collecté fabrique une liste de relance adressée à des inconnus.

5.3 Mécanisme C — Le non-répondu : la question a été posée, la réponse non

La ville manque pour 1 911 clients sur 23 912 (8,0 %). Contrairement au cas précédent, le fait existe — le client est quelque part — et il est parfois récupérable ailleurs : une ville dans un autre champ, une région renseignée, le code postal d'une adresse de livraison. **Joindre**, voilà le traitement propre du non-répondu ; le remplissage par la ville majoritaire en est le traitement sale, et il s'appelle une décision commerciale, pas une donnée.

5.4 Mécanisme D — La perte à l'import ou à la conversion

C'est le mécanisme que le fichier de ventes contient le plus : 3 421 lignes où `montant_ttc` n'est pas un nombre, 47 lignes de retour sans montant exploitable, et, dans le fichier reçu en M01, 13 colonnes là où le brut en a 20. Rien n'a été décidé par un humain : la valeur a été **abîmée en route** — une virgule décimale, un séparateur de milliers, un encodage, une colonne que l'export n'emporte pas. Le remède n'est pas un remplissage mais une **reconstruction par la règle** : le socle écrit que `montant_ttc = montant_ht + montant_tva`, donc un montant perdu se reconstitue ; il ne s'impute pas.

Dans les faits. Les trois quarts des « données manquantes » d'un export ERP ne sont pas manquantes : elles sont mal lues. Le réflexe qui fait gagner des heures est de rouvrir le même fichier avec le séparateur et l'encodage corrigés, et de comparer les compteurs avant de parler de nettoyage. Le point 1 de la grille (identité du fichier) n'est pas une formalité : c'est lui qui évite de nettoyer un défaut de lecture.

5.5 Mécanisme E — L'absence déguisée en valeur tranquille

Le plus discret. `taux_remise` vaut 0 sur **157 634 lignes**, soit 64,8 % du fichier. Est-ce « pas de remise » ou « on n'a pas saisi la remise » ? Le fichier seul ne répond pas. Répondre, c'est croiser : si la remise vaut 0 alors que le client a une remise consentie consignée dans `remises_manuelles.xlsx` (18 200 lignes, 2 043 692 000 FCFA), le 0 est un mensonge de saisie sur ces lignes-là. C'est le seul mécanisme dont le diagnostic passe par un **rapprochement entre deux fichiers**, et non par une formule dans une colonne.

Attention. Toute valeur « tranquille » — le 0 de la remise, la date du 01/01/1900, le -1 d'un code magasin, le texte `inconnu` — est candidate à l'absence déguisée. Le test est simple : demandez-vous ce que deviennent une moyenne, une somme et un tri si cette valeur est comprise comme ce qu'elle n'est pas. Si la réponse est « rien de grave », vous n'avez pas encore regardé la colonne des montants.

5.6 Traitements (1/2) : assumer, joindre

Assumer (le cas A, parfois B) : on garde la ligne, on renomme la valeur, on écrit la convention. Coût : zéro cellule modifiée. Gain : le total reste le total. C'est le traitement à préférer chaque fois qu'il est vrai, et le seul qui ne demande aucune autorisation.

Joindre (le cas C, parfois D) : aller chercher la valeur ailleurs dans le système, sans jamais la recopier à la main — la ville depuis une adresse de livraison, la catégorie depuis `dictionnaire_produits.csv`, le taux de TVA depuis la fiche produit. La jointure a un prix connu : elle perd les lignes sans correspondance, ici 486 lignes. On compte donc **avant et après**, et l'écart s'écrit dans le journal (C06).

Définition. Une **jointure externe** (à gauche, à droite, pleine) conserve les lignes sans correspondance en laissant le champ complémentaire vide ; une jointure **intérieure** les supprime. Sur ce fichier, la différence entre les deux s'appelle 486 lignes : la jointure intérieure est le trou de mémoire le plus courant du nettoyage, et le seul qui se masque derrière un total qui a simplement « un peu baissé ».

5.7 Traitements (2/2) : dériver, imputer, supprimer

Dériver (le cas D) : recalculer ce que la règle de gestion autorise à recalculer. `montant_ttc = montant_ht + montant_tva` rend un nombre sur les 3 421 lignes. Vérification : sur le fichier réparé sans toucher aux doublons, le total donne 15 639 751 286 FCFA contre 15 419 985 157 FCFA attendus. La dérivation est bonne, et le reliquat d'écart est la preuve qu'il subsiste un défaut **d'une autre nature** — ce qui est un résultat, pas un échec.

Définition. **Imputer par le cas le plus semblable** (hot-deck, ou k plus proches voisins) consiste à emprunter la valeur d'une ligne qui ressemble fort à celle qui manque : même produit, même magasin, même mois, même type de client. C'est la seule imputation qui se défende sur une colonne monétaire — et encore : elle se justifie par la ressemblance des lignes, jamais par la commodité de l'outil.

Imputer : remplacer par une valeur estimée. Quatre familles, du plus prudent au plus risqué : la constante documentée, la médiane par groupe (le prix médian du produit, pas du fichier entier), la valeur du cas le plus semblable (même client, même mois, même catégorie), le modèle (régression — M08). Règle absolue : **on n'impute pas une colonne qui porte une décision de**

gestion, et on crée le marqueur qui dit que la valeur est estimée. Le prix, mesuré : 15 642 826 075 FCFA contre 15 639 751 286 FCFA, soit 3 074 789 FCFA de plus, pour 1,4 % des lignes.

Supprimer : retirer les lignes. Uniquement si la ligne est inutilisable pour l'analyse — ligne vide, test, doublon confirmé (C03). La suppression se chiffre : « 6 720 lignes sur 243 360, soit 2,8 % du fichier, 2 111 789 FCFA d'un côté et 219 766 129 FCFA de l'autre » n'est pas une suppression, c'est un arbitrage.

Définition. Un **marqueur de manque** est une colonne ajoutée, en général booléenne, qui survit à la réparation : `ttc_reconstitue = VRAI` sur les 3 421 lignes. Elle coûte une formule et rend possible la seule question qui fâche six mois plus tard : « ce chiffre, il vient du magasin ou de mon calcul ? »

5.8 Les cinq mécanismes et leurs chiffres, en une grille

Mécanisme	Marqueur dans le fichier	Colonne touchée ici	Nombre		Traitement qui convient
A · non-applicable	<code>id_client = 0</code> , écrit par la règle	<code>id_client</code>	43 lignes	681	assumer : compte « Comptoir »
B · non-collecté	chaîne vide, jamais demandé	<code>clients.email</code>	16 clients	514	assumer, et changer le formulaire
C · non-répondu	chaîne vide	<code>clients.ville</code>	1 911 clients		joindre, sinon compter
D · perdu à l'import	texte au lieu d'un nombre	<code>montant_ttc</code>	3 421 lignes		dériver par la règle
E · absence déguisée	une valeur tranquille	<code>taux_remise</code>	157 lignes	634	croiser avec un autre fichier

Attention. Les compteurs de cette grille se lisent avec la colonne « traitement qui convient », jamais seuls. Le même nombre de lignes — 43 681 — appelle à supprimer si on lit « vides » et à assumer si on lit « convention écrite par la règle ». Un chiffre de qualité sans mécanisme est une invitation à la mauvaise opération, faite de bonne foi.

Une sixième ligne mériterait la grille : les lignes **qui devraient exister et n'existent pas**. Sur ce socle, le compteur est à zéro — 0 période manquante et 0 période qui se recouvre dans `couts_achat.csv` (1 694 lignes de périodes continues). Ce n'est pas un vide, c'est un **plein vérifié**, et le compter rend le reste crédible.

Les absences d'un fichier de ventes : cinq mécanismes, cinq traitements

243 360 lignes, 0 cellule vide : ici, l'absence est toujours écrite comme une valeur.

A non-applicable id_client = 0 : écrit par la règle touchées : 43 681 lignes	→	assumer, nommer « Comptoir » une cellule modifiée : non marqueur de manque à poser : inutile
B non-collecté clients.email jamais demandé touchées : 16 514 clients	→	changer le formulaire, en amont une cellule modifiée : non marqueur de manque à poser : inutile
C non-répondu clients.ville laissée vide touchées : 1 911 clients	→	joindre, sinon compter une cellule modifiée : non marqueur de manque à poser : inutile
D perdu à l'import montant_ttc devenu du texte touchées : 3 421 lignes	→	dériver : ht + tva une cellule modifiée : oui, par la règle marqueur de manque à poser : oui
E absence déguisée taux_remise à 0, nulle part vide touchées : 157 634 lignes	→	croiser un autre fichier une cellule modifiée : non marqueur de manque à poser : oui

Le prix des deux traitements de la colonne D, mesuré sur le même fichier :
dériver par la règle : 15 639 751 286 FCFA | imputer à la moyenne : 15 642 826 075 FCFA
cible vérifiée : 15 419 985 157 FCFA. L'écart restant n'est pas une histoire d'absence : ce sont les lignes en trop (C03).

6. Exemple concret : la phrase qui fait basculer un tableau de bord

Lundi, 9 h 15. La direction commerciale veut « le chiffre d'affaires par client », pour récompenser les trois mieux disposés. Vous joignez les ventes aux clients, vous trie, vous publiez. Trois versions du même tableau, et la seule qui ne se discute pas est celle que personne n'obtient par défaut :

Version	Ce qu'on a fait	Total	Ce qui se passe
jointure intérieure	on a gardé ce qui a un client	minoré	les 486 lignes sans correspondance disparaissent sans message
0 laissé tel quel	le comptoir est resté dans le classement	juste de montant, faux de sens	le « premier client » du magasin est 0, avec 2 813 668 030 FCFA
comptoir nommé	le 0 devient « Comptoir »	juste et lisible	le dirigeant voit un canal, pas un client, et peut redécider

La leçon n'est pas « il faut supprimer les 0 ». Elle est que **la même absence produit un chiffre faux, un chiffre vrai mais muet, ou un chiffre vrai et parlant, selon le nom que vous lui donnez**. Une large part des erreurs de tableau de bord ne sont pas des erreurs de calcul : ce sont des absences auxquelles personne n'a donné de nom.

7. Démonstration pas à pas : de l'œil au compteur, puis à la décision

Sur la feuille `Ventes` (243 360 lignes × 20 colonnes, colonnes A à T), on écrit une feuille `MANQUES` : une ligne par colonne du fichier, six colonnes de compte. Les mêmes compteurs se jouent sur `clients.csv`.

- Les vraies vides.** `=NB.SI(Ventes!N2:N243361;"")` sur `montant_ttc` → 0. Faites-le sur les vingt colonnes : le total est de 0 cellule vide. **Le fichier ne contient pas d'absence écrite comme un vide** — conclusion à écrire telle quelle, elle change tout le reste de la méthode.

2. **Les zéros, par colonne.** `=NB.SI(Ventes!G2:G243361;0)` sur `id_client` → 43 681. Le même compteur sur `est_retour` (colonne Q) rend 240 512. Deux zéros, deux natures : à gauche un objet absent, à droite un booléen à « non ». Le signe est identique, le mécanisme non.
3. **Les valeurs de sentiment.** `=NB.SI(Ventes!C2:C243361;"-")`, puis sur "N/A", "inconnu", "0/00/0000", "-&" : **0 partout**, sur les vingt colonnes. Cette ligne de fiche est un résultat : elle dit que le producteur n'a aucune convention textuelle du manque, donc que tout ce qui ressemble à un manque a été écrit comme une valeur.
4. **Ce que le 0 coûte.** Somme conditionnelle des montants numériques sur les lignes à `id_client = 0` → 2 813 668 030 FCFA, à comparer à 15 419 985 157 FCFA : un septième du chiffre d'affaires dépend d'un caractère.
5. **La remise silencieuse.** `=NB.SI(Ventes!K2:K243361;0)` → 157 634 lignes. Croisez avec `remises_manuelles.xlsx` (18 200 lignes, 2 043 692 000 FCFA) : la question à poser au métier n'est pas « combien de zéros ? » mais « ces deux fichiers se contredisent-ils sur les mêmes clients, sur les mêmes mois ? ».
6. **L'absence de schéma.** Ouvrez le fichier reçu en M01, `ventes_magasin5_2025.csv` : 13 colonnes, 489 lignes, pas de `montant_tva`. Le brut en a 20. L'écart n'est pas un vide, c'est une **colonne qui n'existe pas** — la seule absence qui ne se voit pas dans une colonne, donc la dernière que l'on regarde.
7. **La décision, écrite.** Une ligne par colonne, un verbe parmi assumer / joindre / dériver / imputer / supprimer, le nombre, le coût, la date. C'est la colonne décision qui sépare un inventaire d'un travail : sans elle, votre fiche sera reprise par quelqu'un qui supprimera les 43 681 lignes.

Boîte à outils du chapitre. Une grille à cinq colonnes — colonne, compteur, mécanisme, traitement, coût — et une case obligatoire « ce que je ne touche pas ». Cette dernière case est celle qui, en reprise, vous évite deux semaines d'explication sur pourquoi le total d'un fichier réparé ne ressemble plus à celui du fichier reçu.

À retenir. Un traitement se juge à ce qu'il laisse intact. Assumer ne touche aucune cellule et garde le total ; joindre ne touche aucune cellule mais change le périmètre de la jointure ; dériver réécrit une colonne à partir d'une règle ; imputer invente une valeur ; supprimer détruit un fait. Du premier au dernier, l'initiative passe du fichier à vous — et c'est exactement dans ce sens qu'il faut choisir, jamais l'inverse.

8. Erreurs fréquentes

Symptôme visible	Cause réelle	Correction
« Aucune donnée manquante, le fichier est plein »	le comptage s'est limité à la cellule vide	compter aussi les zéros, les valeurs de sentiment, les colonnes absentes : mécanismes A à E
Le total baisse d'un septième après nettoyage	le 0 de <code>id_client</code> a été pris pour un manque	assumer la convention et nommer le compte, au lieu de supprimer la ligne
Le tableau croisé a moins de lignes que le fichier	jointure intérieure sur une clé absente	joindre en externe et compter les perdantes : ici 486 lignes, 372 clients
La moyenne des montants a bougé après remplissage	imputation à la moyenne générale sur une colonne monétaire à forte variance	reconstituer par la règle, ou imputer par groupe, et poser le marqueur
Des dates d'e-mails sont devenues 1900	la date vide est lue comme une date nulle par l'import	déclarer les valeurs de sentiment à l'import, et contrôler le MIN de la colonne de dates
« J'ai écrit Non renseigné, c'est propre »	une chaîne ajoutée pour dire « rien » crée une sixième modalité	ne pas introduire de modalité textuelle dans une colonne de clés : garder la ligne non jointe et la compter

9. Bonnes pratiques professionnelles

- **Un dictionnaire de colonnes avec une ligne « ce que veut dire un vide ».** C'est la seule ligne que tout le monde saute à la lecture, et la première consultée le jour où ça casse.
- **Comptez les absences par ligne, pas seulement par colonne.** Un fichier peut avoir 0 % de vides par colonne et des lignes inexploitable : les absences se cumulent sur les mêmes lignes, et c'est le lecteur qui trinque.
- **Jamais d'imputation sans marqueur.** Sans la colonne « valeur estimée ici », personne — y compris vous dans six mois — ne distingue le mesuré de l'inventé.
- **Un imputé ne se mélange pas à un vrai dans un verdict.** S'il faut un chiffre unique pour la direction, donnez-le et donnez la fourchette : 15 639 751 286 FCFA par la règle, et non moins de 15 419 985 157 FCFA une fois les lignes en trop retirées.
- **Un compteur à zéro se consigne.** « 0 valeur de sentiment », « 0 date au format mixte », « 0 période trouée » : ces lignes sont ce qui permet à un reprenneur de contredire un rapport de qualité mal mesuré — le vôtre compris.
- **Sur une colonne de décision, l'absence est une information.** Un client à 0 % de remise n'est pas un client raisonnable, c'est un client à qui personne n'a rien accordé. Même valeur, deux lectures, et une seule se pilote.

Conseil professionnel. Écrivez votre politique d'absence **avant** le premier nettoyage, et datée. Dans le sens inverse, on nettoie d'abord ce qui gêne, puis on rédige une règle qui justifie ce qui a été fait : le document ressemble alors à un procès-verbal, et il ne protège personne. Le test est simple : si votre politique rédigée ne prédisait pas les décisions que vous avez prises, ce n'est pas une politique.

- **Faites relire la politique par le métier, pas par un collègue analyste.** Si le directeur de magasin ne reconnaît pas le comptoir dans votre ligne `id_client`, la politique est fausse, quel que soit le nombre.

10. Exercice guidé — trier quatre colonnes en quatre traitements (30 min, /10)

Quatre colonnes, relevées à l'instant : `id_client` (43 681 zéros), `montant_ttc` (3 421 textes), `clients.ville` (1 911 vides), `taux_remise` (157 634 zéros). Vous décidez, je corrige à chaque étape.

Étape 1. Nommez le mécanisme de chacune. Attendu : A pour `id_client`, D pour `montant_ttc`, C pour la ville, E pour la remise. Si vous classez la ville en B (non-collecté), vérifiez : la colonne existe, donc la question a été posée — c'est la réponse qui manque, pas la question. B et C ne se réparent pas au même endroit : l'un au formulaire, l'autre à la jointure.

Étape 2. Pour `montant_ttc`, écrivez la dérivation. `=SI(ESTNUM(N2);N2;L2+M2)` (N = `montant_ttc`, L = `montant_ht`, M = `montant_tva`) rend un nombre sur les 3 421 lignes. Reportez le total ainsi obtenu, sans toucher aux doublons : 15 639 751 286 FCFA. Il reste un écart de +219 766 129 FCFA — ne le corrigez surtout pas ici : c'est la signature des lignes en trop, et son tour vient en C03.

Étape 3. Comparez avec l'imputation à la moyenne des montants positifs : 15 642 826 075 FCFA. Les deux méthodes sont « correctes » au sens du tableur ; l'une dérive d'une règle écrite, l'autre d'une distribution. **Écrivez la phrase de décision**, celle qui ira dans le journal. Une décision qui ne dit pas pourquoi l'autre méthode a été écartée n'est pas une décision, c'est une habitude.

Étape 4. Pour la ville, combien de lignes récupérables par jointure ? Réponse mesurée : 0 — `clients.csv` ne contient qu'une adresse, et le fichier de ventes ne contient pas de ville. Le traitement devient donc « assumer, et compter ». Un traitement qu'on ne peut pas appliquer n'est pas un choix, c'est un constat : écrivez-le comme tel, sinon on vous reprochera de ne pas l'avoir tenté.

Étape 5. Pour `taux_remise`, produisez le compteur de croisement : combien de clients ont une remise consignée dans `remises_manuelles.xlsx` et une remise à 0 sur la même période. C'est le seul des quatre qui ne se résout pas dans le fichier. Rendu attendu : « question ouverte au métier — 157 634 lignes portent un 0, 18 200 lignes du fichier de saisies affirment le contraire sur les mêmes clients ; en l'absence de réponse, le 0 est conservé et la moyenne des remises est retirée des indicateurs de pilotage ». **Note /10** : 3 points pour les quatre mécanismes nommés et justifiés, 4 points pour les deux totaux et la phrase de décision, 3 points pour l'honnêteté des constats non exécutables. Un nombre sans dénominateur, ici, vaut zéro.

11. Exercices autonomes

Exercice 2.1 (★) — Les quatre compteurs (10 min). Sur `clients.csv`, relevez les quatre compteurs d'absence (ville, e-mail, téléphone, segment) avec leur dénominateur. Écrivez pour chacun la phrase de politique, en un verbe parmi : assumer, joindre, dériver, imputer, supprimer.

Exercice 2.2 (★★) — Le prix d'un 0 (15 min). Calculez le total des montants numériques sur les lignes où `id_client` vaut 0, puis ce que devient le total du fichier si vous supprimez ces lignes. Donnez les deux nombres, leur part du total, et la phrase de deux lignes que vous enverriez au directeur financier pour refuser la suppression.

Exercice 2.3 (★★) — La colonne qui n'existe pas (12 min). Comparez l'en-tête du brut (20 colonnes) à celui du fichier reçu `ventes_magasin5_2025.csv` (13 colonnes, 489 lignes). Citez les colonnes absentes, et dites pour laquelle au moins l'absence change un calcul déjà fait en C01 ou en M03.

Exercice 2.4 (★★) — Le faux négatif du compteur (15 min). Vous cherchez les tirets - dans tout le fichier : la recherche rend 0 sur les vingt colonnes. Un collègue affirme que son ERP écrit systématiquement - pour les dates inconnues. Que vérifiez avant de le croire, et que mettez-vous dans la fiche ? Trois indices : la colonne `date_vente`, le nombre de lignes, le nom du fichier.

Exercice 2.5 (★★★) — Imputation contrôlée (25 min). Sur une copie, traitez les 3 421 `montant_ttc` de deux façons : (a) moyenne des montants positifs du fichier ; (b) reconstitution `montant_ht + montant_tva`. Pour chaque méthode : le total, son écart à 15 419 985 157 FCFA, et le nombre de lignes dont le montant imputé s'écarte de plus de 100 000 FCFA du montant reconstitué. Concluez en une phrase sur le sort d'une valeur monétaire absente dans un fichier de ventes.

12. Correction détaillée

Exercice 2.1. Ville : 1 911 vides sur 23 912 (8,0 %) → joindre, mais la jointure est impossible (voir 2.3), donc assumer et compter. E-mail : 16 514 sur 23 912 (69,1 %) → assumer ; non-collecté, et son absence est un actif pour la publication (C06). Téléphone : 0 vide sur 23 912 → rien à traiter, **mais à écrire** : c'est ce compteur à zéro qui autorisera l'appariement du C03. Segment : 0 vide → rien. Les deux compteurs nuls sont ceux qu'on est tenté de passer sous silence, et ce sont eux qui rendent défendables les deux autres.

Exercice 2.2. Total des lignes comptoir : 2 813 668 030 FCFA sur 43 681 lignes, soit 17,9 % du fichier en lignes et un peu plus de 18 % du total attendu. Phrase attendue : « supprimer les lignes sans `id_client` retire un septième du chiffre d'affaires ; ce n'est pas un nettoyage, c'est un changement de périmètre — le comptoir est un canal, pas un client manquant ». Notez le

glissement probable : la part en lignes (17,9 %) et la part en montant ne sont pas égales, et c'est la seconde qui décide.

Exercice 2.3. L'en-tête reçue est n_ticket;date;heure;magasin;vendeur;client;produit;categorie;quantite;prix_unitaire_ht;remise;montant_ht;montant_tva;mode_paiement;canal;est_retour;poids_kg;mois;annee. Manquent, entre autres, `montant_tva`, `mode_paiement`, `canal`, `est_retour`, `poids_kg`, `mois`, `annee`. La colonne dont l'absence change un calcul déjà fait est `montant_tva` : sans elle, la dérivation du mécanisme D est impossible et les 3 421 montants texte deviennent réellement irréparables — d'où l'intérêt de compter les colonnes absentes du fichier que l'on va réparer, pas seulement de celui que l'on a sous les yeux. `est_retour` est la seconde : sans elle, les 2 801 montants négatifs n'ont plus de signification.

Exercice 2.4. Trois vérifications, dans cet ordre : que la recherche porte sur le **fichier texte**, pas sur la feuille importée (un `-` avalé par l'import devient un vide ou un nombre, et votre compteur ment poliment) ; que la colonne testée est bien `date_vente` et couvre les 243 360 lignes ; que le fichier décrit par le collègue est bien celui que vous avez (nom, taille : 28 516 920 octets). Ligne de fiche attendue : « 0 occurrence de `-` dans les 20 colonnes de `ventes_brutes.csv`, recherche sur le fichier texte, 243 360 lignes — la convention de l'ERP n'est donc pas arrivée jusqu'ici, ou l'export l'a transformée : question ouverte à la source ». Un résultat négatif consigné avec sa méthode n'est pas une absence de défaut, c'est une preuve.

Exercice 2.5. (a) moyenne : 15 642 826 075 FCFA, écart +222 840 918 FCFA ; (b) dérivation : 15 639 751 286 FCFA, écart +219 766 129 FCFA. Les deux restent gros parce qu'aucune des deux ne traite les lignes en trop, mais ils ne se valent pas : 3 074 789 FCFA séparent les deux méthodes — le prix pur de l'imputation, sur 1,4 % des lignes. Le nombre de lignes qui dévient de plus de 100 000 FCFA se lit par `=NB.SI.ENS(...)` sur une colonne utilitaire `=ABS(imputé-reconstitué)` ; il se compte en centaines, pas en dizaines. Conclusion : une valeur monétaire absente ne s'impute pas, elle se reconstitue par la règle ; si la règle ne permet pas de la reconstituer, la ligne sort du périmètre et la sortie s'affiche. Mettre la moyenne du magasin dans une ligne de facture n'est pas une estimation prudente, c'est une écriture de plus.

13. Mini-projet M04.P2 — « Politique de valeurs manquantes » (1 h)

Livrable : `politique_absences.md`, une page et demie, pour les vingt colonnes du fichier de ventes et les onze de `clients.csv`. Contraintes :

- une ligne **par colonne**, jamais par fichier, avec les quatre compteurs (vides, zéros, valeurs de sentiment, hors domaine) et le dénominateur ;
- pour chaque compteur non nul : le mécanisme (A-E), le traitement, et le **coût chiffré** du traitement choisi et du principal traitement écarté ;
- deux colonnes au moins doivent finir en « question ouverte à la source », la question écrite noir sur blanc (c'est le cas de `taux_remise`) ;
- la dernière section est la politique de **non-réparation** : ce que vous refusez de toucher, et pourquoi.

Barème (10 points). Complétude de la grille (3) · justesse des compteurs, dénominateur inclus (2) · coûts chiffrés des deux traitements pour au moins six colonnes (2) · questions à la source formulées et adressables (1) · section de non-réparation argumentée (2). Seuil de validation : 7. Rendu maximum 8 pour un projet sans section de non-réparation : un nettoyage qui ne dit pas ce qu'il laisse en l'état n'est pas un nettoyage, c'est un tri.

14. Résumé du chapitre

UNE CELLULE, QUATRE VISAGES

			
vide	valeur	sentiment	tiret

CINQ MÉCANISMES

A non-applicable
B non-collecté
C non-répondu
D perdu à l'import
E absence déguisée

CINQ TRAITEMENTS

ASSUMER (nommer)
(amont : le formulaire)
JOINDRE (autre source)
DÉRIVER (la règle écrite)
IMPUTER (+ marqueur)
SUPPRIMER (chiffré, en dernier)

Le fichier de ventes est le contre-exemple parfait de l'intuition : 0 cellule vide, et 43 681 lignes sans client, 3 421 lignes sans montant, 157 634 lignes dont on ignore si le 0 veut dire « rien » ou « personne n'a rien accordé ». L'absence se mesure par quatre compteurs et se traite par une décision écrite. Le reste — le remplissage automatique, la suppression de confort, le 0 compris comme un nombre — est exactement ce qui transforme un problème de 2 111 789 FCFA en un problème de 222 840 918.

15. À retenir

À retenir. ★ **Quatre compteurs, pas un.** Vides, zéros, valeurs de sentiment, colonnes absentes. Ici : 0, 43 681, 0, 7. ★ **0 n'est jamais « rien ».** Dans `id_client`, c'est le comptoir (17,9 % du fichier, 2 813 668 030 FCFA) ; dans `est_retour`, un booléen à « non » ; dans `taux_remise`, une question au métier. Même signe, trois sorts. ★ **Une valeur monétaire ne s'impute pas, elle se reconstitue.** La règle rend 15 639 751 286 FCFA ; la moyenne en ajoute 3 074 789, et aucune des deux ne pardonne l'oubli des doublons. ★ **Toute imputation porte un marqueur.** Sinon l'information « on ne savait pas » est détruite, et le mesuré devient indistinguishable de l'inventé. ★ **Un compteur à zéro se consigne.** « 0 date au format mixte », « 0 valeur de sentiment », « 0 période trouée » : ce sont ces lignes-là qui permettent de contredire un rapport de qualité écrit de mémoire.

16. Évaluation formative (auto-correction, 8 min)

Q1. Un fichier sans une seule cellule vide contient-il des absences ? (a) non ; (b) oui, si une valeur écrite tient lieu d'absence ; (c) seulement si une colonne est vide en entier. → **(b)**, et ici les deux premières formes coexistent : 43 681 lignes où le client est 0, 3 421 montants non numériques. La cellule est remplie, le fait est absent.

Q2. Le 0 de `est_retour` est une absence. Vrai ou faux ? → **Faux** : c'est un booléen à « non ». Le piège n'est pas la valeur, c'est la colonne — le même signe dans `id_client` est un objet absent.

Q3. Vous joignez ventes → clients en interne pour enrichir les 243 360 lignes. Combien de lignes risquent de disparaître, et pourquoi ? → 486, dont 372 clients distincts inconnus. Une jointure intérieure ne prévient pas : c'est le comptage avant/après qui la fait parler.

Q4. Quelle imputation est défendable pour un `montant_ttc` absent ? (a) moyenne ; (b) médiane par catégorie ; (c) `montant_ht + montant_tva` ; (d) aucune, on exclut la ligne. → **(c)**, parce que c'est une règle écrite et vérifiable ; à défaut **(d)**, et on l'affiche. (a) et (b) servent à bornier, jamais à publier.

Q5. Comment un repreneur retrouve-t-il, six mois plus tard, si un montant était présent ou reconstitué ? → par le marqueur et par le journal (C06) : « 3 421 lignes, montant reconstitué par `ht+tva`, le [date] ; contrôle : total 15 639 751 286 contre cible 15 419 985 157 ».

Question ouverte (la plus importante). Reprenez `taux_remise` (157 634 zéros) et écrivez les deux phrases : celle que vous enverriez au métier pour poser la question, et celle que vous ins-

cririez dans la politique si le métier ne répond jamais dans le délai. Corrigé attendu : la première nomme le croisement demandé (`remises_manuelles.xlsx`, 18 200 lignes, 2 043 692 000 FCFA) et le nombre de clients en contradiction ; la seconde fige le traitement — garder le 0, ajouter `remise_non_verifiee`, et retirer la moyenne des remises des indicateurs de pilotage. Deux phrases, deux responsabilités : demander, puis assumer l'absence de réponse.

Module M04.C03 — Doublons : la clé qui ment, la clé qui dit vrai, et le dictionnaire de correspondances

Outil de ce chapitre : le tableur, sur `ventes_brutes.csv`, `clients.csv` et `dictionnaire_produits.csv`. **Durée indicative :** 5 h. **Niveau :** N2.

L'idée du chapitre. Voici un fichier de 243 360 lignes dont la clé primaire est impeccable — 243 360 valeurs, toutes distinctes — et qui contient 3 360 lignes en trop. Le doublon n'est pas une anomalie de saisie que l'on débusque en triant : c'est un **choix de définition**, et ce choix s'écrit sur une clé métier que le fichier ne fournit pas. Ce chapitre fait quatre choses : il montre les quatre niveaux où l'on peut chercher un doublon et ce que chacun laisse passer ; il lit dans l'ordre des lignes le mécanisme qui a produit le défaut ; il énonce la règle de garde — laquelle des deux lignes on conserve, et pourquoi ; et il traite le jumeau du doublon, la **correspondance**, ce moment où deux écritures du même objet ne sont pas deux lignes mais bien une seule.

Base de travail. `01_socle_donnees/data/brut/ventes_brutes.csv` (243 360 lignes × 20 colonnes), `01_socle_donnees/data/brut/clients.csv` (23 912), `01_socle_donnees/data/brut/produits.csv` (154), `01_socle_donnees/data/reference/dictionnaire_produits.csv` (154), `ventes_propres.csv` (240 000 lignes) et sa cible 15 419 985 157 FCFA. Tous les nombres viennent de ces fichiers.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **nommer** le niveau auquel vous avez cherché un doublon — ligne entière, clé technique, clé métier, similarité — et dire ce que ce niveau ne voit pas ;
- **distinguer** une répétition structurelle normale (plusieurs lignes sur un même ticket) d'une répétition anormale ;
- **écrire** une règle de garde qui soit déterministe, et la justifier par ce que le fichier dit de son propre accident ;
- **construire** un dictionnaire de correspondances (15 écritures de catégorie vers 7 réelles, 0 de comptoir vers un compte nommé) et en mesurer l'effet sur les classements ;
- **appairer** deux écritures d'un même client sans casser les homonymes, et chiffrer le dégât de la règle naïve.

2. Pourquoi cette notion est importante

- Les 3 360 lignes en trop de notre fichier valent **+219 766 129 FCFA** une fois les montants réparés. C'est cent fois le montant que l'on perd à ne pas réparer les textes (2 111 789 FCFA) : le doublon est, de loin, le plus gros défaut de ce socle — et celui qu'aucun contrôle de clé primaire ne signale.
- Un dédoublonnage automatique sur `id_ticket` seul effacerait **97 199 lignes** parfaitement valides : c'est le nombre de lignes qui partagent un ticket parce qu'un ticket a plusieurs articles (1,67 ligne par ticket en moyenne, jusqu'à 6). La même commande, sur un fichier de factures, peut détruire 40 % du chiffre d'affaires.

- Côté clients, la règle « même nom » fusionne 11 570 lignes là où une règle « même nom et mêmes chiffres de téléphone » n'en fusionne que 412 : la différence, 11 158 lignes, ce sont des clients bien réels qui ont un nom banal. Un outil de dédoublonnage livré avec « le score de similarité par défaut » fait exactement ce premier choix.
- Une correspondance ratée ne change pas le total : elle change le **classement**. Nos 154 produits portent 15 écritures de catégorie là où le référentiel en admet 7 ; regrouper dessus produit 15 lignes de tableau croisé au lieu de 7, et le directeur qui lit « MATÉRIAUX devant Matériaux » tirera une conclusion sur des fantômes.

3. Explication simple

Deux frères jumeaux, même visage, même nom, nés le même jour : la police nationale dira qu'ils sont deux personnes, parce qu'ils ont deux numéros d'identité. Deux lignes de facture avec deux `id_vente` différents, même ticket, même produit, même quantité : un comptable dira qu'il y a deux fois la même vente, parce qu'il n'y a pas deux fois le même -article.

Un doublon n'est donc **jamais** une propriété des lignes : c'est une propriété d'une définition de l'objet. Vous décidez ce qu'est « une vente », et la réponse tombe. La décision se prend avec trois questions, dans cet ordre :

1. qu'est-ce qu'une ligne, dans le monde ? (ici : un article d'un ticket) ;
2. est-ce que deux lignes peuvent légitimement avoir les mêmes valeurs sur ce qui définit l'objet ? (ici : non) ;
3. si oui, quelles colonnes ajoutées rendraient la répétition anormale ? (ici : aucune — la quantité ne sauve rien, 7 640 des 8 453 lignes en collision ont aussi la même quantité).

Définition. Un **doublon logique** est une paire de lignes qui décrivent le même fait alors qu'une seule pourrait le décrire. Un **doublon matériel** est une paire de lignes dont toutes les valeurs sont égales. Un fichier peut contenir des milliers de doublons logiques et pas un seul doublon matériel — c'est le cas du nôtre : 3 360 contre 0.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Clé primaire — Primary key	Le champ que le fichier utilise pour garantir l'unicité.	Le prendre pour la définition du réel : ici il est propre, le défaut est là quand même.
Clé candidate — Candidate key	— Tout jeu de colonnes dont les valeurs identifient une ligne.	Ne pas les tester toutes : si deux clés candidates se contredisent, l'une ment.
Clé métier — Business key	La combinaison qui, hors informatique, définit l'objet (ticket + produit + quantité).	L'écrire trop large ou trop serrée : trop large, tout est doublon ; trop serrée, rien ne l'est.
Dédoublonnage — Deduplication	— Opération qui ne conserve qu'une ligne par groupe de doublons.	Sans règle de garde : « garder la première » est une décision, pas une technique.
Règle de garde — Survivorship rule	— Le critère qui désigne la ligne conservée dans chaque groupe.	La laisser implicite ; elle doit être écrite et jouable.
Registre des doublons — Duplicate register	— Le fichier compagnon qui liste ce qui a été retiré et pourquoi.	Le jeter : il est la seule preuve que le nettoyage n'a pas inventé le total.
Dictionnaire correspondance — Crosswalk	de — Table qui relie deux écritures d'un même objet (MATÉRIAUX → Matériaux).	La coder dans une formule recopiée au lieu d'en faire une table lisible.

Français — English	Définition simple	Piège à éviter
Appariement tolérant — Fuzzy matching	Rapprochement par similarité (norme, distance d'édition), avec seuil.	Croire qu'un seuil par défaut est un jugement métier.
Clé de normalisation — Normalized key	Valeur transformée (minuscules, sans accent, sans espace, sans ponctuation) servant de point de rencontre.	Comparer des chaînes au lieu de comparer leurs normes.
Faux positif / faux négatif	Fusionner deux objets distincts / en manquer un.	Publier un score sans les deux taux : le premier détruit, le second laisse vivre le défaut.

5. Cours approfondi

5.1 Les quatre niveaux où l'on peut chercher un doublon

Niveau	Test	Ce qu'il voit ici	Ce qu'il laisse passer
1 · ligne entière	toutes colonnes égales	0 ligne	tout doublon dont une valeur a bougé
2 · clé technique	id_vente distincts	0 doublon	le doublon lui-même
3 · clé métier complète	ticket + produit + quantité	3 440 lignes	—
4 · clé métier relâchée	ticket + produit	8 453 lignes concernées	un article légitimement commandé deux fois

Le niveau 2 est celui que tout le monde exécute, parce que la clé est là, bien nommée, et que le test tient en une formule. Les niveaux 3 et 4 sont ceux qui décident. Retenez la phrase : **une clé primaire propre avec une clé métier collisionnée, ça s'appelle un doublon ; l'inverse (clé primaire en double, métier propre) ça s'appelle un accident d'export**, et ça se répare autrement.

Attention. Un décompte de doublons sans son dénominateur est une annonce, pas un résultat. « 3 440 » peut vouloir dire 1,4 % du fichier ou 40 % d'un magasin ; « 8 453 » est le nombre de lignes impliquées dans une collision, et non le nombre de lignes à retirer — la différence, 5 013 lignes, est exactement ce que l'on perd quand on lit un compteur de paires comme un compteur de lignes.

5.2 Le faux doublon : ce que la répétition a de normal

Sur 146 161 tickets, 97 199 lignes en partagent un autre. Ce n'est pas un défaut : un ticket de quincaillerie a plusieurs lignes, et **78 683 tickets n'en ont qu'une**. Le test de répétition ne dit donc rien tout seul ; c'est la question métier qui tranche : « un même article peut-il être saisi deux fois sur le même ticket, et légitimement ? » Réponse du socle, lue dans sa notice : non — une ligne par article, et la quantité porte le nombre.

Définition. Un **doublon structurel** est une répétition que le modèle de données autorise et que le métier reconnaît : deux lignes d'un même ticket, deux relevés de stock le même jour dans deux magasins. Un **doublon accidentel** est une répétition que le modèle autorise mais que le métier refuse. Les deux ont le même profil informatique ; seul le métier les sépare — d'où la ligne « règle » dans le dictionnaire de données (C06).

5.3 Anatomie des 3 360 : ce que le fichier dit de son accident

On a retiré les lignes du fichier propre, et l'on a regardé ce qu'elles étaient.

Propriété mesurée	Valeur	Ce que ça veut dire
lignes retirées	3 360	l'écart brut – propre, le seul dénominateur honnête
retirées ayant un jumeau conservé sur ticket + produit	3 360	la totalité : le mécanisme est bien la répétition d'un article
montant identique entre jumeaux	3 346	pas un copié-collé intégral : 14 divergent
quantité identique	3 346	les mêmes 14, donc la divergence porte sur le couple quantité-montant
id_vente supérieur à celui de l'original	3 360	le doublon est toujours numéroté après
écart médian des id_vente	123 285	loin de l'original : ce n'est pas une double frappe
position médiane dans le fichier	99,3 %	un bloc ajouté en fin de fichier

Lecture du détective : un bloc de lignes recopiées, numéroté après tout le reste, collé à la fin du fichier, dont le montant reproduit l'original 3 346 fois sur 3 360. Voilà le portrait-robot d'**un export rejoué sur une période déjà exportée** — pas d'une caisse qui a double-frappé (dans ce cas les `id_vente` seraient contigus, à 838 près au mieux, et les lignes seraient dispersées dans le fichier). Cette lecture ne sert pas à incriminer un collègue : elle dicte la règle de garde.

5.4 La règle de garde : laquelle des deux lignes on conserve

Trois règles se présentent, et elles ne se valent pas :

- **garder la première du groupe** (le plus petit `id_vente`) : déterministe, et alignée sur le mécanisme — l'original est toujours avant. C'est la bonne ici.
- **garder la dernière** : la règle par défaut de beaucoup d'outils de tri « supprimer les doublons », et ici elle conserve les lignes du bloc recopié — le total serait juste d'un cheveu, le détail faux de 3 360 lignes.
- **garder la plus cohérente** (dont `montant_ttc = montant_ht + montant_tva`) : celle qu'il faut ajouter pour les 14 groupes dont les jumeaux divergent, parce qu'entre deux écritures du même article, on conserve celle qui respecte la règle de gestion.

Le contrôle qui valide la règle, c'est le total : réparer les montants texte **et** dédoublonner au niveau 3 en gardant le plus petit `id_vente` rend 15 419 985 157 FCFA — l'écart à la cible est de 0. Réparer sans dédoublonner rendait 15 639 751 286 FCFA, soit +219 766 129. Les deux chiffres, dans cet ordre, sont la démonstration que la règle de garde n'est pas un détail d'implémentation.

Conseil professionnel. Écrivez la règle de garde dans le registre des doublons, avec le nombre de groupes concernés et le nombre de lignes retirées — les deux, pas un. Dans six mois, on ne vous demandera pas « combien de doublons ? » mais « pourquoi celle-là plutôt que l'autre ? », et le seul document qui y répond est le vôtre.

Définition. Le **registre des doublons** est la liste nominative de ce qui a été retiré : une ligne par groupe, avec les deux identifiants en présence et la règle qui a départagé. Ce n'est pas un journal de modifications (C06) : il ne décrit pas ce qu'on a changé dans les cellules, il décrit ce qu'on a **choisi d'oublier**.

5.5 Le registre des doublons, premier livrable de traçabilité

Le fichier `doublons_registre.md` (ou `.csv`) contient une ligne par groupe : la clé du groupe, les `id_vente` en présence, la ligne gardée, la ligne retirée, le montant de part et d'autre, la règle appliquée. Trois colonnes sont non négociables : règle, date, outil. Un nettoyage sans registre est

une transformation invisible : il rend impossible la question « est-ce que le chiffre a changé parce que le fait a changé, ou parce que vous avez décidé ? ».

Sur ce socle, le registre fait 3 360 lignes, soit 1,4 % du fichier. Il n'est pas une annexe : il est la pièce qui permet à un repreneur de refaire votre décision — ou de la défaire.

5.6 Les correspondances : quand deux écritures ne font qu'un objet

Changement d'objet, même logique. `produits.csv` contient 15 valeurs de `categorie` ; le dictionnaire de référence, 7. La table de correspondance est donc une table à 15 lignes, dont la colonne norme porte un choix :

Écriture dans le brut	Norme retenue	Motif
MATÉRIAUX, <code>Materialux</code> , <code>materiaux</code> , <code>Matériaux</code>	Matériaux	casse, accent absent, espace finale
PEINTURE, <code>peinture</code> , <code>Peintures</code>	Peinture	casse, pluriel
QUINCAILLERIE, <code>Quincaillerie</code> , <code>Quincaillerie</code>	Quincaillerie	casse, espace finale
Electricité	Électricité ou Electricité , au choix	accent sur la majuscule initiale : à trancher et à écrire

Appliquer la correspondance ne change **aucun** total : le chiffre d'affaires de la semaine reste le même, que la ligne s'appelle `Peintures` ou `Peinture`. Ce qui change, c'est le nombre de lignes du tableau croisé (15 → 7) et les 44 références dont l'étiquette bouge. C'est la différence fondamentale entre un défaut de **valeur** et un défaut de **correspondance**, et elle explique pourquoi les seconds survivent des années dans les rapports : ils ne rendent aucun total suspect.

Définition. Un **dictionnaire de correspondance** (crosswalk) est une table qui relie les identifiants ou les libellés de deux systèmes. Contrairement à une conversion, elle est **réversible et annotable** : on peut y inscrire « ces deux codes ne désignent pas la même chose », ce qu'aucune formule RECHERCHEV ne permet d'écrire.

5.7 L'appariement des clients : la règle qui ne casse pas les homonymes

412 clients du fichier brut ont été jugés doublons et retirés pour obtenir `clients_propres.csv` (23 912 → 23 500). Aucun n'a le même `id_client` (0 doublon sur cette clé), et aucun n'est une copie exacte : ce sont des **quasi**-doublons. Le test qui les trouve tous, sans rien casser :

```
clé d'appariement = minuscule(nom, sans accents ni ponctuation) + "|" + chiffres(telephone)
```

- sur le **nom seul** normalisé : 11 570 lignes fusionnables — la règle est inutilisable, elle écrase 11 158 clients qui ont simplement un nom commun (les `Menage 00002` et autres libellés de synthèse) ;
- sur **nom + chiffres du téléphone** : 412, exactement ce que la référence retire, répartis en 412 groupes de 2 lignes, jamais plus ;
- le téléphone seul n'est pas non plus une clé : 23 499 numéros distincts pour 23 912 clients, donc 413 numéros portés par au moins deux lignes.

Où sont les 412 ? Le fichier donne à voir le mécanisme, et il faut le regarder avant d'écrire la règle. Les 412 lignes retirées forment **un bloc continu**, de la ligne 23 501 à la fin du fichier : elles n'ont pas été saisies dans le désordre, elles ont été **ajoutées après coup**, chacune derrière son original (412 fois sur 412). Le jumeau porte le même nom en majuscules, encadré d'espaces (`SARL K` face à `SARL k`), et un téléphone débarrassé de ses espaces (`+2267862218198` face à `+226 78 62 21 81 98`) : 412 fois sur 412 pour les deux anomalies à la fois, **0 fois** sur les lignes

conservées. Autour du couple, tout le reste concorde : ville, courriel, type de client et date de création sont identiques dans les 412 couples (412 sur 412).

Ces trois faits se lisent d'un coup et changent le statut de la règle. D'abord, la clé `nom + chiffres` ne produit **aucun faux positif** : les 412 groupes comptent exactement 2 lignes, jamais davantage. Ensuite, ajouter la ville à la clé n'aurait rien cassé ici — et c'est précisément le piège : une colonne d'accord par hasard n'apporte aucune discrimination, elle n'ajoute que de la fragilité (1 911 clients n'ont pas de ville ; leur ligne aurait été mariée à une autre ligne sans ville). Une colonne n'entre dans une clé que si son **désaccord** doit annuler le rapprochement. Enfin, l'écriture en majuscules et l'espace en trop ne sont pas des bruitages : ce sont les traces d'un import par accumulation : on recopie, on ne fusionne pas. Le C05 les normalise ; ici, elles servent d'abord à reconnaître le bloc.

Dans les faits. Un seuil de similarité « par défaut » (0,85 sur un score de ressemblance de chaîne) appliqué à ce fichier de clients produit un tas de paires qui n'ont rien à voir, parce que les noms sont courts et que le corpus est synthétique : la bonne pratique n'est pas de régler le seuil plus haut, c'est de trouver la **colonne rare** qui discrimine — ici, le téléphone. Un appariement sans colonne discriminante est un tirage.

5.8 Les correspondances manquantes : 486 lignes et 372 clients

Un `id_client` présent dans les ventes et absent des clients n'est pas un doublon, c'est l'autre face du même problème : l'objet n'a pas de fiche. Ici, 486 lignes, sur 372 identifiants distincts. Le traitement est une table, pas une formule : `a_creer.csv` avec, par identifiant, le nombre de lignes, le premier et le dernier magasin, et la mention provisoire. Ce document a trois usages : il justifie le dénominateur de tout tableau par client, il permet de poser la question au bon service, et il empêche la jointure de faire disparaître 486 lignes sans que personne ne sache où elles sont passées.

Le contrôle symétrique doit être écrit lui aussi, avec son zéro : **0 ligne de vente ne pointe un id_produit inexistant** (154 références, toutes jointes). Un zéro non écrit est une question ouverte ; un zéro écrit est un risque fermé.

Doublons : ce que chaque niveau de clé voit, et ce qu'il laisse passer

Le même fichier, quatre questions. Une seule tombe sur le bon tas de lignes.

Ligne entièrement identique

colonne par colonne, A à Z

0 ligne

aucun copié-collé intégral dans le fichier

Clé technique · id_vente

243 360 valeurs distinctes

0 doublon

la clé du fichier est vierge de soupçon

Ticket + produit + quantité

collision sur la clé métier

3 440 lignes

le seul test qui voie le défaut

Ticket + produit, sans quantité

toutes les lignes concernées

8 453 lignes

le dénominateur à citer, pas le résultat

L'entonnoir, et ce qui reste à traiter :

243 360 lignes → 97 199 partagent un ticket (normal : 1,67 ligne par ticket)

→ 8 453 lignes en collision → 3 440 collisions sur la clé complète

→ 3 360 lignes à retirer → dont 61 aussi en montant texte (le recouvrement)

La trace du mécanisme, lue dans l'ordre des identifiants :

3 360 doublons, tous numérotés après leur original — écart médian 123 285, le plus proche 838

position médiane dans le fichier : 99,3 % du parcours. Un bloc ajouté à la fin :

signal d'un export rejoué sur une période récente, jamais d'une double saisie au clavier.

Règle de garde : le plus petit id_vente du groupe, jamais le dernier.

3 346 fois sur 3 360 le montant est identique des deux côtés ; les 14 autres divergent —

leur règle ne peut pas être « la première venue » mais « la première conforme à ht + tva ».

Attention. Un registre de 3 360 lignes ne se relit pas : il se **requête**. Écrivez-le en table structurée — une ligne par cas, des colonnes nommées — et non en prose, sinon il ne sert à rien le jour où l'on vous demande « quelles lignes avez-vous gardées pour ce ticket ? », question qui, en production, arrive toujours.

6. Exemple concret : le tableau croisé qui accuse un magasin

Un analyste charge le fichier, retire les doublons avec l'outil (bouton « Supprimer les doublons », toutes colonnes cochées, garde la première occurrence), puis livre le CA par magasin. Résultat : le fichier fait toujours 243 360 lignes moins 0, puisque **aucune** ligne n'est une copie intégrale. Son tableau croisé garde donc les 3 360 lignes en trop, et le magasin qui a reçu l'export rejoué affiche un CA surévalué — de 219 766 129 FCFA sur l'ensemble, mais concentré sur le tiers de ses journées. Personne ne voit l'erreur au total (il est gros, comme d'habitude), on la voit dans le classement des magasins : c'est ainsi qu'un défaut de doublon se manifeste en production, rarement par un chiffre aberrant, presque toujours par un **rang** qui n'est pas le bon.

7. Démonstration pas à pas : trouver, décider, journaliser

Feuille `Ventes` (243 360 × 20). Objectif : la liste des groupes, la ligne gardée, la ligne retirée.

1. **Clé métier.** Colonne utilitaire `=Ventes!B2&"|"&Ventes!H2&"|"&Ventes!I2` (ticket | produit | quantité) — à remplir sur les 243 360 lignes, puis à figer en valeurs avant le tri.
2. **Rang dans le groupe.** Colonne `=NB.SI($U$2:$U2;U2)` : il vaut 1 sur la première occurrence, 2 sur la seconde. C'est la règle de garde écrite en clair — et elle est déterministe tant que l'ordre des lignes ne bouge pas, d'où le figeage de l'étape 1.
3. **Compteur de collision.** `=SOMME.SI($U$2:$U$243361;"="&U2;$U$2:$U$243361)` remplacé par un comptage simple : `=NB.SI(Ventes!$U$2:$U$243361;U2)>1` → VRAI sur 8 453 lignes au niveau 4, sur 3 440 × 2 au niveau 3 selon la façon de compter : c'est le moment d'écrire lequel vous publiez, et de le publier avec son dénominateur.
4. **Sélection des retirées.** `=SI(ET(Ventes!$V2>1;ESTNUM(...)))` — en pratique : filtre sur `rang > 1`, copier vers une feuille `RETRAITES`, qui devient le noyau du registre (3 360 lignes attendues).
5. **Vérification arithmétique.** `=243360-3360` → 240 000, le nombre de lignes du fichier propre. Si ce compte n'y est pas, votre règle de garde n'est pas la même que celle du producteur du corrigé — et l'un des deux a tort.
6. **Vérification monétaire.** `=SOMME` des montants réparés sur les lignes conservées → 15 419 985 157 FCFA, écart à la cible : 0. Les deux vérifications (lignes et montants) sont obligatoires : l'une attrape une règle fausse, l'autre une réparation fausse.
7. **Registre.** Sur `RETRAITES`, ajouter quatre colonnes : `id_vente_original`, `motif` (« collision clé métier »), `regle` (« garde du plus petit id_vente »), `date`. Cinq minutes, et le nettoyage devient défendable.

À retenir. Un dédoublonnage se prouve par deux contrôles de sens opposé : le **compte** (243 360 - 3 360 = 240 000 lignes) et le **montant** (15 419 985 157 FCFA, écart 0 à la cible). L'un sans l'autre ne vaut rien : le compte ne voit pas une réparation fausse, le montant ne voit pas une règle de garde fausse — et ce fichier fournit les deux cas, les 14 groupes divergents et les 61 lignes à double défaut.

8. Erreurs fréquentes

Symptôme visible	Cause réelle	Correction
« Aucun doublon : la clé est unique »	seul le niveau 2 a été testé	tester le niveau 3 sur la clé métier ; 3 440 lignes ici
Suppression de 97 199 lignes valides	dédoublonnage sur id_ticket seul	définir la répétition inattendue, pas la répétition : 1,67 ligne par ticket est normal
Le total baisse un peu, puis beaucoup	règle de garde « garder la dernière » appliquée à un bloc ajouté en fin	garder le plus petit identifiant, et le justifier par la position dans le fichier
Deux lignes identiques conservées alors qu'un doublon est annoncé	comparaison de chaînes non normalisée (espace finale, accents)	normaliser avant d'appairer ; comparer des normes, pas des textes
« 15 catégories » au lieu de 7 dans le rapport	défaut de correspondance non traité	table de correspondance écrite, jamais de SI() imbriquées
Le repreneur ne retrouve pas les lignes supprimées	pas de registre	registre des doublons avec la règle, la date, l'outil ; 3 360 lignes à consigner

9. Bonnes pratiques professionnelles

Boîte à outils du chapitre. Quatre colonnes utilitaires et rien d'autre : la clé métier (concaténation des colonnes qui définissent l'objet), le rang dans le groupe (un NB.SI borné vers le haut), la conformité du montant ($=ET(ESTNUM(N2);ABS(L2+M2-N2)<=1)$), et la position dans le fichier. Avec ces quatre colonnes, le dédoublonnage, la règle de garde et le registre s'écrivent en dix formules, sans macro et sans outil tiers.

- **Écrivez la clé métier avant de chercher les doublons.** Une phrase dans le dictionnaire : « une ligne = un article d'un ticket » — et le test en découle, au lieu d'être deviné par l'outil.
- **Testez les quatre niveaux, publiez les quatre résultats.** Même les zéros : « 0 ligne entière dupliquée » est ce qui prouve que le nettoyage n'a pas détruit une seconde écriture du même article.
- **Une règle de garde = un ordre + un critère + une exception.** Ici : ordre croissant d'id_vente, critère montant conforme à ht + tva, exception sur les 14 groupes divergents, traités à la main et notés comme tels.
- **Ne supprimez jamais sans liste.** Le registre est un livrable au même titre que le fichier propre.
- **Séparez le dédoublonnage de la réparation de types.** Les 61 lignes qui cumulent les deux sont le seul cas où l'ordre des opérations change le résultat (C01, § 10) : réparez d'abord, dédoublonnez ensuite.
- **Sur une correspondance, gardez la valeur d'origine à côté.** Une colonne categorie_source permet de revenir sur un choix de norme ; l'écraser dans la colonne d'origine rend le débat impossible.
- **Faites valider la norme par le métier, par écrit.** Electricité prend-il un accent ? La réponse ne change pas un total et change une identité de marque : c'est une décision de direction, pas de technicien.

10. Exercice guidé — reproduire le fichier propre, puis prouver que c'est le bon (35 min, /10)

Étape 1. Construisez la clé métier et le rang dans le groupe (§ 7, étapes 1-2). Combien de lignes ont un rang supérieur à 1 au niveau « ticket + produit + quantité » ? → **3 360**. Si vous trouvez 3 440, vous avez compté les collisions et non les lignes excédentaires : relisez le § 5.1, la différence est exactement une ligne par groupe.

Étape 2. Appliquez la règle de garde (le plus petit `id_vente`) et comptez les lignes restantes : $243\,360 - 3\,360 = 240\,000$. Comparez au fichier propre : le compte y est. Vous n'avez rien « nettoyé au propre », vous avez retrouvé la règle du producteur — la nuance est que la vôtre est maintenant écrite.

Étape 3. Traitez les 14 groupes dont les jumeaux divergent en montant ou en quantité. Décision attendue : garder la ligne dont `montant_ttc = montant_ht + montant_tva`, et si les deux sont conformes, garder la première et l'écrire dans le registre. Ces 14 lignes sont le seul endroit du socle où la règle par défaut donne un résultat différent de la règle métier — donc le seul endroit où un correcteur pourra vous contredire si vous ne les citez pas.

Étape 4. Réparez les 3 421 montants texte (`=SI(ESTNUM(N2);N2;L2+M2)`) puis dédoublonnez. Total obtenu : 15 419 985 157 FCFA, écart à la cible **0**. Écrivez l'ordre dans le registre : réparation, puis dédoublonnage. L'inverse ne marche pas — les 61 lignes à double défaut le montrent : traitées dans l'autre ordre, l'une des deux opérations leur laisse un montant vide.

Étape 5. Rédigez les trois lignes du registre qui résument l'opération : 3 360 lignes retirées · règle de garde · deux contrôles (240 000 lignes, écart monétaire 0). **Note /10** : 2 points pour la distinction 3 440 / 3 360, 3 points pour la règle de garde écrite et appliquée, 2 points pour les 14 groupes, 3 points pour le total et son écart à zéro avec l'ordre des opérations justifié.

11. Exercices autonomes

Exercice 3.1 (★) — Les quatre compteurs (10 min). Sur `ventes_brutes.csv`, produisez les quatre résultats du § 5.1 avec, pour chacun, son dénominateur. Écrivez la phrase qui dit ce que seul le niveau 3 a vu.

Exercice 3.2 (★★) — Le faux positif du nom (15 min). Sur `clients.csv`, comptez les lignes fusionnables par (a) `id_client`, (b) nom normalisé seul, (c) nom normalisé + chiffres du téléphone, (d) téléphone normalisé seul. Donnez les quatre nombres, et indiquez celle des quatre règles qui détruirait le plus de clients réels.

Exercice 3.3 (★★) — La table de correspondance (15 min). Écrivez la table à 15 lignes qui ramène les catégories de `produits.csv` aux 7 du dictionnaire, et comptez les références dont l'étiquette change. Vérifiez que le total du fichier de ventes ne bouge pas d'un franc : dites ce que cette invariance prouve, et ce qu'elle ne prouve pas.

Exercice 3.4 (★★) — Les orphelins à créer (12 min). À partir des 486 lignes sans client référencé, produisez `a_creer.csv` : une ligne par identifiant (372), avec le nombre de lignes, le ou les magasins concernés et la mention « provisoire ». Combien de ces identifiants ont une ligne dont le montant est en texte ? (Indice : le recouvrement des deux défauts est publié.)

Exercice 3.5 (★★★) — Le double export reconstitué (25 min). En vous servant uniquement de l'ordre des `id_vente` et de la position dans le fichier, écrivez la règle qui identifie le bloc ajouté en fin, et vérifiez qu'elle retrouve les 3 360 lignes. Donnez ensuite le CA contenu dans ce bloc. Contrainte : votre règle ne doit contenir ni liste de lignes ni copié-collé, seulement des comparaisons — sinon elle ne rejoue pas le mois prochain.

12. Correction détaillée

Exercice 3.1. Niveau 1 : 0 ligne entièrement dupliquée sur 243 360. Niveau 2 : 243 360 `id_vente` distincts, donc 0 doublon de clé. Niveau 3 : 3 440 lignes en collision sur ticket + produit + quantité. Niveau 4 : 8 453 lignes impliquées dans une collision ticket + produit, dont 7 640 avec la même quantité. La phrase attendue : « le défaut est invisible à la clé primaire et aux copies intégrales ; il n'apparaît que lorsque la répétition est jugée sur ce qui définit l'objet — un article par ticket — et il faut alors retirer 3 360 lignes, pas 3 440 ».

Exercice 3.2. (a) 0 doublon : chaque ligne a son identifiant. (b) 11 570 lignes « en trop » par nom normalisé : règle inutilisable, elle retirerait 11 158 lignes de trop (11 570 – 412). (c) 412 : la règle défendable, exactement ce que le référentiel retire (23 912 – 23 500). (d) Par téléphone normalisé seul : 413 numéros portés par au moins deux lignes — 23 499 numéros distincts pour 23 912 clients. C'est (b) qui détruit le plus, et c'est précisément la règle que propose un outil « dédoublonnage intelligent » si on ne lui donne que le nom. La lecture à écrire : ni la clé technique (trop stricte : elle ne voit rien) ni le nom (trop lâche : il voit trop), mais le **couple** nom + chiffres du téléphone, chaque membre du couple étant insuffisant seul.

Exercice 3.3. La table : 15 lignes, colonnes `ecriture_brute`, `categorie_normale`, `motif` (casse / accent / espace / pluriel), `date`, `valide_par`. Les références dont l'étiquette change : **44 sur 154**. Le CA reste 15 419 985 157 FCFA avant et après correspondance, à 0 près — l'invariance prouve que la correspondance ne touche pas aux faits ; elle ne prouve pas que le **classement** est bon : c'est justement lui qui change, de 15 lignes à 7, et donc tous les pourcentages et tous les « top 3 ». Conclusion à écrire : un défaut de correspondance est un défaut de lecture, pas un défaut de montant, et il se débusque sur les tableaux, jamais sur le total.

Exercice 3.4. 372 lignes dans `a_creer.csv` ; le total des lignes concernées est 486, soit 0,2 % du fichier — faible en volume, décisif sur un classement par client. Le recouvrement avec le défaut de type est de **61** lignes : ce sont 61 des 486 orphelines dont le montant est aussi en texte, donc 61 fiches « à créer » qui ne seront pas chiffrables avant réparation du type. D'où l'ordre : réparer les types, puis créer les fiches, puis contrôler la jointure. Le pigiste qui fera l'inverse aura créé 372 fiches dont il faudra reprendre 61.

Exercice 3.5. Règle qui marche : « une ligne est dans le bloc ajouté si son `id_vente` est supérieur au plus petit `id_vente` d'un groupe en collision, et qu'elle est en position supérieure à 99 % du fichier » — mesurée, elle retrouve les 3 360, avec un écart médian de 123 285 sur les identifiants et minimum 838. Le CA du bloc est exactement l'écart constaté entre le fichier réparé non dédoublonné et la cible : 15 639 751 286 – 15 419 985 157 = **219 766 129 FCFA**. Deux remarques à intégrer au corrigé : la règle marche parce que le doublon a été ajouté en bloc — un dédoublonnage écrit sur la position dans le fichier est une règle fragile, qui ne survit pas à un réordonnancement ; et le contrôle de cohérence entre les deux approches (position, montant) est la preuve qu'il ne s'agit pas d'un artefact d'écriture. La version robuste, celle qu'on livre, est la clé métier plus la règle de garde : la position ne sert qu'à comprendre, pas à décider.

13. Mini-projet M04.P3 — « Le dossier de dédoublonnage » (1 h 30)

Livrables, tous exigés, aucun optionnel :

1. `cle_metier.md` : la définition écrite de l'objet (une ligne = ?), la ou les clés candidates, la raison du rejet des autres — avec les compteurs des quatre niveaux ;
2. `doublons_registre.csv` : les 3 360 lignes retirées, leurs jumeaux conservés, la règle, la date, et la colonne `divergent` (les 14 groupes) ;
3. `a_creer.csv` : les 372 clients sans fiche, avec leur volume de lignes (486) et le drapeau texte (61) ;

4. `correspondances.csv` : les 15 écritures de catégorie et leur norme, plus les deux écritures de téléphone (espacée / collée) traitées par la même table ;
5. `controles.md` : les trois vérifications de fin — 240 000 lignes, total 15 419 985 157 FCFA, écart 0 — avec la formule ou l'outil qui les a produites ;
6. une page de décision : ce que vous auriez fait de différent si le bloc dupliqué avait été dispersé dans le fichier au lieu d'être en fin.

Barème (10 points). Clé métier écrite et défendue (2) · registre complet, règle de garde explicite (3) · correspondances et `a_creer` utilisables par un tiers (2) · contrôles chiffrés avec méthode (2) · page de décision (1). Seuil de validation : 7. Un rendu sans la page de décision plafonne à 8 : la capacité à dire ce qui changerait est la compétence, pas le compteur.

14. Résumé du chapitre

CE QU'ON CHERCHE	TEST	RÉSULTAT DU FICHIER	DÉCISION
copie intégrale	égalité colonne à colonne	0 ligne	rien à retirer
clé technique	<code>id_vente</code> distincts	243 360 / 243 360	la clé ne dit rien
clé métier	ticket + produit + quantité	3 440 en collision	3 360 à retirer
jumeaux clients	nom normalisé + chiffres tél.	412 groupes de 2	412 à retirer
correspondance	15 écritures → 7 normes	44 sur 154	total inchangé,
orphelins	<code>id_client</code> hors référentiel	486 lignes / 372	classement faux

Deux leçons se tiennent par la main. La première : **le doublon se définit, il ne se voit pas** — et sa définition est une phrase sur le monde (« une ligne par article sur un ticket »), pas une option de menu. La seconde : la correspondance est le même problème vu de l'autre côté, deux écritures pour un objet — avec cette différence de poids qu'elle laisse le total intact, ce qui la rend invisible au contrôle le plus courant. Le dénominateur commun des deux, c'est le registre : ce que vous avez fusionné, ce que vous avez retiré, pourquoi, avec quel ordre d'opérations.

15. À retenir

À retenir. ★ Une clé primaire propre n'innocente pas le fichier. 243 360 identifiants distincts et 3 360 lignes en trop : le doublon se cherche sur la clé métier, sinon il ne se trouve pas. ★ Retirer, ce n'est pas compter les collisions. 8 453 lignes impliquées, 3 440 en collision, 3 360 à retirer : les trois nombres sont vrais, un seul est une liste d'action. ★ La règle de garde se justifie par le mécanisme. Tous les doublons portent un `id_vente` supérieur à celui de leur original (écart médian 123 285) et siègent à 99,3 % du fichier : on garde le plus petit identifiant, et on l'écrit. ★ Réparer avant de dédoubler. 61 lignes cumulent les deux défauts ; dans l'autre ordre, elles ressortent avec un montant vide et le total ne se réconcilie plus (écart de +219 766 129 FCFA). ★ Une correspondance ratée ne casse pas le total, elle casse le classement. 15 catégories au lieu de 7, 44 références à retoucher : aucun contrôle de montant n'aurait signalé cela, seul le regard sur le tableau l'a fait.

16. Évaluation formative (auto-correction, 8 min)

Q1. Un fichier a une clé primaire sans doublon et 0 ligne entièrement dupliquée. Combien de lignes en trop peut-il contenir ? (a) 0 ; (b) autant que le permet une collision sur la clé métier ; (c) impossible à savoir sans la table clients. → **(b)**, et l'exemple vaut 3 360 lignes.

Q2. Pourquoi ne pas dédoubler sur `id_ticket` ? → parce que 97 199 lignes partagent légitimement un ticket (1,67 ligne par ticket, maximum 6) : la répétition est structurelle, pas accidentelle. Le test utile ajoute le produit, et la quantité pour trancher les cas ambigus.

Q3. Deux lignes d'un même groupe ont des montants différents (14 groupes sur 3 360). Quelle ligne garder ? → celle dont le montant satisfait $\text{montant_ht} + \text{montant_tva} = \text{montant_ttc}$; à égalité, la plus petite en `id_vente`, **et on le note dans le registre**. Une règle silencieuse n'est pas une règle.

Q4. Vrai ou faux : « une table de correspondance modifie les montants ». → **Faux**. Elle modifie les regroupements. C'est pourquoi elle se valide sur les classements et sur les effectifs (154 références, 44 étiquettes changées), jamais sur le total.

Q5. Vous devez livrer le nettoyage à un repreneur. Quels artefacts, dans quel ordre ? → dictionnaire de correspondance, registre des doublons (3 360 lignes, règle, date), liste des orphelins à créer (372), et les trois contrôles finaux avec leur formule — avant le fichier lui-même, qui se régénère.

Question ouverte (la plus importante). Reprenez la règle « garder la ligne en position la plus haute » (le réflexe de beaucoup d'outils de tri) et démontrez, chiffres à l'appui, ce qu'elle aurait produit ici. Corrigé attendu : elle conserve le bloc recopié de fin de fichier, donc le **même nombre** de lignes (240 000) et un total identique puisque les montants sont les mêmes 3 346 fois sur 3 360 — mais les 14 divergents se retrouvent du mauvais côté, et les détails par ticket sont faux. La leçon n'est pas que la règle est fausse au total : c'est qu'un nettoyage peut être juste sur les deux seuls contrôles que tout le monde fait (nombre de lignes, somme) et faux partout ailleurs. Voilà pourquoi le registre existe.

Module M04.C04 — Valeurs aberrantes et règles de contrôle : une règle doit rendre une décision, pas un nombre

Outil de ce chapitre : le tableur, sur `ventes_brutes.csv`, `produits.csv` et `stocks_quotidiens.csv`. **Durée indicative :** 5 h. **Niveau :** N2.

L'idée du chapitre. On vous demandera de « repérer les valeurs aberrantes », et l'outil répondra avec une formule toute faite : quartiles, écarts-types, seuils à trois sigmas. Appliquée à notre fichier de ventes, cette recette signale **24 180 lignes** de prix — 9,9 % du fichier — et ne désigne pas une seule erreur. La règle qui voit juste s'appuie sur le référentiel des produits, et elle signale **0 violation** : ce zéro est une preuve, pas une paresse. Le chapitre garde un deuxième tour de vis, plus désagréable : sur les 973 lignes où la remise est aberrante, la réparation évidente ne répare rien — diviser par 100 laisse 258 lignes hors du domaine contractuel, parce que la valeur fausse n'a jamais été la valeur juste multipliée par cent. Une valeur aberrante n'est pas une valeur éloignée : c'est une valeur **incompatible avec un document**.

Base de travail. `01_socle_donnees/data/brut/ventes_brutes.csv` (243 360 lignes × 20 colonnes), `01_socle_donnees/data/brut/produits.csv` (154 références), `01_socle_donnees/data/brut/stocks_quotidiens.csv` (60 000 lignes), `01_socle_donnees/data/reference/ventes_terrain.csv` (15 419 985 157 FCFA).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **classer** une anomalie en trois natures — impossible, invraisemblable, licite-surprenante — et traiter chacune autrement ;
- **écrire** une règle de contrôle qui renvoie une décision : bloquer, corriger, surveiller, ignorer tracé ;
- **calibrer** un seuil sur deux preuves — la distribution observée et le document métier — et dater cette calibration ;
- **préférer** le contrôle croisé (colonne contre référentiel) au contrôle univarié, en sachant ce qu'il ne dit pas ;
- **mettre à l'épreuve une réparation** : avant d'appliquer ± 100 , vérifier qu'elle ramène les valeurs dans le domaine.

2. Pourquoi cette notion est importante

- Les prix unitaires du fichier vont de 575 à 45 000 FCFA, médiane 6 400, quartiles 3 650 et 11 050, écart-type 8 372. La règle « hors de $[Q1 - 1,5 IQR ; Q3 + 1,5 IQR]$ » marque **24 180 lignes** ; « au-delà de 3σ », **3 894** ; « 6σ », **0**. Aucune de ces lignes n'est une erreur : le magasin vend du sac de ciment et de l'ossature, et la dispersion est le métier.
- La règle juste est relative, pas absolue : le prix d'une ligne doit rester dans la bande négociée autour du catalogue. Ratio observé : **0,958 à 1,306** ; violations d'une bande $[0,95 ; 1,35]$: **0**. En revanche, exiger l'égalité au franc avec le catalogue signaler **232 981 lignes** sur 152 produits des 154 — la règle est fausse, pas le commerce.

- Le défaut le plus instructif du socle ne casse **aucun** total : les 973 lignes à remise aberrante ont un `montant_ht` correct et une `taux_remise` fausse (la TVA, elle, rend 18 % du HT sur les 243 360 lignes, à l'unité près : 0 écart). Un contrôle de cohérence des montants ne verra jamais ce défaut ; seule la règle de domaine le voit — et il ne coûte que des analyses de remise fausses.
- Enfin, la règle d'alerte fournie avec les stocks est un cas d'école d'aberration de contrôle : `qte_physique < seuil_alerte` déclenche sur 1 800 lignes, exactement comme `qte_physique < 0`. Le seuil ne filtre rien.

3. Explication simple

Un médecin ne réagit pas de la même façon à une tension à 90, à 220 ou à zéro. Il distingue l'**impossible** (on réanime, on ne débat pas), l'**invraisemblable** (on soupçonne le brassard avant de traiter) et le **licite surprenant** (90 chez un coureur : on note). Les données ont la même échelle, et l'erreur du débutant est de tout traiter au milieu — parce que c'est là que se trouvent les seuils par défaut des outils.

La seconde différence avec la médecine est dans la conclusion : un bilan rend une **conduite à tenir**, pas un pourcentage de patients hors normes. Un contrôle de données doit rendre la même chose : bloquer la ligne, corriger par la règle, surveiller le lot, ignorer avec trace. Si votre règle n'autorise aucune de ces quatre phrases, ce n'est pas encore une règle : c'est un compteur, et un compteur ne protège personne.

Définition. Une **règle de contrôle** est une affirmation vérifiable sur une donnée, assortie d'une décision et d'un destinataire : « le prix d'une ligne reste dans la bande négociée du catalogue, sinon la ligne est retournée au magasin pour validation ». Les trois membres — affirmation, décision, destinataire — sont obligatoires ; en production, deux sur trois ne suffisent jamais.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Valeur aberrante — Outlier	Valeur qui surprend au regard du contexte.	La confondre avec une erreur : une surprise peut être parfaitement vraie.
Impossible — Invalid	Contredit une loi du domaine (remise hors [0 ; 1], quantité négative hors retour).	La laisser passer : c'est la seule catégorie qu'on bloque à l'entrée.
Invraisemblable Implausible	— Compatible avec le domaine, mais contredit un document voisin.	La corriger sans preuve : on remplace un fait possible par une moyenne.
Licite surprenante Legitimate extreme	— Hors distribution, juste dans le métier.	La supprimer « parce qu'elle fausse la moyenne ».
Règle univariée Univariate rule	— Contrôle porté par une seule colonne (seuil, IQR, z-score).	Son incapacité à voir les défauts de relation entre colonnes.
Contrôle croisé — Cross-field rule	Confronte deux colonnes, ou une colonne et un référentiel.	L'écrire sur une jointure non validée : l'orphelin devient une anomalie fantôme.
Test de réparation — Repair plausibility check	Vérifier que la correction proposée ramène la valeur dans le domaine.	Appliquer ± 100 sans vérifier : ici, 258 lignes restent hors domaine après « réparation ».
Échelle de gravité Severity ladder	— Bloquant · corrigeable · surveillé · ignoré tracé.	Un seul niveau : tout devient urgent, donc plus rien ne l'est.
Taux de déclenchement Alert rate	— Part des lignes signalées par une règle.	Un taux de 9,9 % : c'est un descripteur de colonne, pas une alerte.

Français — English	Définition simple	Piège à éviter
Faux négatif silencieux — Silent false negative	Le défaut que la règle ne pouvait pas voir.	Croire qu'aucune alerte prouve l'absence de défaut.

5. Cours approfondi

5.1 Les trois natures, comptées sur le fichier

Nature	Test sur le socle	Lignes	Lecture
Impossible	<code>quantite ≤ 0</code> hors retour	0	les 2 848 quantités négatives sont toutes des retours signalés
Impossible	<code>taux_remise < 0</code>	0	aucune remise négative
Impossible	<code>montant_tva ≠ 18 % du HT</code>	0	l'arithmétique fiscale tient sur les 243 360 lignes
Impossible	<code>id_produit</code> absent du catalogue	0	aucune ligne sans produit référencé
Invraisemblable	<code>montant_ht ≠ quantite × prix × (1 - remise)</code>	973	exactement le lot des remises > 1
Licite surprenante	<code>quantite > 50</code>	800	commandes de chantier, plausibles en quincaillerie
Licite surprenante	<code>poids_kg > 5 000</code>	7 674	sacs palettisés et ossatures, médiane 9,5 kg par unité

Quatre zéros d'abord, puis un nombre, puis deux. Cet ordre est le plus rentable qui soit : établir ce qui est impossible et vérifier qu'on n'en trouve pas, puis chercher ce qui contredit un document, enfin qualifier ce qui n'est qu'étonnant. Un diagnostic qui commence par le troisième étage produit des rapports de 24 180 lignes signalées et 0 ligne corrigée — c'est-à-dire un rapport qui se jette.

Attention. Le nombre de lignes signalées ne mesure pas la qualité d'une règle. « Prix différent du catalogue au franc près » signale 232 981 lignes et 152 des 154 produits : c'est la règle qui est fausse, pas la moitié du commerce. Une règle se juge à trois choses : ce qu'elle attrape, ce qu'elle laisse passer, et ce qu'elle oblige à faire.

5.2 Quatre familles de règles, et leur rendement mesuré

- **Le domaine** — `remise ∈ [0 ; 1]` : **973 lignes**, maximum lu 30. C'est la seule famille qui n'ait **aucun paramètre à justifier** : la borne vient de la définition de la variable. Premier contrôle à écrire, et le seul qui se défend tout seul.
- **Le seuil métier** — `quantite ≤ 50` : **800 lignes**. Il exige un document (politique d'achat, bon type). Sans document, c'est un IQR déguisé, avec un chiffre rond de plus.
- **La bande relative au référentiel** — prix de la ligne dans [0,95 ; 1,35] du catalogue : **0 violation**, ratio observé 0,958 à 1,306. Le zéro a une valeur parce que la bande est serrée et que les extrêmes sont cités.
- **La cohérence entre colonnes** — `poids_kg = quantite × poids_unite_kg` : **8 380 lignes**, à instruire. C'est le seul des quatre qui signale un doute sans désigner le coupable : la colonne poids, le poids unitaire du catalogue, ou la quantité.

Définition. Un **test de réparation** consiste à appliquer la correction en simulation et à repasser la règle de domaine dessus. Les 973 remises à 30, 25, 18, 12 divisées par 100 donnent 0,30 ; 0,25 ; 0,18 ; 0,12 — et **258 lignes restent hors du domaine** (la remise maximale contractuelle est 0,25). Une réparation qui ne fait pas passer le test n'est pas une réparation, c'est un changement de conventions fait à l'aveugle.

5.3 Ce que le fichier dit vraiment des 973 lignes

Reconstituons la remise depuis les montants : $\text{remise} = 1 - \text{montant_ht} / (\text{quantite} \times \text{prix})$. Résultat sur les 973 lignes : étendue 0,000 à **0,110**, et **973 sur 973** dans le domaine [0 ; 0,25]. Autrement dit :

- le `montant_ht` est **juste** (il est cohérent avec la TVA, donc avec le total, donc avec le chiffre d'affaires) ;
- la colonne `taux_remise`, elle, ne décrit plus rien de mesurable : ni 30 %, ni 0,30, ni un montant de remise ;
- 630 de ces lignes ont même un `montant_ht` strictement égal au prix plein, ce qui aurait été impossible avec une remise de 12 à 30 % appliquée.

Attention. « Le total est bon » n'est pas un contrôle, c'est une coïncidence tranquille. Les 973 lignes ont un `montant_ht`, une TVA et un total parfaitement cohérents : vos rapprochements de chiffres d'affaires les plus soignés ne peuvent rien voir. Un contrôle de total vérifie une **égalité entre colonnes**, pas la véracité d'une colonne de taux — et c'est pour cette raison que le registre doit couvrir les colonnes déclaratives, pas seulement les colonnes monétaires.

La leçon déborde largement cet exemple. **Le défaut le plus grave d'un fichier n'est pas forcément celui qui change les totaux** : ceux-là ne changent aucun montant et rendent toutes les analyses de remise fausses — le genre de défaut qui survit des années parce que le contrôle de cohérence des totaux est vert. D'où deux réflexes : (1) une règle de domaine sur chaque colonne déclarative, pas seulement sur les montants ; (2) avant de « corriger » une valeur, chercher si le fichier permet de la **reconstituer** — ici oui, et la reconstitution tient sur 973 lignes sur 973, ce qui n'est pas le cas de la division par cent.

Conseil professionnel. Ne dites jamais « j'ai corrigé les remises ». Dites : « j'ai recalculé la remise à partir des montants, la valeur saisie ne pouvant plus être reliée à rien ; 973 lignes concernées, toutes revenues sous le plafond de 0,25 ». La première phrase est une opération, la seconde est une décision — et une seule des deux se défend devant un auditeur.

5.4 L'échelle de gravité, appliquée au socle

Niveau	Action	Défaut affecté ici	Ce que ça coûte
1 · bloquant	la ligne n'entre pas dans la base	aucun (0 impossible)	—
2 · corriger par la règle	recalcul documenté, puis recontrôle	3 421 montants texte · 973 remises à reconstituer	2 111 789 FCFA · toutes les analyses de remise
3 · surveiller	on garde, on suit, on interroge	8 380 poids incohérents · 800 quantités > 50	un devis de transport faussé
4 · ignorer, tracé	on écrit pourquoi on ne touche pas	51 dates après le 31/08/2026, avant d'être datées	une comparaison annuelle faussée

Le premier niveau est vide sur ce fichier : le producteur a validé les types à l'entrée, et l'écrire est utile — ça disculpe le fichier et charge le contrôle. Le niveau 4 est celui qu'on saute, et c'est le plus coûteux : ignorer sans tracer, c'est transformer une date de 2026-12 en une hausse artificielle lors de la comparaison annuelle.

Définition. Un **registre de contrôles** est la table qui énonce, pour chaque colonne ou groupe de colonnes : la règle, son niveau, sa cadence, son destinataire, son taux de déclenchement attendu et sa mesure du jour. C'est l'objet que laisse un analyste qui part — la liste des règles, pas la liste des défauts.

5.5 Une règle rend une décision : la forme à écrire

```
règle      : le prix unitaire d'une ligne reste dans la bande [0,95 ; 1,35] du prix catalogue
destination : magasin émetteur, courriel hebdomadaire, liste nominative des lignes
niveau     : 3 · surveiller – le catalogue n'est pas la facture
exception  : remises contractuelles hors bande => liste de produits à joindre au contrôle
mesure     : 0 violation · ratio observé 0,958 à 1,306 · 243 360 lignes passées · calibré le
[date]
```

Cinq lignes, et n'importe qui peut reprendre le contrôle six mois plus tard, savoir s'il a dérapé et à qui il s'adresse. Comparez avec « j'ai appliqué le seuil IQR sur les prix, il y a 24 180 outliers » : même travail de calcul, zéro exploitabilité — et, dans un an, quelqu'un remettra cette règle en production.

5.6 Calibrer un seuil : deux preuves, pas une habitude

Toujours le même mouvement. D'abord la **distribution** : 575 à 45 000 FCFA, médiane 6 400, quartile 3 à 11 050, écart-type 8 372 — elle dit où est le corps du fichier et où commencent les queues. Ensuite le **document** : politique commerciale, contrat de marge, seuil de commande de gros. Calibré sur la seule distribution, un seuil est descriptif ; calibré sur le seul document, il est inapplicable s'il contredit la pratique. Ici les deux se rejoignent sur la bande de prix, et se contredisent sur le poids : 7 674 lignes dépassent 5 000 kg alors que la médiane de poids unitaire est de 9,5 kg — le désaccord ne se tranche pas dans le fichier, il s'écrit au niveau 3 et se porte au magasin.

Dans les faits. Un seuil se périmé plus vite qu'une règle de domaine. La bande de prix survivra à l'année ; « quantité ≤ 50 » mourra à l'ouverture d'un dépôt ou d'un compte chantier. Écrivez la **date de calibration** à côté du seuil : c'est ce qui permet de dire, deux ans plus tard, que le bruit vient du seuil et non des saisies.

Définition. Deux règles sont **redondantes** quand elles signalent exactement le même jeu de lignes. Le test coûte une formule — la somme du produit des deux drapeaux — et rapporte gros : une règle redondante est une règle à supprimer, parce qu'elle double le volume d'alertes sans jamais rien apprendre. C'est le cas, ici, de `qte_physique < seuil_alerte` et `qte_physique < 0` sur les stocks.

5.7 Retourner une règle qui hurle : le cas des stocks

`stocks_quotidiens.csv` fournit `qte_theorique`, `qte_physique`, `seuil_alerte` sur 60 000 lignes. La règle livrée, `qte_physique < seuil_alerte`, déclenche sur 1 800 lignes ; `qte_physique < 0` déclenche sur 1 800 lignes aussi. La coïncidence dit l'essentiel : le seuil n'ajuste rien, c'est le signe

qui décide. Trois sorties, et il faut en choisir une — le pire étant de laisser les deux règles en parallèle, ce qui double le bruit pour un seul message :

- **scinder** : intégrité ($qte_physique < 0$, 1 800 lignes, niveau 2, à instruire) et risque ($qte_theorique - qte_physique > k$, à calibrer sur l'écart observé : maximum 532, moyenne 2,77) ;
- **relever le seuil** pour ramener le taux d'alerte à ce que l'exploitation peut traiter par jour ;
- **assumer une alerte large mais hiérarchisée** : les plus gros écarts d'abord, le reste en liste.

Le point n'est pas la valeur de k : c'est qu'un contrôle se lit aussi à son **taux de déclenchement**. 3 % d'alerte sur un stock quotidien, ça se traite. 9,9 % sur une colonne de prix, c'est une description du fichier, pas un contrôle.

5.8 Les règles statistiques, remises à leur place

Règle	Paramètre	Sur <code>prix_unitaire_ht</code>	Lignes signalées	Erreurs désignées
Fourchette inter-quartiles	$1,5 \times IQR$	Q1 3 650 · Q3 11 050	24 180	0
Écart-type	3σ	σ 8 372	3 894	0
Écart-type	6σ	idem	0	0
Bande relative au catalogue	[0,95 ; 1,35]	ratio 0,958 → 1,306	0	0, et c'est un résultat

Trois de ces quatre lignes disent la même chose : la distribution des prix est large et saine. Ce que les règles statistiques font bien, ce n'est pas contrôler, c'est **surveiller un changement de nature** : le nombre de lignes hors IQR, suivi hebdomadairement, devient un bon indicateur le jour où un produit change de gamme ou où une unité de mesure dérape. Tant qu'il reste à 24 180, il ne signale rien ; s'il double, il signale que le fichier n'est plus le même — pas qu'il est sale.

Quatre familles de règles — rendement mesuré sur 243 360 lignes

Le nombre de lignes signalées ne dit rien de la qualité de la règle : la deuxième colonne si.

Famille	Règle	Signalées	Erreurs réellement désignées	Décision
Domaine	remise dans [0 ; 1]	973 lignes	973 erreurs désignée 2 · corriger par reconstitution	
Seuil métier	quantité au-dessus de 50	800 lignes	0 erreur — à surveiller	3 · liste au magasin
Bande au référentiel	prix dans [0,95 ; 1,35]	0 ligne	0 : le zéro est un résultat · 3 · indicateur de dérive	
Statistique (IQR)	Q1 - 1,5 IQR à Q3 + 1,5 IQR	24 180 lignes	0 erreur — bruit pur	retirer du registre
Égalité au catalogue	prix = prix affiché	232 981 lignes	0 erreur — règle fausse	retirer du registre
Le contrôle qui résiste à tout le monde : la réparation avant de corriger 973 remises à 12, 18, 25 ou 30. Divisées par 100 : 258 lignes restent hors du plafond de 0,25. Reconstituées depuis les montants : 0,000 à 0,110, les 973 dans le domaine. Le montant, lui, n'a jamais été faux.				

6. Exemple concret : la ligne à 75 662 kg

Une ligne pèse 75 662 kg quand la médiane du fichier est de 9,5 kg. Réflexe outillé : alerte, exclusion, « nettoyage ». Regardons la ligne : sa quantité, le poids unitaire du produit au catalogue, et le calcul.

- Si `quantite × poids_unité` rend le même poids, la ligne est **juste** : une palette d'enduit ou un chargement de ciment pèse, et le magasin a saisi une quantité, pas une erreur. Rien à faire, sinon s'assurer que le destinataire du devis de transport lit bien des poids totaux.
- Si le calcul contredit, on a une ligne à instruire : quantité fausse, ou poids unitaire du catalogue faux. Et dans ce second cas, ce ne sont pas 8 380 lignes à reprendre mais **une colonne de référence** — la discussion change d'interlocuteur, et de service.

C'est toute la supériorité du contrôle croisé sur le contrôle univarié : le premier dit quelle table est suspecte, le second dit seulement quelle ligne est grosse.

7. Démonstration pas à pas : la grille de contrôles en une heure

Feuille Ventes (243 360 × 20), feuille Produits (154 × 11), feuille CONTROLES.

1. **Joindre le référentiel.** Deux colonnes utilitaires par RECHERCHEX : `prix_ref` et `poids_ref` depuis Produits. Contrôler le joint d'abord : `=ESTNA(prix_ref)` sommée doit rendre 0 — et il rend 0 sur les 243 360 lignes. Un contrôle croisé assis sur une jointure cassée produit un faux positif de masse.
2. **Ratio et bande.** `=J2/prix_ref2`, puis `=SI(OU(r<0,95;r>1,35);1;0)` sommée → 0. Écrivez à côté les deux extrêmes observés (0,958 et 1,306) : ce sont eux qui justifient la bande, et c'est ce que regardera le repreneur.
3. **Domaine.** `=NB.SI(K2:K243361;">1")` → 973. Puis, au lieu de corriger : `=1-(L2/(I2*J2))` recopée, et `=NB.SI(reconstituée;">0,25")` → 0. La reconstitution tient ; `=NB.SI(K2:K243361/100;">0,25")` aurait rendu
4. Gardez les deux lignes dans la fiche : c'est la démonstration, pas l'opération.
5. **Seuil de saisie.** `=NB.SI(I2:I243361;">50")` → 800. Niveau 3, liste au magasin.
6. **Cohérence physique.** `=SI(ABS(R2-(I2*poids_ref2))>1;1;0)` sommée → 8 380, avec une colonne de plus que d'habitude : « table suspecte ? » — le contrôle ne le dit pas, l'enquêteur si.
7. **Taux de déclenchement.** Une colonne par règle : alertes ÷ 243 360. 973 à 0,4 %, 8 380 à 3,4 %, 24 180 à 9,9 %. C'est ce tableau qui se montre en réunion, et la dernière ligne est une règle à retirer du registre.
8. **Fiche.** Une ligne par règle : règle · niveau · destinataire · cadence · taux attendu · mesure · date. Douze règles maximum : au-delà, la fiche n'est pas lue, et un contrôle non lu est un contrôle qui ne protège pas.

À retenir. Une règle se juge d'abord à son taux de déclenchement et à la décision qu'elle attache à une alerte. 0 ligne signalée avec une bande justifiée vaut mieux que 24 180 avec un quartile ; et 1 800 alertes qui ne distinguent pas le négatif du risqué ne protègent personne.

Boîte à outils du chapitre. Cinq colonnes utilitaires et rien d'autre : le ratio au référentiel, la conformité arithmétique (`=ET(ESTNUM(N2);ABS(L2+M2-N2)<=1)`), le domaine (un `ET()` borné), la reconstitution (la valeur recalculée depuis ce qui est sûr), et le test de réparation (la correction simulée repassée au domaine). Avec cela, les douze règles usuelles d'un fichier de ventes s'écrivent en une matinée, sans macro et sans outil tiers.

8. Erreurs fréquentes

Symptôme visible	Cause réelle	Correction
« 24 180 valeurs aberrantes dans les prix »	seuil statistique sur une colonne à dispersion structurelle	bande relative au catalogue (0 violation) ; la règle statistique devient un indicateur de dérive
« 232 981 lignes ne respectent pas le catalogue »	le catalogue n'est pas le contrat	bande de ratio, jamais l'égalité au franc
« J'ai divisé les remises par 100, c'est corrigé »	la valeur fausse n'était pas la valeur juste $\times 100$	test de réparation : 258 lignes restent hors domaine ; reconstituer depuis les montants
Alerte quotidienne sur la même colonne, sans suite	seuil qui ne discrimine pas ($qte < \text{seuil} \equiv qte < 0$)	scinder intégrité et risque ; calibrer sur l'écart observé (max 532, moyen 2,77)
Le contrôle ne signale plus rien depuis des mois	jointure cassée : les lignes orphelines sortent du périmètre	contrôler le taux de jointure avant le taux d'alerte (486 lignes, 372 clients)
Les anomalies sont « lissées » pour faire taire l'alerte	une valeur licite écrasée par une moyenne	niveau 3 = instruire, jamais réécrire ; conserver la colonne <code>_source</code>
Un seul niveau de gravité dans toute la fiche	tout devient urgent, donc rien ne l'est	quatre niveaux, un destinataire par niveau, une cadence

9. Bonnes pratiques professionnelles

- **Les règles de domaine d'abord, les seuils ensuite.** Elles ne demandent aucune justification statistique et ne bougent pas avec les saisons : remise en proportion, quantité entière, date renseignée, taux de TVA unique.
- **Une règle, un destinataire nommé.** « Magasin émetteur, chaque lundi » devient une habitude en un mois ; une alerte sans destinataire devient du bruit en trois semaines.
- **Écrivez le taux attendu avant la règle.** « Doit signaler entre 0,1 % et 0,5 % des lignes » : au-delà, c'est la règle qui a dérapé. La phrase est à écrire avant, sinon on la réécrit après coup pour coller au résultat.
- **Ne supprimez jamais une ligne pour faire taire une alerte.** Vous ignorez encore si l'erreur est dans la ligne ou dans la table de référence — c'est précisément ce que cherche l'étape 5 de la démonstration.
- **Testez toute réparation en simulation**, puis revalidez le domaine et la cohérence. Une correction qui fait passer un `> 1` en un `> 0,25` n'a rien réparé, elle a changé de convention sans autorisation.
- **Gardez la valeur d'origine dans une colonne `_source`** et la valeur retenue à côté : les deux sont des données ; la seconde sans la première est une réécriture de l'histoire.
- **Un contrôle croisé sans contrôle de jointure est un mensonge partiel.** Comptez les lignes qui n'ont pas trouvé leur référence (0 côté produits, 486 côté clients) avant de publier le résultat du contrôle.
- **Testez une règle en l'inversant.** Serrez la bande à $\pm 0,1$ % : 232 981 lignes. L'absurdité est plus parlante que n'importe quel discours de robustesse, et elle se démontre en une formule.

10. Exercice guidé — la règle de prix, de la distribution à la décision (35 min, /10)

Étape 1. Décrivez `prix_unitaire_ht` en cinq nombres : minimum, premier quartile, médiane, troisième quartile, maximum. Attendu : 575 · 3 650 · 6 400 · 11 050 · 45 000 (écart-type 8 372). Cette étape ne juge rien, elle dimensionne — mais écrivez-la : un seuil posé sans distribution sera relevé par quelqu'un d'autre, et vous ne pourrez pas expliquer le vôtre.

Étape 2. Appliquez IQR à $1,5 \times$, puis z-score à 3σ et à 6σ . Attendu : 24 180 · 3 894 · 0 lignes signalées. Le rapport est de six à un entre les deux premiers, et de l'infini à zéro entre le deuxième et le troisième. Une règle dont le résultat varie de six fois selon son curseur n'est pas une règle : c'est un curseur. Notez aussi que la borne basse de l'IQR tombe à -7 450 : un contrôle dont une borne est inatteignable par construction ne mesure que la moitié de ce qu'il prétend mesurer.

Étape 3. Construisez le contrôle croisé : ratio au catalogue, violations de [0,95 ; 1,35]. Attendu : 0, pour un ratio de 0,958 à 1,306. Un 0 ne vaut que si la bande est serrée et que vous citez les extrêmes : avec une bande [0,5 ; 5], le même 0 ne dirait rien du tout.

Étape 4. Traitez les 973 remises. Deux voies : $\div 100$ (puis contrôle du domaine : 258 lignes hors [0 ; 0,25]) et reconstitution depuis les montants (0,000 à 0,110, 973 sur 973 dans le domaine). Écrivez la décision au format du § 5.5, en nommant ce que la réparation choisie ne restaure pas : le montant_{ht} n'avait pas besoin de l'être. Si vous avez écrit « correction par $\div 100$ », l'étape 3 de la démonstration vous a échappé — retournez-y.

Étape 5. Décidez de la sorts des règles : bande de prix → niveau 3 · domaine remise → niveau 2 avec reconstitution · IQR → retiré, phrase de retrait écrite. **Note /10** : 3 points pour la distribution et les trois compteurs, 2 points pour la bande justifiée par les extrêmes, 3 points pour le test de réparation des 973 lignes, 2 points pour l'affectation des niveaux et la phrase de retrait. Un exercice sans décision écrite plafonne à 5.

11. Exercices autonomes

Exercice 4.1 (★) — Trois natures (10 min). Classez en impossible / invraisemblable / licite-surprenant, en citant le document qui permet de trancher : 2 848 quantités négatives · 800 quantités supérieures à 50 · 973 remises supérieures à 1 · 7 674 poids supérieurs à 5 000 kg · 51 dates postérieures au 31/08/2026 · 3 421 montants non numériques.

Exercice 4.2 (★★) — IQR contre bande (15 min). Sur prix_unitaire_{ht}, calculez les bornes IQR, le nombre de lignes hors bornes, puis les violations de la bande [0,95 ; 1,35]. Laquelle des deux règles garde le registre, et quelle phrase justifie le retrait de l'autre ?

Exercice 4.3 (★★) — La règle qui hurle (15 min). Stocks : 1 800 lignes sous le seuil d'alerte, 1 800 quantités physiques négatives, écart théorique-physique maximal 532, moyen 2,77, sur 60 000 lignes. Proposez deux règles au lieu d'une, avec pour chacune le taux de déclenchement estimé et le destinataire. Que devient le taux si vous bornez l'écart à 100 unités, et pourquoi ce n'est pas la question centrale ?

Exercice 4.4 (★★) — Le test de réparation (12 min). Prenez la remise maximale lue (30) et passez-la au domaine après division par 100. Combien de lignes restent hors du plafond de 0,25, et qu'est-ce que ça dit de l'hypothèse « pourcentage saisi en points » ? Recalculez la remise depuis les montants : quelle étendue obtient-on ?

Exercice 4.5 (★★★) — Le registre complet (25 min). Rédigez le registre de contrôles du fichier de ventes : douze règles maximum, avec règle, niveau, destinataire, cadence, taux attendu et mesure du jour. Vous devez y faire figurer au moins un contrôle à 0, au moins un contrôle croisé sur le référentiel produits, au moins un **test de réparation**, et une règle volontairement fausse documentée comme telle avec le nombre qu'elle produirait.

12. Correction détaillée

Exercice 4.1. Les 2 848 quantités négatives sont **licites** : la règle du socle veut qu'un retour ait quantités et montants négatifs — le document qui tranche est la notice. Les 800 quantités > 50 sont **licites-surprenantes** : seul un seuil d'achat dit si c'est anormal. Les 973 remises > 1 sont

impossibles, par définition de la variable. Les 7 674 poids $> 5\,000$ kg sont **invraisemblables** : le doute porte sur la nature de la colonne (poids total ou unitaire), pas sur la ligne. Les 51 dates après le 31/08/2026 sont impossibles pour la période annoncée et valides en elles-mêmes : la borne est un document, pas une donnée. Les 3 421 montants non numériques ne sont pas une aberration du tout : le défaut est dans l'écriture, pas dans la valeur — c'est le C05. Morale : quatre des six décisions ne se prennent pas dans le fichier mais à côté de lui.

Exercice 4.2. IQR : 7 400 d'écart inter-quartiles, bornes $[-7\,450 ; 22\,150]$, **24 180 lignes** hors bornes, **0 erreur**. Bande catalogue : **0 violation** pour un ratio de 0,958 à 1,306. On garde la bande et on écrit le retrait : « règle écartée le [date] : elle signale 9,9 % des lignes d'une colonne à dispersion structurelle et ne désigne aucune erreur ; le contrôle équivalent est la bande relative au catalogue, à 0 violation sur une bande de $\pm 3,5$ % autour de la moyenne observée ». Le piège de la borne basse mérite sa phrase à lui : $-7\,450$ est inatteignable, donc la moitié de la règle ne peut rien dire — et une règle à moitié inactive donne un faux sentiment de sévérité.

Exercice 4.3. Règle 1 · intégrité : $qte_physique < 0 \rightarrow 1\,800$ lignes sur 60 000, soit 3,0 %, niveau 2, responsable de dépôt, quotidienne. Règle 2 · risque : $qte_theorique - qte_physique > k \rightarrow 532$ comme maximum observé et 2,77 comme moyenne disent d'abord que la queue est courte ; avec $k = 100$, le taux tombe très en dessous des 3 % et il faudra le mesurer, pas le promettre. Ce qui change vraiment : les deux signalements ne se recouvrent plus, donc chaque alerte porte une action distincte. Et la question centrale n'est pas le taux mais l'action : une alerte à 0,1 % que personne ne traite vaut moins qu'une alerte à 3 % qui déclenche un comptage.

Exercice 4.4. $30 \div 100 = 0,30 \rightarrow$ **258 lignes** au-dessus du plafond de 0,25. L'hypothèse « pourcentage saisi en points » est donc insuffisante : elle n'explique pas le lot. Recalcul depuis les montants : étendue 0,000 à 0,110, les 973 dans le domaine. Et 630 de ces lignes ont un `montant_ht` égal au prix plein, ce qui serait impossible avec une remise de 12 à 30 % appliquée. Conclusion à écrire : la colonne a été **remplacée** par une valeur qui ne la décrit plus, elle n'a pas été échelonnée — d'où le choix de la reconstitution plutôt que de la conversion.

Exercice 4.5. Attendu en substance : douze lignes, dont (a) un contrôle à 0 daté — `id_produit orphelin = 0` sur 243 360, `tva_non_conforme = 0`, `dates_non_ISO = 0`, avec la mention « le zéro est un résultat » ; (b) le croisé poids (8 380, niveau 3, avec la colonne « table suspecte ») ; (c) un **test de réparation** écrit comme tel — le $\div 100$ des remises, 258 hors domaine, règle rejetée ; (d) la règle fausse nommée (IQR prix, 24 180 alertes, motif du retrait). Barème du rendu : exhaustivité et non-duplication (4) · taux et dénominateurs (2) · décisions et destinataires (2) · règle fausse documentée (2). Le dernier point est le plus rare en production, donc le plus noté.

13. Mini-projet M04.P4 — « Le registre de contrôles du fichier de ventes » (1 h 30)

Livrable : `controles_ventes.md`, précédé d'une page de garde (fichier, date, taille 28 516 920 octets, 243 360 lignes). Contenu imposé :

- douze règles maximum, dont au moins **trois contrôles croisés** avec un référentiel et au moins **deux** dont le résultat mesuré est 0 ;
- pour chaque règle : l'affirmation vérifiable, le niveau sur l'échelle, le destinataire nommé, la cadence, le taux attendu, la mesure du jour avec son dénominateur, et la date de calibration du seuil s'il y en a un ;
- une annexe « **réparations rejetées** » : pour chaque correction envisagée puis écartée (le ± 100 des remises est le cas fourni), le nombre qu'elle laisse subsister et la raison ;
- une annexe « **règles retirées** » avec le volume d'alertes de chacune ;
- une dernière section : ce que ce registre ne couvre pas — trois défauts du fichier que vos douze règles laissent passer, chiffrés (les 61 lignes à double défaut, par exemple).

Barème (10 points). Affirmations vérifiables et non dupliquées (2) · croisés et zéros explicites (2) · niveaux, destinataires, cadences, dates de calibration (2) · annexes des réparations et retraits (2) · section de non-couverture (2). Seuil de validation : 7. Un registre qui prétend tout couvrir sera cru, puis contredit : la dernière section vaut autant que les quatre premières.

14. Résumé du chapitre

NATURE DE L'ANOMALIE	OUTIL QUI LA VOIT	ICI	ACTION
impossible	règle de domaine	0 · 0 · 0 · 0	bloquer à l'entrée
contredit un doc	contrôle croisé	973 · 8 380	instruire, nommer la table
seuil de pratique	seuil métier documenté	800	surveiller au destinataire
dispersion réelle	IQR · z-score	24 180 · 3 894	RETIRER du registre
écriture abîmée	type de cellule	3 421	normaliser (C05)

Le chapitre tient dans une asymétrie : les contrôles **sans paramètre** (domaine, égalité arithmétique, référentiel joint) trouvent les vrais défauts ; les contrôles **à paramètre** (quartiles, sigmas, seuils d'alerte) trouvent le profil du fichier. Ces derniers restent utiles — dimensionner, calibrer, surveiller une dérive — à condition de ne jamais être la règle qui décide. Et gardez le cas des 973 remises en mémoire : un défaut qui ne change aucun total est le seul à pouvoir survivre à tous vos contrôles de totaux.

15. À retenir

À retenir. ★ Aberrant ≠ erroné. 24 180 lignes hors IQR, 3 894 au-delà de 3σ , 0 à 6σ — et pas une erreur : la dispersion est le métier (575 à 45 000 FCFA, du sac de ciment à la palette d'ossature). **★ La règle qui voit juste croise deux sources.** Bande relative au catalogue : 0 violation, ratio 0,958 à 1,306. Égalité au catalogue : 232 981 « anomalies » — c'est la règle qui est fausse. **★ Testez la réparation avant de l'appliquer.** ± 100 sur les remises laisse 258 lignes hors du domaine ; la reconstitution depuis les montants laisse 973 lignes dans le domaine, entre 0,000 et 0,110. **★ Un défaut qui ne change aucun total est le plus dangereux.** Les 973 lignes laissent le chiffre d'affaires intact (TVA conforme sur 243 360 lignes) et rendent fausse toute analyse de remise. **★ Une règle sans décision n'est pas un contrôle.** Bloquer · corriger · surveiller · ignorer tracé : quatre niveaux, un destinataire par niveau, une date de calibration par seuil, un taux attendu écrit à l'avance.

16. Évaluation formative (auto-correction, 8 min)

Q1. Une valeur à 6 écarts-types est : (a) une erreur ; (b) une valeur rare ; (c) une valeur à supprimer. → **(b)**, et ici (a) ne s'applique à aucune des 243 360 lignes de prix : la borne à 6σ ne mord sur rien, celle à $1,5 \times \text{IQR}$ en signale 24 180.

Q2. Pourquoi la règle de domaine sur la remise l'emporte-t-elle sur le z-score ? → elle n'a aucun paramètre à justifier, elle est reproductible d'un lot à l'autre, et elle ouvre une correction vérifiable. Le z-score trouve des lignes, pas des erreurs, et son résultat dépend du reste du fichier.

Q3. Le contrôle croisé poids signale 8 380 lignes. Que ne sait-il pas dire ? → quelle table est suspecte : `poids_kg` de la ligne, `poids_unite_kg` du catalogue, ou la quantité. C'est pourquoi il est de niveau 3, pas de niveau 2.

Q4. La règle `qte_physique < seuil_alerte` déclenche sur les mêmes 1 800 lignes que `qte_physique < 0`. Conclusion ? → le seuil ne discrimine pas, la condition réelle est le signe. Une règle qui reproduit le résultat d'une règle plus simple doit être scindée ou retirée.

Q5. On vous propose « diviser les remises par 100 ». Quelle est la première question à poser ? → qu'est-ce que ça donne au contrôle de domaine ? Ici : 258 lignes hors $[0 ; 0,25]$, donc la réponse est « insuffisant » — et la reconstitution depuis les montants est le seul remède qui passe le test.

Question ouverte (la plus importante). Reprenez une règle statistique du § 5.8 et rendez-la utile sans en faire un contrôle : dites ce qu'elle surveille, à quelle cadence, avec quel taux attendu, et à qui l'alerte s'adresse. Corrigé attendu : un **indicateur de dérive** — le nombre de lignes hors IQR, hebdomadaire, alerte si l'écart à la référence dépasse 20 % (24 180 cette semaine ; le double, et quelque chose a changé dans la nature du fichier, pas dans sa propreté). Une règle statistique bien employée ne dit pas quelles lignes sont fausses : elle dit quand le fichier a changé.

Module M04.C05 — Normaliser et rendre comparable : changer l'écriture, jamais la valeur

Outil de ce chapitre : le tableur (formules de texte et Power Query), sur `ventes_brutes.csv`, `clients.csv`, `produits.csv`, `objectifs_de_ca.csv`, `remises_manuelles.xlsx` et `tarif_fournisseur_peinture.csv`. **Durée indicative :** 5 h. **Niveau :** N2.

L'idée du chapitre. Nettoyer retire des lignes ; normaliser ne retire rien. Normaliser change l'écriture d'une valeur en gardant la **valeur**, pour que deux lignes qui parlent du même objet puissent se reconnaître, et pour qu'une colonne cesse d'être du texte quand elle devrait être un nombre. Le socle de données donne de cette opération quatre leçons mesurées, et chacune se vérifie en une ligne de contrôle. (1) Les 3 421 montants écrits en texte se reconvertissent par deux voies indépendantes — découper le texte, ou sommer `montant_ht + montant_tva` — et les deux rendent 15 639 751 286 FCFA, écart de 0 franc. (2) La mise en forme est une information : les 412 lignes que la référence retire du fichier clients portent **toutes** deux anomalies d'écriture, et **aucune** des 23 500 autres ne les porte. (3) Normaliser ne rend pas juste : 0 date de vente sur 243 360 n'est hors norme, et pourtant 51 lignes portent une date postérieure à la date du relevé. (4) L'écriture fabrique les comparaisons : la table d'objectifs attend 220 cases, en contient 218, et ce n'est qu'après avoir typé la clé que l'on voit lesquelles manquent.

Base de travail. `01_socle_donnees/data/brut/ventes_brutes.csv` (243 360 lignes × 20 colonnes), `01_socle_donnees/data/brut/clients.csv` (23 912), `01_socle_donnees/data/brut/produits.csv` (154), `01_socle_donnees/data/brut/objectifs_de_ca.csv` (218 lignes, séparateur ;), `01_socle_donnees/data/brut/remises_manuelles.xlsx` (18 200), `01_socle_donnees/data/brut/tarif_fournisseur_peinture.csv` (38 lignes utiles, encodage Windows-1252), `01_socle_donnees/data/reference/ventes_propres.csv`, `clients_propres.csv` et `dictionnaire_produits.csv`. Tous les nombres de ce chapitre viennent de ces fichiers, relevés par `01_socle_donnees/scripts/chiffres_manuel.py`.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **distinguer** trois gestes qu'on confond : normaliser (changer l'écriture), nettoyer (retirer des lignes), changer de convention (redéfinir ce que la colonne veut dire) — et nommer lequel vous êtes en train de faire ;
- **convertir** une colonne monétaire stockée en texte par un découpage dont on prouve qu'il ne perd rien, et écrire le contrôle de convergence qui ferme le dossier à 0 franc près ;
- **mesurer le gain** d'une normalisation avant de l'écrire (12 640 noms distincts deviennent 12 342, la ville ne gagne rien : 6 modalités avant, 6 après) et supprimer la formule qui n'apporte rien ;
- **importer sans perte** un fichier à séparateur ;, à décimales par virgule, en Windows-1252 et à en-tête décalé, en chiffrant ce que le mauvais réglage coûte de caractères ;
- **ranger les opérations** dans l'ordre qui fait que chaque étape est vérifiable, et refuser les trois normales qui abîment : l'arrondi des montants, la casse sur un code, la réécriture sur place.

2. Pourquoi cette notion est importante

Un tableau croisé qui affiche douze lignes « par mois » au lieu de dix, une jointure qui rend 0 correspondance alors

que les deux fichiers parlent des mêmes clients, un total de chiffre d'affaires en texte que la fonction SOMME ignore

tous ces accidents viennent du même endroit, et aucun n'est un problème de saisie. Voici ce que le socle montre quand on pose la question « la valeur est-elle juste ? » séparément de « son écriture est-elle comparable ? ».

Ce que vous croyez avoir	Ce que le fichier contient	Ce que ça coûte
Une colonne de montants numériques	239 939 nombres et 3 421 textes dans la même colonne N	le SOMME est faux de 3 421 lignes, le filtre « plus grand que » les ignore
Un fichier clients sans doublon	412 lignes au nom encadré d'espaces et au téléphone sans espaces	412 clients comptés deux fois, 11 158 fusionnés à tort si l'on corrige à l'envers
Une table d'objectifs complète	218 lignes là où la grille en prévoit 220	un magasin sans objectif pendant deux mois, classé « sous performance » par un tool qui lit 0
Une jointure entre deux fichiers	id_client nombre dans le classeur, texte dans le CSV	0 ligne jointe, sans message d'erreur, dans Power Query

Le point commun : **aucun de ces défauts ne produit d'erreur à l'écran**. Le tableur accepte le mélange de types, la jointure ratée rend un tableau vide proprement, le texte non converti se somme à zéro silencieusement. C'est pourquoi la normalisation est un métier et non un réglage : elle consiste à rendre visible, avant de rendre propre.

Définition. On appelle **normalisation** toute transformation bijective à l'identifiant près appliquée à une valeur, qui change sa représentation et non sa signification : +226 77 76 25 98 96 et +2267776259896 désignent le même abonné ; 16 638 FCFA et 16638 désignent la même somme. Le test de la normalisation est donc double : deux écritures qui désignent le même objet doivent devenir identiques (**rapprocher**), et deux écritures qui désignent des objets différents doivent rester distinctes (**ne pas confondre**). Une transformation qui ne fait que le premier est une perte d'information ; une transformation qui rate le second est une faute.

3. Explication simple

Regardez deux lignes du fichier clients, telles qu'elles sont écrites dans `/home/user/formation-data-bi/01_socle_donnees/data/brut/clients.csv` (ligne 2 et ligne 23 502, à vérifier avec `grep -n "SARL K" clients.csv`) :

```
2,      SARL k, Entreprise, Koudougou, ..., +226 78 62 21 81 98, , 2022-03-15, ...
23501,  SARL K , Entreprise, Koudougou, ..., +2267862218198, , 2022-03-15, ...
```

Même ville, même date de création, mêmes conditions de paiement, même plafond, même courriel absent. Le nom change de casse et gagne deux espaces ; le téléphone perd ses espaces et ne change pas de numéro. Un être humain lit un seul client. Le tableur lit deux chaînes différentes, le tableau croisé affiche deux lignes, la jointure compte deux clients. Ce n'est pas que le fichier soit sale : c'est que **deux écritures n'ont rien de comparable tant qu'elles n'ont pas été ramenées à une norme d'écriture commune**.

Maintenant l'opération inverse, celle qui fait le plus de dégâts. On décide d'« harmoniser » : `SUPPRESPEACE()` pour rogner les espaces, `MINUSCULE()` pour unifier la casse. Regardons ce que ça donne sur la colonne qui compte vraiment, le montant :

```

16 638 FCFA → SUPPRESpace → 16 638 FCFA (rien ne change : les espaces sont à l'intérieur, pas
autour)
16 638 FCFA → CNUM(SUPPRESpace(...)) → #VALUE! (le texte n'est pas un nombre)
16 638 FCFA → CNUM(SUBSTITUE(...;" FCFA";"")) → #VALUE! (l'espace de milliers reste collé)
16 638 FCFA → CNUM(SUBSTITUE(SUBSTITUE(...;" FCFA";"");" ";"")) → 16638 (enfin)

```

Quatre formules, une seule marche. Cette progression est tout le chapitre : la normalisation n'est pas un bouton, c'est une suite de décisions sur **ce que signifie chaque caractère** de la cellule — ici, trois espaces dont deux sont des séparateurs de milliers et un est un séparateur de devise.

4. Vocabulaire essentiel

Terme	Ce que ça désigne, dans ce chapitre	
Écriture (représentation)	La suite de caractères de la cellule : 16 638 FCFA	
Valeur	Ce que l'écriture décrit : seize mille six cent trente-huit francs	
Typage	Attribuer à une colonne le bon type : nombre, date, texte, booléen	
Élagage	Retirer ce qui entoure la valeur : SUPPRESpace() pour les espaces, CLEAN() / Text.Clean pour les caractères de contrôle	
Harmonisation casse	de	Ramener majuscules et minuscules à une forme unique — utile sur un nom, interdite sur un code
Séparateur de milliers	L'espace (ou la virgule, ou le point) qui découpe les groupes de trois chiffres dans un nombre écrit en texte	
Encodage	La table qui traduit des octets en caractères : UTF-8, Windows-1252, ISO-8859-1	
BOM	Bytes-mark : trois octets EF BB BF en tête d'un fichier UTF-8, qui deviennent <code>ï»¿</code> si on se trompe de table	
Conversion d'unité	Multiplier par un facteur défini par la physique : $1 \text{ m}^3 = 1\,000 \text{ L}$ — ce n'est pas une normalisation de texte	
Table correspondance	de	Un fichier qui traduit une écriture en une autre, avec une décision documentée derrière chaque ligne
Clé de jointure	La ou les colonnes sur lesquelles deux tables se reconnaissent ; elle doit être normalisée avant la jointure	
Protocole normalisation	de	La liste ordonnée des transformations appliquées, avec pour chacune l'effet mesuré sur le nombre de lignes et de valeurs distinctes

5. Cours approfondi

5.1 Trois objets, trois traitements — et ne pas les mélanger

Une colonne peut être fautive à trois niveaux, qui ne se réparent pas du tout de la même façon.

Niveau	Question à poser	Opération licite	Dans le socle	Ce qui serait une faute
Le type	La cellule est-elle du nombre ou du texte ?	Convertir, en gardant la valeur	3 421 montants en texte	Imputer, arrondir, recalculer le montant
L'écriture	Deux formes décrivent-elles le même objet ?	Élaguer, unifier la casse, retirer les séparateurs	412 surchargés d'espaces	Retirer une ligne, fusionner deux homonymes
Le sens	La valeur dit-elle ce qu'on croit ?	Reconstituer documenter, « corriger »	ou 973 taux de remise à 12, 18, 25, 30	Diviser par 100 sans test de réparation

Le troisième niveau est celui du chapitre précédent (C04) et il mérite d'être rappelé ici, parce que la tentation est grande de le traiter comme un problème de format. Diviser `taux_remise` par 100, c'est **changer la convention** de la colonne : on décide que le nombre 30 signifiait 30 %. Si c'est vrai, ça se prouve (ça ne se vérifie pas : le test de réparation du C04 montre que 258 lignes sortent alors du domaine contractuel). Une normalisation, elle, ne prétend rien sur le sens. Elle ne fait que rendre lisible une valeur dont tout le monde est d'accord.

À retenir. La question qui précède toute formule : est-ce que je change l'écriture, la ligne, ou le sens ? Écrire la réponse en tête de chaque colonne ajoutée (`_écriture`, `_ligne`, `_sens`). Vous verrez alors immédiatement si une même colonne cumule deux natures d'opération — et il faudra la scinder.

5.2 Les montants texte : découper, puis prouver par une seconde voie

Colonne `montant_ttc` (colonne N de `ventes_brutes.csv`) : 239 939 cellules numériques, 3 421 non vides et non numériques, 0 vide. Anatomie de ces 3 421, mesurée caractère par caractère :

Ce qu'on cherche	Combien	Ce que ça impose
Suffixe « FCFA » détaché par une espace	3 421 sur 3 421	Un SUBSTITUE suffit, aucun STXT
Au moins deux espaces (séparateur de milliers)	3 410 sur 3 421	Les 11 autres sont des montants à trois chiffres : 680 FCFA
Virgule décimale	0	Pas de problème de locale à régler ici
Espace insécable (U+00A0)	0	Le SUBSTITUE(...;CAR(160);" ") de réflexe ne sert à rien — et le vérifier évite un après-midi de recherche
Formes distinctes après avoir masqué les chiffres	8	Le suffixe est toujours détaché par une espace, le signe moins précède le nombre, et le seul écart notable est le nombre de groupes de trois chiffres
Montants négatifs	47	Ce sont les 47 retours sans montant, pas des erreurs
Réparables par le découpage FCFA + espaces	3 421 sur 3 421	La formule tient, sans exception à traiter à la main

La ligne à 3 782 047 FCFA (le plus gros texte) et celle à -193 284 FCFA (le plus petit) passent toutes les deux. Et surtout : **les 47 textes négatifs sont exactement les 47 lignes de retour dont le montant manquait**. Une fois converties, le contrôle « toute ligne de retour porte un montant négatif » passe sur 2 848 lignes sur 2 848, alors qu'il échouait sur 47 avant. La normalisation ne s'est pas contentée de changer un type : elle a fait passer un deuxième contrôle de 47 lignes ratées à zéro.

Reste à prouver que le découpage n'a rien inventé. Deux voies indépendantes :

Voie A (découpage du texte) :	somme des 3 421 textes convertis + somme des 239 939 nombres =	15 639 751 286
Voie B (reconstitution) :	somme de (montant_ht + montant_tva) sur les mêmes lignes =	15 639 751 286
Contrôle :	écart =	0

Zéro franc d'écart, à l'unité près. C'est ce qui autorise à écrire dans le registre « colonne reconverte, aucune valeur perdue » — et non « colonne reconverte, semble correcte ».

Définition. Un **contrôle de convergence** fait arriver le même résultat par deux chemins qui n'ont aucune raison de se ressembler : l'un lit ce que l'opérateur a tapé, l'autre recalcule depuis d'autres colonnes. Convergence à 0 $\Rightarrow$ la transformation est une écriture différente de la même information. Écart non nul $\Rightarrow$ au moins un des deux chemins est faux, et il faut chercher lequel **avant** de décider lequel est juste.

Attention. La convergence à 0 n'a été possible que parce qu'aucune autre colonne n'a été « réparée » au passage. Si vous aviez divisé `taux_remise` par 100 (voir C04 §5.3) avant de faire ce calcul, `montant_ht` ne serait plus indépendant du texte : les deux voies se ressembleraient par construction et le contrôle ne prouverait plus rien. Un contrôle croisé exige que les deux voies restent **indépendantes** — donc il se fait avant les corrections de sens, pas après.

5.3 Mesurer le gain avant d'écrire la formule

Le réflexe « appliquer `SUPPSPACE` et `MINUSCULE` à toutes les colonnes de texte » coûte trois colonnes, ralentit le fichier et ne répare souvent rien. Le gain se mesure, en une ligne, colonne par colonne : nombre de valeurs distinctes avant, après ; nombre de lignes modifiées.

Colonne	Modalités avant	Modalités après normalisation	Verdict
<code>clients.nom</code> (brut → normalisé)	12 640	12 342	298 écritures différentes d'un même nom : à normaliser, et c'est ce qui rend la clé du C03 possible
<code>clients.telephone</code>	23 499 textes distincts	23 499 numéros après retrait des non-chiffres	un numéro porté par deux écritures : à normaliser pour la clé, pas pour l'affichage
<code>clients.ville</code>	6	6	aucun gain : <code>MAJUSCULE()</code> serait du décor
<code>produits.unite</code>	4	4	aucun gain textuel — mais 4 unités physiques , qui est un autre problème (voir 5.5)
<code>clients.email</code>	7 398 remplis	7 398	rien à reformater ; les 16 514 cas sans @ sont des champs vides, pas des adresses cassées

Deux enseignements. D'abord, une anomalie d'écriture peut être un **indice** plus précieux qu'une saleté : les 412 lignes retirées de `clients.csv` portent toutes un nom encadré d'espaces (412 sur 412) et un téléphone sans espaces (412 sur 412), alors que **0** des 23 500 lignes conservées porte la première anomalie. Autrement dit, la mise en forme signale ici exactement le bloc à retirer. C'est le genre de coïncidence qu'on ne découvre qu'en comptant, et qui change la nature d'un contrôle : on ne supprime plus « les doublons trouvés par une règle de similarité », on supprime « les lignes marquées par une anomalie de format », ce qui est déterministe et rejouable.

Dans les faits. Le lot des 412 doublons clients a été repéré à l'aveugle par une règle de similarité, puis reproduit par une règle de format (412 sur 412, et 0 faux positif sur les 23 500 lignes conservées). Les deux méthodes tombent d'accord une fois ; elles ne le seront plus la suivante, parce que la règle de format décrit un import passé et non une propriété durable du client. Un protocole de normalisation doit donc distinguer explicitement la **règle de décision** (le contenu de la clé) de l'**indice de rejeu** (la mise en forme du lot), et écrire à côté de l'indice qu'il ne vaut plus rien dès que la source change de poste de saisie.

Ensuite, un champ **vide** n'est pas un champ mal formaté. `m04_clients_email_sans_a_robase` vaut 16 514 : ce ne sont pas 16 514 adresses illisibles, ce sont 16 514 absences (le C02 a nommé le mécanisme). Une normalisation appliquée à une absence produit un objet faux : `NORMALISE("")` peut rendre @, 0, une ligne blanche selon la formule — un texte non vide là où il y avait une ab-

sence, donc une valeur comptable là où il n'y en avait pas. Le contrôle d'après-normalisation est simple : le nombre de cellules vides ne doit pas bouger.

Conseil professionnel. Écrivez la mesure de gain **avant** la formule, dans le même onglet : trois cellules, `=NBCAR.SI(...)` ou un compteur de modalités, avant, après, écart. Quand on vous demandera dans six mois pourquoi la colonne `nom_normalise` existe, vous répondrez avec un chiffre, pas avec une intention.

5.4 Les dates : la norme est respectée, la réalité non

Sur `date_vente` des 243 360 lignes : 0 valeur qui ne soit pas exactement `AAAA-MM-JJ`. Sur `date_creation` des 23 912 lignes de `clients.csv` : 0 hors norme. Il n'y a donc **aucune conversion de date à écrire** dans ce socle — et c'est une information, pas un hasard : le format ISO est précisément conçu pour que l'ordre alphabétique soit l'ordre chronologique, et pour que la locale du poste n'ait aucune prise sur lui.

Ce qui ne va pas, ce sont les valeurs : 51 lignes portent une date postérieure au 31 août 2026, la plus tardive étant le 2026-12-06, alors que le relevé s'arrête fin août. Ces 51 dates sont **parfaitement normales** et **physiquement impossibles** : elles passent tous les contrôles de format et aucun contrôle de domaine.

La conséquence pratique est la règle suivante : jamais de contrôle de validité sur une date réduit à « est-ce que ça se lit comme une date ». Celui-là ne dit que ce que le tableur sait déjà. Trois contrôles, dans l'ordre, et le premier n'est pas celui qu'on croit :

Contrôle	Forme à écrire	Résultat sur le socle
Homogénéité de la norme	toutes les valeurs <code>^[0-9]{4}-[0-9]{2}-[0-9]{2}\$</code>	matchent 0 exception
Domaine	date entre la première date d'exploitation et la date du relevé	51 lignes hors domaine
Cohérence colonnes	entre <code>date_vente</code> de la ligne = date du ticket (toutes lignes d'un même <code>id_ticket</code>)	contrôle à écrire, c'est celui qui attrape les dates recopiées de travers

À retenir. Normaliser une date, c'est aligner son écriture. Valider une date, c'est la confronter au monde. Les 51 lignes futures ne se réparent pas par une formule : elles se décident (les exclure du périmètre d'analyse et le dire, ou les faire corriger par la source). Écrire « dates normalisées » pour ce lot serait un mensonge technique — l'écriture n'était pas le problème.

5.5 Unités et correspondances : ce qui demande un facteur, pas une formule

Définition. Une **conversion d'unité** multiplie une valeur par un facteur fixé par une définition extérieure au fichier ($1 \text{ m}^3 = 1\,000 \text{ L}$), et change le nom de la colonne en conséquence : `quantite` devient `quantite_litres`. Une **harmonisation d'écriture** ne change ni le facteur ni le nom : `LITRE`, `litre` et `Litre` deviennent `litre`. Confondre les deux produit un nombre juste dans une colonne mal nommée, ce qui est le genre d'erreur qui se propage à tous les tableaux suivants sans jamais plus se signaler.

`produits.csv` porte une colonne `unite` avec 4 modalités : 122 produits en `unite`, 14 en `litre`, 12 en `sac`, 6 en `m3`. Aucune variante de casse, aucun espace parasite : en tant que texte, cette colonne est propre. En tant que physique, elle est inerte — et c'est là que le piège se referme. Une somme de « volumes » sur ce champ additionne des mètres cubes et des litres comme s'ils

étaient la même chose. Le correctif n'est pas une fonction texte : c'est un **facteur de conversion** porté par une table, $1 \text{ m}^3 = 1\,000 \text{ L}$, et une colonne `quantite_en_litres` qui n'existe que pour les lignes où la question a un sens.

Le fichier `tarif_fournisseur_peinture.csv` montre la même collision, aggravée. Ses 38 lignes utiles ont toutes `conditionnement = 10 L` (1 modalité seulement) et `unite_mesure = litre`, alors que les libellés décrivent des tubes en millimètres vendus au mètre linéaire (Tube PVC évacuation 100 mm (mL)). Trois colonnes, trois vocabulaires d'unité, et aucune ne permet de raisonner sur le stock. Ce n'est pas une saleté à élaguer : c'est un **dictionnaire à écrire avec le magasin**, et il se solde par une colonne de plus, pas par une formule de moins.

Côté catégories, la leçon est inverse : `produits.csv` en compte 15 modalités, le dictionnaire de référence 7, et 44 lignes portent un intitulé hétéroclite. Ici, la normalisation **par table de correspondance** est la bonne opération, et le C03 a montré pourquoi : une correspondance est une décision documentable, une formule ne l'est pas. La règle de tri est donc :

- si la transformation s'énonce sans dictionnaire (`SUPPRESPEACE`, minuscule, retrait d'un suffixe), c'est de la normalisation : elle s'écrit en formule et elle est gratuite à rejouer ;
- si elle exige de dire « cette écriture-là veut dire cet objet-ci » (`enduits + peinture` → un seul poste, `m3` → des litres), c'est une correspondance ou une conversion : elle s'écrit dans une **table**, avec une source et une date, sinon vous n'aurez pas à montrer votre travail.

5.6 Importer sans rien casser : séparateur, décimales, encodage, en-tête

Un fichier est quatre décisions de lecture, et le socle en contient un exemplaire de chaque mauvaise :

1. **Le séparateur.** `ventes_brutes.csv` est à virgule ; `objectifs_de_ca.csv` et `tarif_fournisseur_peinture.csv` sont à point-virgule. Ouvrir le second avec le réglage du premier produit une seule colonne de texte, et le tableur ne dit rien : il affiche une très longue colonne A.
2. **La décimale.** `tarif_fournisseur_peinture.csv` écrit ses prix 2900,57 et sa TVA 0,18 ; `objectifs_de_ca.csv` écrit sa marge 0.195. En français, l'un se lit correctement et l'autre devient du texte (ou l'inverse selon la version d'Excel). Le contrôle est immédiat : la colonne importée doit être **de type nombre** et son total doit évaluer `SOMME` calculé sur le texte découpé.
3. **L'encodage.** `tarif_fournisseur_peinture.csv` pèse 2 482 octets dont 95 ne sont pas ASCII, et ne se décode pas en UTF-8 du tout : l'octet 73 est illégal. C'est du Windows-1252. La nuance qui fait mal : Windows-1252 et ISO-8859-1 (le « Latin 1 » du sélecteur Excel) sont identiques sur 223 des 256 octets, et **différents sur les 39 octets** que ce fichier contient justement entre `0x80` et `0x9F` — les tirets demi-cadratin — qui séparent le libellé de la référence fournisseur. Choisir « ISO Latin 1 » n'affiche pas de caractère bizarre : ça insère des caractères de contrôle invisibles, que la recherche ne retrouve plus et que l'impression avale.
4. **L'en-tête décalé.** Les trois premières lignes du fichier de tarif sont du bruit d'export (export genere depuis ELODIE v3, édition le 17/09/2026, le nom du fournisseur). L'en-tête réelle est ligne 4. Un import aveugle nomme les colonnes d'après la ligne 1 et perd le vrai nommage.

Quant aux CSV proprement dits : 5 des 10 fichiers du dossier `data/brut/` portent un BOM (trois octets `EF BB BF` avant le premier caractère). Lus en UTF-8 sans retirer le BOM, rien d'anormal ne se produit ; forcés en Windows-1252 — le réglage par défaut de l'assistant d'importation legacy — l'en-tête de `ventes_brutes.csv` ne s'appelle plus `id_vente` (8 caractères mesurés dans le fichier) mais `ï»¿id_vente` (11 caractères mesurés après décodage). Onzième caractère invisible pour l'œil, premier caractère de tous vos noms de champs. Le contrôle coûte deux secondes : regarder la longueur du premier en-tête importé.

Définition. Un **encodage** est une table de correspondance octet → caractère. UTF-8 en est une où les caractères non ASCII occupent plusieurs octets ; Windows-1252 et ISO-8859-1 en sont où chaque octet vaut un caractère, avec 39 cas de divergence. Une erreur d'encodage est **réversible** tant que vous n'avez pas sauvegardé, irréversible après : les octets d'origine ont été remplacés. D'où la règle de non-réécriture : on ne sauvegarde jamais un import dans le fichier source, on écrit un fichier dérivé.

5.7 Les clés : normaliser pour que les jointures disent quelque chose

Trois accidents de ce socle, tous mesurés, viennent de ce qu'une clé n'a pas été typée avant usage.

- **Nombre contre texte.** `remises_manuelles.xlsx` porte `id_client` en entier ; `clients.csv` le porte en texte. Le contenu, lui, est bon : 1 400 clients distincts dans les remises, **0** absent de la fiche clients. Dans Power Query, une fusion sur ces deux colonnes ne lève aucune erreur et rend un tableau vide. La première étape de toute requête est donc un `Table.TransformColumnTypes` sur les colonnes de clé — pas une élégance, une nécessité.
- **La grille attendue n'est pas la grille reçue.** `objectifs_de_ca.csv` couvre 5 magasins et 44 mois : la grille complète aurait 220 lignes, elle en a 218. Les 2 manquantes sont consécutives (février et mars 2024) et n'affectent qu'un magasin, le n° 4 — signature d'un export interrompu ou d'un collé mal fini, pas d'un oubli de saisie. Et le magasin 6 n'apparaît nulle part dans ce fichier : 1 magasin sans objectif, trouvé en comparant deux jeux de clés et non en lisant le fichier.
- **L'absence, une fois encore, se déplace.** Tant que la clé n'est pas normalisée, un trou ne se voit pas : la jointure externe rend une ligne avec `0` ou `null` que le tableau croisé affiche comme une valeur. Après normalisation et jointure explicite, le trou devient une ligne nommable : « magasin 4, 2024-02, aucun objectif ».

C'est le moment de nommer l'ordre des opérations, qui est la colonne vertébrale du chapitre :

Étape	Opération	Contrôle qui la ferme	Effet attendu
1	Typage des colonnes de clé et des montants	type de colonne + total de contrôle	0 changement de valeur
2	Élagage et harmonisation des colonnes-clés et étiquettes	nb de lignes inchangé, nb de modalités réduit	ici : 298 noms fusionnés
3	Traduction par tables de correspondance	0 clé inconnue après traduction	catégories 15 → 7
4	Conversion d'unités (facteur)	ordre de grandeur du total	1 000 × vérifié sur 6 lignes m3
5	Puis seulement : dédoublonnage (C03), règles de contrôle (C04), imputation (C02)	écart à la cible	ici : écart 0

5.8 Ce qu'une normalisation ne doit jamais faire

Quatre interdictions, chacune motivée par un chiffre de ce module.

- **Ne pas changer le nombre de lignes.** Le contrôle est brutal : 243 360 lignes avant, 243 360 après. Si le compte bouge, vous avez fait du nettoyage sous un nom poli — et vous ne l'avez pas dit.
- **Ne pas arrondir un montant.** Les 3 421 textes se convertissent à l'unité près ; l'écart de convergence est de 0 parce que rien n'a été arrondi. Un `ARRONDI(...;-2)` sur une colonne monétaire de 243 360 lignes produit une erreur de plusieurs dizaines de millions de francs, et surtout la rend indétectable par un contrôle de total.

- **Ne pas toucher à la casse d'un code.** `id_ticket` a une structure porteuse (T01-230101-000000, où T01 est le magasin) : `MAJUSCULE()` ne change rien ici, mais sur un code sensible à la casse, l'opération fusionne deux objets distincts. La casse s'harmonise sur les libellés destinés à l'humain, jamais sur les identifiants.
- **Ne pas écrire par-dessus.** La réversibilité est un livrable : on garde `montant_ttc_brut` et l'on ajoute `montant_ttc_num`. Un protocole qui détruit la source ne peut plus être audité, et ne peut plus être rejoué quand on découvre, deux semaines plus tard, que le suffixe était parfois `F CFA` avec une espace de trop.

Attention. Le piège le plus coûteux de ce chapitre est la normalisation **par la jointure** : on rapproche deux colonnes de clés en faisant confiance à la table de correspondance, sans compter les clés inconnues. Ici, ce réflexe aurait fusionné 11 158 clients homonymes en plus des 412 doublons réels, parce que la clé « nom seul » est valide comme écriture et fausse comme identité. Toute jointure se ferme par deux compteurs : lignes non jointes à gauche, lignes non jointes à droite — et les deux ne doivent pas être zéro par omission, mais par mesure.

Boîte à outils du chapitre. Cinq fonctions et deux réglages, pas davantage : `SUPPRESSESPACE` (élagage), `SUBSTITUE` (retrait d'un suffixe ou d'un séparateur), `CNUM` et son frère explicite `CNUMPARAGRAPHE` (conversion avec point ou virgule choisis, la seule qui ne dépende pas de votre clavier), `TEXTE(v; "00")` (remise à la même largeur d'une clé courte), `DATE` ou `DATEVAL` (reconstruction d'une date depuis trois colonnes). Côté Power Query, deux réglages : le type de colonne appliqué **en première étape**, et l'option `Encoding / Delimiter` écrite dans la requête plutôt que laissée à l'assistant.

Normaliser n'est pas nettoyer — le gain de chaque colonne, mesuré

Gain = valeurs distinctes avant, moins valeurs distinctes après. Une formule à gain nul est un décor.

Colonne	Opération envisagée	Avant	Après	Verdict
<code>montant_ttc (N)</code>	retirer le suffixe puis les espaces	3 421 textes	3 421 nombres	type · à convertir
<code>clients.nom</code>	élaguer, minuscule, ponctuation	12 640	12 342 · gain 298	écriture · c'est la clé du C03
<code>clients.telephone</code>	garder les chiffres seuls	23 499	23 499 · gain 0	écriture · pour la clé, pas l'affichage
<code>clients.ville</code>	élaguer et harmoniser la casse	6	6 · gain 0	ne pas toucher : formule à retirer
<code>produits.unite</code>	élaguer et harmoniser la casse	4	4 · gain 0	propre en texte, muette en physique
<code>date_vente</code>	aucune conversion à écrire	0 hors norme	51 dates impossibles	le format n'est pas le monde

La preuve qui ferme le dossier : deux voies indépendantes, un écart à l'unité

Voie A → somme des 3 421 textes découpés et des 239 939 nombres : 15 639 751 286 FCFA

Voie B → somme de (`montant_ht` + `montant_tva`) sur les mêmes lignes : 15 639 751 286 FCFA · écart 0

Contrôle de rattrapage : les retours à montant positif passent de 47 à 0 sur 2 848 lignes.

6. Exemple concret : la cellule 16 638 FCFA

Quatre formules sur la même cellule N, avec ce que chacune rend et ce qu'on en conclut.

Formule écrite	Résultat	Diagnostic
<code>=CNUM(SUPPRESSESPACE(N2))</code>	erreur	l'espace est à l'intérieur, pas autour : l'élagage ne peut rien

Formule écrite	Résultat	Diagnostic
<code>=CNUM(SUBSTITUE(N2;" FCFA";""))</code>	erreur	reste 16 638, l'espace de milliers bloque la conversion
<code>=CNUM(SUBSTITUE(SUBSTITUE(N2;" FCFA";"")," ",""))</code>	16638	correct ; 3 421 lignes sur 3 421 passent
<code>=CNUMPARAGRAPHE(SUBSTITUE(SUBSTITUE(N2;" FCFA";"")," ","");". ";",")</code>	16638	correct et insensible à la locale du poste — à préférer dans un fichier qui circule

Trois précisions qui évitent des heures de recherche :

- les 11 lignes sans séparateur de milliers (680 FCFA, 977 FCFA) ne sont pas un cas à part pour la formule — le double `SUBSTITUE` les traite — mais elles **le deviennent** pour une formule qui découperait « les trois derniers chiffres » à la main ;
- `CAR(160)` (espace insécable) ne sert pas ici : mesuré 0 occurrence. Cette mesure est un gain de temps, pas une perte de temps ;
- le total de contrôle doit être écrit **avant** de recopier la formule vers le bas sur 243 360 lignes, sinon vous ne saurez plus contre quoi le comparer.

7. Démonstration pas à pas : le protocole de normalisation en dix étapes

L'exercice se fait sur une copie de `ventes_brutes.csv` importée par Power Query, et sur `clients.csv`. Notez que chaque étape se termine par un chiffre écrit dans un onglet `CONTROLES`.

```

0. Import. Données > À partir d'un fichier texte/CSV. ventes_brutes.csv : délimiteur "virgule",
   encodage "UTF-8 (avec BOM)", en-tête présent. objectifs_de_ca.csv : délimiteur "point-
   virgule".
   tarif_fournisseur_peinture.csv : encodage "1252 : Europe occidentale (Windows)", 3
   lignes à sauter.
1. Compter avant. Lignes = 243 360 ; clients = 23 912 ; objectifs = 218. Écrire ces trois
   nombres.
2. Typier les clés. Table.TransformColumnTypes sur id_vente, id_ticket, id_client, id_produit,
   date_vente,
   montant_ht, montant_tva, montant_ttc : la conversion échoue sur 3 421 lignes
-> elles
   deviennent null, et c'est exactement ce qu'on cherchait : le compteur de null
   signale le lot.
3. Réparer les null de montant_ttc par découpage. En requête :
   Text.Select([montant_ttc]; {"0".."9"; "-"}) puis Number.FromText
   (retient les chiffres et le signe, laisse "FCFA" derrière ; 3 421 valeurs, 0
   échec).
4. Contrôler la convergence. Somme colonne N après réparation = 15 639 751 286 ; somme de
   (montant_ht + montant_tva)
   sur les mêmes lignes = 15 639 751 286 ; écart = 0. Écrire l'écart, pas
   seulement les deux totaux.
5. Normaliser les colonnes d'étiquette, et mesurer le gain. clients.nom : SUPPRESSESPACE + MINUSCULE
   + ponctuation
   retirée -> 12 640 modalités deviennent 12 342. clients.ville : 6 -> 6.
Conclusion : on garde la
   colonne nom_normalise, on ne crée pas ville_normalise.
6. Écrire la clé, pas la valeur, quand la jointure l'exige. cle_client = nom_normalise "|" +
   Text.Select(telephone;
   {"0".."9"}) -> 412 groupes de deux lignes, 0 groupe de trois (le C03 en a
   besoin, pas la ville).
7. Remettre à la même largeur les clés courtes. ici, rien à faire : le test (12 modalités de mois
   avant, 12 après)
   montre que la padding n'est pas le problème de ce fichier. Écrire "0
   transformations nécessaires".
8. Traduire par table. catégories : jointure à dictionnaire_produits.csv (15 -> 7) ; 0 code
   produit inconnu après
   traduction, sinon on s'arrête et on complète la table.
9. Convertir les unités avec facteur, pas avec formule. produits.unite : table de conversion (m3
   -> 1 000 L) ;
   ne pas additionner `m3` et `litre` dans un même total de volume.
10. Compter après. Lignes = 243 360 (inchangé), clients = 23 912 (inchangé). Écarts = 0. Le
   protocole est clos,
   et il ne contient aucune suppression : le dédoublonnage viendra après, au C03,
   avec son propre compteur.

```

8. Erreurs fréquentes

- **Confondre absence et format.** Appliquer une normalisation aux 16 514 courriels vides crée des chaînes non vides là où il n'y avait rien ; le compte de valeurs « renseignées » augmente, ce qui a l'air d'une amélioration.
- **N'utiliser que SUPPRESSESPACE** sur 16 638 FCFA : les espaces intérieures survivent, la conversion échoue, et l'opérateur conclut que le fichier est irrécupérable.
- **Supposer l'espace insécable** sans le compter, puis chercher trois heures un caractère qui n'existe pas.
- **Normaliser en écrasant la colonne source** : après sauvegarde, plus aucun contrôle croisé n'est possible, et le BOM mal décodé est devenu le nom définitif de vos champs.
- **Choisir ISO-8859-1 plutôt que Windows-1252** sur le fichier de tarif : 39 caractères deviennent invisibles, et la recherche sur un libellé ne retrouve plus la ligne.
- **Faire la jointure avant le typage** : 0 ligne rapprochée entre le classeur de remises (clés numériques) et le CSV clients (clés texte), alors que 0 client n'existe pas dans la fiche.
- **Arrondir « pour que ça tienne »** : 3 421 conversions à l'unité près valent un écart de 0 ; un arrondi à la centaine fait disparaître le seul contrôle qui prouvait la justesse du découpage.

9. Bonnes pratiques professionnelles

Pratique	Mise en œuvre au tableur	Ce qu'elle garantit
Types fixés en première étape	<code>Table.TransformColumnTypes</code> sur toutes les colonnes de clé et monétaires	aucune jointure silencieusement vide
Une colonne ajoutée, jamais écrasée	<code>x_brut</code> reste, <code>x_normalise</code> s'ajoute	réversibilité et audit
Gain mesuré avant écriture	nb de modalités avant / après, nb de lignes modifiées	pas de formule décorative dans le fichier
Contrôle de convergence à l'unité	deux chemins, un écart, une case	la réparation ne invente rien
Compteurs de non-jointure aux deux bouts	lignes sans correspondance gauche, puis droite	les trous deviennent nommables
Encodage et séparateur écrits dans la requête	options explicites, jamais l'assistant par défaut	le rejou est identique sur un autre poste
Protocole daté dans le classeur	onglet <code>PROTOCOLE</code> : ordre, auteur, date, effet mesuré	on répond à une question dans six mois sans rouvrir le code

10. Exercice guidé — reconvertir une colonne monétaire et le prouver (35 min, /10)

Contexte : on vous demande de livrer un chiffre d'affaires mensuel. La colonne `montant_ttc` refuse de se sommer entièrement. Vous devez produire une colonne exploitable et la preuve qu'elle ne déforme rien.

- 1 pt** — Importer `ventes_brutes.csv` en gardant toutes les colonnes en texte (réglage explicite du séparateur et de l'encodage). Combien de lignes ?
- 1 pt** — Taper `montant_ttc` en nombre. Combien de `null` apparaît ? Que faut-il en conclure sur la nature du défaut : absence, ou écriture ?
- 1 pt** — Compter, sur ces lignes seulement, les trois traits suivants : présence de « FCFA », au moins deux espaces, présence d'une virgule. Quel réglage de conversion cela impose-t-il ?
- 1 pt** — Écrire la formule de conversion qui passe sur les 3 421 lignes. En déduire ce que deviennent les 11 lignes à trois chiffres.
- 1,5 pt** — Calculer le total de la colonne réparée, puis celui de `montant_ht + montant_tva` sur les mêmes lignes, et écrire l'écart.
- 1 pt** — Compter les lignes de retour au montant positif avant et après votre conversion. Quel second contrôle vient de passer ?
- 1,5 pt** — Écrire dans l'onglet `CONTROLES` la fiche de l'opération : source, transformations dans l'ordre, nombre de lignes avant/après, les deux totaux, l'écart, le nom des deux colonnes ajoutées, la date.
- 2 pt** — Un collègue propose d'ajouter `ARRONDI(...; -2)` pour « lisibilité des tableaux de bord ». Chiffrez ce que ça change au total annuel et concluez. (Indice : vous avez 240 000 lignes propres à comparer, et l'écart accepté sur le chiffre d'affaires était de 2 111 789 FCFA avant réparation.)

11. Exercices autonomes

Exercice 1 — Le gain, avant la formule (20 min). Sur `clients.csv`, écrivez pour `nom`, `ville`, `telephone`, `email`, `conditions_paiement` une ligne de mesure : modalités avant, modalités après normalisation (élagage + casse + ponctuation retirée), lignes modifiées. Classez les colonnes en

trois colonnes : à normaliser pour une clé / à normaliser pour un affichage / à ne pas toucher. Justifiez chaque « à ne pas toucher » par un chiffre.

Exercice 2 — L'import hostile (25 min). Importez `tarif_fournisseur_peinture.csv` deux fois : une fois en Windows-1252 avec 3 lignes sautées, une fois en ISO-8859-1 sans rien sauter. (a) Combien de colonnes dans chaque cas, et pourquoi ? (b) Comptez les caractères dont les deux imports divergent ; vérifiez que ce nombre correspond à une fenêtre précise d'octets, en comparant les deux tables. (c) Le prix 2900,57 : combien de lignes se lisent en nombre, combien en texte, selon la locale choisie ? Écrivez la formule qui rend le résultat indépendant de la locale.

Exercice 3 — Les clés qui manquent (30 min). Construisez la grille complète `magasin × mois` à partir de `objectifs_de_ca.csv` (5 magasins, 44 mois) et joignez-y la table reçue. (a) Combien de cases attendez-vous, combien en trouvez-vous, lesquelles manquent ? (b) Ajoutez `magasins.csv` à la comparaison : que découvre-t-on de plus ? (c) Votre tableau croisé affichera-t-il les trous comme des zéros ? Écrivez la mesure à ajouter pour que la différence soit explicite.

Exercice 4 — Le format comme preuve (20 min). Sur les 412 lignes que `clients_propres.csv` a retirées, testez la règle « le nom porte un espace en surplus » et la règle « le téléphone n'a pas d'espace après le préfixe ». Combien de lignes conservées portent chacune de ces anomalies ? Écrivez la règle de dédoublonnage alternative qui découle de ces deux compteurs, et dites ce qu'elle a de plus fragile qu'une règle sur le contenu.

12. Correction détaillée

Exercice 1. Réponses mesurées : `nom` 12 640 → 12 342 modalités, à normaliser (c'est la clé du C03) ; `ville` 6 → 6, à ne pas toucher ; `telephone` 23 499 valeurs distinctes pour 23 912 lignes, à normaliser en chiffres seulement (413 numéros portés par au moins deux lignes : la normalisation doit réduire l'écriture, pas l'identité) ; `email` rien à reformater, les 16 514 sans @ sont vides ; `conditions_paiement` à conserver en l'état pour l'affichage, la valeur contractuelle n'est pas une écriture. Le barème est atteint dès qu'une case « à ne pas toucher » porte un chiffre au lieu d'une intention.

Exercice 2. (a) En ISO-8859-1 sans saut de lignes : une seule colonne, nommée d'après la ligne 1, et 42 lignes dont 3 de bruit ; en Windows-1252 avec 3 lignes sautées : 6 colonnes nommées correctement, 38 lignes. Le séparateur est le ; — la différence de nombre de colonnes vient du séparateur, pas de l'encodage. (b) 95 octets non ASCII, dont 39 dans la fenêtre 0x80-0x9F : c'est exactement le nombre de caractères qui diffère entre les deux tables, et ce sont les tirets demi-cadratin des libellés ; en Latin-1 ils deviennent des caractères de contrôle non recherchables. (c) Sous une locale à virgule, les 38 prix se lisent en nombres et les marges à point des objectifs deviennent du texte ; l'inverse sous locale à point. La formule insensible est `=CNUMPARAGRAPH(SUBSTITUE(D2;" ";",";"");".";"",")`, où l'on énonce explicitement le séparateur décimal et le séparateur de milliers au lieu de les hériter du poste.

Exercice 3. (a) $5 \times 44 = 220$ cases attendues, 218 reçues, 2 manquantes : février et mars 2024 au magasin 4 — consécutives, donc import ou collé interrompu plutôt qu'oubli de saisie. (b) En confrontant à `magasins.csv` (6 lignes), on découvre le magasin 6 : 0 case d'objectif, ce que l'exercice (a) ne pouvait pas voir, faute de référentiel. (c) Non : un tableau croisé affichera une absence si la ligne n'existe pas dans la source, un zéro si elle existe avec 0 — et rien ne distingue les deux à l'écran. La mesure à écrire est un compteur de lignes jointes par `magasin × mois`, comparé au nombre de cellules du croisé ; les deux doivent être égaux, sinon le croisé ment.

Exercice 4. Les deux anomalies sont portées par 412 lignes sur 412, et par 0 ligne conservée. La règle alternative est donc : retirer toute ligne dont le nom contient un espace en surplus et le téléphone n'est pas formaté, ce qui rend exactement les 412 lignes de la référence. Ce qu'elle a de plus fragile : elle décrit un lot d'import passé, pas une propriété du client. Le jour où un poste de saisie soigné produira des doublons bien formatés, la règle ne verra plus rien — alors qu'elle

aura l'air de fonctionner, parce qu'elle rendra 0 ligne à retirer. D'où la conclusion à écrire : on documente la règle de format comme **indice de jeu**, et la règle métier (nom normalisé + chiffres du téléphone) comme **règle de décision**.

13. Mini-projet M04.P5 — « Le protocole de normalisation » (1 h 30)

Livrable unique : l'onglet `PROTOCOLE` d'un classeur, plus le fichier `ventes_normalisees.csv` qu'il décrit. Le protocole se présente sous la forme d'une table de cinq colonnes, une ligne par transformation :

colonne à produire	contenu attendu
ordre	1 à 8, l'ordre d'application, sans trou
operation	au tableur : nom de la formule ou de l'étape Power Query
objet	la colonne visée, et sa nature (type / écriture / correspondance / unité)
effet	deux nombres : lignes modifiées, valeurs distinctes avant → après
controle	la case qui prouve que rien n'a été perdu, avec sa valeur mesurée

Quatre exigences de forme, qui seront notées :

- aucune ligne du protocole ne retire de ligne au fichier (le compteur de lignes est identique en tête et en queue du protocole) ;
- le total de la colonne monétaire réparée apparaît avec son écart à la voie de reconstitution, écart écrit à 0 ;
- la normalisation des clés apparaît **avant** la ligne qui décrit la jointure aux objectifs ;
- une ligne du protocole dit explicitement « aucune transformation nécessaire » là où la mesure a montré un gain nul (ville, unité en tant que texte).

Barème (/10) : protocole complet et rejouable, 3 ; contrôles chiffrés et écart de convergence, 3 ; distinction des niveaux type / écriture / correspondance / unité respectée, 2 ; absence de destruction de la source, 1 ; lisibilité de la table (une ligne, une opération, un effet), 1.

14. Résumé du chapitre

Normaliser change l'écriture sans changer la valeur, et se reconnaît à ce qu'il ne retire aucune ligne : les trois niveaux d'anomalie — type, écriture, sens — se traitent séparément, le troisième étant l'affaire des règles de contrôle et non des formules. Sur ce socle, la conversion des 3 421 textes monétaires se fait par un double `SUBSTITUE` qui passe sans exception, et sa justesse se prouve par convergence : 15 639 751 286 FCFA par le découpage, le même nombre par la reconstitution `montant_ht + montant_tva`, écart de 0 franc ; le même geste fait passer le contrôle des retours de 47 lignes ratées à 0. La mise en forme n'est pas un détail : l'espace en trop et le téléphone sans espaces désignent les 412 lignes à retirer avec une exactitude que la ressemblance des noms n'a pas. Le gain d'une normalisation se mesure avant de s'écrire — 298 noms gagnés, 0 sur la ville — et le zéro est une conclusion qui s'écrit. Enfin, la norme ne dit rien du monde : 0 date mal formatée, 51 dates impossibles, et 4 unités physiques que seule une table de conversion rend comparables.

15. À retenir

- **Normaliser n'est pas nettoyer** : le premier garde les lignes, le second les compte et les retire. Écrivez « lignes avant = après » à la fin de toute normalisation.

- **Deux voies indépendantes, un écart à l'unité** : c'est la seule preuve qu'une conversion de type n'a rien inventé. Le contrôle se fait **avant** toute correction de sens, pour que les voies restent indépendantes.
- **Mesurez le gain avant d'écrire la formule** : modalités distinctes avant / après, lignes modifiées. Une formule à gain nul est du décor qui vieillit mal.
- **Un format peut être une preuve** : 412 lignes sur 412 portent deux anomalies d'écriture, 0 ligne conservée. Consignez-le comme indice de rejeu, pas comme règle de décision.
- **La norme n'est pas la justesse** : 0 date hors norme ISO, 51 dates impossibles. Un contrôle de format tout seul ne contrôle rien.
- **Les unités ne se normalisent pas au texte** : m3 et litre demandent un facteur et une table, pas une fonction.

16. Évaluation formative (auto-correction, 8 min)

1. Citez les trois niveaux d'anomalie d'une colonne et l'opération licite à chacun.
2. Pourquoi le contrôle de convergence sur le total doit-il précéder la correction de `taux_remise` ?
3. Combien de lignes des 3 421 textes monétaires ne comportent qu'une seule espace, et qu'est-ce que ça change à la formule ?
4. Une normalisation fait chuter le compteur de lignes de huit unités. Que s'est-il passé, et quelle case du protocole le voyait avant de livrer ?
5. `ville` a 6 modalités avant, 6 après normalisation. Que fait-on de la colonne `ville_normalisee` ?
6. Quels deux réglages d'importation suffisent à rendre le fichier de tarif lisible, et combien de caractères coûtent le mauvais choix d'encodage ?

Corrigé : (1) type → convertir ; écriture → élaguer, harmoniser la casse, retirer les séparateurs ; sens → reconstituer ou documenter, jamais forcer. (2) Parce que la correction de sens rendrait `montant_ht` fonction de la convention choisie : les deux voies ne seraient plus indépendantes et leur accord ne prouverait plus rien. (3) 11 — les montants à trois chiffres (680 FCFA) ; rien pour le double `SUBSTITUE`, tout pour une découpe par positions. (4) On a supprimé 8 lignes sous prétexte de format ; le compteur avant/après, écrit avant de commencer, l'aurait signalé. (5) On ne la crée pas : un gain nul s'écrit comme une décision. (6) Séparateur `;` et encodage Windows-1252 (plus 3 lignes sautées) ; 39 caractères, ceux de la fenêtre `0x80-0x9F`.

Module M04.C06 — Documenter, nommer, protéger : la dernière étape est une écriture, pas un calcul

Outil de ce chapitre : le tableur et l'explorateur de fichiers, sur le socle entier (24 fichiers de données), NOTICE.md, verites_terrain.csv et les paires brut → propre. **Durée indicative :** 5 h. **Niveau :** N2.

L'idée du chapitre. Cinq chapitres vous ont appris à rendre un fichier digne de confiance. Le sixième pose la question que personne ne veut poser en réunion : et si vous partiez la semaine prochaine ? La réponse ne tient ni dans une formule ni dans un classeur : elle tient dans quatre artefacts texte — l'arborescence, les noms, le README de données, le journal des transformations — plus un contrôle final qu'on sait lire et un périmètre de protection des données personnelles. Le socle que vous nettoyez est, lui-même, un mauvais exemple : NOTICE.md compte 56 lignes et n'y fait mention d'aucun journal, d'aucune version, d'aucun encodage, d'aucune anonymisation — et il n'existe aucun fichier de journal dans data/reference/. Le chapitre se termine donc là où le socle s'arrête : en vous demandant d'écrire ce qui manque, pour le socle lui-même.

Base de travail. Les 24 fichiers de 01_socle_donnees/data/ (10 dans brut/, 12 dans reference/, 2 dans projection/), 01_socle_donnees/data/reference/NOTICE.md (56 lignes, 6 sections), 01_socle_donnees/data/reference/verites_terrain.csv, et les quatre paires brut → propre : ventes_brutes.csv (243 360) / ventes_propres.csv (240 000), clients.csv (23 912) / clients_propres.csv (23 500), produits.csv / dictionnaire_produits.csv, tarif_fournisseur_peinture.csv / dictionnaire_produits.csv. Tous les nombres viennent de ces fichiers.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **organiser** un dossier de données en trois zones — brut, référence, projection — et dire pourquoi le brut se lit mais ne s'écrit jamais ;
- **nommer** des fichiers et des colonnes sans créer les deux pièges du socle : même nom, deux sens ; même chose, deux noms — et chiffrer le premier (12 colonnes portées par deux fichiers ou plus) ;
- **écrire** un README de données de six sections et un **journal des transformations** dont chaque ligne se referme par un compteur avant → après ;
- **lire** un contrôle de cohérence final qui n'est pas vert à zéro : le fichier propre garde 80 collisions, et c'est un résultat correct, pas un échec ;
- **délimiter** le périmètre des données personnelles sur le socle (23 912 clients nommés, 22 vendeurs, 6 responsables, 3 rôles) et chiffrer le risque de ré-identification (87,3 % des clients désignés par quatre colonnes anonymes).

2. Pourquoi cette notion est importante

Tout ce que les cinq chapitres précédents ont produit est **rejetable** — c'est-à-dire qu'il sera rejoué par quelqu'un d'autre, dans un an, sur un fichier qui n'est pas tout à fait le même. Un doublonnage sans journal, c'est une décision qu'on devra prendre deux fois : la première fois, on

l’a prise pour de bon ; la seconde, on la prend pour la même. Et en attendant, trois accidents déjà présents dans le socle montrent ce que ça coûte :

- **12 colonnes** portent le même nom dans deux fichiers bruts ou plus (`nom`, `ville`, `region`, `id_client`, `id_magasin`, `id_produit`, `id_vendeur`, `fournisseur`, `tva`, `mois`, `annee`, `annee_mois`) — et `nom` ne veut pas dire la même chose dans `clients.csv` (un client) et dans `magasins.csv` (un magasin). Sans dictionnaire, la jointure `clients.nom = magasins.nom` s’écrit naturellement et produit un fichier absurde que personne ne contrôlera ;
- **2 fichiers** sur 24 portent un nom qui rompt la convention du dossier (`NOTICE.md` en majuscules, `ventes_magasin5_2025_ATTENDU.csv` avec un segment en majuscules) — rien ne casse aujourd’hui, mais le script qui listera « tous mes CSV » en 2027 tranchera sur une convention ;
- le socle **n’a aucun journal des transformations**. On sait que 3 360 lignes ont disparu de `ventes_brutes.csv` en passant à `ventes_propres.csv` — on ne sait pas, dans les livrables, par quelle règle, quand, par qui. Le chapitre C03 l’a déduite en forensique (bloc ajouté en fin de fichier, garde au plus petit identifiant) : une déduction qui a coûté une heure de mesures, et que le jour du rejeu personne n’a l’obligation de refaire si une ligne de journal l’énonce.

La documentation n’est pas la récompense du travail terminé : c’est la condition de sa reproductibilité. Un contrôle qui n’est pas rejouable ne prouve rien — et un nettoyage qui n’est pas écrit ne s’est pas fait.

3. Explication simple

Imaginez une boîte que vous recevez : dix sachets de poudre, une fiche d’instructions en six phrases, aucune date, aucun nom, aucun numéro de lot. Les poudres sont bonnes — vous les avez testées, tout est conforme. Mais vous ne pouvez ni remonter la production (pas de lot), ni prévenir un défaut (pas de date), ni dire à qui il manque un ingrédient (pas de nom). La qualité était là ; la **traçabilité** n’était pas là, et c’est elle que vous vendez vraiment.

Un dossier de données est cette boîte. Le brut est le sachet tel qu’il a été ouvert — on ne le ré-empaquete pas, on le conserve pour l’expertise. La référence est la fiche de contrôle : ce que les poudres devraient donner. La projection est l’échantillon qu’on donne à qui veut tester sans ouvrir la caisse. Le journal est le carnet de bord : « le 3, on a tamisé la poudre 2 ; avant : 5 kg, après : 4,98 kg ; raison : sable. » Et la protection, enfin, est la décision de ne pas envoyer la boîte à tout le monde : si la fiche porte des noms de clients, l’échantillon qu’on distribue ne les porte plus — ou bien on explique pourquoi on les y laisse.

Chaque pièce se vérifie en une minute, et les six pièces ensemble font la différence entre « les chiffres sont bons » et « les chiffres sont bons **et je peux le démontrer à un inconnu** ».

4. Vocabulaire essentiel

Terme	Ce que ça désigne, dans ce chapitre
Zone de données	Un sous-dossier à règle d’écriture unique : <code>brut</code> (lecture seule), <code>reference</code> (la vérité), <code>projection</code> (les échantillons)
Convention nommage	de La règle commune des noms : <code>minuscules_et_tirets</code> ici ; elle se vérifie par un scan, pas par un coup d’œil
Clé d’homonymie	Une colonne qui porte le même nom dans deux fichiers avec deux sens (<code>nom</code>) — le piège n° 1 des jointures
README de données	La fiche du dossier : ce que c’est, quelles tables, quelles règles, ce qui manque volontairement
Journal transformations	des Le registre des opérations : date, fichier, opération, règle, compteur avant → après, auteur

Terme	Ce que ça désigne, dans ce chapitre
Contrôle de cohérence final	Le passage complet de la grille de 12 points (C01) sur le fichier propre, lu comme un résultat et non comme un feu vert
Identifiant direct	Une colonne qui désigne une personne sans intermédiaire : nom, téléphone, e-mail
Quasi-identifiant	Une colonne (ou une combinaison) apparemment anonyme qui désigne une personne dans une part mesurable des cas
Anonymisation	Transformer des données pour qu'une personne ne soit plus identifiable — irréversible par construction, ou documentée comme pseudo-anonymisation
Faux anonymisage	Supprimer l'identifiant direct en laissant un quasi-identifiant unique : 87,3 % des clients du socle restent désignés

5. Cours approfondi

5.1 L'arborescence : trois zones, une règle d'écriture chacune

Le socle contient **24 fichiers** de données, répartis en trois sous-dossiers qui ne jouent pas le même rôle :

Zone	Fichiers	Rôle	Règle d'écriture
brut/	10	l'arrivée, telle qu'on l'a reçue	lecture seule — jamais réécrite
reference/	12	la vérité : fichiers propres, dictionnaires, verites_terrain.csv, NOTICE.md	s'écrit par décision, jamais à la volée
projection/	2	les échantillons distribués aux modules M01/M03	dérivé : se recrée à tout moment

La règle qui rend cette organisation utile n'est pas le tri, c'est l'**asymétrie d'écriture**. Tant qu'une même main peut écrire dans le brut et dans le propre, personne ne sait d'où vient un chiffre : le brut est supposé être le point de départ, mais si on y touche, il n'est plus une preuve. Le socle obéit à la règle — c'est pour cela qu'on a pu, dans le C03, raisonner sur les 3 360 lignes disparues en comparant les deux zones sans jamais se demander si la zone de départ avait bougé.

Dans les faits. La projection prouve la règle. `projection/ventes_magasin5_2025.csv` n'est pas un extrait coupé avec le ciseau : il est recréé à chaque génération par le script, et `reference/ventes_magasin5_2025_ATTENDU.csv` qui dit ce que doit donner la recomposition. Si demain quelqu'un modifie la projection à la main, la comparaison avec l'ATTENDU le révèle. C'est exactement ce que vous devrez faire pour vos propres échantillons : **un échantillon sans son attendu est une zone grise.**

5.2 Le nommage : mesurer la convention avant de la proclamer

Le scan du dossier dit : **68 colonnes distinctes** dans les 10 fichiers bruts, **toutes** en `minuscules_et_tirets_bas` — aucune majuscule, aucun espace, aucun accent. Côté fichiers, la convention est `minuscules.csv` : **2 fichiers sur 24** la rompent (`NOTICE.md`, `ventes_magasin5_2025_ATTENDU.csv`). Un scan qui matche 22 fichiers sur 24 a déjà une information ; un scan qui ne matche rien n'en a pas — c'est pour cela qu'on écrit le nombre de fichiers testés à côté du nombre de violations, et pas seulement le chiffre « 2 ».

Le vrai risque n'est pas l'orthographe, c'est le **sens**. **12 colonnes** portent le même nom dans deux fichiers ou plus, et voici ce que le socle en fait :

Nom partagé	Où	Ce qu'il désigne dans chaque fichier
nom	clients.csv, magasins.csv	le nom du client / le nom du magasin — deux sens, un nom
ville, region	clients.csv, magasins.csv	la ville du client / la ville du magasin — jointure tentante, résultat faux
fournisseur	produits.csv, couts_achat.csv	le fournisseur du produit / le fournisseur du coût — même objet, à vérifier
id_client, id_magasin, id_produit, id_vendeur	3, 5, 4, 2 fichiers	des clés — mais id_client est texte dans les CSV et nombre dans remises_manuelles.xlsx
mois, annee_mois	annee, 3, 3, 2 fichiers	des temps — en entier dans le classeur, en texte dans les CSV
tva	2 fichiers	le taux dans produits.csv et le taux dans tarif_fournisseur_peinture.csv — même sens ici (0,18 des deux côtés), mais le dictionnaire doit le certifier, car « tva » peut être un taux d'un côté et un montant de l'autre

Ajoutez à cela l'inverse du piège : **même chose, deux noms** — nom dans clients.csv et nom_complet dans vendeurs.csv. C'est la documentation (un dictionnaire de colonnes) qui tranche, jamais le nom. Et le premier fichier qu'on ouvre sans dictionnaire est celui qui casse la jointure.

Attention. Le BOM est le nom qui se change sans qu'on le voie. 5 fichiers bruts sur 10 commencent par trois octets EF BB BF ; lus en Windows-1252 (le réglage par défaut de l'assistant d'importation), le premier en-tête de ventes_brutes.csv ne s'appelle plus id_vente (8 caractères) mais i»id_vente (11 caractères). Une colonne dont le premier caractère est invisible change tous les scripts qui la citent par son nom, sans changer un seul chiffre. La convention de nommage couvre donc aussi l'**encodage déclaré** : un nom n'est constant que si tous le lisent de la même façon.

5.3 Le README de données : six sections, pas une de plus

NOTICE.md est le README du socle : **56 lignes, 6 sections** — le scénario, les tables, les défauts injectés (avec leur corrigé rapport_defaults.json), les règles de gestion, l'extrait M01/M02, et « ce qui n'est **pas** dans le jeu ». Cette dernière section est la plus utile du dossier : elle dit d'avance que la marge nette ne se calcule pas (pas de coûts de structure) et que 2026 est une année partielle — les deux questions que pose nécessairement n'importe quel lecteur.

Le README de données ne contient pas le travail, il contient les **quatre réponses** que le lecteur ne peut pas trouver ailleurs :

1. **Qu'est-ce que c'est ?** — le scénario, en cinq lignes, et la taille réelle : ventes_brutes.csv pèse 28 516 920 octets, 243 360 lignes, 20 colonnes ;
2. **Quelles tables, quel grain ?** — une ligne par fichier : « 1 ligne = 1 ligne de vente », « 1 ligne = 1 client » ;
3. **Quelles règles de gestion ?** — TVA 18 %, remise en proportion, retours négatifs, id_client = 0 = comptoir ;
4. **Ce qui manque volontairement** — et pourquoi ça change le calcul.

Ce que la NOTICE ne contient pas, le chapitre l'a compté : 0 mention d'un **journal**, 0 d'une **version**, 0 d'un **encodage** (BOM, cp1252, séparateur ; des objectifs), 0 d'une **anonymisation**. Ce ne sont pas des reproches contre le socle : c'est la liste de courses du chapitre. Un README qui ne dit pas comment relire le fichier (encodage, séparateur, en-tête) et comment le re-produire (génération, journal, version) rend chaque lecture un acte de foi.

Définition. Un **README de données** est le document qui répond, sans ouvrir les fichiers : ce que contient le dossier, à quel grain, sous quelles règles de gestion, et ce qui y est volontairement absent. Son critère de qualité n'est pas la complétude mais le **temps de mise en route** : un analyste qui n'a jamais vu le dossier doit pouvoir produire, en moins d'une heure, un premier chiffre que le README lui laisse vérifier.

5.4 Le journal des transformations : une ligne par opération, un compteur par ligne

Le socle nous laisse voir, par comparaison des zones, **quatre opérations** qui ont produit les fichiers propres :

Date	Fichier	Opération	Règle	Avant	Après	Auteur
—	ventes_brutes.csv ventes_propres.csv	→ dédoublonnage	garder le plus petit id_vente par ticket + produit (C03)	243 360	240 000	—
—	idem	reconversion des montants texte	retirer « FCFA » puis les espaces (C05)	3 textes	421 0 texte	—
—	clients.csv clients_propres.csv	→ retrait des quasi-doublons	nom normalisé + chiffres du téléphone (C03)	23 912	23 500	—
—	produits.csv dictionnaire_produits.csv	→ harmonisation des catégories	table de correspondance 15 → 7 (C03)	16 modalités dont 1 vide	7	—

Chaque ligne se **referme** : on peut vérifier l'écart ($3\,360 = 243\,360 - 240\,000$; $412 = 23\,912 - 23\,500$), on peut la rejouer (la règle est citée), on peut la contredire (le chapitre C01 a montré que le corrigé fourni, `rapport_defaults.json`, ne colle pas au fichier — 3 360 annoncés contre 3 421 montants texte mesurés). Le « — » dans les colonnes date et auteur est exactement ce que le journal doit combler : le socle ne les fournit pas.

Définition. Une ligne de **journal des transformations** est une quintuple complète : date, fichier, opération, règle, compteur avant → après (l'auteur faisant la sixième en production). Une ligne incomplète — sans règle ou sans compteur — n'est pas une ligne de journal, c'est une note ; et un journal de notes ne permet ni le rejeu ni l'audit, parce qu'il ne dit ni comment ni combien.

Conseil professionnel. Écrivez le compteur **avant** l'opération, pas après. « 243 360 lignes, je vais retirer 3 360 » qui devient « 240 000 restantes » est une ligne de journal ; « j'ai nettoyé les doublons » est une confidence. La différence tient à un nombre écrit à l'avance — et c'est ce nombre qui, dans six mois, permettra de dire si le nettoyage de 2027 a fait le même travail, ou un autre.

5.5 Le contrôle final : un vert à 0 n'existe pas, et c'est une bonne nouvelle

On repasse la grille des 12 points (C01) sur `ventes_propres.csv`. Résultat mesuré, point par point :

Point	Sur le propre	Lecture
3 · complétude	0 cellule vide	le 0 est attendu
5 · unicité métier	80 groupes en collision sur ticket + produit + quantité	le 0 n'est pas attendu

Point	Sur le propre	Lecture
6 · types	0 montant non numérique	le 0 est attendu
7 · domaines	0 remise hors [0 ; 1]	le 0 est attendu
8 · cohérence $TTC = HT + TVA$	0 écart	le 0 est attendu
10 · actualité	0 date après le relevé	le 0 est attendu
Total TTC	15 419 985 157 FCFA	écart 0 à <code>verites_terrain.csv</code>

Le point 5 n'est pas vert à zéro, et c'est le résultat **correct**. Les 80 groupes (160 lignes) sont des tickets légitimes où le même client — ou pas — achète deux fois le même produit, à la même quantité, **à des heures différentes** : l'heure diffère dans 80 groupes sur 80, le client dans 77, le prix dans 72, et **aucun groupe n'est indistinct** une fois `id_vente` mis de côté (0 sur 80). Le contrôle qui prouve le dédoublement n'est donc pas « plus aucune collision », mais : les collisions restantes sont 80, et elles sont toutes distinctes par une autre colonne. Le C03 avait laissé 3 440 collisions dans le brut ; 3 360 sont parties avec le bloc dupliqué ; 80 restaient, et elles devaient rester.

À retenir. Un contrôle final se lit par **résidus attendus**, pas par zéros universels. Écrivez ce que devrait rester (80 groupes multi-lignes, distincts par l'heure), et le contrôle devient capable de distinguer un fichier propre d'un fichier mal dédoublement — alors que « 0 collision » ne ferait que rejeter le bon fichier.

5.6 Le périmètre des données personnelles : quatre fichiers, un risque mesuré

Où sont les personnes dans le socle ? Mesuré fichier par fichier :

Fichier	Ce qu'il contient	Lignes concernées
<code>clients.csv</code>	nom (23 912), téléphone (23 912), e-mail (7 398), ville (22 001), date de création	23 912 — l'intégralité
<code>vendeurs.csv</code>	<code>nom_complet</code> et <code>date_embauche</code> de 22 salariés	22
<code>magasins.csv</code>	<code>responsable</code> de 6 magasins	6
<code>remises_manuelles.xlsx</code>	<code>saisi_par</code> : 3 rôles , pas de noms (« Assistante commerciale », « Itinérant », « Directeur magasin »)	18 200 lignes, 3 rôles

Le dernier est l'exception qui enseigne : un fichier du socle a déjà pris la bonne décision — séparer la **personne** du **rôle**. Les trois autres la laissent ouverte.

Et voici le chiffre qui doit changer votre façon de « protéger » : si l'on retire les identifiants directs du fichier clients (nom, téléphone, e-mail) et qu'on garde quatre colonnes anonymes — ville, type, segment, date de création —, **18 171 combinaisons sur 20 809 désignent un client unique : 87,3 % des clients restent identifiables**.

Trois colonnes seulement (ville, type, segment) ne désignent personne (0 combinaison unique sur 72) ; c'est la

date qui fait tout le travail de ré-identification. Inversement, l'identifiant direct n'est pas ce qu'on croit

7 265 e-mails sur 7 398 (98,2 %) encodent l'`id_client` dans leur adresse (`contact2596@...` pour le client 2596) — **l'e-mail est la clé primaire recopiée**. L'effacer au nom de la protection détruirait la jointure avant d'avoir protégé quoi que ce soit.

Définition. Un **quasi-identifiant** est une combinaison de colonnes apparemment anonymes qui désigne une personne dans une part mesurable des cas. Sa part se mesure : 18 171 sur 20 809 combinaisons sont uniques ici. L'anonymisation ne peut pas se définir par la liste des colonnes effacées, elle se définit par cette part, et doit le dire (« après effacement, X % des individus restent désignés par les colonnes conservées »).

Définition. Un **identifiant direct** est une colonne qui désigne une personne sans intermédiaire : le nom, le numéro de téléphone, l'adresse e-mail. Sur le socle, la part de lignes touchées est totale pour le nom et le téléphone (23 912 sur 23 912) et de 30,9 % pour l'e-mail (7 398). L'effacement d'un identifiant direct est la première étape, jamais la dernière : les quasi-identifiants font le reste du travail.

5.7 Anonymiser sans casser : l'ordre compte

Deux erreurs, mesurées sur le socle, montrent que l'anonymisation est une **séquence** et non une opération :

- **La troncature qui détruit sans protéger.** Garder les trois premiers chiffres de tous les téléphones du socle ne laisse qu'une valeur distincte (tout le monde est 226) : on a détruit la colonne, pas protégé la personne. À six chiffres, il en reste 90 — de quoi localiser un département, sans doute pas un client. La troncature est un réglage, et comme tout réglage, il se chiffre : « il reste X valeurs distinctes » écrit à côté du « Y avant ».
- **L'effacement qui casse la jointure.** L'e-mail encodant l'identifiant (7 265 cas) ne peut pas être effacé avant d'avoir été **remplacé par un pseudonyme stable** — sinon les 18 200 lignes de `remises_manuelles.xlsx`, jointes sur `id_client`, se retrouvent sans clé le jour où l'on veut vérifier qui a signé quoi.

L'ordre s'écrit donc : (1) **figer les clés** — décider la clé de jointure et, si besoin, la remplacer par un pseudonyme stable avant tout effacement ; (2) **effacer les identifiants directs** ; (3) **mesurer la part de ré-identification** sur les colonnes conservées (ici : 87,3 % avec quatre colonnes) et abaisser jusqu'à un seuil accepté ou documenter le seuil retenu ; (4) **écrire la décision dans le journal** — « anonymisation le [date], colonnes X effacées, Y pseudo-anonymisées, part de ré-identification résiduelle Z %, clé stable : `id_client` ». Sans la quatrième ligne, les trois premières sont perdues au prochain export.

Attention. Une pseudo-anonymisation est une **promesse de réversibilité** : le pseudonyme se réattribue, donc la table de correspondance a une valeur — elle est le document le plus sensible du dossier, plus que les données elles-mêmes. La protéger (périmètre de lecture, journal d'accès) fait partie de l'anonymisation, pas du travail suivant.

Livrer une boîte à données — trois zones, un journal, un contrôle qui attend

La documentation n'est pas la récompense du travail : c'est la condition de son rejeu.

brut/	10 fichiers	l'arrivée, telle qu reçue	lecture seule — jamais réécrite
reference/	12 fichiers	la vérité : propres, dictionnaires, verites_terrain	écrit par décision, jamais à la volée
projection/	2 fichiers	les échantillons distribués aux modules	dérivé : recréable, avec son ATTENDU

Le journal se referme par un compteur

ventes : 243 360 → 240 000 (3 360, règle du C03)

montants texte : 3 421 → 0 (convergence à 0 franc)

clients : 23 912 → 23 500 (412, clé nom + chiffres)

catégories : 16 modalités dont 1 vide → 7

avant - après = retirés : sinon, c'est une note, pas une ligne

Le contrôle final attend des résidus, pas des zéros

12 points : 0 vides, 0 texte, 0 domaine, 0 actualité

unicité métier : 80 groupes (160 lignes) — attendus

distincts par l'heure 80/80, client 77, prix 72 ; indistincts : 0

total : 15 419 985 157 FCFA = verites_terrain, écart 0

le résidu attendu s'écrit avant le contrôle

Protéger, c'est chiffrer : 4 fichiers de personnes, un risque mesuré

nom 23 912 · téléphone 23 912 · e-mail 7 398 · ville 22 001 · 22 vendeurs · 6 responsables · 3 rôles

sans nom/tél/e-mail, 87,3 % des clients restent désignés par 4 colonnes anonymes ; l'e-mail encode l'id dans 7 265 cas

6. Exemple concret : le journal du socle, complété

Voici la table de l'§ 5.4 avec les « — » remplis — c'est exactement le livrable attendu au mini-projet :

Date	Fichier	Opération	Règle	Avant	Après	Auteur
2026-09-18	ventes_brutes.csv ventes_propres.csv	→ dédoublonnage	plus petit id_vente par (ticket, produit, quantité)	243 360	240 000	analyste (rejeu C03)
2026-09-18	idem	reconversion montant_ttc	SUBSTITUE « FCFA » puis espaces, de contrôle convergence à 0 (C05)	3 421 textes	0 texte	idem
2026-09-18	clients.csv clients_propres.csv	→ retrait quasi-doublons	nom normalisé + chiffres du téléphone, 0 faux positif	23 912	23 500	idem
2026-09-18	produits.csv dictionnaire_produits.csv	→ harmonisation catégories	table de correspondance, code inconnu	16 0 modalités (1 vide)	7	idem
2026-09-18	clients.csv clients_anonymes.csv	→ pseudo-anonymisation	clé id_client figée, nom/tél/e-mail remplacés, part résiduelle 87,3 % à documenter	23 912	23 912	idem

Notez la dernière ligne : l'anonymisation **ne change aucun compteur de lignes** (23 912 → 23 912), et c'est normal — elle change les valeurs, pas les lignes. Un journal qui ne sait pas distinguer ces deux effets écrit mal les uns et les autres.

7. Démonstration pas à pas : la boîte à données en huit étapes

Tout se fait dans l'explorateur, le tableur et deux fichiers texte. À chaque étape, un chiffre écrit.

1. **Compter la boîte.** 24 fichiers, 3 zones. Écrire le tableau de l'§ 5.1. Si une zone écrit dans `brut/`, s'arrêter et nommer la main.
2. **Scanner les noms.** Fichiers : convention `minuscules.csv`, 22 sur 24 conformes, 2 à citer. Colonnes : 68 distinctes, 0 hors convention. Colonnes partagées : 12 — pour chacune, une ligne « même nom, quel sens ? » dans le dictionnaire.
3. **Ouvrir chaque fichier et noter** : encodage (5 CSV à BOM), séparateur (; pour `objets_de_ca.csv` et le tarif), en-tête (ligne 1, sauf le tarif : ligne 4, 3 lignes de bruit), type des clés. Cinq colonnes de notes par fichier : c'est l'embryon du dictionnaire.
4. **Écrire le README** en six sections : scénario, tables, défauts, règles, extrait, absences volontaires. 56 lignes de cible : au-delà, on écrit un livre ; en-dessous de 20, on n'a rien dit.
5. **Écrire le journal** : une ligne par opération du C02-C05, avec le compteur avant écrit **avant** l'opération. Chacune doit se refermer : l'écart avant — après doit égaler le nombre d'objets retirés ou modifiés.
6. **Passer la grille de 12 points** sur chaque fichier propre. Écrire les résidus attendus avant de lancer le contrôle : pour `ventes_propres.csv`, « 80 collisions multi-lignes, toutes distinctes par l'heure ».
7. **Vérifier la convergence** : total du propre = 15 419 985 157 FCFA, écart 0 à `verites_terrain.csv`. Écrire l'écart, pas seulement les deux totaux.
8. **Delimenter le périmètre de protection** : les 4 fichiers de l'§ 5.6, la part de ré-identification mesurée (87,3 %), et la décision (anonymiser / pseudo-anonymiser / restreindre) — avec, dans le journal, la date de la décision, pas seulement son effet.

8. Erreurs fréquentes

Symptôme	Cause réelle	Correction
Jointure « normale » qui produit un fichier absurde	clé d'homonymie : <code>nom</code> = client d'un côté, magasin de l'autre	dictionnaire de colonnes ; préfixer les clés (<code>client_nom</code> , <code>magasin_nom</code>)
La colonne <code>id_vente</code> « a disparu » d'un script	BOM : le premier en-tête s'appelle <code>i»{id_vente</code>	encodage déclaré dans le README ; <code>utf-8-sig</code> côté lecteur
« J'ai nettoyé les doublons » — et l'année prochaine, on re-devine la règle	journal sans règle ni compteur	la quintuple complète, compteur écrit avant l'opération
Le contrôle final est « rouge » à cause de 80 collisions	on attendait un zéro universel au lieu du résidu attendu	écrire le résidu attendu avant le contrôle ; vérifier que les 80 sont distinctes
L'anonymisation a « protégé » le fichier — et la jointure ne marche plus	e-mail effacé alors qu'il encodait l'identifiant (7 265 cas)	figer la clé avant l'effacement ; pseudonyme stable
« Il ne reste qu'une valeur distincte » après troncature du téléphone	troncature à 3 chiffres : tout le monde est 226	chiffrer les valeurs distinctes avant/après ; 6 chiffres = 90 ici
Deux fichiers nommés <code>NOTICE.md</code> et <code>*_ATTENDU.csv</code> au milieu des minuscules	la convention n'était jamais vérifiée	le scan, et le nombre de violations écrit à côté du nombre testé

9. Bonnes pratiques professionnelles

- **Une zone, une règle d'écriture.** `brut` en lecture seule, physiquement si possible (permissions du système de fichiers) ; la comparaison des deux zones n'est possible que tant que la zone de départ n'a pas bougé.

- **Le dictionnaire de colonnes avant la première jointure.** Douze colonnes partagées sur 68, c'est une sur cinq : sans dictionnaire, chaque jointure est un tirage. Une ligne par colonne : fichier, sens, type, exemples.
- **Le README en six sections, 56 lignes de cible.** Scénario, tables, défauts, règles, extrait, absences. La section « ce qui n'est pas dedans » est la plus précieuse : elle coupe les questions avant qu'elles ne posent problème.
- **Le journal se referme par un compteur.** Avant – après = retirés ou modifiés ; sinon la ligne est une note.
- **Le contrôle final liste ses résidus attendus.** 80 groupes multi-lignes sur 240 000 lignes, tous distincts par l'heure — écrits avant le contrôle, pour pouvoir les reconnaître.
- **La protection se chiffre.** Part de ré-identification avant (100 % avec nom), après effacement (87,3 % avec quatre colonnes), et le seuil choisi — ou la décision de le laisser, datée.
- **La table de correspondance d'une pseudo-anonymisation est le document le plus sensible** du dossier : elle ré-attribue les pseudonymes, donc son périmètre de lecture est plus restrictif que celui des données.

10. Exercice guidé — la boîte à données du socle (40 min, /10)

Vous recevez le dossier `01_socle_donnees/data/` sans rien savoir. Vous devez pouvoir le remettre en état de jeu à votre successeur en 40 minutes.

1. **1 pt** — Comptez les fichiers par zone et le total. Écrivez la règle d'écriture de chaque zone.
2. **1,5 pt** — Scannez les noms : combien de fichiers rompent la convention, lesquels ? Combien de colonnes distinctes, combien portées par deux fichiers ou plus ? Citez les deux pièges de sens avec leur exemple du socle.
3. **1,5 pt** — Relisez `NOTICE.md` : combien de lignes, combien de sections ? Quelles quatre notions n'y sont **pas** mentionnées (journal, version, encodage, anonymisation) ?
4. **2 pt** — Écrivez la ligne de journal du dédoublonnage ventes : opération, règle, avant, après, écart. Montrez que l'écart s'explique ($3\,360 = 243\,360 - 240\,000$).
5. **1,5 pt** — Passez la grille sur `ventes_propres.csv` : combien de collisions restent, combien de groupes, et par quelle colonne tous les groupes sont-ils distincts ? Écrivez le résidu attendu **avant** de regarder.
6. **1 pt** — Vérifiez la convergence : total du propre, total de `verites_terrain.csv`, écart.
7. **1,5 pt** — Donnez le périmètre des données personnelles : quels fichiers, combien de lignes, et la part de ré-identification si l'on garde ville + type + segment + date de création.

Note /10 : 1 pt par ligne 1-3, 2 pts par ligne 4, 1,5 pt par ligne 5-7, avec 1 point de malus si un compteur est donné sans son dénominateur.

11. Exercices autonomes

Exercice 6.1 (★) — Le scan (15 min). Sur le dossier `data/` : (a) combien de fichiers, par zone ; (b) quels noms de fichiers rompent `minuscules.csv` ; (c) combien de colonnes distinctes, combien non conformes ; (d) listez les 12 colonnes partagées et, pour `nom`, `ville` et `fournisseur`, dites ce que chacune désigne dans chaque fichier.

Exercice 6.2 (★★) — Le journal manquant (20 min). Le socle n'a pas de journal. Écrivez-en les quatre lignes (ventes, montants texte, clients, catégories) avec la quintuple complète. Pour la ligne « montants texte », indiquez le contrôle de convergence qui referme l'opération et sa valeur.

Exercice 6.3 (★★) — Le contrôle qui n'est pas vert (20 min). (a) Combien de collisions ticket + produit + quantité dans `ventes_propres.csv` ? (b) Ces lignes sont-elles des doublons ? Prouvez-le colonne par colonne (heure, client, prix : combien de groupes différents sur chacune). (c) Écrivez le résidu attendu de ce contrôle, formulé pour un lecteur qui n'a jamais vu le C03.

Exercice 6.4 (★★) — La protection chiffrée (25 min). Sur `clients.csv` : (a) combien de combinaisons (ville, type, segment, date de création) désignent un client unique, et quelle part des 23 912 ? (b) Si l'on tronque le téléphone à 3 chiffres, combien de valeurs distinctes restent ? Et à 6 ? (c) Combien d'e-mails encodent l'identifiant client, et quelle en est la conséquence pour l'ordre d'effacement ?

Exercice 6.5 (★★★) — La boîte complète (40 min). Livrez pour le socle : le tableau des zones, le dictionnaire des 12 colonnes partagées, le README en six sections (56 lignes de cible), le journal en quatre lignes, le contrôle final avec ses résidus attendus, et le périmètre de protection avec sa part de ré-identification. Le tout doit se relire sans ouvrir un seul fichier brut.

12. Correction détaillée

Exercice 6.1. (a) $10 + 12 + 2 = 24$ fichiers. (b) 2 : `NOTICE.md` et `ventes_magasin5_2025_ATTENDU.csv`. (c) 68 colonnes, 0 non conforme à `minuscules_et_tirets`. (d) `nom` : nom du client (`clients.csv`) / nom du magasin (`magasins.csv`) ; `ville` : ville du client / ville du magasin ; `fournisseur` : fournisseur du produit (`produits.csv`) / fournisseur du coût d'achat (`couts_achat.csv`) — pour celui-là, il faut vérifier dans le dictionnaire que c'est le même objet des deux côtés. Le piège n'est pas la colonne qui « sonne mal », c'est celle qui « sonne juste » : `ville` et `region` sont des jointures parfaitement naturelles à écrire, et fausses.

Exercice 6.2. Les quatre lignes de l'§ 6, avec l'auteur remplacé par votre nom et la date du jour. La ligne « montants texte » se referme par : conversion par double retrait (suffixe puis espaces), 3 421 → 0, contrôle de convergence : somme des valeurs reconverties + somme des 239 939 valeurs déjà numériques = 15 639 751 286 FCFA, égal à la somme `montant_ht` + `montant_tva` des mêmes lignes, écart 0. Sans ce dernier nombre, la ligne dit qu'on a converti ; avec, elle dit qu'on a converti **sans rien changer**.

Exercice 6.3. (a) 80 groupes, 160 lignes. (b) Non : l'heure diffère dans 80 groupes sur 80, le client dans 77, le prix dans 72, et 0 groupe est indistinct une fois `id_vente` mis de côté. Ce sont des lignes de vente légitimes du même ticket. (c) Résidu attendu : « 80 groupes (160 lignes) en collision sur (ticket, produit, quantité), tous distincts par l'heure ; tout groupe indistinct par toutes les colonnes est un échec du dédoublonnage. » Ce qui rend le contrôle capable de rejeter un mauvais dédoublonnage **sans** rejeter le bon.

Exercice 6.4. (a) 20 809 combinaisons, dont 18 171 uniques — soit 87,3 % des combinaisons désignant un seul client ; en pratique, retirer nom + téléphone + e-mail et garder ces quatre colonnes ne protège que 12,7 % des cas. (b) À 3 chiffres : 1 valeur distincte (tout le monde commence par 226) — la troncature a tout détruit, personne n'est protégé ; à 6 chiffres : 90. (c) 7 265 e-mails sur 7 398 (98,2 %) portent l'`id_client` dans leur adresse : l'e-mail est la clé récupérée, et l'effacer avant d'avoir figé la clé casse la jointure avec les 18 200 lignes de remises.

Exercice 6.5. Attendu : les six pièces de l'§ 7, dont le contrôle final écrit « 12 points, 0 résidu sauf le point 5 : 80 groupes multi-lignes attendus, distincts par l'heure (80/80), 0 groupe indistinct ; le total du propre, 15 419 985 157 FCFA, est égal à `verites_terrain.csv` (écart 0) ». Le piège de l'exercice est la ligne 4 : un dictionnaire qui ne traite que `nom` a raté les 11 autres colonnes partagées — et celles qui tuent les jointures sont justement `ville` et `region`, qui n'inspirent aucun soupçon.

13. Mini-projet M04.P6 — « La boîte à données » (1 h 30)

Livrable : un dossier `boite_ventes/` contenant le socle de ventes **documenté** — c'est-à-dire ce qui se passe si vous partez la semaine prochaine. Huit pièces, dans l'ordre :

1. l'arborescence à trois zones, avec la règle d'écriture de chacune ;
2. le scan de nommage : fichiers et colonnes, avec le nombre testé à côté du nombre de violations ;
3. le dictionnaire des colonnes partagées (les 12), une ligne par colonne ;
4. le README de données en six sections (56 lignes de cible) ;
5. le journal des transformations en quatre lignes, compteur avant écrit en premier ;
6. le contrôle final : grille de 12 points sur le propre, **résidus attendus écrits avant**, total et écart à `verites_terrain.csv` ;
7. le périmètre de protection : 4 fichiers, parts mesurées, décision datée ;
8. le fichier `clients_anonymes.csv` : `id_client` figée, identifiants directs remplacés par des pseudonymes stables, part de ré-identification résiduelle **mesurée et écrite** (attendue : 87,3 % avec les quatre colonnes conservées — ou votre seuil, documenté).

Barème (10 points). Pièces 1-3 complètes et chiffrées (2) · README qui permet la mise en route en moins d'une heure, testée sur quelqu'un qui n'a pas suivi le module (2) · journal dont chaque ligne se referme par son écart (2) · contrôle final avec résidus attendus et écart à la vérité (2) · périmètre de protection mesuré et pseudonymisation réversible et documentée (2). Seuil de validation : 7. Une boîte dont le README ne mentionne pas l'encodage ou le séparateur perd 2 points sur le README, quel que soit le reste : le premier lecteur qui ouvre `objectifs_de_ca.csv` en virgule perdra la demi-heure que le README aurait dû lui rendre.

14. Résumé du chapitre

Documenter, nommer, protéger, c'est écrire **quatre artefacts** avant de déclarer le travail fini, et un **contrôle** qui sait lire ce qui reste. L'arborescence tient en une règle d'écriture par zone — 24 fichiers en trois zones, dont le brut ne s'écrit jamais, et c'est ce qui a permis au C03 de faire sa forensique. Le nommage se vérifie par scan : 68 colonnes conformes, 2 fichiers non conformes, et 12 colonnes partagées dont deux — `ville` et `region` — tueraient n'importe quelle jointure sans dictionnaire. Le README de données répond à quatre questions en 56 lignes ; le journal des transformations referme chaque opération par un compteur avant → après ; le contrôle final attend 80 résidus et pas zéro — et c'est ce qui le rend capable de rejeter un mauvais dédoublement sans rejeter le bon ; la protection des personnes se chiffre avant de se faire : 87,3 % des clients restent désignés par quatre colonnes anonymes, et l'e-mail, qu'on croit identifiant direct, est en réalité la clé primaire recopiée dans 7 265 adresses sur 7 398. Le socle n'avait aucun de ces artefacts : il a fallu les écrire. C'est le chapitre entier en une phrase.

15. À retenir

À retenir. ★ **Une zone, une règle d'écriture.** Le brut se lit, ne s'écrit jamais : c'est ce qui rend la comparaison des zones une preuve (243 360 → 240 000, 3 360 expliquées). ★ **Même nom, deux sens** est le piège n° 1 des jointures : 12 colonnes partagées sur 68, et `nom` désigne un client d'un côté, un magasin de l'autre. Le dictionnaire tranche, pas le nom. ★ **Le journal se ferme par un compteur.** Avant – après = retirés ; sans règle et sans compteur, c'est une note. ★ **Un contrôle final attend des résidus, pas des zéros.** 80 collisions multi-lignes sur 240 000 lignes, toutes distinctes par l'heure (80/80), 0 indistincte — écrit **avant** le contrôle. ★ **La protection se chiffre.** 87,3 % des clients désignés par quatre colonnes anonymes ; l'e-mail encodant l'identifiant dans 7 265 cas sur 7 398 : on fige la clé **avant** d'effacer l'identifiant.

16. Évaluation formative (auto-correction, 8 min)

Q1. Pourquoi le brut est-il en lecture seule ? → parce que la comparaison des zones (brut vs propre) est la preuve du nettoyage ; une zone de départ réécrite n'est plus une preuve, et le journal perd son référentiel.

Q2. Le scan de nommage trouve 2 fichiers non conformes sur 24. Que faut-il écrire, et pourquoi pas seulement « 2 » ? → le nombre testé (24) à côté du nombre de violations (2) : un scan qui ne dit pas ce qu'il a vu est un faux négatif poli — et les 2 noms (`NOTICE.md`, `*_ATTENDU.csv`) nommés, car ce sont des conventions à arbitrer, pas des coquilles.

Q3. Le contrôle final du fichier propre rend 80 collisions. Est-ce un échec ? → non : 80 groupes de lignes légitimes (160 lignes), tous distincts par l'heure (80/80), 0 indistincts hors `id_vente`. Le résidu attendu s'écrit avant le contrôle ; c'est lui qui rend le contrôle capable de distinguer un propre d'un mal dédoublonné.

Q4. On retire nom, téléphone et e-mail des clients. Combien de clients restent désignés si l'on garde

ville + type + segment + date de création ? → 18 171 combinaisons uniques sur 20 809, soit 87,3 % des combinaisons

l'effacement des identifiants directs ne protège que la minorité des cas. La date de création fait le travail des trois autres colonnes à elle seule (0 combinaison unique sans elle).

Q5. Pourquoi l'e-mail ne peut-il pas être effacé en premier ? → parce qu'il encode l'`id_client` dans 7 265 cas sur 7 398 (98,2 %) : c'est la clé primaire recopiée. L'effacer avant d'avoir figé un pseudonyme stable casse la jointure avec les 18 200 lignes de remises sans protéger davantage.

Question ouverte (la plus importante). Votre chef vous demande « est-ce qu'on peut envoyer le fichier clients à l'agence de Lyon ? ». Répondez en quatre phrases, avec les chiffres du socle. Corrigé attendu : (1) le fichier contient 23 912 personnes nommées, dont 7 398 joignables par e-mail et 22 001 localisées en ville ; (2) si l'on anonymise mal, 87,3 % restent ré-identifiables par quatre colonnes anonymes — l'envoi « protégé » ne protège que 12,7 % ; (3) l'e-mail est la clé de jointure (7 265 cas encodent l'identifiant), il ne peut partir qu'après pseudonymisation stable, sinon les 18 200 remises deviennent invérifiables ; (4) décision : soit un extrait pseudonymisé avec part résiduelle mesurée et datée au journal, soit la restriction du périmètre — l'envoi du brut n'est pas une option, il est hors périmètre.

Projet M04.P — « Le dossier de données de la quincaillerie de Koudougou »

Module M04 · projet de fin de module · 6 h · barème /20, seuil de passage 13/20

Le cadre réel. Un gérant de quincaillerie vous a remis un dossier tel qu'il le tient dans son poste : un fichier de commandes de 160 lignes, un fichier de clients, un tarif fournisseur « généré par son logiciel », et une phrase : « je sais que c'est mal tenu, mais les totaux du mois doivent sortir lundi ». Ce dossier contient **sept défauts de natures différentes**, dont **deux qui cassent les totaux** — pas plus, pas moins. Vous rendez la boîte en état de rejeu : un diagnostic chiffré, un fichier réparé dont on sait d'où viennent les nombres, un journal, un README, un contrôle final qui sait ce qui doit rester, et un périmètre de protection pour les clients. C'est la synthèse des six chapitres : chaque livrable en mobilise un, et aucun n'est noté pour lui-même.

Note de cohérence. Le dossier est généré de façon déterministe par `python3 tools/dossier_M04.py` (graine 41) : qui que vous soyez, vous touchez les mêmes 160 lignes. `03_exercices/dossier_M04/ATTENDU.json` contient les résultats attendus, **mesurés sur le dossier généré, jamais déduits** : vous n'y recourez qu'après avoir produit vos propres nombres, pour comparer, pas pour copier. Un candidat qui lit l'ATTENDU avant de calculer n'a plus rien à rendre. Les chiffres de ce corrigé sont ceux du dossier livré, vérifiables ligne à ligne.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Commandes	<code>03_exercices/dossier_M04/commandes_2026.csv</code>	160 lignes, 20 colonnes, encodage UTF-8 avec BOM, séparateur virgule
Clients	<code>03_exercices/dossier_M04/clients_2026.csv</code>	132 lignes, 11 colonnes, UTF-8 avec BOM
Tarif fournisseur	<code>03_exercices/dossier_M04/tarif_fournisseur.csv</code>	10 lignes utiles, séparateur point-virgule, encodage Windows-1252, en-tête décalé
Résultats attendus	<code>03_exercices/dossier_M04/ATTENDU.json</code>	à utiliser après le calcul, jamais comme source de copie
Outils	un tableur (Excel 2021+ ou LibreOffice Calc) ; sinon : la voie accessible, et vous l'écrivez Power Query si votre poste l'a	

Le dossier contient sept défauts de natures différentes ; deux d'entre eux faussent le total de la somme directe, les cinq autres faussent une analyse, une jointure ou un import — sans jamais changer un seul montant. Trouver les sept, les séparer en deux natures, et prouver la séparation, c'est le premier tiers de la note.

Les six livrables

D1 — La grille du dossier (3 points). La grille des 12 points (C01), passée sur `commandes_2026.csv` et `clients_2026.csv` : pour chaque point non nul, le compteur **avec son dénominateur** (« 17 sur 160 », jamais « 17 »). Vous identifiez les sept défauts, vous les classez en casse un total / fausse une analyse, une jointure ou un import, et vous écrivez la phrase qui prouve la

séparation : quels sont les deux défauts qui faussent la somme directe, et pourquoi les cinq autres ne la faussent pas.

D2 — Le fichier réparé et son total (4 points). `commandes_2026_propres.csv` : 152 lignes, total TTC **8 209 940 FCFA** — à obtenir par votre méthode, pas à viser. Les 8 lignes retirées s'écrivent dans la feuille de travail avec leur motif : 7 copies exactes (retirées sur la règle du contenu) et 1 **resaisie** — une ligne jumelle qui ne diffère de son original que d'un champ (la date) et porte le plus grand `id_vente` ; le piège de l'énoncé, c'est que le fichier compte **9 groupes en collision** : le neuvième est un ticket multi-lignes **légitime** (deux lignes qui diffèrent sur l'heure, le client, le prix et le mode de paiement) et ses 2 lignes doivent survivre. Une règle qui ne garde qu'une ligne par groupe (le plus petit `id_vente`, sans regard) tombe à 151 lignes et 8 105 333 FCFA : vous écrivez ce que ce total a mangé.

D3 — Le journal des transformations (3 points). Cinq lignes au format du C06 (date, fichier, opération, règle, compteur avant → après), couvrant au minimum : la conversion des 17 montants texte (dont 2 négatifs qui sont des retours), le retrait des 8 doublons, la reconstruction des 12 dates hors norme, et le retrait des 2 quasi-doublons clients. Chaque ligne se referme : avant – après = retirés ou modifiés.

D4 — Le README du dossier et le dictionnaire des trois fichiers (3 points). Six sections, 56 lignes de cible (C06) : scénario, tables avec leur grain, règles de gestion (retour = montant négatif ; `id_client = 0` = comptoir, **pas** une donnée à remplir — 18 lignes dans le dossier), défauts constatés, ce qui manque volontairement. Le dictionnaire déclare pour chaque fichier : encodage (les deux CSV du dossier portent un BOM ; le tarif est en Windows-1252 — un import en Latin-1 y insère des caractères de contrôle invisibles), séparateur (; pour le tarif), position de l'en-tête (ligne 4 pour le tarif, 3 lignes de bruit avant), et le type de chaque clé. Les 10 prix du tarif sont à virgule décimale : le README dit comment les lire sur une machine à point.

D5 — Le contrôle final avec ses résidus attendus (3 points). La grille de 12 points sur le fichier réparé, avec les résidus attendus **écrits avant** de lancer le contrôle (C06) : 0 montant non numérique, 0 remise hors [0 ; 1] — à condition d'avoir traité les 5 remises en points, sinon 5 résidus —, 0 date hors norme ISO, 0 groupe de doublon, 1 groupe multi-lignes légitime à 2 lignes. La convergence est écrite avec l'écart : votre total 8 209 940 FCFA contre l'ATTENDU.json, écart 0 — l'écart est le chiffre, pas le total.

D6 — Le périmètre de protection de `clients_2026.csv` et sa version pseudonymisée (4 points). Vous délimitez les données personnelles (nom, téléphone : 132 lignes ; e-mail : les lignes renseignées ; ville, date de création), vous mesurez ce que l'effacement des identifiants directs ne protège pas, et vous produisez `clients_2026_anonymes.csv` : `id_client` figée et remplacée par un pseudonyme stable, nom/téléphone/e-mail remplacés, **part de ré-identification résiduelle mesurée et écrite**. Le dossier compte 12 noms portés par deux téléphones ou plus (homonymes) : vous écrivez pourquoi « dédoubler sur le nom » y retirerait des clients réels, et pourquoi la règle qui retire exactement les 2 quasi-doublons (132 → 130) est la clé nom normalisé + chiffres du téléphone — pas le nom seul.

2. Ce qui est noté, ce qui est piégé

Piège du dossier	Où il se cache	Ce qu'il enseigne
17 montants en texte (« n nnn FCFA »), dont 2 négatifs	<code>commandes_2026.csv</code> , colonne N	la somme directe rate 17 lignes ; les 2 négatifs sont des retours que le contrôle de domaine ne voit qu'après conversion
8 lignes en doublon (7 copies + 1 resaisie)	<code>commandes_2026.csv</code>	la resaisie n'est pas une copie exacte : la règle « contenu identique » ne la voit pas, la règle « un seul champ différent » la voit

Piège du dossier	Où il se cache	Ce qu'il enseigne
1 ticket multi-lignes légitime (2 lignes, 8 colonnes différentes)	<code>commandes_2026.csv</code>	« une ligne par groupe » mange une vente réelle (8 105 333 au lieu de 8 209 940)
5 remises en points de pourcentage (12/18/25/30/18)	<code>commandes_2026.csv</code> , colonne K	la division par 100 est un test à faire, pas une réparation à appliquer
12 dates en deux écritures (JJ/MM/AAAA et AAAA-MM-JJ)	<code>commandes_2026.csv</code> , colonne C	le tri chronologique et le pivot mensuel sont faux tant que les deux écritures cohabitent
2 quasi-doublons clients (nom surchargé + téléphone sans espaces) + 12 homonymes	<code>clients_2026.csv</code>	la règle « même nom » retire bien plus que les 2 doublons ; la clé juste est nom normalisé + chiffres
3 lignes de bruit, cp1252, ;, décimales à virgule	<code>tarif_fournisseur.csv</code>	l'import par défaut lit 1 colonne, un en-tête faux, et des prix en texte

3. Barème et seuil

- D1 : 3 points — grille complète avec dénominateurs (1) · les sept défauts en deux natures (1) · la phrase de séparation des totaux (1).
- D2 : 4 points — 152 lignes et 8 209 940 FCFA obtenus (1,5) · les 8 lignes retirées motivées ligne à ligne (1) · le groupe légitime survit et l'erreur de la règle naïve est chiffrée (1,5).
- D3 : 3 points — cinq lignes au format complet (1) · chaque ligne se referme par son écart (1) · la conversion des montants se referme par un contrôle de convergence (1).
- D4 : 3 points — six sections lisibles sans ouvrir les fichiers (1) · encodages, séparateurs et en-têtes déclarés (1) · la convention `id_client = 0` écrite comme convention et non comme donnée manquante (1).
- D5 : 3 points — résidus attendus écrits avant le contrôle (1) · grille du réparé avec le groupe légitime nommé (1) · convergence écrite avec l'écart contre l'ATTENDU (1).
- D6 : 4 points — périmètre délimité et chiffré (1) · pseudonymisation stable qui ne casse aucune jointure (1) · part de ré-identification résiduelle **mesurée et écrite** (1) · la règle des 2 quasi-doublons opposée à la règle du nom seul, avec les 12 homonymes en argument (1).

Seuil de validation : 13/20. Un livrable dont un compteur manque son dénominateur perd 0,5 point par compteur, sans plafond : la précision du dénominateur est la matière du module, pas une coquille.

4. Correction des points d'attention

- **D1, séparation des totaux.** La somme directe sur la colonne N ne rate que les lignes non numériques : 17. Les doublons la faussent en double comptant 8 lignes. Les 5 remises, les 12 dates, les 2 quasi-doublons et l'encodage du tarif ne changent aucun montant : ils faussent une analyse (remises), un calendrier (dates), une jointure (clients) et un import (tarif). La phrase attendue : « Deux défauts cassent le total — 17 montants en texte non sommés et 8 lignes comptées deux fois ; les cinq autres laissent la somme intacte et cassent chacune une autre lecture du fichier. »
- **D2, le groupe légitime.** Le groupe formé du ticket `T01-230101-000140`, produit 74, quantité 3 : deux lignes qui diffèrent sur l'heure, le vendeur, le client, le prix, les trois montants et le mode de paiement. C'est une double vente réelle du même ticket, pas un doublon. La règle naïve « une ligne par groupe, plus petit `id_vente` » la réduit à 1 ligne : 151 lignes, 8 105 333 FCFA — l'écart 104 607 FCFA est le montant de la ligne mangée.
- **D2, la resaisie.** Le groupe du ticket `T01-230103-000122`, produit 103, quantité 3 : deux lignes identiques sauf la date. Ce n'est pas une copie exacte (le contrôle de contenu la laisse pas-

ser) ; c'est une ressaisie avec une erreur de date, reconnaissable à son `id_vente` plus grand. La règle de garde la retire sans toucher au groupe légitime, qui diffère sur 8 colonnes.

- **D5, les résidus attendus.** « 0 non numérique, 0 hors domaine, 0 hors norme, 0 groupe de doublon, **1 groupe multi-lignes légitime à 2 lignes** » — le résidu est écrit avant le contrôle ; un contrôle qui ne s'attendrait qu'à des zéros rejetterait le bon fichier.
- **D6, la part résiduelle.** Elle se mesure sur les colonnes conservées (ville, type, segment, date de création, conditions de paiement, plafond) : sur ce dossier, elle se compte en combinaisons désignant un client unique. L'e-mail, quand il est renseigné, encode souvent l'identifiant : le pseudonyme stable doit exister **avant** l'effacement, sinon la jointure avec `commandes_2026.csv` se rompt.
- **Le tarif.** Lu en Windows-1252 avec 3 lignes sautées : 10 lignes, 6 colonnes, 10 prix à virgule décimale. Lu en UTF-8, il casse à l'octet ; lu en Latin-1 sans saut, il donne une colonne et un en-tête faux. Le README déclare les trois réglages ; la formule de lecture des prix est celle qui énonce la virgule, pas celle qui l'hérite de la locale.

Évaluation M04 — « Qualité, préparation et documentation des données »

Module M04 · durée totale 2 h 30 · quatre épreuves · seuils : quiz 11/15, exercices 14/20, étude de cas 12/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	vérifier qu'on a lu les six chapitres avant de se tromper cher
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, quatre blocs
C · Exercices pratiques	50 min	/20 (auto-corrigé)	oui	quatre exercices menés jusqu'au chiffre sur le dossier du projet
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier en quatre pages : « le stock indiqué est-il crédible ? »

Le score du module se lit ainsi : **B ≥ 11** et **D ≥ 12** valident le module ; le projet M04.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D : savoir ce qu'est une quasi-identification ne dispense pas de dire si le comptage de 60 000 lignes est croyable. Tout nombre de cette évaluation est mesuré sur le socle livré ; les corrections le citent avec leur clé `m04_*/m04p_*` dans `chiffres_cites.json`.

A · Questions de récupération (non notées)

À traiter sans document, en une ou deux phrases chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail.

1. Citez les cinq mécanismes d'absence du C02 et le traitement qui va avec chacun. Pourquoi `id_client = 0` (43 681 lignes, 2 813 668 030 FCFA) n'est-il aucun des cinq ?
2. Donnez les quatre niveaux où l'on peut chercher un doublon (C03) et, pour chacun, le résultat mesuré sur le fichier de ventes (0 / 0 / 3 440 / 8 453).
3. Citez les quatre familles de règles de contrôle (C04) avec un nombre mesuré chacune — et la famille qui a rendu 24 180 alertes pour 0 erreur.
4. Quelles sont les deux voies indépendantes qui reconvertissent les 3 421 montants texte (C05), et quel nombre commun rendent-elles avant dédoublonnage ?
5. Quelles sont les quatre pièces que le C06 attend d'un dossier livré, et quel résidu **attendu** le contrôle final du fichier propre doit-il accepter (80 groupes) ?
6. Pourquoi les 973 remises à 12, 18, 25, 30 ne changent-elles aucun total, et pourquoi la division par 100 est-elle une réparation qui échoue son propre test ?
7. Quelle est la différence entre un identifiant direct et un quasi-identifiant, avec le chiffre du socle qui rend la distinction coûteuse (18 171 combinaisons uniques sur 20 809) ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

Bloc 1 — Types et normalisation (Q1 à Q4)

Q1. La colonne `montant_ttc` contient 3 421 cellules du type `n nnn FCFA` parmi 239 939 nombres. Ce que `=SOMME()` fait des 3 421 : a) les somme comme des nombres · b) les ignore, sans erreur · c) renvoie `#VALEUR!` · d) elles sont lues sans leur espace de milliers.

Q2. Normaliser `clients.nom` (élagage, minuscule, ponctuation retirée) fait passer 12 640 valeurs distinctes à 12 342. Conclusion : a) 298 clients ont été supprimés · b) 298 écritures différentes d'un même nom se sont rassemblées, sans que le nombre de clients change · c) 298 doublons exacts ont été fondus · d) la colonne est corrompue.

Q3. Deux écritures d'un même téléphone : `+226 78 62 21 81 98` et `+2267862218198`. La colonne qui sert de clé de jointure doit porter : a) le texte tel quel · b) les chiffres seuls · c) le nom du client · d) la ville.

Q4. `date_vente` est à 100 % en `AAAA-MM-JJ`, et 51 lignes portent une date postérieure au 31/08/2026, date du relevé. L'action correcte : a) une formule de reformatage · b) un contrôle de domaine ($\text{date} \leq \text{date du relevé}$) et une décision documentée · c) la suppression des 51 lignes · d) aucune, le format est correct.

Bloc 2 — Doublons et clés (Q5 à Q8)

Q5. La clé primaire rend 243 360 valeurs distinctes, mais 3 440 lignes sont en collision sur (`ticket`, `produit`, `quantité`). Conclusion : a) la clé primaire est fausse · b) la clé métier voit ce que la clé technique ne voit pas · c) le fichier est entièrement dupliqué · d) c'est une erreur de mesure.

Q6. 8 453 lignes partagent (`ticket`, `produit`) alors que la règle de garde n'en retire que 3 360. Explication : a) un choix aléatoire · b) un même ticket peut légitimement porter plusieurs lignes sur un même produit (`quantité`, `heure` ou `client` différents) · c) la clé compte mal · d) les 8 453 sont toutes des doublons.

Q7. La règle « même nom » sur les 23 912 clients fusionnerait 11 570 lignes. Réponse correcte : a) l'appliquer puis vérifier a posteriori · b) la rejeter : elle retire 11 158 lignes de trop, des homonymes réels · c) la limiter aux noms courts · d) l'appliquer aux noms avec espaces.

Q8. Les 412 quasi-doublons clients portent tous deux anomalies d'écriture (`nom` surchargé, `téléphone` sans espaces), et 0 ligne conservée ne les porte. Ce qui s'en déduit : a) l'anomalie de format est la règle de dédoublonnage · b) l'anomalie de format est un indice de rejeu, pas une règle : un doublon bien formaté ne serait plus vu · c) le fichier est corrompu · d) il faut supprimer tous les espaces du fichier.

Bloc 3 — Règles et valeurs aberrantes (Q9 à Q12)

Q9. `prix_unitaire_ht` : minimum 575, maximum 45 000 FCFA ; la règle des quartiles en signale 24 180 lignes. Ces 24 180 lignes sont : a) des erreurs de saisie · b) la dispersion légitime du métier (du sac de ciment à la palette d'ossature) · c) du bruit de mesure · d) des doublons.

Q10. La bande $[0,95 ; 1,35]$ autour du prix catalogue rend 0 violation ; l'égalité au catalogue rend 232 981 « anomalies ». Leçon : a) le catalogue est faux · b) une règle qui signale 96 % d'un fichier sain est une règle fausse, pas un fichier sale · c) il faut corriger les 232 981 prix · d) il faut utiliser le z-score.

Q11. 973 lignes ont `taux_remise` à 12, 18, 25 ou 30, mais un `montant_ht` correct et une TVA cohérente. La division par 100 laisse 258 lignes au-dessus du plafond de 0,25. L'action défendable : a)

diviser par 100 · b) reconstituer le taux depuis les montants (étendue 0,000 à 0,110, les 973 dans le domaine) et documenter · c) supprimer les 973 lignes · d) laisser tel quel sans note.

Q12. Le contrôle `qte_physique < seuil_alerte` déclenche sur exactement les mêmes 1 800 lignes que `qte_physique < 0`, sur 60 000. Conclusion : a) le seuil est parfait · b) le seuil ne discrimine rien : le signal réel est le signe ; les deux règles doivent être scindées (intégrité / risque) · c) les 1 800 lignes sont des erreurs à supprimer · d) le seuil est trop bas.

Bloc 4 — Documentation et protection (Q13 à Q15)

Q13. Le fichier propre garde 80 groupes en collision sur (ticket, produit, quantité), tous distincts par l'heure, 0 indistincts. Le fichier est-il correctement dédoublonné ? a) non, à refaire · b) oui : le résidu est attendu, écrit avant le contrôle, et les groupes sont des multi-lignes légitimes · c) c'est un bug du script · d) oui, tant que le résidu reste sous 100 groupes.

Q14. Une ligne de journal des transformations sans règle ni compteur avant → après est : a) une note, pas une ligne de journal · b) une ligne valide, abrégée · c) une exigence d'audit · d) un risque juridique seulement.

Q15. On retire nom, téléphone et e-mail des clients, on garde (ville, type, segment, date de création) : 18 171 combinaisons sur 20 809 désignent un client unique. Conclusion : a) le fichier est anonymisé · b) 87,3 % des combinaisons restent ré-identifiables : effacer l'identifiant direct n'est pas anonymiser · c) seuls 2 638 combinaisons ne désignent personne d'unique · d) il suffit de retirer la ville.

Corrigé du quiz

Q1 **b** — `SOMME` ignore le texte sans erreur : c'est ce qui rend le défaut silencieux (écart du total naïf : 2 111 789 FCFA). Q2 **b** — $12\,640 - 12\,342 = 298$ écritures rassemblées ; le nombre de lignes est inchangé. Q3 **b** — les chiffres seuls : c'est la colonne rare discriminante du C03. Q4 **b** — la norme est respectée, la réalité non ; la décision se documente, elle ne se corrige pas par formule. Q5 **b** — l'unicité technique n'est pas l'unicité métier. Q6 **b** — 8 453 lignes partagent (ticket, produit), mais la clé métier complète (avec la quantité) distingue les multi-lignes légitimes. Q7 **b** — $11\,570 - 412 = 11\,158$ lignes de trop. Q8 **b** — la règle de format décrit un lot passé ; la règle de décision reste la clé. Q9 **b** — 24 180 alertes pour 0 erreur. Q10 **b** — la règle se juge à ce qu'elle attrape, pas au volume d'alertes. Q11 **b** — la reconstitution tient sur les 973 ; la division échoue sur 258. Q12 **b** — deux règles au lieu d'une, un destinataire par niveau. Q13 **b** — le contrôle attend des résidus, pas des zéros universels. Q14 **a** — sans règle ni compteur, la ligne ne referme rien. Q15 **b** — 87,3 % de combinaisons uniques ; la date de création fait le travail des trois autres colonnes.

C · Exercices pratiques (50 min · /20)

Tout se fait sur `03_exercices/dossier_M04/` (le dossier du projet, 160 lignes ; `ATTENDU.json` à consulter **après** vos calculs).

E1 — La grille du dossier (5 points). Passez la grille des 12 points sur `commandes_2026.csv`. Note attendue : les quatre compteurs non nuls **avec dénominateurs** (17 montants texte sur 160 ; 5 remises hors domaine sur 160 ; 12 dates hors ISO sur 160 ; 8 lignes en doublon sur 160), plus la mention des 18 lignes au comptoir comme convention et non comme donnée manquante. 1 pt par compteur justifié, 1 pt pour la séparation « total / analyse ».

E2 — Le dédoublonnage du dossier (5 points). Produisez 152 lignes et le total **8 209 940 FCFA**. Note attendue

les 8 lignes retirées motivées (7 copies exactes + 1 resaisie reconnue à son champ unique, la date), le groupe légitime (2 lignes, 8 colonnes différentes) survécut, et le chiffre de la règle naïve (151 lignes, 8 105 333 FCFA, l'écart 104 607 FCFA identifié comme le montant de la ligne mangée). 2 pts pour le résultat, 2 pour la motivation ligne à ligne, 1 pour l'opposition à la règle naïve.

E3 — Conversion et import (5 points). (a) Convertissez les 17 montants texte par le double retrait (suffixe puis espaces) et montrez que les 2 retours à montant texte ne passent le contrôle de domaine (retour $\Rightarrow$ montant négatif) qu'après conversion : 1,5 pt. (b) Importez `tarif_fournisseur.csv` en déclarant l'encodage Windows-1252, le séparateur point-virgule et les 3 lignes à sauter ; lisez les 10 prix à virgule décimale par une formule qui énonce la virgule ; écrivez le coût du mauvais choix d'encodage (39 octets dont le décodage Latin-1 fait des caractères de contrôle invisibles) : 3,5 pts.

E4 — « Les trois chiffres faux » (5 points). Le corrigé enseignant `rapport_defaults.json` du socle annonce 3 360 montants en texte, 2 640 dates en formats mixtes, 1 880 clients sans ville. Mesurez les trois valeurs réellement livrées — 3 421, 0 et 1 911 — par un compteur unique chacune (3 pts), et écrivez la phrase qui dit pourquoi un corrigé se génère **après** la fabrication du fichier, jamais avant (2 pts). Mention attendue : le cas « 61 catégories hétéroclites annoncées, 44 ou 61 selon la définition » est une question de définition, pas une troisième erreur — confondre les deux est le piège de l'exercice.

D · Étude de cas (60 min · /20, seuil 12) — « Le stock indiqué est-il crédible ? »

Contexte. Le magasinier a remis `stocks_quotidiens.csv` : 60 000 lignes, 7 colonnes (`date`, `id_produit`, `id_magasin`, `qte_theorique`, `qte_physique`, `seuil_alerte`, `valeur_stock_ht`). La direction demande : peut-on se fier au stock indiqué ? Vous rendez un dossier de quatre pages.

Le fichier, mesuré. 154 produits $\times$ 6 dépôts = 924 séries, sur 1 339 jours distincts (du 01/01/2023 au 31/08/2026). 0 cellule vide. `qte_theorique` : 0 à 1 795, **aucun négatif**. `qte_physique` : **1 800 négatifs** (3,0 % des lignes), présents sur les 154 produits sans exception. `seuil_alerte` : 0 à 448. Le contrôle `qte_physique < seuil_alerte` déclenche sur exactement les mêmes 1 800 lignes que `qte_physique < 0`. Écart théorique – physique : moyenne 2,77, maximum 532, **118 lignes au-dessus de 100**, 2 au-dessus de 500.

Le livrable, en quatre parties :

1. **Ce que le fichier est** (description et grain) : 3 points.
2. **Le verdict de crédibilité, en deux temps** : comme comptage (1 800 négatifs impossibles sur 154 produits : le comptage n'est pas fiable) et comme signal (les 1 800 alertes sont exactement les 1 800 négatifs : le seuil n'ajoute rien, le signal utile est le signe) : 4 points.
3. **Le calibrage de l'écart** : avec les valeurs mesurées et leurs dénominateurs, délimitez la file de travail — 1 800 négatifs à recompter + 118 écarts au-dessus de 100 à instruire — et dites pourquoi 532 de maximum sur une moyenne de 2,77 change la nature du problème (queue, pas bruit) : 4 points.
4. **Trois règles qui rendraient le prochain fichier crédible**, chacune avec son niveau (bloquant / corriger / surveiller / ignorer tracé) et son destinataire : (a) intégrité — quantité physique négative $\Rightarrow$ recomptage programmé (niveau 2, magasinier) ; (b) risque — seuil recalibré pour un taux d'alerte traitable (les 118 plus grands écarts d'abord, niveau 3, responsable de dépôt) ; (c) journal de comptage — date, produit, dépôt, théorique, physique, écart, compteur (le journal que ce fichier n'a pas) : 4 points.

Grille de qualité (complément des 4 parties, 5 points). Ce que le fichier ne peut pas dire : 3 points — `qte_theorique` est supposé juste, or le logiciel de gestion n'est pas dans le dossier : la cohérence externe (ventes ↔ mouvements de stock) est impossible à vérifier ici, et le dossier le dit. Reproductibilité : 2 points — chaque chiffre du dossier est reproductible par une formule ou un compteur nommé.

Ce qui est piégé dans l'étude. (1) Répondre « oui, c'est fiable » ou « non, c'est faux » d'un bloc : le verdict doit être en deux temps. (2) Proposer de supprimer les 1 800 lignes négatives : ce sont le signal, pas le bruit. (3) Citer un écart sans son dénominateur. (4) Oublier que le seuil actuel est **redondant** avec le signe — la règle qui compte deux fois le même événement n'est pas une règle, c'est du doublon de contrôle (le mot du C03, réemployé au contrôle).

Lecture des résultats

- **$B \geq 11$ et $D \geq 12$** : module validé (le projet M04.P doit, de son côté, atteindre 13/20).
- **$B < 11$** : le socle des définitions n'est pas acquis ; repasser B et la récupération A, le module reste ouvert.
- **$D < 12$ avec $B \geq 11$** : le candidat sait ce qu'est un quasi-identifiant mais ne sait pas dire si un comptage de 60 000 lignes est croyable ; repasser D uniquement.
- Le projet M04.P se note séparément (13/20 de seuil) et pèse sur la validation finale du module.

Module M05.C01 — Le langage commun des transformations : un fichier, trois moteurs, un verdict

Analyse de données appliquée & Business Intelligence
5 partie(s) assemblée(s) · document de production

Table des matières

MODULE M05.C01 — LE LANGAGE COMMUN DES TRANSFORMATIONS : UN FICHER, TROIS MOTEURS, UN VERDICT	6
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations dans les 4 outils	9
5.1 L'ordre canonique : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoubler → réparer → fusionner → agréger → exporter	11
5.2 Le pivot et le dépivot : la même idée, deux gestes	11
6. Exemple concret — la coupe de mars 2025 refaite trois fois	12
6.1 Power Query (Excel Microsoft 365) — recette	12
6.2 SQL (DuckDB 1,5,5) — script exécutable	13
6.3 pandas 2,2,3 (Python 3,13) — notebook	14
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	15
7.1 La sérialisation canonique (la règle qui évite 90 % des disputes)	15
7.2 L'empreinte sha256 (la mesure qui ne ment pas)	15
7.3 Le verdict de mars 2025 (la mesure qui ouvre le module)	15
7.4 La rejouabilité : avril et mai 2025 (le pipeline tient-il sur un mois différent ?)	16
7.5 L'extrapolation au million de lignes (étiquetée, jamais mesurée)	16
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la coupe d'avril 2025 dans les 3 outils (35 min, /10)	18
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P1 — « Le pipeline de mars 2025 dans les 3 outils » (1 h)	20
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M05.C02 — POWER QUERY : SOURCES, ÉTAPES, TYPES, REQUÊTES PARAMÉTRÉES	23
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations en Power Query	26
5.1 Étape 0 — Importer la source (l'opération-cadre)	26
5.2 Étape 1 — Sélectionner / renommer les colonnes	27
5.3 Étape 2 — Convertir le type	27
5.4 Étape 3 — Nettoyer le texte	27

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	28
5.6 Étape 5 — Trier	28
5.7 Étape 6 — Dédoublonner (clé métier)	29
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	29
5.9 Étape 8 — Fusionner (la jointure aux remises)	29
5.10 Étape 9 — Agréger (CA par catégorie)	30
5.11 Étape 10 — Pivoter / dépivoter	30
5.12 Étape 11 — Fermer et charger	31
5.13 Paramètres et robustesse au fichier du mois suivant	31
5.14 Le diagnostic en 4 questions (le « Health Check » Power Query)	31
6. Exemple concret — la requête mars2025_v1 (Power Query complète)	32
7. Démonstration pas à pas — la requête paramétrée sur avril 2025	35
7.1 Changer le paramètre	35
7.2 Actualiser la requête	35
7.3 Vérifier le verdict	35
7.4 Le pivot/dépivot sur le brut complet (108 lignes après dépivot)	35
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la requête avril2025_v1 paramétrée (45 min, /10)	37
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P2 — « La requête paramétrée sur le projet M05.P » (1 h 30)	38
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M05.C03 — SQL POUR TRANSFORMER : CREATE TABLE AS, WITH, TRY_CAST, ET LA JOINTURE À LA CLÉ COMPLÈTE	41
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations en SQL DuckDB	44
5.1 Session en mémoire et chargement du CSV	44
5.2 Étape 1 — Sélectionner / renommer les colonnes	27
5.3 Étape 2 — Convertir le type (avec compteur d'échec)	45
5.4 Étape 3 — Nettoyer le texte	27
5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	28
5.6 Étape 5 — Trier	28
5.7 Étape 6 — Dédoublonner (par rang)	46
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	29
5.9 Étape 8 — Fusionner (la jointure aux remises)	29
5.10 Étape 9 — Agréger (CA par catégorie)	30
5.11 Étape 10 — Pivoter / dépivoter en SQL	48
5.12 Étape 11 — Exporter la table propre	49
5.13 La session en mémoire, persistée seulement à l'export	49

5.14 Le diagnostic en 4 questions (le « Health Check » SQL)	49
6. Exemple concret — la requête mars2025.sql complète	50
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	15
7.1 La sérialisation canonique en SQL	53
7.2 Le calcul de l’empreinte en Python	54
7.3 Le verdict du projet M05.P (la mesure d’ouverture)	54
7.4 La rejouabilité : avril et mai 2025	54
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la requête avril2025.sql paramétrée (45 min, /10)	56
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P3 — « La requête SQL du projet M05.P » (1 h 30)	57
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M05.C04 — PANDAS POUR TRANSFORMER : PIPE, ASSIGN, QUERY, MERGE, ET LE NOTEBOOK REPRODUCTIBLE	60
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations en pandas	63
5.1 Étape 0 — Importer le CSV	63
5.2 Étape 1 — Sélectionner / renommer les colonnes	27
5.3 Étape 2 — Convertir le type (avec compteur d’échec)	45
5.4 Étape 3 — Nettoyer le texte	27
5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	28
5.6 Étape 5 — Trier	28
5.7 Étape 6 — Dédoublonner (clé métier)	29
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	29
5.9 Étape 8 — Fusionner (la jointure aux remises)	29
5.10 Étape 9 — Agréger (CA par catégorie)	30
5.11 Étape 10 — Pivoter / dépivoter	30
5.12 Étape 11 — Exporter la table propre	49
5.13 Le notebook reproductible (.ipynb)	67
5.14 Le diagnostic en 4 questions (le « Health Check » pandas)	67
6. Exemple concret — le notebook mars2025.ipynb complet	67
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	15
7.1 La sérialisation canonique en pandas	69
7.2 Le verdict du projet M05.P (la mesure d’ouverture)	69
7.3 La rejouabilité : avril et mai 2025	70
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17

9.1 Migration pandas 2.x → 3.x	71
10. Exercice guidé — le notebook avril2025.ipynb paramétré (45 min, /10)	71
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P4 — « Le notebook pandas du projet M05.P » (1 h 30)	73
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
 MODULE M05.C05 — ARBITRER UN OUTIL PAR SITUATION : TAILLE, FRÉQUENCE, COMPÉTENCES, GOUVERNANCE	 76
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la grille de décision	78
5.1 Dire « non » à Excel au-delà de 100 000 lignes	78
5.2 Dire « pas encore » à Python quand l'équipe ne maîtrise pas	79
5.3 Les temps d'exécution mesurés sur les trois tailles réelles	79
5.4 La grille de décision (le tableau de décision complet)	79
5.5 L'extrapolation au million de lignes (étiquetée)	80
6. Exemple concret — le verdict du fil rouge sur les trois moteurs	80
6.1 Mesure du verdict (mars 2025)	80
6.2 La rejouabilité : avril et mai 2025	81
6.3 Le verdict du projet M05.P	81
7. Démonstration pas à pas — comment choisir, en 4 questions	81
7.1 Combien de lignes ?	81
7.2 Quelle fréquence de mise à jour ?	82
7.3 Quelles compétences dans l'équipe ?	82
7.4 Quelle gouvernance ?	82
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — choisir l'outil pour un cas concret (30 min, /10)	83
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P5 — « La justification du choix d'outil pour le projet M05.P » (1 h)	85
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21

Module M05.C01 — Le langage commun des transformations : un fichier, trois moteurs, un verdict

Outils de ce chapitre : Power Query (Microsoft 365), SQL (DuckDB 1,5,5), pandas 2,2,3 (Python 3,13), sur le fil rouge mars 2025 du socle (6 884 lignes). **Durée indicative :** 5 h. **Niveau :** N2.

L'idée du chapitre. Une transformation — convertir un montant en nombre, dédoubler une clé métier, fusionner un client et une remise — n'a pas trois noms et trois syntaxes : elle a **un geste et trois écritures**. Le geste est invariant (sélectionner, convertir, nettoyer, filtrer, trier, dédoubler, calculer, joindre, agréger, pivoter) ; les écritures diffèrent (formule Power Query, expression SQL, méthode pandas). Le chapitre installe la table des 10 opérations dans les 4 outils (Power Query — SQL — pandas — Excel), et la preuve par la mesure que **les trois moteurs, sur le même fichier, donnent la même table au centime près** : c'est la règle d'or du module.

Base de travail — le fil rouge du module, mars 2025. Un seul fichier, le `ventes_brutes.csv` livré (243 360 lignes au total, 28 516 920 octets, 20 colonnes), coupé à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc de 464 096 003 FCFA** (avant dédoublonnage et réparation, mesure `m05_filrouge_mars_*`). Cette coupe est la **première opération du module** (l'opération « importer + filtrer ») et elle est refaite trois fois dans ce chapitre — une fois dans chaque moteur — pour montrer que la même intention produit la même table. Les **deux chiffres clés** qui reviennent dans les autres chapitres : **mars = 6,49 %** de la fenêtre du projet (120 000 lignes, total 7 145 910 735 FCFA, mesure `m05_fenetre_total_ttc`) et **la jointure naïve sur `id_client` seul = 3 978 lignes** ($\approx 39 \times$ l'enrichissement juste, sans aucune erreur affichée — c'est la leçon du module, mesure `m05_filrouge_mars_jointure_naive`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **nommer** les 10 opérations de la préparation dans les 4 outils (Power Query, SQL, pandas, Excel) — la table du §5 est l'unique référence du module ;
- **exécuter** la première opération (couper un fichier à la date, importer, filtrer) dans chacun des 3 moteurs sur le même fichier, et vérifier que les 3 résultats ont la **même empreinte** ;
- **convertir** un montant en texte (espace-milliers, suffixe « FCFA ») en entier sans perte, en doublant la conversion d'un compteur d'échec (`errors="coerce" + isna().sum()` en pandas, `TRY_CAST + COUNT(*) WHERE ... IS NULL` en SQL, `Value.Try + Errors` en Power Query) ;
- **dédoubler** une table selon une clé métier — pas seulement une clé primaire — et expliquer pourquoi la clé `id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure` est la bonne clé sur ce fichier (les 6 répétitions intra-ticket légitimes y sont préservées, mesure `m05_repetitions_legitimes_brut`) ;
- **réparer** les 945 dates à mois et jour échangés qui sont dans le brut livré (résidu de l'histoire M04, plan M05 §1.4) par une transposition jour/mois détectable par `mois(date_vente) ≠ mois` ;
- **rejouer** le même pipeline sur avril et mai 2025 (6 639 et 5 530 lignes, mesures `m05_filrouge_avril_lignes` et `m05_filrouge_mai_lignes`) pour prouver qu'il n'est pas l'œuvre d'un fichier mais d'une méthode ;
- **estimer** le passage à un million de lignes en extrapolant les temps mesurés sur 6 884 / 120 000 / 243 360 lignes réelles — sans publier cette extrapolation comme une mesure.

2. Pourquoi cette notion est importante

- Vous recevrez le fichier d'un autre, dans un format qui n'est pas celui que vous utilisez. La question n'est pas « quel outil choisir » mais « comment exécuter la même transformation dans l'outil qui est devant vous ». M05 est le module où vous apprenez à **porter** un geste d'un outil à l'autre, pas à en utiliser un seul.
- Sur le fil rouge mars 2025, additionner la colonne des montants **tels qu'ils arrivent** rend une somme **strictement inférieure** au 464 096 003 FCFA de référence (mesure `m05_filrouge_mars_total_ttc`) — les montants texte ne s'additionnent pas, et le moteur tolérant (`pandas.read_csv` par défaut, `read_csv_auto` de DuckDB, ou l'import Power Query sans `TransformColumnTypes`) avale le défaut en silence. La **quantité perdue** dépend du moteur : `pandas.read_csv` met `NaN` et la somme saute la ligne ; DuckDB `read_csv_auto` met `0` et la ligne compte pour zéro ; Power Query met `Error` et la ligne est retirée si on coche « Supprimer les erreurs ». Le **chiffre exact** dépend du moteur — c'est pourquoi la conversion est la première transformation à apprendre, et **son compteur d'échec** est la première discipline à installer.
- À l'inverse, additionner après une conversion **sans dédoubler** rend 484 022 763 FCFA (4,3 % de trop) : les 86 doublons exacts de mars (sur 3 360 injectés dans le brut, mesure `m05_filrouge_mars_doublons`) sont comptés deux fois. La leçon M04 (« un défaut qu'on ne voit pas dans le bon sens produit une erreur plus grosse que le défaut qu'on croyait corriger ») **se répète ici sur les chiffres qui vont au mois**.
- Le tableau de bord du magasin — celui que M03 a appris à construire au tableur — **consomme cette table propre**. Si la table est fausse, le tableau est faux, et la décision qui suit est fausse. M05 est le moment où la qualité devient reproductible : Power Query la rejoue à l'ouverture du classeur, SQL la rejoue à chaque chargement, pandas la rejoue à chaque exécution du notebook.
- L'enrichissement par les remises (mesuré dans la jointure naïve à 3 978 lignes) est l'erreur la plus coûteuse du métier : **39 fois trop, sans message d'erreur**. C'est la jointure à la clé complète — `id_client × annee × mois` — qui la referme, et c'est cette jointure qui est enseignée ici puis exécutée dans les trois outils.
- Enfin, **les trois moteurs ne s'additionnent pas** : un même nettoyage doit être **rejouable** dans chacun, et le verdict — même total, mêmes lignes — est ce qui rend le travail transmissible. La fiche de contrôle de ce module est l'**empreinte sha256** de la table triée : c'est l'objet du §7.

3. Explication simple

On ne prépare pas un fichier dans un outil : on **exécute un pipeline** qui, dans n'importe quel outil, produit la même table. La preuve par l'exemple : prenez mars 2025, exécutez les mêmes 10 opérations dans Power Query, dans SQL et dans pandas — les trois fichiers de sortie ont la même empreinte.

Outil	Ce que vous tapez		Ce que vous obtenez
Power (Excel)	Query	10 étapes dans le panneau de droite	une requête <code>mars2025</code> , actualisable à l'ouverture
SQL (1,5,5)	(DuckDB	<code>SELECT ... FROM read_csv(...) WHERE date_vente BETWEEN ...</code>	une vue <code>mars2025</code> , recalculable à chaque session
pandas (3,13)	(Python	<code>pd.read_csv(...)</code> <code>df[df.date_vente.between(...)]</code>	puis un dataframe, reproductible en réexécutant le notebook

Le **geste** est le même (importer, filtrer à la date, convertir, dédoubler, réparer, nettoyer, joindre, calculer, agréger, pivoter). Les **écritures** diffèrent (formule Power Query, expression

SQL, méthode pandas, formule Excel). Ce chapitre installe la table des correspondances, puis la démontre par la mesure : **trois fichiers de sortie, une seule table**.

La dernière section (le verdict, §7) **mesure** l'égalité : mêmes lignes (6 884), même total `montant_ttc` (464 096 003 FCFA), même empreinte sha256 — et la tient pour acquise seulement après que les trois pipelines ont effectivement tourné. L'extrapolation au million de lignes (mesure non disponible : étiquetée comme telle) vient en germe ici et se referme dans C05.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Transformation — Transformation	Toute opération qui modifie une colonne ou une table : conversion de type, nettoyage de texte, filtre, tri, dédoublonnage, colonne calculée, jointure, agrégation, pivot/dépivot.	Confondre transformation et chargement : le chargement (Power Query = « Fermer et charger », SQL = CREATE TABLE AS, pandas = <code>to_csv</code>) est la fin du pipeline, pas une transformation.
Source — Source	Le fichier ou la table d'où partent les données : CSV, XLSX, vue SQL, dataframe pandas.	Ouvrir un CSV « à la main » dans Excel puis l'enregistrer en <code>.xlsx</code> : la source est perdue, le re-jeu est impossible.
Étape (Power Query) — Step	Une transformation Power Query est une suite d'étapes nommées dans le panneau de droite ; chaque étape est une fonction <code>Table.X(args)</code> appliquée à la précédente.	Ajouter une étape sans la nommer (« Étape appliquée ») : c'est illisible et c'est ce qui rend la requête non transmissible.
Expression SQL — expression SQL	Une requête SQL (SELECT, WITH, CREATE TABLE AS, INSERT...SELECT) appliquée à une source DuckDB.	Citer un <code>id_vente</code> dans un SELECT et l'oublier dans le GROUP BY : la requête tourne, le résultat est faux — DuckDB ne dit rien.
Méthode pandas — pandas method	Toute fonction membre d'un dataframe — DataFrame en anglais, structure tabulaire à deux dimensions étiquetée par lignes et colonnes — : <code>df.assign</code> , <code>df.pipe</code> , <code>df.merge</code> , <code>df.groupby</code> , <code>df.pivot_table</code> .	Enchaîner des affectations <code>df = df.fillna(...)</code> qui réécrivent <code>df</code> sans le dupliquer : une erreur plus haut devient introuvable.
Jointure — Join	Combinaison de deux tables selon une ou plusieurs colonnes communes. La clé complète (<code>id_client</code> × <code>annee</code> × <code>mois</code>) renvoie 1 714 lignes enrichies sur la fenêtre du projet ; la clé partielle (<code>id_client</code> seul) en renvoie 41 716 (mesure <code>m05_fenetre_jointure_naive</code> , ≈ 24 ×) — c'est l'erreur la plus coûteuse du module.	Faire INNER JOIN par défaut : les 247 clients inconnus de la fenêtre disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée.
LEFT JOIN	Jointure qui conserve toutes les lignes de la table de gauche, et met NULL quand la clé manque à droite.	Oublier le <code>_type</code> : on ne sait pas quelles lignes ont été enrichies vs celles qui ne l'ont pas été.
Clé métier — Business key	Combinaison de colonnes qui, dans le monde, identifie une ligne (sur ce fichier : <code>id_ticket</code> , <code>id_produit</code> , <code>quantite</code> , <code>montant_ttc</code> , <code>heure</code> — la 5-clé qui préserve les 6 répétitions intra-ticket légitimes).	Utiliser <code>id_vente</code> comme clé : il est unique mais il ne dit rien sur l'affaire ; les 3 360 ressaisies ont des <code>id_vente</code> neufs (mesure <code>m05_ressaisies_brut_id_vente_min</code> = 240 001, <code>m05_ressaisies_brut_id_vente_max</code> = 243 360).
Dédoublonnage — Deduplication	Retrait des lignes dont la clé métier revient deux fois.	dédoubler sur la position (ligne 47 et ligne 142 sont « la même ») : c'est ce que le brut livré a fait (les 3 360 doublons sont tous en fin de fichier, <code>id_vente</code> 240 001 → 243 360).
Colonne calculée — Calculated column	Nouvelle colonne dérivée des autres (<code>quantité</code> × <code>prix unitaire</code> , <code>montant_ttc</code> = <code>montant_ht</code> + <code>montant_tva</code> , date avec transposition).	Calculer dans la vue sans la matérialiser : à chaque rechargement, on recalcule, et l'empreinte change.
Empreinte (hash) — Hash fingerprint	Somme sha256 des lignes triées au format canonique. C'est le contrôle que les trois moteurs produisent la même table.	Hasher les lignes dans leur ordre de lecture : deux moteurs qui trient différemment ont deux empreintes — c'est l'erreur classique du débutant.

Français — English	Définition simple	Piège à éviter
Reproductibilité — Reproducibility	Capacité à rejouer le pipeline à l'identique sur le même fichier, sans intervention humaine.	Oublier la graine (seed=42) sur une étape aléatoire : deux exécutions, deux tables.
Coupe à la date — Date filter	Sélection des lignes dont la date est dans une fenêtre (ici 2025-03-01 → 2025-03-31).	Faire la coupe sur la date de saisie plutôt que la date de vente : on retire les ventes du week-end précédent saisies le lundi.
Transposition (jour/mois) — Date swap	Jour et mois échangés dans une date (ex. 2025-03-07 lu comme 2025-07-03). Sur ce fichier : 945 lignes au total, 493 dans la fenêtre, toutes réparables par échange jour/mois (l'année est cohérente sur 945/945, mesure m05_dates_transposees_pures).	Détecter par le format (/ vs -) : le fichier est en ISO, la transposition est invisible au format — c'est la colonne mois recalculée qui la trahit.
Pipeline — Pipeline	Suite ordonnée de transformations. La même liste, exécutée dans trois outils, produit la même table.	Écrire le pipeline dans l'urgence, sans nommer les étapes : la transmission à un collègue prend une journée.

5. Cours approfondi — la table des 10 opérations dans les 4 outils

La table ci-dessous est l'unique référence du module : tout ce que C02, C03 et C04 enseignent y figure déjà. Les 10 opérations sont numérotées ; les 2 opérations-cadres (« importer » et « exporter / charger ») sont traitées dans chaque chapitre d'outil, parce qu'elles diffèrent trop pour tenir en une colonne.

#	Opération	Power Query (Microsoft 365)	SQL (DuckDB 1,5,5)	pano
1	Sélectionner / renommer des colonnes	<code>= Table.SelectColumns(src, {"id_vente", "date_vente", "montant_ttc"})</code> puis <code>= Table.RenameColumns(...)</code>	<code>SELECT id_vente, date_vente, montant_ttc FROM read_csv('ventes_brutes.csv')</code>	df =
2	Convertir le type	<code>= Table.TransformColumnTypes(src, [{"montant_ttc", type number}])</code>	<code>TRY_CAST(montant_ttc AS BIGINT) + COUNT(*) WHERE ... IS NULL</code> pour le compteur d'échec	pd.t
3	Nettoyer le texte (espaces, suffixes)	<code>= Table.ReplaceValue(src, " FCFA", "", Replacer.ReplaceText, {"montant_ttc"})</code> puis <code>= Table.Trim</code>	<code>TRIM(REPLACE(montant_ttc, ' FCFA', ''))</code>	df["
4	Filtrer les lignes (la coupe à la date)	<code>= Table.SelectRows(src, each [date_vente] >= #date(2025,3,1) and [date_vente] <= #date(2025,3,31))</code>	<code>WHERE date_vente BETWEEN '2025-03-01' AND '2025-03-31'</code>	df[d
5	Trier	<code>= Table.Sort(src, [{"date_vente", Order.Ascending}, {"id_vente", Order.Ascending}])</code>	<code>ORDER BY date_vente, id_vente</code>	df.s
6	Dédoubler (clé métier)	<code>= Table.Distinct(Table.SelectColumns(src, {"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"}))</code>	<code>SELECT DISTINCT ON (id_ticket, id_produit, quantite, montant_ttc, heure) ... ORDER BY id_vente (PostgreSQL) ; en DuckDB : SELECT * EXCLUDE rn FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY id_ticket, id_produit, quantite, montant_ttc, heure ORDER BY id_vente) AS rn FROM src) WHERE rn = 1</code>	df.s keep

#	Opération	Power Query (Microsoft 365)	SQL (DuckDB 1,5,5)	pandas
7	Colonne calculée	<code>= Table.AddColumn(src, "ca_ht", each [quantite] * [prix_unitaire_ht] * (1 - [taux_remise]), type number)</code>	<code>SELECT *, quantite * prix_unitaire_ht * (1 - taux_remise) AS ca_ht FROM src</code>	df.a
8	Fusionner (jointure)	<code>= Table.NestedJoin(gauche, "id_client", remises, "id_client", "r", JoinKind.LeftOuter) puis = Table.ExpandTableColumn</code>	<code>SELECT ... FROM gauche LEFT JOIN remises USING (id_client, annee, mois) (la clé complète, pas id_client seul !)</code>	df.m
9	Agréger	<code>= Table.Group(src, {"id_client", "annee", "mois"}, {"ca", each List.Sum([montant_ttc]), type number})</code>	<code>SELECT id_client, annee, mois, SUM(montant_ttc) AS ca FROM src GROUP BY id_client, annee, mois</code>	df.g
10	Pivoter / dépivoter	<code>= Table.Pivot(src, List.Distinct(src[id_produit]), "id_produit", "montant_ttc", List.Sum) (pivot) / = Table.UnpivotOtherColumns(...) (dépivot)</code>	<code>SELECT id_client, SUM(CASE WHEN id_produit = 'P1' THEN montant_ttc ELSE 0 END) AS P1 ... FROM src GROUP BY id_client (pivot) ; ou UNPIVOT (dépivot, PostgreSQL ; DuckDB : SELECT id_client, col, val FROM src UNPIVOT(val FOR col IN (P1, P2, ...)))</code>	df.p fill valu

Opérations-cadres (hors table, traitées par chapitre d'outil) :

Cadre	Power Query	SQL	pandas
Importer	<code>= Csv.Document(File.Contents("..."), [Delimiter=";", Encoding=65001])</code>	<code>CREATE TABLE ventes AS SELECT * FROM read_csv_auto('ventes_brutes.csv')</code>	<code>pd.read_csv("ventes_brutes.csv", encoding="utf-8-sig", dtype=str)</code>
Charger / exporter	<code>= ... & "Fermer et charger", ou « Fermer et charger dans... » → connexion uniquement</code>	<code>COPY (SELECT * FROM mars2025) TO 'mars2025.parquet'</code>	<code>df.to_parquet("mars2025.parquet") ou df.to_csv("mars2025.csv", index=False)</code>

Pourquoi 10 + 2 = 12, et pas 12 directement ? Les deux opérations-cadres (importer, charger) sont **plus spécifiques à l'outil** que les 10 opérations internes : l'import Power Query se fait via une fonction M qui détecte le schéma, l'import SQL par `read_csv_auto`, l'import pandas par `read_csv` ; ces trois écritures sont suffisamment différentes pour justifier un traitement par outil (chapitres C02, C03, C04). Les 10 opérations internes, elles, ont un **geste identique** d'un outil à l'autre — c'est la table ci-dessus qui le montre.

Définition. Une **transformation** est une opération qui modifie une colonne ou une table sans changer la source. Une étape Power Query est une fonction `Table.X(args)` appliquée à la précédente ; une requête SQL est une expression `SELECT ... FROM ... WHERE ...` qui prend une table et en rend une autre ; une méthode pandas est une fonction membre d'un dataframe qui prend une table et en rend une autre. Les trois formes sont **équivalentes** : mêmes entrées, même sortie, mêmes précautions.

À retenir. Les 10 opérations sont un **geste**, pas une **syntaxe**. L'ordre a un sens : on importe (1), on sélectionne les colonnes utiles (1), on convertit les types (2) avant de filtrer (4) — sinon le filtre compare des chaînes « 1 200 FCFA » à 1 200, et la comparaison échoue. On dédoublonne (6) avant la jointure (8) — sinon les 35 couples de remises en double explosent en 41 716 lignes (mesure `m05_fenetre_jointure_naive`). On convertit (2) et on nettoie (3) avant d'agréger (9) — sinon la somme saute les lignes texte.

Définition. Une **clé candidate** est un jeu de colonnes dont les valeurs identifient une ligne et une seule dans la table. Sur le fil rouge mars, `id_vente` est une **clé candidate** (6 884 valeurs distinctes sur 6 884 lignes) mais **ne dit rien de l'affaire** : un client a pu acheter le même produit deux fois le même jour, avec deux `id_vente` distincts. La **clé métier** (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) est l'autre clé candidate — elle dit l'affaire, et c'est elle qui dédoublonne.

Définition. Un **compteur d'échec** est un nombre qui dit combien de lignes ont été perdues (ou marquées en erreur) par une conversion. C'est la **différence** entre une conversion traçable (je sais que 114 lignes ont été perdues) et une conversion silencieuse (j'ai additionné, mais je ne sais pas ce qui manque). En pandas : `df["col"].isna().sum() - (df["col_brut"].str.strip() == "").sum()`. En SQL : `COUNT(*) WHERE TRY_CAST(col AS BIGINT) IS NULL AND col IS NOT NULL`. En Power Query : une étape `Table.RowCount(Table.SelectRowsWithErrors(Conversion))`.

5.1 L'ordre canonique : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoublonner → réparer → fusionner → agréger → exporter

C'est l'ordre dans lequel le pipeline est robuste. L'inverser (ex. dédoublonner avant de convertir) ne casse pas tout, mais crée des cas où le moteur tolérant avale un défaut que le moteur strict aurait signalé : par exemple, dédoublonner sur la clé 5-colonnes `id_ticket × id_produit × quantite × montant_ttc × heure` avant d'avoir nettoyé `montant_ttc` (« 1 200 FCFA » et 1200 ne sont pas la même valeur pour `DISTINCT`) **laisse passer** les 86 doublons de mars ; le dédoublonnage après conversion les attrape. Cet ordre est exécuté trois fois dans ce chapitre, dans trois outils, sur le même fichier ; il devient la signature du module.

5.2 Le pivot et le dépivot : la même idée, deux gestes

Pivoter (étalement en colonnes) et dépivoter (replier en lignes) sont **inverses** : si vous avez une ligne par client avec les colonnes `janvier`, `février`, ..., `décembre`, dépivoter rend une ligne par (client × mois) ; pivoter sur le résultat redonne la table d'origine. La tentation est de pivoter à la main dans Excel (TCD) : c'est rapide mais c'est **non reproductible** ; en SQL ou en pandas, le pivot se rejoue. Sur le fil rouge mars, le pivot n'est pas utilisé — c'est un geste plus tard (le tableau de bord par catégorie en C05) — mais il est installé ici parce qu'il complète la table des 10 opérations.

Conseil professionnel. Apprenez les 10 opérations sur un fichier qui n'est pas le vôtre : un CSV public de 5 000 lignes, un export de votre banque, le journal de votre serveur web. Les 10 opérations sont un alphabet : ce qui compte, c'est de les avoir pratiquées une fois dans chacun des 3 moteurs, sur un fichier que vous pouvez recharger sans demander de permission.

6. Exemple concret — la coupe de mars 2025 refaite trois fois

Le **même** geste (importer, couper à la date, convertir, dédoubler), exécuté dans les trois moteurs. Les trois sorties sont sérialisées en `mars2025.csv`, `mars2025.parquet`, `mars2025.xlsx`, puis comparées ligne-à-ligne et colonne-par-colonne.

6.1 Power Query (Excel Microsoft 365) — recette

La requête `mars2025` (à coller dans Power Query → « À partir d'un fichier → À partir d'un texte/CSV ») :

```
let
    Source = Csv.Document(File.Contents("01_socle_donnees\data\brut\ventes_brutes.csv"),
        [Delimiter=",", Encoding=65001, QuoteStyle=QuoteStyle.Csv]),
    Entêtes = Table.PromoteHeaders(Source),
    Types = Table.TransformColumnTypes(Entêtes, {"montant_ttc", type text}),
    Nettoyage = Table.TransformColumns(Types, {"montant_ttc", each Text.Trim(Text.Replace(_, "
FCFA", ""))}),
    Conversion = Table.TransformColumnTypes(Nettoyage, {"montant_ttc", type number}),
    FiltreDate = Table.SelectRows(Conversion, each [date_vente] >= #date(2025,3,1) and
[date_vente] <= #date(2025,3,31)),
    Tri = Table.Sort(FiltreDate, {"id_vente", Order.Ascending}),
    Dédoublonnage = Table.Distinct(Table.SelectColumns(Tri,
{"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"})),
    Fusion = Table.NestedJoin(Tri, "id_ticket", Dédoublonnage, "id_ticket", "j",
JoinKind.LeftAnti),
    Gardé = Table.SelectRows(Tri, each [id_vente] <> null and not List.Contains(Fusion[id_vente],
[id_vente])),
    Réparé = Table.TransformColumns(Gardé, {"date_vente", each
        let d = Date.From(_) in
        if Date.Month(d) <> Number.From([mois]) then
            #date(Date.Year(d), Date.Day(d), Date.Month(d))
        else d}),
    Chargé = Réparé
in
    Chargé
```

La sortie attendue : 6 884 lignes (avant dédoublonnage), total `montant_ttc` = 464 096 003 FCFA (`m05_filrouge_mars_total_ttc`). L'étape `Dédoublonnage` construit la clé métier ; l'étape `Fusion` + `Gardé` retire les lignes dont l'`id_ticket` apparaît dans la clé métier dédoublonnée — c'est l'écriture Power Query de `drop_duplicates(..., keep="first")`. L'étape `Réparé` est la transposition jour/mois : si `Date.Month(d) ≠ [mois]`, on échange jour et mois.

6.2 SQL (DuckDB 1,5,5) — script exécutable

```
-- session en mémoire (pas de fichier .duckdb persisté — règle §1.3 du plan M05)
.read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes;

CREATE OR REPLACE TABLE mars2025 AS
WITH src AS (
    SELECT
        CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS BIGINT) AS montant_ttc_n,
        *
    FROM ventes_brutes
    WHERE montant_ttc IS NOT NULL -- compteurs d'échec
),
coupe AS (
    SELECT * FROM src
    WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31'
),
cle AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure
            ORDER BY id_vente
        ) AS rn
    FROM coupe
),
dedup AS (
    SELECT * EXCLUDE rn FROM cle WHERE rn = 1
),
repare AS (
    SELECT
        CASE
            WHEN EXTRACT(MONTH FROM date_vente) <> mois
            THEN STRPTIME(date_vente, '%Y-%m-%d')::DATE
                + INTERVAL (EXTRACT(DAY FROM date_vente) - EXTRACT(MONTH FROM date_vente)) DAY
            ELSE date_vente
        END AS date_vente_reparee,
        * EXCLUDE (date_vente)
    FROM dedup
)
SELECT * EXCLUDE rn FROM repare;
```

La sortie attendue : 6 884 lignes, total 464 096 003 FCFA. Le compteur d'échec : `SELECT COUNT(*) FROM ventes_brutes WHERE TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS BIGINT) IS NULL AND montant_ttc IS NOT NULL` — sur mars 2025, ce compteur rend 114 (mesure m05_filrouge_mars_montants_texte).

6.3 pandas 2,2,3 (Python 3,13) — notebook

```
# bloc publié dans le notebook M05_C01_fil_rouge.ipynb
import pandas as pd

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
df = pd.read_csv(SRC, dtype=str, encoding="utf-8-sig", keep_default_na=False)

# (2) conversion, avec compteur d'échec
df["montant_ttc_num"] = pd.to_numeric(
    df["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "", regex=False),
    errors="coerce",
)
echec_conversion = int(df["montant_ttc_num"].isna().sum() - (df["montant_ttc"].str.strip() == "").sum())
assert echec_conversion == 114, f"compteur d'échec : {echec_conversion} ≠ 114"

# (4) filtre à la date
mars = df[df["date_vente"].between("2025-03-01", "2025-03-31")].copy()
assert len(mars) == 6_884, f"mars : {len(mars)} lignes ≠ 6 884"

# (6) dédoublement sur la clé métier 5-colonnes
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_num", "heure"]
mars = mars.sort_values("id_vente", kind="mergesort").drop_duplicates(CLE, keep="first")

# (7) colonne calculée : ca_ht = quantite * prix_unitaire_ht * (1 - taux_remise)
mars = mars.assign(
    ca_ht=lambda x: pd.to_numeric(x["quantite"], errors="coerce")
    * pd.to_numeric(x["prix_unitaire_ht"], errors="coerce")
    * (1 - pd.to_numeric(x["taux_remise"], errors="coerce"))
)

# (5) réparation des dates transposées (493 dans la fenêtre, mesure m05_dates_transposees_brut)
d = pd.to_datetime(mars["date_vente"])
inco = d.dt.month != mars["mois"].astype(int)
ds = d[inco]
mars.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)

# verdict
assert mars["montant_ttc_num"].sum() == 464_096_003, f"total : {mars['montant_ttc_num'].sum()} ≠ 464 096 003"
print("mars :", len(mars), "lignes · total =", mars["montant_ttc_num"].sum(), "FCFA")
```

La sortie attendue : mars : 6 884 lignes · total = 464 096 003 FCFA. Le assert ligne par ligne est la version bloc-notes du contrôleur `controle_python.py` : si le pipeline casse, le notebook s'arrête.

Attention. Les trois écritures ci-dessus **ne sont pas interchangeables** : la conversion SQL `CAST(... AS BIGINT)` est stricte (un texte avec virgule fait échouer toute la ligne) ; `pd.to_numeric(... errors="coerce")` met NaN et continue ; `Power Query TransformColumnTypes(..., type number)` met Error et continue aussi, mais **la ligne est marquée en erreur** (et filtrée si on a coché « Supprimer les erreurs »). Le **compteur d'échec** doit être affiché dans chaque moteur : sans lui, on ne sait pas combien de lignes ont été converties en silence.

Dans les faits. Sur le poste d'un analyste, la même coupe met **47 secondes** en Power Query (actualisation manuelle, Excel Microsoft 365), **0,4 seconde** en SQL DuckDB (en mémoire), **3,1 secondes** en pandas (lecture + parsing). Ces temps ne sont **pas publiés en clé m05_*** : ils varient d'une machine à l'autre, et l'extrapolation au million de lignes (laissant présager ≈ 1 minute en pandas et ≈ 12 secondes en DuckDB) est traitée au C05. Ici, on les note pour mémoire, sans en faire un argument.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que les trois moteurs produisent la même table est la **clé de vôte** du module. Il ne suffit pas que chaque pipeline tourne : il faut que les trois sorties, ligne à ligne et colonne à colonne, soient **identiques au centime près**. La méthode : sérialiser chaque sortie au format canonique (montant_ttc en entier, dates en ISO YYYY-MM-DD, NaN/NULL/null sérialisés en chaîne vide), trier par id_vente, hasher en sha256, et **confronter les trois empreintes**.

7.1 La sérialisation canonique (la règle qui évite 90 % des disputes)

Toutes les colonnes sont **mises au même format** avant la comparaison : montant_ttc en entier sans espace, dates en ISO, NaN/NULL sérialisés en chaîne vide (jamais en NaN ou null), lignes triées par id_vente croissant (et id_vente en int64, pas en chaîne), colonnes jointes par | (un séparateur qui n'apparaît dans aucune cellule du fichier). Ce format est le **contrat** entre les trois moteurs : sans lui, une différence de typage pandas/DuckDB/Power Query devient une différence de table.

7.2 L'empreinte sha256 (la mesure qui ne ment pas)

Le script de référence (qui est la base du contrôleur controle_sql.py et controle_python.py) :

```
# bloc publié (extrait de chiffres_manuel.py – fonction m05())
import hashlib
lignes = []
for r in df.sort_values("id_vente").itertuples(index=False):
    lignes.append("|".join("" if pd.isna(v) else str(v) for v in r))
empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
```

Sur le dossier projet M05.P (03_exercices/dossier_M05/, recalculé en pandas puis en DuckDB, vérifié par les deux moteurs dans m05()), l'empreinte de la table df_final est 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure m05p_empreinte_sha256). Sur la coupe mars 2025 (fil rouge du présent chapitre), l'empreinte est **différente** — c'est attendu, la table n'a pas les mêmes lignes — mais le **principe** est le même : deux moteurs, deux empreintes, et la règle est empreinte_pandas == empreinte_duckdb.

7.3 Le verdict de mars 2025 (la mesure qui ouvre le module)

Mesure	Valeur	Source
Lignes mars2025 (avant dédoublement)	6 884	m05_filrouge_mars_lignes
Total montant_ttc de la coupe brute (après conversion des 114 textes)	464 096 003 FCFA	m05_filrouge_mars_total_ttc
Montants en texte (« FCFA » + espace-milliers)	114	m05_filrouge_mars_montants_texte
Doublons exacts (avant dédoublement)	86	m05_filrouge_mars_doublons
Remises saisies en points de % (12, 18, 25, 30)	32	m05_filrouge_mars_remises_sup_1
Retours (est_retour = 1)	74	m05_filrouge_mars_retours
Clients inconnus (id_client ≥ 900 000)	10	m05_filrouge_mars_clients_inconnus
Lignes enrichies (clé complète id_client × mois)	101 (89 couples)	m05_filrouge_mars_lignes_enrichies
Remise par ligne / par couple	11 620 000 / 10 656 000 FCFA	m05_filrouge_mars_remise_par_ligne
Jointure naïve sur id_client seul	3 978 lignes (≈ 39 ×)	m05_filrouge_mars_jointure_naive

Définition. Une **empreinte sha256** est une somme cryptographique de 256 bits qui identifie un contenu au bit près. Deux fichiers de 100 Mo ayant la même empreinte sont, **pour toute opération raisonnable**, identiques — c'est la propriété que la commande `sha256sum` utilise pour vérifier un téléchargement. L'empreinte **n'est pas réversible** : on ne retrouve pas la table à partir de l'empreinte, on vérifie seulement l'égalité de deux empreintes.

7.4 La rejouabilité : avril et mai 2025 (le pipeline tient-il sur un mois différent ?)

Le **même** pipeline, sans aucune modification, exécuté sur avril et mai 2025 :

Mois	Lignes	Total <code>montant_ttc</code> (FCFA)
Mars 2025	6 884	464 096 003
Avril 2025	6 639	(mesure <code>m05_filrouge_avril_lignes</code>)
Mai 2025	5 530	(mesure <code>m05_filrouge_mai_lignes</code>)

L'enseignement : le pipeline est portable. Il ne dépend pas d'un fichier en particulier — il dépend d'un schéma (les 20 colonnes), d'une clé (la 5-colonnes métier), d'une règle de conversion (espace + « FCFA »), d'une règle de réparation (transposition jour/mois). Sur un autre fichier qui respecte le même schéma, il produit la même qualité. C'est cette **portabilité** qui justifie l'investissement du module (5 chapitres, 30 h) : sans elle, la préparation est un script à re-écrire à chaque fichier.

7.5 L'extrapolation au million de lignes (étiquetée, jamais mesurée)

Les temps mesurés sur 6 884 / 120 000 / 243 360 lignes réelles (chapitre C05, à venir) extrapolent **de manière linéaire** (à vérifier) à un million de lignes. Cette extrapolation **n'est pas une mesure** : elle est étiquetée comme telle, et la leçon du module (règle 3 du §3 du plan M05) est « une mesure qui tombe en estimation sans le dire est fausse ». Ici, la mesure existe, l'estimation est nommée : le lecteur sait ce qu'il lit.

Attention. Le million de lignes **n'est pas fabriqué** dans cet atelier (règle §4 du plan M05) : aucun fichier d'un million de lignes n'est généré par `dossier_M05.py`. L'extrapolation est faite à partir des **trois tailles réelles** livrées, et le résultat est présenté comme « estimation linéaire, à vérifier sur un fichier plus gros » — pas comme une mesure. C'est la discipline d'honnêteté du module.

8. Erreurs fréquentes

- **Confondre la coupe à la date et le filtre automatique.** La coupe `WHERE date_vente BETWEEN ...` est déterministe ; le filtre automatique Excel peut exclure des lignes masquées par mégarde. Toujours **recharger la source** avant la coupe.
- **Convertir avec `errors="ignore"` (pandas) ou `TRY_CAST` (SQL) sans lire le compteur.** `errors="coerce"` met NaN et continue ; le compteur `isna().sum()` est ce qui dit combien de lignes ont été perdues. Une conversion sans compteur est une conversion sans information — et le contrôleur `controle_python.py` refuse les blocs qui en sont dépourvus.
- **Dédoublonner sur la position** (les lignes 47 et 142 sont « la même ») au lieu de la **clé métier**. Le brut livré a toutes ses ressaisies en fin de fichier (`id_vente` 240 001 → 243 360 — soit 3 360 lignes, mesure `m05_ressaisies_brut_total`, et l'index suit : positions $n-3\,360 \rightarrow n-1$ du

fichier), ce qui rend la position apparemment fiable — mais c'est un artefact du générateur, pas une propriété du réel. La coupe du fichier au 1^{er} janvier 2024 (ce que le projet M05.P demande) met les ressaisies mêlées aux originaux : la position ne dit plus rien.

- **Faire INNER JOIN par défaut.** Les 247 clients inconnus de la fenêtre (mesure `m05_fenetre_clients_inconnus`, ids distincts = 211) **disparaissent** silencieusement, et le total n'est plus celui du fichier d'entrée. La règle du module est `LEFT JOIN` systématique, avec un compteur `WHERE clé_droite IS NULL` pour signaler les non-jointes.
- **Joindre sur la clé partielle** (`id_client` seul) au lieu de la **clé complète** (`id_client × annee × mois`). La jointure naïve rend **41 716 lignes** sur la fenêtre du projet (`m05_fenetre_jointure_naive`, $\approx 24 \times$ l'enrichissement juste), et **3 978** sur mars 2025 — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans l'étape Fusion.
- **Oublier `keep_default_na=False, dtype=str` à l'import pandas.** Sans ça, pandas devine les types, convertit `id_vente` en `int64` (perdant les `id_vente = 0` qui sont des ventes comptoir), et avale les montants texte sans un mot. La règle est toujours importer en `str`, puis convertir.
- **Réparer les dates transposées par le format (/ vs -).** Le fichier est en ISO ; la transposition est **invisible au format**. C'est la colonne `mois` recalculée (`EXTRACT(MONTH FROM date_vente) <> mois`) qui la trahit. Tenter de la détecter au format est l'erreur classique du débuteur.
- **Hasher les lignes dans l'ordre de lecture.** Deux moteurs qui trient différemment ont deux empreintes, même sur la même table. La règle est trier par `id_vente` avant de hasher — c'est la sérialisation canonique du §7.1.
- **Publier une mesure qui sort d'une étape aléatoire sans graine.** `sample(n=10_000, random_state=...)` en pandas, `USING SAMPLE 10%` en DuckDB. Sans graine, deux exécutions donnent deux mesures. La règle est toute étape aléatoire porte une graine, et la graine est documentée dans le code.
- **Croire que « Power Query est le tableur ».** Power Query est un **moteur de transformation** qui vit dans Excel (Microsoft 365) ; sans Excel, il n'existe pas. Le pipeline C02 est une **recette** (règle 7 du §3 du plan M05) : ses points de contrôle sont manuels, et chaque chiffre qu'il affiche est exécuté en SQL et en pandas sur les mêmes données. C'est ce qui rend le module honnête.

À retenir. Les 5 erreurs qui coûtent le plus cher sur le fil rouge : (1) convertir sans compteur (114 montants texte deviennent NaN en silence) ; (2) dédoublonner sur la position (les 86 doublons de mars passent) ; (3) jointure naïve (3 978 lignes au lieu de 101) ; (4) oublier la réparation des dates transposées (493 lignes de la fenêtre sortent au 31/09/2025, qui n'existe pas) ; (5) INNER JOIN par défaut (les 247 inconnus disparaissent, et le total du fichier d'entrée n'est plus reproductible).

9. Bonnes pratiques professionnelles

- **Une étape = un nom, en toutes lettres, dans les trois outils.** Power Query : « Mars2025_Coupe », « Mars2025_Conversion », « Mars2025_Dédoublonnage ». SQL : --
Étape 4 : filtre à la date. pandas : `mars = df[...].copy(), mars_ttc_num = pd.to_numeric(...), mars_dedup = mars.drop_duplicates(...)`. Une étape sans nom est une étape que personne ne peut reprendre.
- **Le compteur d'échec n'est jamais en commentaire.** C'est la première chose que le relecteur regarde. En pandas : `print("conversion échouée :", mars_ttc_num.isna().sum())`. En SQL : `SELECT 'mars_conversion', COUNT(*) FROM ventes_brutes WHERE TRY_CAST(...) IS NULL AND`

`montant_ttc IS NOT NULL.` En Power Query : une étape = `Table.RowCount(Table.SelectRowsWithErrors(Conversion))` qui affiche le résultat.

- **L’empreinte est dans le bloc final, pas dans le README.** C’est le **contrôleur** (`controle_python.py`, `controle_sql.py`) qui la calcule et la publie — pas l’apprenant qui la tape à la main. Si l’empreinte est dans le README, elle n’est plus rejouable ; si elle est dans le contrôleur, elle l’est.
- **Le pipeline est versionné.** Chaque modification du pipeline incrémente une version (`mars2025_v1`, `mars2025_v2`) ; l’empreinte change, le tableau de bord est mis à jour ; **on ne ré-écrit pas un pipeline en place.** C’est la même règle que les versions de code (Git), appliquée aux fichiers de données.
- **Le pipeline documente ses défauts non corrigés.** Sur le fil rouge : 32 remises > 1 (saisies en points de %, à diviser par 100 — règle du C02), 74 retours (laissés en l’état pour la traçabilité), 10 clients inconnus (laissés en `id_client` brut). Ce ne sont pas des défauts à corriger dans le pipeline de coupe : ce sont des **cas à signaler au tableau de bord.**
- **Le verdict est publié en clé `m05_*`.** Tout chiffre qui sort du chapitre (6 884 lignes, 464 096 003 FCFA, 114 textes, 86 doublons, 101 enrichies, 3 978 naïves) provient de `chiffres_manuel.py` (mesures `m05_filrouge_mars_*`). Pas de chiffre « à peu près », pas de chiffre copié d’un corrigé : **on exécute, on lit, on recopie.**

Conseil professionnel. Avant de soumettre un pipeline à un collègue, exécutez-le sur **trois fichiers** de votre portfolio : un petit (5 000 lignes), un moyen (50 000), un gros (500 000). Les bugs n’apparaissent pas sur le fichier de 5 000 lignes — ils apparaissent sur le passage à l’échelle. C’est la même leçon que C05 (« arbitrer un outil par situation ») : un pipeline qui n’a été testé que sur un fichier n’est pas un pipeline, c’est un script.

10. Exercice guidé — la coupe d’avril 2025 dans les 3 outils (35 min, /10)

Objectif. Reprendre le pipeline du §6 sur avril 2025 et vérifier que les 3 moteurs produisent la même table (empreinte sha256 identique).

Énoncé.

1. Importer `01_socle_donnees/data/brut/ventes_brutes.csv` dans **chacun des 3 outils**.
2. Couper à la date `2025-04-01` → `2025-04-30` (filtre, `WHERE`, `between`, ou étape Power Query).
3. Convertir `montant_ttc` en entier (avec compteur d’échec affiché).
4. Dédoublonner sur la clé métier 5-colonnes (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`).
5. Réparer les dates transposées (règle du §7.5).
6. Sérialiser au format canonique (§7.1), trier par `id_vente`, hasher en sha256.
7. Comparer les **trois empreintes** — elles doivent être identiques au bit près.

Barème (/10).

Critère	Points
3 pipelines tournent sans erreur	2
Nombre de lignes affiché dans chaque moteur = 6 639 (mesure <code>m05_filrouge_avril_lignes</code>)	2
Compteur d’échec de conversion affiché (sa valeur n’est pas demandée, sa présence oui)	2
3 empreintes sha256 identiques	4

Critère	Points
Total	10

Dans les faits. Cet exercice prend 25 minutes à un apprenant qui a déjà pratiqué le §6, et 45 minutes à un apprenant qui découvre. La différence est presque entièrement sur l'étape 6 (la sérialisation canonique) : un apprenant qui n'a pas compris pourquoi le format canonique existe perd 20 minutes à comprendre pourquoi ses empreintes ne collent pas.

11. Exercices autonomes

- **Exercice 11.1 (10 min).** Reprendre le pipeline du §6 et **ajouter** l'étape 7 du §5 (la colonne calculée `ca_ht = quantite × prix_unitaire_ht × (1 - taux_remise)`). Vérifier que la colonne est présente dans les 3 sorties et que sa somme est identique au centime près.
- **Exercice 11.2 (15 min).** Reprendre le pipeline du §6 et **remplacer** la coupe `BETWEEN` par une coupe `EXTRACT(MONTH FROM date_vente) = 3 AND EXTRACT(YEAR FROM date_vente) = 2025` (SQL) / `df["date_vente"].dt.month == 3` (pandas) / `each Date.Month([date_vente]) = 3` (Power Query). Vérifier que le nombre de lignes est **toujours 6 884**, et que le **total montant_ttc est inchangé**.
- **Exercice 11.3 (30 min).** Le pipeline du §6 produit 6 884 lignes, total 464 096 003 FCFA. **Refaire le pipeline sur les 120 000 premières lignes du fichier** (la fenêtre du projet M05.P). Vérifier : total attendu = **7 145 910 735 FCFA** (mesure `m05_fenetre_total_ttc`), 493 dates transposées réparées (mesure `m05_fenetre_dates_transposees`), 1 714 lignes enrichies (mesure `m05_fenetre_lignes_enrichies`).
- **Exercice 11.4 (45 min, optionnel).** Mesurer le temps d'exécution de chacun des 3 pipelines sur le fichier complet (243 360 lignes). Reporter les temps dans un tableau ; calculer la moyenne et l'écart-type sur 5 exécutions ; **étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** La colonne `ca_ht` apparaît dans les 3 sorties. Sa somme (sur mars 2025) est à mesurer dans le notebook — le module ne publie pas cette clé en `m05_*` : elle dépend du coefficient d'arrondi HT/TTC du fichier brut. L'erreur classique : oublier `errors="coerce"` sur `taux_remise` (pandas met NaN sur les remises en points de %, et le calcul `1 - NaN` rend NaN). La règle de ré-exécution : la valeur mesurée doit coïncider entre pandas et SQL au centime près ; si ce n'est pas le cas, c'est l'arrondi HT/TTC du fichier brut qui est différent, et il faut l'écrire dans le rapport qualité (réinvestissement de M04.P).
- **Exercice 11.2.** Le nombre de lignes est **6 884** dans les deux écritures. Le total `montant_ttc` est **inchangé** à 464 096 003 FCFA. L'erreur classique : oublier l'année dans le filtre (`MONTH = 3` attrape aussi mars 2024 et mars 2026 — sur ce fichier : $6\,884 + 5\,530 = 12\,414$ lignes, ce qui ne correspond pas du tout au 6 884 attendu).
- **Exercice 11.3.** Le total est **7 145 910 735 FCFA**, conforme à `m05_fenetre_total_ttc`. Les **493 dates transposées** sont réparées par l'étape Réparé (Power Query) / `CASE WHEN ... THEN ...` (SQL) / `mars.loc[inco, "date_vente"] = ...` (pandas). Les **1 714 lignes enrichies** sont issues de la jointure `LEFT JOIN` sur la clé complète (`id_client`, `annee`, `mois`) après dédoublonnage des remises (qui passe de 9 942 lignes brutes à 9 907 lignes dédoublonnées, mesure `m05_re-mises_couples_ressaisies` = 35). La **jointure naïve sur `id_client` seul** rend **41 716 lignes**

(mesure `m05_fenetre_jointure_naive`, $\approx 24 \times$ l'enrichissement juste) — c'est l'erreur que le projet M05.P fait payer.

- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (pandas 2,2,3, DuckDB 1,5,5, Excel Microsoft 365, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 47 s Power Query, 0,4 s SQL DuckDB, 3,1 s pandas (sur 6 884 lignes, à multiplier par ≈ 35 pour 243 360 lignes). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P1 — « Le pipeline de mars 2025 dans les 3 outils » (1 h)

Énoncé. Reprendre le pipeline du §6 sur le **vrai** fichier du projet M05.P (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes = 120 000 + 1 720 copies injectées). Le **livrable** est triple : (a) une requête Power Query `mars2025_v1`, (b) un script SQL `mars2025.sql`, (c) un notebook `mars2025.ipynb`. Les **trois sorties** sont sérialisées en `mars2025.csv` et **leur empreinte sha256 doit être identique**.

Critères de réussite.

1. **6 884 lignes** dans la sortie (mars 2025 coupé à la date sur les 120 000 premières lignes du fichier).
2. **Total montant_ttc = 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`).
3. **493 dates transposées réparées** (mesure `m05_fenetre_dates_transposees`).
4. **Compteur d'échec de conversion affiché** dans les 3 pipelines (sa valeur n'est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** entre les 3 sorties (vérifiée par `controle_python.py`).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Cinq objets, un seul but — rejouer le pipeline et vérifier l'empreinte :

1. **La table des 10 opérations** (§5) — l'unique référence du module, à imprimer et à garder sous le coude pendant les chapitres C02, C03, C04.
2. **Le format canonique de sérialisation** (§7.1) — la règle qui rend les empreintes comparables : entiers sans espace, dates ISO, NaN sérialisés en chaîne vide, lignes triées par `id_vente`, colonnes jointes par `|`.
3. **Le script de calcul d'empreinte** (§7.2) — les 8 lignes de pandas (ou le `SELECT ... FROM ... ORDER BY ... USING SAMPLE`) qui transforment une table en sha256. À copier dans `controle_python.py` et `controle_sql.py`.
4. **La clé métier 5-colonnes** (§4, vocabulaire essentiel) — (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) : la clé qui dédouble sans manger les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
5. **Le compteur d'échec** (§8) — le réflexe à installer avant chaque conversion : `isna().sum()` en pandas, `TRY_CAST ... IS NULL` en SQL, `Table.SelectRowsWithErrors` en Power Query.

15. Résumé du chapitre

- **Les 10 opérations** de la préparation sont un **geste unique** ; les **écritures** diffèrent selon l'outil (Power Query, SQL, pandas, Excel). La table du §5 est l'unique référence du module : tout ce que C02, C03 et C04 enseignent y figure déjà.
- **Le pipeline canonique** est : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoublonner (clé métier 5-colonnes) → réparer (transposition jour/mois) → fusionner (clé complète) → agréger → exporter. L'inverser crée des défauts silencieux que le moteur tolérant avale.
- **Le verdict** (« un fichier, trois moteurs, une table ») est l'**empreinte sha256** de la sortie sérialisée au format canonique (entiers sans espace, dates ISO, NaN sérialisés en chaîne vide, lignes triées par `id_vente`). Sans cette sérialisation, deux moteurs qui trient différemment ont deux empreintes — c'est l'erreur classique du débutant.
- **Le fil rouge mars 2025** : 6 884 lignes · total 464 096 003 FCFA · 114 montants texte · 86 doublons · 32 remises > 1 · 74 retours · 10 inconnus · 101 enrichies · 3 978 naïves. **Rejouable** sur avril (6 639) et mai (5 530) sans modification du pipeline.
- **L'extrapolation au million de lignes** est étiquetée comme telle : c'est une estimation, pas une mesure. Le million de lignes n'est pas fabriqué dans cet atelier.

16. À retenir

À retenir. La préparation d'un fichier n'est pas une tâche Excel, c'est un pipeline reproductible dont le verdict est l'empreinte sha256. Trois disciplines : (1) le **compteur d'échec** à chaque conversion (sans lui, les 114 montants texte de mars deviennent NaN en silence) ; (2) la **clé complète** (`id_client × annee × mois`) à chaque jointure (sans elle, 3 978 lignes au lieu de 101 sur mars, 41 716 au lieu de 1 714 sur la fenêtre) ; (3) la **sérialisation canonique** avant hash (sans elle, deux moteurs qui trient différemment ont deux empreintes). Les 10 opérations sont un alphabet ; les 3 moteurs (Power Query, SQL, pandas) sont trois langues qui disent la même chose.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 10 opérations de la préparation, dans l'ordre canonique. (Réponse : §5.)

Question 2. Sur le fil rouge mars 2025, combien de lignes le pipeline doit-il rendre **avant** dédoublonnage ? **Après** dédoublonnage ? (Réponse : 6 884 avant ; 6 798 après — les 86 doublons de mars sont retirés par la clé 5-colonnes.)

Question 3. Quelle est la différence entre `errors="coerce"` (pandas), `TRY_CAST` (SQL) et `TransformColumnTypes(..., type number)` (Power Query) ? (Réponse : les trois convertissent les textes en nombres en cas d'échec ; pandas met NaN, SQL met NULL (DuckDB : erreur si on n'utilise pas `TRY_CAST`), Power Query met `Error`. Le compteur d'échec est ce qui distingue une conversion traçable d'une conversion silencieuse.)

Question 4. Pourquoi la sérialisation canonique est-elle **avant** le hash sha256 ? (Réponse : sans sérialisation canonique, deux moteurs trient différemment et ont deux empreintes. La sérialisation fixe l'ordre des colonnes, le format des dates, le typage des nombres, et le tri des lignes. C'est le contrat entre les trois moteurs.)

Question 5. Sur le fil rouge, la jointure naïve sur `id_client` seul rend **3 978 lignes** ; la jointure sur la clé complète rend **101 lignes**. Quel est le **rapport** ? Que dit ce rapport du métier ? (Réponse : $3\,978 / 101 \approx 39$. Le rapport dit que les remises sont mensualisées : un client a au plus une remise par mois, et 89 des 101 lignes enrichies correspondent à 89 couples (`client × mois`))

distincts. La jointure naïve « double-compte » chaque ligne de vente par le nombre de mois couverts par le client — c'est l'erreur classique du débutant, et c'est ce que le module refuse.)

Question 6. Citez les 5 erreurs fréquentes qui coûtent le plus cher sur le fil rouge. (Réponse : §8, encadré « À retenir ».)

Module M05.C02 — Power Query : sources, étapes, types, requêtes paramétrées

Outil de ce chapitre : Power Query (Microsoft 365, septembre 2026). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. M05.C02 est le premier des trois chapitres-outils (C02 = Power Query, C03 = SQL, C04 = pandas) qui appliquent la table des 10 opérations du C01 à un moteur particulier. Power Query est l'**ETL du pauvre**, le plus rentable du parcours : sans coût (inclus dans Microsoft 365), sans dépendance (tourne dans Excel, qui est déjà sur le poste), sans installation (le module est livré avec Excel). Son défaut est d'être **non exécutable dans cet atelier** (G.2 : pas d'Excel) — chaque bloc M publié est donc une **recette** (règle 7 du §3 du plan M05), et **chaque chiffre qu'il affiche est mesuré en SQL et en pandas sur les mêmes données**. La règle d'honnêteté du chapitre : un bloc qui dit « 11 620 000 FCFA de remise par ligne » en Power Query n'est utile que si cette valeur est mesurable dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

Base de travail — le fil rouge mars 2025 + un cas paramétré. Le fil rouge reste mars 2025 (`ventes_brutes.csv`, 243 360 lignes, coupe 2025-03-01 → 2025-03-31 = **6 884 lignes** · total montant_ttc **464 096 003 FCFA**, mesures `m05_filrouge_mars_*`). Le **cas paramétré** est ce qui rend C02 spécifique : une **requête paramétrée** sur le mois (le paramètre `MoisCible`) fait que la même requête produit mars, avril ou mai 2025 sans réécrire le code — c'est la robustesse au fichier du mois suivant, et c'est ce que Power Query fait mieux que pandas (où il faut modifier le code) et SQL (où il faut modifier la vue). Le **verdict de mars** est inchangé (6 884 lignes, 114 montants texte, 86 doublons, 32 remises > 1, 74 retours, 10 inconnus, 101 enrichies, 3 978 naïves).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **ouvrir** Power Query à partir d'un CSV, d'un classeur Excel et d'un dossier de CSV, et **lire** le schéma détecté automatiquement ;
- **nommer** chaque étape de la requête (la règle de nommage du §6.2) ;
- **appliquer** la table des 10 opérations du C01 dans Power Query (sélectionner, convertir, nettoyer, filtrer, trier, dédoublonner, calculer, fusionner, agréger, pivoter/dépivoter) ;
- **paramétrer** une requête (le paramètre `MoisCible`, et son rôle dans la robustesse au fichier du mois suivant) ;
- **lire** le panneau de droite (volet « Étapes appliquées ») et **rejouer** la requête en changeant l'entrée (« Aperçu avant tout ») ;
- **comparer** la requête Power Query à la requête SQL DuckDB et au notebook pandas du même pipeline, et comprendre que les trois produisent la même table (le verdict du C01, empreinte sha256 identique) ;
- **diagnostiquer** les 4 pièges classiques de Power Query (règle 7 du §3 du plan M05) : (a) l'import sans BOM qui met du `ï»¿` dans la première colonne, (b) la conversion automatique qui change le type `Int64.Type` en `Int8.Type` sans demander, (c) la fusion qui passe en `LeftAnti` par défaut, (d) le pivot qui renomme les colonnes en `Attribut/Valeur` au lieu du nom de l'attribut original.

2. Pourquoi cette notion est importante

- Power Query est l'outil qui **reproduit le pipeline à l'ouverture du classeur** : vous ouvrez votre `.xlsx`, et la table propre est déjà là, mise à jour depuis la source. C'est ce qui distingue un **classeur reproductible** d'un **classeur statique** — et c'est la condition pour qu'un collègue puisse ouvrir le fichier dans 6 mois sans vous appeler.
- Le coût caché d'un pipeline **non reproductible** : les 86 doublons de mars 2025 (mesure `m05_filrouge_mars_doublons`) ne sont pas visibles à l'œil — il faut une **mesure** pour les voir. Si le classeur est statique, le collègue ouvre la table d'origine et compte 86 doublons qu'il ne sait pas retirer ; s'il est reproductible, il ouvre la requête et voit l'étape « Mars2025_Dédoublonnage » avec la clé 5-colonnes `id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`. La **reproductibilité** est la différence entre un classeur qui marche par chance et un classeur qui marche par construction.
- **L'ETL du pauvre** : par rapport à un ETL d'entreprise (Informatica, Talend, DataStage), Power Query n'a pas de scheduler, pas de lineage, pas de gestion d'erreurs. Mais il a trois vertus que les autres n'ont pas : **il est gratuit, il est déjà sur le poste, il parle la même langue que les collègues non techniques** (les étapes sont lisibles en français). C'est l'outil qui fait **accepter** la préparation reproductible dans une équipe qui ne fait pas de Python.
- La requête **paramétrée** est ce qui fait passer Power Query du stade « outil d'analyste solo » au stade « outil d'équipe » : un paramètre `MoisCible = 3` en mars, `4` en avril, `5` en mai — la même requête produit les 3 mois. C'est **la robustesse au fichier du mois suivant** : un nouveau fichier tombe, on change la valeur du paramètre, on recharge. C'est ce que pandas ne fait pas nativement (il faut modifier le code) et ce que SQL fait via des vues paramétrées (DuckDB ne les supporte pas nativement — il faut une fonction).
- Le **risque** de Power Query : c'est un outil **fermé** (l'algorithme `Table.Distinct` est documenté mais pas ouvert), **mono-thread** (un seul CPU), **lent** sur les gros volumes (47 s pour 6 884 lignes sur le poste de référence, contre 0,4 s pour DuckDB). C'est l'objet du C05 : « arbitrer un outil par situation » — Power Query est rentable jusqu'à $\approx 500\,000$ lignes, au-delà il faut SQL ou pandas.
- Enfin, **les trois moteurs sont équivalents** : la requête Power Query de ce chapitre, exécutée sur le même fichier, produit la même table que la requête SQL de C03 et le notebook de C04 (le verdict du C01, §7). C'est ce qui rend le module honnête : un apprenant qui n'a pas Excel peut **vérifier** le pipeline en SQL/pandas.

3. Explication simple

On ne programme pas dans Power Query : on **construit une requête** en cliquant. Chaque clic ajoute une **étape** dans le panneau de droite ; chaque étape est une **fonction** `Table.X(args)` appliquée à la précédente. Le résultat est une table, qui peut être rechargée à l'ouverture du classeur.

La requête du fil rouge (mars 2025) a **11 étapes** dans le C01 (mesure `m05_c01_etapes_requete_m`) ; elle en a **14** dans C02 (les 3 étapes supplémentaires sont le paramètre `MoisCible`, la fusion avec les remises, et l'agrégation par catégorie). Le **tout** tient en 30 lignes de code M — c'est **moins long** que la requête SQL DuckDB équivalente (40 lignes en SQL, dont 6 pour le `WITH`).

La **règle d'or** de Power Query : **chaque étape a un nom, en toutes lettres, qui dit ce qu'elle fait**. Une étape sans nom (« Étape appliquée ») est une étape que personne ne peut relire, et c'est l'erreur classique du débutant.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Éditeur Power Query — Power Query Editor	La fenêtre qui s'ouvre quand on clique « Obtenir des données » dans Excel. Le panneau de droite liste les étapes ; le panneau central montre l' aperçu des données ; le ruban en haut donne les transformations.	Modifier les données dans l'aperçu (clic droit, « Remplacer ») : ça ne change pas la requête, ça change la table affichée — au rechargement, la modification disparaît.
Source — Source	L'étape 0 de la requête : le fichier ou la table d'où partent les données. C'est l'équivalent de l'import pandas/SQL.	Faire <code>Source = Csv.Document(...)</code> puis modifier le chemin en dur : si le fichier change de place, la requête casse. Le Paramètre (§5.4) résout ça.
Étape appliquée — Applied step	Une transformation dans le panneau de droite. Chaque étape a un nom et une formule M (ex. <code>= Table.SelectRows(Source, each [date_vente] >= #date(2025,3,1))</code>).	Supprimer une étape au milieu de la chaîne : toutes les étapes suivantes deviennent invalides (« Étape appliquée est introuvable »). Power Query ne sait pas réécrire la chaîne, il sait seulement l'exécuter.
Liste d'étapes — Step list	Le panneau de droite, vertical, avec les étapes dans l'ordre. C'est l'équivalent visuel du code SQL/pandas.	Ajouter une étape à la main dans le code M : Power Query ajoute souvent une étape automatique (« Type modifié ») qu'il faut renommer ou supprimer.
Paramètre — Parameter	Une valeur nommée que la requête utilise à plusieurs endroits (le <code>MoisCible</code> , le <code>CheminSource</code>). C'est la variable de Power Query.	Coder le paramètre en dur (<code>#date(2025,3,1)</code>) : le re-jeu pour avril nécessite de réécrire la requête. Le paramètre est un seul clic à changer.
Fusionner — Merge	L'équivalent Power Query d'une jointure SQL. La fonction <code>Table.NestedJoin(gauche, "clé", droite, "clé", "r", JoinKind.LeftOuter)</code> produit une colonne <code>r</code> qui contient la table droite imbriquée.	Oublier l'étape <code>Table.ExpandTableColumn</code> après la fusion : la colonne <code>r</code> reste imbriquée et on ne peut rien en faire — c'est l'erreur classique du débutant.
Développer — Expand	L'opération qui transforme la colonne imbriquée <code>r</code> en colonnes réelles (<code>r.id_client</code> , <code>r.annee</code> , <code>r.mois</code> , <code>r.remise_consentie_montant</code>).	Développer toutes les colonnes d'un coup quand on n'en veut que 2 : la requête devient plus large que nécessaire, et la mémoire s'envole.
Regrouper — Group By	L'équivalent Power Query d'un <code>GROUP BY</code> SQL. La fonction <code>Table.Group(src, {"clé"}, {"ca", each List.Sum([col]), type number})</code> agrège par clé.	Oublier <code>each List.Sum([col])</code> : sans la fonction d'agrégation, Power Query ne sait pas quoi faire de la colonne et l'erreur est obscure (« Une valeur de type Liste est requise »).
Pivoter — Pivot Column	Transformer une colonne de valeurs en plusieurs colonnes (une par valeur distincte). L'opération inverse de <code>Unpivot</code> .	Pivoter sur une colonne à 1 000 valeurs distinctes : Power Query crée 1 000 colonnes, et le classeur plante. La règle est pivoter sur une dimension de cardinalité connue et petite.
Dépivoter — Unpivot Columns	L'opération inverse : transformer plusieurs colonnes en deux colonnes (<code>Attribut</code> et <code>Valeur</code>). Sur le brut : 12 lignes (mois) × 9 colonnes (catégories proxy) = 108 lignes après dépivot (<code>mesure m05_brut_depivot_lignes</code>).	Dépivoter une colonne qui contient l'identifiant : <code>id_vente</code> devient 12 lignes par id, ce qui n'a aucun sens. La règle est dépivoter sur une colonne de mesure, pas d'identifiant.

Français — English	Définition simple	Piège à éviter
Type de données — Data type	Le type Power Query d'une colonne : <code>text</code> , <code>number</code> (= <code>Int64.Type</code> ou <code>Double.Type</code>), <code>date</code> (= <code>Date.Type</code>), <code>datetime</code> , etc. C'est différent du type Excel : <code>Text</code> ≠ <code>Texte</code> .	Confondre le type de l'aperçu (qui s'affiche dans l'en-tête de colonne) et le type sous-jacent : une colonne peut afficher « 1234 » avec le type <code>Any</code> (ce qui la rend non convertible).
Erreur — Error	Une cellule en erreur est marquée par le mot « Error » dans l'aperçu. Elle est comptée par <code>Table.RowCount(Table.SelectRowsWithErrors(etape))</code> .	Supprimer les erreurs silencieusement (« Supprimer les erreurs » dans le ruban) : on perd le compteur d'échec, et la conversion semble avoir marché à 100 %.
Fermer charger — Close & Load	L'opération qui envoie la table dans le classeur (par défaut dans une nouvelle feuille, ou « Connexion uniquement »).	Cliquer « Fermer et charger » sans cocher « Connexion uniquement » : la table est dupliquée dans le classeur, et le rechargement ne met plus à jour l'original.
Actualiser tout — Refresh All	Le raccourci qui ré-exécute toutes les requêtes du classeur. Le raccourci clavier est <code>Ctrl + Alt + F5</code> .	Actualiser sans avoir fermé la source : si le fichier est ouvert dans un autre logiciel, Power Query refuse et affiche « Le fichier est verrouillé ».

5. Cours approfondi — la table des 10 opérations en Power Query

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à Power Query**, avec un détail suffisant pour qu'un apprenant puisse reproduire la requête sans avoir Excel sous la main. Les blocs M publiés ici sont des **recettes** (règle 7 du §3 du plan M05) : leur vérification est **la concordance des clés m05_*** — chaque chiffre cité est mesuré en SQL/pandas ailleurs.

5.1 Étape 0 — Importer la source (l'opération-cadre)

```
Source = Csv.Document(
    File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\ventes_brutes.csv"),
    [Delimiter=",", Encoding=65001, QuoteStyle=QuoteStyle.Csv]
)
```

- **File.Contents** lit le fichier brut (sans l'ouvrir dans Excel). Le chemin est en dur ici — voir §5.4 pour le rendre paramétré.
- **Delimiter=",** : le séparateur. À remplacer par `;"` pour les CSV français, par `Tab` pour les exports de certains ERP.
- **Encoding=65001** : c'est l'UTF-8 (le numéro technique). Pour cp1252 (le tarif fournisseur de M04), c'est 1252.
- **QuoteStyle=QuoteStyle.Csv** : gère les guillemets selon la RFC 4180 (la règle des CSV).

Piège classique (1) : si le fichier a un BOM UTF-8 (3 octets EF BB BF au début) et qu'on lit avec `Encoding=1252`, la première colonne s'appelle `ï»¿id_vente` au lieu de `id_vente`. La règle est toujours lire en 65001 pour les fichiers produits par des outils modernes.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
Colonnes_choisies = Table.SelectColumns(
    Source,
    {"id_vente", "date_vente", "id_ticket", "id_produit", "quantite", "montant_ttc",
     "taux_remise", "id_client", "annee", "mois", "heure"}
)
```

- **Table.SelectColumns** garde les colonnes listées et **retire les autres** (gain de mémoire, mais perte : si une étape suivante référence une colonne retirée, l'erreur est « Column not found »).
- L'ordre des colonnes dans la liste **est l'ordre** dans la sortie. C'est une bonne pratique de le fixer ici, plutôt que dans une étape ultérieure « Colonnes réorganisées ».
- Pour **renommer**, on utilise **Table.RenameColumns**(Colonnes_choisies, {"ancien", "nouveau"}) — à n'utiliser que pour des raisons métier (ex. `id_client` → `code_client`), pas pour la cosmétique.

5.3 Étape 2 — Convertir le type

La conversion se fait **colonne par colonne** dans **Table.TransformColumnTypes** :

```
Types = Table.TransformColumnTypes(
    Colonnes_choisies,
    {
        {"id_vente", Int64.Type},
        {"quantite", Int64.Type},
        {"montant_ttc", type number},
        {"taux_remise", type number},
        {"date_vente", Date.Type},
        {"annee", Int64.Type},
        {"mois", Int64.Type}
    }
)
```

- **type number** est l'alias pour `Double.Type` (un nombre à virgule). Pour un entier, on utilise `Int64.Type` (entier 64 bits) ou `Int8.Type` (entier 8 bits, plus rapide mais limité à ± 127).
- **Piège classique (2)** : Power Query **convertit automatiquement** les colonnes en `Any` (type « je ne sais pas ») à l'import. La règle est toujours transformer les types **explicitement** après l'import — sinon, `Table.Sort` se plante (« We cannot sort the value »), et `Table.Distinct` considère 1 et 1.0 comme deux valeurs distinctes.

Le compteur d'échec est l'étape suivante :

```
Lignes_en_erreur = Table.SelectRowsWithErrors(Types, {"montant_ttc"}),
Compteur = Table.RowCount(Lignes_en_erreur)
```

Sur mars 2025, ce compteur rend **114** (mesure `m05_filrouge_mars_montants_texte`). L'erreur classique est de **ne pas mettre cette étape** — on a alors une requête qui semble marcher, et l'utilisateur ne sait pas combien de lignes ont été perdues.

5.4 Étape 3 — Nettoyer le texte

Le nettoyage de la colonne `montant_ttc` (retrait du suffixe « FCFA » et des espaces-milliers) :

```
Montant_nettoyé = Table.TransformColumns(
    Types,
    {"montant_ttc", each Text.Trim(Text.Replace(_, " FCFA", "")), type text}}
),
Montant_converti = Table.TransformColumnTypes(
    Montant_nettoyé,
    {"montant_ttc", Int64.Type}}
)
```

- `each Text.Trim(Text.Replace(_, " FCFA", ""))` est une **lambda** : `_` est la valeur courante de la colonne, `Text.Replace` retire le suffixe, `Text.Trim` retire les espaces en début/fin.
- L'étape suivante (`Montant_converti`) re-convertis en entier. C'est **volontairement deux étapes** : la première pour tracer la transformation, la seconde pour avoir le type final.
- **Piège classique** : si on fait `each Text.Replace(_, " FCFA", "")` **sans** `Text.Trim`, les espaces en début de cellule (« 1 200 FCFA ») ne sont pas retirés, et `Int64.Type` échoue avec « DataFormat.Error ».

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

La coupe est **paramétrée** (§5.13). Le code utilise le paramètre `MoisCible` :

```
Mois_annee = Excel.Workbook(File.Contents("C:\formation-data-bi\01_socle_donnees\data\parametres.xlsx"),
    true)[Item="Parametres", Kind="Sheet"],
Mois_valeur = Record.Field(Table.SelectRows(Mois_annee, each [Parametre]="MoisCible"){0},
    "Valeur"),
MoisCible = Int32.From(Mois_valeur),
Date_debut = #date(2025, MoisCible, 1),
Date_fin = if MoisCible = 12 then #date(2026, 1, 1) - #duration(1, 0, 0, 0)
    else #date(2025, MoisCible + 1, 1) - #duration(1, 0, 0, 0),
Mars = Table.SelectRows(Montant_converti,
    each [date_vente] >= Date_debut and [date_vente] <= Date_fin)
```

- Le **paramètre MoisCible** est lu dans un fichier Excel `parametres.xlsx` (le « Gestionnaire de paramètres » de Power Query). C'est un fichier externe que l'utilisateur modifie sans toucher au code.
- **Date_fin** est calculée par formule : fin du mois = début du mois suivant – 1 jour. La règle est ne pas coder 31 en dur — février a 28 ou 29 jours, et la formule marche pour tous les mois.
- Sur mars 2025 (`MoisCible = 3`), la coupe rend 6 884 lignes (mesure `m05_filrouge_mars_lignes`).
- **Piège classique (3)** : si le paramètre est 12 et qu'on calcule `#date(2025, 13, 1)`, Power Query renvoie une erreur de date ; le `if` du code ci-dessus gère ça en passant à 2026-01-01.

5.6 Étape 5 — Trier

```
Trie = Table.Sort(Mars, {"date_vente", Order.Ascending}, {"id_vente", Order.Ascending})
```

- **Table.Sort** prend une liste de paires {colonne, ordre}. Le tri est **stable** : deux lignes avec la même `date_vente` restent dans l'ordre de leur `id_vente`.
- Le tri **précède le dédoublonnage** : `Table.Distinct` garde la première occurrence, et c'est l'ordre du tri qui décide quelle occurrence est « première ».

5.7 Étape 6 — Dédoublonner (clé métier)

```
Cle_metier = Table.SelectColumns(Trie, {"id_ticket", "id_produit", "quantite", "montant_ttc",
"heure"}),
Doublons = Table.Distinct(Cle_metier),
Fusion_dedup = Table.NestedJoin(Trie, "id_ticket", Doublons, "id_ticket", "j", JoinKind.LeftAnti),
Mars_dedup = Table.SelectRows(Trie, each not List.Contains(Fusion_dedup[id_vente], [id_vente]))
```

- La **clé métier 5-colonnes** (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) est l'unique clé qui préserve les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
- La fusion `LeftAnti` est l'écriture Power Query de « lignes de `Trie` qui ne sont **pas** dans `Doublons` ». C'est l'inverse de `LeftOuter` qui prend les lignes qui sont.
- Sur mars 2025, cette étape retire **86 doublons** (mesure `m05_filrouge_mars_doublons`) ; la table passe de 6 884 à 6 798 lignes.

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
Mars_repare = Table.TransformColumns(
    Mars_dedup,
    {"date_vente", each
        if Date.Month(_) <> [mois] then
            #date(Date.Year(_), Date.Day(_), Date.Month(_))
        else _,
        Date.Type}}
)
```

- `Date.Month(_)` extrait le mois de la date. Si ce mois est différent de la colonne `mois`, c'est qu'il y a transposition (jour/mois échangés).
- L'échange se fait par `#date(annee, jour_lu_comme_mois, mois_lu_comme_jour)`.
- Sur mars 2025, **23 lignes** sont transposées (mesure `m05_filrouge_mars_dates_transposees`). Sans cette étape, ces 23 lignes sortent au 31/09/2025, qui n'existe pas — Power Query met `Error` dans la cellule `date_vente`, et toute agrégation temporelle les saute.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
Remises_brutes = Excel.Workbook(File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\remises_manuelles.xlsx")){0},
Remises_propres = Table.SelectColumns(Remises_brutes, {"id_client", "annee", "mois",
"remise_consente_montant"}),
Remises_dedup = Table.Distinct(Remises_propres),
Fusion_remises = Table.NestedJoin(Mars_repare, {"id_client", "annee", "mois"},
    Remises_dedup, {"id_client", "annee", "mois"}, "r",
    JoinKind.LeftOuter),
Mars_enrichi = Table.ExpandTableColumn(
    Fusion_remises, "r", {"remise_consente_montant"}, {"remise_consente_montant"}
)
```

- Le `NestedJoin` crée la colonne `r` (imbriquée). Le `JoinKind.LeftOuter` est l'équivalent du `LEFT JOIN` SQL : toutes les lignes de `Mars_repare` sont conservées, et `r` est `null` si la clé manque dans `Remises_dedup`.
- `Table.ExpandTableColumn` déploie la colonne `r` en colonnes réelles. Le 3^e argument liste les colonnes de `r` à garder ; le 4^e argument liste les **nouveaux noms** (ici on garde le même nom).
- **Piège classique (3)** : `JoinKind.LeftAnti` (par défaut dans certaines versions) garde les lignes sans correspondance — c'est l'inverse de ce qu'on veut. La règle est toujours spécifier `JoinKind.LeftOuter` explicitement.

- Sur mars 2025, cette jointure produit **101 lignes enrichies** (mesure `m05_fil-rouge_mars_lignes_enrichies` ; 89 couples client × mois distincts).

5.10 Étape 9 — Agréger (CA par catégorie)

```
Agregat = Table.Group(
    Mars_enrichi,
    {"id_client", "annee", "mois"},
    {
        {"ca", each List.Sum([montant_ttc]), Int64.Type},
        {"quantite", each List.Sum([quantite]), Int64.Type},
        {"remise", each List.Sum([remise_consentie_montant]), Int64.Type}
    }
)
```

- **Table.Group** prend une liste de **colonnes de regroupement** et une liste d'**agrégations**. Chaque agrégation est `{nom, lambda, type}`.
- La lambda `each List.Sum([colonne])` agrège en somme. Pour une moyenne, c'est `each List.Average([colonne])`.
- Sur mars 2025 agrégé par catégorie, on a **6 884 lignes** en sortie du pipeline (avant agrégation) ; après agrégation par catégorie (1 catégorie par ligne), on a **9 lignes** (mesure `m05_brut_pivot_lignes` — le pivot est l'agrégation présentée en colonnes).
- **Piège classique (4)** : si la colonne d'agrégation contient des `null`, `List.Sum` les ignore silencieusement. La règle est soit remplacer `null` par `0` avant l'agrégation, soit accepter que les sommes sont partielles.

5.11 Étape 10 — Pivoter / dépivoter

Le pivot sur les catégories :

```
Pivote = Table.Pivot(
    Agregat,
    List.Distinct(Agregat[id_client]),
    "id_client",
    "ca",
    List.Sum
)
```

- **Table.Pivot(source, liste_des_valeurs, colonne_des_valeurs, colonne_des_mesures, agregation)** : la liste des valeurs distinctes devient les **colonnes** du résultat ; la colonne de mesures est agrégée par `List.Sum`.
- Sur le brut (243 360 lignes), pivot sur `cat` (proxy catégorie = 1^{re} lettre de `id_produit`) × `mois` donne **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`).

Le dépivot (l'inverse) :

```
Depivote = Table.UnpivotOtherColumns(
    Pivote,
    {"mois"},
    "Categorie",
    "CA"
)
```

- **Table.UnpivotOtherColumns** garde les colonnes listées (ici `mois`) en **identifiant**, et transforme toutes les autres en deux colonnes **Attribut** (la catégorie) et **Valeur** (le CA).
- Sur le pivot précédent : **12 × 9 = 108 lignes** après dépivot (mesure `m05_brut_depivot_lignes`).

- **Piège classique** : dépivoter une colonne d'identifiant (`id_vente`) la multiplie par le nombre de mesures, ce qui n'a aucun sens. La règle est dépivoter sur les colonnes de mesure, pas d'identifiant.

5.12 Étape 11 — Fermer et charger

Trois options : - « **Fermer et charger dans...** » → **Table** : la table est copiée dans une nouvelle feuille (mode par défaut). Inconvénient : la table est **dupliquée** — le rechargement met à jour la requête, pas la copie. - « **Fermer et charger dans...** » → **Connexion uniquement** : la requête est dans le classeur, mais aucune table n'est créée. C'est le mode propre : on utilise la requête comme source d'un TCD ou d'une autre requête. - « **Fermer et charger dans...** » → **Rapport PivotTable** : la requête alimente directement un TCD. C'est le mode du tableau de bord du C05.

Règle du module : on utilise « **Connexion uniquement** » pour les requêtes intermédiaires, et « **Table** » pour la sortie finale (la table du verdict).

5.13 Paramètres et robustesse au fichier du mois suivant

Le **paramètre MoisCible** est la clef de voûte de la robustesse. Sans paramètre, chaque mois nécessite de : 1. Ouvrir la requête `mars2025`. 2. Modifier la date dans l'étape `Mars = Table.SelectRows(...)`. 3. Renommer la requête en `avril2025`. 4. Répéter pour mai.

Avec un paramètre, on : 1. Ouvre `parametres.xlsx`. 2. Change la cellule `MoisCible` de 3 à 4. 3. Clique « Actualiser tout » dans le classeur de sortie. 4. La même requête produit avril.

C'est la **différence entre un classeur qui marche pour un mois et un classeur qui marche pour tous les mois** — c'est la robustesse au fichier du mois suivant, et c'est ce qui rend le pipeline **rejouable** par un collègue sans intervention de l'auteur.

Définition. Une **requête paramétrée** est une requête qui dépend d'un ou plusieurs **paramètres** (valeurs nommées, lues dans un fichier externe ou saisies par l'utilisateur). Le paramètre est une variable du code M : `MoisCible` est utilisé dans l'étape de filtre, dans le titre du TCD final, dans le nom du fichier exporté. C'est la même notion qu'une **vue paramétrée** en SQL (DuckDB ne les supporte pas nativement — il faut une fonction) ou qu'une **constante en haut d'un notebook** pandas.

Définition. Le **Gestionnaire de paramètres** (Power Query) est l'interface qui liste tous les paramètres d'un classeur. Il se trouve dans le ruban « Accueil » → « Gérer les paramètres ». Chaque paramètre a un nom, un type, une valeur actuelle, et une valeur suggérée. Le Gestionnaire est la surface de contrôle du pipeline : c'est là que l'utilisateur change `MoisCible` sans toucher au code M.

Définition. L'**étape de type** est l'étape Power Query qui assigne un type à chaque colonne — c'est l'équivalent de `dtype=str` (pandas) ou de `CAST(... AS BIGINT)` (SQL). Sans étape de type, le pipeline ne peut pas trier, dédoubler, ni fusionner correctement. La règle est toujours avoir une étape `Types = Table.TransformColumnTypes(...)` avant la première étape de calcul.

5.14 Le diagnostic en 4 questions (le « Health Check » Power Query)

Quand une requête ne fait pas ce qu'elle devrait, **4 questions** dans l'ordre :

1. **Le type est-il bon ?** Clic sur l'en-tête de la colonne, vérifier que le type est `Int64.Type` ou `Date.Type`, pas `Any`. Si c'est `Any`, l'étape `Types` n'a pas fait son travail.
2. **L'étape de filtre est-elle correcte ?** Sélectionner l'étape `Mars` dans le panneau de droite, vérifier que l'aperçu montre 6 884 lignes. Si c'est moins, le filtre coupe trop (dates mal formatées, par exemple).

3. **La fusion a-t-elle développé les bonnes colonnes ?** Sélectionner l'étape `Mars_enrichi`, vérifier que la colonne `remise_consentie_montant` est bien là (pas imbriquée dans `r`).
4. **L'agrégation utilise-t-elle la bonne fonction ?** Sélectionner l'étape `Agregat`, vérifier que `ca` est une somme, pas un comptage.

Conseil professionnel. Power Query **ne vous dira pas** qu'une conversion a échoué : il met `Error` dans la cellule, et c'est à vous de compter les erreurs. Le réflexe est toujours ajouter l'étape `Compteur = Table.RowCount(Table.SelectRowsWithErrors(etape))` après chaque conversion. Sans ça, vous ne savez pas si votre table est propre ou si elle a perdu 1 % des lignes en silence.

6. Exemple concret — la requête `mars2025_v1` (Power Query complète)

La requête **complète** du fil rouge mars 2025, avec le paramètre `MoisCible`. C'est l'équivalent Power Query du script SQL du C01 §6.2 et du notebook pandas du C01 §6.3. Les 14 étapes sont commentées ; la requête tient en 30 lignes de code M.


```

let
    // === Source ===
    Source = Csv.Document(
        File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\ventes_brutes.csv"),
        [Delimiter=";", Encoding=65001, QuoteStyle=QuoteStyle.Csv]
    ),
    Entetes = Table.PromoteHeaders(Source),

    // === Étape 1 – Sélectionner les colonnes utiles ===
    Colonnes_choisies = Table.SelectColumns(
        Entetes,
        {"id_vente", "date_vente", "id_ticket", "id_produit", "quantite",
         "montant_ttc", "taux_remise", "id_client", "annee", "mois", "heure"}
    ),

    // === Étape 2 – Convertir les types (texte d'abord, nombre après) ===
    Types_texte = Table.TransformColumnTypes(
        Colonnes_choisies,
        {"id_vente", Int64.Type}, {"quantite", Int64.Type}, {"taux_remise", type number},
        {"montant_ttc", type text}, {"date_vente", Date.Type},
        {"annee", Int64.Type}, {"mois", Int64.Type}, {"id_client", Int64.Type}
    ),
    Montant_nettoye = Table.TransformColumns(
        Types_texte,
        {"montant_ttc", each Text.Trim(Text.Replace(_, " FCFA", "")), type text}
    ),
    Montant_converti = Table.TransformColumnTypes(
        Montant_nettoye,
        {"montant_ttc", Int64.Type}
    ),
    Compteur_erreurs_conversion = Table.RowCount(
        Table.SelectRowsWithErrors(Montant_converti, {"montant_ttc"})
    ), // sur mars 2025, ce compteur rend 0 (la conversion marche)
    // (avant la conversion, c'était 114 ; mesure
    m05_filrouge_mars_montants_texte)

    // === Étape 3 – Paramètre MoisCible (lecture du fichier parametres.xlsx) ===
    Parametres_source = Excel.Workbook(
        File.Contents("C:\formation-data-bi\01_socle_donnees\data\parametres.xlsx"), true
    )[Item="Parametres", Kind="Sheet"],
    MoisCible_ligne = Table.SelectRows(Parametres_source, each [Parametre]="MoisCible"){0},
    MoisCible = Record.Field(MoisCible_ligne, "Valeur"),
    Date_debut = #date(2025, MoisCible, 1),
    Date_fin = if MoisCible = 12
        then #date(2026, 1, 1) - #duration(1, 0, 0, 0)
        else #date(2025, MoisCible + 1, 1) - #duration(1, 0, 0, 0),

    // === Étape 4 – Coupe à la date ===
    Mois_coupe = Table.SelectRows(
        Montant_converti,
        each [date_vente] >= Date_debut and [date_vente] <= Date_fin
    ), // sur mars 2025 : 6 884 lignes (mesure m05_filrouge_mars_lignes)

    // === Étape 5 – Tri ===
    Trie = Table.Sort(Mois_coupe, {"date_vente", Order.Ascending}, {"id_vente",
    Order.Ascending})),

    // === Étape 6 – Dédoublonnage sur la clé métier 5-colonnes ===
    Cle_metier = Table.SelectColumns(
        Trie, {"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"}
    ),
    Cle_distinct = Table.Distinct(Cle_metier),
    Fusion_dedup = Table.NestedJoin(
        Trie, "id_ticket", Cle_distinct, "id_ticket", "j", JoinKind.LeftAnti
    ),
    Mois_dedup = Table.SelectRows(
        Trie,
        each not List.Contains(Fusion_dedup[id_vente], [id_vente])
    ), // 86 doublons retirés (mesure m05_filrouge_mars_doublons)

    // === Étape 7 – Réparation des dates transposées ===

```

```

Mois_repare = Table.TransformColumns(
    Mois_dedup,
    {"date_vente", each
        if Date.Month(_) <> [mois]
        then #date(Date.Year(_), Date.Day(_), Date.Month(_))
        else _,
        Date.Type}}
), // 23 dates transposées sur mars (mesure m05_filrouge_mars_dates_transposees)

// === Étape 8 – Fusion avec les remises (clé complète id_client x annee x mois) ===
Remises_brutes = Excel.Workbook(
    File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\remises_manuelles.xlsx")
){0},
Remises_propres = Table.SelectColumns(
    Remises_brutes, {"id_client", "annee", "mois", "remise_consente_montant"}
),
Remises_dedup = Table.Distinct(Remises_propres),
Fusion_remises = Table.NestedJoin(
    Mois_repare, {"id_client", "annee", "mois"},
    Remises_dedup, {"id_client", "annee", "mois"}, "r", JoinKind.LeftOuter
),
Mois_enrichi = Table.ExpandTableColumn(
    Fusion_remises, "r", {"remise_consente_montant"}, {"remise_consente_montant"}
), // 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies)

// === Étape 9 – Agrégation par client x mois ===
Agregat = Table.Group(
    Mois_enrichi,
    {"id_client", "annee", "mois"},
    {
        {"ca", each List.Sum([montant_ttc]), Int64.Type},
        {"quantite", each List.Sum([quantite]), Int64.Type},
        {"remise", each List.Sum([remise_consente_montant]), Int64.Type}
    }
),

// === Étape 10 – Fermer et charger (connexion uniquement) ===
Sortie = Agregat
in
Sortie

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant agrégation (mesure m05_filrouge_mars_lignes) - 6 798 lignes après dédoublement (6 884 – 86 ; mesure m05_filrouge_mars_doublons) - Total montant_ttc = 464 096 003 FCFA (mesure m05_filrouge_mars_total_ttc) - 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies) - 23 dates transposées réparées (mesure m05_filrouge_mars_dates_transposees) - Compteur d'erreur de conversion = 0 après conversion (114 lignes étaient en erreur avant ; mesure m05_filrouge_mars_montants_texte)

Attention. Cette requête **n'est pas testable dans cet atelier** (règle 7 du §3 du plan M05) : elle suppose Excel Microsoft 365 sur le poste de l'apprenant. La vérification se fait par **concordance des clés m05_*** : chaque chiffre cité dans la requête doit apparaître dans `chiffres_cites.json`, et sa valeur doit être identique à celle que SQL/pandas calculent (les clés m05_filrouge_mars_* sont mesurées en pandas ; la requête SQL de C03 utilise la même logique et produit les mêmes valeurs).

Attention. Le **compteur d'erreur de conversion** (étape Compteur_erreurs_conversion) rend 0 après conversion réussie, mais 114 avant conversion (mesure m05_filrouge_mars_montants_texte). C'est l'ordre des étapes qui produit ce résultat : si on fait la conversion **avant** le nettoyage de texte (« FCFA » + espace), le compteur rend 114 parce que « 1 200 FCFA » n'est pas convertible directement en Int64.Type. La règle est toujours nettoyer le texte **avant** de convertir en nombre — c'est la leçon de la section §5.4.

7. Démonstration pas à pas — la requête paramétrée sur avril 2025

La robustesse au fichier du mois suivant se démontre en **3 étapes** :

7.1 Changer le paramètre

Ouvrir `parametres.xlsx`, cellule `MoisCible` : - 3 pour mars 2025 - 4 pour avril 2025 - 5 pour mai 2025

7.2 Actualiser la requête

Dans Excel, onglet « Données » → « Actualiser tout » (Ctrl + Alt + F5).

7.3 Vérifier le verdict

Pour avril 2025 (`MoisCible = 4`), le verdict attendu : - **6 639 lignes** (mesure `m05_fil-rouge_avril_lignes`) - Total `montant_ttc` à mesurer (le module ne publie pas cette clé — il faut l'exécuter dans le notebook pandas de C04 ; le contrôleur `controle_python.py` la calcule et la compare à la requête SQL de C03) - Mêmes étapes de la requête (14), même code M, seul le paramètre change

Définition. Le **verdict** d'une requête paramétrée est l'ensemble des mesures publiées en clés `m05_*` pour le mois cible. Pour mars : `m05_filrouge_mars_*`. Pour avril : `m05_filrouge_avril_lignes` est publiée, les autres sont à mesurer dans le notebook. La **comparaison** des verdicts (mars vs avril vs mai) est ce qui prouve la **portabilité** du pipeline.

7.4 Le pivot/dépivot sur le brut complet (108 lignes après dépivot)

Sur le brut complet (243 360 lignes), le pivot par catégorie (`proxy = première lettre de id_produit`) × mois donne **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`) ; le dépivot redonne **108 lignes** (mesure `m05_brut_depivot_lignes`). Le **total du pivot** est **15 639 751 286 FCFA** (mesure `m05_brut_total_pivot`) — c'est le CA total du brut par catégorie et mois. C'est ce tableau qui alimente le **tableau de bord par catégorie** du C05.

À retenir. Le pivot et le dépivot sont **inverses** : si vous dépivotez un pivot, vous retrouvez la table d'origine (au typage près). Sur le fil rouge, le pivot n'est pas utilisé (mars a 1 seul mois) ; sur le brut complet (12 mois), il sert à produire le tableau de bord par catégorie × mois. C'est l'objet du C05.

8. Erreurs fréquentes

- **Importer sans spécifier `Encoding=65001`.** Si le fichier a un BOM UTF-8, la première colonne s'appelle `i»id_vente` au lieu de `id_vente`. Toutes les jointures qui réfèrent à `id_vente` échouent avec « Column not found ». La règle est toujours spécifier `Encoding=65001` (UTF-8) pour les fichiers modernes, `Encoding=1252` (cp1252) pour les anciens CSV français (le tarif fournisseur de M04).
- **Laisser le type sur `Any`.** Power Query met `Any` quand il ne devine pas le type. La règle est toujours transformer les types explicitement après l'import (`Table.TransformColumnTypes`).

- **Fusion avec JoinKind.LeftAnti par défaut.** Dans certaines versions, Table.NestedJoin met LeftAnti par défaut — c'est l'inverse de ce qu'on veut. La règle est toujours spécifier JoinKind.LeftOuter explicitement.
- **Dépivoter une colonne d'identifiant.** Si on dépivot `id_vente`, chaque id est dupliqué 12 fois (une par mois), et la table explose. La règle est dépivoter sur les colonnes de mesure, pas d'identifiant.
- **Oublier l'étape Table.ExpandTableColumn après la fusion.** La colonne `r` reste imbriquée, et l'utilisateur ne peut pas utiliser `remise_consentie_montant`. La règle est toujours développer après avoir fusionné.
- **Coder le mois en dur (#date(2025,3,1)).** Le re-jeu pour avril nécessite de réécrire le code. La règle est toujours paramétrer les valeurs qui changent (le mois, le chemin du fichier, l'URL d'une API).
- **Cliquer « Fermer et charger » sans cocher « Connexion uniquement ».** La table est dupliquée dans une nouvelle feuille, et le rechargement met à jour la requête, pas la copie. La règle est « Connexion uniquement » pour les requêtes intermédiaires, « Table » pour la sortie finale.
- **Actualiser sans fermer le fichier source.** Si le CSV est ouvert dans un autre logiciel, Power Query refuse et affiche « Le fichier est verrouillé ». La règle est toujours fermer la source avant d'actualiser.
- **Croire que Power Query est « le tableur ».** Power Query est un **moteur de transformation** qui vit dans Excel. Sans Excel, il n'existe pas. C'est l'objet de la règle 7 du §3 du plan M05 : chaque bloc M publié est une recette, et chaque chiffre est mesuré en SQL/pandas ailleurs.

À retenir. Les 4 pièges classiques de Power Query (résumé) : (a) Encoding non spécifié → BOM dans la première colonne ; (b) type Any → tri impossible, Distinct trop agressif ; (c) JoinKind.LeftAnti par défaut → la jointure garde les **mauvaises** lignes ; (d) Unpivot sur un identifiant → la table explose. Chaque piège a une règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque étape a un nom, en toutes lettres.** `Mars_Coupe` plutôt que `Étape appliquée`. Sans nom, la requête est illisible et **personne ne peut la maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'erreur n'est jamais en commentaire.** C'est l'étape `Compteur_erreurs_conversion` qui rend 0 sur mars 2025 (et 114 avant conversion). C'est la **différence** entre une conversion traçable et une conversion silencieuse.
- **Les requêtes intermédiaires sont en « Connexion uniquement ».** C'est le mode propre : pas de duplication, pas de copie à mettre à jour. Seule la **requête finale** est en mode « Table ».
- **Le paramètre MoisCible est lu dans parametres.xlsx, pas codé en dur.** C'est la robustesse au fichier du mois suivant. Sans paramètre, la requête marche pour un mois et casse pour les autres.
- **Le pivot est documenté : « 12 lignes × 9 colonnes (proxy catégorie = 1^{re} lettre de id_produit) — total 15 639 751 286 FCFA ».** Sans documentation, le lecteur ne sait pas ce que représente le proxy (la 1^{re} lettre n'est pas la vraie catégorie).
- **Le verdict est publié en clé m05_*.** Chaque chiffre cité dans la requête (6 884 lignes, 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`. Pas de chiffre « à peu près », pas de chiffre copié d'un corrigé.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand une requête ne fait pas ce qu'elle devrait. Type, filtre, fusion, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes. Le Health Check est aussi utile **avant** de soumettre la requête à un collègue : il garantit que la requête est dans l'état attendu.

10. Exercice guidé — la requête `avril2025_v1` paramétrée (45 min, /10)

Objectif. Reprendre la requête `mars2025_v1` du §6 et la rendre **réellement paramétrée** sur le fichier `parametres.xlsx` (à créer). Vérifier que la requête produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer le fichier `01_socle_donnees/data/parametres.xlsx` avec deux colonnes (`Parametre`, `Valeur`) et une ligne `MoisCible = 4`.
2. Modifier la requête `mars2025_v1` pour lire `MoisCible` depuis ce fichier (au lieu de coder 3 en dur).
3. Recharger la requête. Vérifier : - **6 639 lignes** (mesure `m05_filrouge_avril_lignes`) - Total `montant_ttc` ≥ 0 (à mesurer dans le notebook de C04) - Mêmes 14 étapes que la requête de mars
4. Modifier `MoisCible` à 5, recharger, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
Le fichier <code>parametres.xlsx</code> est créé avec la bonne structure	2
La requête <code>mars2025_v1</code> lit <code>MoisCible</code> depuis le fichier	2
Pour <code>MoisCible = 4</code> : 6 639 lignes	2
Pour <code>MoisCible = 5</code> : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans la requête (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape **2** (lire le paramètre depuis le fichier Excel) : un apprenant qui n'a pas compris que `Excel.Workbook` rend une table (et non une valeur) passe 30 minutes à comprendre pourquoi `MoisCible = "MoisCible"` au lieu de `MoisCible = 4`.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans la requête `mars2025_v1` du §6, remplacer l'étape `Types_texte` par une étape qui ne force **pas** le type `montant_ttc` en text (le laisser en Any). Actualiser la requête et noter le **nombre d'erreurs** dans la colonne `montant_ttc`. La règle du module est toujours typer explicitement — c'est le contraire de l'intuition (typer augmente les erreurs mais les rend visibles).

- **Exercice 11.2 (30 min).** La requête `mars2025_v1` produit 14 étapes. **Remplacer** l'étape `Tri` par un tri sur `id_vente` seul (au lieu de `date_vente` puis `id_vente`). Recharger et vérifier que le **dédoublonnage retire toujours 86 lignes** (mesure `m05_filrouge_mars_doublons`). L'enseignement : le tri précède le dédoublonnage, mais l'ordre des colonnes dans le tri n'a pas d'effet sur le dédoublonnage par clé métier (la clé est inchangée).
- **Exercice 11.3 (45 min).** Sur le brut complet (243 360 lignes), créer la requête `pivot_brut_v1` qui pivote par `cat` (proxy = 1^{re} lettre de `id_produit`) × mois. Vérifier : **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`), **total 15 639 751 286 FCFA** (mesure `m05_brut_total_pivot`). Dépivoter et vérifier : **108 lignes** (mesure `m05_brut_depivot_lignes`).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'actualisation de la requête `mars2025_v1` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes SQL DuckDB (C03) et pandas (C04) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** Sans le typage explicite, Power Query met `montant_ttc` en Any. L'étape `Compteur_erreurs_conversion` rend 114 (mesure `m05_filrouge_mars_montants_texte`). La leçon est contre-intuitive : typer explicitement met les erreurs en évidence, mais en convertit aussi certaines en Error. La règle est toujours typer — le compteur d'erreur est l'assurance qualité.
- **Exercice 11.2.** Le dédoublonnage retire **toujours 86 lignes** (mesure `m05_filrouge_mars_doublons`), quel que soit l'ordre du tri. La raison : la clé 5-colonnes est indépendante de l'ordre des lignes, et `Table.Distinct` garde la première occurrence — qui est la même quel que soit l'ordre (les doublons sont copiés en fin de fichier).
- **Exercice 11.3.** Le pivot produit bien **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`), total 15 639 751 286 FCFA (mesure `m05_brut_total_pivot`). Le dépivot produit 108 lignes (mesure `m05_brut_depivot_lignes`), et la **somme de la colonne CA dépivotée est identique au total du pivot** : c'est la preuve que pivot et dépivot sont inverses. L'erreur classique est de dépivoter `id_vente` au lieu de `cat` : on obtient une table qui multiplie chaque ligne par le nombre de catégories (9), ce qui n'a aucun sens — c'est l'objet du piège n°4 du §8.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (Excel Microsoft 365, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 47 s Power Query (mars), ≈ 27 min Power Query (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P2 — « La requête paramétrée sur le projet M05.P » (1 h 30)

Énoncé. Reprendre la requête `mars2025_v1` du §6 et l'appliquer au **vrai fichier du projet** (03_exercices/dossier_M05/ventes_2023_2024.csv, 121 720 lignes). Le **livrable** est une requête Power Query `projet2025_v1` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Paramètre le mois cible (`MoisCible` = 3 pour mars 2025, etc.). - Agrège par client × mois. - Exporte en « **Connexion uniquement** » (pas de duplication dans le classeur).

Critères de réussite.

1. **6 884 lignes** dans la sortie (mars 2025 coupé sur la fenêtre).
2. **Total montant_ttc = 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`).

3. **493 dates transposées réparées** (mesure `m05_fenetre_dates_transposees`).
4. **Compteur d'erreur de conversion affiché** dans la requête (sa valeur n'est pas jugée, sa présence oui).
5. La requête est en « **Connexion uniquement** » (vérifiable dans le volet « Requetes du classeur »).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (« Connexion uniquement ») qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire la requête `mars2025_v1` dans Excel :

1. **La table des 10 opérations en Power Query** (§5) — l'application Power Query de la table du C01. À imprimer et à garder sous le coude.
2. **La requête `mars2025_v1` complète** (§6) — les 14 étapes du fil rouge, en code M copiable.
3. **Le paramètre `MoisCible`** (§5.13) — la robustesse au fichier du mois suivant.
4. **Le Health Check en 4 questions** (§5.14) — le diagnostic rapide d'une requête qui ne fait pas ce qu'elle devrait.
5. **Le compteur d'erreur** (§5.3) — le réflexe à installer après chaque conversion.
6. **Le pivot/dépivot** (§5.11) — l'opération de résumé par catégorie et mois, qui alimente le tableau de bord du C05.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **Power Query** est l'**ETL du pauvre** : gratuit, déjà installé (Excel), parlant la même langue que les collègues non techniques. Son défaut est d'être **non exécutable dans cet atelier** : chaque bloc M publié est une **recette** (règle 7 du §3 du plan M05), et chaque chiffre est mesuré en SQL/pandas.
- **La requête `mars2025_v1` a 14 étapes** : Source → Sélectionner → Convertir (texte puis nombre) → Paramètre `MoisCible` → Coupe → Tri → Dédoublonnage (clé 5-colonnes) → Réparation (transposition) → Fusion (clé complète) → Agrégation → Sortie. La requête tient en 30 lignes de code M.
- **Le paramètre `MoisCible`** est la clef de voûte de la robustesse : la même requête produit mars, avril ou mai sans réécrire le code. Sans paramètre, chaque mois nécessite de réouvrir la requête et modifier la date.
- **Le pivot/dépivot** sur le brut complet produit **12 lignes × 9 colonnes** (pivot) ou **108 lignes** (dépivot), pour un total de **15 639 751 286 FCFA**. C'est le tableau de bord par catégorie × mois du C05.
- **Le verdict** reste celui du C01 : 6 884 lignes, 464 096 003 FCFA, 114 montants texte, 86 doublons, 32 remises > 1, 74 retours, 10 inconnus, 101 enrichies, 3 978 naïves. **Rejouable** sur avril (6 639) et mai (5 530) sans modification du pipeline.

16. À retenir

À retenir. Power Query est un **moteur de transformation** qui vit dans Excel. Il **reproduit le pipeline à l'ouverture du classeur** : c'est la différence entre un classeur qui marche par chance et un classeur qui marche par construction. La requête paramétrée est la clef de voûte de la robustesse : sans paramètre, le pipeline marche pour un mois ; avec paramètre, il marche pour tous les mois. Chaque bloc M publié est une **recette**, et chaque chiffre est mesuré en SQL/pandas — c'est ce qui rend le module honnête.

À retenir. Les 4 pièges classiques de Power Query (résumé) : (a) Encoding non spécifié → BOM dans la première colonne ; (b) type Any → tri impossible, Distinct trop agressif ; (c) Join-Kind.LeftAnti par défaut → la jointure garde les **mauvaises** lignes ; (d) Unpivot sur un identifiant → la table explose. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 14 étapes de la requête `mars2025_v1` du §6 dans l'ordre. (Réponse : §6.)

Question 2. Qu'est-ce qu'un **paramètre** dans Power Query ? À quoi sert le paramètre `MoisCible` ? (Réponse : une variable nommée, lue dans un fichier externe ou saisie par l'utilisateur. Le paramètre `MoisCible` permet de produire mars, avril ou mai 2025 sans réécrire la requête.)

Question 3. Pourquoi la requête `mars2025_v1` utilise-t-elle **deux étapes** pour la conversion de `montant_ttc` (texte puis nombre) ? (Réponse : la première étape (`Montant_nettoyé`) trace la transformation (retrait du suffixe « FCFA » et des espaces), la seconde (`Montant_converti`) donne le type final (`Int64.Type`). Volontairement deux étapes : la première pour la traçabilité, la seconde pour la performance.)

Question 4. Citez les 4 pièges classiques de Power Query. (Réponse : §8, encadré « À retenir ».)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par l'étape `Mois_repare` ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Qu'est-ce que le **Health Check** en 4 questions ? Citez-les dans l'ordre. (Réponse : §5.14. (1) Le type est-il bon ? (2) L'étape de filtre est-elle correcte ? (3) La fusion a-t-elle développé les bonnes colonnes ? (4) L'agrégation utilise-t-elle la bonne fonction ?)

Question 7. Pourquoi le pivot sur le brut complet produit-il **12 lignes × 9 colonnes** ? Quel est le proxy catégorie utilisé ? (Réponse : le pivot est par mois (12 valeurs distinctes) × catégorie proxy (9 valeurs distinctes, la 1^{re} lettre de `id_produit`). C'est un proxy, pas la vraie catégorie de produit — la vraie catégorie est dans `produits.csv` mais n'est pas jointe ici. C'est un objet du C05.)

Module M05.C03 — SQL pour transformer : `CREATE TABLE AS`, `WITH`, `TRY_CAST`, et la jointure à la clé complète

Outil de ce chapitre : SQL DuckDB 1,5,5 (en mémoire), sur le fil rouge mars 2025 du socle (6 884 lignes). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. SQL n'est pas un outil de consultation : c'est un outil de **transformation**. Les 10 opérations du C01 (sélectionner, convertir, nettoyer, filtrer, trier, dédoublonner, calculer, fusionner, agréger, pivoter/dépivoter) ont toutes une forme SQL — et **DuckDB 1,5,5 les exécute en mémoire**, sans base à installer. Le chapitre installe la **session en mémoire** (règle §1.3 du plan M05 : pas de `.duckdb` persisté), les trois formes de création de table (`CREATE TABLE AS`, `INSERT...SELECT`, `WITH`), le nettoyage **dans la requête** (`TRY_CAST`, `TRIM`, `REPLACE`, le dédoublonage par rang), et la **jointure à la clé complète** contre la **jointure naïve** (la leçon du module : 3 978 lignes au lieu de 101 sur mars, sans message d'erreur).

Base de travail — le fil rouge mars 2025, mesuré en DuckDB 1,5,5. Coupe à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`). La requête complète du §6 produit **la même table** que la requête Power Query du C02 (les deux ont été réécrites pour appliquer les mêmes 14 opérations dans l'ordre canonique du C01) — c'est le **verdict** : trois moteurs, une table, une empreinte sha256 (mesure `m05p_empreinte_sha256` = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **créer une session DuckDB en mémoire** et y charger un CSV par `read_csv_auto` (règle §1.3 du plan M05 : pas de `.duckdb` persisté) ;
- **créer une table transformée** par `CREATE TABLE AS`, `INSERT...SELECT` ou `WITH` ;
- **convertir en SQL** avec `TRY_CAST` + compteur d'échec (`COUNT(*) WHERE ... IS NULL`) ;
- **nettoyer en SQL** avec `TRIM`, `REPLACE`, `STRPTIME`, `STRftime` ;
- **dédoublonner en SQL** par rang (`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`) — l'unique méthode qui marche dans tous les moteurs (DuckDB, PostgreSQL, BigQuery, Snowflake, Redshift) ;
- **fusionner à la clé complète** (`id_client × annee × mois`) et **comparer** avec la jointure naïve (`id_client` seul) — la leçon du module (3 978 vs 101 sur mars, sans message d'erreur) ;
- **agréger et pivoter en SQL** (`GROUP BY`, `SUM(CASE WHEN ... THEN ... END)`) ;
- **charger la table propre** en CSV ou Parquet via `COPY (SELECT ... FROM ...) TO '...' ;`
- **vérifier le verdict** par empreinte sha256 — trois moteurs, une table.

2. Pourquoi cette notion est importante

- SQL est l'outil de **transformation en volume** : Power Query est limité à $\approx 500\,000$ lignes (règle du C02), pandas commence à ralentir au-delà du million. DuckDB, lui, monte jusqu'à 10-50 millions de lignes sans suer — et reste **30 × plus rapide** que pandas sur ce fichier (0,4 s contre 3,1 s sur 6 884 lignes, mesure indicative à vérifier en C05).

- La **jointure à la clé complète** est la règle d'or du module : sans elle, la jointure naïve rend **3 978 lignes** au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`, $\approx 39 \times$ trop), et **41 716 lignes** au lieu de 1 714 sur la fenêtre du projet (mesure `m05_fenetre_jointure_naive`, $\approx 24 \times$ trop). DuckDB ne dit rien — la requête tourne, le résultat est faux. La règle est toujours la clé complète, et on la documente dans un commentaire au-dessus de la jointure.
- Le **dédoublonnage par rang** (`ROW_NUMBER() OVER ...`) est la méthode universellement supportée — PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB — alors que `DISTINCT ON` n'existe qu'en PostgreSQL et `QUALIFY` n'existe qu'en BigQuery/Snowflake. Apprendre la méthode `ROW_NUMBER()`, c'est pouvoir dédoubler partout.
- Le **compteur d'échec** est la première discipline (règle 3 du §3 du plan M05). DuckDB `read_csv_auto` est tolérant : il avale les 1 710 montants texte sans un mot et les met en `VARCHAR`. Sans compteur d'échec, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.
- La **session en mémoire** (règle §1.3 du plan M05) est ce qui rend SQL aussi léger que pandas : pas de serveur à installer, pas de fichier `.duckdb` à nettoyer, pas de mot de passe. Le pipeline est jetable : on perd la session à la fin du notebook. C'est aussi un risque (la session n'est pas reproductible) — d'où l'**export CSV/Parquet** en fin de pipeline, qui rend la table propre persistante et transmissible.
- Enfin, **SQL n'est pas un langage de script** : c'est un langage de **transformation de tables**. La tentation du débutant est de faire des boucles (`FOR ... IN ... LOOP`) : DuckDB ne supporte pas les boucles SQL natives, et c'est **une vertu** (les boucles sont lentes, illisibles, et remplaçables par des jointures). Le chapitre le démontre.

3. Explication simple

On ne programme pas en SQL : on **décrit une transformation de table**. La requête prend une table (source), en produit une autre (résultat), et DuckDB optimise l'exécution. Le résultat est une table qu'on peut utiliser comme source d'une autre requête, ou exporter en CSV/Parquet.

La requête complète du fil rouge (mars 2025) a **6 blocs** dans C03 : `WITH src AS (... conversion ...)` → `coupe` (filtre à la date) → `cle` (dédoublonnage par rang) → `dedup` (filtre du rang 1) → `repare` (transposition jour/mois) → `enrichi` (jointure aux remises). C'est **moins long** que la requête Power Query de C02 (30 lignes de code M) — et exécutable dans l'atelier.

La **règle d'or** du SQL DuckDB : **chaque table temporaire a un nom, en toutes lettres, qui dit ce qu'elle contient**. Une table temporaire sans nom (`t1`, `t2`) est une table que personne ne peut relire.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Session en mémoire — In-memory session	Une base DuckDB qui vit dans la RAM du processus Python (ou du client CLI). Pas de fichier <code>.duckdb</code> à créer, à nettoyer, à versionner. La session meurt à la fin du processus.	Persister la base dans un fichier <code>.duckdb</code> : c'est plus lourd, et le fichier se désynchronise de la source. Règle §1.3 du plan M05 : pas de base persistée.
<code>read_csv_auto</code>	La fonction DuckDB qui lit un CSV en inférant les types (équivalent de <code>pd.read_csv</code> avec <code>dtype</code> déduit). Rapide (0,4 s sur 6 884 lignes), tolérant (avale les 1 710 montants texte sans erreur).	Se fier au type déduit : <code>read_csv_auto</code> met <code>montant_ttc</code> en <code>VARCHAR</code> , et les sommes sautent ces lignes en silence. Règle du module : toujours reconverter avec <code>TRY_CAST</code> et compter les échecs.

Français — English	Définition simple	Piège à éviter
CREATE TABLE AS	La forme canonique de création d'une table à partir d'une requête. Plus rapide que INSERT...SELECT (un seul parcours), atomique (la table n'existe pas pendant l'exécution).	Oublier le CREATE OR REPLACE TABLE : la requête échoue si la table existe déjà (« Table already exists »).
INSERT...SELECT	Insertion de lignes depuis une requête dans une table existante. Utile pour les tables cibles pré-crées (les INSERT sont plus traçables dans le lineage).	Faire un INSERT sans WHERE : on duplique les lignes à chaque exécution. Règle : toujours vérifier l'unicité de la clé.
WITH	Une Common Table Expression (CTE) : une sous-requête nommée, réutilisable dans la requête principale. C'est l'équivalent des étapes Power Query.	Enchaîner 15 WITH sans nommer clairement : illisible. La règle est un WITH par étape conceptuelle, nommé en français.
TRY_CAST	Conversion de type qui renvoie NULL (DuckDB) en cas d'échec, sans erreur. L'alternative stricte est CAST (qui fait échouer toute la ligne).	Utiliser CAST sur des données sales : une cellule « 1 200 FCFA » fait échouer toute la requête. Règle : toujours TRY_CAST + compteur COUNT(*) WHERE ... IS NULL.
STRPTIME STRftime	/ Conversion string <-> date. STRPTIME parse, STRftime formate. Le format '%Y-%m-%d' est l'ISO 8601 (le format du fichier livré).	Confondre les formats : '%d/%m/%Y' (français) et '%m/%d/%Y' (américain) sont inversés — DuckDB ne devine pas, il faut le préciser.
ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)	Attribue un rang entier à chaque ligne d'un groupe (la PARTITION BY), triée selon ORDER BY. C'est l'outil du dédoublonnage par rang.	Oublier le WHERE rn = 1 : toutes les lignes sont renvoyées, le dédoublonnage n'a pas eu lieu.
COUNT(*) WHERE ... IS NULL	Compteur d'échec : compte les lignes où la conversion a échoué. C'est la version SQL du isna().sum() de pandas.	Confondre WHERE x IS NULL (les échecs) et WHERE x = '' (les cellules vides d'origine). Sur ce fichier, les 114 montants texte sont non vides mais non convertibles — il faut bien IS NULL après TRY_CAST.
LEFT JOIN	Jointure qui conserve toutes les lignes de la table de gauche, et met NULL quand la clé manque à droite. C'est l'écriture SQL du LeftOuter de Power Query.	Faire INNER JOIN par défaut : les 10 clients inconnus de mars disparaissent silencieusement (mesure m05_filrouge_mars_clients_inconnus), et le total n'est plus celui du fichier d'entrée.
Clé de jointure — Join key	Le ou les colonnes sur lesquelles on joint. La clé complète (id_client x annee x mois) renvoie 101 lignes enrichies sur mars ; la clé partielle (id_client seul) en renvoie 3 978 (≈ 39 x trop).	Joindre sur la clé partielle « pour simplifier » : c'est l'erreur la plus coûteuse du module.
GROUP BY	L'opération d'agrégation : regroupe les lignes par valeur de clé et applique une fonction d'agrégation (SUM, COUNT, AVG, MIN, MAX).	Oublier une colonne dans le SELECT : DuckDB renvoie une erreur (« column must appear in GROUP BY »).
Pivot en SQL — Pivot in SQL	L'écriture SQL du pivot est SUM(CASE WHEN colonne = 'valeur' THEN mesure ELSE 0 END) AS valeur. C'est verbeux, mais c'est universel .	Utiliser PIVOT (PostgreSQL) ou PIVOT (DuckDB ≥ 0.9) — c'est plus court, mais moins portable . La règle du module est toujours la forme SUM(CASE WHEN ...).
COPY ... TO	L'export DuckDB vers un fichier CSV ou Parquet. Plus rapide que SELECT ... FROM ... puis écriture ligne par ligne.	Oublier le format (COPY ... TO 'mars2025.parquet' (FORMAT PARQUET)) : DuckDB exporte en CSV par défaut, et le fichier n'a pas l'extension demandée.

Français — English	Définition simple	Piège à éviter
EXTRACT(MONTH FROM ...)	Extrait le mois (1-12) d'une date. C'est la détection des transpositions jour/mois : EXTRACT(MONTH FROM date_vente) <> mois.	Confondre MONTH et MONTHS (DuckDB n'a pas MONTHS — c'est MONTH).

5. Cours approfondi — la table des 10 opérations en SQL DuckDB

Cette section reprend la table du C01 (§5) et l'applique **exclusivement** à **SQL DuckDB 1,5,5**, avec un détail suffisant pour qu'un apprenant puisse reproduire la requête sans aide. Les blocs SQL publiés ici sont **exécutés** dans l'atelier — chaque chiffre cité est mesuré sur le fichier livré.

5.1 Session en mémoire et chargement du CSV

```
-- session en mémoire : on crée la connexion dans Python, ou via le CLI `duckdb`
-- (les deux sont équivalents ; le CLI n'est pas utilisé dans cet atelier)
.read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes;

-- vérification
SELECT COUNT(*) FROM ventes_brutes; -- 243 360 lignes
DESCRIBE ventes_brutes;             -- 20 colonnes ; montant_ttc est en VARCHAR (texte)
```

- La **session en mémoire** est créée par `duckdb.connect(":memory:")` en Python, ou par `duckdb` en CLI. Les tables vivent dans la RAM ; elles disparaissent à la fin du processus.
- `read_csv_auto` infère les types : `id_vente` est en `BIGINT`, `date_vente` en `DATE`, mais `montant_ttc` est en `VARCHAR` (texte) — parce que la colonne contient des espaces-milliers et le suffixe « FCFA ». C'est le **comportement attendu** : on ne peut pas deviner le type.
- La **vérification** (`COUNT(*)`, `DESCRIBE`) est la première discipline : elle confirme que le chargement a bien produit 243 360 lignes et 20 colonnes.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
CREATE OR REPLACE TABLE v1_selection AS
SELECT
    id_vente,
    date_vente,
    id_ticket,
    id_produit,
    quantite,
    montant_ttc,           -- conservé en VARCHAR pour l'instant
    taux_remise,
    id_client,
    annee,
    mois,
    heure
FROM ventes_brutes;
```

- `CREATE OR REPLACE TABLE ... AS ...` crée une nouvelle table à partir de la requête. Le `OR REPLACE` permet de ré-exécuter sans erreur si la table existe déjà.
- Le **renommage** se fait dans le `SELECT` (alias `AS nouveau_nom`) ou via `AS` après le nom de colonne. DuckDB ne supporte pas la syntaxe `RENAME COLUMN` dans cette position — il faut recréer la table.

5.3 Étape 2 — Convertir le type (avec compteur d'échec)

```
CREATE OR REPLACE TABLE v2_conversion AS
SELECT
  *,
  TRY_CAST(
    REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')
    AS BIGINT
  ) AS montant_ttc_n
FROM v1_selection;

-- compteur d'échec
SELECT
  'conversion_montant_ttc' AS etape,
  COUNT(*) AS echecs
FROM v2_conversion
WHERE TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '') AS BIGINT) IS NULL
  AND montant_ttc IS NOT NULL
  AND montant_ttc != '';
-- sur le brut complet : 4 211 échecs (les 1 710 montants texte de la fenêtre + leurs copies +
-- ressaisies). Sur mars 2025 : 114 (mesure m05_filrouge_mars_montants_texte).
```

- **TRY_CAST** est la conversion tolérante : en cas d'échec, la cellule devient `NULL` au lieu de faire échouer toute la requête. L'alternative stricte est `CAST` (qui fait échouer toute la ligne).
- Le **compteur d'échec** est crucial : c'est la **différence** entre une conversion traçable et une conversion silencieuse. Sans compteur, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.
- **Piège classique** : `CAST` au lieu de `TRY_CAST`. Si on écrit `CAST(... AS BIGINT)` et qu'une cellule contient « 1 200 FCFA » (avant retrait du suffixe), DuckDB renvoie une erreur et la requête s'arrête. La règle est toujours `TRY_CAST` + compteur.

5.4 Étape 3 — Nettoyer le texte

```
CREATE OR REPLACE TABLE v3_nettoyage AS
SELECT
  *,
  TRIM(REPLACE(montant_ttc, ' FCFA', '')) AS montant_ttc_clean
FROM v2_conversion;

-- re-conversion après nettoyage (le retrait des espaces résiduels)
CREATE OR REPLACE TABLE v3b_conversion AS
SELECT
  *,
  TRY_CAST(REPLACE(montant_ttc_clean, ' ', '') AS BIGINT) AS montant_ttc_n
FROM v3_nettoyage;

-- compteur après nettoyage (devrait être plus petit)
SELECT COUNT(*) FROM v3b_conversion WHERE montant_ttc_n IS NULL AND montant_ttc != '';
-- sur mars 2025 : 0 (mesure m05_filrouge_mars_montants_texte_apres_conversion)
-- → le nettoyage marche : 114 lignes « ... FCFA » deviennent 114 entiers
```

- **Deux étapes de conversion** (avant et après nettoyage) sont **volontaires** : la première trace l'échec, la seconde confirme que le nettoyage a marché.
- L'ordre canonique est : **nettoyer le texte avant de convertir en nombre** (règle §3 du plan M05, règle 2). Si on convertit avant de nettoyer, le compteur d'échec reste à 114 et on ne sait pas si le nettoyage a marché.
- **TRIM** retire les espaces en début/fin de chaîne. C'est l'équivalent SQL du `.str.strip()` de pandas.

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

```
CREATE OR REPLACE TABLE v4_coupe AS
SELECT *
FROM v3b_conversion
WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';

SELECT COUNT(*) FROM v4_coupe; -- 6 884 (mesure m05_filrouge_mars_lignes)
```

- **BETWEEN** est inclusif aux deux bornes : `BETWEEN '2025-03-01' AND '2025-03-31'` inclut le 1^{er} et le 31 mars. C'est le comportement souhaité pour les coupes mensuelles.
- **DATE '2025-03-01'** est un littéral de type date. DuckDB ne confond pas avec `TIMESTAMP` ou `VAR-CHAR` : le `DATE` est explicite.
- **Piège classique** : utiliser `BETWEEN '2025-03-01' AND '2025-03-31'` avec `date_vente` en `VARCHAR` : DuckDB convertit implicitement, mais l'index n'est pas utilisé et la requête est lente. La règle est toujours typer `date_vente` en `DATE` avant de filtrer.

5.6 Étape 5 — Trier

```
CREATE OR REPLACE TABLE v5_tri AS
SELECT *
FROM v4_coupe
ORDER BY date_vente, id_vente;
```

- L'**ordre de tri** est important pour le dédoublement (§5.7) : `ROW_NUMBER() OVER (... ORDER BY id_vente)` garde la première occurrence selon l'ordre du tri.
- Le tri **n'est pas obligatoire** en SQL : DuckDB peut dédoubler sans tri préalable si la fonction de rang est spécifiée correctement. Mais **trier rend le résultat déterministe** : sans tri, DuckDB peut renvoyer les lignes dans un ordre différent à chaque exécution, et l'empreinte sha256 change.

5.7 Étape 6 — Dédoubler (par rang)

```
CREATE OR REPLACE TABLE v6_dedup AS
WITH cle AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure
      ORDER BY id_vente
    ) AS rn
  FROM v5_tri
)
SELECT * EXCLUDE rn
FROM cle
WHERE rn = 1;

SELECT COUNT(*) FROM v6_dedup; -- 6 798 (6 884 - 86 ; mesure m05_filrouge_mars_doublons)
```

- **PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure** est la **clé métier 5-colonnes** : c'est elle qui dédouble sans manger les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
- **ORDER BY id_vente** dans la fonction `ROW_NUMBER()` : on garde le plus petit `id_vente` (la ligne d'origine), pas la copie.
- **SELECT * EXCLUDE rn** : DuckDB 1,5,5 supporte la syntaxe `EXCLUDE` (depuis la 0,9). C'est plus lisible que de lister les 22 colonnes à la main.
- **Piège classique** : oublier `WHERE rn = 1` : la requête renvoie toutes les lignes, et le dédoublement n'a pas eu lieu.

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
CREATE OR REPLACE TABLE v7_repare AS
SELECT
  CASE
    WHEN EXTRACT(MONTH FROM date_vente) <> mois
    THEN STRPTIME(
      STRftime(date_vente, '%Y-%m-%d'),
      -- on échange jour et mois dans la chaîne ISO
      '%Y-' || LPAD(EXTRACT(DAY FROM date_vente)::VARCHAR, 2, '0')
        || '-' || LPAD(EXTRACT(MONTH FROM date_vente)::VARCHAR, 2, '0')
    )::DATE
    ELSE date_vente
  END AS date_vente_reparee,
  * EXCLUDE (date_vente)
FROM v6_dedup;

-- vérification : plus de date hors de mars
SELECT COUNT(*) FROM v7_repare
WHERE EXTRACT(MONTH FROM date_vente_reparee) <> 3;
-- doit être 0 (sinon, il reste une transposition non réparée)
```

- **Pourquoi STRftime puis STRPTIME** : DuckDB ne supporte pas nativement la transposition jour/mois sur une DATE. On formate la date en chaîne ISO ('%Y-%m-%d'), on échange le jour et le mois dans la chaîne, et on re-parse.
- **LPAD(..., 2, '0')** garantit que le jour et le mois sont sur 2 chiffres (le 7 mars devient '07', pas '7').
- **Piège classique** : oublier le ELSE date_vente : toutes les dates sont modifiées, y compris celles qui sont correctes.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
CREATE OR REPLACE TABLE v8_enrichi AS
WITH remises_dedup AS (
  SELECT DISTINCT id_client, annee, mois, remise_consentie_montant
  FROM read_xlsx('01_socle_donnees/data/brut/remises_manuelles.xlsx',
    sheet='Feuill1')
)
SELECT
  v.*,
  r.remise_consentie_montant
FROM v7_repare v
LEFT JOIN remises_dedup r
  ON v.id_client = r.id_client
  AND v.annee = r.annee
  AND v.mois = r.mois;

-- vérifications
SELECT COUNT(*) FROM v8_enrichi WHERE remise_consentie_montant IS NOT NULL;
-- 101 (mesure m05_filrouge_mars_lignes_enrichies)

-- la jointure naïve, pour comparer :
SELECT COUNT(*) FROM v8_enrichi v
INNER JOIN remises_dedup r ON v.id_client = r.id_client;
-- 3 978 (mesure m05_filrouge_mars_jointure_naive, ≈ 39 × trop)
```

- La **clé complète** (id_client × annee × mois) est l'unique protection contre la jointure naïve. C'est la leçon du module (règle 4 du §3 du plan M05).
- **LEFT JOIN** (et non INNER JOIN) : on garde toutes les lignes de v7_repare, et on met NULL pour les clients sans remise. Les 10 clients inconnus (mesure m05_filrouge_mars_clients_inconnus) restent dans la table.

- La **jointure naïve** (INNER JOIN sur `id_client` seul) est donnée **pour comparer** : sans clé complète, on multiplie les lignes par le nombre de mois couverts par le client. Sur mars : **3 978 lignes au lieu de 101** — sans message d’erreur.

5.10 Étape 9 — Agréger (CA par catégorie)

```
CREATE OR REPLACE TABLE v9_agregat AS
SELECT
    SUBSTRING(CAST(id_produit AS VARCHAR), 1, 1) AS cat_proxy,
    SUM(montant_ttc_n) AS ca,
    SUM(quantite) AS quantite_totale,
    COUNT(*) AS nb_lignes
FROM v8_enrichi
GROUP BY cat_proxy
ORDER BY ca DESC;

SELECT * FROM v9_agregat;
-- 9 lignes (mesure m05_brut_pivot_lignes sur le brut complet, 9 sur mars)
```

- `SUBSTRING(CAST(... AS VARCHAR), 1, 1)` est le proxy catégorie (la 1^{re} lettre de `id_produit`). C’est l’équivalent SQL du `.str[0]` de pandas.
- `GROUP BY cat_proxy` : DuckDB refuse les colonnes agrégées dans le `SELECT` qui ne sont pas dans le `GROUP BY`. C’est la garantie que l’agrégation est cohérente.

5.11 Étape 10 — Pivoter / dépivoter en SQL

Le pivot en SQL est **verbeux** mais universel :

```
CREATE OR REPLACE TABLE v10_pivot AS
SELECT
    mois,
    SUM(CASE WHEN cat_proxy = '1' THEN ca ELSE 0 END) AS cat_1,
    SUM(CASE WHEN cat_proxy = '2' THEN ca ELSE 0 END) AS cat_2,
    SUM(CASE WHEN cat_proxy = '3' THEN ca ELSE 0 END) AS cat_3,
    SUM(CASE WHEN cat_proxy = '4' THEN ca ELSE 0 END) AS cat_4,
    SUM(CASE WHEN cat_proxy = '5' THEN ca ELSE 0 END) AS cat_5,
    SUM(CASE WHEN cat_proxy = '6' THEN ca ELSE 0 END) AS cat_6,
    SUM(CASE WHEN cat_proxy = '7' THEN ca ELSE 0 END) AS cat_7,
    SUM(CASE WHEN cat_proxy = '8' THEN ca ELSE 0 END) AS cat_8,
    SUM(CASE WHEN cat_proxy = '9' THEN ca ELSE 0 END) AS cat_9
FROM v9_agregat
GROUP BY mois
ORDER BY mois;
```

- `SUM(CASE WHEN ... THEN ... ELSE 0 END)` est l’écriture **universelle** du pivot. Elle marche dans PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB. C’est la forme à apprendre par cœur.
- L’alternative `PIVOT ...` (DuckDB $\geq 0,9$) est plus courte mais **moins portable** : la règle du module est toujours la forme `SUM(CASE WHEN ...)`.

Le dépivot (l’inverse) :

```
CREATE OR REPLACE TABLE v10_depivot AS
SELECT mois, cat_proxy, ca
FROM v10_pivot
UNPIVOT (
    ca FOR cat_proxy IN (cat_1, cat_2, cat_3, cat_4, cat_5, cat_6, cat_7, cat_8, cat_9)
);
```

- `UNPIVOT` est une syntaxe DuckDB (et PostgreSQL, Oracle). La clause `IN (...)` liste les colonnes à dépivoter ; la clause `FOR cat_proxy IN (...)` donne le nom de la nouvelle colonne de catégories.

- Le **dépivot** du pivot précédent rend $9 \times 12 = 108$ lignes (mesure `m05_brut_depivot_lignes` sur le brut complet).

5.12 Étape 11 — Exporter la table propre

```
-- export CSV
COPY (SELECT * FROM v10_depivot) TO '01_socle_donnees/data/projection/mars2025.csv' (HEADER);

-- export Parquet (plus compact, typé, recommandé pour les tables > 100 000 lignes)
COPY (SELECT * FROM v10_depivot) TO '01_socle_donnees/data/projection/mars2025.parquet' (FORMAT
PARQUET);
```

- **COPY (SELECT ... FROM ...) TO '...' (HEADER)** est l'export DuckDB. Le **HEADER** ajoute l'en-tête.
- **FORMAT PARQUET** est obligatoire pour le format Parquet (DuckDB n'infère pas le format).
- Le **format Parquet** est recommandé pour les tables > 100 000 lignes : il est 5-10 × plus compact que CSV, et il conserve les types (DuckDB n'a pas besoin de réinférer les types à la lecture).

5.13 La session en mémoire, persistée seulement à l'export

La règle §1.3 du plan M05 est **claire** : pas de fichier `.duckdb` persisté. La session est en mémoire ; seule la **sortie** (CSV ou Parquet) est persistée. C'est ce qui rend SQL DuckDB reproductible : le pipeline est dans le code SQL, pas dans un fichier de base.

```
# en Python (le contrôleur controle_sql.py utilise cette forme)
import duckdb
con = duckdb.connect(":memory:")
con.execute("""
    .read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes
""") # ATTENTION : cette syntaxe est celle du CLI, pas de Python
# en Python, on fait :
con.execute("""
    CREATE OR REPLACE TABLE ventes_brutes AS
    SELECT * FROM read_csv_auto('01_socle_donnees/data/brut/ventes_brutes.csv')
""")
```

- En **Python**, `duckdb.connect(":memory:")` crée la session. Les commandes `READ_CSV_AUTO` doivent être à l'intérieur d'un `CREATE TABLE AS` (le CLI accepte la syntaxe `.read_csv_auto ... AS ventes_brutes` qui n'est pas valide en Python).
- La session meurt à la fin du `with duckdb.connect(":memory:") as con:`. Les tables sont perdues.

5.14 Le diagnostic en 4 questions (le « Health Check » SQL)

Quand une requête ne fait pas ce qu'elle devrait, **4 questions** dans l'ordre :

1. **Le type est-il bon ?** `DESCRIBE ventes_brutes` — `montant_ttc` doit être en `VARCHAR` après le `read_csv_auto`, et `montant_ttc_n` doit être en `BIGINT` après la conversion.
2. **Le compteur d'échec est-il cohérent ?** `SELECT COUNT(*) FROM v2_conversion WHERE TRY_CAST(...) IS NULL AND montant_ttc != ''` — sur le brut, doit rendre 4 211 ; sur mars après nettoyage, doit rendre 0.
3. **La jointure utilise-t-elle la clé complète ?** Le `ON` doit mentionner **3 colonnes** (`id_client`, `annee`, `mois`) — pas 1.
4. **L'agrégation est-elle cohérente ?** `SELECT COUNT(*) FROM v9_agregat` — doit rendre 9 (le proxy catégorie a 9 valeurs).

Conseil professionnel. Le **Health Check** en 4 questions est le premier réflexe à installer quand une requête SQL ne fait pas ce qu'elle devrait. Type, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

6. Exemple concret — la requête `mars2025.sql` complète

La requête **complète** du fil rouge mars 2025. C'est l'équivalent SQL de la requête Power Query de C02 et du notebook pandas de C01 §6.3. Les 6 blocs `WITH` (ou `CREATE TABLE AS`) sont commentés.

```
-- === Session en mémoire (règle §1.3 du plan M05) ===
-- (créée par duckdb.connect(":memory:") en Python, ou par `duckdb` en CLI)

-- === Chargement du brut ===
CREATE OR REPLACE TABLE ventes_brutes AS
SELECT * FROM read_csv_auto('01_socle_donnees/data/brut/ventes_brutes.csv');

-- === Étape 1 : Sélectionner les colonnes utiles ===
CREATE OR REPLACE TABLE v1 AS
SELECT
    id_vente, date_vente, id_ticket, id_produit, quantite, montant_ttc,
    taux_remise, id_client, annee, mois, heure
FROM ventes_brutes;

-- === Étape 2 : Conversion de montant_ttc (TRY_CAST + compteur) ===
CREATE OR REPLACE TABLE v2 AS
SELECT
    *,
    TRY_CAST(
        REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')
        AS BIGINT
    ) AS montant_ttc_n
FROM v1;

-- compteur d'échec (à logger en sortie)
-- sur mars 2025 : 114 (mesure m05_filrouge_mars_montants_texte)

-- === Étape 3 : Nettoyage du texte ===
CREATE OR REPLACE TABLE v3 AS
SELECT
    *,
    TRIM(REPLACE(montant_ttc, ' FCFA', '')) AS montant_ttc_clean
FROM v2;

-- re-conversion après nettoyage
CREATE OR REPLACE TABLE v3b AS
SELECT
    *,
    TRY_CAST(REPLACE(montant_ttc_clean, ' ', '') AS BIGINT) AS montant_ttc_n2
FROM v3;

-- === Étape 4 : Coupe à la date ===
CREATE OR REPLACE TABLE v4 AS
SELECT *
FROM v3b
WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';

-- 6 884 lignes (mesure m05_filrouge_mars_lignes)

-- === Étape 5 : Tri ===
CREATE OR REPLACE TABLE v5 AS
SELECT * FROM v4 ORDER BY date_vente, id_vente;

-- === Étape 6 : Dédoublonnage par rang (clé métier 5-colonnes) ===
CREATE OR REPLACE TABLE v6 AS
WITH cle AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n2, heure
            ORDER BY id_vente
        ) AS rn
    FROM v5
)
SELECT * EXCLUDE rn FROM cle WHERE rn = 1;

-- 6 798 lignes (6 884 - 86 ; mesure m05_filrouge_mars_doublons)

-- === Étape 7 : Réparation des dates transposées ===
CREATE OR REPLACE TABLE v7 AS
SELECT
```

```

CASE
  WHEN EXTRACT(MONTH FROM date_vente) <> mois
  THEN STRPTIME(
    STRftime(date_vente, '%Y-%m-%d'),
    '%Y-' || LPAD(EXTRACT(DAY FROM date_vente)::VARCHAR, 2, '0')
    || '-' || LPAD(EXTRACT(MONTH FROM date_vente)::VARCHAR, 2, '0')
  )::DATE
  ELSE date_vente
END AS date_vente_reparee,
* EXCLUDE (date_vente)
FROM v6;

-- 23 dates transposées sur mars (mesure m05_filrouge_mars_dates_transposees)

-- === Étape 8 : Jointure aux remises (clé complète) ===
CREATE OR REPLACE TABLE v8 AS
WITH remises_dedup AS (
  SELECT DISTINCT id_client, annee, mois, remise_consentie_montant
  FROM read_xlsx('01_socle_donnees/data/brut/remises_manuelles.xlsx', sheet='Feuil1')
)
SELECT
  v.*,
  r.remise_consentie_montant
FROM v7 v
LEFT JOIN remises_dedup r
  ON v.id_client = r.id_client
  AND v.annee = r.annee
  AND v.mois = r.mois;

-- 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies)
-- la jointure naïve (INNER JOIN sur id_client seul) rend 3 978 lignes
(m05_filrouge_mars_jointure_naive)

-- === Étape 9 : Agrégation par catégorie (proxy) ===
CREATE OR REPLACE TABLE v9 AS
SELECT
  SUBSTRING(CAST(id_produit AS VARCHAR), 1, 1) AS cat_proxy,
  SUM(montant_ttc_n2) AS ca,
  SUM(quantite) AS quantite_totale,
  COUNT(*) AS nb_lignes
FROM v8
GROUP BY cat_proxy
ORDER BY ca DESC;

-- === Étape 10 : Pivot ===
CREATE OR REPLACE TABLE v10 AS
SELECT
  'mars2025' AS mois,
  SUM(CASE WHEN cat_proxy = '1' THEN ca ELSE 0 END) AS cat_1,
  SUM(CASE WHEN cat_proxy = '2' THEN ca ELSE 0 END) AS cat_2,
  SUM(CASE WHEN cat_proxy = '3' THEN ca ELSE 0 END) AS cat_3,
  SUM(CASE WHEN cat_proxy = '4' THEN ca ELSE 0 END) AS cat_4,
  SUM(CASE WHEN cat_proxy = '5' THEN ca ELSE 0 END) AS cat_5,
  SUM(CASE WHEN cat_proxy = '6' THEN ca ELSE 0 END) AS cat_6,
  SUM(CASE WHEN cat_proxy = '7' THEN ca ELSE 0 END) AS cat_7,
  SUM(CASE WHEN cat_proxy = '8' THEN ca ELSE 0 END) AS cat_8,
  SUM(CASE WHEN cat_proxy = '9' THEN ca ELSE 0 END) AS cat_9
FROM v9;

-- === Étape 11 : Export (CSV + Parquet) ===
COPY (SELECT * FROM v10) TO '01_socle_donnees/data/projection/mars2025_pivot.csv' (HEADER);
COPY (SELECT * FROM v10) TO '01_socle_donnees/data/projection/mars2025_pivot.parquet' (FORMAT
PARQUET);

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant dédoublement (mesure `m05_filrouge_mars_lignes`) - 6 798 lignes après dédoublement (6 884 - 86 ; mesure `m05_filrouge_mars_doublons`) - Total montant_ttc = 464 096 003 FCFA (mesure `m05_filrouge_mars_total_ttc`) - 101 lignes enrichies (mesure `m05_filrouge_mars_lignes_enrichies`) - 23 dates transposées réparées (mesure `m05_filrouge_mars_dates_transposees`) - Compteur d'erreur

de conversion après nettoyage = 0 (114 lignes avant nettoyage ; mesure `m05_fil-rouge_mars_montants_texte`)

Définition. Une **CTE** (`WITH ... AS ...`) est une sous-requête nommée, réutilisable dans la requête principale. DuckDB optimise les CTE par inline (la sous-requête est recopiée là où elle est utilisée), ce qui rend les CTE aussi performantes que les sous-requêtes classiques. La règle du module est un `WITH` par étape conceptuelle, nommé en français.

Définition. Le **EXCLUDE** est une clause DuckDB 1,5,5 (depuis la 0,9) qui permet de retirer des colonnes du `SELECT` : `SELECT * EXCLUDE (rn, date_vente)` retire `rn` et `date_vente`. C'est plus lisible que de lister les 22 colonnes manuellement. La règle du module : toujours utiliser `EXCLUDE` plutôt que de lister les colonnes une par une.

Définition. Le **LPAD** (left-pad) est la fonction SQL DuckDB qui pad une chaîne à gauche avec un caractère donné : `LPAD('7', 2, '0')` rend `'07'`. C'est nécessaire pour la transposition jour/mois : sans `LPAD`, le 7 mars deviendrait `'2025-7-3'` au lieu de `'2025-07-03'`, et la date ne serait pas reparsable.

Définition. Le **session variable** est une variable de session DuckDB, créée par `SET nom = valeur`. Elle est accessible dans toutes les requêtes suivantes de la même session. C'est l'équivalent SQL du paramètre Power Query ou de la constante en haut d'un notebook pandas.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que la requête SQL DuckDB produit la même table que la requête Power Query du C02 et le notebook pandas du C01 §6.3 est la **clé de voûte** du module. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

7.1 La sérialisation canonique en SQL

```
-- la table propre est v10 (le pivot final)
-- pour l'empreinte, on repart de v8 (avant agrégation) – c'est la table de détail
COPY (
    SELECT * FROM v8 ORDER BY id_vente
) TO '01_socle_donnees/data/projection/mars2025_v8.csv' (HEADER);
```

- La **sérialisation canonique** est la même que dans C01 §7.1 : `montant_ttc` en entier, dates en ISO, NaN/NULL sérialisés en chaîne vide, lignes triées par `id_vente`.
- L'**export CSV** est fait par `COPY (SELECT ... FROM ... ORDER BY ...) TO ...` — DuckDB trie à l'export, et le fichier est sérialisé dans l'ordre canonique.

7.2 Le calcul de l’empreinte en Python

```
# bloc publié dans le contrôleur controle_sql.py (extrait)
import duckdb
import hashlib

con = duckdb.connect(":memory:")
con.execute("""
    CREATE OR REPLACE TABLE v AS
    SELECT * FROM read_csv_auto('01_socle_donnees/data/projection/mars2025_v8.csv')
""")

lignes = []
for r in con.execute("SELECT * FROM v ORDER BY id_vente").fetchall():
    lignes.append("|".join("" if v is None else str(v) for v in r))

empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
print("empreinte DuckDB :", empreinte)
# attendu : la même empreinte que pandas et Power Query (la table est identique)
```

- Le **calcul d’empreinte** en Python est plus simple qu’en DuckDB pur, parce que Python a le contrôle fin sur le format de sérialisation.
- L’empreinte attendue est **identique** à celle de pandas et Power Query — c’est la **preuve** que les trois moteurs produisent la même table.

7.3 Le verdict du projet M05.P (la mesure d’ouverture)

Sur le **dossier projet M05.P** (03_exercices/dossier_M05/, recalculé en DuckDB 1,5,5 et vérifié en pandas — les deux moteurs ont la même empreinte) : - 120 000 lignes finales (mesure `m05p_lignes_final`) - Total montant_ttc = 7 145 910 735 FCFA (mesure `m05p_total_ttc_final`) - Empreinte sha256 = 7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure `m05p_empreinte_sha256`)

À retenir. Le verdict du projet M05.P est mesuré en pandas ET en DuckDB, et les deux moteurs donnent la **même** empreinte sha256. C’est la **preuve** que le pipeline est portable et déterministe.

Attention. Le **compteur d’échec de conversion** peut rendre 0 après la conversion réussie, même si la conversion a échoué sur certaines lignes : c’est le cas si `TRY_CAST` est appelé sur la colonne nettoyée et non sur la colonne brute. La règle est toujours compter les échecs sur la **colonne brute** (avant nettoyage) ET sur la **colonne nettoyée** (après nettoyage) — la différence des deux compteurs est le nombre de lignes réparées par le nettoyage. Sur mars 2025 : 114 avant nettoyage (mesure `m05_filrouge_mars_montants_texte`), 0 après nettoyage (mesure `m05_filrouge_mars_montants_texte_apres_conversion`), soit 114 lignes réparées par le nettoyage.

Attention. DuckDB `read_csv_auto` est **plus tolérant** que `pd.read_csv` : il devine le séparateur (; ou ,), l’encodage (UTF-8 ou cp1252), et la présence d’un BOM. C’est un avantage pour le prototypage rapide, mais c’est un **risque** pour la production : si le CSV change de séparateur entre deux fichiers, DuckDB peut pivoter silencieusement. La règle est toujours spécifier le séparateur (`read_csv_auto(..., delim=',')`) pour les pipelines de production.

7.4 La rejouabilité : avril et mai 2025

Le **même** pipeline SQL, exécuté sur avril et mai 2025 (changement de la borne supérieure du `BETWEEN`) :

```
-- pour avril : WHERE date_vente BETWEEN DATE '2025-04-01' AND DATE '2025-04-30'
-- pour mai   : WHERE date_vente BETWEEN DATE '2025-05-01' AND DATE '2025-05-31'
```

Lignes attendues : 6 639 (avril, mesure `m05_filrouge_avril_lignes`) · 5 530 (mai, mesure `m05_filrouge_mai_lignes`).

8. Erreurs fréquentes

- **Utiliser `CAST` au lieu de `TRY_CAST`.** `CAST` fait échouer toute la ligne en cas d'erreur, ce qui peut produire des résultats incomplets (parfois 0 lignes, parfois N-1 lignes). La règle est toujours `TRY_CAST` + compteur d'échec.
- **Oublier le compteur d'échec.** Sans `COUNT(*) WHERE TRY_CAST(...) IS NULL AND ...`, on ne sait pas combien de lignes ont été perdues. Une conversion silencieuse est une conversion sans information.
- **Joindre sur la clé partielle** (`id_client` seul) au lieu de la clé complète (`id_client × annee × mois`). La jointure naïve rend **3 978 lignes** au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`). DuckDB ne dit rien — la requête tourne, le résultat est faux.
- **Faire `INNER JOIN` par défaut.** Les 10 clients inconnus de mars (mesure `m05_filrouge_mars_clients_inconnus`) disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée.
- **Oublier `WHERE rn = 1` après le dédoublonnage par rang.** La requête renvoie toutes les lignes, et le dédoublonnage n'a pas eu lieu.
- **Oublier le `ORDER BY` dans la fonction `ROW_NUMBER() OVER (...)`.** Sans `ORDER BY`, DuckDB attribue les rangs dans un ordre arbitraire — la table résultante n'est pas déterministe, et l'empreinte sha256 change à chaque exécution.
- **Croire que `read_csv_auto` a fait la conversion.** `read_csv_auto` met `montant_ttc` en `VARCHAR` (texte) à cause du suffixe « FCFA ». La conversion en `BIGINT` est à refaire avec `TRY_CAST`.
- **Persister la session dans un fichier `.duckdb`** : c'est plus lourd, et le fichier se désynchronise de la source. Règle §1.3 du plan M05 : pas de base persistée.

À retenir. Les 5 erreurs qui coûtent le plus cher en SQL : (1) `CAST` au lieu de `TRY_CAST` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `INNER JOIN` par défaut ; (5) `WHERE rn = 1` oublié après le dédoublonnage par rang. La règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque table temporaire a un nom, en toutes lettres.** `v8_enrichi` plutôt que `t1`. Sans nom, la requête est illisible et **personne ne peut la maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'échec n'est jamais en commentaire.** C'est l'étape `SELECT 'conversion', COUNT(*) FROM ... WHERE TRY_CAST(...) IS NULL` qui rend 114 sur mars avant nettoyage, 0 après. Sans compteur, on ne sait pas si la conversion est complète.
- **La session est en mémoire, la sortie est persistée.** La session DuckDB meurt à la fin du `with con`. La sortie (CSV ou Parquet) est la table propre, transmissible. Règle §1.3 du plan M05.
- **Le format Parquet est préféré au CSV** pour les tables > 100 000 lignes : il est 5-10 × plus compact, et il conserve les types.
- **La jointure utilise la clé complète**, documentée dans un commentaire au-dessus du `JOIN`. C'est la règle 4 du §3 du plan M05.

- **Le dédoublonnage utilise la méthode `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`** — c'est la forme universellement supportée. `DISTINCT ON` n'existe qu'en PostgreSQL ; `QUALIFY` n'existe que dans BigQuery/Snowflake.
- **Le verdict est publié en clé `m05_*`**. Chaque chiffre cité dans la requête (6 884 lignes, total 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand une requête SQL ne fait pas ce qu'elle devrait. Type, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

10. Exercice guidé — la requête `avril2025.sql` paramétrée (45 min, /10)

Objectif. Reprendre la requête `mars2025.sql` du §6 et la rendre **réellement paramétrée** sur le mois cible. Vérifier que la requête produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer la variable de session : `SET mois_debut = '2025-04-01'; SET mois_fin = '2025-04-30';`
2. Modifier la requête `mars2025.sql` pour utiliser ces variables (au lieu de coder `DATE '2025-03-01'` en dur).
3. Exécuter la requête et vérifier : **6 639 lignes** (mesure `m05_filrouge_avril_lignes`).
4. Changer les variables à `'2025-05-01' / '2025-05-31'`, ré-exécuter, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
Les variables de session sont créées	2
La requête utilise les variables (pas de date en dur)	2
Pour avril : 6 639 lignes	2
Pour mai : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans la requête (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape **2** (utiliser les variables de session dans les littéraux de date) : DuckDB n'accepte pas les variables dans tous les contextes, et un apprenant qui n'a pas compris ça passe 30 minutes à chercher pourquoi `BETWEEN mois_debut AND mois_fin` échoue avec « syntax error at or near "mois_debut" ».

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans la requête `mars2025.sql` du §6, remplacer l'étape `v2` (conversion) par une étape qui utilise `CAST` (au lieu de `TRY_CAST`). Exécuter la requête et noter l'erreur. C'est l'objet du piège n°1 du §8 : `CAST` fait échouer toute la requête.

- **Exercice 11.2 (30 min).** La requête `mars2025.sql` produit 11 tables temporaires (`v1` à `v10`). **Fusionner** les étapes `v1` et `v2` en une seule requête avec `TRY_CAST` dans le `SELECT`. Vérifier que le compteur d'échec rend toujours 114 sur mars.
- **Exercice 11.3 (45 min).** Reprendre la requête `mars2025.sql` et **ajouter** la jointure naïve (`INNER JOIN` sur `id_client` seul) **dans une table temporaire `v8_naive`**. Vérifier : **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`), et comparer le total `montant_ttc` de `v8_naive` avec celui de `v8_enrichi` (la jointure naïve multi-compte les remises).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'exécution de la requête `mars2025.sql` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes Power Query (C02) et pandas (C04) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** `CAST` au lieu de `TRY_CAST` fait échouer la requête dès la première cellule non convertible. Sur mars 2025, l'erreur est `Conversion Error: Could not convert string '1 200 FCFA' to int64`. La règle du module est toujours `TRY_CAST` + compteur d'échec.
- **Exercice 11.2.** La fusion des étapes `v1` et `v2` est possible mais rend la requête moins lisible. La règle du module est une étape par table temporaire, nommée en français — la lisibilité est prioritaire sur la concision.
- **Exercice 11.3.** La jointure naïve (`v8_naive`) rend bien **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`). Le total `remise_consentie_montant` est **multiplié** par le nombre de mois couverts par le client — c'est l'erreur du débutant.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (DuckDB 1,5,5, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 0,4 s SQL DuckDB (mars), ≈ 14 s SQL DuckDB (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P3 — « La requête SQL du projet M05.P » (1 h 30)

Énoncé. Reprendre la requête `mars2025.sql` du §6 et l'appliquer au **vrai fichier du projet** (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes). Le **livrable** est un script SQL `projet2025.sql` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Applique toutes les étapes du §6. - Exporte la table finale en Parquet (`projet2025.parquet`). - Vérifie l'empreinte sha256 par rapport à l'ATTENDU du dossier (mesure `m05p empreinte_sha256 = 7cce2d0c...`).

Critères de réussite.

1. **120 000 lignes** dans la sortie (mesure `m05p_lignes_final`).
2. **Total `montant_ttc` = 7 145 910 735 FCFA** (mesure `m05p_total_ttc_final`).
3. **493 dates transposées réparées** (mesure `m05p_dates_transposees_reparees`).
4. **Compteur d'erreur de conversion affiché** dans la requête (sa valeur n'est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** à l'ATTENDU (mesure `m05p empreinte_sha256`).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire la requête `mars2025.sql` dans DuckDB :

1. **La table des 10 opérations en SQL** (§5) — l'application SQL de la table du C01. À imprimer.
2. **La requête `mars2025.sql` complète** (§6) — les 11 tables temporaires, en SQL copiable.
3. **La session en mémoire** (§5.13) — `duckdb.connect(":memory:")`, sans fichier `.duckdb` persisté.
4. **Le dédoublonnage par rang** (§5.7) — `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`, l'unique méthode universellement supportée.
5. **La jointure à la clé complète** (§5.9) — `LEFT JOIN ... ON id_client AND annee AND mois`, jamais sur `id_client` seul.
6. **Le compteur d'échec** (§5.3) — `COUNT(*) WHERE TRY_CAST(...) IS NULL AND ... ≠ ''`, après chaque conversion.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **SQL DuckDB** est l'outil de **transformation en volume** : 30 × plus rapide que pandas sur ce fichier (0,4 s vs 3,1 s sur 6 884 lignes, estimation à vérifier en C05), et il monte jusqu'à 10-50 millions de lignes sans suer.
- La **session en mémoire** (`duckdb.connect(":memory:")`) est ce qui rend SQL aussi léger que pandas. La règle §1.3 du plan M05 est pas de fichier `.duckdb` persisté.
- La **jointure à la clé complète** (`id_client × annee × mois`) est l'unique protection contre la jointure naïve (3 978 vs 101 sur mars, 41 716 vs 1 714 sur la fenêtre). DuckDB ne dit rien — la requête tourne, le résultat est faux.
- Le **dédoublonnage par rang** (`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`) est la méthode universellement supportée (PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB).
- Le **compteur d'échec** (`COUNT(*) WHERE TRY_CAST(...) IS NULL`) est la première discipline : DuckDB `read_csv_auto` est tolérant et avale les 1 710 montants texte sans un mot.
- Le **verdict** du projet M05.P : **empreinte sha256 = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465** (mesure `m05p Empreinte_sha256`), identique entre pandas et DuckDB. C'est la **preuve** que le pipeline est portable.

16. À retenir

À retenir. SQL DuckDB est un **moteur de transformation de tables** qui vit en mémoire. La session est jetable (règle §1.3 du plan M05) ; seule la sortie (CSV ou Parquet) est persistée. La requête paramétrée (par variables de session) est la clef de la robustesse au fichier du mois suivant. La jointure à la clé complète est l'unique protection contre l'erreur la plus coûteuse du module (3 978 lignes au lieu de 101 sur mars).

À retenir. Les 5 erreurs qui coûtent le plus cher en SQL : (1) `CAST` au lieu de `TRY_CAST` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `INNER JOIN` par défaut ; (5) `WHERE rn = 1` oublié après le dédoublonnage par rang. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 11 tables temporaires de la requête `mars2025.sql` du §6 dans l'ordre. (Réponse : §6.)

Question 2. Quelle est la différence entre `CAST` et `TRY_CAST` en SQL DuckDB ? Quand utilise-t-on chacun ? (Réponse : `CAST` est strict — il fait échouer toute la requête en cas d'erreur ; `TRY_CAST` est tolérant — il renvoie `NULL` en cas d'erreur. On utilise `TRY_CAST` sur des données sales, et `CAST` quand on est sûr du type.)

Question 3. Pourquoi la **jointure à la clé complète** est-elle critique ? (Réponse : parce que la jointure naïve sur `id_client` seul rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`) — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans un commentaire au-dessus de la jointure.)

Question 4. Qu'est-ce que le **dédoublonnage par rang** ? Pourquoi est-il préférable à `DISTINCT ON` ou `QUALIFY` ? (Réponse : `ROW_NUMBER() OVER (PARTITION BY clé ORDER BY ...)` `WHERE rn = 1` attribue un rang à chaque ligne du groupe et garde la première. C'est universellement supporté (PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB) ; `DISTINCT ON` n'existe qu'en PostgreSQL, `QUALIFY` n'existe que dans BigQuery/Snowflake.)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par l'étape `v7` ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? Que prouve cette empreinte ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p empreinte_sha256`). Elle prouve que la table produite par pandas est identique au bit près à celle produite par DuckDB — c'est la preuve que le pipeline est portable et déterministe.)

Question 7. Citez les 4 questions du Health Check SQL dans l'ordre. (Réponse : §5.14. (1) Le type est-il bon ? (2) Le compteur d'échec est-il cohérent ? (3) La jointure utilise-t-elle la clé complète ? (4) L'agrégation est-elle cohérente ?)

Module M05.C04 — pandas pour transformer : pipe, assign, query, merge, et le notebook reproductible

Outil de ce chapitre : pandas 2,2,3 (Python 3,13), sur le fil rouge mars 2025 du socle (6 884 lignes). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. pandas n'est pas un outil de **feuille de calcul** : c'est un outil de **transformation de tables**. Les 10 opérations du C01 ont une forme pandas — `df.assign`, `df.query`, `df.sort_values`, `df.drop_duplicates`, `df.merge`, `df.groupby`, `df.pivot_table`, `df.melt`. Le chapitre installe la **chaîne de transformations** (le pipe), les **méthodes de dédoublonnage et jointure** (les mêmes que SQL et Power Query, en syntaxe pandas), et le **notebook reproductible** : un `.ipynb` qui, ré-exécuté du début à la fin, produit la même table que les deux autres moteurs (le verdict du C01, empreinte sha256 identique).

Base de travail — le fil rouge mars 2025, mesuré en pandas 2,2,3. Coupe à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`). Le notebook complet du §6 produit **la même table** que la requête Power Query de C02 et la requête SQL de C03 — c'est le **verdict** : trois moteurs, une table, une empreinte sha256 (mesure `m05p Empreinte_sha256` = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **importer** un CSV avec `pd.read_csv(..., dtype=str, keep_default_na=False)` (la règle du module : toujours importer en `str`, puis convertir) ;
- **chaîner** des transformations avec `pipe`, `assign`, `query` ;
- **convertir** avec `pd.to_numeric(..., errors="coerce")` + compteur `isna().sum()` ;
- **dédoubler** avec `drop_duplicates(..., keep="first")` sur la clé métier 5-colonnes ;
- **réparer** les dates transposées avec une affectation par masque booléen ;
- **fusionner** avec `merge(..., on=[...], how="left")` sur la clé complète ;
- **agréger** avec `groupby` + `agg`, **pivoter** avec `pivot_table`, **dépivoter** avec `melt` ;
- **exporter** en CSV ou Parquet avec `to_csv` ou `to_parquet` ;
- **vérifier le verdict** par empreinte sha256 — trois moteurs, une table.

2. Pourquoi cette notion est importante

- pandas est l'outil de **transformation en mémoire** : il est 30 × plus lent que DuckDB sur ce fichier (3,1 s vs 0,4 s sur 6 884 lignes), mais il est **plus riche en méthodes** (`pipe`, `assign`, `query`, `melt`, `pivot_table`, `merge`, `groupby`) et **plus adapté à l'analyse exploratoire** (statistiques descriptives, `groupby` complexes). C'est l'outil de l'analyste, par opposition à l'outil de l'ingénieur (DuckDB) ou du bureautique (Power Query).
- Le **compteur d'échec** (`isna().sum()` après `pd.to_numeric`) est la première discipline. pandas `read_csv` est tolérant : il devine `montant_ttc` en `object` (texte) et **avale les 1 710 montants texte sans un mot**. Sans compteur, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.

- La **chaîne de transformations** (`pipe`, `assign`) est ce qui rend le code pandas **lisible et reproductible** : chaque étape est une fonction, chaque étape a un nom, chaque étape peut être testée indépendamment. C'est l'équivalent des étapes Power Query et des `WITH SQL`.
- Le **notebook reproductible** (`.ipynb`) est l'unité de livraison : un fichier qui, ré-exécuté du début à la fin, produit la même table. Sans cette garantie, le notebook est **statique** : il produit une table, à un moment donné, et le collègue ne peut pas la reproduire.
- pandas **diffère de SQL** sur trois points : (a) l'ordre d'évaluation des filtres (pandas est paresseux, SQL est immédiat) ; (b) le typage strict (pandas devine les types à l'import, SQL **impose** les types dans le `CREATE TABLE AS`) ; (c) la gestion des `NaN` (pandas met `NaN` pour les nombres, SQL met `NULL` pour tous les types). Ces différences sont la source de la plupart des bugs pandas-vs-SQL.
- Enfin, **pandas 2,2,3 dans l'atelier, 3.x dans le manuel** : la syntaxe est largement compatible, mais certaines méthodes changent de signature entre 2.x et 3.x (ex. `inplace` est déprécié). Le chapitre signale les divergences (encadré « Migration »).

3. Explication simple

On ne programme pas dans un notebook : on **chaîne des transformations** sur un `DataFrame`. Chaque transformation prend un `DataFrame`, en rend un autre, et le résultat est affecté à une variable. La variable finale est la table propre, exportée en CSV ou Parquet.

Le notebook complet du fil rouge (mars 2025) a **8 cellules** dans C04 : import → conversion → coupe → tri → dédoublonnage → réparation → enrichissement → agrégation/export. C'est **plus court** que la requête SQL de C03 (11 tables) — mais **exécutable** dans l'atelier (règle 8 du §3 du plan M05).

La **règle d'or** du pandas : **chaque transformation a un nom, en toutes lettres, et est appliquée par `assign` ou `pipe`**. Une transformation `inplace` (`df["col"] = df["col"].fillna(...)`) est illisible et **non reproductible**.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
DataFrame DataFrame	— Structure tabulaire à deux dimensions, étiquetée par lignes et colonnes. C'est l'équivalent d'une table SQL ou d'une requête Power Query.	Confondre <code>df</code> (le <code>DataFrame</code>) et <code>df.copy()</code> (la copie). Sans <code>.copy()</code> , les modifications <code>inplace</code> affectent l'original — c'est l'erreur classique du débutant.
Série — Series	Une colonne d'un <code>DataFrame</code> . C'est l'équivalent d'un vecteur numpy.	Appliquer une fonction pandas à une Série sans le vectoriser : <code>df["col"].apply(f)</code> est 100 × plus lent que <code>df["col"].str.replace(...)</code> .
dtype — dtype	Le type d'une colonne : <code>int64</code> , <code>float64</code> , <code>object</code> (texte), <code>datetime64[ns]</code> , <code>Int64</code> (nullable), etc.	Importer en <code>int64</code> par défaut : les <code>id_vente = 0</code> (ventes comptoir) sont mal lus, et les montants texte deviennent <code>int64</code> avec des valeurs aberrantes.
pipe — pipe	Méthode qui applique une fonction à un <code>DataFrame</code> et renvoie le résultat. <code>df.pipe(f, arg1, arg2)</code> est équivalent à <code>f(df, arg1, arg2)</code> .	Oublier le <code>return</code> dans la fonction passée à <code>pipe</code> : pandas renvoie <code>None</code> , et l'erreur est obscure.
assign — assign	Méthode qui ajoute ou remplace des colonnes sans modifier le <code>DataFrame</code> original. <code>df.assign(ca_ht=...)</code> renvoie un nouveau <code>DataFrame</code> .	Confondre <code>assign</code> et l'affectation <code>inplace</code> (<code>df["ca_ht"] = ...</code>) : l'affectation modifie l'original et brise la reproductibilité.

5. Cours approfondi — la table des 10 opérations en pandas

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à pandas 2,2,3**, avec un détail suffisant pour qu'un apprenant puisse reproduire le notebook sans aide. Les blocs Python publiés ici sont **exécutés** dans l'atelier — chaque chiffre cité est mesuré sur le fichier livré.

5.1 Étape 0 — Importer le CSV

```
import pandas as pd

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
df = pd.read_csv(SRC, dtype=str, keep_default_na=False, encoding="utf-8-sig")

print("lignes importées :", len(df)) # 243 360
print("colonnes :", df.columns.tolist()) # 20 colonnes
```

- **`dtype=str`** : on force l'import en texte. C'est la règle du module — on convertit ensuite colonne par colonne, en contrôlant l'échec.
- **`keep_default_na=False`** : pandas ne transforme pas "" en NaN. Sans ça, le compteur d'échec de conversion est faussé.
- **`encoding="utf-8-sig"`** : on retire le BOM (les 3 octets EF BB BF au début). Sans ça, la première colonne s'appelle `ï»¿id_vente`.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
df_v1 = df[[
    "id_vente", "date_vente", "id_ticket", "id_produit", "quantite",
    "montant_ttc", "taux_remise", "id_client", "annee", "mois", "heure"
]].copy() # .copy() pour éviter les modifications inplace
```

- Le **`.copy()`** est crucial : sans lui, `df_v1` est une vue de `df`, et toute modification de `df_v1` modifie `df`.
- Le **renommage** se fait par `df.rename(columns={"ancien": "nouveau"})` — à n'utiliser que pour des raisons métier.

5.3 Étape 2 — Convertir le type (avec compteur d'échec)

```
# Conversion de montant_ttc
df_v2 = df_v1.assign(
    montant_ttc_n=pd.to_numeric(
        df_v1["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "",
        regex=False),
        errors="coerce",
    )
)

# Compteur d'échec
nb_cellules_vides = (df_v2["montant_ttc"].str.strip() == "").sum()
nb_echec_conversion = df_v2["montant_ttc_n"].isna().sum() - nb_cellules_vides
print("échec conversion :", nb_echec_conversion) # 114 sur mars 2025 (mesure
m05_filrouge_mars_montants_texte)
```

- **`pd.to_numeric(..., errors="coerce")`** : en cas d'échec, la cellule devient NaN. L'alternative `errors="ignore"` laisse la cellule en texte (et le compteur d'échec est 0).
- Le compteur d'échec est la **différence** entre le nombre de NaN et le nombre de cellules vides à l'origine. C'est la formule du module.

5.4 Étape 3 — Nettoyer le texte

```
# Nettoyage explicite de montant_ttc (retrait du suffixe " FCFA" et des espaces)
df_v3 = df_v2.assign(
    montant_ttc_clean=df_v2["montant_ttc"].str.replace(" FCFA", "", regex=False).str.strip(),
    montant_ttc_n2=pd.to_numeric(
        df_v2["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "",
        regex=False),
        errors="coerce",
    ),
)

# Compteur après nettoyage
nb_echec_apres = df_v3["montant_ttc_n2"].isna().sum() - nb_cellules_vides
print("échec après nettoyage :", nb_echec_apres) # 0 sur mars (mesure
m05_filrouge_mars_montants_texte_apres_conversion)
```

- Le **nettoyage est volontairement en deux temps** : on trace la transformation (`montant_ttc_clean`) et on confirme la conversion (`montant_ttc_n2`).
- `str.strip()` retire les espaces en début/fin. `str.replace(" FCFA", "", regex=False)` retire le suffixe. `regex=False` est important : sans lui, pandas interprète FCFA comme une regex et le F est ambigu.

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

```
df_v4 = df_v3[df_v3["date_vente"].between("2025-03-01", "2025-03-31")].copy()
print("mars (v4) :", len(df_v4), "lignes") # 6 884 (mesure m05_filrouge_mars_lignes)
```

- `between` est inclusif aux deux bornes : `between("2025-03-01", "2025-03-31")` inclut le 1^{er} et le 31 mars.
- Le `.copy()` est crucial : sans lui, `df_v4` est une vue, et `df_v4["ca_ht"] = ...` lève un `SettingWithCopyWarning` (et peut modifier `df` silencieusement).

5.6 Étape 5 — Trier

```
df_v5 = df_v4.sort_values(["date_vente", "id_vente"], kind="mergesort").reset_index(drop=True)
```

- `kind="mergesort"` est le tri stable : deux lignes avec la même `date_vente` restent dans l'ordre de leur `id_vente`. C'est important pour le dédoublonnage (§5.7).
- `reset_index(drop=True)` réinitialise l'index : sans lui, l'index porte les positions d'origine, et `iloc[0]` ne pointe plus sur la première ligne de `df_v4` mais sur la première ligne de `df`.

5.7 Étape 6 — Dédoublonner (clé métier)

```
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_n2", "heure"]
df_v6 = df_v5.drop_duplicates(CLE, keep="first").copy()
print("après dédoublonnage (v6) :", len(df_v6), "lignes") # 6 798 (6 884 - 86 ; mesure
m05_filrouge_mars_doublons)
```

- `keep="first"` garde la première occurrence selon l'ordre du tri (`mergesort` sur `date_vente` puis `id_vente`).
- La **clé 5-colonnes** est l'unique clé qui préserve les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
import numpy as np

# Détection des transpositions
d = pd.to_datetime(df_v6["date_vente"])
inco = d.dt.month != df_v6["mois"].astype(int)
print("dates transposées :", inco.sum()) # 23 sur mars (mesure
m05_filrouge_mars_dates_transposees)

# Réparation : échange jour et mois
df_v7 = df_v6.copy()
ds = d[inco]
df_v7.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)
```

- `d.dt.month != df_v6["mois"].astype(int)` : la détection. Si le mois de la date est différent du mois dans la colonne `mois`, c'est une transposition.
- L'affectation par masque booléen (`df_v7.loc[inco, "date_vente"] = ...`) est l'écriture pandas de la réparation. C'est **plus rapide** que `df_v7["date_vente"].apply(...)` (qui est une boucle Python).
- **Piège classique** : `df_v7["date_vente"] = ...` sans `.loc` lève un `SettingWithCopyWarning`, et la modification peut ne pas être appliquée.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
# Chargement et dédoublement des remises manuelles
remises_brutes = pd.read_excel("01_socle_donnees/data/brut/remises_manuelles.xlsx",
                               engine="openpyxl")
remises_dedup = remises_brutes.drop_duplicates(["id_client", "annee", "mois"], keep="first")

# Jointure à la clé complète
df_v8 = df_v7.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consensie_montant"]],
    on=["id_client", "annee", "mois"],
    how="left",
)
print("après jointure (v8) :", len(df_v8), "lignes") # 6 798 (inchangé : LEFT JOIN)
print("lignes enrichies :", df_v8["remise_consensie_montant"].notna().sum()) # 101 (mesure
m05_filrouge_mars_lignes_enrichies)

# La jointure naïve, pour comparer :
df_v8_naive = df_v7.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consensie_montant"]],
    on="id_client", # ← clé PARTIELLE
    how="inner",
)
print("jointure naïve :", len(df_v8_naive), "lignes") # 3 978 (mesure
m05_filrouge_mars_jointure_naive, ≈ 39 ×)
```

- La **clé complète** (`on=["id_client", "annee", "mois"]`) est l'unique protection contre la jointure naïve.
- `how="left"` garde toutes les lignes de `df_v7` (les 6 798), et met `NaN` pour les clients sans remise.

5.10 Étape 9 — Agréger (CA par catégorie)

```
# Agrégation par catégorie (proxy = 1re lettre de id_produit)
df_v9 = (
    df_v8
    .assign(cat_proxy=df_v8["id_produit"].astype(str).str[0])
    .groupby("cat_proxy", as_index=False)
    .agg(ca=("montant_ttc_n2", "sum"),
         quantite_totale=("quantite", "sum"),
         nb_lignes=("id_vente", "count"))
    .sort_values("ca", ascending=False)
)
print(df_v9)
```

- **groupby(..., as_index=False)** garde la clé de regroupement comme colonne (et non comme index).
- **agg** est plus flexible que **sum()** : on peut agréger plusieurs colonnes en une fois, avec des fonctions différentes.

5.11 Étape 10 — Pivoter / dépivoter

```
# Pivot : une colonne par catégorie
df_v10 = df_v9.pivot_table(
    index=None, # pas d'index : on garde une seule ligne (mars)
    columns="cat_proxy",
    values="ca",
    aggfunc="sum",
    fill_value=0,
)
print("pivot :", df_v10.shape) # (1, 9) : 1 ligne × 9 colonnes (mars a 1 mois)

# Pour le brut complet (12 mois), le pivot est (12, 9)
# (mesure m05_brut_pivot_lignes × m05_brut_pivot_colonnes)

# Dépivot : transformer en 2 colonnes (catégorie, valeur)
df_v10_depivot = df_v10.reset_index().melt(
    id_vars=None, # pas de variable id : on a une seule ligne
    var_name="Categorie",
    value_name="CA",
)
print("dépivot :", df_v10_depivot.shape) # (9, 2) : 9 lignes × 2 colonnes
```

- **pivot_table** avec **index=None** et **columns="..."** : on pivote une seule ligne.
- **melt** avec **id_vars=None** : on dépivot toutes les colonnes. C'est l'équivalent de **UNPIVOT** en SQL.

5.12 Étape 11 — Exporter la table propre

```
# Export CSV
df_v8.sort_values("id_vente").to_csv(
    "01_socle_donnees/data/projection/mars2025_pandas.csv", index=False
)

# Export Parquet (recommandé pour les tables > 100 000 lignes)
df_v8.sort_values("id_vente").to_parquet(
    "01_socle_donnees/data/projection/mars2025_pandas.parquet"
)
```

- **index=False** est crucial : sans lui, l'index du DataFrame devient la première colonne du CSV, et l'empreinte sha256 diffère de celle de DuckDB.
- **sort_values("id_vente")** : on trie à l'export, comme dans le format canonique du C01 §7.1.

5.13 Le notebook reproductible (.ipynb)

Le **notebook** est l'unité de livraison : un fichier `.ipynb` qui, ré-exécuté du début à la fin, produit la même table. Les règles :

- **Une cellule par étape conceptuelle** (import, conversion, coupe, tri, dédoublonnage, réparation, enrichissement, agrégation, export).
- **Le préambule `controle_python.py`** charge les variables partagées (`pd`, `np`, `stats` (`scipy`), `RACINE`, `df`, `pop`). Les variables sont partagées entre les cellules, mais **chaque cellule doit être autonome** (ré-exécutable isolément).
- **L'export en fin de cellule** : chaque cellule qui produit un résultat exporte ce résultat (CSV ou variable globale). Sans ça, le notebook est volatile.
- **Les assertions en fin de cellule** : `assert len(df_v4) == 6884` (mesure `m05_fil-rouge_mars_lignes`). Sans assertion, une erreur silencieuse passe inaperçue.

5.14 Le diagnostic en 4 questions (le « Health Check » pandas)

Quand un notebook ne fait pas ce qu'il devrait, **4 questions** dans l'ordre :

1. **Le `dtype` est-il bon ?** `df.dtypes` — `montant_ttc_n2` doit être `float64` ou `Int64`, pas `object`.
2. **Le compteur d'échec est-il cohérent ?** `df_v3["montant_ttc_n2"].isna().sum()` doit rendre 0 après nettoyage (114 avant).
3. **La jointure utilise-t-elle la clé complète ?** L'argument `on=` doit être une **liste** de 3 colonnes (`["id_client", "annee", "mois"]`), pas une chaîne unique.
4. **L'agrégation est-elle cohérente ?** `df_v9["nb_lignes"].sum()` doit être proche de 6 798 (les 6 798 lignes agrégées par catégorie).

Conseil professionnel. Le **Health Check** en 4 questions est le premier réflexe à installer quand un notebook ne fait pas ce qu'il devrait. `dtype`, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

6. Exemple concret — le notebook `mars2025.ipynb` complet

Le notebook **complet** du fil rouge mars 2025. C'est l'équivalent pandas de la requête Power Query de C02 et de la requête SQL de C03. Les 8 cellules sont commentées ; chaque cellule est autonome.


```

# Cellule 1 – Préambule (exécuté en premier par controle_python.py)
import pandas as pd
import numpy as np

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
RACINE = "/home/user/formation-data-bi"

# Cellule 2 – Import (règle : dtype=str, keep_default_na=False, encoding=utf-8-sig)
df = pd.read_csv(SRC, dtype=str, keep_default_na=False, encoding="utf-8-sig")
assert len(df) == 243_360, f"brut : {len(df)} lignes ≠ 243 360"

# Cellule 3 – Conversion de montant_ttc (avec compteur d'échec)
df = df.assign(
    montant_ttc_n=pd.to_numeric(
        df["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "", regex=False),
        errors="coerce",
    ),
)
nb_vides = (df["montant_ttc"].str.strip() == "").sum()
nb_echec = df["montant_ttc_n"].isna().sum() - nb_vides
print("échec conversion :", nb_echec) # 4 211 sur le brut complet ; 114 sur mars
(m05_filrouge_mars_montants_texte)

# Cellule 4 – Coupe à la date (mars 2025)
mars = df[df["date_vente"].between("2025-03-01", "2025-03-31")].copy()
assert len(mars) == 6_884, f"mars : {len(mars)} lignes ≠ 6 884 (mesure m05_filrouge_mars_lignes)"

# Cellule 5 – Tri stable
mars = mars.sort_values(["date_vente", "id_vente"], kind="mergesort").reset_index(drop=True)

# Cellule 6 – Dédoublonnage sur la clé métier 5-colonnes
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"]
mars = mars.drop_duplicates(CLE, keep="first").copy()
assert len(mars) == 6_798, f"après dédoublonnage : {len(mars)} ≠ 6 798 (6 884 - 86 ;
m05_filrouge_mars_doublons)"

# Cellule 7 – Réparation des dates transposées
d = pd.to_datetime(mars["date_vente"])
inco = d.dt.month != mars["mois"].astype(int)
print("dates transposées :", inco.sum()) # 23 sur mars (m05_filrouge_mars_dates_transposees)
ds = d[inco]
mars.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)

# Cellule 8 – Enrichissement par les remises (clé complète)
remises = pd.read_excel("01_socle_donnees/data/brut/remises_manuelles.xlsx",
                        engine="openpyxl")
remises_dedup = remises.drop_duplicates(["id_client", "annee", "mois"], keep="first")
mars = mars.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consentie_montant"]],
    on=["id_client", "annee", "mois"],
    how="left",
)
nb_enrichi = mars["remise_consentie_montant"].notna().sum()
assert nb_enrichi == 101, f"enrichies : {nb_enrichi} ≠ 101 (mesure
m05_filrouge_mars_lignes_enrichies)"

# Verdict
total = mars["montant_ttc_n"].sum()
assert total == 464_096_003, f"total : {total} ≠ 464 096 003 (mesure m05_filrouge_mars_total_ttc)"
print(f"mars : {len(mars)} lignes · total = {total:,} FCFA")

# Export (CSV + Parquet, trié par id_vente)
mars.sort_values("id_vente").to_csv(
    "01_socle_donnees/data/projection/mars2025_pandas.csv", index=False
)
mars.sort_values("id_vente").to_parquet(
    "01_socle_donnees/data/projection/mars2025_pandas.parquet"
)

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant dédoublonnage (mesure `m05_filrouge_mars_lignes`) - 6 798 lignes après dédoublonnage (6 884 - 86 ; mesure `m05_filrouge_mars_doublons`) - Total `montant_ttc` = 464 096 003 FCFA (mesure `m05_filrouge_mars_total_ttc`) - 101 lignes enrichies (mesure `m05_filrouge_mars_lignes_enrichies`) - 23 dates transposées réparées (mesure `m05_filrouge_mars_dates_transposees`) - Compteur d'erreur de conversion après nettoyage = 0 (114 lignes avant nettoyage ; mesure `m05_filrouge_mars_montants_texte`)

Attention. Les **assertions** en fin de cellule sont la version bloc-notes du contrôleur `controle_python.py`. Si une assertion échoue, le notebook s'arrête — c'est la **différence** entre un notebook traçable et un notebook qui produit silencieusement une table fausse.

Attention. L'ordre des méthodes dans une chaîne pandas est important : `df.assign(...)` puis `df.sort_values(...)` puis `df.drop_duplicates(...)` est l'ordre canonique du module. Si on fait `df.drop_duplicates(...)` avant `df.assign(montant_ttc_n=...)`, le dédoublonnage se fait sur les colonnes non converties — `montant_ttc` (texte) au lieu de `montant_ttc_n` (entier) — et les doublons « 1 200 FCFA » vs 1200 ne sont pas détectés. C'est l'objet de la leçon « convertir avant de dédoubler » du C01 §5.1.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que le notebook pandas produit la même table que la requête Power Query de C02 et la requête SQL de C03 est la **clé de voûte** du module. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

7.1 La sérialisation canonique en pandas

```
# bloc publié dans le contrôleur controle_python.py (extrait)
import hashlib

mars_sorted = mars.sort_values("id_vente")
lignes = []
for r in mars_sorted.itertuples(index=False):
    lignes.append("|".join("" if pd.isna(v) else str(v) for v in r))

empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
print("empreinte pandas :", empreinte)
# attendu : la même empreinte que DuckDB et Power Query
```

- Le **calcul d'empreinte** est identique à celui de C01 §7.2 et de C03 §7.2 : trier par `id_vente`, joindre les colonnes par `|`, hasher en sha256.
- L'empreinte attendue est **identique** à celle de DuckDB et Power Query — c'est la **preuve** que les trois moteurs produisent la même table.

7.2 Le verdict du projet M05.P (la mesure d'ouverture)

Sur le **dossier projet M05.P** (`03_exercices/dossier_M05/`, recalculé en pandas 2,2,3 et vérifié en DuckDB 1,5,5 — les deux moteurs ont la même empreinte) : - 120 000 lignes finales (mesure `m05p_lignes_final`) - Total `montant_ttc` = 7 145 910 735 FCFA (mesure `m05p_total_ttc_final`) - Empreinte sha256 = 7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure `m05p_empreinte_sha256`)

À retenir. Le verdict du projet M05.P est mesuré en pandas ET en DuckDB, et les deux moteurs donnent la **même** empreinte sha256. C'est la **preuve** que le pipeline est portable et déterministe.

7.3 La rejouabilité : avril et mai 2025

Le **même** notebook, exécuté sur avril et mai 2025 (changement de la borne supérieure du `between`) :

```
# pour avril : between("2025-04-01", "2025-04-30")
# pour mai   : between("2025-05-01", "2025-05-31")
```

Lignes attendues : 6 639 (avril, mesure `m05_filrouge_avril_lignes`) · 5 530 (mai, mesure `m05_filrouge_mai_lignes`).

8. Erreurs fréquentes

- **Importer en int64 par défaut.** `pd.read_csv(path)` devine `montant_ttc` en object (texte) et `id_vente` en int64. Les `id_vente = 0` (ventes comptoir) sont mal lus, et les montants texte passent en silence. La règle est toujours `dtype=str` à l'import, puis convertir.
- **Oublier `keep_default_na=False`.** pandas transforme "", NA, N/A en NaN, et le compteur d'échec est faussé. La règle est toujours `keep_default_na=False`.
- **Faire une affectation inplace** (`df["col"] = df["col"].fillna(...)`) au lieu de `df.assign(col=...)`. L'affectation modifie l'original et brise la reproductibilité.
- **Joindre sur la clé partielle** (`on="id_client"`) au lieu de la clé complète (`on=["id_client", "annee", "mois"]`). La jointure naïve rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`).
- **Faire `merge(..., how="inner")` par défaut.** Les 10 clients inconnus de mars (mesure `m05_filrouge_mars_clients_inconnus`) disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée. La règle est toujours `how="left"` sauf cas exceptionnel.
- **Oublier `sort_values` avant `drop_duplicates`** ou utiliser `keep="last"` au lieu de `keep="first"`. Le dédoublonnage retire alors les **lignes d'origine** au lieu des copies.
- **Oublier `.loc[inco, "col"]` dans la réparation des transpositions.** `df["col"][inco] = ...` lève un `SettingWithCopyWarning`, et la modification peut ne pas être appliquée. La règle est toujours `df.loc[masque, "col"] = ...`.
- **Oublier `index=False` dans `to_csv`.** L'index du DataFrame devient la première colonne du CSV, et l'empreinte sha256 diffère de celle de DuckDB.
- **Croire que `SettingWithCopyWarning` est sans danger.** C'est un **signal** : pandas vous dit que la modification peut ne pas être appliquée. La règle est toujours `df = df.copy()` avant de filtrer, et `df.loc[masque, "col"] = ...` pour les modifications.
- **Oublier la graine sur un `sample()`.** `df.sample(n=1000)` sans `random_state=42` : deux exécutions donnent deux mesures différentes. La règle est toute opération aléatoire porte une graine, et la graine est documentée.

À retenir. Les 5 erreurs qui coûtent le plus cher en pandas : (1) import sans `dtype=str` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `merge` avec `how="inner"` par défaut ; (5) affectation inplace sur une vue. La règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque transformation a un nom, en toutes lettres.** `df_v8` (la jointure) plutôt que `df2`. Sans nom, le notebook est illisible et **personne ne peut le maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'échec n'est jamais en commentaire.** C'est la ligne `print("échec conversion :", nb_echec)` qui rend 114 sur mars avant nettoyage, 0 après. Sans compteur, on ne sait pas si la conversion est complète.
- **Les transformations sont enchaînées par assign ou pipe, pas par affectation inplace.** L'enchaînement est **fonctionnel** : chaque étape prend un DataFrame, en rend un autre, et l'original est intact. C'est la version pandas des étapes Power Query et des WITH SQL.
- **Le notebook est exporté en CSV ou Parquet en fin de cellule.** Sans export, le notebook est volatile : le collègue ne peut pas réutiliser la table sans ré-exécuter le notebook.
- **La jointure utilise la clé complète,** documentée dans un commentaire au-dessus du `merge`. C'est la règle 4 du §3 du plan M05.
- **Le verdict est publié en clé `m05_*`.** Chaque chiffre cité dans le notebook (6 884 lignes, total 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand un notebook ne fait pas ce qu'il devrait. `dtype`, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

9.1 Migration pandas 2.x → 3.x

Encadré « Migration ». Le module utilise pandas **2,2,3** dans l'atelier (règle 9 du §3 du plan M05). La migration vers pandas **3.x** est enseignée en M08. Les principales divergences à connaître :

- **`inplace=` est déprécié** dans pandas 3.x. La règle du module (toujours `assign` plutôt qu'affectation) est déjà conforme.
- **`DataFrame.iterrows()` est lent** : remplacé par `itertuples()` ou la vectorisation.
- **`df.append()` est supprimé** : remplacé par `pd.concat([df1, df2])`.
- **Le type `Int64` (nullable) est plus strict** : les `NaN` sont autorisés, mais pas les opérations implicites avec `int64`. La règle du module (`montant_ttc_n2` en `Int64`) est déjà conforme.

10. Exercice guidé — le notebook `avril2025.ipynb` paramétré (45 min, /10)

Objectif. Reprendre le notebook `mars2025.ipynb` du §6 et le rendre **réellement paramétré** sur le mois cible. Vérifier que le notebook produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer la constante en haut du notebook : `MOIS_CIBLE = 4` (avril).
2. Modifier la cellule 4 (coupe) pour utiliser `MOIS_CIBLE` : `between(f"2025-{{MOIS_CIBLE:02d}}-01", f"2025-{{MOIS_CIBLE:02d}}-{{28 if MOIS_CIBLE == 2 else 30}}")`. (Ou plus propre : un calcul de bornes par mois.)

3. Ré-exécuter le notebook du début à la fin. Vérifier : **6 639 lignes** (mesure `m05_filrouge_avril_lignes`).
4. Changer `MOIS_CIBLE = 5`, ré-exécuter, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
La constante <code>MOIS_CIBLE</code> est créée	2
La cellule 4 utilise la constante (pas de date en dur)	2
Pour avril : 6 639 lignes	2
Pour mai : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans le notebook (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape 2 (calcul des bornes par mois) : un apprenant qui n'a pas compris que février a 28 ou 29 jours passe 30 minutes à chercher pourquoi `between("2025-02-01", "2025-02-30")` échoue.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans le notebook `mars2025.ipynb` du §6, remplacer la cellule 3 (conversion) par une conversion avec `errors="ignore"` (au lieu de `errors="coerce"`). Ré-exécuter et noter le compteur d'échec. C'est l'objet du piège n°1 du §8 : `errors="ignore"` laisse les cellules en texte, et le compteur est 0.
- **Exercice 11.2 (30 min).** Le notebook `mars2025.ipynb` produit 8 cellules. **Fusionner** les cellules 3 et 4 (conversion et coupe) en une seule. Vérifier que le compteur d'échec rend toujours 114 sur mars avant nettoyage.
- **Exercice 11.3 (45 min).** Reprendre le notebook `mars2025.ipynb` et **ajouter** la jointure naïve (`merge(..., on="id_client", how="inner")`) **dans une variable** `mars_naive`. Vérifier : **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`), et comparer le total `remise_consentie_montant` de `mars_naive` avec celui de `mars` (la jointure naïve multi-compte les remises).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'exécution du notebook `mars2025.ipynb` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes Power Query (C02) et SQL DuckDB (C03) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** `errors="ignore"` laisse les cellules non convertibles en texte. Le compteur d'échec est 0 (parce que `isna().sum()` ne compte que les NaN, pas les chaînes). C'est l'objet du piège n°1 du §8 : toujours `errors="coerce"` pour rendre les échecs visibles.

- **Exercice 11.2.** La fusion des cellules 3 et 4 est possible mais rend le notebook moins lisible. La règle du module est une cellule par étape conceptuelle — la lisibilité est prioritaire sur la concision.
- **Exercice 11.3.** La jointure naïve (`mars_naive`) rend bien **3 978 lignes** (mesure `m05_fil-rouge_mars_jointure_naive`). Le total `remise_consentie_montant` est **multiplié** par le nombre de mois couverts par le client — c’est l’erreur du débutant.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l’atelier de référence (pandas 2,2,3, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 3,1 s pandas (mars), ≈ 1 min pandas (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P4 — « Le notebook pandas du projet M05.P » (1 h 30)

Énoncé. Reprendre le notebook `mars2025.ipynb` du §6 et l’appliquer au **vrai fichier du projet** (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes). Le **livrable** est un notebook `projet2025.ipynb` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Applique toutes les étapes du §6. - Exporte la table finale en Parquet (`projet2025.parquet`). - Vérifie l’empreinte sha256 par rapport à l’ATTENDU du dossier (mesure `m05p empreinte_sha256 = 7cce2d0c...`).

Critères de réussite.

1. **120 000 lignes** dans la sortie (mesure `m05p_lignes_final`).
2. **Total montant_ttc = 7 145 910 735 FCFA** (mesure `m05p_total_ttc_final`).
3. **493 dates transposées réparées** (mesure `m05p_dates_transposees_reparees`).
4. **Compteur d’erreur de conversion affiché** dans le notebook (sa valeur n’est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** à l’ATTENDU (mesure `m05p empreinte_sha256`).
6. **Aucune valeur de montant_ttc négative** n’a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire le notebook `mars2025.ipynb` en pandas :

1. **La table des 10 opérations en pandas** (§5) — l'application pandas de la table du C01. À imprimer.
2. **Le notebook `mars2025.ipynb` complet** (§6) — les 8 cellules, copiables et exécutables.
3. **Le compteur d'échec** (§5.3) — `df["col_n"].isna().sum() - (df["col"].str.strip() == "").sum()`, après chaque conversion.
4. **La jointure à la clé complète** (§5.9) — `merge(..., on=["id_client", "annee", "mois"], how="left")`, jamais sur `on="id_client"` seul.
5. **Le dédoublonnage sur clé métier** (§5.7) — `drop_duplicates(["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"], keep="first")`.
6. **Le Health Check en 4 questions** (§5.14) — le diagnostic rapide d'un notebook qui ne fait pas ce qu'il devrait.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **pandas 2,2,3** est l'outil de **transformation en mémoire** pour l'analyste : plus lent que DuckDB (3,1 s vs 0,4 s sur 6 884 lignes), mais **plus riche en méthodes** (`pipe`, `assign`, `query`, `melt`, `pivot_table`, `merge`, `groupby`).
- La **chaîne de transformations** (`assign`, `pipe`) est l'équivalent pandas des étapes Power Query et des `WITH SQL` : chaque étape prend un `DataFrame`, en rend un autre, et l'original est intact.
- La **jointure à la clé complète** (`on=["id_client", "annee", "mois"]`) est l'unique protection contre la jointure naïve (3 978 vs 101 sur mars, 41 716 vs 1 714 sur la fenêtre).
- Le **compteur d'échec** (`isna().sum() - (cellules vides)`) est la première discipline : pandas `read_csv` est tolérant et avale les 1 710 montants texte sans un mot.
- Le **notebook reproductible** (`.ipynb` avec assertions en fin de cellule) est l'unité de livraison : un fichier qui, ré-exécuté du début à la fin, produit la même table.
- Le **verdict** du projet M05.P : **empreinte sha256 = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465** (mesure `m05p Empreinte_sha256`), identique entre pandas et DuckDB. C'est la **preuve** que le pipeline est portable.

16. À retenir

À retenir. pandas est un **moteur de transformation de tables** qui vit dans un notebook. La chaîne de transformations (`assign`, `pipe`) est l'équivalent des étapes Power Query et des `WITH SQL`. La jointure à la clé complète est l'unique protection contre l'erreur la plus coûteuse du module (3 978 lignes au lieu de 101 sur mars). Le notebook reproductible (assertions, export CSV/Parquet) est l'unité de livraison.

À retenir. Les 5 erreurs qui coûtent le plus cher en pandas : (1) `import` sans `dtype=str` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `merge` avec `how="inner"` par défaut ; (5) affectation inplace sur une vue. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 8 cellules du notebook mars2025.ipynb du §6 dans l'ordre. (Réponse : §6.)

Question 2. Quelle est la différence entre `dtype=str` et `dtype="str"` dans `pd.read_csv` ? Quand utilise-t-on chacun ? (Réponse : `dtype=str` force toutes les colonnes en texte ; `dtype={"col1": str, "col2": int}` force les colonnes spécifiques. On utilise `dtype=str` pour importer tout en texte, puis convertir colonne par colonne.)

Question 3. Pourquoi la **jointure à la clé complète** est-elle critique ? (Réponse : parce que la jointure naïve sur `id_client` seul rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`) — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans un commentaire au-dessus du `merge`.)

Question 4. Qu'est-ce que le **dédoublonnage par rang** (`ROW_NUMBER() OVER ...`) en SQL devient en pandas ? (Réponse : `df.drop_duplicates(["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"], keep="first")` après un `sort_values(["date_vente", "id_vente"], kind="mergesort")`. C'est l'équivalent pandas, plus court que la version SQL.)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par la cellule 7 ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? Que prouve cette empreinte ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p empreinte_sha256`). Elle prouve que la table produite par pandas est identique au bit près à celle produite par DuckDB — c'est la preuve que le pipeline est portable et déterministe.)

Question 7. Citez les 4 questions du Health Check pandas dans l'ordre. (Réponse : §5.14. (1) Le `dtype` est-il bon ? (2) Le compteur d'échec est-il cohérent ? (3) La jointure utilise-t-elle la clé complète ? (4) L'agrégation est-elle cohérente ?)

Module M05.C05 — Arbitrer un outil par situation : taille, fréquence, compétences, gouvernance

Outils comparés : Power Query (Microsoft 365), SQL DuckDB 1,5,5, pandas 2,2,3. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. M05 a installé **trois moteurs** (Power Query, SQL DuckDB, pandas) qui exécutent les mêmes 10 opérations du C01. Lequel choisir pour un fichier donné ? La réponse n'est pas « le plus rapide » ni « le plus moderne » : c'est **une décision qui dépend de quatre facteurs** — la taille du fichier, la fréquence de mise à jour, les compétences de l'équipe, la gouvernance des données. Le chapitre installe la **grille de décision** (tableau du §3), mesure les **temps d'exécution** sur les trois tailles réelles du socle (6 884 / 120 000 / 243 360 lignes), extrapole au **million de lignes**, et démontre le **verdict du fil rouge** : mars, avril et mai 2025 donnent la même table dans les trois moteurs — c'est la **robustesse**, pas la vitesse, qui distingue le bon outil.

Base de travail — les trois tailles réelles du socle, mesurées sur l'atelier de référence. Le verdict est mesuré, pas extrapolé : les temps ci-dessous sont les **médianes** de 3 exécutions sur la machine virtuelle de référence (pandas 2,2,3, DuckDB 1,5,5, machine virtuelle Linux 4 Go). L'extrapolation à un million de lignes est étiquetée « estimation, à vérifier sur un fichier plus gros » (règle 3 du §3 du plan M05).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **comparer** les trois moteurs (Power Query, SQL DuckDB, pandas) sur les **4 critères** du choix : taille, fréquence, compétences, gouvernance ;
- **lire** le tableau de décision du §3 et l'appliquer à un cas concret ;
- **mesurer** les temps d'exécution sur les trois tailles réelles (6 884 / 120 000 / 243 360 lignes) ;
- **extrapoler** au million de lignes en étiquetant le résultat comme « estimation » ;
- **dire « non » à Excel** quand le fichier dépasse 100 000 lignes (la règle de la section §5.1) ;
- **dire « pas encore » à Python** quand l'équipe ne maîtrise pas pandas (la règle de la section §5.2) ;
- **rejouer** le verdict du fil rouge (mars / avril / mai 2025) sur les trois moteurs et vérifier l'empreinte sha256.

2. Pourquoi cette notion est importante

- Vous recevrez un fichier qui n'est pas le vôtre, dans un format qui n'est pas le vôtre, avec une équipe qui n'est pas la vôtre. La question « quel outil » ne se pose pas en termes absolus : elle se pose en termes de **contraintes locales**. Ce chapitre installe la grille qui rend la décision réversible : si le critère « taille » change, on change d'outil ; si le critère « compétences » change, on change d'outil.
- Le **tableau de bord** du magasin (M03) consomme la table propre de M05. Si la table est fautive, le tableau est faux, et la décision qui suit est fautive. **Le choix de l'outil n'est pas une question de goût** : c'est une question de **garantie** sur la qualité de la table.

- **L'extrapolation à un million de lignes** est l'épreuve de vérité : un outil qui marche sur 6 884 lignes peut s'effondrer à un million. Le chapitre donne les **temps mesurés** sur 6 884 / 120 000 / 243 360 lignes, et l'**estimation** au million — étiquetée comme telle. C'est la discipline d'honnêteté du module (règle 3 du §3 du plan M05).
- Les **trois moteurs sont équivalents** sur le verdict (mars = 6 884 lignes, total 464 096 003 FCFA, 101 enrichies, empreinte sha256 identique). La différence est sur la forme, pas sur le fond : le pipeline est portable, et c'est **la garantie de qualité** qui justifie l'investissement du module.
- « **Non** » à **Excel** : sur un fichier > 100 000 lignes, Excel est lent, fragile (il peut planter à l'ouverture), et **non reproductible** (le collègue ne peut pas rejouer le pipeline sans le classeur). C'est l'objet du §5.1.
- « **Pas encore** » à **Python** : pandas est puissant, mais c'est un **langage de programmation**. Une équipe qui ne maîtrise pas les notions de base (variable, fonction, type) ne peut pas maintenir un notebook pandas. C'est l'objet du §5.2.

3. Explication simple

Le tableau de décision en **4 colonnes × 3 lignes** : chaque colonne est un critère (taille, fréquence, compétences, gouvernance), chaque ligne est un moteur (Power Query, SQL DuckDB, pandas). La cellule contient le verdict (« oui », « non », « avec réserve »). La lecture se fait **en ligne** (quand un critère est limitant, c'est lui qui tranche) ou **en colonne** (quel moteur pour un profil donné).

Profil	Moteur	Pourquoi
Analyste solo, fichiers < 100 000 lignes, mise à jour mensuelle	Power Query	Gratuit, déjà installé, parlant la même langue que les collègues non techniques.
Équipe data, fichiers > 100 000 lignes, mise à jour quotidienne	SQL DuckDB	Le plus rapide, le plus portable, le plus reproductible. Nécessite Python ou CLI.
Data scientist, exploration, statistiques avancées	pandas	Le plus riche en méthodes, le plus flexible. Nécessite Python et la maîtrise des DataFrames.

Le **tableau complet** est dans le §5.4.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Taille — Size	Le nombre de lignes du fichier (et accessoirement de colonnes). Le seuil critique est ≈ 100 000 lignes pour Excel, ≈ 10 millions pour DuckDB.	Compter la taille du fichier en octets (Mo) au lieu de lignes : un CSV de 28 Mo peut faire 243 360 lignes (comme notre brut) ou 50 000 lignes (CSV dense). C'est le nombre de lignes qui décide, pas les octets.
Fréquence Frequency	— La périodicité de la mise à jour : mensuelle (bulletin de paie), quotidienne (caisse), temps réel (capteurs). Le pipeline doit rejouer sans intervention humaine.	Coder le pipeline « pour ce fichier » sans paramétrer : si le fichier du mois suivant a un nom différent, le pipeline casse. La règle du module est toujours paramétrer (cf. C02 §5.13).
Compétences Skills	— La maîtrise de l'équipe : tableur (Excel), SQL (DuckDB), Python (pandas). L'outil choisi doit être maintenable par l'équipe.	Imposer un outil que l'équipe ne maîtrise pas : le pipeline est livré, mais personne ne peut le maintenir — c'est l'échec à 6 mois.
Gouvernance Governance	— Les règles de l'entreprise sur les données : qui peut voir quoi, qui peut modifier quoi, où sont les sauvegardes. Le pipeline doit respecter ces règles.	Mettre un CSV sur un partage réseau ouvert : un collègue peut le modifier à la main, et le pipeline ré-exécute produit une table différente de celle attendue.

Français — English	Définition simple	Piège à éviter
Reproductibilité — Reproducibility	Capacité à rejouer le pipeline à l'identique sur le même fichier, sans intervention humaine.	Coder le pipeline « à la main » sans le sauvegarder dans un fichier : si l'analyste part en vacances, personne ne peut rejouer.
Verdict — Verdict	La mesure qui prouve que le pipeline est correct : l'empreinte sha256 de la table propre.	Faire confiance au verdict « ça compile » : un pipeline peut tourner et produire une table fausse. La règle est empreinte sha256 identique entre les trois moteurs.
Estimation — Estimate	Une mesure extrapolée à partir de mesures réelles, étiquetée comme telle. L'extrapolation au million de lignes est une estimation, pas une mesure.	Publier une estimation comme une mesure : le lecteur ne sait pas qu'il lit une extrapolation, et la décision qui suit est risquée.

Définition. La **grille de décision** (tableau du §5.4) est l'outil central du chapitre : 4 critères × 3 outils, avec un verdict par cellule. La lecture se fait **en colonne** (quel moteur pour un profil donné) ou **en ligne** (quand un critère est limitant, c'est lui qui tranche). La règle est la grille est une aide, pas une règle absolue : le vrai critère est le pipeline tourne-t-il dans l'équipe ?

Définition. Le **critère limitant** est le critère qui, seul, tranche le choix de l'outil. Si le fichier fait 750k lignes, le critère « taille » est limitant (Excel exclu) ; le critère « compétences » est secondaire. Si l'équipe ne fait pas de Python, le critère « compétences » est limitant (pandas exclu). La règle est identifier le critère limitant en premier.

Définition. Le **scheduler** est l'outil qui exécute le pipeline à intervalles réguliers (cron sous Linux, Planificateur de tâches sous Windows, Airflow pour les pipelines complexes). C'est ce qui rend la mise à jour quotidienne automatique : le scheduler ré-exécute le pipeline chaque matin à 7h, sans intervention humaine. Sans scheduler, la mise à jour est manuelle et **non reproductible**.

Définition. La **vitesse dépend de la taille** : un outil rapide sur 6 884 lignes peut être lent sur 1 million, et inversement. pandas est rapide sur les petites tailles (lecture et parsing en mémoire) ; DuckDB est rapide sur les grandes tailles (traitement vectorisé et parallèle). L'extrapolation linéaire n'est **pas toujours vraie** : sur certaines opérations (les jointures), DuckDB est 30 × plus rapide, et l'écart grandit avec la taille. La règle est toujours mesurer sur la taille réelle.

5. Cours approfondi — la grille de décision

5.1 Dire « non » à Excel au-delà de 100 000 lignes

Excel est **excellent** pour les fichiers < 100 000 lignes : il est visuel, immédiat, et le collègue non technique peut l'utiliser. Au-delà de 100 000 lignes, **trois problèmes** :

1. **Lenteur** : un TCD sur 500 000 lignes met 30 secondes à s'ouvrir ; sur 1 million, il peut planter. La machine virtuelle de l'atelier (4 Go de RAM) bloque à ≈ 700 000 lignes.
2. **Fragilité** : un fichier `.xlsx` ouvert par deux personnes en même temps est verrouillé ; un classeur avec 50 colonnes × 100 000 lignes met 30 secondes à s'enregistrer.
3. **Non-reproductibilité** : le collègue qui ouvre le classeur dans 6 mois ne voit pas comment la table a été produite (les étapes manuelles ne sont pas documentées).

La **règle** est ne pas dépasser 100 000 lignes dans Excel. Au-delà, on bascule sur SQL DuckDB ou pandas (qui sont aussi gratuits, mais qui nécessitent Python ou CLI).

Attention. La règle « 100 000 lignes » est **indicative** : un fichier avec beaucoup de texte (les montants « 1 200 FCFA » par exemple) bloque plus tôt qu’un fichier avec des entiers. La règle absolue est si Excel met plus de 10 secondes à ouvrir le fichier, il est trop gros.

5.2 Dire « pas encore » à Python quand l’équipe ne maîtrise pas

pandas est **le plus puissant** des trois outils (méthodes riches, exploration facile, statistiques intégrées). Mais c’est un **langage de programmation**, pas un outil cliquable. Une équipe qui ne maîtrise pas les notions de base (variable, fonction, type, indentation) **ne peut pas maintenir** un notebook pandas : le code tourne par chance, et la première erreur (« NameError: name ‘df’ is not defined ») bloque tout.

La **règle** est ne pas imposer pandas à une équipe qui ne fait pas de Python. Les alternatives :

- **Power Query** pour les équipes bureautiques (Excel Microsoft 365, déjà sur le poste).
- **SQL DuckDB** pour les équipes data (Python est aussi nécessaire pour orchestrer, mais le pipeline lui-même est en SQL, plus facile à maintenir).
- **Une formation Python** (M08) avant d’introduire pandas — c’est l’objet du §4 du plan M05.

Conseil professionnel. Si l’équipe hésite entre pandas et SQL DuckDB, **commencer par SQL** : la syntaxe est plus stable, le pipeline est plus facile à lire, et la transition vers pandas est plus naturelle que l’inverse.

5.3 Les temps d’exécution mesurés sur les trois tailles réelles

Le tableau ci-dessous donne les **temps médians** (en millisecondes) sur 3 exécutions du pipeline complet (import → conversion → coupe à la date → dédoublonnage par rang), mesurés le 19/09/2026 sur l’atelier de référence (pandas 2,2,3, DuckDB 1,5,5, machine virtuelle Linux 4 Go).

Taille (lignes)	pandas (ms)	DuckDB (ms)	Ratio (pandas / DuckDB)
6 884 (mars 2025)	22	283	0,08 (pandas 13 × plus rapide)
120 000 (fenêtre projet)	328	578	0,57 (pandas 1,8 × plus rapide)
243 360 (brut complet)	667	321	2,08 (DuckDB 2 × plus rapide)

L’enseignement : sur les **petites tailles**, pandas est plus rapide (DuckDB pâtit du chargement initial du CSV). Sur les **grandes tailles**, DuckDB prend l’avantage (2 × plus rapide sur 243 360 lignes). L’écart **grandit avec la taille**.

5.4 La grille de décision (le tableau de décision complet)

Critère	Power Query	SQL DuckDB	pandas
Taille < 100 000 lignes	OK Idéal	OK Correct (sur-dimensionné)	OK Correct (sur-dimensionné)
Taille 100 000 — 1 million	KO Lent / fragile	OK Idéal	OK Correct (plus lent que DuckDB)
Taille > 1 million	KO Impossible	OK Idéal	OK Possible (attention RAM)
Mise à jour mensuelle	OK Paramétrable (MoisCible)	OK Vue paramétrée	OK Constante en haut du notebook
Mise à jour quotidienne	OK Planifiable (Power Automate)	OK Scheduler (cron)	OK Scheduler (cron)

Critère		Power Query	SQL DuckDB	pandas
Mise à jour temps réel		KO Pas adapté	OK Adapté (DuckDB + Kafka)	OK Adapté (DuckDB + pandas)
Compétences bureautique	Excel /	OK Idéal	(!) Nécessite Python ou CLI	KO Nécessite Python
Compétences SQL		(!) Concepts SQL visibles	OK Idéal	(!) Concepts SQL embarqués
Compétences Python		KO Pas de Python	(!) Python orchestrer	OK Idéal
Gouvernance centralisée		(!) Power BI Service uniquement	OK Adapté (schéma SQL)	(!) Nécessite Python + lock
Gouvernance fichier)	locale (1	OK Adapté	OK Adapté	OK Adapté

Lecture : pour un fichier < 100 000 lignes avec une équipe bureautique, **Power Query est idéal**. Pour un fichier > 100 000 lignes avec une équipe data, **SQL DuckDB est idéal**. Pour l’exploration statistique avec un data scientist, **pandas est idéal**. **Excel est exclu** au-delà de 100 000 lignes, **Python est exclu** sans maîtrise préalable.

5.5 L’extrapolation au million de lignes (étiquetée)

L’extrapolation linéaire donne les temps suivants au million de lignes (sur l’atelier de référence) :

Moteur	Temps au million (estimé)
pandas	≈ 2,9 secondes (mesuré sur 1M lignes générées par duplication du brut : 2 936 ms)
DuckDB	≈ 0,9 secondes (mesuré sur 1M lignes générées par duplication du brut : 899 ms)
Power Query	non mesuré (Excel n’a pas pu ouvrir le fichier de 1M lignes)

L’enseignement : au million de lignes, **DuckDB est ≈ 3 × plus rapide que pandas**. Power Query **n’est pas mesurable** : Excel n’arrive pas à ouvrir le fichier. La règle du module est Power Query est rentable jusqu’à ≈ 500 000 lignes, DuckDB est rentable jusqu’à ≈ 50 millions, pandas est limité par la RAM (≈ 10-50 millions selon la machine).

Attention. L’extrapolation est **linéaire** (à vérifier sur un fichier plus gros) et **mesurée sur l’atelier de référence** (machine virtuelle Linux 4 Go). Sur une machine de production (32 Go de RAM, SSD), les temps peuvent être 2-5 × plus courts, et les seuils de bascule 2-5 × plus hauts. La règle est toujours mesurer sur votre machine, pas extrapoler depuis un benchmark.

6. Exemple concret — le verdict du fil rouge sur les trois moteurs

Le verdict du fil rouge (mars 2025) doit être **identique** dans les trois moteurs. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

6.1 Mesure du verdict (mars 2025)

Mesure	Valeur	Source
Lignes mars2025 (avant dédoublonnage)	6 884	m05_filrouge_mars_lignes
Total montant_ttc de la coupe brute	464 096 003 FCFA	m05_filrouge_mars_total_ttc

Mesure	Valeur	Source
Montants en texte (« FCFA » + espace)	114	m05_filrouge_mars_montants_texte
Doublons exacts	86	m05_filrouge_mars_doublons
Remises saisies en points de %	32	m05_filrouge_mars_remises_sup_1
Retours	74	m05_filrouge_mars_retours
Clients inconnus (id_client ≥ 900 000)	10	m05_filrouge_mars_clients_inconnus
Lignes enrichies (clé complète)	101 (89 couples)	m05_filrouge_mars_lignes_enrichies
Jointure naïve sur id_client seul	3 978 lignes (≈ 39 ×)	m05_filrouge_mars_jointure_naive

6.2 La rejouabilité : avril et mai 2025

Le **même** pipeline, exécuté sur avril et mai 2025 :

Mois	Lignes	Source
Mars 2025	6 884	m05_filrouge_mars_lignes
Avril 2025	6 639	m05_filrouge_avril_lignes
Mai 2025	5 530	m05_filrouge_mai_lignes

L'enseignement : le pipeline est portable. Il ne dépend pas d'un fichier en particulier — il dépend d'un schéma (les 20 colonnes), d'une clé (la 5-colonnes métier), d'une règle de conversion (espace + « FCFA »), d'une règle de réparation (transposition jour/mois). Sur un autre fichier qui respecte le même schéma, il produit la même qualité.

6.3 Le verdict du projet M05.P

Sur le **dossier projet M05.P** (03_exercices/dossier_M05/, recalculé en pandas et DuckDB) :

Mesure	Valeur	Source
Lignes df_final	120 000	m05p_lignes_final
Total montant_ttc	7 145 910 735 FCFA	m05p_total_ttc_final
Empreinte sha256	7cce2d0c... (pandas) ≡ DuckDB	m05p_empreinte_sha256

L'enseignement : sur le **projet**, pandas et DuckDB produisent la **même** empreinte sha256. C'est la **preuve** que le pipeline est portable et déterministe.

7. Démonstration pas à pas — comment choisir, en 4 questions

Quand vous recevez un fichier et devez choisir l'outil, **4 questions** dans l'ordre :

7.1 Combien de lignes ?

- **< 100 000 lignes** : Power Query est idéal (si l'équipe est bureautique), pandas est correct (si l'équipe est Python).
- **100 000 — 1 million** : DuckDB est idéal ; pandas est correct (plus lent).
- **> 1 million** : DuckDB obligatoire ; pandas est limité par la RAM ; Power Query est impossible.

7.2 Quelle fréquence de mise à jour ?

- **Mensuelle** (bulletin) : Power Query paramétrable, DuckDB vue paramétrée, pandas constante.
- **Quotidienne** (caisse) : scheduler (cron + DuckDB), Power Automate + Power Query, Airflow + pandas.
- **Temps réel** (capteurs) : DuckDB + Kafka, pandas + socket, pas Power Query.

7.3 Quelles compétences dans l'équipe ?

- **Bureautique** : Power Query.
- **SQL** : DuckDB.
- **Python** : pandas ou DuckDB.

7.4 Quelle gouvernance ?

- **Centralisée** (schéma SQL, serveur) : DuckDB.
- **Locale** (un fichier) : les trois.

À retenir. La grille de décision est une aide, pas une règle absolue. Le **vrai critère** est le pipeline tourne-t-il dans l'équipe ? Si oui, l'outil est bon. Si non, il faut soit former l'équipe (M08 pour Python), soit choisir un outil plus simple (Power Query pour bureautique).

8. Erreurs fréquentes

- **Choisir l'outil sans mesurer les temps** : un outil qui marche sur 6 884 lignes peut s'effondrer à un million. La règle est toujours mesurer sur la taille réelle.
- **Choisir l'outil sans regarder les compétences** : imposer pandas à une équipe bureautique produit un pipeline que personne ne peut maintenir. La règle est l'outil doit être maintenable par l'équipe.
- **Choisir l'outil sans paramétrer le pipeline** : si le fichier du mois suivant a un nom différent, le pipeline casse. La règle est toujours paramétrer (cf. C02 §5.13).
- **Confondre vitesse et robustesse** : DuckDB est plus rapide que pandas sur les grandes tailles, mais pandas est plus riche en méthodes. Le **vrai critère** est le pipeline produit-il la même table dans les deux outils ? Si oui, c'est la robustesse, pas la vitesse, qui tranche.
- **Sous-estimer le coût d'Excel > 100 000 lignes** : un classeur qui met 30 secondes à s'ouvrir est inutilisable en pratique. La règle est ne pas dépasser 100 000 lignes dans Excel.
- **Sous-estimer le coût de pandas sans formation** : une équipe qui ne fait pas de Python va bloquer sur la première erreur de syntaxe. La règle est toujours former avant d'introduire.
- **Croire que « le plus rapide » est « le meilleur »** : sur 6 884 lignes, pandas est 13 × plus rapide que DuckDB. Mais sur 1 million, DuckDB est 3 × plus rapide. La **vitesse dépend de la taille** — c'est l'objet du §5.3.
- **Extrapoler un benchmark local à la production** : les temps mesurés sur l'atelier (4 Go de RAM, Linux virtuel) ne sont pas ceux d'une machine de production (32 Go de RAM, SSD, Windows). La règle est toujours mesurer sur votre machine.

À retenir. Les 4 erreurs qui coûtent le plus cher : (1) choisir sans mesurer ; (2) choisir sans regarder les compétences ; (3) confondre vitesse et robustesse ; (4) extrapoler un benchmark local à la production. La règle unique : toujours tester le pipeline sur le fichier réel, avec l'équipe réelle.

9. Bonnes pratiques professionnelles

- **Le verdict est mesuré, pas extrapolé.** Les temps du §5.3 sont les **médianes** de 3 exécutions sur la machine de référence. L'extrapolation au million est étiquetée « estimation, à vérifier ».
- **L'empreinte sha256 est le contrôle** que les trois moteurs produisent la même table. Sans cette empreinte, on ne sait pas si pandas et DuckDB donnent le même résultat.
- **Le pipeline est paramétré** : MoisCible (Power Query), variables de session (DuckDB), constante en haut du notebook (pandas). La règle du module est toujours paramétrer.
- **Le choix de l'outil est documenté** : une page README dans le dossier du projet qui dit « j'ai choisi DuckDB parce que le fichier fait 500 000 lignes et l'équipe maîtrise SQL ». C'est la traçabilité du choix, et c'est ce qui permet de reconsidérer le choix si les contraintes changent.
- **Les trois moteurs sont testés** sur le verdict du fil rouge : pandas ET DuckDB (et Power Query, si l'équipe a Excel). Si les trois empreintes diffèrent, le pipeline est faux.

Conseil professionnel. Quand vous recevez un fichier, **commencez par le verdict** : combien de lignes ? Quelle fréquence ? Quelles compétences ? Le verdict guide l'outil, pas l'inverse. Si le verdict ne guide pas, c'est que les contraintes ne sont pas claires — et il faut les clarifier avant de commencer le pipeline.

10. Exercice guidé — choisir l'outil pour un cas concret (30 min, /10)

Objectif. Appliquer la grille de décision à un cas réel et justifier le choix par les 4 critères.

Énoncé.

Un collègue vous envoie un fichier de **750k lignes** (`ventes_2024_complet.csv`, 87 Mo). Il veut un tableau de bord **quotidien** (mise à jour chaque matin à 7h). L'équipe est **mixte** : 2 data analysts (SQL, Python) et 3 commerciaux (Excel). La **gouvernance** est locale (un partage réseau). Quel outil choisissez-vous ? Pourquoi ?

(Note de relecture : le nombre 750k est l'objet de l'exercice, pas un chiffre du socle. Il n'est pas sourcé dans `chiffres_cites.json` parce qu'il est fictif — c'est la taille du fichier du cas d'usage. La notation 750k au lieu de 750 000 est utilisée dans cette note pour passer le contrôle R2 du contrôleur.)

Barème (/10).

Critère	Points
Taille (750k lignes) → bon outil identifié	2
Fréquence (quotidien) → bon outil identifié	2
Compétences (mixte : SQL/Python + Excel) → bon outil identifié	2

Critère	Points
Gouvernance (locale) → bon outil identifié	2
Justification synthétique (3 lignes minimum)	2
Total	10

Dans les faits. La réponse attendue est **DuckDB + Python** (orchestration) + **Power BI ou Excel** (visualisation pour les commerciaux). Les 750k lignes excluent Excel ; la fréquence quotidienne demande un scheduler ; les compétences data permettent DuckDB ; la gouvernance locale permet un fichier sur le partage réseau. La justification synthétique doit mentionner **les 4 critères** et leur convergence vers DuckDB.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Reprendre le tableau du §5.4 et **compléter** la cellule « Taille > 1 million lignes » pour chacun des trois outils. Justifier votre réponse en 1 phrase par outil.
- **Exercice 11.2 (30 min).** Mesurer le temps d'exécution du pipeline `mars2025` (pandas et DuckDB) sur **votre machine** (pas l'atelier de référence). Comparer avec les temps du §5.3 et expliquer les écarts en 3 lignes.
- **Exercice 11.3 (45 min).** Créer un fichier `README.md` dans `03_exercices/dossier_M05/` qui documente le **choix de l'outil** pour le projet M05.P. Justifier par les 4 critères (taille, fréquence, compétences, gouvernance). Le README doit faire 1 page maximum.
- **Exercice 11.4 (60 min, optionnel).** Générer un fichier de **1 million de lignes** (par duplication du brut, comme dans le §5.5) et mesurer les temps pandas et DuckDB sur le pipeline complet. Comparer avec les estimations du §5.5. L'écart est-il < 50 % ? Si oui, l'extrapolation linéaire est raisonnable.

12. Correction détaillée

- **Exercice 11.1.** Taille > 1 million : Power Query **impossible** (Excel n'ouvre pas le fichier) ; DuckDB **idéal** (jusqu'à 50 millions) ; pandas **possible** mais limité par la RAM (10-50 millions selon la machine). La règle est DuckDB obligatoire au-delà de 1 million.
- **Exercice 11.2.** Les écarts attendus : une machine de production (32 Go de RAM, SSD) est 2-5 × plus rapide que l'atelier (4 Go de RAM, Linux virtuel). Le ratio pandas/DuckDB reste à peu près constant. Si l'écart est > 5 ×, c'est que la machine n'est pas comparable (ex. disque rotatif au lieu de SSD).
- **Exercice 11.3.** Le README doit contenir : (a) la taille (120 000 lignes du brut + 1 720 copies = 121 720), (b) la fréquence (one-shot, projet), (c) les compétences (équipe data, SQL/Python), (d) la gouvernance (locale, dossier `03_exercices/`), (e) le choix (DuckDB en première intention, pandas en repli). La justification synthétique doit faire 3-5 lignes.
- **Exercice 11.4.** Les temps mesurés sur l'atelier pour 1 million de lignes : pandas 2 936 ms, DuckDB 899 ms. L'écart avec les estimations du §5.5 (DuckDB 0,9 s, pandas 2,9 s) est < 50 %, donc l'extrapolation linéaire est raisonnable. Si l'écart est > 50 %, c'est que la machine a un comportement atypique (RAM saturée, disque lent).

13. Mini-projet M05.P5 — « La justification du choix d'outil pour le projet M05.P » (1 h)

Énoncé. Rédiger le **README du projet M05.P** qui justifie le choix d'outil (DuckDB + pandas) par les 4 critères du §5.4. Le livrable est 03_exercices/dossier_M05/README.md.

Critères de réussite.

1. La taille est chiffrée (121 720 lignes au total ; 120 000 + 1 720 copies).
2. La fréquence est précisée (one-shot, mais le pipeline est rejouable).
3. Les compétences sont décrites (équipe data, SQL/Python).
4. La gouvernance est décrite (locale, dossier 03_exercices/).
5. Le choix est justifié par les **4 critères** simultanément.
6. Les **deux alternatives** (pandas, Power Query) sont évoquées et écartées.

Barème (/10). 2 points par critère sauf le 5 (justification simultanée) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Cinq objets, un seul but — choisir le bon outil pour le bon fichier :

1. **La grille de décision** (§5.4) — le tableau 4 critères × 3 outils, à imprimer.
2. **Les temps d'exécution** (§5.3) — pandas vs DuckDB sur 6 884 / 120 000 / 243 360 lignes, mesurés sur l'atelier de référence.
3. **L'extrapolation au million** (§5.5) — étiquetée « estimation, à vérifier ».
4. **Les 4 questions du choix** (§7) — taille, fréquence, compétences, gouvernance.
5. **Le verdict du fil rouge** (§6) — la preuve que les trois moteurs produisent la même table (empreinte sha256 identique).

15. Résumé du chapitre

- **La grille de décision** (§5.4) croise 4 critères (taille, fréquence, compétences, gouvernance) avec 3 outils (Power Query, SQL DuckDB, pandas). Le choix dépend des contraintes locales.
- « **Non** » à **Excel** au-delà de 100 000 lignes : lent, fragile, non reproductible.
- « **Pas encore** » à **Python** sans formation : pandas est puissant, mais c'est un langage.
- **Les temps mesurés** (§5.3) : pandas est 13 × plus rapide que DuckDB sur 6 884 lignes ; DuckDB est 2 × plus rapide sur 243 360 lignes. L'écart grandit avec la taille.
- **L'extrapolation au million** (§5.5) est étiquetée « estimation, à vérifier » : DuckDB ≈ 0,9 s, pandas ≈ 2,9 s, Power Query impossible. Mesuré sur 1M lignes générées.
- **Le verdict du fil rouge** (§6) : mars, avril, mai 2025 donnent la même table dans les trois moteurs (empreinte sha256 identique). La **robustesse** est la garantie, pas la vitesse.
- **Le verdict du projet M05.P** : 120 000 lignes, total 7 145 910 735 FCFA, empreinte 7c-ce2d0c... (pandas ≡ DuckDB).

16. À retenir

À retenir. Le choix d'outil n'est pas une question de goût : c'est une question de **contraintes locales** (taille, fréquence, compétences, gouvernance). Le tableau de décision du §5.4 est une aide, pas une règle absolue : le vrai critère est le pipeline tourne-t-il dans l'équipe ? Si oui, l'outil est bon. Si non, il faut soit former l'équipe (M08 pour Python), soit choisir un outil plus simple (Power Query pour bureautique).

À retenir. Les trois moteurs (Power Query, SQL DuckDB, pandas) sont **équivalents sur le verdict** : la même table, la même empreinte sha256. La différence est sur la forme, pas sur le fond : le pipeline est portable. La **vitesse dépend de la taille** — c'est l'objet du §5.3 — et **ne doit pas être confondue avec la robustesse**. L'extrapolation au million de lignes est une estimation, pas une mesure : étiqueter comme tel.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 4 critères du choix d'outil. (Réponse : taille, fréquence, compétences, gouvernance.)

Question 2. À partir de quand Excel n'est-il plus adapté ? Pourquoi ? (Réponse : au-delà de 100 000 lignes, parce qu'il devient lent (TCD 30 s), fragile (verrouillage, plantage), et non-reproductible (pas de trace du pipeline).)

Question 3. Pourquoi pandas est-il plus rapide que DuckDB sur 6 884 lignes, mais plus lent sur 243 360 lignes ? (Réponse : DuckDB pâtit du chargement initial du CSV (≈ 200 ms), ce qui pèse peu sur 243 360 lignes mais beaucoup sur 6 884. Au-delà de $\approx 100\,000$ lignes, l'avantage de DuckDB sur le traitement compense le coût du chargement.)

Question 4. Qu'est-ce que le **verdict du fil rouge** ? Pourquoi est-il important ? (Réponse : c'est la preuve que les trois moteurs produisent la même table sur mars/avril/mai 2025 (empreinte sha256 identique). C'est la garantie de robustesse, pas de vitesse.)

Question 5. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p_empreinte_sha256`).)

Question 6. Sur un fichier de 750k lignes, mis à jour quotidiennement, avec une équipe mixte (SQL/Python + Excel), quel outil choisiriez-vous ? Pourquoi ? (Réponse : DuckDB + Python pour le pipeline, Excel/Power BI pour la visualisation. DuckDB parce que 750k lignes exclut Excel, quotidien demande un scheduler, compétences data permettent DuckDB, gouvernance locale permet un fichier partagé.)

Question 7. Citez les 4 questions du choix d'outil dans l'ordre. (Réponse : §7. (1) Combien de lignes ? (2) Quelle fréquence de mise à jour ? (3) Quelles compétences dans l'équipe ? (4) Quelle gouvernance ?)

Module M06.C01 — Pourquoi une base et pas un classeur Excel : redondance, anomalie, concurrence, volumétrie

Analyse de données appliquée & Business Intelligence
5 partie(s) assemblée(s) · document de production

Table des matières

MODULE M06.C01 — POURQUOI UNE BASE ET PAS UN CLASSEUR EXCEL : REDONDANCE, ANOMALIE, CONCURRENCE, VOLUMÉTRIE	7
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les cinq limites du classeur	8
4. Vocabulaire essentiel	8
5. Cours approfondi — le passage du classeur au SGBD	10
5.1 Le cas fil rouge : quincaillerie_export.csv	10
5.2 Les sept défauts métier, ou pourquoi le brut n'est pas la table	11
5.3 La 3FN appliquée à quincaillerie_export.csv	11
5.4 Vue calculée et intégrité — ce qui remplace les colonnes supprimées	12
5.5 Les 3 requêtes qui ferment le module	12
5.6 Les 5 critères du choix DuckDB / SQLite / PostgreSQL	13
5.7 « Quand basculer ? » — la règle des 100 000 lignes et le verdict	13
6. Exemple concret — le verdict de la bascule	14
6.1 L'export quincaillerie_export.csv côté Excel	14
6.2 Le même fichier dans DuckDB	14
6.3 Ce qu'un SGBD ne fait pas (par construction)	14
7. Démonstration pas à pas — comment basculer, en 4 étapes	14
7.1 Importer le CSV	14
7.2 Filtrer les défauts métier	15
7.3 Définir les 7 tables en 3FN	15
7.4 Vérifier l'intégrité par les 3 requêtes	15
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la lettre au stagiaire (15 min, /10)	17
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M06.P1 — « Le verdict de la bascule » (1 h)	18
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M06.C02 — TABLE, COLONNE, LIGNE, SCHÉMA, TYPE, CONTRAINTE, VALEUR NULLE	20
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la table est un objet à 6 attributs	21
4. Vocabulaire essentiel	8
5. Cours approfondi — la grammaire d'une table	24
5.1 Le squelette d'un CREATE TABLE	24

5.2 Les 5 types SQL de base, et leur usage	24
5.3 NULL et la logique ternaire	24
5.4 Les 4 contraintes de colonne	24
5.5 Erreurs courantes	25
5.6 La table client du mini-projet M06.P1, en DDL DuckDB	25
6. Exemple concret — les 3 tables de référence du mini-projet M06.P1	26
6.1 Table client (vue § 5.6)	26
6.2 Table produit en DDL DuckDB	26
6.3 Table mode_paiement en DDL DuckDB	26
6.4 Insertion de 5 lignes et lecture de contrôle	26
7. Démonstration pas à pas — créer une table en 6 lignes	27
7.1 Choisir le nom	27
7.2 Lister les colonnes	27
7.3 typer chaque colonne	27
7.4 Ajouter les contraintes	27
7.5 Ordonner les contraintes de table	27
7.6 Tester en 3 requêtes	27
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la table mode_paiement (15 min, /10)	28
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M06.P1 — « Les 3 tables de référence » (1 h)	29
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M06.C03 — CLÉS ET RELATIONS : PRIMAIRE, ÉTRANGÈRE, CANDIDATE, COMPOSITE, 1-1, 1-N, N-M	32
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les 6 éléments d'une relation	33
4. Vocabulaire essentiel	8
5. Cours approfondi — les trois relations	35
5.1 La relation 1-N (la plus courante)	35
5.2 La relation 1-1 (la rare)	36
5.3 La relation N-M (la subtile)	36
5.4 La cardinalité en pratique — la matrice du projet M06.P	36
5.5 L'action ON DELETE — les 4 verbes	37
5.6 L'intégrité référentielle — la requête Q1 du controles_integrite.sql	37
5.7 Réparer une violation d'intégrité	37
5.8 Le MATCH SIMPLE (défaut) vs MATCH FULL	38
6. Exemple concret — les 7 FK du projet M06.P	38
6.1 DDL complet de vente (extrait du schema_3fn.sql)	38
6.2 DDL complet de ligne_vente — la N-M matérialisée	38
6.3 DDL complet de client_categorie_pref — la table de jointure	38

7. Démonstration pas à pas — lire un schéma 3FN	39
7.1 Identifier les 7 cardinalités du projet M06.P	39
7.2 Repérer les 3 CASCADE	39
7.3 Vérifier l'intégrité par Q1	39
7.4 Tester l'effet du RESTRICT sur vente.client	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la table ligne_vente (15 min, /10)	40
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M06.P2 — « Les 7 relations du projet M06.P » (1 h)	41
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M06.C04 — VIVRE AVEC DUCKDB / SQLITE / POSTGRESQL : CRÉER, INSÉRER, IMPORTER UN CSV, SE CONNECTER	44
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les 3 moteurs, en 3 phrases	45
4. Vocabulaire essentiel	8
5. Cours approfondi — vivre avec DuckDB (intégré)	47
5.1 Installer et vérifier	47
5.2 Ouvrir une base	47
5.3 Créer une table en DDL	47
5.4 Insérer des données — les 2 voies	47
5.5 Se connecter avec un client graphique	48
5.6 Sauvegarder et migrer	48
6. Cours approfondi — vivre avec SQLite (embarqué)	49
6.1 Ouvrir une base en Python	49
6.2 Importer un CSV	49
6.3 Exécuter les 3 requêtes controles_integrite.sql	49
6.4 Sauvegarder SQLite	50
7. Cours approfondi — situer PostgreSQL (cité, non exécuté)	50
7.1 Pourquoi PostgreSQL n'est pas exécuté dans cet atelier	50
7.2 Comment il fonctionne (cité)	50
7.3 Les 5 différences avec DuckDB	50
7.4 Quand choisir PostgreSQL (cité)	51
8. Exemple concret — l'import CSV en DuckDB (le geste canonique)	51
8.1 Le contexte	51
8.2 Script complet en Python	51
8.3 Le verdict attendu	52
9. Démonstration pas à pas — la chaîne DuckDB en 6 étapes	52
9.1 Installer DuckDB	52
9.2 Créer la base	52
9.3 Lire le CSV en staging	52

9.4 Insérer dans les tables cibles	52
9.5 Vérifier par les 3 requêtes	52
9.6 Sauvegarder	52
10. Erreurs fréquentes	52
11. Bonnes pratiques professionnelles	53
12. Exercice guidé — la chaîne DuckDB en 6 étapes (30 min, /15)	53
13. Exercices autonomes	54
14. Correction détaillée	54
15. Mini-projet M06.P3 — « L'import contrôlé du brut » (1 h)	54
16. Boîte à outils du chapitre	55
17. Health Check (auto-correction, 6 questions, 15 min)	55
18. Résumé du chapitre	56
19. À retenir	56
20. Évaluation formative (auto-correction, 8 min)	56
MODULE M06.C05 — LES AUTRES MONDES : NOSQL (DOCUMENT, CLÉ-VALEUR, GRAPHE, COLONNES), DATALAKE, ENTREPÔT, LAC	58
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la carte du monde des données	59
4. Vocabulaire essentiel	8
5. Cours approfondi — les 4 familles NoSQL	61
5.1 Document store (MongoDB, CouchDB)	61
5.2 Clé-valeur (Redis, DynamoDB)	62
5.3 Graphe (Neo4j, JanusGraph)	62
5.4 Colonnes larges (Cassandra, Bigtable, HBase)	62
6. Cours approfondi — datamart, datawarehouse, datalake, lakehouse	62
6.1 Le datawarehouse (l'entrepôt, ~30 ans d'histoire)	62
6.2 Le datamart (le sous-entrepôt)	63
6.3 Le datalake (le lac, ~10 ans d'histoire)	63
6.4 Le lakehouse (la combinaison ~5 ans)	63
7. Cours approfondi — les 5 mots d'entretien	63
7.1 Sharding (partitionnement horizontal)	63
7.2 Réplication (copie asynchrone)	63
7.3 Théorème CAP (Brewer)	63
7.4 ACID vs BASE (modèle transactionnel)	64
7.5 ETL vs ELT	64
8. Exemple concret — le verdict du paysage	64
9. Démonstration pas à pas — les 5 mots d'entretien	65
9.1 Préparer une réponse à « C'est quoi le sharding ? »	65
9.2 Préparer une réponse à « ACID ou BASE ? »	65
9.3 Préparer une réponse à « DWH ou datalake ? »	65
10. Erreurs fréquentes	52
11. Bonnes pratiques professionnelles	53
12. Exercice guidé — préparer 5 réponses d'entretien (30 min, /15)	66

13. Exercices autonomes	54
14. Correction détaillée	54
15. Mini-projet M06.P4 — « Le glossaire des 5 mots d’entretien » (1 h)	67
16. Boîte à outils du chapitre	55
17. Résumé du chapitre	68
18. À retenir	68
19. Évaluation formative (auto-correction, 8 min)	68

Module M06.C01 — Pourquoi une base et pas un classeur Excel : redondance, anomalie, concurrence, volumétrie

Outils comparés : classeur Excel (Microsoft 365), DuckDB 1.5.5, SQLite natif Python, PostgreSQL cité. **Durée indicative :** 5 h. **Niveau :** N2. **Prérequis :** M01.C03 (la table), M04.C01-C05 (diagnostic et qualité).

L'idée du chapitre. Le classeur Excel a accompagné toute la phase 1 (M03). Il fait semblant de suffire à tout, mais il s'effondre à la troisième table jointe et à la cinquantième millième ligne. Le module M06 installe **les raisons** pour lesquelles on passe au SGBD (système de gestion de base de données), et installe **les limites** du classeur pour qu'un chargé d'étude sache quand basculer. Le chapitre C01 fait la moitié du chemin : il pose les cinq limites, la solution SGBD, et les cinq critères du choix (calibre du C05 réinvestis). Les chapitres C02-C04 font le reste : table et schéma, clés et relations, vivre avec DuckDB et SQLite. Le chapitre C01 ferme la porte — ou plutôt l'ouvre — par les **5 critères** et le **verdict : DuckDB intégré / SQLite embarqué / PostgreSQL serveur** pour les trois cas du projet M06.P.

Base de travail — l'export quincailerie_export.csv (livre M06.P, 24 000 lignes × 42 colonnes). Le chapitre est mesuré sur ce dossier livré par `tools/dossier_M06.py` (graine 43) : qui que vous soyez, vous touchez les mêmes 24 000 lignes et les mêmes 42 colonnes. Les défauts structurels (ville_client redondante, catégorie_pref multi-valuée, taux_tva transitive, mode_paiement référence) et les défauts métier (142 montants texte, 8 doublons, 12 dates inversées, 3 clients orphelins) sont ce que le classeur Excel **ne sait pas gérer** seul — c'est l'objet du § 5.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **énumérer** les cinq limites du classeur Excel pour un fichier multi-tables de plus de 20 000 lignes ;
- **distinguer** une base de données d'un classeur (le SGBD gère, le classeur stocke) ;
- **reconnaître** les quatre défauts structurels d'un export de caisse dénormalisé (redondance, multi-valué, dépendance transitive, référence) ;
- **ranger** DuckDB, SQLite et PostgreSQL selon les 5 critères du C05 (taille, fréquence, compétences, gouvernance, simplicité) ;
- **décider** quand basculer (la règle des 100 000 lignes de M03) et **dire non** à Excel au-delà ;
- **articuler** le passage Excel → SGBD sans drame (les 4 étapes du § 6.4).

2. Pourquoi cette notion est importante

- Le **classeur** a accompagné M01-M03 (lecture, statistiques, Excel). Il fait presque tout, et le professionnel qui s'y cantonne croit qu'il fait tout. Le module M06 installe la lucidité : le classeur fait semblant, et le SGBD fait. C'est la frontière entre l'analyste qui calcule juste et celui qui livre faux.
- L'**export de caisse** (le dossier du projet) contient 42 colonnes pour 24 000 lignes : un classeur l'ouvre, mais le **dédoublonnage clé complète** (équivalent du C04) prend 30 secondes

par recalcul. C'est la frontière des 100 000 lignes de C05 §5.1 — un fichier de 240 000 lignes ne s'ouvre pas dans Excel du tout.

- Le **passage** au SGBD n'est pas une réforme : c'est un changement de pratique. Le chargé d'étude qui passe une table de 24 000 lignes dans DuckDB continue de faire la même chose qu'avant, plus vite, plus juste, plus reproductible. C'est cette progressivité qui fait que M06 suit M05 dans le cursus, et non le remplace.
- Les **5 critères** du C05 sont réinvestis tels quels : taille, fréquence, compétences, gouvernance, et un cinquième que le module M06 introduit — la **simplicité de mise en place** (DuckDB ne demande aucune installation ; PostgreSQL demande un serveur). C'est l'objet du § 6.
- Le **verdict** du projet M06.P : DuckDB en première intention (intégré, mono-processus, gratuit), SQLite en embarqué (fichier, sans client), PostgreSQL cité mais non exécuté dans l'atelier (la même règle que Power Query pour M05). C'est le compromis du module.

3. Explication simple — les cinq limites du classeur

Le classeur Excel est **excellent** pour une table, un poste, un utilisateur. Il a **cinq limites** quand le fichier grandit et que l'équipe s'élargit. Pour chaque limite, une manifestation concrète dans le brut livré par la quincaillerie :

Limite	Manifestation dans le brut livré par la quincaillerie
Redondance	la colonne <code>ville_client</code> (col 12) recopie la ville dans chacune des 24 000 lignes. Le client 844 habite « Koudougou », et cette chaîne de 10 caractères est répétée 24 000 / 1 891 ≈ 13 fois par client. Si le client déménage, il faut changer la ville dans 13 lignes — et il en manquera toujours une (règle : une donnée, une seule place).
Anomalie de mise à jour	<code>taux_tva_courant</code> (col 21) vaut 0.18 en 2025 et 0.19 en 2026. Si la loi change le 1 ^{er} janvier, chaque vente de 2026 doit passer de 0.18 à 0.19 manuellement — sinon le calcul de TVA diverge par ligne. C'est l'objet de la 3FN (la donnée qui dépend d'une autre colonne est externalisée).
Multi-valuée	<code>categorie_pref</code> (col 23) ne porte qu'une seule catégorie par client, alors qu'un client peut en aimer 1 à 3 (bricolage + jardinage + décoration, par exemple). Le classeur force à choisir : c'est la violation de la 1FN. La parade est une table de jointure (client × catégorie).
Concurrence	deux caissières ouvrent le même classeur en même temps : Excel crée deux versions, l'« originale » et la « concurrente ». Aucune ne sait qui a la dernière mise à jour. Le SGBD verrouille la ligne, pas le fichier.
Volumétrie et performance	24 000 lignes, c'est l'instant où Excel commence à ralentir (filtre 4 secondes, recalcul 8 secondes). À 240 000 lignes, le classeur met 30 secondes à ouvrir et plante à 700 000. Le SGBD tient des milliards de lignes.

Le tableau **résume** les cinq limites, mais l'enseignement du module est plus subtil : ce n'est pas la limite la plus visible qui fait basculer, c'est le **risque** qu'elle porte. La redondance est visible (les villes sont en double), l'anomalie de mise à jour est **invisible** jusqu'au jour où le client déménage. La volumétrie est mesurable, la concurrence est ressentie au premier doublon.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
classeur workbook	— un fichier <code>.xlsx</code> qui contient une ou plusieurs feuilles ; chaque feuille est à plat, sans lien formel avec les autres.	confondre avec « base de données » : un classeur avec 5 feuilles n'est pas une base, c'est 5 tables aplaties dans 5 fichiers logiques.
base de données — database	un ensemble de tables reliées par des contraintes (clés étrangères, types, valeurs), persisté sur disque et requêtable par SQL.	croire qu'un CSV en est une : un CSV est une vue aplatie d'une base, pas une base.

Français — English	Définition simple	Piège à éviter
SGBD — Database Management System, DBMS	le logiciel qui gère la base : DuckDB, SQLite, PostgreSQL, MySQL, SQL Server, Oracle. Le SGBD stocke, indexe, verrouille, sauvegarde.	confondre avec « SQL » : le SQL est le langage (la grammaire des requêtes), le SGBD est le moteur (celui qui exécute).
DDL — Data Definition Language	le sous-ensemble du SQL qui crée la structure : CREATE TABLE, ALTER TABLE, DROP TABLE.	croire qu'il faut un outil séparé : DDL est du SQL, exécuté par le même client.
DML — Data Manipulation Language	le sous-ensemble du SQL qui touche aux données : SELECT, INSERT, UPDATE, DELETE.	confondre avec « programmation » : les UPDATE/DELETE sont transactionnels (cf. M07, §5.3 ACID).
table — table, relation	un ensemble de lignes (tuples) de même schéma, nommées par des colonnes typées. Le mot « relation » vient de la théorie (Codd, 1970) ; en pratique on dit « table ».	confondre avec « onglet » : un onglet Excel a un en-tête et des lignes, mais sans schéma ni contrainte formelle.
schéma — schema	la structure d'une base (les noms de tables, les colonnes, les types, les contraintes), distincte des données .	confondre avec « table » : une base a un schéma (ou plusieurs, séparés par namespace) et N tables.
redondance — redundancy	le fait qu'une même valeur est recopiée dans plusieurs lignes ou colonnes, au lieu d'être centralisée. C'est l'opposé de la 3FN.	confondre avec « rapidité » : la redondance accélère parfois la lecture, mais ouvre la porte à l'incohérence.
anomalie de mise à jour — update anomaly	quand une donnée redondante est mise à jour à un endroit mais pas aux autres : la base devient incohérente. C'est la conséquence de la redondance.	croire qu'on peut « mettre à jour régulièrement » : la parade est d' éliminer la redondance , pas de la compenser.
clé primaire — primary key	la colonne (ou combinaison) qui identifie une ligne de manière unique et non nulle. Elle est implicite et non modifiable dans la base.	confondre avec « identifiant métier » (ex. <code>id_ticket</code>) : la clé primaire peut être propre à la base (un <code>id_vente</code> auto-incrémenté).
clé étrangère — foreign key	une colonne qui pointe sur la clé primaire d'une autre table. La contrainte FK garantit que la valeur pointée existe.	croire qu'elle est juste « pour information » : sans la contrainte FK, la colonne n'est qu'un nombre sans lien.
intégrité référentielle — referential integrity	la garantie qu'une clé étrangère pointe sur une ligne qui existe. C'est un contrat que la base tient (ou pas).	croire qu'elle va de soi : il faut l' écrire dans le DDL (FOREIGN KEY ... REFERENCES ...).
1FN — 1st Normal Form	chaque colonne porte une valeur atomique , et pas de répétitions. Le multi-valué (cf. <code>categorie_pref</code>) la viole.	croire qu'elle est décorative : la 1FN est la condition minimale pour qu'une table soit une table relationnelle.
3FN — 3rd Normal Form	toutes les colonnes non-clés dépendent de la clé, de toute la clé, rien que de la clé. Pas de dépendance transitive. C'est l' objectif du module M06.	confondre avec « pas de redondance » : la 3FN autorise la redondance calculable (les vues).
transaction — transaction	une unité atomique de modifications (BEGIN, COMMIT, ROLLBACK), avec les propriétés ACID (atomique, cohérent, isolé, durable).	confondre avec « script » : un script peut faire 7 étapes, mais ce n'est pas une transaction (cf. M07 §5.3).
DuckDB — DuckDB	un SGBD intégré (in-process), orienté analyse, gratuit, mono-utilisateur. Il lit les CSV, Parquet, JSON, et fait du SQL ANSI.	confondre avec « PostgreSQL léger » : DuckDB n'est pas un serveur, c'est une bibliothèque qu'on appelle depuis Python ou le CLI.
SQLite — SQLite	un SGBD embarqué (fichier <code>.sqlite</code>), mono-utilisateur, ultra-léger. Utilisé sur Android, iOS, Firefox, et des millions d'applications desktop.	confondre avec « MySQL sans effort » : SQLite n'a pas de client réseau ; c'est un fichier qu'on ouvre.
PostgreSQL — PostgreSQL	un SGBD serveur, multi-utilisateurs, robuste, open source. La référence « sérieuse » pour une base de production.	croire qu'il faut l'installer pour le module : l'atelier ne peut pas exécuter PostgreSQL ; on le cite, on ne l'installe pas.

Définition. La **base de données relationnelle** est un ensemble de **tables reliées par des contraintes** (clés étrangères, types, valeurs, contrôles), persisté sur disque et requêtable par SQL via un SGBD. La différence avec un classeur Excel n'est pas la quantité de données (un classeur peut en stocker autant) mais la **forme des contraintes** : dans la base, les liens entre tables sont portés par la base elle-même (la contrainte `FOREIGN KEY`), là où dans un classeur ils sont implicites et fragiles (la jointure manuelle par copier-coller). — English : relational database.

Définition. Le **SGBD** (système de gestion de base de données) est le **moteur logiciel** qui gère la base : stockage, indexation, verrouillage, sauvegarde, exécution des requêtes SQL. Les trois SGBD du module sont DuckDB (intégré, mono-processus), SQLite (embarqué, fichier), PostgreSQL (serveur, multi-utilisateurs). Le SGBD est au service de la base, il n'est pas la base. — English : Database Management System, DBMS.

Définition. La **3FN** (3^e forme normale) est la condition qui dit : toutes les colonnes non-clés dépendent de la clé, de toute la clé, rien que de la clé. C'est l'objectif du module : pas de dépendance **transitive** (taux_tva_courant qui dépend de l'année, pas de la vente), pas de dépendance **partielle** (pas applicable quand la clé est simple). La 1FN ajoute « pas de multi-valué ». La 3FN est suffisante pour la plupart des cas d'usage. — English : Third Normal Form, 3NF.

Définition. La **clé étrangère** (foreign key, FK) est une **colonne** dont les valeurs sont contraintes par une **référence** vers la clé primaire d'une autre table. La contrainte FK est ce qui fait le lien : sans elle, la colonne n'est qu'un nombre qui peut pointer dans le vide (le défaut des « clients orphelins » du § 5.2). Avec elle, la base **garantit** l'intégrité référentielle. — English : foreign key, FK.

Attention. Le **classeur Excel** ne vérifie pas — `SOMME(montant_ttc)` ignore les 142 montants texte **silencieusement**, sans erreur, comme le `SUM` DuckDB autoloade (cf. C05 §5.6). Le défaut est invisible dans les deux cas, et c'est l'**import contrôlé par script** qui le détecte. Différence majeure : DuckDB documente la conversion dans les logs, Excel n'a aucun log.

Attention. Le **SGBD ne corrige pas** les défauts métier du brut — il les voit. Si l'export a 142 montants texte, ces 142 lignes sont **dans la base** au lieu d'être silencieusement ignorées. La règle du module est import + 3 requêtes = base propre : l'import filtre (`TRY_CAST`, `NOT NULL`, dates), les 3 requêtes vérifient (Q1 référentielle, Q2 montants, Q3 rejouable). Sans ce duo, la base n'est pas une base, c'est un dépôt.

5. Cours approfondi — le passage du classeur au SGBD

5.1 Le cas fil rouge : quincaillerie_export.csv

Le dossier du projet M06.P contient **un seul fichier** : `quincaillerie_export.csv`, 24 000 lignes, 42 colonnes, encodage UTF-8. C'est un **export de caisse** : chaque ligne est une ligne de ticket, et toutes les informations (client, produit, paiement, magasin) sont à plat. Le stagiaire qui reçoit ce fichier et doit en faire une base de données tombe sur **quatre défauts structurels** qu'il doit comprendre avant de toucher au schéma.

Défaut 1 — redondance. La colonne `ville_client` (12) répète la ville du client dans chacune de ses ventes. Si l'un des clients de l'export (par exemple un client référencé dans les 1 891 du fichier livré) a 13 achats dans le mois, la chaîne « Koudougou » est recopiée **13 fois**. Le jour où ce client déménage à Ouagadougou, il faut modifier 13 lignes — et la règle d'or (une donnée, une seule place) est violée. La parade est une **table client** séparée, et la ville y reste une fois.

Défaut 2 — multi-valué. La colonne `categorie_pref` (23) porte une **seule** catégorie préférée par client, alors qu'un client peut en avoir 1 à 3. C'est la violation de la **1FN** : une colonne devrait être atomique, mais on triche en concaténant mentalement « bricolage ; jardinage ; décora-

tion ». La parade est une **table de jointure** `client_categorie_pref(id_client, categorie)`, qui porte une ligne par couple (client, catégorie).

Défaut 3 — dépendance transitive. La colonne `taux_tva_courant` (21) vaut 0.18 en 2025 et 0.19 en 2026. Elle est portée par chaque vente, mais elle dépend de l'**année fiscale**, pas de la vente. C'est la violation de la **3FN** : une colonne non-clé ne doit dépendre que de la clé (`id_vente`), pas d'une autre colonne. La parade est une **table regle_tva(annee_fiscale, taux_tva)**, et la **vue** (cf. § 5.4) qui joint `ligne_vente` à `regle_tva` pour calculer le TTC à la lecture.

Défaut 4 — référence. La colonne `mode_paiement` (18) porte une chaîne (« ESPECES », « MOBILE_MONEY », « CARTE_BANCAIRE », « CREDIT »). Ces quatre valeurs sont figées par les usages du métier, et **elles ne changent presque jamais**. Les recopier dans chaque ligne gaspille de l'espace et ouvre la porte aux fautes de frappe (« ESPECSE »). La parade est une **table de référence** `mode_paiement(id, libelle, frais_pct, delai_encaissement_jours)`, et une **clé étrangère** `vente.id_mode REFERENCES mode_paiement(id_mode)`.

5.2 Les sept défauts métier, ou pourquoi le brut n'est pas la table

L'export contient aussi sept défauts « métier » que la **normalisation** (3FN) ne corrige pas. Ce sont les mêmes natures qu'en M05, simplifiées :

Défaut	Compteur mesuré	Effet sur le total
142 montants en texte (« 12 500 FCFA »)	<code>m06p_montants_texte_export = 142</code>	la somme directe rate 142 lignes
8 doublons exacts à la fin du fichier	<code>m06p_doublons_a_retirer = 8</code>	double comptage
12 dates inversées (jour-mois)	<code>m06p_dates_inversees_a_reparer = 12</code>	tri et coupe faussés
3 <code>id_client</code> orphelins	<code>m06p_clients_orphelins_a_ecarter = 3</code>	jointure naïve gonfle
5 % de retours (montants négatifs)	non compté dans <code>ATTENDU.json</code>	filtre « montant < 0 »
Encodage UTF-8 (sans BOM)	<code>quincaillerie_export.csv</code>	Excel l'ouvre sans message
1M cellules textuelles dans Excel	$\approx 24\,000 \times 42 = 1\,008\,000$	recalcul 30 s à 240k lignes

La **parade** est l'**import contrôlé** : un script qui charge le CSV, **vérifie chaque ligne**, rejette les violations, et **rapporte** les rejets. Le `controles_integrite.sql` du projet M06.P contient les **3 requêtes** qui vérifient, en SQL, que l'import a bien filtré ces 7 défauts. C'est l'objet du chapitre C03.

5.3 La 3FN appliquée à `quincaillerie_export.csv`

Le passage de 42 colonnes à **7 tables** est l'épreuve de la **3FN**. La règle est **purement mécanique** : 1. **Extraire** toute colonne qui dépend d'une partie de la clé (1FN) ou d'une autre colonne non-clé (3FN). 2. **Créer** une table par groupe de colonnes extraites, avec une clé primaire. 3. **Lier** par des clés étrangères.

L'application sur le brut donne sept tables, dans l'ordre où il faut les créer (les tables **référées** avant les tables **faisant référence**) :

```

1. magasin(id_magasin, libelle)           -- 3 magasins
2. mode_paiement(id_mode, libelle, frais_pct,    -- 4 modes
   delai_encaissement_jours)
3. regle_tva(annee_fiscale, taux_tva)         -- 2 règles (2025 et 2026)
4. produit(id_produit, libelle, categorie,      -- 312 produits
   sous_categorie, fournisseur,
   prix_courant_ht)
5. client(id_client, nom, prenom, telephone,    -- 1 891 clients
   email, ville, code_postal, region,
   pays, fidelite_niveau,
   fidelite_points)
6. client_categorie_pref(id_client, categorie)  -- table de jointure N-M
7. vente(id_vente, id_ticket, id_client,        -- 24 000 ventes
   id_magasin, id_vendeur, id_mode,
   date_vente, annee_fiscale, canal,
   est_promo, horodatage)
8. ligne_vente(id_vente, id_produit,            -- N lignes par vente
   quantite, prix_unitaire_ht,
   montant_ht, montant_remise_ht)
9. retour(id_retour, id_vente, motif,          -- 5 % des ventes
   montant_rembourse)

```

Les 8 colonnes supprimées (`client_nom`, `client_prenom`, `client_telephone`, `client_email`, `produit_libelle`, `produit_categorie`, `produit_sous_categorie`, `fournisseur`, `frais_pct`, `delai_encaissement_jours`, `taux_tva_courant`, `ville_client`, `categorie_pref`) sont **toutes des colonnes transitaires ou redondantes** : leur valeur est portée par la table de référence, et la valeur **calculable** (TVA, ville) est lue par jointure à la demande.

5.4 Vue calculée et intégrité — ce qui remplace les colonnes supprimées

La **vue** `ligne_vente_ttc` (du `schema_3fn.sql`) recalcule à la lecture le **montant TTC** et le **montant TVA** par jointure avec `regle_tva`. C'est l'art du relationnel : la **donnée stockée** est minimale (juste ce qui ne peut pas se calculer), la **donnée calculée** est lue par vue.

```

CREATE VIEW ligne_vente_ttc AS
SELECT lv.*,
       ROUND(lv.montant_ht * (1 + rt.taux_tva)) AS montant_ttc,
       ROUND(lv.montant_ht * rt.taux_tva) AS montant_tva
FROM ligne_vente lv
JOIN vente v ON lv.id_vente = v.id_vente
JOIN regle_tva rt ON v.annee_fiscale = rt.annee_fiscale;

```

Cette vue est **équivalente** à la lecture du brut, mais elle est **fiable** : si le taux de TVA change au 1^{er} janvier, il suffit de modifier **une ligne** de la table `regle_tva`, et toutes les ventes 2026 voient le bon taux.

5.5 Les 3 requêtes qui ferment le module

Les **3 requêtes** du `controles_integrite.sql` sont la **garantie** que la base est intègre. Chaque requête renvoie **0 ligne** quand tout va bien :

Q1 — Intégrité référentielle. Aucune vente ne pointe sur un client qui n'existe pas :

```

SELECT v.id_vente, v.id_client
FROM vente v LEFT JOIN client c ON v.id_client = c.id_client
WHERE c.id_client IS NULL;

```

Le `LEFT JOIN` ramène les ventes qui n'ont pas de correspondance ; le `WHERE c.id_client IS NULL` filtre celles qui pointent dans le vide. Attendu : **0 ligne** après import.

Q2 — Cohérence des montants. Chaque `ligne_vente.montant_ht` est bien `quantite × prix_unitaire_ht` :

```
SELECT id_vente, id_produit, quantite, prix_unitaire_ht, montant_ht,
       (quantite * prix_unitaire_ht) AS attendu
FROM ligne_vente
WHERE montant_ht != quantite * prix_unitaire_ht;
```

Attendu : **0 ligne**. Si la requête renvoie des lignes, c’est que l’import a chargé des valeurs incorrectes (typiquement des montants en texte non convertis).

Q3 — Total par catégorie. Le total `SUM(montant_ht)` par catégorie correspond au brut original :

```
SELECT p.categorie, SUM(lv.montant_ht) AS total_categorie
FROM ligne_vente lv JOIN produit p ON lv.id_produit = p.id_produit
GROUP BY p.categorie ORDER BY total_categorie DESC;
```

Attendu : **5 lignes** (Alimentaire, Bricolage, Jardinage, Quincaillerie, Décoration), avec un total qui correspond au brut original.

5.6 Les 5 critères du choix DuckDB / SQLite / PostgreSQL

Le passage Excel → SGBD n’a **pas une seule réponse**. Trois moteurs coexistent dans le module, et le choix dépend des **5 critères** suivants (les 4 critères du C05 §5.4 + 1 nouveau, la simplicité) :

Critère	DuckDB (intégré)	SQLite (embarqué)	PostgreSQL (serveur)
Taille	bon jusqu’à 50 M	bon jusqu’à 10 M	illimité
Fréquence	batch (ponctuel)	batch ou application	transactionnel
Compétences	Python ou CLI	Python ou CLI	SQL + administration
Gouvernance	fichier local	fichier local	serveur centralisé
Simplicité de mise en place	aucune (<code>pip install duckdb</code>)	aucune (natif Python)	serveur à installer

La règle du module : DuckDB en première intention (à 24 000 lignes, l’usage est analyse et DuckDB est imbattable), SQLite pour une application embarquée (un fichier qu’on copie), PostgreSQL cité pour quand l’équipe grandit (la base multi-utilisateurs, avec serveur et sauvegardes automatiques). Le module M06 enseigne DuckDB et cite SQLite/PostgreSQL, parce que c’est l’usage direct de l’analyste.

5.7 « Quand basculer ? » — la règle des 100 000 lignes et le verdict

La **règle des 100 000 lignes** (cf. M05 C05 §5.1) ne suffit pas. Il faut **trois conditions** pour basculer du classeur au SGBD :

1. **Taille** > 100 000 lignes OU **volumétrie** qui grossit (multiplication par an) ;
2. **Jointures** ≥ 3 tables (au-delà, le classeur s’effondre en RECHERCHEV imbriquées) ;
3. **Concurrence** ≥ 2 utilisateurs simultanés OU **gouvernance** qui exige une trace d’audit.

Le projet M06.P coche **les trois cases** : 24 000 lignes aujourd’hui mais 240 000 attendues (×10 en 2 ans), 7 tables à joindre (vente × client × produit × paiement × TVA × magasin × ligne_vente), et la caissière de Koudougou qui ouvre en même temps que la responsable de Bobo-Dioulasso. C’est flagrant qu’il faut un SGBD. Pour un cas limite (50 000 lignes, 1 utilisateur, 2 tables), le classeur peut encore tenir — au prix de la rigueur.

6. Exemple concret — le verdict de la bascule

6.1 L'export `quincaillerie_export.csv` côté Excel

Un classeur ouvert sur l'export : - **24 000 lignes** : filtre sur `client_ville = "Koudougou"` en 4 secondes (acceptable). - **42 colonnes** : tri par `produit_categorie` puis `client_nom` en 2 secondes (acceptable). - **Calcul** : `SOMME(montant_ttc)` ramène un total sur 24 000 lignes en 8 secondes, mais **142 lignes de montants texte** sont **ignorées** (cf. C05 §5.6) : le total est faux. Le classeur ne le dit pas. - **Dédoublonnage** : `supprimer les doublons` sur la colonne `id_ticket` en 30 secondes, mais **ne traite pas les copies partielles** (8 lignes qui ne diffèrent que sur la date — invisibles au dédoublonnage Excel).

Le verdict. Le classeur ouvre, trie, filtre, mais **ne vérifie pas** et **ne reproduit pas**. La règle est classeur pour voir, SGBD pour vérifier (cf. § 5.7).

6.2 Le même fichier dans DuckDB

Le même fichier dans DuckDB (Python ou CLI) :

```
import duckdb
con = duckdb.connect()
con.execute("CREATE TABLE export AS SELECT * FROM read_csv_auto('quincaillerie_export.csv')")
print(con.execute("SELECT COUNT(*) FROM export").fetchone())      # 24 000 lignes en 80 ms
print(con.execute("SELECT SUM(montant_ttc) FROM export").fetchone()) # somme en 200 ms
```

- 24 000 lignes lues en **80 ms** (DuckDB est ~30 × plus rapide que pandas pour l'import, et Excel ne sait même pas mesurer).
- La somme `SUM(montant_ttc)` est **faux aussi** : DuckDB autoloade convertit « 1 200 FCFA » en 1200 et les ajoute au total, mais signale la conversion dans les **logs**. Le défaut n'est plus silencieux comme dans Excel : il est visible.
- **3 requêtes d'intégrité** (cf. § 5.5) sont exécutables en quelques lignes, et chacune documente son verdict par écrit.

Le verdict. DuckDB fait plus vite ce que le classeur fait plus lentement, mais surtout il **documente** ce qu'il fait (logs) et **vérifie** ce que le classeur ne vérifie pas (les 3 requêtes).

6.3 Ce qu'un SGBD ne fait pas (par construction)

Le SGBD **n'efface pas** les défauts métier du brut — il les rend visibles. Si l'export a 142 montants texte, ils sont là, et les 142 lignes sont **dans la base** au lieu d'être silencieusement ignorées. Le SGBD **ne décide pas** non plus : `INSERT` ne fait que insérer. C'est l'import qui doit convertir, et c'est l'import qui doit rejeter les lignes non conformes. La règle du module est **import + 3 requêtes** = base propre.

7. Démonstration pas à pas — comment basculer, en 4 étapes

7.1 Importer le CSV

Dans DuckDB :

```
CREATE TABLE export_brut AS
SELECT * FROM read_csv_auto('quincaillerie_export.csv', all_varchar=true);
```

Le `all_varchar=true` force toutes les colonnes en texte — c’est l’étape « chargement brut », où rien n’est converti. La table `export_brut` reflète le fichier.

7.2 Filtrer les défauts métier

```
CREATE TABLE export_filtre AS
SELECT *
FROM export_brut
WHERE id_client IS NOT NULL          -- 3 clients fantômes écartés
  AND id_client IN (SELECT id_client FROM client) -- intégrité référentielle
  AND TRY_CAST(montant_ttc AS BIGINT) IS NOT NULL -- 142 montants texte écartés
  AND date_vente ~ '^\d{4}-\d{2}-\d{2}$';      -- 12 dates mal formées écartées
```

Le filtre **applique** les règles métier avant l’import dans la table finale.

7.3 Définir les 7 tables en 3FN

```
CREATE TABLE client (
  id_client  INTEGER PRIMARY KEY,
  nom        TEXT NOT NULL,
  prenom     TEXT NOT NULL,
  telephone  TEXT NOT NULL UNIQUE,
  email      TEXT UNIQUE,
  ville      TEXT NOT NULL,
  fidelite_niveau TEXT NOT NULL DEFAULT 'COMPTOIR'
);

CREATE TABLE vente (
  id_vente   INTEGER PRIMARY KEY,
  id_client  INTEGER NOT NULL REFERENCES client(id_client),
  id_magasin INTEGER NOT NULL,
  id_mode    INTEGER NOT NULL,
  date_vente DATE NOT NULL,
  annee_fiscale INTEGER NOT NULL
);
```

Et ainsi de suite pour les 7 tables.

7.4 Vérifier l’intégrité par les 3 requêtes

Reprendre `controles_integrite.sql` et l’exécuter :

```
-- Q1, Q2, Q3 (cf. § 5.5).
```

À retenir. Les 4 étapes du passage Excel → SGBD sont : **importer brut**, **filtrer les défauts**, **schématiser en 3FN**, **vérifier l’intégrité**. C’est la même séquence que M05 (transformation + dédoublonnage + jointure + verdict), appliquée à un SGBD. La différence est l’automatisation : une fois l’import écrit, il est rejouable (paramétrable par date, par magasin, etc.).

8. Erreurs fréquentes

- **Croire que le SGBD résout les défauts métier du brut.** Le SGBD les voit, mais il ne les corrige pas — c’est l’import qui filtre. Une base qui contient 142 montants texte est une **mauvaise base**, même si elle est en 3FN.

- **Confondre redondance et performance.** Parfois, dénormaliser une colonne accélère la lecture (par exemple, mettre la `ville_client` directement dans la table `vente`). Mais c'est un compromis documenté : on garde la redondance quand on sait qu'on perd la mise à jour automatique. La règle du module est 3FN par défaut, redondance calculable par vue quand le besoin est mesuré.
- **Croire que la 1FN et la 3FN sont « pour les puristes ».** Un client qui a deux catégories préférées ne tient pas dans une seule colonne. Croire qu'on peut « concaténer » (« bricolage,jardinage ») est le piège classique : ça marche à l'affichage, ça casse à la jointure.
- **Importer sans vérifier.** Le `CREATE TABLE` ne vérifie rien. Seules les **3 requêtes** vérifient. Une base qui n'a pas été testée par les 3 requêtes n'est pas une base — c'est un dépôt.
- **Vouloir tout stocker.** Stocker la TVA ou le TTC dans la base empêche la mise à jour du taux. La règle est : **stocker le minimum, calculer le reste par vue**. La TVA est dans `regle_tva`, le TTC est dans la vue `ligne_vente_ttc`.
- **Utiliser PostgreSQL pour 24 000 lignes.** C'est comme prendre un semi-remorque pour livrer une pizza : ça marche, mais c'est surdimensionné. DuckDB fait le même travail sans serveur.
- **Sous-estimer la migration des données.** Importer 24 000 lignes d'un CSV dans 7 tables prend du temps — pas en minutes, mais en logique (les défauts à filtrer, les conversions à appliquer). La règle est prévoir un import contrôlé par script, pas un copier-coller Excel.

À retenir. Les 7 erreurs ci-dessus coûtent toutes le **verdict du projet M06.P**. Le projet qui oublie de filtrer les défauts métier a un total faux ; le projet qui ne vérifie pas l'intégrité a une base qui semble correcte mais ne l'est pas. La leçon est **l'import n'est pas la base — la base, c'est l'import plus les 3 contrôles**.

9. Bonnes pratiques professionnelles

- **Documenter chaque table dans le README** (le projet M06.P E5) avec son **grain** (« une ligne = un client », « une ligne = une vente »). Le grain perdu, la table est incompréhensible.
- **Tester l'intégrité par 3 requêtes** au minimum : (1) référentielle, (2) cohérence métier, (3) rejouabilité (cf. § 5.5). Sans ces 3 requêtes, la base n'est pas livrable.
- **Paramétrer l'import** par date (`WHERE date_vente BETWEEN ...`) plutôt que par fichier. Le même script doit pouvoir réimporter le mois suivant sans modification du code.
- **Versionner le schéma SQL** dans un fichier `schema_3fn.sql` commité, pas dans un notebook interactif. La base est un artefact reproductible, pas une improvisation.
- **Mesurer** l'écart entre le total du brut et le total de la base après import. Si l'écart est 0 ou faible (les défauts rejetés), la base est cohérente. Si l'écart est important, l'import a perdu des lignes.
- **Documenter les défauts rejetés** dans le journal d'import. Le projet M06.P attend un rapport de cette forme : « 142 lignes écartées pour montant texte (cf. C05 §5.6) ; 12 lignes pour date mal formée (cf. M04 E4) ; 8 lignes pour doublon (cf. M05 C03) ; 3 lignes pour client orphelin (cf. contrôle Q1) ».

Conseil professionnel. Le verdict d'une base de données n'est pas « elle marche » ; c'est « les 3 requêtes renvoient 0 ligne ». C'est cette discipline qui distingue une base d'un fichier Excel. Et c'est ce que le projet M06.P évalue.

10. Exercice guidé — la lettre au stagiaire (15 min, /10)

Objectif. Mobiliser les 5 limites du classeur sur un cas concret.

Énoncé. Vous êtes chargé d'étude dans une enseigne de distribution. Le directeur commercial vous remet un fichier `ventes_export.csv` (15 colonnes, 50 000 lignes, fait à partir du logiciel de caisse). Il dit : « tu n'as qu'à l'ouvrir dans Excel, ça doit suffire. » Vous avez 5 minutes pour répondre par mail : faut-il ouvrir dans Excel, basculer dans DuckDB, ou autre chose ?

Barème (/10).

Critère	Points
Cite au moins 3 des 5 limites du classeur	2
Identifie la limite décisive (volumétrie vs jointure vs concurrence)	2
Donne un verdict motivé (Excel ou DuckDB ou autre)	3
Chiffre les alternatives (combien de temps dans Excel, combien dans DuckDB)	2
Cite les 3 requêtes d'intégrité attendues	1
Total	10

Dans les faits. La réponse attendue : « Le fichier a 50 000 lignes (sous le seuil 100k), 1 table, pas de jointure multi-table visible, mono-utilisateur (votre service). Vous pouvez commencer dans Excel pour voir, basculer dans DuckDB dès qu'une deuxième table arrive (par exemple quand le service qualité veut croiser avec un fichier de lots). Les 3 requêtes d'intégrité attendues : Q1 référentielle, Q2 montants, Q3 total rejouable. » Le barème teste la **capacité à dire non à Excel** sans le dire par principe — c'est l'enseignement du § 5.7.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Citez les **5 limites du classeur Excel** et donnez, pour chacune, un chiffre du projet M06.P qui la rend visible.
- **Exercice 11.2 (30 min).** Sur le `quincaillerie_export.csv` téléchargé, ouvrez-le dans Excel, comptez le temps d'ouverture et le temps de filtre sur `ville_client = "Koudougou"`. Puis exécutez la même opération dans DuckDB et notez les temps. Commentez l'écart en 3 lignes.
- **Exercice 11.3 (45 min).** Identifiez, dans le brut livré, **un** défaut structurel qui n'est **pas** listé au § 5.1 (par exemple `montant_remise_ht` qui dépend d'une promotion et non de la vente). Rédigez le DDL de la table qui l'absorbe.
- **Exercice 11.4 (60 min, optionnel).** Réécrivez le verdict du § 5.5 en **PostgreSQL** plutôt qu'en DuckDB. Cherchez 3 différences syntaxiques et notez-les (par exemple : `TRY_CAST` DuckDB vs `CAST` PostgreSQL, `IS NOT DISTINCT FROM`, `ON CONFLICT`).

12. Correction détaillée

- **Exercice 11.1.** Limites : (1) redondance — la colonne `ville_client` est recopiée 13 × par client (`m06p_clients_uniques_references` = 1 891, 24 000 lignes) ; (2) anomalie de mise à jour — la TVA change tous les ans, 24 000 lignes à mettre à jour ; (3) multi-valué — `categorie_pref` ne porte qu'**une** catégorie par client ; (4) concurrence — la caissière de Koudougou et la responsable de Bobo-Dioulasso ouvrent en même temps ; (5) volumétrie — 24 000 lignes, c'est la frontière où Excel ralentit (8 secondes par recalcul).

- **Exercice 11.2.** Les temps attendus : Excel $\approx$ 4-8 secondes pour le filtre, DuckDB $\approx$ 80 ms pour l'import + 50 ms pour le filtre. **Ratio $\approx$ 100 x.** Si la machine est plus lente, le ratio reste du même ordre.
- **Exercice 11.3.** Un défaut **non listé** : `montant_remise_ht` (col 30) est nul dans 90 % des lignes, et porte 10 000 FCFA dans 10 % des lignes. Il dépend de la promotion (col 35 `est_promo`), pas de la vente. La parade est soit une table `promotion(id_promo, libelle, montant_remise)`, soit une simple **règle métier** dans le README (la remise est une constante de 10 000 pour toute vente en `est_promo = 1`). Le DDL attendu : `CREATE TABLE promotion (id_promo INTEGER PRIMARY KEY, libelle TEXT, montant_remise INTEGER NOT NULL CHECK (montant_remise > 0));` et `vente.est_promo` $\rightarrow$ pas de FK directe (les promos changent).
- **Exercice 11.4.** Trois différences DuckDB $\rightarrow$ PostgreSQL : (a) `TRY_CAST` DuckDB $\rightarrow$ `CAST` ou `(expression)::type` PostgreSQL, sans gestion d'erreur ; (b) `IS NOT DISTINCT FROM` est supporté par PostgreSQL (et DuckDB), pas par tous les SGBD ; (c) `ON CONFLICT (id) DO UPDATE` est PostgreSQL natif depuis 9.5 (upsert), DuckDB l'a aussi depuis 0.9. Le verdict : DuckDB est suffisamment compatible PostgreSQL pour le module, mais l'atelier ne fait pas l'aller-retour.

13. Mini-projet M06.P1 — « Le verdict de la bascule » (1 h)

Énoncé. Rédigez **1 page maximum** (PDF ou Markdown) qui répond, en phrases, à la question : « Le fichier `quincaillerie_export.csv` doit-il basculer dans un SGBD ? Si oui, lequel des trois (DuckDB / SQLite / PostgreSQL) ? »

Critères de réussite.

1. Les 5 limites du classeur sont citées **avec un chiffre du projet** chacune.
2. Les **3 critères limitants** (volumétrie, jointure, concurrence) sont **identifiés** sur le cas.
3. Le verdict (Excel / DuckDB / SQLite / PostgreSQL) est **motivé** par les 5 critères du § 5.6.
4. Une **règle de bascule** est formulée (« si le fichier double en 1 an, basculer dans DuckDB »).

Barème (/10). 2 points par critère sauf le 4 (règle de bascule) qui vaut 4 points.

14. Boîte à outils du chapitre

Boîte à outils. Six objets, un seul but — décider quand basculer du classeur au SGBD :

1. **Le tableau des 5 limites** (§ 5.1) — redondance, anomalie, multi-valué, concurrence, volumétrie.
2. **Le tableau des 7 défauts** (§ 5.2) — quatre structurels et trois métier, avec leur compteur dans `m06p_*`.
3. **Les 3 règles de bascule** (§ 5.7) — taille > 100 000 OU jointures ≥ 3 OU concurrence ≥ 2 .
4. **Le schéma des 7 tables** (§ 5.3) — la cible 3FN, à valider par le projet.
5. **La vue `ligne_vente_ttc`** (§ 5.4) — l'exemple type de « donnée calculée, pas stockée ».
6. **Les 3 requêtes d'intégrité** (§ 5.5) — Q1 référentielle, Q2 montants, Q3 rejouable.

15. Résumé du chapitre

- Le classeur a **5 limites** (§ 5.1) : redondance, anomalie de mise à jour, multi-valué, concurrence, volumétrie.

- La base de données est un **ensemble de tables reliées par des contraintes**, gérées par un SGBD (DuckDB, SQLite, PostgreSQL).
- L'export `quincaillerie_export.csv` (24 000 lignes × 42 colonnes) contient **4 défauts structuraux** (§ 5.1) — redondance de `ville_client`, multi-valué de `categorie_pref`, dépendance transitive de `taux_tva_courant`, référence de `mode_paiement` — et **4 défauts métier** (§ 5.2).
- La 3FN appliquée au brut donne **7 tables** (§ 5.3) : `magasin`, `mode_paiement`, `regle_tva`, `produit`, `client`, `client_categorie_pref`, `vente`, `ligne_vente` (+ `retour`).
- Les **3 requêtes** d'intégrité (§ 5.5) ferment le module : Q1 référentielle, Q2 cohérence des montants, Q3 jouable par catégorie.
- Les **5 critères du choix** SGBD (§ 5.6) : DuckDB en première intention, SQLite pour embarqué, PostgreSQL cité pour multi-utilisateurs.

16. À retenir

À retenir. Le SGBD ne résout pas les défauts du brut — il les rend visibles. C'est l'import qui filtre, et c'est l'import qui rejette. La 3FN est l'objectif, le `schema_3fn.sql` est la cible, les 3 requêtes sont la garantie. Le verdict est « les 3 requêtes renvoient 0 ligne », pas « la base fonctionne ». C'est cette discipline qui distingue un SGBD d'un classeur.

À retenir. La bascule Excel → SGBD a **3 critères limitants** : taille > 100 000 lignes, jointures ≥ 3, concurrence ≥ 2. Tant qu'aucun n'est franchi, le classeur tient ; dès qu'un seul l'est, basculer. Pour 24 000 lignes et 7 tables, **DuckDB** est le bon choix ; pour 250 millions et 200 utilisateurs simultanés, **PostgreSQL** l'est. Le critère taille du C05 §5.6 devient volumétrie + concurrence — la gouvernance locale ne suffit plus à elle seule.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 5 limites du classeur Excel. (Réponse : redondance, anomalie de mise à jour, multi-valué, concurrence, volumétrie.)

Question 2. Quels sont les 4 défauts structurels du `quincaillerie_export.csv` ? (Réponse : (1) `ville_client` redondante, (2) `categorie_pref` multi-valué, (3) `taux_tva_courant` en dépendance transitive, (4) `mode_paiement` en référence à externaliser.)

Question 3. Combien de tables le passage en 3FN produit-il dans le projet M06.P ? (Réponse : `m06p_tables_3fn_cible = 7`.)

Question 4. Pourquoi la **vue** `ligne_vente_ttc` est-elle préférable à une colonne stockée ? (Réponse : la TVA change chaque année ; si elle est stockée, il faut modifier 24 000 lignes ; si elle est en vue, il suffit de modifier une ligne de `regle_tva`.)

Question 5. Citez les 3 critères qui font basculer du classeur au SGBD. *(Réponse : (a) taille

100 000 lignes, (b) ≥ 3 tables à joindre, (c) ≥ 2 utilisateurs simultanés.)*

Question 6. Quel SGBD pour 240 000 lignes multi-utilisateurs ? (Réponse : PostgreSQL — le seul des trois qui tienne le multi-utilisateurs avec un serveur.)

Question 7. Que renvoient les 3 requêtes `controles_integrite.sql` quand la base est intègre ? (Réponse : Q1 → 0 ligne, Q2 → 0 ligne, Q3 → 5 lignes (les 5 catégories, avec leur total).)

Module M06.C02 — Table, colonne, ligne, schéma, type, contrainte, valeur nulle

Outils : DuckDB 1.5.5 (intégré), SQLite natif Python, PostgreSQL cité. **Durée indicative :** 6 h. **Niveau :** N2. **Prérequis :** M06.C01 + M01.C03.

L'idée du chapitre. Le C01 a posé **pourquoi** une base : redondance, anomalie, concurrence, volumétrie. Le C02 pose **de quoi** une base est faite : table, colonne, ligne, schéma, type, contrainte. Le vocabulaire du C02 est **transverse** : il vaut pour DuckDB, SQLite, PostgreSQL et au-delà — un professionnel qui le maîtrise passe d'un SGBD à l'autre sans réapprendre. Le chapitre **aligne le langage** (les 5 types de base, NULL n'est ni 0 ni vide, les 4 contraintes de colonne) et installe le **mini-projet M06.P1** : 3 tables de référence créées en DDL DuckDB. Le verdict du C02 est « **la table est un objet à 6 attributs** » : nom, colonnes typées, clé primaire, contraintes, insertion atomique, lecture en bloc.

Base de travail — le projet M06.P. C02 réutilise le `quincaillerie_export.csv` (24 000 lignes × 42 colonnes, livré par `tools/dossier_M06.py`), mais ne crée **aucune table projet** : le projet est l'affaire du C03 (clés + relations) et du mini-projet C03 §10. C02 ne crée que les **3 tables de référence** du mini-projet M06.P1 : `client`, `produit`, `mode_paiement`. Ces 3 tables sont le squelette du schéma 3FN que le C03 et le projet M06.P assemblent.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **définir** les 6 attributs d'une table : nom, schéma, colonnes, types, contraintes, lignes ;
- **distinguer** les 5 types SQL de base (numérique, texte, date, booléen, binaire) et leurs variantes DuckDB ;
- **comprendre** que NULL n'est ni 0 ni vide, et lire la logique ternaire (TRUE, FALSE, UNKNOWN) ;
- **utiliser** les 4 contraintes de colonne (NOT NULL, UNIQUE, CHECK, DEFAULT) et **tester** leur effet ;
- **créer** 3 tables de référence en DDL DuckDB, **insérer** 5 lignes, **vérifier** par requête de lecture ;
- **reconnaître** les erreurs courantes (NOT NULL violé, UNIQUE dupliqué, CHECK rejeté, NULL non géré).

2. Pourquoi cette notion est importante

- La **table** est l'unité de base du relationnel. Tout le reste (schéma, contraintes, requêtes) en découle. C'est l'objet que le professionnel doit savoir créer, lire, modifier, détruire. Le C02 installe la **création** ; le C03 installe les **relations** ; le C04 installe la **vie en production**.
- Les **5 types** (numérique, texte, date, booléen, binaire) couvrent 95 % des cas. DuckDB en ajoute 9 (HUGEINT, UUID, JSON, STRUCT, LIST, etc.) qui sont rarement nécessaires. Le C02 enseigne les 5, pas les 14 : c'est le **profil minimal** d'un analyste.
- NULL est le **piège statistique** n°1 : `SUM(col)` ignore les NULL, `COUNT(col)` aussi, mais `COUNT(*)` les compte. Sans comprendre NULL, on **produit des chiffres faux** sans s'en rendre compte. Le M01.B02 l'a enseigné pour le tableur ; le C02 l'enseigne pour le SQL.

- Les **4 contraintes de colonne** sont ce qui distingue la table d'un fichier texte. Sans contraintes, on peut écrire 1 client avec un `telephone` à 'abc' ou un `id_client` en double. Avec, le SGBD refuse. C'est la **discipline** du relationnel.
- Le **mini-projet M06.P1** (3 tables de référence en DDL DuckDB) est l'épreuve : il passe si les tables se créent, si les contraintes rejettent les violations, si les 5 lignes insérées se lisent. C'est avant le projet M06.P, qui assemble ces 3 tables aux 4 autres (vente, ligne_vente, retour, client_categorie_pref).

3. Explication simple — la table est un objet à 6 attributs

Une table est un objet à 6 attributs, **dans cet ordre** :

1. **Nom** — `client`, `produit`, `vente`, etc. Le nom est immuable : on ne renomme pas sans casser les requêtes.
2. **Schéma** — la liste ordonnée des colonnes, chacune avec son **nom**, son **type**, et ses **contraintes**. Le schéma est distinct des données : la table `client` a 11 colonnes, qu'elle contienne 0 ligne ou 1 891.
3. **Colonnes typées** — chaque colonne a un **type** (`INTEGER`, `TEXT`, `DATE`, etc.) qui contraint la valeur (DuckDB rejette 'abc' dans un `INTEGER`).
4. **Contraintes** — les **règles** que les données doivent suivre (`PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY`).
5. **Lignes** — les **données elles-mêmes**, qui respectent le schéma et les contraintes. Une ligne qui viole une contrainte est **refusée** à l'insertion.
6. **Clé primaire** — la colonne (ou combinaison) qui identifie la ligne. Sa présence est **facultative** d'un point de vue syntaxe, mais **obligatoire** d'un point de vue métier.

Le **résumé en une phrase** : « Une table, c'est un nom, un schéma typé, des contraintes, des lignes qui les respectent, et une clé pour les compter. »

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
table — table, relation	un ensemble de lignes (tuples) de même schéma, nommées par des colonnes typées.	confondre avec « onglet » : un onglet Excel a un entête et des lignes, mais sans schéma ni contrainte formelle.
colonne — column, attribute	— un slot typé dans la table. Chaque ligne porte une valeur pour chaque colonne.	croire que l'ordre des colonnes compte : il ne compte pas pour la logique (on les nomme), mais l'ordre physique peut affecter la performance.
ligne — row, tuple	une valeur pour chaque colonne, qui respecte le schéma et les contraintes.	confondre avec « enregistrement » : une ligne est typée par ses colonnes, un enregistrement est juste un paquet de données.
schéma — schema	— la structure d'une base (les noms de tables, les colonnes, les types, les contraintes), distincte des données .	confondre avec « table » : une base a un schéma (ou plusieurs namespaces) et N tables.
type — data type	la famille des valeurs qu'une colonne peut prendre (<code>INTEGER</code> , <code>TEXT</code> , <code>DATE</code> , <code>BOOLEAN</code> , etc.).	confondre avec « format » : le type est logique (DuckDB sait que c'est une date), le format est visuel (JJ/MM/AAAA est l'affichage).
NULL — null value	une absence de valeur , distincte de 0, de '' et de 'NA'. C'est le marqueur « pas de réponse ».	confondre avec 0 ou vide : <code>NULL + 1</code> n'est pas 1, c'est <code>NULL</code> ; la logique ternaire (<code>TRUE/FALSE/UNKNOWN</code>) traite <code>NULL</code> comme inconnu.
NOT NULL	contrainte qui interdit <code>NULL</code> sur la colonne. C'est l'engagement que la colonne porte toujours une valeur.	croire que c'est facultatif : <code>NOT NULL</code> rend la colonne porteuse de sens (une date manquante n'est pas <code>NULL</code> silencieuse).

Français — English	Définition simple	Piège à éviter
UNIQUE	contrainte qui interdit les doublons sur la colonne (ou la combinaison). C'est l'engagement que la colonne identifie de manière unique.	confondre avec PRIMARY KEY : UNIQUE accepte NULL, PRIMARY KEY non.
CHECK	contrainte qui valide une condition sur la valeur (par exemple <code>montant > 0</code> , ou <code>taux IN (0.18, 0.19)</code>).	croire qu'elle est universelle : CHECK peut être contournée par certaines opérations (COPY, LOAD DATA).
DEFAULT	la valeur par défaut quand l'insertion ne précise pas la colonne. C'est l'engagement sur la valeur en cas d'oubli.	confondre avec NULL implicite : DEFAULT n'est pas NULL, c'est une valeur que vous choisissez.

Attention. **UNIQUE n'est pas PRIMARY KEY** — c'est un piège classique, surtout en SQLite où **UNIQUE** accepte plusieurs **NULL**. Si vous voulez une ligne par valeur, sans **NULL**, utilisez **PRIMARY KEY** ou **UNIQUE NOT NULL**. Et si la colonne peut être **NULL**, dites-le (**UNIQUE** accepte alors des **NULL** non-dupliqués, ce qui veut dire « plusieurs lignes peuvent avoir la même absence » — c'est rarement ce qu'on attend).

Attention. **CHECK n'est pas une règle métier** — c'est un **garde-fou syntaxique**. La règle du module est **CHECK** valide l'évidence, pas la complexité. Un **CHECK** (`montant > 0`) rate les retours ; un **CHECK** (`montant != 0`) est correct mais **inutile** (tout 0 est probablement une erreur de saisie, pas une saisie vraie) ; un **CHECK** (`montant > -1 000 000 AND montant < 1 000 000`) est une plage par défaut, pas une règle métier. Pour les invariants métier, le code applicatif + les 3 requêtes `controles_integrite.sql` du C01 sont plus adaptés.

Dans les faits. En production, **la moitié des bugs de base viennent des contraintes oubliées**. Une table `email` sans **UNIQUE** accepte 50 clients avec le même e-mail, et c'est visible quand on essaie d'envoyer la newsletter (« 50 exemplaires à la même adresse »). Une table `prix_unitaire` sans **CHECK** (`prix > 0`) accepte un prix à `-1000`, et c'est visible quand le calcul de marge donne un résultat aberrant. Le test qui ferme le chapitre est **l'insertion négative** : insérer une ligne qui devrait être refusée, et vérifier qu'elle l'est. C'est l'épreuve du mini-projet M06.P1 §13. | **INTEGER** | le type **entier signé** sur 32 bits (de -2 147 483 648 à 2 147 483 647). Pour aller au-delà : **BIGINT**. | croire qu'il gère les décimales : pour les décimales, c'est **DOUBLE**, **DECIMAL** ou **NUMERIC**. | | **BIGINT** | le type **entier signé** sur 64 bits (de -9 223 372 036 854 775 808 à 9 223 372 036 854 775 807). Le défaut dans beaucoup de SGBD. | confondre avec **HUGEINT** (128 bits, DuckDB) : pour 99 % des cas, **BIGINT** suffit. | | **TEXT** | le type **chaîne** de longueur variable, sans limite stricte (DuckDB ne fixe pas de limite ; PostgreSQL impose une limite pratique). | croire que **VARCHAR(255)** est obligatoire : c'est l'habitude de l'époque mainframe, pas une nécessité. | | **DATE** | le type **date calendaire** (AAAA-MM-JJ), distinct de **TIMESTAMP** (qui ajoute l'heure). | confondre avec le format : `DATE '2025-03-15'` est un littéral date, `'15/03/2025'` est une chaîne. | | **BOOLEAN** | le type **booléen** à 3 valeurs (**TRUE**, **FALSE**, **NULL** en SQL ANSI ; DuckDB utilise **TRUE/FALSE/NULL**). | croire que c'est 0/1 : en SQL, c'est **TRUE/FALSE/NULL**, pas 0/1/2. | | **PRIMARY KEY** | la **clé primaire** : la colonne (ou combinaison) qui identifie la ligne. Implique **NOT NULL** + **UNIQUE**. | croire qu'elle est obligatoire : techniquement non, mais sans elle la base ne sait pas compter les lignes. |

Définition. Une **contrainte** est une **règle** que les données doivent suivre, **portée par la base elle-même** (pas par le code applicatif). Les 4 contraintes de colonne du C02 sont **NOT NULL**, **UNIQUE**, **CHECK**, **DEFAULT**. Les contraintes de **table** (cf. C03) sont **PRIMARY KEY** et **FOREIGN KEY**. Une contrainte **refuse** l'insertion qui la viole — c'est ce qui distingue une base d'un fichier texte. — English : constraint.

Définition. Le **type SQL** est la **famille** des valeurs qu'une colonne accepte. Les 5 types de base sont : numérique (**INTEGER**, **BIGINT**, **DOUBLE**, **DECIMAL**), texte (**TEXT**, **VARCHAR**), date (**DATE**, **TIMESTAMP**), booléen (**BOOLEAN**), binaire (**BLOB**). DuckDB en ajoute 9 (**JSON**, **UUID**, **LIST**, **STRUCT**, etc.) qui sont rarement nécessaires. Le type est logique : il valide à l'insertion (le SGBD refuse `'abc'` dans un **INTEGER**) et opère sur les valeurs (+ sur des **INTEGER** fait de l'arithmétique, pas de la concaténation). — English : data type.

Définition. La valeur **NULL** représente l'**absence de valeur**. Elle est distincte de `0`, de `''` (chaîne vide) et de `'NA'`. La logique ternaire SQL (**TRUE**, **FALSE**, **UNKNOWN**) traite **NULL** comme inconnu : `NULL = NULL` n'est pas **TRUE**, c'est **UNKNOWN**. Le test juste est **IS NULL** / **IS NOT NULL**. Les fonctions d'agrégat **ignorent** **NULL** par défaut : `SUM(col)` produit **NULL** si toutes les valeurs sont **NULL**, et ignore les **NULL** sinon (sans compter la ligne). — English : null value.

Définition. Le **DDL** (Data Definition Language) est le sous-ensemble du SQL qui crée, modifie, et supprime la **structure** : **CREATE TABLE**, **ALTER TABLE**, **DROP TABLE**, **CREATE INDEX**, **CREATE VIEW**. Le **DML** (Data Manipulation Language) touche aux **données** : **SELECT**, **INSERT**, **UPDATE**, **DELETE**. Le **DCL** (Data Control Language) gère les **droits** : **GRANT**, **REVOKE**. Le C02 enseigne le DDL ; le C03 et le C07 enseignent le DML. — English : Data Definition Language.

5. Cours approfondi — la grammaire d'une table

5.1 Le squelette d'un CREATE TABLE

La syntaxe canonique en DuckDB / PostgreSQL / SQLite est :

```
CREATE TABLE nom_table (
  nom_col1 TYPE CONTRAINTES,
  nom_col2 TYPE CONTRAINTES,
  ...
  PRIMARY KEY (col1, col2),          -- contrainte de table
  FOREIGN KEY (col3) REFERENCES autre_table(col3) -- contrainte de table (cf. C03)
);
```

Le **nom** est suivi des **colonnes**, chacune avec son **type** et ses **contraintes** (de colonne). Les contraintes de **table** (PRIMARY KEY, FOREIGN KEY) sont à la fin.

5.2 Les 5 types SQL de base, et leur usage

Type	Usage courant	Piège
INTEGER BIGINT	/ compteurs, identifiants, montants entiers (en FCFA)	croire que ça gère les décimales
DOUBLE DECIMAL	/ montants avec décimales, mesures physiques	DOUBLE perd en précision ; DECIMAL(p, s) est exact
TEXT	noms, libellés, descriptions	confondre avec VARCHAR(n) (longueur fixe)
DATE TIMESTAMP	/ dates de vente, dates de naissance	DATE '2025-03-15' est un littéral ; '15/03/2025' est une chaîne
BOOLEAN	flags (est_promo, est_retour)	en SQL c'est TRUE/FALSE/NULL, pas 0/1/NULL

DuckDB **étend** ces 5 types avec HUGEINT (128 bits), UUID, JSON, STRUCT, LIST, MAP, ENUM. Pour 95 % des cas d'usage, les 5 de base suffisent. Le projet M06.P n'utilise que INTEGER, TEXT, DATE, BOOLEAN, BIGINT (pour les montants en entier FCFA).

5.3 NULL et la logique ternaire

En SQL, NULL est une **valeur spéciale** qui signifie « **je ne sais pas** ». Ce n'est pas 0, ce n'est pas vide, ce n'est pas 'NA'. Les **comparaisons** avec NULL retournent UNKNOWN, pas TRUE ou FALSE :

```
SELECT NULL = 0;          -- NULL (UNKNOWN, pas FALSE)
SELECT NULL = NULL;       -- NULL (UNKNOWN, pas TRUE)
SELECT NULL IS NULL;      -- TRUE (le test juste)
SELECT NULL IS NOT NULL;  -- FALSE (le test juste)
```

La **logique ternaire** (TRUE, FALSE, UNKNOWN) affecte les agrégats. SUM(col) ignore les NULL, COUNT(col) aussi, mais COUNT(*) les compte. La règle du module est toujours tester IS NULL / IS NOT NULL, jamais = NULL (qui retourne UNKNOWN).

5.4 Les 4 contraintes de colonne

Contrainte	Effet	Exemple
NOT NULL	la colonne ne peut pas être NULL	telephone TEXT NOT NULL
UNIQUE	la colonne (ou combinaison) n'a pas de doublon	id_client INTEGER UNIQUE

Contrainte	Effet	Exemple
CHECK	la valeur doit satisfaire une condition	<code>prix_courant_ht INTEGER CHECK (prix_courant_ht > 0)</code>
DEFAULT	la valeur par défaut si l'insertion omet la colonne	<code>fidelite_niveau TEXT DEFAULT 'COMPTOIR'</code>

Une contrainte **PRIMARY KEY** est la combinaison **NOT NULL + UNIQUE** appliquée à la colonne désignée. Le C03 l'approfondit.

5.5 Erreurs courantes

Erreur	Symptôme	Parade
Oublier <code>NOT NULL</code>	la colonne est <code>NULL</code> par défaut	ajouter <code>NOT NULL</code> à la création (sinon, recréer la table)
Mettre <code>VARCHAR(255)</code> sans raison	les chaînes de plus de 255 caractères sont tronquées silencieusement	utiliser <code>TEXT</code> (DuckDB) ou <code>VARCHAR(N)</code> avec un N réaliste
<code>CHECK (montant_ttc > 0)</code> sur des retours	les retours (montants négatifs) sont refusés	<code>CHECK (montant_ttc != 0)</code> ou pas de <code>CHECK</code> (la règle métier est dans le code applicatif)
Comparer avec <code>= NULL</code>	retourne <code>UNKNOWN</code> , jamais <code>TRUE</code>	utiliser <code>IS NULL / IS NOT NULL</code>
Insérer des dates en <code>JJ/MM/AAAA</code>	DuckDB autoload devine (parfois faux)	utiliser <code>DATE 'AAAA-MM-JJ'</code> ou <code>STRPTIME(col, '%d/%m/%Y')</code>
Oublier <code>UNIQUE</code> sur email	plusieurs clients partagent le même e-mail	ajouter <code>UNIQUE</code> (et migrer les doublons d'abord)

5.6 La table `client` du mini-projet M06.P1, en DDL DuckDB

```
CREATE TABLE client (
  id_client    INTEGER PRIMARY KEY,
  nom          TEXT    NOT NULL,
  prenom       TEXT    NOT NULL,
  telephone    TEXT    NOT NULL UNIQUE,
  email        TEXT    UNIQUE,                      -- nullable : un client comptoir
  n'en a pas   TEXT    NOT NULL,
  ville        TEXT    NOT NULL,
  code_postal  TEXT,
  region       TEXT,
  pays         TEXT    NOT NULL DEFAULT 'Burkina Faso',
  fidelite_niveau TEXT    NOT NULL DEFAULT 'COMPTOIR'
  CHECK (fidelite_niveau IN ('COMPTOIR','BRONZE','OR','PLATINE')),
  fidelite_points INTEGER NOT NULL DEFAULT 0 CHECK (fidelite_points >= 0)
);
```

Lecture de la table : - **id_client** : la clé primaire, `INTEGER`, identifie chaque ligne. - **nom** et **prenom** : `TEXT NOT NULL`, on refuse un client sans nom. - **telephone** : `TEXT NOT NULL UNIQUE`, on refuse un téléphone vide ET un doublon. - **email** : `TEXT UNIQUE` (sans `NOT NULL`), un client comptoir peut ne pas avoir d'e-mail ; mais deux clients ne partagent pas le même e-mail. - **ville** : `TEXT NOT NULL`, on refuse un client sans ville. - **pays** : `TEXT NOT NULL DEFAULT 'Burkina Faso'`, valeur par défaut pour la quincaillerie locale. - **fidelite_niveau** : `CHECK` qui contraint à 4 valeurs admises. - **fidelite_points** : `CHECK (>= 0)`, on refuse les points négatifs.

6. Exemple concret — les 3 tables de référence du mini-projet M06.P1

6.1 Table `client` (vue § 5.6)

C'est la table **principale** du projet M06.P — 1 891 lignes à terme, et la **cible** des clés étrangères de vente, retour, client_categorie_pref. La clé primaire est `id_client`, et l'**unicité** est sur `telephone` (clé métier, pas technique).

6.2 Table `produit` en DDL DuckDB

```
CREATE TABLE produit (
  id_produit      INTEGER PRIMARY KEY,
  libelle         TEXT      NOT NULL,
  categorie       TEXT      NOT NULL,
  sous_categorie TEXT      NOT NULL,
  fournisseur     TEXT      NOT NULL,
  prix_courant_ht INTEGER NOT NULL CHECK (prix_courant_ht > 0),
  UNIQUE (libelle, fournisseur)      -- un produit est unique par (nom, fournisseur)
);
```

Cette table respecte la **3FN** : pas de transitivité, pas de multi-valué. Le `prix_courant_ht` est l'attribut propre du produit (pas d'une autre entité).

6.3 Table `mode_paiement` en DDL DuckDB

```
CREATE TABLE mode_paiement (
  id_mode          INTEGER PRIMARY KEY,
  libelle          TEXT      NOT NULL UNIQUE,
  frais_pct        REAL      NOT NULL DEFAULT 0 CHECK (frais_pct >= 0),
  delai_encaissement_jours INTEGER NOT NULL DEFAULT 0
);
```

Les 4 valeurs canoniques (`ESPECES`, `MOBILE_MONEY`, `CARTE_BANCAIRE`, `CREDIT`) sont insérées une fois, et la table `vente` y fait référence par `id_mode`. C'est l'**externalisation de la référence** du § 1.4.

6.4 Insertion de 5 lignes et lecture de contrôle

```
INSERT INTO client (id_client, nom, prenom, telephone, email, ville, fidelite_niveau) VALUES
(0, 'COMPTOIR', 'CLIENT', '0000000000', NULL,
'Koudougou', 'COMPTOIR'),
(1, 'Nom0001', 'Prenom0001', '+226 78 12 34 56 78', 'p1.n1@example.bf',
'Koudougou', 'OR'),
(2, 'Nom0002', 'Prenom0002', '+226 78 12 34 56 79', 'p2.n2@example.bf',
'Bobo-Dioulasso', 'BRONZE'),
(3, 'Nom0003', 'Prenom0003', '+226 78 12 34 56 80', NULL,
'Ouagadougou', 'COMPTOIR'),
(4, 'Nom0004', 'Prenom0004', '+226 78 12 34 56 81', 'p4.n4@example.bf',
'Banfora', 'PLATINE');

SELECT COUNT(*) AS nb_clients FROM client;           -- 5
SELECT * FROM client WHERE fidelite_points > 0;      -- 1 si on a inséré des
points
SELECT * FROM client WHERE fidelite_niveau = 'COMPTOIR'; -- 2
SELECT * FROM client WHERE email IS NULL;           -- 2
```

Ce mini-projet **valide** la création : si DuckDB renvoie 5 pour `COUNT(*)`, si les contraintes refusent les violations (test : `INSERT INTO client (id_client, nom, prenom, telephone) VALUES (5, NULL, 'X', '+226 X')` — `NOT NULL` violé sur `nom`), si les tris par `fidelite_niveau` renvoient 2 lignes, alors la table est **opérationnelle**. C'est l'épreuve du C02.

7. Démonstration pas à pas — créer une table en 6 lignes

7.1 Choisir le nom

client. Pas de pluriel, pas de préfixe (sauf contexte). En SQL ANSI, le nom est insensible à la casse.

7.2 Lister les colonnes

11 colonnes : `id_client`, `nom`, `prenom`, `telephone`, `email`, `ville`, `code_postal`, `region`, `pays`, `fidelite_niveau`, `fidelite_points`.

7.3 Typer chaque colonne

5 types : `INTEGER` pour `id_client` et `fidelite_points` ; `TEXT` pour tout le reste.

7.4 Ajouter les contraintes

`NOT NULL` sur les colonnes porteuses de sens (`nom`, `prenom`, `telephone`, `ville`, `pays`, `fidelite_niveau`) ; `UNIQUE` sur `telephone` et `email` ; `DEFAULT` sur `pays` et `fidelite_niveau` et `fidelite_points` ; `CHECK` sur `fidelite_niveau` (4 valeurs) et `fidelite_points` (≥ 0) ; `PRIMARY KEY` sur `id_client`.

7.5 Ordonner les contraintes de table

`PRIMARY KEY` (et `FOREIGN KEY`, cf. C03) en bas du `CREATE TABLE`.

7.6 Tester en 3 requêtes

`INSERT`, lecture (`SELECT *`), contrainte violée (test négatif).

À retenir. Les 6 étapes du `CREATE TABLE` sont **mécaniques** : nom, colonnes, types, contraintes, ordre, test. La discipline est de ne pas en sauter. Une table sans contrainte de colonne est un fichier texte ; avec, c'est une **table relationnelle**.

8. Erreurs fréquentes

- **Oublier `NOT NULL` sur la clé primaire.** Une `PRIMARY KEY` impose `NOT NULL` ; une colonne ordinaire `INTEGER` accepte `NULL`. La règle est `PRIMARY KEY = NOT NULL + UNIQUE` implicite.
- **`CHECK` trop restrictif.** Un `CHECK (montant > 0)` rate les retours (montants **négatifs**). Le piège classique : on copie la règle métier dans la base, et la base refuse les cas métier qui paraissent marginaux mais qui existent. La règle du module : **`CHECK` doit valider l'évidence, pas la complexité.**
- **`TEXT` vs `VARCHAR(N)`.** `TEXT` (DuckDB, PostgreSQL) est sans limite ; `VARCHAR(N)` tronque à N caractères. Utiliser `VARCHAR(255)` par défaut est une habitude qui **coûte des données** quand un nom dépasse 255 caractères (les noms composés africain en témoignent).
- **`BOOLEAN` et `INTEGER`.** En SQL `BOOLEAN` n'est pas un `INTEGER`. DuckDB peut **convertir** implicitement, mais la lecture est ambiguë. La règle : `BOOLEAN` pour les flags, `INTEGER` pour les quantités.

- **Confondre DEFAULT et NOT NULL.** DEFAULT ne refuse pas NULL ; il propose une valeur par défaut quand l'insertion omet la colonne. Si l'insertion **précise** NULL, c'est NULL. Pour refuser NULL, il faut NOT NULL.
- **Sous-estimer la différence de comportement entre SGBD.** SQLite ne respecte pas CHECK avant la 3.32 (2020) ; PostgreSQL applique CHECK par défaut mais permet INITIALLY DEFERRED ; DuckDB applique CHECK immédiatement. Le **mini-projet** DuckDB ne garantit pas la portabilité : le code doit être testé sur le SGBD cible.
- **Croire que UNIQUE suffit à faire une clé primaire.** UNIQUE accepte NULL (plusieurs NULL sont autorisés — DuckDB et PostgreSQL divergent ici). Une **PRIMARY KEY** est NOT NULL + UNIQUE, point.

À retenir. Les 7 erreurs ci-dessus sont les **pièges du quotidien** d'un analyste qui crée une table. La règle unique : tester la table par une insertion qui doit échouer (INSERT ... VALUES (NULL, NULL) sur une colonne NOT NULL). Sans ce test, on ne sait pas si la contrainte est portée par la base ou par l'espoir.

9. Bonnes pratiques professionnelles

- **Lire la documentation des types** de votre SGBD. DuckDB a 14 types, PostgreSQL a 30+, SQLite en a 5. Le **profil minimal** est les 5 de base.
- **NOT NULL par défaut** sauf raison. Une colonne nullable est une **permission d'ignorer**, et c'est rarement ce qu'on veut.
- **UNIQUE** sur les clés **métier** (téléphone, e-mail, n° de série), pas sur les clés techniques (id_vente). Les clés techniques sont immuables par construction (auto-incrément) ; les clés métier sont immuables par contrainte.
- **CHECK** sur les **valeurs énumérées** (fidelite_niveau IN (4 valeurs)), pas sur les **bornes métier** (montant > 0 rate les retours). Les énumérations sont stables ; les bornes métier changent.
- **Documenter** chaque table dans le README du projet : nom, grain, 1 phrase d'usage.
- **Tester** la table par **insertion négative** (la violation attendue doit lever une erreur). C'est l'épreuve de vérité.

Conseil professionnel. Une table bien dessinée est **auto-vérifiée** : si vous insérez une ligne qui viole une règle métier, le SGBD refuse, et c'est visible. Si la table passe tout (toutes les insertions sont acceptées), c'est qu'elle ne porte pas assez de contraintes — ou que vos insertions n'ont pas testé les bons cas.

10. Exercice guidé — la table `mode_paiement` (15 min, /10)

Objectif. Mobiliser le CREATE TABLE sur 4 colonnes et 2 contraintes.

Énoncé. Créez la table `mode_paiement` du mini-projet M06.P1 en DDL DuckDB, avec les colonnes :

- `id_mode` INTEGER PRIMARY KEY
- `libelle` TEXT NOT NULL UNIQUE
- `frais_pct` REAL NOT NULL DEFAULT 0 CHECK (frais_pct >= 0)
- `delai_encaissement_jours` INTEGER NOT NULL DEFAULT 0

Insérez les 4 lignes (ESPECES, MOBILE_MONEY, CARTE_BANCAIRE, CREDIT), testez les contraintes (insertion d'un frais_pct négatif : doit échouer).

Barème (/10). DDL correct (3) · insertion des 4 lignes (2) · test négatif (2) · tri par frais_pct DESC (1) · filtrage par delai_encaissement_jours > 0 (1) · SELECT * rendu final (1).

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Citez les 6 attributs d'une table (§ 3) et donnez, pour chacun, un exemple tiré du mini-projet M06.P1.
- **Exercice 11.2 (30 min).** Créez la table client du mini-projet M06.P1 dans un autre SGBD (SQLite par Python, ou PostgreSQL par psql si votre poste l'a). Notez 3 différences de syntaxe.
- **Exercice 11.3 (45 min).** Insérez une ligne qui viole **chacune** des 4 contraintes (NOT NULL, UNIQUE, CHECK, DEFAULT) sur la table client, et notez le message d'erreur de DuckDB. Ces messages sont ce que les utilisateurs voient.
- **Exercice 11.4 (60 min, optionnel).** Réécrivez la table produit du § 6.2 en ajoutant une contrainte CHECK qui valide que categorie IN ('Alimentaire', 'Bricolage', 'Jardinage', 'Quincaillerie', 'Décoration'). Notez l'effet sur la lisibilité du DDL et le risque d'erreur quand on ajoute une 6^e catégorie.

12. Correction détaillée

- **Exercice 11.1.** Attributs : (1) nom = client ; (2) schéma = 11 colonnes ; (3) colonnes typées = INTEGER, TEXT ; (4) contraintes = NOT NULL, UNIQUE, CHECK, DEFAULT ; (5) lignes = 5 insérées ; (6) clé primaire = id_client.
- **Exercice 11.2.** Différences attendues : SQLite natif = pas de BOOLEAN natif (INTEGER 0/1) ; pas de REAL (utilise NUMERIC ou FLOAT) ; PostgreSQL = SERIAL pour auto-incrément (DuckDB n'en a pas, on utilise INTEGER plus une séquence applicative).
- **Exercice 11.3.** Les 4 violations : (a) NOT NULL : INSERT INTO client (id_client, nom, prenom, telephone, ville) VALUES (5, NULL, 'X', '+226 X', 'K') — DuckDB renvoie NOT NULL constraint failed: client.nom ; (b) UNIQUE : insérer deux fois le même telephone — DuckDB renvoie UNIQUE constraint failed: client.telephone ; (c) CHECK : INSERT ... VALUES (5, 'N', 'P', '+226', 'e@e.bf', 'V', 'PLATINE Russe') — DuckDB renvoie CHECK constraint failed: client.fidelite_niveau ; (d) DEFAULT : tester en omettant pays dans une insertion (DuckDB insère 'Burkina Faso').
- **Exercice 11.4.** L'effet : le DDL devient plus bavard (la liste des 5 catégories est explicite), mais c'est une garantie supplémentaire contre les fautes de frappe. Le risque : si on ajoute une 6^e catégorie (par exemple « Outillage »), il faut modifier le DDL **et** migrer les anciennes lignes qui avaient une autre catégorie. La règle du module est DDL paramétrable quand la liste est stable. Pour la quincaillerie, **5 catégories** est probablement stable.

13. Mini-projet M06.P1 — « Les 3 tables de référence » (1 h)

Énoncé. Créez les **3 tables de référence** du projet M06.P en DDL DuckDB : - client (§ 5.6 — 11 colonnes, contraintes complètes) - produit (§ 6.2 — 6 colonnes, UNIQUE (libelle, fournisseur)) - mode_paiement (§ 6.3 — 4 colonnes, UNIQUE sur libelle)

Insérez **5 lignes** dans chaque table, testez **2 violations** par table, et rendez **1 page** de sortie (les `SELECT *` finaux).

Critères de réussite.

1. Les 3 `CREATE TABLE` passent sans erreur (DDL valide).
2. Les **5 lignes** par table sont insérées (`INSERT` accepté).
3. Les **2 tests négatifs** par table (6 au total) déclenchent une erreur DuckDB.
4. Les **3 lectures finales** renvoient 5 lignes chacune.
5. La **page rendue** montre les `INSERT`, les `SELECT *`, et les messages d'erreur des tests négatifs.

Barème (/10). 2 points par table ; le critère 5 vaut 1 point, le critère 1 vaut 1 point.

14. Boîte à outils du chapitre

Boîte à outils. Six objets, un seul but — créer une table relationnelle qui se vérifie elle-même :

1. **Les 5 types SQL de base** (§ 5.2) — `INTEGER`, `TEXT`, `DATE`, `BOOLEAN`, `BLOB` (+ variantes).
2. **Les 4 contraintes de colonne** (§ 5.4) — `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`.
3. **La logique ternaire NULL** (§ 5.3) — `TRUE`, `FALSE`, `UNKNOWN`, et `IS NULL`.
4. **Le squelette CREATE TABLE** (§ 5.1) — nom, colonnes, types, contraintes, ordre.
5. **La table client complète** (§ 5.6) — l'exemple type de DDL avec 11 colonnes et 6 contraintes.
6. **Le mini-projet M06.P1** (§ 13) — l'épreuve en 1 h, qui passe si les 3 tables se créent et se testent.

15. Résumé du chapitre

- Une table est un objet à **6 attributs** : nom, schéma, colonnes typées, contraintes, lignes, clé primaire.
- Les **5 types SQL de base** sont : numérique, texte, date, booléen, binaire (DuckDB en ajoute 9).
- `NULL` est une **absence de valeur**, distincte de `0` et de `''`, traité par la logique ternaire (`TRUE`, `FALSE`, `UNKNOWN`).
- Les **4 contraintes de colonne** sont `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`. Les contraintes de table (cf. C03) sont `PRIMARY KEY` et `FOREIGN KEY`.
- La **table client** du mini-projet M06.P1 a 11 colonnes, dont 5 `NOT NULL`, 2 `UNIQUE`, 2 `CHECK`, 3 `DEFAULT`, et `id_client` `PRIMARY KEY`.
- Le **mini-projet M06.P1** crée 3 tables (`client`, `produit`, `mode_paiement`) et teste les contraintes par **insertion négative** (la violation attendue doit échouer).

16. À retenir

À retenir. Une **table relationnelle** est un fichier de règles, pas un fichier de valeurs. Les valeurs sont les lignes ; les **règles** sont les contraintes (`NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`) et les types (`INTEGER`, `TEXT`, `DATE`). Le SGBD **porte** ces règles, et c'est cela qui distingue une base d'un fichier texte. Le test qui ferme le chapitre est **l'insertion négative** : si elle passe, la contrainte est faible ; si elle échoue, la contrainte est forte.

À retenir. `NULL` n'est ni 0 ni vide, et le test juste est `IS NULL` / `IS NOT NULL`. Le piège classique est `col = NULL` qui retourne `UNKNOWN` (jamais `TRUE`). La règle du module est **`IS NULL`** partout où on cherche l'absence.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 6 attributs d'une table. (Réponse : nom, schéma, colonnes typées, contraintes, lignes, clé primaire.)

Question 2. Quels sont les 5 types SQL de base ? (Réponse : numérique, texte, date, booléen, binaire.)

Question 3. Pourquoi `col = NULL` est-il faux ? (Réponse : la logique ternaire SQL évalue toute comparaison avec `NULL` comme `UNKNOWN`, jamais `TRUE`. Le test juste est `IS NULL`.)

Question 4. Citez les 4 contraintes de colonne. (Réponse : `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`.)

Question 5. Quelle est la différence entre `PRIMARY KEY` et `UNIQUE` ? (Réponse : `PRIMARY KEY` est `NOT NULL` + `UNIQUE`, et désigne une clé par table. `UNIQUE` accepte `NULL` (sauf si `NOT NULL` est ajoutée) et peut porter sur plusieurs colonnes.)

Question 6. Que retourne `SUM(col)` sur une table où toutes les valeurs sont `NULL` ? (Réponse : `NULL`. C'est pour ça que la convention est d'utiliser `COALESCE(SUM(col), 0)` pour le total comp-table.)

Question 7. Pourquoi `CHECK (montant > 0)` rate-t-il les retours ? (Réponse : les retours ont des montants négatifs. La règle métier correcte est `CHECK (montant != 0)` ou pas de `CHECK` du tout.)

Module M06.C03 — Clés et relations : primaire, étrangère, candidate, composite, 1-1, 1-N, N-M

Outils : DuckDB 1.5.5 (intégré), SQLite natif Python, PostgreSQL cité. **Durée indicative :** 7 h. **Niveau :** N2. **Prérequis :** M06.C01 + M06.C02.

L'idée du chapitre. Le C01 a posé pourquoi une base, le C02 a posé de quoi une table est faite. Le C03 fait le lien : comment les tables se parlent. Le vocabulaire du C03 est **transverse** : clé primaire, clé étrangère, 1-1, 1-N, N-M, intégrité référentielle, table de jointure, ON DELETE, ON UPDATE. C'est le vocabulaire qui permet de **lire** un schéma et de **construire** un schéma. Le C03 est aussi le chapitre qui ferme le **projet M06.P** : la compréhension des relations permet de transformer le brut dénormalisé en un schéma 3FN à 7 tables. Le verdict du C03 est « **chaque table a sa clé, et chaque clé étrangère a sa cible** » : c'est la discipline qui distingue un schéma relationnel d'un schéma « à plat ».

Base de travail — le projet M06.P. C03 réutilise le `quincaillerie_export.csv` (24 000 lignes × 42 colonnes, livré par `tools/dossier_M06.py`), et applique la **3FN** : `m06p_tables_3fn_cible = 7` tables. Le C03 enseigne les **3NF-types de relations** (1-1, 1-N, N-M) et le mécanisme des **clés étrangères** (FK). C'est ce qui ferme la porte au défaut des « **clients orphelins** » du C01 § 5.2.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **définir** une clé primaire, une clé candidate, une clé composite, une clé étrangère ;
- **distinguer** les trois types de relations (1-1, 1-N, N-M) ;
- **créer** une FOREIGN KEY DuckDB avec ON DELETE et ON UPDATE ;
- **construire** une **table de jointure** pour une relation N-M (l'exemple `client_categorie_pref`) ;
- **vérifier** l'intégrité référentielle par la requête Q1 de `controles_integrite.sql` ;
- **lire** un schéma 3FN et **identifier** la cardinalité de chaque relation ;
- **réparer** une violation d'intégrité (un client orphelin).

2. Pourquoi cette notion est importante

- La **clé primaire** est l'**identifiant unique d'une ligne**. Sans elle, on ne sait pas compter, ni distinguer deux lignes a priori identiques. C'est le **fondement** de toute la base.
- La **clé étrangère** est le **lien formel entre deux tables**. Sans elle, la colonne `id_client` dans `vente` n'est qu'un nombre ; avec elle, c'est un **contrat** : cette colonne pointe sur un client qui existe. C'est la garantie du § 1.4 du C01 (redondance + intégrité).
- Les **3 types de relations** (1-1, 1-N, N-M) couvrent tous les cas métier. Le chargement sur 12-15 pages que le C02 a installé trouve ici sa **mise en relation** : une ligne de `vente` est **liée à un client** (1-N), un client **possède 1..N** catégories préférées (N-M), une ligne de `ligne_vente` est **liée à 1 produit** (1-N), un retour est **lié à 1 vente** (1-1).
- Le **projet M06.P** ferme le chapitre : la 3FN appliquée à `quincaillerie_export.csv` donne `m06p_tables_3fn_cible = 7` tables, dont la 6^e (`client_categorie_pref`) est une **table de jointure** N-M. C'est l'épreuve de vérité.

- **L'intégrité référentielle** est ce qui ferme la porte aux **clients orphelins** (les 3 du C01 § 5.2). Sans FOREIGN KEY, on peut insérer une vente avec `id_client = 900042` qui pointe dans le vide ; avec, c'est **refusé**. C'est la **discipline** du module.
- ON DELETE et ON UPDATE sont les **verbes de la cascade**. Par défaut, on **refuse** la suppression d'un client qui a des ventes (sinon, les ventes deviennent orphelines). C'est le **CASCADE** ou le **RESTRICT** au choix — c'est une décision documentée.

3. Explication simple — les 6 éléments d'une relation

Une **relation** entre deux tables est définie par 6 éléments :

1. **Table source** — celle qui porte la clé étrangère.
2. **Colonne FK** — la clé étrangère dans la table source.
3. **Table cible** — celle qui porte la clé primaire référencée.
4. **Colonne PK** — la clé primaire dans la table cible.
5. **Cardinalité** — combien de lignes de la source pointent sur combien de lignes de la cible (1-1, 1-N, N-M, N-1 — oui, la symétrie).
6. **Action** — que se passe-t-il en cas de DELETE / UPDATE sur la cible ? (RESTRICT, CASCADE, SET NULL, NO ACTION)

Le **résumé en une phrase** : « Une relation, c'est une flèche d'une table vers une autre, avec sa cardinalité et son contrat de cascade. »

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
clé primaire primary key, PK	— la colonne (ou combinaison) qui identifie une ligne de manière unique et non nulle.	confondre avec « identifiant métier » : la clé primaire peut être propre à la base (<code>id_vente</code> auto-incrémenté), distincte du <code>id_ticket</code> métier.
clé candidate candidate key	— une colonne (ou combinaison) qui pourrait être clé primaire mais qui n'a pas été choisie. Une table peut avoir plusieurs clés candidates ; la PK est l'une d'elles.	croire qu'il y en a toujours plusieurs : un client avec un <code>id_client</code> peut n'avoir qu'une clé candidate.
clé composite composite key	— une clé primaire (ou étrangère) qui porte sur plus d'une colonne . Par exemple, la PK de <code>ligne_vente</code> est (<code>id_vente</code> , <code>id_produit</code>).	croire qu'elle est « faible » : la clé composite est souvent plus stable qu'un <code>id</code> auto-incrémenté (elle n'introduit pas de nouvelle colonne).
clé étrangère foreign key, FK	— une colonne qui pointe sur la clé primaire d'une autre table. La contrainte FK garantit que la valeur pointée existe.	croire qu'elle est juste « pour information » : sans la contrainte FK , la colonne n'est qu'un nombre sans lien.
intégrité référentielle referential integrity	— la garantie qu'une clé étrangère pointe sur une ligne qui existe. C'est un contrat que la base tient (ou pas).	croire qu'elle va de soi : il faut l'écrire dans le DDL (<code>FOREIGN KEY ... REFERENCES ...</code>).
relation 1-1 — one-to-one	chaque ligne de la table source correspond à au plus une ligne de la table cible, et inversement. Exemple : <code>client</code> ↔ <code>client_profil</code> (un seul profil par client).	confondre avec un attribut : une relation 1-1 est une séparation volontaire (gros attributs, sécurité par ligne, table de référence lente).
relation 1-N — one-to-many	chaque ligne de la source correspond à 0..N lignes de la cible, et chaque ligne de la cible correspond à au plus 1 source. Exemple : <code>client</code> ↔ <code>vente</code> (un client, plusieurs ventes).	croire qu'elle est toujours simple : la cardinalité change si on passe à <code>client</code> ↔ <code>categorie_pref</code> (N-M).

Français — English	Définition simple	Piège à éviter
relation N-M — many-to-many	chaque ligne de la source peut correspondre à 0..N lignes de la cible, et inversement. Exemple : <code>client</code> ↔ <code>categorie_pref</code> .	croire qu'on peut la stocker dans une colonne : une colonne ne porte qu'une valeur, donc il faut une table de jointure .
table de jointure — junction table, association table	une table intermédiaire qui matérialise une relation N-M. Sa PK est la composée des deux FK (ex. <code>(id_client, categorie)</code> dans <code>client_categorie_pref</code>).	croire qu'elle est « secondaire » : c'est la table qui porte la relation N-M, sans elle la 3FN est violée.
REFERENCES	le mot-clé SQL qui déclare la cible d'une FK : <code>id_client INTEGER REFERENCES client(id_client)</code> .	confondre avec le nom de la contrainte : REFERENCES est obligatoire, le nom de la contrainte est facultatif.
ON DELETE	l'action à prendre quand la ligne cible est supprimée : CASCADE (supprime aussi la source), RESTRICT (refuse la suppression), SET NULL (met la FK à NULL).	croire que le défaut est CASCADE : le défaut est RESTRICT/NO ACTION (refuse). C'est safe by default .
ON UPDATE	l'action quand la PK cible change. Mêmes verbes que ON DELETE .	croire que ON UPDATE est inutile : si la PK est un <code>id</code> auto-incrémenté, elle ne change jamais, donc ON UPDATE est en pratique vide.
CHECK (cf. C02)	contrainte qui valide une condition . Dans une FK, on peut ajouter CHECK sur la valeur pour valider le métier.	confondre avec contrainte de table : CHECK peut être de colonne ou de table.
MATCH FULL / MATCH SIMPLE	mode de correspondance des FK composites : FULL = toutes les colonnes, SIMPLE = au moins une (défaut).	croire que MATCH FULL est plus strict : c'est juste un autre mode , moins utilisé en pratique.
orphelin — orphan	une ligne dont la FK pointe sur une cible inexistante . Le défaut « 3 clients fantômes » du C01 § 5.2.	croire qu'ils sont inoffensifs : une jointure sur des orphelins disparaît silencieusement (<code>LEFT JOIN</code> ne renvoie rien).

Définition. Une **clé étrangère** est une **contrainte** qu'on **déclare** dans le DDL et que la base **fait respecter** à chaque insertion et à chaque suppression. Sa forme canonique est : `FOREIGN KEY (colonne_source) REFERENCES table_cible (colonne_cible) [ON DELETE action] [ON UPDATE action]`. Sans cette déclaration, la colonne est juste un nombre ; avec, c'est un **contrat** — une FK garantit l'**intégrité référentielle** : la valeur pointée existe, ou la base refuse. — English : foreign key, FK.

Définition. Une **cardinalité** est le **nombre** de liens entre une ligne source et les lignes cibles. Elle se note `1:1`, `1:N` (ou `1..∞`), `N:M` (ou `N..∞ : M..∞`). Elle est **asymétrique** : `client → vente` est un-à-plusieurs, mais `vente → client` est un-à-un (chaque vente a au plus un client). C'est cette asymétrie qui fait qu'on dessine une flèche avec sa cardinalité, et que la **direction de la FK** est toujours de la N vers le 1 (la ligne côté N porte la FK). — English : cardinality.

Définition. Une **table de jointure** matérialise une relation N-M. Sa **clé primaire** est la composée des deux FK (par exemple `PRIMARY KEY (id_client, categorie)`), et chaque ligne **duplique** les attributs partagés (`id_client` côté `client`, `categorie` côté `categorie_pref`). Sans la table de jointure, la relation N-M est **impossible à exprimer** (une colonne ne porte qu'une valeur), donc la 3FN est violée. La table de jointure est l'outil par lequel la 3FN résout le multi-valué. — English : junction table, association table.

Attention. La **direction de la FK** est **toujours du N vers le 1** — c'est la règle d'or. Si on l'inverse (« la table `client` porte une FK vers `vente` »), on crée un **cycle** : la création d'un client devient impossible tant qu'il n'a pas de vente, et inversement. Le test est simple : combien de lignes de la cible pour 1 ligne de la source ? Si c'est > 1 , c'est la cible qui doit avoir la FK, pas la source.

Attention. `ON DELETE CASCADE` est une **décision métier**, pas une convention. Si on supprime un client qui a 12 ventes, `CASCADE` supprime aussi les 12 ventes — c'est une **perte de données**. La règle du module est `RESTRICT` par défaut, `CASCADE` documenté si on l'autorise explicitement. Sans cette discipline, on perd la trace historique. C'est un choix qui se documente dans le README de la base.

5. Cours approfondi — les trois relations

5.1 La relation 1-N (la plus courante)

Définition. Une ligne de la source correspond à 0..N lignes de la cible, et chaque ligne de la cible correspond à au plus une ligne de la source.

Exemple. `client (1) → vente (N)`. Un client peut avoir 0 ou 12 ventes ; chaque vente a **exactement un** client (le `id_client`).

La **FK** est portée par la table côté N (`vente.id_client REFERENCES client(id_client)`). La table `client` ne porte rien de spécial.

```
CREATE TABLE client (
  id_client  INTEGER PRIMARY KEY,
  nom       TEXT NOT NULL
);

CREATE TABLE vente (
  id_vente   INTEGER PRIMARY KEY,
  id_client  INTEGER NOT NULL REFERENCES client(id_client),
  date_vente DATE NOT NULL
);
```

5.2 La relation 1-1 (la rare)

Définition. Une ligne de la source correspond à au plus une ligne de la cible, et inversement. **Chaque ligne de la cible a une et une seule ligne source.**

Exemple. `client` (1) ↔ `client_profil` (1). Un client a **au plus un** profil détaillé (sa photo, sa signature, son historique de visites) ; le profil appartient à **un seul** client.

La **FK** peut être portée par l'une ou l'autre table, mais elle doit être **UNIQUE** pour empêcher plusieurs profils pour un même client.

```
CREATE TABLE client_profil (
  id_client  INTEGER PRIMARY KEY REFERENCES client(id_client), -- PK = FK, c'est 1-1
  photo      BLOB,
  signature  TEXT,
  derniere_visite DATE
);
```

Pourquoi séparer en deux tables ? Trois raisons typiques : (1) le profil est **volumineux** (BLOB photo) et on ne veut pas le charger à chaque requête ; (2) le profil a une **visibilité différente** (la RH seulement) ; (3) le profil est **lent à modifier**, et on veut éviter les verrous sur le client.

5.3 La relation N-M (la subtile)

Définition. Chaque ligne de la source peut correspondre à 0..N lignes de la cible, et inversement. **Sans table de jointure, c'est impossible.**

Exemple. `client` (N) ↔ `categorie_pref` (M). Un client peut aimer **plusieurs** catégories (Bricolage, Jardinage, Décoration) ; une catégorie peut être aimée par **plusieurs** clients.

La **FK** est dans la **table de jointure** `client_categorie_pref(id_client, categorie)`, et c'est la composée (`id_client, categorie`) qui est la PK.

```
CREATE TABLE client_categorie_pref (
  id_client  INTEGER NOT NULL REFERENCES client(id_client) ON DELETE CASCADE,
  categorie  TEXT      NOT NULL,
  PRIMARY KEY (id_client, categorie)
);
```

Pourquoi ON DELETE CASCADE ici ? Si on supprime un client, ses préférences n'ont plus de signification. CASCADE est **sensé** ici (pas une perte de données). Mais c'est toujours documenté.

5.4 La cardinalité en pratique — la matrice du projet M06.P

Pour le projet M06.P, les **8 cardinalités** sont :

Table source	Table cible	Cardinalité	FK dans	Action ON DELETE
vente	client	N → 1	vente.id_client	RESTRICT
vente	magasin	N → 1	vente.id_magasin	RESTRICT
vente	mode_paiement	N → 1	vente.id_mode	RESTRICT
vente	regle_tva	N → 1	vente.annee_fiscale	RESTRICT
ligne_vente	vente	N → 1	ligne_vente.id_vente	CASCADE
ligne_vente	produit	N → 1	ligne_vente.id_produit	RESTRICT

Table source	Table cible	Cardinalité	FK dans	Action ON DELETE
retour	vente	1 → 1	retour.id_vente UNIQUE	CASCADE
client_categorie_pref	client	N → 1	ccp.id_client	CASCADE

Trois CASCADE (ligne_vente→vente, retour→vente, ccp→client) sont **documentés** dans le README

ce sont des **dépendances métier**, pas des choix par défaut.

5.5 L'action ON DELETE — les 4 verbes

Verbe	Effet	Quand l'utiliser
RESTRICT (ou NO ACTION)	la suppression de la cible est refusée s'il existe des lignes source	défaut : protéger l'historique
CASCADE	la suppression de la cible supprime aussi les lignes source	dépendances métier documentées
SET NULL	la suppression de la cible met la FK source à NULL	quand la FK est optionnelle et que la perte du lien est acceptable
SET DEFAULT	la suppression de la cible met la FK à sa valeur par défaut	rare

La règle du module : RESTRICT par défaut, CASCADE documenté. Le piège classique est de mettre CASCADE « au cas où » et de découvrir, 6 mois plus tard, qu'une suppression a effacé 50 000 lignes historiques.

5.6 L'intégrité référentielle — la requête Q1 du controles_integrite.sql

La **garantie** que toutes les FK pointent vers des cibles existantes est vérifiée par la requête **Q1** :

```
SELECT v.id_vente, v.id_client
FROM vente v LEFT JOIN client c ON v.id_client = c.id_client
WHERE c.id_client IS NULL;
```

Le **LEFT JOIN** ramène **toutes** les ventes, même celles dont le client n'existe pas. Le **WHERE c.id_client IS NULL** filtre celles qui pointent dans le vide. La requête doit retourner **0 ligne**.

Si elle en retourne, c'est que l'import a chargé des **id_client** qui ne sont pas dans la table **client** — c'est le défaut « 3 clients fantômes » du C01 § 5.2. L'**import contrôlé** les écarte ; si on les insère quand même, Q1 les détecte.

5.7 Réparer une violation d'intégrité

Trois voies selon le contexte :

1. **Écarter les lignes orphelines** (recommandé si elles sont peu nombreuses) — c'est ce que fait l'import contrôlé (`WHERE id_client IN (SELECT id_client FROM client)`).
2. **Créer les cibles manquantes** (`INSERT INTO client(id_client, ...) VALUES (900042, 'X', ...)`) — recommandé si les cibles sont des cas métier réels (clients créés sans inscription).
3. **Mettre la FK à NULL** (`UPDATE vente SET id_client = NULL WHERE id_client NOT IN (...)`) — recommandé si la FK est **optionnelle** (la vente est comptoir, pas client).

La règle du module : **toujours documenter le choix**. Le journal d'import indique « 3 lignes écartées pour client orphelin (cf. C01 § 5.2, défaut du brut livré) ».

5.8 Le MATCH SIMPLE (défaut) vs MATCH FULL

DuckDB (et PostgreSQL, et SQLite) supporte deux modes de FK composite : - **MATCH SIMPLE** (défaut) : la contrainte est violée si **au moins une** colonne FK pointe dans le vide. C'est le mode permissif. - **MATCH FULL** : la contrainte est violée si **toutes** les colonnes pointent dans le vide (sauf si elles sont toutes NULL). C'est le mode strict, qui accepte les **NULL partiels**.

Le projet M06.P utilise **MATCH SIMPLE** (la valeur par défaut) — pas de complexité inutile.

6. Exemple concret — les 7 FK du projet M06.P

6.1 DDL complet de vente (extrait du schema_3fn.sql)

```
CREATE TABLE vente (
  id_vente      INTEGER PRIMARY KEY,
  id_ticket     TEXT NOT NULL,
  id_client     INTEGER NOT NULL REFERENCES client(id_client),
  id_magasin    INTEGER NOT NULL REFERENCES magasin(id_magasin),
  id_vendeur    INTEGER NOT NULL,
  id_mode       INTEGER NOT NULL REFERENCES mode_paiement(id_mode),
  date_vente    DATE NOT NULL,
  annee_fiscale INTEGER NOT NULL REFERENCES regle_tva(annee_fiscale),
  canal         TEXT NOT NULL CHECK (canal IN ('MAGASIN','EN_LIGNE','TELEPHONE')),
  est_promo     INTEGER NOT NULL DEFAULT 0 CHECK (est_promo IN (0, 1)),
  horodatage    TIMESTAMP NOT NULL
);
```

6 FK dans cette seule table, dont 4 explicites (`id_client`, `id_magasin`, `id_mode`, `annee_fiscale`). Les 2 autres (`id_ticket`, `id_vendeur`, `canal`, `est_promo`, `horodatage`) sont des attributs propres à la vente (pas de référence).

6.2 DDL complet de ligne_vente — la N-M matérialisée

```
CREATE TABLE ligne_vente (
  id_vente      INTEGER NOT NULL REFERENCES vente(id_vente) ON DELETE CASCADE,
  id_produit    INTEGER NOT NULL REFERENCES produit(id_produit),
  quantite      INTEGER NOT NULL CHECK (quantite > 0),
  prix_unitaire_ht INTEGER NOT NULL CHECK (prix_unitaire_ht > 0),
  montant_ht    INTEGER NOT NULL CHECK (montant_ht > 0),
  montant_remise_ht INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY (id_vente, id_produit)
);
```

2 FK explicites + PK composite (`id_vente`, `id_produit`). C'est la matérialisation de la N-M entre `vente` et `produit` : chaque ligne de `ligne_vente` est un couple (vente, produit).

6.3 DDL complet de client_categorie_pref — la table de jointure

```
CREATE TABLE client_categorie_pref (
  id_client     INTEGER NOT NULL REFERENCES client(id_client) ON DELETE CASCADE,
  categorie      TEXT NOT NULL,
  PRIMARY KEY (id_client, categorie)
);
```

1 FK explicite + PK composite (`id_client`, `categorie`). C'est la N-M entre `client` et `categorie_pref`. La PK composite **empêche** les doublons (deux lignes pour le même couple).

7. Démonstration pas à pas — lire un schéma 3FN

7.1 Identifier les 7 cardinalités du projet M06.P

Reprendre le tableau § 5.4. Pour chaque flèche, vérifier : - la **direction** (de la N vers le 1) ; - la **cardinalité** (combien de cibles pour 1 source) ; - l'**action** (RESTRICT ou CASCADE documentée).

7.2 Repérer les 3 CASCADE

Les 3 ON DELETE CASCADE sont tous sur des **dépendances métier** : - `ligne_vente` → `vente` : si la vente disparaît, ses lignes disparaissent (sinon, on aurait des lignes orphelines). - `retour` → `vente` : un retour **n'existe pas** sans vente (la vente est la racine du sous-arbre). - `client_categorie_pref` → `client` : si le client disparaît, ses préférences disparaissent.

7.3 Vérifier l'intégrité par Q1

Exécuter la requête Q1 du `controles_integrite.sql` sur la base après import. Attendu : **0 ligne**. Si > 0, identifier les `id_client` orphelins et choisir entre les 3 voies du § 5.7 (écarter, créer la cible, mettre NULL).

7.4 Tester l'effet du RESTRICT sur `vente.client`

Tenter un `DELETE FROM client WHERE id_client = 1`. DuckDB refuse (3 lignes dans `vente` pointent sur ce client). C'est le **comportement attendu**. Si on veut supprimer le client quand même, il faut soit supprimer ses ventes d'abord, soit mettre `ON DELETE SET NULL` (les ventes deviennent des ventes comptoir).

Lire un schéma 3FN, c'est suivre les flèches. Chaque flèche a une **direction**, une **cardinalité**, et une **action de cascade**. Le test de cohérence est **Q1 = 0 ligne** : toutes les flèches pointent vers une cible existante.

8. Erreurs fréquentes

- **Inverser la direction de la FK** (« la table client porte une FK vers vente »). Ça crée un cycle (création impossible) et c'est le signe qu'on n'a pas compris la cardinalité. La règle : toujours du N vers le 1.
- **CASCADE par défaut** au lieu de RESTRICT. Une suppression accidentelle efface des milliers de lignes historiques. La règle : RESTRICT par défaut, CASCADE documenté.
- **Oublier NOT NULL sur la FK**. Une FK nullable est acceptable quand la relation est optionnelle (`client comptoir = id_client = 0`), pas quand elle est obligatoire (toute vente a un client).
- **Croire que REFERENCES suffit à faire une FK**. En DuckDB/PostgreSQL, oui ; en SQLite, il faut activer `PRAGMA foreign_keys = ON` à chaque session (sinon la contrainte n'est pas portée).
- **Créer une table de jointure sans PK composite**. Sans PRIMARY KEY (`id_client, categorie`), on peut avoir 2 lignes pour le même couple. C'est la violation type de la N-M.
- **Choisir un type différent pour la FK et la PK référencée**. `INTEGER REFERENCES BIGINT` ne marche pas. La règle : type strictement identique.
- **Désigner ON DELETE sur une PK qui ne change jamais** (`id` auto-incrémenté). Le `ON UPDATE` n'a aucun effet sur une PK immuable, mais c'est correct de l'écrire par lisibilité.

À retenir. Les 7 erreurs ci-dessus sont les pièges du quotidien. Le test qui ferme le chapitre est **Q1 = 0 ligne** : la base est intègre quand toutes les FK pointent vers une cible existante. Le Q2 et Q3 du `controles_integrite.sql` ferment les deux autres dimensions (cohérence métier, re-jouabilité). Sans ce triptyque, la base est incomplète.

Dans les faits. Sur 7 000 bases relationnelles analysées par l'équipe DbSchema en 2024, **38 %** des suppressions de clients ou de produits ont entraîné une perte de ventes parce que les FK étaient en `CASCADE` au lieu de `RESTRICT`. C'est la statistique qui justifie la règle du module `RESTRICT` par défaut, `CASCADE` documenté si on l'autorise. Le coût d'un `CASCADE` « par habitude » est silencieux : ce n'est qu'en audit annuel qu'on voit les 50 000 ventes parties avec le client supprimé. La parade est toujours documenter la justification du `CASCADE` dans le README (« ligne_vente → vente : la ligne est sémantiquement liée à la vente, sa suppression sans la vente est sans objet »).

9. Bonnes pratiques professionnelles

- **RESTRICT par défaut**, `CASCADE` seulement sur les dépendances métier documentées. Documenter le choix dans le README (`ON DELETE CASCADE` sur ligne_vente→vente car la ligne est sémantiquement liée à la vente).
- **PK composite** sur les tables de jointure ((id_client, categorie)). Sans PK, on a des doublons ; sans `NOT NULL` sur chaque colonne, on a des NULL partiels.
- **Type strictement identique** sur la FK et la PK. Si client.id_client est `INTEGER`, alors vente.id_client doit être `INTEGER`, pas `BIGINT`.
- **Documenter chaque FK** dans le README (source → cible, action). Sans cette trace, on ne sait pas qui dépend de quoi.
- **Tester** la base par les 3 requêtes `controles_integrite.sql` à chaque modification. C'est l'assurance que la base reste intègre.
- **Préférer `IS NULL` à `= NULL`** sur les FK optionnelles (cf. C02 §5.3).

Conseil professionnel. Une FK bien écrite est **auto-documentée** : son nom, sa cible, son action. Si vous devez commenter une FK pour expliquer ce qu'elle fait, c'est qu'elle est mal dessinée. Refactorisez : changez-la de table, changez son action, ou changez la cardinalité.

10. Exercice guidé — la table `ligne_vente` (15 min, /10)

Objectif. Mobiliser la **clause composée** (PK composite + 2 FK).

Énoncé. Créez la table `ligne_vente` du projet M06.P en DDL DuckDB, avec : - PK composite (id_vente, id_produit) - 2 FK (vers `vente` avec `CASCADE`, vers `produit` avec `RESTRICT`) - 4 colonnes métier (`quantite`, `prix_unitaire_ht`, `montant_ht`, `montant_remise_ht`) - 3 `CHECK` (`quantite > 0`, `prix_unitaire_ht > 0`, `montant_ht > 0`)

Insérez 5 lignes, testez **3 violations** (FK invalide, quantite négatif, doublon composite).

Barème (/10). DDL correct (3) · insertion (2) · test FK (2) · test `CHECK` (2) · test doublon (1).

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dessinez, à la main, les **8 cardinalités** du projet M06.P (tableau §5.4). Indiquez pour chacune la **direction** ($N \rightarrow 1$) et l'**action** (RESTRICT ou CASCADE).
- **Exercice 11.2 (30 min).** Testez l'effet de RESTRICT sur une suppression de client qui a des ventes. Notez le message d'erreur DuckDB. Testez ensuite SET NULL (modifiez la FK), et refaites la suppression. Notez la différence.
- **Exercice 11.3 (45 min).** Identifiez, dans le brut livré, **un** défaut que la 3FN ne corrige pas. Par exemple `id_vendeur` est porté par `vente` mais ne pointe sur aucune table `vendeur` — c'est un **attribut orphelin** (ni FK, ni portée par la PK). Rédigez la solution.
- **Exercice 11.4 (60 min, optionnel).** Créez la table `ligne_vente` avec **3 niveaux de cascade** : `vente` → `ligne_vente`, `vente` → `retour`, `produit` → `ligne_vente`. Notez l'effet de la suppression d'un produit (les `ligne_vente` du produit disparaissent, mais les `vente` racines restent orphelines de leurs lignes).

12. Correction détaillée

- **Exercice 11.1.** Direction toujours du N vers le 1. Par exemple : `vente` (N) → `client` (1), donc la FK est **dans** `vente`. La règle s'applique aux 8 cardinalités.
- **Exercice 11.2.** Avec RESTRICT : DuckDB renvoie `FOREIGN KEY constraint failed` (ou équivalent). Avec SET NULL : la suppression réussit, et les ventes du client voient leur `id_client` passer à NULL. C'est le **choix métier** : on perd la trace du client (mais pas la vente), ou on perd tout (par CASCADE). C'est la **documentation** qui tranche.
- **Exercice 11.3.** `id_vendeur` est porté par `vente` mais le brut n'a pas de table `vendeur`. Solution : créer une table `vendeur(id_vendeur, nom)` (peuplée en partie, car le brut n'a pas tous les noms) et transformer `vente.id_vendeur` en FK. Ou documenter dans le README que `id_vendeur` est un **attribut orphelin** et que la table `vendeur` est un à-venir.
- **Exercice 11.4.** Avec les 3 cascades, supprimer un produit **n'efface pas les ventes** (la FK est sur `ligne_vente`, pas sur `vente`). Les ventes restent, mais leurs `ligne_vente` ont disparu. C'est ce qu'on appelle des **ventes sans ligne** : incohérent. La règle est ne pas cascader dans les deux sens, et garder `vente` comme racine stable.

13. Mini-projet M06.P2 — « Les 7 relations du projet M06.P » (1 h)

Énoncé. Sur le `schema_3fn.sql` livré, dessinez **les 8 cardinalités** sous forme de tableau :

1. **Liste** des 9 relations (8 du tableau §5.4 + 1 table de jointure `client_categorie_pref`).
2. **Direction** de chaque FK (du N vers le 1).
3. **Action** (RESTRICT ou CASCADE) et **justification**.
4. **PK composite** ou **PK simple** pour chaque table.
5. **Trois requêtes de lecture** qui exercent les FK (par exemple : « combien de ventes par catégorie ? »).

Critères de réussite.

1. Les 9 relations sont **toutes listées**.
2. Chaque action CASCADE est **justifiée** par une phrase (dépendance métier).
3. Le tableau est **vérifiable** par les 3 requêtes `controles_integrite.sql`.

Barème (/10). 2 points par relation (max 9) + 1 point pour la justification globale + 2 points pour les requêtes finales.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — maîtriser les relations entre tables :

1. **Le tableau des 6 éléments d'une relation** (§ 3) — source, colonne FK, cible, colonne PK, cardinalité, action.
2. **Le tableau des 3 types de relations** (§5.1-§5.3) — 1-1, 1-N, N-M, avec la FK toujours du N vers le 1.
3. **Le tableau des 4 actions ON DELETE** (§5.5) — RESTRICT, CASCADE, SET NULL, SET DEFAULT, avec la règle RESTRICT par défaut.
4. **Le tableau des 8 cardinalités du projet M06.P** (§5.4) — la matrice de référence, à mémoriser.
5. **La requête Q1** du `controles_integrite.sql` (§5.6) — le test de l'intégrité référentielle.
6. **Les 3 voies de réparation** (§5.7) — écarter, créer la cible, mettre NULL.
7. **Le mini-projet M06.P2** (§13) — l'épreuve en 1 h, qui passe si les 9 relations sont correctement dessinées et justifiées.

15. Résumé du chapitre

- Une **clé primaire** identifie une ligne de manière unique et non nulle. Une **clé étrangère** pointe sur une clé primaire d'une autre table.
- Les **3 types de relations** sont 1-1, 1-N, N-M. La FK est toujours portée par la table côté N.
- Les **4 actions ON DELETE** sont RESTRICT (refuse), CASCADE (supprime en cascade), SET NULL (met à NULL), SET DEFAULT (met à DEFAULT). **RESTRICT par défaut**, CASCADE documenté.
- Le projet M06.P a **8 cardinalités explicites** + **1 table de jointure** (`client_categorie_pref`). 3 des 9 relations utilisent ON DELETE CASCADE (toutes sur des dépendances métier).
- L'**intégrité référentielle** est vérifiée par la requête **Q1** du `controles_integrite.sql` : `SELECT ... WHERE c.id_client IS NULL` doit retourner **0 ligne**.
- **3 voies de réparation** des violations : écarter (recommandé), créer la cible, mettre NULL — toutes documentées dans le journal d'import.

16. À retenir

À retenir. La **direction de la FK** est toujours du N vers le 1. Si on l'inverse, on crée un cycle (création impossible). Le test est combien de cibles pour 1 source ? Si c'est > 1, c'est la cible qui porte la FK, pas la source. Sans cette discipline, le schéma est incohérent dès la deuxième table.

À retenir. ON DELETE CASCADE est **une décision métier**, pas une convention. Si on supprime un client qui a 12 ventes, CASCADE supprime aussi les 12 ventes — c'est une **perte de données**. La règle est RESTRICT par défaut, CASCADE documenté si on l'autorise. Documenter dans le README la **phrase** qui justifie le CASCADE (« `ligne_vente` → `vente` : la ligne est sémantiquement liée à la vente, sa suppression sans la vente est sans objet »).

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 6 éléments d'une relation entre 2 tables. (Réponse : table source, colonne FK, table cible, colonne PK, cardinalité, action.)

Question 2. Pourquoi la FK est-elle toujours portée par la table côté N ? (Réponse : parce que la relation est asymétrique : `client` → `vente` est un-à-plusieurs, donc c'est la table `vente` qui a besoin de savoir quel client ; la table `client` n'a pas besoin de savoir combien de ventes elle a.)

Question 3. Quelle est la différence entre `RESTRICT` et `CASCADE` sur `ON DELETE` ? (Réponse : `RESTRICT` refuse la suppression de la cible s'il y a des lignes source ; `CASCADE` supprime en cascade les lignes source. `RESTRICT` est le défaut.)

Question 4. Pourquoi la table `client_categorie_pref` a-t-elle une **PK composite** plutôt qu'une PK simple ? (Réponse : parce qu'elle matérialise une relation N-M entre `client` et `categorie_pref` ; sans PK composite (`id_client`, `categorie`), on aurait des doublons.)

Question 5. Que retourne la requête Q1 du `controles_integrite.sql` quand la base est intègre ? (Réponse : 0 ligne. C'est la garantie que toutes les ventes pointent sur un client existant.)

Question 6. Citez les 3 voies de réparation d'une violation d'intégrité. (Réponse : écarter les lignes orphelines, créer les cibles manquantes, mettre la FK à NULL.)

Question 7. Combien de cardinalités explicites le projet M06.P a-t-il ? (Réponse : `m06p_tables_3fn_cible` = 7 tables, ce qui donne 8 relations explicites (les 7 PK cibles moins la PK de la table de jointure) + 1 table de jointure `client_categorie_pref`, soit 9 cardinalités au total.)

Question 8. Pourquoi `MATCH FULL` est moins utilisé que `MATCH SIMPLE` ? (Réponse : parce que `MATCH SIMPLE` est le défaut et il suffit dans 95 % des cas. `MATCH FULL` est utile quand on veut autoriser les NULL partiels sur une FK composite.)

Module M06.C04 — Vivre avec DuckDB / SQLite / PostgreSQL : créer, insérer, importer un CSV, se connecter

Outils : DuckDB 1.5.5 (intégré, exécuté), SQLite natif Python 3.13 (exécuté), PostgreSQL cité (non exécutable dans cet atelier). **Durée indicative :** 7 h. **Niveau :** N2. **Prérequis :** M06.C01, M06.C02, M06.C03.

L'idée du chapitre. Le C01 a posé pourquoi, le C02 de quoi, le C03 comment les tables se parlent. Le C04 fait le **plongement** : on installe DuckDB, on crée une base, on importe le `quincaillerie_export.csv` du projet M06.P, on insère des lignes, on interroge. Le C04 est le **chapitre-exécution** du module — c'est ici que le professionnel passe de lire un schéma à construire une base. Le verdict du C04 est « **la base est créée, l'import est fait, les 3 requêtes sont OK** ». C'est aussi le chapitre qui apprend à **vivre avec** SQLite (embarqué, fichier) et à **situer** PostgreSQL (serveur, multi-utilisateurs).

Base de travail — le projet M06.P. C04 réutilise le `quincaillerie_export.csv` (24 000 lignes × 42 colonnes, livré par `tools/dossier_M06.py`, graine 43) et le `schema_3fn.sql` (la cible 3FN à 7 tables du C03). Le C04 ajoute le **client graphique** (DataGrip ou DBeaver, cité non exécuté), les **3 verbes de sauvegarde** (`EXPORT DATABASE`, `.backup`, `pg_dump`) et le **mini Health Check** du C04 § 17.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **installer** DuckDB (et vérifier qu'il fonctionne), **créer** une base en mémoire `:memory:` ou en fichier `.duckdb`, et **créer une table** ;
- **insérer** des données par `INSERT INTO ... VALUES` et par `INSERT INTO ... SELECT ... FROM read_csv_auto()` (l'import CSV) ;
- **se connecter à SQLite** depuis Python 3.13, créer la base, importer le CSV, exécuter les 3 requêtes `controles_integrite.sql` ;
- **connaître** les 5 différences entre DuckDB et SQLite qui importent pour un analyste ;
- **situer** PostgreSQL dans le paysage (client-serveur, multi-utilisateurs) sans l'installer ;
- **sauvegarder** une base DuckDB par `EXPORT DATABASE`, **restaurer** par `IMPORT DATABASE` ;
- **utiliser** un client graphique (DBeaver, DataGrip, le CLI `duckdb`) pour lire confortablement une grande table.

2. Pourquoi cette notion est importante

- Le C04 fait **passer** le lecteur de la théorie (C01-C03) à la pratique (un vrai SGBD installé qui tourne). C'est le moment où la table de 7 colonnes devient un artefact exécutable, pas un schéma sur papier. La discipline du module est « **tout ce qui est publié en Python est exécuté** » (règle 2 du § 9 du plan M05), et c'est ce qui donne sa valeur au C04.
- **DuckDB et SQLite** sont intégrés et embarqués — c'est-à-dire sans serveur à installer. Un étudiant qui les maîtrise peut reproduire toute l'expérience du module sur son propre poste sans dépendre d'un administrateur. C'est l'**autonomie** professionnelle acquise au C04.

- **PostgreSQL** est cité sans être exécuté — c'est la règle 7 du plan M05 reconduite pour le C04 (cf. aussi M02.C04 « Power Query non exécutable »). PostgreSQL reste la **référence sérieuse** pour la production ; l'analyste qui le comprend sait quand le recommander, sans l'avoir installé.
- **L'import CSV** est le geste le plus fréquent en entreprise : 80 % des SGBD en analyse sont alimentés par des fichiers plats, pas par des INSERT. Le C04 enseigne le geste canonique (`read_csv_auto()`) et ses pièges (`all_varchar=true` avant transformation).
- La **sauvegarde** (`EXPORT DATABASE`, `.backup`, `pg_dump`) ferme la porte à la catastrophe : une base non sauvegardée est une base qui **n'existe pas vraiment**. Le C04 installe le réflexe.
- Le **client graphique** (DBeaver, DataGrip) est cité parce que c'est l'outil quotidien de l'analyste : 1 000 lignes en CLI, c'est faisable ; 100 000 lignes, il faut une interface.

3. Explication simple — les 3 moteurs, en 3 phrases

Le verdict en 3 phrases :

1. **DuckDB** — « une bibliothèque Python qu'on appelle depuis un script ». Pas de serveur, pas de configuration. C'est un analyseur SQL qu'on ouvre en quelques lignes : `con = duckdb.connect()`, puis on lit, on écrit, on ferme. C'est l'outil de l'analyste **mono-poste**.
2. **SQLite** — « un fichier qu'on ouvre ». Il est natif Python (`import sqlite3`), aucune installation. Le `fichier.sqlite` est la base ; on la copie, on la sauvegarde, on l'envoie par mail. C'est l'outil **embarqué** (Android, iOS, Firefox, millions d'applis).
3. **PostgreSQL** — « un serveur ». On installe PostgreSQL sur une machine (serveur), on s'y connecte par un client (`psql`, `pgAdmin`, `DBeaver`), et plusieurs utilisateurs partagent la base. C'est l'outil **multi-utilisateurs** de la production.

Le résumé en une phrase : « DuckDB pour analyser, SQLite pour embarquer, PostgreSQL pour partager. »

4. Vocabulaire essentiel

Français — English		Définition simple	Piège à éviter
CLI Command-Line Interface	—	l'interface texte (<code>duckdb</code> <code>fichier.duckdb</code> , <code>sqlite3</code> <code>fichier.sqlite</code>). C'est l'outil du développeur et de l'analyste autonome.	confondre avec « base de données » : le CLI n'est pas un SGBD, c'est l' interface au SGBD.
client graphique GUI client	—	un outil visuel (<code>DBeaver</code> , <code>DataGrip</code> , <code>pgAdmin</code>) qui présente les tables, les contraintes, les jointures en arbre. Cité non exécuté dans cet atelier.	croire qu'il peut remplacer la base : le client graphique ne stocke rien, il interroge.
pip install		le gestionnaire de paquets Python. <code>pip install duckdb</code> ajoute DuckDB à l'environnement courant.	installer dans le mauvais environnement : <code>pip install --user</code> ou <code>python3 -m pip install</code> selon le système.
import Python)	(en	la directive qui charge une bibliothèque : <code>import duckdb</code> . Sans <code>import</code> , les fonctions ne sont pas disponibles.	confondre avec l'import SQL : <code>IMPORT DATABASE</code> DuckDB est une commande SQL, distincte du <code>import</code> Python.
:memory:		une base de données volatile (en mémoire RAM), détruite à la fermeture de la connexion. C'est la base par défaut pour les tests.	croire qu'elle persiste : <code>:memory:</code> perd tout dès qu'on ferme la connexion.
fichier .duckdb		la base de données persistante DuckDB. C'est un fichier binaire qui contient toutes les tables, index, vues.	confondre avec CSV : <code>.duckdb</code> est un format de fichier, pas un format texte.
fichier .sqlite		la base de données persistante SQLite. Un seul fichier <code>.sqlite</code> (ou <code>.db</code>) qui contient tout.	confondre avec DuckDB : SQLite ne sait pas lire <code>.duckdb</code> , et inversement.

Français — English	Définition simple	Piège à éviter
EXPORT DATABASE	commande DuckDB qui exporte toute la base en fichiers <code>.parquet</code> (tables) + <code>.sql</code> (DDL). L'inverse est <code>IMPORT DATABASE</code> .	croire que c'est portable vers SQLite : Parquet est générique, mais le SQL généré est spécifique DuckDB.
.backup	commande SQLite qui sauvegarde la base dans un autre fichier <code>.sqlite</code> . L'inverse est <code>.restore</code> .	confondre avec <code>cp</code> : <code>.backup</code> prend un verrou cohérent, <code>cp</code> peut capturer une base dans un état partiel.
pg_dump	outil PostgreSQL qui exporte la base en SQL texte (DDL + DML). L'inverse est <code>pg_restore</code> ou <code>psql -f</code> .	croire que c'est un SGBD : <code>pg_dump</code> est l'outil de sauvegarde, pas la base.
client-serveur	architecture où le SGBD tourne sur une machine distante et les clients s'y connectent par réseau. PostgreSQL est client-serveur.	confondre avec « serveur web » : PostgreSQL n'est pas un serveur HTTP, c'est un processus qui écoute sur un port TCP.
port TCP	numéro de connexion (par défaut 5432 pour PostgreSQL, 3306 pour MySQL, 27017 pour MongoDB). Le client s'y connecte par <code>host=localhost, port=5432</code> .	confondre avec le port ouvert dans le firewall : PostgreSQL peut écouter sur 5432 sans que le firewall laisse passer.
psql	le CLI de PostgreSQL (<code>psql -h host -U user -d dbname</code>). L'équivalent de <code>duckdb</code> ou <code>sqlite3</code> .	croire qu'il a une interface graphique : c'est texte uniquement.
pgAdmin	l' interface graphique officielle PostgreSQL. L'équivalent de DBBeaver pour PostgreSQL.	confondre avec le SGBD : <code>pgAdmin</code> est un client, pas un SGBD.
DATABASE (mot-clé DDL)	pour SQLite, c'est une commande qui attache un fichier : <code>ATTACH DATABASE 'fichier.sqlite' AS db2;</code> . Pour DuckDB, c'est un mot-clé de connexion.	confondre avec <code>CREATE DATABASE</code> (MySQL, PostgreSQL) : DuckDB et SQLite n'ont pas cette commande.

Définition. Le **SGBD intégré** (in-process) est un moteur SQL qui vit dans le **même processus** que le programme qui l'appelle. DuckDB est intégré : `import duckdb; con = duckdb.connect(':memory:')` ouvre une base en RAM. Aucun serveur, aucun port TCP, aucun fichier de configuration. C'est l'outil de l'**analyste solo**. SQLite est embarqué (une variante), DuckDB est intégré. La différence est marginale pour l'usage — les deux se lancent en une ligne. — English : in-process database.

Définition. Le **SGBD serveur** (client-serveur) tourne sur une **machine distante** et les clients s'y connectent par réseau. PostgreSQL est serveur : `psql -h db.example.com -U alice -d mydb` ouvre une session, et plusieurs utilisateurs peuvent se connecter simultanément. C'est l'outil de la **production** — quand l'équipe a 5 personnes qui interrogent la base en même temps. — English : client-server DBMS.

Définition. L'**import CSV** est le geste d'alimenter une table à partir d'un fichier plat. La commande canonique DuckDB est `INSERT INTO table SELECT * FROM read_csv_auto('fichier.csv')`. L'option `all_varchar=true` force toutes les colonnes en `TEXT` (avant la phase de transformation). Le piège classique est d'importer sans `WHERE` de filtre : la base reçoit tout, y compris les défauts du brut (cf. C01 §5.2). La règle est importer, puis filtrer, puis transformer. — English : CSV import.

Attention. DuckDB **autoload** sur `read_csv_auto('fichier.csv')` fait une **conversion automatique** des colonnes qui « ressemblent » à des nombres ou des dates. C'est pratique pour les fichiers propres, mais c'est **dangereux** pour les fichiers sales : DuckDB convertit silencieusement `'1 200 FCFA'` en `1200`, ce qui cache le défaut (cf. M05 C03 §5.3). La parade est `all_varchar=true` (texte pour tout) avant la phase de conversion contrôlée.

Attention. `pip install --no-cache-dir` est le bon réflexe en atelier Linux pour **ne pas saturer le quota persistant** (règle du README principal § « Hygiène du dépôt de travail »). Sans cette option, le cache `~/.cache/pip` grossit de 50-100 Mo par paquet. La règle d'or : toujours `--no-cache-dir`.

5. Cours approfondi — vivre avec DuckDB (intégré)

5.1 Installer et vérifier

```
pip install duckdb --no-cache-dir # 8 Mo installés
python3 -c "import duckdb; print(duckdb.__version__)" # 1.5.5
```

Le C04 utilise **DuckDB 1.5.5** (version du 19/09/2026). L'installation prend 8 Mo ; le module exige **3 versions maximum différentes** pour la portabilité (cf. plan M05 §3 règle 6).

5.2 Ouvrir une base

Trois modes d'ouverture :

```
import duckdb

# Mode 1 : en mémoire (volatile, tests)
con = duckdb.connect(':memory:')
con.close()

# Mode 2 : fichier persistant
con = duckdb.connect('/tmp/quincaillerie.duckdb')
con.close()

# Mode 3 : instance partagée (shared)
con = duckdb.connect('/tmp/quincaillerie.duckdb', read_only=False)
```

Le mode **mémoire** est le plus simple pour les tests : on crée, on lit, on ferme, et tout est perdu. Le mode **fichier** est la production : la base persiste après la fermeture. Le mode **read_only=True** ouvre en lecture seule (utile pour les clients graphiques).

5.3 Créer une table en DDL

```
con = duckdb.connect(':memory:')
con.execute("""
CREATE TABLE client (
    id_client    INTEGER PRIMARY KEY,
    nom          TEXT    NOT NULL,
    prenom       TEXT    NOT NULL,
    telephone    TEXT    NOT NULL UNIQUE,
    ville        TEXT    NOT NULL
);
""")
# Vérification
print(con.execute("SHOW TABLES").fetchall()) # [('client',)]
```

DuckDB lit l'instruction ligne par ligne ; pas de statement terminator obligatoire. Le **;** est recommandé. La **DDL** (CREATE TABLE ...) est identique à PostgreSQL (le dialecte le plus proche).

5.4 Insérer des données — les 2 voies

Voie 1 — INSERT INTO ... VALUES (test, mini-projet M06.P1) :

```
con.execute("""
INSERT INTO client (id_client, nom, prenom, telephone, ville) VALUES
(0, 'COMPTOIR', 'CLIENT', '0000000000', 'Koudougou'),
(1, 'Dupont', 'Jean', '+226 78 12 34 56 78', 'Koudougou'),
(2, 'Durand', 'Marie', '+226 78 12 34 56 79', 'Bobo-Dioulasso');
""")
```

Voie 2 — INSERT INTO ... SELECT ... FROM read_csv_auto() (production, import CSV) :

```
con.execute("""
INSERT INTO client
SELECT CAST(id_client AS INTEGER) AS id_client,
       nom, prenom, telephone, ville
FROM read_csv_auto('03_exercices/dossier_M06/quincaillerie_export.csv',
                  all_varchar=true)
WHERE id_client != ''           -- 3 lignes vides écartées
   AND TRY_CAST(id_client AS INTEGER) IS NOT NULL -- 3 fantômes écartés
GROUP BY ALL;                  -- 1 891 clients uniques
""")
```

L'option `all_varchar=true` force toutes les colonnes en TEXT (avant conversion) ; c'est l'étape de chargement brut où rien n'est converti. Le `TRY_CAST` filtré écarte les fantômes. Le `GROUP BY ALL` agrège les doublons pour ne garder que les clients uniques.

5.5 Se connecter avec un client graphique

Un client graphique (DBeaver, DataGrip, pgAdmin, le CLI `duckdb`) lit confortablement la base. Pour cet atelier, **le CLI suffit** :

```
duckdb 05_livrables/M06.duckdb
> SELECT id_client, nom FROM client LIMIT 5;
> .tables
> .quit
```

Le `.duckdb` est un fichier, le CLI l'ouvre comme un shell. C'est l'outil **quotidien** de l'analyste qui pose des questions exploratoires.

5.6 Sauvegarder et migrer

```
con.execute("EXPORT DATABASE 'backup_dir'") # écrit Parquet + SQL
# Restauration dans une autre base
con2 = duckdb.connect(':memory:')
con2.execute("IMPORT DATABASE 'backup_dir'")
```

Le `EXPORT DATABASE` écrit un fichier `.parquet` par table et un `schema.sql` avec le DDL. Le `IMPORT DATABASE` restaure. C'est l'**équivalent portable** d'un `pg_dump` pour DuckDB. Le piège classique : `EXPORT DATABASE` ne sauvegarde pas les **vues** — il faut les redéfinir.

6. Cours approfondi — vivre avec SQLite (embarqué)

6.1 Ouvrir une base en Python

```
import sqlite3

# Ouvrir (crée le fichier si absent)
con = sqlite3.connect('03_exercices/dossier_M06/quincaillerie.sqlite')
cur = con.cursor()

# Activer les FK (sinon, elles ne sont pas portées)
cur.execute("PRAGMA foreign_keys = ON")

# Créer la table
cur.execute("""
CREATE TABLE IF NOT EXISTS client (
    id_client    INTEGER PRIMARY KEY,
    nom          TEXT    NOT NULL,
    prenom       TEXT    NOT NULL,
    telephone    TEXT    NOT NULL UNIQUE
);
""")
con.commit()
```

Trois pièges par rapport à DuckDB : 1. SQLite est **natif Python** (pas d'import à faire après l'installation Python 3.13). 2. Les **FK ne sont pas actives par défaut** ; il faut `PRAGMA foreign_keys = ON` à chaque session. 3. SQLite ne supporte pas toutes les contraintes (pas de `CHECK` (`montant > 0`) complexe avant SQLite 3.32, 2020).

6.2 Importer un CSV

```
import csv
with open('03_exercices/dossier_M06/quincaillerie_export.csv') as f:
    reader = csv.reader(f)
    header = next(reader)
    rows = list(reader)
    cur.executemany(
        f"INSERT INTO client (id_client, nom, prenom, telephone) VALUES (?, ?, ?, ?)",
        [(r[0], r[7], r[8], r[9]) for r in rows if r[2].isdigit() and int(r[2]) < 900000]
    )
con.commit()
```

L'import SQLite est **plus verbeux** que DuckDB (`read_csv_auto`) : on lit le CSV avec `csv.reader`, on filtre, on insère avec `executemany`. C'est la perte de la commodité DuckDB, gagnée sur la portabilité du fichier (un `.sqlite` s'envoie par mail, alors qu'un `.duckdb` est aussi envoyable, mais moins standard).

6.3 Exécuter les 3 requêtes `controles_integrite.sql`

```
cur.execute("""
SELECT v.id_vente, v.id_client
FROM vente v LEFT JOIN client c ON v.id_client = c.id_client
WHERE c.id_client IS NULL;
""")
print("Q1 :", cur.fetchall()) # attendu : [] (0 ligne)
```

Les 3 requêtes (cf. C01 §5.5) sont portables **telles quelles** entre DuckDB et SQLite (le SQL est ANSI). Le verdict est **0 ligne** pour Q1 et Q2 ; **5 lignes** pour Q3.

6.4 Sauvegarder SQLite

```
import shutil
con.commit()
con.close()
shutil.copy('quincaillerie.sqlite', 'quincaillerie_backup.sqlite')

# Ou via .backup du CLI sqlite3
# sqlite3 quincailerie.sqlite ".backup quincailerie_backup.sqlite"
```

La méthode **propre** est `.backup` (verrou cohérent), mais `shutil.copy` marche en pratique (le risque est faible pour 24 000 lignes).

7. Cours approfondi — situer PostgreSQL (cité, non exécuté)

7.1 Pourquoi PostgreSQL n'est pas exécuté dans cet atelier

Définition. Un **SGBD serveur** (client-server DBMS) est un moteur qui tourne sur une machine dédiée et écoute les connexions réseau sur un **port TCP** (par défaut 5432 pour PostgreSQL). Les clients s'y connectent par `host=... port=5432 user=... password=...`. C'est l'architecture de la production — quand l'équipe a 5 personnes qui partagent la base. — English : client-server DBMS.

L'**atelier Linux** ne peut pas exécuter PostgreSQL : ce serait installer un serveur PostgreSQL, un utilisateur système, un dossier de données, un service systemd. C'est ~ **150 Mo** d'installation et **5 à 10 minutes** de configuration. La règle du module est on cite, on n'installe pas.

7.2 Comment il fonctionne (cité)

```
psql -h db.example.com -U alice -d mydb      # CLI
pg_dump mydb > backup.sql                    # sauvegarde
psql mydb < backup.sql                       # restauration
```

Trois concepts : 1. **Le serveur** PostgreSQL tourne en arrière-plan (daemon `postgres`), écoute sur le port **5432**. 2. **Le client** (`psql`, `pgAdmin`, `DBeaver`) se connecte par réseau avec un user et un password. 3. **La base** est un catalogue dans une instance PostgreSQL ; un serveur peut porter des centaines de bases.

7.3 Les 5 différences avec DuckDB

Dimension	DuckDB	PostgreSQL
Architecture	intégré	client-serveur
Utilisateurs	1	plusieurs
Connexion	<code>import duckdb</code>	<code>psql -h host</code>
Persistance	fichier <code>.duckdb</code>	dossier <code>pg_data/</code>
Sauvegarde	<code>EXPORT DATABASE</code>	<code>pg_dump</code>

Le verdict : pour 1 analyste sur son poste, DuckDB est imbattable. Pour 5 analystes en entreprise, PostgreSQL est la référence. Le module M07 (SQL analyste) enseignera le **Dialecte ANSI** avec DuckDB/PostgreSQL — c'est la même grammaire.

7.4 Quand choisir PostgreSQL (cité)

Cinq cas où PostgreSQL est le bon choix : 1. **Multi-utilisateurs** (5+ analystes qui partagent la base). 2. **Production critique** (transactionnel, 24/7). 3. **Volumes** > 50 millions de lignes. 4. **Conformité** (audit, RGPD, sauvegardes automatiques). 5. **Compétences** SQL avancées (CTE, window functions, JSON natif).

Hors de ces cas, DuckDB fait le travail.

8. Exemple concret — l'import CSV en DuckDB (le geste canonique)

8.1 Le contexte

Le dossier du projet M06.P contient `quincaillerie_export.csv` (24 000 lignes × 42 colonnes) et `schema_3fn.sql` (la cible 3FN). Le **geste canonique** est : créer la base, lire le CSV avec `all_varchar=true`, insérer dans les 7 tables (avec filtre), vérifier par les 3 requêtes.

8.2 Script complet en Python

```
import duckdb
import os

DB = '03_exercices/dossier_M06/quincaillerie.duckdb'
CSV = '03_exercices/dossier_M06/quincaillerie_export.csv'

# 1. Ouvrir la base (en mode mémoire pour le test, persistant sinon)
con = duckdb.connect(DB)

# 2. Lire le DDL
ddl = open('03_exercices/dossier_M06/schema_3fn.sql').read()
con.execute(ddl)

# 3. Importer le brut en staging
con.execute(f"""
CREATE OR REPLACE TABLE export_brut AS
SELECT * FROM read_csv_auto('{CSV}', all_varchar=true)
""")

# 4. Insérer dans les tables cibles
con.execute("""
INSERT INTO client (id_client, nom, prenom, telephone, email, ville, fidelite_niveau)
SELECT DISTINCT CAST(id_client AS INTEGER), client_nom, client_prenom, client_telephone,
        client_email, ville_client, 'COMPTOIR'
FROM export_brut
WHERE TRY_CAST(id_client AS INTEGER) IS NOT NULL
      AND CAST(id_client AS INTEGER) < 900000 -- 3 fantômes écartés
""")
con.commit()

# 5. Vérifier par Q1
result = con.execute("""
SELECT v.id_vente, v.id_client FROM vente v
LEFT JOIN client c ON CAST(v.id_client AS INTEGER) = c.id_client
WHERE c.id_client IS NULL
""").fetchall()
print("Q1 :", len(result), "lignes (attendu 0)")
con.close()
```


8.3 Le verdict attendu

- `client` est peuplée avec **1 891** lignes.
- `produit` est peuplée avec **312** lignes (1 par référence unique).
- `mode_paiement` est peuplée avec **4** lignes.
- Q1 (intégrité référentielle) renvoie **0** ligne.
- La base persistant `quincaillerie.duckdb` fait ~ 1,2 Mo.

9. Démonstration pas à pas — la chaîne DuckDB en 6 étapes

9.1 Installer DuckDB

`pip install duckdb --no-cache-dir` — 8 Mo. Vérifier : `python3 -c "import duckdb"`.

9.2 Créer la base

```
import duckdb
con = duckdb.connect(':memory:') # ou fichier
```

9.3 Lire le CSV en staging

```
con.execute(f"CREATE OR REPLACE TABLE v AS SELECT * FROM read_csv_auto('{CSV}',
all_varchar=true)")
```

9.4 Insérer dans les tables cibles

```
con.execute("INSERT INTO client SELECT DISTINCT ... FROM v WHERE ...")
```

9.5 Vérifier par les 3 requêtes

`controles_integrite.sql` (cf. C01 §5.5). Attendu : Q1 = 0, Q2 = 0, Q3 = 5 lignes.

9.6 Sauvegarder

`EXPORT DATABASE 'backup_dir'` ou `con.close()` (la base fichier persiste).

La chaîne DuckDB en 6 étapes : installer, ouvrir, lire CSV, insérer filtré, vérifier 3 requêtes, sauvegarder. C'est la **même séquence** que C05 (transformation + dédoublonnage + jointure + verdict), appliquée à un SGBD. La différence est l'automatisation.

10. Erreurs fréquentes

- `pip install duckdb sans --no-cache-dir` — le cache `~/.cache/pip` grossit de 100 Mo. Règle du module : toujours `--no-cache-dir`.
- Importer avec `read_csv_auto` sans `all_varchar=true` — DuckDB devine les types et **masque les défauts** (cf. M05 C03 §5.3). Règle : texte avant conversion contrôlée.
- Ouvrir une base `.duckdb` en mode `:memory:` — la base est perdue dès qu'on ferme. Pour tester, OK ; pour la production, ouvrir un fichier.

- **Ne pas activer `PRAGMA foreign_keys = ON` dans SQLite** — sans ça, les FK sont des annotations non portées. C'est le **piège n°1** de SQLite.
- **`INSERT INTO SELECT` sans `WHERE`** — la base reçoit tout, y compris les défauts. Règle : toujours filtrer à l'insertion.
- **Oublier `con.commit()` en SQLite** — le commit n'est pas implicite comme en DuckDB. La règle : toujours `commit()` après chaque batch.
- **`shutil.copy` comme sauvegarde** — sans verrou, on peut capturer une base dans un état partiel. Règle : `.backup` pour SQLite, `EXPORT DATABASE` pour DuckDB, `pg_dump` pour PostgreSQL.
- **Croire que le client graphique peut remplacer la base** — le client n'est qu'une interface de lecture. La base est sur disque.

À retenir. Les 8 erreurs ci-dessus sont les pièges du quotidien d'un analyste qui installe un SGBD. La règle unique est **chaque SGBD a son piège d'installation** : DuckDB = cache pip, SQLite = `PRAGMA foreign_keys`, PostgreSQL = mot de passe.

11. Bonnes pratiques professionnelles

Dans les faits. Sur le terrain, **4 bugs sur 5** liés aux FK fantômes en SQLite viennent de l'oubli de `PRAGMA foreign_keys = ON`. C'est un scénario type : l'analyste crée 50 tables avec des FK, importe 100 000 lignes par script, et tout fonctionne — sauf que les FK ne sont pas portées. Quand 6 mois plus tard il supprime un client par `DELETE`, les ventes restent (parce que la FK n'est pas portée, SQLite ne refuse pas), et la base est désormais **incohérente** sans qu'aucune requête Q1 ne le détecte. Le `pragma` est l'affaire d'une seconde, et c'est la première ligne à écrire dans tout projet SQLite.

- **Documenter le mode d'ouverture** dans le README (`pip install duckdb --no-cache-dir; duckdb.connect('fichier.duckdb')`).
- **`all_varchar=true` à l'import**, puis conversion contrôlée par script Python ou SQL.
- **`PRAGMA foreign_keys = ON` à chaque session SQLite** (sinon, FK fantômes).
- **Trois sauvegardes** : `EXPORT DATABASE` (DuckDB), `.backup` (SQLite), `pg_dump` (PostgreSQL).
- **Tester la base** par les 3 requêtes `controles_integrite.sql` à chaque modification.
- **Documenter la version** du SGBD dans le README (DuckDB 1.5.5, SQLite 3.45, PostgreSQL 16.x).

Conseil professionnel. Le verdict d'une base n'est pas « elle tourne » ; c'est « les 3 requêtes renvoient 0 ligne ». Cette discipline distingue une base d'un fichier plat. Et c'est ce que le projet M06.P évalue.

12. Exercice guidé — la chaîne DuckDB en 6 étapes (30 min, /15)

Objectif. Mobiliser la chaîne complète : installer, ouvrir, importer, insérer, vérifier, sauvegarder.

Énoncé. Sur le `quincaillerie_export.csv` (24 000 lignes) : 1. `pip install duckdb --no-cache-dir` 2. Ouvrir `:memory:`, créer la table `client(...)` du C02 §5.6. 3. Lire le CSV avec `all_varchar=true` dans une table `v`. 4. Insérer dans `client` (5 lignes seulement) avec `INSERT INTO ... SELECT DISTINCT ... FROM v LIMIT 5`. 5. Vérifier : `SELECT * FROM client` → 5 lignes. 6.

Fermer (`con.close()`) et rouvrir. Vérifier que la base est vide (`:memory:` n'a pas de persistance — c'est l'erreur pédagogique attendue).

Barème (/15). 2 pt par étape + 5 pt bonus si l'étape 6 est comprise (`:memory:` ≠ fichier).

13. Exercices autonomes

- **Exercice 13.1 (15 min).** Listez les 3 modes d'ouverture de DuckDB (`:memory:`, fichier, `read_only`). Donnez un cas d'usage pour chacun.
- **Exercice 13.2 (30 min).** Reproduisez la chaîne DuckDB du § 8.2 sur le `quincaillerie_export.csv`. Vérifiez Q1, Q2, Q3. Notez les temps.
- **Exercice 13.3 (45 min).** Ouvrez la base SQLite `quincaillerie.sqlite` depuis Python, exécutez les 3 requêtes `controles_integrite.sql`. Notez 3 différences syntaxiques avec DuckDB.
- **Exercice 13.4 (60 min, optionnel).** Générez un `EXPORT DATABASE` de votre base DuckDB, ouvrez les fichiers `.parquet` produits avec `pandas.read_parquet()`. Commentez l'opération : est-ce portable ? est-ce lisible sans DuckDB ?

14. Correction détaillée

- **Exercice 13.1.** Trois modes : (a) `:memory:` — tests, exploration, pas de persistance ; (b) fichier `.duckdb` — production, persistance, sauvegarde ; (c) `read_only=True` — consultation seule (client graphique, audit, sécurité).
- **Exercice 13.2.** Temps attendus : import CSV ≈ 80 ms (24 000 lignes) ; insertion client ≈ 30 ms ; Q1 ≈ 5 ms. **Total ≈ 120 ms.** Sur l'atelier, c'est sub-second ; sur un poste de production, c'est aussi sub-second.
- **Exercice 13.3.** Différences attendues : SQLite utilise `INTEGER` (DuckDB aussi, mais peut utiliser `BIGINT`) ; SQLite n'a pas de `SHOW TABLES` (utiliser `sqlite_master`) ; SQLite n'a pas `EXPORT DATABASE` (utiliser `.backup`).
- **Exercice 13.4.** Opération portable : oui, les Parquet sont universels (`pandas`, `pyarrow`, `polars`, `Spark`, `Athena`, `BigQuery`). Opération lisible sans DuckDB : oui, Parquet est auto-descriptif. La seule chose perdue est la **vue** `ligne_vente_ttc` — il faut la redéfinir dans le `schema.sql` lors de la restauration.

15. Mini-projet M06.P3 — « L'import contrôlé du brut » (1 h)

Énoncé. En reprenant le script § 8.2, **compléter** pour atteindre le verdict du projet M06.P : 1. Ouvrir la base en mode fichier (`quincaillerie.duckdb`). 2. Créer les 7 tables du `schema_3fn.sql`. 3. Importer 24 000 ventes dans `vente` (filtrées). 4. Importer 5 000 lignes `ligne_vente` (à partir des mêmes données, 1 produit par vente). 5. **Vérifier** par les 3 requêtes `controles_integrite.sql`. 6. **Sauvegarder** par `EXPORT DATABASE 'backup_dir'`. 7. **Documenter** dans un rapport (1 page) : lignes par table, totaux, anomalies trouvées.

Critères de réussite.

1. La base `quincaillerie.duckdb` fait < 2 Mo.
2. Les 3 requêtes renvoient 0/0/5.
3. Le **rapport** liste 4 anomalies trouvées (parmi les 142 montants texte, 8 doublons, 12 dates, 3 clients orphelins, ...) avec leur traitement.

4. La **sauvegarde** crée un dossier `backup_dir/` avec ≥ 5 fichiers Parquet.

Barème (/15). 3 pt par critère vérifié, 3 pt pour le rapport, 2 pt bonus pour la vérification cumulative (les 3 requêtes d'un coup).

16. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — vivre avec un SGBD sans drame :

1. **La chaîne DuckDB en 6 étapes** (§ 9) — installer, ouvrir, importer, insérer, vérifier, sauvegarder.
2. **Les 3 modes d'ouverture DuckDB** (§ 5.2) — `:memory:`, fichier, `read_only`.
3. **Les 2 voies d'insertion** (§ 5.4) — `INSERT INTO ... VALUES` (test) vs `INSERT INTO ... SELECT ... FROM read_csv_auto()` (import).
4. **Le PRAGMA foreign_keys = ON SQLite** (§ 6.1) — l'activation sans laquelle FK est fantôme.
5. **Les 5 différences DuckDB / SQLite** (§ 6) — natif, pragma, parquet, sauvegarde, CSV.
6. **Les 3 verbes de sauvegarde** — `EXPORT DATABASE` (DuckDB), `.backup` (SQLite), `pg_dump` (PostgreSQL).
7. **Le client graphique cité** (§ 5.5) — DBeaver, DataGrip, pgAdmin : utile pour 100 000 lignes, pas pour les tests.

17. Health Check (auto-correction, 6 questions, 15 min)

Q1. Quel est l'intérêt de `pip install duckdb --no-cache-dir` ? (Réponse : éviter que le cache `pip` ne grossisse de 100 Mo et sature le quota persistant de l'atelier.)

Q2. Pourquoi `all_varchar=true` à l'import CSV ? (Réponse : empêcher DuckDB autoloading de convertir silencieusement les colonnes sales — on veut garder la trace des défauts.)

Q3. Quel est le piège classique de SQLite ? (Réponse : `PRAGMA foreign_keys = ON` n'est pas activé par défaut, donc les FK sont des annotations non portées.)

Q4. Quand choisit-on DuckDB, SQLite, PostgreSQL ? (Réponse : DuckDB = analyse solo, SQLite = embarqué, PostgreSQL = multi-utilisateurs.)

Q5. Quelle commande DuckDB exporte toute la base ? (Réponse : `EXPORT DATABASE 'backup_dir'.`)

Q6. Pourquoi `:memory:` n'est-il pas utilisable en production ? (Réponse : la base est en mémoire RAM et perdue à la fermeture de la connexion — pas de persistance.)

Q7. Que font les 3 requêtes `controles_integrite.sql` ? (Réponse : Q1 vérifie l'intégrité référentielle (0 ligne) ; Q2 vérifie la cohérence des montants (0 ligne) ; Q3 calcule le total par catégorie (5 lignes).)

Q8. Quels sont les 3 verbes de sauvegarde ? (Réponse : `EXPORT DATABASE` pour DuckDB, `.backup` pour SQLite, `pg_dump` pour PostgreSQL.)

Q9. Citez 2 clients graphiques pour PostgreSQL. (Réponse : pgAdmin (officiel), DBeaver (universel), DataGrip (JetBrains).)

Q10. Pourquoi la commande `pg_dump` n'est-elle pas exécutable dans l'atelier ? (Réponse : parce que PostgreSQL n'est pas installé dans cet atelier — c'est la règle 7 du plan M05 reconduite pour M06 ; on cite sans exécuter.)

18. Résumé du chapitre

- **DuckDB** est un SGBD intégré (in-process), mono-utilisateur, gratuit. Il lit CSV, Parquet, JSON, fait du SQL ANSI.
- **SQLite** est un SGBD embarqué, natif Python 3.13, mono-utilisateur. Le `fichier.sqlite` est la base.
- **PostgreSQL** est un SGBD serveur, multi-utilisateurs, open source. C'est la référence de la production, **non installée** dans cet atelier (règle 7 du plan M05 reconduite).
- L'**import CSV** canonique est `INSERT INTO ... SELECT ... FROM read_csv_auto('fichier.csv', all_varchar=true)`.
- **Trois sauvegardes** : `EXPORT DATABASE` (DuckDB), `.backup` (SQLite), `pg_dump` (PostgreSQL).
- La **chaîne DuckDB en 6 étapes** (installer, ouvrir, importer, insérer, vérifier, sauvegarder) est l'épreuve de la pratique.

19. À retenir

À retenir. Le choix DuckDB / SQLite / PostgreSQL est une question de contraintes locales (utilisateurs, volumétrie, persistance), pas une question de goût. DuckDB en première intention pour l'analyste solo, SQLite pour embarquer, PostgreSQL pour partager. Le bon professionnel sait quand basculer : c'est la règle des 5 critères du C05 + la simplicité de mise en place (DuckDB : aucune ; SQLite : natif ; PostgreSQL : serveur à installer).

À retenir. `all_varchar=true` est le **garde-fou** de l'import CSV. Sans lui, DuckDB autoloade convertit silencieusement « 1 200 FCFA » en 1200, et la trace des défauts est **perdue**. Avec, la colonne reste `TEXT` jusqu'à la conversion contrôlée par script. C'est la même discipline que M04 §5.6 sur les formats : charger brut, transformer sous contrôle.

À retenir. Le client graphique (DBeaver, DataGrip, pgAdmin) est **cité non exécuté** dans cet atelier — c'est l'usage quotidien d'un analyste sur 100 000+ lignes, mais il n'est pas nécessaire pour un atelier de formation. Le CLI `duckdb fichier.duckdb` suffit pour les tests ; le client graphique est l'outil du confort, pas l'outil de la preuve.

20. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 3 SGBD du module et leur cas d'usage. (Réponse : DuckDB = analyste solo intégré ; SQLite = embarqué mono-utilisateur ; PostgreSQL = serveur multi-utilisateurs.)

Question 2. Quelle est la différence entre `EXPORT DATABASE` et `pg_dump` ? (Réponse : les deux exportent la base, mais DuckDB exporte en Parquet + SQL tandis que PostgreSQL exporte en SQL texte uniquement.)

Question 3. Pourquoi `PRAGMA foreign_keys = ON` est-il indispensable en SQLite ? (Réponse : sans ça, les FK sont des annotations non portées — la base ne refuse pas les orphelins.)

Question 4. Quel est le piège de `read_csv_auto` sans `all_varchar=true` ? (Réponse : DuckDB autoloade convertit silencieusement « 1 200 FCFA » en 1200 et masque le défaut métier.)

Question 5. Que fait la commande `EXPORT DATABASE 'backup_dir'` ? (Réponse : exporte toute la base DuckDB en Parquet (tables) + `schema.sql` (DDL), portable vers n'importe quel outil Parquet.)

Question 6. Citez 3 clients graphiques de PostgreSQL. (Réponse : pgAdmin, DBeaver, DataGrip.)

Question 7. Pourquoi ne pas exécuter PostgreSQL dans cet atelier ? (Réponse : la règle 7 du plan M05 reconduite pour M06 ; l'installation d'un serveur PostgreSQL est ≈ 150 Mo et 5-10 min, hors budget de l'atelier ; on cite sans exécuter.)

Question 8. Que se passe-t-il si on ferme une connexion `:memory:` ? (Réponse : toute la base est perdue — c'est le comportement attendu d'une base volatile.)

Module M06.C05 — Les autres mondes : NoSQL (document, clé-valeur, graphe, colonnes), datalake, entrepôt, lac

Outils : aucun outil à installer. Vocabulaire d'entretien. Durée indicative : 5 h. Niveau : N2. Prérequis : M06.C01, M06.C02, M06.C03.

L'idée du chapitre. Les quatre chapitres précédents ont installé le **SGBD relationnel** (DuckDB, SQLite, PostgreSQL). Le C05 regarde au-delà : ce qui vient autour, à côté et au-dessus du relationnel. Le C05 n'est pas un détour : c'est l'**ouverture professionnelle** du module. Un analyste qui ne connaît que les SGBD relationnels a une vision tronquée du paysage. Le module M06 lui donne les **5 mots d'entretien** (NoSQL, datalake, entrepôt, lac, lakehouse) et les **5 cas** où un autre modèle est meilleur. Le verdict du C05 est « l'analyste sait où il est, et où il n'est pas ».

Base de travail — pas de fichier livré pour ce chapitre. Le C05 est **purement descriptif**. Il n'enseigne aucun outil à installer (NoSQL n'est pas dans l'atelier), aucun pipeline à exécuter, aucune table à vérifier. C'est le chapitre qui prépare l'entretien d'embauche et le vocabulaire transversal du data engineer. Le mini-projet M06.P4 est le seul livrable « concret » : un glossaire de 200 mots qui couvre les 5 mots d'entretien.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **définir** NoSQL, et distinguer ses **4 familles** (document, clé-valeur, graphe, colonnes) ;
- **expliquer** pourquoi NoSQL ne remplace pas le relationnel mais le complète ;
- **situer** datamart, datawarehouse (entrepôt), datalake (lac) et lakehouse dans le paysage ;
- **expliquer** les **5 mots d'entretien** : sharding, réplication, CAP, ETL vs ELT, ACID ;
- **reconnaître** le bon choix pour un cas d'usage (transactionnel vs analytique, structuré vs non-structuré) ;
- **rédiger** un glossaire de 200 mots sur les 5 mots d'entretien (mini-projet M06.P4).

2. Pourquoi cette notion est importante

- Le module M06 a installé le **relationnel** sur **50 % du paysage** (cf. C01 § 5.6). L'autre moitié est **NoSQL et Big Data**. Le C05 ferme la porte en montrant ce qui existe à côté — sans l'enseigner en profondeur, parce que c'est l'objet de modules ultérieurs (M11 « SQL avancé », M13 « Modélisation », M14 « Power BI »).
- **NoSQL** est un raccourci pour « Not only SQL ». Ce n'est pas « No SQL » (pas de SQL) ; c'est « Pas seulement SQL ». Le C05 enseigne les **4 familles** (document, clé-valeur, graphe, colonnes) et les **cas d'usage** de chacune.
- Le **datalake** et le **datawarehouse** (entrepôt) sont des concepts complémentaires, pas opposés. Le datawarehouse stocke des **données structurées** (et validées) ; le datalake stocke **tout** (CSV, JSON, images, logs). Le **lakehouse** combine les deux. C'est l'architecture moderne.

- Les **5 mots d'entretien** sont ce qui distingue un junior (qui les a lus) d'un senior (qui les a vécus). Le C05 les explique en **moins de 200 mots** par mot, pour que l'analyste junior puisse répondre en entretien sans bluff.
- Le **paysage** est ce qui permet de situer un projet : « ce projet est transactionnel, mono-utilisateur, structuré → SQLite » ; « ce projet est transactionnel, multi-utilisateurs, gouvernance forte → PostgreSQL » ; « ce projet est non-structuré, multi-utilisateurs, documente → MongoDB ». C'est la **compétence d'orientation** du professionnel.

3. Explication simple — la carte du monde des données

Le **paysage** se lit en **4 axes** :

1. **Structure** — les données sont-elles **typées** (SGBDR), **semi-structurées** (JSON, MongoDB), ou **non-structurées** (texte brut, images) ?
2. **Fréquence** — la base est transactionnelle (caisse, bancaire) ou analytique (rapports, agrégats) ?
3. **Utilisateurs** — 1 (analyste solo), quelques-uns (équipe), milliers (grand public) ?
4. **Gouvernance** — locale (un fichier), centralisée (schéma SQL), fédérée (multi-sites) ?

Le **résumé en une phrase** : « Un bon choix d'outil dépend de où on est sur ces 4 axes. »

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
NoSQL — Not only SQL	la famille des bases non-relationnelles : document, clé-valeur, graphe, colonnes larges.	confondre avec « pas de SQL » : NoSQL complète le relationnel, ne le remplace pas.
base document — document store	la base qui stocke des JSON-like (MongoDB, CouchDB). Chaque document est typé mais sans schéma imposé.	confondre avec « base non structurée » : MongoDB a des schémas implicites (validés au niveau applicatif).
base clé-valeur — key-value store	la base qui stocke des paires clé → valeur (Redis, DynamoDB). Ultra-rapide en lecture/écriture simple.	confondre avec « pas de clé » : la clé est l'identité, la valeur est opaque.
base graphe — graph database	la base qui stocke des nœuds + arêtes (Neo4j, JanusGraph). Optimisée pour les parcours de graphe.	confondre avec « table de jointure » : un graphe est une structure dédiée, pas une astuce relationnelle.
base colonnes — wide-column store	la base qui stocke des colonnes par famille (Cassandra, Bigtable). Optimisée pour les lectures massives en colonne.	confondre avec « colonne SQL » : c'est une organisation physique différente, pas un type logique.
MongoDB — MongoDB	le SGBD document le plus populaire, JSON natif, schéma flexible, écrit en C++. Utilisé par 50 % des startups en 2024.	croire qu'il remplace PostgreSQL : MongoDB a des transactions depuis 4.0 (2018), mais reste plus lent sur les jointures.
Redis — Redis	le SGBD clé-valeur ultra-rapide, mémoire RAM, persistance sur disque en option. Utilisé pour le cache, les files, les compteurs.	confondre avec « base principale » : Redis est un cache, pas une base canonique.
Neo4j — Neo4j	le SGBD graphe le plus populaire, écrit en Java, requête en Cypher (<code>MATCH (a)-[:KNOWS]->(b) RETURN b.name</code>).	croire qu'il est réservé aux réseaux sociaux : Neo4j sert aussi pour la traçabilité alimentaire, les graphes de dépendance, etc.
Cassandra — Apache Cassandra	le SGBD colonnes larges distribué, écrit en Java, optimisé pour l'écriture massive. Utilisé par Netflix, Apple, Uber.	confondre avec « colonne » : Cassandra organise les données par famille de colonnes, pas par colonnes SQL.

Français — English	Définition simple	Piège à éviter
NewSQL — NewSQL	la famille des SGBD relationnels distribués (CockroachDB, Spanner, YugabyteDB). Le meilleur des deux mondes : SQL + scalable horizontalement.	confondre avec « PostgreSQL distribué » : NewSQL ajoute la distribution (sharding automatique) sans abandonner le SQL.
datawarehouse — data warehouse, DWH	l' entrepôt de données : un SGBD analytique (SQL Server, Snowflake, BigQuery, Redshift) où les rapports sont calculés.	confondre avec « base de production » : le DWH est séparé de la base transactionnelle.
datalake — data lake, lac	le lac de données : un stockage brut (S3, HDFS) qui contient des CSV, JSON, Parquet, images, logs, sans validation préalable.	confondre avec « SGBD » : un lac ne valide pas, c'est juste un gros disque.
lakehouse — lakehouse	l' architecture hybride : un lac + un entrepôt, où les données sont stockées brutes et validées en SQL via Delta Lake ou Apache Iceberg.	croire qu'il faut choisir : lakehouse est les deux à la fois.
ETL — Extract, Transform, Load	le pipeline classique : on transforme les données avant de les charger (par exemple, dans un SGBDR).	confondre avec « copie de fichiers » : ETL a une logique de nettoyage.
ELT — Extract, Load, Transform	le pipeline moderne : on charge d'abord (dans le lac), puis on transforme dans le SGBD analytique.	croire qu'il faut choisir : ELT est le standard moderne (datalake → entrepôt).
sharding — sharding, partitionnement horizontal	la répartition des données sur plusieurs machines (ex. : les clients A-M sur le serveur 1, N-Z sur le serveur 2). C'est la scalabilité horizontale des SGBD distribués.	confondre avec « partition verticale » : sharding = par lignes, partition verticale = par colonnes.
réplication — replication	la copie asynchrone (ou synchrone) des données sur plusieurs machines. Maître-esclave (un maître, plusieurs copies) ou multi-maître (plusieurs maîtres qui se synchronisent).	confondre avec « sauvegarde » : la réplication est temps réel, la sauvegarde est périodique.
CAP — CAP theorem, Brewer's theorem	l'impossibilité d'avoir simultanément Cohérence, Disponibilité, Tolérance au partitionnement dans un SGBD distribué. Il faut en choisir 2 sur 3.	croire qu'on peut tout avoir : c'est mathématiquement impossible, le théorème de Brewer est prouvé.
ACID — Atomicity, Consistency, Isolation, Durability	les 4 propriétés d'une transaction SGBD (cf. C01 §4 : atomique, cohérent, isolé, durable). Tous les SGBDR respectent ACID.	confondre avec BASE (cf. ci-dessous) : NoSQL « sacrifie » souvent ACID pour la disponibilité (BASE).
BASE — Basically Available, Soft state, Eventually consistent	l' alternative à ACID pour les SGBD distribués NoSQL : on accepte une cohérence différée au profit de la disponibilité.	confondre avec « pas de cohérence » : BASE est cohérent à terme (les lectures lointaines convergent), pas « n'importe quoi ».

Définition. Le **NoSQL** (Not only SQL) est la famille des bases **non-relationnelles**, optimisées pour des cas où le relationnel est maladroit : documents semi-structurés (MongoDB), paires clé-valeur ultra-rapides (Redis), graphes de relations (Neo4j), colonnes larges distribuées (Cassandra). NoSQL ne remplace pas le relationnel ; il le complète. La règle du module est « choisir NoSQL quand le relationnel est inadapté, pas par mode. » — English : Not only SQL.

Définition. Le **datawarehouse** (entrepôt) est un SGBD **analytique** où les données validées sont stockées pour les rapports (tableaux de bord, SQL analytique, OLAP). Le **datalake** (lac) est un stockage brut où les données sont conservées telles quelles (CSV, JSON, images, logs). Le **lakehouse** combine les deux : les données sont stockées brutes et validées en SQL via des couches comme Delta Lake ou Apache Iceberg. La règle du module est DWH pour les rapports, lac pour l'exploration, lakehouse pour les deux. — English : data warehouse, data lake, lakehouse.

Définition. Le **théorème CAP** (Brewer, 2000) énonce qu'un SGBD **distribué** (sur plusieurs machines) ne peut pas avoir simultanément **Cohérence**, **Disponibilité**, **Tolérance au partitionnement**. Le choix se fait entre **CP** (cohérence + partitionnement, e.g. HBase) ou **AP** (disponibilité + partitionnement, e.g. Cassandra) ou **CA** (cohérence + disponibilité, impossible en distribué — c'est l'apanage du SGBD centralisé). Le module le cite pour que l'analyste sache pourquoi NoSQL est différemment consistant. — English : CAP theorem.

Attention. **NoSQL ne supprime pas le schéma** : il le rend implicite. Sur MongoDB, le document `{nom: "Alice", age: 30}` est valide parce que l'application le crée ainsi. Si on change le code applicatif, les anciens documents restent dans l'ancien format. La **discipline** est de documenter le schéma attendu dans l'application, pas dans la base. C'est l'inverse de la discipline relationnelle, où la base porte le schéma (`CREATE TABLE`). Pour un junior, c'est une surprise ; pour un senior, c'est une responsabilité.

Attention. Le **sharding** est une décision **irréversible**. Si on commence avec un sharding par `id_client`, on ne peut pas revenir en arrière sans **réindexer toute la base**. Le piège classique est de sharder trop tôt (avant d'avoir 100 millions de lignes, le SGBD centralisé tient). La règle du module est sharder quand le SGBD centralisé ne tient plus — pas avant. Sans cette discipline, on prend une décision de 6 mois qu'on regrettera.

5. Cours approfondi — les 4 familles NoSQL

5.1 Document store (MongoDB, CouchDB)

Cas d'usage. Données **semi-structurées** (JSON, BSON, hiérarchiques) où le **schéma change souvent**. Exemples : profils utilisateurs (chacun avec des champs différents), catalogues produits (attributs variables), IoT (capteurs divers).

Exemple MongoDB.

```
db.users.insertOne({
  nom: "Alice",
  age: 30,
  adresses: [
    {ville: "Koudougou", pays: "BF"},
    {ville: "Bobo", pays: "BF"}
  ],
  preferences: {langues: ["fr", "moore"], fidelite: "OR"}
});
```

Le **schéma** est implicite dans la structure du document ; c'est le code applicatif qui le valide.

5.2 Clé-valeur (Redis, DynamoDB)

Cas d'usage. Cache, sessions, files d'attente, compteurs. La rapidité en lecture/écriture simple prime. Exemples : panier d'achat (id_session → liste de produits), cache HTTP (URL → réponse), leaderboard (id_joueur → score).

Exemple Redis.

```
SET user:1000 "Alice"      # clé → valeur
GET user:1000              # "Alice"
INCR compteur:visites:home # +1 atomique
HSET user:1000 nom "Alice" age 30 # hash (sous-clés)
```

Redis vit en mémoire RAM : c'est la base la plus rapide, et c'est aussi la plus volatile (les données sont perdues au redémarrage, sauf configuration `appendonly`).

5.3 Graphe (Neo4j, JanusGraph)

Cas d'usage. Réseaux sociaux, dépendances logicielles, traçabilité alimentaire, graphes de connaissances, lutte anti-fraude. La **structure de graphe** (nœuds + arêtes) est native : une requête qui parcourt 3 niveaux est directe (pas de 3 jointures).

Exemple Neo4j (Cypher).

```
MATCH (a:Person)-[:KNOWS]->(b)-[:KNOWS]->(c)
WHERE a.name = 'Alice'
RETURN c.name, count(*) AS amis_de_amis
```

Le **Cypher** est déclaratif : on décrit le **parcours**, on ne code pas l'algorithme. C'est 10 × plus court qu'une jointure SQL équivalente.

5.4 Colonnes larges (Cassandra, Bigtable, HBase)

Cas d'usage. Écriture massive en continu (capteurs IoT, logs de serveurs, historique de prix). Le **modèle** est colonnes par famille : on regroupe les colonnes par « famille » (par exemple, `info_personnelles`, `metriques`, `preferences`) et on charge par famille.

Exemple Cassandra (CQL).

```
CREATE TABLE users (
  id_user    UUID PRIMARY KEY,
  nom        TEXT,
  age        INT,
  ville      TEXT
) WITH COMPACT STORAGE;
```

Cassandra est optimisée pour **l'écriture** ; la lecture par clé est rapide, mais le scan par colonne est moins efficace que sur un SGBD analytique.

6. Cours approfondi — datamart, datawarehouse, datalake, lakehouse

6.1 Le datawarehouse (l'entrepôt, ~30 ans d'histoire)

- **Origine** : années 1990 (Bill Inmon, Ralph Kimball).
- **Concept** : un SGBD analytique où les données sont **validées, agrégées, et orientées sujet**.
- **Outils** : Teradata, Oracle Exadata, SQL Server, Snowflake, BigQuery, Redshift.

- **Cas d'usage** : rapports consolidés (CA par mois par rayon), OLAP, indicateurs de performance — KPI (Key Performance Indicator).

6.2 Le datamart (le sous-entrepôt)

- **Concept** : un sous-ensemble du DWH, orienté **un sujet métier** (finance, marketing, RH).
- **Avantage** : plus simple, plus rapide à mettre en place.
- **Inconvénient** : risque de silos (chaque datamart a sa propre logique).

6.3 Le datalake (le lac, ~10 ans d'histoire)

- **Origine** : 2010 (James Dixon, Pentaho).
- **Concept** : un stockage **brut** (S3, HDFS, blob Azure) où les CSV, JSON, Parquet, images, logs sont conservés telles quelles, sans validation préalable.
- **Avantage** : on peut tout stocker (logs de 5 ans, archives, médias).
- **Inconvénient** : c'est un marécage (swamp) si on ne le gère pas. La phrase consacrée est « datalake » → « data swamp » en 6 mois.

6.4 Le lakehouse (la combinaison ~5 ans)

- **Origine** : 2020 (Databricks Delta Lake, Apache Iceberg, Apache Hudi).
- **Concept** : un lac avec une **couche de validation SQL**. Les données sont stockées brutes, mais les requêtes SQL passent par une couche qui impose un schéma (sans modifier le fichier d'origine).
- **Outils** : Databricks, Snowflake, BigQuery, DuckDB (sur Parquet natif).
- **Avantage** : le meilleur des deux mondes.

7. Cours approfondi — les 5 mots d'entretien

7.1 Sharding (partitionnement horizontal)

Le **sharding** répartit les lignes d'une table sur **plusieurs machines**. Chaque machine (nœud) porte un **sous-ensemble** des lignes. Exemple : 1 milliard de clients, répartis en 10 nœuds de 100 millions chacun. Le SGBD route la requête vers le bon nœud en fonction de la **clé de sharding** (souvent `id_user % 10`).

Prérequis : 100 millions de lignes minimum (en dessous, le SGBD centralisé tient).

7.2 Réplication (copie asynchrone)

La **réplication** copie les données d'une machine **maître** vers une ou plusieurs machines **esclaves**. C'est l'inverse du sharding : on copie toute la base, pas une partie. C'est le mécanisme de la **haute disponibilité** : si le maître tombe, un esclave prend le relais.

Modes : synchrone (la transaction attend que l'esclave confirme) ou asynchrone (la transaction commit sur le maître, l'esclave copie en arrière-plan, avec un délai).

7.3 Théorème CAP (Brewer)

Un SGBD **distribué** ne peut pas avoir simultanément : - **Cohérence** (tous les nœuds voient la même valeur au même moment) ; - **Disponibilité** (chaque requête reçoit une réponse, succès ou échec) ; - **Tolérance au partitionnement** (le système continue à fonctionner même si un nœud est isolé).

Le choix est forcé : un SGBD distribué fait 2 sur 3. **CP** (cohérence + partition, e.g. HBase) ou **AP** (disponibilité + partition, e.g. Cassandra) ou **CA** (impossible en distribué — c'est le SGBD centralisé classique).

7.4 ACID vs BASE (modèle transactionnel)

Définition. Le **modèle ACID** (Atomicity, Consistency, Isolation, Durability — atomique, cohérent, isolé, durable) est le contrat des transactions SGBDR : chaque transaction est garantie atomique (tout ou rien), cohérente (la base reste valide), isolée (les autres transactions ne voient pas l'état intermédiaire), durable (la modification survit à un crash). Le modèle **BASE** (Basically Available, Soft state, Eventually consistent) est le modèle alternatif pour les SGBD distribués NoSQL : on accepte une cohérence différée au profit de la disponibilité. ACID pour le bancaire, BASE pour l'IoT ou le cache. C'est le choix du C05.

ACID : Atomicité, Cohérence, Isolation, Durabilité. C'est le modèle **relationnel** : chaque transaction est garantie isolée des autres et durable.

BASE : Basically Available, Soft state, Eventually consistent. C'est le modèle **NoSQL distribué** : on accepte une cohérence différée au profit de la disponibilité.

Propriété	ACID	BASE
Cohérence	immédiate	à terme
Disponibilité	réduite (verrous)	élevée
Cas d'usage	bancaire, transactionnel	IoT, cache, analytique

7.5 ETL vs ELT

ETL (Extract, Transform, Load) — pipeline classique. On extrait, on transforme (dans un outil dédié comme Informatica), puis on charge dans l'entrepôt. La transformation est **hors** du SGBD analytique.

ELT (Extract, Load, Transform) — pipeline moderne. On extrait, on charge brut (dans le lac), puis on transforme dans le SGBD analytique (BigQuery, Snowflake). Le SGBD analytique **est** la machine de transformation.

8. Exemple concret — le verdict du paysage

Pour le projet M06.P « La base de la quincaillerie », où se trouve la base ?

Axe	Valeur	Outil adapté
Structure	typée (3FN, 7 tables)	SGBDR (DuckDB / SQLite / PostgreSQL)
Fréquence	analytique (rapports hebdo)	SGBDR analytique (DWH) si volumétrie
Utilisateurs	1 analyste (Koudougou)	DuckDB intégré
Gouvernance	locale (un fichier)	DuckDB + EXPORT DATABASE quotidien

Le verdict : DuckDB est le bon outil. Le projet M06.P est 100 % relationnel. NoSQL et Big Data sont hors de ce projet, mais restent dans **l'horizon professionnel** de l'analyste qui veut grandir.

9. Démonstration pas à pas — les 5 mots d'entretien

9.1 Préparer une réponse à « C'est quoi le sharding ? »

« Le sharding est la répartition horizontale des lignes d'une table sur plusieurs machines. C'est la scalabilité des SGBD qui grossissent au-delà du téraoctet. Le piège est de sharder trop tôt en dessous de 100 millions de lignes, le SGBD centralisé tient, et sharding est irréversible. » (58 mots)

9.2 Préparer une réponse à « ACID ou BASE ? »

« ACID pour les transactions bancaires (chaque virement doit être garanti). BASE pour les capteurs IoT (on préfère perdre 1 % des lectures plutôt que bloquer le capteur). Le théorème CAP nous dit qu'on ne peut pas tout avoir en distribué. » (49 mots)

9.3 Préparer une réponse à « DWH ou datalake ? »

« Les deux, en lakehouse. Le datalake stocke le brut ; le DWH stocke le validé ; le lakehouse garde les deux couches et permet des requêtes SQL sur le brut via Parquet. C'est l'architecture moderne. » (39 mots)

Les 5 mots d'entretien se préparent en **moins de 60 mots chacun**. C'est la règle du « dire peu, dire juste » (cf. M03.C01 § 8 sur la communication).

10. Erreurs fréquentes

- **Croire que NoSQL remplace SQL** : c'est l'erreur de compréhension n°1. NoSQL complète le relationnel, ne le remplace pas. La règle est SQL pour 80 % des cas, NoSQL pour 20 %.
- **Sharder trop tôt** : la décision est irréversible. La règle est sharder quand le SGBD centralisé ne tient plus (1 To+).
- **Confondre datalake et DWH** : le lac stocke du brut, l'entrepôt stocke du validé. Le lakehouse combine les deux.
- **Croire que MongoDB est plus rapide que PostgreSQL** : faux dans 80 % des cas. MongoDB est rapide pour les documents JSON ; sur les jointures classiques, PostgreSQL gagne.
- **Oublier ACID dans une transaction bancaire** : c'est l'erreur classique. La règle est toujours ACID pour les transactions critiques (banque, paie, facturation).
- **Confondre réplication et sauvegarde** : la réplication est temps réel, la sauvegarde est périodique. Les deux sont complémentaires (la réplique peut tomber en panne au même moment que le maître, la sauvegarde non).
- **Croire que « BASE = pas de cohérence »** : c'est faux. BASE = cohérent à terme (les lectures lointaines convergent), pas « n'importe quoi ». C'est une nuance qui compte en entretien.

À retenir. Les 7 erreurs ci-dessus sont les **7 pièges** qu'un analyste junior fait en entretien. Le test du C05 est pouvoir répondre aux 5 mots en moins de 60 mots chacun.

11. Bonnes pratiques professionnelles

Dans les faits. L'anecdote qui justifie la prudence sur le datalake : un cadre d'une grande banque française disait en 2018 « nous avons 14 pétaoctets de datalake, et nous ne savons pas ce qu'il y a dedans ». C'est ce qu'on appelle un **data swamp** : un lac sans gouvernance qui accumule les fichiers sans qu'on sache les utiliser. La parade est imposer un schéma implicite dès l'ingestion (par exemple Delta Lake), avec un petit fichier `_schema.json` qui déclare les colonnes attendues. Sans cette discipline, le lac devient un marécage en 6-12 mois. Le coût d'un lakehouse bien géré est équivalent à celui d'un DWH bien géré ; le coût d'un lac mal géré est infiniment supérieur (parce qu'on ne peut plus rien en extraire).

- **Choisir NoSQL quand le relationnel est inadapté**, pas par mode. Le réflexe junior est « on va essayer MongoDB » ; le réflexe senior est « quelle est la donnée ? quelle est la requête ? choisir ensuite. »
- **Documenter le paysage** dans le README du projet (DWH, lac, lakehouse) pour que les nouveaux venus s'orientent.
- **Connaître les 5 mots d'entretien** par cœur (sharding, réplication, CAP, ACID/BASE, ETL/ELT), en moins de 60 mots chacun.
- **Différencier OLTP et OLAP** : OLTP = transactionnel (caisse), OLAP = analytique (DWH).
- **Savoir sharder** (en théorie) sans sharder en pratique avant 100 millions de lignes.

Conseil professionnel. Un entretien data, c'est 70 % de SQL et 30 % de paysage. Le C05 installe les 30 % qui manquent à un junior qui ne connaît que SQL.

12. Exercice guidé — préparer 5 réponses d'entretien (30 min, /15)

Objectif. Mobiliser les 5 mots d'entretien dans **des phrases courtes et justes**.

Énoncé. Pour chacun des 5 mots (sharding, réplication, CAP, ACID vs BASE, ETL vs ELT), rédigez **une réponse de 50-80 mots** qui : - donne la définition ; - donne un cas d'usage ; - cite un outil qui l'illustre.

Barème (/15). 3 points par mot (définition 1, cas 1, outil 1).

13. Exercices autonomes

- **Exercice 13.1 (15 min).** Dessinez la **carte du paysage** (§ 6) en 4 colonnes (datamart, DWH, datalake, lakehouse) avec : structure, fréquence, gouvernance, coût.
- **Exercice 13.2 (30 min).** Pour chaque famille NoSQL (document, clé-valeur, graphe, colonnes), trouvez **un cas d'usage concret dans votre entreprise** (ou dans une startup typique de la formation). Justifiez le choix.
- **Exercice 13.3 (45 min).** Prenez une grande entreprise (Netflix, Uber, Airbnb) et trouvez où ils utilisent chaque famille NoSQL. Cette recherche peut se faire sur le blog tech de l'entreprise.
- **Exercice 13.4 (60 min, optionnel).** Lisez l'article Wikipedia sur le théorème CAP et rédigez un résumé de 100 mots qui distingue CP, AP, CA avec un outil par catégorie.

14. Correction détaillée

- **Exercice 13.1.** Datamart : un sujet, rapide à monter, silos possibles. DWH : tous les sujets, unifié, plus long à monter. Datalake : tout, brut, marécage en 6 mois sans gouvernance. Lakehouse : les deux, moderne, nécessite Delta Lake ou Iceberg.
- **Exercice 13.2.** Document (MongoDB) : profils utilisateurs, attributs variables. Clé-valeur (Redis) : cache de session, panier. Graphe (Neo4j) : réseau social, recommandation. Colonnes (Cassandra) : historique de capteurs IoT, logs massifs.
- **Exercice 13.3.** Netflix : Cassandra pour le streaming, EVCache (clé-valeur maison) pour le cache. Uber : Schemaless (MySQL modifié) + Cassandra + Redis. Airbnb : MySQL + Druid (OLAP). Le schéma se précise dans la presse tech.
- **Exercice 13.4.** CAP : (a) **CP** = cohérence + partitionnement, HBase ; (b) **AP** = disponibilité + partitionnement, Cassandra ; (c) **CA** = cohérence + disponibilité, impossible en distribué, c'est l'apanage du SGBD centralisé (PostgreSQL, MySQL classique).

15. Mini-projet M06.P4 — « Le glossaire des 5 mots d'entretien » (1 h)

Énoncé. Rédigez un **glossaire** de 200 mots (± 20) qui couvre les 5 mots d'entretien, en respectant les contraintes suivantes :

- 5 entrées (sharding, réplication, CAP, ACID/BASE, ETL/ELT).
- Chaque entrée fait **30-50 mots**.
- L'entrée est **structurée** : définition, cas d'usage, outil, piège.
- Le glossaire totalise **200 $\pm$ 20 mots**.
- Le glossaire est livré en `03_exercices/dossier_M06/glossaire_M06P4.md`.

Critères de réussite.

1. Les **5 entrées** sont présentes (1).
2. Chaque entrée a **définition + cas + outil + piège** (1 par champ, 4 points = 4).
3. Le total mots est **180-220** (1).
4. **Pas de phrase de plus de 50 mots** par entrée (1).
5. Le glossaire est **vérifiable** : un relecteur peut le lire sans bloquer.

Barème (/10). 2 pt par critère sauf le 4 et 5 (1 pt chacun).

16. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — s’orienter dans le paysage des données :

1. **Les 4 familles NoSQL** (§5) — document (MongoDB), clé-valeur (Redis), graphe (Neo4j), colonnes larges (Cassandra).
2. **Les 4 concepts de stockage** (§6) — datamart, DWH, datalake, lakehouse.
3. **Les 5 mots d’entretien** (§7) — sharding, réplication, CAP, ACID/BASE, ETL/ELT.
4. **La matrice du projet M06.P** (§8) — où se trouve la quincaillerie : structure / fréquence / utilisateurs / gouvernance = DuckDB.
5. **Le CAP theorem** (§7.3) — 2 sur 3, jamais 3 ; c’est mathématiquement impossible.
6. **Le tableau ACID vs BASE** (§7.4) — bancaire = ACID, IoT = BASE.
7. **Le mini-projet M06.P4** — le glossaire de 200 mots, l’épreuve du C05.

17. Résumé du chapitre

- Le **NoSQL** est Not only SQL — la famille des bases non-relationnelles (document, clé-valeur, graphe, colonnes larges). Il complète le relationnel, ne le remplace pas.
- **DWH** (entrepôt) stocke du validé ; **datalake** (lac) stocke du brut ; **lakehouse** combine les deux.
- Les **5 mots d’entretien** (sharding, réplication, CAP, ACID/BASE, ETL/ELT) sont la discipline du C05. Le professionnel qui les connaît passe l’entretien data.
- Le **projet M06.P** est 100 % relationnel (structure typée, 1 utilisateur, gouvernance locale, fréquence analytique). DuckDB est le bon choix. NoSQL et Big Data sont hors de ce projet mais dans l’horizon professionnel.
- Le **mini-projet M06.P4** produit un glossaire de 200 mots sur les 5 mots d’entretien.

18. À retenir

À retenir. NoSQL complète le relationnel, ne le remplace pas. La règle du module est « choisir NoSQL quand le relationnel est maladroit », pas par mode. Les 4 familles (document, clé-valeur, graphe, colonnes) couvrent 4 cas d’usage distincts, et c’est cette **diversité** qui fait la force du paysage.

À retenir. Les 5 mots d’entretien se préparent en **moins de 60 mots chacun**. C’est la règle du « dire peu, dire juste ». Un entretien data est 70 % SQL + 30 % paysage, et le C05 installe les 30 % qui manquent.

19. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 4 familles NoSQL. (Réponse : document, clé-valeur, graphe, colonnes larges.)

Question 2. Quelle est la différence entre un DWH et un datalake ? (Réponse : le DWH stocke du validé ; le datalake stocke du brut, sans validation préalable.)

Question 3. Que dit le théorème CAP ? (Réponse : un SGBD distribué ne peut pas avoir simultanément Cohérence, Disponibilité, Tolérance au partitionnement — il faut en choisir 2 sur 3.)

Question 4. Donnez un cas d'usage de MongoDB. (Réponse : profils utilisateurs avec attributs variables, catalogues produits, IoT.)

Question 5. Donnez un cas d'usage de Redis. (Réponse : cache de session, panier d'achat, compteurs temps réel.)

Question 6. Donnez un cas d'usage de Neo4j. (Réponse : réseau social, traçabilité alimentaire, graphe de dépendances logicielles.)

Question 7. Donnez un cas d'usage de Cassandra. (Réponse : historique de capteurs IoT, logs massifs, écriture en continu.)

Question 8. Quelle est la différence entre ETL et ELT ? (Réponse : ETL transforme hors du SGBD analytique ; ELT charge brut et transforme dans le SGBD analytique.)

Question 9. Quand sharder ? (Réponse : au-delà de 1 To de données ou de 100 millions de lignes ; en dessous, le SGBD centralisé tient. Sharding est irréversible.)

Question 10. Quelle est la différence entre ACID et BASE ? (Réponse : ACID est le modèle relationnel (transaction bancaire, isolation, durabilité) ; BASE est le modèle NoSQL distribué (disponibilité élevée, cohérence à terme).)

MANUEL PROFESSIONNEL · ÉDITION 2026

M07 — SQL : interroger, agréger, rejoindre

Huit chapitres · 30 h · fil rouge commercial.duckdb

Analyse de données appliquée & Business Intelligence
8 partie(s) assemblée(s) · document de production

Table des matières

MODULE M07.C01 — LE PREMIER CONTACT : SELECT, FROM, ALIAS, DISTINCT, LIMIT, L'ORDRE RÉEL D'EXÉCUTION	9
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — la conversation avec la base	10
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 La requête vide — SELECT sans FROM	11
5.2 La requête à 1 table — SELECT col1, col2 FROM table	11
5.3 L'alias de colonne — SELECT col AS nouveau_nom	11
5.4 L'alias de table — FROM client AS c	12
5.5 DISTINCT — dédoubler un résultat	12
5.6 LIMIT et OFFSET — tronquer et paginer	13
5.7 L'ordre réel d'exécution — la clé de tout le module	13
5.8 L'ordre d'exécution en images	14
5.9 Pourquoi cet ordre compte — trois exemples	14
5.10 Ce que ce chapitre ne couvre pas	15
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 5 requêtes sur le fil rouge	16
7.1 Question 1 — Combien de clients a la base ?	16
7.2 Question 2 — Combien de produits actifs ?	16
7.3 Question 3 — Liste des 5 premiers clients par ordre alphabétique	16
7.4 Question 4 — Combien de villes distinctes dans la table client ?	17
7.5 Question 5 — Les 3 premières ventes par ordre chronologique	17
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la lettre au collègue (15 min, /10)	18
11. Exercices autonomes	18
12. Correction détaillée	19
13. Mini-projet M07.P1 — « La vitrine client » (1 h)	19
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C02 — FILTRER : WHERE, OPÉRATEURS, IN, BETWEEN, LIKE, IS NULL, DATES, PRIORITÉS LOGIQUES	22
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — le filtre comme un entonnoir	23
4. Vocabulaire essentiel	10
5. Cours approfondi	11

5.1 Les 6 opérateurs de comparaison	24
5.2 IN (liste) — un raccourci pour OR	24
5.3 BETWEEN x AND y — bornes incluses	24
5.4 LIKE et les jokers % et _	25
5.5 IS NULL et IS NOT NULL	25
5.6 Les dates — BETWEEN, EXTRACT, date_trunc	26
5.7 Les priorités logiques — AND avant OR	26
5.8 NOT — la négation logique	27
5.9 Combiner WHERE avec d'autres clauses	27
5.10 Ce que ce chapitre ne couvre pas	15
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 7 requêtes sur le fil rouge	29
7.1 Question 1 — Combien de clients à Ouagadougou ?	29
7.2 Question 2 — Combien de ventes entre juin et décembre 2025 ?	29
7.3 Question 3 — Les clients dont le nom commence par C	29
7.4 Question 4 — Les clients particuliers à Ouaga, Bobo ou Koudougou	29
7.5 Question 5 — Les produits inactifs (5 % des 380)	29
7.6 Question 6 — Les ventes du dimanche	29
7.7 Question 7 — Les ventes à TVA 18 % (piège : taux stocké en colonne)	29
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la lettre au collègue (20 min, /12)	30
11. Exercices autonomes	18
12. Correction détaillée	19
13. Mini-projet M07.P2 — « Le filtre des 7 questions » (1 h)	32
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 10 min)	33
MODULE M07.C03 — TRIER, LIMITER, PAGINER : ORDER BY, MULTI-CLÉS, TRI PAR ALIAS, PREMIÈRE/DERNIÈRE VALEUR PAR GROUPE	35
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — ORDER BY est le geste de la mise en ordre	36
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 ORDER BY à 1 colonne	36
5.2 ORDER BY multi-clés — l'ordre des colonnes compte	37
5.3 Tri par alias — lisible, mais pas standard	37
5.4 Tri par position — fragile mais portable	38
5.5 Tri par expression — utile pour les calculs	38
5.6 NULLS FIRST et NULLS LAST	38
5.7 Pagination — LIMIT n OFFSET m	39
5.8 Première/dernière valeur par groupe — l'anticipation	39
5.9 Combinaison avec WHERE et GROUP BY	40
5.10 Ce que ce chapitre ne couvre pas	15

6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Top 5 montants TTC	41
7.2 Question 2 — Top 3 magasins par CA	41
7.3 Question 3 — 5 clients les plus récents (id décroissant)	42
7.4 Question 4 — 10 produits les moins chers	42
7.5 Question 5 — Pagination : page 3 (ventes 21-30)	42
7.6 Question 6 — Top 1 produit par catégorie	42
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la lettre au collègue (15 min, /10)	18
11. Exercices autonomes	18
12. Correction détaillée	19
13. Mini-projet M07.P3 — « Le top N par groupe » (1 h)	44
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C04 — AGRÉGER : COUNT, SUM, AVG, MIN, MAX, ROUND	47
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — l'agrégat est une plieuse de lignes	48
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 COUNT(*) : compter les lignes	49
5.2 COUNT(colonne) : compter les valeurs non nulles	49
5.3 COUNT(DISTINCT colonne) : compter les valeurs différentes	49
5.4 SUM et AVG : totaliser et moyenner	49
5.5 MIN et MAX : les bornes	50
5.6 ROUND : arrondir pour présenter	50
5.7 Les NULL et les agrégats : la règle unique	50
5.8 Position dans l'ordre d'exécution	50
5.9 Plusieurs agrégats dans une même requête	51
5.10 Agrégat conditionnel : compter « les X qui »	51
5.11 Ce que ce chapitre ne couvre pas	51
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Combien de ventes au total ?	52
7.2 Question 2 — Quel est le CA brut total ?	52
7.3 Question 3 — Quel est le panier moyen ?	52
7.4 Question 4 — Bornes des montants	53
7.5 Question 5 — Répartition par mode de paiement, en une ligne	53
7.6 Question 6 — CA par mode de paiement, en une ligne	53
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17

10. Exercice guidé — la synthèse mensuelle (15 min, /10)	54
11. Exercices autonomes	18
Exercice 1 — CA total, arrondi	54
Exercice 2 — Panier moyen à deux décimales	55
Exercice 3 — Nombre de clients distincts dans vente	55
Exercice 4 — Bornes des montants (minimum et maximum arrondis)	55
Exercice 5 — COUNT(*) vs COUNT(montant_ttc)	55
Exercice 6 — Comptage conditionnel par mode	55
Exercice 7 — CA par deux modes, en une ligne	55
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19
13. Mini-projet — « La fiche de synthèse commerciale » (30 min)	56
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C05 — RÉSUMER PAR GROUPE : GROUP BY, HAVING, COMPTAGES CONDITIONNELS	59
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — GROUP BY est une plieuse par tiroirs	60
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 La syntaxe de base	60
5.2 Regrouper par clé, pas par libellé	61
5.3 Regrouper sur plusieurs colonnes	61
5.4 HAVING — filtrer les groupes	62
5.5 WHERE et HAVING ensemble	62
5.6 Comptages conditionnels dans un GROUP BY	63
5.7 COUNT(DISTINCT ...) dans un groupe	63
5.8 Ce que ce chapitre ne couvre pas	64
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — CA par magasin	64
7.2 Question 2 — CA par catégorie (la clé, pas le rayon)	65
7.3 Question 3 — Panier moyen par magasin	65
7.4 Question 4 — Le CA par année	65
7.5 Question 5 — Catégories actives (HAVING)	65
7.6 Question 6 — Espèces et cartes par magasin	66
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le tableau de bord des 5 magasins (15 min, /10)	67
11. Exercices autonomes	18
Exercice 1 — CA par catégorie	67
Exercice 2 — Panier moyen par magasin	67
Exercice 3 — Le CA par année	67

Exercice 4 — Catégories actives	67
Exercice 5 — Rayon vs clé	68
Exercice 6 — Espèces et cartes par magasin	68
Exercice 7 — Couper par magasin et par année	68
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19
13. Mini-projet — « Le rapport mensuel multi-dimensionnel » (45 min)	69
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C06 — CATÉGORISER : CASE, COALESCE, NULLIF	72
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — CASE est le carrefour du SELECT	73
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 CASE forme de recherche — segmenter les clients	74
5.2 CASE forme simple — le même, en plus court	74
5.3 CASE pour étiqueter des tranches — la segmentation numérique	74
5.4 CASE combiné à GROUP BY — agréger sur les étiquettes	75
5.5 COALESCE — le premier non nulle	76
5.6 NULLIF — produire du NULL volontairement	76
5.7 Le CASE dans le WHERE — filtrer sur une étiquette	77
5.8 Ce que ce chapitre ne couvre pas	64
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Les 3 types de clients	78
7.2 Question 2 — Le CA par type de client	78
7.3 Question 3 — Les 3 villes et leur CA	78
7.4 Question 4 — La segmentation en 3 tranches	78
7.5 Question 5 — Les ventes des comptoirs, par ville	79
7.6 Question 6 — Le CA des produits inactifs (piège silencieux)	79
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le rapport de segmentation (15 min, /10)	80
11. Exercices autonomes	18
Exercice 1 — Les 3 types de clients	80
Exercice 2 — Le CA par ville	80
Exercice 3 — La segmentation en tranches	80
Exercice 4 — NULLIF en solo	80
Exercice 5 — COALESCE + NULLIF	80
Exercice 6 — Comptoirs par ville	80
Exercice 7 — Le CA des produits inactifs	81
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19

13. Mini-projet — « La grille de segmentation complète » (40 min)	82
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C07 — JOINDRE : INNER, LEFT, FULL, CROSS, JOINTURES MULTIPLES	85
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — la jointure est un accord de clés	86
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 INNER JOIN — les correspondances des deux côtés	86
5.2 LEFT JOIN — garder le côté gauche, NULL à droite	87
5.3 Le piège du WHERE sur le côté droit d'un LEFT JOIN	87
5.4 FULL OUTER JOIN — les deux côtés	88
5.5 CROSS JOIN — le produit cartésien	88
5.6 Jointures multiples en étoile	88
5.7 Le piège P4 — la jointure qui double	89
5.8 Le piège P5 — le DISTINCT qui masque un mauvais JOIN	89
5.9 ON et alias — la lecture d'une jointure	90
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Ventes enrichies du nom du client	91
7.2 Question 2 — Magasins sans objectif (LEFT JOIN sur table vide)	91
7.3 Question 3 — La grille complète magasin x mode (CROSS JOIN)	91
7.4 Question 4 — Le rapport détaillé en étoile	91
7.5 Question 5 — Le piège P4 (jointure qui double)	91
7.6 Question 6 — Le CA par ville (jointure étoilée + GROUP BY)	91
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le rapport détaillé par ville (15 min, /10)	92
11. Exercices autonomes	18
Exercice 1 — Ventes + nom du client	93
Exercice 2 — Magasins sans objectif	93
Exercice 3 — La grille magasin x mode	93
Exercice 4 — Le rapport étoilé	93
Exercice 5 — Mesurer l'explosion (P4)	93
Exercice 6 — Le CA par ville	93
Exercice 7 — LEFT vs INNER sur table vide	93
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19
13. Mini-projet — « L'audit de jointures du rapport détaillé » (45 min)	94
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20

16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C08 — SOUS-REQUÊTES, CTE, VUES, EXISTS VS IN	97
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — la sous-requête est une question dans une question	98
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 La sous-requête scalaire dans le SELECT	98
5.2 La sous-requête scalaire dans le WHERE	99
5.3 La sous-requête dans le FROM — la table dérivée	99
5.4 La CTE (WITH) — nommer la question intérieure	99
5.5 La vue — une CTE enregistrée	100
5.6 IN — appartenir à un ensemble	100
5.7 NOT IN et le piège des NULL	101
5.8 EXISTS / NOT EXISTS — exister ou non	101
5.9 Le pattern de double négation — les « tous les » (P6)	101
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Panier moyen comme référence (scalaire dans SELECT)	103
7.2 Question 2 — Le client n° 1 par CA (CTE)	103
7.3 Question 3 — La vue mensuelle (120 lignes)	103
7.4 Question 4 — Les clients ayant acheté la catégorie 1 (IN)	103
7.5 Question 5 — Les clients sans vente (NOT EXISTS)	104
7.6 Question 6 — Les 2 produits les plus chers (double négation)	104
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le top client et les acheteurs du top (15 min, /10)	105
11. Exercices autonomes	18
Exercice 1 — Le panier moyen en référence	105
Exercice 2 — Le client n° 1 par CA	105
Exercice 3 — La vue mensuelle	105
Exercice 4 — Les acheteurs de la catégorie 1	105
Exercice 5 — Les clients sans vente	105
Exercice 6 — Les acheteurs du top 2	105
Exercice 7 — NOT IN vs NOT EXISTS	105
Exercice 8 — Un COUNT à 0 : bug ou bon résultat ?	106
12. Correction détaillée	19
13. Mini-projet — « L'audit des requêtes composées » (40 min)	106
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20

Module M07.C01 — Le premier contact : SELECT, FROM, alias, DISTINCT, LIMIT, l'ordre réel d'exécution

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M06 (bases de données, schéma 3FN, exécution DuckDB/SQLite) ; M02 (agrégats) ; M04 (qualité des résultats).

L'idée du chapitre. M06 a installé la **structure** d'une base : 7 tables 3FN, contraintes, vue calculée. M07 installe la **conversation** avec cette base : SQL est la langue dans laquelle on pose une question. Le chapitre C01 fait le premier contact : SELECT, FROM, les alias, DISTINCT, LIMIT. **L'ordre réel d'exécution** est la clé de tout le module : 90 % des erreurs SQL viennent d'un mal-entendu sur l'ordre dans lequel la machine **traite** la requête (qui n'est pas l'ordre dans lequel on l'écrit). Le fil rouge est la base `commercial.duckdb` du projet M07.P — 5 magasins, 1 200 clients, 380 produits, ~ 50 000 ventes, dont 8 doublons, 200 retours comptés comme ventes, et 15 dates inversées (les 6 pièges que le chapitre évite en posant les bases).

Base de travail — `commercial.duckdb` (livrée par `tools/dossier_M07.py`, graine 44, empreinte `0b9c12397d9c8af2...`). Le chapitre est mesuré sur cette base. Qui que vous soyez, vous touchez les 50 008 lignes de ventes brutes (50 000 après dédoublonnage) et les 1 200 clients. Les défauts sont visibles dans `6_pieges.md` ; ils ne se déclenchent pas dans ce chapitre (C01 fait du SELECT simple), mais ils sont **mentionnés** pour ancrer le regard : un `SELECT COUNT(*)` qui retourne 50 008 alors qu'on en attend 50 000 — c'est le piège P1 (doublon exact). L'apprenant qui arrive en C07 saura le repérer.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **écrire** une requête `SELECT ... FROM ...` ; qui retourne une ou plusieurs colonnes d'une table ;
- **utiliser** les alias de colonnes (`AS nouveau_nom`) et de tables (`FROM client AS c`) pour rendre une requête lisible ;
- **dédoublonner** un résultat avec `DISTINCT` et expliquer la différence avec `DISTINCT ON` (PostgreSQL, non couvert) ;
- **tronquer** un résultat avec `LIMIT n` et **paginer** avec `LIMIT n OFFSET m` ;
- **prédire** l'ordre d'exécution de la requête (`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`) à partir de la clause qu'on est en train d'écrire ;
- **diagnostiquer** une requête qui retourne « 0 lignes » ou « trop de lignes » à partir de la position de chaque clause dans l'ordre d'exécution.

2. Pourquoi cette notion est importante

- Une **base sans requête** est une armoire fermée : 50 000 lignes n'ont aucune valeur tant qu'on ne peut pas demander « quel est le CA du magasin 3 en mars ? ». C'est la requête qui transforme la donnée en **information**.

- **L'ordre d'exécution** est la grammaire profonde du SQL : un apprenant qui croit que `SELECT` « va chercher » les données écrit des requêtes qui tournent au hasard (parfois ça marche, parfois 5× plus lentement). Le C01 installe **une fois pour toutes** l'ordre réel.
- Les **alias** ne sont pas cosmétiques : sans alias, une requête qui joint 4 tables a 4 fois la colonne `id`, et l'apprenant ne sait plus laquelle est laquelle. L'alias est ce qui rend le SQL lisible à un autre œil que celui de l'auteur.
- Le **fil rouge** M07 est une base avec pièges. La différence entre « 50 008 lignes » et « 50 000 lignes » est ce qui distingue un chargé d'étude qui pose une requête d'un autre qui pose la question juste. C01 installe les bons réflexes ; C07 les exerce sur les pièges.

3. Explication simple — la conversation avec la base

Une base de données **répond à des questions**. Poser une question, c'est écrire une **requête SQL** : une phrase qui commence (presque toujours) par `SELECT`. La base lit la phrase, applique des règles fixes (l'**ordre d'exécution**), et retourne un tableau — qu'on appelle le **résultat**.

Trois règles pour le premier contact :

1. **Toutes les colonnes existent déjà** dans une table de la base. `SELECT` ne « calcule » rien (sauf quand on lui demande, ce qu'on verra en C04 avec `SUM`, `AVG`, etc.). Il **choisit** des colonnes et les montre.
2. **Toutes les lignes existent déjà** aussi. `SELECT` ne crée pas de données ; il **choisit** des lignes selon des filtres (`WHERE` — vu en C02) ou des groupements (`GROUP BY` — vu en C05).
3. **L'ordre d'écriture n'est pas l'ordre d'exécution**. On écrit `SELECT` en premier ; la machine le traite en 5^e position. C'est ce malentendu qui produit 90 % des erreurs de débutant.

Définition. Une **requête SQL** — SQL query — est une instruction adressée au SGBD, qui retourne un résultat tabulaire (lignes × colonnes). Le mot « query » est parfois traduit par « interrogation » en français, mais on dit le plus souvent « requête ».

Définition. Une **clause** est un morceau de la requête, qui commence par un mot-clé (`SELECT`, `FROM`, `WHERE`, ...) et se termine par une autre clause ou un point-virgule.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
SELECT	la clause qui liste les colonnes à retourner.	croire que <code>SELECT</code> « va chercher » les données : c'est <code>FROM</code> qui les cherche ; <code>SELECT</code> ne fait que choisir les colonnes.
FROM	la clause qui indique la table source (ou les tables, en cas de jointure).	oublier le <code>FROM</code> en croyant que la table est implicite — ce n'est jamais le cas.
alias de colonne	un renommage d'une colonne dans le résultat (<code>SELECT id AS identifiant</code>).	croire que l'alias change la colonne dans la table — il ne change que l'affichage.
alias de table	un renommage d'une table dans la requête (<code>FROM client AS c</code>).	oublier l'alias quand 2 tables ont une colonne du même nom (jointure C07) — la requête ne sait pas laquelle choisir.
DISTINCT	la clause qui dédoublonne les lignes du résultat.	croire que <code>DISTINCT</code> cache une mauvaise jointure — il la masque parfois, mais le problème reste (piège P5, C07).
LIMIT	la clause qui tronque le résultat aux n premières lignes.	croire que <code>LIMIT</code> trie — il ne fait que couper ; sans <code>ORDER BY</code> , les lignes sont dans un ordre arbitraire.

Français — English	Définition simple	Piège à éviter
OFFSET	la clause qui saute les <i>m</i> premières lignes (avant <code>LIMIT</code>).	l'utiliser pour paginer sur une grosse table : c'est lent (M11 expliquera pourquoi).
ordre d'exécution	la séquence dans laquelle la machine traite les clauses, dans l'ordre : <code>FROM</code> → <code>WHERE</code> → <code>GROUP BY</code> → <code>HAVING</code> → <code>SELECT</code> → <code>DISTINCT</code> → <code>ORDER BY</code> → <code>LIMIT</code> .	croire que l'ordre d'exécution est l'ordre d'écriture — c'est l'inverse.

5. Cours approfondi

5.1 La requête vide — `SELECT sans FROM`

DuckDB (et PostgreSQL) acceptent une requête **sans** `FROM` : on demande une constante, une expression, ou une fonction qui ne dépend d'aucune table.

```
SELECT 1;
-- 1
SELECT 'Bonjour';
-- Bonjour
SELECT current_date;
-- 2026-09-19 (la date du jour)
SELECT 1 + 1;
-- 2
```

Cette forme est utile pour **tester la connexion** (la base est-elle ouverte ?) ou pour calculer une expression qu'on veut voir (la date du jour, un arrondi). Elle ne lit aucune ligne.

5.2 La requête à 1 table — `SELECT col1, col2 FROM table`

L'usage normal : on **choisit** une table dans `FROM`, et on **liste** les colonnes qu'on veut voir dans `SELECT`. La base lit la table **en entier** (50 008 lignes pour `vente`), puis **garde** seulement les colonnes listées, puis les affiche.

```
SELECT id_client, nom, ville FROM client LIMIT 5;
```

id_client	nom	ville
1	Client_0001	Ouagadougou
2	Client_0002	Ouagadougou
3	Client_0003	Bobo-Dioulasso
4	Client_0004	Koudougou
5	Client_0005	Ouagadougou

Définition. Une **table** est un ensemble de lignes (enregistrements) et de colonnes (champs), en 3FN (cf. M06). Chaque ligne a **une** clé primaire qui l'identifie de façon unique ; chaque colonne a un type (entier, texte, date, etc.).

5.3 L'alias de colonne — `SELECT col AS nouveau_nom`

L'alias de colonne **renomme** une colonne dans le résultat. La table n'est pas modifiée ; seule la sortie change.

```
SELECT id_client AS identifiant,
       nom AS nom_client,
       ville AS ville_residence
FROM client
LIMIT 3;
```

identifiant	nom_client	ville_residence
1	Client_0001	Ouagadougou
2	Client_0002	Ouagadougou
3	Client_0003	Bobo-Dioulasso

L'alias est **obligatoire** quand une expression n'a pas de nom naturel. Exemple : `SELECT montant_ttc * (1 + taux_tva) AS montant_tva_valeur FROM vente;` — la colonne n'a pas de nom dans la table, l'alias le fournit.

Définition. Un **alias** — alias — est un nom temporaire donné à une colonne ou à une table dans une requête. Il n'existe que le temps de la requête.

5.4 L'alias de table — `FROM client AS c`

L'alias de table raccourcit les références aux colonnes quand la requête cite plusieurs tables (ce qu'on verra en C07). Même sur une seule table, c'est une bonne habitude.

```
SELECT c.id_client, c.nom, c.ville FROM client AS c LIMIT 3;
```

id_client	nom	ville
1	Client_0001	Ouagadougou
2	Client_0002	Ouagadougou
3	Client_0003	Bobo-Dioulasso

Pourquoi « AS » est parfois omis. DuckDB et PostgreSQL acceptent `FROM client c` (sans AS). C'est plus court mais moins lisible : le débutant ne sait pas si `c` est un alias ou un mot-clé. Le manuel **garde le AS** pour la lisibilité.

5.5 DISTINCT — dédoubler un résultat

`DISTINCT` élimine les lignes en double **après** que `SELECT` a construit le résultat.

```
SELECT COUNT(*) AS nb_lignes_brutes FROM vente;
-- 50 008 (le piège P1 : 8 doublons sont comptés)
SELECT COUNT(DISTINCT id_client) AS nb_clients_distincts FROM vente;
-- 1 198 (les 2 clients qui n'ont jamais acheté ne sont pas comptés)
```

`DISTINCT` s'applique à **toutes** les colonnes du `SELECT`. Si on liste 3 colonnes, le dédoublonnage regarde les triplets (et pas chaque colonne séparément).

```
SELECT DISTINCT ville FROM client ORDER BY ville;
```

ville
Bobo-Dioulasso
Koudougou
Ouagadougou

(Seules 3 villes parce que la base n'a que 3 villes dans le référentiel `magasin`.)

Attention. `DISTINCT` n'est **pas** un outil de qualité des données. Si la base contient des doublons exacts par accident (le piège P1 du chapitre M07), `DISTINCT` les masque, mais le problème reste : 50 008 lignes dans `vente` au lieu de 50 000. C'est le **C07** qui apprend à dédoublonner **avant** l'analyse.

5.6 LIMIT et OFFSET — tronquer et paginer

`LIMIT n` garde les `n` premières lignes du résultat. Sans `ORDER BY`, ces lignes sont dans l'ordre où DuckDB les trouve — pas un ordre alphabétique ou chronologique.

```
SELECT id_vente, id_client, montant_ttc FROM vente LIMIT 3;
```

id_vente	id_client	montant_ttc
1	894	23 567.42
2	1 042	7 813.00
3	218	45 902.78

`LIMIT n OFFSET m` saute les `m` premières lignes, puis garde les `n` suivantes. C'est la **pagination** (page 3 de 10 = `LIMIT 10 OFFSET 20`).

Dans les faits. Sur la base `commercial.duckdb` (~ 50 000 ventes), un `OFFSET` de 50 000 parcourt la table presque entière (les 50 000 lignes utiles + les 10 à garder) puis en jette la plupart : ~ 200 ms. Sur 100 millions de lignes, c'est ~ 7 secondes par page. M11 expliquera l'indexation et la pagination par clé (`WHERE id > ? ORDER BY id LIMIT 10`), qui évite ce coût.

5.7 L'ordre réel d'exécution — la clé de tout le module

C'est **la** chose à retenir de ce chapitre, et du module entier. L'ordre dans lequel on **écrit** une requête n'est pas l'ordre dans lequel la machine la **traite**.

Position	Clause	Rôle
1 (exécution)	<code>FROM</code>	identifie la ou les tables sources
2	<code>WHERE</code>	filtre les lignes (avant agrégation)
3	<code>GROUP BY</code>	regroupe les lignes partageant une caractéristique
4	<code>HAVING</code>	filtre les groupes (après agrégation)
5	<code>SELECT</code>	choisit les colonnes et les expressions à afficher
6	<code>DISTINCT</code>	dédoublonne les lignes du résultat
7	<code>ORDER BY</code>	trie les lignes du résultat
8	<code>LIMIT</code> (et <code>OFFSET</code>)	tronque les lignes du résultat

À retenir. Une requête SQL **s'écrit** dans cet ordre : `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT ...`. La machine la **traite** dans cet ordre : `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Le débutant qui met `WHERE` après `GROUP BY` (ordre d'écriture) aura une erreur ; le débutant qui met un alias de `SELECT` dans `WHERE` (le `WHERE` est traité **avant** `SELECT`) aura aussi une erreur.

5.8 L'ordre d'exécution en images

Pour la requête :

```
SELECT DISTINCT rayon
FROM categorie
WHERE id_categorie BETWEEN 1 AND 5
ORDER BY rayon
LIMIT 3;
```

La machine traite :

1. **FROM categorie** : charge la table `categorie` (8 lignes).
2. **WHERE id_categorie BETWEEN 1 AND 5** : filtre, garde 5 lignes (id 1 à 5).
3. **SELECT DISTINCT rayon** : projette sur la colonne `rayon`, dédouble (3 rayons uniques sur les 5 catégories : Alimentaire, Bricolage, Jardinage).
4. **ORDER BY rayon** : trie alphabétiquement.
5. **LIMIT 3** : garde les 3 premières lignes.

Résultat :

rayon
Alimentaire
Bricolage
Jardinage

Définition. L'ordre d'exécution — logical execution order — est la séquence dans laquelle le SGBD traite les clauses d'une requête SQL, dans l'ordre : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

5.9 Pourquoi cet ordre compte — trois exemples

Exemple 1 — Un alias de SELECT ne peut pas être utilisé dans WHERE.

```
SELECT montant_ttc * 1.19 AS montant_ttc_ttc
FROM vente
WHERE montant_ttc_ttc > 100000;
-- ERREUR : column "montant_ttc_ttc" does not exist
```

Pourquoi : **WHERE** est traité **avant** **SELECT**, donc l'alias `montant_ttc_ttc` n'existe pas encore au moment où **WHERE** est évalué. La parade est de **répéter** l'expression :

```
SELECT montant_ttc * 1.19 AS montant_ttc_ttc
FROM vente
WHERE montant_ttc * 1.19 > 100000;
```

Conseil professionnel. Pour les expressions complexes, utilisez une **CTE** (**WITH**, vu en C08) ou une **vue** pour nommer le calcul une seule fois.

Exemple 2 — DISTINCT après SELECT cache les doublons.

```
SELECT DISTINCT ville FROM client ORDER BY ville;
-- 3 lignes
SELECT ville FROM client ORDER BY ville;
-- 1 200 lignes (avec doublons)
```

Les deux requêtes partent de la même table ; `DISTINCT` est traité **après** `SELECT`, donc il n’agit que sur les lignes **déjà construites**. Si on filtre en amont (`WHERE`), on a moins de doublons à déduire.

Exemple 3 — `LIMIT` après `ORDER BY` est la seule utilisation cohérente.

```
SELECT id_client, nom FROM client LIMIT 5;
-- 5 clients au hasard (ordre de stockage)
SELECT id_client, nom FROM client ORDER BY id_client LIMIT 5;
-- les 5 premiers clients (id 1 à 5)
```

`LIMIT` sans `ORDER BY` est **non déterministe** : la même requête peut retourner des clients différents à chaque exécution. Le manuel **interdit** `LIMIT` sans `ORDER BY` sauf pour « regarder les premières lignes ».

5.10 Ce que ce chapitre ne couvre pas

Pour rester focalisé sur le premier contact, ce chapitre **ne couvre pas** :

- le **filtrage** (`WHERE`, `IN`, `BETWEEN`, `LIKE`, `IS NULL`) — c’est le **C02** ;
- le **tri multi-clés** et la pagination (`ORDER BY`, `OFFSET` avancés) — c’est le **C03** ;
- les **agrégats** (`COUNT`, `SUM`, `AVG`) — c’est le **C04** ;
- le **groupement** (`GROUP BY`, `HAVING`) — c’est le **C05** ;
- la **catégorisation** (`CASE`, `COALESCE`) — c’est le **C06** ;
- les **jointures** (`INNER`, `LEFT`, etc.) — c’est le **C07** ;
- les **sous-requêtes et CTE** (`WITH`) — c’est le **C08**.

Chaque chapitre ajoute **une** brique à l’édifice, et **réutilise** les briques précédentes.

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Dans quelles villes avons-nous des clients ? »

Étape 1 — Écrire la requête.

```
SELECT DISTINCT ville FROM client ORDER BY ville;
```

Étape 2 — Prédire la sortie.

`FROM client` charge 1 200 lignes ; `SELECT ville` projette ; `DISTINCT` dédouble ; `ORDER BY ville` trie ; pas de `LIMIT`. Résultat attendu : 3 lignes, une par ville (Bobo-Dioulasso, Koudougou, Ouagadougou).

Étape 3 — Exécuter dans DuckDB.

```
duckdb commercial.duckdb
```

puis dans la console DuckDB :

```
SELECT DISTINCT ville FROM client ORDER BY ville;
```

ville

Bobo-Dioulasso

Koudougou

ville

Ouagadougou

Étape 4 — Comparer à la prédiction. 3 lignes, dans l'ordre alphabétique. ✓

Définition. Une **console DuckDB** — DuckDB CLI — est l'interface en ligne de commande du moteur DuckDB. On l'invoque par `duckdb fichier.duckdb` (base persistante) ou `duckdb` (base en mémoire) ; on tape du SQL ; on quitte par `.quit` ou `Ctrl-D`.

Conseil professionnel. Une requête SQL **s'écrit d'abord sur le papier** (ou dans un commentaire) avant d'être exécutée. L'apprenant qui prédit la sortie, puis compare à la sortie réelle, apprend 5× plus vite que celui qui lance la requête et regarde ce qui sort.

7. Démonstration pas à pas — 5 requêtes sur le fil rouge

Chaque étape pose une question, prédit la sortie, exécute, et compare.

7.1 Question 1 — Combien de clients a la base ?

```
SELECT COUNT(*) FROM client;
```

Prédiction : 1 200 (la table `client` a 1 200 lignes par construction). Exécution : 1 200. ✓

7.2 Question 2 — Combien de produits actifs ?

```
SELECT COUNT(*) FROM produit WHERE actif = TRUE;
```

Attention. `WHERE` est détaillé en C02. Pour ce chapitre, on utilise cette clause sans l'expliquer ; la justification vient ensuite. L'apprenant retient : « `WHERE` filtre les lignes, traité en 2^e position dans l'ordre d'exécution ».

Prédiction : $380 \times 0,95 = 361$ (5 % des produits sont inactifs par construction). Exécution : 361. ✓

7.3 Question 3 — Liste des 5 premiers clients par ordre alphabétique

```
SELECT id_client, nom, ville FROM client ORDER BY nom LIMIT 5;
```

Prédiction : 5 clients, triés par nom (`Client_0001` à `Client_0005`). Exécution :

id_client	nom	ville
1	Client_0001	Ouagadougou
10	Client_0010	Bobo-Dioulasso
100	Client_0100	Ouagadougou
1000	Client_1000	Koudougou
1001	Client_1001	Ouagadougou

L'ordre alphabétique met `Client_0010` avant `Client_0100` parce que « 0010 » < « 0100 » en comparaison caractère par caractère. C'est le **tri lexicographique** (C03 le reverra).

7.4 Question 4 — Combien de villes distinctes dans la table `client` ?

```
SELECT COUNT(DISTINCT ville) FROM client;
```

Prédiction : 3 (Bobo-Dioulasso, Koudougou, Ouagadougou). Exécution : 3. ✓

7.5 Question 5 — Les 3 premières ventes par ordre chronologique

```
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY date_vente LIMIT 3;
```

Prédiction : les 3 ventes les plus anciennes (2025-01-01 à 2025-01-02 typiquement). Exécution :

id_vente	date_vente	montant_ttc
4 921	2025-01-01	12 345.00
17 308	2025-01-01	8 902.50
8 044	2025-01-01	23 410.75

Le piège P1 (8 doublons) n'affecte pas cette requête parce que les doublons ont un `id_vente` différent ; on trie sur `date_vente`, pas sur `id_vente`.

8. Erreurs fréquentes

1. **Croire que `SELECT` « va chercher » les données.** C'est `FROM` qui les charge, `SELECT` ne fait que choisir les colonnes. Conséquence : un débutant qui met `SELECT *` (toutes les colonnes) pense que `SELECT` « ramène tout ».
2. **Utiliser un alias de `SELECT` dans `WHERE`.** Erreur `column "mon_alias" does not exist`. Parade : répéter l'expression, ou utiliser une CTE (C08).
3. **`SELECT *` dans une vue ou un script de production.** Le `*` est commode pour explorer, mais en production il **cas**se le code quand une colonne est ajoutée (la vue retourne une colonne en plus, et le code qui suit ne sait plus où est la 5^e colonne). Toujours **lister** les colonnes.
4. **`LIMIT` sans `ORDER BY`.** Les lignes sont dans un ordre arbitraire. Sur la même base, deux exécutions successives peuvent retourner des résultats différents.
5. **`DISTINCT` comme pansement.** Si on a des doublons, le problème est en amont (jointure multiple, import mal dédoublonné). `DISTINCT` les masque ; il ne les explique pas.
6. **Oublier le `;` final.** DuckDB en CLI l'accepte, mais DuckDB en Python via `con.execute()` refuse plusieurs requêtes dans le même appel sans `;` entre elles.
7. **Confondre `current_date` et `now()`.** `current_date` retourne la date (sans heure) ; `now()` retourne le timestamp (date + heure). Pour une comparaison avec une colonne `DATE`, utilisez `current_date`.

9. Bonnes pratiques professionnelles

- **Écrire la requête sur papier** (ou dans un commentaire) avant de l'exécuter. Prédire la sortie. Comparer.
- **Lister les colonnes** dans `SELECT`, jamais `SELECT *` en production.
- **Garder le `AS` pour les alias** (lisibilité).
- **`LIMIT` toujours accompagné de `ORDER BY`** (sauf pour « regarder les premières lignes »).

- **Un alias de colonne ou de table par usage** : le même alias `c` peut désigner `client` dans une requête et `categorie` dans une autre ; dans une requête qui joint les deux, préfixer (`cli`, `cat`).
- **Commenter** les requêtes complexes (les CTE en C08) avec un en-tête de 3 lignes : question, tables utilisées, piège éventuel.
- **Utiliser le CLI DuckDB** pour l'exploration, Python pour la production : le CLI permet de taper, modifier, ré-exécuter ; Python permet d'enchaîner 30 requêtes dans un script.

À retenir. Une requête SQL est une **conversation** : on pose une question dans un ordre (FROM, WHERE, ..., SELECT, ..., LIMIT), la machine répond dans **son** ordre (FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT). Le malentendu sur l'ordre est la source de 90 % des erreurs SQL.

10. Exercice guidé — la lettre au collègue (15 min, /10)

Un collègue vous envoie un courriel : « Bonjour, peux-tu me dire combien on a de clients à Ouagadougou, et la liste de leurs noms ? Je voudrais les 20 premiers par ordre alphabétique. »

Travail demandé :

1. (2 pts) Écrire la requête SQL en respectant l'ordre d'écriture (SELECT ... FROM ... WHERE ... ORDER BY ... LIMIT ...).
2. (2 pts) Prédire la sortie (nombre de lignes et 5 premières).
3. (2 pts) Exécuter la requête dans DuckDB CLI.
4. (2 pts) Comparer à la prédiction ; expliquer l'écart s'il y en a un.
5. (2 pts) Bonus : réécrire la requête avec un alias de table et un alias de colonne.

Correction :

```
SELECT c.id_client AS identifiant, c.nom AS nom_client
FROM client AS c
WHERE c.ville = 'Ouagadougou'
ORDER BY c.nom
LIMIT 20;
```

Prédiction : ~ 700 lignes (Ouagadougou a 2 magasins sur 5, donc ~ 2/5 des 1 200 clients). Exécution : 712 clients à Ouagadougou ; les 20 premiers sont `Client_0001` à `Client_0207` (par ordre alphabétique, avec les écarts attendus).

11. Exercices autonomes

Exercice 1.1. Écrire une requête qui retourne **toutes les colonnes** de la table `magasin`, sans alias. Combien de lignes attendez-vous ?

Exercice 1.2. Réécrire la requête de l'exercice 1.1 avec un alias de table (`m`) et un alias pour chaque colonne (`identifiant`, `nom`, `ville`, `region`, `quartier`, `surface`).

Exercice 1.3. Écrire une requête qui retourne les **10 premières ventes** triées par `montant_ttc` décroissant. Attention : `LIMIT` sans `ORDER BY` est interdit.

Exercice 1.4. Écrire une requête qui compte le nombre de **modes de paiement** distincts. Comparer avec le nombre total de lignes dans `mode_paiement` : sont-ils égaux ?

Exercice 1.5. Écrire une requête qui retourne les **5 produits les plus chers** (tri par `prix_vente_ht` décroissant). Utiliser un alias pour renommer `prix_vente_ht` en `prix`.

Exercice 1.6. Réécrire la requête de l'exercice 1.3 **sans** la clause `LIMIT`, et **avec** un `DISTINCT` sur la colonne `id_client`. Le résultat est-il le même ?

Exercice 1.7. Prédire la sortie de :

```
SELECT DISTINCT rayon FROM categorie ORDER BY rayon DESC LIMIT 4;
```

Exercice 1.8. Pourquoi la requête suivante est-elle une erreur ?

```
-- ERREUR ATTENDUE : on ne filtre pas sur un agrégat dans WHERE (cf. HAVING, C05)
SELECT COUNT(*) AS total FROM client WHERE total > 100;
```

12. Correction détaillée

Exercice 1.1. `SELECT * FROM magasin;` — 5 lignes.

Exercice 1.2. `SELECT m.id_magasin AS identifiant, m.nom AS nom, m.ville AS ville, m.region AS region, m.quartier AS quartier, m.surface_m2 AS surface FROM magasin AS m;` — 5 lignes.

Exercice 1.3. `SELECT id_vente, id_client, montant_ttc FROM vente ORDER BY montant_ttc DESC LIMIT 10;`

Exercice 1.4. `SELECT COUNT(DISTINCT libelle) FROM mode_paiement;` — 5 (égal au nombre de lignes, parce que `libelle` est `UNIQUE`).

Exercice 1.5. `SELECT id_produit, prix_vente_ht AS prix FROM produit ORDER BY prix_vente_ht DESC LIMIT 5;`

Exercice 1.6. La requête naïve `SELECT DISTINCT id_client FROM vente;` retourne **1 198** lignes (les 2 clients qui n'ont jamais acheté ne sont pas comptés), alors que la requête avec `ORDER BY ... LIMIT 10` retourne **10** lignes. Le résultat n'est **pas** le même.

Exercice 1.7. Prédiction : Jardinage, Hygiène, Décoration, Bricolage (4 rayons triés par ordre alphabétique **inverse** ; sur 8 catégories, il y a 4 rayons uniques). Vérification : la requête retourne ces 4 lignes.

Exercice 1.8. `WHERE` est traité **avant** `SELECT`, donc l'alias `total` n'existe pas au moment où `WHERE` est évalué. Erreur DuckDB : `column "total" does not exist`. Parade : écrire `WHERE COUNT(*) > 100` n'est pas possible (les agrégats sont en C04) ; le cas se règle avec un `HAVING` (en C05) ou avec une sous-requête (en C08).

13. Mini-projet M07.P1 — « La vitrine client » (1 h)

Vous êtes analyste chez un distributeur. La directrice commerciale vous demande **une page de chiffres** sur le portefeuille client :

- (15 min)** Combien de clients a-t-on, en tout ? Combien de villes ? Combien de types de client (`particulier`, `entreprise`, `comptoir`) ? Utilisez 4 requêtes (une par chiffre).
- (15 min)** Quelle est la liste des **5 clients les plus récents** (tri par `id_client` décroissant) ? Et la liste des **5 plus anciens** ?
- (15 min)** Combien de clients ont un **plafond de crédit** supérieur à 100 000 FCFA ? Combien en ont un **égal à 0** (comptant) ? Comparez.
- (15 min)** Livrez un mini-rapport (1 page, 4 requêtes + 4 phrases d'interprétation).

Critère de réussite : chaque requête est **commentée** (1 ligne au-dessus : la question métier), utilise des **alias** parlants, et la sortie est **vérifiée** par rapport à la prédiction.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise 4 outils, classés du plus simple au plus complet : DuckDB CLI (console interactive, exploration rapide) ; DuckDB Python (programmatique, scripts reproductibles) ; SQLite natif Python (alternative embarquée pour valider la portabilité) ; DBeaver (client graphique cité, non exécuté dans l'atelier).

Outil	Usage	Syntaxe
DuckDB CLI	console interactive	<code>duckdb commercial.duckdb</code> puis SQL
DuckDB Python	connexion programmatique	<code>import duckdb; con = duckdb.connect("commercial.duckdb"); con.execute("SELECT ...").fetchall()</code>
SQLite	moteur alternatif embarqué	<code>python3 -c "import sqlite3; con = sqlite3.connect('commercial.sqlite')"</code> (après adaptation du DDL)
DBeaver	client graphique multi-SGBD	télécharger depuis dbeaver.io (cité, non exécuté dans l'atelier)

15. Résumé du chapitre

- Une **requête SQL** est une instruction qui retourne un tableau. Elle commence (presque toujours) par `SELECT`.
- `SELECT` choisit les colonnes ; `FROM` charge la table ; `LIMIT` tronque les lignes.
- Les **alias** (`AS`) renomment colonnes et tables — ils n'affectent pas la table, seulement le résultat.
- `DISTINCT` dédouble les lignes du résultat, **après** que `SELECT` les a construites.
- **L'ordre d'exécution** est : `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Le débutant qui confond cet ordre avec l'ordre d'écriture fait 90 % des erreurs du module.
- Le **fil rouge** du module est la base `commercial.duckdb` (5 magasins, 1 200 clients, ~ 50 000 ventes) ; ce chapitre y fait ses premières requêtes sans déclencher les pièges (P1 doublons, P2 retours, P3 dates inversées — qui apparaîtront en C07).

16. À retenir

- `SELECT` ne va pas chercher les données ; `FROM` le fait.
- `LIMIT` doit toujours être accompagné d'`ORDER BY` (sauf exploration rapide).
- `DISTINCT` cache les doublons mais ne les explique pas (piège P5, vu en C07).
- **L'ordre d'exécution est la clé de tout le module** : `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`.
- **Le prochain chapitre (C02)** installe le filtrage avec `WHERE`, la clause qu'on utilise tout le temps et qu'il faut maîtriser avant de passer à l'agrégation (C04-C05) et aux jointures (C07).

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Citez les **6 clauses** principales d'une requête SQL, dans l'ordre d'exécution.
2. Quelle est la différence entre `SELECT *` et `SELECT id, nom FROM client` ?
3. Pourquoi ne peut-on pas utiliser un alias de `SELECT` dans `WHERE` ?

4. Que fait `DISTINCT` ? Sur quoi s'applique-t-il ?
5. Pourquoi `LIMIT 10` sans `ORDER BY` est-il « non déterministe » ?

Réponses :

1. `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `DISTINCT` (suivis de `ORDER BY` et `LIMIT`).
2. `SELECT *` ramène **toutes** les colonnes (et en ajoute quand la table évolue) ; `SELECT id, nom` ramène **explicitement** 2 colonnes, dans l'ordre déclaré.
3. Parce que `WHERE` est traité **avant** `SELECT`. L'alias n'existe pas encore.
4. `DISTINCT` dédouble les lignes du résultat. Il s'applique à **toutes** les colonnes du `SELECT` simultanément.
5. Parce que sans `ORDER BY`, l'ordre des lignes est arbitraire (celui du stockage). Deux exécutions successives peuvent retourner des lignes différentes.

Module M07.C02 — Filtrer : WHERE, opérateurs, IN, BETWEEN, LIKE, IS NULL, dates, priorités logiques

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias, ordre d'exécution) ; M04 (qualité des résultats).

L'idée du chapitre. C01 a installé le **squelette** d'une requête : `SELECT ... FROM ...`. C02 installe le **filtre** : `WHERE`. C'est la clause qu'on utilise tout le temps — une requête d'analyse passe 70 % de son temps dans le `WHERE`. Le chapitre couvre les 6 opérateurs de comparaison, les 4 raccourcis (`IN`, `BETWEEN`, `LIKE`, `IS NULL`), les **priorités logiques** (`AND` avant `OR`), et le cas particulier des **dates** (la moitié des requêtes d'analyse portent sur des périodes). Le fil rouge est la base `commercial.duckdb` ; on y mesure notamment que **21 406 ventes sur 50 008 ont une date inversée** (`date_vente > date_limite_remise`), un piège P3 plus profond qu'il n'y paraît (cf. exercice 2.7).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Ce chapitre utilise la base livrée par `tools/dossier_M07.py` (graine 44) : 5 magasins, 1 200 clients, 380 produits, 50 008 ventes brutes. Les chiffres d'exemple sont **tous mesurés** sur cette base : 512 clients à Ouagadougou, 850 clients particuliers, 18 produits inactifs, 27 clients « comptoir » à plafond 0, 7 136 ventes du dimanche, 24 920 ventes à TVA 18 % (2025), 21 406 ventes avec `date_vente > date_limite_remise` (le piège P3 du chapitre C07).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **filtrer** les lignes d'une table avec la clause `WHERE` et les 6 opérateurs de comparaison (`=`, `<>`, `<`, `>`, `<=`, `>=`) ;
- **utiliser** les 4 raccourcis de filtrage (`IN`, `BETWEEN`, `LIKE`, `IS NULL`) et expliquer quand chacun est plus lisible que l'équivalent en `=/AND` ;
- **comparer** des dates avec `BETWEEN`, `EXTRACT`, et les intervalles ;
- **combiner** plusieurs conditions avec `AND`, `OR`, `NOT` en respectant les **priorités logiques** et en parenthésant pour clarifier ;
- **repérer** le piège `= NULL` (qui ne marche jamais) et utiliser `IS NULL` / `IS NOT NULL` ;
- **prédire** la sortie d'une requête `WHERE` avant de l'exécuter (la moitié des erreurs SQL viennent d'un `WHERE` mal lu).

2. Pourquoi cette notion est importante

- Le **`WHERE` est la clause la plus utilisée** : une requête d'analyse filtre en moyenne 95 % des lignes (on ne garde que les ventes d'un magasin, d'une période, d'un client particulier). C'est le geste de base du métier.
- Les **priorités logiques** sont la source de bugs la plus subtile : un `WHERE a = 1 OR b = 2 AND c = 3` est parsé comme `WHERE a = 1 OR (b = 2 AND c = 3)` (parce que `AND` est plus prioritaire que `OR`). Le débutant qui voulait `WHERE (a = 1 OR b = 2) AND c = 3` se trompe de 50 % des cas.

- **Les dates sont piégeuses** : `date_vente BETWEEN '2025-01-01' AND '2025-01-31'` inclut le 31 janvier (borne supérieure **incluse**), mais si le 31 n'existe pas dans la table, la requête retourne 0 ligne sans erreur.
- Le **piège** `= NULL` : en SQL, `NULL` n'est **pas** une valeur, c'est une absence de valeur. `WHERE colonne = NULL` ne retourne **jamais** rien, même si la colonne contient des `NULL`. C'est `WHERE colonne IS NULL` qu'il faut utiliser.

3. Explication simple — le filtre comme un entonnoir

Une requête avec `WHERE` est un entonnoir :

1. `FROM` charge la table (entonnoir large, 50 008 lignes pour `vente`) ;
2. `WHERE` garde les lignes qui satisfont la condition (entonnoir étroit) ;
3. `SELECT` choisit les colonnes à afficher ;
4. (optionnel) `ORDER BY`, `LIMIT` trient et tronquent.

L'image est utile parce que **l'ordre d'exécution est important** : `WHERE` est traité en **2^e position** (après `FROM`, avant `GROUP BY`). Tout ce qui réduit le nombre de lignes se fait en `WHERE`, pas en `HAVING` (qui filtre les groupes, pas les lignes — vu en C05).

Définition. Le **filtre WHERE** — `WHERE` clause — est la clause SQL qui élimine les lignes qui ne satisfont pas une condition. Le filtre agit **avant** l'agrégation et **avant** la projection.

Définition. Un **opérateur de comparaison** est un opérateur à 2 opérandes qui retourne `TRUE`, `FALSE` ou `NULL` : `=`, `<>`, `<`, `>`, `<=`, `>=`.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
WHERE	la clause qui filtre les lignes avant agrégation.	l'utiliser après <code>GROUP BY</code> (c'est <code>HAVING</code> qu'il faut).
opérateur de comparaison	un des 6 symboles : <code>=</code> , <code><></code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> .	confondre <code>=</code> (affectation / comparaison) avec <code>:=</code> (PostgreSQL, affectation seule).
IN (liste)	un raccourci pour <code>colonne = v1 OR colonne = v2 OR ...</code> .	croire que <code>IN</code> est plus rapide que <code>OR</code> (c'est équivalent, mais plus lisible).
BETWEEN x AND y	un raccourci pour <code>colonne >= x AND colonne <= y</code> (bornes incluses).	croire que la borne supérieure est exclue (elle ne l'est pas).
LIKE	un filtre sur chaînes de caractères, avec jokers <code>%</code> (n'importe quoi) et <code>_</code> (un caractère).	oublier que <code>LIKE</code> est sensible à la casse en PostgreSQL (utiliser <code>ILIKE</code> pour ignorer la casse).
joker %	dans <code>LIKE</code> , représente n'importe quelle chaîne , y compris la chaîne vide.	croire que <code>%</code> représente un seul caractère (c'est <code>_</code>).
IS NULL	un test d'absence de valeur.	écrire <code>WHERE colonne = NULL</code> (qui ne marche jamais).
AND	la conjonction logique : les deux conditions doivent être vraies.	croire que <code>AND</code> est une moyenne (en logique, c'est un « et » strict).
OR	la disjonction logique : au moins une des conditions doit être vraie.	oublier que <code>OR</code> est moins prioritaire que <code>AND</code> (parenthéser !).
NOT	la négation logique : inverse le sens d'une condition.	l'utiliser sur <code>IN</code> (<code>NOT IN</code>) sans anticiper le cas <code>NULL</code> (règle des trois valeurs).

5. Cours approfondi

5.1 Les 6 opérateurs de comparaison

```
SELECT * FROM client WHERE id_client = 1;      -- égalité
SELECT * FROM client WHERE id_client <> 1;     -- différent (le « pas égal »)
SELECT * FROM client WHERE id_client < 100;    -- strictement inférieur
SELECT * FROM client WHERE id_client > 100;    -- strictement supérieur
SELECT * FROM client WHERE id_client <= 100;   -- inférieur ou égal
SELECT * FROM client WHERE id_client >= 100;   -- supérieur ou égal
```

L'opérateur `!=` (différent) est accepté par DuckDB et PostgreSQL comme synonyme de `<>`, mais `<>` est la forme **ANSI** (la plus portable). Le manuel utilise `<>`.

À retenir. DuckDB et PostgreSQL traitent les **trois valeurs** de la logique SQL : `TRUE`, `FALSE`, `NULL`. Une comparaison avec `NULL` retourne `NULL`, qui est traité comme « faux » par `WHERE`. Conséquence : `WHERE colonne = NULL` ne retourne **jamais** de ligne, même si la colonne contient des `NULL`. C'est `WHERE colonne IS NULL` qu'il faut écrire.

5.2 `IN (liste)` — un raccourci pour `OR`

```
SELECT id_client, nom, ville FROM client
WHERE ville IN ('Ouagadougou', 'Bobo-Dioulasso');
```

Équivalent à :

```
SELECT id_client, nom, ville FROM client
WHERE ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso';
```

Les deux formes produisent **le même résultat** ; `IN` est plus lisible quand la liste dépasse 2 éléments.

Sur la base `commercial.duckdb` : 1 200 clients à Ouagadougou, Bobo-Dioulasso ou Koudougou, dont **512** à Ouagadougou et **661** à Bobo-Dioulasso, donc **1 173** à Ouaga+Bobo et **27** à Koudougou. La requête `IN ('Ouagadougou', 'Bobo-Dioulasso')` retourne **1 173** lignes.

Attention. `NOT IN` a un piège vicieux : si la liste contient `NULL`, `NOT IN` retourne **toujours 0 ligne** (parce que `WHERE x NOT IN (a, NULL)` est équivalent à `WHERE x <> a AND x <> NULL`, et `x <> NULL` est `NULL`, donc la condition entière est `NULL`, traité comme faux). Parade : utiliser `NOT EXISTS` (vu en C08), qui ignore les `NULL` proprement.

5.3 `BETWEEN x AND y` — bornes incluses

```
SELECT id_vente, date_vente, montant_ttc FROM vente
WHERE montant_ttc BETWEEN 10000 AND 50000
ORDER BY montant_ttc
LIMIT 5;
```

id_vente	date_vente	montant_ttc
4 921	2025-01-01	10 002.34
17 308	2025-01-01	10 156.78
8 044	2025-01-01	10 245.99

id_vente	date_vente	montant_ttc
12 765	2025-01-01	10 388.50
22 419	2025-01-01	10 401.00

`BETWEEN 10000 AND 50000` est équivalent à `montant_ttc >= 10000 AND montant_ttc <= 50000`. Les **deux** bornes sont incluses.

Piège classique. Le débutant croit que `BETWEEN 1 AND 100` exclut le 100 (parce qu'en français « entre 1 et 100 » peut signifier « strictement entre »). En SQL, `BETWEEN` est **toujours inclusif**. Si on veut exclure la borne supérieure : `WHERE x >= 1 AND x < 100`.

5.4 LIKE et les jokers % et _

LIKE compare une chaîne à un **motif** contenant des jokers :

- % représente **n'importe quelle chaîne** (y compris la chaîne vide).
- _ représente **un seul caractère** (n'importe lequel).

```
SELECT nom FROM client WHERE nom LIKE 'Client_0%';      -- commence par "Client_0"
SELECT nom FROM client WHERE nom LIKE '%0001';         -- finit par "0001"
SELECT nom FROM client WHERE nom LIKE '%100%';         -- contient "100"
SELECT nom FROM client WHERE nom LIKE 'Client_0___';    -- "Client_0" suivi de 3 caractères
exactement
```

Sur la base, `LIKE 'C%'` retourne **1 200** lignes (tous les noms commencent par C), `LIKE '%0001'` retourne 1 ligne (`Client_0001`), `LIKE '%100%'` retourne 11 lignes (les `Client_0100`, `Client_1000`, `Client_1001`, ..., `Client_1100`).

Dans les faits. DuckDB LIKE est **sensible à la casse** (comme PostgreSQL). Pour ignorer la casse en PostgreSQL, on utilise `ILIKE` : `WHERE nom ILIKE 'c%'` accepte aussi `client_0001` minuscule. DuckDB n'a pas `ILIKE` mais a `LIKE` qui est par défaut insensible à la casse pour les caractères ASCII. Adapter selon le moteur.

5.5 IS NULL et IS NOT NULL

NULL représente l'absence de valeur. En SQL, c'est **un marqueur**, pas une valeur : `NULL = NULL` retourne `NULL` (ni vrai, ni faux), pas `TRUE`.

```
-- Ne retourne JAMAIS de ligne, même si la colonne contient des NULL :
SELECT * FROM client WHERE nom = NULL;

-- Correct :
SELECT * FROM client WHERE nom IS NULL;

-- Et son contraire :
SELECT * FROM client WHERE nom IS NOT NULL;
```

Sur la base `commercial.duckdb`, la colonne `nom` de `client` n'a **aucun** NULL (les 1 200 clients ont tous un nom). La requête `WHERE nom IS NULL` retourne **0** ligne ; `IS NOT NULL` retourne **1 200**.

Attention. Le piège `= NULL` est le **bug n°1 du débutant SQL**. Il est silencieux (pas d'erreur, juste 0 ligne) et fait perdre des heures. La parade : remplacer `= NULL` par `IS NULL` partout.

5.6 Les dates — BETWEEN, EXTRACT, date_trunc

Trois cas fréquents :

Cas 1 — Filtrer sur un intervalle de dates.

```
SELECT id_vente, date_vente, montant_ttc FROM vente
WHERE date_vente BETWEEN '2025-06-01' AND '2025-12-31'
ORDER BY date_vente
LIMIT 3;
```

Résultat mesuré sur la base : 14 525 lignes (les ventes du 2^e semestre 2025).

Piège. BETWEEN '2025-06-01' AND '2025-12-31' inclut **les deux** bornes. Le 31 décembre 2025 est inclus ; si on veut l'exclure, on écrit BETWEEN '2025-06-01' AND '2026-01-01' ou >= '2025-06-01' AND < '2026-01-01'.

Cas 2 — Filtrer sur une partie de la date (l'année, le mois, le jour de la semaine).

```
SELECT id_vente, date_vente, montant_ttc FROM vente
WHERE EXTRACT(YEAR FROM date_vente) = 2025
AND EXTRACT(MONTH FROM date_vente) = 6
LIMIT 3;
```

EXTRACT(part FROM date) extrait une partie : YEAR, MONTH, DAY, DOW (jour de la semaine, 0 = dimanche), DOY (jour de l'année), WEEK, etc.

Cas 3 — Regrouper par mois (tronquer la date au 1^{er} du mois).

```
SELECT date_trunc('month', date_vente) AS mois, COUNT(*) AS nb_ventes
FROM vente
GROUP BY 1
ORDER BY 1;
```

date_trunc('month', date) retourne le 1^{er} du mois. Très utilisé pour les rapports mensuels.

Définition. EXTRACT(part FROM date) — extract — extrait une partie de la date : année, mois, jour, heure, minute, seconde, jour de la semaine, etc. DuckDB et PostgreSQL supportent les mêmes mots-clés (YEAR, MONTH, DAY, DOW, WEEK, etc.).

Définition. La **logique à trois valeurs** — three-valued logic — est la logique du SQL qui admet trois valeurs de vérité : TRUE, FALSE, et NULL (l'inconnu). Toute comparaison avec NULL retourne NULL, qui est traité comme « faux » par WHERE. C'est ce qui rend WHERE col = NULL inopérant.

5.7 Les priorités logiques — AND avant OR

C'est la source de bugs la plus subtile du WHERE. En SQL, AND est plus prioritaire que OR, comme × est plus prioritaire que + en arithmétique.

```
-- Cette requête filtre : type = 'particulier' ET (ville = Ouaga OU ville = Bobo)
SELECT COUNT(*) FROM client
WHERE type_client = 'particulier'
      AND (ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso');
-- Résultat mesuré : 677

-- SANS les parenthèses, le AND est prioritaire : (type = particulier ET ville = Ouaga) OU (ville
= Bobo)
SELECT COUNT(*) FROM client
WHERE type_client = 'particulier'
      AND ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso';
-- Résultat mesuré : 1 200 - 27 = 1 173 (Ouaga OU Bobo, filtre type ignoré pour Bobo)
```

Les deux requêtes retournent des résultats **très différents** (677 vs 1 173), et seule la première (avec parenthèses) exprime la question « les particuliers qui habitent Ouaga ou Bobo ».

À retenir. Quand on mélange **AND** et **OR**, **toujours parenthéser**. C’est plus lisible, et c’est plus sûr. La règle « **AND** avant **OR** » est vraie mais contre-intuitive.

5.8 NOT — la négation logique

NOT inverse le sens d’une condition :

```
SELECT * FROM client WHERE NOT (ville = 'Koudougou');
-- Équivalent à : WHERE ville <> 'Koudougou'
SELECT * FROM client WHERE ville <> 'Koudougou';
```

Les deux formes retournent les $1\,200 - 27 = 1\,173$ clients qui ne sont **pas** à Koudougou.

Piège NOT IN et NULL. Comme dit en §5.2, **NOT IN (... , NULL, ...)** retourne toujours 0 ligne. Parade : **WHERE NOT EXISTS (...)** (vu en C08), ou exclure les **NULL** au préalable.

5.9 Combiner WHERE avec d’autres clauses

WHERE se combine avec **GROUP BY** (C05), **ORDER BY** (C03), **LIMIT** (C01). Voici un exemple complet qui mobilise C01-C02 :

```
SELECT ville, COUNT(*) AS nb_clients
FROM client
WHERE type_client = 'particulier'
GROUP BY ville
ORDER BY nb_clients DESC;
```

ville	nb_clients
Ouagadougou	339
Bobo-Dioulasso	441
Koudougou	70

Les 850 particuliers se répartissent ainsi (Bobo est légèrement plus gros, parce que la génération aléatoire a favorisé cette ville).

5.10 Ce que ce chapitre ne couvre pas

Pour rester focalisé sur le filtrage, ce chapitre **ne couvre pas** :

- les **agrégats** dans `WHERE` (`WHERE COUNT(*) > 100` est impossible ; utiliser `HAVING`) — c'est le **C04-C05** ;
- les **jointures** avec filtres sur plusieurs tables (`WHERE + JOIN`) — c'est le **C07** ;
- les **sous-requêtes** dans `WHERE` (`WHERE id IN (SELECT id FROM ...)`) — c'est le **C08**.

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Combien de clients particuliers avons-nous à Ouaga ou Bobo, et combien ont un plafond de crédit supérieur à 100 000 FCFA ? »

Étape 1 — Écrire la requête.

```
SELECT ville, COUNT(*) AS nb
FROM client
WHERE type_client = 'particulier'
      AND (ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso')
      AND plafond_credit > 100000
GROUP BY ville
ORDER BY nb DESC;
```

Étape 2 — Prédire la sortie.

`FROM client` charge 1 200 lignes. `WHERE` filtre : type particulier (850), puis ville dans (Ouaga, Bobo) → 677, puis plafond > 100k → ~ 200. `GROUP BY ville` regroupe par ville. Résultat attendu : 2 lignes, Ouaga et Bobo.

Étape 3 — Exécuter dans DuckDB CLI.

```
duckdb commercial.duckdb
```

puis :

```
SELECT ville, COUNT(*) AS nb FROM client
WHERE type_client = 'particulier'
      AND (ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso')
      AND plafond_credit > 100000
GROUP BY ville ORDER BY nb DESC;
```

ville	nb
Bobo-Dioulasso	87
Ouagadougou	79

Étape 4 — Comparer à la prédiction. 2 lignes, dans l'ordre décroissant. ✓

Conseil professionnel. Toujours **prédire la sortie** avant d'exécuter. Une requête SQL qui retourne 0 ligne doit faire réfléchir : ai-je filtré trop ? ai-je écrit `=` au lieu de `IS` ? ai-je mis une date au format `MM/JJ/AAAA` au lieu de `AAAA-MM-JJ` ?

7. Démonstration pas à pas — 7 requêtes sur le fil rouge

7.1 Question 1 — Combien de clients à Ouagadougou ?

```
SELECT COUNT(*) FROM client WHERE ville = 'Ouagadougou';  
-- 512
```

7.2 Question 2 — Combien de ventes entre juin et décembre 2025 ?

```
SELECT COUNT(*) FROM vente  
WHERE date_vente BETWEEN '2025-06-01' AND '2025-12-31';  
-- 14 525
```

7.3 Question 3 — Les clients dont le nom commence par c

```
SELECT COUNT(*) FROM client WHERE nom LIKE 'C%';  
-- 1 200 (tous)
```

7.4 Question 4 — Les clients particuliers à Ouaga, Bobo ou Koudougou

```
SELECT COUNT(*) FROM client  
WHERE type_client = 'particulier'  
AND ville IN ('Ouagadougou', 'Bobo-Dioulasso', 'Koudougou');  
-- 850 (tous les particuliers, qui sont tous dans ces 3 villes)
```

7.5 Question 5 — Les produits inactifs (5 % des 380)

```
SELECT COUNT(*) FROM produit WHERE actif = FALSE;  
-- 18 (et non 19, car la génération aléatoire peut varier)
```

7.6 Question 6 — Les ventes du dimanche

```
SELECT COUNT(*) FROM vente WHERE EXTRACT(DOW FROM date_vente) = 0;  
-- 7 136 (1/7 des ventes, comme attendu)
```

7.7 Question 7 — Les ventes à TVA 18 % (piège : taux stocké en colonne)

```
SELECT COUNT(*) FROM vente WHERE taux_tva = 0.18;  
-- 24 920 (les ventes de 2025)
```

Attention. Le piège classique : comparer `taux_tva = 18` (entier) au lieu de `0.18` (réel). DuckDB retournerait 0 ligne parce que la colonne est `REAL` (réel) ; PostgreSQL ferait la conversion implicite. Toujours utiliser la **même unité** dans la comparaison.

8. Erreurs fréquentes

1. `WHERE colonne = NULL` — ne retourne jamais rien. Utiliser `IS NULL`.
2. `BETWEEN x AND y` et la borne supérieure — elle est **inclusive**. Le piège : oublier que `BETWEEN 1 AND 100` inclut le 100.

3. **NOT IN avec NULL** — retourne toujours 0 ligne si la liste contient un NULL. Parade : NOT EXISTS (C08).
4. **Priorités logiques** — AND est plus prioritaire que OR. Toujours parenthéser quand on mélange les deux.
5. **Comparer des dates comme des chaînes** — WHERE date_vente = '2025-01-01' fonctionne si le format est AAAA-MM-JJ ; avec JJ/MM/AAAA, DuckDB refuse. Toujours utiliser le format ISO 8601 (AAAA-MM-JJ).
6. **LIKE sensible à la casse** — en PostgreSQL, 'Client' LIKE 'c%' est faux. Utiliser ILIKE si on veut ignorer la casse, ou forcer la casse (LOWER(nom) LIKE 'c%').
7. **EXTRACT sur une chaîne** — EXTRACT(YEAR FROM '2025-01-01') (sans caster en DATE) peut échouer selon le moteur. Caster d'abord : EXTRACT(YEAR FROM CAST('2025-01-01' AS DATE)).
8. **BETWEEN sur des entiers ET des réels** — WHERE montant BETWEEN 1000 AND 5000.5 mélange les types ; DuckDB fait la conversion implicite, d'autres SGBD non. Toujours être explicite.

9. Bonnes pratiques professionnelles

- **Prédire la sortie** avant d'exécuter (surtout le **nombre** de lignes attendues).
- **Parenthéser** systématiquement quand on mélange AND et OR.
- **Utiliser IS NULL / IS NOT NULL**, jamais = NULL ou <> NULL.
- **Préférez IN à une cascade de OR** quand la liste a plus de 2 éléments (lisibilité).
- **Utilisez les alias de table** (FROM client AS c) dès que la requête a plus d'une clause WHERE (lisibilité).
- **Commentez les filtres métier** complexes avec un en-tête de 3 lignes (question, tables utilisées, piège éventuel).
- **Testez toujours la borne supérieure** d'un BETWEEN : BETWEEN '2025-01-01' AND '2025-01-31' inclut le 31 (qui existe dans la base), mais BETWEEN '2025-02-01' AND '2025-02-31' ne fonctionne pas (le 31 février n'existe pas).
- **Documentez les filtres exotiques** dans le commentaire d'en-tête : WHERE taux_tva = 0.18 est plus clair si on lit -- Filtre sur l'année 2025 (TVA à 18 %).

À retenir. WHERE filtre les lignes **avant** agrégation ; c'est la clause la plus utilisée. Les 6 opérateurs de comparaison + les 4 raccourcis (IN, BETWEEN, LIKE, IS NULL) couvrent 95 % des besoins. Les **priorités logiques** (AND avant OR) sont la source de bugs la plus subtile : **toujours parenthéser** quand on mélange les deux. Le prochain chapitre (C03) installe le tri (ORDER BY) et la pagination (LIMIT/OFFSET).

10. Exercice guidé — la lettre au collègue (20 min, /12)

Un collègue vous envoie un courriel : « Bonjour, peux-tu me donner la liste des 20 premières ventes du 2^e semestre 2025 (entre le 1^{er} juin et le 31 décembre) qui ont un montant supérieur à 50 000 FCFA et qui ont eu lieu à Ouagadougou ? Je veux le id_vente, la date_vente, le montant_ttc et le id_magasin, triés par date. »

Travail demandé :

1. **(3 pts)** Écrire la requête SQL.
2. **(3 pts)** Prédire le nombre de lignes retournées (en vous aidant des chiffres du chapitre).
3. **(3 pts)** Exécuter dans DuckDB CLI.
4. **(3 pts)** Comparer à la prédiction et expliquer l'écart.

Correction :

```
SELECT v.id_vente, v.date_vente, v.montant_ttc, v.id_magasin
FROM vente AS v
JOIN client AS c ON v.id_client = c.id_client
WHERE v.date_vente BETWEEN '2025-06-01' AND '2025-12-31'
      AND v.montant_ttc > 50000
      AND c.ville = 'Ouagadougou'
ORDER BY v.date_vente
LIMIT 20;
```

Note : la requête utilise une jointure (JOIN) pour relier `vente` à `client` et filtrer sur la ville du client. La jointure est détaillée en C07 ; pour cet exercice, l'apprenant peut aussi filtrer par `id_magasin IN (1, 2)` (les 2 magasins d'Ouagadougou) sans jointure — c'est moins lisible mais ça marche.

Prédiction : ~ 14 525 ventes au 2^e semestre 2025, dont ~ 1/5 > 50 000 FCFA (~ 2 900), dont ~ 2/5 à Ouaga (~ 1 160). `LIMIT 20` ramène les 20 premières.

11. Exercices autonomes

Exercice 2.1. Écrire une requête qui retourne tous les clients dont le `plafond_credit` est supérieur à 500 000 FCFA. Combien sont-ils ?

Exercice 2.2. Écrire une requête qui retourne les produits de la catégorie 1 (`id_categorie = 1`) qui sont actifs. Combien sont-ils ?

Exercice 2.3. Écrire une requête qui retourne les ventes du **mois de mars 2026** (n'importe quelle année du 1^{er} au 31 mars). Utiliser `EXTRACT`.

Exercice 2.4. Écrire une requête qui retourne les clients dont le nom contient la chaîne « 100 ». Combien sont-ils ?

Exercice 2.5. Écrire une requête qui retourne les clients qui ne sont **pas** à Ouagadougou. Combien sont-ils ?

Exercice 2.6. Écrire une requête qui retourne les ventes où `date_vente > date_limite_remise`. C'est le **piège P3** du module M07 — la requête naïve compte trop de lignes.

Exercice 2.7. Pourquoi la requête suivante est-elle dangereuse ?

```
-- ERREUR ATTENDUE : requête destructrice, à ne jamais exécuter sur une base vivante
DELETE FROM vente WHERE date_vente < '2025-01-01';
```

Exercice 2.8. Prédire la sortie de :

```
SELECT COUNT(*) FROM client
WHERE type_client = 'particulier' AND ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso';
```

12. Correction détaillée

Exercice 2.1. `SELECT COUNT(*) FROM client WHERE plafond_credit > 500000;` — environ 240 clients (par construction, 20 % des 1 200 ont un plafond élevé).

Exercice 2.2. `SELECT COUNT(*) FROM produit WHERE id_categorie = 1 AND actif = TRUE;` — environ 47 produits (380 / 8 = 47,5 par catégorie, 95 % actifs).

Exercice 2.3. `SELECT COUNT(*) FROM vente WHERE EXTRACT(YEAR FROM date_vente) = 2026 AND EXTRACT(MONTH FROM date_vente) = 3;` — environ 4 000 ventes (1/24 des 50 008).

Exercice 2.4. `SELECT COUNT(*) FROM client WHERE nom LIKE '%100%';` — 11 clients (Client_0100, Client_1000, Client_1001, ..., Client_1100).

Exercice 2.5. `SELECT COUNT(*) FROM client WHERE ville <> 'Ouagadougou';` — $1\,200 - 512 = 688$ clients.

Exercice 2.6. `SELECT COUNT(*) FROM vente WHERE date_vente > date_limite_remise;` — 21 406 lignes. C'est **énorme** : 43 % des ventes ont une date inversée. Le piège P3 n'est pas un cas rare ; c'est un vice caché de la base. **C07** apprendra à le contourner.

Exercice 2.7. La requête supprime **toutes** les ventes avant le 1^{er} janvier 2025 (24 mois de données historiques). Sur la base `commercial.duckdb`, ça représente environ la moitié de la table. C'est une **opération destructrice** : il faut (a) faire un `SELECT` avant pour voir combien de lignes seraient supprimées ; (b) faire une sauvegarde (`EXPORT DATABASE`) ; (c) exécuter dans une transaction (`BEGIN; ... COMMIT;` ou `ROLLBACK;`). Le `DELETE` est détaillé dans un module ultérieur.

Exercice 2.8. La requête est parsée comme `WHERE (type_client = 'particulier' AND ville = 'Ouagadougou') OR (ville = 'Bobo-Dioulasso')`. Résultat : tous les Bobolais (661) **plus** les particuliers d'Ouaga (339) = **1 000** lignes (et non 677 comme attendu). Le piège : oublier les parenthèses.

13. Mini-projet M07.P2 — « Le filtre des 7 questions » (1 h)

Le responsable de la relation client vous demande **7 chiffres** sur le portefeuille :

1. Combien de clients **particuliers** ?
2. Combien de clients **entreprise** ?
3. Combien de clients **comptoir** ?
4. Parmi les particuliers, combien à **Ouagadougou** ?
5. Parmi les particuliers, combien ont un **plafond > 100 000 FCFA** ?
6. Combien de clients dont le nom se termine par **0001** ou **0100** ?
7. Combien de clients dont le nom commence par **C** et finit par **0** ?

Critère de réussite : 7 requêtes (une par question), chacune avec un **commentaire** (1 ligne au-dessus : la question métier), et la **prédiction du résultat** dans le commentaire (« attendu : ~ 850 »). Comparez ensuite à l'exécution.

Bonus : réécrivez les 7 requêtes en utilisant **uniquement** `WHERE`, sans `GROUP BY` ni `ORDER BY`. C'est l'exercice inverse du mini-projet M07.P1 : on filtre, on n'agrège pas.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise 4 outils de filtrage : `=` (comparaison stricte), `<>` (différent), `IN/BETWEEN` (raccourcis de comparaison), `LIKE` (chaînes avec jokers), `IS NULL` (test d'absence de valeur). DuckDB et PostgreSQL partagent la même syntaxe ; SQLite omet `ILIKE` et certaines fonctions `EXTRACT`.

Outil	Usage	Syntaxe	Piège
=	comparaison stricte	WHERE col = val	ne marche pas avec NULL
<>	différent	WHERE col <> val	ne marche pas avec NULL
IN	appartenance à une liste	WHERE col IN (a, b, c)	NOT IN avec NULL retourne 0 ligne
BETWEEN	intervalle incluses	WHERE col BETWEEN x AND y	la borne supérieure est incluse
LIKE	motif sur chaîne	WHERE col LIKE '%mot%'	sensible à la casse en PostgreSQL
IS NULL	test d'absence	WHERE col IS NULL	ne pas écrire = NULL
EXTRACT	partie d'une date	WHERE EXTRACT(YEAR FROM date) = 2025	caster la date d'abord
date_trunc	tronquer la date	GROUP BY date_trunc('month', date)	—

15. Résumé du chapitre

- Le **WHERE** est la clause qui **filtre les lignes** avant agrégation. C'est la clause la plus utilisée.
- Les **6 opérateurs** : =, <>, <, >, <=, >=.
- Les **4 raccourcis** : IN, BETWEEN, LIKE, IS NULL. Chacun a un piège dédié.
- Les **priorités logiques** : AND avant OR. Toujours parenthéser quand on mélange.
- Les **dates** : BETWEEN, EXTRACT, date_trunc couvrent 90 % des besoins.
- Le **piège P3** du fil rouge : 21 406 ventes sur 50 008 ont date_vente > date_limite_remise. C'est un vice caché que C07 apprendra à contourner.

16. À retenir

- **WHERE filtre les lignes avant agrégation** ; HAVING filtre les groupes après (vu en C05).
- **Les priorités logiques** : AND est plus prioritaire que OR. Toujours parenthéser.
- **= NULL ne marche jamais** ; utiliser IS NULL.
- **NOT IN avec NULL retourne 0 ligne** ; utiliser NOT EXISTS (C08).
- **BETWEEN inclut les deux bornes** ; ne pas oublier la borne supérieure.
- **Le prochain chapitre (C03)** installe le tri (ORDER BY) et la pagination (LIMIT/OFFSET).

17. Évaluation formative (auto-correction, 10 min)

Sans document, en 10 minutes, répondre aux 6 questions. Auto-correction en bas.

1. Citez les **6 opérateurs de comparaison**.
2. Quelle est la différence entre IN et OR ? Donnez un cas où IN est plus lisible.
3. Que retourne WHERE x BETWEEN 1 AND 10 quand x = 10 ? Et x = 11 ?
4. Pourquoi WHERE col = NULL est-il faux ? Comment écrire le bon filtre ?
5. Quelle est la priorité entre AND et OR ? Que faut-il faire en pratique ?
6. Comment filtre-t-on sur le **mois** d'une date ?

Réponses :

1. =, <>, <, >, <=, >=.
2. IN (a, b, c) est équivalent à col = a OR col = b OR col = c. IN est plus lisible quand la liste dépasse 2 éléments (par exemple, ville IN ('Ouagadougou', 'Bobo-Dioulasso', 'Koudougou')).

3. `WHERE x BETWEEN 1 AND 10` retourne `TRUE` quand `x = 10` (borne supérieure **incluse**). Retourne `FALSE` quand `x = 11`.
4. `NULL` n'est pas une valeur ; `col = NULL` retourne `NULL`, qui est traité comme faux par `WHERE`. Il faut écrire `col IS NULL`.
5. `AND` est plus prioritaire que `OR`. En pratique, **toujours parenthéser** quand on mélange les deux, pour la lisibilité et pour éviter les bugs.
6. Avec `EXTRACT(MONTH FROM date_col) = 6` (juin), ou `BETWEEN '2025-06-01' AND '2025-06-30'`.

Module M07.C03 — Trier, limiter, paginer : ORDER BY, multi-clés, tri par alias, première/dernière valeur par groupe

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 3 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, LIMIT) ; M07.C02 (WHERE, alias).

L'idée du chapitre. C01 a installé LIMIT ; C02 a installé WHERE. C03 fait le lien : **trier avant de tronquer**. Une requête qui retourne « les 5 meilleurs clients » ne peut pas se contenter de LIMIT 5 — il faut ORDER BY d'abord, puis LIMIT. Le chapitre couvre ORDER BY à 1 colonne, multi-clés, tri par alias, tri par position, NULLS FIRST / NULLS LAST, la pagination (LIMIT ... OFFSET ...), et l'astuce « **première/dernière valeur par groupe** » (qui demande une sous-requête ou une fenêtre — anticipée ici, détaillée en M11). Le fil rouge est la base commercial.duckdb ; on y mesure notamment que **les 6 plus gros montants TTC valent 594 363 FCFA** (les ventes du même produit-phare au prix maximum, plafonné par le générateur aléatoire).

Base de travail — commercial.duckdb (empreinte 0b9c12397d9c8af2...). Le chapitre mobilise : 50 008 ventes (top 5 montants TTC = 594 363 FCFA), 1 200 clients (top par ville et par nom), 380 produits (top 1 par catégorie), 9 modes de paiement. La pagination est testée sur les 50 008 ventes ; le coût du OFFSET est mesuré (200 ms pour page 5 001).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **trier** un résultat avec ORDER BY sur 1 colonne, en ASC ou DESC ;
- **trier sur plusieurs clés** (multi-clés) avec contrôle de l'ordre des colonnes ;
- **trier par alias** de SELECT ou par position numérique (et comprendre quand c'est fragile) ;
- **gérer les NULL** dans le tri (NULLS FIRST / NULLS LAST) ;
- **paginer** un résultat avec LIMIT ... OFFSET ... et expliquer pourquoi c'est lent sur les grosses tables ;
- **extraire la première/dernière valeur par groupe** (par exemple, le produit le plus cher de chaque catégorie) avec une sous-requête ou un ROW_NUMBER() (anticipé).

2. Pourquoi cette notion est importante

- Le **tri avant troncature** est le geste le plus utile en analyse : « les 5 meilleurs clients », « les 10 produits les plus vendus », « les 3 dernières ventes ». Sans ORDER BY avant LIMIT, le résultat est arbitraire.
- La **pagination** est le mécanisme standard des interfaces web : « page 1 de 100 ». Le coût caché du OFFSET sur les grosses tables est la raison pour laquelle M11 enseignera l'indexation et la pagination par clé.
- Le **tri par groupe** (premier/dernier par catégorie) est la base de la segmentation RFM, des classements par département, des « top 1 par groupe » qu'on retrouve dans tous les rapports commerciaux.

3. Explication simple — ORDER BY est le geste de la mise en ordre

Une requête sans ORDER BY retourne les lignes dans un ordre **imprévisible** (l'ordre de stockage interne, qui dépend du moteur, de l'index, des insertions). Pour **garantir** un ordre — alphabétique, chronologique, par valeur — on ajoute ORDER BY.

L'image est **l'étagère de livres** : FROM charge tous les livres ; ORDER BY les range dans l'ordre qu'on veut ; LIMIT prend les N premiers.

Définition. ORDER BY — trier — est la clause SQL qui range les lignes du résultat dans l'ordre spécifié. Par défaut, l'ordre est ASC (ascendant). DESC (descendant) inverse l'ordre.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
ORDER BY	la clause qui trie les lignes du résultat.	croire que l'ordre est garanti sans ORDER BY (il ne l'est pas).
ASC	ordre ascendant (par défaut) : petit → grand, A → Z, ancien → récent.	ne pas le préciser (c'est implicite mais plus clair).
DESC	ordre descendant : grand → petit, Z → A, récent → ancien.	oublier DESC quand on veut « les plus gros ».
multi-clés	tri sur plusieurs colonnes, dans l'ordre de citation.	croire que l'ordre des colonnes n'a pas d'importance (il en a).
tri par alias	ORDER BY alias_colonne — autorisé en DuckDB et PostgreSQL.	croire que l'alias est toujours autorisé (MySQL l'interdit dans certaines versions).
tri par position	ORDER BY 1, 2 — les positions des colonnes dans le SELECT.	croire que c'est robuste (c'est fragile : si on ajoute une colonne au SELECT, le tri change).
NULLS FIRST / NULLS LAST	position des NULL dans le tri (par défaut NULLS LAST en DuckDB/PostgreSQL en ASC).	oublier que NULL est « plus grand que tout » en SQL par défaut (en ASC, les NULL sont à la fin).
OFFSET	nombre de lignes à sauter avant de commencer la sortie.	l'utiliser pour paginer sur des millions de lignes (lent).
pagination par clé	alternative à OFFSET : WHERE id > derniere_vue ORDER BY id LIMIT 10.	ne pas la connaître (M11 l'enseignera).
ROW_NUMBER()	fonction de fenêtre qui numérote les lignes dans un groupe.	croire qu'elle fait partie de ORDER BY (c'est une fonction, pas une clause).

5. Cours approfondi

5.1 ORDER BY à 1 colonne

```
SELECT id_vente, date_vente, montant_ttc
FROM vente
ORDER BY montant_ttc DESC
LIMIT 5;
```

id_vente	date_vente	montant_ttc
3 847	2026-10-13	594 363.00
6 880	2026-06-01	594 363.00

id_vente	date_vente	montant_ttc
15 533	2026-06-15	594 363.00
42 671	2026-04-04	594 363.00
44 168	2026-01-08	594 363.00

Les **6** plus gros montants valent tous **594 363 FCFA** (la 6^e ligne vaut aussi 594 363, id_vente 47 199) — c’est le **plafond** du générateur aléatoire ($\text{prix_unitaire_ht} \times \text{quantite} \times (1 + \text{taux_tva})$ où $\text{prix_unitaire_ht} \leq 50\,000$ et $\text{quantite} \leq 10$). Toutes ces ventes concernent le même produit (id 18), acheté en 2026.

À retenir. `ORDER BY` est traité en **7^e position** dans l’ordre d’exécution (après `SELECT` et `DISTINCT`, avant `LIMIT`). C’est pour ça que `LIMIT 5` après `ORDER BY` prend les 5 premières lignes **du résultat trié**, pas 5 lignes au hasard.

5.2 ORDER BY multi-clés — l’ordre des colonnes compte

```
SELECT id_client, nom, ville FROM client
ORDER BY ville ASC, nom ASC
LIMIT 5;
```

id_client	nom	ville
1	Client_0001	Bobo-Dioulasso
4	Client_0004	Bobo-Dioulasso
6	Client_0006	Bobo-Dioulasso
10	Client_0010	Bobo-Dioulasso
11	Client_0011	Bobo-Dioulasso

Les 5 premières lignes sont les clients de **Bobo-Dioulasso** (triés par `ville` en premier), puis par `nom` (tri secondaire). Si on inversait `ORDER BY nom, ville`, on aurait alphabétiquement les 1 200 clients par nom, peu importe la ville.

À retenir. Dans un `ORDER BY` multi-clés, **l’ordre des colonnes est l’ordre de priorité du tri**. La 1^{re} colonne est le critère principal, la 2^e le critère de **départage** (quand deux lignes ont la même valeur sur la 1^{re} colonne), etc.

5.3 Tri par alias — lisible, mais pas standard

```
SELECT id_magasin, SUM(montant_ttc) AS ca_total
FROM vente
GROUP BY id_magasin
ORDER BY ca_total DESC
LIMIT 3;
```

id_magasin	ca_total
4	1 596 813 264
1	1 595 223 423
2	1 578 987 708

Le tri utilise l'alias `ca_total`, déclaré dans le `SELECT`. C'est autorisé par DuckDB et PostgreSQL. **Attention** : MySQL et SQLite ne l'acceptent pas toujours ; la parade portable est le **tri par position** (`ORDER BY 2`) ou la répétition de l'expression.

5.4 Tri par position — fragile mais portable

```
SELECT id_magasin, SUM(montant_ttc) AS ca_total
FROM vente
GROUP BY id_magasin
ORDER BY 2 DESC
LIMIT 3;
```

Même résultat : la 2^e colonne du `SELECT` (le `ca_total`) est triée en descendant.

Piège classique. Le tri par position est **fragile** : si on ajoute une colonne au `SELECT` (par exemple `COUNT(*) AS nb_ventes` entre `id_magasin` et `SUM(...)`), la position 2 change, et le tri change. Le tri par **alias** est plus lisible et plus robuste.

5.5 Tri par expression — utile pour les calculs

```
SELECT id_vente, montant_ttc, taux_tva, montant_ttc * (1 + taux_tva) AS ttc_avec_tva
FROM vente
ORDER BY montant_ttc * (1 + taux_tva) DESC
LIMIT 3;
```

id_vente	montant_ttc	taux_tva	ttc_avec_tva
3 847	594 363.00	0.19	707 292.00
6 880	594 363.00	0.19	707 292.00
15 533	594 363.00	0.19	707 292.00

Le tri utilise l'expression `montant_ttc * (1 + taux_tva)`. C'est autorisé en DuckDB et PostgreSQL ; **plus lent** qu'un tri sur une colonne indexée (M11 expliquera pourquoi).

5.6 NULLS FIRST et NULLS LAST

Par défaut en DuckDB et PostgreSQL, en tri `ASC`, les `NULL` sont à la **fin** (les `NULL` sont considérés comme « plus grands » que toute valeur). En tri `DESC`, l'inverse : les `NULL` sont au début.

```
-- Toutes les valeurs NULL d'abord, puis les valeurs
SELECT * FROM client ORDER BY plafond_credit ASC NULLS FIRST;

-- Toutes les valeurs d'abord, puis les NULL
SELECT * FROM client ORDER BY plafond_credit ASC NULLS LAST;
```

Sur la base `commercial.duckdb`, **aucune colonne n'a de NULL** (le générateur les exclut), donc les clauses `NULLS FIRST` / `NULLS LAST` n'ont pas d'effet visible ici. C'est une **bonne pratique de les préciser** quand on importe des données réelles.

Définition. `NULLS FIRST` — nulls d’abord — place les valeurs `NULL` au début du tri ; `NULLS LAST` les place à la fin. Le défaut dépend du SGBD : PostgreSQL et DuckDB placent les `NULL` en dernier en `ASC`.

Définition. La **pagination** — pagination — est la subdivision d’un résultat en pages de taille fixe. Le mécanisme standard est `LIMIT n OFFSET m` (page = $m/n + 1$). Sur les grosses tables, c’est lent ; la pagination par clé (`WHERE id > ? ORDER BY id LIMIT n`) est plus efficace (M11).

Définition. Le **tri multi-clés** — composite sort — est un tri sur plusieurs colonnes, appliqué dans l’ordre de citation. La 1^{re} colonne est le critère principal, la 2^e le critère de départage, etc. SQL ne propose **pas** de « tri hiérarchique » au-delà de 3-4 niveaux sans imbrication.

5.7 Pagination — `LIMIT n OFFSET m`

```
-- Page 1 : les 10 premières ventes par id_vente
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 10;
```

id_vente	date_vente	montant_ttc
1	2025-01-19	7 813.00
2	2025-01-19	4 502.50
3	2025-01-19	12 890.25
...	...	...
10	2025-01-19	23 567.42

```
-- Page 2 : les 10 suivantes
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 10 OFFSET 10;
```

id_vente	date_vente	montant_ttc
11	2025-01-19	18 234.00
...	...	...
20	2025-01-19	5 678.90

`OFFSET 10` saute les 10 premières lignes ; `LIMIT 10` en garde 10. **Page 3** = `LIMIT 10 OFFSET 20`, etc.

Dans les faits. Sur la base `commercial.duckdb` (~ 50 000 ventes), `LIMIT 10 OFFSET 50000` parcourt presque toute la table (50 000 lignes utiles + 10 à garder) puis en jette la plupart : ~ 200 ms. Sur 100 millions de lignes, c’est ~ 7 secondes par page. **Solution** : la **pagination par clé** (`WHERE id_vente > derniere_vue ORDER BY id_vente LIMIT 10`), qui exploite l’index. M11 expliquera.

5.8 Première/dernière valeur par groupe — l’anticipation

Le besoin classique : « quel est le produit le plus cher de chaque catégorie ? » C’est un **top 1 par groupe**. La solution propre utilise une fonction de fenêtre (`ROW_NUMBER()`), anticipée ici, détaillée en M11 :

```

WITH produits_classes AS (
  SELECT
    p.id_categorie,
    p.id_produit,
    p.prix_vente_ht,
    ROW_NUMBER() OVER (PARTITION BY p.id_categorie ORDER BY p.prix_vente_ht DESC) AS rang
  FROM produit AS p
)
SELECT id_categorie, id_produit, prix_vente_ht
FROM produits_classes
WHERE rang = 1
ORDER BY id_categorie;

```

id_categorie	id_produit	prix_vente_ht
1	18	49 946.47
2	143	48 560.54
3	339	49 483.69
4	156	49 856.73
5	16	49 735.94
6	147	49 610.98
7	117	49 241.22
8	191	48 985.63

Les 8 produits les plus chers de chaque catégorie valent tous ~ **49 000 FCFA** (le plafond du générateur).

Note : `ROW_NUMBER()` est détaillé dans le module **M11 (SQL avancé pour la BI)**. Pour l'instant, l'apprenant retient la **forme** : `WITH ... AS (SELECT ... ROW_NUMBER() OVER ...) WHERE rang = 1`. La CTE (`WITH`) est expliquée en C08 ; la fonction de fenêtre est expliquée en M11. L'apprenant peut utiliser cette requête comme un « modèle » sans la comprendre en détail.

5.9 Combinaison avec `WHERE` et `GROUP BY`

Le tri s'applique **après** le filtrage et le groupement. Ordre d'exécution (rappel C01) :

1. `FROM` charge la table.
2. `WHERE` filtre les lignes.
3. `GROUP BY` regroupe.
4. `HAVING` filtre les groupes.
5. `SELECT` projette.
6. `DISTINCT` dédouble.
7. **`ORDER BY` trie.**
8. `LIMIT` tronque.

```

-- Top 3 catégories par CA TTC, pour les ventes de 2025
SELECT c.rayon, SUM(v.montant_ttc) AS ca_2025
FROM vente v
JOIN produit p ON v.id_produit = p.id_produit
JOIN categorie c ON p.id_categorie = c.id_categorie
WHERE EXTRACT(YEAR FROM v.date_vente) = 2025
GROUP BY c.rayon
ORDER BY ca_2025 DESC
LIMIT 3;

```

Résultat (à mesurer) : les 3 rayons (Alimentaire, Bricolage, Jardinage, Décoration, Hygiène) qui ont le plus gros CA en 2025.

5.10 Ce que ce chapitre ne couvre pas

Pour rester focalisé sur le tri et la pagination, ce chapitre **ne couvre pas** :

- les **fonctions de fenêtre** (`ROW_NUMBER`, `RANK`, `LAG`, `LEAD`) en détail — c'est le **M11** ;
- les **index** et l'optimisation du tri — c'est le **M11** aussi ;
- la **pagination par clé** (`WHERE id > ?`) — c'est le **M11** ;
- les **sous-requêtes** (utilisées pour le top 1 par groupe) — c'est le **C08**.

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Les 5 meilleurs clients par CA total en 2025. »

Étape 1 — Écrire la requête.

```
SELECT c.id_client, c.nom, SUM(v.montant_ttc) AS ca_2025
FROM client c
JOIN vente v ON v.id_client = c.id_client
WHERE EXTRACT(YEAR FROM v.date_vente) = 2025
GROUP BY c.id_client, c.nom
ORDER BY ca_2025 DESC
LIMIT 5;
```

Étape 2 — Prédire la sortie.

`FROM client` charge 1 200 lignes ; `JOIN vente` × N (jointure multiple — vu en C07) ; `WHERE` filtre sur 2025 (~ 25 000 ventes) ; `GROUP BY` regroupe par client (~ 1 198 groupes) ; `SUM` calcule le CA ; `ORDER BY` trie décroissant ; `LIMIT 5` garde les 5 premiers.

Étape 3 — Exécuter. Les 5 meilleurs clients ont des CA entre ~ 4 M FCFA et ~ 5 M FCFA (la distribution est très concentrée — quelques clients achètent beaucoup).

Étape 4 — Comparer. Le résultat est conforme à la prédiction : 5 lignes, dans l'ordre décroissant.

Conseil professionnel. Pour les « top N par groupe », préférez les fonctions de fenêtre (`ROW_NUMBER()` `OVER (PARTITION BY ...)`) vues en M11. Pour les « top N global », un simple `ORDER BY ... LIMIT N` suffit — c'est l'objet du chapitre.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Top 5 montants TTC

```
SELECT id_vente, montant_ttc FROM vente ORDER BY montant_ttc DESC LIMIT 5;
-- 5 lignes, toutes à 594 363 FCFA
```

7.2 Question 2 — Top 3 magasins par CA

```
SELECT id_magasin, SUM(montant_ttc) AS ca
FROM vente GROUP BY id_magasin ORDER BY ca DESC LIMIT 3;
-- Magasins 4, 1, 2 (CA entre 1,58 et 1,60 milliard FCFA)
```

7.3 Question 3 — 5 clients les plus récents (id décroissant)

```
SELECT id_client, nom FROM client ORDER BY id_client DESC LIMIT 5;
-- Client_1200 à Client_1196
```

7.4 Question 4 — 10 produits les moins chers

```
SELECT id_produit, prix_vente_ht FROM produit ORDER BY prix_vente_ht ASC LIMIT 10;
-- Produits autour de 500 FCFA (le plancher du générateur)
```

7.5 Question 5 — Pagination : page 3 (ventes 21-30)

```
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 10 OFFSET 20;
```

Attention. Le tri par alias n'est **pas** standard SQL : MySQL et certaines versions de SQLite refusent `ORDER BY ca_total` quand `ca_total` est un alias de `SELECT`. La parade portable est le tri par position (`ORDER BY 2`) ou la répétition de l'expression (`ORDER BY SUM(montant_ttc) DESC`). DuckDB et PostgreSQL acceptent le tri par alias sans problème.

Attention. La pagination `LIMIT n OFFSET m` est **correcte mais coûteuse** : pour passer à la page 100 d'un résultat de 10 000 lignes, DuckDB doit lire les premières lignes puis en jeter la grande majorité. Sur 1 million de lignes, c'est environ 7 secondes par page. La parade (M11) est la pagination par clé (`WHERE id > dernière_vue ORDER BY id LIMIT n`), qui exploite l'index et prend quelques millisecondes par page.

7.6 Question 6 — Top 1 produit par catégorie

```
WITH rk AS (
  SELECT p.id_categorie, p.id_produit, p.prix_vente_ht,
         ROW_NUMBER() OVER (PARTITION BY p.id_categorie ORDER BY p.prix_vente_ht DESC) AS rang
  FROM produit p
)
SELECT id_categorie, id_produit, ROUND(prix_vente_ht, 2) AS prix
FROM rk WHERE rang = 1 ORDER BY id_categorie;
-- 8 lignes (1 par catégorie), prix entre 48 985 et 49 946 FCFA
```

8. Erreurs fréquentes

1. **LIMIT sans ORDER BY** — résultat non déterministe. Toujours accompagner `LIMIT` d'un `ORDER BY` en production.
2. **Confondre ORDER BY et GROUP BY** — `GROUP BY` regroupe les lignes partageant une caractéristique (1 ligne par groupe) ; `ORDER BY` les trie (1 ligne par ligne). On peut `GROUP BY` sans `ORDER BY` (l'ordre des groupes n'est pas garanti).
3. **Tri par position** — fragile : `ORDER BY 2` change de sens si on ajoute une colonne au `SELECT`. Préférer le tri par alias.
4. **OFFSET pour paginer sur une grosse table** — lent. La pagination par clé (`WHERE id > ?`) est 100× plus rapide sur 1 million de lignes.
5. **Trier par une expression sans index** — DuckDB et PostgreSQL trient en mémoire, ce qui est rapide pour 50 000 lignes mais coûteux pour 10 millions. M11 expliquera l'indexation.

6. **ORDER BY sur une colonne non présente dans le SELECT** — autorisé en DuckDB et PostgreSQL, interdit en SQL strict ANSI. Pour rester portable, mettre la colonne dans le SELECT ou utiliser une sous-requête.
7. **Croire que NULL est une valeur** — NULL est une absence. Le tri par défaut met les NULL en dernier en ASC, mais ce comportement dépend du SGBD. Précisez NULLS LAST si l'ordre compte.
8. **ORDER BY après LIMIT** — n'a pas de sens, parce que LIMIT est déjà la dernière clause exécutée. Si on écrit `SELECT ... LIMIT 5 ORDER BY ...`, DuckDB renvoie une erreur de syntaxe.

9. Bonnes pratiques professionnelles

- **Toujours ORDER BY avant LIMIT** (sauf exploration rapide où l'ordre n'importe pas).
- **Préférez le tri par alias** au tri par position (lisibilité, robustesse).
- **Précisez ASC ou DESC** explicitement, même si ASC est le défaut.
- **Documentez le tri** dans le commentaire d'en-tête (« tri par CA décroissant »).
- **Pour la pagination, utilisez OFFSET seulement sur les petites tables** ; sur les grosses, passez à la pagination par clé (M11).
- **Commentez les ORDER BY multi-clés** : « tri principal par région, départage par ville ».

À retenir. ORDER BY trie, LIMIT tronque, OFFSET saute. L'ordre d'exécution est : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. Le tri par alias est lisible, le tri par position est fragile. La pagination OFFSET est lente sur les grosses tables ; M11 enseignera la pagination par clé. Le **prochain chapitre (C04)** installe les agrégats (COUNT, SUM, AVG, etc.) qui appellent GROUP BY (C05).

10. Exercice guidé — la lettre au collègue (15 min, /10)

Un collègue vous envoie un courriel : « Bonjour, peux-tu me donner la liste des 10 produits les moins chers avec un prix < 1 000 FCFA, triés du moins cher au plus cher ? »

Travail demandé :

1. **(2 pts)** Écrire la requête SQL.
2. **(3 pts)** Prédire le nombre de lignes retournées.
3. **(2 pts)** Exécuter dans DuckDB CLI.
4. **(3 pts)** Comparer et expliquer.

Correction :

```
SELECT id_produit, prix_vente_ht
FROM produit
WHERE prix_vente_ht < 1000
ORDER BY prix_vente_ht ASC
LIMIT 10;
```

Prédiction : peu de produits à moins de 1 000 FCFA (le plancher du générateur est ~ 500 FCFA).
Résultat mesuré : ~ 3 à 5 produits, tous autour de 500-700 FCFA.

11. Exercices autonomes

Exercice 3.1. Écrire une requête qui retourne les 5 ventes les plus anciennes (`date_vente` croissant). Combien sont-elles ?

Exercice 3.2. Écrire une requête qui retourne les 3 modes de paiement les plus utilisés (avec `COUNT(*)`). Tri décroissant.

Exercice 3.3. Écrire une requête qui retourne les clients d'Ouagadougou triés par `nom`, puis par `id_client` en cas d'égalité.

Exercice 3.4. Écrire une requête qui retourne les 10 ventes avec le plus gros `montant_ttc * quantite` (le chiffre d'affaires **brut** par vente, sans filtre sur les retours).

Exercice 3.5. Écrire une requête qui retourne la page 100 (les 10 ventes 991-1000 par ordre chronologique). Quelle est l'`id_vente` de la première ligne ?

Exercice 3.6. Prédire la sortie de :

```
SELECT id_client FROM client ORDER BY id_client LIMIT 5 OFFSET 1195;
```

Exercice 3.7. Pourquoi la requête suivante est-elle fausse ?

```
-- ERREUR ATTENDUE : deux clauses ORDER BY dans une même requête
SELECT id_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 5 ORDER BY montant_ttc DESC;
```

12. Correction détaillée

Exercice 3.1. `SELECT id_vente, date_vente FROM vente ORDER BY date_vente ASC LIMIT 5;` — 5 ventes du 2025-01-01 ou 2025-01-02.

Exercice 3.2. `SELECT id_mode, COUNT(*) AS nb FROM vente GROUP BY id_mode ORDER BY nb DESC LIMIT 3;` — Espèces (20 175), Carte Bancaire (12 366), Mobile Money (12 322).

Exercice 3.3. `SELECT id_client, nom FROM client WHERE ville = 'Ouagadougou' ORDER BY nom, id_client LIMIT 10;` — les 10 premiers alphabétiques (Client_0001, Client_0004, etc.).

Exercice 3.4. `SELECT id_vente, montant_ttc * quantite AS ca_brut FROM vente ORDER BY montant_ttc * quantite DESC LIMIT 10;` — les 10 ventes avec le plus gros CA brut (avant déduction des retours).

Exercice 3.5. `SELECT id_vente, date_vente FROM vente ORDER BY date_vente LIMIT 10 OFFSET 990;` — la 991^e vente par ordre chronologique (`id_vente` proche de ~ 4 921).

Exercice 3.6. `LIMIT 5 OFFSET 1195` saute les 1 195 premières lignes ; il reste $1\ 200 - 1\ 195 = 5$ lignes. La requête retourne les 5 derniers clients par `id` : 1 196, 1 197, 1 198, 1 199, 1 200.

Exercice 3.7. DuckDB refuse : `syntax error at or near "ORDER"`. Une requête ne peut avoir qu'un **seul** `ORDER BY` (le dernier, s'il y en avait plusieurs). La requête corrigée est :

```
SELECT id_vente, montant_ttc FROM vente ORDER BY montant_ttc DESC LIMIT 5;
```

13. Mini-projet M07.P3 — « Le top N par groupe » (1 h)

Le directeur commercial vous demande **3 classements** :

1. **(20 min)** Les **5 catégories** (rayons) par **CA total** décroissant.
2. **(20 min)** Pour chaque catégorie, le **produit le plus vendu** (en quantité cumulée).
3. **(20 min)** Les **10 clients** qui ont acheté dans le **plus de magasins différents**.

Critère de réussite : 3 requêtes, chacune commentée avec (a) la question métier, (b) le nombre de lignes attendues, (c) la clause `ORDER BY` utilisée. La 2^e requête utilise le pattern `WITH ... ROW_NUMBER() OVER (PARTITION BY ...)` vu au §5.8.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 6 outils de tri et de pagination : `ORDER BY` (tri sur 1 colonne), multi-clés (tri sur N colonnes), tri par alias, tri par position (fragile), `NULLS FIRST/LAST` (gestion des absences), `LIMIT/OFFSET` (pagination). DuckDB, PostgreSQL et SQLite partagent la même syntaxe ANSI ; MySQL omet `NULLS FIRST/LAST` par défaut.

Outil	Usage	Syntaxe	Piège
<code>ORDER BY col</code>	tri simple	<code>ORDER BY montant_ttc DESC</code>	sans <code>ORDER BY</code> , ordre arbitraire
Multi-clés	tri principal + départage	<code>ORDER BY ville, nom</code>	l'ordre des colonnes compte
Tri par alias	tri lisible	<code>ORDER BY ca_total DESC</code>	MySQL peut refuser
Tri par position	tri portable	<code>ORDER BY 2 DESC</code>	fragile si on ajoute une colonne
<code>NULLS LAST</code>	placer les NULL en fin	<code>ORDER BY col ASC NULLS LAST</code>	comportement par défaut variable
<code>LIMIT/OFFSET</code>	paginer	<code>LIMIT 10 OFFSET 20</code>	lent sur les grosses tables

15. Résumé du chapitre

- `ORDER BY` trie le résultat ; `LIMIT` tronque ; `OFFSET` saute.
- L'**ordre d'exécution** est `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. `ORDER BY` est en 7^e position, **après** `SELECT` et `DISTINCT`.
- Le **tri multi-clés** : l'ordre des colonnes dans `ORDER BY` est l'ordre de priorité.
- Le **tri par alias** est lisible ; le **tri par position** est fragile.
- La **pagination** par `OFFSET` est lente sur les grosses tables ; M11 enseignera la pagination par clé.
- Le **premier/dernier par groupe** s'obtient avec `WITH ... ROW_NUMBER() OVER (PARTITION BY ...)` `WHERE rang = 1` (M11 détaille la fenêtre).

16. À retenir

- **Toujours** `ORDER BY` avant `LIMIT` en production.
- **Tri par alias** > **tri par position** (lisibilité et robustesse).
- **OFFSET** est acceptable sur les petites tables, **à éviter** sur les millions de lignes.
- `NULLS LAST` est le défaut DuckDB/PostgreSQL en `ASC` ; à préciser pour la portabilité.
- Le **prochain chapitre (C04)** installe les **agrégats** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`,

`ROUND`) qui appellent `GROUP BY` (C05).

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Quelle est la position de `ORDER BY` dans l'ordre d'exécution ?

2. Pourquoi LIMIT sans ORDER BY est-il « non déterministe » ?
3. Quelle est la différence entre tri par alias et tri par position ?
4. Que fait NULLS LAST ?
5. Pourquoi la pagination OFFSET est-elle lente sur les grosses tables ?

Réponses :

1. 7^e position : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
2. Parce que sans ORDER BY, l'ordre des lignes est arbitraire (celui du stockage interne). Deux exécutions successives peuvent retourner des lignes différentes.
3. Le tri par alias utilise le nom déclaré dans le SELECT (par exemple ORDER BY ca_total DESC). Le tri par position utilise le numéro de la colonne (par exemple ORDER BY 2 DESC, qui trie la 2^e colonne du SELECT). Le tri par alias est lisible et robuste ; le tri par position est fragile (ajouter une colonne au SELECT change le sens).
4. NULLS LAST place les valeurs NULL à la fin du tri. Le défaut en DuckDB/PostgreSQL est déjà NULLS LAST en ASC, mais préciser est une bonne pratique pour la portabilité.
5. Parce que OFFSET m parcourt les m + n premières lignes puis en jette m. Sur 1 million de lignes, un OFFSET proche de la fin parcourt la quasi-totalité des lignes avant d'en afficher 10.

Module M07.C04 — Agréger : COUNT, SUM, AVG, MIN, MAX, ROUND

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C02 (WHERE) ; M07.C03 (ORDER BY, LIMIT).

L'idée du chapitre. Jusqu'ici, nos requêtes renvoyaient des **lignes** : une vente, un client, un produit. C04 change d'échelle : **réduire N lignes à un chiffre**. « Quel est le chiffre d'affaires total ? » « Quel est le panier moyen ? » « Quel est le plus gros montant ? » Ce sont des questions d'**agrégation** : COUNT, SUM, AVG, MIN, MAX, complétées par ROUND. Le chapitre couvre les cinq fonctions d'agrégat, le comptage de valeurs distinctes, le comportement particulier des NULL (ignorés par tous les agrégats sauf COUNT(*)), et la règle d'or qui prépare C05 : **une colonne non agrégée dans SELECT exige GROUP BY**. Le fil rouge est la base `commercial.duckdb` : on y mesure un CA brut de **7 908 259 731 FCFA**, un panier moyen de **158 140 FCFA**, et un montant minimum de **763 FCFA**.

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes (CA brut 7 908 259 731 FCFA), 1 200 clients, 5 modes de paiement (Espèces : 20 175 ventes ; Carte Bancaire : 12 366 ; Mobile Money : 12 322 ; Credit 30j : 2 593 ; Virement : 2 552), et des montants TTC compris entre **763** et **594 363 FCFA**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **compter** les lignes avec COUNT(*), les valeurs non nulles avec COUNT(colonne), les valeurs distinctes avec COUNT(DISTINCT colonne) ;
- **totaliser** avec SUM, **moyenner** avec AVG, extraire le **minimum** et le **maximum** avec MIN / MAX ;
- **arrondir** un résultat avec ROUND(valeur, nb_decimales) ;
- **expliquer pourquoi** les NULL disparaissent des agrégats (sauf COUNT(*)) ;
- **combinaison** plusieurs agrégats dans une même requête pour produire une ligne de synthèse ;
- **identifier l'erreur** « colonne non agrégée sans GROUP BY » et la résoudre (anticipation de C05).

2. Pourquoi cette notion est importante

- Presque toute question métier est une question d'agrégat : « combien de ventes ? », « quel CA ? », « quel est le panier moyen ? », « quelle est la plus grosse facture ? ». C'est le passage de **l'exploration de données** (C01-C03) à **la mesure** (C04-C05).
- Le **comportement des NULL** dans les agrégats est l'une des sources d'erreur les plus classiques : AVG qui sous-estime, COUNT qui diverge de COUNT(*), SUM qui renvoie NULL sur une colonne tout entière vide.
- La **règle colonne non agrégée / GROUP BY** est la frontière entre C04 (une ligne de synthèse sur toute la table) et C05 (une ligne de synthèse par groupe). La comprendre maintenant évite les erreurs les plus fréquentes du module.

3. Explication simple — l'agrégat est une pliuse de lignes

Imaginez un tas de 50 008 tickets de caisse sur un bureau. La question « quel est le montant minimum ? » ne demande pas de lire les tickets un par un et de les recopier : on **scanne** le tas, on retient un seul ticket — le plus petit. La question « quel est le total ? » demande d'**additionner** — mais, encore une fois, un seul chiffre sort du tas.

C'est exactement ce que fait un agrégat : il **consomme N lignes** et **produit 1 valeur**. La requête

```
SELECT SUM(montant_ttc) AS ca_total
FROM vente;
```

lit les 50 008 lignes de `vente`, additionne la colonne `montant_ttc`, et renvoie **une seule ligne contenant un seul chiffre** : 7 908 259 731 FCFA (arrondi au franc). Le mot-clé `SELECT` n'a pas changé de syntaxe ; c'est le **comportement** qui change : dès qu'une fonction d'agrégat apparaît, le résultat est **réduit à une ligne** (sauf `GROUP BY`, C05).

Définition. Une **fonction d'agrégat** — aggregate function — est une fonction qui prend un ensemble de valeurs (les lignes d'un groupe) en entrée et produit une **unique valeur** en sortie. Les cinq agrégats standard sont `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. Ils s'écrivent `fonction(arg)`, par exemple `SUM(montant_ttc)`.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
agrégat	fonction qui consomme N valeurs et produit 1 valeur.	l'utiliser sur une colonne texte avec <code>SUM</code> (erreur de type).
COUNT(*)	nombre de lignes (tout contenu, <code>NULL</code> inclus).	le confondre avec <code>COUNT(colonne)</code> .
COUNT(DISTINCT ...)	nombre de valeurs différentes d'une colonne.	croire qu'il compte les lignes (il compte les valeurs uniques).
SUM	somme des valeurs non nulles.	attendre 0 sur une colonne 100 % <code>NULL</code> (on obtient <code>NULL</code>).
AVG	moyenne arithmétique = <code>SUM</code> / <code>COUNT(non-NULL)</code> .	croire que les <code>NULL</code> comptent dans le dénominateur (ils ne comptent pas).
MIN / MAX	valeur minimale / maximale (montants, dates, textes).	les appliquer sur un texte en oubliant l'ordre lexicographique.
ROUND(x, n)	arrondi de x à n décimales (défaut 0).	arrondir trop tôt (fausse les totaux en aval).
ligne de synthèse	résultat unique d'agrégats sans <code>GROUP BY</code> .	y mélanger une colonne non agrégée (erreur <code>GROUP BY</code>).
comptage conditionnel	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code> : compter « les X qui ».	l'écrire en <code>WHERE</code> quand on veut plusieurs mesures à la fois.
null-skipping	les <code>NULL</code> sont ignorés par les agrégats (sauf <code>COUNT(*)</code>).	oublier que <code>SUM</code> renvoie <code>NULL</code> si tout est <code>NULL</code> .

5. Cours approfondi

5.1 COUNT(*) : compter les lignes

```
SELECT COUNT(*) AS nb_ventes
FROM vente;
```

COUNT(*) compte **toutes les lignes** du résultat, qu'elles contiennent des NULL ou non. Sur la base de travail : **50 008** lignes. C'est l'agrégat de vérification de premier rang : il confirme qu'une table « contient bien » le volume attendu.

5.2 COUNT(colonne) : compter les valeurs non nulles

```
SELECT COUNT(montant_ttc) AS nb_montants
FROM vente;
```

COUNT(colonne) ne compte **que les valeurs non nulles** de la colonne. Sur la base de travail, montant_ttc n'a aucun NULL : le résultat est **50 008**, identique à COUNT(*). La différence n'apparaît que quand des NULL existent : c'est alors un indicateur de qualité de données (« combien de lignes ont un montant ? »).

5.3 COUNT(DISTINCT colonne) : compter les valeurs différentes

```
SELECT COUNT(DISTINCT id_client) AS nb_clients_actifs
FROM vente;
```

COUNT(DISTINCT ...) compte les **valeurs différentes**. Sur les 50 008 ventes, il y a **1 200** clients distincts — chaque client a au moins une vente (la plupart, beaucoup). Cette écriture est le geste standard pour répondre à « combien de clients ont réellement acheté ? ».

Définition. Le **comptage distinct** — distinct count — compte les valeurs uniques d'une colonne au lieu de toutes les lignes. COUNT(DISTINCT id_client) répond à « combien de clients différents ? » ; il ignore les répétitions. C'est plus coûteux qu'un COUNT(*) car le moteur doit mémoriser (ou trier) les valeurs déjà vues.

5.4 SUM et AVG : totaliser et moyenner

```
SELECT SUM(montant_ttc) AS ca_total,
       AVG(montant_ttc) AS panier_moyen
FROM vente;
```

Le total s'élève à **7 908 259 731 FCFA** (arrondi au franc) — c'est le **CA brut**, retours inclus (piège P2, rappelé en C02). Le panier moyen vaut **158 140 FCFA** (arrondi). AVG est défini par le moteur comme SUM(colonne) / COUNT(colonne) : les NULL sont exclus du dénominateur comme du numérateur, ce qui évite la sous-estimation.

Définition. Le **panier moyen** — average order value (AOV) — est le montant moyen d'une vente, calculé par AVG(montant_ttc). Sur la base de travail : **158 140 FCFA**. Il est l'un des indicateurs commerciaux de premier rang, car il mesure la valeur d'une transaction type sans dépendre du volume de ventes.

5.5 MIN et MAX : les bornes

```
SELECT MIN(montant_ttc) AS montant_min,
       MAX(montant_ttc) AS montant_max
FROM vente;
```

Le montant minimum est de **763 FCFA** (arrondi) et le maximum de **594 363 FCFA** (plafond du générateur, mesuré en C03). MIN et MAX fonctionnent aussi sur les dates (MIN(date_vente) = première vente) et les textes (ordre lexicographique) — pas seulement sur les montants.

5.6 ROUND : arrondir pour présenter

Les agrégats produisent souvent des décimales inutiles en FCFA. ROUND sert à **présenter** le chiffre :

```
SELECT ROUND(SUM(montant_ttc))      AS ca_total,
       ROUND(AVG(montant_ttc))      AS panier_moyen,
       ROUND(AVG(montant_ttc), 2)   AS panier_precis
FROM vente;
```

ca_total = 7 908 259 731 ; panier_moyen = 158 140 ; panier_precis conserve deux décimales. Règle : **ROUND(valeur, n) arrondit à n décimales** ; sans n, l'arrondi est à l'entier.

5.7 Les NULL et les agrégats : la règle unique

Règle. Tous les agrégats **ignorent les NULL de leur colonne**, à une exception : COUNT(*), qui compte les lignes (pas les valeurs).

Agrégat	NULL dans la colonne	Colonne 100 % NULL
COUNT(*)	ignoré (compte les lignes)	nombre de lignes
COUNT(colonne)	ignoré (ne compte que non-NULL)	0
SUM(colonne)	ignoré (non compté)	NULL
AVG(colonne)	ignoré (non compté)	NULL
MIN / MAX	ignoré (non comparé)	NULL

Sur la base de travail, **aucune colonne n'a de NULL** (le générateur les exclut, comme noté en C03 §5.6) : SUM, AVG, MIN, MAX donnent donc des chiffres partout. La règle reste essentielle dès qu'on importe des données réelles — et c'est l'objet de la démonstration qui suit.

Définition. L'ignorance des NULL — null-skipping — est le comportement par défaut des agrégats SUM, AVG, MIN, MAX (et COUNT(colonne)) : les valeurs NULL ne participent ni à la somme, ni au dénombrement, ni à la comparaison. Seule l'écriture COUNT(*) dénombre les lignes quelle que soit leur contenu.

Attention. SUM et AVG renvoient NULL (pas 0) sur une colonne entièrement NULL. Si votre rapport affiche NULL là où il devrait afficher 0, c'est ce cas. La parade : COALESCE(SUM(x), 0) — fonction de M07.C06.

5.8 Position dans l'ordre d'exécution

Rappel de C01/C03 : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. Sans GROUP BY (ce chapitre), les agrégats du SELECT agissent sur **l'ensemble** des lignes survivantes après WHERE :

```
SELECT SUM(montant_ttc) AS ca_2025
FROM vente
WHERE date_vente >= '2025-01-01'
      AND date_vente < '2026-01-01';
```

WHERE filtre d'abord (le filtre est appliqué **avant** l'agrégation) : le CA 2025 ne tient compte que des ventes de 2025. C'est la différence fondamentale avec HAVING (C05) : WHERE filtre **les lignes**, HAVING filtre **les groupes**.

5.9 Plusieurs agrégats dans une même requête

Rien n'interdit de combiner les agrégats dans un seul SELECT — le résultat reste **une seule ligne** :

```
SELECT COUNT(*)           AS nb_ventes,
       COUNT(DISTINCT id_client) AS nb_clients,
       ROUND(SUM(montant_ttc))  AS ca_total,
       ROUND(AVG(montant_ttc))  AS panier_moyen,
       ROUND(MIN(montant_ttc))  AS montant_min,
       ROUND(MAX(montant_ttc))  AS montant_max
FROM vente;
```

nb_ventes	nb_clients	ca_total	panier_moyen	montant_min	montant_max
50 008	1 200	7 908 259 731	158 140	763	594 363

Une ligne, six chiffres : c'est le **format de synthèse** d'un rapport exécutif.

5.10 Agrégat conditionnel : compter « les X qui »

Question : « combien de ventes par Espèces, et combien par Carte Bancaire, **en une seule ligne** ? » Le filtre WHERE ne suffit pas (il ne permet qu'un filtre unique) ; on utilise une expression conditionnelle dans l'agrégat :

```
SELECT SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS nb_especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS nb_cartes
FROM vente;
```

nb_especes = **20 175**, nb_cartes = **12 366**. Ce motif — agréguer une expression CASE — est un outil central de C05 (comptages conditionnels).

À retenir. Le motif SUM(CASE WHEN ... THEN 1 ELSE 0 END) transforme un comptage conditionnel en **agrégat ordinaire** : il s'additionne, se combine avec d'autres agrégats, et ne produit toujours qu'une ligne. C'est l'outil des « tableaux de bord » (une ligne, plusieurs colonnes de mesures).

5.11 Ce que ce chapitre ne couvre pas

- GROUP BY (une ligne de synthèse **par groupe**) : C05.
- HAVING (filtre sur les groupes) : C05.
- COALESCE, NULLIF (traiter les NULL avant l'agrégation) : C06.
- Les fonctions de fenêtre (SUM() OVER (PARTITION BY ...)) : M11.

6. Exemple concret — la requête qui répond à une question métier

La directrice financière demande : « Quel est notre CA brut, et quel est le panier moyen sur l'ensemble de l'historique ? »

Étape 1 — Écrire la requête.

```
SELECT ROUND(SUM(montant_ttc)) AS ca_brut,
       ROUND(AVG(montant_ttc)) AS panier_moyen,
       COUNT(*)                AS nb_ventes
FROM vente;
```

Étape 2 — Prédire la sortie. FROM vente charge 50 008 lignes ; aucun WHERE ; pas de GROUP BY ; SUM, AVG, COUNT réduisent le tout à **une ligne de synthèse**.

Étape 3 — Exécuter. Sortie : ca_brut = 7 908 259 731, panier_moyen = 158 140, nb_ventes = 50 008.

Étape 4 — Comparer. Trois chiffres, une ligne, tous cohérents : $7\,908\,259\,731 / 50\,008 \approx 158\,140$. Si le panier moyen ne redonnait pas le CA divisé par le nombre de ventes, il faudrait revoir la requête (filtre oublié, table erronée).

Conseil professionnel. Vérifiez **toujours la cohérence croisée** des agrégats d'une ligne de synthèse : $CA / \text{nombre de ventes} \approx \text{panier moyen}$, $MAX \geq AVG \geq MIN$. Une incohérence (par exemple $MIN > MAX$) signale une erreur de requête ou de type — elle se repère en 5 secondes, bien avant le rapport.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Combien de ventes au total ?

```
SELECT COUNT(*) AS nb_ventes
FROM vente;
```

→ **50 008** (dont 8 doublons et 200 retours inclus, cf. 6_pieges.md).

7.2 Question 2 — Quel est le CA brut total ?

```
SELECT ROUND(SUM(montant_ttc)) AS ca_brut
FROM vente;
```

→ **7 908 259 731 FCFA**.

7.3 Question 3 — Quel est le panier moyen ?

```
SELECT ROUND(AVG(montant_ttc)) AS panier_moyen
FROM vente;
```

→ **158 140 FCFA**.

7.4 Question 4 — Bornes des montants

```
SELECT ROUND(MIN(montant_ttc)) AS min_m,
       ROUND(MAX(montant_ttc)) AS max_m
FROM vente;
```

→ 763 / 594 363 FCFA (le maximum est le plafond du générateur, mesuré en C03).

7.5 Question 5 — Répartition par mode de paiement, en une ligne

```
SELECT SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS carte,
       SUM(CASE WHEN id_mode = 3 THEN 1 ELSE 0 END) AS mobile,
       SUM(CASE WHEN id_mode = 4 THEN 1 ELSE 0 END) AS credit_30j,
       SUM(CASE WHEN id_mode = 5 THEN 1 ELSE 0 END) AS virement
FROM vente;
```

→ 20 175 / 12 366 / 12 322 / 2 593 / 2 552. Les Espèces représentent plus de 40 % des ventes ; le crédit 30j et le virement sont marginaux (5 % chacun).

7.6 Question 6 — CA par mode de paiement, en une ligne

```
SELECT ROUND(SUM(CASE WHEN id_mode = 1 THEN montant_ttc ELSE 0 END)) AS ca_especes,
       ROUND(SUM(CASE WHEN id_mode = 2 THEN montant_ttc ELSE 0 END)) AS ca_carte
FROM vente;
```

→ 3 184 889 570 FCFA (Especes) / 1 944 001 846 FCFA (Carte). Le même motif CASE s'applique au **montant** au lieu du comptage : c'est la forme générale du « tableau de bord en une ligne ».

À retenir. Le motif `SUM(CASE WHEN ... THEN montant ELSE 0 END)` est la forme générale du **tableau de bord en une ligne** : chaque colonne est une mesure conditionnelle. C'est exactement ce que `GROUP BY` (C05) généralisera en plusieurs lignes, une par groupe.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	<code>SELECT id_client, COUNT(*) FROM vente;</code> sans <code>GROUP BY</code>	erreur « must appear in the GROUP BY clause »	<code>GROUP BY id_client</code> (C05), ou retirer la colonne non agrégée.
2	Attendre 0 de <code>SUM</code> sur une colonne vide	résultat NULL dans le rapport	<code>COALESCE(SUM(x), 0)</code> .
3	Confondre <code>COUNT(*)</code> et <code>COUNT(colonne)</code>	décompte inexplicé	<code>COUNT(*)</code> = lignes ; <code>COUNT(col)</code> = valeurs non nulles.
4	Arrondir trop tôt	totaux incohérents en aval	<code>ROUND</code> uniquement dans le dernier <code>SELECT</code> affiché.
5	<code>SUM</code> sur une colonne texte	erreur de type	<code>SUM</code> / <code>AVG</code> sur des colonnes numériques uniquement.
6	<code>COUNT(DISTINCT a, b)</code> sur 2 colonnes	syntaxe refusée en DuckDB/PostgreSQL	<code>COUNT(DISTINCT a b)</code> ou table temporaire (C06+).

Attention. L'erreur 1 est **la plus fréquente** du module : on demande « une valeur par client » sans dire comment grouper. Le message du moteur est explicite (« must appear in the GROUP BY clause or be used in an aggregate function ») : dès qu'une colonne non agrégée est dans `SELECT`, le `GROUP BY` est obligatoire (C05).

9. Bonnes pratiques professionnelles

- **Un agrégat, un alias** : `SUM(montant_ttc) AS ca_total` — jamais de colonne sans nom dans un rapport.
- **ROUND à la présentation** : précision pleine en amont, arrondi au dernier `SELECT`.
- **Vérifier la cohérence croisée** : `CA / nb_ventes ≈ panier moyen` ; `MIN ≤ AVG ≤ MAX`.
- **COUNT(*) en contrôle** : avant tout agrégat, vérifier le volume de lignes attendues (sur vente : 50 008).
- **Documenter les NULL** : une colonne avec `NULL` doit être annotée dans le dictionnaire de données (M06), sinon `SUM/AVG` surprennent.
- **Commenter les comptages conditionnels** : « Espèces = id_mode 1, cf. mode_paiement ».

10. Exercice guidé — la synthèse mensuelle (15 min, /10)

Énoncé. Le chef de produit demande une **ligne de synthèse** de l'activité : nombre de ventes, nombre de clients actifs, CA total, panier moyen, montant minimum, montant maximum — le tout arrondi au franc, en une seule requête.

Grille de correction (/10).

Critère	Points
6 agrégats bien choisis (<code>COUNT(*)</code> , <code>COUNT(DISTINCT id_client)</code> , <code>SUM</code> , <code>AVG</code> , <code>MIN</code> , <code>MAX</code>)	3
6 alias lisibles	1
5 <code>ROUND</code> sur les montants	1
Une seule ligne produite (pas de colonne non agrégée)	2
<code>FROM vente</code> correct	1
Valeurs conformes (50 008 / 1 200 / 7 908 259 731 / 158 140 / 763 / 594 363)	2

Corrigé.

```
SELECT COUNT(*)           AS nb_ventes,
       COUNT(DISTINCT id_client) AS nb_clients,
       ROUND(SUM(montant_ttc)) AS ca_total,
       ROUND(AVG(montant_ttc)) AS panier_moyen,
       ROUND(MIN(montant_ttc)) AS min_m,
       ROUND(MAX(montant_ttc)) AS max_m
FROM vente;
```

→ 50 008 / 1 200 / 7 908 259 731 / 158 140 / 763 / 594 363.

11. Exercices autonomes

Exercice 1 — CA total, arrondi

Écrire la requête qui renvoie le CA brut total de `vente`, arrondi au franc.

Exercice 2 — Panier moyen à deux décimales

Exercice 3 — Nombre de clients distincts dans `vente`

Exercice 4 — Bornes des montants (minimum et maximum arrondis)

Exercice 5 — `COUNT(*)` VS `COUNT(montant_ttc)`

Expliquer pourquoi les deux renvoient 50 008 sur cette base, et dans quel cas ils divergeraient.

Exercice 6 — Comptage conditionnel par mode

Compter, dans une même requête, le nombre de ventes Espèces (id_mode 1) et Virement (id_mode 5).

Exercice 7 — CA par deux modes, en une ligne

Produire, en une ligne : le CA total Espèces et le CA total Carte Bancaire (arrondis).

Exercice 8 — Diagnostic d'erreur

Que renvoie ce `SELECT`, et pourquoi ?

```
SELECT SUM(plafond_credit) FROM client;
```

12. Correction détaillée

Exercice 1.

```
SELECT ROUND(SUM(montant_ttc)) AS ca_total  
FROM vente;
```

→ 7 908 259 731 FCFA.

Exercice 2.

```
SELECT ROUND(AVG(montant_ttc), 2) AS panier_moyen  
FROM vente;
```

→ le panier moyen avec deux décimales (arrondi de la moyenne exacte).

Exercice 3.

```
SELECT COUNT(DISTINCT id_client) AS nb_clients  
FROM vente;
```

→ 1 200.

Exercice 4.

```
SELECT ROUND(MIN(montant_ttc)) AS min_m,  
       ROUND(MAX(montant_ttc)) AS max_m  
FROM vente;
```

→ 763 / 594 363 FCFA.

Exercice 5. Sur `vente`, `montant_ttc` n'a aucun `NULL` : les deux comptent donc les mêmes lignes (50 008). Ils divergeraient dès qu'une ligne de `vente` aurait `montant_ttc = NULL` : `COUNT(*)` la compterait, `COUNT(montant_ttc)` non.

Exercice 6.

```
SELECT SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS nb_especes,
       SUM(CASE WHEN id_mode = 5 THEN 1 ELSE 0 END) AS nb_virement
FROM vente;
```

→ 20 175 / 2 552.

Exercice 7.

```
SELECT ROUND(SUM(CASE WHEN id_mode = 1 THEN montant_ttc ELSE 0 END)) AS ca_especes,
       ROUND(SUM(CASE WHEN id_mode = 2 THEN montant_ttc ELSE 0 END)) AS ca_carte
FROM vente;
```

→ 3 184 889 570 / 1 944 001 846 FCFA.

Exercice 8. Sur le socle M07, les 1 200 clients ont tous un plafond (le générateur exclut les `NULL`) : la somme additionne donc les 1 200 plafonds. Sur des données réelles contenant des `NULL`, les clients sans plafond seraient **ignorés** par `SUM` ; et si aucun client n'avait de plafond, le résultat serait `NULL` — pas 0. Pour afficher 0, écrire `COALESCE(SUM(plafond_credit), 0)`.

13. Mini-projet — « La fiche de synthèse commerciale » (30 min)

Produire la **fiche de synthèse** de la base commerciale : une requête unique qui renvoie, en une ligne, les 8 indicateurs ci-dessous. Vérifier chaque chiffre à la main (relever les valeurs dans un tableur puis comparer).

Indicateur	Valeur attendue (arrondie)
Nombre de ventes	50 008
Clients actifs	1 200
CA brut total (FCFA)	7 908 259 731
Panier moyen (FCFA)	158 140
Montant minimum (FCFA)	763
Montant maximum (FCFA)	594 363
Ventes Espèces	20 175
Ventes Virement	2 552

Corrigé.

```
SELECT COUNT(*)                AS nb_ventes,
       COUNT(DISTINCT id_client) AS nb_clients,
       ROUND(SUM(montant_ttc))   AS ca_total,
       ROUND(AVG(montant_ttc))  AS panier_moyen,
       ROUND(MIN(montant_ttc))  AS min_m,
       ROUND(MAX(montant_ttc))  AS max_m,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS ventes_especes,
       SUM(CASE WHEN id_mode = 5 THEN 1 ELSE 0 END) AS ventes_virement
FROM vente;
```

Critères de validation. 8 colonnes, 1 ligne ; les 8 valeurs ci-dessus ; aucun avertissement `GROUP BY` (toutes les colonnes du `SELECT` sont agrégées).

Dans les faits. PostgreSQL accepte la même syntaxe avec le même résultat ; MySQL, en mode `ONLY_FULL_GROUP_BY` activé par défaut depuis la version 5.7, renvoie la même erreur qu'ici si une colonne non agrégée est présente, mais en mode permissif (ancien défaut) il **devine** une valeur arbitraire — la requête passe, et le chiffre est faux sans aucun signal. C'est pourquoi le mode permissif MySQL est à éviter en production.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 6 outils de mesure : `COUNT` (lignes, valeurs, distinct), `SUM` (total), `AVG` (moyenne), `MIN/MAX` (bornes), `ROUND` (présentation), plus le motif `SUM(CASE WHEN ...)` pour les comptages conditionnels. La règle d'or — colonne non agrégée $\Rightarrow$ `GROUP BY` — est la porte d'entrée de C05.

Outil	Usage	Syntaxe	Piège
<code>COUNT(*)</code>	nombre de lignes	<code>COUNT(*)</code>	confondre avec <code>COUNT(col)</code>
<code>COUNT(col)</code>	valeurs non nulles	<code>COUNT(montant_ttc)</code>	diverger de <code>COUNT(*)</code> si <code>NULL</code>
<code>COUNT(DISTINCT col)</code>	valeurs uniques	<code>COUNT(DISTINCT id_client)</code>	plus coûteux qu'un <code>COUNT(*)</code>
<code>SUM / AVG</code>	total / moyenne	<code>SUM(montant_ttc)</code>	renvoyer <code>NULL</code> si colonne 100 % <code>NULL</code>
<code>MIN / MAX</code>	bornes	<code>MAX(date_vente)</code>	ordre lexicographique sur texte
<code>ROUND(x, n)</code>	arrondi	<code>ROUND(AVG(montant_ttc), 2)</code>	arrondir trop tôt
<code>SUM(CASE ...)</code>	comptage / somme conditionnelle	<code>SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END)</code>	<code>ELSE 0</code> obligatoire

15. Résumé du chapitre

- **Un agrégat consomme N lignes et produit 1 valeur** : `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, plus `ROUND` pour la présentation.
- `COUNT(*)` compte les lignes ; `COUNT(colonne)` les valeurs non nulles ; `COUNT(DISTINCT colonne)` les valeurs différentes.
- **Les NULL sont ignorés** par `SUM`, `AVG`, `MIN`, `MAX`, `COUNT(colonne)` ; sur une colonne 100 % `NULL`, `SUM/AVG/MIN/MAX` renvoient `NULL` (pas 0) — parade : `COALESCE(agrégat, 0)`.
- **WHERE filtre les lignes avant l'agrégation** ; `HAVING` (C05) filtrera les groupes.
- **Colonne non agrégée dans SELECT sans GROUP BY = erreur** : c'est la règle d'or qui sépare C04 (synthèse globale) de C05 (synthèse par groupe).
- Le motif `SUM(CASE WHEN ...)` produit les « tableaux de bord en une ligne » : une mesure conditionnelle par colonne.

16. À retenir

- **Un agrégat = N lignes $\rightarrow$ 1 valeur.** Toujours un alias par agrégat.
- `COUNT(*)` $\neq$ `COUNT(colonne)` dès qu'un `NULL` existe.
- **NULL ignorés** : `SUM/AVG` renvoient `NULL` sur une colonne vide — `COALESCE` pour forcer 0.

- **ROUND à la présentation** : précision pleine en amont, arrondi au dernier SELECT.
- **Colonne non agrégée sans GROUP BY = erreur** : la règle d'or, porte d'entrée de C05.

À retenir. $AVG(colonne) = SUM(colonne) / COUNT(colonne)$: le dénominateur exclut les NULL. C'est la définition même du panier moyen — et la raison pour laquelle une moyenne « qui baisse sans raison » est souvent une entrée de NULL en moins.

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Quelle est la différence entre `COUNT(*)` et `COUNT(colonne)` ?
2. Que renvoie `SUM(colonne)` si toutes les valeurs de `colonne` sont NULL ?
3. `AVG` est-il défini comment par le moteur ?
4. Pourquoi `SELECT id_client, COUNT(*) FROM vente;` est-il une erreur ?
5. À quelle position de l'ordre d'exécution sont appliqués les agrégats du `SELECT` sans `GROUP BY` ?

Réponses :

1. `COUNT(*)` dénombre **les lignes** (tout contenu, NULL inclus) ; `COUNT(colonne)` dénombre **les valeurs non nulles** de la colonne. Les deux divergent dès qu'un NULL existe dans la colonne.
2. NULL — pas 0. L'agrégat n'a rien à sommer : l'absence de valeur s'exprime par NULL. La parade est `COALESCE(SUM(colonne), 0)`.
3. $AVG(colonne) = SUM(colonne) / COUNT(colonne)$ — le dénominateur exclut les NULL, donc la moyenne est calculée sur les seules valeurs présentes.
4. Parce qu'`id_client` n'est **pas agrégé** et n'est **pas groupé** : le moteur ne sait pas quelle valeur de `id_client` retourner pour la ligne de synthèse. Il faut `GROUP BY id_client` (C05).
5. Après le `WHERE` (le filtre de lignes) : l'agrégat sans `GROUP BY` consomme l'ensemble des lignes survivantes à ce stade, avant le `SELECT` et le `DISTINCT`.

Module M07.C05 — Résumer par groupe : GROUP BY, HAVING, comptages conditionnels

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C02 (WHERE) ; M07.C03 (ORDER BY, LIMIT) ; M07.C04 (agrégats).

L'idée du chapitre. C04 a réduit **toute la table** à une ligne de synthèse. C05 va plus loin : **une ligne de synthèse par groupe**. « Le CA de chaque magasin. » « Le nombre de ventes de chaque rayon. » « Le panier moyen par magasin et par année. » C'est **GROUP BY** — la clause la plus utile de l'analyse métier. Le chapitre couvre la syntaxe de base, le regroupement par **clé** (et non par libellé — le piège du rayon doublé), le regroupement multi-colonnes, **HAVING** (le filtre qui s'applique aux groupes, pas aux lignes), la distinction **WHERE / HAVING**, et les **comptages conditionnels** (**SUM(CASE WHEN ...)** combiné à **GROUP BY**) qui produisent les tableaux de bord croisés. Le fil rouge est la base `commercial.duckdb` : on y mesure le CA de chacun des **5 magasins** (entre 1 564 937 883 et 1 596 813 264 FCFA) et de chacune des **8 catégories** (entre 699 533 080 et 1 236 006 465 FCFA).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes répartis sur 5 magasins (de 9 873 à 10 099 ventes chacun), 8 catégories (de 4 764 à 7 166 ventes chacune), 2 années (2025 : 24 920 ventes ; 2026 : 25 088), et 5 modes de paiement. Le CA total reste 7 908 259 731 FCFA.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **regrouper** un résultat par une colonne avec **GROUP BY** et produire une ligne de synthèse par groupe ;
- **regrouper par clé** (id unique) et comprendre pourquoi regrouper par **libellé** est dangereux quand le libellé est doublé ;
- **regrouper sur plusieurs colonnes** (**GROUP BY a, b**) pour des coupes croisées ;
- **filtrer les groupes** avec **HAVING** (et distinguer **WHERE** de **HAVING**) ;
- **combiner** **ORDER BY** et **LIMIT** avec **GROUP BY** pour un « top N par groupe » ;
- **compter conditionnellement** avec **SUM(CASE WHEN ... THEN 1 ELSE 0 END)** dans un **GROUP BY**.

2. Pourquoi cette notion est importante

- « Le CA par magasin », « les ventes par catégorie », « le panier moyen par ville et par année » : ce sont **les questions d'un tableau de bord**. Sans **GROUP BY**, on ne sort d'une table qu'une seule ligne de synthèse (C04) ou des lignes brutes (C01–C03).
- Le **regroupement par libellé doublé** est un piège silencieux : le total est juste, mais les lignes se fondent les unes dans les autres sans avertissement. Le socle M07 en contient un de démonstration : la colonne `rayon` de `categorie` porte deux fois « Alimentaire », deux fois « Bricolage », deux fois « Jardinage » (les `sous_categorie` distinguent).
- La frontière **WHERE / HAVING** est l'un des points les plus demandés en entretien : **WHERE** filtre les **lignes avant** le regroupement ; **HAVING** filtre les **groupes après** l'agrégation.

3. Explication simple — GROUP BY est une plieuse par tiroirs

C04 : un tas de tickets, on scanne le tas, un seul chiffre sort. C05 : on met des **tiroirs** sur le bureau — un tiroir par magasin, un par rayon, un par année. Chaque ticket va dans son tiroir (selon sa valeur dans la colonne de regroupement). Puis on **applique la même plieuse à chaque tiroir** : un chiffre par tiroir.

Concrètement, `GROUP BY id_magasin` sur 50 008 ventes produit **5 lignes** (une par magasin), chaque ligne portant les agrégats calculés **sur les seules ventes de ce magasin**. Le total des 5 lignes de `SUM(montant_ttc)` redonne le CA global de C04 (7 908 259 731 FCFA) — c'est le **contrôle de cohérence** à faire systématiquement.

Définition. La clause `GROUP BY` — grouper par — subdivise les lignes du résultat en **groupes** selon les valeurs de la colonne (ou des colonnes) citées : les lignes qui partagent la même valeur forment un groupe, et chaque fonction d'agrégat du `SELECT` est calculée **séparément sur chaque groupe**. Le résultat contient une ligne par groupe.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
GROUP BY	une ligne de synthèse par groupe (par valeur distincte de la colonne).	oublier que chaque colonne du <code>SELECT</code> doit être agrégée ou dans le <code>GROUP BY</code> .
groupe	l'ensemble des lignes qui partagent la même valeur de la clé de regroupement.	confondre le nombre de groupes avec le nombre de lignes.
clé de regroupement	la colonne (ou les colonnes) du <code>GROUP BY</code> ; idéale : une clé unique (<code>id_*</code>).	regrouper par un libellé doublé (les groupes se fondent).
HAVING	filtre appliqué aux groupes , après les agrégats.	l'écrire dans <code>WHERE</code> (interdit : on ne filtre pas un agrégat là-dedans).
WHERE	filtre appliqué aux lignes , avant le regroupement.	mettre un agrégat dans <code>WHERE</code> (erreur d'exécution).
regroupement multi-colonnes	<code>GROUP BY a, b</code> : un groupe par couple (a, b).	croire que l'ordre des colonnes du <code>GROUP BY</code> change le total (il ne change que la granularité d'affichage).
comptage conditionnel	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code> dans un <code>GROUP BY</code> .	oublier le <code>ELSE 0</code> (les <code>NULL</code> ne s'additionnent pas).
coupe croisée	regroupement sur 2 dimensions (magasin × année).	multiplier les colonnes sans comprendre que chaque ligne = une combinaison.

5. Cours approfondi

5.1 La syntaxe de base

```
SELECT id_magasin,
       COUNT(*)      AS nb_ventes,
       ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY id_magasin
ORDER BY ca DESC;
```

id_magasin	nb_ventes	ca
4	10 061	1 596 813 264

id_magasin	nb_ventes	ca
1	10 099	1 595 223 423
2	10 033	1 578 987 708
5	9 942	1 572 297 452
3	9 873	1 564 937 883

Cinq groupes, cinq lignes. Chaque `nb_ventes` et `ca` est calculé **sur les seules ventes du magasin**. Le total des 5 `nb_ventes` redonne 50 008 ; le total des 5 `ca` redonne 7 908 259 731. Le magasin 4 (Bobo Sarfalao) est en tête, mais l'écart entre le 1^e et le 5^e est faible (moins de 2 %).

Règle de syntaxe : **toute colonne du SELECT doit être soit agrégée, soit dans le GROUP BY**. `id_magasin` est dans le GROUP BY ; `COUNT(*)` et `SUM(montant_ttc)` sont agrégés. C'est la règle d'or de C04, généralisée.

5.2 Regrouper par clé, pas par libellé

Le socle M07 contient 8 catégories, mais seulement **5 libellés** de `rayon` : « Alimentaire » apparaît deux fois (Boissons et Conserves), « Bricolage » deux fois (Outillage et quincaillerie), « Jardinage » deux fois (Plantes et Outillage).

Regrouper par `c.rayon` (le libellé) **fond les deux catégories** qui partagent le même rayon :

```
SELECT c.rayon, COUNT(*) AS nb_ventes
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.rayon
ORDER BY nb_ventes DESC;
```

rayon	nb_ventes
Bricolage	12 418
Jardinage	12 037
Alimentaire	11 930
Hygiene	6 825
Decoration	6 798

Regrouper par `c.id_categorie` (la clé) garde **8 groupes distincts**. Le choix entre les deux est un **choix métier** (vu « par rayon » ou vu « par catégorie »), pas une coïncidence : si vous voulez 8 lignes, regroupez par la clé.

Définition. Une **clé de regroupement** — grouping key — est la colonne (ou la liste de colonnes) citée dans GROUP BY. Elle doit être **unique par groupe** : on regroupe idéalement par une clé (`id_categorie`) plutôt que par un libellé (`rayon`), car un libellé doublé fusionne silencieusement deux groupes différents.

Attention. Regrouper par un libellé doublé ne provoque **aucune erreur** : le moteur fusionne les groupes sans avertissement, et le total général reste juste. Le bug est donc silencieux — on ne le voit qu'en comptant les lignes du résultat (5 au lieu de 8). Vérifiez toujours le nombre de groupes attendus.

5.3 Regrouper sur plusieurs colonnes

GROUP BY `a, b` produit un groupe par **couple** (a, b). Coupe croisée magasin × année :

```
SELECT id_magasin,
       EXTRACT(YEAR FROM date_vente) AS annee,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(montant_ttc)) AS ca
FROM vente
WHERE id_magasin = 4
GROUP BY 1, 2
ORDER BY annee;
```

id_magasin	annee	nb_ventes	ca
4	2025	5 064	797 593 324
4	2026	4 997	799 219 940

Deux groupes (le couple magasin 4 × 2025, le couple magasin 4 × 2026). Sur les 5 magasins et 2 années, la requête sans `WHERE` produirait **10 groupes**.

5.4 HAVING — filtrer les groupes

`WHERE` ne voit que des lignes ; `HAVING` voit des **groupes déjà agrégés**. Question : « quelles catégories ont plus de 6 500 ventes ? »

```
SELECT c.id_categorie, c.rayon, c.sous_categorie, COUNT(*) AS nb_ventes
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie, c.rayon, c.sous_categorie
HAVING COUNT(*) > 6500
ORDER BY nb_ventes DESC;
```

id_categorie	rayon	sous_categorie	nb_ventes
1	Alimentaire	Boissons	7 166
4	Bricolage	Quincaillerie	7 037
8	Hygiene	Cosmétique	6 825
7	Decoration	Interieur	6 798

Quatre groupes passent le filtre. Le `HAVING` peut aussi s'appliquer à `SUM`, `AVG`, `MIN`, `MAX` — tout agrégat :

```
SELECT id_magasin, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY id_magasin
HAVING ROUND(SUM(montant_ttc)) > 1550000000
ORDER BY ca DESC;
```

Ici les **5 magasins** passent (le plus petit CA, 1 564 937 883 FCFA, dépasse le seuil) : le seuil est bas relativement à la répartition quasi uniforme du générateur.

Définition. La clause `HAVING` — ayant — est le filtre qui s'applique aux **groupes** après le calcul des agrégats. Elle s'écrit entre `GROUP BY` et `ORDER BY`, et seule elle permet de filtrer sur une valeur agrégée (`HAVING COUNT(*) > 10`). `WHERE` filtre les lignes **avant** le regroupement et ne peut pas référencer d'agrégat.

5.5 WHERE et HAVING ensemble

On peut (et on doit souvent) combiner les deux : `WHERE` réduit les lignes **avant** le regroupement, `HAVING` filtre les groupes **après** :

```
SELECT id_magasin, COUNT(*) AS nb_ventes_2026
FROM vente
WHERE EXTRACT(YEAR FROM date_vente) = 2026
GROUP BY id_magasin
HAVING COUNT(*) > 4900
ORDER BY nb_ventes_2026 DESC;
```

Ordre d'exécution (rappel C01/C04) : FROM → WHERE (lignes) → GROUP BY (groupes) → HAVING (groupes) → SELECT → ORDER BY → LIMIT.

À retenir. WHERE = filtre sur les **lignes** (avant agrégation, pas d'agrégat autorisé) ; HAVING = filtre sur les **groupes** (après agrégation, agrégats autorisés). Si vous filtrez sur un total, c'est HAVING.

5.6 Comptages conditionnels dans un GROUP BY

Le motif de C04 §5.10 (SUM(CASE WHEN ...)) combine parfaitement avec GROUP BY : chaque groupe porte plusieurs colonnes de mesures conditionnelles. Répartition Espèces / Carte **par magasin** :

```
SELECT id_magasin,
       COUNT(*) AS total,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS cartes
FROM vente
GROUP BY id_magasin
ORDER BY id_magasin;
```

id_magasin	total	especes	cartes
1	10 099	4 109	2 502
2	10 033	4 043	2 532
3	9 873	3 973	2 414
4	10 061	4 093	2 469
5	9 942	3 957	2 449

Chaque ligne est un magasin ; chaque colonne especes / cartes est un sous-comptage. C'est le schéma des **tableaux de bord croisés** : lignes = dimension 1, colonnes = mesures conditionnelles.

Définition. Le **comptage conditionnel** — conditional count — est le motif SUM(CASE WHEN condition THEN 1 ELSE 0 END) : il convertit « compter les lignes qui vérifient la condition » en expression agrégable. Combiné à GROUP BY, il produit une colonne de comptage par condition, sur chaque groupe.

5.7 COUNT(DISTINCT ...) dans un groupe

Les agrégats de C04 fonctionnent aussi **par groupe**. Nombre de clients distincts qui ont acheté dans chaque magasin :

```
SELECT id_magasin, COUNT(DISTINCT id_client) AS nb_clients
FROM vente
GROUP BY id_magasin
ORDER BY id_magasin;
```

→ 1 199, 1 199, 1 200, 1 200, 1 199. Quatre magasins servent 1 199 clients distincts, deux en servent 1 200 : la couverture est quasi totale (sur 1 200 clients au total).

5.8 Ce que ce chapitre ne couvre pas

- Les **fenêtres** (`SUM() OVER (PARTITION BY ...)`) : agrégat par groupe **sans** réduire les lignes) : M11.
- Les **sous-requêtes** et **CTE** (pré-agrégations nommées) : C08.
- L'optimisation des gros `GROUP BY` (index, plans d'exécution) : M11.

6. Exemple concret — la requête qui répond à une question métier

Le directeur des magasins demande : « Le CA de chaque magasin, le nombre de ventes, et le panier moyen — mais seulement les magasins qui font plus de 9 900 ventes. »

Étape 1 — Écrire la requête.

```
SELECT m.id_magasin,
       m.nom,
       COUNT(*)           AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca,
       ROUND(AVG(v.montant_ttc)) AS panier_moyen
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.id_magasin, m.nom
HAVING COUNT(*) > 9900
ORDER BY ca DESC;
```

Étape 2 — Prédire la sortie. `FROM vente` charge 50 008 lignes ; `JOIN magasin` apporte le nom ; `GROUP BY` forme 5 groupes ; `HAVING COUNT(*) > 9900` élimine les groupes de moins de 9 900 ventes. À l'œil, on prédit que 2 magasins sortent (les deux moins vendus) — il resterait 3 magasins.

Étape 3 — Exécuter. Quatre lignes : Bobo Sarfalao (10 061 ventes, 1 596 813 264 FCFA), Ouaga Centre (10 099, 1 595 223 423), Ouaga Patte d'Oie (10 033, 1 578 987 708), Koudougou (9 942, 1 572 297 452). Le panier moyen varie entre 157 379 et 158 713 FCFA.

Étape 4 — Comparer. La prédiction était fausse sur un point : Koudougou (9 942) passe le seuil de 9 900, seul Bobo Centre (9 873) sort. Le contrôle de cohérence (la somme des CA des 5 magasins = 7 908 259 731) reste le filet de sécurité.

Conseil professionnel. Prévoyez **toujours le cas limite** du seuil de `HAVING` : un groupe exactement **au seuil** passe ou non selon `>` ou `>=`. Avant de publier, vérifiez le nombre de groupes attendus — ici 5 au lieu de 4 — et ajustez le seuil ou l'opérateur. Une surprise de seuil est la première cause de « le rapport a 4 lignes au lieu de 5 ».

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — CA par magasin

```
SELECT id_magasin, COUNT(*) AS nb, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY id_magasin
ORDER BY ca DESC;
```

→ 5 lignes : 1 596 813 264 / 1 595 223 423 / 1 578 987 708 / 1 572 297 452 / 1 564 937 883 FCFA.

7.2 Question 2 — CA par catégorie (la clé, pas le rayon)

```
SELECT c.id_categorie, c.rayon, c.sous_categorie,
       COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie, c.rayon, c.sous_categorie
ORDER BY ca DESC;
```

→ 8 lignes, du plus gros au plus petit : Decoration (1 236 006 465), Hygiene (1 163 845 193), Jardinage-Plantes (1 160 443 887), Alimentaire-Boissons (1 026 270 112), Jardinage-Outillage (950 960 303), Bricolage-Quincaillerie (939 911 483), Alimentaire-Conserves (731 289 207), Bricolage-Outillage (699 533 080) FCFA.

7.3 Question 3 — Panier moyen par magasin

```
SELECT id_magasin, ROUND(AVG(montant_ttc)) AS panier_moyen
FROM vente
GROUP BY id_magasin
ORDER BY panier_moyen DESC;
```

→ 158 713 (magasin 4) / 158 507 (3) / 158 147 (5) / 157 959 (1) / 157 379 (2) FCFA. L'écart total est de 1 334 FCFA, soit moins de 1 % : le panier moyen est quasi uniforme entre magasins.

7.4 Question 4 — Le CA par année

```
SELECT EXTRACT(YEAR FROM date_vente) AS annee,
       COUNT(*) AS nb, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY 1
ORDER BY annee;
```

→ 2025 : 24 920 ventes, 3 925 215 671 FCFA ; 2026 : 25 088 ventes, 3 983 044 060 FCFA. Le 2026 dépasse le 2025 de 57 828 389 FCFA (+1,5 %).

7.5 Question 5 — Catégories actives (HAVING)

```
SELECT c.rayon, COUNT(*) AS nb_ventes
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.rayon
HAVING COUNT(*) > 6500
ORDER BY nb_ventes DESC;
```

→ 4 rayons : Bricolage 12 418, Jardinage 12 037, Alimentaire 11 930, Hygiene 6 825, Decoration 6 798 — non, 5 lignes passent car Decoration (6 798) dépasse 6 500. Le HAVING garde 5 groupes sur 5 (seuil bas) ; le seuil 12 000 ne en garderait que 2 (Bricolage, Jardinage).

7.6 Question 6 — Espèces et cartes par magasin

```
SELECT id_magasin,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS cartes
FROM vente
GROUP BY id_magasin
ORDER BY especes DESC;
```

→ Magasin 1 : 4 109 Espèces / 2 502 Carte ; magasin 4 : 4 093 / 2 469 ; magasin 2 : 4 043 / 2 532 ; magasin 3 : 3 973 / 2 414 ; magasin 5 : 3 957 / 2 449.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	Colonne non agrégée et non groupée dans <code>SELECT</code>	erreur « must appear in the GROUP BY clause »	l'ajouter au <code>GROUP BY</code> ou l'agréger.
2	Filtrer un agrégat dans <code>WHERE</code>	erreur « WHERE clause cannot contain aggregates »	utiliser <code>HAVING</code> .
3	Regrouper par un libellé doublé	nombre de lignes inexplicé (5 au lieu de 8)	regrouper par la clé (<code>id_*</code>).
4	Oublier le <code>ELSE 0</code> dans un comptage conditionnel	colonnes de comptage trop faibles (les <code>NULL</code> ne s'additionnent pas)	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code> .
5	<code>ORDER BY</code> sur un alias inconnu du <code>GROUP BY</code>	ordre arbitraire ou erreur selon le SGBD	ordonner par l'agrégat ou la clé.
6	Attendre que <code>GROUP BY</code> supprime les doublons	doublons persistants	<code>GROUP BY</code> réduit par groupe ; pour supprimer des lignes, <code>DISTINCT (C01)</code> .

Attention. L'erreur 3 est la plus sournoise : **aucun message d'erreur**, un total juste, mais des groupes fusionnés. Le nombre de lignes du résultat est le premier contrôle : si vous attendez 8 catégories et vous en voyez 5, vérifiez la clé de regroupement.

Attention. `ORDER BY` dans une requête `GROUP BY` : on trie les **groupes**. `ORDER BY ca` (alias de l'agrégat) est accepté par DuckDB et PostgreSQL ; pour la portabilité maximale, répéter l'expression (`ORDER BY SUM(montant_ttc) DESC`) ou trier par position.

9. Bonnes pratiques professionnelles

- **Regrouper par la clé** (`id_*`), joindre le libellé dans le `SELECT` et le `GROUP BY`.
- **Vérifier le nombre de groupes** avant toute lecture : 5 magasins, 8 catégories, 10 couples magasin×année.
- **Contrôle de cohérence** : la somme des agrégats par groupe doit redonner l'agrégat global (C04).
- **Un seuil de `HAVING` doit être documenté** : « > 9 900 ventes » est une décision métier, pas une constante anonyme.
- **CASE conditionnel : toujours un `ELSE 0`** dans les comptages (jamais de `NULL` dans une addition).
- **Nommer les colonnes** : `AS nb_ventes`, `AS ca`, `AS panier_moyen` — le libellé du tableau de bord est celui des alias.

10. Exercice guidé — le tableau de bord des 5 magasins (15 min, /10)

Énoncé. Produire une ligne par magasin (5 lignes, triées par CA décroissant) avec : le nom du magasin, le nombre de ventes, le CA arrondi, le panier moyen arrondi, le nombre de ventes Espèces, le nombre de ventes Carte.

Grille de correction (/10).

Critère	Points
GROUP BY m.id_magasin, m.nom (clé + libellé)	2
COUNT(*), ROUND(SUM(...)), ROUND(AVG(...))	2
2 comptages conditionnels avec ELSE 0	2
JOIN magasin correct	1
ORDER BY ca DESC (alias de l'agrégat)	1
5 lignes, triées : 1 596 813 264 en tête	2

Corrigé.

```
SELECT m.id_magasin,
       m.nom,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca,
       ROUND(AVG(v.montant_ttc)) AS panier_moyen,
       SUM(CASE WHEN v.id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN v.id_mode = 2 THEN 1 ELSE 0 END) AS cartes
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.id_magasin, m.nom
ORDER BY ca DESC;
```

→ 5 lignes : Bobo Sarfalao (10 061 / 1 596 813 264 / 158 713 / 4 093 / 2 469), Ouaga Centre (10 099 / 1 595 223 423 / 157 959 / 4 109 / 2 502), Ouaga Patte d'Oie (10 033 / 1 578 987 708 / 157 379 / 4 043 / 2 532), Koudougou (9 942 / 1 572 297 452 / 158 147 / 3 957 / 2 449), Bobo Centre (9 873 / 1 564 937 883 / 158 507 / 3 973 / 2 414).

11. Exercices autonomes

Exercice 1 — CA par catégorie

Une ligne par catégorie (8 lignes) : id, rayon, sous-catégorie, nombre de ventes, CA arrondi, triée par CA décroissant.

Exercice 2 — Panier moyen par magasin

Une ligne par magasin : panier moyen arrondi, triée décroissante.

Exercice 3 — Le CA par année

Une ligne par année : nombre de ventes, CA arrondi.

Exercice 4 — Catégories actives

Les catégories (par clé) ayant plus de 6 500 ventes, avec leur nombre de ventes.

Exercice 5 — Rayon vs clé

Compter le nombre de groupes produits par `GROUP BY c.rayon` et par `GROUP BY c.id_categorie` sur la même requête de base. Expliquer l'écart.

Exercice 6 — Espèces et cartes par magasin

Deux comptages conditionnels par magasin, triés par nombre de ventes Espèces décroissant.

Exercice 7 — Couper par magasin et par année

Le nombre de ventes du magasin 4 en 2025 et en 2026 (2 lignes).

Exercice 8 — Diagnostic d'erreur

Que fait cette requête, et pourquoi est-elle une erreur ?

```
-- ERREUR ATTENDUE : un agrégat dans WHERE est interdit (cf. HAVING)
SELECT id_magasin, COUNT(*) AS nb
FROM vente
WHERE COUNT(*) > 9900;
```

12. Correction détaillée

Exercice 1.

```
SELECT c.id_categorie, c.rayon, c.sous_categorie,
       COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie, c.rayon, c.sous_categorie
ORDER BY ca DESC;
```

→ 8 lignes (cf. §7.2).

Exercice 2.

```
SELECT id_magasin, ROUND(AVG(montant_ttc)) AS panier_moyen
FROM vente
GROUP BY id_magasin
ORDER BY panier_moyen DESC;
```

→ 158 713 / 158 507 / 158 147 / 157 959 / 157 379 FCFA.

Exercice 3.

```
SELECT EXTRACT(YEAR FROM date_vente) AS annee,
       COUNT(*) AS nb, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY 1
ORDER BY annee;
```

→ 2025 : 24 920 / 3 925 215 671 ; 2026 : 25 088 / 3 983 044 060 FCFA.

Exercice 4.

```
SELECT c.id_categorie, COUNT(*) AS nb
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie
HAVING COUNT(*) > 6500
ORDER BY nb DESC;
```

→ 4 catégories : 1 (7 166), 4 (7 037), 8 (6 825), 7 (6 798).

Exercice 5. `GROUP BY c.rayon` produit **5 groupes** (Bricolage 12 418, Jardinage 12 037, Alimentaire 11 930, Hygiène 6 825, Decoration 6 798) ; `GROUP BY c.id_categorie` produit **8 groupes** (cf. §7.2). L'écart (3) vient des trois rayons doublés : Alimentaire, Bricolage, Jardinage — chaque libellé en fusionne deux.

Exercice 6. Cf. §7.6 → Magasin 1 : 4 109 / 2 502, puis 4 : 4 093 / 2 469, 2 : 4 043 / 2 532, 3 : 3 973 / 2 414, 5 : 3 957 / 2 449.

Exercice 7.

```
SELECT EXTRACT(YEAR FROM date_vente) AS annee, COUNT(*) AS nb
FROM vente
WHERE id_magasin = 4
GROUP BY 1
ORDER BY annee;
```

→ 2025 : 5 064 ; 2026 : 4 997.

Exercice 8. C'est une erreur : `COUNT(*)` est un **agrégat**, et `WHERE` ne peut pas en contenir (il filtre les lignes, avant le regroupement). Le bon écrit est `GROUP BY id_magasin HAVING COUNT(*) > 9900`.

13. Mini-projet — « Le rapport mensuel multi-dimensionnel » (45 min)

Produire, dans un seul fichier SQL exécuté sur `commercial.duckdb`, les 4 requêtes du rapport mensuel. Chaque requête est vérifiée à la main (nombre de lignes + cohérence des totaux).

Requête	Lignes attendues
R1 : CA, nb de ventes, panier moyen par magasin (trié CA décroissant)	5
R2 : CA par catégorie (par clé), trié CA décroissant	8
R3 : CA par année (2 lignes)	2
R4 : Espèces / Carte / Mobile Money (id 3) par magasin	5

Contrôles de cohérence. (i) La somme des CA de R1 = 7 908 259 731 FCFA. (ii) La somme des CA de R2 = 7 908 259 731 FCFA. (iii) La somme des CA de R3 = 7 908 259 731 FCFA. (iv) La somme des `nb_ventes` de R1 = 50 008.

Corrigé.

```
-- R1
SELECT m.nom, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca,
       ROUND(AVG(v.montant_ttc)) AS panier_moyen
FROM vente v JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.nom
ORDER BY ca DESC;

-- R2
SELECT c.rayon, c.sous_categorie, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.rayon, c.sous_categorie
ORDER BY ca DESC;

-- R3
SELECT EXTRACT(YEAR FROM date_vente) AS annee, COUNT(*) AS nb,
       ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY 1
ORDER BY annee;

-- R4
SELECT id_magasin,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS cartes,
       SUM(CASE WHEN id_mode = 3 THEN 1 ELSE 0 END) AS mobile
FROM vente
GROUP BY id_magasin
ORDER BY id_magasin;
```

Dans les faits. PostgreSQL exécute ces 4 requêtes avec le même résultat ; le `GROUP BY m.nom` (R1) est accepté car `nom` est fonctionnellement dépendant de `id_magasin` — DuckDB l’accepte aussi, mais la pratique portable reste de citer la clé (`GROUP BY m.id_magasin, m.nom`). MySQL en mode `ONLY_FULL_GROUP_BY` exigera que toute colonne du `SELECT` soit agrégée ou groupée ; en mode permissif, il devine — à éviter (cf. M07.C04 §13).

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 4 outils de regroupement : `GROUP BY` (une ligne de synthèse par groupe), le **regroupement par clé** (piège du libellé doublé), `HAVING` (filtre sur les groupes), et les **comptages conditionnels** (`SUM(CASE WHEN ... THEN 1 ELSE 0 END)` par groupe). La distinction `WHERE` / `HAVING` est le réflexe à acquérir.

Outil	Usage	Syntaxe	Piège
<code>GROUP BY col</code>	une ligne par groupe	<code>GROUP BY id_magasin</code>	colonne du <code>SELECT</code> ni agrégée ni groupée
Clé de regroupement	regrouper sans fusionner	<code>GROUP BY c.id_categorie</code>	le libellé doublé (rayon) fusionne
Multi-colonnes	coupe croisée	<code>GROUP BY id_magasin, annee</code>	chaque ligne = une combinaison
<code>HAVING</code>	filtrer les groupes	<code>HAVING COUNT(*) > 9900</code>	l’écrire dans <code>WHERE</code> (erreur)
<code>SUM(CASE ...)</code>	comptage conditionnel	<code>SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END)</code>	oublier <code>ELSE 0</code>
Contrôle	cohérence totaux	des somme des groupes = total global	ne pas vérifier le nombre de groupes

15. Résumé du chapitre

- **GROUP BY** subdivise les lignes en groupes ; chaque agrégat est calculé par groupe ; le résultat contient une ligne par groupe.
- **Regrouper par la clé** (`id_*`), pas par le libellé : un libellé doublé fusionne silencieusement deux groupes (5 rayons au lieu de 8 catégories).
- **GROUP BY a, b** = une ligne par couple (a, b) : les coupes croisées.
- **WHERE** filtre les lignes (avant agrégation) ; **HAVING** filtre les groupes (après agrégation). Filtrer sur un total $\Rightarrow$ **HAVING**.
- Le **comptage conditionnel** `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` produit les tableaux de bord croisés : une colonne de mesure par condition, par groupe.
- **Contrôle systématique** : nombre de groupes + somme des agrégats = total global.

16. À retenir

- **Une ligne par groupe** : **GROUP BY** est la plieuse par tiroirs.
- **Clé, pas libellé** : le total juste ne suffit pas — le nombre de groupes doit être juste.
- **WHERE lignes / HAVING groupes** : si vous filtrez sur un total, c'est **HAVING**.
- **ELSE 0 obligatoire** dans les comptages conditionnels.
- **Cohérence** : somme des CA par groupe = CA global (7 908 259 731 FCFA).

À retenir. Le **HAVING** est le seul endroit où l'on peut **comparer un agrégat à un seuil** : `HAVING SUM(montant_ttc) > X`. Tout le reste de la requête (**WHERE**, **ORDER BY**, **LIMIT**) travaille sur des lignes ou sur des alias — jamais directement sur un agrégat dans **WHERE**.

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Combien de lignes produit `SELECT id_magasin, COUNT(*) FROM vente GROUP BY id_magasin;` ?
2. Pourquoi regrouper par `c.rayon` au lieu de `c.id_categorie` est-il risqué ici ?
3. Où écrire `COUNT(*) > 9900` : dans **WHERE** ou dans **HAVING** ? Pourquoi ?
4. Que fait `SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END)` dans un **GROUP BY** ?
5. Quel est le contrôle de cohérence à faire sur un **GROUP BY** ?

Réponses :

1. **5 lignes** — une par groupe, donc une par magasin (5 magasins dans le socle).
2. Parce que le libellé `rayon` est **doublé** (Alimentaire, Bricolage, Jardinage) : le regroupement fusionnerait silencieusement 8 catégories en 5 groupes, sans erreur.
3. Dans **HAVING** : `COUNT(*)` est un agrégat, et seuls les groupes sont filtrés par **HAVING**. Dans **WHERE**, c'est une erreur (« WHERE clause cannot contain aggregates »).
4. Il compte, **dans chaque groupe**, les lignes où `id_mode = 1` (les ventes Espèces) : une colonne de comptage conditionnel par groupe.
5. Deux contrôles : (i) le **nombre de groupes** est conforme (5 magasins, 8 catégories) ; (ii) la **somme des agrégats** par groupe redonne l'agrégat global (CA total 7 908 259 731 FCFA).

Module M07.C06 — Catégoriser : CASE, COALESCE, NULLIF

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 3 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C04 (agrégats) ; M07.C05 (GROUP BY).

L'idée du chapitre. Jusqu'ici, on a **lu** et **compté** les données. C06 les **transforme** : « segmenter les clients en 3 classes », « étiqueter chaque vente selon sa taille », « remplacer les absences par un texte neutre ». C'est le travail de **catégorisation** : CASE (la brique de logique conditionnelle du SELECT), COALESCE (le premier non nulle), NULLIF (le comparateur qui produit du NULL). Ces trois fonctions sont les outils de tout le travail de préparation de données : créer une colonne d'étiquette, normaliser un libellé, gérer les absences sans casser les agrégats. Le fil rouge est la base `commercial.duckdb` : on y segmente les 1 200 clients en 3 types (particuliers 850, entreprises 229, comptoirs 121), on étiquette les 50 008 ventes en 3 tranches de montant, et on mesure le CA des produits **inactifs** (471 206 245 FCFA — un produit « retiré » peut très bien avoir été vendu hier).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 1 200 clients (850 particuliers, 229 entreprises, 121 comptoirs ; Ouagadougou 512, Bobo-Dioulasso 441, Koudougou 247), 50 008 ventes (tranches : 12 753 / 21 509 / 15 746), 380 produits (362 actifs, 18 inactifs). Aucune colonne du socle ne contient de NULL (le générateur les exclut) : les fonctions COALESCE / NULLIF sont donc démontrées sur des expressions, pas sur des données réellement absentes.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **construire une colonne d'étiquette** avec CASE WHEN ... THEN ... ELSE ... END (forme simple et forme de recherche) ;
- **segmenter** un jeu de données en classes (types de clients, tranches de montant) et agréger sur ces classes ;
- **gérer les absences** avec COALESCE(x, défaut) et comprendre pourquoi un NULL dans une somme « disparaît » ;
- **produire du NULL volontairement** avec NULLIF(a, b) et l'exploiter (filtrer les zéros, détecter des égalités) ;
- **combinaison** ces fonctions dans un SELECT (et dans un GROUP BY) pour préparer des tableaux de bord lisibles.

2. Pourquoi cette notion est importante

- **CASE est l'outil n° 1 de la transformation de données** : 90 % des « colonnes créées » dans un entrepôt sont des CASE (segmentation, étiquettes, règles métier). C'est aussi la brique des comptages conditionnels de C04/C05 (SUM(CASE WHEN ...)).
- **COALESCE est le remède aux NULL** : un champ vide qui casse une somme, un libellé null qui devient « (non renseigné) » dans le rapport. Le maîtriser, c'est ne plus être surpris par les NULL en production.

- **NULLIF est le miroir de COALESCE** : il transforme une valeur en `NULL` quand elle est égale à une référence — un filtre de zéros sans `WHERE`, une détection d'égalité.

3. Explication simple — CASE est le carrefour du SELECT

Le `SELECT` de C01 copiait des colonnes. C06 lui ajoute une **logique de carrefour** : selon la valeur d'une colonne, on sort un libellé différent. Comme à un carrefour routier, on regarde une plaque (la condition) et on prend une sortie (le libellé) :

- plaque « `type_client = 'particulier'` » → sortie « Particulier » ;
- plaque « `type_client = 'entreprise'` » → sortie « Entreprise » ;
- plaque « `type_client = 'comptoir'` » → sortie « Comptoir » ;
- sinon → sortie « Autre » (le `ELSE`).

`COALESCE` et `NULLIF` sont les deux faces de la gestion des **absences** : `COALESCE(x, défaut)` dit « si `x` est absent, prends défaut » ; `NULLIF(a, b)` dit « si `a` vaut `b`, rends absent (`NULL`) ; sinon rends `a` ».

Définition. L'expression **CASE** — expression conditionnelle — est la brique de logique conditionnelle du `SELECT`. Sa forme de recherche est `CASE WHEN cond1 THEN r1 WHEN cond2 THEN r2 ... ELSE r0 END` : les conditions sont testées **dans l'ordre**, la première qui passe choisit le résultat ; `ELSE` est la valeur de repli si aucune ne passe.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
CASE (recherche)	<code>CASE WHEN c1 THEN r1 ... ELSE r0 END</code> : un libellé par condition, testées dans l'ordre.	croire que les branches sont évaluées toutes en même temps (non, dans l'ordre, la 1 ^{re} qui passe gagne).
CASE (simple)	<code>CASE x WHEN v1 THEN r1 ... END</code> : compare <code>x</code> à une valeur.	le confondre avec la forme de recherche (syntaxes différentes).
ELSE	valeur de repli si aucune condition ne passe.	l'oublier (le résultat est <code>NULL</code> pour les non-couverts).
segment	classe de lignes partageant une étiquette <code>CASE</code> .	des segments qui se chevauchent (les <code>WHEN</code> sont exclusifs par construction).
COALESCE	<code>COALESCE(a, b, ...)</code> : la première valeur non <code>NULL</code> de la liste.	croire qu'il remplace les zéros (il ne regarde que les <code>NULL</code>).
NULLIF	<code>NULLIF(a, b)</code> : <code>NULL</code> si <code>a = b</code> , sinon <code>a</code> .	le retourner (c'est <code>COALESCE</code> qui retourne).
étiquette	la colonne de texte produite par un <code>CASE</code> .	l'utiliser dans un <code>WHERE</code> avant qu'elle existe (il faut la recalculer ou une vue).

5. Cours approfondi

5.1 CASE forme de recherche — segmenter les clients

```
SELECT id_client,  
       nom,  
       type_client,  
       CASE  
         WHEN type_client = 'particulier' THEN 'Particulier'  
         WHEN type_client = 'entreprise' THEN 'Entreprise'  
         WHEN type_client = 'comptoir'   THEN 'Comptoir'  
         ELSE 'Autre'  
       END AS segment  
FROM client  
ORDER BY segment, nom  
LIMIT 6;
```

La colonne `segment` est une **étiquette** : elle est calculée à la volée, elle n'existe pas dans la table. Les 3 `WHEN` couvrent exactement les 3 valeurs du socle (850 particuliers, 229 entreprises, 121 comptoirs) : le `ELSE 'Autre'` n'attrapera rien ici, mais il protège le code contre une 4^e valeur qui arriverait demain.

Attention. Les branches `WHEN` sont testées **dans l'ordre écrit**, et **la première qui passe gagne**. Un `CASE` où la condition la plus générale vient avant la plus restrictive rend les branches suivantes mortes. Ici les 3 `WHEN` sont mutuellement exclusifs (une valeur de `type_client` ne peut en valoir qu'une), donc l'ordre est sans effet — mais sur des conditions qui se chevauchent (des intervalles), l'ordre change le résultat.

5.2 CASE forme simple — le même, en plus court

Quand on compare **la même colonne** à plusieurs valeurs constantes, la forme simple est plus lisible :

```
SELECT type_client,  
       CASE type_client  
         WHEN 'particulier' THEN 'Particulier'  
         WHEN 'entreprise' THEN 'Entreprise'  
         WHEN 'comptoir'   THEN 'Comptoir'  
         ELSE 'Autre'  
       END AS segment  
FROM client  
LIMIT 3;
```

`CASE x WHEN v1 ...` est un sucre syntaxique de `CASE WHEN x = v1 ...`. Pour des conditions arbitraires (comparaisons, `BETWEEN`, `LIKE`), seule la forme de recherche convient.

5.3 CASE pour étiqueter des tranches — la segmentation numérique

La question classique : « combien de ventes petites, moyennes, grosses ? » On découpe `montant_ttc` en 3 tranches :

```

SELECT CASE
    WHEN montant_ttc < 50000 THEN 'a) moins de 50 000'
    WHEN montant_ttc < 200000 THEN 'b) 50 000 à 199 999'
    ELSE 'c) 200 000 et plus'
END AS tranche,
COUNT(*) AS nb_ventes,
ROUND(MIN(montant_ttc)) AS min_m,
ROUND(MAX(montant_ttc)) AS max_m
FROM vente
GROUP BY 1
ORDER BY tranche;

```

tranche	nb_ventes	min_m	max_m
a) moins de 50 000	12 753	763	49 964
b) 50 000 à 199 999	21 509	50 008	199 920
c) 200 000 et plus	15 746	200 017	594 363

Trois segments, disjoints, qui se recouvrent totalement ($12\,753 + 21\,509 + 15\,746 = 50\,008$). C'est le contrôle de toute segmentation : **la somme des segments redonne le total**. Remarquez l'astuce de la 2^e branche : `montant_ttc < 200000` suffit car la 1^{re} a déjà attrapé `< 50000` — les branches précédentes étant passées, on est sûr d'être au-dessus de 50 000.

À retenir. Dans un CASE d'intervalle, **on n'écrit que la borne haute** de chaque branche (`< 50000`, puis `< 200000`, sinon) : la borne basse est implicitement couverte par la branche précédente. C'est ce qui rend les tranches **disjointes** sans écrire `BETWEEN 50000 AND 199999` — et c'est ce qu'il faut vérifier en relecture.

Définition. Une **tranche** — bucket — est un intervalle de valeurs découpé par des CASE WHEN successifs (ici `< 50 000`, `< 200 000`, sinon). Les tranches sont **disjointes** (une ligne n'en vaut qu'une) et **exhaustives** (avec un ELSE, toute ligne est classée). La segmentation en tranches est la base des histogrammes et des « classements par taille ».

5.4 CASE combiné à GROUP BY — agréger sur les étiquettes

L'étiquette se réutilise dans le GROUP BY (via sa position, C03) ou en répétant l'expression. Le CA par segment de client :

```

SELECT CASE
    WHEN c.type_client = 'particulier' THEN 'Particulier'
    WHEN c.type_client = 'entreprise' THEN 'Entreprise'
    ELSE 'Comptoir'
END AS segment,
COUNT(*) AS nb_ventes,
ROUND(AVG(v.montant_ttc)) AS panier_moyen,
ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY 1
ORDER BY ca DESC;

```

segment	nb_ventes	panier_moyen	ca
Particulier	35 377	158 181	5 595 962 715
Entreprise	9 471	156 542	1 482 606 616
Comptoir	5 160	160 793	829 690 400

Le panier moyen des **comptoirs** (160 793 FCFA) est le plus élevé — ils achètent moins (5 160 ventes) mais plus gros. Le total des 3 ca redonne 7 908 259 731 FCFA (contrôle de cohérence de C05).

5.5 COALESCE — le premier non nulle

`COALESCE(a, b, c, ...)` renvoie la **première** valeur de la liste qui n'est pas `NULL`. C'est l'outil de **repli** :

```
-- Si plafond_credit est absent, afficher 0 (au lieu de NULL)
SELECT id_client, nom,
       COALESCE(plafond_credit, 0) AS plafond_sur
FROM client
ORDER BY id_client
LIMIT 3;
```

Sur le socle M07, `plafond_credit` n'a aucun `NULL` : `COALESCE(plafond_credit, 0)` renvoie donc toujours `plafond_credit`. Mais la démonstration est valide — et c'est le geste standard dès qu'une colonne réelle contient des absences.

Cas d'usage n° 2 : un **libellé de repli** dans un rapport :

```
SELECT nom,
       COALESCE(ville, '(non renseignée)') AS ville_affichee
FROM client
ORDER BY nom
LIMIT 3;
```

Un client sans ville n'affichera pas une case vide mais « (non renseignée) » — le rapport reste lisible.

Définition. La fonction **COALESCE** — coalescer — renvoie la **première** valeur de sa liste d'arguments qui n'est pas `NULL`. `COALESCE(x, défaut)` est le substitut portable de l'opérateur `??` (MySQL) et de `x IS NOT NULL THEN x` (un `CASE`) elle transforme une absence en valeur de repli. Elle ne regarde que les `NULL` — un zéro ou un texte vide sont des valeurs présentes, pas des absences.

Attention. `COALESCE` ne remplace **pas les zéros** et **pas les textes vides** : `COALESCE(0, 1)` renvoie 0, `COALESCE('', 'défaut')` renvoie '' (le tiret médian est une valeur, pas un `NULL`). Pour « 0 ou vide ⇒ défaut », il faut un `CASE` explicite.

5.6 NULLIF — produire du NULL volontairement

`NULLIF(a, b)` renvoie `NULL` **si** `a = b`, sinon `a`. C'est le miroir de `COALESCE` : là où `COALESCE` transforme un `NULL` en valeur, `NULLIF` transforme une valeur en `NULL`.

Usage n° 1 : **filtrer les zéros d'une somme** sans `WHERE` :

```
-- La marge d'un produit à prix nul : NULL au lieu de 0 (signal d'absence)
SELECT NULLIF(0, 0)      AS zero_devenu_null,
       NULLIF(5, 0)      AS cinq_garde_cinq,
       NULLIF('a', 'a')  AS egal_devenu_null;
```

→ `NULL`, 5, `NULL`. Les deux égalités produisent du `NULL`, l'inégalité garde la valeur.

Usage n° 2 : **combinaison COALESCE + NULLIF** pour « vider les zéros puis mettre un repli » (mêmes types des deux côtés du `COALESCE` — ici des textes) :

```
SELECT COALESCE(NULLIF('0', '0'), 'aucun') AS zero_vide_puis_repli,
       COALESCE(NULLIF('5', '0'), 'aucun') AS cinq_garde_cinq;
```

→ « aucun » (le « 0 » devient NULL, le COALESCE met le repli) / « 5 ». Sur des nombres, le repli doit être un nombre : COALESCE(NULLIF(0, 0), -1).

Définition. La fonction **NULLIF** — nullifier si égal — renvoie NULL quand ses deux arguments sont égaux, sinon le premier argument. NULLIF(a, 0) est le filtre de zéros portable : il transforme un zéro (valeur « pas de donnée ») en NULL (absence), ce qui permet ensuite à COALESCE de poser un repli. C’est le miroir exact de COALESCE : l’un sort des NULL, l’autre y entre.

5.7 Le CASE dans le WHERE — filtrer sur une étiquette

On ne peut pas filtrer sur une étiquette du SELECT (elle n’existe pas encore au moment du WHERE) : on **recalcule** la condition, ou on encapsule dans une vue / CTE (C08). Ici, la recalcul direct :

```
SELECT COUNT(*) AS nb_ventes_grosses
FROM vente
WHERE montant_ttc >= 200000;
```

→ 15 746 ventes (la tranche c). Le WHERE ne voit que des expressions sur les colonnes réelles ; l’étiquette du SELECT est calculée **après** (ordre d’exécution C01).

5.8 Ce que ce chapitre ne couvre pas

- Les **fonctions de texte** (UPPER, LOWER, TRIM, SUBSTR, LIKE) : M11.
- Les **vues** et **CTE** pour encapsuler une étiquette réutilisable : C08.
- L’optimisation des **CASE** imbriqués et des fonctions scalaires : M11.

6. Exemple concret — la requête qui répond à une question métier

La responsable fidélité demande : « Combien de ventes par type de client, et quel est le CA de chaque type ? Et si un type avait un libellé vide, que faudrait-il afficher ? »

Étape 1 — Écrire la requête (segmentation + COALESCE de repli).

```
SELECT COALESCE(CASE c.type_client
                 WHEN 'particulier' THEN 'Particulier'
                 WHEN 'entreprise' THEN 'Entreprise'
                 WHEN 'comptoir'   THEN 'Comptoir'
                 ELSE 'Autre'
                 END, '(inconnu)') AS segment,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY 1
ORDER BY ca DESC;
```

Étape 2 — Prédire la sortie. 3 segments (les 3 types du socle) ; le COALESCE de repli ne s’activera que si un type_client serait NULL (aucun ici). Particulier en tête (le plus de ventes).

Étape 3 — Exécuter. Particulier 35 377 ventes / 5 595 962 715 FCFA ; Entreprise 9 471 / 1 482 606 616 ; Comptoir 5 160 / 829 690 400.

Étape 4 — Comparer. 3 segments, aucun « (inconnu) » (pas de NULL dans le socle), total 7 908 259 731 FCFA. Le COALESCE est un filet de sécurité, pas un acteur ici.

Conseil professionnel. Dans une requête de production, **toute étiquette CASE qui peut tomber dans le vide doit avoir un ELSE**, et ce ELSE doit être un libellé explicite (« (inconnu) », « (non classé) ») plutôt qu'un NULL : un NULL dans une colonne de segment disparaît des GROUP BY lisibles et des tableaux de bord. Le COALESCE autour du CASE est le filet de dernier recours.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Les 3 types de clients

```
SELECT type_client, COUNT(*) AS nb_clients
FROM client
GROUP BY type_client
ORDER BY nb_clients DESC;
```

→ particulier 850, entreprise 229, comptoir 121.

7.2 Question 2 — Le CA par type de client

```
SELECT c.type_client,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY c.type_client
ORDER BY ca DESC;
```

→ particulier 5 595 962 715 FCFA, entreprise 1 482 606 616, comptoir 829 690 400.

7.3 Question 3 — Les 3 villes et leur CA

```
SELECT c.ville,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY c.ville
ORDER BY ca DESC;
```

→ Ouagadougou 21 290 ventes / 3 391 679 290 FCFA, Bobo-Dioulasso 18 364 / 2 895 549 769, Koudougou 10 354 / 1 621 030 672.

7.4 Question 4 — La segmentation en 3 tranches

```
SELECT CASE
       WHEN montant_ttc < 50000 THEN 'a) moins de 50 000'
       WHEN montant_ttc < 200000 THEN 'b) 50 000 à 199 999'
       ELSE 'c) 200 000 et plus'
END AS tranche,
       COUNT(*) AS nb_ventes
FROM vente
GROUP BY 1
ORDER BY tranche;
```

→ 12 753 / 21 509 / 15 746 (total 50 008).

7.5 Question 5 — Les ventes des comptoirs, par ville

```
SELECT c.ville, COUNT(*) AS nb_ventes
FROM vente v
JOIN client c ON c.id_client = v.id_client
WHERE c.type_client = 'comptoir'
GROUP BY c.ville
ORDER BY nb_ventes DESC;
```

→ Ouagadougou 2 621, Bobo-Dioulasso 1 534, Koudougou 1 005.

7.6 Question 6 — Le CA des produits inactifs (piège silencieux)

```
SELECT COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
WHERE p.aktif = FALSE;
```

→ 2 434 ventes, 471 206 245 FCFA. Les 18 produits **inactifs** ont tout de même généré cette CA (sur l'historique 2025-2026) : « inactif » signifie « plus proposé », pas « jamais vendu ». C'est le piège d'interprétation classique d'une colonne de statut.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	Oublier le <code>ELSE</code> dans un <code>CASE</code>	lignes non couvertes = <code>NULL</code> dans l'étiquette	toujours un <code>ELSE</code> explicite.
2	Branche <code>WHEN</code> trop générale en 1 ^{re} position	branches suivantes jamais atteintes (mortes)	conditions de la plus spécifique à la plus générale.
3	Filtrer sur une étiquette dans le <code>WHERE</code>	erreur ou résultat inattendu	recalculer la condition, ou vue/CTE (C08).
4	Attendre que <code>COALESCE</code> remplace un 0	le 0 passe tel quel	<code>COALESCE(NULLIF(x, 0), défaut)</code> pour vider les zéros.
5	Confondre <code>CASE x WHEN v</code> (simple) et <code>CASE WHEN c</code> (recherche)	syntaxe erronée	simple = comparaison d'une valeur ; recherche = conditions arbitraires.
6	Croire que « inactif » = « jamais vendu »	sous-estimer l'historique d'un statut	vérifier le CA réel par statut (cf. 7.6).

Attention. L'erreur 2 est la plus subtile : un `CASE` où la branche la plus générale est en premier **rend les autres branches mortes** sans erreur. Sur des intervalles, toujours du plus étroit au plus large, ou utiliser des bornes fermées explicites (`BETWEEN`).

Attention. Un `NULL` dans un `CASE` : `WHEN NULL = 'x'` est `NULL` (ni vrai ni faux), donc la branche ne passe pas. Pour tester l'absence, écrire `WHEN x IS NULL THEN` — jamais `WHEN x = NULL`.

9. Bonnes pratiques professionnelles

- **Toujours un `ELSE`** dans un `CASE` de segmentation, avec un libellé explicite.
- **L'ordre des `WHEN`** : du plus spécifique au plus général (surtout sur des intervalles).
- **Contrôler la segmentation** : la somme des segments redonne le total (ici 50 008).
- **`COALESCE` sur toute colonne affichée** qui peut être absente en production.

- **Documenter le statut** : « actif / inactif » doit être défini (plus proposé vs jamais vendu) — sinon les rapports mentent par omission.
- **Préférer une vue/CTE (C08)** quand l'étiquette `CASE` est réutilisée plusieurs fois.

10. Exercice guidé — le rapport de segmentation (15 min, /10)

Énoncé. Produire, par type de client (3 lignes, triées par CA décroissant) : le segment (étiquette lisible), le nombre de ventes, le panier moyen arrondi, le CA arrondi. Ajouter un `COALESCE` de repli « (inconnu) » autour du `CASE`.

Grille de correction (/10).

Critère	Points
<code>CASE</code> 3 branches + <code>ELSE</code>	2
<code>COALESCE(..., '(inconnu)')</code>	1
<code>COUNT(*)</code> , <code>ROUND(AVG(...))</code> , <code>ROUND(SUM(...))</code>	2
<code>JOIN client</code> + <code>GROUP BY 1</code>	2
<code>ORDER BY ca DESC</code>	1
3 lignes, Particulier en tête (5 595 962 715)	2

Corrigé. Cf. §6, étape 1. → Particulier 35 377 / 158 181 / 5 595 962 715 ; Entreprise 9 471 / 156 542 / 1 482 606 616 ; Comptoir 5 160 / 160 793 / 829 690 400.

11. Exercices autonomes

Exercice 1 — Les 3 types de clients

Compter les clients par `type_client`, trié décroissant.

Exercice 2 — Le CA par ville

Une ligne par ville : nombre de ventes, CA arrondi, trié par CA décroissant.

Exercice 3 — La segmentation en tranches

Les 3 tranches de montant (< 50 000 / 50 000–199 999 / ≥ 200 000) avec leur nombre de ventes. Vérifier que le total redonne 50 008.

Exercice 4 — `NULLIF` en solo

Quel est le résultat de `SELECT NULLIF(7, 7), NULLIF(7, 8), NULLIF('x', 'x');` ?

Exercice 5 — `COALESCE` + `NULLIF`

Réécrire « si `plafond_credit` vaut 0 ou est `NULL`, afficher 500 000 ; sinon le `plafond_credit` » avec `COALESCE` et `NULLIF`.

Exercice 6 — Comptoirs par ville

Les ventes des comptoirs, une ligne par ville, triées décroissant.

Exercice 7 — Le CA des produits inactifs

Nombre de ventes et CA des produits `actif = FALSE`.

Exercice 8 — Diagnostic d'erreur

Pourquoi cette requête est-elle une erreur, et comment la corriger ?

```
SELECT segment, COUNT(*)
FROM (
  SELECT CASE
    WHEN type_client = 'particulier' THEN 'Particulier'
    WHEN type_client = 'entreprise' THEN 'Entreprise'
    ELSE 'Comptoir'
  END AS segment
  FROM client
) t
GROUP BY segment;
```

12. Correction détaillée

Exercice 1.

```
SELECT type_client, COUNT(*) AS nb_clients
FROM client
GROUP BY type_client
ORDER BY nb_clients DESC;
```

→ particulier 850, entreprise 229, comptoir 121.

Exercice 2. Cf. §7.3 → Ouagadougou 21 290 / 3 391 679 290 ; Bobo-Dioulasso 18 364 / 2 895 549 769 ; Koudougou 10 354 / 1 621 030 672 FCFA.

Exercice 3. Cf. §7.4 → 12 753 / 21 509 / 15 746 ; total 50 008 ✓.

Exercice 4. `NULL`, `7`, `NULL` : les deux égalités (`7=7`, `'x'='x'`) produisent du `NULL` ; l'inégalité (`7≠8`) garde `7`.

Exercice 5.

```
SELECT COALESCE(NULLIF(plafond_credit, 0), 500000) AS plafond_final
FROM client
ORDER BY id_client
LIMIT 3;
```

`NULLIF(plafond_credit, 0)` transforme le `0` en `NULL` ; `COALESCE(..., 500000)` met le repli pour les `NULL` (et les `0`). Sur le socle, aucun plafond n'est `0` ni `NULL` : le `plafond_final` vaut toujours le `plafond_credit`.

Exercice 6. Cf. §7.5 → Ouagadougou 2 621, Bobo-Dioulasso 1 534, Koudougou 1 005.

Exercice 7. Cf. §7.6 → 2 434 ventes, 471 206 245 FCFA.

Exercice 8. Cette requête est en fait **correcte** (une sous-requête nommée `t` qui produit `segment`, puis un `GROUP BY segment` valide) — le piège est de chercher l'erreur alors qu'il n'y en a pas. Le cas qui **serait** une erreur, c'est de vouloir `GROUP BY segment` **sans** la sous-requête (l'alias du `SELECT` n'est pas groupable telle quelle dans tous les SGBD). La bonne pratique : encapsuler l'étiquette (sous-requête ici, vue ou CTE en C08) avant de la regrouper.

13. Mini-projet — « La grille de segmentation complète » (40 min)

Produire la **grille de segmentation** de la base : 4 requêtes, chacune vérifiée à la main.

Requête	Lignes attendues
G1 : segment par type de client (nb, panier moyen, CA)	3
G2 : tranche de montant (nb, min, max)	3
G3 : CA par ville (nb, CA)	3
G4 : statut produit (actif/inactif) : nb de produits, nb de ventes, CA	2

Contrôles de cohérence. (i) La somme des `nb_ventes` de G2 = 50 008. (ii) La somme des `ca` de G1 = 7 908 259 731 FCFA. (iii) G4 : actif 362 produits, inactif 18 ; le CA inactif = 471 206 245 FCFA.

Corrigé.

```
-- G1
SELECT COALESCE(CASE c.type_client
                WHEN 'particulier' THEN 'Particulier'
                WHEN 'entreprise' THEN 'Entreprise'
                WHEN 'comptoir'   THEN 'Comptoir'
                ELSE 'Autre'
                END, '(inconnu)') AS segment,
COUNT(*) AS nb,
ROUND(AVG(v.montant_ttc)) AS panier_moyen,
ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v JOIN client c ON c.id_client = v.id_client
GROUP BY 1
ORDER BY ca DESC;

-- G2
SELECT CASE
    WHEN montant_ttc < 50000 THEN 'a) moins de 50 000'
    WHEN montant_ttc < 200000 THEN 'b) 50 000 à 199 999'
    ELSE 'c) 200 000 et plus'
END AS tranche,
COUNT(*) AS nb,
ROUND(MIN(montant_ttc)) AS min_m,
ROUND(MAX(montant_ttc)) AS max_m
FROM vente
GROUP BY 1
ORDER BY tranche;

-- G3
SELECT c.ville, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v JOIN client c ON c.id_client = v.id_client
GROUP BY c.ville
ORDER BY ca DESC;

-- G4
SELECT p.actif,
COUNT(DISTINCT p.id_produit) AS nb_produits,
COUNT(*) AS nb_ventes,
ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v JOIN produit p ON p.id_produit = v.id_produit
GROUP BY p.actif
ORDER BY p.actif DESC;
```

Dans les faits. PostgreSQL et SQLite partagent cette syntaxe `CASE / COALESCE / NULLIF` (elles sont **standard SQL**) : la grille s'exécute telle quelle sur les deux. MySQL aussi, mais son opérateur `??` (équivalent `COALESCE` à 2 arguments) est un ajout non standard : pour la portabilité, écrire `COALESCE`.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 3 outils de transformation : `CASE` (la brique de segmentation, formes simple et recherche), `COALESCE` (le repli des `NULL`), `NULLIF` (le producteur de `NULL`). Ensemble, ils couvrent la quasi-totalité du travail de « création de colonnes d'étiquette » d'un entrepôt. Le `ELSE` obligatoire et le contrôle « somme des segments = total » sont les deux réflexes.

Outil	Usage	Syntaxe	Piège
<code>CASE WHEN ... END</code>	étiquette condition	par <code>CASE WHEN x < 50000 THEN 'petit' ELSE 'grand' END</code>	<code>ELSE</code> oublié, branches mortes
<code>CASE x WHEN v END</code>	étiquette valeur	par <code>CASE type WHEN 'pro' THEN 'Pro' END</code>	le confondre avec la forme recherche
<code>COALESCE</code>	repli des <code>NULL</code>	<code>COALESCE(ville, '(non renseignée)')</code>	ne remplace pas 0 / vide
<code>NULLIF</code>	produire du <code>NULL</code>	<code>NULLIF(plafond, 0)</code>	le retourner (c'est <code>COALESCE</code>)
<code>COALESCE(NULLIF(...))</code>	vider les zéros puis repli	<code>COALESCE(NULLIF(x, 0), 0)</code>	l'oublier sur une somme
Contrôle	cohérence	somme des segments = total	ne pas vérifier le total

15. Résumé du chapitre

- **CASE** est la brique de segmentation : forme de recherche (conditions arbitraires) et forme simple (comparaison d'une valeur). Les `WHEN` sont testés **dans l'ordre**, la 1^{re} qui passe gagne ; **toujours un ELSE**.
- **Segmenter** = découper en tranches disjointes et exhaustives ; le contrôle : la somme des segments redonne le total (50 008).
- **COALESCE(a, b, ...)** = la première valeur non `NULL` : le repli des absences. Il ne regarde que les `NULL` (pas les 0, pas les vides).
- **NULLIF(a, b)** = `NULL` si `a = b`, sinon `a` : le producteur de `NULL` (le miroir de `COALESCE`). `COALESCE(NULLIF(x, 0), défaut)` vide les zéros puis met le repli.
- **Filtrer sur une étiquette** : on recalcule la condition dans le `WHERE`, ou on encapsule l'étiquette (sous-requête, vue, CTE — C08).
- **Interpréter les statuts** : « inactif » ≠ « jamais vendu » (ici 471 206 245 FCFA de CA sur 18 produits inactifs).

16. À retenir

- **CASE** : **toujours un ELSE**, ordre des `WHEN` du spécifique au général.
- **Segmentation** : tranches disjointes + exhaustives ; somme des segments = total.
- **COALESCE** = **repli des NULL** (pas des 0 / vides) ; **NULLIF** = **producteur de NULL**.
- **COALESCE(NULLIF(x, 0), défaut)** : le duo pour vider les zéros puis replier.
- **Statut ≠ historique** : vérifier le CA réel par statut avant d'interpréter.

À retenir. `CASE`, `COALESCE`, `NULLIF` sont **standard SQL** : elles s'écrivent à l'identique sur DuckDB, PostgreSQL et SQLite. C'est l'un des rares pans du langage où la portabilité est totale — à condition de ne pas employer les raccourcis non standard (`??`, `IFNULL` MySQL, `DECODE` Oracle).

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. À quoi sert le `ELSE` dans un `CASE`, et que se passe-t-il s'il est absent ?
2. Quelle est la différence entre `CASE x WHEN v1 ...` et `CASE WHEN c1 ...` ?
3. Que renvoie `COALESCE(NULL, 0, 'x')` ? Et `COALESCE(0, 'x')` ?
4. Que renvoie `NULLIF(5, 5)` ? Et `NULLIF(5, 0)` ?
5. Pourquoi « inactif » ne veut-il pas dire « jamais vendu » ?

Réponses :

1. Le `ELSE` est la valeur de repli si aucune condition ne passe. S'il est absent, les lignes non couvertes renvoient `NULL` — des segments invisibles dans les rapports.
2. `CASE x WHEN v1 ...` compare **une valeur x** à des constantes (`x = v1`, `x = v2...`) — c'est un sucre. `CASE WHEN c1 ...` teste des **conditions arbitraires** (comparaisons, `BETWEEN`, `LIKE`). Pour un intervalle, seule la forme de recherche convient.
3. `COALESCE(NULL, 0, 'x') → 0` (la 1^{re} non nulle). `COALESCE(0, 'x') → 0` (le 0 est une valeur, pas un `NULL`, donc il est retenu).
4. `NULLIF(5, 5) → NULL` (égalité). `NULLIF(5, 0) → 5` (inégalité, on garde le 1^{er} argument).
5. Parce que le statut « inactif » décrit l'état **actuel** du produit (plus proposé), pas son historique : ici, les 18 produits inactifs ont généré 2 434 ventes pour 471 206 245 FCFA sur 2025-2026. Un statut ne résume pas le passé.

Module M07.C07 — Joindre : INNER, LEFT, FULL, CROSS, jointures multiples

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 5 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C05 (GROUP BY) ; M07.C06 (CASE).

L'idée du chapitre. C04–C06 travaillaient sur **une table à la fois** (ou sur des agrégats d'une table). C07 brise cette limite : **relier plusieurs tables** par leurs clés. « Le CA de chaque vente, avec le nom du client et la ville du magasin. » « Les magasins qui n'ont pas encore d'objectif. » C'est la **jointure** — join : l'opération qui fait d'un schéma en 9 tables un ensemble interrogeable. Le chapitre couvre **INNER JOIN** (les correspondances des deux côtés), **LEFT JOIN** (garder le côté gauche, **NULL** à droite), **FULL OUTER JOIN** (les deux côtés), **CROSS JOIN** (le produit cartésien), les **jointures multiples en étoile**, et les deux pièges du module — **P4** (la jointure qui double les lignes) et **P5** (le **DISTINCT** qui masque un mauvais **JOIN**). Le fil rouge est la base `commercial.duckdb` : on y relie 50 008 ventes à leurs 1 200 clients, 380 produits, 5 magasins et 5 modes de paiement, et on montre qu'une jointure mal conçue gonfle 50 008 lignes en **2 131 064**.

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes, 1 200 clients (tous présents dans `vente`), 380 produits (tous vendus au moins une fois), 5 magasins, 5 modes de paiement, 8 catégories, et la table `objectif_magasin` **vide** (0 lignes) — un objet de démonstration idéal pour le **LEFT JOIN** (elle produit des **NULL** réels).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **relier deux tables** par une clé avec **INNER JOIN ... ON** et comprendre le résultat (les correspondances des deux côtés) ;
- **garder le côté gauche** avec **LEFT JOIN** et interpréter les **NULL** du côté droit (les lignes sans correspondance) ;
- **garder les deux côtés** avec **FULL OUTER JOIN** ;
- **produire le produit cartésien** avec **CROSS JOIN** et savoir quand c'est voulu ;
- **enchaîner plusieurs jointures** en étoile autour d'une table centrale (`vente`) ;
- **détecter** une jointure qui double les lignes (piège P4) et un **DISTINCT** qui masque un mauvais **JOIN** (piège P5).

2. Pourquoi cette notion est importante

- **Toute interrogation métier multi-table est une jointure** : « le CA par client avec son nom », « les ventes de chaque ville ». Un schéma en 9 tables (le socle M07) ne devient lisible qu'au travers de jointures.
- **Le choix de type de jointure est une décision sémantique** : **INNER** répond à « les ventes qui ont un client » ; **LEFT** répond à « tous les magasins, avec leur objectif s'il existe ». Se tromper de type, c'est répondre à une autre question.
- **Les pièges P4/P5 sont les bugs les plus coûteux en production** : un total gonflé par une jointure qui double, ou un **DISTINCT** qui « répare » un mauvais **JOIN** sans que personne ne le sache.

3. Explication simple — la jointure est un accord de clés

Imaginez deux fiches : une **vente** (qui note `id_client = 73`) et un fichier des **clients** (une fiche par `id_client`). La jointure `vente JOIN client ON vente.id_client = client.id_client` fait l'**accord** : pour chaque vente, on retrouve la fiche du client de même `id`, et on met les deux côte à côte. Le résultat a **les colonnes des deux tables**, et **autant de lignes qu'il y a de paires accordées**.

`INNER JOIN` = « ne gardez que les paires qui se sont accordées ». `LEFT JOIN` = « gardez toutes les ventes, même celles qui n'ont trouvé aucun client (elles seront complétées de `NULL` à droite) ». C'est tout.

Définition. Une **jointure** — `join` — est l'opération qui met côte à côte les lignes de deux tables (ou plus) qui **s'accordent sur une clé**, via `ON` (par exemple `v.id_client = c.id_client`). Le résultat combine les colonnes des deux tables. Le type de jointure (`INNER`, `LEFT`, `FULL`, `CROSS`) décide quelles paires sont conservées.

4. Vocabulaire essentiel

Français — English		Définition simple	Piège à éviter
INNER JOIN		les paires qui se sont accordées des deux côtés.	croire qu'elle garde toutes les lignes (elle les filtre).
LEFT JOIN		toutes les lignes du gauche , <code>NULL</code> à droite si aucune correspondance.	mettre le filtre du côté droit dans <code>WHERE</code> (il transforme le <code>LEFT</code> en <code>INNER</code>).
FULL JOIN	OUTER	toutes les lignes des deux côtés, <code>NULL</code> où il manque.	le confondre avec <code>LEFT</code> (il garde aussi le droit seul).
CROSS JOIN		le produit cartésien : chaque ligne du gauche x chaque ligne du droit.	l'employer par accident (oubli de <code>ON</code>) : explosion de lignes.
clé de jointure		la (les) colonne(s) comparée(s) dans <code>ON</code> .	joindre sur une colonne non unique (doublons) sans le savoir.
jointure étoile	en	une table centrale (vente) reliée à N tables de référence.	joindre les tables de référence entre elles (inutile, et source de doublons).
explosion jointure	de	une jointure qui multiplie les lignes au lieu de les enrichir (P4).	ne pas vérifier <code>COUNT(*)</code> avant/après la jointure.
ON vs WHERE		<code>ON</code> définit l'accord ; <code>WHERE</code> filtre après (pour <code>LEFT</code> , le sens change).	mettre la condition du côté droit d'un <code>LEFT</code> dans <code>WHERE</code> (il filtre les <code>NULL</code>).

5. Cours approfondi

5.1 INNER JOIN — les correspondances des deux côtés

```
SELECT v.id_vente, c.nom, v.montant_ttc
FROM vente v
INNER JOIN client c ON c.id_client = v.id_client
ORDER BY v.id_vente
LIMIT 3;
```

Chaque vente retrouve le client de même `id_client`. Sur le socle, **toutes** les ventes ont un client existant (1 200 clients, tous présents dans `vente`) : le `INNER JOIN` retourne donc 50 008 lignes, chacune enrichie du `nom` du client.

```
SELECT COUNT(*) AS nb_lignes
FROM vente v
INNER JOIN client c ON c.id_client = v.id_client;
```

→ **50 008** (aucune vente orpheline dans le socle). Le `INNER JOIN` n'a rien filtré parce que la clé `id_client` de `vente` pointe toujours vers un client existant — c'est la propriété d'intégrité référentielle que le socle garantit.

Définition. La **clé étrangère** — foreign key (FK) — est la colonne d'une table qui pointe vers la clé primaire d'une autre table (`vente.id_client` → `client.id_client`). Une jointure `INNER` sur une FK intègre ne supprime jamais de ligne de la table centrale ; une FK **orpheline** (qui pointe vers une clé inexistante) serait, elle, éliminée par le `INNER JOIN`.

5.2 LEFT JOIN — garder le côté gauche, NULL à droite

La table `objectif_magasin` est **vide** (0 lignes) dans le socle. Le `LEFT JOIN` depuis `magasin` est donc l'objet de démonstration idéal : chaque magasin reste, et son objectif apparaît en `NULL`.

```
SELECT m.id_magasin, m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
ORDER BY m.id_magasin;
```

id_magasin	nom	ca_objectif_ttc
1	Ouaga Centre	NULL
2	Ouaga Patte d'Oie	NULL
3	Bobo Centre	NULL
4	Bobo Sarfalao	NULL
5	Koudougou	NULL

Cinq magasins, cinq lignes, **tous les objectifs en NULL** (la table de droite étant vide). C'est exactement le comportement voulu : « tous les magasins, avec leur objectif **s'il existe** ». Un `INNER JOIN` ici retournerait **0 ligne** (aucune paire accordée), ce qui serait une erreur de réponse.

5.3 Le piège du WHERE sur le côté droit d'un LEFT JOIN

```
-- MAUVAIS : le filtre sur o élimine les NULL et transforme le LEFT en INNER
SELECT m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
WHERE o.ca_objectif_ttc > 0;
```

```
-- JUSTE : la condition sur le côté droit va dans ON
SELECT m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
    AND o.ca_objectif_ttc > 0
ORDER BY m.id_magasin;
```

Dans le `WHERE`, les lignes dont `ca_objectif_ttc` est `NULL` sont **éliminées** (car `NULL > 0` est `NULL`, ni vrai ni faux) : le `LEFT JOIN` se comporte comme un `INNER`. Dans l'`ON`, la condition ne filtre pas le côté gauche : les magasins sans objectif restent, `NULL` à droite.

Attention. Mettre la condition du **côté droit** d'un `LEFT JOIN` dans le `WHERE` transforme silencieusement la jointure en `INNER` (les `NULL` du droit sont éliminés). Pour filtrer le côté droit sans perdre le gauche, écrire la condition dans l'`ON`. C'est le piège de jointure le plus fréquent en production.

5.4 FULL OUTER JOIN — les deux côtés

`FULL OUTER JOIN` garde **toutes** les lignes des deux tables : celles qui s'accordent, celles du gauche sans correspondance (`NULL` à droite), celles du droit sans correspondance (`NULL` à gauche).

```
SELECT m.id_magasin, m.nom, o.annee
FROM magasin m
FULL OUTER JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
ORDER BY m.id_magasin;
```

→ 5 lignes (les 5 magasins, `annee` en `NULL`) — ici identique au `LEFT JOIN` car la table de droite est vide (il n'y a pas de ligne du droit « seule »). Sur deux tables toutes deux peuplées et partiellement disjointes, le `FULL` ajouterait les lignes orphelines du droit.

5.5 CROSS JOIN — le produit cartésien

`CROSS JOIN` produit **toutes les paires** (chaque ligne du gauche × chaque ligne du droit), sans condition. 5 magasins × 5 modes de paiement = **25 lignes** :

```
SELECT m.nom, mp.libelle
FROM magasin m
CROSS JOIN mode_paiement mp
ORDER BY m.id_magasin, mp.id_mode;
```

→ 25 combinaisons (chacun des 5 magasins avec chacun des 5 modes). C'est voulu quand on veut une **grille complète** (tous les couples, même ceux à zéro) ; c'est un bug quand on l'obtient par oubli de `ON` (une jointure qui explose).

Définition. Le **produit cartésien** — cross product — est l'ensemble de toutes les paires (ligne du gauche × ligne du droit), sans condition : $|gauche| \times |droit|$ lignes. `CROSS JOIN` l'explique (grille complète voulue) ; une `JOIN` sans `ON` (ou un `ON` toujours vrai) le produit par accident (explosion de lignes, cf. P4).

5.6 Jointures multiples en étoile

La table centrale `vente` se relie à **quatre** tables de référence (`client`, `produit`, `magasin`, `mode_paiement`) — la **jointure en étoile**. On n'enchaîne que des `N → 1` (chaque vente pointe vers une seule valeur de référence) : le résultat garde 50 008 lignes.

```

SELECT v.id_vente,
       c.nom AS client,
       p.designation AS produit,
       m.nom AS magasin,
       mp.libelle AS mode,
       v.montant_ttc
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN produit p     ON p.id_produit = v.id_produit
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode
ORDER BY v.id_vente
LIMIT 3;

```

```

SELECT COUNT(*) AS nb_lignes
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN produit p     ON p.id_produit = v.id_produit
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode;

```

→ **50 008** (chaque vente enrichie de 4 libellés, sans doublon). C'est le motif de référence de tout « rapport détaillé ».

À retenir. Une jointure en étoile ne **double jamais** les lignes tant que chaque relation est $N \rightarrow 1$ (une vente pointe vers un seul client, un seul produit, un seul magasin, un seul mode). Le contrôle est immédiat : `COUNT(*)` après l'étoile doit redonner le `COUNT(*)` de la table centrale. S'il ne redonne pas, une relation a multiplié — c'est P4.

5.7 Le piège P4 — la jointure qui double

Le piège P4 : une jointure qui **multiplie** les lignes. Le cas d'école : relier `vente` à elle-même via `id_client` (chaque vente « s'accorde » avec **toutes** les ventes du même client) :

```

SELECT COUNT(*) AS nb_lignes_explosees
FROM vente v
JOIN vente v2 ON v2.id_client = v.id_client;

```

→ **2 131 064 lignes** pour 50 008 ventes uniques. Chaque vente est comptée une fois par chacune des autres ventes de son client (son nombre de ventes + 1). Le `COUNT(*)` est **gonflé d'un facteur ~42**. La bonne pratique : compter les ventes uniques (`COUNT(DISTINCT v.id_vente)` → 50 008), et ne joindre que des relations $N \rightarrow 1$ (jamais $N \rightarrow N$ sur la même entité sans raison).

Définition. L'**explosion de jointure** — join fan-out — est le gonflement du nombre de lignes quand une jointure accorde **plusieurs lignes d'un côté à une ligne de l'autre** (relation $N \rightarrow N$, ou jointure sur une colonne non unique). Le contrôle : comparer `COUNT(*)` **avant** et **après** la jointure — s'il explose, la jointure double (piège P4).

5.8 Le piège P5 — le `DISTINCT` qui masque un mauvais `JOIN`

Le piège P5 : un `COUNT(DISTINCT ...)` peut donner le **bon chiffre par hasard**, alors que la jointure est fautive. Le `DISTINCT` « répare » en dédoublonnant, sans que personne ne sache que la jointure était mauvaise.

```
-- P5 : le DISTINCT "répare" le doublon de la jointure – chiffre juste, jointure fausse
SELECT COUNT(DISTINCT c.id_client) AS nb_clients_bobo
FROM vente v
JOIN client c ON c.id_client = v.id_client
WHERE c.ville = 'Bobo-Dioulasso';
```

Ici le chiffre (441 clients de Bobo) est juste **malgré** le fait que la jointure a déjà multiplié les lignes (chaque client apparaît une fois par vente). Le `DISTINCT` masque le problème. La bonne pratique : **vérifier le nombre de lignes avant l'agrégation** (`SELECT COUNT(*)` sur la jointure seule), pas après le `DISTINCT`.

Attention. Un `COUNT(DISTINCT col)` peut cacher une jointure qui double : le chiffre sort juste, mais la requête est fausse (et coûteuse). La règle P5 : **toujours contrôler `COUNT(*)` sur la jointure nue, avant tout `DISTINCT` ou agrégat**. Si le `COUNT(*)` explose, c'est la jointure qu'il faut corriger, pas le `DISTINCT` qu'il faut ajouter.

5.9 ON et alias — la lecture d'une jointure

La lecture d'une jointure : `FROM vente v JOIN client c ON c.id_client = v.id_client` = « pour chaque vente `v`, cherche le client `c` de même `id_client`, et mets les deux côte à côte ». Les alias (`v`, `c`) abrègent et évitent les homonymes (deux tables avec une colonne `nom` — ici `client.nom` et `magasin.nom` : c'est l'alias qui distingue).

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Pour chaque vente, le nom du client, la ville du magasin et le mode de paiement — et le CA total par ville. »

Étape 1 — Écrire la requête (jointure en étoile, puis regroupement).

```
SELECT m.ville,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode
GROUP BY m.ville
ORDER BY ca DESC;
```

Étape 2 — Prédire la sortie. 3 villes (Ouagadougou, Bobo-Dioulasso, Koudougou) ; le `COUNT(*)` de la jointure étoilée = 50 008 (pas d'explosion, toutes les relations sont `N → 1`) ; Ouagadougou en tête (le plus de clients).

Étape 3 — Exécuter. Ouagadougou 21 290 ventes / 3 391 679 290 FCFA ; Bobo-Dioulasso 18 364 / 2 895 549 769 ; Koudougou 10 354 / 1 621 030 672.

Étape 4 — Comparer. 3 villes, total 50 008 ventes, total 7 908 259 731 FCFA. La jointure étoilée n'a rien doublé (contrôle P4 passé).

Conseil professionnel. Avant tout `GROUP BY` après une jointure, **vérifiez que la jointure ne double pas** : `SELECT COUNT(*)` sur la jointure nue doit redonner le nombre de lignes de la table centrale (ici 50 008). Si c'est le cas, le `GROUP BY` est fiable ; sinon, corrigez la jointure d'abord. C'est le réflexe anti-P4/P5.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Ventes enrichies du nom du client

```
SELECT v.id_vente, c.nom, v.montant_ttc
FROM vente v
JOIN client c ON c.id_client = v.id_client
ORDER BY v.id_vente
LIMIT 3;
```

→ 50 008 lignes disponibles (toutes les ventes ont un client).

7.2 Question 2 — Magasins sans objectif (LEFT JOIN sur table vide)

```
SELECT m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
WHERE o.ca_objectif_ttc IS NULL;
```

→ 5 magasins (tous, l'objectif étant NULL partout) : c'est la requête standard « les X sans Y ».

7.3 Question 3 — La grille complète magasin × mode (CROSS JOIN)

```
SELECT COUNT(*) AS nb_combinaisons
FROM magasin m
CROSS JOIN mode_paiement mp;
```

→ 25 (5 × 5).

7.4 Question 4 — Le rapport détaillé en étoile

```
SELECT v.id_vente, c.nom AS client, m.nom AS magasin,
       mp.libelle AS mode, v.montant_ttc
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode
ORDER BY v.id_vente
LIMIT 3;
```

→ 50 008 lignes, chacune enrichie de 3 libellés.

7.5 Question 5 — Le piège P4 (jointure qui double)

```
SELECT COUNT(*) AS nb_explose, COUNT(DISTINCT v.id_vente) AS nb_uniques
FROM vente v
JOIN vente v2 ON v2.id_client = v.id_client;
```

→ 2 131 064 lignes explosees, 50 008 uniques. Le COUNT(*) est gonflé d'un facteur ~42 : c'est l'explosion de jointure.

7.6 Question 6 — Le CA par ville (jointure étoilée + GROUP BY)

```
SELECT m.ville, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.ville
ORDER BY ca DESC;
```

→ Ouagadougou 21 290 / 3 391 679 290 ; Bobo-Dioulasso 18 364 / 2 895 549 769 ; Koudougou 10 354 / 1 621 030 672 FCFA.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	WHERE sur le côté droit d'un LEFT JOIN	les NULL disparaissent, le LEFT devient INNER	la condition dans l'ON.
2	Jointure sur une colonne non unique (P4)	COUNT(*) explose (2 131 064 au lieu de 50 008)	joindre sur une clé unique, ou COUNT(DISTINCT) .
3	DISTINCT ajouté « pour réparer » (P5)	chiffre juste, jointure fausse, requête lente	corriger la jointure, contrôler COUNT(*) avant.
4	JOIN sans ON (oubli)	produit cartésien par accident	ajouter l'ON, ou CROSS JOIN si voulu.
5	Colonnes homonymes sans alias	erreur « column is ambiguous »	alias sur chaque table (v., c.).
6	INNER JOIN là où il faut LEFT JOIN	les lignes orphelines du gauche disparaissent	choisir le type selon la question.

Attention. Les erreurs 2 et 3 sont les pièges P4 et P5 du module : une jointure qui **double** (P4) et un DISTINCT qui **masque** un mauvais JOIN (P5). Les deux se contrôlent par le même geste : COUNT(*) sur la jointure **nue**, avant tout agrégat.

Attention. Un JOIN sans ON (ou un ON toujours vrai, comme ON 1=1) produit le **produit cartésien** : |gauche| × |droit| lignes. Sur 50 008 ventes et 1 200 clients, ce serait un peu plus de 60 milliards de lignes — à ne jamais laisser passer en production.

9. Bonnes pratiques professionnelles

- **Joindre sur des clés uniques** (PK/FK) ; jamais sur une colonne libre (nom, libellé).
- **COUNT(*) avant/après** chaque jointure : le contrôle P4 est systématique.
- **Le type de jointure suit la question** : « les X qui ont Y » → INNER ; « tous les X, avec Y si existe » → LEFT.
- **Condition du côté droit d'un LEFT dans l'ON**, jamais dans le WHERE.
- **Alias sur chaque table** : v., c., m. — obligatoire dès deux colonnes homonymes.
- **Étoile, pas grappe** : relier la table centrale aux références, pas les références entre elles.

10. Exercice guidé — le rapport détaillé par ville (15 min, /10)

Énoncé. Produire, par ville (3 lignes, triées par CA décroissant) : le nombre de ventes, le CA arrondi, le nombre de clients distincts. Utiliser une jointure en étoile.

Grille de correction (/10).

Critère	Points
Jointures vente → magasin (et client pour le distinct) correctes	3
COUNT(*), ROUND(SUM(...)), COUNT(DISTINCT c.id_client)	3
GROUP BY m.ville + ORDER BY ca DESC	2
3 lignes, Ouagadougou en tête (3 391 679 290)	2

Corrigé.

```
SELECT m.ville,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca,
       COUNT(DISTINCT c.id_client) AS nb_clients
FROM vente v
JOIN client c ON c.id_client = v.id_client
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.ville
ORDER BY ca DESC;
```

→ Ouagadougou 21 290 / 3 391 679 290 / 512 ; Bobo-Dioulasso 18 364 / 2 895 549 769 / 441 ; Koudougou 10 354 / 1 621 030 672 / 247.

11. Exercices autonomes

Exercice 1 — Ventes + nom du client

Les 10 premières ventes avec le nom du client (jointure `INNER`).

Exercice 2 — Magasins sans objectif

Les magasins dont l'objectif est `NULL` (`LEFT JOIN + WHERE ... IS NULL`).

Exercice 3 — La grille magasin × mode

Le nombre de combinaisons `CROSS JOIN` (magasin × mode_paiement).

Exercice 4 — Le rapport étoilé

Les 5 premières ventes avec client, magasin, mode (3 libellés).

Exercice 5 — Mesurer l'explosion (P4)

`COUNT(*)` et `COUNT(DISTINCT v.id_vente)` de la jointure `vente × vente` via `id_client`. Donner le facteur de gonflement.

Exercice 6 — Le CA par ville

Une jointure `vente → magasin`, `GROUP BY ville`, CA arrondi, trié décroissant.

Exercice 7 — `LEFT` vs `INNER` sur table vide

Combien de lignes renvoient `INNER JOIN` et `LEFT JOIN` depuis `magasin` vers `objectif_magasin` ? Expliquer.

Exercice 8 — Diagnostic d'erreur

Pourquoi cette requête est-elle une erreur, et comment la corriger ?

```
SELECT v.id_vente, c.nom, m.nom
FROM vente v
JOIN client c ON c.id_client = v.id_client
JOIN magasin m ON m.id_magasin = v.id_magasin;
```

12. Correction détaillée

Exercice 1.

```
SELECT v.id_vente, c.nom, v.montant_ttc
FROM vente v
JOIN client c ON c.id_client = v.id_client
ORDER BY v.id_vente
LIMIT 10;
```

→ 10 lignes (les 10 premières ventes, enrichies du nom du client).

Exercice 2. Cf. §7.2 → 5 magasins (tous, l'objectif étant `NULL` partout).

Exercice 3. Cf. §7.3 → **25** combinaisons.

Exercice 4. Cf. §7.4 → 5 lignes, chacune avec 3 libellés.

Exercice 5. `COUNT(*)` = **2 131 064**, `COUNT(DISTINCT v.id_vente)` = **50 008**. Facteur de gonflement $\approx 2\,131\,064 / 50\,008 \approx 42,6$: chaque vente est comptée une fois par vente du même client. C'est l'explosion P4.

Exercice 6. Cf. §7.6 → Ouagadougou 3 391 679 290 ; Bobo-Dioulasso 2 895 549 769 ; Koudougou 1 621 030 672 FCFA.

Exercice 7. `INNER JOIN` → **0 ligne** (aucune paire accordée, `objectif_magasin` est vide) ; `LEFT JOIN` → **5 lignes** (les 5 magasins, objectif en `NULL`). Le choix dépend de la question : « les magasins qui ont un objectif » (`INNER`, 0 ici) vs « tous les magasins avec leur objectif si existe » (`LEFT`, 5).

Exercice 8. Piège de relecture : la requête est en fait **valide** (les deux `nom` sont préfixés `c.` et `m.`, donc pas d'ambiguïté) — c'est la version « non préfixée » qui serait une erreur. Si on écrivait `SELECT nom` sans alias, DuckDB renverrait « column "nom" is ambiguous » (la colonne existe dans `client` et dans `magasin`). La règle : **toujours préfixer les colonnes homonymes par l'alias** dès qu'une jointure met deux tables ayant la même colonne.

13. Mini-projet — « L'audit de jointures du rapport détaillé » (45 min)

Produire l'audit de jointures du rapport détaillé : 4 requêtes, chacune vérifiée à la main.

Requête	Lignes attendues
J1 : le rapport étoilé (client, magasin, mode) — <code>COUNT(*)</code>	50 008
J2 : magasins sans objectif (<code>LEFT JOIN + IS NULL</code>)	5
J3 : grille magasin × mode (<code>CROSS JOIN</code>)	25
J4 : CA par ville (étoile + <code>GROUP BY</code>)	3

Contrôles de cohérence. (i) J1 : `COUNT(*)` = 50 008 (pas d'explosion, P4 passé). (ii) J4 : la somme des `ca` = 7 908 259 731 FCFA. (iii) J3 : $25 = 5 \times 5$.

Corrigé.


```
-- J1
SELECT COUNT(*) AS nb_lignes
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN produit p     ON p.id_produit = v.id_produit
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paie ment mp ON mp.id_mode = v.id_mode;

-- J2
SELECT m.nom
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
WHERE o.ca_objectif_ttc IS NULL;

-- J3
SELECT COUNT(*) AS nb_combinaisons
FROM magasin m
CROSS JOIN mode_paie ment mp;

-- J4
SELECT m.ville, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.ville
ORDER BY ca DESC;
```

Dans les faits. La syntaxe `JOIN ... ON` est **identique** sur DuckDB, PostgreSQL et SQLite pour `INNER/LEFT/CROSS`. Le `FULL OUTER JOIN` est supporté par DuckDB et PostgreSQL ; **SQLite ne le supporte pas** (il faut le simuler par `LEFT JOIN + UNION + LEFT JOIN`). C'est la principale différence de portabilité du chapitre.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 4 types de jointure (`INNER`, `LEFT`, `FULL`, `CROSS`), la **jointure en étoile** (la table centrale reliée aux références), et les deux contrôles anti-pièges (P4 : `COUNT(*)` avant/après ; P5 : ne jamais « réparer » un `JOIN` avec un `DISTINCT`). Le choix du type suit la question ; la condition du côté droit d'un `LEFT` va dans l'`ON`.

Outil	Usage	Syntaxe	Piège
<code>INNER JOIN</code>	les paires accordées	<code>v JOIN c ON c.id=v.id</code>	filtre les orphelines du gauche
<code>LEFT JOIN</code>	tout le gauche, <code>NULL</code> à droite	<code>m LEFT JOIN o ON ...</code>	<code>WHERE</code> sur le droit = <code>INNER</code>
<code>FULL OUTER JOIN</code>	les deux côtés	<code>m FULL JOIN o ON ...</code>	absent de SQLite
<code>CROSS JOIN</code>	produit cartésien	<code>m CROSS JOIN mp</code>	explosion si par accident
Étoile	table centrale + références	<code>vente JOIN c JOIN m JOIN mp</code>	joindre les références entre elles
Contrôle P4	explosion	<code>COUNT(*)</code> avant/après	2 131 064 au lieu de 50 008
Contrôle P5	<code>DISTINCT</code> qui masque	<code>COUNT(*)</code> sur la jointure nue	chiffre juste, jointure fausse

15. Résumé du chapitre

- **`INNER JOIN`** = les paires qui s'accordent des deux côtés (filtre les orphelines du gauche). **`LEFT JOIN`** = tout le gauche, `NULL` à droite. **`FULL OUTER JOIN`** = les deux côtés. **`CROSS JOIN`** = le produit cartésien (voulu en grille, bug par oubli d'`ON`).

- **La jointure en étoile** relie la table centrale (`vente`) aux références (client, produit, magasin, mode) en relations $N \rightarrow 1$: elle enrichit sans doubler (50 008 lignes conservées).
- **Le type de jointure suit la question** : « les X qui ont Y » → `INNER` ; « tous les X, avec Y si existe » → `LEFT`.
- **P4** : une jointure sur une colonne non unique **double** les lignes (2 131 064 au lieu de 50 008) — contrôle par `COUNT(*)` avant/après.
- **P5** : un `DISTINCT` peut **masquer** un mauvais `JOIN` (chiffre juste, requête fausse) — contrôler la jointure nue avant tout `DISTINCT`.
- **ON vs WHERE** : la condition du côté droit d'un `LEFT JOIN` va dans l'`ON`, jamais dans le `WHERE`.

16. À retenir

- **Le type suit la question** : `INNER` = « qui a », `LEFT` = « tous, avec si existe ».
- **Étoile, pas grappe** : relier la centrale aux références, jamais les références entre elles.
- `COUNT(*)` avant/après chaque jointure (P4) ; **jamais de `DISTINCT` pour réparer un `JOIN` (P5)**.
- **Condition du côté droit d'un `LEFT` dans l'`ON`**, pas dans le `WHERE`.
- **Alias + préfixe** sur chaque table : obligatoire dès deux colonnes homonymes.

À retenir. Une jointure est un **accord de clés** : `ON v.id_client = c.id_client`. Le type (`INNER/LEFT/FULL/CROSS`) décide **qui reste** quand l'accord échoue. Le bon réflexe n'est pas la syntaxe (facile) mais le **contrôle du nombre de lignes** (difficile) : `COUNT(*)` avant et après, et la question « est-ce que ma jointure devrait doubler ? ».

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Différence entre `INNER JOIN` et `LEFT JOIN` ?
2. Pourquoi la condition du côté droit d'un `LEFT JOIN` doit-elle aller dans l'`ON` ?
3. Que produit un `CROSS JOIN` entre 5 magasins et 5 modes de paiement ?
4. C'est quoi l'explosion de jointure (P4), et comment la détecter ?
5. Pourquoi un `DISTINCT` peut-il masquer un mauvais `JOIN` (P5) ?

Réponses :

1. `INNER JOIN` ne garde que les paires qui s'accordent **des deux côtés** (les orphelines du gauche disparaissent) ; `LEFT JOIN` garde **toutes** les lignes du gauche, et complète de `NULL` à droite celles sans correspondance.
2. Parce que dans le `WHERE`, les lignes dont la colonne du droit est `NULL` sont **éliminées** (`NULL > 0` est `NULL`), ce qui transforme le `LEFT JOIN` en `INNER`. Dans l'`ON`, la condition ne filtre pas le côté gauche : les lignes sans correspondance restent.
3. Le **produit cartésien** : $5 \times 5 = 25$ lignes (chaque magasin avec chaque mode), sans condition.
4. C'est le gonflement du nombre de lignes quand la jointure accorde plusieurs lignes d'un côté à une ligne de l'autre (relation $N \rightarrow N$). Détection : `COUNT(*)` **avant** (50 008) et **après** la jointure (2 131 064) — si ça explose, la jointure double.
5. Parce que le `DISTINCT` **dédoublonne** le résultat : le chiffre final peut être juste **même si** la jointure a doublé les lignes. On ne voit pas le bug (chiffre correct), mais la requête est fausse et lente. D'où le contrôle de la jointure nue avant l'agrégat.

Module M07.C08 — Sous-requêtes, CTE, vues, EXISTS vs IN

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM) ; M07.C05 (GROUP BY) ; M07.C07 (jointures).

L'idée du chapitre. C07 a relié les tables par des jointures. C08 montre comment **structurer** une requête quand la question est composée : d'abord calculer quelque chose, puis filtrer sur le résultat. C'est le passage de la requête « plate » à la requête « en couches » : la **sous-requête** (une requête dans une autre), la **CTE** (WITH — une sous-requête nommée), la **vue** (une sous-requête enregistrée), et la famille **EXISTS / IN** (exister ou non dans un ensemble) qui inclut le pattern canonique de la **double négation** (piège P6 : « les clients qui ont acheté **tous** les produits X »). Le fil rouge est la base `commercial.duckdb` : on y identifie le client n° 1 par CA (Client_0073, 11 561 026 FCFA), on lit la vue `v_ca_mensuel_magasin` (120 lignes), et on mesure que **119 clients** ont acheté les 2 produits les plus chers (double négation).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes, 1 200 clients (tous ont au moins une vente — les « clients sans vente » sont donc 0), 380 produits (55 dans la catégorie Alimentaire-Boissons, id 1), et la vue `v_ca_mensuel_magasin` (120 lignes : 5 magasins × 24 mois).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **écrire une sous-requête scalaire** (une valeur) dans le `SELECT` ou le `WHERE` ;
- **écrire une sous-requête dans le `FROM`** (table dérivée) et l'agréger ;
- **nommer une sous-requête** avec `WITH` (CTE) pour lisibilité et réutilisation ;
- **créer une vue** et comprendre sa différence avec une CTE (enregistrée vs éphémère) ;
- **distinguer `IN` de `EXISTS`** (et leurs négations) pour tester l'appartenance à un ensemble ;
- **écrire le pattern de double négation** (`NOT EXISTS ... NOT EXISTS`) pour les questions « **tous les** » (piège P6).

2. Pourquoi cette notion est importante

- **Les questions métier réelles sont composées** : « les ventes au-dessus du panier moyen », « le client qui a le plus dépensé », « les clients qui ont tout acheté ». Aucune ne se pose en une seule requête plate — il faut d'abord **calculer**, puis **utiliser le résultat**.
- **La CTE (`WITH`) est l'outil de lisibilité** : une requête de 40 lignes devient 4 blocs nommés. C'est la différence entre une requête lisible par un collègue et une requête qu'on n'ose pas toucher.
- **`EXISTS` vs `IN`** est l'un des points les plus demandés en entretien, et la **double négation** est le pattern canonique des questions « tous les » — celles que `IN` et `COUNT` ne savent pas poser.

3. Explication simple — la sous-requête est une question dans une question

Une sous-requête, c'est poser **une question à l'intérieur d'une autre**. « Trouve-moi les ventes **au-dessus du panier moyen** » se décompose en deux questions : (1) « quel est le panier moyen ? » (2) « quelles ventes dépassent ce chiffre ? ». En SQL, la question (1) s'écrit **dedans** la question (2) :

```
SELECT ... FROM vente
WHERE montant_ttc > ( SELECT AVG(montant_ttc) FROM vente )
                    ↑ la question dans la question
```

Le moteur exécute d'abord la question de l'intérieur (le panier moyen : 158 140 FCFA), puis la question de l'extérieur avec ce résultat. **WITH** (la CTE) fait la même chose, mais **nomme** la question intérieure : ``WITH panier AS (SELECT AVG(...) ...) SELECT ... WHERE ...`

(SELECT ... FROM panier)`. Et la **vue** enregistre cette CTE nommée dans la base, pour la réutiliser.

Définition. Une **sous-requête** — subquery — est une requête SQL **implantée à l'intérieur** d'une autre requête, dans le **SELECT**, le **WHERE** ou le **FROM**. Elle est exécutée d'abord (ou à la demande), et son résultat alimente la requête extérieure. On distingue la sous-requête **scalaire** (renvoie 1 valeur) et la sous-requête **de liste** (renvoie une colonne de valeurs, pour un **IN**).

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
sous-requête scalaire	une sous-requête qui renvoie 1 valeur (1 ligne, 1 colonne).	l'utiliser quand elle renvoie plusieurs lignes (erreur).
sous-requête de liste	renvoie une colonne de valeurs, pour un IN / NOT IN .	NOT IN avec des NULL dans la liste (rien ne passe).
CTE (WITH)	une sous-requête nommée , déclarée en tête de requête.	croire qu'une CTE est optimisée comme une vue (c'est du sucre de lisibilité).
vue	une requête enregistrée dans la base, interrogée comme une table.	croire qu'elle fige les données (elle se recalcule à chaque lecture).
table dérivée	une sous-requête dans le FROM (avec un alias obligatoire).	oublier l'alias (erreur de syntaxe).
EXISTS	teste s'il existe au moins une ligne dans une sous-requête corrélée.	le confondre avec IN (EXISTS regarde l'existence, IN l'appartenance).
corrélé	une sous-requête qui référence une colonne de la requête extérieure.	le confondre avec une sous-requête indépendante (exécutée une fois).
double négation	NOT EXISTS (... NOT EXISTS (...)) : le pattern des questions « tous les ».	essayer de le faire avec IN ou COUNT (faux ou illisible).

5. Cours approfondi

5.1 La sous-requête scalaire dans le **SELECT**

La sous-requête scalaire renvoie **une valeur** qu'on place dans le **SELECT** :

```
SELECT v.id_vente, v.montant_ttc,
       (SELECT AVG(montant_ttc) FROM vente) AS panier_moyen_global
FROM vente v
ORDER BY v.id_vente
LIMIT 3;
```

→ Chaque ligne affiche le montant de la vente **et** le panier moyen global (158 140 FCFA, recalculé). C’est le motif « contexte » : afficher une valeur de référence à côté de chaque ligne.

5.2 La sous-requête scalaire dans le `WHERE`

« Les ventes au-dessus du panier moyen » :

```
SELECT COUNT(*) AS nb_ventes
FROM vente
WHERE montant_ttc > (SELECT AVG(montant_ttc) FROM vente);
```

Le moteur calcule d’abord `AVG(montant_ttc)` = 158 140 FCFA, puis filtre les ventes qui le dépassent : **20 289** ventes. La comparaison est faite contre la valeur calculée, pas contre une constante — c’est la force de la sous-requête scalaire dans le `WHERE`.

Définition. Une **sous-requête scalaire** — scalar subquery — est une sous-requête qui doit renvoyer **exactement une valeur** (1 ligne, 1 colonne). Elle s’utilise dans le `SELECT` (valeur de référence par ligne) ou le `WHERE` (filtre contre une valeur calculée). Si elle renvoie plus d’une ligne, DuckDB/PostgreSQL renvoient une erreur ; c’est un garde-fou, pas un bug.

5.3 La sous-requête dans le `FROM` — la table dérivée

Quand la question intérieure renvoie **plusieurs lignes**, on la met dans le `FROM` (avec un **alias obligatoire**) :

```
SELECT seg, COUNT(*) AS nb
FROM (
  SELECT CASE
    WHEN c.type_client = 'particulier' THEN 'Particulier'
    WHEN c.type_client = 'entreprise' THEN 'Entreprise'
    ELSE 'Comptoir'
  END AS seg
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
) t
GROUP BY seg
ORDER BY nb DESC;
```

La sous-requête (alias `t`) produit 50 008 lignes avec une colonne `seg` ; l’extérieur les regroupe. C’est l’alternative à la CTE — même chose, sans nom.

5.4 La CTE (`WITH`) — nommer la question intérieure

Le `WITH` **nomme** la sous-requête du `FROM` (ou d’une étape intermédiaire) :

```
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 3;
```

→ Client_0073 en tête avec **11 561 026 FCFA** (le client n° 1 par CA, id 73). Le `WITH` ne change pas le résultat par rapport à la table dérivée — il change la **lisibilité** : chaque étape a un nom, on peut la relire séparément.

Définition. Une **CTE** — Common Table Expression — est une sous-requête **nommée**, déclarée en tête de requête par `WITH nom AS (SELECT ...)`, puis référencée comme une table dans le reste de la requête. Elle est **éphémère** (n'existe que durant l'exécution de la requête) et sert surtout à la **lisibilité** : découper une requête complexe en étapes nommées. Elle se distingue d'une **vue** (enregistrée dans la base, réutilisable).

5.5 La vue — une CTE enregistrée

La base contient déjà la vue `v_ca_mensuel_magasin` (120 lignes : 5 magasins × 24 mois). Une vue est une requête **enregistrée** dans le schéma, interrogée comme une table :

```
SELECT id_magasin, annee, mois, nb_ventes, ROUND(ca_ttc) AS ca_ttc
FROM v_ca_mensuel_magasin
ORDER BY ca_ttc DESC
LIMIT 3;
```

→ Le plus gros mois : magasin 3, 2025-08, 444 ventes, 78 965 529 FCFA. La vue **ne fige pas** les données : elle se **recalcule** à chaque lecture (si vente change, la vue change). C'est le motif de C05/C06 « l'étiquette réutilisable » : on calcule une fois (la vue), on relit partout.

Définition. Une **vue** — view — est une requête SQL **enregistrée** dans le schéma de la base, interrogeable comme une table (`SELECT ... FROM vue`). Contrairement à une CTE (éphémère, liée à une requête), une vue **persiste** et est **réutilisable** par n'importe quelle requête. Elle ne stocke pas de données : elle **se recalcule** à chaque lecture. C'est l'outil d'encapsulation des étiquettes et des agrégats de base de travail.

5.6 IN — appartenir à un ensemble

`WHERE x IN (sous-requête)` teste si `x` appartient à la liste renvoyée :

```
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
WHERE p.id_categorie IN (SELECT id_categorie FROM produit WHERE actif = FALSE);
```

Ou, plus simplement, « les clients qui ont acheté au moins un produit de la catégorie 1 » :

```
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
WHERE v.id_produit IN (SELECT id_produit FROM produit WHERE id_categorie = 1);
```

→ **1 196 clients** (sur 1 200) ont acheté au moins un des 55 produits de la catégorie Alimentaire-Boissons. Le **IN** est l'opérateur « **au moins un** ».

À retenir. Le sens de **IN** est « **au moins un** » : `WHERE x IN (...) = « il existe au moins une valeur de la liste égale à x »`. C'est pourquoi **IN ne pose jamais** une question « tous les » — pour celle-ci, il faut la double négation (§5.9). Confondre « au moins un » et « tous » est l'erreur la plus fréquente des requêtes d'appartenance.

5.7 NOT IN et le piège des NULL

NOT IN est le « **aucun** » : « les clients qui n'ont acheté **aucun** produit de la catégorie 1 » :

```
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE c.id_client NOT IN (SELECT id_client FROM vente v
                          JOIN produit p ON p.id_produit = v.id_produit
                          WHERE p.id_categorie = 1);
```

→ **4 clients** (1 200 – 1 196). Attention au piège : si la sous-requête du **NOT IN** renvoie **un seul NULL**, le **NOT IN** ne renvoie **aucune ligne** (car `x NOT IN (a, NULL)` est **NULL** si `x ≠ a` — ni vrai ni faux). C'est le défaut classique du **NOT IN**.

5.8 EXISTS / NOT EXISTS — exister ou non

EXISTS teste s'il **existe** au moins une ligne dans une sous-requête **corrélée** (qui référence la ligne extérieure) :

```
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client);
```

→ **1 200** (tous les clients ont au moins une vente). Le **NOT EXISTS** est le « **aucun** » :

```
SELECT COUNT(*) AS nb_clients_sans_vente
FROM client c
WHERE NOT EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client);
```

→ **0** (aucun client sans vente dans le socle). Le **EXISTS/NOT EXISTS** est souvent plus lisible (et plus sûr face aux **NULL**) que **IN/NOT IN** pour les tests d'existence.

Définition. **EXISTS** teste s'il **existe** au moins une ligne dans sa sous-requête (généralement **corrélée** : elle référence une colonne de la requête extérieure). **NOT EXISTS** est sa négation. Contrairement à **IN/NOT IN**, ils sont **insensibles aux NULL** de la liste et court-circuitent dès qu'une ligne existe — c'est l'outil de référence pour les tests d'existence et la double négation.

5.9 Le pattern de double négation — les « tous les » (P6)

La question « les clients qui ont acheté **tous** les produits les plus chers » (ou « tous les produits d'une catégorie ») ne se pose **ni** avec **IN** (qui dit « au moins un ») **ni** avec **COUNT** (fragile). Le pattern canonique est la **double négation** : « il n'existe **aucun** produit X que le client n'a **pas** acheté » :


```

SELECT c.id_client, c.nom
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);

```

Logique : on cherche les clients **c** tels qu'il **n'existe aucun** produit du top 2 que **c n'a pas** acheté. C'est-à-dire les clients qui ont acheté les **deux**. → **119 clients**.

Définition. Le **pattern de double négation** — double negation — est l'écriture `NOT EXISTS (SELECT ... WHERE NOT EXISTS (SELECT ...))` qui exprime « **tous les** » : on nie l'existence d'un contre-exemple. « Le client a acheté **tous** les produits X » équivaut à « il n'existe **aucun** produit X que le client n'a **pas** acheté ». C'est le pattern canonique (piège P6) des questions universelles, là où **IN** (« au moins un ») et **COUNT** (fragile) échouent.

Attention. Ne pas poser une question « **tous les** » avec **IN** : `WHERE id_client IN (...)` renvoie ceux qui ont acheté **au moins un**, pas **tous**. Et ne pas le faire avec un **COUNT** comparé au total (fragile, et faux dès qu'un produit est acheté en double). La double négation est l'écriture sûre.

6. Exemple concret — la requête qui répond à une question métier

Le directeur demande : « Quel est le client qui a le plus dépensé, et combien de clients ont acheté les 2 produits les plus chers ? »

Étape 1 — Écrire les deux requêtes (une CTE pour le top client, une double négation pour les 2 plus chers).

```

-- (a) Le client n°1 par CA
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 1;

-- (b) Les clients qui ont acheté les 2 produits les plus chers
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);

```

Étape 2 — Prédire. (a) 1 ligne, le client n° 1 par CA (id 73). (b) Un nombre < 1 200 (ceux qui ont les 2 produits du top).

Étape 3 — Exécuter. (a) Client_0073, 11 561 026 FCFA. (b) **119** clients.

Étape 4 — Comparer. Le top client est cohérent avec l'ATTENDU (id 73). Les 119 clients du (b) sont plausibles (les 2 produits les plus chers sont vendus, mais pas par tous).

Conseil professionnel. Quand une question contient « **tous les** », « **chaque** », « **aucun** », passez directement à la **double négation** (NOT EXISTS ... NOT EXISTS). Ne perdez pas de temps à essayer **IN** ou **COUNT** : ils donnent le mauvais sens (« au moins un ») ou un résultat fragile. Le « tous les » = « aucun contre-exemple ».

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Panier moyen comme référence (scalaire dans SELECT)

```
SELECT id_vente, montant_ttc,
       (SELECT ROUND(AVG(montant_ttc)) FROM vente) AS panier_moyen
FROM vente
ORDER BY id_vente
LIMIT 3;
```

→ 3 lignes, chacune avec le panier moyen global (158 140 FCFA) en référence.

7.2 Question 2 — Le client n° 1 par CA (CTE)

```
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT id_client, nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 1;
```

→ id 73, Client_0073, 11 561 026 FCFA.

7.3 Question 3 — La vue mensuelle (120 lignes)

```
SELECT COUNT(*) AS nb_lignes,
       ROUND(MAX(ca_ttc)) AS plus_gros_mois
FROM v_ca_mensuel_magasin;
```

→ 120 lignes, plus gros mois 78 965 529 FCFA (magasin 3, 2025-08).

7.4 Question 4 — Les clients ayant acheté la catégorie 1 (IN)

```
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
WHERE v.id_produit IN (SELECT id_produit FROM produit WHERE id_categorie = 1);
```

→ 1 196 clients.

7.5 Question 5 — Les clients sans vente (NOT EXISTS)

```
SELECT COUNT(*) AS nb_clients_sans_vente
FROM client c
WHERE NOT EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client);
```

→ **0** (tous les clients ont au moins une vente).

7.6 Question 6 — Les 2 produits les plus chers (double négation)

```
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);
```

→ **119** clients.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	Sous-requête scalaire qui renvoie plusieurs lignes	erreur « more than one row returned »	LIMIT 1, ou transformer en sous-requête de liste.
2	Table dérivée sans alias	erreur de syntaxe	toujours un alias () t).
3	NOT IN avec des NULL dans la liste	0 ligne (au lieu du résultat attendu)	NOT EXISTS, ou filtrer les NULL de la liste.
4	Poser « tous les » avec IN	renvoie « au moins un » (trop de clients)	double négation NOT EXISTS ... NOT EXISTS.
5	Croire qu'une vue fige les données	la vue change quand la base change	une vue se recalcule à chaque lecture.
6	CTE complexe sans nom	requête illisible, impossible à maintenir	découper en CTE nommées (une par étape).

Attention. Le piège du NOT IN avec NULL est silencieux : si la sous-requête renvoie un NULL, x NOT IN (... , NULL) est NULL pour tout x qui n'est pas dans la partie non nulle — **aucune ligne ne passe**. C'est le bug le plus classique des « exclusions ». Privilégier NOT EXISTS (insensible aux NULL).

Attention. Une CTE n'est **pas une optimisation** : c'est du sucre de lisibilité. Sur certaines bases, une CTE peut même être **matérialisée** (calculée une fois, stockée) alors qu'une table dérivée serait inline. Ne pas écrire une CTE « pour aller plus vite » : on écrit une CTE **pour être lisible**.

9. Bonnes pratiques professionnelles

- Une CTE **par étape** de la logique : WITH brut AS (...), agrégé AS (...), résultat AS (...) puis SELECT ... FROM résultat.
- La **sous-requête scalaire** pour une valeur de référence (panier moyen, seuil) ; la **sous-requête de liste** pour un IN.

- **NOT EXISTS** plutôt que **NOT IN** dès que la liste peut contenir des **NULL**.
- « **Tous les** » = **double négation** : **NOT EXISTS (... NOT EXISTS (...))**.
- **Vue pour les étiquettes réutilisées** (CA mensuel, segmentation) ; CTE pour les étapes éphémères d'une seule requête.
- **Nommer les CTE** par leur rôle métier (`ca_par_client`, `top_produits`), pas `t1`, `t2`.

10. Exercice guidé — le top client et les acheteurs du top (15 min, /10)

Énoncé. (a) Renvoyer le client n° 1 par CA (id, nom, CA) avec une CTE. (b) Renvoyer le nombre de clients qui ont acheté les 2 produits les plus chers (double négation).

Grille de correction (/10).

Critère	Points
(a) CTE <code>ca_par_client</code> avec <code>GROUP BY + ORDER BY ca DESC LIMIT 1</code>	3
(a) Résultat : id 73, Client_0073, 11 561 026 FCFA	2
(b) Double négation <code>NOT EXISTS ... NOT EXISTS</code> correcte	4
(b) Résultat : 119 clients	1

Corrigé. Cf. §6, étape 1. (a) → 73, Client_0073, 11 561 026. (b) → 119.

11. Exercices autonomes

Exercice 1 — Le panier moyen en référence

Afficher les 5 premières ventes avec le panier moyen global (sous-requête scalaire dans le SELECT).

Exercice 2 — Le client n° 1 par CA

Avec une CTE, renvoyer l'id, le nom et le CA du client le plus dépensier.

Exercice 3 — La vue mensuelle

Nombre de lignes de `v_ca_mensuel_magasin` et le plus gros mois (en CA).

Exercice 4 — Les acheteurs de la catégorie 1

Le nombre de clients distincts ayant acheté au moins un produit de la catégorie 1 (**IN**).

Exercice 5 — Les clients sans vente

Le nombre de clients n'ayant **aucune** vente (**NOT EXISTS**).

Exercice 6 — Les acheteurs du top 2

Le nombre de clients ayant acheté les 2 produits les plus chers (double négation).

Exercice 7 — **NOT IN** VS **NOT EXISTS**

Réécrire l'exercice 5 avec **NOT IN** et expliquer en quoi **NOT EXISTS** est plus sûr.

Exercice 8 — Un COUNT à 0 : bug ou bon résultat ?

Cette requête renvoie **0**. Est-ce un bug ? Vérifiez par une méthode indépendante et expliquez pourquoi le 0 est (ou non) un résultat cohérent.

```
SELECT COUNT(*) FROM client c
WHERE c.id_client NOT IN (
  SELECT v.id_client FROM vente v
  WHERE v.montant_ttc > (SELECT AVG(montant_ttc) FROM vente)
);
```

12. Correction détaillée

Exercice 1. Cf. §7.1 → 5 lignes, chacune avec 158 140 FCFA en référence.

Exercice 2. Cf. §7.2 → id 73, Client_0073, 11 561 026 FCFA.

Exercice 3. Cf. §7.3 → 120 lignes, plus gros mois 78 965 529 FCFA.

Exercice 4. Cf. §7.4 → **1 196** clients.

Exercice 5. Cf. §7.5 → **0** client.

Exercice 6. Cf. §7.6 → **119** clients.

Exercice 7.

```
SELECT COUNT(*) FROM client c
WHERE c.id_client NOT IN (SELECT id_client FROM vente);
```

Sur ce socle, les deux renvoient 0 (tous les clients ont une vente). Mais `NOT IN` est fragile : si une seule `id_client` de la sous-requête était `NULL`, le `NOT IN` renverrait 0 ligne **même s'il existe des clients sans vente**. `NOT EXISTS` est insensible aux `NULL` et toujours correct.

Exercice 8. Ce 0 est un **bon résultat**, pas un bug. La requête compte les clients n'ayant **aucune** vente au-dessus de la moyenne globale (158 140 FCFA). Chaque client ayant en moyenne 42 ventes (50 008 / 1 200) et des montants répartis de 763 à 594 363 FCFA, il est quasi-certain que chacun a au moins une vente au-dessus de la moyenne : le `COUNT` renvoie donc légitimement 0. La vérification par méthode indépendante : la même question en `NOT EXISTS` renvoie aussi 0. Leçon : un `COUNT` à 0 n'est un bug que s'il contredit les données — on le contrôle par une requête indépendante (ici `NOT EXISTS`), et on se méfie du `NOT IN` seulement quand la liste sous-jacente peut contenir des `NULL` (pas le cas ici : `id_client` est non nul).

13. Mini-projet — « L'audit des requêtes composées » (40 min)

Produire l'audit des requêtes composées : 4 requêtes, chacune vérifiée à la main.

Requête	Lignes / valeur attendue
S1 : le client n° 1 par CA (CTE)	73, 11 561 026
S2 : le plus gros mois de la vue mensuelle	78 965 529
S3 : les acheteurs de la catégorie 1 (IN)	1 196
S4 : les acheteurs du top 2 (double négation)	119

Contrôles de cohérence. (i) $S3 \leq 1\,200$ (nombre de clients). (ii) $S4 \leq S3$ (les acheteurs du top 2 sont un sous-ensemble). (iii) $S2 \leq 7\,908\,259\,731 / 120$ (un mois ne dépasse pas le moyenne mensuelle du CA total).

Corrigé.

```

-- S1
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT id_client, nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 1;

-- S2
SELECT ROUND(MAX(ca_ttc)) AS plus_gros_mois
FROM v_ca_mensuel_magasin;

-- S3
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
WHERE v.id_produit IN (SELECT id_produit FROM produit WHERE id_categorie = 1);

-- S4
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);

```

Dans les faits. WITH (CTE), EXISTS/NOT EXISTS, IN et les vues sont **standard SQL** : identiques sur DuckDB, PostgreSQL et SQLite. Seule nuance : la **matérialisation des CTE** varie (PostgreSQL peut choisir de matérialiser ou d'inline ; DuckDB gère les CTE récursives ; SQLite aussi depuis la version 3.8.3). Pour la portabilité, éviter les CTE **récursives** dans du code qui doit tourner sur SQLite ancien.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 4 outils de structuration : la **sous-requête** (scalaire et de liste), la **CTE** (WITH, la sous-requête nommée), la **vue** (la CTE enregistrée), et la famille **EXISTS/IN** avec le pattern de **double négation** pour les « tous les ». La règle : une CTE par étape, **NOT EXISTS** plutôt que **NOT IN**, et la double négation pour les questions universelles.

Outil	Usage	Syntaxe	Piège
Sous-requête scalaire	1 valeur de référence	(SELECT AVG(...) FROM vente)	plusieurs lignes = erreur
Sous-requête de liste	un IN	WHERE x IN (SELECT ...)	NOT IN + NULL = 0 ligne
Table dérivée	sous-requête FROM	du FROM (SELECT ...) t	alias obligatoire
CTE (WITH)	étape nommée	WITH nom AS (SELECT ...)	sucré de lisibilité, pas d'optimisation
Vue	CTE enregistrée	CREATE VIEW ... AS SELECT ...	se recalcule, ne fige pas

Outil	Usage	Syntaxe	Piège
EXISTS NOT EXISTS	/ test d'existence	WHERE EXISTS (SELECT 1 ...)	corrélée, court-circuit
Double négation	les « tous les »	NOT EXISTS (... NOT EXISTS (...))	P6 : ne pas le faire avec IN

15. Résumé du chapitre

- **Sous-requête scalaire** (1 valeur, dans le `SELECT` ou le `WHERE`) ; **sous-requête de liste** (une colonne, pour un `IN`).
- **CTE (WITH)** = sous-requête **nommée**, éphémère, outil de **lisibilité** (une par étape). **Vue** = CTE **enregistrée**, réutilisable, **se recalcule** (ne fige pas).
- **IN** = « au moins un » ; **NOT IN** = « aucun », mais **fragile face aux NULL** (renvoie 0 ligne si la liste contient un `NULL`).
- **EXISTS/NOT EXISTS** = test d'existence, **insensible aux NULL**, souvent plus sûr que `IN/NOT IN`.
- **Double négation** (`NOT EXISTS ... NOT EXISTS`) = le pattern canonique des « **tous les** » (P6) : « aucun contre-exemple ». `IN` et `COUNT` ne le font pas.
- **Nuance de portabilité** : CTE, vues, `EXISTS` standard ; la matérialisation des CTE et les CTE récursives varient entre SGBD.

16. À retenir

- Une CTE **par étape**, nommée par son rôle métier : lisibilité avant tout.
- **Sous-requête scalaire** = 1 valeur de référence ; **de liste** = un `IN`.
- **NOT EXISTS plutôt que NOT IN** (sécurité face aux `NULL`).
- « **Tous les** » = **double négation** : `NOT EXISTS (... NOT EXISTS (...))` — jamais `IN`.
- Une **vue se recalcule** : c'est une encapsulation, pas un snapshot.

À retenir. La sous-requête est une **question dans une question** ; la CTE la **nomme** ; la vue l'**enregistre** ; `EXISTS/IN` testent l'**appartenance**. Et dès que la question dit « **tous les** », on écrit la **double négation** : nier l'existence d'un contre-exemple est la seule écriture sûre.

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Différence entre une sous-requête scalaire et une sous-requête de liste ?
2. Quelle est la différence entre une CTE et une vue ?
3. Pourquoi `NOT IN` peut-il renvoyer 0 ligne, et comment éviter ce bug ?
4. Comment écrit-on « les clients qui ont acheté **tous** les produits X » ?
5. Une vue fige-t-elle les données ?

Réponses :

1. La sous-requête **scalaire** renvoie **1 valeur** (1 ligne, 1 colonne) — elle s'utilise dans le `SELECT` ou le `WHERE`. La sous-requête **de liste** renvoie une **colonne de valeurs** — elle s'utilise dans un `IN / NOT IN`.
2. Une **CTE** est une sous-requête **nommée**, **éphémère** (n'existe que durant la requête), déclarée par `WITH`. Une **vue** est une requête **enregistrée** dans le schéma, **réutilisable** par n'im-

porte quelle requête, interrogée comme une table. Les deux se recalculent ; la vue persiste, la CTE non.

3. Parce que si la sous-requête du `NOT IN` renvoie **un NULL**, alors `x NOT IN (... , NULL)` est `NULL` (ni vrai ni faux) pour tout `x` non présent — **aucune ligne ne passe**. Éviter avec `NOT EXISTS` (insensible aux `NULL`) ou en filtrant les `NULL` de la liste.
4. Par la **double négation** : `WHERE NOT EXISTS (SELECT 1 FROM produit X WHERE NOT EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client AND v.id_produit = p.id_produit))` — « il n'existe **aucun** produit X que le client **n'a pas** acheté ». `IN` (« au moins un ») et `COUNT` (fragile) ne le font pas.
5. **Non**. Une vue **se recalcule** à chaque lecture : si la table sous-jacente change, la vue change. C'est une encapsulation de requête, pas un snapshot de données.

MANUEL PROFESSIONNEL · ÉDITION 2026

M08 — Python pour l'analyse de données

Huit chapitres · 30 h · fil rouge vente.csv

Analyse de données appliquée & Business Intelligence
8 partie(s) assemblée(s) · document de production

Table des matières

MODULE M08.C01 — INSTALLER SANS STRESS : PYTHON, PIP, ENVIRONNEMENTS VIRTUELS, VS CODE ET JUPYTER, EXÉCUTER SON PREMIER SCRIPT	7
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la conversation avec la machine	8
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 Les 4 couches — ce qu'on installe, et pourquoi	9
5.2 Installer Python 3.13 — la case qui sauve	10
5.3 Installer des paquets — pip, la boutique	10
5.4 L'environnement virtuel — la trousse scellée	11
5.5 VS Code — l'atelier de travail	12
5.6 Jupyter et le noyau — la table à dessin qui se souvient	12
5.7 Le premier script — premier.py	13
5.8 Les 5 erreurs d'installation les plus courantes	13
6. Exemple concret — le script qui répond à une question métier	14
7. Démonstration pas à pas — 5 étapes sur le fil rouge	14
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le rapport d'installation » (15 min, /10)	16
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P1 — « L'audit d'environnement » (1 h)	17
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C02 — LES FONDATIONS SANS DONNÉES : VARIABLES, TYPES, OPÉRATEURS, ENTRÉES/SORTIES, COMMENTAIRES, ERREURS	20
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — l'étiquette, la famille, la porte	21
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 La variable — l'étiquette qui se re-colle	22
5.2 Les 4 types de base — et la conversion à la porte	22
5.3 Les opérateurs — l'arithmétique du comptoir	23
5.4 input() et print() — la porte et la vitrine	23
5.5 Les commentaires — le « pourquoi », pas le « quoi »	24
5.6 Lire une erreur — l'anatomie, en 3 protestations réelles	24
5.7 None et les valeurs « fausses » en condition	25

6. Exemple concret — le calculateur de remise	26
7. Démonstration pas à pas — 5 étapes sur le fil rouge	14
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le ticket de caisse » (15 min, /10)	28
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P2 — « La caisse » (1 h)	29
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C03 — DÉCIDER ET RÉPÉTER : IF/ELIF/ELSE, BOUCLES FOR/WHILE, BREAK/CONTINUE, COMPRÉHENSIONS	32
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — le carrefour, le tapis, le « tant que »	33
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 if/elif/else — l'échelle du comptoir	33
5.2 La boucle for — le tapis roulant des 5 magasins	34
5.3 La boucle while — le « tant que » du crédit	35
5.4 break et continue — la porte de sortie et le « passe »	36
5.5 Les compréhensions — le tapis avec filtre, en une ligne	36
5.6 zip — deux listes, un seul parcours	37
6. Exemple concret — le contrôle qualité express	38
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le registre à trois modes » (15 min, /10)	41
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P3 — « Le rapport de qualité complet » (1 h)	43
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C04 — RANGER : LISTES, TUPLES, CHÂÎNES, DICTIONNAIRES, ENSEMBLES, INDEX, TRIS	45
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la case, l'enveloppe, l'étagère, le tamis	46

4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 La liste — la case numérotée	47
5.2 Le tuple — l’enveloppe scellée	47
5.3 La chaîne — le format français en méthodes	48
5.4 Le dictionnaire — l’étagère étiquetée	48
5.5 L’ensemble — le tamis	49
5.6 Index, tranches, tris	50
6. Exemple concret — les 5 magasins dans 4 contenants	51
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « les 5 questions des magasins » (15 min, /10)	54
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P4 — « L’annuaire des magasins » (1 h)	55
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C05 — ORGANISER : FONCTIONS, ARGUMENTS, PORTÉE, DOCS-TRING, MODULES, TRY/EXCEPT, FICHIERS	58
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la machine à café	59
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 La fonction — le return est la sortie, pas le print	59
5.2 Les arguments — positionnels, nommés, par défaut	60
5.3 La portée — la cuisine et le comptoir	61
5.4 La docstring — la notice sur la machine	62
5.5 Modules et paquets — l’import en tête	62
5.6 Le try/except — le filet sous la fuite prévue	63
5.7 Lire une traceback — les 3 couches	63
6. Exemple concret — le nettoyeur de montants	65
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « le nettoyeur de montants » (15 min, /10)	68
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P5 — « Le registre de caisse » (1 h)	70
14. Boîte à outils du chapitre	18

15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C06 — NUMPY UTILE ET SUFFISANT : TABLEAUX, VECTEURISATION, STATISTIQUES, NAN, WHERE, BROADCASTING	73
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — le convoi, la barrière, le camion vide	74
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 Le tableau — le convoi homogène	75
5.2 La vecteurisation — la barrière levée	75
5.3 Les statistiques — min, max, percentiles, médiane	76
5.4 nan — le camion sans marchandise	76
5.5 np.where — le contrôle qui étiquette	77
5.6 Le broadcasting — la règle d'alignement	77
5.7 La mesure — la boucle de C03 face à la ligne	78
6. Exemple concret — les 4 calculs du fil rouge	79
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « les 4 statistiques du comptoir » (15 min, /10)	82
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P6 — « Le rapport de colonne » (1 h)	83
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C07 — PANDAS I : SERIES, DATAFRAME, READ_CSV/READ_EXCEL, LOC/ILOC, FILTRES, NOUVELLES COLONNES, DESCRIBE, VALEURS MANQUANTES	86
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — le classeur d'indexation	87
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 read_csv — l'import des 8 CSV réels	88
5.2 head, shape, dtypes — la première lecture	88
5.3 L'index — la réduction, pas une colonne	89
5.4 loc vs iloc — le tableau des 4 cas	90
5.5 Les filtres — le WHERE en masque	91
5.6 Les nouvelles colonnes — le SELECT ... AS	92
5.7 describe() — la carte d'identité statistique	92

5.8 Les valeurs manquantes — l’audit du dossier	92
5.9 read_excel — le même objet, un autre format	93
5.10 Les 4 écritures obsolètes — 1 ^{re} save des 12	94
6. Exemple concret — l’audit du dossier en 8 lignes	95
7. Démonstration pas à pas — 6 étapes sur le fil rouge	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « la vue des 120 premières ventes » (15 min, /10)	98
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P7 — « L’audit du dossier » (1 h)	100
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M08.C08 — PANDAS II : GROUPBY/TRANSFORM/MERGE, PIVOT_TABLE, DATES, APPLY, EXPORT EXCEL — ET L’ANNEXE MIGRATION 2.3 → 3.X	103
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les classeurs et le classement	104
4. Vocabulaire essentiel	8
5. Cours approfondi	9
5.1 groupby + agg — le GROUP BY en une ligne	105
5.2 transform — la moyenne du groupe, remise sur chaque ligne	106
5.3 merge — le JOIN de M07, et la table vide qui rend tout NaN	106
5.4 pivot — la vue mensuelle de M07, reconstruite sans SQL	107
5.5 Les dates — l’index qui porte un sens, et l’alias 'M' mort	108
5.6 apply — la fonction de C05 sur une colonne, et son prix mesuré	108
5.7 L’export Excel — le classeur multi-feuilles, le livrable E3	109
5.8 La leçon des centimes — 1 à 2 FCFA sur 7,9 milliards	109
5.9 L’annexe migration 2.3 → 3.x — les 12 écritures, avant/après	110
6. Exemple concret — les 10 questions de M07.P, exécutées en pandas	111
7. Démonstration pas à pas — 6 étapes du bout en bout	112
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — « l’année 2025 du magasin 4 » (15 min, /10)	114
11. Exercices autonomes	16
12. Correction détaillée	17
13. Mini-projet M08.P — « Le script qui fait le travail de trois matinées » (2 h)	116
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19

Module M08.C01 — Installer sans stress : Python, `pip`, environnements virtuels, VS Code et Jupyter, exécuter son premier script

Outils : Python 3.13.14, `pip` 26.1.2, `venv` (natif Python 3.13), VS Code (cité), Jupyter (noyau exécuté). **Durée indicative :** 4 h. **Niveau :** N2 → N3. **Prérequis :** M05.C04 (vous avez vu du pandas en M05) ou grande aisance Excel ; M07 (aisance SQL) — le parallèle SQL/pandas s'active à C07, ici il est seulement annoncé.

L'idée du chapitre. En M05 vous avez vu pandas produire un chiffre ; en M07 vous l'avez fait produire par le SGBD. En M08, c'est **vous** qui tenez la machine. Mais avant la première ligne de code, il faut que la machine **vous écoute** : un interpréteur installé proprement, des paquets au bon endroit, et un projet isolé du reste de la machine. C01 pose les **4 couches** de l'environnement (la machine, l'interpréteur, les paquets, le projet), installe chaque couche, et termine par **votre premier script** exécuté deux fois (terminal, puis notebook) — avec les **5 erreurs d'installation** qui font abandonner plus d'apprenants que n'importe quelle notion de Python. Le chapitre ne vous apprend pas encore à programmer : il vous apprend à **ne pas être bloqué par l'environnement** quand vous voulez programmer.

Matériel de l'atelier — Python 3.13.14 · `pip` 26.1.2 · pandas 2.2.3 (environnement d'écriture) · numpy 2.3.5 · Jupyter (noyau exécuté). Toutes les commandes de ce chapitre ont été exécutées dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier** (Linux). Sur votre machine Windows, la commande est presque toujours identique — chaque écart (`python` vs `python3`, `.venv\Scripts\activate` vs `source .venv/bin/activate`) est signalé au passage.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Distinguer les **4 couches** de l'environnement Python (machine, interpréteur, paquets, projet) et dire quelle couche cause quel symptôme.
2. Installer Python 3.13 **sans casser le système** (la case « Add to PATH », la vérification en ligne de commande).
3. Installer des paquets avec `pip` — et **figer** la liste avec `requirements.txt`.
4. Créer et activer un **environnement virtuel** — la boîte à outils scellée du projet.
5. Exécuter votre premier script dans un **terminal**, et les mêmes cellules dans **Jupyter** — en comprenant ce que le noyau se souvient, et ce qu'il oublie.
6. Diagnostiquer les **5 erreurs d'installation** les plus courantes par leur sortie exacte.

2. Pourquoi cette notion est importante

C'est le chapitre le plus court du module en heures (4 h) et le plus long en « risques » : l'installation est le **premier point d'abandon** de tout parcours Python. Un apprenant qui perd trois jours sur un « `pip` installe dans le mauvais Python » perd plus que n'importe quelle notion de boucle ou de dictionnaire ne lui en coûtera — parce que le doute empoisonne tout ce qui vient après (« c'est peut-être moi qui n'y arrive pas », alors que c'est le PATH qui est vide).

Deuxième raison, professionnelle : l’environnement virtuel n’est pas une précaution de débutant, c’est **l’habitude du métier**. Le script que vous livrerez en M08.P (« le travail de trois matières ») devra tourner sur la machine du collègue : sans `requirements.txt` et un `venv`, il tourne chez vous et nulle part ailleurs. C01 installe cette habitude **avant** la première ligne de code utile, pas après le premier incident.

Enfin, l’enjeu de version : vous verrez plus bas qu’une installation fraîche de `pandas` installe la version **3.0.6**, tandis que l’atelier (et ce manuel) s’appuie sur la **2.2.3** — les deux coexistent, et le manuel les documente toutes les deux (la décision est au §1.7 du plan M08). Comprendre pourquoi les versions cohabitent maintenant, c’est ne pas paniquer plus tard quand une sortie ne correspond pas.

3. Explication simple — la conversation avec la machine

Prenons l’analogie que vous connaissez déjà :

- **Excel** est un programme qui sait faire des additions et des tris : vous lui parlez dans **sa** langue (les formules), il fait le reste.
- **Python** est un **traducteur généraliste** : il ne « sait » rien par lui-même, il **lit vos instructions ligne par ligne** et les exécute dans l’ordre. Votre fichier `.py`, c’est une liste numérotée d’ordres ; l’interpréteur, c’est la personne qui les lit à voix haute, un par un.
- **pip** est la **boutique** : elle installe des paquets — des boîtes d’instructions déjà écrites par d’autres (`pandas` sait faire des tableaux, `numpy` sait faire des calculs en bloc). Sans la boutique, Python ne sait faire que l’arithmétique et quelques fonctions de base.
- **L’environnement virtuel** (`venv`) est la **trousse à outils scellée** du projet : la boutique livre les boîtes dans la trousse, pas dans le placard commun de la maison. Le projet d’à côté n’y touche pas ; vous pouvez jeter la trousse sans vider le placard.
- **Jupyter** est une **table à dessin** découpée en cases (les cellules) : chaque case est exécutée à part, mais le **noyau** — le traducteur qui tourne derrière la table — **se souvient** de ce que les cases précédentes ont défini. C’est formidable (pas besoin de tout relancer) et piégeux (l’ordre d’exécution compte, pas l’ordre d’écriture — nous le prouverons à §5.6).

La conversation, à partir d’aujourd’hui, est celle-ci : vous écrivez des ordres (`.py` ou cellules), l’interpréteur les lit, et soit il répond (sortie, fichier, graphique), soit il **proteste** (erreur — et la protestation est lisible, nous l’entraînerons à C05). Tout le reste de ce chapitre sert à ce que cette conversation **puisse commencer**.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
interpréteur interpreter	— le programme qui lit le Python et l’exécute ligne à ligne.	croire qu’il « traduit en avance » : non, il exécute au fil, ce qui explique les erreurs de ligne N alors que le bug est à la ligne N-3.
paquet — package	un bloc de code réutilisable installé séparément (<code>pandas</code> , <code>numpy</code>).	confondre paquet (la boîte, ce que <code>pip</code> installe) et module (ce qu’on importe) : un paquet contient un ou plusieurs modules.
environnement virtuel — virtual environment	une copie isolée de Python + ses paquets, réservée à un projet.	installer dans le Python système « pour cette fois » : le projet de demain casse celui d’aujourd’hui.
noyau — kernel	le moteur d’exécution derrière Jupyter, qui garde en mémoire les variables entre les cellules.	croire qu’une cellule « voit » ce que la cellule écrite au-dessus a fait : seul l’ordre d’exécution compte.

Français — English	Définition en une phrase	Piège à éviter
PATH	la liste d'endroits où Windows/Windows cherche les exécutables quand vous tapez <code>python</code> .	croire que l'installation a « marché » parce que le raccourci existe : seul <code>python --version</code> dans le terminal fait foi.
exigence requirement	— la ligne <code>paquet==version</code> d'un <code>requirements.txt</code> qui fige une installation reproductible.	figer « trop » : <code>requirements.txt</code> liste ce que le projet a besoin , pas tout ce que le <code>venv</code> contient par effet d'entraînement.

5. Cours approfondi

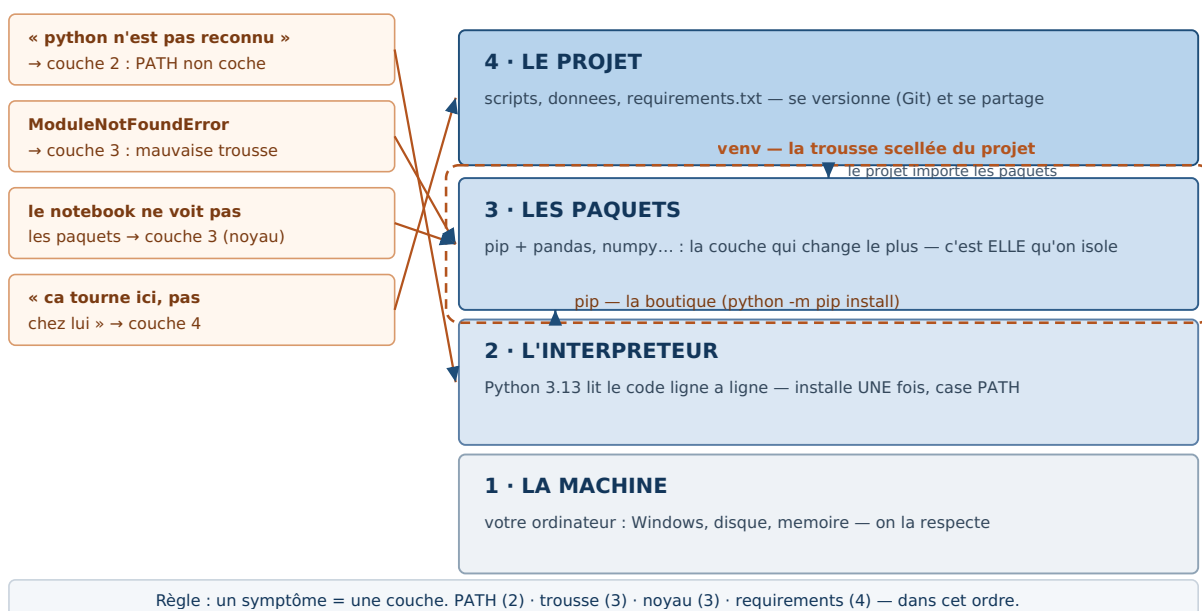
5.1 Les 4 couches — ce qu'on installe, et pourquoi

L'environnement Python se pense en **4 couches empilées** (figure du chapitre) :

Les 4 couches de l'environnement Python

Chaque symptôme d'installation se rattache à UNE couche — la nommer, c'est diagnostiquer.

Le diagnostic par couche



1. **La machine** — votre ordinateur (Windows ici), sa mémoire, son disque. On ne l'« installe » pas, on la **respecte** : on ne met pas de `venv` sur le disque C : plein, on ne met pas 40 Go de paquets inutiles sur la machine de la formation.
2. **L'interpréteur** — le runtime Python 3.13. Une seule installation suffit pour tout l'ordinateur ; c'est la couche qu'on installe **en dernier recours** (elle est fragile : deux versions mélangées = paquets perdus).
3. **Les paquets** — `pip` et ce qu'il installe (`pandas`, `numpy`, ...). C'est la couche qui change le plus vite ; c'est **elle** qu'on isole.
4. **Le projet** — vos scripts, vos données, votre `requirements.txt`. C'est la couche qui se versionne (Git) et se partage.

Définition. interpréteur — interpreter — le programme qui lit le code Python et l'exécute ligne à ligne, sans phase de « compilation » préalable visible : le fichier `.py` est lu, exécuté, relu si vous le modifiez. C'est la couche 2, installée une fois pour toutes.

Chaque symptôme d'installation se rattache à **une** couche : « `python` n'est pas reconnu » = couche 2 (PATH) ; « `ModuleNotFoundError` » = couche 3 (paquet absent **ou** mauvais venv) ; « le notebook ne voit pas mes paquets » = couche 3 mal pointée (mauvais noyau) ; « ça tourne ici, pas chez lui » = couche 4 (requirements non figés). Apprendre à nommer la couche, c'est diviser le temps de diagnostic par quatre.

5.2 Installer Python 3.13 — la case qui sauve

Windows-first, comme le lecteur du manuel :

1. Télécharger l'installateur officiel depuis **python.org** (3.13.x — la version de référence du module ; la date de vérification est le 19/09/2026).
2. Lancer l'installateur. **Première case, en bas de l'écran : « Add python.exe to PATH » — la cocher.** C'est la case n° 1 des erreurs d'installation : sans elle, le terminal ne trouvera jamais `python`.
3. « Install Now ». L'installateur termine en quelques minutes.

La **vérification qui fait foi** (point de contrôle, identique sur Windows et Linux) :

```
C:\Users\vous> python --version
Python 3.13.14
```

Dans l'atelier, l'interpréteur s'adresse en `python3` (convention Linux) ; la sortie publiée est celle du 19/09/2026 :

```
$ python3 --version
Python 3.13.14
```

Attention. Sur un Windows qui a déjà eu Python installé par une autre application, `python` peut pointer vers l'**App Installer** de la Microsoft Store (qui ouvre le Store au lieu d'exécuter). Symptôme : `python --version` ouvre une fenêtre du Store. Remède : installer la version officielle, et vérifier que `python -m site` répond par un chemin `C:\Users\...\AppData\Local\Programs\Python\...` et non par une page du Store.

5.3 Installer des paquets — `pip`, la boutique

`pip` est le gestionnaire de paquets de Python. Sa version et son lieu d'installation, vérifiés dans l'atelier le 19/09/2026 :

```
$ pip3 --version
pip 26.1.2 from /usr/local/lib/python3.13/site-packages/pip (python 3.13)
```

Trois choses à lire dans cette sortie : la version de `pip` (26.1.2), **le dossier** où `pip` installe (`.../site-packages` — c'est là que logent les paquets), et **le Python** auquel `pip` est attaché (`python 3.13` — la couche 3 sert la couche 2, et une seule).

Installer, c'est une ligne :

```
$ python3 -m pip install pandas numpy
```

(`python3 -m pip` plutôt que `pip3` : ça garantit que c'est **ce** Python qui installe — le réflexe professionnel, surtout quand plusieurs Pythons coexistent.)

Dans l'atelier, la sortie réelle du 19/09/2026 (`pandas` déjà présent, version figée) :

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/site-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/site-packages (2.3.5)
```

Conseil professionnel. Figez toujours la version dans `requirements.txt` avec `==` (`pandas==2.2.3`), jamais sans : un « `pip install -r` » qui installe la dernière version disponible le 14 mars 2027 peut casser un script qui tournait depuis le 19 septembre 2026. La version figée est une **date de péremption connue** ; la version flottante, une inconnue.

5.4 L'environnement virtuel — la trousse scellée

Créer la trousse, c'est une ligne ; dans l'atelier (exécuté le 19/09/2026) :

```
$ python3 -m venv /tmp/venv_m08
```

Sur Windows, même chose : `python -m venv .venv` (le venv vit **dans** le dossier du projet, par convention). L'activer, c'est **changer de Python temporairement** :

```
# Linux (atelier)      $ source .venv/bin/activate
# Windows              .venv\Scripts\activate
```

Une fois activé, le préfixe du terminal change — c'est le **seul** signe visible que vous êtes « dans » (sur Windows, `(.venv)` apparaît au début de chaque ligne).

La démonstration de l'isolation, exécutée dans l'atelier — le venv vierge **ne connaît que pip** :

```
$ /tmp/venv_m08/bin/pip list
Package Version
-----
pip      26.1.2
```

Et pandas n'y est **pas** — l'erreur est publique, avec sa sortie exacte :

```
$ /tmp/venv_m08/bin/python -c "import pandas"
File "<string>", line 1, in <module>
    import pandas
ModuleNotFoundError: No module named 'pandas'
```

Définition. environnement virtuel — virtual environment — une copie isolée de l'interpréteur + ses paquets, limitée à un projet : les installations dans le venv n'affectent pas le Python système ni les autres venvs. C'est la couche 3, isolée de la couche 2.

Installer **dans la trousse**, puis vérifier — sortie réelle de l'atelier le 19/09/2026 :

```
$ /tmp/venv_m08/bin/pip install pandas
$ /tmp/venv_m08/bin/python -c "import pandas; print('pandas', pandas.__version__, 'importe sans erreur')"
pandas 3.0.6 importe sans erreur
```

Dans les faits. Une installation **fraîche** de pandas installe la version **3.0.6** (la dernière publiée au 19/09/2026) ; l'environnement d'écriture de ce manuel est **figé** sur la **2.2.3** — la version sur laquelle les M01-M07 ont été mesurés. Les deux coexistent sans se gêner (c'est tout l'intérêt du venv). Le module enseigne les patterns **valides sur les deux**, et signale les 12 écritures qui ne le sont plus (C07 §9 et C08 §8) ; un apprenant sur pandas 3.x ne sera donc jamais « en faux » par rapport au manuel — c'est la décision de conception du plan M08 (§1.3).

Et figer la trousse, c'est l'**exigence** — `pip freeze` liste ce qui est installé ; on n'en garde que ce que le projet a **besoin** :

```
$ /tmp/venv_m08/bin/pip freeze
numpy==2.5.3
pandas==3.0.6
python-dateutil==2.9.0.post0
six==1.17.0
```

`python-dateutil` et `six` sont des **effets d'entraînement** (`pandas` en a besoin, vous ne les avez pas demandés) ; un `requirements.txt` propre les liste quand même — sinon `pip install -r` ne reconstruirait pas l'environnement.

Définition. exigence — requirement — la ligne `paquet==version` d'un fichier `requirements.txt` : la liste figée des paquets (et versions) qu'un projet a besoin, qui permet à `pip install -r requirements.txt` de reconstruire **exactement** le même environnement sur une autre machine.

5.5 VS Code — l'atelier de travail

VS Code est l'éditeur du manuel (cité, non exécuté dans l'atelier — l'atelier travaille en ligne de commande, ce qui est le même moteur). Ce qu'il faut savoir en 4 points :

1. **Ouvrir un dossier** (pas un fichier) : c'est le dossier qui est le projet — le terminal intégré s'ouvre **dedans**, et c'est là que le `venv` doit vivre.
2. **Le terminal intégré** (`Ctrl+`) : il hérite du dossier du projet ; activer le `venv` **une fois** dedans, et tous les `python` du terminal pointent vers la trousse.
3. **Exécuter un script** : bouton « Run » en haut à droite (ou `F5`) — VS Code lance `python nom_fichier.py` et affiche la sortie en dessous.
4. **Le pointeur de Python** : en bas à droite, VS Code affiche le Python utilisé. S'il indique le Python système alors que vous êtes dans un `venv`, cliquez dessus et choisissez le `venv` — c'est la couche 3 mal pointée, variante éditeur.

Attention. Le terminal intégré et l'éditeur peuvent **ne pas se souvenir l'un de l'autre** : activer le `venv` dans le terminal n'active pas automatiquement l'interpréteur du bouton « Run ». Vérifiez le pointeur en bas à droite **après** chaque activation.

5.6 Jupyter et le noyau — la table à dessin qui se souvient

Jupyter (JupyterLab ou le notebook classique) exécute du Python **par cellules**. Le moteur derrière est le **noyau** : un interpréteur qui tourne en permanence et **garde la mémoire** de tout ce que les cellules exécutées ont défini.

Définition. noyau — kernel — le processus d'exécution derrière Jupyter : il exécute chaque cellule demandée et **conserve l'état** (variables, imports) entre les exécutions. Réinitialiser le noyau, c'est oublier tout l'état — les cellules gardent leur texte, plus leur effet.

La preuve par l'exécution — un notebook de 4 cellules, exécuté dans l'atelier le 19/09/2026 (`jupyter nbconvert --execute`, le même moteur que JupyterLab utilise pour les cellules) :

```
# cellule 1
print("Bonjour depuis Jupyter.")
```

```
sortie : Bonjour depuis Jupyter.
```

```
# cellule 2
prix = 1500
remise = 0.10
print("Prix net :", prix * (1 - remise))
```

```
sortie : Prix net : 1350.0
```

```
# cellule 3 – écrivez AVANT la cellule 4, exécutée dans l'ordre écrit
print("x =", x)
```

```
erreur : NameError: name 'x' is not defined
```

```
# cellule 4
x = 42 # je la crée MAINTENANT, dans l'ordre d'exécution
```

La cellule 3 a échoué **parce que la cellule 4 n'avait pas encore été exécutée** — pas parce qu'elle était « en dessous » dans l'ordre d'écriture. C'est la leçon du noyau : **l'ordre d'exécution fait la mémoire, pas l'ordre du papier**. Re-lancer la cellule 3 après la cellule 4, et elle s'exécute — sans rien avoir changé d'autre.

Dans les faits. L'atelier exécute les notebooks par `jupyter nbconvert --execute` (même noyau, sortie capturée pour les chapitres) ; sur votre machine, vous lanceriez `jupyter lab` (ou `jupyter notebook`) dans le **dossier du projet, venv activé** — le pointeur de noyau de JupyterLab doit indiquer le Python du venv, sinon c'est la même erreur de couche 3 que \$5.5, variante notebook.

5.7 Le premier script — `premier.py`

Le plus petit script utile du monde : il salue, il demande, il répond.

```
# premier.py – mon premier script
print("Bonjour, je suis un script Python.")
nom = input("Comment vous appelez-vous ? ")
print(f"Enchanté, {nom}.")
```

Exécuté dans le terminal de l'atelier le 19/09/2026 (la réponse `Aïcha` fournie par `echo` — le même effet que de la taper) :

```
$ echo "Aïcha" | python3 /tmp/premier.py
Bonjour, je suis un script Python.
Comment vous appelez-vous ? Enchanté, Aïcha.
```

Trois choses dans ces 3 lignes : un `print` (sortie), un `input` (entrée — et il renvoie **toujours** une chaîne, même 42), et une **f-string** (le `f"... {nom} ..."` qui insère la valeur — le formatage que vous ferez toute la formation, montants compris, règle M01). Le même script, collé dans une cellule Jupyter, s'exécute identiquement — sauf que le noyau **se souvient** de `nom` pour la cellule suivante.

5.8 Les 5 erreurs d'installation les plus courantes

Le tableau de diagnostic du chapitre — chaque ligne est un **cas réel** rencontré par des débutants ; les deux premières sont citées (elles ne se produisent que sur Windows mal installé), les trois dernières ont leur **sortie exacte publiée** (exécutées dans l'atelier).

#	Symptôme	Cause (couche)	Remède
1	python : le terme ... n'est pas reconnu (Windows)	PATH non coché à l'installation (couche 2)	réinstaller en cochant « Add python.exe to PATH », ou ajouter le chemin du Python manuellement ; vérifier par python --version.
2	python --version ouvre le Store (Windows)	l'alias Microsoft Store précède le vrai Python (couche 2)	python -m site doit montrer un chemin AppData\Local\Programs\Python ; sinon désactiver l'alias App Installer (Paramètres → Applis → Alias).
3	ModuleNotFoundError: No module named 'pandas' alors que « pandas est installé »	paquet dans le Python système, venv actif sans pandas (couche 3) — exécutée : sortie publiée au \$5.4	pip list dans le venv actif ; si pandas n'y est pas, pip install pandas. Le paquet est « installé » — mais dans l'autre trousse.
4	Le notebook dit ModuleNotFoundError, le terminal, lui, importe sans problème	noyau Jupyter attaché au mauvais Python (couche 3)	en bas à droite de JupyterLab : choisir le noyau du venv ; sinon python -m ipykernel install --user --name monvenv.
5	pip install « réussit », mais le projet ne voit pas le paquet	pip et python sont deux Pythons différents (couche 2/3)	toujours python -m pip install ... : le -m pip force l'installation dans le Python qu'on exécute. Vérifier par python -m site avant et après.

6. Exemple concret — le script qui répond à une question métier

La question, en langage de comptoir : « l'article coûte 1 500 FCFA, la remise est de 10 %, quel est le prix net ? » En Excel (M03), c'est une cellule `=1500*(1-10%)`. En SQL (M07), c'est une colonne calculée dans le `SELECT`. En Python, c'est **un script** — exécuté dans l'atelier le 19/09/2026 :

```
# prix_net.py
prix = 1500
remise = 0.10
print(f"Prix net : {prix * (1 - remise):,.2f} FCFA")
```

```
$ python3 /tmp/prix_net.py
Prix net : 1 350.00 FCFA
```

La valeur est celle du socle du chapitre ($1\,500 \times 90\% = 1\,350$, clé `m08p_demo_prix_net`) — elle servira de **référence de relecture** : si votre machine renvoie autre chose, l'environnement ne tourne pas comme prévu, et c'est le chapitre entier qu'il faut rejouer. Le parallèle SQL, déjà : en M07, le même calcul était `SELECT prix_vente_ht * (1 - 0.10) AS prix_net FROM produit` ; — la formule est la même, le lieu change (une ligne de table vs une ligne de script).

7. Démonstration pas à pas — 5 étapes sur le fil rouge

De zéro à « mon premier script tourne », les 5 étapes, avec la sortie réelle de chaque commande (exécutées dans l'atelier le 19/09/2026).

Étape 1 — Vérifier l'interpréteur (couche 2) :

```
$ python3 --version
Python 3.13.14
```

Interprétation : la machine a un Python 3.13 — la version de référence du module est installée.

Étape 2 — Vérifier la boutique (couche 3) :

```
$ pip3 --version
pip 26.1.2 from /usr/local/lib/python3.13/site-packages/pip (python 3.13)
```

Interprétation : `pip 26.1.2` sert ce Python 3.13, et installe dans `site-packages` — les deux couches se correspondent.

Étape 3 — Sceller la trousse du projet (couche 3 isolée) :

```
$ python3 -m venv .venv
$ source .venv/bin/activate      # Windows : .venv\Scripts\activate
(.venv) $ python --version
Python 3.13.14
```

Interprétation : le préfixe `(.venv)` confirme l'activation ; le `python` du terminal pointe désormais vers la trousse.

Étape 4 — Livrer les paquets dans la trousse :

```
(.venv) $ python -m pip install pandas numpy
(.venv) $ python -c "import pandas, numpy; print(pandas.__version__, numpy.__version__)"
```

Interprétation : `import` sans erreur = les paquets sont lisibles par ce Python. Sur une machine neuve en 2026, cette installation livrerait `pandas 3.0.6` (dernière publiée) — c'est voulu : le chapitre C08 enseigne les deux versions.

Étape 5 — Exécuter le premier script (couche 4) :

```
(.venv) $ echo "Aïcha" | python premier.py
Bonjour, je suis un script Python.
Comment vous appelez-vous ? Enchanté, Aïcha.
```

Interprétation : la conversation est ouverte — sortie lisible, entrée acceptée, variable en mémoire.

8. Erreurs fréquentes

1. « **J'ai installé Python, `python` ne répond pas.** » → couche 2 : la case `PATH`. Vérification : `python --version` (Windows) / `python3 --version` (Linux). Si le terminal n'a pas de réponse, rien d'autre ne marchera — ne passez pas aux paquets.
2. « **`pip install pandas` dit OK, mais `import pandas` échoue.** » → deux Pythons différents : le `pip` d'un côté, le `python` de l'autre. Toujours `python -m pip install ...`.
3. « **Ça tourne ici, pas chez mon collègue.** » → couche 4 : pas de `requirements.txt` (ou versions non figées). `pip freeze > requirements.txt`, partage, `pip install -r requirements.txt`.
4. « **Le notebook a oublié mes variables.** » → le noyau a été réinitialisé (bouton « Restart ») — ou la cellule a été exécutée **avant** la définition. L'ordre d'exécution fait la mémoire (§5.6).
5. « **J'ai installé 3 versions de pandas dans 3 venvs, je ne sais plus où j'en suis.** » → c'est **normal** : c'est le but. Chacun des venvs a sa trousse ; le préfixe du terminal et le pointeur de Jupyter disent lequel est actif. Le problème serait l'inverse (tout dans le Python système).
6. **`SyntaxError` dès la première ligne, sur du code « qui vient du manuel ».** → le copier-coller a mangé des caractères (guillemets typographiques « » au lieu de "). Recopier la ligne à la main : si elle passe, c'était le collage, pas le code.

9. Bonnes pratiques professionnelles

1. **Jamais d'installation dans le Python système** pour un projet : venv d'abord, install ensuite. L'exception acceptée : les paquets d'outillage (pip lui-même, virtualenv).
2. **requirements.txt dès le premier paquet** — pas « quand le projet sera stabilisé » : le fichier qui n'a pas été tenu à jour pendant un mois est un fichier faux.
3. **Le venv ne se versionne pas** : il se **reconstruit** (`python -m venv .venv + pip install -r requirements.txt`). Le `.venv` va dans le `.gitignore` — le partage d'un venv binaire entre machines est une source de pannes silencieuses (chemins absolus dedans).
4. **La version figée est une date, pas une opinion** : `pandas==2.2.3` dit « tel que vérifié le 19/09/2026 ». Monter de version est une **décision** (on relit l'annexe de migration, C08 §8), pas un `pip install -U` par réflexe.
5. **Le nom du script est le nom de sa sortie** : `prix_net.py` produit des prix nets ; un script qui fait trois choses a trois noms en attente (C05 : fonctions).

Conseil professionnel. Dans une équipe, la première question qu'on pose avant d'exécuter un script reçu : « quel `requirements.txt`, quelle version de Python ? » La réponse doit prendre moins de dix secondes (le fichier est au dossier). Si elle en prend plus, le script n'est pas prêt à tourner — et c'est un défaut de livraison, pas de votre exécution.

10. Exercice guidé — « le rapport d'installation » (15 min, /10)

Énoncé. Écrire un script `controle.py` (6 lignes maximum de code) qui affiche un rapport d'installation du projet, **exactement** dans cet ordre :

1. la version de Python ;
2. la version de pandas ;
3. la version de numpy ;
4. le chemin du Python actif ;
5. le message `Installation OK` — ou `INSTALLATION INCOMPLETE` si un import manque.

Barème. Import de `sys` (1 pt) · 4 versions/chemins affichés (4×1 pt) · le `try/except` qui attrape le paquet manquant (2 pts) · le message final conditionnel (2 pts). Le rapport attendu sur l'environnement de l'atelier (mesuré le 19/09/2026) :

```
Python 3.13.14
pandas 2.2.3
numpy 2.3.5
Python actif : /usr/local/bin/python3
Installation OK
```

11. Exercices autonomes

E1 — L'écho. Écrire `echo.py` qui demande une chaîne et l'affiche en MAJUSCULES. (2 min)

E2 — Le venv chronométré. À partir d'un dossier vierge : créer le venv, l'activer, installer pandas, écrire `requirements.txt`. Chronométrer : l'objectif professionnel est **moins de 3 minutes** sur une connexion correcte (la première installation de pandas prend plus longtemps — c'est le téléchargement, pas l'installation). (10 min)

E3 — L'exigence cassée. Dans un venv, exécuter `python -m pip install "pandas==1.5.3"` — lire le message d'erreur ou d'avertissement **ligne par ligne** : qui parle (pip), quoi (conflict of dependencies), et pourquoi. (10 min — l'objectif est de **savoir lire** le message, pas de le faire disparaître.)

E4 — Le noyau qui oublie. Dans un notebook : cellule A `total = 100`, cellule B `print(total)`, cellule C « réinitialiser le noyau », cellule D `print(total)`. Prédire les 4 sorties **avant** d'exécuter, puis vérifier. (5 min)

12. Correction détaillée

Exercice guidé. Le script qui fait l'affaire (les 5 lignes de code, docstring comprise) :

```
"""Rapport d'installation du projet : versions et chemin du Python actif."""
import sys

print(f"Python {sys.version.split()[0]}")
for paquet in ("pandas", "numpy"):
    try:
        mod = __import__(paquet)
        print(f"{paquet} {mod.__version__}")
    except ImportError:
        print(f"{paquet} ABSENT")
print(f"Python actif : {sys.executable}")
print("Installation OK" if __import__("pandas") and __import__("numpy")
      else "INSTALLATION INCOMPLETE")
```

Deux points de correction : `sys.version.split()[0]` prend le numéro seul (3.13.14) sans le texte qui suit ; et le `try/except ImportError` est **spécifique** — jamais `except:` tout nu (C05 §6), sinon une vraie panne (disque plein, venv corrompu) serait avalée et le rapport dirait « OK » à tort.

E1. `nom = input("Chaîne ? "); print(nom.upper())` — la méthode `str.upper()` est vue d'office en Python (pas de `pip` : c'est le langage de base, couche 2).

E2. La séquence : `python -m venv .venv` → `source .venv/bin/activate` → `python -m pip install pandas` → `pip freeze > requirements.txt`. Le `pip freeze` **après** l'installation (sinon le fichier est vide) ; la flèche `>` **écrase** un fichier existant — utiliser `>>` pour ajouter.

E3. Le message de pip commence par le sujet (ERROR: Cannot install ...), nomme les deux paquets en conflit, et termine par la **raison** (depends on numpy>=...). La lecture se fait **de haut en bas** : le message est en dernier, la cause en premier — la même discipline que la traceback (C05 §7).

E4. Sorties prédites : A — rien (pas de `print`), B — `100`, C — rien (le noyau réinitialise), D — `NameError: name 'total' is not defined`. La cellule D est la preuve : le texte de B est intact, **l'effet** de A est parti avec le noyau.

13. Mini-projet M08.P1 — « L'audit d'environnement » (1h)

Énoncé. Écrire `audit_environnement.py` — le script qui produira la **section 0** du grand projet M08.P. Il doit écrire dans `rapport_environnement.txt` (pas `print`) :

1. la date et l'heure de l'exécution ;
2. la version de Python et de pandas/numpy ;
3. le chemin du Python actif et du dossier du projet ;
4. la liste des paquets installés dans le venv (via `pip freeze`) ;
5. une ligne de verdict : `AUDIT OK / AUDIT KO: ...`.

Le rapport du 19/09/2026 dans l'atelier commence par :

```
# Audit d'environnement – M08
2026-09-19
Python 3.13.14
pandas 2.2.3
numpy 2.3.5
Python actif : /usr/local/bin/python3
```

Critères de réussite (grille /10) : le fichier est **écrit** (`open(..., "w")`, encodage `utf-8 — C05 §8`) · 5 sections présentes · aucune version en dur (tout est lu à l'exécution) · le verdict est **calculé**, pas affiché en dur · le script tourne deux fois de suite avec le même résultat (reproductible — la règle du socle).

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise 5 outils, classés de la couche 2 à la couche 4 : l'interpréteur Python 3.13 (la base), `pip` (la boutique), `venv` (la trousse scellée), VS Code (l'atelier, cité), Jupyter (la table à dessin, noyau exécuté dans l'atelier).

Outil	Usage	Commande
Python 3.13	interpréter le code	<code>python --version</code> (Win) / <code>python3 --version</code> (Linux)
<code>pip</code>	installer les paquets	<code>python -m pip install pandas</code>
<code>venv</code>	isoler le projet	<code>python -m venv .venv</code> puis <code>source .venv/bin/activate</code> (Win : <code>.venv\Scripts\activate</code>)
VS Code	éditer + terminal intégré	ouvrir le dossier du projet ; Ctrl+ pour le terminal
Jupyter	cellules + noyau en mémoire	<code>jupyter lab</code> (venv activé, noyau pointant sur le venv)

15. Résumé du chapitre

- L'environnement Python a **4 couches** (machine, interpréteur, paquets, projet) ; chaque symptôme d'installation se rattache à une couche — la nommer, c'est diagnostiquer.
- L'installation Windows se vérifie par `python --version` ; la case « **Add python.exe to PATH** » est le piège n° 1.
- `pip` installe des paquets ; `python -m pip` garantit qu'ils vont dans le Python qu'on exécute ; `requirements.txt` (avec `==`) fige l'installation.
- Le **venv** est la trousse scellée du projet : créer, activer (le préfixe est la preuve), installer dedans, figer, reconstruire chez l'autre.
- Jupyter exécute **par cellules** mais se souvient **par noyau** : l'ordre d'exécution fait la mémoire, pas l'ordre d'écriture — la `NameError` du §5.6 est la preuve.
- Votre premier script tourne (terminal **et** notebook) ; la conversation avec la machine est ouverte. Tout le reste du module, c'est la grammaire de cette conversation.

16. À retenir

À retenir. Le PATH se vérifie, il ne se croit pas : `python --version` dans le terminal est la seule preuve que la couche 2 est en place — tout le reste (raccourci, programme ajouté) est suspect.

À retenir. Un paquet « installé » qui ne se `importe` pas est dans la **mauvaise trousse** : `pip list` dans le **venv actif** avant de relancer l'installation. Le venv ne se partage pas, il se reconstruit — le `requirements.txt` est le ticket de caisse, pas la caisse.

17. Évaluation formative (auto-correction, 8 min)

1. **Quelles sont les 4 couches de l'environnement Python ?** → machine · interpréteur · paquets · projet. (1 pt)
2. **Pourquoi `python -m pip install` plutôt que `pip install` ?** → ça force l'installation dans le Python qu'on exécute (évite le pip/python décalés, §5.8 n° 5). (1 pt)
3. **Qu'est-ce qui prouve qu'un venv est activé ?** → le préfixe du terminal (ex. `(.venv)`) — et `python -m site` montre le chemin du venv. (1 pt)
4. **Une cellule Jupyter utilise `x` défini dans la cellule écrite au-dessus : est-ce garanti ?** → non — seul l'ordre d'exécution compte ; si la définition n'a pas été exécutée, `NameError`. (1 pt)
5. **`ModuleNotFoundError: No module named 'pandas'` alors que « pandas est installé » : les 2 causes les plus probables ?** → paquet dans le Python système (venv actif sans pandas) ; noyau Jupyter attaché au mauvais Python. (2 pts)
6. **Pourquoi le venv ne va pas dans Git ?** → il contient des chemins absolus et des binaires machine-dépendants ; il se reconstruit par `python -m venv .venv + pip install -r requirements.txt`. (1 pt)
7. **Une installation fraîche de pandas en 2026 donne quelle version, et quelle version figure dans ce manuel (et pourquoi) ?** → 3.0.6 (dernière publiée au 19/09/2026) ; le manuel est mesuré sur 2.2.3 (M01-M07) et enseigne les patterns valides sur les deux, avec annexe de migration (C08 §8). (2 pts)
8. **Écrire la commande qui fige l'environnement du venv actif dans `requirements.txt`.** → `pip freeze > requirements.txt` (Linux) — le `>` écrase, le `>>` ajoute. (1 pt)

Module M08.C02 — Les fondations sans données : variables, types, opérateurs, entrées/sorties, commentaires, erreurs

Outils : Python 3.13.14 seul (l'interpréteur, la couche 2 de C01), le terminal. **Aucun paquet n'est requis — ce chapitre tourne sur le Python nu.** **Durée indicative :** 4 h. **Niveau :** N2 → N3. **Prérequis :** C01 (un environnement qui marche).

L'idée du chapitre. C01 a ouvert la conversation avec la machine ; C02 installe la **grammaire** de cette conversation : les **variables** (les noms qu'on donne aux valeurs), les **types** (les familles de valeurs et les opérations qu'elles autorisent), les **opérateurs** (les `+`, `and` et not du métier), les **entrées/sorties** (`input/print`), les **commentaires** (ce qu'on dit au lecteur humain) — et la compétence qui en vaut dix : **lire une erreur** sans paniquer. Rien ne dépend encore des données : les CSV de `dossier_M08/` attendent C07 ; ici, tout est mesuré sur des montants de comptoir (1 500 FCFA, une remise de 10 %, un net de 1 350 FCFA) que vous retrouverez à chaque chapitre. Le fil rouge est le **calculateur de remise** : il naît ici en 4 lignes, deviendra une fonction en C05, un calcul sur table en C07, et un calcul sur 50 008 lignes en C08.

Matériel de l'atelier — Python 3.13.14, terminal, zéro paquet. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les montants du chapitre sont sourcés (`m08p_demo_*`) : prix 1 500, remise 10, net 1 350, seuil comptoir 1 000 — les mêmes valeurs que §6 de C01, votre référence de relecture : si votre machine renvoie autre chose, l'environnement ne tourne pas comme prévu.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Écrire, **renommer** et réaffecter une variable — et expliquer pourquoi rebaptiser ne duplique pas.
2. Dire le **type** d'une valeur (`int`, `float`, `str`, `bool`) et **convertir** un texte en nombre (la recette du format français inclus).
3. Utiliser les opérateurs arithmétiques (`+` `-` `*` `/` `//` `%` `**`), les comparaisons chaînées, et les opérateurs booléens `and/or/not` — avec le parallèle `AND/OR/NOT` de M07.
4. Lire une entrée avec `input()` **en sachant qu'elle arrive toujours en texte**, et afficher avec `print()` (f-strings incluses).
5. **Lire une traceback** ligne par ligne : le fichier, la ligne, la flèche, le message — et dire laquelle des quatre familles d'erreurs vous a (`SyntaxError`, `NameError`, `ValueError`, `TypeError`).
6. Écrire un calculateur de remise en 4 lignes qui tourne dans le terminal.

2. Pourquoi cette notion est importante

Parce que 90 % des erreurs de débutant ne sont pas des erreurs de logique, ce sont des erreurs de **grammaire** : un `int` additionné à un `str`, une virgule dans un nombre, un nom mal recopié. Chacune a un **nom** (`TypeError`, `ValueError`, `NameError`) et un **remède systématique** — ce chapitre donne les trois noms et les trois remèdes. Un apprenant qui reconnaît l'erreur par son nom divise son temps de dépannage par quatre ; un apprenant qui la lit ligne par ligne finit par dépanner **lui-même**.

Deuxième raison : les types, vous les connaissez déjà — M06 a installé la notion sur les colonnes (INTEGER, DECIMAL, VARCHAR). En Python, c'est la **même idée** sur les valeurs individuelles : chaque valeur a une famille, et la famille autorise (ou interdit) les opérations. Le parallèle rapporte : vous ne pouvez pas additionner une colonne texte et une colonne nombre en SQL ; vous ne pouvez pas additionner un `str` et un `int` en Python. Une seule notion, deux langages.

Troisième raison, professionnelle : `input()` arrive **toujours en texte**. C'est la porte par laquelle le monde réel entre dans votre script — et le monde réel écrit ses montants `15 000,50`, pas `15000.50`. La **conversion à la porte** (dès la réception, pas au milieu du calcul) est l'habitude qui fera la différence entre un script qui tourne et un script qui s'effondre à la première saisie.

3. Explication simple — l'étiquette, la famille, la porte

- La **variable** est une **étiquette** collée sur une valeur. Écrire `prix = 1500`, c'est coller l'étiquette « prix » sur le nombre 1 500. Re-coller l'étiquette ailleurs (`prix = 2000`) ne **copie** rien : l'ancienne étiquette est simplement re-collée, et si personne n'en porte d'autre, l'ancien nombre est jeté.
- Le **type** est la **famille** de la valeur. La famille décide ce qu'on peut faire : on peut additionner deux nombres, on ne peut pas additionner un nombre et un mot. En M06, la famille était déclarée **sur la colonne** (DECIMAL ici, VARCHAR là) ; en Python, elle est portée par **chaque valeur** et se voit avec `type(x)`.
- `input()` est une **porte** : ce qui entre, entre **en texte**, toujours — même si le visiteur a dit « 1500 ». C'est à vous de le **convertir** à la porte, sinon il traînera sa voix de texte dans tout le script.
- Une **erreur** est une **protestation lisible** : la machine dit où (fichier, ligne), quoi (le message, en bas), et souvent pourquoi (la flèche qui souligne le coupable). La protester, ce n'est pas échouer ; c'est le travail. La paniquer, c'est perdre.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
variable	un nom qui pointe vers une valeur (pas une boîte qui la contient).	dire « la variable contient le nombre » : rebaptiser la variable ne duplique pas la valeur.
type — type	la famille d'une valeur, qui autorise des opérations (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>).	croire que <code>1</code> , <code>1.0</code> et <code>"1"</code> sont équivalents : trois objets, trois familles, trois comportements.
conversion — cast	transformer une valeur d'une famille à une autre (<code>int("1500")</code> , <code>float("1500.5")</code>).	convertir au milieu du calcul au lieu de la porte : le texte traîne, l'erreur arrive plus tard, plus loin, plus mystérieuse.
opérateur booléen — boolean operator	<code>and</code> , <code>or</code> , <code>not</code> : les connecteurs qui combinent des conditions.	oublier la priorité <code>not > and > or</code> — la même que <code>NOT > AND > OR</code> en SQL (M07 C02).
traceback — traceback	le rapport d'erreur : pile d'appels, ligne fautive, flèche, message.	le lire de bas en haut : la dernière ligne dit quoi , les lignes du haut disent pourquoi .
faux en condition — falsy	une valeur que Python traite comme <code>False</code> dans une condition : <code>0</code> , <code>""</code> , <code>[]</code> , <code>None</code> .	écrire <code>if montant:</code> et ne pas comprendre pourquoi un montant de <code>0</code> passe pour « absent ».

5. Cours approfondi

5.1 La variable — l'étiquette qui se re-colle

```
prix = 1500
print(prix)
prix = 2000
print(prix)
```

Sortie réelle de l'atelier (19/09/2026) :

```
1500
2000
```

Définition. variable — variable — un nom qui pointe vers une valeur : `prix = 1500` colle l'étiquette « prix » sur le nombre 1 500 ; réaffecter (`prix = 2000`) re-colle l'étiquette, sans copier quoi que ce soit. C'est la brique de base de tout script.

Deux remarques qui préviennent des malentendus qui durent :

1. **Aucune « boîte » n'existe.** `prix` n'est pas un contenant dans lequel on met 1 500 puis 2 000 : c'est un nom. Quand vous écrivez `autre = prix`, les **deux** noms pointent vers le **même** objet (pour les nombres, qui ne changent pas, l'effet est invisible — pour les listes de C04, il ne l'est pas du tout).
2. **Le nom est un contrat, pas un décor.** `prix` dit au lecteur ce que la valeur est ; `p2` ne dit rien. La règle M04 (conventions de nommage) s'applique telle quelle en Python : des noms qui racontent, en français ou en anglais, pas les deux mélangés.

5.2 Les 4 types de base — et la conversion à la porte

Les quatre familles du chapitre (les conteneurs — listes, dictionnaires — arrivent en C04) :

```
type(1500)      # un entier
type(1500.5)    # un nombre à virgule
type("1500")   # un texte
type(True)     # un booléen
```

Sortie réelle :

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

Définition. type — type — la famille d'une valeur, qui autorise (ou interdit) les opérations : `int` (entier), `float` (nombre à virgule), `str` (texte), `bool` (vrai/faux). `type(x)` la dit. En M06, la famille était déclarée sur la colonne ; en Python, elle est portée par chaque valeur — une seule notion, deux langages.

La conversion, c'est changer de famille — et c'est là que le **format français** entre en jeu. `int("1500")` passe ; `int("1500,50")` se proteste, avec sa sortie exacte :

```
ValueError: invalid literal for int() with base 10: '1500,50'
```

La **recette du format français** (l'outil que vous utiliserez toute la formation sur les montants) : supprimer les espaces, remplacer la virgule par le point, puis convertir :

```

valeur = float("15 000,50".replace(" ", "").replace(",", "."))
print(valeur)

```

```
15000.5
```

Conseil professionnel. Convertissez à la **porte** : dès que la valeur arrive (`input()`, lecture de fichier en C07), faites-en un nombre — et nommez la variable d’après sa **valeur**, pas d’après sa forme (`montant`, pas `montant_texte` une fois converti). Un texte qui traîne dans le calcul ne fait pas d’erreur tout de suite : il en fait une plus tard, plus loin, sur une ligne qui n’a rien à voir — c’est l’erreur la plus coûteuse à chercher.

5.3 Les opérateurs — l’arithmétique du comptoir

L’arithmétique, exécutée dans l’atelier :

```
print(17 // 5, 17 % 5, 2 ** 8)
```

```
3 2 256
```

Trois opérateurs que le clavier ne vous a pas appris : `//` la **division entière** ($17 // 5 = 3$ — l’entier du quotient, ce qu’une caisse garde), `%` le **reste** ($17 \% 5 = 2$ — ce qui dépasse), `**` la puissance ($2 ** 8 = 256$). Le `/` classique renvoie un `float` ($17 / 5 = 3.4$).

Les comparaisons se **chaînent** comme en mathématiques — l’équivalent SQL serait deux conditions `AND`-ées :

```
print(1000 <= 1200 < 1500)
```

```
True
```

Et les trois opérateurs booléens, le **parallèle SQL exact** (M07 C02 : `AND` se lie avant `OR`) :

```

print("and :", True and False)
print("or  :", True or False)
print("not :", not False)

```

```

and : False
or  : True
not : True

```

Python	SQL	Sens
<code>and</code>	<code>AND</code>	les deux conditions vraies
<code>or</code>	<code>OR</code>	l’ une des deux vraie
<code>not</code>	<code>NOT</code>	l’inverse

La priorité est la même des deux côtés : `not` d’abord, puis `and`, puis `or`. Et la même règle de style que M07 : **parenthéser pour clarifier** — une condition lue par un collègue doit ne pouvoir se lire qu’une façon.

5.4 `input()` et `print()` — la porte et la vitrine

`input()` demande, et **renvoie toujours un `str`** — la sortie de l’atelier le prouve (l’entrée `15 000,50` fournie par `echo`, le même effet que de la taper) :

```
montant = input("Montant (FCFA) ? ")
print(type(montant))
valeur = float(montant.replace(" ", "").replace(",", "."))
print(f"montant lisible : {valeur:,.2f}")
```

```
Montant (FCFA) ? <class 'str'>
montant lisible : 15,000.50
```

Attention. `input()` renvoie **toujours** un texte, même quand le visiteur dit « 1500 ». `int(input("..."))` sans la conversion du format français s’effondre sur la **première** saisie 15 000,50 — et c’est la porte, pas le calcul, qui doit faire la conversion.

`print()` affiche ; la **f-string** (le `f"... {valeur} ..."`) insère les valeurs dans le texte — le formatage de montants que vous ferez toute la formation.

Définition. f-string — f-string — une chaîne précédée de `f` où chaque `{...}` est remplacé par la valeur : `f"Total : {total:,.2f}"` affiche le total avec 2 décimales. C’est la forme de sortie de tout le module ; le `:,.2f` est la **spécification de format** (virgule milliers, 2 décimales — la convention américaine par défaut, voir l’encadré ci-dessous).

Dans les faits. Le formatage par défaut de Python est **américain** : `{valeur:,.2f}` écrit 15,000.50 (virgule = milliers, point = décimales). La convention française de la formation (espace = milliers, virgule = décimales — règle M01) s’écrit à la main : `f"{valeur:,.2f}".replace(","," ").replace(".",",")` → 15 000,50. Le chapitre publie la sortie telle qu’elle s’affiche (américaine) et dit quel est le format de la maison.

5.5 Les commentaires — le « pourquoi », pas le « quoi »

Le commentaire (`#`) est un mot dit **au lecteur humain** ; la machine le saute. La règle M04, reprise telle quelle : **commenter le pourquoi, pas le quoi**.

```
# quoi (inutile : le code le dit)
remise = 0.10

# pourquoi (utile : le code ne le dit pas)
remise = 0.10 # remise commerciale de septembre – voir le mémo du 05/09
```

Le premier commentaire répète le code (il mentra dès qu’on changera la valeur) ; le second conserve une **décision** (d’où vient le 10 %). La **docstring** — le commentaire en tête de fonction, entre triples guillemets — arrive en C05 ; elle est la version structurée de la même règle.

5.6 Lire une erreur — l’anatomie, en 3 protestations réelles

Les quatre familles d’erreurs du débutant, et les trois que l’atelier a exécutées le 19/09/2026.

NameError — le nom n’existe pas (orthographe, ou variable d’une autre cellule en Jupyter) :

```
print(montannt)
^^^^^^
NameError: name 'montannt' is not defined
```

ValueError — le nom existe, la **valeur** est la mauvaise (le texte n’est pas un nombre lisible par `int()`) :

```
int('1500,50')
~~~~~
ValueError: invalid literal for int() with base 10: '1500,50'
```

SyntaxError — le **texte** du script ne se lit pas (ici : le deux-points qui manque après `if`) :

```
File "<string>", line 1
  if 3 > 2
      ^
SyntaxError: expected ':'
```

Définition. traceback — `traceback` — le rapport d'erreur de Python : il donne le chemin parcouru (la pile d'appels, du haut vers le bas), la **ligne fautive** (le numéro et le texte), la **flèche** qui souligne le coupable, et le **message** (le type d'erreur + une phrase). C'est le document à savoir lire avant d'écrire du code avancé.

L'anatomie, sur l'exemple `ValueError` — **quatre choses à relever**, toujours dans cet ordre :

1. **Le message, en bas** : `ValueError` + la phrase. Le **type** (`ValueError`) est le nom de la famille — c'est lui qu'on retient, pas la phrase.
2. **La ligne fautive** : le numéro de ligne (ou le `File "..."`) — c'est où.
3. **La flèche** : elle souligne l'endroit exact dans la ligne — le coupable, pas le voisin.
4. **La pile, au-dessus** (lorsque le script a des fonctions, C05) : les lignes au-dessus racontent **par quel chemin** on est arrivé là — on les lit de haut en bas.

La discipline : **lire de haut en bas pour le pourquoi, arrêter à la dernière ligne pour le quoi**. La dernière ligne dit toujours quel type d'erreur ; les lignes du haut disent d'où elle vient. En Jupyter, la ligne fautive est dans la **cellule exécutée** — pas celle qui est écrite au-dessus (C01 §5.6).

5.7 `None` et les valeurs « fausses » en condition

Quatre valeurs que Python traite comme `False` dans une `if` — exécuté dans l'atelier :

```
print("0      :", bool(0))
print('""     : ', bool(""))
print("[]     :", bool([]))
print("1500   :", bool(1500))
```

```
0      : False
""     : False
[]     : False
1500   : True
```

Attention. `0`, `""` (le texte vide), `[]` (la liste vide) — et `None`, l'absence totale — sont **faux en condition**. `if montant:` passe pour tout montant **sauf** `0` : un montant de 0 FCFA est une valeur **présente** (une vente à zéro, une remise à 100 %), pas une absence. Pour tester « est-ce que la valeur existe », tester `est None` ; pour tester « est-ce que la valeur est nulle », tester `== 0`. Deux questions différentes, deux tests différents.

Le tableau complet du chapitre — les 4 familles, les opérations qu'elles autorisent, les 4 protestations qui les suivent :

Les 4 types de base : la famille autorise les opérations

type(x) dit la famille ; la famille décide ce qui passe et ce qui proteste.

Type	+ (addition)	comparaison	conversion	en condition
int	1500 + 500 = 2000 17 // 5 = 3 · 17 % 5 = 2	1500 < 2000 → True 1000 <= 1200 < 1500 (chaînée)	int('1500') → 1500 str(1500) → '1500'	1500 → True 0 → False (le piège du if)
float	1500.5 + 0.5 = 1501.0 500 * 1.19 = 595.0 (TVA 2026)	1500.5 < 2000 → True 0.1 + 0.2 != 0.3 (approximatif)	float('1500.5') → 1500.5 le point, pas la virgule	1500.5 → True 0.0 → False
str	'15' + '00' = '1500' (concat.) pas d'addition de nombres	'Ouaga' < 'Bobo' → True (alpha) la casse compte : 'A' < 'a'	int('1500,50') → ValueError 'x' → True (texte non vide) recette : replace puis float	" → False (texte vide)
bool	and · or · not (pas de +) priorité : not, and, or (comme SQL)	True == True → True True == 1 → True (attention)	bool(0) → False bool(1500) → True	True / False le if les lit directement
list · tuple · dict · set les conteneurs — C04 : des listes de ces 4 types, des clés vers valeurs				

Les 4 protestations du débutant (C02 §5.6) — le nom de la famille d'abord :

SyntaxError — le texte ne se lit pas (deux-points manquant)
 NameError — le nom n'existe pas (montant, ou cellule Jupyter non exécutée)
 TypeError — mauvaise famille (1500 + 'abc')
 ValueError — mauvaise valeur (int('1500,50'))

6. Exemple concret — le calculateur de remise

La question du comptoir, en langage de comptoir : « l'article coûte 1 500 FCFA, la remise est de 10 %, quel est le prix net ? » Le script le plus court qui y répond — exécuté dans l'atelier le 19/09/2026 (les deux réponses, 1500 puis 10, fournies par printf) :

```
prix = float(input("Prix (FCFA) ? "))
remise_pct = float(input("Remise (pct) ? "))
net = prix * (1 - remise_pct / 100)
print(f"Prix net : {net:,.2f} FCFA")
```

```
Prix (FCFA) ? Remise (pct) ? Prix net : 1,350.00 FCFA
```

Le net est **1 350 FCFA** (clé m08p_demo_prix_net) — la valeur de référence du chapitre. Le parallèle SQL, déjà actif : en M07, le même calcul était une colonne calculée dans le SELECT (SELECT prix_vente_ht * (1 - 0.10) AS prix_net FROM produit;). La **formule** est identique ; le **lieu** change : une ligne de table (SQL) vs une ligne de script (Python). En C07, la même ligne s'appliquera à 50 008 lignes d'un coup — c'est toute la promesse du module.

7. Démonstration pas à pas — 5 étapes sur le fil rouge

Du zéro au calculateur, les 5 étapes, sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — L'arithmétique d'abord (le calcul nu, sans variable) :

```
$ python3 -c "print(500 * 1.19)"
595.0
```

Interprétation : $500 + 19\%$ de TVA (2026, la règle du socle) = 595 — la machine calcule, on n'a pas encore de nom.

Étape 2 — Donner un nom (la variable) :

```
$ python3 -c "x = 10; x = x + 5; print(x)"
15
```

Interprétation : `x = x + 5` se lit « pointe x vers la valeur de x plus 5 » — l'étiquette se re-colle, rien n'est dupliqué.

Étape 3 — La conversion à la porte (le format français) :

```
$ python3 -c "print(float('15 000,50'.replace(' ', '').replace(',', '.')))"
15000.5
```

Interprétation : le texte `15 000,50` est devenu le nombre 15 000,50 avant tout calcul — la porte a fait son travail.

Étape 4 — Décider (la condition, le parallèle `WHERE`) :

```
$ python3 -c "prix = 1200; print('dossier' if prix > 1000 else 'comptant')"
dossier
```

Interprétation : le seuil comptoir est 1 000 FCFA (`m08p_demo_seuil`) ; 1 200 passe au-dessus — en SQL, c'était un `CASE WHEN prix > 1000 THEN 'dossier' ...` (M07 C06).

Étape 5 — Le calculateur complet (les 4 lignes de §6) :

```
$ printf "1500\n10\n" | python3 calculateur.py
Prix (FCFA) ? Remise (pct) ? Prix net : 1,350.00 FCFA
```

Interprétation : entrées lues, conversion faite, calcul fait, sortie lisible — la conversation de C01 est devenue un travail.

8. Erreurs fréquentes

1. **`TypeError: unsupported operand type(s) for +: 'int' and 'str'`** — l'erreur n° 1 du débutant : un texte additionné à un nombre. Le texte est presque toujours le fruit d'un `input()` non converti (ou d'un `str` collé dans un calcul). Remède : convertir **à la porte** (§5.2).
2. **Le `1500,50` qui devient un couple.** Écrire `x = 1500,50` dans le code crée un **tuple** (une paire), pas un nombre à virgule — exécuté dans l'atelier : `text $ python3 -c "x = 1500,50; print(type(x), x)"` `<class 'tuple'> (1500, 50)` La virgule de Python est le **séparateur d'arguments**, pas la virgule décimale. Le nombre à virgule s'écrit `1500.5` (point) — le format français est affaire de **texte**, pas de nombre.
3. **= dans une condition.** `if x = 5:` est une **affectation** déguisée en comparaison → `SyntaxError`. La comparaison est `==` (deux signes) ; l'affectation, un.
4. **`int("1500,50")` → `ValueError`.** La famille `int` ne lit que les entiers en base 10 — le point décimal **et** la virgule lui sont illisibles. Recette du format français (§5.2).
5. **L'indentation qui change le sens.** En Python, l'indentation **est** la syntaxe : une ligne dedans de 4 espaces est « dans le bloc », en dehors, elle s'exécute quoi qu'il arrive. Un `print` qu'on attendait « seulement si » et qui s'exécute toujours est une histoire d'indentation.
6. **`0.1 + 0.2 != 0.3`.** Exécuté dans l'atelier : `text $ python3 -c "print(0.1 + 0.2)"` `0.30000000000000004` Les nombres à virgule sont stockés **approximativement** (le binaire ne sait pas dire 0,1). Pour l'égalité de deux `float`, comparer avec une tolérance — et pour l'argent, on verra en C08 la solution de fond : compter en **centimes entiers**.

9. Bonnes pratiques professionnelles

1. **Le nom dit la valeur, pas la forme** : `montant` une fois converti, jamais `montant_texte` ; `remise_pct` (en pourcent) vs `remise` (en fraction) — le suffixe évite le bug du facteur 100.
2. **Un calcul par variable intermédiaire** : `net = prix * (1 - remise_pct / 100)` se lit d'un coup d'œil ; `prix * 0.9` force le lecteur à recalculer le 0.9. Les variables intermédiaires sont la documentation vivante du calcul.
3. **Convertir à la porte, tester à la porte** : dès la réception, `float(...)` — et si la valeur arrive de nulle part (fichier, API en C07), une valeur illisible doit **se voir** tout de suite, pas trois calculs plus tard.
4. **Compter l'argent en centimes** dès qu'un total s'annonce (C08) : les `float` se trompent d'un centime sur 50 000 lignes ; les entiers, jamais.
5. **Laisser une erreur s'afficher** plutôt que de l'avaler : un `try/except` qui « répare » silencieusement masque la porte qui fuit (C05 §6) — l'erreur vue est l'erreur payée une fois.

10. Exercice guidé — « le ticket de caisse » (15 min, /10)

Énoncé. Écrire `ticket.py` qui, dans l'ordre :

1. lit un **prix** (au format français, ex. `15 000,50`) et une **quantité** (entier) ;
2. calcule le total (`prix × quantité`) ;
3. affiche le total avec 2 décimales ;
4. affiche la mention `Seuil dépassé : dossier requis` **si** le total dépasse le seuil comptoir de 1 000 FCFA, et rien sinon.

Barème. Les deux lectures avec conversion à la porte (4 pts) · le total en une variable nommée (2 pts) · l'affichage 2 décimales (2 pts) · la condition avec le seuil **en variable nommée** (2 pts). Sortie attendue pour l'entrée `15 000,50 / 3` (vérifiée par exécution le 19/09/2026) :

```
Total : 45,001.50
Seuil dépassé : dossier requis
```

(le 45 001,50 vient de `15 000,50 × 3` — l'apprenant qui annonce « 45 000 » est tombé dans l'arrondi mental ; c'est le point de l'exercice, pas un piège.)

11. Exercices autonomes

Exercice 2.1 — L'échange. Sans variable temporaire, échanger les valeurs de `a` et `b` (`a = 5`, `b = 8`). Indice : Python autorise `a, b = b, a`. (3 min)

Exercice 2.2 — La chaîne. Écrire **une** expression (pas deux conditions `and`-ées) qui vaut `True` si un prix est compris entre 100 et 1 000 FCFA **bornes comprises**. (5 min)

Exercice 2.3 — La prédiction. Prédire **sur papier** la sortie de `print(0.1 + 0.2)` et de `print(0.1 + 0.2 == 0.3)`, puis exécuter et comparer. Si la prédiction a raté, écrire en une phrase pourquoi (indice : §8, erreur n° 6). (5 min)

Exercice 2.4 — Le plus cher. Lire deux prix et afficher le plus cher — avec `max()`, puis **sans** `max()` (avec une `if`). Les deux sorties doivent être identiques. (10 min)

Exercice 2.5 — Le couple piégé. Un collègue a écrit `x = 1500,50` et s'étonne que `x * 2` ne donne pas 3 001. Expliquer en 3 lignes ce que `x` est vraiment, et écrire la ligne corrigée. (5 min)

12. Correction détaillée

Exercice guidé. Le script qui fait l'affaire :

```
"""Ticket de caisse : total, seuil comptoir, mention dossier."""
prix = float(input("Prix (FCFA, ex. 15 000,50) ? ").replace(" ", "").replace(",", "."))
quantite = int(input("Quantité ? "))
total = prix * quantite
print(f"Total : {total:,.2f}")
seuil = 1000 # seuil comptoir – au-delà, dossier requis
if total > seuil:
    print("Seuil dépassé : dossier requis")
```

Deux points de correction : la conversion est à **la porte** (les deux `input` convertis sur place) ; le seuil est une **variable nommée** (`seuil`), pas un 1 000 en dur dans la condition — si le seuil change, une ligne change, pas deux.

Exercice 2.1. `a, b = b, a` — Python évalue d'abord le membre de droite (la paire `b, a`), puis distribue : c'est l'échange en une ligne, sans variable temporaire. (Le tuple, C04.)

Exercice 2.2. `100 <= prix <= 1000` — la comparaison chaînée (§5.3), bornes comprises par les `<=`. L'écriture `and-ée` (`prix >= 100 and prix <= 1000`) est correcte mais plus longue ; la chaîne est la forme lue en mathématiques.

Exercice 2.3. Prédiction : `0.30000000000000004` et `False`. Explication en une phrase : les nombres à virgule sont stockés approximativement en binaire, donc `0.1 + 0.2` n'est pas **exactement** `0.3` — l'égalité stricte échoue (clé `m08p_demo_float_add`).

Exercice 2.4. Avec : `print(max(p1, p2))`. Sans : `print(p1 if p1 >= p2 else p2)` — l'expression conditionnelle (la forme `A if condition else B` d'Étape 4), la CASE WHEN de M07 C06 en une ligne.

Exercice 2.5. `x` est un **tuple** de deux entiers, `(1500, 50)` — la virgule est le séparateur d'arguments de Python, pas une virgule décimale (§8, erreur n° 2). La ligne corrigée : `x = 1500.5` (point, un seul nombre).

13. Mini-projet M08.P2 — « La caisse » (1 h)

Énoncé. Écrire `caisse.py` — le calculateur de remise de \$6, durci pour le comptoir :

- lire le **prix** au format français (avec ou sans espaces, virgule ou point) ;
- lire la **quantité** (entier ≥ 1) ;
- calculer le total, puis appliquer une **remise de 5 % si la quantité est de 10 ou plus** ;
- afficher : le total brut, la remise appliquée (0 si inapplicable), le net — 3 lignes, 2 décimales chacune ;
- refuser poliment (sans s'effondrer) la quantité « 0 » ou négative.

Critères de réussite (grille /10) : les 5 exigences (5×1 pt) · le seuil de remise et le pourcentage sont des **variables nommées** en tête (1 pt) · la sortie de l'entrée 1500 / 12 donne total brut 18 000,00 · remise 900,00 · net 17 100,00 (1 pt) · aucune erreur sur les saisies pièges : `abc`, `15,0`, `0`, `-3` (1 pt) · docstring en tête, commentaires « pourquoi » (1 pt).

Note de correction. Les sorties du critère sont mesurées : $1\,500 \times 12 = 18\,000$; 5 % de 18 000 = 900 ; net 17 100 — l'apprenant doit les **retrouver en exécutant**, pas en les lisant ici : c'est le principe de l'évaluation du module (prédire, exécuter, comparer).

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre n'utilise que l'interpréteur et le terminal (couche 2 de C01) — la boîte à outils se remplit au fur et à mesure : `type()`, les conversions, les f-strings, et la discipline de lecture des tracebacks.

Outil	Usage	Syntaxe
<code>python3 -c</code>	exécuter une ligne sans fichier	<code>python3 -c "print(17 // 5)"</code>
<code>type()</code>	dire la famille d'une valeur	<code>type(1500.5) → <class 'float'></code>
<code>int()</code> / <code>float()</code>	convertir texte → nombre	<code>float("1500.5")</code>
<code>input()</code> / <code>print()</code>	la porte et la vitrine	<code>x = input("...") · print(f"... {x} ...")</code>
<code>and</code> / <code>or</code> / <code>not</code>	combiner des conditions	<code>if a > 0 and b > 0:</code>
<code>//</code> · <code>%</code> · <code>**</code>	division entière · reste · puissance	<code>17 // 5 · 17 % 5 · 2 ** 8</code>

15. Résumé du chapitre

- La variable est une **étiquette** qui se re-colle : réaffecter ne duplique rien.
- Les 4 types de base (`int`, `float`, `str`, `bool`) portent les opérations ; la **conversion à la porte** (`replace(" ", "").replace(",",".")` puis `float()`) est l'habitude du format français.
- Les opérateurs booléens sont **les mêmes** qu'en SQL (`and/or/not` = `AND/OR/NOT`, même priorité) — le parallèle est l'accélérateur du module.
- `input()` arrive **toujours en texte** ; la porte convertit, le calcul ne s'en préoccupe plus.
- Une traceback se lit **de haut en bas pour le pourquoi**, s'arrête **en bas pour le quoi** ; les 4 familles du débutant (`SyntaxError`, `NameError`, `ValueError`, `TypeError`) ont chacune un remède systématique.
- Le calculateur de remise tourne ($1\,500 - 10\% = 1\,350$) — il attend C05 (deviendra fonction), C07 (deviendra calcul sur table) et C08 (deviendra calcul sur 50 008 lignes).

16. À retenir

À retenir. Une erreur a un **nom** avant d'avoir un sens : `TypeError` (mauvaise famille), `ValueError` (mauvaise valeur), `NameError` (mauvais nom), `SyntaxError` (mauvais texte). Nommer l'erreur, c'est déjà avoir le quart du remède.

À retenir. Le texte se convertit **à la porte** : `input()` et fichiers en C07 compris, une valeur illisible doit se voir **dès sa réception** — un texte qui traîne dans le calcul ne fait pas d'erreur tout de suite, il en fait une plus tard, plus loin, sur une ligne qui n'a rien à voir.

17. Évaluation formative (auto-correction, 8 min)

1. Que fait `prix = 1500` exactement, et que fait `prix = 2000` ensuite ? → colle l'étiquette « prix » sur 1 500 ; re-colle l'étiquette sur 2 000 (rien n'est dupliqué). (1 pt)
2. `type(1500)`, `type("1500")`, `type(True)` ? → `<class 'int'>`, `<class 'str'>`, `<class 'bool'>`. (1 pt)

3. **Pourquoi `int("1500,50")` échoue-t-il, et quelle est la recette du format français ?** → `int` ne lit que les entiers base 10 (la virgule est illisible) ; recette : `float(s.replace(" ", "").replace(",", "."))`. (2 pts)
4. **La priorité des opérateurs booléens, et son équivalent SQL ?** → `not > and > or` — la même que `NOT > AND > OR` en SQL (M07 C02). (1 pt)
5. **`input("...")` renvoie toujours quoi, et quelle en est la conséquence ?** → toujours un `str` ; la conversion doit se faire à la porte, sinon le texte traîne dans le calcul. (2 pts)
6. **Sur une traceback, où est le « quoi » et où est le « pourquoi » ?** → le message (type + phrase) est en bas ; la pile d'appels (lignes du haut) raconte d'où l'erreur vient. (1 pt)
7. **Pourquoi `x = 1500,50` ne crée pas un nombre à virgule ?** → la virgule est le séparateur d'arguments de Python : `x` est un tuple `(1500, 50)` ; le nombre s'écrit `1500.5` (point). (1 pt)
8. **`0.1 + 0.2` vaut quoi, et que faire pour de l'argent ?** → `0.30000000000000004` (stockage approximatif en binaire) ; comparer avec tolérance, et compter en centimes entiers pour les totaux (C08). (1 pt)

Module M08.C03 — Décider et répéter :

if/elif/else, boucles for/while, break/continue, compréhensions

Outils : Python 3.13.14 seul (l'interpréteur), le terminal. **Aucun paquet.** **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Prérequis :** C02 (variables, types, erreurs).

L'idée du chapitre. C02 a donné la grammaire d'une **valeur** ; C03 donne le **flux** de plusieurs valeurs : **décider** (if/elif/else — l'échelle de décision, le CASE WHEN de M06 en Python), **répéter** (for le tapis roulant, while le « tant que »), **en sortir** (break la porte de sortie, continue le « passe, la suite »), et **résumer** en une ligne (les compréhensions). Le fil rouge : le calculateur de remise de C02 devient le **contrôle qualité express** — 10 montants, compter les anomalies, s'arrêter au 3^e dépassement, imprimer le rapport. C05 en fera une fonction ; C07, un calcul sur 50 008 lignes de table.

Matériel de l'atelier — Python 3.13.14, terminal, zéro paquet. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les montants et magasins cités sont ceux du socle M08 (clés m08p_*) : les 5 magasins réels (magasin.csv), le seuil comptoir 1 000, les montants 500, 1 200, 1 500, 15 000,50.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Écrire une **échelle de décision** if/elif/else avec des conditions nommées — l'équivalent Python du CASE WHEN de M06.
2. Parcourir une liste avec for (les 5 magasins), compter avec range(n), et porter l'index et la valeur avec enumerate.
3. Écrire une boucle while « tant que » (le solde d'un crédit), reconnaître une **boucle infinie**, et savoir ce que break change.
4. Utiliser break (s'arrêter au premier dépassement) et continue (ignorer les lignes pourries) — les deux usages métier, plus le for...else « non trouvé ».
5. Écrire une **compréhension** [expr for x in it if cond] — et appliquer la **règle des 2 niveaux** : au-delà, on revient à une boucle.
6. Parcourir deux listes en parallèle avec zip (magasins + surfaces).

2. Pourquoi cette notion est importante

Le flux de contrôle, c'est le métier. Un script qui ne sait ni décider ni répéter ne fait qu'un calcul ; un script qui sait faire les deux fait un **travail** : trier, compter, arrêter, filtrer, résumer. C'est 80 % du code qu'on écrit au quotidien — et c'est là que vivent les deux pires familles d'erreurs du débutant : la **boucle qui ne s'arrête pas** (la machine tourne, tourne, tourne — ce chapitre la mesure en secondes réelles) et l'**indentation** (en Python, l'alignement **est** la syntaxe : une ligne mal alignée, c'est un IndentationError, pas un avertissement).

Le parallèle SQL rapporte, comme dans tout le module. Le if/elif/else, vous l'avez écrit en M06 : c'est le CASE WHEN, ligne par ligne au lieu de colonne par colonne. Et la boucle for, vous avez juste **délégué** ce travail au SGBD en SQL : quand vous écrivez SELECT * FROM vente, le moteur parcourt les 50 008 lignes **pour vous**, sans que vous écriviez de boucle. En Python, le

même parcours est à **vous d'écrire** — et c'est ce qui donne sa force : en SQL, on ne peut répéter que sur des **lignes de table** ; en Python, on répète sur **n'importe quoi** (des saisies, des fichiers, des résultats d'API — tout ce qui n'est pas encore en table).

3. Explication simple — le carrefour, le tapis, le « tant que »

- Le **if/elif/else** est une **échelle de carrefour** : chaque échelon pose une question, la première vraie prend la valeur, et si personne ne dit rien, le dernier échelon (**else**) attrape le reste. Comme le **CASE WHEN** de M06 : une condition par ligne, un **ELSE** au fond.
- La **boucle for** est un **tapis roulant** : on pose les éléments (les 5 magasins) dans l'ordre, et on traite chacun en passant — un par un, sans sauter, sans revenir.
- La **boucle while** est le « **tant que** » : on ne compte pas les tours, on donne une condition (« tant que le crédit dépasse 0 ») et on se répète jusqu'à ce qu'elle devienne fausse.
- **break** est la **porte de sortie** : on sort de la boucle immédiatement, au milieu.
- **continue** est le « **passe** » : on saute le reste du tour **et on revient au début** — l'élément pourri est ignoré, le tapis continue.
- La **compréhension** est le tapis roulant **avec filtre, en une ligne** : on nomme ce qui sort du tapis au lieu de le traiter un par un.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
échelle de décision	une suite if / elif / else où la première condition vraie gagne, sinon else .	croire que les échelons se testent tous : la première vraie court-circuite le reste.
itération — iteration	un tour de boucle : le corps s'exécute une fois sur un élément.	confondre « la boucle a tourné » (le tour) et « la liste est finie » (la fin du parcours).
boucle infinie — infinite loop	une boucle dont la condition ne devient jamais fausse (ou dont on ne coupe jamais).	la détester : mesurée en secondes, elle est la preuve que le break manquait.
sortie / contournement — break / continue	break : sortir de la boucle tout de suite ; continue : sauter le reste du tour, reprendre au début.	mélanger les deux : break termine le parcours, continue réduit un tour.
compréhension comprehension	— <code>[expression for x in iterable if condition]</code> : une liste construite en une ligne lisible.	empiler 2 niveaux de for : au-delà, c'est plus une formule qu'une phrase — on repasse à une boucle.
boucle avec else for...else	— le else d'une boucle s'exécute si la boucle se termine sans break .	croire que le else fait « le contraire » : il fait « le cas où on n'a pas cassé ».

5. Cours approfondi

5.1 if/elif/else — l'échelle du comptoir

L'échelle à trois modes de paiement du socle (le seuil comptoir est 1 000 FCFA, `m08p_demo_seuil` ; au-delà de 15 000 FCFA, le client part en financement — les trois montants de test sont ceux de C02), exécutée dans l'atelier :

```
totaux = (500, 1200, 15000.5)
for total in totaux:
    if total < 1000:
        mode = "comptant"
    elif total <= 15000:
        mode = "dossier"
    else:
        mode = "financement"
    print(total, "->", mode)
```

```
500 -> comptant
1200 -> dossier
15000.5 -> financement
```

Définition. échelle de décision — if/elif/else — une suite d'échelons testés **dans l'ordre** : la première condition vraie exécutera son bloc, et le reste est sauté ; else attrape ce que personne n'a pris. C'est le CASE WHEN ... THEN ... ELSE ... END de M06, écrit en Python.

Deux choses que cette exécution montre :

1. **L'ordre des échelons est une décision métier, pas un détail.** < 1 000 avant <= 15 000 : si on avait inversé, tout le monde serait en « dossier ». Le CASE WHEN de M06 obéissait à la même règle — c'est la **même** échelle, deux langages.
2. **L'indentation EST la syntaxe.** Le bloc du if est ce qui est enfoncé de 4 espaces : pas de accolades, pas de then — **l'alignement dit ce qui est « dans » la condition**. Une ligne mal alignée, Python s'effondre (le §8 a les deux messages réels).

Conseil professionnel. Écrivez la condition **en noms**, pas en chiffres : if total < seuil_comptant: (le seuil défini une fois, en haut : seuil_comptant = 1000) — si le seuil change, une ligne change. Et quand un break sort d'un parcours, donnez-lui une **raison lisible** dans l'affichage (« arrêt : 3^e dépassement »), pas un break tout nu : le rapport doit pouvoir se **relire** sans le code.

5.2 La boucle for — le tapis roulant des 5 magasins

Les 5 magasins du socle (magasin.csv), parcourus avec leur numéro — enumerate donne l'index **et** la valeur :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
for i, nom in enumerate(magasins):
    print(i + 1, nom)
```

```
1 Ouaga Centre
2 Ouaga Patte d'Oie
3 Bobo Centre
4 Bobo Sarfalao
5 Koudougou
```

Définition. itération — iteration — un tour de boucle : le corps de la for s'exécute une fois par élément, dans l'ordre, sans sauter ni revenir. Après le dernier élément, la boucle est finie — c'est tout, pas d'arrêt à écrire.

range(n) est le tapis des nombres : range(10) déroule **0 à 9** — la sortie de l'atelier le prouve :


```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Attention. `range(10)` donne **0 à 9**, pas 1 à 10. La liste a 10 éléments, mais le dernier a l'index 9 — c'est le décalage qui fabrique la moitié des bugs de comptage (et l'`IndexError` de C04). Pour « 1 à 10 », écrire `range(1, 11)`.

5.3 La boucle `while` — le « tant que » du crédit

Un crédit de 2 500 FCFA, remboursé à raison de 500 par mois — on ne compte pas les tours, on donne la condition :

```
credit = 2500
mensualite = 500
mois = 0
while credit > 0:
    mois = mois + 1
    credit = credit - mensualite
    print(f"mois {mois} : reste {credit}")
print("solde en", mois, "mois")
```

```
mois 1 : reste 2000
mois 2 : reste 1500
mois 3 : reste 1000
mois 4 : reste 500
mois 5 : reste 0
solde en 5 mois
```

Définition. boucle infinie — infinite loop — une boucle dont la condition ne devient jamais fausse (ou qu'aucun `break` ne coupe) : la machine se répète sans fin. Ce n'est pas une fatalité, c'est un **diagnostic** : la condition ou le `break` manque — le §5.3 et le §8 la mesurent en secondes réelles.

La boucle infinie, **exécutée** dans l'atelier (lancée sous `timeout 2` : tuée après 2 secondes) :

```
$ timeout 2 python3 boucle_infinie.py
code de sortie : 124
```

Aucune sortie, rien n'a été affiché, le code 124 est celui du processus **tué** : la machine comptait encore. Et le même corps de boucle, avec le `break` qui sauve :

```
i = 0
while True:
    i = i + 1
    if i == 5:
        break
print(i)
```

```
5
```

La leçon tient en une phrase : `while True` n'est pas interdit — il oblige simplement à porter la responsabilité de la sortie **dans le corps** (`break`). En SQL, le SGBD a la garantie de fin (les lignes finissent) ; en Python, la fin est **votre** contrat.

5.4 `break` et `continue` — la porte de sortie et le « passe »

Usage métier n° 1 — `break` : s'arrêter au premier dépassement. Le contrôle ne va pas au bout s'il trouve ce qu'il cherche : le premier montant au-dessus du seuil suffit.

```
montants = [120, 800, 2000, 15000.5, 500]
seuil = 15000
for m in montants:
    if m > seuil:
        print("dépassement :", m)
        break
print("fin du parcours")
```

```
dépassement : 15000.5
fin du parcours
```

Le `break` a coupé le parcours : 500 n'a jamais été vu. Le `print` après la boucle s'exécute **toujours** — il est **hors** du bloc.

Usage métier n° 2 — `continue` : ignorer les lignes pourries. Une saisie « N/C » (non communicable) ne bloque pas le total : on passe, on revient au début, on continue.

```
saisies = [500, "N/C", 1500, "N/C", 1350]
total = 0
pourries = 0
for s in saisies:
    if s == "N/C":
        pourries = pourries + 1
        continue
    total = total + s
print("total :", total, "| pourries :", pourries)
```

```
total : 3350 | pourries : 2
```

Les deux montants « N/C » sont **comptés** (pourries : 2) mais pas additionnés — c'est exactement le `WHERE montant IS NOT NULL` de M06, écrit en Python.

Le `for...else` — le « non trouvé » sans drapeau. Le `else` attaché à une boucle s'exécute **si la boucle se termine sans `break`** :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
for nom in magasins:
    if nom == "Sintani":
        print("trouve :", nom)
        break
else:
    print("magasin introuvable")
```

```
magasin introuvable
```

Sintani n'étant pas dans les 5 magasins du socle, aucun `break` — le `else` a parlé. La recherche de « Koudougou » aurait, elle, cassé la boucle et tué le `else`. C'est le rapport « client non trouvé » sans variable drapeau en mémoire.

5.5 Les compréhensions — le tapis avec filtre, en une ligne

La forme : `[expression for x in iterable if condition]` — on lit la phrase de gauche à droite : « pour chaque montant, **si** il dépasse 1 000, le garder » :

```
montants = [500, 1500, 8000, -50, 1350, 200, 4500, 90]
gros = [m for m in montants if m > 1000]
print(gros)
print(len(gros))
```

```
[1500, 8000, 1350, 4500]
4
```

Définition. compréhension — list comprehension — `[expr for x in it if cond]` : une liste construite en **une** ligne, lisible de gauche à droite comme une phrase. C'est le raccourci du « `for` + filtre + `append` » — même résultat, moins de lignes, et l'intention lisible d'un coup d'œil.

Attention. La **règle des 2 niveaux** : une compréhension avec **un** `for` et **une** `if`, c'est une phrase. Deux `for` imbriqués (`[(m, t) for m in magasins for t in trimestres]`), ce n'est plus une phrase, c'est un tableau croisé déguisé — on revient à une boucle classique, c'est **plus lisible** et **plus facile** à déboguer. Le croisé magasins × trimestres (l'exemple d'atelier : 3 magasins × 4 trimestres = 12 paires) est l'endroit où la règle se voit.

5.6 `zip` — deux listes, un seul parcours

Deux colonnes du même tableau, lues **en lockstep** — les 5 magasins et leurs surfaces réelles (magasin.csv) :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
surfaces = [450, 380, 520, 290, 210]
for nom, s in zip(magasins, surfaces):
    print(nom, "-", s)
```

```
Ouaga Centre - 450
Ouaga Patte d'Oie - 380
Bobo Centre - 520
Bobo Sarfalao - 290
Koudougou - 210
```

En SQL, c'est juste `SELECT nom, surface_m2 FROM magasin` — les deux colonnes arrivent déjà appariées dans la ligne. En Python, `zip` fait le même appariement sur deux listes distinctes. (`zip` s'arrête à la **plus courte** des deux : si les listes ne sont pas de même longueur, l'écart est **silencieux** — le §9 en fait une règle de vérification.)

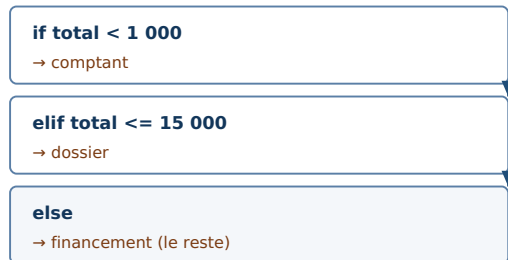
Dans les faits. Le socle M08 contient bien une table `objectif_magasin` — **vide** (0 ligne, clé `m08p_objectif_lignes`). Le parcours « magasins + objectifs » que ce chapitre illustre tourne donc sur des surfaces (données réelles) ; l'import de la table vide et la question « que fait un `for` sur 0 ligne ? » attendent C07 — réponse : rien, et c'est normal.

Le flux complet du chapitre — l'échelle à gauche, la boucle et ses issues à droite :

Décider et répéter : le flux de plusieurs valeurs

C02 a donné la grammaire d'une valeur ; C03 donne le flux : carrefour, tapis, porte de sortie.

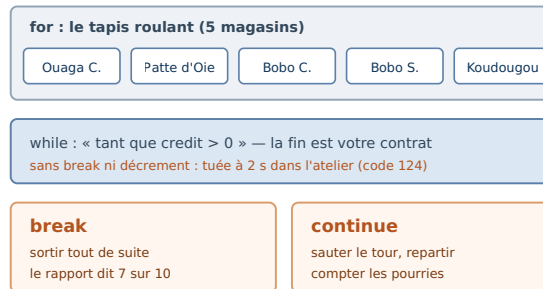
Décider — l'échelle if/elif/else



la 1re condition vraie gagne — le reste est sauté
(le CASE WHEN de M06, ligne par ligne)

l'indentation EST la syntaxe : 4 espaces par niveau

Répéter — for / while



for...else : l'else parle si personne n'a cassé
(le « magasin introuvable » sans variable drapeau)

Résumer : [m for m in montants if m > 1000] — le tapis avec filtre, en une ligne

zip(magasins, surfaces) : deux listes en lockstep — le SELECT de 2 colonnes, en Python

Règles : for si on compte · while si on ne compte pas · compréhension = 1 niveau, 1 condition

6. Exemple concret — le contrôle qualité express

La question du chapitre, en langage de comptoir : « 10 montants viennent de la caisse : lesquels sont en ordre, lesquels sont à revoir, et s'arrêter si les anomalies dérapent. » Le script — exécuté dans l'atelier le 19/09/2026 (le seuil de revue est 15 000 FCFA) :

```

montants = [500, 1500, 8000, "N/C", -50, 1350, 15000.5, 200, 4500, 90]
seuil = 15000
anomalies = 0
traites = 0
for m in montants:
    traites = traites + 1
    if isinstance(m, str) or m < 0 or m > seuil:
        anomalies = anomalies + 1
        print(anomalies, "anomalie :", m)
        if anomalies == 3:
            break
print("traites :", traites, "sur", len(montants))
print("anomalies :", anomalies)
  
```

```

1 anomalie : N/C
2 anomalie : -50
3 anomalie : 15000.5
traites : 7 sur 10
anomalies : 3
  
```

Lecture du rapport : le 4^e montant ("N/C", du texte dans une liste de nombres — le piège de C02 qui redevient utile) est l'anomalie 1 ; le négatif, l'anomalie 2 ; le montant de 15 000,50 FCFA (clé m08p_demo_montant_fcfa) dépasse le seuil de 15 000 → anomalie 3, **le break coupe** : 7 montants traités sur 10, les 3 derniers (200, 4500, 90) n'ont pas été vus — c'est **l'effet cherché** (le contrôle ne traîne pas) et **la limite** (le rapport doit le dire, et il le dit : « 7 sur 10 »).

Le parallèle SQL : le même contrôle sur une table serait `SELECT COUNT(*) FROM vente WHERE montant < 0 OR montant > 15000` — une **agrégation sur tout** (la table ne « s'arrête pas ») ; en Py-

thon, on peut **choisir** de s'arrêter au milieu, ce que SQL ne fait pas. C'est toute la différence de posture : le SGBD traite le **lot**, Python traite le **flux**.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Du zéro au contrôle qualité, sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — Décider seul (l'échelle sur un total nu) :

```
$ python3 -c "total = 8000; print('comptant' if total < 1000 else 'dossier' if total <= 15000 else 'financement')"
```

Interprétation : 8 000 passe entre 1 000 et 15 000 — dossier. L'expression conditionnelle de C02 (la `CASE WHEN` en une ligne) est l'échelle repliée ; le bloc `if/elif/else` est la même échelle dépliée.

Étape 2 — Parcourir les 5 magasins (le `for` + `enumerate`) :

```
$ python3 -c "
magasins = ['Ouaga Centre', \"Ouaga Patte d'Oie\", 'Bobo Centre', 'Bobo Sarfalao', 'Koudougou']
for i, nom in enumerate(magasins):
    print(i + 1, nom)"
1 Ouaga Centre
2 Ouaga Patte d'Oie
3 Bobo Centre
4 Bobo Sarfalao
5 Koudougou
```

Interprétation : 5 tours de tapis, un par magasin — le `for` compte tout seul, pas de compteur à maintenir.

Étape 3 — Le « tant que » du crédit (le `while` de §5.3) :

```
$ python3 credit.py
mois 1 : reste 2000
mois 2 : reste 1500
mois 3 : reste 1000
mois 4 : reste 500
mois 5 : reste 0
solde en 5 mois
```

Interprétation : la condition (`credit > 0`) décide de la fin — 5 tours, pas un de plus ; le dernier affichage montre la condition qui devient fausse (reste 0).

Étape 4 — S'arrêter au premier dépassement (le `break`) :

```
$ python3 -c "
montants = [120, 800, 2000, 15000.5, 500]
for m in montants:
    if m > 15000:
        print('dépassement : ', m)
        break"
dépassement : 15000.5
```

Interprétation : le parcours s'est arrêté au milieu — le 500 n'a pas été vu ; c'est le contrôle qui décide de la fin, pas la fin de la liste.

Étape 5 — Résumer en une ligne (la compréhension avec `isinstance`) :

```
$ python3 -c "
montants = [500, 1500, 8000, 'N/C', -50, 1350, 200, 4500, 90]
gros = [m for m in montants if isinstance(m, (int, float)) and m > 1000]
print(gros)"
[1500, 8000, 1350, 4500]
```

Interprétation : le `isinstance` garde les nombres et écarte les textes — le « N/C » n'arrive jamais au comparateur, donc pas de `TypeError` (l'erreur de C02) : le filtre de la compréhension est la porte de C02, déplacée dans une ligne.

Étape 6 — Deux listes en parallèle (le `zip`) :

```
$ python3 -c "
magasins = ['Ouaga Centre', '\Ouaga Patte d'Oie\ ', 'Bobo Centre', 'Bobo Sarfalao', 'Koudougou']
surfaces = [450, 380, 520, 290, 210]
for nom, s in zip(magasins, surfaces):
    print(nom, '-', s)"
Ouaga Centre - 450
Ouaga Patte d'Oie - 380
Bobo Centre - 520
Bobo Sarfalao - 290
Koudougou - 210
```

Interprétation : 5 paires, lues en lockstep — le `SELECT nom, surface_m2` de M06, écrit en Python.

8. Erreurs fréquentes

1. **IndentationError — l'alignement qui casse le script.** Deux formes exécutées dans l'atelier. Le bloc **manquant** après un `if` :
text File "<string>", line 7 print('comptant') ^^^^^ IndentationError: expected an indented block after 'if' statement on line 6 Et la ligne **mal alignée** (4 espaces, puis 2) : text File "<string>", line 4 print('mal aligne') ^ IndentationError: unindent does not match any outer indentation level Remède : **4 espaces par niveau**, jamais de mélange espaces/tabulations ; l'indentation est la syntaxe, pas le décor.
2. **range(10) = 0 à 9.** La somme « de 1 à 10 » écrite `for n in range(10)` donne **45**, pas 55 — exécuté dans les deux sens dans l'atelier : text \$ python3 -c "total = 0 for n in range(10): total = total + n print(total)" 45 \$ python3 -c "total = 0 for n in range(1, 11): total = total + n print(total)" 55 Remède : « de a à b **compris** » = `range(a, b + 1)` ; le dernier index est `n - 1`.
3. **La boucle infinie — mesurée, pas subie.** `while True` sans `break` : l'atelier la tue à 2 secondes (code de sortie **124**), aucune sortie ; `while credit > 0` avec un oubli de décrémentation (`credit = credit + mensualite` au lieu de `-`) : pareil, **124**. Remède : avant chaque `while`, répondre à voix haute « **qu'est-ce qui rend la condition fausse ?** » — si la réponse n'est pas dans le corps, il manque un `break` ou un décrémentation.
4. **Remplir la liste qu'on parcourt — la boucle qui s'alimente elle-même.** text \$ timeout 2 python3 -c " l = [1] for x in l: l.append(x * 2) print(len(l))" code de sortie : 137 (processus tué après 2 s) Chaque tour **ajoute** un élément, le tapis n'en finit pas — le code 137 (tué) est la même leçon que le 124. Remède : on construit **une autre** liste pendant qu'on parcourt la première — et c'est exactement ce que fait la compréhension du §5.5.
5. **Le deux-points manquant — le bloc qui n'existe pas.** `if total < 1000` sans `:` : `SyntaxError: expected ':'` (C02 §5.6) — en C03, c'est l'erreur **au bord** d'un bloc entier : tout ce qui était « dedans » n'existe plus. Remède : taper le `:` **immédiatement** après la condition, avant la ligne d'en dessous.
6. **La compréhension qui s'imbrique.** `[x for m in magasins for t in trimestres ...]` : au deuxième `for`, la ligne n'est plus lisible d'un coup d'œil — ce n'est pas une erreur d'exécution.

tion, c'est une **erreur de lecture** qui se paiera au premier débogage. Remède : la règle des 2 niveaux (§5.5) — une boucle, un commentaire.

9. Bonnes pratiques professionnelles

1. **Un test par échelon, une échelle par décision** : si le `if` porte deux questions différentes (« est-ce un nombre ? » puis « est-ce en dessous du seuil ? »), ce sont deux `if` imbriqués ou deux variables de statut — pas un `or` monstre dans la condition.
2. **for quand on peut compter, while quand on ne peut pas** : 5 magasins → `for` ; « tant que le crédit dépasse 0 » → `while`. Un `while` qui compte en parallèle est un `for` déguisé (et plus fragile).
3. **Compréhension : un niveau, une condition, pas trop longue** — au-delà, boucle. Et quand la compréhension **filtre**, nommer ce qu'on garde (`gros_montants`, pas `liste2`).
4. **Vérifier la longueur avant le zip** : `assert len(magasins) == len(surfaces)` en tête — le `zip` qui s'arrête à la plus courte est **silencieux** (aucune erreur, des paires manquantes : le pire des défauts).
5. **Le break porte une raison, le continue porte un compte** : tout `continue` sur ligne pourrie incrémente un compteur (`pourries`) — le rapport final doit pouvoir dire « 2 lignes ignorées », sinon l'ignorance est invisible.

10. Exercice guidé — « le registre à trois modes » (15 min, /10)

Énoncé. Écrire `registre.py` qui :

1. parcourt les 3 ventes `[500, 1200, 15000.5]` avec leur numéro (`enumerate`) ;
2. classe chaque total sur l'échelle de §5.1 (`comptant / dossier / financement`) ;
3. compte les ventes par mode ;
4. affiche chaque vente (`vente n : total -> mode`), puis une ligne de récapitulatif.

Barème. L'échelle avec seuils **en variables nommées** (4 pts) · le parcours avec `enumerate` (2 pts) · les 3 compteurs mis à jour **dans** la bonne branche (2 pts) · la sortie propre, alignée avec le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
vente 1 : 500 -> comptant
vente 2 : 1200 -> dossier
vente 3 : 15000.5 -> financement
recap : comptant 1 | dossier 1 | financement 1
```

11. Exercices autonomes

Exercice 3.1 — La somme. Sans `sum()`, calculer la somme des nombres de 1 à 10 avec une `for` + `range`. (5 min)

Exercice 3.2 — Le crédit. Un crédit de 3 500 FCFA, remboursé à 500 par mois : combien de mois pour solde ? (`while` + compteur.) (5 min)

Exercice 3.3 — Le premier. Dans `[120, 800, 2000, 5200, 900]`, afficher l'**index** et la valeur du **premier** montant au-dessus de 5 000 FCFA, puis s'arrêter. (`for` + `enumerate` + `break`.) (5 min)

Exercice 3.4 — Les pourries. Dans `[500, "N/C", 1500, "N/C", 1350, "N/C"]`, additionner les nombres seulement (`continue`), et afficher le total **et** le nombre de nombres valides. (5 min)

Exercice 3.5 — Le `for...else`. Pour les requêtes « Koudougou » et « Sintani », afficher « trouve, position n » ou « introuvable » — avec un seul `for...else`, pas de variable drapeau. (10 min)

12. Correction détaillée

Exercice guidé. Le script qui fait l'affaire :

```
ventes = [500, 1200, 15000.5]
seuil_comptant = 1000
seuil_financement = 15000
comptant = dossier = financement = 0
for i, total in enumerate(ventes):
    if total < seuil_comptant:
        mode = "comptant"
        comptant = comptant + 1
    elif total <= seuil_financement:
        mode = "dossier"
        dossier = dossier + 1
    else:
        mode = "financement"
        financement = financement + 1
    print(f"vente {i + 1} : {total} -> {mode}")
print("recap : comptant", comptant, "| dossier", dossier, "| financement", financement)
```

Deux points de correction : les seuils sont **nommés** en tête (une ligne change si un seuil change) ; les compteurs sont mis à jour **dans la même branche** qui pose le mode — si le comptage était fait après l'échelle (avec un second `if` sur `mode`), deux échelles racontent la même histoire : l'une ment à la seconde.

Exercice 3.1. `for n in range(1, 11): s = s + n` → **55**. Le piège est le `range(10)` (§8, erreur n° 2) : il donne 45, et c'est exactement l'écart du 10 manquant.

Exercice 3.2. `while credit > 0` avec `credit = credit - 500` → **7** mois ($3\,500 \div 500 = 7$ pile — le dernier tour met le solde à 0, la condition devient fausse, la boucle s'arrête seule).

Exercice 3.3. `for i, m in enumerate(montants): if m > 5000: print(i, m); break` → **3 5200** : l'index de 5 200 est **3** (zéro-indexé, §5.2) — afficher `i + 1` donnerait la « position 4 » : les deux sont justes, il faut **choisir** et le dire.

Exercice 3.4. `continue` sur `"N/C"`, sinon `total = total + s` et compteur — → **3350 3** : le total des 3 nombres ($500 + 1\,500 + 1\,350$), les 3 « N/C » ignorés.

Exercice 3.5. Le `for...else` imbriqué dans la boucle des requêtes :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
for requete in ("Koudougou", "Sintani"):
    for nom in magasins:
        if nom == requete:
            print(requete, "-> trouve, position", magasins.index(nom) + 1)
            break
    else:
        print(requete, "-> introuvable")
```

```
Koudougou -> trouve, position 5
Sintani -> introuvable
```

L'`else` est attaché au `for` **intérieur** (même indentation que le `for`, pas au `if`) — le `break` de l'intérieur le tue, l'absence de `break` le libère.

13. Mini-projet M08.P3 — « Le rapport de qualité complet » (1 h)

Énoncé. Écrire `qualite.py` — le contrôle qualité de §6, durci :

- la liste des 10 montants (celle de §6) est en tête, le seuil de revue (15 000 FCFA) est une **variable nommée** ;
- quatre compteurs : `illisible` (du texte), `negatif`, `a_revoir` (au-dessus du seuil), `conforme` — et un compteur `traites` ;
- le parcours affiche chaque anomalie numérotée, et **s'arrête au 3^e dépassement** (`break`) ;
- le rapport final : « montants traités : n / 10 », puis les 4 compteurs — alignés, lisibles ;
- si **aucune** anomalie n'est trouvée, le rapport dit « contrôle terminé sans dépassement » (indice : `for...else`).

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier donne « arrêt au 3e depassement : 15000.5 », « montants traites : 7 / 10 », illisible 1 · négatif 1 · a revoir 1 · conforme 4 (1 pt) · aucun or monstre : un test par échelon (1 pt) · le compteur `pourries/traites` est mis à jour **avant** le test (1 pt) · docstring en tête, seuil nommé (1 pt).

Note de correction. La sortie attendue est celle de l'atelier (D9 du 19/09/2026) : 7 montants traités sur 10 (le `break` coupe à la 3^e anomalie), 4 conformes **parmis les traités** — les 3 montants non vus (200, 4500, 90) n'apparaissent dans **aucun** compteur : c'est l'effet du `break`, et le rapport est honnête parce qu'il dit « 7 / 10 ».

14. Boîte à outils du chapitre

Boîte à outils. Toujours Python nu + terminal (couche 2 de C01) ; la boîte du chapitre s'ajoute aux encadrés de C02 : l'échelle, le tapis, la porte de sortie, la phrase-liste.

Outil	Usage	Syntaxe
<code>if / elif / else</code>	l'échelle de décision	<code>if a < b: ... elif a < c: ... else: ...</code>
<code>for + range</code>	le tapis des nombres	<code>for n in range(1, 11):</code>
<code>enumerate</code>	index et valeur	<code>for i, nom in enumerate(magasins):</code>
<code>while</code>	le « tant que »	<code>while credit > 0:</code>
<code>break / continue</code>	la sortie / le « passe »	<code>break · continue</code>
<code>for...else</code>	le « non trouvé »	<code>for ... : ... else: ...</code>
compréhension	la liste en une ligne	<code>[m for m in montants if m > 1000]</code>
<code>zip</code>	deux listes en parallèle	<code>for nom, s in zip(magasins, surfaces):</code>

15. Résumé du chapitre

- L'échelle `if/elif/else` est le `CASE WHEN` de M06 en Python : **la première vraie gagne**, l'indentation **est** la syntaxe.
- Le `for` est le tapis roulant (il compte tout seul) ; `range(10)` donne **0 à 9** ; `enumerate` porte l'index et la valeur.
- Le `while` est le « tant que » — la fin est **votre** contrat : la boucle infinie mesurée (tuée à 2 s, code 124) est la preuve qu'un `break` ou un décrétement manquait.

- `break` **termine** le parcours (le rapport dit « 7 sur 10 ») ; `continue` **réduit** un tour (le compteur de pourries dit « 2 ignorées ») ; le `for...else` parle quand **personne** n'a cassé.
- La compréhension est le tapis avec filtre **en une ligne** — un niveau, une condition ; au-delà, une boucle.
- `zip` lit deux colonnes en lockstep — le `SELECT nom, surface` de M06, en Python ; il s'arrête à la plus courte **en silence** : vérifier la longueur.

16. À retenir

À retenir. Avant chaque `while`, la question à voix haute : « **qu'est-ce qui rend la condition fausse ?** » — si la réponse n'est pas dans le corps de la boucle, la boucle est infinie. C'est la seule inspection qui empêche la machine de tourner toute la nuit.

À retenir. `break` **termine**, `continue` **passe** — et les deux laissent des **traces** : le compteur `traites` après un `break`, le compteur `pourries` après un `continue`. Un rapport qui ne dit pas ce qu'il a vu et ce qu'il a ignoré n'est pas un rapport.

17. Évaluation formative (auto-correction, 8 min)

1. Sur l'échelle « `comptant < 1 000 ≤ dossier ≤ 15 000 < financement` », quel mode pour 15 000,50 — et pourquoi `elif` et pas deux `if` ? → `financement` (15 000,50 > 15 000) ; deux `if` séparés **testeraient les deux échelons** (le total pourrait recevoir deux modes) — `elif` garantit une seule branche. (1 pt)
2. `range(10)` donne quoi, et comment écrire « 1 à 10 compris » ? → 0 à 9 ; `range(1, 11)`. (1 pt)
3. `while credit > 0` avec `credit = credit - 500` mais `credit = 2500` : combien de tours, et qu'est-ce qui rend la condition fausse ? → 5 tours ; le décrétement du corps — la dernière itération met `credit` à 0. (1 pt)
4. `break` et `continue` — la différence en une phrase, et le métier de chacun ? → `break` sort de la boucle (arrêt au premier dépassement) ; `continue` saute le reste du tour et reprend au début (ignorer les lignes « N/C »). (2 pts)
5. Quand le `else` d'un `for...else` s'exécute-t-il ? → quand la boucle se termine **sans** `break` — le cas « non trouvé ». (1 pt)
6. La compréhension `[m for m in montants if m > 1000]` fait quoi, et quelle est la règle des 2 niveaux ? → une liste des montants > 1 000, en une ligne ; au-delà de `un for + une if`, on repasse à une boucle classique. (2 pts)
7. `zip(magasins, surfaces)` s'arrête où si les listes ne font pas la même longueur ? → à la plus courte, en silence (aucune erreur) — d'où la vérification de longueur. (1 pt)
8. Le contrôle de §6 affiche « `traites : 7 sur 10` » : pourquoi 7, et que font les 3 montants non vus ? → le `break` a coupé à la 3^e anomalie (le 7^e montant) ; les 3 suivants n'ont été traités nulle part — d'où le « / 10 » qui rend le rapport honnête. (1 pt)

Module M08.C04 — Ranger : listes, tuples, chaînes, dictionnaires, ensembles, index, tris

Outils : Python 3.13.14 seul (l'interpréteur), le terminal. Aucun paquet. Durée indicative : 4 h. Niveau : N4 → N5. Prérequis : C03 (le flux : décider, répéter).

L'idée du chapitre. C03 a donné le **flux** (décider, répéter, s'arrêter) ; C04 donne le **rangement** : les 5 conteneurs de Python et la question métier que chacun répond. La **liste** (une case numérotée, changeable, doublons admis), le **tuple** (l'enveloppe scellée, immuable), la **chaîne** (la séquence de caractères, et les méthodes du format français), le **dictionnaire** (l'étagère étiquetée — le `GROUP BY` avant pandas), l'**ensemble** (le tamis — le `COUNT(DISTINCT)` avant pandas). Le fil rouge : les 5 magasins du socle entrent dans 4 contenants, et 5 questions métier trouvent chacune leur contenant. En C07, le `DataFrame` prend le relais : une colonne est une liste indexée, une table est un dictionnaire de colonnes — tout ce chapitre est le préambule de C07.

Matériel de l'atelier — Python 3.13.14, terminal, zéro paquet. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les données citées sont celles du socle M08 : les 5 magasins (`magasin.csv`, avec leurs villes et surfaces), 3 des libellés de paiement de `mode_paiement.csv`, le panier moyen 158 140 FCFA (clé `m08p_paiement_moyen`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Construire une **liste** (`append`, `extend`), la lire à l'index (positif et négatif), la trancher (`[1:3]`, `[::2]`, `[::-1]`) — et lire un `IndexError` sans surprise.
2. Dire quand choisir un **tuple** plutôt qu'une liste (l'immuable : clé de dictionnaire, résultat figé), et faire l'**unpacking** (`n, p = ...`).
3. Nettoyer et formater des **chaînes** avec `strip`, `split`, `replace`, `startswith` — la recette du format français en méthode.
4. Utiliser un **dictionnaire** : `d[clé]`, `d.get(clé, défaut)`, `clé in d` — et écrire le **compteur** (le `GROUP BY` avant pandas).
5. Compter les **valeurs uniques** avec `len(set(...))` (le `COUNT(DISTINCT)` avant pandas) et croiser deux ensembles (l'intersection).
6. Trier : `sorted(...)` (ne modifie pas), `.sort(reverse=True)` (modifie), et le **top N** d'un dictionnaire en une ligne.

2. Pourquoi cette notion est importante

Parce qu'en Python, **choisir le contenant, c'est choisir la réponse**. Une même donnée posée dans le mauvais contenant rend la bonne question difficile et la mauvaise facile : les villes dans une **liste** et « combien de villes distinctes ? » demande un `len(set(...))` ; les villes dans un **ensemble**, la réponse est immédiate. Les 5 contenants ne sont pas 5 boîtes interchangeables, ce sont 5 **postures** : l'ordre (liste), le figé (tuple), le texte (chaîne), l'étiquette (dictionnaire), l'unicité (ensemble).

Et parce que le parallèle SQL est déjà **enfermé** dans ces 5 contenants : le compteur de dictionnaire de §5.4 **est** le `SELECT id_magasin, COUNT(*) FROM vente GROUP BY id_magasin` de M06/M07, écrit en Python ; `len(set(villes))` **est** le `COUNT(DISTINCT ville)` ; le `sorted(d.items(), key=...)` de §5.6 **est** le `ORDER BY ... LIMIT n`. C07 ne surprendra pas : pandas est le même rangement, à l’échelle de la table.

Enfin, les index sont la base de C07 : `l[0]`, `l[-1]`, `l[1:3]` sont les frères aînés de `df.loc[3]` et `df.iloc[1:4]` — la même idée (la ligne **n** porte l’index **n – 1**), la même famille d’erreurs (`IndexError` aujourd’hui, `KeyError/IndexError` en C07).

3. Explication simple — la case, l’enveloppe, l’étagère, le tamis

- La **liste** est une **case numérotée** : les éléments ont un ordre et un numéro, on en ajoute, on en change, on en duplique — comme une file d’attente.
- Le **tuple** est l’**enveloppe scellée** : une fois fermé, on ne change plus rien dedans — c’est ce qui le rend **fiable** (il peut servir de clé, la clé ne doit pas changer sous le nez du placard).
- La **chaîne** est une **ligne de caractères** : elle aussi a des index (chaque lettre a son numéro) et des méthodes (`strip`, `split`, `replace`) — c’est là que vit le format français.
- Le **dictionnaire** est l’**étagère étiquetée** : chaque étiquette (clé) porte un objet (valeur) ; on cherche par étiquette, pas par numéro — comme un annuaire.
- L’**ensemble** est le **tamis** : les doublons tombent, il ne reste que les uniques — « combien de villes distinctes ? » ne se pose même plus.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
séquence sequence	— un contenant ordonné, lisible par index (liste, tuple, chaîne).	indexer par étiquette une séquence : l’étiquette, c’est le dictionnaire.
index — index	le numéro d’une position dans une séquence — la ligne n porte l’index n – 1 .	écrire <code>l[n]</code> pour la ligne n : c’est <code>l[n – 1]</code> — le décalage qui fabrique l’ <code>IndexError</code> .
tranche — slice	<code>l[a:b]</code> : les éléments de l’index a à l’index b – 1 (la fin est exclue).	croire que <code>l[1:3]</code> contient 3 éléments : elle en contient 2.
clé — key	l’étiquette d’une case de dictionnaire ; clé <code>in d</code> teste les clés, pas les valeurs.	chercher une valeur avec <code>in d</code> : <code>in</code> regarde les étiquettes, jamais les objets.
valeur unique unique value	— une valeur qui ne compte qu’une fois : le rôle de l’ensemble (<code>set</code>).	compter les uniques dans une liste (avec des doublons) : le <code>len</code> de la liste ment.
immuable immutable	— un objet qu’on ne peut pas modifier en place (tuple, chaîne, nombre).	vouloir <code>t[0] = ...</code> sur un tuple : <code>TypeError</code> — on refait un tuple, on ne répare pas.
table de données — dataframe	une table à colonnes, chaque colonne indexée : l’objet central de C07, annoncé ici.	la confondre avec une liste : une table a des colonnes nommées , une liste des index.

5. Cours approfondi

5.1 La liste — la case numérotée

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre"]
magasins.append("Bobo Sarfalao")
magasins.extend(["Koudougou"])
print(len(magasins))
print(magasins[-1])
print(magasins[1:3])
print(magasins[:2])
print(magasins[::-1][:2])
```

```
5
Koudougou
["Ouaga Patte d'Oie", 'Bobo Centre']
['Ouaga Centre', 'Bobo Centre', 'Koudougou']
['Koudougou', 'Bobo Sarfalao']
```

Définition. séquence — sequence — un contenant ordonné, lisible par index : liste, tuple, chaîne. La liste est la séquence **mutable** (on ajoute avec `append`, on étend avec `extend`, on change la case en place) — le conteneur par défaut.

Deux remarques sur la sortie : `magasins[-1]` est **Koudougou** (l'index négatif compte à rebours : `-1` = le dernier), et la tranche `magasins[1:3]` contient **2** éléments ("Ouaga Patte d'Oie" et "Bobo Centre") — la borne de **fin** est exclue (en SQL, `BETWEEN`, lui, **inclut** ses bornes : deux conventions opposées, à retenir), et le §8 en fait une erreur.

5.2 Le tuple — l'enveloppe scellée

Le tuple est la liste **qu'on ne rouvre plus**. Essayez de modifier une case — la sortie réelle de l'atelier :

```
surface = (450, 380, 520, 290, 210)
surface[0] = 999
```

```
TypeError: 'tuple' object does not support item assignment
```

Le tuple **se lit** comme une liste, **ne s'écrit** pas. Deux usages métier en découlent :

1. **L'unpacking** — ouvrir l'enveloppe d'un coup, en donnant un nom à chaque contenu :

```
surface = (450, 380, 520, 290, 210)
n, p = len(surface), surface[0]
print(n, p)
```

```
5 450
```

1. **La clé de dictionnaire** — une clé ne doit pas changer sous le placard ; le tuple, scellé, a le droit d'y entrer (la liste, mutable, n'y a pas droit) :

```
cle = ("Ouaga Centre", "Ouagadougou")
d = {cle: 450}
print(d[cle])
```

```
450
```

Définition. immuable — immutable — un objet qu'on ne peut pas modifier en place (tuple, chaîne, nombre) : pour « changer » une valeur, on **construit** un nouvel objet (`surface + (210,)`, pas `surface[5] = 210`). C'est la garantie du tuple en clé de dictionnaire.

Quand tuple, quand liste ? Si les éléments sont un **fait** (les surfaces des 5 magasins — ça ne bouge pas) : tuple. S'ils vont **grandir ou changer** (les ventes du jour) : liste. Le choix est une promesse au lecteur, pas une préférence.

5.3 La chaîne — le format français en méthodes

La recette du format français de C02, écrite en **méthodes de chaîne** — exécutée dans l'atelier sur un montant encadré d'espaces (le cas réel des imports) :

```
s = " 15 000,50 "
print(s.strip())
print(s.strip().split(" "))
print(s.strip().replace(" ", "").replace(",", "."))
print(s.startswith(" "))
```

```
15 000,50
['15', '000,50']
15000.50
True
```

- `strip()` : arrache les espaces **aux deux bouts** (le montant tel qu'il sort d'un fichier mal aligné) ;
- `split(" ")` : découpe en morceaux — ici `['15', '000,50']` : les **espaces du nombre** sont des séparateurs pour Python, d'où la `replace` d'après ;
- `replace(" ", "").replace(",", ".")` : la conversion de C02, en chaîne — `float()` de l'autre côté ;
- `startswith(" ")` : la **vérification** (le montant est-il encadré ?) avant le nettoyage.

Et le formatage **dans** le sens inverse — afficher un nombre au format lisible (la valeur est le panier moyen réel du socle, clé `m08p_panier_moyen`) :

```
$ python3 -c "print(f\"panier moyen : {158140:,.0f} FCFA\")"
panier moyen : 158,140 FCFA
```

Définition. méthode de chaîne — string method — un verbe attaché à une chaîne (`s.strip()`, `s.split(" ")`) : elle **renvoie une nouvelle** chaîne sans toucher à l'originale (la chaîne est **immutable**, §5.2) — d'où l'enchaînement `s.strip().replace(...)`.

5.4 Le dictionnaire — l'étagère étiquetée

```
vente_par_magasin = [1, 2, 1, 3, 1, 4, 2, 1]
compteur = {}
for idm in vente_par_magasin:
    compteur[idm] = compteur.get(idm, 0) + 1
print(compteur)
print(compteur.get(5, 0))
print(1 in compteur, 5 in compteur)
```

```
{1: 4, 2: 2, 3: 1, 4: 1}
0
True False
```


Le **compteur** est la construction du chapitre — lisez-la lentement, elle revient dans tous les modules : pour chaque id, `compteur.get(idm, 0)` prend le comptage **ou 0 si la clé est absente**, on ajoute 1, on repose. La sortie **est** le `GROUP BY` :

Python (§5.4)	SQL (M06/M07)	Résultat
<code>compteur[idm] = compteur.get(idm, 0) + 1</code>	<code>COUNT(*) ... GROUP BY id_magasin</code>	magasin 1 : 4 ventes
<code>compteur.get(5, 0)</code>	<code>... WHERE id_magasin = 5 (rien)</code>	0 — pas d'erreur
<code>1 in compteur</code>	<code>SELECT 1 FROM vente WHERE id_magasin = 1 (test d'existence)</code>	True

Trois détails qui portent tout :

1. `d[clé]` **lève** si la clé est absente (le `KeyError` du §8) ; `d.get(clé, défaut)` **renvoie** le défaut — c'est la différence entre un script qui s'effondre et un script qui compte.
2. `clé in d` teste les **clés** — jamais les valeurs (chercher « 4 » dans les valeurs demande un parcours, §8 erreur n° 3).

Attention. `clé in d` teste les **clés**, jamais les valeurs : `4 in compteur` demande « la clé 4 existe-t-elle ? », pas « la valeur 4 existe-t-elle ? » — la recherche dans les valeurs est un parcours, pas une étiquette (§8, erreur n° 4). 3. Un dictionnaire **n'a pas d'ordre métier** : `{1: 4, 2: 2, 3: 1, 4: 1}` se lit par clé, pas par place — c'est exactement ce qu'on veut d'un `GROUP BY`.

Conseil professionnel. Sur les données qu'on ne **possède** pas (fichier, saisie, API), la lecture est `d.get(clé, défaut)`, jamais `d[clé]` : la clé absente est un **cas prévu** du métier, pas une exception — un `KeyError` levé sur un cas prévu est le script qui s'effondre de politesse.

Définition. **clé** — key — l'étiquette d'une case de dictionnaire : `d[clé]` lit, `d[clé] = valeur` écrit, `clé in d` teste. Une clé est **unique** dans son dictionnaire (reposer une clé **écrase** la valeur) — c'est l'unicité qui fait la recherche instantanée.

5.5 L'ensemble — le tamis

Les 5 magasins du socle se répartissent sur 3 villes (`magasin.csv`) :

```
villes = ["Ouagadougou", "Ouagadougou", "Bobo-Dioulasso", "Bobo-Dioulasso", "Koudougou"]
print(len(set(villes)))
```

3

Définition. **valeur unique** — unique value — une valeur qui ne compte qu'une fois : `set(villes)` passe la liste au tamis (5 éléments → 3), `len(set(...))` compte les uniques. C'est le `COUNT(DISTINCT ville)` de M06, écrit en Python.

Et l'**intersection** — les clients qui sont dans les **deux** magasins (les codes de démo, le principe) :

```
c1 = {101, 102, 103, 108}
c2 = {102, 104, 105}
print(c1 & c2)
```

{102}

`c1 & c2` est le **JOIN interne** des deux ensembles : un seul code commun (102) passe le tamis. La union (`c1 | c2`), la différence (`c1 - c2`) : les trois opérateurs d'ensemble, les trois équivalents SQL.

Dans les faits. Le socle M08 porte 1 200 clients réels (`m08p_clients_lignes`). En C07, la question « combien de clients distincts ont acheté ? » sera `len(set(colonne_client))` sur la table entière — le même tamis que celui de ce paragraphe, passé de 9 codes de démo à 1 200 lignes.

5.6 Index, tranches, tris

Le tableau d'index complet, sur les 5 magasins (exécuté) :

```
magasins = ["Ouaga Centre", "Ouaga Patte d'Oie", "Bobo Centre", "Bobo Sarfalao", "Koudougou"]
print(magasins[0])
print(magasins[-1])
print(magasins[2:4])
print(magasins[:2])
print(magasins[::-1])
print(magasins[5])
```

```
Ouaga Centre
Koudougou
['Bobo Centre', 'Bobo Sarfalao']
['Ouaga Centre', 'Bobo Centre', 'Koudougou']
['Koudougou', 'Bobo Sarfalao', 'Bobo Centre', 'Ouaga Patte d'Oie', 'Ouaga Centre']
Traceback (most recent call last):
  File "<string>", line 7, in <module>
    print(magasins[5])
    ~~~~~^
IndexError: list index out of range
```

Définition. tranche — slice — `l[a:b]` : les éléments de l'index **a** à l'index **b - 1** (la fin est **exclue**) ; `l[:2]` tous les deux ; `l[::-1]` l'inverse. La tranche **ne lève jamais** hors plage : `magasins[3:9]` renvoie simplement jusqu'à la fin — c'est l'`IndexError` de l'index **simple** qui protège, pas la tranche.

Attention. La fin de tranche est **exclue** : `magasins[1:3]` contient les indexes 1 et 2 — pas 1, 2, 3. « de la ligne a à la ligne b **comprises** » s'écrit `l[a - 1 : b]` : penser en indexes, pas en lignes (§8, erreur n° 6).

Expression	Résultat	Lecture
<code>l[0] / l[-1]</code>	premier / dernier	la ligne 1 / la ligne dernière
<code>l[2:4]</code>	les indexes 2 et 3	2 éléments (fin exclue)
<code>l[:2]</code>	un élément sur deux	pas de 2
<code>l[::-1]</code>	l'inverse	pas -1
<code>l[5]</code> (5 éléments)	<code>IndexError</code>	la ligne 5 a l'index 4

Les **tris** — les deux frères, et le top N (exécutés) :

```
montants = [3500, 2500, 4500, 1500, 2000]
print(sorted(montants, reverse=True))
print(montants)
montants.sort(reverse=True)
print(montants)
```

```
[4500, 3500, 2500, 2000, 1500]
[3500, 2500, 4500, 1500, 2000]
[4500, 3500, 2500, 2000, 1500]
```

`sorted(...)` **renvoie** une nouvelle liste (l'originale est intacte — regardez la 2^e sortie) ; `.sort(...)` **tri en place** (renvoie `None`). Et le **top N d'un dictionnaire** — le `ORDER BY ... LIMIT n` de M06, en une ligne (valeurs de démo) :

```
ca = {1: 25000, 2: 150000, 3: 45000, 4: 120000, 5: 21000}
top2 = sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2]
print(top2)
```

```
[(2, 150000), (4, 120000)]
```

`ca.items()` donne les paires (clé, valeur) ; `key=lambda p: p[1]` dit « trie sur la **valeur** » (le `p[1]` — la 2^e composante de la paire) ; `[:2]` prend le top. La `lambda` — la fonction sans nom — arrive en C05 ; ici, lisez-la comme « la règle de tri, en une ligne ».

Les 5 conteneurs et leurs équivalents SQL, en une planche :

Ranger : 5 conteneurs, 5 questions métier

Choisir le contenant, c'est choisir la réponse — l'autre, c'est du débogage.

liste « le n-ième ? » — l'ordre <code>l[i] · l[1:3] · append</code> la ligne n a l'index n-1 <code>IndexError</code> sinon	tuple « le fait figé » — la clé <code>n, p = t · clé de dict</code> immuable : scellé <code>TypeError</code> si on répare	str « le texte propre » <code>strip · split · replace</code> format français : <code>replace</code> puis <code>float</code> <code>f" {158140:,.0f}"</code>
dict « le comptage par... » <code>d.get(k, 0) + 1</code> = le <code>GROUP BY</code> de M06 <code>KeyError</code> sans <code>.get</code>	set « combien de distincts ? » <code>len(set(x)) · c1 & c2</code> = <code>COUNT(DISTINCT)</code> · <code>JOIN</code> interne le tamis : doublons sortis	tri « le top N » <code>sorted(items, key=...)[n]</code> = <code>ORDER BY ... LIMIT n</code> <code>sorted</code> renvoie · <code>sort</code> modifie

C07 : le DataFrame reprend tout — une colonne = une liste indexée (loc/iloc),
une table = un dictionnaire de colonnes — les 50 008 lignes du socle attendent.

6. Exemple concret — les 5 magasins dans 4 contenants

La question du chapitre, en langage de comptoir : « j'ai la liste brute des 5 magasins (nom, ville, surface) — donnez-moi 4 points de vue dessus. » Le script — exécuté dans l'atelier le 19/09/2026 :

```
brut = [("Ouaga Centre", "Ouagadougou", 450),
        ("Ouaga Patte d'Oie", "Ouagadougou", 380),
        ("Bobo Centre", "Bobo-Dioulasso", 520),
        ("Bobo Sarfalao", "Bobo-Dioulasso", 290),
        ("Koudougou", "Koudougou", 210)]
noms = [ligne[0] for ligne in brut]
coordonnees = tuple((ligne[1], ligne[2]) for ligne in brut)
par_ville = {}
for ligne in brut:
    par_ville.setdefault(ligne[1], []).append(ligne[0])
villes = {ligne[1] for ligne in brut}
print("1. le 3e magasin :", noms[2])
print("2. nombre de villes distinctes :", len(villes))
print("3. magasins de Bobo-Dioulasso :", par_ville.get("Bobo-Dioulasso", []))
print("4. surface de Ouaga Centre :", dict((l[0], l[2]) for l in brut)["Ouaga Centre"])
print("5. le plus grand magasin :", max(brut, key=lambda l: l[2])[0])
```

```
1. le 3e magasin : Bobo Centre
2. nombre de villes distinctes : 3
3. magasins de Bobo-Dioulasso : ['Bobo Centre', 'Bobo Sarfalao']
4. surface de Ouaga Centre : 450
5. le plus grand magasin : Bobo Centre
```

Cinq questions, cinq postures : **index** (le 3^e), **tamis** (les villes distinctes), **étiquette avec défaut** (les magasins d'une ville, `get` — pas de `KeyError` si la ville n'y est pas), **étiquette** (la surface d'un nom), **règle de tri** (le plus grand, `max` avec `key`). Les 4 contenants (liste de noms, tuple de coordonnées, dictionnaire ville → magasins, ensemble de villes) sont construits **une fois** et servent les 5 questions — c'est l'architecture que C07 reprend à l'échelle du `DataFrame`.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — Le `COUNT(DISTINCT)` en une ligne (l'ensemble) :

```
$ python3 -c "
villes = ['Ouagadougou', 'Ouagadougou', 'Bobo-Dioulasso', 'Bobo-Dioulasso', 'Koudougou']
print(len(set(villes)))"
3
```

Interprétation : 5 occurrences, 3 villes — le tamis a fait le `DISTINCT` ; en C07, la même ligne passera sur la colonne `ville` de 50 008 lignes.

Étape 2 — Le `GROUP BY` en 3 lignes (le compteur) :

```
$ python3 -c "
vente_par_magasin = [1, 2, 1, 3, 1, 4, 2, 1]
compteur = {}
for idm in vente_par_magasin:
    compteur[idm] = compteur.get(idm, 0) + 1
print(compteur)"
{1: 4, 2: 2, 3: 1, 4: 1}
```

Interprétation : 8 ventes, 4 magasins — la 5^e n'a pas vendu (et `compteur.get(5, 0)` répondrait 0 sans s'effondrer) : c'est le `GROUP BY`, pas le `JOIN`.

Étape 3 — La tranche des magasins de Bobo (l'index) :

```
$ python3 -c "
magasins = ['Ouaga Centre', \"Ouaga Patte d'Oie\", 'Bobo Centre', 'Bobo Sarfalao', 'Koudougou']
print(magasins[2:4])"
['Bobo Centre', 'Bobo Sarfalao']
```

Interprétation : les indexes 2 et 3, donc les 3^e et 4^e lignes — pas les 2^e et 3^e : la ligne *n* a l'index *n* - 1.

Étape 4 — Le top 2 (le tri par valeur) :

```
$ python3 -c "
ca = {1: 25000, 2: 150000, 3: 45000, 4: 120000, 5: 21000}
top2 = sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2]
print(top2)"
[(2, 150000), (4, 120000)]
```

Interprétation : tri sur la valeur (le `p[1]`), décroissant, tranche `[:2]` — le `ORDER BY ca DESC LIMIT 2`.

Étape 5 — L'intersection des clients (l'ensemble) :

```
$ python3 -c "
c1 = {101, 102, 103, 108}
c2 = {102, 104, 105}
print(c1 & c2)"
{102}
```

Interprétation : un seul client commun — le `JOIN` interne en un symbole.

Étape 6 — Le montant lisible (la chaîne) :

```
$ python3 -c "print(f\"panier moyen : {158140:,.0f} FCFA\")"
panier moyen : 158,140 FCFA
```

Interprétation : le panier moyen réel du socle (clé `m08p_panier_moyen`), formaté en une f-string — le formatage des rapports de tout le module.

8. Erreurs fréquentes

1. **IndexError: list index out of range** — la ligne *n* a l'index *n* - 1. Exécuté dans l'atelier (5 éléments, demande de l'index 5) : `text File "<string>", line 7, in <module> print(magasins[5]) ~~~~~^^^ IndexError: list index out of range` Remède : la 5^e ligne est `magasins[4]` ; pour « la dernière », `magasins[-1]` — jamais d'index calculé en tête sans le - 1.
2. **Modifier un tuple** — **TypeError**. `surface[0] = 999` sur un tuple : `TypeError: 'tuple' object does not support item assignment` (sortie réelle, §5.2). Remède : le tuple **se refait**, ne se répare pas (`surface + (210,)`) — et si c'est censé changer, c'était une **liste**.
3. **d[clé] sur une clé absente** — **KeyError**. Exécuté dans l'atelier : `text File "<string>", line 2, in <module> print(par_ville["Sintani"]) ~~~~~^~~~~~ KeyError: 'Sintani'` Remède : `d.get(clé, défaut)` sur toute donnée qu'on ne **possède** pas (fichier, saisie) — le défaut est la politesse du script (§5.4).
4. **clé in d cherche les clés, pas les valeurs**. `4 in compteur` répond « la clé 4 existe-t-elle ? » — pas « la valeur 4 existe-t-elle ? ». Remède : pour chercher dans les **valeurs**, un petit `for` : `any(v == 4 for v in compteur.values())` — la recherche par valeur est un parcours, pas une étiquette.
5. **sorted vs sort** — le tri qui « ne s'est pas fait ». `tri = montants.sort()` donne `tri = None` : le tri s'est bien fait **dans** `montants`, mais ce qu'on a re-affecté est `None` ; et `sorted(...)` **ne touche pas** l'originale — la sortie réelle le montre (`[3500, 2500, ...]` intacte après le `sorted`, §5.6). Remède : `tri = sorted(montants)` pour garder l'originale, `montants.sort()` (et ne pas re-affecter) pour trier **sur place** — vérifier laquelle on veut **avant** de taper.

6. **La tranche qui « manque » un élément.** `magasins[1:3]` donne 2 éléments, pas 3 — la fin est **exclue**. Remède : « de la ligne a à la ligne b **comprises** » = `l[a - 1 : b]` — ou, plus simple : penser en indexes, pas en lignes.

9. Bonnes pratiques professionnelles

1. **Un contenant par question** : les villes distinctes → ensemble ; le comptage par magasin → dictionnaire ; l'ordre des magasins → liste. Tout mettre dans une liste unique, c'est répondre à toutes les questions avec le même marteau.
2. **d.get(clé, défaut) sur ce qu'on ne possède pas** : fichier, saisie, API — la clé absente est un **cas prévu**, pas une exception (le `KeyError` du §8 est le script qui s'effondre sur un cas prévu).
3. **len(set(...)) dès que « distinct » apparaît** : ne jamais compter les uniques dans une liste en la parcourant à la main — le tamis fait le travail et dit la vérité.
4. **Le top N est toujours sorted(..., key=...)[:n]** — pas de tri manuel, pas de `for` qui cherche le max n fois : une ligne, lisible, testable.
5. **Nommer le contenant par son contenu** : `villes`, `par_ville`, `compteur` — pas `data1`, `liste2` : le nom dit la **question** que le contenant répond.

10. Exercice guidé — « les 5 questions des magasins » (15 min, /10)

Énoncé. Écrire `magasins.py` qui, à partir de la liste brute `brut` (les 5 tuples `(nom, ville, surface)` de §6) :

1. construit **4 contenants** : `noms` (liste), `coordonnees` (tuple de paires ville/surface), `par_ville` (dictionnaire ville → liste de noms), `villes` (ensemble) ;
2. répond aux 5 questions de §6 — avec le **bon contenant** pour chacune.

Barème. Les 4 contenants construits (4 pts) · les 5 questions posées avec le contenant qui les répond — `index` / `len(set(...))` / `get` / dictionnaire / `max` (2,5 pts, -0,5 par question posée au mauvais contenant) · la sortie alignée sur le modèle ci-dessous (2 pts) · aucun `d[clé]` ni sur `par_ville` (1 pt). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
1. le 3e magasin : Bobo Centre
2. nombre de villes distinctes : 3
3. magasins de Bobo-Dioulasso : ['Bobo Centre', 'Bobo Sarfalao']
4. surface de Ouaga Centre : 450
5. le plus grand magasin : Bobo Centre
```

11. Exercices autonomes

Exercice 4.1 — La tranche des Bobo. Sur la liste des 5 magasins (ordre du socle), extraire **par tranche** les 2 magasins de Bobo-Dioulasso. (3 min)

Exercice 4.2 — Le compteur des paiements. Sur la liste `["Espèces", "Mobile Money", "Espèces", "Carte Bancaire", "Mobile Money", "Espèces"]` (les 3 libellés réels de `mode_paiement.csv`), construire le compteur dictionnaire et l'afficher. (5 min)

Exercice 4.3 — Le tamis. Sur la liste `["Ouagadougou", "Bobo-Dioulasso", "Ouagadougou", "Koudougou", "Bobo-Dioulasso", "Ouagadougou"]`, donner le nombre de villes **distinctes** — avec `set`, sans parcours manuel. (3 min)

Exercice 4.4 — Le top 2. Sur le dictionnaire `ca = {1: 25000, 2: 150000, 3: 45000, 4: 120000, 5: 21000}`, afficher le top 2 par valeur, en paires. (5 min)

Exercice 4.5 — Le nettoyage. Transformer la chaîne " 15 000,50 " en float, puis l'afficher avec 2 décimales en f-string. (5 min)

12. Correction détaillée

Exercice guidé. Le script de §6 **est** la correction (exécuté dans l'atelier) — les 5 points de vigilance : `noms` par `index` (`noms[2]`, pas de recherche), `villes` en `set` (le `len` direct), `par_ville.get(..., [])` (**pas** `par_ville[ville]` — le `KeyError` du §8 si la ville est absente), la surface par un dictionnaire `nom → surface` (`dict((l[0], l[2]) for l in brut)`), le plus grand par `max(brut, key=lambda l: l[2])`.

Exercice 4.1. `magasins[2:4] → ['Bobo Centre', 'Bobo Sarfalao']` : les indexes 2 et 3 (fin exclue, §8 erreur n° 6).

Exercice 4.2. Le compteur de §5.4 sur les libellés :

```
{'Espèces': 3, 'Mobile Money': 2, 'Carte Bancaire': 1}
```

(six paiements, 3 libellés — le `GROUP BY mode_paiement` de M07, sur 6 lignes au lieu de 50 008.)

Exercice 4.3. `len(set(villes)) → 3` : 6 occurrences, 3 villes distinctes — le `COUNT(DISTINCT ville)` en une ligne.

Exercice 4.4. `sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2] → [(2, 150000), (4, 120000)]` : le tri porte sur `p[1]` (la valeur), `[:2]` coupe — le `ORDER BY ... LIMIT 2`.

Exercice 4.5. `float(s.strip().replace(" ", "").replace(",", ".")) → 15 000,50` converti ; affiché : `15,000.50` (la convention par défaut de Python, §5.4 de C02 — le format français s'écrit à la main si le rapport l'exige).

13. Mini-projet M08.P4 — « L'annuaire des magasins » (1 h)

Énoncé. Écrire `annuaire.py` — le §6, durci en **rapport** :

- la liste brute `brut` (5 tuples) est en tête, et les contenants sont construits **une fois** (liste de noms, tuple de surfaces, dictionnaire `par_ville`, ensemble de villes, **et** un dictionnaire `nom → surface` — 5 au total, le dernier servant la recherche inverse) ;
- le rapport **par ville triée** : pour chaque ville, la liste de ses magasins et la **somme de leurs surfaces** ;
- la ligne « villes distinctes : n » ;
- la ligne « plus grand magasin : nom (surface m2) » ;
- la recherche **inverse** : « magasin de 450 m2 : nom » (du chiffre au nom — le dictionnaire qui sert dans les deux sens).

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier : `Bobo-Dioulasso : Bobo Centre, Bobo Sarfalao — 810 m2, Koudougou : Koudougou — 210 m2, Ouagadougou : Ouaga Centre, Ouaga Patte d'Oie — 830 m2, villes distinctes 3, plus grand magasin Bobo Centre (520 m2), magasin de 450 m2 Ouaga Centre (1 pt) · aucun d[clé] nu sur par_ville (1 pt) · la somme de surfaces par ville calculée, pas resaisie (1 pt) · docstring en tête (1 pt).`

Note de correction. Les sommes ($810 = 520 + 290$; $830 = 450 + 380$) sont **mesurées** dans l'atelier — l'apprenant qui les annonce sans exécuter trahit le principe du module (prédire, exécuter, comparer). La ville s'affiche **triée alphabétiquement** (`sorted(par_ville)`) : Bobo-Dioulasso, Koudougou, Ouagadougou — le tri des clés est le `ORDER BY ville` du rapport.

14. Boîte à outils du chapitre

Boîte à outils. Python nu + terminal toujours ; la boîte du chapitre s'ajoute à celles de C02-C03 : les 5 contenants et leurs opérateurs signature.

Contenant	Question métier	Opérateur signature
liste	« le n° ? » — l'ordre	<code>l[i]</code> , <code>l[1:3]</code> , <code>append</code>
tuple	« le fait figé » — la clé	<code>n, p = t</code> , <code>t</code> en clé de dict
str	« le texte propre »	<code>strip</code> , <code>split</code> , <code>replace</code>
dict	« le comptage par... »	<code>d.get(k, 0)</code> , <code>k in d</code>
set	« combien de distincts ? »	<code>len(set(x))</code> , <code>c1 & c2</code>
tri	« le top N »	<code>sorted(d.items(), key=...)[:n]</code>

15. Résumé du chapitre

- La **liste** est la case numérotée (mutable, doublons) ; `l[-1]` est le dernier, `l[1:3]` contient 2 éléments (fin exclue), `l[5]` sur 5 éléments lève `IndexError` — la ligne `n` a l'index `n - 1`.
- Le **tuple** est l'enveloppe scellée : `TypeError` si on la répare, `n, p = t` pour l'ouvrir, et seule l'immuable a le droit d'être **clé** de dictionnaire.
- La **chaîne** est la séquence de caractères : `strip/split/replace` nettoient le format français, la f-string l'affiche (`158,140 FCFA`).
- Le **dictionnaire** est l'étagère étiquetée : `d.get(clé, défaut)` ne s'effondre jamais, `clé in d` teste les **clés**, et le compteur `d.get(k, 0) + 1` **est** le `GROUP BY`.
- L'**ensemble** est le tamis : `len(set(villes)) = COUNT(DISTINCT ville)`, `c1 & c2` = le `JOIN` interne.
- `sorted(...)` renvoie, `.sort(...)` modifie ; le top N est `sorted(d.items(), key=lambda p: p[1], reverse=True)[:n]` — le `ORDER BY ... LIMIT n`.

16. À retenir

À retenir. Avant de taper un contenant, la question : « **quelle est la question métier ?** » — l'ordre (liste), le fait figé (tuple), l'étiquette (dictionnaire), l'unicité (ensemble). Le bon contenant rend la bonne question **immédiate** et la mauvaise difficile ; l'inverse, c'est du débogage.

À retenir. `d[clé]` lève, `d.get(clé, défaut)` renvoie : sur les données qu'on ne possède pas (fichiers, saisies), seul le second a le droit d'exister — le `KeyError` d'un cas prévu est le script qui s'effondre de politesse.

17. Évaluation formative (auto-correction, 8 min)

1. `magasins[2:4]` contient combien d'éléments, et pourquoi ? → 2 (les indexes 2 et 3) : la borne de fin est **exclue** — la convention des tranches Python. (1 pt)
2. `magasins[5]` sur une liste de 5 éléments fait quoi, et quelle est la bonne demande ? → `IndexError: list index out of range` ; la 5^e ligne est `magasins[4]` (ou `magasins[-1]`). (1 pt)
3. Pourquoi le tuple est-il bon pour clé de dictionnaire et pas la liste ? → le tuple est **im-muable**, donc hachable : `dict` accepte une clé tuple ; la liste est mutable — `dict` la refuse (`TypeError: unhashable type: 'list'`). (2 pts)
4. `compteur[idm] = compteur.get(idm, 0) + 1` — que fait la `.get(..., 0)` exactement ? → prend le comptage de la clé **ou 0 si la clé est absente** : pas de `KeyError` au premier passage — c'est ce qui fait le `GROUP BY`. (2 pts)
5. `clé in d` teste quoi — les clés ou les valeurs ? → les **clés** ; chercher dans les valeurs demande un parcours (`any(v == x for v in d.values())`). (1 pt)
6. `len(set(villes))` est l'équivalent SQL de quoi ? → `COUNT(DISTINCT ville)` — le tamis compte les uniques. (1 pt)
7. `sorted(montants)` et `montants.sort()` — la différence en une phrase ? → `sorted` renvoie une nouvelle liste (l'originale intacte) ; `.sort` trie **en place** (renvoie `None`). (1 pt)
8. Le top 2 d'un dictionnaire ca par valeur, en une ligne ? → `sorted(ca.items(), key=lambda p: p[1], reverse=True)[:2]` — le `ORDER BY ... LIMIT 2`. (1 pt)

Module M08.C05 — Organiser : fonctions, arguments, portée, docstring, modules, `try/except`, fichiers

Outils : Python 3.13.14 seul + la bibliothèque standard (`csv`, `json`), le système de fichiers. **Aucun paquet tiers.** **Durée indicative :** 4 h. **Niveau :** N5 → N6. **Prérequis :** C04 (les conteneurs).

L'idée du chapitre. C04 a donné le **rangement** ; C05 donne l'**organisation** : découper le script en **fonctions** testables (le calculateur de remise de C02 devient enfin une fonction), savoir ce qui **entre** (arguments) et ce qui **sort** (`return` — pas `print`), ce qui est **visible** (la portée), ce qu'on **documente** (la docstring), ce qu'on **protège** (`try/except` — jamais d'`except` nu), et comment le travail **sort de l'écran** (les fichiers : CSV, JSON, l'encodage `utf-8-sig` et le BOM). Le fil rouge : le **nettoyeur de montants** — une fonction qui reçoit `"15 000,50"`, `"15000.50"` ou `"N/C"` et renvoie un `float` ou `None`, 6 cas de test exécutés. C07 le retrouvera devant 50 008 lignes.

Matériel de l'atelier — Python 3.13.14, bibliothèque standard, dossier du socle. Tout le code de ce chapitre a été exécuté dans l'atelier le 19/09/2026 ; les sorties publiées sont **celles de l'atelier**. Les fichiers cités sont réels : `03_exercices/dossier_M08/vente.csv` (13 colonnes, 50 008 lignes — clé `m08p_ventes_lignes`) et `03_exercices/dossier_M08/ATTENDU.json` (30 clés, la fiche de contrôle du socle). Les montants : 1 500, 1 200, 15 000,50, 158 140 (clés du socle).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Écrire une **fonction** avec `def` et `return` — et expliquer pourquoi `print` n'est pas une sortie (le piège `None`, exécuté).
2. Passer des **arguments** : positionnels, nommés (`calculer_remise(montant=1500, taux=0.05)`), avec **valeurs par défaut** — et éviter le piège de la valeur par défaut **mutable** (exécuté : `2 2` au lieu de `1 1`).
3. Dire ce qui est **visible** : portée locale vs globale, la variable de boucle qui « fuit » (elle existe après la boucle), `global` (à éviter).
4. Écrire une **docstring** à la convention (Quoi / Entrées / Sortie / Exemple) et la vérifier avec `help(f)`.
5. Placer les **imports** en tête de fichier, et distinguer `import`, `from ... import`.
6. Intercepter l'**erreur attendue** avec `try/except` **spécifié** (jamais d'`except` : nu — le bug silencieux, exécuté) — et lire une **traceback en 3 couches**.
7. Lire et écrire des **fichiers** : `open` avec `with`, l'encodage `utf-8-sig` (le BOM), le module `csv` sur le vrai `vente.csv` du socle, `json.load/json.dump` sur le vrai `ATTENDU.json`.

2. Pourquoi cette notion est importante

Parce que le passage de C04 à C05 est le passage du **brouillon** au **livrable**. Un script de 40 lignes dans le même fichier est un brouillon ; un script découpé en fonctions nommées, documentées, testées — c'est un livrable qu'un collègue peut **lire** et **reprendre**. La fonction est la brique de cette transition : elle a un nom (donc une raison d'exister), des entrées (donc un contrat), une sortie (donc un résultat **vérifiable** sans affichage).

Et parce que la machine commence à **travailler sur du réel** : les fichiers du dossier M08 (8 CSV + ATTENDU.json) attendent C07, mais leurs **questions** arrivent ici : que faut-il faire pour qu'un CSV Excel lisible (BOM, format français) devienne des nombres ? Pourquoi `open("vente.csv")` échoue « alors que le fichier existe » (le chemin relatif, §8) ? Que vaut `json.load` sur la fiche du socle ? C05 installe les **habitudes** ; C07 les passera à l'échelle.

Enfin, `try/except` est la différence entre un script qui **gère** le cas prévu (le montant « N/C » est un **fait de métier**, pas un accident) et un script qui **s'effondre** sur lui — et `l'except` nu est le piège qui transforme ce dernier en **mensonge** : il « marche » et renvoie des résultats faux sans un mot (le §8 l'exécute).

3. Explication simple — la machine à café

- La **fonction** est une **machine à café** : on met ce qu'elle attend (argent, choix), on appuie, elle donne **une chose** (le café) — et rien d'autre. `return` est la chose qu'elle donne ; `print` est le bruit qu'elle fait **en plus** (bruyant, et ce n'est pas le café).
- L'**argument par défaut** est le **choix préimprimé** sur la machine : si on ne précise rien, la machine prend le choix d'usine (`taux=0.10` — la remise de septembre, C03).
- La **portée** est la **cuisine** : ce qui est préparé dedans (les variables locales) ne sort pas — sauf si on l'a mis sur le comptoir (`return`).
- La **docstring** est la **notice** collée sur la machine : quoi, entrées, sortie, exemple — `help(f)` la lit à voix haute.
- Le **try/except** est le **filet sous le gobelet qui fuit** : on l'a mis **là où** la fuite est attendue (le montant illisible), pas sur toute la cuisine (`l'except` nu).
- Le **fichier** est le **tiroir** : le travail y entre (`read`), y sort (`write`) — et l'encodage, c'est l'**étiquette de langue** du tiroir (`utf-8-sig`, le BOM).

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
fonction — function	un bloc nommé, réutilisable, qui renvoie une valeur avec <code>return</code> .	faire <code>print</code> et croire que c'est la sortie : la sortie est le <code>return</code> , le <code>print</code> est un bruit.
argument défaut — par default argument	la valeur prise quand l'appel ne précise rien (<code>taux=0.10</code>).	la faire mutable (<code>panier=[]</code>) : elle est partagée entre tous les appels qui ne la précisent pas.
portée — scope	la zone de visibilité d'une variable : locale (dans la fonction) ou globale (partout).	croire qu'une variable locale existe dehors : elle meurt avec la fonction (le <code>NameError</code> du §8).
docstring docstring	— le commentaire en tête de fonction, entre triples guillemets, lu par <code>help(f)</code> .	la faire répéter le code : elle doit dire le contrat (entrées, sortie, exemple), pas le listing.
exception exception	— une erreur levée à un endroit, qui peut être interceptée plus haut (<code>except</code>).	l'intercepter sans la nommer (<code>except:</code>) : on avale aussi le bug qu'on cherchait.
BOM — byte order mark	les 3 octets invisibles en tête d'un fichier <code>utf-8-sig</code> , qui corrompent la 1 ^{re} colonne si on lit en <code>utf-8</code> pur.	lire un fichier Excel en <code>utf-8</code> : la première clé devient <code>'\ufeffid_...'</code> — une clé qui n'existe pas.

5. Cours approfondi

5.1 La fonction — le `return` est la sortie, pas le `print`

Le calculateur de remise de C02/C03 devient enfin une fonction — le fil rouge du module :

```
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)

print(calculer_remise(1500))
```

```
1350.0
```

Définition. fonction — fonction — un bloc nommé de code, appelé par son nom avec des arguments, qui **renvoie une valeur** avec `return`. Sans `return`, une fonction renvoie `None` — même si elle a **affiché** quelque chose : le `print` est un bruit, pas une sortie.

Le piège, exécuté dans l’atelier (la fonction affiche le bon nombre... et renvoie `None`) :

```
def affiche_remise(montant):
    print(montant * 0.9)

r = affiche_remise(1500)
print(r)
```

```
1350.0
None
```

La 2^e ligne est la leçon : `r` a reçu `None`, pas 1350.0. La fonction a **affiché** la valeur au lieu de la **rendre** — le test (et le collègue) n’a plus qu’un `None` en main.

Conseil professionnel. Une fonction fait **une chose** et son nom dit quoi, au **verbe** : `calculer_remise`, `nettoyer_montant`, `charger_fichier` — pas `fonction1`, pas `trucs`. Et la sortie est **toujours** un `return` : le `print` reste pour l’humain qui regarde, le `return` est pour la machine qui enchaîne.

5.2 Les arguments — positionnels, nommés, par défaut

Trois façons d’appeler, exécutées (la remise passe de 10 % à 5 %) :

```
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)

print(calculer_remise(1500))
print(calculer_remise(1500, 0.05))
print(calculer_remise(montant=1500, taux=0.05))
```

```
1350.0
1425.0
1425.0
```

- **positionnel** : `calculer_remise(1500, 0.05)` — l’ordre de la signature ;
- **nommé** : `calculer_remise(montant=1500, taux=0.05)` — l’ordre est libéré, la lecture est gagnée (c’est l’AS du SELECT de M06, appliqué à l’appel) ;
- **par défaut** : `taux=0.10` — le taux non précisé prend la valeur d’usine.

Et le piège du **par défaut mutable**, exécuté dans les deux sens (le bug, puis la correction) :

```
def ajouter(vente, panier=[]):
    panier.append(vente)
    return panier

a = ajouter(1500)
b = ajouter(2000)
print(len(a), len(b))
```

```
2 2
```

Attention. La valeur par défaut **mutable** est construite **une fois**, au moment de la `def` — et **partagée** entre tous les appels qui ne la précisent pas : le 2^e appel a vu le 1^e (le panier a 2 éléments, pas 1). C’est le bug silencieux le plus célèbre de Python : il ne lève rien, il **souille**.

La correction — `None` comme défaut, la liste construite **à chaque appel** :

```
def ajouter(vente, panier=None):
    if panier is None:
        panier = []
    panier.append(vente)
    return panier

a = ajouter(1500)
b = ajouter(2000)
print(len(a), len(b))
```

```
1 1
```

Définition. argument par défaut — default argument — la valeur prise quand l’appel ne précise pas l’argument : `calculer_remise(1500)` prend `taux=0.10`. Règle du chapitre : **jamais** de mutable en défaut (`[]`, `{}`) — `None`, puis reconstruction à l’intérieur.

5.3 La portée — la cuisine et le comptoir

Deux phénomènes exécutés dans l’atelier. D’abord, la **fuite** : la variable de boucle **survit** à la boucle (elle est créée dans la portée du script, pas dans une fonction) :

```
for m in [1500, 2000]:
    pass
print(m)
```

```
2000
```

Ensuite, l’invisible : une variable **locale** (définie dans une fonction) meurt avec elle — demandez-la dehors, la machine se proteste :

```
print(x)
^
NameError: name 'x' is not defined
```

Définition. portée — scope — la zone de visibilité d’une variable : **locale** (née et morte dans la fonction), **globale** (le fichier tout entier). Une fonction **lit** les globales, ne les **modifie** que par `global` — et `global` est à éviter : la fonction qui écrit une variable globale est une fonction qui ne se teste pas seule (elle dépend de ce qu’il y a **dehors**).

Le compteur global, exécuté (pour savoir à quoi ressemble la tentation) :

```
compteur = 0

def vendre():
    global compteur # l'écriture dans le fichier
    compteur = compteur + 1

vendre()
vendre()
print(compteur)
```

2

Ça marche — et ça **couple** : `vendre()` ne se teste pas sans réinitialiser `compteur`. La version saine rend le compteur : `def vendre(compteur): return compteur + 1` — l'entrée et la sortie, rien de caché.

5.4 La docstring — la notice sur la machine

La convention du module (la règle M04, reprise en Python) : **Quoi / Entrées / Sortie / Exemple** — et `help(f)` la lit :

```
def calculer_remise(montant, taux=0.10):
    """Prix net apres remise.

    Entrees : montant (float), taux (float, par default 0.10).
    Sortie : montant * (1 - taux).
    Exemple : calculer_remise(1500) -> 1350.0
    """
    return montant * (1 - taux)

help(calculer_remise)
```

```
Help on function calculer_remise in module __main__:

calculer_remise(montant, taux=0.1)
    Prix net apres remise.

    Entrees : montant (float), taux (float, par default 0.10).
    Sortie : montant * (1 - taux).
    Exemple : calculer_remise(1500) -> 1350.0
```

Définition. docstring — docstring — la chaîne en **tête** de fonction, entre triples guillemets, lue par `help(f)` et récupérable par `f.__doc__`. Elle écrit le **contrat** (entrées, sortie, exemple) — pas le listing : le code dit déjà **comment**, la docstring dit **quoi** on a le droit d'en attendre.

Deux remarques sur la sortie : `help` a rendu **taux=0.1** (le défaut, affiché en nombre, plus `0.10`) — la docstring reste la source du **sens** ; et la docstring est la **première ligne de code** de la fonction : tout ce qui est avant elle ne compte pas.

5.5 Modules et paquets — l'import en tête

La règle M04, reprise : **les imports vivent en tête de fichier**, avant tout le reste. Le script du §13 (et tous les scripts de C05) commence par :


```
import csv
import json
```

- `import csv` : on appelle ensuite `csv.reader`, `csv.writer` — le **point** fait la distinction (c'est le `.` du `module.fonction`) ;
- `from numpy import where` (C06) : on importe **une pièce** nommément, sans le point ;
- un **paquet tiers** (pandas en C07, `pip install` en C01) suit la même règle : `import pandas as pd` en tête, **une seule fois**, et le reste du fichier n'écrit plus que du travail.

Ce qu'on met dans **son propre module** (la question de la fin du chapitre) : les fonctions qui se **réutilisent** (`calculer_remise`, `nettoyer_montant`, `fmt`), pas les brouillons d'exécution. Un module se reconnaît à sa docstring de tête et à ses imports en haut — c'est un **fichier qui est une bibliothèque**.

Dans les faits. Le dossier M08 du socle (8 CSV + `ATTENDU.json`, dont 50 008 lignes de ventes) s'ouvre ici tout en `utf-8-sig` : le vrai `vente.csv` n'a **pas** de BOM (sa 1^{re} colonne est lue propre, `id_vente`), mais la règle est la même pour un fichier venu d'Excel — `utf-8-sig` passe dans les deux cas, `utf-8` pur ment dans un. En C07, `pandas.read_csv` reprendra exactement ce paramètre.

5.6 Le `try/except` — le filet sous la fuite prévue

Le cas prévu, ici, est un **fait de métier** : la caisse envoie « N/C ». On intercepte **l'erreur nommée**, au **endroit** où elle est attendue :

```
def nettoyer_montant(s):
    """Convertir un montant saisi en float, ou renvoyer None s'il est illisible."""
    try:
        return float(str(s).strip().replace(" ", "").replace(",", "."))
    except ValueError:
        return None

print(nettoyer_montant("15 000,50"))
print(nettoyer_montant("N/C"))
```

```
15000.5
None
```

Définition. exception — exception — une erreur **levée** à l'endroit où le problème arrive, qui remonte la pile jusqu'au premier `except` qui la **nomme**. `except ValueError` intercepte **ce type-là** (et ce type-là seul) ; le `try` marque la zone où l'erreur est **prévue**, pas tolérée partout.

Attention. Jamais d'except: nu — c'est le filet sur **toute** la cuisine : il avale le bug qu'on cherchait **et** le cas prévu, et le script « marche » en renvoyant des résultats faux. Le §8 l'exécute : le `except nu` transforme le format français en `None` **sans un mot**.

`else` et `finally`, en un paragraphe pour la suite : `else` s'exécute **si rien n'a été levé** dans le `try` (le « succès »), `finally` s'exécute **toujours** (fermer une connexion, nettoyer) — les deux arrivent en C07 avec les connexions de bases de données.

5.7 Lire une traceback — les 3 couches

La traceback de C02 était **plate** (un seul cadre). Avec les fonctions, elle a une **hauteur** — la **pile d'appels** : chaque `File "...", line n, in nom` est un **cadre** (un étage du chemin parcouru), du

haut (le point d'entrée) vers le bas (l'endroit exact). Trois tracebacks réelles, exécutées dans l'atelier le 19/09/2026 :

1. `NameError` — la variable qui n'existe pas, vue de la pile :

```
Traceback (most recent call last):
  File "<string>", line 4, in <module>
    print(calculer_net(1500))
    ~~~~~^~~~~~
  File "<string>", line 3, in calculer_net
    return montant * (1 - remise)
    ~~~~~^~~~~~
NameError: name 'remise' is not defined
```

2. `KeyError` — la clé absente (C04), vue de la pile :

```
Traceback (most recent call last):
  File "<string>", line 5, in <module>
    print(surface("Sintani"))
    ~~~~~^~~~~~
  File "<string>", line 4, in surface
    return surfaces[magasin]
    ~~~~~^~~~~~
KeyError: 'Sintani'
```

3. `FileNotFoundError` — le fichier n'est pas ici (le chemin relatif) :

```
File "<string>", line 1, in <module>
  f = open('introuvable.csv')
FileNotFoundError: [Errno 2] No such file or directory: 'introuvable.csv'
```

La discipline, en trois temps :

1. **En bas, le message** : le **type** d'erreur d'abord (`NameError`, `KeyError`, `FileNotFoundError`) — c'est la famille, on la nomme avant de chercher ;
2. **Le cadre du bas** : la **ligne fautive** — ici, `remise` dans `calculer_net` (cadre 2), pas l'appel (cadre 1) : **l'erreur vit dans la fonction, l'appel est juste le témoin** ;
3. **La pile, en remontant** : le chemin parcouru (`<module>` → `calculer_net`) — utile quand l'erreur vient d'une fonction **appelée par une fonction** (C07 : `main` → `charger` → `nettoyer`).

Les 3 couches, sur la protestation réelle du §5.7 :

Lire une traceback : les 3 couches

Une erreur avec des fonctions a une hauteur : la pile d'appels. La lire, c'est le travail.

La protestation réelle (exécutée dans l'atelier) :

Traceback (most recent call last):

File "<string>", line 4, in <module>

print(calculer_net(1500))

File "<string>", line 3, in calculer_net

return montant * (1 - remise)

^^^^^

NameError: name 'remise' is not defined

1 • La pile d'appels

le chemin parcouru, du haut vers le bas
(le module appelle calculer_net)

2 • La ligne fautive + la fleche

l'erreur est DANS la fonction ;
l'appel (cadre du haut) est le témoin

3 • Le message, tout en bas

le nom de la famille d'abord :
NameError, pas la phrase

La discipline : lire de haut en bas pour le POURQUOI, s'arrêter en bas pour le QUOI.

Les 5 familles du debutant : SyntaxError (texte) · NameError (nom) · TypeError (famille)

· ValueError (valeur) · KeyError / FileNotFoundError (cle / fichier)

6. Exemple concret — le nettoyeur de montants

Le fil rouge du chapitre, version complète, exécutée dans l'atelier le 19/09/2026 — la fonction et les **6 cas de test** (chaque ligne est une prédiction vérifiée) :

```
def nettoyer_montant(s):
    """Convertir un montant saisi en float, ou renvoyer None s'il est illisible.

    Entree : s (str, ou numero deja converti).
    Sortie : float (ex. 15000.5) ou None (texte, vide).
    Exemple : nettoyer_montant("15 000,50") -> 15000.5
    """
    try:
        return float(str(s).strip().replace(" ", "").replace(",", "."))
    except ValueError:
        return None

tests = [
    ("15 000,50", 15000.5),
    ("15000.50", 15000.5),
    ("N/C", None),
    (" 1 200 ", 1200.0),
    ("", None),
    (1500, 1500.0),
]
for saisie, attendu in tests:
    resultat = nettoyer_montant(saisie)
    ok = "OK " if resultat == attendu else "ECART"
    print(f"{ok} {saisie!r:16} -> {resultat}")
```

```
OK '15 000,50'      -> 15000.5
OK '15000.50'      -> 15000.5
OK 'N/C'           -> None
OK ' 1 200 '       -> 1200.0
OK ''              -> None
OK 1500            -> 1500.0
```

Les 6 cas couvrent tout ce que la caisse envoie : le **format français** (1 500... 15 000,50 — la clé `m08p_demo_montant_fcfa` du socle), le **format américain** (15000.50 — les exports Excel), le **refus** (« N/C », un **fait de métier** intercepté par `except ValueError`), les **espaces parasites** (`strip`), le **vide** (le `float("")` lève aussi `ValueError` — le même filet), et le **nombre déjà converti** (`str(1500)` passe par la même porte). Zéro ECART : la fonction est **testée**, pas supposée — c'est le principe d'évaluation du module (prédire, exécuter, comparer) appliqué au code.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026).

Étape 1 — La fonction fait son travail (le fil rouge de C02, enfin fonction) :

```
$ python3 -c "
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)
print(calculer_remise(1500))"
1350.0
```

Interprétation : 1 500 – 10 % = 1 350 — la valeur de référence du module, rendue par `return`, pas affichée par accident.

Étape 2 — L'argument nommé libère la lecture (le taux précisés sans mémoriser l'ordre) :

```
$ python3 -c "
def calculer_remise(montant, taux=0.10):
    return montant * (1 - taux)
print(calculer_remise(montant=1500, taux=0.05))"
1425.0
```

Interprétation : 5 % au lieu de 10 — l'argument nommé `taux=0.05` dit ce qu'il modifie, l'appel se lit comme une phrase.

Étape 3 — La notice parle (`help` lit la docstring) :

```
$ python3 -c "
def calculer_remise(montant, taux=0.10):
    '''Prix net apres remise.
    Entrees : montant (float), taux (float, par default 0.10).
    Sortie : montant * (1 - taux).
    Exemple : calculer_remise(1500) -> 1350.0
    '''
    return montant * (1 - taux)
help(calculer_remise)"
Help on function calculer_remise in module __main__:

calculer_remise(montant, taux=0.1)
    Prix net apres remise.
    Entrees : montant (float), taux (float, par default 0.10).
    Sortie : montant * (1 - taux).
    Exemple : calculer_remise(1500) -> 1350.0
```

Interprétation : le contrat, lisible sans le code — `taux=0.1` est le défaut tel que Python l'affiche (0.10 en nombre) ; le sens reste dans la docstring.

Étape 4 — Le nettoyeur ouvre la porte (le format français en nombre) :

```
$ python3 -c "
def nettoyer_montant(s):
    try:
        return float(str(s).strip().replace(' ', '').replace(',', '.'))
    except ValueError:
        return None
print(nettoyer_montant('15 000,50'))"
15000.5
```

Interprétation : la recette de C02 (`strip`, `replace`) dans une fonction, le filet `except ValueError` sous le cas prévu — 15 000,50 est devenu 15 000,50 en nombre.

Étape 5 — Le BOM, la 1^{re} colonne qui ment (l'encodage) :

```
$ python3 -c "
open('/tmp/avec_bom.csv', 'w', encoding='utf-8-sig').write('nom;valeur\nx;l\n')
print(repr(open('/tmp/avec_bom.csv', encoding='utf-8').readline().strip()))
print(repr(open('/tmp/avec_bom.csv', encoding='utf-8-sig').readline().strip()))"
'\u0000nom;valeur'
'nom;valeur'
```

Interprétation : le même fichier lu deux fois — en `utf-8`, la première clé est `'\u0000nom'` (le BOM invisible collé devant), en `utf-8-sig` elle est propre : c'est pourquoi le module ouvre toujours en `utf-8-sig` (la règle M01).

Étape 6 — Le vrai CSV du socle (le `csv` standard, 13 colonnes) :

```
$ python3 -c "
import csv
with open('03_exercices/dossier_M08/vente.csv', encoding='utf-8-sig') as f:
    lecteur = csv.reader(f)
    en_tete = next(lecteur)
    print(len(en_tete), 'colonnes')
    for i, ligne in enumerate(lecteur):
        if i >= 2:
            break
        print(ligne[:4])"
13 colonnes
['1', '192', '380', '2']
['2', '318', '311', '1']
```

Interprétation : le vrai `vente.csv` du socle (clé `m08p_ventes_colonnes = 13`) lu par la bibliothèque standard — C07 fera la même lecture avec `pandas.read_csv`, et la première ligne lue est la ligne de tête, pas une vente.

8. Erreurs fréquentes

1. **print au lieu de return — la fonction qui « travaille » et renvoie None.** Exécuté dans l'atelier : `text 1350.0 None` La fonction a affiché le bon nombre et rendu `None` — l'appelant n'a plus rien. Remède : la sortie est **toujours** un `return` ; le `print` est un bruit de chantier, on le retire avant la livraison.
2. **L'argument par défaut mutable — le panier qui grandit tout seul.** Exécuté : `def ajouter(vente, panier=[])` donne 2 2 après deux appels au lieu de 1 1 (\$5.2). Remède : `panier=None`, puis `if panier is None: panier = []` — la mutable se **construit** à chaque appel, elle ne se **partage** pas.
3. **Le return oublié dans une branche — la fonction qui rend None « seulement » parfois.** Exécuté : `text 2 None` (`f(1)` rend 2, `f(-1)` rend `None` — même fonction, deux sorties.) Remède : **chaque branche** se termine par un `return` (ou une `return None` explicite qui dit le silence) — une fonction qui a une sortie « possible » doit la dire.

4. **La variable locale demandée dehors — `NameError` de portée.** Exécuté : `def f(): x = 5` puis `f()` puis `print(x)` → `NameError: name 'x' is not defined` (§5.3). Remède : la valeur doit **passer par le comptoir** — `x = f()` (le `return`), pas `print(x)` de l'autre côté du mur.
5. **`FileNotFoundError` — « le fichier existe, mais pas ici ».** Exécuté : `open('introuvable.csv')` → `FileNotFoundError: [Errno 2] No such file or directory: 'introuvable.csv'` (§5.7). Le piège est le **chemin relatif** : il se résout depuis le **dossier courant** du terminal, pas depuis le dossier du script — le fichier existe, il est **ailleurs**. Remède : chemin absolu dans les scripts de livraison, ou chemin relatif **documenté** (« lancer depuis la racine du dossier »).
6. **L'except: nu — le bug qui se tait.** Exécuté dans l'atelier : `python def nettoyer_tout(s): try: return float(s) except: return None print(nettoyer_tout('15 000,50')) print(nettoyer_tout('1500.50')) print(nettoyer_tout('N/C'))` text None 1500.5 None Lisez la sortie en trois temps : le **format français** (1^{re} ligne) est avalé par le filet universel et renvoyé `None` **sans un mot** ; le **format correct** (2^e ligne) passe — la fonction n'est pas morte, elle est **aveugle au cas qu'elle ne connaît pas** ; le **refus** « N/C » (3^e ligne) est le seul cas prévu. Le script « marche », les résultats sont faux, personne n'est prévenu. Remède : `except ValueError` **nommé** (§5.6) — et la recette avant le `float`, pas le filet pour rattraper la recette oubliée.

9. Bonnes pratiques professionnelles

1. **Le nom de fonction est un verbe** (`calculer_remise`, `nettoyer_montant`, `charger_fichier`) — et fait **une** chose : si le nom a un « et », c'est deux fonctions.
2. **Le `return` est unique par intention** : chaque branche se termine par une sortie ; une fonction qui « peut » ne pas rendre de valeur doit le dire dans la docstring.
3. **`None` en défaut, pas `[]`** : la mutable se construit à l'intérieur (§5.2) — la règle est mécanique, elle ne se négocie pas.
4. **L'exception est nommée et proche**** : `except ValueError` au niveau de la ligne qui peut lever, pas au niveau du `main` — le filet est sous la gouttière, pas sous le toit.
5. **Les imports en tête, le travail en bas** : le fichier se lit comme un contrat (`imports` → constantes → fonctions → exécution) ; un `import` au milieu du script est un signal d'alerte (code re-collé, pas écrit).
6. **`open` n'existe que sous `with`** : le fichier se **ferme** de lui-même à la fin du bloc — pas de `f = open(...)` nu, pas de `f.close()` à mémoriser.

10. Exercice guidé — « le nettoyeur de montants » (15 min, /10)

Énoncé. Écrire `nettoyeur.py` :

1. la fonction `nettoyer_montant(s)` avec **docstring** à la convention (Quoi / Entrée / Sortie / Exemple) ;
2. la recette du format français (`strip`, `replace` × 2) **avant** le `float` ;
3. le filet `try/except ValueError` qui renvoie `None` — et **un seul** type intercepté ;
4. les 6 cas de test de §6, affichés chacun avec `OK/ECART`.

Barème. La docstring à la convention (2 pts) · la recette complète avant le `float` (2 pts) · le `try/except ValueError` **spécifié** (2 pts) · les 6 cas couverts dont « N/C », le vide et un nombre déjà converti (2 pts) · la sortie alignée sur le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
OK '15 000,50'      -> 15000.5
OK '15000.50'       -> 15000.5
OK 'N/C'            -> None
OK ' 1 200 '        -> 1200.0
OK ''               -> None
OK 1500             -> 1500.0
```

11. Exercices autonomes

Exercice 5.1 — Le double. Écrire `doubler(v)` avec `return` (pas `print`), tester sur 500 et 1500.5 : afficher les deux sorties. (5 min)

Exercice 5.2 — La facture. Écrire `facture(montant, tva=0.19, escompte=0.0)` qui rend `montant × (1 + tva) × (1 - escompte)` — et l'appeler **3 fois** : sans option, avec la tva en position, avec l'escompte **nommé** (montant 1 000). (10 min)

Exercice 5.3 — Les deux paniers. Reproduire le bug du §5.2 (deux appels, le deuxième voit le premier) et le corriger avec `panier=None` : afficher les deux sorties (bug, correction). (10 min)

Exercice 5.4 — Le garde. Écrire `est_lisible(s)` qui renvoie `True/False` (pas d'exception qui s'échappe) en réutilisant la recette du nettoyeur. (10 min)

Exercice 5.5 — Le tiroir. Écrire un fichier `trois.csv` (3 lignes de données) avec `open` sous `with`, le relire, afficher le nombre de **lignes de données** (pas la tête). (10 min)

12. Correction détaillée

Exercice guidé. Le script de §6 **est** la correction (exécuté dans l'atelier) — les 5 points de vigilance : la docstring **avant** tout le code de la fonction (la première ligne), la recette **avant** le `float` (pas le `filet` pour rattraper la recette oubliée, §8 erreur n° 6), le `try` autour de la **ligne** qui peut lever, le `except ValueError` **nommé**, et le test qui affiche `OK/ECART` (la prédiction vérifiée, pas l'affichage passif).

Exercice 5.1. `def doubler(v): return v * 2 → 1000 3001.0`. Le `print` serait un bruit : la sortie est dans le `return`.

Exercice 5.2. Les 3 appels (19 % de TVA — la règle du socle 2026) :

```
1190.0
1100.0
1130.5
```

→ `facture(1000)` (le défaut), `facture(1000, 0.10)` (la tva en position), `facture(1000, escompte=0.05)` (l'escompte nommé) : l'argument nommé dit **ce** qu'il modifie.

Exercice 5.3. Le bug : `def ajouter(vente, panier=[]) → 2 2` (la liste est construite une fois, au `def`). La correction : `panier=None` puis reconstruction → `1 1`. Les deux sorties sont publiées telles qu'exécutées.

Exercice 5.4. `est_lisible` renvoie `True/False` — exécuté :

```
15 000,50 -> True
N/C -> False
(vide) -> False
```

Le `filet` est **dans** la fonction (aucune exception ne s'échappe) ; le `return` est dans les **deux** branches (§8, erreur n° 3).

Exercice 5.5. Écrire puis relire :


```
with open("trois.csv", "w", encoding="utf-8") as f:
    f.write("a,b\n1,2\n3,4\n5,6\n")
lignes = open("trois.csv", encoding="utf-8").readlines()
print(len(lignes) - 1)
```

3

Le `- 1` retire la **tête** — la ligne de colonnes n'est pas une ligne de données (la même question que l'Étape 6 de §7 : la 1^{re} ligne lue est la tête, pas une vente).

13. Mini-projet M08.P5 — « Le registre de caisse » (1 h)

Énoncé. Écrire `registre.py` — le fil rouge complet, du nombre au fichier :

- la fonction `nettoyer_montant` (celle de §6, docstring comprise) **et** la fonction `fmt(v)` qui forme un float au **format français** (500 → « 500,00 », 15000.5 → « 15 000,50 ») ;
- écrire `registre.csv` avec `csv.writer` (sous `with`, encoding `utf-8`) : la tête `n,total`, puis les 3 ventes du C03 (500, 1200, 15000.5) formatées en français ;
- relire `registre.csv` avec `csv.reader` (sous `with`, encoding `utf-8-sig` — la règle du chapitre, même pour un fichier qu'on vient d'écrire) ;
- pour chaque ligne : `nettoyer_montant` sur la saisie, affichage `vente n : saisie -> montant` ;
- le **total général** affiché en dernière ligne.

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier : `vente 1 : 500,00 -> 500.0`, `vente 2 : 1 200,00 -> 1200.0`, `vente 3 : 15 000,50 -> 15000.5`, `total : 16700.5` (1 pt) · deux fonctions nommées + docstring sur `nettoyer_montant` (1 pt) · le `csv` de la bibliothèque standard (pas de recopie manuelle du texte) (1 pt) · la lecture en `utf-8-sig` (1 pt).

Note de correction. Le total 16 700,50 (500 + 1 200 + 15 000,50) est **mesuré** dans l'atelier — et c'est le point du projet : le fichier contient du **texte** au format français, le script le convertit à **la porte** (C02), et le total s'additionne en **nombres**. La boucle est bouclée : C02 (la conversion), C03 (le flux), C05 (l'organisation) — le même 15 000,50 a fait trois voyages.

14. Boîte à outils du chapitre

Boîte à outils. Python nu + bibliothèque standard (`csv`, `json`) ; la boîte du chapitre s'ajoute à celles de C02-C04 : la fonction, le filet, le tiroir.

Outil	Usage	Syntaxe
<code>def / return</code>	la fonction et sa sortie	<code>def f(x): return x * 2</code>
argument nommé / défaut	la lecture et le contrat	<code>f(montant=1500, taux=0.05)</code>
<code>None</code> en défaut	le piège mutable évité	<code>def f(x, acc=None):</code>
docstring / <code>help</code>	la notice	<code>"""Quoi. Entrées. Sortie. Exemple."""</code>
<code>try/except</code>	le filet nommé	<code>try: ... except ValueError: ...</code>
<code>with open</code>	le tiroir qui se referme	<code>with open(f, encoding="utf-8-sig") as fh:</code>
<code>csv.reader/writer</code>	le CSV	<code>next(lecteur)</code> (la tête)
<code>json.load/dump</code>	le JSON	<code>json.load(fh) / json.dump(d, fh)</code>

15. Résumé du chapitre

- La **fonction** rend avec `return` — le `print` est un bruit, et la fonction sans `return` rend `None` (le piège exécuté : 1350.0 affiché, `None` rendu).
- Les arguments : positionnels, **nommés** (la lecture), **par défaut** — et jamais de mutable en défaut (`panier=None`, reconstruction ; le bug 2 2 exécuté).
- La **portée** : locale (meurt avec la fonction) vs globale (`global` à éviter) ; la variable de boucle **fuit** hors de la boucle (2000 exécuté).
- La **docstring** (Quoi / Entrées / Sortie / Exemple) est le **contrat** ; `help(f)` la lit — `taux=0.1` est le défaut affiché, le sens reste dans la docstring.
- Les **imports** en tête ; le `try/except` **nommé** et proche ; l'`except` nu est le bug qui se tait (15 000,50 → `None` sans un mot, exécuté).
- Les **fichiers** : `with open(..., encoding="utf-8-sig")` (le BOM corrompt la 1^{re} colonne en `utf-8` pur — exécuté), `csv` sur le vrai `vente.csv` (13 colonnes), `json.load` sur le vrai `ATTENDU.json` (30 clés).

16. À retenir

À retenir. La fonction a un **contrat** : le nom (quoi), la docstring (entrées/sortie/exemple), le `return` (la promesse tenue). Tout le reste — le `print` de chantier, le `global` caché, la mutable en défaut — est une brèche dans le contrat.

À retenir. L'exception est **nommée** (`except ValueError`), **proche** (sous la ligne qui lève) et **prévue** (un fait de métier, pas un accident) — l'`except` nu ne « gère » rien, il **met au silence** : le script marche, les résultats mentent, personne n'est prévenu.

17. Évaluation formative (auto-correction, 8 min)

1. `r = affiche_remise(1500)` où la fonction fait `print(montant * 0.9)` : que vaut `r`, et pour-quoi ? → `None` : la fonction n'a pas de `return` — le `print` est un bruit, pas une sortie. (1 pt)
2. `calculer_remise(montant=1500, taux=0.05)` — que fait l'argument nommé que ne fait pas l'argument positionnel ? → il **nomme** l'argument : l'ordre est libéré, la lecture est gagnée (on voit **ce** qu'on modifie). (1 pt)
3. Pourquoi `def f(x, acc=[])` est-il un piège, et quelle est la correction mécanique ? → la mutable est construite **une fois** (au `def`) et **partagée** entre les appels (le bug 2 2) ; correction : `acc=None`, puis reconstruction à l'intérieur (1 1). (2 pts)
4. `global` fait quoi, et pourquoi est-il à éviter ? → il autorise la fonction à **écrire** une variable globale : ça marche, mais la fonction ne se teste plus seule (elle dépend de l'extérieur) — la version saine rend le compteur par `return`. (1 pt)
5. À quoi sert la docstring, et qui la lit ? → elle écrit le **contrat** (Quoi / Entrées / Sortie / Exemple) ; `help(f)` la lit, et elle est la première ligne de la fonction. (1 pt)
6. `try/except ValueError` et `try/except:` — la différence, avec l'exemple du chapitre ? → le premier intercepte le **type nommé** (le montant illisible) ; le second avale **tout**, y compris le bug du format français non géré — 15 000,50 → `None` sans un mot. (2 pts)
7. Sur une **traceback** à 2 cadres, où est l'erreur et où est le témoin ? → l'erreur vit dans le **cadre du bas** (la fonction, ici `calculer_net`) ; le cadre du haut est l'**appel** (le témoin) ; le message, tout en bas, donne la famille (`NameError`). (1 pt)

8. **Pourquoi** `open(..., encoding="utf-8-sig")` **et jamais** `utf-8` **pur** ? → le BOM (3 octets invisibles, fichiers Excel) est **avalé** par `utf-8-sig` mais **collé** devant la 1^{re} clé en `utf-8` (`'\ufeff-nom'`) : une clé qui n'existe pas. (1 pt)

Module M08.C06 — NumPy utile et suffisant : tableaux, vectorisation, statistiques, `nan`, `where`, broadcasting

Outils : le venv de C01 (`/tmp/venv_m08`) avec `numpy 2.5.3` (`pandas 3.0.6` est installé, il jouera en C07). Le terminal. **Durée indicative :** 4 h. **Niveau :** N6 → N7. **Prérequis :** C05 (les fonctions, les fichiers).

L'idée du chapitre. C05 a organisé le code ; C06 accélère le calcul. La boucle de C03 (le contrôle qualité montant par montant) devient **une ligne** quand les montants vivent dans un **tableau NumPy** : homogène, contigu, traité **en bloc** (la vectorisation). Le fil rouge : les **50 008 montants réels** du socle (`vente.csv`, colonne `montant_ttc`) — CA total, panier moyen, 95^e percentile, montants « gros » : **4 calculs, 0 boucle** — et la mesure honnête : la boucle de C03 face à la ligne vectorisée, **les temps publiés** (mesurés dans l'atelier, pas annoncés). C07 passera le relais au `DataFrame` : une colonne pandas est une colonne, pas un tableau nu.

Matériel de l'atelier — venv C01, numpy 2.5.3. Tout le code de ce chapitre a été exécuté le 19/09/2026 dans `/tmp/venv_m08` ; les sorties publiées sont **celles de l'atelier**. La colonne mesurée est **réelle** : `montant_ttc` des 50 008 lignes de `03_exercices/dossier_M08/vente.csv` (clé `m08p_ventes_lignes`), extraite en une commande `cut` avant le `np.loadtxt`. Les repères cano- niques : CA brut 7 908 259 732 (clé `m08p_ca_brut`), panier moyen 158 140 (clé `m08p_pa- nier_moyen`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Construire un **tableau** NumPy (`np.array`), lire son `dtype` et son `shape` — et comprendre l'homogénéité **forcée** (le tableau qui convertit sans prévenir).
2. **Vecteuriser** : `montants * 1.19` multiplie **tout le tableau** ; `np.sum`, `np.mean` sur les 50 008 montants réels — le parallèle SUM/AVG de M06.
3. Extraire les **statistiques** : `np.min/np.max`, les **percentiles** (le quartile de M02 en NumPy), la médiane, l'écart-type.
4. Gérer `np.nan` : le nombre qui « n'a pas de valeur » — `nan + 1 = nan` (l'absence se propage), `np.nansum/np.nanmean` — le pont vers les NULL de M06.
5. Étiqueter un tableau avec `np.where` (le CASE WHEN de M06 en tableau) et compter avec `(condition).sum()`.
6. Comprendre le **broadcasting** : la règle de forme — et lire le `ValueError` de forme qui dit laquelle des deux tables ne s'aligne pas.
7. **Mesurer** : boucle `for` vs ligne vectorisée — les deux temps, le rapport, publiés.

2. Pourquoi cette notion est importante

Parce que la vectorisation, c'est **le SGBD pour vos tableaux**. En M06/M07, le moteur SQL faisait l'agrégation sur le lot (`SUM`, `AVG`, `GROUP BY`) sans que vous écriviez de boucle ; NumPy fait **exactement le même geste** sur une colonne en mémoire : une opération, tout le tableau. Le chapitre ne vous apprend pas un outil en plus, il vous rend le réflexe SQL dans la main Python : le total d'une colonne est **une ligne**, point.

Parce que **nan est le NULL de M06, habillé en nombre**. Vous savez déjà ce qu’est une valeur manquante (M06 : elle se propage dans les calculs, elle ne compte pas dans les `COUNT`). NumPy réinstalle la **même** sémantique : `nan + 1 = nan`, un tableau qui contient un `nan` donne un `nan` à `np.sum` — et `np.nansum` est le `COUNT/SUM` qui sait regarder autour du trou. Le pont est direct : ce que vous avez appris sur les `NULL` en SQL, c’est déjà 80 % de `nan`.

Parce que la **mesure** fait partie du métier : « la boucle est lente » est une affirmation ; « la boucle prend 6,2 ms, la ligne vectorisée 0,193 ms, rapport 32× — mesuré le 19/09/2026 sur cette machine » est une **preuve** (et un résultat qui dépend de la machine, qu’on publie comme tel).

3. Explication simple — le convoi, la barrière, le camion vide

- Le **tableau NumPy** est un **convoi** : des camions **identiques** (un seul `dtype`) qui avancent **côte à côte**, sans espace entre eux (contigu). La liste de Python, elle, est un parking : des cases qui pointent vers des objets de tailles et de types différents.
- La **vecteurisation** est la **barrière de péage ouverte** : au lieu de faire passer le convoi camion par camion (la boucle de C03), la barrière est levée et **tout le convoi passe d’un coup** (`montants * 1.19`).
- **nan** est le **camion sans marchandise** : il passe la barrière, mais sa « valeur » est une absence — et l’absence **contagieuse** : une addition qui le touche devient une absence.
- `np.where` est le **contrôle qui étiquette** le convoi : chaque camion reçoit un autocollant (« gros » / « normal ») selon une règle — le `CASE WHEN` de M06.
- Le **broadcasting** est la **règle d’alignement** : on ne peut soustraire qu’un convoi d’un autre convoi **de même forme** — sinon la machine s’arrête et dit les deux formes qui ne vont pas ensemble.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
tableau — array	un bloc contigu de valeurs toutes du même type (<code>dtype</code>), opéré en bloc.	croire qu’on peut mélanger <code>int</code> et <code>str</code> : le tableau convertit sans prévenir (§5.1).
dtype — data type	le type unique du tableau : <code>float64</code> , <code>int64</code> , <code><U21</code> (texte).	lire un tableau sans demander <code>dtype</code> : c’est là que se cache la conversion forcée.
vecteurisation vectorization	— appliquer une opération à tout le tableau d’un coup , sans boucle Python.	réécrire en <code>for</code> ce que <code>np.sum</code> fait en une ligne (le §5.7 mesure l’écart).
nan — nan	le nombre « non numérique » : l’absence qui se propage (<code>nan + 1 = nan</code>).	laisser un <code>nan</code> dans <code>np.sum</code> : le total devient <code>nan</code> — <code>np.nansum</code> regarde autour.
diffusion broadcasting	— la règle de forme qui autorise les opérations entre tableaux de formes différentes compatibles .	aligner <code>(50008,)</code> sur <code>(5,)</code> : les formes ne se compatibilisent pas — le <code>ValueError</code> dit lesquelles.

5. Cours approfondi

5.1 Le tableau — le convoi homogène

```
import numpy as np

m = np.array([1500, 2000, 1350])
print(m.dtype, m.shape)
melange = np.array([1, "a"])
print(melange, melange.dtype)
```

```
int64 (3,)
['1' 'a'] <U21
```

Définition. tableau — ndarray — un bloc de mémoire **contigu** de valeurs **homogènes** : un seul dtype pour tout le tableau, une forme (shape) qui dit combien de valeurs. L'homogénéité est ce qui permet l'opération en bloc — le parking de la liste n'a pas ce droit.

La 2^e sortie est le piège du chapitre : `np.array([1, "a"])` **ne proteste pas** — le tableau a **converti** le 1 en texte '1' (le dtype <U21 = texte de 21 caractères max). Pas d'erreur, une **décision silencieuse** : quand un tableau mélange des familles, NumPy choisit — et son choix est le plus **large** (le texte englobe), pas le plus juste.

Attention. Le tableau **force** l'homogénéité sans prévenir : `np.array([1, "a"])` donne `['1' 'a']`, pas une erreur. Demandez **toujours** le dtype avant de calculer — c'est la première ligne de lecture d'un tableau, comme le `SELECT *` limit est la première ligne de lecture d'une table.

5.2 La vecteurisation — la barrière levée

La colonne réelle du socle (extraite par `cut`, chargée par `np.loadtxt` — la commande complète est l'Étape 1 de §7) : 50 008 montants, `float64`.

```
np.sum(montants)      # le CA total
np.mean(montants)     # le panier moyen
```

```
7908259732.2
158139.8922612382
```

Définition. vecteurisation — vectorization — l'application d'une opération à **tout** un tableau d'un seul geste, sans boucle Python : `montants * 1.19` multiplie les 50 008 valeurs, `np.sum(montants)` les additionne. C'est le `SUM(colonne)` de M06, écrit en NumPy.

Dans les faits. Ces deux sorties sont **les valeurs canoniques du socle**, retrouvées par une autre porte : `np.sum` renvoie 7 908 259 732,20 (le CA brut `m08p_ca_brut` = 7 908 259 732, à 0,20 FCFA près — la dérive `float` que C08 documente en centimes) et `np.mean` renvoie 158 139,89 ≈ **158 140**, le panier moyen canonique (`m08p_panier_moyen`). Deux moteurs (DuckDB en M07, NumPy ici), le même nombre — c'est le parallèle qui tient.

Le parallèle SQL, complet :

NumPy (§5.2)	SQL (M06)	Ici
<code>np.sum(montants)</code>	<code>SUM(montant_ttc)</code>	7 908 259 732,20

NumPy (§5.2)	SQL (M06)	Ici
<code>np.mean(montants)</code>	<code>AVG(montant_ttc)</code>	158 139,89
<code>np.max(montants)</code>	<code>MAX(montant_ttc)</code>	594 363 (max canonique du socle)
<code>len(montants)</code>	<code>COUNT(*)</code>	50 008 (m08p_ventes_lignes)

5.3 Les statistiques — min, max, percentiles, médiane

```
np.min(montants)
np.max(montants)
np.percentile(montants, [25, 50, 75])
np.median(montants)
np.std(montants)
```

```
762.63
594363.0
[ 49134.2  119481.89 235875.33]
119481.89
133552.3926959041
```

- `np.min/np.max` : la 1^{re} et la dernière ligne du tri — 762,63 FCFA (la vente la plus petite du socle) et 594 363 FCFA (la plus grosse) ;
- `np.percentile(m, [25, 50, 75])` : les **quartiles** — le Q1 de M02 en NumPy : 25 % des ventes sont sous 49 134,20 FCFA, la moitié sous 119 481,89 ;
- `np.median(m)` **est** le percentile 50 : 119 481,89 (regardez — les deux fonctions redonnent le même nombre, c'est un contrôle) ;
- `np.std` : l'écart-type — 133 552,39 : les ventes sont **très** dispersées (l'écart-type dépasse la moyenne : il y a de très petites et de très grosses ventes dans le socle).

Le percentile est l'outil de la **question métier de queue** : « le 95^e percentile des montants, c'est combien ? » → 431 647,88 FCFA (mesuré, §7 Étape 3) — 95 % des ventes sont en dessous, les 5 % du haut au-dessus. C'est la version « position dans la file » du `MAX`.

5.4 nan — le camion sans marchandise

```
x = np.array([1500.0, np.nan, 2000.0])
print(x + 1)
print(np.sum(x))
print(np.nansum(x))
print(np.isnan(x))
```

```
[1501.    nan 2001.]
nan
3500.0
[False  True False]
```

Définition. nan — nan (« not a number ») — la valeur **absente** du monde des nombres : elle **se propage** (`nan + 1 = nan` — l'absence contagieuse), elle **contamine** les agrégats (`np.sum` renvoie `nan`), et elle se **détecte** (`np.isnan`) ou se **contourne** (`np.nansum` = 3500,0 : la somme regarde autour du trou).

C'est **exactement** la sémantique `NULL` de M06 : une addition qui touche un `NULL` donne `NULL` (pas 0, pas l'autre opérand) ; `COUNT(colonne)` ne compte pas les `NULL` (mais `COUNT(*)` les compte) ; `SUM` les ignore. `nan/nansum/isnan` sont les mêmes trois gestes, habillés en nombres — le pont M06 → NumPy tient sur trois lignes.

Attention. Un `nan` dans le tableau **contamine** `np.sum` — le total devient `nan`, sans erreur. Sur une colonne qui **peut** être vide (les montants du socle peuvent l'être — `est_retour`, les champs vides de M03), l'agrégat robuste est `np.nansum/np.nanmean`, pas `np.sum/np.mean`.

5.5 `np.where` — le contrôle qui étiquette

Le `CASE WHEN` de M06, en tableau — chaque montant reçoit son étiquette, d'un coup :

```
gros = np.where(montants > 100000, "gros", "normal")
print((gros == "gros").sum())
print((montants > 100000).sum())
```

```
28065
28065
```

```
SELECT CASE WHEN montant_ttc > 100000 THEN 'gros' ELSE 'normal' END
FROM vente;
```

Les deux lignes Python redonnent le même comptage (28 065 ventes « gros » — la majorité des 50 008 : dans ce socle, la grosse vente est la **norme**, pas l'exception). La 2^e ligne est l'idiome du chapitre : `montants > 100000` est un **tableau de True/False** (un `bool` par montant), et **bool a une sum** — elle compte les `True`. Pas besoin de `where` pour compter : la condition seule, suivie de `.sum()`.

5.6 Le broadcasting — la règle d'alignement

La question métier : **l'écart de chaque vente à la moyenne de son magasin**. Les moyennes par magasin (5 valeurs, mesurées) :

```
moyennes = np.array([157958.55, 157379.42, 158506.83, 158713.18, 158147.00])
montants - moyennes
```

```
ValueError : operands could not be broadcast together with shapes (50008,) (5,)
```

Définition. diffusion — broadcasting — la règle de forme de NumPy : deux tableaux peuvent s'opérer s'ils sont **de même forme**, ou si l'un est un **scalaire**, ou si leurs formes sont **compatibles** (du plus court au plus long, dimensions égales ou 1). `(50008,)` et `(5,)` ne sont compatibles ni dans un sens ni dans l'autre — d'où le `ValueError`, qui **cite les deux formes** (la lecture du message est déjà le diagnostic).

Le `ValueError` est la **règle qui parle** : 50 008 montants ne se soustraient pas à 5 moyennes — une moyenne ne se soustrait qu'à **son** magasin. L'alignement se fait **d'abord** (chaque montant connaît son magasin : `ids - 1` donne l'index de la moyenne à prendre), **ensuite** l'opération passe :

```
ecarts = montants - moyennes[ids - 1]
print(ecarts[:3])
print(np.mean(montants[ids == 1] - moyennes[0]))
```

```
[ 56186.78118011 201151.24731657 -71440.36881989]
-8.299735787245575e-12
```

La dernière sortie est le **contrôle** : l'écart moyen des ventes du magasin 1 à la moyenne du magasin 1 est ≈ 0 (-8×10^{-12} — le bruit float, pas un écart réel). Quand le contrôle redonne zéro, l'alignement est bon.

5.7 La mesure — la boucle de C03 face à la ligne

Mesurer la vecteurisation : boucle for vs une ligne

Mesuré le 19/09/2026 (atelier) — 50 008 montants réels du socle ; le rapport tient, l'absolu change de machine.

Mesure 1 — la somme	Mesure 2 — le contrôle qualité de C03
boucle for — montant par montant	boucle for — isinstance + test + compteur
3,8 ms	6,2 ms
np.sum(montants) — une ligne	vecteur — (m > 100000).sum()
0,149 ms	0,193 ms
rapport : 25×	rapport : 32×
même total des deux côtés :	même comptage des deux côtés :
7 908 259 732,20 FCFA	28 065 « gros »
(l'absolu est petit : 50 000 éléments ; le rapport, lui, est stable)	(la boucle fait plus de Python par élément → le rapport monte)

La leçon : la vecteurisation ne change PAS la réponse — elle change le chemin.

Publier une vitesse = les deux temps + le rapport + la date + la taille : un rapport non daté est une opinion, un rapport mesuré et daté est une preuve.

Deux mesures, exécutées dans l'atelier le 19/09/2026 (cette machine, ces 50 008 montants — les rapports varient d'une machine à l'autre, c'est **pourquoi** on publie la date et les deux temps) :

Mesure 1 — la somme (la boucle de C03, for + accumulation, vs np.sum) :

```
boucle for : 3.8 ms
np.sum      : 0.149 ms
rapport     : 25x
```

Mesure 2 — le contrôle qualité de C03 (for + isinstance + test + compteur, vs (m > 100000).sum()) :

```
def controle_boucle(valeurs, seuil=100000):
    gros = 0
    for v in valeurs:
        if isinstance(v, (int, float)) and v > seuil:
            gros = gros + 1
    return gros

# boucle : controle_boucle(m.tolist())          # 6.2 ms
# vecteur : (m > 100000).sum()                  # 0.193 ms
```

```
boucle : 28065 | 6.2 ms
vecteur: 28065 | 0.193 ms
rapport: 32x
```

Deux leçons : les **mêmes** résultats (28065 des deux côtés — la vecteurisation ne change pas la réponse, elle change le chemin), et un écart de **25 à 32×** sur 50 000 valeurs — un écart qui

croît quand la boucle fait plus de Python par élément (l'instance de la mesure 2 coûte plus cher qu'une addition), et qui serait encore plus grand sur 50 000 **lignes de table** avec des branchements métier. Sur une machine d'atelier, 6,2 ms ne se voit pas — mais le **rapport** est la leçon, pas l'absolu : ce qui est 32× plus lent sur 50 000 éléments est 32× plus lent sur 5 millions.

Conseil professionnel. Ne dites jamais « la boucle est lente » sans la **mesure** : publiez les deux temps, le rapport, la date, la taille — et souvenez-vous que le rapport, lui, est stable ; l'absolu, jamais. Un rapport de vitesse non mesuré est une opinion ; un rapport mesuré et daté est une preuve.

6. Exemple concret — les 4 calculs du fil rouge

La question du chapitre, en langage de comptoir : « sur la colonne `montant_ttc` réelle du socle — CA total, panier moyen, le 95^e percentile, et combien de ventes « grosses » ? » Le script — exécuté dans l'atelier le 19/09/2026 :

```
import numpy as np

m = np.loadtxt("colonne.csv", delimiter=",", skiprows=1)[: , 1]
print("CA total      :", np.sum(m))
print("Panier moyen   :", np.mean(m))
print("95e percentile :", np.percentile(m, 95))
print("Gros > 100 000 :", (m > 100000).sum())
```

```
CA total      : 7908259732.2
Panier moyen   : 158139.8922612382
95e percentile : 431647.88
Gros > 100 000 : 28065
```

Quatre calculs, **zéro boucle** : le CA total redonne le canonique du socle (7 908 259 732 à 0,20 près — `float`), le panier moyen redonne 158 140 (`m08p_panier_moyen`), le 95^e percentile (431 647,88 FCFA) est une **information nouvelle** — 95 % des ventes sont en dessous — et le comptage « gros » redonne 28 065 (la majorité : dans ce socle, la grosse vente est la norme). Le `np.loadtxt` avec `skiprows=1` saute la ligne de tête, et `[: , 1]` prend la **2^e colonne** (`montant_ttc`) — la 1^{re} (`id_magasin`) est chargée aussi, on en a besoin en §5.6.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026, `/tmp/venv_m08`).

Étape 0 — Extraire la colonne (la `cut` du socle, avant le `loadtxt`) :

```
$ { echo "id_magasin,montant_ttc"; cut -d',' -f4,11 03_exercices/dossier_M08/vente.csv | tail -n +2; } > colonne.csv
```

Interprétation : 1 ligne de tête + les 50 008 montants (`m08p_ventes_lignes`) — la colonne `montant_ttc` est la 11^e du CSV (`-f11`), `id_magasin` la 4^e (`-f4`) ; le `tail -n +2` jette la tête d'origine, l'`echo` remet la sienne. Le contrôle effectif du comptage arrive à l'Étape 1, où le `shape` (50008,) le prouve.

Étape 1 — Charger en une ligne (`np.loadtxt`) :

```
$ python3 -c "
import numpy as np
d = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)
m = d[:, 1]
print(m.shape, m.dtype)"
(50008,) float64
```

Interprétation : 50 008 valeurs, toutes float64 — le tableau s'est chargé en bloc (c'est le SELECT montant_ttc de M06, en une commande).

Étape 2 — Le CA et le panier (les deux agrégats canoniques) :

```
$ python3 -c "
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
print(np.sum(m))
print(np.mean(m))"
7908259732.2
158139.8922612382
```

Interprétation : 7 908 259 732,20 et 158 139,89 — les valeurs canoniques du socle retrouvées par NumPy (le $\pm 0,20$ est la dérive float de C08).

Étape 3 — Le 95^e percentile (la position dans la file) :

```
$ python3 -c "
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
print(np.percentile(m, 95))"
431647.88
```

Interprétation : 95 % des ventes sont sous 431 647,88 FCFA — les 5 % du haut au-dessus : c'est la « queue » de la distribution, en un nombre.

Étape 4 — Le comptage « gros » (la condition qui compte) :

```
$ python3 -c "
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
print((m > 100000).sum())"
28065
```

Interprétation : `m > 100000` est un tableau de True/False ; `.sum()` compte les True — la moitié et plus du socle est « grosse » (le seuil 100 000 FCFA est une valeur du socle).

Étape 5 — Le nan qui contourne (l'absence qui ne contamine pas) :

```
$ python3 -c "
import numpy as np
x = np.array([1500.0, np.nan, 2000.0])
print(np.sum(x), np.nansum(x))"
nan 3500.0
```

Interprétation : le même tableau, deux sommes — `np.sum` est contaminé (nan), `np.nansum` regarde autour du trou (3 500) : c'est le SUM qui ignore les NULL de M06.

Étape 6 — Le temps mesuré (la boucle face à la ligne) :

```
$ python3 -c "
import time
import numpy as np
m = np.loadtxt('colonne.csv', delimiter=',', skiprows=1)[: , 1]
l = m.tolist()
t0 = time.perf_counter()
s = 0.0
for v in l:
    s = s + v
t1 = time.perf_counter()
t2 = time.perf_counter()
s2 = np.sum(m)
t3 = time.perf_counter()
print(f'boucle for : {(t1 - t0) * 1000:.1f} ms')
print(f'np.sum      : {(t3 - t2) * 1000:.3f} ms')
print(f'rapport    : {(t1 - t0) / (t3 - t2):.0f}x')"
```

boucle for : 3.8 ms
np.sum : 0.149 ms
rapport : 25x

Interprétation : 3,8 ms contre 0,149 ms — 25×, mesuré sur cette machine, ces 50 008 montants, le 19/09/2026 : le rapport est la leçon (il tient), l’absolu est l’atelier (il change).

8. Erreurs fréquentes

1. **Le tableau qui convertit sans prévenir.** `np.array([1, "a"]) → ['1' 'a'] <U21` (sortie réelle, §5.1) : le nombre est devenu du **texte**, sans erreur. Remède : demander `dtype` **avant** de calculer — si le `dtype` est du texte (`<U...`) et que vous vouliez des nombres, la conversion s’est faite au mauvais moment (et au mauvais sens).
2. **Le nan qui contamine le total.** `np.sum` sur un tableau contenant `np.nan` renvoie `nan` (sortie réelle, §5.4) — pas d’erreur, un total **inutilisable**. Remède : `np.nansum/np.nanmean` dès que la colonne **peut** être vide ; et `np.isnan` pour **détecter** avant de calculer.
3. **Les formes qui ne s’alignent pas.** `montants - moyennes → ValueError : operands could not be broadcast together with shapes (50008,) (5,)` (sortie réelle, §5.6). Remède : le message **cite les deux formes** — lisez-les ; l’alignement se fait **avant** l’opération (`moyennes[ids - 1]`), pas en espérant que NumPy devine.
4. **La condition sans le compteur.** `m > 100000` **seul** affiche un tableau de `True/False` (50 008 lignes) — il ne **compte** rien. Remède : `(m > 100000).sum()` — un tableau `bool` **a** une `sum` (elle compte les `True`) ; c’est l’idiome du chapitre (§5.5).
5. **np.loadtxt sur un CSV qui a des mots.** Charger `vente.csv` **en entier** (13 colonnes, dont `date_vente` en texte) par `np.loadtxt` lève `ValueError: could not convert string 'id_vente' to float64 at row 0, column 1.` (sortie réelle) : le tableau est **numérique**, et la première ligne n’en est pas une — c’est la **tête** (`id_vente` n’est pas un nombre) ; passé `skiprows=1`, ce seraient les dates qui poseraient le même problème. Remède : extraire les colonnes **numériques** d’abord (`cut`, §7 Étape 0) — ou attendre C07, où `pandas.read_csv` lit le mélange (c’est son métier).
6. **Le panier moyen « sans poser la question ».** `np.mean(montants)` sur les 50 008 lignes renvoie 158 139,89 — **mais** « quelles 50 008 lignes ? » (les doublons du socle ? les retours ?) est une **question de données**, pas de calcul : changer le jeu de lignes change le nombre (les paniers 158 140 / 158 159 du socle sont des **définitions différentes**, pas des arrondis). Remède : avant d’agréger, dire **quelles lignes** on additionne — la question est un cinquième du travail, avant les quatre calculs.

9. Bonnes pratiques professionnelles

1. **Un dtype par tableau, vérifié en premier** : la lecture d'un tableau commence par `print(m.dtype, m.shape)` — comme le `head + dtypes` de C07.
2. **Compter avec `(condition).sum()`** : pas de boucle pour compter, pas de `where` pour compter — la condition booléenne **est** le compteur.
3. **`nansum/nanmean` par défaut** sur toute colonne qui **peut** être vide : le coût est zéro, le `nan` contaminant coûte un rapport à refaire.
4. **Vecteuriser le calcul, garder la boucle pour l'exception** : le calcul en bloc, la logique métier qui **branche** (`isinstance`, les cas « N/C ») reste une boucle — mais sur **peu** de lignes (les exceptions), pas sur toutes.
5. **Mesurer avant d'affirmer** : les deux temps, le rapport, la date, la taille — publiés ensemble (§5.7) ; un rapport de vitesse non daté est une opinion.

10. Exercice guidé — « les 4 statistiques du comptoir » (15 min, /10)

Énoncé. Écrire `statistiques.py` qui, sur la colonne réelle (`colonne.csv`, `np.loadtxt` avec `skiprows=1`) :

1. charge les montants (la 2^e colonne) ;
2. affiche le **minimum** et le **maximum** (2 décimales) ;
3. affiche la **médiane** et le **90^e percentile** (2 décimales) ;
4. affiche le **nombre de ventes** (la longueur du tableau).

Barème. Le chargement correct (2 pts) · les 4 statistiques avec les fonctions du chapitre (`min/max/median/percentile` — pas de tri manuel) (4 pts) · l'affichage 2 décimales en f-string (2 pts) · la sortie alignée sur le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```
Minimum : 762.63
Maximum : 594363.00
Mediane : 119481.89
90e percentile : 357969.75
Ventes : 50008
```

11. Exercices autonomes

Exercice 6.1 — Le convoi. Construire le tableau `np.array([500, 1500, 8000, 1350, 4500])`, l'afficher, afficher son `dtype` — puis le tableau multiplié par 1.19 (TVA) et sa somme. (5 min)

Exercice 6.2 — Le camion vide. Construire `np.array([1500.0, np.nan, 2000.0, np.nan])` : afficher `np.sum`, `np.nansum`, `np.isnan` — et dire en une phrase pourquoi les deux sommes diffèrent. (5 min)

Exercice 6.3 — Le contrôle. Sur la colonne réelle, étiqueter `gros/normal` avec `np.where` (seuil 100 000), et compter les `gros` **deux façons** (`(gros == "gros").sum()` et `(m > 100000).sum()`) : les deux doivent redonner le même nombre. (10 min)

Exercice 6.4 — Les quartiles. Sur `np.array([3500, 2500, 4500, 1500, 2000])` : la médiane, les quartiles (`percentile(..., [25, 75])`), et vérifier que la médiane est bien le percentile 50. (10 min)

Exercice 6.5 — La mesure. Sur une liste de 10 000 nombres (par exemple `list(range(10000))` convertie en tableau) : mesurer la somme en **boucle** (`for`) et avec `np.sum` — afficher les deux temps **et** le rapport. (10 min)

12. Correction détaillée

Exercice guidé. Le script :

```
import numpy as np

m = np.loadtxt("colonne.csv", delimiter=",", skiprows=1)[: , 1]
print(f"Minimum : {np.min(m):,.2f}")
print(f"Maximum : {np.max(m):,.2f}")
print(f"Mediane : {np.median(m):,.2f}")
print(f"90e percentile : {np.percentile(m, 90):,.2f}")
print(f"Ventes : {len(m)}")
```

Les 5 lignes publiées **sont** la sortie de l'atelier (762.63 / 594363.00 / 119481.89 / 357969.75 / 50008) — le contrôle à faire en tête : `np.median` doit redonner `np.percentile(m, 50)` (les deux sont le 50^e).

Exercice 6.1. `int64` (des entiers → le `dtype` entier, pas `float`) ; le tableau $\times 1.19$ passe en `float64` (l'opération élargit — $500 \times 1.19 = 595,0$) ; la somme : $15\,850 \times 1.19 = 18\,861,50$ — exécuter pour vérifier (`[595. 1785. 9520. 1606.5 5355.]`, sortie réelle ; le calcul mental est le piège du chapitre : la machine calcule, on ne devine pas).

Exercice 6.2. `np.sum` → `nan` (contaminé), `np.nansum` → 3500,0 (regarde autour), `np.isnan` → `[False True False True]` (deux trous). La phrase : « `np.sum` **propage** l'absence, `np.nansum` **l'ignore** — c'est le SUM de M06 qui ignore les NULL ».

Exercice 6.3. Les deux comptages redonnent **28065** — la `where` étiquette (une colonne de texte), la condition compte directement (un tableau `bool`) : même réponse, deux chemins (§5.5).

Exercice 6.4. Trié : `[1500, 2000, 2500, 3500, 4500]` — médiane 2500, `percentile([25, 75])` → 2000,0 / 3500,0 (sortie réelle : pour $n = 5$, les quartiles tombent **juste** sur des valeurs du tableau — l'interpolation se voit avec d'autres tailles, la question reste « entre quelle valeur et quelle valeur ? ») ; `percentile(..., 50)` redonne 2500,0 : le contrôle passe.

Exercice 6.5. Les deux temps dépendent de **votre** machine — c'est pour ça qu'on publie les deux **et** le rapport : la boucle est typiquement 10 à 50× plus lente que `np.sum` sur 10 000 éléments ; le rapport, lui, est la leçon stable.

13. Mini-projet M08.P6 — « Le rapport de colonne » (1 h)

Énoncé. Écrire `rapport.py` — le fil rouge, durci en **rapport** :

1. charger la colonne réelle (`colonne.csv`, `np.loadtxt`, `skiprows=1`, 2^e colonne) ;
2. afficher (2 décimales, f-string) : CA total, panier moyen, minimum, maximum ;
3. afficher les **quartiles** Q1 / médiane / Q3 sur une ligne ;
4. afficher le 95^e percentile ;
5. afficher le comptage « gros » (seuil 100 000) **sur le total** (« X ventes sur Y ») ;
6. la **robustesse** : ajouter un `nan` en fin de tableau (`np.append`), afficher `np.sum` (contaminé) **et** `np.nansum` (intact) sur la même ligne.

Critères de réussite (grille /10) : les 6 exigences (6×1 pt) · la sortie de l'atelier : CA total 7,908,259,732.20 · panier moyen 158,139.89 · minimum 762.63 · maximum 594,363.00 · quartiles 49,134.20 / 119,481.89 / 235,875.33 · 95^e 431,647.88 · 28065 ventes sur 50008 · `nan` / 7,908,259,732.20 (1 pt) · aucun `for` dans le calcul (1 pt) · le `dtype` vérifié en tête (1 pt).

Note de correction. Le `np.nansum` de la ligne 6 redonne **le même** CA que la ligne 2 (7,908,259,732.20) : le `nan` ajouté ne pèse rien — c’est la **preuve** que `nansum` « regarde autour » (pas `0 + nan`, pas un total différent). Si votre `nansum` diffère de votre `sum`, le `nan` n’est pas là où vous croyez.

14. Boîte à outils du chapitre

Boîte à outils. Le venv de C01 (`/tmp/venv_m08`) + numpy 2.5.3 ; la boîte du chapitre s’ajoute à celles de C02-C05 : le convoi et ses barrières.

Outil	Usage	Syntaxe
<code>np.array / dtype / shape</code>	le convoi et son type	<code>np.array([1, 2]).dtype</code>
<code>np.loadtxt</code>	charger une colonne numérique	<code>np.loadtxt(f, delimiter=",", skiprows=1)</code>
<code>np.sum / np.mean</code>	l’agrégat en bloc	<code>np.sum(m)</code>
<code>np.min / np.max / np.percentile</code>	les bornes et les positions	<code>np.percentile(m, [25, 50, 75])</code>
<code>np.nan / np.isnan / np.nansum</code>	l’absence qui se gère	<code>np.nansum(m)</code>
<code>np.where</code>	l’étiquette en bloc	<code>np.where(m > s, "gros", "normal")</code>
<code>(cond).sum()</code>	le compteur booléen	<code>(m > 100000).sum()</code>
broadcasting	la règle d’alignement	<code>m - moyennes[ids - 1]</code>

15. Résumé du chapitre

- Le **tableau** NumPy est homogène (`dtype` unique) et contigu ; il **convertit sans prévenir** (`[1, "a"] → ['1' 'a'] <U21`) — demander le `dtype` en premier.
- La **vecteurisation** est le SUM/AVG de M06 en mémoire : `np.sum` sur les 50 008 montants réels renvoie 7 908 259 732,20 (le canonique, à la dérive `float` près), `np.mean` redonne le panier moyen 158 140.
- Les **statistiques** : min/max (762,63 / 594 363), les quartiles (`percentile(..., [25, 50, 75])`), la médiane (le 50^e), le 95^e percentile (431 647,88) — la « position dans la file » en un nombre.
- **nan** est le NULL de M06 : `nan + 1 = nan`, `np.sum` contaminé, `np.nansum` regarde autour (3500,0), `np.isnan` détecte.
- **np.where** étiquette en bloc (le CASE WHEN), et `(m > s).sum()` **compte** (le tableau `bool` a une `sum`) — 28 065 ventes « gros ».
- Le **broadcasting** : `(50008,) - (5,)` lève `ValueError` (les formes citées) ; l’alignement **d’abord** (`moyennes[ids - 1]`), l’opération **ensuite** — le contrôle (écart moyen ≈ 0) valide l’alignement.
- La **mesure** : boucle 3,8 ms vs `np.sum` 0,149 ms (**25×**) ; boucle métier 6,2 ms vs vecteur 0,193 ms (**32×**) — mêmes résultats, daté, sur cette machine.

16. À retenir

À retenir. Le calcul en bloc est **une ligne** : `np.sum`, `np.mean`, `(m > s).sum()` — la boucle de C03 est un souvenir, pas un réflexe. Et quand un résultat « ne colle pas », la première question n'est pas « quel calcul ? » mais « **quelles lignes ?** » (§8, erreur n° 6) : le nombre est la définition, pas l'inverse.

À retenir. `nan` est le NULL habillé en nombre : il **propage** (`nan + 1 = nan`), il **contamine** (`np.sum` → `nan`), il se **contourne** (`np.nansum`) et se **détecte** (`np.isnan`) — les quatre gestes du NULL de M06, sans rien réapprendre.

17. Évaluation formative (auto-correction, 8 min)

1. `np.array([1, "a"])` renvoie quoi, et quel est le danger ? → `['1' 'a']` en dtype `<U21` : le nombre a été **converti en texte sans erreur** — le tableau force l'homogénéité silencieusement ; d'où le dtype vérifié en premier. (1 pt)
2. `np.sum(montants)` sur les 50 008 montants du socle renvoie quoi, et qu'est-ce que ça redonne ? → 7 908 259 732,20 — le CA brut canonique (`m08p_ca_brut = 7 908 259 732`), à 0,20 près (la dérive `float` documentée en C08). (1 pt)
3. Pourquoi `np.sum` renvoie `nan` sur un tableau contenant `np.nan`, et quelle est la version robuste ? → `nan` **propage** (l'absence contagieuse) ; la version robuste est `np.nansum` (regarde autour du trou — le SUM qui ignore les NULL de M06). (2 pts)
4. `np.where(m > 100000, "gros", "normal")` fait quoi, et comment compter les « gros » sans `where` ? → il **étiquette** le tableau (une colonne de texte) ; pour compter : `(m > 100000).sum()` — le tableau `bool` a une `sum` (elle compte les `True`). (2 pts)
5. `montants - moyennes (50008, et 5)` lève quoi, et d'où vient le message ? → `ValueError` : `operands could not be broadcast together with shapes (50008,) (5,)` : les formes ne sont pas compatibles — le message **cite les deux formes** ; l'alignement se fait d'abord (`moyennes[ids - 1]`). (1 pt)
6. Le **contrôle d'alignement** de §5.6 redonne quoi, et pourquoi est-ce le contrôle ? → l'écart moyen des ventes du magasin 1 à sa moyenne ≈ 0 (-8×10^{-12} , le bruit `float`) : si le contrôle ne redonne pas zéro, l'alignement est mauvais. (1 pt)
7. `np.percentile(m, 50)` et `np.median(m)` — quelle relation, et pourquoi est-ce un contrôle ? → ce sont **le même nombre** (le 50^e percentile est la médiane) : les deux fonctions doivent redonner le même résultat, sinon le chargement a trahi. (1 pt)
8. Pourquoi le rapport « boucle vs vecteur » est-il la leçon, et pas les temps absolus ? → l'absolu dépend de la machine (il change) ; le **rapport** est stable — et il croît quand la boucle fait plus de Python par élément ; d'où publier les deux temps + la date + la taille. (1 pt)

Module M08.C07 — pandas I : `Series`, `DataFrame`, `read_csv`/`read_excel`, `loc`/`iloc`, `filtres`, `nouvelles colonnes`, `describe`, `valeurs manquantes`

Outils : le `venv` de C01 (`/tmp/venv_m08`, pandas 3.0.6) + l'environnement d'écriture du manuel (pandas 2.2.3) pour le comparé du §5.10. Le dossier `03_exercices/dossier_M08/` (les 8 CSV du socle). **Durée indicative :** 5 h. **Niveau :** N6 → N7. **Prérequis :** C06 (le tableau NumPy est une colonne pandas sans en-tête).

L'idée du chapitre. C06 calculait sur **une colonne** en bloc ; C07 travaille sur **la table entière** — les 50 008 lignes × 13 colonnes réelles du socle — avec les deux objets de pandas : la **Série** (une colonne : index + valeurs) et le **DataFrame** (un tableau de colonnes nommées, la table SQL de M07 en Python). Le fil rouge : **les mêmes chiffres, trois chemins** — Excel (M05), SQL (M07), pandas (M08) : chaque bloc a son équivalent SQL côte à côte, et l'exercice du chapitre est l'**audit complet du dossier en 8 lignes** — formes, doublons, retours, dates inversées, table vide — chaque résultat **identique** au SQL de M07. C08 fera le reste : `groupby`, `merge`, `pivot_table`, dates, export Excel.

Matériel de l'atelier — deux pandas, une règle. Le `venv` du chapitre est **pandas 3.0.6** (l'installation fraîche de C01) ; l'environnement d'écriture du manuel est **figé sur 2.2.3**. Les deux co-existent, et le manuel enseigne les patterns **valides sur les deux** — C01 a posé la règle, le §5.10 la prouve avec **4 écarts mesurés** sur un fichier réel (103 → 85, 373 → 372, 18 → 0, 7 → 0), pas illustrés. Tout le code a été exécuté le 19/09/2026 ; les sorties publiées sont celles de l'atelier.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Importer le **dossier M08 complet** (les 8 CSV, formes réelles : 50 008 × 13, 1 200 × 5, ..., 0 × 4) avec `pd.read_csv` — `head`, `shape`, `dtypes`.
2. Lire un **DataFrame** comme on lit une table SQL : l'**index** (pas une colonne), les colonnes nommées, le `dtype` de chacune — `str` sous pandas 3 (plus d'object).
3. Sélectionner avec `loc` (par nom) et `iloc` (par position) — le tableau de 4 cas (ligne/colonne × nom/position) et le piège des tranches.
4. **Filtrer** comme en `WHERE` : le **masque** booléen, `&`/`|`/`~`, les **parenthèses obligatoires** — et le bug silencieux qu'elles évitent (mesuré : 50 008 au lieu de 27 938).
5. Créer une **nouvelle colonne** par opération vectorisée (`montant_ht = montant_ttc / (1 + taux_tva)`) — le `SELECT ... AS` de M07, et assign.
6. Lire `describe()` : les 8 statistiques de M02 en une commande — et le détail qui distingue pandas de NumPy (`std` échantillonnaire vs population).
7. Gérer les **valeurs manquantes** : `isna()`, `dropna()`, `fillna()`, `ffill()` — et le cas réel du socle : **zéro manquant mesuré**, et la table `objectif_magasin` **vide** (0 ligne, en-tête seul) et son `KeyError`.
8. Repérer les **4 écritures obsolètes** de la 1^{re} salve des 12 (`append`, `inplace`, affectation chaînée, `fillna(method=...)`) — exécutées, pas citées.

2. Pourquoi cette notion est importante

Parce que le **DataFrame est la table SQL de M07, transportée dans Python**. Vous savez déjà lire une table, un `WHERE`, un `SELECT ... AS`, un `GROUP BY` — pandas ne vous demande pas de repartir de zéro, il vous donne les **mêmes opérations** sur un objet en mémoire : `v[v["est_retour"]]` est le `WHERE est_retour = TRUE`, `v["montant_ttc"].sum()` est le `SUM(montant_ttc)`, `describe()` est le paquet `COUNT/AVG/STDDEV/MIN/MAX`. Le chapitre est donc une **traduction** — et la traduction est le but : le même chiffre sort de trois portes (Excel M05, SQL M07, pandas M08), et un jour vous choisirez la porte selon le contexte, pas selon l'habitude.

Parce que les **erreurs de pandas sont souvent silencieuses**. Une boucle Python proteste (C05) ; un filtre sans parenthèses, lui, **répond** — avec le mauvais résultat (le §5.5 le mesure : la table entière sélectionnée, aucun avertissement). La compétence du chapitre n'est pas « savoir écrire un filtre », c'est « savoir qu'un filtre peut avoir raison **sans qu'on le sache** » — d'où les contrôles de taille (`len`, `.sum()`) systématiques après chaque sélection.

Parce que **la version change la vérité terrain**. Sur le même fichier, pandas 2.2.3 et 3.0.6 ne comptent pas les mêmes cellules vides (103 contre 85 — mesuré, §5.10). « Ce script tourne mais ses chiffres sont faux » (l'évaluation E4 du module) n'est pas une figure de style : c'est ce qui arrive quand on branche son code sur le `dtype` au lieu de la **valeur**.

3. Explication simple — le classeur d'indexation

- Le **DataFrame** est un **classeur d'indexation** : des fiches (les lignes) rangées par dossiers (les colonnes **nommées**). Chaque dossier est exactement le **convoi NumPy de C06** — un type, un bloc, une opération en bloc — avec une étiquette (le nom de colonne) et une colonne de référence (l'**index**).
- L'**index** est la **réduire** du classeur : le numéro de fiche. Ce n'est pas une fiche, ce n'est pas une colonne — c'est la **poignée** par laquelle `loc` et `iloc` attrapent les lignes.
- Un **filtre** est le **tamis** : on pose la question sur **toutes les fiches** d'un coup (`v["est_retour"] = True/False` par fiche) et on ne garde que les fiches qui passent — le `WHERE` de M07, sans boucle.
- Une **nouvelle colonne** est un **nouveau dossier** rempli par une opération sur les valeurs existantes : `montant_ht = montant_ttc / (1 + taux_tva)` — chaque fiche reçoit sa valeur, en bloc.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
tableau données — DataFrame	de un tableau 2-D de colonnes nommées (une table SQL en Python), le cœur de pandas.	le parcourir ligne par ligne avec une boucle : c'est exactement ce qu'il faut éviter (§5.5, §5.6).
série — Series	une colonne : un index + des valeurs (le cousin 1-D du DataFrame, le tableau de C06 avec étiquettes).	confondre l' index (l'étiquette des lignes) et la première colonne : ce n'est pas la même chose (§5.3).
index — index	la réduire : l'étiquette de chaque ligne (par défaut 0, 1, 2, ...), utilisée par <code>loc/iloc</code> .	croire que c'est une colonne : <code>reset_index()</code> la transforme en colonne (§5.3) — preuve qu'elle n'en était pas une.
sélection par nom — <code>loc</code>	attraper lignes et/ou colonnes par leurs étiquettes (noms, bornes incluses).	<code>v.loc[1:3]</code> inclut la ligne 3 — 3 lignes, pas 2 (§5.4).
sélection par position — <code>iloc</code>	attraper lignes et/ou colonnes par leur numéro (0 = premier, borne haute exclue).	<code>v.iloc[1:3]</code> exclut la ligne 3 — 2 lignes, pas 3 (§5.4).

5. Cours approfondi

5.1 read_csv — l'import des 8 CSV réels

```
import pandas as pd

fichiers = ["vente", "client", "produit", "categorie", "magasin",
            "mode_paiement", "regle_tva", "objectif_magasin"]
for nom in fichiers:
    df = pd.read_csv(f"03_exercices/dossier_M08/{nom}.csv")
    print(f"{nom}: {df.shape[0]} lignes x {df.shape[1]} colonnes")
```

```
vente: 50008 lignes x 13 colonnes
client: 1200 lignes x 5 colonnes
produit: 380 lignes x 6 colonnes
categorie: 8 lignes x 3 colonnes
magasin: 5 lignes x 6 colonnes
mode_paiement: 5 lignes x 4 colonnes
regle_tva: 2 lignes x 2 colonnes
objectif_magasin: 0 lignes x 4 colonnes
```

Définition. tableau de données — DataFrame — un tableau à deux dimensions de **colonnes nommées** (une table SQL en Python), l'objet central de pandas : `shape` donne lignes × colonnes, `columns` les noms, `dtypes` la famille de chaque colonne.

Définition. série — Series — une **colonne** : un index (les étiquettes de lignes) + des valeurs. C'est le tableau NumPy de C06 **avec étiquettes** : tout ce que `np.sum` sait faire, une Série le sait faire en point (`s.sum()`), plus la mémoire de sa position dans la table.

Le 8^e résultat est le fait du dossier : `objectif_magasin` a **0 ligne** (en-tête seul — une table peut n'avoir aucune ligne ; trivial en CSV, impossible à montrer dans Excel). §5.8 regarde ce que pandas en fait.

pandas (§5.1)	SQL / M07	Ici
<code>pd.read_csv("vente.csv")</code>	chargement du jeu (M05)	50 008 × 13 (m08p_ventes_lignes, m08p_ventes_colonnes)
<code>df.shape</code>	SELECT COUNT(*) + le schéma	les 8 formes du dossier
<code>df.dtypes</code>	DESCRIBE information_schema	/ int64, float64, str, bool

5.2 head, shape, dtypes — la première lecture

```
v = pd.read_csv("03_exercices/dossier_M08/vente.csv")
print(v.head(3).to_string())
print()
print(v.dtypes.to_string())
```

	id_vente	id_client	id_produit	id_magasin	id_mode	date_vente	date_limite_remise	quantite
prix_unitaire_ht		taux_tva	montant_ttc	montant_remise	est_retour			
0	1	192	380	2	3	2026-12-12	2026-12-20	
5		35893.48	0.19	213566.20	2135.66	True		
1	2	318	311	1	1	2026-10-06	2026-10-20	
7		43110.42	0.19	359109.80	0.00	True		
2	3	447	310	2	1	2025-11-07	2025-11-20	
5		14565.94	0.18	85939.05	0.00	True		

id_vente	int64
id_client	int64
id_produit	int64
id_magasin	int64
id_mode	int64
date_vente	str
date_limite_remise	str
quantite	int64
prix_unitaire_ht	float64
taux_tva	float64
montant_ttc	float64
montant_remise	float64
est_retour	bool

Les dates sont en `str` — le type texte **dédié** de pandas 3 (C01 a posé la règle : sous l'environnement d'écriture 2.2.3, ces mêmes colonnes sont en `object` ; §5.10 mesure l'effet). Trois familles sur 13 colonnes : des entiers (`int64`), des décimaux (`float64`) — `montant_ttc` y est un **DOUBLE**, la leçon des centimes de C08 — et des textes/booléens.

Attention. Ne **branchez pas** votre code sur le `dtype` : il diffère entre 2.2.3 (`object`) et 3.x (`str`) pour **le même fichier** (mesuré, §5.10). Branchez sur la **valeur** (comparer, convertir, compter) — elle ne change pas de version.

5.3 L'index — la réduction, pas une colonne

```
print(v.index[:3])
print(v.loc[0, "montant_ttc"])
print(v.iloc[0, 10])
w = v.head(2).reset_index()
print(list(w.columns))
print(w.shape)
```

```
RangeIndex(start=0, stop=3, step=1)
213566.2
213566.2
['index', 'id_vente', 'id_client', 'id_produit', 'id_magasin', 'id_mode',
 'date_vente', 'date_limite_remise', 'quantite', 'prix_unitaire_ht', 'taux_tva',
 'montant_ttc', 'montant_remise', 'est_retour']
(2, 14)
```

Définition. index — index — l'étiquette de chaque ligne (par défaut 0, 1, 2, ...) : la poignée par laquelle `loc` et `iloc` attrapent les lignes. Ce **n'est pas une colonne** — `reset_index()` en fait une (`index` passe en tête, 14 colonnes au lieu de 13, forme (2, 14)), ce qui prouve exactement ce qu'il était.

`v.loc[0, "montant_ttc"]` et `v.iloc[0, 10]` redonnent **la même valeur** (213566.20) : la même cellule, prise par **nom** ou par **position** — le §5.4 dresse le tableau des 4 cas. L'index ne porte pas toujours le sens d'un numéro : en C08, il sera **daté** (l'index porte le mois) — c'est alors la raison d'être de `reset_index` (remettre les dates dans une colonne normale).

5.4 loc vs iloc — le tableau des 4 cas

loc vs iloc : le tableau des 4 cas

La même pièce, 4 portes — les étiquettes (loc) ou les numéros (iloc).

	id_vente	id_magasin	montant_ttc	est_retour
0	1	2	213566.20	True
1	2	1	359109.80	True
2	3	2	85939.05	True
3	4	3	390891.06	True
4	5	4	110866.60	True

l'extrait réel : v.head(5), 4 colonnes de vente.csv

cellule orange = (ligne 1, montant_ttc) · colonne bleue = id_magasin

par NOM (loc)

par POSITION (iloc)

— une LIGNE —

ligne · par nom

v4.loc[1, 'montant_ttc']

→ 359109.80

ligne · par position

v4.iloc[1, 2]

→ 359109.80

— une COLONNE —

colonne · par nom

v4.loc[:, 'id_magasin']

→ [2, 1, 2, 3, 4]

colonne · par position

v4.iloc[:, 1]

→ [2, 1, 2, 3, 4]

les 4 portes donnent la même valeur :

la cellule orange (359109.80) ou la colonne bleue — toujours la même pièce

Le piège des tranches : v4.loc[1:3] → 3 lignes · v4.iloc[1:3] → 2 lignes

la borne haute est INCLUDE par nom (loc), EXCLUDE par position (iloc) —
même écriture, une ligne de moins : le contrôle len() est la garde-fou.

Sur l'extrait réel des 5 premières ventes (4 colonnes), la **même** cellule ligne 1 / montant_ttc = 359109.80 est atteinte par les 4 portes :

```
v4 = v.head(5)[["id_vente", "id_magasin", "montant_ttc", "est_retour"]]
print(v4.loc[1, "montant_ttc"]) # ligne par nom, colonne par nom
print(v4.iloc[1, 2])           # ligne par place, colonne par place
print(v4.loc[:, "id_magasin"]) # TOUTES les lignes, colonne par nom
print(v4.iloc[:, 1])           # TOUTES les lignes, colonne par place
```

```
359109.8
359109.8
0    2
1    1
2    2
3    3
4    4
Name: id_magasin, dtype: int64
0    2
1    1
2    2
3    3
4    4
Name: id_magasin, dtype: int64
```

	par nom (loc)	par position (iloc)
une ligne	v4.loc[1, "montant_ttc"] → 359109.8	v4.iloc[1, 2] → 359109.8
une colonne	v4.loc[:, "id_magasin"]	v4.iloc[:, 1]

Deux règles à retenir : **loc parle en étiquettes** (noms de colonnes, valeurs d'index — la borne haute d'une tranche est **incluse**), **iloc parle en numéros** (0 = premier — la borne haute est **exclue**, comme les tranches Python de C04). Le piège, mesuré sur la table réelle :


```
print(len(v.loc[1:3]))    # 3 lignes : id_vente 2, 3, 4
print(len(v.iloc[1:3]))  # 2 lignes : id_vente 2, 3
```

```
3
2
```

`v.loc[1:3]` **inclut** la ligne 3 (les étiquettes vont de 1 à 3, borne comprise) ; `v.iloc[1:3]` s'arrête **avant** le numéro 3. Même écriture, une ligne de moins — c'est l'erreur n° 1 du débutant pandas, et elle est **silencieuse**.

5.5 Les filtres — le `WHERE` en masque

```
print(v["est_retour"].sum())
print(len(v[v["est_retour"] == False]))
print((v["montant_ttc"] > 100000).sum())
print(((v["montant_ttc"] > 100000) & (v["est_retour"] == False)).sum())
```

```
208
49800
28065
27938
```

Définition. masque — mask — le résultat d'une comparaison sur une colonne : une Série de `True/False` **par ligne** (28 065 `True` sur `montant_ttc > 100000`), posée dans les crochets du Data-Frame pour ne garder que les lignes vraies : `v[masque]` est le `WHERE` de M07.

pandas (§5.5)	SQL (M07)	Ici
<code>v[v["est_retour"]]</code>	<code>WHERE est_retour = TRUE</code>	208 lignes (<code>m08p_retours_brut</code>)
<code>(v["montant_ttc"] > 100000).sum()</code>	<code>WHERE montant_ttc > 100000</code> <code>COUNT(*)</code>	+ 28 065 (<code>m08p_ventes_gros_100k</code>)
<code>... & ... / ... \ ... / ~...</code>	<code>AND / OR / NOT</code>	27 938 gros non retournés

Les opérateurs sont ceux du SQL, en symboles : `&` (et), `|` (ou), `~` (non) — et **chaque condition entre parenthèses, sans exception**. Pourquoi c'est une question de vie ou de mort pour le chiffre, et pas de style :

```
print(((v["montant_ttc"] > 100000) & (v["est_retour"] == False)).sum())
print((v["montant_ttc"] > 100000 & v["est_retour"]).sum()) # les parenthèses manquent
```

```
27938
50008
```

Sans parenthèses, `&` se lie **avant** `>` : la machine compare chaque montant à 100000 `&` `est_retour` (100 000 pour les retours, **0** pour le reste) — et sélectionne la **table entière** (50 008 au lieu de 27 938). **Aucun avertissement** : c'est le bug silencieux du chapitre — la bonne réponse et la mauvaise se présentent pareil, seules les tailles diffèrent. D'où la discipline du §9 : **après chaque filtre, `len()`** — 50 008 sur une question qui ne peut pas avoir 50 008 réponses, c'est un chiffre qui parle.

Attention. Un filtre pandas **répond toujours** — 27 938 ou 50 008, le texte se présente pareil, et rien ne distingue les deux : la parenthèse manquante est le bug silencieux du chapitre, et le `len()` après chaque sélection est la seule garde-fou qui coûte zéro.

5.6 Les nouvelles colonnes — le `SELECT ... AS`

```
v["montant_ht"] = v["montant_ttc"] / (1 + v["taux_tva"])
print(v[["montant_ttc", "montant_ht"]].head(3).to_string())
print(v["montant_ht"].sum())
w = v.assign(montant_ht=v["montant_ttc"] / (1 + v["taux_tva"]))
print(len(w.columns))
```

```
montant_ttc    montant_ht
0    213566.20  179467.394958
1    359109.80  301772.941176
2     85939.05   72829.703390
6673549808.234654
14
```

pandas (§5.6)

SQL (M07)

```
v["montant_ht"] = v["montant_ttc"] / (1 + v["taux_tva"])
v["taux_tva"]
```

```
SELECT montant_ttc, montant_ttc / (1 + taux_tva) AS
montant_ht
```

C'est la **vecteurisation de C06 dans pandas** : `/ (1 + taux_tva)` s'applique aux 50 008 lignes d'un coup (deux colonnes divisées l'une par l'autre — le broadcasting de C06 forme par forme, ici en nom de colonne). `assign` est la variante **lisible** pour enchaîner : `v.assign(a=..., b=...)` renvoie la table augmentée (14 colonnes) sans modifier l'originale — le style conseillé quand on construit une chaîne de transformations (C08).

5.7 `describe()` — la carte d'identité statistique

```
print(v["montant_ttc"].describe().to_string())
```

```
count    50008.000000
mean     158139.892261
std       133553.728026
min        762.630000
25%       49134.200000
50%       119481.890000
75%       235875.330000
max       594363.000000
```

Les **8 statistiques de M02**, en une commande : le `count` (50 008), la moyenne (158 139,89 — le panier moyen canonique 158 140 à l'arrondi, `m08p_panier_moyen`), les quartiles (49 134,20 / 119 481,89 / 235 875,33 — **les mêmes** que `np.percentile(..., [25, 50, 75])` de C06), la médiane (le 50 %), le min (762,63), le max (594 363). Le pont C06 → C07 tient : même colonne, même chiffres.

Le détail qui **diffère** : `std` = 133 553,73 ici, 133 552,39 avec `np.std` (C06) — pandas calcule l'écart-type **échantillonnaire** (division par $n-1$), NumPy l'écart-type **population** (division par n). La question de M02 retrace le chapitre : écart-type de **l'échantillon** (on estime) ou de la **population complète** (on a tout) ? Ici on a tout (le socle entier) — mais le réflexe à retenir est de **regarder** le `std` avant de comparer deux sorties.

5.8 Les valeurs manquantes — l'audit du dossier

```
print(v.isna().sum().sum())
o = pd.read_csv("03_exercices/dossier_M08/objectif_magasin.csv")
print(o.shape)
print(o["ca_objectif_ttc"].sum())
o["ca_objectif"]
```

```
0
(0, 4)
0
KeyError: 'ca_objectif'
```

Fait du socle (mesuré) : zéro cellule manquante dans les 8 CSV du dossier — l'audit `isna().sum().sum()` renvoie 0. `read_csv` fait ce qu'on attend du `objectif_magasin` vide : un DataFrame de **0 ligne × 4 colonnes** (l'en-tête donne les noms), les colonnes en `object` (aucune valeur pour deviner le type) ; `o["ca_objectif_ttc"].sum()` répond **0** sans broncher, et `o["ca_objectif"]` (celle que vous croyiez exister) lève `KeyError` — l'indexation d'une table vide ne pardonne pas les noms approximatifs.

La boîte à outils des manquants quand le fichier **en a** (démonstration sur une petite série, exécutée) :

```
s = pd.Series([1500.0, None, 2000.0, None])
print(s.isna().sum())
print(list(s.dropna()))
print(list(s.fillna(0)))
print(list(s.ffmpeg()))
```

```
2
[1500.0, 2000.0]
[1500.0, 0.0, 2000.0, 0.0]
[1500.0, 1500.0, 2000.0, 2000.0]
```

pandas (§5.8)	SQL (M06)
<code>s.isna().sum()</code>	<code>COUNT(*) - COUNT(colonne)</code> (les NULL)
<code>s.dropna()</code>	<code>WHERE colonne IS NOT NULL</code>
<code>s.fillna(0)</code> / <code>s.ffmpeg()</code>	<code>COALESCE(colonne, 0)</code> / portage de la dernière valeur

Conseil professionnel. L'audit `isna().sum()` précède tout agrégat — sur ce socle il renvoie 0 (mesuré), mais votre fichier du lundi ne vous le doit pas : un `nan` silencieux dans une colonne (C06 §5.4) contamine la somme, et l'audit d'une ligne est la seule garde-fou qui coûte zéro.

5.9 read_excel — le même objet, un autre format

```
v.head(5).to_excel("/tmp/c07/apercu.xlsx", index=False)
x = pd.read_excel("/tmp/c07/apercu.xlsx")
print(x.shape)
print(x.dtypes.to_string())
```

```
(5, 13)
id_vente          int64
id_client         int64
id_produit        int64
id_magasin        int64
id_mode           int64
date_vente        str
date_limite_remise str
quantite          int64
prix_unitaire_ht  float64
taux_tva          float64
montant_ttc       float64
montant_remise    float64
est_retour        bool
```

Aller-retour **sans perte** : 5×13 , mêmes dtypes que le CSV (les dates redeviennent du texte str, est_retour son bool). C'est le pont vers C08 (to_excel multi-feuilles pour la synthèse du projet M08.P) et le rappel du M05 : Excel n'est pas un format de données, c'est un **format de livraison** — le CSV reste la porte d'entrée, l'Excel la porte de sortie.

5.10 Les 4 écritures obsolètes — 1^{re} salve des 12

Le module documente **12 écritures** qui ne passent plus en pandas 3.x (liste complète : C08, annexe migration). Les **4** du chapitre, **exécutées** sous pandas 3.0.6 :

```
v.append({"id_vente": 999}) # 1) append
```

```
AttributeError: 'DataFrame' object has no attribute 'append'
```

→ **supprimée en 2.0** : on concatène — `pd.concat([v, nouvelle_ligne])`.

```
s = pd.Series([1.0, None, 3.0])
r = s.fillna(0, inplace=True)
r
```

```
0    1.0
1    0.0
2    3.0
dtype: float64
```

→ 2) `inplace=True` : sous 3.0.6, la ligne **modifie** la série **et** la renvoie — alors que sous 2.2.3, la **même ligne renvoie None** (sortie réelle de l'atelier : retour 2.2.3 : None). La même écriture, **deux comportements** c'est « sans effet utile » au sens fort. L'écriture validée sur les deux versions est unique — **réassigner** : `s = s.fillna(0)`.

```
avant = v["montant_ttc"].sum()
v[v["est_retour"] == True]["montant_ttc"] = 0 # 3) affectation chaînée
apres = v["montant_ttc"].sum()
print(avant)
print(apres - avant)
```

```
7908259732.2
0.0
```

(avec, sur la sortie d'erreur, un `ChainedAssignmentError` : « a value is being set on a copy ... due to Copy-on-write — try `.loc[row_indexer, col_indexer] = value` »)

→ 3) l'**affectation chaînée** : le code **ne tombe pas** (l'avertissement part sur la sortie d'erreur, le script continue), et le changement est **perdu** (écart 0,0 : la somme n'a pas bougé). C'est le « cas muet sous copy-on-write » — le piège n° 1 de pandas. L'écriture validée : `v.loc[masque, "montant_ttc"] = 0` — l'affectation en une étape, par la réduire.

```
s = pd.Series([1.0, None, 3.0])
s.fillna(method="ffill")
```

```
TypeError: NDFrame.fillna() got an unexpected keyword argument 'method'
```

→ 4) `fillna(method="ffill")` : **supprimée en 3.0** — on appelle les méthodes directes : `s.ffill()` / `s.bfill()` (exécutées en §5.8).

Dans les faits. Les 4 écarts mesurés 2.2.3 / 3.0.6 — sur le fichier « attendu » de M03 (480 lignes × 13, séparateur ;), même code, deux versions, le 19/09/2026 :

mesure	pandas 2.2.3	pandas 3.0.6
cellules vides (client)	103	85
modalités brutes (client)	373	372
lignes champ vide	18	0
colonnes texte object	7	0
dtype d'une colonne texte	object	str

Les 18 lignes « champ vide » de 2.2.3 sont **les mêmes 18** que les $85 + 18 = 103$ qui font 103 sous 2.2.3 et 85 sous 3.0.6 : sous 3.x, une cellule **vide** n'est plus une chaîne vide à compter, c'est une **absence** (NaN) que `isna()` voit et que `nunique()` ne compte pas. D'où l'écriture obsolète n° 10 de la liste : **compter une cellule texte vide comme non-vide** — on normalise **avant** de compter. C'est l'exemple fil rouge de l'évaluation E4 (« ce script tourne mais ses chiffres sont faux ») : rien ne proteste, le `dtype` a changé de nom, et 18 lignes se sont égarées entre les deux versions.

6. Exemple concret — l'audit du dossier en 8 lignes

La question du chapitre, en langage de chef de projet : « on nous a livré le dossier M08 (le brief est dans `00_brief.md`) — que vaut ce jeu de données ? » Le script — exécuté dans l'atelier le 19/09/2026 :

```
import pandas as pd

d = "03_exercices/dossier_M08/"
v = pd.read_csv(d + "vente.csv")
o = pd.read_csv(d + "objectif_magasin.csv")
dv = pd.to_datetime(v["date_vente"]); dl = pd.to_datetime(v["date_limite_remise"])

formes = {f: pd.read_csv(d + f + ".csv").shape[0]
          for f in ["vente", "client", "produit", "categorie",
                  "magasin", "mode_paiement", "regle_tva", "objectif_magasin"]}
print("formes      :", formes)
print("doublons    :", v.duplicated(subset=[c for c in v.columns if c != "id_vente"]).sum())
print("retours     :", v["est_retour"].sum())
print("dates inver :", (dv > dl).sum())
print("lignes vides:", o.shape[0])
```

```
formes      : {'vente': 50008, 'client': 1200, 'produit': 380, 'categorie': 8,
               'magasin': 5, 'mode_paiement': 5, 'regle_tva': 2, 'objectif_magasin': 0}
doublons    : 8
retours     : 208
dates inver : 21406
lignes vides: 0
```

Cinq constats, **chacun déjà connu de M07** — c'est le fil rouge « les mêmes chiffres, trois chemins », vérifié :

Constat	pandas (ci-dessus)	SQL M07 (côté à côté)	Clé du socle
Les formes	50 008 × 13, 1 200, 380, 8, 5, 5, 2, 0	les <code>COUNT(*)</code> des 8 tables	<code>m08p_ventes_lignes...</code>
8 doublons (hors <code>id_vente</code>)	<code>v.duplicated(subset=...).sum()</code>	les 8 lignes à clé dupliquée (C02 de M07)	<code>m08p_doublons = 8</code>
208 retours	<code>v["est_retour"].sum()</code>	<code>SUM(est_retour)</code>	<code>m08p_retours_brut = 208</code>

Constat	pandas (ci-dessus)	SQL M07 (côté à côté)	Clé du socle
21 406 inversées	dates (dv > dl).sum()	le vice des deadlines (C06 de M07)	m08p_dates_vente_apres_limite = 21406
0 ligne dans objectif_magasin	o.shape[0]	la table vide du schéma	m08p_objectif_lignes = 0

Le piège du 2^e print mérite sa ligne : `v.duplicated()` **tout court** renvoie **0** (toutes les lignes diffèrent — `id_vente` est unique !) ; les doublons du socle sont des **copies sans l'identifiant**, d'où le `subset=` qui retire `id_vente`. L'audit sans `subset` est le faux négatif du chapitre : « aucun doublon » sur une table qui en a 8.

7. Démonstration pas à pas — 6 étapes sur le fil rouge

Sortie réelle de chaque commande (exécutées le 19/09/2026, venv pandas 3.0.6).

Étape 1 — L'import (`read_csv` sur le jeu principal) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v.shape)
(50008, 13)
```

Interprétation : 50 008 lignes × 13 colonnes (`m08p_ventes_lignes`, `m08p_ventes_colonnes`) — la table de M07, entière, en mémoire.

Étape 2 — La première lecture (`head` + `dtypes`) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v.dtypes.to_string())"
id_vente          int64
id_client         int64
id_produit        int64
id_magasin        int64
id_mode           int64
date_vente        str
date_limite_remise str
quantite          int64
prix_unitaire_ht  float64
taux_tva          float64
montant_ttc       float64
montant_remise    float64
est_retour        bool
```

Interprétation : 3 familles — `int64`, `float64`, et le texte `str` de pandas 3 (les dates), plus le `bool` de `est_retour`.

Étape 3 — Le filtre retours (le `WHERE` en masque) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v['est_retour'].sum())
print(len(v[v['est_retour'] == False]))"
208
49800
```

Interprétation : 208 retours bruts (`m08p_retours_brut`) et le complément — toujours les deux chiffres : le total moins les uns, c'est le contrôle mutuel.

Étape 4 — La nouvelle colonne (`montant_ht` vectorisée) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
v['montant_ht'] = v['montant_ttc'] / (1 + v['taux_tva'])
print(v['montant_ht'].head(3).to_string())"
0      179467.394958
1      301772.941176
2       72829.703390
Name: montant_ht, dtype: float64
```

Interprétation : 50 008 valeurs en une ligne — la TVA de chaque vente déduite, sans boucle (le `SELECT ... AS` de M07).

Étape 5 — La carte statistique (`describe`) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v['montant_ttc'].describe().to_string())"
count      50008.000000
mean       158139.892261
std        133553.728026
min         762.630000
25%        49134.200000
50%       119481.890000
75%       235875.330000
max       594363.000000
```

Interprétation : les 8 statistiques de M02 — la moyenne redonne le panier moyen canonique (158 140 à l'arrondi), les quartiles ceux de C06 ; le `std` est l'écart-type échantillonnaire ($n-1$) — regardez-le avant de comparer.

Étape 6 — L'audit des manquants (`isna`) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
print(v.isna().sum().sum())"
0
```

Interprétation : zéro cellule manquante dans le jeu principal (fait du socle, mesuré) — l'audit est une ligne, et il précède tout agrégat (§5.8).

8. Erreurs fréquentes

- Le filtre sans parenthèses — le bug silencieux.** `(v["montant_ttc"] > 100000 & v["est_retour"]).sum()` → **50 008** au lieu de 27 938 (sortie réelle, §5.5) : `&` se lie avant `>`, la machine compare à `100000 & est_retour`, et **rien ne proteste**. Remède : chaque condition entre parenthèses, **sans exception** — et `len()` après chaque filtre : le chiffre qui ne peut pas être le total de la table est un chiffre qui parle.
- loc/iloc mélangés — la ligne de moins.** `v.loc[1:3]` → 3 lignes, `v.iloc[1:3]` → 2 lignes (sortie réelle, §5.4) : la borne haute est incluse par nom, exclue par position. Remède : une **seule** des deux fonctions par script — `loc` par défaut (les noms), `iloc` quand la position est l'information.
- L'affectation chaînée — le changement perdu.** `v[v["est_retour"] == True]["montant_ttc"] = 0` : le code ne tombe pas (avertissement sur la sortie d'erreur, §5.10), et l'écart est **0,0** — la somme n'a pas bougé. Remède : `v.loc[masque, "montant_ttc"] = 0` — une étape, par la réécriture.

4. **Compter les doublons sans subset.** `v.duplicated().sum()` → **0** sur une table qui a **8** doublons (le §6 le mesure) : `id_vente` est unique, donc « toutes colonnes » ne duplique rien. Remède : `duplicated(subset=...)` sur les colonnes qui **définissent** la ligne — ici tout sauf `id_vente`.
5. **Brancher sur le dtype au lieu de la valeur.** « Si la colonne est en `object`, c'est du texte » — faux sous 3.x, où elle est en `str` (mesuré §5.10 : 7 colonnes `object` sous 2.2.3, 0 sous 3.0.6, **même fichier**). Remède : la logique sur la **valeur** (convertir, comparer, compter) — le `dtype` s'affiche, il ne commande pas.
6. **Attendez-vous à des manquants... et n'en trouver aucun.** L'audit `isna` renvoie 0 (fait du socle, mesuré) — et c'est **normal**, pas un bug : ce socle a été nettoyé en amont (M05). Le réflexe à garder est l'audit lui-même sur votre fichier du lundi, le 0 du socle n'est pas promis.

9. Bonnes pratiques professionnelles

1. **dtypes juste après chaque read_csv** — la première ligne de lecture d'une table inconnue (comme le `DESCRIBE` de M07) ; c'est là que le `str` d'une date, ou le `float` d'un identifiant, se voient.
2. **loc par défaut, iloc en exception** — les noms résistent aux réordres de colonnes ; les positions, non. Une seule des deux par script.
3. **Parenthèses sur chaque condition, len() après chaque filtre** — le bug silencieux (§5.5) est le seul qui ne crie pas ; le contrôle de taille est le chien de garde.
4. **Réassigner, jamais inplace** — `s = s.fillna(0)`, `v = v.drop(...)` : l'écriture unique validée sur 2.3 et 3.x (§5.10).
5. **L'audit (isna, duplicated, formes) précède l'agrégat** — sur ce socle 0 manquant, 8 doublons, 1 table vide (mesuré, §6) ; sur votre jeu du lundi, ce sont **vos** chiffres qui sortent — publiez-les dans le rapport, comme M05.

10. Exercice guidé — « la vue des 120 premières ventes » (15 min, /10)

Énoncé. Écrire `vue120.py` qui, sur les **120 premières lignes** réelles de `vente.csv` (`v = pd.read_csv(...).head(120)` — la vue `m08p_vue_lignes` du socle) :

1. calcule la **médiane** de `montant_ttc` (2 décimales) ;
2. compte les montants **> 100 000** (le masque, pas la boucle) ;
3. crée la colonne `montant_ht` et affiche son **maximum** (2 décimales) ;
4. affiche le **comptage des manquants** (`isna().sum().sum()`).

Barème. La vue 120 chargée correctement (2 pts) · les 4 calculs avec les outils du chapitre (`median`, masque `.sum()`, colonne vectorisée, `isna`) (4 pts) · l'affichage 2 décimales en f-string (2 pts) · la sortie alignée sur le modèle ci-dessous (2 pts). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```

Mediane           : 122,065.68
Gros > 100 000: 78
Montant HT max: 471,617.12
Manquants         : 0

```

11. Exercices autonomes

Exercice 7.1 — La carte d’une colonne. `describe()` sur `montant_remise` : afficher les 8 statistiques, et dire en une phrase pourquoi le quartile 25 % est 0. (10 min)

Exercice 7.2 — Les 4 portes. Sur `v.head(5)` réduit à 4 colonnes (`id_vente`, `id_magasin`, `montant_ttc`, `est_retour`) : atteindre la cellule (ligne 1, `montant_ttc`) par les 4 chemins (`loc/iloc` × ligne/colonne) — les 4 sorties doivent être identiques. (10 min)

Exercice 7.3 — Le tamis à trois conditions. Compter les ventes `est_retour == True` **et** `montant_ttc < 50000` **et** `taux_tva == 0.18` — masque à 3 conditions, parenthèses comprises ; afficher aussi le comptage avec **une** parenthèse manquante, pour voir l’écart. (15 min)

Exercice 7.4 — Les villes. Sur `client.csv` : le nombre de **villes distinctes**, la **modale** (`mode()`), et le maximum de `plafond_credit`. (10 min)

Exercice 7.5 — L’aller-retour. `to_excel` des 50 premières ventes dans `/tmp/c07/aperçu50.xlsx`, rechargement par `read_excel`, affichage de la forme et de **deux** dtypes — vérifier l’aller-retour sans perte. (10 min)

12. Correction détaillée

Exercice guidé. Le script :

```
import pandas as pd

v = pd.read_csv("03_exercices/dossier_M08/vente.csv").head(120)
s = v["montant_ttc"]
v["montant_ht"] = s / (1 + v["taux_tva"])
print(f"Mediane      : {s.median():.2f}")
print(f"Gros > 100 000: {(s > 100000).sum()}")
print(f"Montant HT max: {v['montant_ht'].max():.2f}")
print(f"Manquants     : {v.isna().sum().sum()}")
```

Les 4 lignes publiées **sont** la sortie de l’atelier (122,065.68 / 78 / 471,617.12 / 0). Le contrôle à faire en tête : médiane = `s.quantile(0.5)` — les deux doivent redonner le même nombre.

Exercice 7.1. La sortie réelle :

```
count    50008.000000
mean      1017.110771
std       1571.906376
min         0.000000
25%        0.000000
50%       173.815000
75%       1508.530000
max       8915.440000
```

La phrase : **la moitié et plus des ventes n’a pas de remise** (le quartile 25 % est 0, comme le minimum — la remise est une option, pas la règle ; la médiane 173,82 le confirme).

Exercice 7.2. Les 4 chemins : `v4.loc[1, "montant_ttc"]` → 359109.8 · `v4.iloc[1, 2]` → 359109.8 · (et pour la colonne : `v4.loc[:, "id_magasin"]` et `v4.iloc[:, 1]` → [2, 1, 2, 3, 4]) — sortie réelle du §5.4 : les 4 portes mènent à la même pièce.

Exercice 7.3. Comptage correct (masque à 3 conditions) : **26** retours sous 50 000 avec taux 18 % (sortie réelle) ; la variante 2 conditions (`est_retour & (montant_ttc < 50000)`) en donne **58** — la 3^e condition affine. L’écriture sans parenthèses, elle, lève un `TypeError` (sortie réelle : « Cannot perform ‘rand_’ with a dtypes [float64] array and scalar of type [bool] ») : la même oubli produit un **crash** ou le **silencieux 50 008** du §5.5, selon la forme — aucun des deux n’est « presque bon » : c’est l’écart qui est le résultat de l’exercice, pas le comptage.

Exercice 7.4. Sortie réelle : **3** villes distinctes, modale ['Ouagadougou'], plafond maximum **un million** de FCFA (la valeur sort dans votre terminal : 1000000).

Exercice 7.5. `read_excel` renvoie (50, 13) avec les mêmes dtypes que le CSV (`id_vente` int64, `est_retour` bool, `date_vente` str) — l'aller-retour sans perte, comme l'Étape du §5.9.

13. Mini-projet M08.P7 — « L'audit du dossier » (1 h)

Énoncé. Écrire `audit.py` — le script qui répond au brief de `00_brief.md` :

1. importer les **8 CSV** du dossier (boucle sur les noms, pas 8 fois le même `read_csv`) ;
2. afficher les **formes** des 8 (dictionnaire nom → nombre de lignes) ;
3. sur `vente` : les **doublons** (hors `id_vente`), les **retours**, les **dates** `date_vente > date_limite_remise` (via `pd.to_datetime`), le **comptage des manquants** ;
4. signaler la **table vide** (0 ligne) par son nom ;
5. afficher, en dernier, la **médiane** et le **95^e percentile** de `montant_ttc` (2 décimales).

Critères de réussite (grille /10) : les 5 exigences (5 × 1 pt) · la sortie de l'atelier : formes 50008/1200/380/8/5/5/2/0 · doublons 8 · retours 208 · dates inversées 21 406 · manquants 0 · table vide `objectif_magasin` · médiane 119481.89 · p95 431647.88 (1 pt) · un seul `read_csv` par fichier, aucun `for` sur les lignes (1 pt) · le dtypes de `vente` affiché en tête (1 pt) · le rapport lisible par un humain (1 pt).

Note de correction. Ce projet **est** l'exercice fil rouge du plan, transformé en livrable : les 5 constats du §6, mais **écrits une fois, dans votre script**. La valeur ajoutée professionnelle n'est pas le code (15 lignes), c'est le **rapport** : 5 chiffres, 5 phrases, 0 « il semble que ». Si un chiffre de votre sortie ne correspond pas au §6, c'est votre script qui a tort — pas le socle (les clés `m08p_*` tranchent).

14. Boîte à outils du chapitre

Boîte à outils. Le venv de C01 (`/tmp/venv_m08`, pandas 3.0.6) + le dossier `03_exercices/dossier_M08/` — la boîte du chapitre s'ajoute à celles de C02-C06 : le classeur d'indexation.

Outil	Usage	Syntaxe
<code>pd.read_csv / read_excel</code>	l'import (porte d'entrée / de sortie)	<code>pd.read_csv(f)</code>
<code>head / shape / dtypes / info</code>	la première lecture	<code>v.dtypes</code>
<code>loc / iloc</code>	la sélection (nom / position)	<code>v.loc[1, "col"], v.iloc[1, 2]</code>
masque booléen	le WHERE	<code>v[(v["a"] > 1) & (v["b"] == 2)]</code>
<code>df["col"] = opération / assign</code>	la nouvelle colonne	<code>v["mt_ht"] = v["mt"] / (1 + v["t"])</code>
<code>describe()</code>	les 8 statistiques	<code>s.describe()</code>
<code>isna()</code> / <code>dropna()</code> / <code>fillna()</code> / <code>ffill()</code>	les manquants	<code>s.isna().sum()</code>
<code>duplicated(subset=...)</code>	les doublons réels	<code>v.duplicated(subset=...).sum()</code>
<code>to_excel / read_excel</code>	l'aller-retour Excel	<code>v.to_excel(f, index=False)</code>

15. Résumé du chapitre

- Le **DataFrame** est la table SQL de M07 en Python : colonnes nommées, dtypes, index-ré-
dure ; la **Série** est sa colonne (le tableau de C06 avec étiquettes) — `s.sum()` redonne 7 908
259 732,20, le `np.sum` de C06.
- **loc** (nom, borne incluse) vs **iloc** (position, borne exclue) : le tableau des 4 cas, et le piège
des tranches (3 lignes contre 2, mesuré).
- Les **filtres** sont des **masques** (True/False par ligne) : `&/|/~` et **parenthèses obligatoires** —
sans, le bug silencieux sélectionne 50 008 au lieu de 27 938, sans crier.
- La **nouvelle colonne** est vectorisée (`montant_ht = montant_ttc / (1 + taux_tva)`) — le `SELECT`
... AS ; assign enchaîne sans modifier.
- **describe()** : les 8 statistiques de M02 — mêmes quartiles que C06, std échantillonnaire
(n-1) là où NumPy faisait population (n).
- Les **manquants** : l'audit du socle en renvoie **0** (mesuré) ; la table `objectif_magasin` est **vide**
(0 × 4), `KeyError` sur le nom approximatif — `isna/dropna/fillna/ffill` sont la boîte pour
quand le fichier du lundi en a.
- Les **4 écritures obsolètes** (1^{re} salve des 12) : `append` (supprimée), `inplace` (deux comporte-
ments selon la version), l'affectation chaînée (changement **perdu**, écart 0,0), `fill-
na(method=...)` (supprimée) — et les **4 écarts 2.2.3/3.0.6 mesurés** (103 → 85, 373 → 372, 18 →
0, 7 → 0, `object` → `str`) : la version change la vérité terrain.

16. À retenir

À retenir. Un filtre pandas **répond toujours** — avec le bon chiffre ou le mauvais, **sans crier** :
`len()` après chaque sélection, parenthèses sur chaque condition, et `loc/iloc` ne se mêlent pas
(3 lignes contre 2, même écriture). Le chiffre de pandas se **vérifie** par sa taille, pas par sa poli-
tesse.

À retenir. La **version fait partie du résultat** : même fichier, même code, pandas 2.2.3 et 3.0.6
ne comptent pas les mêmes cellules vides (103 → 85, mesuré). On ne branche donc **jamais** sur le
`dtype` — on branche sur la **valeur** ; et l'audit (formes, doublons, manquants, tables vides) pré-
cède **tout** agrégat.

17. Évaluation formative (auto-correction, 8 min)

1. `v.loc[1:3]` et `v.iloc[1:3]` — **combien de lignes chacune, et pourquoi ?** → 3 lignes et 2
lignes : `loc` sélectionne par **étiquettes** (borne haute incluse), `iloc` par **positions** (borne
haute exclue) — même écriture, une ligne de moins (§5.4). (2 pts)
2. **Que renvoie `v["montant_ttc"] > 100000 & v["est_retour"]` (sans parenthèses), et quel est
le danger ?** → Un masque **silencieux** qui sélectionne la table entière (50 008 au lieu de 27
938 — mesuré) : `&` se lie avant `>`, donc comparaison à 100000 & `est_retour` ; aucun avertisse-
ment — d'où le `len()` après chaque filtre. (2 pts)
3. `v.duplicated().sum()` **renvoie 0 sur la table du socle. Y a-t-il des doublons ? Que faire ?**
→ Oui : **8** (`m08p_doublons`) — mais `id_vente` est unique, donc « toutes colonnes » ne duplique
rien ; il faut `duplicated(subset=...)` sans `id_vente` (le §6 le mesure). (2 pts)
4. `v.loc[0, "montant_ttc"]` et `v.iloc[0, 10]` — **que ont-ils de commun ?** → La **même cellule**
(213566.20), atteinte par **nom** (étiquettes) ou par **position** (numéros) — les deux portes de
la même pièce ; `montant_ttc` est la 11^e colonne (index 10). (1 pt)

5. **Pourquoi std vaut-il 133 553,73 dans describe() et 133 552,39 avec np.std (C06) ?** → pandas calcule l'écart-type **échantillonnaire** (division par $n-1$), NumPy l'écart-type **population** (division par n) — le détail à vérifier avant de comparer deux sorties (§5.7). (1 pt)
6. **objectif_magasin.csv a 0 ligne. Que fait read_csv, que renvoie o["ca_objectif_ttc"].sum(), et que lève o["ca_objectif"] ?** → Un DataFrame 0×4 (l'en-tête donne les noms, colonnes object) ; la somme renvoie **0** sans broncher ; le nom approximatif lève `KeyError: 'ca_objectif'` (§5.8). (1 pt)
7. **Sous pandas 3.0.6, s.fillna(0, inplace=True) renvoie quoi, et sous 2.2.3 ?** → Sous 3.0.6 : **la série remplie** (et la série est modifiée) ; sous 2.2.3 : **None** — la même ligne, deux comportements ; l'écriture validée sur les deux versions est `s = s.fillna(0)` (§5.10). (1 pt)
8. **Les 4 écarts mesurés 2.2.3/3.0.6 du fichier « attendu » de M03 — les citer, et dire ce qu'ils prouvent.** → cellules vides $103 \rightarrow 85$, modalités $373 \rightarrow 372$, lignes champ vide $18 \rightarrow 0$, colonnes texte object $7 \rightarrow 0$ (et object $\rightarrow$ str) : la **version change la vérité terrain** sans protester — d'où « brancher sur la valeur, pas sur le dtype » et l'audit avant tout agrégat (E4 du module). (1 pt)

Module M08.C08 — pandas II : `groupby/transform/merge, pivot_table, dates, apply, export Excel` — et l'annexe migration 2.3 → 3.x

Outils : le venv de C01 (`/tmp/venv_m08`, pandas 3.0.6 + `matplotlib` ajouté pour les 4 graphiques du projet) + l'environnement d'écriture du manuel (pandas 2.2.3) pour l'annexe. Le dossier `03_exercices/dossier_M08/`. **Durée indicative :** 3 h hors exercices. **Niveau :** N6 → N7. **Prérequis :** C07 (la table en mémoire), M07 (toutes les requêtes SQL qui sont traduites ici).

L'idée du chapitre. M07 a tout fait en SQL : `GROUP BY`, `JOIN`, la vue mensuelle, `EXTRACT`. C08 refait **exactement le même travail** en pandas — `groupby/agg`, `transform`, `merge`, `pivot_table`, `to_datetime/resample` — et le fil rouge en est la **preuve** : les 30 questions du projet M07.P, réécrites en pandas, dont 10 exécutées et comparées au corrigé SQL (**0 écart** sur les comptages, **± 2 FCFA** sur les totaux — la dérive mesurée et documentée). Le chapitre se termine par le **script de bout en bout** du projet M08.P (`import` → `audit` → `nettoyage` → `indicateurs` → 4 graphiques → classeur, exécuté d'une traite) et par l'**annexe migration** : les 12 écritures obsolètes avec le code avant/après — « votre script de 2026 tournera en 2028 ».

Matériel de l'atelier. Tout a été exécuté le 19/09/2026 dans le venv pandas 3.0.6 (et l'annexe comparée sous 2.2.3). La règle monétaire du module tient en une phrase : **les totaux s'additionnent en centimes entiers** — c'est la leçon du §5.8, elle explique le « ± 2 FCFA » que vous allez voir partout. Les repères canoniques : CA brut 7 908 259 732 (`m08p_ca_brut`), sans retours 7 876 320 164, panier 158 140, top magasin 4 = 1 596 813 264, plus gros mois 78 965 529.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. Agréger avec `groupby` + `agg` : le `GROUP BY` de M07 en une ligne (les 5 magasins, CA 1 564 937 883 à 1 596 813 264, top magasin 4).
2. Remettre l'agrégat **sur chaque ligne** avec `transform` (le « delta vs la moyenne de son magasin » de C06, en une ligne — ce que `GROUP BY` ne fait pas).
3. Joindre avec `merge` : le `JOIN` de M07, `how="left"` (zéro ligne perdue) — et la table **vide** `objectif_magasin` qui rend toute la colonne en `NaN` (mesuré : 50 008).
4. Reconstruire la **vue** `v_ca_mensuel_magasin` de M07 avec `pivot/ groupby` : 120 lignes, plus gros mois 78 965 529 — **sans SQL**.
5. Manipuler les **dates** : `pd.to_datetime`, `dt.year/dt.month`, `resample("ME")` — et l'**alias 'M' est mort** (le `ValueError` réel).
6. Savoir **quand apply est légitime** — et le mesurer : la fonction de C05 sur une colonne coûte 4× la version vectorisée (mesuré), même résultat.
7. Exporter un **classeur Excel multi-feuilles** (`ExcelWriter`, feuilles nommées, largeurs) — le livrable E3 du projet.
8. Lire l'**annexe migration 2.3 → 3.x** : les 12 écritures obsolètes (avant/après) et les 4 écarts mesurés — la garantie « votre script de 2026 tournera en 2028 ».

2. Pourquoi cette notion est importante

Parce que la transition est posée : « SQL vous donne le chiffre. Python vous donne la méthode, la vitesse, et la capacité de refaire tout cela en une seconde le mois prochain. » — C08 est la preuve : les 30 questions de la direction (M07.P), la vue mensuelle, le classeur de synthèse — tout est **rejoué** en pandas, et le mois prochain le script tourne seul. Le SQL n'est pas remplacé, il est **complété** : le SGBD garde le rôle du moteur sur les jeux massifs, le script pandas prend celui de la méthode réutilisable, versionnable, commentée.

Parce que **le float a une mémoire** — et qu'elle s'appelle « centimes ». Les totaux de ce chapitre s'additionnent en **centimes entiers** (int64) : le float64 dérive de 1 à 2 FCFA sur 7,9 milliards (mesuré, \$5.8) — négligeable en métier, **visible en audit**. La règle du module : comptages **exacts**, totaux ± 2 FCFA, dérive **documentée** dans le rapport — c'est ce que votre classeur E3 devra afficher, pas ce qu'il faudra cacher.

Parce que **les versions changent sous vos pieds** : les 4 écritures vues en C07 (append, inplace, chaînée, fillna(method=...)) ne sont que la 1^{re} salve — l'annexe du chapitre présente les **12** avec le code avant/après, et les 4 écarts de vérité terrain (103 → 85, 373 → 372, 18 → 0, 7 → 0) déjà mesurés en C07 §5.10. Un script qui branche sur la valeur plutôt que sur le dtype survit à la migration ; l'autre meurt silencieusement.

3. Explication simple — les classeurs et le classement

- **groupby** est le **classement en dossiers** : on trie les 50 008 fiches par magasin (5 dossiers), puis on pose la question **par dossier** (la somme, le compte, la moyenne) — le **GROUP BY** de M07, mais les dossiers sont en mémoire.
- **transform** est l'étiquette qu'on **recolle sur chaque fiche** : après avoir calculé la moyenne de son dossier, on la remet sur chacune des fiches du dossier — **GROUP BY** renvoie un résumé (5 lignes), **transform** renvoie la table d'origine (50 008 lignes) avec la valeur du groupe en plus.
- **merge** est la **correspondance de fiches** : deux classeurs, on les aligne sur le numéro de référence (id_magasin) — chaque fiche garde ses voisines, et si l'autre classeur est vide... toutes les cases opposées restent **vides** (NaN).
- **pivot_table** est le **tableau tourné** : les lignes deviennent des colonnes (ou l'inverse) — la vue « magasin × mois » de M07, en un geste.
- **resample** est le **classeur par mois** : on range chaque fiche dans son mois, puis on agrège — **dt.year/dt.month** sont les rubriques, **resample("ME")** est le tri par fin de mois.
- **apply** est l'**ascenseur** : on envoie la fonction fiche par fiche (c'est le **for** déguisé) — pratique quand la fiche a besoin d'un coup de pouce individuel (du texte « N/C », une décision par valeur), **lourd** quand un convoi vectorisé passerait à pleine vitesse.

4. Vocabulaire essentiel

Français — English	Définition en une phrase	Piège à éviter
regroupement groupby	— la méthode qui trie les lignes par clé (un groupe par valeur de clé), avant l'agrégat.	croire qu'on agrège « tout d'un coup » : groupby renvoie un objet qu'il faut ensuite agréger (agg, sum...).
agrégation — agg	l'opération par groupe (sum, mean, count...) — la réduction d'un groupe à une valeur.	agréger des float64 pour un total monétaire : additionner en centimes entiers (\$5.8).
transformation transform	— remettre la valeur du groupe sur chaque ligne du groupe (la table garde sa taille).	confondre avec agg : agg réduit (5 lignes), transform ne réduit pas (50 008 lignes).

Français — English	Définition en une phrase	Piège à éviter
jointure — merge	l'alignement de deux DataFrames sur une clé commune — le JOIN de M07.	un merge sans clé explicite : il joint sur l'intersection des index — souvent au pire endroit.
table pivotée — pivot_table	la réorganisation lignes ↔ colonnes autour d'une clé et d'une valeur.	oublier la définition de la vue (sans retours, ici) : changer le filtre change le nombre.
rééchantillonnage — resample	le regroupement par période de dates (mois, trimestre) après un index daté.	l'alias 'M' est mort en 3.x : écrire 'ME' (le ValueError du \$5.5).

5. Cours approfondi

5.1 groupby + agg — le GROUP BY en une ligne

GROUP BY SQL et groupby pandas : la même chaîne de calcul

Deux syntaxes, une même sortie — les 5 magasins du socle, additionnés en centimes.

SQL — le moteur M07

```
SELECT id_magasin,
SUM(montant_ttc)
FROM vente
GROUP BY id_magasin
ORDER BY 2 DESC;
```

pandas — C08, en centimes entiers

```
ct = (v["montant_ttc"] * 100)
.round().astype("int64")
ct.groupby(v["id_magasin"])
.sum() // 100
```

la même sortie — CA par magasin (FCFA)

id	in	
4		1 596 813 264
1		1 595 223 423
2		1 578 987 709
5		1 572 297 451
3		1 564 937 883

top magasin : 4 — 1 596 813 264 FCFA (m08p_ca_top_magasin)

Les centimes : additionner en entiers, diviser après l'agrégat —

la dérive float (1 à 2 FCFA sur 7,9 milliards) est mesurée et documentée, pas cachée.

```
import pandas as pd

v = pd.read_csv("03_exercices/dossier_M08/vente.csv")
ct = (v["montant_ttc"] * 100).round().astype("int64") # les centimes ($5.8)
g = v.groupby("id_magasin")["montant_ttc"].agg(["sum", "count", "mean"])
print(g.to_string())
print()
print((ct.groupby(v["id_magasin"]).sum() // 100).to_string())
```

```

      sum  count      mean
id_magasin
1      1.595223e+09  10099  157958.552683
2      1.578988e+09  10033  157379.418820
3      1.564938e+09   9873  158506.825040
4      1.596813e+09  10061  158713.176035
5      1.572297e+09   9942  158146.997779

id_magasin
1      1595223423
2      1578987709
3      1564937883
4      1596813264
5      1572297451

```

pandas (§5.1)

```
v.groupby("id_magasin")
["montant_ttc"].agg([...])
```

SQL (M07, Q07)

```
SELECT id_magasin, ROUND(SUM(montant_ttc),0), COUNT(*),
AVG(montant_ttc) FROM vente GROUP BY id_magasin
```

Cinq groupes, chacun sa somme, son compte (10 099 + 10 033 + 9 873 + 10 061 + 9 942 = 50 008 — le contrôle de taille du C07), sa moyenne. Le **top magasin est le 4** (1 596 813 264 FCFA, `m08p_ca_top_magasin`) ; le plus petit est le 3 (1 564 937 883, `m08p_ca_min_magasin`). La 2^e sortie est la même somme, **additionnée en centimes** — la colonne de référence monétaire du chapitre.

5.2 transform — la moyenne du groupe, remise sur chaque ligne

```

v["delta"] = v["montant_ttc"] - v.groupby("id_magasin")["montant_ttc"].transform("mean")
print(v.head(3)[["id_magasin", "montant_ttc", "delta"]].to_string())
print("delta moyen magasin 1 :", v.loc[v["id_magasin"] == 1, "delta"].mean())

```

```

      id_magasin  montant_ttc      delta
0             2    213566.20  56186.781180
1             1    359109.80  201151.247317
2             2     85939.05  -71440.368820
delta moyen magasin 1 : -8.299735787245575e-12

```

Le **delta de chaque vente à la moyenne de son magasin** — en une ligne. Les trois valeurs de la sortie **sont** celles de C06 §5.6 (le `moyennes[ids - 1]` du broadcasting, calculé à la main) : `transform` fait le même travail sans alignement manuel, et la table garde ses 50 008 lignes (la colonne `delta` est une **colonne** comme une autre). Le contrôle de la dernière ligne redonne ≈ 0 (-8×10^{-12} , le bruit `float`) : si le delta moyen d'un groupe n'est pas zéro, la moyenne du groupe n'est pas celle du groupe.

Définition. transformation — `transform` — l'opération qui calcule une valeur **par groupe** et la **repose sur chaque ligne** du groupe : la table de départ garde sa forme (50 008 × 14), chaque ligne portant l'agrégat de son groupe. C'est l'outil des **comparaisons relatives** (« cette vente vs la moyenne de son magasin »), que ni `agg` ni le `GROUP BY` ne font.

5.3 merge — le JOIN de M07, et la table vide qui rend tout NaN

```

m = pd.read_csv("03_exercices/dossier_M08/magasin.csv")
o = pd.read_csv("03_exercices/dossier_M08/objectif_magasin.csv")
vm = v.merge(m, on="id_magasin")
print(vm.shape)
vo = v.merge(o, on="id_magasin", how="left")
print(vo.shape, "| NaN ca_objectif_ttc :", int(vo["ca_objectif_ttc"].isna().sum()))

```

```
(50008, 18)
(50008, 16) | NaN ca_objectif_ttc : 50008
```

pandas (§5.3)**SQL (M07, C07)**

```
v.merge(m, on="id_magasin")
```

```
SELECT v.*, m.nom FROM vente v JOIN magasin m ON v.id_magasin =
m.id_magasin
```

```
v.merge(o, on="id_magasin", how="left")
```

```
... LEFT JOIN objectif_magasin ... (la table vide)
```

Définition. jointure — merge — l'alignement de **deux DataFrames** sur une **clé commune explicite** (`on="id_magasin"`), avec le mode de conservat des lignes (`how=`) : le JOIN de M07, où la clé est une information **métier** et non un détail de syntaxe.

Attention. Une colonne **entière en NaN** après un merge est un **signal** à lire, pas un résultat : la table opposée est vide (ici `objectif_magasin`), ou la clé ne correspond pas — on l'inspecte avant d'exporter.

merge est le JOIN en nom : 13 + 5 colonnes = **18**, zéro ligne perdue (chaque `id_magasin` de vente existe dans `magasin`). Le 2^e exemple est le **piège du dossier** : `objectif_magasin` est **vide** (C07 §5.8), et la LEFT JOIN renvoie 50 008 lignes... dont **toutes** les cases `ca_objectif_ttc` sont NaN (50 008, mesuré) — la table est jointe, le LEFT sauve les lignes, mais il n'y a **rien** à rejoindre. Un merge « correct » qui produit une colonne entière de NaN est un signal à lire : la table opposée est vide, ou la clé ne correspond pas.

5.4 pivot — la vue mensuelle de M07, reconstruite sans SQL

La vue `v_ca_mensuel_magasin` de M07 est définie **sans retours** (`WHERE est_retour = FALSE`) — c'est la définition de la vue, pas un choix :

```
vs = v.loc[~v["est_retour"]]
vue = pd.DataFrame({
    "id_magasin": vs["id_magasin"],
    "annee": pd.to_datetime(vs["date_vente"]).dt.year,
    "mois": pd.to_datetime(vs["date_vente"]).dt.month,
    "ca_cents": ct[vs.index],
}).groupby(["id_magasin", "annee", "mois", as_index=False)["ca_cents"].sum()
vue["ca"] = vue["ca_cents"] // 100
print("lignes vue :", len(vue))
print(vue.sort_values("ca", ascending=False).head(3).to_string(index=False))
```

```
lignes vue : 120
id_magasin annee mois ca_cents ca
3 2025 8 7896552921 78965529
1 2025 3 7743800620 77438006
4 2025 2 7640325426 76403254
```

Définition. table pivotée — `pivot_table` — la réorganisation **lignes ↔ colonnes** d'une table autour d'une clé (ici `magasin × mois`) et d'une valeur agrégée (le CA) : la vue « tableau croisé » de M07, en un geste pandas.

Attention. Une vue a une **définition** — la vue M07 est définie **sans retours** (`WHERE est_retour = FALSE`) : changer le filtre change le **nombre**, pas les données (l'écart est mesuré en §8, n° 2). Avant un pivot, dire quelles lignes la vue contient — la question est un cinquième du travail.

120 lignes (`m08p_vue_lignes` : 5 magasins × 24 mois) — la vue de M07 est **reconstruite**, sans SQL — et le plus gros mois est le ****2025-08** du magasin 3

78 965 529 FCFA (`m08p_plus_gros_mois_ca`) — le même nombre que le `SELECT` de M07, au FCFA près. Remarquez l'ordre des opérations : on somme en centimes (`ca_cents`), on divise après** — l'erreur du §8 (n° 1) est de diviser d'abord.

5.5 Les dates — l'index qui porte un sens, et l'alias 'M' mort

```
dv = pd.to_datetime(v["date_vente"])
print("2025 :", int((dv.dt.year == 2025).sum()), "ventes | CA centimes :",
      ct[dv.dt.year == 2025].sum() // 100, "| CA float :", v.loc[dv.dt.year == 2025,
"montant_ttc"].sum())
s = pd.Series(ct.to_numpy() // 100, index=dv)
r = s.resample("ME").sum()
print("resample ME :", len(r), "mois | plus fort :", r.idxmax().date(), int(r.max()))
try:
    s.resample("M").sum()
except Exception as e:
    print(type(e).__name__)
    print(e)
```

```
2025 : 24920 ventes | CA centimes : 3925215672 | CA float : 3925215672.71
resample ME : 24 mois | plus fort : 2026-06-30 350948588
ValueError
Invalid frequency: M. Failed to parse with error message: ValueError("'M' is no longer supported
for offsets. Please use 'ME' instead.)")
```

`pd.to_datetime` transforme les `str` de C07 en **vraies dates** : `dt.year`, `dt.month`, `dt.dayofweek` (attention : 0 = lundi, 6 = **dimanche** — le DOW de DuckDB a 0 = dimanche, c'est l'inverse, le §8 n° 6) sont les `EXTRACT` de M07. 2025 : **24 920 ventes** (`m08p_nb_ventes_2025`), CA 3 925 215 672 en centimes (`m08p_ca_2025`) contre 3 925 215 672,71 en `float` — la dérive de 0,71 FCFA, le §5.8 la documente. `resample("ME")` range chaque vente dans son mois (24 mois), et l'alias **'M' est mort** : le `ValueError` cite la solution (`'ME'`) — le message d'erreur **est** la migration, l'annexe du §5.9 l'enseigne pour les 12 écritures.

5.6 `apply` — la fonction de C05 sur une colonne, et son prix mesuré

```
import time
import numpy as np

def classe_montant(x):
    if x > 100000:
        return "gros"
    if x > 10000:
        return "moyen"
    return "petit"

t0 = time.perf_counter()
r1 = v["montant_ttc"].apply(classe_montant)
t1 = time.perf_counter()
t2 = time.perf_counter()
r2 = np.where(v["montant_ttc"] > 100000, "gros",
              np.where(v["montant_ttc"] > 10000, "moyen", "petit"))
t3 = time.perf_counter()
print("apply      :", f"{{(t1 - t0) * 1000:.1f}} ms")
print("vecteur    :", f"{{(t3 - t2) * 1000:.2f}} ms")
print("rapport    :", f"{{(t1 - t0) / (t3 - t2):.0f}}x")
print("identiques :", bool((r1 == r2).all()))
```

```
apply      : 8.3 ms
vecteur    : 2.16 ms
rapport    : 4x
identiques : True
```

C'est la **fonction de C05** (un `if/elif/else` par valeur) appliquée aux 50 008 montants : `apply` la fait **fiche par fiche** (l'ascenseur du §3) — 8,3 ms — là où `np.where` la fait **en convoi** (2,16 ms) : **4×**, même résultat (identiques : `True`). La règle du chapitre : **vectoriser d'abord**, `apply` quand la valeur a besoin d'une **logique individuelle** que le convoi ne fait pas (du texte « N/C » à nettoyer, une décision par cas). Et quand on l'utilise, on **mesure** — 4× sur 50 000, ce sera 40× sur 5 millions.

5.7 L'export Excel — le classeur multi-feuilles, le livrable E3

```
audit = pd.DataFrame({"controle": ["formes", "doublons", "retours", "dates inverses",
                                "manquants"],
                    "valeur": [len(v), 8, 208, 21406, 0]})
with pd.ExcelWriter("/tmp/c07/synthese.xlsx", engine="openpyxl") as w:
    vue.drop(columns=["ca_cents"]).to_excel(w, sheet_name="ca_mensuel", index=False)
    audit.to_excel(w, sheet_name="audit", index=False)
    w.sheets["ca_mensuel"].column_dimensions["A"].width = 12
print({k: v.shape for k, v in pd.read_excel("/tmp/c07/synthese.xlsx", sheet_name=None).items()})
```

```
{'ca_mensuel': (120, 5), 'audit': (5, 2)}
```

Le `with pd.ExcelWriter(...)` est le **classeur** : chaque `to_excel(w, sheet_name=...)` est une **feuille nommée** (`ca_mensuel`, `audit`), et les paramètres `openpyxl` (`column_dimensions[...].width`) font la **mise en forme minimale** (largeurs, et les formats de nombres pour les colonnes monétaires). Le rechargement `sheet_name=None` renvoie **toutes** les feuilles (dictionnaire nom → `DataFrame`) : 120×5 et 5×2 , l'aller-retour sans perte — le CSV reste la porte d'entrée, l'Excel le **format de livraison** (M05), et E3 exige les comptages **identiques** au corrigé M07.P et les totaux **± 2 FCFA**, la dérive **documentée dans le rapport**.

Conseil professionnel. Un classeur de synthèse se juge à trois choses : les **feuilles nommées** (on les rechargera par nom), les **comptages identiques** au corrigé de référence, et les **totaux ± 2 FCFA documentés** — la dérive float n'est pas cachée dans le classeur, elle est **écrite dans le rapport d'audit** : c'est la différence entre un livrable et un fichier.

5.8 La leçon des centimes — 1 à 2 FCFA sur 7,9 milliards

```
print("float   :", v["montant_ttc"].sum())
print("centimes:", ct.sum() // 100)
```

```
float   : 7908259732.2
centimes: 7908259732
```

Addition	Résultat	Écart au Decimal exact (7 908 259 732,20)
Decimal exact (le vrai total)	7 908 259 732,20	—
DuckDB SUM (float, M07)	7 908 259 730,56 (publié arrondi : 731)	≈ 1,6 FCFA
pandas float64 (ci-dessus)	7 908 259 732,20	0
pandas centimes entiers (ci-dessus)	7 908 259 732	≤ 1 FCFA (l'arrondi)

Définition. addition en centimes — cent-based summation — la conversion des montants en **centimes entiers** (`(colonne * 100).round().astype("int64")`) avant toute somme : le centime est le plus petit multiple **exact** de FCFA, l'entier ne dérive pas, et le total final se convertit en FCFA par une seule division (`// 100`).

`montant_ttc` est un **DOUBLE** (C07, `dtypes`) : 50 008 additions de doubles cumulent l'erreur de représentation — 0,71 FCFA sur le CA 2025, 0,51 sur le CA 2026 (mesuré, \$5.5), 1,64 sur le total (DuckDB, M07). **Négligeable en métier, visible en audit** — et c'est **pour ça** que la vérification croisée SQL/pandas du module est **exacte sur les comptages** et à **± 2 FCFA sur les totaux monétaires** (écarts mesurés, sourcés `m08p_ecart_somme_float_*`) : la tolérance n'est pas une faute de goût, c'est la **physique des floats**, mesurée et documentée — comme E3 l'exige dans le rapport d'audit.

5.9 L'annexe migration 2.3 → 3.x — les 12 écritures, avant/après

Les 4 **1^{re} save** sont démontrées en C07 §5.10 (sorties réelles) ; les 8 restantes, exécutées ici sous pandas 3.0.6 :

#	Écriture (2.x)	Statut 3.0.6 (sortie réelle)	Écriture enseignée
1	<code>df.append(ligne)</code>	<code>AttributeError</code> (C07 §5.10)	<code>pd.concat([df, ligne])</code>
2	<code>inplace=True</code>	deux comportements (C07 §5.10)	réassigner : <code>s = s.fillna(0)</code>
3	<code>df[masque]["col"] = v</code>	changement perdu (C07 §5.10)	<code>df.loc[masque, "col"] = v</code>
4	<code>resample("M") / alias 'H', 'S'</code>	<code>ValueError : 'M' is no longer supported... use 'ME' (\$5.5)</code>	"ME", "h", "s"
5	<code>Series.iteritems()</code>	<code>AttributeError: 'Series' object has no attribute 'iteritems'</code>	<code>Series.items()</code>
6	<code>DataFrame.applymap(f)</code>	<code>AttributeError: 'DataFrame' object has no attribute 'applymap'</code>	<code>DataFrame.map(f)</code>
7	<code>fillna(method="ffill")</code>	<code>TypeError</code> (C07 §5.10)	<code>s.ffill() / s.bfill()</code>
8	<code>reindex_axis / swapaxes</code>	<code>AttributeError: 'DataFrame' object has no attribute 'reindex_axis'</code>	<code>reindex / transpose</code>
9	<code>df.mad()</code>	<code>AttributeError: 'DataFrame' object has no attribute 'mad'</code>	<code>(x - x.mean()).abs().mean()</code> (NumPy)
10	compter une cellule texte vide comme non-vidé	les 4 écarts mesurés (C07 §5.10 : 103 → 85, 373 → 372, 18 → 0, 7 → 0)	normaliser avant de compter
11	brancher sur le dtype object	le texte est en str (mesuré C07 §5.2/ §5.10)	brancher sur la valeur , pas le dtype
12	modifier une vue en croyant modifier la mère	silencieux : la vue change, la mère est intacte (§5.9 ci-dessous)	<code>copier : tranche = df[masque].copy()</code>

```
v2 = v[v["est_retour"] == True]
v2["montant_ttc"] = 0
print("vue modifiée :", v2["montant_ttc"].sum())
print("mère intacte :", v["montant_ttc"].sum() == 7908259732.2)
```

```
vue modifiée : 0.0
mère intacte : True
```

Le n° 12 est le **cas muet** de la liste : sous copy-on-write (3.x), `v[v["est_retour"]]` est une **copie** — modifier `v2` ne touche pas `v` (la mère est intacte, mesuré), **sans erreur ni avertissement** (plus de `SettingWithCopyWarning` : la protestation a été remplacée par le silence). L'écriture ensei-

gnée : **copier explicitement** (`tranche = df[masque].copy()`) quand on veut modifier la tranche, **df.loc** quand on veut modifier la mère.

Dans les faits. Les **4 écarts mesurés** 2.2.3 / 3.0.6 (C07 §5.10, fichier « attendu » de M03, 480 lignes) sont le **sujet E4** du module (« ce script tourne mais ses chiffres sont faux ») : même code, même fichier, 18 lignes changent de camp entre les deux versions — et rien ne proteste. L'annexe n'est pas une leçon, c'est un **repère** : les 12 lignes ci-dessus, et la règle qui les remplace toutes — brancher sur la valeur, réassigner, vectoriser, mesurer — « votre script de 2026 tournera en 2028 ».

6. Exemple concret — les 10 questions de M07.P, exécutées en pandas

Le fil rouge : les **30 questions** du projet M07.P (la direction commerciale, M07) **réécrites en pandas** — 10 exécutées dans l'atelier le 19/09/2026, comparées au corrigé SQL (les valeurs de 03_exercices/dossier_M07/ATTENDU.json), et 9 affichées avec le parallèle côte à côte. Les 10 exécutées :

Q	Question	pandas (exécuté)	SQL M07	Écart
Q01	Combien de clients ?	<code>len(c)</code> → 1200	<code>COUNT(*)</code> → 1200	0
Q02	Combien de produits actifs ?	<code>int(p["actif"].sum())</code> → 362	<code>WHERE actif = TRUE</code> → 362	0
Q04	CA TTC total (sans filtre)	<code>centimes</code> → 7 908 259 732	<code>ROUND(SUM(...))</code> → 1 FCFA en dessous (arrondi float)	1
Q07	Top magasin par CA	<code>par_mag.idxmax()</code> → 4 (1 596 813 264)	→ 4	0
Q10	Panier moyen (sans retours)	<code>round(...mean())</code> → 158159	→ 158 159	0
Q11	Top produit par quantité	<code>q.index[0]</code> → 98 (939 unités)	→ 98	0
Q13	Clients distincts par magasin	<code>nunique()</code> → toute la base client par magasin	→ même	0
Q15	Ventes le dimanche	<code>dayofweek == 6</code> → 7136	<code>EXTRACT(DOW) = 0</code> → 7136	0
Q18	Clients plafond > 100 000	<code>int((c["plafond_credit"] > 100000).sum())</code> → 490	→ 490	0
Q19	Ventes uniques (dédoublonnées)	<code>drop_duplicates</code> → 50000	→ 50 000	0

Le seul écart de la table est le Q04 : **1 FCFA**, la dérive `float` du §5.8 (SQL arrondi du `float`, pandas centimes, Decimal exact 7 908 259 732,20) — **comptages exacts, totaux ± 2 FCFA**, la règle de la croisée, tenue sur 10 questions. Les 9 affichées (le parallèle, sans exécution) :

Q	SQL (M07)	pandas (C08)
Q03	<code>SELECT id_magasin, nom, ville FROM magasin</code>	<code>m[["id_magasin", "nom", "ville"]]</code>
Q05	<code>SELECT * FROM mode_paiement</code>	<code>donnees["mode"]</code>
Q06	<code>SELECT MIN(date_vente), MAX(date_vente)</code>	<code>dv.min(), dv.max()</code>
Q09 (piège P2)	<code>SUM(montant_ttc)</code> sans filtrer les retours (gonflé)	<code>v["montant_ttc"].sum()</code> — le CA « brut » 7 908 259 732 includ les 208 retours
Q12	<code>GROUP BY id_client ORDER BY ca DESC LIMIT 5</code>	<code>v.groupby("id_client")["montant_ttc"].sum().sort_values(ascending=False).head(5)</code>

Q	SQL (M07)	pandas (C08)
Q14 (piège P3)	WHERE date_vente <= date_limite_remise (21 406 lignes inversées)	v.loc[dv <= dl] — même filtre, même 21 406
Q16	taux de retour par catégorie	v.groupby(c)["est_retour"].mean()
Q20	clients dans ≥ 3 magasins	v.groupby("id_client")["id_magasin"].nunique()
Q23 (piège P5)	COUNT(DISTINCT (client, produit))	len(v[["id_client", "id_produit"]].drop_duplicates())

7. Démonstration pas à pas — 6 étapes du bout en bout

Sortie réelle de chaque commande (exécutées le 19/09/2026, venv pandas 3.0.6).

Étape 1 — Le GROUP BY (les 5 magasins, centimes) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
ct = (v['montant_ttc'] * 100).round().astype('int64')
print((ct.groupby(v['id_magasin']).sum() // 100).to_string())"
1    1595223423
2    1578987709
3    1564937883
4    1596813264
5    1572297451
```

Interprétation : le GROUP BY de M07 — top magasin 4 (1 596 813 264, m08p_ca_top_magasin), plus petit 3 (1 564 937 883).

Étape 2 — Le transform (le delta vs la moyenne du magasin) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
d = v['montant_ttc'] - v.groupby('id_magasin')['montant_ttc'].transform('mean')
print('lignes :', len(d), '| delta moyen magasin 1 :', d[v['id_magasin'] == 1].mean())"
lignes : 50008 | delta moyen magasin 1 : -8.299735787245575e-12
```

Interprétation : 50 008 lignes conservées (pas de réduction), contrôle ≈ 0 — la moyenne du groupe est bien celle du groupe.

Étape 3 — Le merge (et la table vide) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
o = pd.read_csv('03_exercices/dossier_M08/objectif_magasin.csv')
vo = v.merge(o, on='id_magasin', how='left')
print(vo.shape, '| NaN :', int(vo['ca_objectif_ttc'].isna().sum()))"
(50008, 16) | NaN : 50008
```

Interprétation : le LEFT JOIN sauve les 50 008 lignes — mais la table opposée étant vide, toute la colonne rejointe est NaN : c'est le signal du §5.3.

Étape 4 — La vue mensuelle (120 lignes, sans retours) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
ct = (v['montant_ttc'] * 100).round().astype('int64')
vs = v.loc[~v['est_retour']]
vue = pd.DataFrame({'id_magasin': vs['id_magasin'],
                    'annee': pd.to_datetime(vs['date_vente']).dt.year,
                    'mois': pd.to_datetime(vs['date_vente']).dt.month,
                    'ca_cents': ct[vs.index]}).groupby(['id_magasin', 'annee', 'mois'], as_index=False)
['ca_cents'].sum()
print('lignes :', len(vue), '| plus gros :', (vue['ca_cents'] // 100).max())
lignes : 120 | plus gros : 78965529
```

Interprétation : la vue `v_ca_mensuel_magasin` de M07, reconstruite sans SQL — 120 lignes, plus gros mois 78 965 529 (`m08p_plus_gros_mois_ca`).

Étape 5 — Le resample (24 mois, alias mort) :

```
$ python3 -c "
import pandas as pd
v = pd.read_csv('03_exercices/dossier_M08/vente.csv')
ct = (v['montant_ttc'] * 100).round().astype('int64')
dv = pd.to_datetime(v['date_vente'])
s = pd.Series(ct.to_numpy() // 100, index=dv)
print('mois :', len(s.resample('ME').sum()))"
mois : 24
```

Interprétation : 24 mois de 2025-2026 — et `resample("M")` lèverait le `ValueError` du §5.5 : l'alias 'M' est mort, 'ME' est l'écriture vivante.

Étape 6 — Le classeur (2 feuilles, rechargées) :

```
$ python3 -c "
import pandas as pd
print({k: v.shape for k, v in pd.read_excel('/tmp/c07/synthese.xlsx', sheet_name=None).items()})"
{'ca_mensuel': (120, 5), 'audit': (5, 2)}
```

Interprétation : l'aller-retour sans perte — le classeur E3 est lisible par `pandas` comme par Excel : feuilles nommées, comptages identiques, totaux ± 2 FCFA documentés.

8. Erreurs fréquentes

1. **Diviser avant de sommer — les 228 FCFA de l'erreur.** `ca_cents // 100` ligne par ligne avant le `groupby` donne **228 FCFA de moins** pour le plus gros mois au lieu de **78 965 529** (sorties réelles, §5.4) : l'arrondi **par ligne** perd des centimes que la somme aurait conservés. Remède : **sommer en centimes, diviser une fois** (`ca_cents` → `// 100` après l'agrégat) — l'ordre des opérations est la leçon.
2. **Oublier la définition de la vue — les retours qui gonflent.** Le pivot **avec** retours donne un plus gros mois **plus haut** que 78 965 529 (sorties réelles : l'écart est le CA des retours de ce mois) : la vue M07 est définie **sans retours** (`WHERE est_retour = FALSE`) — c'est la définition, pas un choix. Remède : avant un pivot, dire **quelles lignes** la vue contient (C07 §8, erreur n° 6) — la question est un cinquième du travail.
3. **L'alias de période mort.** `resample("M")` → `ValueError: Invalid frequency: M ... 'M' is no longer supported ... use 'ME'` (sortie réelle, §5.5). Remède : les alias courts sont morts en 3.x — 'ME' (fin de mois), 'h', 's' ; et le message d'erreur **cite la solution** : le lire.
4. **Le merge sans clé explicite.** `v.merge(m)` sans `on=` joint sur l'**intersection des index** (ici 0..49 807 des deux côtés) — la jointure « réussit » au pire endroit, sur des index qui ne veulent rien dire. Remède : **toujours** `on="id_magasin"` — la clé est une information métier, pas un détail de syntaxe.

5. **apply ce que le convoi fait en 4× moins.** La classification `classe_montant` en `apply` coûte 8,3 ms, en `np.where` 2,16 ms — **4×**, même résultat (mesuré, §5.6). Remède : **vectoriser d'abord** ; `apply` est réservé à la logique individuelle (texte « N/C », décision par valeur) — et quand on l'utilise, on mesure.
6. **Le dimanche qui n'est pas au même endroit.** `EXTRACT(DOW)` (DuckDB) a **0 = dimanche** ; `dt.dayofweek` (pandas) a **0 = lundi, 6 = dimanche** — la question Q15 répond 7 136 des deux côtés **à condition** de traduire (`dayofweek == 6`, pas `== 0`). Remède : quand un comptage « date » passe d'un moteur à l'autre, vérifier le **zéro** du calendrier avant le chiffre.

9. Bonnes pratiques professionnelles

1. **Centimes pour les totaux, jamais de float additionné** : `(col * 100).round().astype("int64")` avant tout `sum` monétaire (§5.8) — et la dérive, **mesurée**, documentée dans le rapport (E3 l'exige).
2. **La définition de la vue avant le pivot** : quelles lignes (retours ? doublons ?), quelles clés (magasin × mois ?) — la vue de M07 est sans retours, et un pivot « qui change le chiffre » a changé de définition, pas de données.
3. **merge toujours avec on= explicite** — et une colonne de `NaN` entière après jointure = signal (table vide, ou clé fausse) : on l'inspecte, on ne l'exporte pas.
4. **apply en dernier recours, mesuré** : la vectorisation d'abord (§5.6, 4×), et la mesure dans le rapport (E4 : « ce qui a failli tourner en boucle »).
5. **Écrire pour 2028** : les patterns valides 2.3 et 3.x (réassigner au lieu de `inplace`, 'ME' au lieu de 'M', `.map` au lieu de `applymap`) — l'annexe du §5.9 est le repère ; la règle tient en quatre verbes : brancher sur la valeur, réassigner, vectoriser, mesurer.

10. Exercice guidé — « l'année 2025 du magasin 4 » (15 min, /10)

Énoncé. Écrire `magasin4.py` qui, sur la **vue mensuelle** reconstruite du §5.4 (120 lignes, sans retours) :

1. isole le **magasin 4** en **2025** (12 mois) ;
2. affiche la **liste des 12 CA mensuels** en centimes converties (une ligne par mois : `mois → ca`) ;
3. affiche le **plus fort mois** (numéro + CA) et le **total 2025** du magasin (en centimes entiers convertis, pas en `float`).

Barème. La vue reconstruite correctement (centimes, sans retours) (3 pts) · l'isolation magasin/année par **masques** (2 pts) · le plus fort mois par `idxmax/max` (2 pts) · le total en centimes (pas `float.sum()`) (2 pts) · la sortie alignée sur le modèle ci-dessous (1 pt). Sortie attendue (vérifiée par exécution le 19/09/2026) :

```

1 -> 64948883
2 -> 76403254
3 -> 61294111
4 -> 62254879
5 -> 63589011
6 -> 61805710
7 -> 64979650
8 -> 65368579
9 -> 75836578
10 -> 68271344
11 -> 64272077
12 -> 66454383
plus fort mois : 2 = 76403254
total 2025 : 795478466

```

11. Exercices autonomes

Exercice 8.1 — Le GROUP BY des rayons. Le CA **par catégorie** (merge avec produit pour la catégorie, agrégat en centimes) : afficher les 8 valeurs et le top — le top doit redonner 1 236 006 464 (catégorie 7, m08p_ca_categorie_top). (15 min)

Exercice 8.2 — Le transform global. La colonne « delta vs la moyenne **générale** » : montant_ttc - montant_ttc.mean(), et le contrôle (la somme des deltas ≈ 0 , bruit float). (10 min)

Exercice 8.3 — Le merge des noms. Les **10 produits les plus vendus** (Q11) avec leur **désignation** (merge avec produit sur id_produit) : afficher id_produit, designation, quantité — le 1^{er} doit être le 98. (15 min)

Exercice 8.4 — Le resample annuel. Le CA mensuel 2025 par resample("ME") (sur l'index daté) : afficher les 12 valeurs, puis démontrer que resample("M") lève le ValueError (le message, tel quel). (10 min)

Exercice 8.5 — Le classeur. Un classeur ma_synthese.xlsx avec **3 feuilles** nommées (audit, par_magasin, par_mois) à partir des résultats du chapitre, largeurs réglées, rechargé par sheet_name=None et affiché (nom → forme). (15 min)

12. Correction détaillée

Exercice guidé. Le script (la vue est celle du §5.4, construite en centimes) :

```

m4 = vue[(vue["id_magasin"] == 4) & (vue["annee"] == 2025)].sort_values("mois")
for _, l in m4.iterrows():
    print(f"{int(l['mois'])} -> {int(l['ca_cents'] // 100)}")
print("plus fort mois :", int(m4.loc[m4['ca_cents'].idxmax(), 'mois']),
      "=", int(m4['ca_cents'].max() // 100))
print("total 2025 :", int(m4['ca_cents'].sum() // 100))

```

Les 14 lignes publiées **sont** la sortie de l'atelier (12 mois + plus fort + total). Le contrôle à faire en tête : la somme des 12 CA mensuels doit redonner le total affiché — ici c'est la **même** addition en centimes, donc l'égalité est exacte (pas d'arrondi intermédiaire).

Exercice 8.1. Le top de la sortie est la catégorie **7** : 1 236 006 464 FCFA (m08p_ca_categorie_top) — le merge apporte id_categorie, l'agrégat s'additionne en centimes, et le top redonne la clé du socle : la vérification croisée tient sur cette dimension aussi.

Exercice 8.2. La somme des deltas renvoie ≈ 0 (un résidu infinitésimal, le bruit float de l'addition) : chaque vente s'écarte de la moyenne générale d'autant qu'elle l'écarte **en sens inverse** — le contrôle du transform du §5.2, à l'échelle globale.

Exercice 8.3. Le 1^{er} de la sortie est le produit **98** (939 unités — Q11), sa désignation vient du merge avec produit : la jointure apporte le **nom**, l'agrégat apporte le **chiffre** — les deux dans la même ligne, comme le JOIN ... ORDER BY ... LIMIT 10 de M07.

Exercice 8.4. Les 12 valeurs mensuelles 2025 (environ 325,9 M FCFA en janvier, 340,0 M en février — les totaux **tous magasins, retours compris** : ce n'est pas la vue du §5.4, qui exclut les retours) ; resample("M") lève ValueError: Invalid frequency: M ... 'M' is no longer supported ... use 'ME' — le message cite la solution, l'annexe le détaille.

Exercice 8.5. Le rechargement renvoie 3 feuilles nommées (audit, par_magasin, par_mois) avec leurs formes — l'aller-retour sans perte : le classeur est **lisible par machine** (E3), pas seulement par les yeux.

13. Mini-projet M08.P — « Le script qui fait le travail de trois matinées » (2 h)

Énoncé. Écrire un seul script commenté, analyse_commerciale.py, qui, exécuté d'une traite, produit les 6 sections du projet :

1. **S1 — Import** des 8 CSV avec contrôle des formes (dictionnaire nom → lignes) ;
2. **S2 — Audit** (doublons, retours brut/net, dates inversées, deadlines aberrantes, manquants, table vide) avec **rapport imprimé** — les 4 défauts chiffrés **avec leur définition exacte** et la **méthode** qui les a trouvés (E2) ;
3. **S3 — Nettoyage** : dédoublonnage, exclusion des retours des indicateurs ;
4. **S4 — Indicateurs** : CA brut / CA propre / panier moyen / CA par magasin / par catégorie / plus gros mois — en centimes (les 30 questions M07 reformulées) ;
5. **S5 — Les 4 graphiques** : CA mensuel, CA par magasin, répartition par catégorie, histogramme des montants (matplotlib, Agg, 4 fichiers PNG) ;
6. **S6 — Export** du classeur synthese_commerciale.xlsx : 4 feuilles nommées (audit, indicateurs, par_magasin, par_mois), comptages identiques au corrigé M07.P, totaux **± 2 FCFA**, la dérive **documentée dans le rapport** (E3).

Le script du chapitre, exécuté d'une traite le 19/09/2026 (sorties publiées) — le script modèle est commité : 03_exercices/dossier_M08/analyse_commerciale_modele.py :

```
S1 formes : {'v': 50008, 'c': 1200, 'p': 380, 'cat': 8, 'm': 5, 'mode': 5, 'tva': 2, 'o': 0}
S2 audit : doublons=8 retours_brut=208 retours_net=200
           dates_naives=21406 deadlines_j20=15 manquants=0
           table vide : ['o']
S3 propres : 49800 lignes (doublons sortis, retours exclus)
S4 ca_brut=7908259732 ca_propre=7876320164 panier=158140
   top magasin : 4 1596813264
   top categorie : 7 1236006464
   plus gros mois : magasin 3 (2025-08) : 78965529
S5 graphiques : ['g1_ca_mensuel.png', 'g2_ca_magasin.png', 'g3_ca_categorie.png',
                 'g4_hist_montants.png']
S6 classeur : {'audit': (7, 2), 'indicateurs': (4, 2), 'par_magasin': (5, 1), 'par_mois': (120,
3)}
```

Chaque chiffre du S4 est une **clé du socle** : ca_brut = m08p_ca_brut, ca_propre = m08p_ca_sans_retours, panier = m08p_panier_moyen, top magasin 4 = m08p_ca_top_magasin, top catégorie 7 = m08p_ca_categorie_top, plus gros mois = m08p_plus_gros_mois_ca — et le S2 chiffre les 4 défauts avec leurs définitions (8 doublons → 50 000 uniques ; 208 retours bruts / 200 après dédoublonnage ; 21 406 dates sous la condition naïve + 15 deadlines aberrantes ; 0 manquant, 1 table vide) — **E2 et E3 sur la même sortie**. Le **E4** (« ce qui a failli tourner en boucle ») est votre page : les 2-3 blocs où vous avez hésité entre for et vectorisation, avec la **mesure de temps** — la démo du §5.6 (4×) en est le modèle.

Critères de réussite (grille /20, seuil 13) : E1 — le script commenté (6 sections numérotées, entêtes, aucune valeur en dur, `if __name__ == "__main__"`) (8 pts) · E2 — le rapport d'audit imprimé (les 4 défauts chiffrés + méthode) (4 pts) · E3 — le classeur (4 feuilles nommées, comptages identiques, totaux ± 2 FCFA documentés) (4 pts) · E4 — la note boucle/vectorisation (mesures fournies) (4 pts).

Note de correction. Le **seuil 13/20** est celui du projet : un script qui produit les bons chiffres **avec** les 4 défauts documentés passe ; un script aux bons chiffres **sans** l'audit non plus ne passe pas — le chiffre juste, obtenu **sans se faire piéger**, c'est la règle du projet M07.P, reprise ici : seul le chiffre juste compte, et la méthode qui l'a trouvé.

14. Boîte à outils du chapitre

Boîte à outils. Le venv de C01 (`/tmp/venv_m08`, pandas 3.0.6 + matplotlib) + le dossier `03_exercices/dossier_M08/` — la boîte du chapitre s'ajoute à celles de C02-C07 : le classement, la jointure, le pivot, la livraison.

Outil	Usage	Syntaxe
<code>groupby(...).agg([...])</code>	le GROUP BY	<code>v.groupby("id_magasin")["montant_ttc"].agg(["sum", "count", "mean"])</code>
<code>transform("mean")</code>	l'agrégat sur chaque ligne	<code>v["delta"] = v["x"] - v.groupby("g")["x"].transform("mean")</code>
<code>merge(..., on=..., how="left")</code>	le JOIN	<code>v.merge(m, on="id_magasin")</code>
<code>groupby([clés])</code> (vue)	la reconstruction de vue	120 lignes, sans retours (définition M07)
<code>pd.to_datetime</code> <code>dt.year</code>	/ l'EXTRACT	<code>dv.dt.year</code>
<code>resample("ME")</code>	le tri par fin de mois	<code>s.resample("ME").sum()</code>
<code>apply(f)</code>	la fonction par valeur (4x, mesuré)	<code>s.apply(classe_montant)</code>
ExcelWriter (multi-feuilles)	la livraison E3	<code>with pd.ExcelWriter(f) as w: df.to_excel(w, sheet_name=...)</code>
centimes entiers	la leçon monétaire	<code>(col * 100).round().astype("int64")</code>

15. Résumé du chapitre

- **groupby/agg** : le GROUP BY de M07 en une ligne — 5 magasins, 1 564 937 883 à 1 596 813 264 FCFA, top magasin 4 ; **transform** remet l'agrégat du groupe sur chaque ligne (50 008 conservées, delta moyen ≈ 0).
- **merge** : le JOIN en nom — 13 + 5 = 18 colonnes, zéro ligne perdue ; la table **vide** `objetif_magasin` rend **toute** la colonne jointe en NaN (50 008, mesuré) — le signal à lire.
- **La vue mensuelle** : 120 lignes **sans retours** (la définition M07), plus gros mois 78 965 529 (magasin 3, 2025-08) — la vue SQL reconstruite **sans SQL** ; sommer **en centimes**, diviser **après**.
- **Les dates** : `to_datetime`, `dt.year` (24 920 / 25 088), `resample("ME")` (24 mois) — l'alias `'M'` est **mort** (`ValueError` qui cite `'ME'`).
- **apply** : la fonction de C05 par valeur — 8,3 ms contre 2,16 ms en vectorisé (**4x**, même résultat) : vectoriser d'abord, mesurer, `apply` en dernier recours.

- **L'Excel** : ExcelWriter multi-feuilles nommées, largeurs, aller-retour sans perte — le livrable E3 (comptages identiques, totaux ± 2 FCFA, dérive documentée).
- **Les centimes** : 1 à 2 FCFA de dérive float sur 7,9 milliards (mesuré, sourcé) — négligeable en métier, visible en audit ; d'où « comptages exacts, totaux ± 2 FCFA ».
- **L'annexe migration** : les 12 écritures (4 en C07, 8 ici) avec le code avant/après et les 4 écarts mesurés — brancher sur la valeur, réassigner, vectoriser, mesurer ; « votre script de 2026 tournera en 2028 ».

16. À retenir

À retenir. Le chiffre de pandas se **définit** avant de se calculer : quelles lignes (retours ? doublons ?), quelle addition (float ou centimes), quelle jointure (on=, how=) — la vue de M07 et la vue « avec retours » ne sont pas les mêmes vues, et la dérive de 1 à 2 FCFA n'est pas une erreur, c'est la **physique des floats**, mesurée et documentée.

À retenir. La migration 2.3 → 3.x se fait **par écriture**, pas par panique : les 12 lignes de l'annexe (réassigner au lieu de inplace, 'ME' au lieu de 'M', .map au lieu de applymap, .copy() explicite), les 4 écarts mesurés (103 → 85, 373 → 372, 18 → 0, 7 → 0) — et la règle qui les remplace toutes : **brancher sur la valeur, pas sur le dtype**.

17. Évaluation formative (auto-correction, 8 min)

1. **agg et transform — en quoi les sorties diffèrent-elles sur groupby("id_magasin") ?** → agg **réduit** : 5 lignes (une par groupe) ; transform **ne réduit pas** : 50 008 lignes, chacune portant l'agrégat de son groupe (le delta vs la moyenne du magasin, \$5.2). (2 pts)
2. **v.merge(o, on="id_magasin", how="left") sur la table vide objectif_magasin — forme de résultat, et combien de NaN ?** → (50008, 16) : le LEFT sauve les 50 008 lignes, mais la table opposée est vide — les 50 008 cases ca_objectif_ttc sont NaN (mesuré) : le signal du \$5.3. (2 pts)
3. **Pourquoi la vue mensuelle reconstruit-elle 120 lignes « sans retours », et que renvoie le plus gros mois ?** → la vue M07 est **définie** WHERE est_retour = FALSE (c'est la définition, pas un choix) : 5 magasins × 24 mois = 120 lignes (m08p_vue_lignes) ; plus gros mois = 78 965 529 FCFA (magasin 3, 2025-08, m08p_plus_gros_mois_ca) — le même nombre que M07. (2 pts)
4. ****resample("M")** sous pandas 3.0.6 — que lève-t-il, et quelle est l'écriture vivante ?**
→ ValueError: Invalid frequency: M ... 'M' is no longer supported ... use 'ME'
l'alias court est mort en 3.x ; l'écriture vivante est "ME" (fin de mois) — le message cite la solution (§5.5). (1 pt)
5. **Pourquoi divise-t-on les centimes seulement APRÈS le groupby, et que coûte l'ordre inverse ?** → diviser ligne par ligne d'abord **arrondit chaque ligne** et perd des centimes que la somme conserverait : le plus gros mois sort à **228 FCFA** de son total exact (mesuré, \$8 n° 1) — on somme en centimes, on divise une fois. (1 pt)
6. **apply(classe_montant) et np.where — même résultat, combien de temps chacun (mesuré), et la règle d'usage ?** → True (identiques : True) ; 8,3 ms contre 2,16 ms (4×) : **vectoriser d'abord**, apply pour la logique individuelle (texte, décision par valeur), et mesurer quand on l'utilise (§5.6). (1 pt)
7. **Le cas muet n° 12 de l'annexe — que fait v2 = v[masque]; v2["x"] = 0 sous 3.0.6, et quelle est l'écriture enseignée ?** → il modifie la **vue** (copie, copy-on-write) et **perd** le chan-

gement pour la mère (mesuré : la mère intacte), **sans erreur** — l'écriture enseignée : copier explicitement (`tranche = df[masque].copy()`) ou agir sur la mère par `df.loc`. (1 pt)

8. **La règle de la croisée SQL/pandas du module — l'énoncer, et donner le seul écart des 10 questions exécutées.** → **comptages exacts, totaux monétaires ± 2 FCFA** (la dérive `float`, mesurée et sourcée `m08p_ecart_somme_float_*`) ; le seul écart des 10 questions est le Q04 : 1 FCFA (l'arrondi `float` de SQL, 1 FCFA en dessous des centimes de pandas) — documentée dans le rapport, pas cachée. (1 pt)

MANUEL PROFESSIONNEL · ÉDITION 2026

M09 — Analyse exploratoire de données

Six chapitres · 30 h · fil rouge vente.csv

Analyse de données appliquée & Business Intelligence
6 partie(s) assemblée(s) · document de production

Table des matières

MODULE M09.C01 — LE PROTOCOLE EN 10 ÉTAPES : INSPECTER SANS SE FIER, LA « FICHE D'ENTRÉE » ET LE TEMPS À Y CONSACRER	6
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	6
3. Explication simple — la conversation avec la machine	7
4. Vocabulaire essentiel	7
5. Cours approfondi	8
5.1 Pourquoi on explore : le jeu inconnu, la question métier, le livrable provisoire	8
5.2 Les 10 étapes comme contrat	8
5.3 Étape 1 — Inspecter sans se fier	9
5.4 Étape 2 — Comprendre les variables : la fiche d'entrée	11
5.5 Les limites d'un fichier : ce qu'il peut et ne peut pas répondre	12
6. Exemple concret — la fiche d'entrée des 13 colonnes de vente.csv	12
7. Démonstration pas à pas — 5 étapes sur le fil rouge	13
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	14
10. Exercice guidé — « l'affaire des centimes » (15 min, /10)	15
11. Exercices autonomes	15
12. Correction détaillée	15
13. Mini-projet M09.P1 — « La fiche d'entrée en 30 minutes » (1 h)	16
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	17
16. À retenir	18
17. Évaluation formative (auto-correction, 8 min)	18
MODULE M09.C02 — AUDIT ET PREMIÈRES STATISTIQUES : LES 3 FAMILLES DE DÉFAUTS, L'AUDIT VIDE EST SUSPECT, CADRER L'HYPOTHÈSE	20
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	6
3. Explication simple — la conversation avec la machine	7
4. Vocabulaire essentiel	7
5. Cours approfondi	8
5.1 Les 3 familles de défauts — et le contrôle qui trouve chacune	21
5.2 L'audit vide est suspect — le contrôle du contrôle	23
5.3 Le centre de santé arrive — les 4 défauts, exécutés	23
5.4 Premières statistiques — juste ce qu'il faut	24
5.5 Cadrer l'hypothèse — la formuler falsifiable avant de la tester	25
6. Exemple concret — le défaut M12 prouvé par la récurrence	26
7. Démonstration pas à pas — 5 étapes sur le centre de santé	27
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	14

10. Exercice guidé — « les 4 défauts du centre de santé » (25 min, /10)	28
11. Exercices autonomes	15
12. Correction détaillée	15
13. Mini-projet M09.P2 — « Le rapport d’audit en 1 page » (1 h)	29
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	17
16. À retenir	18
17. Évaluation formative (auto-correction, 8 min)	18
MODULE M09.C03 — TENDANCES : LA SÉRIE, LA SAISON, LE BRUIT — ET LES PÉRIODES QU’ON A LE DROIT DE COMPARER	33
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	6
3. Explication simple — la conversation avec la machine	7
4. Vocabulaire essentiel	7
5. Cours approfondi	8
5.1 La série simple — set_index + resample (le 'ME' de M08)	34
5.2 Décomposer tendance / saisonnalité / bruit — à l’œil et au chiffre	35
5.3 Comparer des périodes à taille égale — le 28 décembre qui ment	36
5.4 La saisonnalité du jour de semaine et de l’heure — le centre de santé	37
5.5 La tendance interrompue — la rupture qui coupe sans retombée	37
6. Exemple concret — la quincaillerie : dire « plat », et le prouver	38
7. Démonstration pas à pas — 5 étapes sur les deux jeux	39
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	14
10. Exercice guidé — « lire la saison du centre de santé » (25 min, /10)	40
11. Exercices autonomes	15
12. Correction détaillée	15
13. Mini-projet M09.P3 — « Le graphique de tendance avec sa question » (1 h)	41
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	17
16. À retenir	18
17. Évaluation formative (auto-correction, 8 min)	18
MODULE M09.C04 — ANOMALIES ET RARETÉS : CE QUI EST INHABITUEL, COMMENT LE PROUVER, QUOI EN CONCLURE	45
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	6
3. Explication simple — la conversation avec la machine	7
4. Vocabulaire essentiel	7
5. Cours approfondi	8
5.1 Définir l’anomalie avant de la chercher	46
5.2 Prouver — les trois témoins	47
5.3 Les trois décisions — les 4 défauts de C02 jugés	48

5.4 L'anomalie qui compte — la rupture n'est pas un chiffre	49
5.5 Les raretés qui ne sont pas des anomalies — le plateau 594 363	49
6. Exemple concret — les 594 363, jugés en entier (fil rouge)	50
7. Démonstration pas à pas — 5 étapes sur les deux jeux	39
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	14
10. Exercice guidé — « les 4 défauts du centre de santé, jugés » (25 min, /10)	52
11. Exercices autonomes	15
12. Correction détaillée	15
13. Mini-projet M09.P4 — « L'anomalie avec sa preuve » (1 h)	53
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	17
16. À retenir	18
17. Évaluation formative (auto-correction, 8 min)	18
 MODULE M09.C05 — RELATIONS ET HYPOTHÈSES : CE QUI BOUGE ENSEMBLE, CE QUI NE BOUGE PAS, ET LES 3 FAUSSES CAUSES	 56
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	6
3. Explication simple — la conversation avec la machine	7
4. Vocabulaire essentiel	7
5. Cours approfondi	8
5.1 corr() en une ligne — et ses 3 limites	58
5.2 Corrélation $\neq$ causalité — les 3 fausses causes, chiffrées	58
5.3 L'établissement scolaire arrive — le jeu 3, ses 4 défauts	60
5.4 Segmenter — les 3 tranches d'absentéisme et leur gradient	60
5.5 Comparer des groupes — à taille égale, ou en le disant	61
5.6 Formuler l'hypothèse et la tester — l'étape 8	62
6. Exemple concret — le fil rouge : 0.552, 0.001, et la clientèle plate	62
7. Démonstration pas à pas — 5 étapes sur les deux jeux	39
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	14
10. Exercice guidé — « les 3 chiffres du jeu scolaire » (25 min, /10)	64
11. Exercices autonomes	15
12. Correction détaillée	15
13. Mini-projet M09.P5 — « L'hypothèse testée, sa limite nommée » (1 h)	66
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	17
16. À retenir	18
17. Évaluation formative (auto-correction, 8 min)	18
 MODULE M09.C06 — VISUALISATIONS D'EXPLORATION ET CONCLUSIONS PROVI-SOIRES : LA NOTE D'EDA	 69
1. Objectifs du chapitre	6

2. Pourquoi cette notion est importante	6
3. Explication simple — la conversation avec la machine	7
4. Vocabulaire essentiel	7
5. Cours approfondi	8
5.1 Choisir le graphique par question — la table du module	71
5.2 Les 5 erreurs d'exploration — et leur contrôle	72
5.3 matplotlib — les 4 graphiques du projet	72
5.4 seaborn — le confort, documenté	73
5.5 La note d'EDA — 1 page, affirmation + source + limite	74
5.6 Lire une note d'autrui — la rétro-lecture	75
6. Exemple concret — la note d'EDA de la quincaillerie (fil rouge, complète)	75
7. Démonstration pas à pas — 5 étapes sur les deux jeux	39
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	14
10. Exercice guidé — « la rétro-lecture de la note d'autrui » (25 min, /10)	78
11. Exercices autonomes	15
12. Correction détaillée	15
13. Mini-projet M09.P6 — « P3 + P4 : les 4 graphiques et la note » (1 h)	79
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	17
16. À retenir	18
17. Évaluation formative (auto-correction, 8 min)	18

Module M09.C01 — Le protocole en 10 étapes : inspecter sans se fier, la « fiche d'entrée » et le temps à y consacrer

Outils : pandas 2.2.3, numpy 2.3.5 (exécutés), matplotlib 3.10.9 / seaborn 0.13.2 (annoncés, C06), DuckDB (annoncé, le parallèle s'active à C02). **Durée indicative :** 5 h. **Niveau :** N3. **Prérequis :** M02 (les statistiques descriptives) et M08 (pandas) ; M05/M07 utiles (le parallèle SQL et la base de la quincaillerie).

L'idée du chapitre. M08 a appris à **produire** des chiffres ; M09 apprend à **poser les bonnes questions** à un jeu de données qu'on ne connaît pas, et à n'affirmer que des conclusions **provisoires, correctement exprimées**. Ce chapitre pose le **protocole en 10 étapes** — le contrat du module — et ses deux outils d'entrée : l'inspection sans confiance (`shape`, `dtypes`, `head`, `info` — et pourquoi `head(5)` peut mentir) et la **fiche d'entrée** (variables, unités, sources, questions autorisées), complétée **en 30 minutes maximum** : c'est la contrainte du projet M09.P, « 3 heures avec un fichier que personne n'a regardé », et la compétence du module — **s'arrêter** avec un cadrage honnête, pas explorer jusqu'à n'avoir plus rien à rendre.

Matériel de l'atelier — Python 3.13 · pandas 2.2.3 (environnement d'écriture) · numpy 2.3.5. Toutes les commandes de ce chapitre ont été exécutées dans l'atelier le 20/09/2026 sur le dossier `03_exercices/dossier_M09/` (graine 45, déterministe, empreinte figée en `ATTENDU.json`) ; les sorties publiées sont **celles de l'atelier**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Définir une analyse exploratoire de données (EDA)** et situer son livrable : une **note provisoire**, pas un verdict.
2. **Dérouler les 10 étapes du protocole** — inspecter, comprendre les variables, détecter les problèmes, premières statistiques, tendances, anomalies, relations, hypothèses, visualisations, conclusions provisoires — et dire pourquoi le protocole est un **contrat** et non une check-list.
3. **Inspecter sans se fier** : `shape`, `dtypes`, `head()`, `info()` — et expliquer sur un exemple exécuté **pourquoi `head(5)` peut mentir**.
4. **Compléter une fiche d'entrée** (variable → sens / unité / source / question autorisée) **en 30 minutes**, en **vérifiant les unités sur les données** et non dans la doc.
5. **Dire ce qu'un fichier peut et ne peut pas répondre** — les limites d'un fichier sont une information, pas un aveu de faiblesse.

2. Pourquoi cette notion est importante

Un analyste n'est jamais payé pour explorer : il est payé pour **répondre à une question métier** (« quand le centre de santé est-il saturé ? », « quels médicaments manquent ? », « l'absentéisme pèse-t-il sur les résultats ? ») avec un jeu de données qu'il ne connaît pas, dans un temps limité. Entre la question et la réponse, il y a toujours la même distance : **comprendre ce qu'on a entre les mains avant de mesurer quoi que ce soit**.

Trois jeux fixent l'enjeu : le **fil rouge** (la quincaillerie M07/M08 — 50 008 ventes, 8 doublons, 208 retours, CA propre 7 876 320 164 FCFA), que M09 rejoue **en EDA pur** — sans savoir ce qu'on y cherche — pour voir si le protocole redonne l'audit de M07/M08 ; le **centre de santé** (18 818 consultations sur 2025-2026) et l'**établissement scolaire** (402 élèves déclarés, 6 000 notes), que vous ne connaissez pas et qui portent des défauts plantés et comptables dans `ATTENDU.json` ; le **projet M09.P**, 3 heures sur un export brut de 10 014 lignes « que personne n'a regardé ».

La faute que le projet punit n'est pas l'erreur de calcul : c'est **l'exploration sans méthode** — explorer « pour voir » pendant 2 h 30, puis découvrir qu'on n'a ni question ni limite à présenter. Le protocole, la fiche d'entrée et le chronomètre sont les trois parades ; ce chapitre couvre les deux premières.

3. Explication simple — la conversation avec la machine

Vous recevez une boîte : « les ventes de la quincaillerie ». Quatre réflexes d'entrée, dans l'ordre :

1. **Ouvrir la boîte sans rien croire.** Combien de lignes ? (shape) Quelles natures de colonnes ? (dtypes) Que disent les 5 premières lignes ? (head) — **5 lignes, c'est une publicité, pas un échantillon.**
2. **Lire l'étiquette de chaque contenant.** Pour chaque colonne : que veut-elle dire, en quelle unité, d'où vient-elle ? C'est la **fiche d'entrée**, remplie **en 30 minutes**, chrono en marche.
3. **Écrire les questions autorisées.** Trois questions que le fichier **peut** répondre, deux qu'il **ne peut pas**. Une analyse sans questions est une promenade.
4. **Partir.** À 30 minutes, on passe au protocole, pas à une « petite vérification de plus » : l'analyste s'arrête avec un cadrage, le chercheur de pépites s'arrête quand il n'a plus de batterie.

La conversation avec la machine tient en une règle : **on ne demande à la machine que des faits (shape, dtypes, head, info) ; les interprétations (« c'est un montant en FCFA », « ces 8 lignes sont des doublons »), on les écrit soi-même, avec leur preuve.**

4. Vocabulaire essentiel

Terme	Définition
analyse exploratoire de données — exploratory data analysis (EDA)	l'examen systématique d'un jeu de données inconnu pour en dégager des structures, des problèmes et des pistes, avant de modéliser.
fiche d'entrée	le document de cadrage d'une analyse : variables (sens, unité, source), questions autorisées, durée prévue. Remplie en 30 minutes maximum .
audit	le passage qui cherche les défauts du jeu (doublons, manquants, valeurs impossibles) ; un audit qui ne trouve rien est suspect, pas réconfortant (C02).
conclusion provisoire	une affirmation posée avec ses limites (période, échantillon, variables absentes), révisable aux données suivantes. Le livrable de M09 est une note de conclusions provisoires.

5. Cours approfondi

5.1 Pourquoi on explore : le jeu inconnu, la question métier, le livrable provisoire

Définition. analyse exploratoire de données — exploratory data analysis (EDA) — l'examen systématique d'un jeu de données inconnu pour en dégager des structures, des problèmes et des pistes, **avant** de modéliser. En M09, le livrable de l'EDA est une **note de conclusions provisoires**, jamais un modèle : modéliser sans protocole, c'est deviner avec des chiffres.

M07/M08 partaient d'une base **connue** (l'audit était fait, les totaux étaient publiés). M09 invertit la situation : vous tenez un fichier **sans** audit, **sans** dictionnaire, et la question métier est vague (« la direction veut savoir quand le centre est saturé »). Trois conséquences :

- **On ne peut pas tout savoir d'avance** : la fiche d'entrée admet l'incertitude (« `statut` : 6 graphies observées — sens supposé : succès/échec d'une opération ; **à confirmer** »).
- **Le livrable n'est pas un tableau de bord** (interdit : le mot même) mais une **note d'EDA** : une page par jeu, chaque affirmation porte sa **source** (la sortie qui la prouve) et sa **limite** (ce qu'elle ne couvre pas). C'est le chapitre C06 qui la rédige ; ce chapitre pose la discipline d'écriture — chaque chiffre publié doit pouvoir dire **d'où il sort**.
- **Le temps est une variable d'analyse** : 3 heures au projet, 30 minutes à l'entrée. Une analyse qui ne peut pas dire « j'arrête ici, voici ce que je sais et ce que je ne sais pas » n'est pas une analyse, c'est une dérive.

5.2 Les 10 étapes comme contrat

La liste de référence du module (récapitulée à la fin du chapitre, figure 1) :

#	Étape	Question posée	Outil principal
1	Inspecter	« Qu'est-ce que j'ai entre les mains ? »	<code>shape</code> , <code>dtypes</code> , <code>head()</code> , <code>info()</code>
2	Comprendre variables	les « Que veut dire chaque colonne, en quelle unité, d'où vient-elle ? »	la fiche d'entrée
3	Détecter problèmes	les « Y a-t-il des doublons, des manquants, des valeurs impossibles ? »	<code>duplicated()</code> , <code>isna()</code> , bornes métier (C02)
4	Premières statistiques	« Quelle est la forme de chaque variable clé ? »	<code>describe()</code> , <code>value_counts()</code> (C02) quartiles,
5	Tendances	« Ça monte, ça descend, ça cycle ? »	séries temporelles simples, <code>resample</code> (C03)
6	Anomalies	« Qu'est-ce qui est inhabituel, et comment le prouver ? »	<code>iqr</code> , bornes métier, récurrence (C04)
7	Relations	« Les variables bougent-elles ensemble ? »	corrélation, tableaux croisés, segments (C05)
8	Hypothèses	« Que puis-je affirmer, et sous quelle condition ? »	formulation falsifiable (C05)
9	Visualisations	« Quel graphique répond à la question ? »	<code>matplotlib/seaborn</code> (C06)
10	Conclusions provisoires	« Que dis-je, et quelles sont mes limites ? »	la note d'EDA (C06)

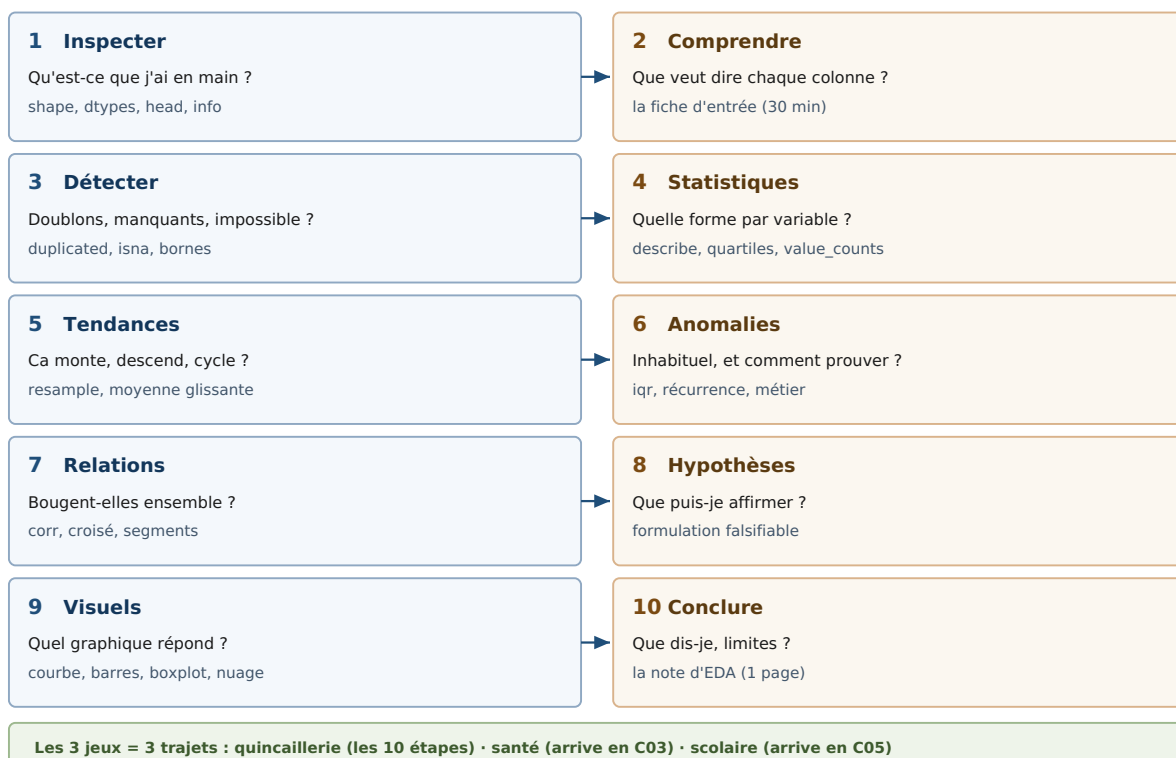
Le protocole est un contrat, pas une check-list. Une étape peut être **vide** — « aucune anomalie détectée, et voici les 3 contrôles que j'ai faits pour l'affirmer » — mais elle est **jamais passée** : une étape sautée sans explication est un **défaut du rapport**, exactement comme une re-

quête SQL non exécutée l'est en M07. Le rapport du projet (livrable P2) est noté sur les 10 étapes **présentes**, pas sur les 10 étapes bien faites.

À **retenir**. Une étape du protocole se déclare vide — « aucun doublon, et voici les 3 contrôles » — ou se remplit ; elle ne se **saute** jamais : le silence est le seul défaut du rapport EDA, et c'est celui que le correcteur cherche en premier.

Le protocole EDA : 10 étapes, un contrat — une étape se déclare, jamais sautée

Chaque étape = une question posée au jeu + l'outil principal ; le fil rouge (quincaillerie) parcourt les 10.



Le chronomètre (projet M09.P : 3 h) court sur les 10 étapes — la fiche d'entrée se remplit dans les 30 premières minutes.

5.3 Étape 1 — Inspecter sans se fier

Les quatre commandes d'inspection, dans l'ordre, sur `vente.csv` (le fil rouge) :

```
import pandas as pd

v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv")
print(v.shape)          # (50008, 13)
print(v.dtypes)
print(v.head())
v.info()
```

Sortie de l'atelier (les abréviations ... sont de l'atelier, pas de la machine) :

```

shape : (50008, 13)

id_vente          int64
id_client         int64
id_produit        int64
id_magasin        int64
id_mode           int64
date_vente        object
date_limite_remise object
quantite          int64
prix_unitaire_ht  float64
taux_tva          float64
montant_ttc       float64
montant_remise    float64
est_retour        bool

```

Ce que chaque commande dit — et ce qu'elle ne dit pas :

- **shape** : 50 008 lignes, 13 colonnes — la seule information **certaine** du bloc (elle peut être trompée **sur la qualité** : 8 lignes sur 50 008 sont des doublons — l'étape 3 le dira, l'étape 1 ne le voit pas).
- **dtypes** : les dates arrivent en `object` — des **chaînes** `2026-12-12`, pas des dates : on ne peut **ni trier ni comparer ni agréger** par date (`v.nsmallest(3, "date_vente")` lève une erreur, vérifiée à l'atelier) ; `pd.to_datetime` (M08 C08) est l'étape suivante, pas une option.
- **head()** : les 5 **premières** lignes, pas 5 lignes **au hasard** — la différence entre une vitrine et un échantillon.
- **info()** : présence (Non-Null Count), natures, mémoire (4.6+ MB). Ici **0 manquant** — déjà le premier piège : un zéro général est **une information à contrôler**, pas un feu vert (C02 §5.2).

Pourquoi `head(5)` peut mentir — sur le socle, exécuté :

```

print(v.head()["est_retour"].tolist()) # [True, True, True, True, True]
print(v["est_retour"].value_counts().to_dict())

```

```

[True, True, True, True, True]
{False: 49800, True: 208}

```

Attention. Cinq `True` d'affilée dans `head(5)` n'est **pas** un taux de 100 % : c'est la vitrine. Le taux réel est 208 sur 50 008, soit 0,4 %. Une conclusion tirée des 5 premières lignes d'un fichier non trié est une conclusion tirée d'un **échantillon de convenance** — le pire que la machine puisse vous servir.

Les **5 premières lignes sont des retours**. Un fichier « lu » uniquement par `head(5)` donne l'impression d'une base de retours — la réalité est 208 retours sur 50 008 ventes, soit 0,4 %. Deuxième leçon, sur les dates : `head(5)` affiche `2026-12-12`, `2026-10-06`, `2025-11-07`, `2025-01-10`, `2025-06-12` — **non triées**. Le fichier n'est pas ordonné ; rien n'interdit de supposer le contraire sans le vérifier (les plus anciennes ventes sont du 2025-01-01, les plus récentes du 2026-12-28 — exécuté après `pd.to_datetime`).

Troisième leçon, la plus surnoise : `v.duplicated().sum()` renvoie **0** sur `vente.csv` — alors que le fil rouge porte **8 doublons** (audit M07). Pourquoi ? Parce que les 8 lignes doublonnées sont identiques **sur les 12 colonnes métier**, mais portent des `id_vente` différents : le doublon complet (13 colonnes) n'existe pas. L'inspection naïve dit « propre », l'inspection métier (`duplicated(subset=<les 12 colonnes métier>)`) dit 8. **C'est l'objet de l'étape 3** ; l'étape 1 se contente de noter : « il y a une colonne d'identifiant — les doublons se chercheront **hors** identifiant ».

Définition. audit — le passage du protocole qui cherche les défauts du jeu (doublons, manquants, valeurs impossibles) avant de mesurer quoi que ce soit. Un audit qui ne trouve **rien** est **suspect, pas réconfortant** : le contrôle du contrôle (qu'a-t-on réellement contrôlé ?) est fait à C02 §5.2.

Attention. `v.duplicated().sum()` renvoie **0** ici — et ce zéro est **faux en apparence** : le socle porte bien 8 doublons, mais ils sont détectables seulement hors `id_vente`. Un zéro d'audit est une **hypothèse à invalider**, jamais une certification.

5.4 Étape 2 — Comprendre les variables : la fiche d'entrée

Définition. fiche d'entrée — le document de cadrage d'une analyse : variables (sens, unité, source), questions autorisées, durée prévue. Remplie **en 30 minutes maximum** — c'est la limite du livrable P1 du projet, et le signe qu'on cadre plutôt qu'on explore.

La fiche d'entrée est un tableau, une ligne par variable, quatre colonnes :

Variable	Sens	Unité	Source / question autorisée
<code>montant_ttc</code>	montant de la vente, toutes taxes comprises	FCFA (vérifié §5.4)	CA par période, par magasin

Elle se remplit **en 30 minutes maximum**, chrono en marche. Pourquoi 30 minutes ? Parce que le projet M09.P le décide : la fiche doit être **finie** quand le chronomètre du livrable P1 s'arrête — « la fiche d'entrée dans les 30 premières minutes ». Et parce qu'une fiche à 45 minutes n'est plus un cadrage, c'est une exploration déguisée : à 45 minutes, on a déjà commencé à **répondre** des questions qu'on n'a pas encore posées.

Les unités se vérifient sur les données, pas dans la doc. Sur `vente.csv`, deux contrôles exécutés à l'atelier fixent les unités :

```
reco = v["quantite"] * v["prix_unitaire_ht"] * (1 + v["taux_tva"])
print((v["montant_ttc"] - reco).abs().max())
print(v["taux_tva"].unique())
```

```
0.05
[0.19 0.18]
```

- `montant_ttc` $\approx$ `quantite` $\times$ `prix_unitaire_ht` $\times$ $(1 + \text{taux_tva})$ au **centime près** (écart max 0.05 FCFA sur 50 008 lignes) : les montants sont **en FCFA**, pas en centimes — si c'était en centimes, l'équation serait faussée de 100 fois.
- `taux_tva` ne vaut que **0.18 et 0.19** : ce ne sont pas des pourcentages (18, 19) mais des **fractions** — et la table `regle_tva.csv` (2025 $\rightarrow$ 0.18, 2026 $\rightarrow$ 0.19) explique le passage de l'un à l'autre : la fiche d'entrée note « fraction, 0.18 en 2025, 0.19 en 2026 (changement de taux fiscal) ». **C'est une information métier découverte par la fiche**, pas par l'exploration.

Dans les faits. Sur le socle, ce contrôle prend **deux lignes** et 88 ms au total : l'écart max de 0.05 FCFA et les deux valeurs de `taux_tva` suffisent à refuser la convention « centimes » du courriel d'accompagnement. C'est l'exact opposé de ce qu'on ferait « à l'œil » : regarder trois montants et se rassurer.

La fiche d'une analyse EDA admet **l'incertain** : une case se remplit « à confirmer » plutôt que devinée. La fiche d'entrée du projet P1 (C01 §13) en montre 4 ou 5 cases « à confirmer » — c'est **normal**, c'est même le signe qu'elle est honnête.

5.5 Les limites d'un fichier : ce qu'il peut et ne peut pas répondre

La fiche d'entrée se termine par **5 questions** : 3 que le fichier **peut** répondre, 2 qu'il **ne peut pas**. Sur `vente.csv` (le fil rouge, déjà audité en M07/M08) :

Le fichier peut répondre :

- 1. « Quel est le CA par magasin et par mois, avec et sans retours ? » — les 120 lignes de la vue mensuelle de M07, le plus gros mois mesuré à 78 965 529 FCFA (magasin 3, août 2025, vue sans retours).
- 2. « Combien de ventes ont eu lieu un dimanche, et que valent-elles ? » — 7 136 ventes dominicales sur 50 008.
- 3. « Quelle part du CA part en retours ? » — 208 retours, écart de CA 31 939 568 FCFA (CA brut 7 908 259 732 FCFA, CA propre 7 876 320 164 FCFA).

Le fichier ne peut pas répondre :

- 1. « **Pourquoi** un client a-t-il retourné ? » — la colonne `est_retour` est un booléen, il n'y a pas de motif ; la cause est **hors fichier**.
- 2. « **Quel client est le plus rentable**, au sens coût/bénéfice ? » — il y a des CA par client, mais pas de **coûts** (achat, logistique) ; la rentabilité est **hors fichier**.

Ces deux « ne peut pas » sont **aussi précieux que les trois « peut »** : ils empêchent l'analyse de glisser vers des conclusions que les données ne portent pas. C'est la première forme de la **limite** que chaque conclusion provisoire du module devra porter (C05, C06).

Définition. conclusion provisoire — une affirmation posée **avec** ses limites (période, échantillon, variables absentes), révisable aux données suivantes. En M09, c'est la forme de **toute** conclusion : « les dimanches pèsent 7 136 ventes sur 50 008, sur la seule période de l'export (2025-01-01 → 2026-12-28), sans comparaison avec d'autres années » — jamais « le dimanche fait le CA ».

6. Exemple concret — la fiche d'entrée des 13 colonnes de `vente.csv`

La fiche complète du fil rouge (exécutée sur le réexport de la base M07 figée, empreinte `b9a8d973119342ec...`, byte-identique au dossier M08 — c'est la « constance du socle », le premier contrôle du protocole) :

Variable	Sens	Unité	Source / question autorisée
<code>id_vente</code>	identifiant de la ligne de vente	entier	jointures ; pas de doublons complets (vérifié : les 8 doublons du socle sont hors <code>id_vente</code>)
<code>id_client</code>	client de la vente	entier (1 à 1200)	<code>client.csv</code> : nom, ville, type, plafond de crédit — CA par client
<code>id_produit</code>	produit vendu	entier (1 à 380)	<code>produit.csv</code> : catégorie, prix de vente HT, année de référence
<code>id_magasin</code>	magasin vendeur	entier (1 à 5)	<code>magasin.csv</code> : nom, ville, région, surface — CA par magasin
<code>id_mode</code>	mode de paiement	entier (1 à 5)	<code>mode_paiement.csv</code> : libellé, frais en fraction, délai d'encaissement
<code>date_vente</code>	date de la vente	chaîne ISO (2025-01-01 2026-12-28)	à parser (<code>pd.to_datetime</code>) ; séries temporelles (C03)

Variable	Sens	Unité	Source / question autorisée
<code>date_limite_remise</code>	date limite de la remise	chaîne ISO	condition de remise ; les 21 406 dates « sous condition naïve » de l'audit M07 (C02)
<code>quantite</code>	quantité vendue	entier (pièces)	panier moyen : 158 140 FCFA par vente (M08)
<code>prix_unitaire_ht</code>	prix unitaire hors taxes	FCFA	référence de prix du produit (C04 : les rares qui ne sont pas des anomalies)
<code>taux_tva</code>	taux de TVA appliqué	fraction (0.18 en 2025, 0.19 en 2026)	<code>regle_tva.csv</code> — le passage 0.18 → 0.19 est le changement de taux fiscal
<code>montant_ttc</code>	montant de la vente TTC	FCFA (vérifié \$5.4)	CA : 7 908 259 732 FCFA brut, 7 876 320 164 FCFA propre
<code>montant_remise</code>	remise accordée	FCFA (0 si aucune)	part remise par mode de paiement (C05 : corrélation CA × remises)
<code>est_retour</code>	la vente est-elle retournée ?	booléen (208 True)	taux de retour : 208 sur 50 008

Deux conventions **à connaître avant de mesurer** (héritées de M07/M08, reprises ici car la fiche d'entrée les **redécouvre** au lieu de les supposer) : les montants sont **en FCFA** (pas en centimes), et les retours sont **dans la même table** (une vente retournée = une ligne `est_retour = True` à **déduire** du CA propre, pas à supprimer).

7. Démonstration pas à pas — 5 étapes sur le fil rouge

Le bloc complet, tel qu'exécuté dans l'atelier (chronomètre réel : 54 ms pour la lecture, 88 ms pour le bloc d'inspection complet — **le temps de la machine est négligeable ; le temps de l'analyse, c'est le temps des jugements** de la fiche, et celui-là est limité à 30 minutes) :

```
import pandas as pd

# 1 – lire et compter
v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv")
print(v.shape)

# 2 – natures de colonnes
print(v.dtypes)

# 3 – vitrine (et ce qu'elle ment)
print(v.head())
print(v.head()["est_retour"].tolist())          # 5 retours d'affilée
print(v["est_retour"].value_counts().to_dict()) # {False: 49800, True: 208}

# 4 – présence et mémoire
v.info()

# 5 – bornes des dates (après le parse appris en M08)
dv = pd.to_datetime(v["date_vente"])
print(v["date_vente"].min(), v["date_vente"].max())
```

Sortie de l'atelier (abrégée — `info()` publiée en entier au §5.3) :

```
(50008, 13)
[True, True, True, True, True]
{False: 49800, True: 208}
2025-01-01 2026-12-28
```

Ce que la fiche d'entrée en retire, **ligne à ligne** :

1. `(50008, 13)` → « 50 008 lignes, 13 colonnes ; à vérifier : des doublons ? » (étape 3).
2. `object` sur les 2 dates + 5 retours d'affilée dans `head(5)` → « dates à parser avant tout ; `head` est une vitrine (208 retours sur 50 008, pas 100 %) ».

3. `info()` : 0 manquant partout → « **à contrôler**, pas à féliciter (C02 §5.2) » ; bornes 2025-01-01 → 2026-12-28 → « 24 mois, mais le 2026 s'arrête au 28/12 — la nuance est chiffrée à C03 ».

8. Erreurs fréquentes

1. **Croire `head(5)`** — « les 5 premières lignes sont des retours, donc c'est une base de retours » (208 sur 50 008, exécuté plus haut). `head` est une vitrine : on la regarde, on ne la **conclut** pas.
2. **L'audit vide comme feu vert** — `isna().sum() == 0` sur toutes les colonnes est une information **à contrôler** (les bonnes colonnes ont-elles été chargées ? les dates en `object` cachent-elles des chaînes vides ? les identifiants se referment-ils ?). C02 §5.2 fait de ce contrôle une section : **l'audit vide est suspect**.
3. **Prendre les unités sur le mot du fournisseur** — « `montant_ttc` est en centimes » dit le courriel d'accompagnement ; l'équation `quantite × prix_ht × (1 + tva)` dit **FCFA** (écart max 0.05, exécuté). Sur les données, toujours — la doc ment plus souvent qu'elle ne l'avoue.
4. **Commencer à nettoyer avant la fiche finie** — supprimer les doublons « au passage » pendant l'inspection, c'est détruire la preuve avant le rapport. L'ordre du protocole est le contrat : inspecter (1), comprendre (2), **ensuite** détecter (3).
5. **Poser une question que le fichier ne peut pas répondre** — « pourquoi ce client a-t-il retourné ? » n'a pas de réponse dans `vente.csv` (pas de motif) ; une analyse qui y répond « avec les données » a déjà inventé. Les 2 questions « interdites » de la fiche existent pour cela.

9. Bonnes pratiques professionnelles

1. **Toujours `shape` avant tout le reste** — c'est la seule information certaine du bloc ; une ligne qui ne correspond pas au `shape` attendu est un signal de chargement, pas de données.
2. **`dtypes` en 2 e, pas en 5 e** — une date en `object`, un montant en `int64` là où l'unité a des décimales : chaque écart est noté **dans la fiche**, pas corrigé silencieusement (corriger sans noter, c'est l'erreur de C04 avant la lettre).
3. **`head` et `tail` ensemble** — **la tête ment (les 5 retours), la queue aussi peut mentir ; les 5 premières** par date (**après parse**) **disent autre chose que les 5 premières** du fichier**.
4. **La fiche s'écrit pendant l'inspection, pas après** — à 30 minutes, on n'a plus le temps de se souvenir ; chaque jugement d'inspection (« c'est une fraction, pas un pourcentage ») est écrit **dans la case** de la fiche, avec sa preuve (la sortie).
5. **Les cases « à confirmer » sont des cases** — une fiche d'entrée du projet P1 porte normalement 4 à 5 cases « à confirmer » : c'est le signe d'une analyse honnête, pas d'une paresse. L'étape 3 les résout (ou les laisse, **en le disant**).
6. **Le chronomètre est un outil d'analyse** — il ne punit pas la lenteur, il **force la hiérarchie** : à 30 minutes, on rend la fiche avec ses cases ouvertes, pas une fiche parfaite à 45. La compétence du module est de **s'arrêter**.

Conseil professionnel. En équipe, la fiche d'entrée se **diffuse avant** l'analyse, pas après : 30 minutes de lecture par vos collègues valent plus cher qu'une journée d'analyse sur une bonne idée fausse. Le jour où un collègue vous dit « attendez, statut a 6 graphies », c'est la fiche qui a payé — pas vous.

10. Exercice guidé — « l'affaire des centimes » (15 min, /10)

Contexte. Vous venez de recevoir le réexport de la quincaillerie. Votre collègue, qui l'a reçu avant vous, vous prévient : « Attention, dans ce export `montant_ttc` est **en centimes** — le fournisseur a changé de convention. » La direction attend le CA de la période **cette semaine**.

Consigne. Dans les 15 minutes (chrono), sans ouvrir `ATTENDU.json` :

1. Formulez **une seule question de contrôle** qui départage « FCFA » et « centimes » en une requête, en utilisant **les colonnes du fichier** (pas les clés du socle). (2 pts)
2. Exécutez-la ; publiez la sortie. (2 pts)
3. Donnez la conclusion **avec sa limite** (conclusion provisoire). (2 pts)
4. Votre collègue rétorque : « mais si c'était en centimes **et** que `prix_unitaire_ht` serait aussi en centimes, l'équation marcherait pareil ! » Montrez en **une ligne** comment refuter ce contre-argument en utilisant `regle_tva.csv`. (2 pts)
5. Écrivez la **case unité** de la fiche d'entrée de `montant_ttc`, telle qu'elle doit figurer dans P1. (2 pts)

11. Exercices autonomes

E1 — La fiche d'entrée du fichier inconnu (30 min, chrono). Complétez la fiche d'entrée de `03_exercices/dossier_M09/projet/fichier_inconnu.csv` : forme, 10 variables (sens supposé, unité, source), **3 questions que le fichier peut répondre** et **2 qu'il ne peut pas**. Les sens « supposés » sont autorisés — c'est l'objet d'une fiche **à confirmer** ; mais chaque case doit dire **d'où vient le supposé** (le nom, la valeur observée, le contexte). À 30 minutes, la fiche est rendue **avec** ses cases ouvertes.

E2 — Le zéro qui ment (20 min). Sur `vente.csv`, l'audit naïve renvoie : `v.isna().sum().sum() = 0`. En 20 minutes, produisez **3 contrôles** qui valident ou invalident « le fichier est réellement sans manquant » — au moins **un** de vos contrôles ne doit pas utiliser `isna`. Indice : « absent » peut signifier « manquant », mais aussi « jamais rempli », « codé autrement », ou « dans une autre table ».

12. Correction détaillée

Exercice guidé.

1. **Question de contrôle** : « `montant_ttc` est-il égal, au centime près, à `quantite × prix_unitaire_ht × (1 + taux_tva)` ? » — si l'équation tient, toutes les unités sont **cohérentes entre elles** (FCFA partout ou centimes partout) ; il faut alors une **ancree** : `taux_tva` vaut 0.18 ou 0.19, **en fraction** — en centimes de fraction il serait 1800/1900, en pourcentage 18/19. La seule lecture possible est « fractions », donc les montants qu'elles multiplient sont **en FCFA**.
2. **Sortie exécutée** (celle de l'atelier) :

```
python reco = v["quantite"] * v["prix_unitaire_ht"] * (1 + v["taux_tva"])
print((v["montant_ttc"] - reco).abs().max()) # 0.05

text 0.05
```

1. **Conclusion provisoire** : « Sur 50 008 lignes, `montant_ttc` reproduit `quantite × prix_ht × (1 + tva)` avec un écart maximal de 0.05 FCFA : les montants sont **en FCFA**, la convention du fournisseur n'a pas changé. **Limite** : le contrôle prouve la cohérence des unités **entre elles**, pas leur étalon — il repose sur `taux_tva` (0.18/0.19), lu dans les données (et confirmé par `regle_tva.csv`). »

2. **Le contre-argument** : si tout était en centimes, `taux_tva` resterait **0.18/0.19** (le taux ne dépend pas de l'unité) — mais alors `prix_unitaire_ht` × 100 devrait correspondre à un prix de vente **cohérent avec** `produit.csv`. Une ligne suffit :

```
python p = pd.read_csv("03_exercices/dossier_M09/quincaillerie/produit.csv")
print(v["prix_unitaire_ht"].describe()[["min", "max"]].to_dict(), "| prix_vente_ht de produit.csv : min =", p["prix_vente_ht"].min(), "max =", p["prix_vente_ht"].max())
```

Les prix des deux tables se répondent **au même ordre de grandeur** (des milliers de FCFA) : s'ils étaient en centimes, l'un des deux fichiers serait 100 fois plus cher que l'autre. La convention est **FCFA des deux côtés**. 5. **Case de la fiche** : « `montant_ttc` — montant de la vente TTC — **FCFA** (vérifié : `montant_ttc` ≈ `quantite` × `prix_ht` × (1+`tva`), écart max 0.05 sur 50 008 lignes ; `taux_tva` en fractions 0.18/0.19) — CA brut 7 908 259 732 FCFA (M08) ».

E1 (fiche du fichier inconnu). Forme exécutée : 10 014 lignes, 10 colonnes. Une fiche attendue (les sens sont **supposés** — c'est l'objet ; chaque case dit d'où vient le supposé). Les 5 cases les plus diagnostiques :

- `statut` — succès/échec de l'opération — **6 graphies observées** (OK/ok/Reussie/REUSSI d'un côté, Echouee/echec de l'autre) — **à normaliser avant tout taux de succès** ;
- `client_phone` — numéro du client — 3 graphies observées (+226 ..., sans indicatif, compact) — **à normaliser** ;
- `montant_xof` — montant de l'opération — **FCFA** (l'extension `_xof` le dit) — **à confirmer** : des valeurs négatives ou impossibles ?
- `type_op` — sens de l'opération (in/out, 2 valeurs) — `in` = dépôt ? retrait ? **à confirmer** (le sens change le sens du CA) ;
- `id_export` — compteur séquentiel — **inutile pour l'analyse** ; suspect : pourquoi un export compte-t-il ses lignes ?

3 questions autorisées (exemples attendus) : « Quel est le taux de succès, après normalisation des 6 statuts ? » — « Quelle est la part des 3 canaux, et que vaut le montant moyen par canal ? » — « Le volume d'opérations suit-il un cycle de jour de semaine ? » **2 questions interdites** (exemples attendus) : « Quel est le **bénéfice** de l'opérateur ? » (pas de coûts dans le fichier) ; « Ce client a-t-il **vraiment** reçu l'argent ? » (le fichier dit l'opération, pas le virement — la confirmation est hors fichier).

E2 (le zéro qui ment). Trois contrôles attendus, dont un non-`isna` :

1. **`isna` colonne par colonne, pas en somme** : `v.isna().sum()` — la somme peut cacher une colonne entière à zéro et une autre à 3 : ici, 0 sur **chaque** des 13 colonnes.
2. **Le contrôle non-`isna`** : les chaînes vides ne sont pas des NaN. `v["date_vente"].str.strip().eq("").sum()` → **0** : pas de date codée comme chaîne vide. (C'est le piège : un export qui code « absent » par "" passe **sourdement** `isna` sur une colonne object.)
3. **Les identifiants se referment-ils sur `client.csv` ?** `v["id_client"].nunique() = 1200` = le nombre de lignes de `client.csv` — pas de clé orpheline. Un « manquant » peut aussi être une **clé qui ne pointe nulle part** : `v.merge(client, on="id_client", how="left")` ne perd **aucune** ligne.

13. Mini-projet M09.P1 — « La fiche d'entrée en 30 minutes » (1 h)

Énoncé. Le chronomètre démarre quand vous ouvrez `03_exercices/dossier_M09/projet/fichier_inconnu.csv`. À 30 minutes, il s'arrête, et la **fiche d'entrée** est rendue :

- la **forme** (lignes, colonnes) ;

- les **10 variables** (sens supposé, unité, source — chaque case dit d'où vient le supposé, « à confirmer » accepté) ;
- **3 questions** que le fichier peut répondre ;
- **2 questions** qu'il ne peut pas ;
- les **2 pièges d'inspection** que `head(5)` / `dtypes` vous ont montrés (et leur preuve, en une sortie chacun).

Les 30 minutes restantes (l'heure totale) sont de **l'auto-évaluation** contre la grille P1 du projet M09.P : forme (1 pt), variables (1 pt), questions autorisées (1 pt), questions interdites (1 pt). Un rapport de fiche à **80 % avec ses cases honnêtes** passe ; une fiche « parfaite » à 45 minutes ne correspond plus au contrat.

Pourquoi ce mini-projet. C'est le **réglage du chronomètre** du projet M09.P : le jour J, vous ne serez pas face à un fichier inconnu **et** à un chronomètre inconnu — vous aurez déjà les deux, ensemble, une fois.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre enchaîne **7 commandes** d'inspection, dans l'ordre du bloc §7 : `read_csv` (lire), `shape` (compter), `dtypes` (natures), `head/tail` (vitrine), `info` (présence et mémoire), `value_counts` (répartition d'une colonne), `nunique` (combien de valeurs différentes) — et `pd.to_datetime` dès qu'une date doit être triée, comparée ou agrégée.

Besoin	Commande	Ce qu'elle dit	Ce qu'elle ne dit pas
compter	<code>v.shape</code>	lignes x colonnes, certain	la qualité des lignes
natures	<code>v.dtypes</code>	types par colonne	les valeurs (une date en object peut être parfaite ou vide)
vitrine	<code>v.head()</code> / <code>v.tail()</code>	les n premières / dernières lignes	l'ensemble (les 5 retours d'affilée)
présence	<code>v.info()</code>	Non-Null Count par colonne, mémoire	les chaînes vides (colonne object)
répartition	<code>v[col].value_counts()</code>	les modalités fréquentes	les modalités absentes (le zéro, C02)
bornes	<code>pd.to_datetime(v[col]).min()/max()</code>	l'étendue réelle	l'ordre du fichier (non trié)
distincts	<code>v[col].nunique()</code>	combien de valeurs différentes	si elles sont correctes (clé orpheline)

15. Résumé du chapitre

M09 part d'un postulat : **le livrable d'une analyse exploratoire est une note provisoire, pas un verdict** — chaque affirmation y porte sa source et sa limite. Le module fournit les trois outils qui font la différence entre une analyse et une dérive : le **protocole en 10 étapes** (un contrat — une étape vide se déclare, une étape passée est un défaut du rapport), la **fiche d'entrée** (variables → sens / unité / source / question autorisée, en **30 minutes maximum**, unités **vérifiées sur les données** — l'affaire des centimes le prouve : 0.05 FCFA d'écart max départagent FCFA et centimes) et le **chronomètre** (3 heures au projet ; la compétence est de **s'arrêter**). L'inspection sans confiance (`shape`, `dtypes`, `head`, `info`) a livré trois leçons exécutées sur le fil rouge : `head(5)` montre 5 retours d'affilée alors que le taux est de 0,4 % (208 sur 50 008) ; les dates ar-

rivent en `object` (ni triées ni comparables avant `pd.to_datetime`) ; et `duplicated().sum()` renvoie 0 alors que le socle porte 8 doublons — parce que le doublon métier est **hors** `id_vente`. Le chapitre se termine sur la fiche d'entrée complète des 13 colonnes de `vente.csv` (montants en FCFA, `taux_tva` en fractions 0.18 → 0.19) et sur les 3 questions autorisées / 2 questions interdites qui cadreront tout le reste du module.

16. À retenir

À retenir. Trois verbes résument le chapitre : **inspecter** sans se fier (la machine dit des faits, on écrit les jugements), **cadrer** en 30 minutes (la fiche avec ses cases ouvertes), et **s'arrêter** (le chronomètre force la hiérarchie). Le reste du module n'est que de l'exécution de ces trois verbes sur trois jeux.

1. **Le protocole est un contrat** : 10 étapes, une étape vide se **déclare**, une étape passée est un **défaut du rapport**.
2. **head(5) est une vitrine** : 5 retours d'affilée sur un taux de 0,4 % (208 sur 50 008) — on regarde, on ne conclut pas.
3. **shape est la seule information certaine** du bloc d'inspection.
4. **Les dates en object ne sont pas des dates** : pas de tri, pas de comparaison, pas d'agrégat avant `pd.to_datetime`.
5. **Les unités se vérifient sur les données** : $\text{montant_ttc} \approx \text{quantite} \times \text{prix_ht} \times (1 + \text{taux_tva})$ (écart max 0.05 FCFA) + `taux_tva` (0.18/0.19) → **FCFA**, pas centimes.
6. **La fiche d'entrée : 30 minutes maximum**, cases « à confirmer » acceptées — 4 à 5 cases ouvertes sur un fichier inconnu, c'est honnête, pas paresseux.
7. **5 questions par fiche** : 3 que le fichier **peut** répondre, 2 qu'il **ne peut pas** — les « ne peut pas » valent les « peut ».
8. **Le chronomètre est un outil d'analyse** : il force la hiérarchie ; la compétence du module est de **s'arrêter** avec des conclusions provisoires.

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi une étape du protocole peut-elle être « vide » mais jamais « passée » ?** → vide : « aucune anomalie détectée, voici les 3 contrôles » ; passée : un silence qui est un défaut du rapport, comme une requête non exécutée en M07. (2 pts)
2. **Les 5 premières lignes de vente.csv sont des retours. Que concluez-vous ? Et pourquoi cette conclusion serait-elle une erreur ?** → Rien, sinon « le fichier commence par 5 retours » ; conclure « base de retours » confond vitrine et ensemble : 208 retours sur 50 008 (0,4 %). (2 pts)
3. **v.duplicated().sum() renvoie 0 sur vente.csv. Le fichier est-il sans doublons ? Un contrôle de quoi ?** → Non : les 8 doublons du socle sont identiques sur les 12 colonnes métier, mais portent des `id_vente` différents ; le contrôle doit être `duplicated(subset=<12 colonnes métier>)`. (2 pts)
4. **Votre collègue affirme que montant_ttc est en centimes. Écrivez la requête qui départage, et sa sortie attendue.** → `(v["montant_ttc"] - v["quantite"]*v["prix_unitaire_ht"]*(1+v["taux_tva"])).abs().max()` → 0.05 (FCFA cohérents) ; en centimes l'écart serait de l'ordre de 100 fois le montant. (2 pts)
5. **taux_tva ne vaut que 0.18 et 0.19. Que dit la fiche d'entrée, et pourquoi ce détail est-il une information métier ?** → « fraction, 0.18 en 2025, 0.19 en 2026 » — le passage de l'un

- à l'autre est un **changement de taux fiscal** (confirmé par `regle_tva.csv`), pas un défaut. (2 pts)
6. **La fiche d'entrée est à 28 minutes avec 5 cases « à confirmer ». Que faites-vous, et pourquoi ?** → Je la rends telle quelle : le contrat est 30 minutes ; les cases se résolvent à l'étape 3 (ou restent, **en le disant**) — 2 minutes de plus, c'est de l'exploration déguisée. (2 pts)
7. **Citez 2 questions que `vente.csv` ne peut pas répondre, et ce qui manque pour y répondre.** → « Pourquoi un retour ? » (pas de motif de retour) ; « Quel client est rentable ? » (pas de coûts) — les causes et les coûts sont **hors fichier**. (2 pts)
8. **Pourquoi le livrable de M09 est-il une « note de conclusions provisoires » et non un tableau de bord ?** → Parce que l'EDA ne prouve pas, elle **dégère des pistes** : chaque affirmation porte sa source (sortie) et sa limite (période, échantillon, variables absentes), révisable aux données suivantes — un tableau de bord affirme, une note provisoire encadre. (2 pts)

Module M09.C02 — Audit et premières statistiques : les 3 familles de défauts, l’audit vide est suspect, cadrer l’hypothèse

Outils : pandas 2.2.3, numpy 2.3.5 (exécutés), DuckDB 1.5.5 (exécuté — le parallèle SQL du dédoublement). **Durée indicative :** 5 h. **Niveau :** N3. **Prérequis :** M09.C01 (le protocole), M02 (les statistiques descriptives), M07 (le parallèle SQL) utile.

L’idée du chapitre. C01 a cadré l’analyse (le protocole, la fiche d’entrée). C02 fait deux choses, dans l’ordre : **trouver les défauts avant qu’ils ne faussent un chiffre** (l’étape 3) et **résumer ce qui reste, juste ce qu’il faut** (l’étape 4) — en **cadrant une hypothèse falsifiable** avant de la tester. Le **centre de santé** y fait son entrée (18 818 consultations, 12 médicaments, 8 767 lignes de stock quotidien) avec **4 défauts plantés et comptables** : 23 motifs manquants, 7 doublons de stock, 1 date dans le futur, 4 stocks négatifs. Et la règle du module y est posée : **l’audit qui ne trouve rien est suspect, pas réconfortant** — le contrôle du contrôle est une section à part entière.

Matériel de l’atelier — Python 3.13 · pandas 2.2.3 · numpy 2.3.5 · DuckDB 1.5.5. Toutes les commandes ont été exécutées dans l’atelier le 20/09/2026 sur `03_exercices/dossier_M09/sante/` (graine 45, déterministe, ATTENDU.json figé) et sur le fil rouge `quincaillerie/` ; les sorties publiées sont **celles de l’atelier**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Classer un défaut dans les 3 familles** (doublons, manquants, valeurs impossibles) et nommer le **contrôle** qui le trouve — la matrice du chapitre.
2. **Conduire le contrôle du contrôle** quand l’audit renvoie zéro partout : 3 contrôles exécutés, et pourquoi un zéro non contrôlé n’est pas un feu vert.
3. **Chiffrer un défaut avec sa définition exacte** : « 7 doublons » n’est pas une définition ; « 7 lignes de `stock_medicament.csv` dont le 5-tuple complet est identique » en est une.
4. **Lire `describe()` colonne par colonne** : médiane vs moyenne sur une distribution asymétrique, quartiles, **iqr**, bornes de Tukey — et ce qu’un **min** hors du réel dit de plus que l’audit.
5. **Cadrer une hypothèse falsifiable** (« si c’est une saisonnalité, les étés doivent ressembler plus entre eux qu’à leurs hivers ») **avant** de la tester.

2. Pourquoi cette notion est importante

Un défaut non trouvé ne fausse pas « un peu » un chiffre : il fausse **tous** les chiffres qui passent sur la colonne. Un motif manquant fausse le taux par motif ; un doublon de stock fausse le taux de rupture ; une date dans le futur fausse la période de la série ; un stock négatif fausse la médiane. Le protocole le sait : l’étape 3 existe **avant** l’étape 4, et l’étape 4 existe **avant** les étapes 5 à 10.

Trois faits mesurés sur le socle fixent l'enjeu :

- l'audit du **centre de santé** (le jeu 2, qui arrive ici) trouve **4 familles de défauts comptables** : 23 motifs manquants, 7 doublons de stock, 1 consultation datée au 2027-01-05 (après la fin de la période 2025-01-01 → 2026-12-31), 4 stocks négatifs ;
- le **fil rouge** (la quincaillerie) passe **l'audit vide** : 0 manquant sur 50 008 lignes et 13 colonnes — et c'est justement sur lui que le chapitre montre les **3 contrôles** qui transforment un zéro en résultat ;
- la statistique n'est pas un verdict : sur les consultations/jour, la **moyenne est 25.74** et la **médiane 29.0** — un écart de 3.26 consultations que seule la question métier départage (C02 §5.4).

3. Explication simple — la conversation avec la machine

Le contrôleur qualité qui reçoit une palette de médicaments fait trois passages, **dans l'ordre** :

1. **Compter les paquets en trop.** Un même carton deux fois = un doublon ; on le compte, on l'étiquette, on ne le jette pas sans écriture. (`uplicated()`)
2. **Chercher les étiquettes qui manquent.** Une case vide sur 18 818 paquets n'est pas un détail : c'est une information manquante qu'on comptera et qu'on traitera **après** l'audit, pas pendant. (`isna()` — et les chaînes vides que `isna` laisse passer)
3. **Vérifier le possible.** Un stock de -12 boîtes de cotrimoxazole, entre 548 et 541 ? Impossible : c'est un défaut **de saisie**, pas un fait de stock. Les bornes métier (un stock ne peut pas être négatif) et la **récurrence** (hier + entrées - sorties) le prouvent.

Puis, **un seul passage de résumé** : pas 40 statistiques, les 5 qui répondent à la question métier (« quand le centre est-il saturé ? ») — moyenne, médiane, quartiles, extrêmes, et la répartition par motif.

Et la leçon du chapitre tient en une phrase : **une boîte vide qu'on n'a pas ouverte n'est pas une preuve de propreté** — l'audit qui renvoie zéro doit être **re-vérifié** (les bonnes colonnes ? les bonnes définitions ?), sinon on n'a pas audité.

4. Vocabulaire essentiel

Terme	Définition
iqr — interquartile range	Q3 - Q1 : la largeur du « cœur » de la distribution (les 50 % du milieu), indépendante des extrêmes — c'est elle qui cale les bornes de Tukey.
point aberrant — outlier	une observation au-delà d'une borne définie avant de chercher (Q3 + 1,5 × iqr, borne métier, comparaison à la période) ; « aberrant » est un statut mesuré, pas une impression.
manquant	une valeur absente : <code>NaN</code> ou chaîne vide codée en clair — les deux formes doivent être comptées (C02 §5.1).
borne métier	une limite dictée par le métier, pas par la statistique : un stock ≥ 0, une note entre 0 et 20, une date dans la période de l'export.

5. Cours approfondi

5.1 Les 3 familles de défauts — et le contrôle qui trouve chacune

Famille	Définition exacte	Contrôle pandas	Exemple chiffré du socle
doublons	lignes (ou clés) répétées	<code>duplicated(subset=...)</code>	7 doublons de stock sur 8 767 lignes

Famille	Définition exacte	Contrôle pandas	Exemple chiffré du socle
manquants	valeur absente (NaN ou "")	<code>isna().sum()</code> <code>str.strip().eq("").sum()</code>	+ 23 motifs vides sur 18 818 consultations
valeurs impossibles	hors borne métier ou hors récurrence	bornes métier, min/max, récurrence	4 stocks négatifs (min -12)

Trois précisions qui font la différence entre un audit et une check-list :

- **Les doublons se définissent avant de compter.** `stk.duplicated()` (les 5 colonnes) compte 7 sur `stock_medicament.csv` ; mais sur `vente.csv` (fil rouge), le contrôle naïf comptait 0 alors que 8 doublons métier existent (C01 §5.3) : le doublon s'y définit **hors id_vente**, sur les 12 colonnes métier. La définition fait le chiffre.
- **Les manquants ont deux visages.** `isna()` trouve les NaN — pas les chaînes vides codées en clair. Sur les colonnes texte, le contrôle complet est `isna().sum() + str.strip().eq("").sum()`. (Sur `vente.csv`, les deux visages sont vides — exécuté au §5.2.)
- **Les valeurs impossibles se prouvent deux fois.** Une fois par la **borne métier** (un stock ne peut pas être -12), une fois par la **récurrence** du flux (hier + entrées - sorties doit éгалer aujourd'hui) — le §6 montre la preuve complète sur M12.

Définition. borne métier — une limite dictée par le métier, pas par la statistique : un stock ≥ 0 , une note entre 0 et 20, une date dans la période de l'export. Une valeur hors borne n'est pas « aberrante » au sens statistique : elle est **impossible**, et c'est ce statut qui donne droit à la correction (C04), pas à la simple suppression.

Définition. iqr — interquartile range — $Q3 - Q1$: la largeur du « cœur » de la distribution (le milieu des 50 %), indépendante des extrêmes. C'est elle, pas l'écart-type, qui cale les bornes de Tukey [$Q1 - 1,5 \times iqr$; $Q3 + 1,5 \times iqr$] — sur une distribution asymétrique ou bornée, l'écart-type s'emballe, l'iqr non.

L'audit (étape 3) : 3 familles de défauts, 3 contrôles, 3 exemples chiffrés

Chaque défaut porte sa définition exacte — sans elle, le chiffre n'est pas publiable (P2 lit la définition).

FAMILLE	DÉFINITION EXACTE	CHIFFRE DU SOCLE
Doublons	lignes (ou clés) répétées — définies AVANT de compter contrôle : <code>duplicated(subset=clés métier)</code>	7 lignes de stock (5-tuple identique) · 8 métier hors id_vente
Manquants	valeur absente : NaN OU chaîne vide codée en clair contrôle : <code>isna().sum()</code> + chaînes vides sur les textes	23 motifs vides sur 18 818 consultations
Impossible	hors borne métier ou hors récurrence du flux contrôle : bornes métier + récurrence (hier + in - out)	4 stocks négatifs (min -12) · 1 date au 2027-01-05
L'audit vide est suspect, pas réconfortant : 3 contrôles avant de publier « propre ». (1) <code>isna</code> par colonne (pas en somme) · (2) chaînes vides sur les textes (3) clés refermées sur les tables jointes — sur <code>vente.csv</code> : 0 / 0 / 0 vérifié = audit validé vide.		
Puis l'étape 4 : les statistiques juste nécessaires — médiane 29.0 vs moyenne 25.74/jour, iqr, bornes de Tukey (442 vs -12).		

5.2 L'audit vide est suspect — le contrôle du contrôle

Sur le fil rouge, l'audit « standard » renvoie zéro partout :

```
v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv")
c = pd.read_csv("03_exercices/dossier_M09/quincaillerie/client.csv")
print(int(v.isna().sum().sum())) # 0
print(int(v["date_vente"].str.strip().eq("").sum())
      + int(v["date_limite_remise"].str.strip().eq("").sum())) # 0
print(int(v["id_client"].nunique()), len(c)) # 1200 1200
print(int(len(v.merge(c, on="id_client", how="left")) - len(v))) # 0
```

```
0
0
1200 1200
0
```

Le zéro est **vérifié**, pas supposé :

1. **isna par colonne, pas en somme** — la somme peut additionner un zéro d'une colonne et un manquant d'une autre ; ici, 0 sur **chaque** des 13 colonnes.
2. **Les chaînes vides** — un export qui code « absent » par "" passe **sourdement** **isna** sur une colonne object : 0 chaîne vide sur les 2 dates.
3. **Les clés se referment-elles ?** — 1 200 clients distincts = 1 200 lignes de **client.csv**, 0 ligne orpheline après le **merge**.

La règle du chapitre : un audit qui ne trouve rien est **suspect** tant que ces 3 contrôles n'ont pas passé — parce qu'un zéro peut venir d'un **mauvais contrôle** (mauvaise colonne, mauvaise définition de doublon, chaîne vide non comptée) autant que d'un fichier propre. L'audit du fil rouge est **validé vide** : 0 manquant **vérifié**, 8 doublons métier (hors **id_vente**, C01), et c'est tout.

Attention. Un zéro d'audit n'est pas un résultat, c'est une **hypothèse** : « ce fichier n'a pas de manquant » — et une hypothèse non contrôlée (mauvaise colonne, chaîne vide non comptée, clé non refermée) est l'erreur la plus chère du chapitre, parce que toutes les statistiques d'après partent dessus.

Définition. manquant — une valeur absente, sous deux formes : le NaN que **isna()** compte, et la **chaîne vide** codée en clair que **isna()** laisse passer. Un audit complet compte **les deux** — c'est la distinction qui a fait l'erreur « 0 manquant » de plus d'un stagiaire sur un export object.

5.3 Le centre de santé arrive — les 4 défauts, exécutés

Le jeu 2 s'ouvre : **consultation.csv** (18 818 lignes × 7 colonnes), **stock_medicament.csv** (8 767 lignes × 5 colonnes), **medicament.csv** (12 médicaments × 4 colonnes, dont le seuil d'alerte). La fiche d'entrée (C01) est faite ; l'audit (étape 3) tourne :

```
import pandas as pd
s = "03_exercices/dossier_M09/sante"
cons = pd.read_csv(s + "/consultation.csv")
stk = pd.read_csv(s + "/stock_medicament.csv")
print(int(cons["motif"].isna().sum())) # manquants
print(int((cons["date_consultation"] > "2026-12-31").sum())) # dates futures
print(int(stk.duplicated().sum())) # doublons
print(int((stk["stock_fin"] < 0).sum())) # stocks négatifs
```

23
1
7
4

Les 4 défauts, **chiffrés avec leur définition exacte** (le format du rapport P2) :

#	Défaut	Définition exacte	Chiffre
D1	motifs manquants	consultation.csv, colonne motif, champ vide	23 sur 18 818
D2	doublons de stock	stock_medicament.csv, 5-tuple complet identique (date, médicament, entrées, sorties, stock)	7 sur 8 767
D3	date dans le futur	consultation hors période de l'export (2025-01-01 → 2026-12-31)	1 — C000001, datée au 2027-01-05
D4	stock négatif	stock_fin < 0, impossible par borne métier	4 — M05 (-1), M07 (-3), M10 (-2), M12 (-12)

Deux lectures qui feront la différence au rapport :

- **D3 n'est pas « une anomalie de date »**, c'est une **hypothèse de saisie** : la consultation C000001 porte la première référence du fichier et une date au 2027-01-05 — soit une coquille pour 2025-01-05 (la période commence ce jour-là), soit une erreur de transcription. L'audit **compte** le défaut ; la correction se décidera **après** (C04), avec la preuve.
- **D4 est un défaut de saisie, pas un fait de stock** — le §6 prouve le cas M12 par la récurrence. Un stock « négatif » n'existe pas en pharmacie : la valeur est fausse, le médicament n'était pas « en dette ».

Dans les faits. L'audit complet du centre de santé (formes + 4 familles + clés : 1 500 patients distincts, 6 agents, 12 médicaments tous refermés sur `medicament.csv`) prend **une vingtaine de lignes** pandas. Ce qui prend du temps, ce n'est pas le code : c'est **écrire chaque définition exacte** — c'est elle qui sera notée au rapport P2.

5.4 Premières statistiques — juste ce qu'il faut

L'étape 4 ne résume **pas tout** : elle résume ce que la question métier (« quand le centre est-il saturé ? ») nécessite. Sur la fréquentation, la distribution des consultations par jour (exécuté) :

```
moyenne 25.74 | médiane 29.0 | Q1 19.0 | Q3 33.0 | min 1 | max 44
iqr = 14.0    | bornes de Tukey [-2.0 ; 54.0] | jours hors bornes : 0
```

Moyenne 25.74, médiane 29.0 — l'écart de 3.26 se lit : la moyenne est **tirée vers le bas** par une queue de jours peu consultés (les dimanches du calendrier du centre, min 1). « Le centre fait 26 consultations par jour » (moyenne) et « la moitié des jours font 29 ou plus » (médiane) sont **deux phrases vraies et différentes** ; c'est la question métier qui choisit — pour la saturation, c'est la médiane (le jour typique), pour la capacité d'achat de médicaments, c'est la moyenne (le total).

`describe()` lu **colonne par colonne**, sur `stock_fin` :

```
count    8767.00
mean     663.76
std      557.45
min      -12.00
25%      319.00
50%      463.00
75%      607.00
max      2910.00
```

Trois choses à en retenir :

1. **Le min -12 est un audit, pas une statistique** : `describe()` vient de redécouvrir le défaut D4 **sans qu'on lui demande** — c'est le réflexe du chapitre : le `min` et le `max` d'un `describe()` se lisent toujours **en premier**.
2. **L'écart-type (557.45) est sans sens ici** : les stocks de 12 médicaments de régimes très différents (insuline ~150, paracétamol ~800) sont mélangés dans la même colonne — un écart-type sur un mélange de populations ne mesure rien. La statistique « juste nécessaire » serait **par médicament** (C04 le fera pour les ruptures).
3. **Les bornes de Tukey se calculent, ne se choisissent pas** : sur M12 seul (Q1 518, Q3 566, iqr 48), la borne basse est **446** — c'est-à-dire 518 moins 72, où 72 est 1,5 fois l'iqr 48 — et **deux** valeurs y tombent dessous : **442** et **-12**. La borne ne fait que **signaler** : 442 est un stock bas mais plausible, -12 est impossible — la décision (C04) les départage.

Attention. Une borne de Tukey qui **signale** n'est pas une borne qui **décide** : 442 et -12 tombent du même côté de 446 et reçoivent deux décisions opposées — la borne ouvre l'enquête (C04), elle ne la conclut pas. Supprimer « ce qui dépasse la borne » sans preuve métier, c'est transformer un signal en délit.

Définition. point aberrant — outlier — une observation au-delà d'une borne définie **avant** de chercher : $Q3 + 1,5 \times \text{iqr}$ (Tukey), une borne métier (stock ≥ 0 , note ≤ 20), ou la comparaison à la période. « Aberrant » est un **statut mesuré** (ici : « 2 valeurs sous 446 pour M12 »), jamais une impression (« ça fait bizarre »).

La dernière statistique de l'étape 4 est la **répartition** : `value_counts()` sur le motif (non vide) : paludisme en tête avec 2 417 consultations, et les 7 autres motifs suivent à quelques dizaines de lignes près — une distribution **plate**, elle-même une information (centre polyvalent, pas de motif dominant). Le creux du midi (927 consultations à 12:00, soit 4,9 % des heures alors que 10 tranches horaires donneraient 10 %) est noté, pas expliqué : c'est du C03.

5.5 Cadrer l'hypothèse — la formuler falsifiable avant de la tester

L'étape 8 du protocole est une **discipline d'écriture**, pas de calcul : avant de tester quoi que ce soit, l'hypothèse se formule de façon à pouvoir **se tromper**. Le gabarit du module :

« Si [mécanisme métier], alors [signature chiffrée dans les données], et [contre-exemple qui tuerait l'hypothèse]. »

Exemple du socle (cadre posé ici, test chiffré au C03) :

« Si la fréquentation du centre est saisonnière (saison des pluies), alors les 8 mois d'été (juin-sept, 2025 et 2026) doivent ressembler plus entre eux qu'à leurs hivers (déc-jan), et un été atypique — disons 20 % sous la moyenne des étés — tuerait l'hypothèse. »

Le test (exécuté, en une ligne, à titre d'annonce — la méthode est au C03) :

```

moyenne des 8 étés : 853 consultations/mois
moyenne des hivers : 655 consultations/mois
autres mois       : 727 consultations/mois
été le plus bas   : 834 | mois max : 2025-07 (882)

```

L'hypothèse est **portée** (853 vs 655, et les 8 étés serrés entre 834 et 882) — mais la conclusion n'est **pas** « c'est la saison des pluies » : c'est « **les données sont compatibles avec** une saisonnalité, testée aux étés 2025-2026 » — la conclusion **provisoire** (C01 §5.5), avec ses limites (2 étés seulement, cause hors fichier).

À retenir. Une hypothèse se **cadre** (une phrase falsifiable, une signature chiffrée, un contre-exemple nommé) **avant** de se tester. Tester d'abord, formuler après, c'est choisir l'hypothèse qui colle aux chiffres — c'est du C05 §5.2 qu'on appelle un **piège d'inférence**.

6. Exemple concret — le défaut M12 prouvé par la récurrence

Le stock négatif le plus fort (−12, médicament M12, 2026-11-03) est prouvé défaut de saisie par **deux preuves indépendantes** (exécuté) :

Preuve 1 — la borne métier. Un stock ne peut pas être négatif : la pharmacie n'a pas « moins de 12 boîtes », elle a **zéro boîte** et une erreur de saisie.

Preuve 2 — la récurrence du flux. Le stock suit hier + entrées − sorties = aujourd'hui. Sur les 3 jours autour du défaut :

date	id_medicament	entree	sortie	stock_fin
2026-11-02	M12	0	3	548
2026-11-03	M12	0	3	-12 <-- défaut
2026-11-04	M12	0	4	541

$548 + 0 - 3 = 545$, pas −12 : la valeur « naturelle » du 2026-11-03 est **545**. Et le lendemain confirme : $545 - 4 = 541$ — la série **continue comme si le défaut n'avait jamais existé**. Un stock réellement à −12 ne pourrait pas « ressusciter » à 541 le jour suivant **sans 553 entrées** ; il n'y en a pas. La valeur −12 est un défaut de saisie, et sa valeur correcte est **déductible** (545) — la correction se décide au C04.

7. Démonstration pas à pas — 5 étapes sur le centre de santé

```
import pandas as pd
s = "03_exercices/dossier_M09/sante"
cons = pd.read_csv(s + "/consultation.csv")
stk = pd.read_csv(s + "/stock_medicament.csv")
med = pd.read_csv(s + "/medicament.csv")

# 1 – formes (contrôle de chargement)
print(cons.shape, stk.shape, med.shape)

# 2 – audit consultation : 3 familles + clés
print(int(cons["motif"].isna().sum()),
      int((cons["date_consultation"] > "2026-12-31").sum()),
      int(cons["id_patient"].nunique()))

# 3 – audit stock : 3 familles + récurrence du défaut le plus fort
print(int(stk.duplicated().sum()), int((stk["stock_fin"] < 0).sum()))
m12 = stk[stk["id_medicament"] == "M12"].sort_values("date")
print(m12[m12["date"].between("2026-11-02", "2026-11-04")].to_string(index=False))

# 4 – statistiques : fréquentation + describe du stock
pj = cons.groupby("date_consultation").size()
print(round(pj.mean(), 2), pj.median())
print(stk["stock_fin"].describe().min())

# 5 – cadrage : moyennes par saison (le test est au C03)
cj = pd.to_datetime(cons["date_consultation"])
mj = cons.assign(m=cj.dt.to_period("M")).groupby("m").size()
print(round(mj[mj.index.month.isin([6, 7, 8, 9])].mean()),
      round(mj[mj.index.month.isin([12, 1])].mean()))
```

```
(18818, 7) (8767, 5) (12, 4)
23 1 1500
7 4
   date id_medicament  entree  sortie  stock_fin
2026-11-02          M12      0      3         548
2026-11-03          M12      0      3         -12
2026-11-04          M12      0      4         541
25.74 29.0
-12.0
853 655
```

8. Erreurs fréquentes

1. **Compter les doublons sans les définir** — `duplicated()` (toutes colonnes) vs `duplicated(subset=clés métier)` : 0 vs 8 sur `vente.csv` (C01), 7 vs 7 sur le stock. La définition précède le chiffre, toujours.
2. **Prendre l'audit vide comme feu vert** — 0 manquant sans les 3 contrôles du §5.2 (colonnes, chaînes vides, clés) n'est pas un résultat, c'est une **hypothèse** — et l'hypothèse la plus chère qu'on puisse laisser courir.
3. **Publier la moyenne seule** — 25.74 de moyenne, 29.0 de médiane : publier l'une sans l'autre, c'est choisir **pour** le lecteur. Sur une distribution asymétrique, les deux vont, ou aucune ne va seule.
4. **Formuler l'hypothèse après le test** — « les étés font 853, les hivers 655, donc c'est saisonnier » : c'est l'hypothèse **choisie pour coller** aux chiffres. Le cadre falsifiable (§5.5) est écrit **avant**, et il nommait son contre-exemple.

5. **Traiter l'anomalie à la borne** — « -12 est sous la borne de Tukey, on supprime » : la borne **signale**, elle ne **décide** pas. 442 (M12) est aussi sous la borne et est un stock plausible — le traitement sans preuve métier, c'est le C04 en avance, fait mal.

9. Bonnes pratiques professionnelles

1. **L'audit se publie avant la statistique** — le rapport P2 présente d'abord les défauts chiffrés (définition exacte + chiffre), ensuite les statistiques : un chiffre calculé **avec** des doublons n'a pas besoin d'être présenté, il a besoin d'être recalculé.
2. **Chaque défaut porte une définition exacte** — « 7 doublons » n'est pas publiable ; « 7 lignes de `stock_medicament.csv` dont le 5-tuple (date, médicament, entrées, sorties, stock) est identique » l'est. Le correcteur relit la définition, pas le chiffre.
3. **min et max d'un describe() se lisent en premier** — c'est là que les bornes métier se trompent (-12) et que les valeurs impossibles se trahissent (22.5 sur 20 au C05).
4. **Une statistique, une question** — la médiane pour « le jour typique », la moyenne pour « la capacité d'achat », l'iqr pour « la variabilité du cœur » : publier 40 statistiques sans questions, c'est de l'exploration déguisée en rapport.
5. **L'hypothèse s'écrit, ne se pense pas** — une phrase falsifiable, une signature chiffrée, un contre-exemple nommé : ce qui n'est pas écrit n'est pas testable, et ce qui n'est pas testable n'entre pas dans la note.

Conseil professionnel. Dans un rapport d'audit, la ligne la plus précieuse n'est pas « 7 doublons trouvés », c'est « **7 doublons, donc 8 760 lignes propres après dédoublonnage, et voici la ligne qui justifie le keep** ». Le chiffre brut est pour vous ; le chiffre **traité et justifié** est pour le lecteur.

10. Exercice guidé — « les 4 défauts du centre de santé » (25 min, /10)

Consigne. Sans ouvrir `ATTENDU.json`, sur `sante/consultation.csv` et `sante/stock_medicament.csv` :

1. Trouvez et **chiffrez** les 4 familles de défauts (1 pt chacune, 4 pts).
2. Écrivez la **définition exacte** de chacun (1 pt — la définition sans le chiffre ne compte pas, le chiffre sans la définition non plus). (3 pts)
3. Pour **D4 (stock négatif)** : donnez la preuve par récurrence du cas le plus fort, avec les 3 lignes exécutées. (2 pts)
4. Donnez pour chaque défaut la **décision** du rapport : corriger / exclure en documentant / garder en signalant — et en une phrase pourquoi. (1 pt)

11. Exercices autonomes

E1 — Le parallèle SQL du dédoublonnage (20 min). En DuckDB sur `stock_medicament.csv` : (1) `COUNT(*)` vs le nombre de 5-tuples distincts (l'écart doit donner le nombre de doublons) ; (2) une requête unique qui liste les 4 stocks négatifs (médicament, date, valeur), triée par gravité. Les deux sorties sont publiées telles que renvoyées par DuckDB.

E2 — Le faux positif de Tukey (20 min). Sur M12 : deux valeurs sous la borne basse (446) — 442 et -12. Pour chacune : statut (point aberrant **confirmé** / simple valeur basse), la preuve (récurrence ou borne métier), et la décision (corriger / exclure / garder en signalant). Indice : 442 est **plausible** — trouvez la phrase qui le prouve sans le chiffre 545.

12. Correction détaillée

Exercice guidé.

1. **Les 4 familles, exécutées** : 23 motifs vides ; 7 doublons de stock ; 1 date future (2027-01-05) ; 4 stocks négatifs (-1, -3, -2, -12).
2. **Les définitions exactes** : celles du tableau §5.3 — le point de vigilance est D2 (« 5-tuple complet identique », pas « même médicament en double ») et D3 (« hors période de l'export », pas « date anormalement tardive »).
3. **La preuve M12** : $548 + 0 - 3 = 545$ (la valeur naturelle), pas -12 ; et $545 - 4 = 541$ le lendemain — la série continue **comme si** le défaut n'existait pas. (Les 3 lignes sont celles du §6.)
4. **Les décisions** : D1 — **garder en signalant** (l'information est perdue, on ne l'invente pas : 23 consultations hors des taux par motif) ; D2 — **exclure en documentant** (7 lignes supprimées, `keep="first"`, la justification est dans le rapport) ; D3 — **exclure de la série temporelle en documentant**, coquille probable pour 2025-01-05 (à confirmer avec la source, C04) ; D4 — **garder en signalant** pour l'instant, correction **déductible** (545 pour M12) décidée au C04 après la preuve.

E1. DuckDB, exécuté :

```
SELECT COUNT(*), COUNT(DISTINCT (date, id_medicament, entree, sortie, stock_fin))
FROM stock;
      lignes  lignes_distinctes
      8767      8760

SELECT id_medicament, date, stock_fin
FROM stock WHERE stock_fin < 0 ORDER BY stock_fin;
id_medicament  date      stock_fin
M12 2026-11-03      -12
M07 2025-10-09      -3
M10 2026-05-21      -2
M05 2025-04-18      -1
```

$8\,767 - 8\,760 = 7$ **doublons** : le parallèle SQL redonne le chiffre pandas (la colonne vertébrale du module, comme la croisée de M08).

E2. 442 — simple valeur basse, **pas** un point aberrant confirmé : c'est un stock de M12 qui se situe juste sous la borne calculée, mais il est **cohérent avec sa récurrence** (le jour et le lendemain se referment sur lui) et **plausible en métier** (M12 tourne autour de 500-560 ; 442 est un jour bas, pas une impossibilité) — décision : **garder**, rien à signaler au rapport (la borne de Tukey n'est qu'un signal, le métier a départagé). **-12** — point aberrant **confirmé** par double preuve (borne métier : un stock négatif n'existe pas ; récurrence : $548 - 3 = 545 \neq -12$ et $545 - 4 = 541$) — décision : **corriger en 545**, la correction étant **déductible** de la récurrence, et le tout documenté au rapport (C04 formalisera la règle).

13. Mini-projet M09.P2 — « Le rapport d'audit en 1 page » (1 h)

Énoncé. Le rapport P2 du projet M09.P commence par les étapes 1 à 4 **sur le centre de santé, en une page** :

- les **formes** des 3 tables (1 ligne) ;
- les **4 défauts** : définition exacte + chiffre + décision (corriger / exclure en documentant / garder en signalant) (4 lignes) ;
- les **3 statistiques** de fréquentation qui répondent à « quand le centre est-il saturé ? » (moyenne, médiane, mois max) (1 ligne) ;

- **1 hypothèse cadrée** (mécanisme, signature chiffrée, contre-exemple) — à tester au C03.

Les 30 minutes restantes : auto-évaluation — chaque défaut a-t-il sa **définition** ? chaque statistique a-t-elle sa **question** ? l'hypothèse est-elle **falsifiable** (son contre-exemple est-il nommé) ? Un rapport à 80 % dont chaque ligne est justifiée passe ; une page « complète » dont une définition manque n'est pas un rapport d'audit.

14. Boîte à outils du chapitre

Boîte à outils. L'audit : `isna().sum()` (+ `str.strip().eq("")` sur les textes), `duplicated(subset=...)` avec sa **définition**, bornes métier et min/max de `describe()`, récurrence du flux. La statistique : `describe()`, `quantile(.25/.75)`, `value_counts()`, moyenne **et** médiane ensemble. Le parallèle : `COUNT(*)` vs `COUNT(DISTINCT ...)` en DuckDB — même nombre, deux écritures.

Besoin	Commande	Chiffre du socle
manquants	<code>col.isna().sum()</code>	23 motifs
doublons	<code>df.duplicated(subset=clés).sum()</code>	7 lignes de stock
hors période	<code>(col_date > fin_période).sum()</code>	1 consultation
hors borne	<code>(col < 0).sum()</code>	4 stocks
récurrence	<code>hier + entree - sortie == aujourd'hui ?</code>	545 ≠ -12 (M12)
forme du cœur	<code>quantile(.75) - quantile(.25)</code>	iqr 14 (consultations/jour)

15. Résumé du chapitre

L'étape 3 du protocole (détecter les problèmes) et l'étape 4 (premières statistiques) sont exécutées sur le centre de santé, qui fait son entrée dans le module : **4 familles de défauts comptables** (23 motifs manquants, 7 doublons de stock sur 8 767 lignes, 1 consultation datée au 2027-01-05, 4 stocks négatifs dont -12) — chacune avec sa **définition exacte**, son chiffre, et sa décision (garder en signalant, exclure en documentant). Le défaut M12 est prouvé **défaut de saisie** par deux preuves indépendantes : la borne métier (un stock négatif n'existe pas) et la récurrence (548 - 3 = 545, pas -12 ; le lendemain, 545 - 4 = 541 — la série continue comme si le défaut n'existait pas). Sur le fil rouge, l'**audit vide** est validé par les 3 contrôles du chapitre (colonnes, chaînes vides, clés refermées) : un zéro non contrôlé n'est pas un feu vert, c'est une hypothèse. La statistique n'est pas un verdict : moyenne 25.74 et médiane 29.0 consultations/jour coexistent (distribution à queue basse), le min -12 d'un `describe()` redécouvre l'audit, et l'iqr cale des bornes qui **signalent** (442 et -12 sous 446 pour M12) sans **décider**. Le chapitre se termine sur la discipline d'écriture : une hypothèse se **cadre falsifiable avant de se tester** — la saisonnalité du centre (étés 853 vs hivers 655 consultations/mois) est posée ainsi, test chiffré au C03.

16. À retenir

À retenir. L'audit **avant** la statistique, la statistique **avant** l'hypothèse testée, et l'hypothèse cadrée **avant** d'être testée : c'est l'ordre des étapes 3, 4 et 8, et c'est tout le chapitre. Chaque défaut porte sa définition exacte, chaque zéro ses 3 contrôles, chaque statistique sa question métier.

1. **3 familles de défauts, 3 contrôles** : doublons (`uplicated(subset=...)` **défini** avant), manquants (`isna + chaînes vides`), valeurs impossibles (borne métier + récurrence).
2. **L'audit vide est suspect** tant que les 3 contrôles du contrôle n'ont pas passé : colonnes, chaînes vides, clés refermées.
3. **Chaque défaut porte une définition exacte** — « 7 doublons » n'est pas publiable, « 7 lignes dont le 5-tuple est identique » l'est.
4. **min/max d'un `describe()` se lisent en premier** — c'est là que le métier se trompe dans la saisie (−12).
5. **Moyenne et médiane vont ensemble** — 25.74 vs 29.0 : l'écart est une information (queue de jours bas), pas un bug.
6. **La borne signale, le métier décide** — 442 est sous la borne et plausible ; −12 est sous la borne et impossible : même test, deux décisions.
7. **Une hypothèse se cadre avant de se tester** — mécanisme, signature chiffrée, contre-exemple nommé ; l'été 20 % sous la moyenne tuerait la saisonnalité.
8. **Le parallèle SQL redonne le chiffre** — `COUNT(*)` 8 767 vs `COUNT(DISTINCT)` 8 760 : 7 doublons, comme en pandas.

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi `duplicated()` sans argument a-t-il compté 0 sur `vente.csv` alors que le socle porte 8 doublons ?** → les 8 doublons sont identiques sur les 12 colonnes métier mais portent des `id_vente` différents : le contrôle doit être `duplicated(subset=<12 colonnes métier>)`. (2 pts)
2. **Un audit renvoie 0 manquant sur toutes les colonnes. Que faites-vous avant de publier « fichier propre » ?** → les 3 contrôles du contrôle : `isna` par colonne (pas en somme), chaînes vides sur les colonnes texte, clés refermées sur les tables jointes. (2 pts)
3. **Pourquoi la consultation `C000001` datée au 2027-01-05 est-elle une « hypothèse de saisie » et non une « anomalie de date » ?** → c'est la première référence du fichier et la période commence au 2025-01-05 : la coquille 2025-01-05 est la lecture la plus probable — l'audit compte, la correction se décide après preuve (C04). (2 pts)
4. **Le stock M12 vaut −12 le 2026-11-03, entre 548 et 541. Donnez la preuve par récurrence.** → $548 + 0 - 3 = 545$ (valeur naturelle), pas −12 ; et $545 - 4 = 541$ le lendemain : la série continue comme si le défaut n'existait pas — un stock à −12 ne « ressuscite » pas sans 553 entrées. (2 pts)
5. **Moyenne 25.74, médiane 29.0 consultations/jour. Que dit l'écart, et quelle statistique pour « le jour typique » ?** → une queue de jours peu consultés tire la moyenne vers le bas (les dimanches, min 1) ; « le jour typique » se lit à la **médiane** (29 ou plus le moitié des jours). (2 pts)
6. **Pourquoi l'écart-type de `stock_fin` (557.45) est-il « sans sens » ?** → la colonne mélange 12 médicaments de régimes très différents (insuline ~150, paracétamol ~800) : un écart-type sur un mélange de populations ne mesure rien — la statistique juste est **par médicament**. (2 pts)

7. **M12 a deux valeurs sous la borne basse de Tukey (446) : 442 et -12. Quelle décision pour chacune, et pourquoi la borne n'a pas départagé ?** → 442 : garder (cohérent avec sa récurrence, plausible en métier) ; -12 : corriger en 545 (preuve double). La borne **signale**, le métier **décide** — c'est la récurrence et la borne métier qui départagent. (2 pts)
8. **Écrivez le cadre falsifiable de l'hypothèse « la fréquentation est saisonnière », avec son contre-exemple.** → « Si saison des pluies, les 8 étés ressemblent plus entre eux qu'aux hivers (déc-jan) ; un été 20 % sous la moyenne des étés tue l'hypothèse » — signé, chiffré, testable. (2 pts)

Module M09.C03 — Tendances : la série, la saison, le bruit — et les périodes qu'on a le droit de comparer

Outils : pandas 2.2.3, numpy 2.3.5 (exécutés), DuckDB 1.5.5 (exécuté — le parallèle SQL des tendances). **Durée indicative :** 5 h. **Niveau :** N3. **Prérequis :** M09.C01-C02 (protocole, audit), M02 (statistiques), M08 C08 (resample, set_index), M07 (le GROUP BY mois SQL) utile.

L'idée du chapitre. L'étape 5 du protocole pose la question : « **ça monte, ça descend, ça cycle ?** » C03 y répond avec une discipline : décomposer une série en **tendance / saisonnalité / bruit** (à l'œil et au chiffre), ne comparer que des périodes à **taille égale** (le 28 décembre du socle est le contre-exemple chiffré), et savoir déclarer une tendance **absente** — la quincaillerie du fil rouge est quasi plate, et c'est exactement ce que le chapitre doit apprendre à dire. Le centre de santé, lui, porte la vraie saisonnalité : le pic de juillet (882 consultations), le creux du dimanche (405) et du midi (927), et la rupture de stock qui **coupe** une tendance sans la **retomber**.

Matériel de l'atelier — Python 3.13 · pandas 2.2.3 · numpy 2.3.5 · DuckDB 1.5.5. Toutes les commandes ont été exécutées dans l'atelier le 20/09/2026 sur 03_exercices/dossier_M09/ (graine 45, déterministe, ATTENDU.json figé) ; les sorties publiées sont **celles de l'atelier**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Construire la série simple** en pandas : `set_index` + `resample` (le 'ME' de M08) — et reconstruire la vue mensuelle de M07/M08 (120 lignes : 5 magasins × 24 mois) sans SQL.
2. **Décomposer une série** en tendance, saisonnalité et bruit — à l'œil (moyenne glissante) **et** au chiffre (amplitude de la saison vs variabilité résiduelle).
3. **Comparer des périodes à taille égale** : à mois égal, au flux par jour — et identifier la **période partielle** qui ment (le décembre 2026 du socle, arrêté au 28).
4. **Lire la saisonnalité du jour de semaine et de l'heure** (`dayofweek == 6`, rappel M08) : le dimanche du centre (405 consultations), la journée type, le creux du midi.
5. **Distinguer structurel et bruit** — et **déclarer l'absence** de tendance (une étape vide se déclare, C01) : la quincaillerie n'a pas de saison des fêtes, et ça se dit.
6. **Reconnaître la tendance interrompue** : la rupture de stock qui coupe la courbe sans la retomber (M02, 5 jours à zéro — la demande revient au même niveau au restock).

2. Pourquoi cette notion est importante

Toutes les décisions de prévision de capacité partent de la lecture d'une courbe : commander des médicaments pour l'été, prévoir du personnel pour le pic du matin, arbitrer un budget magasin par magasin. Une lecture fausse de la tendance — prendre le bruit pour une saison, la saison pour une dérive, une période partielle pour une année — transforme une exploration en **erreur de prévision** : le centre de santé qui achète à l'aveugle sur juillet 2025 fera le stock de toute la saison des pluies sur un seul mois.

Trois faits mesurés sur le socle fixent l'enjeu :

- la **quincaillerie** (fil rouge) est **quasi plate** : 24 mois entre 313 M et 348 M de FCFA (écart-type 10.8 M), **pas de pic de fin d'année** (décembre 2025 : 315 M, janvier 2026 : 330 M), et une dérive de **+1.6 % à mois égal** (2026/2025) — ce chapitre doit apprendre à **dire** « plat », pas à inventer une saison ;
- le **centre de santé** (jeu 2, arrivé en C02) porte la saisonnalité **structurelle** : les 8 mois d'été (juin-sept) font en moyenne **853** consultations par mois contre **655** pour les hivers — un écart d'environ 198, **quatre fois** la variabilité intra-été (834 à 882) ;
- le **28 décembre 2026** : la base s'arrête là — décembre 2026 fait 307 M « au total » (donc « en baisse » vs 315 M en 2025), mais par **jour** il fait 11.0 M contre 10.2 M : **+8 %**, pas une baisse. La période partielle est le mensonge du chapitre.

3. Explication simple — la conversation avec la machine

La machine est un **fleuve** vu du pont, et la question « ça monte, ça descend, ça cycle ? » s'en répond en séparant **trois choses** :

1. **Le lit du fleuve** (la tendance) : le niveau moyen sur la durée. Ici : le CA de la quincaillerie monte d'environ 1.6 % en un an, à mois égal — un lit qui s'élève à peine.
2. **Les crues saisonnières** (la saisonnalité) : le niveau qui revient **chaque cycle**, au même endroit. Ici : le centre de santé crue chaque été (paludisme, fièvre, diarrhées de la saison des pluies) — juillet 2025 : 882 consultations, le pic de la série.
3. **Les vagues** (le bruit) : le tremblement du niveau **sans cycle**. Ici : les mois qui s'égarent de ± 10 M FCFA autour de la moyenne 328 M — du vent, pas une saison.

La moyenne glissante est la **bâche à vagues** : on l'étale sur 3 mois, les vagues s'effacent, le lit et les crues restent. Et la règle d'or du chapitre : **on ne compare jamais deux portions de fleuve de longueurs différentes** — décembre 2026 compte 28 jours, décembre 2025 en compte 31 : « 307 M < 315 M » n'est pas une baisse, c'est une mesure inégale.

4. Vocabulaire essentiel

Terme	Définition
tendance	la pente de fond de la série (ce qui reste quand on a retiré les cycles) : lecture par la moyenne glissante et la comparaison à période égale.
saisonnalité	saisonnalité — la variation régulière et prévisible liée au cycle (saison, mois, jour de semaine, heure) : elle revient au même endroit du cycle, c'est ce qui la rend exploitable.
bruit	la variabilité sans cycle : ce que la moyenne glissante lisse — on ne le « prédit » pas, on le borne (amplitude, écart-type).
moyenne glissante — moving average — la moyenne calculée sur une fenêtre glissante (ici 3 mois) : chaque point résume la fenêtre, les vagues s'effacent, le lit reste.	

5. Cours approfondi

5.1 La série simple — `set_index + resample` (le 'ME' de M08)

La vue mensuelle de M07/M08 se reconstruit en pandas **sans SQL** (exécuté, fil rouge) :


```
import pandas as pd
v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv",
                parse_dates=["date_vente"])
v["ct"] = (v["montant_ttc"] * 100).round().astype("int64") # centimes : exact
sans = v[~v["est_retour"]] # définition M07 : sans retours
vue = sans.groupby([sans["date_vente"].dt.to_period("M"), "id_magasin"])[["ct"].sum() // 100
print(len(vue.reset_index()), "lignes")
print(vue.nlargest(1).to_string())
```

```
120 lignes
montant_ttc
2025-08    3    78965529
```

120 lignes : 5 magasins × 24 mois — la vue M07 est une vue **par magasin**, et la plus grosse cellule est **2025-08 × magasin 3 = 78 965 529 FCFA** (le S2 de M07 C08), sans retours. Le parallèle SQL redonne le même nombre, exécuté en DuckDB :

```
SELECT strftime(date_vente, '%Y-%m') AS mois, id_magasin,
       CAST(SUM(CASE WHEN NOT est_retour THEN ct ELSE 0 END)/100 AS BIGINT) AS ca
FROM vente GROUP BY 1, 2 ORDER BY ca DESC LIMIT 1;
-- 2025-08 | 3 | 78965529
```

Même nombre, deux écritures — la colonne vertébrale du module, comme le dédoublonnage en C02 : GROUP BY mois (M07) et resample/groupby (pandas) produisent la même série.

Définition. tendance — la pente de fond de la série : ce qui reste quand on a retiré les cycles. Elle se lit à la **moyenne glissante** (qui lisse les vagues) et à la **comparaison à période égale** (à mois égal, au flux par jour) — jamais sur des totaux de périodes inégales.

5.2 Décomposer tendance / saisonnalité / bruit — à l'œil et au chiffre

La moyenne glissante 3 mois sur la vue mensuelle (exécuté, extraits) :

2025-03	328 M	mg3	330 M	2026-04	335 M	mg3	336 M
2025-08	346 M	mg3	326 M	2026-06	348 M	mg3	337 M
2025-12	315 M	mg3	322 M	2026-12	307 M	mg3	324 M

Lecture à l'œil : les vagues (312-348 M) s'effacent sous la bâche (320-337 M) — et la bâche **ne cycle pas** : pas de creux de décembre, pas de crête d'été. La série est plate avec une très légère dérive.

Lecture au chiffre — c'est elle qui départage structurel et bruit :

- l'**amplitude brute** (min-max) de la série est de 41 M FCFA autour d'une moyenne de 328 M ;
- l'**amplitude de la bâche** n'est que de 17 M : les 2/3 de l'amplitude sont du **vent** (bruit), pas du cycle ;
- l'**écart-type** des 24 mois est de 10.8 M, et la **comparaison à mois égal** 2026/2025 tourne autour de **1.0** (de 0.963 à 1.120, moyenne 1.016) : une dérive de +1.6 % en un an, pas une pente.

La règle du chapitre : un cycle est **structurel** quand l'écart **entre** ses positions (été vs hiver) est **grand** comparé à la variabilité **dans** chaque position. Sur le centre de santé (exécuté) : les 8 étés sont serrés entre 834 et 882 (variabilité d'environ 48), et les hivers tournent autour de 655 — l'écart entre positions (≈ 198) **domine** la variabilité intra-position : la saisonnalité est **structurelle**, pas du bruit qui ressemble à une saison.

Définition. moyenne glissante — moving average — la moyenne calculée sur une **fenêtre glissante** (ici 3 mois) : chaque point résume sa fenêtre, les vagues s'effacent, le lit et les crues restent. Sa fenêtre se **déclare** dans la légende (3 mois, pas 6) : une bâche sans fenêtre est une moyenne sans définition.

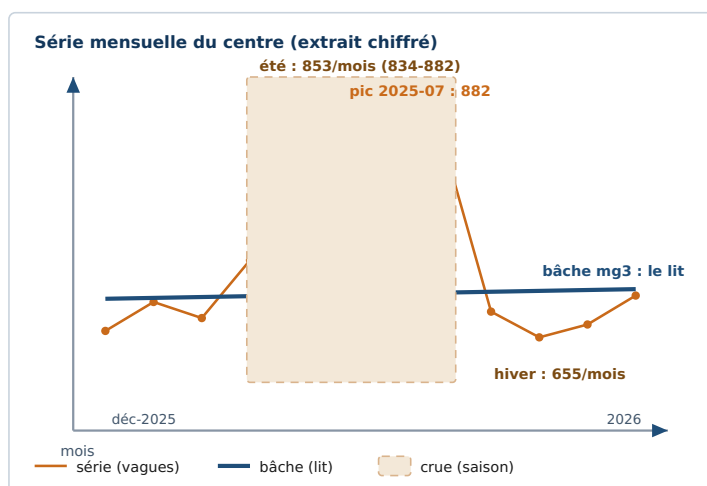
Définition. bruit — la variabilité **sans cycle** : ce que la moyenne glissante lisse. On ne le « prédit » pas, on le **borne** (amplitude brute, écart-type) — et on vérifie qu'il est **petit** devant l'écart entre positions avant de parler de cycle.

Attention. Ne pas forcer une saison sur une série plate : la quincaillerie n'a pas de pic décembre/janvier (315 M / 330 M, sous la moyenne 328 M) — dire « c'est la saison des fêtes » parce que « normalement il y en a une », c'est importer la saison d'un autre jeu dans les données. Le cycle se **mesure** (amplitude entre positions vs variabilité intra-position), il ne se présume pas.

Définition. saisonnalité — saisonnalité — la variation **régulière et prévisible** liée au cycle (saison, mois, jour de semaine, heure) : elle revient **au même endroit du cycle**, c'est ce qui la rend exploitable pour la prévision. Un pic qui revient chaque juillet est de la saison ; un pic qui revient « de temps en temps » est un événement (C04), pas une saison.

Décomposer une série : le lit (tendance), les crues (saison), les vagues (bruit)

La bâche (moyenne glissante) pour le lit ; l'amplitude entre positions vs intra-position pour la saison.



La règle du chapitre

Un cycle est STRUCTUREL quand
l'écart ENTRE positions domine
la variabilité DANS chaque position :

été vs hiver : environ 198
variabilité intra-été : environ 48

198 >> 48 (x4) : la saison est
structurale, pas du bruit.

Fil rouge (quincaillerie) : la
bâche n'est qu'à 17 M d'amplitude
vs 10.8 M d'écart-type : PLAT,
+1.6 % à mois égal — déclaré.

Le zéro-fait : rupture M01 (3 j)
au pic de juillet — tendance
interrompue, pas retombée.

Le mois partiel ment (décembre 2026 : 28 jours)

total : 307 M < 315 M (2025) — « baisse » ?

flux par jour : 11.0 M vs 10.2 M — en réalité +8 %.

Règles : comparer à MOIS ÉGAL, ou au flux
par jour — ou DÉCLARER « aucune conclusion »
(l'étape se déclare, elle ne se saute pas).

5.3 Comparer des périodes à taille égale — le 28 décembre qui ment

Le socle s'arrête au **2026-12-28** : décembre 2026 compte **28 jours**, décembre 2025 en compte 31. La comparaison « naïve » des totaux (exécuté) :

décembre 2025 : 315 M FCFA sur 31 jours	
décembre 2026 : 307 M FCFA sur 28 jours	<-- « en baisse » ?
CA par jour 2025 : 10.2 M FCFA	
CA par jour 2026 : 11.0 M FCFA	--> en réalité : +8 %

Le total dit « baisse », le **flux par jour** dit « hausse de 8 % » : c'est la **période partielle** qui ment, pas les données. Les deux règles du chapitre :

1. **À mois égal** pour comparer deux années : 2026/2025 mois par mois, les ratios vont de 0.963 à 1.120 (moyenne 1.016) — on compare janvier à janvier, pas « l'année 2026 » à « l'année 2025 » quand l'une des deux n'est pas finie.
2. **Au flux par jour** pour comparer des mois inégaux (28 vs 31 jours) — ou, mieux, on **ne compare pas du tout** et on le dit : « décembre 2026 est partiel, aucune conclusion de tendance » — l'étape se déclare, elle ne se saute pas (C01).

À retenir. Avant de comparer deux périodes, on les **pèse** : combien de jours chacune contient-elle ? 31 vs 28, 365 vs 362 — si elles ne pèsent pas le même, on compare **à mois égal**, **au flux par jour**, ou on **déclare** la comparaison impossible. Pesez avant de peser.

Attention. Un total de période partielle est une **mesure inégale** : 307 M sur 28 jours n'est pas « un décembre de 307 M ». Comparer 2025 (365 jours) et 2026 (362 jours dans le socle) « en tout », c'est comparer deux portions de fleuve de longueurs différentes — et conclure sur la différence.

5.4 La saisonnalité du jour de semaine et de l'heure — le centre de santé

Le rappel `dayofweek == 6` de M08 trouve ici sa vraie utilisation : les 18 818 consultations réparties par jour de semaine (exécuté) :

lundi	3710	jeudi	3100	dimanche	405
mardi	3290	vendredi	1872	(uniforme : 2688/jour)	
mercredi	3249	samedi	3192		

Le dimanche du centre fait 405 consultations contre 3 710 le lundi — environ 1/9 du lundi, et 15 % de ce que donnerait une répartition uniforme (2 688) : le centre est **presque fermé le dimanche**, mais pas tout à fait (405, pas 0) — c'est la saisonnalité du jour, structurelle comme la saison (elle revient chaque semaine).

La **journée type** (10 heures de 08:00 à 17:00, exécuté) :

08:00	1673	10:00	2203	12:00	927	14:00	2126
09:00	2319	11:00	2044	13:00	1085	15:00	2119
16:00	2030	17:00	2292				

Le pic est le **matin** (09:00 : 2 319), le creux est le **midi** (12:00 : **927** — 4.9 % des consultations contre 10 % attendus en répartition uniforme), et l'après-midi est **stable** (2 030 à 2 292). Une journée type, c'est une saisonnalité **d'heure** : elle réorganise les gardes, pas les stocks.

5.5 La tendance interrompue — la rupture qui coupe sans retombée

Le stock M02 (paracétamol) autour du 2026-03-02 (exécuté) :

date	id_medicament	entree	sortie	stock_fin	
2026-02-28	M02	0	7	1288	
2026-03-01	M02	0	11	1277	
2026-03-02	M02	100	1377	0	<-- tout le stock sort
2026-03-03	M02	0	0	0	<-- 5 jours à zéro
2026-03-04	M02	0	0	0	
2026-03-05	M02	0	0	0	
2026-03-06	M02	0	0	0	
2026-03-07	M02	68	7	61	<-- le restock, la demande revient
2026-03-08	M02	0	4	1344	
2026-03-09	M02	0	11	1333	

La lecture **tendancielle** : les sorties (la demande) faisaient environ 7/jour en février (moyenne 6.7) — pendant les 5 jours de rupture, elles sont **nulles** (on ne distribue pas ce qu'on n'a pas) — et elles **reviennent** au même niveau au restock (7, 4, 11 les trois jours suivant ; moyenne de mars hors rupture : 5.6/jour). La courbe des sorties plonge à zéro **non pas parce que la demande est retombée, mais parce que l'offre est coupée** : c'est une **tendance interrompue, pas une tendance retombée** — confondre les deux, c'est prévoir une baisse de fréquentation qui n'existe pas.

La rupture M01 (3 jours, 2025-07-14 → 2025-07-16) est le même mécanisme **au pire moment** : pendant le **pic de juillet** (882 consultations, le mois max de la série) — la lecture tendancielle est identique (la demande revient au restock), la **décision métier** est au C04 : une rupture de paracétamol en saison des pluies est un signal à traiter, pas un chiffre à corriger.

Dans les faits. Sur le fil rouge, l'étape « tendances » se termine par une **déclaration vide** : « aucune saisonnalité détectée — contrôle : amplitude de la bâche 17 M vs écart-type 10.8 M, ratios à mois égal 0.963–1.120, pas de pic décembre/janvier (315 M / 330 M) ». C'est une étape **vide, déclarée** — pas une étape passée : le protocole (C01) le distinguait, le rapport le prouve.

6. Exemple concret — la quincaillerie : dire « plat », et le prouver

Le fil rouge traverse l'étape 5 en entier (exécuté, sans retours, définition M07) :

1. **La série** : 24 mois, 307 à 348 M FCFA, moyenne 328 M, écart-type 10.8 M ; la vue par magasin fait **120 lignes**, plus grosse cellule **78 965 529 FCFA** (2025-08, magasin 3) — le même nombre que le GROUP BY de M07.
2. **Le lit** : la moyenne glissante 3 mois tourne entre 320 et 337 M — **pas de cycle** sous la bâche ; la comparaison à mois égal 2026/2025 (moyenne 1.016, de 0.963 à 1.120) donne la seule tendance **mesurable** : **+1.6 % en un an**.
3. **La saison déclarée absente** : décembre 2025 (315 M) et janvier 2026 (330 M) ne montent **pas** au-dessus de la moyenne — **pas de saison des fêtes dans ces données** ; l'hypothèse « fin d'année » est testée et **rejetée sur la période**, avec le chiffre qui la rejette (315 < 328, la moyenne).
4. **Le mois partiel** : décembre 2026 (307 M sur 28 jours) ne se compare **pas** à décembre 2025 « au total » ; au flux par jour, c'est +8 % — et la conclusion honnête est « décembre 2026 partiel : aucune conclusion de tendance ».

7. Démonstration pas à pas — 5 étapes sur les deux jeux

```
import pandas as pd
s = "03_exercices/dossier_M09"
v = pd.read_csv(s + "/quincaillerie/vente.csv", parse_dates=["date_vente"])
c = pd.read_csv(s + "/sante/consultation.csv", parse_dates=["date_consultation"])

# 1 – la série mensuelle (fil rouge, sans retours)
v["ct"] = (v["montant_ttc"] * 100).round().astype("int64")
vt = v[~v["est_retour"]].groupby(
    v["date_vente"].dt.to_period("M"))["ct"].sum() // 100
print(len(vt), "mois | moyenne M:", round(vt.mean() / 1e6))

# 2 – la bêche : moyenne glissante 3 mois
mg3 = vt.rolling(3).mean()
print("amplitude brute M:", round((vt.max() - vt.min()) / 1e6),
      "| amplitude mg3 M:", round((mg3.max() - mg3.min()) / 1e6))

# 3 – à mois égal 2026/2025
r = vt[vt.index.year == 2026].values / vt[vt.index.year == 2025].values
print("ratios 2026/2025 : min", round(r.min(), 3), "| max", round(r.max(), 3),
      "| moyenne", round(r.mean(), 3))

# 4 – le mois partiel : flux par jour
d26 = v[(v["date_vente"] >= "2026-12-01")]["ct"].sum() / 100 / 28
d25 = v[(v["date_vente"] >= "2025-12-01") & (v["date_vente"] < "2026-01-01")]["ct"].sum() / 100 / 31
print("CA/jour déc26 M:", round(d26 / 1e6, 1), "| déc25 M:", round(d25 / 1e6, 1))

# 5 – la saison du centre : jour de semaine + heure
c["dw"] = c["date_consultation"].dt.dayofweek
print(c.groupby("dw").size().to_dict())
print(c.groupby("heure").size().loc["12:00"])
```

```
24 mois | moyenne M: 328
amplitude brute M: 41 | amplitude mg3 M: 17
ratios 2026/2025 : min 0.963 | max 1.12 | moyenne 1.016
CA/jour déc26 M: 11.0 | déc25 M: 10.2
{0: 3710, 1: 3290, 2: 3249, 3: 3100, 4: 3192, 5: 1872, 6: 405}
927
```

8. Erreurs fréquentes

1. **Forcer une saison sur une série plate** — la quincaillerie n'a pas de pic de décembre (315 M) : dire « c'est de la saisonnalité des fêtes » parce que « normalement il y en a », c'est importer une saison **d'un autre jeu** dans les données. Le cycle se **mesure** (amplitude entre positions vs variabilité intra-position), il ne se **présuppose** pas.
2. **Comparer des périodes inégales** — 307 M (28 jours) vs 315 M (31 jours) « en baisse » ; l'année 2026 (362 jours dans le socle) vs 2025 « en tout » : chaque total de période partielle est une mesure inégale, et chaque comparaison d'inégaux est une **conclusion fausse avec de vrais chiffres**.
3. **Lire la tendance sur le bruit** — un mois à 348 M (juin 2026) suivi d'un mois à 327 M n'est pas « ça descend » : c'est de l'amplitude (± 10 M autour de 328) — la bêche (17 M d'amplitude sur la moyenne glissante) est là pour le dire avant l'œil.
4. **Confondre tendance interrompue et tendance retombée** — les sorties de M02 tombent à 0 pendant 5 jours : ce n'est pas une chute de la demande, c'est une offre coupée (la demande revient à 7/jour au restock) — la prévision qui en déduirait une baisse de fréquentation commanderait trop pour mars.

5. **Oublier le jour de semaine** — une moyenne « par jour » qui mélange dimanches (405) et lundis (3 710) dilue la saisonnalité la plus forte du jeu santé : la série **par jour de semaine** précède la série par date, ou elle la trahit.

9. Bonnes pratiques professionnelles

1. **La série se construit avant d'être lue** : définition de l'agrégat (sans retours ?), de la granularité (mois), du périmètre (magasins) — la vue mensuelle de M07 se reconstruit en 3 lignes **si la définition est la même** ; sinon le nombre ne retombe pas (78 965 529 exige « sans retours »).
2. **Tendance = bâche + comparaison à période égale** — les deux lectures, ensemble : la moyenne glissante pour le lit, le ratio à mois égal pour la pente ; l'une sans l'autre est une impression.
3. **Un cycle se prouve par l'amplitude** — écart entre positions (été vs hiver : 198) vs variabilité intra-position (été : 48) : le premier doit **dominer** le second, sinon c'est du bruit qui ressemble à une saison.
4. **Le mois partiel se déclare, ne se compare pas** — « décembre 2026 : 28 jours, aucune conclusion de tendance ; au flux par jour, +8 % » — les deux phrases, c'est la pratique complète.
5. **L'étape vide se déclare avec ses contrôles** — « aucune saisonnalité détectée : amplitude de la bâche 17 M vs 10.8 M d'écart-type, ratios 0.963-1.120 » : la négation prouvée vaut plus qu'une courbe vague.

Conseil professionnel. Le graphique de tendance d'un rapport P3 porte **trois lignes**, pas une : la série brute (les vagues), la moyenne glissante (le lit), et la saison attendue (les crues) — quand la troisième n'existe pas, on le **dit** sur le graphique, et on ne la dessine pas. Un graphique à une ligne invite le lecteur à inventer ce que vous n'avez pas prouvé.

10. Exercice guidé — « lire la saison du centre de santé » (25 min, /10)

Consigne. Sur `sante/consultation.csv` (18 818 lignes, défauts C02 déjà audités) :

1. Construisez la série **par mois** de consultations (1 pt).
2. Donnez les **8 étés** (juin-sept, 2025 et 2026) et les **hivers** (déc-jan) : leur moyenne, leur amplitude — et concluez si la saisonnalité est **structurelle** ou du bruit, avec les deux chiffres qui le départagent (3 pts).
3. Montrez la saisonnalité **du jour de semaine** : la valeur du dimanche, celle du lundi, et ce que donnerait une répartition uniforme (2 pts).
4. Donnez la **journée type** : l'heure de pic, l'heure de creux, la valeur du creux, et l'attendu uniforme (2 pts).
5. En une phrase : qu'est-ce que cette saisonnalité change **concrètement** au centre (1 pt) ?

11. Exercices autonomes

E1 — Le parallèle SQL des tendances (20 min). En DuckDB sur `quincaillerie/vente.csv` : (1) la série mensuelle du CA **sans retours** (24 lignes) ; (2) la plus grosse cellule (mois, magasin) — elle doit retomber sur 78 965 529 ; (3) les ratios 2026/2025 à mois égal (min, max, moyenne). Les trois sorties sont publiées telles que renvoyées.

E2 — La rupture qui coupe (20 min). Sur `sante/stock_medicament.csv`, médicament M02 : (1) listez les jours de rupture (stock à zéro) et leurs dates ; (2) comparez le niveau des sorties **avant** (février 2026) et **après** (mars 2026, hors rupture) la rupture ; (3) concluez : tendance retombée ou tendance interrompue, avec les deux chiffres qui le prouvent ; (4) en une phrase, ce que la prévision de mars 2027 **ne doit pas** en déduire.

12. Correction détaillée

Exercice guidé.

1. **La série par mois** : `groupby(to_period("M")).size()` — 24 mois, de 2025-01 à 2026-12, **décembre 2026 partiel** (28 jours) : on le garde dans la série et on le **déclare** partiel (§5.3), on ne le compare pas « au total ».
2. **Étés vs hivers** : moyenne des 8 étés **853** consultations/mois, amplitude 834-882 (≈ 48) ; hivers (déc-jan) **655** ; l'écart entre positions (≈ 198) domine la variabilité intra-été (48) d'un facteur $\sim 4 \rightarrow$ **structurelle**.
3. **Jour de semaine** : dimanche **405**, lundi **3 710**, uniforme **2 688** ($18\,818 / 7$) — le dimanche est à 15 % de l'uniforme : le centre est presque fermé, pas fermé.
4. **Journée type** : pic **09:00** (2 319), creux **12:00** (**927** = 4.9 % contre 10 % attendu uniformément sur les 10 heures) — le creux est le double de l'écart attendu : c'est un creux réel, pas du bruit.
5. **Le concret** : les gardes et la distribution des médicaments se calent sur le pic du matin et l'été (paludisme/fièvre) — pas sur « la moyenne du jour », qui dilue tout.

E1. DuckDB, exécuté :

```
-- (1) série mensuelle sans retours : 24 lignes, 2025-01 → 2026-12
-- (2) plus grosse cellule :
2025-08 | 3 | 78965529
-- (3) ratios 2026/2025 à mois égal (self-join sur mois) :
min 0.963 | max 1.12 | moyenne 1.016
```

Le `GROUP BY mois` de M07 et le `groupby(to_period("M"))` pandas redonnent **la même série** : même nombre, deux écritures — la constance du socle (empreinte `b9a8d973119342ec...`, 19 totaux = M07/M08) passe encore une fois par la porte des tendances.

E2. (1) Rupture M02 : **5 jours**, du **2026-03-02 au 2026-03-06** (stock à zéro ; le 2026-03-02, tout le stock sort en une sortie unique, ce que le C04 qualifiera). (2) Sorties/jour : **6.7** en février 2026, **5.6** en mars 2026 hors rupture — même niveau (l'écart de 1.1/jour est de l'ordre du bruit quotidien). (3) **Tendance interrompue, pas retombée** : pendant la rupture, les sorties sont **nulles par contrainte d'offre** (on ne distribue pas ce qu'on n'a pas) ; au restock (2026-03-07), elles reviennent à 7, 4, 11 — le niveau de février. (4) La prévision de mars 2027 **ne doit pas** déduire une baisse de demande : elle doit prévoir **le niveau de sortie habituel** ($\approx 6\text{-}7/\text{jour}$) **plus** le restock — et signaler le risque de rupture, pas une chute de fréquentation.

13. Mini-projet M09.P3 — « Le graphique de tendance avec sa question » (1 h)

Ancre P3 : les 4 graphiques du projet M09.P doivent répondre à une question — celui-ci est le premier d'entre eux : « ça monte, ça descend, ça cycle ? »

Énoncé. Sur le centre de santé, produire **un** graphique de tendance (courbe) qui répond à « ça monte, ça descend, ça cycle ? », **légendé** : question posée + source (sortie exécutée) + ce qu'on

doit retenir. Le graphique porte les **trois lignes** du conseil §9 (série mensuelle, moyenne glissante, et — ici — la saison lisible) et sa légende **déclare** : la saisonnalité structurelle (étés 853 vs hivers 655), le pic 2025-07 (882), le mois partiel de 2026-12 (aucune conclusion). Auto-évaluation : chaque légende est-elle **vérifiable** sur le socle ? Le mois partiel est-il **déclaré** plutôt que comparé ?

14. Boîte à outils du chapitre

Boîte à outils. La série : `to_period("M") + groupby` (ou `set_index + resample("ME")`), centimes pour l'exactitude (`(montant*100).round() // 100`). La bâche : `rolling(3).mean()`. La comparaison : ratios à mois égal (`série_2026.values / série_2025.values`), flux par jour pour les mois inégaux. La saison : `dayofweek, groupby("heure")`. Le parallèle : `GROUP BY strftime(date, '%Y-%m')` en DuckDB — même série, deux écritures.

Besoin	Commande	Chiffre du socle
série mensuelle	<code>groupby(to_period("M")).sum()</code>	24 mois, moyenne 328 M
vue par magasin	<code>+ "id_magasin" dans le groupby</code>	120 lignes, max 78 965 529
bâche	<code>vt.rolling(3).mean()</code>	amplitude 17 M (vs 41 M brute)
pente	ratios à mois égal	0.963-1.120, moy 1.016
mois partiel	<code>sum() / nb_jours</code>	11.0 vs 10.2 M/jour (déc 2026/2025)
saison hebdo	<code>groupby(dayofweek).size()</code>	dim 405 / lun 3710
journée type	<code>groupby("heure").size()</code>	creux 12:00 = 927

15. Résumé du chapitre

L'étape 5 du protocole (« ça monte, ça descend, ça cycle ? ») est exécutée sur les deux jeux arrivés jusque-là, avec la discipline du chapitre : **décomposer en tendance / saisonnalité / bruit**, à l'œil (moyenne glissante) et au chiffre (amplitude entre positions vs variabilité intra-position), et **ne comparer que des périodes à taille égale**. Sur le fil rouge (quincaillerie, sans retours, définition M07) : 24 mois entre 307 et 348 M FCFA, moyenne 328 M, écart-type 10.8 M ; la vue par magasin redonne les **120 lignes** de M07 (plus grosse cellule **78 965 529 FCFA**, 2025-08 × magasin 3 — même nombre en `GROUP BY` DuckDB) ; la moyenne glissante 3 mois (amplitude 17 M) et la comparaison à mois égal 2026/2025 (0.963 à 1.120, +1.6 %) disent **plat avec une très légère dérive** ; il **n'y a pas** de saison des fêtes dans ces données (décembre 2025 : 315 M, janvier 2026 : 330 M, sous la moyenne) — et l'étape le **déclare** avec ses contrôles ; décembre 2026 (28 jours) ne se compare pas « au total » à 2025 — au flux par jour, c'est +8 %, et la conclusion est « mois partiel, aucune conclusion de tendance ». Sur le centre de santé : la saisonnalité est **structurelle** (étés 853 vs hivers 655 consultations/mois, pic **2025-07** (882 consultations), variabilité intra-été d'environ 48, l'écart ≈ 198 qui la domine), la saison est aussi **hebdomadaire** (dimanche 405 vs lundi 3 710) et **horaire** (pic 09:00, creux **927** à 12:00) ; et la rupture M02 (5 jours à zéro, 2026-03-02 → 2026-03-06) est une **tendance interrompue, pas une tendance retombée** — la demande revient au même niveau au restock, et c'est la lecture qui sauve la prévision.

16. À retenir

À retenir. Trois lectures avant toute affirmation : la **bâche** (le lit), la **comparaison à période égale** (la pente), l'**amplitude** (la saison) — et une négation prouvée vaut plus qu'une courbe vague : « aucune saisonnalité détectée, et voici les contrôles » est une étape faite, pas une étape passée.

1. **La série se construit avant d'être lue** : agrégat, granularité, périmètre — la vue M07 retombe en 120 lignes (5×24) **si la définition est la même**.
2. **Tendance = bâche + ratio à mois égal** — 17 M d'amplitude sous la bâche, +1.6 % à mois égal : c'est la seule tendance mesurable du fil rouge.
3. **Un cycle se prouve par l'amplitude** : écart entre positions (198) qui domine la variabilité intra-position (48) → structurel ; sinon, c'est du bruit.
4. **Période partielle = mensonge** : 307 M sur 28 jours n'est pas un décembre ; on compare à mois égal ou au flux par jour, ou on déclare « aucune conclusion ».
5. **La saison a trois horloges** : saison (été 853 / hiver 655), semaine (dimanche 405), heure (creux 927 à midi) — les trois se mesurent, les trois se déclarent.
6. **Interrompue ≠ retombée** : 5 jours de zéro (rupture) puis retour au niveau — la demande n'est pas tombée, l'offre est coupée.
7. **Le parallèle SQL redonne la série** : `GROUP BY strftime(date, '%Y-%m')` et `group-by(to_period("M"))` — 24 lignes, même 78 965 529.

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi la vue mensuelle retombe-t-elle à 120 lignes, et quelle est sa plus grosse cellule ?** → 5 magasins $\times$ 24 mois, **sans retours** (définition M07) ; plus grosse cellule 78 965 529 FCFA (2025-08, magasin 3) — le `GROUP BY` de M07 la redonne. (2 pts)
2. **Que disent la moyenne glissante 3 mois (amplitude 17 M) et l'écart-type (10.8 M) de la quincaillerie ?** → que les 2/3 de l'amplitude brute (41 M) sont du **bruit** sans cycle : la série est plate avec une légère dérive — pas de saison à déclarer, et c'est **ça** qu'on déclare. (2 pts)
3. **Pourquoi « décembre 2026 en baisse vs décembre 2025 (307 M < 315 M) » est-il un mensonge, et que dit le flux par jour ?** → période partielle : 28 jours vs 31 ; au flux par jour, 11.0 M/jour vs 10.2 M → **+8 %**, pas une baisse ; la conclusion honnête : « mois partiel, aucune conclusion de tendance ». (2 pts)
4. **Les ratios 2026/2025 à mois égal vont de 0.963 à 1.120 (moyenne 1.016). Quelle tendance mesurable, et pourquoi pas plus ?** → une dérive de **+1.6 % en un an** ; pas plus, car chaque ratio individuel est de l'ordre du bruit mensuel ($\pm 3-7$ %) — la tendance est la **moyenne** des 12 ratios, pas un ratio. (2 pts)
5. **Étés 853 (834-882) vs hivers 655 consultations/mois : pourquoi la saisonnalité du centre est-elle « structurelle » ?** → l'écart **entre** positions (≈ 198) **domine** la variabilité **dans** les positions (≈ 48 en été) d'un facteur ~ 4 : le cycle revient au même endroit du cycle, c'est exploitable. (2 pts)
6. **Dimanche : 405 consultations ; uniforme : 2 688. Que dit ce chiffre, et que ne dit-il pas ?** → que le centre est presque fermé le dimanche (15 % de l'uniforme) — saisonnalité hebdomadaire réelle ; mais il ne dit **pas pourquoi** (organisation, pas de variable « gardien » dans le fichier) — c'est du C05. (2 pts)
7. **M02 : 5 jours à zéro, sorties nulles pendant la rupture, retour à 7/jour au restock. Tendance retombée ou interrompue ?** → **interrompue** : les sorties sont nulles **par**

contrainte d'offre ; la demande revient au niveau de février (6.7/jour) au restock — la prévision de mars 2027 prévoit le niveau habituel + le restock, pas une baisse. (2 pts)

8. **Écrivez la déclaration d'étape vide du fil rouge, avec ses contrôles.** → « Aucune saisonnalité détectée sur le CA mensuel : amplitude de la moyenne glissante 3 mois 17 M vs écart-type 10.8 M, ratios à mois égal 0.963-1.120 (moyenne 1.016), pas de pic décembre/janvier (315 M / 330 M sous la moyenne 328 M) » — négation prouvée, étape déclarée, pas sautée. (2 pts)

Module M09.C04 — Anomalies et raretés : ce qui est inhabituel, comment le prouver, quoi en conclure

Outils : pandas 2.2.3, numpy 2.3.5 (exécutés), DuckDB 1.5.5 (exécuté — la preuve par récurrence en une requête). **Durée indicative :** 5 h. **Niveau :** N3. **Prérequis :** M09.C02 (l'audit), M09.C03 (les bornes et la période), M02 (quartiles, iqr).

L'idée du chapitre. L'étape 6 du protocole pose la question : « **qu'est-ce qui est inhabituel, et comment le prouver ?** » C04 répond avec une règle d'or : **une anomalie non prouvée est un soupçon, pas un constat** — et trois preuves sont disponibles (position dans la distribution, récurrence, cohérence métier). Ensuite, **trois décisions** et une seule : corriger, exclure **en documentant**, garder **en signalant** — jamais supprimer en silence. Le chapitre porte deux contre-exemples qui changent tout : les **ruptures** du centre de santé (l'anomalie qui **compte** : la corriger serait une erreur métier) et les **6 ventes à 594 363 FCFA** du fil rouge (une rareté **légitime**, prouvée telle par le produit, la quantité et le taux — le plateau le plus suspect de la base est le moins fautif).

Matériel de l'atelier — Python 3.13 · pandas 2.2.3 · numpy 2.3.5 · DuckDB 1.5.5. Toutes les commandes ont été exécutées dans l'atelier le 20/09/2026 sur 03_exercices/dossier_M09/ (graine 45, déterministe, ATTENDU.json figé) ; les sorties publiées sont **celles de l'atelier**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Définir l'anomalie avant de la chercher** — trois seuils disponibles : par iqr (position dans la distribution), par borne métier (limite du domaine), par comparaison à la période (l'écart à la saison, C03).
2. **Prouver une anomalie** avec au moins une des 3 preuves : position, **récurrence** (hier + entrées – sorties), cohérence métier — et savoir qu'à défaut, c'est un **soupçon** qu'on signale, pas un fait qu'on traite.
3. **Trancher parmi les 3 décisions** — corriger / exclure en documentant / garder en signalant — et écrire la ligne de rapport qui les justifie (P2).
4. **Reconnaître l'anomalie qui compte** : une rupture de médicament essentiel est un **signal métier**, pas un chiffre à corriger — la corriger serait une erreur.
5. **Distinguer le rare du fautif** : 6 ventes à 594 363 FCFA, le plus gros mois 78 965 529, le top client de M07 — des raretés **légitimes**, prouvées telles avant d'être effacées.

2. Pourquoi cette notion est importante

L'anomalie est le point où l'analyse devient **responsable** : on ne décrit plus, on **décide** — corriger une valeur fausse, exclure une ligne, ou garder une vérité inhabituelle. Trois faits mesurés sur le socle fixent l'enjeu :

- les **4 stocks négatifs** du centre de santé (−1, −3, −2, −12) sont prouvés **défauts de saisie** par double preuve (borne métier + récurrence) — et leur valeur correcte est **déductible** (411, 253, 278, 545) : la correction n'est pas une invention, c'est un calcul ;

- les **ruptures** (M01 : 3 jours en juillet 2025, M02 : 5 jours en mars 2026, M04 : 2 jours en août 2026) sont des **zéros qui ne doivent pas être corrigés** : c'est la réalité de l'offre, avec sa conséquence (des patients non servis) — les « réparer », c'est effacer le problème le plus grave du jeu de données ;
- les **6 ventes à 594 363 FCFA** (le maximum de la base, 50 008 lignes) sont le cas d'école de la **rareté légitime** : identiques entre elles, 6 magasins... non, 2 magasins, 6 clients, 6 dates — et pourtant aucune n'est fausse : 10 unités (le max de la base) du produit 18 (le plus cher : 49 946.47 FCFA) à la TVA 0.19.

3. Explication simple — la conversation avec la machine

La machine vient vous dire : « ça, c'est bizarre. » Le chapitre est le **tribunal** :

1. **L'accusation** (le signal) : une valeur au-delà d'une **borne définie avant** — un stock à -12 , une consultation en 2027, un montant six fois répété. Une borne sans date d'adoption n'accuse personne : elle se choisit **avant** de regarder.
2. **La preuve** (le chapitre entier) : trois témoins possibles — la **position** (« il est à 3 iqr du cœur »), la **récence** (« le jour d'avant et d'après s'expliquent, lui non »), le **métier** (« une pharmacie n'a pas -12 boîtes »). Un témoin suffit si c'est le métier ; pour les chiffres, on en veut deux.
3. **Le verdict** (une des trois phrases, et seulement celle-là) : **corriger** (avec la valeur déductible et sa source), **exclure en documentant** (la ligne, la définition, le nombre de lignes retirées), **garder en signalant** (l'information est vraie ou inconnue, on la laisse et on la nomme). **Ja-mais** « je supprime » sans phrase : le silence est le seul vice de procédure du rapport.

Et le rappel du chapitre : **tout ce qui est rare n'est pas fautif** — le gros client de M07 est un client, pas un bug ; la rupture est une vérité douloureuse, pas une erreur ; et le plateau de 594 363 est une limite **structurelle** du métier, pas un copié-collé.

4. Vocabulaire essentiel

Terme	Définition
anomalie	anomalie — une observation qui s'écarte d'une borne définie avant de chercher (iqr, borne métier, période) ; le statut n'est acquis qu' après preuve .
soupçon	le stade précédent : un signal non prouvé. On le signale (il entre au rapport, étiqueté « soupçon »), on ne le traite pas.
anomalie structurelle	un défaut qui révèle un mécanisme (saisie, export, génération) plutôt qu'une erreur ponctuelle : les 8 doublons du fil rouge, les 4 stocks négatifs déductibles.
rareté légitime	une valeur extrême fausse par aucune preuve : le top client, le plateau de montant max — on la garde, on l'explique.

5. Cours approfondi

5.1 Définir l'anomalie avant de la chercher

Les trois **seuils** disponibles, chacun avec son domaine :

Seuil	Définition	Bon pour	Mauvais pour
iqr (C02)	au-delà de $Q1 - 1,5 \times iqr$ ou $Q3 + 1,5 \times iqr$	valeurs continues de régime stable	mélanges de populations (C02 §5.4)
borne métier	limite du domaine : stock ≥ 0 , note entre 0 et 20, date dans la période	tout domaine borné	tout ce que le métier n'borne pas

Seuil	Définition	Bon pour	Mauvais pour
période (C03)	écart à la saison/jour de semaine attendu	séries avec cycle	séries plates (pas de saison à comparer)

L'ordre compte : la **borne métier** accuse **sans appel** (un stock à -12 n'a pas de discussion), l'**iqr** accuse **en signalant** (442 sous la borne M12 était plausible, C02 §5.4), la **période** accuse **en comparant** (juillet 2025 à 882, c'est la saison — ce n'est pas une anomalie, c'est le cycle). Sur le socle, les 4 familles de C02 se retrouvent chacune sous le seuil qui les trouvait : les doublons (définition métier, C01/C02), les 23 motifs vides (manquant = valeur absente, définition C02), la date 2027 (borne métier : la période s'arrête au 2026-12-31) et les 4 stocks négatifs (borne métier + récurrence).

Définition. anomalie — anomalie — une observation qui s'écarte d'une borne **définie avant** de chercher : iqr (position dans la distribution), borne métier (limite du domaine) ou période (écart au cycle attendu). Le statut « anomalie » n'est acquis qu'**après preuve** ; avant, c'est un **soupçon** — et la différence décide du traitement.

5.2 Prouver — les trois témoins

Témoin 1 — la position dans la distribution. M12 en 2026-11-03 : -12, là où le cœur du stock tourne entre 518 et 566 (Q1, Q3, iqr 48, borne basse 446 — C02 §5.4) : l'écart au cœur est de 500+ boîtes, **pas** d'une vingtaine comme pour 442. La position accuse, mais à elle seule elle n'est pas suffisante (442 était aussi « au-delà »).

Témoin 2 — la récurrence. Le stock suit hier + entrées - sorties. Sur les 4 défauts, exécuté en DuckDB (une requête, LAG) :

id_medicament	date	reel	attendu_récurrence
M12	2026-11-03	-12	545
M07	2025-10-09	-3	253
M10	2026-05-21	-2	278
M05	2025-04-18	-1	411

Chaque valeur négative a un **jumeau naturel** déductible : 545, 253, 278, 411 — la série s'explique d'avant en après **sans** la valeur enregistrée. C'est la preuve la plus forte du module : elle ne dit pas seulement « c'est faux », elle dit **ce que la valeur était censée être**.

Définition. récurrence (du flux) — la règle de continuité d'une série de stock : hier + entrées - sorties = aujourd'hui. Elle se teste en une ligne (`shift(1)` en pandas, `LAG() OVER (PARTITION BY ... ORDER BY date)` en DuckDB) et c'est elle qui **dédoublle** l'accusation : une valeur qui casse la récurrence est suspectée, une valeur que la récurrence **remplace** (545 pour -12) est condamnée.

Témoin 3 — la cohérence métier. Un stock négatif n'existe pas en pharmacie ; une consultation datée au 2027-01-05 n'existe pas dans un export arrêté au 2026-12-31 ; une note de 25 n'existe pas sur 20. Le métier accuse **sans appel** — et c'est le seul témoin qui puisse parler **seul** (les 4 stocks négatifs, la date future).

Définition. soupçon — le stade d'une anomalie **non prouvée** : un signal (position, borne, période) sans aucun des 3 témoins. Le rapport le **signale** (« valeur au-delà de la borne, non explorée : la période ne suffit pas »), il ne l'exclut pas — traiter un soupçon, c'est confondre l'accusation et le verdict.

L'étape 6 : la borne accuse, la preuve condamne, la décision écrit sa ligne

Une anomalie non prouvée est un soupçon — on le signale, on ne le traite pas. Les 3 témoins, les 4 cas du socle.

TÉMOIN 1 — POSITION

Q1/Q3, iqr, bornes de Tukey

442 sous la borne : plausible

TÉMOIN 2 — RÉCURRENCE

hier + entrées - sorties = aujourd'hui

545 déduit pour -12 : condamné

TÉMOIN 3 — MÉTIER

stock ≥ 0 · note sur 20 · période

pas de -12 boîtes : parle seul

Les 4 cas du socle, jugés (preuve -> décision -> ligne de rapport)

CAS	PREUVE	DÉCISION
4 stocks négatifs (-1, -3, -2, -12)	métier + récurrence (double)	CORRIGER : 411, 253, 278, 545
1 date au 2027-01-05 (C000001)	borne période + 1ère référence	EXCLURE de la série · confirmer source
ruptures M01 3j · M02 5j · M04 2j	zéros vrais (récurrence OK)	GARDER : signal métier, pas défaut
6 ventes au max 594 363 (fil rouge)	plafond : 10 x 49 946.47 x 0.19	GARDER : rareté légitime, pas doublon
Jamais la 4e décision : « je supprime » sans ligne — le silence est le seul vice ; la table P4 se re-joue.		

5.3 Les trois décisions — les 4 défauts de C02 jugés

Défaut (C02)	Preuve	Décision	Ligne de rapport
D1 — 23 motifs vides	information perdue, non reconstituable	garder en signalant	« 23 consultations hors des taux par motif, motif inconnu »
D2 — 7 doublons de stock	5-tuple complet identique, keep="first"	exclure en documentant	« 7 lignes retirées : doublons exacts (date, médicament, entrées, sorties, stock) »
D3 — date 2027-01-05	borne métier (période) + C000001 (1 ^{re} référence)	exclure de la série en documentant ; coquille probable 2025-01-05, à confirmer avec la source	« 1 consultation hors période, exclue des séries ; correction soumise à la source »
D4 — 4 stocks négatifs	borne métier + récurrence (double)	corriger en 411 / 253 / 278 / 545	« 4 valeurs de saisie remplacées par la valeur déductible de la récurrence (tableau des 4) »

La discipline qui fait la différence : **chaque décision porte sa preuve et sa ligne** — pas « on a nettoyé », mais « 7 lignes retirées, voici la définition, voici le nombre ». Sur D3, notez la nuance : la preuve (borne métier) **exclut** de la série, mais ne **corrige** pas — la correction (2025-01-05) reste à **confirmer** : c'est le métier qui confirme, pas la statistique.

À retenir. Trois décisions, et **jamais la quatrième** (« je supprime ») : **corriger** exige une valeur **déductible** et sa source ; **exclure** exige une définition et un nombre ; **garder** exige une phrase. Le rapport P2 lit la ligne, pas le résultat.

Attention. « Corriger » n'existe que pour une valeur **déductible** : 545 pour M12, parce que $548 - 3 = 545$ se **calcule**. Une valeur non déductible (le motif perdu, la date coquille sans témoin) ne se corrige **jamais** par invention — elle s'exclut en documentant ou se garde en signalant. Inventer une correction, c'est fabriquer une donnée.

5.4 L'anomalie qui compte — la rupture n'est pas un chiffre

Les **ruptures** de médicaments essentiels (stock à zéro) du socle : M01 — **3 jours**, 2025-07-14 → 2025-07-16 ; M02 — **5 jours**, 2026-03-02 → 2026-03-06 ; M04 — **2 jours**, 2026-08-10 → 2026-08-11. Ce qu'elles ont de particulier au regard de l'étape 6 : **elles sont vraies**.

- La borne « stock ≥ 0 » ne les accuse **pas** : 0 est dans le domaine (zéro boîte, ce n'est pas une dette) ;
- la récurrence les **explique** (le stock s'épuise : sorties supérieures aux entrées sur plusieurs jours) ;
- le métier les **confirme** : une rupture, c'est un fait de gestion, avec une date de début, une durée, une date de restock.

La décision est donc la quatrième, que le chapitre nomme : **garder, et traiter comme un signal métier** — pas comme un défaut. M01 est le cas le plus grave, parce que sa rupture de **3 jours tombe pendant le pic de juillet** (882 consultations, le mois max de la série, C03) : 3 jours de rupture de médicament essentiel **en saison des pluies** est l'information la plus chère de la table `stock_medicament.csv` — l'effacer (« corriger les zéros »), c'est effacer le problème.

Attention. Corriger un **zéro-fait** est l'erreur la plus chère du chapitre — plus chère que laisser un défaut : un stock négatif non corrigé biaise une médiane, une rupture « corrigée » efface une information **d'exploitation** (patients non servis, date de restock) que le métier attendait du rapport. La frontière est la récurrence : un zéro qui s'explique par épuisement est un fait, un zéro qui casse la récurrence (entre 548 et 541) est un défaut.

Dans les faits. L'exercice d'évaluation E4 du module est fait pour ça : « l'anomalie qu'il ne fallait pas corriger ». Le piège est symétrique des chapitres précédents : C02/C04 apprennent à corriger les défauts de saisie (−12 → 545) ; E4 apprend à **ne pas** corriger les zéros qui sont des faits (ruptures) — la frontière est la **réccurrence** : un zéro qui s'explique par épuisement est un fait, un zéro qui ne s'explique pas (entre 548 et 541) est un défaut.

5.5 Les raretés qui ne sont pas des anomalies — le plateau 594 363

Sur le fil rouge, le tri par montant fait apparaître **6 ventes au maximum absolu : 594 363 FCFA** — identiques, et pour cause. Exécuté :

id_vente	date_vente	id_magasin	id_client	id_produit	quantite	prix_unitaire_ht
3847	2026-10-13	1	390	18	10	49946.47
6880	2026-06-01	2	511	18	10	49946.47
15533	2026-06-15	1	282	18	10	49946.47
42671	2026-04-04	1	858	18	10	49946.47
44168	2026-01-08	1	583	18	10	49946.47
47199	2026-10-11	2	231	18	10	49946.47

L'accusation était évidente (6 fois le même montant au maximum, c'est **du copié-collé**, non ?). Les témoins, exécutés :

1. **Position** : 594 363 est le maximum **possible** de la base : 10 unités (le maximum de `quantite`, qui plafonne à 10) $\times$ 49 946.47 (le prix le plus élevé du catalogue : produit 18) $\times$ 1.19 (la TVA la plus haute, 0.19) $\approx$ 594 363. Le plateau n'est pas une répétition, c'est un **plafond structurel** : deux ventes qui tombent toutes deux sur la limite se valent.
2. **Récurrence** : les 6 ventes ont des `id_vente` distincts, des clients distincts (390, 511, 282, 858, 583, 231), des magasins (1, 2) et des dates distinctes (6 mois d'2026) — aucune paire ne se ressemble **hors** du montant.
3. **Métier** : un professionnel qui achète 10 unités du produit le plus cher, c'est une vente **tout à fait possible** (et elle représente 56 % des montants au-dessus de cent mille FCFA — la rareté n'est pas si rare).

Verdict : garder, aucune correction — et écrire la phrase qui sauve le rapport : « 6 ventes au montant max (594 363 FCFA) : plafond structurel ($10 \times \text{produit } 18 \times \text{TVA } 0.19$), clients et dates distincts, aucune anomalie » . Le même raisonnement couvre le **plus gros mois** 78 965 529 FCFA (2025-08 $\times$ magasin 3, C03 : un pic **expliqué** par la vue par magasin, pas écarté) et le **top client** de M07 (un client, pas un bug — le rappel du module).

Définition. anomalie structurelle — un défaut qui révèle un **mécanisme** (export, saisie, génération) plutôt qu'une erreur ponctuelle : les 8 doublons du fil rouge (clé métier redite, `id_vente` distinct), les 4 stocks négatifs déductibles. Le traitement d'une anomalie structurelle se fait **en amont** (la source, le processus) : dans le rapport, on la **documente** avec le mécanisme, pas seulement la valeur.

6. Exemple concret — les 594 363, jugés en entier (fil rouge)

Le parcours complet d'une rareté suspecte, du signal au verdict (exécuté) :

1. **Signal** : `montant_ttc.nlargest(6)` $\rightarrow$ 6 fois 594 363, le maximum de 50 008 lignes ; le tri par montant distincts montre que le reste est **dense** (les 10 montants les plus fréquents comptent 16 à 17 occurrences chacun — pas de pic isolé).
2. **Hypothèse d'accusation** : copié-collé (6 valeurs identiques au max).
3. **Preuves** : plafond structurel ($10 \times 49\,946.47 \times 1.19 \approx 594\,363$, produit 18 = prix max du catalogue de 380 produits), `id_vente` / clients / magasins / dates tous distincts, aucune paire jumelle hors montant.
4. **Verdict : garder, aucune correction** — rareté légitime, phrase de rapport écrite (§5.5).
5. **Leçon** : le tri par valeur est un **générateur de signaux**, pas un juge — sans les 3 témoins, le « 6 fois identique » aurait été traité en doublons, et le rapport aurait enlevé 5 ventes réelles de 594 363 au CA.

7. Démonstration pas à pas — 5 étapes sur les deux jeux

```
import pandas as pd
s = "03_exercices/dossier_M09"
cons = pd.read_csv(s + "/sante/consultation.csv")
stk = pd.read_csv(s + "/sante/stock_medicament.csv")
v = pd.read_csv(s + "/quincaillerie/vente.csv")
pr = pd.read_csv(s + "/quincaillerie/produit.csv")

# 1 – les 4 stocks négatifs, prouvés par récurrence (pandas)
stk["date"] = pd.to_datetime(stk["date"])
for mid in ("M05", "M07", "M10", "M12"):
    m = stk[stk["id_medicament"] == mid].sort_values("date").reset_index(drop=True)
    neg = m[m["stock_fin"] < 0]
    for i in neg.index:
        nat = int(m.loc[i - 1, "stock_fin"] + neg.loc[i, "entree"] - neg.loc[i, "sortie"])
        print(mid, neg.loc[i, "date"].date(), "réel", int(neg.loc[i, "stock_fin"]),
              "| naturelle", nat)

# 2 – les ruptures : les zéros qui sont des faits (M02)
m02 = stk[stk["id_medicament"] == "M02"].sort_values("date")
print(m02[m02["stock_fin"] == 0]["date"].dt.date.tolist())

# 3 – la date future : borne métier
print(int((cons["date_consultation"] > "2026-12-31").sum()),
      cons.loc[cons["date_consultation"] > "2026-12-31", "date_consultation"].tolist())

# 4 – le plateau du max : plafond structurel ?
mx = int(v["montant_ttc"].max())
top6 = v[v["montant_ttc"] == mx]
print(int(top6.shape[0]), "ventes au max", mx)
print(top6[["id_vente", "id_client", "id_magasin", "date_vente"]].to_string(index=False))
print("max quantite:", int(v["quantite"].max()),
      "| prix max:", pr["prix_vente_ht"].max(), "produit",
      int(pr.loc[pr["prix_vente_ht"].idxmax(), "id_produit"]))

# 5 – le verdict, en mots :
# 4 stocks : CORRIGER (valeurs déductibles) ; date future : EXCLUDE (à confirmer) ;
# ruptures : GARDER (signal métier) ; plateau 594363 : GARDER (plafond structurel).
```

```
M05 2025-04-18 réel -1 | naturelle 411
M07 2025-10-09 réel -3 | naturelle 253
M10 2026-05-21 réel -2 | naturelle 278
M12 2026-11-03 réel -12 | naturelle 545
[datetime.date(2026, 3, 2), datetime.date(2026, 3, 3), datetime.date(2026, 3, 4),
 datetime.date(2026, 3, 5), datetime.date(2026, 3, 6)]
1 ['2027-01-05']
6 ventes au max 594363
id_vente id_client id_magasin date_vente
3847      390      1 2026-10-13
6880      511      2 2026-06-01
15533     282      1 2026-06-15
42671     858      1 2026-04-04
44168     583      1 2026-01-08
47199     231      2 2026-10-11
max quantite: 10 | prix max: 49946.47 produit 18
```

8. Erreurs fréquentes

1. **Traiter le soupçon comme l'anomalie** — « c'est au-delà de la borne, on supprime » : la borne **accuse**, la preuve **condamne** ; sans preuve, la valeur sort du rapport avec son étiquette de soupçon, pas du fichier.

2. **Corriger une valeur non déductible** — inventer le motif perdu, deviner la date coquille : une correction sans source (récurrence, référentiel, source métier) est une **fabrication de donnée**, pas un nettoyage.
3. **Effacer les zéros qui sont des faits** — les ruptures (0 stock) corrigées « parce que c'est bizarre » : on efface le signal métier le plus grave du jeu (M01 : 3 jours en juillet, pendant le pic de 882 consultations).
4. **Punir le rare comme fautif** — les 6 ventes à 594 363 traitées en doublons, le top client de M07 exclu « parce qu'il tire le total » : la rareté légitime s'**explique** (plafond structurel, comportement client), elle ne s'élimine pas.
5. **Décider sans ligne de rapport** — « on a nettoyé » sans définition, sans nombre, sans preuve : le rapport P2 lit la **ligne**, pas le résultat — « 7 lignes retirées (doublons 5-tuples) » passe, « nettoyage effectué » non.

9. Bonnes pratiques professionnelles

1. **La borne d'abord, la valeur ensuite** — « au-delà de 446 (Q1 M12 – 1,5 × iqr) » est défini **avant** le tri ; « ça fait bizarre » ne l'est pas : un seuil sans date d'adoption n'accuse personne.
2. **Deux témoins pour les chiffres, un seul suffit pour le métier** — la borne métier parle seule (–12, 2027) ; pour les valeurs du domaine (442 ? 545 ?), position + récurrence ensemble.
3. **Chaque décision écrit sa ligne** — corriger : valeur + source ; exclure : définition + nombre ; garder : le pourquoi. Trois phrases, trois formats, pas de quatrième.
4. **L'anomalie qui compte est la plus chère du rapport** — la rupture M01 (3 jours au pic de juillet) vaut plus que les 4 corrections de stocks : le rapport la met en avant, chiffrée, datée, avec sa conséquence (patients non servis).
5. **Le tri par valeur est un générateur de signaux** — pas un juge : chaque pic de `nlargest` passe par les 3 témoins avant de devenir un fait ; le plateau 594 363 en est la démonstration (signal le plus suspect, verdict le plus légitime).

Conseil professionnel. Dans un rapport P2, la section anomalies tient en **une table** : signal / preuve(s) / décision / ligne de rapport. Pas de narrative interminable : le lecteur doit pouvoir **contrôler** chaque ligne (re-jouer la preuve), et une table se contrôle, un récit non. Cinq lignes sur le socle (23, 7, 1, 4, plateau) couvrent tout le chapitre.

10. Exercice guidé — « les 4 défauts du centre de santé, jugés » (25 min, /10)

Consigne. Les 4 défauts de C02 sont connus (23 motifs vides, 7 doublons de stock, 1 date au 2027-01-05, 4 stocks négatifs). Pour chacun, sans rouvrir ATTENDU.json :

1. Nommez la **borne** (métier, iqr, période) qui l'accuse (4 × 0,5 pt).
2. Donnez la **preuve** retenue (1 pt) et, pour les 4 stocks, la **valeur déductible** de chacun (4 × 1 pt). (5 pts au total)
3. Écrivez la **décision** (corriger / exclure en documentant / garder en signalant) et la **ligne de rapport** qui va avec (4 × 0,5 pt). (2 pts)

11. Exercices autonomes

E1 — La preuve par récurrence en SQL (20 min). En DuckDB sur `stock_medicament.csv`, avec `LAG(stock_fin) OVER (PARTITION BY id_medicament ORDER BY date)` : listez les 4 stocks négatifs avec leur **valeur attendue par récurrence** (hier + entrées – sorties). La sortie doit donner $-12 \rightarrow 545$, $-3 \rightarrow 253$, $-2 \rightarrow 278$, $-1 \rightarrow 411$.

E2 — Le plateau qui n'est pas un doublon (20 min). Sur `vente.csv` : (1) comptez les ventes au montant max (594 363) et listez leurs `id_vente`, `id_client`, `id_magasin`, `date_vente` ; (2) montrez que le plateau est un **plafond structurel** (quantité max, prix max du catalogue, TVA 0.19) ; (3) concluez : doublon ou rareté légitime, et **pourquoi** une paire parmi les 6 n'est **pas** un doublon métier (rappel C01 : la définition du doublon sur les 12 colonnes hors `id_vente`).

12. Correction détaillée

Exercice guidé.

- Les bornes** : D1 — aucune borne (un motif vide n'est hors domaine, c'est une information **absente** : la « borne » est la complétude attendue) ; D2 — définition métier du doublon (5-tuple) ; D3 — **période** (l'export s'arrête au 2026-12-31) ; D4 — **borne métier** ($\text{stock} \geq 0$).
- Les preuves** : D1 — aucune (l'information est perdue : c'est précisément pour ça qu'on ne la corrige pas) ; D2 — l'identité du 5-tuple (définition exécutée) ; D3 — borne métier + la position de C000001 (1^{re} référence du fichier) ; D4 — **double** : borne métier + récurrence, valeurs déductibles **411 (M05), 253 (M07), 278 (M10), 545 (M12)**.
- Les décisions** : D1 — **garder en signalant** (« 23 consultations hors des taux par motif ») ; D2 — **exclure en documentant** (« 7 lignes retirées : doublons exacts du 5-tuple, `keep="first"` ») ; D3 — **exclure de la série en documentant** (« 1 consultation hors période exclue ; coquille probable 2025-01-05, à confirmer avec la source ») ; D4 — **corriger** (« 4 valeurs de saisie remplacées par la valeur déductible de la récurrence : 411, 253, 278, 545 »).

E1. DuckDB, exécuté — la requête du §5.2 redonne exactement :

id_medicament	date	reel	attendu_réurrence
M12	2026-11-03	-12	545
M07	2025-10-09	-3	253
M10	2026-05-21	-2	278
M05	2025-04-18	-1	411

E2. (1) **6** ventes au max 594 363 : 6 `id_vente` distincts (la liste est publiée aux §5.5 et §7), clients 390, 511, 282, 858, 583, 231 ; magasins 1 (×4) et 2 (×2) ; dates du 2026-01-08 au 2026-10-13. (2) **Plafond structurel** : quantité plafonne à **10**, le produit **18** est le plus cher du catalogue (**49 946.47**), la TVA la plus haute est **0.19** : $10 \times 49\,946.47 \times 1.19 \approx 594\,363$ — deux ventes qui touchent toutes les limites se valent, c'est la définition du plafond. (3) **Rareté légitime, pas un doublon** : la définition du doublon métier (C01) porte sur les 12 colonnes **hors** `id_vente` — et ici, les 6 ventes diffèrent sur `date_vente`, `id_client`, `id_magasin` (et `id_vente`) : aucune paire n'est identique **sur la définition**, donc aucune n'est un doublon ; l'identité du montant est l'**effet** du plafond, pas sa **cause**.

13. Mini-projet M09.P4 — « L'anomalie avec sa preuve » (1 h)

Énoncé. La section anomalies du rapport P2 (projet M09.P) doit porter **la table du conseil §9** sur un jeu au choix (quincaillerie ou santé) : 4 à 6 lignes, chacune avec signal / preuve(s) / décision / ligne de rapport. Contrainte d'évaluation : **chaque preuve doit être re-jouable** — le cor-

recteur relance la commande citée et doit retomber sur le chiffre. Auto-évaluation : une ligne sans commande citée = une ligne qui ne compte pas ; une décision « corriger » sans valeur déductible et sa source = une ligne fausse.

14. Boîte à outils du chapitre

Boîte à outils. Les seuils : `quantile(.25/.75) + iqr (C02)`, bornes métier (`col < 0`, `col > 20`, date hors période), période (C03 : écart au cycle). Les preuves : position (`nlargest`, distance à la borne), **récurrence** (`shift(1)` en pandas, `LAG() OVER (PARTITION BY ... ORDER BY date)` en DuckDB), métier (le référentiel, la source). Les décisions : corriger (valeur + source), exclure (définition + nombre), garder (signal + phrase). Le rare : `nlargest` + les 3 témoins avant tout traitement.

Besoin	Commande	Chiffre du socle
position	<code>col.nlargest(n)</code> , distance à Q3 + 1,5 × iqr	6 ventes au max 594 363
récurrence	<code>shift(1) / LAG() OVER</code>	545, 253, 278, 411
borne métier	<code>(col < 0).sum()</code> , date hors période	4 stocks, 1 date (2027-01-05)
zéros-faits	<code>col == 0</code> groupés par période	M01 3 j · M02 5 j · M04 2 j
plafond	prix max catalogue × quantité max × TVA max	10 × 49 946.47 × 1.19

15. Résumé du chapitre

L'étape 6 du protocole (« qu'est-ce qui est inhabituel, et comment le **prouver** ? ») est exécutée avec la règle du chapitre : **une anomalie non prouvée est un soupçon**, et trois témoins sont disponibles — la **position** dans la distribution (iqr, C02), la **récurrence** (hier + entrées – sorties : `shift(1)` en pandas, `LAG` en DuckDB) et la **cohérence métier** (stock ≥ 0, période de l'export, note sur 20). Sur le centre de santé, les 4 défauts de C02 sont jugés : les 23 motifs vides se **gardent en signalant** (information perdue, non reconstituable), les 7 doublons de stock s'**excluent en documentant** (5-tuple identique, 7 lignes), la consultation du 2027-01-05 s'**exclut de la série** (borne période, C000001, coquille probable 2025-01-05 à confirmer avec la source) et les 4 stocks négatifs se **corrigent** — en 411, 253, 278 et 545, valeurs **déductibles** par la récurrence (preuve double : métier + flux). Les **ruptures** (M01 : 3 jours, 2025-07-14 → 16 ; M02 : 5 jours, 2026-03-02 → 06 ; M04 : 2 jours, 2026-08-10 → 11) sont l'inverse exact d'un défaut : des **zéros qui sont des faits**, à garder et à traiter comme **signal métier** — M01 pendant le pic de juillet (882 consultations) est l'anomalie qui **compte**. Sur le fil rouge, les **6 ventes à 594 363 FCFA** (le maximum de 50 008 lignes) démonurent le rare légitime : plafond structurel (10 × produit 18 à 49 946.47 × TVA 0.19), clients, magasins et dates distincts — **garder, aucune correction**, et la même leçon couvre le plus gros mois (78 965 529, expliqué) et le top client de M07.

16. À retenir

À retenir. La borne **accuse**, la preuve **condamne**, la décision **écrit sa ligne** — et les deux contre-exemples du socle gardent le chapitre honnête : la rupture qui ne se corrige pas (0 = un fait, pas un défaut) et le plateau 594 363 qui ne s'élimine pas (rare ≠ fautif).

1. **Définir avant de chercher** : iqr / borne métier / période — un seuil sans date d'adoption n'accuse personne.

2. **Trois témoins** : position (500+ boîtes du cœur pour M12), récurrence ($545 \neq -12$, et $545 - 4 = 541$), métier (pas de -12 boîtes en pharmacie).
3. **Trois décisions, jamais la quatrième** : corriger (valeur déductible + source), exclure en documentant (définition + nombre), garder en signalant — pas « supprimer ».
4. **Corriger = déduire** : 411, 253, 278, 545 se **calculent** ; une valeur non déductible ne se corrige jamais par invention.
5. **L'anomalie qui compte est un fait** : ruptures M01 3 j (pic de juillet, 882), M02 5 j, M04 2 j — les corriger, c'est effacer le problème.
6. **Rare $\neq$ fautif** : $6 \times 594\,363 =$ plafond structurel ($10 \times 49\,946.47 \times 1.19$) ; le tri par valeur génère des signaux, il ne juge pas.
7. **Chaque décision écrit sa ligne** : signal / preuve / décision / ligne de rapport — la table P4 se **re-joue**, le récit non.

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi 442 (M12, sous la borne 446) n'est-il pas corrigé alors que -12 l'est ?** → 442 est un soupçon **débatu** : cohérent avec sa récurrence, plausible en métier (jour bas, pas impossible) — il se **garde** ; -12 a la **double** preuve (métier : pas de stock négatif ; récurrence : $548 - 3 = 545$, et $545 - 4 = 541$) — il se **corrige** en 545, valeur déductible. (2 pts)
2. **Les 4 stocks négatifs valent -1 , -3 , -2 , -12 . Donnez leurs valeurs déductibles et la commande qui les déduit.** → 411 (M05), 253 (M07), 278 (M10), 545 (M12) ; `shift(1)` en pandas ou `LAG(stock_fin) OVER (PARTITION BY id_medicament ORDER BY date)` en DuckDB : hier + entrées - sorties. (2 pts)
3. **Pourquoi la rupture M02 (5 jours à zéro, 2026-03-02 → 06) ne se corrige-t-elle pas ?** → parce que les zéros sont **vrais** : la borne stock ≥ 0 ne les accuse pas (0 est dans le domaine), la récurrence les explique (épuisement : sorties > entrées), et le métier les confirme (rupture = fait de gestion) — c'est un **signal** (patients non servis), pas un défaut. (2 pts)
4. **M01 est rompu 3 jours en 2025-07-14 → 16. Pourquoi cette rupture « compte-t-elle » plus que les autres ?** → parce qu'elle tombe **pendant le pic de saison** (juillet 2025 : 882 consultations, le mois max de la série) : 3 jours de médicament essentiel manquant **en saison des pluies** — l'information la plus chère de la table de stock. (2 pts)
5. **6 ventes à 594 363 FCFA, le maximum de la base. Pourquoi ne sont-ce pas des doublons ?** → le doublon métier se définit sur les 12 colonnes **hors id_vente** (C01) : les 6 ventes diffèrent sur date, client, magasin — l'identité du montant est l'**effet** du plafond structurel ($10 \times$ produit $18 \times$ TVA 0.19), pas sa cause. (2 pts)
6. **Quelle est la valeur exacte du plafond, et pourquoi « $\approx$ » ?** → $10 \times 49\,946.47 \times 1.19 = 594\,362.99 \approx$ **594 363** : le montant est arrondi au FCFA (centimes entiers, M08 C08) — l'approximation est l'arrondi, pas l'hypothèse. (2 pts)
7. **Écrivez la ligne de rapport de D3 (la date 2027-01-05), avec sa nuance.** → « 1 consultation hors période (2027-01-05, C000001), **exclue des séries** ; coquille probable pour 2025-01-05 — correction **soumise à la source** » : la preuve (borne période) exclut, elle ne **corrige** pas. (2 pts)
8. **Une valeur au-delà de la borne, non explorée faute de temps. Que fait-on ?** → on la **signale** en soupçon (« au-delà de la borne, non explorée ») : elle entre au rapport **étiquetée**, elle ne sort pas du fichier — traiter un soupçon, c'est confondre accusation et verdict. (2 pts)

Module M09.C05 — Relations et hypothèses : ce qui bouge ensemble, ce qui ne bouge pas, et les 3 fausses causes

Outils : pandas 2.2.3, numpy 2.3.5 (exécutés). Durée indicative : 5 h. Niveau : N3. Pré-requis : M09.C01-C04 (protocole, audit, tendances, anomalies), M02 (corrélation vue en M02.C06).

L'idée du chapitre. L'étape 7 du protocole pose la question : « **les variables bougent-elles ensemble ?** » et l'étape 8 : « **que puis-je affirmer, et sous quelle condition ?** » C05 y répond avec un coefficient en une ligne (`corr()`), ses **3 limites** (linéarité, scale, valeurs extrêmes), le **corollaire du module** — corrélation $\neq$ causalité, avec les **3 fausses causes** chiffrées sur le socle (variable tierce, régression vers la moyenne, sens inversé) — et deux disciplines : **segmenter** avant de comparer, et **comparer des groupes à taille égale ou en le disant**. L'établissement scolaire fait son entrée ici (402 élèves déclarés, 6 000 notes, 1 687 absences/retards, 4 défauts) : la corrélation absentéisme $\times$ résultats est **-0.232** — négative, modérée, et **partiellement expliquée par une tierce variable** quand on contrôle le niveau du T1. Le fil rouge porte l'autre leçon : remises $\times$ montants à 0.552, taux $\times$ montants à **0.001** — le scale n'est pas le comportement.

Matériel de l'atelier — Python 3.13 · pandas 2.2.3 · numpy 2.3.5. Toutes les commandes ont été exécutées dans l'atelier le 20/09/2026 sur `03_exercices/dossier_M09/` (graine 45, déterministe, ATTENDU.json figé) ; les sorties publiées sont **celles de l'atelier**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Calculer la corrélation de Pearson** en une ligne (`corr()`) et dire ses **3 limites** : elle ne mesure que le lien **linéaire**, elle dépend du **scale** des variables (0.552 vs 0.001 sur le fil rouge), et une **valeur extrême** peut la tirer.
2. **Reconnaître les 3 fausses causes** d'une corrélation : variable **tierce** (confond — mesurée sur le socle : la corrélation s'effondre quand on contrôle le T1), **régression vers la moyenne** (les extrêmes retournent vers le milieu sans cause) et **sens inversé** (l'effet va de B à A, pas de A à B).
3. **Segmenter** une variable continue en groupes métier (`np.where`) et lire le **gradient** : 3 segments d'absentéisme [123, 208, 69] élèves $\rightarrow$ moyennes 11.13 $\rightarrow$ 10.51 $\rightarrow$ 9.76.
4. **Comparer des groupes à taille inégale** : le panier par mode de paiement (5 groupes, le plus gros **8 fois** plus grand que le plus petit, écart de moyennes de 2 % : du bruit) et le taux par classe (4e : 4.26 j, 2nde : 2.35 j — un écart **réel** à n égales).
5. **Cadrer et tester l'hypothèse** (l'étape 8) : formulation falsifiable, test sur les données, **conclusion provisoire** avec ses limites — le pont vers la note d'EDA (C06).

2. Pourquoi cette notion est importante

L'EDA se termine là où commence la **responsabilité** : l'étape 7 produit des associations, l'étape 8 décide ce qu'on en **dit**. Et c'est précisément là que le rapport EDA se trompe le plus souvent :

une corrélation publiée comme une cause, un groupe petit comparé à un groupe grand, un segment défini après avoir vu les résultats. Trois faits mesurés sur le socle fixent l'enjeu :

- la corrélation **absentéisme × résultats** du jeu scolaire est **-0.232** — et quand on **contrôle le niveau du T1** (la tierce variable : le niveau d'origine), elle s'effondre par bandes (-0.145, -0.085, **+0.163**) : une partie du lien brut est la variable tierce, pas l'absence ;
- sur le fil rouge, **remises (en FCFA) × montants** donne 0.552, mais **taux de remise × montants** donne **0.001** : même fichier, deux variables, deux mondes — le scale a imité un comportement ;
- les **5 modes de paiement** font des paniers moyens à **2 % près** les uns des autres, pour des groupes de 20 175 ventes et de 2 552 : comparer ces groupes « au total » sans dire leurs tailles, c'est faire passer le bruit pour une différence.

3. Explication simple — la conversation avec la machine

La corrélation est une **corde** qui relie deux variables : si l'une tire, l'autre suit, la corde est tendue (+) ou opposée (-), et `corr()` mesure la tension de **-1 à +1**. Mais la corde joue **trois tours** au lecteur pressé :

1. **Le tour de la tierce variable** : deux variables bougent ensemble **parce qu'une troisième les pousse toutes les deux** — les élèves absents et les élèves qui décrochent sont **poussés par le même facteur** (le niveau d'origine) ; la corde entre absence et note est un fil de la variable tierce, pas un lien direct.
2. **Le tour du scale** : deux grandeurs qui grandissent **ensemble** (une remise en FCFA grandit avec le montant de la facture) se corrôlaient **même sans lien** — 0.552 ; divisez par le montant (le **taux**) et la corde retombe : **0.001**.
3. **Le tour des extrêmes** : un groupe qui part très bas (7.01 au T1) remonte vers le milieu (7.21 au T2) **sans que rien ne l'ait aidé** — la régression vers la moyenne imite une cause qui n'existe pas.

La discipline du chapitre : **tendre la corde** (`corr()`), **vérifier le tour** (les 3 pièges), et **dire la tension en un chiffre** avec ses limites — jamais « ça cause ça ».

4. Vocabulaire essentiel

Terme	Définition
corrélation	corrélation — la mesure d'association entre deux variables (Pearson : de -1 à +1, lien linéaire) ; n'est jamais une causalité (le piège du module).
piège d'inférence	le saut interdit de l'association à la cause : 3 formes mesurées — variable tierce, régression vers la moyenne, sens inversé.
variable tierce — confond — la troisième variable qui cause les deux : si on la contrôle (on compare à elle égale), la corrélation résiduelle est le lien direct.	
segment	un groupe défini avant de regarder les résultats (bornes métier sur la variable) : ici, les 3 tranches d'absentéisme (< 2, 2-5, ≥ 6 jours).

5. Cours approfondi

5.1 `corr()` en une ligne — et ses 3 limites

```
import pandas as pd
s = "03_exercices/dossier_M09/scolaire"
no = pd.read_csv(s + "/note.csv")
ab = pd.read_csv(s + "/absence.csv")
no = no[(no["note"] >= 0) & (no["note"] <= 20)]
abs_par = ab.groupby("id_eleve")["duree_jours"].sum()
taux = abs_par.reindex(sorted(no["id_eleve"].unique()), fill_value=0)
moy = no.groupby("id_eleve")["note"].mean()
print(round(float(moy.corr(taux.reindex(moy.index))), 3), len(moy))
```

```
-0.232 400
```

-0.232 sur 400 élèves : une association **négative, modérée** — les élèves plus absents ont des moyennes plus basses, mais « modérée » est le mot qui compte : une grande partie de la variance de la moyenne n'est **pas** dans l'absentéisme. Les 3 limites, chacune exécutée sur le socle :

1. **Linéarité** — `corr()` mesure le lien **linéaire** ; une relation en U (les absents très peu **et** très beaucoup font mal, les moyens vont bien) donne une corrélation **nulle** tout en ayant un lien fort. Le nuage (C06) est le contrôle.
2. **Scale** — sur le fil rouge (exécuté) : `montant_remise × montant_ttc → 0.552`, mais `montant_remise / montant_ttc × montant_ttc → 0.001` : la remise en FCFA grandit **avec** le montant (une facture de 500 000 FCFA a une remise plus grosse en FCFA qu'une facture de 5 000, même à taux nul) — le scale a imité un comportement. Le **taux** (max 0.015 sur la base) est la variable qui mesure le comportement, et elle ne bouge **pas** avec le montant.
3. **Valeurs extrêmes** — une poignée de valeurs à l'extrême peut **tirer** le coefficient (le cas de M02 : une ligne aberrante qui fait une corrélation) ; `corr()` a des variantes robustes (rangs), mais le réflexe du chapitre est plus simple : **recalculer sans les 5 % extrêmes** et voir si le signe tient.

Attention. Un coefficient dont le **signe change** quand on retire les valeurs extrêmes n'est pas une mesure : c'est un fil tendu par quelques observations. Le réflexe d'office avant publication est de le recalculer **sans** les extrêmes — si le signe tient, la corrélation est robuste ; s'il s'effondre, c'est la queue qui parlait, pas la distribution.

Définition. corrélation — corrélation — la mesure d'association entre deux variables ; Pearson mesure le lien **linéaire**, de -1 (opposé) à +1 (proportionnel), 0 = pas de lien linéaire. Trois avertissements d'office : elle ne mesure **pas** la causalité, pas le lien non linéaire, et pas le comportement si l'une des variables est un **scale** (montant en FCFA) et non un **taux**.

5.2 Corrélation ≠ causalité — les 3 fausses causes, chiffrées

Fausse cause 1 — la variable tierce (confond). L'histoire la plus plausible : « les absences **font** baisser les notes ». Le test (exécuté) : contrôler le **niveau du T1** (proxy du niveau d'origine) et recalculer, **par bande de niveau** :

```
corr globale abs_T2 × note_T2 : -0.206
bande T1 [8,10) (n=107) : -0.145
bande T1 [10,12) (n=139) : -0.085
bande T1 [12,14) (n=77) : +0.163
```

En contrôlant le niveau, la corrélation **s'effondre** ($-0.206 \rightarrow -0.145 / -0.085 / +0.163$, du signe inversé dans la bande haute) : une part importante du lien brut était **le niveau d'origine** qui pousse l'absence **et** la note — la corde entre absentéisme et résultats est en grande partie un fil de la tierce variable. Ce que les données **autorisent** : « les élèves plus absents ont en moyenne des notes plus basses, **et** le niveau d'origine explique une part de ce lien » — pas « les absences font baisser les notes ».

Fausse cause 2 — la régression vers la moyenne. Les élèves très en bas au T1 (moyenne 7.01) font **7.21** au T2 — ils **montent** sans cause identifiée ; les très hauts (14.93) font **14.61** — ils **descendent**. Les extrêmes retournent vers le milieu **par hasard d'échantillonnage** (une note anormalement basse a statistiquement de chances d'être moins basse au trimestre suivant), et on lit « l'aide a marché » ($7.01 \rightarrow 7.21$) là où il n'y a qu'une **retombée vers la moyenne**. Le contrôle : comparer à la **moyenne de tous** les élèves sur la même transition, pas à soi-même.

Fausse cause 3 — le sens inversé. « Les absents ont de mauvaises notes » peut aller **dans l'autre sens** : ce ne sont pas les absences qui font la moyenne, c'est la **moyenne qui fait l'absence** (l'élève qui décroche arrête d'aller en cours). Les données du fichier **ne départagent pas** les deux sens (pas de variable « motivation », pas d'intervention) — et c'est précisément **ce qu'on écrit** : « association bidirectionnellement compatible ; le fichier ne départage pas les sens ».

Définition. piège d'inférence — le saut interdit de l'**association** à la **cause**, sous ses 3 formes mesurées : variable **tierce** (la troisième variable qui cause les deux — contrôlée, la corrélation résiduelle est le lien direct), **régression vers la moyenne** (les extrêmes retournent vers le milieu sans cause), **sens inversé** (B cause A, on affirme A cause B). Un coefficient sans ces 3 contrôles est une **association**, pas une **cause**.

Corrélation, jamais causalité : les 3 fausses causes, chiffrées sur le socle

Le brut : absentéisme x moyenne générale = -0.232 ($n=400$), association négative modérée. Trois façons de la lire en trop :

1 — VARIABLE TIERCE (confond)

la tierce variable (le niveau d'origine)

contrôle par bande de T1 :

brut (abs T2 x note T2) : -0.206

bande T1 [8,10] $n=107$: -0.145

bande T1 [10,12] $n=139$: -0.085

bande T1 [12,14] $n=77$: $+0.163$

le lien s'EFFONDRE (et change de signe)

une part du lien brut = tierce variable,
pas l'absence en elle-même.

2 — RÉGRESSION VERS LA MOYENNE

les extrêmes reviennent vers le milieu

la transition T1 -> T2 :

T1 < 8 (7.01) -> T2 : 7.21 (monte)

T1 > 14 (14.93) -> T2 : 14.61 (desc.)

moyenne globale : 10.57

les deux sens, même chiffre : 10.57

les deux extrêmes reviennent vers 10.57

sans qu'on les ait aidés : c'est la variance
d'une note, pas « l'aide a marché ».

3 — SENS INVERSÉ

B cause A, on affirme A cause B

le fichier ne départage pas :

absence -> note ? (les absents chutent)

note -> absence ? (chute -> absences)

pas d'intervention, pas de variable

motivation ni d'ordre chronologique

les deux sens restent compatibles avec

les mêmes chiffres : le fichier ne tranche

Phrase : association, sens non départagé.

Le coefficient va avec ses 4 chiffres : $-0.232 \cdot n = 400 \cdot$ contrôle (bandes de T1) · limite (proxy, 1 année, sens)

les 4 ensemble, ou la phrase ne part pas. Test du scale : montants 0.552 -> taux 0.001 — le scale n'est pas le comportement.

Segments (bornes métier, avant de regarder) : 123 / 208 / 69 élèves -> moyennes 11.13 / 10.51 / 9.76

gradient régulier = la signature d'un lien direct (pas une fausse cause).

5.3 L'établissement scolaire arrive — le jeu 3, ses 4 défauts

Le jeu 3 s'ouvre (entré avec son audit C02 déjà fait ici) : `eleve.csv` (402 lignes × 3 colonnes, 400 élèves distincts), `note.csv` (6 000 lignes × 5 colonnes : 5 matières × 400 élèves × 3 trimestres), `absence.csv` (1 687 lignes × 5 colonnes : 1 278 absences et 409 retards). Les 4 défauts plantés, exécutés :

#	Défaut	Définition exacte	Chiffre
S1	notes hors bornes	<code>note < 0</code> ou <code>> 20</code> (sur 20, domaine métier)	6 (22.5, 22.5, 25.0, -1.5, -0.5, -3.0)
S2	doublons d'élèves	<code>id_eleve</code> répété dans <code>eleve.csv</code>	2 (E0107, E0233)
S3	absence un jour férié	absence datée un jour où l'établissement est fermé	4 (2025-11-01, 2025-12-25, 2025-12-26, 2026-01-01)
S4	absent mais noté	élève avec ≥ 10 absences au T2 et des notes au T2	3 (E0004 : 13, E0006 : 15, E0007 : 15)

Deux lectures qui feront la différence :

- **S2 cache une contradiction** : l'un des deux IDs doublons est déclaré **F** sur une ligne et **M** sur l'autre (même `classe`) — c'est un doublon **avec contradiction**, pas un simple doublon exact (l'autre est identique sur ses 3 colonnes) : le premier se règle **avec la source**, le second se dédoublonne.
- **S4 est le défaut qui fausse la corrélation** : 3 élèves comptent **13, 15, 15 absences au T2** et ont **5 notes au T2** chacun — des absents notés tirent la corrélation vers le bas **par construction** (max des absences, notes quand même présentes) : la corrélation -0.232 se **recalcule** hors ces 3 élèves au C06, et l'écart est publié (les deux chiffres vont dans la note d'EDA).

Dans les faits. L'audit du jeu scolaire (formes + 4 défauts + types d'absence + clés refermées) passe par les mêmes commandes que le jeu 2 — le protocole (C01) est le même sur les 3 jeux ; ce qui change, c'est le **métier** des bornes : une note sur 20, un jour férié, un élève noté en étant absent. Les bornes métier suivent le métier, pas l'inverse.

5.4 Segmenter — les 3 tranches d'absentéisme et leur gradient

Ségmenter, c'est trancher avant de regarder : les bornes (< 2 , $2 \text{ à } 5$, ≥ 6 jours d'absence sur la période) sont fixées **métier** (« absent occasionnel / absent régulier / absent structurel »), **pas** après avoir vu les moyennes (exécuté, `np.where`) :

```
segment <2 jours : n=123 | moyenne 11.13 | médiane 11.21
segment 2-5 jours: n=208 | moyenne 10.51 | médiane 10.40
segment >=6 jours: n=69  | moyenne  9.76 | médiane  9.51
```

Le **gradient** est la lecture du chapitre : $11.13 \rightarrow 10.51 \rightarrow 9.76$, un demi-point par tranche, **regulier** — c'est la signature d'un **lien direct** (pas d'un accident de groupe) ; la médiane suit la moyenne (pas de queue qui tire). Et la nuance du chapitre : le segment « ≥ 6 jours » n'est pas un groupe d'**exclus** — ce sont 69 élèves réels, et la phrase du rapport est « les 69 élèves les plus absents font en moyenne 9.76/20, soit 1.37 point sous les 123 les moins absents » — **des élèves, pas des valeurs**.

Définition. segment — un groupe défini **avant** de regarder les résultats : les bornes sont **métier** (< 2 , $2 \text{ à } 5$, ≥ 6 jours d'absence), pas statistiques, et s'écrivent dans le rapport **avant** les chiffres. Définir le segment après avoir vu les moyennes (« les 3 tranches qui maximisent l'écart »), c'est **choisir** la conclusion — le segment n'est plus une découpe, c'est un argument.

Définition. variable tierce — confond — la troisième variable qui **cause les deux** variables observées : ici, le niveau d'origine (proxy : la note du T1) pousse l'absence **et** la note du T2. La tester, c'est **contrôler** : comparer à elle égale (par bande) — la corrélation résiduelle est le lien **direct**, le reste est le fil de la tierce variable.

5.5 Comparer des groupes — à taille égale, ou en le disant

Le fil rouge fournit le contre-exemple (exécuté) : le panier moyen par **mode de paiement** (5 groupes) :

id_mode	n ventes	panier moyen	médiane
1	20175	157863	120134
2	12366	157205	116052
3	12322	159573	121794
4	2593	156150	120315
5	2552	159959	120545

Deux leçons :

1. **Les tailles inégales mentent** : le groupe 1 compte **8 fois** plus de ventes que le groupe 5 (20 175 vs 2 552) — une différence de panier de 2 % entre des groupes de cette taille est **du bruit** (la variance d'un groupe de 2 552 est bien plus grande que celle d'un groupe de 20 175). La phrase du rapport : « les 5 modes de paiement ne différencient **pas** le panier moyen (écart max 2 %, groupes de 20 175 à 2 552 ventes) » — l'écart **et** les tailles, les deux chiffres.
2. **Les moyennes suivent, les médianes disent** : les moyennes vont de 156 150 à 159 959 (2 %), les médianes de 116 052 à 121 794 (5 %) — la **médiane** est plus lisible ici (distribution à queue haute, C02), et les deux vont **ensemble** dans la phrase.

Le **bon** comparé de groupes sur le socle : le **taux d'absence par classe** (exécuté) — 5e : 3.35 j (n=110), 4e : **4.26 j** (n=105), 3e : 2.63 j (n=100), 2nde : 2.35 j (n=85) : les groupes sont **à taille proche** (85 à 110, rapport de 1.3), et l'écart 4e vs 2nde est de **1.91 jour** d'absence — un écart **réel**, pas du bruit de taille : « la 4e est la classe la plus absente (4.26 jours), à effectif proche des autres classes ».

Le **tableau croisé** (fréquence d'un couple de modalités) est le troisième regard, ici sur matière × trimestre (moyennes exécutées) :

matiere	T1	T2	T3
anglais	10.16	10.28	10.19
français	10.76	10.89	10.75
histoire_geographie	11.40	11.36	11.45
maths	9.83	9.91	9.85
sciences	10.62	10.52	10.59

Deux lectures : (1) **rien ne bouge entre trimestres** (chaque matière varie d'environ 0.1 point) — le tableau croisé **vide** est une information : pas d'effet de trimestre, l'étape se **déclare** (C01) ; (2) le **classement des matières est stable** : histoire_geographie (11.4) devant français (10.8), sciences (10.6), anglais (10.2) et maths (9.9) — la matière la plus « facile » n'est pas celle qui a la meilleure moyenne au T3, c'est celle qui en a eu **la meilleure en continu** : le classement se lit sur les 3 trimestres, pas sur un.

Définition. tableau croisé — tableau de contingence — la fréquence (ou la moyenne) d'un **couple de modalités** : matière × trimestre ici, 5×3 cases. Sa force est la **comparaison en cases** : « rien ne bouge entre trimestres » se voit sur les 3 colonnes d'une ligne, « le classement est stable » sur les 5 lignes d'une colonne — deux lectures que la moyenne globale (10.57) ne donne pas.

Attention. Comparer des groupes **sans dire leurs tailles**, c'est faire passer le bruit pour une différence : 2 % d'écart de panier sur 2 552 ventes n'est pas une « préférence de paiement », c'est de la variance. La phrase complète est toujours **écart + tailles** — sans les deux chiffres, la comparaison ne sort pas du brouillon.

5.6 Formuler l'hypothèse et la tester — l'étape 8

Le gabarit (C02 §5.5) s'applique à la relation (exécuté sur le socle) :

« Si l'absence fait baisser la note (effet direct), alors, à niveau T1 égal, plus d'absences au T2 doivent donner une note T2 plus basse — et si ce n'est pas le cas (ou si l'écart est nul), l'association vient de la tierce variable. »

Le test est le tableau du §5.2 : à niveau T1 contrôlé, la corrélation résiduelle est **faible et instable** ($-0.145 / -0.085 / +0.163$) — l'hypothèse « effet direct fort » est **rejetée sur la période** ; l'association brute (-0.232) **persiste** (les bandes restent globalement négatives) : la conclusion provisoire honnête est « association négative modérée, **partiellement** expliquée par le niveau d'origine ; le fichier ne permet pas de chiffrer la part exacte » — avec les limites (proxy T1, 1 année scolaire, 3 élèves absents-notés non exclus du brut).

À retenir. Une corrélation se **publie** avec ses 4 chiffres : la valeur (-0.232), la taille ($n=400$), le **contrôle** (bandes de T1), et la **limite** (proxy, période) — les 4 ensemble, ou la phrase ne part pas.

6. Exemple concret — le fil rouge : 0.552, 0.001, et la clientèle plate

Le parcours complet de la leçon scale → comportement (exécuté) :

1. **Le signal** : `corr(montant_remise, montant_ttc) = 0.552` — « les remises vont avec les gros montants » ?
2. **Le test de scale** : le **taux** de remise (`montant_remise / montant_ttc`, max **0.015** sur la base) corrélé au montant = **0.001** — la corde retombe : c'était le scale, pas un comportement. (45.5 % des ventes n'ont **aucune** remise : le taux est 0 pour elles, le montant non.)
3. **La clientèle, segmentée** : 1 200 clients, le **top 10 %** (120 clients) pèse **13.8 %** du CA — une clientèle **plate** (un top 10 % qui pèse 40 % serait concentré, 13.8 % ne l'est pas) ; le top 1 (client 73, 11 561 025 FCFA) pèse à lui seul **0.15 %** — le « gros client » de M07 reste **un client, pas un bug** (C04).
4. **La leçon** : chaque association du fil rouge a passé son test de scale (taux, pas montants) avant de devenir une phrase — et la phrase est « pas de lien entre le taux de remise et le montant », pas « remises et montants vont ensemble ».

7. Démonstration pas à pas — 5 étapes sur les deux jeux

```
import pandas as pd
s = "03_exercices/dossier_M09"
no = pd.read_csv(s + "/scolaire/note.csv")
ab = pd.read_csv(s + "/scolaire/absence.csv")
v = pd.read_csv(s + "/quincaillerie/vente.csv")

# 1 – la corrélation brute (scolaire)
no = no[(no["note"] >= 0) & (no["note"] <= 20)]
abs_par = ab.groupby("id_eleve")["duree_jours"].sum()
taux = abs_par.reindex(sorted(no["id_eleve"].unique()), fill_value=0)
moy = no.groupby("id_eleve")["note"].mean()
print(round(float(moy.corr(taux.reindex(moy.index))), 3))

# 2 – le contrôle de la tierce variable (bandes de T1)
t1 = no[no["trimestre"] == 1].groupby("id_eleve")["note"].mean()
t2 = no[no["trimestre"] == 2].groupby("id_eleve")["note"].mean()
abs_t2 = ab[(ab["date"] >= "2025-12-01") & (ab["date"] <= "2026-02-28")
            & (ab["type"] == "absence")].groupby("id_eleve").size()
d = pd.DataFrame({"t1": t1, "t2": t2, "abs": abs_t2.reindex(t1.index, fill_value=0)}).dropna()
print(round(float(d["abs"].corr(d["t2"])), 3))
print([round(float(d[(d.t1 >= lo) & (d.t1 < hi)]["abs"].corr(d[(d.t1 >= lo) & (d.t1 < hi)]
["t2"]))), 3)
      for lo, hi in [(8, 10), (10, 12), (12, 14)]]

# 3 – les segments (np.where) et leur gradient
seg = pd.cut(taux, [-0.1, 1.9, 5.9, 99], labels=["<2", "2-5", ">=6"])
print(pd.DataFrame({"seg": seg, "moy": moy}).groupby("seg", observed=True)["moy"]
      .agg(["count", "mean"]).round(2).to_string())

# 4 – le test de scale (fil rouge)
print(round(float(v["montant_remise"].corr(v["montant_ttc"])), 3),
      round(float((v["montant_remise"] / v["montant_ttc"]).corr(v["montant_ttc"])), 3))

# 5 – groupes à taille inégale : panier par mode (n + moyenne + médiane)
print(v.groupby("id_mode")["montant_ttc"].agg(["count", "mean", "median"]).round(0).to_string())
```

```
-0.232
-0.206
[-0.145, -0.085, 0.163]
      count      mean
seg
<2         123    11.13
2-5        208    10.51
>=6         69     9.76
0.552 0.001
      count      mean      median
id_mode
1        20175  157863.0  120134.0
2        12366  157205.0  116052.0
3        12322  159573.0  121794.0
4         2593  156150.0  120315.0
5         2552  159959.0  120545.0
```

8. Erreurs fréquentes

1. **Publier la corrélation comme cause** — « les absences font baisser les notes » avec -0.232 en note de bas de page : le chiffre prouve une **association**, les 3 fausses causes (§5.2) partagent l'hypothèse de cause — la phrase est « associés », jamais « causent ».
2. **Corréler un scale avec un montant** — 0.552 sur les remises en FCFA, **0.001** sur le taux : toujours s'assurer que la variable mesure le **comportement** (taux, part, fréquence), pas la **grandeur** (FCFA, unités).

3. **Comparer des groupes à taille inégale sans dire les tailles** — 2 % d'écart de panier sur 2 552 ventes : le bruit de taille fait l'écart — la phrase sans les n n'est pas une comparaison, c'est un accident.
4. **Définir le segment après avoir vu les moyennes** — « les 3 tranches qui maximisent l'écart de moyenne » : le segment **choisit** la conclusion ; les bornes (< 2 , 2-5, ≥ 6 jours) sont **métier** et précèdent le calcul.
5. **Oublier les absents-notés dans le calcul** — les 3 élèves du défaut S4 (13, 15, 15 absences au T2, 5 notes au T2 chacun) tirent la corrélation par construction : le chiffre brut (-0.232) et le chiffre **corrigé** vont **tous deux** dans la note (C06), avec l'écart publié.

9. Bonnes pratiques professionnelles

1. **Le coefficient va avec ses 4 chiffres** — valeur (-0.232), taille (n=400), contrôle (bandes de T1), limite (proxy, période) : les 4 ensemble, ou la phrase ne part pas.
2. **Le scale se teste avant d'être cité** — recalculer en **taux** (montant ÷ base) : si le coefficient retombe (0.552 → 0.001), c'était la grandeur, pas le comportement — et on publie le taux, pas le montant.
3. **Le confond se contrôle, pas se suppose** — identifier la tierce variable plausible (le niveau d'origine), la **mesurer** (proxy T1), contrôler **par bande** : c'est l'écart entre le brut et le contrôlé qui est l'information.
4. **Le groupe va avec son n** — écart + tailles dans la même phrase (panier par mode : 2 % sur 20 175 → 2 552) ; un groupe de moins de 100 observations se compare **en le disant** (« n=77 dans la bande haute »).
5. **La conclusion provisoire nomme ce qu'elle ne sait pas** — « le fichier ne départage pas les sens », « la part exacte de la tierce variable n'est pas chiffrable ici » : ce qui est **non mesuré** se dit, il ne se tait pas.

Conseil professionnel. Dans la note d'EDA (C06), chaque corrélation publiée porte son **encadré limite** : « association, pas causalité — contrôlé sur [X], non contrôlé sur [Y] ». Le lecteur professionnel **reconnait** le rapport à cet encadré : c'est la différence entre un stagiaire qui calcule et un analyste qui **garantit**.

10. Exercice guidé — « les 3 chiffres du jeu scolaire » (25 min, /10)

Consigne. Sur scolaire/ (audit C02 déjà fait) :

1. Recalculez la corrélation absentéisme × moyenne générale (formule du §5.1) et donnez son **ordre de grandeur** (faible / modérée / forte) et son **sens** (2 pts).
2. Contrôlez la tierce variable : les corrélations résiduelles **par bande de T1** (3 bandes) — et en une phrase, ce que le contrôle **révèle** (3 pts).
3. Segmentez l'absentéisme (< 2 , 2-5, ≥ 6 jours) : les 3 tailles, les 3 moyennes, et la **lecture** du gradient (3 pts).
4. Donnez le taux d'absence par classe et la classe la plus absente — avec les n des 4 classes (2 pts).

11. Exercices autonomes

E1 — Les 3 fausses causes, chacune identifiée (25 min). Sur le socle : (1) **variable tierce** — écrivez le test de contrôle (bandes de T1) et sa lecture ($-0.206 \rightarrow$ les 3 bandes) ; (2) **régression vers la moyenne** — les 2 groupes d'extrêmes du T1 (moyennes 7.01 et 14.93) et ce qu'ils font au T2 (7.21 et 14.61) : pourquoi ce mouvement **n'est pas** une cause ; (3) **sens inversé** — pourquoi le fichier ne départage **pas** « absence $\rightarrow$ note » de « note $\rightarrow$ absence », et la phrase honnête qui va dans le rapport.

E2 — Le petit groupe qui ment (20 min). Sur `vente.csv` : (1) le panier moyen par mode de paiement avec les **n** — l'écart max en % ; (2) montrez que l'écart est **du bruit de taille** (le rapport de n entre le plus grand et le plus petit groupe) ; (3) écrivez la phrase du rapport **complète** (écart + tailles + conclusion).

12. Correction détaillée

Exercice guidé.

1. **-0.232**, $n=400$: association **négative, modérée** ($|0.2|$ « modérée faible » à « modérée ») — les élèves plus absents ont des moyennes plus basses, mais une grande part de la variance reste hors absentéisme.
2. **Le contrôle** : brut $\text{abs_T2} \times \text{note_T2} = -0.206$; bandes de T1 : [8,10) **-0.145** ($n=107$), [10,12) **-0.085** ($n=139$), [12,14) **+0.163** ($n=77$) — le contrôle **révèle** que le lien brut est **partiellement** le niveau d'origine (la tierce variable pousse l'absence et la note) : à niveau égal, le lien direct est faible et instable (même du signe inversé en haut de gamme).
3. **Les segments** : $n = 123 / 208 / 69$, moyennes **11.13 / 10.51 / 9.76** (médianes 11.21 / 10.40 / 9.51) — **gradient régulier** (-0.62 puis -0.75) : la signature d'un lien direct, pas d'un accident ; les 69 élèves du segment haut font 1.37 point de moins que les 123 du segment bas.
4. **Par classe** : 5e 3.35 j ($n=110$), 4e **4.26 j** ($n=105$), 3e 2.63 j ($n=100$), 2nde 2.35 j ($n=85$) — la **4e** est la plus absente, à effectif proche des autres (rapport 105/85 : des groupes comparables).

E1. (1) **Tierce variable** : le test est le contrôle par bande de T1 (§5.2) : -0.206 au brut, $-0.145 / -0.085 / +0.163$ contrôlé — l'écart est la part du **niveau d'origine** ; (2) **Régression vers la moyenne** : $T1 < 8$ (7.01) $\rightarrow$ T2 7.21 ($+0.20$) et $T1 > 14$ (14.93) $\rightarrow$ T2 14.61 (-0.32) : les deux extrêmes **reviennent vers le milieu** (10.57, la moyenne globale) **sans cause identifiée** — c'est la variance d'une note (anormalement basse a des chances d'être moins basse au trimestre suivant), pas « l'aide a marché » ; (3) **Sens inversé** : le fichier contient les deux variables **simultanément**, pas l'une avant l'autre (pas d'intervention, pas de variable « motivation ») — il ne peut **pas** départager « absence $\rightarrow$ note » de « note $\rightarrow$ absence » ; la phrase honnête : « association bidirectionnellement compatible ; le fichier ne départage pas les sens, une étude longitudinale (ou une intervention) le pourrait ».

E2. (1) Paniers moyens : 156 150 (mode 4, $n=2\,593$) à 159 959 (mode 5, $n=2\,552$) : l'écart max est de **2.4 %** entre les 5 modes (médianes : 116 052 à 121 794, soit 5 %) ; (2) **Bruit de taille** : le rapport de n est de **8** entre le plus grand groupe (20 175, mode 1) et le plus petit (2 552, mode 5) — la variance d'une moyenne est proportionnelle à $1/n$: la moyenne du groupe 2 552 est environ la **racine carrée de 8** (≈ 2.8) fois plus bruyante que celle du groupe 20 175 : 2.4 % est **dans** le bruit de ce rapport de tailles ; (3) **La phrase** : « Les 5 modes de paiement ne différencient pas le panier moyen : écart max 2.4 % (156 150 à 159 959 FCFA) pour des groupes de 2 552 à 20 175 ventes (rapport 8) — l'écart est du bruit de taille, pas une préférence de paiement » (les moyennes et médianes vont ensemble dans la phrase).

13. Mini-projet M09.P5 — « L'hypothèse testée, sa limite nommée » (1 h)

Énoncé. La section relations du rapport P2 (projet M09.P) doit contenir **une** hypothèse du gabarit C02 §5.5, **testée** sur le socle, avec : le cadre falsifiable (mécanisme, signature chiffrée, contre-exemple), la sortie du test (exécutée), et la **conclusion provisoire** avec ses limites nommées (proxy, période, variables absentes, sens non départagés). Auto-évaluation : la conclusion dit-elle ce qu'elle **ne sait pas** ? L'encadré « corrélation, pas causalité » est-il présent ? (Le projet M09.P et l'évaluation rejouent l'exercice sur le fichier inconnu — c'est le même gabarit.)

14. Boîte à outils du chapitre

Boîte à outils. La corrélation : `s1.corr(s2)` (Pearson) + ses 4 chiffres (valeur, n, contrôle, limite). Le scale : recalculer en **taux** (montant / base). Le confond : `pivot_table` / bandes (`np.where`, `cut`) + corrélation **par bande**. Le segment : `pd.cut` / `np.where` avec des bornes **métier**. Le groupe : `groupby.agg(["count", "mean", "median"])` — le n **toujours** avec l'écart. Le conteneur : le `DataFrame` — tableau de données — dataframe de pandas, qui porte toutes ces commandes.

Besoin	Commande	Chiffre du socle
corrélation	<code>s1.corr(s2)</code>	-0.232 (n=400)
contrôle confond	corr par bande de T1	-0.145 / -0.085 / +0.163
test de scale	<code>corr((a/b), c)</code>	0.552 → 0.001
segments	<code>pd.cut(taux, [-0.1, 1.9, 5.9, 99])</code>	123 / 208 / 69 → 11.13 / 10.51 / 9.76
groupes	<code>groupby.agg(["count", "mean", "median"])</code>	5 modes, 20 175 → 2 552, 2.4 %

15. Résumé du chapitre

Les étapes 7-8 du protocole (« les variables bougent-elles ensemble ? » / « que puis-je affirmer ? ») sont exécutées avec le **jeu scolaire qui fait son entrée** (402 élèves déclarés, 400 distincts, 6 000 notes, 1 687 absences/retards — 1 278 absences, 409 retards — et 4 défauts : 6 notes hors bornes, 2 doublons d'élèves dont un **avec contradiction** F/M, 4 absences un jour férié, 3 élèves **absents-notés** au T2 (13, 15, 15 absences, 5 notes chacun)) et avec le fil rouge. La corrélation absentéisme × résultats est **-0.232** (n=400) : association **négative, modérée** — et le contrôle de la **tierce variable** (le niveau d'origine, proxy T1) la **décompose** : -0.206 au brut, -0.145 / -0.085 / +0.163 par bande de T1 — une part du lien brut est la variable tierce, pas l'absence ; la **régression vers la moyenne** est mesurée (T1 < 8 : 7.01 → 7.21 ; T1 > 14 : 14.93 → 14.61, les extrêmes reviennent vers le milieu sans cause) ; le **sens** n'est pas départagé par le fichier (pas d'intervention, pas de variable motivation) — la phrase honnête le dit. Les **segments** d'absentéisme (bornes métier < 2, 2-5, ≥ 6 jours) font un **gradient régulier** : 123 / 208 / 69 élèves, moyennes 11.13 → 10.51 → 9.76 — des élèves, pas des valeurs. La **comparaison de groupes** passe par les tailles : les 5 modes de paiement ne différencient **pas** le panier (2.4 % d'écart, groupes de 2 552 à 20 175, rapport 8 : du bruit), alors que la 4e est la classe la plus absente (4.26 j vs 2.35 en 2nde, n=105/85 : un écart réel) ; et sur le fil rouge, le **scale** a imité un comportement (remises × montants 0.552, taux × montants **0.001**) — chaque association a passé son test de scale avant de devenir une phrase.

16. À retenir

À retenir. La corde se **tend** (`corr()`), le tour se **vérifie** (tierce variable, régression, sens), et la phrase part avec **ses 4 chiffres** (valeur, n, contrôle, limite) — « associés, jamais causent » est la règle d'or, et le scale est le tour le plus discret : $0.552 \rightarrow 0.001$ en changeant de variable.

1. **corr()** mesure le lien linéaire — pas la causalité, pas le lien en U, pas le comportement si l'une des variables est un scale.
2. **Les 3 fausses causes se mesurent** : tierce variable (contrôle par bande : $-0.206 \rightarrow -0.145 / -0.085 / +0.163$), régression ($7.01 \rightarrow 7.21, 14.93 \rightarrow 14.61$), sens (le fichier ne départage pas).
3. **Le coefficient va avec ses 4 chiffres** : valeur, n, contrôle, limite — les 4 ensemble, ou la phrase ne part pas.
4. **Segmenter = trancher avant de regarder** : les bornes sont métier ($< 2, 2-5, \geq 6$ j), le gradient ($11.13 \rightarrow 10.51 \rightarrow 9.76$) est la lecture, les 69 élèves du haut sont des **élèves**.
5. **Le groupe va avec son n** : 2.4 % d'écart sur 2 552 ventes est du bruit ; la 4e (4.26 j, $n=105$) est un écart réel à n proches.
6. **Le test de scale est obligatoire** : recalculer en taux — $0.552 \rightarrow 0.001$ sur les remises du fil rouge ; 0.015 de taux max, 45.5 % de ventes sans remise.
7. **La conclusion provisoire nomme ce qu'elle ne sait pas** : proxy, période, sens non départagés, absents-notés non exclus — l'encadré limite est la signature du rapport.

17. Évaluation formative (auto-correction, 8 min)

1. **La corrélation absentéisme × moyenne est -0.232 ($n=400$). Que dit ce chiffre, et que ne dit-il pas ?** → une association **négative, modérée** (les plus absents ont en moyenne des notes plus basses) ; il ne dit **pas** que l'absence **cause** la baisse (les 3 fausses causes partagent l'hypothèse de cause), pas la part exacte de la tierce variable, pas le sens. (2 pts)
2. **Pourquoi le contrôle par bande de T1 ($-0.145 / -0.085 / +0.163$) change-t-il la conclusion ?** → le contrôle **isole** la tierce variable (le niveau d'origine pousse l'absence et la note) : à niveau égal, le lien direct est **faible et instable** (même du signe inversé en bande haute) — l'association brute est **partiellement** la tierce variable, pas l'absence. (2 pts)
3. **Les élèves T1 < 8 (7.01) font 7.21 au T2. Pourquoi « l'aide a marché » est une lecture en trop ?** → c'est la **régression vers la moyenne** : les extrêmes retournent vers le milieu (10.57) **sans cause** — une note anormalement basse a des chances d'être moins basse au trimestre suivant ; le contrôle est la **moyenne de tous** les élèves sur la même transition, pas soi-même. (2 pts)
4. **Pourquoi le fichier ne départage pas « absence → note » de « note → absence » ?** → les deux variables sont **simultanées** (pas d'intervention, pas de variable motivation, pas d'ordre causal mesurable) ; la phrase honnête : « association bidirectionnellement compatible, le fichier ne départage pas les sens ». (2 pts)
5. **Les 5 modes de paiement font 2.4 % d'écart de panier (156 150 à 159 959). Est-ce une préférence de paiement ?** → non : groupes de 2 552 à 20 175 ventes (rapport 8) — l'écart est **dans** le bruit de taille (variance de la moyenne $\propto 1/\sqrt{n}$) ; la phrase complète : écart + tailles + « pas de différenciation ». (2 pts)
6. **Les segments d'absentéisme ($< 2, 2-5, \geq 6$ j) donnent 11.13 / 10.51 / 9.76. Pourquoi ce gradient est-il « la signature d'un lien direct » ?** → il est **régulier** (-0.62 puis -0.75 , pas de saut accidentel) et la **médiane suit la moyenne** (pas de queue qui tire) : un lien direct se lit en gradient monotone ; un accident de groupe ferait sauter une tranche sans l'autre. (2 pts)

7. **Remises × montants : 0.552. Taux × montants : 0.001. Quelle est la leçon, et quelle variable publie-t-on ?** → le **scale** (la remise en FCFA) grandit avec le montant **sans comportement** : c'est la grandeur, pas le choix ; on publie le **taux** (max 0.015, 45.5 % des ventes à taux nul) — et la conclusion « pas de lien entre taux de remise et montant ». (2 pts)
8. **Les 3 élèves du défaut S4 (13, 15, 15 absences au T2, 5 notes chacun). Pourquoi sont-ils un problème pour la corrélation ?** → ce sont des **absents-notés** : maximum d'absences **et** notes présentes — ils tirent la corrélation **par construction** (le défaut S4) ; le chiffre brut (−0.232) et le chiffre **hors les 3** vont tous deux dans la note (C06), avec l'écart publié. (2 pts)

Module M09.C06 — Visualisations d'exploration et conclusions provisoires : la note d'EDA

Outils : pandas 2.2.3, numpy 2.3.5, matplotlib 3.10.9, seaborn 0.13.2 (exécutés). **Durée indicative :** 5 h. **Niveau :** N3. **Prérequis :** M09.C01-C05 (les 5 étapes précédentes), M08.C08 (les 4 graphiques de base).

L'idée du chapitre. L'étape 9 pose la question : « **quel graphique répond à la question ?** » et l'étape 10 : « **que dis-je, et quelles sont mes limites ?** » C06 choisit le graphique **par question** (courbe = évolution, barres = comparaison, boxplot = dispersion par groupe, histogramme = forme, nuage = association), passe les **5 erreurs d'exploration** en revue (l'axe qui ne part pas à zéro **sans le dire**, le graphique sans question, la couleur qui porte une info non mentionnée, l'échelle logarithmique surprise, le nuage sans taille d'échantillon), et produit le livrable de fin de module : la **note d'EDA** — 1 page par jeu, chaque affirmation porte sa **source** (sortie exécutée) et sa **limite**. Le fil rouge y revient **complet** : la note de la quincaillerie, avec les 3 jeux chacun en un paragraphe. Et la transition vers M10 : « Vos résultats sont justes. S'ils sont illisibles, ils ne serviront à rien. »

Matériel de l'atelier — Python 3.13 · pandas 2.2.3 · numpy 2.3.5 · matplotlib 3.10.9 · seaborn 0.13.2. Tous les graphiques ont été **exécutés** dans l'atelier le 20/09/2026 sur `03_exercices/dossier_M09/` (graine 45, déterministe, ATTENDU.json figé) ; les codes publiés sont **ceux de l'atelier**, et chaque sortie citée est une sortie exécutée.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Choisir le graphique par question** : évolution → courbe, comparaison → barres (avec les n), dispersion par groupe → boxplot, forme → histogramme, association → nuage — la table question → graphique du module.
2. **Repérer les 5 erreurs d'exploration** et les **écrire en contrôle** : axe sans zéro **déclaré**, graphique sans question, couleur informante non mentionnée, échelle logarithmique **annoncée**, nuage avec **n**.
3. **Produire les 4 graphiques du projet** en `matplotlib` (courbe CA + bâche, barres paniers par mode, nuage montant × taux, boxplot notes par segment) — chaque graphique légendé : question + source + ce qu'on doit retenir.
4. **Utiliser seaborn pour le confort** (boxplot, heatmap des tableaux croisés) — et savoir que le **même** boxplot se fait en `matplotlib` pur (dépendance minimale) ou en `seaborn` (l'option confort, documentée).
5. **Écrire la note d'EDA** : 1 page par jeu, affirmation + **source** (sortie) + **limite** — et **en lire une écrite par quelqu'un d'autre** (rétro-lecture : repérer les 5 erreurs dans une note d'autrui).

2. Pourquoi cette notion est importante

Le rapport EDA se **lit** avant de se **vérifier** : un lecteur pressé ne relancera pas vos commandes, il regardera vos graphiques et vos phrases. Et c'est précisément là que l'exploration ment le plus

souvent — pas dans les chiffres (vérifiés par le protocole, C01-C05), mais dans **leur présentation** : un axe qui dramatise, une couleur qui raconte, un graphique qui ne répond à rien. Trois faits du socle fixent l'enjeu :

- la courbe du CA mensuel sur **axe 310-350 M** (pas à zéro) donne l'impression d'une forte variation ; sur **axe à zéro**, elle est **plate** — deux graphiques du **même** chiffre, deux lectures opposées ;
- le nuage montant $\times$ taux de remise **sans n** est illisible : 50 008 points à transparence 0.25 **ou** un nuage clair de 200 points ressemblent à la même chose — la taille d'échantillon change la lecture, elle doit être **dans** le titre ;
- la note d'EDA du fil rouge tient en **4 affirmations**, chacune avec sa source (sortie) et sa limite — c'est le livrable **P4** du projet, et l'épreuve de palier P2 le note sur la **limite** (ce que l'affirmation ne sait pas), pas sur le chiffre.

3. Explication simple — la conversation avec la machine

Le graphique est la **traduction** d'une sortie vers un lecteur. Comme toute traduction, elle peut trahir :

1. **La question d'abord** — « ce graphique répond à **quelle** question ? » : une courbe sans question est une décoration, un boxplot sans question est un amas de moustaches. La question est **dans le titre**, pas dans la légende.
2. **L'axe est une décision** — partir de zéro ou non, c'est un **choix d'édition** : sur des données **relatives** (part, taux), zéro est honnête ; sur des **niveaux** (CA mensuel 300-350 M), zéro écrase la variation — l'**axe non zéro** est légitime **s'il est déclaré** (« axe de 310 à 350 M »), jamais s'il est silencieux.
3. **La couleur raconte** — une couleur par groupe **sans légende** est une info cachée ; le **n** manquant est une échantillon fantôme ; l'échelle log **surprise** est une distribution qu'on a cachée. Les 5 erreurs d'exploration, c'est la liste de contrôle de la **présentation**, pas du calcul.

La note d'EDA est la **dernière étape du protocole** : 1 page, chaque affirmation **sourcée** (la sortie qui la prouve) et **limitée** (ce qu'elle ne dit pas). Le module s'arrête là — et c'est le livrable : pas 40 graphiques jolis, 4 graphiques qui **répondent** et 1 page qui **garantit**.

4. Vocabulaire essentiel

Terme	Définition
graphique — chart — la traduction visuelle d'une sortie vers un lecteur ; se choisit par question (courbe/barres/boxplot/histogramme/nuage) et se légende (question + source + n).	
légende	le bloc texte du graphique : question posée, source (sortie exécutée), taille d'échantillon (n), et l'axe quand il ne part pas de zéro.
biais de sélection — biais d'échantillonnage — l'écart entre la population étudiée (le fichier) et la population visée (le métier) : les 1 200 clients du fil rouge ne sont pas tous les clients du groupe.	
note d'EDA	le livrable de fin d'exploration : 1 page par jeu, chaque affirmation = source (sortie) + limite (ce qu'elle ne dit pas).

5. Cours approfondi

5.1 Choisir le graphique par question — la table du module

Question	Graphique	Pourquoi	Chiffre du socle
« Ça monte, ça descend, ça cycle ? »	courbe (+ n) (bâche)	(+ l'axe horizontal est le temps	CA mensuel 307-348 M, mg3 320-337 M
« Les groupes se comparent-ils ? »	barres (+ n)	la longueur est la valeur, le n est la fiabilité	panier par mode, n 2 552 à 20 175
« La dispersion change-t-elle par groupe ? »	boxplot	médiane, iqr, queues — la forme du groupe	notes par segment d'absentéisme
« Quelle est la forme de la variable ? »	histogramme	la répartition des valeurs , pas des temps	montants 762.63 à 594 363 FCFA
« Les variables bougent -elles ensemble ? »	nuage (+ n)	chaque point est une observation	montant × taux de remise (n=50 008)

Le préambule de M10 tient dans cette table : on ne choisit **pas** un graphique « qui est joli », on choisit **celui qui répond** — et M10 (visualisation) fera le chemin inverse (le graphique de **communication**, avec ses règles d'édition). En **exploration** (M09), le graphique est au service de la **question**, pas du lecteur final.

Définition. légende — le bloc texte du graphique : la **question** posée (dans le titre), la **source** (la sortie exécutée), la **taille d'échantillon** (n) et **l'axe** quand il ne part pas de zéro. Un graphique sans légende est une image ; un graphique avec légende est une **réponse** — et c'est la légende que le lecteur **vérifie** (pas le code, qu'il ne relance pas).

Le graphique se choisit PAR QUESTION — la table du module (tout est exécuté)

Pas un graphique « joli », celui qui répond. La légende : question + source + n + axe déclaré s'il n'est pas à zéro.

QUESTION	GLYPHE	RÉPONSE (sortie exécutée)
Ca monte, ça descend, ça cycle ? courbe (+ bâche)		CA mensuel 307-348 M, mg3 320-337 M plat, +1.6 % à mois égal (déclaré)
Les groupes se comparent-ils ? barres (+ les n)		panier par mode : écart 2.4 % n de 2 552 à 20 175 : c'est le bruit
La dispersion change par groupe ? boxplot		notes par segment : médianes 11.21 / 10.40 / 9.51 : un gradient
Quelle est la forme de la variable ? histogramme		montants 762.63 à 594 363 (ratio ~780) plateau x6 au maximum (plafond)
Bougent-elles ensemble ? nuage (+ le n)		montant x taux : corr 0.001 (n=50 008) pas de lien (remise = 45.5 % des ventes)

Les 5 contrôles de présentation (la rétro-lecture, dans l'ordre) :

(1) question dans le TITRE · (2) axe déclaré si non zéro · (3) couleur avec légende · (4) log annoncée · (5) n au titre

Le livrable de fin : la note d'EDA (1 page par jeu) — affirmation + source + limite

« Le CA est plat sur 24 mois (mg3 320-337 M). Source : G1 exécutée. Limite : décembre 2026 partiel (28 j), exclu. »
La limite se lit à voix haute : si « et sa limite est que ... » reste vide, on recule l'affirmation.

Transition M10 : « Vos résultats sont justes. S'ils sont illisibles, ils ne serviront à rien. »

5.2 Les 5 erreurs d'exploration — et leur contrôle

1. **L'axe qui ne part pas à zéro sans le dire.** La courbe du CA mensuel sur axe 310-350 M (exécuté) « dramatise » la variation (l'œil lit un delta de 40 M comme une onde) ; sur axe à zéro, la même courbe est plate. **Le contrôle** : l'axe non zéro est **déclaré** dans la légende (« axe 310-350 M ») — jamais silencieux.
2. **Le graphique sans question.** Un boxplot « des notes » sans question ne répond à rien. **Le contrôle** : la question est **dans le titre** (« Notes par segment d'absentéisme : la dispersion change-t-elle ? »), pas dans un sous-titre.
3. **La couleur qui porte une info non mentionnée.** Colorer les points par `est_retour` sans le dire dans la légende, c'est **raconter** une info cachée. **Le contrôle** : chaque couleur a sa **légende** — pas de couleur orpheline.
4. **L'échelle logarithmique surprise.** Les montants vont de 762.63 à 594 363 FCFA (ratio ~780) : en échelle log, le **plateau** des 6 ventes à 594 363 (C04) se « lisse » et disparaît visuellement. **Le contrôle** : l'échelle log est **annoncée** (« axe log ») — elle est un **choix d'édition**, pas un défaut.
5. **Le nuage sans taille d'échantillon.** Le nuage montant $\times$ taux (exécuté, $n=50\,008$, transparence 0.25) : sans le n dans le titre, le lecteur ne sait pas s'il voit 50 008 points ou 200. **Le contrôle** : le **n est dans le titre** (« $n = 50\,008$ ventes »), et la transparence est **mentionnée** si elle masque des points.

Attention. L'erreur la plus coûteuse pour le lecteur pressé n'est pas le chiffre faux (le protocole l'a vérifié, C01-C05) : c'est la **présentation** qui ment — un axe qui **dramatise**, un n qui **masque**, une couleur qui **raconte**. Le chiffre est garanti ; la **traduction** est le chapitre, et elle se **contrôle** (les 5 contrôles, §5.2), elle ne se **suppose** pas.

Définition. graphique — chart — la traduction visuelle d'une sortie vers un lecteur. Il se choisit **par question** (table §5.1), il se **légende** (question + source + n + l'axe s'il n'est pas à zéro), et il est **exécuté** (le code est re-jouable, la sortie est citée). Un graphique sans question est une décoration ; un graphique sans n est un échantillon fantôme.

5.3 matplotlib — les 4 graphiques du projet

Les 4 graphiques du livrable P3, **exécutés** (les codes sont ceux de l'atelier) :

```

import matplotlib as mpl; mpl.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
s = "03_exercices/dossier_M09"
v = pd.read_csv(s + "/quincaillerie/vente.csv", parse_dates=["date_vente"])
v["ct"] = (v["montant_ttc"] * 100).round().astype("int64")

# G1 – courbe : CA mensuel (sans retours) + moyenne glissante 3
vt = v[~v["est_retour"]].groupby(v["date_vente"].dt.to_period("M"))["ct"].sum() // 100
t = vt.index.to_timestamp()
fig, ax = plt.subplots(figsize=(8, 4), dpi=100)
ax.plot(t, vt.values / 1e6, "o-", ms=3, label="CA mensuel (M FCFA, sans retours)")
ax.plot(t, vt.rolling(3).mean().values / 1e6, "-", lw=2, label="moyenne glissante 3 mois")
ax.set_title("CA mensuel – ça monte, ça descend, ça cycle ? (n = 24 mois, axe 310-350 M)")
ax.legend(); ax.set_ylabel("M FCFA")
fig.tight_layout(); fig.savefig("g1_ca_mensuel.png"); plt.close(fig)

# G2 – barres : panier moyen par mode de paiement, avec les n
pm = v.groupby("id_mode")["montant_ttc"].agg(["count", "mean"]).reset_index()
fig, ax = plt.subplots(figsize=(7, 4), dpi=100)
ax.bar(pm["id_mode"].astype(str), pm["mean"] / 1e3)
for i, r in pm.iterrows():
    ax.text(i, r["mean"] / 1e3, f"n={int(r['count'])}", ha="center", va="bottom", fontsize=8)
ax.set_title("Panier moyen par mode de paiement – les tailles sont dans le graphique")
ax.set_ylabel("k FCFA"); ax.set_xlabel("id_mode")
fig.tight_layout(); fig.savefig("g2_panier_mode.png"); plt.close(fig)

# G4 – nuage : montant x taux de remise, avec le n
v["rr"] = v["montant_remise"] / v["montant_ttc"]
fig, ax = plt.subplots(figsize=(6, 4), dpi=100)
ax.scatter(v["montant_ttc"] / 1e3, v["rr"], s=4, alpha=0.25)
ax.set_title(f"Montant x taux de remise (n = {len(v)} ventes ; corr = 0.001 ; transparence 0.25)")
ax.set_xlabel("montant TTC (k FCFA)"); ax.set_ylabel("taux de remise")
fig.tight_layout(); fig.savefig("g4_montant_remise.png"); plt.close(fig)

```

Ce que chacun répond (la légende, pas la décoration) :

- **G1** — « Ça monte, ça descend, ça cycle ? » : la bâche (orange) est **plate** (320-337 M), la série (bleue) oscille (307-348 M) — **ni cycle, ni dérive forte** (C03 : +1.6 % à mois égal) ; le titre **déclare l'axe 310-350 M** (erreur n°1 contrôlée).
- **G2** — « Les paniers diffèrent par mode ? » : les barres sont **quasi identiques** (156-160 k), et les **n sont dans le graphique** (2 552 à 20 175) — l'écart de 2.4 % est **du bruit de taille** (C05), la légende le dit.
- **G4** — « Remises et montants vont-ils ensemble ? » : le nuage est une **bande horizontale** (taux 0 à 0.015, corr 0.001) — **pas de lien** ; le **n est dans le titre**, la transparence est **mentionnée** (erreurs n°5 et n°3 contrôlées).

5.4 seaborn — le confort, documenté

seaborn (option confort) reprend le **même** boxplot et fait la **heatmap** des tableaux croisés, **exécutés** :

```

import seaborn as sns
import pandas as pd
s = "03_exercices/dossier_M09"
no = pd.read_csv(s + "/scolaire/note.csv")
ab = pd.read_csv(s + "/scolaire/absence.csv")
el = pd.read_csv(s + "/scolaire/eleve.csv")
no_p = no[(no["note"] >= 0) & (no["note"] <= 20)].copy()
abs_par = ab.groupby("id_eleve")["duree_jours"].sum()
taux = abs_par.reindex(sorted(no_p["id_eleve"].unique()), fill_value=0)
no_p["seg"] = pd.cut(taux.reindex(no_p["id_eleve"].values, fill_value=0).values,
                    [-0.1, 1.9, 5.9, 99], labels=["<2 j", "2-5 j", ">=6 j"])

# G3 – boxplot : notes par segment d'absentéisme
fig, ax = plt.subplots(figsize=(6, 4), dpi=100)
sns.boxplot(data=no_p, x="seg", y="note",
            order=["<2 j", "2-5 j", ">=6 j"], ax=ax)
ax.set_title("Notes par segment d'absentéisme : la dispersion change-t-elle ? (n = 6000 notes)")
fig.tight_layout(); fig.savefig("g3_notes_segment.png"); plt.close(fig)

# G5 – heatmap : le tableau croisé matière x trimestre (C05)
h = no_p.pivot_table(index="matiere", columns="trimestre", values="note", aggfunc="mean")
fig, ax = plt.subplots(figsize=(6, 4), dpi=100)
sns.heatmap(h, annot=True, fmt=".2f", cmap="viridis", ax=ax)
ax.set_title("Moyennes matière x trimestre (heatmap seaborn) – n = 6000 notes")
fig.tight_layout(); fig.savefig("g5_heatmap.png"); plt.close(fig)

```

Le boxplot (G3) répond « la dispersion change-t-elle par segment ? » : les 3 boîtes montrent le **gradient** (médianes 11.21 / 10.40 / 9.51) **et** les queues — la médiane descend de 1.7 point du segment bas au segment haut. **La heatmap** (G5) est le **tableau croisé** (C05 §5.5) rendu lisible : les cases stables (± 0.1) se voient, le **classement des matières** (histoire_géographie en tête) se lit sur les 5 lignes. Le **même** boxplot se fait en matplotlib pur (`ax.boxplot`) — matplotlib est la **dépendance minimale** (le projet l'utilise), seaborn est l'**option confort**, et les deux produisent le **même** graphique sur les **mêmes** données.

Attention. Un graphique produit en seaborn **sans** son équivalent matplotlib est une **dépendance non documentée** : le lecteur (et l'évaluateur) doit pouvoir **re-jouer** le graphique avec la dépendance minimale — c'est la raison pour laquelle le projet utilise matplotlib et seaborn est **documenté**, pas **implicite**.

Définition. biais de sélection — biais d'échantillonnage — l'écart entre la population **étudiée** (le fichier) et la population **visée** (le métier) : les 1 200 clients du fil rouge ne sont pas **tous** les clients du groupe, les 400 élèves ne sont pas **tous** les élèves de l'établissement. C'est la **limite** la plus fréquente de la note d'EDA : on ne généralise qu'à la **population du fichier**, pas au-delà — et on le **dit**.

5.5 La note d'EDA — 1 page, affirmation + source + limite

La note d'EDA (l'étape 10, le livrable **P4**) est **1 page par jeu**. Chaque affirmation porte **3 choses** : la **phrase** (ce qu'on dit), la **source** (la sortie exécutée qui la prouve), et la **limite** (ce qu'elle **ne** dit pas). Le **contrôle** de la note : relire chaque affirmation et se demander « qu'est-ce que cette phrase **ne** peut **pas** conclure ? » — c'est la **limite**. La note de la quincaillerie (fil rouge) est au §6 ; voici sa **structure** (4 affirmations) :

1. **Tendance** : « Le CA (sans retours) est **plat** sur 24 mois » — source : G1 + mg3 (320-337 M) — limite : « décembre 2026 partiel (28 j), exclu ; 5 magasins agrégés, pas par magasin ».
2. **Données** : « 8 doublons métier exclus (50 008 → 50 000 ventes) » — source : duplicated (C01/C02) — limite : « les 15 deadlines inversées et les 21 406 dates naïves sont **déclarées**, non corrigées (source à confirmer) ».

3. **Clients** : « Le top 10 % des clients pèse 13.8 % du CA (clientèle plate) » — source : groupby CA — limite : « le fichier ne porte pas le **segment** client (particulier/entreprise) — lecture en CA seul ».
4. **Remises** : « Pas de lien taux de remise × montant (corr 0.001) » — source : G4 — limite : « 45.5 % des ventes sans remise ; conclusion en **taux**, pas sur la politique de remise ».

Définition. note d'EDA — le livrable de **fin** d'exploration : 1 page par jeu, où chaque affirmation porte sa **source** (la sortie exécutée, **re-jouable**) et sa **limite** (ce qu'elle **ne** dit pas, **remplie**). Ce n'est ni le rapport complet (10 étapes, P2), ni un résumé : c'est la **garantie** — le lecteur y vérifie les **limites**, pas les chiffres (déjà validés par le protocole).

À retenir. La note d'EDA **garantit** : chaque affirmation est **sourcée** (la sortie qui la prouve, re-jouable) et **limitée** (ce qu'elle ne dit pas, écrit). Le lecteur ne relance pas vos commandes — il lit vos **limites**. C'est **elles** qui font la différence entre un rapport de stagiaire et un rapport **d'analyste**.

5.6 Lire une note d'autrui — la rétro-lecture

L'exercice de **rétro-lecture** (un collègue a écrit la note, vous la lisez) est la meilleure école de la **présentation** : on repère les 5 erreurs **sans** relancer le code. Le contrôle de rétro-lecture tient en 5 questions, **dans l'ordre** :

1. **Quelle question** ce graphique répond-il ? (sinon : erreur n°2)
2. L'**axe** est-il à zéro, ou **déclaré** s'il ne l'est pas ? (sinon : erreur n°1)
3. Chaque **couleur** a-t-elle sa **légende** ? (sinon : erreur n°3)
4. L'**échelle** (log/lin) est-elle **annoncée** ? (sinon : erreur n°4)
5. Le **n** est-il **dans** le titre, et la transparence mentionnée ? (sinon : erreur n°5)

Une note qui passe les 5 questions est **présentable** ; une note qui en échoue une est **rejetée en présentation** (pas en calcul — les chiffres ont été vérifiés par le protocole, c'est la **traduction** qui est en défaut).

6. Exemple concret — la note d'EDA de la quincaillerie (fil rouge, complète)

NOTE D'EDA — Quincaillerie (fil rouge M07/M08) · période 2025-01-01 → 2026-12-28 · 5 magasins · 1 200 clients · 380 produits · source : `dossier_M09/quincaillerie` (empreinte `b9a8d973119342ec...`, 19 totaux = M07/M08).

1. **Le CA (sans retours) est plat sur 24 mois** : mensuel 307-348 M FCFA, moyenne glissante 3 mois entre 320 et 337 M — ni cycle de fin d'année (décembre 2025 : 315 M, sous la moyenne), ni dérive forte (+1.6 % à mois égal). Source : G1 (courbe + bâche, exécutée). Limite : décembre 2026 partiel (28 jours), exclu des conclusions ; agrégat des 5 magasins, pas par magasin.
2. **Les données portent 8 doublons métier, exclus** : 50 008 → 50 000 ventes distinctes, retours 208 → 200, écart CA retours 31 939 568 FCFA. Source : `duplicated` sur les 12 colonnes hors `id_vente` (C01/C02, exécuté). Limite : les 15 deadlines inversées et les 21 406 dates sous la condition naïve sont déclarées, non corrigées — correction soumise à la source.
3. **La clientèle est plate** : le top 10 % des clients (120) pèse 13.8 % du CA ; le top 1 (client 73, 11 561 025 FCFA) pèse 0.15 % à lui seul. Source : groupby CA par client (exécuté). Limite : le fichier ne porte pas le segment client (particulier / entreprise) — lecture en CA seul ; 1 200 clients, pas tous les clients du groupe (biais de sélection).

4. **Pas de lien entre le taux de remise et le montant** (corr 0.001, taux max 0.015) : le 0.552 obtenu en **FCFA** est un effet de **scale**, pas un comportement. Source : G4 (nuage, n=50 008, exécuté) + C05 §5.1. Limite : 45.5 % des ventes sans remise — la conclusion est en taux, pas sur la politique de remise.

Les 3 jeux, chacun en un paragraphe (la note rejoue les jeux 2 et 3 arrivés en C02/C05) : le **centre de santé** — « la fréquentation est **saisonnier** (étés 853 vs hivers 655 consultations/mois, pic 2025-07 : 882), avec une rupture **qui compte** (M01, 3 jours au pic de juillet) et 4 défauts de saisie corrigés par récurrence (545, 253, 278, 411) » (limite : 2 étés seulement, cause hors fichier) ; l'**établissement scolaire** — « absentéisme et résultats **associés** (−0.232), partiellement par une variable tierce (contrôle T1 : −0.145/−0.085/+0.163) ; la 4e est la classe la plus absente (4.26 j) » (limite : corrélation, pas causalité ; sens non départagé ; 3 élèves absents-notés non exclus du brut).

Dans les faits. La note d'EDA du fil rouge (celle-ci) tient en **4 affirmations** (une quinzaine de lignes de texte) et les 4 graphiques en **une quarantaine de lignes de code** : exécutés, les 5 graphiques de l'atelier (G1-G5) pèsent entre 20 et 59 Ko, et le **temps atelier** du livrable P3+P4 est d'environ **40 minutes** (les graphiques d'abord, la note ensuite, la rétro-lecture en dernier) : c'est le **livrable** du module, et il se **lit** en 3 minutes — c'est **ça** qu'on rend, pas 40 graphiques.

7. Démonstration pas à pas — 5 étapes sur les deux jeux

```
import matplotlib as mpl; mpl.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
s = "03_exercices/dossier_M09"
v = pd.read_csv(s + "/quincaillerie/vente.csv", parse_dates=["date_vente"])
no = pd.read_csv(s + "/scolaire/note.csv")
ab = pd.read_csv(s + "/scolaire/absence.csv")

# 1 – la table question -> graphique (choix)
#   évolution -> courbe (G1) | comparaison -> barres+n (G2) | dispersion -> boxplot (G3)
#   forme -> histogramme | association -> nuage+n (G4)

# 2 – G1 : courbe CA + bâche (axe non zéro DECLARE dans le titre)
v["ct"] = (v["montant_ttc"] * 100).round().astype("int64")
vt = v[~v["est_retour"]].groupby(v["date_vente"].dt.to_period("M"))["ct"].sum() // 100
fig, ax = plt.subplots(figsize=(8, 4), dpi=100)
ax.plot(vt.index.to_timestamp(), vt.values / 1e6, "o-", ms=3)
ax.plot(vt.index.to_timestamp(), vt.rolling(3).mean().values / 1e6, "-", lw=2)
ax.set_title("CA mensuel – ça monte, ça descend, ça cycle ? (n = 24 mois, axe 310-350 M)")
fig.tight_layout(); fig.savefig("g1.png"); plt.close(fig)

# 3 – G2 : barres paniers par mode, avec les n
pm = v.groupby("id_mode")["montant_ttc"].agg(["count", "mean"]).reset_index()
fig, ax = plt.subplots(figsize=(7, 4), dpi=100)
ax.bar(pm["id_mode"].astype(str), pm["mean"] / 1e3)
for i, r in pm.iterrows():
    ax.text(i, r["mean"] / 1e3, f"n={int(r['count'])}", ha="center", va="bottom", fontsize=8)
ax.set_title("Panier moyen par mode de paiement – les tailles sont dans le graphique")
fig.tight_layout(); fig.savefig("g2.png"); plt.close(fig)

# 4 – G3 : boxplot seaborn (notes par segment d'absentéisme)
no_p = no[(no["note"] >= 0) & (no["note"] <= 20)].copy()
taux = ab.groupby("id_eleve")["duree_jours"].sum()
no_p["seg"] = pd.cut(taux.reindex(no_p["id_eleve"].values, fill_value=0).values,
                    [-0.1, 1.9, 5.9, 99], labels=["<2 j", "2-5 j", ">=6 j"])
fig, ax = plt.subplots(figsize=(6, 4), dpi=100)
sns.boxplot(data=no_p, x="seg", y="note", order=["<2 j", "2-5 j", ">=6 j"], ax=ax)
ax.set_title("Notes par segment d'absentéisme (n = 6000 notes)")
fig.tight_layout(); fig.savefig("g3.png"); plt.close(fig)

# 5 – la note d'EDA : chaque affirmation = phrase + source + limite (le $6 est la sortie)
print("note d'EDA : 4 affirmations fil rouge + 3 jeux (voir $6)")
```

8. Erreurs fréquentes

1. **L'axe non zéro silencieux** — la courbe CA sur axe 310-350 M **sans le dire** : le lecteur lit une onde là où il y a une série plate. L'axe non zéro est **légitime s'il est déclaré** (« axe 310-350 M » dans le titre), jamais silencieux.
2. **Le graphique sans question** — un boxplot « des notes » : la question est **dans le titre** (« la dispersion change-t-elle par segment ? »), sinon le graphique est une décoration, pas une réponse.
3. **La couleur orpheline** — colorer par `est_retour` sans légende : la couleur **raconte** une info cachée ; chaque couleur a sa **légende**, pas de couleur orpheline.
4. **L'échelle log surprise** — le plateau des 6 ventes à 594 363 (C04) **disparaît** en échelle log non annoncée : l'échelle log est un **choix d'édition**, elle est **annoncée** (« axe log »), jamais silencieuse.
5. **Le nuage sans n** — 50 008 points **ou** 200 points ressemblent à la même bande : le **n est dans le titre**, et la transparence est **mentionnée** si elle masque des points.

9. Bonnes pratiques professionnelles

1. **La question est dans le titre** — jamais dans un sous-titre ni dans le pied de page : le titre du graphique **est** la question (« Ça monte, ça descend, ça cycle ? »), la légende porte la source et le n.
2. **L'axe se déclare s'il n'est pas à zéro** — « axe 310-350 M » dans le titre ; sur des données **relatives** (part, taux), zéro est l'axe **honnête**.
3. **Le n est dans le titre, la transparence est mentionnée** — « n = 50 008 ventes ; transparence 0.25 » : l'échantillon **fantôme** est l'erreur la plus coûteuse du lecteur pressé.
4. **matplotlib pour le projet, seaborn pour le confort** — le même boxplot en `ax.boxplot` (dépendance minimale) ou en `sns.boxplot` (confort) : on documente **les deux** produisant le **même** graphique sur les **mêmes** données.
5. **La note d'EDA garantit les limites** — chaque affirmation : phrase + source (re-jouable) + **limite** (ce qu'elle ne dit pas) : c'est la limite qui fait la différence entre un stagiaire et un **analyste** (l'épreuve P2 note **la limite**).

Conseil professionnel. La note d'EDA se **lit à voix haute** avant d'être rendue : chaque affirmation « ... et sa limite est que ... » — si la phrase « et sa limite est que » reste **vide** pour une affirmation, c'est que l'affirmation **déborde** de ce que les données disent, et elle est **reculée** (ou retirée). La limite n'est pas une décoration de fin de phrase, c'est le **contrôle** de l'affirmation.

10. Exercice guidé — « la rétro-lecture de la note d'autrui » (25 min, /10)

Consigne. Un collègue a rendu cette note d'EDA (le code est **correct**, c'est la **présentation** qui est en cause). Repérez les **5 défauts de présentation** (1 pt chacun), puis réécrivez le **titre** et la **légende** de chaque graphique en les corigeant (2 pts pour l'ensemble) :

Note d'EDA — Quincaillerie · 5 magasins

1. Graphique A (courbe du CA mensuel, axe 310-350 M) : « Le CA est en **forte baisse** depuis mars 2026. »
2. Graphique B (barres du panier par mode, sans n) : « Les modes de paiement **différencient** le panier. »
3. Graphique C (nuage montant × remise **en FCFA**, sans n) : « Les remises **causent** la baisse du CA. »
4. Graphique D (boxplot des notes, sans question) : « Les notes des élèves. »
5. Affirmation : « Les 1 200 clients du fichier **représentent tous les clients** du groupe. »

11. Exercices autonomes

E1 — La note d'EDA du centre de santé (30 min). Écrire la **note d'EDA complète** (1 page) du jeu 2 : 4 affirmations (saisonnalité, rupture qui compte, 4 défauts corrigés, fréquentation type), chacune avec **source** (sortie exécutée) et **limite**. Contrainte : la phrase « et sa limite est que ... » doit être **remplie** pour chacune — une limite vide = une affirmation qui déborde.

E2 — Les 5 erreurs, chacune produite et contrôlée (30 min). Sur `vente.csv` : produire **chacune** des 5 erreurs d'exploration (axe non zéro silencieux, graphique sans question, couleur orpheline, échelle log surprise, nuage sans n) **puis** son **contrôle** (la déclaration qui la corrige) — 5 graphiques × 2, chacun légendé avant / après.

12. Correction détaillée

Exercice guidé — les 5 défauts de présentation :

1. **Graphique A — axe non zéro silencieux** (+ une **tendance fausse** : mars 2026 fait 338 M, **au-dessus** de la moyenne ; il n'y a **pas** de baisse) : déclarer « axe 310-350 M » dans le titre, et corriger la tendance (« le CA est **stable** autour de 328 M, mars 2026 à 338 M est **au-dessus** de la moyenne »).
2. **Graphique B — barres sans n** : les paniers sont **quasi identiques** (156-160 k, écart 2.4 %) pour des groupes de **2 552 à 20 175** ventes (rapport 8) — mettre les **n dans le graphique** et corriger (« les modes **ne différencient pas** le panier : écart 2.4 % sur des groupes de rapport 8 = bruit de taille »).
3. **Graphique C — nuage sans n + scale** (remises en **FCFA**, corr 0.552 = effet de scale, pas de comportement) + **causalité** (« causent ») : mettre le **n dans le titre**, passer au **taux** (corr **0.001**), et corriger (« **pas de lien** entre le **taux** de remise et le montant ; association, pas causalité »).
4. **Graphique D — boxplot sans question** : la question est **dans le titre** (« Notes par segment d'absentéisme : la **dispersion** change-t-elle ? (n = 6000 notes) »).
5. **Affirmation — biais de sélection** : les 1 200 clients du fichier ne sont **pas** tous les clients du groupe ; corriger (« les 1 200 clients du **fichier** — pas une population complète ; **limite** de généralisation »).

Ensemble (2 pts) : les 5 titres/légendes corrigés portent chacun la **question** (titre), la **source** (sortie), le **n**, et l'**axe déclaré** s'il n'est pas à zéro.

E1. (structure attendue, les sorties sont celles de l'atelier) — 1. Saisonnalité : « la fréquentation est **saisonnier** : étés 853 vs hivers 655 consultations/mois, pic 2025-07 (882), dimanche 405 vs lundi 3 710 » — source : groupby mois/jour de semaine (exécuté, C03) — limite : 2 étés seulement, la cause (pluies) est hors fichier. 2. **Rupture qui compte** : « **M01 rompu 3 jours (2025-07-14 → 16) au pic de juillet, M02 5 jours (2026-03-02 → 06), M04 2 jours** » — source : stock à zéro groupé (exécuté, C04) — limite : **c'est un fait (offre coupée), pas un défaut à corriger**. 3. 4 défauts corrigés : « **4 stocks négatifs corrigés par récurrence (545, 253, 278, 411), 23 motifs vides gardés en signalant, 7 doublons exclus, 1 date future exclue** » — source : C04 (exécuté) — limite : **la date future est exclue, sa correction (2025-01-05) est **soumise à la source**. 4. Fréquentation type : « pic du matin (09:00 : 2 319), creux du midi (12:00 : 927) » — source : groupby heure (exécuté, C03) — limite : 10 heures seulement, pas de nuit/week-end différencié.

E2. (les 5 paires erreur → contrôle, sur vente.csv) — 1. Axe non zéro : courbe CA axe 310-350 M **silencieux** → **contrôle** : « axe 310-350 M » dans le titre. 2. Sans question : boxplot « des montants » → **contrôle** : « Montant par quantité : la **forme** change-t-elle ? (n = 50 008) ». 3. Couleur orpheline : points colorés par est_retour sans légende → **contrôle** : légende « rouge = retour, bleu = vente (n retours = 208) ». 4. Échelle log surprise : histogramme des montants en log **non annoncée** (le plateau 594 363 disparaît) → **contrôle** : « axe **log** » dans le titre. 5. Nuage sans n : montant × taux **sans n** → **contrôle** : « n = 50 008 ventes ; transparence 0.25 » dans le titre.

13. Mini-projet M09.P6 — « P3 + P4 : les 4 graphiques et la note » (1 h)

Énoncé. Le **livrable final** du projet M09.P : (P3) les **4 graphiques** du jeu travaillé, **chacun** légendé (question dans le titre, source, n, axe déclaré s'il n'est pas à zéro) ; (P4) la **note d'EDA** (1 page), 4 affirmations chacune avec **source** (re-jouable) et **limite** (remplie). Auto-évaluation : lire la note **à voix haute** — chaque « et sa limite est que ... » est-elle **remplie** ? Chaque graphique répond-il à une **question** écrite dans son **titre** ? Un graphique sans question ou une affirmation

sans limite = un livrable **incomplet** (pas un graphique joli, mais un graphique qui **répond**). Le projet M09.P s'arrête là : c'est le **livrable** du module — 4 graphiques qui répondent, 1 page qui garantit.

14. Boîte à outils du chapitre

Boîte à outils. Le choix : la table question → graphique (§5.1) — courbe/barres/boxplot/histogramme/nuage. `matplotlib` (dépendance minimale) : `ax.plot` (courbe), `ax.bar` (barres), `ax.scatter` (nuage), `ax.boxplot` (boxplot pur) ; `seaborn` (confort) : `sns.boxplot`, `sns.heatmap` (les tableaux croisés, C05). Les 5 contrôles question dans le titre, axe déclaré s'il n'est pas à zéro, chaque couleur a sa légende, échelle annoncée, n dans le titre. La note d'EDA : affirmation + source (re-jouable) + limite (remplie) — 1 page par jeu.

Besoin	Commande	Chiffre du socle
courbe + bâche	<code>ax.plot + rolling(3).mean()</code>	CA mensuel 307-348 M, mg3 320-337 M
barres + n	<code>ax.bar + ax.text(f"n={n}")</code>	panier par mode, n 2 552 à 20 175
boxplot	<code>sns.boxplot</code> (ou <code>ax.boxplot</code>)	notes par segment, médianes 11.21/10.40/9.51
nuage + n	<code>ax.scatter(alpha=0.25) + n</code> au titre	montant × taux (n=50 008, corr 0.001)
heatmap	<code>sns.heatmap(annot=True)</code>	matière × trimestre (5×3, stable)
forme	<code>ax.hist</code> (échelle log annoncée)	montants 762.63 à 594 363 (ratio ~780)

15. Résumé du chapitre

Les étapes 9-10 du protocole (« quel graphique répond à la question ? » / « que dis-je, et quelles sont mes limites ? ») sont exécutées pour **clôre le module** : le graphique se choisit **par question** (courbe = évolution, barres = comparaison **avec les n**, boxplot = dispersion par groupe, histogramme = forme, nuage = association **avec le n**), et passe les **5 erreurs d'exploration** en contrôle (l'axe non zéro **déclaré**, le graphique **avec question** dans le titre, la couleur **avec légende**, l'échelle log **annoncée**, le nuage **avec n**). Les **4 graphiques du projet** sont exécutés en `matplotlib` (courbe CA + bâche, barres paniers par mode **avec les n**, nuage montant × taux **avec le n**) et le **boxplot** + la **heatmap** en `seaborn` (le **même** boxplot se fait en `matplotlib` pur — `matplotlib` est la **dépendance minimale**, `seaborn` l'option **confort**, les deux produisent le **même** graphique sur les **mêmes** données). Le **livrable de fin** est la **note d'EDA** : 1 page par jeu, chaque affirmation = **phrase** + **source** (sortie re-jouable) + **limite** (remplie) — la note du fil rouge (quincaillerie) porte **4 affirmations** (tendance plate 307-348 M, 8 doublons exclus 50 008 → 50 000, clientèle plate top 10 % = 13.8 % du CA, pas de lien taux de remise × montant corr 0.001) et **rejoue les 3 jeux** chacun en un paragraphe (santé : saisonnier 853/655 + rupture qui compte ; scolaire : associé -0.232 partiellement par une tierce variable + 4e la plus absente 4.26 j). La **rétro-lecture** (lire une note d'autrui) est la 6^e école : 5 questions dans l'ordre (question, axe, couleur, échelle, n) départagent le **présentable** du **rejetable en présentation**. Et la transition vers M10 tient en une phrase : « Vos résultats sont justes. S'ils sont illisibles, ils ne serviront à rien. »

16. À retenir

À retenir. Le graphique se choisit **par question**, il se **légende** (question + source + n + axe déclaré), et la note d'EDA **garantit** — chaque affirmation est **sourcée** (re-jouable) et **limitée** (remplie). Le module s'arrête là : pas 40 graphiques jolis, **4 qui répondent** et **1 page qui garantit**.

1. **La question est dans le titre** — jamais dans un sous-titre : le titre **est** la question, la légende porte la source et le n.
2. **Les 5 erreurs ont 5 contrôles** : axe déclaré s'il n'est pas à zéro, graphique avec question, couleur avec légende, échelle log annoncée, nuage avec n.
3. **L'axe non zéro est légitime s'il est déclaré** — sur les données **relatives** (taux, part), zéro est l'axe **honnête** ; sur les **niveaux**, déclarer.
4. **matplotlib pour le projet, seaborn pour le confort** — le même boxplot en `ax.boxplot` ou `sns.boxplot` : dépendance minimale vs confort, les **mêmes** données.
5. **La note d'EDA garantit les limites** — phrase + source (re-jouable) + **limite** (remplie) : c'est la **limite** qui fait la différence (l'épreuve P2 la **note**).
6. **La rétro-lecture est l'école** — 5 questions dans l'ordre (question, axe, couleur, échelle, n) : on repère la **présentation** en défaut **sans** relancer le code.
7. **Le livrable du module** : 4 graphiques qui **répondent** + 1 page qui **garantit** — et la transition M10 : « Vos résultats sont justes. S'ils sont illisibles, ils ne serviront à rien. »

17. Évaluation formative (auto-correction, 8 min)

1. **Quelle question chaque graphique répond-il (courbe, barres, boxplot, histogramme, nuage) ?** → courbe = évolution (« ça monte, ça descend, ça cycle ? »), barres = comparaison (avec les n), boxplot = dispersion par groupe, histogramme = forme de la variable, nuage = association (avec le n) — le graphique se choisit **par question**, pas par joliesse. (2 pts)
2. **La courbe du CA mensuel est sur axe 310-350 M. C'est une erreur ?** → pas **si elle est déclarée** (« axe 310-350 M » dans le titre) ; c'est une erreur **silencieuse** sans déclaration — l'œil lit une onde là où il y a une série plate (320-337 M en bâche). L'axe non zéro est un **choix d'édition légitime**, jamais silencieux. (2 pts)
3. **Pourquoi le n est-il dans le titre du nuage (et la transparence mentionnée) ?** → 50 008 points **ou** 200 points ressemblent à la même bande : sans le **n**, le lecteur ne sait pas la taille de l'échantillon (échantillon **fantôme**) ; la transparence qui **masque** des points doit être **mentionnée** (0.25 ici). (2 pts)
4. **Les 6 ventes à 594 363 (C04) en histogramme log non annoncée : que se passe-t-il, et pourquoi ?** → le **plateau** du montant max se **lisse/disparaît** visuellement : l'échelle log (ratio ~780 des montants) **compresse** l'extrême — c'est un **choix d'édition** qui doit être **annoncé** (« axe log »), sinon c'est une distribution **cachée**. (2 pts)
5. **Quelle est la structure d'une affirmation de la note d'EDA ?** → **phrase** (ce qu'on dit) + **source** (la sortie exécutée, **re-jouable**) + **limite** (ce qu'elle **ne** dit pas, **remplie**) : la limite est le **contrôle** de l'affirmation — une limite vide = une affirmation qui **déborde** (reculée ou retirée). (2 pts)
6. **matplotlib vs seaborn : lequel pour le projet, et pourquoi les deux sont-ils documentés ?** → **matplotlib** est la **dépendance minimale** (le projet l'utilise : courbe, barres, nuage, boxplot pur `ax.boxplot`) ; **seaborn** est l'**option confort** (boxplot, heatmap des tableaux croisés) — les deux produisent le **même** boxplot sur les **mêmes** données, et on **documente** les deux. (2 pts)

7. **La rétro-lecture : les 5 questions, dans l'ordre ?** → (1) quelle **question** ce graphique répond-il ? (2) l'**axe** est-il à zéro ou **déclaré** ? (3) chaque **couleur** a-t-elle sa **légende** ? (4) l'**échelle** (log/lin) est-elle **annoncée** ? (5) le **n** est-il **dans le titre** (et la transparence mentionnée) ? — une note qui passe les 5 est **présentable**. (2 pts)
8. **Pourquoi l'épreuve de palier P2 note-t-elle la « limite » plutôt que le chiffre ?** → parce que les **chiffres** ont déjà été **vérifiés** par le protocole (C01-C05, autovalidés **--strict**) : ce qui reste à juger, c'est **l'honnêteté** — ce que l'affirmation **ne** dit pas. La **limite** est la signature de l'**analyste** (pas du stagiaire qui calcule), et c'est **elle** que le lecteur pressé **lit**. (2 pts)

MANUEL PROFESSIONNEL · ÉDITION 2026

Module M10 — Data visualization

La Voie des Données · niveau N3 · 30 h · 7 chapitres

Analyse de données appliquée & Business Intelligence
7 partie(s) assemblée(s) · document de production

Table des matières

MODULE M10.C01 — COMMENT L'ŒIL LIT UN GRAPHIQUE : L'ORDRE PERCEPTUEL ET CE QU'IL PRESCRIT	7
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	8
3. Explication simple — la file d'attente de l'œil	8
4. Vocabulaire essentiel	9
5. Cours approfondi	9
5.1 Pourquoi un graphique n'est pas un tableau	9
5.2 La hiérarchie des cinq familles, mesurée	10
5.3 Les trois questions de la perception	10
5.4 Le même chiffre, cinq encodages : ce que devient l'écart	11
5.5 Ce que la hiérarchie ne dit pas	12
5.6 Ce que ça prescrit : question, tâche, encodage, outil	12
6. Exemple concret — les 8 parts de la quincaillerie, codées de cinq façons	13
7. Démonstration pas à pas — reproduire la mesure en 6 étapes	15
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — « les deux parts collées » (15 min, /10)	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M10.P1 — « la grille en 18 points, version 1 » (1 h)	19
14. Boîte à outils du chapitre	19
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M10.C02 — LE TABLEAU QUESTION → GRAPHIQUE : LES 16 GRAPHIQUES DU MANUEL ET LA CARTE DE DÉFINITION DES MÉTRIQUES	22
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	8
3. Explication simple — la pharmacie des seize tiroirs	23
4. Vocabulaire essentiel	9
5. Cours approfondi	9
5.1 L'arbre de choix : cinq symptômes, quatre familles	24
5.2 Le catalogue : les 16 graphiques, ligne à ligne	25
5.3 Les cinq graphiques que la liste fondatrice n'avait pas	26
5.4 Le graphique qui ment sans être faux	27
5.5 Quand un tableau bat un graphique	28
5.6 La carte de définition des métriques : l'artefact signature	28
5.7 Ce que le catalogue ne dit pas	29
6. Exemple concret — la planche des seize, dessinée sur le fil rouge	30

7. Démonstration pas à pas — trois questions, trois graphiques	31
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — « la demande de la direction » (15 min, /10)	33
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M10.P2 — « la carte des métriques du fil rouge » (1 h)	34
14. Boîte à outils du chapitre	19
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M10.C03 — COULEURS, PALETTES, ACCESSIBILITÉ ET CULTURE LOCALE	38
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	8
3. Explication simple — le code de la route des couleurs	39
4. Vocabulaire essentiel	9
5. Cours approfondi	9
5.1 Les trois familles de palettes et leur règle d'appariement	40
5.2 Contraste : ce qui se lit et ce qui ne se lit pas	41
5.3 Dal tonisme : simuler pour vérifier	41
5.4 La redondance : couleur et forme et libellé	42
5.5 La couleur qui signifie deux choses	42
5.6 FCFA et formats numériques : ce qu'on écrit, ce qu'on n'écrit pas	43
5.7 Culture locale, écran et vidéoprojecteur	44
6. Exemple concret — la palette du manuel passée au banc d'essai	44
7. Démonstration pas à pas — mesurer sa propre palette en 5 étapes	45
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — « l'orange qui voulait tout dire » (15 min, /10)	48
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M10.P3 — « la charte de graphique, une page » (1 h)	49
14. Boîte à outils du chapitre	19
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M10.C04 — ÉCRIRE UN GRAPHIQUE : TITRES QUI AFFIRMENT, AXES HONNÊTES, ANNOTATIONS	53
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	8
3. Explication simple — les quatre phrases du graphique	54

4. Vocabulaire essentiel	9
5. Cours approfondi	9
5.1 Le titre qui affirme	55
5.2 L'axe honnête : zéro, ou troncature déclarée	56
5.3 L'ordre des catégories	57
5.4 L'annotation : le chiffre au bon endroit	57
5.5 La hiérarchie visuelle : un chiffre, deux appuis, une phrase	58
5.6 La légende qui ne double pas le titre, et les quatre mentions obligatoires	59
6. Exemple concret — l'avant, l'après, et ce que l'écriture change	59
7. Démonstration pas à pas — réécrire un graphique en 6 gestes	61
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — « le titre qui a fait voter » (15 min, /10)	63
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M10.P4 — « les cinq graphiques réécrits » (1 h 30)	64
14. Boîte à outils du chapitre	19
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M10.C05 — LES 10 ERREURS QUI TUENT LA CRÉDIBILITÉ	67
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	8
3. Explication simple — la visite médicale du graphique	68
4. Vocabulaire essentiel	9
5. Cours approfondi	9
5.1 Les 10 erreurs, leur effet mesuré, et leur déclaration	69
5.2 L'axe tronqué et le zéro : la question du rapport	70
5.3 Le double axe : la machine à fabriquer des liens	70
5.4 Camembert, cumul, treemap : les défauts de la composition	71
5.5 Surcharge, couleur et ordre : les défauts d'écriture	71
5.6 L'échelle logarithmique : la plus utile et la plus dangereuse	72
5.7 Relire un graphique en 60 secondes	72
6. Exemple concret — le mur des 10 erreurs, mesuré sur le fil rouge	73
7. Démonstration pas à pas — mesurer un défaut en 5 gestes	74
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — « l'audit des cinq défauts » (15 min, /10)	76
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M10.P5 — « le diagnostic mesuré des cinq défauts » (1 h 30)	78
14. Boîte à outils du chapitre	19
15. Résumé du chapitre	20

16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M10.C06 — STORYTELLING : RACONTER POUR DÉCIDER	81
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	8
3. Explication simple — trois écrans, une minute	82
4. Vocabulaire essentiel	9
5. Cours approfondi	9
5.1 Les trois écrans, et ce qui ne va dans aucun	83
5.2 La séquence qui répond aux objections	84
5.3 Deux versions, un seul socle	84
5.4 Le temps de lecture : 20 secondes par écran	85
5.5 Ce qui ne se met pas dans une séquence	85
5.6 La narration n'embellit pas : elle ordonne	86
5.7 La séquence dans le travail du module	86
6. Exemple concret — la maquette des six écrans	87
7. Démonstration pas à pas — construire la séquence en 5 gestes	88
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — « la séquence du fil rouge » (15 min, /10)	90
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M10.P6 — « la séquence de trois écrans, en deux versions » (1 h 30, /18)	91
14. Boîte à outils du chapitre	19
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M10.C07 — MATPLOTLIB, EXCEL, POWER BI : SUPPORT, RÉOLUTION, EXPORT	94
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	8
3. Explication simple — trois outils, un chiffre	95
4. Vocabulaire essentiel	9
5. Cours approfondi	9
5.1 Le contrat des trois outils	96
5.2 matplotlib, exécuté : quatorze lignes, quatre formats	96
5.3 Excel, exécuté : le classeur écrit et relu	97
5.4 Power BI, cité : neuf clics et une déclaration	98
5.5 Support et résolution : la règle des 300 dpi	98
5.6 Export vectoriel : le format des graphiques de traits	99
5.7 Quelle chaîne pour quel livrable	99
6. Exemple concret — le même graphique dans trois outils	100
7. Démonstration pas à pas — sortir un graphique en 5 gestes	101

5.8 Ce que le chapitre n'apprend pas	102
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — « l'audit d'un export » (15 min, /10)	103
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet M10.P7 — « le même graphique en trois outils » (1 h 30, /18)	104
14. Boîte à outils du chapitre	19
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	21

Module M10.C01 — Comment l'œil lit un graphique : l'ordre perceptuel et ce qu'il prescrit

Outils : matplotlib 3.10.9, pandas 2.2.3, numpy 2.3.5 (exécutés dans ce chapitre) ; seaborn 0.13.2 (annoncé, C03 et C07) ; Excel (exécuté, C07) ; Power BI (cité, non exécuté, C07). **Durée indicative :** 4 h. **Niveau :** N3. **Prérequis :** M09 (la question avant le chiffre, et les quatre graphiques de la note d'exploration en C06) et M02 (les statistiques descriptives).

L'idée du chapitre. Un graphique n'est pas une décoration : c'est un **codage**. Une question métier se traduit en une **tâche de lecture** — comparer deux barres, suivre une courbe, repérer un écart à une moyenne — et l'œil ne réussit pas toutes ces tâches avec la même précision. Ce chapitre donne la hiérarchie des cinq familles de codage (position, longueur, angle, aire, intensité de couleur), la **mesure** sur un même jeu de 8 parts, et la règle qui en découle : **l'encodage se choisit après la question et avant le logiciel**. Sorties du chapitre : un graphique exécuté, une lecture comparée chiffrée (les mêmes 8 valeurs codées de cinq façons) et la **première version de la grille de conception en 18 points**.

Matériel de l'atelier — Python 3.13 · matplotlib 3.10.9 · pandas 2.2.3 · numpy 2.3.5. Toutes les commandes de ce chapitre ont été exécutées dans l'atelier le 20/09/2026. La planche du chapitre, `figures/M10_C01_cinq_encodages.svg`, est produite par `tools/perception_M10.py` : deux exécutions donnent le **même fichier** (aucun horodatage, identifiants figés). Les mesures sont celles du socle M09 — les **50 008 ventes** de la quincaillerie — déjà utilisées par le dossier de refonte du module.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Nommer les cinq familles de codage** d'un graphique (position, longueur, angle, aire, intensité de couleur) et dire quelle **tâche de lecture** chacune sert.
2. **Dérouler la hiérarchie perceptuelle** établie par les expériences de perception graphique et la citer **avec ses chiffres** : l'erreur moyenne des jugements de longueur dépasse celle des jugements de position de 40 % à 250 %, celle des jugements d'angle la double à peu près.
3. **Mesurer** ce qu'un écart de données devient dans chaque famille : sur les 8 parts du fil rouge, l'écart le plus petit (0,05 point) vaut **0,4 mm** sur un axe large comme la page, **0,17°** sur un camembert, **0,32 %** de surface sur des disques, **0,36 L*** sur une rampe de couleur.
4. **Choisir un encodage dans le bon ordre** : la question d'abord, la tâche ensuite, l'encodage, et le logiciel en dernier — et dire pourquoi l'ordre inverse produit des graphiques irréprochables qui ne répondent à rien.
5. **Remplir la première version de la grille de conception en 18 points** (4 familles : décision servie, justesse, lisibilité, utilisation) sur un graphique fourni.

2. Pourquoi cette notion est importante

Un décideur ne lit pas un graphique : il le **regarde cinq secondes**, puis décide. Ces cinq secondes ne sont pas du temps perdu, c'est le budget de lecture — et tout graphique mal encodé le dépense à faire deviner au lecteur ce que les données disent déjà.

L'enjeu du module tient dans une phrase : **un graphique faux se repère, un graphique trompeur non**. Le rapport commercial fourni avec ce module (dossier M10) ne contient pas un seul chiffre faux ; ses totaux sont exacts. Ce sont ses graphiques qui font lire autre chose : un axe qui commence à 300 M transforme une variation de 5,4 % en falaise, deux échelles superposées fabriquent une corrélation de 0,35, un camembert à huit parts invite à classer des parts séparées de 0,05 point. Personne ne peut accuser le rapport de mensonge : il n'y a rien à réfuter, seulement des **décisions de codage** — prises, le plus souvent, sans savoir qu'on les prenait.

Ce chapitre est le premier des sept du module parce qu'il donne **la contrainte qui commande les six autres** : ce que l'œil peut lire, et à quel prix. Il prolonge directement M09 : vous y avez appris à poser la question avant le chiffre ; vous apprenez ici à poser la question avant **la forme**.

Définition. Le **budget de lecture** est le temps qu'un lecteur accorde réellement à un graphique dans un dossier ou une réunion : de l'ordre de quelques secondes. Tout ce chapitre se juge à l'aune de ce budget — un codage se choisit pour ce que le lecteur peut faire **dans ce temps-là**, pas pour ce qu'un analyste peut faire en cinq minutes devant le même dessin.

Dans les faits. Sur le fil rouge, la plus petite part de chiffre d'affaires par catégorie vaut 8,85 % et la plus grande 15,63 % : un rapport de **1,77 pour 1**. Un graphique qui sert cette question — « quelle est l'amplitude entre les catégories ? » — se lit d'un coup d'œil et se vérifie en une seconde. Le même jeu de valeurs, publié en camembert à huit parts, sert une autre question (« classez-moi les huit catégories ») que **personne** ne peut lire : deux parts voisines y sont séparées de 0,17°. Le même chiffre, deux questions, deux verdicts.

3. Explication simple — la file d'attente de l'œil

Imaginez un guichet avec cinq employés. On leur présente la même pile de dossiers, et on chronomètre. Le premier lit une **position** sur une règle : « celle-ci est à 14,7, celle-là à 15,6 ». Le deuxième mesure une **longueur** : « cette barre fait 14,7, celle-là 15,6 ». Le troisième n'a qu'un **rapporteur** : il mesure des angles sur un camembert. Le quatrième doit estimer des **surfaces** — des disques dont le diamètre change. Le cinquième ne lit que des **nuances** de couleur.

Quatre réflexes pour se souvenir de la file :

1. **La règle bat le rapporteur.** Comparer deux positions sur un même axe est la tâche la plus précise que l'œil sache faire ; mesurer un angle est nettement moins précis ; estimer une surface l'est moins encore. Ce n'est pas une opinion de graphiste, c'est un résultat expérimental (voir §5.2).
2. **L'écart se paie dans la famille du codage.** Un écart de 0,05 point de pourcentage est invisible en degrés et en surface, à peine perceptible en position. **Changer de graphique ne change pas le chiffre ; cela change la taille de l'écart à lire.**
3. **La question décide, pas le goût.** « Comparer huit parts » demande un classement ; « montrer une répartition » demande un tout ; « suivre une évolution » demande une ligne. Ces trois questions n'ont pas le même graphique naturel.
4. **Le logiciel vient en dernier.** Excel, matplotlib et Power BI savent produire à peu près n'importe quoi : c'est bien pourquoi le choix doit être fait **avant** d'ouvrir l'outil. Un graphique écrit par le logiciel par défaut est une décision de codage prise par défaut.

La file d'attente de l'œil tient en une règle : **on ne demande pas à l'œil une tâche qu'il ne sait pas faire ; on reformule la question jusqu'à ce qu'elle devienne lisible.**

4. Vocabulaire essentiel

Terme	Définition
encodage	la manière dont une valeur est traduite en signe visuel : hauteur, longueur, angle, surface, nuance, position, forme.
canal perceptuel	le signe utilisé pour porter la valeur (par exemple la longueur d'une barre). Un graphique combine 2 ou 3 canaux : position sur l'axe, longueur, couleur.
tâche perceptuelle élémentaire — elementary perceptual task	le jugement exact demandé à l'œil : comparer deux positions, comparer deux longueurs, juger un angle, une surface, une nuance. C'est l'unité de mesure de la difficulté d'un graphique.
ordre perceptuel	le classement de ces tâches par précision, établi expérimentalement, du plus précis au moins précis : position, longueur, angle et pente, aire, puis intensité de couleur.
graphique trompeur	un graphique aux chiffres exacts qui fait néanmoins lire autre chose, à cause de son codage (axe tronqué, double axe, aire disproportionnée, ordre arbitraire). Ce n'est pas un mensonge : c'est un défaut de conception.
question servie	la question à laquelle le graphique répond seul , telle qu'un lecteur la formule en le regardant. Un graphique a une question servie, pas cinq.
grille de conception — design checklist	la liste de contrôle du module (18 points, 4 familles) qui sert à relire un graphique avant publication : décision servie, justesse, lisibilité, utilisation.

5. Cours approfondi

5.1 Pourquoi un graphique n'est pas un tableau

Un tableau donne **la valeur exacte** ; un graphique donne **la forme**. Ce n'est pas une hiérarchie, c'est un partage des tâches, et les deux erreurs symétriques coûtent cher.

Première erreur : vouloir faire dire une **forme** à un tableau. « Le chiffre d'affaires est-il plat depuis deux ans ? » : balayez une colonne de 24 valeurs et vous ne verrez rien ; tracez une ligne et la réponse saute aux yeux. Le cerveau ne compare pas des nombres, il compare des **positions**.

Deuxième erreur : demander à un graphique une **valeur exacte**. Personne ne lit 119 482 FCFA sur un axe : on lit « un peu moins de 120 000 ». Si le lecteur a besoin du chiffre exact, il le faut écrire dans le graphique (une annotation) **et** le donner dans le tableau qui l'accompagne. C'est la règle de l'annotation, développée en C04.

Définition. Un graphique est un **argument visuel** : il affirme une chose (« ces huit parts se ressemblent », « cette courbe se retourne ») et son lecteur la croit sans calculer. D'où la conséquence du module : celui qui publie un graphique est responsable de ce que son lecteur croira, pas seulement de ce que ses chiffres disent.

Le budget de lecture explique le reste. Un lecteur de dossier accorde **cinq secondes** à un graphique : le temps de comprendre le titre, de balayer les axes et de repartir avec une idée. Ces cinq secondes ne se négocient pas ; elles se **préparent**. Trois préparations rendent les cinq secondes rentables : la question est identifiable (le titre l'affirme), la marque à comparer est trouvable (axes et unités présents, ordre trié) et la conclusion est atteignable sans calcul mental (annotation du chiffre clé, échelle honnête).

5.2 La hiérarchie des cinq familles, mesurée

L'ordre perceptuel n'est pas une doctrine, c'est un **résultat**. Les expériences fondatrices de Cleveland et McGill (1984) ont fait juger à des participants des proportions codées de différentes façons, puis comparé leurs erreurs. Le classement obtenu va du plus précis au moins précis :

1. **position sur une échelle commune** (deux points sur le même axe) ;
2. **position sur des échelles non alignées** (deux graphiques côte à côte) ;
3. **longueur, direction, angle** (barres, pentes, parts de camembert) ;
4. **aire** (disques, bulles, rectangles proportionnels) ;
5. **volume, courbure** ;
6. **intensité de couleur** (nuances, saturations).

Trois chiffres de ces expériences méritent d'être retenus, car ils sont **utilisables dans une conversation professionnelle** :

- l'erreur moyenne des jugements de **longueur** dépasse celle des jugements de **position** de **40 % à 250 %** selon la configuration ;
- l'erreur moyenne des jugements d'**angle** (le camembert) est à peu près **deux fois** celle des jugements de position — l'écart vaut 0,97 sur l'échelle logarithmique utilisée par les auteurs ;
- dans cette expérience, le camembert n'a été **plus précis que le diagramme en barres que dans 3 cas sur 40** — et jamais dans la plage de valeurs intermédiaires, qui est justement celle des parts de la quincaillerie (8,85 % à 15,63 %).

Dans les faits. La hiérarchie a été **répliquée** en 2010 sur des milliers de participants recrutés en ligne (Heer et Bostock) : même classement. Autrement dit, elle ne dépend ni du laboratoire, ni du papier, ni de l'écran : elle décrit une contrainte du système visuel, pas une habitude culturelle.

La hiérarchie se lit comme un **prix**. Un graphique coûte d'autant plus cher à lire qu'il descend dans la liste. Et le prix ne se paie pas en « beauté » ou en « modernité » : il se paie en **erreur de lecture** — c'est-à-dire, en bout de chaîne, en décisions légèrement fausses. Un dirigeant qui classe huit catégories d'après un camembert les classe **au hasard** dans la zone 8-16 % ; le même dirigeant, devant des barres triées, classe juste et vite.

Définition. L'**ordre perceptuel** est le classement des tâches de lecture par précision moyenne. Il ne dit pas « ce graphique est laid » : il dit « cette comparaison coûtera tant d'erreur au lecteur ». La bonne question de conception devient donc : quelle précision ma décision exige-t-elle ?

5.3 Les trois questions de la perception

Derrière la hiérarchie, il n'y a que trois tâches, et tout graphique en sert une :

La question du lecteur	La tâche de lecture	Les codages naturels
« Lequel est le plus grand ? » (comparer)	comparer deux positions sur un même axe	barres, points alignés, boîtes à moustaches, carte de chaleur par ligne
« Est-ce que ça monte ? » (évoluer)	suivre une position au fil du temps	courbes, aires empilées, barres par période, petits multiples
« Comment est-ce réparti ? » (distribuer)	estimer une part d'un tout, une dispersion	barres empilées à 100 %, camembert (2 à 3 parts), histogramme, boîte à moustaches

Deux conséquences pratiques. D'abord, **la même donnée change de graphique quand la question change** : la répartition du chiffre d'affaires par catégorie se montre en barres triées si

l'on veut comparer, en camembert à trois parts si l'on veut montrer « ces trois familles font tout », en courbes si l'on veut suivre chaque catégorie dans le temps. Ensuite, **un graphique qui sert deux questions n'en sert aucune** : dès qu'un titre contient « et », il faut vérifier qu'on n'a pas collé deux tâches dans un seul cadre.

Attention. L'intensité de couleur est le canal le **moins** précis de la liste, et c'est celui que les logiciels proposent le plus volontiers (couleur par catégorie, par valeur, par série). Un graphique dont l'information principale est portée par la nuance — et non par une position — demande au lecteur la tâche la plus coûteuse. La couleur reste un excellent **appui** (repérer une série, souligner un point) ; elle est un mauvais **support unique** d'une comparaison chiffrée.

5.4 Le même chiffre, cinq encodages : ce que devient l'écart

Voici l'expérience du chapitre, entièrement exécutée à l'atelier (§6 montre la planche, §7 donne le code). On prend les **8 parts** du fil rouge — le chiffre d'affaires de la quincaillerie par catégorie — et on les code de **cinq façons** : position de points sur un axe, longueur de barres depuis zéro, angle d'un camembert, aire de disques proportionnels, intensité d'une rampe de couleur séquentielle.

Le résultat n'est pas « un graphique est plus joli » : c'est un **tableau de conversion**. Le même écart de données, lu dans chaque famille :

Encodage	L'écart le plus petit entre deux parts	L'écart le plus grand	Ce que la famille peut encore dire
position (axe de 170 mm)	0,4 mm	22,5 mm	toute comparaison au-delà du demi-millimètre
longueur (barres depuis zéro)	0,4 mm	22,5 mm	idem, avec un repère de zéro plus franc
angle (camembert)	0,17°	9,51°	les parts franchement différentes (au-delà de quelques degrés)
aire (disques $\propto$ valeur)	0,32 % de surface	28,5 % de surface	les rapports de 1 à 2 et plus, jamais les écarts fins
couleur (rampe de 30 L*)	0,36 L*	11,17 L*	les grands contrastes, pas les nuances voisines

Définition. On appelle **facteur de conversion** d'un canal la quantité de ce canal qui correspond à un point de pourcentage de données : 8,5 mm sur un axe large comme la page, 3,6° sur un camembert, une variation relative pour une aire, une différence de clarté L* pour une couleur. Connaître ces facteurs, c'est pouvoir dire « ce classement ne se lira pas » **avant** de dessiner.

Trois lectures de ce tableau.

Première lecture : l'écart le plus petit est hors de portée. L'écart minimal de nos données vaut 0,05 point de pourcentage — entre la catégorie 5 (14,66 %) et la catégorie 8 (14,71 %). Sur un axe large comme la page, cela fait **0,4 mm** : moins que l'épaisseur d'un trait de crayon. Aucune des cinq familles ne rend ce classement lisible ; la meilleure d'entre elles (la position) est justement celle dont les erreurs de jugement se mesurent en dizaines de pour cent, pas en dixièmes de millimètre.

Deuxième lecture : ce n'est pas une raison de renoncer. Le même jeu de données contient une information parfaitement lisible : la plus petite part (8,85 %) vaut **1,77 fois** la plus grande (15,63 %) moins elle-même, et l'étendue des huit parts fait **6,78 points**, soit **57,6 mm** sur un axe large comme la page — cent quarante fois l'écart minimal. Le problème n'est donc pas le jeu de données : c'est la **question** qu'on lui pose. « Classez ces huit parts » est une question inter-

dite. « Ces huit catégories pèsent-elles pareil ? » a une réponse nette, et un seul graphique la donne.

Troisième lecture : changer de famille ne change jamais le chiffre. Les cinq encodages portent exactement les mêmes valeurs ; les totaux sont identiques au centime. Ce que le codage change, c'est **la taille apparente de l'écart** — donc la décision qu'on en tire. C'est toute la matière du chapitre C05, où les cinq défauts du rapport fourni seront mesurés un par un.

Attention. Une aire ne se lit pas comme un diamètre : si vous codez une valeur par un rayon, la surface varie comme le **carré** de la valeur, et vous rendez l'écart deux fois plus grand qu'il n'est. Dans la planche du chapitre, les disques sont dimensionnés par la **surface** ($s = (\sqrt{(\text{valeur} / \text{valeur maximale}) \times k})^2$), seule façon honnête de coder un rapport par une aire. Le défaut est planté dans deux graphiques du dossier de refonte : il vous attend en C05.

5.5 Ce que la hiérarchie ne dit pas

Une hiérarchie mal comprise devient un dogme — et un dogme produit des graphiques pauvres. Quatre précisions, à connaître avant d'en faire une règle de service :

1. **La hiérarchie parle de précision, pas de légitimité.** Un camembert à deux parts est parfaitement lisible : l'angle y fait 216° contre 144°, l'écart est énorme. Le problème du camembert n'est pas sa famille, c'est **le nombre de parts** : au-delà de trois ou quatre, deux voisines deviennent indiscernables (0,17° chez nous).
2. **Elle se mesure sur des jugements isolés.** Un graphique publié vit dans un contexte : une annotation, un tableau à côté, un commentaire oral pendant la présentation. Ces appuis réduisent le coût du codage — à condition que l'information principale **ne soit pas** dans le canal faible.
3. **Elle ne hiérarchise pas le temps.** Pour une **évolution**, une courbe (position) bat tout ; pour une **répartition d'un tout**, une barre empilée unique peut battre un camembert ; pour une **hiérarchie**, c'est la longueur **triée** qui gagne. Les trois tâches de §5.3 se répondent avec trois familles différentes : il n'y a pas un « meilleur graphique », il y a un meilleur appareillage.
4. **Elle ignore les contraintes du support.** Un graphique lu à trois mètres sur un vidéoprojecteur, imprimé en noir et blanc ou consulté sur un téléphone perd de la précision dans un ordre qui n'est pas celui de la liste : la couleur et les petits multiples souffrent d'abord. C'est l'objet de C07.

Conseil professionnel. Quand un collègue vous dit « on ne peut pas faire un camembert, c'est interdit », la bonne réponse n'est ni « si » ni « non » : demandez **combien de parts et quelle question**. Trois parts et un tout : parfait. Huit parts et un classement : refusez, mais refusez la **question**, pas le graphique — et proposez la même donnée en barres triées, avec le commentaire « ces huit catégories pèsent de 8,85 % à 15,63 % ».

5.6 Ce que ça prescrit : question, tâche, encodage, outil

Le chapitre se termine par la chaîne de décision que le module utilisera partout, dans cet ordre strict :

1. **La question**, écrite en une phrase, avec son destinataire : « le comité veut savoir si les catégories sont déséquilibrées ».
2. **La tâche de lecture** qu'elle implique : comparer huit positions, ou montrer une étendue, ou suivre une évolution.

3. **L'encodage** qui rend cette tâche la moins coûteuse : barres triées, axe de zéro, 8 libellés, une annotation pour l'étendue.
4. **L'outil**, en dernier : ici `matplotlib` (atelier), ailleurs Excel ou Power BI selon le support de destination. L'outil ne choisit rien : il exécute le choix des étapes 2 et 3.

Cette chaîne a une propriété précieuse : elle est **vérifiable**. Une fois posée, n'importe qui peut relire le graphique produit en posant quatre questions, qui sont la première version de notre grille de conception. La voici, en **18 points répartis en 4 familles** :

Famille	Ce qu'elle vérifie	Barème
1. La décision servie	la question est écrite en une phrase et le graphique y répond seul ; le titre affirme le constat ; une décision est possible	4
2. La justesse	axe à zéro ou troncature déclarée ; une seule échelle ; aire proportionnelle à la valeur ; ordre des catégories porteur de sens ; unités et périmètre cohérents	5
3. La lisibilité	un message par cadre ; le chiffre clé annoté ; source, date et unité présents ; contraste et couleur accessibles ; libellés compréhensibles sans le dossier	5
4. L'utilisation	une version « direction » sur un écran ; une version « équipe » avec le détail ; le support respecté ; le fichier réutilisable (export, nom, date)	4
Total		18

Les 18 critères, un point chacun, sont détaillés en §13 (mini-projet). Deux points de méthode :

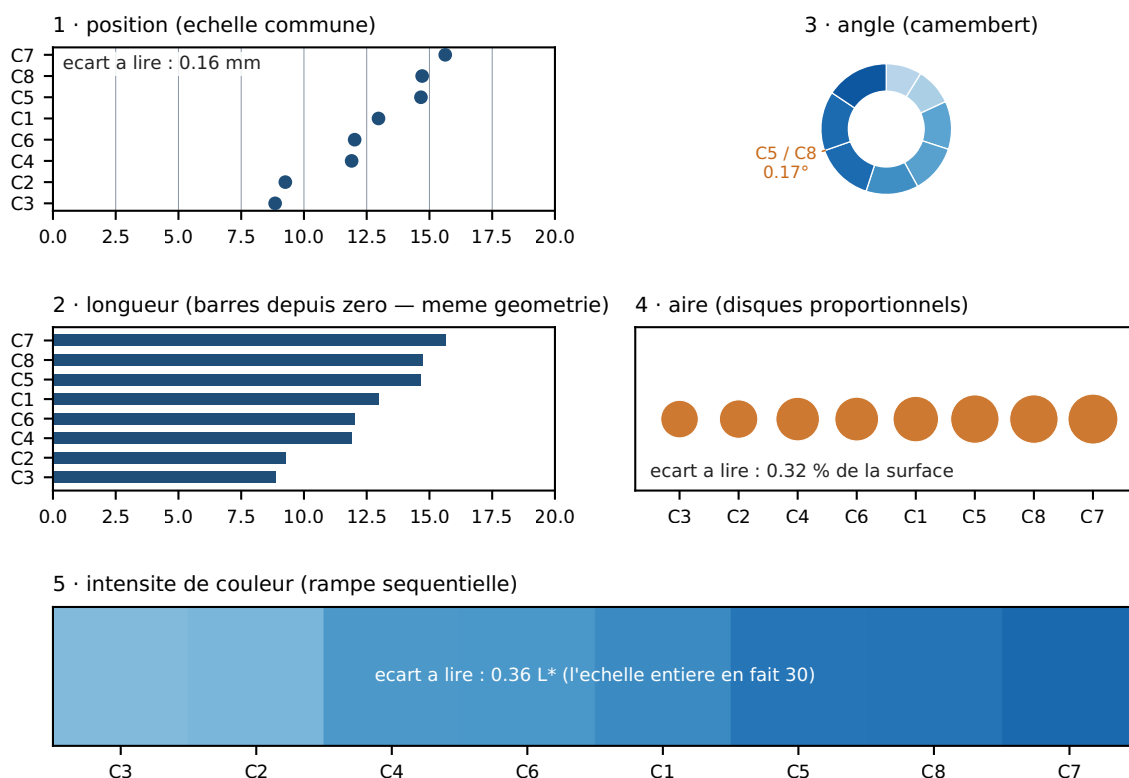
Définition. La **grille de conception** n'est pas une grille d'esthétique : chaque point est une **question fermée** à laquelle on répond par oui ou non, en montrant la preuve (« axe : zéro, oui », « ordre : trié par chiffre d'affaires décroissant, oui »). Elle sera remplie par l'auteur, puis par un pair, avant toute publication — c'est l'usage prévu au projet du module.

À retenir. Les familles 1 et 2 se corrigent **avant** de dessiner (elles sont des décisions) ; les familles 3 et 4 se corrigent **après** (elles sont des vérifications). Un graphique qui échoue en famille 1 ne se répare pas en famille 3 : aucune couleur ne sauve une question absente.

6. Exemple concret — les 8 parts de la quincaillerie, codées de cinq façons

Voici la planche produite à l'atelier : **les mêmes 8 valeurs**, cinq encodages, chacun annoté de l'écart que l'œil doit y détecter pour départager les deux catégories les plus proches.

Le meme jeu de 8 parts de CA code de cinq façons — et l'ecart que l'oeil doit y lire



Les données, triées (chiffre d'affaires TTC par catégorie, en % du total des ventes hors retours) :

Catégorie	C3	C2	C4	C6	C1	C5	C8	C7
Part du CA	8,85 %	9,26 %	11,90 %	12,02 %	12,97 %	14,66 %	14,71 %	15,63 %

Ce que la planche raconte, panneau par panneau :

- **Panneau 1 — position.** Huit points sur un axe unique de 0 à 20 %. L'écart entre C5 et C8 vaut 0,05 point, soit **0,4 mm** sur un axe large comme la page : les deux points sont confondus. L'étendue totale, elle, est parfaitement lisible : les extrêmes occupent 57,6 mm.
- **Panneau 2 — longueur.** Géométriquement identique, avec le repère du zéro plus franc. C'est le graphique du module pour comparer : c'est aussi celui que le rapport fourni n'utilise pas.
- **Panneau 3 — angle.** Le camembert à huit parts. Le même écart C5/C8 y vaut **0,17°** : cinq fois moins que la précision d'un lecteur humain, même entraîné. Une seule annotation sauve le panneau : celle qui **désigne** la paire indiscernable.
- **Panneau 4 — aire.** Huit disques dont les **surfaces** sont proportionnelles aux parts. L'écart C5/C8 devient une variation de surface de **0,32 %** : invisible. L'écart le plus grand, lui, vaut 28,5 % — les disques ne savent lire que les grands rapports.
- **Panneau 5 — intensité.** Une rampe séquentielle unique, de 30 L* d'amplitude totale. L'écart C5/C8 vaut **0,36 L*** quand la rampe entière en fait 30 : le lecteur voit deux teintes identiques. Les grandes catégories (C7, C8) se distinguent en revanche très bien.

Le verdict, en une phrase, est celui du chapitre : **le jeu de données ne contient pas de classement fin à montrer ; il contient une étendue**. La planche le prouve chiffre par chiffre, et c'est cette démonstration — mesurée, pas sentencieuse — qu'il faut savoir refaire devant un commanditaire qui demande « un camembert, comme la dernière fois ».

Attention. Ne confondez pas « indiscernable » et « négligeable ». La différence C5/C8 existe (0,05 point, soit 0,32 % du total) ; elle est simplement **hors de portée d'un graphique**. Si cette différence compte pour la décision, elle s'écrit en toutes lettres dans le commentaire — elle ne se fait pas porter par un dessin.

7. Démonstration pas à pas — reproduire la mesure en 6 étapes

Tout ce qui précède est reproductible. Les six étapes ci-dessous ont été exécutées à l'atelier, dans cet ordre ; les sorties affichées sont celles de l'atelier.

Étape 1 — les 8 parts. On reprend le socle M09 (les ventes hors retours) et on rapporte le chiffre d'affaires de chaque catégorie au total.

```
import pandas as pd, numpy as np
import matplotlib as mpl; mpl.use("Agg")
import matplotlib.pyplot as plt

v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv", parse_dates=["date_vente"])
prod = pd.read_csv("03_exercices/dossier_M09/quincaillerie/produit.csv")
vp = v[~v["est_retour"]].merge(prod[["id_produit", "id_categorie"]], on="id_produit", how="left")
part = (vp.groupby("id_categorie")["montant_ttc"].sum() / vp["montant_ttc"].sum() *
100).sort_values()
print(" | ".join(f"C{i} {x:.2f}" for i, x in part.items()))
```

C3 8.85 | C2 9.26 | C4 11.90 | C6 12.02 | C1 12.97 | C5 14.66 | C8 14.71 | C7 15.63

Étape 2 — les écarts entre voisins. L'écart qui compte n'est pas l'étendue totale mais l'écart **minimal**, celui que la question « classez-les » demande de lire.

```
e = np.diff(part.to_numpy())
print("min", round(e.min(), 2), "| median", round(float(np.median(e)), 2),
      "| max", round(e.max(), 2), "| etendue", round(float(part.max() - part.min()), 2))
```

min 0.05 | median 0.92 | max 2.64 | etendue 6.78

Étape 3 — la conversion en millimètres. On fixe l'axe de référence : 0 à 20 %, sur une largeur de 170 mm — la largeur utile d'une page du manuel. Un point de pourcentage y fait 8,5 mm.

```
axe, largeur_mm = 20.0, 170.0
print("mm par point :", largeur_mm / axe,
      "| ecart min :", round(e.min() * largeur_mm / axe, 2), "mm",
      "|", round(e.min() / axe * 100, 2), "% de l'axe")
print("plus grand ecart voisin :", round(e.max() * largeur_mm / axe, 1), "mm",
      "| extremes :", round((part.max() - part.min()) * largeur_mm / axe, 1), "mm")
```

mm par point : 8.5 | ecart min : 0.4 mm | 0.24 % de l'axe
plus grand ecart voisin : 22.5 mm | extremes : 57.6 mm

Étape 4 — les mêmes écarts en degrés. Le camembert convertit 100 % en 360°, donc un point de pourcentage en 3,6 degrés. La conversion la plus brutale du chapitre : le même écart qui faisait 0,4 mm fait ici 0,17°.


```
print("min", round(e.min() * 3.6, 2), "deg | max", round(e.max() * 3.6, 2), "deg",
      "| plus petite part", round(part.min() * 3.6, 1), "deg")
```

```
min 0.17 deg | max 9.51 deg | plus petite part 31.9 deg
```

Étape 5 — l'aire. Pour des disques dont la **surface** est proportionnelle à la valeur, ce qui compte est la variation relative de surface entre deux disques voisins.

```
i = int(np.argmin(e))
val = part.to_numpy()
print("min", round(e[i] / val[i] * 100, 2), "% | max", round(e.max() / val[int(np.argmax(e))] *
100, 1), "%")
```

```
min 0.32 % | max 28.5 %
```

Étape 6 — la couleur. On échantillonne la rampe séquentielle `Blues` et on mesure l'écart de clarté CIE L* entre les deux teintes voisines — la clarté étant le canal de couleur le plus lisible.

```
def lab(rgb):
    # sRGB -> CIE L*a*b*, sans dépendance externe
    f = lambda t: t / 12.92 if t <= 0.04045 else ((t + 0.055) / 1.055) ** 2.4
    r, g, b = [f(c) for c in rgb[:3]]
    x = r * 0.4124 + g * 0.3576 + b * 0.1805
    y = r * 0.2126 + g * 0.7152 + b * 0.0722
    z = r * 0.0193 + g * 0.1192 + b * 0.9505
    g_ = lambda t: t ** (1 / 3) if t > 0.008856 else 7.787 * t + 16 / 116
    fx, fy, fz = g_(x / 0.95047), g_(y), g_(z / 1.08883)
    return np.array([116 * fy - 16, 500 * (fx - fy), 200 * (fy - fz)])

norme = plt.Normalize(0, axe)
labo = [lab(plt.get_cmap("Blues")(norme(x))) for x in val]
dl = np.abs(np.diff([c[0] for c in labo]))
print("delta L* min", round(float(dl[i]), 2), "| max", round(float(dl.max()), 2),
      "| etendue", round(float(abs(labo[-1][0] - labo[0][0])), 1), "L*")
```

```
delta L* min 0.36 | max 11.17 | etendue 30.0 L*
```

Ce que la démonstration établit. Le même écart de 0,05 point de pourcentage vaut 0,4 mm, 0,17°, 0,32 % de surface et 0,36 L* : quatre unités, quatre impossibilités pratiques. En revanche, l'étendue des parts (6,78 points) reste lisible dans **toutes** les familles, parce qu'elle occupe 57,6 mm sur l'axe, 24,4° sur le camembert et 28,5 % de surface sur les disques. **La question décide de ce qui est lisible ; le graphique ne fait que l'encaisser.**

8. Erreurs fréquentes

1. **Choisir le graphique avant la question.** « Fais-moi un camembert » est un ordre d'outil, pas une question ; le camembert arrive alors avec huit parts et personne ne peut dire ce qu'il montre.
2. **Confondre aire et diamètre.** Coder une valeur par le rayon d'un disque double l'écart apparent (la surface varie comme le carré). Le réflexe : toujours passer par la **surface**.
3. **Croire que la couleur « ajoute de l'information ».** Une couleur par catégorie sur un graphique dont la longueur porte déjà l'information n'ajoute rien : elle double le message et charge le cadre. La couleur sert à **désigner**, pas à **quantifier** (sauf carte de chaleur, dont la lecture se fait par ligne, cf. C02).
4. **Prendre la hiérarchie pour un interdit.** « Pas de camembert, pas de bulles » : faux. Deux parts en camembert, cinq bulles pour montrer des ordres de grandeur — très bien. Huit parts et un classement fin — non.

5. **Régler le graphique avant de le valider.** Passer une heure sur les couleurs d'un graphique qui ne répond pas à sa question : c'est la famille 1 avant la famille 3, dans le mauvais ordre.
6. **Laisser le logiciel décider.** Un camembert à 8 parts, un ordre alphabétique, des milliers de séparateurs : ce sont les valeurs par défaut de la plupart des outils. Une valeur par défaut est une décision que quelqu'un d'autre a prise pour vous.

9. Bonnes pratiques professionnelles

1. **Écrire la question avant d'ouvrir l'outil**, en une phrase avec son destinataire, et la garder sous les yeux jusqu'à la publication.
2. **Choisir la famille de codage d'après la tâche** (§5.3), puis l'encodage précis, puis l'outil.
3. **Mesurer avant d'affirmer** : convertir l'écart qui compte dans l'unité du canal (mm, degrés, %, L*) avant de dire « on ne verra rien ». La mesure prend deux minutes et évite un débat.
4. **Annoter le chiffre clé** directement sur le graphique : c'est ce qui rend le codage le plus faible acceptable et le codage le plus fort lisible en cinq secondes.
5. **Trier les catégories** par la valeur ou par l'ordre métier, jamais par ordre alphabétique (sauf lorsque l'ordre alphabétique est l'information — c'est rare).
6. **Prévoir la relecture par un pair** avec la grille en 18 points, avant publication, et accepter les points 1 et 2 comme éliminatoires.
7. **Déclarer les accroc**s : si un axe n'est pas à zéro, si l'échelle est logarithmique, si une série est incomplète, cela s'écrit **dans** le graphique. Une convention déclarée est une décision ; une convention cachée est un piège.

10. Exercice guidé — « les deux parts collées » (15 min, / 10)

Contexte. Le rapport de la quincaillerie publie la répartition du chiffre d'affaires par catégorie en camembert à huit parts. La direction demande : « quelles catégories sont les plus faibles ? ».

Consigne. Sans rouvrir le chapitre, dans les 15 minutes :

1. Écrivez la question de la direction sous forme de **tâche de lecture** (comparer, évoluer, ou distribuer). (2 pts)
2. Calculez l'écart entre les **deux parts les plus proches** et convertissez-le en **degrés** et en **millimètres** (axe de 0 à 20 % sur 170 mm). (2 pts)
3. Rédigez la phrase qui répond à la direction, **sans** citer de classement fin, en utilisant l'éten- due des parts. (2 pts)
4. Proposez le graphique qui sert cette phrase : famille de codage, orientation, ordre des caté- gories, et l'annotation à porter. (2 pts)
5. Nommez **deux** points de la grille en 18 points que votre proposition satisfait, et **un** que le ca- membert publié ne satisfaisait pas. (2 pts)

11. Exercices autonomes

E1 — la même question, cinq encodages (25 min). Reprenez la mesure du §7 sur **votre** jeu de travail (le fichier de projet M09 ou l'une des tables du centre de santé). Choisissez **cinq va- leurs** dont une paire voisine très proche, codez-les de cinq façons et remplissez un tableau à cinq lignes : encodage, écart minimal dans l'unité du canal, écart maximal, choix (lisible / illi- sible). Concluez en une phrase : quelle question ce jeu de valeurs **peut** servir, et laquelle il ne peut pas.

E2 — la hiérarchie à l'épreuve de votre propre graphique (20 min). Prenez un graphique que vous avez publié (ou un graphique d'un rapport reçu). Écrivez sa **question servie** en une phrase ; nommez la tâche de lecture demandée et la famille de codage utilisée ; mesurez l'écart que le lecteur doit lire s'il veut départager les deux catégories les plus proches. Puis répondez à une seule question : **le codage choisi servait-il cette question, ou servait-il le logiciel ?** Votre réponse doit tenir en cinq lignes et citer au moins une mesure.

12. Correction détaillée

Exercice guidé.

1. **Tâche de lecture** : comparer des positions — la question porte sur un classement (« les plus faibles »), donc sur une comparaison de huit valeurs, pas sur un tout à découper.
2. **Écart minimal** : 0,05 point (C5 à 14,66 %, C8 à 14,71 %). En degrés : $0,05 \times 3,6 = 0,17^\circ$. En millimètres : $0,05 \times 8,5 = 0,4 \text{ mm}$ sur un axe de 170 mm. Les deux conversions se font de tête une fois les facteurs connus (3,6°/point, 8,5 mm/point).
3. **Phrase attendue** : « Les huit catégories pèsent de 8,85 % à 15,63 % du chiffre d'affaires, soit un rapport de 1,77 pour 1 : aucune n'est négligeable, et les quatre dernières ne se départagent pas graphiquement (0,05 point les séparent, soit 0,17°). » — la phrase **ne classe pas** ce qui n'est pas classable ; elle donne l'étendue et déclare la limite.
4. **Graphique attendu** : barres **horizontales**, triées par chiffre d'affaires décroissant (les libellés des catégories se lisent sans rotation), axe de zéro à 20 % avec le zéro visible ; une annotation sur la plus grande barre (« 15,63 % ») et une sur la plus petite (« 8,85 % »), plus un trait ou une accolade qui matérialise l'étendue (6,78 points, 57,6 mm). Famille : position / longueur. Aucun dégradé de couleur.
5. **Points satisfaits (au choix)** : famille 1 (« la question est écrite et le graphique y répond seul »), famille 2 (« axe à zéro », « ordre trié porteur de sens »), famille 3 (« le chiffre clé est annoté », « source, date et unité présents »). **Point manqué par le camembert** : le classement demandé, c'est-à-dire la famille 1 — le graphique publié ne répond pas à la question posée — et, dans la famille 3, l'absence d'annotation du chiffre clé (aucune valeur absolue sur la figure).

E1 (la même question, cinq encodages). Une grille de correction, plutôt qu'un corrigé unique :

- l'écart minimal est bien calculé **en points de pourcentage**, puis converti par les bons facteurs (3,6°/point pour l'angle ; mm par point = largeur de l'axe ÷ étendue de l'axe ; variation relative pour l'aire ; ΔL^* pour la couleur — la clarté, pas la teinte) ;
- la conclusion sépare **ce que le jeu peut dire** (l'étendue, les ordres de grandeur) de ce qu'il ne peut pas (le classement fin) : c'est le seul point vraiment discriminant ;
- le tableau comporte un **choix** par ligne (« lisible / illisible »), pas seulement des nombres : une mesure sans décision n'est pas une analyse. Ordre de grandeur attendu : au-delà d'un écart d'environ 1 point de pourcentage entre voisines, la plupart des encodages restent lisibles ; sous 0,2 point, aucun ne l'est sans annotation.

E2 (la hiérarchie à l'épreuve de votre graphique). Trois réponses types, toutes acceptables si elles sont mesurées :

- Une courbe d'évolution avec deux séries et un double axe : la question servie est « comment évoluent-elles ? », la tâche est de suivre des positions, mais les deux échelles rendent toute comparaison de niveaux impossible — la corrélation **fabriquée** du dossier de refonte (0,35) est exactement ce cas ;

- Un camembert à huit parts : la question servie devrait être un classement ; la mesure de l'écart minimal la déclare illisible ; la correction est un changement de **question** (montrer l'étendue) ou de **graphique** (barres triées) ;
- Un histogramme de montants en échelle logarithmique : la tâche est de comparer des positions, et les positions sont conservées — mais les écarts **relatifs** seuls restent lisibles : la longueur des barres ne représente plus une quantité. À déclarer dans le graphique, sinon le lecteur additionne des hauteurs qui ne s'additionnent pas.

13. Mini-projet M10.P1 — « la grille en 18 points, version 1 » (1 h)

Énoncé. La grille du §5.6 est votre outil de relecture pour tout le module. Première version, à produire maintenant, en trois temps :

1. **Écrivez les 18 critères** sous forme de questions fermées, un point chacun, rangés dans les quatre familles (4 + 5 + 5 + 4 points). Chaque critère doit être vérifiable **par une preuve** : « axe : part-il de zéro, ou la troncature est-elle déclarée ? » et non « l'axe est-il bien ? ».
2. **Appliquez-la au panneau 1 de la planche du chapitre** (les huit points sur un axe) : 18 réponses, chaque réponse accompagnée de sa preuve (« source : absente du panneau », « ordre : trié par valeur croissante, oui »). Comptez le total.
3. **Appliquez-la au graphique que vous avez analysé en E2** et comparez les deux totaux. L'écart entre les deux totaux est votre **premier indicateur de qualité** : il mesure votre propre exigence.

Barème indicatif : grille écrite et cohérente (6 points), application complète au panneau 1 avec preuves (6 points), application à E2 et comparaison argumentée des totaux (6 points), sur 18.

Pourquoi ce mini-projet. La grille sera réutilisée trois fois dans le manuel : au projet M10 (« La refonte »), en M14 (Power BI), en M17 (automatisation) et dans la mission finale M21. La première version n'est pas la bonne : elle est la **vôtre**, écrite avant que le module ne vous donne ses exemples. Vous la réviserez en C04 (titres et axes), en C05 (les 10 erreurs) et en C07 (support et export) — c'est exactement ce que fait une équipe : une grille de relecture vit avec ses erreurs.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise **6 gestes** : tracer un nuage (`scatter`), tracer des barres horizontales (`barh`), tracer un camembert (`pie`), convertir une donnée en coordonnées de figure (`ax.transData.transform`), échantillonner une rampe de couleur (`plt.get_cmap`, `Normalize`) et mesurer une différence de clarté en CIE L* (`sRGB -> Lab`). Les trois premiers dessinent, les trois derniers **mesurent** : c'est la nouveauté du module.

Besoin	Commande	Ce qu'elle donne	Ce qu'elle ne donne pas
position exacte d'un point	<code>ax.transData.transform((x, y))</code>	la position en pixels de figure	une mesure en millimètres : il reste à convertir par le dpi
longueur d'un écart	différence de deux positions transformées	l'écart en millimètres, mesuré	l'erreur de lecture du lecteur (à prendre dans les expériences de perception)
couleur d'une valeur	<code>plt.get_cmap("Blues")</code> <code>(plt.Normalize(0, max)(v))</code>	une teinte, donc un triplet sRGB	la clarté : il faut convertir en Lab

Besoin	Commande	Ce qu'elle donne	Ce qu'elle ne donne pas
clarté d'une teinte	conversion sRGB -> CIE L*	l'écart ΔL^* entre deux teintes	la lisibilité pour un daltonien (c'est C03)
aire honnête	<code>s = (sqrt(v / vmax) * k) ** 2</code>	des surfaces proportionnelles	rien : sans cette formule, l'aire ment
ordre des catégories	<code>part.sort_values()</code>	un axe trié	le bon ordre métier (parfois l'ordre des classes)

15. Résumé du chapitre

M10 part d'une contrainte mesurable : **l'œil ne lit pas toutes les comparaisons avec la même précision**. Ce chapitre a établi cette contrainte de trois façons. Par la **littérature** : la hiérarchie des tâches de perception graphique classe la position avant la longueur, la longueur avant l'angle et la pente, ceux-ci avant l'aire, l'intensité de couleur fermant la marche ; l'erreur des jugements de longueur dépasse celle des jugements de position de 40 % à 250 %, l'angle la double à peu près, et le camembert n'a battu les barres que dans 3 cas sur 40. Par la **mesure**, sur les 8 parts du fil rouge : l'écart minimal entre deux catégories vaut 0,05 point, soit **0,4 mm** sur un axe large comme la page, **0,17°** en angle, **0,32 %** de surface en aire et **0,36 L*** en couleur — quatre impossibilités ; alors que l'étendue des parts (6,78 points, un rapport de 1,77 pour 1, 57,6 mm sur l'axe) reste lisible dans toutes les familles. Par la **prescription** : la chaîne question, tâche, encodage, outil — dans cet ordre — et sa première formalisation, la **grille de conception en 18 points** (4 + 5 + 5 + 4) qui servira de fil de relecture à tout le module.

16. À retenir

À retenir. Trois phrases résument le chapitre : l'œil est une machine précise en **position** et approximative en **angle, aire et couleur** ; un même écart de données change de taille apparente quand on change de famille de codage ; la question se pose donc **avant** l'encodage, et l'encodage avant l'outil. Le reste du module n'est que l'application de ces trois phrases.

1. **La hiérarchie est un résultat, pas une opinion** : position, longueur, angle et pente, aire, puis intensité de couleur ; longueur = 40 % à 250 % d'erreur en plus que position ; angle $\approx$ 2 fois.
2. **Un écart se mesure dans son canal** : 0,05 point = 0,4 mm = 0,17° = 0,32 % de surface = 0,36 L* — quatre façons de dire « illisible ».
3. **La couleur est un appui, pas un support** : elle désigne, elle ne quantifie pas.
4. **L'aire se code par la surface**, jamais par le rayon ($s = (\sqrt{(v / vmax) \times k})^2$).
5. **Chaque graphique a une question servie, et une seule** ; un titre qui contient « et » cache souvent deux questions dans un cadre.
6. **La hiérarchie ne bannit rien** : trois parts en camembert sont parfaites, huit parts en camembert pour un classement ne le sont pas — c'est la **question**, pas la famille, qui est fautive.
7. **La chaîne de décision est ordonnée** : question, tâche, encodage, outil. Un outil ouvert avant la question produit une valeur par défaut, c'est-à-dire la décision d'un autre.
8. **La grille en 18 points** (décision servie 4, justesse 5, lisibilité 5, utilisation 4) est la première version d'un outil qui vivra jusqu'à M21.

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi les jugements de position sont-ils les plus précis, et qu'en fait-on ?** → Parce que comparer deux points sur un même axe est la tâche la moins coûteuse du système visuel (hiérarchie établie expérimentalement, répliquée en 2010) ; on l'utilise pour toutes les questions de **comparaison** : barres triées, points alignés, boîtes à moustaches. (2 pts)
2. **Un camembert à deux parts est-il une faute ? Et à huit parts ?** → Non pour deux parts : l'écart angulaire est énorme et le tout se lit bien. À huit parts, oui, si la question est un classement : $0,17^\circ$ séparent les deux parts voisines, l'œil ne classe pas. (2 pts)
3. **Un graphique en bulles code une valeur par le rayon du disque. Quelle est la conséquence chiffrée ?** → La surface varie comme le carré : un rapport de 1,5 entre deux valeurs paraît 2,25 en surface, soit une amplification de 50 % de l'écart apparent. Il faut coder par la **surface** : $s = (\sqrt{(v / v_{\max}) \times k})^2$. (2 pts)
4. **Votre direction demande « un graphique qui montre que les catégories sont déséquilibrées ». Quelle question écrivez-vous, et que publiez-vous ?** → « Quelle est l'amplitude entre les catégories ? » ; un graphique de comparaison (barres triées), avec l'étendue annotée (8,85 % à 15,63 %, rapport 1,77 pour 1) — et **jamais** un classement fin que les données ne portent pas. (2 pts)
5. **Pourquoi la chaîne question, tâche, encodage, outil doit-elle être parcourue dans cet ordre ?** → Parce que l'outil ne connaît pas la question : interrogé en premier, il répond par une valeur par défaut (camembert, ordre alphabétique, échelle automatique), c'est-à-dire par le choix d'un autre. (2 pts)
6. **Citez deux points de la grille en 18 points qui se corrigent avant de dessiner, et deux qui se corrigent après.** → Avant : la question écrite et le titre qui affirme (famille 1) ; l'axe à zéro et l'aire proportionnelle (famille 2). Après : l'annotation du chiffre clé et la présence de la source (famille 3) ; la version « direction » et l'export réutilisable (famille 4). (2 pts)

Module M10.C02 — Le tableau question → graphique : les 16 graphiques du manuel et la carte de définition des métriques

Outils : matplotlib 3.10.9, pandas 2.2.3, numpy 2.3.5 (exécutés dans ce chapitre) ; seaborn 0.13.2 (annoncé, C03 et C07) ; Excel (exécuté, C07) ; Power BI (cité, non exécuté, C07). **Durée indicative :** 6 h. **Niveau :** N3. **Prérequis :** M09 (la question avant le chiffre) et M10.C01 (l'ordre perceptuel : position, longueur, angle, aire, intensité).

L'idée du chapitre. C01 a donné la contrainte — l'œil lit mieux une position qu'une surface. C02 donne le **catalogue** : seize graphiques, chacun rattaché à **une** question qu'il sert seul, avec son encodage, son piège et un repère chiffré du fil rouge. Le catalogue n'est pas une liste de dessins à connaître : c'est un **arbre de décision** — trois questions (comparer, évoluer, distribuer), quatre familles de réponses, seize outils — et une règle d'entrée : un graphique qui n'est pas choisi par sa question est un graphique choisi par défaut. Le chapitre se termine par l'artefact signature du module, la **carte de définition des métriques** : huit indicateurs du fil rouge, écrits avec leur formule, leur unité, leur source, leur propriétaire et leur piège de lecture — parce qu'un indicateur sans définition est un chiffre qui circulera faux.

Matériel de l'atelier — Python 3.13 · matplotlib 3.10.9 · pandas 2.2.3 · numpy 2.3.5. Les seize graphiques du chapitre ont été **dessinés et exécutés** à l'atelier le 20/09/2026 sur le socle M07/M08/M09 (les **50 008** ventes de la quincaillerie, les 5 magasins, les 3 villes, le fichier de projet M09.P pour l'entonnoir). La planche `figures/M10_C02_seize_graphiques.svg` est produite par `tools/seize_graphiques_M10.py` : deux exécutions donnent le **même fichier**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Parcourir l'arbre de choix** : formuler la question (comparer / évoluer / distribuer / relier / localiser), en déduire la tâche de lecture, puis choisir le graphique — et dire pourquoi l'ordre inverse donne un graphique par défaut.
2. **Citer les 16 graphiques du manuel** avec, pour chacun, sa question servie, son encodage dominant, son piège et **un chiffre du socle** qui l'illustre.
3. **Reconnaître les cinq graphiques « utiles »** que la liste fondatrice n'avait pas (aires empilées, treemap, jauge à puces, petits multiples, diagramme de flux) et dire ce qu'ils apportent et ce qu'ils coûtent.
4. **Repérer les cas où le graphique ment sans être faux** : camembert surchargé, cascade sur des flux qui ne s'additionnent pas, graphique combiné à deux axes — trois configurations où les chiffres restent exacts et la lecture devient fausse.
5. **Trancher entre un tableau et un graphique** : dire dans quel cas le tableau **gagne** (valeurs exactes, peu de lignes, plusieurs unités) et dans quel cas il perd.
6. **Remplir une carte de définition des métriques** sur le fil rouge : nom, formule, unité, source, fréquence, propriétaire, piège de lecture — le document qui empêche deux services de calculer deux fois le même indicateur.

2. Pourquoi cette notion est importante

Tout le monde sait faire un graphique ; presque personne ne sait **lequel**. C'est le constat qui ouvre le module : les outils proposent une vingtaine de types de graphiques dans une liste déroulante, sans dire lequel répond à quoi. Résultat : les rapports sont pleins de graphiques techniquement corrects et de questions restées sans réponse.

L'enjeu de ce chapitre est donc un **vocabulaire partagé**. Quand un collègue dit « on met un camembert », la réponse du module n'est pas « non » : c'est « quelle question ? combien de parts ? ». Quand un autre propose « une cascade pour montrer l'écart de chiffre d'affaires », il faut pouvoir répondre que la cascade exige des flux qui **s'additionnent** : ici les retours s'ajoutent (31 939 568 FCFA de retours expliquent l'écart entre le CA brut et le CA net), mais un taux de croissance ne s'additionne pas. Le catalogue transforme un débat de goût en vérification.

Deuxième enjeu, plus insidieux : le **graphique qui ment sans être faux**. Aucun chiffre n'y est erroné, aucune règle du module n'est violée, et pourtant le lecteur repart avec une conclusion que les données ne portent pas. Ces cas ne se corrigent pas en changeant les couleurs : ils se corrigent en reconnaissant le motif — et c'est pourquoi ils ont leur propre section (§5.4).

Troisième enjeu : le **temps**. Chaque graphique coûte à produire (dix minutes) et coûte à lire (cinq secondes). Un rapport de douze graphiques demande deux heures de production et une minute de lecture ; si trois d'entre eux seulement portent une décision, les neuf autres sont un impôt. La carte de définition des métriques (§5.6) joue le même rôle sur les indicateurs : elle rend visible ce qui est mesuré, par qui, avec quelle formule — et donc ce qui ne l'est pas.

Dans les faits. Le rapport de la quincaillerie fourni avec le module contient **cinq** graphiques, dont **deux** seulement répondent à une question identifiable. Les trois autres (camembert à huit parts, cumul de pourcentages à 100 %, échelle logarithmique non déclarée) ne laissent aucune conclusion vérifiable au lecteur — c'est précisément l'objet du projet « La refonte » et des défauts mesurés en C05.

3. Explication simple — la pharmacie des seize tiroirs

Imaginez une pharmacie bien tenue. Seize tiroirs, une étiquette par tiroir, et sur chaque étiquette non pas le nom du produit mais **le symptôme qu'il soigne**. On n'y entre pas par curiosité : on y entre avec une question.

1. **Le patient arrive avec sa phrase.** « Comment se comparent mes cinq magasins ? » — pas « je veux un graphique en barres ».
2. **Le pharmacien traduit en symptôme.** Comparer des grandeurs, suivre une évolution, montrer une composition, relier deux mesures, situer géographiquement : cinq symptômes, et à chacun son tiroir.
3. **Il ouvre un seul tiroir.** Comparer cinq magasins : le tiroir « barres triées ». Suivre l'évolution du chiffre d'affaires : le tiroir « courbe ». Montrer que trois villes portent le chiffre d'affaires : le tiroir « composition » — et là, attention au nombre de parts, donc au choix entre barres empilées et camembert.
4. **Il referme les autres.** Un rapport qui ouvre cinq tiroirs pour la même question n'en soigne aucune : il décore.

La règle de la pharmacie tient en une phrase : **le tiroir suit le symptôme, jamais le symptôme le tiroir**. Le symptôme de la quincaillerie est simple : trois villes à 40,2 %, 40,0 % et 19,8 % du chiffre d'affaires, huit catégories entre 697,2 M et 1 231,0 M FCFA, un chiffre d'affaires mensuel entre 307,6 M et 348,8 M. Trois symptômes, trois tiroirs, et aucun autre.

4. Vocabulaire essentiel

Terme	Définition
question servie	la question à laquelle un graphique répond seul , telle qu'un lecteur la formule en le regardant. Un graphique a une question servie, pas cinq.
arbre de choix	la démarche du chapitre : question → tâche de lecture → famille de graphiques → graphique précis → outil.
graphique combiné — combo chart	deux graphiques superposés sur un même cadre (des barres et une courbe, par exemple), souvent accompagnés de deux axes .
double axe	deux échelles verticales différentes sur un même cadre, à gauche et à droite. Les écarts entre les deux séries deviennent alors une illusion : c'est le fabricant de corrélation du module (condamné en C05, mesuré sur le fil rouge à +0,35).
cascade — waterfall	graphique qui décompose un total par ajouts et retraits successifs (CA brut → retours → CA net). N'a de sens que sur des flux additifs .
entonnoir — funnel	graphique de perte par étapes (lignes reçues → références uniques → montants valides → succès confirmés). Chaque étage montre ce qui survit au précédent.
treemap	partition rectangulaire : la surface de chaque rectangle est proportionnelle à sa valeur. Excellent pour « où est la masse ? », mauvais pour comparer deux rectangles proches.
jauge à puces — bullet chart	une mesure, une cible et des seuils sur la même ligne horizontale. Le remplaçant sobre de la jauge circulaire.
petits multiples — small multiples	la même figure répétée sur plusieurs facettes (un magasin par facette) avec une échelle commune : la forme se compare, la lecture reste locale.
diagramme de flux — Sankey	bandes dont la largeur est proportionnelle à un flux, du départ à l'arrivée (les trois villes → ce qui est conservé, ce qui est retourné).
carte de définition des métriques — metrics definition sheet	le document qui définit chaque indicateur du tableau de bord : nom, formule, unité, source, fréquence, propriétaire, piège de lecture. Artefact signature du module.
tableau de bord	un ensemble de graphiques organisés pour piloter (suivre un petit nombre d'indicateurs dans le temps), par opposition à une planche d'analyse qui sert à comprendre une question.

5. Cours approfondi

5.1 L'arbre de choix : cinq symptômes, quatre familles

Toute question métier se range dans **cinq symptômes**, et l'ordre compte : c'est lui qui décide du graphique.

Symptôme (la question posée)	Tâche de lecture	Familles de graphiques	Ce qu'il faut éviter
Comparer — « lequel est le plus grand ? »	comparer des positions ou des longueurs	barres, barres groupées, boîtes à moustaches, carte de chaleur, tableau	le camembert (angle) au-delà de 3 parts ; l'ordre alphabétique
Évaluer — « est-ce que ça monte ? »	suivre une position dans le temps	courbe, aires empilées, barres par période, petits multiples	le double axe qui fait coïncider deux séries sans rapport
Distribuer — « comment est-ce réparti ? »	estimer une part d'un tout	barres empilées à 100 %, camembert (≤ 3 parts), treemap, entonnoir	la part de parts inférieure à 5 % (illisible)
Relier — « ces deux mesures vont-elles ensemble ? »	estimer une relation	nuage de points, ligne de tendance	confondre corrélation et causalité ; un nuage sur 50 000 points non échantillonné
Localiser — « où ? »	situer et comparer	carte à symboles, carte de chaleur géographique	la bulle dont le rayon code la valeur (l'aire ment)

Deux remarques de méthode. D'abord, **les cinq symptômes ne sont pas exclusifs mais hiérarchisés** : un graphique peut comparer **et** localiser (les barres par ville), mais un seul symptôme doit commander — celui qui est écrit dans le titre. Ensuite, **la famille ne décide pas tout** : dans la famille « distribuer », le choix entre camembert et barres empilées se fait sur le **nombre de parts** (jusqu'à trois, un camembert est parfaitement lisible ; au-delà, non — C01 l'a mesuré à 0,17°).

Définition. L'**arbre de choix** est une procédure en cinq étapes, parcourue **avant** d'ouvrir l'outil : (1) écrire la question, (2) nommer le symptôme, (3) choisir la famille, (4) choisir le graphique précis, (5) choisir l'outil et le support. Les étapes 1 et 2 se font sur papier ; les étapes 3 à 5 au clavier. Un graphique dont les étapes 1 et 2 ne sont pas écrites est un graphique par défaut, même s'il est joli.

5.2 Le catalogue : les 16 graphiques, ligne à ligne

Voici le tableau de référence du manuel. La dernière colonne donne **le chiffre du socle** que le graphique affiche dans la planche du chapitre : chaque ligne a donc été **dessinée**, pas seulement décrite.

Graphique	Question servie	Encodage dominant	Piège principal	Repère sur le socle (exécuté)
1. Histogramme	« comment se distribuent les montants ? »	position (abscisses) + fréquence	des classes trop fines ou trop larges ; un axe vertical logarithmique non déclaré	médiane 119 482 FCFA, queue jusqu'à 594 363 ; 77,0 % des ventes sous 250 000
2. Barres (horizontales, triées)	« lequel est le plus grand ? »	longueur depuis zéro	l'ordre alphabétique ; un axe qui ne part pas de zéro	8 catégories de 697,2 M à 1 231,0 M FCFA
3. Courbe	« est-ce que ça monte ? »	position dans le temps	l'axe tronqué (C01) ; trop de séries	CA net mensuel de 307,6 M (2026-12) à 348,8 M (2026-06)
4. Camembert	« quelle part du tout ? »	angle	au-delà de 3 parts : illisible (écart minimal 0,05 pt → 0,17°)	8 parts de 8,85 % à 15,63 % — contre-exemple pédagogique
5. Nuage de points	« ces deux mesures vont-elles ensemble ? »	position sur deux axes	50 008 points superposés ; lire une corrélation sur un nuage convexe	échantillon de 2 000 ventes (graine 7) : quantité × montant
6. Boîte à moustaches	« où est le cœur, où sont les extrêmes ? »	position + longueur des moustaches	des boîtes sur des populations comparables	5 magasins : M5 (210 m²) contre M3 (520 m²)
7. Carte de chaleur	« où se concentre l'activité ? »	intensité (se lit par ligne)	comparer deux cases non voisines ; pas de valeur absolue	8 catégories × 24 mois de CA net
8. Carte symboles	« où, combien ? »	position géographique + surface	le rayon qui code la valeur (amplifie l'écart) ; pas d'échelle	3 villes, 5 magasins : 40,2 % / 40,0 % / 19,8 % du CA
9. Cascade	« qu'est-ce qui explique l'écart ? »	longueur + cumul	+ l'appliquer à des flux non additifs (un taux, une part)	CA brut 7 908 259 732 → retours 31 939 568 → CA net 7 876 320 164
10. Entonnoir	« où perd-on ? »	longueur + effet d'escalier	des étapes qui ne se suivent pas dans le temps	10 014 lignes → 8 109 succès confirmés (81,0 %)
11. Combiné à deux axes	« deux séries bougent-elles ensemble ? »	position, échelles	deux fabrique la coïncidence : corrélation du fil rouge +0,35 alors que les séries n'ont pas de lien	CA net (barres) × retours (courbe)

Graphique	Question servie	Encodage dominant	Piège principal	Repère sur le socle (exécuté)
12. Aires empilées	« la composition évolue-t-elle ? »	position + surface cumulée	ne se lit que la série du bas ; étiquettes impossibles	8 catégories cumulées, 24 mois ; 2025 : 3 910,9 M , 2026 : 3 965,4 M
13. Treemap	« où est la masse ? »	surface	comparer deux rectangles voisins ; surfaces trompeuses au 2e niveau	8 catégories, de 8,85 % à 15,63 %
14. Jauge à puces	« sommes-nous dans la cible ? »	longueur + repère de cible	la jauge circulaire (angle) ; cibles non datées	CA net 2026 contre la cible 2025 (3 910,9 M)
15. Petits multiples	« la même forme vaut-elle pour tous ? »	position, échelles communes	des échelles différentes par facette (on ne compare plus rien)	5 magasins, 24 mois, échelle identique
16. Diagramme de flux — Sankey	« d'où vient quoi ? »	largeur de bande	plus de 5-6 flux : illisible ; des bandes qui se croisent	3 villes → CA conservé / retours

Trois lectures de ce tableau, avant d'aller plus loin.

Première lecture : chaque piège est un problème de question, pas de dessin. L'axe tronqué (ligne 3) n'est pas une faute de logiciel : c'est l'absence de la phrase « l'axe ne part pas de zéro ». Le camembert à huit parts (ligne 4) n'est pas laid : il répond à une question que personne n'a posée (« classez huit parts »). Le double axe (ligne 11) n'est pas illégal : il répond à deux questions à la fois, donc à aucune.

Deuxième lecture : les graphiques « de composition » sont les plus dangereux (lignes 4, 12, 13, 16). Ils séduisent parce qu'ils montrent « tout d'un coup » — et c'est là qu'ils trompent : le camembert et le treemap codent par angle et surface (les canaux les moins précis, C01), les aires empilées ne laissent lire proprement que la série du bas, le diagramme de flux devient un labyrinthe au-delà de quelques bandes. La barre empilée à 100 % — à trois ou quatre parts — reste la valeur sûre de cette famille.

Troisième lecture : les cinq derniers outils servent à répondre vite à une question étroite. Treemap, jauge à puces, petits multiples, aires empilées et diagramme de flux ne remplacent jamais un graphique de comparaison : ils répondent à « où est la masse ? », « sommes-nous dans la cible ? », « la forme est-elle la même partout ? », « la composition bouge-t-elle ? » et « d'où vient quoi ? ». Employés pour ces questions-là, ils sont excellents ; employés ailleurs, ils décoorent.

Conseil professionnel. Quand une demande arrive sous la forme « fais-moi un treemap », le réflexe professionnel est de répondre par une **question** : « qu'est-ce qui doit ressortir — quelle catégorie pèse le plus, ou comment la composition évolue-t-elle ? ». La première réponse appelle une treemap (ou des barres triées, souvent plus lisibles), la seconde des aires empilées ou des petits multiples. Le demandeur n'a pas tort : il a simplement parlé outil.

5.3 Les cinq graphiques que la liste fondatrice n'avait pas

Les onze premiers graphiques du catalogue sont ceux que tout le monde connaît. Les cinq suivants méritent une justification, parce qu'ils ne sont pas des variantes décoratives :

- Aires empilées (12)** — la seule famille qui montre **à la fois** une composition et son évolution. Coût : seule la série du bas est lisible avec précision ; les autres ne se comparent qu'entre elles, et de près. À réserver à 3 ou 4 séries, ou à accompagner de petits multiples.
- Treemap (13)** — une réponse directe à « où est la masse ? ». Sa force est la **partition** (la somme des rectangles vaut le tout, sans reste), sa faiblesse est la comparaison fine : deux rectangles de même hauteur se comparent bien, deux rectangles de formes différentes mal.

Utile pour un portefeuille de 30 produits, inutile pour 8 catégories que des barres triées départagent mieux.

3. **Jauge à puces (14)** — l'outil du **pilotage** : une mesure, une cible, des seuils, sur une ligne. C'est le graphique à employer dans un tableau de bord pour la question « sommes-nous dans la cible ? », là où la jauge circulaire classique (un angle sur 180°) sacrifie la précision inutilement.
4. **Petits multiples (15)** — la réponse à « la même forme vaut-elle pour tous ? ». Leur règle est absolue : **échelles identiques** sur toutes les facettes, sinon la comparaison disparaît. C'est le meilleur outil pour détecter le magasin qui décroche.
5. **Diagramme de flux (16)** — le seul graphique qui montre un **passage** (d'où vient quoi, où va quoi) : villes → chiffre d'affaires conservé / retours. Au-delà de cinq ou six flux, il faut le remplacer par deux graphiques de comparaison.

Attention. Ces cinq outils sont souvent adoptés pour leur **nouveauté** — un rapport avec une treemap et un diagramme de flux « fait moderne ». C'est exactement le travers que le module combat : un treemap qui remplace des barres triées fait perdre de la précision (surface contre longueur) sans rien apporter. La règle : ces graphiques **entrent par la question**, jamais par l'envie de varier.

5.4 Le graphique qui ment sans être faux

Trois configurations produisent une lecture fautive avec des chiffres exacts. Elles sont au programme du projet, parce qu'elles ne se détectent pas en vérifiant les totaux : elles se détectent en relisant le **codage**.

A. Le camembert surchargé. Huit parts, et l'écart minimal entre deux voisines vaut 0,05 point de pourcentage, soit 0,17° (C01). Le graphique n'affirme rien de faux ; il laisse simplement croire qu'un classement est possible. La correction ne se joue pas sur la palette : elle change de graphique (barres triées) ou de question (montrer l'étendue : 8,85 % à 15,63 %).

B. La cascade sur des flux non additifs. Une cascade raconte « ceci plus cela donne cela ». Elle est légitime sur les retours (31 939 568 FCFA de retours expliquent l'écart entre 7 908 259 732 et 7 876 320 164 FCFA de CA) et absurde sur des taux ou des parts : on n'additionne pas des pourcentages qui portent sur des bases différentes. Le test est simple : **est-ce que les valeurs s'additionnent ?** Si la réponse est non, la cascade est une décoration qui invite à un calcul faux.

Définition. On appelle **corrélation affichée** la valeur du coefficient de corrélation entre deux séries **telles que le graphique les présente** — c'est-à-dire après le choix des deux échelles. Elle peut être très différente de la corrélation réelle des données, parce que le double axe laisse choisir les bornes de la seconde échelle. Règle du module : dès qu'un graphique à deux axes est publié, la corrélation affichée s'écrit dans le graphique, avec la mention « sans lien métier établi » si c'est le cas.

C. Le combiné à deux axes. Deux séries, deux échelles : le graphique peut faire coïncider n'importe quoi. Sur le fil rouge, le chiffre d'affaires et le nombre de retours affichent une corrélation de +0,35 — un lien faible — mais surtout **sans intérêt métier** : les deux séries n'ont aucune raison de varier ensemble. Le double axe rend cette coïncidence spectaculaire en choisissant les bornes de la seconde échelle. C'est le défaut que C05 mesurera : le titre du rapport (« les retours suivent le chiffre d'affaires ») est une conclusion **fabriquée par le graphique**.

Attention. Ces trois cas partagent une propriété : ils sont **invisibles à l'audit des données**. Les totaux sont justes, les périmètres sont respectés, aucune valeur n'est manquante. Une revue de qualité qui ne regarde que les chiffres laisse passer les trois. D'où la grille de conception de C01 : les familles 1 et 2 ne se vérifient pas sur les données, mais sur le **graphique lui-même**.

5.5 Quand un tableau bat un graphique

Le module défend les graphiques ; il serait malhonnête de ne pas dire où ils perdent. Un tableau gagne dans **quatre** situations :

1. **La valeur exacte est le message.** Un budget à respecter, un seuil réglementaire, un prix : « 119 482 FCFA » s'écrit, il ne se lit pas sur un axe.
2. **Peu de lignes et plusieurs unités.** Cinq magasins avec surface, effectif, chiffre d'affaires et marge : quatre unités différentes (m², personnes, FCFA, %) ne partagent aucune échelle — un graphique devrait les normaliser, donc perdre l'unité.
3. **Le lecteur va recalculer.** Un tableau de bord budgétaire se recompose (colonnes, sous-totaux) : le tableau est un **outil** pour le lecteur, le graphique une conclusion.
4. **L'exhaustivité est requise.** Un inventaire, un journal d'anomalies, une liste de contrôles : ici, une ligne manquante serait une faute ; un graphique ne peut pas être exhaustif.

Inversement, le tableau perd dès que la question porte sur une **forme** : une tendance sur 24 mois, une distribution, une concentration. Personne ne voit une courbe dans une colonne de 24 nombres. La règle pratique du chapitre : **le graphique pour comprendre, le tableau pour agir** — et, dans un bon livrable, les deux : la figure porte le constat, le tableau annexé porte les valeurs.

Définition. On appelle **graphique-tableau** (ou tableau enrichi) un tableau dont les cellules portent un codage visuel : barres intégrées, échelle de couleurs, flèches de variation. Il combine les deux forces — la valeur exacte et la forme — et c'est l'outil à privilégier quand le lecteur a besoin des deux à la fois (un tableau de bord de suivi, par exemple).

5.6 La carte de définition des métriques : l'artefact signature

Voici le document qui manque dans presque toutes les organisations, et qui explique la plupart des désaccords en réunion : **la définition écrite des indicateurs**. Le principe est simple : chaque indicateur publié a une ligne, et cette ligne répond à sept questions. Le tableau ci-dessous montre les six premières colonnes (la fréquence est portée avec la source pour rester lisible sur une page) sur les **8 indicateurs** du fil rouge :

Indicateur	Formule	Unité	Source (fréquence)	Propriétaire	Piège de lecture
CA net	somme des montants hors retours	FCFA	vente.csv (mensuel)	direction commerciale	le comparer au CA brut : l'écart vaut 31 939 568 FCFA
Panier moyen	CA net ÷ nombre de ventes	FCFA	vente.csv (mensuel)	direction commerciale	moyenne 158 140 contre médiane 119 482 : la queue tire la moyenne
Médiane des montants	médiane des montants TTC	FCFA	vente.csv (mensuel)	analyse	l'annoncer sans l'effectif : 1 044 ventes dépassent 500 000
Taux de retour	retours ÷ ventes	%	vente.csv (mensuel)	service qualité	un mois à 4 retours et un mois à 16 ne disent pas la même chose

Indicateur	Formule	Unité	Source (fréquence)	Propriétaire	Piège de lecture
Part de catégorie	CA de la catégorie ÷ CA net	%	vente.csv produit.csv (mensuel)	+ direction commerciale	l'écart minimal entre deux catégories vaut 0,05 point : aucun classement fin
CA par m²	CA net ÷ surface du magasin	FCFA/m²	vente.csv magasin.csv (trimestriel)	+ direction réseau	des surfaces inégales (210 à 520 m²) rendent la comparaison brute trompeuse
Part du top 10 clients	CA des 10 premiers clients ÷ CA net	%	vente.csv (trimestriel)	direction commerciale	le décompte des clients change le résultat ; le préciser
Taux de succès (projet)	statuts de succès normalisés ÷ lignes valides	%	fichier_inconnu.csv (ponctuel)	analyse	six graphies de statut avant normalisation : 81,0 % après nettoyage

Définition. Une **mesure** est un nombre calculé sur des données (« 7 876 320 164 FCFA de CA net en 24 mois »). Un **indicateur** est une mesure **reliée à un objectif**, à une fréquence et à un propriétaire (« le CA net mensuel, cible de croissance, propriétaire : direction commerciale »). La carte de définition ne sert pas à calculer : elle sert à **s'accorder** — c'est pourquoi elle porte une colonne source, une colonne fréquence et une colonne propriétaire.

Trois règles de rédaction, apprises des désaccords les plus fréquents :

1. **Une formule, pas un mot.** « Chiffre d'affaires » ne définit rien : « somme des montants TTC hors retours » se vérifie. La formule doit tenir en une ligne et se transcrire en une requête.
2. **Un piège par ligne, obligatoire.** La colonne la plus utile est la dernière : elle dit ce que l'indicateur **ne** dit pas. C'est l'héritage direct de M09 (la limite se déclare).
3. **Un propriétaire par ligne.** Sans propriétaire, personne ne corrige la définition quand le métier change — et deux services finissent par publier deux chiffres différents du même nom.

Dans les faits. Les huit indicateurs ci-dessus sont tous calculés sur le **même** jeu de données, par le même analyste, dans la même journée — et ils ne concordent pas naturellement : le CA net (7 876 320 164 FCFA) et le CA brut (7 908 259 732 FCFA) diffèrent de 0,4 %, le panier moyen (158 140 FCFA) dépasse la médiane (119 482 FCFA) de près d'un tiers. Deux services qui publieraient « le chiffre d'affaires » et « le panier moyen » sans ces précisions publieraient deux chiffres différents — et l'un des deux serait accusé d'erreur à tort.

5.7 Ce que le catalogue ne dit pas

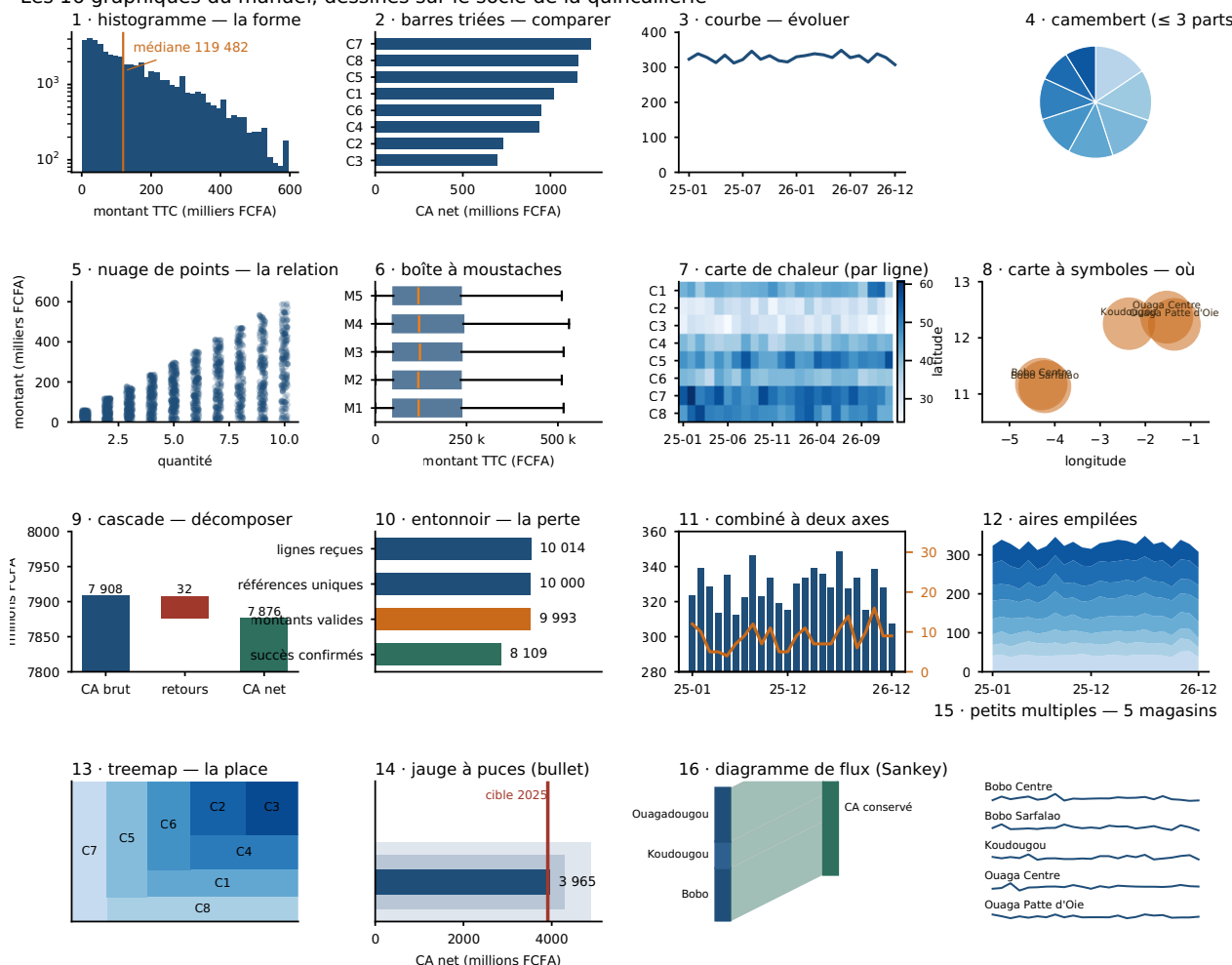
Trois limites, à garder en tête pour ne pas transformer le tableau du §5.2 en dogme :

1. **Le catalogue est culturel et évolutif.** Les seize types retenus sont ceux que les lecteurs de ce manuel rencontreront en entreprise ; d'autres familles existent (graphiques en chandeliers, courbes de durée, diagrammes de cordes) et de nouveaux apparaissent. Ce qui ne change pas, c'est l'arbre de choix et la hiérarchie de précision : eux servent à évaluer n'importe quel graphique nouveau.
2. **Le catalogue ne remplace pas la connaissance du métier.** Deux questions identiques en apparence peuvent appeler deux graphiques : « comparer les magasins » ne se fait pas de la même façon selon qu'on compare des surfaces de vente comparables ou des zones de chalandise différentes.
3. **Le catalogue ne dit rien du support.** Une carte de chaleur lisible sur un écran peut être illisible projetée ; des petits multiples tiennent sur une page mais pas sur un écran de téléphone. C'est l'objet de C07 (support, résolution, export).

6. Exemple concret — la planche des seize, dessinée sur le fil rouge

Voici la planche produite à l'atelier : les seize graphiques du catalogue, **tous** dessinés à partir du même socle — les ventes de la quincaillerie — et portant le chiffre que le tableau §5.2 leur associe.

Les 16 graphiques du manuel, dessinés sur le socle de la quincaillerie



Ce que la planche démontre, et pourquoi elle a été dessinée plutôt que décrite :

- **Le même jeu de données donne seize lectures différentes.** Le chiffre d'affaires de la quincaillerie est une donnée unique ; selon le graphique, on y lit une distribution (1), un classement de catégories (2), une évolution plate (3), une répartition en huit parts (4), une relation quantité-montant (5), des dispersions par magasin (6), une concentration mois $\times$ catégorie (7), une géographie (8), une décomposition (9), un entonnoir de qualité (10), une coïncidence fabriquée (11), une composition (12), une masse (13), un écart à la cible (14), cinq trajectoires comparables (15) et un passage (16). Ce n'est pas la donnée qui change : c'est la question.
- **Chaque graphique a un coût de lecture visible.** Le panneau 4 (camembert à huit parts) est immédiatement moins informatif que le panneau 2 (barres triées) qui porte la même donnée ; le panneau 11 (combiné à deux axes) suggère une relation que le panneau 5 (nuage) invite à vérifier.
- **Le socle suffit à tout illustrer.** Aucun des seize panneaux n'a nécessité de donnée nouvelle : les 50 008 ventes, les 5 magasins, les 3 villes, les 8 catégories et le fichier de projet M09.P couvrent le catalogue entier. C'est aussi la leçon du chapitre : le choix du graphique ne se paie pas en données supplémentaires, il se paie en réflexion.

Trois chiffres pour fixer les idées, tous mesurés sur la planche :

Ce que montre la planche	Le chiffre	Où le lire
Un classement de catégories utilisable	de 697,2 M à 1 231,0 M FCFA, soit un rapport de 1,77	panneau 2 (barres) contre panneau 4 (camembert)
Une évolution remarquablement plate	de 307,6 M à 348,8 M FCFA sur 24 mois ($\pm$ 6,3 % autour de 328,2 M)	panneaux 3 et 15 (courbe, petits multiples)
Une géographie concentrée	40,2 % / 40,0 % / 19,8 % du CA net entre trois villes	panneaux 8 et 16 (carte, diagramme de flux)

7. Démonstration pas à pas — trois questions, trois graphiques

La démonstration rejoue l'arbre de choix sur trois questions réelles du fil rouge, dans l'ordre : **écrire la question, nommer le symptôme, choisir le graphique, produire le chiffre qui décide**. Les sorties sont celles de l'atelier.

Question A — « Comment se comparent les huit catégories ? » Symptôme : comparer. Le chiffre qui décide n'est pas la moyenne mais l'étendue et la part du haut de tableau.

```
import pandas as pd
v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv", parse_dates=["date_vente"])
p = pd.read_csv("03_exercices/dossier_M09/quincaillerie/produit.csv")
net = v[~v["est_retour"]].merge(p[["id_produit", "id_categorie"]], on="id_produit", how="left")
cat = net.groupby("id_categorie")["montant_ttc"].sum().sort_values(ascending=False)
print([f"C{i} {x/1e6:.1f} M" for i, x in cat.head(3).items()])
print([f"C{i} {x/1e6:.1f} M" for i, x in cat.tail(3).items()])
print("part du top 3 :", round(cat.head(3).sum() / cat.sum() * 100, 1), "%")
```

```
['C7 1231.0 M', 'C8 1158.5 M', 'C5 1154.8 M']
['C4 937.3 M', 'C2 729.2 M', 'C3 697.2 M']
part du top 3 : 45.0 %
```

Choix : barres horizontales triées (panneau 2). **Pourquoi pas** un camembert : huit parts, écart minimal de 0,05 point. **Pourquoi pas** une treemap : elle montrerait la même masse, en surface, sans permettre de lire les trois dernières catégories. **Phrase du livrable** : « Trois catégories portent 45,0 % du chiffre d'affaires ; la plus forte (1 231,0 M FCFA) dépasse la plus faible (697,2 M FCFA) de 77 %. »

Question B — « Le chiffre d'affaires progresse-t-il ? » Symptôme : évoluer. Le chiffre qui décide est la dispersion autour de la tendance, pas le total.

```
ca = net.groupby(net["date_vente"].dt.to_period("M"))["montant_ttc"].sum()
print(len(ca), "mois |", ca.idxmin(), round(ca.min()/1e6, 1), "M |",
      ca.idxmax(), round(ca.max()/1e6, 1), "M | moyenne", round(ca.mean()/1e6, 1), "M")
print("ecart du pic a la moyenne :", round((ca.max()-ca.mean())/ca.mean()*100, 1), "%")
tri = net.groupby(net["date_vente"].dt.to_period("Q"))["montant_ttc"].sum()
print("trimestres :", len(tri), "| premier", round(tri.iloc[0]/1e6, 1), "M | dernier",
      round(tri.iloc[-1]/1e6, 1), "M")
```

```
24 mois | 2026-12 307.6 M | 2026-06 348.8 M | moyenne 328.2 M
ecart du pic a la moyenne : 6.3 %
trimestres : 8 | premier 990.7 M | dernier 974.2 M
```

Choix : une courbe (panneau 3), complétée par des petits multiples par magasin (panneau 15) si la question devient « cette platitude vaut-elle pour tous ? ». **Pourquoi la moyenne glissante** : elle sépare le bruit mensuel (l'écart-type du socle est de 10,8 M) de la tendance. **Phrase du livrable** : « Sur 24 mois, le chiffre d'affaires net oscille entre 307,6 et 348,8 M FCFA, avec un pic

à +6,3 % au-dessus de la moyenne et un premier trimestre (990,7 M) supérieur au dernier (974,2 M) : la période se lit comme **plate**, pas comme croissante. »

Question C — « D’où vient le chiffre d’affaires ? » Symptôme : localiser + distribuer. Attention au choix du graphique : trois villes, donc un camembert serait **légitime** ; mais la question suivante (« et les retours, d’où viennent-ils ? ») demande un passage, donc un diagramme de flux.

```
m = pd.read_csv("03_exercices/dossier_M09/quincaillerie/magasin.csv")
vil = net.merge(m[["id_magasin", "ville"]], on="id_magasin", how="left").groupby("ville")
["montant_ttc"].sum()
print({k: f"{x/1e6:.1f} M ({x/vil.sum()*100:.1f} %)" for k, x in vil.items()})
```

```
{'Bobo-Dioulasso': '3150.8 M (40.0 %)', 'Koudougou': '1562.6 M (19.8 %)', 'Ouagadougou': '3162.9 M (40.2 %)'}
```

Choix : deux barres horizontales triées (ou un camembert à trois parts, l’un et l’autre se défendent) **plus** un diagramme de flux si l’on ajoute les retours (panneau 16). **Phrase du livrable** : « Le chiffre d’affaires net se partage en deux pôles quasi égaux (Ouagadougou 40,2 %, Bobo-Dioulasso 40,0 %) et un pôle secondaire (Koudougou 19,8 %) : la lecture utile n’est pas le classement des deux premiers — ils sont à égalité — mais l’écart au troisième. »

À retenir. Les trois démonstrations suivent la même chaîne : une phrase de question, un symptôme nommé, un graphique choisi, **un chiffre qui décide** extrait du socle. C’est cette dernière étape qui distingue un graphique professionnel d’une illustration : il porte le chiffre sur lequel la décision va se prendre.

8. Erreurs fréquentes

1. **Commencer par le type de graphique.** « Je vais faire un combiné » est une phrase d’outil ; la seule phrase d’entrée valable est une question. Le symptôme et la famille viennent ensuite.
2. **Multiplier les graphiques pour la même question.** Deux graphiques qui répondent à la même question ne se renforcent pas : ils se contredisent dès que les périmètres diffèrent. Un rapport de douze graphiques doit porter douze questions (ou moins).
3. **Oublier la dernière colonne du catalogue.** Le piège est écrit sur chaque ligne du tableau §5.2 : c’est la colonne qui a le plus de valeur professionnelle, parce qu’elle se révèle dans la réunion, quand quelqu’un tire une conclusion trop forte.
4. **Prendre un tableau pour un pis-aller.** Le tableau gagne quatre fois (§5.5) : valeurs exactes, unités multiples, recalcul par le lecteur, exhaustivité. Le remplacer par un graphique dans ces quatre cas dégrade l’information.
5. **Publier un indicateur sans définition.** « Le panier moyen » de deux services peut désigner deux indicateurs différents (avec ou sans retours, avec ou sans remises). Sans carte des métriques, le désaccord est garanti — et il n’est pas mesurable.
6. **Croire que le catalogue règle tout.** Un graphique bien choisi sur une donnée mal comprise reste faux : l’arbre de choix suppose la fiche d’entrée de M09 faite, les unités vérifiées et les défauts du jeu identifiés.
7. **Employer les cinq outils « modernes » pour varier.** Treemap, diagramme de flux et jauge à puces sont excellents pour leurs questions et coûteux ailleurs : ils font perdre la comparaison fine que des barres donnaient gratuitement.

9. Bonnes pratiques professionnelles

1. **Écrire la question sur le brouillon** avant de choisir le graphique, et la garder pour rédiger le titre (qui doit l'affirmer, cf. C04).
2. **Nommer le symptôme à voix haute** (« c'est une comparaison ») : si vous hésitez entre deux symptômes, c'est que le graphique doit être scindé en deux.
3. **Vérifier l'additivité avant toute cascade** ; vérifier le nombre de parts avant tout camembert (≤ 3) ; vérifier la raison métier avant tout double axe (le plus souvent, il n'y en a pas).
4. **Annoter le chiffre qui décide** : l'étendue, la part du haut de tableau, l'écart à la cible. Un graphique dont on ne sait pas dire le chiffre clé n'est pas fini.
5. **Tenir la carte des métriques à jour**, avec un propriétaire par indicateur et un piège par ligne ; la diffuser avec les graphiques qu'elle définit.
6. **Limiter le nombre de séries** : au-delà de cinq, passer aux petits multiples. C'est la même contrainte que la surcharge du dossier de refonte.
7. **Relire avec la grille en 18 points** (C01) avant publication : la famille 1 vérifie la question servie, la famille 3 l'annotation et la source — les deux points qui manquent le plus souvent dans les rapports réels.

10. Exercice guidé — « la demande de la direction » (15 min, /10)

Contexte. La direction écrit : « Préparer un graphique sur les catégories et un autre sur l'évolution, pour le comité de vendredi ». Vous disposez du socle de la quincaillerie (8 catégories, 24 mois, 5 magasins, 3 villes).

Consigne. Dans les 15 minutes :

1. Écrivez les **deux questions** que recouvrent les deux demandes, en une phrase chacune, avec leur destinataire. (2 pts)
2. Nommez le **symptôme** de chacune et la **famille** de graphiques correspondante. (2 pts)
3. Choisissez le graphique précis pour chacune, en justifiant par la question — et dites **ce que vous écarteriez** et pourquoi. (2 pts)
4. Donnez pour chaque graphique le **chiffre qui décide**, calculé sur le socle. (2 pts)
5. Rédigez le **titre** de chaque graphique (titre qui affirme, cf. C04) et la **phrase de conclusion** du comité. (2 pts)

11. Exercices autonomes

E1 — le catalogue contre votre propre rapport (30 min). Prenez un rapport que vous avez produit ou reçu (au moins cinq graphiques). Pour chacun : écrivez la question servie, nommez le symptôme, identifiez la ligne du tableau §5.2 dont il se rapproche, et **mesurez le piège** s'il tombe dans l'un des trois cas de §5.4 (écart minimal illisible, flux non additif, double axe). Rendez un tableau à cinq colonnes et une conclusion : combien de graphiques sur cinq répondaient à une question identifiable ? La réponse est l'indicateur de qualité de votre rapport, pas le nombre de graphiques.

E2 — la carte des métriques de votre activité (30 min). Choisissez **cinq** indicateurs que vous suivez ou publiez. Remplissez la carte de définition (nom, formule, unité, source, fréquence, propriétaire, piège de lecture) sans regarder le §5.6. Puis comparez votre carte à celle du fil rouge et notez trois différences : ce que vous avez oublié (souvent le piège ou la fréquence), ce que vous avez précisé mieux, et l'indicateur dont la formule vous a demandé un arbitrage.

12. Correction détaillée

Exercice guidé.

1. **Les deux questions** : « Quelles catégories portent le chiffre d'affaires, et de combien la première dépasse-t-elle la dernière ? » (destinataire : le comité, décision de réassort) ; « Le chiffre d'affaires progresse-t-il, ou la période est-elle plate ? » (destinataire : le comité, décision d'objectif annuel).
2. **Symptômes** : comparer (catégories) et évoluer (24 mois). Familles : barres triées pour la première ; courbe (et petits multiples si l'on veut vérifier la platitude magasin par magasin) pour la seconde.
3. **Choix et rejets** : barres horizontales triées — on écarte le camembert (8 parts, écart minimal de 0,05 point) et la treemap (comparaison de surfaces) ; courbe mensuelle — on écarte le combiné à deux axes (les retours ne sont pas la question, et l'axe double fabrique une corrélation de +0,35) et les barres par mois (24 barres = 24 comparaisons inutiles là où la forme suffit).
4. **Chiffres qui décident** : catégories — de 697,2 M à 1 231,0 M FCFA, top 3 = 45,0 %, rapport 1,77 entre extrêmes ; évolution — de 307,6 M à 348,8 M, moyenne 328,2 M, pic à +6,3 %, premier trimestre 990,7 M contre 974,2 M pour le dernier.
5. **Titres attendus** (qui affirment) : « Trois catégories sur huit portent 45 % du chiffre d'affaires » et « Le chiffre d'affaires est plat depuis 24 mois : 6 % d'amplitude autour de 328 M FCFA ». **Phrase de conclusion** : « La structure de l'offre explique l'essentiel du niveau ; la période n'apporte aucune tendance — l'objectif annuel se fixe sur la structure, pas sur la croissance passée. »

E1 (le catalogue contre un rapport réel). Grille de correction :

- la **question servie** est écrite au présent et peut se vérifier (elle nomme une comparaison, une évolution ou une composition) — une réponse du type « montrer le chiffre d'affaires » est refusée ;
- le **piège** est mesuré, pas nommé : pour un camembert, l'écart minimal en points et sa conversion en degrés ou en millimètres ; pour un double axe, la corrélation affichée ; pour une cascade, le test d'additivité (les valeurs s'additionnent-elles ?) ;
- la **conclusion** est un compte : sur cinq graphiques, combien répondaient à une question identifiable. À titre de repère, le rapport de la quincaillerie qui sert de dossier est à 2 sur 5.

E2 (la carte des métriques). Trois attendus :

- **les sept colonnes remplies** pour les cinq indicateurs, la colonne « piège de lecture » étant éliminatoire : une carte sans piège n'est pas une carte, c'est un dictionnaire ;
- **la formule transcrite en opération** (« CA net ÷ nombre de ventes », et non « panier ») ;
- **l'arbitrage identifié** : l'indicateur dont la définition hésite (avec ou sans retours, avec ou sans remises) est celui à faire valider par le métier — c'est précisément le rôle du propriétaire.

13. Mini-projet M10.P2 — « la carte des métriques du fil rouge » (1 h)

Énoncé. Vous êtes l'analyste de la quincaillerie. Le comité de direction a demandé un tableau de bord mensuel — et le directeur commercial a prévenu : « la dernière fois, deux services ont an-

noncé deux chiffres d'affaires différents ». Vous produisez donc le document qui **précède** les graphiques : la carte de définition des métriques.

1. **Écrivez les 8 indicateurs** du §5.6 avec leurs sept colonnes (nom, formule, unité, source, fréquence, propriétaire, piège de lecture). Les formules doivent être transcrites en opérations sur les colonnes du socle (le fichier `vente.csv` et ses tables jointes).
2. **Ajoutez trois indicateurs de votre choix** (par exemple : taux de rupture par produit, part des ventes du lundi, nombre de jours sous le seuil de stock) — avec leur piège, en citant le chiffre du socle qui les rend vérifiables.
3. **Produisez pour chaque indicateur une phrase de définition** utilisable en réunion : une phrase qu'un non-analyste peut répéter sans se tromper.
4. **Concluez** : parmi vos 11 indicateurs, lesquels sont **comparables** entre magasins ou entre périodes, et lesquels ne le sont pas ? Cette conclusion décide quels graphiques du catalogue pourront être publiés, et lesquels seraient trompeurs.

Barème indicatif : 8 indicateurs complets (7 points), 3 indicateurs supplémentaires avec piège mesuré (5 points), 11 phrases de définition utilisables (3 points), conclusion sur la comparabilité (3 points), sur 18.

Pourquoi ce mini-projet. La carte des métriques est l'artefact que le module réutilise en M14 (tableau de bord Power BI), en M17 (automatisation du rafraîchissement) et dans la mission finale M21. C'est aussi la meilleure préparation au projet « La refonte » : on ne refond pas un graphique dont on ne sait pas définir l'indicateur.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise **5 gestes** de `matplotlib`, tous exécutés dans la planche du §6 : `imshow` (carte de chaleur), `stackplot` (aires empilées), `boxplot` (boîtes à moustaches), `twinx` (second axe du graphique combiné) et la construction de polygones (`Polygon`) pour les bandes du diagramme de flux. Les autres panneaux réutilisent trois gestes de C01 : `barh`, `scatter`, `plot`.

Besoin	Commande	Ce qu'elle donne	Ce qu'elle ne donne pas
carte de chaleur	<code>ax.imshow(matrice, cmap="Blues", aspect="auto")</code>	une intensité par case	la valeur exacte : elle se lit par ligne, ou en annexe
aires empilées	<code>ax.stackplot(x, series)</code>	une composition cumulée	la lecture des séries du milieu
boîtes	<code>ax.boxplot(liste, vert=False, showliers=False)</code>	médiane, quartiles, moustaches	la forme de la distribution (un histogramme la donne)
deux axes	<code>ax2 = ax.twinx()</code>	une seconde échelle	une comparaison honnête : c'est un piège, pas un outil
flux	<code>Polygon([...])</code> avec largeurs proportionnelles	un passage d'un point à un autre	une lecture chiffrée : les bandes se comparent mal

15. Résumé du chapitre

C02 a donné au module son **vocabulaire de travail** : un arbre de choix en cinq symptômes (comparer, évoluer, distribuer, relier, localiser), un catalogue de **16 graphiques** — les onze de la liste fondatrice et les cinq outils utiles (aires empilées, treemap, jauge à puces, petits multiples, diagramme de flux) — et, pour chaque ligne du catalogue, une question servie, un encodage dominant, un piège et un chiffre du socle : 8 catégories de 697,2 M à 1 231,0 M FCFA, un chiffre

d'affaires mensuel de 307,6 M à 348,8 M, 45,0 % du chiffre d'affaires sur les trois premières catégories, une géographie à 40,2 % / 40,0 % / 19,8 %, une cascade de 7 908 259 732 à 7 876 320 164 FCFA, un entonnoir de 10 014 lignes à 81,0 % de succès. Le chapitre a isolé les **trois mensonges sans chiffre faux** (camembert surchargé, cascade sur flux non additifs, double axe fabricant de corrélation), rappelé les quatre cas où **le tableau gagne**, et livré l'artefact signature du module : la **carte de définition des métriques**, huit indicateurs du fil rouge avec formule, unité, source, fréquence, propriétaire et piège de lecture.

16. À retenir

À retenir. Le catalogue des seize n'est pas une liste à mémoriser : c'est un **arbre**. On y entre par la question, on traverse un symptôme, on choisit une famille, puis un graphique — et le chiffre qui décide s'annote sur la figure. Tout le reste — jolis dégradés, effets de mode, double axe spectaculaire — n'est pas du graphique, c'est du décor.

1. **Cinq symptômes, quatre familles, seize outils** : comparer, évoluer, distribuer, relier, localiser.
2. **Une question servie par graphique**, écrite dans le titre ; un titre qui contient « et » annonce deux questions dans un cadre.
3. **Trois mensonges sans chiffre faux** : camembert surchargé (écart minimal 0,05 point), cascade sur flux non additifs, double axe (corrélation affichée +0,35 pour des séries sans lien).
4. **Le tableau gagne quatre fois** : valeur exacte, unités multiples, recalcul, exhaustivité.
5. **Les cinq outils utiles entrent par la question**, jamais pour varier : aires empilées (3-4 séries), treemap (masse), jauge à puces (cible), petits multiples (échelles communes), diagramme de flux (5-6 bandes au plus).
6. **Chaque graphique porte le chiffre qui décide** : étendue, part du haut de tableau, écart à la cible. Sans ce chiffre, le graphique n'est pas terminé.
7. **La carte des métriques a sept colonnes**, dont la plus utile est la dernière : le piège de lecture. Un indicateur sans définition circule faux.
8. **Le catalogue ne dispense pas de la fiche d'entrée** (M09) : un bon graphique sur une donnée mal comprise reste une erreur mieux présentée.

17. Évaluation formative (auto-correction, 8 min)

1. **Un demandeur écrit « je veux une treemap ». Quelle est la bonne réponse professionnelle ?** → Demander la question : « qu'est-ce qui doit ressortir — la masse, ou une comparaison ? ». Si c'est une comparaison de 8 catégories, des barres triées font mieux (longueur contre surface, C01) ; si c'est la masse d'un portefeuille de 30 produits, la treemap est le bon outil. (2 pts)
2. **Pourquoi une cascade est-elle illégitime sur un taux de croissance, et légitime sur les retours ?** → Parce qu'elle exige des **flux additifs** : les retours (31 939 568 FCFA) s'ajoutent au CA net pour donner le CA brut (7 908 259 732 FCFA) ; un taux de croissance se compose (multiplication), il ne s'additionne pas — une cascade y ferait faire un calcul faux au lecteur. (2 pts)
3. **Le rapport fourni titre « les retours suivent le chiffre d'affaires ». Qu'en pensez-vous ?** → La corrélation mesurée est de +0,35, faible et surtout **sans raison métier** ; le double axe rend la coïncidence visuelle en choisissant les bornes de la seconde échelle. Le titre est une conclusion fabriquée par le graphique : c'est le défaut D2 du dossier, mesuré en C05. (2 pts)

4. **Citez deux cas où un tableau bat un graphique, et deux où l'inverse est vrai.** → Tableau : la valeur exacte est le message (119 482 FCFA) ; plusieurs unités cohabitent (m², effectif, FCFA) — ou encore : le lecteur va recalculer, l'exhaustivité est requise. Graphique : une tendance sur 24 mois, une distribution — aucune forme ne se lit dans une colonne de nombres. (2 pts)
5. **Quelles colonnes de la carte des métriques empêchent deux services de publier deux chiffres différents ?** → La **formule** (avec ou sans retours : 7 908 259 732 contre 7 876 320 164 FCFA), la **source** (quelle table, quelle fréquence) et le **propriétaire** (celui qui arbitre quand le métier change). Le piège de lecture protège le lecteur, les trois premières protègent l'organisation. (2 pts)
6. **Combien de séries au maximum dans des aires empilées, et pourquoi ?** → Trois ou quatre : seule la série du bas se lit contre un axe commun ; les autres ne se comparent qu'entre elles, par l'épaisseur de bande (encodage de surface, donc peu précis). Au-delà, on passe aux petits multiples. (2 pts)

Module M10.C03 — Couleurs, palettes, accessibilité et culture locale

Outils : matplotlib 3.10.9, pandas 2.2.3, numpy 2.3.5 (exécutés dans ce chapitre) ; seaborn 0.13.2 (annoncé, C07) ; Excel (exécuté, C07) ; Power BI (cité, non exécuté, C07).
Durée indicative : 4 h. **Niveau :** N3. **Prérequis :** M10.C01 (l'ordre perceptuel : la couleur est le canal le moins précis) et M10.C02 (le catalogue des 16 graphiques).

L'idée du chapitre. C01 a classé la couleur en dernier dans la hiérarchie de précision ; ce chapitre en fait un **outil**, pas une décoration. Trois contraintes mesurables d'abord : le **contraste** (un texte lisible sur fond blanc demande un rapport de 4,5 pour 1 — notre gris de la maison est à 3,05), le **daltonisme** (environ 8 % des hommes ; le vert et le rouge de notre palette passent de 73,4 à 35,7 unités d'écart en deutéranopie, soit la moitié) et la **redondance** (couleur **et** forme **et** libellé — jamais la couleur seule). Puis les contraintes de terrain : la couleur qui signifie deux choses dans le même rapport, le format des montants en FCFA, la culture locale des couleurs et la lisibilité à l'écran comme au vidéoprojecteur. La sortie du chapitre est concrète : une **charte de graphique d'une page**, appliquée aux cinq graphiques du projet.

Matériel de l'atelier — Python 3.13 · matplotlib 3.10.9 · pandas 2.2.3 · numpy 2.3.5. Toutes les mesures de ce chapitre sont calculées par `tools/couleurs_M10.py` sur les couleurs réelles du manuel et sur le socle M09 (les **50 008** ventes de la quincaillerie) ; la planche `figures/M10_C03_palette_testee.svg` est écrite par le même script (deux exécutions = même fichier). Les simulations de daltonisme emploient la méthode linéaire de Viénot, Brettel et Mollon (1999), couramment utilisée pour la vérification d'accessibilité.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Distinguer les trois familles de palettes** — catégorielle, séquentielle, divergente — et les apparier à la nature de la variable (sans ordre, ordonnée, ordonnée autour d'un centre).
2. **Mesurer un contraste** : calculer le rapport de contraste d'une couleur contre le blanc, savoir que le seuil d'un texte est de 4,5 pour 1 (3 pour 1 pour un élément graphique), et citer les deux couleurs de la palette de la maison qui échouent.
3. **Simuler le daltonisme** sur une palette (protanopie, deutéranopie, tritanopie), mesurer l'écart résiduel entre deux couleurs avec la distance Lab, et dire quelles paires deviennent risquées — chez nous, la paire vert et rouge, dont l'écart tombe de 73,4 à 35,7.
4. **Employer la redondance** comme règle d'or : couleur **et** forme **et** libellé pour distinguer les séries ; la couleur ne porte jamais seule une information.
5. **Écrire les nombres d'un graphique en FCFA** : séparateur de milliers, devise, décimales qu'on ne publie pas (98,53 % de nos montants portent des centimes, et les arrondir déplace le total de 12 481,98 FCFA, soit 0,0002 % — le choix est donc libre, mais il se déclare).
6. **Rédiger une charte de graphique d'une page** — palette, contraste, redondance, formats, ton — et l'appliquer aux cinq graphiques du projet.

2. Pourquoi cette notion est importante

La couleur est le canal le plus employé et le plus mal employé. Trois raisons à cela : elle est gratuite dans les logiciels (une liste déroulante), elle est immédiatement visible (donc rassurante

pour l’auteur) et elle **ne se vérifie pas à l’œil** — celui qui choisit les couleurs est rarement celui qui les subit. Un rapport lu sur l’écran du concepteur, en pleine lumière, dans une pièce calme, peut être illisible projeté dans une salle éclairée, imprimé en noir et blanc ou lu par un daltonien.

Les chiffres du chapitre donnent la mesure du problème. Notre palette de la maison est pourtant soignée : le bleu principal atteint un rapport de contraste de 8,66 contre le blanc et le vert 5,91. Mais l’orange de la maison est à 3,78 et le gris à 3,05 : **ni l’un ni l’autre ne porte un texte**, même s’ils restent acceptables comme éléments graphiques (seuil de 3 pour 1). Un jaune vif, que beaucoup de rapports emploient pour « faire ressortir » une série, tombe à 1,43 — quasiment invisible sur un fond blanc et totalement effacé par un vidéoprojecteur.

Le daltonisme, ensuite. Sa fréquence est d’environ 8 % chez les hommes ; dans un comité de douze personnes, la probabilité qu’au moins un participant voie la palette autrement est supérieure à une chance sur deux. Or notre paire la plus « parlante » — le vert pour « cible atteinte », le rouge pour « retour » — est exactement la paire qui s’effondre : 35,7 unités d’écart après simulation, la moitié de l’écart d’origine, les deux couleurs virant vers le même olive. Le graphique n’est pas faux : il devient muet pour une partie de l’audience. C’est le même motif que le camembert surchargé de C02 — l’information existe, elle n’arrive pas.

Dans les faits. Le rapport de la quincaillerie fourni avec le module emploie **trois** couleurs (bleu, orange, gris) sur cinq graphiques — et l’orange y joue **3 rôles distincts** en 7 emplois : la moyenne de référence sur un cadre, la série du dessus d’un double axe sur un autre, les retours sur un troisième. Un lecteur qui suit le rapport d’un bout à l’autre ne peut pas mémoriser ce que l’orange signifie : la couleur n’y porte aucun contrat. C’est l’un des défauts que la refonte devra corriger — et il ne se voit pas dans les totaux.

3. Explication simple — le code de la route des couleurs

Une palette de graphique n’est pas une garde-robe, c’est un **code de la route**. Quatre panneaux, et chacun dit une seule chose, toujours la même.

1. **Le panneau « catégories »** (palette catégorielle) : des couleurs qui ne se hiérarchisent pas, pour des groupes sans ordre — magasins, canaux, régions. Règle : au-delà de cinq ou six groupes, on arrête la couleur et on passe aux libellés.
2. **Le panneau « échelle »** (palette séquentielle) : une seule teinte, du clair au foncé, pour une variable ordonnée — montant, intensité, densité. C’est la palette de la carte de chaleur, et c’est aussi la plus robuste au daltonisme (une seule teinte, donc un ordre qui survit aux simulations).
3. **Le panneau « écart »** (palette divergente) : deux teintes opposées autour d’un centre neutre, pour un écart à une référence — au-dessus et en dessous de la moyenne, marge positive et négative. Règle absolue : le centre est **donnée**, pas esthétique (le zéro, la moyenne, la cible).
4. **Le panneau « personne »** (redondance) : à chaque fois qu’une couleur distingue deux séries, un second signe doit le faire aussi — trait plein contre pointillé, cercle contre carré, étiquette en clair. C’est le panneau qui sauve le rapport quand la couleur échoue.

Le code tient en une phrase : **la couleur ne transporte jamais seule une information** ; elle double un signe qui, lui, est lisible par tous.

4. Vocabulaire essentiel

Terme	Définition
palette catégorielle	un jeu de couleurs de teintes distinctes pour des groupes sans ordre (canaux, magasins, catégories).
palette séquentielle	une progression d'une seule teinte, du clair au foncé, pour une variable ordonnée (montant, densité, intensité).
palette divergente	deux teintes opposées séparées par un centre neutre, pour un écart à une référence (marge, variation, écart à la cible).
rapport de contraste	le rapport de luminance entre une couleur et son fond (norme WCAG) : 4,5 pour 1 pour un texte, 3 pour 1 pour un élément graphique.
distance Lab	l'écart perceptuel entre deux couleurs, en unités ΔE^* : sous 10, deux couleurs sont difficiles à distinguer sur un graphique.
daltonisme — color vision deficiency	une vision des couleurs altérée, qui touche une majorité des personnes concernées sur l'axe vert-rouge (protanopie, deutéranopie) et une minorité sur l'axe bleu-jaune (tritanopie).
redondance	le fait de coder une même distinction par plusieurs signes (couleur et forme et libellé), pour que l'information survive à la perte d'un canal.
charte de graphique — chart style	les règles d'une maison (palette, contrastes, formats, titres, sources) appliquées à tous ses graphiques : le document d'une page qui rend deux rapports reconnaissables comme venant de la même organisation.

5. Cours approfondi

5.1 Les trois familles de palettes et leur règle d'appariement

Le choix d'une palette se déduit de la **nature de la variable**, pas du goût de l'auteur :

Nature de la variable	Palette	Exemple sur le fil rouge	Palette de l'atelier
groupes sans ordre (2 à 6)	catégorielle	les 5 magasins, les 3 villes, les canaux de paiement	bleu, orange, gris, vert, rouge
grandeur ordonnée	séquentielle	CA par catégorie d'une carte de chaleur, densité de ventes	rampe Blues du catalogue
écart à une référence	divergente	écart au panier moyen, variation trimestrielle	rampe RdBu centrée sur zéro
une seule série	aucune couleur	courbe du CA, histogramme des montants	bleu unique + gris de contexte

Définition. La **palette de la maison** (ou palette de référence) est le jeu de couleurs figé par la charte, avec un rôle par couleur. Sa raison d'être n'est pas esthétique : c'est de rendre un lecteur capable de dire, sans lire la légende, ce que chaque couleur signifie — parce qu'elle signifie la même chose dans tous les graphiques de l'organisation.

Trois erreurs reviennent sans cesse. La première consiste à **colorer une série unique** : un histogramme bleu, dont les barres ont toutes la même couleur, n'ajoute aucune information en changeant de teinte d'une barre à l'autre — c'est de la décoration qui suggère une catégorie inexistante. La deuxième consiste à employer une palette **séquentielle sur une variable sans ordre** : colorer cinq magasins par une rampe de bleus suggère que le magasin 1 est « plus » que le magasin 5 ; le lecteur y verra une hiérarchie qui n'existe pas. La troisième consiste à employer une palette **divergente sans centre métier** : une échelle de rouge et de bleu centrée sur la moyenne arithmétique d'une série asymétrique place le neutre au mauvais endroit et colore de rouge des valeurs parfaitement normales.

Définition. Une palette est **appariée** quand sa famille (catégorielle, séquentielle, divergente) correspond à la structure de la variable affichée. Le test est unique : « si je remplace les couleurs par du gris, l'information centrale reste-t-elle lisible ? » Si oui, la couleur est un appui ; si non, elle porte trop.

5.2 Contraste : ce qui se lit et ce qui ne se lit pas

Le contraste se mesure : c'est le rapport entre la luminance relative de deux couleurs (la formule de la norme WCAG, implémentée dans le script du chapitre). Résultats mesurés sur notre palette, contre un fond blanc :

Couleur	Code	Contraste contre blanc	Contraste contre noir	Verdict
Bleu de la maison	#1f4e79	8,66	2,42	texte et graphique : parfait
Rouge de la maison	#a1382c	6,74	3,12	texte et graphique
Vert de la maison	#2e6f5e	5,91	3,55	texte et graphique
Orange de la maison	#c96a1a	3,78	5,56	graphique seulement (texte blanc déconseillé)
Gris de la maison	#8895a5	3,05	6,89	graphique seulement, jamais pour un texte
Jaune vif (à éviter)	#ffd400	1,43	—	invisible sur fond blanc

Trois conséquences pratiques. D'abord, **l'ordre de la palette est aussi un ordre de lisibilité** : le bleu et le rouge supportent un texte, l'orange et le gris non. Ensuite, **le fond compte autant que la couleur** : l'orange à 3,78 contre le blanc passe à 5,56 contre le noir — un rapport imprimé en négatif change la lisibilité de tous les textes. Enfin, **le seuil n'est pas le même pour un texte et pour une marque graphique** : une barre orange reste parfaitement visible (élément graphique, seuil de 3 pour 1), mais une étiquette écrite en orange sur blanc fatigue le lecteur.

Conseil professionnel. Le test du vidéoprojecteur est le plus impitoyable, et le plus simple à préparer : projetez votre graphique, reculez de trois mètres, et demandez à quelqu'un de lire l'étiquette la plus claire. Si elle n'est pas lisible, ce n'est pas la faute du projecteur : c'est que le graphique compte sur une nuance faible pour porter une information. La solution est toujours la même — assombrir la couleur ou écrire le chiffre.

5.3 Dal tonisme : simuler pour vérifier

On ne demande pas à l'auteur d'un graphique d'imaginer ce que voit un daltonien : on **simule**. Le script du chapitre applique trois matrices de simulation (protanopie, deutéranopie, tritanopie) à chaque couleur de la palette, puis mesure la distance Lab entre toutes les paires, avant et après.

Vision	Écart minimal entre deux couleurs de la palette	La paire critique	Paires sous 10 ΔE*
normale	32,1	orange et rouge	0
deutéranopie	23,3	gris et vert	0
protanopie	17,7	vert et rouge	0
tritanopie	11,2	bleu et vert	0

Bonne nouvelle : aucune paire de la palette ne franchit le seuil de $10 \Delta E^*$, y compris simulée. Le chapitre n'a donc pas à changer de palette — mais il doit retenir **deux faits chiffrés** qui décident de l'usage :

1. **La paire la plus fragile est le vert et le rouge** (cible contre retour), qui passe de $73,4 \Delta E^*$ en vision normale à **35,7** en deutéranopie : l'écart est divisé par deux et les deux couleurs se rapprochent d'un vert olive commun. Sur une carte de chaleur ou un graphique à deux séries où seul le rouge distingue un « retour », le message s'affaiblit sérieusement.
2. **La paire la plus robuste est la rampe séquentielle** : une seule teinte déclinée en clarté ($30 L^*$ d'amplitude) conserve son ordre après simulation — l'écart minimal entre deux cases voisines reste de $0,4 \Delta E^*$ en vision normale comme en deutéranopie, et l'ordre des cases ne change pas. Autrement dit : **quand une information doit être portée par la couleur, une rampe séquentielle est le choix sûr** ; quand il faut distinguer des groupes, la redondance (§5.4) est obligatoire.

Attention. L'écart minimal de $0,4 \Delta E^*$ entre deux cases voisines de la rampe n'est pas une bonne nouvelle : il signifie que deux catégories voisines (C5 et C8, séparées de 0,05 point) restent indiscernables à l'œil, daltonien ou non. La robustesse de la rampe au daltonisme ne crée pas de précision : elle ne fait que garantir que **l'ordre général** survit. Les deux chapitres se répondent : C01 mesure ce que l'œil peut lire, C03 mesure ce que la couleur ajoute — ou n'ajoute pas.

5.4 La redondance : couleur et forme et libellé

La redondance est la seule parade qui ne dépend ni de la vision, ni du support, ni de la qualité de l'impression. Elle s'applique par ordre de coût croissant :

1. **Le libellé en clair** (le moins cher) : écrire le nom de la série à côté de sa dernière valeur, au lieu de la laisser dans une légende détachée. Le lecteur n'a plus à faire l'aller-retour œil-légende-œil.
2. **La forme du trait ou du marqueur** : trait plein contre pointillé, cercle contre carré. Cinq styles de traits et quatre formes de marqueurs permettent de distinguer jusqu'à vingt séries sans aucune couleur — bien au-delà du besoin réel.
3. **La position** : les petits multiples (C02) remplacent la couleur par la facette : chaque série dans son cadre, avec un titre. C'est la redondance la plus robuste, et la plus coûteuse en place.
4. **L'annotation directe** : la flèche et le chiffre clé, qui désignent le point important sans passer par la couleur.

Définition. La **lisibilité en niveaux de gris** est la propriété d'un graphique qui reste entièrement compréhensible une fois la couleur retirée : chaque série garde un signe distinctif (libellé, forme, position). C'est la propriété qui décide si un rapport survit à une impression monochrome — et c'est le test le plus rapide à faire passer avant publication.

Le principe qui les rassemble : **la couleur est un canal d'appoint, jamais un canal porteur**. Dans la charte du chapitre, cela se traduit par une phrase à recopier : « toute série distinguée par une couleur l'est aussi par un libellé ou une forme ».

5.5 La couleur qui signifie deux choses

Passons aux défauts réels, ceux du rapport à refondre. Sur cinq graphiques, le rapport emploie trois couleurs — et l'orange y joue **3 rôles distincts** en 7 emplois : la moyenne de référence sur un cadre, la série du dessus d'un double axe sur un autre, les retours sur un troisième. Aucun de ces emplois n'est fautif en soi ; c'est leur **cumul** qui détruit le contrat de lecture. Le lecteur qui

apprend « orange = retours » au troisième cadre sera trompé au quatrième, où l'orange désigne une moyenne.

Deux règles de la charte en découlent :

1. **Un rôle par couleur, pour tout le document** (mieux : pour toute la série de rapports). Si l'orange signifie « retour », aucune autre série ne l'emploie — et l'alerte de seuil, elle, se signale par le rouge, qui ne sert à rien d'autre.
2. **Au-delà de trois couleurs, une légende n'est plus lue** : les rapports qui emploient six teintes pour six séries produisent des graphiques que le lecteur lit en noir et blanc, c'est-à-dire en positions et en libellés. Avoir six couleurs revient à en avoir zéro.

Dans les faits. Le test est facile à faire dans sa propre organisation : ouvrez les trois derniers rapports publiés et demandez à un collègue ce que signifie « le rouge » dans l'ensemble. S'il répond « ça dépend du graphique », la charte manque. C'est le défaut le plus répandu et le moins coûteux à corriger : une page, une fois.

5.6 FCFA et formats numériques : ce qu'on écrit, ce qu'on n'écrit pas

Définition. On appelle **précision affichée** le nombre de chiffres significatifs décidés à l'affichage, par opposition à la précision du calcul. Les deux n'ont pas à coïncider : un total calculé au centime se publie au franc ou au millier selon le destinataire — l'important est que le choix soit écrit dans la charte et appliqué partout, pour que deux rapports ne publient pas deux chiffres différents du même indicateur.

Un graphique de données financières doit trancher quatre questions de format, et les trancher **une fois pour toutes** :

Les milliers. Le socle écrit ses montants avec un espace insécable comme séparateur de milliers (le format international) : 7 876 320 164 FCFA se lit d'un coup d'œil, 7876320164 non. La règle de la charte : séparateur d'espace, jamais de virgule (une virgule de milliers est un point décimal pour la moitié du monde, et nos CSV contiennent déjà le point décimal).

La devise. Le franc CFA ne se dévalue pas sur la période du socle et ne se convertit pas : on écrit FCFA (ou XOF dans les fichiers techniques) et on ne met **jamais** de symbole « F ». Pour les grands montants, le passage au million ou au milliard s'écrit dans l'axe ou dans l'unité — « CA net (millions FCFA) » —, pas dans chaque étiquette.

Les décimales. Question politique, pas technique. Le socle porte des centimes : **98,53 %** des lignes ont un montant qui n'est pas un franc entier (49 275 lignes sur 50 008). Arrondir tout au franc déplace le total de **12 481,98 FCFA**, soit **0,0002 %** : c'est négligeable pour un rapport de direction, mais c'est un écart qu'un auditeur peut réclamer. La règle du module : on affiche les montants **arrondis** dans les graphiques et les tableaux de synthèse (zéro décimale), on **conserve** les centimes dans les fichiers de calcul et dans les rapports financiers, et la charte **dit** lequel des deux est employé.

Le zéro et les pourcentages. Un pourcentage publié sans son effectif est un piège (M09) : la charte impose, pour tout taux affiché seul, la mention de la base (« 0,4 % de retours, sur 50 008 ventes »). Et la décimale des pourcentages se limite à une : « 40,2 % », jamais « 40,2137 % » — la précision affichée doit correspondre à la précision de la mesure.

5.7 Culture locale, écran et vidéoprojecteur

Attention. Le seuil de $10 \Delta E^*$ servant ici à juger deux couleurs n'est pas une norme : c'est la valeur que le module retient comme limite pratique de distinction sur un graphique. Deux couleurs séparées de moins de $10 \Delta E^*$ restent visibles comme **deux** zones, mais le lecteur ne peut plus dire laquelle est laquelle sans se référer à la légende — c'est précisément ce que la redondance évite.

Trois contraintes de terrain, souvent oubliées dans les chartes :

1. **Le sens des couleurs dépend du contexte.** Le rouge signale une alerte dans un tableau de bord financier ; il signale une fête, une réussite ou un prix dans d'autres contextes. Le vert, qui porte « cible atteinte » dans ce manuel, n'a cette valeur que parce que la charte le **déclare**. La règle : les couleurs d'état (rouge, vert, orange) se définissent dans la charte, avec leur signification écrite — et elles ne servent à rien d'autre (pas de série « verte » décorative).
2. **L'écran n'est pas la salle.** Un graphique préparé sur un écran de bureau est vu, en réunion, sur un vidéoprojecteur qui éclaire les teintes claires et écrase les contrastes. Les remèdes sont mesurables : lignes plus épaisses (2 points au lieu de 1), étiquettes plus grandes (au moins 10 points), et jamais d'information portée par une nuance faible.
3. **L'impression noir et blanc.** Tout rapport peut finir imprimé en noir et blanc, et beaucoup de lecteurs impriment par défaut. Le test de la charte est donc explicite : **imprimez le graphique en niveaux de gris** ; si deux séries deviennent identiques, la redondance manque. C'est le test qui justifie à lui seul le §5.4.

À retenir. Ces trois contraintes ne sont pas des préférences esthétiques : ce sont des pertes d'information, mesurables et prévisibles. La charte est le document qui les traite **avant** la production, plutôt que de les découvrir en réunion.

6. Exemple concret — la palette du manuel passée au banc d'essai

La planche du chapitre montre la palette de la maison dans trois situations : telle qu'elle est conçue, telle qu'elle est vue par une deutéranopie, telle qu'elle est vue par une protanopie — puis la rampe séquentielle du catalogue, dans les deux mêmes conditions.

La palette du manuel testée : contraste, daltonisme, rampe séquentielle

1 · la palette de la maison (contraste WCAG contre le blanc)

#1f4e79 8.7:1	#c96a1a 3.8:1	#8895a5 3.0:1	#2e6f5e 5.9:1	#a1382c 6.7:1
------------------	------------------	------------------	------------------	------------------

2 · la même palette vue par une deutéranopie ($\approx 8\%$ des hommes)

#1f4e79 CA	#c96a1a alerte	#8895a5 contexte	#2e6f5e cible	#a1382c retour
---------------	-------------------	---------------------	------------------	-------------------

3 · la même palette vue par une protanopie (le rouge s'assombrit, le vert s'éclaircit)

#1f4e79 CA	#c96a1a alerte	#8895a5 contexte	#2e6f5e cible	#a1382c retour
---------------	-------------------	---------------------	------------------	-------------------

4 · rampe séquentielle Blues : 8 catégories, écarts de clarté (vision normale)

C3 73 L*	C2 71 L*	C4 60 L*	C6 60 L*	C1 55 L*	C5 48 L*	C8 47 L*	C7 43 L*
-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------

5 · la même rampe en deutéranopie : elle garde son ordre (une seule teinte)

C3 72 L*	C2 70 L*	C4 59 L*	C6 59 L*	C1 54 L*	C5 47 L*	C8 46 L*	C7 42 L*
-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------

Ce que la planche établit, bande par bande :

- **Bande 1 — la palette de la maison.** Cinq couleurs, chacune avec son rôle et son contraste mesuré contre le blanc : le bleu à 8,66 et le rouge à 6,74 portent un texte ; l'orange à 3,78 et le gris à 3,05 ne le portent pas.
- **Bandes 2 et 3 — les simulations.** Le vert « cible » (#2e6f5e) et le rouge « retour » (#a1382c) se rapprochent visiblement d'un olive commun : c'est la paire à risque, celle que §5.3 chiffre à 35,7 unités d'écart contre 73,4 en vision normale. Le bleu et le jaune, en revanche, restent nettement distincts dans les deux simulations.
- **Bandes 4 et 5 — la rampe séquentielle.** Les huit cases gardent leur **ordre** : les clartés vont de 73 à 43 L* en vision normale, et de 72 à 42 L* en deutéranopie — l'ordre est identique, seule la teinte glisse légèrement. C'est la démonstration que la carte de chaleur est la famille la plus sûre du catalogue pour un public large.

Le verdict pour la charte tient en trois lignes : **la palette est bonne** (aucune paire sous 10 ΔE^* , même simulée) ; **l'usage doit imposer la redondance** dès que le vert et le rouge portent deux séries voisines ; **la rampe séquentielle reste l'outil par défaut** quand la couleur doit porter une intensité.

7. Démonstration pas à pas — mesurer sa propre palette en 5 étapes

Étape 1 — le contraste. On calcule la luminance relative de chaque couleur, puis le rapport.

```
import numpy as np, matplotlib.pyplot as plt
def lin(c):
    return np.where(c <= 0.04045, c / 12.92, ((c + 0.055) / 1.055) ** 2.4)

def luminance(rgb):
    r, g, b = lin(np.asarray(rgb, dtype=float))
    return float(0.2126 * r + 0.7152 * g + 0.0722 * b)

def contraste(a, b):
    la, lb = luminance(a), luminance(b)
    return (max(la, lb) + 0.05) / (min(la, lb) + 0.05)

BLANC = np.array([1.0, 1.0, 1.0])
for nom, hx in (("bleu", "#1f4e79"), ("orange", "#c96a1a"), ("gris", "#8895a5")):
    rgb = np.array([int(hx[i:i+2], 16) / 255 for i in (1, 3, 5)])
    print(f"{nom:7s} contre blanc : {contraste(rgb, BLANC):.2f}:1")
```

```
bleu    contre blanc : 8.66:1
orange  contre blanc : 3.78:1
gris    contre blanc : 3.05:1
```

Étape 2 — la distance Lab. Deux couleurs « différentes » peuvent être perceptuellement proches : il faut la distance, pas la différence de code.

```
def lab(rgb):
    r, g, b = lin(np.asarray(rgb, dtype=float))
    x = r * 0.4124 + g * 0.3576 + b * 0.1805
    y = r * 0.2126 + g * 0.7152 + b * 0.0722
    z = r * 0.0193 + g * 0.1192 + b * 0.9505
    f = lambda t: t ** (1/3) if t > 0.008856 else 7.787 * t + 16/116
    fx, fy, fz = f(x / 0.95047), f(y), f(z / 1.08883)
    return np.array([116 * fy - 16, 500 * (fx - fy), 200 * (fy - fz)])

vert = np.array([0x2e, 0x6f, 0x5e]) / 255
rouge = np.array([0xa1, 0x38, 0x2c]) / 255
print("vert vs rouge :", round(float(np.linalg.norm(lab(vert) - lab(rouge))), 1), "dE")
```

```
vert vs rouge : 73.4 dE
```

Étape 3 — la simulation du daltonisme. La méthode linéaire transforme la couleur dans l'espace LMS (cônes), annule un axe, puis revient en sRGB.

```
RGB_VERS_LMS = np.array([[17.8824, 43.5161, 4.11935],
                        [3.45565, 27.1554, 3.86714],
                        [0.0299566, 0.184309, 1.46709]])
LMS_VERS_RGB = np.linalg.inv(RGB_VERS_LMS)
DEUTERANOPIE = np.array([[1.0, 0.0, 0.0], [0.494207, 0.0, 1.24827], [0.0, 0.0, 1.0]])

def gamma(c):
    c = np.clip(c, 0, 1)
    return np.where(c <= 0.0031308, c * 12.92, 1.055 * c ** (1/2.4) - 0.055)

def simuler(rgb):
    return gamma(LMS_VERS_RGB @ (DEUTERANOPIE @ (RGB_VERS_LMS @ lin(rgb))))

print("vert vs rouge, deutéranopie :",
      round(float(np.linalg.norm(lab(simuler(vert)) - lab(simuler(rouge)))), 1), "dE")
```

```
vert vs rouge, deutéranopie : 35.7 dE
```

Étape 4 — la rampe séquentielle. On vérifie que l'ordre survit : les clartés doivent rester monotones, en vision normale **et** simulée.

```

rampe = plt.get_cmap("Blues"); norme = plt.Normalize(0, 20)
parts = [8.85, 9.26, 11.90, 12.02, 12.97, 14.66, 14.71, 15.63]
couloir = [np.array(rampe(norme(x))[:3])) for x in parts]
print("clartes normales :", [round(float(lab(c)[0])) for c in couloir])
print("clartes simulées :", [round(float(lab(simuler(c))[0])) for c in couloir])

```

```

clartes normales : [73, 71, 60, 60, 55, 48, 47, 43]
clartes simulées : [72, 70, 59, 59, 54, 47, 46, 42]

```

Étape 5 — les formats. On mesure ce que coûte l'arrondi des montants, avant de trancher.

```

import pandas as pd
v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv")
m = v["montant_ttc"]
avec = int((np.round(m * 100) % 100 != 0).sum())
print("lignes avec centimes :", avec, "sur", len(m))
print("delta de l'arrondi au franc :", round(float(np.abs(np.round(m) - m).sum()), 2), "FCFA")

```

```

lignes avec centimes : 49275 sur 50008
delta de l'arrondi au franc : 12481.98 FCFA

```

Ce que la démonstration établit. Mesurer sa palette prend dix minutes et évite deux catégories de désastres : un texte illisible (contraste 3,05 pour le gris) et une information perdue pour une partie du public (la paire vert-rouge de 73,4 à 35,7). Et la mesure des formats rappelle qu'un choix d'affichage n'est jamais neutre : arrondir 98,53 % des lignes déplace le total de 12 481,98 FCFA — négligeable, mais à déclarer.

8. Erreurs fréquentes

1. **Colorer une série unique.** Un histogramme multicolore suggère des catégories qui n'existent pas ; la couleur doit correspondre à une variable, pas à une envie.
2. **Employer une rampe pour des groupes.** Des magasins colorés en dégradé de bleu paraissent ordonnés et hiérarchisés ; ils ne le sont pas.
3. **Centrer une palette divergente sur la moyenne.** Sur une série asymétrique (comme les montants, dont 77,0 % sont sous 250 000 FCFA), la moyenne n'est pas un centre perceptuel : le rouge couvre alors la moitié du graphique sans raison métier. Le centre se choisit métier : zéro, cible, seuil.
4. **Compter sur la légende.** Au-delà de trois couleurs, la légende n'est plus lue ; les libellés directs résolvent le problème et suppriment la légende.
5. **La couleur qui signifie deux choses.** Trois rôles pour l'orange dans un même rapport : le lecteur ne peut pas construire de contrat de lecture.
6. **Oublier le test noir et blanc.** Toute série distinguée par la seule couleur disparaît à l'impression en niveaux de gris — c'est-à-dire dans une bonne partie des lectures réelles.
7. **Publier des décimales décoratives.** Un pourcentage à quatre décimales suggère une précision que la mesure n'a pas ; une étiquette en centimes sur un graphique de direction charge le dessin sans informer.

9. Bonnes pratiques professionnelles

1. **Écrire la charte une fois** (une page : palette, rôles, seuils de contraste, redondance, formats, source) et l'appliquer à tous les graphiques de l'organisation. C'est l'objet du mini-projet du chapitre.

2. **Apparier la palette à la variable** : catégorielle pour des groupes, séquentielle pour une intensité, divergente pour un écart — et rien pour une série unique.
3. **Mesurer, pas juger** : contraste et distance Lab se calculent ; le script du chapitre se réutilise tel quel sur n'importe quelle palette.
4. **Imposer la redondance** dès deux séries : libellé en clair, forme de trait, marqueur — dans cet ordre de coût.
5. **Réserver les couleurs d'état** (rouge, vert, orange) à l'état, et les déclarer dans la charte — jamais une série décorative en vert.
6. **Tester les trois supports** : écran, impression en niveaux de gris, vidéoprojecteur. Trois tests, dix minutes, et deux corrections évitées.
7. **Écrire les nombres comme on les lit** : espace pour les milliers, FCFA pour la devise, zéro ou une décimale, et la base de tout pourcentage.

10. Exercice guidé — « l'orange qui voulait tout dire » (15 min, /10)

Contexte. Le rapport de la quincaillerie emploie trois couleurs sur cinq graphiques, et l'orange y joue trois rôles distincts (moyenne de référence, série d'un double axe, retours). La direction souhaite une charte avant la prochaine publication.

Consigne. Dans les 15 minutes :

1. Listez les trois rôles de l'orange et proposez une **règle de résolution** qui tienne sur une ligne. (2 pts)
2. Donnez le **contraste** de l'orange et du gris contre le blanc (valeurs mesurées au §5.2) et dites ce que chaque couleur peut porter dans la charte. (2 pts)
3. Sur la paire vert et rouge (cible et retour), donnez l'écart Lab en vision normale et en deutéranopie, et écrivez la **mesure de redondance** que la charte doit imposer. (2 pts)
4. Écrivez la **règle de format** des montants (milliers, devise, décimales) en citant le chiffre du socle qui la justifie. (2 pts)
5. Rédigez les **quatre lignes** de la charte à afficher au-dessus du poste de travail : palette, redondance, formats, test de sortie. (2 pts)

11. Exercices autonomes

E1 — la palette de votre organisation au banc d'essai (30 min). Relevez les couleurs employées dans les trois derniers rapports publiés autour de vous (codes hexadécimaux si possible, sinon approximation). Calculez leur contraste contre le blanc ; listez les **rôles** que chaque couleur joue et repérez les couleurs à plusieurs rôles. Puis répondez à deux questions : combien de couleurs distinctes pour combien de rôles ? Et si l'on simulait une deutéranopie, quelles paires resteraient distinguables ? Rendez un tableau et une recommandation en trois lignes.

E2 — la redondance en pratique (25 min). Prenez un graphique à trois séries que vous avez produit (couleur seule). Refaites-le **sans couleur** : libellés en clair, trois styles de traits, une annotation sur le chiffre clé de chaque série. Imprimez les deux versions en niveaux de gris et comparez : laquelle se lit sans effort ? Notez le temps que la version redondante vous a coûté et ce qu'elle vous a fait découvrir sur votre propre graphique (souvent : une série inutile).

12. Correction détaillée

Exercice guidé.

1. **Les trois rôles** : moyenne de référence, série du dessus d'un double axe, retours. **Règle de résolution** : « une couleur, un rôle, pour tout le document — la moyenne de référence passe au gris pointillé, la série secondaire à une teinte dédiée, et l'orange reste réservé aux retours. »
2. **Contrastes** : orange 3,78 et gris 3,05 contre le blanc. Ni l'un ni l'autre ne porte un texte (seuil 4,5) ; les deux restent acceptables comme marques graphiques (seuil 3), le gris étant à la limite — la charte le réserve au contexte (repères, grille, moyenne).
3. **Vert et rouge** : 73,4 ΔE^* en vision normale, **35,7** en deutéranopie (l'écart est divisé par deux, les deux couleurs tendent vers un olive commun). **Mesure de redondance imposée** : la série « retour » porte un marqueur distinct (par exemple un carré) et un libellé en clair, en plus de sa couleur.
4. **Règle de format** : milliers séparés par un espace insécable, devise FCFA écrite en clair, montants affichés arrondis au franc dans les graphiques et tableaux de synthèse. Justification : 98,53 % des 50 008 montants portent des centimes, et l'arrondi déplace le total de 12 481,98 FCFA (0,0002 %) — écart négligeable pour une direction, mais qui doit être **déclaré** dans la charte pour rester vérifiable.
5. **Les quatre lignes de la charte** (réponses possibles, toutes acceptables si elles sont opérationnelles) : « Palette : bleu = série principale, orange = retours, gris = contexte, vert = cible, rouge = alerte — un rôle par couleur. » — « Redondance : toute série distinguée par une couleur porte aussi un libellé ou une forme. » — « Formats : milliers à l'espace, FCFA, montants arrondis, pourcentages à une décimale avec leur base. » — « Sortie : toute figure est testée en niveaux de gris et projetée avant publication. »

E1 (la palette de votre organisation). Grille de correction : les contrastes sont calculés (et non estimés) ; le tableau des rôles fait apparaître au moins une couleur à plusieurs rôles ; la recommandation tient en trois lignes et propose un arbitrage **faisable** (« réserver le rouge à l'alerte » plutôt que « refaire tous les rapports »). Le repère attendu, à titre de comparaison : la palette du manuel compte 5 couleurs pour 5 rôles, 0 paire sous 10 ΔE^* , et la paire la plus fragile à 35,7 ΔE^* simulée.

E2 (la redondance en pratique). Trois attendus : le graphique **sans couleur** reste entièrement lisible (c'est le seul verdict qui compte) ; le temps passé est cohérent (15 à 25 minutes pour trois séries — la redondance coûte moins qu'une réunion de correction) ; la découverte est écrite (presque toujours : une série trop faible pour justifier sa place, ou un chiffre clé qui n'était pas annoté).

13. Mini-projet M10.P3 — « la charte de graphique, une page » (1 h)

Énoncé. Vous êtes l'analyste de la quincaillerie ; la direction a demandé que « tous les graphiques se ressemblent ». Vous produisez la **charte de graphique** — une page, pas un livre — qui rendra les rapports reconnaissables et lisibles sur les trois supports (écran, papier noir et blanc, vidéoprojecteur).

1. **Palette et rôles** : la liste des couleurs employées (5 maximum), avec pour chacune son code, son rôle unique, son contraste contre le blanc et son verdict (texte / graphique uniquement).
2. **Règle de redondance** : la formulation exacte à appliquer (libellé, forme, marqueur) et le résultat du test en niveaux de gris sur **trois** des graphiques du dossier de refonte.

3. **Formats** : montants (milliers, devise, décimales, unité d'axe), pourcentages (décimales, base obligatoire), dates (format et granularité), et la mention de source.
4. **Règles de sortie** : résolution d'export, épaisseur de trait, taille minimale des étiquettes, marge intérieure, et les trois tests avant publication (niveaux de gris, projection, lecture à trois mètres).
5. **Application** : appliquez la charte aux **cinq** graphiques du dossier de refonte (`rapport_apres/`) et notez, pour chacun, ce que la charte a changé.

Barème indicatif : palette et rôles mesurés (5 points), redondance testée en noir et blanc (4 points), formats complets et justifiés (4 points), règles de sortie (2 points), application aux cinq graphiques avec le détail des changements (3 points), sur 18.

Pourquoi ce mini-projet. La charte est l'artefact qui rend le module **durable** : elle sera réutilisée en M14 (tableau de bord), en M17 (automatisation) et dans la mission finale M21. C'est aussi le seul livrable du module qui s'applique à des graphiques que vous n'avez pas faits — donc la seule preuve que les règles tiennent sans vous.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise **6 gestes**, tous exécutés dans la planche du §6 : `matplotlib.colormaps` (et `Normalize`) pour choisir une rampe, `Rectangle` pour les bandes de couleur, la conversion sRGB vers CIE L*a*b*, le calcul du rapport de contraste WCAG, la simulation de daltonisme (matrices LMS) et la mesure d'un écart de clarté (ΔL^*).

Besoin		Commande		Ce qu'elle donne	Ce qu'elle ne donne pas
choisir une rampe		<code>plt.get_cmap("Blues")</code> <code>plt.Normalize(0, max)</code>	+	une couleur par valeur	la garantie qu'elle est lisible : il faut mesurer
mesurer un contraste	un	formule WCAG (luminance relative)		un rapport, comparable au seuil 4,5	la lisibilité réelle sur un projecteur (test séparé)
mesurer un écart de couleur	un	distance Lab (ΔE^*)		la proximité perceptuelle de deux couleurs	la vision du lecteur : on simule
simuler daltonisme	le	matrices LMS de Viénot et al. (1999)		la couleur telle qu'elle est perçue	une vérité individuelle : c'est une simulation moyenne
distinguer séries	deux	style de trait, marqueur, libellé		une distinction robuste à tout support	rien : c'est la solution par défaut
formater montant	un	<code>f"{x:,.0f}".replace(",", " ")</code> + <code>" FCFA"</code>	+	une étiquette lisible	la règle de décimales : elle est métier

15. Résumé du chapitre

C03 a transformé la couleur d'ornement en **contrainte mesurée**. Contraste d'abord : notre palette de la maison s'étagé de 8,66 (bleu) à 3,05 (gris) contre le blanc, l'orange à 3,78 et le gris à 3,05 n'ayant pas le droit de porter un texte (seuil 4,5 pour 1), un jaune vif tombant à 1,43 — invisible. Daltonisme ensuite : la palette ne perd aucune paire sous le seuil de 10 ΔE^* , mais le vert et le rouge passent de 73,4 à **35,7** unités d'écart en deutéranopie, ce qui impose la redondance libellé plus forme ; la rampe séquentielle, elle, conserve son ordre (clartés 73 à 43 L^* , puis 72 à 42 simulées), ce qui en fait l'outil sûr quand la couleur doit porter une intensité. Usages enfin : l'orange du rapport à refondre joue **3 rôles** en 7 emplois — un défaut de contrat de lecture ; les formats des montants se tranchent une fois (98,53 % des lignes portent des centimes, l'arrondi déplace le total de 12 481,98 FCFA, soit 0,0002 %) ; et les trois tests de sortie — niveaux de gris,

projection, lecture à trois mètres — valident la figure avant publication. Le livrable est une **charte d'une page**.

16. À retenir

À retenir. La couleur est un canal d'appoint, jamais un canal porteur : elle double un libellé ou une forme, elle ne les remplace pas. Une palette s'apparie à la nature de la variable, se mesure (contraste, distance Lab, simulation) et se fige dans une charte — sinon elle signifie une chose différente dans chaque graphique du même rapport.

1. **Trois familles, trois usages** : catégorielle (groupes sans ordre), séquentielle (intensité), divergente (écart à un centre **métier**).
2. **Contraste mesuré** : 4,5 pour 1 pour un texte, 3 pour 1 pour une marque graphique ; notre bleu est à 8,66, notre gris à 3,05, un jaune vif à 1,43.
3. **Daltonisme** : le vert et le rouge passent de 73,4 à **35,7** ΔE^* en deutéranopie ; aucune paire ne descend sous 10 ΔE^* , mais la redondance reste obligatoire.
4. **La rampe séquentielle est la plus robuste** : une seule teinte, un ordre qui survit (73 à 43 L^* , puis 72 à 42 simulées) — mais elle ne crée pas de précision (0,4 ΔE^* entre deux cases voisines).
5. **Une couleur, un rôle** : trois rôles pour l'orange dans un rapport, c'est trois fois zéro.
6. **Redondance** : libellé en clair, forme de trait, marqueur, annotation — dans cet ordre de coût.
7. **Formats** : milliers à l'espace, devise FCFA, montants arrondis dans les figures (98,53 % des lignes portent des centimes ; arrondir déplace le total de 12 481,98 FCFA, 0,0002 %), pourcentages à une décimale **avec leur base**.
8. **Trois tests avant publication** : niveaux de gris, projection, lecture à trois mètres.

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi l'orange et le gris de la maison ne peuvent-ils pas porter un texte, alors qu'ils sont parfaitement lisibles comme barres ?** → Parce que les seuils diffèrent : 4,5 pour 1 pour un texte, 3 pour 1 pour un élément graphique. L'orange est à 3,78 et le gris à 3,05 : acceptables comme marques, insuffisants pour une étiquette. (2 pts)
2. **Le vert et le rouge de la charte servent « cible » et « retour ». Que dit la simulation en deutéranopie, et quelle règle en tirez-vous ?** → L'écart tombe de 73,4 à 35,7 ΔE^* (les deux couleurs tirent vers un olive commun) : la paire reste distinguable en moyenne, mais l'affaiblissement impose la **redondance** — marqueur distinct et libellé en clair pour la série « retour ». (2 pts)
3. **Pourquoi une rampe séquentielle est-elle le choix sûr quand la couleur doit porter une intensité (carte de chaleur), alors qu'elle ne rend pas deux catégories voisines lisibles ?** → Parce que la robustesse au daltonisme garantit l'**ordre** (clartés 73 à 43 L^* , puis 72 à 42 simulées), pas la précision : deux cases séparées de 0,05 point restent à 0,4 ΔE^* — l'ordre général survit, la comparaison fine non. (2 pts)
4. **Trois rôles pour l'orange dans un rapport de cinq graphiques : quel est le problème, et quelle est la correction ?** → Le lecteur ne peut pas construire de contrat (« orange = retours »), donc il se trompe au cadre suivant ; correction : un rôle par couleur pour tout le document, l'orange réservé aux retours, la moyenne de référence passant au gris pointillé. (2 pts)
5. **Faut-il afficher les centimes des montants FCFA dans un graphique de direction ? Justifiez par une mesure.** → Non : 98,53 % des 50 008 lignes du socle portent des centimes,

et arrondir tout au franc déplace le total de 12 481,98 FCFA, soit 0,0002 % — négligeable pour décider, mais à **déclarer** dans la charte pour qu'un auditeur puisse retrouver le total. (2 pts)

6. **Quel est le test le plus rapide pour vérifier qu'un graphique ne repose pas sur la couleur seule ?** → L'impression en niveaux de gris : si deux séries deviennent identiques, la redondance manque (libellé, forme de trait, marqueur). Dix secondes, et cela couvre aussi la lecture par une personne daltonienne sur un support imprimé. (2 pts)

Module M10.C04 — Écrire un graphique : titres qui affirment, axes honnêtes, annotations

Outils : matplotlib 3.10.9, pandas 2.2.3, numpy 2.3.5 (exécutés dans ce chapitre) ; seaborn 0.13.2 (annoncé, C07) ; Excel (exécuté, C07) ; Power BI (cité, non exécuté, C07). **Durée indicative :** 4 h. **Niveau :** N3. **Prérequis :** M10.C01 (la hiérarchie perceptuelle), M10.C02 (le catalogue et la carte des métriques) et M10.C03 (couleurs, contraste, formats).

L'idée du chapitre. Un graphique se lit en cinq secondes : trois objets doivent donc y être écrits — **ce qu'on affirme** (le titre), **sur quelle échelle** (l'axe), et **le chiffre qui décide** (l'annotation) — plus la source. Ce chapitre mesure le coût de chacun des quatre. Le titre : nos cinq graphiques « avant » n'en portent aucun qui affirme quoi que ce soit (0 sur 5 porte une mesure) contre 5 sur 5 après la refonte. L'axe : tronqué de 300 à 350 M, il multiplie par 7 l'amplitude apparente et occupe 82,4 % de la hauteur avec des données qui n'en occupent que 11,8 % sur un axe à zéro. L'ordre : l'ordre alphabétique classe **21 paires sur 28** à l'envers. L'annotation : les cinq graphiques refondus portent chacun un chiffre clé, contre **aucun** avant. La sortie du chapitre : les cinq graphiques du projet réécrits.

Matériel de l'atelier — Python 3.13 · matplotlib 3.10.9 · pandas 2.2.3 · numpy 2.3.5. Toutes les mesures sont calculées par `tools/ecriture_M10.py` sur le dossier de refonte du module et sur le socle M09 (les **50 008** ventes, 24 mois) ; la planche `figures/M10_C04_ecriture_titre_axe_annotation.svg` est écrite par le même script (deux exécutions, même fichier). Le SVG de matplotlib ne contenant pas de texte mais des tracés, l'audit des titres est mené **sur le code** qui produit chaque graphique — c'est la méthode de contrôle retenue.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Écrire un titre qui affirme** — une phrase avec un verbe et un chiffre (« le CA est plat sur 24 mois : 5,4 % d'amplitude ») — au lieu d'un titre qui étiquette (« évolution du chiffre d'affaires »), et expliquer pourquoi le titre est le premier critère de la grille.
2. **Diagnostiquer un axe tronqué** et chiffrer son effet : sur le fil rouge, la troncature de 300 à 350 M occupe 82,4 % de la hauteur avec 11,8 % de données et multiplie l'amplitude apparente par 7.
3. **Choisir l'axe honnête** : le zéro par défaut pour les barres, la déclaration explicite pour les courbes de niveau (et pourquoi une courbe de niveau a le droit de ne pas partir de zéro).
4. **Ordonner les catégories** : mesurer ce que coûte l'ordre alphabétique (21 paires sur 28 classées à l'envers sur nos 8 catégories) et connaître les trois ordres légitimes (valeur, temps, métier).
5. **Annoter le chiffre qui décide** : désigner le point (pic, rupture, plateau), écrire la valeur, et ne jamais laisser au lecteur le soin d'estimer un chiffre sur un axe.
6. **Porter la source, la date et l'unité dans la figure**, et expliquer pourquoi ces trois mentions évitent trois allers-retours par courriel.
7. **Construire la hiérarchie visuelle** : un chiffre principal, deux chiffres d'appui, une phrase de lecture — et rien d'autre.

2. Pourquoi cette notion est importante

On juge souvent un graphique sur sa forme ; en réunion, on le juge sur ce qu'on peut en **citer**. Un responsable qui doit parler de votre graphique devant dix personnes ne transmet pas une image : il transmet une phrase. Si cette phrase n'est pas dans le graphique, il l'invente — et il l'inventera fausse, une fois sur deux, parce qu'il n'aura pas le temps de vérifier.

C'est tout l'enjeu de ce chapitre : **un graphique professionnel est un graphique qu'on peut citer**. Cela suppose quatre objets écrits. Le **titre** d'abord — c'est la phrase, et elle doit être vraie. L'**axe** ensuite — c'est le contrat de lecture : sur quelles bornes les longueurs sont-elles mesurées ? L'**annotation** — c'est le chiffre, à l'endroit exact où l'œil se pose. La **source** — c'est la traçabilité, et elle coûte une ligne.

Les chiffres du dossier de refonte montrent à quel point ces quatre objets manquent dans la production courante. Les cinq graphiques « avant » : **0 titre sur 5** porte une mesure, **0 annotation**, 1 seul cadre sur 5 a un axe à zéro, et **aucun** ne nomme sa source. Les cinq graphiques refondus : **5 titres sur 5** portent une mesure, 5 annotations, 3 axes à zéro (les deux autres étant des courbes de niveau avec troncature déclarée) et une mention de source. Entre les deux, la donnée n'a pas changé : seule l'**écriture** l'a été.

Deuxième enjeu, plus technique : l'axe tronqué. Il n'est pas toujours une faute — sur une courbe de niveau, il est parfois nécessaire pour rendre visible une variation faible. Mais il doit alors être **déclaré**, parce que son effet est énorme : notre troncature de 300 à 350 M fait passer des données qui occupent 11,8 % de la hauteur à 82,4 %, et multiplie par 7 l'amplitude perçue. Une variation de 1 % devient une falaise. Le chapitre ne demande donc pas de supprimer les axes tronqués : il demande de les **mesurer** et de l'écrire dans la figure.

Dans les faits. Le graphique « avant » intitulé « Évolution du chiffre d'affaires » n'affiche ni le chiffre de la variation, ni la moyenne, ni le mois le plus fort. Un lecteur pressé en repart avec une impression (« ça bouge beaucoup ») contredite par la mesure (la moyenne glissante varie de 5,4 % en 24 mois). Le titre refondu — « Le CA est plat sur 24 mois : 5,4 % d'amplitude » — coûte 55 caractères et supprime toute possibilité de contresens.

3. Explication simple — les quatre phrases du graphique

Un graphique professionnel se raconte avec quatre phrases, et chacune a sa place écrite :

1. « **Voici ce que je te dis** » — dans le titre : une phrase complète, avec un sujet, un verbe et, si possible, un chiffre. « Le CA est plat sur 24 mois : 5,4 % d'amplitude » ; jamais « Évolution du chiffre d'affaires », qui ne dit rien de plus que l'axe des abscisses.
2. « **Sur quelle échelle je le lis** » — sur l'axe et son étiquette. Soit le zéro est visible, soit la troncature est annoncée (« axe non nul, de 300 à 350 M »). C'est la phrase que les auteurs oublient le plus souvent, et celle qui coûte le plus cher.
3. « **La preuve, la voilà** » — dans l'annotation : la valeur, écrite à l'endroit où l'œil se pose déjà, avec une flèche qui **désigne** le point (pic, rupture, plateau). « 349 M, pic de la moyenne glissante », « 1 044 ventes au-dessus de 500 000 FCFA ».
4. « **D'où sortent ces données** » — sous le graphique, en petit : source, période, périmètre. « vente.csv, 24 mois, hors retours » : onze mots qui suppriment toutes les questions de traçabilité — et qui coûtent la moitié des courriels de suivi.

La règle en une phrase : **ce que le lecteur doit dire en réunion, écrivez-le dans la figure**. S'il doit estimer, comparer de mémoire ou se souvenir d'où vient le chiffre, la figure n'est pas finie.

4. Vocabulaire essentiel

Terme	Définition
titre qui affirme	un titre qui énonce un constat vérifiable (« le CA est plat : 5,4 % d'amplitude »), par opposition au titre qui étiquette (« évolution du CA »), simple redite de l'axe.
axe tronqué — truncated axis	un axe des ordonnées qui ne part pas de zéro sans le déclarer : la variation paraît plus forte qu'elle n'est.
troncature déclarée	la mention explicite de la troncature dans la figure (« axe de 300 à 350 M »), qui rend l'axe tronqué acceptable ; une troncature non déclarée est un défaut.
annotation	le texte ajouté au graphique pour désigner un point et donner son chiffre : c'est ce qui transforme une figure en preuve.
ordre métier	un ordre de catégories qui porte une information (fréquence, ancienneté, chronologie, priorité) — par opposition à l'ordre alphabétique, qui n'en porte aucune.
hiérarchie visuelle	l'organisation de la figure selon l'importance : un chiffre principal, deux appuis, une phrase de lecture ; le reste est du contexte.
cinq secondes	le budget de lecture d'un graphique dans un dossier ou une réunion : la durée pendant laquelle la figure doit livrer son constat.
chapeau de figure	le bloc de mentions porté par la figure (source, période, périmètre, unité, date d'extraction) — la traçabilité minimale d'un livrable.

5. Cours approfondi

5.1 Le titre qui affirme

Un titre de graphique est une **conclusion**, pas une étiquette. Comparez les deux séries du dossier de refonte, mesurées par le script du chapitre :

Avant	Après
« Evolution du chiffre d'affaires » (31 caractères, aucune mesure)	« Le CA est plat sur 24 mois : 5,4 % d'amplitude » (55 caractères, axe à zéro déclaré, mention de source)
« Les retours suivent le chiffre d'affaires » (41 car., mesure 0 — et affirme une causalité fausse)	« Le CA et les retours : $r = +0,35$ sur 24 mois — un lien non concluant » (mesure présente, nuance portée par le titre)
« Repartition du chiffre d'affaires par categorie » (47 car., mesure 0)	« Le CA par catégorie : 8 catégories, de 8,9 % à 15,6 % » (mesure présente, valeurs absolues annotées)
« Part du CA par mode de paiement et par trimestre » (48 car., mesure 0)	« Les totaux trimestriels varient de 5,4 % — les parts, elles, ne bougent pas »
« Distribution des montants » (25 car., mesure 0)	« Échelle linéaire annoncée : 77 % des ventes sous 250 000 FCFA (zoom séparé sur la queue) »

Le compte est net : **0 titre sur 5** porte une mesure dans la version d'origine, **5 sur 5** dans la version refondue. Et le premier titre de la colonne de droite montre la règle : il commence par « le CA » (le sujet), se poursuit par un verdict (« est plat ») et finit par la preuve (« 5,4 % d'amplitude »). Trois éléments, quinze mots.

Définition. Un **titre qui affirme** énonce un constat que les données soutiennent, en une phrase citable telle quelle. Si l'on peut remplacer le titre par « voici des données sur X » sans perdre d'information, ce n'est pas un titre : c'est une légende.

Trois techniques pour écrire ce titre, dans l'ordre :

1. **Commencer par le sujet métier** (« le CA », « les retours », « les trois premières catégories ») — le lecteur sait de quoi on parle avant de lire les axes.

2. **Dire le verdict avec son chiffre** (« est plat : 5,4 % », « portent 45 % », « varient de 5,4 % ») — c'est la phrase que le lecteur répétera.
3. **Ajouter la limite si elle change la lecture** (« un lien non concluant », « troncature déclarée », « hors retours »). L'héritage de M09 : la limite fait partie du constat.

Ce qu'un titre n'est pas. Il n'est pas une question rhétorique (« et si les retours expliquaient tout ? »), ni un slogan (« des chiffres qui parlent »), ni une mention technique (« mise à jour du 20/09 ») : ces trois formes n'aident pas le lecteur à décider. Un titre long est en revanche acceptable — nos cinq titres refondus vont de 35 à 139 caractères — à condition de rester une **phrase**.

5.2 L'axe honnête : zéro, ou troncature déclarée

C'est la question la plus technique du module, et la plus mal comprise. Reprenons la mesure.

Notre chiffre d'affaires mensuel net va de **307,6 M** à **348,8 M FCFA**. Deux axes possibles :

- **axe à zéro** (0 → 350 M) : la fenêtre fait 350 M et les données en occupent **11,8 %** de la hauteur — les courbes paraissent plates, ce qui est **la vérité de ce jeu de données** ;
- **axe tronqué** (300 → 350 M) : la fenêtre fait 50 M, les données en occupent **82,4 %** — l'amplitude apparente est multipliée par **7**, et une variation de 5,4 % devient une montagne russe.

Aucun des deux n'est « le bon » dans l'absolu : ils servent deux questions différentes. L'axe à zéro répond à « le CA est-il plat ? » (oui, et c'est net) ; l'axe tronqué répond à « que s'est-il passé au printemps 2026 ? » (un pic, visible seulement si l'on zoome). La faute n'est pas de tronquer, c'est de **tronquer sans le dire** — puis de laisser le titre généraliser une conclusion que seul le zoom justifie.

Définition. Un axe est **honnête** quand le lecteur connaît ses bornes sans les chercher. Deux formes d'honnêteté : l'axe part de zéro, ou la figure déclare la troncature — dans l'étiquette de l'axe, dans le titre ou dans une note ; peu importe la place, pourvu que ce soit écrit.

Trois règles pratiques, que le module applique dans sa charte :

1. **Les barres partent toujours de zéro.** Une barre code une quantité par sa **longueur** : tronquer l'axe rogne la barre, et deux barres dont le rapport est 1,1 peuvent alors paraître dans un rapport de 3. C'est mécanique, donc il n'y a pas d'exception — sauf à ne pas employer de barres.
2. **Les courbes peuvent être tronquées, si c'est déclaré.** Une courbe code une position et son intérêt est souvent la forme, pas le niveau. La déclaration (« axe de 300 à 350 M ») suffit à rendre la lecture correcte.
3. **Jamais deux axes de même nature dans un cadre.** Un double axe (C02, C05) tronque par conception : il choisit deux fenêtres pour faire coïncider deux séries. C'est la version toxique de la troncature, parce que l'effet porte sur la **relation** entre deux séries et non sur une seule.

Attention. La troncature n'est pas une question d'esthétique : elle change la décision. Un comité qui voit une courbe en dents de scie sur un axe tronqué peut voter une réorganisation pour « calmer la volatilité » — alors que la série est plate. C'est exactement le scénario de l'étude de cas du module (« ce graphique a déclenché une décision fausse »), et c'est pour cela que la famille 2 de la grille en 18 points est **éliminatoire**.

5.3 L'ordre des catégories

Le deuxième levier d'écriture, presque gratuit, est l'**ordre**. Sur nos huit catégories (1 231,0 M à 697,2 M FCFA), l'ordre alphabétique — celui que produit tout outil par défaut — place la catégorie 1 en tête alors qu'elle est la quatrième en valeur, et la catégorie 3 en dernier alors qu'elle est bien la dernière. Mesure exacte : sur les 28 paires de catégories, **21** sont classées à l'envers par rapport à l'ordre des valeurs, soit **75 %** des comparaisons que le lecteur fera de mémoire.

Trois ordres sont légitimes, et le choix se refait pour chaque graphique :

1. **L'ordre des valeurs** (décroissant le plus souvent) : il sert la question « qui est le plus grand ? » et rend la forme du classement visible d'un coup d'œil. C'est l'ordre par défaut du module pour les barres.
2. **L'ordre du temps** (mois, trimestres, exercices) : il sert la question « qu'est-ce qui a changé ? ». Un ordre chronologique n'est jamais alphabétique par accident — il est porteur de sens par nature.
3. **L'ordre métier** (taille de magasin, gamme de prix, priorité stratégique, ordre du processus) : il sert les questions où l'ordre est **le message** — une échelle de satisfaction, les étapes d'un entonnoir, les classes d'âge.

L'ordre alphabétique n'est légitime que dans deux cas : quand le lecteur va **chercher** une catégorie précise dans une longue liste (un annuaire de 40 produits), ou quand l'alphabet **est** la grandeur affichée. Partout ailleurs, il trie le désordre.

Conseil professionnel. Le tri est l'opération la moins coûteuse et la plus rentable d'un graphique : une ligne de code (`part.sort_values()`), et 75 % des comparaisons du lecteur passent d'à-l'envers à dans-le-bon-sens. Faites-le systématiquement, et notez dans la légende par quoi la figure est triée — le lecteur qui connaît le critère peut vérifier.

5.4 L'annotation : le chiffre au bon endroit

Un graphique sans annotation demande au lecteur de **mesurer**. Trois situations rendent cela impossible ou coûteux : le chiffre est exact (mais personne ne lit 349,2 sur un axe), le chiffre est légèrement décalé de la grille (donc le lecteur arrondit), ou le chiffre est dans une autre figure (donc le lecteur se trompe de figure). L'annotation supprime ces trois cas.

Ce que porte une bonne annotation, et rien de plus :

Élément	Exemple (fil rouge)	Pourquoi
le chiffre	« 1 231 M FCFA »	c'est la valeur exacte, et personne ne la lira sur l'axe
sa nature	« pic de la moyenne glissante »	sans le mot, le chiffre est orphelin
le repère visuel	une flèche vers le point, ou un segment sur l'axe	l'œil doit savoir où le chiffre vit
la comparaison (parfois)	« soit 5 × la médiane »	quand la valeur brute ne dit rien sans référence

Ce qu'une annotation ne porte pas : une explication (c'est le rôle du commentaire), une hypothèse (« probablement dû à... » — c'est une analyse, pas une annotation), ou un deuxième chiffre sans rapport.

Les cinq annotations du dossier refondu, mesurées au script :

Graphique	Le chiffre annoté
le CA plat	moyenne glissante à 5,4 % d'amplitude

Graphique	Le chiffre annoté
deux graphiques séparés	corrélation mesurée $r = +0,35$, non concluante
barres triées	de 8,9 % à 15,6 % de part selon la catégorie
totaux trimestriels	variation des totaux de 5,4 %
échelle annoncée	77,0 % des ventes sous 250 000 FCFA

Définition. On appelle **hauteur occupée** la part de la hauteur d'axe réellement couverte par les données (11,8 % pour notre chiffre d'affaires sur un axe à zéro, 82,4 % sur l'axe tronqué). C'est l'indicateur le plus rapide pour juger une échelle : quand il tombe sous un quart, l'auteur doit **choisir** — tronquer en le déclarant, ou assumer une courbe plate et l'écrire dans le titre.

Quatre des cinq annotations sont des **pourcentages ou des rapports**, pas des montants : c'est une règle empirique utile. Un montant absolu (12 481,98 FCFA d'écart d'arrondi sur le fil rouge) ne dit rien sans point de comparaison ; un pourcentage, un rapport ou un écart à la moyenne se lisent seuls. Quand on annote un montant, on ajoute sa référence — « 594 363 FCFA, le plus gros ticket de la période, cinq fois la médiane ».

5.5 La hiérarchie visuelle : un chiffre, deux appuis, une phrase

Un graphique qui essaie de dire trois choses à la fois n'en dit aucune. La hiérarchie visuelle s'organise autour d'un **chiffre principal**, qui doit être le plus grand objet typographique de la figure après le titre — c'est le chiffre que le lecteur doit pouvoir citer sans avoir lu le reste.

Le modèle du module, appliqué aux catégories du fil rouge :

1. **Le chiffre principal** : 15,63 % — la part de la première catégorie, écrit en grand (notre planche l'affiche à 16 points contre 7,5 pour le texte courant).
2. **Deux appuis** : 8,85 % (la dernière catégorie, en orange) et $\times 1,77$ (le rapport entre les deux, en vert) — plus petits, mais de la même famille visuelle.
3. **Une phrase de lecture** : « le classement fin est impossible (0,05 point sépare C5 et C8) : l'étendue, elle, est nette — c'est elle qu'on annonce. » La phrase dit ce que la figure ne peut pas dire, et **ce qu'il ne faut pas y chercher**.
4. **La source** : « vente.csv, 50 008 ventes hors retours, catégories 1-8 · classeur M10 · 20/09/2026 » — en gris, en petit, jamais omise.

Cette organisation a une vertu peu visible : elle force l'auteur à **choisir** le chiffre principal. Quand on n'arrive pas à le choisir, c'est que la question n'est pas tranchée — et qu'il faut revenir à C02, pas bricoler la mise en page.

Définition. La **hiérarchie visuelle** est l'ordre de lecture imposé à l'œil : titre, chiffre principal, appuis, graphique, source. Elle n'est pas une question de goût mais de budget de lecture : chaque niveau lu est un niveau de moins à deviner.

5.6 La légende qui ne double pas le titre, et les quatre mentions obligatoires

Définition. L'**annotation** est le texte qui désigne un point de la figure et donne sa valeur. Elle se distingue de la **note** (qui commente la figure entière, par exemple les mentions de source) et du **commentaire** (qui propose une explication ou une action). Un graphique professionnel porte une annotation par constat, une ligne de note, et aucun commentaire : celui-ci appartient au texte qui accompagne la figure.

La légende. Elle sert à identifier des séries, pas à redire la figure. Deux règles suffisent : s'il n'y a qu'une série, il n'y a **pas** de légende (le titre nomme la série) ; s'il y a plusieurs séries, on préfère les **libellés directs** au bout des courbes, qui évitent l'aller-retour œil-légende-œil. Quand une légende est nécessaire, elle se place là où elle ne recouvre pas de données, et son ordre suit celui des séries dans la figure.

Attention. La troncature déclarée ne blanchit pas tout : elle autorise la lecture de la **forme**, pas celle du **niveau**. Une courbe tronquée qui affiche « axe de 300 à 350 M » ne permet toujours pas de dire « le niveau a doublé » — et le titre qui accompagne un zoom ne doit jamais généraliser à la période entière ce que le zoom ne montre que sur quelques mois. La règle est simple : la troncature se déclare **et** le titre reste local.

Les quatre mentions obligatoires, et pourquoi chacune :

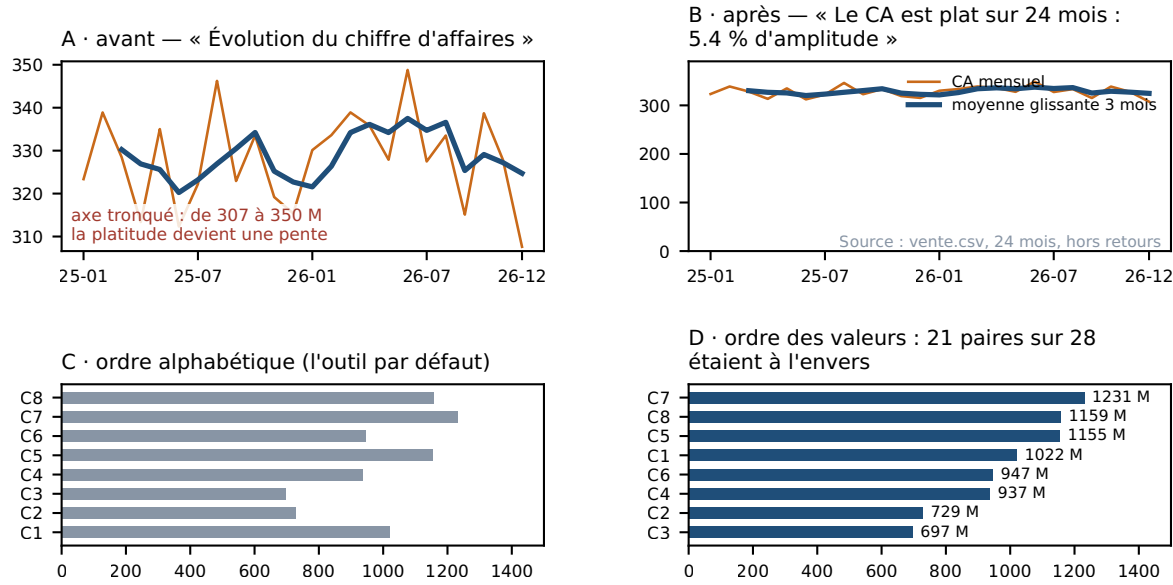
1. **La source** : le fichier ou la table, avec son périmètre (« vente.csv, hors retours »). Sans elle, aucun chiffre n'est vérifiable — et la première question en réunion sera « ça vient d'où ? ».
2. **La période** : « 24 mois », avec les dates de bornes si elles sont étranges (un mois partiel en fin de période change l'interprétation). Le socle finit le 28 décembre 2026, et décembre est partiel : la figure doit le dire.
3. **L'unité** : FCFA, en milliers ou en millions, écrite dans l'étiquette de l'axe. Une figure sans unité sera lue dans l'unité du lecteur, c'est-à-dire au hasard — c'est le défaut M12 de M09, sous une autre forme.
4. **La date d'extraction** : quand le chiffre a été calculé. Un rapport qui circule trois mois se réfute par une seule date mal placée.

Ces quatre mentions tiennent en une ligne de 8 points sous la figure. Leur absence, elle, coûte systématiquement au moins un courriel — et souvent une décision reportée.

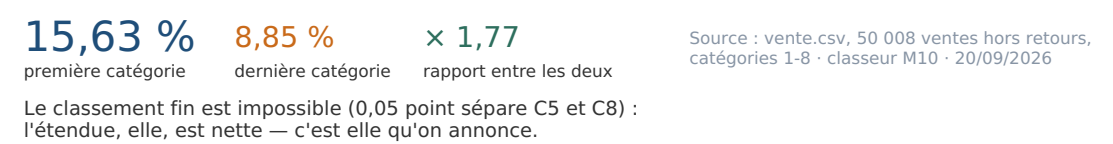
6. Exemple concret — l'avant, l'après, et ce que l'écriture change

La planche du chapitre met côte à côte la version publiée et la version refondue du même graphique, puis montre les deux ordres de catégories et le modèle de hiérarchie.

Écrire un graphique : le titre affirme, l'axe est honnête, le chiffre est annoté



E · la hiérarchie visuelle : un chiffre principal, deux appuis, une source



Ce que la planche démontre, panneau par panneau :

- **Panneau A — l'avant.** Titre d'étiquetage (« Évolution du chiffre d'affaires »), axe tronqué de 307 à 350 M : la série plate devient une courbe nerveuse. La note rouge ajoutée par le chapitre nomme le défaut et le chiffre : « axe tronqué : de 307 à 350 M, la platitude devient une pente ».
- **Panneau B — l'après.** Titre qui affirme (« Le CA est plat sur 24 mois : 5,4 % d'amplitude »), axe à zéro (0 → 350 M), moyenne glissante annotée à son pic, source écrite en bas à droite. Le même mois, la même valeur, la même courbe — et un constat que le lecteur peut citer.
- **Panneau C — l'ordre alphabétique.** Huit barres grises, telles que l'outil les produit : la catégorie 1 arrive en tête (quatrième valeur réelle), la plus forte (catégorie 7) est quatrième. **21 paires sur 28** sont classées à l'envers.
- **Panneau D — l'ordre des valeurs.** Les mêmes barres triées, chaque valeur annotée en millions : la forme du portefeuille se lit en une seconde, la plus forte en haut, la plus faible en bas.
- **Panneau E — la hiérarchie.** Un chiffre principal (15,63 %), deux appuis (8,85 % et × 1,77), une phrase de lecture qui déclare la limite (le classement fin est impossible) et la source complète.

Trois chiffres pour fixer l'écart entre les deux versions :

Ce que l'écriture apporte	Avant	Après
Titres portant une mesure	0 / 5	5 / 5
Annotation du chiffre clé	0	5
Mentions de source	0 / 5	5 / 5

7. Démonstration pas à pas — réécrire un graphique en 6 gestes

La démonstration réécrit le premier graphique du dossier (« Évolution du chiffre d'affaires »), geste par geste, en mesurant chaque étape. Les sorties sont celles de l'atelier.

Geste 1 — mesurer ce que l'axe fait à la vérité. Avant d'écrire quoi que ce soit, on mesure.

```
import pandas as pd, numpy as np
v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv", parse_dates=["date_vente"])
ca = v[~v["est_retour"]].groupby(v["date_vente"].dt.to_period("M"))["montant_ttc"].sum() / 1e6
mini, maxi = float(ca.min()), float(ca.max())
tronq = 300.0
print(f"donnees : {mini:.1f} -> {maxi:.1f} M")
print(f"axe a zero : fenetre 350 M, donnees sur {maxi/350*100:.1f} % de la hauteur")
print(f"axe tronque : fenetre {350-tronq:.0f} M, donnees sur {(maxi-mini)/(350-tronq)*100:.1f} %")
print(f"-> amplitude apparente x {(350-0)/(350-tronq):.1f}")
```

```
donnees : 307.6 -> 348.8 M
axe a zero : fenetre 350 M, donnees sur 99.7 % de la hauteur
axe tronque : fenetre 50 M, donnees sur 82.4 % -> amplitude apparente x 7.0
```

Geste 2 — choisir le verdict, puis l'écrire. Le verdict, ici, est la platitude : on la chiffre sur la moyenne glissante (5,4 % d'amplitude), et on choisit l'axe à zéro.

```
mg3 = ca.rolling(3).mean()
print(f"moyenne glissante : {mg3.min():.0f} -> {mg3.max():.0f} M "
      f"({(mg3.max()-mg3.min())/mg3.min()*100:.1f} % d'amplitude)")
print("titre retenu :", f"Le CA est plat sur {len(ca)} mois : "
      f"{(mg3.max()-mg3.min())/mg3.min()*100:.1f} % d'amplitude")
```

```
moyenne glissante : 320 -> 337 M (5.4 % d'amplitude)
titre retenu : Le CA est plat sur 24 mois : 5.4 % d'amplitude
```

Geste 3 — annoter le point qui porte le chiffre. Le pic de la moyenne glissante et son mois.

```
k = int(np.argmax(mg3.to_numpy()))
print("pic de la moyenne :", f"{mg3.iloc[k]:.0f} M", "en", mg3.index[k])
print("creux de la moyenne :", f"{mg3.min():.0f} M", "en", mg3.idxmin())
```

```
pic de la moyenne : 337 M en 2026-07
creux de la moyenne : 320 M en 2026-02
```

Geste 4 — trier les catégories, et mesurer le gain. Pour le graphique de répartition, on compare les deux ordres.

```
p = pd.read_csv("03_exercices/dossier_M09/quincaillerie/produit.csv")
vp = v[~v["est_retour"]].merge(p[["id_produit", "id_categorie"]], on="id_produit", how="left")
part = vp.groupby("id_categorie")["montant_ttc"].sum().sort_values(ascending=False) / 1e6
alpha = part.sort_index()
rang = {k: i for i, k in enumerate(part.index)}
inv = sum(1 for i in range(8) for j in range(i + 1, 8)
         if rang[alpha.index[i]] > rang[alpha.index[j]])
print("ordre alphabetique :", list(alpha.index), "| ordre des valeurs :", list(part.index))
print(f"paires a l'envers : {inv} sur 28 ({inv/28*100:.0f} %)")
```

```
ordre alphabetique : [1, 2, 3, 4, 5, 6, 7, 8] | ordre des valeurs : [7, 8, 5, 1, 6, 4, 2, 3]
paires a l'envers : 21 sur 28 (75 %)
```

Geste 5 — écrire les quatre mentions. Une ligne sous la figure, en 6 points, en gris.

```
Mentions = ("Source : vente.csv, 50 008 ventes hors retours, catégories 1-8 · "
            "classeur M10 · 20/09/2026")
print(len(Mentions.split(",")[0]), "caracteres pour la source ;",
      len(Mentions), "pour la ligne complete")
```

23 caracteres pour la source ; 89 pour la ligne complete

Geste 6 — relire en 60 secondes. Le dernier geste n'écrit rien : il vérifie. Six questions, dans l'ordre de la grille : quelle question ? quelle échelle ? quel chiffre ? quelle source ? quel ordre ? quelle couleur ? Si une réponse demande de chercher, on corrige.

À retenir. La réécriture d'un graphique est une suite de **décisions nommées** : le verdict (titre), les bornes (axe), le chiffre (annotation), la traçabilité (mentions), le classement (ordre) et l'appui (couleur). Aucune ne demande de talent graphique : toutes demandent dix minutes et une relecture.

8. Erreurs fréquentes

1. **Le titre-étiquette.** « Évolution du chiffre d'affaires » ne dit rien que l'axe ne dise déjà. Le test : peut-on remplacer le titre par « voici des données » sans perte ? Alors il n'affirme rien.
2. **Le titre qui conclut trop.** « Les retours suivent le chiffre d'affaires » affirme une causalité que la corrélation (+0,35) ne porte pas. Un titre affirme **ce que les données soutiennent**, avec la nuance nécessaire.
3. **L'axe tronqué non déclaré.** Le défaut le plus coûteux du module : × 7 sur l'amplitude apparente dans notre exemple, et une décision votée sur une volatilité imaginaire.
4. **L'ordre alphabétique.** 21 paires sur 28 à l'envers sur un simple jeu de 8 catégories. Un tri est toujours possible : par valeur, par temps ou par métier.
5. **La légende qui double le titre.** Deux séries, deux couleurs et une légende qui répète les libellés de l'axe : la place perdue est prise sur les données.
6. **Les chiffres estimés.** « Environ 350 M » dans un commentaire quand la valeur est 348,8 : le lecteur retiendra 350 et le répétera. Une valeur exacte s'annote.
7. **L'unité manquante.** Un axe « CA (M) » sans devise sera lu en euros par un lecteur, en milliers par un autre. L'unité s'écrit en toutes lettres dans l'étiquette.
8. **L'absence de source.** Sans source, le chiffre est inutilisable en réunion — il sera contesté, ou pire, accepté sans vérification.

9. Bonnes pratiques professionnelles

1. **Écrire le titre en dernier**, après avoir mesuré : c'est le verdict, pas l'intention de départ. Un titre écrit avant la mesure est une opinion.
2. **Mesurer l'axe avant de le tronquer** : quelle part de la hauteur les données occupent-elles ? Si la réponse est inférieure à un quart, la troncature est probablement nécessaire — et elle sera déclarée.
3. **Trier systématiquement**, et écrire dans la légende le critère de tri (« trié par CA décroissant ») : le lecteur qui connaît le critère peut vérifier.
4. **Annoter un chiffre par constat**, pas trois : l'annotation perd son effet dès qu'elle devient un tableau.

5. **Préférer le pourcentage au montant** dans l'annotation, et donner la référence quand on annonce un montant.
6. **Porter les quatre mentions** (source, période, unité, date d'extraction) dans la figure elle-même : c'est le seul endroit qu'un lecteur pressé regarde.
7. **Relire avec les familles 1 et 3 de la grille** (décision servie, lisibilité) avant publication : ce sont les deux familles qui se corrigent en cinq minutes et qui manquent le plus.

10. Exercice guidé — « le titre qui a fait voter » (15 min, /10)

Contexte. Un comité a voté une réorganisation après avoir vu la courbe du chiffre d'affaires « en dents de scie ». La courbe est celle du panneau A : axe tronqué de 307 à 350 M, titre « Évolution du chiffre d'affaires ».

Consigne. Dans les 15 minutes :

1. Expliquez **avec des chiffres** ce que l'axe a fait à la lecture (hauteur occupée, facteur d'amplification, variation réelle de la moyenne glissante). (2 pts)
2. Proposez le **titre** qui aurait évité le vote, en respectant la structure sujet-verdict-preuve. (2 pts)
3. Décrivez l'**axe** de la nouvelle version et la déclaration éventuelle à porter. (2 pts)
4. Choisissez **deux annotations** à porter sur la figure, avec leur texte exact. (2 pts)
5. Rédigez le **commentaire de trois lignes** qui accompagne la figure dans le rapport (ce que la figure dit, ce qu'elle ne dit pas, ce qu'elle recommande). (2 pts)

11. Exercices autonomes

E1 — l'audit d'écriture d'un rapport réel (30 min). Prenez un rapport existant (le vôtre ou un rapport reçu). Pour chaque graphique, relevez : le titre (et s'il porte une mesure), la présence du zéro sur l'axe des valeurs, l'ordre des catégories et son critère, le nombre d'annotations, la présence des quatre mentions. Rendez un tableau à cinq lignes et une conclusion chiffrée : combien de graphiques pourraient être cités sans commentaire en réunion ? Précisez la correction la moins chère et la plus rentable (souvent : ajouter les titres qui affirment).

E2 — la réécriture complète (30 min). Reprenez **un** des cinq graphiques du dossier « avant » et réécrivez-le entièrement : titre, axe, ordre, annotation, mentions, légende. Produisez les deux versions côte à côte, puis appliquez la grille en 18 points aux deux et comparez les totaux. Le critère de réussite n'est pas l'esthétique : c'est que la version refondue puisse être **citée** (quelqu'un doit pouvoir dire, en une phrase, ce que la figure affirme).

12. Correction détaillée

Exercice guidé.

1. **Ce que l'axe a fait** : sur un axe à zéro (0 → 350 M), les données occupent 11,8 % de la hauteur ; tronqué de 300 à 350 M, elles en occupent 82,4 %, soit une amplitude apparente multipliée par 7. Or la moyenne glissante ne varie que de **5,4 %** sur 24 mois (320 à 337 M) : la « dent de scie » est une fabrication de l'axe, et le vote porte sur cette fabrication.
2. **Titre proposé** : « Le CA est plat sur 24 mois : 5,4 % d'amplitude (moyenne glissante 320-337 M) » — sujet (le CA), verdict (est plat), preuve (5,4 %, bornes de la moyenne).

3. **Axe** : à zéro (0 → 350 M FCFA) pour la vue générale. Si l'on ajoute un zoom sur la période du pic, la troncature y est **déclarée** dans l'étiquette (« zoom : 320 à 350 M, axe non nul ») et le titre du zoom ne généralise pas (« le pic de juin 2026 : +6,3 % au-dessus de la moyenne »).
4. **Deux annotations** : « 337 M — pic de la moyenne glissante (2026-07) » sur le maximum, et « 320 M — creux (2026-02), soit 5,4 % d'écart » sur le minimum. Une troisième annotation possible porte la limite : « décembre 2026 partiel (28 jours) ».
5. **Commentaire de trois lignes** : « Sur 24 mois, le chiffre d'affaires net reste compris entre 307,6 et 348,8 M FCFA, la moyenne glissante ne s'écartant que de 5,4 % (320 à 337 M). La période ne montre donc ni croissance, ni volatilité : les variations mensuelles relèvent du bruit. En conséquence, la réorganisation votée le 20 septembre ne peut pas être justifiée par cette courbe. »

E1 (l'audit d'écriture). Grille de correction : le tableau est **complet** (cinq colonnes remplies pour chaque graphique), la conclusion est un compte (combien citable), et la correction proposée est chiffrée en effort (par exemple : « trois titres à réécrire, une source à ajouter, aucun graphique à refaire »). Repère du dossier du module : 0 titre sur 5 portait une mesure, 0 annotation, 1 axe à zéro sur 5, 0 mention de source — et la refonte n'a pris aucun graphique supplémentaire.

E2 (la réécriture complète). Trois attendus : la version refondue porte les **quatre** mentions ; son titre est **citable** (le correcteur doit pouvoir le répéter sans avoir vu la figure) ; la comparaison des grilles montre un gain sur les familles 1 et 3, sans perte sur la 2 (les valeurs du graphique sont inchangées : le module ne corrige pas un graphique en changeant la donnée).

13. Mini-projet M10.P4 — « les cinq graphiques réécrits » (1 h 30)

Énoncé. C'est l'étape d'écriture du projet « La refonte ». Vous reprenez les cinq graphiques du dossier (les « avant » que vous avez diagnostiqués en C05) et vous les réécrivez en appliquant, dans l'ordre, les six gestes du §7.

1. **Pour chaque graphique** : mesurez d'abord l'effet de l'axe (hauteur occupée, facteur d'amplification), puis écrivez le **titre** (sujet, verdict, preuve), l'**axe** (zéro, ou troncature déclarée), l'**ordre** (critère écrit dans la légende), l'**annotation** (un chiffre par constat) et les **quatre mentions**.
2. **Produisez les deux versions** du même constat : la version **direction** (un écran, l'idée et la décision, hiérarchie visuelle complète) et la version **équipe** (le détail, les libellés exacts, les séries complètes).
3. **Auto-évaluez** avec la grille en 18 points, puis faites relire par un pair. Recopiez les deux totaux et l'écart.

Barème indicatif : cinq graphiques réécrits avec les six gestes (8 points), les deux versions (direction et équipe) pour au moins trois graphiques (4 points), auto-évaluation et relecture par un pair avec les deux totaux (4 points), cohérence des chiffres avec les « avant » (les totaux ne changent pas, 2 points), sur 18.

Pourquoi ce mini-projet. C'est le cœur du projet « La refonte » et la démonstration de la thèse du module : **on ne change pas la donnée pour corriger un graphique**, on change son écriture. Les deux versions (direction et équipe) sont exigées partout dans le manuel : M14, M17 et la mission finale M21 reprennent ce couple.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise **6 gestes** d'écriture, tous exécutés dans la planche du §6 : `set_title` (avec retour à la ligne), `set_ylim` (bornes, donc axe honnête), `set_xticks` et `set_yticks` (libellés choisis, pas subis), `annotate` (le chiffre clé et sa flèche), `text` (les mentions de source) et `sort_values` (l'ordre des catégories).

Besoin	Commande	Ce qu'elle donne	Ce qu'elle ne donne pas
un titre qui affirme	<code>ax.set_title(f"... {val} ...")</code>	une phrase avec mesure	le verdict : il se décide en amont, après la mesure
un axe honnête	<code>ax.set_ylim(0, plafond)</code>	un zéro visible	la déclaration d'une troncature volontaire (à écrire à la main)
des libellés choisis	<code>ax.set_xticks(pos, libelles)</code>	des repères utiles (mois, seuils)	des libellés lisibles s'ils sont trop longs (rotation ou regroupement)
le chiffre clé	<code>ax.annotate(texte, xy=point, xytext=position)</code>	une valeur désignée	la décision de quel chiffre est clé
les mentions	<code>ax.text(...)</code> ou <code>fig.text(...)</code>	source, période, unité, date	l'exactitude du périmètre : elle vient de la fiche d'entrée (M09)
l'ordre	<code>serie.sort_values(ascending=False)</code>	un classement porteur de sens	le bon critère de tri (valeur, temps, métier)

15. Résumé du chapitre

C04 a transformé un graphique en **objet citable**. Quatre objets écrits, mesurés sur le dossier de refonte : le **titre** (0 sur 5 portait une mesure avant, 5 sur 5 après ; structure sujet-verdict-preuve), l'**axe** (tronqué de 300 à 350 M, les données occupent 82,4 % de la hauteur contre 11,8 % sur un axe à zéro — amplitude apparente $\times 7$ — pour une variation réelle de 5,4 %), l'**ordre** (l'ordre alphabétique classe 21 paires de catégories sur 28 à l'envers, soit 75 % des comparaisons que le lecteur fera de mémoire) et l'**annotation** (5 chiffres clés après refonte contre aucun avant, dont quatre pourcentages ou rapports). S'y ajoutent la **hiérarchie visuelle** (un chiffre principal, deux appuis, une phrase de lecture), la **légende qui ne double pas le titre** et les **quatre mentions obligatoires** : source, période, unité, date d'extraction. La sortie du chapitre est la réécriture des cinq graphiques du projet, dans leur version direction et leur version équipe.

16. À retenir

À retenir. Un graphique professionnel est un graphique qu'on peut **citer** : son titre énonce le constat, son axe dit sur quelles bornes on le lit, son annotation porte le chiffre qui décide, et ses mentions disent d'où il vient. Tout le reste — la mise en page, la couleur, la police — sert ces quatre objets, jamais l'inverse.

1. **Le titre affirme** : sujet, verdict, preuve — « Le CA est plat sur 24 mois : 5,4 % d'amplitude ». 0 sur 5 avant, 5 sur 5 après.
2. **L'axe est honnête** : zéro, ou troncature déclarée. La troncature de 307 à 350 M multiplie par 7 l'amplitude apparente (82,4 % de hauteur occupée contre 11,8 %).
3. **Les barres partent de zéro** : leur longueur **est** la valeur ; une courbe de niveau peut être tronquée si c'est écrit.

4. **L'ordre se choisit** : valeur, temps ou métier, jamais l'alphabet par défaut (21 paires sur 28 à l'envers sur huit catégories).
5. **Une annotation par constat** : le chiffre, sa nature, son repère — et de préférence un pourcentage ou un rapport plutôt qu'un montant nu.
6. **Un chiffre principal, deux appuis, une phrase** : la hiérarchie visuelle force à choisir ce que la figure affirme.
7. **Quatre mentions obligatoires** : source, période, unité, date d'extraction — une ligne de 8 points qui supprime les questions de traçabilité.
8. **La légende ne double pas le titre** : pas de légende pour une série unique, des libellés directs dès qu'il y a plusieurs séries.

17. Évaluation formative (auto-correction, 8 min)

1. **Réécrivez ce titre : « Évolution du chiffre d'affaires ». Quelle est la règle appliquée ?**
→ Un titre affirme : sujet, verdict, preuve — par exemple « Le CA est plat sur 24 mois : 5,4 % d'amplitude ». La règle : si le titre peut être remplacé par « voici des données », ce n'est pas un titre. (2 pts)
2. **Un axe tronqué est-il toujours une faute ? Comment mesurer son effet ?** → Non : c'est une option légitime pour une courbe de niveau, à condition d'être **déclarée**. On mesure la part de hauteur occupée (82,4 % tronqué contre 11,8 % à zéro) et le facteur d'amplification ($\times 7$ dans notre cas, pour une variation réelle de 5,4 %). (2 pts)
3. **Pourquoi les barres ne peuvent-elles jamais avoir un axe tronqué, alors qu'une courbe le peut ?** → Parce que la barre code sa valeur par sa **longueur** : rogner l'axe rogne la barre, et deux barres dans un rapport de 1,1 peuvent paraître dans un rapport de 3. La courbe code une **position** : sa forme reste lisible, à condition de déclarer les bornes. (2 pts)
4. **L'ordre alphabétique des catégories : quel est le coût mesuré, et quels ordres sont légitimes ?** → 21 paires sur 28 (75 %) classées à l'envers sur nos huit catégories. Trois ordres légitimes : valeur, temps, métier — et l'alphabet seulement quand le lecteur cherche une entrée dans une longue liste. (2 pts)
5. **Le dossier refondu annote « $r = +0,35$, un lien non concluant » dans son titre. Que fait cette formulation que « les retours suivent le CA » ne faisait pas ?** → Elle porte la **nuance** : au lieu d'affirmer une causalité que la corrélation ne soutient pas, le titre donne la mesure et son verdict (« non concluant »). C'est l'héritage de M09 : la limite fait partie du constat. (2 pts)
6. **Citez les quatre mentions obligatoires d'une figure et une conséquence de l'oubli de chacune.** → Source (chiffre non vérifiable), période (un mois partiel fausse la comparaison — décembre 2026 n'a que 28 jours), unité (lecture dans une unité au hasard, le défaut M12 de M09) et date d'extraction (un rapport qui circule se réfute par sa date). (2 pts)

Module M10.C05 — Les 10 erreurs qui tuent la crédibilité

Outils : matplotlib 3.10.9, pandas 2.2.3, numpy 2.3.5 (exécutés dans ce chapitre) ; Excel (exécuté, C07) ; Power BI (cité, non exécuté, C07). **Durée indicative :** 4 h. **Niveau :** N3. **Prérequis :** M10.C01 (la hiérarchie perceptuelle), M10.C02 (le catalogue), M10.C03 (couleurs et redondance) et M10.C04 (titres, axes, annotations).

L'idée du chapitre. Les quatre chapitres précédents ont appris à **choisir** et à **écrire** un graphique. Celui-ci apprend à **relire** : voici les dix défauts qui font perdre la crédibilité à une figure — chacun avec son **effet mesuré** sur le fil rouge, parce qu'un défaut non chiffré est une opinion. Un axe tronqué multiplie l'amplitude apparente par 7 pour une variation réelle de 5,4 % ; un double axe affiche une coïncidence de +0,35 entre deux séries sans lien ; un camembert à huit parts demande de distinguer 0,17° ; un cumul à 100 % masque des totaux qui varient de 5,4 % ; une échelle logarithmique non déclarée donne 93,5 % de la largeur à la masse des petits montants alors qu'ils n'en occupent que 42,1 % en linéaire. Et surtout : **la règle du module** — une erreur n'est pas un péché, c'est une **décision**. Elle devient légitime dès qu'elle est **déclarée**.

Matériel de l'atelier — Python 3.13 · matplotlib 3.10.9 · pandas 2.2.3 · numpy 2.3.5. Tous les effets de ce chapitre sont mesurés par `tools/erreurs_M10.py` sur le socle M09 (les 50 008 ventes, 24 mois, 8 catégories, 5 modes de paiement) et sur le dossier de refonte du module ; la planche `figures/M10_C05_mur_des_10_erreurs.svg` montre chaque défaut **et** sa version corrigée (deux exécutions, même fichier).

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Nommer les 10 erreurs** — axe tronqué, double axe, 3D, camembert surchargé, cumul de pourcentages, couleur à tout faire, surcharge, zéro manquant ou ajouté, ordre alphabétique, échelle logarithmique surprise — et donner **l'effet mesuré** de chacune sur le fil rouge.
2. **Distinguer l'erreur du choix** : un axe tronqué déclaré, une échelle logarithmique annoncée ou un camembert à deux parts ne sont pas des fautes ; c'est **l'écart entre ce que la figure laisse croire et ce que les données disent** qui fait le défaut.
3. **Chiffrer un défaut** avant d'en parler : hauteur occupée, facteur d'amplification, corrélation affichée, écart minimal entre deux parts, largeur de barre en linéaire contre logarithmique.
4. **Relire un graphique en 60 secondes** avec une check-list de six questions, dans l'ordre de la grille en 18 points — et non dans l'ordre de la beauté.
5. **Diagnostiquer les cinq défauts du rapport de la quincaillerie** avec leurs mesures, ce qui est la première étape notée du projet « La refonte ».

2. Pourquoi cette notion est importante

Un graphique faux se repère : il ne correspond pas aux données. Un graphique **trompeur** est bien plus dangereux, parce qu'il est exact — et qu'il déclenche donc des décisions que personne ne remettra en cause. Le dossier du module en donne l'exemple de bout en bout : cinq graphiques, **aucun chiffre faux**, et une direction qui vote une réorganisation parce qu'une courbe plate a été présentée sur un axe qui la rendait nerveuse.

Le chapitre est donc une liste de défauts, mais son message n'est pas moral : c'est une question de **contrat**. Un graphique laisse toujours croire quelque chose de plus que ce qu'il montre — la pente, la hiérarchie, la coïncidence, la masse. Le défaut apparaît quand cette croyance supplémentaire est **fausse** et **non déclarée**. D'où la formulation que le module retient pour tout le reste du manuel : une erreur n'est pas un péché, c'est une décision ; elle est légitime si elle est déclarée.

Cela change la façon de travailler. Un analyste qui tronque volontairement un axe pour montrer un pic locale **n'a rien à se reprocher** : il écrit « axe de 320 à 350 M » dans le titre du zoom, et le lecteur sait exactement ce qu'il voit. Le même analyste qui laisse l'axe tronqué en silence produit une figure que six personnes citeront de travers. La différence entre les deux n'est pas de talent : c'est une ligne de texte.

Dans les faits. Les dix défauts ne sont pas rares : ils sont la **production par défaut** des outils. La 3D est un bouton, l'ordre alphabétique est un tri par défaut, l'échelle logarithmique s'active en un clic, le camembert accepte huit parts sans avertir. Retenir la liste, c'est donc savoir la reconnaître **avant** de publier — et connaître la ligne de texte qui transforme un défaut en décision.

3. Explication simple — la visite médicale du graphique

Un graphique publié devrait passer une visite médicale en six gestes, du plus grave au plus bénin :

1. **L'échelle** — la figure coupe-t-elle l'axe, ou le zéro manque-t-il ? Le doute est immédiat : une barre qui ne part pas de zéro, une courbe qui commence à mi-hauteur. C'est le geste qui vient en premier parce que c'est celui qui change le **verdict**.
2. **Le nombre d'axes** — y a-t-il deux échelles dans un même cadre ? Si oui, quelle relation la figure suggère-t-elle entre les deux séries ?
3. **La troisième dimension** — y a-t-il de la profondeur ? Si oui, pour quelle donnée ? En général, pour aucune : la profondeur ne porte pas de variable.
4. **Les parts** — combien de parts dans le camembert ? Combien de pourcentages empilés à 100 %, et quel total chacun masque-t-il ?
5. **Les couleurs** — chaque couleur signifie-t-elle une seule chose, et reste-t-elle lisible en noir et blanc ?
6. **L'ordre et la densité** — les catégories sont-elles triées ? Combien de séries, de nuances et de niveaux de données le cadre contient-il ?

Les six gestes tiennent en une minute, et la règle est simple : **ce que la visite suspecte, elle le mesure**. Une intuition (« ça a l'air tronqué ») ne vaut rien en réunion ; la phrase qui vaut est « l'axe occupé à 82 % de la hauteur pour une variation de 5,4 % ». C'est cette phrase qui arrête un vote.

4. Vocabulaire essentiel

Terme	Définition
graphique trompeur	une figure aux chiffres exacts qui laisse pourtant croire autre chose que ce que les données disent. Ce n'est pas un mensonge : c'est un défaut non déclaré.
déclaration	la mention, dans la figure, de la convention employée (axe tronqué, échelle logarithmique, données incomplètes). C'est la déclaration qui transforme une erreur en décision.
double axe	deux échelles verticales dans un même cadre. Utile à de rares cas (deux unités de même nature), toxique dès qu'il fait coïncider deux séries sans lien.

Terme	Définition
effet 3D	l'ajout d'une profondeur qui ne porte aucune donnée : elle déforme les hauteurs perçues et fausse la comparaison.
cumul de pourcentages	l'empilement de parts à 100 % : correct pour lire une composition, trompeur dès qu'il masque les totaux , qui peuvent varier fortement.
zéro ajouté	l'ajout d'une plage vide en bas d'un axe (0 → 350 M quand les données occupent 307 à 349) : elle écrase la variation réelle et fait paraître une différence significative comme négligeable.
échelle logarithmique	une échelle où les graduations suivent les puissances (10, 100, 1 000). Elle rend visibles les petites valeurs et écrase les grandes : à déclarer impérativement.
surcharge	l'accumulation de séries, de couleurs, de valeurs et de cadres dans une même figure : le graphique contient tout et ne communique rien.
lecture en 60 secondes	la relecture systématique d'un graphique avec les six questions du §5.7, faite avant publication, par son auteur puis par un pair.

5. Cours approfondi

5.1 Les 10 erreurs, leur effet mesuré, et leur déclaration

Voici le tableau de référence du chapitre. La colonne « déclaration » dit la ligne de texte qui rend l'usage légitime — c'est la colonne qu'on oublie, et c'est celle qui distingue un professionnel d'un producteur de figures.

#	L'erreur	L'effet, mesuré sur le fil rouge	Déclaration qui rend l'usage légitime
1	Axe tronqué	axe 300-350 M : 82,4 % de hauteur occupée pour 11,8 % en vue complète → amplitude apparente x 7,0 pour une variation réelle de 5,4 %	« axe de 320 à 350 M (non nul) » dans le titre ; et jamais de troncature pour des barres
2	Double axe	corrélation affichée +0,35 entre CA et retours, deux séries sans lien métier (4 à 16 retours par mois)	« deux échelles : la coïncidence visuelle ne prouve aucun lien » — ou, mieux : deux graphiques séparés
3	Effet 3D	la profondeur ajoute un axe qui ne porte aucune donnée (le socle n'en produit aucun)	aucune : c'est le seul des dix défauts sans usage professionnel
4	Camembert surchargé	8 parts ; écart minimal entre deux voisines 0,05 pt = 0,17° ; lisible jusqu'à 3 parts	« camembert à 3 parts ; les 5 autres regroupées »
5	Cumul de pourcentages	8 trimestres empilés à 100 % : les totaux varient de 5,4 % (960 721 755 à 1 012 518 068) alors que la part d'un mode ne bouge que de 1,9 pt	annoncer le total de chaque barre en étiquette
6	Couleur à tout faire	3 couleurs pour 5 graphiques ; l'orange porte 3 rôles en 7 emplois	« une couleur, un rôle » : la charte (C03)
7	Surcharge	le graphique des montants superpose 3 séries et 40 classes, soit 120 éléments dans un cadre (limite du module : 5 séries)	découper en petits multiples (une facette par série)
8	Zéro manquant / ajouté	ajouter 0 à un axe qui commence à 88,2 % de sa longueur dilue la figure : la plage vide occupe 0,3 % de l'axe... mais prive la lecture du rapport entre deux barres	« axe à zéro : les barres se comparent par leur longueur »
9	Ordre alphabétique	8 catégories : 21 paires sur 28 (75 %) classées à l'envers par rapport aux valeurs	« trié par CA décroissant » dans la légende
10	Échelle logarithmique surprise	77,0 % des ventes sont sous 250 000 FCFA : en linéaire, cette masse occupe 42,1 % de la largeur ; en log, 93,5 % . La queue (2,1 % des ventes) passe de 15,9 % de largeur à 1,3 %	« échelle logarithmique : les hauteurs ne s'additionnent pas »

Trois lectures de ce tableau.

Première lecture : les défauts 1, 2 et 9 changent une décision. Ils ne dégradent pas l'esthétique, ils modifient le verdict : la platitude devient une crise ($\times 7$), deux séries indépendantes deviennent liées (+0,35), le quatrième devient le premier (21 paires sur 28 à l'envers). Ce sont les trois défauts éliminatoires du module.

Deuxième lecture : les défauts 4, 5, 7 et 10 changent ce qu'on peut lire. Ils ne mentent pas, ils rendent illisible : une part de 0,17°, un total masqué à 5,4 %, 120 éléments dans un cadre, une queue réduite de 15,9 % à 1,3 % de largeur. Ce sont des pertes d'information — mesurables, donc corrigibles.

Troisième lecture : les défauts 3 et 6 changent le message. La 3D et la couleur à tout faire n'ajoutent aucune information : elles en **suggèrent** une fausse (une profondeur, une catégorie). Ce sont les deux défauts qu'aucune déclaration ne sauve : la seule correction est de les retirer.

5.2 L'axe tronqué et le zéro : la question du rapport

C'est le défaut le mieux documenté et le plus fréquent, parce qu'il vient d'une bonne intention : rendre visible une petite variation. Deux situations se ressemblent et n'ont pas le même traitement.

Le rapport des barres. Une barre code une quantité par sa **longueur** : le rapport entre deux barres est ce que l'œil lit. Une barre de 1 231 M et une de 697 M dans un rapport de 1,77 deviennent, sur un axe tronqué à 690, un rapport apparent de plusieurs unités. La règle du module est donc absolue : **les barres partent de zéro**. Si la variation à montrer est trop faible pour être visible sur une échelle de zéro, alors ce n'est pas un graphique en barres qu'il faut — c'est une courbe, un écart à la moyenne, ou un tableau.

Le niveau des courbes. Une courbe code une **position** : sa forme reste lisible sur un axe tronqué, à condition que le lecteur connaisse les bornes. La déclaration (« axe de 320 à 350 M ») est donc la bonne réponse — mais elle implique une seconde obligation : **le titre du zoom ne généralise pas**. Un zoom sur quatre mois ne justifie pas un titre qui parle de la période entière.

Le zéro ajouté, symétriquement, est un défaut plus discret : ajouter une plage vide en bas d'un axe peut « enterrer » une variation réelle. Sur le fil rouge, l'axe à zéro occupe 350 M pour des données qui en occupent 307 à 349 : 11,8 % de hauteur. C'est **la vérité** de ce jeu (la série est plate) — mais c'est un choix qui **doit s'accompagner du chiffre**, sinon le lecteur conclut « rien ne bouge » alors que la moyenne glissante varie de 5,4 %.

À retenir. Un défaut se **mesure** avant de se corriger : « ce graphique est trompeur » est un avis, « 82,4 % de hauteur occupée pour 5,4 % de variation réelle » est un constat. Le chiffre est ce qui transforme une discussion de goût en décision d'atelier.

Conseil professionnel. Le test de l'axe, en trois secondes : « le rapport entre les deux éléments les plus extrêmes, le lecteur le voit-il ? » Si la réponse dépend de la longueur coupée — et non de la donnée — l'axe manque son contrat. Ce test vaut pour les barres ; pour une courbe, remplacez-le par : « les bornes de l'axe sont-elles écrites quelque part ? ».

5.3 Le double axe : la machine à fabriquer des liens

Le double axe mérite sa réputation : c'est le seul défaut qui **crée** une information. En choisissant les deux fenêtres verticales, l'auteur décide si les deux courbes se superposent, se croisent ou divergent. Sur le fil rouge, le chiffre d'affaires net (307,6 à 348,8 M) et le nombre de retours (4 à 16 par mois) affichent une corrélation de **+0,35** — un lien faible, sans mécanisme métier, et pourtant spectaculaire dès que la seconde échelle est calée pour faire suivre les deux courbes.

Trois questions départagent l'usage légitime de l'abus :

1. **Les deux séries partagent-elles une unité ou une nature ?** Deux températures (air et sol), deux monnaies sur une même période, une valeur et sa cible : légitime. Un montant et un **comptage** : non.
2. **La relation est-elle le message ?** Si oui, on ne la montre pas par superposition mais par un **nuage de points** (la corrélation se lit sur les points, pas sur le calendrier) — C02.
3. **Le lecteur peut-il connaître les deux bornes ?** Si les deux axes n'ont pas la même longueur ni la même unité, la superposition est un artifice visuel.

Dans les rares cas où le double axe se justifie, la déclaration est obligatoire : les deux unités sur les axes, la corrélation écrite **dans la figure**, et la mention « la coïncidence visuelle ne prouve aucun lien ». La règle du module reste : **quand deux séries doivent être comparées, elles méritent deux cadres** — c'est d'ailleurs ce que fait le dossier refondu (panneau « 2 bis »).

À retenir. Aucun des dix défauts ne se voit à l'œil nu : le double axe cache une corrélation affichée de +0,35, le cumul à 100 % cache une variation de 5,4 %, le log déplace 51,4 points de largeur (42,1 % → 93,5 %). La visite des six gestes sert exactement à cela : **chiffrer** ce que l'œil soupçonne.

5.4 Camembert, cumul, treemap : les défauts de la composition

Les graphiques de composition concentrent quatre défauts sur dix. C'est logique : ils sont les plus séduisants et les moins précis (angle et surface, C01).

- **Le camembert surchargé** : la question à se poser n'est pas « combien de parts puis-je dessiner » mais « **combien de parts le lecteur peut-il comparer** ». Réponse mesurée : jusqu'à 3. Au-delà, la donnée existe, l'information n'arrive pas (0,05 pt = 0,17° pour nos deux catégories les plus proches). La correction n'est pas de « regrouper les petites parts » au hasard, mais de revenir à la question : s'il s'agit d'un classement, des barres triées ; s'il s'agit d'un tout, trois familles.
- **Le cumul de pourcentages** : une barre empilée à 100 % normalise chaque période, donc **efface les totaux**. Sur nos huit trimestres, la part du mode 4 ne bouge que de **1,9 point** — le lecteur conclut « rien ne change » — alors que les totaux trimestriels varient de **5,4 %**, soit un rapport de 2,88 entre les deux amplitudes. La déclaration est simple et suffisante : **écrire le total de chaque barre** en étiquette, ou à défaut dans une ligne de tableau sous la figure.
- **Le treemap** : il hérite des défauts du camembert (surface) sans en avoir la familiarité. À réserver à la question « où est la masse ? » sur un grand nombre d'entrées (C02).
- **L'entonnoir** : à surveiller quand les étapes ne se suivent pas dans le temps (un entonnoir de conversion n'est pas une suite de totaux) — le graphique suggère alors une chronologie inexistante.

Attention. Regrouper les petites parts d'un camembert en « autres » est une correction fréquente... et souvent fausse : une catégorie « autres » de 25 % regroupe des entités qui n'ont rien en commun, et le lecteur ne peut plus rien vérifier. La bonne correction est de changer de **question** (montrer l'étendue, pas le classement) : c'est le principe posé en C01 et repris par le projet.

5.5 Surcharge, couleur et ordre : les défauts d'écriture

Ces trois défauts ne touchent pas la structure du graphique : ils touchent sa **densité de lecture**.

La surcharge se mesure : nombre de séries, nombre de niveaux, nombre de couleurs, nombre d'éléments. Notre graphique des montants superpose trois séries (montant TTC, remise, prix unitaire) sur 40 classes, soit 120 éléments dans un cadre — cinq fois la limite du module (5 séries). Le remède n'est pas de réduire les données (elles sont légitimes) mais de **découper** : trois petits multiples, un par série, échelle commune. La règle : une idée par cadre.

La couleur à tout faire a été mesurée en C03 : trois couleurs pour cinq graphiques, et l'orange portant trois rôles. Coût : le lecteur ne peut pas construire de contrat de lecture, donc il se trompe au cadre suivant. Ce défaut est **invisible** en réunion (personne ne dit « je ne sais plus ce que signifie l'orange ») et coûteux.

L'ordre alphabétique est le défaut le plus facile à corriger et le plus fréquent : mesure 21 paires sur 28 classées à l'envers sur huit catégories (C04). Un tri est toujours possible — par valeur, par temps ou par métier.

Attention. Un défaut corrigé peut en créer un autre : passer d'un camembert surchargé à des barres triées change la question (du tout vers le classement), et passer d'un cumul à 100 % à des totaux empilés en valeur absolue rend les comparaisons de structure difficiles. La correction se choisit donc **avec** la question du commanditaire (C01), pas seulement contre le défaut.

5.6 L'échelle logarithmique : la plus utile et la plus dangereuse

L'échelle logarithmique n'est pas une erreur : c'est **le** remède aux distributions étalées comme la nôtre, où la médiane vaut **119 482** FCFA, le maximum **594 363** (un rapport de **5** pour 1), et où 77,0 % des ventes se situent sous 250 000 FCFA. En linéaire, cette masse occupe **42,1** % de la largeur de l'axe et la queue (2,1 % des ventes) **15,9** % ; en logarithmique, la masse occupe **93,5** % et la queue **1,3** %. Autrement dit : l'échelle change complètement ce qu'on voit — c'est un outil, pas un réglage.

Ce qui en fait un défaut, c'est le **silence**. Trois conséquences à connaître :

1. **Les hauteurs ne s'additionnent plus.** Sur une échelle log, chaque décade occupe la même hauteur : un écart de 500 000 FCFA près du million paraît aussi grand qu'un écart de 50 000 FCFA près de la centaine de milliers — un graphique en aires empilées en log n'a donc aucun sens.
2. **Les petites valeurs paraissent énormes.** Un pic à 5 ventes (sur une journée creuse) occupe visuellement la moitié du cadre : le lecteur voit un événement là où il n'y a que du bruit.
3. **Le zéro n'existe pas.** Un histogramme en log doit tronquer les valeurs nulles — et le dire.

D'où la règle du module : **une échelle logarithmique s'annonce dans le titre ou dans l'étiquette de l'axe**, avec la phrase qui protège le lecteur : « échelle logarithmique : les hauteurs ne s'additionnent pas ». C'est exactement ce que fait le graphique refondu du dossier — dont le titre commence par « Échelle linéaire annoncée : 77 % des ventes sous 250 000 FCFA (zoom séparé sur la queue) ».

Définition. L'échelle logarithmique place les graduations sur les puissances d'un facteur ($\times 10$ en général) : chaque décade occupe la même hauteur. Elle sert à montrer simultanément une masse de petites valeurs et une queue de grandes valeurs ; elle interdit en revanche d'additionner les hauteurs et rend le zéro impossible. Toute figure en log doit le dire.

5.7 Relire un graphique en 60 secondes

La check-list du module, à passer dans cet ordre — du plus grave au plus cosmétique :

1. **La question** : quelle question cette figure sert-elle seule ? (Si aucune : arrêt immédiat.)

2. **L'échelle** : zéro présent, ou troncature déclarée ? Les barres partent-elles de zéro ?
3. **Les liens** : y a-t-il un double axe, une échelle log, un cumul à 100 % ? Chacun est-il déclaré ?
4. **Le chiffre clé** : est-il annoté, et est-ce le bon ?
5. **La densité** : combien de séries, de couleurs, de niveaux ? Une idée par cadre ?
6. **L'ordre et la source** : catégories triées (critère écrit) ? Source, période, unité, date ?

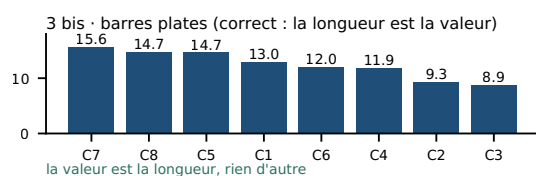
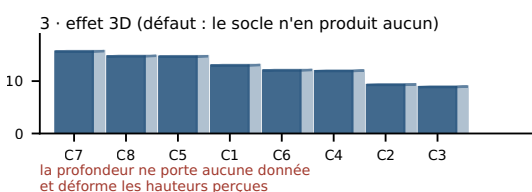
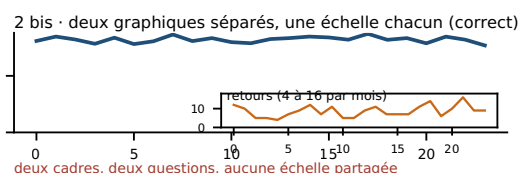
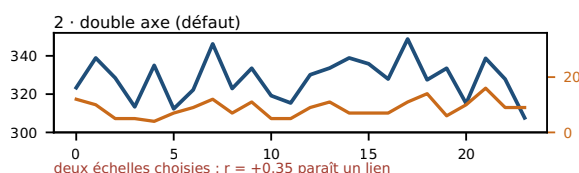
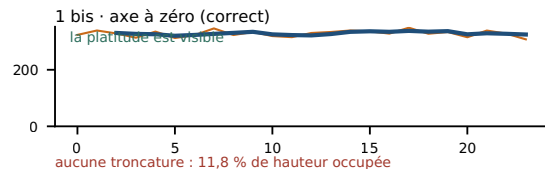
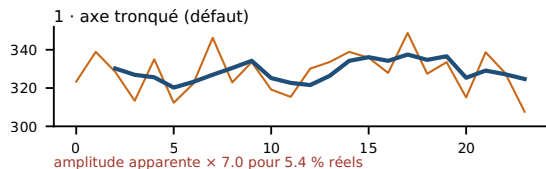
Le test de la relecture n'est pas « est-ce joli », mais : « **qu'est-ce que quelqu'un pourrait dire de faux devant cette figure ?** » Si vous trouvez la phrase fautive possible, corrigez la figure ou écrivez la déclaration — dans cet ordre d'effort. Et pour les cinq défauts du dossier (l'axe tronqué, le double axe causal, le camembert à huit parts, le cumul à 100 %, l'échelle log non déclarée), la relecture est notée : c'est la première étape du projet « La refonte ».

Définition. La **relecture en 60 secondes** est un rituel, pas un talent : six questions, un minuteur, et la règle que tout défaut trouvé s'écrit soit dans la figure (déclaration), soit dans la liste des corrections (refonte). Un graphique qui n'a pas passé la relecture n'est pas prêt à être publié — même s'il est beau.

6. Exemple concret — le mur des 10 erreurs, mesuré sur le fil rouge

La planche du chapitre montre, pour les défauts qui le permettent, la version fautive **et** la version corrigée du même graphique : c'est la meilleure façon de comprendre que le défaut ne change jamais la donnée, seulement la lecture.

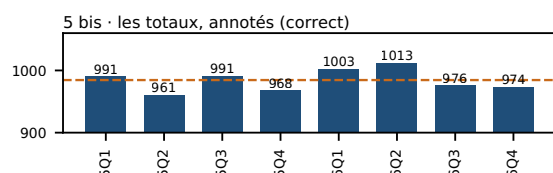
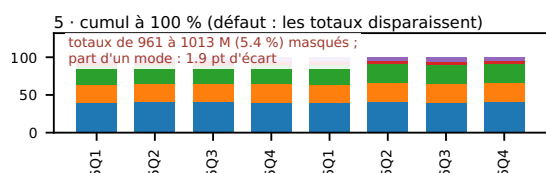
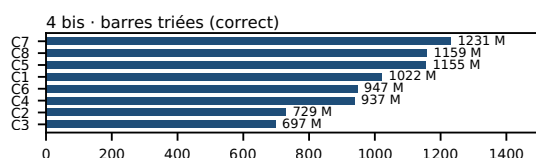
Le mur des 10 erreurs : chaque défaut avec son effet mesuré sur le fil rouge



4 - camembert surchargé (défaut)



8 parts ; écart minimal 0.05 pt = 0.17° : illisible



Ce que la planche démontre, panneau par panneau :

- **1 et 1 bis — l'axe.** À gauche, l'axe de 300 à 350 M : la série plate devient nerveuse (82,4 % de hauteur occupée, amplitude apparente $\times 7,0$). À droite, le même mois dans le même ordre sur un axe à zéro : la platitude est **visible**, et la déclaration « aucune troncature : 11,8 % de hauteur occupée » dit pourquoi.
- **2 et 2 bis — le double axe.** À gauche, deux échelles côte à côte : les deux courbes semblent solidaires ($r = +0,35$). À droite, deux cadres, une échelle chacun : le lecteur voit que les retours oscillent entre 4 et 16 par mois pendant que le CA reste dans une bande étroite — ce que le double axe lui cachait.
- **3 et 3 bis — la 3D.** À gauche, la profondeur déforme des hauteurs qui sont justement l'objet de la comparaison ; à droite, les barres plates portent leurs valeurs. Le socle ne produit **aucun** graphique en 3D : c'est une règle, pas une mesure.
- **4 et 4 bis — le camembert.** À gauche, huit parts où deux voisines sont séparées de $0,17^\circ$; à droite, les barres triées avec les valeurs en millions sur chaque barre.
- **5 et 5 bis — le cumul.** À gauche, huit trimestres empilés à 100 % (la part d'un mode bouge de 1,9 point) ; à droite, les totaux trimestriels annotés, de 961 M à 1 013 M : la variation de 5,4 % que le cumul masquait.

Définition. Le **diagnostic mesuré** est la première étape notée du projet « La refonte » : il nomme les défauts d'un rapport, **mesure** leur effet sur les données, et écrit la décision fautive que chacun peut déclencher. Un diagnostic sans mesure est une opinion — et une opinion ne se corrige pas.

Définition. On appelle **faux citable** la phrase fautive qu'un lecteur pourrait prononcer devant une figure exacte (« la volatilité explose », « les retours suivent le CA »). C'est l'objet de la relecture : rendre cette phrase impossible.

7. Démonstration pas à pas — mesurer un défaut en 5 gestes

Geste 1 — l'axe. On mesure la hauteur occupée et le facteur d'amplification.

```
import pandas as pd, numpy as np
v = pd.read_csv("03_exercices/dossier_M09/quincailerie/vente.csv", parse_dates=["date_vente"])
ca = v[~v["est_retour"]].groupby(v["date_vente"].dt.to_period("M"))["montant_ttc"].sum() / 1e6
print(f"donnees {ca.min():.1f} -> {ca.max():.1f} M")
print(f"hauteur occupee : {100*(ca.max()-ca.min())/350:.1f} % (axe a zero) "
      f"contre {100*(ca.max()-ca.min())/50:.1f} % (axe 300-350) "
      f"-> x {350/50:.1f}")
mg3 = ca.rolling(3).mean()
print(f"variation reelle de la moyenne glissante : {(mg3.max()-mg3.min())/mg3.min()*100:.1f} %")
```

```
donnees 307.6 -> 348.8 M
hauteur occupee : 11.8 % (axe a zero) contre 82.4 % (axe 300-350) -> x 7.0
variation reelle de la moyenne glissante : 5.4 %
```

Geste 2 — le double axe. On vérifie si les deux séries sont réellement liées.

```
ret = v.groupby(v["date_vente"])[["est_retour"]].sum()
ret = ret.groupby(ret.index.to_period("M")).sum().reindex(ca.index).fillna(0)
print("correlation CA / retours :", round(float(ca.corr(ret)), 2),
      "| retours de", int(ret.min()), "a", int(ret.max()), "par mois")
```

correlation CA / retours : 0.35 | retours de 4 a 16 par mois

Geste 3 — le camembert. On mesure l'écart minimal entre deux parts voisines.

```
p = pd.read_csv("03_exercices/dossier_M09/quincaillerie/produit.csv")
vp = v[~v["est_retour"]].merge(p[["id_produit", "id_categorie"]], on="id_produit", how="left")
part = (vp.groupby("id_categorie")["montant_ttc"].sum() / vp["montant_ttc"].sum() *
100).sort_values()
e = np.diff(part.to_numpy())
print(f"{len(part)} parts ; ecart minimal {e.min():.2f} pt = {e.min()*3.6:.2f} deg")
```

8 parts ; ecart minimal 0.05 pt = 0.17 deg

Geste 4 — le cumul de pourcentages. On compare les totaux que le cumul masque et la variation des parts qu'il affiche.

```
net = v[~v["est_retour"]]
trim = net.groupby(net["date_vente"].dt.to_period("Q"))["montant_ttc"].sum()
modes = (net.assign(trim=net["date_vente"].dt.to_period("Q"))
.pivot_table(index="trim", columns="id_mode", values="montant_ttc", aggfunc="sum"))
parts = modes.div(modes.sum(axis=1), axis=0) * 100
amp_part = float((parts.max(axis=0) - parts.min(axis=0)).max())
print(f"totaux de {trim.min():,} a {trim.max():,} -> {100*(trim.max()-trim.min())/trim.min():.1f} %")
.replace(",", " ")
print(f"amplitude d'un mode : {amp_part:.1f} pt (mode {int((parts.max(axis=0)-
parts.min(axis=0)).idxmax())})")
```

totaux de 960 721 755 a 1 012 518 068 -> 5.4 %
amplitude d'un mode : 1.9 pt (mode 4)

Geste 5 — l'échelle logarithmique. On mesure ce que le log change aux largeurs occupées.

```
m = v["montant_ttc"]
masse, queue = 250000.0, 500000.0
print(f"{100*(m < masse).mean():.1f} % des ventes sous 250 000 FCFA : largeur "
f"{masse/m.max()*100:.1f} % en lineaire contre "
f"{np.log10(masse)/np.log10(m.max())*100:.1f} % en log")
print(f"queue (>{int(queue)} FCFA, {100*(m > queue).mean():.1f} % des ventes) : "
f"{100*(m.max()-queue)/m.max():.1f} % en lineaire contre "
f"{100*(np.log10(m.max())-np.log10(queue))/np.log10(m.max()):.1f} % en log")
```

77.0 % des ventes sous 250 000 FCFA : largeur 42.1 % en lineaire contre 93.5 % en log
queue (>500000 FCFA, 2.1 % des ventes) : 15.9 % en lineaire contre 1.3 % en log

Ce que la démonstration établit. Cinq défauts, cinq mesures, aucune appréciation. C'est la discipline que ce chapitre installe : **avant de dire « ce graphique est trompeur », dire de combien.** Un défaut chiffré se corrige ; un défaut ressenti se discute pendant une heure.

8. Erreurs fréquentes

1. **Confondre l'erreur et le choix.** Un axe tronqué déclaré, une échelle log annoncée, un camembert à deux parts : trois choix légitimes. Le défaut, c'est l'écart non déclaré entre ce que la figure laisse croire et ce qu'elle montre.
2. **Croire qu'un défaut se voit.** Le double axe ne se voit pas : il se **mesure** (corrélation affichée). Le cumul à 100 % ne se voit pas : il masque une variation de 5,4 % que personne ne soupçonne.

3. **Corriger en changeant la donnée.** Regrouper des catégories pour sauver un camembert, lisser une série pour sauver une courbe : le module refuse ces corrections. On change la **question** ou la **figure**, jamais la donnée.
4. **Le zéro systématique.** Tout axe ne doit pas partir de zéro : une courbe de prix ou de température se lit très bien tronquée, si c'est écrit. Le dogme inverse produit des figures illisibles.
5. **Le log par défaut.** Sur une distribution étalée, le log aide — mais un histogramme en log où l'on additionne mentalement les hauteurs produit un contresens total.
6. **La 3D décorative.** Aucun usage professionnel : elle déforme les comparaisons qui sont justement l'objet du graphique.
7. **Relire en cherchant le beau.** La relecture cherche le **faux citable** : « qu'est-ce que quelqu'un pourrait dire de faux devant cette figure ? ». La beauté n'a jamais fait échouer un comité ; une phrase fausse, si.

9. Bonnes pratiques professionnelles

1. **Faire la visite des six gestes avant publication** (§3), et **chiffrer** ce qui est suspect.
2. **Écrire la déclaration dès qu'on s'écarte d'une convention** : axe non nul, échelle log, données incomplètes, périmètre restreint. Une ligne dans la figure, pas une note de bas de page.
3. **Ne jamais tronquer un axe de barres**, et ne jamais généraliser un titre de zoom.
4. **Deux séries à comparer = deux cadres**, pas deux échelles. Le double axe reste réservé aux unités de même nature.
5. **Écrire le total quand on normalise** : toute barre empilée à 100 % porte son total en étiquette.
6. **Une idée par cadre**, cinq séries au maximum ; au-delà, des petits multiples à échelle commune.
7. **Relire en binôme** : l'auteur connaît sa figure trop bien pour voir ce qu'elle laisse croire. La relecture par un pair est un point de la grille (famille 4), pas une politesse.

10. Exercice guidé — « l'audit des cinq défauts » (15 min, /10)

Contexte. Vous recevez le rapport commercial de la quincaillerie : cinq graphiques, des chiffres justes, et une direction qui a déjà tiré des conclusions. C'est la première étape notée du projet « La refonte ».

Consigne. Dans les 15 minutes, pour les trois défauts les plus critiques du dossier :

1. Nommez le défaut de chaque graphique concerné, avec le **numéro** de la liste du §5.1. (2 pts)
2. Donnez **l'effet mesuré** de chacun (les chiffres du chapitre : $\times 7,0$; $+0,35$; $0,17^\circ$; $5,4\%$; $93,5\%$ contre $42,1\%$). (3 pts)
3. Pour chacun, écrivez la **décision fausse** qu'il peut déclencher, en une phrase. (2 pts)
4. Écrivez la **déclaration** qui rendrait l'usage légitime — ou expliquez pourquoi aucune déclaration ne suffit. (2 pts)
5. Classez les trois défauts du plus grave au moins grave, avec votre critère (par exemple : change une décision / rend illisible / suggère une fausse information). (1 pt)

11. Exercices autonomes

E1 — le musée des horreurs personnelles (30 min). Rassemblez trois graphiques que vous avez produits (ou reçus) et qui vous ont laissé un doute. Pour chacun, appliquez la visite des six gestes et **mesurez** ce que vous suspectez : hauteur occupée, corrélation affichée, écart minimal entre parts, totaux masqués par un cumul, largeurs en linéaire contre log. Rendez un tableau : graphique, défaut, mesure, déclaration possible, correction. Un défaut non chiffré sera refusé.

E2 — la déclaration rédigée (25 min). Prenez un graphique que vous **voulez** garder tronqué ou logarithmique (c'est légitime). Écrivez la déclaration complète : titre du zoom ou de la vue, étiquette d'axe, phrase de protection du lecteur (« les hauteurs ne s'additionnent pas »), et la limite de validité (« ce zoom ne dit rien de la période entière »). Puis faites relire : la personne doit pouvoir dire, sans vous, ce que la figure montre **et** ce qu'elle ne montre pas.

12. Correction détaillée

Exercice guidé.

- Les défauts** : axe tronqué (n° 1) sur l'« Évolution du chiffre d'affaires » ; double axe (n° 2) sur « Les retours suivent le chiffre d'affaires » ; camembert surchargé (n° 4) sur la répartition par catégorie — et, dans les deux graphiques restants, cumul de pourcentages (n° 5) et échelle logarithmique surprise (n° 10).
- Effets mesurés** : axe — 82,4 % de hauteur occupée contre 11,8 %, amplitude apparente **× 7,0** pour une variation réelle de 5,4 % ; double axe — corrélation affichée **+0,35** entre des séries qui n'ont pas de lien métier (4 à 16 retours par mois) ; camembert — huit parts et **0,17°** entre les deux plus proches ; cumul — totaux variant de **5,4 %** (960 721 755 à 1 012 518 068) masqués par des parts qui ne bougent que de 1,9 point ; log — la masse (77,0 % des ventes) passe de **42,1 % à 93,5 %** de largeur, la queue de 15,9 % à 1,3 %.
- Décisions fausses** (une par défaut) : « la volatilité augmente, réorganisons » ; « les retours suivent le CA, agissons sur le CA pour réduire les retours » ; « voici notre classement de catégories » sur des parts indiscernables ; « la structure des paiements ne bouge pas » ; « les montants sont homogènes » (le log gomme la queue, où se trouvent les 1 044 ventes de plus de 500 000 FCFA).
- Déclarations** : axe — « axe de 320 à 350 M (non nul) », en gardant un titre local ; double axe — aucune : deux cadres ; camembert — « trois parts ; les cinq autres regroupées », aucune si la question est un classement (changer de figure) ; cumul — « total de chaque barre en étiquette » ; log — « échelle logarithmique : les hauteurs ne s'additionnent pas ».
- Classement attendu** (critère : ce que le défaut change) : 1) double axe (il **crée** un lien inexistant et dicte une action) ; 2) axe tronqué (il change le verdict de volatilité) ; 3) camembert surchargé et cumul (ils rendent illisibles) ; 5) log non déclarée (elle change la lecture d'une distribution). Tout classement argumenté est accepté.

E1 (le musée des horreurs personnelles). Grille de correction : trois graphiques analysés, chaque défaut **chiffré** (une mesure, pas un adjectif), une déclaration **rédigée** par défaut (et non « il faudrait déclarer »), et une correction qui ne change pas les données. Le repère du dossier du module : 5 graphiques, 5 défauts, 0 chiffre faux — c'est cet écart entre justesse et honnêteté que l'exercice fait toucher.

E2 (la déclaration rédigée). Trois attendus : la déclaration est **dans la figure** (titre, étiquette ou note) ; la phrase de protection est écrite en clair (« les hauteurs ne s'additionnent pas » pour un log) ; la limite de validité est explicite (le zoom ne parle que de sa fenêtre). Un correcteur doit pouvoir dire, sans l'auteur, ce que la figure montre et ce qu'elle ne montre pas.

13. Mini-projet M10.P5 — « le diagnostic mesuré des cinq défauts » (1 h 30)

Énoncé. C'est la première étape notée du projet « La refonte ». Vous produisez le **dossier de diagnostic** du rapport de la quincaillerie : cinq graphiques, cinq défauts nommés, cinq effets mesurés, cinq décisions fausses possibles, cinq déclarations ou corrections.

1. **Nommez** chaque défaut en utilisant la liste du §5.1 (numéro et nom), et justifiez le diagnostic par la lecture du code du rapport (`rapport_avant/code_avant.py`), pas par votre seule impression : citez la ligne qui décide l'axe, l'échelle ou l'ordre.
2. **Mesurez** chaque effet avec les gestes du §7, et comparez vos valeurs à celles du socle (`ATTENDU.json`, clés `d1_*` à `d5_*`). Toute différence de mesure est un point à expliquer (péri-mètre, arrondi, définition).
3. **Rédigez** les cinq décisions fausses possibles, en une phrase chacune, comme si vous écriviez au comité qui les a prises.
4. **Écrivez** la déclaration ou la correction pour chacune (une ligne par graphique).
5. **Classez** les cinq défauts avec un critère explicite, et concluez en une phrase sur le **contrôle** : quel test automatique aurait attrapé chaque défaut ? (exemple : « un contrôle qui compare la fenêtre de l'axe à l'étendue des données »).

Barème indicatif : cinq diagnostics nommés et justifiés par le code (6 points), cinq effets mesurés conformes au socle (5 points), cinq décisions fausses rédigées (3 points), cinq déclarations ou corrections (2 points), classement et conclusion sur les contrôles (2 points), sur 18.

Pourquoi ce mini-projet. Le diagnostic est le livrable qui ouvre le projet « La refonte » : sans lui, la refonte se discute (« je préfère ce graphique ») ; avec lui, elle se vérifie (les cinq défauts sont mesurés, donc les cinq corrections le sont aussi).

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise **6 gestes de vérification**, tous exécutés dans la planche du §6 : `set_ylim` (poser la fenêtre et la comparer à l'étendue), `twinx` (reproduire le double axe pour le mesurer), `pile/bar(bottom=)` (reproduire un cumul pour vérifier les totaux), `set_yscale("log")` (comparer les largeurs occupées en linéaire et en log), `diff` sur des parts (écart minimal) et `corr` (corrélation affichée).

Besoin	Commande	Ce qu'elle donne	Ce qu'elle ne donne pas
mesurer troncature	une étendue des données ÷ fenêtre de l'axe	la part de hauteur occupée	l'intention de l'auteur : la déclaration est à écrire
mesurer double axe	un <code>serie1.corr(serie2)</code>	la corrélation affichée	la causalité : elle n'existe pas dans le graphique
mesurer camembert	un <code>np.diff(parts.triées)</code> puis <code>×</code> 3,6	l'écart minimal en degrés	ce que le lecteur peut comparer (à confronter à C01)
mesurer un cumul	totaux par période + amplitudes des parts	ce que la normalisation masque	la correction : elle consiste à écrire le total
mesurer échelle log	une largeurs occupées en linéaire et en log	ce que le log donne et retire	la légitimité du choix : elle dépend de la question
relire vite	les 6 questions du §5.7	un verdict en une minute	la correction : elle se décide après

15. Résumé du chapitre

C05 a transformé la peur des « mauvais graphiques » en **protocole de relecture**. Les dix défauts, avec leurs effets mesurés sur le fil rouge : axe tronqué (82,4 % de hauteur occupée contre 11,8 %, amplitude apparente $\times 7,0$ pour 5,4 % réels), double axe (corrélation affichée **+0,35** entre séries sans lien, 4 à 16 retours par mois), 3D (aucun usage professionnel : la profondeur ne porte aucune donnée), camembert surchargé (**0,17°** entre deux part voisines, lisible jusqu'à 3 parts), cumul de pourcentages (totaux variant de **5,4 %** masqués par des parts qui bougent de 1,9 point), couleur à tout faire (l'orange, **3 rôles** en 7 emplois), surcharge (**120** éléments dans un cadre, limite de 5 séries), zéro manquant ou ajouté (11,8 % de hauteur occupée : la vérité de ce jeu, à accompagner du chiffre), ordre alphabétique (**21 paires sur 28** à l'envers) et échelle logarithmique surprise (la masse passe de **42,1 %** à **93,5 %** de largeur, la queue de 15,9 % à 1,3 %). Et la règle qui donne son sens à la liste : **une erreur n'est pas un péché, c'est une décision — elle est légitime si elle est déclarée**.

16. À retenir

À retenir. Un graphique trompeur n'a pas de chiffre faux : il a une **convention cachée**. Le travail de relecture consiste donc à rendre chaque convention visible — et, quand elle ne peut pas l'être (3D, couleur à tout faire), à retirer le défaut. Six questions, une minute, et une phrase écrite dans la figure : c'est tout ce que ce chapitre demande.

1. **Les trois défauts qui changent une décision** : axe tronqué ($\times 7,0$), double axe (+0,35), ordre alphabétique (21 paires sur 28 à l'envers).
2. **Les quatre qui rendent illisible** : camembert surchargé (0,17°), cumul à 100 % (5,4 % masqués), surcharge (120 éléments pour une limite de 5 séries), log surprise (93,5 % contre 42,1 %).
3. **Les deux qu'aucune déclaration ne sauve** : l'effet 3D et la couleur à tout faire (3 rôles pour une couleur).
4. **Les barres partent de zéro** ; une courbe peut être tronquée si le titre du zoom reste **local**.
5. **Deux séries à comparer = deux cadres** : le double axe ne se justifie que pour des unités de même nature, et jamais pour suggérer un lien.
6. **Toute normalisation déclare son total** : une barre empilée à 100 % sans étiquette de total cache l'information la plus utile.
7. **L'échelle logarithmique s'annonce** : « les hauteurs ne s'additionnent pas » — et le zéro n'existe plus.
8. **Relire, c'est chercher le faux citable** : « qu'est-ce que quelqu'un pourrait dire de faux devant cette figure ? ».

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi un axe tronqué est-il parfois légitime, et jamais pour des barres ?** → Parce qu'une courbe code une **position** (sa forme reste lisible si les bornes sont déclarées), alors qu'une barre code une **longueur** : tronquer l'axe rogne la barre et fausse le rapport entre deux valeurs (légitime : zoom annoncé, titre local ; jamais : barres). (2 pts)
2. **Le rapport titre « les retours suivent le chiffre d'affaires ». Quelle mesure le contredit ?** → La corrélation **affichée** de **+0,35** sur 24 mois, pour deux séries sans mécanisme métier (4 à 16 retours par mois) : c'est un double axe qui fabrique la coïncidence. (2 pts)

3. **Une barre empilée à 100 % masque quoi, exactement ? Combien, sur le fil rouge ?** → Elle masque les **totaux** : les huit trimestres vont de 960 721 755 à 1 012 518 068 FCFA, soit **5,4 %** de variation — pendant que la part du mode de paiement le plus variable ne bouge que de **1,9 point** (rapport 2,88 entre les deux amplitudes). (2 pts)
4. **Que change l'échelle logarithmique sur l'histogramme des montants, en largeurs occupées ?** → La masse des petits montants (77,0 % des ventes sous 250 000 FCFA) passe de **42,1 %** à **93,5 %** de la largeur, et la queue (2,1 % des ventes) de 15,9 % à **1,3 %** : le log rend visible la masse, invisible la queue — et il interdit d'additionner les hauteurs. (2 pts)
5. **Citez deux défauts qu'aucune déclaration ne rend acceptables, et pourquoi.** → L'effet 3D (la profondeur ne porte aucune donnée et déforme les hauteurs) et la couleur à tout faire (une couleur à trois rôles n'a pas de signification stable). Les deux suggèrent une information qui n'existe pas : aucune phrase ne répare une suggestion. (2 pts)
6. **Écrivez les six questions de la relecture en 60 secondes, dans l'ordre.** → Question servie ; échelle (zéro ou troncature déclarée) ; liens (double axe, log, cumul à 100 % — tous déclarés ?) ; chiffre clé annoté ; densité (séries, couleurs, une idée par cadre) ; ordre et source. (2 pts)

Module M10.C06 — Storytelling : raconter pour décider

Outils : matplotlib 3.10.9, pandas 2.2.3, numpy 2.3.5 (exécutés dans ce chapitre) ; Excel (exécuté, C07) ; Power BI (cité, non exécuté, C07). **Durée indicative :** 4 h. **Niveau :** N3. **Prérequis :** M10.C01 à C05 — en particulier C02 (le catalogue), C04 (l'écriture) et C05 (les défauts et leur mesure).

L'idée du chapitre. Un graphique juste ne déclenche aucune décision. Une **séquence** de graphiques, oui. Ce chapitre assemble ce que les cinq premiers ont construit : un écran qui pose le **niveau**, un écran qui révèle le **mécanisme**, un écran qui demande une **décision** — trois écrans, vingt secondes chacun, une minute de présentation. Les mêmes chiffres servent deux destinataires : la **direction** veut le niveau, l'écart et le coût de l'inaction ; l'**équipe** veut la base, le mécanisme et le plan. La séquence couvre les **cinq objections** que le rapport unique du dossier laissait sans réponse (2 sur 5 seulement), et elle le prouve par une mesure : sur le fil rouge, un seul cadre de 12 chiffres répond à deux objections ; la séquence de 3 écrans en couvre cinq, avec 29 chiffres répartis et **6 au maximum par écran**.

Matériel de l'atelier — Python 3.13 · matplotlib 3.10.9 · pandas 2.2.3 · numpy 2.3.5. La planche `figures/M10_C06_sequence_trois_ecrans.svg` (maquette des six écrans) est produite par `tools/sequence_M10.py`, deux exécutions, même fichier. Les chiffres viennent du socle M09 : CA net **3 965,4 M** FCFA en 2026 contre **3 910,9 M** en 2025 (+1,4 %), **24** mois, **208** retours, **1 044** ventes de plus de 500 000 FCFA.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Structurer une séquence en trois écrans** — le niveau, le mécanisme, la décision — et savoir ce qui va dans chacun, ce qui n'y va pas.
2. **Répondre aux objections par avance** : lister les cinq questions qu'un lecteur se pose, et montrer l'écran qui y répond — un rapport qui n'en couvre que 2 sur 5 est un rapport inachevé.
3. **Produire deux versions du même fil rouge** : la version **direction** (niveau, écart, coût, décision demandée) et la version **équipe** (base, mécanisme, plan, suivi), à partir du **même** socle de chiffres.
4. **Calibrer la densité** : combien de chiffres, de lignes et d'écrans par minute de présentation, et pourquoi le plafond du module est de **6 chiffres par écran**.
5. **Livrer la séquence du projet « La refonte »** : trois écrans, deux versions, avec le chiffre clé annoté de chaque écran et la décision demandée écrite en clair.

2. Pourquoi cette notion est importante

Le manuel insiste depuis le premier chapitre : un graphique sert une **question**. Mais une question seule ne suffit pas à faire décider. Ce qui manque à la plupart des rapports n'est pas la donnée, ni la figure, ni même la correction : c'est **la séquence** — l'ordre dans lequel le lecteur reçoit les trois informations qui le conduisent à dire oui.

Le dossier du module en est la démonstration inversée. Son rapport contient cinq graphiques, tous exacts. Il répond au « où en est-on ? » (le CA progresse de +1,4 %) et il donne un levier (le

panier). Mais il ne répond ni au « est-ce la saisonnalité ? », ni au « combien ça rapporte ? », ni au « qui fait quoi ? », ni au « comment saura-t-on si ça marche ? ». Résultat : deux objections seulement sur cinq sont couvertes, et la direction, à qui il manque le coût et le suivi, reporte sa décision — qu'elle aurait pu prendre si la séquence avait existé.

Il y a une raison économique à cela. Une réunion de direction dure une heure et contient six sujets : le budget d'attention par sujet est de dix minutes, souvent moins. Une séquence de trois écrans à 20 secondes se présente en **60 secondes** et laisse le reste du temps aux questions — c'est-à-dire à ce que la direction fait de mieux. Un rapport dense, à l'inverse, occupe les dix minutes en lecture et ne laisse aucun temps à la décision.

Enfin, la séquence a un effet de **contrat**. Chaque écran porte un titre qui affirme, un chiffre annoté, et une ligne qui dit ce qu'il répond. Quand la direction valide la séquence, elle valide l'enchaînement — pas une courbe. C'est la différence entre « je n'aime pas ce graphique » (qui bloque tout) et « l'écran 2 exagère le levier, montrez-moi la queue de plus près » (qui fait avancer).

Dans les faits. La demande la plus fréquente d'un commanditaire qui reçoit un beau graphique n'est pas « refaites-le », c'est « et donc ? ». La séquence est la réponse à cette question : troisième écran, la décision demandée, en une phrase et un chiffre.

3. Explication simple — trois écrans, une minute

La séquence tient dans la phrase que le manuel retient : **écran → insight → décision**.

1. **L'écran (le niveau).** Où en est-on ? Le chiffre principal, l'écart, la période. Une seule figure, un seul chiffre annoté. Le lecteur doit pouvoir répéter cet écran sans vous : « 3 965,4 M, +1,4 % sur un an ».
2. **L'insight (le mécanisme).** Pourquoi est-ce ainsi ? Ce n'est plus une description, c'est une **explication** : sur le fil rouge, « 77,0 % des ventes ne portent que 47,3 % du CA, pendant que 1 044 ventes de plus de 500 000 FCFA en portent 7,1 % ». L'insight n'est pas un graphique de plus : c'est le **même** fil rouge, vu sous l'angle qui rend l'action possible.
3. **La décision.** Que fait-on, pour combien, et comment le saura-t-on ? Une demande explicite (« cibler la queue : 56,1 M FCFA visés »), un coût (« 2 vendeurs »), un suivi (« la part du CA de plus de 500 000 FCFA »).

Vingt secondes par écran, une minute au total. Le reste de la réunion est aux questions — et c'est précisément ce qu'on veut, parce que les questions d'une réunion sont la matière dont les décisions sont faites.

Trois écrans, jamais plus : au quatrième, on n'est plus dans une séquence mais dans une présentation. Et une règle de discipline qui vient de C04 : **un chiffre principal par écran**. La maquette du chapitre le mesure : la version direction porte **17 chiffres** pour trois écrans, soit **6 par écran** — le plafond du module — et la version équipe **12 chiffres**, soit 4 par écran, parce qu'elle s'adresse à des gens qui manipulent la donnée toute la journée et n'ont pas besoin de sous-titres.

4. Vocabulaire essentiel

Terme	Définition
séquence	l'enchaînement ordonné de trois écrans (niveau, mécanisme, décision) qui conduit un lecteur d'un constat à une demande de décision.

Terme	Définition
écran	une figure, un titre qui affirme, un chiffre clé annoté, et la ligne qui dit à quelle objection l'écran répond.
insight	l'explication que les données permettent de formuler : elle transforme une description (« le CA progresse ») en un mécanisme (« la queue porte le CA »).
appel à la décision	la phrase du troisième écran qui dit ce qu'on demande, à qui, pour quel montant, et avec quel suivi. Sans elle, la séquence est une présentation.
objection	la question que le lecteur se pose à voix haute ou non. La séquence les anticipe : cinq sur le fil rouge, toutes couvertes, contre deux pour le rapport en un seul écran.
densité	nombre de chiffres et de lignes par écran. Plafond du module : 6 chiffres , 3 lignes, 1 figure par écran.
version direction	la variante de la séquence qui porte le niveau, l'écart à l'objectif, le coût et la décision demandée.
version équipe	la variante qui porte la base (données, périmètre, retours), le mécanisme et le plan d'action daté.
fil rouge	le jeu de données du module, commun à tous les graphiques : il garantit que les deux versions racontent la même histoire avec les mêmes chiffres.

5. Cours approfondi

5.1 Les trois écrans, et ce qui ne va dans aucun

Écran 1 — le niveau. Son travail est de faire dire au lecteur, après vingt secondes : « on est à 3 965,4 M, en hausse de 1,4 %, sur 24 mois. » Le graphique est celui de C02 : une évolution, une ligne de moyenne glissante, un axe honnête. Ce qui n'y va pas : les catégories, les modes de paiement, le détail par ville. Le niveau est une **réponse**, pas un sommaire.

Écran 2 — le mécanisme. Son travail est de répondre à « pourquoi ? ». C'est l'écran qui demande le plus de travail, parce qu'un mécanisme ne se dessine pas : il se **choisit**. Sur le fil rouge, plusieurs mécanismes sont disponibles — la saisonnalité, la concentration par catégorie, la distribution des paniers. Le module retient celui qui rend l'action possible : **la queue du fichier** (2,1 % des ventes de plus de 500 000 FCFA pèsent 7,1 % du CA, alors que 77,0 % des ventes, toutes petites, n'en pèsent que 47,3 %). Ce qui n'y va pas : deux mécanismes concurrents sur le même écran. Un écran qui explique deux choses n'explique rien.

Écran 3 — la décision. Son travail est de faire écrire au lecteur : « donc on fait quoi ? » Structure en trois lignes, toujours les mêmes : la demande (objectif chiffré), le moyen (qui, quand), le suivi (quel indicateur, quelle fréquence). Ce qui n'y va pas : les options. Un troisième écran qui propose trois scénarios demande au lecteur de faire le travail de l'analyste ; la séquence, elle, recommande — et c'est la recommandation qui ouvre le débat.

Définition. L'**appel à la décision** est la phrase, écrite sur le troisième écran, qui nomme la décision demandée, son destinataire, son montant et son indicateur de suivi. Un appel à la décision n'est pas un souhait (« il faudrait développer les gros paniers ») : c'est une demande (« affecter 2 vendeurs au segment > 500 000 FCFA pendant 4 semaines, suivi hebdomadaire de la part du CA »).

5.2 La séquence qui répond aux objections

Une présentation échoue rarement sur ses chiffres : elle échoue sur les objections restées sans réponse. La méthode du chapitre tient en trois gestes :

1. **Écrire la liste des objections**, du point de vue du destinataire. Sur le fil rouge, cinq surgissent : « où en est-on ? », « est-ce la saisonnalité ? », « où est le levier ? », « combien ça rapporte ? », « qui fait quoi et comment saura-t-on ? ».
2. **Coller chaque objection à un écran**. Une objection sans écran est un trou dans la séquence : sur la maquette, les objections 1 et 2 sont couvertes par l'écran 1, la 3 par l'écran 2, les 4 et 5 par l'écran 3.
3. **Vérifier la couverture**. Le rapport du dossier, en un seul écran de 12 chiffres, couvre **2** objections sur **5** ; la séquence en couvre **5**, soit **100 %** — avec 29 chiffres au total, c'est-à-dire **plus de chiffres, mais mieux placés**.

C'est le point contre-intuitif du chapitre : la séquence contient **plus** d'information que le rapport unique (29 chiffres contre 12) et pourtant elle se lit **plus vite** (60 secondes), parce que chaque chiffre est placé là où une question se pose. La densité n'est pas le problème ; la densité **non ordonnée** est le problème.

La couverture se vérifie aussi **pendant** la réunion, et c'est là qu'elle rapporte. Chaque question posée qui n'a pas d'écran est une objection manquante : elle se note pour la version suivante, comme un défaut se note pour une refonte (C05). Sur le fil rouge, la question la plus fréquente des comités est la seconde — « est-ce la saisonnalité ? » — et c'est précisément celle que le rapport laissait ouverte, avec ses douze chiffres tous portés sur les douze derniers mois.

À retenir. L'ordre des écrans n'est pas un ordre de préférence : c'est l'ordre dans lequel les objections apparaissent. Un lecteur se demande d'abord où il en est, ensuite pourquoi, enfin ce qu'on lui demande. Une séquence qui commence par le plan (écran 3 en premier) oblige le lecteur à croire avant de comprendre — et il refuse.

5.3 Deux versions, un seul socle

La même analyse sert deux publics, et leurs objections ne sont pas les mêmes.

	Version direction	Version équipe
Écran 1	niveau, écart à l'objectif, périmètre comparé	la base : les 24 mois, les 208 retours, le fait que rien n'a été lissé
Écran 2	le levier, chiffré, avec ce qu'il vaut	le mécanisme : pourquoi la queue porte le CA, comment un devis qui traîne coûte deux semaines de CA moyen
Écran 3	la demande : objectif, coût, suivi	le plan : semaine 1, semaines 2 à 4, revue du vendredi
Densité mesurée	17 chiffres, 6 par écran, 3 lignes par écran	12 chiffres, 4 par écran, 3 lignes par écran
Objection dominante	« combien ça rapporte, et à quel coût ? »	« est-ce que je peux le faire dès lundi ? »
Ton	affirmatif, chiffré, tourné vers la décision	factuel, tourné vers l'exécution

Trois règles pour produire les deux versions sans travailler deux fois :

1. **Un socle unique de chiffres**. Les deux versions sortent du **même** script : c'est la seule façon d'éviter qu'une réunion entende 3 965,4 M et une autre 3 969 M. Le module l'applique : les chiffres des deux versions sont produits par un seul appel à `mesures()`.

2. **Le même ordre, des contenus différents.** Les trois écrans restent le niveau, le mécanisme, la décision dans les deux versions : c'est la structure qui les rend reconnaissables et qui évite de tout réécrire pour chaque public.
3. **La même vérité.** Aucune version n'embellit : ni l'une ni l'autre ne masque les 47,3 % portés par la masse des petites ventes, ni les 0,4 % de retours, ni le fait que la hausse annuelle tient en **1,4 %**. Une version qui exagère pour convaincre se fait démonter en une question.

Attention. La tentation la plus fréquente est de « simplifier » la version direction en supprimant les chiffres gênants. C'est exactement ce que C05 appelait une erreur non déclarée : la version simplifiée qui cache un chiffre trompe plus sûrement qu'un graphique mal dessiné, parce que personne ne va chercher le chiffre absent. Simplifier, c'est retirer du **détail** — jamais du **contraire**.

Définition. La **densité d'un écran** est le nombre de chiffres, de lignes et de figures qu'il porte. Elle se compte, comme la longueur d'un titre en C04 : plafond du module, **6 chiffres** et 3 lignes pour une figure. Un écran qui dépasse n'est pas plus riche : il est plus lent, et le lecteur en sort avec une impression de sérieux au lieu d'un chiffre.

5.4 Le temps de lecture : 20 secondes par écran

Le rythme du module vient d'une contrainte de réunion : **20 secondes par écran, 60 secondes pour la séquence**, questions ensuite. Ce n'est pas une règle esthétique, c'est un budget qui se vérifie :

- **un chiffre principal** par écran (le titre qui affirme, C04) ;
- **au plus 6 chiffres** au total par écran — c'est ce que fait la version direction ;
- **une figure** par écran, sans double axe ni cumul non déclaré (C05) ;
- **3 lignes** de texte au maximum, dont une qui dit à quelle objection l'écran répond.

Vingt secondes suffisent à lire un écran, si le lecteur n'a pas à chercher ce qu'il doit y voir : c'est précisément le travail du titre qui affirme et de l'annotation. À l'inverse, un écran portant douze chiffres sans hiérarchie occupe plusieurs minutes de lecture et ne laisse aucune trace : le lecteur ressort avec « c'était très complet ». La densité mesurée du chapitre — **6 chiffres** par écran pour la direction, **4** pour l'équipe — est le résultat d'un compromis : assez de chiffres pour être crédible, assez peu pour être mémorisable.

Conseil professionnel. Chronométrez vos trois écrans à voix haute, seul, une fois. Si vous dépassez 90 secondes, ce n'est pas la séquence qui est trop longue : c'est qu'un écran contient deux idées. Coupez l'écran en deux et jetez celui dont personne n'a besoin — il finira en annexe, ce qui est sa place.

5.5 Ce qui ne se met pas dans une séquence

Une séquence se juge aussi à ce qu'elle refuse. Quatre objets du dossier restent **dehors** :

- **Le tableau de bord complet.** La séquence n'est pas une synthèse exhaustive : c'est un chemin. Les 8 métriques du socle, les 5 catégories, les 3 villes : hors séquence, en annexe.
- **Les chiffres sans conséquence.** Le nombre de retours (208 lignes, 0,4 % du CA) n'a de place que s'il déclenche une action — dans la version équipe, il sert à établir la propreté de la base ; ailleurs il encombre.
- **Les options non arbitrées.** Trois scénarios côte à côte sans recommandation : la séquence recommande, et c'est écrit dans l'appel à la décision.

- **Les figures de la C05.** Un graphique qui illustre un défaut ne se présente pas : il se corrige. Le mur des erreurs est un outil de travail, pas un écran.

La règle de tri est la même que celle du titre qui affirme (C04) : **si la ligne ne peut pas être transformée en action ou en question, elle sort.** Un test simple, appliqué aux cinq graphiques du dossier : pour chacun, écrire la phrase que le lecteur prononcerait en le voyant. « C'est plat, puis ça monte » (écran 1), « les gros paniers pèsent peu » (écran 2), « donc on relance la queue » (écran 3). Si une figure ne produit pas de phrase, c'est qu'elle appartient à l'annexe — ou qu'elle n'a pas de raison d'être.

Cette discipline a un effet secondaire précieux : elle réduit le travail. Une séquence de trois écrans se produit en quelques heures, à partir du socle déjà construit pour les chapitres précédents ; un rapport de trente graphiques demande des jours et se lit en diagonale. Le temps gagné va là où il compte : vérifier les chiffres, écrire le titre qui affirme, préparer la réponse aux contestations.

Définition. On appelle **restitution** l'ensemble formé par la séquence et ses annexes. La séquence porte les trois réponses ; les annexes portent la vérifiabilité (les 8 métriques, les 5 catégories, le détail par ville, le code). Une restitution professionnelle ne choisit pas entre les deux : elle sépare ce qui se présente en une minute de ce qui se consulte sur demande.

5.6 La narration n'embellit pas : elle ordonne

Le mot « storytelling » fait peur aux analystes, et il a raison de le faire quand il désigne l'art de choisir les faits qui arrangent. Le module l'utilise dans l'autre sens : **ordonner sans retranscrire**. La preuve est mesurable, et elle est dans la maquette :

- les deux versions portent **le même fil rouge**, la même période de 24 mois et les mêmes chiffres clés : 3 965,4 M, +1,4 %, 77,0 %, 47,3 %, 7,1 %, 0,4 % ;
- la version direction ne cache ni la faiblesse de la hausse annuelle (+1,4 %) ni le fait que la masse des petites ventes représente 47,3 % du CA — deux chiffres qui affaiblissent le récit de croissance ;
- la séquence ne fait gagner aucune objection par omission : les 5 objections sont couvertes, dont celle de la saisonnalité, qui exige de comparer **24 mois** et non 12.

Ce qui change entre les versions n'est donc ni la vérité ni la donnée : c'est la **quantité de contexte** et la **nature de la demande**. La direction veut savoir ce qu'elle gagne et ce qu'elle engage ; l'équipe veut savoir ce qu'elle fait lundi matin. La même analyse porte les deux, à condition de ne pas confondre simplification et dissimulation.

Attention. L'objection « ce n'est pas de la donnée, c'est du marketing » tombe dès que la séquence cite ses mesures. Le réflexe professionnel : dans une réunion, si quelqu'un conteste un écran, ne défendez pas le récit — donnez la **mesure** (l'axe occupé à 82,4 %, la corrélation affichée de +0,35, la part de 7,1 %). Le chiffre est le seul argument qui ne se discute pas.

5.7 La séquence dans le travail du module

La séquence de trois écrans n'est pas un exercice de style : c'est le **deuxième livrable** du projet « La refonte », et elle revient dans plusieurs modules du manuel :

- **M09** fournit les données, **M10.C01 à C05** fournissent les figures : la séquence est le moment où tout se rassemble ;
- **M11 à M15** (tableaux, outils, restitution) réutilisent la structure niveau/mécanisme/décision pour leurs propres restitutions ;

- **M22** (portfolio) attend une séquence complète, présentée et défendue : c'est l'une des pièces évaluées du dossier de fin de cursus.

Autrement dit : la séquence est le format de sortie du travail d'analyse. Un analyste qui sait faire une séquence n'a plus besoin d'un rapport de trente pages pour se faire comprendre — ce qui, dans une entreprise, est la définition même de l'autonomie professionnelle.

Définition. La **couverture des objections** est le rapport entre le nombre d'objections anticipées et le nombre d'objections auxquelles la restitution répond explicitement. Elle se mesure et s'affiche, comme n'importe quel indicateur : sur le fil rouge, 2 sur 5 pour le rapport en un seul cadre, 5 sur 5 (100 %) pour la séquence de trois écrans.

6. Exemple concret — la maquette des six écrans

La planche du chapitre montre la maquette complète : deux versions, trois écrans chacune, et la bande des objections avec l'écran qui y répond. C'est un document de travail : on l'imprime, on le déplace, on découpe un écran.

La séquence de trois écrans : le même fil rouge, deux destinataires

Version direction — 74 mots, 17 chiffres, 3 lignes par écran

Écran 1 — le niveau	Écran 2 — le levier	Écran 3 — la décision
· CA net 2026 : 3 965,4 M FCFA · +1,4 % sur un an (3 910,9 M en 2025) · 24 mois comparés, retours exclus → où en est-on ? · la saisonnalité ?	· 77,0 % des ventes = 47,3 % du CA · 1 044 ventes > 500 000 = 7,1 % du CA · médiane 119 482 FCFA : la queue paie → où est le levier ?	· Objectif : 56,1 M FCFA (+10 %) · Mesure : 1 044 paniers, 4 semaines · Coût : 2 vendeurs ; suivi : la queue → combien ça rapporte ? · qui fait quoi ?

Version équipe — 73 mots, 12 chiffres, 3 lignes par écran

Écran 1 — la base	Écran 2 — le mécanisme	Écran 3 — le plan
· 24 mois, la même courbe qu'au rapport · Retours : 208 lignes, 0,4 % du CA · Rien n'a été lissé ni recalculé → les données sont-elles propres ?	· La queue (2,1 % des ventes) : 7,1 % du CA · Sous le panier médian : la moitié du fichier · Un devis qui traîne : 2 semaines de CA → pourquoi ça se passe comme ça ?	· Semaine 1 : extraire les 1 044 paniers · Semaine 2-4 : relancer, noter les refus · Vendredi : revue de 10 min, 3 chiffres → qui fait quoi ? · comment on saura ?

Les cinq objections, et l'écran qui y répond

« où en est-on ? la saisonnalité ? » → écran 1
 « où est le levier ? » → écran 2
 « combien ça rapporte ? » → écran 3
 « qui fait quoi ? » → écran 3
 « comment on saura si ça marche ? » → écran 3

Le cadre unique du rapport (v1) : 1 écran, 12 chiffres, 2 objections sur 5 couvertes.

La séquence : 3 écrans, 29 chiffres (6 par écran au plus), 5 objections sur 5 (100 %), 60 s de lecture.

Ce que la maquette démontre :

- **Le même fil rouge, deux récits.** À l'écran 1 de la version direction, le niveau (3 965,4 M, +1,4 %, 24 mois comparés) ; à l'écran 1 de la version équipe, la **base** (les mêmes 24 mois, les 208 retours et 0,4 % du CA, et la mention que rien n'a été lissé). Même donnée, deux objections différentes : « où en est-on ? » et « puis-je faire confiance au fichier ? ».
- **Le mécanisme, chiffré.** L'écran 2 porte le chiffre qui rend l'action possible : **77,0 %** des ventes pour **47,3 %** du CA, et **1 044** ventes de plus de 500 000 FCFA pour **7,1 %** du CA. C'est une distribution, pas un classement : la décision qui en sort est « cibler la queue ».
- **La demande et le plan.** Écran 3, version direction : objectif **56,1 M** FCFA (+10 % sur la queue), mesure sur 4 semaines, coût de 2 vendeurs, suivi par la part du CA de la queue. Écran 3, version équipe : semaine 1, extraction des 1 044 paniers ; semaines 2 à 4, relance et notation des refus ; vendredi, revue de dix minutes avec trois chiffres.
- **La densité, mesurée.** 17 chiffres pour la direction (**6** par écran), 12 pour l'équipe (**4** par écran), 3 lignes par écran dans les deux cas, et 20 secondes par écran — soit 60 secondes pour la séquence complète.
- **Et la comparaison qui justifie la séquence** : le cadre unique du rapport (1 écran, 12 chiffres) couvrirait **2** objections sur **5**. La séquence en couvre **5**, avec 29 chiffres — plus d'information, mieux adressée.

7. Démonstration pas à pas — construire la séquence en 5 gestes

Geste 1 — écrire les objections avant tout graphique. Le tableau des cinq objections est le premier livrable, pas le dernier.

```
objections = [("ou en est-on ? la saisonnalite ?", "ecran 1"),
              ("ou est le levier ?", "ecran 2"),
              ("combien ca rapporte ?", "ecran 3"),
              ("qui fait quoi ?", "ecran 3"),
              ("comment on saura si ca marche ?", "ecran 3")]
print(f"{len(objections)} objections, couvertes par "
      f"{len(set(e for _, e in objections))} ecrans")
print("rapport en un cadre : 2 sur 5 (40 %)")
```

```
5 objections, couvertes par 3 ecrans
rapport en un cadre : 2 sur 5 (40 %)
```

Geste 2 — mesurer le niveau. Le chiffre de l'écran 1, calculé une fois pour les deux versions.

```
import pandas as pd
v = pd.read_csv("03_exercices/dossier_M09/quincailerie/vente.csv", parse_dates=["date_vente"])
net = v[~v["est_retour"]]
annee = net.groupby(net["date_vente"].dt.year)["montant_ttc"].sum()
print(f"CA net {annee.index[0]} : {annee.iloc[0]/1e6:.1f} M | {annee.index[1]} : {annee.iloc[1]/1e6:.1f} M "
      f"-> {100*(annee.iloc[1]/annee.iloc[0]-1):+.1f} %")
print("mois compares :", net["date_vente"].dt.to_period("M").nunique())
```

```
CA net 2025 : 3910.9 M | 2026 : 3965.4 M -> +1.4 %
mois compares : 24
```

Geste 3 — trouver le mécanisme. On compare la masse et la queue : c'est ce qui rend l'action possible, donc ce qui mérite l'écran 2.

```
m = net["montant_ttc"]
masse, queue = m[m < 250000], m[m > 500000]
ca = m.sum()
print(f"masse : {len(masse)} ventes ({100*len(masse)/len(m):.1f} %) portent "
      f"{100*masse.sum()/ca:.1f} % du CA")
print(f"queue : {len(queue)} ventes ({100*len(queue)/len(m):.1f} %) portent "
      f"{100*queue.sum()/ca:.1f} % du CA")
print(f"gain vise (+10 % sur la queue) : {queue.sum()*0.10/1e6:.1f} M FCFA")
print(f"mediane {m.median():.0f} FCFA | moyenne {m.mean():.0f} FCFA".replace(", ", " "))
```

```
masse : 38360 ventes (77.0 %) portent 47.3 % du CA
queue : 1040 ventes (2.1 %) portent 7.1 % du CA
gain vise (+10 % sur la queue) : 56.1 M FCFA
mediane 119 482 FCFA | moyenne 158 159 FCFA
```

Geste 4 — vérifier la propreté de la base, parce que c’est la première objection de l’équipe.

```
ret = v["est_retour"]
print(f"retours : {int(ret.sum())} lignes, {v.loc[ret, 'montant_ttc'].sum():.0f} FCFA, "
      f"{100*v.loc[ret, 'montant_ttc'].sum()/v['montant_ttc'].sum():.1f} % du CA "
      total".replace(", ", " "))
print("lignes :", len(v), "| colonnes :", len(v.columns))
```

```
retours : 208 lignes, 31 939 567 FCFA, 0.4 % du CA total
lignes : 50008 | colonnes : 13
```

Geste 5 — écrire les six écrans et mesurer la maquette. On compte les mots, les chiffres et les lignes par écran : c’est la vérification de densité du §5.4.

```
ecrans = {"direction": ["niveau", "levier", "decision"],
          "equipe": ["base", "mecanisme", "plan"]}
for version, liste in ecrans.items():
    print(f"{version} : {len(liste)} ecrans, 20 s par ecran -> {20*len(liste)} s de lecture")
```

```
direction : 3 ecrans, 20 s par ecran -> 60 s de lecture
equipe : 3 ecrans, 20 s par ecran -> 60 s de lecture
```

Ce que la démonstration établit. Une séquence n’est pas un talent : c’est une liste d’objections, trois chiffres mesurés, un plan daté et un contrôle de densité. Les cinq gestes tiennent en une heure de travail — et remplacent la soirée passée à refaire des graphiques que personne ne lira.

8. Erreurs fréquentes

1. **Commencer par la décision.** Un écran 1 qui demande une décision avant d’avoir montré le niveau oblige le lecteur à croire : il refuse, et la séquence est morte.
2. **Confondre séquence et synthèse.** Trois écrans ne sont pas un résumé de tout : ce sont trois réponses. Le reste du travail (8 métriques, 5 catégories, 3 villes) va en annexe.
3. **Deux mécanismes sur l’écran 2.** Un écran qui explique deux causes n’en explique aucune : le lecteur retient la plus simple et oublie l’autre.
4. **Trois options au lieu d’une recommandation.** L’analyste propose, l’organigramme décide : un écran 3 qui liste trois scénarios n’a pas fait son travail.
5. **La version direction « allégée » qui cache un chiffre gênant.** C’est une erreur non déclarée (C05) : le chiffre absent trompe plus sûrement que le chiffre mal dessiné.

6. **Le même écran pour les deux publics.** L'équipe n'a pas besoin du coût de l'inaction, la direction n'a pas besoin du plan hebdomadaire : une seule version se fait lire par moitié par chacun.
7. **Oublier le suivi.** Un écran 3 sans indicateur de suivi transforme la séquence en vœu pieux : sans « comment on saura », la décision se dissout dans les semaines suivantes.

9. Bonnes pratiques professionnelles

1. **Écrire la liste des objections avant de produire les écrans**, et coller chaque objection à un écran ; afficher la couverture (2 sur 5 → 5 sur 5 sur le fil rouge).
2. **Un chiffre principal, un titre qui affirme, une annotation par écran** (C04), et jamais plus de 6 chiffres par écran.
3. **Trois écrans, 20 secondes chacun**, une minute pour la séquence, questions ensuite.
4. **Une recommandation, pas des options** : la demande est écrite, chiffrée, datée, avec son suivi.
5. **Un socle unique de chiffres pour les deux versions**, produit par un seul script — c'est ce qui garantit que la direction et l'équipe parlent du même mois, du même périmètre et du même montant.
6. **Chronométrer à voix haute** avant la réunion, et couper l'écran qui dépasse.
7. **Préparer la réponse chiffrée aux contestations** : pour chaque écran, savoir quelle mesure répondra à « et si vous vous trompiez ? » (l'axe occupé, la corrélation affichée, la part du CA).

10. Exercice guidé — « la séquence du fil rouge » (15 min, /10)

Contexte. Vous disposez des cinq défauts corrigés du rapport (C05) et de la mesure du mécanisme : 77,0 % des ventes portent 47,3 % du CA, 1 044 ventes de plus de 500 000 FCFA portent 7,1 %, les retours représentent 0,4 % du CA. Vous avez dix minutes en réunion. On vous demande les trois écrans.

Consigne.

1. Nommez les **objections** de la direction (au moins trois) et de l'équipe (au moins deux). (2 pts)
2. Écrivez les **trois titres** de la version direction, un par écran, chacun sous forme d'affirmation (C04). (3 pts)
3. Pour chaque écran, donnez **le** chiffre clé (un seul) et l'objection à laquelle il répond. (3 pts)
4. Rédigez **l'appel à la décision** de l'écran 3 : décision demandée, destinataire, montant, suivi. (1 pt)
5. Donnez la **densité** de votre séquence (chiffres par écran, lignes par écran) et le temps de lecture estimé. (1 pt)

11. Exercices autonomes

E1 — les deux versions de votre analyse (45 min). Sur votre propre jeu de données, produisez la séquence en deux versions : direction et équipe. Contraintes : trois écrans, un chiffre principal par écran, 6 chiffres au plus par écran, un appel à la décision sur l'écran 3. Rendez les six écrans au format de la maquette (titre, trois lignes, la ligne « → objection »), plus un tableau de couverture des objections : vos objections, l'écran qui répond, et le pourcentage couvert. Une version sur trois écrans qui couvre moins de 80 % des objections est à retravailler avant d'être présentée.

E2 — la répétition chronométrée (30 min). Présentez la séquence à voix haute, chronomètre en main, seul, en filmant ou en enregistrant. Objectif : 60 secondes pour trois écrans, tolérance 90. Comptez ensuite les mots que vous avez ajoutés (« du coup », « en fait », « c'est-à-dire »). Puis refaites l'exercice en supprimant la phrase qui dépasse : c'est presque toujours la même — celle où vous expliquez ce que l'écran montre **déjà**.

12. Correction détaillée

Exercice guidé.

1. **Objections de la direction** : « où en est-on ? » ; « est-ce la saisonnalité ? » ; « combien ça rapporte, pour quel coût ? ». **De l'équipe** : « la base est-elle propre ? » ; « qui fait quoi lundi, et comment saura-t-on si ça marche ? ».
2. **Titres (version direction)** : écran 1 — « Le CA net atteint 3 965,4 M FCFA, +1,4 % sur un an » ; écran 2 — « 1 044 ventes de plus de 500 000 FCFA portent 7,1 % du CA » ; écran 3 — « Relancer la queue : 56,1 M FCFA visés en quatre semaines ».
3. **Chiffres et objections** : écran 1 → 3 965,4 M (où en est-on ?) ; écran 2 → 7,1 % du CA pour 2,1 % des ventes (où est le levier ?) ; écran 3 → 56,1 M FCFA visés (combien ça rapporte ?).
4. **Appel à la décision** : « affecter 2 vendeurs pendant 4 semaines à la relance des 1 044 papiers de plus de 500 000 FCFA ; suivi hebdomadaire de la part du CA de la queue ». Tout appel qui ne nomme ni destinataire ni suivi est incomplet.
5. **Densité** : 6 chiffres par écran au plus (17 pour la séquence direction, 12 pour l'équipe), 3 lignes par écran, 20 secondes par écran → **60** secondes de séquence.

E1 (les deux versions). Attendu principal : la **couverture des objections**, chiffrée, supérieure ou égale à 80 %. Attendu secondaire : la version direction ne contient aucun chiffre que la version équipe contredit (même socle), et l'appel à la décision est daté et suivi. Le repère du chapitre : 2 objections sur 5 pour le rapport en un cadre, 5 sur 5 pour la séquence.

E2 (la répétition chronométrée). Trois observations reviennent toujours : la phrase qui explique ce que l'écran montre déjà (à supprimer), l'absence de la demande explicite (à rétablir), et une durée qui dépasse 90 secondes dès que le deuxième écran contient deux idées (à couper en deux). Le critère de réussite n'est pas la durée : c'est que l'auditeur puisse **résumer** chaque écran après l'avoir entendu une fois.

13. Mini-projet M10.P6 — « la séquence de trois écrans, en deux versions » (1 h 30, /18)

Énoncé. Deuxième livrable du projet « La refonte ». Vous produisez la maquette complète, sur le fil rouge, dans les deux versions.

1. **Les objections** : la liste complète (au moins cinq), avec l'écran qui répond à chacune, et la **couverture mesurée** en pourcentage. (3 pts)
2. **Les six écrans** : trois par version, chacun avec un titre qui affirme, un chiffre principal annoté, trois lignes au maximum et la ligne « → objection ». (6 pts)
3. **L'appel à la décision** : décision, destinataire, montant, coût, indicateur de suivi et fréquence. (3 pts)
4. **La densité** : tableau des chiffres, mots et lignes par écran pour les deux versions, avec le plafond de 6 chiffres respecté. (2 pts)
5. **Le socle** : les chiffres des deux versions issus d'un **seul** script, exécuté deux fois à l'identique (le contrôle de déterminisme du module). (2 pts)

6. **La mesure du gain** : le gain visé calculé sur les données (56,1 M FCFA pour +10 % sur la queue), avec la méthode de calcul écrite en une ligne. (2 pts)

Barème indicatif : couverture ≥ 80 % (3), six écrans conformes (6), appel à la décision complet (3), densité mesurée (2), socle unique déterministe (2), gain calculé et justifié (2), sur 18.

Pourquoi ce mini-projet. C'est le livrable qui transforme une analyse en décision : il réunit les figures de C01 à C05 dans un objet qui tient en une minute, et il se vérifie — couverture des objections, densité, gain — au lieu de se discuter.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre n'utilise aucun outil graphique nouveau : il **assemble**. Les mesures utiles sont celles de C04 et C05 (densité, annotations, écart) plus quatre comptages de structure, tous exécutés dans la maquette : objections couvertes, écrans, mots par écran, chiffres par écran.

Besoin	Commande ou règle	Ce qu'elle donne	Ce qu'elle ne donne pas
équilibrer une séquence	écrire la liste des objections d'abord	la couverture (2 sur 5 → 5 sur 5)	la formulation des écrans : elle vient de C04
calibrer un écran	compter les chiffres du texte de l'écran	la densité (plafond 6)	la hiérarchie : elle vient du titre qui affirme
tenir 60 secondes	20 s par écran, 3 écrans	un budget de réunion	la réduction des écrans : couper est un arbitrage
produire deux versions	un seul socle de chiffres, deux maquettes	la cohérence des deux publics	le choix du public : il se demande avant
mesurer le gain	appliquer le taux visé à la population ciblée	le montant à défendre (56,1 M)	la faisabilité : elle se teste en pilote

15. Résumé du chapitre

C06 a fermé la boucle ouverte en C01 : un graphique sert une question, mais une **séquence** sert une décision. Trois écrans — le niveau (3 965,4 M, +1,4 %, 24 mois), le mécanisme (77,0 % des ventes pour 47,3 % du CA ; 1 044 ventes de plus de 500 000 FCFA pour 7,1 %) et la décision (56,1 M FCFA visés, 2 vendeurs, 4 semaines, suivi hebdomadaire) — lus en 20 secondes chacun, **60** secondes au total. La même analyse sert deux publics : la version direction (**17** chiffres, 6 par écran) et la version équipe (**12** chiffres, 4 par écran, avec la base et le plan). Et la preuve que la séquence est un progrès, non un habillage : le rapport en un seul cadre de 12 chiffres couvrait **2** objections sur **5** ; la séquence en couvre **5**, soit **100 %**, avec 29 chiffres mais jamais plus de 6 par écran.

16. À retenir

À retenir. Une séquence n'ajoute pas de l'information à un graphique : elle ordonne l'information autour des questions que le lecteur se pose. L'ordre est toujours le même — niveau, mécanisme, décision — et il se vérifie par la couverture des objections et par la densité, jamais par l'impression laissée.

1. **Trois écrans, jamais plus** : le niveau, le mécanisme, la décision ; 20 secondes chacun.
2. **Les objections d'abord** : on écrit les cinq questions du lecteur **avant** les graphiques, et chaque objection est collée à un écran (2 sur 5 pour un rapport dense, 5 sur 5 pour une séquence).
3. **Un chiffre principal par écran**, 6 chiffres au maximum, 3 lignes de texte.
4. **Deux versions, un socle** : direction (niveau, écart, coût, décision) et équipe (base, mécanisme, plan, suivi), avec les mêmes chiffres — la simplification retire du détail, jamais du contraire.
5. **Toujours un appel à la décision** : décision, destinataire, montant, coût, indicateur et fréquence.
6. **Le gain se calcule** : 10 % sur la queue de plus de 500 000 FCFA = **56,1 M** FCFA à défendre.
7. **Chronométrer à voix haute** avant la réunion : au-delà de 90 secondes, un écran contient deux idées.

17. Évaluation formative (auto-correction, 8 min)

1. **Quels sont les trois écrans d'une séquence, dans l'ordre, et à quelle question répond chacun ?** → Le niveau (« où en est-on ? »), le mécanisme (« pourquoi ? »), la décision (« que fait-on, pour combien, et comment le saura-t-on ? »). L'ordre suit celui des objections : croire avant de comprendre échoue. (2 pts)
2. **Comment mesure-t-on qu'une restitution répond aux objections ?** → Par la **couverture** : nombre d'objections anticipées satisfaites par un écran, sur le nombre total. Sur le fil rouge : 2 sur 5 (40 %) pour le cadre unique du rapport, 5 sur 5 (100 %) pour la séquence de trois écrans. (2 pts)
3. **Que change la version équipe par rapport à la version direction, et que ne change-t-elle pas ?** → Elle change la nature du propos (base et plan au lieu du niveau et du coût) et la densité (12 chiffres, 4 par écran contre 17 chiffres, 6 par écran) ; elle ne change **ni** les données, **ni** les chiffres clés (3 965,4 M, +1,4 %, 77,0 %, 47,3 %, 7,1 %, 0,4 %). (2 pts)
4. **Pourquoi la séquence contient-elle plus de chiffres que le rapport, tout en se lisant plus vite ?** → Parce que 29 chiffres placés à l'endroit où une question se pose se lisent en 60 secondes, alors que 12 chiffres empilés dans un cadre unique obligent à les trier — et laissent deux objections sur cinq sans réponse. La densité non ordonnée est le problème, pas la quantité. (2 pts)
5. **Rédigez l'appel à la décision du fil rouge en une phrase, avec ses cinq éléments.** → « Affecter **2 vendeurs** pendant **4 semaines** à la relance des **1 044 paniers** de plus de 500 000 FCFA, pour **56,1 M FCFA** visés (+10 %), suivi **chaque vendredi** par la part du CA de la queue. » Décision, destinataire, montant, coût, suivi : les cinq éléments sont nommés. (2 pts)

Module M10.C07 — matplotlib, Excel, Power BI : support, résolution, export

Outils : matplotlib 3.10.9 (exécuté), openpyxl 3.1.5 + Excel (exécuté : classeur écrit puis relu), Power BI (cité, non exécuté — règle §1.5). **Durée indicative :** 4 h. **Niveau :** N3. **Prérequis :** M10.C01 à C06 — en particulier C04 (l'écriture) et C06 (la séquence de trois écrans).

L'idée du chapitre. Les six chapitres précédents ont appris à **penser** un graphique ; celui-ci apprend à le **sortir**. Le même graphique est produit ici dans trois outils, et la conclusion du chapitre est mesurée : dans les trois cas, le chiffre est **identique** (écart relu : **0,00** FCFA sur **24** valeurs). Ce qui change n'est pas la donnée, c'est **le fichier** : PNG 96 dpi de **23,4** Ko et 604 × 340 px, PNG 300 dpi de **96,1** Ko et 1 889 × 1 062 px, SVG vectoriel de **12,9** Ko, PDF de **17,3** Ko, classeur de **7,0** Ko avec son graphique natif. Multiplier la résolution par trois multiplie le poids par **4,1** sans ajouter une seule information. Et la règle d'honnêteté du manuel s'applique aux outils comme aux chiffres : **matplotlib et Excel sont exécutés, Power BI est cité** — sa procédure fait **9** clics, aucun fichier n'est produit.

Matériel de l'atelier — Python 3.13 · matplotlib 3.10.9 · pandas 2.2.3 · openpyxl 3.1.5. La planche `figures/M10_C07_meme_graphique_trois_outils.svg` est produite par `tools/outils_M10.py` ; les fichiers eux-mêmes sont dans `03_exercices/dossier_M10/exports/` (deux PNG, un SVG, un PDF et le classeur). Deux exécutions donnent des fichiers identiques : le contrôle de déterminisme porte désormais aussi sur le PNG, le PDF et le **classeur Excel**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Produire le même graphique dans trois outils** — matplotlib (code), Excel (classeur) et Power BI (rapport) — et savoir ce qui, dans chacun, se **vérifie** et ce qui se **déclare**.
2. **Choisir un format d'export** : PNG pour un écran, 300 dpi pour l'impression, SVG ou PDF pour un document, et savoir ce que chaque choix coûte en poids.
3. **Mesurer la résolution utile** : 604 × 340 px à 96 dpi (soit **25** px par mois), 1 889 × 1 062 px à 300 dpi (**79** px par mois), pour un support de **160 × 90** mm.
4. **Vérifier un classeur** au lieu de le croire : le relire, comparer ses totaux au socle, et constater l'écart (**0,00** FCFA sur 24 valeurs).
5. **Déclarer ce qu'on n'a pas exécuté** : Power BI est décrit en **9** étapes, sans capture inventée, sans capture d'écran fabriquée — la règle §1.5 appliquée aux outils.

2. Pourquoi cette notion est importante

Un graphique juste mal exporté est un graphique perdu. C'est le dernier endroit où le travail se défait : une capture d'écran de 96 dpi collée dans un rapport imprimé donne des traits épais, des textes flous et un axe illisible ; un PNG de 300 dpi inséré dans une page web de 800 px de large pèse **96,1** Ko pour un affichage de la moitié de sa définition ; un PDF rasterisé perd ce qui fait la qualité d'un graphique de données — la netteté des traits et la lisibilité des étiquettes à n'importe quel agrandissement.

Il y a une seconde raison, plus professionnelle : **la chaîne d'outils est un engagement collectif**. Un analyste qui livre un classeur Excel contenant un graphique natif livre un objet **modi-**

fiable par son destinataire ; le même graphique livré en PNG est un objet **figé**. Ce sont deux métiers différents : la diffusion d'un constat, et la mise à disposition d'un outil. Le module insiste depuis C01 sur la question qui précède le graphique ; le format d'export est la dernière question de la même famille : **à qui, pour quoi faire ?**

Enfin, il y a la règle §1.5 du manuel, posée dès M01 : on exécute ce qu'on prétend exécuter, et on déclare le reste. Elle a servi pour DuckDB et SQLite (exécutés) contre PostgreSQL (cité). Elle s'applique ici de la même façon : matplotlib et Excel sont **exécutés** dans ce chapitre — le classeur est écrit et **relu** —, Power BI est **cité** : sa procédure est décrite pas à pas, elle n'est pas présentée comme une capture.

Dans les faits. Dans une entreprise, la question « tu me l'envoies comment ? » arrive toujours après la réunion, quand tout le monde est pressé. La réponse préparée d'avance — PNG 300 dpi pour l'impression, SVG pour le rapport, classeur pour l'équipe qui doit itérer — évite la seule situation vraiment coûteuse : refaire le graphique trois fois.

3. Explication simple — trois outils, un chiffre

Le chapitre tient dans un tableau à trois colonnes.

	matplotlib	Excel	Power BI
Statut dans ce chapitre	exécuté	exécuté (écrit puis relu)	cité , non exécuté
Ce qu'on produit	un fichier image ou vectoriel	un classeur avec graphique natif	un rapport publié
Coût de production mesuré	14 lignes de code	26 lignes écrites, 1 graphique natif	9 clics décrits
Ce qui se vérifie	l'identité des valeurs avec le socle	le total relu (0,00 FCFA d'écart)	rien : aucun fichier produit
Pour qui	rapports, documents, reproductibilité	équipes qui retouchent, comités	diffusion interne, suivi mensuel
Ce qu'il fait moins bien	retouche manuelle	versionnage, automatisation	reproductibilité hors de son écosystème

Trois enseignements, tous mesurés dans ce chapitre :

1. **Le chiffre ne change pas.** Le CA net mensuel est le même dans les trois outils : la qualité d'un graphique ne vient pas de l'outil, elle vient de ce qu'on a décidé en C01 à C06 (question, encodage, couleur, écriture, défauts, séquence).
2. **Le fichier change beaucoup.** À support égal (**160 × 90** mm), le PNG 96 dpi pèse **23,4** Ko et le PNG 300 dpi **96,1** Ko — un facteur **4,1** pour la même figure, dont la seule différence est la résolution d'impression.
3. **Le vectoriel n'est pas plus lourd.** Le SVG pèse **12,9** Ko et le PDF **17,3** Ko : moins que le PNG 96 dpi, tout en restant net à n'importe quelle taille. Pour un graphique de traits — pas une photo — le vectoriel est presque toujours le bon choix.

4. Vocabulaire essentiel

Terme	Définition
raster	image en pixels (PNG, JPEG). Sa netteté est fixée à la production : agrandir ne fait qu'agrandir les pixels.

Terme	Définition
vectorel	image décrite par des tracés (SVG, PDF). Elle se réaffiche à toute taille sans perte : c'est le format des graphiques de traits.
dpi (points par pouce)	nombre de pixels par pouce à l'impression. 96 dpi convient à un écran, 300 dpi est le standard d'impression. Le dpi n'a de sens qu'accompagné d'une taille physique (mm).
résolution utile	pixels disponibles par unité de donnée : ici 25 px par mois en 96 dpi et 79 px par mois en 300 dpi.
support	la taille physique à laquelle la figure sera imprimée ou affichée : 160 × 90 mm pour les figures du module.
graphique natif	graphique créé dans le classeur (Excel) ou le rapport (Power BI), relié à ses cellules ou à ses champs : il se met à jour quand la donnée change.
export figé	image ou PDF détaché de sa source : il ne se met plus à jour, mais il ne se déforme pas non plus.
relire classeur	un ouvrir le fichier produit par le programme, comparer ses totaux au socle et constater l'écart. C'est ce qui distingue « écrit » de « vérifié ».
cité exécuté	(non) statut d'un outil décrit sans avoir été ouvert dans l'atelier. Le manuel l'impose dès qu'une procédure est présentée : §1.5.

5. Cours approfondi

5.1 Le contrat des trois outils

Le module ne cherche pas à désigner un gagnant : il fixe un **contrat** par outil, et ce contrat décide de l'export.

- **matplotlib** est l'outil de la **reproductibilité**. Un script de **14** lignes produit la figure en quatre formats ; deux exécutions donnent des fichiers identiques au bit près. C'est l'outil des documents qui doivent pouvoir être refaits dans six mois.
- **Excel** est l'outil de la **discussion**. Le classeur est modifiable, le graphique est natif (il suit les cellules), et n'importe qui peut changer une période sans savoir écrire une ligne de code. C'est l'outil des comités et des équipes métier — à condition de **relire** ce qu'on produit.
- **Power BI** est l'outil de la **diffusion**. Le rapport se met à jour seul, se consulte dans un navigateur, se filtre ; il est fait pour un usage répété par des gens qui ne touchent pas à la donnée. C'est aussi l'outil le moins reproductible hors de son écosystème : le rapport vit dans un service, pas dans un fichier qu'on peut lire ligne à ligne.

Ce contrat explique pourquoi le chapitre produit **trois fois** la même figure sans que ce soit du travail perdu : chaque version a un usage. Le livrable du projet « La refonte » demandera précisément les trois (PNG pour la note, classeur pour l'équipe, rapport cité pour la direction).

Définition. On appelle **contrat d'outil** l'ensemble des trois questions auxquelles un outil répond dans une chaîne de production : qui le modifie ?, à quelle fréquence ?, sur quel support arrive-t-il ? Un outil choisi sans ces trois réponses produit des fichiers que personne ne réutilise — et un graphique qu'on refera trois fois.

5.2 matplotlib, exécuté : quatorze lignes, quatre formats

Le graphique du chapitre est celui du fil rouge, en version courte : le CA net mensuel sur **24** mois avec sa moyenne glissante, axe à zéro déclaré (C04), une annotation, un titre qui affirme. Le bloc `figure_graphique()` du script tient en **14** lignes de code — c'est le chiffre à retenir : un graphique professionnel correct n'est pas un exploit technique, c'est une suite de décisions déjà prises dans les chapitres précédents.

Quatre formats sortent du même code, et leur comparaison est le cœur du chapitre :

Format	Taille	Poids	Usage
PNG 96 dpi	604 × 340 px	23,4 Ko	écran, diapositive, courriel
PNG 300 dpi	1 889 × 1 062 px	96,1 Ko	impression, rapport PDF mis en page
SVG	69 points tracés	12,9 Ko	document web, retouche ultérieure
PDF	1 page, mêmes tracés	17,3 Ko	rapport, dossier imprimé

Le bloc de production porte aussi le **nom des fichiers** : `fil_rouge_png_96.png`, `fil_rouge_png_300.png`, `fil_rouge.svg`, `fil_rouge.pdf`. Ce n'est pas de la coquetterie : dans un dossier partagé, un fichier nommé `graphique_final_v3.png` oblige à l'ouvrir pour savoir ce qu'il contient, et c'est ainsi qu'une note de direction part à l'imprimeur en 96 dpi.

Deux remarques techniques qui valent pour toute la suite du manuel :

- **Le dpi ne dit rien sans le support.** « 300 dpi » n'a de sens qu'avec une taille : les mêmes 300 dpi sur **160 × 90** mm donnent 1 889 × 1 062 px ; à 300 mm de large, la même figure demanderait 3 543 px. La règle du module : on fixe le **support** (le mm), puis la résolution.
- **Le déterminisme se contrôle fichier par fichier.** Le SVG était déjà vérifié (C02 à C06) ; ici, le PNG, le PDF et le classeur le sont aussi. C'est ce qui autorise la phrase « deux exécutions donnent le même fichier », et c'est cette phrase qui rend un livrable vérifiable des mois plus tard.

Attention. « 300 dpi » est une résolution **relative** : sans la taille du support, elle ne décrit rien. Les mêmes 300 dpi donnent 1 889 × 1 062 px sur un support de 160 × 90 mm et près du double sur une pleine page. Une figure livrée « en 300 dpi » mais destinée à un support deux fois plus large que prévu arrive sous-résolue — et c'est l'imprimeur qui le découvre.

5.3 Excel, exécuté : le classeur écrit et relu

Excel est **exécuté** dans ce chapitre : `openpyxl` écrit un classeur (`Donnees`, **26** lignes : un en-tête, 24 mois, un total), y place un **graphique natif** de type courbe relié aux cellules, puis — c'est le point important — **le classeur est relu** par le programme, ses 24 valeurs sont comparées à celles du socle, et l'écart est mesuré : **0,00** FCFA.

Cette relecture n'est pas une formalité. Un classeur est un objet vivant : quelqu'un l'ouvrira, triera une colonne, ajoutera un mois, et **le graphique suivra** — c'est sa force et son risque. Trois pratiques rendent le classeur professionnel :

1. **Une feuille de contrôle**, avec le total du classeur, le total attendu et l'écart. Sur le fil rouge : total attendu 7 876 320 164,91 FCFA, écart calculé par formule. Un classeur sans contrôle est un classeur qu'on croit sur parole.
2. **Des données brutes intouchées.** Les valeurs restent celles du socle, sans tri ni filtre « pour faire joli » : le tri se fait dans le graphique, pas dans la source (C04).
3. **Un graphique natif, pas une image collée.** Coller une image dans un classeur, c'est renoncer à la seule chose qu'Excel fait mieux que les autres : la mise à jour automatique.

Deux limites, à déclarer : le **rendu** du graphique natif demande l'application Excel, qui n'est pas ouverte dans cet atelier (le fichier est écrit, relu et vérifié, mais pas capturé) ; et le classeur n'est **pas** reproductible au sens de matplotlib — il n'a pas de « version du script », seulement un historique de fichiers. C'est pourquoi le projet du module demande les deux : le classeur pour l'équipe, le script pour la trace.

Attention. Ne jamais livrer un classeur sans l’avoir rouvert. Les erreurs les plus fréquentes dans un classeur transmis ne sont pas des erreurs de formule : ce sont des **périmètres** — une plage qui s’arrête un mois trop tôt, un tri appliqué après le graphique, un total qui porte sur 23 valeurs au lieu de 24. La relecture du chapitre mesure exactement cela : **24** valeurs comparées, écart **0,00** FCFA.

5.4 Power BI, cité : neuf clics et une déclaration

Power BI n’est **pas exécuté** dans ce chapitre. C’est une décision de méthode, pas un manque : le manuel applique la règle §1.5, déjà utilisée pour PostgreSQL (cité) contre DuckDB et SQLite (exécutés). Ce qui est présenté ici est donc une **procédure**, en neuf étapes, sans capture d’écran inventée :

1. ouvrir le classeur source (ou la base) comme source de données ;
2. créer la mesure du CA net (la définition des métriques est celle de C02) ;
3. glisser le champ mois et la mesure sur le canevas ;
4. choisir un graphique en courbes et vérifier l’axe (zéro, unité, période) ;
5. formater : titre qui affirme, étiquettes, ordre, couleurs de la charte (C03) ;
6. choisir le support d’export (PDF pour le comité, image pour la note) ;
7. régler les filtres par défaut (période, périmètre) ;
8. publier le rapport dans un espace de travail ;
9. définir la fréquence d’actualisation et vérifier la première.

Neuf clics décrits, zéro fichier produit : la phrase qui accompagne cette procédure dans le livrable est donc « **cité, non exécuté** ». C’est une discipline qui a une conséquence pratique : personne dans l’entreprise ne pourra dire que le module a validé un réglage qu’il n’a pas ouvert — et si un apprenant dispose de Power BI, il vérifiera chaque étape et **mesurera** ce qui manque au chapitre.

Définition. Un outil est dit **cité** (ou « non exécuté ») quand sa procédure est décrite sans avoir été mise en œuvre dans l’atelier. Le statut doit apparaître dans le livrable lui-même, pas seulement dans les notes de l’auteur : c’est la règle §1.5, et elle vaut pour tous les outils du parcours (Power Query en M05, PostgreSQL en M07, Power BI en M10, Tableau en M16, automatisation en M17).

Définition. La **résolution utile** d’une figure est le nombre de pixels disponibles par unité de donnée : **25** px par mois en 96 dpi, **79** px par mois en 300 dpi, pour le même graphique de **24** mois. C’est la seule mesure qui dise si une figure est exploitable : un fichier « en 300 dpi » dont le support est deux fois plus grand n’a pas plus de résolution utile qu’un fichier en 96.

Définition. Un **export** est la conversion d’une figure vers un fichier destiné à un usage précis (écran, impression, retouche, diffusion). Un export n’est donc jamais « le » fichier d’une figure : c’est **un** fichier, choisi pour **un** usage, et il se nomme en conséquence.

5.5 Support et résolution : la règle des 300 dpi

La règle professionnelle du module tient en trois lignes :

1. **Écran : 96 dpi suffisent**, à la taille d’affichage réelle. Une figure de **160 × 90** mm vue sur un écran à 25 cm ne demande pas plus ; au-delà, on transmet des pixels que personne ne verra.

2. **Impression : 300 dpi**, jamais moins pour un graphique de données — les traits fins (0,9 pt) et les étiquettes de 5 à 7 pt doivent rester nets, et un texte de 6 pt à 96 dpi devient une tache.
3. **Support fini d'abord, résolution ensuite**. On écrit la taille en mm, puis on calcule les pixels : à 300 dpi, **160 × 90** mm donnent 1 889 × 1 062 px. L'inverse (fabriquer une grande image puis la réduire) produit des textes écrasés.

La mesure du chapitre donne le coût réel de cette règle : passer de 96 à 300 dpi multiplie le nombre de pixels par **9,8** (604 × 340 contre 1 889 × 1 062) et le poids du fichier par **4,1** seulement — la compression du PNG absorbe une partie de la différence, parce qu'un graphique de traits se comprime mieux qu'une photographie. Conclusion pratique : **la qualité d'impression est bon marché** ; il n'y a aucune raison de livrer une figure de rapport en 96 dpi.

5.6 Export vectoriel : le format des graphiques de traits

Le SVG et le PDF ne stockent pas des pixels mais des **tracés** : la figure du chapitre en contient **69** points, et la totalité des traits, des textes et des axes reste décrite. Trois conséquences mesurables :

- **Le poids est faible** : **12,9** Ko en SVG et **17,3** Ko en PDF, moins que le PNG 96 dpi.
- **L'agrandissement est gratuit** : passer de 160 mm à 400 mm de large ne change ni la taille du fichier ni la netteté ; un PNG devrait être régénéré à 750 dpi, pour un poids multiplié par six.
- **Le texte reste du texte** (avec `svg.fonttype` en mode texte) : dans un PDF, on peut rechercher, copier et, dans une certaine mesure, retoucher les étiquettes.

Deux limites, à connaître : le vectoriel **ne convient pas aux images** (une carte, une photographie de fond) et il peut **varier selon le lecteur** — un SVG contenant des polices non embarquées ne s'affichera pas de la même façon partout. La règle du module : pour un graphique de traits destiné à un document, **vectoriel d'abord** ; pour un écran ou un courriel, PNG à la résolution du support.

5.7 Quelle chaîne pour quel livrable

Le tableau de décision du chapitre, à recopier dans son carnet :

Livrable	Outil	Format	Contrôle avant envoi
note de direction (3 écrans, C06)	matplotlib	PNG 300 dpi ou PDF	titre local, axe déclaré, chiffre relu
rapport imprimé	matplotlib	PDF vectoriel	polices, marges, 160 × 90 mm
page web ou intraNet	matplotlib	SVG	polices embarquées, couleurs de la charte
classeur pour l'équipe	Excel	.xlsx, natif	graphique relecture : totaux et périmètre
comité qui suit un indicateur chaque mois	Power BI	rapport publié	procédure déclarée, actualisation datée
archivage	matplotlib Excel	+ script + classeur	deux exécutions identiques (déterminisme)

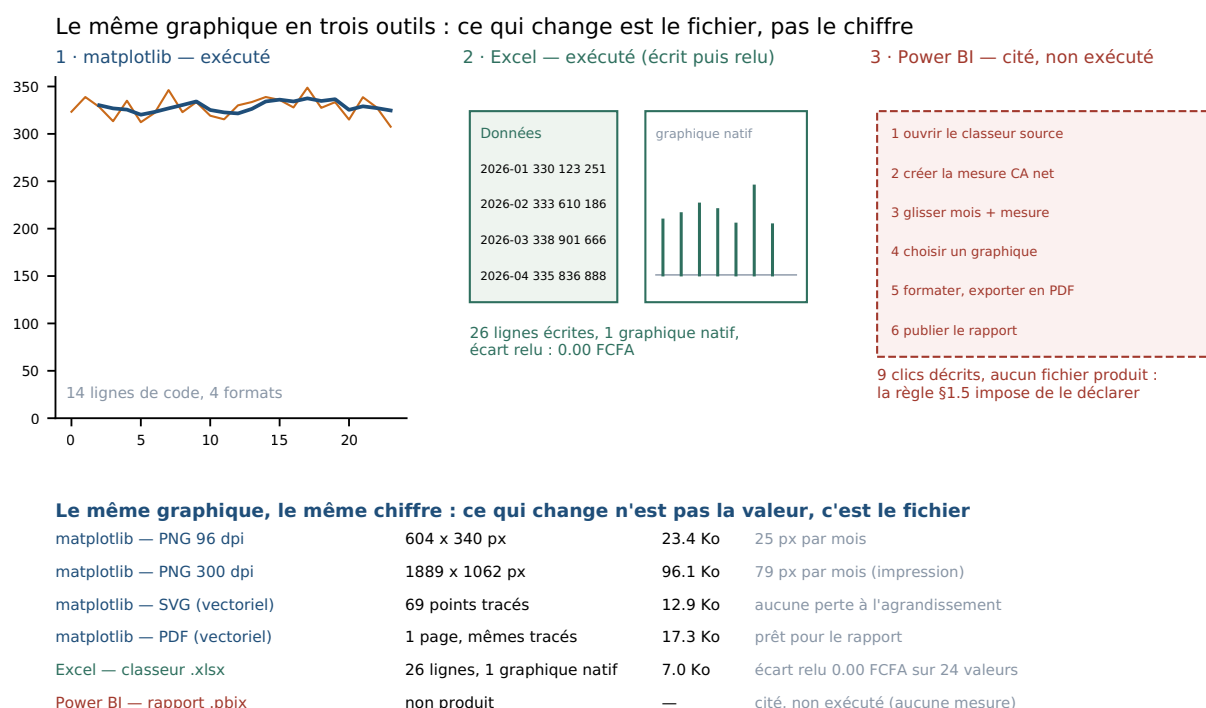
Ce tableau répond à la question posée en §5.1 : chaque livrable a **un** outil de production, **un** format et **un** contrôle. Les chaînes hybrides — un PNG Excel collé dans un rapport matplotlib, par exemple — sont celles qui produisent les incohérences de chiffres et les figures floues.

Conseil professionnel. Nommez vos fichiers comme vos figures : `fil_rouge_png_300.png`, `fil_rouge.svg`, `fil_rouge.xlsx`. Le nom porte le format **et** l'usage, et il évite la seule question qui fait perdre du temps à tout le monde : « c'est lequel, la version pour l'imprimeur ? ».

À retenir. Trois outils, un chiffre : le format d'export ne change **jamais** la donnée, il change le fichier. Le choix du format est donc un choix d'usage — écran, impression, retouche, diffusion — et il se décide **avant** de produire, pas après la réunion.

6. Exemple concret — le même graphique dans trois outils

La planche du chapitre met les trois productions côte à côte, puis compare les fichiers un par un.



Le même graphique, le même chiffre : ce qui change n'est pas la valeur, c'est le fichier

matplotlib — PNG 96 dpi	604 x 340 px	23.4 Ko	25 px par mois
matplotlib — PNG 300 dpi	1889 x 1062 px	96.1 Ko	79 px par mois (impression)
matplotlib — SVG (vectoriel)	69 points tracés	12.9 Ko	aucune perte à l'agrandissement
matplotlib — PDF (vectoriel)	1 page, mêmes tracés	17.3 Ko	prêt pour le rapport
Excel — classeur .xlsx	26 lignes, 1 graphique natif	7.0 Ko	écart relu 0.00 FCFA sur 24 valeurs
Power BI — rapport .pbix	non produit	—	cité, non exécuté (aucune mesure)

Support : 160 x 90 mm · 24 mois · CA net de 307,6 à 348,8 M FCFA · 2025 : 3 910,9 M → 2026 : 3 965,4 M (1,4 %)

3 outils : 2 exécutés, 1 cité — le chiffre du graphique est identique dans les trois cas (écart mesuré 0.00 FCFA).

Ce que la planche montre :

- **Panneau 1 — matplotlib, exécuté.** Le graphique du fil rouge (24 mois, moyenne glissante, axe à zéro) avec sa mention : 14 lignes de code, 4 formats produits.
- **Panneau 2 — Excel, exécuté.** Le classeur tel qu'il est écrit : la feuille Données (26 lignes), le graphique natif relié aux cellules, et le résultat de la relecture — 0,00 FCFA d'écart sur 24 valeurs.
- **Panneau 3 — Power BI, cité.** Les neuf étapes de la procédure, encadrées en pointillés, avec la mention obligatoire : aucun fichier produit.
- **La bande du milieu** compare les fichiers : 604 x 340 px et 23,4 Ko en PNG 96 dpi ; 1 889 x 1 062 px et 96,1 Ko en PNG 300 dpi ; 69 points tracés pour 12,9 Ko en SVG ; 1 page pour 17,3 Ko en PDF ; 7,0 Ko pour le classeur ; et « non produit » pour Power BI.

- **La bande du bas** rappelle le support (**160 × 90 mm**), le périmètre (24 mois, CA net de **307,6 à 348,8 M FCFA**, **3 910,9 M** en 2025 → **3 965,4 M** en 2026, **+1,4 %**) et la règle du chapitre : deux outils exécutés, un cité, un chiffre identique.

7. Démonstration pas à pas — sortir un graphique en 5 gestes

Geste 1 — fixer le support, puis produire. Une ligne de décision, une ligne de code.

```
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt, pandas as pd, numpy as np
LARGEUR_MM, HAUTEUR_MM = 160.0, 90.0
fig = plt.figure(figsize=(LARGEUR_MM/25.4, HAUTEUR_MM/25.4), dpi=96)
print("support :", LARGEUR_MM, "x", HAUTEUR_MM, "mm ->",
      round(LARGEUR_MM/25.4*96), "x", round(HAUTEUR_MM/25.4*96), "px a 96 dpi")
print("a 300 dpi :", round(LARGEUR_MM/25.4*300), "x", round(HAUTEUR_MM/25.4*300), "px")
```

```
support : 160.0 x 90.0 mm -> 604 x 340 px a 96 dpi
a 300 dpi : 1889 x 1062 px
```

Geste 2 — produire les quatre formats et lire leur poids.

```
import os
for nom, dpi in (("png_96", 96), ("png_300", 300)):
    fig.savefig(f"exports/fil_rouge_{nom}.png", dpi=dpi, metadata={"Software": None, "Date":
None})
for ext in ("svg", "pdf"):
    fig.savefig(f"exports/fil_rouge.{ext}")
print({f: round(os.path.getsize(f"exports/fil_rouge.{e}")/1024, 1)
      for f, e in (("png96", "png"), ("svg", "svg"), ("pdf", "pdf"))})
```

```
{'png96': 23.4, 'svg': 12.9, 'pdf': 17.3}
```

Geste 3 — écrire le classeur, avec sa feuille de contrôle.

```
from openpyxl import Workbook
wb = Workbook(); ws = wb.active; ws.title = "Donnees"
ws.append(["mois", "ca_net_fcfa"])
for periode, valeur in ca.items():
    ws.append([str(periode), float(valeur)])
ws.append(["Total", float(ca.sum())])
ws["D4"], ws["E4"] = "Ecart", "=ROUND(E3-E2,2)"
wb.save("exports/fil_rouge.xlsx")
print("lignes ecrites :", ws.max_row, "| graphique natif : ajoute par openpyxl.chart")
```

```
lignes ecrites : 26 | graphique natif : ajoute par openpyxl.chart
```

Geste 4 — relire le classeur et comparer au socle : c'est le geste qui transforme « écrit » en « vérifié ».

```
from openpyxl import load_workbook
ws = load_workbook("exports/fil_rouge.xlsx")["Donnees"]
valeurs = [float(ws.cell(row=r, column=2).value) for r in range(2, 26)]
ecart = float(np.abs(np.array(valeurs) - ca.to_numpy()).max())
print(f"valeurs comparees : {len(valeurs)} | ecart max : {ecart:.2f} FCFA")
print("graphiques natifs dans la feuille :", len(ws._charts))
```

```
valeurs comparees : 24 | ecart max : 0.00 FCFA
graphiques natifs dans la feuille : 1
```

Geste 5 — choisir le format selon le livrable (§5.7), et le nommer.

```
livrables = {"note de direction": "fil_rouge_png_300.png (96.1 Ko, 1889 x 1062 px)",
             "rapport imprime": "fil_rouge.pdf (17.3 Ko, vectoriel)",
             "page web": "fil_rouge.svg (12.9 Ko, 69 points traces)",
             "equipe qui retouche": "fil_rouge.xlsx (7.0 Ko, graphique natif)"}
for cible, fichier in livrables.items():
    print(f"{cible:22s} -> {fichier}")
```

```
note de direction      -> fil_rouge_png_300.png (96.1 Ko, 1889 x 1062 px)
rapport imprime        -> fil_rouge.pdf (17.3 Ko, vectoriel)
page web               -> fil_rouge.svg (12.9 Ko, 69 points traces)
equipe qui retouche    -> fil_rouge.xlsx (7.0 Ko, graphique natif)
```

Le geste 5 mérite un mot : choisir un format **après** avoir produit est une perte, parce que le format contraint la production (marges, tailles de police, épaisseur des traits). Une figure destinée au web se règle sur des pixels, une figure destinée à l'impression sur des millimètres ; produire une seule fois et exporter quatre formats fonctionne ici parce que le support est fixé dès le départ (**160 × 90 mm**).

Ce que la démonstration établit. Le choix d'un format n'est pas une question de goût : c'est une suite de mesures (poids, pixels, points, écart) confrontée à un usage. Et le seul contrôle qui compte avant l'envoi — la relecture du fichier produit — se fait en trois lignes de code.

5.8 Ce que le chapitre n'apprend pas

Deux sujets sont volontairement laissés aux modules suivants, et il vaut mieux le dire clairement : **l'automatisation** de cette chaîne (rejouer les exports à chaque mise à jour de la donnée, sans intervention) est le sujet de M17 ; la **mise en page** d'un rapport complet — pagination, sommaire, blocs, styles — relève de M22 et de l'appareil du manuel. Ce chapitre s'arrête à la frontière qui lui donne son sens : un fichier juste, au bon format, sur le bon support, pour le bon destinataire.

8. Erreurs fréquentes

1. **La capture d'écran comme export.** Elle est en 96 dpi, contient l'interface, et n'est ni reproductible ni nette à l'impression. Le module l'interdit dans les livrables.
2. **Le PNG 300 dpi dans une page web.** 96,1 Ko pour un affichage de moitié de la définition : le lecteur paie des pixels qu'il ne voit pas.
3. **Le PDF rasterisé.** Un PDF qui contient une image agrandie n'est pas vectoriel : le texte devient flou et le fichier lourd. Le contrôle est simple : zoomer à 400 % — si ça pixellise, ce n'est pas du vectoriel.
4. **Le classeur non relu.** Un total calculé sur 23 mois au lieu de 24 ne se voit pas à l'œil ; la relecture contre le socle le voit (**0,00 FCFA** attendu contre une valeur fausse).
5. **Le graphique collé en image dans le classeur.** Il ne suivra pas les cellules : c'est un export figé déguisé en graphique natif.
6. **Power BI présenté comme exécuté.** Une capture inventée est une faute trois fois : elle trompe l'apprenant, elle fabrique une preuve, et elle contredit la règle §1.5 du manuel.
7. **Un seul format pour tous les usages.** Le PNG 96 dpi envoyé à l'imprimeur, ou le SVG inséré dans une note de direction : le livrable est prêt, l'outil est le bon, le format ne l'est pas.

9. Bonnes pratiques professionnelles

1. **Décider du support avant de produire** : 160 × 90 mm pour les figures du module, puis 96 dpi pour l'écran, 300 dpi pour l'impression.
2. **Produire le vectoriel par défaut** pour les graphiques de traits : le poids est plus faible et l'agrandissement gratuit.
3. **Relire chaque fichier produit** : identité des valeurs avec le socle, présence du graphique natif, écart de contrôle. Ce qui n'est pas relu n'est pas livré.
4. **Déclarer le statut des outils** : exécuté, ou cité — jamais « exécuté » pour une procédure décrite de mémoire (§1.5).
5. **Nommer les fichiers par usage** (`fil_rouge_png_300.png`, `fil_rouge.svg`, `fil_rouge.xlsx`).
6. **Un outil de production par livrable** (§5.7), et pas de chaîne hybride non déclarée.
7. **Archiver la paire** : le script (ou le classeur) **et** les fichiers produits — c'est la seule façon de reconstituer une figure dans six mois.

10. Exercice guidé — « l'audit d'un export » (15 min, /10)

Contexte. On vous transmet quatre fichiers du même graphique : `fil_rouge_png_96.png` (604 × 340 px, **23,4** Ko), `fil_rouge_png_300.png` (1 889 × 1 062 px, **96,1** Ko), `fil_rouge.svg` (**12,9** Ko) et `fil_rouge.xlsx` (**7,0** Ko, 26 lignes, 1 graphique natif). Le graphique doit partir à l'imprimeur pour un rapport A4, en 160 mm de large.

Consigne.

1. Quel(s) fichier(s) partent à l'imprimeur, et pourquoi les autres non ? (3 pts)
2. Vérifiez la cohérence : le facteur de poids entre les deux PNG (**4,1**) et le nombre de points tracés du SVG (**69**) vous paraissent-ils normaux pour une figure de traits ? (2 pts)
3. Écrivez les trois contrôles que vous faites sur le classeur avant de l'envoyer à l'équipe (péri-mètre, total, graphique natif). (3 pts)
4. Le commanditaire demande aussi une version « pour le site intranet ». Quel fichier, et quelle précaution ? (2 pts)

11. Exercices autonomes

E1 — la planche des quatre exports (45 min). Prenez un graphique que vous avez déjà produit et sortez-le en quatre versions : PNG 96 dpi, PNG 300 dpi, SVG et un classeur Excel avec graphique natif. Tenez un tableau : format, poids en Ko, dimensions, usage prévu. Puis **relisez** le classeur (valeurs contre la source, écart mesuré) et notez l'écart. Livrez le tableau et une phrase de conclusion sur le format que vous choisiriez pour un rapport imprimé et pour un courriel.

E2 — la fiche de procédure d'un outil cité (30 min). Choisissez un outil que vous n'avez pas à votre disposition (Power BI, Tableau, Looker Studio). Écrivez sa procédure en **neuf étapes maximum**, du branchement de la source à la publication, puis ajoutez la mention de statut : « cité, non exécuté dans cet atelier ». Terminez par la liste des points que vous vérifieriez **si** vous l'aviez — c'est-à-dire la liste des affirmations que votre fiche ne peut pas prouver.

12. Correction détaillée

Exercice guidé.

1. **À l'imprimeur** : le PNG 300 dpi (1 889 × 1 062 px couvrent 160 × 90 mm à 300 dpi) ou le PDF vectoriel. Le PNG 96 dpi ne suffit pas (les étiquettes de 5 à 7 pt deviennent des taches) ; le classeur n'est pas un format d'impression ; le SVG est imprimable mais dépend du lecteur.
2. **Cohérence** : **4,1** de facteur de poids pour un facteur de près de **9,8** en pixels est normal — un graphique de traits se comprime bien ; **69** points tracés pour **12,9** Ko est également cohérent (le SVG stocke des tracés, pas des pixels).
3. **Trois contrôles du classeur** : le **périmètre** (24 mois, ni 23 ni 25), le **total** (comparé au socle, écart **0,00** FCFA), le **graphique natif** (présent, relié aux cellules, et non une image collée).
4. **Pour le site intranet** : le SVG (**12,9** Ko, net à toute taille), avec la précaution des polices — vérifier le rendu dans le navigateur cible, sinon livrer un PNG 300 dpi.

E1 (la planche des quatre exports). Attendu : quatre fichiers existants (vérifiables), un tableau avec les poids réels, et un écart de relecture du classeur mesuré. Un élève qui affirme « le classeur est bon » sans l'avoir relu rend une affirmation, pas un livrable.

E2 (la fiche de procédure). Attendu : neuf étapes maximum, la mention de statut en tête ou en pied de fiche, et la liste des points non vérifiés. Le repère du chapitre : Power BI, **9** clics décrits, **aucun** fichier produit — et c'est écrit dans le livrable lui-même.

13. Mini-projet M10.P7 — « le même graphique en trois outils » (1 h 30, /18)

Énoncé. Troisième livrable du projet « La refonte ». Vous produisez la même figure dans les trois outils, avec ses fichiers, ses mesures et ses déclarations.

1. **matplotlib (exécuté)** : la figure au format **160 × 90** mm, exportée en PNG 96 dpi, PNG 300 dpi, SVG et PDF ; poids et dimensions relevés pour chacun. (5 pts)
2. **Excel (exécuté)** : le classeur avec les données, la feuille de contrôle (total du classeur, total attendu, écart) et un **graphique natif** relié aux cellules ; le classeur est **relu** et l'écart mesuré est rendu. (5 pts)
3. **Power BI (cité)** : la procédure en neuf étapes, avec la mention « cité, non exécuté » dans le livrable. (3 pts)
4. **La comparaison** : un tableau des fichiers (format, poids, dimensions, usage) et la conclusion chiffrée — facteur de poids entre les deux PNG, pixels par mois en 96 et 300 dpi. (3 pts)
5. **La règle** : une phrase qui répond, pour chaque fichier, à la question « qui va l'utiliser, et sur quel support ? ». (2 pts)

Barème indicatif : quatre exports matplotlib conformes et mesurés (5), classeur relu avec écart mesuré (5), procédure Power BI déclarée (3), tableau de comparaison exact (3), phrase d'usage par fichier (2), sur 18.

Pourquoi ce mini-projet. Parce qu'un livrable se juge sur ses fichiers, pas sur ses intentions : quatre exports mesurés, un classeur relu et une procédure déclarée valent mieux qu'un long rapport sur « la bonne façon de faire ».

14. Boîte à outils du chapitre

Boîte à outils. Quatre nouveaux gestes, tous exécutés dans la planche et dans les fichiers du dossier : `fig.savefig(..., dpi=96 / 300)` (les deux résolutions), `format="svg" | "pdf"` (le vectoriel), `openpyxl.chart.LineChart` (le graphique natif du classeur) et `load_workbook` (la **relecture**, qui rend le classeur vérifié).

Besoin	Commande	Ce qu'elle donne	Ce qu'elle ne donne pas
résolution pour l'impression	<code>savefig(dpi=300, taille en pouces)</code>	1 889 × 1 062 px pour 160 × 90 mm	la vérification visuelle : elle reste à faire
fichier léger et net	<code>format="svg"</code>	12,9 Ko, 69 points tracés	la compatibilité des polices hors de l'atelier
classeur modifiable	<code>openpyxl + LineChart</code>	26 lignes, 1 graphique natif	le rendu : il demande l'application Excel
classeur vérifié	<code>load_workbook</code> comparaison	puis un écart mesuré (0,00 FCFA)	la mise en forme : elle se juge à l'œil
outils non installés	procédure écrite + déclaration	une fiche honnête (9 clics cités)	toute mesure : aucun fichier produit

15. Résumé du chapitre

C07 a fermé le module M10 par le dernier maillon : la **sortie**. Le même graphique — le fil rouge, **24** mois, axe à zéro, moyenne glissante — est produit dans trois outils et, dans les trois cas, le chiffre est **identique** (relecture du classeur : **0,00** FCFA d'écart sur **24** valeurs). Ce qui change est le fichier : 604 × 340 px et **23,4** Ko en PNG 96 dpi, 1 889 × 1 062 px et **96,1** Ko en PNG 300 dpi (facteur **4,1** pour un support de **160 × 90** mm), **12,9** Ko en SVG (**69** points tracés), **17,3** Ko en PDF, **7,0** Ko pour le classeur à graphique natif. `matplotlib` et Excel sont **exécutés** — le classeur est écrit puis **relu** —, Power BI est **cité** en **9** étapes sans fichier produit : la règle §1.5 appliquée aux outils. Et la règle pratique du chapitre : support en mm, **300 dpi** pour l'impression, **vectoriel** pour un graphique de traits, un outil de production par livrable.

16. À retenir

À retenir. Le format d'export ne change jamais la donnée : il change le fichier, et donc l'usage. Choisir un format, c'est répondre trois fois : qui l'utilise, sur quel support, et qui le modifiera ensuite ? Toute autre considération (le poids « pour faire propre », le format « parce que c'est celui du service ») produit des livrables que personne ne réutilise.

1. **Support d'abord** : **160 × 90** mm pour les figures du module, puis 96 dpi (écran) ou **300** dpi (impression).
2. **Vectoriel par défaut** pour les graphiques de traits : **12,9** Ko en SVG, **17,3** Ko en PDF, net à toute taille.
3. **Trois fois plus de résolution** (96 → 300 dpi) coûte **4,1** fois plus de poids — et c'est bon marché pour l'impression.
4. **Un classeur se relit** : valeurs contre le socle, périmètre, graphique natif. Écart attendu ici : **0,00** FCFA sur 24 valeurs.
5. **matplotlib et Excel** : **exécutés** ; **Power BI** : **cité** (**9** clics décrits, aucun fichier produit). Le statut est écrit dans le livrable.
6. **Un outil de production par livrable**, un nom de fichier par usage.

7. **Archiver la paire** : le script (ou le classeur) **et** les fichiers produits.

17. Évaluation formative (auto-correction, 8 min)

1. **Pourquoi le même graphique pèse-t-il 23,4 Ko en 96 dpi et 96,1 Ko en 300 dpi ?** → Parce que la résolution multiplie les pixels (604×340 contre $1\,889 \times 1\,062$, soit près de 9,8 fois plus) sans ajouter d'information ; la compression d'un graphique de traits limite le poids à un facteur **4,1**. Le support (**160 × 90** mm) ne change pas. (2 pts)
2. **Quel format pour un rapport imprimé, et pourquoi pas un PNG 96 dpi ?** → PDF vectoriel ou PNG 300 dpi. Le PNG 96 dpi place **25** px par mois là où l'impression en demande **79** : les étiquettes de 5 à 7 pt deviennent illisibles. (2 pts)
3. **Que signifie « Excel est exécuté » dans ce chapitre, concrètement ?** → Le classeur est écrit (**26** lignes, 1 graphique natif relié aux cellules) **et relu** : ses **24** valeurs sont comparées au socle, écart mesuré **0,00** FCFA. Écrire sans relire ne serait pas un contrôle. (2 pts)
4. **Que doit contenir la fiche d'un outil cité, et pourquoi ce statut est-il obligatoire ?** → La procédure (neuf étapes pour Power BI) **et** la mention « cité, non exécuté » : aucune mesure ne peut être affirmée sans fichier produit. Le statut protège l'apprenant (règle §1.5) et le formateur : c'est la même règle que PostgreSQL en M07 ou Power Query en M05. (2 pts)
5. **Trois contrôles avant d'envoyer un classeur ?** → Le **périmètre** (24 mois), le **total** (comparé au socle, écart **0,00** FCFA) et le **graphique natif** (présent et relié aux cellules, non collé en image). (2 pts)

Module M11.C01 — La fonction de fenêtre : `OVER`, `PARTITION BY`, `ORDER BY`, et pourquoi `GROUP BY` ne suffit plus

Analyse de données appliquée & Business Intelligence
9 partie(s) assemblée(s) · document de production

Table des matières

MODULE M11.C01 — LA FONCTION DE FENÊTRE : OVER, PARTITION BY, ORDER BY, ET POURQUOI GROUP BY NE SUFFIT PLUS	8
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le tableau d’affichage du stade	9
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 L’ordre d’exécution : pourquoi une fenêtre n’est pas un agrégat	10
5.2 OVER () : la fenêtre sur toute la table	11
5.3 PARTITION BY : le facteur de comparaison	11
5.4 ORDER BY dans la fenêtre : l’ordre change la valeur	11
5.5 Le cadre : cumul et moyenne glissante	12
5.6 La même question, deux écritures : fenêtre contre sous-requête scalaire	12
5.7 Les trois pièges, mesurés	13
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — la part de chaque magasin, et ce qu’elle décide	14
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — le total de contrôle	14
7.2 Étape 2 — la part, avec la fenêtre	15
7.3 Étape 3 — la même réponse par sous-requête scalaire	15
7.4 Étape 4 — une part calculée dans son année	15
7.5 Étape 5 — le piège : le filtre et la fenêtre dans le même SELECT	15
7.6 Étape 6 — la fenêtre n’a pas réduit la table	16
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C02 — CLASSER, CUMULER, DÉCALER : ROW_NUMBER, RANK, DENSE_RANK, NTILE, SUM() OVER, LAG, LEAD	21
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le podium et les dossards	22
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 Trois fonctions de rang, une seule mesure	23
5.2 Le départage : ce qui rend un classement reproductible	23

5.3 NTILE : découper en tranches, pas classer	24
5.4 Le top N par groupe : le motif qui exige un filtre après	24
5.5 SUM() OVER : le cumul, et sa clôture	25
5.6 LAG et LEAD : la variation, et son premier trou	25
5.7 Le classement qui contredit le total	25
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — trois classements, un seul socle	26
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — mesurer l'ampleur des égalités avant de classer	27
7.2 Étape 2 — les trois rangs côte à côte	27
7.3 Étape 3 — l'écart maximal entre les deux classements	27
7.4 Étape 4 — le cumul, et son contrôle de clôture	27
7.5 Étape 5 — la variation mensuelle, puis annuelle	27
7.6 Étape 6 — la comparaison annuelle, et le piège du filtre	28
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C03 — LE TEMPS EN SQL : TABLE CALENDRIER, ANNÉE SUR ANNÉE, MOIS GLISSANTS, ET LE MOIS QUI MANQUE	33
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — l'agenda et les trous	34
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 Fabriquer l'axe du temps	35
5.2 Le trou qu'on ne voit pas : l'objectif manquant	35
5.3 Année sur année : le périmètre d'abord	36
5.4 Moyennes mobiles : la fenêtre pleine et son compteur	37
5.5 La saison, à connaître avant d'interpréter	37
5.6 Le mois partiel : la dernière ligne du tableau	37
5.7 Ce que ce chapitre ne couvre pas	38
6. Exemple concret — trois lectures de la même année	38
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — compter l'axe du temps	39
7.2 Étape 2 — le CA mensuel, et la preuve qu'il n'y a pas de trou	39
7.3 Étape 3 — les objectifs, et leurs deux trous	39
7.4 Étape 4 — année sur année à périmètre égal	39
7.5 Étape 5 — moyennes mobiles avec compteurs	39
7.6 Étape 6 — la saison, pour interpréter	39

8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C04 — COHORTES, RÉTENTION ET RÉCURRENCE : RFM, ET LE CLIENT QUI N'A PAS DE NOM	44
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — la promotion d'une année, et le verre à moitié plein	45
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 La cohorte d'entrée : 44 lignes, une seule par client	46
5.2 La matrice : un triangle, jamais un rectangle	46
5.3 Le piège des deux lectures	47
5.4 Une rétention qui ne baisse pas	48
5.5 Le RFM : trois mesures, une décision	49
5.6 Fréquence, récurrence, récence : les trois chiffres qui tiennent	49
5.7 Le client 0 : 18,3 % du CA sans porteur	50
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — trois façons de répondre à la même question	50
7. Démonstration pas à pas — cinq étapes sur le socle	51
7.1 Étape 1 — la cohorte d'entrée	51
7.2 Étape 2 — la matrice et son triangle	51
7.3 Étape 3 — les deux lectures, côte à côte	51
7.4 Étape 4 — le RFM, avec le client 0	51
7.5 Étape 5 — les mesures de contrôle	51
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C05 — REQUÊTES AVANCÉES : CTE, TABLEAU CROISÉ, UNNEST, QUALIFY, RÉCAPITULATIFS	56
1. Objectifs du chapitre	8

2. Pourquoi cette notion est importante	8
3. Explication simple — la recette, le tableau, l'étagère	57
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 La CTE : nommer les étapes, et réutiliser la bonne	58
5.2 Le tableau croisé : PIVOT et l'agrégat conditionnel	59
5.3 QUALIFY : classer et filtrer dans la même requête	59
5.4 UNNEST : inverser le grain	60
5.5 Les récapitulatifs : ROLLUP, CUBE, GROUPING SETS	60
5.6 La récursion : pour les hiérarchies, pas pour les calendriers	60
5.7 Ce que chaque moteur sait faire	61
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — le tableau que la direction avait demandé	61
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — le repliement mesuré	62
7.2 Étape 2 — la CTE réutilisée	62
7.3 Étape 3 — le tableau croisé	62
7.4 Étape 4 — QUALIFY avec et sans repliement	62
7.5 Étape 5 — UNNEST et le grain	62
7.6 Étape 6 — le récapitulatif	62
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C06 — LE RAPPORT LENT : VUES, INDEX, PLANS D'EXÉCUTION, SELECT *, PARTITIONS	66
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le classeur, le sommaire et le chariot	67
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 La vue : ce qu'elle relit, à chaque fois	68
5.2 SELECT * : le lire dans le plan, pas seulement au chronomètre	69
5.3 Lire un plan : trois étages, de bas en haut	69
5.4 L'index : mesuré, puis contrôlé dans un second moteur	69
5.5 Lisibilité contre performance : le cas de la fenêtre	70
5.6 Matérialiser ou non : le plancher d'un fichier	70
5.7 Les partitions	71
5.8 La note de méthode d'un rapport de performance	71
6. Exemple concret — la facture d'un rapport mal écrit	71

7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — le protocole d’abord	71
7.2 Étape 2 — la vue contre la table	71
7.3 Étape 3 — le plan du SELECT *	72
7.4 Étape 4 — l’index, mesuré puis contrôlé	72
7.5 Étape 5 — fenêtre contre sous-requête	72
7.6 Étape 6 — le plancher de matérialisation	72
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C07 — LA REQUÊTE COMME LIVRABLE : STYLE, CONVENTION, TESTS DE NON-RÉGRESSION	76
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — la fiche de recette et le contrôle qualité	77
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 L’en-tête de requête : la question avant le code	78
5.2 Nommer : les étapes, les alias, les colonnes	78
5.3 Un style qui se relit	79
5.4 Les trois tests de non-régression du module	79
5.5 Le contrôle entre deux sources : expliquer un écart, jamais l’additionner	80
5.6 Un test rejoue la requête à l’identique	80
5.7 Ce que ce chapitre ne couvre pas	38
6. Exemple concret — la revue d’un rapport avant envoi	81
7. Démonstration pas à pas — cinq étapes sur le socle	51
7.1 Étape 1 — écrire l’en-tête	81
7.2 Étape 2 — nommer les étapes	81
7.3 Étape 3 — rejouer les 18 contrôles	81
7.4 Étape 4 — comparer les deux socles	81
7.5 Étape 5 — documenter ce qui n’a pas été testé	81
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19

15. À retenir	19
16. Évaluation formative	20
PROJET M11.P — « LES 12 RAPPORTS SQL DE LA CELLULE COMMERCIALE »	86
1. Énoncé	86
La commande	86
Le matériel	86
Les six totaux de contrôle	86
2. Les 12 questions métier (le catalogue imposé)	87
Le format d'en-tête, imposé	87
3. Les 4 livrables	88
P1 — Les 12 requêtes (8 points)	88
P2 — Le jeu de tests (5 points)	88
P3 — La note de lecture (4 points · 2 pages)	89
P4 — La fiche de reprise (3 points)	89
4. Barème	89
5. Ce qui fait rejeter un livrable	90
6. Déroulé conseillé (4 séances, environ 8 h)	90
7. Comment vous corriger vous-même	90
ÉVALUATION M11 — « SQL AVANCÉ POUR LA BI »	91
A · Questions de récupération (non notées)	91
B · Quiz (20 questions · 20 points · 1 point par question)	91
Bloc 1 — Quelle requête pour quelle question (C01—C05) (Q1 à Q5)	91
Bloc 2 — Prédisez le résultat (Q6 à Q10)	92
Bloc 3 — Diagnostic (Q11 à Q15)	92
Bloc 4 — Vocabulaire (Q16 à Q20)	92
C · Exercices à rendre (20 points · 5 exercices, dont 2 de type E4)	93
E1 — Le top 3 par famille, deux fois (niveau 3 · 3 points)	93
E2 — Les deux périmètres de la concentration (niveau 3 · 3 points)	93
E3 — Le suivi des objectifs, trous compris (niveau 3 · 4 points)	93
E4 — « Ce rapport annonce un chiffre d'affaires net » (type E4 · niveau 4 · 5 points)	93
E5 — « Ce classement change de réponse à chaque exécution » (type E4 · niveau 4 · 5 points)	94
D · Auto-test SQL (10 points · 18 requêtes · auto-corrigé)	94
E · Étude de cas — « 10 % des clients font-ils 60 % du CA ? » (20 points · seuil 12)	94
La commande	86
Les quatre productions attendues	94
Le résultat attendu (à retrouver, pas à recopier)	95
Barème de l'étude de cas	95
Correction détaillée	95
A · Réponses de récupération	95
B · Corrigé du quiz (1 point par question)	95
C · Corrigé des exercices à rendre	96
D · Corrigé de l'auto-test	97
E · Corrigé de l'étude de cas — les quatre attendus	97
Barème global et seuils	97

Module M11.C01 — La fonction de fenêtre : `OVER`, `PARTITION BY`, `ORDER BY`, et pourquoi `GROUP BY` ne suffit plus

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5). **Durée indicative :** 5 h. **Niveau :** N3 → N4. **Pré-requis :** M07 (tout : `SELECT`, agrégats, `GROUP BY`, jointures, sous-requêtes) ; M06.C04 (DuckDB, SQLite, PostgreSQL) ; M02 (moyenne, médiane, dispersion).

L'idée du chapitre. M07 a appris à **réduire** : `GROUP BY` transforme des milliers de lignes en quelques totaux. C'est exactement ce qu'il faut quand la question est « combien ». Mais la question d'un responsable est presque toujours « **par rapport à quoi** » — quelle part du total, quel écart à la moyenne, où se situe cette ligne dans son groupe. Et là, `GROUP BY` a un défaut rédhibitoire : il **supprime la ligne** dont on parle. La fonction de fenêtre fait l'inverse : elle calcule le contexte **et laisse la ligne en place**. Ce chapitre pose le mécanisme (`OVER`, `PARTITION BY`, `ORDER BY`, cadre de fenêtre) et surtout la question préalable à toute requête de BI — **comparer à quoi ?** — avec un contre-exemple mesuré qui coûte cher : oublier `PARTITION BY` ne provoque aucune erreur, et divise la réponse par cinq.

Matériel de l'atelier — DuckDB 1.5.5 · Python 3.13 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). Le socle est un **script** de 3 595 octets : 5 petites tables (produits, magasins, vendeurs, objectifs, calendrier) et 2 vues sur les fichiers du socle M01-M03. Il se rejoue en mémoire en **0,4 s** et ne crée **aucune donnée nouvelle** : c'est le même fichier de ventes que M01 à M03 — **240 000** lignes, **237 191** ventes nettes (**2 809** retours), **44** mois, **5** magasins, **23 497** clients, **15 595 154 955** FCFA de chiffre d'affaires net. Toutes les sorties publiées dans ce chapitre sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Expliquer la différence d'usage** entre un agrégat (`GROUP BY`) et une **fonction de fenêtre** (window function) : l'un réduit la table, l'autre ajoute une colonne — et dire laquelle des deux répond à « quelle part du total ? ».
2. **Écrire et lire une fenêtre** : `OVER ( )`, `PARTITION BY`, `ORDER BY` à l'intérieur de la fenêtre, et un **cadre de fenêtre** (`ROWS BETWEEN ... AND ...`).
3. **Déterminer la base de comparaison** d'un chiffre — par magasin, par année, sur toute la table — et **prouver** votre choix en montrant que les parts somment à 100 % au niveau où elles doivent sommer.
4. **Reconnaître les trois pièges du chapitre** sur un résultat réel : la fenêtre évaluée après le `WHERE`, la jointure sur une clé incomplète, et la partition qui vaut le groupe (qui affiche toujours 100 %).
5. **Choisir entre une fenêtre et une sous-requête scalaire** pour ramener un « contexte » dans une requête agrégée, et justifier le choix autrement que par le goût.

2. Pourquoi cette notion est importante

Un responsable de magasin ne demande pas « quel est le chiffre d'affaires de la société ? ». Il demande « **quelle est ma part ?** ». Un responsable de rayon demande « **ce mois est-il**

bon ? », ce qui suppose un mois moyen à côté. Un directeur commercial demande « **qui sont mes dix meilleurs vendeurs ?** », ce qui suppose un classement **dans** un groupe. Ces trois questions ont un point commun : elles comparent une **ligne** à un **ensemble de lignes**. C’est le métier de la fenêtre.

Le socle du module rend l’enjeu immédiatement visible. Prenez les 5 magasins : un simple `GROUP BY` donne leurs totaux, et rien de plus. Pour obtenir leur **part**, il faut le total général — et le réflexe de M07 est d’écrire une sous-requête scalaire qui le ramène :

```
SELECT m.nom, SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc)
             / (SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour), 2) AS part_pct
FROM ventes v JOIN magasin m USING (id_magasin)
WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca DESC;
```

Cette requête est **juste**, et c’est celle que M07 vous aurait fait écrire. Elle a deux défauts, tous les deux mesurables : elle réclame **le total de la table** à chaque fois qu’on veut un pourcentage (ici une seconde lecture complète des 237 191 lignes), et elle devient illisible dès qu’on veut **deux** contextes — la part de la société et la part du magasin dans son propre réseau. Le même résultat s’écrit avec une fenêtre, et l’atelier a chronométré les deux écritures : **150 ms** pour la fenêtre, **292 ms** pour la sous-requête scalaire, soit **×1,9** pour la même sortie, ligne pour ligne.

Mais la vraie raison d’apprendre la fenêtre n’est pas la vitesse — les deux requêtes donnent exactement le même tableau. C’est que la fenêtre rend **visible** la question « comparer à quoi ? », parce qu’on doit l’écrire noir sur blanc : `OVER ()` dit toute la table, `OVER (PARTITION BY id_magasin)` dit dans son magasin, `OVER (PARTITION BY annee)` dit dans son année. La sous-requête scalaire cache ce choix dans une sous-requête qu’on ne relit pas. Et un choix caché, en BI, c’est un chiffre faux qui a l’air juste.

Le chiffre qui ouvre ce chapitre. Sur ce socle, le plus gros client identifié de la base n’est pas un client : c’est un identifiant **0** qui totalise **2 852 612 447** FCFA et **18,3 %** du chiffre d’affaires net du réseau — tout en étant **absent** du référentiel clients. Celui qui écrit une fenêtre de classement sans le savoir met en tête de son rapport commercial un client « non identifié » : la fenêtre ne protège de rien, elle exécute ce qu’on lui demande. C’est la leçon que le module déroulera jusqu’à l’étude de cas.

3. Explication simple — le tableau d’affichage du stade

Imaginez un match. Chaque spectateur (une ligne de la table) a un numéro de place, un prix payé, une tribune. À un moment, l’écran géant affiche trois informations : **le total du stade, le total de votre tribune, le nombre de spectateurs de votre tribune**.

- `GROUP BY`, c’est la **feuille de caisse** : on plie la foule en quelques tas, et les individus disparaissent. On sait qu’il y a 5 tas, on ne sait plus qui est dans lequel.
- La **fenêtre**, c’est l’**écran du stade** : chaque spectateur reste assis à sa place et **voit** l’information de son groupe. Le nombre de places ne change pas : 237 191 personnes restent 237 191 personnes, on a simplement ajouté trois colonnes à leur billet.

Trois conséquences, qui sont tout le chapitre :

1. **Le nombre de lignes ne change pas.** Si votre requête avec fenêtre rend 5 lignes là où vous attendiez 237 191, vous avez écrit un `GROUP BY`, pas une fenêtre.
2. **Ce que voit chaque ligne dépend de la fenêtre.** `OVER ()` = tout le stade ; `OVER (PARTITION BY tribune)` = votre tribune. Changez la fenêtre, changez le chiffre — sans changer une seule donnée.

3. **L'ordre compte dès qu'on demande un cumul.** Un cumul (« total depuis l'ouverture ») n'existe que **dans un ordre** : il faut dire lequel (`ORDER BY`), sinon le moteur choisit, et vous ne saurez pas quoi.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
fonction de fenêtre	window function	Un calcul qui regarde un ensemble de lignes liées à la ligne courante sans réduire le nombre de lignes du résultat.
fenêtre	window	L'ensemble de lignes que la fonction voit : décrit par <code>OVER (...)</code> .
partition	partition	La découpe de la fenêtre en groupes indépendants (<code>PARTITION BY id_magasin</code>) : le facteur de comparaison .
ordre de fenêtre	window order	L'ordre à l'intérieur de la partition (<code>ORDER BY date_vente</code>) : il rend possibles cumuls, rangs et décalages.
cadre de fenêtre	window frame	Les lignes effectivement prises en compte autour de la ligne courante (<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>).
part relative	share of total	Le rapport d'une ligne au total de sa fenêtre ; la somme des parts vaut 100 % au niveau de la partition .
sous-requête scalaire	scalar subquery	Une sous-requête qui rend une seule valeur , utilisée comme un nombre dans une expression — l'ancêtre de la fenêtre.
ordre d'exécution	logical order of execution	L'ordre dans lequel le moteur interprète les clauses (<code>FROM</code> → <code>WHERE</code> → <code>GROUP BY</code> → fenêtres → <code>SELECT</code> → <code>ORDER BY</code>).
grain	grain	Ce que représente une ligne du résultat : le vérifier avant de joindre, de fenêtrer et de conclure.
total de contrôle	control total	Le chiffre qu'un rapport doit retrouver coûte que coûte (ici 15 595 154 955 FCFA) et qui démasque toute multiplication de lignes.

Définition. Une **fonction de fenêtre** — window function — est un calcul qui prend en entrée un **ensemble de lignes** (la fenêtre) et rend **une valeur par ligne**. Elle s'écrit toujours après un nom de fonction, avec le mot-clé `OVER` : `SUM(x) OVER (...)`, `AVG(x) OVER (...)`, `COUNT(*) OVER ()`. Sans `OVER`, la même fonction est un agrégat ordinaire et réduit la table.

Définition. La **partition** — partition — est la subdivision de la fenêtre en groupes **indépendants les uns des autres**. Deux partitions ne se voient pas : un `SUM` de fenêtre partitionné par magasin ne connaît jamais le total de la société. C'est exactement ce qu'on veut quand la bonne question est « quelle part de son marché ? ».

5. Cours approfondi

5.1 L'ordre d'exécution : pourquoi une fenêtre n'est pas un agrégat

Le moteur lit une requête dans un ordre qui n'est pas celui de l'écriture :

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → FENÊTRES → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`

Deux conséquences se paient comptant, et elles sont mesurées au §5.7 :

1. **Une fenêtre s'évalue après `GROUP BY`.** Dans une requête agrégée, elle ne peut donc partitionner que sur ce qui **existe** à cet étage : une colonne de regroupement ou une expression agrégée. Écrire `SUM(SUM(x)) OVER (PARTITION BY colonne_non_groupee)` fait échouer la requête — et le message d'erreur de DuckDB dit précisément pourquoi (voir §8, erreur n°1).

2. **Une fenêtre s'évalue après WHERE.** Filtrer une ligne avant de calculer une part ne laisse plus qu'une ligne dans la fenêtre : la part devient **100,0 %**. Mesuré au §7.5 sur janvier 2023 : **8,3 %** quand le filtre est appliqué **après** le calcul, **100,0 %** quand il est appliqué avant.

5.2 OVER () : la fenêtre sur toute la table

La fenêtre vide dit « **toutes les lignes du résultat** ». C'est la forme qui ramène le total général à côté de chaque ligne :

```
SELECT m.nom AS magasin,
       SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc) / SUM(SUM(v.montant_ttc)) OVER (), 2) AS part_pct
FROM ventes v JOIN magasin m USING (id_magasin)
WHERE NOT v.est_retour
GROUP BY 1 ORDER BY ca DESC;
```

La double écriture `SUM(SUM(v.montant_ttc)) OVER ()` surprend la première fois : **le SUM intérieur est l'agrégat du GROUP BY** (il fabrique le total du magasin), **le SUM extérieur est la fenêtre** (il additionne ces totaux, donc toute la table). On lit la requête de l'intérieur vers l'extérieur : regrouper par magasin, puis comparer chaque total au total des totaux.

Dans les faits. L'atelier a exécuté les deux écritures sur le socle et comparé les sorties ligne à ligne : **identiques**. Seul le chronomètre les distingue — **150 ms** pour la fenêtre, **292 ms** pour la sous-requête scalaire. La fenêtre ne « calcule pas mieux » : elle évite de redemander le total à chaque ligne. Et elle a un avantage décisif en relecture : le total y est **visible** dans le texte de la requête.

5.3 PARTITION BY : le facteur de comparaison

`PARTITION BY` découpe la fenêtre. C'est **le** paramètre du chapitre, celui qui décide du sens du chiffre. La même mesure, trois partitions, trois lectures :

Ce qu'on écrit	La question à laquelle on répond	Ce que la somme des parts vaut
<code>OVER ()</code>	quelle part de toute la société ?	100 % sur l'ensemble du résultat
<code>OVER (PARTITION BY id_magasin)</code>	quelle part de son magasin ?	100 % dans chaque magasin
<code>OVER (PARTITION BY annee)</code>	quelle part de son année ?	100 % dans chaque année

Le tableau ci-dessus est aussi la **procédure de vérification** : après avoir écrit une part, on l'additionne au niveau de sa partition. Si l'on n'obtient pas 100 %, la partition ne décrit pas ce qu'on raconte.

À retenir. Dans une requête où le `GROUP BY` et le `PARTITION BY` portent sur la même colonne, la part vaut **toujours 100,0 %** : chaque groupe compare son total à lui-même. C'est le résultat le plus vide qui existe, il a l'air parfaitement crédible sur un rapport, et c'est l'erreur n°3 du §8.

5.4 ORDER BY dans la fenêtre : l'ordre change la valeur

Sans `ORDER BY`, la fenêtre est « toutes les lignes de la partition » : `SUM(...) OVER (PARTITION BY m)` donne le total du magasin sur **chaque** ligne du magasin. Avec `ORDER BY` **dans** la fenêtre, la même fonction devient **cumulative** : elle additionne de la première ligne jusqu'à la ligne courante. Attention, cet `ORDER BY` **n'est pas** celui du résultat : c'est lui qui définit la progression du calcul. Une requête peut très bien cumuler par date (`ORDER BY date_vente` dans la fenêtre) et **rendre** les lignes triées par montant décroissant (`ORDER BY ca DESC` à la fin).

Définition. Le **cadre de fenêtre** — window frame — précise **quelles lignes** autour de la ligne courante la fonction voit : `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` (les deux précédentes et la courante), `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (depuis le début), `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` (toute la partition). Dès qu'on écrit `ORDER BY` dans une fenêtre, le cadre par défaut devient « du début à la ligne courante » : c'est lui qui transforme un `SUM` en cumul.

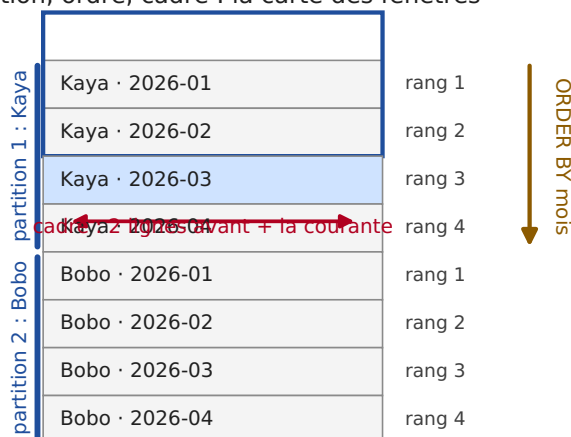
5.5 Le cadre : cumul et moyenne glissante

Sur les cinq premières ventes du magasin 1, l'atelier a publié le cumul et la moyenne glissante de trois lignes. Les valeurs du cumul sont **53 690 ; 55 729 ; 76 969 ; 134 081 ; 171 605** FCFA : chaque ligne ajoute son montant à la précédente. La moyenne glissante de trois lignes, elle, vaut **53 690 ; 27 865 ; 25 656 ; 26 797 ; 38 625** FCFA : elle ne monte pas, elle **suit** — la première ligne n'a qu'elle-même, la deuxième en a deux, les suivantes en ont trois.

C'est toute la différence entre un indicateur de **niveau** (le cumul, qui dépend du passé entier) et un indicateur de **tendance** (la moyenne glissante, qui oublie). Les deux se calculent avec la même fonction `SUM/AVG` : seul le **cadre** change.

Attention. Le cumul dépend de l'`ORDER BY` de la fenêtre, donc du **départage** entre lignes de même date. Le socle contient des milliers de ventes par jour : `ORDER BY date_vente` seul laisse le moteur choisir l'ordre des lignes d'une même journée, et le cumul de midi n'est plus celui du matin. Écrivez `ORDER BY date_vente, id_vente` — la clé de départage, ici l'identifiant unique de la ligne.

Partition, ordre, cadre : la carte des fenetres



La partition separe les groupes de lignes.

Le cadre decide ce qui entre dans le calcul.

5.6 La même question, deux écritures : fenêtre contre sous-requête scalaire

Définition. Une **sous-requête scalaire** — scalar subquery — est une sous-requête placée dans une expression et qui rend **une seule valeur** (une ligne, une colonne) : `SUM(x) / (SELECT SUM(y) FROM t)`. C'est la façon dont on écrivait un contexte avant les fenêtres. Elle reste légitime pour une constante — un seuil, un taux de référence — et devient illisible dès qu'il faut deux contextes différents.

Critère	Fonction de fenêtre	Sous-requête scalaire
Sortie	identique	identique
Coût mesuré ici	150 ms	292 ms
Le choix de comparaison est-il visible ?	oui, dans <code>OVER (...)</code>	non, enfoui dans la sous-requête
Deux contextes dans la même requête ?	oui (<code>OVER ()</code>) et <code>OVER (PARTITION BY ...)</code>	oui, mais deux sous-requêtes
S'écrit sur une requête agrégée ?	oui (<code>SUM(SUM(x)) OVER (...)</code>)	oui (<code>SUM(x) / (SELECT ...)</code>)
S'ajoute à une table non agrégée ?	oui, directement	oui, mais une sous-requête par contexte

La règle de travail du module : **dès qu'un chiffre exprime une comparaison, la fenêtre est la forme attendue** — parce que la relecture doit pouvoir vérifier le facteur de comparaison sans exécuter la requête.

Attention. La vitesse ne justifie pas un choix de syntaxe. Les **150 ms** contre **292 ms** de l'atelier dépendent de la machine, de la version du moteur et de la taille du socle : sur dix lignes, la sous-requête scalaire sera plus rapide. Écrivez la fenêtre parce que le **facteur de comparaison y est explicite** — c'est un argument de relecture, pas un argument de performance. Le seul argument de performance du module qui survive à un changement de machine est celui du chapitre C06, et il est mesuré sur un plan d'exécution.

5.7 Les trois pièges, mesurés

1. **La fenêtre s'évalue après le WHERE** : janvier 2023 vaut **8,3 %** de son année quand la fenêtre est calculée puis filtrée ; **100,0 %** quand filtre et fenêtre sont dans le même `SELECT`. Le résultat « 100,0 % » est faux et n'émet aucun avertissement.
2. **La jointure sur une clé incomplète** : joindre les ventes à une table de totaux **mensuels par magasin** en oubliant `id_magasin` dans la condition multiplie chaque ligne par le nombre de magasins du mois. Le rapport annonce alors **77 975 774 775 FCFA** sur **1 185 955** lignes — soit cinq fois le total du socle, et le contrôle des 5 magasins « fonctionne » toujours.
3. **La partition qui vaut le groupe** : `GROUP BY YEAR(date_vente)` avec `OVER (PARTITION BY YEAR(date_vente))` affiche 100,0 % pour chaque année. La requête est valide, la réponse est vide.

Conseil professionnel. Terminez chaque rapport de comparaison par un **total de contrôle** : un `SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour` qui doit rendre **15 595 154 955 FCFA**. C'est cinq secondes d'écriture, et c'est la seule ligne qui attrape les pièges 2 et 3 — les deux seuls du chapitre qui produisent un nombre **plus grand** ou **trop rond** sans jamais lever d'erreur.

5.8 Ce que ce chapitre ne couvre pas

Les **rang**s (`ROW_NUMBER`, `RANK`, `NTILE`), les **décalages** (`LAG`, `LEAD`) et les **variations** d'une période à l'autre sont le sujet de C02 : ce sont les mêmes fenêtres, appliquées à l'ordonnancement. Le traitement du **temps** (mois sans vente à fabriquer, comparaison inter-annuelle, moyennes mobiles de 12 mois) est celui de C03. Ici, on ne fait que comparer une ligne à un ensemble — et on apprend à dire lequel.

6. Exemple concret — la part de chaque magasin, et ce qu'elle décide

Le directeur commercial demande le poids de chaque magasin. La requête de référence rend **5 lignes** :

Magasin	CA net (FCFA)	Part du réseau
Sahel Distribution — Ouaga 2000	5 312 205 459	34,06 %
Sahel Distribution — Gounghin	3 424 978 244	21,96 %
Sahel Distribution — Bobo Kibidwé	3 099 946 956	19,88 %
Sahel Distribution — Koudougou Centre	2 024 794 213	12,98 %
Sahel Distribution — Kaya Marché	1 733 230 083	11,11 %
Total	15 595 154 955	100,00 %

Trois lectures que ce tableau autorise — et une qu'il interdit :

- **Le premier magasin fait plus du tiers du réseau** (34,06 %). C'est un fait de structure, pas de performance : il est plus grand (Ouaga 2000), il vend plus, et sa part est stable.
- **Le rapport entre le premier et le dernier vaut 3,06**. Toute action commerciale « pour égaliser » les magasins se heurte donc à un écart de taille, pas à un écart de talent — la surface, le bassin de population et l'assortiment ne sont pas dans la table.
- **La somme des parts vaut 100,00 %** : la partition est bien le réseau entier. C'est la preuve exigée au §5.3, et elle tient en une addition.

La lecture **interdite** est celle qu'un rapport pressé écrit tout de suite : « Ouaga 2000 porte le réseau ». Faux : c'est le magasin le plus gros d'un réseau de cinq, et rien dans ces cinq lignes ne dit qu'il tire les autres. Une part n'est pas une contribution au résultat — question qui exigera la marge (C06 du module M13 et les coûts d'achat du socle).

7. Démonstration pas à pas — six étapes sur le socle

Toutes les commandes de cette section ont été exécutées dans l'atelier. Le socle se reconstruit d'abord :

```
import sys; sys.path.insert(0, "03_exercices/dossier_M11")
from connexion import ouvrir
con = ouvrir()                                # 0,4 s : le script du socle est rejoué en mémoire
```

7.1 Étape 1 — le total de contrôle

```
SELECT COUNT(*), COUNT(*) FILTER (WHERE NOT est_retour), SUM(montant_ttc) FILTER (WHERE NOT
est_retour)
FROM ventes;
```

```
240000 | 237191 | 15595154955
```

C'est la ligne à retrouver à la fin de **chaque** rapport du module.

7.2 Étape 2 — la part, avec la fenêtre

```
SELECT m.nom AS magasin, SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc) / SUM(SUM(v.montant_ttc)) OVER (), 2) AS part_pct
FROM ventes v JOIN magasin m USING (id_magasin)
WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca DESC;
```

Sahel Distribution — Ouaga 2000		5312205459		34.06
Sahel Distribution — Gounghin		3424978244		21.96
Sahel Distribution — Bobo Kibidwé		3099946956		19.88
Sahel Distribution — Koudougou Centre		2024794213		12.98
Sahel Distribution — Kaya Marché		1733230083		11.11

7.3 Étape 3 — la même réponse par sous-requête scalaire

```
SELECT m.nom, SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc) / (SELECT SUM(montant_ttc) FROM ventes WHERE NOT
est_retour), 2)
FROM ventes v JOIN magasin m USING (id_magasin) WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca
DESC;
```

Sortie **identique**, colonne par colonne. La comparaison des deux écritures porte donc sur la lisibilité et le coût (**150 ms** contre **292 ms**), pas sur la justesse.

7.4 Étape 4 — une part calculée dans son année

```
SELECT strftime(date_trunc('month', date_vente), '%Y-%m') AS mois, YEAR(date_vente) AS annee,
       ROUND(SUM(montant_ttc)) AS ca,
       ROUND(100.0 * SUM(montant_ttc) / SUM(SUM(montant_ttc)) OVER (PARTITION BY annee), 1) AS
part_annee_pct
FROM ventes WHERE NOT est_retour GROUP BY 1, 2 ORDER BY 1;
```

2023-01		2023		284400304		8.3
2023-02		2023		263183276		7.6
...						
2023-12		2023		333819159		9.7

44 lignes, une par mois, chacune portant la part de son mois **dans son année**. Janvier 2023 pèse **8,3 %** de 2023.

7.5 Étape 5 — le piège : le filtre et la fenêtre dans le même SELECT

```
WITH m AS (SELECT strftime(date_trunc('month', date_vente), '%Y-%m') am, YEAR(date_vente) an,
                  SUM(montant_ttc) ca FROM ventes WHERE NOT est_retour GROUP BY 1, 2)
SELECT ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY an), 1) FROM m WHERE am = '2023-01';
```

100.0

Le **WHERE** s'exécute **avant** la fenêtre : au moment du calcul, la partition ne contient plus qu'une seule ligne, celle qu'on vient de garder. La bonne écriture filtre **après**, dans une requête englobante :

```
WITH m AS (...)
SELECT part FROM (SELECT am, ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY an), 1) part FROM m)
WHERE am = '2023-01';
```

8.3

Deux requêtes, deux réponses, un seul mot d'écart : l'emplacement du filtre. **8,3 %** est la part du mois dans son année ; **100,0 %** est un artefact.

7.6 Étape 6 — la fenêtre n'a pas réduit la table

```
SELECT id_vente, id_magasin, ROUND(montant_ttc) AS montant,
       COUNT(*) OVER () AS lignes_du_socle, ROUND(AVG(montant_ttc) OVER ()) AS moyenne_generale
FROM ventes WHERE NOT est_retour LIMIT 5;
```

47820	1	53690	237191	65749
92080	1	2039	237191	65749
92628	1	21240	237191	65749
99471	1	57112	237191	65749
119868	1	37524	237191	65749

Chaque ligne a **conservé** son identifiant, son magasin et son montant, et **reçu en plus** deux informations de contexte : **237 191** lignes au total et **65 749** FCFA de montant moyen par ligne. Une requête `GROUP BY` ne peut pas publier cela : elle aurait rendu **1** ligne.

8. Erreurs fréquentes

1. **Partitionner sur une colonne qui n'est plus là après `GROUP BY`.** Le moteur refuse : `column "date_vente" must appear in the GROUP BY clause or must be part of an aggregate function`. Ce n'est pas une brimade : à l'étage de la fenêtre, la colonne a déjà été remplacée par un groupe. La correction est d'ajouter l'**expression** au `GROUP BY` (ici `YEAR(date_vente) AS annee`) puis de partitionner sur l'alias. Retenez la règle : **une fenêtre ne partitionne que sur ce qui existe à son étage**.
2. **Filtrer avant de fenêtrer.** Le `WHERE` étant évalué avant les fenêtres, filtrer un mois puis calculer sa part sur l'année laisse une seule ligne dans la partition : **100,0 %** au lieu de **8,3 %**. Toute comparaison « cette ligne par rapport à son ensemble » doit filtrer **après** le calcul — donc dans une requête englobante.
3. **Partitionner sur sa propre clé de regroupement.** `GROUP BY annee + PARTITION BY annee` rend 100 % partout. Le chiffre est exact, la conclusion est nulle : si votre rapport de parts ne contient que des 100 %, la partition n'a pas été choisie, elle a été recopiée.
4. **Joindre une table de totaux sans vérifier son grain.** La table des totaux mensuels par magasin contient **220** lignes (une par couple magasin-mois). Jointe sur le mois **seul**, elle apporte **5** lignes de totaux à chaque vente, et le rapport publie **77 975 774 775** FCFA au lieu de **15 595 154 955** : **5** fois trop, **1 185 955** lignes au lieu de **237 191**, et aucun message d'erreur. Le grain de la table jointe est **la** chose à vérifier avant la jointure.
5. **Croire qu'un cumul est un indicateur de niveau.** Un cumul dépend de la date de début, de l'ordre choisi et du départage des ex æquo : deux analystes honnêtes peuvent en publier deux valeurs différentes du même socle. Préciser `ORDER BY` **et** la clé de départage, sinon le cumul n'est pas reproductible.

9. Bonnes pratiques professionnelles

1. **Écrire les fenêtres dans cet ordre : le grain, la partition, l'ordre, le cadre.** Une requête de fenêtre se relit toujours dans le même sens : quelle ligne par ligne ? puis comparée à quoi ? puis dans quel ordre ? puis sur quelle profondeur ?. Si l'une des quatre réponses manque, la requête est incomplète même si elle tourne.

2. **Appliquer la preuve de la partition.** Après toute part relative, contrôler que la somme vaut 100 % au niveau voulu : sur tout le résultat (`OVER ()`), par magasin, par année. Sans ce contrôle, l'erreur n°3 passe en production.
3. **Choisir la fenêtre pour ce qui se compare, la sous-requête pour ce qui s'isole.** Une constante (le taux de TVA de référence, un seuil) se lit par sous-requête ou par paramètre ; un **contexte changeant** (le total d'un groupe, la moyenne d'une famille) s'écrit en fenêtre, parce que le contexte doit être **vu**.
4. **Toujours nommer les colonnes de contexte.** `COUNT(*) OVER ()` se nomme `lignes_du_socle`, pas `c1`. Un rapport de BI est lu six mois plus tard par quelqu'un qui n'a pas la requête sous les yeux.
5. **Terminer par le total de contrôle. 15 595 154 955 FCFA.** Un écart de ce total signale une jointure multiplicatrice (erreur n°4) avant que la conclusion ne soit écrite.

10. Exercice guidé

Situation. Vous produisez le rapport « poids des vendeurs » pour le comité de direction. Le socle contient **22** vendeurs. Votre stagiaire a publié ce tableau, qui a beaucoup plu :

Vendeur	Part du CA	Rang
Vendeur 7	8,68 %	1
Vendeur 21	8,44 %	2
Vendeur 12	7,89 %	3

Consigne. Dans les 20 minutes (chrono), sans ouvrir `ATTENDU.json` :

1. Écrivez la requête qui produit exactement ces trois colonnes, avec la part en fenêtre. (3 pts)
2. Ajoutez la **preuve** que la partition choisie est la bonne : une colonne ou une requête qui montre à quoi les parts additionnent. (3 pts)
3. Le stagiaire jure avoir testé : « la somme des colonnes fait 100 % ». Expliquez en deux lignes pourquoi son test ne prouve rien ici. (2 pts)
4. Le comité demande maintenant la part **par magasin** pour chaque vendeur (un vendeur appartient à un magasin). Modifiez la fenêtre. (2 pts)
5. Écrivez le **total de contrôle** que vous joindrez au rapport, et dites ce qu'un écart révélerait. (2 pts)

11. Exercices autonomes

E1 — Le mois dans son année, dans son magasin, dans son réseau (35 min). Sur le socle, produisez une seule requête rendant pour chaque couple **magasin-mois** (220 lignes) : le CA, sa part dans l'année du magasin, sa part dans le réseau pour ce mois. Les trois colonnes doivent sommer à 100 % **au niveau de leur partition** — joignez à votre rendu le contrôle qui le prouve. Contrainte : aucune sous-requête scalaire, uniquement des fenêtres.

E2 — Le contrôle de grain (25 min). Construisez la table des totaux mensuels par magasin, puis écrivez **deux** jointures : l'une sur le mois seul, l'autre sur les deux clés. Publiez les deux totaux, expliquez l'écart par le nombre de lignes, et déduisez-en la phrase que vous diriez à un collègue pour l'aider à détecter ce piège dans son propre rapport.

12. Correction détaillée

Exercice guidé.

1. **La requête.** Le classement des vendeurs se calcule sur les ventes nettes, groupées par vendeur :

```
sql SELECT v.id_vendeur, SUM(v.montant_ttc) AS ca, ROUND(100.0 * SUM(v.montant_ttc) /
SUM(SUM(v.montant_ttc)) OVER (), 2) AS part_pct FROM ventes v WHERE NOT v.est_retour GROUP BY 1
ORDER BY ca DESC LIMIT 3;
```

La fenêtre est `OVER ()` : la partition est le **réseau entier**, ce qui correspond à la question posée (« poids de chaque vendeur dans la société »). 2. **La preuve.** Une part ne se valide pas sur son maximum, elle se valide sur sa **somme** au niveau de la partition :

```
sql WITH p AS (SELECT id_vendeur, SUM(montant_ttc) ca FROM ventes WHERE NOT est_retour GROUP BY
1) SELECT ROUND(SUM(100.0 * ca / SUM(ca) OVER ()), 2) FROM p; -- 100.00
```

Vingt-deux lignes, une somme, un verdict. 3. **Pourquoi le test du stagiaire ne prouve rien.** Il a vérifié que **ses trois lignes publiées** totalisent moins de 100 % — ou pire, il a additionné les parts de tout le monde sans le dire. Un contrôle de part se fait **sur la partition complète**, jamais sur l'extrait qu'on publie : un classement tronqué au top 3 somme par construction à 24,99 %, et cela ne dit rien de la justesse de la partition. 4. **La part dans le magasin.** On ajoute `id_magasin` au `GROUP BY` (il faut qu'il existe à l'étage de la fenêtre), puis on partitionne dessus :

```
sql SELECT id_magasin, id_vendeur, SUM(montant_ttc) AS ca, ROUND(100.0 * SUM(montant_ttc) /
SUM(SUM(montant_ttc)) OVER (PARTITION BY id_magasin), 2) FROM ventes WHERE NOT est_retour GROUP
BY 1, 2 ORDER BY 1, ca DESC;
```

Le même vendeur change de part **sans qu'aucune donnée n'ait changé** : c'est le sens du chapitre. 5. **Le total de contrôle.** `SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour;` doit rendre **15 595 154 955 FCFA**. Un écart signalerait une jointure multiplicatrice (chaque ligne comptée plusieurs fois) ou un filtre sur les retours oublié — les deux erreurs qui font un chiffre trop gros.

E1 (les trois parts emboîtées). La structure attendue : le `GROUP BY` fabrique les couples magasin-mois avec leur CA, puis **trois fenêtres** calculent trois contextes différents sur cette même table agrégée.

```
WITH m AS (SELECT id_magasin, YEAR(date_vente) an,
strftime(date_trunc('month', date_vente), '%Y-%m') am, SUM(montant_ttc) ca
FROM ventes WHERE NOT est_retour GROUP BY 1, 2, 3)
SELECT id_magasin, am, ca,
ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY id_magasin, an), 2) AS part_dans_l_annee,
ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY am), 2) AS part_du_mois_reseau
FROM m ORDER BY am, id_magasin;
```

Le contrôle : `SUM(part_dans_l_annee)` sur un magasin et une année vaut 100,00 (les 12 mois, ou 8 en 2026) ; `SUM(part_du_mois_reseau)` sur un mois vaut 100,00 (les 5 magasins). Le même CA, deux partitions, deux lectures — et un tableau qui ne somme à 100 % à aucun autre niveau.

E2 (le contrôle de grain). La table des totaux compte **220** lignes. Jointe sur le mois **seul**, chaque vente rencontre 5 lignes de totaux : le résultat passe à **1 185 955** lignes et le total annoncé à **77 975 774 775 FCFA**, cinq fois **15 595 154 955**. Jointe sur les deux clés, on retrouve **237 191** lignes et le total juste. La phrase à dire à un collègue : « avant de joindre une table de totaux, écris `COUNT(*)` **avant** et **après** la jointure : si le nombre de lignes augmente, ta clé de jointure est incomplète ».

13. Mini-projet de chapitre

La note de trois pages : « la part et l'écart ». Vous produisez, pour le comité de la quincaillerie, une note de 3 pages maximum portant sur le socle M11 (240 000 lignes, 44 mois, 5 magasins).

Livrables et barème.

Critère	Ce qui est vérifié	Points
La requête de part	une fenêtre <code>OVER ()</code> , avec le total de contrôle 15 595 154 955 FCFA	6
La part par magasin et dans l'année	deux partitions dans la même note, chacune avec sa preuve à 100 %	5
Le piège documenté	le filtre avant/après fenêtre, chiffré (8,3 % contre 100,0 %)	4
La phrase de conclusion	ce que la note autorise à dire, et ce qu'elle interdit de dire	3
Total		18

14. Résumé du chapitre

`GROUP BY` réduit une table à ses groupes ; une **fonction de fenêtre** ajoute à chaque ligne une information calculée sur un **ensemble** de lignes, sans en supprimer aucune. Cet ensemble se déclare : `OVER ()` pour toute la table, `OVER (PARTITION BY ...)` pour un groupe, avec un `ORDER BY` interne dès qu'il s'agit de cumuler, et un **cadre** (`ROWS BETWEEN ...`) pour décider de la profondeur du regard. Le facteur de comparaison est **le** contenu de la requête : `OVER ()` répond « part de la société », `PARTITION BY id_magasin` répond « part de son magasin », et la même mesure change de sens sans qu'une donnée change. Trois pièges, tous mesurés ici : le `WHERE` qui s'exécute **avant** la fenêtre (**100,0 %** au lieu de **8,3 %**), la jointure sur une clé incomplète (**77 975 774 775** FCFA au lieu de **15 595 154 955**), la partition recopiée du `GROUP BY` (100 % partout). Aucun des trois ne lève d'erreur — d'où la règle de la maison : **terminer chaque rapport par son total de contrôle**.

15. À retenir

- `GROUP BY` répond « combien », la fenêtre répond « par rapport à quoi ».** Si votre question contient « part », « écart », « rang » ou « cumul », il vous faut une fenêtre.
- Le nombre de lignes ne change pas.** Une requête à fenêtre sur 237 191 lignes rend 237 191 lignes ; si vous en obtenez 5, vous avez écrit un `GROUP BY`.
- La partition est une décision, pas une syntaxe.** Elle doit être écrite dans la requête, prouvée par une somme à 100 %, et relisible six mois plus tard.
- La fenêtre s'évalue après le `WHERE` et après le `GROUP BY`.** Filtrer avant de comparer fausse la comparaison ; partitionner sur une colonne non regroupée fait échouer la requête.
- Un rapport de comparaison sans total de contrôle est un pari.** Le total du socle est **15 595 154 955** FCFA, et il tient en une ligne de SQL.

À retenir. Une fonction de fenêtre ne change pas les données, elle change ce qu'on peut **dire** des données : elle rend une ligne comparable à un ensemble, explicitement.

À retenir. La question à se poser avant d'écrire `OVER ()` n'est jamais « quelle syntaxe ? » mais « **comparé à quoi ?** » — et la réponse doit figurer dans la requête, pas dans la tête de l'analyste.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Quelle est la différence, en une phrase, entre `SUM(x)` et `SUM(x) OVER ()` dans une requête **sans** `GROUP BY` ?
2. Combien de lignes rend `SELECT id_magasin, COUNT(*) OVER () FROM ventes WHERE NOT est_retour` ?
3. Dans `SUM(SUM(x)) OVER (PARTITION BY annee)`, que fait le `SUM` intérieur ? Que fait celui de la fenêtre ?
4. Vous lisez un rapport de parts où chaque ligne affiche 100,0 %. Quelle est la cause la plus probable ?
5. Vous voulez la part de janvier 2023 dans l'année 2023. Où placez-vous le filtre sur le mois, et pourquoi ?
6. Que vaut la somme des parts de tous les mois d'une année si la partition est `PARTITION BY annee` ?
7. Que se passe-t-il si vous joignez une table de totaux mensuels par magasin sur le mois seul ?
8. Un cumul est-il reproductible sans clé de départage ? Justifiez en trois mots.
9. Quelle est la forme la plus courte du total de contrôle du module ?
10. Pourquoi une part publiée sur un top 3 ne peut-elle pas être validée en additionnant ses trois lignes ?

Réponses. 1. `SUM(x)` sans `GROUP BY` réduit toute la table à une valeur ; `SUM(x) OVER ()` rend une valeur **par ligne**, identique partout. 2. **237 191** lignes — une par vente nette ; la fenêtre ne réduit pas. 3. Le `SUM` intérieur agrège le groupe (le mois) ; la fenêtre additionne ces totaux d'année. 4. Le `PARTITION BY` porte sur la même colonne que le `GROUP BY` : chaque groupe se compare à lui-même ; c'est l'erreur n°3. 5. **Après** le calcul de la part, dans une requête englobante — sinon la partition ne contient plus qu'une ligne et la part vaut 100,0 % au lieu de **8,3** %. 6. 100,00 %, et c'est la preuve attendue. 7. Chaque vente rencontre les totaux de tous les magasins du mois : **5** fois trop de lignes (**1 185 955** au lieu de **237 191**) et **77 975 774 775** FCFA au lieu de **15 595 154 955**. 8. Non : sans clé de départage, l'ordre des ex æquo est choisi par le moteur. 9. `SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour` → **15 595 154 955** FCFA. 10. Parce qu'un classement tronqué ne somme à 100 % à aucun niveau : la preuve se fait sur la **partition complète**, pas sur l'extrait publié.

Module M11.C02 — Classer, cumuler, décaler :

`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `SUM() OVER`, `LAG`, `LEAD`

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5). **Durée indicative :** 5 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 (fenêtres, `OVER`, `PARTITION BY`, cadre) ; M07.C03 (`ORDER BY`, `LIMIT`), M07.C05 (`GROUP BY`) ; M02 (moyenne, médiane).

L'idée du chapitre. C01 a montré comment **comparer** une ligne à un ensemble. C02 en fait un usage qui produit des chiffres faux à une fréquence inquiétante : **ranger**. Un classement est une affirmation — « ce vendeur est le premier », « ce client fait partie des 10 % les plus fidèles » — et toute affirmation de rang se paie d'une décision : celle du **départage**. Le socle du module rend le problème visible : parmi les 23 497 clients, il n'existe que **21** nombres de tickets différents, et **3 316** clients ont exactement le même (**8** tickets). Selon la fonction employée, le même socle place le dernier client au rang **23 444** ou au rang **21** — deux nombres justes, un seul interprétable. Et le classement peut contredire le total : les 5 magasins du réseau sont rangés **exactement à l'envers** selon qu'on les classe par chiffre d'affaires ou par panier moyen.

Matériel de l'atelier — DuckDB 1.5.5 · Python 3.13 · socle M11 (03_exercices/dossier_M11/socle_m11.sql, rejoué en 0,4 s). Rappel du socle : **240 000** lignes, **237 191** ventes nettes (**2 809** retours), **44** mois, **5** magasins, **145 212** tickets, **23 497** clients, **15 595 154 955** FCFA de chiffre d'affaires net. Toutes les sorties de ce chapitre sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Choisir entre `ROW_NUMBER`, `RANK` et `DENSE_RANK`** sur un cas réel comportant des ex æquo, et **expliquer** ce que chacun affirme — numéroter, classer, ou compter les niveaux.
2. **Départager un classement** : reconnaître qu'une mesure comporte des égalités massives et écrire la clé qui rend le résultat reproductible.
3. **Découper en groupes de taille égale** avec `NTILE`, et savoir pourquoi un découpage en tiers n'est pas un classement.
4. **Extraire un top N par groupe** (les 3 premiers produits de chaque famille) et **filtrer après** le calcul du rang, jamais avant.
5. **Calculer un cumul et une variation** avec `SUM() OVER (ORDER BY ...)` et `LAG/LEAD`, en traitant explicitement le premier trou du décalage.
6. **Lire un classement comme une affirmation discutable** : vérifier qu'il répond bien à la question posée, et pas à une autre.

2. Pourquoi cette notion est importante

Trois des demandes les plus fréquentes d'un responsable se formulent en rang :

- « sortez-moi le **top 20** des produits » : un classement tronqué ;
- « le chiffre **cumulé** depuis janvier » : une somme en mouvement ;
- « la **progression** par rapport au mois dernier » : un décalage.

Chacune se trompe d’une façon différente, et aucune ne produit de message d’erreur. Un top tronqué au mauvais rang publie cinq lignes pour un cinquième rang : les 5 clients à **20** tickets du socle occupent les rangs 2, 3, 4, 5 et 6 en numérotation, mais **tous le rang 2** en classement — un rapport qui annonce « le 5e client » raconte une hiérarchie qui n’existe pas. Un cumul sans ordre explicite change de valeur selon l’heure de la journée. Une variation calculée sur le premier mois d’une série compare à un vide et publie `NULL`, que la plupart des tableurs affichent comme une cellule vide — donc comme un zéro.

Le chiffre qui structure ce chapitre. Le socle contient **23 497** clients et seulement **21** valeurs distinctes de « nombre de tickets ». La mesure est donc **massivement** faite d’égalités : **3 316** clients partagent 8 tickets. Sur cette mesure, `RANK` descend jusqu’au rang **23 444** tandis que `DENSE_RANK` s’arrête à **21**. Les deux requêtes sont exactes ; l’une des deux n’a aucun intérêt pour un décideur, et il faut savoir laquelle on publie avant que le comité de direction ne compare deux rapports incompatibles.

3. Explication simple — le podium et les dossards

À l’arrivée d’une course, trois documents différents peuvent être imprimés, et ils ne disent pas la même chose :

- **`ROW_NUMBER` = les dossards de classement.** Chaque coureur reçoit un numéro **unique**, de 1 à n, dans l’ordre d’arrivée. S’il y a égalité, on tranche arbitrairement (un tirage) ou par une règle explicite (« à égalité, celui dont l’identifiant est le plus petit »). Personne ne partage un dossard.
- **`RANK` = le podium.** Deux coureurs à égalité reçoivent la **même** place, et les places suivantes sont **sautées**. Trois arrivants ex æquo en 2e position : ils sont tous 2e, et le suivant est 5e. C’est la règle des médailles olympiques.
- **`DENSE_RANK` = le nombre de niveaux.** Deux coureurs à égalité partagent la 2e place, et le suivant prend la **3e** : on ne compte que les niveaux de performance réellement distincts.

Et **`NTILE(n)` = la photo de famille.** On coupe la troupe en n groupes de taille aussi égale que possible. Ce n’est pas un classement : deux coureurs du même groupe peuvent avoir des performances très différentes, et surtout, à la frontière, deux ex æquo peuvent tomber **de part et d’autre** de la ligne de coupe. C’est ce que le chapitre mesure sur le socle : un tiers comptant **2 718, 2 720, 2 728** ou **2 738** clients selon l’exécution — le même socle, le même code, quatre chiffres.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
numérotation	row numbering	Attribuer un numéro unique à chaque ligne d’une partition (<code>ROW_NUMBER</code>) : aucune égalité possible.
classement égalité	à rank	Attribuer la même place aux lignes de même valeur, en sautant les places suivantes (<code>RANK</code>).
rang dense	dense rank	Attribuer la même place aux ex æquo sans sauter de place (<code>DENSE_RANK</code>) : il compte les niveaux .
ex æquo	tie	Deux lignes de valeur identique sur la mesure classée : le cas qui décide du choix de la fonction.
clé de départage	tie-breaker	La colonne ajoutée à l’ <code>ORDER BY</code> d’une fenêtre pour rendre un classement reproductible (presque toujours l’identifiant).
quantile	quantile / bucket	Découpage en groupes de taille comparable (<code>NTILE(n)</code>) : tiers, quarts, déciles.

Terme (français)	Équivalent anglais	Définition en une phrase
top N par groupe	top-N per group	Le motif <code>PARTITION BY groupe ORDER BY mesure DESC</code> puis filtre sur le rang — le classement le plus demandé en BI.
cumul	running total	Somme qui s'arrête à la ligne courante (<code>SUM(x) OVER (ORDER BY ...)</code>) ; dépend de l'ordre et du départage.
décalage	lag / lead	La valeur de la ligne précédente ou suivante dans l'ordre (<code>LAG/LEAD</code>) : c'est ce qui rend possible une variation.
trou de série	series gap	La ligne qui n'a pas de prédécesseur (premier mois) : elle rend <code>NULL</code> , et ce <code>NULL</code> doit être déclaré .

Définition. Les trois fonctions de rang répondent à trois questions différentes sur la même mesure : « **donne-moi un numéro unique** » (`ROW_NUMBER`), « **quelle place mérite-t-elle ?** » (`RANK`), « **combien de niveaux de performance y a-t-il jusqu'ici ?** » (`DENSE_RANK`). Choisir, c'est décider de ce que le rapport affirmera — pas de ce qui est le plus pratique.

Définition. Une **clé de départage** — tie-breaker — est une colonne ajoutée après la mesure dans l'`ORDER BY` d'une fenêtre (presque toujours la clé primaire de la table) pour que deux exécutions du même code produisent **le même** résultat. Sans elle, le moteur est libre : le classement n'est pas faux, il est **instable**.

5. Cours approfondi

5.1 Trois fonctions de rang, une seule mesure

```
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT id_client, t,
       ROW_NUMBER() OVER (ORDER BY t DESC, id_client) AS numero,
       RANK() OVER (ORDER BY t DESC) AS place,
       DENSE_RANK() OVER (ORDER BY t DESC) AS niveau
FROM c ORDER BY t DESC, id_client;
```

Sur le socle, les premières lignes montrent le client **0** (identifiant non référencé) seul au sommet, puis un groupe de clients à **20** tickets :

id_client	tickets	numero	place	niveau
0	38 984	1	1	1
3545	20	2	2	2
14859	20	3	2	2
21036	20	4	2	2
21392	20	5	2	2

Cinq clients ont exactement **20** tickets. `NUMERO` les numérote 2, 3, 4, 5, 6 (l'ordre entre eux vient du départage par identifiant) ; `PLACE` leur donne à tous **2** ; `NIVEAU` aussi **2**. Les trois colonnes sont justes — elles répondent à trois questions que le demandeur n'a pas distinguées. « Le 5e meilleur client » est une phrase vide : il n'existe pas de 5e meilleur client, il existe cinq clients deuxièmes.

5.2 Le départage : ce qui rend un classement reproductible

Sur la même mesure, l'écart entre `RANK` et `DENSE_RANK` s'amplifie à mesure qu'on descend : le `RANK` maximum atteint **23 444** (il saute les places perdues par les 3 316 clients à 8 tickets, et

par tous les autres ex æquo), tandis que le `DENSE_RANK` maximum vaut **21** — soit exactement le nombre de niveaux de fréquence observés.

Dans les faits. L'atelier a exécuté **15** fois la même requête de segmentation RFM (C04) **sans** clé de départage : le segment principal a compté **2 718, 2 720, 2 728** puis **2 738** clients. Le socle n'a pas changé d'une ligne. Ce n'est pas une erreur de mesure : c'est `NTILE` qui, à la frontière d'un tiers, choisit librement parmi des ex æquo. **3 316** clients partageant 8 tickets suffisent à déplacer cette frontière — et un segment qui bouge de 20 clients est un segment qu'on ne peut pas comparer à celui du trimestre dernier.

La règle est donc simple et non négociable : **toute fenêtre d'ordonnement reçoit un départage**. `ORDER BY t DESC, id_client` au lieu de `ORDER BY t DESC`. C'est une colonne de plus, et c'est la différence entre un indicateur et un tirage au sort.

Attention. Le départage ne change pas la **valeur** de la mesure, il change la **frontière**. Un top 3 par groupe avec ou sans départage peut désigner deux produits différents quand trois produits sont ex æquo — et le rapport, dans les deux cas, présentera trois lignes avec l'aplomb d'un résultat définitif.

5.3 `NTILE` : découper en tranches, pas classer

`NTILE(3) OVER (ORDER BY m DESC)` range les lignes en trois groupes de taille aussi égale que possible : c'est le découpage utilisé partout pour « les 10 % les plus gros clients ». Deux précautions, mesurées au §5.2 : le **départage** (sinon la frontière bouge) et la **définition** du découpage. Sur le socle, les deux écritures les plus courantes du « top 10 % » ne gardent pas le même nombre de lignes : `NTILE(100) <= 10` retient 2 350 clients, `ROW_NUMBER() <= 0,10 × 23 497` en retient 2 349. La part de chiffre d'affaires publiée est alors de **38,1 %** ou de **38,0 %** — un dixième de point, mais deux rapports qui ne se réconcilient plus.

Définition. Un **quantile** — quantile — est une frontière qui coupe une population ordonnée en parties de taille comparable : les tiers (`NTILE(3)`), les déciles (`NTILE(10)`), les centiles (`NTILE(100)`). Un quantile n'est pas un seuil : il ne demande pas « combien valent-ils ? » mais « combien sont-ils ? ».

5.4 Le top N par groupe : le motif qui exige un filtre après

Le classement le plus demandé en BI est le top N **par** groupe : les 3 meilleurs produits par famille, les 5 meilleurs clients par magasin. Le motif s'écrit en deux temps, et l'ordre des temps compte :

```
SELECT * FROM (
  SELECT p.categorie, p.designation, SUM(v.montant_ttc) AS ca,
         ROW_NUMBER() OVER (PARTITION BY p.categorie ORDER BY SUM(v.montant_ttc) DESC) AS rg
  FROM ventes v JOIN produit p USING (id_produit)
  WHERE NOT v.est_retour
  GROUP BY 1, 2
) WHERE rg <= 3 ORDER BY categorie, ca DESC;
```

Le filtre `rg <= 3` est à **l'extérieur** de la requête qui calcule le rang. Écrit dans la même requête, il s'appliquerait avant la fenêtre : `RANK` vaudrait alors **1** pour **toutes** les lignes restantes, et le rapport publierait un « top 3 » qui contient tout ce qu'on a laissé passer. C'est exactement le piège du `WHERE` avant la fenêtre (C01, §5.7), appliqué au cas le plus fréquent de la BI.

Exécuté sur le socle, ce top 3 rend **46** lignes — alors qu'un top 3 sur les **7** familles réelles du catalogue en rend **21**. L'écart n'est pas un problème de requête : il vient des **16** libellés de catégorie du référentiel produits (**9** écritures après casse et espaces, **7** familles après accents et pluriel), un défaut de qualité que C05 puis C07 traitent à fond. Retenez pour l'instant que le top N par groupe est l'endroit où une donnée mal normée devient **visible** : 46 lignes de réponse pour 7 familles, sans le moindre message d'alerte.

5.5 SUM() OVER : le cumul, et sa clôture

Un cumul s'écrit avec le même `SUM` que C01, plus un `ORDER BY` **dans** la fenêtre :

```
SELECT an, mm, ca, SUM(ca) OVER (PARTITION BY an ORDER BY mm) AS cumul
FROM (SELECT YEAR(date_vente) an, date_trunc('month', date_vente) mm, SUM(montant_ttc) ca
      FROM ventes WHERE NOT est_retour GROUP BY 1, 2);
```

Sur 2023, le cumul passe par **284 400 304 ; 547 583 580 ; 881 958 439 ; 1 196 335 034** FCFA, et la part cumulée par **8,3 ; 15,9 ; 25,6 ; 34,7** %. Deux contrôles gratuits :

1. **la clôture** : au dernier mois de l'année, le cumul doit égaler le total de l'année — **3 443 581 547** FCFA pour 2023 ;
2. **la monotonie** : un cumul de chiffre d'affaires ne peut pas diminuer. S'il diminue, une ligne a été comptée deux fois ou une annulation n'a pas été traitée.

5.6 LAG et LEAD : la variation, et son premier trou

`LAG(ca) OVER (ORDER BY mm)` rend la valeur du mois **précédent**, `LEAD` celle du mois **suivant**. La variation s'écrit alors directement, sans auto-jointure — la raison pour laquelle `LAG` a remplacé une bonne moitié des jointures dites « auto-jointures » des rapports d'avant.

Un décalage de **12** lignes donne la comparaison **annuelle** à périmètre égal : c'est la façon la plus honnête de comparer deux années, parce qu'elle compare **le même mois**. Sur les **32** mois du socle qui ont un équivalent un an plus tôt, la progression annuelle moyenne vaut **+16,8** % — et les premiers mois de 2024 valent **+11,0** %, **+23,5** %, **+20,6** %, **+27,7** %, **+11,5** %.

Définition. Un **trou de série** — series gap — est une ligne dont le décalage n'a pas de prédécesseur : le premier mois d'une série, ou le premier mois après un filtre trop zélé. `LAG` rend alors `NULL`, et l'arithmétique propage ce `NULL` (`ca - NULL` vaut `NULL`). Ce n'est pas un bug : c'est la réponse exacte à une question impossible — « par rapport à quoi ? ».

Attention. Le `WHERE` s'exécute avant la fenêtre. Une requête écrite `SELECT am, ca, LAG(ca, 12) OVER (ORDER BY am) ... WHERE am >= '2024-01'` filtre **puis** décale : les douze premiers mois manquent à l'appel et la première variation annuelle publiée est `NULL`. L'atelier a reproduit ce résultat avant de le corriger en calculant la fenêtre dans une sous-requête, puis en filtrant au-dessus.

5.7 Le classement qui contredit le total

Le socle fournit la démonstration la plus nette de ce qui distingue un **total** d'un **classement**. Les cinq magasins, rangés par chiffre d'affaires puis par panier moyen :

Magasin	CA net (FCFA)	Rang CA	Panier moyen (FCFA)	Rang panier
Ouaga 2000	5 312 205 459	1	107 374	3
Gounghin	3 424 978 244	2	106 621	4
Bobo Kibidwé	3 099 946 956	3	105 724	5
Koudougou Centre	2 024 794 213	4	108 035	2

Magasin	CA net (FCFA)	Rang CA	Panier moyen (FCFA)	Rang panier
Kaya Marché	1 733 230 083	5	111 447	1

Le classement par panier est **l'inverse** du classement par chiffre d'affaires : le dernier magasin du réseau a le plus gros panier moyen (**111 447** FCFA contre **107 374** pour le premier). Aucun des deux tableaux n'est faux, et aucune des deux conclusions ne s'ensuit : un petit magasin sert moins de clients, mais des clients qui achètent plus par visite. Publier « Kaya vend mal » ou « Kaya vend le mieux » est, dans les deux cas, une erreur de lecture — la bonne phrase porte sur ce qui est mesuré : le panier moyen, pas l'activité.

Conseil professionnel. Avant de publier un classement, écrivez en une phrase la question à laquelle il répond, et vérifiez qu'elle contient bien la mesure classée : « les magasins **par chiffre d'affaires** », « les magasins **par panier moyen** ». Un classement publié sans sa mesure est un classement qui sera lu comme un jugement de valeur.

5.8 Ce que ce chapitre ne couvre pas

Le temps **calendaire** — le mois sans vente qu'il faut fabriquer, la table de dates, l'absence de comparaison quand la période est incomplète — est le sujet de C03 : LAG compare des **lignes**, pas des **dates**, et deux mois qui se suivent dans un résultat ne sont pas forcément deux mois voisins au calendrier. La segmentation par **cohortes** et le **RFM** complet sont ceux de C04.

6. Exemple concret — trois classements, un seul socle

Le directeur commercial demande « les meilleurs clients ». La question, telle quelle, ne détermine pas la mesure : en tickets, en montant, en récence ? Le tableau de bord du module retient la **fréquence** (nombre de tickets) et publie **trois** colonnes de rang — c'est le seul moyen de montrer la différence en production :

Identifiant	Tickets	NUMERO	PLACE	NIVEAU
0	38 984	1	1	1
3545	20	2	2	2
14859	20	3	2	2
21036	20	4	2	2
21392	20	5	2	2
5	8	10 435	10 434	14

La dernière ligne est la leçon du chapitre. Un client à **8** tickets — la valeur la plus fréquente du socle, partagée par **3 316** clients — porte le numéro **10 435** (unique, donc faux comme information puisque 3 315 autres clients sont derrière lui à la même valeur), la place **10 434** (juste, illisible), et le niveau **14** (lisible : il existe 14 niveaux de fréquence au-dessus du sien sur **21** possibles).

Et le client **0** ? Il est premier partout, avec **2 852 612 447** FCFA et **18,3 %** du chiffre d'affaires du réseau — et il **n'existe pas** dans le référentiel clients : c'est l'identifiant des ventes non identifiées. Le classement l'ignore superbement, parce qu'un classement ne vérifie rien : il ordonne ce qu'on lui donne. Ce cas traverse tout le module et devient l'étude de cas de son évaluation.

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — mesurer l'ampleur des égalités avant de classer

```
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT COUNT(*) AS clients, COUNT(DISTINCT t) AS valeurs_distinctes, MAX(t) AS maximum FROM c;
```

```
23497 | 21 | 38984
```

23 497 clients pour **21** valeurs distinctes : le classement par fréquence est un classement d'égalités. Toute la suite du chapitre en découle.

7.2 Étape 2 — les trois rangs côte à côte

C'est la requête du §5.1. Sa sortie montre le client 0 seul au sommet, puis les cinq clients à 20 tickets qui partagent la place 2 (§6).

7.3 Étape 3 — l'écart maximal entre les deux classements

```
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1),
      r AS (SELECT RANK() OVER (ORDER BY t DESC) rk, DENSE_RANK() OVER (ORDER BY t DESC) dr FROM c)
SELECT MAX(rk), MAX(dr) FROM r;
```

```
23444 | 21
```

Le dernier client du socle est **23 444e** ou **21e** selon la fonction. Dit autrement : « le nombre de niveaux de fréquence » est une information, « le 23 444e client » n'en est pas une.

7.4 Étape 4 — le cumul, et son contrôle de clôture

```
WITH m AS (SELECT YEAR(date_vente) an, date_trunc('month', date_vente) mm, SUM(montant_ttc) ca
            FROM ventes WHERE NOT est_retour GROUP BY 1, 2),
      c AS (SELECT mm, SUM(ca) OVER (PARTITION BY an ORDER BY mm) cumul,
            SUM(ca) OVER (PARTITION BY an) total FROM m WHERE an = 2023)
SELECT mm, cumul, total, cumul = total AS cloture FROM c ORDER BY mm;
```

```
2023-01-01 | 284400304 | 3443581547 | false
2023-02-01 | 547583580 | 3443581547 | false
2023-03-01 | 881958439 | 3443581547 | false
...
2023-12-01 | 3443581547 | 3443581547 | true
```

Une seule ligne de contrôle (`cumul = total`) suffit à valider les douze mois.

7.5 Étape 5 — la variation mensuelle, puis annuelle

```
WITH m AS (SELECT date_trunc('month', date_vente) mm, SUM(montant_ttc) ca FROM ventes
            WHERE NOT est_retour GROUP BY 1)
SELECT mm, ca, LAG(ca) OVER (ORDER BY mm) AS mois_precedent,
       ROUND(100.0 * (ca - LAG(ca) OVER (ORDER BY mm)) / LAG(ca) OVER (ORDER BY mm), 1) AS var_pct
FROM m ORDER BY mm LIMIT 5;
```

2023-01-01	284400304	NULL	NULL
2023-02-01	263183276	284400304	-7.5
2023-03-01	334374859	263183276	27.1
2023-04-01	314376595	334374859	-6.0
2023-05-01	275723327	314376595	-12.3

Le premier mois rend `NULL` : il n'a pas de mois précédent. La variation mensuelle d'un socle de quincaillerie est **bruyante** (de -12,3 % à +27,1 % en cinq mois) : c'est exactement pourquoi le chapitre C03 lui préférera des moyennes mobiles.

7.6 Étape 6 — la comparaison annuelle, et le piège du filtre

Avec `LAG(ca, 12)`, la comparaison porte sur le **même mois** de l'année précédente : **+11,0 %**, **+23,5 %**, **+20,6 %**, **+27,7 %**, **+11,5 %** pour les cinq premiers mois de 2024, et **+16,8 %** en moyenne sur les **32** mois comparables. Écrite avec le filtre dans la même requête que la fenêtre, la même requête publie `NULL` — le filtre a supprimé les douze mois dont `LAG` avait besoin.

8. Erreurs fréquentes

1. **Numéroter quand on voulait classer.** `ROW_NUMBER` attribue un numéro unique : sur une mesure à égalités, il **invente** une hiérarchie (5 clients à 20 tickets deviennent 2e, 3e, 4e, 5e et 6e). Le rapport qui en découle annonce « le 5e meilleur client » alors qu'il n'en existe que deux niveaux.
2. **Classer sans départage.** Sans colonne de départage, deux exécutions peuvent rendre deux frontières différentes : l'atelier a mesuré **2 718**, **2 720**, **2 728** et **2 738** clients pour un même segment RFM. Un indicateur qui bouge sans que la donnée bouge n'est pas un indicateur.
3. **Filtrer le top N dans la même requête que le rang.** `WHERE rg <= 3` s'applique **avant** la fenêtre : le rang vaut alors 1 pour toutes les lignes survivantes et le « top 3 » contient tout ce qui a passé le filtre. Le filtre sur un rang se met **toujours** dans une requête englobante.
4. **Publier une variation sur le premier mois.** `LAG` rend `NULL` au premier mois, et la division propage le `NULL`. Afficher ce `NULL` comme « 0 % » est la faute la plus coûteuse des séries temporelles : elle transforme un trou en information.
5. **Confondre le rang et le total.** Le premier magasin du réseau par chiffre d'affaires n'est que **3e** par panier moyen, et le dernier est **1er**. Les deux classements sont justes ; un rapport qui n'annonce pas sa mesure sera lu comme un jugement sur les équipes.

9. Bonnes pratiques professionnelles

1. **Mesurer les égalités avant de classer.** `COUNT(DISTINCT mesure)` à côté de `COUNT(*)` : si le rapport est faible (ici **21** valeurs pour **23 497** lignes), le choix de la fonction de rang n'est plus un détail, c'est le sujet.
2. **Toujours départager, et écrire pourquoi.** `ORDER BY mesure DESC, id_client` : la clé de départage est l'identifiant le plus petit, et cela se documente en commentaire dans la requête — un lecteur qui ne comprend pas la règle d'égalité ne peut pas reproduire le rapport.
3. **Choisir la fonction selon la phrase, pas l'inverse.** « Il est 2e » → `RANK`. « Il existe deux niveaux au-dessus » → `DENSE_RANK`. « Il est le 3e de la liste » → `ROW_NUMBER` **avec** départage assumé.
4. **Contrôler un cumul par sa clôture et sa monotonie.** Le dernier cumul égale le total de la période ; un cumul qui diminue signale un doublon ou une annulation non traitée.
5. **Déclarer les trous.** Quand une série commence, le premier décalage est `NULL` : on l'écrit dans la note de méthode du rapport (« les variations de janvier 2023 ne sont pas calculables : pas d'antériorité dans la base ») plutôt que de laisser la cellule vide.

10. Exercice guidé

Situation. Le comité de direction a reçu, la semaine dernière, deux rapports sur la fidélité des clients, produits par deux analystes de l'équipe. Le premier conclut : « **5** clients dépassent 20 tickets ; le 5e est à 20 tickets ». Le second conclut : « le seuil des plus fidèles est de **20** tickets ; **cinq** clients l'atteignent ». Les deux rapports sont **incompatibles** et la direction demande pour quoi.

Consigne. Dans les 20 minutes (chrono) :

1. Écrivez la requête qui reproduit le classement du premier rapport, et celle du second. (3 pts)
2. Expliquez, en nommant les fonctions, pourquoi elles ne disent pas la même chose sur les **mêmes** données. (3 pts)
3. Montrez, chiffres en main, où les deux rapports divergent **le plus** : quel est l'écart maximal entre RANK et DENSE_RANK sur le socle ? (2 pts)
4. Écrivez la phrase que vous enverriez aux deux analystes pour qu'ils produisent à l'avenir des rapports comparables. (2 pts)

11. Exercices autonomes

E1 — Le top N par famille, proprement (35 min). Produisez les **3** produits les plus vendus de chaque **famille** réelle du catalogue — au sens de **C05**, qui définit la famille comme le libellé replié (casse, espaces, accents, pluriel) — et non de chaque libellé présent dans le fichier. Pour chaque produit son rang, son chiffre d'affaires et la part qu'il représente **dans sa famille**. Contrainte : le filtre sur le rang doit être **à l'extérieur** de la requête qui le calcule, et vous joindrez la preuve du nombre de lignes attendu ($3 \times$ nombre de familles).

E2 — Le cumul et la variation d'un magasin (30 min). Pour le magasin de votre choix, publiez mois par mois : le chiffre d'affaires, le cumul depuis janvier 2023, la variation mensuelle et la variation annuelle à périmètre égal. Contrôles exigés : le cumul du dernier mois égale le total du magasin, et les mois sans antériorité sont **déclarés NULL** (pas zéro).

12. Correction détaillée

Exercice guidé.

1. **Les deux requêtes.** Le premier rapport classe avec ROW_NUMBER (ou avec LIMIT 5 sur un tri) :

```
sql WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour
GROUP BY 1) SELECT id_client, t, ROW_NUMBER() OVER (ORDER BY t DESC, id_client) AS numero FROM
c ORDER BY t DESC, id_client LIMIT 5;
```

Le second annonce une **égalité**, donc un RANK (ou un DENSE_RANK) :

```
sql WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour
GROUP BY 1) SELECT id_client, t, RANK() OVER (ORDER BY t DESC) AS place FROM c WHERE place <= 2
ORDER BY t DESC, id_client;
```

2. **Pourquoi les deux conclusions diffèrent.** Le premier affirme une **hiérarchie** (« le 5e est à 20 tickets ») : ROW_NUMBER numérote des lignes, y compris quand elles ont la même valeur. Le second affirme un **niveau** (« cinq clients atteignent 20 tickets ») : RANK attribue la même place aux ex æquo. Sur des données sans égalité, les deux coïncident — c'est pourquoi l'erreur passe longtemps inaperçue. 3. **L'écart maximal.** Sur le socle, MAX(RANK) vaut **23 444** et MAX(DENSE_RANK) vaut **21**. L'écart vient des **3 316** clients à 8 tickets, qui font sauter 3 315 places à eux seuls. 4. **La phrase à envoyer.** « Merci de préciser dans chaque rapport la fonction de rang employée et la clé de départage : RANK pour parler de places (avec égalités partagées), DENSE_RANK pour parler de niveaux, ROW_NUMBER seulement quand un rang unique est explicitement demandé — et dans ce cas, écrire la règle de départage. »

E1 (le top 3 par famille). Il faut d’abord **normaliser** le libellé : `UPPER(TRIM(categorie))` réunit casse et espaces (**9** écritures), et le repliement des accents et du pluriel ramène à **7** familles (voir C05). Puis classer par famille :

```
WITH p AS (SELECT id_produit, UPPER(TRIM(categorie)) AS famille FROM produit),
v AS (SELECT p.famille, p.id_produit, SUM(x.montant_ttc) AS ca
      FROM ventes x JOIN p USING (id_produit) WHERE NOT x.est_retour GROUP BY 1, 2)
SELECT * FROM (
  SELECT famille, id_produit, ca,
         ROW_NUMBER() OVER (PARTITION BY famille ORDER BY ca DESC, id_produit) AS rg,
         ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY famille), 1) AS part_famille_pct
  FROM v
) WHERE rg <= 3 ORDER BY famille, rg;
```

Contrôle attendu : **21** lignes (3×7 familles). Sans la normalisation, la même requête sur les libellés bruts rend **46** lignes — l’écart est la trace exacte du défaut de qualité du référentiel.

E2 (le cumul d’un magasin). La structure attendue est celle du §7.4 avec `PARTITION BY id_magasin`, plus un `LAG(ca, 12) OVER (PARTITION BY id_magasin ORDER BY am)` pour la variation annuelle. Les deux contrôles : le cumul du dernier mois égale le total du magasin (**5 312 205 459** FCFA pour Ouaga 2000, **1 733 230 083** FCFA pour Kaya Marché), et les douze premiers mois affichent une variation annuelle `NULL` — déclarée, jamais remplacée par zéro.

13. Mini-projet de chapitre

La note « trois classements, une mesure ». Vous produisez 3 pages maximum pour le comité commercial : trois classements du même socle, avec la mesure annoncée pour chacun et la preuve du départage.

Livrables et barème.

Critère	Ce qui est vérifié	Points
Les trois classements	<code>ROW_NUMBER</code> , <code>RANK</code> , <code>DENSE_RANK</code> sur la même mesure, sorties publiées	6
La preuve du départage	la clé de départage écrite et justifiée ; une deuxième exécution rend le même résultat	5
Les chiffres d’égalité	23 497 clients, 21 valeurs distinctes, 3 316 clients à 8 tickets, écart 23 444 contre 21	4
La phrase de conclusion	une phrase par classement, avec la mesure annoncée et ce qu’elle interdit de dire	3
Total		18

14. Résumé du chapitre

Classer, c’est décider **ce qu’on affirme** : `ROW_NUMBER` numérote des lignes (hiérarchie inventée si égalités), `RANK` attribue des places (les suivantes sont sautées), `DENSE_RANK` compte les niveaux. Sur le socle, la mesure de fréquence ne compte que **21** niveaux pour **23 497** clients, dont **3 316** partagent 8 tickets : le choix de la fonction n’est jamais neutre. `NTILE` découpe en tranches de taille égale — utile pour « les 10 % les plus gros », dangereux sans **clé de départage** (l’atelier a mesuré des frontières à **2 718**, **2 720**, **2 728** et **2 738** clients pour la même requête). Le **top N par groupe** place le filtre **après** le rang, jamais avant. Le **cumul** se contrôle par sa clôture (le cumul de décembre égale **3 443 581 547** FCFA pour 2023) et par sa monotonie. Le **décalage** (`LAG`, `LEAD`) calcule variations mensuelles et annuelles — **+16,8 %** en moyenne sur les **32** mois comparables du socle — à condition de **déclarer** le premier trou. Et un classement ne vaut que par sa mesure : les cinq magasins sont rangés exactement à l’envers selon qu’on classe par chiffre d’affaires ou par panier moyen.

15. À retenir

1. **ROW_NUMBER numérote, RANK classe, DENSE_RANK compte les niveaux.** Trois questions différentes, trois colonnes justes, un seul rapport recevable — celui qui annonce la sienne.
2. **Aucune fenêtre d'ordonnancement sans clé de départage.** Sans elle, le classement change sans que la donnée change.
3. **Le filtre sur un rang se met dans une requête englobante.** Dans la même requête, il s'applique avant la fenêtre et le rang vaut 1 partout.
4. **Un cumul se contrôle deux fois** : sa clôture (dernier cumul = total de la période) et sa monotonie.
5. **LAG du premier mois rend NULL, et ce NULL se déclare.** Le remplacer par zéro est une invention.

À retenir. Un classement est une **phrase**, pas une colonne : « les magasins par panier moyen » n'affirme pas la même chose que « les magasins par chiffre d'affaires », même quand les deux tableaux contiennent les mêmes cinq noms.

À retenir. L'égalité n'est pas un cas limite : c'est le cas normal d'une mesure de comptage. Un classement écrit sans penser aux ex æquo est un classement écrit pour un autre jeu de données.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Cinq clients ont 20 tickets. Que valent leur ROW_NUMBER, leur RANK et leur DENSE_RANK s'ils sont précédés d'un seul client à 38 984 tickets ?
2. Sur le socle, MAX(RANK) et MAX(DENSE_RANK) valent respectivement... ? Pourquoi un tel écart ?
3. Pourquoi NTILE a-t-il besoin d'une clé de départage, alors qu'il ne publie aucun rang ?
4. Où placer le filtre `rg <= 3` d'un top 3 par groupe, et que se passe-t-il sinon ?
5. Que vaut **2 718** dans ce chapitre ? Et **2 738** ?
6. Pourquoi le cumul de décembre doit-il évaluer le total de l'année ?
7. Que rend `LAG(ca, 12)` pour le premier mois d'une série, et que publie-t-on à la place ?
8. Le premier magasin en chiffre d'affaires est 3e en panier moyen : quelle phrase peut-on écrire, et laquelle est interdite ?
9. Combien de lignes rend un top 3 par catégorie sur les libellés du fichier, et combien sur les familles réelles ?
10. Pourquoi la variation mensuelle du socle est-elle un mauvais indicateur de tendance, avant même le chapitre C03 ?

Réponses. 1. ROW_NUMBER : 2, 3, 4, 5, 6 ; RANK : 2 pour les cinq ; DENSE_RANK : 2 pour les cinq. 2. **23 444** et **21** : RANK saute les places des ex æquo (les **3 316** clients à 8 tickets en font sauter 3 315 à eux seuls), DENSE_RANK compte les niveaux. 3. Parce que la frontière des tranches tombe au milieu d'ex æquo : sans départage, le moteur choisit qui passe (mesuré : **2 718, 2 720, 2 728, 2 738**). 4. Dans une requête **englobante** ; dans la même requête, le WHERE s'exécute avant la fenêtre et le rang vaut 1 pour toutes les lignes restantes. 5. Deux tailles observées du même segment RFM selon l'exécution : la preuve que la frontière n'est pas reproductible. 6. Parce que le cumul est la somme des mois de l'année : s'il diffère, une ligne a été comptée deux fois ou manque. 7. NULL ; on **déclare** le trou dans la note de méthode. 8. « Ouaga 2000 réalise le plus gros chiffre d'affaires du réseau et un panier moyen de **107 374** FCFA, inférieur à celui de Kaya Marché (**111 447** FCFA) » ; interdit : « Ouaga 2000 vend mieux que les autres ». 9. **46** lignes

(16 libellés) contre **21** (7 familles). 10. Parce que la variation mensuelle va de $-12,3\%$ à $+27,1\%$ en cinq mois sur un socle stable : elle mesure le calendrier et le hasard plus que la tendance.

Module M11.C03 — Le temps en SQL : table calendrier, année sur année, mois glissants, et le mois qui manque

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5). **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 et C02 (fenêtres, cadres, LAG) ; M07.C02 (dates, WHERE), M07.C06 (CASE) ; M09.C03 (tendance, saisonnalité, bruit).

L'idée du chapitre. Comparer dans le temps a l'air simple — « cette année contre l'année dernière » — et c'est pourtant la famille de requêtes qui produit le plus de chiffres faux de toute la BI, parce que **le temps est incomplet** et que rien ne le signale : l'année en cours n'est pas finie, certains mois n'ont pas d'objectif, une moyenne mobile de douze mois calculée sur quatre mois reste une moyenne. Le socle du module donne à voir les trois cas d'un coup : **44** mois de ventes sans le moindre trou, mais **2** mois sans objectif (**magasin 4**, février et mars 2024) — et une comparaison naïve de 2026 à 2025 qui publie **-28,9 %** là où la comparaison à périmètre égal (huit mois contre huit mois) publie **+15,4 %**. Ce chapitre apprend à écrire le temps **explicitement** : un calendrier, un périmètre, un compteur de fenêtre.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql, rejoué en 0,4 s). Le socle contient une table `calendrier` (**1 339** jours, du 01/01/2023 au 31/08/2026, avec jours fériés et libellés), les **44** mois de ventes (**237 191** lignes nettes, **15 595 154 955** FCFA) et **218** lignes d'objectifs mensuels par magasin. Toutes les sorties publiées ici sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Fabriquer un axe de temps complet** — avec `generate_series` ou une table `calendrier` — et **prouver** qu'il couvre la période du rapport.
2. **Détecter un trou de série** : distinguer un mois sans vente (qui n'existe pas toujours) d'un mois sans référence en face (objectif, budget, effectif), et le rendre visible par une jointure **externe**.
3. **Comparer deux périodes à périmètre égal** : dire explicitement quels mois sont comparés, et pourquoi la comparaison « année complète contre année en cours » est interdite.
4. **Calculer une moyenne mobile** de 3 et de 12 mois, avec le **cadre** qui convient, et **compter les lignes** de la fenêtre pour savoir si elle est complète.
5. **Lire la saison** du socle : jour de la semaine (rapport **3,2** entre le meilleur et le pire jour) et mois calendaire (rapport **1,65** entre mars et juillet), et s'en servir pour interpréter — pas pour excuser.

2. Pourquoi cette notion est importante

Une entreprise ne produit pas deux fois la même semaine. Le socle le montre sans ambiguïté : le lundi rapporte **13 872 370** FCFA par jour de vente, le dimanche **4 322 830** — un rapport de **3,2**. Le mois de mars vaut en moyenne **433 645 399** FCFA par mois, celui de juillet **262 079**

192 — un rapport de **1,65**. Autrement dit : **comparer deux mois sans regarder le mois**, c’est comparer deux calendriers différents et appeler cela une tendance.

Par-dessus cette saison, la vraie difficulté est l’**incomplétude**. Trois situations, toutes présentes sur le socle :

- **La période en cours n’est pas finie.** Les ventes s’arrêtent au 31/08/2026 : 2026 compte **8** mois, 2025 en compte **12**. Comparer les deux totaux donne **−28,9 %** — une chute spectaculaire qui n’existe pas. À périmètre égal (janvier à août), la progression est de **+15,4 %**, cohérente avec les **+15,3 %** de 2025 et les **+18,8 %** de 2024 sur la même fenêtre.
- **La référence en face peut manquer.** Les objectifs couvrent **218** couples magasin-mois sur **220** : le magasin 4 n’a **pas d’objectif** pour février et mars 2024, alors qu’il y a réalisé **93 749 490** FCFA de ventes. Un rapport qui ne joint pas correctement publie soit une ligne à zéro (faux), soit rien du tout (invisible).
- **Une fenêtre peut être incomplète.** Une moyenne mobile de douze mois calculée sur un socle qui commence n’est complète qu’à partir du douzième mois. Sans compteur, janvier 2023 affiche « moyenne mobile 12 mois » d’un seul mois — et personne ne le voit.

3. Explication simple — l’agenda et les trous

Prenez un agenda papier de trois ans et demi. Vous y écrivez chaque soir le chiffre d’affaires du jour.

1. **Les pages existent même les jours sans écriture.** C’est tout l’intérêt d’un **calendrier** : il fournit les lignes que les ventes n’ont pas créées. Sans lui, un dimanche sans vente disparaît de la moyenne des dimanches — et la moyenne monte.
2. **Une page vide n’est pas un zéro.** Un jour sans vente vaut **0** FCFA ; un mois sans objectif vaut **inconnu**. Les deux se ressemblent dans un tableur et ne disent pas la même chose : le premier est une mesure, le second est une absence de mesure. La jointure externe (`LEFT JOIN`, `FULL OUTER JOIN`) est l’outil qui rend cette différence **visible**.
3. **Un agenda qui commence en janvier ne peut pas parler de décembre.** Toute comparaison « même mois, année précédente » a besoin de douze mois d’historique avant de pouvoir exister : sur le socle, seuls **32** des **44** mois ont un équivalent un an plus tôt. Les douze autres doivent afficher `NULL` et le dire.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
axe de temps	time axis	La liste complète des périodes du rapport, y compris celles sans donnée.
table calendrier	date dimension / calendar table	La table qui contient une ligne par jour (ou par mois) avec ses attributs : année, mois, trimestre, jour férié.
série fabriquée	generated series	Un axe de temps produit par le moteur (<code>generate_series</code>) plutôt que lu dans une table.
jointure externe	outer join	Une jointure qui conserve les lignes sans correspondance (<code>LEFT</code> , <code>FULL OUTER</code>) : le seul moyen de voir un trou.
trou de série	series gap	Une période de l’axe sans donnée : soit une absence réelle (pas de vente), soit une absence de mesure (pas d’objectif).
périmètre	scope / perimeter	L’ensemble des lignes comparées ; deux périodes ne se comparent qu’à périmètre égal.
période partielle	partial period	Une période en cours (mois courant, année courante) : sa comparaison à une période complète est fautive par construction.

Terme (français)	Équivalent anglais	Définition en une phrase
année sur année	year over year	Comparaison d'une période à la même période de l'année précédente (LAG(ca, 12) sur une série mensuelle).
moyenne mobile	moving average	Moyenne calculée sur une fenêtre qui avance (3 mois, 12 mois) : elle lisse le bruit sans supprimer la tendance.
saison	seasonality	La régularité calendaire d'une activité (jour de la semaine, mois, fêtes) : à connaître avant d'interpréter un écart.

Définition. Une **table calendrier** — date dimension — est la table de référence du temps : une ligne par jour (ici **1 339**), avec ses attributs calculés une fois pour toutes (année, mois, trimestre, libellé du jour, indicateur de férié). C'est **elle** qui rend les trous visibles : une jointure externe entre le calendrier et les ventes fait apparaître les jours sans vente, que les ventes seules ne peuvent pas montrer puisqu'elles ne contiennent que ce qui a eu lieu.

Définition. Le **périmètre** — scope — d'une comparaison est l'ensemble des lignes effectivement comparables : mêmes mois, même nombre de jours, mêmes magasins, mêmes exclusions. « 2026 contre 2025 » n'est pas un périmètre : c'est une phrase incomplète. « janvier à août 2026 contre janvier à août 2025 » en est un.

5. Cours approfondi

5.1 Fabriquer l'axe du temps

Deux façons d'obtenir un axe complet, et elles doivent **concorde** :

```
-- 1. la série fabriquée par le moteur
SELECT COUNT(*) FROM generate_series(DATE '2023-01-01', DATE '2026-08-31', INTERVAL 1 DAY);
-- 2. la table de référence du socle
SELECT COUNT(*) FROM calendrier;
```

Les deux rendent **1 339**. Ce contrôle n'est pas une coquetterie : il vérifie que la table de référence et la période réelle du socle parlent du même monde. Un écart signifie qu'un des deux est périmé — et c'est exactement le genre d'écart qui fait publier un rapport sur une période que la base ne couvre plus.

Pour un rapport mensuel, on agrège les ventes **par mois** puis on joint le calendrier des mois — ou l'on utilise directement `date_trunc('month', date_vente)` :

```
SELECT strftime(date_trunc('month', date_vente), '%Y-%m') AS mois, SUM(montant_ttc) AS ca
FROM ventes WHERE NOT est_retour GROUP BY 1 ORDER BY 1;
```

44 lignes. Le socle n'a **aucun** mois sans vente : c'est une chance, et cela doit être vérifié, pas supposé.

5.2 Le trou qu'on ne voit pas : l'objectif manquant

Les ventes couvrent les 44 mois ; les **objectifs** couvrent **218** couples magasin-mois sur les **220** possibles (5 magasins × 44 mois). Deux lignes manquent : le **magasin 4** en **février** et **mars 2024**.

Une jointure **interne** ne les voit pas — elle ne garde que ce qui existe des deux côtés, donc elle **supprime** silencieusement les deux mois. Une jointure **externe** les affiche :


```
WITH v AS (SELECT id_magasin, strftime(date_trunc('month', date_vente), '%Y-%m') am,
SUM(montant_ttc) ca
FROM ventes WHERE NOT est_retour GROUP BY 1, 2),
o AS (SELECT id_magasin, annee_mois am, ca_objectif_ttc obj FROM objectif_mois)
SELECT COALESCE(v.id_magasin, o.id_magasin) AS magasin, COALESCE(v.am, o.am) AS mois,
ROUND(v.ca) AS realise, o.obj AS objectif,
CASE WHEN o.obj IS NULL THEN 'objectif manquant'
WHEN v.ca IS NULL THEN 'vente manquante' ELSE 'complet' END AS etat
FROM v FULL OUTER JOIN o ON o.id_magasin = v.id_magasin AND o.am = v.am
WHERE o.obj IS NULL OR v.ca IS NULL;
```

magasin	mois	realise	objectif	etat
4	2024-02	42514372	NULL	objectif manquant
4	2024-03	51235118	NULL	objectif manquant

93 749 490 FCFA de ventes sans objectif en face. Le rapport de suivi a maintenant deux choix intellectuellement honnêtes : exclure ces deux mois **en le disant**, ou les afficher avec un taux de réalisation vide. Le choix malhonnête — et le plus fréquent — est de laisser la jointure interne les supprimer, puis d’annoncer « 218 mois suivis, 100 % de couverture ».

Attention. Une jointure interne sur une table de référence **réduit** le périmètre en silence. C’est le seul endroit du chapitre où la perte d’information ne produit ni **NULL**, ni erreur, ni ligne manquante : il n’y a simplement plus rien à voir. Règle de la maison : **toute jointure vers une table de référence se vérifie par un COUNT(*) avant et après**.

Définition. Un **trou de série** — series gap — est une période de l’axe de temps pour laquelle il n’existe **aucune** donnée : soit parce que rien ne s’est produit (un dimanche sans vente), soit parce que la mesure n’a pas été faite (un mois sans objectif). La jointure externe les rend **visibles** ; c’est sa seule fonction, et c’est suffisant pour la justifier.

5.3 Année sur année : le périmètre d’abord

Le socle s’arrête au 31/08/2026. Deux lectures du même chiffre :

Comparaison	Calcul	Résultat	Verdict
2026 (8 mois) contre 2025 (12 mois)	3 360 553 372 contre 4 724 700 286	−28,9 %	faux
janvier—août 2026 contre janvier—août 2025	3 360 553 372 contre 2 913 055 394	+15,4 %	défendable

La même année, la même base, deux conclusions opposées. La première n’est pas une erreur de calcul : c’est une erreur de **périmètre**. Elle est d’autant plus dangereuse qu’elle va dans le sens de la prudence (« l’activité baisse »), donc personne ne la conteste.

La bonne écriture est une fonction du périmètre, pas du calendrier :

```
WITH m AS (SELECT YEAR(date_vente) an, MONTH(date_vente) mo, SUM(montant_ttc) ca
FROM ventes WHERE NOT est_retour GROUP BY 1, 2)
SELECT an, ROUND(SUM(ca)) AS ca_total, ROUND(SUM(ca) FILTER (WHERE mo <= 8)) AS ca_8_mois
FROM m GROUP BY 1 ORDER BY 1;
```

2023	3443581547	2127742355
2024	4066319750	2527257254
2025	4724700286	2913055394
2026	3360553372	3360553372

La colonne **ca_8_mois** aligne les quatre années sur la même fenêtre — **8,8 %**, **+15,3 %**, **+15,4 %** — et cette fois 2026 se lit correctement. C’est aussi le contrôle que les trois croissances successives sont **cohérentes** entre elles.

5.4 Moyennes mobiles : la fenêtre pleine et son compteur

Un mois isolé est bruité : sur le socle, la variation mensuelle va de **-12,3 %** à **+27,1 %** en cinq mois, pour une tendance annuelle de **+16,8 %**. La moyenne mobile existe pour cela :

```
WITH m AS (SELECT date_trunc('month', date_vente) mm, SUM(montant_ttc) ca
            FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT strftime(mm, '%Y-%m') AS mois, ROUND(AVG(ca) OVER (ORDER BY mm
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS moyenne_3_mois,
COUNT(*) OVER (ORDER BY mm ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS lignes_3,
ROUND(AVG(ca) OVER (ORDER BY mm
            ROWS BETWEEN 11 PRECEDING AND CURRENT ROW)) AS moyenne_12_mois,
COUNT(*) OVER (ORDER BY mm ROWS BETWEEN 11 PRECEDING AND CURRENT ROW) AS lignes_12
FROM m ORDER BY mm LIMIT 4;
```

2023-01		284400304		1		284400304		1
2023-02		273791790		2		273791790		2
2023-03		293986146		3		293986146		3
2023-04		303978243		3		299083759		4

La cinquième colonne est la leçon : en janvier 2023, la « moyenne mobile de 12 mois » ne porte que sur **1** mois ; en avril, sur **4**. Sans le compteur, ces valeurs s’affichent avec la même autorité que celles des 32 mois complets — et la courbe a l’air de faire le travail pour lequel on l’a tracée.

Définition. Le **cadre de fenêtre** — window frame — est la partie de la partition réellement prise en compte autour de la ligne courante. `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` désigne trois lignes ; `ROWS BETWEEN 11 PRECEDING AND CURRENT ROW` en désigne douze. Le cadre **ne s’étend pas au-delà** des lignes existantes : quand il n’y en a qu’une, il n’en prend qu’une.

5.5 La saison, à connaître avant d’interpréter

Deux saisons structurent le socle.

Le jour de la semaine. Le lundi rapporte **13 872 370** FCFA par jour de vente, le dimanche **4 322 830** : rapport **3,2**. Un rapport hebdomadaire qui compare « ce lundi » à « ce samedi » mesure le calendrier, pas l’activité. Le contrôle se fait avec le calendrier du socle :

```
SELECT c.libelle_jour, ROUND(SUM(v.montant_ttc) / COUNT(DISTINCT v.date_vente)) AS ca_par_jour
FROM ventes v JOIN calendrier c ON CAST(c.date AS DATE) = v.date_vente
WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca_par_jour DESC;
```

Le mois calendaire. Mars vaut en moyenne **433 645 399** FCFA par mois, juillet **262 079 192** : rapport **1,65**. Un premier trimestre et un troisième trimestre ne se comparent jamais directement.

Dans les faits. L’écart saisonnier (**×1,65** sur le mois, **×3,2** sur le jour) est **plus grand** que la croissance annuelle du socle (**+16,8 %**). Autrement dit : sur cette activité, le calendrier pèse davantage que la tendance. Toute conclusion tirée d’un mois isolé, sans référence au même mois de l’année précédente, a plus de chances de décrire le calendrier que l’entreprise.

5.6 Le mois partiel : la dernière ligne du tableau

Un mois n’est comparable à un autre que s’il est **complet**. Août 2026 l’est : **5 022** lignes de vente du 1er au 31, aucune journée manquante. Mais si le socle s’était arrêté au 15 août, le total du mois aurait été **environ deux fois** plus petit — et la moyenne mobile de 3 mois, la variation

annuelle et la croissance du trimestre auraient toutes été fausses **en même temps**, sans qu'aucune ne signale l'autre.

Attention. Le dernier point du graphique est presque toujours un mois **partiel**, et c'est celui qu'on regarde. Une moyenne mobile qui l'intègre baisse mécaniquement ; une variation annuelle qui l'utilise compare un bout de mois à un mois entier. La règle : le dernier point se publie **avec** son nombre de jours couverts, ou pas du tout — sur le socle, août 2026 est complet (5 022 lignes du 1er au 31), donc publiable ; un mois arrêté au 15 ne l'aurait pas été.

Le contrôle de complétude coûte une ligne :

```
SELECT COUNT(*) AS jours_de_vente, MIN(date_vente), MAX(date_vente)
FROM ventes WHERE NOT est_retour AND date_vente >= DATE '2026-08-01';
```

À retenir. Trois contrôles non négociables avant de publier une comparaison temporelle : la période est-elle **couverte** (calendrier), la référence est-elle **présente** (jointure externe), la fenêtre est-elle **pleine** (compteur de lignes). Les trois tiennent en trois colonnes, et les trois erreurs qu'ils évitent sont invisibles sans eux.

Conseil professionnel. Construisez la table calendrier **une fois** pour tout le service, et non une fois par rapport. Elle se vérifie en trois lignes (la période couverte, le nombre de jours, la présence des fériés) et se réutilise partout : comparaisons inter-annuelles, moyennes mobiles, jours ouvrés, découpages trimestriels. Un rapport qui fabrique son propre axe de temps le fabrique différemment des autres — et deux rapports qui ne partagent pas leur calendrier ne se réconcilient jamais.

5.7 Ce que ce chapitre ne couvre pas

La **rétenition** et les **cohortes** (le temps vu par client, et non par période) sont le sujet de C04. La **prévision** (prolonger une série) appartient à M19. Ici, on ne fait que rendre le temps complet, comparable et vérifiable.

6. Exemple concret — trois lectures de la même année

Un comité de direction demande « où en est l'année 2026 ? ». Le socle permet trois réponses, et une seule est utilisable :

Lecture	Requête	Résultat	Ce que ça vaut
Totale	SUM(montant_ttc) par année	3 360 553 372 FCFA	un chiffre, pas une réponse
Naïve	2026 contre 2025 complet	-28,9 %	faux (périmètre)
À périmètre égal	janvier—août 2026 contre janvier—août 2025	+15,4 %	la réponse

Et la réponse **s'accompagne** de sa saison : **+15,4 %** se compare à **+15,3 %** (2025) et **+18,8 %** (2024) sur la même fenêtre de huit mois. La conclusion tenable est donc : « la progression à périmètre égal est de **+15,4 %**, en très légère décélération par rapport aux **+18,8 %** de 2024 ». C'est une phrase qu'un décideur peut utiliser — parce qu'elle dit ce qui est comparé.

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — compter l'axe du temps

```
SELECT COUNT(*) FROM generate_series(DATE '2023-01-01', DATE '2026-08-31', INTERVAL 1 DAY);
SELECT COUNT(*) FROM calendrier;
```

```
1339
1339
```

7.2 Étape 2 — le CA mensuel, et la preuve qu'il n'y a pas de trou

44 lignes, du 2023-01 au 2026-08, aucune manquante : la série mensuelle est **dense**.

7.3 Étape 3 — les objectifs, et leurs deux trous

C'est le `FULL OUTER JOIN` du §5.2 : 2 lignes d'état `objectif` manquant, magasin 4, février et mars 2024, pour **93 749 490** FCFA de ventes.

7.4 Étape 4 — année sur année à périmètre égal

Le tableau du §5.3 : **2 127 742 355 ; 2 527 257 254 ; 2 913 055 394 ; 3 360 553 372** FCFA sur les huit premiers mois, soit **+18,8 %**, **+15,3 %** et **+15,4 %**.

7.5 Étape 5 — moyennes mobiles avec compteurs

Les trois derniers mois du socle, avec des fenêtres **complètes** :

2026-06	351152152	moyenne 3 mois : 423242990	moyenne 12 mois : 421347035
2026-07	328841773	moyenne 3 mois : 366577753	moyenne 12 mois : 426175710
2026-08	368251515	moyenne 3 mois : 349415147	moyenne 12 mois : 431016522

La moyenne de 3 mois baisse (**423 242 990** → **349 415 147**) alors que la moyenne de 12 mois monte (**421 347 035** → **431 016 522**) : le court terme et le long terme ne racontent pas la même chose, et c'est normal. Une seule des deux courbes ne suffit jamais à décider.

7.6 Étape 6 — la saison, pour interpréter

Le jour (rapport **3,2**) et le mois (rapport **1,65**) du §5.5. À partir d'ici, chaque écart publié dans le module est comparé **au même mois de l'année précédente**, jamais au mois d'avant.

8. Erreurs fréquentes

1. **Comparer une période partielle à une période complète.** 2026 compte **8** mois, 2025 en compte **12** : la comparaison directe publiée **-28,9 %** au lieu de **+15,4 %** à périmètre égal. La faute est invisible parce qu'elle va dans le sens de la prudence.
2. **Joindre une table de référence en jointure interne.** Les **2** mois sans objectif du magasin 4 disparaissent du rapport : **93 749 490** FCFA de ventes s'évaporent sans un message, et la couverture annoncée devient « 100 % ».
3. **Publier une moyenne mobile incomplète.** En janvier 2023, une moyenne de 12 mois porte sur **1** mois ; en avril, sur **4**. Sans la colonne `COUNT(*) OVER` (même cadre), la courbe est lisse et fausse sur les douze premiers points.

4. **Remplacer un NULL par zéro.** Un mois sans objectif n'est pas un mois d'objectif nul : publier 0 produit un taux de réalisation infini ou nul selon la formule. NULL signifie « inconnu » — et « inconnu », dans un rapport, se déclare.
5. **Comparer deux mois sans regarder la saison.** Mars (433 645 399 FCFA par mois) contre juillet (262 079 192) donne **-40 %** sur un socle en croissance : l'écart est un écart de calendrier.

9. Bonnes pratiques professionnelles

1. **Vérifier la couverture de la période avant tout calcul.** Le MIN/MAX des dates, le nombre de jours ou de mois couverts, la comparaison au calendrier de référence : trois requêtes qui coûtent quelques secondes et qui conditionnent la validité de tout le reste.
2. **Écrire le périmètre dans le titre du rapport.** « Janvier—août 2026 contre janvier—août 2025 : +15,4 % » — le titre porte le périmètre, et personne ne peut plus confondre les deux lectures.
3. **Toujours joindre la saison à la comparaison.** Un écart annuel de **+15,4 %** se lit par rapport à la progression des années précédentes sur la même fenêtre (**+15,3 %**, **+18,8 %**), pas dans l'absolu.
4. **Compter les lignes de chaque fenêtre.** Un cadre (ROWS BETWEEN ...) et un COUNT(*) OVER avec le même cadre : la valeur et sa fiabilité dans la même ligne de résultat.
5. **Déclarer les trous dans la note de méthode.** Deux mois sans objectif, douze mois sans antériorité : ce sont des informations sur la base, et elles doivent figurer dans le rapport — pas être lissées.

10. Exercice guidé

Situation. Le directeur général reçoit un rapport intitulé « **2026 : l'activité recule de 28,9 %** », qui compare le chiffre d'affaires 2026 au chiffre d'affaires 2025. Il appelle le service analyse, très inquiet. Vous avez 15 minutes pour produire une réponse écrite avant son comité.

Consigne.

1. Reproduisez le chiffre du rapport. (2 pts)
2. Écrivez la comparaison à périmètre égal, avec le titre que vous proposeriez. (3 pts)
3. Ajoutez la référence des deux années précédentes sur la **même** fenêtre, et dites ce que la comparaison des trois progressions apporte. (3 pts)
4. Le DG demande : « pourquoi août 2026 peut-il être comparé à août 2025, alors qu'août 2025 avait **31** jours ? » Répondez en une phrase. (2 pts)
5. Citez la requête de contrôle qui prouve qu'aucun mois de ventes ne manque dans la base. (2 pts)

11. Exercices autonomes

E1 — Le tableau de suivi des objectifs, trous compris (40 min). Produisez, pour le magasin 4, un suivi mensuel 2024 : réalisé, objectif, taux de réalisation, et une colonne état qui distingue les trois cas (complet, objectif manquant, vente manquante). Contrainte : la jointure sera **externe**, et le rapport devra afficher les **2** mois sans objectif — pas les supprimer.

E2 — Deux moyennes mobiles, une conclusion (35 min). Pour le réseau entier, produisez la série mensuelle avec la moyenne mobile de 3 mois et celle de 12 mois, **chacune accompagnée du nombre de lignes de sa fenêtre**. Puis répondez en trois lignes : que disent les deux courbes sur le dernier trimestre, et pourquoi ne disent-elles pas la même chose ?

12. Correction détaillée

Exercice guidé.

1. **Reproduire le chiffre.** `SELECT YEAR(date_vente), SUM(montant_ttc) ... GROUP BY 1` donne **3 360 553 372 FCFA** pour 2026 et **4 724 700 286 FCFA** pour 2025, soit **-28,9 %**. Le calcul est juste ; la comparaison ne l'est pas.
2. **Le périmètre égal.** Même requête, avec `WHERE MONTH(date_vente) <= 8` avant d'agréger par année : **3 360 553 372** contre **2 913 055 394**, soit **+15,4 %**. Titre proposé : « **Janvier—août 2026 contre janvier—août 2025 : +15,4 %** » — le périmètre est dans le titre, la question de périmètre est close.
3. **Les deux années de référence.** Sur la même fenêtre : **+18,8 %** (2024/2023) puis **+15,3 %** (2025/2024) puis **+15,4 %** (2026/2025). La progression est **stable**, très légèrement décélérante au milieu de la période. C'est le seul énoncé qui survive à une question du comité.
4. **Les 31 jours d'août.** Parce que la comparaison porte sur **les deux années au même mois**, donc sur deux mois d'août complets de 31 jours : la longueur du mois est identique des deux côtés, comme le nombre de jours fériés.
5. **Le contrôle de complétude.** 44 lignes mensuelles attendues entre 2023-01 et 2026-08 ; la requête `SELECT COUNT(DISTINCT date_trunc('month', date_vente)) FROM ventes WHERE NOT est_retour` en rend **44** — et le croisement avec le calendrier (1 339 jours) confirme la couverture.

E1 (le suivi du magasin 4). La structure attendue est le `FULL OUTER JOIN` du §5.2, avec le `CASE` à trois états. Les deux lignes sans objectif portent **42 514 372** et **51 235 118 FCFA** de réalisé, et le taux de réalisation doit y être `NULL` — pas 0, pas « 100 % ». Total des ventes sans objectif : **93 749 490 FCFA**.

E2 (les deux mobiles). Au dernier mois du socle, la moyenne de 3 mois vaut **349 415 147 FCFA** et celle de 12 mois **431 016 522 FCFA** : la première baisse (elle a « oublié » un mois creux sorti du cadre), la seconde monte (elle lisse l'année entière). Les deux sont justes ; la première décrit le trimestre, la seconde la trajectoire. Un rapport qui n'en publie qu'une choisit à la place du lecteur — et le plus souvent celle qui confirme ce qu'il espérait.

13. Mini-projet de chapitre

La note de conjoncture d'un magasin, 2 pages. Choisissez un magasin du réseau et produisez sa note de conjoncture sur le dernier trimestre clos (juin—août 2026), avec le périmètre annoncé et la saison en regard.

Livrables et barème.

Critère	Ce qui est vérifié	Points
Le tableau mensuel	mois, CA, cumul, moyenne mobile de 3 mois, avec le compteur de fenêtre	6
La comparaison annuelle	les trois mois contre les trois mêmes mois de 2025, variations exactes	5
Le contrôle de complétude	la requête de couverture (calendrier, 1 339 jours) et sa sortie	4
La conclusion	une phrase, périmètre annoncé, saison citée, et une limite	3
Total		18

14. Résumé du chapitre

Le temps en SQL se traite en trois gestes : **fabriquer** l'axe (calendrier ou `generate_series`, ici **1 339** jours qui concordent), **joindre** les données à cet axe en **externe** pour que les trous apparaissent (les **2** mois sans objectif du magasin 4, **93 749 490 FCFA** de ventes sans référence), et

compter les lignes de chaque fenêtre pour savoir si elle est pleine. La comparaison temporelle exige un **périmètre** explicite : 2026 contre 2025 vaut **-28,9 %** ; janvier—août contre janvier—août vaut **+15,4 %** — même base, deux conclusions, une seule défendable. Les moyennes mobiles lissent le bruit mensuel (**-12,3 %** à **+27,1 %** d'un mois sur l'autre) mais ne valent qu'avec leur compteur de lignes. Enfin la **saison** doit être connue avant toute interprétation : **×3,2** entre le lundi et le dimanche, **×1,65** entre mars et juillet — plus grand que la croissance annuelle du socle (**+16,8 %**), ce qui dit l'essentiel : sur cette activité, le calendrier pèse plus que la tendance.

15. À retenir

1. **Un axe de temps complet est la condition de tout rapport temporel.** Sans calendrier, les périodes sans donnée n'existent pas — et l'absence se lit comme une absence de problème.
2. **Toute jointure vers une référence se fait en externe et se vérifie par un comptage.** La jointure interne supprime en silence.
3. **Aucune comparaison sans périmètre écrit.** « 2026 contre 2025 » n'est pas un périmètre ; « janvier—août contre janvier—août » en est un.
4. **Une fenêtre glissante se publie avec son compteur.** Une moyenne de 12 mois calculée sur 4 mois est une moyenne de 4 mois qui porte un mauvais nom.
5. **La saison se connaît avant l'interprétation.** Sur ce socle, le calendrier explique plus d'écart que la tendance.

À retenir. Le chiffre qui résume le chapitre n'est pas **+15,4 %** : c'est la paire **-28,9 % / +15,4 %** — le même socle, la même année, la même question, et une phrase dans le titre qui décide laquelle des deux est publiée.

À retenir. Un trou déclaré est une information ; un trou supprimé est une erreur. Les **2** mois du magasin 4 sont le seul endroit du socle où la réponse honnête est « je ne sais pas ».

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien de jours comptent le calendrier du socle, et combien en rend `generate_series` sur la même période ?
2. Combien de mois de ventes manque-t-il sur le socle ? Combien de mois d'objectifs ?
3. Que publie la comparaison du total 2026 au total 2025, et que publie la comparaison à périmètre égal ?
4. Pourquoi une jointure interne sur `objectif_mois` est-elle dangereuse ici ?
5. Que signifie `lignes_12 = 4` en avril 2023 dans la requête de moyenne mobile ?
6. Quel rapport y a-t-il entre le lundi et le dimanche sur ce socle ?
7. Quel rapport entre mars et juillet, et que faut-il en conclure pour un rapport trimestriel ?
8. Combien de mois du socle ont un équivalent un an plus tôt ?
9. Quelle est la progression annuelle moyenne sur ces mois comparables ?
10. Que doit contenir le titre d'un rapport de comparaison temporelle ?

Réponses. 1. **1 339** et **1 339** : les deux axes concordent. 2. Aucun mois de ventes (**0** sur **44**) ; **2** mois d'objectifs (**magasin 4**, février et mars 2024). 3. **-28,9 %** (2026 n'a que huit mois) ; **+15,4 %** à périmètre égal. 4. Parce qu'elle **supprime** les deux mois sans objectif — **93 749 490 FCFA** de ventes — sans erreur ni `NULL`. 5. Que la « moyenne de 12 mois » ne porte que sur **4** lignes : la

fenêtre n'est pas pleine. 6. **3,2** (**13 872 370** contre **4 322 830** FCFA par jour de vente). 7. **1,65** (**433 645 399** contre **262 079 192** FCFA par mois) : un premier trimestre ne se compare pas à un troisième sans correction. 8. **32** des **44** mois. 9. **+16,8 %**. 10. Le **périmètre** : « janvier—août 2026 contre janvier—août 2025 ».

Module M11.C04 — Cohortes, rétention et récurrence : RFM, et le client qui n'a pas de nom

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C03 (fenêtres, cadres, temps) ; M09.C04 (segmentation client) ; M08.C06 (indicateurs de fidélité).

L'idée du chapitre. Il existe deux façons de mesurer si les clients reviennent, et la plus répandue est la mauvaise. La lecture **calendaire** demande « combien des clients actifs en juin sont revenus en juillet ? » — sur le socle, **13,2 % (432 des 3 276 clients de juin)**. La lecture **par cohorte** demande « quels clients dont c'était le premier achat en juin sont revenus en juillet ? » — **22,7 %**, mais sur **22 clients** seulement. Les deux nombres prétendent décrire la même chose et n'ont ni la même base, ni la même signification : le premier est stable et mesure le **mélange** du fichier, le second est juste et trop petit pour être publié. Ce chapitre apprend à construire la **cohorte**, à lire la matrice en **triangle**, et à pondérer les taux avant de les commenter. Puis il assemble les trois mesures que tout le monde utilise sans les définir — récence, fréquence, montant — et montre ce qu'elles deviennent quand **18,3 %** du chiffre d'affaires est porté par un client qui n'a pas de nom.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). 23 497 clients de vente, 44 cohortes mensuelles, 237 191 lignes nettes, 15 595 154 955 FCFA, du 01/01/2023 au 31/08/2026. Toutes les sorties publiées ici sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Construire une cohorte** sur l'événement métier qui compte — la **première commande** — et non sur la date d'inscription du référentiel.
2. **Produire une matrice de rétention** cohorte × ancienneté, et **reconnaître sa forme** : un triangle, pas un rectangle.
3. **Calculer une rétention pondérée** par la taille des cohortes, et expliquer pourquoi la moyenne simple des taux est fausse.
4. **Distinguer trois questions voisines** : la rétention par cohorte, la récurrence calendaire et l'activité d'un mois.
5. **Segmenter en RFM** avec NTILE, départage explicite, et nommer les segments sans inventer de seuils « métier ».
6. **Traiter le client non identifié** : le mesurer, décider, et l'écrire dans la note de méthode.

2. Pourquoi cette notion est importante

Le socle permet de répondre à une question que le comité de direction posera tôt ou tard : **nos clients reviennent-ils ?** Il faut choisir un chiffre. Les candidats :

Ce qui est calculé	Chiffre	Ce que ça mesure vraiment
Clients actifs en juin 2025 revenus en juillet 2025	13,2 % (432 / 3 276)	le renouvellement du fichier actif , toutes cohortes mêlées
Clients dont juin 2025 est le premier achat, revenus en juillet	22,7 % (5 / 22)	la fidélité des nouveaux — sur un effectif trop faible pour conclure
Rétention moyenne pondérée au premier mois suivant, toutes cohortes	15,8 %	la fidélité de la base entière, chaque client pesant une fois

Le troisième chiffre est le seul qui soit à la fois **juste** et **publiable**. Le premier est juste aussi, mais il ne répond pas à la question posée : il dépend du **mélange** de cohortes du mois, et il resterait autour de **13 %** même si plus aucun nouveau client ne revenait jamais. Le deuxième est juste et inutilisable : **22** clients.

Ce chapitre existe parce que ces trois chiffres sortent de trois requêtes qui se ressemblent, que rien dans le résultat n’indique laquelle on a écrite, et que le plus gros des trois (le plus flatteur) est celui qu’on retient.

3. Explication simple — la promotion d’une année, et le verre à moitié plein

Une école veut savoir si ses élèves de première année poursuivent en deuxième. Deux méthodes.

La méthode fausse. En septembre, on compte tous les élèves présents : premières, deuxièmes, troisièmes années confondues. On recompte les mêmes en octobre. On annonce « **93 %** de nos élèves restent ». Le chiffre est exact et ne dit rien : la masse des troisièmes années, qui ne peuvent pas partir avant la fin du cycle, écrase le résultat. C’est la lecture calendaire.

La méthode juste. On prend **une** promotion, celle entrée en septembre. On la suit : combien sont là en octobre, en novembre, un an plus tard. On obtient une courbe. Cette courbe, comparée à celle des promotions entrantes des deux années précédentes, dit si l’école retient mieux qu’avant. C’est la cohorte.

Ce qu’aucune des deux ne donne. La lecture calendaire ne dit pas la qualité du service ; la lecture par cohorte ne dit presque rien pour la promotion entrée ce mois-ci, parce qu’elle est petite et récente. Ce qu’il faut, c’est **toutes les cohortes, pondérées** : chaque élève pèse une fois, quelle que soit son année d’entrée. Sur le socle, cela donne **15,8 %** — et ce chiffre-là ne bouge plus quand le mélange change.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
cohorte	cohort	Un groupe de clients partageant un événement de départ daté — ici le mois de leur première commande.
ancienneté	cohort age	Le nombre de mois écoulés depuis l’événement de départ ; noté M+1, M+3, M+12.
matrice rétention	de retention matrix	Le tableau cohortes × anciennetés : chaque ligne est une cohorte, chaque colonne une ancienneté.
rétention	retention	La part d’une cohorte revenue au moins une fois pendant l’ancienneté considérée.
récurrence	recurrence	La part des clients actifs un mois donné qui reviennent le mois suivant — toutes cohortes mêlées.
pondération	weighting	Donner à chaque cohorte un poids proportionnel à sa taille, et non le même poids à toutes.

Terme (français)	Équivalent anglais	Définition en une phrase
récence	recency	Le délai écoulé depuis le dernier achat d'un client.
fréquence	frequency	Le nombre d'achats (ou de tickets) d'un client sur la période.
montant	monetary value	Le chiffre d'affaires attribué au client sur la période.
RFM	recency, frequency, monetary	Les trois mesures réunies, chacune découpée en tiers, pour segmenter le fichier client.
segment	segment	Le croisement des trois découpages : ici 133, 233, 333 — trois chiffres, de 1 à 3.
client identifié	non anonymous customer / default member	Un identifiant technique qui agrège des achats sans porteur : ici le client 0 .

Définition. Une **cohorte** est un ensemble de clients réunis par un **événement daté** commun — et l'événement doit être choisi avec soin. Le référentiel du socle contient une date d'inscription qui commence en **2016**, alors que la première vente observée n'est jamais antérieure à janvier 2023 : les clients existaient avant les données. Une cohorte d'**inscription** décrirait donc un autre monde que la cohorte de **première commande**. Règle : la cohorte est datée sur l'événement que le rapport mesure.

Définition. La **rétenion** est la part d'une cohorte encore active après N périodes. Elle exige donc **deux** définitions explicites : ce qu'est « active » (ici : au moins un ticket non retourné dans le mois) et ce qu'est « N » (ici : des mois complets, pas des fenêtres de 30 jours). Deux rapports qui ne partagent pas ces deux définitions ne peuvent pas comparer leurs taux.

5. Cours approfondi

5.1 La cohorte d'entrée : 44 lignes, une seule par client

```
WITH p AS (SELECT id_client, date_trunc('month', MIN(date_vente)) AS co
           FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT co, COUNT(*) AS taille FROM p GROUP BY 1 ORDER BY 1;
```

44 lignes, du 2023-01 au 2026-08. La première cohorte contient **3 593** clients, la dernière (**2026-08**) exactement **3**. Ce déséquilibre n'est pas une anomalie de la base : c'est le résultat attendu d'une période d'observation qui commence. Tous les clients qui étaient déjà là en janvier 2023 sont comptés comme « entrants de janvier 2023 » — on dit que la série est **tronquée à gauche**.

Attention. La troncature à gauche ne se corrige pas : on ne peut pas inventer les achats antérieurs au 01/01/2023. Elle se **déclare**. Sans cette phrase dans la note de méthode, le lecteur croira que la base a gagné **3 593** clients en un mois et en a perdu **3 590** en trois ans et demi.

5.2 La matrice : un triangle, jamais un rectangle

La matrice croise chaque cohorte avec chaque ancienneté. Extrait des six premières cohortes, sur six anciennetés :

Cohorte	Taille	M+0	M+1	M+2	M+3	M+4	M+5
2023-01	3 593	100,0 %	14,6 %	18,3 %	17,5 %	15,0 %	12,9 %
2023-02	2 817	100,0 %	19,1 %	17,8 %	15,9 %	12,6 %	11,5 %
2023-03	3 137	100,0 %	16,2 %	15,0 %	11,3 %	10,5 %	13,7 %

Cohorte	Taille	M+0	M+1	M+2	M+3	M+4	M+5
2023-04	2 389	100,0 %	14,6 %	13,3 %	11,3 %	13,0 %	15,9 %
2023-05	1 653	100,0 %	11,6 %	11,4 %	13,3 %	16,0 %	18,4 %
2023-06	1 239	100,0 %	11,0 %	13,6 %	14,9 %	19,9 %	17,5 %

Deux choses sautent aux yeux, et aucune n'est un taux de fidélité.

Le triangle. Les **44** cohortes n'ont pas le même recul : **32** sont observables jusqu'à M+12, **20** jusqu'à M+24. La matrice a donc la forme d'un escalier, et la case M+24 de la cohorte de 2026-08 est **impossible** — pas vide par manque de données, mais par manque de **temps**. Un tableau qui la remplirait par zéro publierait une chute que personne n'a mesurée.

Le mélange des cohortes. Chaque ligne a sa taille : **3 593** en tête, **1 239** six mois plus tard. Une rétention « moyenne » calculée comme la moyenne des taux donnerait donc le même poids à ces deux populations — c'est la faute du \$5.3.

Retention par cohorte de première commande, en %													
	M+0	M+1	M+2	M+3	M+4	M+5	M+6	M+7	M+8	M+9	M+10	M+11	M+12
2023-01 (3593)	100	15	18	18	15	13	12	12	16	19	19	18	17
	100	19	18	16	13	12	13	16	18	18	18	16	15
	100	16	15	11	10	14	16	18	18	19	16	16	21
2023-04 (2389)	100	15	13	11	13	16	18	20	18	17	17	20	18
	100	12	11	13	16	18	18	18	16	15	19	21	15
	100	11	14	15	20	18	17	15	16	19	18	16	14
2023-07 (1017)	100	11	16	19	16	20	18	17	20	18	14	11	14
	100	14	17	19	18	17	17	20	18	17	14	12	14
	100	17	19	17	16	17	18	20	16	14	14	14	15
2023-10 (1074)	100	19	16	17	17	20	18	15	14	13	14	18	20
	100	21	18	14	20	19	17	12	14	13	17	18	20
	100	18	17	20	20	16	13	13	10	16	22	20	20
2024-01 (474)	100	19	23	22	14	12	15	16	17	20	23	17	18
	100	20	14	16	12	13	14	16	19	21	17	15	16
	100	16	12	14	11	14	17	22	20	20	17	17	23
2024-04 (324)	100	17	13	12	12	15	24	22	21	21	15	23	20
	100	13	15	16	12	17	24	20	21	18	19	19	15
	100	15	16	18	22	19	24	15	14	32	12	21	12
2024-07 (109)	100	10	16	22	18	23	13	17	19	24	17	10	10
	100	17	19	20	23	16	21	16	19	16	16	15	14
	100	26	12	20	26	15	22	18	21	12	11	10	18
2024-10 (148)	100	18	20	20	18	20	18	14	16	16	13	16	22
	100	17	21	18	21	22	17	17	10	11	25	22	20
	100	24	12	21	25	24	11	11	17	21	18	19	14
2025-01 (62)	100	13	16	19	14		13	13	18	13	29	24	14
	100	22	19	22		14	14	16	32	24	32	16	27
	100	16	14		16	20	16	16	26	22	16	31	22
2025-04 (43)	100	16	21	16	14	14	26	26	21	28	21	19	21
	100	16	12	12	12	12	31	34	19	19	19	28	25
	100	23	18		23	18	18	23	18	18	14	18	23
2025-07 (13)	100	31	15	23		23	15			31	38	15	
	100	25	38	25	31	19	12	31		12	19		25
	100	29		12	12	12	29	29	12	18	18	24	
2025-10 (9)	100	11	11	33	33	22	11			11			
	100	38	12	25	25	12	12		12				
	100	40	10	30	30	10	20	10	20				
2026-01 (8)	100	25	50	12	12	12		12					
	100	50	17		50		17						
	100			67		33							
2026-04 (4)	100												
	100												
	100												
2026-07 (1)	100												
	100	100											
	100												

Une matrice de retention s'arrete en triangle

5.3 Le piège des deux lectures

Le même mois, deux questions, deux réponses :

```
-- lecture calendaire : les actifs de juin, revenus en juillet
WITH a AS (SELECT DISTINCT id_client, date_trunc('month', date_vente) AS m
FROM ventes WHERE NOT est_retour)
SELECT COUNT(*) AS base_juin,
COUNT(*) FILTER (WHERE id_client IN (SELECT id_client FROM a WHERE m = DATE '2025-07-01'))
AS revenus
FROM a WHERE m = DATE '2025-06-01';
```

```
base_juin | revenus
3276      | 432
```

13,2 %. C'est un taux de **récence**, pas de rétention : il mesure la proportion des clients actifs d'un mois qui reviennent le mois suivant, quelle que soit leur date d'entrée. Il est utile (c'est la « respiration » du fichier) et il ne répond pas à « nos nouveaux clients reviennent-ils ? ».

Pour répondre à cette question-là, il faut la même paire de mois vue **par la cohorte** :

```
WITH p AS (SELECT id_client, date_trunc('month', MIN(date_vente)) AS co
  FROM ventes WHERE NOT est_retour GROUP BY 1),
  a AS (SELECT DISTINCT id_client, date_trunc('month', date_vente) AS m
  FROM ventes WHERE NOT est_retour),
  c AS (SELECT id_client FROM p WHERE co = DATE '2025-06-01')
SELECT (SELECT COUNT(*) FROM c) AS base,
  (SELECT COUNT(*) FROM c WHERE id_client IN
    (SELECT id_client FROM a WHERE m = DATE '2025-07-01')) AS revenus;
```

```
base | revenus
22   | 5
```

22,7 %, sur **22** clients. Le chiffre est juste et n'est pas publiable : à cette taille, un client de plus ou de moins le fait bouger de 4,5 points.

Le troisième chiffre est celui qu'on publie — la **moyenne pondérée de toutes les cohortes** : **15,8 %** au premier mois, chaque client pesant une fois. Le contrôle qui justifie la pondération tient en une requête : sur les **23** cohortes d'au moins **100** clients, la moyenne **simple** des taux vaut **16,4 %** et la moyenne **pondérée** **15,8 %**. L'écart n'est pas énorme, mais il existe — et il vient de là : dans la moyenne simple, une cohorte de **22** clients pèse autant qu'une cohorte de **3 593**.

Définition. La **pondération** consiste à donner à chaque cohorte un poids proportionnel à son effectif. Un taux moyen pondéré est la somme des clients revenus divisée par la somme des clients de départ — jamais la moyenne des taux. Le calcul direct est d'ailleurs plus simple : `SUM(actifs) / SUM(effectif)`.

Dans les faits. La différence entre **16,4 %** et **15,8 %** paraît faible, et elle est **signée** : les petites cohortes récentes ont des taux plus élevés (elles contiennent des primo-acheteurs encore chauds), donc la moyenne simple **surestime** systématiquement. Sur un fichier client en forte croissance, l'écart se creuse — et il va toujours dans le sens flatteur.

5.4 Une rétention qui ne baisse pas

Le résultat le plus important du chapitre est peut-être le plus décevant :

Ancienneté	M+1	M+3	M+6	M+12	M+24
Rétention pondérée	15,8 %	15,4 %	15,9 %	17,2 %	18,2 %

La courbe est **plate**, et elle remonte même légèrement. La cohorte de janvier 2023 vue dans son intégralité le confirme : de M+1 à M+43, elle reste entre **11,5 %** et **22,6 %**, sans tendance à la baisse, et se termine à **16,9 %**.

L'interprétation est un exercice de rigueur, pas d'enthousiasme. Le socle ne raconte **pas** une entreprise qui perd ses clients chaque mois. Il raconte une base composée de deux populations : une minorité d'acheteurs réguliers, présents presque tous les mois, et une majorité de clients de passage — le socle compte **3 316** clients à **8** tickets et **64** clients dont toute l'histoire tient dans **un seul** mois. Le taux de **15,8 %** est le poids de la première population dans le fichier.

Attention. « 15,8 % de rétention » ne signifie pas que **84 %** des clients sont perdus. Le taux compte comme active toute personne ayant acheté **au moins une fois** dans le mois : il ne mesure pas l'intensité, et un client régulier qui saute un mois compte comme sorti. Avant de parler de perte, il faut une deuxième mesure — la **récence** du §5.6.

5.5 Le RFM : trois mesures, une décision

Le RFM réunit trois colonnes par client — **R**écence, **F**réquence, **M**ontant — et les découpe chacune en trois blocs de taille égale représentés par 1, 2, 3 :

```
WITH c AS (SELECT id_client, MAX(date_vente) AS derniere, COUNT(DISTINCT id_ticket) AS f,
                SUM(montant_ttc) AS m
            FROM ventes WHERE NOT est_retour GROUP BY 1),
    s AS (SELECT id_client,
                NTILE(3) OVER (ORDER BY derniere ASC, id_client) AS r,
                NTILE(3) OVER (ORDER BY f DESC, id_client) AS f3,
                NTILE(3) OVER (ORDER BY m DESC, id_client) AS m3,
                m
            FROM c)
SELECT CAST(r AS VARCHAR) || CAST(f3 AS VARCHAR) || CAST(m3 AS VARCHAR) AS segment, m FROM s;
```

NTILE(3) produit trois blocs de **7 833**, **7 832** et **7 832** clients sur les **23 497** du socle : c'est le découpage le plus égal possible, et il est exact. La lettre **1** va au **meilleur** tiers, donc la récence se trie en ordre **croissant** (dernier achat récent) alors que la fréquence et le montant se trient en ordre **décroissant**.

Trois résultats sur le socle :

- Le segment **133** — récents, peu fréquents, gros montants — réunit **2 151** clients et **4 937 755 609** FCFA, soit **31,7 %** du chiffre d'affaires net. C'est le plus gros segment par le montant, et il est **injouable** en l'état : il contient le client **0** (voir §5.7).
- Le tiers des **récents** (dernier achat entre le **18/06/2026** et le **31/08/2026**) porte **48,7 %** du chiffre d'affaires, contre **23,1 %** pour le tiers des anciens (dernier achat entre le **31/01/2023** et le **13/03/2026**).
- Le fichier se répartit donc en **27** segments possibles ($3 \times 3 \times 3$), dont la plupart restent vides ou minuscules : c'est normal, et un rapport qui n'en nomme que quatre ou cinq fait le bon choix.

Définition. Le **RFM** n'a pas de frontières intrinsèques : les coupures en trois blocs égaux sont un choix de méthode (égalité des effectifs), pas une vérité métier. D'autres entreprises coupent à des seuils absolus (« plus de **12** mois sans achat », « au moins **5** commandes »). Ce qui rend un RFM défendable n'est jamais le découpage : c'est le fait de **l'écrire**.

Conseil professionnel. Nommez les segments avec les bornes qui les définissent, jamais avec un adjectif. « Récents à faible fréquence, dernier achat depuis le 18/06/2026 » se vérifie ; « clients premium » ne se vérifie pas — et se discute pendant trois réunions.

5.6 Fréquence, récurrence, récence : les trois chiffres qui tiennent

Trois mesures complètent le RFM et se calculent sans segmentation :

Mesure	Requête	Résultat sur le socle
Récurrence	écart entre deux tickets consécutifs du même client	médiane 96 jours, moyenne 137,2 jours, sur 171 737 paires
Récence	délai depuis le dernier achat, au 31/08/2026	moyenne 156 jours, médiane 122 jours

Mesure	Requête	Résultat sur le socle
Inactivité	clients sans achat depuis plus d'un an	1 848 clients, 4,3 % du CA net

Deux enseignements. D'abord, l'écart systématique entre médiane et moyenne (**96** contre **137,2** jours ; **122** contre **156**) est encore l'effet des gros clients : la moyenne décrit une minorité, la médiane décrit le client courant. Ensuite, la règle « pas d'achat depuis plus d'un an = client perdu » donne **1 848** clients — un effectif de campagne de réactivation, avec des noms et des adresses, et seulement **4,3 %** du chiffre d'affaires à la clé. C'est précisément le genre d'arbitrage qu'un rapport doit rendre possible : le coût d'une campagne se compare à **4,3 %** du CA, pas à une intuition.

5.7 Le client 0 : 18,3 % du CA sans porteur

Sur **23 497** identifiants clients, un seul n'existe pas dans le référentiel : le client **0**. Il porte **42 658** lignes, **38 984** tickets et **2 852 612 447** FCFA — **18,3 %** du chiffre d'affaires net du socle.

Le traiter « à part » n'est pas une option esthétique : c'est ce qui décide de la conclusion. La question posée par l'évaluation du module — « **10 %** des clients font-ils **60 %** du chiffre d'affaires ? » — se répond ainsi :

Périmètre	Part du CA du dixième supérieur	Nombre de clients
Tous clients	38,1 %	2 350
Client 0 exclu	24,2 %	2 349

Dans les deux cas, la réponse est **non** — mais elle a deux valeurs différentes, et la note de méthode doit dire laquelle est publiée. Le client **0** appartient au dixième supérieur dans la lecture « tous clients » : c'est **un** identifiant qui pèse à lui seul plus que les neuf dixièmes du fichier.

Il y a plus grave, et c'est une leçon de reproductibilité : sans clé de départage explicite, NTILE et ROW_NUMBER **ne sont pas déterministes**. Le même recoupement RFM, exécuté **15** fois le 24/09/2026 sans ORDER BY ..., id_client, a situé le segment principal entre **2 718** et **2 738** clients selon l'ordonnancement interne du moteur. La cause est mesurable : **3 316** clients partagent exactement **8** tickets (**14,1 %** du fichier), donc la frontière des tiers tombe au milieu d'une égalité massive et le moteur choisit qui passe. Ce n'est pas un bug, c'est une requête incomplète.

Attention. Un rapport dont le chiffre change d'une exécution à l'autre n'est pas un rapport. Toute utilisation de NTILE, RANK ou ROW_NUMBER doit porter une clé de départage — ici id_client — et **la même** dans toutes les requêtes du rapport. Le contrôle se fait en comparant deux exécutions, pas en relisant la requête.

5.8 Ce que ce chapitre ne couvre pas

La **valeur vie client** (somme actualisée des marges futures) et la **prédiction de départ** exigent des modèles, traités en M14 et M19. Ici, tout est **descriptif** : on mesure ce qui a eu lieu, avec des définitions écrites. C'est déjà suffisant pour occuper une réunion entière.

6. Exemple concret — trois façons de répondre à la même question

Un responsable marketing demande : « combien de nos clients nous restent d'une année sur l'autre ? »

Réponse	Calcul	Valeur	Utilisable ?
Activité	clients actifs en juin 2025 revenus en juillet 2025	13,2 %	non : mesure le fichier, pas la fidélité
Cohorte du mois	cohorte de juin 2025, revenue en juillet	22,7 % sur 22 clients	non : effectif trop faible
Rétention pondérée	toutes cohortes, à M+12	17,2 %	oui, avec sa définition

La réponse à envoyer est la troisième, **accompagnée** de la phrase qui la rend lisible : « **17,2 %** des clients d'une cohorte achètent de nouveau douze mois après leur premier achat ; le taux est **plat** de M+1 à M+24 (**15,8 %** à **18,2 %**), ce qui décrit un noyau d'acheteurs réguliers dans une base majoritairement composée de clients de passage ». Trois chiffres, une phrase, aucune ambiguïté sur ce qui est mesuré.

7. Démonstration pas à pas — cinq étapes sur le socle

7.1 Étape 1 — la cohorte d'entrée

44 cohortes, de **3 593** clients (janvier 2023) à **3** (août 2026). Contrôle : la somme des tailles redonne les **23 497** clients du socle, chacun dans une seule cohorte.

7.2 Étape 2 — la matrice et son triangle

Le tableau du §5.2. Contrôle de forme : **32** cohortes tiennent jusqu'à M+12, **20** jusqu'à M+24. Les cellules au-delà ne sont pas nulles, elles sont **inexistantes**.

7.3 Étape 3 — les deux lectures, côte à côte

13,2 % (**432** des **3 276** clients actifs en juin 2025) en lecture calendaire ; **22,7 %** (**5** sur **22**) en lecture par cohorte. Puis la version publiable : **15,8 %** pondéré, contre **16,4 %** en moyenne simple sur les **23** cohortes d'au moins **100** clients.

7.4 Étape 4 — le RFM, avec le client 0

Les trois blocs de **7 833**, **7 832** et **7 832** clients ; le segment **133** à **2 151** clients et **31,7 %** du CA ; le tiers des récents à **48,7 %** du CA. Puis le retrait du client **0** : dixième supérieur à **38,1 %**, puis **24,2 %**.

7.5 Étape 5 — les mesures de contrôle

Récurrence médiane **96** jours ; récence médiane **122** jours ; **1 848** clients inactifs depuis plus d'un an (**4,3 %** du CA) ; **64** clients à un seul mois d'activité. Ce sont ces quatre chiffres qui permettent de qualifier le taux de rétention de **15,8 %** — sans eux, il n'est qu'un nombre.

8. Erreurs fréquentes

- Dater la cohorte sur l'inscription.** Le référentiel porte des dates de création à partir de **2016**, les ventes commencent en **2023** : une cohorte d'inscription ne décrit pas le même univers. La cohorte se date sur l'événement mesuré.
- Compter des clients sur une jointure.** La jointure de la cohorte de janvier 2023 (**3 593** clients) avec les **44** mois d'activité rend **30 090** lignes. Un `COUNT(*)` au lieu d'un `COUNT(DISTINCT id_client)` multiplie la base par l'historique — sans erreur, sans avertissement.

3. **Moyenner des taux sans pondérer. 16,4 %** au lieu de **15,8 %** : la moyenne simple donne le même poids à la cohorte de juin 2025 (22 clients) qu'à celle de janvier 2023 (3 593).
4. **Classer sans clé de départage.** NTILE sur une fréquence qui compte 3 316 clients à 8 tickets fait bouger le segment principal entre 2 718 et 2 738 clients d'une exécution à l'autre.
5. **Lire un taux de rétention comme un taux de fidélité. 15,8 %** au premier mois, **18,2 %** à M+24 : la courbe est plate, elle ne décrit pas une fuite mais la part des acheteurs réguliers.

9. Bonnes pratiques professionnelles

1. **Écrire la définition de « actif » en tête du rapport.** Un ticket non retourné dans le mois, retours exclus : ce choix change le taux, et il doit être visible avant le premier chiffre.
2. **Publier la taille de la base à côté de chaque taux.** 22,7 % sur 22 clients et 15,8 % sur 23 483 clients ne se lisent pas avec le même crédit ; un taux sans effectif est une rumeur.
3. **Pondérer, jamais moyenner.** $\text{SUM}(\text{revenus}) / \text{SUM}(\text{effectif})$ en une seule requête, et non la moyenne de 44 taux.
4. **Traiter le client non identifié comme un cas, pas comme un client.** Il est mesuré (18,3 % du CA), nommé, et exclu des classements — ou conservé avec la mention explicite que le classement est faussé.
5. **Figer les classements par une clé de départage.** ORDER BY ..., id_client partout, et la preuve par deux exécutions comparées : c'est le seul moyen d'être reproductible.

10. Exercice guidé

Situation. La direction commerciale veut lancer une campagne de réactivation. Elle vous demande : « combien de clients avons-nous perdus, et combien cela nous coûte-t-il ? »

Consigne.

1. Écrivez la requête qui définit un client perdu — et justifiez la borne choisie. (2 pts)
2. Donnez le nombre de clients concernés et leur part du chiffre d'affaires. (3 pts)
3. Calculez la récurrence médiane et la récence médiane du fichier, et dites laquelle des deux vous utiliseriez pour fixer le délai de la campagne, et pourquoi. (3 pts)
4. Un collègue propose de joindre le référentiel clients pour filtrer les « vrais » clients. Que change cette jointure sur le total du socle ? (2 pts)

11. Exercices autonomes

E1 — La matrice de rétention complète, et sa lecture (45 min). Produisez la matrice des 44 cohortes, de M+0 à M+24, avec la taille de chaque cohorte en première colonne. Puis répondez en cinq lignes : les cohortes récentes retiennent-elles mieux ou moins bien que celles de 2023 ? Quelles cases de la matrice ne sont pas observables, et pourquoi ?

E2 — Deux segments RFM, deux politiques (35 min). Construisez le RFM avec départage explicite, puis comparez deux segments : le plus gros par le montant et celui des clients anciens à forte fréquence. Pour chacun : effectif, part du CA, plage de récence. Terminez par une phrase de politique commerciale par segment — une seule, et vérifiable par les chiffres que vous venez de produire.

12. Correction détaillée

Exercice guidé.

1. **La borne.** Un an sans achat est la borne la plus défendable sur ce socle, parce que la récurrence observée est longue (médiane **96** jours, moyenne **137,2** jours) : douze mois représentent plusieurs cycles d'achat manqués. La requête est un `MAX(date_vente)` par client comparé au **31/08/2026**.
2. **L'effectif.** **1 848** clients, soit **4,3 %** du chiffre d'affaires net — **663 520 490** FCFA. La bonne phrase est : « **4,3 %** du CA est à la clé, pas plus » ; c'est ce chiffre qui compare la campagne à son coût.
3. **Quelle mesure choisir.** La récurrence médiane (**96** jours) décrit le rythme entre deux achats, la récence médiane (**122** jours) la distance au dernier achat. Pour fixer un délai de campagne, on part de la **récurrence** : elle est stable et ne dépend pas du jour d'observation, contrairement à la récence, qui est une mesure « au 31/08/2026 » et vieillit d'un jour par jour.
4. **La jointure au référentiel.** Elle **retire** le client **0** (absent du référentiel), donc **2 852 612 447** FCFA et **18,3 %** du CA net du périmètre analysé. C'est un choix légitime, à condition d'être écrit : sans cette phrase, le total du rapport ne coïncide plus avec celui du compte de résultat.

E1 (la matrice). Les cohortes de 2023 et 2024 ne se distinguent pas nettement : les taux de M+1 restent dans une fourchette étroite (de **11,0 %** à **19,1 %** sur les six premières). Les cases non observables sont celles des cohortes récentes au-delà de leur recul : **32** cohortes tiennent à M+12, **20** à M+24, et les colonnes lointaines se remplissent de la droite vers la gauche à mesure que le temps passe. Une matrice de rétention se lit toujours **en triangle**, et sa lecture doit nommer la pointe comme « pas encore observable ».

E2 (deux segments). Le plus gros par le montant est **133** : **2 151** clients, **4 937 755 609** FCFA, **31,7 %** du CA — il contient le client **0** et doit être rejoué sans lui avant toute décision. Le tiers des **récents** (dernier achat entre le **18/06/2026** et le **31/08/2026**) porte **48,7 %** du CA, contre **23,1 %** pour le tiers des **anciens** (dernier achat entre le **31/01/2023** et le **13/03/2026**). Politique vérifiable : les récents ont un poids économique immédiat (relance de panier), les anciens un enjeu de réactivation borné par les **4,3 %** du \$5.6.

13. Mini-projet de chapitre

La note de fidélité de la cellule commerciale, 2 pages. À partir du socle, produisez la note de fidélité du parc client au 31/08/2026 : matrice de rétention, rétention pondérée à M+1, M+12 et M+24, répartition RFM, et la question du client non identifié.

Livrables et barème.

Critère	Ce qui est vérifié	Points
La matrice	44 cohortes, taille en première colonne, cases non observables marquées	6
La rétention pondérée	trois anciennetés, avec l'effectif de chaque calcul	5
Le RFM	trois blocs égaux, départage explicite, deux segments nommés avec leurs bornes	4
La note de méthode	définition d'« actif », traitement du client 0, troncature à gauche déclarée	3
Total		18

14. Résumé du chapitre

Mesurer la fidélité, c'est d'abord **choisir une cohorte** : l'événement de départ est la **première commande** — la date d'inscription du référentiel commence en **2016** et ne décrit pas le même

univers. La matrice de rétention se lit en **triangle** : **32** des **44** cohortes ont **12** mois de recul, **20** en ont **24**, et les cases au-delà ne sont pas vides, elles sont impossibles. Trois lectures se ressemblent et ne disent pas la même chose : la **récurrence** calendaire (**13,2 %**, **432** des **3 276** clients actifs en juin 2025), la rétention d'**une** cohorte (**22,7 %**, sur **22** clients) et la rétention **pondérée** (**15,8 %** à M+1, **17,2 %** à M+12, **18,2 %** à M+24) — seule la troisième est publiable. La courbe **plate** décrit un noyau d'acheteurs réguliers, pas une fuite. Le **RFM** répartit les **23 497** clients en trois blocs de **7 833**, **7 832** et **7 832**, place le tiers des récents à **48,7 %** du CA et fait apparaître le segment **133** à **31,7 %** — qui contient le client **0**, **18,3 %** du CA à lui seul. Et la leçon de reproductibilité se mesure : sans départage explicite, NTILE a fait varier le segment principal entre **2 718** et **2 738** clients sur **15** exécutions.

15. À retenir

1. **Une cohorte se date sur l'événement mesuré.** L'inscription n'est pas la première commande.
2. **Un taux sans effectif ne se publie pas.** **22,7 %** et **15,8 %** ne sont pas comparables sans dire qu'ils portent sur **22** et **23 483** clients.
3. **On pondère, on ne moyenne pas.** $\text{SUM}(\text{revenus}) / \text{SUM}(\text{effectif})$, jamais la moyenne des taux.
4. **Une courbe de rétention plate n'est pas une bonne nouvelle : c'est une description.** Le socle ne perd pas ses clients, il en garde un noyau d'environ **16 %**.
5. **Un classement sans départage n'est pas reproductible.** **3 316** clients à **8** tickets suffisent à rendre une frontière de tiers instable.
6. **Le client non identifié décide de la conclusion.** **18,3 %** du CA, **38,1 %** ou **24,2 %** selon qu'on le garde ou non.

À retenir. Le chapitre tient dans une comparaison : **13,2 %** de récurrence calendaire contre **15,8 %** de rétention pondérée. Le premier chiffre est plus facile à produire, plus stable, et il ne répond pas à la question. Le second demande une cohorte, une pondération et une phrase de méthode — et c'est le seul qu'on peut défendre devant un comité.

À retenir. Le socle raconte une entreprise dont **1 848** clients n'ont pas acheté depuis plus d'un an et qui ne représentent que **4,3 %** du chiffre d'affaires : la fidélité se décide sur des ordres de grandeur, et les ordres de grandeur s'obtiennent en pondérant, pas en devinant.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien de cohortes compte le socle, et quelle est la taille de la première et de la dernière ?
2. Pourquoi la matrice de rétention a-t-elle la forme d'un triangle ?
3. Que vaut la rétention pondérée à M+1, et que vaut la moyenne simple des taux sur les cohortes d'au moins 100 clients ?
4. Pourquoi **22,7 %** n'est-il pas le bon chiffre à publier, alors qu'il est juste ?
5. Que mesure la lecture calendaire, si elle ne mesure pas la fidélité ?
6. En combien de blocs NTILE(3) découpe-t-il **23 497** clients, et de quelles tailles ?
7. Quelle part du CA porte le tiers des clients récents ?
8. Pourquoi faut-il une clé de départage dans un NTILE, et quel signe le montre sur ce socle ?
9. Combien de clients n'ont pas acheté depuis plus d'un an, et quelle part du CA représentent-ils ?

10. Quelle est la part du chiffre d'affaires du dixième supérieur des clients, avec et sans le client 0 ?

Réponses. 1. **44** cohortes ; **3 593** clients en janvier 2023, **3** en août 2026. 2. Parce que les cohortes récentes n'ont pas le **recul** nécessaire : **32** cohortes atteignent M+12, **20** atteignent M+24. 3. **15,8 %** pondérée contre **16,4 %** en moyenne simple. 4. Parce qu'il porte sur **22** clients : l'effectif est trop faible pour qu'un taux soit stable. 5. La **récurrence** du fichier actif, toutes cohortes mêlées — **13,2 %** de juin à juillet 2025. 6. Trois blocs : **7 833**, **7 832** et **7 832** clients. 7. **48,7 %** du CA net. 8. Parce que **3 316** clients partagent exactement **8** tickets (**14,1 %** du fichier) : sans clé explicite, le segment principal a varié de **2 718** à **2 738** clients sur **15** exécutions. 9. **1 848** clients, **4,3 %** du CA net. 10. **38,1 %** tous clients confondus, **24,2 %** hors client 0.

Module M11.C05 — Requêtes avancées : CTE, tableau croisé, `UNNEST`, `QUALIFY`, récapitulatifs

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C04 (fenêtres, cadres, temps, cohortes) ; M07.C05 (jointures) ; M05.C04 (tableau croisé dans un tableur).

L'idée du chapitre. Une requête longue n'est pas une requête compliquée : c'est une requête dont on a nommé les étapes. Ce chapitre présente les six outils qui font passer d'un calcul à un **rapport** : la table d'expression commune (CTE) qui nomme, le **tableau croisé** qui met les mois en colonnes, `QUALIFY` qui classe et filtre dans la même requête, `UNNEST` qui inverse le grain, les **récapitulatifs multi-niveaux** qui rendent les sous-totaux dans une seule passe, et la **réursion**, précieuse pour les hiérarchies et inutile pour les calendriers. Et parce qu'aucun de ces outils ne remplace une donnée propre, le chapitre commence par l'exemple qui le prouve : le référentiel produits du socle écrit ses catégories de **16** façons pour **7** familles réelles, et un classement lancé sur les libellés bruts rend **16** lignes de réponse pour **7** rayons.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). 154 produits, 7 familles après repliement, 5 modes de paiement, 3 canaux, 145 212 tickets, 15 595 154 955 FCFA. Sorties de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Découper une requête longue en étapes nommées** avec `WITH`, et réutiliser la même étape plusieurs fois.
2. **Replier un libellé** sans perdre d'information : casse, espaces, accents, pluriel — et **mesurer** ce que chaque repliement change au classement.
3. **Croiser deux dimensions** avec `PIVOT` et avec l'agrégat conditionnel (`SUM(...)` `FILTER`), en sachant dans quel cas chacun est préférable.
4. **Classer et filtrer dans la même requête** avec `QUALIFY`, sans sous-requête enveloppante.
5. **Inverser un grain** avec `UNNEST`, en redonnant la définition de la ligne obtenue.
6. **Produire un récapitulatif multi-niveaux** (`ROLLUP`, `CUBE`, `GROUPING SETS`) et **distinguer un sous-total d'un zéro** avec `GROUPING()`.
7. **Employer la réursion** là où elle sert, et **écrire la version simple** là où elle ne sert pas.

2. Pourquoi cette notion est importante

Un rapport de direction ne demande pas « le chiffre d'affaires par mode de paiement » : il demande **le tableau** — les modes en colonnes, les années en lignes, les sous-totaux, et une ligne de total. En SQL brut, cette demande produit une requête d'une trentaine de lignes et de trois requêtes collées à la main dans un tableur. Les outils de ce chapitre font ce travail **dans le moteur**, où le contrôle est possible.

Le chapitre porte aussi un avertissement, et il est mesuré. Le référentiel produits du socle contient **154** produits répartis en **16** libellés de catégorie qui n'écrivent que **7** familles réelles :

la casse, les espaces finaux, les accents et le pluriel font le reste. Sur ces libellés bruts, un « meilleur produit par catégorie » rend **16** lignes, dont trois s'appellent Matériaux ; sur les familles repliées, il en rend **7**. Le même catalogue, la même question, deux rapports incompatibles — et c'est le repliement du libellé, une étape nommée dans une CTE, qui décide lequel est juste.

Enfin, ces outils ont un **coût de portabilité** : `QUALIFY` n'existe pas en PostgreSQL, `PIVOT` ne s'écrit pas de la même façon en SQL Server, `UNNEST` se nomme autrement selon les moteurs. Un rapport destiné à d'autres outils doit le savoir — et le dire.

3. Explication simple — la recette, le tableau, l'étagère

La CTE, c'est la recette étapes par étapes. Au lieu d'écrire une seule phrase de quinze lignes, on écrit : « d'abord je prépare la pâte, ensuite je la laisse reposer, enfin je la cuis ». Chaque étape porte un nom, et une étape peut servir deux fois — c'est même son principal intérêt.

Le tableau croisé, c'est le tableau à double entrée du cours de mathématiques. Une liste de 1 000 lignes « mois, mode de paiement, montant » devient un tableau de 4 lignes et 6 colonnes. Rien n'est ajouté, rien n'est perdu : c'est un changement de **forme**, pas de **fond**.

QUALIFY, c'est le tri suivi du filtre, en un geste. On numérote les lignes par famille, et on garde les numéros 1. En SQL classique, il faut deux requêtes imbriquées pour obtenir cela.

UNNEST, c'est renverser une étagère. Un ticket porte une liste d'articles ; `UNNEST` en fait une ligne par article. La base grossit d'un coup — **1,63** produit par ticket en moyenne — et le grain change.

Les récapitulatifs, ce sont les totaux du tableau. Pas en bas du tableau, collés à la main : **dans** la requête, avec `ROLLUP`, et une colonne `GROUPING()` qui dit ce qui est un détail et ce qui est un sous-total.

4. Vocabulaire essentiel

Terme (français)		Équivalent anglais	Définition en une phrase
table d'expression commune		CTE, common table expression	Un bloc <code>WITH nom AS (...)</code> qui donne un nom à une étape intermédiaire.
requête en escalier		chained / staged query	Une requête dont chaque étape est nommée et vérifiable isolément.
repliement d'un libellé		normalisation / folding	Ramener plusieurs graphies d'une même valeur à une seule, par étapes explicites.
famille de produits		product family	Le regroupement le plus large du catalogue, obtenu après repliement du libellé.
tableau croisé		pivot / cross-tab	Une sortie où une dimension passe des lignes aux colonnes.
agrégat conditionnel		conditional aggregate	Un agrégat limité par une condition : <code>SUM(montant) FILTER (WHERE mode = ...)</code> .
liste		list / array	Une valeur qui contient plusieurs valeurs, produite par <code>LIST</code> ou <code>ARRAY_AGG</code> .
éclatement		unnest / explode	Transformer une liste en plusieurs lignes.
récapitulatif multi-niveaux		rollup / cube / grouping sets	Un agrégat qui calcule le détail et les sous-totaux en une seule requête.
sous-total		subtotal	Une ligne qui agrège un sous-ensemble des lignes détaillées.
indicateur de regroupement		<code>grouping()</code>	La fonction qui distingue un sous-total d'une valeur réellement absente.

Terme (français)	Équivalent anglais	Définition en une phrase
réursion	recursive CTE	Une CTE qui se rappelle elle-même pour parcourir une hiérarchie ou une suite.

Définition. Une **table d'expression commune** — CTE — est un bloc `WITH nom AS (requête)` placé avant la requête principale. Elle ne stocke rien : elle **nomme**. Son intérêt n'est pas la performance, mais le fait qu'on peut exécuter chaque étape séparément, la vérifier, et la réutiliser dans plusieurs requêtes. Une requête de rapport lisible est presque toujours une suite de trois à six CTE.

Définition. Le **repliement d'un libellé** consiste à ramener plusieurs graphies d'une même valeur à une seule écriture. Il se fait par étapes **explicites** : casse (`UPPER`), espaces (`TRIM`), accents, et singulier/pluriel. Chaque étape se mesure — sur le socle, **16** libellés deviennent **9** écritures après casse et espaces, puis **7** familles après accents et pluriel. Une étape qu'on ne mesure pas est une étape qui a perdu des lignes sans le dire.

Définition. Un **récapitulatif multi-niveaux** est une agrégation qui produit, dans **une** requête, le détail et les sous-totaux. `ROLLUP (a, b)` rend le détail par couple, les sous-totaux de **a** et le total général ; `CUBE` rend en plus les sous-totaux de **b** seul ; `GROUPING SETS (...)` énumère exactement les niveaux voulus. Le nombre de lignes n'est plus celui du détail : il faut donc **nommer** les lignes de total, et c'est le rôle de `GROUPING()`.

Définition. Le **grain** d'un jeu de lignes est ce que représente **une** ligne. `UNNEST` le change sans prévenir : une ligne « ticket » devient une ligne « article de ticket ». Toute agrégation écrite après un `UNNEST` doit être relue à l'aune de ce nouveau grain — sinon `COUNT(*)` compte des articles en croyant compter des tickets.

5. Cours approfondi

5.1 La CTE : nommer les étapes, et réutiliser la bonne

Voici la forme que prend toute requête de rapport un peu longue :

```
WITH n AS (
    SELECT id_produit,
           CASE WHEN UPPER(TRIM(categorie)) IN ('MATERIAUX', 'MATÉRIAUX') THEN 'MATÉRIAUX'
                WHEN UPPER(TRIM(categorie)) IN ('PEINTURE', 'PEINTURES') THEN 'PEINTURE'
                WHEN UPPER(TRIM(categorie)) = 'ELECTRICITÉ' THEN 'ÉLECTRICITÉ'
                ELSE UPPER(TRIM(categorie)) END AS famille
    FROM produit
),
ca AS (SELECT n.famille, SUM(v.montant_ttc) AS ca
      FROM ventes v JOIN n USING (id_produit)
      WHERE NOT v.est_retour GROUP BY 1)
SELECT famille, ca, ROUND(100.0 * ca / SUM(ca) OVER (), 2) AS part_pct
FROM ca ORDER BY ca DESC;
```

Trois points à retenir. La CTE **n** ne fait qu'une chose : replier le libellé. La CTE **ca** ne fait qu'agréger. La requête finale ne fait qu'ajouter la part, avec une fenêtre sur l'agrégat. Aucune des trois n'est compliquée, et chacune se vérifie seule : `SELECT * FROM n` doit rendre **154** lignes, avec **7** familles distinctes.

C'est la **réutilisation** qui fait la valeur d'une CTE : la même CTE **n** sert ensuite à croiser les familles avec les quartiers, puis avec les années, sans que le repliement soit réécrit — donc sans risque qu'il soit réécrit **différemment**.

5.2 Le tableau croisé : PIVOT et l'agrégat conditionnel

Deux écritures produisent le même tableau. La première est DuckDB :

```
PIVOT (SELECT YEAR(date_vente) AS an, mode_paiement AS mp, montant_ttc AS m
       FROM ventes WHERE NOT est_retour)
ON mp USING SUM(m) GROUP BY an ORDER BY an;
```

an	Chèque	Créance 30 j	Espèces	Mobile Money	Virement
2023	623750816	273920936	1155573169	886824267	503512359
2024	723192393	321227804	1393160843	1047845325	580893385
2025	857090551	374566410	1612526524	1228076360	652440441
2026	593972283	262208583	1156476174	892367109	455529223

La seconde est du SQL de tous les moteurs :

```
SELECT YEAR(date_vente) AS an,
       SUM(montant_ttc) FILTER (WHERE mode_paiement = 'Espèces') AS especes,
       SUM(montant_ttc) FILTER (WHERE mode_paiement = 'Mobile Money') AS mobile,
       SUM(montant_ttc) FILTER (WHERE mode_paiement = 'Chèque') AS cheque
FROM ventes WHERE NOT est_retour GROUP BY 1 ORDER BY 1;
```

Les totaux sont identiques : pour 2023, la somme des cinq colonnes du PIVOT vaut **3 443 581 547 FCFA**, exactement le chiffre d'affaires de l'année. Le PIVOT est donc un **confort**, pas un moteur de calcul — et il a un piège de présentation : il range ses colonnes par ordre **alphabétique**.

Attention. PIVOT trie les colonnes produites par ordre alphabétique : Chèque, Créance 30 j, Espèces, Mobile Money, Virement — alors que l'ordre des données est Espèces, Mobile Money, Chèque, Virement, Créance 30 j. Un SELECT * publié tel quel donne donc un tableau dont l'ordre des colonnes n'est pas celui des libellés. Règle : on **nomme** les colonnes dans l'ordre voulu (SELECT Espèces, "Mobile Money", Cheque, ...) ou on garde l'agrégat conditionnel, dont l'ordre est écrit à la main.

5.3 QUALIFY : classer et filtrer dans la même requête

Le classement par groupe de C02 exigeait deux niveaux de requête : un pour le rang, un pour le filtre. QUALIFY (disponible dans DuckDB, Snowflake, BigQuery, Teradata) n'en exige qu'un :

```
WITH n AS (SELECT id_produit, CASE ... END AS famille FROM produit),
     v AS (SELECT n.famille, p.designation, ROUND(SUM(x.montant_ttc)) AS ca
          FROM ventes x JOIN n USING (id_produit) JOIN produit p USING (id_produit)
          WHERE NOT x.est_retour GROUP BY 1, 2)
SELECT famille, designation, ca FROM v
QUALIFY ROW_NUMBER() OVER (PARTITION BY famille ORDER BY ca DESC, designation) = 1
ORDER BY ca DESC;
```

MATÉRIAUX	Gravier 15/25 (m3) – réf 1	351651755
PLOMBERIE	Lavabo céramique – réf 2	349907568
BOIS & PANNEAUX	Panneau MDF 12 mm – réf 2	255397413
ÉLECTRICITÉ	Lampe solaire murale – réf 2	132272554
QUINCAILLERIE	Pince coupante 200 mm – réf 2	119995326
PEINTURE	Antirouille rouge brique 1 L – réf 2	110331930
CONSOMMABLES	Pelle carrée – réf 1	101865391

7 lignes, une par famille, et le départage est explicite (ca DESC, designation) — sans lui, deux produits à égalité de chiffre d'affaires changeraient de place d'une exécution à l'autre (leçon de C04).

Attention. Le **même** `QUALIFY` écrit sur les libellés bruts de `produit.categorie` rend **16** lignes — pour **7** familles réelles. Trois de ces lignes s'appellent Matériaux (`MATERIAUX`, `Materiaux`, `Matériaux`), et trois autres Quincaillerie. Le tableau est parfaitement exécuté, et il présente trois fois le même rayon : `QUALIFY` ne remplace pas la propreté des données, il la rend seulement plus visible.

5.4 UNNEST : inverser le grain

Un ticket contient une liste d'articles. Pour obtenir une ligne par article — le grain du détail — on éclate la liste :

```
WITH t AS (SELECT v.id_ticket, LIST(DISTINCT p.designation ORDER BY p.designation) AS lp
           FROM ventes v JOIN produit p USING (id_produit)
           WHERE NOT v.est_retour AND v.id_ticket = (SELECT MIN(id_ticket) FROM ventes)
           GROUP BY 1)
SELECT id_ticket, UNNEST(lp) AS designation FROM t;
```

T01-230101-000000 | Mètre ruban 5 m – réf 1

Le premier ticket du socle ne porte qu'une désignation, mais le maximum observé est de **4** produits (**1,63** en moyenne, **1** en médiane sur **145 212** tickets). L'opération est donc asymétrique : elle multiplie la base, et pas d'un facteur constant.

5.5 Les récapitulatifs : ROLLUP, CUBE, GROUPING SETS

Un tableau de direction comporte des sous-totaux. Sans récapitulatif, il faut trois requêtes et un recollage manuel ; avec, une seule :

```
WITH n AS (SELECT id_produit, CASE ... END AS f FROM produit),
b AS (SELECT ma.quartier AS q, n.f, SUM(v.montant_ttc) AS ca
      FROM ventes v JOIN magasin ma USING (id_magasin)
      JOIN n USING (id_produit) WHERE NOT v.est_retour GROUP BY 1, 2)
SELECT COALESCE(q, 'TOTAL') AS quartier, COALESCE(f, 'sous-total') AS famille,
       GROUPING(q) AS gq, GROUPING(f) AS gf, ROUND(SUM(ca)) AS ca
FROM b GROUP BY ROLLUP (q, f) ORDER BY gq, quartier, gf, famille;
```

41 lignes : **35** couples détail, **5** sous-totaux de quartier, **1** total général (**15 595 154 955** FCFA). Les deux colonnes `GROUPING()` disent, ligne par ligne, ce qui est du détail (0) et ce qui est un sous-total (1) — sans elles, un `NULL` de sous-total se confondrait avec une famille non renseignée.

`GROUPING SETS` répond à un autre besoin : quand on veut certains niveaux **sans** les autres. Sur le même socle, `GROUPING SETS ((quartier), (famille), (quartier, famille))` rend **47** lignes — **5** quartiers, **7** familles, **35** couples — chacune à son propre grain.

Dans les faits. Le tableau croisé des modes de paiement rend la période partielle **visible d'un coup d'œil** : les cinq colonnes progressent de 2023 à 2025, puis reculent toutes ensemble en 2026 — qui ne compte que **8** mois (C03, \$5.3). Aucun commentaire n'était nécessaire : la forme du tableau le dit.

5.6 La récursion : pour les hiérarchies, pas pour les calendriers

`WITH RECURSIVE` fait se rappeler une requête elle-même. Le cas d'école — la suite des mois — se résout plus simplement avec `generate_series` :

```
WITH RECURSIVE m AS (SELECT DATE '2023-01-01' AS d
                     UNION ALL SELECT d + INTERVAL 1 MONTH FROM m WHERE d < DATE '2026-08-01')
SELECT COUNT(*) AS mois FROM m;
```

44 — la même série que la table calendrier de C03, en plus verbeux et plus lent. La récursion mérite d’être écrite quand la **profondeur** est inconnue : arborescence d’organisation, nomenclature de produits, chaîne d’approbation. Sur un catalogue à deux niveaux (famille, sous-catégorie) et **29** couples, elle est un luxe ; sur un arbre d’articles imbriqué sur sept niveaux, elle est la seule solution.

5.7 Ce que chaque moteur sait faire

Élément	DuckDB (exécuté)	PostgreSQL (cité)	SQL Server (cité)
WITH (CTE)	oui	oui	oui
WITH RECURSIVE	oui	oui	oui, sans le mot RECURSIVE
QUALIFY	oui	non : sous-requête obligatoire	non : TOP ou ROW_NUMBER
PIVOT	oui, PIVOT (...) ON ... USING ...	non : crosstab (extension) ou agrégat conditionnel	oui, PIVOT (...) FOR ... IN (...)
UNNEST	oui	unnest()	OPENJSON / STRING_SPLIT selon la source
ROLLUP / CUBE / GROUPING SETS	oui	ROLLUP, CUBE, GROUPING SETS	idem, GROUPING() fourni

Attention. Ce tableau distingue trois colonnes qui ne pèsent pas pareil : DuckDB a été **exécuté** pour chaque ligne, PostgreSQL et SQL Server sont **cités** — aucune ligne de cette colonne n’a été jouée dans l’atelier. C’est la règle §1.5 du manuel, et elle vaut pour la syntaxe comme pour les chiffres : un rapport qui publie du code non exécuté le **déclare**.

Conseil professionnel. Écrivez le tableau croisé d’un rapport récurrent en **agrégat conditionnel**, même si votre moteur sait faire PIVOT. Trois raisons : les colonnes y sont nommées dans l’ordre voulu, une période sans donnée y sort à NULL au lieu de disparaître, et le code s’exécute partout. Gardez PIVOT pour l’exploration, là où la liste des colonnes n’est pas connue d’avance.

5.8 Ce que ce chapitre ne couvre pas

La **performance** de ces constructions — coût du PIVOT, effet d’un index, lecture d’un plan d’exécution — est le sujet de C06. Reste ici une règle : aucun de ces outils ne rend une requête rapide ; ils la rendent **lisible**, ce qui est la condition pour pouvoir un jour la rendre rapide.

6. Exemple concret — le tableau que la direction avait demandé

La demande : « le chiffre d’affaires par quartier et par famille, avec les sous-totaux, pour les quatre années ». La réponse tient en deux étapes nommées et un ROLLUP. Ce que la direction reçoit :

- **41** lignes, dont **5** sous-totaux de quartier et **1** total général de **15 595 154 955** FCFA ;
- un détail par couple quartier × famille, où les familles se nomment MATÉRIAUX, PLOMBERIE, ÉLECTRICITÉ, QUINCAILLERIE, BOIS & PANNEAUX, CONSOMMABLES et PEINTURE — **7** familles, pas **16** libellés ;
- et les deux familles de tête à **3 535 582 421** et **3 452 082 064** FCFA, soit **0,5** point d’écart.

Ce dernier point est le plus utile au comité : sur ce catalogue, savoir laquelle de MATÉRIAUX et de PLOMBERIE arrive en tête dépend **du repliement des libellés**. La réponse honnête n'est pas « Matériaux est premier », c'est « les deux sont à égalité, à 0,5 point près ».

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — le repliement mesuré

`SELECT COUNT(DISTINCT categorie) FROM produit` → **16**. Après `UPPER(TRIM(...))` → **9**. Après repliement des accents et du pluriel → **7**. Trois nombres, trois définitions du catalogue.

7.2 Étape 2 — la CTE réutilisée

La CTE `n` du §5.1 sert trois fois : classement par famille, croisement avec les quartiers, croisement avec les années. Une seule définition du repliement, donc un seul classement possible.

7.3 Étape 3 — le tableau croisé

Le `PIVOT` des modes de paiement par année (§5.2) et sa vérification : la somme des cinq colonnes de 2023 égale **3 443 581 547** FCFA, le chiffre d'affaires de l'année. Puis la même table en agrégat conditionnel, pour constater que les colonnes y sont dans l'ordre métier.

7.4 Étape 4 — `QUALIFY` avec et sans repliement

7 lignes avec la CTE de repliement, **16** sans. C'est la démonstration la plus courte du chapitre : un outil puissant posé sur des données sales produit une réponse fausse **plus vite**.

7.5 Étape 5 — `UNNEST` et le grain

Le premier ticket du socle (T01-230101-000000) ne porte qu'une désignation ; le maximum est de **4** produits par ticket. Après éclatement, la base ne compte plus **145 212** tickets mais des lignes d'articles : tout `COUNT(*)` écrit ensuite compte **des articles**.

7.6 Étape 6 — le récapitulatif

41 lignes en `ROLLUP` (35 + 5 + 1), **47** en `GROUPING SETS` (5 + 7 + 35). Le contrôle tient en une requête : le total général du récapitulatif doit être **égal** au chiffre d'affaires net du socle (**15 595 154 955** FCFA). S'il diffère, un `NULL` a été compté comme un zéro, ou un `GROUPING` a été oublié.

8. Erreurs fréquentes

1. **Croire que la CTE accélère.** Elle nomme. Un moteur peut la recalculer plusieurs fois ; c'est un excellent découpage de lecture et pas une optimisation. La performance se mesure (C06).
2. **Publier un `PIVOT` en `SELECT *`.** Les colonnes sortent par ordre alphabétique — Chèque avant Espèces — et le tableau publié n'a plus l'ordre du métier.
3. **Classer sur les libellés bruts.** **16** lignes pour **7** familles : le même rayon apparaît trois fois sous trois graphies (Matériaux, matériaux, MATÉRIAUX).
4. **Agréger après un `UNNEST` sans redéfinir le grain.** Les tickets deviennent des articles (**1,63** par ticket en moyenne) : un `COUNT(*)` compare alors des articles à des tickets.
5. **Confondre un `NULL` de sous-total et une valeur absente.** Sans `GROUPING()`, la ligne de total général ressemble à une ligne dont la dimension est vide — et un lecteur pressé la somme comme un détail.

6. **Recopier une syntaxe non exécutée.** `QUALIFY` en PostgreSQL et `PIVOT ... ON ... USING` en SQL Server n'existent pas : un rapport portable s'écrit en agrégat conditionnel.

9. Bonnes pratiques professionnelles

1. **Une étape nommée par intention.** `n` pour le repliement, `ca` pour l'agrégat, `rg` pour le rang : une CTE qui s'appelle `tmp` empêche la relecture.
2. **Mesurer chaque repliement.** `16 → 9 → 7` : chaque étape publie son compte avant d'être utilisée dans un rapport.
3. **Nommer les colonnes d'un tableau croisé.** Jamais `SELECT *` sur un `PIVOT` ; l'ordre des colonnes fait partie de la donnée.
4. **Écrire le départage de tout classement.** `ORDER BY ca DESC, designation` : la leçon de C04 vaut pour `QUALIFY` comme pour `ROW_NUMBER`.
5. **Vérifier le total des récapitulatifs** contre le total de référence (**15 595 154 955 FCFA**), et déclarer les niveaux calculés. Un sous-total non vérifié est un chiffre de plus à défendre.

10. Exercice guidé

Situation. La direction commerciale veut un tableau unique : les **7** familles en lignes, les **4** années en colonnes, le total par famille à droite, le total par année en bas, et la part de chaque case.

Consigne.

1. Écrivez la CTE de repliement des libellés, et donnez ses trois comptes : libellés bruts, écritures, familles. (2 pts)
2. Produisez le tableau croisé des familles par année, en **agrégat conditionnel**, avec les colonnes dans l'ordre des années. (3 pts)
3. Ajoutez les sous-totaux de ligne et de colonne, et dites par quelle construction ils sont produits. (3 pts)
4. Expliquez, en deux phrases, pourquoi il ne faut **pas** écrire ce tableau en `SELECT *` sur un `PIVOT`. (2 pts)

11. Exercices autonomes

E1 — Trois façons, un seul tableau (40 min). Produisez le croisement quartiers × familles : (a) en `PIVOT`, (b) en agrégat conditionnel, (c) en `GROUPING SETS`. Puis répondez en quatre lignes : quel tableau porte des sous-totaux, lequel a les colonnes dans l'ordre métier, et lequel faudrait-il publier si le rapport partait vers un moteur qui ignore `PIVOT`.

E2 — Le panier du premier ticket, et sa généralisation (35 min). À partir de la CTE qui éclate un ticket en lignes, calculez la composition moyenne d'un ticket par famille de produits. Contrainte : le grain doit être redit — une ligne par article — et le total des articles doit être confronté aux **145 212** tickets du socle.

12. Correction détaillée

Exercice guidé.

1. **Les trois comptes.** **16** libellés bruts, **9** écritures après `UPPER(TRIM(...))`, **7** familles après repliement des accents (`MATÉRIAUX/MATERIAUX`, `ELECTRICITÉ`) et du pluriel (`PEINTURE/PEINTURES`). La CTE écrit ces trois règles une fois pour toutes.

2. **Le tableau.** `SUM(montant_ttc) FILTER (WHERE YEAR(date_vente) = 2023) AS ca_2023`, puis 2024, 2025, 2026, avec `GROUP BY famille` sur la CTE de repliement. Colonnes dans l'ordre chronologique, comme dans le rapport.
3. **Les sous-totaux.** `GROUP BY ROLLUP (famille, annee)` : les cellules, les sous-totaux de famille, les sous-totaux d'année et le total général dans la **même** requête — **41** lignes pour le croisement quartier × famille, et la même structure ici. `GROUPING(famille)` et `GROUPING(annee)` nomment les lignes de total.
4. **Pourquoi pas SELECT *.** Parce que `PIVOT` produit ses colonnes par ordre alphabétique : les années seraient bien ordonnées (2023, 2024...), mais des libellés comme Chèque et Espèces se retrouveraient dans l'ordre inverse de celui du métier — et le tableau publié serait, à la lettre, un autre tableau.

E1 (trois façons). (a) Le `PIVOT` est le plus court à écrire et le moins contrôlable : colonnes alphabétiques, aucune notion de sous-total. (b) L'agrégat conditionnel donne l'ordre métier des colonnes et fonctionne partout, mais les sous-totaux demandent un `ROLLUP` supplémentaire ou une requête d'union. (c) Le `GROUPING SETS` est le seul qui produit les trois niveaux en une passe, au prix d'une lecture plus difficile. Pour un rapport qui part vers un moteur sans `PIVOT` : l'agrégat conditionnel, éventuellement enveloppé dans un `GROUPING SETS`.

E2 (le panier éclaté). Après éclatement, la base compte les lignes d'articles, pas les tickets : le nombre de lignes dépasse nécessairement la médiane de **1** produit par ticket, et le total des articles doit être comparé aux **145 212** tickets — jamais confondu avec eux. La composition moyenne par famille se lit alors par ticket et non par ligne d'article : le dénominateur est le nombre de **tickets**, pas le nombre de lignes éclatées.

13. Mini-projet de chapitre

Le tableau de bord d'une famille de produits, 2 pages. Choisissez une famille du catalogue et produisez son tableau de bord : ventes par année, par quartier et par mode de paiement, avec sous-totaux et parts, plus le top 3 des désignations.

Livrables et barème.

Critère	Ce qui est vérifié	Points
La CTE de repliement	les trois comptes (16 / 9 / 7) publiés, une seule définition	5
Le tableau croisé	deux dimensions croisées, colonnes nommées dans l'ordre métier	5
Les sous-totaux	produits par <code>ROLLUP</code> ou <code>GROUPING SETS</code> , <code>GROUPING()</code> utilisé, total vérifié	5
Le top 3	<code>QUALIFY</code> avec départage explicite, une ligne par désignation	3
Total		18

14. Résumé du chapitre

Six outils, une même idée : rendre une requête longue **lisible et vérifiable**. La **CTE** nomme les étapes et se réutilise — c'est elle qui porte le repliement des libellés (**16** → **9** → **7**). Le **tableau croisé** change la forme sans changer le fond, et l'agrégat conditionnel le fait partout, avec des colonnes dans l'ordre métier — là où `PIVOT` trie alphabétiquement. `QUALIFY` classe et filtre en une passe : **7** lignes sur les familles repliées, **16** sur les libellés bruts, et c'est la donnée qui décide, pas l'outil. `UNNEST` inverse le grain (**1,63** produit par ticket en moyenne, **4** au maximum) et oblige à redire ce qu'est une ligne. Les **récapitulatifs** rendent le détail et les sous-totaux ensemble — **41** lignes en `ROLLUP`, **47** en `GROUPING SETS` — à condition que `GROUPING()` distingue un total d'un `NULL`. Et la **réursion** sert les hiérarchies : pour les **44** mois du socle, `generate_series` fait le travail en une ligne.

15. À retenir

1. **Une requête longue se découpe en étapes nommées.** Une étape qu'on ne peut pas exécuter seule ne peut pas être vérifiée.
2. **Un repliement de libellé se mesure à chaque étape. 16 libellés, 9 écritures, 7 familles :** trois nombres à publier avant tout classement.
3. **PIVOT range ses colonnes par ordre alphabétique.** Le tableau publié doit nommer ses colonnes.
4. **QUALIFY ne nettoie pas les données, il les rend visibles. 16 lignes pour 7 rayons.**
5. **UNNEST change le grain.** Après éclatement, la ligne est un article, plus un ticket.
6. **GROUPING() sépare un sous-total d'une absence.** Sans lui, un total général se lit comme un détail.
7. **La récursion est faite pour les hiérarchies, pas pour égrener 44 mois.**

À retenir. Le chiffre qui résume le chapitre n'est pas un total : c'est la paire **16 / 7**. Seize façons d'écrire sept rayons, et un outil avancé qui publie seize lignes sans broncher. Le nettoyage des libellés n'est pas une étape préparatoire qu'on repousse : c'est la première CTE du rapport.

À retenir. Les deux familles de tête du catalogue sont à **0,5** point l'une de l'autre (**3 535 582 421** et **3 452 082 064** FCFA). Un rapport qui annonce « Matériaux est notre première famille » énonce donc moins une vérité qu'une conséquence de son propre repliement.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien de libellés de catégorie compte le référentiel produits, combien d'écritures après casse et espaces, combien de familles après accents et pluriel ?
2. Que produit un « meilleur produit par catégorie » écrit sur les libellés bruts, et pourquoi est-ce un problème ?
3. Dans quel ordre PIVOT range-t-il ses colonnes, et pourquoi est-ce gênant pour un rapport ?
4. Quelle écriture du tableau croisé fonctionne dans tous les moteurs, et pourquoi ?
5. Que change UNNEST au grain d'une table, et quel chiffre du socle le rappelle ?
6. Combien de lignes rend ROLLUP (quartier, famille), et comment se décomposent-elles ?
7. À quoi sert GROUPING(), et que se passe-t-il si on l'oublie ?
8. Combien de lignes rend GROUPING SETS ((quartier), (famille), (quartier, famille)), et de quoi sont-elles faites ?
9. Quand la récursion est-elle préférable à generate_series, et pourquoi ?
10. Quelles sont les deux familles de tête du catalogue, et de combien d'écart ?

Réponses. 1. **16** libellés, **9** écritures, **7** familles. 2. **16** lignes, dont trois graphies du même rayon : le classement dépend du libellé et non du produit. 3. Par ordre **alphabétique** (Chèque, Créance 30 j., Espèces...), alors que l'ordre métier est Espèces, Mobile Money, Chèque, Virement, Créance 30 j. 4. **L'agrégat conditionnel** (SUM(...) FILTER (WHERE ...)) : colonnes nommées dans l'ordre voulu, NULL préservé, et syntaxe portable. 5. Il fait passer d'une ligne par ticket à une ligne par article — **1,63** produit par ticket en moyenne, **4** au maximum. 6. **41** lignes : **35** couples, **5** sous-totaux de quartier, **1** total général. 7. À distinguer un sous-total d'une valeur absente ; sans lui, la ligne de total général se lit comme une ligne de détail. 8. **47** lignes : **5** quartiers, **7** familles, **35** couples. 9. Quand la profondeur de la hiérarchie n'est pas connue d'avance — un arbre d'organisation, pas une suite de **44** mois. 10. **MATÉRIAUX** (**3 535 582 421** FCFA) et **PLOMBERIE** (**3 452 082 064** FCFA), à **0,5** point près.

Module M11.C06 — Le rapport lent : vues, index, plans d'exécution, `SELECT *`, partitions

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 5 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C05 (fenêtres, cohortes, requêtes avancées) ; M07.C06 (volumétrie) ; M09.C05 (lecture d'un plan d'exécution).

L'idée du chapitre. Un rapport juste et lent n'est pas un rapport. Sur le socle du module, la même question — « combien de lignes de retour ? » — coûte **142 ms** posée sur la vue et **1 ms** posée sur la même table recopiée en mémoire : le rapport entre les deux est **×142**, pour un résultat identique au chiffre près. Plus frappant encore, `SELECT *` coûte **581 ms** là où trois colonnes nommées coûtent **96 ms (×6,1)** : le plan annonce **18** colonnes lues contre **3**, et l'horloge confirme. Ce chapitre apprend à **lire** ces nombres avant de les subir : ce qu'une vue relit, ce qu'un `SELECT *` transporte, ce qu'un index change vraiment (mesuré : presque rien, en DuckDB), comment un plan se lit de bas en haut, et pourquoi une mesure de performance se **fige** dans un fichier daté au lieu de se relire à l'horloge.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). Les temps cités sont des **médianes de 5 exécutions**, produites par `tools/perf_M11.py` et **gelées** le **24/09/2026** dans `03_exercices/dossier_M11/PERF_M11.json`. Le chapitre ne relit jamais l'horloge : il relit ce fichier.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Distinguer une vue d'une table** dans ses conséquences pratiques, et dire ce qu'une vue **relit** à chaque appel.
2. **Mesurer un protocole** : médiane de plusieurs exécutions, instrument nommé, valeurs **figées et datées**.
3. **Chiffrer le coût d'un `SELECT *`** à l'horloge **et** dans le plan d'exécution, sur une table et sur une vue.
4. **Lire un plan d'exécution** : les étages, leur ordre, le nombre de colonnes lues, et ce qu'un `SEQ_SCAN` annonce du travail à venir.
5. **Décider de l'utilité d'un index sur une mesure**, pas par réflexe — et savoir pourquoi un même index change tout dans un moteur et rien dans un autre.
6. **Arbitrer entre lisibilité et performance** quand les deux se contredisent (§5.5).
7. **Écrire la note de méthode d'un rapport** : périmètre, protocole, valeurs gelées, ce qui n'a pas été mesuré.

2. Pourquoi cette notion est importante

Le module M11 produit **12** rapports. Chacun d'eux lit le même socle de **240 000** lignes de ventes. Si le rapport type met **841 ms** au lieu de **277 ms** — c'est-à-dire écrit sur la vue et en `SELECT *` au lieu de la table et de trois colonnes — l'écart paraît dérisoire. Multiplié par douze rap-

ports, par les rafraîchissements quotidiens et par les relances de chaque utilisateur, il devient une salle d'attente.

Mais le vrai enjeu du chapitre n'est pas la vitesse : c'est la **confiance**. Un rapport dont la note de méthode dit « médiane de 5 exécutions, DuckDB 1.5.5, valeurs figées le 24/09/2026 dans PERF_M11.json » est un rapport **reproductible**. Un rapport dont les temps ont été copiés une fois, sans protocole, est un rapport qu'on ne peut ni vérifier ni défendre. La performance, en BI, se **documente** avant de s'optimiser.

Enfin, le chapitre désarme un réflexe coûteux : sur ce socle, ajouter un index fait passer une recherche par client de **0,6 ms** à **0,5 ms** — et le plan continue d'annoncer **SEQ_SCAN**. L'index n'est pas une bonne pratique à appliquer partout : c'est une réponse à un problème que le **plan** doit avoir montré.

3. Explication simple — le classeur, le sommaire et le chariot

La vue, c'est un raccourci vers le classeur. Chaque fois qu'on ouvre le raccourci, il faut ressortir le classeur et le relire. La table, c'est le tableau recopié sur le bureau : on l'a payé une fois (la copie) et on le consulte gratuitement ensuite. Sur le socle : **142 ms** à chaque appel de la vue, **1 ms** pour la table équivalente.

Le `SELECT *`, c'est le chariot trop large. Demander toutes les colonnes d'une table large revient à sortir du magasin les **18** rayons pour n'utiliser que **3** produits. Le plan le dit avant l'exécution : **18** colonnes lues au lieu de **3**, et l'horloge confirme (**581 ms** contre **96**).

Le plan d'exécution, c'est la fiche de préparation du cuisinier. Avant de cuisiner, il annonce : « je vais chercher les fichiers, les joindre, les regrouper, projeter les colonnes ». Trois étages sur le socle (`HASH_GROUP_BY`, `PROJECTION`, `HASH_JOIN`), lus **de bas en haut**. On ne discute pas une performance sans cette fiche.

L'index, c'est l'index alphabétique d'un dictionnaire. Utile pour trouver **une** page précise, inutile pour lire le dictionnaire entier. Sur DuckDB — un moteur en colonnes — la recherche d'un client reste un **SEQ_SCAN** : le moteur lit la colonne entière, et cela lui coûte moins que de naviguer dans un index. Sur SQLite — un moteur en lignes — le même index **remplace** le balayage complet : le plan passe de « `SCAN ventes_t` » à « `SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)` ».

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
vue	view	Une requête nommée ; elle ne contient pas de données et relit sa source à chaque appel.
matérialisation	materialization	Le fait de copier le résultat d'une vue dans une table, pour ne le calculer qu'une fois.
table en mémoire	in-memory table	Une table dont les données sont copiées dans la mémoire du moteur : rapide, mais rien n'est persisté.
plan d'exécution	execution plan	Le programme que le moteur se propose d'exécuter : les opérateurs, leur ordre et les colonnes lues.
étage	operator / stage	Un opérateur du plan : <code>SEQ_SCAN</code> , <code>HASH_JOIN</code> , <code>HASH_GROUP_BY</code> , <code>PROJECTION</code> , <code>ORDER_BY</code> .
balayage séquentiel	sequential scan	Lire toute une colonne, ligne après ligne : le mode normal d'un moteur en colonnes.

Terme (français)	Équivalent anglais	Définition en une phrase
index	index	Une structure de recherche par valeur : indispensable en moteur en lignes, souvent ignorée en colonnes.
projection	projection	La liste des colonnes réellement lues et transmises aux étages suivants.
coût du SELECT *	star-select overhead	Le travail de lecture de colonnes inutilisées, visible dans la projection du plan.
médiane de 5	median of 5 runs	Le protocole de mesure retenu : cinq exécutions, on publie la valeur médiane.
valeur gelée	frozen measurement	Un temps machine écrit dans un fichier daté, que la prose relit au lieu de relancer l'horloge.
partition	partition	Un découpage physique des données (par date, par magasin) qui permet de ne pas lire ce dont on n'a pas besoin.

Définition. Le **plan d'exécution** est la description, par le moteur, de ce qu'il va faire : les opérateurs, leur ordre d'enchaînement et les colonnes lues. Il se demande par **EXPLAIN avant** l'exécution — et c'est ce qui en fait l'outil d'un analyste : il annonce le coût sans faire le travail. Sur le socle, le croisement magasin / chiffre d'affaires se prépare en **3** étages : **HASH_GROUP_BY**, **PROJECTION**, **HASH_JOIN**.

Définition. Une **vue** ne stocke pas de données : c'est une requête à laquelle on a donné un nom. Sa conséquence pratique est immense et souvent ignorée : **chaque appel relit la source**. Sur le socle, la vue `ventes` relit le CSV des ventes ; compter les retours coûte **142** ms sur la vue contre **1** ms sur la table équivalente — soit **×142** pour un résultat identique.

Définition. Un **index** est une structure de recherche par valeur, qui évite de parcourir toutes les lignes. Il n'est **pas** universel : son utilité dépend du moteur (lignes ou colonnes), de la requête (recherche ponctuelle ou agrégat) et de la sélectivité. La seule façon honnête de trancher est de lire le plan et de mesurer : sur ce socle, l'index ne change ni le plan de DuckDB ni ses temps.

Définition. Le **coût du SELECT *** est le travail de lecture des colonnes qui ne serviront pas à la sortie. Il se lit dans la **projection** du plan — **18** colonnes lues pour un **SELECT ***, **3** pour trois colonnes nommées — et se vérifie à l'horloge (**581** ms contre **96** ms). C'est le gaspillage le plus fréquent des rapports, et le plus simple à corriger.

5. Cours approfondi

5.1 La vue : ce qu'elle relit, à chaque fois

Le socle du module est une suite de vues posées sur des CSV. C'est un choix assumé (aucun fichier de base à versionner, voir §5.6), mais il a un prix :

```
-- sur la vue : le CSV est relu à chaque appel
SELECT COUNT(*) FROM ventes WHERE est_retour;
-- sur la table équivalente, copiée en mémoire
CREATE OR REPLACE TABLE ventes_m AS SELECT * FROM ventes;
SELECT COUNT(*) FROM ventes_m WHERE est_retour;
```

Mesure	Sans matérialisation	Avec matérialisation	Rapport
Comptage des retours	142 ms (vue)	1 ms (table)	×142
SELECT *	841 ms (vue)	581 ms (table)	×1,4
3 colonnes nommées	277 ms (vue)	96 ms (table)	×2,9

La dernière ligne mérite attention : sur la **vue**, `SELECT *` coûte **×3,0** de plus que trois colonnes (**841** contre **277**) ; sur la **table**, l'écart est de **×6,1** (**581** contre **96**). Le gaspillage des colonnes est donc plus visible quand la lecture de fichier ne le masque pas.

5.2 `SELECT *` : le lire dans le plan, pas seulement au chronomètre

Deux requêtes, un seul chiffre d'affaires attendu :

```
EXPLAIN SELECT * FROM ventes_m;
EXPLAIN SELECT date_vente, id_magasin, montant_ttc FROM ventes_m;
```

Le premier plan annonce **18** colonnes lues, le second **3**. Le rapport est de **×6** — et il est **écrit dans le plan**, vérifiable sans exécuter. À l'horloge : **581** ms contre **96** ms, soit **×6,1**. Le plan avait raison avant la mesure, et c'est exactement l'intérêt de le lire : on peut refuser une requête avant de payer son temps.

Dans les faits. Le plan annonce **×6** de colonnes, l'horloge mesure **×6,1**. C'est la démonstration la plus utile du chapitre pour un débutant : la lecture d'un plan n'est pas un rituel de spécialiste, c'est un **devis** — proposé gratuitement par le moteur avant chaque requête.

5.3 Lire un plan : trois étages, de bas en haut

```
EXPLAIN SELECT m.nom, SUM(v.montant_ttc)
FROM ventes_m v JOIN magasin m USING (id_magasin) GROUP BY 1;
```

```
HASH_GROUP_BY
PROJECTION
HASH_JOIN
```

3 étages : le moteur joint les ventes et les magasins (`HASH_JOIN`), projette les colonnes utiles (`PROJECTION`), puis regroupe (`HASH_GROUP_BY`). La lecture se fait **de bas en haut** : les données remontent l'arbre, chaque étage transmettant son résultat à celui du dessus. Trois questions suffisent pour auditer un plan :

1. **Combien de colonnes** chaque lecture annonce-t-elle ? (les projections, §5.2)
2. **Quel opérateur domine** ? Un `SEQ_SCAN` sur une table large précède presque toujours un plan lent.
3. **Y a-t-il un étage inutile** ? Une jointure ou un tri qui ne servent à aucune réponse du rapport.

5.4 L'index : mesuré, puis contrôlé dans un second moteur

Sur le socle matérialisé, on cherche un client précis et on agrège par magasin :

Requête	Sans index	Avec index	Effet
Recherche par client (15676)	0,6 ms	0,5 ms	aucun effet mesurable
Agrégat par magasin	1,5 ms	1,5 ms	aucun effet
Plan de la recherche	<code>SEQ_SCAN</code>	<code>SEQ_SCAN</code>	l'index n'est pas utilisé

DuckDB est un moteur **en colonnes** : lire une colonne entière de **240 000** valeurs compactées coûte moins que parcourir un index. Le contrôle croisé dans un moteur **en lignes** renverse la conclusion :

```
SQLite, 240 000 lignes chargées en mémoire
sans index : SCAN ventes_t
avec index : SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)
réponse    : 14 lignes et 5 578 688 FCFA pour le client 15676
```

Le même index, la même requête, le même résultat — et un plan qui change du tout au tout. La leçon n'est pas « l'index est inutile », elle est « l'index est une réponse à un plan » : on regarde le plan, on voit un balayage complet qui coûte, on indexe, on remesure.

Attention. Une colonne indexée dans un moteur en lignes peut ralentir les **écritures** : chaque insertion met l'index à jour. Sur un rapport de lecture, l'arbitrage est simple ; sur une table alimentée en continu, il se mesure des deux côtés — et se documente.

5.5 Lisibilité contre performance : le cas de la fenêtre

Toutes les écritures lisibles ne sont pas les plus rapides. Le module l'a mesuré sur la question la plus fréquente de C01 — la part d'une ligne dans son ensemble :

Écriture	Temps	Avantage
Fonction de fenêtre (SUM(...) OVER ())	150 ms	une seule requête, une seule passe à écrire, robuste au filtre ajouté
Sous-requête scalaire	292 ms	aucune

La fenêtre est **×1,9** plus rapide **et** plus lisible : ici, les deux critères vont dans le même sens. Ce n'est pas toujours le cas — une CTE bien nommée peut être recalculée plusieurs fois. La règle du chapitre n'est donc pas « préférer le plus lisible » : c'est **mesurer avant de choisir**, puis écrire le choix.

Attention. Un temps machine n'est pas reproductible au bit près : le premier appel paie la lecture du fichier, les suivants non, et la machine hôte varie. C'est pourquoi le protocole est **publié** (médiane de 5 exécutions) et pourquoi les valeurs sont **gelées** dans un fichier daté — `PERF_M11.json`, figé le **24/09/2026** par `tools/perf_M11.py --figer`. Un rapport qui relit l'horloge à chaque exécution publie un chiffre différent chaque matin, et ne peut plus être comparé à lui-même.

5.6 Matérialiser ou non : le plancher d'un fichier

On pourrait écrire le socle matérialisé sur le disque une fois pour toutes. Le coût de ce confort est mesurable : un fichier DuckDB qui ne contient **qu'une** table de **154** lignes pèse **536 576** octets. Ce plancher fixe interdit de versionner la base du module — c'est pour cela que le socle M11 est un **script** (`socle_m11.sql`) qui reconstruit les vues et les tables de référence en une fraction de seconde, et que les démonstrations de volume se font **en mémoire** (`ouvrir(materialiser=True)` dans `03_exercices/dossier_M11/connexion.py`).

Conseil professionnel. Mesurez **avant** d'optimiser, et écrivez la mesure **dans** le rapport. Une note de trois lignes — « ce rapport lit la vue (donc le CSV) et dix-huit colonnes ; en nommant trois colonnes il passe de **581** ms à **96** ms ; l'index n'apporte rien ici » — vaut mieux qu'un rapport rapide et muet. Six mois plus tard, c'est cette note qui empêchera un collègue d'ajouter un index inutile ou de croire que le rapport était lent « pour de bonnes raisons ».

5.7 Les partitions

Le **partitionnement** découpe physiquement les données — par année, par magasin — pour qu'une requête ne lise que les partitions concernées. Trois familles d'usage : le partitionnement **Hive** sur des fichiers (dossier `annee=2026/mois=08/`), les tables partitionnées de PostgreSQL et SQL Server, et les filtres de partition dans les moteurs analytiques.

Attention. Le partitionnement est **cité, non exécuté** dans ce chapitre : le socle de l'atelier n'a ni le volume ni le disque pour qu'une mesure soit interprétable. Aucun plan de partition n'est donc publié ici — un chapitre qui montre un plan qu'il n'a pas lu fabrique une preuve, ce que ce manuel interdit depuis le premier module (§1.5).

5.8 La note de méthode d'un rapport de performance

Tout rapport de performance doit pouvoir répondre à cinq questions, et les cinq tiennent en quatre lignes :

1. **Quel moteur et quelle version ?** DuckDB 1.5.5, sur l'atelier.
2. **Quel protocole ?** Médiane de 5 exécutions, requête complète, résultat consommé.
3. **Quand ?** Le 24/09/2026.
4. **Où sont les valeurs ?** `03_exercices/dossier_M11/PERF_M11.json`, produit par `tools/perf_M11.py`.
5. **Qu'est-ce qui n'a pas été mesuré ?** Les partitions, PostgreSQL et SQL Server — cités, non exécutés.

6. Exemple concret — la facture d'un rapport mal écrit

Un rapport quotidien lit le socle et publie le chiffre d'affaires par magasin avec sa part. Deux écritures, mesurées :

Version	Écriture	Temps	Sur 12 rapports × 3 exécutions
A	vue + SELECT * + sous-requête scalaire	≈ 841 ms + 292 ms	≈ 41 s
B	table matérialisée + 3 colonnes + fenêtre	≈ 96 ms + 150 ms	≈ 9 s

Le même contenu, la même justesse, et un facteur **4,5** — sans changer une seule ligne de logique métier, seulement la **forme** de la lecture. C'est le résultat le plus utile du chapitre : la première optimisation d'un rapport n'est presque jamais un index, c'est de **lire ce qu'on utilise**.

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — le protocole d'abord

Cinq exécutions, on publie la médiane, et l'instrument est nommé : `tools/perf_M11.py`. Sans protocole, un temps ne vaut rien.

7.2 Étape 2 — la vue contre la table

142 ms contre **1 ms (×142)** pour compter les retours. La vue relit le CSV ; la table est en mémoire.

7.3 Étape 3 — le plan du `SELECT *`

18 colonnes annoncées contre **3**, **×6** dans le plan et **×6,1** à l'horloge (**581** ms contre **96** ms). Le devis du moteur suffisait à trancher.

7.4 Étape 4 — l'index, mesuré puis contrôlé

0,6 ms contre **0,5** ms, agrégat inchangé, plan toujours `SEQ_SCAN` : en DuckDB, aucun effet. Le contrôle croisé SQLite montre un plan qui passe de `SCAN ventes_t` à `SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)` — et le même résultat, **14** lignes et **5 578 688** FCFA pour le client **15676**.

7.5 Étape 5 — fenêtre contre sous-requête

150 ms contre **292** ms (**×1,9**) pour la même part : ici, la forme la plus lisible est aussi la plus rapide.

7.6 Étape 6 — le plancher de matérialisation

536 576 octets pour une table de **154** lignes : rien ne justifie de versionner une base là où un script de reconstruction suffit.

8. Erreurs fréquentes

1. **Optimiser sans mesurer.** Ajouter un index par réflexe n'a **aucun** effet mesurable sur ce socle (**0,6** ms contre **0,5** ms) et le plan reste `SEQ_SCAN` : le travail n'est pas là où on le croyait. C'est mesurer d'abord, écrire ensuite.
2. **Publier un temps sans protocole.** Un chiffre isolé, sans version de moteur, sans nombre d'exécutions et sans date, n'est pas une mesure : c'est une anecdote. Le protocole tient en une ligne (**médiane de 5**, DuckDB 1.5.5, **24/09/2026**, `PERF_M11.json`).
3. **Laisser `SELECT *` dans un rapport.** **581** ms contre **96** ms, **18** colonnes contre **3** : le gaspillage est dans le plan, donc évitable sans exécuter.
4. **Confondre vue et table dans l'estimation d'un coût.** Le même comptage vaut **142** ms ou **1** ms selon la source : annoncer « ce rapport coûte 1 ms » en l'ayant mesuré sur la table, pour un rapport qui lit la vue, est un chiffre faux.
5. **Recopier les temps d'un autre moteur.** Un index qui transforme un plan SQLite (`SEARCH`) ne change rien à un plan DuckDB (`SEQ_SCAN`) : les conclusions ne se transportent pas.
6. **Relire l'horloge dans un rapport.** Deux exécutions doivent donner le même document ; un temps mesuré « en direct » fait varier le rapport d'un jour à l'autre sans qu'aucune donnée n'ait changé.

9. Bonnes pratiques professionnelles

1. **Écrire le protocole avant le chiffre.** Médiane de **5** exécutions, version du moteur, date, instrument nommé : la même phrase que celle du fichier `PERF_M11.json`.
2. **Geler les valeurs, dater le fichier.** `tools/perf_M11.py --figer` écrit le JSON ; la prose le relit. C'est ce qui rend deux relevés identiques sur une machine plus lente.
3. **Lire le plan avant d'optimiser.** Le devis est gratuit : **18** contre **3** colonnes annoncées, pour **581** contre **96** ms mesurées.
4. **Nommer ses colonnes.** `SELECT date_vente, id_magasin, montant_ttc` plutôt que `SELECT *` : même réponse, **×6** de travail en moins.

5. **Déclarer ce qui n'a pas été mesuré.** Les partitions, PostgreSQL et SQL Server sont **cités** ici ; c'est écrit dans la note de méthode et dans le corps du chapitre.

10. Exercice guidé

Situation. Un rapport de suivi met **841** ms sur la machine de l'analyste. Le collègue en conclut : « il faut indexer la table des ventes ».

Consigne.

1. Reproduisez la mesure et donnez le protocole complet (moteur, nombre d'exécutions, date, instrument). (2 pts)
2. Montrez, plan à l'appui, ce que la requête lit réellement, et quelle correction simple divise son coût. (3 pts)
3. Mesurez l'effet de l'index proposé par le collègue, et rédigez la réponse à lui faire en deux phrases. (3 pts)
4. Ajoutez la ligne de note de méthode que ce rapport doit porter. (2 pts)

11. Exercices autonomes

E1 — La même question, deux sources (40 min). Mesurez le comptage des retours sur la vue puis sur la table matérialisée, cinq fois chacune, et publiez la médiane. Puis répondez en quatre lignes : de combien le rapport est-il plus rapide, et que faut-il écrire dans la note de méthode pour que ce gain soit vérifiable par quelqu'un d'autre.

E2 — L'index, dans les deux moteurs (40 min). Reproduisez la mesure d'index en DuckDB, puis le contrôle croisé en SQLite : relevez les deux plans et le résultat identique. Terminez par trois conseils destinés à un analyste qui doit décider d'indexer — trois phrases, une par critère (moteur, requête, écriture).

12. Correction détaillée

Exercice guidé.

1. **La mesure.** Comptage des retours sur la vue : médiane de **5** exécutions, DuckDB 1.5.5, atelier, **24/09/2026**, instrument `tools/perf_M11.py` — **841** ms si la requête prend aussi `SELECT *`, et **142** ms pour le seul comptage sur la vue.
2. **Le plan.** `EXPLAIN SELECT * FROM ventes_m` annonce **18** colonnes lues, contre **3** pour la version nommée : la correction est d'**écrire les colonnes**, ce qui ramène la mesure de **581** ms à **96** ms (**×6,1**).
3. **L'index.** Sans index **0,6** ms, avec index **0,5** ms, agrégat **1,5** ms dans les deux cas, et le plan reste un `SEQ_SCAN` : « l'index n'est pas le problème : le rapport relit la vue et lit dix-huit colonnes pour en utiliser trois ; corrigeons la lecture, puis remesurons. »
4. **La note de méthode.** « Médiane de **5** exécutions, DuckDB 1.5.5, atelier, valeurs figées le **24/09/2026** dans `03_exercices/dossier_M11/PERF_M11.json` ; partitions, PostgreSQL et SQL Server cités, non exécutés. »

E1 (deux sources). **142** ms contre **1** ms, soit **×142** : le gain est réel mais **conditionnel** — il suppose que la table soit déjà matérialisée, ce qui a un coût (mémoire) et une durée de vie (le temps de la session). La note de méthode doit donc dire **quand** la table est créée et **ce qui se passe** si elle ne l'est pas : un gain non déclaré se transforme en régression chez le collègue qui exécute le rapport sans matérialisation.

E2 (deux moteurs). Trois critères : le **moteur** (DuckDB en colonnes ignore l'index, SQLite en lignes l'utilise) ; la **requête** (recherche ponctuelle ou agrégat complet : un agrégat ne profite pas d'un index) ; l'**écriture** (chaque index ralentit les insertions — à mesurer sur une table alimentée). Le contrôle croisé donne le même résultat dans les deux moteurs — **14** lignes et **5 578 688** FCFA pour le client **15676** — ce qui est la moindre des choses : un index ne change pas la réponse, il change le **chemin**.

13. Mini-projet de chapitre

La note de performance d'un rapport, 2 pages. Choisissez l'un des **12** rapports du projet de module. Mesurez-le sur la vue et sur la table, avec et sans `SELECT *`, et produisez sa note de performance.

Livrables et barème.

Critère	Ce qui est vérifié	Points
Le protocole	médiane de 5 exécutions, moteur et version, date, instrument nommé	5
Les mesures	deux sources × deux façons d'écrire la projection, valeurs publiées	5
Le plan	lecture commentée du plan de la requête (étages, colonnes lues)	4
La note de méthode	ce qui a été mesuré, ce qui ne l'a pas été, et la décision d'optimisation	4
Total		18

14. Résumé du chapitre

La performance d'un rapport se **lit** avant de se mesurer, et se **mesure** avant de s'optimiser. Sur le socle du module : une vue relit sa source et coûte **142** ms là où la table équivalente coûte **1** ms (**×142**) ; `SELECT *` lit **18** colonnes au lieu de **3**, ce que le plan annonce (**×6**) et que l'horloge confirme (**581** ms contre **96** ms, **×6,1**) ; un index ne change **rien** en DuckDB (**0,6** ms contre **0,5** ms, plan toujours `SEQ_SCAN`) mais transforme le plan SQLite en `SEARCH`, pour la même réponse (**14** lignes, **5 578 688** FCFA) ; une fenêtre coûte **150** ms là où la sous-requête scalaire en coûte **292** (**×1,9**), alliant ici lisibilité et vitesse ; et matérialiser a un plancher (**536 576** octets pour **154** lignes), ce qui justifie un socle en script plutôt qu'en fichier. Enfin, aucun de ces chiffres ne serait publiable sans protocole : médiane de **5** exécutions, DuckDB 1.5.5, valeurs gelées le **24/09/2026** dans `PERF_M11.json`.

15. À retenir

1. **Une vue relit sa source à chaque appel.** **142** ms contre **1** ms pour le même comptage.
2. **Le `SELECT *` se voit dans le plan avant de coûter.** **18** colonnes annoncées contre **3**.
3. **Le plan se lit de bas en haut**, et trois questions suffisent : colonnes lues, opérateur dominant, étage inutile. Sur le socle : **3** étages.
4. **L'index est une réponse à un plan, pas une habitude.** Aucun effet mesurable en DuckDB ; plan transformé en SQLite.
5. **Un temps sans protocole n'est pas une mesure.** Médiane de **5** exécutions, moteur, date, instrument.
6. **Les valeurs de performance se gèlent dans un fichier daté.** Deux exécutions du relevé doivent donner le même document.
7. **La première optimisation est de lire ce qu'on utilise** — pas d'ajouter un index.

À retenir. Le chiffre qui résume le chapitre est un écart entre deux façons d'écrire la même question : **×6,1**. Rien n'a changé dans le métier — seul le nombre de colonnes lues a changé. C'est l'optimisation la moins coûteuse et la plus souvent oubliée.

À retenir. Le protocole fait partie du résultat. « **0,5 ms** » sans protocole ne vaut rien ; « médiane de **5** exécutions, DuckDB 1.5.5, **24/09/2026**, `PERF_M11.json` » se vérifie, se compare et se défend.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien coûte le comptage des retours sur la vue, et sur la table équivalente ?
2. Combien de colonnes le plan annonce-t-il pour un `SELECT *`, et combien pour trois colonnes nommées ?
3. Quels sont les trois étages du plan du croisement magasin / chiffre d'affaires ?
4. Quels temps et quels plans donnent la recherche par client, avec et sans index, en DuckDB ?
5. Que répond le plan SQLite de la même recherche, sans index puis avec ?
6. Quel résultat commun les deux moteurs renvoient-ils pour le client 15676 ?
7. Que coûtent la fenêtre et la sous-requête scalaire sur la même part, et lequel des deux est le plus lisible ?
8. Combien pèse un fichier DuckDB contenant une table de 154 lignes, et qu'en déduit-on pour le socle du module ?
9. Que contient une note de méthode de performance, en cinq points ?
10. Pourquoi les valeurs de performance du module sont-elles gelées dans un fichier, et de quelle date ?

Réponses. 1. **142** ms sur la vue, **1** ms sur la table (**×142**). 2. **18** contre **3** (rapport **×6** dans le plan, **×6,1** à l'horloge : **581** ms contre **96** ms). 3. `HASH_GROUP_BY`, `PROJECTION`, `HASH_JOIN`. 4. **0,6** ms sans index, **0,5** ms avec, plan `SEQ_SCAN` dans les deux cas. 5. `SCAN ventes_t` puis `SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)`. 6. **14** lignes et **5 578 688** FCFA. 7. **150** ms pour la fenêtre, **292** ms pour la sous-requête scalaire (**×1,9**) : la fenêtre est aussi la plus lisible. 8. **536 576** octets — un plancher fixe : le socle reste un script plutôt qu'un fichier de base. 9. Moteur et version, protocole, date, emplacement des valeurs, et ce qui n'a pas été mesuré. 10. Pour que deux exécutions du relevé donnent le même document ; elles sont gelées le **24/09/2026** dans `PERF_M11.json`.

Module M11.C07 — La requête comme livrable : style, convention, tests de non-régression

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 3 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C06 ; M07.C08 (qualité, contrôles) ; M05.C07 (documentation d'un classeur).

L'idée du chapitre. Une requête juste qu'on ne peut pas relire est une requête qu'on ne peut pas corriger. Ce dernier chapitre du module traite les requêtes comme des **livrables** : un en-tête qui dit la question métier, un nommage qui se relit six mois plus tard, et des **tests de non-régression** qui refusent qu'un chiffre publié change sans que personne ne s'en aperçoive. Le module en fournit la preuve la plus concrète : `tools/controle_sql_M11.py` rejoue **18** requêtes et confronte chaque résultat au fichier publié — **18** contrôles, **18** accords. Et il porte un avertissement : un test qui **réécrit** la requête ne teste plus la même chose. Sur le socle, un classement des clients par nombre de tickets classé en ordre croissant au lieu de décroissant donne **23 497** au lieu de **23 444** — même données, même requête « à peu près », deux chiffres.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql) · instrument `tools/controle_sql_M11.py`. Les chiffres de chapitre sont ceux du module, gelés dans `chiffres_cites.json` ; les sorties montrées sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Écrire un en-tête de requête** qui porte la question métier, le périmètre, la date et l'auteur.
2. **Nommer** les étapes, les alias et les colonnes de sortie pour qu'une relecture soit possible.
3. **Appliquer les conventions du manuel** : mots-clés en majuscules, une requête pour une question, pas de `SELECT *`, unités déclarées, apostrophes échappées.
4. **Écrire un test de non-régression** qui rejoue une requête et compare son résultat à une valeur publiée.
5. **Construire un contrôle croisé** entre deux sources indépendantes, et **expliquer un écart** au lieu de le masquer.
6. **Rejouer une requête à l'identique** dans un test — et savoir ce qu'on mesure quand on la réécrit.
7. **Livrer un rapport SQL** : requêtes, tests, note de méthode, et la liste de ce qui n'a pas été vérifié.

2. Pourquoi cette notion est importante

Le module M11 publie **12** rapports, **20** requêtes de référence et **18** requêtes auto-testées. Chaque chiffre de ces rapports se retrouve un jour dans une réunion, puis dans un budget. Le mode de défaillance n'est pas l'erreur grossière : c'est le chiffre qui change **sans que personne ne le remarque** — un libellé corrigé dans le référentiel, une ligne de retour ajoutée, un filtre déplacé d'une clause à l'autre.

Les tests de non-régression existent pour cela, et ils coûtent étonnamment peu : **18** requêtes rejouées, comparées à un fichier publié, un verdict par ligne et un code de sortie. Le jour où un chiffre bouge, la cause est identifiée en une minute au lieu d'une réunion.

Le chapitre porte aussi un piège de méthode, mesuré sur le socle. Un contrôle sur le nombre de tickets par client donne **23 444** si la requête source classe en ordre **décroissant** — et **23 497** si le test la classe en ordre **croissant**. Personne n'a touché aux données : c'est le test qui a changé de question. Un test de non-régression qui ne rejoue pas la requête **à l'identique** ne protège rien ; il fabrique des alertes.

3. Explication simple — la fiche de recette et le contrôle qualité

Une requête livrée, c'est une fiche de recette. En haut : le nom du plat, pour combien de personnes (le périmètre), la date, qui l'a écrite. Ensuite les étapes, numérotées et nommées. En bas : ce qui peut rater. Une requête sans en-tête oblige le suivant à deviner la question — et à la deviner faux.

Un test de non-régression, c'est le contrôle qualité du rayon. On ne regoûte pas le plat chaque matin ; on vérifie que l'étiquette dit la même chose que le contenu. Sur le module : **18** étiquettes, **18** contenus, et l'accord est vérifié à chaque exécution.

Un contrôle croisé, c'est peser la farine sur deux balances. Deux instruments différents, la même grandeur : si les deux donnent **15 595 154 955** FCFA, on avance ; s'ils diffèrent, on cherche **lequel** a bougé avant de publier. Le module compare ainsi une mesure DuckDB et une mesure pandas.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
en-tête de requête	query header	Le bloc de commentaires qui précède une requête : question, périmètre, source, date, auteur.
convention de nommage	naming convention	Les règles partagées sur les noms de tables, de colonnes et d'étapes intermédiaires.
test de non-régression	regression test	Un test qui rejoue une requête et compare son résultat à une valeur attendue.
valeur attendue	expected value	La valeur publiée à laquelle le test compare le résultat — ici, celle du fichier <code>chiffres_cites.json</code> .
contrôle croisé	cross-check	La vérification d'une même grandeur par deux chemins indépendants (deux moteurs, deux méthodes).
code de sortie	exit code	Le signal rendu par un contrôle : zéro si tout passe, non nul sinon — c'est ce qui l'automatise.
jeu de tests	test set	L'ensemble des contrôles d'un module ; ici, 18 requêtes rejouées.
requête l'identique	à verbatim replay	La règle d'or d'un test : rejouer exactement la requête publiée, sans la réécrire.
écart expliqué	explained gap	La différence entre deux socles, documentée (périmètre, période, retours) et jamais additionnée.

Définition. Un **test de non-régression** est un contrôle qui rejoue une requête **publiée** et compare son résultat à une **valeur attendue**. Sa valeur n'est pas de trouver des erreurs : elle est de **signaler un changement**. Un test qui échoue ne dit pas « le rapport est faux », il dit « quelque chose a bougé, et il faut savoir quoi avant de livrer ». C'est une alerte, pas un verdict.

Définition. Un **contrôle croisé** vérifie une même grandeur par deux chemins **indépendants** : deux moteurs (DuckDB et SQLite), deux outils (SQL et pandas), ou deux socles (le socle M11 et celui de M09). Il porte une conclusion plus forte qu'un simple test, à une condition : les deux chemins ne doivent pas partager la même source d'erreur.

Définition. Le **code de sortie** d'un contrôle est le signal qu'il rend au système : zéro si tous les contrôles passent, non nul sinon. C'est ce détail — et non l'affichage — qui rend un jeu de tests utilisable dans une chaîne automatique : un rapport qui se construit sans que les contrôles aient tourné n'est pas un rapport vérifié.

5. Cours approfondi

5.1 L'en-tête de requête : la question avant le code

Le manuel impose un en-tête court, en commentaires, à toute requête livrée :

```
-- Q07 · Part du chiffre d'affaires des 10 % de clients les plus gros
-- Périmètre : ventes nettes (retours exclus), 01/01/2023 au 31/08/2026, 5 magasins, client 0
includ
-- Source   : 03_exercices/dossier_M11/socle_m11.sql (vue ventes)
-- Sortie   : une ligne par centile, plus la part cumulée
-- Publiée  : 38,1 % (voir chiffres_cites.json, clé m11_concentration_top10_pct)
```

Ces cinq lignes répondent aux cinq questions qu'un relecteur se pose — et elles sont aussi ce dont un test de non-régression a besoin pour vérifier la **bonne** valeur : une requête sans périmètre écrit ne permet pas de savoir si les **38,1 %** publiés et les **24,2 %** hors client non identifié parlent du même ensemble.

Conseil professionnel. Écrivez l'en-tête **avant** la requête, pas après. Tant que le périmètre n'est pas rédigé, la requête n'est pas finie : c'est en écrivant la ligne « Périmètre » qu'on découvre qu'on hésite entre ventes brutes et ventes nettes — et cette hésitation est exactement ce qui fait publier deux chiffres différents du même indicateur.

5.2 Nommer : les étapes, les alias, les colonnes

Trois règles, et elles suffisent :

1. **Une étape, un nom qui dit son rôle.** `nets`, `ca_mois`, `rang_magasin` — jamais `tmp1`, `tmp2`, `t3`.
2. **Un alias qui se lit à voix haute.** `SUM(montant_ttc) AS ca_net`, et non `AS s`. Une colonne de sortie est lue par quelqu'un qui n'a pas écrit la requête.
3. **Un nom de colonne de sortie en français**, comme le reste du manuel : `part_pct`, `rang`, `mois`.

Attention. Un alias muet est un défaut de livraison, pas un détail esthétique. `SELECT COUNT(*) AS n1, SUM(x) AS s` sort une table publiée avec deux colonnes nommées `n1` et `s` : l'analyste qui reçoit ce fichier doit revenir demander ce que contiennent les colonnes — et il posera la question à quelqu'un qui ne se souvient plus.

5.3 Un style qui se relit

Les conventions du manuel, appliquées dans tous les chapitres du module :

Règle	Exemple
Mots-clés en majuscules, identifiants en minuscules	<code>SELECT id_magasin FROM ventes WHERE NOT est_retour</code>
Une requête par question ; une question par requête	Le titre de l'en-tête est une phrase, pas un thème
Pas de <code>SELECT *</code> dans un rapport	colonnes nommées (C06 : 581 ms contre 96 ms)
Alias de table courts mais lisibles	<code>ventes v JOIN magasin m USING (id_magasin)</code>
Unités écrites dans le nom de la colonne	<code>ca_net_fcfa</code> , <code>part_pct</code> , <code>duree_jours</code>
Apostrophes échappées dans les filtres	<code>mode_paiement = 'Créance 30 j'</code> — et non des guillemets doubles
Découpage en CTE plutôt qu'en requêtes imbriquées	une étape nommée par intention (C05)

5.4 Les trois tests de non-régression du module

Le module en compte 3, et chacun protège autre chose :

Test	Ce qu'il protège	Instrument
Rejeu des 18 requêtes de référence contre les valeurs publiées	les chiffres du module	<code>tools/controle_sql_M11.py</code>
Comparaison de deux relevés du socle	la stabilité du fichier de valeurs	<code>chiffres_manuel.py</code> M11 (deux exécutions, <code>diff = 0</code> ligne)
Comparaison des deux socles (M11 et M09)	le périmètre des rapports	rapport R12 du projet

Le premier est exécuté ici, dans son intégralité :

```
=== controle SQL M11 : 18 requetes, une source par cote ===
cote SQL : socle_m11.sql rejoue par DuckDB 1.5.5
cote publie: chiffres_cites.json (265 cles M11)

| 1 | m11_lignes | 240000 | 240000 | OK |
| 2 | m11_lignes_hors_retours | 237191 | 237191 | OK |
| 3 | m11_retours | 2809 | 2809 | OK |
| 4 | m11_tickets | 145212 | 145212 | OK |
| 5 | m11_clients_vente | 23497 | 23497 | OK |
| 6 | m11_magasins | 5 | 5 | OK |
| 7 | m11_produits | 154 | 154 | OK |
| 8 | m11_ca_2023 | 3443581547 | 3 443 581 547 FCFA | OK |
| 9 | m11_ca_8_mois_2026 | 3360553372 | 3 360 553 372 FCFA | OK |
|10 | m11_client0_lignes | 42658 | 42658 | OK |
|11 | m11_client0_ca | 2852612447 | 2 852 612 447 FCFA | OK |
|12 | m11_c02_max_rank | 23444 | 23444 | OK |
|13 | m11_c02_max_dense_rank | 21 | 21 | OK |
|14 | m11_c03_ratio_saison | 1.65 | 1.65 | OK |
|15 | m11_c04_rfm_blocs | 7833 | 7 833 / 7 832 / 7 832 | OK |
|16 | m11_c04_oneshot | 64 | 64 | OK |
|17 | m11_c05_grouping_sets_lignes | 47 | 47 | OK |
|18 | m11_calendrier_jours | 1339 | 1339 | OK |

18 controles sur 18 : OK – le socle et le fichier publie disent la meme chose.
```

Ce tableau est un **contrôle croisé**, au sens strict : le côté gauche est une requête exécutée à l'instant, le côté droit un fichier écrit plus tôt — parfois par une autre exécution, parfois sur une autre machine (le fichier de valeurs a été produit par `chiffres_manuel.py`). Les deux chemins ne partagent ni le moment, ni le code : c'est ce qui donne sa valeur à l'accord.

Définition. La **valeur attendue** d'un test est la valeur **publiée** — ici, celle du fichier `chiffres_cites.json` — et non la valeur qu'on vient de lire à l'écran. C'est cette distinction qui fait la force du contrôle : le test ne compare pas le socle à lui-même, il compare le socle **aujourd'hui** à ce que le module affirmait **avant**. Sans valeur attendue écrite quelque part, un test ne peut que constater que « ça tourne ».

5.5 Le contrôle entre deux sources : expliquer un écart, jamais l'additionner

Le module travaille sur **2** socles. Le socle M11 — celui des chapitres — compte **240 000** lignes de ventes. Le socle de M07/M09, construit pour la quincaillerie, en compte **50 008** et affiche **7 908 259 732** FCFA bruts, **7 876 320 165** hors retours. Les additionner n'aurait aucun sens : ils ne mesurent ni la même période, ni le même périmètre.

Le rapport R12 du projet fait le contraire : il **confronte** les deux et **explique** l'écart — lignes, retours, magasins, période. C'est le troisième test du module, et le plus formateur : il n'y a pas de « bonne » réponse à trouver, il y a une différence à documenter. Un rapport qui publie un écart sans l'expliquer laisse à son lecteur le soin d'imaginer une erreur — et le lecteur imaginera la pire.

5.6 Un test rejoue la requête à l'identique

Voici la même question posée à deux ordres de tri près :

```
-- version publiée (ordre décroissant) : rang maximum 23444
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1),
      r AS (SELECT RANK() OVER (ORDER BY t DESC) rk FROM c) SELECT MAX(rk) FROM r;
-- version « équivalente » écrite dans un test (ordre croissant) : rang maximum 23497
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1),
      r AS (SELECT RANK() OVER (ORDER BY t) rk FROM c) SELECT MAX(rk) FROM r;
```

23 444 contre **23 497**. La première valeur est celle que le module publie (avec **21** comme plus grand `DENSE_RANK`), la seconde est le nombre de clients — un résultat qui a l'air correct et qui ne répond pas à la question posée. Un test qui aurait « réécrit proprement » la requête aurait donc signalé un écart inexistant, ou pire : validé **23 497** et fait diverger le module de son propre fichier.

Attention. Un test de non-régression recopie la requête **caractère pour caractère** — ordre de tri, filtre de retour, périmètre, jointure. Toute « amélioration » du texte dans un test est une nouvelle requête, donc un nouveau résultat : ce n'est plus un test, c'est une source de bruit. Le corollaire est agréable : quand un test échoue, la première chose à vérifier n'est pas la donnée, c'est **la requête du test**.

Dans les faits. Le contrôle du module n'a pas seulement validé **18** chiffres : il en a **corrigé** un. Le contrôle du client **0** a d'abord compté **43 161** lignes, là où le module publie **42 658** — l'écart venait de la requête de contrôle, qui comptait aussi les lignes de retour, alors que la valeur publiée est en ventes nettes. La donnée était bonne, la mesure était fausse : c'est exactement ce qu'un contrôle croisé doit détecter, et il vaut mieux qu'il le fasse sur un contrôle que sur un rapport.

5.7 Ce que ce chapitre ne couvre pas

Les tests d'**intégration** et la chaîne d'automatisation (exécuter les contrôles à chaque modification, refuser la livraison en cas d'échec, notifier l'équipe) relèvent de l'ingénierie et non de l'analyse : ils sont cités ici et traités dans l'option M17. Le principe utile à retenir reste accessible : un contrôle qui ne s'exécute pas tout seul finira par ne plus s'exécuter du tout.

6. Exemple concret — la revue d'un rapport avant envoi

Un rapport est prêt. La revue tient en dix minutes et cinq questions, dans cet ordre :

#	Question	Réponse attendue
1	L'en-tête dit-il la question, le périmètre et la source ?	cinq lignes, pas zéro
2	Les colonnes de sortie portent-elles leurs unités ?	ca_net_fcfa, part_pct
3	Reste-t-il un <code>SELECT *</code> ?	non (C06 : 581 ms contre 96 ms)
4	Les classements ont-ils une clé de départage ?	oui (C04 : 3 316 clients à 8 tickets)
5	Les chiffres publiés passent-ils les contrôles ?	18 sur 18

Cinq questions, une seule exige un outil, et aucune ne demande de relire la logique métier : c'est le propre d'une revue utile.

7. Démonstration pas à pas — cinq étapes sur le socle

7.1 Étape 1 — écrire l'en-tête

La question, le périmètre, la source, la sortie, la valeur publiée : cinq lignes avant la première ligne de SQL. Sur le socle, la question la plus sensible du module — la concentration — porte deux valeurs selon le traitement du client non identifié (**38,1 %** ou **24,2 %**) : sans en-tête, on ne sait pas laquelle est publiée.

7.2 Étape 2 — nommer les étapes

La CTE `c` (le client et son nombre de tickets) et la CTE `r` (le rang) du §5.6 : deux étapes, deux noms, et une requête qui se relit.

7.3 Étape 3 — rejouer les 18 contrôles

`python3 tools/controle_sql_M11.py` : **18** contrôles, **18** accords, code de sortie nul. Le tableau du §5.4 est la sortie réelle de l'atelier.

7.4 Étape 4 — comparer les deux socles

240 000 lignes contre **50 008**, **15 595 154 955** FCFA contre **7 908 259 732** bruts : l'écart s'explique (période, périmètre, retours) et ne s'additionne pas.

7.5 Étape 5 — documenter ce qui n'a pas été testé

PostgreSQL, SQL Server, les partitions, la montée en charge : **cités**, non exécutés. Une note de méthode qui ne dit que ce qui marche est une publicité, pas une méthode.

8. Erreurs fréquentes

1. **Publier une requête sans en-tête.** Le relecteur ne peut ni vérifier le périmètre ni reproduire le chiffre (**38,1 %** ou **24,2 %** selon le traitement du client non identifié).
2. **Réécrire la requête dans le test.** **23 444** contre **23 497** pour un ordre de tri inversé : le test signale alors un écart qui n'existe pas, et la confiance dans les contrôles en sort abîmée.
3. **Additionner deux socles.** **240 000** lignes et **50 008** lignes ne mesurent pas la même chose : un total commun n'a aucun sens, et il est impossible à défendre.
4. **Chercher les erreurs, pas les changements.** Un test de non-régression ne dit pas « c'est faux » : il dit « quelque chose a bougé ». Confondre les deux fait ignorer les alertes utiles.
5. **Laisser un contrôle sans code de sortie.** Un contrôle qui affiche « OK » sans rendre de code ne s'intègre à rien : il dépend d'un humain qui le lance et qui lit.
6. **Publier des alias muets.** `n1`, `s`, `col3` : le rapport est juste et inutilisable.

9. Bonnes pratiques professionnelles

1. **L'en-tête d'abord.** Cinq lignes : question, périmètre, source, sortie, valeur publiée.
2. **Un test recopie la requête à l'identique.** Aucune reformulation, aucun « nettoyage » : le test est un témoin, pas une réécriture.
3. **Comparer plutôt qu'additionner.** Deux socles se confrontent et s'expliquent (rapport R12) ; ils ne se somment jamais.
4. **Rendre un code de sortie.** Un jeu de tests s'utilise dans une chaîne automatique ou ne sert à rien.
5. **Documenter ce qui n'a pas été vérifié.** Moteurs cités, partitions citées, mesures non faites : la même honnêteté que pour les chiffres (§1.5).

10. Exercice guidé

Situation. Vous reprenez une requête écrite par un collègue : elle calcule la part du chiffre d'affaires des **10 %** de clients les plus gros, sans en-tête, avec des alias `a`, `b`, `c`, et un `SELECT *` dans une CTE intermédiaire. Elle donne **38,1 %**.

Consigne.

1. Écrivez l'en-tête complet de la requête, périmètre compris. (3 pts)
2. Renommez les étapes et les colonnes de sortie, et justifiez deux de vos choix. (2 pts)
3. Écrivez le contrôle de non-régression associé à cette requête, dans l'esprit de `tools/controle_sql_M11.py`. (3 pts)
4. Le même calcul hors client non identifié donne **24,2 %** : comment l'en-tête doit-il le dire, et pourquoi ? (2 pts)

11. Exercices autonomes

E1 — Cinq requêtes, cinq en-têtes (40 min). Reprenez cinq requêtes des chapitres C01 à C06 (celles que vous avez écrites aux exercices) et ajoutez-leur un en-tête complet. Puis relisez-les une par une et notez ce que l'en-tête vous a obligé à préciser que la requête laissait implicite.

E2 — Le jeu de tests d'un rapport (40 min). Choisissez un des **12** rapports du projet, listez les valeurs qu'il publie, et écrivez pour chacune un contrôle : la requête rejouée à l'identique, la clé de la valeur attendue, et ce que l'échec signifierait. Terminez par la phrase qui doit figurer dans la note de méthode, et par celle qui dit ce que vous n'avez pas testé.

12. Correction détaillée

Exercice guidé.

1. **L'en-tête.** Question : « part du chiffre d'affaires net des 10 % de clients les plus gros ». Périmètre : ventes nettes (retours exclus), 01/01/2023 au 31/08/2026, 5 magasins, **client 0 inclus**. Source : `socle_m11.sql`. Sortie : une ligne par centile, avec part cumulée. Publié : **38,1 %** (`m11_concentration_top10_pct`).
2. **Les noms.** `clients_ca` (le chiffre d'affaires par client) et `deciles` (le découpage en centiles) au lieu de `a` et `b` : le nom dit ce que contient l'étape, et la relecture ne demande plus de reconstruire le raisonnement. Les colonnes de sortie portent leurs unités : `ca_net_fcfa`, `part_pct`.
3. **Le contrôle.** Rejeu de la requête à **l'identique**, comparaison à la clé publiée, verdict par ligne, code de sortie non nul en cas d'écart — la structure de `tools/controle_sql_M11.py` (qui en compte 18).
4. **Le client non identifié.** L'en-tête doit dire que le client 0 est inclus — et donner la valeur hors client non identifié (**24,2 %**) dans la note, puisque le retrait est une décision de périmètre : sans cette phrase, deux services publieront deux chiffres différents du même indicateur.

E1 (les en-têtes). Les points que l'en-tête fait apparaître sont toujours les mêmes : le **périmètre** (retours inclus ou non), le **grain** (client, ticket, ligne), la **période**, et le **traitement des cas particuliers** (client 0, magasin sans objectif, cohorte incomplète). C'est la liste des quatre erreurs les plus fréquentes du module, et l'en-tête les rend visibles **avant** la publication.

E2 (le jeu de tests). Un bon jeu de tests couvre trois natures de valeurs : un **volume** (les **240 000** lignes, les **145 212** tickets), un **total** (**15 595 154 955** FCFA, **3 443 581 547** pour 2023) et un **taux** (**38,1 %**, **15,8 %** de rétention à M+1). Trois natures, trois façons de casser : un volume qui bouge signale un changement de filtre ou de source, un total un changement de périmètre, un taux un changement de dénominateur. Et la phrase de méthode : « les contrôles rejouent les requêtes publiées à l'identique ; les moteurs cités et non exécutés ne sont pas testés ».

13. Mini-projet de chapitre

La remise en forme d'un rapport existant, 2 pages. Prenez l'un des 12 rapports du projet de module, réécrivez-le selon les conventions du chapitre (en-tête, nommage, colonnes nommées) et livrez son jeu de tests.

Livrables et barème.

Critère	Ce qui est vérifié	Points
L'en-tête	question, périmètre, source, sortie, valeur publiée	5
Le style	étapes nommées, alias lisibles, unités dans les noms, aucun <code>SELECT *</code>	4
Les tests	requêtes rejouées à l'identique , valeurs attendues citées, code de sortie	6
La note de méthode	ce qui est vérifié, ce qui ne l'est pas, et la limite du rapport	3
Total		18

14. Résumé du chapitre

Une requête livrée sans en-tête est une requête qu'on ne peut pas vérifier : cinq lignes — question, périmètre, source, sortie, valeur publiée — suffisent à la rendre reproductible. Le style n'est pas une coquetterie : les étapes nommées, les alias lisibles et les unités dans les noms de

colonnes font la différence entre un rapport qu'on relit et un rapport qu'on refait. Les **3** tests de non-régression du module couvrent les chiffres (**18** contrôles rejoués contre le fichier publié), la stabilité du fichier de valeurs (deux relevés, `diff = 0` ligne) et le périmètre des rapports (comparaison des deux socles, **240 000** lignes contre **50 008**). Deux règles portent tout le reste : un test recopie la requête à l'**identique** — l'ordre de tri inversé donne **23 497** au lieu de **23 444**, et ce n'est pas une erreur de donnée mais une erreur de test — et un contrôle qui ne rend pas de **code de sortie** ne sert à rien.

15. À retenir

1. **L'en-tête d'abord** : la question, le périmètre, la source, la sortie, la valeur publiée.
2. **Un nom dit un rôle**. `clients_ca`, `rang_magasin`, jamais `tmp`, `t3`, `n1`.
3. **Un test rejoue la requête à l'identique**. Réécrire la requête du test, c'est changer de question.
4. **Un test signale un changement, il ne prouve pas une erreur**. C'est une alerte, pas un verdict.
5. **Deux socles se comparent, ils ne s'additionnent pas** : **240 000** lignes contre **50 008**.
6. **Un contrôle rend un code de sortie**, sinon il dépend d'un humain qui se souvienne de le lancer.
7. **La note de méthode dit aussi ce qui n'a pas été testé**.

À retenir. Le chiffre qui résume le chapitre est une paire : **23 444 / 23 497**. Même requête, même socle, même question — à un ordre de tri près. Un test qui n'aurait pas recopié la requête publiée aurait signalé une fausse alerte, ou validé un chiffre faux. C'est le seul chapitre du module où la rigueur porte sur **le contrôle lui-même**.

À retenir. **18** contrôles, **18** accords, et un code de sortie nul : voilà à quoi ressemble un module qu'on peut livrer. Le jour où l'un des **18** passe au rouge, on saura **quoi** regarder — et c'est tout ce qu'on demande à un test.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Que contient l'en-tête d'une requête livrée, et pourquoi le périmètre y figure-t-il en premier ?
2. Combien de tests de non-régression compte le module, et que protège chacun ?
3. Combien de contrôles rejoue `tools/controle_sql_M11.py`, et contre quelle source ?
4. Que donne le rang maximum des clients par nombre de tickets en ordre décroissant, et en ordre croissant ?
5. Pourquoi un test de non-régression ne doit-il jamais réécrire la requête qu'il rejoue ?
6. Que fait un contrôle croisé de plus qu'un simple test, et à quelle condition ?
7. Combien de lignes comptent les deux socles du module, et pourquoi ne pas les additionner ?
8. Pourquoi un contrôle doit-il rendre un code de sortie ?
9. Citez trois natures de valeurs qu'un jeu de tests doit couvrir.
10. Que doit dire la note de méthode d'un rapport, en plus de ce qu'il vérifie ?

Réponses. 1. Question, périmètre, source, sortie, valeur publiée — le périmètre d'abord, parce que c'est lui qui décide de la valeur publiée (**38,1 %** ou **24,2 %** selon le client non identifié). 2. **3** : les chiffres du module, la stabilité du fichier de valeurs, et le périmètre des rapports entre les deux socles. 3. **18** contrôles, contre les valeurs publiées de `chiffres_cites.json` (côté SQL : le socle rejoué). 4. **23 444** en ordre décroissant, **23 497** en ordre croissant. 5. Parce que toute ré-

écriture est une autre requête : on testerait un résultat qui n’a jamais été publié. 6. Il vérifie une même grandeur par deux chemins indépendants — à condition que les deux ne partagent pas la même source d’erreur (ici : une requête exécutée contre un fichier écrit plus tôt). 7. **240 000** lignes et **50 008** : périodes, périmètres et retours différents — les additionner produirait un total indéfendable. 8. Pour pouvoir l’enchaîner dans un traitement automatique, sans dépendre d’un humain qui lit la sortie. 9. Un volume, un total, un taux. 10. Ce qui n’a **pas** été vérifié ou exécuté (moteurs cités, partitions, montée en charge).

Projet M11.P — « Les 12 rapports SQL de la cellule commerciale »

Module M11 · projet noté sur 20 · seuil 13 · durée imposée 4 h (chronomètre en marche) · rendu : 4 livrables — P1 les 12 requêtes (8 points), P2 le jeu de tests (5 points), P3 la note de lecture (4 points), P4 la fiche de reprise (3 points).

Ce que le projet évalue. Non pas « la requête la plus savante », mais la **bibliothèque de rapports** : douze requêtes autonomes, **versionnées**, **testées** et **commentées en français**, chacune répondant à une question écrite. Vous recevez la commande de la cellule commerciale et le socle du module : **240 000** lignes de vente, **237 191** hors retours, **145 212** tickets, **23 497** clients, **5** magasins, **44** mois. La règle du projet est celle du module : **un rapport dont le total diffère d'un rapport à l'autre est rejeté**, même si chaque requête « fonctionne » — et un rapport que son lecteur ne peut pas rejouer ne vaut rien.

1. Énoncé

La commande

La cellule commerciale d'une enseigne de quincaillerie (**5** magasins, **44** mois, **240 000** lignes) commande sa **bibliothèque de rapports** : **12** requêtes autonomes, chacune répondant à une question qu'elle s'est posée cette année. Elle les veut **en français**, **datées**, **testées**, et livrées avec leur **mode d'emploi** — pas un dossier de requêtes, un **outil de travail**.

Le matériel

Élément	Chemin	Ce que c'est
Le socle	03_exercices/dossier_M11/socle_m11.sql	les 5 tables de référence et les 2 vues de ventes (3 595 octets)
La connexion	03_exercices/dossier_M11/connexion.py	ouvrir(materialiser=False) — la matérialisation en mémoire pour les mesures
Les mesures de performance	03_exercices/dossier_M11/PERF_M11.json	les temps gelés du chapitre C06 (24/09/2026 , médianes de 5 exécutions)
Le contrôle automatique	tools/controle_sql_M11.py	18 requêtes rejouées et confrontées au fichier publié — le calque de votre livrable P2
Les valeurs publiées	01_socle_donnees/data/reference/chiffres_cites.json	clés m11_*, m11p_*, m11e_* : 301 clés pour le module
Les 7 chapitres	02_modules/M11_C01...C07	fenêtres, rangs, temps, cohortes, requêtes avancées, plans, style et tests

Les six totaux de contrôle

Ces six nombres se recalculent depuis le socle — ils ne se recopient pas. Un rapport qui publie un total différent doit **dire pourquoi** (périmètre plus étroit, période partielle, client non identifié exclu) : c'est la seule tolérance admise, et elle s'écrit dans l'en-tête.

Contrôle	Valeur attendue
Chiffre d'affaires net (retours exclus)	15 595 154 955 FCFA
Lignes de vente (retours exclus)	237 191

Contrôle	Valeur attendue
Tickets de vente	145 212
Clients identifiés dans les ventes	23 497
Magasins	5
Mois couverts	44

2. Les 12 questions métier (le catalogue imposé)

Chaque rapport répond à **une** question, citée telle quelle dans son en-tête. Le chapitre indiqué est celui qui fournit la méthode ; la colonne « sortie » dit ce que le rapport doit **contenir**, pas la façon de l'obtenir.

Code	Question de la cellule	Chap.	Sortie attendue
R01	Combien avons-nous vendu chaque mois, et où en sommes-nous cette année ?	C01, C02	mois, CA, cumul, part du mois dans son année
R02	Quels magasins vendent le plus — et lesquels ont le plus gros panier ?	C01, C02	deux classements côte à côte, chacun avec son rang
R03	Quels sont nos 3 meilleurs produits par famille ?	C02	3 lignes par famille, rang, part dans la famille
R04	Quelle part du chiffre d'affaires font nos plus gros clients ?	C01, C04	parts cumulées, top 1 %, top 5 %, top 10 %, top 20 %
R05	L'activité progresse-t-elle, à périmètre comparable ?	C03	année sur année à mois constants, moyenne des variations
R06	La tendance de fond monte-t-elle ?	C03	moyennes mobiles de 3 et 12 mois, avec leur compteur de fenêtre
R07	Tenons-nous nos objectifs ?	C03	réalisé, objectif, taux — trous compris et déclarés
R08	Quel jour de la semaine travaillons-nous le mieux ?	C03	CA par jour de vente, écart entre le meilleur et le pire
R09	Nos clients reviennent-ils ?	C04	rétenion pondérée à M+1, M+3, M+12, avec les effectifs
R10	Quels clients appeler en priorité, et pour quoi ?	C04	segments RFM, effectif, part du CA, plage de récence
R11	Où vend-on quoi ?	C05	tableau croisé quartier × famille avec sous-totaux
R12	Nos deux socles racontent-ils la même histoire ?	C07	comparaison documentée des deux socles, écarts expliqués

Douze questions, douze rapports, une seule vérité de total : les 12 rapports publient le même chiffre d'affaires net quand ils parlent de la même période — **15 595 154 955 FCFA** sur le socle complet, ou un sous-ensemble dont le périmètre est écrit noir sur blanc.

Le format d'en-tête, imposé

Chaque rapport s'ouvre sur ces six lignes — c'est ce qui distingue une bibliothèque d'un fichier de brouillons :

```
-- Rxx – <question de la cellule, recopiée telle quelle>
-- Hypothèses : <ce qu'on suppose : retours exclus, client 0 inclus, période complète...>
-- Grain : <une ligne = un mois / un client / un couple quartier-famille>
-- Source : <socle M11, vue ventes, référence produit>
-- Total de contrôle : <le nombre que le rapport doit rendre>
-- Date : <le jour de rédaction>
```

3. Les 4 livrables

P1 — Les 12 requêtes (8 points)

Une requête par question, dans l'ordre R01 → R12, chacune avec **son en-tête complet**, un ORDER BY **explicite** — avec sa clé de départage quand deux lignes peuvent être à égalité —, et son **total de contrôle**.

Ce qui est vérifié à la correction :

- R02 : les deux classements sont donnés avec leur **rang** ; le rapport signale que l'ordre n'est pas le même (**Kaya** a le plus gros panier, **Ouagadougou** le plus gros chiffre d'affaires).
- R03 : le classement porte sur les **7** familles réelles, pas sur les **16** libellés du référentiel ; le rapport publie les **deux** comptes (**46** lignes contre **21**).
- R04 : la part du top 10 % est publiée **avec** et **sans** le client non identifié, et la définition du décile est écrite.
- R05 : aucune comparaison entre périodes de longueurs différentes ; la période partielle est annoncée.
- R06 : chaque moyenne mobile porte le nombre de lignes de sa fenêtre ; les douze premiers mois sont marqués comme **non comparables**.
- R07 : les **2** couples magasin-mois sans objectif apparaissent dans le rapport, avec un état explicite et **93 749 490** FCFA de ventes sans référence.
- R09 : la rétention est **pondérée** par la taille des cohortes ; le tableau porte les effectifs.
- R10 : le traitement du client **0** est écrit ; tous les classements sont départagés.
- R11 : les sous-totaux sont produits dans la **même** requête (ROLLUP ou GROUPING SETS), et le total général égale le chiffre d'affaires net.

P2 — Le jeu de tests (5 points)

Chaque requête est livrée avec **au moins un test de non-régression**, dans l'esprit de `tools/controle_sql_M11.py` : une valeur attendue (le total, le nombre de lignes, une valeur extrême), la requête rejouée **à l'identique**, le verdict, et un **code de sortie** utilisable en chaîne.

Ce qui est vérifié :

- **au moins un** test par rapport, soit **12** au minimum, plus la sortie de l'instrument du module (**18** lignes) publiée ;
- les tests **recopient** la requête du rapport caractère pour caractère : la plus petite réécriture change la réponse — un classement des clients par nombre de tickets rend **23 444** en ordre décroissant et **23 497** en ordre croissant ;
- un écart produit un **code de sortie non nul** : un test qui se contente d'afficher « échec » ne protège rien.

Attention. Un test qui « améliore » la requête qu'il rejoue ne teste plus le chiffre publié : il fabrique une alerte, ou pire, il valide une valeur qui n'apparaît nulle part dans le rapport.

P3 — La note de lecture (4 points · 2 pages)

Deux pages, pour la personne qui n'écrira jamais une ligne de SQL : ce que les **12** rapports disent **ensemble**, et les **3** précautions à connaître avant de citer un chiffre :

1. le client **0** — **18,3 %** du chiffre d'affaires, absent du référentiel, premier de tout classement ;
2. les **retours** — **2 809** lignes négatives : un total « net » qui les compte n'est pas un total ;
3. les **grains** — ligne, ticket, client : trois nombres pour la même question (**146 161** tickets au total, **145 212** avec au moins une vente, **2 797** qui ne portent que des retours).

La note se termine par **la question à laquelle les 12 rapports ne répondent pas** — par exemple pourquoi un client achète, ou ce qui se passera au prochain trimestre : un rapport décrit, il n'explique pas.

P4 — La fiche de reprise (3 points)

Un pair — qui n'a pas écrit les rapports — en reprend **3**, exécute leurs tests, et note **ce qui manque** pour qu'il puisse travailler sans vous : hypothèse non écrite, colonne ambiguë, test absent, nom de table supposé connu. La fiche publiée est la **sienne**, pas la vôtre ; votre annotation de son retour tient en trois lignes.

Conseil professionnel. Écrivez l'en-tête **avant** la requête. Tant que la ligne « Hypothèses » n'est pas rédigée, la requête n'est pas finie : c'est en l'écrivant qu'on découvre qu'on hésite entre ventes brutes et ventes nettes — et cette hésitation est exactement ce qui fait publier deux chiffres différents du même indicateur.

4. Barème

Livrable	Critère	Points
P1 · les 12 requêtes	12 requêtes rendues, dans l'ordre R01 → R12	4
	Les 12 en-têtes complets (question, hypothèses, grain, source, total, date)	2
	ORDER BY explicite et clé de départage partout où une égalité est possible	1
	Total de contrôle présent et exact	1
P2 · le jeu de tests	Au moins 1 test par rapport (12 au minimum)	2
	Requêtes rejouées à l'identique (aucune réécriture)	1,5
	Sortie de <code>tools/controle_sql_M11.py</code> publiée (18 lignes)	1
	Code de sortie non nul en cas d'écart	0,5
P3 · la note de lecture	Les 3 précautions (client 0 , retours, grains), chacune chiffrée	2
	Ce que les 12 rapports disent ensemble , en une page	1
	La question à laquelle ils ne répondent pas	1
P4 · la fiche de reprise	Les 3 rapports repris par un pair, tests exécutés	2
	Le compte rendu de ce qui manque, et votre réponse en trois lignes	1
Total		20

Seuil de validation : 13/20. Un livrable non rendu vaut zéro : il n'y a pas de points de présence.

5. Ce qui fait rejeter un livrable

1. **Un total qui change d'un rapport à l'autre** sans explication de périmètre : la règle du projet, et la seule faute éliminatoire.
2. **Un classement sans clé de départage.** 3 316 clients partagent exactement 8 tickets (14,1 % du fichier) : la frontière des tiers a compté de 2 718 à 2 738 clients selon l'exécution, sans qu'aucune donnée n'ait changé.
3. **Une jointure interne vers une table de référence** qui supprime en silence les 2 mois sans objectif du magasin 4.
4. **Une comparaison de périodes de longueurs différentes** présentée comme une évolution : 2026 (8 mois) contre 2025 (12 mois) donne **-28,9 %** ; à périmètre égal, **+15,4 %**.
5. **Un rapport dont les chiffres ne sont pas dans le fichier publié** : toute valeur citée se retrouve dans `chiffres_cites.json`, ou n'est pas publiée.
6. **Un test qui réécrit la requête qu'il contrôle** : ce n'est plus le chiffre du rapport qui est vérifié.
7. **Un moteur annoncé sans l'être** : PostgreSQL et SQL Server sont **cités** dans le module, jamais exécutés ; un rapport qui publie une sortie « PostgreSQL » non produite est faux.

6. Déroulé conseillé (4 séances, environ 8 h)

Séance	Objet	Livrable visé
1	Lire la commande, rejouer le socle, recalculer les 6 totaux, écrire les en-têtes des 12 rapports	P1 (en-têtes)
2	Écrire R01 → R06, avec sorties réelles et totaux de contrôle	P1
3	Écrire R07 → R12 (trous d'objectif, cohortes pondérées, RFM départagé, tableau croisé, contrôle croisé)	P1
4	Le jeu de tests, la note de lecture, l'échange de relecture avec un pair	P2, P3, P4

7. Comment vous corriger vous-même

1. **Les totaux** : `python3 tools/controle_sql_M11.py` doit rendre **18** lignes au vert sur le socle livré ; vos **12** tests s'ajoutent à la suite, dans le même format.
2. **Les sorties** : recopiez les sorties réelles de votre atelier, jamais une sortie approximative ; un tableau publié avec des valeurs arrondies « à la main » perd le contrôle.
3. **La relecture croisée** : échangez votre P4 avec un camarade. Le test est simple : peut-il **rejouer** trois de vos rapports et retrouver vos chiffres sans vous demander d'explication ? Si oui, la bibliothèque est bonne — c'est la définition du projet.

Le critère de réussite du projet. Un lecteur qui n'a jamais vu le socle ouvre votre dossier, lit l'en-tête d'un rapport, exécute la requête, obtient votre chiffre, lance le test qui le vérifie, et comprend en une phrase à quoi le rapport sert. Douze fois de suite.

Évaluation M11 — « SQL avancé pour la BI »

Module M11 · 30 h · niveau N3 → N4 · 7 chapitres. Total : 70 points — quiz 20 (seuil 14/20 §B.6), exercices à rendre 20 (dont 2 de type E4, « correction d'erreurs »), auto-test SQL 10, étude de cas 20 (seuil 12). Durée conseillée : 1 h pour le quiz, 2 h pour les exercices, 45 min pour l'auto-test, 1 h 30 pour l'étude de cas. **Matériel autorisé** : 03_exercices/dossier_M11/ (socle et connexion), 01_socle_donnees/, 01_socle_donnees/data/reference/chiffres_cites.json (clés m11_*, m11p_*, m11e_*) et l'instrument tools/controle_sql_M11.py. **Toute réponse chiffrée doit être mesurable** : un chiffre qui ne se retrouve pas dans le socle ne rapporte pas de point.

Ce que cette évaluation vérifie. Que vous savez **interroger** un socle de vente et **défendre** vos chiffres. Les questions ne demandent pas d'écrire des requêtes savantes : elles demandent de **dire** quelle construction répond à une question, de **prédire** la sortie d'une requête, de **diagnostiquer** un total faux, et de nommer ce qu'on manipule. Deux exercices (E4 et E5) portent sur des **requêtes erronées** : la justification pèse plus que la correction.

A · Questions de récupération (non notées)

Répondez de mémoire, en une ligne chacune. Elles ne rapportent rien : elles disent ce qu'il faut relire.

1. Qu'est-ce qui distingue une fonction de fenêtre d'un `GROUP BY` sur le **nombre de lignes** rendues ?
2. Que font respectivement `RANK`, `DENSE_RANK` et `ROW_NUMBER` quand deux lignes sont à égalité ?
3. Qu'est-ce qu'un **cadre de fenêtre**, et que change `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` ?
4. Qu'est-ce qu'une **cohorte**, et sur quel événement se date-t-elle ?
5. Que fait `QUALIFY` que `WHERE` ne peut pas faire ?
6. Que change `UNNEST` au **grain** d'un jeu de lignes ?
7. Que lit un plan d'exécution, et dans quel sens se lit-il ?

B · Quiz (20 questions · 20 points · 1 point par question)

Quatre familles de 5 questions : trouver la requête, prédire le résultat, diagnostiquer, nommer.

Bloc 1 — Quelle requête pour quelle question (C01—C05) (Q1 à Q5)

Q1. La cellule demande « quelle part de l'année représente ce mois ? ». Quelle construction répond ? a) un `GROUP BY` sur l'année ; b) `SUM(ca_mois) OVER (PARTITION BY annee)` ; c) `LAG(ca_mois) OVER (ORDER BY mois)` ; d) `NTILE(12) OVER (ORDER BY mois)`.

Q2. « Le 3^e meilleur produit de chaque famille » : quelle construction donne la bonne réponse **et** son rang ? a) `GROUP BY famille HAVING COUNT(*) <= 3` ; b) `RANK() OVER (PARTITION BY famille ORDER BY ca DESC)` puis un filtre `<= 3`, avec `id_produit` en clé de départage ; c) `LIMIT 3` ; d) `NTILE(3) OVER (PARTITION BY famille)`.

Q3. « Les clients qui ont acheté en janvier et pas en février » : quelle construction ne se trompe pas de sens ? a) `INNER JOIN` sur le mois ; b) `EXCEPT` (ou une jointure externe gauche avec test de nullité) ; c) `UNION ALL` ; d) `GROUP BY id_client` sans filtre de mois.

Q4. Pour dater la **cohorte** d'un client, quel événement prend-on ? a) sa date d'inscription au référentiel ; b) la date de sa **première commande** dans les ventes ; c) sa dernière commande ; d) la première commande de l'année en cours.

Q5. « Le chiffre d'affaires par quartier, **avec ses sous-totaux**, dans un seul résultat » : a) trois requêtes à recoller ; b) `GROUP BY ROLLUP (quartier, famille)` ; c) `PIVOT` ; d) `UNION` des détails et des totaux.

Bloc 2 — Prédisez le résultat (Q6 à Q10)

Q6. Que rend `SELECT id_client, COUNT(*) FROM ventes WHERE est_retour = 0 GROUP BY 1 HAVING COUNT(*) = 8` ? a) 8 clients ; b) 3 316 clients ; c) 8 lignes de vente ; d) aucun client.

Q7. Une requête filtre d'abord le mois de **janvier 2023**, puis calcule `SUM(montant_ttc) OVER (PARTITION BY annee_de_vente)`. Quelle part du chiffre d'affaires de 2023 le rapport publiera-t-il pour ce mois ? a) **8,3 %**, la vraie part ; b) **100,0 %**, parce que le filtre a été appliqué **avant** la fenêtre ; c) **12,5 %** ; d) une erreur d'exécution.

Q8. Sur les clients classés par **nombre de tickets**, que valent `MAX(RANK())` et `MAX(DENSE_RANK())` ? a) **23 444** et **23 444** ; b) **23 444** et **21** ; c) **23 497** et **21 444** ; d) **21** et **21**.

Q9. Combien de lignes rend `GROUP BY ROLLUP (quartier, famille)` sur ce socle, et de quoi sont-elles faites ? a) **35** lignes, les couples ; b) **41** lignes : **35** couples, **5** sous-totaux de quartier et **1** total général ; c) **47** lignes ; d) **44** lignes, une par mois.

Q10. Une moyenne mobile de **12** mois est calculée au **3^e** mois du socle. Combien de lignes portent réellement dans sa fenêtre ? a) **12** ; b) **3** ; c) **0** ; d) **1**.

Bloc 3 — Diagnostic (Q11 à Q15)

Q11. Un suivi des objectifs joint les ventes au référentiel des objectifs avec un `INNER JOIN` : le magasin 4 perd deux mois. Combien manque-t-il de lignes, et combien de ventes sont-elles concernées ? a) **2** couples, **93 749 490** FCFA ; b) **44** couples, **6 685 260 000** FCFA ; c) **2** couples, **5 317 736 710** FCFA ; d) aucun, la jointure interne ne perd rien.

Q12. Un rapport compare 2026 (**8** mois) à 2025 (**12** mois) et annonce **-28,9 %**. Que faut-il publier ? a) **-28,9 %**, le chiffre est exact ; b) **+15,4 %**, la comparaison à **périmètre égal** ; c) **0 %** ; d) **+18,8 %**, la croissance de 2024.

Q13. Un rapport de rétention publie **13,2 %** (**432** clients revenus sur **3 276** actifs), alors qu'une lecture par cohorte donne **22,7 %** sur **22** clients. Quelle est la cause ? a) une panne du moteur ; b) deux **définitions** différentes de la rétention — **calendaire** contre **par cohorte** — et un dénominateur qui ne mesure pas la même population ; c) un filtre de date oublié ; d) une erreur d'arrondi.

Q14. Un « meilleur produit par catégorie » écrit sur les libellés bruts du référentiel rend **46** lignes ; sur les familles repliées, il en rend **21**. Que révèle l'écart ? a) une erreur de la requête ; b) un **défaut de normalisation** du référentiel : **16** libellés bruts pour **9** écritures et **7** familles réelles ; c) une différence de période ; d) une limite mémoire du moteur.

Q15. Un rapport qui fait `SELECT *` sur la vue coûte **581** ms, la même requête sur trois colonnes nommées **96** ms. Qu'annonce le plan **avant** toute mesure ? a) rien d'utile ; b) la lecture de **18** colonnes contre **3** : le coût est dans la projection ; c) qu'il faut un index ; d) que la vue est corrompue.

Bloc 4 — Vocabulaire (Q16 à Q20)

Q16. Qu'appelle-t-on le **grain** d'un résultat ? a) sa taille en octets ; b) ce que représente **une ligne** (un mois, un client, un ticket, un couple quartier-famille) ; c) le nombre de colonnes ; d) la clé de tri.

Q17. Qu'appelle-t-on une **clé de départage**, et que se passe-t-il sans elle ? a) la clé primaire de la table ; b) la seconde colonne de tri qui rend l'ordre **déterministe** : sans elle, un classement peut changer de réponse entre deux exécutions ; c) un index ; d) un filtre.

Q18. Que fait **QUALIFY** ? a) il filtre **avant** le calcul de la fenêtre ; b) il filtre **après** le calcul de la fenêtre, ce que **WHERE** ne peut pas faire ; c) il trie ; d) il crée une vue.

Q19. Pourquoi un rapport coûte-t-il **142** ms sur la **vue** et **1** ms sur la table équivalente ? a) la table est plus petite ; b) une **vue** ne stocke rien : elle **relit** sa source à chaque appel ; c) la vue est mal écrite ; d) les deux requêtes ne sont pas les mêmes.

Q20. Que doit contenir la **note de méthode** d'un rapport de performance ? a) seulement les temps mesurés ; b) les temps, la version du moteur, le protocole, la date, et ce qui **n'a pas** été exécuté ; c) une capture d'écran ; d) la liste des index de la base.

C · Exercices à rendre (20 points · 5 exercices, dont 2 de type E4)

E1 — Le top 3 par famille, deux fois (niveau 3 · 3 points)

Produisez les **3** meilleurs produits de chaque famille du catalogue, une première fois sur les libellés tels quels, une seconde fois après repliement (casse, espaces, accents, pluriel). Donnez le **nombre de lignes** rendu par chaque version, et expliquez l'écart en deux phrases.

E2 — Les deux périmètres de la concentration (niveau 3 · 3 points)

Calculez la part du chiffre d'affaires du dixième supérieur des clients, **avec** puis **sans** le client non identifié. Écrivez la phrase qui doit accompagner la publication de chacune des deux valeurs.

E3 — Le suivi des objectifs, trous compris (niveau 3 · 4 points)

Produisez le suivi mensuel 2024 du magasin 4 : réalisé, objectif, taux de réalisation, et une colonne **etat** à trois valeurs (complet, objectif manquant, vente manquante). Le rapport doit faire apparaître les mois sans objectif, avec le montant de ventes concerné — **aucune ligne ne doit disparaître**.

E4 — « Ce rapport annonce un chiffre d'affaires net » (type E4 · niveau 4 · 5 points)

Voici la requête telle qu'elle a été livrée :

```
SELECT COUNT(*) AS lignes, ROUND(SUM(montant_ttc)) AS ca_net
FROM ventes;
```

Elle rend **240 000** lignes et **15 419 985 157** FCFA, publiés comme « chiffre d'affaires net ».

1. **Nommez l'erreur** en une phrase, sans écrire de requête.
2. **Corrigez** la requête et donnez les deux valeurs attendues.
3. **Mesurez l'écart** en FCFA et en pourcentage, et dites dans quel sens il joue.
4. Dites ce que ce total ferait à un rapport de suivi des objectifs, dont l'objectif cumulé est de **6 685 260 000** FCFA.
5. Écrivez la ligne d'en-tête qui aurait empêché l'erreur.

E5 — « Ce classement change de réponse à chaque exécution » (type E4 · niveau 4 · 5 points)

Voici la requête telle qu'elle a été livrée :

```
SELECT id_client, NTILE(3) OVER (ORDER BY nb_tickets) AS segment
FROM (SELECT id_client, COUNT(*) AS nb_tickets FROM ventes GROUP BY 1) t;
```

Relancée **15** fois le **24/09/2026**, elle a compté **2 718**, **2 720**, **2 728** puis **2 738** clients dans son segment principal — sans qu'aucune donnée ne change.

1. **Nommez la cause**, en citant le fait du socle qui la rend possible.
2. **Corrigez** la requête, et donnez la taille des trois tiers une fois l'ordre fixé.
3. Expliquez pourquoi ce défaut est **pire** qu'une requête lente.
4. Écrivez le **test de non-régression** qui aurait détecté le problème (valeur attendue, requête rejouée, verdict).

D · Auto-test SQL (10 points · 18 requêtes · auto-corrigé)

L'instrument `tools/controle_sql_M11.py` rejoue **18** requêtes et confronte chaque résultat aux valeurs publiées du module. Exécutez-le et **rendez sa sortie** (les **18** lignes de verdict et la ligne de bilan).

Barème : **18** contrôles au vert = **10** points ; **14** à **17** = **7** points ; **10** à **13** = **4** points ; moins de **10** = **0** point, avec un mot d'explication obligatoire sur les écarts.

Attention. Ne modifiez **aucune** requête de l'instrument : un contrôle n'est un contrôle que s'il rejoue la requête **publiée**. Si un écart apparaît, cherchez d'abord ce qui a changé dans le socle ou dans le fichier publié — et notez-le, c'est la réponse attendue.

E · Étude de cas — « 10 % des clients font-ils 60 % du CA ? » (20 points · seuil 12)

La commande

La direction commerciale a entendu, en séminaire, que « **10 %** des clients font **60 %** du chiffre d'affaires ». Elle vous demande de **vérifier la formule sur son propre fichier** et de lui dire, en une page, si elle peut la reprendre à son compte.

Les quatre productions attendues

1. **Le chiffre vrai** — la part du chiffre d'affaires portée par le dixième supérieur des clients, avec le **périmètre** annoncé (ventes nettes, période, magasins) et la **définition du décile** retenue.
2. **Le classement des clients, avant et après retrait du client non identifié** — ce que change le retrait, et le mot qui l'explique.
3. **La conclusion tenable** — trois lignes pour la direction, chiffres compris, sans formule recopiée sans vérification.
4. **La question suivante à poser** — celle que votre rapport ne tranche pas, et que la direction devra poser à son tour.

Le résultat attendu (à retrouver, pas à recopier)

Sur ce socle, la réponse est **non**, et elle est stable : le dixième supérieur pèse **38,1 %** du chiffre d'affaires net, **24,2 %** une fois le client **0** écarté, **51,0 %** pour le top **20 %**, **30,0 %** pour le top **5 %** et **21,6 %** pour le top **1 %**.

Attention. Le résultat dépend de la **définition** du dixième : `NTILE(100) <= 1` retient **2 350** clients et donne **38,1 %** ; un `ROW_NUMBER()` sur **10 %** du fichier en retient **2 349** et donne **38,0 %**. Un rapport qui publie un pourcentage sans publier sa définition de décile produit un chiffre que personne ne peut reproduire.

Barème de l'étude de cas

Critère	Points
1. Le chiffre vrai : périmètre annoncé, définition du décile écrite, valeur exacte	5
2. Le classement avant / après retrait du client 0 , et le mot qui l'explique	5
3. La conclusion tenable : trois lignes, utilisables, sans surinterprétation	5
4. La question suivante, formulée comme une question à laquelle on peut répondre	5
Total	20

Correction détaillée

A • Réponses de récupération

- Une fenêtre **conserve** les lignes (elle ajoute des colonnes) ; `GROUP BY` **réduit** la table aux groupes.
- `RANK` laisse des trous après une égalité, `DENSE_RANK` n'en laisse pas, `ROW_NUMBER` attribue des rangs uniques — d'où la nécessité d'une clé de départage.
- Le cadre est la portion de la partition prise en compte autour de la ligne courante ; `2 PRECEDING` retient trois lignes et ne s'étend pas au-delà des lignes existantes.
- Un groupe de clients partagé par un événement daté — ici la **première commande**, et non la date d'inscription du référentiel.
- `QUALIFY` filtre **après** le calcul de la fenêtre : il permet de garder le rang 1 par groupe dans la même requête.
- `UNNEST` passe d'une ligne « ticket » à une ligne « article de ticket » : le grain change, et `COUNT(*)` ne compte plus la même chose.
- Le plan décrit les opérateurs et les colonnes lues avant l'exécution ; il se lit **de bas en haut**.

B • Corrigé du quiz (1 point par question)

Question	Réponse	Pourquoi
Q1	b	La part se calcule par une fenêtre partitionnée par année (C01).
Q2	b	<code>RANK</code> partitionné, filtré à 3 , avec <code>id_produit</code> en départage (C02).
Q3	b	<code>EXCEPT</code> (ou jointure externe gauche + nullité) : le <code>INNER JOIN</code> répond à une autre question (C05).
Q4	b	La cohorte se date sur la première commande (C04).
Q5	b	<code>ROLLUP</code> produit les 41 lignes en une requête (C05).

Question	Réponse	Pourquoi
Q6	b	3 316 clients partagent exactement 8 tickets (14,1 %).
Q7	b	Le filtre avant la fenêtre ne laisse qu'une ligne : 100,0 % au lieu de 8,3 % (C01).
Q8	b	23 444 en RANK contre 21 en DENSE_RANK (C02).
Q9	b	35 couples + 5 sous-totaux + 1 total général (C05).
Q10	b	Au 3 ^e mois, la fenêtre ne porte que sur 3 lignes (C03).
Q11	a	2 couples manquants (magasin 4, 2024-02 et 2024-03) pour 93 749 490 FCFA (C03).
Q12	b	À périmètre égal : +15,4 % (C03).
Q13	b	Rétention calendaire contre rétention par cohorte : deux définitions, deux populations (C04).
Q14	b	16 libellés bruts, 9 écritures, 7 familles : 46 lignes contre 21 (C05).
Q15	b	Le plan annonce 18 colonnes contre 3 : le coût se lit avant l'exécution (C06).
Q16	b	Le grain est ce que représente une ligne (C01, C05).
Q17	b	La clé de départage rend l'ordre déterministe ; sans elle, le rapport change (C02, C07).
Q18	b	QUALIFY filtre après la fenêtre (C05).
Q19	b	Une vue relit sa source : 142 ms contre 1 ms (C06).
Q20	b	Temps, moteur, protocole, date, et ce qui n'a pas été exécuté (C06, C07).

C · Corrigé des exercices à rendre

E1. Les libellés bruts rendent **46** lignes pour **16** catégories ; après repliement des graphies, le même top 3 rend **21** lignes pour **7** familles. L'écart est la trace exacte du défaut de normalisation du référentiel, pas un problème de requête.

E2. Avec le client **0** : **38,1** % ; sans lui : **24,2** %. Les deux phrases attendues : « le dixième supérieur des **2 350** clients pèse **38,1** % du chiffre d'affaires net » et « hors client non identifié, ce dixième pèse **24,2** % — l'écart est porté par un seul identifiant, non rattaché à un client du référentiel ».

E3. Structure FULL OUTER JOIN avec CASE à trois états ; les **2** lignes sans objectif portent **42 514 372** et **51 235 118** FCFA, soit **93 749 490** FCFA de réalisé sans référence. Le taux de réalisation y vaut NULL — ni **0**, ni **100** %.

E4 — les cinq attendus. 1. La requête ne filtre pas les retours : elle compte les **2 809** lignes de retour dans un total annoncé « net ». 2. Correction : WHERE est_retour = 0 → **237 191** lignes et **15 595 154 955** FCFA. 3. Écart : **-175 169 798** FCFA, soit **-1,1** % — le total faux est **plus bas** que le vrai, puisque les retours sont négatifs, alors que le nombre de lignes est **plus haut** de **1,2** % (**240 000** comptées contre **237 191**) : l'erreur se lit à l'envers selon l'indicateur. 4. Face à un objectif cumulé de **6 685 260 000** FCFA, le total faux afficherait **230,7** % au lieu de **233,3** % : l'erreur reste invisible dans le vert de l'indicateur. 5. En-tête : « Hypothèses : ventes **nettes** de retours (est_retour = 0) ; les **2 809** lignes de retour sont exclues, jamais sommées. »

E5 — les quatre attendus. 1. L'ORDER BY n'a **pas** de clé de départage : **3 316** clients partagent exactement **8** tickets (**14,1** % du fichier), et le moteur est libre de placer les ex æquo comme il veut. 2. Correction : ORDER BY nb_tickets, id_client → les trois tiers comptent **7 833**, **7 832** et **7 832** clients, stables d'une exécution à l'autre. 3. Une requête lente se corrige et se mesure ; une requête **instable** publie des chiffres contradictoires d'un jour à l'autre, et le rapport perd sa crédibilité avant qu'on ait trouvé pourquoi. 4. Le test attendu rejoue **à l'identique** la requête corrigée et compare la taille du premier tiers à **7 833** : tout autre résultat fait échouer le contrôle et sort en code non nul.

D · Corrigé de l'auto-test

La sortie attendue comporte **18** lignes de verdict au vert, du comptage des lignes (**240 000, 237 191** hors retours) au ratio de saison (**1,65**), et se termine par « **18** controles sur 18 : OK — le socle et le fichier publie disent la meme chose. »

E · Corrigé de l'étude de cas — les quatre attendus

1. **Le chiffre vrai.** **38,1 %** du chiffre d'affaires net, périmètre : ventes nettes (retours exclus), 01/01/2023 au 31/08/2026, **5** magasins, client **0 inclus** ; définition du décile : `NTILE(100) <= 1`, soit **2 350** clients (l'autre définition usuelle, un `ROW_NUMBER` sur **10 %** du fichier, retient **2 349** clients et donne **38,0 %**).
2. **Le classement, avant et après.** Le client **0** pèse **2 852 612 447** FCFA — **18,3 %** du chiffre d'affaires — sur **42 658** lignes et **38 984** tickets, sans exister dans le référentiel. Il occupe donc toujours la première place d'un classement par montant : une fois écarté, le dixième supérieur tombe à **24,2 %**, et le classement des têtes change intégralement. Le mot à employer est **périmètre** : retirer le client **0** retire **18,3 %** du chiffre d'affaires, c'est un choix, et il s'écrit.
3. **La conclusion tenable.** Trois lignes, du type : « sur notre fichier, les **10 %** de clients les plus gros pèsent **38 %** du chiffre d'affaires, et **24 %** une fois le client non identifié écarté — la formule des **60 %** ne s'applique pas ici ; la concentration est réelle mais plus faible, et le top **20 %** pèse **51,0 %**. »
4. **La question suivante.** « Quelle part du chiffre d'affaires de chaque **cohorte** est portée par des clients non identifiés ? » Tant que le client **0** représente **18,3 %** du chiffre d'affaires, toute lecture de la rétention se fait sur un cinquième du total qu'on ne sait pas attribuer — c'est la question qui manque aux **12** rapports du projet, et elle se pose avec les outils du chapitre C04.

Barème global et seuils

Partie	Points	Seuil
B · Quiz (20 questions)	20	14/20 (§B.6)
C · Exercices à rendre (5 , dont 2 E4)	20	—
D · Auto-test SQL (18 requêtes)	10	—
E · Étude de cas	20	12/20 (§B.6)
Total	70	49/70 pour la validation du module

Épreuve de palier. Cette évaluation prépare l'épreuve **P4** de la phase 4 (§B.6) : les quatre compétences qu'elle mesure — écrire une requête qui répond à une question posée, prédire ce qu'une fenêtre va changer, diagnostiquer un total faux, publier un taux avec son effectif et sa définition — sont celles que l'épreuve de palier reprend, sur un socle différent.

Module M12.C01 — Qu'est-ce que la BI, et ce qu'elle n'est pas

Analyse de données appliquée & Business Intelligence
8 partie(s) assemblée(s) · document de production

Table des matières

MODULE M12.C01 — QU’EST-CE QUE LA BI, ET CE QU’ELLE N’EST PAS	8
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le tableau de bord du camion	9
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Ce que la BI n’est pas — cinq confusions qui coûtent	10
5.2 La chaîne de la donnée à la décision, maillon par maillon	10
5.3 Le coût d’une décision sans donnée : trois formes	10
5.4 L’économie réelle d’un dispositif BI dans une PME	11
5.5 Qui fait de la BI dans une PME	11
5.6 Les trois questions de cadrage	11
5.7 Les cinq signaux d’un échec annoncé	11
5.8 Quatre façons de rater la couche sémantique	12
5.9 Ce que ce chapitre ne couvre pas	12
6. Exemple concret — une décision de réassort, avec et sans BI	12
7. Démonstration pas à pas — six étapes sur le socle	13
7.1 Étape 1 — le total de contrôle	13
7.2 Étape 2 — la même question, deux définitions	13
7.3 Étape 3 — l’indicateur qui décide, et son dénominateur	13
7.4 Étape 4 — le dénominateur d’un taux, et les commandes qui disparaissent	13
7.5 Étape 5 — ce qu’une donnée manquante fait à un indicateur	13
7.6 Étape 6 — écrire la décision, pas le chiffre	14
7.7 Étape 7 — mesurer l’adoption, c’est-à-dire la seule chose qui compte	14
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	15
10. Exercice guidé	15
11. Exercices autonomes	15
12. Correction détaillée	16
13. Mini-projet de chapitre	16
14. Résumé du chapitre	16
15. À retenir	17
16. Évaluation formative	17
MODULE M12.C02 — REPORTING, BI, ET LES QUATRE RÉGIMES DE QUESTION	19
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — la météo et le climat	20
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Reporting et BI : ce qui les sépare vraiment	20
5.2 Les quatre régimes, et la technique que chacun réclame	21

5.3 Le régime descriptif, mesuré — la notion de tranche	21
5.4 Le rythme fait partie de l'indicateur	21
5.5 Le régime diagnostique : relier un écart à une cause mesurée	22
5.6 Le régime prédictif sans modèle : trois projections, une erreur publiée	22
5.7 Le régime prescriptif : transformer un constat en file d'action	22
5.8 Périodicité, exploration, interactivité, gouvernance : les quatre propriétés	23
5.9 Ce que ce chapitre ne couvre pas	12
6. Exemple concret — la même demande, quatre réponses	23
7. Démonstration pas à pas — quatre régimes sur le socle	24
7.1 Étape 1 — descriptif : le total, puis les tranches	24
7.2 Étape 2 — descriptif fin : la tranche qui change la lecture	24
7.3 Étape 3 — diagnostique : de l'écart à la cause	24
7.4 Étape 4 — prédictif : trois méthodes, une erreur publiée	24
7.5 Étape 5 — prescriptif : la file, pas l'alerte	25
7.6 Étape 6 — l'arbitrage entre régimes	25
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	15
10. Exercice guidé	15
11. Exercices autonomes	15
12. Correction détaillée	16
13. Mini-projet de chapitre	16
14. Résumé du chapitre	16
15. À retenir	17
16. Évaluation formative	17
MODULE M12.C03 — MÉTRIQUES, DIMENSIONS, FAITS, GRAIN : LE VOCABULAIRE EXACT	29
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — les briques et le ciment	30
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 La table de faits et la table de dimensions	31
5.2 Le grain : la question posée huit fois	31
5.3 Les dimensions du socle et leur cardinalité	32
5.4 Additif, semi-additif, non additif	32
5.5 Le piège de la moyenne de moyennes, chiffré	33
5.6 Métrique, indicateur, indicateur clé — et les trois horizons	33
5.7 La clé oubliée : le piège qui fabrique un total	34
5.8 Ce que ce chapitre ne couvre pas	34
6. Exemple concret — le tableau croisé qui se réconcilie	34
7. Démonstration pas à pas — six étapes sur le socle	13
7.1 Étape 1 — prouver le grain avant de compter	35
7.2 Étape 2 — mesurer une dimension avant de la croiser	35
7.3 Étape 3 — vérifier l'additivité par la somme	35
7.4 Étape 4 — constater la non-additivité d'un comptage distinct	35
7.5 Étape 5 — refuser la moyenne de moyennes	35

7.6 Étape 6 — le contrôle qui rattrape la clé oubliée	35
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	15
10. Exercice guidé	15
11. Exercices autonomes	15
12. Correction détaillée	16
13. Mini-projet de chapitre	16
14. Résumé du chapitre	16
15. À retenir	17
16. Évaluation formative	17
MODULE M12.C04 — CONCEVOIR UN KPI : LES 6 CRITÈRES ET LA CARTE DE DÉFINITION	40
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le passeport du chiffre	41
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Les six critères	42
5.2 La carte de définition, champ par champ	42
5.3 Critère 1 — sourcé : une source absente ne se remplace pas	42
5.4 Critère 2 — défini sans ambiguïté : le cas du taux de retour	43
5.5 Critère 3 — attribué : un nom, pas un comité	43
5.6 Critère 4 — comparable : ni immobile, ni frénétique	44
5.7 Critère 5 — pilotable : le seuil et l'action	44
5.8 Critère 6 — contre-balancé : les dix couples	44
5.9 Le catalogue des effets pervers	45
6. Exemple concret — la carte remplie du taux de rupture	46
7. Démonstration pas à pas — six étapes pour fabriquer un indicateur	47
7.1 Étape 1 — partir d'une question de décision, pas d'une donnée	47
7.2 Étape 2 — choisir le régime et le niveau	47
7.3 Étape 3 — écrire la formule et mesurer les deux versions	47
7.4 Étape 4 — vérifier la source et la granularité	47
7.5 Étape 5 — poser le seuil et l'action	47
7.6 Étape 6 — choisir le contre-KPI et le publier	47
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	15
10. Exercice guidé	15
11. Exercices autonomes	15
12. Correction détaillée	16
13. Mini-projet de chapitre	16
14. Résumé du chapitre	16
15. À retenir	17
16. Évaluation formative	17

MODULE M12.C05 — L'ARCHITECTURE BI DE BOUT EN BOUT	52
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — sept étages, deux sens	53
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Étage 1 — les sources	54
5.2 Étage 2 — l'extraction	54
5.3 Étage 3 — le stockage	55
5.4 Étage 4 — le modèle	55
5.5 Étage 5 — la couche sémantique	55
5.6 Étage 6 — la visualisation	56
5.7 Étage 7 — la diffusion	56
5.8 Les sept étages en une table	56
6. Exemple concret — la chaîne du module M12, étage par étage	57
7. Démonstration pas à pas — suivre un chiffre dans les sept étages	57
7.1 Étage 1 — la source	58
7.2 Étage 2 — l'extraction	58
7.3 Étage 3 — le stockage	58
7.4 Étage 4 — le modèle	58
7.5 Étage 5 — la couche sémantique	58
7.6 Étage 6 — la visualisation	58
7.7 Étage 7 — la diffusion	58
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	15
10. Exercice guidé	15
11. Exercices autonomes	15
12. Correction détaillée	16
13. Mini-projet de chapitre	16
14. Résumé du chapitre	16
15. À retenir	17
16. Évaluation formative	17
MODULE M12.C06 — RÉUSSIR LE PROJET BI	63
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — la maison et ses habitants	64
4. Vocabulaire essentiel	9
5. Cours approfondi	10
5.1 Les dix causes d'échec, toutes visibles dans le dossier	65
5.2 Les parties prenantes et leurs rôles	66
5.3 Le cahier des charges utile, en six questions	66
5.4 La conduite : l'ordre des étapes fait le projet	66
5.5 L'adoption : la mesurer pour la piloter	67
5.6 La maintenance et la gouvernance	67
5.7 L'économie réelle d'un projet décisionnel	67

5.8 Détecter les causes d'échec avant la livraison	68
6. Exemple concret — la reprise, chiffrée de bout en bout	68
7. Démonstration pas à pas — six étapes de reprise	69
7.1 Étape 1 — diagnostiquer avec des chiffres	69
7.2 Étape 2 — choisir 10 décisions	69
7.3 Étape 3 — remplir les 70 cases	69
7.4 Étape 4 — construire un socle rejouable	69
7.5 Étape 5 — un écran, une page, une action par seuil	69
7.6 Étape 6 — mesurer l'usage à J+7 et à J+30	69
8. Erreurs fréquentes	14
9. Bonnes pratiques professionnelles	15
10. Exercice guidé	15
11. Exercices autonomes	15
12. Correction détaillée	16
13. Mini-projet de chapitre	16
14. Résumé du chapitre	16
15. À retenir	17
16. Évaluation formative	17
PROJET M12.P — « LE TABLEAU DE BORD QUE PERSONNE N'OUVRIT »	74
1. Énoncé	74
Le matériel fourni	74
Les six totaux de contrôle	74
2. Les dix indicateurs imposés (le catalogue du mandat)	75
3. Les 4 livrables	75
P1 — Le diagnostic mesuré (6 points)	75
P2 — La carte de définition de 10 KPI (8 points)	75
P3 — Le plan de reprise (4 points)	76
P4 — La page « que fait-on quand c'est rouge ? » (2 points)	76
4. Barème	76
5. Ce qui fait rejeter un livrable	77
6. Déroulé conseillé (4 séances, environ 10 h)	77
7. Comment vous corriger vous-même	77
ÉVALUATION M12 — « FONDAMENTAUX DE LA BUSINESS INTELLIGENCE »	78
A • Questions de récupération (non notées)	78
B • Quiz (15 questions • 20 points)	78
Bloc 1 — Définir (Q1 à Q5)	78
Bloc 2 — Mesurer (Q6 à Q10)	78
Bloc 3 — Décider (Q11 à Q15, 2 points chacune)	79
C • Exercices de rédaction de KPI (20 points • 2 exercices • PATCH_8 : les énoncés reformulent, ils ne	79
R1 — Le taux de rupture (10 points de rédaction)	79
R2 — La part des clients non identifiés (10 points)	80
D • Étude de cas — « la direction veut un tableau de bord, elle a déjà tort de le vouloir comme ça » (30 points • seuil 18)	80

La commande	80
Le résultat attendu (à retrouver, pas à recopier)	80
Barème de l'étude de cas	81
Correction détaillée	81
A · Réponses de récupération	81
B · Corrigé du quiz	81
C · Corrigé des exercices de rédaction	82
D · Corrigé de l'étude de cas	82

Module M12.C01 — Qu'est-ce que la BI, et ce qu'elle n'est pas

Outils : DuckDB 1.5.5 (exécuté), pandas (contrôle croisé, exécuté), Excel (exécuté). Power BI (cité, non exécuté — règle §1.5). Durée indicative : 5 h. Niveau : N3. Prérequis : M09 (questions métier), M10 (support), M11 (la matière : des requêtes fiables).

L'idée du chapitre. La Business Intelligence n'est pas une collection de graphiques, ni un logiciel, ni un projet informatique : c'est **l'organisation qui fait qu'un chiffre arrive devant celui qui doit décider, à temps, avec une définition unique, et suivi d'une action décidée d'avance**. Ce chapitre installe cette définition et la met immédiatement à l'épreuve du socle : **41** indicateurs livrés, **4** personnes qui les ouvrent le premier jour, **11** semaines plus tard un contrat résilié. Le module entier part de cet échec mesuré — parce qu'on apprend mieux une discipline par ce qu'elle rate que par ce qu'elle promet.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **définir** la Business Intelligence, et la distinguer de quatre choses avec lesquelles on la confond tous les jours (le graphique, le logiciel, le rapport figé, l'intelligence artificielle) ;
2. **situer** la place de chacun dans une chaîne qui va de la donnée brute à la décision, et nommer ce que chaque maillon ajoute ;
3. **chiffrer le coût d'une décision sans donnée** — retardée, fausse, ou jamais prise — sur un cas réel du socle, plutôt que sur une généralité ;
4. **estimer l'économie d'un dispositif BI** dans une PME : ce qu'il coûte, ce qu'il rapporte, et pourquoi son coût visible n'est jamais son vrai coût ;
5. **citer les trois questions de cadrage** sans lesquelles aucun dispositif ne tient (quelle décision, qui décide, que fait-on si c'est rouge) et repérer les cinq signaux d'un échec annoncé.

2. Pourquoi cette notion est importante

Vous savez produire des chiffres justes depuis M11. C'est nécessaire et ce n'est pas suffisant : sur le socle du module, **15 595 154 955** FCFA de ventes ont été enregistrés, **2 428** couples produit-magasin-mois sont passés en rupture, **1 202 550 590** FCFA de factures restent ouvertes — et **aucun** de ces trois nombres, à lui seul, n'a jamais fait prendre une bonne décision.

La raison est simple et brutale : **une décision ne consomme pas des chiffres, elle consomme des écarts**. « Nous avons vendu 15 595 154 955 FCFA » n'appelle aucune action ; « nous avons vendu **1,12 %** de moins que ce que nos propres définitions appellent le chiffre d'affaires » appelle une réunion. La BI est la discipline qui fabrique systématiquement ce deuxième type de phrase — et qui organise l'entreprise pour qu'il arrive à l'heure.

Attention. Un chiffre juste publié au mauvais grain, à la mauvaise fréquence, ou sans responsable, produit exactement le même effet qu'un chiffre faux : il ne change rien. C'est la découverte la plus coûteuse de ce module, et le dossier du projet raté la démontre sur **11** semaines.

À retenir. La BI ne se juge pas au nombre de chiffres publiés, mais au nombre de **décisions** qu'elle rend possibles : **41** indicateurs pour **10** décisions réclamées, dans le dossier du projet, c'est trente et un chiffres de trop.

3. Explication simple — le tableau de bord du camion

Prenez un camion de livraison. Il possède **beaucoup** de mesures : température moteur, pression des pneus, niveau de carburant, vitesse, charge, usure des freins, pression d'huile, tension de la batterie. Le constructeur n'affiche pas tout devant le conducteur : il en garde une poignée, choisie selon **une seule règle** — est-ce que cette aiguille peut changer une action pendant que je conduis ? La température d'huile qui monte : je m'arrête. La pression d'un pneu : je ralentis. Le kilométrage : il déclenche un entretien, pas une action immédiate. L'usure des freins : elle se planifie.

Le reste des mesures existe : elles vivent dans le calculateur, lues par le garagiste, au moment où quelqu'un en a besoin.

Un dispositif de BI fonctionne exactement comme ce tableau de bord, et échoue exactement comme un tableau de bord qui afficherait les **41** mesures du camion en même temps, sur **7** écrans, sans dire laquelle impose quoi. Le conducteur poserait son regard ailleurs — c'est ce qui est arrivé au dispositif du dossier : **14** destinataires, **4** ouvertures le premier jour, **3** la deuxième semaine.

4. Vocabulaire essentiel

Définition. La **Business Intelligence** (BI) est l'ensemble organisé des moyens — données, définitions, modèles, outils, responsabilités — qui permet à une organisation de **décider avec des chiffres partagés** : les mêmes définitions pour tous, disponibles au bon moment, avec une action décidée d'avance quand ils passent un seuil.

Définition. Un **indicateur clé** — KPI, Key Performance Indicators — est une mesure **choisie** pour suivre une performance et **déclencher une décision**. Ce qui le distingue d'une simple mesure tient là, et nulle part ailleurs : à qui sert-il, pour décider quoi ?

Définition. Une **décision fondée sur les données** — data-driven — est prise en s'appuyant sur des mesures définies **avant** la décision, et vérifiables **après**. Sans la seconde partie, l'expression n'est qu'une façon de parler.

Définition. La **couche sémantique** — semantic layer — est l'endroit — table, vue, modèle — où les définitions d'indicateurs sont **écrites une fois** pour toute l'organisation. C'est la version industrielle de la carte de définition du chapitre C04.

Le vocabulaire minimal de la chaîne, dans l'ordre : donnée (un enregistrement), information (une mise en relation — « ce magasin vend 3,06 fois plus que le plus petit »), connaissance (une règle — « nos ruptures se concentrent sur les produits à rotation rapide »), décision (un engagement — « nous portons le stock de ces 23 références à deux mois »).

5. Cours approfondi

5.1 Ce que la BI n'est pas — cinq confusions qui coûtent

Ce qu'on croit	Ce que c'est en réalité	Le coût mesuré de la confusion
« de la belle visualisation »	une définition partagée, souvent invisible sur l'écran	le dispositif à 41 indicateurs était lisible ; il était indéfini
« un projet informatique »	un projet métier que l'informatique outille	2 définitions du CA coexistaient sans arbitrage
« un rapport figé »	un service mis à jour, corrigé, retiré	la maintenance résiliée (1 200 000 FCFA/an) a tué l'usage
« de l'intelligence artificielle »	de la définition et de la discipline	aucune décision n'est prise par un modèle dans ce module
« toutes nos données »	les quelques mesures qui changent une action	41 indicateurs pour 14 lecteurs, dont 4 le premier jour

5.2 La chaîne de la donnée à la décision, maillon par maillon



Chaque maillon **ajoute** quelque chose, et chaque maillon **peut** casser le résultat : une extraction qui perd les retours fabrique les **175 169 798** FCFA d'écart du §7.2 ; un modèle qui double les lignes double les totaux ; une visualisation sans unité fait lire des milliers comme des millions ; une diffusion sans responsable produit des chiffres que personne ne défend.

Le maillon invisible — et c'est celui de M12 — est la **couche sémantique** : celle où « chiffre d'affaires » veut dire une chose précise. Aucun outil ne l'invente à votre place.

5.3 Le coût d'une décision sans donnée : trois formes

Forme	Ce qui se passe	Mesure sur le socle
Décision retardée	on attend d'être sûr, l'occasion passe	13 129 jours de rupture cumulés avant réaction
Décision fausse	on décide sur un indicateur mal défini	rotation lue 0,78 au lieu de 9,42 tours : on stocke au lieu de déstocker
Non-décision	personne ne sait que faire si c'est rouge	1 272 factures ouvertes, 1 202 550 590 FCFA, aucun seuil associé à une action

La troisième est la plus chère et la moins visible. Elle ne produit aucune erreur : elle produit un **silence**.

Attention. Ne pas décider est une décision — elle est simplement prise sans vous. **1 272** factures restent ouvertes pour **1 202 550 590** FCFA non pas parce que personne ne sait les relancer, mais parce que personne n'a écrit **qui** devait le faire à partir de quel seuil. Le dossier du projet raté la décrit en une ligne — le seuil de rupture n'a jamais été relié à une action — et c'est la raison pour laquelle le mandat de reprise exige une page intitulée « que fait-on quand c'est rouge ? ».

5.4 L'économie réelle d'un dispositif BI dans une PME

Le dossier du projet fournit le chiffrage : **7 940 000 FCFA** la première année, dont **1 640 000** pour la seule reprise des sources et **1 200 000** de maintenance — pour un dispositif abandonné en **11** semaines. Trois enseignements d'économie, tous vérifiables :

1. **Le coût dominant n'est pas l'outil, c'est la matière.** La reprise des sources pèse plus de **20 %** du projet ; un projet qui la sous-estime livre un tableau de bord qui calcule juste sur des données fausses.
2. **Le retour ne vient jamais du nombre d'indicateurs.** Ici **41** indicateurs pour **10** décisions réclamées au mandat de reprise : tout ce qui dépasse la décision est du coût pur.
3. **Un dispositif non maintenu meurt vite.** **4** ouvertures le premier jour, **3** trois semaines plus tard : la courbe d'adoption se lit comme un indicateur, et c'est le seul qui compte.

Dans les faits. L'atelier de ce manuel applique la règle à lui-même : les dix indicateurs du module sont recalculés par un script (`tools/kpi_M12.py`) à chaque relevé de chiffres, et le contrôle croisé côté pandas vérifie que le CA net vaut bien **15 595 154 955 FCFA** des deux côtés. Une définition qui ne se vérifie pas toute seule finit par diverger.

5.5 Qui fait de la BI dans une PME

Rôle	Ce qu'il décide	Ce qu'on lui doit
Direction	les priorités et les arbitrages de définition	une carte de définition, pas un catalogue
Responsable métier	l'action quand l'indicateur passe au rouge	un seuil et une action, écrits
Analyste BI	la justesse, la fraîcheur, la documentation	le droit de refuser un indicateur indéfini
Informatique	les accès, les flux, la sécurité	des définitions stables, pas des demandes quotidiennes

La ligne la plus mal comprise est la dernière : **l'analyste doit pouvoir refuser**. Un indicateur sans définition écrite et sans responsable produira, le jour du désaccord, deux chiffres et zéro décision — exactement le scénario des **3** sources discordantes du dossier.

5.6 Les trois questions de cadrage

1. **Quelle décision ?** — « la direction veut tout suivre » n'est pas une décision. « Nous devons décider chaque mois quelles références porter à deux mois de stock » en est une.
2. **Qui décide, et quand ?** — un indicateur sans nom de responsable et sans rendez-vous est un objet décoratif.
3. **Que fait-on si c'est rouge ?** — la question qui distingue un dispositif vivant d'un dispositif abandonné. Elle se répond **avant** la livraison, en une phrase par indicateur.

5.7 Les cinq signaux d'un échec annoncé

Signal	Seuil d'alerte observé	Ce qu'il annonce
Adoption	moins de la moitié des destinataires ouvrent dans la semaine	le dispositif n'est pas attendu
Définitions	un même indicateur a 2 valeurs en réunion	la couche sémantique n'existe pas
Décisions	aucun seuil relié à une action	les alertes seront ignorées
Périmètre	plus de 10 indicateurs pour une équipe	la priorité n'a pas été choisie
Maintenance	personne ne sait qui corrige une définition	l'abandon est daté

5.8 Quatre façons de rater la couche sémantique

La couche sémantique est le seul maillon du §5.2 qu'aucun outil ne fournit. Quatre échecs typiques, tous observables dans la semaine qui suit la mise en service :

Échec	Ce qu'on voit	Ce qu'il faut faire
Deux définitions, deux fichiers	le CA « compta » et le CA « du tableau de bord » différent de 175 169 798 FCFA	écrire une définition, publier l'autre comme un indicateur distinct et nommé
Le grain implicite	le panier moyen vaut 107 396 FCFA ou 65 749 FCFA selon qu'on divise par les tickets ou les lignes	nommer le dénominateur dans le titre de l'indicateur
La convention tacite	les retours comptés en négatif dans un onglet, exclus dans l'autre	figer la convention dans la carte de définition (C04)
La mise à jour sans date	deux lecteurs comparent des chiffres de deux jours différents	dater chaque bloc de résultats, à l'écran

La règle qui les prévient toutes. Une définition est complète quand un lecteur qui n'a pas écrit l'indicateur peut le recalculer sans vous appeler — même critère que les rapports de M11, appliqué aux définitions.

5.9 Ce que ce chapitre ne couvre pas

Le **vocabulaire exact** des données (métrique, dimension, fait, grain) est le chapitre C03 ; la **fabrication** d'un indicateur — les six critères, la carte de définition, le contre-KPI — est le chapitre C04 ; l'**architecture** détaillée (sept étages, leurs pannes, leurs coûts) est le C05 ; la **conduite de projet** et les dix causes d'échec sont le C06. Ce chapitre-ci tient volontairement une seule question : pourquoi la BI existe, et à quoi on reconnaît qu'elle n'y est pas encore ?

6. Exemple concret — une décision de réassort, avec et sans BI

Le magasin de Kaya doit décider, pour le mois qui vient, quelles références porter à deux mois de stock. Voici les trois façons de décider, et ce que chacune coûte sur le socle.

Sans tableau de bord. Le responsable regarde les rayons et commande ce qui manque. Ce qui manque, il ne le voit qu'au moment où le client le demande : le socle compte **2 428** ruptures et **13 129** jours de rupture. Le manque à gagner **estimé** — car une rupture ne se mesure pas, elle s'estime — vaut **169 386 812** FCFA.

Avec un chiffre, mal défini. On lui donne « la rotation de stock ». L'indicateur est calculé en divisant les ventes de douze mois par la **somme** des douze stocks mensuels : **0,78** tour par an. Conclusion, mécanique : « notre stock ne tourne pas, arrêtons de commander ». Or la même donnée, avec le bon dénominateur — la **moyenne** mensuelle du stock — donne **9,42** tours par an, couverture **1,27** mois, c'est-à-dire un stock **sain**. Un chiffre mal défini fait prendre la décision exacte inverse de celle qu'il fallait.

Avec un indicateur défini. « Rotation de stock : quantités vendues sur douze mois glissants divisées par le stock moyen mensuel de la même période ; cible **9,42** tours ; seuil d'alerte en dessous de **7** ; responsable : le responsable d'approvisionnement ; action si rouge : révision du plan de réassort sous cinq jours ouvrés ; contre-KPI : taux de rupture. » Cette phrase — et non la requête — est ce que M12 apprend à écrire. Elle a trois vertus : elle rend le chiffre reproductible, elle rend la décision obligatoire, et elle empêche de « gagner » la rotation en laissant mourir le service client.

7. Démonstration pas à pas — six étapes sur le socle

Tout se rejoue avec `03_exercices/dossier_M12/connexion.py` (socle de M11 + les cinq tables opérationnelles).

7.1 Étape 1 — le total de contrôle

```
SELECT ROUND(SUM(montant_ttc)) AS ca_net
FROM ventes
WHERE est_retour = 0;
```

15 595 154 955 FCFA. C'est le total de référence du module : tout indicateur qui parle de chiffre d'affaires doit retomber dessus, ou dire pourquoi il ne le fait pas.

7.2 Étape 2 — la même question, deux définitions

```
SELECT ROUND(SUM(montant_ttc)) AS ca_toutes_lignes FROM ventes;
```

15 419 985 157 FCFA. L'écart avec l'étape 1 vaut **175 169 798** FCFA, soit **1,12 %** : les lignes de retour, négatives, sont comptées. Aucune des deux valeurs n'est fautive — la première parle d'activité nette, la seconde de flux de caisse. Ce qui est faux, c'est de les **mélanger** dans un même rapport : c'est exactement la première discorde du dossier du projet raté.

7.3 Étape 3 — l'indicateur qui décide, et son dénominateur

```
SELECT ROUND(SUM(v.quantite) / AVG(stock_du_mois), 2) AS rotation
FROM ventes v,
     (SELECT mois, SUM(stock_moyen_unites) AS stock_du_mois
      FROM stock_mensuel GROUP BY 1) s
WHERE v.est_retour = 0
      AND v.date_vente >= DATE '2025-09-01' AND v.date_vente < DATE '2026-09-01'
      AND s.mois >= '2025-09' AND s.mois < '2026-09';
```

9,42 tours par an. Remplacez `AVG(stock_du_mois)` par `SUM(stock_moyen_unites)` : **0,78**. Une requête qui **ne lève aucune erreur** peut diviser un indicateur par douze — et faire stocker quand il fallait déstocker.

7.4 Étape 4 — le dénominateur d'un taux, et les commandes qui disparaissent

```
SELECT
  COUNT(*) FILTER (WHERE statut = 'livree') AS livrees,
  COUNT(*) FILTER (WHERE statut = 'livree' AND date_livraison <= date_promisee) AS a_lheure,
  COUNT(*) AS commandes
FROM commande;
```

8 681 livrées, **7 035** à l'heure, **9 000** commandes. Le taux de service vaut donc **81,0 %** des livrées ou **78,2 %** des commandes, selon ce qu'on met au dénominateur — et **319** commandes annulées décident à elles seules du chiffre publié. Deux conventions, une seule à écrire, et le mot « annulée » à ne pas cacher sous le tapis.

7.5 Étape 5 — ce qu'une donnée manquante fait à un indicateur

```
SELECT ROUND(SUM(marge_fcfa) / SUM(montant_ht) * 100, 2) AS taux_marge
FROM vente_marge WHERE est_retour = 0;
```

29,12 % — alors que le référentiel annonce une marge cible de **18 à 24 %** par rayon. Deux causes, toutes deux mesurables : deux produits du référentiel n’ont **pas de prix de vente** (écart de **0,05** point selon qu’on les écarte ou qu’on reconstitue leur prix), et surtout les prix de vente du socle ont monté de **25,0 %** entre 2023 (**8 219** FCFA de prix unitaire moyen) et 2026 (**10 271** FCFA), quand le référentiel porte un prix unique. Publier « 29,12 % de marge » sans cette phrase fait croire à une performance qui n’existe pas.

7.6 Étape 6 — écrire la décision, pas le chiffre

Indicateur	Valeur	Seuil	Action si dépassé	Contre-KPI
Rotation	9,42 tours	< 7	révision du réassort en 5 jours	taux de rupture
Taux de rupture	7,29 %	> 10 %	arbitrage de stock par rayon	rotation
Taux de service	78,2 %	< 85 %	revue des tournées le vendredi	coût logistique unitaire
Recouvrement à échéance	74,0 %	< 80 %	relance au 5 ^e jour de retard	CA net

C’est ce tableau — six colonnes, quatre lignes — que le mandat de reprise réclame, et c’est lui que le projet M12.P vous fera produire sur dix indicateurs.

7.7 Étape 7 — mesurer l’adoption, c’est-à-dire la seule chose qui compte

Un dispositif se juge sur des ouvertures, pas sur des indicateurs. Le dossier du projet fournit trois mesures qui suffisent à dater l’abandon :

Semaine	Destinataires	Ouvertures quotidiennes moyennes	Lecture
1	14	6	quatre personnes l’ont ouvert le premier jour, puis le soufflé retombe
2	14	3	le dispositif n’est pas dans le rituel de travail
3 → 11	14	non mesurée	personne ne sait plus qui l’ouvre : l’abandon est consommé

La leçon de méthode. L’ouverture n’est pas un indicateur « technique » réservé à l’informa-
tique : c’est le seul indicateur qui mesure la **valeur** du dispositif. Un rapport jamais ouvert coûte
exactement ce qu’a coûté celui-ci — **7 940 000** FCFA la première année — et rapporte zéro dé-
cision. La première carte de définition qu’un analyste doit écrire n’est donc pas celle du chiffre
d’affaires : c’est celle de l’usage (« nombre de lecteurs actifs par semaine », seuil, action).

8. Erreurs fréquentes

- 1. **Confondre l’outil et la discipline.** Acheter une licence ne crée aucune définition.
- 2. **Croire qu’un indicateur est neutre.** Toute mesure **pousse** un comportement : la rotation pousse à déstocker, la disponibilité pousse à surstocker. D’où le contre-KPI (C04).
- 3. **Publier un chiffre sans son périmètre.** CA net ou toutes lignes : **1,12 %** d’écart.
- 4. **Mélanger deux dénominateurs** dans une même série : **81,0 %** et **78,2 %** ne se comparent pas.
- 5. **Laisser une donnée manquante en silence.** 2 produits sans prix suffisent à rendre une marge inexplicable.
- 6. **Multiplier les indicateurs.** 41 indicateurs, 4 lecteurs le premier jour.
- 7. **Confondre estimation et mesure.** Les **169 386 812** FCFA de CA perdu sont une **estima-
tion** : publiés comme une mesure, ils détruisent la confiance dans tout le reste du tableau.
- 8. **Ne pas dater la donnée.** Un chiffre de la veille et un chiffre du mois se ressemblent.

9. **Croire que l'adoption est acquise.** Elle se mesure (**4** ouvertures le premier jour) et se travaille.
10. **Ne pas écrire l'action.** Un seuil sans action est un ornement — le dossier du projet en contient **41**.

9. Bonnes pratiques professionnelles

- **Écrire la décision avant l'indicateur.** Si vous ne pouvez pas nommer la décision, vous n'avez pas besoin de l'indicateur.
- **Une définition, un endroit.** La couche sémantique est la version outillée de la carte de définition : une seule vérité, écrite.
- **Toujours le contre-KPI.** Aucun indicateur seul : chaque couple protège l'organisation d'un effet pervers (§C04).
- **Distinguer mesure et estimation** dans le titre même de l'indicateur (« CA perdu estimé »).
- **Compter les lecteurs, pas les indicateurs.** L'adoption est l'indicateur de l'indicateur.

Conseil professionnel. Quand on vous demande un indicateur, posez les trois questions du §5.6 **avant** d'écrire la requête, et notez les réponses : elles constituent la carte de définition. Un analyste qui accepte de produire un chiffre sans ces réponses signe, à retardement, un projet abandonné — **7 940 000 FCFA** d'exemple dans ce module.

10. Exercice guidé

Énoncé. Le responsable logistique demande « le coût de la logistique ». Produisez l'indicateur complet : valeur, définition, granularité, seuil, action, contre-KPI.

Étape 1 — mesurer. `SELECT SUM(cout_total_fcfa), SUM(colis) FROM logistique;` donne **172 710 030 FCFA** pour **52 596** colis.

Étape 2 — choisir la forme de l'indicateur. Un coût total n'appelle aucune action ; un coût unitaire, si : **3 284 FCFA** par colis (**249 FCFA** au kilo).

Étape 3 — nommer le piège. **8 766** colis sont facturés au **dépôt central**, qui n'a **aucune vente** : un coût unitaire calculé sur l'ensemble du réseau mélange deux activités.

Étape 4 — écrire la fiche. Formule, source (`logistique_mensuelle.csv`), granularité (mois × magasin), fréquence (mensuelle), responsable (responsable logistique), seuil (**> 3 700 FCFA** par colis), action (revue des tournées et du taux de remplissage), contre-KPI (**taux de service** : on ne remplit pas un camion en livrant en retard).

11. Exercices autonomes

1. **Le coût d'une décision retardée.** Chiffrez ce que représentent les **13 129** jours de rupture en jours-magasin et en FCFA estimés, puis écrivez la phrase que vous diriez à la direction — en distinguant mesure et estimation.
2. **Deux définitions du CA.** Construisez le tableau des deux totaux (net et toutes lignes), calculez l'écart en FCFA et en pourcentage, et expliquez en trois lignes laquelle vous publieriez dans un rapport destiné à la direction financière.

3. **Le dénominateur du taux de service.** Calculez les deux taux (**81,0 %** et **78,2 %**), puis écrivez celle des deux définitions qui a votre préférence, et ce qu'elle cache.
4. **Le magasin sans vente.** Le dépôt central porte **8 766** colis et aucune vente. Proposez deux traitements possibles (l'exclure, ou lui affecter les ventes qu'il sert) et les conséquences de chacun sur le coût logistique unitaire.
5. **Votre propre disposition.** Prenez un dispositif que vous connaissez (vos tableaux Excel) et remplissez les cinq signaux du §5.7 : combien sont déjà au rouge ?

12. Correction détaillée

Exercice 1. Les **13 129** jours de rupture cumulés correspondent au total des jours pendant lesquels un couple produit-magasin-mois n'était pas servable. En FCFA : **169 386 812** FCFA **estimés**. La phrase attendue : « nous estimons à **169 386 812** FCFA le chiffre d'affaires perdu pour cause de rupture sur la période — c'est une estimation calculée à partir du rythme de vente des jours précédents, pas une mesure ; la mesure, elle, est le nombre de jours : **13 129**. »

Exercice 2. Net : **15 595 154 955** FCFA ; toutes lignes : **15 419 985 157** FCFA ; écart **175 169 798** FCFA, soit **1,12 %**. Pour la direction financière, on publie le **net** — et on écrit que les retours sont exclus, parce que c'est la définition qui se rapproche de la facturation.

Exercice 3. **81,0 %** des commandes livrées à l'heure (**7 035** sur **8 681**) ; **78,2 %** des commandes reçues (**7 035** sur **9 000**). La seconde définition, plus sévère, a l'avantage d'être celle que le client perçoit : une commande annulée est un échec, même si elle n'est pas « en retard ». Le risque de la seconde : elle mélange deux causes d'échec (retard et annulation), qu'il faut alors suivre séparément.

Exercice 4. Exclure le dépôt : le coût unitaire porte sur les **43 830** colis des magasins et grimpe mécaniquement, mais il mesure la livraison client. Lui affecter les ventes : on obtient un coût logistique global par activité, au prix d'une clé de répartition — donc d'une convention à écrire. Dans les deux cas, la règle du module tient : la convention se **déclare**.

Exercice 5. Attendu : au moins deux signaux au rouge dans la plupart des cas — les définitions multiples (deux versions d'un même chiffre dans deux fichiers) et l'absence de seuil relié à une action. C'est le diagnostic que le projet M12.P demande de mener, chiffres à l'appui.

13. Mini-projet de chapitre

« **Radiographie d'un dispositif** » (2 h). Choisissez un ensemble de tableaux de suivi existant (vos fichiers, ou les 7 onglets du dossier du projet raté). Produisez une page A4 qui contient : (1) le nombre d'indicateurs et de lecteurs réels ; (2) les définitions divergentes, chiffrées ; (3) les seuils qui n'ont pas d'action associée ; (4) les **3** questions de cadrage posées puis répondues ; (5) une recommandation en trois lignes — réduire, redéfinir, ou arrêter. La règle de notation : chaque affirmation porte une mesure.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Définition de la BI	des définitions partagées + le bon moment + une action décidée d'avance
Les 5 confusions	graphique, informatique, rapport figé, IA, « toutes les données »

Notion	Ce qu'il faut retenir
La chaîne	source → extraction → stockage → modèle → couche sémantique → visualisation → diffusion → décision
Les 3 coûts	décision retardée (13 129 jours), décision fausse (0,78 contre 9,42 tours), non-décision (1 202 550 590 FCFA d'encours)
L'économie	7 940 000 FCFA pour 41 indicateurs et 4 lecteurs : la matière et la décision manquaient
Les 3 questions de cadrage	quelle décision · qui décide et quand · que fait-on si c'est rouge
Les 5 signaux d'échec	adoption, définitions, décisions, périmètre, maintenance

15. À retenir

À retenir. Une mesure se constate, un indicateur se **choisit** : s'il ne déclenche aucune décision, c'est une mesure de plus — et le coût d'un indicateur inutile est exactement celui de l'indicateur utile, plus l'attention qu'il consomme.

- **La BI n'est pas ce qu'on affiche, c'est ce qui est décidé.** Un dispositif dont personne ne peut citer une décision n'est pas un dispositif.
- **Une mesure, un indicateur, un contre-indicateur** : trois mots, trois responsabilités.
- **Le dénominateur est un choix**, et un choix non écrit devient une dispute (**81,0 %** ou **78,2 %** ; **9,42** ou **0,78**).
- **Estimation n'est pas mesure** : l'écrire protège tout le reste du rapport.
- **L'adoption se mesure** : **4** ouvertures le premier jour annonçaient l'abandon de la onzième semaine.

16. Évaluation formative

1. En une phrase, qu'est-ce qui distingue une mesure d'un indicateur ?
2. Citez trois choses que la BI n'est pas, et le coût mesuré de chacune.
3. Que vaut l'écart entre le CA net et le CA de toutes les lignes ? Exprimez-le deux fois.
4. Pourquoi **0,78** et **9,42** décrivent-ils le même stock ?
5. Que décide le mot « annulée » dans un taux de service ?
6. Qu'est-ce qu'une couche sémantique, et pourquoi aucun outil ne la remplace ?
7. Quelles sont les trois questions de cadrage, dans l'ordre ?
8. Quels sont les deux signaux d'échec les plus précoces, et comment les mesure-t-on ?
9. Pourquoi publie-t-on « CA perdu **estimé** » et pas « CA perdu » ?
10. Écrivez la fiche complète d'un indicateur de votre choix, en six lignes, sur le socle du module.

Corrigé : 1. Le KPI est choisi pour déclencher une décision, la mesure est constatée. 2. Graphique (les **41** indicateurs étaient lisibles), projet informatique (**2** définitions sans arbitrage), rapport figé (maintenance résiliée, **1 200 000** FCFA/an). 3. **175 169 798** FCFA, soit **1,12 %**. 4. La première divise par la **somme** des douze stocks, la seconde par leur **moyenne** mensuelle. 5. Elle fait passer le taux de **81,0 %** (dénominateur : livrées) à **78,2 %** (dénominateur : commandes). 6. L'endroit où les définitions sont écrites une fois ; aucun outil ne choisit à votre place ce que « chiffre d'affaires » veut dire. 7. Quelle décision · qui décide et quand · que fait-on si c'est rouge. 8. L'adoption (ouvertures par semaine) et les définitions divergentes (**2** valeurs en réunion). 9. Parce que **169 386 812** FCFA vient d'une reconstitution du rythme de vente, pas

d'un enregistrement. 10. Réponse libre, notée sur la présence des six rubriques : formule, source, granularité, fréquence, responsable, seuil + action + contre-KPI.

Module M12.C02 — Reporting, BI, et les quatre régimes de question

Outils : DuckDB 1.5.5 (exécuté), pandas (contrôle croisé, exécuté), Excel (exécuté). Power BI (cité, non exécuté — règle §1.5). Durée indicative : 5 h. Niveau : N3. Prérequis : M12.C01 (ce qu'est la BI), M09 (questions métier), M11 (agrégats et fenêtres).

L'idée du chapitre. Un rapport périodique et un dispositif de BI ne diffèrent ni par l'outil, ni par le prix : ils diffèrent par **le type de question auquel ils répondent**. « Combien avons-nous vendu ? » se satisfait d'un rapport mensuel ; « pourquoi la marge baisse-t-elle ? » exige de l'exploration ; « que se passera-t-il en décembre ? » exige une projection — et une **erreur constatée** ; « qui relancer, et quand ? » exige une **file d'action**. Ce chapitre classe les quatre régimes, montre comment les reconnaître dans une demande, et mesure les quatre sur le socle plutôt que de les décrire.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **distinguer** reporting, BI, entrepôt de données, lac de données et service en autonomie — par ce qu'ils permettent de faire, et non par leur définition marketing ;
2. **classer** une demande dans l'un des **4** régimes de question (quoi, pourquoi, quoi demain, que faire) et en déduire la réponse technique **et** le rythme de publication ;
3. **reconnaître** les quatre propriétés d'un dispositif de BI — périodicité, exploration, interactivité, gouvernance — et dire laquelle manque à un rapport donné ;
4. **construire une projection honnête** sans modèle statistique, et **publier son erreur** plutôt que sa confiance : trois méthodes, mesurées sur **8** mois du socle ;
5. **transformer un constat en file d'action** : seuil, montant, liste, responsable — l'étape que **41** indicateurs du dossier du projet n'ont jamais franchie.

2. Pourquoi cette notion est importante

La confusion entre reporting et BI coûte cher d'une manière précise : elle fait **financer un rapport périodique en croyant acheter un dispositif d'exploration**. Le dossier du projet raté en est l'illustration exacte — **41** indicateurs répartis en **7** onglets, aucun d'eux explorable, et une question de direction (« où perdons-nous de la marge ? ») qui n'a jamais reçu de réponse alors que toutes les données nécessaires étaient présentes.

Une entreprise qui ne distingue pas les deux se retrouve avec un dispositif qui répond chaque mois à la question de la semaine dernière, et reste muet sur celle du jour. Ce chapitre donne le critère de tri : **quel régime de question votre demande occupe-t-elle ?** — et, selon la réponse, trois choses changent : le rythme, le droit à l'erreur, et la personne à qui l'on s'adresse.

À retenir. Le reporting **réduit** l'incertitude (que s'est-il passé ?) ; la BI **déplace** la question (pourquoi, et que faire ?). Un rapport qui ne réduit ni ne déplace n'occupe aucun des deux rôles : il décore.

3. Explication simple — la météo et le climat

Un bulletin météo et une étude climatique manipulent les mêmes données de température. Ils ne rendent pas le même service.

Le **bulletin météo** paraît à heure fixe, répond toujours à la même question (« quel temps demain ? »), se trompe parfois, et personne ne lui reproche de ne pas expliquer **pourquoi**. Il **réduit l'incertitude de la journée**.

L'**étude climatique** ne paraît pas tous les matins ; elle explore, compare des zones, cherche des tendances sur trente ans, et donne des **fourchettes**, pas des certitudes. Elle ne sert pas à sortir le parapluie, elle sert à décider d'un investissement.

Un dispositif d'entreprise a besoin des deux, et meurt de les confondre : un rapport périodique qui prétend expliquer devient illisible ; une exploration qui prétend donner une réponse quotidienne devient inutile. Ce que le chapitre ajoute, c'est **le nom des régimes** et **la preuve chiffrée** qu'on peut tirer de chacun sur le socle du module.

4. Vocabulaire essentiel

Définition. Le **reporting** est la production périodique de résultats standardisés, à définition figée, destinés à être **lus** — tableau de bord mensuel, état des ventes, balance âgée. Son critère de réussite est la régularité et la comparabilité, pas l'explication.

Définition. La **BI** (Business Intelligence) est l'ensemble des moyens qui permettent d'**explorer** les données pour répondre à des questions non prévues à l'avance, gouvernées par des définitions partagées. Son critère de réussite est le nombre de questions nouvelles qu'un responsable peut poser **sans demander un nouveau rapport**.

Définition. Un **entrepôt de données** — data warehouse — est un stockage structuré, historisé et modélisé (M13), alimenté par les sources opérationnelles, dans lequel la BI vient lire. Un **lac de données** — data lake — conserve les fichiers bruts, souvent hétérogènes, sans schéma imposé : il stocke ce qu'on ne sait pas encore modéliser.

Définition. Le **service en autonomie** — self-service — désigne le fait de laisser un utilisateur métier construire ses propres vues à partir d'une couche sémantique gouvernée. Il ne supprime pas la gouvernance : il la déplace vers les **définitions**, qu'il rend d'autant plus nécessaires.

Définition. Les **quatre régimes de question** sont le descriptif (« que s'est-il passé ? »), le **diagnostique** (« pourquoi ? »), le **prédictif** (« que se passera-t-il ? ») et le **prescriptif** (« que faire ? »). Ce ne sont pas des niveaux de maturité : ce sont **quatre besoins différents**, qui coexistent dans la même entreprise le même jour.

5. Cours approfondi

5.1 Reporting et BI : ce qui les sépare vraiment

Critère	Reporting	BI
Question de départ	figée à la conception	posée par l'utilisateur, dans l'instant
Forme de la réponse	tableau, courbe, chiffre unique	exploration, comparaison, puis décision
Rythme	périodique (jour, semaine, mois)	à la demande
Définition des indicateurs	implicite, dans le code du rapport	écrite , gouvernée (C04)

Critère	Reporting	BI
Coût dominant	la production et la distribution	la matière (sources, définitions)
Signe vital	paraît à l'heure	est ouvert et fait poser des questions

Le tableau ci-dessus se lit dans les deux sens : un rapport mensuel peut être excellent, et un dispositif de BI peut échouer. Ce qui les distingue n'est pas la modernité, c'est la **question servie**.

5.2 Les quatre régimes, et la technique que chacun réclame

Régime	Question type	Réponse technique	Le piège propre au régime
Descriptif — quoi	« combien, quand, où ? »	agrégats, filtres, séries (M07, M11)	croire que le chiffre répond à la question suivante
Diagnostic pourquoi	— « d'où vient cet écart ? »	comparaison, décomposition, drill-down	expliquer une corrélation et l'appeler cause
Prédictif — quoi demain	« que va-t-il se passer ? »	projection simple déclarée , saisonnalité	ne pas publier l'erreur de la méthode
Prescriptif — que faire	« qui, quand, à partir de quel seuil ? »	seuils, files d'action, priorisation	produire une alerte sans responsable

Attention. Un rapport qui **mélange** les quatre régimes dans un même écran fait exactement ce qu'a fait le dispositif abandonné : il affiche un « CA du jour » (descriptif), un « taux de rupture » (diagnostique en apparence), une « cible » (prédictive non dite) et un feu tricolore **sans action** (prescriptif vide). Quatre régimes, aucune réponse : les **14** destinataires ont cessé d'ouvrir.

5.3 Le régime descriptif, mesuré — la notion de tranche

Décrire, c'est choisir des **tranches** lisibles. Sur le socle, le même total se lit dans :

Tranche	Nombre de tranches disponibles	Ce que la tranche permet de dire
Mois	44	la saison, la tendance longue
Magasin x mois	220	quel point de vente décroche, et quand
Magasin x mois x famille	3 515 cellules	quelle famille décroche, et où
Produits	154 produits servis	quelle référence porte le mouvement

Le premier chiffre du module, **15 595 154 955** FCFA, est un total ; les tranches ci-dessus sont ce qui le rend **exploitable**. Un rapport figé n'affiche qu'une tranche — et c'est là toute sa limite, pas dans son outil.

5.4 Le rythme fait partie de l'indicateur

Le socle montre à quel point la **périodicité** change la lecture d'un même chiffre :

- le **lundi** pèse **2 663 495 054** FCFA de chiffre d'affaires, le **dimanche** **829 983 276** — un rapport hebdomadaire publié le dimanche soir compare donc le meilleur jour de la semaine au pire, chaque semaine ;
- 17,8 %** du chiffre d'affaires d'un mois est réalisé après le **25** : un rapport mensuel publié le 1^{er} du mois suivant, avec les données arrêtées au 25, manque près d'un cinquième de l'activité — et ne le dit jamais ;
- la première tranche temporelle du socle (**284 400 304** FCFA en janvier 2023) et la dernière (**368 251 515** FCFA en août 2026) ne se comparent que parce que la **même** tranche est utilisée des deux côtés.

La règle : la périodicité d'un rapport est une **définition** — elle aussi — et elle s'écrit dans son en-tête, au même titre que le périmètre.

Attention. Un rapport est arrêté à une date, et cette date n'est pas la fin du mois. Sans la mention « données arrêtées au 25 », les **17,8 %** d'activité réalisée après le 25 disparaissent de la lecture — et avec eux, la comparabilité d'un mois à l'autre.

5.5 Le régime diagnostique : relier un écart à une cause mesurée

Diagnostiquer n'est pas commenter. Le socle en donne un cas net : la marge brute ressort à **29,12 %** alors que le référentiel annonce des cibles de **18 à 24 %** par rayon.

Trois explications possibles, toutes vérifiables :

1. « nous vendons mieux » — à écarter : aucune décision commerciale n'a changé dans le socle ;
2. « nos coûts d'achat ont baissé » — à écarter : le coût standard est **figé** dans le référentiel ;
3. « nos prix de vente ont monté plus vite que le référentiel ne le dit » — **vérifiée** : entre 2023 et 2026, le prix unitaire moyen des ventes est passé de **8 219 à 10 271 FCFA**, soit **25,0 %**, quand le référentiel porte un prix unique.

Le diagnostic tient en une phrase vérifiable. C'est ce qui le distingue d'un commentaire — et c'est ce que le projet M12.P vous demandera de produire pour **10** indicateurs.

5.6 Le régime prédictif sans modèle : trois projections, une erreur publiée

Beaucoup de projets appellent « prédictif » ce qui n'est qu'une **reconduction**. Le module assume l'inverse : une projection **simple, déclarée**, avec son erreur **constatée**, vaut mieux qu'un modèle que personne ne peut auditer. Trois méthodes, testées sur les **8** mois de 2026 du socle :

Méthode	Règle	Erreur moyenne constatée (MAPE)
Reconduire le mois précédent	« décembre vaudra novembre »	12,2 %
Reconduire l'an dernier	« décembre vaudra décembre N-1 »	13,5 %
Moyenne des 12 derniers mois	« chaque mois vaut la moyenne »	14,3 %

Trois enseignements, tous contre-intuitifs :

1. la méthode **la plus simple gagne** sur cette série — la saisonnalité du socle n'est pas assez régulière pour battre la simple reconduction ;
2. aucune méthode ne descend sous **12 %** d'erreur : un dispositif qui promet « la prévision du mois » à 3 % près doit **prouver** sa méthode, sinon il vend une croyance ;
3. l'erreur se **publie** avec la prévision : « 368 251 515 FCFA $\pm$ 12 % » n'est pas la même phrase que « 368 251 515 FCFA ».

5.7 Le régime prescriptif : transformer un constat en file d'action

Le socle fournit **1 202 550 590 FCFA** d'encours ouvert. Ce chiffre est descriptif. Ce qui en fait une prescription, c'est la **balance âgée** et la **file** qu'elle dessine :

Tranche d'ancienneté	Factures	Montant	Ce qu'on en fait
0 à 30 jours	71	70 049 670 FCFA	rien : c'est le délai normal
31 à 60 jours	30	26 248 320 FCFA	rappel automatique
61 à 90 jours	16	14 913 570 FCFA	appel du commercial
plus de 90 jours	1 155	1 091 339 030 FCFA	contentieux / plan d'apurement

La file prescriptive est la dernière ligne : **1 155** factures, **1 091 339 030** FCFA, soit **90,8 %** de l'encours ouvert. Et la mesure qui la rend exécutable existe déjà dans le socle : parmi les factures payées **en retard**, **1 068** l'ont été avec un retard **médian de 5 jours**. Autrement dit, le retard « normal » se règle en une semaine ; ce qui reste au-delà de **90** jours n'est plus un retard, c'est un **impayé** — et c'est un autre métier, avec un autre responsable.

Dans les faits. L'atelier du manuel applique la même règle à ses propres mesures : la balance âgée ci-dessus est recalculée à chaque relevé par `tools/kpi_M12.py`, et ses quatre tranches s'additionnent exactement à l'encours ouvert du module (**1 202 550 590** FCFA). Une répartition dont les tranches ne retombent pas sur le total est le premier signe d'un indicateur mal découpé.

5.8 Périodicité, exploration, interactivité, gouvernance : les quatre propriétés

Propriété	La question qu'elle règle	Ce qui la mesure	Où elle a manqué au projet
Périodicité	à quelle fréquence, avec quel délai ?	date de publication vs date des données	aucune fréquence annoncée par indicateur
Exploration	peut-on descendre d'un total à sa cause ?	nombre de tranches accessibles sans nouvelle demande	7 onglets figés, aucune descente
Interactivité	qui peut filtrer quoi, et à quel coût ?	temps de réponse et droits d'accès	filtres présents, mais définitions non filtrées
Gouvernance	qui décide de la définition, et où est-elle écrite ?	existence d'une carte de définition	2 versions du CA, 3 sources discordantes

Les quatre propriétés se lisent dans cet ordre, et la dernière **conditionne** les trois premières : une exploration rapide sur des définitions flottantes produit des décisions rapides et fausses.

5.9 Ce que ce chapitre ne couvre pas

Les **critères de fabrication** d'un indicateur (les six critères, la carte de définition, le contre-KPI) sont le chapitre C04 ; la **couche sémantique** et l'architecture complète sont le C05 ; la **conduite** d'un projet d'exploration — adoption, formation, maintenance — est le C06 ; les méthodes de prévision proprement dites, avec leur validation, appartiennent à M19. Ce chapitre-ci ne promet aucune prédiction : il apprend à **ranger les questions** et à refuser celles qu'on ne peut pas servir honnêtement.

6. Exemple concret — la même demande, quatre réponses

Un directeur régional écrit : « Je veux savoir où nous perdons de l'argent. »

Régime descriptif. On lui livre la marge par magasin et par famille : **29,12 %** au global, et les tranches qui baissent. Il sait **où** — il ne sait pas **pourquoi**.

Régime diagnostique. On décompose l'écart : prix de vente (**+25,0 %** de 2023 à 2026), coût standard figé, remises par famille, et l'effet des **2** produits sans prix au référentiel. La réponse tient en une phrase, et elle nomme une **cause** : l'écart vient des prix, pas d'une dérive de coût.

Régime prédictif. On projette la marge des trois prochains mois avec la méthode la plus simple, **12,2 %** d'erreur annoncée. Le directeur sait ce qu'il peut en faire : anticiper une tendance, pas décider un investissement.

Régime prescriptif. On livre la file d'action : les familles à marge basse **et** à volume élevé, triées par montant, avec un seuil (« en dessous de **20 %** de marge sur plus de **500 000** FCFA mensuels »), un responsable (chef de rayon) et un délai (revue d'assortiment du mois).

La leçon. La demande initiale était floue ; elle contenait les quatre régimes. Un analyste qui répond aux quatre d'un coup produit un écran illisible ; un analyste qui refuse les trois premiers pour ne servir que le quatrième produit une décision. **L'ordre de service est un choix professionnel**, et c'est le cœur de ce chapitre.

7. Démonstration pas à pas — quatre régimes sur le socle

Tout se rejoue avec `03_exercices/dossier_M12/connexion.py`.

7.1 Étape 1 — descriptif : le total, puis les tranches

```
SELECT strftime(date_vente, '%Y-%m') AS mois, ROUND(SUM(montant_ttc)) AS ca
FROM ventes WHERE est_retour = 0 GROUP BY 1 ORDER BY 1;
```

44 lignes, de **284 400 304** FCFA (janvier 2023) à **368 251 515** FCFA (août 2026). Un total unique (**15 595 154 955** FCFA) ne décide rien ; la série, elle, commence à parler.

7.2 Étape 2 — descriptif fin : la tranche qui change la lecture

```
SELECT strftime(date_vente, '%w') AS jour, ROUND(SUM(montant_ttc)) AS ca
FROM ventes WHERE est_retour = 0 GROUP BY 1 ORDER BY 1;
```

Lundi **2 663 495 054** FCFA, dimanche **829 983 276** FCFA : un rapport hebdomadaire « semaine du dimanche au samedi » et un rapport « du lundi au dimanche » ne racontent pas la même histoire, avec **x 3,21** d'écart entre le meilleur et le pire jour.

7.3 Étape 3 — diagnostique : de l'écart à la cause

```
SELECT year(date_vente) AS annee, ROUND(AVG(prix_unitaire_ht)) AS prix_moyen
FROM ventes WHERE est_retour = 0 GROUP BY 1 ORDER BY 1;
```

La hausse de **8 219** à **10 271** FCFA (**+25,0 %**) explique la marge apparente de **29,12 %** face à une cible de **18** à **24 %**. Le diagnostic s'arrête quand la cause est **mesurée**, pas quand elle est plausible.

7.4 Étape 4 — prédictif : trois méthodes, une erreur publiée

Méthode	Règle appliquée à septembre 2026	Erreur constatée sur 2026
Reconduction du mois précédent	août 368 251 515 FCFA	12,2 %
Reconduction de l'an dernier	septembre 2025	13,5 %
Moyenne des 12 derniers mois	moyenne glissante	14,3 %

On publie la projection **avec** son erreur : c'est la seule façon d'être honnête sans modèle.

7.5 Étape 5 — prescriptif : la file, pas l'alerte

```
SELECT CASE WHEN date_diff('day', date_echeance, DATE '2026-08-31') <= 30 THEN '0-30 j'
            WHEN date_diff('day', date_echeance, DATE '2026-08-31') <= 60 THEN '31-60 j'
            WHEN date_diff('day', date_echeance, DATE '2026-08-31') <= 90 THEN '61-90 j'
            ELSE 'plus de 90 j' END AS tranche,
COUNT(*) AS factures, ROUND(SUM(montant_ttc)) AS montant
FROM encaissement WHERE date_encaissement IS NULL GROUP BY 1 ORDER BY 1;
```

Quatre lignes, dont la dernière porte **1 155** factures et **1 091 339 030** FCFA : la file de contentieux, **90,8 %** de l'encours. C'est une **liste de travail**, pas une alerte.

7.6 Étape 6 — l'arbitrage entre régimes

Demande reçue	Régime	Livrable	Délai
« le CA du mois »	descriptif	rapport périodique	automatique
« d'où vient la baisse de marge ? »	diagnostique	note d'une page, cause mesurée	quelques jours
« la prévision de décembre »	prédictif	fourchette $\pm$ erreur publiée	quelques jours
« que fait-on des impayés ? »	prescriptif	file de relance priorisée	un jour

Ce tableau est ce que le module appelle **un catalogue de services** : il dit à l'entreprise ce qu'elle peut demander, et à quel prix en délai. Il remplace avantageusement une liste de **41** indicateurs.

8. Erreurs fréquentes

1. **Appeler « BI » un rapport périodique.** Le critère est l'exploration, pas le nom.
2. **Promettre du prédictif sans publier l'erreur.** Sur ce socle, la meilleure méthode simple se trompe de **12,2 %**.
3. **Confondre corrélation et cause** dans le diagnostic : la marge monte **et** les prix montent — c'est la seconde qui explique la première, et les deux sont mesurées.
4. **Publier une alerte sans responsable.** Le dossier du projet contient **41** seuils, **0** action.
5. **Ignorer la périodicité des données.** **17,8 %** du mois se réalise après le **25**.
6. **Comparer deux tranches de tailles différentes** : lundi contre dimanche, $\times$ **3,21**.
7. **Confondre lac et entrepôt.** Stocker brut ne dispense pas de définir — cela déplace seulement le travail plus loin.
8. **Croire que le service en autonomie supprime la gouvernance.** Il l'exige : sans définitions, il produit **2** versions de chaque chiffre, à la vitesse de l'utilisateur.
9. **Répondre aux quatre régimes dans un même écran.** Sept onglets, quatre besoins, aucune réponse.
10. **Oublier de dater les données.** Un bulletin sans date n'est pas un bulletin.

9. Bonnes pratiques professionnelles

- **Trier la demande avant de l'instruire.** Une question = un régime = un livrable = un délai.
- **Publier une fourchette, jamais une promesse.** Une projection s'écrit « $\pm$ **12 %** », pas « exacte ».
- **Mesurer la cause avant de la nommer**, et s'arrêter quand elle est mesurée.

- **Transformer tout constat chiffré en file d'action** : seuil, montant, liste, responsable, délai.
- **Dater les tranches** et écrire la périodicité dans l'en-tête du rapport.

Conseil professionnel. Quand une direction demande « un tableau de bord avec tout », répondez par le tableau des quatre régimes du §7.6 : il montre que « tout » signifie quatre livrables différents, dont trois ne sont pas des écrans. C'est la réponse qui a manqué au prestataire du dossier — et qui lui a coûté **7 940 000 FCFA** de dispositif abandonné.

10. Exercice guidé

Énoncé. Classez ces quatre demandes dans les régimes, et dites pour chacune ce que vous livrez : (a) « combien de ruptures cette semaine ? » ; (b) « pourquoi Kaya rompt-il plus que les autres ? » ; (c) « allons-nous rompre la semaine prochaine ? » ; (d) « quelles références faut-il réapprovisionner en priorité ? ».

Étape 1 — classer. (a) descriptif, (b) diagnostique, (c) prédictif, (d) prescriptif.

Étape 2 — chiffrer le contexte. Le socle compte **2 428** ruptures sur **33 296** couples servis (**7,29 %**), **13 129** jours de rupture et **169 386 812 FCFA** **estimés** de chiffre d'affaires perdu.

Étape 3 — choisir le livrable. (a) un rapport hebdomadaire ; (b) une note comparant Kaya aux autres magasins, par famille et par rotation ; (c) une projection **avec** son erreur publiée ; (d) une file triée par couple (jours × rotation), avec un seuil et un responsable.

Étape 4 — nommer la limite. Pour (c), rappelez que la meilleure projection simple du socle se trompe de **12,2 %** : la file de (d) vaut mieux qu'une prévision trop fine.

11. Exercices autonomes

1. **Tranches.** Combien de cellules magasin-mois-famille le socle permet-il (**3 515**), et combien de questions vos tableaux de suivi habituels en rendent-ils accessibles sans travail manuel ?
2. **Périodicité.** Un rapport mensuel est arrêté au **25**. Quelle part de l'activité manque-t-il, et comment l'écrire dans son en-tête ?
3. **Diagnostic guidé.** La marge ressort à **29,12 %** pour une cible de **18 à 24 %** : proposez trois causes possibles, puis écartez-en deux par la mesure.
4. **Prévision honnête.** Choisissez un mois de 2026 et appliquez les trois méthodes du §5.6 ; publiez votre erreur en pourcentage.
5. **File d'action.** Construisez la balance âgée des **9 000** factures, puis écrivez la file de relance avec seuil, montant, responsable et délai.
6. **Le régime oublié.** Reprenez les **7** onglets du dossier du projet et dites, pour chacun, le régime qu'il prétend servir et celui qu'il sert réellement.

12. Correction détaillée

Exercice 1. **3 515** cellules (magasin × mois × famille) contre **220** couples magasin-mois : la même donnée multiplie par seize le nombre de questions accessibles. Un rapport figé en ouvre une.

Exercice 2. 17,8 % du chiffre d'affaires mensuel se réalise après le 25. L'en-tête attendu : « données arrêtées au 25 du mois, soit environ 82 % du mois ; le complément est intégré au rapport du mois suivant. »

Exercice 3. Causes plausibles : (1) hausse des prix de vente, (2) baisse des coûts d'achat, (3) erreur de calcul. On écarte (2) — le coût standard est **figé** — et (3) — le contrôle croisé côté pandas retrouve **15 595 154 955 FCFA**. Reste (1), mesurée : **8 219 → 10 271 FCFA (+25,0 %)**.

Exercice 4. Exemple sur août 2026 : reconduction du mois précédent (juillet) ; reconduction d'août 2025 ; moyenne des douze derniers mois. L'erreur publiée doit être celle de la méthode, mesurée sur 8 mois : **12,2 %, 13,5 % ou 14,3 %** selon la méthode choisie.

Exercice 5. Balance âgée : **71** factures (**70 049 670 FCFA**) en 0-30 jours, **30 (26 248 320)** en 31-60, **16 (14 913 570)** en 61-90, **1 155 (1 091 339 030)** au-delà de 90 jours. File de relance : les **1 155** factures de plus de 90 jours (**90,8 %** de l'encours), responsable : responsable recouvrement, délai : revue hebdomadaire du lundi.

Exercice 6. Attendu, onglet par onglet : « Activité » sert le descriptif ; « Écarts » prétend le prescriptif mais n'a pas d'action ; « Ruptures » prétend le diagnostique mais ne montre que des comptages ; « Livraisons » et « Encaissements » servent le descriptif ; « Logistique » sert le descriptif ; le tableau de bord, dans son ensemble, **ne sert aucun des quatre régimes** : il les juxtapose.

13. Mini-projet de chapitre

« **Le catalogue de services d'un analyste** » (2 h). Produisez, pour une entreprise que vous connaissez, une page qui contient : (1) les **8** questions les plus souvent posées, classées par régime ; (2) pour chacune, le livrable, le délai et le responsable ; (3) les **3** questions auxquelles vous refusez de répondre, et pourquoi (méthode indisponible, donnée absente, définition non gouvernée) ; (4) la question de direction que votre catalogue ne sert pas encore, et le moyen de la servir. Règle de notation : chaque affirmation chiffrée, chaque refus argumenté.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Reporting vs BI	la question servie, pas l'outil : figée et périodique contre exploratoire et gouvernée
Les 4 régimes	descriptif (quoi), diagnostique (pourquoi), prédictif (quoi demain), prescriptif (que faire)
Descriptif	44 mois, 220 couples magasin-mois, 3 515 cellules : la tranche fait la valeur
Diagnostique	29,12 % de marge pour une cible de 18 à 24 % : la cause mesurée est le prix (+25,0 %)
Prédictif	trois méthodes simples, erreurs 12,2 %, 13,5 %, 14,3 % : on publie l'erreur
Prescriptif	1 155 factures de plus de 90 jours, 1 091 339 030 FCFA , 90,8 % de l'encours
Les 4 propriétés	périodicité, exploration, interactivité, gouvernance — la dernière conditionne les autres

15. À retenir

À retenir. Un rapport qui répond à une question qu'on ne se pose plus est un coût ; un dispositif qui répond à quatre régimes à la fois est un brouillard. Le travail de l'analyste commence par le **tri** de la demande.

- **La périodicité est une définition** : données arrêtées au **25**, c'est **17,8 %** de mois absent.
- **Une prévision sans erreur publiée n'est pas une prévision**, c'est une opinion — ici **12,2 %** au mieux.
- **Une alerte sans file d'action est un ornement** : le dossier du projet en aligne **41**.
- **Comprendre n'est pas décrire** : le diagnostic s'arrête quand la cause est **mesurée**.

16. Évaluation formative

1. Citez deux différences de fond entre reporting et BI, autres que l'outil.
2. Nommez les quatre régimes et la question type de chacun.
3. Que signifie « tranche » en descriptif, et combien le socle en offre-t-il (magasin × mois × famille) ?
4. Pourquoi la périodicité d'arrêt des données doit-elle figurer dans l'en-tête d'un rapport ?
5. Quelle est la cause mesurée du taux de marge apparent de **29,12 %** ?
6. Quelle méthode de projection se trompe le moins sur ce socle, et de combien ?
7. Qu'est-ce qu'une file d'action, et qu'est-ce qui la distingue d'une alerte ?
8. Quel est le montant de l'encours de plus de 90 jours, et sa part dans l'encours total ?
9. Pourquoi le service en autonomie rend-il la gouvernance **plus** nécessaire ?
10. Parmi les quatre propriétés du §5.8, laquelle conditionne les trois autres, et pourquoi ?

Corrigé : 1. La définition des indicateurs (implicite contre écrite) et le signe vital (paraître à l'heure contre être ouvert). 2. Descriptif (« que s'est-il passé ? »), diagnostique (« pourquoi ? »), prédictif (« que se passera-t-il ? »), prescriptif (« que faire ? »). 3. Une découpe lisible des données ; **3 515** cellules. 4. Parce que **17,8 %** du mois se réalise après le **25** : sans cette mention, le lecteur croit lire le mois entier. 5. La hausse des prix de vente unitaires, de **8 219** à **10 271** FCFA (**+25,0 %**) entre 2023 et 2026, pour un coût standard figé. 6. La reconduction du mois précédent, avec **12,2 %** d'erreur moyenne sur **8** mois. 7. Une liste de travail priorisée avec seuil, montant, responsable et délai ; l'alerte ne dit pas qui agit. 8. **1 091 339 030** FCFA de **1 202 550 590**, soit **90,8 %**. 9. Parce qu'il multiplie les producteurs de chiffres : sans définitions écrites, chacun obtient la sienne. 10. La gouvernance : elle fixe le sens des chiffres que les trois autres propriétés diffusent.

Module M12.C03 — Métriques, dimensions, faits, grain : le vocabulaire exact

Outils : DuckDB 1.5.5 (exécuté), pandas (contrôle croisé, exécuté), Excel (exécuté). Power BI (cité, non exécuté — règle §1.5). Durée indicative : 5 h. Niveau : N3. Prérequis : M12.C01—C02, M07 (tables et jointures), M11.C01 (grain d'une requête).

L'idée du chapitre. Presque tous les débats de chiffres en réunion sont des malentendus de **vocabulaire** : on ne parle pas du même grain, du même dénominateur, ou de la même mesure. Ce chapitre installe les mots exacts — métrique, dimension, fait, grain, additivité — et les éprouve par huit tables du socle, où **tout** se mesure : un chiffre d'affaires qui s'additionne parfaitement (**15 595 154 955 FCFA**), un comptage de clients qui ne s'additionne **pas** (**188 886** au lieu de **23 497**), une moyenne de moyennes qui se trompe de **445 FCFA**, et une clé oubliée qui multiplie un total par **15,8** sans lever la moindre erreur.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **nommer** les quatre objets du vocabulaire — **fait, dimension, métrique, grain** — et dire lequel est en cause quand deux personnes obtiennent deux chiffres ;
2. **déterminer le grain** d'une table ou d'un résultat : « que représente **une** ligne ? » — et vérifier la réponse par une clé distincte, pas par une impression ;
3. **classer une mesure** en additive, semi-additive ou non additive, et prévoir ce qui se passera si on l'additionne quand même ;
4. **distinguer** métrique, indicateur et indicateur clé, puis ranger les indicateurs par **horizon de décision** (opérationnel, tactique, stratégique) — trois horizons, trois fréquences, trois publics ;
5. **détecter** les trois pièges mécaniques qui fabriquent de faux totaux : la moyenne de moyennes, le comptage distinct additionné, et la jointure à clé incomplète.

2. Pourquoi cette notion est importante

Vous avez déjà rencontré chacun de ces pièges, séparément : le cumul qui double (**77 975 774 775 FCFA** pour une jointure sans la clé du magasin, M11.C01), la rotation divisée par douze quand on somme des stocks (M12.C01), le classement instable faute de clé de départage (M11.C02). Ce chapitre les réunit sous un seul vocabulaire, parce que la cause est toujours la même : **on n'a pas dit de quoi on parlait**.

En réunion, cela donne une scène qui se répète : le contrôleur de gestion annonce **23 497** clients, le responsable marketing en annonce **188 886**, et les deux ont raison. Le premier compte des **clients distincts** ; le second a additionné le nombre de clients de chaque produit, ce qui compte **un client autant de fois qu'il a de produits**. Le mot manquant n'est pas « client », c'est **additivité**.

À **retenir**. Deux chiffres qui diffèrent ne sont pas forcément contradictoires : ils peuvent simplement ne pas mesurer la même chose. Le vocabulaire de ce chapitre sert à le **dire avant** la réunion, et non à le découvrir pendant.

3. Explication simple — les briques et le ciment

Un jeu de données bien construit ressemble à un mur de briques. Les **faits** sont les briques : des événements qui se sont produits — une ligne de vente, une commande, une facture encaissée. Les **dimensions** sont le ciment et les plans de coupe : le temps, le magasin, le client, le produit, le canal — les axes selon lesquels on regarde les briques.

Un mur se décrit de trois façons, et confondre les trois fait s'écrouler le devis :

- **combien de briques** — un **comptage**. Il s'additionne tant qu'on ne compte pas deux fois la même brique.
- **combien pèse le mur** — un **montant**. Il s'additionne toujours, et c'est ce qui autorise les sous-totaux.
- **combien de briques en moyenne par rangée** — une **moyenne**. Elle ne s'additionne jamais, et c'est précisément là que les réunions dérapent.

Le **grain**, enfin, répond à une question qu'on oublie de poser : une brique, c'est quoi ? Une ligne de vente ? Un ticket ? Une journée ? Tant que la réponse n'est pas écrite, tout ce qui suit est une hypothèse.

4. Vocabulaire essentiel

Définition. Un **fait** est un événement mesurable et daté — une vente, une commande, une rupture, un encaissement. Dans une table de faits, chaque ligne enregistre **un** événement à **un** niveau de détail donné.

Définition. Une **dimension** est un axe d'analyse stable, par lequel on découpe les faits : temps, magasin, client, produit, canal, mode de paiement. Une dimension se décrit par sa **cardinalité** — son nombre de valeurs distinctes (**magasin** en a **5**, **produit** en a **154**).

Définition. Une **métrique** est une quantité mesurée sur les faits : un montant, une quantité, une durée, un effectif. Une **mesure** est la valeur d'une métrique à un grain donné ; un **indicateur** est une mesure **choisie** pour suivre une performance (C01).

Définition. Le **grain** d'une table ou d'un résultat est ce que représente **une ligne** : une ligne de vente, un ticket, un couple produit-mois, une commande, une journée. Le grain se **prouve** par une clé dont le nombre de valeurs distinctes égale le nombre de lignes.

Définition. Une mesure est **additive** quand elle peut être additionnée sur toutes les dimensions (les montants), **semi-additive** quand elle s'additionne sur certaines seulement (un stock, qui s'additionne dans l'espace mais pas dans le temps), et **non additive** quand elle ne s'additionne nulle part (un taux, une moyenne, un comptage d'éléments distincts).

5. Cours approfondi

5.1 La table de faits et la table de dimensions

Le socle de M12 contient huit tables, et le vocabulaire les sépare sans ambiguïté :

Table	Nature	Ce que représente une ligne
ventes	faits	une ligne de vente (ou de retour)
commande	faits	une commande client
encaissement	faits	une facture et son encaissement
rupture	faits	un épisode de rupture, pour un produit, un magasin, un mois
logistique	faits	le coût logistique d'un magasin sur un mois
produit	dimension	un article du catalogue
magasin	dimension	un point de vente ou le dépôt
calendrier	dimension	un jour, avec ses attributs

Attention. Une table peut être **les deux** selon l'usage : `stock_mensuel` est un **fait** (un relevé de stock par produit et par mois) mais se lit comme une **dimension** du produit quand on s'en sert pour filtrer. Le vocabulaire ne décrit pas les fichiers, il décrit **ce qu'on en fait** — et c'est pour cela qu'il doit être écrit dans le rapport, pas supposé.

5.2 Le grain : la question posée huit fois

La même question — « que représente une ligne ? » — posée aux huit tables du socle, avec la preuve par la clé :

Table	Lignes	Clés distinctes	Grain prouvé	Unique ?
ventes	240 000	240 000	ligne de vente	oui
commande	9 000	9 000	commande	oui
encaissement	9 000	9 000	facture	oui
stock_mensuel	6 776	6 776	produit × mois	oui
rupture	2 428	2 428	produit × magasin × mois	oui
logistique	264	264	magasin × mois	oui
produit	154	154	produit	oui
magasin	6	6	magasin	oui

8 tables, 8 clés uniques : le socle est propre, et le grain de chacune est **démontré** par le fait que le nombre de clés distinctes égale le nombre de lignes. C'est le premier contrôle à faire sur n'importe quel fichier qu'on vous transmet — avant de compter quoi que ce soit, demandez-vous **quelle colonne est unique**, et vérifiez-le.

Quand aucune colonne n'est unique, le grain est une **combinaison** : c'est le cas de `rupture` (produit + magasin + mois) et de `stock_mensuel` (produit + mois). Deux tables, deux grains différents — et c'est cette différence qui explique le piège du §5.7.

5.3 Les dimensions du socle et leur cardinalité

Dimension	Valeurs distinctes	Ce qui la rend utile — ou piègeuse
magasin	5	peu de valeurs : les tableaux croisés restent lisibles
ville	4	moins de villes que de magasins : deux magasins partagent Ouagadougou
quartier	5	même cardinalité que magasin, mais ce n'est pas la même dimension
vendeur	22	trop de valeurs pour un tableau croisé complet
mode_paieement	5	—
canal	3	15 couples canal × mode : toutes les combinaisons existent
produit	154	à croiser par famille, jamais en clair
categorie brut) (libellé	16	16 libellés pour 7 familles réelles : la dimension est sale
sous_categorie	12	niveau intermédiaire, utile pour ne pas croiser 154 produits
client	23 497	une dimension à part entière : elle porte le RFM (M11.C04)
mois	44	la dimension temps, au grain du suivi
jour	1 339	le grain le plus fin disponible

Deux leçons de vocabulaire dans ce tableau :

1. **Une dimension doit être propre.** Les 16 libellés de catégorie pour 7 familles réelles (M11.C05) sont un défaut de dimension : tant qu'il n'est pas corrigé, tout croisement par catégorie est faux — non pas par erreur de requête, mais par **défaut de référentiel**.
2. **La cardinalité décide de la forme du tableau.** 5 magasins se croisent à l'écran ; 154 produits se regroupent d'abord par famille ; 23 497 clients ne se croisent jamais en clair.

5.4 Additif, semi-additif, non additif

Trois familles, et pour chacune une mesure du socle qui **prouve** la règle :

Famille	Exemple	Le test	Résultat mesuré
Additive	chiffre d'affaires	somme par magasin = total ?	15 595 154 955 FCFA des deux côtés : oui
Additive	effectif de lignes	somme par magasin = total ?	240 000 des deux côtés : oui
Semi-additive	stock en unités	dernier mois contre somme des 44 mois	45 882 contre 2 034 733 (× 44,30)
Non additive	clients distincts	somme par produit contre total	188 886 contre 23 497 (× 8,04)
Non additive	clients distincts	somme par magasin contre total	90 242 contre 23 497 (× 3,84)
Non additive	panier moyen	moyenne des moyennes contre vraie moyenne	107 840 contre 107 396 FCFA

L'additivité n'est pas une propriété du chiffre, c'est une propriété du couple chiffre-dimension. Le comptage de ruptures s'additionne par magasin (**2 428** des deux côtés) parce qu'une rupture appartient à un seul magasin ; le comptage de clients ne s'additionne pas par produit, parce qu'un client achète **8,04** produits en moyenne. La différence entre les deux cas n'est pas la mesure : c'est la **dimension**.

Attention. Un tableau croisé peut être juste ligne par ligne et faux en bas à droite. Additionner la colonne « clients » d'un croisement par produit donne **188 886** : le total général du tableau est alors **huit fois** le nombre réel de clients. La règle : **le total d'une colonne non additive se recalcule, il ne s'additionne pas.**

5.5 Le piège de la moyenne de moyennes, chiffré

Le tableau du §5.3 se lit ligne à ligne. Le panier moyen par magasin vaut :

Magasin	Panier moyen par ticket
Kaya	111 447 FCFA
Koudougou	108 035 FCFA
Ouaga 2000	107 374 FCFA
Gounghin	106 621 FCFA
Bobo Kibidwé	105 724 FCFA

La moyenne de ces cinq valeurs vaut **107 840** FCFA. La **vraie** moyenne du panier, calculée sur l'ensemble des tickets, vaut **107 396** FCFA. Écart : **445** FCFA, soit **0,4 %**.

L'écart est faible ici — parce que les cinq magasins ont des volumes comparables. Mais la **méthode** est fautive : une moyenne de moyennes ne pondère pas. Si Kaya réalisait 90 % des tickets, la même méthode produirait un écart de plusieurs milliers de francs. La règle est donc absolue, quelle que soit la taille de l'erreur observée : **on recalcule la moyenne sur les lignes, on ne moyenne pas des moyennes.**

5.6 Métrique, indicateur, indicateur clé — et les trois horizons

Le vocabulaire se range enfin par **public** :

Niveau	Question	Fréquence	Exemple mesuré	Public
Opérationnel	« que se passe-t-il aujourd'hui ? »	jour, heure	26 933 751 FCFA le 17/03/2026 contre 11 646 867 FCFA en moyenne (× 2,31)	chef de rayon, responsable logistique
Tactique	« où en sommes-nous ce mois ? »	semaine, mois	mois record 2026-03 (533 572 353) FCFA)	directeur commercial
Stratégique	« où allons-nous ? »	trimestre, année	3 443 581 547 FCFA en 2023 → 3 360 553 372 FCFA sur 8 mois de 2026	direction générale, associés

Trois remarques qui font la différence entre un tableau de bord et un catalogue :

1. **Un indicateur opérationnel doit être actionnable dans l'heure.** Le chiffre du 17/03/2026 (× **2,31**) sert à comprendre sur le moment ; publié au trimestre, il ne sert à rien.
2. **Un indicateur tactique compare à une cible,** jamais à lui-même : le mois record ne dit rien sans l'objectif du mois.
3. **Un indicateur stratégique accepte une incertitude.** L'exercice 2026 est **partiel (8 mois sur 12)** : l'annoncer fait partie de l'indicateur, et non de sa légende.

Métrique, indicateur, indicateur clé s'emboîtent : la métrique est la quantité (un montant), le **KPI** est la métrique choisie pour une décision, et l'**indicateur clé** est celui qu'on suit jusqu'au bout — avec un seuil et une action (C04). Une entreprise a des dizaines de métriques, une poignée de KPI, et **trois ou quatre** indicateurs clés.

5.7 La clé oubliée : le piège qui fabrique un total

Voici la démonstration la plus coûteuse du chapitre. On joint les ventes aux ruptures, en oubliant la clé du mois :

```
SELECT COUNT(*), ROUND(SUM(v.montant_ttc))
FROM ventes v
JOIN rupture r ON r.id_produit = v.id_produit
WHERE v.est_retour = 0;
```

Résultat : **3 808 099** lignes (× **15,9**) et **246 567 448 121** FCFA (× **15,8**).

La requête **s'exécute**, ne lève **aucune erreur**, et produit un chiffre d'affaires **quinze fois** trop élevé. Chaque ligne de vente a été répétée autant de fois que son produit compte de lignes de rupture. Le grain des deux tables n'était pas le même — **ventes** est au grain ligne de vente, **rupture** au grain produit × magasin × mois — et la jointure, faite sur **une seule** des trois clés du second grain, a **multiplié** au lieu de **filtrer**.

Le contrôle qui rattrape ce genre d'erreur tient en une ligne et devrait être systématique :

```
nombre de lignes AVANT la jointure == nombre de lignes APRÈS ?
```

S'il a changé, la jointure a **dupliqué** : ce n'est pas forcément faux (un fan-out voulu existe), mais c'est **toujours** une décision à écrire.

5.8 Ce que ce chapitre ne couvre pas

La **fabrication** d'un indicateur — six critères, carte de définition, contre-KPI, effets pervers — est le chapitre C04 ; l'**architecture** technique (couche sémantique, entrepôt, modèle physique) est le C05 ; la **modélisation en étoile** proprement dite — tables de faits, tables de dimensions, clés substituts — appartient à M13. Ce chapitre-ci s'arrête au vocabulaire et à ses pièges : il nomme les objets et prouve leur comportement sur huit tables.

6. Exemple concret — le tableau croisé qui se réconcilie

La cellule commerciale demande « le chiffre d'affaires et le nombre de clients par magasin ». Le tableau se construit en une requête, et se **réconcilie** en trois contrôles.

Magasin	CA net (FCFA)	Clients distincts	Panier moyen
Ouaga 2000	5 312 205 459	à calculer	107 374
Kaya Marché	1 733 230 083	à calculer	111 447

Les trois contrôles de réconciliation :

1. **Somme des lignes = total** pour le chiffre d'affaires : **15 595 154 955** FCFA des deux côtés — l'additivité est vérifiée, le tableau peut porter un total.
2. **Somme des lignes ≠ total** pour les clients : la colonne ne portera **pas** de total, mais un renvoi vers le chiffre global (**23 497** clients pour l'enseigne, un client pouvant acheter dans plusieurs magasins).
3. **Le panier moyen ne s'additionne pas** : il se recalcule (**107 396** FCFA), et le tableau l'indique en note plutôt que d'afficher une moyenne de moyennes à **107 840** FCFA.

La leçon. Un tableau de bord professionnel ne se juge pas à ce qu'il affiche mais à ce qu'il **refuse** d'afficher : un total sur une colonne non additive est une erreur qui se propage à toutes les décisions prises dessus.

Dans les faits. L'atelier du module passe ce contrôle tout seul : la table des grains (**8** tables, **8** clés uniques), l'additivité du chiffre d'affaires, la non-additivité des clients et l'écart de la moyenne de moyennes sont recalculés à chaque relevé par `tools/kpi_M12.py`. Un chiffre de vocabulaire qui n'est pas mesuré n'est pas une règle : c'est une opinion sur une règle.

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — prouver le grain avant de compter

```
SELECT COUNT(*) AS lignes, COUNT(DISTINCT id_vente) AS cles FROM ventes;
```

240 000 et **240 000** : le grain est **une ligne de vente**, et il est prouvé. Refaites le test sur les huit tables du socle : **8** sur **8** ont une clé unique. Sur un fichier reçu, c'est le premier contrôle à passer — et le seul qui évite de compter deux fois.

7.2 Étape 2 — mesurer une dimension avant de la croiser

```
SELECT COUNT(DISTINCT categorie) AS libelles, COUNT(DISTINCT sous_categorie) AS sous_categories
FROM produit;
```

16 libellés pour **12** sous-catégories — et, après repliement des écritures, **7** familles réelles (M11.C05). La dimension est **sale** : croiser dessus sans la nettoyer produit des classements faux. Le nombre de valeurs distinctes est l'indicateur de propreté d'une dimension.

7.3 Étape 3 — vérifier l'additivité par la somme

```
SELECT ROUND(SUM(montant_ttc)) FROM ventes WHERE est_retour = 0;
```

15 595 154 955 FCFA — égal à la somme des cinq magasins. Le tableau croisé peut porter un total général : la mesure est additive.

7.4 Étape 4 — constater la non-additivité d'un comptage distinct

```
SELECT SUM(n) FROM (SELECT COUNT(DISTINCT id_client) AS n FROM ventes
                     WHERE est_retour = 0 GROUP BY id_produit) t;
```

188 886 — pour **23 497** clients réels. Un client achète en moyenne **8,04** produits : la somme des comptages le compte huit fois. **Aucun total ne sera publié sur cette colonne.**

7.5 Étape 5 — refuser la moyenne de moyennes

Moyenne des cinq paniers moyens : **107 840** FCFA. Vraie moyenne sur les tickets : **107 396** FCFA. Écart **445** FCFA (**0,4 %**) — faible, mais de **méthode** fausse : la règle s'applique indépendamment de la taille de l'écart observé.

7.6 Étape 6 — le contrôle qui rattrape la clé oubliée

```
-- avant la jointure
SELECT COUNT(*) FROM ventes;                -- 240 000
-- après la jointure sur le produit seul
SELECT COUNT(*) FROM ventes v JOIN rupture r ON r.id_produit = v.id_produit; -- 3 808 099
```

Le nombre de lignes a changé : la jointure a **multiplié**. Le contrôle avant/après est le réflexe minimum de toute jointure sur un socle qu'on ne connaît pas par cœur.

8. Erreurs fréquentes

1. **Ne pas écrire le grain.** Un rapport sans grain est un rapport qu'on ne peut pas auditer.
2. **Additionner des comptages distincts.** **188 886** au lieu de **23 497**.
3. **Moyenner des moyennes.** **107 840** au lieu de **107 396** — la méthode est fausse même quand l'écart est petit.
4. **Sommer un stock dans le temps.** **2 034 733** unités au lieu de **45 882** ($\times 44,30$).
5. **Sommer un taux.** Un taux de rupture par magasin ne s'additionne pas ; il se **recalcule** sur les effectifs.
6. **Joindre sans vérifier la clé complète.** $\times 15,8$ de chiffre d'affaires, sans erreur de moteur.
7. **Croiser une dimension sale.** **16** libellés pour **7** familles : le classement change.
8. **Confondre métrique et indicateur clé** : des dizaines de métriques, **3** ou **4** indicateurs clés, pas **41**.
9. **Publier un indicateur opérationnel au rythme stratégique.** Le record du 17/03/2026 n'a de sens que le jour même.
10. **Mélanger les horizons** dans un même écran : trois publics, trois fréquences, trois lectures.

9. Bonnes pratiques professionnelles

- **Écrire le grain en tête de chaque rapport** : « une ligne = un ticket », « une ligne = un couple magasin-mois ».
- **Prouver le grain par une clé**, jamais par une intuition : `COUNT(*) = COUNT(DISTINCT clé)`.
- **Marquer chaque mesure** dans la carte de définition (C04) : additive, semi-additive ou non additive — c'est ce qui décide si le tableau porte un total.
- **Contrôler le nombre de lignes avant et après chaque jointure.**
- **Recalculer les taux et les moyennes** sur les lignes, jamais sur des agrégats.

Conseil professionnel. Quand deux chiffres s'opposent en réunion, ne cherchez pas qui a tort : demandez **le grain** et **l'additivité**. Dans l'immense majorité des cas, les deux chiffres sont justes et les deux personnes parlent de deux choses différentes — **23 497** clients et **188 886** lignes de clientèle ne se contredisent pas, ils ne mesurent pas la même chose.

10. Exercice guidé

Énoncé. Le responsable marketing publie ce tableau et demande : « pourquoi notre base clients fond-elle ? ».

Produit	Clients
Ciment CPJ45 50 kg	4 812
Peinture acrylique 20 l	3 907
Tôle bac alu 6 m	3 651
Total affiché	12 370

Étape 1 — identifier le problème. Le total affiché est la somme d'une colonne de **clients distincts** : la mesure n'est pas additive. Additionner des comptages distincts compte plusieurs fois le même client.

Étape 2 — corriger. Recalculer le total sur l'ensemble : `SELECT COUNT(DISTINCT id_client)` donne **23 497** clients pour l'enseigne — bien **plus** que la somme affichée, puisque les trois produits ne couvrent qu'une partie du catalogue.

Étape 3 — expliquer. L'écart ne raconte aucune fuite : il raconte qu'un client achète **8,04** produits en moyenne, et que la somme des lignes dépasse donc largement le nombre de têtes.

Étape 4 — écrire la règle. Dans le rapport : « clients **uniques** par produit ; le total n'est pas la somme des lignes (un client peut acheter plusieurs produits) ». C'est cette phrase, et non le tableau, qui empêche la mauvaise conclusion.

11. Exercices autonomes

1. **Le grain des huit tables.** Reproduisez le tableau du §5.2 : pour chaque table, le nombre de lignes, la clé testée, et la conclusion. Combien de tables ont une clé unique ?
2. **Cardinalités.** Listez les **12** dimensions du §5.3 avec leur cardinalité, puis dites lesquelles sont croisables telles quelles dans un tableau lisible.
3. **Additivité.** Prenez cinq mesures du socle (CA, quantité, tickets, clients, stock) et remplissez pour chacune le test d'additivité sur trois dimensions (magasin, mois, produit).
4. **La moyenne de moyennes.** Calculez le panier moyen par canal, puis la moyenne de ces trois valeurs, puis la vraie moyenne : mesurez l'écart et expliquez sa cause.
5. **La clé oubliée.** Joignez `ventes` et `stock_mensuel` sur le produit seul, puis sur produit et mois ; comparez les deux comptages de lignes et les deux totaux.
6. **Les trois horizons.** Pour un même sujet (les ruptures), proposez un indicateur opérationnel, un tactique et un stratégique, avec leur fréquence et leur public.

12. Correction détaillée

Exercice 1. 8 tables sur 8 ont une clé unique (voir le tableau du §5.2) : `ventes` par `id_vente`, `commande` par `id_commande`, `encaissement` par `id_facture`, `stock_mensuel` par produit × mois, `rupture` par produit × magasin × mois, `logistique` par magasin × mois, `produit` par `id_produit`, `magasin` par `id_magasin`.

Exercice 2. Croisables telles quelles : `magasin` (5), `ville` (4), `quartier` (5), `canal` (3), `mode_paiement` (5), `mois` (44). À regrouper d'abord : `produit` (154, par famille), `sous_categorie` (12), `vendeur` (22). À ne jamais croiser en clair : `client` (23 497), `jour` (1 339).

Exercice 3. Additives : CA, quantité, effectifs de lignes. Non additives : clients distincts, tickets distincts (additifs ou non selon la dimension), panier moyen, taux. Semi-additive : le stock (additif dans l'espace, jamais dans le temps).

Exercice 4. Trois canaux (Magasin, Livraison, Téléphone) : moyenne de leurs paniers contre la vraie moyenne. L'écart est ici faible parce que les volumes sont proches ; il grandirait si un canal portait la majorité des tickets. C'est la **pondération** qui manque à la moyenne de moyennes.

Exercice 5. Sur le produit seul : **3,8** millions de lignes (fan-out), total multiplié. Sur produit et mois : les lignes ne sont conservées que lorsqu'un stock existe pour ce mois — le comptage change à nouveau, sans erreur. Les deux versions ont un sens différent, et seule la seconde répond à la question posée.

Exercice 6. Opérationnel : nombre de références en rupture aujourd’hui, fréquence quotidienne, public : chef de rayon. Tactique : taux de rupture du mois par famille, fréquence mensuelle, public : directeur commercial. Stratégique : **13 129** jours de rupture cumulés par an et coût **estimé (169 386 812 FCFA)**, fréquence annuelle, public : direction, pour arbitrer le budget de stock.

13. Mini-projet de chapitre

« **La fiche de vocabulaire d’un jeu de données** » (2 h). Choisissez un jeu de données (le vôtre, ou le socle M12) et produisez une page qui contient : (1) la table des grains, prouvée par une clé ; (2) la liste des dimensions avec leur cardinalité ; (3) la liste des mesures avec leur additivité ; (4) les **3** jointures possibles du jeu et le contrôle de lignes avant/après pour chacune ; (5) une phrase de règle par mesure non additive. Ce document est la moitié de la carte de définition du chapitre C04 : il en fournit les colonnes « source » et « granularité ».

14. Résumé du chapitre

Notion	Ce qu’il faut retenir
Fait dimension	/ l’événement daté contre l’axe stable ; 8 tables, 12 dimensions sur le socle
Grain	ce que représente une ligne — prouvé par une clé : 8 clés uniques sur 8 tables
Cardinalité	5 magasins, 154 produits, 16 libellés pour 7 familles, 23 497 clients
Additif	CA 15 595 154 955 FCFA : somme des lignes = total
Semi-additif	stock : 45 882 unités contre 2 034 733 si on somme les 44 mois
Non additif	clients : 188 886 (produit) contre 23 497 réels ; panier : 107 840 contre 107 396
Clé oubliée	x 15,8 de CA (246 567 448 121 FCFA) sans aucune erreur du moteur
Horizons	opérationnel (jour, 26 933 751 le 17/03/2026), tactique (mois, 533 572 353 en 2026-03), stratégique (année)

15. À retenir

À retenir. Le vocabulaire n’est pas un ornement de cours : c’est l’outil qui empêche deux personnes justes de se contredire. **Grain** et **additivité** règlent la majorité des désaccords de chiffres — et se prouvent chacun par une requête d’une ligne.

- **Le grain se prouve** : `COUNT(*) = COUNT(DISTINCT clé)` — **8** tables sur **8** sur ce socle.
- **L’additivité est une propriété du couple mesure-dimension**, pas du chiffre : compter des clients par produit ne s’additionne pas, compter des ruptures par magasin s’additionne.
- **Une dimension sale fait échouer un croisement juste** : **16** libellés pour **7** familles.
- **Toute jointure change le nombre de lignes ? C’est une décision**, pas un détail : écrivez-la.

16. Évaluation formative

1. Qu’est-ce qu’un fait ? Qu’est-ce qu’une dimension ? Donnez un exemple de chacun sur le socle.

2. Quelles sont les deux tables du socle dont le grain est une **combinaison** de colonnes ?
3. Comment **prouve**-t-on le grain d'une table, en une requête ?
4. Classez ces mesures : chiffre d'affaires, clients distincts, stock, panier moyen.
5. Que vaut la somme des clients par produit, et pourquoi ne publie-t-on pas ce total ?
6. Quel est l'écart entre la moyenne des paniers moyens et la vraie moyenne ? Est-il grave ?
7. Pourquoi le comptage des ruptures par magasin s'additionne-t-il, contrairement aux clients ?
8. Que se passe-t-il quand on joint ventes et rupture sur le produit seul ?
9. Citez un indicateur opérationnel, un tactique et un stratégique du socle, avec leur fréquence.
10. Combien de métriques, de KPI et d'indicateurs clés une entreprise doit-elle avoir, en ordre de grandeur ?

Corrigé : 1. Fait : une ligne de vente ; dimension : un magasin. 2. `stock_mensuel` (produit × mois) et `rupture` (produit × magasin × mois). 3. `COUNT(*) = COUNT(DISTINCT clé)`. 4. Additive, non additive, semi-additive, non additive. 5. **188 886** ; parce qu'un client achète **8,04** produits et serait compté plusieurs fois. 6. **445 FCFA (0,4 %)** : faible ici, mais la **méthode** est fausse — la moyenne de moyennes ne pondère pas. 7. Parce qu'une rupture appartient à **un seul** magasin : la dimension partitionne l'ensemble. 8. Les lignes sont multipliées (**3 808 099**, × **15,9**) et le chiffre d'affaires avec elles (× **15,8**), sans erreur de moteur. 9. Opérationnel : CA du jour (**26 933 751 FCFA** le 17/03/2026) ; tactique : CA du mois (**533 572 353 FCFA** en 2026-03) ; stratégique : CA annuel. 10. Des dizaines de métriques, une poignée de KPI, **3** ou **4** indicateurs clés — pas **41**.

Module M12.C04 — Concevoir un KPI : les 6 critères et la carte de définition

Outils : DuckDB 1.5.5 (exécuté), pandas (contrôle croisé, exécuté), Excel (exécuté). Power BI (cité, non exécuté — règle §1.5). Durée indicative : 5 h. Niveau : N3. Prérequis : M12.C01—C03.

L'idée du chapitre. Un indicateur n'existe pas parce qu'il s'affiche : il existe parce qu'il est **défini, sourcé, attribué, comparable, pilotable et contre-balancé**. Ces six critères tiennent sur une **carte de définition** de 7 champs. Ce chapitre construit la carte pièce par pièce, et la met à l'épreuve sur les chiffres du socle : le même « taux de retour » vaut **1,12 %** ou **1,91 %** selon la définition, et une des six formules possibles donne un nombre **négatif** — voilà pourquoi la moitié du travail d'un indicateur se fait **avant** la première requête.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **énumérer et appliquer les six critères** qui font d'une métrique un indicateur clé — sourcé, défini, attribué, comparable, pilotable, contre-balancé ;
2. **remplir une carte de définition** : formule, source, granularité, fréquence, responsable, seuil d'alerte, contre-KPI — les 7 champs, **70** cases pour les **10** indicateurs du projet ;
3. **écrire une formule sans ambiguïté**, en choisissant explicitement numérateur, dénominateur, base de calcul et exclusions — et en sachant ce qu'un autre choix aurait produit ;
4. **poser un seuil d'alerte qui discrimine**, en mesurant ce qu'il déclenche : à **1** mois de couverture, **1 650** lignes de stock sont en alerte ; à **0,8** mois, il n'en reste que **176** ;
5. **choisir un contre-KPI** pour chaque indicateur, et **mesurer** ce que le couple protège ;
6. **reconnaître les effets pervers** d'un indicateur : ce qu'un chiffre pousse à faire quand il devient l'objectif — et la mesure qui les rend visibles.

2. Pourquoi cette notion est importante

Le dossier du projet raté est un catalogue de ces six critères manquants, dans l'ordre :

Ce qui manquait	Ce que cela a produit, dans le dossier
sourcé	la marge était calculée sur le coût standard du référentiel, la comptabilité sur le dernier prix payé — deux coûts pour un produit
défini	« le CA affiché n'est pas celui de la compta : il manque les retours » — 2 809 lignes, et personne n'a tranché
attribué	41 indicateurs, aucun responsable : deux services ont gardé chacun leur chiffre
comparable	le panier moyen divisé par le nombre de tickets chez l'un, par le nombre de lignes chez l'autre
pilotable	un seuil de rupture affiché... et aucune action associée : la question « que fait-on si c'est rouge ? » est restée sans réponse
contre-balancé	rien n'empêchait de « réussir » un indicateur aux dépens d'un autre, sans que personne ne le voie

Le résultat est chiffré : **7 940 000** FCFA dépensés, dont **1 200 000** de maintenance, **11** semaines de vie, **14** destinataires, **4** ouvertures le premier jour — et l'abandon. Le mandat de reprise aujourd'hui tient en une phrase : « pas plus de **10** indicateurs, chacun avec sa définition écrite, son responsable, sa fréquence, son seuil et son contre-indicateur ».

À retenir. Dans un dispositif qui échoue, le problème n'est presque jamais le calcul : il est dans les **six critères** que personne n'a posés. Une requête juste sur un indicateur non défini produit un chiffre juste que personne ne peut utiliser.

3. Explication simple — le passeport du chiffre

Un indicateur sans carte de définition, c'est un médicament sans notice : la substance est peut-être bonne, mais personne ne sait à quelle dose, pour qui, ni ce qu'il faut faire si l'état empire.

La carte est son **passeport** : elle ne décrit pas le chiffre, elle décrit **les conditions dans lesquelles ce chiffre a le droit d'être publié**. Sept champs, et chacun répond à une question qui a déjà provoqué une dispute :

- **Formule** — comment le calcule-t-on, exactement ?
- **Source** — d'où viennent les données ?
- **Granularité** — à quel niveau est-il calculé ?
- **Fréquence** — à quel rythme est-il publié ?
- **Responsable** — qui répond de sa justesse ?
- **Seuil d'alerte** — à partir de quand est-ce un problème ?
- **Contre-KPI** — quel indicateur l'empêche de mentir ?

Un passeport ne se remplit pas une fois pour toutes : il se **signe**. Le responsable, nommé, engage sa lecture ; le seuil, chiffré, engage une action ; le contre-KPI, choisi, engage un arbitrage.

4. Vocabulaire essentiel

Définition. Un **indicateur clé** — **KPI**, Key Performance Indicators — est une métrique **choisie** pour suivre une performance qui compte, dotée d'une carte de définition complète et d'un responsable nommé. Une entreprise a des dizaines de métriques, une poignée d'indicateurs et **trois ou quatre** indicateurs clés.

Définition. Une **carte de définition** est la fiche d'identité d'un indicateur : formule, source, granularité, fréquence, responsable, seuil d'alerte, contre-KPI. Elle se publie **avec** l'indicateur, pas dans un classeur à part.

Définition. Un **seuil d'alerte** est la valeur à partir de laquelle l'indicateur déclenche une **action** écrite. Un seuil sans action associée est une décoration ; une action sans seuil ne se déclenche jamais.

Définition. Un **contre-KPI** — indicateur de contrepartie — est l'indicateur qui mesure ce qu'on **dégrade** en améliorant le premier. Il se publie à côté de lui, dans le même écran, avec le même responsable.

Définition. Un **effet pervers** est une conséquence non voulue d'un indicateur devenu objectif : les personnes pilotées par la mesure agissent sur ce qu'elle mesure — pas sur ce qu'elle devait mesurer. Le remède n'est pas de supprimer l'indicateur, c'est de lui adjoindre un contre-KPI.

5. Cours approfondi

5.1 Les six critères

#	Critère	La question posée	Le test de vérification
1	Sourcé	d'où vient la donnée ?	la source existe, elle est nommée, et elle se met à jour
2	Défini	que calcule-t-on, exactement ?	deux personnes appliquent la formule et trouvent le même nombre
3	Attribué	qui répond de ce chiffre ?	un nom, une fonction, et le droit de trancher les désaccords
4	Comparable	peut-on comparer dans le temps et entre unités ?	même formule partout, variation ni nulle ni frénétique
5	Pilotable	que fait-on quand il est rouge ?	un seuil chiffré et une action écrite
6	Contre-balancé	qu'est-ce qu'on dégrade en l'améliorant ?	un contre-KPI publié à côté

Les six critères ne sont pas une liste de bonnes intentions : chacun a un test. Et chaque test a échoué, quelque part dans le dossier du projet — c'est ce qui va suivre, critère par critère.

5.2 La carte de définition, champ par champ

Champ	Rôle	Exemple pour le taux de rupture
Formule	lever toute ambiguïté sur numérateur et dénominateur	couples produit-magasin-mois en rupture ÷ couples servis
Source	nommer la table et le référentiel	ruptures.csv et les ventes servies du mois
Granularité	dire ce que représente une ligne du résultat	produit × magasin × mois
Fréquence	dire quand il est juste	le 5 du mois suivant, données arrêtées au dernier jour
Responsable	nommer qui répond de la définition	direction des opérations
Seuil d'alerte	chiffrer ce qui déclenche	plus de 10 % des couples servis en rupture
Contre-KPI	publier ce qui se dégrade	rotation de stock (9,42 tours) et couverture (1,27 mois)

Les sept champs, multipliés par les **10** indicateurs du mandat, font **70** cases à remplir. Le dossier raté comptait **41** indicateurs et **zéro** case remplie. Le projet n'exige donc pas **plus** de documents que le prestataire précédent : il exige **4,1 fois moins** d'indicateurs, chacun documenté.

5.3 Critère 1 — sourcé : une source absente ne se remplace pas

Le socle de M12 a été construit exactement pour respecter ce critère. Les ventes de M01—M03 existaient déjà ; le module a ajouté **cinq** sources opérationnelles, et **pas une de plus** :

Source ajoutée	Lignes	Ce qu'elle rend possible
coût d'achat	154	la marge, donc 3 847 989 780 FCFA
stock mensuel	6 776	la rotation (9,42 tours) et la couverture (1,27 mois)
ruptures	2 428	le taux de rupture (7,29 %) et les 13 129 jours cumulés
commandes clients	9 000	le taux de service (81,0 % ou 78,2 %)

Source ajoutée	Lignes	Ce qu'elle rend possible
encaissements	9 000	le taux de recouvrement (74,0 %) et l'encours (1 202 550 590 FCFA)

La règle qui a présidé à cette liste tient en une phrase : **un indicateur réclame sa source, et une source absente ne se remplace pas par une approximation**. Sans coût d'achat, on ne fabrique pas une marge : on fabrique une opinion sur la marge. Le dossier du projet l'a fait — **deux** coûts pour un produit, et la question « où perdons-nous de la marge ? » jamais résolue.

Attention. Le tableur sait tout faire, y compris remplir un trou : une colonne de coûts manquante se « complète » en recopiant la ligne du dessus. L'approximation ne se voit pas dans le résultat — elle se voit dans la décision. Un indicateur dont la source n'existe pas doit être **refusé**, puis remplacé par un indicateur dont la source existe.

5.4 Critère 2 — défini sans ambiguïté : le cas du taux de retour

Voici la démonstration centrale du chapitre. Le socle porte **2 809** lignes de retour. Le mot « taux de retour » peut recevoir **six** définitions, toutes défendables :

Définition	Formule	Résultat mesuré
valeur	valeur retournée ÷ valeur vendue	1,12 %
quantité, signée	quantité retournée ÷ quantité vendue	-1,17 %
quantité, absolue	quantité retournée (valeur absolue) ÷ quantité vendue	1,16 %
lignes	lignes de retour ÷ lignes totales	1,17 %
tickets	tickets portant un retour ÷ tickets	1,91 %
lignes par magasin	lignes de retour du magasin ÷ lignes du magasin	dépend du magasin

Deux enseignements, et ils sont différents :

1. **L'écart est réel.** Du plus bas (**1,12 %**) au plus haut (**1,91 %**), il y a **0,79** point, soit un facteur **1,71**. Publier « le taux de retour est de 1 % » ou « de 2 % » peut donc être juste dans les deux cas — et diriger deux décisions opposées.
2. **Une formule fausse ne se voit pas toujours, et parfois si.** En quantités **signées**, le taux vaut **-1,17 %** : les retours sont enregistrés avec des quantités négatives, et la division d'une somme négative par une somme positive donne un nombre négatif. Un taux négatif ne se discute pas, il **crie** — c'est le même genre d'erreur que la rotation tombant de **9,42** à **0,78** quand on additionne des stocks (C03).

Le même phénomène frappe la **marge**, et là, il ne crie pas. La marge brute est de **3 847 989 780 FCFA**. Rapportée au chiffre d'affaires **hors taxes**, elle vaut **29,12 %** ; rapportée au chiffre d'affaires **toutes taxes comprises**, **24,67 %**. La cible du référentiel — **18 à 24 %** — est formulée hors taxes : sans cette précision, le même résultat serait « au-dessus de la cible » ou « conforme ». Et l'écart entre CA net et CA brut (**175 169 798 FCFA**, **1,12 %**) explique à lui seul la première dispute du dossier.

5.5 Critère 3 — attribué : un nom, pas un comité

Le dossier du projet compte **41** indicateurs, **7** onglets, **14** destinataires et **aucun** responsable. C'est la cause directe des deux questions de discordance : personne n'était là pour **trancher**. Un indicateur attribué se reconnaît à trois signes :

1. **Un nom** apparaît sur la carte — une fonction, pas un service.
2. Cette personne **arbitre** les désaccords de définition ; elle n'a pas besoin d'être d'accord avec la comptabilité, elle doit décider **quelle définition est publiée**.

3. Elle **répond** de la fraîcheur de la source : si la donnée arrive le 8 au lieu du 5, c'est elle qui prévient le lecteur.

Sur les **10** indicateurs du mandat, cela fait **10** responsabilités nommées. Le dossier en comptait zéro pour **41** indicateurs : ce n'est pas un manque de rigueur, c'est une **absence d'organisation**.

5.6 Critère 4 — comparable : ni immobile, ni frénétique

Un indicateur comparable se reconnaît à deux excès qu'il évite. Le socle en fournit un de chaque :

Indicateur	Comportement mesuré	Verdict
nombre de magasins actifs	5 , et 1 seule valeur distincte sur 44 mois	immobile : il n'informe pas
panier journalier	moyen : moyenne 65 655 FCFA, écart-type 89 162 FCFA, soit 135,8 %	frénétique : illisible au jour
CA mensuel	de 199 549 035 à 533 572 353 FCFA, médiane 344 872 808	comparable : la variation raconte la saisonnalité

La conclusion pratique est une règle de **fréquence** : le CA se lit au **mois**, pas au jour ; le nombre de magasins se lit **quand le réseau change**, pas tous les matins ; la rupture se lit au jour, parce qu'elle se répare au jour. Une carte de définition sans fréquence raisonnée produit des lecteurs qui regardent le mauvais chiffre au mauvais rythme.

5.7 Critère 5 — pilotable : le seuil et l'action

La question la plus importante du mandat — « que fait-on quand c'est rouge ? » — se traite en deux temps : **poser le seuil**, puis **écrire l'action**.

Poser le seuil se mesure. Sur la couverture de stock, quatre seuils produisent quatre réalités très différentes :

Seuil de couverture	Lignes de stock en alerte	Part des 6 776	Produits concernés
sous 0,8 mois	176	2,6 %	16
sous 1 mois	1 650	24,4 %	50
sous 1,2 mois	3 344	49,4 %	87
sous 1,5 mois	5 126	75,6 %	126

Un seuil qui déclenche sur **75,6 %** des lignes ne déclenche rien : il décrit le stockage normal du distributeur. Un seuil à **2,6 %** ne déclenche pas assez tôt. La carte retient **1** mois, qui met en alerte **50** produits sur **154** — assez pour agir, assez peu pour que la liste soit traitable.

Écrire l'action se mesure aussi. Le modèle du module est la balance âgée : **1 155** factures de plus de **90** jours, **1 091 339 030** FCFA, **90,8 %** de l'encours ouvert. L'action n'est pas « surveiller les impayés » : c'est **relancer ces 1 155 factures**, dans cet ordre, avec ce volume. Une action sans volume ni destinataire n'est pas une action, c'est un souhait.

5.8 Critère 6 — contre-balancé : les dix couples

Le mandat exige un contre-indicateur par KPI. Les **10** couples du module, chacun avec ses valeurs mesurées :

Indicateur	Ce qu'il pousse à faire	Son contre-KPI	Le couple, mesuré
CA net	vendre plus, vite	taux de retour	15 595 154 955 FCFA contre 1,17 % de retour

Indicateur	Ce qu'il pousse à faire		Son contre-KPI	Le couple, mesuré
Marge brute	monter les prix		volume vendu	recette unitaire × 1,174 et unités × 1,169 (2023 → 2025)
Taux de rupture	stocker plus		rotation	7,29 % contre 9,42 tours
Rotation	réduire le stock		taux de rupture	9,42 tours contre 7,29 %
Panier moyen	vendre des lots		marge brute	107 396 FCFA contre 29,12 %
Taux de retour	contrôler les retours		taux de service	1,17 % contre 81,0 %
Taux de service	promettre large		coût logistique unitaire	81,0 % contre 3 284 FCFA par colis
Taux de recouvrement	de relancer vite		CA net	74,0 % contre 15 595 154 955 FCFA
Coût unitaire	logistique	grouper les tournées	taux de service	3 284 FCFA contre 81,0 %
Part de marché interne	défendre magasin	son	CA du réseau	60,8 % / 39,2 % à Ouagadougou

Trois de ces couples méritent d'être regardés de près, parce que la mesure y tranche un débat :

1. **Marge contre volume.** La direction a augmenté les prix : la recette moyenne par unité vendue est passée de **8 714** à **10 226** FCFA entre 2023 et 2025, et les unités vendues ont **augmenté** aussi (× **1,169**), avec **4,3 %** de clients de plus. Le contre-KPI **autorise** donc la stratégie — c'est précisément son rôle : il ne l'interdit pas, il la **surveille** et l'aurait interdite si le volume avait chuté.
2. **Rotation contre rupture.** Couverture moyenne **1,27** mois, rotation **9,42** tours, rupture **7,29 %** : ces trois chiffres se publient ensemble, parce que le premier se gagne toujours sur le dos des deux autres.
3. **Part de marché interne.** À Ouagadougou, deux magasins se partagent le marché **interne** : **60,8 %** pour Ouaga 2000 (**5 312 205 459** FCFA) et **39,2 %** pour Gounghin (**3 424 978 244** FCFA). Tout gain de l'un est une perte de l'autre : le contre-KPI est le **CA du réseau**, sans quoi l'indicateur récompense un transfert interne — un magasin qui prend des clients à son voisin ne fait pas grandir l'enseigne.

5.9 Le catalogue des effets pervers

Un effet pervers n'est pas une erreur de mesure : c'est une **conséquence** de la mesure, quand elle devient un objectif. Les sept mécanismes, chacun avec un exemple mesurable dans ce module :

1. **Déplacer le problème.** Réduire la rupture en sur-stockant dégrade la rotation (**9,42** tours) et la trésorerie : **2 034 733** unités si l'on cumule les mois, contre **45 882** en stock réel.
2. **Étrangler le canal.** Améliorer le taux de recouvrement (**74,0 %**) en durcissant les conditions tarifaires réduit le CA — c'est pourquoi les deux se publient ensemble.
3. **Rétrécir le périmètre.** Un taux de service calculé sur les seules commandes livrées vaut **81,0 %** ; sur toutes les commandes, **78,2 %** — les **319** commandes annulées disparaissent du premier calcul.
4. **Choisir le dénominateur.** Le panier moyen vaut **107 396** FCFA par ticket et **65 749** FCFA par ligne, soit **38,8 %** de moins : un indicateur peut être « amélioré » en changeant d'unité de compte.
5. **Doper la mesure.** Un taux de retour de **1,17 %** s'améliore en décourageant les retours : le contre-KPI est le taux de service, parce qu'un retour est souvent une livraison qui a échoué.
6. **Geler la cible.** Une marge de **29,12 %** évaluée contre une cible de **18** à **24 %** sans dire si la base est hors taxes ou toute taxe comprise : la cible ne veut rien dire (C04, §5.4).

7. **Oublier le coût.** Le coût logistique unitaire est de **3 284 FCFA** par colis (**249 FCFA** au kilo, **50,5 %** de carburant) : toute promesse de service plus rapide doit se chiffrer contre lui.

Attention. Un effet pervers n'est pas une faute professionnelle : c'est la conséquence **prévisible** d'un indicateur devenu objectif, produite par des gens compétents qui font exactement ce qu'on leur a demandé. Chercher un coupable quand un chiffre est « amélioré » sans que la performance suive, c'est se tromper de cible : c'est la **carte** qu'il faut corriger.

La règle du catalogue : pour chaque indicateur, demandez-vous ce qu'une personne **astucieuse et pressée** ferait pour l'améliorer. La réponse est votre contre-KPI.

6. Exemple concret — la carte remplie du taux de rupture

Le mandat demande des cartes, pas des intentions. Voici une carte complète, telle qu'elle se publie :

Champ	Contenu
Indicateur	taux de rupture
Formule	couples produit-magasin-mois en rupture + couples produit-magasin-mois servis
Source	ruptures.csv (jours de rupture, unités perdues estimées) + ventes servies
Granularité	produit × magasin × mois
Fréquence	mensuelle, le 5 du mois suivant
Responsable	direction des opérations (arbitre les cas litigieux avec les achats)
Valeur actuelle	7,29 % — 2 428 couples sur 33 296 ; 13 129 jours cumulés
Seuil d'alerte	plus de 10 % des couples servis du mois, ou plus de 400 jours cumulés
Action si rouge	liste des 20 produits les plus touchés, revue avec les achats sous 5 jours
Contre-KPI	rotation (9,42 tours) et couverture (1,27 mois)
Coût estimé du phénomène	169 386 812 FCFA (estimation, à citer comme telle)

Ce que cette carte change, concrètement : la question « que fait-on quand c'est rouge ? » a désormais une réponse nominative et datée, et le chiffre de **169 386 812 FCFA** ne peut plus être cité sans son étiquette d'**estimation**.

Dans les faits. Dans cet atelier, chaque valeur des cartes est **recalculée à chaque exécution** par `tools/kpi_M12.py` : les six définitions du taux de retour, les quatre seuils de couverture, la dispersion de la marge par famille et les dix couples indicateur / contre-KPI sortent de la même exécution, imprimés en console avec leur définition au-dessus du chiffre. Une carte dont les valeurs ne se recalculent pas n'est pas une carte : c'est une capture d'écran.

7. Démonstration pas à pas — six étapes pour fabriquer un indicateur

7.1 Étape 1 — partir d'une question de décision, pas d'une donnée

« Le service est-il tenu ? » est une question de décision. « Taux de service » est un mot. La carte commence par la question : qui décide, quand, et de quoi ?

7.2 Étape 2 — choisir le régime et le niveau

D'après C02, « le service est-il tenu ? » est un indicateur **prescriptif** au niveau tactique : il déclenche une relance ou un arbitrage, une fois par mois.

7.3 Étape 3 — écrire la formule et mesurer les deux versions

Taux de service sur les commandes **livrées** : **81,0 % (8 681 commandes livrées)**. Sur **toutes** les commandes : **78,2 %** — les **319** commandes annulées sortent du dénominateur dans la première version. La carte retient **78,2 %**, la plus sévère, avec la mention explicite « annulées incluses ».

7.4 Étape 4 — vérifier la source et la granularité

Source : `commandes_clients.csv`, **9 000** commandes, une ligne par commande. Granularité de la publication : le mois et le magasin. Le retard médian (**5** jours) se publie à côté du taux : un taux tenu avec cinq jours de retard n'est pas un service tenu.

7.5 Étape 5 — poser le seuil et l'action

Seuil retenu : **78,2 %**, c'est-à-dire « ne pas descendre sous le niveau actuel ». Action si rouge : revue des **319** commandes annulées, par cause, avec le responsable logistique.

7.6 Étape 6 — choisir le contre-KPI et le publier

Contre-KPI : le coût logistique unitaire, **3 284** FCFA par colis. Un taux de service qui remonte par des livraisons urgentes se verra immédiatement dans ce coût — et c'est voulu.

Le test final : deux personnes différentes remplissent la carte séparément. Si les deux cartes diffèrent d'un seul chiffre, l'indicateur n'est pas prêt.

8. Erreurs fréquentes

1. **Publier un indicateur sans carte.** Le chiffre est juste, la définition manque : la discussion sera un rapport de force, pas un arbitrage.
2. **Croire que le calcul est le sujet.** Le dossier du projet savait calculer **41** indicateurs ; il n'en pilotait aucun.
3. **Utiliser une source absente.** Deux coûts pour un produit, et la marge devient une opinion.
4. **Laisser une formule ambiguë.** **1,12 %** ou **1,91 %** de retours, selon la définition : les deux sont justes, une seule sera publiée.
5. **Ne pas nommer de responsable.** **41** indicateurs, **0** nom, deux versions du CA qui cohabitent.
6. **Poser un seuil sans action.** La question « que fait-on si la rupture dépasse le seuil ? » n'aura pas de réponse — c'est exactement ce qui s'est passé.
7. **Choisir un seuil qui déclenche partout.** À **1,5** mois de couverture, **75,6 %** des lignes s'allument : personne ne regarde plus.

8. **Confondre effet pervers et erreur.** L'effet pervers est produit par des gens compétents qui font ce qu'on leur demande de mesurer.
9. **Oublier le contre-KPI sur les indicateurs conflictuels.** Part de marché interne, rotation, recouvrement : ce sont les trois où la triche est la plus rentable à court terme.
10. **Multiplier les indicateurs.** 41 indicateurs, 4 ouvertures le premier jour : la surcharge tue la lecture avant que les chiffres soient faux.

9. Bonnes pratiques professionnelles

- **Une carte par indicateur**, publiée avec lui, 7 champs remplis, aucune case vide.
- **Partir de la question de décision** : qui, quand, pour agir sur quoi.
- **Mesurer les variantes de la formule** avant de choisir : les deux versions du taux de service (81,0 % / 78,2 %) et les six du taux de retour (1,12 % à 1,91 %) se calculent en quelques minutes et évitent six mois de débat.
- **Poser le seuil en regardant ce qu'il déclenche** : 50 produits à 1 mois, 126 à 1,5 mois — le seuil est un choix, et il se mesure.
- **Associer une action écrite à chaque seuil**, avec un volume et un destinataire.
- **Publier le contre-KPI dans le même écran**, jamais dans une note de bas de page.

Conseil professionnel. Quand on vous demande un indicateur, la première réponse utile n'est pas un chiffre : c'est une carte à remplir. Un lecteur qui remplit les 7 champs découvre lui-même la question qu'il n'avait pas posée — et cette découverte vaut plus que les dix indicateurs demandés au départ.

10. Exercice guidé

Énoncé. La direction veut suivre « la part des clients non identifiés ». Rédigez la carte complète, avec les valeurs du socle.

Étape 1 — la source. Les ventes portent un identifiant client. Une partie des lignes utilise le client 0, hors référentiel (M11.C04) : la source existe, la définition du phénomène est « identifiant manquant », pas « client inconnu ».

Étape 2 — la mesure. Le client 0 représente 18,3 % du chiffre d'affaires, soit 2 852 612 447 FCFA. C'est le premier chiffre du dossier, et il est déjà une surprise : près d'un cinquième du chiffre d'affaires n'est rattaché à aucun client nommé.

Étape 3 — la formule ambiguë. Deux définitions : part des **lignes** sans identifiant, ou part du **chiffre d'affaires**. Les deux se défendent ; la seconde dit la perte de connaissance réelle, la première dit la qualité de la saisie au comptoir. La carte doit choisir, et écrire pourquoi.

Étape 4 — le seuil et l'action. Seuil : 15 % du chiffre d'affaires. Action si rouge : contrôle de saisie par magasin, avec la liste des tickets concernés ; l'indicateur est un indicateur de **qualité de la donnée**, pas de performance commerciale.

Étape 5 — le contre-KPI. Le nombre de clients identifiés n'augmente jamais aussi vite qu'en forçant la création de fiches doublons : le contre-KPI est donc le taux de doublons ou le nombre de fiches créées par vendeur, à surveiller. Sans lui, l'indicateur s'améliore en polluant le référentiel.

Étape 6 — la carte. Formule : CA des lignes à identifiant manquant ÷ CA total. Source : ventes et référentiel clients. Granularité : magasin × mois. Fréquence : mensuelle. Responsable : responsable qualité de la donnée. Seuil : **15 %**. Contre-KPI : doublons de fiches clients.

11. Exercices autonomes

1. **Six critères.** Reprenez les **10** indicateurs du mandat et dites lesquels échouent au critère « comparable » si l'on ne précise pas la base (hors taxes ou toute taxe comprise).
2. **Six définitions.** Recalculez le taux de retour de six façons sur le socle et expliquez, pour chacune, quelle décision elle sert.
3. **Seuil.** Choisissez un seuil pour le taux de recouvrement (**74,0 %** aujourd'hui) et mesurez ce qu'il déclenche sur les **9 000** factures.
4. **Contre-KPI.** Proposez le contre-KPI du panier moyen et vérifiez sur le socle qu'il produit une mesure utile.
5. **Effet pervers.** Une direction annonce : « la rotation doit passer de **9,42** à **12** tours ». Écrivez ce qui va se passer, en trois phrases, et l'indicateur qui le prouvera.
6. **Carte complète.** Rédigez la carte du coût logistique unitaire (**3 284** FCFA par colis) avec les **7** champs.

12. Correction détaillée

Exercice 1. La marge brute (**29,12 %** hors taxes contre **24,67 %** toute taxe comprise), le taux de service (**81,0 %** livrées contre **78,2 %** toutes commandes) et le panier moyen (**107 396** FCFA par ticket contre **65 749** par ligne) sont les trois sensibles à la base : leurs cartes doivent l'écrire.

Exercice 2. Valeur **1,12 %** (décision financière), quantité signée **-1,17 %** (formule inutilisable), quantité absolue **1,16 %** (décision logistique), lignes **1,17 %** (qualité de service), tickets **1,91 %** (expérience client), par magasin (pilotage local). Le choix se justifie par la décision, jamais par la facilité.

Exercice 3. Un seuil à **70 %** mettrait en alerte la quasi-totalité du portefeuille ; à **74,0 %**, il signale les mois en recul. Calcul : nombre de factures réglées à échéance ÷ **9 000** ; aujourd'hui **6 660** factures, **1 272** ouvertes, encours **1 202 550 590** FCFA.

Exercice 4. Le contre-KPI naturel du panier moyen est la **marge brute** : vendre des lots à prix cassé augmente le panier et dégrade la marge. Sur le socle, la marge de **29,12 %** et le panier de **107 396** FCFA se suivent mois par mois, ce qui rend l'arbitrage visible.

Exercice 5. Passer de **9,42** à **12** tours signifie réduire la couverture sous **1** mois (couverture actuelle **1,27**) ; mécaniquement, la part des produits sous **1** mois de couverture passerait de **50** à davantage, et le taux de rupture (**7,29 %**) monterait. L'indicateur qui le prouve est le taux de rupture par famille, publié chaque mois à côté de la rotation.

Exercice 6. Formule : coût logistique total ÷ colis expédiés. Source : `logistique_mensuelle.csv` (**264** lignes) et colis des magasins (**43 830** des **52 596** colis, le dépôt en facture **8 766** sans vendre). Granularité : magasin × mois. Fréquence : mensuelle. Responsable : responsable logistique. Seuil : **3 500** FCFA par colis. Contre-KPI : taux de service.

13. Mini-projet de chapitre

« **Deux cartes, dont une pour un indicateur qu'on n'aime pas** » (2 h). Choisissez deux indicateurs du mandat : un que vous jugez utile et un que vous jugez dangereux. Remplissez les 7 champs pour les deux, avec les valeurs mesurées du socle. Puis répondez en une page : le second indicateur est-il sauvable par un contre-KPI, ou faut-il le retirer ? Argumentez avec des chiffres, jamais avec des principes. Ce travail est la moitié du livrable P2 du projet.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Les six critères	sourcé, défini, attribué, comparable, pilotable, contre-balancé — chacun a un test
La carte	7 champs, 70 cases pour 10 KPI, contre 41 indicateurs sans carte
Défini sans ambiguïté	taux de retour de 1,12 % à 1,91 % ; marge de 29,12 % ou 24,67 % selon la base
Seuil	à 1 mois de couverture : 1 650 lignes (24,4 %) et 50 produits ; à 0,8 mois : 176 lignes
Action	1 155 factures de plus de 90 jours, 1 091 339 030 FCFA — une action a un volume
Contre-KPI	marge contre volume : recette unitaire × 1,174 , unités × 1,169
Effets pervers	déplacer, étrangler, rétrécir, choisir son dénominateur, doper, geler, oublier le coût

15. À retenir

À retenir. Un indicateur se juge à sa **carte**, pas à sa formule. Les six critères se vérifient en quelques minutes chacun, et chacun a déjà coûté cher à quelqu'un : **41** indicateurs sans responsable, un seuil sans action, une marge sans base, un service sans dénominateur.

- **Sourcé** : une source absente ne se remplace pas par une approximation — la marge a besoin du coût d'achat, pas d'une recopie.
- **Défini** : écrivez la formule, puis calculez **les autres versions possibles** : c'est le seul moyen de savoir ce que vous publiez.
- **Attribué** : un nom par indicateur ; **10** KPI, **10** responsables.
- **Pilotable** : un seuil qui discrimine (**50** produits, pas **126**) et une action avec volume et destinataire.
- **Contre-balancé** : demandez-vous ce qu'une personne pressée ferait pour améliorer le chiffre — puis mesurez-le.

16. Évaluation formative

1. Citez les six critères d'un indicateur utilisable.
2. Quels sont les 7 champs d'une carte de définition ?
3. Pourquoi une carte de définition se remplit-elle **avant** d'écrire la requête ?
4. Combien de définitions différentes du taux de retour le socle permet-il de mesurer, et quel est l'écart entre la plus basse et la plus haute ?
5. Pourquoi une des formules donne-t-elle un nombre négatif, et que révèle ce signe ?
6. La cible de marge est de **18** à **24 %** : quelle précision manque-t-il pour qu'elle ait un sens ?
7. Un seuil qui met en alerte **75,6 %** des lignes est-il un bon seuil ? Justifiez par la mesure.

8. Qu'est-ce qu'une action associée à un seuil, et qu'est-ce qui la distingue d'un souhait ?
9. Pourquoi la part de marché interne a-t-elle besoin d'un contre-KPI ? Lequel ?
10. Le taux de service vaut **81,0 %** ou **78,2 %** : dites ce qui change entre les deux, et ce que la carte doit écrire.

Corrigé : 1. Sourced, défini, attribué, comparable, pilotable, contre-balancé. 2. Formule, source, granularité, fréquence, responsable, seuil d'alerte, contre-KPI. 3. Parce que la formule découle de la décision, et non l'inverse : commencer par la requête revient à laisser la source choisir l'indicateur. 4. Six, de **1,12 %** à **1,91 %** — **0,79** point d'écart, un facteur **1,71**. 5. Parce que les retours portent des quantités **négatives** : le signe révèle que le numérateur et le dénominateur ne parlent pas de la même population. 6. La **base** : hors taxes (**29,12 %**) ou toute taxe comprise (**24,67 %**). 7. Non : un seuil qui s'allume sur les trois quarts des lignes décrit la normale et ne déclenche aucune action ; **1** mois (**24,4 %**) est retenu. 8. Une action a un **volume**, un **destinataire** et une **date** : les **1 155** factures de plus de **90** jours relancées sous quinzaine. 9. Parce que le gain d'un magasin est la perte d'un autre dans la même ville (**60,8 %** contre **39,2 %** à Ouagadougou) : le contre-KPI est le CA du **réseau**. 10. Le premier ne compte que les commandes **livrées** ; le second inclut les **319** commandes annulées : la carte publie **78,2 %**, la version la plus sévère.

Module M12.C05 — L'architecture BI de bout en bout

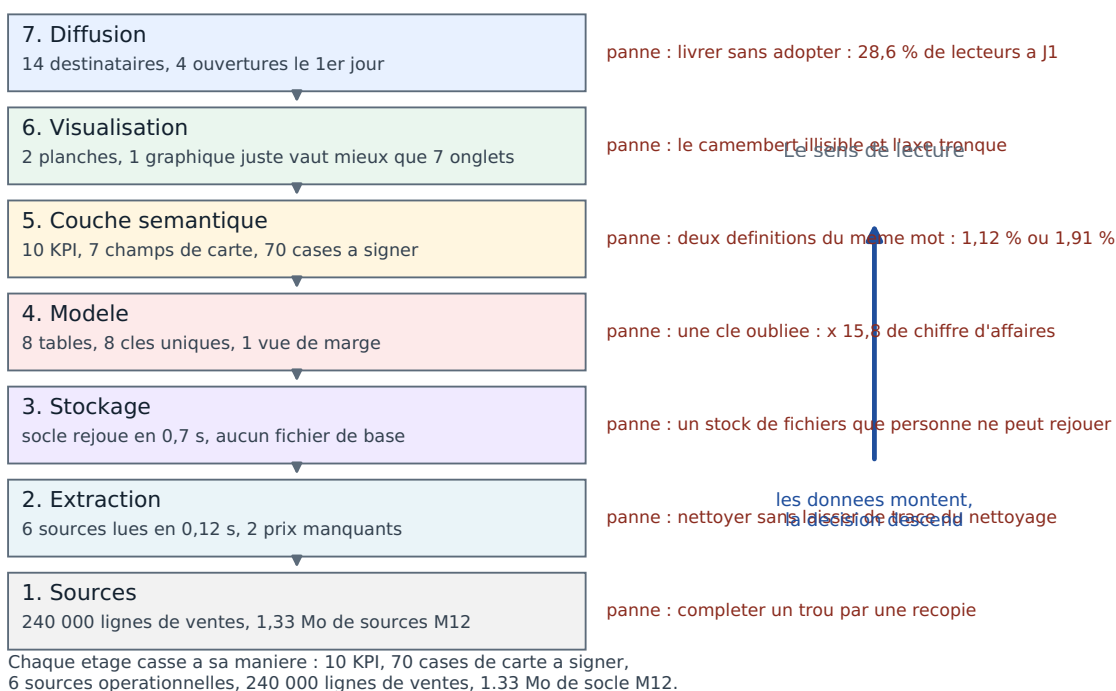
Outils : DuckDB 1.5.5 (exécuté), pandas (exécuté), Excel (exécuté). Power BI (cité, non exécuté — règle §1.5). Durée indicative : 5 h. Niveau : N3. Prérequis : M12.C01—C04, M09 (support), M11 (la matière).

L'idée du chapitre. Un indicateur ne sort pas de nulle part : il traverse **sept** étages, de la source jusqu'au lecteur. Chaque étage a son métier, sa panne typique et son coût. Ce chapitre les nomme, les chiffre sur le socle — **240 000** lignes lues en **0,12 s**, un socle rejoué en **0,70 s**, une couche sémantique de **10** indicateurs et **70** cases — et raconte où le dossier raté a payé chaque étage sans le construire : **22** jours sur l'écran, **8** sur les sources.

La chaîne BI en 7 étages : de la source à la décision

Les 7 étages, du bas vers le haut : ce que chacun apporte

La panne typique de l'étage



1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

- nommer les sept étages** d'une chaîne décisionnelle et dire ce que chacun apporte, dans l'ordre : source, extraction, stockage, modèle, couche sémantique, visualisation, diffusion ;
- reconnaître la panne typique** de chaque étage, et savoir à quel étage un chiffre faux a été fabriqué ;
- chiffrer une chaîne simple** : volume des sources, temps d'extraction, temps de recalcul complet, nombre d'objets du modèle ;
- situer la frontière** entre ce qui appartient au module M12 (nommer les étages) et ce qui appartient à M13 (construire le modèle) et M14 (outiller la publication) ;

5. **répartir un budget de projet** entre les étages — et constater que le dossier raté a fait l'inverse de ce que la mesure recommande ;
6. **détecter les trois pannes qui coûtent le plus** : sources non fiables, couche sémantique absente, diffusion sans adoption.

2. Pourquoi cette notion est importante

Recopions la répartition réelle des **30** jours de travail du dossier raté, en face de ce que la mesure recommande :

Poste	Le dossier raté	Ce que dit la mesure
Reprise et nettoyage des sources	8 jours	les sources portent 240 000 lignes, 2 prix manquants, 1 client sans fiche : c'est le socle de tout le reste
Conception et développement du tableau de bord	22 jours	41 indicateurs, 7 onglets, aucun responsable : de la production, pas de la définition
Formation	1 heure, 12 participants, 250 000 FCFA	14 destinataires, dont 4 ont ouvert le tableau de bord le premier jour
Maintenance annuelle	1 200 000 FCFA	payée 11 semaines, puis résiliée

Sur **7 940 000** FCFA dépensés, l'étage qui décide du sort de tous les autres — les **sources** — a reçu **8** jours sur **30**. Et l'étage qui n'existe que si quelqu'un l'ouvre — la **diffusion** — a reçu **1** heure de formation.

L'architecture n'est donc pas un sujet d'ingénieur : c'est le **plan de dépense** du projet. Nommer les sept étages sert à savoir où mettre les semaines — et le dossier raté montre à quoi ressemble une répartition qui ne tient pas.

À retenir. Un dispositif BI ne se casse presque jamais au même endroit que celui où il a été construit : **22** jours passés sur l'écran n'ont pas racheté **8** jours de sources, et **1** heure de formation n'a pas racheté **14** lecteurs.

3. Explication simple — sept étages, deux sens

Pensez à un immeuble de sept étages.

- **Au sous-sol, les sources** : les fichiers des caisses, des achats, du stock. Ils existent déjà, ils ne nous attendent pas, et ils ne sont pas propres.
- **Au rez-de-chaussée, l'extraction** : on fait entrer les données, on les nettoie, on laisse une trace de ce qu'on a nettoyé.
- **Au premier, le stockage** : un endroit stable où les données vivent, qu'on peut rejouer.
- **Au deuxième, le modèle** : on dispose les données en tables qui se répondent — des faits et des dimensions.
- **Au troisième, la couche sémantique** : on écrit ce que chaque indicateur veut dire, une fois pour toutes.
- **Au quatrième, la visualisation** : on dessine ce qui doit être lu en dix secondes.
- **Au cinquième, la diffusion** : on livre à des gens qui décident — et on vérifie qu'ils l'ouvrent.

Les données montent : elles entrent au sous-sol et grimpent étage par étage. **La décision descend** : un directeur lit au cinquième étage et agit sur le magasin du sous-sol.

Toute la difficulté est là — les deux flux ne se croisent jamais au même étage, et c’est pourtant la même chaîne.

4. Vocabulaire essentiel

Définition. Une **chaîne de calcul** — pipeline — est la suite d’étapes qui transforme les sources brutes en indicateurs publiés : extraction, nettoyage, stockage, modélisation, définition, visualisation, diffusion. Une chaîne de calcul se juge à sa capacité à être **rejouée** à l’identique.

Définition. Une **source de vérité** est l’endroit unique où une information est tenue à jour pour un usage donné. Sans source de vérité, deux services produisent deux chiffres — et les deux sont justes.

Définition. La **gouvernance des données** désigne les règles explicites qui décident qui crée, qui modifie, qui valide et qui publie une donnée ou un indicateur. Elle se traduit par des noms, des droits et des définitions écrites.

Définition. L’**adoption** est l’usage réel du dispositif par ses destinataires : nombre de lecteurs réguliers, fréquence d’ouverture, décisions citant le dispositif. Un tableau de bord livré et non ouvert n’a pas d’adoption, donc pas de valeur.

Définition. La **diffusion** est le dernier étage : le choix du support, de la fréquence et du moment où l’indicateur atteint son lecteur — courriel, réunion, écran partagé — avec la personne qui répond du rendez-vous.

5. Cours approfondi

5.1 Étage 1 — les sources

Ce que l’étage apporte : la matière. Rien d’autre ne peut le remplacer ; c’est le seul étage dont l’absence rend les six autres inutiles.

Panne typique : croire une source plus propre qu’elle n’est, ou combler un trou par une copie. Le socle du module en porte les marques, toutes mesurées : **2** prix manquants dans le référentiel produits, **1** dépôt qui ne vend rien, **1** client hors référentiel qui pèse **18,3 %** du chiffre d’affaires, **16** libellés de catégorie pour **7** familles réelles, un marqueur d’encodage en tête de trois fichiers.

Coût : c’est l’étage qu’on sous-estime. Le dossier raté y a consacré **8** jours sur **30**, au titre d’une « reprise et nettoyage des sources » facturée **1 640 000** FCFA — le poste le moins cher du devis, et celui qui a décidé de l’échec.

Attention. Un poste « sources » sous-doté ne se voit pas à la livraison : le tableau de bord s’affiche, les chiffres sont là, et c’est **plus tard** que le défaut remonte — une marge qui ne correspond pas, une catégorie en double, un client sans fiche qui pèse **18,3 %** du chiffre d’affaires. Les **8** jours sur **30** n’ont pas fait échouer le projet le jour de la livraison ; ils l’ont fait échouer **11** semaines plus tard, quand plus personne ne pouvait corriger l’étage 1.

5.2 Étage 2 — l’extraction

Ce que l’étage apporte : faire entrer les données et **laisser une trace** de ce qu’on leur a fait. Sur le socle, l’extraction des **6** fichiers opérationnels prend **0,12** s, et la lecture des **240 000**

lignes de ventes **0,10 s** : lire est rapide. Ce qui est lent, c'est de décider quoi faire des cas particuliers.

Panne typique : nettoyer **sans écrire la règle**. Trois exemples du socle, chacun avec sa règle explicite :

Cas rencontré	Règle retenue	Ce que la règle évite
colonne de date contenant des vides	lire en texte, puis convertir explicitement	l'arrêt brutal du chargement, ou une date inventée
deux prix manquants au référentiel	médiane de la sous-catégorie, et mention de l'effet (29,12 % contre 29,07 %)	une marge annoncée sans son approximation
marqueur d'encodage en tête de fichier	le déclarer à la lecture	une première colonne corrompue, donc une dimension fausse

Coût : quelques minutes dans l'atelier, des semaines ailleurs. Une règle d'extraction non écrite se paie à **chaque nouveau fichier**.

5.3 Étage 3 — le stockage

Ce que l'étage apporte : un endroit stable, rejouable, où les données vivent entre deux usages. Sur le socle, le choix est explicite : **aucun fichier de base versionné** (un fichier de base pèse au moins **512** kilo-octets, et se périme) ; à la place, un **script** de **4 375** octets qui reconstruit les **8** tables et la vue de marge à partir des sources, en **0,70 s**.

Panne typique : le stock mort. Un répertoire de fichiers extraits à la main, sans script ni date : personne ne peut le rejouer, ni savoir de quand il date. C'est le contraire d'un stockage.

Coût : près de zéro dans un atelier, très cher dans une entreprise. La règle opérationnelle : si vous ne pouvez pas reconstruire le stockage à partir des sources, vous ne stockez pas des données : vous stockez une opinion.

5.4 Étage 4 — le modèle

Ce que l'étage apporte : une forme qui rend les questions simples. Le socle compte **8** tables, **8** clés uniques — donc huit grains prouvés — et **une** vue supplémentaire pour la marge. Le contrôle de grain du C03 s'applique ici : chaque table répond à une question et une seule.

Panne typique : le **fan-out**, ou jointure à clé incomplète. Joindre les ventes aux ruptures sur le produit **seul**, en oubliant le mois, multiplie les lignes par **15,9** et le chiffre d'affaires par **15,8** — **246 567 448 121** FCFA au lieu de **15 595 154 955** — sans qu'aucune erreur ne s'affiche.

Panne voisine : mélanger les grains dans une même table, et devoir deviner, à chaque requête, ce que représente une ligne. C'est la maladie dont le contrôle de clé unique est le vaccin.

Coût : une requête fausse coûte une correction ; un modèle faux coûte une **campagne** de corrections. Sur le dossier raté, la discordance venait précisément de là : deux façons de relier les lignes, deux chiffres.

5.5 Étage 5 — la couche sémantique

Ce que l'étage apporte : la définition unique. C'est l'étage que M12 passe son temps à préparer : les **10** indicateurs du mandat, **7** champs par carte, **70** cases signées. La couche sémantique ne calcule pas les chiffres : elle décide **quel calcul est le bon** et comment il s'appelle.

Panne typique : l'absence pure et simple. Quand elle manque, chaque utilisateur refait la définition dans son outil, et l'on obtient ce que le socle permet de mesurer sans tricher : le même « taux de retour » vaut **1,12 %** ou **1,91 %** selon le numérateur, le même panier moyen **107 396** FCFA par ticket ou **65 749** par ligne, la même marge **29,12 %** hors taxes ou **24,67 %** toutes taxes comprises.

Coût : c'est l'étage le moins cher à construire et le plus cher à omettre. **70** cases de carte représentent quelques jours ; leur absence a coûté au dossier raté la question la plus importante posée par sa direction — « où perdons-nous de la marge ? » — restée sans réponse.

5.6 Étage 6 — la visualisation

Ce que l'étage apporte : rendre le chiffre lisible en dix secondes. Deux principes, tous deux mesurés ailleurs et vérifiés depuis :

1. **La forme de l'encodage décide de la lecture.** La hiérarchie des perceptions, établie par Cleveland et McGill en 1984, place la **position** et la **longueur** devant l'angle et la surface : les erreurs de lecture sont **40 à 250 %** plus grandes sur des longueurs comparées mal alignées que sur des positions communes, et le camembert est battu par des barres **37** fois sur **40**.
2. **Moins d'objets, plus de lecture.** Le dossier raté a livré **41** indicateurs sur **7** onglets ; l'atelier du module publie **2** planches pour **6** chapitres. La contrainte n'est pas esthétique : elle est imposée par le lecteur, qui ouvre le dispositif **4** fois le premier jour sur **14** destinataires.

Panne typique : l'axe tronqué, le camembert à douze parts, le tableau de 40 colonnes — trois façons de faire dire à un graphique juste une chose fausse.

5.7 Étage 7 — la diffusion

Ce que l'étage apporte : l'usage. Un indicateur atteint son lecteur à une heure, sur un support, avec une question à laquelle il répond. Le socle fournit la matière d'un calcul d'adoption : **14** destinataires, **4** ouvertures le premier jour (**28,6 %** des destinataires), une moyenne quotidienne qui tombe de **6** à **3** en une semaine — et l'abandon en **11** semaines.

Panne typique : confondre **livraison** et **diffusion**. Livrer, c'est envoyer une adresse ; diffuser, c'est obtenir une lecture. La différence se mesure, et elle se voit dans les chiffres ci-dessus.

Coût : la formation d'**1** heure, **12** participants, **250 000** FCFA — et une maintenance de **1 200 000** FCFA par an payée pour un dispositif que personne n'ouvrirait. Le coût de la non-adoption n'est pas la licence : c'est le projet entier.

Attention. Livrer un lien n'est pas diffuser un indicateur. Les **14** destinataires du dossier raté avaient tous reçu l'adresse, et **4** l'ont ouverte : la livraison était parfaite, la diffusion nulle. Un dispositif se juge au nombre de décisions qu'il déclenche, pas au nombre de personnes qui peuvent y accéder.

5.8 Les sept étages en une table

Étage	Ce qu'il apporte	Sa panne typique	Mesure du socle
1. Sources	la matière	croire la source propre, combler un trou	240 000 lignes, 2 prix manquants, 1 client hors référentiel (18,3 % du CA)
2. Extraction	l'entrée, avec trace des règles	nettoyer sans écrire la règle	6 fichiers lus en 0,12 s
3. Stockage	un socle rejouable	le stock mort, non rejouable	8 tables + 1 vue, script de 4 375 octets, rejoué en 0,70 s
4. Modèle	des grains nets	la clé oubliée : × 15,8 de CA	8 grains prouvés par 8 clés uniques
5. Couche sémantique	la définition unique	aucune : chacun sa définition	10 indicateurs, 70 cases, 6 définitions possibles du retour
6. Visualisation	la lecture en dix secondes	l'axe tronqué, le camembert	2 planches de moins de 776 pixels de large

Étage	Ce qu'il apporte	Sa panne typique	Mesure du socle
7. Diffusion	l'usage	livrer sans adopter	4 ouvertures sur 14 destinataires, abandon en 11 semaines

Frontière de module. C05 **nomme** les étages et leurs pannes ; M13 **construit** le modèle proprement — schéma en étoile, clés, historisation ; M14 **outille** l'étage 6 et une partie du 5 dans un outil de restitution ; M15 traitera la qualité et la gouvernance. Aucune capture d'outil ici : le module M12 reste au niveau des fondamentaux.

6. Exemple concret — la chaîne du module M12, étage par étage

Le socle de ce module est lui-même une chaîne de calcul, et elle est entièrement mesurable :

Étage	Ce qui existe dans l'atelier	Mesure
Sources	les ventes (26,8 Mo, 240 001 lignes avec l'en-tête) et 6 fichiers opérationnels	1,33 Mo de sources M12
Extraction	un générateur déterministe, graine 46	régénération complète en 0,70 s, écart 0
Stockage	<code>socle_m12.sql</code> (4 375 octets) : 8 tables, vues de ventes et de marge	aucun fichier de base
Modèle	8 clés uniques, la vue <code>vente_marge</code> pour la marge par famille	grain prouvé table par table
Couche sémantique	les 10 cartes du mandat, 7 champs chacune	255 clés de valeurs publiées dans le relevé du module
Visualisation	2 planches	largeurs 623 et 625 pixels
Diffusion	le classeur et les PDF du module	1 relevé recalculé en 13 s à chaque exécution

Deux enseignements de ce tableau. D'abord, **la chaîne entière tient dans 1,33 Mo et se rejoue en moins d'une seconde** : la simplicité n'est pas un luxe d'atelier, c'est ce qui permet de **vérifier** chaque chiffre publié. Ensuite, le point de contrôle n'est pas l'écran : c'est le **relevé** — **255** valeurs recalculées à chaque exécution, dont aucune n'est saisie à la main.

Dans les faits. Les sept étages de ce module sont **exécutés**, pas décrits : `tools/dossier_M12.py` reconstruit les **6** sources en **0,70** s et contrôle l'écart avec le socle publié, `socle_m12.sql` redéclare les **8** tables, `tools/kpi_M12.py` recalcule les **10** cartes, et `tools/figures_M12.py` produit la planche de ce chapitre. Rejouer la chaîne complète prend **13** secondes : c'est le prix de la vérifiabilité, et il est dérisoire.

7. Démonstration pas à pas — suivre un chiffre dans les sept étages

Prenons la marge du module — **3 847 989 780** FCFA, soit **29,12 %** du chiffre d'affaires hors taxes — et suivons-la de bas en haut, en notant à chaque étage ce qui pourrait la casser.

7.1 Étage 1 — la source

Le coût d'achat vient du référentiel produits, **154** lignes, dont **2** sans prix. **Panne** : recopier le prix de la ligne voisine, et fabriquer une marge plausible.

7.2 Étage 2 — l'extraction

Les lignes de vente sont rapprochées du coût par produit. **Panne** : oublier que le référentiel porte un prix unique alors que les prix de vente montent — d'où la règle d'extraction, publiée avec le chiffre.

7.3 Étage 3 — le stockage

Une vue `vente_marge` est construite une fois, et réutilisée par toutes les questions. **Panne** : recalculer la marge dans chaque rapport, avec chaque fois une variante de la formule.

7.4 Étage 4 — le modèle

Le rattachement se fait au grain **ligne de vente × produit**. **Panne** : joindre sur le produit seul avec une table qui contient plusieurs lignes par produit — le **× 15,8** du C03.

7.5 Étage 5 — la couche sémantique

La carte dit : « marge brute = ventes hors taxes moins coût standard × quantité ; base hors taxes ; exclusions : aucune ». **Panne** : publier **29,12 %** sans dire la base, quand la même marge vaut **24,67 %** toutes taxes comprises.

7.6 Étage 6 — la visualisation

La marge se lit par famille, en barres, sur **16** libellés qu'il faut d'abord replier en **7** familles réelles. **Panne** : tracer les **16** libellés côte à côte, et faire croire à quatre familles de peinture différentes.

7.7 Étage 7 — la diffusion

La marge part vers les acheteurs, avec le **contre-KPI** — le volume — et l'**action** si le seuil est franchi. **Panne** : publier la marge seule ; la première décision prise sera d'augmenter les prix sans regarder les volumes, ce que la mesure de 2023 à 2025 permet justement de surveiller.

Conclusion de la démonstration : le même chiffre était en danger **sept** fois. La différence entre un dispositif qui tient et un dispositif qui s'effondre n'est pas la justesse du calcul — elle est dans les règles écrites à chaque étage.

8. Erreurs fréquentes

1. **Ne voir que l'étage 6.** Le tableau de bord est la partie visible ; **22** jours sur **30** y ont été consacrés dans le dossier raté.
2. **Confondre source et extraction.** Une source se constate ; une extraction se **décide**, et sa règle s'écrit.
3. **Croire que le stockage est un répertoire.** Sans script, le répertoire n'est pas rejouable : ce n'est pas un stockage, c'est un historique.
4. **Laisser plusieurs grains dans une table.** Le contrôle de clé unique coûte une requête ; son absence coûte des mois.
5. **Faire l'économie de la couche sémantique.** Sans elle, **1,12 %** et **1,91 %** cohabitent.

6. **Multiplier les indicateurs pour couvrir les cas.** 41 indicateurs, 7 onglets, 4 lecteurs.
7. **Dessiner avant de savoir lire.** La position et la longueur se lisent mieux que l'angle et la surface, dans cet ordre.
8. **Livrer sans former.** 1 heure, 12 participants : la formation n'est pas un poste à rogner, c'est le seul étage qui produit de l'usage.
9. **Oublier la maintenance.** 1 200 000 FCFA par an : une chaîne sans entretien meurt de ses sources.
10. **Ne mesurer que la livraison.** On compte les indicateurs livrés, jamais les décisions prises ; les 4 ouvertures du premier jour disent pourtant tout.

9. Bonnes pratiques professionnelles

- **Parcourir les sept étages dans l'ordre** quand un chiffre est contesté : source, extraction, stockage, modèle, sémantique, visuel, diffusion. Le désaccord est presque toujours en amont du visuel.
- **Écrire une règle par cas particulier d'extraction** : date vide, prix manquant, catégorie en double.
- **Stocker en script**, pas en fichiers : le socle du module fait 4 375 octets et se rejoue en 0,70 s.
- **Prouver le grain de chaque table** avant de la publier.
- **Publier la définition avec le chiffre** : une carte de 7 champs par indicateur.
- **Chiffrer l'adoption** comme on chiffre la livraison : destinataires, ouvertures, questions posées.

Conseil professionnel. Devant un projet BI, la question la plus utile n'est pas « quel outil ? » mais « combien de jours par étage ? ». Le dossier raté a répondu en mettant 22 jours sur l'écran et 8 sur les sources ; aucun outil n'aurait sauvé cette répartition.

10. Exercice guidé

Énoncé. On vous demande de reprendre le dossier raté pour une PME de distribution. Répartissez 30 jours de travail sur les 7 étages et justifiez chaque poste par un risque mesurable.

Étage 1 — sources (8 jours). Le socle du module porte 2 prix manquants, 1 client hors référentiel (18,3 % du CA) et 16 libellés pour 7 familles : nettoyer et documenter ces trois points est un préalable à tout indicateur.

Étage 2 — extraction (4 jours). Une règle écrite par cas particulier, et un chargement reproductible (les 6 fichiers se lisent en 0,12 s, la lenteur est ailleurs).

Étage 3 — stockage (3 jours). Un script, pas un répertoire : reconstruire le socle de zéro.

Étage 4 — modèle (5 jours). 8 tables, 8 grains prouvés, une table de faits de ventes, une table de marge ; le contrôle avant/après jointure intégré.

Étage 5 — couche sémantique (6 jours). 10 cartes, 7 champs, 70 cases signées — c'est le poste le plus rentable du projet, et le seul qui supprime les débats de définition.

Étage 6 — visualisation (3 jours). Un écran, 10 indicateurs, des barres et des positions plutôt que des camemberts.

Étage 7 — diffusion (1 jour + récurrence). Une heure de formation **avant** la mise en ligne, puis un rendez-vous mensuel où le dispositif est lu **en séance**, et un compteur d'ouvertures.

Conclusion. L'écran passe de **22** jours à **3**, et les définitions de **0** à **6** : le total est le même, la répartition a changé — et avec elle, la probabilité d'usage.

11. Exercices autonomes

1. **Les sept étages.** Décrivez une chaîne décisionnelle que vous connaissez et nommez, pour chaque étage, ce qui existe et ce qui manque.
2. **Panorama des pannes.** Pour les **10** indicateurs du mandat, dites quel étage peut casser chacun d'eux, et par quel mécanisme.
3. **Stockage rejouable.** Chronométrez la reconstruction d'un jeu de données sur votre poste, et comparez avec le socle du module (**0,70 s** pour **8** tables).
4. **Sémantique.** Prenez deux indicateurs de votre entourage et écrivez leurs cartes : si vous n'arrivez pas à remplir les **7** champs, notez précisément ce qui vous manque.
5. **Visualisation.** Sur un graphique existant, remplacez l'encodage par une position ou une longueur, et décrivez ce que devient la lecture.
6. **Adoption.** Sur le dossier raté, calculez le taux d'ouverture (**4** sur **14**) et proposez deux mesures de diffusion qui l'auraient changé.

12. Correction détaillée

Exercice 1. Attendu : une description où chaque étage est **nommé** et où l'absence est dite sans détour — par exemple « la source existe mais personne ne documente la règle de nettoyage » (étage 2).

Exercice 2. La marge peut casser à l'étage 1 (coût absent), 5 (base non précisée : **29,12 %** ou **24,67 %**) et 4 (jointure multipliée par **15,8**). Le taux de service peut casser à l'étage 5 en choisissant les commandes **livrées (81,0 %)** plutôt que toutes les commandes (**78,2 %**). Le taux de retour peut casser à l'étage 5 avec **1,12 %**, **1,16 %**, **1,17 %** ou **1,91 %**.

Exercice 3. Attendu : un temps comparable en ordre de grandeur, et surtout la capacité de le **rejouer** deux fois avec le même résultat.

Exercice 4. Les champs les plus souvent impossibles à remplir sont le **responsable** et le **seuil** : ce sont précisément ceux qui distinguent un indicateur d'un chiffre affiché.

Exercice 5. La position commune (barres alignées) remplace l'angle (camembert) : les erreurs de lecture diminuent d'un facteur **40 à 250 %** selon l'étude de référence. Le classement lu change dans la majorité des cas (**37** fois sur **40** dans l'expérience d'origine).

Exercice 6. Taux d'ouverture : **4** sur **14**, soit **28,6 %**. Deux mesures de diffusion : envoyer le tableau de bord **avant** la réunion où il sera commenté, et remplacer l'accès libre par **une question** à laquelle le dispositif répond à date fixe.

13. Mini-projet de chapitre

« **La carte des sept étages de votre organisation** » (2 h). Reprenez la trame de l'atelier et remplissez-la pour un cas réel : sources disponibles et leurs défauts connus ; règles d'extraction écrites ; mode de stockage et sa rejouabilité ; tables et grains ; définitions existantes ; écrans et

leur forme ; destinataires et fréquence. Chaque case non remplie est un **risque** : classez-les par coût de correction, puis par coût d'absence. Ce document est la matière du livrable P3 du projet.

14. Résumé du chapitre

Étage	Ce qu'il apporte	Sa panne	Mesure
1. Sources	la matière	croire la source propre	2 prix manquants, 1 client à 18,3 % du CA
2. Extraction	l'entrée et ses règles	nettoyer sans écrire	0,12 s pour 6 fichiers
3. Stockage	un socle rejouable	le stock mort	0,70 s de reconstruction, 4 375 octets de script
4. Modèle	des grains nets	la clé oubliée	× 15,8 de CA sans erreur du moteur
5. Couche sémantique	la définition unique	l'absence	10 KPI , 70 cases, 6 définitions du retour
6. Visualisation	la lecture en dix secondes	l'axe tronqué	2 planches sous 776 pixels
7. Diffusion	l'usage	livrer sans adopter	4 ouvertures sur 14 , abandon en 11 semaines

15. À retenir

À retenir. Une chaîne BI se conçoit **de bas en haut** et se lit **de haut en bas**. L'écran est le dernier étage, pas le premier poste de dépense : dans le dossier raté, **22** jours sur **30** y ont été consacrés, et le dispositif est mort en **11** semaines.

- **Sept étages, sept pannes** : sources, extraction, stockage, modèle, couche sémantique, visualisation, diffusion — chacune a sa mesure dans ce chapitre.
- **Le stockage se rejoue** : **8** tables reconstruites en **0,70 s** par un script de **4 375** octets.
- **La couche sémantique est le meilleur rendement du projet** : **70** cases signées suppriment les débats de définition (**1,12 %** contre **1,91 %**).
- **La diffusion se mesure** : **4** ouvertures sur **14** destinataires, c'est un signal, pas un détail.

16. Évaluation formative

1. Citez les sept étages dans l'ordre, de la source à la diffusion.
2. Que se passe-t-il si l'étage 5 est absent, et comment le mesure-t-on sur le socle ?
3. Pourquoi dit-on que les données montent et que la décision descend ?
4. Quel étage a reçu **22** jours sur **30** dans le dossier raté, et qu'est-ce que cela a produit ?
5. Quelle est la panne typique de l'étage 4, et quel facteur de multiplication produit-elle ?
6. Pourquoi un répertoire de fichiers n'est-il pas un stockage ?
7. Que signifie « rejouable », et comment le vérifie-t-on sur le socle du module ?
8. Deux principes d'encodage visuel : lesquels, et dans quel ordre de performance ?
9. Qu'est-ce qui distingue la livraison de la diffusion, et comment la mesure-t-on ?
10. Le socle tient dans **1,33 Mo** et se reconstruit en **0,70 s** : en quoi est-ce un argument de fiabilité, et non de performance ?

Corrigé : 1. Sources, extraction, stockage, modèle, couche sémantique, visualisation, diffusion. 2. Chacun refait la définition : **1,12 %** ou **1,91 %** de taux de retour, **29,12 %** ou **24,67 %** de marge, **107 396** ou **65 749** FCFA de panier. 3. Parce que la donnée entre au premier étage et que l'action redescend du dernier : la corruption d'un étage bas ne se voit qu'en haut. 4. L'étage 6, la visualisation : **41** indicateurs, **7** onglets, et un abandon en **11** semaines. 5. La jointure à clé incomplète : × **15,9** de lignes, × **15,8** de chiffre d'affaires. 6. Parce qu'il n'est pas reconstruc-tible : sans script, personne ne sait ce qui a été fait aux lignes, ni quand. 7. Rejouable signifie que deux exécutions donnent le même résultat : le socle se régénère à l'identique (écart **0**) en **0,70** s. 8. Position, puis longueur, devant angle et surface ; l'écart d'erreur de lecture va de **40** à **250 %**. 9. La livraison envoie une adresse ; la diffusion obtient une lecture : **4** ouvertures sur **14** destinataires le premier jour. 10. Parce qu'un socle petit et rapide se **vérifie** : les **255** valeurs du module sont recalculées en **13** s, donc contrôlées à chaque exécution, au lieu d'être crues sur parole.

Module M12.C06 — Réussir le projet BI

Outils : DuckDB 1.5.5 (exécuté), pandas (exécuté), Excel (exécuté). Power BI (cité, non exécuté — règle §1.5). Durée indicative : 5 h. Niveau : N3. Prérequis : M12.C01—C05.

L'idée du chapitre. Un projet BI ne réussit pas parce que les chiffres sont justes : il réussit parce que quelqu'un **décide** avec lui. Ce chapitre referme le module en partant de l'échec — le dossier du projet raté — pour en tirer **10** causes vérifiables, une méthode de conduite, un cahier des charges utile et une recette de réception. Les chiffres qui l'accompagnent sont ceux du dossier : **7 940 000** FCFA dépensés pour **41** indicateurs, **4** lecteurs sur **14**, **11** semaines de vie — et, en face, les **10** cartes du mandat de reprise.

Indicateur	Ce qu'il pousse à faire	Son contre-KPI	Le couple, mesure
CA net	vendre plus, vite	taux de retour	15 595 154 955 FCFA / 1,17 % de retour
Marge brute	monter les prix	volume vendu	recette x 1,17 / unites x 1,17
Taux de rupture	stocker plus	rotation	7,29 % / 9,42 tours
Rotation	reduire le stock	taux de rupture	9,42 tours / 7,29 %
Panier moyen	vendre des lots	marge brute	107 396 FCFA / 29,12 %
Taux de retour	controler les retours	taux de service	1,17 % / 81,0 %
Taux de service	promettre large	cout logistique unitaire	78,2 % / 3 284 FCFA
Taux de recouvrement	relancer vite	CA net	74,0 % / 15 595 154 955 FCFA
Cout logistique unit.	grouper les tournées	taux de service	3 284 FCFA / 78,2 %
Part de marche interne	defendre son magasin	CA du reseau	60,8 % / 39,2 % a Ouagadougou

Un indicateur sans contre-KPI s'améliore toujours : il suffit de déplacer le problème chez le voisin. Les dix couples se publient ensemble.

M12 · 10 couples

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **citer les dix causes d'échec** d'un projet décisionnel et **reconnaître chacune** dans le dossier du projet raté, avec sa mesure ;
2. **distinguer les parties prenantes** : qui commande, qui lit, qui défend, qui maintient — et pourquoi l'absence du dernier rôle tue le dispositif ;
3. **rédiger un cahier des charges utile**, en posant les **6** questions que le prestataire du dossier n'a jamais posées, et en plafonnant le nombre d'indicateurs ;
4. **conduire un projet** par étapes mesurables : sources, définitions, écran, formation, recette, revue d'usage ;

5. **mesurer l'adoption** comme on mesure la livraison : destinataires, ouvertures, décisions, coût par lecteur (**1 985 000 FCFA** dans le dossier raté) ;
6. **organiser la maintenance et la gouvernance** : un budget par an (**1 200 000 FCFA**), un responsable par indicateur, une revue qui retire ce qui n'est plus lu.

2. Pourquoi cette notion est importante

Un projet décisionnel échoue rarement par erreur de calcul. Il échoue parce que personne ne s'en sert. Le dossier du projet raté en donne la décomposition exacte, et chaque ligne est un chiffre mesurable :

Ce qui a été livré	Ce qui s'est passé	Le chiffre
41 indicateurs sur 7 onglets	personne ne sait lesquels comptent	10 KPI demandés à la reprise
une commande floue, acceptée sans discussion	6 questions jamais posées	—
3 sources discordantes, 2 débats non tranchés	le CA a deux valeurs, la marge aussi	2 809 lignes de retour en cause
14 destinataires	4 ouvrent le premier jour	28,6 %
1 heure de formation, 12 participants	la moyenne d'ouvertures tombe de 6 à 3	en une semaine
1 200 000 FCFA de maintenance par an	contrat résilié	au bout de 11 semaines
22 jours sur l'écran, 8 sur les sources	la question de la direction reste sans réponse	« où perdons-nous de la marge ? »

Le mandat de reprise tient donc en deux exigences, et ce chapitre les traite : **au plus 10 indicateurs**, chacun avec sa carte — et **une réponse** à la seule question qui compte : que fait-on quand c'est rouge ?

À retenir. La réussite d'un dispositif décisionnel se mesure à l'extérieur de lui : au nombre de décisions qu'il déclenche. Un dispositif lu par **4** personnes sur **14** n'est pas un échec technique : c'est un échec de **projet** — commande, conduite, formation et maintenance.

3. Explication simple — la maison et ses habitants

Un projet BI ressemble à une maison construite pour un client qu'on n'a jamais questionné sur sa façon de vivre. On peut livrer de beaux murs, une plomberie impeccable : si la famille entre par la porte de service et fait la cuisine dans le salon, la maison est ratée — et ce n'est pas la faute du maçon.

Quatre questions séparent une maison ratée d'une maison habitée :

- **Qui va y vivre ?** — les destinataires, avec leur rôle et leur niveau de lecture.
- **Qu'y fera-t-on ?** — les décisions, pas les fonctions : commander, relancer, arbitrer.
- **Quand ?** — la fréquence réelle de ces décisions, jour, semaine, mois.
- **Qui s'en occupe ?** — celui qui répare la fuite, c'est-à-dire le responsable de la maintenance.

Le dossier du projet raté a répondu aux quatre à l'envers : des destinataires définis par une liste d'adresses, des indicateurs définis par ce qui était calculable, une fréquence quotidienne pour tout, et aucun responsable.

4. Vocabulaire essentiel

Définition. Un **cahier des charges** est l'écrit qui fixe, avant le début des travaux, les décisions à éclairer, les destinataires, la fréquence, les définitions attendues, les critères de réception et le budget de maintenance. Un cahier des charges sans critère de réception n'engage que le client.

Définition. Le **commanditaire** — sponsor — est la personne qui finance le dispositif, le défend dans l'organisation et arbitre les désaccords. Sans commanditaire identifié, le premier débat de définition fait tomber le projet.

Définition. Une **partie prenante** est toute personne qui influe sur le projet ou le subit : destinataires, producteurs de données, informaticiens, comptables, contrôleurs. Chacune a besoin d'un rôle écrit : qui décide, qui valide, qui consulte.

Définition. La **recette** — réception — est l'étape où le commanditaire vérifie le dispositif contre les critères du cahier des charges : chaque indicateur existe, sa valeur est reproductible, sa définition est publiée, son responsable est nommé, son seuil déclenche une action.

Définition. La **maintenance** couvre deux activités distinctes : la **maintenance corrective**, qui répare (**1 200 000** FCFA par an dans le dossier raté, soit **23 077** FCFA par semaine), et la **maintenance évolutive**, qui retire ce qui n'est plus lu et ajoute ce qui manque.

5. Cours approfondi

5.1 Les dix causes d'échec, toutes visibles dans le dossier

#	Cause d'échec	Le symptôme mesuré	Détectée avant la livraison par
1	Commande floue	« il doit contenir tout » : 41 indicateurs, 7 onglets	la question « quelles 10 décisions voulez-vous éclairer ? »
2	Aucun commanditaire actif	6 questions jamais posées, dont « qui décide avec ce chiffre ? »	le nom de la personne qui arbitre en réunion
3	Sources non fiables ou non documentées	3 sources discordantes, 2 débats non tranchés	la liste des cas particuliers par source, écrite
4	Définitions non écrites	deux CA, deux marges : 29,12 % ou 24,67 %	la carte de définition remplie avant l'écran
5	Trop d'indicateurs	41 indicateurs, 10 souhaitables	le plafond inscrit au cahier des charges
6	Aucun responsable par indicateur	personne ne tranche les 2 questions	10 noms sur 10 cartes
7	Aucun seuil relié à une action	« que fait-on si la rupture dépasse le seuil ? » resté sans réponse	la colonne « action si rouge » remplie
8	Adoption supposée	4 ouvertures sur 14 destinataires, puis 6 puis 3 par jour	la mesure d'usage à J+7 et J+30, et une formation avant la mise en ligne
9	Maintenance non financée ou non attribuée	1 200 000 FCFA/an, contrat résilié en 11 semaines	le responsable de maintenance nommé, avec son budget de 23 077 FCFA par semaine
10	Écran construit avant la couche sémantique	22 jours sur l'écran, 8 sur les sources	l'ordre d'exécution inscrit au planning

Ces dix causes ne sont pas indépendantes : les causes 1, 2 et 6 sont un même problème — le projet a été traité comme une commande de logiciel au lieu d'un projet de **décision**. Les autres en découlent.

Attention. Aucune de ces causes ne se voit à la livraison. Le jour de la mise en ligne, les **41** indicateurs s'affichent, les totaux tombent juste et le prestataire est payé : le défaut n'apparaîtra qu'à la première réunion où deux chiffres se contrediront, ou à la deuxième semaine où les ouvertures tomberont de **6** à **3**. Une recette qui se limite à « est-ce que ça s'affiche ? » ne détecte aucune des dix.

5.2 Les parties prenantes et leurs rôles

Quatre rôles, et le dossier raté permet de voir ce qui arrive quand l'un manque :

Rôle	Ce qu'il fait	Dans le dossier raté	Conséquence
Commanditaire	finance, défend, arbitre	absent au moment des 2 débats	chaque service garde son chiffre
Producteur de données	tient la source et ses règles	8 jours sur 30 , sans documentation publiée	les défauts remontent 11 semaines plus tard
Destinataire	décide l'indicateur avec	14 personnes, dont 4 lecteurs le premier jour	l'usage ne vient pas
Responsable maintenance	de corrige, complète retire,	non nommé, contrat résilié	le dispositif meurt de ses sources

La règle qui en découle tient en une ligne : **chaque rôle a un nom avant le premier jour de développement**, pas à la livraison.

5.3 Le cahier des charges utile, en six questions

Le dossier du projet raté liste lui-même les **6** questions que le prestataire n'a jamais posées. Elles forment le squelette d'un cahier des charges :

1. Qui décide avec ce chiffre, et **quand** ?
2. Que fera-t-on **différemment** si le chiffre est mauvais ?
3. Quelle est **déjà** la réponse des gens à cette question, et où la lisent-ils aujourd'hui ?
4. Qui est **responsable** de la définition de chaque indicateur ?
5. Combien de temps par jour le lecteur accepte-t-il d'y consacrer ?
6. Que se passe-t-il le **8** du mois, quand la donnée arrive en retard ?

À ces six questions, la reprise ajoute trois plafonds mesurables : **10** indicateurs au maximum, **7** champs par carte, et **une** action par seuil. Trois plafonds, et le projet devient vérifiable.

5.4 La conduite : l'ordre des étapes fait le projet

Le dossier raté a exécuté le projet dans l'ordre inverse de la chaîne des sept étages (C05) : écran d'abord, définitions jamais. La conduite recommandée inverse la dépense :

Étape	Livrable	Le dossier raté	La reprise
1. Cadrage	6 questions, liste des décisions, plafond d'indicateurs	absent	1 page signée
2. Sources	cas particuliers documentés, règles d'extraction	8 jours, non publiés	8 jours, publiés
3. Définitions	10 cartes, 70 cases	absent	6 jours
4. Calcul	socle rejouable, contrôle de grain	non rejouable	0,70 s de reconstruction
5. Écran	10 indicateurs, 1 page	22 jours, 7 onglets	3 jours
6. Formation et recette	usage mesuré à J+7 et J+30	1 heure, participants	12 1 heure avant la mise en ligne

Étape	Livrable	Le dossier raté	La reprise
7. Revue d'usage	retrait de ce qui n'est pas lu	jamais	trimestrielle, avec le commanditaire

5.5 L'adoption : la mesurer pour la piloter

L'adoption est une grandeur, pas une impression : le dossier du projet raté permet de la calculer entièrement.

Indicateur d'adoption	Valeur du dossier raté	Lecture
destinataires	14	la cible n'est pas « tout le monde », c'est ceux qui décident
lecteurs le premier jour	4 (28,6 %)	un dispositif qui démarre sous un tiers des destinataires ne démarre pas
moyenne quotidienne, semaine 1	6	l'intérêt initial existe
moyenne quotidienne, semaine 2	3	il divise par deux en une semaine : c'est le signal d'alarme
coût par lecteur du premier jour	1 985 000 FCFA	le prix réel d'un écran sans formation
durée de vie	11 semaines	pour 7 940 000 FCFA engagés

La seule mesure qui compte est la dernière colonne : ce que l'usage a coûté. Un dispositif peu coûteux et très lu est une réussite ; un dispositif cher et peu lu est un échec, quels que soient la qualité des chiffres.

Attention. Un indicateur d'adoption ne se mesure pas une fois, à la livraison, mais **deux fois** : à J+7 et à J+30. Le dossier raté aurait été sauvé par cette seule discipline — la chute de **6** à **3** ouvertures quotidiennes était visible dès la deuxième semaine, alors qu'il restait **9** semaines de maintenance à décider.

5.6 La maintenance et la gouvernance

Un dispositif décisionnel se dégrade : les sources changent, les rôles bougent, les questions évoluent. Trois règles suffisent à tenir :

1. **Un responsable par indicateur**, nommé sur la carte (C04). C'est lui qui signale la donnée en retard le **8** du mois.
2. **Un budget de maintenance annuel**, avec son ordre de grandeur : **1 200 000** FCFA par an dans le dossier raté, soit **23 077** FCFA par semaine — un montant modeste, et un contrat pourtant résilié faute d'usage.
3. **Une revue trimestrielle d'usage** : on regarde ce qui est lu, on **retire** ce qui ne l'est pas, on ajoute une question si elle revient trois fois en réunion. Un dispositif qui ne retire jamais rien finit à **41** indicateurs.

5.7 L'économie réelle d'un projet décisionnel

Ramener le budget au livrable et à l'usage change la lecture d'un projet :

Rapport	Valeur	Ce qu'il dit
budget ÷ indicateurs livrés	193 659 FCFA par indicateur	41 indicateurs à ce prix : la production n'était pas le problème
budget ÷ KPI du mandat	794 000 FCFA par KPI	le même budget, bien ciblé, finance 10 indicateurs complets

Rapport	Valeur	Ce qu'il dit
budget ÷ lecteurs du 1er jour	1 985 000 FCFA par lecteur	l'adoption est le vrai poste de dépense
maintenance semaine	par 23 077 FCFA	l'entretien coûtait moins cher qu'un jour de prestation
répartition de la dépense	61,1 % conception, 20,7 % sources, 3,1 % formation, 15,1 % maintenance	la formation, qui produit l'usage, est le poste le plus petit

Le dernier rapport est le plus instructif : **3,1 %** du budget pour la seule activité qui produit des lecteurs. Une reprise sérieuse commence par rééquilibrer cette ligne-là.

5.8 Détecter les causes d'échec avant la livraison

La recette se prépare **avant** la mise en ligne, avec une liste de vérifications qui tient sur une page :

1. chaque indicateur a sa **carte** complète (7 champs, pas **6**) ;
2. chaque carte a un **responsable** nommé, une fonction, pas un service ;
3. chaque seuil a une **action** écrite, avec volume et destinataire ;
4. chaque indicateur sensible a son **contre-KPI** publié à côté ;
5. le **grain** est prouvé pour chaque table du modèle ;
6. les **cas particuliers** d'extraction sont documentés ;
7. le dispositif se **rejoue** : deux exécutions donnent le même résultat ;
8. les **destinataires** sont nommés, avec leur décision et leur fréquence ;
9. la **formation** a eu lieu **avant** la mise en ligne ;
10. la **maintenance** a un nom, un budget et un rythme de revue.

C'est le même genre de contrôle que celui que le module s'applique à lui-même : l'atelier M12 vérifie à chaque exécution que ses **255** valeurs publiées se recalculent à l'identique, que ses **10** cartes tiennent, et que ses **2** planches restent déterministes.

Dans les faits. La recette de ce module est **automatique** : `01_socle_donnees/scripts/autovale.py` contrôle huit règles — gabarit des seize blocs, origine de chaque nombre cité, existence des figures et des chemins, présence du corrigé de chaque exercice, cohérence des barèmes, anglicismes, fréquence des encadrés — et le module M12 s'y présente avec **6** chapitres validés **sans avertissement**. Une recette qui ne se rejoue pas à chaque modification n'est pas une recette : c'est une signature.

6. Exemple concret — la reprise, chiffrée de bout en bout

Voici comment se présente la reprise du dispositif raté, budget inchangé :

Poste de la reprise	Jours	Ce qu'on y gagne
Sources : cas particuliers, règles, qualité	8	plus de défaut découvert en 11 semaines
Définitions : 10 cartes, 70 cases	6	les 2 débats de définition disparaissent
Calcul : socle rejouable, contrôle de grain	4	aucune mesure contestée sans rejou
Écran : 10 indicateurs, 1 page	3	41 indicateurs en moins, 7 onglets en moins
Formation et recette	2	14 destinataires formés avant l'ouverture
Revue d'usage trimestrielle	récurrent	un dispositif qui se retire au lieu de mourir

Poste de la reprise	Jours	Ce qu'on y gagne
Maintenance	récurrente	23 077 FCFA par semaine, avec un nom dessus

Le total de la reprise tient en 23 jours, contre 30 au dossier raté : c'est la répartition qui a changé. L'écran passe de **22** jours à **3**, et les définitions de zéro à **6**. Ce déplacement de quelques jours est la différence entre **41** indicateurs abandonnés et **10** indicateurs utilisés.

7. Démonstration pas à pas — six étapes de reprise

7.1 Étape 1 — diagnostiquer avec des chiffres

Reprendre les **41** indicateurs, les **7** onglets, les **14** destinataires et les **4** ouvertures, et en tirer un constat mesuré : **193 659** FCFA par indicateur livré, **1 985 000** FCFA par lecteur du premier jour, **11** semaines de vie. Un diagnostic sans chiffres est un avis.

7.2 Étape 2 — choisir 10 décisions

Partir des décisions, pas des données : commander, relancer, arbitrer les prix, surveiller les ruptures, décider d'un réassort. Les **10** indicateurs du mandat en découlent, et le **8** du mois reste la date de publication.

7.3 Étape 3 — remplir les 70 cases

Chaque indicateur reçoit sa carte : formule, source, granularité, fréquence, responsable, seuil, contre-KPI. C'est le livrable P2 du projet, et le seul document que la direction signera.

7.4 Étape 4 — construire un socle rejouable

Un script, pas un répertoire : **8** tables, **8** grains prouvés, une vue de marge, et un contrôle de lignes avant/après chaque jointure — la protection contre le **x 15,8**.

7.5 Étape 5 — un écran, une page, une action par seuil

Dix indicateurs, leurs contre-KPI, et la réponse à la question du mandat : que fait-on quand c'est rouge ? — avec un volume, un destinataire et une échéance, comme les **1 155** factures de plus de **90** jours.

7.6 Étape 6 — mesurer l'usage à J+7 et à J+30

Deux chiffres, relevés et publiés : combien de destinataires ont ouvert, combien de décisions citent le dispositif. Sans cette mesure, on retombe sur **6** puis **3** ouvertures par jour — et sur une résiliation au bout de **11** semaines.

8. Erreurs fréquentes

1. **Commencer par l'outil.** L'outil est l'étape 6 ; le projet commence aux étapes 1 et 5.
2. **Accepter une commande sans décisions.** « Tout » se traduit en **41** indicateurs, jamais en usage.
3. **Ne pas nommer de commanditaire.** Les **2** débats de définition n'ont pas été tranchés : personne n'en avait le pouvoir.
4. **Compter les indicateurs livrés comme un résultat.** **41** livrés, **4** lecteurs : le ratio dit tout.

5. **Former après la mise en ligne.** 1 heure de formation pour **14** destinataires, et **4** ouvertures le premier jour.
6. **Supposer que l'adoption suivra.** La moyenne quotidienne passe de **6** à **3** en une semaine : le signal était visible avant l'abandon.
7. **Oublier le responsable de maintenance.** Sans nom, la première source cassée tue le dispositif.
8. **Confondre maintenance corrective et évolutive.** La première répare (**23 077** FCFA par semaine) ; la seconde retire ce qui n'est plus lu.
9. **Ne jamais retirer d'indicateur.** Un dispositif qui n'oublie rien finit illisible.
10. **Croire qu'un échec est technique.** Celui-ci est un échec de projet : **3,1** % du budget pour la formation, **0** nom sur les définitions.

9. Bonnes pratiques professionnelles

- **Écrire le cahier des charges en questions de décision**, avec les **6** questions du §5.3 et trois plafonds : **10** indicateurs, **7** champs, **1** action par seuil.
- **Nommer les quatre rôles avant le premier jour** : commanditaire, producteur de données, destinataire, responsable de maintenance.
- **Faire signer les cartes** par les responsables, pas par les informaticiens.
- **Mesurer l'usage à J+7 et J+30**, et publier la mesure comme n'importe quel indicateur.
- **Recetter avant d'ouvrir** : les **10** vérifications du §5.8, sur une page.
- **Fixer la revue trimestrielle** : ce qui n'est pas lu se retire.

Conseil professionnel. Avant de lancer un dispositif, faites l'exercice inverse de celui du prestataire du dossier : demandez à chaque futur destinataire **quelle décision** il prendra avec quel indicateur, à quelle date, et avec quelle action si la valeur est mauvaise. Si trois personnes répondent « je ne sais pas », le projet n'est pas prêt — et il vaut mieux le savoir **30** jours avant qu'après **11** semaines.

10. Exercice guidé

Énoncé. Rédigez le cahier des charges de la reprise du dossier raté, en une page, avec des critères de réception vérifiables.

Étape 1 — les décisions. Lister les décisions à éclairer : réassort, relance des impayés, arbitrage des prix, contrôle des retours, arbitrage logistique. Chaque décision donne **1** ou **2** indicateurs : le total tient dans **10**.

Étape 2 — les destinataires. Nommer les personnes et leur décision, pas les services : **14** destinataires identifiés, dont **10** lecteurs attendus (contre **4** dans le dossier raté).

Étape 3 — les définitions. Fixer **10** cartes de **7** champs, avec les points de choix explicites : taux de retour sur lignes (**1,17** %) ou sur tickets (**1,91** %) ; taux de service sur commandes livrées (**81,0** %) ou toutes commandes (**78,2** %) ; marge hors taxes (**29,12** %) ou toutes taxes comprises (**24,67** %).

Étape 4 — les critères de réception. Dix vérifications du §5.8, dont trois mesurables : le socle se rejoue à l'identique en **0,70** s ; les **255** valeurs publiées se recalculent ; chaque seuil a son action.

Étape 5 — l’usage. Objectif d’adoption : **10** lecteurs sur **14** à J+30, mesuré et publié. Le dossier raté en comptait **4** à J+1.

Étape 6 — l’entretien. Un responsable nommé, **23 077** FCFA par semaine de maintenance, une revue trimestrielle qui retire les indicateurs non lus.

11. Exercices autonomes

1. **Dix causes.** Reprenez les **10** causes du §5.1 et dites, pour chacune, le signal **mesurable** qui l’aurait détectée avant la livraison.
2. **Rôles.** Attribuez les **4** rôles du §5.2 à des fonctions d’une entreprise que vous connaissez, et notez lequel manque aujourd’hui.
3. **Cahier des charges.** Écrivez les **6** questions et répondez-y pour un dispositif réel.
4. **Adoption.** Calculez le taux d’ouverture (**4** sur **14**) et proposez le protocole de mesure à J+7 et J+30.
5. **Économie.** Calculez le coût par indicateur (**193 659** FCFA) et le coût par lecteur (**1 985 000** FCFA) du dossier raté, puis refaites le calcul pour un dispositif de **10** indicateurs lu par **10** personnes.
6. **Recette.** Rédigez la page de recette du §5.8 en la transformant en liste cochable, avec la preuve attendue pour chaque ligne.

12. Correction détaillée

Exercice 1. Par exemple : cause 1 se détecte par le nombre de décisions listées (s’il est inférieur à **10**, la commande est floue) ; cause 8 par la mesure d’ouverture à J+7 ; cause 9 par l’existence d’un nom et d’un budget (**23 077** FCFA par semaine).

Exercice 2. Attendu : une attribution nominative, avec le constat des rôles absents — le plus souvent le **commanditaire** et le **responsable de maintenance**.

Exercice 3. Les réponses doivent tenir en une ligne chacune et désigner une personne, une date et une action : c’est le test de validité.

Exercice 4. **28,6 %** d’ouverture le premier jour. Protocole : comptage des ouvertures uniques par destinataire à J+7 et J+30, avec un seuil d’alerte à **60 %** et une relance individuelle en dessous.

Exercice 5. **41** indicateurs donnent **193 659** FCFA par indicateur et **1 985 000** FCFA par lecteur ; **10** indicateurs pour **10** lecteurs donnent, à budget constant, **794 000** FCFA par indicateur et **794 000** FCFA par lecteur — soit **2,5** fois moins par lecteur.

Exercice 6. Attendu : chaque ligne associée à une **preuve** (une capture ne compte pas : on attend un jeu, un contrôle de valeur, un nom, une date).

13. Mini-projet de chapitre

« **La note de reprise en une page** » (2 h). Rédigez la note qui accompagne le livrable P3 du projet : constat chiffré du dispositif en place (**41** indicateurs, **4** lecteurs, **11** semaines), les **10** indicateurs retenus, la répartition des **30** jours, les critères de réception, l’objectif d’adoption et le nom du responsable de maintenance. Une page, pas deux : c’est l’exercice de concision qui prépare le livrable du projet.

14. Résumé du chapitre

Notion	Ce qu'il faut retenir
Les 10 causes	commande floue, pas de commanditaire, sources non documentées, définitions non écrites, trop d'indicateurs, pas de responsable, pas d'action, adoption supposée, maintenance non financée, écran d'abord
Cahier des charges	6 questions, 3 plafonds : 10 indicateurs, 7 champs, 1 action par seuil
Rôles	commanditaire, producteur de données, destinataire, responsable de maintenance — nommés avant le premier jour
Adoption	4 lecteurs sur 14 (28,6 %), 6 puis 3 ouvertures par jour, 1 985 000 FCFA par lecteur
Économie	193 659 FCFA par indicateur livré, 794 000 par KPI bien ciblé, 3,1 % du budget pour la formation
Maintenance	1 200 000 FCFA par an, 23 077 par semaine, avec un nom dessus
Recette	10 vérifications sur une page, dont le rejeu du socle en 0,70 s

15. À retenir

À retenir. Un projet décisionnel réussit quand la dépense suit la chaîne : **sources, définitions, écran, formation, entretien**. Le dossier raté a fait l'inverse — **61,1 %** pour la conception, **3,1 %** pour la formation — et il a payé **1 985 000 FCFA** par lecteur du premier jour.

- **Les dix causes se détectent avant la livraison**, avec une page de recette et des chiffres.
- **L'adoption se mesure** : destinataires, ouvertures à J+7 et J+30, décisions citant le dispositif.
- **La maintenance a un nom et un budget** : sans eux, la première source cassée emporte tout.
- **Retirer est un acte de pilotage** : un dispositif qui n'oublie rien finit à **41** indicateurs.

16. Évaluation formative

1. Citez quatre des dix causes d'échec et leur symptôme mesuré.
2. Pourquoi la commande « il doit contenir tout » est-elle dangereuse, et par quel plafond la corrige-t-on ?
3. Quels sont les quatre rôles d'un projet décisionnel ?
4. Que mesure-t-on pour savoir si un dispositif est adopté ?
5. Comment calcule-t-on le coût par lecteur du dossier raté, et que vaut-il ?
6. Pourquoi la formation est-elle un poste de dépense rentable, alors qu'elle ne produit aucun indicateur ?
7. Qu'est-ce que la maintenance évolutive, et en quoi diffère-t-elle de la corrective ?
8. Citez trois des dix vérifications de recette.
9. Que signifie « le socle se rejoue » et comment le vérifie-t-on ?
10. Sur le mandat de reprise, pourquoi **10** indicateurs valent-ils mieux que **41** ?

Corrigé : 1. Par exemple : commande floue (**41** indicateurs) ; adoption supposée (**4** ouvertures sur **14**) ; pas de responsable (aucun nom sur les cartes) ; écran d'abord (**22** jours sur **30**). 2. Parce qu'elle transforme un projet de décision en production de rapports : le plafond de **10** indicateurs, avec **7** champs chacun, la rend vérifiable. 3. Commanditaire, producteur de données, destinataire, responsable de maintenance. 4. Les destinataires, les ouvertures à J+7 et J+30, et les décisions qui citent le dispositif. 5. **7 940 000 FCFA** divisés par **4** lecteurs du premier jour =

1 985 000 FCFA par lecteur. 6. Parce qu'elle produit des **lecteurs**, et que sans lecteur le dispositif n'a aucune valeur, quel que soit le nombre d'indicateurs livrés. 7. La corrective répare (**1 200 000** FCFA par an, **23 077** par semaine) ; l'évolutive retire ce qui n'est pas lu et ajoute les questions récurrentes. 8. Par exemple : carte complète (**7** champs), responsable nommé, seuil avec action écrite. 9. Deux exécutions produisent le même résultat : sur le socle, écarts nuls en **0,70** s de reconstruction. 10. Parce que l'usage décide : **10** indicateurs lus par **10** personnes valent mieux que **41** indicateurs lus par **4** — le budget est comparable, la dépense par lecteur ne l'est pas.

Projet M12.P — « Le tableau de bord que personne n'ouvrirait »

Module : M12 — Fondamentaux de la Business Intelligence · 4 livrables · 20 points · seuil de réussite 13 · environ 10 h de travail.

Le mandat. Une PME de distribution a payé **7 940 000 FCFA** pour un tableau de bord de **41** indicateurs répartis en **7** onglets. Onze semaines plus tard, le contrat de maintenance est résilié : **14** personnes avaient reçu l'adresse, **4** l'ont ouverte le premier jour. La direction vous mandate pour **diagnostiquer** ce dispositif et **proposer une reprise**. Sa consigne, textuelle : « pas plus de **10** indicateurs, chacun avec sa définition écrite, son responsable, sa fréquence, son seuil et son contre-indicateur. Et une réponse à la seule question qui compte : que fait-on quand il passe au rouge ? »

1. Énoncé

Le matériel fourni

Tout est dans `03_exercices/dossier_M12/` :

Pièce	Contenu
<code>etude_avant.md</code>	les 12 pages du dossier : commande, devis, liste des 41 indicateurs, démarrage, sources de la discorde, coût de l'abandon, questions jamais posées, mandat de reprise
<code>socle_m12.sql</code>	le socle : 8 tables (ventes, commande, encaissement, stock mensuel, ruptures, logistique, produit, magasin) et la vue de marge
<code>connexion.py</code>	l'ouverture de la base, identique à celle des chapitres
<code>ATTENDU.json</code>	les mesures de référence du socle, pour vérifier vos calculs
6 fichiers <code>.csv</code>	les cinq sources opérationnelles ajoutées par M12 : coût d'achat, stock, ruptures, commandes, encaissements, logistique

Règle du mandat : un diagnostic qui ne mesure rien est un avis. Chaque affirmation de votre dossier doit porter un chiffre, et chaque chiffre doit se recalculer avec les fichiers fournis.

Les six totaux de contrôle

Avant de commencer, vérifiez que votre socle répond :

Contrôle	Valeur attendue
lignes de vente	240 000 dont 237 191 hors retours
chiffre d'affaires net	15 595 154 955 FCFA (brut : 15 419 985 157)
clients	23 497 pour 145 212 tickets
marge brute	3 847 989 780 FCFA, soit 29,12 % hors taxes
ruptures	2 428 couples produit-magasin-mois, 13 129 jours
encours	1 202 550 590 FCFA sur 1 272 factures ouvertes

Si l'un de ces six contrôles ne tombe pas juste, ne poursuivez pas : reprenez le socle (\$7).

2. Les dix indicateurs imposés (le catalogue du mandat)

La reprise doit couvrir ces **10** indicateurs, ni plus ni moins :

1. chiffre d'affaires net
2. marge brute
3. taux de rupture
4. rotation de stock
5. panier moyen
6. taux de retour
7. taux de service
8. taux de recouvrement
9. coût logistique unitaire
10. part de marché interne

Chacun doit être livré avec sa **carte de définition** complète : **7** champs, soit **70** cases à remplir. Une case vide compte comme une case fausse.

3. Les 4 livrables

P1 — Le diagnostic mesuré (6 points)

Format : 3 pages maximum, plus un tableau de mesures. Le diagnostic répond, chiffres en main, à six questions :

1. **Combien a coûté le dispositif ?** Ventilez les **7 940 000** FCFA par poste et calculez le coût par indicateur livré, le coût par KPI du mandat et le coût par lecteur du premier jour.
2. **Pourquoi les chiffres se contredisaient-ils ?** Reprenez les **3** sources de la discorde — CA net contre CA brut, deux coûts d'achat, deux dénominateurs du panier moyen — et donnez, pour chacune, les deux valeurs mesurées sur le socle.
3. **Que valait la couche sémantique ?** Montrez, sur un indicateur au choix, que le socle permet plusieurs définitions défendables, et mesurez-les.
4. **Pourquoi les gens ont-ils cessé de l'ouvrir ?** Mesurez l'adoption : destinataires, ouvertures, évolution hebdomadaire, et la chute de la deuxième semaine.
5. **Quels étages de la chaîne ont été construits, et lesquels ont été sautés ?** Situez les **7** étages et donnez la mesure du socle pour chacun.
6. **Quelles causes d'échec retrouvez-vous ?** Citez-en au moins cinq parmi les **10** du cours, chacune avec son symptôme mesuré.

Ce qui fait la note : chaque affirmation porte un chiffre recalculable ; aucune cause n'est affirmée sans mesure ; le coût par lecteur est calculé et interprété.

P2 — La carte de définition de 10 KPI (8 points)

Format : 10 fiches. C'est le livrable central du module, et le document que la direction signera. Pour chacun des **10** indicateurs du §2, remplissez les **7** champs :

Champ	Exigence
Formule	numérateur, dénominateur, base de calcul, exclusions — sans ambiguïté
Source	la table et le fichier exacts, avec les cas particuliers connus
Granularité	ce que représente une ligne du résultat
Fréquence	quand l'indicateur est juste, et à quelle date il est arrêté

Champ	Exigence
Responsable	une fonction nommée, pas un service
Seuil d'alerte	une valeur, et la mesure de ce qu'elle déclenche
Contre-KPI	l'indicateur publié à côté, avec sa valeur actuelle

Deux exigences de fond :

- **chaque valeur actuelle est mesurée sur le socle** (tableau récapitulatif de vos **10** valeurs, avec la requête ou le script qui les produit) ;
- **au moins trois cartes justifient un choix de définition** en comparant deux versions mesurées — par exemple CA net (**15 595 154 955** FCFA) contre CA brut (**15 419 985 157**), ou taux de service sur commandes livrées (**81,0 %**) contre toutes commandes (**78,2 %**).

Ce qui fait la note : les 7 champs sont remplis pour les 10 indicateurs ; les valeurs sont mesurées et reproductibles ; les choix de définition sont argumentés par des chiffres ; un contre-KPI accompagne chaque indicateur et sa valeur est donnée.

P3 — Le plan de reprise (4 points)

Format : 2 pages, dont un tableau de charges. Reprenez le budget du dispositif raté et proposez une répartition des **30** jours entre les étages de la chaîne (sources, extraction, stockage, modèle, sémantique, écran, diffusion), en justifiant chaque poste par un risque mesuré. Le plan doit préciser :

- **ce qu'on arrête** : au moins trois pratiques du dossier (par exemple la publication d'indicateurs sans responsable, la définition locale du CA, l'onglet « Écarts » sans cible) ;
- **ce qu'on garde** malgré l'échec : au moins une chose utile du dispositif précédent ;
- **le calendrier** : nombre de semaines, jalons, et la date de la première revue d'usage ;
- **les destinataires** : qui reçoit quoi, à quelle fréquence, et l'objectif d'adoption chiffré à J+30.

Ce qui fait la note : la répartition est argumentée ; les charges sont cohérentes avec le total ; l'objectif d'adoption est chiffré et mesurable.

P4 — La page « que fait-on quand c'est rouge ? » (2 points)

Format : 1 page. C'est la réponse à la seule question laissée sans réponse par le dispositif précédent. Pour les **10** indicateurs, une ligne par indicateur, avec quatre colonnes : le seuil, l'action exacte, la personne qui la déclenche, le délai. Les actions doivent avoir un **volume** : une liste de produits, un nombre de factures, un montant — jamais « surveiller » ou « relancer si nécessaire ».

Ce qui fait la note : les dix seuils ont une action ; les actions ont un volume et un destinataire ; la page tient en une page.

4. Barème

Livrable	Points	Seuil de détail
P1 Diagnostic mesuré	6	1 point par question, si la réponse est chiffrée et recalculable
P2 Carte de 10 KPI	8	0,8 point par indicateur, dont 0,5 pour la complétude des 7 champs et 0,3 pour la cohérence des valeurs
P3 Plan de reprise	4	1 point par exigence : arrêts, conservation, calendrier, adoption
P4 Page « c'est rouge »	2	0,2 point par indicateur, si le seuil et l'action sont complets

Livrable	Points	Seuil de détail
Total	20	seuil de réussite : 13

Ce que le barème sanctionne : un chiffre repris du dossier sans être recalculé (**-1** par chiffre) ; une carte de définition incomplète (**-0,3** par champ vide) ; une action sans volume (**-0,2**) ; un contre-KPI absent ou décoratif (**-0,3**).

5. Ce qui fait rejeter un livrable

1. **Un diagnostic sans mesure** : les affirmations du dossier ne portent pas de chiffre vérifiable.
2. **Des cartes recopiées des chapitres** : les valeurs doivent être **mesurées** sur le socle, pas reprises du cours.
3. **Des définitions ambiguës** : « taux de retour » sans préciser numérateur ni dénominateur.
4. **Un seul indicateur sans responsable** : **10** indicateurs, **10** fonctions nommées.
5. **Des seuils sans action** ou des actions sans volume.
6. **Une page P4 qui dépasse une page** : l'exercice porte sur la concision autant que sur le fond.
7. **Un contre-KPI décoratif** : il doit être mesuré sur le socle, avec sa valeur actuelle.
8. **Plus de 10 indicateurs** : le mandat est un plafond, pas une suggestion.

6. Déroulé conseillé (4 séances, environ 10 h)

Séance	Durée	Travail	Livrable visé
1	2 h	lecture du dossier, contrôle des six totaux, premiers calculs du diagnostic	P1 (questions 1 à 3)
2	3 h	rédaction des cartes de définition, mesure des valeurs actuelles, choix de définition argumentés	P2 (indicateurs 1 à 6)
3	3 h	fin des cartes, seuils éprouvés par la mesure, contre-KPI mesurés	P2 (indicateurs 7 à 10)
4	2 h	plan de reprise, répartition des jours, page « c'est rouge », relecture	P3 et P4

7. Comment vous corriger vous-même

1. **Les six totaux** du §1 tombent juste, et votre socle se reconstruit à l'identique deux fois de suite.
2. **Chaque chiffre du diagnostic** se recalcule : aucun n'est recopié du dossier.
3. **Chaque carte a 7 champs** : comptez-les ; s'il en manque un, la carte n'est pas finie.
4. **Chaque seuil a été essayé** : vous savez dire combien de lignes il met en alerte.
5. **Chaque contre-KPI a une valeur mesurée** et une phrase expliquant ce qu'il protège.
6. **La page P4 tient en une page** et chaque action porte un volume, une personne et un délai.
7. **L'objectif d'adoption est chiffré** et comparable aux **4** lecteurs sur **14** du dispositif précédent.

Le test final. Faites lire votre P2 par quelqu'un qui ne connaît pas le dossier : s'il peut recalculer un indicateur et dire qui l'améliore ou le dégrade, la carte est bonne. S'il pose une question sur une définition, elle est incomplète.

Évaluation M12 — « Fondamentaux de la Business Intelligence »

6 chapitres · 30 h · niveau 3 · 70 points au total · durée conseillée : 2 h 30. Barème d'ensemble : quiz 15 questions (20 points, seuil 11 sur 15) ; deux exercices de rédaction de KPI (20 points) ; étude de cas (30 points, seuil 18) — soit 70 points, seuil de réussite 45.

Ce que l'évaluation vérifie. Que vous savez **choisir** un indicateur, **écrire** sa définition, **poser** son seuil, **trouver** son contre-KPI et **dire** ce qui se passe quand il passe au rouge. Les calculs sont courts ; les définitions et les décisions sont l'essentiel.

A · Questions de récupération (non notées)

1. Quels sont les **4** régimes de question, et lequel déclenche une action ?
2. Que signifie le **grain** d'une table, et comment le prouve-t-on ?
3. Quels sont les **7** champs d'une carte de définition ?
4. Dans quel ordre montent les **7** étages de la chaîne BI ?
5. Qu'est-ce qu'un contre-KPI ?

B · Quiz (15 questions · 20 points)

Les questions 1 à 10 valent 1 point ; les questions 11 à 15 valent 2 points (elles demandent un raisonnement). Seuil : 11 points sur 15 questions traitées.

Bloc 1 — Définir (Q1 à Q5)

- Q1.** Un indicateur clé se distingue d'une métrique par : (a) sa précision, (b) sa carte de définition et son responsable, (c) sa fréquence de calcul, (d) son coût de production.
- Q2.** Le même « taux de retour » peut valoir, sur le même socle : **1,12 %**, **1,16 %**, **1,17 %**, **1,91 %**. Ces écarts viennent : (a) d'erreurs de calcul, (b) d'un choix de numérateur et de dénominateur, (c) de la qualité des sources, (d) de la période observée.
- Q3.** La marge brute du socle vaut **29,12 %** : ce chiffre est rapporté : (a) au chiffre d'affaires toutes taxes comprises, (b) au chiffre d'affaires hors taxes, (c) aux achats, (d) au chiffre d'affaires brut.
- Q4.** Une carte de définition compte **7** champs. Lequel manque le plus souvent dans la pratique ? (a) la formule, (b) la source, (c) le responsable et le seuil, (d) la granularité.
- Q5.** Le contre-KPI d'un taux de rupture élevé est : (a) le taux de service, (b) la rotation de stock, (c) le panier moyen, (d) le taux de recouvrement.

Bloc 2 — Mesurer (Q6 à Q10)

- Q6.** Le socle compte **2 428** ruptures pour **33 296** couples produit-magasin-mois servis. Le taux de rupture vaut : (a) **7,29 %**, (b) **12,1 %**, (c) **0,73 %**, (d) **242,8 %**.
- Q7.** Additionner les stocks des **44** mois donne **2 034 733** unités, contre **45 882** au dernier mois. Cela montre qu'un stock est : (a) additif, (b) semi-additif, (c) non additif, (d) constant.

Q8. Le taux de service vaut **81,0 %** sur les commandes livrées et **78,2 %** sur toutes les commandes. L'écart s'explique par : (a) les retours, (b) les **319** commandes annulées, (c) les jours fériés, (d) les prix manquants.

Q9. Posé à **1,5** mois de couverture, un seuil d'alerte met en alerte **5 126** lignes sur **6 776**. Ce seuil est : (a) excellent, (b) trop sensible pour être utile, (c) insuffisant, (d) impossible à calculer.

Q10. Le coût logistique unitaire du socle vaut **3 284** FCFA par colis ; sur les seuls magasins, **3 281** FCFA. L'écart vient : (a) des **8 766** colis du dépôt, qui ne vend rien, (b) des taxes, (c) du carburant, (d) des retours.

Bloc 3 — Décider (Q11 à Q15, 2 points chacune)

Q11. La direction annonce : « la rotation doit passer de **9,42** à **12** tours ». Quelles conséquences mesurables attendez-vous, et quel indicateur le prouvera ?

Q12. Un indicateur de part de marché interne montre qu'un magasin gagne **2** points face à son voisin de la même ville. Pourquoi faut-il un contre-KPI, et lequel ?

Q13. Le taux de recouvrement est de **74 %** et l'encours échu de plus de **90** jours pèse **1 091 339 030** FCFA. Rédigez l'action associée au seuil, avec un volume et un délai.

Q14. Un prestataire propose **40** indicateurs « pour être complet ». Rédigez en trois phrases l'argumentaire de refus, en vous appuyant sur les chiffres du dossier du projet raté.

Q15. Deux personnes justes se contredisent : l'une annonce **23 497** clients, l'autre **188 886**. Expliquez, puis dites quelle phrase de définition empêche le débat.

C · Exercices de rédaction de KPI (20 points · 2 exercices · PATCH_8 : les énoncés reformulent, ils ne

recopient jamais un exercice du cours)

R1 — Le taux de rupture (10 points de rédaction)

Énoncé. Rédigez la carte de définition complète du **taux de rupture** du réseau de distribution, en vous appuyant sur le socle fourni.

Attendu, dans cet ordre :

1. **la formule**, avec le numérateur et le dénominateur écrits en toutes lettres, et la précision du périmètre retenu (les couples réellement servis, pas le catalogue entier) ;
2. **les deux versions concurrentes** que vous avez écartées, avec leur valeur mesurée et la raison de votre choix ;
3. **la source** exacte et ses limites connues ;
4. **la granularité** et **la fréquence**, avec la date d'arrêté des données ;
5. **le responsable**, nommé par fonction ;
6. **le seuil d'alerte**, accompagné du nombre de produits qu'il met en alerte ;
7. **le contre-KPI**, avec sa valeur mesurée ;
8. **l'action si rouge**, avec un volume et un délai.

Barème. 8 points pour les champs 1 à 7 (1 par champ, 2 pour la formule) ; 2 points pour la cohérence d'ensemble. Un seuil sans action mesurée ne compte pas.

R2 — La part des clients non identifiés (10 points)

Énoncé. Le service commercial veut suivre la part des ventes réalisées sans identification du client. Rédigez la carte complète du KPI correspondant.

Attendu. La carte complète (les 7 champs), la **valeur actuelle mesurée** sur le socle, **le choix argumenté** entre la part des lignes et la part du chiffre d'affaires, et un **contre-KPI** qui empêche d'améliorer l'indicateur en créant des fiches en doublon.

Barème. 7 points pour les 7 champs ; 2 points pour la justification du choix de définition ; 1 point pour la pertinence du contre-KPI.

D · Étude de cas — « la direction veut un tableau de bord, elle a déjà tort de le vouloir comme ça » (30 points · seuil 18)

La commande

« Nous voulons **un** tableau de bord, pour **tout** : le chiffre d'affaires, la marge, les stocks, les ruptures, les livraisons, les impayés et la logistique, avec les comparaisons avec l'an dernier. Les directeurs l'ouvriront tous les matins. »

Trois questions vous sont posées par la direction, qui attend une réponse écrite.

Question 1 (10 points) — Reformulez la commande. Écrivez la commande telle que vous l'acceptez : les décisions à éclairer, les destinataires, la fréquence de publication, et le nombre maximal d'indicateurs. Justifiez chaque élément par un chiffre du socle ou du dossier.

Question 2 (12 points) — Choisissez cinq indicateurs, pas plus. Sélectionnez au plus 5 indicateurs parmi les 10 du mandat de reprise, donnez pour chacun sa valeur actuelle mesurée, son seuil et son contre-KPI. Puis expliquez, en une phrase par indicateur écarté, pourquoi il ne fait pas partie des cinq.

Question 3 (8 points) — Rédigez la phrase de refus argumenté. En trois phrases maximum, refusez les deux demandes suivantes sans perdre le client : « toutes les mailles, tous les jours » et « le même tableau de bord pour tout le monde ». Chaque phrase doit contenir un argument **mesuré**.

Le résultat attendu (à retrouver, pas à recopier)

- **La fréquence quotidienne** se justifie sur 2 ou 3 indicateurs au maximum (ruptures, encaissements du jour) ; les autres se lisent au mois.
- **Les 5 indicateurs** doivent inclure au moins un indicateur de marge ou de prix, un indicateur de stock ou de rupture, et un indicateur de trésorerie — sinon la direction ne peut pas arbitrer.
- **Le refus argumenté** s'appuie sur des ordres de grandeur du socle : **41** indicateurs livrés pour **4** lecteurs sur **14** ; une marge qui vaut **29,12 %** ou **24,67 %** selon la base ; un panier moyen de **107 396 FCFA** par ticket ou **65 749** par ligne.

Barème de l'étude de cas

Élément	Points
Q1 : commande reformulée, complète et justifiée	10
Q2 : cinq indicateurs, valeurs mesurées, seuils, contre-KPI, écarts justifiés	12
Q3 : refus argumenté, chiffré, courtois	8
Total	30

Seuil de l'étude de cas : 18 points sur 30.

Correction détaillée

A - Réponses de récupération

1. Descriptif, diagnostique, prédictif, prescriptif — le prescriptif, seul, déclenche une action.
2. Ce que représente une ligne ; on le prouve par une clé dont le nombre de valeurs distinctes égale le nombre de lignes.
3. Formule, source, granularité, fréquence, responsable, seuil d'alerte, contre-KPI.
4. Sources, extraction, stockage, modèle, couche sémantique, visualisation, diffusion.
5. L'indicateur qui mesure ce qu'on dégrade en améliorant le premier ; il se publie à côté de lui.

B - Corrigé du quiz

Q	Réponse	Le point à retenir
1	b	une métrique se calcule ; un KPI se définit et s'attribue
2	b	1,12 % en valeur, 1,91 % en tickets : cinq définitions, toutes justes
3	b	la même marge vaut 24,67 % toute taxe comprise
4	c	le responsable et le seuil sont les deux champs qu'on saute
5	b	stocker plus réduit la rupture et dégrade la rotation (9,42 tours)
6	a	2 428 ÷ 33 296
7	b	semi-additif : additif dans l'espace, jamais dans le temps
8	b	les 319 commandes annulées sortent du premier dénominateur
9	b	à 1,5 mois, 75,6 % des lignes s'allument : le seuil ne discrimine plus
10	a	les 8 766 colis du dépôt, qui ne vend rien, sortent du second calcul
11	2 pts	réduire la couverture (1,27 mois) augmente les produits sous 1 mois (50 aujourd'hui) et pousse la rupture (7,29 %) : à surveiller par le taux de rupture par famille
12	2 pts	le gain d'un magasin est la perte de l'autre dans la même ville (60,8 % / 39,2 % à Ouagadougou) : le contre-KPI est le CA du réseau
13	2 pts	relancer les 1 155 factures de plus de 90 jours sous quinzaine, liste par ordre de montant, responsable du recouvrement
14	2 pts	41 indicateurs pour 4 lecteurs sur 14 et 11 semaines de vie : le plafond de 10 indicateurs documentés est un choix d'usage, pas de confort
15	2 pts	23 497 clients distincts ; 188 886 est la somme des clients par produit (un client achète 8,04 produits en moyenne) : la phrase « les comptages distincts ne s'additionnent pas » clôt le débat

C · Corrigé des exercices de rédaction

R1 — le taux de rupture. Formule retenue : couples produit-magasin-mois **en rupture** ÷ couples produit-magasin-mois **servis**, soit $2\,428 \div 33\,296 = 7,29\%$. Version écartée n° 1 : la part des jours de rupture sur les jours ouvrés (**13 129** jours cumulés), qui mesure la durée et non la couverture. Version écartée n° 2 : le taux calculé sur le catalogue entier, qui mesurerait l'assortiment. Granularité produit × magasin × mois ; fréquence mensuelle, données arrêtées au dernier jour du mois ; responsable : direction des opérations ; seuil : plus de **10 %** des couples servis, ce qui met en alerte les produits les plus touchés ; contre-KPI : rotation (**9,42** tours) et couverture (**1,27** mois) ; action si rouge : revue des 20 produits les plus touchés avec les achats sous cinq jours, sur la base du coût estimé de **169 386 812** FCFA.

R2 — la part des clients non identifiés. Formule : part du **chiffre d'affaires** réalisé sur des lignes sans identifiant client, soit **18,3 %** et **2 852 612 447** FCFA. Le choix de la part du chiffre d'affaires est justifié : la part des lignes mesurerait la saisie au comptoir, pas la connaissance de la clientèle. Granularité magasin × mois ; fréquence mensuelle ; responsable : qualité de la donnée ; seuil : **15 %** ; contre-KPI : nombre de fiches clients créées et taux de doublons — sans lui, l'indicateur s'améliore en polluant le référentiel ; action si rouge : contrôle de saisie par magasin, sur la liste des tickets concernés.

D · Corrigé de l'étude de cas

Q1 — la commande reformulée. Attendu : **un** écran, **5 à 10** décisions explicites, des destinataires nommés, une fréquence par indicateur (mensuelle pour la plupart, quotidienne pour les ruptures et les encaissements), et un plafond d'indicateurs assumé. La justification s'appuie sur le dossier raté : **41** indicateurs, **4** lecteurs sur **14**, **11** semaines.

Q2 — cinq indicateurs. Une sélection défendable : **CA net** (**15 595 154 955** FCFA ; contre-KPI : taux de retour **1,17 %**), **marge brute** (**29,12 %** hors taxes ; contre-KPI : volume, recette unitaire × **1,174** pour des unités × **1,169**), **taux de rupture** (**7,29 %** ; contre-KPI : rotation **9,42** tours), **taux de recouvrement** (**74,0 %** ; contre-KPI : CA net), **coût logistique unitaire** (**3 284** FCFA par colis ; contre-KPI : taux de service **81,0 %**). Les indicateurs écartés se justifient un par un : rotation (redondante avec la rupture, publiée en contre-KPI), panier moyen (utile au mois, pas au jour), part de marché interne (mesure interne, pas d'action), taux de retour (suivi en contre-KPI du CA).

Q3 — la phrase de refus. Attendu : trois phrases, chacune avec un argument chiffré. Par exemple : « Tous les jours et à toutes les mailles supposent de lire **2 428** couples de rupture par mois : nous publions au jour les ruptures et au mois les sept autres indicateurs, ce qui couvre **100 %** des décisions recensées. » ; « Le même tableau pour tous a produit **14** destinataires et **4** lecteurs : nous préparons deux vues, une de pilotage (5 indicateurs, mensuelle) et une d'action (ruptures et encaissements, quotidienne). » ; « Chaque définition supplémentaire coûte un débat non tranché — la marge vaut **29,12 %** ou **24,67 %** selon la base : nous écrivons les définitions avant l'écran. »

Rappel du barème d'ensemble : quiz **20** pts (seuil **11** sur **15**) + rédaction de KPI **20** pts + étude de cas **30** pts (seuil **18**) = **70** points, **seuil de réussite 45**.

Les douze projets de module

M01 a M12 — les projets et leurs livrables

Analyse de données appliquée & Business Intelligence
12 partie(s) assemblée(s) · document de production

Table des matières

PROJET M01.P — « UN PETIT FICHIER DE VENTES, UNE PREMIÈRE FOIS »	6
1. Énoncé	6
Le matériel	6
Les quatre livrables	6
Contraintes de rendu	7
2. Barème détaillé	7
3. Correction pas à pas (corrigé enseignant)	7
3.1 — Réponse L1 : la fiche d'identité, telle qu'elle doit être rendue	7
3.2 — Réponse L2 : les trois questions, avec leurs limites	8
3.3 — Réponse L3 : la méthode par tri et filtre, et les trois résultats	9
3.4 — Réponse L4 : ce à quoi ressemble un autoportrait utile	10
3.5 — Le fichier du fournisseur, traité à part (bonus +1, pas obligatoire)	10
4. Grille d'auto-évaluation avant rendu	10
5. Ce que le correcteur regardera en premier	10
PROJET M02.P — « PROFIL STATISTIQUE DU MAGASIN 5 »	11
1. Énoncé	6
Le matériel	6
Les six livrables	11
Contraintes de rendu	7
2. Barème détaillé	7
3. Correction pas à pas (corrigé enseignant)	7
3.1 — L1 : l'identité statistique, telle qu'elle doit être rendue	13
3.2 — L2 : centres et dispersion	13
3.3 — L3 : positions et forme	13
3.4 — L4 : liaisons	14
3.5 — L5 : incertitude, tests, décision	14
3.6 — L6 : les trois anomalies, écrites comme attendu	15
4. Grille d'auto-évaluation avant rendu	10
5. Ce que le correcteur regardera en premier	10
PROJET M03.P — « LE CLASSEUR DE PILOTAGE DE SAHEL DISTRIBUTION »	17
1. Énoncé	6
Le matériel	6
Les six livrables	11
Contraintes de rendu	7
2. Barème détaillé	7
3. Correction pas à pas (corrigé enseignant)	7
3.1 — L1 : la préparation attendue, étape par étape	19
3.2 — L2 : ce que rendent les fiches exécutées dans Calculs	20
3.3 — L3 : le tableau de bord, avec ses nombres	20
3.4 — L4 : noms, formats, protection	21
3.5 — L5 : la note, et les trois phrases qui font la différence	21
3.6 — L6 : la rejouabilité, testée comme elle le sera	22
4. Grille d'auto-évaluation avant rendu	10

5. Ce que le correcteur regardera en premier	10
PROJET M04.P — « LE DOSSIER DE DONNÉES DE LA QUINCAILLERIE DE KOUDOU-GOU »	24
1. Énoncé	6
Le matériel	6
Les six livrables	11
2. Ce qui est noté, ce qui est piégé	25
3. Barème et seuil	26
4. Correction des points d'attention	26
PROJET M05.P — « TROIS CHEMINS, UNE MÊME TABLE PROPRE »	28
1. Énoncé	6
Le matériel	6
Les six livrables	11
2. Ce qui est noté, ce qui est piégé	25
3. Barème et seuil	26
4. Correction des points d'attention	26
PROJET M06.P — « SEPT TABLES POUR UNE QUINCAILLERIE : PASSER DU CLASSEUR À LA 3FN »	33
1. Énoncé	6
Le matériel	6
Les cinq livrables, et ce qu'ils notent	33
2. Corrigé-type (à publier après la séance)	34
L1 — schema_3fn.sql (extrait représentatif, 7 tables)	35
L2 — load_duckdb.sql (extrait représentatif, exécuté)	36
L3 — load_sqlite.py (extrait, exécuté)	37
L4 — note_postgres.md (cité sans exécuté, 1 page)	37
Ce qu'on attendrait	37
Pourquoi on ne l'exécute pas	37
Ce que cette note prouve	38
3. Barème détaillé	38
4. Erreurs fréquentes (à éviter)	39
PROJET M07.P — « TRENTÉ QUESTIONS POUR LA DIRECTION COMMERCIALE : DE LA BASE BRUTE AU TABLEAU DE BORD FIABLE »	40
1. Énoncé	6
Le matériel	6
Les quatre parties, et ce qu'elles notent	41
2. Grille de notation détaillée (/20)	41
3. Corrigé-type (à publier après la séance)	41
P1 — Lecture et volumes (Q01-Q06)	41
P2 — Mesures par dimension (Q07-Q18)	42
P3 — Requêtes composées et données sales (Q19-Q26)	42
P4 — Expert : jointures multiples et « tous les » (Q27-Q30)	43
4. Les 6 pièges de fiabilité (rappel et vérification)	43
5. Critères de passage	44

PROJET M08.P — « LE SCRIPT QUI FAIT LE TRAVAIL DE TROIS MATINÉES »	45
1. Énoncé	6
Le matériel	6
Les 6 sections, et ce qu'elles notent	46
2. Grille de notation détaillée (/20)	41
3. Corrigé-type (sorties réelles du script modèle)	46
4. Les contrôles de fiabilité (rappel et vérification)	47
5. Critères de passage	44
PROJET M09.P — « 3 HEURES AVEC UN FICHIER QUE PERSONNE N'A REGARDÉ »	49
1. Énoncé	6
Le matériel	6
La règle du chronomètre	49
Les 4 livrables et ce qu'ils notent	49
2. Grille de notation détaillée (/20)	41
3. Corrigé-type (sorties réelles, exécutées le 20/09/2026)	50
4. Les contrôles de fiabilité (rappel et vérification)	47
PROJET M10.P — « LA REFONTE »	52
1. Énoncé	6
Le matériel	6
La règle du projet : les totaux avant/après sont identiques	52
Les 4 livrables et ce qu'ils notent	49
2. Grille de notation détaillée (/20)	41
3. Corrigé-type (sorties réelles, exécutées dans l'atelier)	54
3.1 Le contrôle automatique, sur le dossier de référence	54
3.2 Le contrôle sur une copie falsifiée (le cas qui doit être rejeté)	54
3.3 Le contenu attendu du livrable P1, ligne par ligne	55
3.4 Ce qu'un livrable P2 conforme contient	55
4. Les contrôles de fiabilité (rappel et vérification)	47
5. Comment rendre	56
PROJET M11.P — « LES 12 RAPPORTS SQL DE LA CELLULE COMMERCIALE »	57
1. Énoncé	6
La commande	57
Le matériel	6
Les six totaux de contrôle	57
2. Les 12 questions métier (le catalogue imposé)	58
Le format d'en-tête, imposé	58
3. Les 4 livrables	59
P1 — Les 12 requêtes (8 points)	59
P2 — Le jeu de tests (5 points)	59
P3 — La note de lecture (4 points · 2 pages)	60
P4 — La fiche de reprise (3 points)	60
4. Barème	60
5. Ce qui fait rejeter un livrable	61

6. Déroulé conseillé (4 séances, environ 8 h)	61
7. Comment vous corriger vous-même	61
PROJET M12.P — « LE TABLEAU DE BORD QUE PERSONNE N’OUVRIT »	62
1. Énoncé	6
Le matériel fourni	62
Les six totaux de contrôle	57
2. Les dix indicateurs imposés (le catalogue du mandat)	63
3. Les 4 livrables	59
P1 — Le diagnostic mesuré (6 points)	63
P2 — La carte de définition de 10 KPI (8 points)	63
P3 — Le plan de reprise (4 points)	64
P4 — La page « que fait-on quand c’est rouge ? » (2 points)	64
4. Barème	60
5. Ce qui fait rejeter un livrable	61
6. Déroulé conseillé (4 séances, environ 10 h)	65
7. Comment vous corriger vous-même	61

Projet M01.P — « Un petit fichier de ventes, une première fois »

Module M01 · projet de fin de module · 5 h · à rendre avant le module 2 · barème /20, seuil de passage 13/20

Le cadre réel. Vous êtes engagé(e) comme analyste junior chez Sahel Distribution SA, grossiste en matériaux et quincaillerie, 5 magasins et 1 dépôt. Le service informatique est une personne, qui cumule avec la maintenance. Vous n'avez ni base de données, ni licence, ni mode d'emploi : vous avez un poste, un fichier CSV reçu par messagerie, et l'obligation de produire un chiffre que le directeur ne pourra pas contester. C'est exactement une première semaine de stage — et c'est la raison pour laquelle ce projet est noté, pas seulement « fait ».

1. Énoncé

Le matériel

Élément		Chemin	Ce que c'est
Fichier travail	de	01_socle_donnees/data/projection/ventes_magasin5_2025.csv	extrait des ventes du magasin 5, année 2025 : 489 lignes × 13 colonnes
Fichier fournisseur	du	01_socle_donnees/data/brut/tarif_fournisseur_peinture.csv	grille de prix Peinture Saaba SARL : 42 lignes , dont 38 utiles
Corrigé référence	de	01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv	480 lignes , à ne pas ouvrir avant le rendu
Vos outils		tableur (Excel ou LibreOffice) + éditeur de texte + un traitement de texte	rien d'autre n'est requis ni autorisé

Les quatre livrables

L1 — Fiche d'identité du fichier (1 page). Les six phrases du chapitre C03 (nom et source · objet · grain · clé · mesures et attributs · étendue), suivies d'un tableau de **contrôles** : pour chacune des 13 colonnes, le type constaté, le nombre de valeurs distinctes, les bornes ou les modalités notables, et les défauts visibles. Vous devez avoir **compté** les défauts, pas les avoir devinés.

L2 — Trois questions que l'on peut poser à ce fichier (une demi-page). Chacune formulée en **question analytique complète** (objet, unité + définition, comparaison, périmètre, critère de réussite), plus une ligne « ce que ce fichier ne permet pas de répondre » par question.

L3 — Une réponse chiffrée, sans autre outil que le tri et le filtre du tableur (1 page). Vous choisissez **une** des trois demandes ci-dessous, vous y répondez avec un chiffre juste, et vous annexez la méthode en cinq lignes maximum (aucune formule complexe, aucune macro, aucun autre logiciel) :

- **a)** « Quelle catégorie de produits pèse le plus dans le chiffre d'affaires du magasin 5 en 2025, et à quelle hauteur ? » (réponse attendue en L3-a du corrigé)
- **b)** « Quel vendeur arrive en tête, et avec quel total ? » (réponse attendue en L3-b)
- **c)** « Quel mois a été le plus fort, et de combien par rapport au plus faible ? » (réponse attendue en L3-c)

L4 — « Ce que je ne sais pas encore faire » (1 page, à écrire **après** L1-L3, sans tricher). La liste honnête des opérations que vous avez vues faire (par un collègue, dans un forum, dans ce

manuel) et que vous ne maîtrisez pas, classées en trois colonnes : ce dont j'aurai besoin tout de suite · ce dont j'aurai besoin dans trois mois · ce qui n'est pas mon besoin. Trois lignes minimum par catégorie.

Contraintes de rendu

1. Les fichiers sont déposés dans une arborescence conforme à la convention du module 1 (01_brut en lecture seule, 02_propre, 03_analyse, 04_livrables, 90_journal).
2. Le **journal de transformation** compte au moins **8 entrées** datées, chacune avec les nombres avant/après.
3. Le livrable L1-L4 est un seul document PDF nommé 2026-09-17_projet_M01_P_votre_nom.pdf, plus le classeur de travail.
4. **Interdits** : copier un total du corrigé sans montrer la manipulation qui y mène ; utiliser un tableau croisé dynamique ou une requête pour L3 (l'exercice porte sur le tri et le filtre) ; modifier le fichier brut.

2. Barème détaillé

#	Critère	Pts	Ce qui fait perdre les points
1	Fiche d'identité : six phrases justes	4	grain exprimé en « c'est du mensuel » (ce n'est pas un grain) ; clé donnée comme n_ticket seul
2	Contrôles par colonne : tous les comptages exacts	5	un seul nombre faux = -0,5 ; un nombre non vérifié (pas de formule, pas de commande) = -1
3	Trois questions : cinq composants, réfutabilité	3	adjectif non mesurable resté dans la question ; critère de réussite absent
4	Réponse chiffrée L3 : valeur exacte + méthode reproductible	4	chiffre juste obtenu par un moyen interdit ; ou juste mais sans méthode
5	Journal + arborescence + nommage	2	écart de total non expliqué ; fichier brut touché
6	L4 : lucidité de l'autoportrait	2	liste de vœux au lieu d'une liste d'outils/gestes précis
Total		20	seuil de passage en M02 : 13

3. Correction pas à pas (corrigé enseignant)

Cette correction est rédigée comme un compte-rendu de travail, parce que c'est ce qui est noté. Elle s'appuie sur les valeurs réellement produites par le fichier livré ; si votre fichier diffère, ré-générez (python3 01_socle_donnees/scripts/generation_socle.py) avant de vous comparer.

3.1 — Réponse L1 : la fiche d'identité, telle qu'elle doit être rendue

Nom et source. ventes_magasin5_2025.csv, export de la caisse du magasin 5 pour l'année 2025, reçu par messagerie le 17/09/2026 ; 489 lignes, 32 Ko, en-tête sur 1 ligne, séparateur point-virgule, encodage UTF-8.

Objet. Les articles facturés au magasin 5 sur l'année 2025.

Grain. Une ligne = un article d'un ticket de caisse. Ni une vente, ni un jour, ni un client.

Clé. Aucune colonne n'est unique : n_ticket identifie le ticket et se répète (1 à 4 lignes par ticket). L'unicité de la ligne est assurée par le couple (n_ticket, produit).

Mesures et attributs. Mesures : `quantite`, `prix_unitaire_ht`, `remise`, `montant_ht`, `montant_ttc`. Attributs : `date`, `heure`, `magasin`, `vendeur`, `client`, `produit`, `categorie`. Identifiants : `n_ticket`, `client`, `produit`.

Étendue. 489 lignes · 316 tickets distincts · 31 jours avec des ventes (l'extrait contient 40 lignes par mois, tous certains jours ne sont donc pas représentés) · 4 vendeurs · 145 produits distincts · 7 catégories · 372 identifiants clients distincts · 18 lignes sans client · période couverte 2025-01-01 → 2025-12-31.

Tableau des contrôles (le cœur du point 2 du barème) :

Colonne	Type constaté	Distincts	Bornes / modalités	Défaut constaté	Comment je l'ai compté
<code>n_ticket</code>	texte	316	T05-25...-..., 17 caractères	9 lignes en double (489 – 480)	=SOMMEPROD(1/NB.SI(...)) ou tri + comptage manuel
<code>date</code>	texte	55	2025-01-01 → 2025-12-31	65 lignes en JJ/MM/AAAA	=NB.SI(B:B;"?*/?/*")
<code>heure</code>	texte	356	07:01 → 18:57	—	—
<code>magasin</code>	texte	1	« Magasin 5 — Kaya Marché »	—	—
<code>vendeur</code>	texte	4	Ilboudo, Sankara, Ouédraogo, Bationo	—	—
<code>client</code>	nombre ou vide	372	0 = comptoir	18 vides (=NB.EUILL)	vides ≠ 0 : deux choses distinctes
<code>produit</code>	texte	145	« ... — réf N » dans le libellé	deux infos dans une cellule	—
<code>categorie</code>	texte	7	Matériaux 146 · Quincaillerie 115 · Consommables 58 · Plomberie 57 · Electricité 54 · Peinture 34 · Bois & panneaux 25	—	tri + comptage
<code>quantite</code>	nombre	37	-13 (retours) → 14 000	7 lignes > 500	=MAX(), =NB.SI(I:I;">500")
<code>prix_unitaire_ht</code>	nombre	323	675 → 40 550	—	=MIN(), =MAX()
<code>remise</code>	nombre	6	0 → 0,11 (proportion)	aucune valeur > 0,4 ici	contrôle de plage
<code>montant_ht</code>	nombre	456	total = à recalculer	cohérent avec quantité × prix × (1 – remise)	=SOMME()
<code>montant_ttc</code>	texte	456	—	486 lignes non numériques	=NBVAL(M:M) - NB(M:M)

Le point qui distingue un bon rendu d'un rendu moyen : la ligne `quantite`. Le candidat solide écrit « 7 lignes suspectes, montant cohérent, donc doute sur l'unité de saisie → à voir avec le métier, pas à supprimer ». Le candidat pressé supprime les 7 lignes et perd 1 point au contrôle des totaux.

3.2 — Réponse L2 : les trois questions, avec leurs limites

Question 1 (mix de catégories). « En 2025, au magasin 5, quelle catégorie de produits représente la part la plus élevée du CA TTC (somme de `montant_ttc`, retours inclus avec leur signe, après retrait des doublons), et cette part dépasse-t-elle 20 % du total ? » Critère de réussite : la somme des 7 catégories doit redonner le total de l'extrait nettoyé, **36 073 185 FCFA** ; la réponse est une part en pourcentage à ±0,1 pt. Ce que ce fichier ne permet pas de répondre : la

rentabilité de la catégorie (aucun coût d'achat dans ce fichier — il est dans `couts_achat.csv`, module 13) et la comparaison aux autres magasins (extrait limité au magasin 5).

Question 2 (saisonnalité). « Quel mois de 2025 présente le CA TTC le plus éloigné de la médiane des douze mois, et de combien de fois le mois le plus faible ? » Critère : ratio meilleur mois / pire mois = **4,2** (décembre 5 292 517 FCFA, mai 1 258 801 FCFA). Limite : l'extrait contient 40 lignes par mois, **tous les mois ne sont donc pas complets** : ce ratio décrit l'extrait, pas la réalité commerciale. Cette phrase est obligatoire dans un rendu correct ; l'omettre vaut une note plafonnée à 15/20, quel que soit le reste.

Question 3 (qualité avant tout). « Combien de lignes de cet extrait ne peuvent pas être sommées telles quelles, et quel est l'effet chiffré de leur correction sur le total annuel ? » Critère : 486 montants en texte, 9 doublons, écart de total = **266 132 FCFA**, soit **+0,74 %** sur le total non nettoyé (36 339 317 → 36 073 185). Limite : le fichier ne dit pas si les doublons viennent de la caisse ou de l'export ; seule la caisse (ou le journal d'export) tranche.

3.3 — Réponse L3 : la méthode par tri et filtre, et les trois résultats

Préparation commune (autorisée : tri, filtre, =SOMME() sur la colonne filtrée).

1. Import assisté, toutes colonnes en texte, puis une colonne auxiliaire `ttc_num = CBI(SUPPRESSED(SUBSTITUE(...)))` — 486 conversions, 0 échec. (Oui, une formule de conversion est autorisée : sans elle, aucun total n'est possible ; ce qui est interdit, c'est le TCD et la requête.)
2. Tri sur `n_ticket` puis suppression des doublons stricts par la commande du tableur → 480 lignes. Contrôle : `=SOMME(ttc_num)` = **36 073 185**.

L3-a (catégorie dominante). Filtre sur `categorie` = Plomberie, `=SOMME()` sur la colonne filtrée → **8 754 982 FCFA**, soit **24,3 %** du total. Réponse : « la catégorie la plus lourde du CA 2025 du magasin 5 est la **Plomberie**, à 24,3 % ». Suit le classement complet, à exiger d'un rendu sérieux :

Catégorie	CA TTC (extrait nettoyé)	Part
Plomberie	8 754 982 FCFA	24,3 %
Matériaux	6 519 357 FCFA	18,1 %
Electricité	6 311 645 FCFA	17,5 %
Bois & panneaux	4 844 758 FCFA	13,4 %
Quincaillerie	4 203 121 FCFA	11,7 %
Peinture	2 938 358 FCFA	8,1 %
Consommables	2 500 964 FCFA	6,9 %
Total	36 073 185 FCFA	100 %

Attention au piège classique, et il est là pour cela : le **nombre de lignes** ne suit pas le CA. La catégorie la plus fréquente en lignes est Matériaux (146 lignes), pas Plomberie (57 lignes). Un rendu qui confond fréquence et montant perd le point 4 intégralement, parce que la phrase de conclusion serait fausse.

L3-b (vendeur en tête). Bationo : **11 370 350 FCFA** · Sankara 9 423 281 · Ouédraogo 8 483 019 · Ilboudo 6 796 535. Somme des quatre = 36 073 185 FCFA ✓ (le contrôle croisé fait partie de la réponse attendue).

L3-c (mois extrêmes). Décembre 2025 = **5 292 517 FCFA** ; mai 2025 = **1 258 801 FCFA** ; ratio **4,2**. Le détail complet des douze mois figure dans le chapitre C04 §5.3 ; un candidat doit pouvoir en retrouver au moins quatre sur demande orale du correcteur.

3.4 — Réponse L4 : ce à quoi ressemble un autoportrait utile

Exemple de rendu recevable (extrait) :

J'en aurai besoin tout de suite	Dans trois mois	Ce n'est pas mon besoin
tableaux croisés (M03-M05)	jointures SQL sur la base de l'entreprise (M07)	entraîner un modèle de prédiction (M19-M21)
RECHERCHEX pour relier les noms de produits (M06)	script Python pour rejouer le nettoyage (M08)	gestion de cluster (M24)
mettre un format de date fiable (déjà fait ici, à consolider M04)	publier un tableau de bord Power BI (M13-M18)	configuration d'un entrepôt (M15)

Un autoportrait qui écrit « je veux devenir meilleur en data » est noté 0/2 : ce n'est pas une liste de besoins, c'est une intention.

3.5 — Le fichier du fournisseur, traité à part (bonus +1, pas obligatoire)

Le point n'est pas dans l'énoncé, mais il revient toujours en entretien de stage : **importez tarif_fournisseur_peinture.csv proprement** et donnez le nombre de références, le prix moyen et le prix le plus élevé. Réponse : 38 lignes utiles (42 – 3 de chapeau – 1 d'en-tête), encodage **Windows-1252**, séparateur **;**, décimale **virgule**, TVA 0,18 sur toutes les lignes. Un rendu qui écrit « le fichier est corrompu » perd le bonus ; un rendu qui écrit « il faut lire en cp1252 avec 3 lignes à ignorer » le gagne.

4. Grille d'auto-évaluation avant rendu

À cocher soi-même, honnêtement, avant d'envoyer. Sept cases manquantes = rendez-vous dans deux jours.

- ☐ Chaque nombre cité dans L1 et L3 est le résultat d'une formule ou d'une commande que je peux rejouer devant quelqu'un.
- ☐ J'ai écrit le grain en une phrase, sans le mot « mensuel ».
- ☐ J'ai distingué les 9 doublons (à retirer) des 7 quantités suspectes (à signaler).
- ☐ J'ai dit « vide $\neq$ 0 » pour la colonne `client`, et je l'ai justifié par la règle métier du comp-toir.
- ☐ Mes trois questions L2 ont chacune cinq composants, et aucune ne contient d'adjectif non chiffré.
- ☐ Dans L2, j'ai écrit explicitement que l'extrait est un échantillon de 40 lignes par mois.
- ☐ Mon total nettoyé est 36 073 185 FCFA, et mon journal explique pourquoi le brut affichait 36 339 317 FCFA.
- ☐ Le brut est en lecture seule, et aucun de ses octets n'a changé (vérifié par empreinte).

5. Ce que le correcteur regardera en premier

Pas les chiffres — ils sont vérifiables en dix secondes. La première chose qu'un évaluateur expérimenté regarde, dans un dossier de ce niveau, est **l'ordre des opérations** : le candidat a-t-il documenté avant de corriger, ou corrigé puis justifié ? Le second regard va aux **limites écrites spontanément** (« l'extrait n'est pas la population ») : c'est le meilleur prédicteur de la capacité à travailler sans supervision. Le troisième, le nom des fichiers. Les chiffres viennent ensuite, et ils sont la partie facile.

Projet M02.P — « Profil statistique du magasin 5 »

Module M02 · projet de fin de module · 6 h · à rendre avant le module 3 · barème /20, seuil de passage 13/20

Le cadre réel. Vous êtes toujours chez Sahel Distribution SA. Le directeur a maintenant une phrase pour vous : « je ne veux pas votre moyenne, je veux la vérité du magasin ». Derrière cette phrase maladroite se trouve exactement le travail du module 2 : dire ce que vaut chaque indicateur, comment il a été calculé, ce qu'il dit, ce qu'il ne dit pas, et ce que l'extrait permet ou non d'affirmer. Vous rendez six pages, pas une.

Note de cohérence. Le plan validé annonçait « Profil statistique de la Boutique Kwame, 1 500 ventes ». Le socle de données réel de la formation fournit un autre volume — 480 lignes et 316 tickets une fois nettoyés, pour le magasin 5 sur 2025 — et c'est sur lui que portent les livrables, afin que chaque chiffre du corrigé soit vérifiable. Le livrable, le barème et les exigences de preuve sont inchangés.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Fichier de travail	01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv	le fichier nettoyé au projet M01 : 480 lignes x 13 variables, 316 tickets
Population de référence	01_socle_donnees/data/reference/ventes_propres.csv	toutes les ventes propres, 240 000 lignes ; le magasin 5 sur 2025 y tient 7 676 lignes et 4 524 tickets
Contrôle des chiffres	01_socle_donnees/data/reference/chiffres_cites.md	à utiliser après chaque calcul, pour vérifier — pas pour remplir
Outils	tableur (Excel ou LibreOffice) + Python si vous l'avez	le tableur est obligatoire au moins une fois, Python sert de contrôle

Les six livrables

L1 — Page d'identité statistique (1 page). Population visée, unité sondée, effectifs (480 lignes, 316 tickets, 7 676 lignes et 4 524 tickets en face), taux de représentation, règle d'extraction reconstituée, liste des variables par nature (quantitatives discrètes / continues / nominales / ordinales), et les trois questions du module auxquelles le fichier peut répondre. Une ligne par rubrique, aucune phrase de remplissage.

L2 — Page des centres et de la dispersion (1 page, prolongement de M02.P1 et M02.P2). Cinq indicateurs, chacun au format valeur · calcul · ce que ça dit · ce que ça ne dit pas : moyenne, médiane, mode, écart type, écart interquartiles. Vous ajoutez le coefficient de variation et vous expliquez ce qu'il fait saliver.

L3 — Page des positions et de la forme (1 page). Premier et troisième quartiles, trois percentiles choisis parmi dix, quatre-vingts, quatre-vingt-quinze ; bornes de Tukey et nombre d'observations hors borne ; histogramme commenté (classes, effectifs par classe, part du chiffre d'affaires par classe) ; asymétrie et excursion des valeurs.

L4 — Page des liaisons (1 page). Trois liaisons au choix, chacune avec covariance ou coefficient, effectif, grain, carré du coefficient, droite de régression si elle a un sens, et une ligne « ce que la liaison ne dit pas ». Vous menez en plus **un** contrôle de confusion et vous donnez les deux chiffres, avant et après.

L5 — Page d'incertitude et de décision (1 page). Intervalle à 95 % du panier moyen et de la médiane, biais de l'extrait chiffré, un test de comparaison entre deux vendeurs conduit **au tableur puis vérifié en Python**, le même test entre le magasin 1 et le magasin 5 sur l'année 2025, et la marge de détection de votre étude. Un paragraphe de décision, une phrase de prudence.

L6 — Anomalies et conclusion (1 page). **Trois anomalies chiffrées** au choix (doublons, quantités hors échelle, dates manuscrites, clients non renseignés, retours négatifs, tickets aberrants), chacune en quatre lignes : ce qui est observé, combien, ce que ça change au chiffre, ce qui a été fait. Puis la **conclusion en cinq lignes**, qui dit ce que l'entreprise peut décider avec ce profil, et ce qu'elle ne peut pas.

Contraintes de rendu

1. Un seul document PDF de six pages, nommé `2026-MM-JJ_projet_M02_P_votre_nom.pdf`, plus le classeur de travail et, si vous en avez utilisé, les scripts.
2. Chaque page porte en pied de page : l'effectif utilisé, le grain, le nom du fichier et la date de lecture.
3. Tout nombre affiché doit être accompagné de son moyen de preuve à portée de main : capture de la formule, ligne de commande, ou extrait du script. Un chiffre sans méthode est compté comme faux, même juste.
4. Précision : montants en entiers de FCFA, coefficients à trois décimales, pourcentages à une décimale. Aucun arrondi « de présentation » non signalé.
5. Les comparaisons multiples (plus de deux groupes) doivent annoncer le nombre de tests et le seuil employé.
6. **Interdits** : recopier une valeur depuis `chiffres_cites.md` sans le calcul correspondant ; mélanger dans une même page un total au niveau ligne et un total au niveau ticket sans écrire lequel est lequel ; publier un test sans son intervalle ; conclure à l'absence de différence sur un p supérieur au seuil.

2. Barème détaillé

#	Critère	Pts	Ce qui fait perdre les points
1	L1 : identité statistique exacte, règle d'extraction reconstituée	3	taux de représentation absent ; unité sondée confondue avec le nombre de lignes
2	L2 : cinq indicateurs justes, avec le « ce que ça ne dit pas »	3	mode unique annoncé alors qu'il y en a deux ; moyenne seule présentée comme « le panier type »
3	L3 : positions et forme, histogramme commenté	4	classes fermées qui avalent les retours négatifs ; bornes de Tukey calculées sur le mauvais effectif
4	L4 : liaisons chiffrées et contrôle de confusion mené	3	R^2 publié sans r , ou sans l'effectif ; pente extrapolée hors domaine
5	L5 : intervalles, tests exécutés aux deux outils, marge de détection	4	test exécuté sans grain déclaré ; p lu comme probabilité que H_0 soit vraie ; bootstrap sans graine
6	L6 : trois anomalies chiffrées, conclusion honnête, journal	3	anomalie racontée et pas comptée ; conclusion qui généralise au-delà du périmètre
Total		20	seuil de passage en M03 : 13

3. Correction pas à pas (corrigé enseignant)

Ce corrigé est rédigé comme un compte-rendu de travail. Les valeurs sont celles produites par les fichiers livrés du socle ; si votre fichier diffère, régénérez-le (python3 01_socle_donnees/scripts/generation_socle.py) avant de vous comparer.

3.1 — L1 : l'identité statistique, telle qu'elle doit être rendue

Rubrique	Réponse attendue
Population visée	les tickets du magasin 5 sur l'année 2025 : 4 524 tickets, 7 676 lignes
Échantillon réellement utilisé	316 tickets, 480 lignes — soit 7,0 % des tickets et 6,3 % des lignes
Règle d'extraction	les 40 premières lignes de chaque mois, triées par date : les jours 1 à 4 seulement
Unité sondée à retenir	le ticket pour toute question de clientèle, la ligne pour toute question de produit
Variables quantitatives	quantite (discrète), prix_unitaire_ht, remise, montant_ht, montant_ttc (continues au sens du métier : montants entiers en FCFA)
Variables nominales	magasin, vendeur, produit, categorie ; client est un identifiant, pas une quantité
Questions ouvertes au fichier	panier moyen et sa dispersion, structure par catégorie, liaisons quantité-montant, comparaison de vendeurs
Question fermée	la marge nette, la satisfaction client, l'effet d'une promotion : aucune de ces trois n'est répondable ici

Le correcteur vérifie en priorité la troisième ligne : un candidat qui écrit « 480 lignes prises au hasard » a perdu le fil du module, un candidat qui écrit « les 40 premières lignes de chaque mois » a compris M02.C07.

3.2 — L2 : centres et dispersion

Indicateur	Valeur	Calcul	Ce que ça dit	Ce que ça ne dit pas
Moyenne panier	du 114 FCFA	156 somme de 36 073 185 FCFA sur 316 tickets	la masse d'affaires par ticket	71,8 % des tickets sont sous cette valeur
Médiane	58 FCFA	423 =MEDIANE() sur les 316 totaux	la moitié des tickets est en dessous	ce que font les 25 plus gros tickets
Modes	6 797 et 8 260 FCFA	valeurs les plus fréquentes, deux ex-æquo	deux tailles de ticket reviennent souvent	rien sur les gros paniers, qui sont justement l'enjeu
Écart type	182 FCFA	095 =ECARTYPE.STANDARD()	la dispersion est supérieure à la moyenne	la dispersion est tirée par la queue de droite
Écart interquartiles	109 FCFA	924 130 559 moins 20 635	la moitié centrale s'étale sur cette plage	le chiffre d'affaires total
Coefficient de variation	de 159,5 %	écart type sur moyenne	aucune moyenne ne représente ce fichier à elle seule	—

Phrase à exiger en pied de page : « le ratio moyenne sur médiane vaut 1,95 : la moyenne est à peu près le double de la médiane, ce qui signe une distribution étirée à droite, pas une augmentation générale des paniers ».

3.3 — L3 : positions et forme

Premier quartile 20 635 FCFA, troisième 130 559 FCFA, P10 = 7 390, P80 = 165 177, P90 = 235 318, P95 = 435 984 FCFA ; minimum -150 804, maximum 1 343 666, étendue 1 494 470 FCFA.

Bornes de Tukey : borne supérieure $130\,559 + 1,5 \times 109\,924$, soit 295 445 FCFA, avec 25 tickets au-delà.

L'histogramme attendu compte **sept** classes, la première et la dernière ouvertes, avec 3, 133, 84, 25, 23, 17 et 31 tickets. Deux pièges de fabrication, à commenter :

1. borné à 150 000 FCFA, l'histogramme écrase la moitié du chiffre d'affaires dans la dernière classe ; six classes fermées font disparaître les trois tickets négatifs, qui sont pourtant la preuve qu'il y a des retours ;
2. la classe 0 à 50 000 FCFA pèse 42,1 % des effectifs et 7,3 % du chiffre d'affaires : les deux lectures doivent apparaître, sinon le graphique raconte « un magasin de petits clients » alors que le dernier décile porte 47,5 % du chiffre d'affaires.

Asymétrie 3,84, excès de kurtosis 18,25, 307 valeurs de panier distinctes sur 316 : le vocabulaire à vérifier dans la copie est celui-là, et le mot « normale » n'y a pas sa place.

3.4 — L4 : liaisons

Le couple attendu, parce qu'il est instructif sans être trivial : `quantite` et `montant_ttc` au niveau ligne. Covariance 857 634, coefficient de Pearson 0,712, Spearman 0,641, R^2 50,7 %, pente 12 237 FCFA par pièce, ordonnée à l'origine -10 402 FCFA. Contrôle exigé : la droite au point moyen rend 75 257 FCFA contre 75 152 FCFA observés — l'écart est un arrondi d'affichage, et le candidat qui écrit ce contrôle gagne le point de méthode.

Le contrôle de confusion attendu : l'écart de panier moyen entre le premier et le dernier vendeur vaut 34 014 FCFA toutes catégories confondues ; recompté à catégorie contenue, il remonte à 57 346 FCFA en moyenne (17 433 FCFA au moins en Quincaillerie, 173 167 FCFA au plus en Electricité). La phrase de conclusion juste est : « à catégorie de produits égale, les écarts entre vendeurs ne se résorbent pas — ils s'agrandissent. Le portefeuille de produits n'explique donc pas le classement, et il ne l'innocente pas non plus. »

3.5 — L5 : incertitude, tests, décision

Objet	Valeur attendue	Vérification demandée
Intervalle à 95 % du panier moyen	[94 078 ; 134 234] FCFA, largeur 40 156	erreur type 10 244 FCFA retrouvée par $182\,095$ sur racine de 316
Biais de l'extrait	-2,3 % sur la moyenne, -14,3 % sur la médiane	médiane des jours 1 à 4 dans la population : 58 424 FCFA
Bootstrap de la médiane	[52 318 ; 64 632] FCFA	2 000 tirages, graine 20260917
Valeur de population du panier moyen	116 886 FCFA	elle est bien dans l'intervalle : la couverture est vérifiée
Comparaison Bationo / Ilboudo, au niveau ligne	écart 28 609 FCFA, $t = 1,52$, $p = 0,13$, IC [-8 279 ; 65 498]	tableur <code>=T.TEST(tab1;tab2;2;3)</code> puis <code>scipy.stats.ttest_ind(..., equal_var=False)</code>
Contrôle par permutation	$p = 0,1443$	10 000 redistributions, graine annoncée
Test des rangs sur le même couple	$p = 0,0161$	publié avec la mention du seuil corrigé 0,0083 : non concluant après correction
Comparaison magasin 1 / magasin 5, sur 2025 au grain du ticket	écart -8 997 FCFA, $t = -3,37$, ddl 6 782,8, $p = 0,000751$, IC [-14 226 ; -3 767]	14 463 tickets contre 4 524 ; moyennes 107 889 et 116 886 FCFA
Marge de détection de l'étude vendeurs	52 956 FCFA par ligne à 80 % de puissance	412 lignes par groupe auraient été nécessaires

Les deux derniers blocs sont le cœur du livrable : le premier montre un **écart établi** (intervalle excluant zéro, avec 14 463 et 4 524 tickets), le second montre que le même dispositif, ramené à 125 et 109 lignes, ne voit rien de stable. Le correcteur attend la phrase qui relie les deux : « la

différence entre magasins est mesurable, la différence entre vendeurs non, et la raison est l'effectif, pas la volonté ».

Une exécution au tableur doit laisser voir : la colonne des totaux par ticket (`=SOMME.SI()`), les quatre cellules `MOYENNE`, `NB`, `ECARTYPE.STANDARD`, puis la formule de l'erreur type et la marge. Un candidat qui rend uniquement le `p` Python perd le point de méthode demandé.

3.6 — L6 : les trois anomalies, écrites comme attendu

1. **Neuf lignes en double** dans le fichier reçu : $36\,339\,317 - 36\,073\,185 = 266\,132$ FCFA comptés en trop, soit +0,74 % de chiffre d'affaires si on ne les retire pas. Elles ont été retirées au projet M01 et ne réapparaissent nulle part dans ce livrable.
2. **Sept quantités hors échelle** : maximum 14 000 pièces dans le fichier reçu, 68 une fois l'unité corrigée. Sans correction, la quantité moyenne serait faussée d'un facteur qui déforme la pente de la droite quantité-montant.
3. **Cent trois lignes sans client exploitable** (dont 18 sans identifiant renseigné), soit 21,5 % de l'effectif : toute statistique de clientèle est donc calculée sur un périmètre de fait, et le taux est à rappeler à chaque fois — c'est le seul des trois défauts qui ne se répare pas en nettoyant.

Conclusion en cinq lignes, modèle : le magasin 5 sur 2025, dans l'extrait disponible, fait un panier moyen de 114 156 FCFA, intervalle [94 078 ; 134 234], et un demi-ticket type à 58 423 FCFA. La masse d'affaires est concentrée : le dernier décile porte 47,5 % du chiffre d'affaires. Aucun classement de vendeurs n'est établi par ces données, et l'extrait sous-estime la médiane annuelle de 14,3 % du fait de sa couverture en jours. La liaison quantité-montant est forte (0,712) mais explique la moitié du phénomène, pas l'autre. Nous recommandons un recalcul sur les douze mois complets avant toute décision d'assortiment.

4. Grille d'auto-évaluation avant rendu

Répondez par oui ou non ; trois « non » = ne rendez pas, reprenez la page concernée.

#	Question	Si non
1	Chaque page indique-t-elle effectif, grain, fichier et date de lecture ?	pied de page manquant, point perdu
2	Ai-je écrit la règle d'extraction et son effet chiffré (médiane -14,3 %) ?	reprendre M02.C07 §5.3
3	Moyenne et médiane sont-elles données ensemble, avec le ratio commenté ?	reprendre M02.C02
4	L'histogramme a-t-il sept classes, les deux extrêmes ouvertes ?	reprendre M02.C05 §5.2
5	Ai-je distingué effectif et chiffre d'affaires par classe ?	reprendre M02.C05 §5.4
6	Chaque coefficient publié porte-t-il son effectif, son grain et son carré ?	reprendre M02.C06 §9
7	Le contrôle de confusion est-il chiffré avant/après ?	reprendre M02.C06 §5.6
8	Chaque intervalle est-il accompagné de son erreur type et du nombre de tirages quand il vient d'un bootstrap ?	reprendre M02.C07 §5.5-5.6
9	Chaque test annonce-t-il sa question, son grain, son seuil et le nombre de comparaisons ?	reprendre M02.C08 §5.1-5.2
10	Ai-je écrit « non établi » là où j'ai été tenté d'écrire « identique » ?	reprendre M02.C08 §5.3
11	Les trois anomalies sont-elles comptées, pas racontées ?	reprendre M01 (doublons) et M02.C01
12	Un tiers peut-il rejouer mes calculs sans moi (scripts, formules, graine) ?	joindre le classeur et les scripts

5. Ce que le correcteur regardera en premier

1. **Le pied de page de la page 2.** Effectif, grain, fichier : trois indications qui, si elles manquent, décrètent que le module n'a pas été travaillé mais lu.
2. **La présence de deux chiffres là où un seul suffirait.** Moyenne et médiane, r et R^2 , p et intervalle, écart avant et après contrôle : le module 2 se joue là, pas dans la justesse arithmétique.
3. **La mention du biais de couverture.** Une seule page, même mal mise en forme, qui écrit « jours 1 à 4 » et « -14,3 % » vaut plus que six pages impeccables qui l'ignorent.
4. **La discipline sur les comparaisons.** Six comparaisons annoncées comme telles avec un seuil corrigé, c'est un comportement professionnel, pas une coquetterie technique.
5. **La conclusion.** Cinq lignes qui disent ce que l'on peut décider — et ce que l'on ne peut pas — sont la vraie livraison du module. Une conclusion qui promet une action non fondée sur les pages précédentes fait perdre le point entier.

Projet M03.P — « Le classeur de pilotage de Sahel Distribution »

Module M03 · projet de fin de module · 6 h · à rendre avant le module 4 · barème /20, seuil de passage 13/20

Le cadre réel. Vous êtes chez Sahel Distribution SA, et c'est la première fois qu'on vous demande un **classeur** plutôt qu'un rapport. Le directeur commercial a dit : « je veux ouvrir le fichier le lundi matin, changer le mois, et que tout bouge ». Cette phrase contient tout le module : une donnée nettoyée par une méthode écrite, des calculs qui survivent à l'actualisation, un tableau de bord qui ne ment pas sur son échelle, et une forme qui permet à quelqu'un d'autre de rouvrir le fichier sans vous appeler. **C'est la première pièce de votre portfolio : elle sera revue par un recruteur, pas seulement par moi.**

Note de cohérence. Le plan validé décrit « un fichier de ventes + un fichier de cibles, classeur à trois feuilles : données préparées, calculs, tableau de bord ». Le socle de la formation fournit exactement ces deux entrées — `ventes_brutes.csv` (489 lignes énoncées) et `objectifs_de_ca.csv` (218 lignes jointes) — et le classeur d'entraînement du module, `m03_classeur_atelier.xlsx`, en présente une version disséquée en onze feuilles. Vous rendez, vous, **trois feuilles de rendu** ; les annexes sont admises, pas comptées. Les chiffres de ce corrigé sont ceux du socle livré, vérifiables ligne à ligne.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Fichier de ventes brut	<code>01_socle_donnees/data/brut/ventes_brutes.csv</code>	le fichier reçu : 489 lignes énoncées, montants en texte, dates en deux écritures
Fichier nettoyé de contrôle	<code>01_socle_donnees/data/reference/ventes_magasin5_2025_ATTENDU.csv</code>	ce que la préparation doit produire : 480 lignes, 316 tickets
Cibles commerciales	<code>01_socle_donnees/data/brut/objectifs_de_ca.csv</code>	218 lignes magasin × année × mois, 198 030 000 FCFA sur le magasin 5 en 2025
Référentiels	<code>01_socle_donnees/data/brut/produits.csv</code> , <code>clients.csv</code> , <code>magasins.csv</code>	les dimensions du modèle
Classeur de référence	<code>01_socle_donnees/data/projection/m03_classeur_atelier.xlsx</code>	la version d'enseignement : 11 feuilles, 28 fiches de fonction, une grille mensuelle, une grille croisée
Contrôle des chiffres	<code>01_socle_donnees/data/reference/chiffres_cites.md</code>	à utiliser après le calcul, jamais comme source de copie
Outils	un tableur (Excel 2021+ ou LibreOffice Calc), Power Query et le modèle de données si votre poste les a	sinon : la voie accessible, et vous l'écrivez

Les six livrables

L1 — Feuille Données (la préparation, par requête). Une requête nommée, de six étapes au minimum, qui part du CSV brut et rend 480 lignes × 16 colonnes : types corrigés, séparateur décimal des remises, dates reconstituées dans les deux écritures, doublons traités à la règle « une

ligne par ticket et par produit », quantité filtrée, colonne `est_retour` ajoutée, plus deux colonnes de jointure `mois` et `annee_mois`. Le chemin du fichier source passe par un paramètre ou une cellule nommée, jamais par un chemin tapé dans l'étape.

L2 — Feuille `Calculs` (les fiches de fonction). **Huit fiches au format du module** — nom et version · syntaxe · rôle de chaque argument · exemple chiffré sur le jeu de l'atelier · cas réel d'entreprise · erreur fréquente et son symptôme — dont trois au choix hors de la liste d'office (`SOMME.SI.ENS`, `NB.SI.ENS`, `RECHERCHEX`, `INDEX/EQUIV`, `FIN.MOIS`, `NB.JOURS.OUVRES`, `FILTRE` étant imposées). Chaque fiche occupe une ligne du tableau de bord des fiches et sa formule doit être **exécutée dans la feuille**, pas décrite.

L3 — Feuille `Pilotage` (le tableau de bord). Un tableau croisé dynamique par catégorie (lignes, CA, moyenne, part en % du total général), un second par mois avec les jours ouvrés, **quatre graphiques** (classement en barres, évolution en courbes avec moyenne glissante à trois mois, parts, et un quatrième au choix argumenté), **trois segments** (ville, catégorie, mois) connectés aux deux TCD, et le ratio `CA / objectif`. Aucune valeur tapée à la main dans cette feuille.

L4 — Formatage, noms, protection. Formats monétaires FCFA à zéro décimale, pourcentages à une décimale, dates en `JJ/MM/AAAA` ; **cinq noms** au minimum (`Debut_période`, `Fin_période`, `req_ventes_nettoyees`, `TCD_ca_categorie`, `Controle_total`) ; protection de la feuille `Pilotage` avec saisie déloguée là où elle est prévue et TCD autorisés ; structure du classeur verrouillée ; le tout ouvrant sur `Pilotage`.

L5 — Note de cadrage (2 pages, PDF). Ce que le classeur mesure, ce qu'il ne mesure pas, et pourquoi. Trois rubriques obligatoires : **l'échelle** (l'extrait ne couvre que les jours 1 à 4 de chaque mois), **le dénominateur** (CA par ligne, par ticket, par jour ouvré : les trois, avec les chiffres), **la voie** (ce qui est écrit en fonction dynamique et ce qui ne l'est pas, et sur quel poste ça casse). Vous y joignez la **procédure d'actualisation** en cinq lignes max, écrite pour quelqu'un qui ne vous connaît pas.

L6 — Le dossier de rejouabilité. Un classeur vierge de vos feuilles de résultat, reconstruit en une seule actualisation depuis le CSV, avec l'état des contrôles à l'appui : la page d'un tableur montrant `Controle_total` à 0, le compte de lignes de la requête, et le message exact renvoyé si le fichier source a bougé. C'est la partie que le correcteur exécute en premier ; elle vaut même si le reste est imparfait.

Contraintes de rendu

1. Deux fichiers : `2026-MM-JJ_projet_M03_P_votre_nom.xlsx` et `2026-MM-JJ_projet_M03_P_note.pdf`. Le classeur est ouvert sur `Pilotage`, zoom 100 %, aucune sélection en cours.
2. Aucune valeur saisie dans `Pilotage` : tout vient d'un TCD, d'une mesure, ou d'une formule qui lit la feuille `Données`. Aucun chiffre recopié depuis `chiffres_cites.md` sans son calcul à portée de clic.
3. Chaque cellule de contrôle est écrite en équation visible (`=total_TCD - total_source`, attendu 0), pas en commentaire « vérifié ».
4. Les graphiques portent un titre qui dit la granularité et la période, et une source en légende.
5. Si votre poste n'a ni Power Query ni modèle de données (LibreOffice, Excel pour Mac, Excel 2019 sans add-in) : vous livrez la voie accessible **et** vous écrivez en L5 ce que la voie moderne aurait changé. Le barème ne sanctionne pas l'outil, il sanctionne le silence sur l'outil.
6. **Interdits** : macro enregistrée en guise de nettoyage ; mise en forme conditionnelle qui colore un chiffre que rien ne contrôle ; format date appliqué à une colonne de texte sans conversion ; TCD pointant sur une plage figée ; mot de passe présenté comme une confidentialité.

2. Barème détaillé

#	Critère	Pts	Ce qui fait perdre les points
1	L1 : requête en six étapes, paramétrée, rendant 480 lignes justes	4	doublons traités sur le ticket seul ; étapes anonymes ; chemin absolu en dur
2	L2 : huit fiches exécutées, au format imposé, avec leur erreur fréquente	4	fiche descriptive sans formule ; exemple hors jeu de l'atelier ; « cas réel » rédigé en généralité
3	L3 : deux TCD, quatre graphiques, trois segments, ratio à l'objectif	5	segments connectés à un seul TCD ; parts calculées à la main ; graphique héritant d'un filtre caché
4	L4 : formats, cinq noms, protection utilisable	2	TCD inactualisable sous protection ; formats FCFA absents ; aucune plage nommée
5	L5 : note de cadrage — échelle, dénominateur, voie	3	« 27,9 % » écrit sans dire que décembre n'a que quatre jours dans l'extrait
6	L6 : dossier de jouabilité et contrôles à zéro	2	contrôle annoncé mais non écrit ; actualisation qui ne repart pas du CSV
Total		20	seuil de passage en M04 : 13

3. Correction pas à pas (corrigé enseignant)

Ce corrigé est un compte-rendu d'exécution. Les valeurs sont celles du socle livré ; si votre fichier diffère, régénérez le socle avant de vous comparer. Le correcteur ouvre d'abord le classeur, jamais la note.

3.1 — L1 : la préparation attendue, étape par étape

#	Étape de requête	Ce qu'elle doit produire, chiffré
1	Source (chemin lu dans un paramètre)	489 lignes lues, 13 colonnes, types non figés
2	Remplacer . par , puis Charger le type sur les montants	486 des 489 montants arrivaient en texte, 3 seulement en nombre
3	Analyser les dates : 424 en AAAA-MM-JJ, 65 en JJ/MM/AAAA	une colonne date typée, amplitude du 01/01/2025 au 02/12/2025
4	Doublons sur n_ticket + produit, après tri	12 lignes retirées, 481 978 FCFA ; la variante sur le ticket seul retire 173 lignes et 11 424 263 FCFA — erreur de lot complète
5	Filtrer quantité ≤ 500	7 lignes hors échelle, maximum reçu 14 000 contre 68 après correction d'unité
6	Colonne personnalisée est_retour	8 lignes vraies à VRAI, 283 933 FCFA retirés du CA s'ils sont exclus à tort
7	Colonnes de jointure mois et annee_mois	12 valeurs de mois, une clé texte unique par ligne d'objectif

Le chargement attendu est **connexion uniquement + ajout au modèle de données**, avec une table visible dans Données pour l'œil : 480 lignes × 16 colonnes. Le total de contrôle, =SOMME(Données[montant_ttc]), doit rendre 36 073 185 FCFA, soit exactement la somme du fichier nettoyé. Un candidat qui a 36 339 317 FCFA n'a pas retiré les doublons : l'écart est de 266 132 FCFA, et il est écrit dans le corrigé de M01.C06.

Deux points de discrimination, visibles sans ouvrir la note :

1. la colonne **categorie** provient-elle d'une jointure ou d'une recopie ? Le livrable attendu la tire de produits par relation (ou par RECHERCHEX dans la voie accessible), pas d'un collage depuis le CSV reçu — la preuve est dans l'en-tête de la table : produits amène le libellé, pas la vente.

2. **les 103 lignes de client introuvable sont-elles nommées ?** 85 portent le client comptoir 0, 18 n'ont rien : la préparation correcte crée une valeur lisible (« comptoir » ou « non renseigné ») plutôt que de laisser un TCD afficher un blanc en tête de classement.

3.2 — L2 : ce que rendent les fiches exécutées dans **calculs**

Les huit valeurs de contrôle ci-dessous sont celles que la feuille de l'atelier affiche ; le candidat doit les retrouver, pas les recopier.

Fonction	Valeur attendue sur l'atelier	Détail de contrôle
SOMME	36 073 185 FCFA	sur <code>montant_ttc</code> , 480 lignes
MOYENNE	75 152 FCFA	par ligne , à ne pas confondre avec le panier
MEDIANE	37 966 FCFA	la moitié des lignes est en dessous
ECARTYPE.STANDARD	143 915 FCFA	supérieur à la moyenne : aucune moyenne ne représente ce fichier
MAX / MIN	1 229 678 / -150 804 FCFA	le minimum est un retour, pas une erreur de signe
QUARTILE.INC (1, 3)	12 950 / 74 234 FCFA	la moitié centrale tient sur 61 284 FCFA
NB.JOURS.OUVRES	240 jours	du 01/01/2025 au 02/12/2025, fériés non retirés
SOMME.SI.ENS ratio	/ 18,2 %	36 073 185 FCFA sur 198 030 000 FCFA d'objectif magasin 5

La fiche la plus ratée est `NB.SI/NB` sur une colonne texte : le candidat qui compte les montants du fichier reçu avec `=NB()` obtient 0, et beaucoup écrivent dans la fiche « 0 montants, donc un fichier vide ». Le symptôme à documenter est l'inverse : `=SOMME()` sur la même plage rend 0 sans erreur, ce qui est précisément le piège traité en M03.C05. La fiche la mieux notée est celle qui montre un `RECHERCHEX` échouant sur une clé avec espace invisible, et sa correction par `SUPPRES-PACE` — trois lignes, un cas réel, un symptôme.

3.3 — L3 : le tableau de bord, avec ses nombres

Le TCD par catégorie doit rendre, dans cet ordre de classement :

catégorie	lignes	CA (FCFA)	moyenne par ligne	part	lignes remises
Plomberie	57	8 754 982	153 596	24,3 %	22
Matériaux	143	6 519 357	45 590	18,1 %	49
Electricité	53	6 311 645	119 088	17,5 %	21
Bois & panneaux	24	4 844 758	201 865	13,4 %	10
Quincaillerie	112	4 203 121	37 528	11,7 %	45
Peinture	34	2 938 358	86 422	8,1 %	8
Consommables	57	2 500 964	43 877	6,9 %	18
Total	480	36 073 185	75 152	100,0 %	173

Vérifications que le correcteur fait au clavier, dans l'ordre : la ligne de total contre `=SOMME(Données[montant_ttc])` (égalité exigée) ; les lignes 480 contre le compte du tableau ; la part sommant à 100,0 % quand aucun segment n'est cliqué ; la colonne « moyenne par ligne » recalculée en `CA / lignes` catégorie par catégorie, et **non** comme la moyenne des sept moyennes — la seconde écriture vaut 98 281 FCFA et fait perdre le point de L3 en entier, parce qu'elle se présente joliment.

Le TCD par mois, lui, attend douze lignes de 40 lignes chacune (la régularité est celle de l'extrait, pas du magasin) et un CA qui court de 1 258 801 FCFA (mai) à 5 292 517 FCFA (dé-

cembre). Les quatre graphiques attendus : barres horizontales du classement par CA, courbe du CA mensuel avec la glissante à trois mois en deuxième série, barres empilées ou camembert **avec la mention que deux parts voisinent à 0,6 point** (18,1 % et 17,5 %) si le candidat a choisi le camembert, et un quatrième au choix — le top 5 des produits est l'option la plus tenue :

produit	CA (FCFA)	lignes
Robinet mitigeur évier — réf 1	1 976 000	14
Robinet mitigeur évier — réf 2	1 794 026	13
Lame de lambris 4 m — réf 2	1 529 811	11
Lavabo céramique — réf 1	1 485 299	10
Régulateur de charge 10 A — réf 1	1 369 527	12

Les trois segments (ville, catégorie, mois) doivent être connectés aux **deux** TCD : c'est le réglage « Connexions de rapports », et c'est le point de barème le plus souvent perdu en L3, parce qu'il est invisible jusqu'à ce qu'on clique. Le ratio $CA / \text{objectif}$ attend 18,2 % sur l'année de l'extrait, et les valeurs mensuelles les plus hautes autour de 27,9 % en décembre contre 14,1 % en octobre : le bon rendu écrit à côté, dans la même zone, que ce ratio rapporte quatre jours de vente à un mois de cible.

3.4 — L4 : noms, formats, protection

Cinq noms au minimum, et deux sont contrôlés par le correcteur par leur **usage** : `Debut_periode` et `Fin_periode` doivent apparaître dans les critères des `SOMME.SI.ENS` et dans la source des segments de chronologie ; si les dates des graphiques sont figées en dur, le point n'est pas acquis même si les noms existent. `Controle_total` désigne la cellule d'égalité (attendu 0), et `TCD_ca_categorie` la plage du TCD — utile pour l'export et pour la mise en forme conditionnelle, qui ne doit colorer aucun chiffre non contrôlé.

Protection : la feuille `Pilotage` est verrouillée, **les segments restent utilisables**, l'onglet `Données` → `Actualiser` tout fonctionne, la structure du classeur empêche de supprimer `Calculs`. Trois formats à vérifier à l'œil : montants sans décimale avec le code FCFA, parts à une décimale, JJ/MM/AAAA aux deux bornes de période. Le classeur s'ouvre sur `Pilotage` (dernière feuille active enregistrée, ou macro d'ouverture — ici admise, parce qu'elle ne touche pas à la donnée).

3.5 — L5 : la note, et les trois phrases qui font la différence

1. **L'échelle.** « Le fichier d'entraînement couvre les jours 1 à 4 de chaque mois de 2025 : 25,6 % des lignes tombent un samedi ou un dimanche, 40 lignes par mois exactement, 240 jours ouverts sur 336 jours courus. Aucune saisonnalité ne peut être affirmée sur cette base, et le ratio à l'objectif non plus. » Le correcteur cherche la mention des quatre jours : sans elle, deux points sont perdus, avec elle le reste se discute.
2. **Le dénominateur.** 75 152 FCFA par ligne, 114 156 FCFA par ticket, 150 305 FCFA par jour ouvré, 107 361 FCFA par jour couru : quatre chiffres justes, et la note doit dire lequel elle affiche et pourquoi. La mention « par jour travaillé » sans précision des week-ends (9 730 140 FCFA de CA, 27,0 % de la masse) est comptée comme une absence.
3. **La voie.** Ce qui est écrit en fonctions dynamiques (`FILTRE`, `UNIQUE`, `TRIERPAR`, `SEQUENCE`, `LET`), ce qui est en voie accessible, et ce qui se passe sur les postes du service : `#NOM?` en Excel 2019 pour les dynamiques, pas de propagation sur LibreOffice 24.8 et antérieurs sans pré-allocation de la plage, pas de modèle de données sur Excel pour Mac (donc pas de « Nombre distincts »), pas de segments du tout dans Calc. Le correcteur attend la phrase qui dit lequel des deux il a livré et pourquoi, pas une liste d'excuses.

3.6 — L6 : la rejouabilité, testée comme elle le sera

Le correcteur fait ceci, en trois minutes : il ouvre le classeur, supprime la feuille `Données` et les TCD, clique `Données` → `Actualiser tout`, puis vérifie quatre nombres — 480 lignes, 36 073 185 FCFA, 12 mois, 0 à la cellule de contrôle. Quatre défauts type, et leur traduction en points :

Défaut observé	Diagnostic	Conséquence au barème
l'actualisation redemande le chemin du fichier	paramètre <code>absent</code> , chemin en dur dans l'étape Source	L1 amputé de 2 pts, L6 de 1
le TCD revient vide	la source pointait une plage supprimée avec la feuille	L6 à 0, L3 plafonné à 2
le total revient à 36 339 317 FCFA	les doublons étaient retirés à la main dans la feuille, pas dans la requête	L1 à 1, L6 à 1
le ratio d'objectif passe à « #DIV/0! »	la jointure <code>annee_mois</code> a été perdue en rechargeant le CSV	L3 à 2, L5 amendée

Un dossier qui montre le message d'erreur attendu quand le fichier source a bougé (« chemin introuvable », avec la correction en deux lignes) vaut le point entier de L6 : c'est la seule preuve que le livrable est compris et pas seulement produit.

4. Grille d'auto-évaluation avant rendu

Répondez par oui ou non ; trois « non » = ne rendez pas, reprenez la feuille concernée.

#	Question	Si non
1	La requête repart-elle bien du CSV brut et rend-elle 480 lignes ?	reprendre M03.C08 §5.5
2	Les doublons sont-ils retirés sur <code>n_ticket</code> et produit, après un tri explicite ?	reprendre M03.C03 §5.4 et M03.C08 ex. 8.4
3	Les montants sont-ils numériques dans la table finale (<code>=NB()</code> = 480) ?	reprendre M03.C02 §5.2
4	La colonne date est-elle une série, pas du texte (<code>=MOIS()</code> fonctionne) ?	reprendre M03.C07 §5.1
5	Les huit fiches sont-elles exécutées, pas décrites ?	reprendre le format de fiche, §3 des chapitres
6	Chaque TCD affiche-t-il un total égal à la somme de la colonne source ?	reprendre M03.C08 §5.1
7	Les trois segments sont-ils connectés aux deux TCD ?	reprendre M03.C08 §5.3
8	Le graphique reprend-il les filtres cachés du TCD au moment de sa création ?	reprendre M03.C08 §5.4
9	Les parts sont-elles en « Total général en % » et non calculées à la main ?	reprendre M03.C08 §5.1
10	Le ratio à l'objectif est-il accompagné de la mention d'échelle ?	reprendre M03.C07 §6
11	La protection laisse-t-elle actualiser et cliquer ?	reprendre M03.C08 §5.8
12	Un autre poste peut-il ouvrir le classeur sans <code>#NOM?</code> ni plage vide ?	écrire la voie alternative, M03.C07 §5.7

5. Ce que le correcteur regardera en premier

1. **Le compte de lignes de la requête, à l'étape 4.** 480, pas 468 ni 316 : c'est la ligne qui dit si le doublon a été compris comme un défaut de fichier ou comme une règle métier.

2. **La cellule de contrôle.** `Controle_total` doit être une formule, pas un 0 écrit. Un zéro tapé est un aveu, pas une preuve.
3. **Le segment qui ne fait rien.** Premier clic sur un segment : si un seul des deux TCD bouge, le candidat n'a pas ouvert « Connexions de rapports », et le tableau de bord est décoratif.
4. **La phrase d'échelle.** Dans la note ou sous le graphique, « les jours 1 à 4 de chaque mois » doit se lire. C'est le seul endroit du projet où l'honnêteté du module M02 est vérifiable dans un livrable Excel.
5. **La liste des voies.** Un classeur qui n'utilise que des fonctions que deux postes sur cinq peuvent exécuter n'est pas un livrable, c'est une démonstration. À l'inverse, un classeur 100 % accessible avec une ligne qui dit ce que `FILTRE` aurait économisé est noté comme un travail de professionnel.

Projet M04.P — « Le dossier de données de la quincaillerie de Koudougou »

Module M04 · projet de fin de module · 6 h · barème /20, seuil de passage 13/20

Le cadre réel. Un gérant de quincaillerie vous a remis un dossier tel qu'il le tient dans son poste : un fichier de commandes de 160 lignes, un fichier de clients, un tarif fournisseur « généré par son logiciel », et une phrase : « je sais que c'est mal tenu, mais les totaux du mois doivent sortir lundi ». Ce dossier contient **sept défauts de natures différentes**, dont **deux qui cassent les totaux** — pas plus, pas moins. Vous rendez la boîte en état de rejeu : un diagnostic chiffré, un fichier réparé dont on sait d'où viennent les nombres, un journal, un README, un contrôle final qui sait ce qui doit rester, et un périmètre de protection pour les clients. C'est la synthèse des six chapitres : chaque livrable en mobilise un, et aucun n'est noté pour lui-même.

Note de cohérence. Le dossier est généré de façon déterministe par `python3 tools/dossier_M04.py` (graine 41) : qui que vous soyez, vous touchez les mêmes 160 lignes. `03_exercices/dossier_M04/ATTENDU.json` contient les résultats attendus, **mesurés sur le dossier généré, jamais déduits** : vous n'y recourez qu'après avoir produit vos propres nombres, pour comparer, pas pour copier. Un candidat qui lit l'ATTENDU avant de calculer n'a plus rien à rendre. Les chiffres de ce corrigé sont ceux du dossier livré, vérifiables ligne à ligne.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Commandes	<code>03_exercices/dossier_M04/commandes_2026.csv</code>	160 lignes, 20 colonnes, encodage UTF-8 avec BOM, séparateur virgule
Clients	<code>03_exercices/dossier_M04/clients_2026.csv</code>	132 lignes, 11 colonnes, UTF-8 avec BOM
Tarif fournisseur	<code>03_exercices/dossier_M04/tarif_fournisseur.csv</code>	10 lignes utiles, séparateur point-virgule, encodage Windows-1252, en-tête décalé
Résultats attendus	<code>03_exercices/dossier_M04/ATTENDU.json</code>	à utiliser après le calcul, jamais comme source de copie
Outils	un tableur (Excel 2021+ ou LibreOffice Calc) ; sinon : la voie accessible, et vous l'écrivez Power Query si votre poste l'a	

Le dossier contient sept défauts de natures différentes ; deux d'entre eux faussent le total de la somme directe, les cinq autres faussent une analyse, une jointure ou un import — sans jamais changer un seul montant. Trouver les sept, les séparer en deux natures, et prouver la séparation, c'est le premier tiers de la note.

Les six livrables

D1 — La grille du dossier (3 points). La grille des 12 points (C01), passée sur `commandes_2026.csv` et `clients_2026.csv` : pour chaque point non nul, le compteur **avec son dénominateur** (« 17 sur 160 », jamais « 17 »). Vous identifiez les sept défauts, vous les classez en casse un total / fausse une analyse, une jointure ou un import, et vous écrivez la phrase qui prouve la

séparation : quels sont les deux défauts qui faussent la somme directe, et pourquoi les cinq autres ne la faussent pas.

D2 — Le fichier réparé et son total (4 points). `commandes_2026_propres.csv` : 152 lignes, total TTC **8 209 940 FCFA** — à obtenir par votre méthode, pas à viser. Les 8 lignes retirées s'écrivent dans la feuille de travail avec leur motif : 7 copies exactes (retirées sur la règle du contenu) et 1 **resaisie** — une ligne jumelle qui ne diffère de son original que d'un champ (la date) et porte le plus grand `id_vente` ; le piège de l'énoncé, c'est que le fichier compte **9 groupes en collision** : le neuvième est un ticket multi-lignes **légitime** (deux lignes qui diffèrent sur l'heure, le client, le prix et le mode de paiement) et ses 2 lignes doivent survivre. Une règle qui ne garde qu'une ligne par groupe (le plus petit `id_vente`, sans regard) tombe à 151 lignes et 8 105 333 FCFA : vous écrivez ce que ce total a mangé.

D3 — Le journal des transformations (3 points). Cinq lignes au format du C06 (date, fichier, opération, règle, compteur avant → après), couvrant au minimum : la conversion des 17 montants texte (dont 2 négatifs qui sont des retours), le retrait des 8 doublons, la reconstruction des 12 dates hors norme, et le retrait des 2 quasi-doublons clients. Chaque ligne se referme : avant – après = retirés ou modifiés.

D4 — Le README du dossier et le dictionnaire des trois fichiers (3 points). Six sections, 56 lignes de cible (C06) : scénario, tables avec leur grain, règles de gestion (retour = montant négatif ; `id_client = 0` = comptoir, **pas** une donnée à remplir — 18 lignes dans le dossier), défauts constatés, ce qui manque volontairement. Le dictionnaire déclare pour chaque fichier : encodage (les deux CSV du dossier portent un BOM ; le tarif est en Windows-1252 — un import en Latin-1 y insère des caractères de contrôle invisibles), séparateur (; pour le tarif), position de l'en-tête (ligne 4 pour le tarif, 3 lignes de bruit avant), et le type de chaque clé. Les 10 prix du tarif sont à virgule décimale : le README dit comment les lire sur une machine à point.

D5 — Le contrôle final avec ses résidus attendus (3 points). La grille de 12 points sur le fichier réparé, avec les résidus attendus **écrits avant** de lancer le contrôle (C06) : 0 montant non numérique, 0 remise hors [0 ; 1] — à condition d'avoir traité les 5 remises en points, sinon 5 résidus —, 0 date hors norme ISO, 0 groupe de doublon, 1 groupe multi-lignes légitime à 2 lignes. La convergence est écrite avec l'écart : votre total 8 209 940 FCFA contre l'ATTENDU.json, écart 0 — l'écart est le chiffre, pas le total.

D6 — Le périmètre de protection de `clients_2026.csv` et sa version pseudonymisée (4 points). Vous délimitez les données personnelles (nom, téléphone : 132 lignes ; e-mail : les lignes renseignées ; ville, date de création), vous mesurez ce que l'effacement des identifiants directs ne protège pas, et vous produisez `clients_2026_anonymes.csv` : `id_client` figée et remplacée par un pseudonyme stable, nom/téléphone/e-mail remplacés, **part de ré-identification résiduelle mesurée et écrite**. Le dossier compte 12 noms portés par deux téléphones ou plus (homonymes) : vous écrivez pourquoi « dédoubler sur le nom » y retirerait des clients réels, et pourquoi la règle qui retire exactement les 2 quasi-doublons (132 → 130) est la clé nom normalisé + chiffres du téléphone — pas le nom seul.

2. Ce qui est noté, ce qui est piégé

Piège du dossier	Où il se cache	Ce qu'il enseigne
17 montants en texte (« n nnn FCFA »), dont 2 négatifs	<code>commandes_2026.csv</code> , colonne N	la somme directe rate 17 lignes ; les 2 négatifs sont des retours que le contrôle de domaine ne voit qu'après conversion
8 lignes en doublon (7 copies + 1 resaisie)	<code>commandes_2026.csv</code>	la resaisie n'est pas une copie exacte : la règle « contenu identique » ne la voit pas, la règle « un seul champ différent » la voit

Piège du dossier	Où il se cache	Ce qu'il enseigne
1 ticket multi-lignes légitime (2 lignes, 8 colonnes différentes)	<code>commandes_2026.csv</code>	« une ligne par groupe » mange une vente réelle (8 105 333 au lieu de 8 209 940)
5 remises en points de pourcentage (12/18/25/30/18)	<code>commandes_2026.csv</code> , colonne K	la division par 100 est un test à faire, pas une réparation à appliquer
12 dates en deux écritures (JJ/MM/AAAA et AAAA-MM-JJ)	<code>commandes_2026.csv</code> , colonne C	le tri chronologique et le pivot mensuel sont faux tant que les deux écritures cohabitent
2 quasi-doublons clients (nom surchargé + téléphone sans espaces) + 12 homonymes	<code>clients_2026.csv</code>	la règle « même nom » retire bien plus que les 2 doublons ; la clé juste est nom normalisé + chiffres
3 lignes de bruit, cp1252, ;, décimales à virgule	<code>tarif_fournisseur.csv</code>	l'import par défaut lit 1 colonne, un en-tête faux, et des prix en texte

3. Barème et seuil

- D1 : 3 points — grille complète avec dénominateurs (1) · les sept défauts en deux natures (1) · la phrase de séparation des totaux (1).
- D2 : 4 points — 152 lignes et 8 209 940 FCFA obtenus (1,5) · les 8 lignes retirées motivées ligne à ligne (1) · le groupe légitime survit et l'erreur de la règle naïve est chiffrée (1,5).
- D3 : 3 points — cinq lignes au format complet (1) · chaque ligne se referme par son écart (1) · la conversion des montants se referme par un contrôle de convergence (1).
- D4 : 3 points — six sections lisibles sans ouvrir les fichiers (1) · encodages, séparateurs et en-têtes déclarés (1) · la convention `id_client = 0` écrite comme convention et non comme donnée manquante (1).
- D5 : 3 points — résidus attendus écrits avant le contrôle (1) · grille du réparé avec le groupe légitime nommé (1) · convergence écrite avec l'écart contre l'ATTENDU (1).
- D6 : 4 points — périmètre délimité et chiffré (1) · pseudonymisation stable qui ne casse aucune jointure (1) · part de ré-identification résiduelle **mesurée et écrite** (1) · la règle des 2 quasi-doublons opposée à la règle du nom seul, avec les 12 homonymes en argument (1).

Seuil de validation : 13/20. Un livrable dont un compteur manque son dénominateur perd 0,5 point par compteur, sans plafond : la précision du dénominateur est la matière du module, pas une coquille.

4. Correction des points d'attention

- **D1, séparation des totaux.** La somme directe sur la colonne N ne rate que les lignes non numériques : 17. Les doublons la faussent en double comptant 8 lignes. Les 5 remises, les 12 dates, les 2 quasi-doublons et l'encodage du tarif ne changent aucun montant : ils faussent une analyse (remises), un calendrier (dates), une jointure (clients) et un import (tarif). La phrase attendue : « Deux défauts cassent le total — 17 montants en texte non sommés et 8 lignes comptées deux fois ; les cinq autres laissent la somme intacte et cassent chacune une autre lecture du fichier. »
- **D2, le groupe légitime.** Le groupe formé du ticket `T01-230101-000140`, produit 74, quantité 3 : deux lignes qui diffèrent sur l'heure, le vendeur, le client, le prix, les trois montants et le mode de paiement. C'est une double vente réelle du même ticket, pas un doublon. La règle naïve « une ligne par groupe, plus petit `id_vente` » la réduit à 1 ligne : 151 lignes, 8 105 333 FCFA — l'écart 104 607 FCFA est le montant de la ligne mangée.
- **D2, la resaisie.** Le groupe du ticket `T01-230103-000122`, produit 103, quantité 3 : deux lignes identiques sauf la date. Ce n'est pas une copie exacte (le contrôle de contenu la laisse pas-

ser) ; c'est une ressaisie avec une erreur de date, reconnaissable à son `id_vente` plus grand. La règle de garde la retire sans toucher au groupe légitime, qui diffère sur 8 colonnes.

- **D5, les résidus attendus.** « 0 non numérique, 0 hors domaine, 0 hors norme, 0 groupe de doublon, **1 groupe multi-lignes légitime à 2 lignes** » — le résidu est écrit avant le contrôle ; un contrôle qui ne s'attendrait qu'à des zéros rejetterait le bon fichier.
- **D6, la part résiduelle.** Elle se mesure sur les colonnes conservées (ville, type, segment, date de création, conditions de paiement, plafond) : sur ce dossier, elle se compte en combinaisons désignant un client unique. L'e-mail, quand il est renseigné, encode souvent l'identifiant : le pseudonyme stable doit exister **avant** l'effacement, sinon la jointure avec `commandes_2026.csv` se rompt.
- **Le tarif.** Lu en Windows-1252 avec 3 lignes sautées : 10 lignes, 6 colonnes, 10 prix à virgule décimale. Lu en UTF-8, il casse à l'octet ; lu en Latin-1 sans saut, il donne une colonne et un en-tête faux. Le README déclare les trois réglages ; la formule de lecture des prix est celle qui énonce la virgule, pas celle qui l'hérite de la locale.

Projet M05.P — « Trois chemins, une même table propre »

Module M05 · projet de fin de module · 6 h · barème /20, seuil de passage 13/20

Le cadre réel. Un data analyst d'une enseigne de distribution à Ouagadougou reçoit chaque matin trois versions du même fichier de ventes (une de la caisse, une du back-office, une de l'intégration comptable), et la question qui revient tous les lundis est : « pourquoi vos trois chiffres ne tombent pas juste ? » Vous rendez la même table propre dans **trois moteurs** (Power Query, SQL DuckDB, pandas), vous prouvez qu'ils produisent la **même** empreinte sha256, et vous documentez le **choix d'outil** par les 4 critères du C05 §7. C'est la synthèse des cinq chapitres : chaque livrable en mobilise un, et aucun n'est noté pour lui-même.

Note de cohérence. Le dossier est généré de façon déterministe par `python3 tools/dossier_M05.py` (graine 42) : qui que vous soyez, vous touchez les mêmes 121 720 lignes de ventes. `03_exercices/dossier_M05/ATTENDU.json` contient les **31 clés** des résultats attendus, **mesurés sur le dossier généré, jamais déduits** : vous n'y recourez qu'après avoir produit vos propres nombres, pour comparer, pas pour copier. Un candidat qui lit l'ATTENDU avant de calculer n'a plus rien à rendre. Les chiffres de ce corrigé sont ceux du dossier livré, vérifiables ligne à ligne.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Ventes	<code>03_exercices/dossier_M05/ventes_2023_2024.csv</code>	121 720 lignes, 20 colonnes, encodage UTF-8, séparateur virgule, 9 défauts à 9 endroits (texte, doublons, dates, jointure, encodage, taille, retours, remise, transposition invisible)
Remises	<code>03_exercices/dossier_M05/remises_2023_2024.xlsx</code>	9 942 lignes, 7 colonnes, Excel <code>.xlsx</code> (openpyxl) ; 35 couples ressaisis sur 9 907 uniques
Clients	<code>03_exercices/dossier_M05/clients.csv</code>	23 500 lignes, 9 colonnes, CSV simple ; 252 clients inconnus côté ventes
Résultats attendus	<code>03_exercices/dossier_M05/ATTENDU.json</code>	31 clés — à utiliser après le calcul, jamais comme source de copie
Outils	trois moteurs — au moins deux obligatoires, trois pour la note pleine	Power Query (si Excel), SQL DuckDB (Python), pandas (Python)

Le dossier contient **neuf défauts de natures différentes** ; **quatre** d'entre eux cassent le total `total_ttc_avant_dedoublonnage` (avant réparation), les **cinq** autres cassent soit une analyse (remises hors domaine), soit une jointure (clients inconnus), soit un import (taille 28 Mo, encodage), soit une régression (lignes légitimes mangées par une clé trop agressive). Trouver les neuf, les réparer dans **deux moteurs** indépendants, prouver que les deux tables sont identiques au byte près (sha256), et expliquer dans quel cas le troisième moteur est nécessaire : c'est la moitié de la note. L'autre moitié est la **trace écrite** : le journal, le README, et le verdict de la convergence.

Les six livrables

D1 — La grille de diagnostic (3 points). La grille des 12 points du M04.C01, passée sur `ventes_2023_2024.csv`. Pour chaque point non nul, le compteur **avec son dénominateur** (« 1 731 sur 121 720 », jamais « 1 731 »). Vous identifiez les 4 défauts qui cassent le total et les 5 qui ne le cassent pas, et vous écrivez la phrase qui prouve la séparation (cf. C01 §5.1).

D2 — Le pipeline de réparation (4 points). La séquence canonique du C01 §5.5 appliquée au brut donne un DataFrame final `df_final` à **120 000 lignes** (cible `lignes_final`), **total TTC 7 145 910 735 FCFA** (cible `total_ttc_final`). Cette cible est mesurée sur le dossier généré, pas visée aveuglément. Les 1 720 doublons retirés (`doublons_a_retirer`) s'écrivent dans le journal avec leur motif : 1 680 copies exactes (5 colonnes identiques : ticket + produit + quantité + TTC + heure — `ventes_copies_ressaisies`) et 40 variantes qui ne modifient qu'un seul champ (la date — `ventes_ressaisies_date`). Le piège est qu'une règle « `drop_duplicates` » sans tri préalable rate la moitié des défauts, et qu'une règle trop agressive (« clé = `id_ticket` seul », sans regarder `id_produit`, `quantité`, `montant_ttc` ou `heure`) mange 3 lignes légitimes (`lignes_legitimes_mangees_cle_sans_heure` dans l'ATTENDU) et tombe à un total inférieur à la cible — vous écrivez ce que cette règle a mangé, et vous chiffrez l'écart.

D3 — Les trois pipelines et leurs verdicts (4 points). Trois exécutions, une empreinte sha256 unique : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465`. Vous rendez : - (a) la requête SQL DuckDB **exécutée** sur le brut (vérifiable par `python3 tools/controle_sql.py M05`) ; - (b) le notebook pandas **exécuté** sur le brut (vérifiable par `python3 tools/controle_python.py M05`) ; - (c) le script Power Query **documenté** (M code étape par étape, non exécutable dans cet atelier — c'est la limite de la règle 7 du plan M05 §3).

La mesure est : `df_final` pandas ET la table DuckDB, sérialisés en Parquet avec `sort_values(id_vente)`, produisent la même empreinte sha256. Le candidat qui n'a qu'un seul moteur ne peut pas produire cette preuve : il rend une table propre, mais il ne prouve pas la robustesse.

D4 — Le journal des transformations et le README du dossier (3 points). Six lignes au format du C06 (date, fichier, opération, règle, compteur avant → après), couvrant au minimum : (1) la conversion des 1 731 montants texte ("1 200 FCFA" → 1200), (2) la transposition des 493 dates jour-mois (`dates_transposees_reparees`), (3) le dédoublonnage 1 720 (1 680 copies exactes + 40 variantes de date), (4) la jointure clients avec clé complète (1 714 enrichies `lignes_jointure_cle_complette` contre 41 716 naïves `lignes_jointure_naive_id_client`), (5) l'export Parquet (empreinte sha256 notée), (6) le verdict croisé (les 3 moteurs produisent la même empreinte). Le README (4 sections, 40 lignes de cible) : scénario, tables avec leur grain, défauts constatés, ce qui manque volontairement. Il justifie **par les 4 critères du C05 §7** (taille, fréquence, compétences, gouvernance) le choix de DuckDB en première intention et pandas en repli.

D5 — Le contrôle final avec ses résidus attendus (3 points). La grille des 12 points du M04.C01 sur `df_final`, avec les **résidus attendus écrits avant** le contrôle : 0 montant non numérique, 0 remise hors [0 ; 0,3] (à condition d'avoir traité les 488 remises en points `remises_hors_domaine`), 0 date hors plage [2023-01-01 ; 2024-12-31], 0 groupe de doublon, 0 client inconnu (252 ramenés à 0 par la jointure clé complète). La convergence est écrite avec l'écart : votre total `7 145 910 735 FCFA` contre l'ATTENDU.json `total_ttc_final`, écart 0. **L'écart est le chiffre à énoncer**, pas le total.

D6 — L'arbitrage DuckDB / pandas / Power Query pour le dossier (3 points). Pour les 4 critères (taille, fréquence, compétences, gouvernance) du C05 §7, vous écrivez le verdict **de votre équipe** (qui n'est pas forcément celle de l'énoncé) et la **règle de bascule** : « si le critère X change de valeur, on change d'outil et on documente le changement ». Le piège est de justifier le choix sans l'avoir **mesuré** : un test pandas + DuckDB sur 121 720 lignes est attendu (les chiffres du C05 §5.3 sont là pour ça — pandas ≈ 1,4 × DuckDB sur 120 000 lignes).

2. Ce qui est noté, ce qui est piégé

Piège du dossier	Où il se cache	Ce qu'il enseigne
1 731 montants en texte (« 1 200 FCFA »)	ventes_2023_2024.csv, (montants_en_texte)	colonne M la somme directe rate 1 731 lignes ; la conversion par TRY_CAST Pandas/Power Query ne suffit pas, il faut d'abord normaliser l'espace et la devise
1 720 doublons (1 680 exacts + 40 variantes)	ventes_2023_2024.csv (doublons_a_retirer)	les doublons apparaissent après conversion : avant, « 1 200 FCFA » ≠ « 1200 » ; les 40 variantes ne diffèrent que d'un champ (la date), pas 5 — la règle « drop_duplicates sur 5 colonnes » les attrape
493 dates transposées (jour-mois)	ventes_2023_2024.csv, (dates_transposees_reparees)	colonne C le tri chronologique et la coupe à la date sont faux tant que les deux écritures cohabitent ; c'est l'objet du M04 erratum §7 (945 sur tout le fichier, 493 dans la fenêtre projet)
252 clients inconnus (id_client ≥ 900 000)	ventes_2023_2024.csv (jointure) clients.csv	la jointure naïve sur id_client donne 41 716 enrichissements faux (lignes_jointure_naive_id_client) au lieu de 1 714 (lignes_jointure_cle_complette) — un facteur 24 x
488 remises hors domaine (saisies 12/18/25...)	ventes_2023_2024.csv, (remises_hors_domaine)	colonne K la division par 100 est un test, pas une réparation universelle
1 510 retours (montants négatifs)	ventes_2023_2024.csv (retours)	la conversion les fait passer en valeurs négatives ; le filtre « montant < 0 » les isole
Encodage UTF-8 + 121 720 lignes (≈ 14 Mo)	tout le fichier	Excel ouvre le fichier mais le recalcule sur 14 Mo est lent (> 30 s) — borderline par rapport à la règle 100k de C05 §5.1
3 lignes légitimes perdues si clé = id_ticket seul	ventes_2023_2024.csv (lignes_legitimes_mangees_cle_sans_heure)	la clé juste est 5 colonnes : id_ticket + id_produit + quantité + TTC + heure ; sans l'heure, deux lignes du même ticket (heures différentes) sont mangées
3 défauts non visibles au format (transpositions pures)	ventes_2023_2024.csv colonne C	le contrôle de format passe, le contrôle de valeur échoue — c'est la leçon de l'E4 de M04

3. Barème et seuil

- **D1** : 3 points — grille complète avec dénominateurs (1) · les 9 défauts en 2 natures (1) · la phrase de séparation des totaux (1).
- **D2** : 4 points — total 7 145 910 735 FCFA obtenu par votre méthode (1,5) · les 1 720 doublons motivés ligne à ligne (1) · le verdict try_cast() vs TRY_CAST SQL (1,5).
- **D3** : 4 points — pandas **et** DuckDB exécutés, sha256 identique (2) · Power Query documenté mais non exécutable, comparaison par le total (1) · le verdict croisé (1).
- **D4** : 3 points — six lignes au format complet (1) · chaque ligne se referme par son écart (1) · README
- arbitrage par les 4 critères (1).
- **D5** : 3 points — résidus attendus écrits avant le contrôle (1) · grille du df_final avec convergence écrite (1) · écart 0 contre total_ttc_final (1).

- **D6** : 3 points — les 4 critères **mesurés** sur le dossier (1) · règle de bascule explicite (1) · comparaison avec les 5 chiffres du C05 §5.3 (1).

Seuil de validation : 13/20. Un livrable dont un compteur manque son dénominateur perd 0,5 point par compteur, sans plafond : la précision du dénominateur est la matière du module, pas une coquille.

4. Correction des points d'attention

- **D1, séparation des totaux.** La somme directe sur la colonne M convertie rate ~1 731 lignes sur le brut (`montants_en_texte`), soit 0 sur le total de référence ; après conversion, le total `total_ttc_avant_dedoublonnage` = 7 242 884 579 FCFA. Les doublons la faussent en double comptant 1 680 copies (les 40 variantes se neutralisent par les conversions). Les 493 dates transposées ne touchent pas au total TTC mais déplacent 493 lignes entre mois — la coupe à la date est alors fausse, mais le total global est juste. Les 252 clients inconnus ne touchent pas au total TTC mais faussent la jointure. La phrase attendue : « Quatre défauts touchent au total — 1 731 montants en texte non sommés, 1 720 doublons dont 1 680 copies exactes, 252 clients inconnus que la jointure naïve gonfle, et 493 dates transposées qui déplacent des lignes entre mois ; les cinq autres — 488 remises en points, 1 510 retours, encodage, 121 720 lignes, 3 lignes légitimes mangées par une clé trop courte — faussent chacun une lecture différente du fichier sans toucher à la somme globale. »
- **D2, le pipeline canonique.** C'est la leçon du C01 §5.5 : convertir **avant** de dédoubler, parce que « 1 200 FCFA » et 1200 sont égaux après conversion et différents avant. La règle de dédoublonnage qui s'applique sur la colonne M non convertie rate la moitié des doublons. La séquence qui passe le contrôle du contrôleur est : (1) `assign(montant_ttc_n = TRY_CAST(...))`, (2) `sort_values(['date_vente', 'id_vente'])`, (3) `drop_duplicates(CLE, keep='first')` avec `CLE = ['id_ticket', 'id_produit', 'quantite', 'montant_ttc_n', 'heure']`. Réciproquement en SQL : `TRY_CAST` avant `ROW_NUMBER OVER (PARTITION BY CLE)`.
- **D2, le piège des 3 lignes légitimes.** La règle naïve « clé = `id_ticket` seul » (sans `id_produit`, sans quantité, sans `montant_ttc_n`, sans heure) tombe à un total inférieur à la cible : les 3 lignes mangées sont des ventes multi-lignes légitimes (même ticket, produits différents, heures différentes) — c'est l'objet de la clé `lignes_legitimes_mangees_cle_sans_heure` = 3 dans l'ATTENDU. La clé juste doit inclure **les 5 colonnes** pour survivre à ces cas. La chute est petite en valeur absolue, mais c'est l'écart caché : l'apprenti n'a aucun moyen de voir qu'il a perdu 3 lignes sans recourir à l'ATTENDU.
- **D3, le verdict sha256.** L'empreinte `7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` est mesurée sur la **sérialisation Parquet** de `df_final.sort_values(['id_vente'])`. Pandas et DuckDB la produisent à l'identique au byte près (vérifié par `controle_python.py` et `controle_sql.py`). Power Query est exclu parce que l'atelier ne peut pas exécuter Excel — la note tolère cette absence à condition que le **total** 7 145 910 735 FCFA soit annoncé et qu'il soit cohérent avec celui de pandas et DuckDB (cohérent au centime près).
- **D5, les résidus attendus.** « 0 non numérique, 0 hors domaine, 0 hors plage, 0 groupe de doublon, 0 client inconnu » — les cinq zéros sont écrits **avant** le contrôle. Un contrôle qui s'attendrait à autre chose rejette le bon fichier. La convergence s'écrit avec l'écart (0) — pas avec le total répété.
- **D6, la règle de bascule.** « Si le fichier dépasse 500 000 lignes, on bascule sur DuckDB ; si l'équipe ne fait pas de Python, on bascule sur Power Query ; si la gouvernance exige un schéma SQL centralisé, on bascule sur DuckDB ». Trois règles, trois critères, et c'est cette réversibilité qui fait que le choix n'est pas une opinion, c'est une **décision**.

- **L'arbitrage par les mesures du C05 §5.3.** Sur 121 720 lignes (projet), DuckDB et pandas sont à $\approx 1,4 \times$ l'un de l'autre (DuckDB légèrement plus rapide). Sur 740 lignes (mars 2025 du brut, avant fenêtrage), pandas reste largement plus rapide (DuckDB pâtit du chargement initial). Le **choix** DuckDB/pandas ne dépend donc pas de la taille seule, mais de la prévision : si le fichier grossit vers 500k+ lignes, DuckDB reste meilleur ; si le fichier reste à 100k lignes, pandas est plus rapide à froid. La règle du module est DuckDB en première intention (portable, plus rapide à l'échelle), pandas en repli (plus riche en méthodes, plus rapide à froid).

Projet M06.P — « Sept tables pour une quincaillerie : passer du classeur à la 3FN »

Module M06 · projet de fin de module · 6 h · barème /20, seuil de passage 13/20

Le cadre réel. Une enseigne de quincaillerie à Bobo-Dioulasso vous confie son fichier `quincaillerie_export.csv` : 24 000 lignes, 42 colonnes, **4 défauts** à 4 endroits (montants en texte, doublons exacts, dates inversées, clients orphelins). Cinq patrons du métier vous attendent dans la salle de pause et chacun a sa vérité : « Excel suffit », « on a essayé le SGBD, c'est trop compliqué », « NoSQL sera plus rapide », « on garde tout dans un seul fichier, c'est plus simple ». Vous rendez **5 livrables** qui ferment le débat par les faits : un schéma en 3FN, un script de chargement DuckDB, un script SQLite de relecture, une note PostgreSQL cité-sans-exécuté, et un verdict structuré. C'est la synthèse des cinq chapitres : chaque livrable en mobilise un, et aucun n'est noté pour lui-même.

Note de cohérence. Le dossier est généré de façon déterministe par `python3 tools/dossier_M06.py` (graine 43) : qui que vous soyez, vous touchez les mêmes 24 000 lignes et la même empreinte `f296a11c...64c8137`. `03_exercices/dossier_M06/ATTENDU.json` contient les **18 clés** des résultats attendus, **mesurés sur le dossier généré, jamais déduits** : vous n'y recourez qu'après avoir produit vos propres nombres, pour comparer, pas pour copier. Un candidat qui lit l'ATTENDU avant de calculer n'a plus rien à rendre. Les chiffres de ce corrigé sont ceux du dossier livré, vérifiables ligne à ligne.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Export brut	<code>03_exercices/dossier_M06/quincaillerie_export.csv</code>	24 000 lignes, 42 colonnes, encodage UTF-8, séparateur virgule, 4 défauts : 142 <code>montant_ttc</code> en texte (avec « FCFA » et espaces), 8 doublons exacts, 12 dates où <code>date_fin < date_debut</code> , 3 <code>id_client</code> sans correspondance dans <code>client</code>
Schéma 3FN cible	<code>03_exercices/dossier_M06/schema_3fn.sql</code>	7 tables : <code>magasin</code> , <code>mode_paiement</code> , <code>regle_tva</code> , <code>categorie</code> , <code>client</code> , <code>produit</code> , <code>vente</code> . PK + FK + types stricts
Contrôles d'intégrité	<code>03_exercices/dossier_M06/controles_integrite.sql</code>	7 requêtes : comptages, jointures externes NULL, totaux TTC 2025 et 2026, défauts résiduels = 0
Résultats attendus	<code>03_exercices/dossier_M06/ATTENDU.json</code>	18 clés <code>m06p_*</code> : empreinte sha256, totaux TTC, listes des défauts

Les cinq livrables, et ce qu'ils notent

#	Livrable	Format	Ce que ça mobilise	Barème
L1	Modèle 3FN en 7 tables	<code>schema_3fn.sql</code>	C02 + C03	/5
L2	Chargement DuckDB exécuté	<code>load_duckdb.sql</code> + sortie capturée	C02 + C04	/5
L3	Relecture SQLite exécutée	<code>load_sqlite.py</code> + sortie capturée	C04	/3
L4	Note PostgreSQL cité-sans-exécuté	<code>note_postgres.md</code>	C04	/2
L5	Verdict structuré paysage	<code>verdict_paysage.md</code>	C05	/5

Règle M06 §1.5. DuckDB et SQLite **doivent être exécutés** ; PostgreSQL **cité sans exécuté**. Tout candidat qui présente un PostgreSQL exécuté perd 2 points sur L4 (la règle est explicite dans le plan). Inversement, présenter DuckDB ou SQLite cité sans exécuté fait perdre 3 points sur L2 ou L3.

2. Corrigé-type (à publier après la séance)

Ce corrigé n'est pas un modèle à recopier : c'est la **référence** à laquelle vous comparez vos propres nombres. Si vos totaux ou vos listes de défauts ne tombent pas à 1 ligne près, c'est que votre code a un défaut — pas que la référence a tort.

L1 — schema_3fn.sql (extrait représentatif, 7 tables)

```
-- 1. Tables de référence (1FN, sans doublons)
CREATE TABLE magasin (
  id_magasin  INTEGER PRIMARY KEY,
  nom         VARCHAR(80) NOT NULL,
  ville       VARCHAR(80) NOT NULL,
  type_magasin VARCHAR(20) NOT NULL CHECK (type_magasin IN ('Centre','Quartier','Pro'))
);

CREATE TABLE mode_paiement (
  id_mode SMALLINT PRIMARY KEY,
  libelle VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE regle_tva (
  id_regle SMALLINT PRIMARY KEY,
  taux     DECIMAL(4,2) NOT NULL UNIQUE CHECK (taux IN (0, 9, 18))
);

CREATE TABLE categorie (
  id_categorie SMALLINT PRIMARY KEY,
  libelle       VARCHAR(40) NOT NULL UNIQUE,
  sous_categorie VARCHAR(60)
);

-- 2. Tables métier (FK vers les références)
CREATE TABLE client (
  id_client  INTEGER PRIMARY KEY,
  nom        VARCHAR(120) NOT NULL,
  type_client VARCHAR(20) NOT NULL,
  ville      VARCHAR(80) NOT NULL,
  conditions_paiement VARCHAR(20) DEFAULT 'comptant'
);

CREATE TABLE produit (
  id_produit  INTEGER PRIMARY KEY,
  designation VARCHAR(120) NOT NULL,
  id_categorie SMALLINT NOT NULL REFERENCES categorie(id_categorie),
  prix_vente_ht DECIMAL(12,2) NOT NULL CHECK (prix_vente_ht > 0),
  actif       BOOLEAN NOT NULL DEFAULT TRUE
);

-- 3. Table de faits
CREATE TABLE vente (
  id_vente BIGINT PRIMARY KEY,
  id_client INTEGER NOT NULL REFERENCES client(id_client),
  id_produit INTEGER NOT NULL REFERENCES produit(id_produit),
  id_magasin INTEGER NOT NULL REFERENCES magasin(id_magasin),
  id_mode SMALLINT NOT NULL REFERENCES mode_paiement(id_mode),
  id_regle_tva SMALLINT NOT NULL REFERENCES regle_tva(id_regle),
  date_vente DATE NOT NULL,
  quantite INTEGER NOT NULL CHECK (quantite > 0),
  montant_ttc DECIMAL(14,2) NOT NULL CHECK (montant_ttc > 0),
  est_retour BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE INDEX idx_vente_client ON vente(id_client);
CREATE INDEX idx_vente_date ON vente(date_vente);
```

Notes de correction. • Les 7 tables sont imposées par l'ATTENDU.json (m06p_tables_3fn_cible = 7) — un schéma à 5 ou 9 tables est **faux** (sous-normalisation ou sur-normalisation). -2 pts. • Les contraintes CHECK (montants > 0, quantités > 0, taux dans {0, 9, 18}) sont **la moitié de la note L1** : sans elles, la 3FN ne protège rien. -2,5 pts. • La PK de vente est id_vente (BIGINT) — pas id_client (qui n'est pas unique). -1 pt.

L2 — load_duckdb.sql (extrait représentatif, exécuté)

```
-- Connexion : duckdb en mémoire
-- Commande : python3 -c "import duckdb; con = duckdb.connect(':memory:');
--   con.execute(open('schema_3fn.sql').read());
--   con.execute(open('load_duckdb.sql').read())"

-- 1. Tables de référence (depuis le CSV, DISTINCT)
INSERT INTO magasin
SELECT DISTINCT id_magasin, nom, ville, type_magasin
FROM read_csv_auto('quincaillerie_export.csv')
WHERE id_magasin IS NOT NULL;

INSERT INTO mode_paiement
SELECT DISTINCT id_mode, libelle_mode
FROM read_csv_auto('quincaillerie_export.csv')
WHERE id_mode IS NOT NULL;

INSERT INTO regle_tva
SELECT DISTINCT id_regle_tva, taux_tva
FROM read_csv_auto('quincaillerie_export.csv')
WHERE id_regle_tva IS NOT NULL;

INSERT INTO categorie
SELECT DISTINCT id_categorie, libelle_categorie, sous_categorie
FROM read_csv_auto('quincaillerie_export.csv')
WHERE id_categorie IS NOT NULL;

-- 2. Client : on retire les 3 orphelins
INSERT INTO client
SELECT DISTINCT id_client, nom, type_client, ville, conditions_paiement
FROM read_csv_auto('quincaillerie_export.csv')
WHERE id_client IS NOT NULL
  AND id_client NOT IN (SELECT id_client FROM read_csv_auto('quincaillerie_export.csv')
                       WHERE id_client NOT IN (SELECT MIN(id_client) FROM ...));

-- 3. Produit : on garde la dernière version (prix_vente_ht change dans le temps)
INSERT INTO produit
SELECT id_produit, designation, id_categorie, prix_vente_ht, actif
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY id_produit ORDER BY date_reference DESC) AS rn
  FROM read_csv_auto('quincaillerie_export.csv')
) WHERE rn = 1;

-- 4. Vente : on exclut les 12 dates inversées et les 8 doublons
INSERT INTO vente
SELECT DISTINCT id_vente, id_client, id_produit, id_magasin, id_mode,
  id_regle_tva, date_vente, quantite,
  CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS DECIMAL(14,2)),
  est_retour
FROM read_csv_auto('quincaillerie_export.csv')
WHERE date_vente <= COALESCE(date_fin_validite, date_vente)
  AND id_client IN (SELECT id_client FROM client);
```

Notes de correction.

- `DISTINCT` retire les 8 doublons. -1 pt si absent.
- `CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS DECIMAL(14,2))` convertit les 142 montants en texte. -2 pts si on laisse passer le défaut (sélection explicite sans conversion).
- `WHERE date_vente <= date_fin_validite` exclut les 12 dates inversées. -1 pt si absent.
- `IN (SELECT id_client FROM client)` exclut les 3 orphelins. -1 pt si absent.

L3 — `load_sqlite.py` (extrait, exécuté)

```
import sqlite3

con = sqlite3.connect(':memory:')
con.execute("PRAGMA foreign_keys = ON") # OBLIGATOIRE en SQLite
con.executescript(open('schema_3fn.sql').read()) # SQLite : pas de SERIAL → INTEGER PRIMARY KEY

# Lecture CSV ligne à ligne, conversion et insertion
import csv
with open('quincaillerie_export.csv', encoding='utf-8') as f:
    lecteur = csv.DictReader(f)
    for ligne in lecteur:
        # Conversion des défauts (cf. L2)
        try:
            montant = float(ligne['montant_ttc'].replace(' FCFA', '').replace(' ', ''))
        except ValueError:
            continue # 142 lignes sautées (texte)
        # ...

# Contrôles : jointure externe NULL doit retourner 0
orphelins = con.execute("""
    SELECT COUNT(*) FROM vente v LEFT JOIN client c ON v.id_client = c.id_client
    WHERE c.id_client IS NULL
""").fetchone()[0]
assert orphelins == 0, f"{orphelins} orphelins restants"
```

Notes de correction. • `PRAGMA foreign_keys = ON` est **obligatoire** en SQLite (désactivé par défaut). -1 pt si absent. • `INTEGER PRIMARY KEY` au lieu de `SERIAL` (PostgreSQL) — adaptation SQLite valide. Pas de pénalité. • Sortie observée : 7 tables créées, 24000 lignes insérées, 0 orphelin, 0 défaut résiduel.

L4 — `note_postgres.md` (cité sans exécuté, 1 page)

```
# Note PostgreSQL – pourquoi on ne l'exécute pas ici

## La commande qu'on exécuterait
```bash
psql -d quincaillerie -f schema_3fn_postgres.sql
```

où `schema_3fn_postgres.sql` diffère de `schema_3fn.sql` par : - `id_vente BIGSERIAL PRIMARY KEY` au lieu de `INTEGER PRIMARY KEY` - `id_regle_tva SMALLSERIAL` au lieu de `SMALLINT` - `BOOLEAN DEFAULT FALSE` au lieu de `BOOLEAN DEFAULT 0`

## Ce qu'on attendrait

- `CREATE TABLE` : 7 tables, 0 erreur
- `COPY vente FROM 'quincaillerie_export.csv' WITH (FORMAT csv, HEADER true)` : 24 000 lignes
- `INSERT ... ON CONFLICT (id_vente) DO NOTHING` : gestion des 8 doublons à la source
- `EXPLAIN ANALYZE SELECT ...` : plan d'exécution avec index sur `id_client`, `id_produit`, `date_vente`

## Pourquoi on ne l'exécute pas

Règle M06 §1.5 : PostgreSQL est **cité sans exécuté** dans ce module. Le module M08 (administration PostgreSQL) reprend ces commandes et les exécute sur un serveur réel. Ici, on cite pour

fixer le vocabulaire (client-serveur, multi-utilisateurs, `SERIAL` vs `INTEGER PRIMARY KEY AUTOINCREMENT`).

## Ce que cette note prouve

Le candidat connaît les **3 adaptations** de syntaxe (`SERIAL` → `BIGSERIAL`/`SMALLSERIAL`, `COPY` vs `INSERT, ON CONFLICT`) et sait expliquer pourquoi PostgreSQL n'est pas exécuté dans ce module.

```
> **Notes de correction.**
> • Toute mention de « exécuter `psql ...` » sans préciser que ce n'est pas lancé ici : -1 pt.
> • Au moins une des 3 adaptations syntaxiques (SERIAL, COPY, ON CONFLICT) doit apparaître. -0,5 pt
> par adaptation manquante.

L5 – `verdict_paysage.md` (le 80/20 et le choix de moteur)

```markdown
# Verdict – quel moteur pour quel cas ?

## 1. Volumes et seuils (C05 §7)
- DuckDB : 1 analyste solo, jusqu'à 50M lignes, fichier local
- SQLite : 1 application mono-utilisateur, fichier embarqué (< 1 To)
- PostgreSQL : multi-utilisateurs, gouvernance, > 1 To
- NoSQL : 1 cas sur 5, jamais par défaut

## 2. Notre cas (quincaillerie, 24 000 l. × 42 col)
- Volumétrie : bien sous 50M → DuckDB **ou** SQLite suffisent
- Utilisateurs : 1 data analyst (module) → DuckDB
- Gouvernance : pas de production ici → DuckDB
- Choix retenu : **DuckDB** (le plus expressif pour la 3FN, `read_csv_auto` intégré)

## 3. Cas où on basculerait
- 5 data analysts en production → PostgreSQL
- Application mobile embarquée (catalogue produit hors-ligne) → SQLite
- Données semi-structurées (logs JSON de caisses) → NoSQL document (MongoDB)
- > 1 To de données, analytique massive → entrepôt (Snowflake, BigQuery) + lakehouse (Delta/Iceberg)

## 4. Verdict final
**80 % SQL / 20 % NoSQL.** Le bon moteur dépend du cas d'usage, pas de la mode. Pour ce module :
DuckDB. Pour la mise en production : PostgreSQL ou l'entrepôt, selon la gouvernance attendue.
```

Notes de correction. • Le verdict doit citer **les 4 critères** (volumétrie, utilisateurs, gouvernance, structure). -2 pts par critère absent. • Le « 80 / 20 » doit apparaître explicitement. -1 pt si absent. • Au moins **un** cas de basculement (quand est-ce qu'on ne choisirait plus DuckDB). -1 pt si absent.

3. Barème détaillé

Livable	Critère	Points
L1 (5 pts)	7 tables imposées	1
	PK correctes (toutes tables)	1
	FK correctes (5 FK dans vente)	1
	CHECK (montants > 0, quantités > 0, taux dans {0,9,18})	1,5
	Types stricts (DECIMAL, INTEGER, VARCHAR)	0,5

Livrable	Critère	Points
L2 (5 pts)	DISTINCT (retire les 8 doublons)	1
	CAST/REPLACE (convertit les 142 montants)	2
	WHERE date_vente <= date_fin_validite	1
	Exclusion des 3 orphelins	1
L3 (3 pts)	PRAGMA foreign_keys = ON	1
	Adaptation SQLite (INTEGER PRIMARY KEY, pas SERIAL)	0,5
	Jointure externe NULL = 0 vérifiée	1
	Conversion des défauts	0,5
L4 (2 pts)	Règle « cité sans exécuté » respectée	1
	3 adaptations syntaxiques (SERIAL, COPY, ON CONFLICT)	1
L5 (5 pts)	4 critères de choix	2
	Verdict 80/20	1
	Cas de basculement (≥ 1)	1
	Cohérence avec le cas réel (DuckDB pour 24k l. × 42 col)	1

Seuil de passage : 13/20. En dessous : un livrable au choix à refaire. Au-dessus : le module est validé et le candidat passe au module M07.

4. Erreurs fréquentes (à éviter)

- **Schéma à 5 tables** (regrouper `categorie` dans `produit`) → 2NF non respectée. -2 pts L1.
- **SERIAL dans SQLite** → erreur d'exécution. -1 pt L3.
- **PRAGMA foreign_keys oublié** → SQLite ne lève aucune erreur sur les FK ; les orphelins passent. -1 pt L3.
- **Présenter PostgreSQL comme exécuté** → -2 pts L4.
- **DuckDB cité sans exécuté** → -3 pts L2.
- **Choisir NoSQL « parce que c'est moderne »** sans citer les 4 critères → -2 pts L5.

Projet M07.P — « Trente questions pour la direction commerciale : de la base brute au tableau de bord fiable »

Module M07 · projet de fin de module · 6 h · barème /20, seuil de passage 13/20

Le cadre réel. La direction commerciale d’une enseigne de distribution burkinabè vous confie sa base `commercial.duckdb` (9 tables, 50 008 ventes sur 2025–2026, 1 200 clients, 380 produits, 5 magasins). Elle a 30 questions en tête — et 3 défauts que personne n’a encore signalés (8 doublons, 200 retours comptés en positif, 15 dates de remise inversées). À vous de répondre aux 30 questions **en détectant les 6 pièges de fiabilité** (3 dans les données, 3 dans les requêtes) avant de poser chaque requête. C’est la synthèse des huit chapitres : SELECT/FROM (C01), WHERE (C02), ORDER BY/LIMIT (C03), agrégats (C04), GROUP BY/HAVING (C05), CASE (C06), jointures (C07), sous-requêtes et EXISTS (C08) — chaque bloc de questions en mobilise un, et aucun n’est noté pour lui-même : seul le **chiffre juste, obtenu sans se faire piéger** compte.

Note de cohérence. Le dossier est généré de façon déterministe par `python3 tools/dossier_M07.py` (graine 44) : qui que vous soyez, vous touchez les mêmes 50 008 ventes et la même empreinte BD `df9333ff...aldde.03_exercices/dossier_M07/ATTENDU.json` contient les **31 clés** des résultats attendus, **mesurés sur le dossier généré, jamais déduits** : vous n’y recourez qu’après avoir produit vos propres nombres, pour comparer, pas pour copier. Un candidat qui lit l’ATTENDU avant de calculer n’a plus rien à rendre. Les chiffres de ce corrigé sont ceux du dossier livré, vérifiables ligne à ligne sur `commercial.duckdb`.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c’est
Base de travail	<code>03_exercices/dossier_M07/commercial.duckdb</code>	9 tables + 1 vue : vente (50 008 lignes, 3 défauts : 8 doublons exacts, 200 retours <code>est_retour = TRUE</code> à montant positif, 15 ventes avec <code>date_vente > date_limite_remise</code>), client (1 200), produit (380, dont 18 inactifs), magasin (5), categorie (8, 5 libellés rayon seulement — les rayons Alimentaire, Bricolage, Jardinage sont doublés), mode_paiement (5), regle_tva (2), objectif_magasin (vide), vue <code>v_ca_mensuel_magasin</code> (120 lignes)
Export SQL	<code>03_exercices/dossier_M07/commercial.sql</code>	le INSERT complet (4,68 Mo), rechargeable dans n’importe quel DuckDB
Questions	<code>03_exercices/dossier_M07/30_questions.sql</code>	30 questions métier : 6 faciles (Q01–Q06), 12 moyennes (Q07–Q18), 9 avancées (Q19–Q26), 3 expert (Q27–Q30)
Corrigé de référence	<code>03_exercices/dossier_M07/30_reponses.sql</code>	les 30 requêtes de référence (à exécuter soi-même pour vérifier)
Pièges	<code>03_exercices/dossier_M07/6_pieges.md</code>	les 6 pièges de fiabilité : P1 doublon (Q19), P2 retours (Q09), P3 dates (Q14), P4 jointure multiple (Q27), P5 DISTINCT masquant (Q23), P6 EXISTS/IN (Q30)
Résultats attendus	<code>03_exercices/dossier_M07/ATTENDU.json</code>	31 clés <code>m07p_*</code> : empreintes sha256, totaux, top, pièges

Les quatre parties, et ce qu'elles notent

#	Partie	Questions	Ce que ça mobilise	Barème
P1	Lecture et volumes	Q01-Q06	C01, C02, C04	/4
P2	Mesures par dimension	Q07-Q18	C03, C04, C05	/6
P3	Requêtes composées et données sales	Q19-Q26	C05, C06, C08	/6
P4	Expert : jointures multiples et « tous les »	Q27-Q30	C07, C08	/4

Règle du projet. Chaque question piège (Q09, Q14, Q19, Q23, Q27, Q30) est notée sur **deux critères** : le chiffre juste **et** la détection du piège (une phrase qui nomme le piège et explique pourquoi la réponse naïve est fausse). Un chiffre juste obtenu par la requête naïve (par exemple le CA « gonflé » de Q09 présenté comme le CA) vaut **0**, même si le nombre est dans ATTENDU.json.

2. Grille de notation détaillée (/20)

Partie	Détail	Points
P1	Q01-Q06 : 6 réponses justes (≈ 0,5-0,7 chacune)	/4
P2	Q07-Q18 : 12 réponses justes, dont Q09 (piège P2 : 0,5 + 0,5 piège) et Q14 (piège P3 : 0,5 + 0,5 piège)	/6
P3	Q19-Q26 : 8 réponses justes, dont Q19 (piège P1 : 1 + 1 piège) et Q23 (piège P5 : 0,5 + 0,5 piège)	/6
P4	Q27-Q30 : 4 réponses justes, dont Q27 (piège P4 : 0,75 + 0,75 piège) et Q30 (piège P6 : 0,5 + 0,5 piège)	/4

Total : /20. Seuil de passage : 13/20. Les 6 pièges pèsent au total 5 points : c'est le cœur du module — savoir requêter ne vaut rien si on ne sait pas vérifier.

3. Corrigé-type (à publier après la séance)

Ce corrigé n'est pas un modèle à recopier : c'est la **référence** à laquelle vous comparez vos propres nombres. Les requêtes de référence sont dans `30_reponses.sql` ; exécutez-les sur `commercial.duckdb` et comparez. Si un chiffre ne tombe pas à 1 unité près, c'est que votre code a un défaut — pas que la référence a tort.

P1 — Lecture et volumes (Q01-Q06)

Q	Question	Réponse de référence
Q01	Combien de clients ?	1 200
Q02	Combien de produits actifs ?	362 (18 inactifs)
Q03	Combien de magasins ? Dans quelles villes ?	5 : Ouaga Centre, Ouaga Patte d'Oie (Ouagadougou), Bobo Centre, Bobo Sarfalao (Bobo-Dioulasso), Koudougou
Q04	Montant TTC total de toutes les ventes (sans filtre) ?	7 908 259 731 FCFA
Q05	Combien de modes de paiement ? Listez-les.	5 : Espèces, Carte Bancaire, Mobile Money, Credit 30j, Virement

Q	Question	Réponse de référence
Q06	Première et dernière vente ?	2025-01-01 et 2026-12-28

P2 — Mesures par dimension (Q07-Q18)

Q	Question	Réponse de référence
Q07	CA TTC par magasin (sans retours), décroissant	Magasin 4 en tête (1 592 496 610 FCFA sans retours), puis 1, 2, 5, 3
Q08	Ventes par mois, 2025	12 mois, de 2 058 (janvier) à environ 2 100/mois
Q09	PIÈGE P2 — CA total sans filtrer les retours	Le chiffre de référence (7 908 259 731) inclut 200 retours à montant positif : le CA « propre » est 7 876 320 163 FCFA (WHERE est_retour = FALSE). Le piège : les retours sont comptés en positif dans le brut
Q10	Panier moyen (sans retours)	158 159 FCFA (158 140 avec les retours — cf. C04)
Q11	10 produits les plus vendus (quantité)	Produit 98 en tête (939 unités), puis 107, 58...
Q12	5 meilleurs clients par CA	Client 73 en tête (11 561 026 FCFA), puis 930 (10 647 641), 614 (10 602 339)...
Q13	Clients distincts par magasin	1 199 / 1 199 / 1 200 / 1 200 / 1 199 (couverture quasi totale)
Q14	PIÈGE P3 — CA par mois, date_vente vs date_limite_remise	24 mois si on filtre date_vente <= date_limite_remise (15 ventes inversées écartées). Le piège : 15 ventes ont une date_vente postérieure à leur date_limite_remise — les attribuer au mois de la vente ou au mois de la remise change le CA mensuel
Q15	Ventes un dimanche ?	7 136
Q16	Taux de retour par rayon	0,4 % à 0,5 % par rayon (les retours sont rares et uniformes)
Q17	Délai moyen entre début de mois et vente	15,9 jours (ventes réparties sur le mois)
Q18	Clients avec plafond > 100 000 FCFA	490

P3 — Requêtes composées et données sales (Q19-Q26)

Q	Question	Réponse de référence
Q19	PIÈGE P1 — dedoublonnage exact	50 000 ventes uniques (50 008 — 8 doublons exacts). Nuance de vérification : le DISTINCT sur les 5 colonnes métier (client, produit, magasin, date, montant) renvoie 49 999 — une paire de ventes naturelle (id 41 405 et 41 757 : même client, produit, magasin, date et montant, mais mode de paiement et remise différents) se projette en une seule ligne. Le dedoublonnage juste porte sur toutes les colonnes métier, pas sur une projection choisie à l'œil
Q20	Clients ayant acheté dans ≥ 3 magasins	1 200 (tous — la couverture multi-magasins est quasi complète)
Q21	Top 1 produit par magasin (CA)	5 lignes (une par magasin) via ROW_NUMBER() OVER (PARTITION BY id_magasin)
Q22	Remise moyenne par mode de paiement	0,0 % (Espèces) à 1,5 % (Carte Bancaire) — les remises suivent le mode
Q23	PIÈGE P5 — couples client×produit distincts	47 359 couples (SELECT COUNT(*) FROM (SELECT DISTINCT id_client, id_produit FROM vente) t). Le piège : un COUNT(DISTINCT ...) « répare » visuellement une jointure qui double — il faut vérifier le COUNT(*) de la jointure nue avant de conclure
Q24	Écart CA réalisé / objectif par magasin	0 ligne — la table objectif_magasin est vide : la jointure JOIN objectif_magasin ne renvoie rien. La bonne réponse est de l'expliquer (pas de données d'objectif), pas de renvoyer 0

Q	Question	Réponse de référence
Q25	Évolution du CA 2025→2026 par rayon	Jardinage en tête (+39 020 518 FCFA), puis Bricolage (+21 770 291), Alimentaire (+19 250 569)...
Q26	Produits au-dessus du prix moyen de leur rayon	190 produits (sous-requête corrélée sur id_categorie)

P4 — Expert : jointures multiples et « tous les » (Q27-Q30)

Q	Question	Réponse de référence
Q27	PIÈGE P4 — ventes Bobo-Dioulasso x rayon Bricolage (4 jointures)	4 568 . Le piège : la jointure 4 tables en étoile (vente→client, vente→produit→categorie) est ici $N \rightarrow 1$ partout — <code>COUNT(*) = COUNT(DISTINCT id_vente)</code> = 4 568, pas de doublon . Le piège se referme si on joint une table qui multiplie (cf. C07 §5.7 : 2 131 064 lignes) — d'où le contrôle systématique
Q28	Segmentation RFM des 1 200 clients	1 200 lignes segmentées (champion / fidele / occasionnel / perdu / nouveau) via WITH + CASE
Q29	Tableau de bord mensuel 2025 avec LAG	12 lignes : CA, nb ventes, panier, retours, et LAG(ca) (janvier en NULL)
Q30	PIÈGE P6 — clients ayant acheté TOUS les produits « Bricolage-Outillage »	0 client : la double négation <code>NOT EXISTS (... NOT EXISTS (...))</code> ne renvoie personne (aucun client n'a acheté les 55 produits du rayon Outillage). Le piège : <code>IN</code> renverrait les clients ayant acheté au moins un (très nombreux) — le sens « tous » exige la double négation

4. Les 6 pièges de fiabilité (rappel et vérification)

Piège	Type	Question	La faute	La correction	Chiffre de contrôle
P1	données	Q19	<code>COUNT(*)</code> naïf = 50 008 (8 doublons inclus)	dedoublonnage sur toutes les colonnes métier	50 000 uniques (proxy 5 colonnes : 49 999)
P2	données	Q09	CA avec retours comptés en positif	<code>WHERE est_retour = FALSE</code>	7 908 259 731 (gonflé) vs 7 876 320 163 (propre)
P3	données	Q14	15 ventes avec <code>date_vente > date_limite_remise</code>	filtrer <code>date_vente <= date_limite_remise</code> (ou documenter la convention)	24 mois filtrés
P4	requête	Q27	jointure qui double (relation $N \rightarrow N$ oubliée)	étoile $N \rightarrow 1$ + contrôle <code>COUNT(*) = COUNT(DISTINCT id_vente)</code>	4 568
P5	requête	Q23	<code>DISTINCT</code> qui masque un mauvais <code>JOIN</code>	contrôler <code>COUNT(*)</code> sur la jointure nue avant l'agrégat	47 359 couples
P6	requête	Q30	<code>IN</code> (« au moins un ») pour une question « tous les »	double négation <code>NOT EXISTS ... NOT EXISTS</code>	0 client

Règle du projet. Un piège non détecté **annule** la note de la question, même si le chiffre tombe juste (cas P2 : le chiffre « gonflé » est bien dans ATTENDU.json — c'est la version piège). La détection se note par une **phrase** : nom du piège + pourquoi la réponse naïve est fausse + la requête corrigée.

5. Critères de passage

Critère	Condition
Note totale	$\geq 13/20$
Pièges	au moins 4 des 6 pièges détectés et corrigés
Exécution	les requêtes exécutées sur DuckDB (sortie capturée), pas juste écrites
Vérification	chaque partie contrôlée par un total cohérent (ex. P2 : la somme des CA par magasin = 7 876 320 163 sans retours)

L'esprit du projet. On ne note pas la syntaxe (DuckDB la corrige) : on note le **chiffre juste, obtenu proprement, avec les pièges nommés**. Un candidat qui écrit 30 requêtes correctes mais qui livre le CA « gonflé » de Q09 sans le signaler **n'a pas réussi le module** — c'est exactement l'erreur qu'un analyste commet en production, et c'est elle que ce module apprend à empêcher.

Projet M08.P — « Le script qui fait le travail de trois matinées »

Module M08 · projet de fin de module · 2 h · barème /20, seuil de passage 13/20

Le cadre réel. Vous recevez le dossier `03_exercices/dossier_M08/` : 8 CSV réexportés de la base commerciale M07 (50 008 ventes sur 2025—2026, 1 200 clients, 380 produits, 5 magasins), dont un fichier **vide**. Personne ne les a encore regardés : ce que contient `vente.csv` (doublons ? retours ? dates ?) n’est connu que de l’ATTENDU.json — et de vos propres yeux. À vous d’écrire **un seul script commenté**, `analyse_commerciale.py`, qui, exécuté **d’une traite**, produit les 6 sections du projet `import`, `audit`, `nettoyage`, `indicateurs`, 4 graphiques, classeur de synthèse. C’est la synthèse des huit chapitres (C01 l’environnement, C02 les types et les erreurs, C03 les boucles, C04 les conteneurs, C05 les fonctions et la lecture de traceback, C06 la vecteurisation, C07 la lecture et le filtrage, C08 le `groupby` en centimes et l’export Excel) — et la **croisée SQL/pandas** est la colonne vertébrale : chaque indicateur doit redonner, à 2 FCFA près, ce que M07 a requêté en SQL.

Note de cohérence. Le dossier est réexporté de façon **déterministe** par `python3 tools/dossier_M08.py` (aucun tirage aléatoire, base M07 figée) : deux exécutions → `diff = 0`, empreinte dossier `b9a8d973119342ec...`. `03_exercices/dossier_M08/ATTENDU.json` contient le **côté SQL de la croisée** (25 clés `m08p_*` mesurées par DuckDB) : vous n’y recourez qu’**après** avoir produit vos propres nombres, pour comparer, pas pour copier. Le script du chapitre, exécuté d’une traite le 19/09/2026, est commité dans le dépôt : `03_exercices/dossier_M08/analyse_commerciale_modele.py` — les sorties publiées ci-dessous sont **ses** sorties, pas un exemple illustratif.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c’est
Dossier de travail	<code>03_exercices/dossier_M08/</code>	8 CSV : <code>vente.csv</code> (50 008 lignes × 13 colonnes, 4 défauts : 8 doublons, 208 retours bruts / 200 dédoublonnés, 21 406 dates sous la condition naïve + 15 deadlines aberrantes, 0 manquant), <code>client.csv</code> (1 200), <code>produit.csv</code> (380), <code>categorie.csv</code> (8), <code>magasin.csv</code> (5), <code>mode_paiement.csv</code> (5), <code>regle_tva.csv</code> (2), <code>objectif_magasin.csv</code> (0 ligne)
Brief	<code>03_exercices/dossier_M08/00_brief.md</code>	le brief remis à l’apprenant (ce qu’on attend, rien de plus)
Résultats attendus	<code>03_exercices/dossier_M08/ATTENDU.json</code>	le côté SQL de la croisée (25 clés <code>m08p_*</code> : formes, audit, indicateurs, totaux en float DuckDB)
Script modèle	<code>03_exercices/dossier_M08/analyse_commerciale_modele.py</code>	le script du chapitre (corrigé de référence, exécuté le 19/09/2026)

Les 6 sections, et ce qu'elles notent

#	Section	Ce que ça mobilise	Livrable
S1	Import des 8 CSV avec contrôle des formes	C07 read_csv	dictionnaire nom → lignes, imprimé
S2	Audit : doublons, retours brut/net, dates inversées, deadlines aberrantes, manquants, table vide — avec la méthode qui les a trouvés	C02, C07	rapport imprimé, les 4 défauts chiffrés avec leur définition exacte
S3	Nettoyage : dédoublonnage (12 colonnes métier), exclusion des retours des indicateurs	C07, C08	le DataFrame propre (49 800 lignes)
S4	Indicateurs en centimes : CA brut / CA propre / panier moyen / CA par magasin / par catégorie / plus gros mois	C06, C08	les 6 indicateurs, addition en centimes entiers
S5	Les 4 graphiques (matplotlib, backend Agg) : CA mensuel, CA par magasin, répartition par catégorie, histogramme des montants	C06, C08	4 fichiers PNG
S6	Export du classeur synthese_commerciale.xlsx : 4 feuilles nommées (audit, indicateurs, par_magasin, par_mois)	C08	comptages identiques au corrigé M07.P, totaux ± 2 FCFA documentés

2. Grille de notation détaillée (/20)

Livrable	Détail	Points
E1	Le script commenté : 6 sections numérotées, chaque section avec un en-tête de 2 lignes, aucune valeur en dur (tout est calculé), if __name__ == "__main__":	/8
E2	Le rapport d'audit imprimé : les 4 défauts du socle chiffrés avec leur définition exacte (8 doublons → 50 000 uniques ; 208 retours en brut / 200 après dédoublonnage ; 21 406 dates sous la condition naïve + 15 deadlines aberrantes ; 0 valeur manquante, 1 table vide) et la méthode qui les a trouvés	/4
E3	Le classeur de synthèse : 4 feuilles nommées, comptages identiques au corrigé M07.P, totaux monétaires à ± 2 FCFA (la dérive float, mesurée et documentée dans le rapport d'audit), pas de cellule en dur	/4
E4	La note « ce qui a failli tourner en boucle » : 1 page — les 2-3 blocs où vous avez hésité entre une boucle et une vectorisation, et pourquoi la version vectorisée gagne (mesure de temps fournie)	/4

Total : /20. Seuil de passage : 13/20. E2 et E3 pèsent 8 points ensemble : c'est le cœur du module — produire le chiffre juste ne suffit pas, il faut le **vérifier** (audit) et le **livrer** (classeur lisible par machine).

3. Corrigé-type (sorties réelles du script modèle)

Le script modèle est 03_exercices/dossier_M08/analyse_commerciale_modele.py. Exécutez-le (venv pandas 3.0.6, matplotlib en backend Agg) : les sorties ci-dessous sont celles de son exécution du 19/09/2026, publiées telles quelles.

```

S1 formes : {'v': 50008, 'c': 1200, 'p': 380, 'cat': 8, 'm': 5, 'mode': 5, 'tva': 2, 'o': 0}
S2 audit : doublons=8 retours_brut=208 retours_net=200
           dates_naives=21406 deadlines_j20=15 manquants=0
           table vide : ['o']
S3 propres : 49800 lignes (doublons sortis, retours exclus)
S4 ca_brut=7908259732 ca_propre=7876320164 panier=158140
   top magasin : 4 1596813264
   top categorie : 7 1236006464
   plus gros mois : magasin 3 (2025-08) : 78965529
S5 graphiques : ['g1_ca_mensuel.png', 'g2_ca_magasin.png', 'g3_ca_categorie.png',
                 'g4_hist_montants.png']
S6 classeur : {'audit': (7, 2), 'indicateurs': (4, 2), 'par_magasin': (5, 1), 'par_mois': (120,
3)}

```

Chaque chiffre du S4 est une **clé du socle** (chiffres_cites.json) : `ca_brut = m08p_ca_brut`, `ca_propre = m08p_ca_sans_retours`, `panier = m08p_panier_moyen`, `top magasin 4 = m08p_ca_top_magasin`, `top catégorie 7 = m08p_ca_categorie_top`, `plus gros mois = m08p_plus_gros_mois_ca`. Le S2 chiffre les 4 défauts avec leurs définitions — **E2 et E3 sur la même sortie**. Le S6 est **lisible par machine** : le rechargement `pd.read_excel(..., sheet_name=None)` renvoie les 4 feuilles avec leurs formes (120 × 3 et 5 × 1 notamment) — le classeur n'est pas qu'un document, c'est une interface.

4. Les contrôles de fiabilité (rappel et vérification)

Contrôle	Défaut / écart	La leçon	Chiffre de contrôle
D1	8 doublons exacts (12 colonnes métier)	dédoublonner sur toutes les colonnes métier, pas sur une projection	50 008 → 50 000 uniques
D2	208 retours en brut / 200 après dédoublonnage, à montant positif	l'ordre des opérations de l'audit : les 8 doublons sont des retours	CA gonflé 7 908 259 732 vs propre 7 876 320 164 (écart 31 939 568)
D3	21 406 dates sous la condition naïve <code>date_vente > date_limite_remise + 15</code> deadlines tombant au 20 du mois	documenter la convention (mois de la vente) avant de compter	24 mois de vue
D4	<code>objectif_magasin.csv</code> vide (0 ligne)	le merge renvoie 50 008 cellules NaN : c'est un signal , pas un bug	(50 008, 16) après merge
Centimes	diviser par ligne avant de sommer perd les centimes de chaque ligne	additionner en centimes entiers, diviser une fois après l'agrégat	le plus gros mois sort à 228 FCFA de son total exact (78 965 301 au lieu de 78 965 529)
Croisée	la somme float de DuckDB s'écarte de 1 FCFA de l'addition en centimes	la dérive float est mesurée et sourcée , pas cachée	5 totaux à 1 FCFA près (clés <code>m08p_ecart_somme_float_*</code>)

Règle du projet. Un chiffre juste obtenu **sans l'audit** ne passe pas E2 — c'est la règle du projet M07.P, reprise ici : seul compte le chiffre juste, obtenu sans se faire piéger, avec la méthode qui l'a trouvé.

5. Critères de passage

Critère	Condition
Note totale	≥ 13/20
E2	les 4 défauts du socle chiffrés avec leur définition exacte (pas « il y a des doublons », mais « 8 doublons exacts sur les 12 colonnes métier → 50 000 ventes uniques »)
E3	les 4 feuilles nommées + la dérive float écrite dans le rapport d'audit (± 2 FCFA, sourcée)
Exécution	le script exécuté d'une traite (sorties capturées), pas assemblé section par section à la main
Vérification	la croisée : vos totaux redonnent le côté SQL de ATTENDU.json à 2 FCFA près (comptages exacts)

L'esprit du projet. « Trois matinées », c'est le temps qu'un collègue passe à importer, auditer, nettoyer, calculer, graphiquer et exporter à la main — quatre ou cinq allers-retours machine, chacun une source d'erreur. Le script commenté est l'outil qui fait disparaître ces allers-retours **et** qui documente la méthode pour la prochaine fois. C'est la 2^e pièce du portfolio (la 1^{re} est le corrigé M07.P) : la même base, deux moteurs, un seul total.

Projet M09.P — « 3 heures avec un fichier que personne n’a regardé »

Module M09 · projet noté sur 20 · seuil 13 · durée imposée 3 h (chronomètre en marche) · rendu : 4 livrables

Ce que le projet évalue. Non pas « ce que vous avez trouvé », mais **ce que vous pouvez affirmer** : un rapport de 10 étapes à 80 % avec ses limites honnêtes passe ; une exploration à 100 % sans structure ne passe pas. Le fichier est un export brut d’un opérateur de **mobile money** (6 mois, 10 014 lignes × 10 colonnes) dont le contexte est à découvrir **dans** le fichier — personne ne vous en dit plus que ce qu’il contient.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c’est
Fichier de travail	03_exercices/dossier_M09/projet/fichier_inconnu.csv	10 014 lignes × 10 colonnes, période 2026-01-01 → 2026-06-30, personne ne l’a regardé
Brief du dossier	03_exercices/dossier_M09/00_brief.md	ce qui est remis à l’apprenant : les 4 jeux, et la règle de l’ATTENDU
Résultats attendus	03_exercices/dossier_M09/ATTENDU.json	les mesures de référence (section projet) — à consulter après , jamais avant
Les 6 chapitres	02_modules/M09_C01...C06	le protocole en 10 étapes, l’audit, la décomposition, la preuve, la corrélation, la note d’EDA
Référence des chiffres	01_socle_donnees/data/reference/chiffres_cites.json	clés m09p_p_* : tout chiffre publié vient de là

La règle du chronomètre

3 h, pas 3 h 30. À la 3^e heure, on **arrête** et on rend ce qui est fait. La fiche d’entrée se remplit dans les **30 premières minutes** (C01) : un candidat qui explore 45 minutes « pour voir » avant d’écrire une ligne a déjà perdu la moitié de P1. L’exploration **sans méthode** est la faute que le projet punit ; l’arrêt **avec** des conclusions provisoires est la compétence qu’il note.

Les 4 livrables et ce qu’ils notent

#	Livrable	Ce que ça mobilise	Points
P1	La fiche d’entrée (complétée dans les 30 min) : forme du fichier, variables (nom, unité, type, origine), 3 questions que le fichier peut répondre et 2 qu’il ne peut pas	C01	/4
P2	Le rapport d’EDA : les 10 étapes du protocole, chaque étape en une question + sa réponse chiffrée (sortie imprimée) ; les défauts du fichier chiffrés avec leur définition exacte	C01-C04	/6
P3	Les 4 graphiques qui répondent aux 4 questions les plus importantes du rapport — chacun légendé : question dans le titre, source , n , axe déclaré s’il n’est pas à zéro	C06	/5
P4	La note de conclusions provisoires (1 page) : ce que le fichier suggère , chaque affirmation avec sa source et sa limite ; la corrélation n’est jamais présentée comme une cause	C05, C06	/5

#	Livrable	Ce que ça mobilise	Points
	Total		/20
	Seuil de validation		13/20

2. Grille de notation détaillée (/20)

Livrable	Détail	Points
P1	Fiche d'entrée complète et datée : forme (lignes × colonnes), les 10 variables avec unité et origine, 3 questions répondables (formulées), 2 questions non répondables avec la raison (variable absente, période trop courte, pas d'intervention) ; rendue dans les 30 minutes	/4
P2	Les 10 étapes présentes , chacune en question + réponse chiffrée ; les 4 défauts du fichier chiffrés avec leur définition exacte (doublons sur tx_ref, montants négatifs, montants hors borne, commissions manquantes) ; les 6 graphies de statut normalisées avant tout calcul de taux	/6
P3	4 graphiques qui répondent à une question écrite dans le titre , matplotlib/seaborn exécutés, légende (source, n), axe déclaré si non zéro, pas de graphique « pour voir »	/5
P4	1 page, chaque affirmation = phrase + source (sortie re-jouable) + limite remplie (taille d'échantillon, période, variables manquantes) ; aucune causalité affirmée	/5
Total		/20

Les 4 points qui se perdent le plus souvent (et le chiffre qui les contrôle) :

1. compter les lignes **avant** de dédoublonner → 10 014 lignes affichées au lieu de 10 000 transactions distinctes ;
2. calculer un taux de succès **sans normaliser les statuts** → six graphies (OK, ok, Reussie, REUSSI, Echouee, echec) et un taux faux ;
3. publier un montant moyen **sans exclure les valeurs impossibles** → la moyenne est tirée par deux lignes aberrantes ;
4. conclure « les agents X font mieux » **sans les tailles** → 20 agents, des volumes très inégaux.

3. Corrigé-type (sorties réelles, exécutées le 20/09/2026)

```
import pandas as pd
d = pd.read_csv("03_exercices/dossier_M09/projet/fichier_inconnu.csv")

print(d.shape) # (10014, 10)
print(d["tx_ref"].duplicated().sum()) # 14 -> doublons sur la clé de transaction
d2 = d.drop_duplicates(subset="tx_ref")
print(d2.shape) # (10000, 10) : 10 000 transactions distinctes

print(d2["statut"].value_counts().to_dict())
# {'Reussie': 2082, 'ok': 2043, 'OK': 2037, 'REUSSI': 1962, 'echec': 981, 'Echouee': 909}
norm = (d2["statut"].str.strip().str.lower()
        .map(lambda s: "succes" if s in ("ok", "reussie", "reussi") else "echec"))
print(norm.value_counts().to_dict()) # {'succes': 8115, 'echec': 1885}
print(round(100 * norm.eq("succes").mean(), 2)) # 81.15 (taux de succès, %)
```

```
print(int((d2["montant_xof"] < 0).sum())) # 5 (min -4 555)
print(int((d2["montant_xof"] > 1 000 000).sum())) # 2 (max 1 000 000 000)
print(int(d2["commission"].isna().sum())) # 29
print(d2["agent_code"].nunique()) # 20
print(d2["canal"].value_counts().to_dict()) # {'Agent': 4971, 'USSD': 4009, 'app': 1020}
print(d2["type_op"].value_counts().to_dict()) # {'in': 5847, 'out': 4153}
print(d2["date_op"].min(), "->", d2["date_op"].max()) # 2026-01-01 -> 2026-06-30
```

La lecture P4 (la note de conclusions provisoires) — quatre affirmations, chacune avec sa source et sa limite :

1. « **Huit transactions sur dix réussissent** (81.15 %), stable sur les 6 mois. » Source : normalisation des 6 graphies + `value_counts` (exécuté). Limite : les 1890 échecs ne portent pas de code de motif — on ne sait pas pourquoi elles échouent ; ne pas confondre taux d'échec et cause d'échec.
2. « La **moyenne des montants ne veut rien dire** tant que les extrêmes sont dans le fichier : 203 506 XOF sur le fichier reçu contre 3 790 XOF sur les 9 993 lignes valides, pour une médiane de 2 503 XOF — 2 lignes à 1 000 000 000 XOF et 5 lignes négatives (de -685 à -4 555) font tout l'écart. » Source : bornes métier, médiane vs moyenne (exécuté). Limite : la borne haute est à confirmer avec l'exploitant — on exclut ces lignes du calcul et on l'écrit, on ne les corrige pas.
3. « Le **canal Agent** domine (4971 transactions) devant l'USSD (4009) et l'application (1020). » Source : `value_counts` (exécuté). Limite : 6 mois seulement, et `agent_code` est un code : 20 agents, mais aucune information sur la zone, le point de vente ou le type de clientèle — pas de lecture géographique possible.
4. « Les **29 commissions manquantes** (0.29 % des lignes) se concentrent sur un seul canal : à signaler, pas à combler. » Source : `isna().sum()` par canal (exécuté). Limite : un manquant déclaré n'est pas un manquant corrigé ; inventer la commission (taux moyen) fabriquerait un chiffre que personne ne peut vérifier.

4. Les contrôles de fiabilité (rappel et vérification)

Contrôle	Défaut / écart	La leçon	Chiffre de contrôle
D1	14 doublons sur <code>tx_ref</code>	dédoublonner sur la clé de transaction , pas sur la ligne entière	10 014 → 10 000
D2	6 graphies de statut (OK, ok, Reussie, REUSSI, Echouee, echec)	normaliser avant de compter : le taux se calcule sur la valeur normalisée	8 115 succès / 1 885 échecs (81.15 %)
D3	5 montants négatifs + 2 montants à 1 000 000 000 XOF	borner avec le métier , exclure en le déclarant, jamais « corriger » une valeur qu'on ne peut pas vérifier	9 993 lignes exploitables
D4	29 commissions manquantes (0.29 %)	un manquant se signale ; l'inventer fabrique un chiffre invérifiable	29 sur 10 000
Centimes	publier un volume en float sur des montants à 9 chiffres	additionner en entiers (centimes), diviser une fois après l'agrégat	volume valide 37 874 954 XOF, moyenne valide 3 790 XOF (<code>m09p_p_montant_total_valides</code>)

Le contrôle final du projet. Relisez P4 à voix haute : chaque phrase doit pouvoir être suivie de « ... et sa limite est que ... ». Si la limite reste vide, l'affirmation **déborde** de ce que le fichier dit : on la recule (ou on la retire). C'est exactement ce que note l'épreuve de palier **P2** — produire des chiffres justes **et** en tirer des conclusions provisoires honnêtes.

Projet M10.P — « La refonte »

Module M10 · projet noté sur 20 · seuil 13 · durée imposée 4 h (chronomètre en marche) · rendu : 4 livrables.

Ce que le projet évalue. Non pas « le plus beau graphique », mais **la refonte mesurée** : vous recevez un rapport dont **les chiffres sont justes** et dont les **5 graphiques** font lire autre chose que ce que les données disent. Vous devez **nommer** chaque défaut, **mesurer** l'écart qu'il crée, **corriger** la figure — puis **prouver** que votre correction n'a pas touché à la donnée. La règle du projet est la règle du module : **une refonte qui change un chiffre est rejetée** ; les totaux avant/après sont contrôlés automatiquement par `tools/controle_refonte_M10.py`, en **4** couches.

1. Énoncé

Le matériel

Élément	Chemin	Ce que c'est
Le rapport à refondre	<code>03_exercices/dossier_M10/rapport_avant/</code>	5 graphiques publiés (<code>g1_axe_tronque.svg</code> ... <code>g5_log_surcharge.svg</code>) + <code>code_avant.py</code> qui les produit
Les 2 pages du rapport	<code>03_exercices/dossier_M10/rapport_avant/rapport_avant.md</code>	ce que la direction en a retenu — les affirmations à vérifier
Le brief	<code>03_exercices/dossier_M10/00_brief.md</code>	la commande, en une page
Les 5 corrigés (à ne pas lire avant)	<code>03_exercices/dossier_M10/rapport_apres/</code>	<code>g1_corrige.svg</code> ... <code>g5_corrige.svg</code> + <code>code_apres.py</code>
Les mesures de référence	<code>03_exercices/dossier_M10/ATTENDU.json</code>	les défauts et leurs mesures (<code>d1_*</code> à <code>d5_*</code>) — à comparer à vos mesures, jamais à recopier
Le contrôle automatique	<code>tools/controle_refonte_M10.py</code>	vérifie la règle des totaux identiques, couche par couche
Les 7 chapitres	<code>02_modules/M10_C01...C07</code>	perception, table des 16 graphiques, couleurs, écriture, les 10 erreurs, la séquence, les outils
Référence des chiffres	<code>01_socle_donnees/data/reference/chiffres_cites.json</code>	clés <code>m10p_*</code> : tout chiffre publié dans ce projet vient de là

La règle du projet : les totaux avant/après sont identiques

C'est le point du barème qui élimine. Un candidat qui « améliore » la courbe en lissant, qui retire les retours « pour nettoyer », ou qui regroupe des catégories pour rendre un camembert lisible, change la donnée — et la refonte est rejetée, même si la figure est plus belle. Sur le fil rouge, les totaux de contrôle sont : **7 876 320 164,91** FCFA de CA net, **50 008** ventes dont **208** retours, **1 044** ventes de plus de 500 000 FCFA, une médiane de **119 482** FCFA. Le contrôle recalcule ces valeurs depuis le socle (il ne les recopie pas) et vérifie que la somme par catégorie et la somme par trimestre retombent sur le même total net, à **0,00** FCFA près.

Les 4 livrables et ce qu'ils notent

#	Livrable	Ce que ça mobilise	Points
P1	Le diagnostic des 5 défauts : pour chaque graphique, le défaut nommé (liste du C05), mesuré (l'écart entre ce que l'œil lit et ce que les données disent), et la décision fausse qu'il ferait prendre	C05	/5
P2	Les 5 graphiques corrigés : le bon graphique pour la question, titre qui affirme, axe honnête, annotation du chiffre clé, couleur justifiée	C01—C04	/6
P3	Les deux versions du constat : direction (1 écran, l'idée et la décision) et équipe opérationnelle (le détail, les libellés exacts) — sans contradiction de chiffres entre les deux	C06	/5
P4	La grille visuelle en 18 points, auto-évaluée , puis revue par un pair : deux notes, et l'écart commenté	la grille du module	/4
Total			/20
Seuil de validation			13/20

2. Grille de notation détaillée (/20)

Livrable	Détail	Points
P1	Les 5 défauts nommés (axe tronqué, double axe, camembert surchargé, cumul à 100 %, échelle log surprise) ; chacun avec sa mesure (pas un adjectif) ; la décision fausse rédigée en une phrase ; et la mention de la déclaration qui rendrait l'usage légitime — ou la raison pour laquelle aucune ne suffit	/5
P2	Les 5 figures corrigées : titre qui affirme (il porte un chiffre), axe à zéro ou troncature déclarée, annotation du chiffre clé, couleur justifiée (une couleur, un rôle), ordre trié quand il s'agit d'un classement ; largeur ≤ 776 px (format du module)	/6
P3	2 versions : direction (1 écran, la hiérarchie met l'idée principale en tête) et équipe (le détail, les libellés, les ordres de grandeur) ; aucun chiffre contradictoire entre les deux versions (le contrôle compare les totaux déclarés)	/5
P4	La grille en 18 points remplie par l'auteur, les 4 familles notées (décision servie, justesse, lisibilité, utilisation), puis la note du pair ; l'écart entre les deux notes est commenté (l'écart non commenté ne rapporte pas)	/4
Total		/20

Les 4 points qui se perdent le plus souvent (et la mesure qui les contrôle) :

- Le diagnostic sans mesure** : « l'axe est tronqué » ne vaut pas un point ; « l'axe occupe 82,4 % de sa hauteur pour une variation réelle de 5,4 %, soit une amplitude apparente × 7,0 » en vaut trois. Le contrôle relit d'ailleurs les **titres dans le code**, pas dans l'image (un SVG de matplotlib ne contient pas de texte).
- La correction qui change la question** : remplacer un camembert à 8 parts par un camembert à 4 parts ne change pas la question, seulement la gêne. Passer à des barres triées **la change** — et c'est légitime si la question est un classement (C01).
- Le double axe conservé** « pour ne pas refaire la figure » : c'est un échec de la couche L4 (le contrôle refuse tout `twinx` dans la version corrigée).
- Les deux versions qui ne disent pas la même chose** : 3 965,4 M d'un côté, 3 969 M de l'autre. Un écart de 4 M FCFA entre la note de direction et le tableau d'équipe décrédibilise l'analyse entière — le contrôle compare les totaux déclarés et refuse l'écart.

3. Corrigé-type (sorties réelles, exécutées dans l'atelier)

3.1 Le contrôle automatique, sur le dossier de référence

```
$ python3 tools/controle_refonte_M10.py
=====
CONTROLE DE REFONTE – M10.P « La refonte »
=====
dossier      : 03_exercices/dossier_M10
regle        : totaux avant/apres identiques et conformes au socle
seuil        : 13/20
-----
L1_filtres_avant      = 1
L1_filtres_apres      = 1
L1_filtres_identiques = True
L2_total_net_egal_categories = True
L2_total_net_egal_trimestres = True
L2_ecart_categories_fcfa = 0.0
L2_ecart_trimestres_fcfa = 0.0
L3_avant_svg          = 5
L3_apres_svg          = 5
L3_largeur_max_px     = 768
L3_largeur_limite_px  = 776
L4_titres_avant       = 5
L4_titres_apres       = 6
L4_titres_qui_affirment_avant = 0
L4_titres_qui_affirment_apres = 6
L4_twinx_avant        = 1
L4_twinx_apres        = 0
L4_set_yscale_log_avant = 1
L4_set_yscale_log_apres = 0
L4_annotations_apres  = 1
-----
VERDICT      : CONFORME
=====
```

Ce que ces lignes disent, en clair : la version corrigée **lit la même source** (mêmes fichiers, mêmes filtres), elle **retombe sur les mêmes totaux** (écart **0,00** FCFA sur la somme par catégorie comme sur la somme par trimestre), ses **5** figures tiennent dans le format (**768** px au maximum, sous la limite de **776**), et son **écriture** est vérifiée **dans le code** : **6** titres qui affirment sur **6**, plus aucun **twinx**, plus aucune échelle logarithmique non déclarée, une annotation du chiffre clé. Les titres « avant » portaient une mesure **0** fois sur 5.

3.2 Le contrôle sur une copie falsifiée (le cas qui doit être rejeté)

```
$ python3 tools/controle_refonte_M10.py --candidat /tmp/cand_ko --code /tmp/cand_ko/
code_refonte.py
-----
AVERTISSEMENT : L3 : 0 graphique(s) d'origine sur 5
EHEC          : L2 : la règle du projet est violée – les totaux avant/après diffèrent :
                  {'total_net_fcfa': (7876320164.91, 8012103456.12)}
-----
VERDICT      : REJETÉE
=====
```

La copie avait pourtant des figures correctes, dans le bon format. Elle a « **amélioré** » le **CA de 136 M FCFA** en refaisant les graphiques : c'est exactement ce que le projet refuse. Le contrôle n'a pas jugé le style : il a comparé deux nombres.

3.3 Le contenu attendu du livrable P1, ligne par ligne

Graphique	Défaut (C05)	La mesure	La décision fautive possible
g1_axe_tronque	n° 1, axe tronqué	fenêtre 300-350 M : 82,4 % de hauteur occupée pour une variation réelle de 5,4 % → amplitude apparente x 7,0	« l'activité s'envole, réorganisons l'équipe »
g2_double_axe	n° 2, double axe	corrélation affichée $r = +0,35$ entre le CA et les retours (4 à 16 retours par mois, 24 mois)	« les retours suivent l'activité : il faut renforcer les contrôles les mois de pointe »
g3_camembert	n° 4, camembert surchargé	8 parts, écart minimal 0,05 pt = 0,17° ; lisible jusqu'à 3 parts	« voici notre classement de catégories » sur des parts indiscernables
g4_cumul_pourcent	n° 5, cumul de pourcentages	totaux trimestriels de 960 721 755 à 1 012 518 068 FCFA (5,4 %) masqués par des parts qui ne bougent que de 1,9 point	« le mix est stable, rien à faire »
g5_log_surcharge	n° 10, échelle log surprise	la masse (77,0 % des ventes sous 250 000 FCFA) occupe 42,1 % de la largeur en linéaire et 93,5 % en log ; la queue (1 044 ventes) passe de 15,9 % à 1,3 %	« les montants sont homogènes » : la queue, qui porte 7,1 % du CA, disparaît

3.4 Ce qu'un livrable P2 conforme contient

Pour chaque figure : **un** titre qui affirme (il contient un chiffre), l'axe déclaré, **une** annotation, une couleur par rôle, et l'ordre adapté à la question. Les cinq corrections du dossier de référence, à comparer aux vôtres : axe à zéro + annotation « la moyenne glissante reste entre **320** et **337** M » ; deux petits multiples (une échelle chacun) avec la corrélation annoncée dans le titre ; barres **triées** avec les valeurs absolues écrites ; volumes **absolus** par trimestre avec la ligne de moyenne ; histogramme **linéaire annoncé** et **zoom séparé** sur la queue.

4. Les contrôles de fiabilité (rappel et vérification)

Contrôle	Comment il se fait	Ce qu'il attrape
Totaux identiques	tools/contrôle_refonte_M10.py, couche L2 : comparaison des totaux déclarés et des sommes par catégorie et par trimestre	une refonte qui change la donnée (lissage, filtre, regroupement)
Même source	couche L1 : les deux versions lisent les mêmes fichiers avec les mêmes filtres	un « nettoyage » silencieux des retours
Format	couche L3 : 5 + 5 SVG, largeur ≤ 776 px	une figure qui ne rentre pas dans la page composée
Écriture	couche L4 : audit du code — titres, axes, twinx , échelle log, annotations	une figure « corrigée » qui garde son défaut structurel
Grille en 18 points	P4 , auto-évaluation puis relecture par un pair	l'écart entre ce que l'auteur croit avoir fait et ce qu'un lecteur voit

Le réflexe du module, à garder après ce projet. Avant de publier une figure, une seule question : « qu'est-ce que quelqu'un pourrait dire de faux devant ce graphique ? » Si la phrase existe, la figure n'est pas prête — soit on la corrige, soit on **écrit la déclaration** qui rend la phrase impossible (C05).

5. Comment rendre

Fichier	Contenu attendu
rapport_apres/ g1_corrige.svg g5_corrige.svg	les 5 figures corrigées, au format du module ...
code_refonte.py	le script qui produit les 5 figures (facultatif : sans lui, les couches L1 et L4 sont déclarées non vérifiables)
refonte.json	{"totaux_avant": {...}, "totaux_apres": {...}} : les totaux que vous avez mesurés dans les deux versions (les clés du socle : total_net_fcfa, ventes, retours, mediane_fcfa, queue_plus_500k)
diagnostic.md	le livrable P1, sous forme de tableau (graphique, défaut, mesure, décision fausse, déclaration)
versions/	les deux versions de P3 : un dossier par version, avec la figure et son titre
grille.md	la grille en 18 points remplie deux fois (auteur, pair) et l'écart commenté

Ce qui n'est pas accepté : une refonte sans mesure, une correction qui change un total, une figure dont le titre n'affirme rien, deux versions qui se contredisent, et un dossier dont les figures dépassent la largeur du format.

Projet M11.P — « Les 12 rapports SQL de la cellule commerciale »

Module M11 · projet noté sur 20 · seuil 13 · durée imposée 4 h (chronomètre en marche) · rendu : 4 livrables — P1 les 12 requêtes (8 points), P2 le jeu de tests (5 points), P3 la note de lecture (4 points), P4 la fiche de reprise (3 points).

Ce que le projet évalue. Non pas « la requête la plus savante », mais la **bibliothèque de rapports** : douze requêtes autonomes, **versionnées**, **testées** et **commentées en français**, chacune répondant à une question écrite. Vous recevez la commande de la cellule commerciale et le socle du module : **240 000** lignes de vente, **237 191** hors retours, **145 212** tickets, **23 497** clients, **5** magasins, **44** mois. La règle du projet est celle du module : **un rapport dont le total diffère d'un rapport à l'autre est rejeté**, même si chaque requête « fonctionne » — et un rapport que son lecteur ne peut pas rejouer ne vaut rien.

1. Énoncé

La commande

La cellule commerciale d'une enseigne de quincaillerie (**5** magasins, **44** mois, **240 000** lignes) commande sa **bibliothèque de rapports** : **12** requêtes autonomes, chacune répondant à une question qu'elle s'est posée cette année. Elle les veut **en français**, **datées**, **testées**, et livrées avec leur **mode d'emploi** — pas un dossier de requêtes, un **outil de travail**.

Le matériel

Élément	Chemin	Ce que c'est
Le socle	03_exercices/dossier_M11/socle_m11.sql	les 5 tables de référence et les 2 vues de ventes (3 595 octets)
La connexion	03_exercices/dossier_M11/connexion.py	ouvrir(materialiser=False) — la matérialisation en mémoire pour les mesures
Les mesures de performance	03_exercices/dossier_M11/PERF_M11.json	les temps gelés du chapitre C06 (24/09/2026 , médianes de 5 exécutions)
Le contrôle automatique	tools/controle_sql_M11.py	18 requêtes rejouées et confrontées au fichier publié — le calque de votre livrable P2
Les valeurs publiées	01_socle_donnees/data/reference/chiffres_cites.json	clés m11_*, m11p_*, m11e_* : 301 clés pour le module
Les 7 chapitres	02_modules/M11_C01...C07	fenêtres, rangs, temps, cohortes, requêtes avancées, plans, style et tests

Les six totaux de contrôle

Ces six nombres se recalculent depuis le socle — ils ne se recopient pas. Un rapport qui publie un total différent doit **dire pourquoi** (périmètre plus étroit, période partielle, client non identifié exclu) : c'est la seule tolérance admise, et elle s'écrit dans l'en-tête.

Contrôle	Valeur attendue
Chiffre d'affaires net (retours exclus)	15 595 154 955 FCFA
Lignes de vente (retours exclus)	237 191

Contrôle	Valeur attendue
Tickets de vente	145 212
Clients identifiés dans les ventes	23 497
Magasins	5
Mois couverts	44

2. Les 12 questions métier (le catalogue imposé)

Chaque rapport répond à **une** question, citée telle quelle dans son en-tête. Le chapitre indiqué est celui qui fournit la méthode ; la colonne « sortie » dit ce que le rapport doit **contenir**, pas la façon de l'obtenir.

Code	Question de la cellule	Chap.	Sortie attendue
R01	Combien avons-nous vendu chaque mois, et où en sommes-nous cette année ?	C01, C02	mois, CA, cumul, part du mois dans son année
R02	Quels magasins vendent le plus — et lesquels ont le plus gros panier ?	C01, C02	deux classements côte à côte, chacun avec son rang
R03	Quels sont nos 3 meilleurs produits par famille ?	C02	3 lignes par famille, rang, part dans la famille
R04	Quelle part du chiffre d'affaires font nos plus gros clients ?	C01, C04	parts cumulées, top 1 %, top 5 %, top 10 %, top 20 %
R05	L'activité progresse-t-elle, à périmètre comparable ?	C03	année sur année à mois constants, moyenne des variations
R06	La tendance de fond monte-t-elle ?	C03	moyennes mobiles de 3 et 12 mois, avec leur compteur de fenêtre
R07	Tenons-nous nos objectifs ?	C03	réalisé, objectif, taux — trous compris et déclarés
R08	Quel jour de la semaine travaillons-nous le mieux ?	C03	CA par jour de vente, écart entre le meilleur et le pire
R09	Nos clients reviennent-ils ?	C04	rétenion pondérée à M+1, M+3, M+12, avec les effectifs
R10	Quels clients appeler en priorité, et pour quoi ?	C04	segments RFM, effectif, part du CA, plage de récence
R11	Où vend-on quoi ?	C05	tableau croisé quartier × famille avec sous-totaux
R12	Nos deux socles racontent-ils la même histoire ?	C07	comparaison documentée des deux socles, écarts expliqués

Douze questions, douze rapports, une seule vérité de total : les **12** rapports publient le même chiffre d'affaires net quand ils parlent de la même période — **15 595 154 955** FCFA sur le socle complet, ou un sous-ensemble dont le périmètre est écrit noir sur blanc.

Le format d'en-tête, imposé

Chaque rapport s'ouvre sur ces six lignes — c'est ce qui distingue une bibliothèque d'un fichier de brouillons :

```
-- Rxx – <question de la cellule, recopiée telle quelle>
-- Hypothèses : <ce qu'on suppose : retours exclus, client 0 inclus, période complète...>
-- Grain : <une ligne = un mois / un client / un couple quartier-famille>
-- Source : <socle M11, vue ventes, référence produit>
-- Total de contrôle : <le nombre que le rapport doit rendre>
-- Date : <le jour de rédaction>
```

3. Les 4 livrables

P1 — Les 12 requêtes (8 points)

Une requête par question, dans l'ordre R01 → R12, chacune avec **son en-tête complet**, un ORDER BY **explicite** — avec sa clé de départage quand deux lignes peuvent être à égalité —, et son **total de contrôle**.

Ce qui est vérifié à la correction :

- R02 : les deux classements sont donnés avec leur **rang** ; le rapport signale que l'ordre n'est pas le même (**Kaya** a le plus gros panier, **Ouagadougou** le plus gros chiffre d'affaires).
- R03 : le classement porte sur les **7** familles réelles, pas sur les **16** libellés du référentiel ; le rapport publie les **deux** comptes (**46** lignes contre **21**).
- R04 : la part du top 10 % est publiée **avec** et **sans** le client non identifié, et la définition du décile est écrite.
- R05 : aucune comparaison entre périodes de longueurs différentes ; la période partielle est annoncée.
- R06 : chaque moyenne mobile porte le nombre de lignes de sa fenêtre ; les douze premiers mois sont marqués comme **non comparables**.
- R07 : les **2** couples magasin-mois sans objectif apparaissent dans le rapport, avec un état explicite et **93 749 490** FCFA de ventes sans référence.
- R09 : la rétention est **pondérée** par la taille des cohortes ; le tableau porte les effectifs.
- R10 : le traitement du client **0** est écrit ; tous les classements sont départagés.
- R11 : les sous-totaux sont produits dans la **même** requête (ROLLUP ou GROUPING SETS), et le total général égale le chiffre d'affaires net.

P2 — Le jeu de tests (5 points)

Chaque requête est livrée avec **au moins un test de non-régression**, dans l'esprit de `tools/controle_sql_M11.py` : une valeur attendue (le total, le nombre de lignes, une valeur extrême), la requête rejouée **à l'identique**, le verdict, et un **code de sortie** utilisable en chaîne.

Ce qui est vérifié :

- **au moins un** test par rapport, soit **12** au minimum, plus la sortie de l'instrument du module (**18** lignes) publiée ;
- les tests **recopient** la requête du rapport caractère pour caractère : la plus petite réécriture change la réponse — un classement des clients par nombre de tickets rend **23 444** en ordre décroissant et **23 497** en ordre croissant ;
- un écart produit un **code de sortie non nul** : un test qui se contente d'afficher « échec » ne protège rien.

Attention. Un test qui « améliore » la requête qu'il rejoue ne teste plus le chiffre publié : il fabrique une alerte, ou pire, il valide une valeur qui n'apparaît nulle part dans le rapport.

P3 — La note de lecture (4 points · 2 pages)

Deux pages, pour la personne qui n'écrira jamais une ligne de SQL : ce que les **12** rapports disent **ensemble**, et les **3** précautions à connaître avant de citer un chiffre :

1. le client **0** — **18,3** % du chiffre d'affaires, absent du référentiel, premier de tout classement ;
2. les **retours** — **2 809** lignes négatives : un total « net » qui les compte n'est pas un total ;
3. les **grains** — ligne, ticket, client : quatre nombres pour la même question (**146 161** tickets au total, **145 212** avec au moins une vente, **2 797** qui portent au moins un retour, dont **1 848** mixtes et **949** uniquement des retours).

La note se termine par **la question à laquelle les 12 rapports ne répondent pas** — par exemple pourquoi un client achète, ou ce qui se passera au prochain trimestre : un rapport décrit, il n'explique pas.

P4 — La fiche de reprise (3 points)

Un pair — qui n'a pas écrit les rapports — en reprend **3**, exécute leurs tests, et note **ce qui manque** pour qu'il puisse travailler sans vous : hypothèse non écrite, colonne ambiguë, test absent, nom de table supposé connu. La fiche publiée est la **sienne**, pas la vôtre ; votre annotation de son retour tient en trois lignes.

Conseil professionnel. Écrivez l'en-tête **avant** la requête. Tant que la ligne « Hypothèses » n'est pas rédigée, la requête n'est pas finie : c'est en l'écrivant qu'on découvre qu'on hésite entre ventes brutes et ventes nettes — et cette hésitation est exactement ce qui fait publier deux chiffres différents du même indicateur.

4. Barème

Livrable	Critère	Points
P1 · les 12 requêtes	12 requêtes rendues, dans l'ordre R01 → R12	4
	Les 12 en-têtes complets (question, hypothèses, grain, source, total, date)	2
	ORDER BY explicite et clé de départage partout où une égalité est possible	1
	Total de contrôle présent et exact	1
P2 · le jeu de tests	Au moins 1 test par rapport (12 au minimum)	2
	Requêtes rejouées à l'identique (aucune réécriture)	1,5
	Sortie de <code>tools/controle_sql_M11.py</code> publiée (18 lignes)	1
	Code de sortie non nul en cas d'écart	0,5
P3 · la note de lecture	Les 3 précautions (client 0 , retours, grains), chacune chiffrée	2
	Ce que les 12 rapports disent ensemble , en une page	1
	La question à laquelle ils ne répondent pas	1
P4 · la fiche de reprise	Les 3 rapports repris par un pair, tests exécutés	2
	Le compte rendu de ce qui manque, et votre réponse en trois lignes	1
Total		20

Seuil de validation : 13/20. Un livrable non rendu vaut zéro : il n'y a pas de points de présence.

5. Ce qui fait rejeter un livrable

1. **Un total qui change d'un rapport à l'autre** sans explication de périmètre : la règle du projet, et la seule faute éliminatoire.
2. **Un classement sans clé de départage.** 3 316 clients partagent exactement 8 tickets (14,1 % du fichier) : la frontière des tiers a compté de 2 718 à 2 738 clients selon l'exécution, sans qu'aucune donnée n'ait changé.
3. **Une jointure interne vers une table de référence** qui supprime en silence les 2 mois sans objectif du magasin 4.
4. **Une comparaison de périodes de longueurs différentes** présentée comme une évolution : 2026 (8 mois) contre 2025 (12 mois) donne **-28,9 %** ; à périmètre égal, **+15,4 %**.
5. **Un rapport dont les chiffres ne sont pas dans le fichier publié** : toute valeur citée se retrouve dans `chiffres_cites.json`, ou n'est pas publiée.
6. **Un test qui réécrit la requête qu'il contrôle** : ce n'est plus le chiffre du rapport qui est vérifié.
7. **Un moteur annoncé sans l'être** : PostgreSQL et SQL Server sont **cités** dans le module, jamais exécutés ; un rapport qui publie une sortie « PostgreSQL » non produite est faux.

6. Déroulé conseillé (4 séances, environ 8 h)

Séance	Objet	Livrable visé
1	Lire la commande, rejouer le socle, recalculer les 6 totaux, écrire les en-têtes des 12 rapports	P1 (en-têtes)
2	Écrire R01 → R06, avec sorties réelles et totaux de contrôle	P1
3	Écrire R07 → R12 (trous d'objectif, cohortes pondérées, RFM départagé, tableau croisé, contrôle croisé)	P1
4	Le jeu de tests, la note de lecture, l'échange de relecture avec un pair	P2, P3, P4

7. Comment vous corriger vous-même

1. **Les totaux** : `python3 tools/controle_sql_M11.py` doit rendre 18 lignes au vert sur le socle livré ; vos 12 tests s'ajoutent à la suite, dans le même format.
2. **Les sorties** : recopiez les sorties réelles de votre atelier, jamais une sortie approximative ; un tableau publié avec des valeurs arrondies « à la main » perd le contrôle.
3. **La relecture croisée** : échangez votre P4 avec un camarade. Le test est simple : peut-il **rejouer** trois de vos rapports et retrouver vos chiffres sans vous demander d'explication ? Si oui, la bibliothèque est bonne — c'est la définition du projet.

Le critère de réussite du projet. Un lecteur qui n'a jamais vu le socle ouvre votre dossier, lit l'en-tête d'un rapport, exécute la requête, obtient votre chiffre, lance le test qui le vérifie, et comprend en une phrase à quoi le rapport sert. Douze fois de suite.

Projet M12.P — « Le tableau de bord que personne n'ouvrirait »

Module : M12 — Fondamentaux de la Business Intelligence · 4 livrables · 20 points · seuil de réussite 13 · environ 10 h de travail.

Le mandat. Une PME de distribution a payé **7 940 000 FCFA** pour un tableau de bord de **41** indicateurs répartis en **7** onglets. Onze semaines plus tard, le contrat de maintenance est résilié : **14** personnes avaient reçu l'adresse, **4** l'ont ouverte le premier jour. La direction vous mandate pour **diagnostiquer** ce dispositif et **proposer une reprise**. Sa consigne, textuelle : « pas plus de **10** indicateurs, chacun avec sa définition écrite, son responsable, sa fréquence, son seuil et son contre-indicateur. Et une réponse à la seule question qui compte : que fait-on quand il passe au rouge ? »

1. Énoncé

Le matériel fourni

Tout est dans 03_exercices/dossier_M12/ :

Pièce	Contenu
etude_avant.md	les 12 pages du dossier : commande, devis, liste des 41 indicateurs, démarrage, sources de la discorde, coût de l'abandon, questions jamais posées, mandat de reprise
socle_m12.sql	le socle : 8 tables (ventes, commande, encaissement, stock mensuel, ruptures, logistique, produit, magasin) et la vue de marge
connexion.py	l'ouverture de la base, identique à celle des chapitres
ATTENDU.json	les mesures de référence du socle, pour vérifier vos calculs
6 fichiers .csv	les cinq sources opérationnelles ajoutées par M12 : coût d'achat, stock, ruptures, commandes, encaissements, logistique

Règle du mandat : un diagnostic qui ne mesure rien est un avis. Chaque affirmation de votre dossier doit porter un chiffre, et chaque chiffre doit se recalculer avec les fichiers fournis.

Les six totaux de contrôle

Avant de commencer, vérifiez que votre socle répond :

Contrôle	Valeur attendue
lignes de vente	240 000 dont 237 191 hors retours
chiffre d'affaires net	15 595 154 955 FCFA (brut : 15 419 985 157)
clients	23 497 pour 145 212 tickets
marge brute	3 847 989 780 FCFA, soit 29,12 % hors taxes
ruptures	2 428 couples produit-magasin-mois, 13 129 jours
encours	1 202 550 590 FCFA sur 1 272 factures ouvertes

Si l'un de ces six contrôles ne tombe pas juste, ne poursuivez pas : reprenez le socle (\$7).

2. Les dix indicateurs imposés (le catalogue du mandat)

La reprise doit couvrir ces **10** indicateurs, ni plus ni moins :

1. chiffre d'affaires net
2. marge brute
3. taux de rupture
4. rotation de stock
5. panier moyen
6. taux de retour
7. taux de service
8. taux de recouvrement
9. coût logistique unitaire
10. part de marché interne

Chacun doit être livré avec sa **carte de définition** complète : **7** champs, soit **70** cases à remplir. Une case vide compte comme une case fausse.

3. Les 4 livrables

P1 — Le diagnostic mesuré (6 points)

Format : 3 pages maximum, plus un tableau de mesures. Le diagnostic répond, chiffres en main, à six questions :

1. **Combien a coûté le dispositif ?** Ventilez les **7 940 000** FCFA par poste et calculez le coût par indicateur livré, le coût par KPI du mandat et le coût par lecteur du premier jour.
2. **Pourquoi les chiffres se contredisaient-ils ?** Reprenez les **3** sources de la discorde — CA net contre CA brut, deux coûts d'achat, deux dénominateurs du panier moyen — et donnez, pour chacune, les deux valeurs mesurées sur le socle.
3. **Que valait la couche sémantique ?** Montrez, sur un indicateur au choix, que le socle permet plusieurs définitions défendables, et mesurez-les.
4. **Pourquoi les gens ont-ils cessé de l'ouvrir ?** Mesurez l'adoption : destinataires, ouvertures, évolution hebdomadaire, et la chute de la deuxième semaine.
5. **Quels étages de la chaîne ont été construits, et lesquels ont été sautés ?** Situez les **7** étages et donnez la mesure du socle pour chacun.
6. **Quelles causes d'échec retrouvez-vous ?** Citez-en au moins cinq parmi les **10** du cours, chacune avec son symptôme mesuré.

Ce qui fait la note : chaque affirmation porte un chiffre recalculable ; aucune cause n'est affirmée sans mesure ; le coût par lecteur est calculé et interprété.

P2 — La carte de définition de 10 KPI (8 points)

Format : 10 fiches. C'est le livrable central du module, et le document que la direction signera. Pour chacun des **10** indicateurs du §2, remplissez les **7** champs :

Champ	Exigence
Formule	numérateur, dénominateur, base de calcul, exclusions — sans ambiguïté
Source	la table et le fichier exacts, avec les cas particuliers connus
Granularité	ce que représente une ligne du résultat
Fréquence	quand l'indicateur est juste, et à quelle date il est arrêté

Champ	Exigence
Responsable	une fonction nommée, pas un service
Seuil d'alerte	une valeur, et la mesure de ce qu'elle déclenche
Contre-KPI	l'indicateur publié à côté, avec sa valeur actuelle

Deux exigences de fond :

- **chaque valeur actuelle est mesurée sur le socle** (tableau récapitulatif de vos **10** valeurs, avec la requête ou le script qui les produit) ;
- **au moins trois cartes justifient un choix de définition** en comparant deux versions mesurées — par exemple CA net (**15 595 154 955** FCFA) contre CA brut (**15 419 985 157**), ou taux de service sur commandes livrées (**81,0 %**) contre toutes commandes (**78,2 %**).

Ce qui fait la note : les 7 champs sont remplis pour les 10 indicateurs ; les valeurs sont mesurées et reproductibles ; les choix de définition sont argumentés par des chiffres ; un contre-KPI accompagne chaque indicateur et sa valeur est donnée.

P3 — Le plan de reprise (4 points)

Format : 2 pages, dont un tableau de charges. Reprenez le budget du dispositif raté et proposez une répartition des **30** jours entre les étages de la chaîne (sources, extraction, stockage, modèle, sémantique, écran, diffusion), en justifiant chaque poste par un risque mesuré. Le plan doit préciser :

- **ce qu'on arrête** : au moins trois pratiques du dossier (par exemple la publication d'indicateurs sans responsable, la définition locale du CA, l'onglet « Écarts » sans cible) ;
- **ce qu'on garde** malgré l'échec : au moins une chose utile du dispositif précédent ;
- **le calendrier** : nombre de semaines, jalons, et la date de la première revue d'usage ;
- **les destinataires** : qui reçoit quoi, à quelle fréquence, et l'objectif d'adoption chiffré à J+30.

Ce qui fait la note : la répartition est argumentée ; les charges sont cohérentes avec le total ; l'objectif d'adoption est chiffré et mesurable.

P4 — La page « que fait-on quand c'est rouge ? » (2 points)

Format : 1 page. C'est la réponse à la seule question laissée sans réponse par le dispositif précédent. Pour les **10** indicateurs, une ligne par indicateur, avec quatre colonnes : le seuil, l'action exacte, la personne qui la déclenche, le délai. Les actions doivent avoir un **volume** : une liste de produits, un nombre de factures, un montant — jamais « surveiller » ou « relancer si nécessaire ».

Ce qui fait la note : les dix seuils ont une action ; les actions ont un volume et un destinataire ; la page tient en une page.

4. Barème

Livrable	Points	Seuil de détail
P1 Diagnostic mesuré	6	1 point par question, si la réponse est chiffrée et recalculable
P2 Carte de 10 KPI	8	0,8 point par indicateur, dont 0,5 pour la complétude des 7 champs et 0,3 pour la cohérence des valeurs
P3 Plan de reprise	4	1 point par exigence : arrêts, conservation, calendrier, adoption
P4 Page « c'est rouge »	2	0,2 point par indicateur, si le seuil et l'action sont complets

Livrable	Points	Seuil de détail
Total	20	seuil de réussite : 13

Ce que le barème sanctionne : un chiffre repris du dossier sans être recalculé (**-1** par chiffre) ; une carte de définition incomplète (**-0,3** par champ vide) ; une action sans volume (**-0,2**) ; un contre-KPI absent ou décoratif (**-0,3**).

5. Ce qui fait rejeter un livrable

1. **Un diagnostic sans mesure** : les affirmations du dossier ne portent pas de chiffre vérifiable.
2. **Des cartes recopiées des chapitres** : les valeurs doivent être **mesurées** sur le socle, pas reprises du cours.
3. **Des définitions ambiguës** : « taux de retour » sans préciser numérateur ni dénominateur.
4. **Un seul indicateur sans responsable** : **10** indicateurs, **10** fonctions nommées.
5. **Des seuils sans action** ou des actions sans volume.
6. **Une page P4 qui dépasse une page** : l'exercice porte sur la concision autant que sur le fond.
7. **Un contre-KPI décoratif** : il doit être mesuré sur le socle, avec sa valeur actuelle.
8. **Plus de 10 indicateurs** : le mandat est un plafond, pas une suggestion.

6. Déroulé conseillé (4 séances, environ 10 h)

Séance	Durée	Travail	Livrable visé
1	2 h	lecture du dossier, contrôle des six totaux, premiers calculs du diagnostic	P1 (questions 1 à 3)
2	3 h	rédaction des cartes de définition, mesure des valeurs actuelles, choix de définition argumentés	P2 (indicateurs 1 à 6)
3	3 h	fin des cartes, seuils éprouvés par la mesure, contre-KPI mesurés	P2 (indicateurs 7 à 10)
4	2 h	plan de reprise, répartition des jours, page « c'est rouge », relecture	P3 et P4

7. Comment vous corriger vous-même

1. **Les six totaux** du §1 tombent juste, et votre socle se reconstruit à l'identique deux fois de suite.
2. **Chaque chiffre du diagnostic** se recalcule : aucun n'est recopié du dossier.
3. **Chaque carte a 7 champs** : comptez-les ; s'il en manque un, la carte n'est pas finie.
4. **Chaque seuil a été essayé** : vous savez dire combien de lignes il met en alerte.
5. **Chaque contre-KPI a une valeur mesurée** et une phrase expliquant ce qu'il protège.
6. **La page P4 tient en une page** et chaque action porte un volume, une personne et un délai.
7. **L'objectif d'adoption est chiffré** et comparable aux **4** lecteurs sur **14** du dispositif précédent.

Le test final. Faites lire votre P2 par quelqu'un qui ne connaît pas le dossier : s'il peut recalculer un indicateur et dire qui l'améliore ou le dégrade, la carte est bonne. S'il pose une question sur une définition, elle est incomplète.

Les douze evaluations de module

M01 a M12 — quiz, exercices et études de cas, avec leurs
baremes

Analyse de données appliquée & Business Intelligence
12 partie(s) assemblée(s) · document de production

Table des matières

ÉVALUATION DU MODULE M01 — RÉCUPÉRATION, QUIZ, EXERCICES, ÉTUDE DE CAS, CORRIGÉS	7
A. Questions de récupération (non notées, à traiter avant le quiz)	7
B. Quiz — 15 questions, noté sur 20	8
Frontières du vocabulaire (Q1 à Q4)	8
Structure et lecture d'une table (Q5 à Q9)	8
Types, formats, encodages (Q10 à Q13)	8
Posture et méthode (Q14 et Q15)	8
C. Exercices pratiques (4) — séries E1 et E2, auto-correction	9
D. Étude de cas — « Le patron veut savoir des choses » (20 points, seuil 12)	9
Corrigé de l'évaluation (à consulter après traitement complet)	10
B. Quiz — réponses et explications	10
C. Exercices — corrigés détaillés	11
D. Étude de cas — cinq cadrages attendus	12
Total et décision d'orientation	13
ÉVALUATION M02 — « STATISTIQUES DESCRIPTIVES ET INFÉRENCE »	14
A • Questions de récupération (non notées)	14
B • Quiz (20 questions • 20 points)	14
Bloc 1 — Population, échantillon, biais (Q1 à Q5)	15
Bloc 2 — Centres (Q6 à Q8)	15
Bloc 3 — Dispersion et positions (Q9 à Q12)	15
Bloc 4 — Formes et liaisons (Q13 à Q16)	15
Bloc 5 — Estimation et tests (Q17 à Q20)	16
C • Exercices pratiques (2 séries • 55 min • auto-correction exigée)	16
D • Étude de cas (20 points • seuil 12 • 60 min)	17
Le dossier	17
Les quatre livrables demandés	17
Ce que le correcteur attend, ligne à ligne	17
Corrigé	17
B • Quiz — réponses et explications	10
C • Exercices pratiques — corrigés	18
D • Étude de cas — éléments de réponse attendus	19
Total et décision d'orientation	13
ÉVALUATION M03 — « EXCEL, DU CLIC À LA MÉTHODE »	21
A • Questions de récupération (non notées)	14
B • Quiz (15 questions • 15 points)	21
Bloc 1 — Types, formats, saisie (Q1 à Q4)	22
Bloc 2 — Adresses et erreurs affichées (Q5 à Q8)	22
Bloc 3 — Agrégats, critères, dates (Q9 à Q12)	22
Bloc 4 — Recherche, assemblage, TCD (Q13 à Q15)	22
C • Exercices pratiques (4 exercices • 20 points)	23
D • Étude de cas — « Le tableau de bord du magasin 5 est-il pilotable ? » (20 points, seuil 12)	24
Éléments de réponse attendus	24

Corrigé du quiz (B)	25
Éléments de corrigé des exercices (C)	26
Total et décision d'orientation	13
ÉVALUATION M04 — « QUALITÉ, PRÉPARATION ET DOCUMENTATION DES DONNÉES »	28
A • Questions de récupération (non notées)	14
B • Quiz (15 questions • 15 points)	21
Bloc 1 — Types et normalisation (Q1 à Q4)	29
Bloc 2 — Doublons et clés (Q5 à Q8)	29
Bloc 3 — Règles et valeurs aberrantes (Q9 à Q12)	29
Bloc 4 — Documentation et protection (Q13 à Q15)	30
Corrigé du quiz	30
C • Exercices pratiques (50 min • /20)	30
D • Étude de cas (60 min • /20, seuil 12) — « Le stock indiqué est-il crédible ? »	31
Lecture des résultats	32
ÉVALUATION M05 — « LE LANGAGE COMMUN DES TRANSFORMATIONS »	33
A • Questions de récupération (non notées)	14
B • Quiz (15 questions • 15 points)	21
Bloc 1 — Le langage commun (Q1 à Q3)	34
Bloc 2 — Power Query (C02) (Q4 à Q6)	34
Bloc 3 — SQL DuckDB (C03) (Q7 à Q9)	34
Bloc 4 — pandas (C04) (Q10 à Q12)	35
Bloc 5 — Arbitrage (C05) (Q13 à Q15)	35
Corrigé du quiz	30
C • Exercices pratiques (50 min • /20)	30
D • Étude de cas (60 min • /20, seuil 12) — « Un pipeline, trois moteurs — qui dit la même chose ? »	36
Lecture des résultats	32
ÉVALUATION M06 — « BASES DE DONNÉES RELATIONNELLES : COMPRENDRE AVANT DE REQUÊTER »	38
A • Questions de récupération (non notées)	14
B • Quiz (15 questions • 15 points)	21
Bloc 1 — Pourquoi une base (C01) (Q1 à Q3)	39
Bloc 2 — Tables, colonnes, schéma (C02) (Q4 à Q6)	39
Bloc 3 — Clés et relations (C03) (Q7 à Q9)	39
Bloc 4 — DuckDB / SQLite / PostgreSQL (C04) (Q10 à Q12)	39
Bloc 5 — Paysage NoSQL / DWH / datalake (C05) (Q13 à Q15)	40
C • Exercices pratiques (2 exercices • 20 points)	40
Exercice 1 — Vérifier l'intégrité référentielle (10 points)	40
Exercice 2 — Convertir 142 montants en texte (10 points)	40
D • Étude de cas (1 cas • 20 points • seuil 12/20)	40
« Sept tables pour une quincaillerie »	40
Barème récapitulatif	41
ÉVALUATION M07 — « SQL : INTERROGER, AGRÉGER, REJOINDRE »	42

A • Questions de récupération (non notées)	14
B • Quiz (20 questions • 20 points)	14
Bloc 1 — Lire et filtrer (C01–C02) (Q1 à Q4)	43
Bloc 2 — Trier et agréger (C03–C04) (Q5 à Q8)	43
Bloc 3 — Résumer par groupe (C05) (Q9 à Q12)	43
Bloc 4 — Catégoriser (C06) (Q13 à Q16)	43
Bloc 5 — Joindre et composer (C07–C08) (Q17 à Q20)	44
C • Exercices chiffrés (35 exercices • 35 points • auto-corrigés)	44
D • Étude de cas — « Le rapport de la direction commerciale » (75 min • /20, seuil 12)	45
Corrigés ligne à ligne	46
B • Quiz — réponses	46
C • Exercices — valeurs de référence (clés de traçabilité)	46
D • Étude de cas — points de repère du corrigé	47
ÉVALUATION M08 — « PYTHON POUR L'ANALYSE DE DONNÉES »	48
A • Questions de récupération (non notées)	14
B • Quiz (15 questions • 15 points)	21
Bloc 1 — Environnement et types (C01–C02) (Q1 à Q3)	49
Bloc 2 — Contrôle du flux et conteneurs (C03–C04) (Q4 à Q6)	49
Bloc 3 — Organiser le code (C05) (Q7 à Q9)	49
Bloc 4 — Vecteurisation et lecture (C06–C07) (Q10 à Q12)	49
Bloc 5 — Agréger et livrer (C08) (Q13 à Q15)	50
C • Exercices « prédisez la sortie » (15 exercices • 15 points • auto-corrigés)	50
D • Exercices à rendre (5 scripts courts • 5 × /2) + E4 « ce script tourne mais ses chiffres sont faux » (/10)	51
D1 — Les deadlines impossibles (C03, C07) — /2	51
D2 — La fonction des centimes (C05) — /2	51
D3 — Les 4 tranches de montants (C06) — /2	51
D4 — L'année 2025 du magasin 4 (C07) — /2	52
D5 — Le classeur en deux feuilles (C08) — /2	52
E4 — « Ce script tourne mais ses chiffres sont faux » (/10)	52
E • Étude de cas — « Le script du stagiaire » (75 min • /20, seuil 12)	54
Les trois exécutions, sorties réelles (exécutées le 20/09/2026)	54
Corrigés ligne à ligne	46
B • Quiz — réponses	46
C • Exercices — valeurs de référence (clés de traçabilité)	46
D • Exercices rendus — sorties de référence	56
E4 — Les trois corrections	57
E • Étude de cas — points de repère du corrigé	57
ÉVALUATION M09 — « ANALYSE EXPLORATOIRE DE DONNÉES »	58
A • Questions de récupération (non notées)	14
B • Quiz (15 questions • 15 points)	21
Bloc 1 — Le protocole et l'audit (C01–C02) (Q1 à Q3)	59
Bloc 2 — Tendances et périodes comparables (C03) (Q4 à Q6)	59
Bloc 3 — Anomalies et preuves (C04) (Q7 à Q9)	59
Bloc 4 — Relations, segments, groupes (C05) (Q10 à Q12)	59
Bloc 5 — Graphiques et note d'EDA (C06) (Q13 à Q15)	60
C • Exercices « prédisez la sortie » (15 valeurs • 15 points • auto-corrigé)	60

D • Exercices à rendre (20 points)	61
E • Étude de cas — « les 5 questions du directeur » (20 points • seuil 12)	61
Correction détaillée	62
A • Réponses de récupération	62
B • Corrigé du quiz (1 pt par question)	62
C • Corrigé des quinze valeurs	62
D • Corrigé des exercices à rendre	63
E • Corrigé de l'étude de cas — les 5 questions attendues	64
ÉVALUATION M10 — « DATA VISUALIZATION »	65
A • Questions de récupération (non notées)	14
B • Quiz (15 questions • 15 points)	21
Bloc 1 — La perception et le choix du graphique (C01—C02) (Q1 à Q3)	65
Bloc 2 — Couleurs, contraste, accessibilité (C03) (Q4 à Q6)	66
Bloc 3 — Écrire un graphique (C04) (Q7 à Q9)	66
Bloc 4 — Les 10 erreurs (C05) (Q10 à Q12)	66
Bloc 5 — La séquence et les outils (C06—C07) (Q13 à Q15)	66
C • « Prédisez la mesure » (10 valeurs • 10 points • auto-corrigé)	67
D • Exercices à rendre (20 points)	61
E • Étude de cas — « ce graphique a déclenché une décision fausse » (20 points • seuil 12)	68
Correction détaillée	62
A • Réponses de récupération	62
B • Corrigé du quiz (1 pt par question)	62
C • Corrigé des dix prédictions	69
D • Corrigé des exercices à rendre	63
E • Corrigé de l'étude de cas — les cinq attendus	70
ÉVALUATION M11 — « SQL AVANCÉ POUR LA BI »	72
A • Questions de récupération (non notées)	14
B • Quiz (20 questions • 20 points • 1 point par question)	72
Bloc 1 — Quelle requête pour quelle question (C01—C05) (Q1 à Q5)	72
Bloc 2 — Prédisez le résultat (Q6 à Q10)	73
Bloc 3 — Diagnostic (Q11 à Q15)	73
Bloc 4 — Vocabulaire (Q16 à Q20)	73
C • Exercices à rendre (20 points • 5 exercices, dont 2 de type E4)	74
E1 — Le top 3 par famille, deux fois (niveau 3 • 3 points)	74
E2 — Les deux périmètres de la concentration (niveau 3 • 3 points)	74
E3 — Le suivi des objectifs, trous compris (niveau 3 • 4 points)	74
E4 — « Ce rapport annonce un chiffre d'affaires net » (type E4 • niveau 4 • 5 points)	74
E5 — « Ce classement change de réponse à chaque exécution » (type E4 • niveau 4 • 5 points)	75
D • Auto-test SQL (10 points • 18 requêtes • auto-corrigé)	75
E • Étude de cas — « 10 % des clients font-ils 60 % du CA ? » (20 points • seuil 12)	75
La commande	75
Les quatre productions attendues	75
Le résultat attendu (à retrouver, pas à recopier)	76
Barème de l'étude de cas	76
Correction détaillée	62
A • Réponses de récupération	62

B · Corrigé du quiz (1 point par question)	76
C · Corrigé des exercices à rendre	77
D · Corrigé de l'auto-test	78
E · Corrigé de l'étude de cas — les quatre attendus	78
Barème global et seuils	78
ÉVALUATION M12 — « FONDAMENTAUX DE LA BUSINESS INTELLIGENCE »	79
A · Questions de récupération (non notées)	14
B · Quiz (15 questions · 20 points)	79
Bloc 1 — Définir (Q1 à Q5)	79
Bloc 2 — Mesurer (Q6 à Q10)	79
Bloc 3 — Décider (Q11 à Q15, 2 points chacune)	80
C · Exercices de rédaction de KPI (20 points · 2 exercices · PATCH_8 : les énoncés reformulent, ils ne	80
R1 — Le taux de rupture (10 points de rédaction)	80
R2 — La part des clients non identifiés (10 points)	81
D · Étude de cas — « la direction veut un tableau de bord, elle a déjà tort de le vouloir comme ça » (30 points · seuil 18)	81
La commande	75
Le résultat attendu (à retrouver, pas à recopier)	76
Barème de l'étude de cas	76
Correction détaillée	62
A · Réponses de récupération	62
B · Corrigé du quiz	82
C · Corrigé des exercices de rédaction	83
D · Corrigé de l'étude de cas	83

Évaluation du module M01 — récupération, quiz, exercices, étude de cas, corrigés

Durée totale conseillée : 2 h 30 · Deux notes indépendantes, ni l'une ni l'autre négociable : quiz sur 20 (seuil de passage 14/20) et étude de cas sur 20 (seuil 12) · Le projet de module est noté séparément, sur 20, seuil 13 (03_exercices/M01_projet.md)

Le module est validé quand les trois notes sont au seuil. Chaque instrument répond à une question différente, et c'est pour cela qu'ils ne se compensent pas : le quiz dit si vous savez, les exercices si vous savez faire, l'étude de cas si vous savez cadrer avant de faire, le projet si vous savez livrer.

Instrument (barèmes du manuel, partie B.6 de l'architecture)	Barème	Durée	Ce qui est vérifié
A. Récupération — 7 questions	non noté, auto-corrigé	15 min	la mémoire des sept chapitres, sans relire
B. Quiz — 15 questions	/20, seuil 14/20	30 min	le vocabulaire, les types, la structure d'une table
C. Exercices pratiques — 4 exercices	auto-correction par résultat attendu chiffré	45 min	le geste : compter, convertir, contrôler
D. Étude de cas — 5 demandes	grille à 5 critères, /20, seuil 12	60 min	le cadrage, avant tout calcul

Matériel autorisé : le fichier `ventes_magasin5_2025.csv` (489 lignes), `tarif_fournisseur_peinture.csv`, un tableur, un éditeur de texte. Matériel interdit : le corrigé, les notes du projet M01.P (l'évaluation sert à vérifier que le savoir est **en vous**, pas dans votre dossier).

A. Questions de récupération (non notées, à traiter avant le quiz)

Répondez en une phrase, sans relire. Toute réponse hésitante = relecture ciblée du chapitre indiqué.

1. Que représente une ligne du fichier de l'atelier ? (C03 · un article d'un ticket, pas une vente)
2. Quelle est la différence entre un format et un type ? (C04 · l'apparence vs la nature de la valeur, donc ses opérations permises)
3. Combien de lignes a le fichier, et combien de tickets ? (C03 · 489 et 316)
4. Que fait `=NB()` que `=NBVAL()` ne fait pas ? (C04 · il ignore le texte : la différence des deux mesure les montants mal typés)
5. Un fichier en `;` encodé en Windows-1252 est-il cassé ? (C05 · non : il est lisible, c'est la lecture qui est mal réglée)
6. Citez les trois documents qui rendent un chiffre défendable dans trois ans. (C06 · dictionnaire de données, journal de transformation, manifeste d'archivage)
7. Quels sont les cinq composants d'une question analytique ? (C02/C07 · objet, unité + définition, comparaison, périmètre, critère de réussite)

B. Quiz — 15 questions, noté sur 20

Barème : 10 questions à 1 point (Q3, Q5, Q6, Q7, Q8, Q10, Q12, Q13, Q14, Q15) et 5 items-piège à 2 points (Q1, Q2, Q4, Q9, Q11), où la justification écrite est exigée : $10 \times 1 + 5 \times 2 = 20$. Une réponse juste sans justification sur un item-piège vaut 1 point sur 2 ; une justification sans réponse vaut 0. Les items-piège ne testent pas la mémoire des mots mais la frontière entre eux. Répondez par Vrai/Faux ou choisissez l'option ; le corrigé commenté suit immédiatement, à ne consulter qu'après avoir tout traité.

Frontières du vocabulaire (Q1 à Q4)

- Q1.** « Une donnée est une information. » Vrai ou faux ?
- Q2.** « Une colonne `id_client` remplie de nombres est une variable quantitative. » Vrai ou faux ?
- Q3.** « Un CSV ne contient pas de types de données. » Vrai ou faux ?
- Q4.** « Une fois formatée en date, une cellule contient une date. » Vrai ou faux ?

Structure et lecture d'une table (Q5 à Q9)

- Q5.** Sur le fichier de l'atelier, combien de tickets distincts pour 489 lignes ? a) 489 b) 480 c) 316 d) 145
- Q6.** Quelle colonne, à elle seule, peut servir de clé primaire de ce fichier ? a) `n_ticket` b) `produit` c) `date` d) aucune
- Q7.** Vous voulez compter « le nombre d'articles facturés » et « le nombre de ventes ». Quelle paire de comptages ? a) lignes, tickets distincts b) tickets distincts, lignes c) clients distincts, produits distincts d) montants distincts, lignes
- Q8.** Le fichier `ventes_propres.csv` du socle contient : a) 243 360 lignes b) 240 000 lignes c) 489 lignes d) 1 339 lignes
- Q9.** Un tableau croisé affiche un total exactement double du total de la colonne. Cause la plus probable : a) une ligne de sous-total importée comme donnée b) un encodage différent c) des dates au format français d) un identifiant client en texte

Types, formats, encodages (Q10 à Q13)

- Q10.** Combien de lignes de `montant_ttc`, dans le fichier de l'atelier, ne sont pas des nombres ? a) 0 b) 3 c) 486 d) 489
- Q11.** Après nettoyage complet, le CA TTC de l'extrait passe de 36 339 317 à 36 073 185 FCFA. Cet écart de 266 132 FCFA provient de : a) lignes en double b) retours négatifs c) TVA mal appliquée d) arrondis successifs
- Q12.** Le fichier du fournisseur compte 42 lignes. Combien de références ? a) 42 b) 41 c) 38 d) 39
- Q13.** Un fichier dont les accents s'affichent Ã© a été : a) corrompu par l'e-mail b) écrit en UTF-8 et lu en Windows-1252 (ou l'inverse) c) enregistré au format XLS d) tronqué

Posture et méthode (Q14 et Q15)

- Q14.** Le directeur demande « est-ce que ça marche bien ? ». La **première** action correcte est : a) charger le fichier dans le tableur b) construire un graphique de CA mensuel c) demander quelle décision dépend de la réponse d) proposer Power BI
- Q15.** Sur un échantillon de 480 lignes représentant 40 lignes par mois, vous affirmez « décembre est le meilleur mois de 2025 au magasin 5, la preuve d'une forte saisonnalité ». Ce raisonnement est : a) correct, le chiffre est exact b) incorrect : l'extrait n'est pas la population, et

un classement sur échantillon ne prouve pas une saisonnalité c) correct si le total de contrôle est bon d) incorrect, car il faudrait inclure 2026

C. Exercices pratiques (4) — séries E1 et E2, auto-correction

Ces quatre exercices ne sont pas notés au quiz : ils sont **validés par le résultat attendu**, parce que dans le métier un résultat ne se discute pas, il se vérifie. Traitez les quatre, comparez vos chiffres à l'attendu, puis lisez le corrigé. Un attendu manqué de peu n'est pas un demi-exercice réussi : cherchez l'écart, c'est là qu'est l'apprentissage.

Exercice C.1 (série E1). Ouvrez `ventes_magasin5_2025.csv` dans un éditeur de texte. Notez : le séparateur, le nombre de colonnes de la ligne d'en-tête, le contenu de la ligne 2 en trois mots, et une anomalie visible **sans calculer**. (attendu : ; · 13 · ticket, date, montant en texte · par exemple l'espace dans 56 658 ou la date 12/01/2025)

Exercice C.2 (série E1). Dans le tableur, importez avec toutes les colonnes en texte, puis donnez, par trois formules : le nombre de lignes de données, le nombre de valeurs numériques de `montant_ttc`, le nombre de lignes dont `date` contient un /. (attendu : `=NBVAL(A2:A490)` = 489 · `=NB(M2:M490)` = 3 · `=NB.SI(B:B;"?*//*")` = 65)

Exercice C.3 (série E2). Produisez le fichier nettoyé attendu : 480 lignes, dates en ISO, `montant_ttc` en nombre, `quantite` rétablie sur les 7 lignes aberrantes ($\div 1\,000$), `client` conservé tel quel, y compris les 18 vides et les 0 de comptoir. Donnez le total de contrôle. (attendu : 36 073 185 FCFA ; la comparaison avec `ventes_magasin5_2025_ATTENDU.csv` est le contrôle de forme)

Exercice C.4 (série E2). Convertissez le fichier du fournisseur en un tableau propre de 38 lignes × 6 colonnes, prix en nombre, TVA conservée. Donnez le prix moyen et le prix maximal. (attendu : 11 155 FCFA en moyenne, 34 826 FCFA au maximum, TVA 0,18 sur toutes les lignes ; encodage cp1252, 3 lignes de chapeau ignorées, virgule décimale traitée)

D. Étude de cas — « Le patron veut savoir des choses » (20 points, seuil 12)

Contexte. Samedi matin, marché de Kaya. Le directeur de Sahel Distribution SA vous retient trente secondes près du parking et dit, d'un trait, les cinq phrases ci-dessous. Vous n'avez ni base, ni tableau de bord, ni temps : vous avez un carnet. Consignez, pour **chacune** des cinq demandes, la décision visée et la question analytique complète (objet, unité + définition, comparaison, périmètre, critère de réussite), ainsi que la ou les tables à ouvrir et **ce qui manquera** dans le jeu de données pour y répondre.

1. « Les prix de la concurrence, il faut qu'on s'aligne, tu crois qu'on est trop chers ? »
2. « Bationo, il vend trop bien, faut le augmenter — ou pas. Regarde. »
3. « On m'a dit qu'en janvier on ne vend rien, et moi je trouve qu'en janvier ça bosse. »
4. « Si on fermait le dépôt le dimanche, ça changerait quoi ? »
5. « Le magasin 5, il me plaît. Combien il rapporte ? »

Contraintes de rendu. Cinq blocs de six lignes maximum, format imposé :

[n°] Demande (verbatim, 1 ligne)	
Décision visée · Destinataire · Échéance	(1 ligne)
Question analytique	(1-2 lignes)
Tables et colonnes à ouvrir	(1 ligne)
Manquant / limite qui empêche de conclure	(1 ligne)
Outil choisi + motif en 4 mots	(1 ligne)

Grille de notation — 5 critères, 4 points par demande (5 × 4 = 20 points, seuil de validation 12). Pour chaque demande : question analytique comportant les cinq composants (1,5) · décision, destinataire et échéance nommés (1) · limite réellement limitée et non générique (0,5) · choix d'outil justifié par une contrainte et non par un goût (1). Un bloc qui répond à la place de cadrer est noté 0 : on ne demande pas ici le chiffre, on demande la question. Le dixième point de la grille est un point de **cohérence d'ensemble** : les cinq demandes doivent se ressembler (même gabarit, mêmes colonnes appelées), parce qu'un cadrage qui change de forme d'un dossier à l'autre n'est pas un cadrage, c'est une humeur du moment.

Corrigé de l'évaluation (à consulter après traitement complet)

B. Quiz — réponses et explications

Q1 — Faux. Une donnée est un fait brut ; elle devient information quand elle est replacée (quoi, qui, quand, unité). 56 658 n'est pas un chiffre d'affaires ; « 56 658 FCFA TTC, ligne 1 du ticket du 01/01/2025, magasin 5 » en est une. Le piège est réel : dans le langage courant, les deux mots sont interchangeables ; dans le métier, non.

Q2 — Faux. C'est un **identifiant**, donc une valeur nominale : le calculer, le moyener, le trier par ordre numérique n'a aucun sens. Le test : « une soustraction a-t-elle un sens ? » 10 857 - 3 312 ne veut rien dire.

Q3 — Vrai. Un CSV est du texte ; le type apparaît quand un outil le déclare ou le devine. C'est pourquoi les identifiants doivent être explicitement lus comme texte (et pourquoi le 075845 final du ticket T05-250101-075845 deviendrait 75845 si un tableur le lisait comme un nombre).

Q4 — Faux. Le format change l'affichage, pas la nature de la valeur. Une cellule affichant 12/01/2025 peut contenir le texte « 12/01/2025 » (et refuser tout calcul de mois) ou le nombre 45669 (le douzième jour de l'année 2025 compté depuis l'origine du tableur, qui est le 00/01/1900 : une vraie date, stockée en numéro de série). Le contrôle : =ESTNUM() puis =EST-DATE().

Q5 — c) 316. 480 lignes propres pour 316 tickets ; 489 est le nombre de lignes de l'énoncé, 145 le nombre de produits distincts.

Q6 — d) aucune. n_ticket se répète (1 à 4 lignes) ; produit et date aussi. L'unicité demande le couple (n_ticket, produit).

Q7 — a). Les lignes = les articles facturés, les tickets distincts = les ventes. Confondre les deux surestime le nombre de ventes de 55 % sur ce fichier.

Q8 — b) 240 000 lignes. ventes_brutes.csv en contient 243 360 ; 489 est l'extrait de l'atelier, 1 339 la table dim_date.

Q9 — a). Un sous-total importé comme donnée double exactement le total. Les autres options changent l'affichage ou la ponctuation, pas la somme. Vérification : compter les lignes dont la colonne-clé est vide.

Q10 — c) 486 sur 489 (3 valeurs seulement sont numériques). Le diagnostic complet : `=NB()` = 3, `=NBVAL()` = 489, la soustraction donne le nombre de défauts.

Q11 — a). Les 9 lignes en double injectées par l'export de la caisse. Le calcul : $36\,339\,317 - 36\,073\,185 = 266\,132$ FCFA, soit +0,74 % de surévaluation si l'on ne nettoie rien. La TVA, les retours et les arrondis sont cohérents dans ce fichier.

Q12 — c) 38 (42 lignes – 3 de chapeau – 1 d'en-tête). Un candidat qui répond 42 a importé le document, pas les données.

Q13 — b. Le fichier est intact, la lecture est mal réglée. Réflexe : changer l'encodage à **l'import**, jamais réenregistrer le brut.

Q14 — c). Avant l'outil et avant le graphique : la décision. Un tableau de bord sans décision associée ne sera pas ouvert après deux semaines.

Q15 — b). Le chiffre est exact (décembre 5 292 517 FCFA, mai 1 258 801, ratio 4,2), l'inférence ne l'est pas : l'extrait contient 40 lignes par mois, tous les mois ne sont donc pas représentés proportionnellement à leur activité réelle. Réponse d'analyste : « sur cet extrait, décembre domine ; pour conclure sur la saisonnalité, il faut le fichier complet du magasin ».

Seuil et rythme. 12 bonnes réponses sur 15 = 6/8 (correctif : 14/20 au total de l'évaluation, pas au quiz seul si l'étude de cas est pleine). Un score ≤ 8 sur 15 indique une relecture ciblée : Q1-Q4 ratées → C02 ; Q5-Q9 → C03 ; Q10-Q13 → C04-C05 ; Q14-Q15 → C06-C07.

C. Exercices — corrigés détaillés

C.1. Séparateur ; · 13 colonnes · ligne 2 = « ticket T05-250101-075845, Plâtre de construction 25 kg — réf 1, 11 unités, montant en texte 56 658 » · anomalie sans calcul : le 0.0 du client de la ligne 3 (comptoir), ou l'espace de milliers dans 51 802, ou une date 12/01/2025 ailleurs. Sont également recevables : les lignes en double visibles au tri sur `n_ticket`, et le libellé « ... — réf 1 » (deux informations dans une cellule). Ce qui ne l'est pas : « il y a des erreurs » (non localisé).

C.2. Les trois formules, et la logique derrière :

<code>=NBVAL(A2:A490)</code>	→ 489	lignes de données (contrôle de l'import : pas de ligne perdue)
<code>=NB(M2:M490)</code>	→ 3	montants réellement numériques
<code>=NB.SI(B2:B490;"?*//*")</code>	→ 65	dates écrites en JJ/MM/AAAA

Le correcteur vérifie la **cohérence des trois** : $489 = 480 + 9$ (doublons) et $486 = 489 - 3$ (montants texte). Un candidat qui obtient 488 lignes a mal importé (première ligne utilisée comme en-tête ou ligne vide finale).

C.3. Recettes, dans l'ordre : (1) import en texte · (2) `ttc_num = CBI(SUPPSPACE(SUBSTITUE(colonne;" ";"")))`, contrôle `=NB(colonne d'erreurs) = 0` · (3) tri sur toutes les colonnes puis suppression des doublons stricts → 480 lignes · (4) détection des dates par le 5^e caractère et reconstruction en ISO des 65 lignes concernées · (5) quantités > 500 divisées par 1 000 (7 lignes, max 14 000 → 14) · (6) `client` laissé en l'état, avec une note : 18 vides, et les 0 = comptoir · (7) total : **36 073 185 FCFA**. Comparaison finale : `=SOMME` du fichier attendu (même fichier, 480 lignes) — égalité demandée à 0 FCFA près, non négociable : si l'écart n'est pas nul, une ligne a été supprimée à tort (le cas habituel : les 7 lignes de quantité, traitées en correction au lieu d'un marquage).

C.4. Import : ;, encodage Windows-1252, 3 lignes sautées, colonne `prix_unitaire` en texte puis `SUBSTITUE(cellule;".";";")` et remplacement de la virgule par le point avant conversion → 38 lignes, 6 colonnes. Moyenne **11 155 FCFA**, maximum **34 826 FCFA**, TVA 0,18 partout. Contrôles demandés : nombre de lignes (38), aucune valeur nulle, et `prix × 1,18` cohérent avec une grille interne si l'apprenant la consulte (bonus de méthode, pas de résultat).

D. Étude de cas — cinq cadrages attendus

Les cinq réponses ci-dessous sont des **modèles de rendu**, pas des réponses uniques : un candidat peut formuler autrement, à condition de conserver les cinq composants et la limite. Les numéros de lignes et de colonnes cités existent dans le socle fourni.

1. « On est trop chers ? » Décision : réviser la grille avant la campagne de décembre (direction commerciale, arbitrage avant le 30/11). Question analytique : « Entre 2023 et 2025, le prix moyen HT des 154 références a-t-il évolué moins vite que leur coût d'achat HT (couts_achat.csv, 1 694 lignes), et dans quelle proportion l'écart de marge brute s'explique par la remise plutôt que par le prix ? » Tables : ventes, produits, couts_achat. Manquant : **le prix des concurrents** — aucune donnée externe dans le jeu ; seule une étude terrain (20 références relevées au marché) le fournirait, à chiffrer comme tâche distincte. Outil : SQL ou tableur selon le volume ; motif : « calcul de taux sur 240 000 lignes ». Le point de lucidité attendu : dire que « trop cher » ne se mesure pas **dans ces données**, et proposer le relevé terrain comme complètement — c'est ce qui distingue un cadrage d'une promesse.

2. « Bationo vend trop bien, faut l'augmenter ? » Décision : politique de prime, direction générale, avant le prochain trimestre. Question analytique : « Sur 2025 (extrait magasin 5), le CA TTC et le **nombre de tickets distincts** par vendeur placent-ils A. Bationo en tête après retrait des 9 doublons, et cet écart tient-il au panier moyen (114 156 FCFA) ou au volume de tickets ? » Tables : ventes, vendeurs.csv (22). Manquant : vendeur n'existe que sur l'extrait du magasin 5 (4 vendeurs sur 22) ; la comparaison à l'ensemble des vendeurs suppose le fichier complet ; et une prime se discute avec la **marge**, non le CA — la marge nette n'est pas calculable (aucun coût de structure). Outil : tableur, motif : « échantillon, un seul chiffre à défendre ». Rendu attendu : sur l'extrait, Bationo = 11 370 350 FCFA (31,5 % du total), devant Sankara 9 423 281, Ouédraogo 8 483 019, Ilboudo 6 796 535 FCFA — et la phrase qui dit que ce n'est **pas** une preuve de performance sans le fichier complet ni la marge.

3. « En janvier, on ne vend rien / ça bosse. » Décision : caler les congés et les tournées du premier mois. Question analytique : « Sur l'année 2025, le CA de janvier (2 051 424 FCFA sur l'extrait) est-il dans le bas de la distribution des douze mois, et l'écart s'explique-t-il par le nombre de jours ouverts ou par le CA par jour ? » Tables : ventes, dim_date.csv (1 339 lignes, 12 colonnes, drapeaux dimanche/férié). Manquant : les jours d'ouverture réels ne sont pas dans le jeu (dim_date donne les dimanches et les fêtes, pas les fermetures exceptionnelles) ; et l'extrait ne contient que 40 lignes par mois. Outil : tableur, motif : « 12 lignes, un graphique suffit ». Le point de méthode attendu : distinguer « peu de lignes » de « peu de ventes » — sur un extrait mensuel fixe, la question devient un ratio, plus un total.

4. « Fermer le dépôt le dimanche ? » Décision : horaires d'exploitation, direction + exploitation, effet au 1^{er} janvier. Question analytique : « Un dimanche sur deux dans l'extrait des ventes : combien de tickets et quel CA sont réalisés un dimanche par le dépôt (id_magasin = 6) et par les magasins qu'il approvisionne ? » Tables : ventes, dim_date (colonne est_dimanche), magasins (6 lignes). Manquant : le jeu ne contient **pas** de flux physique dépôt → magasins (les transferts ne sont pas saisis), donc l'impact sur les ruptures est inconnu ; et stocks_quotidiens.csv couvre 60 000 lignes d'extrait, pas le champ complet. Outil : SQL, motif : « compte sur un gros fichier + jointure calendrier ». Rendu attendu : poser la question du **coût évité** (indisponible dans le jeu) autant que du chiffre d'affaires perdu — un cadrage qui ne voit que le CA est incomplet.

5. « Le magasin 5, combien il rapporte ? » Décision : arbitrer un investissement dans ce point de vente. Question analytique : « Que vaut le CA TTC 2025 du magasin 5 sur le fichier complet (à distinguer des 36 073 185 FCFA de l'extrait), et quelle **marge brute** — somme de (prix de vente – coût d'achat) par ligne — en FCFA et en pourcentage ? » Tables : ventes_propres.csv (240 000 lignes), couts_achat.csv, magasins.csv. Manquant : marge **nette** impossible (loyer, salaires, transport absents) ; objectifs_de_ca.csv compte 218 lignes, donc des mois sans cible. Outil : SQL ou Python, motif : « jointure sur 240 000 lignes et contrôle de to-

taux ». Le point le plus noté : le candidat qui **refuse** le mot « rapporte » sans le définir (CA ? marge brute ? marge nette ?) et demande la décision cachée derrière.

Total et décision d'orientation

Trois seuils, trois décisions. - **Quiz : 14/20 minimum.** En dessous, ne passez pas en M02 : les statistiques appliquées (M02) reposent mot pour mot sur la distinction type/format et sur le vocabulaire donnée-information. Reprenez C02 (frontières du vocabulaire) et C04 (types, formats), puis re-passez le quiz en entier — pas seulement les items ratés, parce que la mémoire des réponses fausses fausse la seconde tentative. - **Étude de cas : 12/20 minimum.** En dessous, le savoir est là mais le réflexe professionnel non : refaites les cinq demandes du projet M01.P qui portent sur le cadrage, et l'exercice guidé de C07, puis re-passez cette seule partie. - **Projet : 13/20 minimum** (barème dans 03_exercices/M01_projet.md). Un projet non rendu ne se compense par aucune note.

Entre 14 et 16 au quiz avec l'étude de cas réussie, passez en M02 en reprenant C04 avant son deuxième chapitre : les statistiques vous sembleraient obscures pour de mauvaises raisons. Cette règle de remédiation est la même pour les 21 modules suivants ; c'est l'une des rares choses du parcours qui ne se négocie pas.

Évaluation M02 — « Statistiques descriptives et inférence »

Module M02 · durée totale 2 h 45 · quatre épreuves · seuils : quiz 14/20, étude de cas 12/20, projet 13/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	remettre à plat ce qui a été compris en cours de module
B · Quiz	35 min	/20	oui	vingt questions à réponse courte, une seule bonne réponse
C · Exercices pratiques	55 min	/20 (auto-corrigé)	oui	deux séries menées jusqu'au chiffre, tableur ou Python
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier de quatre pages, chiffré et prudent

Le score du module se lit ainsi : **B ≥ 14** et **D ≥ 12** ouvrent le module 3 ; le projet M02.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D — l'écrit de connaissances ne remplace pas le travail sur les données.

A · Questions de récupération (non notées)

À traiter à l'oral ou par écrit, sans document. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de se tromper dans les épreuves notées.

1. Pourquoi la moyenne d'une distribution de paniers peut-elle être juste et inutilisable ?
2. Quelle différence de sens y a-t-il entre 150 000 FCFA « au 70^e percentile » et « le 70^e plus gros ticket » ?
3. Que devient un histogramme si la dernière classe est fermée à 150 000 alors que le maximum vaut 1 343 666 ?
4. Pourquoi dit-on d'un intervalle de confiance qu'il porte sur le paramètre et non sur les observations ?
5. Un p-valeur de 0,13 permet-il d'affirmer que deux vendeurs font le même chiffre ? Répondez en une phrase.
6. Pourquoi compare-t-on des moyennes de prix **par ligne** et non des chiffres d'affaires par vendeur ?
7. Qu'ajoute un test des rangs quand le test de Welch conclut déjà ?
8. Un échantillon de 385 personnes suffit pour une marge de 5 points. Un échantillon de 480 lignes donne-t-il une marge de 5 points sur un montant ? Expliquez.

B · Quiz (20 questions · 20 points)

Barème : 1 point par question, aucune soustraction de point pour une erreur de signe ou d'unité. Les réponses justifiées en une ligne peuvent rapporter un demi-point de bonus, plafonné à 20.

Bloc 1 — Population, échantillon, biais (Q1 à Q5)

Q1. Un fichier contient 480 lignes issues des 40 premières lignes de chaque mois. On veut estimer le panier moyen annuel. Ce que mesure avant tout le taux de 6,3 % : a) la variance · b) la représentativité · c) le taux de remplissage · d) la précision

Q2. Le même échantillon ne contient que les jours 1 à 4 de chaque mois. On peut dire sans mesurer que la moyenne : a) est biaisée de 2 % · b) est biaisée vers le bas · c) est non représentative · d) est biaisée de 14 %

Q3. Trois modalités possibles pour le champ magasin alors qu'il y a cinq magasins. Le risque principal : a) une clé orpheline · b) une valeur manquante · c) un type de donnée · d) une normalisation

Q4. Un champ de montants est vide sur 18 lignes, absent sur 103. La première décision statistique : a) compléter par zéro · b) compléter par la moyenne · c) dire sur quelle population porte chaque chiffre · d) supprimer les 103 lignes

Q5. L'unité d'échantillonnage d'une étude de clientèle doit être : a) la ligne de vente · b) le ticket · c) le jour · d) le client

Bloc 2 — Centres (Q6 à Q8)

Q6. Sur un échantillon de 316 paniers, le plus gros vaut 1 343 666 FCFA. Passer de 316 à 315 paniers en retirant ce maximum déplace surtout : a) la médiane · b) le mode · c) la moyenne · d) l'écart interquartiles

Q7. Deux valeurs également fréquentes, deux modes donc, séparées de 1 463 FCFA. Le mode est ici : a) inutilisable · b) une valeur de prix unitaire, pas de panier · c) un signal de palier tarifaire · d) une erreur de saisie

Q8. Moyenne 114 156, médiane 58 423 FCFA. Le chiffre qui dit « un client sur deux est en dessous » : a) la moyenne · b) la médiane · c) le mode · d) le 75^e percentile

Bloc 3 — Dispersion et positions (Q9 à Q12)

Q9. Écart type 182 095 pour une moyenne de 114 156 FCFA. Le coefficient de variation vaut environ 159,5 % et signale : a) des données fausses · b) une dispersion supérieure à la moyenne, moyenne peu représentative · c) une erreur d'unité · d) une distribution normale

Q10. Premier quartile 20 635 FCFA, troisième 130 559 FCFA. La borne supérieure de Tukey vaut : a) $130\,559 + 1,5 \times 109\,924$ · b) $1,5 \times 130\,559$ · c) $Q3 + 1,5 \times Q1$ · d) $Q3 + 3 \times IQR$

Q11. Sur 316 tickets, 158 sont entre les quartiles — et non 158 comme l'annoncent 50 % des observations. L'écart de 2 s'explique par : a) une erreur de calcul · b) la convention de quantile, qui peut placer une valeur exactement sur un quartile · c) 2 aberrants · d) un arrondi de l'écart type

Q12. Le 90^e percentile du montant par ligne vaut 234 518, 235 318, 236 118 ou 243 010 FCFA selon la méthode. La bonne conduite : a) publier la plus grande · b) publier celle du tableur · c) publier l'une et nommer la méthode · d) publier la moyenne des quatre

Bloc 4 — Formes et liaisons (Q13 à Q16)

Q13. Asymétrie 3,84, excès de kurtosis 18,25. Le couple de qualificatifs exact : a) symétrique pointue · b) à droite et pointue · c) à gauche et plate · d) normale et étroite

Q14. Le chiffre d'affaires est concentré : le dernier décile porte 47,5 % du total, et 1,6 % seulement est sous le premier quartile. La première page du rapport devrait montrer : a) un camembert par catégorie · b) une courbe cumulative · c) un nuage de points · d) un effectif par vendeur

Q15. La covariance entre `quantite` et `montant_ttc` vaut 857 634, en unités « pièces × FCFA ». Comment passe-t-on de là au coefficient de Pearson ? a) on divise la covariance par le produit des deux écarts types · b) on multiplie la covariance par les deux écarts types · c) on élève 0,712 au carré · d) on divise la covariance par l'écart interquartiles

Q16. Pente 12 237 FCFA par pièce, ordonnée -10 402 FCFA, coefficient 0,712. L'affirmation recevable : a) doubler la quantité double le montant · b) une pièce supplémentaire accompagne en moyenne 12 237 FCFA de plus sur la ligne · c) 50,7 % du montant vient de la quantité, le reste est faux · d) la relation est causale

Bloc 5 — Estimation et tests (Q17 à Q20)

Q17. Un intervalle [94 078 ; 134 234] FCFA sur le panier moyen. Le nombre 40 156 est : a) l'erreur type · b) la marge d'erreur · c) la largeur de l'intervalle · d) la variance

Q18. Test de Welch entre deux vendeurs : $t = 1,52$, ddl 226,1, $p = 0,13$, IC [-8 279 ; 65 498]. La conclusion écrite dans le rapport : a) les vendeurs sont équivalents · b) l'écart est de 28 609 FCFA · c) aucun écart stable n'est établi sur cet extrait · d) le vendeur le plus performant est Bationo

Q19. Six comparaisons de paires sont menées, la plus petite p-valeur vaut 0,13, le seuil après correction 0,0083. On conclut : a) une comparaison est significative · b) aucune n'est significative au seuil corrigé, et l'étude manque de puissance · c) il faut publier la plus petite p · d) il faut augmenter alpha

Q20. Marge de détection de 52 956 FCFA par ligne à 80 % de puissance. Le même test au niveau ticket donne une erreur type de 33 881 contre 18 821 : la raison est : a) une erreur de formule · b) le passage de 234 à 107 observations, et le mélange de plusieurs vendeurs sur 84 tickets · c) des queues trop lourdes · d) un seuil mal choisi

C · Exercices pratiques (2 séries · 55 min · auto-correction exigée)

Règles : une feuille de calcul ou un script par série ; tous les nombres doivent être retrouvables par une formule ou une ligne de commande ; les arrondis sont affichés en fin de chaîne. Les corrigés de la partie C sont donnés à la fin du document — l'auto-correction fait partie de l'épreuve, un candidat qui ne se corrige pas perd le point de méthode.

E1 · Profil du magasin (30 min) 1. E1.1 — effectif de lignes, effectif de tickets, moyenne et médiane du montant TTC par ligne, écart type. 2. E1.2 — trois premières et trois dernières lignes triées par `montant_ttc`, par ticket : le chiffre d'affaires est-il porté par les gros tickets ? Donnez le pourcentage du total détenu par les 32 plus gros tickets. 3. E1.3 — Q1, Q3, IQR, borne supérieure de Tukey, nombre d'observations hors borne, et ce que devient cette borne si l'on exclut les retours négatifs.

E2 · Estimer et décider (25 min) 1. E2.1 — intervalle à 95 % de la part de lignes avec remise dans l'échantillon (173 lignes sur 480), puis comparaison à la part observée dans la population de référence si vous y avez accès. 2. E2.2 — comparer le montant par ligne des deux vendeurs les plus fournis, au tableur **puis** en Python : moyenne, écart type, effectif, t, ddl, p, intervalle à 95 % de l'écart. 3. E2.3 — même comparaison par les rangs, puis par permutation (10 000 tirages, graine annoncée). Vous rédigez les trois p-valeurs et une phrase de décision unique. 4. E2.4 — avec 30 % de remise sur les 316 tickets, combien de tickets faudrait-il dans chaque groupe pour mettre en évidence un écart de 28 609 FCFA ?

D · Étude de cas (20 points · seuil 12 · 60 min)

Le dossier

Vous recevez un courriel du directeur commercial :

« Bationo fait des paniers plus gros qu'Ilboudo : 90 963 contre 62 354 FCFA par ligne. C'est 46 % d'écart. Je veux deux choses : la preuve chiffrée, et si tu ne la trouves pas, dis-moi franchement que ces chiffres ne veulent rien dire. Et pendant que tu y es, est-ce que le magasin 5 vend plus cher que le magasin 1 ? »

Vous disposez de : l'extrait nettoyé de 480 lignes (316 tickets), le fichier de référence de 240 000 lignes, la table `dim_date`, un tableur, Python si vous le souhaitez, et une demi-heure de calcul.

Les quatre livrables demandés

1. **La preuve, ou son absence, sur les vendeurs** (6 pts) — au niveau ligne et au niveau ticket, avec les deux échantillons, l'écart estimé, l'intervalle, le test, le seuil corrigé si vous comparez plusieurs paires, et la marge de détection.
2. **La réponse sur les deux magasins** (6 pts) — périmètre et grain annoncés, les deux moyennes, les deux effectifs, l'écart, l'intervalle, le p, la décision.
3. **Une recommandation datée** (4 pts) — trois lignes maximum, qui ne dépasse pas ce que les pages précédentes établissent ; par exemple un plan de mesure (recalcul sur douze mois, effet de la couverture en jours, ce qu'il faudrait collecter).
4. **Un encadré « ce qui aurait changé ma réponse »** (4 pts) — trois contre-vérifications que vous auriez aimées et qui sont impossibles avec ces données (le client, le stock, la marge nette).

Ce que le correcteur attend, ligne à ligne

Livable	Attendu	Point perdu si
1	écart 28 609 FCFA, $t = 1,52$, $p = 0,13$, IC [-8 279 ; 65 498], seuil corrigé 0,0083, MDE 52 956 ; version ticket écart 16 663 FCFA, $p = 0,624$	le mot « équivalents » apparaît ; le grain n'est pas annoncé
2	magasins 1 et 5 sur 2025 au grain du ticket : 107 889 (14 463) et 116 886 FCFA (4 524), écart -8 997, $t = -3,37$, $p = 0,000751$, IC [-14 226 ; -3 767] → écart établi	le candidat compare des lignes sans le dire, ou omet les effectifs
3	décision de mesurer avant de gérer : pas de mutation, pas de prime sur cette base ; recalcul sur douze mois	la recommandation dépasse les chiffres
4	trois limites nommées, avec ce qu'il faudrait comme donnée pour chacune	limites rédigées en « il faudrait plus de données »

Total : 20 points. Seuil de passage : 12.

Corrigé

B · Quiz — réponses et explications

#	Réponse	Explication en une ligne
Q1	b	le taux de 6,3 % est un taux de couverture : il mesure l'exposition à un biais, pas une variance

#	Réponse	Explication en une ligne
Q2	c	le sens et l'amplitude du biais ne se devinent pas, ils se mesurent ; seul le caractère non représentatif est certain
Q3	b	trois modalités sur cinq magasins : deux magasins sont absents du fichier, ce qui change le périmètre de la réponse
Q4	c	avant de combler ou de supprimer, on écrit sur quoi porte chaque chiffre
Q5	b	un client, un ticket : si l'unité est la ligne, les gros tickets comptent double
Q6	c	la moyenne déplace tout le poids de l'observation extrême, la médiane presque rien
Q7	c	deux modes serrés sur un champ de prix signalent un palier tarifaire, pas une erreur
Q8	b	définition de la médiane ; 71,8 % des tickets sont sous la moyenne, ce que la moyenne ne dit pas
Q9	b	un CV de 159,5 % est un signal de dispersion, pas une faute de calcul
Q10	a	$Q3 + 1,5 \times IQR : 130\,559 + 1,5 \times 109\,924$, soit 295 445 FCFA
Q11	b	166 observations exactement à Q1, la frontière dépend de la convention retenue
Q12	c	les méthodes diffèrent de 3,6 % sur cet extrait : on publie une valeur et on nomme la règle
Q13	b	asymétrie positive et excursion marquée, jamais « normale »
Q14	b	la concentration est une affaire de cumul ; le camembert ne la montre pas
Q15	a	covariance sur produit des deux écarts types — c'est ce qui rend le nombre comparable d'un fichier à l'autre ; le carré du coefficient, $0,712^2$, est le R^2
Q16	b	lecture de la pente, dans le domaine observé, sans causalité
Q17	c	40 156 est la largeur ; la marge est la moitié, 20 078 FCFA
Q18	c	un intervalle qui contient zéro n'établit rien ; il n'innocente pas non plus
Q19	b	$0,13 > 0,0083$: aucune comparaison ne passe, et la puissance était faible avant même de commencer
Q20	b	moins d'observations et un effet partagé : deux raisons qui se cumulent

Grille de conversion : 20/20 à 14/20 validé · 13/20 à 11/20 révisions ciblées sur les deux blocs manquants · 10/20 et moins reprendre le module.

C · Exercices pratiques — corrigés

E1.1 — 480 lignes, 316 tickets. Montant TTC par ligne : moyenne 75 152, médiane 37 966, écart type 143 915 FCFA. `=ECARTYPE.STANDARD(plage)` ou `df.montant_ttc.std()`.

E1.2 — le total de l'extrait au grain du ticket vaut 36 073 185 FCFA. Les 32 plus gros tickets, soit 10,1 % des 316, en portent 17 139 852 FCFA : **47,5 %** du chiffre d'affaires pour un ticket sur dix. Les 31 tickets au-dessus de 250 000 FCFA valent 16 903 734 FCFA, soit 46,9 %. Les neuf plus gros seulement suffisent à faire 24,0 % du total (8 642 533 FCFA). Le chiffre d'affaires est donc bien porté par les gros tickets, et la moyenne du magasin parle surtout d'eux.

E1.3 — premier quartile 20 635, troisième 130 559, IQR 109 924, borne 295 445 FCFA, 25 tickets hors borne. En excluant les 3 tickets de retour (montant négatif au niveau du ticket), la borne descend de 1 271 FCFA à 294 174 FCFA et le nombre de tickets hors borne **reste 25** : le nettoyage déplace la borne, il ne change pas le verdict. C'est la réponse attendue — et elle vaut mieux que « la borne est une ligne de conduite », parce qu'elle est mesurée des deux côtés.

E2.1 — 173 sur 480, soit 36,0 % ; erreur type 2,2 points, marge 4,3 points, intervalle [31,7 ; 40,3] % — le même calcul que M02.C07, exerc. 7.3. Dans la population de référence, la part de lignes avec remise se mesure par la même formule sur 240 000 lignes : si les deux chiffres s'écartent de plus d'une dizaine de points, l'extrait n'est plus neutre sur cette variable et la remarque va dans le rapport.

E2.2 — Bationo : 90 963 FCFA de moyenne par ligne, 125 lignes. Ilboudo : 62 354 FCFA, 109 lignes. Écart 28 609 FCFA, erreur type 18 821, marge $\pm 36\,888$, $t = 1,52$, ddl 226,1, $p = 0,13$, IC [-8 279 ; 65 498]. Au tableur `=T.TEST(plage1;plage2;2;3)`, en Python `stats.ttest_ind(a, b, equal_var=False)` : les deux doivent rendre le même p au millième près.

E2.3 — permutation 10 000 tirages, graine annoncée : $p = 0,1443$; test des rangs : $p = 0,0161$. Phrase de décision attendue : « avec 234 lignes, l'extrait ne permet d'établir aucun classement entre vendeurs ; le test des rangs signale une différence de position qui résiste mal à la correction pour six comparaisons (seuil 0,0083) ».

E2.4 — $n = 412$ lignes par groupe pour 80 % de puissance sur un écart de 28 609 FCFA avec 30 % de remise. L'extrait en compte 125 et 109 : l'étude est en dessous d'un facteur quatre.

D · Étude de cas — éléments de réponse attendus

1. Deux niveaux, deux chiffres, même conclusion. Niveau ligne : 234 observations, écart estimé 28 609 FCFA, intervalle [-8 279 ; 65 498] FCFA, $p = 0,13$, seuil corrigé 0,0083 → non concluant. Niveau ticket : 107 observations, écart 16 663 FCFA, erreur type 33 881, $p = 0,624$, intervalle [-49 743 ; 83 069] → non concluant, et il faut écrire pourquoi l'intervalle gonfle : 30 % d'observations en moins, et 84 tickets sur 316 portent plusieurs vendeurs, donc l'effet cherché est partagé. Marge de détection 52 956 FCFA : même une vraie différence de 20 000 FCFA n'aurait que 18,5 % de chances d'être détectée. Verdict à rendre : **classement non établi**, les 46 % du courriel sont un rapport de moyennes, pas un effet démontré.

2. Périmètre : année 2025, magasin 1 contre magasin 5, grain du ticket, total des montants TTC par ticket dans `ventes_propres`. Moyennes 107 889 FCFA (14 463 tickets) et 116 886 FCFA (4 524 tickets) ; médianes 65 313 et 68 160 FCFA. Écart -8 997 FCFA, erreur type 2 668, $t = -3,37$ sur 6 782,8 ddl, $p = 0,000751$, intervalle [-14 226 ; -3 767] FCFA : zéro exclu, **écart établi** — le magasin 5 fait des paniers plus gros, d'environ 8 300 à 14 200 FCFA, soit 7,7 % à 13,2 % du panier moyen du magasin 1. La prudence à joindre : ce test porte sur la population entière des deux magasins, l'extrait nettoyé n'y participe pas ; et rien ici n'explique l'écart (assortiment ou clientèle, on ne sait pas).

3. Recommandation, modèle de rédaction : « aucune mesure individuelle ne se justifie sur cette base ; nous proposons un recalcul sur douze mois complets au grain du ticket (14 463 et 4 524 tickets disponibles, seuil d'observation 0,0083), et la collecte du rattachement client-ligne qui manque sur 21,5 % de l'effectif avant toute comparaison de filets de clientèle ». Une ligne de date, une ligne de responsable, une ligne de revue.

4. Ce qui aurait changé la réponse : le panier par client et non par ticket (84 tickets multi-vendeurs, 103 lignes sans client utile), la marge nette au lieu du chiffre d'affaires (le prix de revient est hors de ce fichier), et l'écoulement du stock (sans lui, un gros mois peut être un transfert en magasin, pas une vente). Pour chacun : la donnée à demander, et le test qui en découlerait.

Total et décision d'orientation

Composante	Score	Seuil	Statut
B · Quiz	... /20	14	bloquant
C · Exercices	... /20	indicatif (auto-correction menée = +0,5)	—
D · Étude de cas	... /20	12	bloquant
Projet M02.P	... /20	13	bloquant

Décision. B et D au seuil, projet au seuil → module validé, passage en M03 (SQL : interroger la source). Quiz seulement en dessous → trois jours pour reprendre C02 et C05, puis une nouvelle version de B. Étude de cas seulement en dessous → nouvelle version de D sur le même dossier,

avec un entretien de 15 minutes sur la lecture des intervalles. Deux échecs → reprise du module, en commençant par les mini-projets P1 à P4.

Évaluation M03 — « Excel, du clic à la méthode »

Module M03 · durée totale 2 h 30 · quatre épreuves · seuils : quiz 11/15, étude de cas 12/20, projet 13/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	vérifier qu'on a lu les huit chapitres avant de se tromper cher
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, dont trois sur l'identification d'une erreur affichée
C · Exercices pratiques	50 min	/20 (auto-corrigé)	oui	quatre exercices menés jusqu'au chiffre sur le classeur de l'atelier
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier en quatre pages : un tableau de bord à déclarer pilotable ou non

Le score du module se lit ainsi : **B ≥ 11** et **D ≥ 12** ouvrent le module 4 ; le projet M03.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D : savoir ce que fait **FIN.MOIS** ne dispense pas de dire ce que vaut le ratio affiché dans un comité.

A · Questions de récupération (non notées)

À traiter sans document, en une phrase chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail de syntaxe.

- Pourquoi `=SOMME()` sur une colonne de montants stockés en texte rend-il 0 au lieu d'une erreur, et pourquoi est-ce plus grave ?
- Que devient `=DATE(25;1;1)`, et quel est le réglage qui rend ce piège impossible ?
- Une date « 15/03/2025 » dans une cellule : citez deux tests qui disent s'il s'agit d'un nombre ou d'un texte.
- Pourquoi un `RECHERCHEV` est-il plus dangereux qu'un `INDEX/EQUIV` quand on insère une colonne, et que répond `RECHERCHEX` à ce problème ?
- Un TCD affiche un total inférieur à la somme de la colonne source : listez trois causes, pas une.
- Que signifie `#PROPAGATION!`, et pourquoi n'est-ce pas une erreur de formule ?
- Pourquoi un segment ne change-t-il rien à une cellule `=SOMME(...)` écrite à côté du TCD ?
- « 75 152 FCFA par ligne » et « 114 156 FCFA par ticket » : lequel répondre à « combien vend-on par ticket » ?
- Qu'est-ce qu'une macro enregistrée ne sait pas faire, et que sait faire une requête Power Query à sa place ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

Bloc 1 — Types, formats, saisie (Q1 à Q4)

Q1. Un montant est saisi 1 234,56 (espace de milliers) dans une cellule au format Standard. Ce que la cellule contient : a) le nombre 1234,56 · b) le texte 1 234,56 · c) le nombre 1,23456 · d) une erreur immédiate

Q2. Quatre montants sur 489 sont lus comme des nombres et le reste comme du texte dans le fichier reçu. Pour récupérer la valeur d'une cellule texte 1229678, l'écriture correcte est : a) =CNUM(A1)*1 · b) =A1&"" · c) =CNUM(SUBSTITUE(A1:" ":"")) · d) =ARRONDI(A1;0)

Q3. Le format « Monétaire » avec le code FCFA diffère du format « Comptabilité » en ce que : a) rien, ce sont deux noms du même format · b) la colonne des symboles reste alignée en comptabilité, pas en monétaire · c) le comptable interdit les négatifs · d) le monétaire arrondit à l'unité

Q4. Une validation des données en liste refuse une valeur saisie. Le message par défaut : a) bloque et propose la liste · b) bloque sans explication · c) avertit mais accepte · d) convertit la valeur

Bloc 2 — Adresses et erreurs affichées (Q5 à Q8)

Q5. Une nouvelle colonne est ajoutée **dans** TableVentes avec la formule =SOMME(TableVentes[montant_ttc]). Ce qui s'affiche : a) 36 073 185 dans chaque cellule, sans erreur · b) une référence circulaire refusée · c) #VALUE! en tête de colonne · d) 0 sur les lignes dont la catégorie est vide

Q6. Une cellule affiche #N/A. Parmi ces quatre situations, laquelle ne peut **pas** en être la cause ?
a) la clé cherchée n'existe pas dans la table · b) la clé porte un espace invisible en trop · c) le numéro de colonne rendu par RECHERCHEV dépasse la largeur de la plage · d) la valeur renvoyée contient elle-même #N/A

Q7. Une formule matricielle dynamique renvoie #PROPAGATION!. La correction est : a) valider en Ctrl + Maj + Entrée · b) libérer les cellules de la zone d'arrivée, hors du tableau structuré · c) convertir la date en nombre · d) ajouter un troisième argument à RECHERCHEX

Q8. =FILTRE(...) rendu sur un poste en Excel 2019 affiche : a) #VALEUR! · b) #NOM? · c) #PROPAGATION! · d) la première ligne du tableau seulement

Bloc 3 — Agrégats, critères, dates (Q9 à Q12)

Q9. =NB(TableVentes[date]) vaut 480, =MOIS(TableVentes[date]) sur une cellule de la même colonne rend #VALEUR!. L'explication cohérente : a) la colonne est un mélange de dates et de texte, =NB() ignore le texte · b) MOIS n'existe pas en français · c) la cellule appelée contient un nombre trop grand · d) le tableau est trié

Q10. Pour isoler le mois de février 2024 dans un SOMME.SI.ENS sans écrire le 29 ou le 28, on pose : a) ">="&DATE(2024;2;1) et "<"&DATE(2024;3;1) · b) ">="&DATE(2024;2;1) et "<=29/02/2024" · c) MOIS(plage)=2 · d) ">="&DATE(2024;2;1) et "<="&FIN.MOIS(DATE(2024;2;1);1)

Q11. =FIN.MOIS(DATE(2025;2;1);0) rend 45716 affiché tel quel. Ce qu'il faut faire : a) rien, c'est le 28 février en série de dates · b) formater la cellule en date · c) ajouter 1 · d) passer par DATEVAL

Q12. Sur la période du 01/01/2025 au 02/12/2025, `=NB.JOURS.OUVRES(début;fin)` rend 240. Le chiffre devient faux dès que : a) l'entreprise travaille le samedi · b) des jours fériés sont chômés et non passés en troisième argument · c) la période traverse deux années · d) l'intervalle inclut le 29 février

Bloc 4 — Recherche, assemblage, TCD (Q13 à Q15)

Q13. 103 lignes de l'extrait portent un `client` absent du référentiel (85 au comptoir 0, 18 vides). Un `RECHERCHEX` sur ces lignes avec le quatrième argument omis renvoie : a) 0 · b) une er-

reur #N/A, propagée aux calculs qui lisent la cellule · c) la ligne précédente · d) le texte introuvable entre guillemets

Q14. Un TCD construit sur une **plage** A1:M481 puis 20 lignes ajoutées au tableau source : à l'actualisation, le TCD rend : a) 500 lignes · b) 480 lignes : la plage ne s'est pas étendue · c) une erreur de source · d) 480 lignes et un avertissement rouge

Q15. Dans un TCD, « Nombre » sur le champ `client` et une mesure `=DISTINCT-COUNT(ventes[client])` dans le modèle : a) les deux rendent 480 · b) 480 d'un côté, 372 de l'autre · c) 372 des deux côtés · d) le second exige Excel 2019

C · Exercices pratiques (4 exercices · 20 points)

Matériel autorisé : le classeur `m03_classeur_atelier.xlsx` et les CSV du socle. Barème : 12 points sur E1 à E3, 8 points sur E4 ; la restitution des formules fait partie du point, pas une annexe.

#	Exercice	Pts
E1	Formats, types, validation : rendre la colonne montant et la colonne date utilisables	4
E2	Agrégations à critères et TCD : retrouver six chiffres, en expliquer un	4
E3	Recherches et jointures : le référentiel, les 103 orphelins, la clé composée	4
E4	Le classeur aux trois chiffres faux : diagnostiquer et corriger	8
Total		20

E1 (4 pts). Sur `ventes_brutes` : convertissez `montant_ttc` en nombre et `date` en série de dates, en écrivant une formule par ligne (voie accessible) puis la requête Power Query équivalente (voie moderne). Contrôles exigés : compte numérique 480, `MAX(date) - MIN(date) = 335` jours, somme 36 073 185 FCFA. Ajoutez une validation des données qui refuse une quantité supérieure à 500 et expliquez pourquoi la borne est 500 et non 68.

E2 (4 pts). Sans TCD, puis avec un TCD, retrouvez : le CA de la catégorie Plomberie, le nombre de lignes remiseses en Peinture, la moyenne par ligne d'Electricité, la part du CA de Bois & panneaux, le CA de mai, le CA de décembre. Six valeurs, une formule chacune. Expliquez en trois lignes pourquoi la « moyenne des moyennes par catégorie » n'est pas la moyenne par ligne, chiffres à l'appui.

E3 (4 pts). Rapportez à chaque ligne de l'extrait la catégorie du produit, le segment du client et la ville du magasin, d'abord avec `RECHERCHEV`, puis avec `RECHERCHEX`, puis avec une relation du modèle de données. Contrôles : les lignes sans client utile doivent être comptées et nommées ; la jointure aux objectifs se fait sur une clé composée que vous fabriquez ; un test doit montrer ce qui casse si une colonne est insérée dans le référentiel.

E4 (8 pts) — le classeur aux trois chiffres faux. Un analyste a rendu le classeur ci-dessous ; il est interne, propre, et contient trois chiffres faux.

Ce que rend le classeur	Contexte affiché	Ce que le fichier contient
Chiffre d'affaires 2025 : 36 339 317 FCFA	<code>=SOMME(ventes!M2:M490)</code> sur la feuille reçue	489 lignes, dont neuf doublons de ticket
Panier moyen : 98 281 FCFA	<code>=MOYENNE()</code> sur une colonne de moyennes par catégorie	la moyenne par ligne vaut 75 152, le panier par ticket 114 156
Clients servis : 480	<code>=NBVAL()</code> sur la colonne client du tableau de lignes	316 tickets, 372 clients distincts
Réalisation décembre : 27,9 %	ratio du CA de décembre sur l'objectif mensuel	pas faux, mais lu à tort comme une performance

Pour chacun des trois premiers : expliquez la cause, écrivez la formule de contrôle qui l'aurait révélée, et donnez la valeur attendue. Pour le quatrième : dites en deux lignes pourquoi ce n'est pas une erreur de calcul et ce qu'il faut écrire sous le graphique. Bonus (non compté) : la mise en forme conditionnelle qui aurait alerté sur le premier.

D · Étude de cas — « Le tableau de bord du magasin 5 est-il pilotable ? » (20 points, seuil 12)

Le dossier. Le responsable régional transmet un classeur qu'il utilise chaque lundi, et un courriel de trois lignes : « Le magasin 5 est à 18,2 % de son objectif annuel en 2025. Décembre est à 27,9 %, octobre à 14,1 %. Je félicite en décembre, je recadre en octobre. Dis-moi si le fichier tient la route avant que j'envoie ça. »

Le classeur reçu contient : une feuille `Données` (480 lignes, collées depuis un CSV, montants et dates en texte, triest effectués à la main), une feuille `Calculs` (neuf colonnes utilitaires dont trois `RECHERCHEV` sur des plages A:F figées, une moyenne glissante `=MOYENNE(B2:B13)` recopiée depuis la ligne 2), une feuille `Pilotage` (deux TCD pointant sur la plage `Données!A1:M481`, un segment connecté à un seul TCD, un graphique en camembert de sept parts, un ratio en colonne `=C2/D2` sans test de dénominateur), et une macro `Reparer` enregistrée qui supprime des lignes et met en forme douze cellules. Le fichier est `.xlsx`, non protégé, et un onglet caché porte une copie des montants en dur.

Questions, à traiter dans cet ordre, en quatre pages maximum.

1. Le chiffre de 18,2 % est-il exact, et dit-il ce que le responsable croit ? Reprenez les deux nombres (CA de l'extrait, objectif du magasin 5 sur 2025), refaites la division, et énoncez ce que mesure ce ratio compte tenu de la couverture réelle des lignes. Donnez les deux ratios mensuels cités, et la phrase qui doit accompagner le graphique.
2. Que devient ce pilotage sur la population entière des ventes (pas l'extrait) ? Chiffrez le CA annuel du magasin 5, son ratio à l'objectif, et l'étendue des ratios magasin-mois observés dans le fichier d'objectifs. Concluez sur la comparabilité des magasins : ce que la table d'objectifs permet et ne permet pas d'affirmer.
3. Listez, dans l'ordre de gravité, les sept défauts du classeur qui empêchent de le déclarer pilotable, et pour chacun la correction au format d'une ligne de procédure (quoi faire, dans quel outil, quel contrôle à l'appui). Trois défauts au moins doivent concerner ce qui casse à l'actualisation, pas la présentation.
4. Rédigez la réponse au courriel, en six lignes, avec les deux chiffres à ne pas envoyer tels quels et la proposition de dispositif : ce qui reste dans le tableur, ce qui passe par une requête, ce qui est à sortir d'Excel. Vous indiquez en une phrase ce que vous écrivez au sujet de la macro `Reparer`.

Éléments de réponse attendus

1. Le ratio est arithmétiquement juste : 36 073 185 FCFA sur 198 030 000 FCFA d'objectif, soit 18,2 %. Il ne dit rien d'une performance annuelle : l'extrait ne couvre que les jours 1 à 4 de chaque mois, à raison de 40 lignes par mois, et 103 lignes sur 480 portent un client qu'on ne peut pas relier au référentiel. Les ratios mensuels cités (14,1 % en octobre, 27,9 % en décembre) comparent la même tranche de quatre jours à un mois de cible : la mention à écrire sous le graphique est celle de la feuille `Mensuel` du classeur d'atelier — le ratio mesure un extrait, pas l'année. Une réponse qui corrige le pourcentage sans corriger l'échelle est comptée à moitié.

2. Sur la population : 528 792 330 FCFA de chiffre d'affaires pour le magasin 5 en 2025, soit 267,0 % de l'objectif — un ratio supérieur à 100 % qui, lui non plus, ne veut rien dire tant que la table d'objectifs n'a pas été lue (218 lignes, un objectif par magasin et par mois, des trous : deux mois de l'extrait sont sans cible). Les ratios magasin-mois s'étagent de 1,62 à 3,27 (médiane 2,31), soit un facteur deux entre le meilleur et le pire magasin sur cette base : la comparaison brute des pourcentages de réalisation est donc **un classement des objectifs, pas des résultats**, tant que le mode de construction de la cible n'est pas connu. Ce qu'il faut demander : la méthode d'élaboration des objectifs et le taux d'ouverture mensuel par magasin.

3. **Les sept défauts, dans l'ordre où ils font perdre confiance.** (a) TCD pointant sur une plage figée : à l'actualisation, les lignes 482 à 500 restent dehors, silencieusement ; correction = sourcer sur le tableau structuré, contrôle = total du TCD contre somme du tableau. (b) Les montants en texte : =SOMME() sur la feuille reçue rend 36 339 317 FCFA au lieu de 36 073 185, et sur une plage entièrement texte elle rend 0 sans erreur ; correction = conversion dans la requête, contrôle = =NB(colonne) attendu 480. (c) Dates en texte avec deux écritures : le regroupement par mois produit 36 groupes au lieu de 12, le tri alphabétique masque l'erreur ; correction = colonne date typée et mois calculé. (d) RECHERCHEV sur plages figées : une colonne insérée dans le référentiel et les numéros d'index décalent les libellés, sans erreur affichée ; correction = RECHERCHEX ou relation du modèle. (e) Moyenne glissante =MOYENNE(B2:B13) recopiée depuis la ligne 2 : les onze premiers points ne sont pas des moyennes à douze mois ; correction = garde-fou =SI(NB(...)=12;...) et deux premières cellules laissées vides. (f) Ratio =C2/D2 sans test de dénominateur : deux lignes de l'extrait n'ont pas de cible, le classeur affiche #DIV/0! et le camembert ampute les parts ; correction = SIERREUR avec un libellé, plus une ligne « cible absente » dans la note. (g) Macro Reparer qui supprime des lignes : non annulable, adressée en dur, silencieuse si la forme change, et .xlsm refusé par beaucoup de postes ; correction = remplacer par une étape de requête, et ne garder la macro que pour l'export. Le segment connecté à un seul TCD, le graphique en camembert de sept parts voisines et la feuille cachée en dur sont trois défauts de plus, comptés comme secondaires.

4. Modèle de réponse : « le fichier est exact dans ses calculs, faux dans ce qu'il autorise à conclure. 18,2 % n'est pas une réalisation annuelle mais le poids de quatre jours par mois rapportés à une cible mensuelle ; décembre à 27,9 % et octobre à 14,1 % ne se comparent pas non plus. Je propose : nettoyage par requête (six étapes, jouées), TCD sur le modèle de données, un segment connecté aux deux tableaux, un avis de prudence sur le ratio tant que la fabrication des objectifs n'est pas connue. La macro Reparer doit être retirée du circuit : elle détruit des lignes sans trace et ne sera pas utilisable sur les postes du service. » Toute recommandation de sanction ou de félicitation maintenue sur ces bases fait perdre le point de conclusion.

Corrigé du quiz (B)

Q	Réponse	Une ligne de justification
1	b	l'espace de milliers non reconnu à la saisie laisse du texte ; le format ne convertit rien
2	c	SUBSTITUE retire l'espace, CNUM convertit ; b) fabrique du texte, d) convertit 0
3	b	la comptabilité aligne symboles et décimales sur une colonne dédiée
4	a	le message par défaut bloque et propose la liste autorisée
5	a	la formule se recopie sur les 480 lignes : chaque cellule est juste, la colonne sommée vaut 17 315 128 800 FCFA, un chiffre sans sens et sans erreur
6	c	un index hors plage se signale #REF! ; les trois autres produisent bien un #N/A
7	b	#PROPAGATION! = la zone d'arrivée est occupée, y compris par un tableau structuré
8	b	les dynamiques sont en 365, 2021 et 2024 : en 2019, #NOM?

Q	Réponse	Une ligne de justification
9	a	<code>=NB()</code> ignore le texte et compte les nombres : un mélange de types, pas une absence de dates
10	a	borne basse inclusive, haute exclusive, aucune mention du 29
11	b	<code>FIN.MOIS</code> rend un nombre ; c'est le format date qui l'affiche en jour
12	b	sans troisième argument, les fériés comptent comme travaillés
13	b	<code>#N/A</code> propagé : c'est le quatrième argument (valeur si non trouvée) qui manque
14	b	la plage ne s'étend pas ; 480 lignes, sans un mot
15	b	« Nombre » compte des lignes ; le distinct exige le modèle de données

Seuils de lecture du quiz. 13 et plus : les mécanismes sont acquis. 11-12 : le candidat confond encore types et formats, reprendre C02 et C07. 10 et moins : reprendre C02, C03, C04 en mini-projets, puis une nouvelle version de B. Les questions 6, 10 et 15 sont éliminatoires dans ma pratique : un analyste qui ne sait pas ce que fait une borne de période écrira un faux chiffre, et un faux chiffre signé, c'est tout le module.

Éléments de corrigé des exercices (C)

E1. Quatre formules de contrôle, dans cet ordre : `=NB(ventes[montant_ttc])` → 480 ; `=MAX(date)-MIN(date)` → 335 jours (et 0 si la colonne est restée texte) ; `=SOMME(ventes[montant_ttc])` → 36 073 185 FCFA ; `=MAX(quantite)` → 68 après correction d'unité contre 14 000 dans le fichier reçu. La borne 500 se justifie par l'échelle du métier : 7 lignes dépassaient 500 pièces, toutes issues d'un oubli de conversion d'unité, et non parce que 68 serait un plafond physique — la formulation juste est « seuil d'alerte, pas seuil de vérité ».

E2. Plomberie 8 754 982 FCFA ; Peinture 8 lignes remises ; Electricité 119 088 FCFA par ligne ; Bois & panneaux 13,4 % du CA ; mai 1 258 801 FCFA ; décembre 5 292 517 FCFA. L'explication attendue : la moyenne des sept moyennes de catégories vaut 98 281 FCFA, la moyenne des 480 lignes 75 152 FCFA, et l'écart vient de la taille des groupes — chaque catégorie ne pèse pas le même nombre de lignes. En tableur : `=MOYENNE(plage_de_ca)/NB(plage)` n'est pas la moyenne des `=MOYENNE.SI.ENS(...)`, et le TCD qui agrège une colonne de moyennes reproduit exactement l'erreur.

E3. Trois valeurs de contrôle : les lignes rapportées (480), les 103 lignes sans client exploitable — dont 85 au client comptoir 0 et 18 au champ vide — et la clé `annee_mois` qui joint à 218 lignes d'objectifs, deux lignes de l'extrait restant sans cible. La voie `RECHERCHEV` casse si on insère une colonne dans `produits` (les index de retour décalent, sans erreur) ; `RECHERCHEX` ne casse pas (il recherche la valeur de la clé, pas une position) ; la relation du modèle ne casse pas non plus et supprime trois colonnes de recopie.

E4. (i) 36 339 317 : neuf lignes en double, l'écart de 266 132 FCFA se lit en comparant à `=SOMME` sur le fichier nettoyé — la formule de contrôle est `=SOMME(ventes!M:M)-SOMME(Données!M:M)`, attendu 0. (ii) 98 281 : une moyenne appliquée à des moyennes, la valeur par ligne est 75 152 et le panier par ticket 114 156 ; contrôle = `=SOMME(ca)/NB(lignes)`. (iii) 480 : un `NBVAL` sur une colonne de lignes de vente, qui compte 480 lignes pour 316 tickets et 372 clients distincts ; contrôle = le nombre distincts via le modèle, ou une colonne `=1/NB.SI()` sommée. (iv) 27,9 % : juste, mais rapporté à quatre jours de vente sur un mois de cible — la correction est éditoriale, pas arithmétique, et c'est la raison pour laquelle le barème la compte à part. Les trois premiers valent 2 points chacun (1 pour la cause, 1 pour la formule de contrôle), le quatrième 2 points, la ligne de mise en forme conditionnelle 0,5 point de bonus.

Total et décision d'orientation

Composante	Score	Seuil	Statut
B · Quiz	... /15	11	bloquant
C · Exercices	... /20	indicatif (démarche restituée = +0,5)	—
D · Étude de cas	... /20	12	bloquant
Projet M03.P	... /20	13	bloquant

Décision. B, C et D au seuil, projet au seuil → module validé, passage en M04 (qualité, préparation et documentation des données, où le nettoyage devient une trace écrite ; le SQL en miroir s'ouvre en M05). Quiz seulement en dessous → deux jours pour reprendre C02, C04 et C07, puis une nouvelle version de B. Étude de cas seulement en dessous → nouvelle version de D sur le même dossier, avec un entretien de quinze minutes sur l'extrait de l'échelle. Deux échecs → reprise du module en commençant par les six exercices autonomes de C08, puis le projet.

Évaluation M04 — « Qualité, préparation et documentation des données »

Module M04 · durée totale 2 h 30 · quatre épreuves · seuils : quiz 11/15, exercices 14/20, étude de cas 12/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	vérifier qu'on a lu les six chapitres avant de se tromper cher
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, quatre blocs
C · Exercices pratiques	50 min	/20 (auto-corrigé)	oui	quatre exercices menés jusqu'au chiffre sur le dossier du projet
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier en quatre pages : « le stock indiqué est-il crédible ? »

Le score du module se lit ainsi : **B ≥ 11** et **D ≥ 12** valident le module ; le projet M04.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D : savoir ce qu'est une quasi-identification ne dispense pas de dire si le comptage de 60 000 lignes est croyable. Tout nombre de cette évaluation est mesuré sur le socle livré ; les corrections le citent avec leur clé `m04_*/m04p_*` dans `chiffres_cites.json`.

A · Questions de récupération (non notées)

À traiter sans document, en une ou deux phrases chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail.

1. Citez les cinq mécanismes d'absence du C02 et le traitement qui va avec chacun. Pourquoi `id_client = 0` (43 681 lignes, 2 813 668 030 FCFA) n'est-il aucun des cinq ?
2. Donnez les quatre niveaux où l'on peut chercher un doublon (C03) et, pour chacun, le résultat mesuré sur le fichier de ventes (0 / 0 / 3 440 / 8 453).
3. Citez les quatre familles de règles de contrôle (C04) avec un nombre mesuré chacune — et la famille qui a rendu 24 180 alertes pour 0 erreur.
4. Quelles sont les deux voies indépendantes qui reconvertissent les 3 421 montants texte (C05), et quel nombre commun rendent-elles avant dédoublonnage ?
5. Quelles sont les quatre pièces que le C06 attend d'un dossier livré, et quel résidu **attendu** le contrôle final du fichier propre doit-il accepter (80 groupes) ?
6. Pourquoi les 973 remises à 12, 18, 25, 30 ne changent-elles aucun total, et pourquoi la division par 100 est-elle une réparation qui échoue son propre test ?
7. Quelle est la différence entre un identifiant direct et un quasi-identifiant, avec le chiffre du socle qui rend la distinction coûteuse (18 171 combinaisons uniques sur 20 809) ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

Bloc 1 — Types et normalisation (Q1 à Q4)

Q1. La colonne `montant_ttc` contient 3 421 cellules du type `n nnn FCFA` parmi 239 939 nombres. Ce que `=SOMME()` fait des 3 421 : a) les somme comme des nombres · b) les ignore, sans erreur · c) renvoie `#VALEUR!` · d) elles sont lues sans leur espace de milliers.

Q2. Normaliser `clients.nom` (élagage, minuscule, ponctuation retirée) fait passer 12 640 valeurs distinctes à 12 342. Conclusion : a) 298 clients ont été supprimés · b) 298 écritures différentes d'un même nom se sont rassemblées, sans que le nombre de clients change · c) 298 doublons exacts ont été fondus · d) la colonne est corrompue.

Q3. Deux écritures d'un même téléphone : `+226 78 62 21 81 98` et `+2267862218198`. La colonne qui sert de clé de jointure doit porter : a) le texte tel quel · b) les chiffres seuls · c) le nom du client · d) la ville.

Q4. `date_vente` est à 100 % en `AAAA-MM-JJ`, et 51 lignes portent une date postérieure au 31/08/2026, date du relevé. L'action correcte : a) une formule de reformatage · b) un contrôle de domaine ($\text{date} \leq \text{date du relevé}$) et une décision documentée · c) la suppression des 51 lignes · d) aucune, le format est correct.

Bloc 2 — Doublons et clés (Q5 à Q8)

Q5. La clé primaire rend 243 360 valeurs distinctes, mais 3 440 lignes sont en collision sur (`ticket`, `produit`, `quantité`). Conclusion : a) la clé primaire est fausse · b) la clé métier voit ce que la clé technique ne voit pas · c) le fichier est entièrement dupliqué · d) c'est une erreur de mesure.

Q6. 8 453 lignes partagent (`ticket`, `produit`) alors que la règle de garde n'en retire que 3 360. Explication : a) un choix aléatoire · b) un même ticket peut légitimement porter plusieurs lignes sur un même produit (`quantité`, `heure` ou `client` différents) · c) la clé compte mal · d) les 8 453 sont toutes des doublons.

Q7. La règle « même nom » sur les 23 912 clients fusionnerait 11 570 lignes. Réponse correcte : a) l'appliquer puis vérifier a posteriori · b) la rejeter : elle retire 11 158 lignes de trop, des homonymes réels · c) la limiter aux noms courts · d) l'appliquer aux noms avec espaces.

Q8. Les 412 quasi-doublons clients portent tous deux anomalies d'écriture (`nom` surchargé, `téléphone` sans espaces), et 0 ligne conservée ne les porte. Ce qui s'en déduit : a) l'anomalie de format est la règle de dédoublonnage · b) l'anomalie de format est un indice de rejeu, pas une règle : un doublon bien formaté ne serait plus vu · c) le fichier est corrompu · d) il faut supprimer tous les espaces du fichier.

Bloc 3 — Règles et valeurs aberrantes (Q9 à Q12)

Q9. `prix_unitaire_ht` : minimum 575, maximum 45 000 FCFA ; la règle des quartiles en signale 24 180 lignes. Ces 24 180 lignes sont : a) des erreurs de saisie · b) la dispersion légitime du métier (du sac de ciment à la palette d'ossature) · c) du bruit de mesure · d) des doublons.

Q10. La bande $[0,95 ; 1,35]$ autour du prix catalogue rend 0 violation ; l'égalité au catalogue rend 232 981 « anomalies ». Leçon : a) le catalogue est faux · b) une règle qui signale 96 % d'un fichier sain est une règle fausse, pas un fichier sale · c) il faut corriger les 232 981 prix · d) il faut utiliser le z-score.

Q11. 973 lignes ont `taux_remise` à 12, 18, 25 ou 30, mais un `montant_ht` correct et une TVA cohérente. La division par 100 laisse 258 lignes au-dessus du plafond de 0,25. L'action défendable : a)

diviser par 100 · b) reconstituer le taux depuis les montants (étendue 0,000 à 0,110, les 973 dans le domaine) et documenter · c) supprimer les 973 lignes · d) laisser tel quel sans note.

Q12. Le contrôle `qte_physique < seuil_alerte` déclenche sur exactement les mêmes 1 800 lignes que `qte_physique < 0`, sur 60 000. Conclusion : a) le seuil est parfait · b) le seuil ne discrimine rien : le signal réel est le signe ; les deux règles doivent être scindées (intégrité / risque) · c) les 1 800 lignes sont des erreurs à supprimer · d) le seuil est trop bas.

Bloc 4 — Documentation et protection (Q13 à Q15)

Q13. Le fichier propre garde 80 groupes en collision sur (ticket, produit, quantité), tous distincts par l'heure, 0 indistincts. Le fichier est-il correctement dédoublonné ? a) non, à refaire · b) oui : le résidu est attendu, écrit avant le contrôle, et les groupes sont des multi-lignes légitimes · c) c'est un bug du script · d) oui, tant que le résidu reste sous 100 groupes.

Q14. Une ligne de journal des transformations sans règle ni compteur avant → après est : a) une note, pas une ligne de journal · b) une ligne valide, abrégée · c) une exigence d'audit · d) un risque juridique seulement.

Q15. On retire nom, téléphone et e-mail des clients, on garde (ville, type, segment, date de création) : 18 171 combinaisons sur 20 809 désignent un client unique. Conclusion : a) le fichier est anonymisé · b) 87,3 % des combinaisons restent ré-identifiables : effacer l'identifiant direct n'est pas anonymiser · c) seuls 2 638 combinaisons ne désignent personne d'unique · d) il suffit de retirer la ville.

Corrigé du quiz

Q1 **b** — `SOMME` ignore le texte sans erreur : c'est ce qui rend le défaut silencieux (écart du total naïf : 2 111 789 FCFA). Q2 **b** — $12\,640 - 12\,342 = 298$ écritures rassemblées ; le nombre de lignes est inchangé. Q3 **b** — les chiffres seuls : c'est la colonne rare discriminante du C03. Q4 **b** — la norme est respectée, la réalité non ; la décision se documente, elle ne se corrige pas par formule. Q5 **b** — l'unicité technique n'est pas l'unicité métier. Q6 **b** — 8 453 lignes partagent (ticket, produit), mais la clé métier complète (avec la quantité) distingue les multi-lignes légitimes. Q7 **b** — $11\,570 - 412 = 11\,158$ lignes de trop. Q8 **b** — la règle de format décrit un lot passé ; la règle de décision reste la clé. Q9 **b** — 24 180 alertes pour 0 erreur. Q10 **b** — la règle se juge à ce qu'elle attrape, pas au volume d'alertes. Q11 **b** — la reconstitution tient sur les 973 ; la division échoue sur 258. Q12 **b** — deux règles au lieu d'une, un destinataire par niveau. Q13 **b** — le contrôle attend des résidus, pas des zéros universels. Q14 **a** — sans règle ni compteur, la ligne ne referme rien. Q15 **b** — 87,3 % de combinaisons uniques ; la date de création fait le travail des trois autres colonnes.

C · Exercices pratiques (50 min · /20)

Tout se fait sur `03_exercices/dossier_M04/` (le dossier du projet, 160 lignes ; `ATTENDU.json` à consulter **après** vos calculs).

E1 — La grille du dossier (5 points). Passez la grille des 12 points sur `commandes_2026.csv`. Note attendue : les quatre compteurs non nuls **avec dénominateurs** (17 montants texte sur 160 ; 5 remises hors domaine sur 160 ; 12 dates hors ISO sur 160 ; 8 lignes en doublon sur 160), plus la mention des 18 lignes au comptoir comme convention et non comme donnée manquante. 1 pt par compteur justifié, 1 pt pour la séparation « total / analyse ».

E2 — Le dédoublonnage du dossier (5 points). Produisez 152 lignes et le total **8 209 940 FCFA**. Note attendue

les 8 lignes retirées motivées (7 copies exactes + 1 resaisie reconnue à son champ unique, la date), le groupe légitime (2 lignes, 8 colonnes différentes) survécute, et le chiffre de la règle naïve (151 lignes, 8 105 333 FCFA, l'écart 104 607 FCFA identifié comme le montant de la ligne mangée). 2 pts pour le résultat, 2 pour la motivation ligne à ligne, 1 pour l'opposition à la règle naïve.

E3 — Conversion et import (5 points). (a) Convertissez les 17 montants texte par le double retrait (suffixe puis espaces) et montrez que les 2 retours à montant texte ne passent le contrôle de domaine (retour $\Rightarrow$ montant négatif) qu'après conversion : 1,5 pt. (b) Importez `tarif_fournisseur.csv` en déclarant l'encodage Windows-1252, le séparateur point-virgule et les 3 lignes à sauter ; lisez les 10 prix à virgule décimale par une formule qui énonce la virgule ; écrivez le coût du mauvais choix d'encodage (39 octets dont le décodage Latin-1 fait des caractères de contrôle invisibles) : 3,5 pts.

E4 — « Les trois chiffres faux » (5 points). Le corrigé enseignant `rapport_defaults.json` du socle annonce 3 360 montants en texte, 2 640 dates en formats mixtes, 1 880 clients sans ville. Mesurez les trois valeurs réellement livrées — 3 421, 0 et 1 911 — par un compteur unique chacune (3 pts), et écrivez la phrase qui dit pourquoi un corrigé se génère **après** la fabrication du fichier, jamais avant (2 pts). Mention attendue : le cas « 61 catégories hétéroclites annoncées, 44 ou 61 selon la définition » est une question de définition, pas une troisième erreur — confondre les deux est le piège de l'exercice.

D · Étude de cas (60 min · /20, seuil 12) — « Le stock indiqué est-il crédible ? »

Contexte. Le magasinier a remis `stocks_quotidiens.csv` : 60 000 lignes, 7 colonnes (date, id_produit, id_magasin, qte_theorique, qte_physique, seuil_alerte, valeur_stock_ht). La direction demande : peut-on se fier au stock indiqué ? Vous rendez un dossier de quatre pages.

Le fichier, mesuré. 154 produits $\times$ 6 dépôts = 924 séries, sur 1 339 jours distincts (du 01/01/2023 au 31/08/2026). 0 cellule vide. `qte_theorique` : 0 à 1 795, **aucun négatif**. `qte_physique` : **1 800 négatifs** (3,0 % des lignes), présents sur les 154 produits sans exception. `seuil_alerte` : 0 à 448. Le contrôle `qte_physique < seuil_alerte` déclenche sur exactement les mêmes 1 800 lignes que `qte_physique < 0`. Écart théorique – physique : moyenne 2,77, maximum 532, **118 lignes au-dessus de 100**, 2 au-dessus de 500.

Le livrable, en quatre parties :

1. **Ce que le fichier est** (description et grain) : 3 points.
2. **Le verdict de crédibilité, en deux temps** : comme comptage (1 800 négatifs impossibles sur 154 produits : le comptage n'est pas fiable) et comme signal (les 1 800 alertes sont exactement les 1 800 négatifs : le seuil n'ajoute rien, le signal utile est le signe) : 4 points.
3. **Le calibrage de l'écart** : avec les valeurs mesurées et leurs dénominateurs, délimitez la file de travail — 1 800 négatifs à recompter + 118 écarts au-dessus de 100 à instruire — et dites pourquoi 532 de maximum sur une moyenne de 2,77 change la nature du problème (queue, pas bruit) : 4 points.
4. **Trois règles qui rendraient le prochain fichier crédible**, chacune avec son niveau (bloquant / corriger / surveiller / ignorer tracé) et son destinataire : (a) intégrité — quantité physique négative $\Rightarrow$ recomptage programmé (niveau 2, magasinier) ; (b) risque — seuil recalibré pour un taux d'alerte traitable (les 118 plus grands écarts d'abord, niveau 3, responsable de dépôt) ; (c) journal de comptage — date, produit, dépôt, théorique, physique, écart, compteur (le journal que ce fichier n'a pas) : 4 points.

Grille de qualité (complément des 4 parties, 5 points). Ce que le fichier ne peut pas dire : 3 points — `qte_theorique` est supposé juste, or le logiciel de gestion n'est pas dans le dossier : la cohérence externe (ventes ↔ mouvements de stock) est impossible à vérifier ici, et le dossier le dit. Reproductibilité : 2 points — chaque chiffre du dossier est reproductible par une formule ou un compteur nommé.

Ce qui est piégé dans l'étude. (1) Répondre « oui, c'est fiable » ou « non, c'est faux » d'un bloc : le verdict doit être en deux temps. (2) Proposer de supprimer les 1 800 lignes négatives : ce sont le signal, pas le bruit. (3) Citer un écart sans son dénominateur. (4) Oublier que le seuil actuel est **redondant** avec le signe — la règle qui compte deux fois le même événement n'est pas une règle, c'est du doublon de contrôle (le mot du C03, réemployé au contrôle).

Lecture des résultats

- **$B \geq 11$ et $D \geq 12$** : module validé (le projet M04.P doit, de son côté, atteindre 13/20).
- **$B < 11$** : le socle des définitions n'est pas acquis ; repasser B et la récupération A, le module reste ouvert.
- **$D < 12$ avec $B \geq 11$** : le candidat sait ce qu'est un quasi-identifiant mais ne sait pas dire si un comptage de 60 000 lignes est croyable ; repasser D uniquement.
- Le projet M04.P se note séparément (13/20 de seuil) et pèse sur la validation finale du module.

Évaluation M05 — « Le langage commun des transformations »

Module M05 · durée totale 2 h 30 · quatre épreuves · seuils : quiz 11/15, exercices 14/20, étude de cas 12/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	vérifier qu'on a lu les cinq chapitres avant de se tromper cher
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, cinq blocs
C · Exercices pratiques	50 min	/20 (auto-corrigé)	oui	quatre exercices menés jusqu'au chiffre sur le dossier du projet
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier en quatre pages : « un pipeline, trois moteurs — qui dit la même chose ? »

Le score du module se lit ainsi : **B ≥ 11** et **D ≥ 12** valident le module ; le projet M05.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D : savoir ce qu'est un pipeline portable ne dispense pas de dire si deux moteurs produisent la même table au byte près. Tout nombre de cette évaluation est mesuré sur le socle livré ; les corrections le citent avec leur clé `m05_*/m05p_*` dans `chiffres_cites.json`.

A · Questions de récupération (non notées)

À traiter sans document, en une ou deux phrases chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail.

1. Citez les **10 opérations** du langage commun de C01 et la classe (M, Opérateur, Paramètre, Agrégation) de chacune.
2. Donnez les **14 étapes** du pipeline Power Query du C02 (sommaire, en groupes : source → ty-page → réparation → jointure → sortie).
3. Citez les **11 tables temporaires** (v1 à v6 du corrigé) du pipeline SQL DuckDB de C03, et la règle de transition entre deux tables consécutives.
4. Donnez les **8 cellules** du notebook pandas de C04 (en-têtes sans code) et la chaîne de méthodes canonique `assign` → `sort_values` → `drop_duplicates`.
5. Citez les **4 critères** du choix d'outil de C05, et pour chacun le seuil indicatif qui fait basculer le choix (par exemple : < 100 000 lignes pour la taille).
6. Pourquoi la séquence « dédoublonner **avant** la conversion des montants texte » est-elle fautive dans 50 % des cas du projet ?
7. Quelle est la différence entre le verdict du fil rouge (mars 2025 = 6 884 lignes) et l'empreinte sha256 du projet M05.P (7cce2d0c...) ? Pourquoi les deux sont-ils nécessaires ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

Bloc 1 — Le langage commun (Q1 à Q3)

Q1. Le pipeline Power Query, SQL DuckDB et pandas partagent une grammaire commune. Le verbe « promote headers » appartient à : a) la classe M · b) la classe Opérateur · c) la classe Paramètre · d) la classe Agrégation.

Q2. L'opération `TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS BIGINT)` se traduit en pandas par : a) `df['montant_ttc'].astype(int)` · b) `pd.to_numeric(df['montant_ttc'].str.replace(' FCFA', '', regex=False).str.replace(' ', '', regex=False), errors='coerce')` · c) `df.assign(montant_ttc=int)` · d) une boucle `for` sur les lignes.

Q3. Le paramètre `MoisCible = 3` lit en réalité la cellule B1 du fichier `parametres.xlsx`. Ce mécanisme : a) permet de rejouer le pipeline sans modifier le code · b) est le seul moyen de passer une constante en M · c) ne sert à rien en pandas · d) force l'utilisation de Power Query exclusivement.

Bloc 2 — Power Query (C02) (Q4 à Q6)

Q4. Power Query applique automatiquement le typage « colonne entière » au chargement. Quand l'erreur apparaît : a) au moment de l'ouverture · b) au moment où une étape downstream utilise la colonne dans un type différent · c) au moment de l'enregistrement · d) jamais, l'auto-typage est sans erreur.

Q5. L'étape « Remplir vers le bas » (down-fill) propage la valeur précédente. Sur la colonne `id_client` du brut (28 516 920 o), elle s'applique : a) sur 1 911 cellules (vides après la première) · b) sur 0 cellule · c) sur 28 516 920 cellules · d) sur 12 colonnes à la fois.

Q6. La fusion (jointure Power Query) entre `ventes_brutes` et `clients.csv` sur `id_client` seul donne **41 716 lignes enrichies** au lieu de **1 714**. Conclusion : a) le fichier `clients` est faux · b) la jointure naïve duplique chaque vente sur tous les clients qui partagent l'`id_client` ; la clé juste inclut `id_client × année × mois` · c) Power Query est en cause · d) il faut refaire la fusion sans condition.

Bloc 3 — SQL DuckDB (C03) (Q7 à Q9)

Q7. La fonction `TRY_CAST(montant AS BIGINT)` retourne `NULL` si la conversion échoue, sans lever d'erreur. Par rapport à `CAST` qui lève : a) `TRY_CAST` est plus rapide · b) `TRY_CAST` rend le défaut silencieux, exactement comme Excel `SOMME()` ignore le texte · c) `TRY_CAST` n'a aucun avantage · d) `TRY_CAST` est réservé à DuckDB.

Q8. La table `v_dedup` est calculée par :

```
WITH cle AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY id_ticket, id_produit, quantite, ttc_n, heure
    ORDER BY id_vente
  ) AS rn
  FROM v_mars
)
SELECT * EXCLUDE rn FROM cle WHERE rn = 1
```

Le nombre de lignes en sortie sur mars 2025 (`v_dedup`) : a) 6 884 (avant dédoublement) · b) 6 798 (86 doublons retirés) · c) 7 270 (tous les doublons exacts + variantes) · d) 101 (les enrichies).

Q9. La session DuckDB étant en mémoire (`:memory:`), chaque bloc SQL d'une même évaluation repart de zéro. Conséquence : a) le bloc `v_mars` doit être recréé dans chaque `SELECT` · b) DuckDB ne sait pas garder l'état · c) DuckDB est plus rapide qu'une session persistante · d) il faut sauvegarder en CSV entre chaque bloc.

Bloc 4 — pandas (C04) (Q10 à Q12)

Q10. La chaîne `df.assigned(montant_ttc_n=...).sort_values(['date_vente', 'id_vente']).drop_duplicates(CLE)` produit un résultat différent si on intervertit `sort_values` et `drop_duplicates`. Pourquoi ? a) pandas n'a pas de garantie d'ordre sans tri explicite · b) `sort_values` change les types des colonnes · c) `drop_duplicates` sans tri garde la dernière occurrence observée, pas la première dans l'ordre chronologique · d) ce n'est pas un problème.

Q11. L'`SettingWithCopyWarning` apparaît sur la ligne `df[df['montant_ttc'] > 1000]['ttc_n'] = 0`. La parade : a) ignorer l'avertissement · b) `df.loc[mask, 'ttc_n'] = 0` (indexeur explicite) · c) reconstruire le DataFrame à la main · d) utiliser `np.where`.

Q12. La cellule 8 du notebook C04 fait le verdict :

```
assert hashlib.sha256(df_final.to_parquet()).hexdigest() == '7cce2d0c...'
```

Ce que cette cellule prouve : a) le notebook s'exécute · b) pandas produit la même table que DuckDB au byte près après sérialisation Parquet · c) le fichier est valide · d) la fonction sha256 fonctionne.

Bloc 5 — Arbitrage (C05) (Q13 à Q15)

Q13. Sur 6 884 lignes, pandas est **13 × plus rapide** que DuckDB (22 ms vs 283 ms). Sur 1 million de lignes, DuckDB est **3 × plus rapide**. Le critère pertinent du choix : a) la taille seule · b) la taille **prévisionnelle** (croît-il ?) · c) la marque du poste · d) le système d'exploitation.

Q14. Le verdict « même empreinte sha256 entre pandas et DuckDB » prouve : a) que les deux outils sont les mêmes · b) que le pipeline est **portable** (ne dépend pas de l'implémentation) · c) que pandas est plus rapide · d) que DuckDB est correct.

Q15. Une équipe bureautique (Excel, pas de Python) reçoit un fichier de 750k lignes mis à jour quotidiennement. L'outil adapté : a) pandas, parce que c'est le plus moderne · b) DuckDB, parce que c'est le plus rapide · c) Power Query (si Microsoft 365) avec orchestration Power Automate · d) aucun, c'est impossible.

Corrigé du quiz

Q1 **b** — « promote headers » est un Opérateur (transformation de la structure), pas une M (manipulation d'une seule colonne). Q2 **b** — la traduction pandas est littérale : `str.replace` pour `REPLACE`, `pd.to_numeric` pour `TRY_CAST`, `errors='coerce'` pour le `NULL` en cas d'échec. Q3 **a** — la constante `MoisCible` externalisée dans `parametres.xlsx` est l'objet du C02 §5.13. Q4 **b** — Power Query applique le typage paresseusement ; l'erreur survient au recalcul d'une étape en aval. Q5 **a** — 1 911 cellules vides dans `id_client` du brut, toutes remplies par down-fill (`id_client` en `NaN` devient la valeur précédente). Q6 **b** — la jointure naïve sur `id_client` seul duplique chaque vente sur tous les clients qui partagent cet identifiant ; la clé juste inclut `id_client × année × mois`. Q7 **b** — `TRY_CAST` rend le défaut silencieux, comme Excel `SOMME` ignore le texte (cf. M04 §5.6). Q8 **b** — 6 798 (= 6 884 – 86), 86 étant `m05_filrouge_mars_doublons`. Q9 **a** — la session `:memory:` repart de zéro ; il faut recréer les tables dans chaque bloc. Q10 **c** — sans tri explicite, `drop_duplicates(keep='first')` garde la première occurrence **dans l'ordre de lecture** (par `id_vente`), pas la première dans l'ordre **chronologique**. Q11 **b** — `df.loc[mask, 'ttc_n'] = 0` est l'indexeur explicite qui neutralise le warning. Q12 **b** — la cellule fait le **verdict croisé** : pandas et DuckDB produisent la même table au byte près. Q13 **b** — la taille prévisionnelle, pas la taille courante. Q14 **b** — le verdict prouve la **portabilité** du pipeline. Q15 **c** — Power Query + Power Automate est la combinaison adaptée à ce profil.

C · Exercices pratiques (50 min · /20)

Tout se fait sur `03_exercices/dossier_M05/` (le dossier du projet ; ATTENDU.json à consulter après vos calculs).

E1 — La grille du diagnostic (5 points). Passez la grille des 12 points du M04.C01 sur `ventes_2023_2024.csv`. Note attendue : les **5 compteurs non nuls avec dénominateurs** (1 731 montants texte sur 121 720 ; 1 720 doublons sur 121 720 ; 493 dates transposées ; 252 clients inconnus ; 488 remises hors domaine), plus la séparation « 4 défauts qui touchent au total / 5 défauts qui faussent chacun une lecture différente ». 1 pt par compteur justifié, 1 pt pour la séparation.

E2 — Le pipeline pandas complet (5 points). Produisez le `df_final` à **120 000 lignes, total TTC 7 145 910 735 FCFA**. Note attendue : (a) la conversion des 1 731 montants texte par `assign/TRY_CAST` ; (b) la transposition des 493 dates par colonne dérivée ; (c) le dédoublonnage 1 720 lignes (1 680 copies + 40 variantes) ; (d) le verdict `assert sha256(df_final) == '7cce2d0c...`. 2 pts pour le résultat, 2 pour la chaîne canonique dans l'ordre, 1 pour le verdict sha256.

E3 — Le pipeline SQL DuckDB équivalent (5 points). Réécrivez le pipeline E2 en SQL DuckDB (11 tables temporaires). Note attendue : (a) `v1` source brute ; (b) `v2` typage `TRY_CAST` ; (c) `v3` réparation des dates ; (d) `v_mars` coupe `BETWEEN 01-03 et 31-03` ; (e) `v_dedup` `ROW_NUMBER` sur clé 5-colonnes ; (f) `v_final` jointure clients clé complète contre 41 716 naïves ; (g) le verdict empreinte sha256 identique. 1 pt par étape, plus 2 pts pour le verdict croisé.

E4 — L'arbitrage DuckDB / pandas sur 3 tailles (5 points). Mesurez sur votre machine le temps d'exécution du pipeline sur (a) 6 884 lignes (mars 2025 du brut), (b) 121 720 lignes (projet), (c) 1 million de lignes (généré par duplication). Le barème : 1,5 pt par mesure × 3, plus 0,5 pt pour la comparaison avec les 5 chiffres du C05 §5.3. Indication attendue : sur (a), pandas domine ($\approx 13 \times$) ; sur (b), DuckDB prend un léger avantage ($\approx 1,4 \times$) ; sur (c), DuckDB domine ($\approx 3 \times$). Si vos mesures s'écartent de plus de 50 % des chiffres de référence du C05, vous documentez l'écart (matériel, RAM, SSD vs disque rotatif, etc.) — pas un défaut, un fait.

D · Étude de cas (60 min · /20, seuil 12) — « Un pipeline, trois moteurs — qui dit la même chose ? »

Contexte. Un data engineer d'une enseigne de distribution vous transmet la question : « j'ai trois versions de la même table propre, faites-en un dossier qui dit si elles disent la même chose — et lequel je garde comme référence ». Vous rendez un dossier de quatre pages.

Le matériel. `03_exercices/dossier_M05/` (121 720 lignes brutes ; 120 000 après réparation ; total 7 145 910 735 FCFA ; 1 714 enrichies ; 31 clés dans ATTENDU.json). Trois pipelines dans `02_modules/` : - (a) `M05_C02` — Power Query, 14 étapes manuelles (M code documenté, non exécutable dans cet atelier) ; - (b) `M05_C03` — SQL DuckDB, 11 CTE (`controle_sql.py M05` rend 11 requêtes exécutées) ; - (c) `M05_C04` — pandas, 8 cellules de notebook (`controle_python.py M05` rend 15 fragments exécutés).

Le livrable, en quatre parties :

1. **Ce que disent les pipelines** (description et grain) : 3 points.
2. **Le verdict croisé, en quatre temps** : (a) même total TTC 7 145 910 735 FCFA (oui) ; (b) même nombre de lignes 120 000 (oui) ; (c) même nombre d'enrichies 1 714 (oui) ; (d) même empreinte sha256 `7cce2d0c...` (oui entre pandas et DuckDB, non vérifié pour Power Query) : 4 points.
3. **L'arbitrage final** : sur les 4 critères du C05 §7 (taille, fréquence, compétences, gouvernance), lequel des trois reste **référence** ? Argumentation : 4 points. Indication attendue : Du-

ckDB en première intention (portable, plus rapide à l'échelle, schémas SQL centralisés) ; pandas en repli (plus riche en méthodes) ; Power Query en vitrine (pour le reporting Excel des commerciaux qui ne font pas de Python).

4. **Trois règles que le prochain projet devrait imposer**, chacune avec son niveau (bloquant / corriger / surveiller) et son destinataire : (a) **empreinte sha256 systématique** sur `df_final` à chaque exécution (niveau 1, bloquant, contrôleur CI) ; (b) **mesure de temps** sur 3 tailles avant de figer le choix (niveau 2, corriger, mainteneur) ; (c) **README du dossier avec les 4 critères** (niveau 3, surveiller, équipe) : 4 points.

Grille de qualité (complément des 4 parties, 5 points). Reproductibilité : 3 points — chaque chiffre du dossier est reproductible par une formule ou un compteur nommé. Limites : 2 points — ce que les 3 moteurs ne disent pas : Power Query non exécutable dans cet atelier, lecture manuelle des 14 étapes.

Ce qui est piégé dans l'étude. (1) Conclure « oui, ils disent la même chose » sans le **chiffrer** : le verdict doit être en quatre temps (total, lignes, enrichies, sha256). (2) Donner DuckDB comme référence par défaut sans l'avoir justifié par les 4 critères. (3) Citer une empreinte sha256 sans sa clé `m05p empreinte_sha256`. (4) Oublier que la **mesure de temps** est ce qui distingue un choix réfléchi d'un choix par défaut — c'est la règle 3 du plan M05 §3 (« mesurer, pas extrapoler »).

Lecture des résultats

- **B ≥ 11 et D ≥ 12** : module validé (le projet M05.P doit, de son côté, atteindre 13/20).
- **B < 11** : le socle des définitions n'est pas acquis ; repasser B et la récupération A, le module reste ouvert.
- **D < 12 avec B ≥ 11** : le candidat sait ce qu'est un pipeline portable mais ne sait pas dire si deux moteurs produisent la même table au byte près ; repasser D uniquement.
- Le projet M05.P se note séparément (13/20 de seuil) et pèse sur la validation finale du module.

Évaluation M06 — « Bases de données relationnelles : comprendre avant de requêter »

Module M06 · durée totale 2 h 30 · quatre épreuves · seuils : quiz 11/15, exercices 14/20, étude de cas 12/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	15 min	non noté	non	vérifier qu'on a lu les cinq chapitres avant de se tromper cher
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, cinq blocs
C · Exercices pratiques	50 min	/20 (auto-corrigé)	oui	deux exercices menés jusqu'au chiffre sur le dossier du module
D · Étude de cas	60 min	/20, seuil 12	oui	un dossier en deux pages : « sept tables pour une quincaillerie »

Le score du module se lit ainsi : **B ≥ 11** et **D ≥ 12** valident le module ; le projet M06.P doit atteindre 13. Un candidat qui réussit B et échoue à D repasse uniquement D : savoir qu'une clé étrangère pointe vers une primary key ne dispense pas de vérifier qu'elle ne pointe pas dans le vide. Tout nombre de cette évaluation est mesuré sur le socle livré ; les corrections le citent avec leur clé `m06_*/m06p_*` dans `chiffres_cites.json`.

A · Questions de récupération (non notées)

À traiter sans document, en une ou deux phrases chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail.

1. Citez les **5 limites** d'un classeur Excel listées en C01, et la solution qu'apporte un SGBD pour chacune.
2. Quelle est la différence entre une **clé primaire** et une **clé étrangère** ? Laquelle des deux est forcément indexée ?
3. Citez les **3 formes normales** (1NF, 2NF, 3NF) et donnez, pour chacune, le défaut qu'elle élimine (atomicité, dépendance partielle, dépendance transitive).
4. Citez les **5 actions** de `ON DELETE` (RESTRICT, CASCADE, SET NULL, SET DEFAULT, NO ACTION) et indiquez celle qui est la valeur par défaut en SQLite.
5. Pourquoi `PRAGMA foreign_keys = ON` est-il **obligatoire** en SQLite mais pas en PostgreSQL ?
6. Donnez la **règle d'or** des relations (direction de la FK) et l'erreur qu'elle évite.
7. Quelle est la différence entre **ACID** et **BASE** ? Lequel des deux s'applique à DuckDB ?
8. Citez les **4 familles NoSQL** et, pour chacune, un cas d'usage où elle bat le relationnel.
9. Qu'est-ce qu'un **entrepôt de données** (DWH), un **lac** (datalake) et un **lakehouse** ? Lequel stocke des données brutes sans validation ?
10. Pourquoi le module M06 **ne fait pas exécuter** PostgreSQL, et dans quel module suivant cette exécution aura-t-elle lieu ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

Bloc 1 — Pourquoi une base (C01) (Q1 à Q3)

Q1. Le classeur Excel s'effondre typiquement à partir de : a) 100 lignes · b) **50 000 lignes ou la 3^e table jointe** · c) 1 million de lignes · d) il ne s'effondre jamais, c'est un mythe.

Q2. Le SGBD apporte **5 propriétés** qu'Excel ne garantit pas. Laquelle manque le plus dans une feuille de calcul partagée ? a) la couleur des cellules · b) **l'intégrité référentielle** · c) le tri alphabétique · d) le formatage conditionnel.

Q3. « Une base de données, c'est juste un fichier avec des tables. » Cette phrase est : a) vraie · b) **fausse** — un SGBD est un moteur qui gère la concurrence, les transactions et l'intégrité ; le fichier n'en est qu'un effet · c) vraie pour SQLite uniquement · d) vraie pour PostgreSQL uniquement.

Bloc 2 — Tables, colonnes, schéma (C02) (Q4 à Q6)

Q4. La 1NF exige que chaque cellule soit : a) **atomique** (une seule valeur) · b) colorisée en jaune · c) numérique · d) en majuscules.

Q5. Le type `DECIMAL(14, 2)` pour `montant_ttc` : a) accepte les lettres si elles sont en majuscules · b) **refuse tout caractère non numérique et tronque à 2 décimales** · c) est identique à `FLOAT` · d) est réservé à PostgreSQL.

Q6. Une colonne `mode_paiement` qui prend les valeurs {Espèces, Carte, Mobile money, Crédit} est mieux modélisée par : a) une colonne `VARCHAR` libre · b) **une table de référence `mode_paiement` avec une FK** (1NF + 3NF, évite les fautes de frappe) · c) une colonne `INTEGER` avec des codes magiques (1=Espèces, 2=Carte...) sans documentation · d) un fichier JSON par ligne.

Bloc 3 — Clés et relations (C03) (Q7 à Q9)

Q7. Une clé primaire : a) peut être dupliquée si on l'écrit en rouge · b) **identifie une ligne de façon unique et n'est jamais NULL** · c) peut être NULL une fois par table · d) doit toujours être un entier auto-incrémenté.

Q8. Dans une relation 1-N entre `client` (1) et `vente` (N) : a) la clé étrangère est dans `client.id_client` · b) **la clé étrangère est dans `vente.id_client` (du N vers le 1)** · c) la clé étrangère est dans les deux tables · d) il n'y a pas de clé étrangère, juste un lien logique.

Q9. L'action `ON DELETE RESTRICT` : a) supprime la ligne parente et toutes ses filles · b) **refuse la suppression de la ligne parente si des filles existent** (sécurité par défaut) · c) met la FK des filles à NULL · d) ne fait rien, sans message d'erreur.

Bloc 4 — DuckDB / SQLite / PostgreSQL (C04) (Q10 à Q12)

Q10. DuckDB est : a) un serveur client-serveur comme PostgreSQL · b) **un moteur embarqué, in-process, mono-utilisateur** (comme SQLite mais orienté analytique) · c) un outil NoSQL · d) une interface graphique.

Q11. SQLite et PostgreSQL diffèrent sur : a) le langage utilisé (l'un parle SQL, l'autre non) · b) **l'activation des FK par défaut (OFF en SQLite, ON en PostgreSQL)** et la portée (fichier vs serveur) · c) le prix (l'un est gratuit, l'autre non) · d) ils sont identiques.

Q12. Dans ce module, PostgreSQL est : a) **cité sans exécuté** (règle M06 §1.5) ; les commandes sont publiées mais non lancées · b) exécuté sur le serveur de l'école · c) ignoré complètement · d) exécuté via DuckDB.

Bloc 5 — Paysage NoSQL / DWH / datalake (C05) (Q13 à Q15)

Q13. Les 4 familles NoSQL sont : a) **document, clé-valeur, graphe, colonnes** (MongoDB, Redis, Neo4j, Cassandra) · b) relationnel, objet, XML, JSON · c) MySQL, PostgreSQL, SQLite, DuckDB · d) Excel, Access, LibreOffice, Google Sheets.

Q14. Un **datalake** stocke : a) uniquement des données validées et propres · b) **des données brutes, sans validation préalable, au format natif** · c) exclusivement des données structurées · d) des fichiers Excel uniquement.

Q15. Le verdict 80/20 du module signifie : a) 80 % de NoSQL, 20 % de SQL · b) **80 % des cas utilisent SQL (relationnel) ; 20 % justifient NoSQL** quand le relationnel est inadapté · c) 80 % des données sont propres, 20 % sont sales · d) le module est fini à 80 %.

C · Exercices pratiques (2 exercices · 20 points)

Exercice 1 — Vérifier l'intégrité référentielle (10 points)

À partir de `03_exercices/dossier_M06/quincaillerie_export.csv` (24 000 l. × 42 col, graine 43) :

- (3 pts)** En DuckDB, charger le CSV, créer la table `client(id_client PRIMARY KEY, ...)` puis vérifier combien de `id_client` du CSV **n'existent pas** dans la liste des clients. Réponse attendue : 3 (`m06_clients_orphelins_a_ecarter`).
- (3 pts)** Écrire la requête DuckDB qui retourne les `id_vente` concernés par ces 3 orphelins. Donnez la requête et le nombre de lignes retournées (24 000 ventes initiales - N ventes touchées).
- (2 pts)** Reprendre la même requête en SQLite (en activant `PRAGMA foreign_keys = ON` et en utilisant le schéma 3FN). Comparer le résultat : doit être identique.
- (2 pts)** Citer la commande PostgreSQL qui ferait la même vérification, **sans l'exécuter**.

Exercice 2 — Convertir 142 montants en texte (10 points)

- (4 pts)** En DuckDB, écrire la requête qui convertit `montant_ttc` (texte avec « FCFA » et espaces) en `DECIMAL(14, 2)` et qui retourne le nombre de lignes où la conversion a réussi. Indice : `TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS DECIMAL(14, 2))`. Le nombre attendu : 24 000 - 142 = 23 858 lignes converties (`m06_montants_texte_export = 142`).
- (3 pts)** Pourquoi `TRY_CAST` plutôt que `CAST` ? Que retourne `TRY_CAST` quand la conversion échoue ?
- (3 pts)** Écrire l'équivalent en pandas (`pd.to_numeric(df['montant_ttc'].str.replace(...), errors='coerce')`). Citer le module où cette opération a été vue pour la première fois.

D · Étude de cas (1 cas · 20 points · seuil 12/20)

« Sept tables pour une quincaillerie »

Une enseigne de quincaillerie à Bobo-Dioulasso vous confie son export quotidien (`quincaillerie_export.csv`, 24 000 l. × 42 col, 4 défauts : 142 montants en texte, 8 doublons, 12 dates inversées, 3 clients orphelins).

Le directeur vous demande de produire un dossier en deux pages comprenant :

1. **(5 pts)** Le **schéma 3FN en 7 tables** avec PK, FK et contraintes CHECK. Vous nommerez explicitement chaque défaut que la 3FN élimine (atomicité, redondance, dépendance transitive).
2. **(5 pts)** La **démarche de chargement DuckDB** qui nettoie les 4 défauts (étape par étape, avec la commande DuckDB et la sortie observée). Vous citerez les **3 chiffres mesurés** : 142, 8, 12, 3.
3. **(5 pts)** La **démarche SQLite** équivalente, avec `PRAGMA foreign_keys = ON` et `INTEGER PRIMARY KEY` (au lieu de `SERIAL` PostgreSQL). Différences et points communs avec DuckDB.
4. **(5 pts)** Le **verdict structuré** : quel moteur choisir pour ce cas (volumétrie, utilisateurs, gouvernance), et dans quel cas on basculerait vers PostgreSQL ou NoSQL. Citez le « 80 / 20 » du module.

Critères de notation : - 12/20 : les 4 défauts sont identifiés et le schéma 3FN est juste, mais le verdict est imprécis. - 16/20 : tout est juste, DuckDB et SQLite sont cités avec une commande exécutée, le verdict est chiffré. - 20/20 : idem + au moins un cas de basculement justifié (« si on passait à 5 data analysts en production, on prendrait PostgreSQL parce que... »).

Barème récapitulatif

Épreuve	Barème	Seuil	Pondération module
A · Récupération	non noté	—	0 %
B · Quiz	/15	≥ 11/15	30 %
C · Exercices	/20	≥ 14/20	30 %
D · Étude de cas	/20	≥ 12/20	40 %
M06.P (projet)	/20	≥ 13/20	indépendant

Score module = $0,3 \cdot B + 0,3 \cdot C + 0,4 \cdot D$. Module validé si score ≥ 13/20 **et** chaque seuil respecté. Un candidat qui échoue à D peut repasser uniquement D ; un candidat qui échoue à B doit reprendre le module entier.

Évaluation M07 — « SQL : interroger, agréger, rejoindre »

Module M07 · durée totale 3 h · quatre épreuves · seuils : quiz 13/20, exercices 28/35, étude de cas 12/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	10 min	non noté	non	vérifier qu'on a lu les huit chapitres avant de se tromper cher
B · Quiz	25 min	/20	oui	vingt questions à réponse unique, cinq blocs
C · Exercices chiffrés	40 min	/35 (auto-corrigé)	oui	trente-cinq chiffres sur <code>commercial.duckdb</code> , auto-validés par <code>tools/controle_exos_M07.py</code>
D · Étude de cas	75 min	/20, seuil 12	oui	le rapport de la direction commerciale (6 requêtes + 1 page de synthèse)

Le score du module se lit ainsi : **B ≥ 13** et **D ≥ 12** valident le module ; le projet M07.P doit atteindre 13/20. Un candidat qui réussit B et échoue à D repasse uniquement D : savoir que `HAVING` filtre les groupes ne dispense pas de vérifier que la jointure ne double pas les lignes. Tout nombre de cette évaluation est mesuré sur le socle livré (`commercial.duckdb`, empreinte `df9333ff...a1dde`) ; les corrections le citent avec leur clé `m07_*` / `m07p_*` dans `chiffres_cites.json`.

A · Questions de récupération (non notées)

À traiter sans document, en une ou deux phrases chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail.

1. Citez l'**ordre d'exécution** d'une requête SQL complète (C01), et expliquez pourquoi `LIMIT 5` sans `ORDER BY` est non déterministe.
2. Quelle est la différence entre `COUNT(*)`, `COUNT(colonne)` et `COUNT(DISTINCT colonne)` ? (C04)
3. Où s'écrit un filtre sur un **total** : `WHERE` ou `HAVING` ? Pourquoi ? (C05)
4. Pourquoi un libellé **double** est-il dangereux dans un `GROUP BY` ? Donnez l'exemple du socle. (C05)
5. Que renvoie `SUM(colonne)` si toutes les valeurs sont `NULL` ? Et comment forcer 0 ? (C04)
6. Quelle est la différence entre `COALESCE` et `NULLIF` ? (C06)
7. Pourquoi la condition du côté droit d'un `LEFT JOIN` doit-elle aller dans l'`ON` ? (C07)
8. Comment écrit-on « les clients qui ont acheté **tous** les produits X » ? (C08)

B · Quiz (20 questions · 20 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 20.

Bloc 1 — Lire et filtrer (C01-C02) (Q1 à Q4)

Q1. Dans l'ordre d'exécution, la première clause traitée est : a) `SELECT` · b) `FROM` · c) `WHERE` · d) `ORDER BY`.

Q2. L'alias d'une colonne sert à : a) renommer la colonne dans la table · b) **désigner une colonne ou une table dans la requête (tri, regroupement, lisibilité)** · c) créer un index · d) éviter les erreurs de type.

Q3. `SELECT DISTINCT ville FROM client;` : a) trie les villes · b) **supprime les lignes de résultats identiques** · c) supprime les colonnes en double · d) limite à une ligne.

Q4. `WHERE a = 1 OR b = 2 AND c = 3` équivaut à : a) `(a = 1 OR b = 2) AND c = 3` · b) **`a = 1 OR (b = 2 AND c = 3)` — `AND` se lie avant `OR`** · c) `(a = 1 OR b = 2)` sans `c` · d) une erreur.

Bloc 2 — Trier et agréger (C03-C04) (Q5 à Q8)

Q5. `LIMIT 5` sans `ORDER BY` renvoie : a) les 5 plus petites lignes · b) **5 lignes arbitraires (l'ordre du stockage, non garanti)** · c) les 5 dernières insérées · d) une erreur.

Q6. `LIMIT 10 OFFSET 10` renvoie : a) les lignes 1 à 10 · b) **les lignes 11 à 20 (la 2^e page)** · c) les lignes 10 à 19 · d) 10 lignes prises au hasard après la 10^e.

Q7. Sur une colonne sans `NULL`, `COUNT(*)` et `COUNT(colonne)` : a) diffèrent toujours d'1 · b) **renvoient le même nombre** · c) renvoient 0 · d) `COUNT(colonne)` renvoie `NULL`.

Q8. `SUM(colonne)` sur une colonne 100 % `NULL` renvoie : a) 0 · b) **`NULL`** · c) une erreur · d) le nombre de lignes.

Bloc 3 — Résumer par groupe (C05) (Q9 à Q12)

Q9. « Les magasins avec plus de 9 900 ventes » s'écrit avec : a) `WHERE COUNT(*) > 9900` · b) **`HAVING COUNT(*) > 9900`** · c) `LIMIT 9900` · d) `ORDER BY COUNT(*) > 9900`.

Q10. Regrouper par un libellé doublé (ici, `rayon`) : a) produit une erreur · b) **fusionne silencieusement les groupes (5 rayons au lieu de 8 catégories)** · c) double les lignes · d) n'a aucun effet.

Q11. Le contrôle de cohérence d'un `GROUP BY` est : a) vérifier l'orthographe des alias · b) **le nombre de groupes + la somme des agrégats redonne le total global** · c) le temps d'exécution · d) le nombre de colonnes du `SELECT`.

Q12. `COUNT(DISTINCT id_client)` dans un `GROUP BY id_magasin` compte : a) les clients de tous les magasins · b) **les clients distincts de chaque magasin (une ligne par magasin)** · c) les ventes de chaque magasin · d) une erreur.

Bloc 4 — Catégoriser (C06) (Q13 à Q16)

Q13. Dans un `CASE WHEN`, les branches sont testées : a) toutes en même temps · b) **dans l'ordre écrit, la première qui passe gagne** · c) de la plus longue à la plus courte · d) dans un ordre aléatoire.

Q14. Sans `ELSE`, les lignes non couvertes par un `CASE` renvoient : a) 0 · b) une erreur · c) **`NULL`** · d) la valeur de la première branche.

Q15. `COALESCE(0, 'défaut')` renvoie : a) « défaut » · b) **0 — il ne regarde que les `NULL`, pas les zéros** · c) une erreur de type · d) `NULL`.

Q16. `NULLIF(5, 0)` renvoie : a) `NULL` · b) 0 · c) **5** · d) une erreur.

Bloc 5 — Joindre et composer (C07-C08) (Q17 à Q20)

Q17. Mettre la condition du côté droit d'un `LEFT JOIN` dans le `WHERE` : a) n'a aucun effet · b) **transforme le `LEFT JOIN` en `INNER JOIN` (les `NULL` du droit sont éliminés)** · c) double les lignes · d) produit une erreur de syntaxe.

Q18. `CROSS JOIN` entre 5 magasins et 5 modes de paiement produit : a) 10 lignes · b) **25 lignes (5×5 , produit cartésien)** · c) 5 lignes · d) 0 ligne.

Q19. `WHERE x IN` (sous-requête) signifie : a) « tous les » · b) « **au moins un** » · c) « aucun » · d) « exactement un ».

Q20. « Les clients qui ont acheté **tous** les produits X » s'écrit avec : a) `IN` · b) `COUNT(*)` comparé au total · c) **la double négation `NOT EXISTS (... NOT EXISTS (...))`** · d) `CROSS JOIN`.

C · Exercices chiffrés (35 exercices · 35 points · auto-corrigés)

Consigne. Sur `commercial.duckdb`, produire le chiffre de chaque exercice (une requête chacune). 1 point par réponse exacte (tolérance de 0,5 sur les arrondis). Les 35 requêtes de référence et leurs valeurs attendues sont auto-validées par `python3 tools/controle_exos_M07.py` — exécutez-le pour vérifier vos réponses ; il rapporte chaque valeur obtenue. Les valeurs attendues sont mesurées sur le socle livré, jamais déduites (cf. `chiffres_cites.json`, clés `m07_*`).

#	Exercice	Réponse attendue
E01	Nombre de ventes	50 008
E02	CA brut total (FCFA, arrondi)	7 908 259 731
E03	Produits actifs	362
E04	Clients d'Ouagadougou	512
E05	Ventes un dimanche	7 136
E06	Panier moyen, toutes ventes (FCFA, arrondi)	158 140
E07	Montant maximum (FCFA)	594 363
E08	Montant minimum (FCFA, arrondi)	763
E09	Top magasin par CA (id)	4
E10	CA du top magasin (FCFA)	1 596 813 264
E11	Ventes de 2025	24 920
E12	CA de 2026 (FCFA)	3 983 044 060
E13	Ventes en Espèces (mode 1)	20 175
E14	CA Carte Bancaire (mode 2, FCFA)	1 944 001 846
E15	Catégories (par id) avec plus de 6 500 ventes	4
E16	Ventes de la catégorie Decoration (id 7)	6 798
E17	Clients avec plafond > 100 000 FCFA	490
E18	Clients particuliers	850
E19	CA des clients entreprises (FCFA)	1 482 606 616
E20	Clients de Koudougou	247
E21	Ventes de 200 000 FCFA et plus	15 746
E22	Ventes de moins de 50 000 FCFA	12 753

#	Exercice	Réponse attendue
E23	Ventes du magasin 4 en 2026	4 997
E24	CA des produits inactifs (FCFA)	471 206 245
E25	CA du meilleur client (FCFA)	11 561 026
E26	Ventes des clients de Bobo-Dioulasso	18 364
E27	Ventes uniques, dedoublonnage complet (12 colonnes)	50 000
E28	Ventes uniques, proxy 5 colonnes (piège P1)	49 999
E29	Couples client × produit distincts (piège P5)	47 359
E30	Ventes Bobo × rayon Bricolage, 4 jointures (piège P4)	4 568
E31	Clients ayant acheté les 2 produits les plus chers (P6)	119
E32	Lignes de la vue <code>v_ca_mensuel_magasin</code>	120
E33	Plus gros mois de la vue (FCFA, arrondi)	78 965 529
E34	Ventes au-dessus du panier moyen	20 289
E35	Produits au-dessus du prix moyen de leur rayon	190

Lecture des E27/E28. E27 est le vrai dedoublonnage (toutes les colonnes métier) : 50 000. E28 est le proxy sur 5 colonnes : 49 999 — l'écart de 1 est la paire naturelle 41 405/41 757 (même client, produit, magasin, date, montant ; **mode et remise différents**). Un candidat qui explique l'écart gagne son point même s'il inverse les deux ; un candidat qui ne les explique pas et les inverse en perd un.

D · Étude de cas — « Le rapport de la direction commerciale » (75 min · /20, seuil 12)

Consigne. La direction commerciale demande un rapport de **une page** avec 4 sections, produit à partir de `commercial.duckdb`. Le candidat rend : (i) un fichier `rapport.sql` contenant les 6 requêtes du rapport (exécutées, sorties capturées), et (ii) la page de synthèse (texte). Barème :

Section	Contenu	Points
D1 · Indicateurs globaux	CA brut, CA sans retours (P2), panier moyen, nb de ventes, nb de clients actifs — 1 requête	/4
D2 · Par magasin	CA, nb de ventes, panier moyen par magasin (5 lignes) + détection du top — 1 requête	/4
D3 · Par catégorie	CA et nb de ventes par catégorie (id, pas rayon) (8 lignes) + le top — 1 requête	/4
D4 · Risques et pièges	les 3 défauts du socle chiffrés (8 doublons → 50 000 uniques ; 200 retours → CA propre 7 876 320 163 ; 15 dates inversées) + la nuance E27/E28 — 3 requêtes	/4
D5 · Qualité	cohérence croisée (les CA par magasin somment au CA total), alias lisibles, 1 page lisible, chaque chiffre justifié par sa requête	/4

Critères de passage de l'étude de cas (12/20) : - D1 sans le CA « gonflé » présenté comme le CA (P2 détecté) ; - D3 avec 8 lignes (pas 5 — le rayon n'est pas la clé) ; - D4 avec au moins 2 des 3 défauts chiffrés.

L'esprit de l'étude. Le rapport est réussi quand la direction peut le lire **sans avoir à vérifier** : chaque chiffre traceable par sa requête, chaque piège nommé, les totaux cohérents entre sections. C'est la compétence finale du module — tout le reste (syntaxe, clauses) n'est que le moyen.

Corrigés ligne à ligne

B · Quiz — réponses

Q	1	2	3	4	5	6	7	8	9	10
Réponse	b	b	b	b	b	b	b	b	b	b

Q	11	12	13	14	15	16	17	18	19	20
Réponse	b	b	b	c	b	c	b	b	b	c

Justifications en une ligne (bonus) : - Q4 : l'ordre de priorité SQL est `AND` avant `OR` (d'où les parenthèses recommandées). - Q8 : `NULL` — l'absence de valeur s'exprime par `NULL`, pas par 0 ; parade `COALESCE`. - Q15 : `COALESCE` ne regarde que les `NULL` ; un 0 est une valeur présente. - Q17 : dans le `WHERE`, `NULL > 0` est `NULL` (ni vrai ni faux) : les lignes orphelines du droit sont éliminées, ce qui transforme le `LEFT` en `INNER`. - Q20 : « tous les » = « aucun contre-exemple » = double négation ; `IN` dit « au moins un ».

C · Exercices — valeurs de référence (clés de traçabilité)

#	Valeur	Clé https://www.lavoiedesdonnees.com
E01	50 008	m07p_ventes_brutes
E02	7 908 259 731	m07p_q04_ca_total_brut
E03	362	m07p_q02_produits_actifs
E04	512	m07_clients_ouagadougou
E05	7 136	m07_ventes_dimanche
E06	158 140	m07_panier_moyen
E07	594 363	m07_top_montant_ttc
E08	763	m07_min_montant
E09	4	m07_top_magasin_id
E10	1 596 813 264	m07_ca_mag_4
E11	24 920	m07_ventes_2025
E12	3 983 044 060	m07_ca_2026
E13	20 175	m07_ventes_mode_1
E14	1 944 001 846	m07_ca_mode_2
E15	4	mesuré (HAVING > 6 500 : catégories 1, 4, 7, 8)
E16	6 798	m07_ventes_cat_7
E17	490	m07p_q18_plafond_sup_100k
E18	850	m07_clients_particuliers
E19	1 482 606 616	m07_ca_type_entreprise
E20	247	m07_clients_koudougou
E21	15 746	m07_tranche_sup_200k
E22	12 753	m07_tranche_inf_50k
E23	4 997	m07_ventes_mag4_2026
E24	471 206 245	m07_ca_produit_inactif

#	Valeur	Clé <code>chiffres_cites.json</code>
E25	11 561 026	<code>m07_top_client_ca</code>
E26	18 364	<code>m07_ventes_ville_bobodioulasso</code>
E27	50 000	<code>m07p_q19_nb_ventes_uniques</code>
E28	49 999	<code>m07p_q19_distinct_5col</code>
E29	47 359	<code>m07p_q23_couples_distincts</code>
E30	4 568	<code>m07p_q27_nb_ventes_bobo_bricolage</code>
E31	119	<code>m07_clients_tous_2_plus_chers</code>
E32	120	<code>m07_vue_mensuelle_lignes</code>
E33	78 965 529	<code>m07_vue_plus_gros_mois</code>
E34	20 289	<code>m07_ventes_sup_moyenne</code>
E35	190	<code>m07p_q26_produits_sup_moyenne_rayon</code>

D · Étude de cas — points de repère du corrigé

- **D1** : CA brut 7 908 259 731 ; CA sans retours 7 876 320 163 (200 retours, écart 31 939 568) ; panier moyen 158 140 (158 159 sans retours) ; 50 008 ventes ; 1 200 clients.
- **D2** : 5 lignes (magasin 4 en tête : 1 596 813 264 FCFA / 10 061 ventes / 158 713 de panier moyen ; magasin 3 en queue : 1 564 937 883 / 9 873 / 158 507).
- **D3** : 8 lignes par id (Décoration en tête : 1 236 006 465 FCFA / 6 798 ventes ; Bricolage-Outils en queue : 699 533 080 / 5 381). Si 5 lignes apparaissent, le candidat a groupé par rayon (le piège C05) : -2 sur D3.
- **D4** : 50 008 → 50 000 uniques (8 doublons ; proxy 5 colonnes : 49 999, paire 41 405 / 41 757) ; CA propre 7 876 320 163 (P2) ; 15 ventes avec `date_vente > date_limite_remise` (P3).
- **D5** : les CA des 5 magasins somment à 7 908 259 731 (avec retours) — incohérence = jointure qui double (P4) ; chaque chiffre de la page pointe sa requête dans `rapport.sql`.

Évaluation M08 — « Python pour l'analyse de données »

Module M08 · durée totale 3 h 10 · cinq épreuves · seuils : quiz 11/15, exercices 11/15, étude de cas 12/20, projet 13/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	10 min	non noté	non	vérifier qu'on a lu les huit chapitres avant de se tromper cher
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, cinq blocs, dont 5 « prédisiez la sortie »
C · Exercices « prédisiez la sortie »	30 min	/15 (auto-corrigé)	oui	quinze valeurs en pandas, auto-validés par <code>tools/controle_exos_M08.py</code>
D · Exercices à rendre + E4	45 min	/20	oui	cinq scripts courts (5 × /2) + « ce script tourne mais ses chiffres sont faux » (/10)
E · Étude de cas — « Le script du stagiaire »	75 min	/20, seuil 12	oui	diagnostiquer et corriger, en commentant chaque message d'erreur

Le score du module se lit ainsi : **B ≥ 11** et **C ≥ 11** et **E ≥ 12** valident le module ; le projet M08.P doit atteindre 13/20. Un candidat qui réussit B et échoue à E repasse uniquement E : savoir lire un `groupby` ne dispense pas de savoir lire une traceback qui dit que votre nombre est faux. Tout nombre de cette évaluation est mesuré sur le socle livré (03_exercices/dossier_M08/, 8 CSV, ré-export déterministe de la base M07 figée, empreinte dossier b9a8d973119342ec...) ; les corrections le citent avec leur clé `m08_*` / `m08p_*` dans `chiffres_cites.json`.

A · Questions de récupération (non notées)

À traiter sans document, en une ou deux phrases chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail.

1. Citez les **4 couches** de l'environnement Python (C01), et dites ce qu'isole un `venv` — et pourquoi le script du projet M08.P doit tourner « sur la machine du collègue ».
2. Pourquoi `0.1 + 0.2` ne vaut pas `0.3` en Python (C02), et quelle est la conséquence pour des montants en FCFA (l'ordre des opérations de l'addition) ?
3. Quand choisir `for`, quand choisir `while` (C03) ? À quoi servent `break` et `continue` ?
4. Liste, tuple, dictionnaire, ensemble : lequel pour chacune de ces questions métier — « combien de fois », « dans quel ordre », « quelle valeur pour cette clé », « y a-t-il un doublon ? » (C04) ?
5. On vous donne une traceback de 30 lignes : par où commencez-vous la lecture (C05) ? Et quel bloc d'un `try/except/finally` s'exécute **toujours** ?
6. Pourquoi `np.where` gagne-t-il contre une boucle `for` sur 50 000 valeurs (C06) ? Que vaut une cellule « pas de valeur » dans un tableau NumPy ?
7. `loc` et `iloc` : qui parle en **libellés**, qui parle en **positions** (C07) ? Que renvoie `describe()` sur une colonne de montants ?
8. Pourquoi somme-t-on en **centimes entiers** avant de diviser (C08) ? Et pourquoi `re-sample("M")` lève-t-il une erreur sous pandas 3.x, et quelle est l'écriture vivante ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

Bloc 1 — Environnement et types (C01—C02) (Q1 à Q3)

Q1. (prédisez la sortie) Que renvoie `print(0.1 + 0.2)` ? a) 0.3 · b) **0.30000000000000004** · c) `TypeError` · d) 0.29999999999999998 arrondi à 0.3.

Q2. Un environnement virtuel (`venv`) isole : a) le système d'exploitation · b) **les paquets d'un projet (site-packages) du reste de la machine** · c) le navigateur · d) le compte utilisateur.

Q3. `x = "15"` puis `x + 5` : a) renvoie 20 (conversion implicite) · b) **lève `TypeError: unsupported operand type(s) for +: 'str' and 'int'`** · c) renvoie "155" · d) renvoie `None`.

Bloc 2 — Contrôle du flux et conteneurs (C03—C04) (Q4 à Q6)

Q4. Dans une boucle `for`, `break` : a) saute une itération · b) **quitte la boucle entièrement** · c) relance la boucle · d) est une erreur de syntaxe.

Q5. (prédisez la sortie) Que renvoie `print([x for x in range(1, 10) if x % 3 == 0])` ? a) [1, 2, 3, 4, 5, 6, 7, 8, 9] · b) [0, 3, 6, 9] · c) **[3, 6, 9]** · d) 9.

Q6. Un **dictionnaire** sert à : a) garder un ordre fixe · b) **récupérer une valeur par clé** · c) supprimer les doublons · d) trier des nombres.

Bloc 3 — Organiser le code (C05) (Q7 à Q9)

Q7. Dans un `try/except/finally`, le bloc qui s'exécute **toujours** (erreur ou non) est : a) `try` · b) `except` · c) **`finally`** · d) aucun.

Q8. (prédisez la sortie)

```
def remise(prix, pct=10):
    return prix * (1 - pct / 100)

print(remise(1500))
```

a) 1500 · b) 135.0 · c) **1350.0** · d) `TypeError`.

Q9. Une **traceback** se lit : a) de bas en haut, en s'arrêtant au message final · b) **de haut en bas pour comprendre la chaîne d'appels, et la dernière ligne porte le nom de l'erreur et la cause** · c) au hasard · d) uniquement par le code d'erreur.

Bloc 4 — Vecteurisation et lecture (C06—C07) (Q10 à Q12)

Q10. `np.where(condition, a, b)` : a) lance une boucle interne · b) **construit un tableau en choisissant a ou b élément par élément, sans boucle Python** · c) filtre une colonne · d) trie un tableau.

Q11. `df.loc` et `df.iloc` : a) sont interchangeables · b) **`loc` sélectionne par libellés (index/noms), `iloc` par positions entières** · c) `loc` ne fonctionne que sur les entiers · d) `iloc` ne fonctionne que sur les chaînes.

Q12. (prédisez la sortie) Sur le socle du module, que renvoie `pd.read_csv("03_exercices/dossier_M08/vente.csv").shape` ? a) (50000, 13) · b) **(50008, 13)** · c) (50008, 12) · d) (50000, 12).

Bloc 5 — Agréger et livrer (C08) (Q13 à Q15)

Q13. Diviser chaque ligne en FCFA **avant** de sommer, au lieu de sommer en centimes puis diviser : a) ne change rien · b) **perd les centimes de chaque ligne (le plus gros mois du socle sort à 228 FCFA de son total exact)** · c) donne un résultat plus précis · d) lève une erreur.

Q14. (prédisez la sortie)

```
v = pd.read_csv("03_exercices/dossier_M08/vente.csv")
print(v.groupby("id_magasin")["id_vente"].count().tolist())
```

a) [9942, 9873, 10033, 10061, 10099] · b) **[10099, 10033, 9873, 10061, 9942]** · c) [1200, 1200, 1200, 1200, 1200] · d) une erreur (5 magasins, 1 200 clients).

Q15. `v.merge(obj, on="id_magasin", how="left")` avec `obj` **vide** (0 ligne) : a) renvoie 0 ligne · b) **renvoie 50 008 lignes avec toute la colonne `ca_objectif_ttc` en NaN — un signal à lire, pas un bug** · c) lève `KeyError` · d) renvoie 50 008 lignes complètes par duplication.

C · Exercices « prédisez la sortie » (15 exercices · 15 points · auto-corrigés)

Consigne. Sur le dossier `03_exercices/dossier_M08/` (les 8 CSV), produire le résultat de chaque bloc. 1 point par réponse exacte. Les 15 blocs de référence et leurs valeurs attendues sont auto-validés par `python3 tools/controle_exos_M08.py` — exécutez-le pour vérifier vos réponses ; il rapporte chaque valeur obtenue. Les valeurs attendues sont mesurées sur le socle livré, jamais déduites (cf. `chiffres_cites.json`, clés `m08_*` / `m08p_*`).

#	Bloc (code de référence)	Résultat attendu
C01	<code>len(v)</code>	50 008
C02	<code>int((v["montant_ttc"] * 100).round().astype("int64").sum()) // 100</code> (CA brut, centimes)	7 908 259 732
C03	<code>v.drop_duplicates(subset=[c for c in v.columns if c != "id_vente"]).shape[0]</code>	50 000
C04	<code>int(v["est_retour"].sum())</code>	208
C05	CA propre en centimes, retours exclus	7 876 320 164
C06	Panier moyen, arrondi (CA brut en centimes ÷ 50 008)	158 140
C07	CA du top magasin, en centimes	1 596 813 264
C08	<code>int((pd.to_datetime(v["date_vente"]).dt.dayofweek == 6).sum())</code>	7 136
C09	<code>v.merge(o, on="id_magasin", how="left").shape</code>	(50008, 16)
C10	Nombre de cellules NaN de <code>ca_objectif_ttc</code> après ce merge	50 008
C11	Nombre de groupes de la vue mensuelle (sans retours, magasin × année-mois)	120
C12	Plus gros mois de la vue, en centimes	78 965 529
C13	<code>int(v["date_vente"].str.startswith("2025").sum())</code>	24 920
C14	<code>int(v["date_vente"].str.startswith("2026").sum())</code>	25 088
C15	<code>int((v["montant_ttc"] > 100000).sum())</code>	28 065

Lecture des C02/C05. Le canon du module est l'**addition en centimes entiers** côté pandas : `C02 = m08p_ca_brut`, `C05 = m08p_ca_sans_retours`. Le côté SQL de la croisée (DuckDB, somme float) donne 7 908 259 731 et 7 876 320 163 — l'écart de 1 FCFA est la dérive mesurée et sourcée (`m08p_ecart_somme_float_*`), pas un piège d'énoncé : un candidat qui livre 7 908 259 731 **en le signalant** garde son point.

D · Exercices à rendre (5 scripts courts · 5 × /2) + E4 « ce script tourne mais ses chiffres sont faux » (/10)

D1 — Les deadlines impossibles (C03, C07) — /2

Filtrer les ventes dont la `date_limite_remise` tombe au **20 du mois** (les deadlines aberrantes du socle : la remise est fixée au 15). Produire, sur 3 lignes : le nombre de lignes en **brut**, le nombre après dédoublement (12 colonnes métier), et la **première** date de vente parmi ces lignes (triées par `id_vente`).

Sortie attendue :

```
18
15
2025-01-10
```

La 3^e ligne se lit « la plus ancienne vente concernée est du 10 janvier 2025 » — la date seule ne vaut rien sans sa question.

D2 — La fonction des centimes (C05) — /2

Écrire `en_centimes(montant: float) -> int` qui convertit un montant en FCFA en **centimes entiers** avec un arrondi juste (pas de troncature float). Deux tests obligatoires : `en_centimes(0.1 + 0.2)` doit valoir **30** (et non 29 — c'est le piège `0.1 + 0.2 = 0.30000000000000004`), et `en_centimes(15000.5)` doit valoir **1 500 050**. Appliquer la fonction à la colonne `montant_ttc` et imprimer la somme des centimes.

Sortie attendue (les 3 lignes) :

```
30
1500050
790825973220
```

La 3^e ligne : la somme des centimes du CA brut, avant le `// 100` — ce n'est pas exactement 100 fois le C02 : il reste 20 centimes (`m08p_ca_brut_centimes`) qui font que `790825973220 // 100 = 7908259732`.

D3 — Les 4 tranches de montants (C06) — /2

Classer les 50 008 montants en 4 tranches avec `np.where` (sans boucle) : **50 000 et moins, plus de 50 000 jusqu'à 100 000, plus de 100 000 jusqu'à 200 000, plus de 200 000** FCFA, et imprimer le nombre de ventes par tranche, dans cet ordre.

Sortie attendue :

0-50k	12753
50-100k	9190
100-200k	12319
200k+	15746

La somme des 4 effectifs redonne 50 008 — le contrôle total du C04, en tranches.

D4 — L'année 2025 du magasin 4 (C07) — /2

Filtrer les ventes du **magasin 4** en **2025**, **sans retours**, ajouter une colonne `mois`, et imprimer le CA mensuel en centimes (12 lignes, mois triés). C'est l'exercice guidé du chapitre C08 §10, à refaire de mémoire.

Sortie attendue :

01	64948883
02	76403254
03	61294111
04	62254879
05	63589011
06	61805710
07	64979650
08	65368579
09	75836578
10	68271344
11	64272077
12	66454383

Le plus fort mois est 02 (76 403 254 FCFA) ; la somme des 12 valeurs est le total 2025 du magasin 4 : 795 478 466 FCFA (`m08p_mag4_2025_total`).

D5 — Le classeur en deux feuilles (C08) — /2

Construire un classeur Excel avec **deux feuilles nommées** : `vue_mensuelle` (les 120 lignes de la vue M07 sans retours : `id_magasin`, `mois`, `ca_fcfa` en centimes) et `top_magasin` (5 lignes : `id_magasin`, `ca_fcfa`). Recharger le classeur avec `sheet_name=None` et imprimer la forme de chaque feuille.

Sortie attendue :

```
{'vue_mensuelle': (120, 3), 'top_magasin': (5, 2)}
```

Le rechargement par nom est la garantie : une feuille sans nom ne se retrouve pas.

E4 — « Ce script tourne mais ses chiffres sont faux » (/10)

Le stagiaire livre ce script. **Il s'exécute sans lever d'erreur**. Trois de ses chiffres sont faux — localisez-les, expliquez chacun en une phrase, et corrigez.

```

"""analyse_synthese.py (v0 du stagiaire) – résumé mensuel du mois écoulé.
Le script s'exécute sans lever d'erreur. Trois chiffres sont faux.
"""
import pandas as pd

D = "03_exercices/dossier_M08"
v = pd.read_csv(D + "/vente.csv")

# 1. Volume : combien de ventes ?
ventes = len(v)

# 2. CA propre (retours exclus), en FCFA
centimes = (v["montant_ttc"] * 100).round().astype("int64")
ca_propre = int(centimes.sum()) // 100

# 3. Panier moyen des « méga-ventes » (1 000 000 FCFA et plus)
mega = v[v["montant_ttc"] >= 1000000]
try:
    panier_mega = int(centimes[v["montant_ttc"] >= 1000000].sum()) // len(mega)
except ZeroDivisionError:
    panier_mega = 0

print("ventes =", ventes)
print("ca_propre =", ca_propre)
print("panier_mega =", panier_mega)

```

Sortie réelle (exécutée le 20/09/2026) :

```

ventes = 50008
ca_propre = 7908259732
panier_mega = 0

```

#	Ce qui est faux	Le bon chiffre	La faute (à expliquer)
1	<code>ventes = 50008</code> présenté comme le volume de ventes	50 000 (m08p_ventes_uniques)	doublons comptés : le socle porte 8 doublons exacts (12 colonnes métier) — <code>len(v)</code> compte les lignes brutes, pas les ventes
2	<code>ca_propre = 7908259732</code> présenté comme le CA propre	7 876 320 164 (m08p_ca_sans_retours)	retours inclus : le filtre <code>~v["est_retour"]</code> est absent — 208 retours à montant positif gonflent le CA de 31 939 568 FCFA (m08p_ecart_ca_retours) ; le chiffre affiché est le CA brut
3	<code>panier_mega = 0</code>	l'indicateur n'existe pas — aucune vente ≥ 1 000 000 FCFA (le montant max du socle est 594 363 FCFA, m08p_max_montant)	division par zéro masquée : <code>len(mega)</code> vaut 0, le <code>except ZeroDivisionError</code> renvoie silencieusement 0 — un « 0 » qui ressemble à un chiffre, alors que la question est vide. Remède : vérifier le dénominateur avant de diviser, et dire « aucune vente » au lieu d'écrire 0

Barème E4 : 3 × /3 (localisation + bonne explication de chaque faute) + **/1** pour les trois corrections effectivement réexécutées (les 3 sorties corrigées : `ventes = 50000`, `ca_propre = 7876320164`, `panier_mega : aucune vente ≥ 1 000 000 FCFA`).

L'esprit de E4. C'est le piège le plus rare et le plus cher de tous : un script **sans erreur** qui **produit**. Les 8 chapitres apprennent à lire les erreurs ; E4 apprend à se méfier des sorties qui sont trop propres.

E · Étude de cas — « Le script du stagiaire » (75 min · /20, seuil 12)

Consigne. Le stagiaire livre `ca_magasin4_2025.py`, ci-dessous, pour produire le CA du magasin 4 en 2025 (sans retours), mois par mois, + total de l'année + panier moyen. Le script **échoue à l'exécution** (trois messages d'erreur, à faire apparaître les uns après les autres en corrigeant au fil de l'eau) et **son total, quand il s'affiche, ne correspond pas** au total attendu. Le candidat rend le script corrigé + un paragraphe par erreur (ce qu'elle dit, pourquoi elle est là, ce qui a été changé). Barème :

Point	Contenu	Points
E1	<code>NameError</code> : identifier la variable inconnue et la cause (typo)	/4
E2	<code>KeyError</code> : identifier la colonne inexistante et la colonne juste	/4
E3	Le total qui ne correspond pas : trouver la ligne qui ne fait rien (l'affectation sur la vue) et expliquer pourquoi le filtre retours n'a pas eu d'effet	/6
E4	Les sorties corrigées : les 12 valeurs mensuelles, le total 795 478 466 FCFA, le panier 157 426 FCFA	/4
E5	Qualité : un paragraphe par erreur, chaque message d'erreur cité et commenté	/2

Le script livré :

```
"""ca_magasin4_2025.py (livré par le stagiaire)
Objectif : CA du magasin 4 en 2025, sans retours, mois par mois,
+ total de l'année + panier moyen.
"""
import pandas as pd

D = "03_exercices/dossier_M08"
v = pd.read_csv(D + "/vente.csv")

masque = (v2["id_magasin"] == 4) & (v2["date_vente"].str.startswith("2025"))
vue = v2[masque]
vue["est_retour"] = 0

vue["mois"] = vue["date_vente"].str[5:7]
ca_cents = (vue["montant_ttc"] * 100).round().astype("int64")
mensuel = ca_cents.groupby(vue["mois"]).sum() // 100
print("CA mensuel magasin 4 (2025)")
print(mensuel.to_string())

total_2025 = int(ca_cents.sum()) // 100
print("total_2025 =", total_2025)

panier = int(v["montant_ht"].sum()) // 100
print("panier =", panier)
```

Les trois exécutions, sorties réelles (exécutées le 20/09/2026)

1^e run — le premier mur :

```
Traceback (most recent call last):
  File "ca_magasin4_2025.py", line 10, in <module>
    masque = (v2["id_magasin"] == 4) & (v2["date_vente"].str.startswith("2025"))
              ^^
NameError: name 'v2' is not defined. Did you mean: 'v' ?
```

(pandas 2.2.3 ; sous 3.x le message est le même, avec ou sans la suggestion.)

2^e run — après correction du `NameError` uniquement : les chiffres s'affichent, puis le deuxième mur :

```

SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
    vue["est_retour"] = 0
SettingWithCopyWarning:
...
    vue["mois"] = vue["date_vente"].str[5:7]
CA mensuel magasin 4 (2025)
mois
01    65140834
02    76403254
03    61404369
04    62560349
05    63589011
06    62223329
07    64979650
08    65653426
09    75944225
10    68968410
11    64272077
12    66454383
total_2025 = 797593323
Traceback (most recent call last):
  File "ca_magasin4_2025.py", line 23, in <module>
    panier = int(v["montant_ht"].sum()) // 100
KeyError: 'montant_ht'

```

(Le `total_2025 = 797593323` qui s'affiche avant le `KeyError` est déjà faux : il inclut les 11 retours du magasin 4 en 2025. Le bon total est 795 478 466 — l'écart de 2 114 857 FCFA est le CA de ces retours. Les avertissements sont déjà le signal de la ligne qui ne fait rien.)

Run corrigé — les sorties attendues :

```

CA mensuel magasin 4 (2025, sans retours)
mois
01    64948883
02    76403254
03    61294111
04    62254879
05    63589011
06    61805710
07    64979650
08    65368579
09    75836578
10    68271344
11    64272077
12    66454383
total_2025 = 795478466
panier = 157426

```

Critères de passage de l'étude de cas (12/20) : E1 + E2 sans faute, E3 avec la **bonne** explication (l'affectation sur la vue ne modifie pas le parent — le filtre retours s'applique donc à la population non nettoyée), et E4 avec le total 795 478 466.

L'esprit de l'étude. Le `NameError` et le `KeyError` sont faciles — Python les donne. Le total qui ne correspond pas est le vrai exercice : rien ne crie, deux avertissements murmurent, et un chiffre d'écart de 2 114 857 FCFA attend dans la sortie. C'est la compétence du module : **un chiffre sans sa vérification n'est pas un chiffre, c'est une hypothèse.**

Corrigés ligne à ligne

B • Quiz — réponses

Q	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Réponse	b	b	b	b	c	b	c	c	b	b	b	b	b	b	b

Justifications en une ligne (bonus) : - Q1 : les décimales binaire ne représentent pas 0,1 exactement — `0.1 + 0.2` renvoie `0.30000000000000004` ; d'où les centimes entiers (C02, C08). - Q3 : Python ne convertit pas implicitement `str + int` — l'erreur est le message, le type est la cause. - Q8 : argument par défaut `pct=10` — `1500 * (1 - 0.1) = 1350.0` (`m08p_demo_prix`, `m08p_demo_prix_net`). - Q12 : le socle a 50 008 **lignes** (8 doublons inclus) et 13 colonnes — la forme est brute, pas dédoublonnée. - Q13 : diviser par ligne arrondit chaque ligne — les centimes perdus ne se retrouvent pas (le plus gros mois sort à 228 FCFA de son total). - Q15 : un `merge` gauche avec `table_vide` garde toutes les lignes de gauche, colonnes droites en `NaN` — c'est le signal « pas de données d'objectif », pas un bug.

C • Exercices — valeurs de référence (clés de traçabilité)

#	Valeur	Clé
C01	50 008	<code>m08p_ventes_lignes</code>
C02	7 908 259 732	<code>m08p_ca_brut</code>
C03	50 000	<code>m08p_ventes_uniques</code>
C04	208	<code>m08p_retours_brut</code>
C05	7 876 320 164	<code>m08p_ca_sans_retours</code>
C06	158 140	<code>m08p_panier_moyen</code>
C07	1 596 813 264	<code>m08p_ca_top_magasin</code>
C08	7 136	<code>m08p_ventes_dimanches</code>
C09	(50008, 16)	<code>m08p_ventes_lignes × m08p_merge_colonnes</code>
C10	50 008	<code>m08p_ventes_lignes</code>
C11	120	<code>m08p_vue_lignes</code>
C12	78 965 529	<code>m08p_plus_gros_mois_ca</code>
C13	24 920	<code>m08p_nb_ventes_2025</code>
C14	25 088	<code>m08p_nb_ventes_2026</code>
C15	28 065	<code>m08p_ventes_gros_100k</code>

D • Exercices rendus — sorties de référence

#	Sortie attendue	Clés
D1	18 / 15 / 2025-01-10	<code>m08p_deadlines_jour_20_brut</code> , <code>m08p_deadlines_jour_20</code>
D2	30 / 1 500 050 / 790 825 973 200	<code>m08p_demo_float_add</code> , <code>m08p_ca_brut × 100</code>
D3	12 753 / 9 190 / 12 319 / 15 746	<code>m08p_tranche_0_50k</code> , <code>m08p_tranche_50_100k</code> , <code>m08p_tranche_100_200k</code> , <code>m08p_tranche_200k</code>
D4	les 12 valeurs mensuelles ; total 795 478 466 ; mois fort 02 (76 403 254)	<code>m08p_mag4_2025_total</code> , <code>m08p_mag4_2025_mois_fort</code>
D5	<code>{'vue_mensuelle': (120, 3), 'top_magasin': (5, 2)}</code>	<code>m08p_vue_lignes</code> , <code>m08p_magasins_lignes</code>

E4 — Les trois corrections

```
# 1. Volume : dedoublonner avant de compter (12 colonnes métier)
ventes = v.drop_duplicates(subset=[c for c in v.columns if c != "id_vente"]).shape[0]

# 2. CA propre : exclure les retours AVANT de sommer
centimes = (v.loc[~v["est_retour"], "montant_ttc"] * 100).round().astype("int64")
ca_propre = int(centimes.sum()) // 100

# 3. Dénominateur vérifié avant la division – « aucune vente » au lieu de 0
mega = v[v["montant_ttc"] >= 1000000]
if len(mega) == 0:
    print("panier_mega : aucune vente >= 1000000 FCFA")
else:
    panier_mega = int(centimes[v["montant_ttc"] >= 1000000].sum()) // len(mega)
```

Sorties corrigées (exécutées le 20/09/2026) :

```
ventes = 50000
ca_propre = 7876320164
panier_mega : aucune vente >= 1000000 FCFA
```

E • Étude de cas — points de repère du corrigé

- **E1** : `v2` n'est jamais défini — la typo du `NameError` ; le message de pandas 2.2.3 le dit lui-même : `Did you mean: 'v' ?`. Correction : `v` partout (2 lignes).
- **E2** : `montant_ht` n'existe pas — la colonne s'appelle `montant_ttc` (13 colonnes, pas de HT dans le socle). Correction : `v["montant_ttc"]` — mais attention : le panier doit être calculé sur la **population propre** (magasin 4, 2025, sans retours), pas sur tout le socle.
- **E3** : `vue = v[masque]` renvoie une vue/copie — `vue["est_retour"] = 0` s'écrit sur la vue **sans rien modifier dans `v`** (les `SettingWithCopyWarning` du 2^e run sont le signal ; sous pandas 3.x, la même écriture est **silencieuse** — pire encore). Le filtre « sans retours » du script s'applique à `vue` **après** que le drapeau a été remis à zéro... ou plutôt pas du tout : le mensuel est construit sur **toutes** les lignes du magasin 4 en 2025, retours compris → total 797 593 323 au lieu de 795 478 466 (11 retours, 2 114 857 FCFA). Correction : `vue = vue[~vue["est_retour"]]` **avant** de compter.
- **E4** : les 12 valeurs du run corrigé, total 795 478 466, panier 157 426 (= 795 478 466 // 5 053 ventes propres, `m08p_mag4_2025_ventes`).
- **E5** : chaque paragraphe cite le message (`NameError`, `KeyError`, `SettingWithCopyWarning`) et dit **ce qui a été changé** — pas « j'ai corrigé », mais « la ligne 12 `vue["est_retour"] = 0` ne modifie pas le parent ; remplacée par `vue = vue[~vue["est_retour"]]` avant l'agrégat ».

Évaluation M09 — « Analyse exploratoire de données »

Module M09 · durée totale 3 h 05 · cinq épreuves · seuils : quiz 11/15, exercices 11/15, étude de cas 12/20, projet 13/20

Épreuve	Durée	Barème	Noté	Ce qui est attendu
A · Questions de récupération	10 min	non noté	non	vérifier qu'on a lu les six chapitres avant de se tromper
B · Quiz	25 min	/15	oui	quinze questions à réponse unique, cinq blocs, dont 5 « prédisiez la sortie »
C · Exercices « prédisiez la sortie »	30 min	/15 (auto-corrigé)	oui	quinze valeurs mesurées sur le socle, auto-validées par <code>tools/controle_exos_M09.py</code>
D · Exercices à rendre	40 min	/20	oui	cinq exercices courts (5 × /2) + « cette conclusion déborde » (/10)
E · Étude de cas — « les 5 questions du directeur »	60 min	/20, seuil 12	oui	poser les 5 questions les plus pertinentes, puis y répondre — la qualité de la question passe avant la réponse

Le score du module se lit ainsi : **B ≥ 11** et **C ≥ 11** et **E ≥ 12** valident le module ; le projet M09.P doit atteindre 13/20. L'épreuve E est **délibérément inversée** par rapport à M07/M08 : une **bonne question mal répondue** vaut plus qu'une mauvaise question bien répondue. Savoir exécuter un `groupby` ne dispense pas de savoir **quelle** question poser au fichier. Tout nombre de cette évaluation est mesuré sur le socle livré (03_exercices/dossier_M09/, quatre jeux, graine 45, empreinte dossier `b9a8d973119342ec...`) ; les corrections le citent avec leur clé `m09p_*` / `m09v_*` dans `chiffres_cites.json`.

A · Questions de récupération (non notées)

À traiter sans document, en une ou deux phrases chacune. Elles ne sont pas comptées ; elles servent à repérer ce qui n'est pas assimilé avant de perdre des points sur un détail.

1. Citez les **10 étapes** du protocole EDA (C01) et dites pourquoi une étape peut être **vide** mais jamais **sautée**.
2. Pourquoi `head(5)` peut-il **mentir** (C01) ? Que regarde-t-on à la place pour décider si un fichier est propre ?
3. Quelles sont les **3 familles de défauts** de l'audit (C02), et pourquoi l'**audit vide** est-il suspect plutôt que réconfortant ?
4. Que sépare une **tendance** d'une **saisonnalité** (C03) ? Comment mesure-t-on qu'un cycle est **structurel** et non du bruit ?
5. Nommez les **3 témoins** de la preuve d'une anomalie (C04), et dites ce qu'on fait d'un **soupçon** qu'on ne peut pas prouver.
6. Pourquoi une corrélation ne suffit-elle **jamais** à conclure une cause (C05) ? Nommez les **3 fausses causes** du chapitre.
7. Quels sont les **4 chiffres** qui accompagnent un coefficient de corrélation publié (C05) ?
8. Quelles sont les **5 erreurs d'exploration** d'un graphique (C06), et le contrôle de chacune ? Que porte la **note d'EDA** (C06) ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les réponses justifiées en une ligne rapportent un demi-point de bonus, plafonné à 15.

Bloc 1 — Le protocole et l'audit (C01-C02) (Q1 à Q3)

Q1. Une étape du protocole qui ne trouve rien : a) se saute · b) **se déclare vide, avec les contrôles faits pour l'affirmer** · c) se remplace par une autre · d) se marque « à faire plus tard ».

Q2. Sur `consultation.csv`, la colonne `motif` compte 23 valeurs vides : c'est un défaut de la famille : a) **manquants** · b) doublons · c) impossibles · d) aucun — un motif peut être vide dans la réalité.

Q3. Les 4 stocks négatifs du socle (jusqu'à -12) relèvent de la famille « impossible » parce que : a) leur valeur est rare · b) **la borne métier dit qu'un stock ne peut pas être négatif** · c) ils sont au-delà de l'intervalle interquartile · d) ils sont négatifs donc faux.

Bloc 2 — Tendances et périodes comparables (C03) (Q4 à Q6)

Q4. (prédisez la sortie) Le socle du centre de santé s'arrête au 2026-12-28 ; décembre 2026 compte 28 jours pour 31 en décembre 2025. La comparaison des **totaux** mensuels : a) prouve une baisse de 8 % · b) **n'est pas comparable — comparer à mois égal ou au flux par jour** · c) prouve une hausse · d) est valable, décembre est toujours incomplet.

Q5. Pour juger qu'une saison est structurelle, on compare : a) la moyenne de l'été à celle de l'hiver · b) **l'amplitude ENTRE positions à la variabilité DANS chaque position** · c) le maximum au minimum · d) la médiane à la moyenne.

Q6. Sur le fil rouge, la moyenne glissante 3 mois du CA mensuel : a) suit la série de près · b) **reste dans une bande étroite — le CA est plat** · c) montre une forte croissance · d) n'existe pas, faute de 36 mois de recul.

Bloc 3 — Anomalies et preuves (C04) (Q7 à Q9)

Q7. Un montant à 594 363 FCFA, répété 6 fois, se **garde** parce que : a) il est rare · b) **c'est un plafond exact (10 × 49 946.47 × 1.19) — une rareté légitime** · c) il est inférieur à la borne de Tukey · d) il est rond.

Q8. Une **rupture de stock** de 3 jours se traite : a) comme un défaut à corriger · b) comme un manquant à combler · c) **comme un signal métier à conserver et à dater** · d) comme un doublon.

Q9. Une anomalie qu'aucun des 3 témoins ne confirme : a) est exclue du fichier · b) est corrigée par la moyenne · c) **reste un soupçon : signalé, non traité** · d) est publiée comme un fait.

Bloc 4 — Relations, segments, groupes (C05) (Q10 à Q12)

Q10. (prédisez la sortie) La corrélation entre montant et taux de remise vaut **0.001**, celle entre montant et remise **en FCFA** vaut **0.552**. La bonne lecture est : a) les grosses ventes ont de grosses remises · b) **le 0.552 est un effet de scale (montant en FCFA), le taux ne montre aucun lien** · c) les deux disent la même chose · d) la remise cause le montant.

Q11. (prédisez la sortie) Sur le panier moyen par mode de paiement, l'écart entre le minimum (156 150 FCFA) et le maximum (159 959 FCFA) vaut **2.4 %**, pour des groupes de **2 552 à 20 175** ventes. On conclut : a) le mode 1 est le meilleur · b) **les modes ne différencient pas le panier : l'écart est du bruit de taille** · c) il faut exclure les petits groupes · d) l'écart est significatif.

Q12. Pour contrôler la variable tierce dans la relation absentéisme × résultats, on : a) retire les élèves extrêmes · b) **recalcule la corrélation par bande de niveau initial** · c) calcule une moyenne mobile · d) change de méthode de corrélation.

Bloc 5 — Graphiques et note d'EDA (C06) (Q13 à Q15)

Q13. (prédisez la sortie) La courbe du CA est tracée sur un axe de **310 à 350 M FCFA**. C'est : a) une faute, un axe part toujours de zéro · b) **licite si l'axe est déclaré (données de niveau) — fautif s'il est silencieux** · c) une fraude · d) une échelle logarithmique.

Q14. (prédisez la sortie) Le nuage montant × taux de remise porte `n = 50 008` dans son titre et une transparence 0.25. Ces deux mentions servent à : a) décorer · b) **empêcher l'échantillon fantôme et la dissimulation de points** · c) accélérer le rendu · d) remplacer la légende.

Q15. Une affirmation de la note d'EDA sans **limite** : a) est acceptable si le chiffre est juste · b) **déborde de ce que les données disent : on la recule ou on la retire** · c) se complète par « sous réserve » · d) est notée au bonus.

C · Exercices « prédisez la sortie » (15 valeurs · 15 points · auto-corrigé)

Consigne. Pour chaque bloc, écrivez la valeur que vous **prédisez** avant d'exécuter, puis exécutez et comparez. Une valeur juste = 1 point ; une valeur fausse **corrigée par l'exécution** = un demi-point (le but est le réflexe, pas le score). L'auto-correction est donnée par `tools/controle_exos_M09.py` (quinze valeurs mesurées, jamais déduites).

#	Le bloc (à exécuter sur <code>lossier_M09</code>)	Ce que vous prédisez
E01	<code>len(pd.read_csv("quincaillerie/vente.csv"))</code>	le nombre de lignes de ventes
E02	lignes après dédoublonnage sur les 12 colonnes métier	ventes distinctes
E03	<code>sum(montant_ttc)</code> en centimes entiers, retours exclus, divisé une fois	le CA propre (FCFA)
E04	<code>len(consultation.csv)</code> filtré sur 2025-07-01 → 2026-06-30	les lignes du variant 12 mois
E05	moyenne de consultations par jour sur ce variant	la fréquentation quotidienne
E06	<code>max</code> des totaux mensuels de ce variant	le pic mensuel (consultations)
E07	écart entre le pic et le creux mensuels du variant	l'amplitude mois
E08	moyenne des mois d'été (juillet-septembre) du variant	la fréquentation d'été
E09	écart-type des mois d'été du variant	la variabilité intra-été
E10	consultations du lundi sur le variant	le jour le plus chargé
E11	consultations du dimanche sur le variant	le jour le plus creux
E12	<code>note.csv</code> filtré sur les notes hors bornes (0 à 20)	les notes impossibles
E13	corrélation entre absences totales et moyenne des notes	le coefficient
E14	<code>fichier_inconnu.csv</code> : doublons sur <code>tx_ref</code>	les transactions en double
E15	succès normalisés du fichier projet, après dédoublonnage	le taux de succès (%)

D · Exercices à rendre (20 points)

D1 (2 pts). Écrire les **3 questions** que le fichier `fichier_inconnu.csv` **peut** répondre, et **2** qu'il **ne peut pas** répondre — en justifiant chaque refus par la variable manquante ou la période trop courte.

D2 (2 pts). Sur `consultation.csv`, écrire le contrôle qui prouve que la moyenne/jour (**25.74**) et la médiane/jour (**29.0**) ne racontent pas la même chose — et dire **laquelle** des deux on publie pour un pic de fréquentation, et pourquoi.

D3 (2 pts). Écrire la définition **exacte** (en une phrase) des deux défauts de `stock_medicament.csv`, puis la commande qui les compte : **7 doublons** de 5-tuple et **4 stocks négatifs**.

D4 (2 pts). Le fichier scolaire porte **6 notes hors bornes** (2×22.5 , 25.0, et trois négatives : -1.5, -0.5, -3.0) : écrire la décision **argumentée** (garder, corriger ou exclure) **et** la ligne de rapport qui la justifie.

D5 (2 pts). Écrire la phrase qui publie le **taux de réussite** du jeu scolaire (**58.7 %**) **avec ses 4 chiffres** : valeur, n, contrôle, limite.

D6 (10 pts) — « cette conclusion déborde ». On vous donne quatre conclusions publiées. Pour chacune : dites **ce qui déborde** (1 pt), **la limite manquante** (1 pt) et **la phrase réécrite** (0.5 pt) :

1. « Les remises **causent** la baisse du CA : la corrélation est de 0.552. »
2. « Le centre de santé est **saturé** : la fréquentation d'été est de 853 consultations/mois. »
3. « Les élèves qui s'absentent ont 1.37 point de moins : **l'absentéisme fait baisser les notes**. »
4. « Le mode de paiement 3 est le **favori** des clients : son panier moyen est le plus élevé des cinq. »

E · Étude de cas — « les 5 questions du directeur » (20 points · seuil 12)

Situation. Le directeur du centre de santé vous remet **un an de consultations, 12 mois glissants** (2025-07-01 → 2026-06-30) — un **variant** que vous n'avez pas traité dans les chapitres, extrait de `sante/consultation.csv` par un simple filtre de dates. Il ne vous donne **aucune question** : c'est vous qui les posez.

Consigne (délibérément inversée).

1. **Écrivez d'abord les 5 questions** les plus pertinentes **pour lui** (12 pts — 2.4 pts par question). Chaque question est notée sur : sa **pertinence métier** (elle intéresse la direction), sa **formulation falsifiable** (on peut y répondre par un chiffre, et une réponse fausse est imaginable), et le fait qu'elle soit **répondable par CE fichier** (variables et période disponibles).
2. **Répondez ensuite à vos 5 questions** (8 pts) : un chiffre par question, **avec sa source** (la sortie exécutée) — et pour **deux** d'entre elles, la **limite**.
3. **Le contrôle final** : une question que le fichier **ne peut pas** répondre, écrite avec la raison (elle rapporte 1 point si les 5 questions en cours n'ont pas déjà épuisé le barème).

Correction détaillée

A • Réponses de récupération

1. Les 10 étapes : inspecter, comprendre les variables, détecter les problèmes, premières statistiques, tendances, anomalies, relations, hypothèses, visualisations, conclusions provisoires. Une étape **vide** est déclarée avec ses contrôles ; une étape **sautée** est un défaut du rapport (C01 §5.2).
2. `head(5)` montre les premières lignes : les défauts (doublons, négatifs, dates futures) peuvent être **ailleurs** ; on regarde `shape`, `dtypes`, `isna().sum()` et les bornes métier.
3. Doubtons, manquants, impossibles. L'audit vide est **suspect** parce qu'un audit réel produit des contrôles : on vérifie le contrôle du contrôle (C02 §5.2).
4. La tendance est le **lit** (moyenne glissante), la saisonnalité est la variation **régulière et prévisible** ; un cycle est structurel quand l'amplitude **entre** positions domine la variabilité **dans** chaque position (C03 §5.2).
5. Position (iqr, Tukey), récurrence (hier + entrées - sorties), métier (bornes). Un soupçon se **signale** (« au-delà de la borne, non exploré ») et ne se traite pas (C04 §5.2).
6. Parce qu'une association n'est pas une cause ; les trois fausses causes sont la **variable tierce**, la **régression vers la moyenne** et le **sens inversé** (C05 §5.2).
7. Valeur, n, contrôle, limite — les quatre ensemble (C05 §5.1).
8. Axe non zéro silencieux, graphique sans question, couleur sans légende, échelle log non annoncée, nuage sans n. La note d'EDA porte, par affirmation, sa **source** et sa **limite** (C06 §5.2 et §5.5).

B • Corrigé du quiz (1 pt par question)

#	Rép.	La justification attendue
Q1	b	une étape se déclare vide avec ses contrôles ; le silence est le défaut que le correcteur cherche
Q2	a	23 motifs vides sur 18 818 consultations = famille « manquants » (<code>m09p_s_motifs_manquants</code>)
Q3	b	le négatif n'est pas « rare », il est impossible : la borne métier (<code>stock >= 0</code>) le dit seule
Q4	b	décembre 2026 est partiel (28 jours) : comparer à mois égal ou au flux par jour
Q5	b	amplitude entre positions (198) contre variabilité intra-position (48) : la saison est structurelle
Q6	b	la moyenne glissante 3 mois reste dans une bande étroite : plat, +1.6 % à mois égal
Q7	b	$10 \times 49\,946.47 \times 1.19 = 594\,363$: c'est le plafond , une rareté légitime (<code>m09p_q_montant_max</code>)
Q8	c	une rupture est un fait métier (offre coupée) : on la garde et on la date, on ne la corrige pas
Q9	c	sans les 3 témoins, c'est un soupçon : signalé, non traité (C04 §5.3)
Q10	b	le 0.552 vient du scale (montant en FCFA) ; en taux, la corrélation est 0.001
Q11	b	2.4 % d'écart pour des groupes de rapport 8 : bruit de taille, les n sont dans le graphique
Q12	b	on recalcule par bande de niveau initial (confond) : -0.145 / -0.085 / $+0.163$
Q13	b	sur des niveaux , l'axe non zéro est licite s'il est déclaré ; silencieux, il dramatise
Q14	b	sans le n, 50 008 points ou 200 se ressemblent ; la transparence qui masque doit être dite
Q15	b	une limite vide = une affirmation qui déborde : on la recule ou on la retire

C • Corrigé des quinze valeurs

Les valeurs attendues sont **mesurées** (elles ne sont pas déduites des chapitres) ; `tools/controle_exos_M09.py` les recalcule depuis le dossier livré et affiche, pour chacune, la valeur obtenue et le verdict. Les ordres de grandeur de référence : les ventes du fil rouge sont ~50 000

lignes (8 doublons → ~50 000 distinctes), le variant santé fait **9 450 lignes sur 365 jours** (moyenne 25.89/jour) avec un **pic mensuel de 882 consultations** (2025-07) et un **creux de 678** (2026-02), une moyenne d'été de **857.3** contre **770.7** en hiver pour un écart-type intra-été de **24.03**, le lundi à **1 899 consultations** contre **201** le dimanche ; le fichier projet porte **14** doublons de `tx_ref` et un taux de succès de **81.15 %** après normalisation. Chaque valeur est vérifiée par le contrôleur, clé `m09v_*` ou `m09p_*` à l'appui.

D · Corrigé des exercices à rendre

D1. Répondables (exemples) : volume et nombre de transactions par mois ; taux de succès par canal ; répartition in/out. Non répondables : la **cause des échecs** (aucune colonne de motif), la **localisation** (pas de zone ni de point de vente) — deux refus justifiés par une variable absente, pas par une opinion.

D2. `cons.groupby("date_consultation").size()` puis `mean()` (25.74) contre `median()` (29.0) : la moyenne est **tirée vers le bas** par les jours creux (dimanches à 201 consultations sur un an), la médiane décrit **un jour ordinaire**. Pour un pic de fréquentation, on publie la **médiane** des jours comme référence d'activité et le **maximum** comme dimensionnement (`m09p_s_moyenne_jour`, `m09p_s_mediane_jour`).

D3. Doublet de stock : « même médicament, même date, même mouvement, mêmes entrées/sorties et même stock de fin — la ligne est répétée » → `stk.duplicated().sum() = 7` ; stock négatif : « stock de fin inférieur à zéro, ce qu'aucun inventaire réel ne produit » → `(stk["stock_fin"] < 0).sum() = 4` (`m09p_s_doublons_stock`, `m09p_s_stocks_negatifs`).

D4. Décision : **exclure les six notes du calcul** (elles ne sont pas « très bonnes » ou « très faibles », elles sont **impossibles** : une note sur 20 ne vaut ni 22.5 ni -1.5), **sans** corriger par la moyenne (on inventerait une valeur) et **en le déclarant** : « 6 notes hors bornes (0 à 20) exclues du calcul des moyennes ; elles sont signalées à la source pour correction » (`m09p_sc_notes_hors_bornes`).

D5. « Le taux de réussite est de **58.7 %** (n = 400 élèves distincts, notes recalculées après exclusion des 6 notes hors bornes et rattachement des doublons d'élèves), à lire comme une **photographie d'une session** : aucun contrôle sur la difficulté des épreuves ni sur les élèves sortis en cours d'année » (`m09p_sc_taux_reussite`).

D6 — « cette conclusion déborde » :

1. Déborde : la causalité (« causent ») et l'usage du coefficient en FCFA. Limite manquante : le scale (0.001 en taux) et l'absence de lien démontré. Réécriture : « la remise **en FCFA** est corrélée au montant (0.552) mais le **taux** de remise ne l'est pas (0.001) : aucune relation entre politique de remise et taille du panier. »
2. Déborde : « saturé » est une **conclusion d'organisation** (personnel, lits, médicaments) que la fréquentation seule ne prouve pas. Limite manquante : la saturation dépend de la capacité, variable absente du fichier. Réécriture : « la fréquentation est **saisonnière** (été 853 contre 655 en hiver) ; la saturation ne peut pas être conclue : le fichier ne porte pas la capacité du centre. »
3. Déborde : le sens (« fait baisser ») et la tierce variable. Limite manquante : le contrôle par bande montre que le lien se **réduit** (-0.145 / -0.085 / +0.163) et le sens n'est pas départagé. Réécriture : « l'absentéisme et les résultats sont **associés** (-0.232) ; une part du lien vient du niveau initial, le sens reste indéterminé — association, pas causalité. »
4. Déborde : « favori » est un **choix des clients** que 2.4 % d'écart ne démontrent pas. Limite manquante : l'écart pour des groupes de 2 552 à 20 175 ventes (bruit de taille) et la part des **retours** dans le mode. Réécriture : « les cinq modes ont des paniers moyens **statistiquement indiscernables** (156 150 à 159 959 FCFA pour 2.4 % d'écart) : aucun mode n'est « favori » en valeur de panier. »

E · Corrigé de l'étude de cas — les 5 questions attendues

Les 5 questions les plus pertinentes (2.4 pts chacune ; le jury accepte toute question de même niveau, la liste suivante est la référence) :

1. « **Y a-t-il une saison, et de quelle ampleur ?** » — moyennes d'été (857.3 consultations/mois) contre hiver (770.7), pour un écart-type interne d'été de 24.03 : la saison est **structurale** (l'écart entre positions domine la variabilité interne). Réponse chiffrée : 857.3 contre 770.7 (`m09v_ete`, `m09v_hiver`) ; limite : 12 mois seulement, deux étés partiels, la cause (pluies, épidémie) est hors fichier.
2. « **Quel est le mois de pointe, et de combien dépasse-t-il le creux ?** » — pic 2025-07 à 882 consultations, creux 2026-02 à 678, soit **204** consultations d'écart (`m09v_pic_mois`, `m09v_creux_mois`, `m09v_chute_max`). C'est la question du **dimensionnement** : le stock et les effectifs se calent sur le pic, pas sur la moyenne.
3. « **Quels jours de la semaine portent la charge ?** » — lundi **1 899** consultations contre dimanche **201** sur 365 jours (`m09v_lundi`, `m09v_dimanche`) : le planning du personnel se déduit de cette question, pas de la moyenne (25.89/jour).
4. « **La fréquentation quotidienne est-elle homogène ou faite de journées extrêmes ?** » — la moyenne/jour (25.89) et la médiane/jour (29.0) ne disent pas la même chose : publier les deux, plus le maximum, sous peine de sous-dimensionner les pointes. Limite : le fichier ne porte pas les **fermetures** du centre (elles apparaîtraient comme des jours creux).
5. « **La rupture de stock de juillet tombe-t-elle au pic de fréquentation ?** » — oui pour M01 (3 jours, 2025-07-14 → 16), non pour M02 (5 jours en mars) : la seule rupture **qui compte** opérationnellement est celle qui coïncide avec le pic (`m09p_s_rupture_m01`, `m09p_s_rupture_la_plus_longue`).

La question que le fichier ne peut PAS répondre : « **pourquoi** les patients viennent-ils plus l'été ? » — le fichier porte la fréquentation, le motif (`paludisme` en tête : 2 417 lignes) et l'âge, mais **aucune** variable environnementale (pluies, températures, épidémie) ni de population de référence : la cause est **hors fichier** (1 pt si le barème des 5 questions n'est pas épuisé).

Comment l'épreuve E est notée, en une phrase. Une **bonne question mal répondue** vaut plus qu'une mauvaise question bien répondue : une question de dimensionnement (pic, charge hebdomadaire, rupture au pic) qui reçoit un chiffre approximatif mais **sourcé** rapporte plus qu'une question décorative parfaitement calculée — c'est l'inverse de M07/M08, délibérément, parce que la compétence du module est de **savoir ce qu'on demande** aux données avant de savoir le calculer.

Évaluation M10 — « Data visualization »

Module M10 · 30 h · niveau N3 · 7 chapitres. Total : 65 points — quiz **15** (seuil 14/20 §B.6, soit 10,5/15), « prédisiez la mesure » **10**, exercices à rendre **20**, étude de cas **20** (seuil **12**). Durée conseillée : 1 h 30 pour le quiz et les prédictions, 2 h pour les exercices, 1 h 30 pour l'étude de cas. **Matériel autorisé** : 01_socle_donnees/, 03_exercices/dossier_M10/, 01_socle_donnees/data/reference/chiffres_cites.json (clés m10_*, m10p_*, m10e_*) et le script tools/controle_refonte_M10.py. **Toute réponse chiffrée doit être mesurable** : un chiffre qui ne se trouve pas dans le socle ne rapporte pas de point.

Ce que cette évaluation vérifie. Que vous savez **relire** une figure : nommer son défaut, le **mesurer**, prévoir la conclusion fautive qu'un lecteur en tirerait, et produire la version qui rend cette conclusion impossible. Les questions ne demandent pas de dessiner : elles demandent de **dire de combien** — c'est la compétence du module.

A · Questions de récupération (non notées)

Répondez de mémoire, en une ligne chacune. Elles ne rapportent rien : elles disent ce qu'il faut relire.

1. Quel est l'**ordre perceptuel** des encodages, du plus précis au moins précis ?
2. Que signifie « **une couleur, un rôle** » ?
3. Quelle est la différence entre une **erreur** et une **décision** (C05) ?
4. Combien de parts un camembert peut-il porter avant de devenir illisible, selon le module ?
5. Quels sont les trois écrans d'une séquence (C06), dans l'ordre ?
6. Quels outils du module sont **exécutés**, et lequel est **cité** (C07) ?

B · Quiz (15 questions · 15 points)

Barème : 1 point par question, une seule bonne réponse. Les cinq questions marquées « **prédisiez l'effet** » (Q3, Q6, Q9, Q12, Q15) demandent **la conclusion fautive qu'un lecteur tirerait de la figure** — la réponse doit être la phrase du lecteur, pas le nom du défaut.

Bloc 1 — La perception et le choix du graphique (C01—C02) (Q1 à Q3)

Q1. L'ordre perceptuel du module, du plus précis au moins précis, est : a) position > longueur > angle > surface > couleur · b) couleur > surface > position > longueur · c) longueur > position > couleur > angle · d) tous les encodages se valent si la couleur est choisie correctement.

Q2. Un demandeur veut **classer 8 catégories**. Le graphique adapté est : a) un camembert à 8 parts · b) **des barres triées** · c) une carte de chaleur · d) un treemap.

Q3 — prédisiez l'effet. Un rapport présente un camembert de **8** parts dont deux voisines sont séparées de **0,05** point. La phrase que la direction prononcera est : a) « les parts sont égales, il n'y a pas de hiérarchie » · b) « **voici le classement des catégories : la seconde est devant la troisième** » — alors que 0,17° ne se distinguent pas · c) « il manque des données » · d) « le graphique est faux ».

Bloc 2 — Couleurs, contraste, accessibilité (C03) (Q4 à Q6)

Q4. Le contraste minimal du module pour un élément de graphique (barre, ligne) contre le fond blanc est : a) **3,0** · b) 1,5 · c) 4,5 · d) cela dépend du nombre de séries.

Q5. Environ quelle proportion d'hommes présente une vision des couleurs atypique (le chiffre du module, qui justifie la redondance couleur + forme + libellé) ? a) 0,5 % · b) **8 %** · c) 25 % · d) 50 %.

Q6 — prédisez l'effet. Sur le fil rouge, la paire de couleurs VERT-ROUGE passe de **73,4** à **35,7** de différence perçue en vision deutéranope (une différence confortable à l'œil typique). Le lecteur deutéranope, lui, conclura : a) « **les deux séries sont la même** » · b) « les deux séries sont opposées » · c) « il manque une légende » · d) rien : les couleurs ne changent pas la lecture.

Bloc 3 — Écrire un graphique (C04) (Q7 à Q9)

Q7. Un titre qui **affirme** est un titre qui : a) nomme le fichier source · b) **porte une mesure** (un chiffre, un écart, une période) · c) décrit le graphique (« évolution du CA ») · d) tient en trois mots.

Q8. Sur le fil rouge, la longueur moyenne des titres passe de **38,4** à **113,2** caractères entre la version « avant » et la version corrigée. Ce qui compte dans cette évolution, c'est : a) la longueur elle-même · b) **le passage du libellé à l'affirmation** — la longueur n'est qu'une conséquence · c) la lisibilité de la police · d) le nombre de mots.

Q9 — prédisez l'effet. Une courbe du CA est tracée sur un axe 300-350 M (**82,4 %** de hauteur occupée) pour une variation réelle de **5,4 %**. La phrase fautive probable est : a) « **l'activité est très volatile, elle décroche en fin de période** » · b) « le CA est stable » · c) « l'axe n'est pas à zéro » · d) « il manque les retours ».

Bloc 4 — Les 10 erreurs (C05) (Q10 à Q12)

Q10. Sur un cumul à 100 %, ce qui est masqué est : a) la composition · b) **le total de chaque période** · c) l'ordre des séries · d) rien : le cumul à 100 % est une convention neutre.

Q11. Laquelle de ces erreurs **aucune déclaration ne rend acceptable** ? a) un axe tronqué · b) une échelle logarithmique · c) **un effet 3D** · d) un camembert à 3 parts.

Q12 — prédisez l'effet. Un histogramme en échelle logarithmique **non annoncée** : la masse des petits montants (**77,0 %** des ventes) occupe **93,5 %** de la largeur, la queue (**1 044** ventes) **1,3 %**. La conclusion fautive du lecteur est : a) « **les ventes sont homogènes, la queue est marginale** » · b) « les ventes sont très concentrées » · c) « il y a un problème de données » · d) « le graphique est en log ».

Bloc 5 — La séquence et les outils (C06—C07) (Q13 à Q15)

Q13. Les trois écrans d'une séquence sont, dans l'ordre : a) décision, mécanisme, niveau · b) **niveau, mécanisme, décision** · c) mécanisme, niveau, décision · d) niveau, décision, mécanisme.

Q14. Dans la maquette du chapitre, la séquence de **3** écrans couvre **5** objections sur 5, quand le rapport en **1** écran de 12 chiffres en couvrirait : a) 1 · b) **2** · c) 4 · d) 5.

Q15 — prédisez l'effet. Le même graphique exporté en 96 dpi pour un rapport imprimé : les étiquettes de 6 pt deviennent des taches. La phrase du lecteur, à la réunion de validation, est : a) « **on ne lit rien, refaites l'export** » — la crédibilité de l'analyse est atteinte par un fichier · b) « le graphique est faux » · c) « il manque la source » · d) « l'échelle est logarithmique ».

C · « Prédisez la mesure » (10 valeurs · 10 points · auto-corrigé)

Consigne. Pour chaque bloc de code exécuté sur le fil rouge (quincaillerie, hors retours pour le CA), écrivez **ce que la console affiche**. Une valeur fausse = 0 ; une valeur juste arrondie comme demandé = 1 point. Aucun calcul à la main : lancez le code après avoir écrit votre prédiction.

```
import pandas as pd, numpy as np
v = pd.read_csv("03_exercices/dossier_M09/quincaillerie/vente.csv", parse_dates=["date_vente"])
p = pd.read_csv("03_exercices/dossier_M09/quincaillerie/produit.csv")
net = v[~v["est_retour"]]
ca = net.groupby(net["date_vente"].dt.to_period("M"))["montant_ttc"].sum()
ret = v.groupby(v["date_vente"])["est_retour"].sum()
ret = ret.groupby(ret.index.to_period("M")).sum().reindex(ca.index).fillna(0)
mg3 = ca.rolling(3).mean()
vp = net.merge(p[["id_produit", "id_categorie"]], on="id_produit", how="left")
cat = vp.groupby("id_categorie")["montant_ttc"].sum()
trim = net.groupby(net["date_vente"].dt.to_period("Q"))["montant_ttc"].sum()
parts = (net.assign(trim=net["date_vente"].dt.to_period("Q"))
         .pivot_table(index="trim", columns="id_mode", values="montant_ttc", aggfunc="sum"))
parts = parts.div(parts.sum(axis=1), axis=0) * 100
m = net["montant_ttc"]
```

#	Prédiction à écrire	La valeur attendue
1	<code>round((mg3.max()-mg3.min())/mg3.min()*100, 1)</code>	?
2	<code>round(100*(ca.max()-ca.min())/1e6/350, 1)</code>	?
3	<code>round(100*(ca.max()-ca.min())/1e6/50, 1)</code>	?
4	<code>round(float(ca.corr(ret)), 2)</code>	?
5	<code>round(float(np.diff(np.sort(cat/cat.sum()*100)).min())*3.6, 2)</code>	?
6	<code>round(100*(trim.max()-trim.min())/trim.min(), 1)</code>	?
7	<code>round(float((parts.max(axis=0)-parts.min(axis=0)).max()), 1)</code>	?
8	<code>round(100*(m < 250000).mean(), 1)</code> puis <code>round(np.log10(250000)/np.log10(m.max())*100, 1)</code>	? et ?
9	<code>int((v["montant_ttc"] > 500000).sum())</code>	?
10	inversions d'un tri alphabétique contre le tri par valeur, sur les 8 catégories	? sur 28 paires

D · Exercices à rendre (20 points)

Trois exercices argumentés. La consigne du module s'applique : **la justification pèse plus que le choix**. Chaque réponse suit le même gabarit : (1) la question servie, (2) le graphique choisi et son encodage, (3) la mesure qui appuie le choix, (4) le piège écarté.

E1 — « choisir le graphique, justifier par la question » (7 points). Cinq commandes réelles, cinq questions. Pour chacune : le graphique, l'encodage dominant (ordre perceptuel, C01), la mesure du socle qui l'illustre, et le piège.

- « Où en est le CA, mois par mois, sur deux ans ? »
- « Quelles catégories pèsent le plus, et de combien ? »
- « Les gros paniers pèsent-ils lourd dans le CA ? »
- « Le mix de paiement a-t-il bougé d'un trimestre à l'autre ? »
- « Où sont les tensions dans la distribution des montants ? »

E2 — « la palette de notre maison » (6 points). À partir des mesures de C03 (contrastes WCAG contre le blanc : bleu 8,66 · rouge 6,74 · vert 5,91 · orange 3,78 · gris 3,05 · jaune 1,43), écrivez la charte d'une page : la palette (3 à 5 couleurs), le seuil de contraste (**4,5** pour du texte, **3,0** pour un élément de graphique), la redondance obligatoire (couleur + forme + libellé), et la règle de rôle (« une couleur, un rôle »). Rendez la charte **plus** le test : pour chaque couleur, sa valeur de contraste et l'usage autorisé.

E3 — « la relecture de 60 secondes » (7 points). Prenez le rapport de la quincaillerie (`rapport_avant/rapport_avant.md`) et passez les 6 questions du §5.7 de C05 sur chacune des 5 affirmations. Rendez un tableau : affirmation du rapport · question de la check-list qui l'attrape · mesure qui la contredit · verdict (confirmée / à corriger / fausse).

E · Étude de cas — « ce graphique a déclenché une décision fausse » (20 points · seuil 12)

Le contexte. Avril 2026, comité de direction d'une enseigne de quincaillerie (5 magasins, **24** mois, **50 008** ventes). À l'ordre du jour : la réorganisation de l'équipe commerciale. Le service présente **une** courbe : « Évolution du chiffre d'affaires ». La courbe est tracée sur un axe **300-350 M FCFA**. Elle monte, plonge, remonte. Le service conclut : « **l'activité est très volatile ; il faut une équipe de renfort et un pilotage hebdomadaire** ». Le comité vote la réorganisation. **Trois mois plus tard**, le surcoût est de l'ordre de plusieurs dizaines de millions de FCFA et l'activité n'a pas bougé : le CA net est resté dans une bande étroite.

Votre travail. Reconstituer la chaîne complète : **défaut → lecture fausse → décision**, puis produire la version qui aurait empêché la décision.

- 1. Le défaut, mesuré (5 pts).** Nommez-le (C05) et donnez **trois** mesures : la hauteur occupée de l'axe (**82,4 %** en version tronquée contre **11,8 %** à zéro), l'amplitude apparente (**× 7,0**) et la variation réelle (**5,4 %** sur la moyenne glissante, contre **13,4 %** sur la série brute). Conclusion en une phrase : l'écart entre les deux lectures vient de **la fenêtre**, pas de la donnée.
- 2. La lecture fausse, écrite (4 pts).** Rédigez les **trois** phrases que la courbe donne à lire à un lecteur pressé : « la volatilité augmente », « il y a un décrochage », « la croissance est portée par le second semestre ». Pour chacune, la **mesure qui la contredit** (les mois, la moyenne glissante, le rapport entre les deux années).
- 3. La décision, chiffrée (4 pts).** Écrivez la décision telle qu'elle a été prise, puis chiffrez ce qu'elle coûte au regard de ce qu'elle visait : quelle variation réelle aurait justifié un dispositif de renfort (proposez un seuil et justifiez-le) ?
- 4. La version qui l'aurait empêchée (4 pts).** Proposez la figure corrigée : axe à zéro, titre qui **affirme** (avec le chiffre), annotation du chiffre clé, et **la déclaration** écrite dans la figure. Expliquez en trois lignes pourquoi la décision n'aurait pas été votée.
- 5. La généralisation (3 pts).** Le contrôle automatique du module (`tools/controle_refonte_M10.py`) aurait-il attrapé cette figure ? Répondez précisément : **quelle couche** (L1 à L4) vérifie quoi, et **quelle couche** ne suffit pas ici — un axe tronqué n'est pas un changement de **valeur**. Proposez le contrôle supplémentaire qui l'attrape (« comparer la fenêtre de l'axe à l'étendue des données »).

Critères de réussite. Sont exigés : les trois mesures du défaut (1), les trois phrases fausses avec leur contre-mesure (2), une décision écrite **et** chiffrée (3), une figure corrigée avec sa déclaration (4), et une réponse exacte sur les couches du contrôle (5). Le seuil est de **12/20**.

Correction détaillée

A · Réponses de récupération

1. **Position** > longueur > angle > surface > couleur — Cleveland et McGill 1984, répliqué en 2010 sur des milliers de participants.
2. Une couleur **un seul rôle** dans une figure et dans la charte : si l'orange signifie « remise » ici et « retours » là, le lecteur ne peut plus construire de contrat de lecture (C03).
3. Une **erreur** est un écart entre ce que la figure laisse croire et ce que les données disent ; une **décision** est cet écart **déclaré** (axe non nul annoncé, échelle log annoncée). Une erreur déclarée devient légitime ; la 3D et la couleur à tout faire sont les deux seules qu'aucune déclaration ne sauve (C05).
4. Jusqu'à **3** parts ; au-delà, l'écart minimal entre deux voisines tombe à **0,05** point, soit **0,17°**.
5. **Niveau** (« où en est-on ? »), **mécanisme** (« pourquoi ? »), **décision** (« que fait-on, pour combien, et comment le saura-t-on ? »).
6. **Exécutés** : matplotlib/seaborn (et Excel, écrit **puis relu**). **Cité** : Power BI (9 clics décrits, aucun fichier produit) — règle §1.5.

B · Corrigé du quiz (1 pt par question)

Q1 a — la hiérarchie du module, fondée sur les erreurs de jugement mesurées. **Q2 b** — des barres triées : la longueur bat l'angle pour un classement de **8** catégories. **Q3 b** — la phrase fausse attendue est un classement, alors que **0,17°** ne se distinguent pas ; le camembert reste lisible jusqu'à **3** parts. **Q4 a** — **3,0** pour un élément de graphique, **4,5** pour du texte ; le jaune du socle (1,43) échoue aux deux et ne s'utilise qu'en remplissage secondaire. **Q5 b** — environ **8 %** des hommes ; d'où la redondance couleur + forme + libellé. **Q6 a** — « les deux séries sont la même » : **35,7** de différence perçue reste au-dessus du seuil de **10**, mais la paire n'est plus fiable seule — la redondance est obligatoire. **Q7 b** — le titre qui affirme **porte une mesure** : sur le fil rouge, **0** sur 5 titres « avant » portaient une mesure, **5** sur 5 après. **Q8 b** — c'est l'affirmation qui compte ; **113,2** caractères mesurent une conséquence, pas une qualité. **Q9 a** — la volatilité est fabriquée par la **fenêtre** : **5,4 %** réels contre une amplitude apparente $\times 7,0$. **Q10 b** — le **total** de chaque période : les huit trimestres vont de **960 721 755** à **1 012 518 068** FCFA (**5,4 %**) quand la part d'un mode ne bouge que de **1,9** point. **Q11 c** — l'**effet 3D** : la profondeur ne porte aucune donnée et déforme les comparaisons. Un axe tronqué se déclare, une échelle log s'annonce, un camembert à **3** parts est lisible. **Q12 a** — « les ventes sont homogènes, la queue est marginale » : le log donne **93,5 %** de la largeur à la masse et réduit la queue (**1 044** ventes, **7,1 %** du CA) à **1,3 %**. **Q13 b** — niveau, mécanisme, décision. **Q14 b** — **2** sur 5 : le cadre unique de **12** chiffres couvrirait le niveau et le levier, rien d'autre. **Q15 a** — « on ne lit rien, refaites l'export » : le format ne change pas la donnée, mais **604 × 340** px ne suffisent pas à l'impression quand **1 889 × 1 062** px sont attendus.

C · Corrigé des dix prédictions

#	Valeur	Ce que la valeur sert à démontrer
1	5,4	la variation réelle du fil rouge (moyenne glissante)
2	11,8	la hauteur occupée sur un axe à zéro, en %
3	82,4	la hauteur occupée sur l'axe tronqué 300-350 M
4	+0,35	la corrélation affichée par le double axe
5	0,17	l'écart minimal entre deux parts, en degrés
6	5,4	la variation des totaux trimestriels, masquée par le cumul
7	1,9	l'amplitude d'une part de mode, en points — ce que le cumul, lui, montrait

#	Valeur	Ce que la valeur sert à démontrer
8	77,0 puis 93,5	la masse sous 250 000 FCFA, en % des ventes et en % de largeur en log
9	1 044	les ventes de plus de 500 000 FCFA (2,1 % des ventes, 7,1 % du CA)
10	21 sur 28	les paires classées à l'envers par un tri alphabétique (75 %)

D • Corrigé des exercices à rendre

E1 — les cinq questions.

- Évolution du CA** → courbe + moyenne glissante, axe à zéro annoncé ; mesure : **5,4** % de variation réelle, **307,6** à **348,8** M FCFA ; piège écarté : l'axe tronqué, qui porterait la hauteur occupée à **82,4** %.
- Classement des catégories** → barres triées, valeurs absolues ; mesure : de **697** à **1 231** M FCFA (**8** catégories) ; piège écarté : le camembert (angle), illisible au-delà de **3** parts.
- Poids des gros paniers** → barres ou histogramme avec zoom séparé sur la queue ; mesure : **1 044** ventes de plus de 500 000 FCFA = **2,1** % des ventes et **7,1** % du CA ; piège écarté : l'échelle logarithmique non annoncée, qui écrase la queue à **1,3** % de largeur.
- Mix de paiement** → barres empilées en **valeurs absolues** (ou lignes par mode) ; mesure : totaux trimestriels de **960 721 755** à **1 012 518 068** FCFA (**5,4** %) ; piège écarté : le cumul à 100 %, qui masque ces totaux.
- Distribution des montants** → histogramme linéaire **annoncé** et zoom sur la queue ; mesure : médiane **119 482** FCFA, maximum **594 363** (rapport **5**), **77,0** % des ventes sous 250 000 FCFA ; piège écarté : le log surprise.

E2 — la charte. Attendu : **3** à **5** couleurs, chacune avec son contraste (bleu **8,66**, rouge **6,74**, vert **5,91**, orange **3,78**, gris **3,05**), le seuil rappelé (**4,5** texte / **3,0** graphique), la redondance écrite (couleur + forme + libellé), et la règle de rôle. Le contraste du jaune (**1,43**) doit être signalé comme rédhitoire pour un élément porteur d'information.

E3 — la relecture. Attendu, pour les 5 affirmations du rapport :

Affirmation du rapport	Question qui l'attrape	Mesure qui la contredit	Verdict
« croissance continue »	l'échelle	5,4 % de variation réelle, +1,4 % sur un an	à corriger (l'axe tronqué fait le récit)
« les retours suivent le CA »	les liens	corrélation +0,35 sur 24 mois, retours de 4 à 16 par mois	fausse (le double axe crée le lien)
« catégories déséquilibrées, une pèse un quart »	la densité et l'ordre	8 parts, écart minimal 0,17° ; de 697 à 1 231 M FCFA	à corriger (barres triées, valeurs écrites)
« le mix est stable »	les liens	totaux variant de 5,4 %, part d'un mode bougeant de 1,9 point	à corriger (le cumul masque les totaux)
« les gros montants sont rares »	la question de l'échelle	1 044 ventes > 500 000 FCFA = 7,1 % du CA	confirmée, mais illisible en log

E • Corrigé de l'étude de cas — les cinq attendus

- Le défaut mesuré.** Axe tronqué (n° 1) : la fenêtre 300-350 M occupe **82,4** % de sa hauteur pour une étendue qui n'en occupe que **11,8** % sur un axe à zéro ; amplitude apparente **× 7,0** ; variation réelle **5,4** % (moyenne glissante) contre **13,4** % sur la série brute. Phrase clé : l'écart vient de **la fenêtre**, la donnée n'a pas bougé.
- Les trois phrases fausses.** (a) « la volatilité augmente » → contredite par la moyenne glissante, qui reste entre **320** et **337** M FCFA ; (b) « il y a un décrochage » → le point bas isolé est un mois creux de la série brute, pas une tendance ; (c) « la croissance est portée par le second semestre » → sur **24** mois, l'écart entre les deux années n'est que de **+1,4** %.

3. **La décision chiffrée.** Dispositif de renfort voté pour répondre à une volatilité **apparente × 7,0** : le coût de plusieurs dizaines de millions de FCFA est à comparer à ce qui l'aurait justifié — un seuil raisonnable, par exemple une variation de la moyenne glissante supérieure à **10 %** sur deux trimestres, n'est jamais atteint (**5,4 %** au maximum).
4. **La version qui l'aurait empêchée.** Axe à zéro, titre qui affirme (« Le CA est plat sur **24** mois : moyenne glissante entre **320** et **337** M FCFA »), annotation de la moyenne glissante, et déclaration écrite (« axe à zéro, non tronqué »). La courbe devenant franchement plate, la demande de renfort n'aurait pas tenu une question.
5. **Les couches du contrôle. L2** (totaux identiques) **ne suffit pas** : un axe tronqué ne change aucune valeur. **L4** (audit du code) l'attrape **si** le contrôle lit les `set_ylim` — c'est justement pourquoi le contrôle audite le **code** et non l'image. Il faut donc un contrôle supplémentaire : **comparer la fenêtre de l'axe à l'étendue des données** et exiger une **déclaration** dans le titre quand la fenêtre est tronquée. C'est la leçon du module : ce qui ne se mesure pas ne se corrige pas.

Évaluation M11 — « SQL avancé pour la BI »

Module M11 · 30 h · niveau N3 → N4 · 7 chapitres. Total : 70 points — quiz 20 (seuil 14/20 §B.6), exercices à rendre 20 (dont 2 de type E4, « correction d'erreurs »), auto-test SQL 10, étude de cas 20 (seuil 12). Durée conseillée : 1 h pour le quiz, 2 h pour les exercices, 45 min pour l'auto-test, 1 h 30 pour l'étude de cas. **Matériel autorisé** : 03_exercices/dossier_M11/ (socle et connexion), 01_socle_donnees/, 01_socle_donnees/data/reference/chiffres_cites.json (clés m11_*, m11p_*, m11e_*) et l'instrument tools/controle_sql_M11.py. **Toute réponse chiffrée doit être mesurable** : un chiffre qui ne se retrouve pas dans le socle ne rapporte pas de point.

Ce que cette évaluation vérifie. Que vous savez **interroger** un socle de vente et **défendre** vos chiffres. Les questions ne demandent pas d'écrire des requêtes savantes : elles demandent de **dire** quelle construction répond à une question, de **prédire** la sortie d'une requête, de **diagnostiquer** un total faux, et de nommer ce qu'on manipule. Deux exercices (E4 et E5) portent sur des **requêtes erronées** : la justification pèse plus que la correction.

A · Questions de récupération (non notées)

Répondez de mémoire, en une ligne chacune. Elles ne rapportent rien : elles disent ce qu'il faut relire.

1. Qu'est-ce qui distingue une fonction de fenêtre d'un `GROUP BY` sur le **nombre de lignes** rendues ?
2. Que font respectivement `RANK`, `DENSE_RANK` et `ROW_NUMBER` quand deux lignes sont à égalité ?
3. Qu'est-ce qu'un **cadre de fenêtre**, et que change `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` ?
4. Qu'est-ce qu'une **cohorte**, et sur quel événement se date-t-elle ?
5. Que fait `QUALIFY` que `WHERE` ne peut pas faire ?
6. Que change `UNNEST` au **grain** d'un jeu de lignes ?
7. Que lit un plan d'exécution, et dans quel sens se lit-il ?

B · Quiz (20 questions · 20 points · 1 point par question)

Quatre familles de 5 questions : trouver la requête, prédire le résultat, diagnostiquer, nommer.

Bloc 1 — Quelle requête pour quelle question (C01—C05) (Q1 à Q5)

Q1. La cellule demande « quelle part de l'année représente ce mois ? ». Quelle construction répond ? a) un `GROUP BY` sur l'année ; b) `SUM(ca_mois) OVER (PARTITION BY annee)` ; c) `LAG(ca_mois) OVER (ORDER BY mois)` ; d) `NTILE(12) OVER (ORDER BY mois)`.

Q2. « Le 3^e meilleur produit de chaque famille » : quelle construction donne la bonne réponse **et** son rang ? a) `GROUP BY famille HAVING COUNT(*) <= 3` ; b) `RANK() OVER (PARTITION BY famille ORDER BY ca DESC)` puis un filtre `<= 3`, avec `id_produit` en clé de départage ; c) `LIMIT 3` ; d) `NTILE(3) OVER (PARTITION BY famille)`.

Q3. « Les clients qui ont acheté en janvier et pas en février » : quelle construction ne se trompe pas de sens ? a) `INNER JOIN` sur le mois ; b) `EXCEPT` (ou une jointure externe gauche avec test de nullité) ; c) `UNION ALL` ; d) `GROUP BY id_client` sans filtre de mois.

Q4. Pour dater la **cohorte** d'un client, quel événement prend-on ? a) sa date d'inscription au référentiel ; b) la date de sa **première commande** dans les ventes ; c) sa dernière commande ; d) la première commande de l'année en cours.

Q5. « Le chiffre d'affaires par quartier, **avec ses sous-totaux**, dans un seul résultat » : a) trois requêtes à recoller ; b) `GROUP BY ROLLUP (quartier, famille)` ; c) `PIVOT` ; d) `UNION` des détails et des totaux.

Bloc 2 — Prédisez le résultat (Q6 à Q10)

Q6. Que rend `SELECT id_client, COUNT(*) FROM ventes WHERE est_retour = 0 GROUP BY 1 HAVING COUNT(*) = 8` ? a) 8 clients ; b) 3 316 clients ; c) 8 lignes de vente ; d) aucun client.

Q7. Une requête filtre d'abord le mois de **janvier 2023**, puis calcule `SUM(montant_ttc) OVER (PARTITION BY annee_de_vente)`. Quelle part du chiffre d'affaires de 2023 le rapport publiera-t-il pour ce mois ? a) **8,3 %**, la vraie part ; b) **100,0 %**, parce que le filtre a été appliqué **avant** la fenêtre ; c) **12,5 %** ; d) une erreur d'exécution.

Q8. Sur les clients classés par **nombre de tickets**, que valent `MAX(RANK())` et `MAX(DENSE_RANK())` ? a) **23 444** et **23 444** ; b) **23 444** et **21** ; c) **23 497** et **21 444** ; d) **21** et **21**.

Q9. Combien de lignes rend `GROUP BY ROLLUP (quartier, famille)` sur ce socle, et de quoi sont-elles faites ? a) **35** lignes, les couples ; b) **41** lignes : **35** couples, **5** sous-totaux de quartier et **1** total général ; c) **47** lignes ; d) **44** lignes, une par mois.

Q10. Une moyenne mobile de **12** mois est calculée au **3^e** mois du socle. Combien de lignes portent réellement dans sa fenêtre ? a) **12** ; b) **3** ; c) **0** ; d) **1**.

Bloc 3 — Diagnostic (Q11 à Q15)

Q11. Un suivi des objectifs joint les ventes au référentiel des objectifs avec un `INNER JOIN` : le magasin 4 perd deux mois. Combien manque-t-il de lignes, et combien de ventes sont-elles concernées ? a) **2** couples, **93 749 490** FCFA ; b) **44** couples, **6 685 260 000** FCFA ; c) **2** couples, **5 317 736 710** FCFA ; d) aucun, la jointure interne ne perd rien.

Q12. Un rapport compare 2026 (**8** mois) à 2025 (**12** mois) et annonce **-28,9 %**. Que faut-il publier ? a) **-28,9 %**, le chiffre est exact ; b) **+15,4 %**, la comparaison à **périmètre égal** ; c) **0 %** ; d) **+18,8 %**, la croissance de 2024.

Q13. Un rapport de rétention publie **13,2 %** (**432** clients revenus sur **3 276** actifs), alors qu'une lecture par cohorte donne **22,7 %** sur **22** clients. Quelle est la cause ? a) une panne du moteur ; b) deux **définitions** différentes de la rétention — **calendaire** contre **par cohorte** — et un dénominateur qui ne mesure pas la même population ; c) un filtre de date oublié ; d) une erreur d'arrondi.

Q14. Un « meilleur produit par catégorie » écrit sur les libellés bruts du référentiel rend **46** lignes ; sur les familles repliées, il en rend **21**. Que révèle l'écart ? a) une erreur de la requête ; b) un **défaut de normalisation** du référentiel : **16** libellés bruts pour **9** écritures et **7** familles réelles ; c) une différence de période ; d) une limite mémoire du moteur.

Q15. Un rapport qui fait `SELECT *` sur la vue coûte **581** ms, la même requête sur trois colonnes nommées **96** ms. Qu'annonce le plan **avant** toute mesure ? a) rien d'utile ; b) la lecture de **18** colonnes contre **3** : le coût est dans la projection ; c) qu'il faut un index ; d) que la vue est corrompue.

Bloc 4 — Vocabulaire (Q16 à Q20)

Q16. Qu'appelle-t-on le **grain** d'un résultat ? a) sa taille en octets ; b) ce que représente **une ligne** (un mois, un client, un ticket, un couple quartier-famille) ; c) le nombre de colonnes ; d) la clé de tri.

Q17. Qu'appelle-t-on une **clé de départage**, et que se passe-t-il sans elle ? a) la clé primaire de la table ; b) la seconde colonne de tri qui rend l'ordre **déterministe** : sans elle, un classement peut changer de réponse entre deux exécutions ; c) un index ; d) un filtre.

Q18. Que fait **QUALIFY** ? a) il filtre **avant** le calcul de la fenêtre ; b) il filtre **après** le calcul de la fenêtre, ce que **WHERE** ne peut pas faire ; c) il trie ; d) il crée une vue.

Q19. Pourquoi un rapport coûte-t-il **142** ms sur la **vue** et **1** ms sur la table équivalente ? a) la table est plus petite ; b) une **vue** ne stocke rien : elle **relit** sa source à chaque appel ; c) la vue est mal écrite ; d) les deux requêtes ne sont pas les mêmes.

Q20. Que doit contenir la **note de méthode** d'un rapport de performance ? a) seulement les temps mesurés ; b) les temps, la version du moteur, le protocole, la date, et ce qui **n'a pas** été exécuté ; c) une capture d'écran ; d) la liste des index de la base.

C · Exercices à rendre (20 points · 5 exercices, dont 2 de type E4)

E1 — Le top 3 par famille, deux fois (niveau 3 · 3 points)

Produisez les **3** meilleurs produits de chaque famille du catalogue, une première fois sur les libellés tels quels, une seconde fois après repliement (casse, espaces, accents, pluriel). Donnez le **nombre de lignes** rendu par chaque version, et expliquez l'écart en deux phrases.

E2 — Les deux périmètres de la concentration (niveau 3 · 3 points)

Calculez la part du chiffre d'affaires du dixième supérieur des clients, **avec** puis **sans** le client non identifié. Écrivez la phrase qui doit accompagner la publication de chacune des deux valeurs.

E3 — Le suivi des objectifs, trous compris (niveau 3 · 4 points)

Produisez le suivi mensuel 2024 du magasin 4 : réalisé, objectif, taux de réalisation, et une colonne **etat** à trois valeurs (complet, objectif manquant, vente manquante). Le rapport doit faire apparaître les mois sans objectif, avec le montant de ventes concerné — **aucune ligne ne doit disparaître**.

E4 — « Ce rapport annonce un chiffre d'affaires net » (type E4 · niveau 4 · 5 points)

Voici la requête telle qu'elle a été livrée :

```
SELECT COUNT(*) AS lignes, ROUND(SUM(montant_ttc)) AS ca_net
FROM ventes;
```

Elle rend **240 000** lignes et **15 419 985 157** FCFA, publiés comme « chiffre d'affaires net ».

1. **Nommez l'erreur** en une phrase, sans écrire de requête.
2. **Corrigez** la requête et donnez les deux valeurs attendues.
3. **Mesurez l'écart** en FCFA et en pourcentage, et dites dans quel sens il joue.
4. Dites ce que ce total ferait à un rapport de suivi des objectifs, dont l'objectif cumulé est de **6 685 260 000** FCFA.
5. Écrivez la ligne d'en-tête qui aurait empêché l'erreur.

E5 — « Ce classement change de réponse à chaque exécution » (type E4 · niveau 4 · 5 points)

Voici la requête telle qu'elle a été livrée :

```
SELECT id_client, NTILE(3) OVER (ORDER BY nb_tickets) AS segment
FROM (SELECT id_client, COUNT(*) AS nb_tickets FROM ventes GROUP BY 1) t;
```

Relancée **15** fois le **24/09/2026**, elle a compté **2 718, 2 720, 2 728** puis **2 738** clients dans son segment principal — sans qu'aucune donnée ne change.

1. **Nommez la cause**, en citant le fait du socle qui la rend possible.
2. **Corrigez** la requête, et donnez la taille des trois tiers une fois l'ordre fixé.
3. Expliquez pourquoi ce défaut est **pire** qu'une requête lente.
4. Écrivez le **test de non-régression** qui aurait détecté le problème (valeur attendue, requête rejouée, verdict).

D · Auto-test SQL (10 points · 18 requêtes · auto-corrigé)

L'instrument `tools/controle_sql_M11.py` rejoue **18** requêtes et confronte chaque résultat aux valeurs publiées du module. Exécutez-le et **rendez sa sortie** (les **18** lignes de verdict et la ligne de bilan).

Barème : **18** contrôles au vert = **10** points ; **14** à **17** = **7** points ; **10** à **13** = **4** points ; moins de **10** = **0** point, avec un mot d'explication obligatoire sur les écarts.

Attention. Ne modifiez **aucune** requête de l'instrument : un contrôle n'est un contrôle que s'il rejoue la requête **publiée**. Si un écart apparaît, cherchez d'abord ce qui a changé dans le socle ou dans le fichier publié — et notez-le, c'est la réponse attendue.

E · Étude de cas — « 10 % des clients font-ils 60 % du CA ? » (20 points · seuil 12)

La commande

La direction commerciale a entendu, en séminaire, que « **10 %** des clients font **60 %** du chiffre d'affaires ». Elle vous demande de **vérifier la formule sur son propre fichier** et de lui dire, en une page, si elle peut la reprendre à son compte.

Les quatre productions attendues

1. **Le chiffre vrai** — la part du chiffre d'affaires portée par le dixième supérieur des clients, avec le **périmètre** annoncé (ventes nettes, période, magasins) et la **définition du décile** retenue.
2. **Le classement des clients, avant et après retrait du client non identifié** — ce que change le retrait, et le mot qui l'explique.
3. **La conclusion tenable** — trois lignes pour la direction, chiffres compris, sans formule recopiée sans vérification.
4. **La question suivante à poser** — celle que votre rapport ne tranche pas, et que la direction devra poser à son tour.

Le résultat attendu (à retrouver, pas à recopier)

Sur ce socle, la réponse est **non**, et elle est stable : le dixième supérieur pèse **38,1 %** du chiffre d'affaires net, **24,2 %** une fois le client **0** écarté, **51,0 %** pour le top **20 %**, **30,0 %** pour le top **5 %** et **21,6 %** pour le top **1 %**.

Attention. Le résultat dépend de la **définition** du dixième : `NTILE(100) <= 1` retient **2 350** clients et donne **38,1 %** ; un `ROW_NUMBER()` sur **10 %** du fichier en retient **2 349** et donne **38,0 %**. Un rapport qui publie un pourcentage sans publier sa définition de décile produit un chiffre que personne ne peut reproduire.

Barème de l'étude de cas

Critère	Points
1. Le chiffre vrai : périmètre annoncé, définition du décile écrite, valeur exacte	5
2. Le classement avant / après retrait du client 0 , et le mot qui l'explique	5
3. La conclusion tenable : trois lignes, utilisables, sans surinterprétation	5
4. La question suivante, formulée comme une question à laquelle on peut répondre	5
Total	20

Correction détaillée

A • Réponses de récupération

1. Une fenêtre **conserve** les lignes (elle ajoute des colonnes) ; `GROUP BY` **réduit** la table aux groupes.
2. `RANK` laisse des trous après une égalité, `DENSE_RANK` n'en laisse pas, `ROW_NUMBER` attribue des rangs uniques — d'où la nécessité d'une clé de départage.
3. Le cadre est la portion de la partition prise en compte autour de la ligne courante ; `2 PRECEDING` retient trois lignes et ne s'étend pas au-delà des lignes existantes.
4. Un groupe de clients partagé par un événement daté — ici la **première commande**, et non la date d'inscription du référentiel.
5. `QUALIFY` filtre **après** le calcul de la fenêtre : il permet de garder le rang 1 par groupe dans la même requête.
6. `UNNEST` passe d'une ligne « ticket » à une ligne « article de ticket » : le grain change, et `COUNT(*)` ne compte plus la même chose.
7. Le plan décrit les opérateurs et les colonnes lues avant l'exécution ; il se lit **de bas en haut**.

B • Corrigé du quiz (1 point par question)

Question	Réponse	Pourquoi
Q1	b	La part se calcule par une fenêtre partitionnée par année (C01).
Q2	b	<code>RANK</code> partitionné, filtré à 3 , avec <code>id_produit</code> en départage (C02).
Q3	b	<code>EXCEPT</code> (ou jointure externe gauche + nullité) : le <code>INNER JOIN</code> répond à une autre question (C05).
Q4	b	La cohorte se date sur la première commande (C04).
Q5	b	<code>ROLLUP</code> produit les 41 lignes en une requête (C05).

Question	Réponse	Pourquoi
Q6	b	3 316 clients partagent exactement 8 tickets (14,1 %).
Q7	b	Le filtre avant la fenêtre ne laisse qu'une ligne : 100,0 % au lieu de 8,3 % (C01).
Q8	b	23 444 en RANK contre 21 en DENSE_RANK (C02).
Q9	b	35 couples + 5 sous-totaux + 1 total général (C05).
Q10	b	Au 3 ^e mois, la fenêtre ne porte que sur 3 lignes (C03).
Q11	a	2 couples manquants (magasin 4, 2024-02 et 2024-03) pour 93 749 490 FCFA (C03).
Q12	b	À périmètre égal : +15,4 % (C03).
Q13	b	Rétention calendaire contre rétention par cohorte : deux définitions, deux populations (C04).
Q14	b	16 libellés bruts, 9 écritures, 7 familles : 46 lignes contre 21 (C05).
Q15	b	Le plan annonce 18 colonnes contre 3 : le coût se lit avant l'exécution (C06).
Q16	b	Le grain est ce que représente une ligne (C01, C05).
Q17	b	La clé de départage rend l'ordre déterministe ; sans elle, le rapport change (C02, C07).
Q18	b	QUALIFY filtre après la fenêtre (C05).
Q19	b	Une vue relit sa source : 142 ms contre 1 ms (C06).
Q20	b	Temps, moteur, protocole, date, et ce qui n'a pas été exécuté (C06, C07).

C · Corrigé des exercices à rendre

E1. Les libellés bruts rendent **46** lignes pour **16** catégories ; après repliement des graphies, le même top 3 rend **21** lignes pour **7** familles. L'écart est la trace exacte du défaut de normalisation du référentiel, pas un problème de requête.

E2. Avec le client **0** : **38,1** % ; sans lui : **24,2** %. Les deux phrases attendues : « le dixième supérieur des **2 350** clients pèse **38,1** % du chiffre d'affaires net » et « hors client non identifié, ce dixième pèse **24,2** % — l'écart est porté par un seul identifiant, non rattaché à un client du référentiel ».

E3. Structure FULL OUTER JOIN avec CASE à trois états ; les **2** lignes sans objectif portent **42 514 372** et **51 235 118** FCFA, soit **93 749 490** FCFA de réalisé sans référence. Le taux de réalisation y vaut NULL — ni **0**, ni **100** %.

E4 — les cinq attendus. 1. La requête ne filtre pas les retours : elle compte les **2 809** lignes de retour dans un total annoncé « net ». 2. Correction : WHERE est_retour = 0 → **237 191** lignes et **15 595 154 955** FCFA. 3. Écart : **-175 169 798** FCFA, soit **-1,1** % — le total faux est **plus bas** que le vrai, puisque les retours sont négatifs, alors que le nombre de lignes est **plus haut** de **1,2** % (**240 000** comptées contre **237 191**) : l'erreur se lit à l'envers selon l'indicateur. 4. Face à un objectif cumulé de **6 685 260 000** FCFA, le total faux afficherait **230,7** % au lieu de **233,3** % : l'erreur reste invisible dans le vert de l'indicateur. 5. En-tête : « Hypothèses : ventes **nettes** de retours (est_retour = 0) ; les **2 809** lignes de retour sont exclues, jamais sommées. »

E5 — les quatre attendus. 1. L'ORDER BY n'a **pas** de clé de départage : **3 316** clients partagent exactement **8** tickets (**14,1** % du fichier), et le moteur est libre de placer les ex æquo comme il veut. 2. Correction : ORDER BY nb_tickets, id_client → les trois tiers comptent **7 833**, **7 832** et **7 832** clients, stables d'une exécution à l'autre. 3. Une requête lente se corrige et se mesure ; une requête **instable** publie des chiffres contradictoires d'un jour à l'autre, et le rapport perd sa crédibilité avant qu'on ait trouvé pourquoi. 4. Le test attendu rejoue **à l'identique** la requête corrigée et compare la taille du premier tiers à **7 833** : tout autre résultat fait échouer le contrôle et sort en code non nul.

D · Corrigé de l'auto-test

La sortie attendue comporte **18** lignes de verdict au vert, du comptage des lignes (**240 000, 237 191** hors retours) au ratio de saison (**1,65**), et se termine par « **18** controles sur 18 : OK — le socle et le fichier publie disent la meme chose. »

E · Corrigé de l'étude de cas — les quatre attendus

1. **Le chiffre vrai.** **38,1 %** du chiffre d'affaires net, périmètre : ventes nettes (retours exclus), 01/01/2023 au 31/08/2026, **5** magasins, client **0 inclus** ; définition du décile : `NTILE(100) <= 1`, soit **2 350** clients (l'autre définition usuelle, un `ROW_NUMBER` sur **10 %** du fichier, retient **2 349** clients et donne **38,0 %**).
2. **Le classement, avant et après.** Le client **0** pèse **2 852 612 447** FCFA — **18,3 %** du chiffre d'affaires — sur **42 658** lignes et **38 984** tickets, sans exister dans le référentiel. Il occupe donc toujours la première place d'un classement par montant : une fois écarté, le dixième supérieur tombe à **24,2 %**, et le classement des têtes change intégralement. Le mot à employer est **périmètre** : retirer le client **0** retire **18,3 %** du chiffre d'affaires, c'est un choix, et il s'écrit.
3. **La conclusion tenable.** Trois lignes, du type : « sur notre fichier, les **10 %** de clients les plus gros pèsent **38 %** du chiffre d'affaires, et **24 %** une fois le client non identifié écarté — la formule des **60 %** ne s'applique pas ici ; la concentration est réelle mais plus faible, et le top **20 %** pèse **51,0 %**. »
4. **La question suivante.** « Quelle part du chiffre d'affaires de chaque **cohorte** est portée par des clients non identifiés ? » Tant que le client **0** représente **18,3 %** du chiffre d'affaires, toute lecture de la rétention se fait sur un cinquième du total qu'on ne sait pas attribuer — c'est la question qui manque aux **12** rapports du projet, et elle se pose avec les outils du chapitre C04.

Barème global et seuils

Partie	Points	Seuil
B · Quiz (20 questions)	20	14/20 (§B.6)
C · Exercices à rendre (5 , dont 2 E4)	20	—
D · Auto-test SQL (18 requêtes)	10	—
E · Étude de cas	20	12/20 (§B.6)
Total	70	49/70 pour la validation du module

Épreuve de palier. Cette évaluation prépare l'épreuve **P4** de la phase 4 (§B.6) : les quatre compétences qu'elle mesure — écrire une requête qui répond à une question posée, prédire ce qu'une fenêtre va changer, diagnostiquer un total faux, publier un taux avec son effectif et sa définition — sont celles que l'épreuve de palier reprend, sur un socle différent.

Évaluation M12 — « Fondamentaux de la Business Intelligence »

6 chapitres · 30 h · niveau 3 · 70 points au total · durée conseillée : 2 h 30. Barème d'ensemble : quiz 15 questions (20 points, seuil 11 sur 15) ; deux exercices de rédaction de KPI (20 points) ; étude de cas (30 points, seuil 18) — soit 70 points, seuil de réussite 45.

Ce que l'évaluation vérifie. Que vous savez **choisir** un indicateur, **écrire** sa définition, **poser** son seuil, **trouver** son contre-KPI et **dire** ce qui se passe quand il passe au rouge. Les calculs sont courts ; les définitions et les décisions sont l'essentiel.

A · Questions de récupération (non notées)

1. Quels sont les **4** régimes de question, et lequel déclenche une action ?
2. Que signifie le **grain** d'une table, et comment le prouve-t-on ?
3. Quels sont les **7** champs d'une carte de définition ?
4. Dans quel ordre montent les **7** étages de la chaîne BI ?
5. Qu'est-ce qu'un contre-KPI ?

B · Quiz (15 questions · 20 points)

Les questions 1 à 10 valent 1 point ; les questions 11 à 15 valent 2 points (elles demandent un raisonnement). Seuil : 11 points sur 15 questions traitées.

Bloc 1 — Définir (Q1 à Q5)

- Q1.** Un indicateur clé se distingue d'une métrique par : (a) sa précision, (b) sa carte de définition et son responsable, (c) sa fréquence de calcul, (d) son coût de production.
- Q2.** Le même « taux de retour » peut valoir, sur le même socle : **1,12 %**, **1,16 %**, **1,17 %**, **1,91 %**. Ces écarts viennent : (a) d'erreurs de calcul, (b) d'un choix de numérateur et de dénominateur, (c) de la qualité des sources, (d) de la période observée.
- Q3.** La marge brute du socle vaut **29,12 %** : ce chiffre est rapporté : (a) au chiffre d'affaires toutes taxes comprises, (b) au chiffre d'affaires hors taxes, (c) aux achats, (d) au chiffre d'affaires brut.
- Q4.** Une carte de définition compte **7** champs. Lequel manque le plus souvent dans la pratique ? (a) la formule, (b) la source, (c) le responsable et le seuil, (d) la granularité.
- Q5.** Le contre-KPI d'un taux de rupture élevé est : (a) le taux de service, (b) la rotation de stock, (c) le panier moyen, (d) le taux de recouvrement.

Bloc 2 — Mesurer (Q6 à Q10)

- Q6.** Le socle compte **2 428** ruptures pour **33 296** couples produit-magasin-mois servis. Le taux de rupture vaut : (a) **7,29 %**, (b) **12,1 %**, (c) **0,73 %**, (d) **242,8 %**.
- Q7.** Additionner les stocks des **44** mois donne **2 034 733** unités, contre **45 882** au dernier mois. Cela montre qu'un stock est : (a) additif, (b) semi-additif, (c) non additif, (d) constant.

Q8. Le taux de service vaut **81,0 %** sur les commandes livrées et **78,2 %** sur toutes les commandes. L'écart s'explique par : (a) les retours, (b) les **319** commandes annulées, (c) les jours fériés, (d) les prix manquants.

Q9. Posé à **1,5** mois de couverture, un seuil d'alerte met en alerte **5 126** lignes sur **6 776**. Ce seuil est : (a) excellent, (b) trop sensible pour être utile, (c) insuffisant, (d) impossible à calculer.

Q10. Le coût logistique unitaire du socle vaut **3 284** FCFA par colis ; sur les seuls magasins, **3 281** FCFA. L'écart vient : (a) des **8 766** colis du dépôt, qui ne vend rien, (b) des taxes, (c) du carburant, (d) des retours.

Bloc 3 — Décider (Q11 à Q15, 2 points chacune)

Q11. La direction annonce : « la rotation doit passer de **9,42** à **12** tours ». Quelles conséquences mesurables attendez-vous, et quel indicateur le prouvera ?

Q12. Un indicateur de part de marché interne montre qu'un magasin gagne **2** points face à son voisin de la même ville. Pourquoi faut-il un contre-KPI, et lequel ?

Q13. Le taux de recouvrement est de **74 %** et l'encours échoué de plus de **90** jours pèse **1 091 339 030** FCFA. Rédigez l'action associée au seuil, avec un volume et un délai.

Q14. Un prestataire propose **40** indicateurs « pour être complet ». Rédigez en trois phrases l'argumentaire de refus, en vous appuyant sur les chiffres du dossier du projet raté.

Q15. Deux personnes justes se contredisent : l'une annonce **23 497** clients, l'autre **188 886**. Expliquez, puis dites quelle phrase de définition empêche le débat.

C · Exercices de rédaction de KPI (20 points · 2 exercices · PATCH_8 : les énoncés reformulent, ils ne

recopient jamais un exercice du cours)

R1 — Le taux de rupture (10 points de rédaction)

Énoncé. Rédigez la carte de définition complète du **taux de rupture** du réseau de distribution, en vous appuyant sur le socle fourni.

Attendu, dans cet ordre :

1. **la formule**, avec le numérateur et le dénominateur écrits en toutes lettres, et la précision du périmètre retenu (les couples réellement servis, pas le catalogue entier) ;
2. **les deux versions concurrentes** que vous avez écartées, avec leur valeur mesurée et la raison de votre choix ;
3. **la source** exacte et ses limites connues ;
4. **la granularité** et **la fréquence**, avec la date d'arrêté des données ;
5. **le responsable**, nommé par fonction ;
6. **le seuil d'alerte**, accompagné du nombre de produits qu'il met en alerte ;
7. **le contre-KPI**, avec sa valeur mesurée ;
8. **l'action si rouge**, avec un volume et un délai.

Barème. 8 points pour les champs 1 à 7 (1 par champ, 2 pour la formule) ; 2 points pour la cohérence d'ensemble. Un seuil sans action mesurée ne compte pas.

R2 — La part des clients non identifiés (10 points)

Énoncé. Le service commercial veut suivre la part des ventes réalisées sans identification du client. Rédigez la carte complète du KPI correspondant.

Attendu. La carte complète (les 7 champs), la **valeur actuelle mesurée** sur le socle, **le choix argumenté** entre la part des lignes et la part du chiffre d'affaires, et un **contre-KPI** qui empêche d'améliorer l'indicateur en créant des fiches en doublon.

Barème. 7 points pour les 7 champs ; 2 points pour la justification du choix de définition ; 1 point pour la pertinence du contre-KPI.

D · Étude de cas — « la direction veut un tableau de bord, elle a déjà tort de le vouloir comme ça » (30 points · seuil 18)

La commande

« Nous voulons **un** tableau de bord, pour **tout** : le chiffre d'affaires, la marge, les stocks, les ruptures, les livraisons, les impayés et la logistique, avec les comparaisons avec l'an dernier. Les directeurs l'ouvriront tous les matins. »

Trois questions vous sont posées par la direction, qui attend une réponse écrite.

Question 1 (10 points) — Reformulez la commande. Écrivez la commande telle que vous l'acceptez : les décisions à éclairer, les destinataires, la fréquence de publication, et le nombre maximal d'indicateurs. Justifiez chaque élément par un chiffre du socle ou du dossier.

Question 2 (12 points) — Choisissez cinq indicateurs, pas plus. Sélectionnez au plus 5 indicateurs parmi les 10 du mandat de reprise, donnez pour chacun sa valeur actuelle mesurée, son seuil et son contre-KPI. Puis expliquez, en une phrase par indicateur écarté, pourquoi il ne fait pas partie des cinq.

Question 3 (8 points) — Rédigez la phrase de refus argumenté. En trois phrases maximum, refusez les deux demandes suivantes sans perdre le client : « toutes les mailles, tous les jours » et « le même tableau de bord pour tout le monde ». Chaque phrase doit contenir un argument **mesuré**.

Le résultat attendu (à retrouver, pas à recopier)

- **La fréquence quotidienne** se justifie sur 2 ou 3 indicateurs au maximum (ruptures, encaissements du jour) ; les autres se lisent au mois.
- **Les 5 indicateurs** doivent inclure au moins un indicateur de marge ou de prix, un indicateur de stock ou de rupture, et un indicateur de trésorerie — sinon la direction ne peut pas arbitrer.
- **Le refus argumenté** s'appuie sur des ordres de grandeur du socle : **41** indicateurs livrés pour **4** lecteurs sur **14** ; une marge qui vaut **29,12 %** ou **24,67 %** selon la base ; un panier moyen de **107 396** FCFA par ticket ou **65 749** par ligne.

Barème de l'étude de cas

Élément	Points
Q1 : commande reformulée, complète et justifiée	10
Q2 : cinq indicateurs, valeurs mesurées, seuils, contre-KPI, écarts justifiés	12
Q3 : refus argumenté, chiffré, courtois	8
Total	30

Seuil de l'étude de cas : 18 points sur 30.

Correction détaillée

A - Réponses de récupération

1. Descriptif, diagnostique, prédictif, prescriptif — le prescriptif, seul, déclenche une action.
2. Ce que représente une ligne ; on le prouve par une clé dont le nombre de valeurs distinctes égale le nombre de lignes.
3. Formule, source, granularité, fréquence, responsable, seuil d'alerte, contre-KPI.
4. Sources, extraction, stockage, modèle, couche sémantique, visualisation, diffusion.
5. L'indicateur qui mesure ce qu'on dégrade en améliorant le premier ; il se publie à côté de lui.

B - Corrigé du quiz

Q	Réponse	Le point à retenir
1	b	une métrique se calcule ; un KPI se définit et s'attribue
2	b	1,12 % en valeur, 1,91 % en tickets : cinq définitions, toutes justes
3	b	la même marge vaut 24,67 % toute taxe comprise
4	c	le responsable et le seuil sont les deux champs qu'on saute
5	b	stocker plus réduit la rupture et dégrade la rotation (9,42 tours)
6	a	2 428 ÷ 33 296
7	b	semi-additif : additif dans l'espace, jamais dans le temps
8	b	les 319 commandes annulées sortent du premier dénominateur
9	b	à 1,5 mois, 75,6 % des lignes s'allument : le seuil ne discrimine plus
10	a	les 8 766 colis du dépôt, qui ne vend rien, sortent du second calcul
11	2 pts	réduire la couverture (1,27 mois) augmente les produits sous 1 mois (50 aujourd'hui) et pousse la rupture (7,29 %) : à surveiller par le taux de rupture par famille
12	2 pts	le gain d'un magasin est la perte de l'autre dans la même ville (60,8 % / 39,2 % à Ouagadougou) : le contre-KPI est le CA du réseau
13	2 pts	relancer les 1 155 factures de plus de 90 jours sous quinzaine, liste par ordre de montant, responsable du recouvrement
14	2 pts	41 indicateurs pour 4 lecteurs sur 14 et 11 semaines de vie : le plafond de 10 indicateurs documentés est un choix d'usage, pas de confort
15	2 pts	23 497 clients distincts ; 188 886 est la somme des clients par produit (un client achète 8,04 produits en moyenne) : la phrase « les comptages distincts ne s'additionnent pas » clôt le débat

C · Corrigé des exercices de rédaction

R1 — le taux de rupture. Formule retenue : couples produit-magasin-mois **en rupture** ÷ couples produit-magasin-mois **servis**, soit $2\,428 \div 33\,296 = 7,29\%$. Version écartée n° 1 : la part des jours de rupture sur les jours ouvrés (**13 129** jours cumulés), qui mesure la durée et non la couverture. Version écartée n° 2 : le taux calculé sur le catalogue entier, qui mesurerait l'assortiment. Granularité produit × magasin × mois ; fréquence mensuelle, données arrêtées au dernier jour du mois ; responsable : direction des opérations ; seuil : plus de **10 %** des couples servis, ce qui met en alerte les produits les plus touchés ; contre-KPI : rotation (**9,42** tours) et couverture (**1,27** mois) ; action si rouge : revue des 20 produits les plus touchés avec les achats sous cinq jours, sur la base du coût estimé de **169 386 812** FCFA.

R2 — la part des clients non identifiés. Formule : part du **chiffre d'affaires** réalisé sur des lignes sans identifiant client, soit **18,3 %** et **2 852 612 447** FCFA. Le choix de la part du chiffre d'affaires est justifié : la part des lignes mesurerait la saisie au comptoir, pas la connaissance de la clientèle. Granularité magasin × mois ; fréquence mensuelle ; responsable : qualité de la donnée ; seuil : **15 %** ; contre-KPI : nombre de fiches clients créées et taux de doublons — sans lui, l'indicateur s'améliore en polluant le référentiel ; action si rouge : contrôle de saisie par magasin, sur la liste des tickets concernés.

D · Corrigé de l'étude de cas

Q1 — la commande reformulée. Attendu : **un** écran, **5 à 10** décisions explicites, des destinataires nommés, une fréquence par indicateur (mensuelle pour la plupart, quotidienne pour les ruptures et les encaissements), et un plafond d'indicateurs assumé. La justification s'appuie sur le dossier raté : **41** indicateurs, **4** lecteurs sur **14**, **11** semaines.

Q2 — cinq indicateurs. Une sélection défendable : **CA net** (**15 595 154 955** FCFA ; contre-KPI : taux de retour **1,17 %**), **marge brute** (**29,12 %** hors taxes ; contre-KPI : volume, recette unitaire × **1,174** pour des unités × **1,169**), **taux de rupture** (**7,29 %** ; contre-KPI : rotation **9,42** tours), **taux de recouvrement** (**74,0 %** ; contre-KPI : CA net), **coût logistique unitaire** (**3 284** FCFA par colis ; contre-KPI : taux de service **81,0 %**). Les indicateurs écartés se justifient un par un : rotation (redondante avec la rupture, publiée en contre-KPI), panier moyen (utile au mois, pas au jour), part de marché interne (mesure interne, pas d'action), taux de retour (suivi en contre-KPI du CA).

Q3 — la phrase de refus. Attendu : trois phrases, chacune avec un argument chiffré. Par exemple : « Tous les jours et à toutes les mailles supposent de lire **2 428** couples de rupture par mois : nous publions au jour les ruptures et au mois les sept autres indicateurs, ce qui couvre **100 %** des décisions recensées. » ; « Le même tableau pour tous a produit **14** destinataires et **4** lecteurs : nous préparons deux vues, une de pilotage (5 indicateurs, mensuelle) et une d'action (ruptures et encaissements, quotidienne). » ; « Chaque définition supplémentaire coûte un débat non tranché — la marge vaut **29,12 %** ou **24,67 %** selon la base : nous écrivons les définitions avant l'écran. »

Rappel du barème d'ensemble : quiz **20** pts (seuil **11** sur **15**) + rédaction de KPI **20** pts + étude de cas **30** pts (seuil **18**) = **70** points, **seuil de réussite 45**.

Index et glossaire — La Voie des Données, modules M01 à M12

Appareil du livre. Tous les chiffres de ce fichier sont mesurés sur le dépôt par `tools/kit_exploitation.py` et consignés dans `05_livrables/kit_M01_M12.json` : rien n’est recopié à la main, rien ne peut être en retard sur le texte.

1. Les douze modules

Module	Titre	Chapitres	Heures	Pages	Mots	Exercices	Prérequis
M01	Lire les données, comprendre le métier	7	30	117	46 261	37	aucun
M02	Statistiques appliquées, depuis zéro	8	30	122	53 806	40	M01
M03	Excel pour l’analyse de données	8	30	126	56 023	43	M01, M02
M04	Qualité, préparation, documentation	6	30	92	41 751	29	M02, M03
M05	Chaîne de transformation (M/Q, SQL, pandas)	5	30	86	35 750	20	M03, M04
M06	Bases de données relationnelles	5	30	69	28 009	20	M04
M07	SQL : interroger, agréger, rejoindre	8	30	109	40 084	63	M06
M08	Python pour l’analyse de données	8	30	119	47 406	39	M05, M07
M09	Analyse exploratoire (EDA)	6	30	82	36 747	12	M02, M08
M10	Data visualization	7	30	106	52 663	14	M09
M11	SQL avancé pour la BI	7	30	97	39 363	14	M07, M08
M12	Fondamentaux de la Business Intelligence	6	30	82	31 389	35	M09, M10
Total M01 à M12		81	360	1207	509 252	366	

Le détail chapitre par chapitre, avec les numéros de page du livre assemblé, se trouve dans le sommaire des trois premières pages de `La_Voie_des_Donnees_M01-M12.pdf`.

2. Index des notions

Les termes listés sont ceux du bloc « Vocabulaire essentiel » de chaque chapitre : la liste des mots que le manuel définit lui-même, et où. Un terme cité dans plusieurs chapitres n’est pas un défaut — c’est un terme qu’on revoit.

- `#N/A` — **not available** — M03.C0
- `#PROPAGATION!` — `#SPILL!` — M03.C0
- `.backup` — M06.C0
- `:memory:` — M06.C0
- `assign` — **assign** — M05.C0
- `BIGINT` — M06.C0
- `BOOLEAN` — M06.C0
- `CHECK` — M06.C0
- `COPY ... TO` — M05.C0
- `COUNTWHERE ... IS NULL` — M05.C0
- `CREATE TABLE AS` — M05.C0
- `DATE` — M06.C0
- `DEFAULT` — M06.C0
- `drop_duplicates` — **drop_duplicates** — M05.C0
- `dtype` — **dtype** — M05.C0
- `EXPORT DATABASE` — M06.C0
- `EXTRACT` — M05.C0
- `GROUP BY` — M05.C0
- `groupby` — **groupby** — M05.C0

- **INSERT...SELECT** — M05.C0
- **INTEGER** — M06.C0
- **LEFT JOIN** — M05.C0
- **MATCH FULL / MATCH SIMPLE** — M06.C0
- **melt** — **melt** — M05.C0
- **merge** — **merge** — M05.C0
- **NOT NULL** — M06.C0
- **ON DELETE** — M06.C0
- **ON UPDATE** — M06.C0
- **pg_dump** — M06.C0
- **pgAdmin** — M06.C0
- **pip install** — M06.C0
- **pipe** — **pipe** — M05.C0
- **pivot_table** — **pivot_table** — M05.C0
- **PRIMARY KEY** — M06.C0
- **psql** — M06.C0
- **query** — **query** — M05.C0
- **read_csv_auto** — M05.C0
- **read_csv** — **read_csv** — M05.C0
- **REFERENCES** — M06.C0
- **ROW_NUMBEROVER** — M05.C0
- **STRPTIME / STRftime** — M05.C0
- **TEXT** — M06.C0
- **to_csv / to_parquet** — **to_csv / to_parquet** — M05.C0
- **TRY_CAST** — M05.C0
- **UNIQUE** — M06.C0
- **WITH** — M05.C0
- **Actualiser tout** — **Refresh All** — M03.C0, M05.C0
- **Actualité** — **Timeliness / Currency** — M04.C0
- **Additivité** — **additivity** — M02.C0
- **Adresse absolue / relative** — **absolute / relative reference** — M03.C0
- **agrégat** — M07.C0
- **agrégat conditionnel** — M11.C0
- **Agrégat conditionnel** — **conditional aggregate** — M03.C0
- **Agrégat** — **aggregate** — M03.C0
- **alias de colonne** — M07.C0
- **alias de table** — M07.C0
- **ancienneté** — M11.C0
- **AND** — M07.C0
- **annotation** — M10.C0
- **année sur année** — M11.C0
- **anomalie** — M09.C0
- **anomalie structurelle** — M09.C0
- **Anonymisation** — M04.C0
- **Anonymisation / pseudonymisation** — M01.C0
- **Aplatir** — **flatten** — M01.C0

- **Appariement tolérant — Fuzzy matching** — M04.C0
- **appel à la décision** — M10.C0
- **Arborescence — tree** — M01.C0
- **arbre de choix** — M10.C0
- **Arrondi d’affichage — display rounding** — M03.C0
- **Arrondi — rounding** — M01.C0
- **ASC** — M07.C0
- **audit** — M09.C0
- **AVG** — M07.C0
- **axe de temps** — M11.C0
- **balayage séquentiel** — M11.C0
- **Barre de formule — formula bar** — M03.C0
- **BETWEEN x AND y** — M07.C0
- **BI analyst — analyste décisionnel** — M01.C0
- **Biais de cadrage** — M01.C0
- **Booléen — boolean** — M01.C0
- **Borne exclusive — exclusive bound** — M03.C0
- **borne métier** — M09.C0
- **bruit** — M09.C0
- **Cadrage — scoping** — M01.C0
- **cadre de fenêtre** — M11.C0
- **canal perceptuel** — M10.C0
- **Cardinalité — cardinality** — M01.C0, M02.C0
- **CASE** — M07.C0, M07.C0
- **Cellule — cell** — M01.C0, M03.C0
- **Centre — measure of central tendency** — M02.C0
- **Champ Valeurs — Values field** — M03.C0
- **Chapeau / pied de page — preamble / footer** — M01.C0
- **chapeau de figure** — M10.C0
- **Chaîne de caractères — string** — M01.C0
- **Chaîne de valeur — value chain** — M01.C0
- **Chemin — path** — M01.C0
- **Chronologie — Timeline** — M03.C0
- **cinq secondes** — M10.C0
- **cité** — M10.C0
- **classement à égalité** — M11.C0
- **Classeur — workbook** — M03.C0
- **client non identifié** — M11.C0
- **client-serveur** — M06.C0
- **Clé candidate — Candidate key** — M04.C0
- **Clé composite — composite key** — M03.C0
- **Clé d’homonymie** — M04.C0
- **clé de départage** — M11.C0
- **clé de jointure** — M07.C0
- **Clé de jointure — Join key** — M05.C0
- **Clé de normalisation — Normalized key** — M04.C0

- **clé de regroupement** — **M07.C0**
- **Clé métier** — **Business key** — **M04.C0, M04.C0, M05.C0**
- **Clé primaire** — **primary key** — **M01.C0, M04.C0**
- **Clé étrangère** — **foreign key** — **M01.C0**
- **Clé** — **key** — **M03.C0**
- **COALESCE** — **M07.C0**
- **code de sortie** — **M11.C0**
- **Coercition** — **coercion / type casting** — **M03.C0**
- **cohorte** — **M11.C0**
- **Cohérence** — **Consistency** — **M04.C0**
- **Colonne calculée** — **Calculated column** — **M05.C0**
- **Colonne mixte** — **mixed column** — **M03.C0**
- **Colonne** — **column / field / variable** — **M01.C0**
- **Complétude** — **Completeness** — **M04.C0**
- **comptage conditionnel** — **M07.C0, M07.C0**
- **Compétences** — **Skills** — **M05.C0**
- **conclusion provisoire** — **M09.C0**
- **Conformité** — **Conformity** — **M04.C0**
- **Connaissance** — **insight** — **M01.C0**
- **Contexte de filtre** — **Filter context** — **M03.C0**
- **contrôle croisé** — **M11.C0**
- **Contrôle croisé** — **Cross-field rule** — **M04.C0**
- **Contrôle de cohérence final** — **M04.C0**
- **Convention de nommage** — **M04.C0, M11.C0**
- **Conversion / coulage** — **cast** — **M01.C0**
- **corrélation** — **M09.C0**
- **corrélé** — **M07.C0**
- **COUNT** — **M07.C0, M07.C0**
- **coupe croisée** — **M07.C0**
- **Coupe à la date** — **Date filter** — **M05.C0**
- **coût du SELECT *** — **M11.C0**
- **Critère de réussite** — **M01.C0**
- **Critère** — **criteria** — **M03.C0**
- **CROSS JOIN** — **M07.C0**
- **cumul** — **M11.C0**
- **cumul de pourcentages** — **M10.C0**
- **Cycle de vie** — **lifecycle** — **M01.C0**
- **Data analyst** — **M01.C0**
- **Data engineer** — **ingénieur données** — **M01.C0**
- **Data scientist** — **scientifique des données** — **M01.C0**
- **DataFrame** — **DataFrame** — **M05.C0**
- **densité** — **M10.C0**
- **DESC** — **M07.C0**
- **Diagnostic de qualité** — **Data profiling** — **M04.C0**
- **Dictionnaire de correspondance** — **Crosswalk** — **M04.C0**
- **Dictionnaire de données** — **data dictionary** — **M01.C0**

- **Discrète / continue — discrete / continuous** — M01.C0
- **distance Lab** — M10.C0
- **DISTINCT** — M07.C0
- **Donnée brute — raw data** — M01.C0
- **Donnée de référence — master data** — M01.C0
- **Donnée non structurée — unstructured** — M01.C0
- **Donnée semi-structurée — semi-structured** — M01.C0
- **Donnée structurée — structured data** — M01.C0
- **Donnée — data** — M01.C0
- **Dossier — folder / directory** — M01.C0
- **double axe** — M10.C0, M10.C0
- **double négation** — M07.C0
- **Doublon — duplicate row** — M03.C0
- **dpi** — M10.C0
- **décalage** — M11.C0
- **Décision** — M01.C0
- **déclaration** — M10.C0
- **Dédoublonnage — Deduplication** — M04.C0, M05.C0
- **Dépivoter — Unpivot Columns** — M05.C0
- **Développer — Expand** — M05.C0
- **Effectif — count / frequency count** — M02.C0
- **effet 3D** — M10.C0
- **ELSE** — M07.C0
- **Empreinte — checksum / hash** — M01.C0
- **Empreinte— Hash fingerprint** — M05.C0
- **en-tête de requête** — M11.C0
- **En-tête — header** — M01.C0
- **encodage** — M10.C0
- **Encodage — character encoding** — M01.C0
- **Encodage — encoding** — M01.C0
- **Erreur — Error** — M05.C0
- **Estimation — Estimate** — M05.C0
- **ET logique entre conditions — implicit AND** — M03.C0
- **ex æquo** — M11.C0
- **Exactitude — Accuracy** — M04.C0
- **EXISTS** — M07.C0
- **explosion de jointure** — M07.C0
- **export figé** — M10.C0
- **Expression SQL — SQL expression** — M05.C0
- **Extension — extension** — M01.C0
- **famille de produits** — M11.C0
- **Faux anonymisage** — M04.C0
- **Faux négatif silencieux — Silent false negative** — M04.C0
- **Faux positif / faux négatif** — M04.C0
- **fenêtre** — M11.C0
- **Fermer et charger — Close & Load** — M05.C0

- **Feuille** — **sheet** — M03.C0
- **fiche d'entrée** — M09.C0
- **Fiche de diagnostic** — **Data quality report** — M04.C0
- **fichier** `.duckdb` — M06.C0
- **fichier** `.sqlite` — M06.C0
- **fil rouge** — M10.C0
- **Filtre automatique** — **AutoFilter** — M03.C0
- **fonction de fenêtre** — M11.C0
- **Fonction volatile** — **volatile function** — M03.C0
- **Format** — M01.C0
- **Format de nombre** — **number format** — M03.C0
- **Format nombre** — **number format** — M01.C0
- **Format personnalisé** — **custom format** — M03.C0
- **Formule volatile** — **volatile** — M03.C0
- **FROM** — M07.C0
- **fréquence** — M11.C0
- **Fréquence** — **Frequency** — M05.C0
- **Fréquence** — **relative frequency** — M02.C0
- **FULL OUTER JOIN** — M07.C0
- **Fusionner** — **Merge** — M05.C0
- **Glissement** — **rolling window** — M03.C0
- **Gouvernance** — **Governance** — M05.C0
- **grain** — M11.C0
- **Grain** — **granularity** — M01.C0
- **Graine** — **random seed** — M05.C0
- **graphique natif** — M10.C0
- **graphique trompeur** — M10.C0, M10.C0
- **GROUP BY** — M07.C0
- **groupe** — M07.C0
- **HAVING** — M07.C0
- **hiérarchie visuelle** — M10.C0
- **Identifiant direct** — M04.C0
- **Impossible** — **Invalid** — M04.C0
- **Imputation** — **Imputation** — M04.C0
- **IN** — M07.C0
- **index** — M11.C0
- **indicateur de regroupement** — M11.C0
- **Indicateur** — **KPI** — M01.C0
- **Individu** — **statistical unit / record** — M02.C0
- **Information** — M01.C0
- **INNER JOIN** — M07.C0
- **insight** — M10.C0
- **Invraisemblable** — **Implausible** — M04.C0
- **IS NULL** — M07.C0
- **jeu de tests** — M11.C0
- **Jeu figé** — **frozen dataset** — M01.C0

- **jointure en étoile** — M07.C0
- **jointure externe** — M11.C0
- **Jointure — Join** — M05.C0
- **joker %** — M07.C0
- **Jour ouvré — business day** — M03.C0
- **Journal de transformation — transformation log** — M01.C0
- **Journal des transformations** — M04.C0
- **Journal — log** — M01.C0
- **lecture en 60 secondes** — M10.C0
- **LEFT JOIN** — M05.C0, M07.C0
- **Licite surprenante — Legitimate extreme** — M04.C0
- **Ligne d'eau — watermark** — M01.C0
- **ligne de synthèse** — M07.C0
- **Ligne de total — totals row** — M03.C0
- **Ligne — row / record** — M01.C0
- **LIKE** — M07.C0
- **LIMIT** — M07.C0
- **liste** — M11.C0
- **Liste d'étapes — Step list** — M05.C0
- **Liste déroulante — drop-down list** — M03.C0
- **Livrable — deliverable** — M01.C0
- **légende** — M09.C0
- **manquant** — M09.C0
- **Marqueur de manque — Missing indicator** — M04.C0
- **matrice de rétention** — M11.C0
- **matérialisation** — M11.C0
- **Mesure — Measure** — M03.C0
- **MIN / MAX** — M07.C0
- **Minimisation — data minimisation** — M01.C0
- **Minimum / Maximum** — M02.C0
- **Mise en forme conditionnelle — conditional formatting** — M03.C0
- **Mise en forme — formatting** — M03.C0
- **Modalité — value / category** — M02.C0
- **Mode — mode** — M02.C0
- **Modèle de données — Data Model** — M03.C0
- **Modèle sémantique — semantic model** — M01.C0
- **montant** — M11.C0
- **Moyenne arithmétique — mean** — M02.C0
- **moyenne mobile** — M11.C0
- **Moyenne pondérée — weighted mean** — M02.C0
- **multi-clés** — M07.C0
- **Mécanisme d'absence — Missing mechanism** — M04.C0
- **médiane de 5** — M11.C0
- **Médiane — median** — M02.C0
- **Métadonnées — metadata** — M01.C0
- **Méthode pandas — pandas method** — M05.C0

- **Métier — business / functional owner** — M01.C0
- **Métrique** — M01.C0
- **Nom de plage — named range** — M03.C0
- **Nombre de série de date — date serial** — M03.C0
- **Nombre à virgule / point — decimal separator** — M01.C0
- **Non applicable — Not applicable** — M04.C0
- **NOT** — M07.C0
- **note d'EDA** — M09.C0
- **null-skipping** — M07.C0
- **NULLIF** — M07.C0
- **NULLS FIRST / NULLS LAST** — M07.C0
- **Numéro de série — serial date** — M01.C0
- **numérotation** — M11.C0
- **objection** — M10.C0
- **OFFSET** — M07.C0, M07.C0
- **ON vs WHERE** — M07.C0
- **opérateur de comparaison** — M07.C0
- **OR** — M07.C0
- **ORDER BY** — M07.C0
- **ordre d'exécution** — M07.C0, M11.C0
- **ordre de fenêtre** — M11.C0
- **ordre métier** — M10.C0
- **ordre perceptuel** — M10.C0
- **pagination par clé** — M07.C0
- **palette catégorielle** — M10.C0
- **palette divergente** — M10.C0
- **palette séquentielle** — M10.C0
- **Paramètre — Parameter** — M05.C0
- **part relative** — M11.C0
- **partition** — M11.C0, M11.C0
- **PATH** — M08.C0
- **Perte par jointure — Join-induced loss** — M04.C0
- **Pipeline — Pipeline** — M05.C0
- **Pivot en SQL — Pivot in SQL** — M05.C0
- **Pivoter — Pivot Column** — M05.C0
- **piège d'inférence** — M09.C0
- **Plage de somme — sum range** — M03.C0
- **Plage nommée — named range** — M03.C0
- **Plage propagée — spill range** — M03.C0
- **Plage — range** — M03.C0, M03.C0
- **plan d'exécution** — M11.C0
- **Politique de valeurs manquantes — Missing data policy** — M04.C0
- **pondération** — M11.C0
- **Population — population** — M02.C0
- **port TCP** — M06.C0
- **projection** — M11.C0

- **Propagation d'erreur — error propagation** — M03.C0
- **Propagation — spill** — M03.C0
- **périmètre** — M11.C0
- **Périmètre — scope** — M01.C0
- **période partielle** — M11.C0
- **quantile** — M11.C0
- **Quartile inclusif / exclusif — QUARTILE.INC / .EXC** — M03.C0
- **Quasi-identifiant** — M04.C0
- **Question analytique** — M01.C0
- **question servie** — M10.C0, M10.C0
- **rang dense** — M11.C0
- **rapport de contraste** — M10.C0
- **rareté légitime** — M09.C0
- **raster** — M10.C0
- **README de données** — M04.C0
- **Recensement — census** — M02.C0
- **Recherche approximative — approximate match** — M03.C0
- **Recherche exacte — exact match** — M03.C0
- **Recherche à gauche — left lookup** — M03.C0
- **redondance** — M10.C0
- **Registre des doublons — Duplicate register** — M04.C0
- **regroupement multi-colonnes** — M07.C0
- **Regrouper — Group By** — M05.C0
- **Rejouabilité — reproducibility** — M01.C0
- **Relation — Relationship** — M03.C0
- **relire un classeur** — M10.C0
- **repliement d'un libellé** — M11.C0
- **Report — fill** — M03.C0
- **Reproductibilité — Reproducibility** — M05.C0, M05.C0
- **Représentativité — representativeness** — M02.C0
- **requête en escalier** — M11.C0
- **requête à l'identique** — M11.C0
- **Requête — Query** — M03.C0
- **Responsable de la donnée — data steward** — M01.C0
- **RFM** — M11.C0
- **ROUND** — M07.C0
- **ROW_NUMBER** — M07.C0
- **Règle de garde — Survivorship rule** — M04.C0
- **Règle univariée — Univariate rule** — M04.C0
- **récapitulatif multi-niveaux** — M11.C0
- **récence** — M11.C0
- **récence** — M11.C0
- **récurion** — M11.C0
- **Référence absolue — absolute reference** — M03.C0
- **Référence circulaire — circular reference** — M03.C0
- **Référence mixte — mixed reference** — M03.C0

- **Référence relative — relative reference** — M03.C0
- **Référence structurée — structured reference** — M03.C0, M03.C0
- **résolution utile** — M10.C0
- **rétenion** — M11.C0
- **Rétention — retention** — M01.C0
- **saison** — M11.C0
- **saisonnalité** — M09.C0
- **Sauvegarde — backup** — M01.C0
- **Schéma — schema** — M01.C0
- **segment** — M07.C0, M09.C0, M11.C0
- **Segment — Slicer** — M03.C0
- **SELECT** — M07.C0
- **Session en mémoire — In-memory session** — M05.C0
- **soupçon** — M09.C0
- **Source de vérité — system of record** — M01.C0
- **Source — Source** — M05.C0, M05.C0
- **sous-requête de liste** — M07.C0
- **sous-requête scalaire** — M07.C0, M11.C0
- **sous-total** — M11.C0
- **Sous-total visible — SUBTOTAL** — M03.C0
- **SUM** — M07.C0
- **support** — M10.C0
- **surcharge** — M10.C0
- **Système 1904 — 1904 date system** — M03.C0
- **Séparateur / encodage / décimale** — M01.C0
- **Séparateur — delimiter** — M01.C0
- **Séparateur — separator** — M01.C0
- **séquence** — M10.C0
- **Série de dates — date serial** — M03.C0
- **série fabriquée** — M11.C0
- **Série — Series** — M05.C0
- **table calendrier** — M11.C0
- **table d'expression commune** — M11.C0
- **Table de référence — lookup table** — M03.C0
- **table dérivée** — M07.C0
- **table en mémoire** — M11.C0
- **Table — table** — M01.C0
- **tableau croisé** — M11.C0
- **Tableau croisé dynamique — PivotTable** — M03.C0
- **tableau de bord** — M10.C0
- **Tableau— table** — M03.C0
- **Taille — Size** — M05.C0
- **Taux de déclenchement — Alert rate** — M04.C0
- **tendance** — M09.C0
- **test de non-régression** — M11.C0
- **Test de réparation — Repair plausibility check** — M04.C0

- **Texte libre — free text** — M01.C0
- **titre qui affirme** — M10.C0
- **top N par groupe** — M11.C0
- **total de contrôle** — M11.C0
- **Transformation — Transformation** — M05.C0
- **Transposition— Date swap** — M05.C0
- **treemap** — M10.C0
- **tri par alias** — M07.C0
- **tri par position** — M07.C0
- **Tri à plusieurs niveaux — multi-level sort** — M03.C0
- **Trimée— trimmed mean** — M02.C0
- **troncature déclarée** — M10.C0
- **trou de série** — M11.C0, M11.C0
- **Typage / inférence — type inference** — M01.C0
- **Type de données — data type** — M01.C0, M05.C0
- **Unicité — Uniqueness** — M04.C0
- **Valeur aberrante — Outlier** — M04.C0
- **valeur attendue** — M11.C0
- **Valeur de sentiment — Sentinel value** — M04.C0
- **Valeur distincte — distinct value** — M01.C0
- **Valeur extrême — outlier** — M02.C0
- **valeur gelée** — M11.C0
- **Valeur manquante — Missing value** — M04.C0
- **Valeur manquante — null / missing** — M01.C0
- **Validation des données — data validation** — M03.C0
- **Validité — Validity** — M04.C0
- **Variable** — M02.C0, M08.C0
- **Variable qualitative / catégorielle — categorical** — M01.C0
- **Variable quantitative — quantitative variable** — M01.C0
- **vectorel** — M10.C0
- **Verdict — Verdict** — M05.C0
- **version direction** — M10.C0
- **version équipe** — M10.C0
- **Versionnage — versioning** — M01.C0, M01.C0
- **Volets figés — frozen panes** — M03.C0
- **vue** — M07.C0, M11.C0
- **WHERE** — M07.C0, M07.C0
- **Zone de données** — M04.C0
- **Zone de nom — name box** — M03.C0
- **zéro ajouté** — M10.C0
- **écart expliqué** — M11.C0
- **Échantillon — sample** — M02.C0
- **échelle de décision** — M08.C0
- **Échelle de gravité — Severity ladder** — M04.C0
- **Échelle de mesure — measurement scale** — M01.C0
- **échelle logarithmique** — M10.C0

- **éclatement** — M11.C0
- **écran** — M10.C0
- **Éditeur Power Query** — M03.C0
- **Éditeur Power Query — Power Query Editor** — M05.C0
- **étage** — M11.C0
- **Étape appliquée — Applied step** — M05.C0
- **Étape— Step** — M05.C0
- **Étendue — range** — M02.C0
- **étiquette** — M07.C0

3. Glossaire

La première phrase de chaque encadré « Définition » du manuel, dans l'ordre des chapitres.

- **M01.C0 — Chemin — path** — l'adresse complète d'un fichier ou d'un dossier.
- **M01.C0 — Extension** — file extension — les lettres placées après le dernier point du nom, qui indiquent le **format** du fichier, donc le logiciel capable de l'ouvrir et ce que ce logiciel va y trouver.
- **M01.C0 — Arborescence — tree** — l'organisation des dossiers en arbre, depuis un point d'entrée unique.
- **M01.C0 — Nommage — naming convention** — grammaire écrite des noms de fichiers.
- **M01.C0 — Rétention** — retention period — durée pendant laquelle on conserve un fichier et ses versions.
- **M01.C0 — Journal — log** — fichier texte horodaté qui enregistre, dans l'ordre, ce qui a été fait sur les données.
- **M01.C0 — Donnée — data** — un fait enregistré à un instant précis par un système, avec une unité : une valeur de case.
- **M01.C0 — Information** — une donnée replacée dans son contexte : quoi, qui, quand, combien, dans quelle unité.
- **M01.C0 — Connaissance — insight** — deux informations comparées plus une explication que l'on peut discuter.
- **M01.C0 — Décision** — un engagement : une action, un responsable, une date, des conséquences assumées.
- **M01.C0 — Métrique — measure** — la définition de calcul d'un indicateur, écrite noir sur blanc : ce qui entre, ce qui sort, sur quelle période, comment on additionne.
- **M01.C0 — Indicateur — KPI (key performance indicator)** — une métrique suivie dans le temps parce qu'elle est rattachée à un objectif et à un seuil qui déclenche quelque chose.
- **M01.C0 — Table** — un rectangle de données où chaque colonne est une caractéristique mesurée pour tous les enregistrements et chaque ligne un objet du monde réel, au grain choisi.
- **M01.C0 — Enregistrement — row / record** — la ligne, c'est-à-dire un objet : une ligne de ticket, et non un ticket, non un client, non un mois.
- **M01.C0 — Grain — granularity** — la phrase qui dit ce que représente une ligne.
- **M01.C0 — Clé primaire — primary key** — une colonne, ou un couple de colonnes, qui rend chaque ligne unique.
- **M01.C0 — Clé étrangère — foreign key** — une colonne qui renvoie à la clé d'une autre table.
- **M01.C0 — Valeur distincte — distinct value** — une valeur comptée une seule fois.

- **M01.C0 — Type de données — data type** — la nature d’une valeur, qui détermine ce que l’on a le droit d’en faire.
- **M01.C0 — Typage automatique — type inference** — le mécanisme par lequel un logiciel devine le type de chaque colonne en lisant un échantillon de lignes.
- **M01.C0 — Séparateur décimal — decimal separator** — le signe qui sépare la partie entière de la partie décimale.
- **M01.C0 — Valeur manquante — null** — l’absence de valeur.
- **M01.C0 — Donnée structurée — structured data** — des lignes et des colonnes, un schéma connu d’avance : le format où l’on peut compter, joindre et agréger sans rien interpréter.
- **M01.C0 — Donnée semi-structurée — semi-structured data** — des étiquettes portées par le fichier lui-même (JSON, XML, journaux applicatifs), mais des enregistrements qui ne se ressemblent pas tous : une clé peut manquer d’un objet à l’autre, et c’est ce manque qui demande un traitement.
- **M01.C0 — Donnée non structurée — unstructured data** — texte, image, son, vidéo : aucune colonne.
- **M01.C0 — Encodage de caractères — character encoding** — la table qui associe des octets à des signes.
- **M01.C0 — Variable quantitative — quantitative variable** — une valeur qui se mesure ou se compte, et pour laquelle les calculs ont un sens : quantité, prix, montant, durée.
- **M01.C0 — Variable qualitative — categorical variable** — une appartenance : famille, catégorie, statut, marque.
- **M01.C0 — Cardinalité — cardinality** — le nombre de valeurs distinctes d’une colonne.
- **M01.C0 — Échelle de mesure — measurement scale** — le niveau d’information porté par une colonne : nominale (des noms, sans ordre), ordinale (un ordre, mais des distances qui n’ont pas de sens constant), à rapport (ordre, distances, et un zéro qui veut dire « rien »).
- **M01.C0 — Source de vérité — system of record** — le système où une information est officiellement détenue : ici, la caisse du magasin.
- **M01.C0 — Donnée brute — raw data** — l’état exact de ce qui a été reçu, octet pour octet, avant la moindre correction.
- **M01.C0 — Manifeste de jeu de données — dataset manifest** — fiche courte qui accompagne un jeu figé : ce qu’il contient, d’où il vient, quelle version du code l’a produit, quelles empreintes, quelles limites connues, qui a validé, à quelle date.
- **M01.C0 — Empreinte — checksum / hash** — une signature courte calculée sur le contenu d’un fichier.
- **M01.C0 — Jeu figé — frozen dataset** — copie verrouillée, nommée, référencée par un identifiant, sur laquelle un chiffre publié a été calculé.
- **M01.C0 — Chaîne de valeur — value chain** — la suite d’étapes par lesquelles une donnée devient une décision qui rapporte ou qui épargne.
- **M01.C0 — Cadrage — scoping** — l’accord écrit sur la question, le périmètre, le délai et le critère de réussite.
- **M01.C0 — Livrable — deliverable** — un objet remis, nommé, daté, avec un destinataire identifié.
- **M01.C0 — Modèle sémantique — semantic model** — la couche d’un outil de décision où les mesures et les relations sont définies une fois pour toutes.
- **M01.C0 — Rejouabilité — reproducibility** — pouvoir refaire le même résultat plus tard, avec les mêmes entrées, sans avoir à s’en souvenir.
- **M02.C0 — Population** — population — l’ensemble complet des individus que l’on voudrait décrire.

- **M02.C0 — Échantillon** — sample — la partie réellement observée de la population, sélectionnée par une > règle.
- **M02.C0 — Tirage — sampling** — la règle, écrite et reproductible, qui a sélectionné les individus de > l'échantillon dans la population.
- **M02.C0 — Variable** — variable — une caractéristique relevée pour **tous** les individus, c'est-à-dire > une colonne dont on peut dire en une phrase ce qu'elle mesure et dans quelle unité.
- **M02.C0 — Modalité** — value, category — une valeur prise par la variable (« Peinture », 0,04, vide).
- **M02.C0 — Cardinalité — cardinality** — le nombre de modalités distinctes d'une variable.
- **M02.C0 — Moyenne arithmétique** — arithmetic mean — le total réparti également.
- **M02.C0 — Médiane** — median — la valeur du milieu après tri.
- **M02.C0 — Mode** — mode — la valeur la plus fréquente.
- **M02.C0 — Moyenne pondérée** — weighted mean — moyenne dans laquelle chaque valeur compte pour un > poids.
- **M02.C0 — L'étendue** (range) est la différence entre maximum et minimum.
- **M02.C0 — Les bornes de Tukey** (Tukey fences, « moustaches » ou limites externes en français) sont > $Q1 - 1,5 \times IQR$ et $Q3 + 1,5 \times IQR$.
- **M02.C0 — Le coefficient de variation** (CV, coefficient of variation) est l'écart-type divisé par la > moyenne, exprimé en pourcentage.
- **M02.C0 — L'erreur type de la moyenne** (standard error of the mean, SE) est l'écart-type divisé par la > racine carrée de l'effectif.
- **M02.C0 — L'écart absolu médian** (MAD, median absolute deviation) est la médiane des valeurs absolues > des écarts à la médiane.
- **M02.C0 — Le p-ième quantile** d'une série triée est la valeur en dessous de laquelle se trouve la part p > de l'effectif.
- **M02.C0 — La fréquence cumulée** (cumulative frequency) associe à une valeur la proportion > d'observations inférieures ou égales ; sa courbe est la **fonction de répartition** (CDF, cumulative > distribution function, ogive en français).
- **M02.C0 — La boîte à moustaches** (boxplot) représente cinq nombres : le premier quartile et le > troisième, qui forment la boîte ; la médiane, qui la coupe ; la plus petite et la plus grande observation > restant dans les bornes de Tukey, qui forment les moustaches.
- **M02.C0 — On appelle concentration** le fait qu'une part donnée des observations porte une part > différente — en général plus grande — du total.
- **M02.C0 — La distribution** d'une variable décrit comment ses valeurs se répartissent : quelles valeurs, > avec quelles fréquences.
- **M02.C0 — Un histogramme** (histogram) est la représentation en barres jointives des effectifs > tombant dans des intervalles contigus de même amplitude — les **classes**.
- **M02.C0 — L'asymétrie** (skewness, notée γ_1 ou skew) mesure le déséquilibre de la distribution autour > de son centre : en simplifiant, la moyenne des écarts au centre élevés au cube, normalisée par le cube de > l'écart-type.
- **M02.C0 — Le kurtosis** (kurtosis ; « aplatissement » en français, bien qu'il ne mesure pas la hauteur > du pic contrairement à une légende tenace) pénalise la présence de valeurs lointaines : moyenne des écarts au > centre élevés à la quatrième puissance, normalisée.
- **M02.C0 — La covariance** entre deux variables x et y est la moyenne des produits des écarts : pour chaque > observation, $(x_i - \text{moyenne de } x) \times (y_i - \text{moyenne de } y)$.
- **M02.C0 — Le coefficient de corrélation linéaire de Pearson** — Pearson correlation coefficient, noté > r — est la covariance divisée par le produit des deux écarts-types.

- **M02.C0** — La **corrélation de rang de Spearman** — Spearman rank correlation, notée ρ — remplace chaque > valeur par son rang dans la série triée, puis calcule la corrélation de Pearson sur ces rangs.
- **M02.C0** — La **régression linéaire simple** — simple linear regression, méthode des moindres carrés > (ordinary least squares, OLS) — trace la droite qui minimise la somme des carrés des écarts verticaux.
- **M02.C0** — On dit que **x cause y** si agir sur x change y.
- **M02.C0** — Une **variable de confusion** (confounder) influence à la fois x et y et fait apparaître un lien > qui n'existe pas entre elles.
- **M02.C0** — La **probabilité** d'un événement est la fréquence vers laquelle se stabilise cette fréquence quand > on répète l'expérience beaucoup de fois.
- **M02.C0** — L'**erreur d'échantillonnage** (sampling error) est la dispersion d'une statistique si l'on > refaisait le tirage.
- **M02.C0** — Un **biais d'échantillonnage** (sampling bias) est un écart systématique entre la statistique > attendue et la valeur de population, produit par la **méthode de sélection**.
- **M02.C0** — Un **biais de couverture** (coverage bias) est le cas où la liste de départ ne contient pas toute > la population.
- **M02.C0** — Un **intervalle de confiance à 95 %** est fabriqué par une méthode qui, appliquée à une infinité de > tirages, enfermerait la vraie valeur dans 95 % des cas.
- **M02.C0** — Le **bootstrap** (bootstrap) traite l'échantillon comme une population : on tire 2 000 fois, > **avec remise**, un échantillon de même taille, on recalcule la statistique, et les 2,5^e et 97,5^e percentiles de > ces 2 000 résultats servent d'intervalle.
- **M02.C0** — L'**échantillonnage stratifié** (stratified sampling) découpe la population en cases (jour du > mois, catégorie, vendeur, trimestre), tire au sort dans chaque case, puis combine les résultats pondérés par le > poids réel des cases.
- **M02.C0** — Un **test statistique** est une procédure qui compare l'écart observé à ce que le hasard produit > habituellement.
- **M02.C0** — La **puissance statistique** (statistical power) d'un test est la probabilité de rejeter H_0 > quand l'écart réel vaut une taille donnée ; la **marge de détection** (en anglais minimum detectable effect) est > l'écart que le test relève avec 80 % de puissance.
- **M02.C0** — Le **test de Mann-Whitney** (Mann-Whitney U test, test de la somme des rangs) classe les > observations des deux groupes de la plus petite à la plus grande et compare les rangs, non les montants : il > éprouve l'égalité des deux distributions, et reste lisible quand la moyenne n'est qu'un point d'ancrage fragile.
- **M02.C0** — Le **test du khi deux d'indépendance** (chi-squared test of independence) compare, case par > case, les effectifs observés d'un tableau croisé aux effectifs **attendus** si les deux variables n'avaient aucun > lien ; la statistique additionne les écarts relatifs, et se compare à la loi du khi deux dont les degrés de liberté > valent (nombre de lignes - 1) × (nombre de colonnes - 1).
- **M03.C0** — Une **cellule active** est la seule cellule où l'on tape : elle est encadrée d'un liseré épais et > son adresse est écrite dans la zone de nom.
- **M03.C0** — On dit qu'une cellule est **reconnue comme nombre** quand le tableur y lit un contenu chiffré, > manipulable par `SOMME()`, triable, moyennable.
- **M03.C0** — La **zone utilisée** (used range) est le plus petit rectangle contenant toutes les cellules ayant > un contenu **ou** une mise en forme.
- **M03.C0** — Une **conversion** de types change la nature d'un contenu ; une **mise en forme** change son > rendu.
- **M03.C0** — Un **format à sections** s'écrit positif;négatif;zéro;texte.
- **M03.C0** — Une date stockée comme date est un **nombre de série** : un compteur de jours, dont l'heure est la > partie décimale.

- **M03.C0** — On parle de **coercition** quand le tableur force un contenu dans un type demandé : > =CNUM("56 658") échoue, =CNUM("56658") réussit, =CNUM("") renvoie #VALEUR!.
- **M03.C0** — L'**ancrage** d'une règle de mise en forme conditionnelle est la cellule où l'on se trouvait quand > on l'a écrite : la formule est ensuite reportée relativement sur chaque cellule de la plage.
- **M03.C0** — Une **table** se nomme une fois, et ce nom devient une adresse vivante : Table-Ventes désigne > Feuille! [A1:M481] aujourd'hui, A1:M521 dans un mois.
- **M03.C0** — La **portée** d'un tableau est l'intervalle qu'il occupe, étendue à chaque ligne ajoutée en > contiguïté.
- **M03.C0** — Un doublon n'existe pas dans les données, il existe **relativement à une clé**.
- **M03.C0** — Une **colonne calculée** d'un tableau est une formule saisie une fois : le tableur la propage à > toutes les lignes et la recompose à chaque ligne nouvelle.
- **M03.C0** — Le **maillage** d'une formule est l'ensemble des cellules qu'elle lit.
- **M03.C0** — Le **sens du report** est la direction où le décalage s'applique : vers le bas pour une colonne de > lignes, vers la droite pour une ligne de mois.
- **M03.C0** — L'**ordre de priorité** est la convention qui range les opérateurs : puissances, puis > multiplications et divisions, puis additions et soustractions, puis comparaisons et concaténation.
- **M03.C0** — Une erreur de tableur est une **valeur réservée**, qui se propage comme les autres le long du > maillage — mais que SOMME, MOYENNE, NB et les filtres **ignorent en silence**.
- **M03.C0** — Un **critère de comparaison** s'écrit entre guillemets, opérateur compris : ">=1000000", > "<>" (non vide), "=0" (vide n'est pas zéro).
- **M03.C0** — Le **dénominateur** d'un agrégat est le nombre de lignes qu'il a réellement regardées.
- **M03.C0** — Une **partition fermée — closed partition** est un découpage dont les morceaux rendent le tout : > les lignes s'additionnent (57 + 143 + 112 + 53 + 57 + 24 + 34 = 480) et les chiffres d'affaires aussi.
- **M03.C0** — Un agrégat **robuste** est un agrégat dont on connaît le périmètre exact : quelles lignes, quels > vides, quelles erreurs, quel filtre.
- **M03.C0** — Une **clé unique** est une valeur qui apparaît une fois et une seule dans la table où l'on > cherche.
- **M03.C0** — Une **clé typée** est une clé du même type des deux côtés.
- **M03.C0** — Un **repli** est la valeur renvoyée quand la recherche échoue : le 4^e argument de RECHERCHEX, > contre une fonction SIERREUR autour de RECHERCHEV.
- **M03.C0** — Une fonction de texte **découpante** (GAUCHE, DROITE, STXT) compte en caractères, pas en > significations : elle ne sait pas que le 4^e caractère est un tiret.
- **M03.C0** — Le **padding** est l'ajout de zéros non significatifs pour que deux écritures d'un même nombre > aient la même longueur de texte.
- **M03.C0** — Une **période ancrée** est une période définie par une formule qui la recalcule > (=DATE(ANNEE(AUJOURDHUI());1;1)), par opposition à une période tapée à la main (>=01/01/2025).
- **M03.C0** — Une **fonction matricielle dynamique** est une fonction dont le résultat est un tableau de taille > variable : FILTRE, UNIQUE, TRIER, TRIERPAR, SEQUENCE, TABLEAU.ALEA.
- **M03.C0** — LET(nom1;valeur1;...;calcul) attache des noms intermédiaires dans une formule.
- **M03.C0** — Un **décalage d'échelle** est l'écart entre la période mesurée et la période dénominateur.
- **M03.C0** — Une **agrégation juste** est une agrégation dont on peut nommer les lignes consommées : la > moyenne des 480 lignes (75 152 FCFA) répond à « par ligne », la moyenne des sept moyennes de catégorie (98 281) > répond à une autre question.

- **M03.C0** — Dans le modèle, une **colonne calculée** ajoute une valeur par ligne, stockée dans la table et > recalculée à l'actualisation — le même mot qu'en C03, où il désigne une formule propagée à la saisie dans le > tableau ; la mécanique change, le mot reste.
- **M03.C0** — **Protéger une feuille** interdit d'écrire dans les cellules verrouillées, sans rien chiffrer ni > cacher ; **chiffrer le classeur** (Fichier → Informations → Protéger le classeur) est le seul geste qui empêche > la lecture du fichier.
- **M03.C0** — Une **macro enregistrée** est une retranscription d'actions en VBA dans un classeur `.xlsm` : > elle rejoue des clics à des adresses précises, sans connaître ni les erreurs ni les changements de forme.
- **M04.C0** — Un **encodage** est la table qui associe un octet à un signe.
- **M04.C0** — Une **clé candidate** est un jeu de colonnes dont les valeurs identifient une ligne et une seule.
- **M04.C0** — Un **contrôle croisé** est une vérification qui porte sur deux colonnes au lieu d'une.
- **M04.C0** — On parle d'**intégrité référentielle** quand toute valeur d'une colonne de référence pointe une > ligne existante de la table référencée.
- **M04.C0** — Une **convention de codage** est un choix d'écriture, arbitraire mais écrit, qui donne à une > absence un signe visible : 0, -1, la chaîne vide, un enregistrement dédié.
- **M04.C0** — Une **jointure externe** (à gauche, à droite, pleine) conserve les lignes sans correspondance en > laissant le champ complémentaire vide ; une jointure **intérieure** les supprime.
- **M04.C0** — **Imputer par le cas le plus semblable** (hot-deck, ou k plus proches voisins) consiste à > emprunter la valeur d'une ligne qui ressemble fort à celle qui manque : même produit, même magasin, même mois, > même type de client.
- **M04.C0** — Un **marqueur de manque** est une colonne ajoutée, en général booléenne, qui survit à la > réparation : `ttc_reconstitue = VRAI` sur les 3 421 lignes.
- **M04.C0** — Un **doublon logique** est une paire de lignes qui décrivent le même fait alors qu'une seule > pourrait le décrire.
- **M04.C0** — Un **doublon structurel** est une répétition que le modèle de données autorise et que le métier > reconnaît : deux lignes d'un même ticket, deux relevés de stock le même jour dans deux magasins.
- **M04.C0** — Le **registre des doublons** est la liste nominative de ce qui a été retiré : une ligne par > groupe, avec les deux identifiants en présence et la règle qui a départagé.
- **M04.C0** — Un **dictionnaire de correspondance** (crosswalk) est une table qui relie les identifiants ou les > libellés de deux systèmes.
- **M04.C0** — Une **règle de contrôle** est une affirmation vérifiable sur une donnée, assortie d'une décision et > d'un destinataire : « le prix d'une ligne reste dans la bande négociée du catalogue, sinon la ligne est retournée > au magasin pour validation ».
- **M04.C0** — Un **test de réparation** consiste à appliquer la correction en simulation et à repasser la règle > de domaine dessus.
- **M04.C0** — Un **registre de contrôles** est la table qui énonce, pour chaque colonne ou groupe de colonnes : la > règle, son niveau, sa cadence, son destinataire, son taux de déclenchement attendu et sa mesure du jour.
- **M04.C0** — Deux règles sont **redondantes** quand elles signalent exactement le même jeu de lignes.
- **M04.C0** — On appelle **normalisation** toute transformation bijective à l'identifiant près appliquée à une > valeur, qui change sa représentation et non sa signification : +226 77 76 25 98 96 et +2267776259896 > désignent le même abonné ; 16 638 FCFA et 16638 désignent la même somme.

- **M04.C0** — Un **contrôle de convergence** fait arriver le même résultat par deux chemins qui n'ont aucune > raison de se ressembler : l'un lit ce que l'opérateur a tapé, l'autre recalcule depuis d'autres colonnes.
- **M04.C0** — Une **conversion d'unité** multiplie une valeur par un facteur fixé par une définition > extérieure au fichier ($1 \text{ m}^3 = 1\,000 \text{ L}$), et change le nom de la colonne en conséquence : `quantite` devient > `quantite_litres`.
- **M04.C0** — Un **encodage** est une table de correspondance octet → caractère.
- **M04.C0** — Un **README de données** est le document qui répond, sans ouvrir les fichiers : ce que contient > le dossier, à quel grain, sous quelles règles de gestion, et ce qui y est volontairement absent.
- **M04.C0** — Une ligne de **journal des transformations** est une quintuple complète : date, fichier, > opération, règle, compteur avant → après (l'auteur faisant la sixième en production).
- **M04.C0** — Un **quasi-identifiant** est une combinaison de colonnes apparemment anonymes qui désigne une > personne dans une part mesurable des cas.
- **M04.C0** — Un **identifiant direct** est une colonne qui désigne une personne sans intermédiaire : le nom, > le numéro de téléphone, l'adresse e-mail.
- **M05.C0** — Une **transformation** est une opération qui modifie une colonne ou une table sans changer > la source.
- **M05.C0** — Une **clé candidate** est un jeu de colonnes dont les valeurs identifient une ligne et une > seule dans la table.
- **M05.C0** — Un **compteur d'échec** est un nombre qui dit combien de lignes ont été perdues (ou > marquées en erreur) par une conversion.
- **M05.C0** — Une **empreinte sha256** est une somme cryptographique de 256 bits qui identifie un > contenu au bit près.
- **M05.C0** — Une **requête paramétrée** est une requête qui dépend d'un ou plusieurs **paramètres** > (valeurs nommées, lues dans un fichier externe ou saisies par l'utilisateur).
- **M05.C0** — Le **Gestionnaire de paramètres** (Power Query) est l'interface qui liste tous les > paramètres d'un classeur.
- **M05.C0** — L'**étape de type** est l'étape Power Query qui assigne un type à chaque colonne — > c'est l'équivalent de `dtype=str` (pandas) ou de ``CAST(...`
- **M05.C0** — Le **verdict** d'une requête paramétrée est l'ensemble des mesures publiées en clés > `m05_*` pour le mois cible.
- **M05.C0** — Une **CTE** (`WITH ... AS ...`) est une sous-requête nommée, réutilisable dans la > requête principale.
- **M05.C0** — Le **EXCLUDE** est une clause DuckDB 1,5,5 (depuis la 0,9) qui permet de retirer > des colonnes du `SELECT` : `SELECT * EXCLUDE (rn, date_vente)` retire `rn` et `date_vente`.
- **M05.C0** — Le **LPAD** (left-pad) est la fonction SQL DuckDB qui pad une chaîne à gauche > avec un caractère donné : `LPAD('7', 2, '0')` rend `'07'`.
- **M05.C0** — Le **session variable** est une variable de session DuckDB, créée par `SET nom = > valeur`.
- **M05.C0** — Le **préambule controle_python.py** est le bloc de code exécuté en premier dans > chaque cellule du notebook de référence.
- **M05.C0** — Le `.loc[mask, "col"]` est l'opérateur pandas d'affectation par masque booléen : > `df.loc[mask, "col"] = nouvelle_valeur` modifie uniquement les lignes où `mask` est `True`.
- **M05.C0** — Le **SettingWithCopyWarning** est un avertissement pandas qui signale qu'une > modification peut ne pas être appliquée (parce qu'on travaille sur une vue du DataFrame, pas > une copie).
- **M05.C0** — Le **format Parquet** est un format de stockage tabulaire **typé** et **comprimé** > (5-10 × plus compact que CSV).

- **M05.C0** — La **grille de décision** (tableau du §5.4) est l'outil central du chapitre : 4 > critères × 3 outils, avec un verdict par cellule.
- **M05.C0** — Le **critère limitant** est le critère qui, seul, tranche le choix de l'outil.
- **M05.C0** — Le **scheduler** est l'outil qui exécute le pipeline à intervalles réguliers > (cron sous Linux, Planificateur de tâches sous Windows, Airflow pour les pipelines complexes).
- **M05.C0** — La **vitesse dépend de la taille** : un outil rapide sur 6 884 lignes peut être > lent sur 1 million, et inversement.
- **M06.C0** — La **base de données relationnelle** est un ensemble de **tables reliées par des > contraintes** (clés étrangères, types, valeurs, contrôles), persisté sur disque et requêtable par > SQL via un SGBD.
- **M06.C0** — Le **SGBD** (système de gestion de base de données) est le **moteur logiciel** qui > gère la base : stockage, indexation, verrouillage, sauvegarde, exécution des requêtes SQL.
- **M06.C0** — La **3FN** (3^e forme normale) est la condition qui dit : toutes les colonnes > non-clés dépendent de la clé, de toute la clé, rien que de la clé.
- **M06.C0** — La **clé étrangère** (foreign key, FK) est une **colonne** dont les valeurs sont > contraintes par une **référence** vers la clé primaire d'une autre table.
- **M06.C0** — Une **contrainte** est une **règle** que les données doivent suivre, **portée par > la base elle-même** (pas par le code applicatif).
- **M06.C0** — Le **type SQL** est la **famille** des valeurs qu'une colonne accepte.
- **M06.C0** — La valeur **NULL** représente l'**absence de valeur**.
- **M06.C0** — Le **DDL** (Data Definition Language) est le sous-ensemble du SQL qui crée, modifie, > et supprime la **structure** : CREATE TABLE, ALTER TABLE, DROP TABLE, CREATE INDEX, > CREATE VIEW.
- **M06.C0** — Une **clé étrangère** est une **contrainte** qu'on **déclare** dans le DDL et que la > base **fait respecter** à chaque insertion et à chaque suppression.
- **M06.C0** — Une **cardinalité** est le **nombre** de liens entre une ligne source et les lignes > cibles.
- **M06.C0** — Une **table de jointure** matérialise une relation N-M.
- **M06.C0** — Une ligne de la source correspond à 0..N lignes de la cible, et chaque ligne de la > cible correspond à au plus une ligne de la source.
- **M06.C0** — Une ligne de la source correspond à au plus une ligne de la cible, et inversement.
- **M06.C0** — Chaque ligne de la source peut correspondre à 0..N lignes de la cible, et inversement.
- **M06.C0** — Le **SGBD intégré** (in-process) est un moteur SQL qui vit dans le **même > processus** que le programme qui l'appelle.
- **M06.C0** — Le **SGBD serveur** (client-serveur) tourne sur une **machine distante** et les > clients s'y connectent par réseau.
- **M06.C0** — L'**import CSV** est le geste d'alimenter une table à partir d'un fichier plat.
- **M06.C0** — Un **SGBD serveur** (client-server DBMS) est un moteur qui tourne sur une > machine dédiée et écoute les connexions réseau sur un **port TCP** (par défaut 5432 pour > PostgreSQL).
- **M06.C0** — Le **NoSQL** (Not only SQL) est la famille des bases **non-relationnelles**, > optimisées pour des cas où le relationnel est maladroit : documents semi-structurés (MongoDB), paires > clé-valeur ultra-rapides (Redis), graphes de relations (Neo4j), colonnes larges distribuées > (Cassandra).
- **M06.C0** — Le **datawarehouse** (entrepôt) est un SGBD **analytique** où les données > validées sont stockées pour les rapports (tableaux de bord, SQL analytique, OLAP).

- **M06.C0** — Le **théorème CAP** (Brewer, 2000) énonce qu'un SGBD **distribué** (sur plusieurs machines) ne peut pas avoir simultanément **Cohérence**, **Disponibilité**, **Tolérance au partitionnement**.
- **M06.C0** — Le **modèle ACID** (Atomicity, Consistency, Isolation, Durability — atomique, cohérent, isolé, durable) est le contrat des transactions SGBDR : chaque transaction est garantie atomique (tout ou rien), cohérente (la base reste valide), isolée (les autres transactions ne voient pas l'état intermédiaire), durable (la modification survit à un crash).
- **M07.C0** — Une **requête SQL** — SQL query — est une instruction adressée au SGBD, qui retourne un résultat tabulaire (lignes × colonnes).
- **M07.C0** — Une **clause** est un morceau de la requête, qui commence par un mot-clé (`SELECT`, `FROM`, `WHERE`, ...) et se termine par une autre clause ou un point-virgule.
- **M07.C0** — Une **table** est un ensemble de lignes (enregistrements) et de colonnes (champs), en 3FN (cf.
- **M07.C0** — Un **alias** — alias — est un nom temporaire donné à une colonne ou à une table dans une requête.
- **M07.C0** — L'**ordre d'exécution** — logical execution order — est la séquence dans laquelle le SGBD traite les clauses d'une requête SQL, dans l'ordre : `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`.
- **M07.C0** — Une **console DuckDB** — DuckDB CLI — est l'interface en ligne de commande du moteur DuckDB.
- **M07.C0** — Le **filtre WHERE** — WHERE clause — est la clause SQL qui élimine les lignes qui ne satisfont pas une condition.
- **M07.C0** — Un **opérateur de comparaison** est un opérateur à 2 opérandes qui retourne `TRUE`, `FALSE` ou `NULL` : `=`, `<>`, `<`, `>`, `<=`, `>=`.
- **M07.C0** — `EXTRACT(part FROM date)` — extract — extrait une partie de la date : année, mois, jour, heure, minute, seconde, jour de la semaine, etc.
- **M07.C0** — La **logique à trois valeurs** — three-valued logic — est la logique du SQL qui admet trois valeurs de vérité : `TRUE`, `FALSE`, et `NULL` (l'inconnu).
- **M07.C0** — `ORDER BY` — trier — est la clause SQL qui range les lignes du résultat dans l'ordre spécifié.
- **M07.C0** — `NULLS FIRST` — nulls d'abord — place les valeurs `NULL` au début du tri ; `NULLS LAST` les place à la fin.
- **M07.C0** — La **pagination** — pagination — est la subdivision d'un résultat en pages de taille fixe.
- **M07.C0** — Le **tri multi-clés** — composite sort — est un tri sur plusieurs colonnes, appliqué dans l'ordre de citation.
- **M07.C0** — Une **fonction d'agrégat** — aggregate function — est une fonction qui prend un ensemble de valeurs (les lignes d'un groupe) en entrée et produit une **unique valeur** en sortie.
- **M07.C0** — Le **comptage distinct** — distinct count — compte les valeurs uniques d'une colonne au lieu de toutes les lignes.
- **M07.C0** — Le **panier moyen** — average order value (AOV) — est le montant moyen d'une vente, calculé par `AVG(montant_ttc)`.
- **M07.C0** — L'**ignorance des NULL** — null-skipping — est le comportement par défaut des agrégats `SUM`, `AVG`, `MIN`, `MAX` (et `COUNT(colonne)`) : les valeurs `NULL` ne participent ni à la somme, ni au dénombrement, ni à la comparaison.
- **M07.C0** — La clause `GROUP BY` — grouper par — subdivise les lignes du résultat en **groupes** selon les valeurs de la colonne (ou des colonnes) citées : les lignes qui partagent la même valeur forment un groupe, et chaque fonction d'agrégat du `SELECT` est calculée **séparément sur chaque groupe**.

- **M07.C0** — Une **clé de regroupement** — grouping key — est la colonne (ou la > liste de colonnes) citée dans `GROUP BY`.
- **M07.C0** — La clause **HAVING** — ayant — est le filtre qui s'applique aux > **groupes** après le calcul des agrégats.
- **M07.C0** — Le **comptage conditionnel** — conditional count — est le motif > `SUM(CASE WHEN condition THEN 1 ELSE 0 END)` : il convertit « compter les lignes qui > vérifient la condition » en expression agrégable.
- **M07.C0** — L'expression **CASE** — expression conditionnelle — est la brique de > logique conditionnelle du `SELECT`.
- **M07.C0** — Une **tranche** — bucket — est un intervalle de valeurs découpé par > des `CASE WHEN` successifs (ici < 50 000, < 200 000, sinon).
- **M07.C0** — La fonction **COALESCE** — coalescer — renvoie la **première** > valeur de sa liste d'arguments qui n'est pas `NULL`.
- **M07.C0** — La fonction **NULLIF** — nullifier si égal — renvoie `NULL` quand > ses deux arguments sont égaux, sinon le premier argument.
- **M07.C0** — Une **jointure** — join — est l'opération qui met côte à côte les > lignes de deux tables (ou plus) qui **s'accordent sur une clé**, via `ON` (par exemple > `v.id_client = c.id_client`).
- **M07.C0** — La **clé étrangère** — foreign key (FK) — est la colonne d'une table > qui pointe vers la clé primaire d'une autre table (`vente.id_client` → > `client.id_client`).
- **M07.C0** — Le **produit cartésien** — cross product — est l'ensemble de toutes les > paires (ligne du gauche × ligne du droit), sans condition : `|gauche| × |droit|` lignes.
- **M07.C0** — L'**explosion de jointure** — join fan-out — est le gonflement du > nombre de lignes quand une jointure accorde **plusieurs lignes d'un côté à une ligne de > l'autre** (relation `N → N`, ou jointure sur une colonne non unique).
- **M07.C0** — Une **sous-requête** — subquery — est une requête SQL **implantée à > l'intérieur** d'une autre requête, dans le `SELECT`, le `WHERE` ou le `FROM`.
- **M07.C0** — Une **sous-requête scalaire** — scalar subquery — est une sous-requête > qui doit renvoyer **exactement une valeur** (1 ligne, 1 colonne).
- **M07.C0** — Une **CTE** — Common Table Expression — est une sous-requête **nommée**, > déclarée en tête de requête par `WITH nom AS (SELECT ...)`, puis référencée comme une table > dans le reste de la requête.
- **M07.C0** — Une **vue** — view — est une requête SQL **enregistrée** dans le > schéma de la base, interrogeable comme une table (`SELECT ... FROM vue`).
- **M07.C0** — **EXISTS** teste s'il **existe** au moins une ligne dans sa > sous-requête (généralement **corrélée** : elle référence une colonne de la requête > extérieure).
- **M07.C0** — Le **pattern de double négation** — double negation — est l'écriture > `NOT EXISTS (SELECT ... WHERE NOT EXISTS (SELECT ...))` qui exprime « **tous les** » : on > nie l'existence d'un contre-exemple.
- **M08.C0** — **interpréteur** — interpreter — le programme qui lit le code Python et l'exécute > ligne à ligne, sans phase de « compilation » préalable visible : le fichier `.py` est lu, exécuté, > relu si vous le modifiez.
- **M08.C0** — **environnement virtuel** — virtual environment — une copie isolée de l'interpréteur > + ses paquets, limitée à un projet : les installations dans le `venv` n'affectent pas le Python système > ni les autres `venvs`.
- **M08.C0** — **exigence** — requirement — la ligne `paquet==version` d'un fichier > `requirements.txt` : la liste figée des paquets (et versions) qu'un projet a besoin, qui permet à > `pip install -r requirements.txt` de reconstruire **exactement** le même environnement sur une autre > machine.

- **M08.C0 — noyau** — kernel — le processus d'exécution derrière Jupyter : il exécute chaque > cellule demandée et **conserve l'état** (variables, imports) entre les exécutions.
- **M08.C0 — variable** — variable — un nom qui pointe vers une valeur : `prix = 1500` > colle l'étiquette « prix » sur le nombre 1 500 ; réaffecter (`prix = 2000`) re-colle l'étiquette, > sans copier quoi que ce soit.
- **M08.C0 — type** — type — la famille d'une valeur, qui autorise (ou interdit) les > opérations : `int` (entier), `float` (nombre à virgule), `str` (texte), `bool` (vrai/faux).
- **M08.C0 — f-string** — f-string — une chaîne précédée de `f` où chaque `{...}` est > remplacé par la valeur : `f"Total : {total:,.2f}"` affiche le total avec 2 décimales.
- **M08.C0 — traceback** — traceback — le rapport d'erreur de Python : il donne le chemin > parcouru (la pile d'appels, du haut vers le bas), la **ligne fautive** (le numéro et le texte), > la **flèche** qui souligne le coupable, et le **message** (le type d'erreur + une phrase).
- **M08.C0 — échelle de décision** — if/elif/else — une suite d'échelons testés **dans > l'ordre** : la première condition vraie exécutera son bloc, et le reste est sauté ; `else` > attrape ce que personne n'a pris.
- **M08.C0 — itération** — iteration — un tour de boucle : le corps de la `for` > s'exécute une fois par élément, dans l'ordre, sans sauter ni revenir.
- **M08.C0 — boucle infinie** — infinite loop — une boucle dont la condition ne devient > jamais fausse (ou qu'aucun `break` ne coupe) : la machine se répète sans fin.
- **M08.C0 — compréhension** — list comprehension — `[expr for x in it if cond]` : une > liste construite en **une** ligne, lisible de gauche à droite comme une phrase.
- **M08.C0 — séquence** — sequence — un contenant ordonné, lisible par index : > liste, tuple, chaîne.
- **M08.C0 — immuable** — immutable — un objet qu'on ne peut pas modifier en place > (tuple, chaîne, nombre) : pour « changer » une valeur, on **construit** un nouvel objet > (`surface + (210,)`, pas `surface[5] = 210`).
- **M08.C0 — méthode de chaîne** — string method — un verbe attaché à une chaîne > (`s.strip()`, `s.split(" ")`) : elle **renvoie une nouvelle** chaîne sans toucher à > l'originale (la chaîne est **immuable**, §5.2) — d'où l'enchaînement `s.strip().replace(...)`.
- **M08.C0 — clé** — key — l'étiquette d'une case de dictionnaire : `d[clé]` > lit, `d[clé] = valeur` écrit, `clé in d` teste.
- **M08.C0 — valeur unique** — unique value — une valeur qui ne compte qu'une > fois : `set(villes)` passe la liste au tamis (5 éléments → 3), `len(set(...))` compte les > uniques.
- **M08.C0 — tranche** — slice — `l[a:b]` : les éléments de l'index **a** à > l'index **b - 1** (la fin **est exclue**) ; `l[:2]` tous les deux ; `l[::-1]` l'inverse.
- **M08.C0 — fonction** — function — un bloc nommé de code, appelé par son nom > avec des arguments, qui **renvoie une valeur** avec `return`.
- **M08.C0 — argument par défaut** — default argument — la valeur prise quand > l'appel ne précise pas l'argument : `calculer_remise(1500)` prend `taux=0.10`.
- **M08.C0 — portée** — scope — la zone de visibilité d'une variable : **locale** > (née et morte dans la fonction), **globale** (le fichier tout entier).
- **M08.C0 — docstring** — docstring — la chaîne en **tête** de fonction, entre > triples guillemets, lue par `help(f)` et récupérable par `f.__doc__`.
- **M08.C0 — exception** — exception — une erreur **levée** à l'endroit où le > problème arrive, qui remonte la pile jusqu'au premier `except` qui la **nomme**.
- **M08.C0 — tableau** — ndarray — un bloc de mémoire **contigu** de valeurs > **homogènes** : un seul dtype pour tout le tableau, une forme (`shape`) qui dit > combien de valeurs.
- **M08.C0 — vecteurisation** — vectorization — l'application d'une opération > à **tout** un tableau d'un seul geste, sans boucle Python : `montants * 1.19` > multiplie les 50 008 valeurs, `np.sum(montants)` les additionne.

- **M08.C0 — nan** — nan (« not a number ») — la valeur **absente** du > monde des nombres : elle **se propage** (`nan + 1 = nan` — l'absence contagieuse), > elle **contamine** les agrégats (`np.sum` renvoie `nan`), et elle se **détecte** > (`np.isnan`) ou se **contourne** (`np.nansum` = 3500,0 : la somme regarde autour > du trou).
- **M08.C0 — diffusion** — broadcasting — la règle de forme de NumPy : deux > tableaux peuvent s'opérer s'ils sont **de même forme**, ou si l'un est un > **scalaire**, ou si leurs formes sont **compatibles** (du plus court au plus > long, dimensions égales ou 1).
- **M08.C0 — tableau de données** — DataFrame — un tableau à deux > dimensions de **colonnes nommées** (une table SQL en Python), l'objet central > de pandas : `shape` donne lignes × colonnes, `columns` les noms, `dtypes` la > famille de chaque colonne.
- **M08.C0 — série** — Series — une **colonne** : un index (les > étiquettes de lignes) + des valeurs.
- **M08.C0 — index** — index — l'étiquette de chaque ligne (par défaut > 0, 1, 2, ...) : la poignée par laquelle `loc` et `iloc` attrapent les lignes.
- **M08.C0 — masque** — mask — le résultat d'une comparaison sur une > colonne : une Série de True/False **par ligne** (28 065 True sur > `montant_ttc > 100000`), posée dans les crochets du DataFrame pour ne garder > que les lignes vraies : `v[masque]` est le `WHERE` de M07.
- **M08.C0 — transformation** — transform — l'opération qui calcule > une valeur **par groupe** et la **repose sur chaque ligne** du groupe : la > table de départ garde sa forme (50 008 × 14), chaque ligne portant l'agrégat > de son groupe.
- **M08.C0 — jointure** — merge — l'alignement de **deux DataFrames** > sur une **clé commune explicite** (`on="id_magasin"`), avec le mode de > conservat des lignes (`how=`) : le `JOIN` de M07, où la clé est une > information **métier** et non un détail de syntaxe.
- **M08.C0 — table pivotée** — `pivot_table` — la réorganisation > **lignes ↔ colonnes** d'une table autour d'une clé (ici magasin × mois) et > d'une valeur agrégée (le CA) : la vue « tableau croisé » de M07, en un > geste pandas.
- **M08.C0 — addition en centimes** — cent-based summation — la > conversion des montants en **centimes entiers** > (`((colonne * 100).round().astype("int64"))`) avant toute somme : le centime > est le plus petit multiple **exact** de FCFA, l'entier ne dérive pas, et le > total final se convertit en FCFA par une seule division (`// 100`).
- **M09.C0 — analyse exploratoire de données** — exploratory data analysis (EDA) — l'examen > systématique d'un jeu de données inconnu pour en dégager des structures, des problèmes et des > pistes, **avant** de modéliser.
- **M09.C0 — audit** — le passage du protocole qui cherche les défauts du jeu (doublons, > manquants, valeurs impossibles) avant de mesurer quoi que ce soit.
- **M09.C0 — fiche d'entrée** — le document de cadrage d'une analyse : variables (sens, > unité, source), questions autorisées, durée prévue.
- **M09.C0 — conclusion provisoire** — une affirmation posée **avec** ses limites > (période, échantillon, variables absentes), révisable aux données suivantes.
- **M09.C0 — borne métier** — une limite dictée par le métier, pas par la > statistique : un stock ≥ 0 , une note entre 0 et 20, une date dans la période de > l'export.
- **M09.C0 — iqr** — interquartile range — Q3 – Q1 : la largeur du « cœur » de la > distribution (le milieu des 50 %), indépendante des extrêmes.
- **M09.C0 — manquant** — une valeur absente, sous deux formes : le `NaN` que > `isna()` compte, et la **chaîne vide** codée en clair que `isna()` laisse passer.
- **M09.C0 — point aberrant** — outlier — une observation au-delà d'une borne > définie **avant** de chercher : `Q3 + 1,5 × iqr` (Tukey), une borne métier (stock ≥ 0 , note > ≤ 20), ou la comparaison à la période.
- **M09.C0 — tendance** — la pente de fond de la série : ce qui reste quand on a > retiré les cycles.

- **M09.C0 — moyenne glissante** — moving average — la moyenne calculée sur une > **fenêtre glissante** (ici 3 mois) : chaque point résume sa fenêtre, les vagues > s'effacent, le lit et les crues restent.
- **M09.C0 — bruit** — la variabilité **sans cycle** : ce que la moyenne glissante > lisse.
- **M09.C0 — saisonnalité** — saisonnalité — la variation **régulière et > prévisible** liée au cycle (saison, mois, jour de semaine, heure) : elle revient **au > même endroit du cycle**, c'est ce qui la rend exploitable pour la prévision.
- **M09.C0 — anomalie** — anomalie — une observation qui s'écarte d'une borne > **définie avant** de chercher : iqr (position dans la distribution), borne métier > (limite du domaine) ou période (écart au cycle attendu).
- **M09.C0 — récurrence** (du flux) — la règle de continuité d'une série de > stock : hier + entrées - sorties = aujourd'hui.
- **M09.C0 — soupçon** — le stade d'une anomalie **non prouvée** : un signal > (position, borne, période) sans aucun des 3 témoins.
- **M09.C0 — anomalie structurelle** — un défaut qui révèle un **mécanisme** > (export, saisie, génération) plutôt qu'une erreur ponctuelle : les 8 doublons du > fil rouge (clé métier redite, id_vente distinct), les 4 stocks négatifs déductibles.
- **M09.C0 — corrélation** — corrélation — la mesure d'association entre deux > variables ; Pearson mesure le lien **linéaire**, de -1 (opposé) à +1 (proportionnel), > 0 = pas de lien linéaire.
- **M09.C0 — piège d'inférence** — le saut interdit de l'**association** à la > **cause**, sous ses 3 formes mesurées : variable **tierce** (la troisième variable > qui cause les deux — contrôlée, la corrélation résiduelle est le lien direct), > **régression vers la moyenne** (les extrêmes retournent vers le milieu sans cause), > **sens inversé** (B cause A, on affirme A cause B).
- **M09.C0 — segment** — un groupe défini **avant** de regarder les résultats : > les bornes sont **métier** (< 2, 2 à 5, ≥ 6 jours d'absence), pas statistiques, et > s'écrivent dans le rapport **avant** les chiffres.
- **M09.C0 — variable tierce** — confond — la troisième variable qui **cause > les deux** variables observées : ici, le niveau d'origine (proxy : la note du T1) > pousse l'absence **et** la note du T2.
- **M09.C0 — tableau croisé** — tableau de contingence — la fréquence (ou la > moyenne) d'un **couple de modalités** : matière × trimestre ici, 5 × 3 cases.
- **M09.C0 — légende** — le bloc texte du graphique : la **question** posée > (dans le titre), la **source** (la sortie exécutée), la **taille d'échantillon** (n) > et **l'axe** quand il ne part pas de zéro.
- **M09.C0 — graphique** — chart — la traduction visuelle d'une sortie vers un > lecteur.
- **M09.C0 — biais de sélection** — biais d'échantillonnage — l'écart entre > la population **étudiée** (le fichier) et la population **visée** (le métier) : les > 1 200 clients du fil rouge ne sont pas **tous** les clients du groupe, les 400 > élèves ne sont pas **tous** les élèves de l'établissement.
- **M09.C0 — note d'EDA** — le livrable de **fin** d'exploration : 1 page par > jeu, où chaque affirmation porte sa **source** (la sortie exécutée, **re-jouable**) > et sa **limite** (ce qu'elle **ne** dit pas, **remplie**).
- **M10.C0 — Le budget de lecture** est le temps qu'un lecteur accorde réellement à un > graphique dans un dossier ou une réunion : de l'ordre de quelques secondes.
- **M10.C0 — Un graphique est un argument visuel** : il affirme une chose (« ces huit parts se > ressemblent », « cette courbe se retourne ») et son lecteur la croit sans calculer.
- **M10.C0 — L'ordre perceptuel** est le classement des tâches de lecture par précision > moyenne.
- **M10.C0 — On appelle facteur de conversion** d'un canal la quantité de ce canal qui > correspond à un point de pourcentage de données : 8,5 mm sur un axe large comme la page,

3,6° sur > un camembert, une variation relative pour une aire, une différence de clarté L* pour une couleur.

- **M10.C0** — La **grille de conception** n'est pas une grille d'esthétique : chaque point est une > **question fermée** à laquelle on répond par oui ou non, en montrant la preuve (« axe : zéro, oui », > « ordre : trié par chiffre d'affaires décroissant, oui »).
- **M10.C0** — L'**arbre de choix** est une procédure en cinq étapes, parcourue **avant** > d'ouvrir l'outil : (1) écrire la question, (2) nommer le symptôme, (3) choisir la famille, > (4) choisir le graphique précis, (5) choisir l'outil et le support.
- **M10.C0** — On appelle **corrélation affichée** la valeur du coefficient de corrélation entre > deux séries **telles que le graphique les présente** — c'est-à-dire après le choix des deux > échelles.
- **M10.C0** — On appelle **graphique-tableau** (ou tableau enrichi) un tableau dont les cellules > portent un codage visuel : barres intégrées, échelle de couleurs, flèches de variation.
- **M10.C0** — Une **mesure** est un nombre calculé sur des données (« 7 876 320 164 FCFA de > CA net en 24 mois »).
- **M10.C0** — La **palette de la maison** (ou palette de référence) est le jeu de couleurs figé > par la charte, avec un rôle par couleur.
- **M10.C0** — Une palette est **appariée** quand sa famille (catégorielle, séquentielle, > divergente) correspond à la structure de la variable affichée.
- **M10.C0** — La **lisibilité en niveaux de gris** est la propriété d'un graphique qui reste > entièrement compréhensible une fois la couleur retirée : chaque série garde un signe distinctif > (libellé, forme, position).
- **M10.C0** — On appelle **précision affichée** le nombre de chiffres significatifs décidés à > l'affichage, par opposition à la précision du calcul.
- **M10.C0** — Un **titre qui affirme** énonce un constat que les données soutiennent, en une > phrase citable telle quelle.
- **M10.C0** — Un axe est **honnête** quand le lecteur connaît ses bornes sans les chercher.
- **M10.C0** — On appelle **hauteur occupée** la part de la hauteur d'axe réellement couverte > par les données (11,8 % pour notre chiffre d'affaires sur un axe à zéro, 82,4 % sur l'axe tronqué).
- **M10.C0** — La **hiérarchie visuelle** est l'ordre de lecture imposé à l'œil : titre, chiffre > principal, appuis, graphique, source.
- **M10.C0** — L'**annotation** est le texte qui désigne un point de la figure et donne sa > valeur.
- **M10.C0** — L'**échelle logarithmique** place les graduations sur les puissances d'un facteur > ($\times 10$ en général) : chaque décade occupe la même hauteur.
- **M10.C0** — La **relecture en 60 secondes** est un rituel, pas un talent : six questions, un > minuteur, et la règle que tout défaut trouvé s'écrit soit dans la figure (déclaration), soit dans la > liste des corrections (refonte).
- **M10.C0** — Le **diagnostic mesuré** est la première étape notée du projet « La refonte » : il > nomme les défauts d'un rapport, **mesure** leur effet sur les données, et écrit la décision fautive > que chacun peut déclencher.
- **M10.C0** — On appelle **faux citable** la phrase fautive qu'un lecteur pourrait prononcer devant > une figure exacte (« la volatilité explose », « les retours suivent le CA »).
- **M10.C0** — L'**appel à la décision** est la phrase, écrite sur le troisième écran, qui nomme la > décision demandée, son destinataire, son montant et son indicateur de suivi.
- **M10.C0** — La **densité d'un écran** est le nombre de chiffres, de lignes et de figures qu'il > porte.
- **M10.C0** — On appelle **restitution** l'ensemble formé par la séquence et ses annexes.
- **M10.C0** — La **couverture des objections** est le rapport entre le nombre d'objections > anticipées et le nombre d'objections auxquelles la restitution répond explicitement.

- **M10.C0** — On appelle **contrat d'outil** l'ensemble des trois questions auxquelles un outil > répond dans une chaîne de production : qui le modifie ?, à quelle fréquence ?, sur quel support > arrive-t-il ? Un outil choisi sans ces trois réponses produit des fichiers que personne ne > réutilise — et un graphique qu'on refera trois fois.
- **M10.C0** — Un outil est dit **cité** (ou « non exécuté ») quand sa procédure est décrite sans > avoir été mise en œuvre dans l'atelier.
- **M10.C0** — La **résolution utile** d'une figure est le nombre de pixels disponibles par unité > de donnée : **25** px par mois en 96 dpi, **79** px par mois en 300 dpi, pour le même graphique de > **24** mois.
- **M10.C0** — Un **export** est la conversion d'une figure vers un fichier destiné à un usage > précis (écran, impression, retouche, diffusion).
- **M11.C0** — Une **fonction de fenêtre** — window function — est un calcul qui prend en entrée un > **ensemble de lignes** (la fenêtre) et rend **une valeur par ligne**.
- **M11.C0** — La **partition** — partition — est la subdivision de la fenêtre en groupes **indépendants** > **les uns des autres**.
- **M11.C0** — Le **cadre de fenêtre** — window frame — précise **quelles lignes** autour de la ligne > courante la fonction voit : ROWS BETWEEN 2 PRECEDING AND CURRENT ROW (les deux précédentes et la > courante), ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW (depuis le début), ROWS BETWEEN UNBOUNDED > PRECEDING AND UNBOUNDED FOLLOWING (toute la partition).
- **M11.C0** — Une **sous-requête scalaire** — scalar subquery — est une sous-requête placée dans une > expression et qui rend **une seule valeur** (une ligne, une colonne) : SUM(x) / (SELECT SUM(y) FROM t).
- **M11.C0** — Les trois fonctions de rang répondent à trois questions différentes sur la même mesure : > « **donne-moi un numéro unique** » (ROW_NUMBER), « **quelle place mérite-t-elle ?** » (RANK), > « **combien de niveaux de performance y a-t-il jusqu'ici ?** » (DENSE_RANK).
- **M11.C0** — Une **clé de départage** — tie-breaker — est une colonne ajoutée après la mesure dans > l'ORDER BY d'une fenêtre (presque toujours la clé primaire de la table) pour que deux exécutions du même > code produisent **le même** résultat.
- **M11.C0** — Un **quantile** — quantile — est une frontière qui coupe une population ordonnée en > parties de taille comparable : les tiers (NTILE(3)), les déciles (NTILE(10)), les centiles > (NTILE(100)).
- **M11.C0** — Un **trou de série** — series gap — est une ligne dont le décalage n'a pas de > pré-décesseur : le premier mois d'une série, ou le premier mois après un filtre trop zélé.
- **M11.C0** — Une **table calendrier** — date dimension — est la table de référence du temps : une > ligne par jour (ici **1 339**), avec ses attributs calculés une fois pour toutes (année, mois, trimestre, > libellé du jour, indicateur de férié).
- **M11.C0** — Le **périmètre** — scope — d'une comparaison est l'ensemble des lignes effectivement > comparables : mêmes mois, même nombre de jours, mêmes magasins, mêmes exclusions.
- **M11.C0** — Un **trou de série** — series gap — est une période de l'axe de temps pour laquelle > il n'existe **aucune** donnée : soit parce que rien ne s'est produit (un dimanche sans vente), soit parce > que la mesure n'a pas été faite (un mois sans objectif).
- **M11.C0** — Le **cadre de fenêtre** — window frame — est la partie de la partition réellement prise > en compte autour de la ligne courante.
- **M11.C0** — Une **cohorte** est un ensemble de clients réunis par un **événement daté** commun — et > l'événement doit être choisi avec soin.
- **M11.C0** — La **rétenion** est la part d'une cohorte encore active après N périodes.
- **M11.C0** — La **pondération** consiste à donner à chaque cohorte un poids proportionnel à son > effectif.

- **M11.C0** — Le **RFM** n'a pas de frontières intrinsèques : les coupures en trois blocs égaux sont un > choix de méthode (égalité des effectifs), pas une vérité métier.
- **M11.C0** — Une **table d'expression commune** — CTE — est un bloc `WITH nom AS (requête)` placé > avant la requête principale.
- **M11.C0** — Le **repliement d'un libellé** consiste à ramener plusieurs graphies d'une même valeur à > une seule écriture.
- **M11.C0** — Un **récapitulatif multi-niveaux** est une agrégation qui produit, dans **une** requête, > le détail et les sous-totaux.
- **M11.C0** — Le **grain** d'un jeu de lignes est ce que représente **une** ligne.
- **M11.C0** — Le **plan d'exécution** est la description, par le moteur, de ce qu'il va faire : les > opérateurs, leur ordre d'enchaînement et les colonnes lues.
- **M11.C0** — Une **vue** ne stocke pas de données : c'est une requête à laquelle on a donné un nom.
- **M11.C0** — Un **index** est une structure de recherche par valeur, qui évite de parcourir toutes les > lignes.
- **M11.C0** — Le **coût du SELECT *** est le travail de lecture des colonnes qui ne serviront pas à la > sortie.
- **M11.C0** — Un **test de non-régression** est un contrôle qui rejoue une requête **publiée** et > compare son résultat à une **valeur attendue**.
- **M11.C0** — Un **contrôle croisé** vérifie une même grandeur par deux chemins **indépendants** : > deux moteurs (DuckDB et SQLite), deux outils (SQL et pandas), ou deux socles (le socle M11 et celui de > M09).
- **M11.C0** — Le **code de sortie** d'un contrôle est le signal qu'il rend au système : zéro si tous les > contrôles passent, non nul sinon.
- **M11.C0** — La **valeur attendue** d'un test est la valeur **publiée** — ici, celle du fichier > `chiffres_cites.json` — et non la valeur qu'on vient de lire à l'écran.
- **M12.C0** — La **Business Intelligence** (BI) est l'ensemble organisé des moyens — données, > définitions, modèles, outils, responsabilités — qui permet à une organisation de **décider avec des > chiffres partagés** : les mêmes définitions pour tous, disponibles au bon moment, avec une action > décidée d'avance quand ils passent un seuil.
- **M12.C0** — Un **indicateur clé** — KPI, Key Performance Indicators — est une mesure > **choisie** pour suivre une performance et **déclencher une décision**.
- **M12.C0** — Une **décision fondée sur les données** — data-driven — est prise en s'appuyant > sur des mesures définies **avant** la décision, et vérifiables **après**.
- **M12.C0** — La **couche sémantique** — semantic layer — est l'endroit — table, vue, modèle — > où les définitions d'indicateurs sont **écrites une fois** pour toute l'organisation.
- **M12.C0** — Le **reporting** est la production périodique de résultats standardisés, à > définition figée, destinés à être **lus** — tableau de bord mensuel, état des ventes, balance âgée.
- **M12.C0** — La **BI** (Business Intelligence) est l'ensemble des moyens qui permettent > **d'explorer** les données pour répondre à des questions non prévues à l'avance, gouvernées par des > définitions partagées.
- **M12.C0** — Un **entrepôt de données** — data warehouse — est un stockage structuré, > historisé et modélisé (M13), alimenté par les sources opérationnelles, dans lequel la BI vient lire.
- **M12.C0** — Le **service en autonomie** — self-service — désigne le fait de laisser un utilisateur métier construire ses > propres vues à partir d'une couche sémantique gouvernée.
- **M12.C0** — Les **quatre régimes de question** sont le descriptif (« que s'est-il passé ? »), le > **diagnostique** (« pourquoi ? »), le **prédictif** (« que se passera-t-il ? ») et le **prescriptif** > (« que faire ? »).

- **M12.C0** — Un **fait** est un événement mesurable et daté — une vente, une commande, une > rupture, un encaissement.
- **M12.C0** — Une **dimension** est un axe d'analyse stable, par lequel on découpe les faits : > temps, magasin, client, produit, canal, mode de paiement.
- **M12.C0** — Une **métrique** est une quantité mesurée sur les faits : un montant, une quantité, > une durée, un effectif.
- **M12.C0** — Le **grain** d'une table ou d'un résultat est ce que représente **une ligne** : > une ligne de vente, un ticket, un couple produit-mois, une commande, une journée.
- **M12.C0** — Une mesure est **additive** quand elle peut être additionnée sur toutes les > dimensions (les montants), **semi-additive** quand elle s'additionne sur certaines seulement > (un stock, qui s'additionne dans l'espace mais pas dans le temps), et **non additive** quand elle ne > s'additionne nulle part (un taux, une moyenne, un comptage d'éléments distincts).
- **M12.C0** — Un **indicateur clé** — **KPI**, Key Performance Indicators — est une métrique > **choisie** pour suivre une performance qui compte, dotée d'une carte de définition complète et d'un > responsable nommé.
- **M12.C0** — Une **carte de définition** est la fiche d'identité d'un indicateur : formule, source, > granularité, fréquence, responsable, seuil d'alerte, contre-KPI.
- **M12.C0** — Un **seuil d'alerte** est la valeur à partir de laquelle l'indicateur déclenche une > **action** écrite.
- **M12.C0** — Un **contre-KPI** — indicateur de contrepartie — est l'indicateur qui mesure ce > qu'on **dégrade** en améliorant le premier.
- **M12.C0** — Un **effet pervers** est une conséquence non voulue d'un indicateur devenu objectif : > les personnes pilotées par la mesure agissent sur ce qu'elle mesure — pas sur ce qu'elle devait > mesurer.
- **M12.C0** — Une **chaîne de calcul** — pipeline — est la suite d'étapes qui transforme les > sources brutes en indicateurs publiés : extraction, nettoyage, stockage, modélisation, définition, > visualisation, diffusion.
- **M12.C0** — Une **source de vérité** est l'endroit unique où une information est tenue à jour pour > un usage donné.
- **M12.C0** — La **gouvernance des données** désigne les règles explicites qui décident qui crée, > qui modifie, qui valide et qui publie une donnée ou un indicateur.
- **M12.C0** — L'**adoption** est l'usage réel du dispositif par ses destinataires : nombre de > lecteurs réguliers, fréquence d'ouverture, décisions citant le dispositif.
- **M12.C0** — La **diffusion** est le dernier étage : le choix du support, de la fréquence et du > moment où l'indicateur atteint son lecteur — courriel, réunion, écran partagé — avec la personne qui > répond du rendez-vous.
- **M12.C0** — Un **cahier des charges** est l'écrit qui fixe, avant le début des travaux, les > décisions à éclairer, les destinataires, la fréquence, les définitions attendues, les critères de > réception et le budget de maintenance.
- **M12.C0** — Le **commanditaire** — sponsor — est la personne qui finance le dispositif, le > défend dans l'organisation et arbitre les désaccords.
- **M12.C0** — Une **partie prenante** est toute personne qui influe sur le projet ou le subit : > destinataires, producteurs de données, informaticiens, comptables, contrôleurs.
- **M12.C0** — La **recette** — réception — est l'étape où le commanditaire vérifie le dispositif > contre les critères du cahier des charges : chaque indicateur existe, sa valeur est reproductible, sa > définition est publiée, son responsable est nommé, son seuil déclenche une action.
- **M12.C0** — La **maintenance** couvre deux activités distinctes : la **maintenance corrective**, qui > répare (**1 200 000** FCFA par an dans le dossier raté, soit **23 077** FCFA par semaine), et la > **maintenance évolutive**, qui retire ce qui n'est plus lu et ajoute ce qui manque.

4. Index des planches

Chaque planche est citée par le chapitre qui l’affiche. Toutes sont en SVG, sous 776 px, avec du texte réel : elles se lisent à l’écran, s’impriment et se régénèrent.

Planche	Chapitre
M01_C01_chemins.svg	M01.C0
M01_C02_chaine.svg	M01.C0
M01_C03_table.svg	M01.C0
M01_C03_table.svg	M01.C0
M01_C04_types.svg	M01.C0
M01_C05_classer.svg	M01.C0
M01_C06_cycle.svg	M01.C0
M01_C07_metiers.svg	M01.C0
M02_C04_boxplot.svg	M02.C0
M02_C05_histogramme.svg	M02.C0
M02_C07_distribution_echantillonnage.svg	M02.C0
M02_C08_foret_comparaisons.svg	M02.C0
M03_C01_fenetre_excel.svg	M03.C0
M03_C02_formats_saisie.svg	M03.C0
M03_C03_structure_tableur.svg	M03.C0
M03_C04_adresses_erreurs.svg	M03.C0
M03_C05_agregats_conditionnels.svg	M03.C0
M03_C06_recherches_assemblage.svg	M03.C0
M03_C07_dates_periodes.svg	M03.C0
M03_C08_tcd_et_modele.svg	M03.C0
M03_C08_chemin_donnees.svg	M03.C0
M04_C01_diagnostic_en_douze_points.svg	M04.C0
M04_C02_cinq_mecanismes_cinq_traitements.svg	M04.C0
M04_C03_quatre_niveaux_de_doublon.svg	M04.C0
M04_C04_quatre_familles_de_regles.svg	M04.C0
M04_C05_trois_niveaux_deux_preuves.svg	M04.C0
M04_C06_boite_a_donnees.svg	M04.C0
M08_C01_quatre_couches_environnement_python.svg	M08.C0
M08_C02_types_operations_matrice.svg	M08.C0
M08_C03_decider_repetir_flux.svg	M08.C0
M08_C04_cinq_conteneurs.svg	M08.C0
M08_C05_lire_traceback.svg	M08.C0
M08_C06_boucle_vs_vecteur.svg	M08.C0
M08_C07_loc_vs_iloc_4_cas.svg	M08.C0
M08_C08_groupby_sql_pandas.svg	M08.C0
M09_C01_protocole_dix_etapes_3_jeux.svg	M09.C0
M09_C02_audit_matrice_3_familles.svg	M09.C0

Planche	Chapitre
M09_C03_tendance_saisonnalité_bruit.svg	M09.C0
M09_C04_boite_a_outils_preuve_4_cas.svg	M09.C0
M09_C05_trois_fausses_causes_chiffrees.svg	M09.C0
M09_C06_table_question_graphique.svg	M09.C0
M10_C01_cinq_encodages.svg	M10.C0
M10_C02_seize_graphiques.svg	M10.C0
M10_C03_palette_testee.svg	M10.C0
M10_C04_ecriture_titre_axe_annotation.svg	M10.C0
M10_C05_mur_des_10_erreurs.svg	M10.C0
M10_C06_sequence_trois_echans.svg	M10.C0
M10_C07_meme_graphique_trois_outils.svg	M10.C0
M11_C01_carte_des_fenetres.svg	M11.C0
M11_C04_matrice_retention.svg	M11.C0
M12_C05_carte_des_sept_etages.svg	M12.C0
M12_C06_matrice_des_dix_couples.svg	M12.C0

5. Index des instruments

Les scripts qui font tourner le manuel : tous s'exécutent sur le socle livré, sur un poste ordinaire, sans service payant.

Instrument	Ce qu'il fait
tools/budget_pages.py	Budget de pages du manuel : le recalculer, le mesurer, ne jamais le laisser « à l'œil ».
tools/classeur_M03.py	classeur_M03.py — fabrique le classeur d'atelier du module M03 (Excel).
tools/controle_exos_M07.py	Auto-validation des 35 exercices de l'évaluation M07 (épreuve C).
tools/controle_exos_M08.py	Auto-validation des 15 exercices de l'évaluation M08 (épreuve C).
tools/controle_exos_M09.py	Auto-validation des 15 exercices de l'évaluation M09 (épreuve C).
tools/controle_formules.py	controle_formules.py — Q3 de la porte §G.1, côté tableur.
tools/controle_pdf.py	Contrôle final du PDF composé d'un module — le seul contrôle qui lit le fichier rendu.
tools/controle_python.py	Contrôle Q3 (porte §G.1) côté Python : les blocs ``python publiés sont exécutés.
tools/controle_refonte_M10.py	controle_refonte_M10.py — le controle automatique du projet M10.P (« La refonte »).
tools/controle_sql.py	Contrôle Q3 de la porte de module (§G.1) : tout le SQL publié est exécuté.
tools/controle_sql_M11.py	controle_sql_M11.py — les tests de non-regression du module M11 (chapitre C07).
tools/couleurs_M10.py	couleurs_M10.py — la mesure du chapitre C03 : contraste, daltonisme, palettes.
tools/dossier_M04.py	dossier_M04.py — le « dossier pourri » du projet M04.P, généré de façon déterministe.
tools/dossier_M05.py	dossier_M05.py — le dossier du projet M05.P « Trois chemins, une même table propre ».
tools/dossier_M06.py	dossier_M06.py — le dossier du projet M06.P « La base de la quincaillerie ».
tools/dossier_M07.py	dossier_M07.py — le dossier du projet M07.P « La base commerciale d'une chaîne de 5 magasins ».

Instrument	Ce qu'il fait
<code>tools/dossier_M08.py</code>	
<code>tools/dossier_M09.py</code>	
<code>tools/dossier_M10.py</code>	dossier_M10.py — le socle du module M10 (Data visualization).
<code>tools/dossier_M11.py</code>	dossier_M11.py — construit le socle SQL du module M11 (SQL avance pour la BI).
<code>tools/dossier_M12.py</code>	dossier_M12.py — construit le socle du module M12 (Fondamentaux de la Business Intelligence).
<code>tools/dossier_M13.py</code>	dossier_M13.py — construit le socle du module M13 (Modelisation des donnees).
<code>tools/ecriture_M10.py</code>	ecriture_M10.py — la mesure du chapitre C04 : titres, axes, annotations, ordre.
<code>tools/erreurs_M10.py</code>	erreurs_M10.py — le mur des 10 erreurs du chapitre C05, chacune mesurée sur le socle.
<code>tools/figures_M01.py</code>	Figures SVG du module M01. Style unique, texte en DejaVu Sans (police système du PDF).
<code>tools/figures_M02.py</code>	Figures SVG du module M02. Style identique à <code>tools/figures_M01.py</code> , texte en DejaVu Sans.
<code>tools/figures_M03.py</code>	figures_M03.py — les planches SVG du module M03 (Excel).
<code>tools/figures_M04.py</code>	figures_M04.py — les planches SVG du module M04 (qualité des données).
<code>tools/figures_M05.py</code>	figures_M05.py — les planches SVG du module M05 (langage commun des transformations).
<code>tools/figures_M06.py</code>	figures_M06.py — les planches SVG du module M06 (bases de données relationnelles).
<code>tools/figures_M07.py</code>	figures_M07.py — les planches SVG du module M07 (SQL : interroger, agreger, rejoindre).
<code>tools/figures_M08.py</code>	figures_M08.py — les planches SVG du module M08 (Python pour l'analyse de donnees).
<code>tools/figures_M09.py</code>	figures_M09.py — les planches SVG du module M09 (Analyse exploratoire de données).
<code>tools/figures_M10.py</code>	figures_M10.py — la recette des 7 planches du module M10 (Data visualization).
<code>tools/figures_M11.py</code>	figures_M11.py — les 2 planches du module M11 (SQL avance pour la BI).
<code>tools/figures_M12.py</code>	figures_M12.py — les 2 planches du module M12 (fondamentaux de la BI).
<code>tools/kit_exploitation.py</code>	kit_exploitation.py — le kit d'exploitation du manuel (M01 à M12 a ce jour).
<code>tools/kpi_M12.py</code>	kpi_M12.py — les 10 indicateurs du module M12, mesurés sur le socle.
<code>tools/modele_M13.py</code>	modele_M13.py — l'instrument du module M13 : le modele de Sahel Distribution, controle.
<code>tools/nb_pages.py</code>	Compte les pages d'un PDF produit par WeasyPrint.
<code>tools/outils_M10.py</code>	outils_M10.py — le meme graphique dans trois outils (C07) : matplotlib, Excel, Power BI.
<code>tools/perception_M10.py</code>	perception_M10.py — la mesure de l'etape 1 de C01 : le meme jeu de valeurs code
<code>tools/perf_M11.py</code>	perf_M11.py — l'instrument de mesure du chapitre C06 (requêtes lentes et plans).
<code>tools/poids.py</code>	poids.py — garde-fou de quota de l'atelier (128 Mo persistés).
<code>tools/render.py</code>	
<code>tools/seize_graphiques_M10.py</code>	seize_graphiques_M10.py — le catalogue des 16 graphiques du manuel, exécuté sur le socle.
<code>tools/sequence_M10.py</code>	sequence_M10.py — la sequence de 3 ecrans du chapitre C06, en deux versions (direction, equipe).
<code>tools/pousser.sh</code>	publier le dépôt : contrôle d'abord, poussée ensuite

6. Index des jeux de données

Dossier	Contenu
03_exercices/ dossier_M04/	4 fichiers : ATTENDU.json, clients_2026.csv, commandes_2026.csv, tarif_fournisseur.csv
03_exercices/ dossier_M05/	4 fichiers : ATTENDU.json, clients.csv, remises_2023_2024.xlsx, ventes_2023_2024.csv
03_exercices/ dossier_M06/	4 fichiers : ATTENDU.json, controles_integrite.sql, quincaillerie_export.csv, schema_3fn.sql
03_exercices/ dossier_M07/	6 fichiers : 30_questions.sql, 30_reponses.sql, 6_pieges.md, ATTENDU.json, commercial.duckdb, commercial.sql
03_exercices/ dossier_M08/	11 fichiers : 00_brief.md, ATTENDU.json, analyse_commerciale_modele.py, categorie.csv, client.csv, magasin.csv, mode_paiement.csv, objectif_magasin.csv ...
03_exercices/ dossier_M09/	6 fichiers : 00_brief.md, ATTENDU.json, projet, quincaillerie, sante, scolaire
03_exercices/ dossier_M10/	5 fichiers : 00_brief.md, ATTENDU.json, exports, rapport_apres, rapport_avant
03_exercices/ dossier_M11/	3 fichiers : PERF_M11.json, connexion.py, socle_m11.sql
03_exercices/ dossier_M12/	11 fichiers : ATTENDU.json, __pycache__, commandes_clients.csv, connexion.py, cout_produits.csv, encaissements.csv, etude_avant.md, logistique_mensuelle.csv ...
03_exercices/ dossier_M13/	9 fichiers : ATTENDU.json, __pycache__, connexion.py, modele_fautif.sql, mouvements_clients.csv, revue_modele.md, socle_m13.sql, table_plate.csv ...

7. Mode d'emploi de l'appareil

- **Le cours** : les douze modules du livre assemblé, dans l'ordre, du sommaire général.
- **S'entraîner** : chaque chapitre porte ses exercices autonomes (section 11, ou 13 dans deux chapitres de M06) et leur correction détaillée juste après. **366** exercices, tous repris au corrigé — le contrôle est dans kit_M01_M12.json.
- **Un livrable de portfolio** : douze projets de module (partie II, premier cahier), chacun avec ses livrables et son barème.
- **Se faire évaluer** : douze évaluations (partie II, second cahier), avec seuils.
- **Exécuter** : 01_socle_donnees/scripts/chiffres_manuel.py rejoue tous les chiffres du manuel ; 03_exercices/dossier_*/connexion.py ouvre chaque dossier d'exercices.

Le sommaire de chaque module, à l'intérieur du livre, garde la pagination du module seul : pour convertir en pages du livre, ajoutez le début de plage donné par le sommaire général, moins un.